The OpenXR™ 1.1.62 Specification (with all registered extensions)

The Khronos® OpenXR Working Group

Version 1.1.62: from git ref release-1.1.62

Table of Contents

Preamble . . . 1
1. Introduction . . . 3
1.1. What is OpenXR? . . . 3
1.2. The Programmer’s View of OpenXR . . . 3
1.3. The Implementor’s View of OpenXR . . . 3
1.4. Our View of OpenXR . . . 4
1.5. Filing Bug Reports . . . 4
1.6. Document Conventions . . . 4
2. Fundamentals . . . 6
2.1. API Version Numbers and Semantics . . . 6
2.2. String Encoding . . . 8
2.3. Threading Behavior . . . 8
2.4. Multiprocessing Behavior . . . 12
2.5. Runtime . . . 12
2.6. Extensions . . . 12
2.7. API Layers . . . 13
2.8. Type Aliasing . . . 15
2.9. Valid Usage . . . 15
2.10. Return Codes . . . 20
2.11. Handles . . . 40
2.12. Object Handle Types . . . 40
2.13. Buffer Size Parameters . . . 44
2.14. Time . . . 47
2.15. Duration . . . 48
2.16. Prediction Time Limits . . . 49
2.17. Colors . . . 49
2.18. Coordinate System . . . 50
2.19. Common Data Types . . . 53
2.20. Angles . . . 59
2.21. Boolean Values . . . 60
2.22. Events . . . 60
2.23. System resource lifetime . . . 66
3. API Initialization . . . 68
3.1. Exported Functions . . . 68
3.2. Function Pointers . . . 68
3.3. Runtime Interface Negotiation . . . 71

3.4. API Layer Interface Negotiation . . . 77
4. Instance . . . 86
4.1. API Layers and Extensions . . . 86
4.2. Instance Lifecycle . . . 91
4.3. Instance Information . . . 97
4.4. Platform-Specific Instance Creation . . . 99
4.5. Instance Enumerated Type String Functions . . . 100
5. System . . . 104
5.1. Form Factors . . . 104
5.2. Getting the XrSystemId . . . 105
5.3. System Properties . . . 108
6. Path Tree and Semantic Paths . . . 114
6.1. Path Atom Type . . . 114
6.2. Well-Formed Path Strings . . . 116
6.3. Reserved Paths . . . 119
6.4. Interaction Profile Paths . . . 126
7. Spaces . . . 159
7.1. Reference Spaces . . . 160
7.2. Action Spaces . . . 166
7.3. Space Lifecycle . . . 167
7.4. Locating Spaces . . . 174
8. View Configurations . . . 188
8.1. Primary View Configurations . . . 188
8.2. View Configuration API . . . 190
8.3. Example View Configuration Code . . . 197
9. Session . . . 201
9.1. Session Lifecycle . . . 201
9.2. Session Creation . . . 203
9.3. Session Control . . . 207
9.4. Session States . . . 212
10. Rendering . . . 217
10.1. Swapchain Image Management . . . 217
10.2. View and Projection State . . . 235
10.3. Frame Synchronization . . . 240
10.4. Frame Submission . . . 244
10.5. Frame Rate . . . 250
10.6. Compositing . . . 251
11. Input and Haptics . . . 264

11.1. Action Overview . . . 264
11.2. Action Sets . . . 266
11.3. Creating Actions . . . 271
11.4. Suggested Bindings . . . 276
11.5. Current Interaction Profile . . . 282
11.6. Reading Input Action State . . . 286
11.7. Output Actions and Haptics . . . 297
11.8. Input Action State Synchronization . . . 302
11.9. Bound Sources . . . 305
12. List of Current Extensions . . . 312
12.1. XR_KHR_android_create_instance . . . 319
12.2. XR_KHR_android_surface_swapchain . . . 321
12.3. XR_KHR_android_thread_settings . . . 324
12.4. XR_KHR_binding_modification . . . 328
12.5. XR_KHR_composition_layer_color_scale_bias . . . 331
12.6. XR_KHR_composition_layer_cube . . . 334
12.7. XR_KHR_composition_layer_cylinder . . . 337
12.8. XR_KHR_composition_layer_depth . . . 341
12.9. XR_KHR_composition_layer_equirect . . . 345
12.10. XR_KHR_composition_layer_equirect2 . . . 349
12.11. XR_KHR_convert_timespec_time . . . 352
12.12. XR_KHR_D3D11_enable . . . 356
12.13. XR_KHR_D3D12_enable . . . 363
12.14. XR_KHR_extended_struct_name_lengths . . . 371
12.15. XR_KHR_generic_controller . . . 374
12.16. XR_KHR_loader_init . . . 380
12.17. XR_KHR_loader_init_android . . . 383
12.18. XR_KHR_metal_enable . . . 385
12.19. XR_KHR_opengl_enable . . . 392
12.20. XR_KHR_opengl_es_enable . . . 404
12.21. XR_KHR_swapchain_usage_input_attachment_bit . . . 412
12.22. XR_KHR_visibility_mask . . . 413
12.23. XR_KHR_vulkan_enable . . . 419
12.24. XR_KHR_vulkan_enable2 . . . 433
12.25. XR_KHR_vulkan_swapchain_format_list . . . 451
12.26. XR_KHR_win32_convert_performance_counter_time . . . 454
12.27. XR_EXT_active_action_set_priority . . . 457
12.28. XR_EXT_composition_layer_inverted_alpha . . . 460

12.29. XR_EXT_conformance_automation . . . 462
12.30. XR_EXT_debug_utils . . . 471
12.31. XR_EXT_dpad_binding . . . 497
12.32. XR_EXT_eye_gaze_interaction . . . 507
12.33. XR_EXT_frame_synthesis . . . 514
12.34. XR_EXT_future . . . 521
12.35. XR_EXT_hand_interaction . . . 542
12.36. XR_EXT_hand_joints_motion_range . . . 553
12.37. XR_EXT_hand_tracking . . . 556
12.38. XR_EXT_hand_tracking_data_source . . . 576
12.39. XR_EXT_haptic_parametric . . . 581
12.40. XR_EXT_interaction_profile_battery_state_display . . . 600
12.41. XR_EXT_interaction_render_model . . . 605
12.42. XR_EXT_loader_init_properties . . . 628
12.43. XR_EXT_performance_settings . . . 632
12.44. XR_EXT_render_model . . . 642
12.45. XR_EXT_spatial_anchor . . . 679
12.46. XR_EXT_spatial_entity . . . 692
12.47. XR_EXT_spatial_marker_tracking . . . 779
12.48. XR_EXT_spatial_persistence . . . 801
12.49. XR_EXT_spatial_persistence_operations . . . 828
12.50. XR_EXT_spatial_plane_tracking . . . 842
12.51. XR_EXT_stationary_reference_space . . . 859
12.52. XR_EXT_thermal_query . . . 866
12.53. XR_EXT_user_presence . . . 869
12.54. XR_EXT_view_configuration_depth_range . . . 875
12.55. XR_EXT_view_configuration_views_change . . . 878
12.56. XR_EXT_win32_appcontainer_compatible . . . 881
12.57. XR_ALMALENCE_digital_lens_control . . . 883
12.58. XR_ANDROID_anchor_sharing_export . . . 886
12.59. XR_ANDROID_composition_layer_passthrough_mesh . . . 895
12.60. XR_ANDROID_eye_tracking . . . 910
12.61. XR_ANDROID_face_tracking . . . 925
12.62. XR_ANDROID_geospatial . . . 946
12.63. XR_ANDROID_geospatial_anchor . . . 969
12.64. XR_ANDROID_google_cloud_auth . . . 984
12.65. XR_ANDROID_light_estimation . . . 996
12.66. XR_ANDROID_mouse_interaction . . . 1011

12.67. XR_ANDROID_passthrough_camera_state . . . 1015
12.68. XR_ANDROID_performance_metrics . . . 1019
12.69. XR_ANDROID_recommended_resolution . . . 1030
12.70. XR_ANDROID_scene_meshing . . . 1033
12.71. XR_ANDROID_spatial_anchor_space . . . 1060
12.72. XR_ANDROID_spatial_component_subsumed_by . . . 1068
12.73. XR_ANDROID_spatial_discovery_bounds . . . 1076
12.74. XR_ANDROID_spatial_discovery_raycast . . . 1083
12.75. XR_ANDROID_spatial_entity_bound_anchor . . . 1099
12.76. XR_ANDROID_spatial_object_tracking . . . 1110
12.77. XR_ANDROID_trackables_image . . . 1122
12.78. XR_ANDROID_unbounded_reference_space . . . 1150
12.79. XR_BD_body_tracking . . . 1153
12.80. XR_BD_body_tracking_auxiliary_metrics . . . 1169
12.81. XR_BD_dynamic_object_keyboard . . . 1179
12.82. XR_BD_dynamic_object_mouse . . . 1182
12.83. XR_BD_dynamic_object_tracking . . . 1185
12.84. XR_BD_facial_simulation . . . 1194
12.85. XR_BD_future_progress . . . 1237
12.86. XR_BD_spatial_anchor . . . 1240
12.87. XR_BD_spatial_anchor_sharing . . . 1259
12.88. XR_BD_spatial_audio_rendering . . . 1271
12.89. XR_BD_spatial_mesh . . . 1343
12.90. XR_BD_spatial_plane . . . 1349
12.91. XR_BD_spatial_scene . . . 1356
12.92. XR_BD_spatial_sensing . . . 1365
12.93. XR_BD_ultra_controller_interaction . . . 1431
12.94. XR_EPIC_view_configuration_fov . . . 1434
12.95. XR_FB_android_surface_swapchain_create . . . 1436
12.96. XR_FB_body_tracking . . . 1439
12.97. XR_FB_color_space . . . 1456
12.98. XR_FB_composition_layer_alpha_blend . . . 1464
12.99. XR_FB_composition_layer_depth_test . . . 1467
12.100. XR_FB_composition_layer_image_layout . . . 1470
12.101. XR_FB_composition_layer_secure_content . . . 1472
12.102. XR_FB_composition_layer_settings . . . 1475
12.103. XR_FB_display_refresh_rate . . . 1478
12.104. XR_FB_eye_tracking_social . . . 1484

12.105. XR_FB_face_tracking . . . 1496
12.106. XR_FB_face_tracking2 . . . 1511
12.107. XR_FB_foveation . . . 1552
12.108. XR_FB_foveation_configuration . . . 1559
12.109. XR_FB_foveation_vulkan . . . 1563
12.110. XR_FB_hand_tracking_aim . . . 1565
12.111. XR_FB_hand_tracking_capsules . . . 1569
12.112. XR_FB_hand_tracking_mesh . . . 1572
12.113. XR_FB_haptic_amplitude_envelope . . . 1579
12.114. XR_FB_haptic_pcm . . . 1581
12.115. XR_FB_keyboard_tracking . . . 1587
12.116. XR_FB_passthrough . . . 1594
12.117. XR_FB_passthrough_keyboard_hands . . . 1624
12.118. XR_FB_render_model . . . 1628
12.119. XR_FB_scene . . . 1641
12.120. XR_FB_scene_capture . . . 1657
12.121. XR_FB_spatial_entity . . . 1661
12.122. XR_FB_spatial_entity_container . . . 1676
12.123. XR_FB_spatial_entity_query . . . 1680
12.124. XR_FB_spatial_entity_sharing . . . 1692
12.125. XR_FB_spatial_entity_storage . . . 1697
12.126. XR_FB_spatial_entity_storage_batch . . . 1706
12.127. XR_FB_spatial_entity_user . . . 1711
12.128. XR_FB_swapchain_update_state . . . 1717
12.129. XR_FB_swapchain_update_state_android_surface . . . 1721
12.130. XR_FB_swapchain_update_state_opengl_es . . . 1724
12.131. XR_FB_swapchain_update_state_vulkan . . . 1727
12.132. XR_FB_triangle_mesh . . . 1730
12.133. XR_HTC_anchor . . . 1746
12.134. XR_HTC_body_tracking . . . 1752
12.135. XR_HTC_facial_tracking . . . 1772
12.136. XR_HTC_foveation . . . 1798
12.137. XR_HTC_hand_interaction . . . 1807
12.138. XR_HTC_passthrough . . . 1809
12.139. XR_HTC_vive_wrist_tracker_interaction . . . 1819
12.140. XR_HUAWEI_controller_interaction . . . 1821
12.141. XR_LOGITECH_mx_ink_stylus_interaction . . . 1823
12.142. XR_META_automatic_layer_filter . . . 1829

12.143. XR_META_body_tracking_calibration . . . 1830
12.144. XR_META_body_tracking_fidelity . . . 1837
12.145. XR_META_body_tracking_full_body . . . 1844
12.146. XR_META_boundary_visibility . . . 1850
12.147. XR_META_colocation_discovery . . . 1856
12.148. XR_META_detached_controllers . . . 1883
12.149. XR_META_environment_depth . . . 1889
12.150. XR_META_environment_raycast . . . 1915
12.151. XR_META_face_tracking_visemes . . . 1932
12.152. XR_META_foveation_eye_tracked . . . 1940
12.153. XR_META_hand_tracking_frequency_hint . . . 1946
12.154. XR_META_hand_tracking_microgestures . . . 1950
12.155. XR_META_hand_tracking_unextrapolated_poses . . . 1955
12.156. XR_META_hand_tracking_wide_motion_mode2 . . . 1959
12.157. XR_META_headset_id . . . 1962
12.158. XR_META_local_dimming . . . 1964
12.159. XR_META_passthrough_color_lut . . . 1967
12.160. XR_META_passthrough_layer_resumed_event . . . 1978
12.161. XR_META_passthrough_preferences . . . 1981
12.162. XR_META_performance_metrics . . . 1984
12.163. XR_META_recommended_layer_resolution . . . 1994
12.164. XR_META_simultaneous_hands_and_controllers . . . 1998
12.165. XR_META_spatial_entity_discovery . . . 2005
12.166. XR_META_spatial_entity_group_sharing . . . 2018
12.167. XR_META_spatial_entity_mesh . . . 2023
12.168. XR_META_spatial_entity_persistence . . . 2028
12.169. XR_META_spatial_entity_room_mesh . . . 2040
12.170. XR_META_spatial_entity_semantic_label . . . 2049
12.171. XR_META_spatial_entity_sharing . . . 2051
12.172. XR_META_tile_properties_hint . . . 2058
12.173. XR_META_virtual_keyboard . . . 2065
12.174. XR_META_vulkan_swapchain_create_info . . . 2105
12.175. XR_ML_compat . . . 2107
12.176. XR_ML_facial_expression . . . 2111
12.177. XR_ML_frame_end_info . . . 2124
12.178. XR_ML_global_dimmer . . . 2127
12.179. XR_ML_localization_map . . . 2129
12.180. XR_ML_marker_understanding . . . 2150

12.181. XR_ML_spatial_anchors . . . 2186
12.182. XR_ML_spatial_anchors_storage . . . 2199
12.183. XR_ML_system_notifications . . . 2233
12.184. XR_ML_user_calibration . . . 2236
12.185. XR_ML_view_configuration_depth_range_change . . . 2242
12.186. XR_ML_world_mesh_detection . . . 2243
12.187. XR_MND_headless . . . 2279
12.188. XR_OCULUS_audio_device_guid . . . 2281
12.189. XR_OCULUS_external_camera . . . 2284
12.190. XR_OPPO_controller_interaction . . . 2290
12.191. XR_QCOM_tracking_optimization_settings . . . 2293
12.192. XR_SONY_swapchain_color_space . . . 2299
12.193. XR_ULTRALEAP_hand_tracking_forearm . . . 2306
12.194. XR_VALVE_analog_threshold . . . 2308
12.195. XR_VARJO_composition_layer_depth_test . . . 2311
12.196. XR_VARJO_environment_depth_estimation . . . 2315
12.197. XR_VARJO_foveated_rendering . . . 2317
12.198. XR_VARJO_marker_tracking . . . 2325
12.199. XR_VARJO_view_offset . . . 2338
12.200. XR_VARJO_xr4_controller_interaction . . . 2341
12.201. XR_YVR_controller_interaction . . . 2344
13. List of Provisional Extensions . . . 2348
13.1. XR_EXTX_overlay . . . 2349
13.2. XR_HTCX_vive_tracker_interaction . . . 2356
13.3. XR_MNDX_egl_enable . . . 2365
13.4. XR_MNDX_force_feedback_curl . . . 2367
14. List of Deprecated Extensions . . . 2375
14.1. XR_KHR_locate_spaces . . . 2377
14.2. XR_KHR_maintenance1 . . . 2387
14.3. XR_EXT_hp_mixed_reality_controller . . . 2390
14.4. XR_EXT_local_floor . . . 2393
14.5. XR_EXT_palm_pose . . . 2396
14.6. XR_EXT_plane_detection . . . 2400
14.7. XR_EXT_samsung_odyssey_controller . . . 2427
14.8. XR_EXT_uuid . . . 2429
14.9. XR_ANDROID_device_anchor_persistence . . . 2431
14.10. XR_ANDROID_raycast . . . 2452
14.11. XR_ANDROID_trackables . . . 2462

14.12. XR_ANDROID_trackables_marker . . . 2490
14.13. XR_ANDROID_trackables_object . . . 2502
14.14. XR_ANDROID_trackables_qr_code . . . 2510
14.15. XR_BD_controller_interaction . . . 2520
14.16. XR_FB_space_warp . . . 2525
14.17. XR_FB_touch_controller_pro . . . 2530
14.18. XR_FB_touch_controller_proximity . . . 2534
14.19. XR_HTC_vive_cosmos_controller_interaction . . . 2538
14.20. XR_HTC_vive_focus3_controller_interaction . . . 2541
14.21. XR_META_touch_controller_plus . . . 2544
14.22. XR_ML_ml2_controller_interaction . . . 2548
14.23. XR_MND_swapchain_usage_input_attachment_bit . . . 2551
14.24. XR_MSFT_composition_layer_reprojection . . . 2552
14.25. XR_MSFT_controller_model . . . 2559
14.26. XR_MSFT_first_person_observer . . . 2572
14.27. XR_MSFT_hand_interaction . . . 2574
14.28. XR_MSFT_hand_tracking_mesh . . . 2577
14.29. XR_MSFT_holographic_window_attachment . . . 2597
14.30. XR_MSFT_perception_anchor_interop . . . 2601
14.31. XR_MSFT_scene_marker . . . 2606
14.32. XR_MSFT_scene_understanding . . . 2619
14.33. XR_MSFT_scene_understanding_serialization . . . 2671
14.34. XR_MSFT_secondary_view_configuration . . . 2678
14.35. XR_MSFT_spatial_anchor . . . 2689
14.36. XR_MSFT_spatial_anchor_persistence . . . 2697
14.37. XR_MSFT_spatial_graph_bridge . . . 2710
14.38. XR_MSFT_unbounded_reference_space . . . 2721
14.39. XR_OCULUS_android_session_state_enable . . . 2723
14.40. XR_QCOM_hand_tracking_gesture . . . 2725
14.41. XR_VARJO_quad_views . . . 2729
15. Core Revisions (Informative) . . . 2732
15.1. Version 1.1 . . . 2732
15.2. Loader Runtime and API Layer Negotiation Version 1.0 . . . 2735
15.3. Version 1.0 . . . 2736
Appendix . . . 2744
Code Style Conventions . . . 2744
Application Binary Interface . . . 2744
Android Notes . . . 2779

Glossary . . . 2779
Abbreviations . . . 2781
Dedication (Informative) . . . 2783
Contributors (Informative) . . . 2785
Index . . . 2789

Preamble

Copyright (c) 2017-2026 The Khronos Group Inc.

This Specification is protected by copyright laws and contains material proprietary to Khronos. Except as described by these terms, it or any components may not be reproduced, republished, distributed, transmitted, displayed, broadcast or otherwise exploited in any manner without the express prior written permission of Khronos.

Khronos grants a conditional copyright license to use and reproduce the unmodified Specification for any purpose, without fee or royalty, EXCEPT no licenses to any patent, trademark or other intellectual property rights are granted under these terms.

Khronos makes no, and expressly disclaims any, representations or warranties, express or implied, regarding this Specification, including, without limitation: merchantability, fitness for a particular purpose, non-infringement of any intellectual property, correctness, accuracy, completeness, timeliness, and reliability. Under no circumstances will Khronos, or any of its Promoters, Contributors or Members, or their respective partners, officers, directors, employees, agents or representatives be liable for any damages, whether direct, indirect, special or consequential damages for lost revenues, lost profits, or otherwise, arising from or in connection with these materials.

This document contains extensions which are not ratified by Khronos, and as such is not a ratified Specification, though it contains text from (and is a superset of) the ratified OpenXR Specification that can be found at https://registry.khronos.org/OpenXR/specs/1.1-khr/html/xrspec.html (core with KHR and ratified non-KHR extensions).

The Khronos Intellectual Property Rights Policy defines the terms 'Scope', 'Compliant Portion', and 'Necessary Patent Claims'.

Some parts of this Specification are purely informative and so are EXCLUDED from the Scope of this Specification. The Document Conventions section of the Introduction defines how these parts of the Specification are identified.

Where this Specification uses technical terminology, defined in the Glossary or otherwise, that refer to enabling technologies that are not expressly set forth in this Specification, those enabling technologies are EXCLUDED from the Scope of this Specification. For clarity, enabling technologies not disclosed with particularity in this Specification (e.g. semiconductor manufacturing technology, hardware architecture, processor architecture or microarchitecture, memory architecture, compiler technology, object oriented technology, basic operating system technology, compression technology, algorithms, and so on) are NOT to be considered expressly set forth; only those application program interfaces and data structures disclosed with particularity are included in the Scope of this Specification.

For purposes of the Khronos Intellectual Property Rights Policy as it relates to the definition of Necessary Patent Claims, all recommended or optional features, behaviors and functionality set forth in this Specification, if implemented, are considered to be included as Compliant Portions.

Khronos® and Vulkan® are registered trademarks, and glTF™ is a trademark of The Khronos Group Inc. OpenXR™ is a trademark owned by The Khronos Group Inc. and is registered as a trademark in China, the European Union, Japan and the United Kingdom. OpenGL® is a registered trademark and the OpenGL ES™ and OpenGL SC™ logos are trademarks of Hewlett Packard Enterprise used under license by Khronos. All other product names, trademarks, and/or company names are used solely for identification and belong to their respective owners.

Chapter 1. Introduction

This chapter is informative except for the section on Normative Terminology.

This document, referred to as the "OpenXR Specification" or just the "Specification" hereafter, describes OpenXR: what it is, how it acts, and what is required to implement it. We assume that the reader has a basic understanding of computer graphics and the technologies involved in virtual and augmented reality. This means familiarity with the essentials of computer graphics algorithms and terminology, modern GPUs (Graphic Processing Units), tracking technologies, head mounted devices, and input modalities.

The canonical version of the Specification is available in the official OpenXR Registry, located at URL

https://registry.khronos.org/OpenXR

1.1. What is OpenXR?

OpenXR is an API (Application Programming Interface) for XR applications. XR refers to a continuum of real-and-virtual combined environments generated by computers through human-machine interaction and is inclusive of the technologies associated with virtual reality (VR), augmented reality (AR) and mixed reality (MR). OpenXR is the interface between an application and an in-process or out-of-process "XR runtime system", or just "runtime" hereafter. The runtime may handle such functionality as frame composition, peripheral management, and raw tracking information.

Optionally, a runtime may support device layer plugins which allow access to a variety of hardware across a commonly defined interface.

1.2. The Programmer’s View of OpenXR

To the application programmer, OpenXR is a set of functions that interface with a runtime to perform commonly required operations such as accessing controller/peripheral state, getting current and/or predicted tracking positions, and submitting rendered frames.

A typical OpenXR program begins with a call to create an instance which establishes a connection to a runtime. Then a call is made to create a system which selects for use a physical display and a subset of input, tracking, and graphics devices. Subsequently a call is made to create buffers into which the application will render one or more views using the appropriate graphics APIs for the platform. Finally calls are made to create a session and begin the application’s XR rendering loop.

1.3. The Implementor’s View of OpenXR

To the runtime implementor, OpenXR is a set of functions that control the operation of the XR system and establishes the lifecycle of a XR application.

The implementor's task is to provide a software library on the host which implements the OpenXR API, while mapping the work for each OpenXR function to the graphics hardware as appropriate for the capabilities of the device.

1.4. Our View of OpenXR

We view OpenXR as a mechanism for interacting with VR/AR/MR systems in a platform-agnostic way.

We expect this model to result in a specification that satisfies the needs of both programmers and runtime implementors. It does not, however, necessarily provide a model for implementation. A runtime implementation **must** produce results conforming to those produced by the specified methods, but **may** carry out particular procedures in ways that are more efficient than the one specified.

1.5. Filing Bug Reports

Issues with and bug reports on the OpenXR Specification and the API Registry **can** be filed in the Khronos OpenXR GitHub repository, located at URL

https://github.com/KhronosGroup/OpenXR-Docs

Please tag issues with appropriate labels, such as "Specification", "Ref Pages" or "Registry", to help us triage and assign them appropriately. Unfortunately, GitHub does not currently let users who do not have write access to the repository set GitHub labels on issues. In the meantime, they **can** be added to the title line of the issue set in brackets, e.g. "[Specification]".

1.6. Document Conventions

The OpenXR specification is intended for use by both implementors of the API and application developers seeking to make use of the API, forming a contract between these parties. Specification text may address either party; typically the intended audience can be inferred from context, though some sections are defined to address only one of these parties. (For example, Valid Usage sections only address application developers). Any requirements, prohibitions, recommendations or options defined by normative terminology are imposed only on the audience of that text.

1.6.1. Normative Terminology

The key words **must**, **required**, **should**, **may**, and **optional** in this document, when denoted as above, are to be interpreted as described in RFC 2119:

https://tools.ietf.org/html/rfc2119

must

When used alone, this word, or the term **required**, means that the definition is an absolute requirement of the specification. When followed by **not** ("**must** not"), the phrase means that the

definition is an absolute prohibition of the specification.

should

When used alone, this word means that there may exist valid reasons in particular circumstances to ignore a particular item, but the full implications must be understood and carefully weighed before choosing a different course. When followed by **not** ("**should** not"), the phrase means that there may exist valid reasons in particular circumstances when the particular behavior is acceptable or even useful, but the full implications **should** be understood and the case carefully weighed before implementing any behavior described with this label.

may

This word, or the adjective **optional**, means that an item is truly optional. One vendor may choose to include the item because a particular marketplace requires it or because the vendor feels that it enhances the product while another vendor may omit the same item.

The additional terms **can** and **cannot** are to be interpreted as follows:

can

This word means that the particular behavior described is a valid choice for an application, and is never used to refer to runtime behavior.

cannot

This word means that the particular behavior described is not achievable by an application, for example, an entry point does not exist.



There is an important distinction between **cannot** and **must not**, as used in this Specification. **Cannot** means something the application literally is unable to express or accomplish through the API, while **must not** means something that the application is capable of expressing through the API, but that the consequences of doing so are undefined and potentially unrecoverable for the runtime.

Chapter 2. Fundamentals

2.1. API Version Numbers and Semantics

Multi-part version numbers are used in several places in the OpenXR API.

```
// Provided by XR_VERSION_1_0
typedef uint64_t XrVersion;
```

In each such use, the API major version number, minor version number, and patch version number are packed into a 64-bit integer, referred to as `XrVersion`, as follows:

Version Numbers

- The major version number is a 16-bit integer packed into bits 63-48.
- The minor version number is a 16-bit integer packed into bits 47-32.
- The patch version number is a 32-bit integer packed into bits 31-0.

Differences in any of the version numbers indicate a change to the API, with each part of the version number indicating a different scope of change, as follows.



Note

The rules below apply to OpenXR versions 1.0 or later. Prerelease versions of OpenXR may use different rules for versioning.

A difference in patch version numbers indicates that some usually small part of the specification or header has been modified, typically to fix a bug, and **may** have an impact on the behavior of existing functionality. Differences in the patch version number **must** affect neither full compatibility nor backwards compatibility between two versions, nor **may** it add additional interfaces to the API. Runtimes **may** use patch version number to determine whether to enable implementation changes, such as bug fixes, that impact functionality. Runtimes **should** document any changes that are tied to the patch version. Application developers **should** retest their application on all runtimes they support after compiling with a new version.

A difference in minor version numbers indicates that some amount of new functionality has been added. This will usually include new interfaces in the header, and **may** also include behavior changes and bug fixes. Functionality **may** be deprecated in a minor revision, but **must** not be removed. When a new minor version is introduced, the patch version continues where the last minor version left off, making patch versions unique inside major versions. Differences in the minor version number **should** not affect backwards compatibility, but will affect full compatibility.

A difference in major version numbers indicates a large set of changes to the API, potentially including new functionality and header interfaces, behavioral changes, removal of deprecated features, modification or outright replacement of any feature, and is thus very likely to break compatibility. Differences in the major version number will typically require significant modification to application code in order for it to function properly.

The following table attempts to detail the changes that **may** occur versus when they **must** not be updated during an update to any of the major, minor, or patch version numbers:

Table 1. Scenarios Which May Cause a Version Change

Reason	***Major Version***	***Minor Version***	***Patch Version***
*Extensions Added/Removed**	**may**	**may**	**may**
*Spec-Optional Behavior Changed**	**may**	**may**	**may**
*Spec Required Behavior Changed**	**may**	**may**	**must** not
*Core Interfaces Added**	**may**	**may**	**must** not
*Weak Deprecation**	**may**	**may**	**must** not
*Strong Deprecation**	**may**	**must** not	**must** not
*Core Interfaces Changed/Removed**	**may**	**must** not	**must** not

In the above table, the following identify the various cases in detail:

Extensions Added/Removed	An extension **may** be added or removed with a change at this patch level.
Specification-Optional Behavior Changed	Some **optional** behavior laid out in this specification has changed. Usually this will involve a change in behavior that is marked with the normative language **should** or **may**. For example, a runtime that previously did not validate a particular use case **may** now begin validating that use case.
Specification-Required Behavior Changed	A behavior of runtimes that is required by this specification **may** have changed. For example, a previously **optional** validation **may** now have become mandatory for runtimes.
Core Interfaces Added	New interfaces **may** have been added to this specification (and to the OpenXR header file) in revisions at this level.

Weak Deprecation	An interface **may** have been weakly deprecated at this level. This **may** happen if there is now a better way to accomplish the same thing. Applications making this call **should** behave the same as before the deprecation, but following the new path **may** be more performant, lower latency, or otherwise yield better results. It is possible that some runtimes **may** choose to give run-time warnings that the feature has been weakly deprecated and will likely be strongly deprecated or removed in the future.
Strong Deprecation	An interface **may** have been strongly deprecated at this level. This means that the interface **must** still exist (so applications that are compiled against it will still run) but it **may** now be a no-op, or it **may** be that its behavior has been significantly changed. It **may** be that this functionality is no longer necessary, or that its functionality has been subsumed by another call. This **should** not break an application, but some behavior **may** be different or unanticipated.
Interfaces Changed/Removed	An interface **may** have been changed — with different parameters or return types — at this level. An interface or feature **may** also have been removed entirely. It is almost certain that rebuilding applications will be required.

2.2. String Encoding

This API uses strings as input and output for some functions. Unless otherwise specified, all such strings are `NULL` terminated UTF-8 encoded case-sensitive character arrays.

2.3. Threading Behavior

The OpenXR API is intended to provide scalable performance when used on multiple host threads. All functions **must** support being called concurrently from multiple threads, but certain parameters, or components of parameters are defined to be externally synchronized. This means that the caller **must** guarantee that no more than one thread is using such a parameter at a given time.

More precisely, functions use simple stores to update software structures representing objects. A parameter declared as externally synchronized **may** have its software structures updated at any time during the host execution of the function. If two functions operate on the same object and at least one of the functions declares the object to be externally synchronized, then the caller **must** guarantee not only that the functions do not execute simultaneously, but also that the two functions are separated by an appropriate memory barrier if needed.

For all functions which destroy an object handle, the application **must** externally synchronize the object handle parameter and any child handles.

Externally Synchronized Parameters

- The `instance` parameter, and any child handles, in xrDestroyInstance
- The `session` parameter, and any child handles, in xrDestroySession
- The `space` parameter, and any child handles, in xrDestroySpace
- The `swapchain` parameter, and any child handles, in xrDestroySwapchain
- The `actionSet` parameter, and any child handles, in xrDestroyActionSet
- The `action` parameter, and any child handles, in xrDestroyAction
- The `objectHandle` member of the `nameInfo` parameter in xrSetDebugUtilsObjectNameEXT
- The `instance` parameter, and any child handles, in xrCreateDebugUtilsMessengerEXT
- The `messenger` parameter in xrDestroyDebugUtilsMessengerEXT
- The `anchor` parameter, and any child handles, in xrDestroySpatialAnchorMSFT
- The `nodeBinding` parameter, and any child handles, in xrDestroySpatialGraphNodeBindingMSFT
- The `handTracker` parameter, and any child handles, in xrDestroyHandTrackerEXT
- The `bodyTracker` parameter, and any child handles, in xrDestroyBodyTrackerFB
- The `sceneObserver` parameter, and any child handles, in xrDestroySceneObserverMSFT
- The `scene` parameter, and any child handles, in xrDestroySceneMSFT
- The `facialTracker` parameter, and any child handles, in xrDestroyFacialTrackerHTC
- The `profile` parameter, and any child handles, in xrDestroyFoveationProfileFB
- The `mesh` parameter, and any child handles, in xrDestroyTriangleMeshFB
- The `passthrough` parameter, and any child handles, in xrDestroyPassthroughFB
- The `layer` parameter, and any child handles, in xrDestroyPassthroughLayerFB
- The `instance` parameter, and any child handles, in xrDestroyGeometryInstanceFB
- The `markerDetector` parameter, and any child handles, in xrDestroyMarkerDetectorML
- The `map` parameter, and any child handles, in xrDestroyExportedLocalizationMapML
- The `storage` parameter, and any child handles, in xrDestroySpatialAnchorsStorageML
- The `spatialAnchorStore` parameter, and any child handles, in xrDestroySpatialAnchorStoreConnectionMSFT
- The `faceTracker` parameter, and any child handles, in xrDestroyFaceTrackerFB
- The `eyeTracker` parameter, and any child handles, in xrDestroyEyeTrackerFB
- The `keyboard` parameter, and any child handles, in xrDestroyVirtualKeyboardMETA
- The `user` parameter, and any child handles, in xrDestroySpaceUserFB

- The `colorLut` parameter, and any child handles, in xrDestroyPassthroughColorLutMETA
- The `faceTracker` parameter, and any child handles, in xrDestroyFaceTracker2FB
- The `environmentDepthProvider` parameter, and any child handles, in xrDestroyEnvironmentDepthProviderMETA
- The `swapchain` parameter, and any child handles, in xrDestroyEnvironmentDepthSwapchainMETA
- The `renderModel` parameter, and any child handles, in xrDestroyRenderModelEXT
- The `asset` parameter, and any child handles, in xrDestroyRenderModelAssetEXT
- The `passthrough` parameter, and any child handles, in xrDestroyPassthroughHTC
- The `bodyTracker` parameter, and any child handles, in xrDestroyBodyTrackerHTC
- The `bodyTracker` parameter, and any child handles, in xrDestroyBodyTrackerBD
- The `tracker` parameter, and any child handles, in xrDestroyFaceTrackerBD
- The `snapshot` parameter, and any child handles, in xrDestroySenseDataSnapshotBD
- The `provider` parameter, and any child handles, in xrDestroySenseDataProviderBD
- The `anchor` parameter, and any child handles, in xrDestroyAnchorBD
- The `renderer` parameter, and any child handles, in xrDestroySpatialAudioRendererBD
- The `material` parameter, and any child handles, in xrDestroySoundObstacleMaterialBD
- The `soundObstacle` parameter, and any child handles, in xrDestroySoundObstacleBD
- The `soundObject` parameter, and any child handles, in xrDestroySoundObjectBD
- The `soundField` parameter, and any child handles, in xrDestroySoundFieldBD
- The `planeDetector` parameter, and any child handles, in xrDestroyPlaneDetectorEXT
- The `trackableTracker` parameter, and any child handles, in xrDestroyTrackableTrackerANDROID
- The `eyeTracker` parameter, and any child handles, in xrDestroyEyeTrackerANDROID
- The `handle` parameter, and any child handles, in xrDestroyDeviceAnchorPersistenceANDROID
- The `faceTracker` parameter, and any child handles, in xrDestroyFaceTrackerANDROID
- The `layer` parameter, and any child handles, in xrDestroyPassthroughLayerANDROID
- The `future` member of the `cancelInfo` parameter in xrCancelFutureEXT
- The `detector` parameter, and any child handles, in xrDestroyWorldMeshDetectorML
- The `facialExpressionClient` parameter, and any child handles, in xrDestroyFacialExpressionClientML
- The `environmentRaycaster` parameter, and any child handles, in xrDestroyEnvironmentRaycasterMETA
- The `estimator` parameter, and any child handles, in xrDestroyLightEstimatorANDROID

- The `database` parameter, and any child handles, in xrDestroyTrackableImageDatabaseANDROID
- The `tracker` parameter, and any child handles, in xrDestroySceneMeshingTrackerANDROID
- The `snapshot` parameter, and any child handles, in xrDestroySceneMeshSnapshotANDROID
- The `spatialContext` parameter, and any child handles, in xrDestroySpatialContextEXT
- The `snapshot` parameter, and any child handles, in xrDestroySpatialSnapshotEXT
- The `spatialEntity` parameter, and any child handles, in xrDestroySpatialEntityEXT
- The `persistenceContext` parameter, and any child handles, in xrDestroySpatialPersistenceContextEXT
- The `geospatialTracker` parameter, and any child handles, in xrDestroyGeospatialTrackerANDROID

Implicit Externally Synchronized Parameters

- The `session` parameter by any other xrWaitFrame call in xrWaitFrame
- The `session` parameter by any other xrBeginFrame or xrEndFrame call in xrBeginFrame
- The `session` parameter by any other xrBeginFrame or xrEndFrame call in xrEndFrame
- The XrInstance used to create `messenger`, and all of its child handles in xrDestroyDebugUtilsMessengerEXT
- The buffers returned from calls to xrTriangleMeshGetVertexBufferFB and xrTriangleMeshGetIndexBufferFB on `mesh` in xrDestroyTriangleMeshFB
- `soundObject` parameter by any other xrSubmitSoundObjectBufferBD call, as well as the `renderer` parameter by any other xrWaitAudioPeriodBD or xrEndAudioPeriodBD call (if the `renderer` is the parent of `soundObject`) in xrSubmitSoundObjectBufferBD
- `soundField` parameter by any other xrSubmitSoundFieldBufferBD call, as well as the `renderer` parameter by any other xrWaitAudioPeriodBD or xrEndAudioPeriodBD call (if the `renderer` is the parent of `soundField`) in xrSubmitSoundFieldBufferBD
- The `renderer` parameter by any other xrWaitAudioPeriodBD or xrEndAudioPeriodBD call, `soundField` parameter by any other xrSubmitSoundFieldBufferBD call (if the `soundField` is a child of the `renderer`), and `soundObject` parameter by any other xrSubmitSoundObjectBufferBD call (if the `soundObject` is a child of the `renderer`) in xrWaitAudioPeriodBD
- The `renderer` parameter by any other xrWaitAudioPeriodBD or xrEndAudioPeriodBD call, `soundField` parameter by any other xrSubmitSoundFieldBufferBD call (if the `soundField` is a child of the `renderer`), and `soundObject` parameter by any other xrSubmitSoundObjectBufferBD call (if the `soundObject` is a child of the `renderer`) in xrEndAudioPeriodBD

2.4. Multiprocessing Behavior

The OpenXR API does not explicitly recognize nor require support for multiple processes using the runtime simultaneously, nor does it prevent a runtime from providing such support.

2.5. Runtime

An OpenXR runtime is software which implements the OpenXR API. There **may** be more than one OpenXR runtime installed on a system, but only one runtime can be active at any given time.

2.6. Extensions

OpenXR is an extensible API that grows through the addition of new features. Similar to other Khronos APIs, extensions **may** expose new OpenXR functions or modify the behavior of existing OpenXR functions. Extensions are **optional**, and therefore **must** be enabled by the application before the extended functionality is made available. Because extensions are **optional**, they **may** be implemented only on a subset of runtimes, graphics platforms, or operating systems. Therefore, an application **should** first query which extensions are available before enabling.

The application queries the available list of extensions using the xrEnumerateInstanceExtensionProperties function. Once an application determines which extensions are supported, it **can** enable some subset of them during the call to xrCreateInstance.

OpenXR extensions have unique names that convey information about what functionality is provided. The names have the following format:

Extension Name Formatting

- The prefix "XR_" to identify this as an OpenXR extension
- A string identifier for the vendor tag, which corresponds to the company or group exposing the extension. The vendor tag **must** use only uppercase letters and decimal digits. Some examples include:
  - "KHR" for Khronos extensions, supported by multiple vendors.
  - "EXT" for non-Khronos extensions supported by multiple vendors.
- An underscore "_".
- A string uniquely identifying the extension. The string is a compound of substrings which **must** use only lower case letters and decimal digits. The substrings are delimited with single underscores.

For example: XR_KHR_composition_layer_cube is an OpenXR extension created by the Khronos (KHR) OpenXR Working Group to support cube composition layers.

The public list of available extensions known and configured for inclusion in this document at the time of this specification being generated appears in the List of Extensions appendix at the end of this document.

2.7. API Layers

OpenXR is designed to be a layered API, which means that a user or application **may** insert API layers between the application and the runtime implementation. These API layers provide additional functionality by intercepting OpenXR functions from the layer above and performing different operations than would otherwise be performed without the layer. In the simplest cases, the layer simply calls the next layer down with the same arguments, but a more complex layer **may** implement API functionality that is not present in the layers or runtime below it. This mechanism is essentially an architected "function shimming" or "intercept" feature that is designed into OpenXR and meant to replace more informal methods of "hooking" API calls.

2.7.1. Examples of API Layers

Validation Layer

The layered API approach employed by OpenXR allows for potentially expensive validation of correct API usage to be implemented in a "validation" layer. Such a layer allows the application developer to develop their application with a validation layer active to ensure that the application is using the API correctly. A validation layer confirms that the application has set up object state correctly, has provided the required data for each function, ensures that required resources are available, etc. If a validation layer detects a problem, it issues an error message that **can** be logged or captured by the application via a callback. After the developer has determined that the application is correct, they turn off a validation layer to allow the application to run in a production environment without repeatedly incurring the validation expense. (Note that some validation of correct API usage is required to be implemented by the runtime.)

API Logging Layer

Another example of an API layer is an API logging layer that simply serializes all the API calls to an output sink in a text format, including printing out argument values and structure contents.

API Trace Layer

A related API trace layer produces a trace file that contains all the information provided to the API so that the trace file can be played back by a replay program.

2.7.2. Naming API Layers

To organize API layer names and prevent collisions in the API layer name namespace, API layers **must** be named using the following convention:

```
XR_APILAYER_<VENDOR-TAG>_short_name
```

Vendors are responsible for registering a vendor tag with the OpenXR working group, and just like for implementors, they must maintain their vendor namespace.

Example of an API layer name produced by the Acme company for the "check best practices" API layer:

```
XR_APILAYER_ACME_check_best_practices
```

2.7.3. Activating API Layers

Application Activation

Applications **can** determine the API layers that are available to them by calling the xrEnumerateApiLayerProperties function to obtain a list of available API layers. Applications then **can** select the desired API layers from this list and provide them to the xrCreateInstance function when creating an instance.

System Activation

Application users or users performing roles such as system integrator or system administrator **may** configure a system to activate API layers without involvement from the applications. These platform-dependent steps **may** include the installation of API layer-related files, setting environment variables, or other platform-specific operations. The options that are available for configuring the API layers in this manner are also dependent on the platform and/or runtime.

2.7.4. API Layer Extensions

API layers **may** implement OpenXR functions that are not supported by the underlying runtime. In order to expose these new features, the API layer **must** expose this functionality in the form of an OpenXR extension. It **must** not expose new OpenXR functions without an associated extension.

For example, an OpenXR API-logging API layer might expose an API function to allow the application to turn logging on for only a portion of its execution. Since new functions **must** be exposed through an extension, the vendor has created an extension called `XR_ACME_logging_on_off` to contain these new functions. The application **should** query if the API layer supports the extension and then, only if it exists, enable both the extension and the API layer by name during xrCreateInstance.

To find out what extensions an API layer supports, an application **must** first verify that the API layer exists on the current system by calling xrEnumerateApiLayerProperties. After verifying an API layer of interest exists, the application then **should** call xrEnumerateInstanceExtensionProperties and provide the API layer name as the first parameter. This will return the list of extensions implemented by that API layer.

2.8. Type Aliasing

Type aliasing refers to the situation in which the actual type of a element does not match the declared type. Some C and C++ compilers assume that the actual type matches the declared type in some configurations, and **may** be so configured by default at common optimization levels. In such a compiler configured with that assumption, violating the assumption **may** produce undefined behavior. This compiler feature is typically referred to as "strict aliasing," and it **can** usually be enabled or disabled via compiler options. The OpenXR specification **does not** support strict aliasing, as there are some cases in which an application intentionally provides a struct with a type that differs from the declared type. For example, XrFrameEndInfo::`layers` is an array of type `const` XrCompositionLayerBaseHeader code:* `const`. However, each element of the array **must** be of one of the specific layer types, such as XrCompositionLayerQuad. Similarly, xrEnumerateSwapchainImages accepts an array of XrSwapchainImageBaseHeader, whereas the actual type passed **must** be an array of a type such as XrSwapchainImageVulkanKHR.

For OpenXR to work correctly, the compiler **must** support the type aliasing described here.

```
// Provided by XR_VERSION_1_0
#if !defined(XR_MAY_ALIAS)
#if defined(__clang__) || (defined(__GNUC__) && (__GNUC__ > 4))
#define XR_MAY_ALIAS __attribute__((__may_alias__))
#else
#define XR_MAY_ALIAS
#endif
#endif
```

As a convenience, some types and pointers that are known at specification time to alias values of different types have been annotated with the XR_MAY_ALIAS definition. If this macro is not defined before including OpenXR headers, and a new enough Clang or GCC compiler is used, it is defined to a compiler-specific attribute annotation to inform these compilers that those pointers **may** alias. However, there is no guarantee that all aliasing types or pointers have been correctly marked with this macro, so thorough testing is still recommended if you choose (at your own risk) to permit your compiler to perform type-based aliasing analysis.

2.9. Valid Usage

Valid usage defines a set of conditions which **must** be met in order to achieve well-defined run-time behavior in an application. These conditions depend only on API state, and the parameters or objects whose usage is constrained by the condition.

Some valid usage conditions have dependencies on runtime limits or feature availability. It is possible to validate these conditions against the API's minimum or maximum supported values for these limits and features, or some subset of other known values.

Valid usage conditions **should** apply to a function or structure where complete information about the condition would be known during execution of an application. This is such that a validation API layer or linter **can** be written directly against these statements at the point they are specified.

2.9.1. Implicit Valid Usage

Some valid usage conditions apply to all functions and structures in the API, unless explicitly denoted otherwise for a specific function or structure. These conditions are considered implicit. Implicit valid usage conditions are described in detail below.

2.9.2. Valid Usage for Object Handles

Any input parameter to a function that is an object handle **must** be a valid object handle, unless otherwise specified. An object handle is valid if and only if all of the following conditions hold:

Object Handle Validity Conditions

- It has been created or allocated by a previous, successful call to the API.
- It has not been destroyed by a previous call to the API.
- Its parent handle is also valid.

There are contexts in which an object handle is **optional** or otherwise unspecified. In those cases, the API uses XR_NULL_HANDLE, which has the integer value `0`.

2.9.3. Valid Usage for Pointers

Any parameter that is a pointer **must** be a valid pointer when the specification indicates that the runtime uses the pointer. A pointer is valid if and only if it points at memory containing values of the number and type(s) expected by the function, and all fundamental types accessed through the pointer (e.g. as elements of an array or as members of a structure) satisfy the alignment requirements of the host processor.

2.9.4. Valid Usage for Enumerated Types

Any parameter of an enumerated type **must** be a valid enumerant for that type. An enumerant is valid if and only if the enumerant is defined as part of the enumerated type in question.

2.9.5. Valid Usage for Flags

A collection of flags is represented by a bitmask using the type `XrFlags64`:

```
// Provided by XR_VERSION_1_0
typedef uint64_t XrFlags64;
```

Bitmasks are passed to many functions and structures to compactly represent options and are stored in memory defined by the XrFlags64 type. But the API does not use the XrFlags64 type directly. Instead, a Xr*Flags type is used which is an alias of the XrFlags64 type. The API also defines a set of constant bit definitions used to set the bitmasks.

Any Xr*Flags member or parameter used in the API **must** be a valid combination of bit flags. A valid combination is either zero or the bitwise OR of valid bit flags. A bit flag is valid if and only if:

Bit Flag Validity

- The bit flag is one of the constant bit definitions defined by the same Xr*Flags type as the Xr*Flags member or parameter. (Valid flag values **may** also be defined by extensions but will appear in the specification with all other valid flag values for that type.)
- The flag is allowed in the context in which it is being used. For example, in some cases, certain bit flags or combinations of bit flags are mutually exclusive.

2.9.6. Valid Usage for Structure Types

Any parameter that is a structure containing a type member **must** have a value of type which is a valid XrStructureType value matching the type of the structure. As a general rule, the name of this value is obtained by taking the structure name, stripping the leading Xr, prefixing each capital letter with an underscore, converting the entire resulting string to upper case, and prefixing it with XR_TYPE_.

The only exceptions to this rule are API and Operating System names which are converted in a way that produces a more readable value:

Structure Type Format Exceptions

- OpenGL ⇒ _OPENGL
- OpenGLES ⇒ _OPENGL_ES
- EGL ⇒ _EGL
- D3D ⇒ _D3D

2.9.7. Valid Usage for Structure Pointer Chains

Any structure containing a void* next member **must** have a value of next that is either NULL, or points to a valid structure that also contains type and next member values. The set of structures connected by

`next` pointers is referred to as a `next` chain.

In order to use a structure type defined by an extension in a `next` chain, the proper extension **must** have been previously enabled during xrCreateInstance. A runtime **must** ignore all unrecognized structures in a `next` chain, including those associated with an extension that has not been enabled.

Some structures for use in a chain are described in the core OpenXR specification and are mentioned in the Member Descriptions. Any structure described in this document intended for use in a chain is mentioned in a "See also" list in the implicit valid usage of the structure they chain to. Most chained structures are associated with extensions, and are described in the base OpenXR Specification under the List of Extensions. Vendor-specific extensions **may** be found there as well, or **may** only be available from the vendor's website or internal document repositories.

Unless otherwise specified: Chained structs which are output structs **may** be modified by the runtime with the exception of the type and next fields. Upon return from any function, all type and next fields in the chain **must** be unmodified.

Useful Base Structures

As a convenience to runtimes and layers needing to iterate through a structure pointer chain, the OpenXR API provides the following base structures:

The XrBaseInStructure structure is defined as:

```
// Provided by XR_VERSION_1_0
typedef struct XrBaseInStructure {
    XrStructureType                    type;
    const struct XrBaseInStructure*    next;
} XrBaseInStructure;
```

Member Descriptions

- `type` is the XrStructureType of this structure. This base structure itself has no associated XrStructureType value.
- `next` is `NULL` or a pointer to the next structure in a structure chain.

XrBaseInStructure **can** be used to facilitate iterating through a read-only structure pointer chain.

The XrBaseOutStructure structure is defined as:

```
// Provided by XR_VERSION_1_0
typedef struct XrBaseOutStructure {
    XrStructureType                type;
    struct XrBaseOutStructure*     next;
} XrBaseOutStructure;
```

Member Descriptions

- `type` is the XrStructureType of this structure. This base structure itself has no associated XrStructureType value.
- `next` is `NULL` or a pointer to the next structure in a structure chain.

XrBaseOutStructure **can** be used to facilitate iterating through a structure pointer chain that returns data back to the application.

These structures allow for some type safety and can be used by OpenXR API functions that operate on generic inputs and outputs.

Next Chain Structure Uniqueness

Applications **should** ensure that they create and insert no more than one occurrence of each type of extension structure in a given `next` chain. Other components of OpenXR (such as the OpenXR loader or an API Layer) **may** insert duplicate structures into this chain. This provides those components the ability to update a structure that appears in the `next` chain by making a modified copy of that same structure and placing the new version at the beginning of the chain. The benefit of allowing this duplication is each component is no longer required to create a copy of the entire `next` chain just to update one structure. When duplication is present, all other OpenXR components **must** process only the first instance of a structure of a given type, and then ignore all instances of a structure of that same type.

If a component makes such a structure copy, and the original structure is also used to return content, then that component **must** copy the necessary content from the copied structure and into the original version of the structure upon completion of the function prior to proceeding back up the call stack. This is to ensure that OpenXR behavior is consistent whether or not that particular OpenXR component is present and/or enabled on the system.

2.9.8. Valid Usage for Nested Structures

The above conditions also apply recursively to members of structures provided as input to a function, either as a direct argument to the function, or themselves a member of another structure.

Specifics on valid usage of each function are covered in their individual sections.

2.10. Return Codes

The core API is designed to capture most, but not all, instances of incorrect usage. As such, most functions provide return codes. Functions in the API return their status via return codes that are in one of the two categories below.

Return Code Categories

- Successful completion codes are returned when a function needs to communicate success or status information. All successful completion codes are non-negative values.
- Run time error codes are returned when a function needs to communicate a failure that could only be detected at run time. All run time error codes are negative values.

```
// Provided by XR_VERSION_1_0
typedef enum XrResult {
    XR_SUCCESS = 0,
    XR_TIMEOUT_EXPIRED = 1,
    XR_SESSION_LOSS_PENDING = 3,
    XR_EVENT_UNAVAILABLE = 4,
    XR_SPACE_BOUNDS_UNAVAILABLE = 7,
    XR_SESSION_NOT_FOCUSED = 8,
    XR_FRAME_DISCARDED = 9,
    XR_ERROR_VALIDATION_FAILURE = -1,
    XR_ERROR_RUNTIME_FAILURE = -2,
    XR_ERROR_OUT_OF_MEMORY = -3,
    XR_ERROR_API_VERSION_UNSUPPORTED = -4,
    XR_ERROR_INITIALIZATION_FAILED = -6,
    XR_ERROR_FUNCTION_UNSUPPORTED = -7,
    XR_ERROR_FEATURE_UNSUPPORTED = -8,
    XR_ERROR_EXTENSION_NOT_PRESENT = -9,
    XR_ERROR_LIMIT_REACHED = -10,
    XR_ERROR_SIZE_INSUFFICIENT = -11,
    XR_ERROR_HANDLE_INVALID = -12,
    XR_ERROR_INSTANCE_LOST = -13,
    XR_ERROR_SESSION_RUNNING = -14,
    XR_ERROR_SESSION_NOT_RUNNING = -16,
    XR_ERROR_SESSION_LOST = -17,
    XR_ERROR_SYSTEM_INVALID = -18,
    XR_ERROR_PATH_INVALID = -19,
    XR_ERROR_PATH_COUNT_EXCEEDED = -20,
    XR_ERROR_PATH_FORMAT_INVALID = -21,
    XR_ERROR_PATH_UNSUPPORTED = -22,
    XR_ERROR_LAYER_INVALID = -23,
    XR_ERROR_LAYER_LIMIT_EXCEEDED = -24,
```

```
    XR_ERROR_SWAPCHAIN_RECT_INVALID = -25,
    XR_ERROR_SWAPCHAIN_FORMAT_UNSUPPORTED = -26,
    XR_ERROR_ACTION_TYPE_MISMATCH = -27,
    XR_ERROR_SESSION_NOT_READY = -28,
    XR_ERROR_SESSION_NOT_STOPPING = -29,
    XR_ERROR_TIME_INVALID = -30,
    XR_ERROR_REFERENCE_SPACE_UNSUPPORTED = -31,
    XR_ERROR_FILE_ACCESS_ERROR = -32,
    XR_ERROR_FILE_CONTENTS_INVALID = -33,
    XR_ERROR_FORM_FACTOR_UNSUPPORTED = -34,
    XR_ERROR_FORM_FACTOR_UNAVAILABLE = -35,
    XR_ERROR_API_LAYER_NOT_PRESENT = -36,
    XR_ERROR_CALL_ORDER_INVALID = -37,
    XR_ERROR_GRAPHICS_DEVICE_INVALID = -38,
    XR_ERROR_POSE_INVALID = -39,
    XR_ERROR_INDEX_OUT_OF_RANGE = -40,
    XR_ERROR_VIEW_CONFIGURATION_TYPE_UNSUPPORTED = -41,
    XR_ERROR_ENVIRONMENT_BLEND_MODE_UNSUPPORTED = -42,
    XR_ERROR_NAME_DUPLICATED = -44,
    XR_ERROR_NAME_INVALID = -45,
    XR_ERROR_ACTIONSET_NOT_ATTACHED = -46,
    XR_ERROR_ACTIONSETS_ALREADY_ATTACHED = -47,
    XR_ERROR_LOCALIZED_NAME_DUPLICATED = -48,
    XR_ERROR_LOCALIZED_NAME_INVALID = -49,
    XR_ERROR_GRAPHICS_REQUIREMENTS_CALL_MISSING = -50,
    XR_ERROR_RUNTIME_UNAVAILABLE = -51,
  // Provided by XR_VERSION_1_1
    XR_ERROR_EXTENSION_DEPENDENCY_NOT_ENABLED = -1000710001,
  // Provided by XR_VERSION_1_1
    XR_ERROR_PERMISSION_INSUFFICIENT = -1000710000,
  // Provided by XR_KHR_android_thread_settings
    XR_ERROR_ANDROID_THREAD_SETTINGS_ID_INVALID_KHR = -1000003000,
  // Provided by XR_KHR_android_thread_settings
    XR_ERROR_ANDROID_THREAD_SETTINGS_FAILURE_KHR = -1000003001,
  // Provided by XR_MSFT_spatial_anchor
    XR_ERROR_CREATE_SPATIAL_ANCHOR_FAILED_MSFT = -1000039001,
  // Provided by XR_MSFT_secondary_view_configuration
    XR_ERROR_SECONDARY_VIEW_CONFIGURATION_TYPE_NOT_ENABLED_MSFT = -1000053000,
  // Provided by XR_MSFT_controller_model
    XR_ERROR_CONTROLLER_MODEL_KEY_INVALID_MSFT = -1000055000,
  // Provided by XR_MSFT_composition_layer_reprojection
    XR_ERROR_REPROJECTION_MODE_UNSUPPORTED_MSFT = -1000066000,
  // Provided by XR_MSFT_scene_understanding
    XR_ERROR_COMPUTE_NEW_SCENE_NOT_COMPLETED_MSFT = -1000097000,
  // Provided by XR_MSFT_scene_understanding
    XR_ERROR_SCENE_COMPONENT_ID_INVALID_MSFT = -1000097001,
  // Provided by XR_MSFT_scene_understanding
    XR_ERROR_SCENE_COMPONENT_TYPE_MISMATCH_MSFT = -1000097002,
```

```
    // Provided by XR_MSFT_scene_understanding
      XR_ERROR_SCENE_MESH_BUFFER_ID_INVALID_MSFT = -1000097003,
    // Provided by XR_MSFT_scene_understanding
      XR_ERROR_SCENE_COMPUTE_FEATURE_INCOMPATIBLE_MSFT = -1000097004,
    // Provided by XR_MSFT_scene_understanding
      XR_ERROR_SCENE_COMPUTE_CONSISTENCY_MISMATCH_MSFT = -1000097005,
    // Provided by XR_FB_display_refresh_rate
      XR_ERROR_DISPLAY_REFRESH_RATE_UNSUPPORTED_FB = -1000101000,
    // Provided by XR_FB_color_space
      XR_ERROR_COLOR_SPACE_UNSUPPORTED_FB = -1000108000,
    // Provided by XR_FB_spatial_entity
      XR_ERROR_SPACE_COMPONENT_NOT_SUPPORTED_FB = -1000113000,
    // Provided by XR_FB_spatial_entity
      XR_ERROR_SPACE_COMPONENT_NOT_ENABLED_FB = -1000113001,
    // Provided by XR_FB_spatial_entity
      XR_ERROR_SPACE_COMPONENT_STATUS_PENDING_FB = -1000113002,
    // Provided by XR_FB_spatial_entity
      XR_ERROR_SPACE_COMPONENT_STATUS_ALREADY_SET_FB = -1000113003,
    // Provided by XR_FB_passthrough
      XR_ERROR_UNEXPECTED_STATE_PASSTHROUGH_FB = -1000118000,
    // Provided by XR_FB_passthrough
      XR_ERROR_FEATURE_ALREADY_CREATED_PASSTHROUGH_FB = -1000118001,
    // Provided by XR_FB_passthrough
      XR_ERROR_FEATURE_REQUIRED_PASSTHROUGH_FB = -1000118002,
    // Provided by XR_FB_passthrough
      XR_ERROR_NOT_PERMITTED_PASSTHROUGH_FB = -1000118003,
    // Provided by XR_FB_passthrough
      XR_ERROR_INSUFFICIENT_RESOURCES_PASSTHROUGH_FB = -1000118004,
    // Provided by XR_FB_passthrough
      XR_ERROR_UNKNOWN_PASSTHROUGH_FB = -1000118050,
    // Provided by XR_FB_render_model
      XR_ERROR_RENDER_MODEL_KEY_INVALID_FB = -1000119000,
    // Provided by XR_FB_render_model
      XR_RENDER_MODEL_UNAVAILABLE_FB = 1000119020,
    // Provided by XR_VARJO_marker_tracking
      XR_ERROR_MARKER_NOT_TRACKED_VARJO = -1000124000,
    // Provided by XR_VARJO_marker_tracking
      XR_ERROR_MARKER_ID_INVALID_VARJO = -1000124001,
    // Provided by XR_ML_marker_understanding
      XR_ERROR_MARKER_DETECTOR_PERMISSION_DENIED_ML = -1000138000,
    // Provided by XR_ML_marker_understanding
      XR_ERROR_MARKER_DETECTOR_LOCATE_FAILED_ML = -1000138001,
    // Provided by XR_ML_marker_understanding
      XR_ERROR_MARKER_DETECTOR_INVALID_DATA_QUERY_ML = -1000138002,
    // Provided by XR_ML_marker_understanding
      XR_ERROR_MARKER_DETECTOR_INVALID_CREATE_INFO_ML = -1000138003,
    // Provided by XR_ML_marker_understanding
      XR_ERROR_MARKER_INVALID_ML = -1000138004,
```

```
    // Provided by XR_ML_localization_map
      XR_ERROR_LOCALIZATION_MAP_INCOMPATIBLE_ML = -1000139000,
    // Provided by XR_ML_localization_map
      XR_ERROR_LOCALIZATION_MAP_UNAVAILABLE_ML = -1000139001,
    // Provided by XR_ML_localization_map
      XR_ERROR_LOCALIZATION_MAP_FAIL_ML = -1000139002,
    // Provided by XR_ML_localization_map
      XR_ERROR_LOCALIZATION_MAP_IMPORT_EXPORT_PERMISSION_DENIED_ML = -1000139003,
    // Provided by XR_ML_localization_map
      XR_ERROR_LOCALIZATION_MAP_PERMISSION_DENIED_ML = -1000139004,
    // Provided by XR_ML_localization_map
      XR_ERROR_LOCALIZATION_MAP_ALREADY_EXISTS_ML = -1000139005,
    // Provided by XR_ML_localization_map
      XR_ERROR_LOCALIZATION_MAP_CANNOT_EXPORT_CLOUD_MAP_ML = -1000139006,
    // Provided by XR_ML_spatial_anchors
      XR_ERROR_SPATIAL_ANCHORS_PERMISSION_DENIED_ML = -1000140000,
    // Provided by XR_ML_spatial_anchors
      XR_ERROR_SPATIAL_ANCHORS_NOT_LOCALIZED_ML = -1000140001,
    // Provided by XR_ML_spatial_anchors
      XR_ERROR_SPATIAL_ANCHORS_OUT_OF_MAP_BOUNDS_ML = -1000140002,
    // Provided by XR_ML_spatial_anchors
      XR_ERROR_SPATIAL_ANCHORS_SPACE_NOT_LOCATABLE_ML = -1000140003,
    // Provided by XR_ML_spatial_anchors_storage
      XR_ERROR_SPATIAL_ANCHORS_ANCHOR_NOT_FOUND_ML = -1000141000,
    // Provided by XR_MSFT_spatial_anchor_persistence
      XR_ERROR_SPATIAL_ANCHOR_NAME_NOT_FOUND_MSFT = -1000142001,
    // Provided by XR_MSFT_spatial_anchor_persistence
      XR_ERROR_SPATIAL_ANCHOR_NAME_INVALID_MSFT = -1000142002,
    // Provided by XR_MSFT_scene_marker
      XR_SCENE_MARKER_DATA_NOT_STRING_MSFT = 1000147000,
    // Provided by XR_FB_spatial_entity_sharing
      XR_ERROR_SPACE_MAPPING_INSUFFICIENT_FB = -1000169000,
    // Provided by XR_FB_spatial_entity_sharing
      XR_ERROR_SPACE_LOCALIZATION_FAILED_FB = -1000169001,
    // Provided by XR_FB_spatial_entity_sharing
      XR_ERROR_SPACE_NETWORK_TIMEOUT_FB = -1000169002,
    // Provided by XR_FB_spatial_entity_sharing
      XR_ERROR_SPACE_NETWORK_REQUEST_FAILED_FB = -1000169003,
    // Provided by XR_FB_spatial_entity_sharing
      XR_ERROR_SPACE_CLOUD_STORAGE_DISABLED_FB = -1000169004,
    // Provided by XR_META_spatial_entity_persistence
      XR_ERROR_SPACE_INSUFFICIENT_RESOURCES_META = -1000259000,
    // Provided by XR_META_spatial_entity_persistence
      XR_ERROR_SPACE_STORAGE_AT_CAPACITY_META = -1000259001,
    // Provided by XR_META_spatial_entity_persistence
      XR_ERROR_SPACE_INSUFFICIENT_VIEW_META = -1000259002,
    // Provided by XR_META_spatial_entity_persistence
      XR_ERROR_SPACE_PERMISSION_INSUFFICIENT_META = -1000259003,
```

```
    // Provided by XR_META_spatial_entity_persistence
      XR_ERROR_SPACE_RATE_LIMITED_META = -1000259004,
    // Provided by XR_META_spatial_entity_persistence
      XR_ERROR_SPACE_TOO_DARK_META = -1000259005,
    // Provided by XR_META_spatial_entity_persistence
      XR_ERROR_SPACE_TOO_BRIGHT_META = -1000259006,
    // Provided by XR_META_passthrough_color_lut
      XR_ERROR_PASSTHROUGH_COLOR_LUT_BUFFER_SIZE_MISMATCH_META = -1000266000,
    // Provided by XR_META_environment_depth
      XR_ENVIRONMENT_DEPTH_NOT_AVAILABLE_META = 1000291000,
    // Provided by XR_EXT_render_model
      XR_ERROR_RENDER_MODEL_ID_INVALID_EXT = -1000300000,
    // Provided by XR_EXT_render_model
      XR_ERROR_RENDER_MODEL_ASSET_UNAVAILABLE_EXT = -1000300001,
    // Provided by XR_EXT_render_model
      XR_ERROR_RENDER_MODEL_GLTF_EXTENSION_REQUIRED_EXT = -1000300002,
    // Provided by XR_EXT_interaction_render_model
      XR_ERROR_NOT_INTERACTION_RENDER_MODEL_EXT = -1000301000,
    // Provided by XR_QCOM_tracking_optimization_settings
      XR_ERROR_HINT_ALREADY_SET_QCOM = -1000306000,
    // Provided by XR_HTC_anchor
      XR_ERROR_NOT_AN_ANCHOR_HTC = -1000319000,
    // Provided by XR_BD_spatial_sensing
      XR_ERROR_SPATIAL_ENTITY_ID_INVALID_BD = -1000389000,
    // Provided by XR_BD_spatial_sensing
      XR_ERROR_SPATIAL_SENSING_SERVICE_UNAVAILABLE_BD = -1000389001,
    // Provided by XR_BD_spatial_sensing
      XR_ERROR_ANCHOR_NOT_SUPPORTED_FOR_ENTITY_BD = -1000389002,
    // Provided by XR_BD_spatial_anchor
      XR_ERROR_SPATIAL_ANCHOR_NOT_FOUND_BD = -1000390000,
    // Provided by XR_BD_spatial_anchor_sharing
      XR_ERROR_SPATIAL_ANCHOR_SHARING_NETWORK_TIMEOUT_BD = -1000391000,
    // Provided by XR_BD_spatial_anchor_sharing
      XR_ERROR_SPATIAL_ANCHOR_SHARING_AUTHENTICATION_FAILURE_BD = -1000391001,
    // Provided by XR_BD_spatial_anchor_sharing
      XR_ERROR_SPATIAL_ANCHOR_SHARING_NETWORK_FAILURE_BD = -1000391002,
    // Provided by XR_BD_spatial_anchor_sharing
      XR_ERROR_SPATIAL_ANCHOR_SHARING_LOCALIZATION_FAIL_BD = -1000391003,
    // Provided by XR_BD_spatial_anchor_sharing
      XR_ERROR_SPATIAL_ANCHOR_SHARING_MAP_INSUFFICIENT_BD = -1000391004,
    // Provided by XR_BD_spatial_scene
      XR_ERROR_SCENE_CAPTURE_FAILURE_BD = -1000392000,
    // Provided by XR_EXT_plane_detection
      XR_ERROR_SPACE_NOT_LOCATABLE_EXT = -1000429000,
    // Provided by XR_EXT_plane_detection
      XR_ERROR_PLANE_DETECTION_PERMISSION_DENIED_EXT = -1000429001,
    // Provided by XR_ANDROID_trackables
      XR_ERROR_MISMATCHING_TRACKABLE_TYPE_ANDROID = -1000455000,
```

```
    // Provided by XR_ANDROID_trackables
      XR_ERROR_TRACKABLE_TYPE_NOT_SUPPORTED_ANDROID = -1000455001,
    // Provided by XR_ANDROID_device_anchor_persistence
      XR_ERROR_ANCHOR_ID_NOT_FOUND_ANDROID = -1000457000,
    // Provided by XR_ANDROID_device_anchor_persistence
      XR_ERROR_ANCHOR_ALREADY_PERSISTED_ANDROID = -1000457001,
    // Provided by XR_ANDROID_device_anchor_persistence
      XR_ERROR_ANCHOR_NOT_TRACKING_ANDROID = -1000457002,
    // Provided by XR_ANDROID_device_anchor_persistence
      XR_ERROR_PERSISTED_DATA_NOT_READY_ANDROID = -1000457003,
    // Provided by XR_ANDROID_face_tracking
      XR_ERROR_SERVICE_NOT_READY_ANDROID = -1000458000,
    // Provided by XR_ANDROID_composition_layer_passthrough_mesh
      XR_ERROR_MESH_DATA_LIMIT_EXCEEDED_ANDROID = -1000462000,
    // Provided by XR_EXT_future
      XR_ERROR_FUTURE_PENDING_EXT = -1000469001,
    // Provided by XR_EXT_future
      XR_ERROR_FUTURE_INVALID_EXT = -1000469002,
    // Provided by XR_ML_system_notifications
      XR_ERROR_SYSTEM_NOTIFICATION_PERMISSION_DENIED_ML = -1000473000,
    // Provided by XR_ML_system_notifications
      XR_ERROR_SYSTEM_NOTIFICATION_INCOMPATIBLE_SKU_ML = -1000473001,
    // Provided by XR_ML_world_mesh_detection
      XR_ERROR_WORLD_MESH_DETECTOR_PERMISSION_DENIED_ML = -1000474000,
    // Provided by XR_ML_world_mesh_detection
      XR_ERROR_WORLD_MESH_DETECTOR_SPACE_NOT_LOCATABLE_ML = -1000474001,
    // Provided by XR_ML_facial_expression
      XR_ERROR_FACIAL_EXPRESSION_PERMISSION_DENIED_ML = 1000482000,
    // Provided by XR_META_boundary_visibility
      XR_BOUNDARY_VISIBILITY_SUPPRESSION_NOT_ALLOWED_META = 1000528000,
    // Provided by XR_META_colocation_discovery
      XR_ERROR_COLOCATION_DISCOVERY_NETWORK_FAILED_META = -1000571001,
    // Provided by XR_META_colocation_discovery
      XR_ERROR_COLOCATION_DISCOVERY_NO_DISCOVERY_METHOD_META = -1000571002,
    // Provided by XR_META_colocation_discovery
      XR_COLOCATION_DISCOVERY_ALREADY_ADVERTISING_META = 1000571003,
    // Provided by XR_META_colocation_discovery
      XR_COLOCATION_DISCOVERY_ALREADY_DISCOVERING_META = 1000571004,
    // Provided by XR_META_spatial_entity_group_sharing
      XR_ERROR_SPACE_GROUP_NOT_FOUND_META = -1000572002,
    // Provided by XR_ANDROID_anchor_sharing_export
      XR_ERROR_ANCHOR_NOT_OWNED_BY_CALLER_ANDROID = -1000701000,
    // Provided by XR_ANDROID_trackables_image
      XR_ERROR_IMAGE_FORMAT_UNSUPPORTED_ANDROID = -1000709000,
    // Provided by XR_EXT_spatial_entity
      XR_ERROR_SPATIAL_CAPABILITY_UNSUPPORTED_EXT = -1000740001,
    // Provided by XR_EXT_spatial_entity
      XR_ERROR_SPATIAL_ENTITY_ID_INVALID_EXT = -1000740002,
```

```
  // Provided by XR_EXT_spatial_entity
    XR_ERROR_SPATIAL_BUFFER_ID_INVALID_EXT = -1000740003,
  // Provided by XR_EXT_spatial_entity
    XR_ERROR_SPATIAL_COMPONENT_UNSUPPORTED_FOR_CAPABILITY_EXT = -1000740004,
  // Provided by XR_EXT_spatial_entity
    XR_ERROR_SPATIAL_CAPABILITY_CONFIGURATION_INVALID_EXT = -1000740005,
  // Provided by XR_EXT_spatial_entity
    XR_ERROR_SPATIAL_COMPONENT_NOT_ENABLED_EXT = -1000740006,
  // Provided by XR_EXT_spatial_persistence
    XR_ERROR_SPATIAL_PERSISTENCE_SCOPE_UNSUPPORTED_EXT = -1000763001,
  // Provided by XR_EXT_spatial_persistence_operations
    XR_ERROR_SPATIAL_PERSISTENCE_SCOPE_INCOMPATIBLE_EXT = -1000781001,
  // Provided by XR_ANDROID_google_cloud_auth
    XR_ERROR_KEYLESS_AUTH_NOT_SETUP_ANDROID = -1000787000,
  // Provided by XR_ANDROID_google_cloud_auth
    XR_ERROR_KEYLESS_AUTH_FAILED_ANDROID = -1000787001,
  // Provided by XR_ANDROID_geospatial
    XR_ERROR_GEOSPATIAL_TRACKER_NOT_RUNNING_ANDROID = -1000789000,
  // Provided by XR_ANDROID_geospatial
    XR_ERROR_GEOSPATIAL_COORDINATES_INVALID_ANDROID = -1000789001,
  // Provided by XR_ANDROID_geospatial
    XR_ERROR_GEOSPATIAL_CLOUD_AUTH_FAILED_ANDROID = -1000789002,
  // Provided by XR_ANDROID_spatial_entity_bound_anchor
    XR_ERROR_SPATIAL_ANCHOR_ATTACHABLE_COMPONENT_NOT_FOUND_ANDROID = -1000790001,
  // Provided by XR_ANDROID_spatial_anchor_space
    XR_ERROR_SPATIAL_ANCHOR_ENTITY_ID_INVALID_ANDROID = -1000795001,
  // Provided by XR_ANDROID_geospatial_anchor
    XR_ERROR_SURFACE_ANCHOR_LOCATION_UNSUPPORTED_ANDROID = -1000797000,
  // Provided by XR_KHR_maintenance1
    XR_ERROR_EXTENSION_DEPENDENCY_NOT_ENABLED_KHR =
XR_ERROR_EXTENSION_DEPENDENCY_NOT_ENABLED,
  // Provided by XR_KHR_maintenance1
    XR_ERROR_PERMISSION_INSUFFICIENT_KHR = XR_ERROR_PERMISSION_INSUFFICIENT,
    XR_RESULT_MAX_ENUM = 0x7FFFFFFF
} XrResult;
```

All return codes in the API are reported via XrResult return values.

The following are common suffixes shared across many of the return codes:

- _INVALID: The specified handle, atom, or value is formatted incorrectly, or the specified handle was never created or has been destroyed.
- _UNSUPPORTED: The specified handle, atom, enumerant, or value is formatted correctly but cannot be used for the lifetime of this function's parent handle.
- _UNAVAILABLE: The specified handle, atom, enumerant, or value is supported by the handle taken by this function, but is not usable at this moment.

Success Codes

Enum	Description
XR_SUCCESS	Function successfully completed.
XR_TIMEOUT_EXPIRED	The specified timeout time occurred before the operation could complete.
XR_SESSION_LOSS_PENDING	The session will be lost soon.
XR_EVENT_UNAVAILABLE	No event was available.
XR_SPACE_BOUNDS_UNAVAILABLE	The space's bounds are not known at the moment.
XR_SESSION_NOT_FOCUSED	The session is not in the focused state.
XR_FRAME_DISCARDED	A frame has been discarded from composition.
XR_RENDER_MODEL_UNAVAILABLE_FB	The model is unavailable. (Added by the XR_FB_render_model extension)
XR_SCENE_MARKER_DATA_NOT_STRING_MSFT	Marker does not encode a string. (Added by the XR_MSFT_scene_marker extension)
XR_ENVIRONMENT_DEPTH_NOT_AVAILABLE_META	Warning: The requested depth image is not yet available. (Added by the XR_META_environment_depth extension)
XR_ERROR_FACIAL_EXPRESSION_PERMISSION_DENIED_ML	Permission to track facial expressions was not granted (Added by the XR_ML_facial_expression extension)
XR_BOUNDARY_VISIBILITY_SUPPRESSION_NOT_ALLOWED_META	Runtime did not accept the request to suppress the boundary visibility (Added by the XR_META_boundary_visibility extension)
XR_COLOCATION_DISCOVERY_ALREADY_ADVERTISING_META	Colocation advertisement has already been enabled (Added by the XR_META_colocation_discovery extension)
XR_COLOCATION_DISCOVERY_ALREADY_DISCOVERING_META	Colocation discovery has already been enabled (Added by the XR_META_colocation_discovery extension)

Error Codes

Enum	Description
XR_ERROR_VALIDATION_FAILURE	The function usage was invalid in some way.
XR_ERROR_RUNTIME_FAILURE	The runtime failed to handle the function in an unexpected way that is not covered by another error result.

Enum	Description
XR_ERROR_OUT_OF_MEMORY	A memory allocation has failed.
XR_ERROR_API_VERSION_UNSUPPORTED	The runtime does not support the requested API version.
XR_ERROR_INITIALIZATION_FAILED	Initialization of object could not be completed.
XR_ERROR_FUNCTION_UNSUPPORTED	The requested function was not found or is otherwise unsupported.
XR_ERROR_FEATURE_UNSUPPORTED	The requested feature is not supported.
XR_ERROR_EXTENSION_NOT_PRESENT	A requested extension is not supported.
XR_ERROR_LIMIT_REACHED	The runtime supports no more of the requested resource.
XR_ERROR_SIZE_INSUFFICIENT	The supplied size was smaller than required.
XR_ERROR_HANDLE_INVALID	A supplied object handle was invalid.
XR_ERROR_INSTANCE_LOST	The XrInstance was lost or could not be found. It will need to be destroyed and optionally recreated.
XR_ERROR_SESSION_RUNNING	The session is already running.
XR_ERROR_SESSION_NOT_RUNNING	The session is not yet running.
XR_ERROR_SESSION_LOST	The XrSession was lost. It will need to be destroyed and optionally recreated.
XR_ERROR_SYSTEM_INVALID	The provided XrSystemId was invalid.
XR_ERROR_PATH_INVALID	The provided XrPath was not valid.
XR_ERROR_PATH_COUNT_EXCEEDED	The maximum number of supported semantic paths has been reached.
XR_ERROR_PATH_FORMAT_INVALID	The semantic path character format is invalid.
XR_ERROR_PATH_UNSUPPORTED	The semantic path is unsupported.
XR_ERROR_LAYER_INVALID	The layer was NULL or otherwise invalid.
XR_ERROR_LAYER_LIMIT_EXCEEDED	The number of specified layers is greater than the supported number.
XR_ERROR_SWAPCHAIN_RECT_INVALID	The image rect was negatively sized or otherwise invalid.
XR_ERROR_SWAPCHAIN_FORMAT_UNSUPPORTED	The image format is not supported by the runtime or platform.
XR_ERROR_ACTION_TYPE_MISMATCH	The API used to retrieve an action's state does not match the action's type.

Enum	Description
XR_ERROR_SESSION_NOT_READY	The session is not in the ready state.
XR_ERROR_SESSION_NOT_STOPPING	The session is not in the stopping state.
XR_ERROR_TIME_INVALID	The provided XrTime was zero, negative, or out of range.
XR_ERROR_REFERENCE_SPACE_UNSUPPORTED	The specified reference space is not supported by the runtime or system.
XR_ERROR_FILE_ACCESS_ERROR	The file could not be accessed.
XR_ERROR_FILE_CONTENTS_INVALID	The file's contents were invalid.
XR_ERROR_FORM_FACTOR_UNSUPPORTED	The specified form factor is not supported by the current runtime or platform.
XR_ERROR_FORM_FACTOR_UNAVAILABLE	The specified form factor is supported, but the device is currently not available, e.g. not plugged in or powered off.
XR_ERROR_API_LAYER_NOT_PRESENT	A requested API layer is not present or could not be loaded.
XR_ERROR_CALL_ORDER_INVALID	The call was made without having made a previously required call.
XR_ERROR_GRAPHICS_DEVICE_INVALID	The given graphics device is not in a valid state. The graphics device could be lost or initialized without meeting graphics requirements.
XR_ERROR_POSE_INVALID	The supplied pose was invalid with respect to the requirements.
XR_ERROR_INDEX_OUT_OF_RANGE	The supplied index was outside the range of valid indices.
XR_ERROR_VIEW_CONFIGURATION_TYPE_UNSUPPORTED	The specified view configuration type is not supported by the runtime or platform.
XR_ERROR_ENVIRONMENT_BLEND_MODE_UNSUPPORTED	The specified environment blend mode is not supported by the runtime or platform.
XR_ERROR_NAME_DUPLICATED	The name provided was a duplicate of an already-existing resource.
XR_ERROR_NAME_INVALID	The name provided was invalid.
XR_ERROR_ACTIONSET_NOT_ATTACHED	A referenced action set is not attached to the session.
XR_ERROR_ACTIONSETS_ALREADY_ATTACHED	The session already has attached action sets.

Enum	Description
XR_ERROR_LOCALIZED_NAME_DUPLICATED	The localized name provided was a duplicate of an already-existing resource.
XR_ERROR_LOCALIZED_NAME_INVALID	The localized name provided was invalid.
XR_ERROR_GRAPHICS_REQUIREMENTS_CALL_MISSING	The xrGet*GraphicsRequirements call was not made before calling xrCreateSession.
XR_ERROR_RUNTIME_UNAVAILABLE	The loader was unable to find or load a runtime.
XR_ERROR_EXTENSION_DEPENDENCY_NOT_ENABLED	One or more of the extensions being enabled has dependency on extensions that are not enabled.
XR_ERROR_PERMISSION_INSUFFICIENT	Insufficient permissions. This error is included for use by vendor extensions. The precise definition of XR_ERROR_PERMISSION_INSUFFICIENT and actions possible by the developer or user to resolve it can vary by platform, extension or function. The developer should refer to the documentation of the function that returned the error code and extension it was defined.
XR_ERROR_ANDROID_THREAD_SETTINGS_ID_INVALID_KHR	xrSetAndroidApplicationThreadKHR failed as thread id is invalid. (Added by the XR_KHR_android_thread_settings extension)
XR_ERROR_ANDROID_THREAD_SETTINGS_FAILURE_KHR	xrSetAndroidApplicationThreadKHR failed setting the thread attributes/priority. (Added by the XR_KHR_android_thread_settings extension)
XR_ERROR_CREATE_SPATIAL_ANCHOR_FAILED_MSFT	Spatial anchor could not be created at that location. (Added by the XR_MSFT_spatial_anchor extension)
XR_ERROR_SECONDARY_VIEW_CONFIGURATION_TYPE_NOT_ENABLED_MSFT	The secondary view configuration was not enabled when creating the session. (Added by the XR_MSFT_secondary_view_configuration extension)
XR_ERROR_CONTROLLER_MODEL_KEY_INVALID_MSFT	The controller model key is invalid. (Added by the XR_MSFT_controller_model extension)
XR_ERROR_REPROJECTION_MODE_UNSUPPORTED_MSFT	The reprojection mode is not supported. (Added by the XR_MSFT_composition_layer_reprojection extension)
XR_ERROR_COMPUTE_NEW_SCENE_NOT_COMPLETED_MSFT	Compute new scene not completed. (Added by the XR_MSFT_scene_understanding extension)
XR_ERROR_SCENE_COMPONENT_ID_INVALID_MSFT	Scene component id invalid. (Added by the XR_MSFT_scene_understanding extension)

Enum	Description
XR_ERROR_SCENE_COMPONENT_TYPE_MISMATCH_MSFT	Scene component type mismatch. (Added by the XR_MSFT_scene_understanding extension)
XR_ERROR_SCENE_MESH_BUFFER_ID_INVALID_MSFT	Scene mesh buffer id invalid. (Added by the XR_MSFT_scene_understanding extension)
XR_ERROR_SCENE_COMPUTE_FEATURE_INCOMPATIBLE_MSFT	Scene compute feature incompatible. (Added by the XR_MSFT_scene_understanding extension)
XR_ERROR_SCENE_COMPUTE_CONSISTENCY_MISMATCH_MSFT	Scene compute consistency mismatch. (Added by the XR_MSFT_scene_understanding extension)
XR_ERROR_DISPLAY_REFRESH_RATE_UNSUPPORTED_FB	The display refresh rate is not supported by the platform. (Added by the XR_FB_display_refresh_rate extension)
XR_ERROR_COLOR_SPACE_UNSUPPORTED_FB	The color space is not supported by the runtime. (Added by the XR_FB_color_space extension)
XR_ERROR_SPACE_COMPONENT_NOT_SUPPORTED_FB	The component type is not supported for this space. (Added by the XR_FB_spatial_entity extension)
XR_ERROR_SPACE_COMPONENT_NOT_ENABLED_FB	The required component is not enabled for this space. (Added by the XR_FB_spatial_entity extension)
XR_ERROR_SPACE_COMPONENT_STATUS_PENDING_FB	A request to set the component's status is currently pending. (Added by the XR_FB_spatial_entity extension)
XR_ERROR_SPACE_COMPONENT_STATUS_ALREADY_SET_FB	The component is already set to the requested value. (Added by the XR_FB_spatial_entity extension)
XR_ERROR_UNEXPECTED_STATE_PASSTHROUGH_FB	The object state is unexpected for the issued command. (Added by the XR_FB_passthrough extension)
XR_ERROR_FEATURE_ALREADY_CREATED_PASSTHROUGH_FB	Trying to create an MR feature when one was already created and only one instance is allowed. (Added by the XR_FB_passthrough extension)
XR_ERROR_FEATURE_REQUIRED_PASSTHROUGH_FB	Requested functionality requires a feature to be created first. (Added by the XR_FB_passthrough extension)
XR_ERROR_NOT_PERMITTED_PASSTHROUGH_FB	Requested functionality is not permitted - application is not allowed to perform the requested operation. (Added by the XR_FB_passthrough extension)

Enum	Description
XR_ERROR_INSUFFICIENT_RESOURCES_PASSTHROUGH_FB	There were insufficient resources available to perform an operation. (Added by the XR_FB_passthrough extension)
XR_ERROR_UNKNOWN_PASSTHROUGH_FB	Unknown Passthrough error (no further details provided). (Added by the XR_FB_passthrough extension)
XR_ERROR_RENDER_MODEL_KEY_INVALID_FB	The model key is invalid. (Added by the XR_FB_render_model extension)
XR_ERROR_MARKER_NOT_TRACKED_VARJO	Marker tracking is disabled or the specified marker is not currently tracked. (Added by the XR_VARJO_marker_tracking extension)
XR_ERROR_MARKER_ID_INVALID_VARJO	The specified marker ID is not valid. (Added by the XR_VARJO_marker_tracking extension)
XR_ERROR_MARKER_DETECTOR_PERMISSION_DENIED_ML	The com.magicleap.permission.MARKER_TRACKING permission was denied. (Added by the XR_ML_marker_understanding extension)
XR_ERROR_MARKER_DETECTOR_LOCATE_FAILED_ML	The specified marker could not be located spatially. (Added by the XR_ML_marker_understanding extension)
XR_ERROR_MARKER_DETECTOR_INVALID_DATA_QUERY_ML	The marker queried does not contain data of the requested type. (Added by the XR_ML_marker_understanding extension)
XR_ERROR_MARKER_DETECTOR_INVALID_CREATE_INFO_ML	createInfo contains mutually exclusive parameters, such as setting XR_MARKER_DETECTOR_CORNER_REFINE_METHOD_APRIL_TAG_ML with XR_MARKER_TYPE_ARUCO_ML. (Added by the XR_ML_marker_understanding extension)
XR_ERROR_MARKER_INVALID_ML	The marker id passed to the function was invalid. (Added by the XR_ML_marker_understanding extension)
XR_ERROR_LOCALIZATION_MAP_INCOMPATIBLE_ML	The localization map being imported is not compatible with current OS or mode. (Added by the XR_ML_localization_map extension)
XR_ERROR_LOCALIZATION_MAP_UNAVAILABLE_ML	The localization map requested is not available. (Added by the XR_ML_localization_map extension)

Enum	Description
XR_ERROR_LOCALIZATION_MAP_FAIL_ML	The map localization service failed to fulfill the request, retry later. (Added by the XR_ML_localization_map extension)
XR_ERROR_LOCALIZATION_MAP_IMPORT_EXPORT_PERMISSION_DENIED_ML	The com.magicleap.permission.SPACE_IMPORT_EXPORT permission was denied. (Added by the XR_ML_localization_map extension)
XR_ERROR_LOCALIZATION_MAP_PERMISSION_DENIED_ML	The com.magicleap.permission.SPACE_MANAGER permission was denied. (Added by the XR_ML_localization_map extension)
XR_ERROR_LOCALIZATION_MAP_ALREADY_EXISTS_ML	The map being imported already exists in the system. (Added by the XR_ML_localization_map extension)
XR_ERROR_LOCALIZATION_MAP_CANNOT_EXPORT_CLOUD_MAP_ML	The map localization service cannot export cloud based maps. (Added by the XR_ML_localization_map extension)
XR_ERROR_SPATIAL_ANCHORS_PERMISSION_DENIED_ML	The com.magicleap.permission.SPATIAL_ANCHOR permission was not granted. (Added by the XR_ML_spatial_anchors extension)
XR_ERROR_SPATIAL_ANCHORS_NOT_LOCALIZED_ML	Operation failed because the system is not localized into a localization map. (Added by the XR_ML_spatial_anchors extension)
XR_ERROR_SPATIAL_ANCHORS_OUT_OF_MAP_BOUNDS_ML	Operation failed because it is performed outside of the localization map. (Added by the XR_ML_spatial_anchors extension)
XR_ERROR_SPATIAL_ANCHORS_SPACE_NOT_LOCATABLE_ML	Operation failed because the space referenced cannot be located. (Added by the XR_ML_spatial_anchors extension)
XR_ERROR_SPATIAL_ANCHORS_ANCHOR_NOT_FOUND_ML	The anchor references was not found. (Added by the XR_ML_spatial_anchors_storage extension)
XR_ERROR_SPATIAL_ANCHOR_NAME_NOT_FOUND_MSFT	A spatial anchor was not found associated with the spatial anchor name provided (Added by the XR_MSFT_spatial_anchor_persistence extension)
XR_ERROR_SPATIAL_ANCHOR_NAME_INVALID_MSFT	The spatial anchor name provided was not valid (Added by the XR_MSFT_spatial_anchor_persistence extension)

Enum	Description
XR_ERROR_SPACE_MAPPING_INSUFFICIENT_FB	Anchor import from cloud or export from device failed. (Added by the XR_FB_spatial_entity_sharing extension)
XR_ERROR_SPACE_LOCALIZATION_FAILED_FB	Anchors were downloaded from the cloud but failed to be imported/aligned on the device. (Added by the XR_FB_spatial_entity_sharing extension)
XR_ERROR_SPACE_NETWORK_TIMEOUT_FB	Timeout occurred while waiting for network request to complete. (Added by the XR_FB_spatial_entity_sharing extension)
XR_ERROR_SPACE_NETWORK_REQUEST_FAILED_FB	The network request failed. (Added by the XR_FB_spatial_entity_sharing extension)
XR_ERROR_SPACE_CLOUD_STORAGE_DISABLED_FB	Cloud storage is required for this operation but is currently disabled. (Added by the XR_FB_spatial_entity_sharing extension)
XR_ERROR_SPACE_INSUFFICIENT_RESOURCES_META	Resource limitation prevented this operation from executing. Recommend retrying, perhaps after a short delay and/or reducing memory consumption. (Added by the XR_META_spatial_entity_persistence extension)
XR_ERROR_SPACE_STORAGE_AT_CAPACITY_META	Operation could not be completed until resources used are reduced or storage expanded. (Added by the XR_META_spatial_entity_persistence extension)
XR_ERROR_SPACE_INSUFFICIENT_VIEW_META	Look around the environment more for space tracking to function. (Added by the XR_META_spatial_entity_persistence extension)
XR_ERROR_SPACE_PERMISSION_INSUFFICIENT_META	Space operation permission insufficient. Recommend confirming the status of the required permissions needed for using Space APIs. (Added by the XR_META_spatial_entity_persistence extension)
XR_ERROR_SPACE_RATE_LIMITED_META	Operation cancelled due to rate limiting. Recommend retrying after a short delay. (Added by the XR_META_spatial_entity_persistence extension)
XR_ERROR_SPACE_TOO_DARK_META	Environment too dark for tracking to complete operation. (Added by the XR_META_spatial_entity_persistence extension)

Enum	Description
XR_ERROR_SPACE_TOO_BRIGHT_META	Environment too bright for tracking to complete operation. (Added by the XR_META_spatial_entity_persistence extension)
XR_ERROR_PASSTHROUGH_COLOR_LUT_BUFFER_SIZE_MISMATCH_META	The provided data buffer did not match the required size. (Added by the XR_META_passthrough_color_lut extension)
XR_ERROR_RENDER_MODEL_ID_INVALID_EXT	The render model ID is invalid. (Added by the XR_EXT_render_model extension)
XR_ERROR_RENDER_MODEL_ASSET_UNAVAILABLE_EXT	The render model asset is unavailable. (Added by the XR_EXT_render_model extension)
XR_ERROR_RENDER_MODEL_GLTF_EXTENSION_REQUIRED_EXT	A glTF extension is required. (Added by the XR_EXT_render_model extension)
XR_ERROR_NOT_INTERACTION_RENDER_MODEL_EXT	The provided XrRenderModelEXT was not created from a XrRenderModelIdEXT from XR_EXT_interaction_render_model (Added by the XR_EXT_interaction_render_model extension)
XR_ERROR_HINT_ALREADY_SET_QCOM	Tracking optimization hint is already set for the domain. (Added by the XR_QCOM_tracking_optimization_settings extension)
XR_ERROR_NOT_AN_ANCHOR_HTC	The provided space is valid but not an anchor. (Added by the XR_HTC_anchor extension)
XR_ERROR_SPATIAL_ENTITY_ID_INVALID_BD	The spatial entity id is invalid. (Added by the XR_BD_spatial_sensing extension)
XR_ERROR_SPATIAL_SENSING_SERVICE_UNAVAILABLE_BD	The spatial sensing service is unavailable. (Added by the XR_BD_spatial_sensing extension)
XR_ERROR_ANCHOR_NOT_SUPPORTED_FOR_ENTITY_BD	The spatial entity does not support anchor. (Added by the XR_BD_spatial_sensing extension)
XR_ERROR_SPATIAL_ANCHOR_NOT_FOUND_BD	The spatial anchor is not found. (Added by the XR_BD_spatial_anchor extension)
XR_ERROR_SPATIAL_ANCHOR_SHARING_NETWORK_TIMEOUT_BD	The network transmission timeout. (Added by the XR_BD_spatial_anchor_sharing extension)
XR_ERROR_SPATIAL_ANCHOR_SHARING_AUTHENTICATION_FAILURE_BD	The authentication for the user account failed. (Added by the XR_BD_spatial_anchor_sharing extension)

Enum	Description
XR_ERROR_SPATIAL_ANCHOR_SHARING_NETWORK_FAILURE_BD	The network connection failed, e.g. the connection is unstable or disconnected. (Added by the XR_BD_spatial_anchor_sharing extension)
XR_ERROR_SPATIAL_ANCHOR_SHARING_LOCALIZATION_FAIL_BD	The spatial anchor localization failed. (Added by the XR_BD_spatial_anchor_sharing extension)
XR_ERROR_SPATIAL_ANCHOR_SHARING_MAP_INSUFFICIENT_BD	The feature points of spatial anchor map are insufficient. (Added by the XR_BD_spatial_anchor_sharing extension)
XR_ERROR_SCENE_CAPTURE_FAILURE_BD	The scene capture is failed, for example exiting abnormally. (Added by the XR_BD_spatial_scene extension)
XR_ERROR_SPACE_NOT_LOCATABLE_EXT	The space passed to the function was not locatable. (Added by the XR_EXT_plane_detection extension)
XR_ERROR_PLANE_DETECTION_PERMISSION_DENIED_EXT	The permission for this resource was not granted. (Added by the XR_EXT_plane_detection extension)
XR_ERROR_MISMATCHING_TRACKABLE_TYPE_ANDROID	Indicates that the parameters contains multiple trackable types. (Added by the XR_ANDROID_trackables extension)
XR_ERROR_TRACKABLE_TYPE_NOT_SUPPORTED_ANDROID	Indicates that the function is not supported by the given trackable type. (Added by the XR_ANDROID_trackables extension)
XR_ERROR_ANCHOR_ID_NOT_FOUND_ANDROID	XrUuidExt passed to the function was not found to be a persisted anchor. (Added by the XR_ANDROID_device_anchor_persistence extension)
XR_ERROR_ANCHOR_ALREADY_PERSISTED_ANDROID	XrUuidExt passed to the function was already marked to be persisted. (Added by the XR_ANDROID_device_anchor_persistence extension)
XR_ERROR_ANCHOR_NOT_TRACKING_ANDROID	ANchor cannot be persisted because it is not tracking. (Added by the XR_ANDROID_device_anchor_persistence extension)
XR_ERROR_PERSISTED_DATA_NOT_READY_ANDROID	Persisted data stored by this app has not been loaded yet. (Added by the XR_ANDROID_device_anchor_persistence extension)
XR_ERROR_SERVICE_NOT_READY_ANDROID	The underlying tracking service is not yet ready. (Added by the XR_ANDROID_face_tracking extension)

Enum	Description
XR_ERROR_MESH_DATA_LIMIT_EXCEEDED_ANDROID	Returned by passthrough layer functions to indicate the mesh data exceeds the supported limit. (Added by the XR_ANDROID_composition_layer_passthrough_mesh extension)
XR_ERROR_FUTURE_PENDING_EXT	Returned by completion function to indicate future is not ready. (Added by the XR_EXT_future extension)
XR_ERROR_FUTURE_INVALID_EXT	Returned by completion function to indicate future is not valid. (Added by the XR_EXT_future extension)
XR_ERROR_SYSTEM_NOTIFICATION_PERMISSION_DENIED_ML	The com.magicleap.permission.SYSTEM_NOTIFICATION permission was not granted. (Added by the XR_ML_system_notifications extension)
XR_ERROR_SYSTEM_NOTIFICATION_INCOMPATIBLE_SKU_ML	Incompatible SKU detected. (Added by the XR_ML_system_notifications extension)
XR_ERROR_WORLD_MESH_DETECTOR_PERMISSION_DENIED_ML	The world mesh detector permission was not granted. (Added by the XR_ML_world_mesh_detection extension)
XR_ERROR_WORLD_MESH_DETECTOR_SPACE_NOT_LOCATABLE_ML	At the time of the call the runtime was unable to locate the space and cannot fulfill your request. (Added by the XR_ML_world_mesh_detection extension)
XR_ERROR_COLOCATION_DISCOVERY_NETWORK_FAILED_META	The network request failed. (Added by the XR_META_colocation_discovery extension)
XR_ERROR_COLOCATION_DISCOVERY_NO_DISCOVERY_METHOD_META	The runtime does not have any methods available to perform discovery. (Added by the XR_META_colocation_discovery extension)
XR_ERROR_SPACE_GROUP_NOT_FOUND_META	The group UUID was not found within the runtime (Added by the XR_META_spatial_entity_group_sharing extension)
XR_ERROR_ANCHOR_NOT_OWNED_BY_CALLER_ANDROID	Operation not allowed because anchor is not owned by the XrSession in which the function is being called. (Added by the XR_ANDROID_anchor_sharing_export extension)

Enum	Description
XR_ERROR_IMAGE_FORMAT_UNSUPPORTED_ANDROID	Indicates that the input reference image format is not supported. (Added by the XR_ANDROID_trackables_image extension)
XR_ERROR_EXTENSION_DEPENDENCY_NOT_ENABLED_KHR	Alias for XR_ERROR_EXTENSION_DEPENDENCY_NOT_ENABLED (Added by the XR_KHR_maintenance1 extension)
XR_ERROR_PERMISSION_INSUFFICIENT_KHR	Alias for XR_ERROR_PERMISSION_INSUFFICIENT (Added by the XR_KHR_maintenance1 extension)
XR_ERROR_SPATIAL_CAPABILITY_UNSUPPORTED_EXT	The specified spatial capability is not supported by the runtime or the system. (Added by the XR_EXT_spatial_entity extension)
XR_ERROR_SPATIAL_ENTITY_ID_INVALID_EXT	The specified spatial entity id is invalid or an entity with that id does not exist in the environment. (Added by the XR_EXT_spatial_entity extension)
XR_ERROR_SPATIAL_BUFFER_ID_INVALID_EXT	The specified spatial buffer id is invalid or does not exist in the spatial snapshot being used to query for the buffer data. (Added by the XR_EXT_spatial_entity extension)
XR_ERROR_SPATIAL_COMPONENT_UNSUPPORTED_FOR_CAPABILITY_EXT	The specified spatial component is not supported by the runtime or the system for the given capability. (Added by the XR_EXT_spatial_entity extension)
XR_ERROR_SPATIAL_CAPABILITY_CONFIGURATION_INVALID_EXT	The specified spatial capability configuration is invalid. (Added by the XR_EXT_spatial_entity extension)
XR_ERROR_SPATIAL_COMPONENT_NOT_ENABLED_EXT	The specified spatial component is not enabled for the spatial context. (Added by the XR_EXT_spatial_entity extension)
XR_ERROR_SPATIAL_PERSISTENCE_SCOPE_UNSUPPORTED_EXT	The specified spatial persistence scope is not supported by the runtime or the system. (Added by the XR_EXT_spatial_persistence extension)
XR_ERROR_SPATIAL_PERSISTENCE_SCOPE_INCOMPATIBLE_EXT	The scope configured for the persistence context is incompatible for the current spatial entity. (Added by the XR_EXT_spatial_persistence_operations extension)

Enum	Description
XR_ERROR_SPATIAL_ANCHOR_ATTACHABLE_COMPONENT_NOT_FOUND_ANDROID	Spatial entity with parent ID does not have at least one component enumerated by xrEnumerateSpatialAnchorAttachableComponentsANDROID (Added by the XR_ANDROID_spatial_entity_bound_anchor extension)
XR_ERROR_SPATIAL_ANCHOR_ENTITY_ID_INVALID_ANDROID	The specified spatial entity ID is invalid for an anchor. (Added by the XR_ANDROID_spatial_anchor_space extension)

2.10.1. Convenience Macros

```
// Provided by XR_VERSION_1_0
#define XR_SUCCEEDED(result) ((result) >= 0)
```

A convenience macro that **can** be used to test if a function succeeded. Note that this evaluates to true for all success codes, including a qualified success such as XR_FRAME_DISCARDED.

```
// Provided by XR_VERSION_1_0
#define XR_FAILED(result) ((result) < 0)
```

A convenience macro that **can** be used to test if a function has failed in some way. It evaluates to true for all failure codes.

```
// Provided by XR_VERSION_1_0
#define XR_UNQUALIFIED_SUCCESS(result) ((result) == 0)
```

A convenience macro that can be used to test a function's failure. The XR_UNQUALIFIED_SUCCESS macro evaluates to true exclusively when the provided XrResult is equal to XR_SUCCESS (0).

2.10.2. Validation

Except as noted below or in individual API specifications, valid API usage **may** be required by the runtime. Runtimes **may** choose to validate some API usage and return an appropriate error code.

Application developers **should** use validation layers to catch and eliminate errors during development. Once validated, applications **should** not enable validation layers by default.

If a function returns a run time error, unless otherwise specified any output parameters will have undefined contents, except that if the output parameter is a structure with type and next fields, those

fields will be unmodified. Any output structures chained from `next` will also have undefined contents, except that the `type` and `next` will be unmodified.

Unless otherwise specified, errors do not affect existing OpenXR objects. Objects that have already been successfully created **may** still be used by the application.

XrResult code returns **may** be added to a given function in future versions of the specification. Runtimes **must** return only XrResult codes from the set documented for the given application API version.

Runtimes **must** ensure that incorrect usage by an application does not affect the integrity of the operating system, the API implementation, or other API client applications in the system, and does not allow one application to access data belonging to another application.

2.11. Handles

Objects which are allocated by the runtime on behalf of applications are represented by handles. Handles are opaque identifiers for objects whose lifetime is controlled by applications via the create and destroy functions. Example handle types include XrInstance, XrSession, and XrSwapchain. Handles which have not been destroyed are unique for a given application process, but **may** be reused after being destroyed. Unless otherwise specified, a successful handle creation function call returns a new unique handle. Unless otherwise specified, handles are implicitly destroyed when their parent handle is destroyed. Applications **may** destroy handles explicitly before the parent handle is destroyed, and **should** do so if no longer needed, in order to conserve resources. Runtimes **may** detect XR_NULL_HANDLE and other invalid handles passed where a valid handle is required and return `XR_ERROR_HANDLE_INVALID`. However, runtimes are not required to do so unless otherwise specified, and so use of any invalid handle **may** result in undefined behavior. When a function has an **optional** handle parameter, XR_NULL_HANDLE **must** be passed by the application if it does not pass a valid handle.

All functions that take a handle parameter **may** return `XR_ERROR_HANDLE_INVALID`.

Handles form a hierarchy in which child handles fall under the validity and lifetime of parent handles. For example, to create an XrSwapchain handle, applications must call xrCreateSwapchain and pass an XrSession handle. Thus XrSwapchain is a child handle of XrSession.

2.12. Object Handle Types

The type of an object handle used in a function is usually determined by the specification of that function, as discussed in Valid Usage for Object Handles. However, some functions accept or return object handle parameters where the type of the object handle is unknown at execution time and is not specified in the description of the function itself. For these functions, the XrObjectType **may** be used to explicitly specify the type of a handle.

For example, an information-gathering or debugging mechanism implemented in a runtime extension

or API layer extension **may** return a list of object handles that are generated by the mechanism's operation. The same mechanism **may** also return a parallel list of object handle types that allow the recipient of this information to easily determine the types of the handles.

In general, anywhere an object handle of more than one type can occur, the object handle type **may** be provided to indicate its type.

```
// Provided by XR_VERSION_1_0
typedef enum XrObjectType {
    XR_OBJECT_TYPE_UNKNOWN = 0,
    XR_OBJECT_TYPE_INSTANCE = 1,
    XR_OBJECT_TYPE_SESSION = 2,
    XR_OBJECT_TYPE_SWAPCHAIN = 3,
    XR_OBJECT_TYPE_SPACE = 4,
    XR_OBJECT_TYPE_ACTION_SET = 5,
    XR_OBJECT_TYPE_ACTION = 6,
  // Provided by XR_EXT_debug_utils
    XR_OBJECT_TYPE_DEBUG_UTILS_MESSENGER_EXT = 1000019000,
  // Provided by XR_MSFT_spatial_anchor
    XR_OBJECT_TYPE_SPATIAL_ANCHOR_MSFT = 1000039000,
  // Provided by XR_MSFT_spatial_graph_bridge
    XR_OBJECT_TYPE_SPATIAL_GRAPH_NODE_BINDING_MSFT = 1000049000,
  // Provided by XR_EXT_hand_tracking
    XR_OBJECT_TYPE_HAND_TRACKER_EXT = 1000051000,
  // Provided by XR_FB_body_tracking
    XR_OBJECT_TYPE_BODY_TRACKER_FB = 1000076000,
  // Provided by XR_MSFT_scene_understanding
    XR_OBJECT_TYPE_SCENE_OBSERVER_MSFT = 1000097000,
  // Provided by XR_MSFT_scene_understanding
    XR_OBJECT_TYPE_SCENE_MSFT = 1000097001,
  // Provided by XR_HTC_facial_tracking
    XR_OBJECT_TYPE_FACIAL_TRACKER_HTC = 1000104000,
  // Provided by XR_FB_foveation
    XR_OBJECT_TYPE_FOVEATION_PROFILE_FB = 1000114000,
  // Provided by XR_FB_triangle_mesh
    XR_OBJECT_TYPE_TRIANGLE_MESH_FB = 1000117000,
  // Provided by XR_FB_passthrough
    XR_OBJECT_TYPE_PASSTHROUGH_FB = 1000118000,
  // Provided by XR_FB_passthrough
    XR_OBJECT_TYPE_PASSTHROUGH_LAYER_FB = 1000118002,
  // Provided by XR_FB_passthrough
    XR_OBJECT_TYPE_GEOMETRY_INSTANCE_FB = 1000118004,
  // Provided by XR_ML_marker_understanding
    XR_OBJECT_TYPE_MARKER_DETECTOR_ML = 1000138000,
  // Provided by XR_ML_localization_map
    XR_OBJECT_TYPE_EXPORTED_LOCALIZATION_MAP_ML = 1000139000,
  // Provided by XR_ML_spatial_anchors_storage
```

```
    XR_OBJECT_TYPE_SPATIAL_ANCHORS_STORAGE_ML = 1000141000,
  // Provided by XR_MSFT_spatial_anchor_persistence
    XR_OBJECT_TYPE_SPATIAL_ANCHOR_STORE_CONNECTION_MSFT = 1000142000,
  // Provided by XR_FB_face_tracking
    XR_OBJECT_TYPE_FACE_TRACKER_FB = 1000201000,
  // Provided by XR_FB_eye_tracking_social
    XR_OBJECT_TYPE_EYE_TRACKER_FB = 1000202000,
  // Provided by XR_META_virtual_keyboard
    XR_OBJECT_TYPE_VIRTUAL_KEYBOARD_META = 1000219000,
  // Provided by XR_FB_spatial_entity_user
    XR_OBJECT_TYPE_SPACE_USER_FB = 1000241000,
  // Provided by XR_META_passthrough_color_lut
    XR_OBJECT_TYPE_PASSTHROUGH_COLOR_LUT_META = 1000266000,
  // Provided by XR_FB_face_tracking2
    XR_OBJECT_TYPE_FACE_TRACKER2_FB = 1000287012,
  // Provided by XR_META_environment_depth
    XR_OBJECT_TYPE_ENVIRONMENT_DEPTH_PROVIDER_META = 1000291000,
  // Provided by XR_META_environment_depth
    XR_OBJECT_TYPE_ENVIRONMENT_DEPTH_SWAPCHAIN_META = 1000291001,
  // Provided by XR_EXT_render_model
    XR_OBJECT_TYPE_RENDER_MODEL_EXT = 1000300000,
  // Provided by XR_EXT_render_model
    XR_OBJECT_TYPE_RENDER_MODEL_ASSET_EXT = 1000300001,
  // Provided by XR_HTC_passthrough
    XR_OBJECT_TYPE_PASSTHROUGH_HTC = 1000317000,
  // Provided by XR_HTC_body_tracking
    XR_OBJECT_TYPE_BODY_TRACKER_HTC = 1000320000,
  // Provided by XR_BD_body_tracking
    XR_OBJECT_TYPE_BODY_TRACKER_BD = 1000385000,
  // Provided by XR_BD_facial_simulation
    XR_OBJECT_TYPE_FACE_TRACKER_BD = 1000386000,
  // Provided by XR_BD_spatial_sensing
    XR_OBJECT_TYPE_SENSE_DATA_PROVIDER_BD = 1000389000,
  // Provided by XR_BD_spatial_sensing
    XR_OBJECT_TYPE_SENSE_DATA_SNAPSHOT_BD = 1000389001,
  // Provided by XR_BD_spatial_sensing
    XR_OBJECT_TYPE_ANCHOR_BD = 1000389002,
  // Provided by XR_BD_spatial_audio_rendering
    XR_OBJECT_TYPE_SPATIAL_AUDIO_RENDERER_BD = 1000409000,
  // Provided by XR_BD_spatial_audio_rendering
    XR_OBJECT_TYPE_SOUND_FIELD_BD = 1000409001,
  // Provided by XR_BD_spatial_audio_rendering
    XR_OBJECT_TYPE_SOUND_OBJECT_BD = 1000409002,
  // Provided by XR_BD_spatial_audio_rendering
    XR_OBJECT_TYPE_SOUND_OBSTACLE_BD = 1000409003,
  // Provided by XR_BD_spatial_audio_rendering
    XR_OBJECT_TYPE_SOUND_OBSTACLE_MATERIAL_BD = 1000409004,
  // Provided by XR_EXT_plane_detection
```

```
    XR_OBJECT_TYPE_PLANE_DETECTOR_EXT = 1000429000,
  // Provided by XR_ANDROID_trackables
    XR_OBJECT_TYPE_TRACKABLE_TRACKER_ANDROID = 1000455001,
  // Provided by XR_ANDROID_eye_tracking
    XR_OBJECT_TYPE_EYE_TRACKER_ANDROID = 1000456000,
  // Provided by XR_ANDROID_device_anchor_persistence
    XR_OBJECT_TYPE_DEVICE_ANCHOR_PERSISTENCE_ANDROID = 1000457000,
  // Provided by XR_ANDROID_face_tracking
    XR_OBJECT_TYPE_FACE_TRACKER_ANDROID = 1000458000,
  // Provided by XR_ANDROID_composition_layer_passthrough_mesh
    XR_OBJECT_TYPE_PASSTHROUGH_LAYER_ANDROID = 1000462000,
  // Provided by XR_ML_world_mesh_detection
    XR_OBJECT_TYPE_WORLD_MESH_DETECTOR_ML = 1000474000,
  // Provided by XR_ML_facial_expression
    XR_OBJECT_TYPE_FACIAL_EXPRESSION_CLIENT_ML = 1000482000,
  // Provided by XR_META_environment_raycast
    XR_OBJECT_TYPE_ENVIRONMENT_RAYCASTER_META = 1000592000,
  // Provided by XR_ANDROID_light_estimation
    XR_OBJECT_TYPE_LIGHT_ESTIMATOR_ANDROID = 1000700000,
  // Provided by XR_ANDROID_trackables_image
    XR_OBJECT_TYPE_TRACKABLE_IMAGE_DATABASE_ANDROID = 1000709000,
  // Provided by XR_ANDROID_scene_meshing
    XR_OBJECT_TYPE_SCENE_MESHING_TRACKER_ANDROID = 1000718000,
  // Provided by XR_ANDROID_scene_meshing
    XR_OBJECT_TYPE_SCENE_MESH_SNAPSHOT_ANDROID = 1000718001,
  // Provided by XR_EXT_spatial_entity
    XR_OBJECT_TYPE_SPATIAL_ENTITY_EXT = 1000740000,
  // Provided by XR_EXT_spatial_entity
    XR_OBJECT_TYPE_SPATIAL_CONTEXT_EXT = 1000740001,
  // Provided by XR_EXT_spatial_entity
    XR_OBJECT_TYPE_SPATIAL_SNAPSHOT_EXT = 1000740002,
  // Provided by XR_EXT_spatial_persistence
    XR_OBJECT_TYPE_SPATIAL_PERSISTENCE_CONTEXT_EXT = 1000763000,
  // Provided by XR_ANDROID_geospatial
    XR_OBJECT_TYPE_GEOSPATIAL_TRACKER_ANDROID = 1000789000,
    XR_OBJECT_TYPE_MAX_ENUM = 0x7FFFFFFF
} XrObjectType;
```

The XrObjectType enumeration defines values, each of which corresponds to a specific OpenXR handle type. These values **can** be used to associate debug information with a particular type of object through one or more extensions.

The following table defines XrObjectType and OpenXR Handle relationships in the core specification:

XrObjectType	OpenXR Handle Type
XR_OBJECT_TYPE_UNKNOWN	Unknown/Undefined Handle

XrObjectType	OpenXR Handle Type
XR_OBJECT_TYPE_INSTANCE	XrInstance
XR_OBJECT_TYPE_SESSION	XrSession
XR_OBJECT_TYPE_SWAPCHAIN	XrSwapchain
XR_OBJECT_TYPE_SPACE	XrSpace
XR_OBJECT_TYPE_ACTION_SET	XrActionSet
XR_OBJECT_TYPE_ACTION	XrAction

2.13. Buffer Size Parameters

Functions with buffer or array parameters passed as pointers, rather than declared with a static array size, follow different conventions depending on whether the buffer size is known to the application or variable per call.

2.13.1. Variable size buffer parameters

Functions with variable size output buffer parameters take on either parameter form or structure form, as in one of the following examples, with the element type being `float` in this case:

Parameter form:

```
XrResult xrFunction(uint32_t elementCapacityInput, uint32_t* elementCountOutput, float*
elements);
```

Structure form:

```
XrResult xrFunction(XrBuffer* buffer);

struct XrBuffer {
    uint32_t              elementCapacityInput;
    uint32_t              elementCountOutput;
    float*                elements;
};
```

A "two-call idiom" **should** be employed by the application, first calling `xrFunction` (with a valid `elementCountOutput` pointer if in parameter form), but passing `NULL` as `elements` and `0` as `elementCapacityInput`, to retrieve the required buffer size as number of elements (number of floats in this example). After allocating a buffer at least as large as `elementCountOutput` (in a structure) or the value pointed to by `elementCountOutput` (as parameters), a pointer to the allocated buffer **should** be passed as `elements`, along with the buffer's length in `elementCapacityInput`, to a second call to `xrFunction` to perform the retrieval of the data. If the element type of `elements` is a structure with `type` and `next`

fields, the application **must** set the `type` to the correct value, and **must** set `next` to a valid value. A valid value for `next` is generally either `NULL` or another structure with related data, in which `type` and `next` are also valid, recursively. (See Valid Usage for Structure Pointer Chains for details.)

In the following discussion, "set `elementCountOutput`" should be interpreted as "set the value pointed to by `elementCountOutput`" in parameter form and "set the value of `elementCountOutput`" in struct form. These functions have the following behavior with respect to the array/buffer and its size parameters:

Buffer Size Parameter Behavior

- The `elementCapacityInput` and `elementCountOutput` arguments precede the array to which they refer, in argument order.
- `elementCapacityInput` specifies the capacity in number of elements of the buffer to be written, or `0` to indicate a request for the required buffer size.
- Independent of `elementCapacityInput` or `elements` parameters, the application **must** pass a valid pointer for `elementCountOutput` if the function uses parameter form.
- Independent of `elementCapacityInput` or `elements` parameters, the function sets `elementCountOutput`.
- The application **may** pass `0` for the `elementCapacityInput` parameter, to indicate a request for the required array size. That is, passing a capacity of `0` does not return `XR_ERROR_SIZE_INSUFFICIENT`. In this case, the following two points apply.
  - The function **must** set `elementCountOutput` to the required size in number of elements.
  - The `elements` parameter is ignored (any value passed is considered valid usage).
- If the `elementCapacityInput` is non-zero but less than required, the function **must** set `elementCountOutput` to the required capacity, and **must** return `XR_ERROR_SIZE_INSUFFICIENT`. After the function returns, the data in the array `elements` is undefined.
- If the `elementCapacityInput` is non-zero and the function returns successfully, the function sets `elementCountOutput` to the count of the elements that have been written to `elements`.
- If the function returns any `XR_ERROR_*` result other than `XR_ERROR_SIZE_INSUFFICIENT`, the contents of the values of (or pointed to by) `elementCountOutput` and `elements` are undefined.
- For clarity, if the element array refers to a string (`element` is of type `char*`), `elementCapacityInput` and `elementCountOutput` refer to the string `strlen` plus `1` for a `NULL` terminator.

Some functions have a given `elementCapacityInput` and `elementCountOutput` associated with more than one element array (i.e. parallel arrays). In this case, the capacity/count and all its associated arrays will share a common name prefix. All of the preceding general requirements continue to apply.

Some functions fill multiple element arrays of varying sizes in one call. For these functions, the `elementCapacityInput`, `elementCountOutput`, and `elements` array parameters or fields are repeated with

different prefixes. In this case, all of the preceding general requirements still apply, with these additional requirements:

- If the application sets **any** `elementCapacityInput` parameter or field to `0`, the runtime **must** treat **all** `elementCapacityInput` values as if they were set to `0`.
- If all `elementCapacityInput` values are non-zero but **any** is insufficient to fit all elements of its corresponding array, the runtime **must** return `XR_ERROR_SIZE_INSUFFICIENT`. As in the case of the single array, the data in all arrays is undefined when the function returns any `XR_ERROR_*` result.

2.13.2. Known size buffer parameters

Functions with known size input and/or output buffer parameters, or buffer parameters of an application-chosen size, take a slightly different approach than variable size buffer parameters. Such functions also take on either parameter form or structure form, as in the following examples:

Parameter form:

```
XrResult xrFunction(uint32_t elementCount, float* elements);
```

Structure form:

```
XrResult xrFunction(XrBuffer* buffer);

struct XrBuffer {
    uint32_t              elementCount;
    float*                elements;
};
```

Unlike for variable size buffer parameters, only a single "count" is specified per buffer/array. Functions following this convention have the following behavior with respect to the array/buffer and its count parameters:

Buffer Size Parameter Behavior

- The `elementCount` argument precedes the array to which it refers, in argument order.
- `elementCount` specifies the number of elements passed as input, or both the capacity in number of elements of the buffer to be written and the exact number of elements expected.
- `elements` **must** be at least `elementCount` elements in size.
- If the array is used as output or input/output and `elementCount` is not equal to the exact number of elements expected, the function **must** return `XR_ERROR_VALIDATION_FAILURE`.
- If the array is used as output or input/output and the function returns any `XR_ERROR_*`, including `XR_ERROR_VALIDATION_FAILURE`, the contents of the data in the array `elements` is undefined.
- If the array is used as input, all `elementCount` elements of `elements` **must** be valid unless otherwise specified.

Some functions have a given `elementCount` associated with more than one element array (i.e. parallel arrays). In this case, the count and all its associated arrays will share a common name prefix. All of the preceding general requirements continue to apply.

Some functions operate on multiple element arrays of known sizes in one call. For these functions, the `elementCount`, and `elements` array parameters or fields are repeated with different prefixes. As in the case of the single array, the data in all arrays is undefined when the function returns any `XR_ERROR_*` result. All of the preceding general requirements continue to apply.

2.14. Time

Time is represented by a 64-bit signed integer representing nanoseconds (`XrTime`). The passage of time **must** be monotonic and not real-time (i.e. wall clock time). Thus the time is always increasing at a constant rate and is unaffected by clock changes, time zones, daylight savings, etc.

2.14.1. XrTime

```
// Provided by XR_VERSION_1_0
typedef int64_t XrTime;
```

`XrTime` is a base value type that represents time as a signed 64-bit integer, representing the monotonically-increasing count of nanoseconds that have elapsed since a runtime-chosen epoch. `XrTime` always represents the time elapsed since that constant epoch, rather than a duration or a time point relative to some moving epoch such as vsync time, etc. Durations are instead represented by `XrDuration`.

A single runtime **must** use the same epoch for all simultaneous applications. Time **must** be

represented the same regardless of multiple processors or threads present in the system.

The period precision of time reported by the runtime is runtime-dependent, and **may** change. One nanosecond is the finest possible period precision. A runtime **may**, for example, report time progression with only microsecond-level granularity.

Time **must** not be assumed to correspond to a system clock time.

Unless specified otherwise, zero or a negative value is not a valid `XrTime`, and related functions **must** return error `XR_ERROR_TIME_INVALID`. Applications **must** not initialize such `XrTime` fields to a zero value. Instead, applications **should** always assign `XrTime` fields to the meaningful point in time they are choosing to reason about, such as a frame's predicted display time, or an action's last change time.

The behavior of a runtime is undefined when time overflows beyond the maximum positive value that can be represented by an `XrTime`. Runtimes **should** choose an epoch that minimizes the chance of overflow. Runtimes **should** also choose an epoch that minimizes the chance of underflow below 0 for applications performing a reasonable amount of historical pose lookback. For example, if the runtime chooses an epoch relative to its startup time, it **should** push the epoch into the past by enough time to avoid applications performing reasonable pose lookback from reaching a negative `XrTime` value.

An application cannot assume that the system's clock and the runtime's clock will maintain a constant relationship across frames and **should** avoid storing such an offset, as this may cause time drift. Applications **should** instead always use time interop functions to convert a relevant time point across the system's clock and the runtime's clock using extensions, for example, `XR_KHR_win32_convert_performance_counter_time` or `XR_KHR_convert_timespec_time`.

2.15. Duration

Duration refers to an elapsed period of time, as opposed to an absolute timepoint.

2.15.1. XrDuration

```
// Provided by XR_VERSION_1_0
typedef int64_t XrDuration;
```

The difference between two timepoints is a duration, and thus the difference between two `XrTime` values is an `XrDuration` value. `XrDuration` is a base value type that represents duration as a signed 64-bit integer, representing the signed number of nanoseconds between two timepoints.

Functions that refer to durations use `XrDuration` as opposed to `XrTime`. When an `XrDuration` is used as a timeout parameter, the constants `XR_NO_DURATION` and `XR_INFINITE_DURATION` have special meaning. A timeout with a duration that refers to the past (that is, a negative duration) **must** be interpreted as a timeout of `XR_NO_DURATION`.

The interpretation of zero and negative durations in non-timeout uses is specified along with each

such use.

```
// Provided by XR_VERSION_1_0
#define XR_NO_DURATION 0
```

For the case of timeout durations, XR_NO_DURATION **can** be used to indicate that the timeout is immediate.

```
// Provided by XR_VERSION_1_0
#define XR_INFINITE_DURATION 0x7fffffffffffffffLL
```

XR_INFINITE_DURATION is a special value that **can** be used to indicate that the timeout never occurs.

2.16. Prediction Time Limits

Some functions involve prediction or history retrieval for a supplied XrTime timepoint. For example, xrLocateViews accepts a display time for which to return the resulting data. Timepoints provided by applications **may** refer to time in the past or the future. Times in the past **may** be interpolated historical data. Runtimes have different practical limits with respect to the accuracy possible at varying past (historical or backwards prediction) and future timepoints. The runtime **must** treat as valid any future time requested by an application, except when disqualified by size limitations of the underlying types, though predictions **may** become less accurate as they get farther into the future. With respect to backward prediction, the application **can** pass a prediction time equivalent to the timestamp of the most recently received pose plus as much as 50 milliseconds in the past to retrieve accurate historical data. The runtime **must** retain and return at least 50 milliseconds of historical data, interpolating as required, preceding the most recently received pose. Requested times predating this time window, or requested times predating the earliest received pose, **may** result in a best effort data whose accuracy reduced or unspecified.

2.17. Colors

The XrColor3f structure is defined as:

```
// Provided by XR_VERSION_1_1
typedef struct XrColor3f {
    float    r;
    float    g;
    float    b;
} XrColor3f;
```

Member Descriptions

- `r` is the red component of the color.
- `g` is the green component of the color.
- `b` is the blue component of the color.

Unless otherwise specified, colors are encoded as linear (not with sRGB nor other gamma compression) values with individual components being in the range of 0.0 through 1.0.

The XrColor4f structure is defined as:

```
// Provided by XR_VERSION_1_0
typedef struct XrColor4f {
    float    r;
    float    g;
    float    b;
    float    a;
} XrColor4f;
```

Member Descriptions

- `r` is the red component of the color.
- `g` is the green component of the color.
- `b` is the blue component of the color.
- `a` is the alpha component of the color.

Unless otherwise specified, colors are encoded as linear (not with sRGB nor other gamma compression) values with individual components being in the range of 0.0 through 1.0, and without the RGB components being premultiplied by the alpha component.

If color encoding is specified as being premultiplied by the alpha component, the RGB components are set to zero if the alpha component is zero.

2.18. Coordinate System

This API uses a Cartesian right-handed coordinate system.

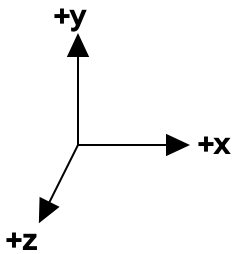


Figure 1. Right Handed Coordinate System

The conventions for mapping coordinate axes of any particular space to meaningful directions depend on and are documented with the description of the space.

The API uses 2D, 3D, and 4D floating-point vectors to describe points and directions in a space.

A two-dimensional vector is defined by the XrVector2f structure:

```
// Provided by XR_VERSION_1_0
typedef struct XrVector2f {
    float    x;
    float    y;
} XrVector2f;
```

Member Descriptions

- x is the x coordinate of the vector.
- y is the y coordinate of the vector.

If used to represent physical distances (rather than e.g. normalized direction) and not otherwise specified, values **must** be in meters.

A three-dimensional vector is defined by the XrVector3f structure:

```
// Provided by XR_VERSION_1_0
typedef struct XrVector3f {
    float    x;
    float    y;
    float    z;
} XrVector3f;
```

Member Descriptions

- `x` is the x coordinate of the vector.
- `y` is the y coordinate of the vector.
- `z` is the z coordinate of the vector.

If used to represent physical distances (rather than e.g. velocity or angular velocity) and not otherwise specified, values **must** be in meters.

A four-dimensional or homogeneous vector is defined by the XrVector4f structure:

```
// Provided by XR_VERSION_1_0
typedef struct XrVector4f {
    float    x;
    float    y;
    float    z;
    float    w;
} XrVector4f;
```

Member Descriptions

- `x` is the x coordinate of the vector.
- `y` is the y coordinate of the vector.
- `z` is the z coordinate of the vector.
- `w` is the w coordinate of the vector.

If used to represent physical distances, `x`, `y`, and `z` values **must** be in meters.

Rotation is represented by a unit quaternion defined by the XrQuaternionf structure:

```
// Provided by XR_VERSION_1_0
typedef struct XrQuaternionf {
    float    x;
    float    y;
    float    z;
    float    w;
} XrQuaternionf;
```

Member Descriptions

- `x` is the x coordinate of the quaternion.
- `y` is the y coordinate of the quaternion.
- `z` is the z coordinate of the quaternion.
- `w` is the w coordinate of the quaternion.

A pose is defined by the XrPosef structure:

```
// Provided by XR_VERSION_1_0
typedef struct XrPosef {
    XrQuaternionf    orientation;
    XrVector3f       position;
} XrPosef;
```

Member Descriptions

- `orientation` is an XrQuaternionf representing the orientation within a space.
- `position` is an XrVector3f representing position within a space.

A construct representing a position and orientation within a space, with position expressed in meters, and orientation represented as a unit quaternion. When using XrPosef the rotation described by `orientation` is always applied before the translation described by `position`.

A runtime **must** return `XR_ERROR_POSE_INVALID` if the `orientation` norm deviates by more than 1% from unit length.

2.19. Common Data Types

Some OpenXR data types are used in multiple structures. Those include the `XrVector*f` family of types, the spatial types specified above, and the following categories of structures:

- offset
- extents
- rectangle
- field of view

Offsets are used to describe the direction and distance of an offset in two dimensions.

A floating-point offset is defined by the structure:

```
// Provided by XR_VERSION_1_0
typedef struct XrOffset2Df {
    float    x;
    float    y;
} XrOffset2Df;
```

Member Descriptions

- `x` is the floating-point offset in the x direction.
- `y` is the floating-point offset in the y direction.

This structure is used for component values that may be real numbers, represented with single-precision floating point. For representing offsets in discrete values, such as texels, the integer variant XrOffset2Di is used instead.

If used to represent physical distances, values **must** be in meters.

An integer offset is defined by the structure:

```
// Provided by XR_VERSION_1_0
typedef struct XrOffset2Di {
    int32_t    x;
    int32_t    y;
} XrOffset2Di;
```

Member Descriptions

- `x` is the integer offset in the x direction.
- `y` is the integer offset in the y direction.

This variant is for representing discrete values such as texels. For representing physical distances, the floating-point variant XrOffset2Df is used instead.

Extents are used to describe the size of a rectangular region in two or three dimensions.

A two-dimensional floating-point extent is defined by the structure:

```
// Provided by XR_VERSION_1_0
typedef struct XrExtent2Df {
    float    width;
    float    height;
} XrExtent2Df;
```

Member Descriptions

- `width` is the floating-point width of the extent.
- `height` is the floating-point height of the extent.

This structure is used for component values that may be real numbers, represented with single-precision floating point. For representing extents in discrete values, such as texels, the integer variant XrExtent2Di is used instead.

If used to represent physical distances, values **must** be in meters.

The `width` and `height` value **must** be non-negative.

The XrExtent3Df structure is defined as:

```
// Provided by XR_VERSION_1_1
typedef struct XrExtent3Df {
    float    width;
    float    height;
    float    depth;
} XrExtent3Df;
```

Member Descriptions

- `width` is the floating-point width of the extent (x).
- `height` is the floating-point height of the extent (y).
- `depth` is the floating-point depth of the extent (z).

This structure is used for component values that may be real numbers, represented with single-precision floating point.

If used to represent physical distances, values **must** be in meters. The width, height, and depth values **must** be non-negative.

A two-dimensional integer extent is defined by the structure:

```
// Provided by XR_VERSION_1_0
typedef struct XrExtent2Di {
    int32_t    width;
    int32_t    height;
} XrExtent2Di;
```

Member Descriptions

- `width` is the integer width of the extent.
- `height` is the integer height of the extent.

This variant is for representing discrete values such as texels. For representing physical distances, the floating-point variant XrExtent2Df is used instead.

The `width` and `height` value **must** be non-negative.

Rectangles are used to describe a specific rectangular region in two dimensions. Rectangles **must** include both an offset and an extent defined in the same units. For instance, if a rectangle is in meters, both offset and extent **must** be in meters.

A rectangle with floating-point values is defined by the structure:

```
// Provided by XR_VERSION_1_0
typedef struct XrRect2Df {
    XrOffset2Df    offset;
    XrExtent2Df    extent;
} XrRect2Df;
```

Member Descriptions

- `offset` is the XrOffset2Df specifying the rectangle offset.
- `extent` is the XrExtent2Df specifying the rectangle extent.

This structure is used for component values that may be real numbers, represented with single-precision floating point.

The `offset` is the position of the rectangle corner with minimum value coordinates. The other three corners are computed by adding the XrExtent2Df::`width` to the `x` offset, XrExtent2Df::`height` to the `y` offset, or both.

A rectangle with integer values is defined by the structure:

```
// Provided by XR_VERSION_1_0
typedef struct XrRect2Di {
    XrOffset2Di    offset;
    XrExtent2Di    extent;
} XrRect2Di;
```

Member Descriptions

- `offset` is the XrOffset2Di specifying the integer rectangle offset.
- `extent` is the XrExtent2Di specifying the integer rectangle extent.

This variant is for representing discrete values such as texels. For representing physical distances, the floating-point variant XrRect2Df is used instead.

The `offset` is the position of the rectangle corner with minimum value coordinates. The other three corners are computed by adding the XrExtent2Di::`width` to the `x` offset, XrExtent2Di::`height` to the `y` offset, or both.

An XrSpheref structure describes the center and radius of a sphere bounds.

```
// Provided by XR_VERSION_1_1
typedef struct XrSpheref {
    XrPosef    center;
    float      radius;
} XrSpheref;
```

Member Descriptions

- `center` is an XrPosef representing the pose of the center of the sphere within the reference frame of the corresponding XrSpace.
- `radius` is the finite non-negative radius of the sphere.

The runtime **must** return `XR_ERROR_VALIDATION_FAILURE` if `radius` is not a finite positive value.

An XrBoxf structure describes the pose and extents of an oriented box.

```
// Provided by XR_VERSION_1_1
typedef struct XrBoxf {
    XrPosef        center;
    XrExtent3Df    extents;
} XrBoxf;
```

Member Descriptions

- `center` is an XrPosef defining the center position and orientation of the oriented bounding box bound within the reference frame of the corresponding XrSpace.
- `extents` is an XrExtent3Df defining the edge-to-edge length of the box along each dimension with `center` as the center.

The runtime **must** return `XR_ERROR_VALIDATION_FAILURE` if width, height or depth values are negative.

An XrFrustumf structure describes the pose, field of view, and far distance of a frustum.

```
// Provided by XR_VERSION_1_1
typedef struct XrFrustumf {
    XrPosef    pose;
    XrFovf     fov;
    float      nearZ;
    float      farZ;
} XrFrustumf;
```

Member Descriptions

- `pose` is an XrPosef defining the position and orientation of the tip of the frustum within the reference frame of the corresponding XrSpace.
- `fov` is an XrFovf for the four sides of the frustum where `angleLeft` and `angleRight` are along the X axis and `angleUp` and `angleDown` are along the Y axis of the frustum space.
- `nearZ` is the positive distance of the near plane of the frustum bound along the -Z direction of the frustum space.
- `farZ` is the positive distance of the far plane of the frustum bound along the -Z direction of the frustum space.

The runtime **must** return `XR_ERROR_VALIDATION_FAILURE` if `farZ` is less than or equal to zero.

The runtime **must** return `XR_ERROR_VALIDATION_FAILURE` if `nearZ` is less than zero.

See XrFovf for validity requirements on fov.

The XrUuid structure is a 128-bit Universally Unique Identifier and is defined as:

```
// Provided by XR_VERSION_1_1
typedef struct XrUuid {
    uint8_t    data[XR_UUID_SIZE];
} XrUuid;
```

Member Descriptions

- data is a 128-bit Universally Unique Identifier.

The structure is composed of 16 octets, with the size and order of the fields defined in RFC 4122 section 4.1.2.

2.20. Angles

Where a value is provided as a function parameter or as a structure member and will be interpreted as an angle, the value is defined to be in radians.

Field of view (FoV) is defined by the structure:

```
// Provided by XR_VERSION_1_0
typedef struct XrFovf {
    float    angleLeft;
    float    angleRight;
    float    angleUp;
    float    angleDown;
} XrFovf;
```

Member Descriptions

- angleLeft is the angle of the left side of the field of view. For a symmetric field of view this value is negative.
- angleRight is the angle of the right side of the field of view.
- angleUp is the angle of the top part of the field of view.
- angleDown is the angle of the bottom part of the field of view. For a symmetric field of view this value is negative.

Angles to the right of the center and upwards from the center are positive, and angles to the left of the center and down from the center are negative. The total horizontal field of view is `angleRight` minus `angleLeft`, and the total vertical field of view is `angleUp` minus `angleDown`. For a symmetric FoV, `angleRight` and `angleUp` will have positive values, `angleLeft` will be -`angleRight`, and `angleDown` will be -`angleUp`.

The angles **must** be specified in radians, and **must** be between -π/2 and π/2 exclusively.

When `angleLeft` > `angleRight`, the content of the view **must** be flipped horizontally. When `angleDown` > `angleUp`, the content of the view **must** be flipped vertically.

2.21. Boolean Values

```
// Provided by XR_VERSION_1_0
typedef uint32_t XrBool32;
```

Boolean values used by OpenXR are of type `XrBool32` and are 32-bits wide as suggested by the name. The only valid values are the following:

Enumerant Descriptions

- `XR_TRUE` represents a true value.
- `XR_FALSE` represents a false value.

```
#define XR_TRUE                         1
```

```
#define XR_FALSE                        0
```

2.22. Events

Events are messages sent from the runtime to the application.

2.22.1. Event Polling

Events are placed in a queue within the runtime. The application **must** read from the queue with regularity. Events are read from the queue one at a time via xrPollEvent. Every type of event is identified by an individual structure type, with each such structure beginning with an XrEventDataBaseHeader.

Example 1. Proper Method for Receiving OpenXR Event Data

```
XrInstance instance; // previously initialized

// Initialize an event buffer to hold the output.
XrEventDataBuffer event {
    .type = XR_TYPE_EVENT_DATA_BUFFER,
};
XrResult result = xrPollEvent(instance, &event);
if (result == XR_SUCCESS) {
    switch (event.type) {
        case XR_TYPE_EVENT_DATA_SESSION_STATE_CHANGED: {
            const XrEventDataSessionStateChanged& session_state_changed_event =
              *reinterpret_cast<XrEventDataSessionStateChanged*>(&event);
            // ...
            break;
        }
        case XR_TYPE_EVENT_DATA_INSTANCE_LOSS_PENDING: {
            const XrEventDataInstanceLossPending& instance_loss_pending_event =
              *reinterpret_cast<XrEventDataInstanceLossPending*>(&event);
            // ...
            break;
        }
    }
}
```

xrPollEvent

The xrPollEvent function is defined as:

```
// Provided by XR_VERSION_1_0
XrResult xrPollEvent(
    XrInstance                                  instance,
    XrEventDataBuffer*                          eventData);
```

Parameter Descriptions

- `instance` is a valid XrInstance.
- `eventData` is a pointer to a valid XrEventDataBuffer.

xrPollEvent polls for the next event and returns an event if one is available. xrPollEvent returns immediately regardless of whether an event was available. The event (if present) is unilaterally

removed from the queue if a valid XrInstance is provided. On return, the `eventData` parameter is filled with the event's data and the type field is changed to the event's type. Runtimes **may** create valid `next` chains depending on enabled extensions, but they **must** guarantee that any such chains point only to objects which fit completely within the original XrEventDataBuffer pointed to by `eventData`.

The runtime **must** discard queued events which contain destroyed or otherwise invalid handles. The runtime **must** not return events containing handles that have been destroyed or are otherwise invalid at the time of the call to xrPollEvent.

Valid Usage (Implicit)

- `instance` **must** be a valid XrInstance handle
- `eventData` **must** be a pointer to an XrEventDataBuffer structure

Return Codes

Success

- `XR_SUCCESS`
- `XR_EVENT_UNAVAILABLE`

Failure

- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_VALIDATION_FAILURE`

Table 2. Event Descriptions

Event	Description
XrEventDataEventsLost	event queue has overflowed and some events were lost
XrEventDataInstanceLossPending	application is about to lose the instance
XrEventDataInteractionProfileChanged	current interaction profile for one or more top level user paths has changed
XrEventDataReferenceSpaceChangePending	runtime will begin operating with updated definitions or bounds for a reference space
XrEventDataSessionStateChanged	the application's session has changed lifecycle state

The XrEventDataBaseHeader structure is defined as:

```
// Provided by XR_VERSION_1_0
typedef struct XrEventDataBaseHeader {
    XrStructureType    type;
    const void*        next;
} XrEventDataBaseHeader;
```

Parameter Descriptions

- `type` is the XrStructureType of this structure. This base structure itself has no associated XrStructureType value.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR.

The XrEventDataBaseHeader is a generic structure used to identify the common event data elements.

Upon receipt, the XrEventDataBaseHeader pointer **should** be type-cast to a pointer of the appropriate event data type based on the `type` parameter.

Valid Usage (Implicit)

- `type` **must** be one of the following XrStructureType values: `XR_TYPE_EVENT_DATA_BOUNDARY_VISIBILITY_CHANGED_META`, `XR_TYPE_EVENT_DATA_COLOCATION_ADVERTISEMENT_COMPLETE_META`, `XR_TYPE_EVENT_DATA_COLOCATION_DISCOVERY_COMPLETE_META`, `XR_TYPE_EVENT_DATA_COLOCATION_DISCOVERY_RESULT_META`, `XR_TYPE_EVENT_DATA_DISPLAY_REFRESH_RATE_CHANGED_FB`, `XR_TYPE_EVENT_DATA_EVENTS_LOST`, `XR_TYPE_EVENT_DATA_EYE_CALIBRATION_CHANGED_ML`, `XR_TYPE_EVENT_DATA_GEOSPATIAL_TRACKER_STATE_CHANGED_ANDROID`, `XR_TYPE_EVENT_DATA_HEADSET_FIT_CHANGED_ML`, `XR_TYPE_EVENT_DATA_IMAGE_TRACKING_LOST_ANDROID`, `XR_TYPE_EVENT_DATA_INSTANCE_LOSS_PENDING`, `XR_TYPE_EVENT_DATA_INTERACTION_PROFILE_CHANGED`, `XR_TYPE_EVENT_DATA_INTERACTION_RENDER_MODELS_CHANGED_EXT`, `XR_TYPE_EVENT_DATA_LOCALIZATION_CHANGED_ML`, `XR_TYPE_EVENT_DATA_MAIN_SESSION_VISIBILITY_CHANGED_EXTX`, `XR_TYPE_EVENT_DATA_MARKER_TRACKING_UPDATE_VARJO`, `XR_TYPE_EVENT_DATA_PASSTHROUGH_LAYER_RESUMED_META`, `XR_TYPE_EVENT_DATA_PASSTHROUGH_STATE_CHANGED_FB`, `XR_TYPE_EVENT_DATA_PERF_SETTINGS_EXT`, `XR_TYPE_EVENT_DATA_RECOMMENDED_RESOLUTION_CHANGED_ANDROID`, `XR_TYPE_EVENT_DATA_REFERENCE_SPACE_CHANGE_PENDING`, `XR_TYPE_EVENT_DATA_SCENE_CAPTURE_COMPLETE_FB`, `XR_TYPE_EVENT_DATA_SENSE_DATA_PROVIDER_STATE_CHANGED_BD`, `XR_TYPE_EVENT_DATA_SENSE_DATA_UPDATED_BD`, `XR_TYPE_EVENT_DATA_SESSION_STATE_CHANGED`, `XR_TYPE_EVENT_DATA_SHARE_SPACES_COMPLETE_META`, `XR_TYPE_EVENT_DATA_SPACE_DISCOVERY_COMPLETE_META`, `XR_TYPE_EVENT_DATA_SPACE_DISCOVERY_RESULTS_AVAILABLE_META`, `XR_TYPE_EVENT_DATA_SPACE_ERASE_COMPLETE_FB`, `XR_TYPE_EVENT_DATA_SPACE_LIST_SAVE_COMPLETE_FB`, `XR_TYPE_EVENT_DATA_SPACE_QUERY_COMPLETE_FB`, `XR_TYPE_EVENT_DATA_SPACE_QUERY_RESULTS_AVAILABLE_FB`, `XR_TYPE_EVENT_DATA_SPACE_SAVE_COMPLETE_FB`, `XR_TYPE_EVENT_DATA_SPACE_SET_STATUS_COMPLETE_FB`, `XR_TYPE_EVENT_DATA_SPACE_SHARE_COMPLETE_FB`, `XR_TYPE_EVENT_DATA_SPACES_ERASE_RESULT_META`, `XR_TYPE_EVENT_DATA_SPACES_SAVE_RESULT_META`, `XR_TYPE_EVENT_DATA_SPATIAL_ANCHOR_CREATE_COMPLETE_FB`, `XR_TYPE_EVENT_DATA_SPATIAL_DISCOVERY_RECOMMENDED_EXT`, `XR_TYPE_EVENT_DATA_START_COLOCATION_ADVERTISEMENT_COMPLETE_META`, `XR_TYPE_EVENT_DATA_START_COLOCATION_DISCOVERY_COMPLETE_META`, `XR_TYPE_EVENT_DATA_STOP_COLOCATION_ADVERTISEMENT_COMPLETE_META`, `XR_TYPE_EVENT_DATA_STOP_COLOCATION_DISCOVERY_COMPLETE_META`, `XR_TYPE_EVENT_DATA_USER_PRESENCE_CHANGED_EXT`, `XR_TYPE_EVENT_DATA_VIEW_CONFIGURATION_VIEWS_CHANGED_EXT`, `XR_TYPE_EVENT_DATA_VIRTUAL_KEYBOARD_BACKSPACE_META`, `XR_TYPE_EVENT_DATA_VIRTUAL_KEYBOARD_COMMIT_TEXT_META`,

XR_TYPE_EVENT_DATA_VIRTUAL_KEYBOARD_ENTER_META,
XR_TYPE_EVENT_DATA_VIRTUAL_KEYBOARD_HIDDEN_META,
XR_TYPE_EVENT_DATA_VIRTUAL_KEYBOARD_SHOWN_META,
XR_TYPE_EVENT_DATA_VISIBILITY_MASK_CHANGED_KHR,
XR_TYPE_EVENT_DATA_VIVE_TRACKER_CONNECTED_HTCX

- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

```
// Provided by XR_VERSION_1_0
typedef struct XrEventDataBuffer {
    XrStructureType    type;
    const void*        next;
    uint8_t            varying[4000];
} XrEventDataBuffer;
```

Parameter Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR.
- `varying` is a fixed sized output buffer big enough to hold returned data elements for all specified event data types.

The XrEventDataBuffer is a structure passed to xrPollEvent large enough to contain any returned event data element. The maximum size is specified by XR_MAX_EVENT_DATA_SIZE.

An application **can** set (or reset) only the `type` member and clear the `next` member of an XrEventDataBuffer before passing it as an input to xrPollEvent. The runtime **must** ignore the contents of the `varying` field and overwrite it without reading it.

A pointer to an XrEventDataBuffer **may** be type-cast to an XrEventDataBaseHeader pointer, or a pointer to any other appropriate event data based on the `type` parameter.

Valid Usage (Implicit)

- `type` **must** be `XR_TYPE_EVENT_DATA_BUFFER`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

```
// Provided by XR_VERSION_1_0
#define XR_MAX_EVENT_DATA_SIZE sizeof(XrEventDataBuffer)
```

XR_MAX_EVENT_DATA_SIZE is the size of XrEventDataBuffer, including the size of the XrEventDataBuffer::type and XrEventDataBuffer::next members.

XrEventDataEventsLost

The XrEventDataEventsLost structure is defined as:

```
// Provided by XR_VERSION_1_0
typedef struct XrEventDataEventsLost {
    XrStructureType    type;
    const void*        next;
    uint32_t           lostEventCount;
} XrEventDataEventsLost;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR.
- lostEventCount is the number of events which have overflowed since the last call to xrPollEvent.

Receiving the XrEventDataEventsLost event structure indicates that the event queue overflowed and some events were removed at the position within the queue at which this event was found.

Valid Usage (Implicit)

- type **must** be XR_TYPE_EVENT_DATA_EVENTS_LOST
- next **must** be NULL or a valid pointer to the next structure in a structure chain

Other event structures are defined in later chapters in the context where their definition is most relevant.

2.23. System resource lifetime

The creator of an underlying system resource is responsible for ensuring the resource's lifetime matches the lifetime of the associated OpenXR handle.

Resources passed as inputs from the application to the runtime when creating an OpenXR handle **should** not be freed while that handle is valid. A runtime **must** not free resources passed as inputs or decrease their reference counts (if applicable) from the initial value. For example, the graphics device

handle (or pointer) passed in to xrCreateSession in `XrGraphicsBinding*` structure **should** be kept alive when the corresponding XrSession handle is valid, and **should** be freed by the application after the XrSession handle is destroyed.

Resources created by the runtime should not be freed by the application, and the application **should** maintain the same reference count (if applicable) at the destruction of the OpenXR handle as it had at its creation. For example, the `ID3D*Texture2D` objects in the `XrSwapchainImageD3D*` are created by the runtime and associated with the lifetime of the XrSwapchain handle. The application **should** not keep additional reference counts on any `ID3D*Texture2D` objects past the lifetime of the XrSwapchain handle, or make extra reference count decrease after destroying the XrSwapchain handle.

Chapter 3. API Initialization

Before using an OpenXR runtime, an application **must** initialize it by creating an XrInstance object. The following functions are useful for gathering information about the API layers and extensions installed on the system and creating the instance.

Instance Creation Functions

- xrEnumerateApiLayerProperties
- xrEnumerateInstanceExtensionProperties
- xrCreateInstance

xrEnumerateApiLayerProperties and xrEnumerateInstanceExtensionProperties **can** be called before calling xrCreateInstance.

3.1. Exported Functions

A dynamically linked library (`.dll` or `.so`) that implements the API loader **must** export all core OpenXR API functions. The application **can** gain access to extension functions by obtaining pointers to these functions through the use of xrGetInstanceProcAddr.

3.2. Function Pointers

Function pointers for all OpenXR functions **can** be obtained with the function xrGetInstanceProcAddr.

```
// Provided by XR_VERSION_1_0
XrResult xrGetInstanceProcAddr(
    XrInstance                                  instance,
    const char*                                 name,
    PFN_xrVoidFunction*                         function);
```

Parameter Descriptions

- `instance` is the instance that the function pointer will be compatible with, or `NULL` for functions not dependent on any instance.
- `name` is the name of the function to obtain.
- `function` is the address of the function pointer to get.

xrGetInstanceProcAddr itself is obtained in a platform- and loader- specific manner. Typically, the

loader library will export this function as a function symbol, so applications **can** link against the loader library, or load it dynamically and look up the symbol using platform-specific APIs. Loaders **must** export function symbols for all core OpenXR functions. Because of this, applications that use only the core OpenXR functions have no need to use xrGetInstanceProcAddr.

Because an application **can** call xrGetInstanceProcAddr before creating an instance, xrGetInstanceProcAddr **must** return a valid function pointer when the `instance` parameter is XR_NULL_HANDLE and the `name` parameter is one of the following strings:

No Instance Required

- xrEnumerateInstanceExtensionProperties
- xrEnumerateApiLayerProperties
- xrCreateInstance

xrGetInstanceProcAddr **must** return `XR_ERROR_HANDLE_INVALID` if `name` is not one of the above strings and `instance` is XR_NULL_HANDLE. xrGetInstanceProcAddr **may** return `XR_ERROR_HANDLE_INVALID` if `name` is not one of the above strings and `instance` is invalid but not XR_NULL_HANDLE.

xrGetInstanceProcAddr **must** return `XR_ERROR_FUNCTION_UNSUPPORTED` if `instance` is a valid instance and the string specified in `name` is not the name of an OpenXR core or enabled extension function.

If `name` is the name of an extension function, then the result returned by xrGetInstanceProcAddr will depend upon how the `instance` was created. If `instance` was created with the related extension's name appearing in the XrInstanceCreateInfo::`enabledExtensionNames` array, then xrGetInstanceProcAddr returns a valid function pointer. If the related extension's name did not appear in the XrInstanceCreateInfo::`enabledExtensionNames` array during the creation of `instance`, then xrGetInstanceProcAddr returns `XR_ERROR_FUNCTION_UNSUPPORTED`. Because of this, function pointers returned by xrGetInstanceProcAddr using one XrInstance may not be valid when used with objects related to a different XrInstance.

The returned function pointer is of type PFN_xrVoidFunction, and **must** be cast by the application to the type of the function being queried.

The table below defines the various use cases for xrGetInstanceProcAddr and return value ("fp" is "function pointer") for each case.

Table 3. xrGetInstanceProcAddr behavior

`instance` **parameter**	`name` **parameter**	**return value**
*	`NULL`	undefined
invalid instance	*	undefined
`NULL`	xrEnumerateInstanceExtensionProperties	fp

`instance` parameter	`name` parameter	return value
`NULL`	xrEnumerateApiLayerProperties	fp
`NULL`	xrCreateInstance	fp
`NULL`	* (any `name` not covered above)	`NULL`
instance	core OpenXR function	fp[1]
instance	enabled extension function for `instance`	fp[1]
instance	* (any `name` not covered above)	`NULL`

1

The returned function pointer **must** only be called with a handle (the first parameter) that is `instance` or a child of `instance`.

Valid Usage (Implicit)

- If `instance` is not XR_NULL_HANDLE, `instance` **must** be a valid XrInstance handle
- `name` **must** be a null-terminated UTF-8 string
- `function` **must** be a pointer to a PFN_xrVoidFunction value

Return Codes

Success

- `XR_SUCCESS`

Failure

- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_OUT_OF_MEMORY`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_VALIDATION_FAILURE`

```
// Provided by XR_VERSION_1_0
typedef void (XRAPI_PTR *PFN_xrVoidFunction)(void);
```

Parameter Descriptions

- no parameters.

PFN_xrVoidFunction is a generic function pointer type returned by queries, specifically those to xrGetInstanceProcAddr.

```
// Provided by XR_LOADER_VERSION_1_0
typedef XrResult (XRAPI_PTR *PFN_xrGetInstanceProcAddr)(XrInstance instance, const char*
name, PFN_xrVoidFunction* function);
```

PFN_xrGetInstanceProcAddr is a function pointer type for xrGetInstanceProcAddr.

```
// Provided by XR_LOADER_VERSION_1_0

typedef struct XrApiLayerCreateInfo XrApiLayerCreateInfo;
typedef XrResult (XRAPI_PTR *PFN_xrCreateApiLayerInstance)(
            const XrInstanceCreateInfo* info,
            const XrApiLayerCreateInfo* apiLayerInfo,
            XrInstance* instance);
```

PFN_xrCreateApiLayerInstance is a function pointer type for xrCreateApiLayerInstance.

Note: This function pointer type is only used by an OpenXR loader library, and never by an application.

3.3. Runtime Interface Negotiation

In order to negotiate the runtime interface version with the loader, the runtime **must** implement the xrNegotiateLoaderRuntimeInterface function.



Note

The API described in this section is solely intended for use between an OpenXR loader and a runtime (and/or an API layer, where noted). Applications use the appropriate loader library for their platform to load the active runtime and configured API layers, rather than making these calls directly. This section is included in the specification to ensure consistency between runtimes in their interactions with the loader.

Be advised that as this is not application-facing API, some of the typical OpenXR API conventions are not followed in this section.

The xrNegotiateLoaderRuntimeInterface function is defined as:

```
// Provided by XR_LOADER_VERSION_1_0
XrResult xrNegotiateLoaderRuntimeInterface(
    const XrNegotiateLoaderInfo*                loaderInfo,
    XrNegotiateRuntimeRequest*                  runtimeRequest);
```

Parameter Descriptions

- `loaderInfo` **must** be a pointer to a valid XrNegotiateLoaderInfo structure.
- `runtimeRequest` **must** be a valid pointer to an XrNegotiateRuntimeRequest structure, with minimal initialization, as subsequently described, to be fully populated by the called runtime.

xrNegotiateLoaderRuntimeInterface **should** be directly exported by a runtime so that using e.g. `GetProcAddress` on Windows or `dlsym` on POSIX platforms returns a valid function pointer to it.

The runtime **must** return `XR_ERROR_INITIALIZATION_FAILED` if any of the following conditions on `loaderInfo` are true:

- XrNegotiateLoaderInfo::`structType` is not `XR_LOADER_INTERFACE_STRUCT_LOADER_INFO`
- XrNegotiateLoaderInfo::`structVersion` is not XR_LOADER_INFO_STRUCT_VERSION
- XrNegotiateLoaderInfo::`structSize` is not `sizeof(XrNegotiateLoaderInfo)`

The runtime **must** also return `XR_ERROR_INITIALIZATION_FAILED` if any of the following conditions on `runtimeRequest` are true:

- XrNegotiateRuntimeRequest::`structType` is not `XR_LOADER_INTERFACE_STRUCT_RUNTIME_REQUEST`
- XrNegotiateRuntimeRequest::`structVersion` is not XR_RUNTIME_INFO_STRUCT_VERSION
- XrNegotiateRuntimeRequest::`structSize` is not `sizeof(XrNegotiateRuntimeRequest)`

The runtime **must** determine if it supports the loader's request. The runtime does not support the loader's request if either of the following is true:

- The runtime does not support any of the interface versions supported by the loader, as specified by the range XrNegotiateLoaderInfo::minInterfaceVersion through XrNegotiateLoaderInfo::maxInterfaceVersion inclusive.
- The runtime does not support any of the API versions supported by the loader, ignoring "patch" version components, as specified by the range XrNegotiateLoaderInfo::minApiVersion through XrNegotiateLoaderInfo::maxApiVersion inclusive.

The runtime **must** return XR_ERROR_INITIALIZATION_FAILED if it does not support the loader's request.

If the function succeeds, the runtime **must** set the XrNegotiateRuntimeRequest::runtimeInterfaceVersion with the runtime interface version it desires to support. The XrNegotiateRuntimeRequest::runtimeInterfaceVersion set **must** be in the range XrNegotiateLoaderInfo::minInterfaceVersion through XrNegotiateLoaderInfo::maxInterfaceVersion inclusive.

If the function succeeds, the runtime **must** set the XrNegotiateRuntimeRequest::runtimeApiVersion with the API version of OpenXR it will execute under. The XrNegotiateRuntimeRequest::runtimeApiVersion set **must** be in the range XrNegotiateLoaderInfo::minApiVersion through XrNegotiateLoaderInfo::maxApiVersion inclusive.

If the function succeeds, the runtime **must** set the XrNegotiateRuntimeRequest::getInstanceProcAddr with a valid function pointer for the loader to use to query function pointers to the remaining OpenXR functions supported by the runtime.

If the function succeeds, the runtime **must** return XR_SUCCESS.

Valid Usage (Implicit)

- loaderInfo **must** be a pointer to a valid XrNegotiateLoaderInfo structure
- runtimeRequest **must** be a pointer to an XrNegotiateRuntimeRequest structure

Return Codes

Success

- XR_SUCCESS

Failure

- XR_ERROR_INITIALIZATION_FAILED

The XrNegotiateLoaderInfo structure is used to pass information about the loader to a runtime or an API layer.

The XrNegotiateLoaderInfo structure is defined as:

```
// Provided by XR_LOADER_VERSION_1_0
typedef struct XrNegotiateLoaderInfo {
    XrLoaderInterfaceStructs    structType;
    uint32_t                    structVersion;
    size_t                      structSize;
    uint32_t                    minInterfaceVersion;
    uint32_t                    maxInterfaceVersion;
    XrVersion                   minApiVersion;
    XrVersion                   maxApiVersion;
} XrNegotiateLoaderInfo;
```

Member Descriptions

- `structType` **must** be `XR_LOADER_INTERFACE_STRUCT_LOADER_INFO`.
- `structVersion` **must** be a valid version of the structure. The value XR_LOADER_INFO_STRUCT_VERSION describes the current latest version of this structure.
- `structSize` **must** be the size in bytes of the current version of the structure (i.e. `sizeof(XrNegotiateLoaderInfo)`).
- `minInterfaceVersion` is the minimum runtime or API layer interface version supported by the loader.
- `maxInterfaceVersion` is the maximum valid version of the runtime or API layer interface version supported by the loader, currently defined using XR_CURRENT_LOADER_RUNTIME_VERSION or XR_CURRENT_LOADER_API_LAYER_VERSION.
- `minApiVersion` is the minimum supported version of the OpenXR API by the loader as formatted by XR_MAKE_VERSION. Patch is ignored.
- `maxApiVersion` is the maximum supported version of the OpenXR API by the loader as formatted by XR_MAKE_VERSION. Patch is ignored.

This structure is an input from the loader to the runtime in an xrNegotiateLoaderRuntimeInterface call, as well as from the loader to an API layer in an xrNegotiateLoaderApiLayerInterface call.

Valid Usage (Implicit)

- `structType` **must** be a valid XrLoaderInterfaceStructs value

The XrLoaderInterfaceStructs enumeration is defined as:

```
// Provided by XR_LOADER_VERSION_1_0
typedef enum XrLoaderInterfaceStructs {
    XR_LOADER_INTERFACE_STRUCT_UNINTIALIZED = 0,
    XR_LOADER_INTERFACE_STRUCT_LOADER_INFO = 1,
    XR_LOADER_INTERFACE_STRUCT_API_LAYER_REQUEST = 2,
    XR_LOADER_INTERFACE_STRUCT_RUNTIME_REQUEST = 3,
    XR_LOADER_INTERFACE_STRUCT_API_LAYER_CREATE_INFO = 4,
    XR_LOADER_INTERFACE_STRUCT_API_LAYER_NEXT_INFO = 5,
    XR_LOADER_INTERFACE_STRUCTS_MAX_ENUM = 0x7FFFFFFF
} XrLoaderInterfaceStructs;
```

This enumeration serves a similar purpose in the runtime and API layer interface negotiation (loader) API as XrStructureType serves in the application-facing API.

```
// Provided by XR_LOADER_VERSION_1_0
#define XR_LOADER_INFO_STRUCT_VERSION 1
```

XR_LOADER_INFO_STRUCT_VERSION is the current version of the XrNegotiateLoaderInfo structure. It is used to populate the XrNegotiateLoaderInfo::`structVersion` field.

```
// Provided by XR_LOADER_VERSION_1_0
#define XR_CURRENT_LOADER_RUNTIME_VERSION 1
```

XR_CURRENT_LOADER_RUNTIME_VERSION is the current version of the overall OpenXR Loader Runtime interface. It is used to populate maximum and minimum interface version fields in XrNegotiateLoaderInfo when loading a runtime.

```
// Provided by XR_LOADER_VERSION_1_0
#define XR_CURRENT_LOADER_API_LAYER_VERSION 1
```

XR_CURRENT_LOADER_API_LAYER_VERSION is the current version of the overall OpenXR Loader API Layer interface. It is used to populate maximum and minimum interface version fields in XrNegotiateLoaderInfo when loading an API layer.

The XrNegotiateRuntimeRequest structure is used to pass information about the runtime back to the loader.

The XrNegotiateRuntimeRequest structure is defined as:

```
// Provided by XR_LOADER_VERSION_1_0
typedef struct XrNegotiateRuntimeRequest {
    XrLoaderInterfaceStructs     structType;
    uint32_t                     structVersion;
    size_t                       structSize;
    uint32_t                     runtimeInterfaceVersion;
    XrVersion                    runtimeApiVersion;
    PFN_xrGetInstanceProcAddr    getInstanceProcAddr;
} XrNegotiateRuntimeRequest;
```

Member Descriptions

- `structType` **must** be `XR_LOADER_INTERFACE_STRUCT_RUNTIME_REQUEST`.
- `structVersion` **must** be a valid version of the structure. The value XR_RUNTIME_INFO_STRUCT_VERSION is used to describe the current version of this structure.
- `structSize` **must** be the size in bytes of the current version of the structure (i.e. `sizeof(XrNegotiateRuntimeRequest)`)
- `runtimeInterfaceVersion` is the version of the runtime interface version being requested by the runtime. Must: not be outside of the bounds of the range XrNegotiateLoaderInfo ::`minInterfaceVersion` through XrNegotiateLoaderInfo::`maxInterfaceVersion` (inclusive).
- `runtimeApiVersion` is the version of the OpenXR API supported by this runtime as formatted by XR_MAKE_VERSION. Patch is ignored.
- `getInstanceProcAddr` is a pointer to the runtime's xrGetInstanceProcAddr implementation that will be used by the loader to populate a dispatch table of OpenXR functions supported by the runtime.

This is an output structure from runtime negotiation. The loader **must** populate `structType`, `structVersion`, and `structSize` to ensure correct interpretation by the runtime, while the runtime populates the rest of the fields in a successful call to xrNegotiateLoaderRuntimeInterface.

Valid Usage (Implicit)

- `structType` **must** be a valid XrLoaderInterfaceStructs value
- `getInstanceProcAddr` **must** be a valid PFN_xrGetInstanceProcAddr value

```
// Provided by XR_LOADER_VERSION_1_0
#define XR_RUNTIME_INFO_STRUCT_VERSION 1
```

XR_RUNTIME_INFO_STRUCT_VERSION is the current version of the XrNegotiateRuntimeRequest structure. It is used to populate the XrNegotiateRuntimeRequest::structVersion field.

3.4. API Layer Interface Negotiation

In order to negotiate the API layer interface version with the loader, an OpenXR API layer **must** implement the xrNegotiateLoaderApiLayerInterface function.



Note

The API described in this section is solely intended for use between an OpenXR loader and an API layer. Applications use the appropriate loader library for their platform to load the active runtime and configured API layers, rather than making these calls directly. This section is included in the specification to ensure consistency between runtimes in their interactions with the loader.

Be advised that as this is not application-facing API, some of the typical OpenXR API conventions are not followed in this section.

The xrNegotiateLoaderApiLayerInterface function is defined as:

```
// Provided by XR_LOADER_VERSION_1_0
XrResult xrNegotiateLoaderApiLayerInterface(
    const XrNegotiateLoaderInfo*                loaderInfo,
    const char*                                 layerName,
    XrNegotiateApiLayerRequest*                 apiLayerRequest);
```

Parameter Descriptions

- loaderInfo **must** be a pointer to a valid XrNegotiateLoaderInfo structure.
- layerName **must** be NULL or a valid C-style NULL-terminated string listing the name of an API layer which the loader is attempting to negotiate with.
- apiLayerRequest **must** be a valid pointer to an XrNegotiateApiLayerRequest structure, with minimal initialization, as subsequently described, to be fully populated by the called API layer.

xrNegotiateLoaderApiLayerInterface **should** be directly exported by an API layer so that using e.g. GetProcAddress on Windows or dlsym on POSIX platforms returns a valid function pointer to it.

The API layer **must** return XR_ERROR_INITIALIZATION_FAILED if any of the following conditions on loaderInfo are true:

- XrNegotiateLoaderInfo::structType is not XR_LOADER_INTERFACE_STRUCT_LOADER_INFO

- XrNegotiateLoaderInfo::structVersion is not XR_LOADER_INFO_STRUCT_VERSION
- XrNegotiateLoaderInfo::structSize is not sizeof(XrNegotiateLoaderInfo)

The API layer **must** also return XR_ERROR_INITIALIZATION_FAILED if any of the following conditions on apiLayerRequest are true:

- XrNegotiateApiLayerRequest::structType is not XR_LOADER_INTERFACE_STRUCT_API_LAYER_REQUEST
- XrNegotiateApiLayerRequest::structVersion is not XR_API_LAYER_INFO_STRUCT_VERSION
- XrNegotiateApiLayerRequest::structSize is not sizeof(XrNegotiateApiLayerRequest)

The API layer **must** determine if it supports the loader's request. The API layer does not support the loader's request if either of the following is true:

- The API layer does not support the interface versions supported by the loader, as specified by the range XrNegotiateLoaderInfo::minInterfaceVersion through XrNegotiateLoaderInfo::maxInterfaceVersion inclusive.
- The API layer does not support the API versions supported by the loader, ignoring "patch" version components, as specified by the range XrNegotiateLoaderInfo::minApiVersion through XrNegotiateLoaderInfo::maxApiVersion inclusive.

The API layer **must** return XR_ERROR_INITIALIZATION_FAILED if it does not support the loader's request.

If the function succeeds, the API layer **must** set the XrNegotiateApiLayerRequest::layerInterfaceVersion with the API layer interface version it desires to support. The XrNegotiateApiLayerRequest::layerInterfaceVersion set **must** be in the range XrNegotiateLoaderInfo::minInterfaceVersion through XrNegotiateLoaderInfo::maxInterfaceVersion inclusive.

If the function succeeds, the API layer **must** set the XrNegotiateApiLayerRequest::layerApiVersion with the API version of OpenXR it will execute under. The XrNegotiateApiLayerRequest::layerApiVersion set **must** be in the range XrNegotiateLoaderInfo::minApiVersion through XrNegotiateLoaderInfo::maxApiVersion inclusive.

If the function succeeds, the API layer **must** set the XrNegotiateApiLayerRequest::getInstanceProcAddr with a valid function pointer for the loader to use to query function pointers to the remaining OpenXR functions supported by the API layer.

If the function succeeds, the API layer **must** set the XrNegotiateApiLayerRequest::createApiLayerInstance with a valid function pointer to an implementation of xrCreateApiLayerInstance for the loader to use to create the instance through the API layer call chain.

If the function succeeds, the API layer **must** return XR_SUCCESS.

The API layer **must** not call into another API layer from its implementation of the xrNegotiateLoaderApiLayerInterface function. The loader **must** handle all API layer negotiations with each API layer individually.

Valid Usage (Implicit)

- `loaderInfo` **must** be a pointer to a valid XrNegotiateLoaderInfo structure
- `layerName` **must** be a null-terminated UTF-8 string
- `apiLayerRequest` **must** be a pointer to an XrNegotiateApiLayerRequest structure

Return Codes

Success

- `XR_SUCCESS`

Failure

- `XR_ERROR_INITIALIZATION_FAILED`

The XrNegotiateApiLayerRequest structure is used to pass information about the API layer back to the loader.

The XrNegotiateApiLayerRequest structure is defined as:

```
// Provided by XR_LOADER_VERSION_1_0
typedef struct XrNegotiateApiLayerRequest {
    XrLoaderInterfaceStructs        structType;
    uint32_t                        structVersion;
    size_t                          structSize;
    uint32_t                        layerInterfaceVersion;
    XrVersion                       layerApiVersion;
    PFN_xrGetInstanceProcAddr       getInstanceProcAddr;
    PFN_xrCreateApiLayerInstance    createApiLayerInstance;
} XrNegotiateApiLayerRequest;
```

Member Descriptions

- `structType` **must** be `XR_LOADER_INTERFACE_STRUCT_API_LAYER_REQUEST`.
- `structVersion` **must** be a valid version of the structure. The value XR_API_LAYER_INFO_STRUCT_VERSION is used to describe the current latest version of this structure.
- `structSize` **must** be the size in bytes of the current version of the structure (i.e. `sizeof(XrNegotiateApiLayerRequest)`).
- `layerInterfaceVersion` is the version of the API layer interface version being requested by the API layer. Should not be outside of the bounds of the range XrNegotiateLoaderInfo::`minInterfaceVersion` through XrNegotiateLoaderInfo::`maxInterfaceVersion` (inclusive).
- `layerApiVersion` is the version of the OpenXR API supported by this API layer as formatted by XR_MAKE_VERSION. Patch is ignored.
- `getInstanceProcAddr` is a pointer to the API layer's xrGetInstanceProcAddr implementation that will be used by the loader to populate a dispatch table of OpenXR functions supported by the API layer.
- `createApiLayerInstance` is a pointer to the API layer's xrCreateApiLayerInstance implementation that will be used by the loader during a call to xrCreateInstance when an API layer is active. This is used because API layers need additional information at xrCreateInstance time.

This is an output structure from API layer negotiation. The loader **must** populate `structType`, `structVersion`, and `structSize` before calling to ensure correct interpretation by the API layer, while the API layer populates the rest of the fields in a successful call to xrNegotiateLoaderApiLayerInterface.

Valid Usage (Implicit)

- `structType` **must** be a valid XrLoaderInterfaceStructs value
- `getInstanceProcAddr` **must** be a valid PFN_xrGetInstanceProcAddr value
- `createApiLayerInstance` **must** be a valid PFN_xrCreateApiLayerInstance value

```
// Provided by XR_LOADER_VERSION_1_0
#define XR_API_LAYER_INFO_STRUCT_VERSION 1
```

XR_API_LAYER_INFO_STRUCT_VERSION is the current version of the XrNegotiateApiLayerRequest structure. It is used to populate the XrNegotiateApiLayerRequest::`structVersion` field.

The xrCreateApiLayerInstance function is defined as:

```
// Provided by XR_LOADER_VERSION_1_0
XrResult xrCreateApiLayerInstance(
    const XrInstanceCreateInfo*                 info,
    const XrApiLayerCreateInfo*                 layerInfo,
    XrInstance*                                 instance);
```

Parameter Descriptions

- `info` is a pointer to the XrInstanceCreateInfo information passed by the application into the outer xrCreateInstance function.
- `layerInfo` is a pointer to an XrApiLayerCreateInfo structure that contains special information required by a API layer during its create instance process. This is generated by the loader.
- `instance` is a pointer to store the returned instance in, just as in the standard xrCreateInstance function.

An API layer's implementation of the xrCreateApiLayerInstance function is invoked during the loader's implementation of xrCreateInstance, if the layer in question is enabled.

An API layer needs additional information during xrCreateInstance calls, so each API layer **must** implement the xrCreateApiLayerInstance function, which is a special API layer function.

An API layer **must** not implement xrCreateInstance.

xrCreateApiLayerInstance **must** be called by the loader during its implementation of the xrCreateInstance function.

The loader **must** call the first API layer's xrCreateApiLayerInstance function passing in the pointer to the created XrApiLayerCreateInfo.

The XrApiLayerCreateInfo::`nextInfo` **must** be a linked-list of XrApiLayerNextInfo structures with information about each of the API layers that are to be enabled. Note that this does not operate like a `next` chain in the OpenXR application API, but instead describes the enabled API layers from outermost to innermost.

The API layer **may** validate that it is getting the correct next information by checking that the XrApiLayerNextInfo::`layerName` matches the expected value.

The API layer **must** use the information in its XrApiLayerNextInfo to call down the call chain to the next xrCreateApiLayerInstance:

- The API layer **must** copy the XrApiLayerCreateInfo structure into its own structure.
- The API layer **must** then update its copy of the XrApiLayerCreateInfo structure, setting XrApiLayerCreateInfo::`nextInfo` to point to the XrApiLayerNextInfo for the next API layer (e.g.

`layerInfoCopy->nextInfo = layerInfo->nextInfo->next;`).

- The API layer **must** then use the pointer to its XrApiLayerCreateInfo structure (instead of the one that was passed in) when it makes a call to the xrCreateApiLayerInstance function.
- If the nested xrCreateApiLayerInstance call succeeds, the API layer **may** choose to setup its own dispatch table to the next API layer's functions using the returned XrInstance and the next API layer's xrGetInstanceProcAddr.
- The API layer **must** return the XrResult returned from the next API layer.

Valid Usage (Implicit)

- `info` **must** be a pointer to a valid XrInstanceCreateInfo structure
- `layerInfo` **must** be a pointer to a valid XrApiLayerCreateInfo structure
- `instance` **must** be a pointer to an XrInstance handle

Return Codes

Success

- `XR_SUCCESS`

Failure

- `XR_ERROR_INITIALIZATION_FAILED`

The XrApiLayerCreateInfo structure contains special information required by a API layer during its create instance process.

The XrApiLayerCreateInfo structure is defined as:

```
// Provided by XR_LOADER_VERSION_1_0
typedef struct XrApiLayerCreateInfo {
    XrLoaderInterfaceStructs    structType;
    uint32_t                    structVersion;
    size_t                      structSize;
    void*                       loaderInstance;
    char                        settings_file_location
[XR_API_LAYER_MAX_SETTINGS_PATH_SIZE];
    XrApiLayerNextInfo*         nextInfo;
} XrApiLayerCreateInfo;
```

Member Descriptions

- `structType` **must** be `XR_LOADER_INTERFACE_STRUCT_API_LAYER_CREATE_INFO`.
- `structVersion` is the version of the structure being supplied by the loader (i.e. XR_API_LAYER_CREATE_INFO_STRUCT_VERSION)
- `structSize` **must** be the size in bytes of the current version of the structure (i.e. `sizeof(XrApiLayerCreateInfo)`)
- `loaderInstance` is deprecated and **must** be ignored.
- `settings_file_location` is the location of any usable API layer settings file. The size of `settings_file_location` is given by XR_API_LAYER_MAX_SETTINGS_PATH_SIZE. This is currently unused.
- `nextInfo` is a pointer to the XrApiLayerNextInfo structure which contains information to work with the next API layer in the chain.

Valid Usage (Implicit)

- `structType` **must** be a valid XrLoaderInterfaceStructs value
- `loaderInstance` **must** be a pointer value
- `settings_file_location` **must** be a null-terminated UTF-8 string whose length is less than or equal to XR_API_LAYER_MAX_SETTINGS_PATH_SIZE
- `nextInfo` **must** be a pointer to an XrApiLayerNextInfo structure

```
// Provided by XR_LOADER_VERSION_1_0
#define XR_API_LAYER_CREATE_INFO_STRUCT_VERSION 1
```

XR_API_LAYER_CREATE_INFO_STRUCT_VERSION is the current version of the XrApiLayerCreateInfo structure. It is used to populate the XrApiLayerCreateInfo::`structVersion` field.

```
// Provided by XR_LOADER_VERSION_1_0
#define XR_API_LAYER_MAX_SETTINGS_PATH_SIZE 512
```

XR_API_LAYER_MAX_SETTINGS_PATH_SIZE is the size of the XrApiLayerCreateInfo ::`settings_file_location` field.

The XrApiLayerNextInfo structure:

The XrApiLayerNextInfo structure is defined as:

```
// Provided by XR_LOADER_VERSION_1_0
typedef struct XrApiLayerNextInfo {
    XrLoaderInterfaceStructs        structType;
    uint32_t                        structVersion;
    size_t                          structSize;
    char                            layerName[XR_MAX_API_LAYER_NAME_SIZE];
    PFN_xrGetInstanceProcAddr       nextGetInstanceProcAddr;
    PFN_xrCreateApiLayerInstance    nextCreateApiLayerInstance;
    struct XrApiLayerNextInfo*      next;
} XrApiLayerNextInfo;
```

Member Descriptions

- `structType` **must** be `XR_LOADER_INTERFACE_STRUCT_API_LAYER_NEXT_INFO`
- `structVersion` **must** be a valid version of the structure and the version being supplied by the loader (i.e. XR_API_LAYER_NEXT_INFO_STRUCT_VERSION).
- `structSize` **must** be the size in bytes of the current version of the structure (i.e. `sizeof(XrApiLayerNextInfo)`)
- `layerName` is the name of the intended next API layer, used to verify and debug the API layer chain.
- `nextGetInstanceProcAddr` is a pointer to the next API layer's xrGetInstanceProcAddr. This is intended for use in populating a dispatch table to the next implementations in the chain.
- `nextCreateApiLayerInstance` is a pointer to the xrCreateApiLayerInstance function implementation in the next API layer. This is to be called **after** the API layer has done any localized creation, but **before** the API layer records any function addresses from the next API layer using xrGetInstanceProcAddr.
- `next` is a pointer to the XrApiLayerNextInfo for the next API layer. If no API layer is after this, it will be `NULL`.

Valid Usage (Implicit)

- `structType` **must** be a valid XrLoaderInterfaceStructs value
- `layerName` **must** be a null-terminated UTF-8 string whose length is less than or equal to `XR_MAX_API_LAYER_NAME_SIZE`
- `nextGetInstanceProcAddr` **must** be a valid PFN_xrGetInstanceProcAddr value
- `nextCreateApiLayerInstance` **must** be a valid PFN_xrCreateApiLayerInstance value
- `next` **must** be a pointer to an XrApiLayerNextInfo structure

```
// Provided by XR_LOADER_VERSION_1_0
#define XR_API_LAYER_NEXT_INFO_STRUCT_VERSION 1
```

XR_API_LAYER_NEXT_INFO_STRUCT_VERSION is the current version of the XrApiLayerNextInfo structure. It is used to populate the XrApiLayerNextInfo::`structVersion` field.

Chapter 4. Instance

```
// Provided by XR_VERSION_1_0
XR_DEFINE_HANDLE(XrInstance)
```

An OpenXR instance is an object that allows an OpenXR application to communicate with an OpenXR runtime. The application accomplishes this communication by calling xrCreateInstance and receiving a handle to the resulting XrInstance object.

The XrInstance object stores and tracks OpenXR-related application state, without storing any such state in the application's global address space. This allows the application to create multiple instances as well as safely encapsulate the application's OpenXR state since this object is opaque to the application. OpenXR runtimes **may** limit the number of simultaneous XrInstance objects that may be created and used, but they **must** support the creation and usage of at least one XrInstance object per process.

Physically, this state **may** be stored in any of the OpenXR loader, OpenXR API layers or the OpenXR runtime components. The exact storage and distribution of this saved state is implementation-dependent, except where indicated by this specification.

The tracking of OpenXR state in the instance allows the streamlining of the API, where the intended instance is inferred from the highest ascendant of an OpenXR function's target object. For example, in:

```
myResult = xrEndFrame(mySession, &myEndFrameDescription);
```

the XrSession object was created from an XrInstance object. The OpenXR loader typically keeps track of the XrInstance that is the parent of the XrSession object in this example and directs the function to the runtime associated with that instance. This tracking of OpenXR objects eliminates the need to specify an XrInstance in every OpenXR function.

4.1. API Layers and Extensions

Additional functionality **may** be provided by API layers or extensions. An API layer **must** not add or modify the definition of OpenXR functions, while an extension **may** do so.

The set of API layers to enable is specified when creating an instance, and those API layers are able to intercept any functions dispatched to that instance or any of its child objects.

Example API layers **may** include (but are not limited to):

- an API layer to dump out OpenXR API calls
- an API layer to perform OpenXR validation

To determine what set of API layers are available, OpenXR provides the xrEnumerateApiLayerProperties function:

```
// Provided by XR_VERSION_1_0
XrResult xrEnumerateApiLayerProperties(
    uint32_t                                    propertyCapacityInput,
    uint32_t*                                   propertyCountOutput,
    XrApiLayerProperties*                       properties);
```

Parameter Descriptions

- propertyCapacityInput is the capacity of the properties array, or 0 to indicate a request to retrieve the required capacity.
- propertyCountOutput is a pointer to the count of properties written, or a pointer to the required capacity in the case that propertyCapacityInput is insufficient.
- properties is a pointer to an array of XrApiLayerProperties structures, but **can** be NULL if propertyCapacityInput is 0.
- See the Buffer Size Parameters section for a detailed description of retrieving the required properties size.

The list of available layers may change at any time due to actions outside of the OpenXR runtime, so two calls to xrEnumerateApiLayerProperties with the same parameters **may** return different results, or retrieve different propertyCountOutput values or properties contents.

Once an instance has been created, the layers enabled for that instance will continue to be enabled and valid for the lifetime of that instance, even if some of them become unavailable for future instances.

Valid Usage (Implicit)

- propertyCountOutput **must** be a pointer to a uint32_t value
- If propertyCapacityInput is not 0, properties **must** be a pointer to an array of propertyCapacityInput XrApiLayerProperties structures

Return Codes

Success

- XR_SUCCESS

Failure

- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SIZE_INSUFFICIENT
- XR_ERROR_VALIDATION_FAILURE

The XrApiLayerProperties structure is defined as:

```
// Provided by XR_VERSION_1_0
typedef struct XrApiLayerProperties {
    XrStructureType    type;
    void*              next;
    char               layerName[XR_MAX_API_LAYER_NAME_SIZE];
    XrVersion          specVersion;
    uint32_t           layerVersion;
    char               description[XR_MAX_API_LAYER_DESCRIPTION_SIZE];
} XrApiLayerProperties;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR.
- layerName is a string specifying the name of the API layer. Use this name in the XrInstanceCreateInfo::enabledApiLayerNames array to enable this API layer for an instance.
- specVersion is the API version the API layer was written to, encoded as described in the API Version Numbers and Semantics section.
- layerVersion is the version of this API layer. It is an integer, increasing with backward compatible changes.
- description is a string providing additional details that **can** be used by the application to identify the API layer.

Valid Usage (Implicit)

- `type` **must** be `XR_TYPE_API_LAYER_PROPERTIES`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

To enable a layer, the name of the layer **should** be added to XrInstanceCreateInfo ::`enabledApiLayerNames` when creating an XrInstance.

Loader implementations **may** provide mechanisms outside this API for enabling specific API layers. API layers enabled through such a mechanism are implicitly enabled, while API layers enabled by including the API layer name in XrInstanceCreateInfo::`enabledApiLayerNames` are explicitly enabled. Except where otherwise specified, implicitly enabled and explicitly enabled API layers differ only in the way they are enabled. Explicitly enabling an API layer that is implicitly enabled has no additional effect.

Instance extensions are able to affect the operation of the instance and any of its child objects. As stated earlier, extensions can expand the OpenXR API and provide new functions or augment behavior.

Examples of extensions **may** be (but are not limited to):

Extension Examples

- an extension to include OpenXR functions to work with a new graphics API
- an extension to expose debug information via a callback

The application can determine the available instance extensions by calling xrEnumerateInstanceExtensionProperties:

```
// Provided by XR_VERSION_1_0
XrResult xrEnumerateInstanceExtensionProperties(
    const char*                                 layerName,
    uint32_t                                    propertyCapacityInput,
    uint32_t*                                   propertyCountOutput,
    XrExtensionProperties*                      properties);
```

Parameter Descriptions

- `layerName` is either `NULL` or a pointer to a string naming the API layer to retrieve extensions from, as returned by xrEnumerateApiLayerProperties.
- `propertyCapacityInput` is the capacity of the `properties` array, or `0` to indicate a request to retrieve the required capacity.
- `propertyCountOutput` is a pointer to the count of `properties` written, or a pointer to the required capacity in the case that `propertyCapacityInput` is insufficient.
- `properties` is a pointer to an array of XrExtensionProperties structures, but **can** be `NULL` if `propertyCapacityInput` is `0`.
- See the Buffer Size Parameters section for a detailed description of retrieving the required `properties` size.

Because the list of available layers may change externally between calls to xrEnumerateInstanceExtensionProperties, two calls **may** retrieve different results if a `layerName` is available in one call but not in another. The extensions supported by a layer may also change between two calls, e.g. if the layer implementation is replaced by a different version between those calls.

Valid Usage (Implicit)

- If `layerName` is not `NULL`, `layerName` **must** be a null-terminated UTF-8 string
- `propertyCountOutput` **must** be a pointer to a `uint32_t` value
- If `propertyCapacityInput` is not `0`, `properties` **must** be a pointer to an array of `propertyCapacityInput` XrExtensionProperties structures

Return Codes

Success

- `XR_SUCCESS`

Failure

- `XR_ERROR_API_LAYER_NOT_PRESENT`
- `XR_ERROR_OUT_OF_MEMORY`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_RUNTIME_UNAVAILABLE`
- `XR_ERROR_SIZE_INSUFFICIENT`
- `XR_ERROR_VALIDATION_FAILURE`

The XrExtensionProperties structure is defined as:

```
// Provided by XR_VERSION_1_0
typedef struct XrExtensionProperties {
    XrStructureType    type;
    void*              next;
    char               extensionName[XR_MAX_EXTENSION_NAME_SIZE];
    uint32_t           extensionVersion;
} XrExtensionProperties;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR.
- `extensionName` is a `NULL` terminated string specifying the name of the extension.
- `extensionVersion` is the version of this extension. It is an integer, incremented with backward compatible changes.

Valid Usage (Implicit)

- `type` **must** be `XR_TYPE_EXTENSION_PROPERTIES`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

4.2. Instance Lifecycle

The xrCreateInstance function is defined as:

```
// Provided by XR_VERSION_1_0
XrResult xrCreateInstance(
    const XrInstanceCreateInfo*                 createInfo,
    XrInstance*                                 instance);
```

Parameter Descriptions

- `createInfo` points to an instance of XrInstanceCreateInfo controlling creation of the instance.
- `instance` points to an XrInstance handle in which the resulting instance is returned.

xrCreateInstance creates the XrInstance, then enables and initializes global API layers and extensions requested by the application. If an extension is provided by an API layer, both the API layer and extension **must** be specified at xrCreateInstance time. If a specified API layer cannot be found, no XrInstance will be created and the function will return `XR_ERROR_API_LAYER_NOT_PRESENT`. Likewise, if a specified extension cannot be found, the call **must** return `XR_ERROR_EXTENSION_NOT_PRESENT` and no XrInstance will be created. Additionally, some runtimes **may** limit the number of concurrent instances that may be in use. If the application attempts to create more instances than a runtime can simultaneously support, xrCreateInstance **may** return `XR_ERROR_LIMIT_REACHED`.

If the XrApplicationInfo::`applicationName` is the empty string the runtime **must** return `XR_ERROR_NAME_INVALID`.

If the XrInstanceCreateInfo structure contains a platform-specific extension for a platform other than the target platform, `XR_ERROR_INITIALIZATION_FAILED` **may** be returned. If a mandatory platform-specific extension is defined for the target platform but no matching extension struct is provided in XrInstanceCreateInfo the runtime **must** return `XR_ERROR_INITIALIZATION_FAILED`.

Valid Usage (Implicit)

- `createInfo` **must** be a pointer to a valid XrInstanceCreateInfo structure
- `instance` **must** be a pointer to an XrInstance handle

Return Codes

Success

- XR_SUCCESS

Failure

- XR_ERROR_API_LAYER_NOT_PRESENT
- XR_ERROR_API_VERSION_UNSUPPORTED
- XR_ERROR_EXTENSION_DEPENDENCY_NOT_ENABLED
- XR_ERROR_EXTENSION_NOT_PRESENT
- XR_ERROR_INITIALIZATION_FAILED
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_NAME_INVALID
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_RUNTIME_UNAVAILABLE
- XR_ERROR_VALIDATION_FAILURE

The XrInstanceCreateInfo structure is defined as:

```
// Provided by XR_VERSION_1_0
typedef struct XrInstanceCreateInfo {
    XrStructureType          type;
    const void*              next;
    XrInstanceCreateFlags    createFlags;
    XrApplicationInfo        applicationInfo;
    uint32_t                 enabledApiLayerCount;
    const char* const*       enabledApiLayerNames;
    uint32_t                 enabledExtensionCount;
    const char* const*       enabledExtensionNames;
} XrInstanceCreateInfo;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR.
- `createFlags` is a bitmask of XrInstanceCreateFlags that identifies options that apply to the creation.
- `applicationInfo` is an instance of XrApplicationInfo. This information helps runtimes recognize behavior inherent to classes of applications. XrApplicationInfo is defined in detail below.
- `enabledApiLayerCount` is the number of global API layers to enable.
- `enabledApiLayerNames` is a pointer to an array of `enabledApiLayerCount` strings containing the names of API layers to enable for the created instance. See the API Layers and Extensions section for further details.
- `enabledExtensionCount` is the number of global extensions to enable.
- `enabledExtensionNames` is a pointer to an array of `enabledExtensionCount` strings containing the names of extensions to enable.

Valid Usage (Implicit)

- `type` **must** be `XR_TYPE_INSTANCE_CREATE_INFO`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain. See also: XrDebugUtilsMessengerCreateInfoEXT, XrInstanceCreateInfoAndroidKHR
- `createFlags` **must** be `0`
- `applicationInfo` **must** be a valid XrApplicationInfo structure
- If `enabledApiLayerCount` is not `0`, `enabledApiLayerNames` **must** be a pointer to an array of `enabledApiLayerCount` null-terminated UTF-8 strings
- If `enabledExtensionCount` is not `0`, `enabledExtensionNames` **must** be a pointer to an array of `enabledExtensionCount` null-terminated UTF-8 strings

The XrInstanceCreateInfo::`createFlags` member is of the following type, and contains a bitwise-OR of zero or more of the bits defined in XrInstanceCreateFlagBits.

```
// Provided by XR_VERSION_1_0
typedef XrFlags64 XrInstanceCreateFlags;
```

Valid bits for XrInstanceCreateFlags are defined by XrInstanceCreateFlagBits.

```
// Provided by XR_VERSION_1_0
// Flag bits for XrInstanceCreateFlags
```

There are currently no instance creation flag bits defined. This is reserved for future use.

The XrApplicationInfo structure is defined as:

```
// Provided by XR_VERSION_1_0
typedef struct XrApplicationInfo {
    char         applicationName[XR_MAX_APPLICATION_NAME_SIZE];
    uint32_t     applicationVersion;
    char         engineName[XR_MAX_ENGINE_NAME_SIZE];
    uint32_t     engineVersion;
    XrVersion    apiVersion;
} XrApplicationInfo;
```

Member Descriptions

- `applicationName` is a non-empty string containing the name of the application.
- `applicationVersion` is an unsigned integer variable containing the developer-supplied version number of the application.
- `engineName` is a string containing the name of the engine (if any) used to create the application. It may be empty to indicate no specified engine.
- `engineVersion` is an unsigned integer variable containing the developer-supplied version number of the engine used to create the application. May be zero to indicate no specified engine.
- `apiVersion` is the version of this API against which the application will run, encoded as described in the API Version Numbers and Semantics section. If the runtime does not support the requested `apiVersion` it **must** return `XR_ERROR_API_VERSION_UNSUPPORTED`.

Useful values for `apiVersion` include XR_API_VERSION_1_0 and XR_API_VERSION_1_1.

Valid Usage (Implicit)

- `applicationName` **must** be a null-terminated UTF-8 string whose length is less than or equal to `XR_MAX_APPLICATION_NAME_SIZE`
- `engineName` **must** be a null-terminated UTF-8 string whose length is less than or equal to `XR_MAX_ENGINE_NAME_SIZE`



Note

When using the OpenXR API to implement a reusable engine that will be used by many applications, `engineName` **should** be set to a unique string that identifies the engine, and `engineVersion` **should** encode a representation of the engine's version. This way, all applications that share this engine version will provide the same `engineName` and `engineVersion` to the runtime. The engine **should** then enable individual applications to choose their specific `applicationName` and `applicationVersion`, enabling one application to be distinguished from another application.

When using the OpenXR API to implement an individual application without a shared engine, the input `engineName` **should** be left empty and `engineVersion` **should** be set to 0. The `applicationName` **should** then be filled in with a unique string that identifies the app and the `applicationVersion` **should** encode a representation of the application's version.

The xrDestroyInstance function is defined as:

```
// Provided by XR_VERSION_1_0
XrResult xrDestroyInstance(
    XrInstance                                  instance);
```

The xrDestroyInstance function is used to destroy an XrInstance.

Parameter Descriptions

- `instance` is the handle to the instance to destroy.

XrInstance handles are destroyed using xrDestroyInstance. When an XrInstance is destroyed, all handles that are children of that XrInstance are also destroyed.

Valid Usage (Implicit)

- `instance` **must** be a valid XrInstance handle

Thread Safety

- Access to `instance`, and any child handles, **must** be externally synchronized

Return Codes

Success

- XR_SUCCESS

Failure

- XR_ERROR_HANDLE_INVALID
- XR_ERROR_RUNTIME_FAILURE

4.3. Instance Information

The xrGetInstanceProperties function provides information about the instance and the associated runtime.

```
// Provided by XR_VERSION_1_0
XrResult xrGetInstanceProperties(
    XrInstance                                  instance,
    XrInstanceProperties*                       instanceProperties);
```

Parameter Descriptions

- instance is a handle to an XrInstance previously created with xrCreateInstance.
- instanceProperties points to an XrInstanceProperties which describes the instance.

The instanceProperties parameter **must** be filled out by the runtime in response to this call, with information as defined in XrInstanceProperties.

Valid Usage (Implicit)

- instance **must** be a valid XrInstance handle
- instanceProperties **must** be a pointer to an XrInstanceProperties structure

Return Codes

Success

- XR_SUCCESS

Failure

- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_VALIDATION_FAILURE

The XrInstanceProperties structure is defined as:

```
// Provided by XR_VERSION_1_0
typedef struct XrInstanceProperties {
    XrStructureType    type;
    void*              next;
    XrVersion          runtimeVersion;
    char               runtimeName[XR_MAX_RUNTIME_NAME_SIZE];
} XrInstanceProperties;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR.
- runtimeVersion is the runtime's version (not necessarily related to an OpenXR API version), expressed in the format of XR_MAKE_VERSION.
- runtimeName is the name of the runtime.

Valid Usage (Implicit)

- type **must** be XR_TYPE_INSTANCE_PROPERTIES
- next **must** be NULL or a valid pointer to the next structure in a structure chain

4.4. Platform-Specific Instance Creation

Some amount of data required for instance creation is exposed through chained structures defined in extensions. These structures may be **optional** or even **required** for instance creation on specific platforms, but not on other platforms. Separating off platform-specific functionality into extension structures prevents the primary XrInstanceCreateInfo structure from becoming too bloated with unnecessary information.

See the List of Extensions appendix for the list of available extensions and their related structures. These structures expand the XrInstanceCreateInfo parent struct using the XrInstanceCreateInfo::next member. The specific list of structures that may be used for extending XrInstanceCreateInfo::next can be found in the "Valid Usage (Implicit)" block immediately following the definition of the structure.

4.4.1. The Instance Lost Error

The XR_ERROR_INSTANCE_LOST error indicates that the XrInstance has become unusable. This **can** happen if a critical runtime process aborts, if the connection to the runtime is otherwise no longer available, or if the runtime encounters an error during any function execution which prevents it from being able to support further function execution. Once XR_ERROR_INSTANCE_LOST is first returned, it **must** henceforth be returned by all non-destroy functions that involve an XrInstance or child handle type until the instance is destroyed. Applications **must** destroy the XrInstance. Applications **may** then attempt to continue by recreating all relevant OpenXR objects, starting with a new XrInstance. A runtime **may** generate an XrEventDataInstanceLossPending event when instance loss is detected.

4.4.2. XrEventDataInstanceLossPending

The XrEventDataInstanceLossPending structure is defined as:

```
// Provided by XR_VERSION_1_0
typedef struct XrEventDataInstanceLossPending {
    XrStructureType    type;
    const void*        next;
    XrTime             lossTime;
} XrEventDataInstanceLossPending;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR.
- lossTime is the absolute time at which the indicated instance will be considered lost and become unusable.

Receiving the XrEventDataInstanceLossPending event structure indicates that the application is about to lose the indicated XrInstance at the indicated `lossTime` in the future. The application should call xrDestroyInstance and relinquish any instance-specific resources. This typically occurs to make way for a replacement of the underlying runtime, such as via a software update.

After the application has destroyed all of its instances and their children and waited past the specified time, it may then re-try xrCreateInstance in a loop waiting for whatever maintenance the runtime is performing to complete. The runtime will return `XR_ERROR_RUNTIME_UNAVAILABLE` from xrCreateInstance as long as it is unable to create the instance. Once the runtime has returned and is able to continue, it **must** resume returning `XR_SUCCESS` from xrCreateInstance if valid data is passed in.

Valid Usage (Implicit)

- `type` **must** be `XR_TYPE_EVENT_DATA_INSTANCE_LOSS_PENDING`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

4.5. Instance Enumerated Type String Functions

Applications often want to turn certain enum values from the runtime into strings for use in log messages, to be localized in UI, or for various other reasons. OpenXR provides functions that turn common enum types into UTF-8 strings for use in applications.

```
// Provided by XR_VERSION_1_0
XrResult xrResultToString(
    XrInstance                                  instance,
    XrResult                                    value,
    char                                        buffer[XR_MAX_RESULT_STRING_SIZE]);
```

Parameter Descriptions

- `instance` is the handle of the instance to ask for the string.
- `value` is the XrResult value to turn into a string.
- `buffer` is the buffer that will be used to return the string in.

Returns the text version of the provided XrResult value as a UTF-8 string.

In all cases the returned string **must** be one of:

Result String Return Values

- The literal string defined for the provide numeric value in the core spec or extension. (e.g. the value 0 results in the string `XR_SUCCESS`)
- `XR_UNKNOWN_SUCCESS_` concatenated with the positive result number expressed as a decimal number.
- `XR_UNKNOWN_FAILURE_` concatenated with the negative result number expressed as a decimal number.

Valid Usage (Implicit)

- `instance` **must** be a valid XrInstance handle
- `value` **must** be a valid XrResult value
- `buffer` **must** be a character array of length `XR_MAX_RESULT_STRING_SIZE`

Return Codes

Success

- `XR_SUCCESS`

Failure

- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_VALIDATION_FAILURE`

The `XR_MAX_RESULT_STRING_SIZE` enumerant defines the size of the buffer passed to xrResultToString.

```
#define XR_MAX_RESULT_STRING_SIZE         64
```

The xrStructureTypeToString function is defined as:

```
// Provided by XR_VERSION_1_0
XrResult xrStructureTypeToString(
    XrInstance                                  instance,
    XrStructureType                             value,
    char                                        buffer[XR_MAX_STRUCTURE_NAME_SIZE]);
```

Parameter Descriptions

- `instance` is the handle of the instance to ask for the string.
- `value` is the XrStructureType value to turn into a string.
- `buffer` is the buffer that will be used to return the string in.

Returns the text version of the provided XrStructureType value as a UTF-8 string.

In all cases the returned string **must** be one of:

Structure Type String Return Values

- The literal string defined for the provide numeric value in the core spec or extension. (e.g. the value of `XR_TYPE_INSTANCE_CREATE_INFO` results in the string `XR_TYPE_INSTANCE_CREATE_INFO`)
- `XR_UNKNOWN_STRUCTURE_TYPE_` concatenated with the structure type number expressed as a decimal number.

Valid Usage (Implicit)

- `instance` **must** be a valid XrInstance handle
- `value` **must** be a valid XrStructureType value
- `buffer` **must** be a character array of length `XR_MAX_STRUCTURE_NAME_SIZE`

Return Codes

Success

- `XR_SUCCESS`

Failure

- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_VALIDATION_FAILURE`

The `XR_MAX_STRUCTURE_NAME_SIZE` enumerant defines the size of the buffer passed to xrStructureTypeToString.

```
#define XR_MAX_STRUCTURE_NAME_SIZE        64
```

Chapter 5. System

This API separates the concept of physical systems of XR devices from the logical objects that applications interact with directly. A system represents a collection of related devices in the runtime, often made up of several individual hardware components working together to enable XR experiences. An `XrSystemId` is returned by xrGetSystem representing the system of devices the runtime will use to support a given form factor. Each system may include: a VR/AR display, various forms of input (gamepad, touchpad, motion controller), and other trackable objects.

The application uses the system to create a session, which can then be used to accept input from the user and output rendered frames. The application also provides suggested bindings from its actions to any number of input sources. The runtime **may** use this action information to activate only a subset of devices and avoid wasting resources on devices that are not in use. Exactly which devices are active once an XR system is selected will depend on the features provided by the runtime, and **may** vary from runtime to runtime. For example, a runtime that is capable of mapping from one tracking system's space to another's **may** support devices from multiple tracking systems simultaneously.

5.1. Form Factors

The first step in selecting a system is for the application to request its desired **form factor**. The form factor defines how the display(s) moves in the environment relative to the user's head and how the user will interact with the XR experience. A runtime **may** support multiple form factors, such as on a mobile phone that supports both slide-in VR headset experiences and handheld AR experiences.

While an application's core XR rendering may span across form factors, its user interface will often be written to target a particular form factor, requiring explicit tailoring to function well on other form factors. For example, screen-space UI designed for a handheld phone will produce an uncomfortable experience for users if presented in screen-space on an AR headset.

```
// Provided by XR_VERSION_1_0
typedef enum XrFormFactor {
    XR_FORM_FACTOR_HEAD_MOUNTED_DISPLAY = 1,
    XR_FORM_FACTOR_HANDHELD_DISPLAY = 2,
    XR_FORM_FACTOR_MAX_ENUM = 0x7FFFFFFF
} XrFormFactor;
```

The predefined form factors which **may** be supported by OpenXR runtimes are:

Enumerant Descriptions

- XR_FORM_FACTOR_HEAD_MOUNTED_DISPLAY. The tracked display is attached to the user's head. The user cannot touch the display itself. A VR headset would be an example of this form factor.
- XR_FORM_FACTOR_HANDHELD_DISPLAY. The tracked display is held in the user's hand, independent from the user's head. The user **may** be able to touch the display, allowing for screen-space UI. A mobile phone running an AR experience using pass-through video would be an example of this form factor.

5.2. Getting the XrSystemId

```
// Provided by XR_VERSION_1_0
XR_DEFINE_ATOM(XrSystemId)
```

An XrSystemId is an opaque atom used by the runtime to identify a system. The value XR_NULL_SYSTEM_ID is considered an invalid system.

```
// Provided by XR_VERSION_1_0
#define XR_NULL_SYSTEM_ID 0
```

The only XrSystemId value defined to be constant across all instances is the invalid system XR_NULL_SYSTEM_ID. No supported system is associated with XR_NULL_SYSTEM_ID. Unless explicitly permitted, it **should** not be passed to API calls or used as a structure attribute when a valid XrSystemId is required.

The xrGetSystem function is defined as:

```
// Provided by XR_VERSION_1_0
XrResult xrGetSystem(
    XrInstance                                  instance,
    const XrSystemGetInfo*                      getInfo,
    XrSystemId*                                 systemId);
```

Parameter Descriptions

- `instance` is the handle of the instance from which to get the information.
- `getInfo` is a pointer to an XrSystemGetInfo structure containing the application's requests for a system.
- `systemId` is the returned `XrSystemId`.

To get an `XrSystemId`, an application specifies its desired form factor to xrGetSystem and gets the runtime's `XrSystemId` associated with that configuration.

If the form factor is supported but temporarily unavailable, xrGetSystem **must** return `XR_ERROR_FORM_FACTOR_UNAVAILABLE`. A runtime **may** return `XR_SUCCESS` on a subsequent call for a form factor it previously returned `XR_ERROR_FORM_FACTOR_UNAVAILABLE`. For example, connecting or warming up hardware might cause an unavailable form factor to become available.

Valid Usage (Implicit)

- `instance` **must** be a valid XrInstance handle
- `getInfo` **must** be a pointer to a valid XrSystemGetInfo structure
- `systemId` **must** be a pointer to an `XrSystemId` value

Return Codes

Success

- `XR_SUCCESS`

Failure

- `XR_ERROR_FORM_FACTOR_UNAVAILABLE`
- `XR_ERROR_FORM_FACTOR_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_VALIDATION_FAILURE`

The XrSystemGetInfo structure is defined as:

```
// Provided by XR_VERSION_1_0
typedef struct XrSystemGetInfo {
    XrStructureType    type;
    const void*        next;
    XrFormFactor       formFactor;
} XrSystemGetInfo;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR.
- `formFactor` is the XrFormFactor requested by the application.

The XrSystemGetInfo structure specifies attributes about a system as desired by an application.

Valid Usage (Implicit)

- `type` **must** be `XR_TYPE_SYSTEM_GET_INFO`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `formFactor` **must** be a valid XrFormFactor value

```
XrInstance instance; // previously initialized

XrSystemGetInfo system_get_info {
    .type = XR_TYPE_SYSTEM_GET_INFO,
    .formFactor = XR_FORM_FACTOR_HEAD_MOUNTED_DISPLAY,
};

XrSystemId systemId {XR_NULL_SYSTEM_ID};
CHK_XR(xrGetSystem(instance, &system_get_info, &systemId));

// create session
// create swapchains
// begin session

// main loop

// end session
// destroy session

// no access to hardware after this point
```

5.3. System Properties

The xrGetSystemProperties function is defined as:

```
// Provided by XR_VERSION_1_0
XrResult xrGetSystemProperties(
    XrInstance                                  instance,
    XrSystemId                                  systemId,
    XrSystemProperties*                         properties);
```

Parameter Descriptions

- `instance` is the instance from which `systemId` was retrieved.
- `systemId` is the `XrSystemId` whose properties will be queried.
- `properties` points to an instance of the XrSystemProperties structure, that will be filled with returned information.

An application **can** call xrGetSystemProperties to retrieve information about the system such as vendor ID, system name, and graphics and tracking properties.

Valid Usage (Implicit)

- `instance` **must** be a valid XrInstance handle
- `properties` **must** be a pointer to an XrSystemProperties structure

Return Codes

Success

- XR_SUCCESS

Failure

- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SYSTEM_INVALID
- XR_ERROR_VALIDATION_FAILURE

The XrSystemProperties structure is defined as:

```
// Provided by XR_VERSION_1_0
typedef struct XrSystemProperties {
    XrStructureType               type;
    void*                         next;
    XrSystemId                    systemId;
    uint32_t                      vendorId;
    char                          systemName[XR_MAX_SYSTEM_NAME_SIZE];
    XrSystemGraphicsProperties    graphicsProperties;
    XrSystemTrackingProperties    trackingProperties;
} XrSystemProperties;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `vendorId` is a unique identifier for the vendor of the system.
- `systemId` is the `XrSystemId` identifying the system.
- `systemName` is a string containing the name of the system.
- `graphicsProperties` is an XrSystemGraphicsProperties structure specifying the system graphics properties.
- `trackingProperties` is an XrSystemTrackingProperties structure specifying system tracking properties.

Valid Usage (Implicit)

- `type` **must** be `XR_TYPE_SYSTEM_PROPERTIES`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain. See also: XrSystemAnchorPropertiesHTC, XrSystemAnchorSharingExportPropertiesANDROID, XrSystemBodyTrackingPropertiesBD, XrSystemBodyTrackingPropertiesFB, XrSystemBodyTrackingPropertiesHTC, XrSystemBoundaryVisibilityPropertiesMETA, XrSystemColocationDiscoveryPropertiesMETA, XrSystemColorSpacePropertiesFB, XrSystemDeviceAnchorPersistencePropertiesANDROID, XrSystemDynamicObjectKeyboardPropertiesBD, XrSystemDynamicObjectMousePropertiesBD, XrSystemDynamicObjectTrackingPropertiesBD, XrSystemEnvironmentDepthPropertiesMETA, XrSystemEnvironmentRaycastPropertiesMETA, XrSystemEyeGazeInteractionPropertiesEXT, XrSystemEyeTrackingPropertiesANDROID, XrSystemEyeTrackingPropertiesFB, XrSystemFaceTrackingProperties2FB, XrSystemFaceTrackingPropertiesANDROID, XrSystemFaceTrackingPropertiesFB, XrSystemFaceTrackingVisemesPropertiesMETA, XrSystemFacialExpressionPropertiesML, XrSystemFacialSimulationPropertiesBD, XrSystemFacialTrackingPropertiesHTC, XrSystemForceFeedbackCurlPropertiesMNDX, XrSystemFoveatedRenderingPropertiesVARJO, XrSystemFoveationEyeTrackedPropertiesMETA, XrSystemGeospatialAnchorPropertiesANDROID, XrSystemGeospatialPropertiesANDROID, XrSystemHandTrackingMeshPropertiesMSFT, XrSystemHandTrackingPropertiesEXT, XrSystemHapticParametricPropertiesEXT, XrSystemHeadsetIdPropertiesMETA, XrSystemImageTrackingPropertiesANDROID, XrSystemKeyboardTrackingPropertiesFB, XrSystemLightEstimationPropertiesANDROID, XrSystemMarkerTrackingPropertiesANDROID, XrSystemMarkerTrackingPropertiesVARJO, XrSystemMarkerUnderstandingPropertiesML, XrSystemPassthroughCameraStatePropertiesANDROID, XrSystemPassthroughColorLutPropertiesMETA, XrSystemPassthroughLayerPropertiesANDROID, XrSystemPassthroughProperties2FB, XrSystemPassthroughPropertiesFB, XrSystemPlaneDetectionPropertiesEXT, XrSystemPropertiesBodyTrackingCalibrationMETA, XrSystemPropertiesBodyTrackingFidelityMETA, XrSystemPropertiesBodyTrackingFullBodyMETA, XrSystemQrCodeTrackingPropertiesANDROID, XrSystemRenderModelPropertiesFB, XrSystemSceneMeshingPropertiesANDROID, XrSystemSimultaneousHandsAndControllersPropertiesMETA, XrSystemSpaceDiscoveryPropertiesMETA, XrSystemSpacePersistencePropertiesMETA, XrSystemSpaceWarpPropertiesFB, XrSystemSpatialAnchorPropertiesBD, XrSystemSpatialAnchorSharingPropertiesBD, XrSystemSpatialEntityGroupSharingPropertiesMETA, XrSystemSpatialEntityPropertiesFB, XrSystemSpatialEntitySharingPropertiesMETA, XrSystemSpatialMeshPropertiesBD, XrSystemSpatialPlanePropertiesBD, XrSystemSpatialScenePropertiesBD, XrSystemSpatialSensingPropertiesBD, XrSystemTrackablesPropertiesANDROID, XrSystemUserPresencePropertiesEXT, XrSystemVirtualKeyboardPropertiesMETA

The runtime **must** report a valid vendor ID for the system. The vendor ID **must** be either the USB vendor ID defined for the physical device or a Khronos vendor ID.

The XrSystemGraphicsProperties structure is defined as:

```
// Provided by XR_VERSION_1_0
typedef struct XrSystemGraphicsProperties {
    uint32_t    maxSwapchainImageHeight;
    uint32_t    maxSwapchainImageWidth;
    uint32_t    maxLayerCount;
} XrSystemGraphicsProperties;
```

Member Descriptions

- `maxSwapchainImageHeight` is the maximum swapchain image pixel height supported by this system.
- `maxSwapchainImageWidth` is the maximum swapchain image pixel width supported by this system.
- `maxLayerCount` is the maximum number of composition layers supported by this system. The runtime **must** support at least XR_MIN_COMPOSITION_LAYERS_SUPPORTED layers.

```
// Provided by XR_VERSION_1_0
#define XR_MIN_COMPOSITION_LAYERS_SUPPORTED 16
```

XR_MIN_COMPOSITION_LAYERS_SUPPORTED defines the minimum number of composition layers that a conformant runtime must support. A runtime **must** return the XrSystemGraphicsProperties ::`maxLayerCount` at least the value of XR_MIN_COMPOSITION_LAYERS_SUPPORTED.

The XrSystemTrackingProperties structure is defined as:

```
// Provided by XR_VERSION_1_0
typedef struct XrSystemTrackingProperties {
    XrBool32    orientationTracking;
    XrBool32    positionTracking;
} XrSystemTrackingProperties;
```

Member Descriptions

- `orientationTracking` is set to `XR_TRUE` to indicate the system supports orientational tracking of the view pose(s), `XR_FALSE` otherwise.
- `positionTracking` is set to `XR_TRUE` to indicate the system supports positional tracking of the view pose(s), `XR_FALSE` otherwise.

Chapter 6. Path Tree and Semantic Paths

OpenXR incorporates an internal *semantic path tree* model, also known as the *path tree*, with entities associated with nodes organized in a logical tree and referenced by path name strings structured like a filesystem path or URL. The path tree unifies a number of concepts used in this specification and a runtime **may** add additional nodes as implementation details. As a general design principle, the most application-facing paths **should** have semantic and hierarchical meaning in their name. Thus, these paths are often referred to as *semantic paths*. However, path names in the path tree model **may** not all have the same level or kind of semantic meaning.

In regular use in an application, path name strings are converted to instance-specific `XrPath` values which are used in place of path strings. The mapping between `XrPath` values and their corresponding path name strings **may** be considered to be tracked by the runtime in a one-to-one mapping in addition to the natural tree structure of the referenced entities. Runtimes **may** use any internal implementation that satisfies the requirements.

Formally, the runtime maintains an instance-specific bijective mapping between well-formed path name strings and valid `XrPath` (`uint64_t`) values. These `XrPath` values are only valid within a single `XrInstance`, and applications **must** not share these values between instances. Applications **must** instead use the string representation of a path in their code and configuration, and obtain the correct corresponding `XrPath` at runtime in each `XrInstance`. The term *path* or *semantic path* **may** refer interchangeably to either the path name string or its associated `XrPath` value within an instance when context makes it clear which type is being discussed.

Given that path trees are a unifying model in this specification, the entities referenced by paths **can** be of diverse types. For example, they **may** be used to represent physical device or sensor *components*, which **may** be of various *component types*. They **may** also be used to represent frames of reference that are understood by the application and the runtime, as defined by an `XrSpace`. Additionally, to permit runtime re-configuration and support hardware-independent development, any syntactically-valid path string **may** be used to retrieve a corresponding `XrPath` without error given sufficient resources, *even if* no logical or hardware entity currently corresponds to that path at the time of the call. Later retrieval of the associated path string of such an `XrPath` using `xrPathToString` **should** succeed if the other requirements of that call are met. However, using such an `XrPath` in a later call to any other API function **may** result in an error if no entity of the type required by the call is available at the path at that later time. A runtime **should** permit the entity referenced by a path to vary over time to naturally reflect varying system configuration and hardware availability.

6.1. Path Atom Type

```
// Provided by XR_VERSION_1_0
XR_DEFINE_ATOM(XrPath)
```

The `XrPath` is an atom that connects an application with a single path, within the context of a single

instance. There is a bijective mapping between well-formed path strings and atoms in use. This atom is used — in place of the path name string it corresponds to — to retrieve state and perform other operations.

As an XrPath is only shorthand for a well-formed path string, they have no explicit life cycle.

Lifetime is implicitly managed by the XrInstance. An XrPath **must** not be used unless it is received at execution time from the runtime in the context of a particular XrInstance. Therefore, with the exception of XR_NULL_PATH, XrPath values **must** not be specified as constant values in applications: the corresponding path string **should** be used instead. During the lifetime of a given XrInstance, the XrPath associated with that instance with any given well-formed path **must** not vary, and similarly the well-formed path string that corresponds to a given XrPath in that instance **must** not vary. An XrPath that is received from one XrInstance **may** not be used with another. Such an invalid use **may** be detected and result in an error being returned, or it **may** result in undefined behavior.

Well-written applications **should** typically use a small, bounded set of paths in practice. However, the runtime **should** support looking up the XrPath for a large number of path strings for maximum compatibility. Runtime implementers **should** keep in mind that applications supporting diverse systems **may** look up path strings in a quantity exceeding the number of non-empty entities predicted or provided by any one runtime's own path tree model, and this is not inherently an error. However, system resources are finite and thus runtimes **may** signal exhaustion of resources dedicated to these associations under certain conditions.

When discussing the behavior of runtimes at these limits, a *new* XrPath refers to an XrPath value that, as of some point in time, has neither been received by the application nor tracked internally by the runtime. In this case, since an application has not yet received the value of such an XrPath, the runtime has not yet made any assertions about its association with any path string. In this context, *new* only refers to the fact that the mapping has not necessarily been made constant for a given value/path string pair for the remaining life of the associated instance by being revealed to the application. It does not necessarily imply creation of the entity, if any, referred to by such a path. Similarly, it does not imply the absence of such an entity prior to that point. Entities in the path tree have varied lifetime that is independent from the duration of the mapping from path string to XrPath.

For flexibility, the runtime **may** internally track or otherwise make constant, in instance or larger scope, any mapping of a path string to an XrPath value even before an application would otherwise receive that value, thus making it no longer *new* by the above definition.

When the runtime's resources to track the path string-XrPath mapping are exhausted, and the application makes an API call that would have otherwise retrieved a *new* XrPath as defined above, the runtime **must** return XR_ERROR_PATH_COUNT_EXCEEDED. This includes both explicit calls to xrStringToPath as well as other calls that retrieve an XrPath in any other way.

The runtime **should** support creating as many paths as memory will allow and **must** return XR_ERROR_PATH_COUNT_EXCEEDED from relevant functions when no more can be created.

```
// Provided by XR_VERSION_1_0
#define XR_NULL_PATH 0
```

The only XrPath value defined to be constant across all instances is the invalid path XR_NULL_PATH. No well-formed path string is associated with XR_NULL_PATH. Unless explicitly permitted, it **should** not be passed to API calls or used as a structure attribute when a valid XrPath is required.

6.2. Well-Formed Path Strings

Even though they look similar, semantic paths are not file paths. To avoid confusion with file path directory traversal conventions, many file path conventions are explicitly disallowed from well-formed path name strings.

A well-formed path name string **must** conform to the following rules:

- Path name strings **must** be constructed entirely from characters on the following list.
  - Lower case ASCII letters: a-z
  - Numeric digits: 0-9
  - Dash: -
  - Underscore: _
  - Period: .
  - Forward Slash: /
- Path name strings **must** start with a single forward slash character.
- Path name strings **must** not end with a forward slash character.
- Path name strings **must** not contain two or more adjacent forward slash characters.
- Path name strings **must** not contain two forward slash characters that are separated by only period characters.
- Path name strings **must** not contain only period characters following the final forward slash character in the string.
- The maximum string length for a path name string, including the terminating `\0` character, is defined by XR_MAX_PATH_LENGTH.

6.2.1. xrStringToPath

The xrStringToPath function is defined as:

```
// Provided by XR_VERSION_1_0
XrResult xrStringToPath(
    XrInstance                                  instance,
    const char*                                 pathString,
    XrPath*                                     path);
```

Parameter Descriptions

- `instance` is an instance previously created.
- `pathString` is the path name string to retrieve the associated `XrPath` for.
- `path` is the output parameter, which **must** point to an `XrPath`. Given a well-formed path name string, this will be populated with an opaque value that is constant for that path string during the lifetime of that instance.

xrStringToPath retrieves the `XrPath` value for a well-formed path string. If such a value had not yet been assigned by the runtime to the provided path string in this XrInstance, one **must** be assigned at this point. All calls to this function with the same XrInstance and path string **must** retrieve the same `XrPath` value. Upon failure, xrStringToPath **must** return an appropriate XrResult, and **may** set the output parameter to XR_NULL_PATH. See Path Atom Type for the conditions under which an error **may** be returned when this function is given a valid XrInstance and a well-formed path string.

If the runtime's resources are exhausted and it cannot create the path, a return value of `XR_ERROR_PATH_COUNT_EXCEEDED` **must** be returned. If the application specifies a string that is not a well-formed path string, `XR_ERROR_PATH_FORMAT_INVALID` **must** be returned.



A return value of `XR_SUCCESS` from xrStringToPath **may** not necessarily imply that the runtime has a component or other source of data that will be accessible through that semantic path. It only means that the path string supplied was well-formed and that the retrieved `XrPath` maps to the given path string within and during the lifetime of the XrInstance given.

Valid Usage (Implicit)

- `instance` **must** be a valid XrInstance handle
- `pathString` **must** be a null-terminated UTF-8 string
- `path` **must** be a pointer to an `XrPath` value

Return Codes

Success

- XR_SUCCESS

Failure

- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_PATH_COUNT_EXCEEDED
- XR_ERROR_PATH_FORMAT_INVALID
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_VALIDATION_FAILURE

6.2.2. xrPathToString

```
// Provided by XR_VERSION_1_0
XrResult xrPathToString(
    XrInstance                                  instance,
    XrPath                                      path,
    uint32_t                                    bufferCapacityInput,
    uint32_t*                                   bufferCountOutput,
    char*                                       buffer);
```

Parameter Descriptions

- instance is an instance previously created.
- path is the valid XrPath value to retrieve the path string for.
- bufferCapacityInput is the capacity of the buffer, or 0 to indicate a request to retrieve the required capacity.
- bufferCountOutput is a pointer to the count of characters written to buffer (including the terminating '\0'), or a pointer to the required capacity in the case that bufferCapacityInput is insufficient.
- buffer is a pointer to an application-allocated buffer that will be filled with the semantic path string. It **can** be NULL if bufferCapacityInput is 0.
- See the Buffer Size Parameters section for a detailed description of retrieving the required buffer size.

xrPathToString retrieves the path name string associated with an XrPath, in the context of a given XrInstance, in the form of a NULL terminated string placed into a *caller-allocated* buffer. Since the mapping between a well-formed path name string and an XrPath is bijective, there will always be exactly one string for each valid XrPath value. This can be useful if the calling application receives an XrPath value that they had not previously retrieved via xrStringToPath. During the lifetime of the given XrInstance, the path name string retrieved by this function for a given valid XrPath will not change. For invalid paths, including XR_NULL_PATH, XR_ERROR_PATH_INVALID **must** be returned.

Valid Usage (Implicit)

- instance **must** be a valid XrInstance handle
- bufferCountOutput **must** be a pointer to a uint32_t value
- If bufferCapacityInput is not 0, buffer **must** be a pointer to an array of bufferCapacityInput char values

Return Codes

Success

- XR_SUCCESS

Failure

- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_PATH_INVALID
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SIZE_INSUFFICIENT
- XR_ERROR_VALIDATION_FAILURE

6.3. Reserved Paths

In order for some uses of semantic paths to work consistently across runtimes, it is necessary to standardize several paths and require each runtime to use the same paths or patterns of paths for certain classes of usage. Those paths are as follows.

6.3.1. Top level /user paths

Some paths are used to refer to entities that are filling semantic roles in the system. These paths are all under the */user* subtree.

The reserved user paths are:

Reserved Semantic Paths

- */user/hand/left* represents the user's left hand. It might be tracked using a controller or other device in the user's left hand, or tracked without the user holding anything, e.g. using computer vision.
- */user/hand/right* represents the user's right hand in analog to the left hand.
- */user/head* represents inputs on the user's head, often from a device such as a head-mounted display. To reason about the user's head, see the `XR_REFERENCE_SPACE_TYPE_VIEW` reference space.
- */user/gamepad* is a two-handed gamepad device held by the user.
- */user/treadmill* is a treadmill or other locomotion-targeted input device.

Runtimes are not required to provide interaction at all of these paths. For instance, in a system with no hand tracking, only */user/head* would be active for interaction. In a system with only one controller, the runtime **may** provide access to that controller via either */user/hand/left* or */user/hand/right* as it deems appropriate.

The runtime **may** change the devices referred to by */user/hand/left* and */user/hand/right* at any time.

If more than two hand-held controllers or devices are active, the runtime **must** determine which two are accessible as */user/hand/left* and */user/hand/right*.

6.3.2. Input subpaths

Interaction profiles define paths for each component that can be bound to an action. This section describes the naming conventions for those input components. Runtimes **must** ignore input subpaths that use identifiers and component names that do not appear in this specification or otherwise do not follow the pattern specified below. Input subpaths further qualify top-level */user* paths to form binding paths. For this reason, they are often shown starting with … or omitting path components before `/input` or `/output` entirely. The input subpaths considered valid when combined with any given top-level */user* path vary by interaction profile.

Each input subpath **must** match the following pattern:

- …*/input*/<identifier>[_<location>][/<component>]

Identifiers are often the label on the component or related to the type and location of the component.

When specifying a suggested binding there are several cases where the component part of the path can be determined automatically. See Suggested Bindings for more details.

See Interaction Profiles for examples of input subpaths.

Standard identifiers

- trackpad - A 2D input source that usually includes click and touch component.
- thumbstick - A small 2D joystick that is meant to be used with the user's thumb. These sometimes include click and/or touch components.
- joystick - A 2D joystick that is meant to be used with the user's entire hand, such as a flight stick. These generally do not have click component, but might have touch components.
- trigger - A 1D analog input component that returns to a rest state when the user stops interacting with it. These sometime include touch and/or click components.
- throttle - A 1D analog input component that remains in position when the user stops interacting with it.
- trackball - A 2D relative input source. These sometimes include click components.
- pedal - A 1D analog input component that is similar to a trigger but meant to be operated by a foot
- system - A button with the specialised meaning that it enables the user to access system-level functions and UI. Input data from system buttons is generally used internally by runtimes and **may** not be available to applications.
- dpad_up, dpad_down, dpad_left, and dpad_right - A set of buttons arranged in a plus shape.
- diamond_up, diamond_down, diamond_left, and diamond_right - Gamepads often have a set of four buttons arranged in a diamond shape. The labels on those buttons vary from gamepad to gamepad, but their arrangement is consistent. These names are used for the A/B/X/Y buttons on a Xbox controller, and the square/cross/circle/triangle button on a PlayStation controller.
- a, b, x, y, start, home, end, select - Standalone buttons are named for their physical labels. These are the standard identifiers for such buttons. Extensions **may** add new identifiers as detailed in the next section. Groups of four buttons in a diamond shape **should** use the diamond-prefix names above instead of using the labels on the buttons themselves.
- volume_up, volume_down, mute_mic, play_pause, menu, view, back - Some other standard controls are often identified by icons. These are their standard names.
- thumbrest - Some controllers have a place for the user to rest their thumb.
- shoulder - A button that is usually pressed with the index finger and is often positioned above a trigger.
- squeeze - An input source that indicates that the user is squeezing their fist closed. This could be a simple button or act more like a trigger. Sources with this identifier **should** either follow button or trigger conventions for their components.
- wheel - A steering wheel.
- thumb_resting_surfaces - Any surfaces that a thumb may naturally rest on. This may include, but is not limited to, face buttons, thumbstick, and thumbrest (Provided by `XR_VERSION_1_1`)
- stylus - Tip that can be used for writing or drawing. May be able to detect various pressure levels (Provided by `XR_VERSION_1_1`)

- trigger_curl - This sensor detects how pointed or curled the user's finger is on the trigger: 0 = fully pointed, 1 = finger flat on surface (Provided by `XR_VERSION_1_1`)
- trigger_slide - This sensor represents how far the user is sliding their index finger along the surface of the trigger: 0 = finger flat on the surface, 1 = finger fully drawn back (Provided by `XR_VERSION_1_1`)

Standard pose identifiers

Input sources whose orientation and/or position are tracked also expose pose identifiers.

Standard pose identifiers for tracked hands or motion controllers as represented by */user/hand/left* and */user/hand/right* are:

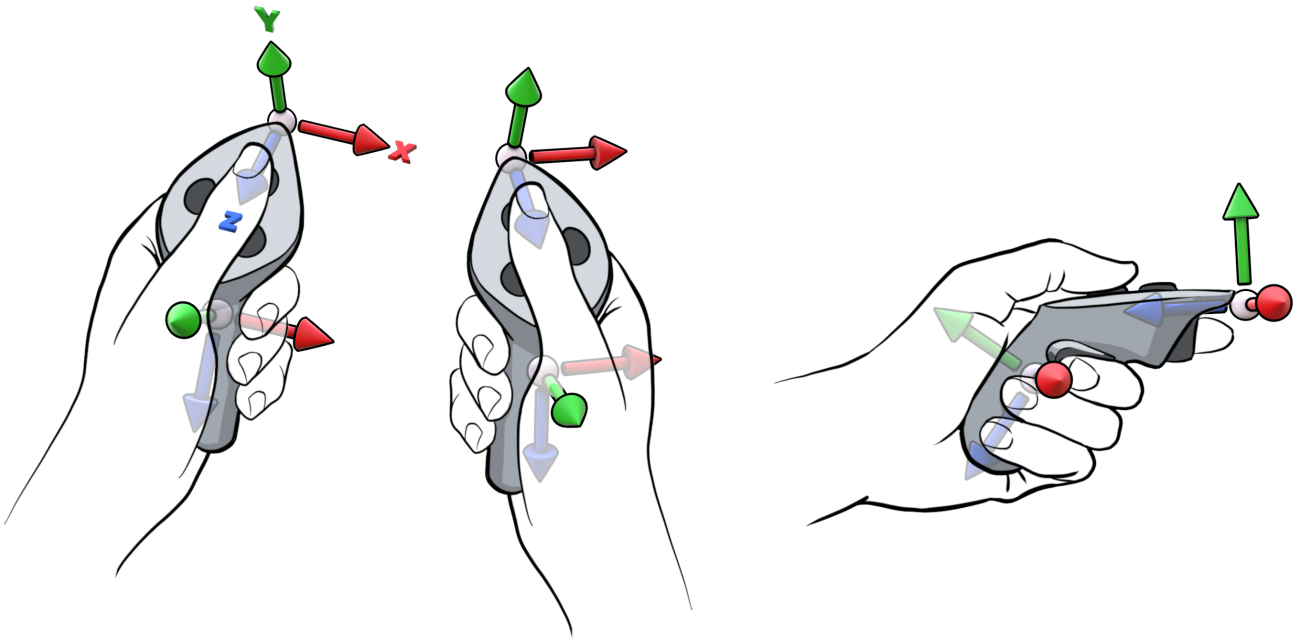


Figure 2. Example grip and aim poses for generic motion controllers

- grip - A pose that allows applications to reliably render a virtual object held in the user's hand, whether it is tracked directly or by a motion controller. The grip pose is defined as follows:
  - The grip position:
    - For tracked hands: The user's palm centroid when closing the fist, at the surface of the palm.
    - For handheld motion controllers: A fixed position within the controller that generally lines up with the palm centroid when held by a hand in a neutral position. This position should be adjusted left or right to center the position within the controller's grip.
  - The grip orientation's +X axis: When you completely open your hand to form a flat 5-finger pose, the ray that is normal to the user's palm (away from the palm in the left hand, into the palm in the right hand).

    - The grip orientation's -Z axis: When you close your hand partially (as if holding the controller), the ray that goes through the center of the tube formed by your non-thumb fingers, in the direction of little finger to thumb.
    - The grip orientation's +Y axis: orthogonal to +Z and +X using the right-hand rule.

- aim - A pose that allows applications to point in the world using the input source, according to the platform's conventions for aiming with that kind of source. The aim pose is defined as follows:
    - For tracked hands: The ray that follows platform conventions for how the user aims at objects in the world with their entire hand, with +Y up, +X to the right, and -Z forward. The ray chosen will be runtime-dependent, often a ray emerging from the hand at a target pointed by moving the forearm.
    - For handheld motion controllers: The ray that follows platform conventions for how the user targets objects in the world with the motion controller, with +Y up, +X to the right, and -Z forward. This is usually for applications that are rendering a model matching the physical controller, as an application rendering a virtual object in the user's hand likely prefers to point based on the geometry of that virtual object. The ray chosen will be runtime-dependent, although this will often emerge from the frontmost tip of a motion controller.

- grip_surface - (Provided by `XR_VERSION_1_1`) A pose that allows applications to reliably anchor visual content relative to the user's physical hand, whether the user's hand is tracked directly or its position and orientation is inferred by a physical controller. The grip_surface pose is defined as follows:
    - The grip_surface position: The user's physical palm centroid, at the surface of the palm. For the avoidance of doubt, the palm does not include fingers.
    - The grip_surface orientation's +X axis: When a user is holding the controller and straightens their index fingers pointing forward, the ray that is normal (perpendicular) to the user's palm (away from the palm in the left hand, into the palm in the right hand).
    - The grip_surface orientation's -Z axis: When a user is holding the controller and straightens their index finger, the ray that is parallel to their finger's pointing direction.
    - The grip_surface orientation's +Y axis: orthogonal to +Z and +X using the right-hand rule.

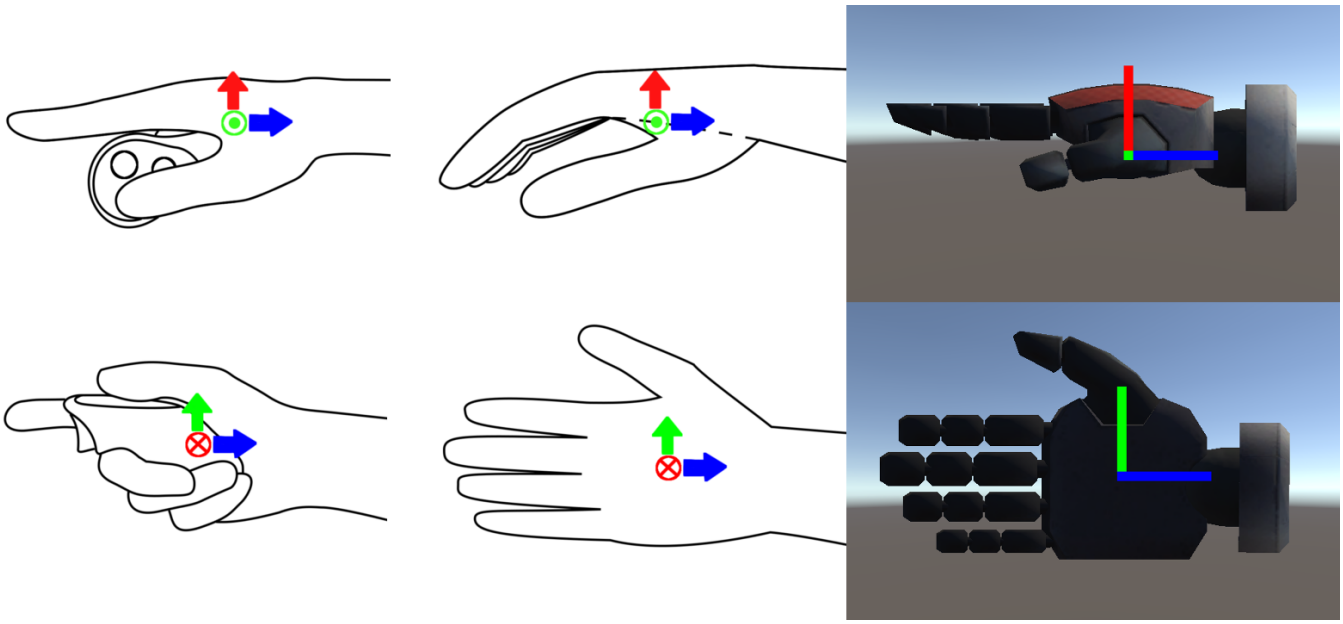


Figure 3. Example grip_surface pose for (from left to right) a generic motion controller, tracked hand, and a digital hand avatar. The X axis is depicted in red. The Y axis is depicted in green. The Z axis is depicted in blue.



Note

When the `XR_EXT_palm_pose` extension is available and enabled, an additional "palm_ext" standard pose identifier is available, and a path is added to all interaction profiles valid for */user/hand/left* or */user/hand/right*. This includes interaction profiles defined in the core spec and in extensions.



Note

When the `XR_EXT_hand_interaction` extension is available and enabled, additional "pinch_ext" and "poke_ext" standard pose identifiers are available, and a path is added to all interaction profiles valid for */user/hand/left* or */user/hand/right*. This includes interaction profiles defined in the core spec and in extensions.

Standard locations

When a single device contains multiple input sources that use the same identifier, a location suffix is added to create a unique identifier for that input source.

Standard locations are:

- left
- right
- left_upper
- left_lower
- right_upper
- right_lower

- upper
- lower

Standard components

Components are named for the specific boolean, scalar, or other value of the input source. Standard components are:

- click - A physical switch has been pressed by the user. This is valid for all buttons, and is common for trackpads, thumbsticks, triggers, and dpads. "click" components are always boolean.
- touch - The user has touched the input source. This is valid for all trackpads, and **may** be present for any other kind of input source if the device includes the necessary sensor. "touch" components are always boolean.
- force - A 1D scalar value that represents the user applying force to the input. It varies from 0 to 1, with 0 being the rest state. This is present for any input source with a force sensor.
- value - A 1D scalar value that varies from 0 to 1, with 0 being the rest state. This is present for triggers, throttles, and pedals. It **may** also be present for squeeze or other components.
- x, y - scalar components of 2D values. These vary in value from -1 to 1. These represent the 2D position of the input source with 0 being the rest state on each axis. -1 means all the way left for x axis or all the way down for y axis. +1 means all the way right for x axis or all the way up for y axis. x and y components are present for trackpads, thumbsticks, and joysticks.
- twist - Some sources, such as flight sticks, have a sensor that allows the user to twist the input left or right. For this component -1 means all the way left and 1 means all the way right.
- pose - The orientation and/or position of this input source. This component **may** exist for dedicated pose identifiers like grip and aim, or **may** be defined on other identifiers such as trackpad to let applications reason about the surface of that part.
- proximity - The user is in physical proximity of input source. This **may** be present for any kind of input source representing a physical component, such as a button, if the device includes the necessary sensor. The state of a "proximity" component **must** be `XR_TRUE` if the same input source is returning `XR_TRUE` for either a "touch" or any other component that implies physical contact. The runtime **may** return `XR_TRUE` for "proximity" when "touch" returns `XR_FALSE` which would indicate that the user is hovering just above, but not touching the input source in question. "proximity" components are always boolean. (Provided by `XR_VERSION_1_1`)

Output paths

Many devices also have subpaths for output features such as haptics. The runtime **must** ignore output component paths that do not follow the pattern:

- .../*output*/<output_identifier>[_<location>]

Standard output identifiers are:

- haptic - A haptic element like an LRA (Linear Resonant Actuator) or vibration motor
- haptic_trigger - A haptic element located in the trigger (Provided by XR_VERSION_1_1)
- haptic_thumb - A haptic element located in the resting place of the thumb, like under the touchpad (Provided by XR_VERSION_1_1)

Devices which contain multiple haptic elements with the same output identifier must use a location suffix as specified above.

6.3.3. Adding input sources via extensions

Extensions **may** enable input source path identifiers, output source path identifiers, and component names that are not included in the core specification, subject to the following conditions:

- EXT extensions **must** include the _ext suffix on any identifier or component name. E.g. *.../input/newidentifier_ext/newcomponent_ext*
- Vendor extensions **must** include the vendor's tag as a suffix on any identifier or component name. E.g. *.../input/newidentifier_vendor/newcomponent_vendor* (where "vendor" is replaced with the vendor's actual extension tag.)
- Khronos (KHR) extensions **may** add undecorated identifier or component names.

These rules are in place to prevent extensions from adding first class undecorated names that become defacto standards. Runtimes **must** ignore input source paths that do not follow the restrictions above.

Extensions **may** also add new location suffixes, and **may** do so by adding a new identifier and location combination using the appropriate suffix. E.g. *.../input/newidentifier_newlocation_ext*

6.4. Interaction Profile Paths

An interaction profile path identifies a collection of buttons and other input sources in a physical arrangement to allow applications and runtimes to coordinate action bindings.

Interaction profile paths are of the form:

- */interaction_profiles/*<vendor_name>/<type_name>



Note

When the XR_EXT_palm_pose extension is available and enabled, an additional input component path is added to all core interaction profiles valid for */user/hand/left* or */user/hand/right*. See the extension for more details.

6.4.1. Khronos Simple Controller Profile

Path: */interaction_profiles/khr/simple_controller*

Valid for user paths:

- */user/hand/left*
- */user/hand/right*

This interaction profile provides basic pose, button, and haptic support for applications with simple input needs. There is no hardware associated with the profile, and runtimes which support this profile **should** map the input paths provided to whatever the appropriate paths are on the actual hardware.

Supported component paths:

- *.../input/select/click*
- *.../input/menu/click*
- *.../input/grip/pose*
- *.../input/aim/pose*
- *.../output/haptic*



Note

When the runtime supports `XR_VERSION_1_1` and use of OpenXR 1.1 is requested by the application, this interaction profile **must** also support

- *.../input/grip_surface/pose*



Note

When the `XR_KHR_maintenance1` extension is available and enabled, this interaction profile **must** also support

- *.../input/grip_surface/pose*



Note

When the `XR_EXT_palm_pose` extension is available and enabled, this interaction profile **must** also support

- *.../input/palm_ext/pose*



Note

When the `XR_EXT_hand_interaction` extension is available and enabled, this interaction profile **must** also support

- *.../input/pinch_ext/pose*
- *.../input/poke_ext/pose*

6.4.2. ByteDance PICO Neo 3 controller Profile

(Provided by `XR_VERSION_1_1`)

Path: */interaction_profiles/bytedance/pico_neo3_controller*

Valid for user paths:

- */user/hand/left*
- */user/hand/right*

This interaction profile represents the input sources and haptics on the ByteDance PICO Neo3 Controller.

- On /user/hand/left only:
  - *.../input/x/click*
  - *.../input/x/touch*
  - *.../input/y/click*
  - *.../input/y/touch*
- On /user/hand/right only:
  - *.../input/a/click*
  - *.../input/a/touch*
  - *.../input/b/click*
  - *.../input/b/touch*
- *.../input/menu/click*
- *.../input/system/click* (**may** not be available for application use)
- *.../input/trigger/click*
- *.../input/trigger/value*
- *.../input/trigger/touch*
- *.../input/thumbstick/y*
- *.../input/thumbstick/x*
- *.../input/thumbstick/click*
- *.../input/thumbstick/touch*
- *.../input/squeeze/click*
- *.../input/squeeze/value*
- *.../input/grip/pose*
- *.../input/aim/pose*

- *.../output/haptic*



Note

When the runtime supports `XR_VERSION_1_1` and use of OpenXR 1.1 is requested by the application, this interaction profile **must** also support

- *.../input/grip_surface/pose*



Note

When the `XR_KHR_maintenance1` extension is available and enabled, this interaction profile **must** also support

- *.../input/grip_surface/pose*



Note

When the `XR_EXT_palm_pose` extension is available and enabled, this interaction profile **must** also support

- *.../input/palm_ext/pose*



Note

When the `XR_EXT_hand_interaction` extension is available and enabled, this interaction profile **must** also support

- *.../input/pinch_ext/pose*
- *.../input/poke_ext/pose*

6.4.3. ByteDance PICO 4 controller Profile

(Provided by `XR_VERSION_1_1`)

Path: */interaction_profiles/bytedance/pico4_controller*

Valid for user paths:

- */user/hand/left*
- */user/hand/right*

This interaction profile represents the input sources and haptics on the ByteDance PICO 4 Controller.

- On /user/hand/left only:
  - *.../input/x/click*
  - *.../input/x/touch*

    - *.../input/y/click*
    - *.../input/y/touch*
    - *.../input/menu/click*
- On /user/hand/right only:
    - *.../input/a/click*
    - *.../input/a/touch*
    - *.../input/b/click*
    - *.../input/b/touch*
- *.../input/system/click* (**may** not be available for application use)
- *.../input/trigger/click*
- *.../input/trigger/value*
- *.../input/trigger/touch*
- *.../input/thumbstick/y*
- *.../input/thumbstick/x*
- *.../input/thumbstick/click*
- *.../input/thumbstick/touch*
- *.../input/squeeze/click*
- *.../input/squeeze/value*
- *.../input/grip/pose*
- *.../input/aim/pose*
- *.../output/haptic*



Note

When the runtime supports `XR_VERSION_1_1` and use of OpenXR 1.1 is requested by the application, this interaction profile **must** also support

- *.../input/grip_surface/pose*



Note

When the `XR_KHR_maintenance1` extension is available and enabled, this interaction profile **must** also support

- *.../input/grip_surface/pose*



Note

When the XR_EXT_palm_pose extension is available and enabled, this interaction profile **must** also support

- *.../input/palm_ext/pose*



Note

When the XR_EXT_hand_interaction extension is available and enabled, this interaction profile **must** also support

- *.../input/pinch_ext/pose*
- *.../input/poke_ext/pose*

6.4.4. ByteDance PICO G3 controller Profile

(Provided by XR_VERSION_1_1)

Path: */interaction_profiles/bytedance/pico_g3_controller*

Valid for user paths:

- */user/hand/left*
- */user/hand/right*

This interaction profile represents the input sources and haptics on the ByteDance PICO G3 Controller.

- *.../input/trigger/click*
- *.../input/trigger/value*
- *.../input/menu/click*
- *.../input/grip/pose*
- *.../input/aim/pose*
- *.../input/thumbstick*
- *.../input/thumbstick/click*



Note

When the runtime supports XR_VERSION_1_1 and use of OpenXR 1.1 is requested by the application, this interaction profile **must** also support

- *.../input/grip_surface/pose*



Note

When the `XR_KHR_maintenance1` extension is available and enabled, this interaction profile **must** also support

- *.../input/grip_surface/pose*



Note

When the `XR_EXT_palm_pose` extension is available and enabled, this interaction profile **must** also support

- *.../input/palm_ext/pose*



Note

When the `XR_EXT_hand_interaction` extension is available and enabled, this interaction profile **must** also support

- *.../input/pinch_ext/pose*
- *.../input/poke_ext/pose*



Note

When designing suggested bindings for this interaction profile, you **may** suggest bindings for both */user/hand/left* and */user/hand/right*. However, only one of them will be active at a given time, so do not design interactions that require simultaneous use of both hands.

6.4.5. Google Daydream Controller Profile

Path: */interaction_profiles/google/daydream_controller*

Valid for user paths:

- */user/hand/left*
- */user/hand/right*

This interaction profile represents the input sources on the Google Daydream Controller.

Supported component paths:

- *.../input/select/click*
- *.../input/trackpad/x*
- *.../input/trackpad/y*
- *.../input/trackpad/click*

- *.../input/trackpad/touch*
- *.../input/grip/pose*
- *.../input/aim/pose*



Note

When the runtime supports `XR_VERSION_1_1` and use of OpenXR 1.1 is requested by the application, this interaction profile **must** also support

- *.../input/grip_surface/pose*



Note

When the `XR_KHR_maintenance1` extension is available and enabled, this interaction profile **must** also support

- *.../input/grip_surface/pose*



Note

When the `XR_EXT_palm_pose` extension is available and enabled, this interaction profile **must** also support

- *.../input/palm_ext/pose*



Note

When the `XR_EXT_hand_interaction` extension is available and enabled, this interaction profile **must** also support

- *.../input/pinch_ext/pose*
- *.../input/poke_ext/pose*

6.4.6. HP Mixed Reality Motion Controller Profile

(Provided by `XR_VERSION_1_1`)

Path: */interaction_profiles/hp/mixed_reality_controller*

Valid for user paths:

- */user/hand/left*
- */user/hand/right*

This interaction profile represents the input sources and haptics on the HP Mixed Reality Motion Controller.

- On */user/hand/left* only:
  - *.../input/x/click*
  - *.../input/y/click*
- On */user/hand/right* only:
  - *.../input/a/click*
  - *.../input/b/click*
- *.../input/menu/click*
- *.../input/squeeze/value*
- *.../input/trigger/value*
- *.../input/thumbstick/x*
- *.../input/thumbstick/y*
- *.../input/thumbstick/click*
- *.../input/grip/pose*
- *.../input/aim/pose*
- *.../output/haptic*



Note

When the runtime supports `XR_VERSION_1_1` and use of OpenXR 1.1 is requested by the application, this interaction profile **must** also support

- *.../input/grip_surface/pose*



Note

When the `XR_KHR_maintenance1` extension is available and enabled, this interaction profile **must** also support

- *.../input/grip_surface/pose*



Note

When the `XR_EXT_palm_pose` extension is available and enabled, this interaction profile **must** also support

- *.../input/palm_ext/pose*



Note

When the `XR_EXT_hand_interaction` extension is available and enabled, this interaction profile **must** also support

- *.../input/pinch_ext/pose*
- *.../input/poke_ext/pose*

6.4.7. HTC Vive Controller Profile

Path: */interaction_profiles/htc/vive_controller*

Valid for user paths:

- */user/hand/left*
- */user/hand/right*

This interaction profile represents the input sources and haptics on the Vive Controller.

Supported component paths:

- *.../input/system/click* (**may** not be available for application use)
- *.../input/squeeze/click*
- *.../input/menu/click*
- *.../input/trigger/click*
- *.../input/trigger/value*
- *.../input/trackpad/x*
- *.../input/trackpad/y*
- *.../input/trackpad/click*
- *.../input/trackpad/touch*
- *.../input/grip/pose*
- *.../input/aim/pose*
- *.../output/haptic*



Note

When the runtime supports `XR_VERSION_1_1` and use of OpenXR 1.1 is requested by the application, this interaction profile **must** also support

- *.../input/grip_surface/pose*



Note

When the `XR_KHR_maintenance1` extension is available and enabled, this interaction profile **must** also support

- *.../input/grip_surface/pose*



Note

When the `XR_EXT_palm_pose` extension is available and enabled, this interaction profile **must** also support

- *.../input/palm_ext/pose*



Note

When the `XR_EXT_hand_interaction` extension is available and enabled, this interaction profile **must** also support

- *.../input/pinch_ext/pose*
- *.../input/poke_ext/pose*

6.4.8. HTC Vive Cosmos Controller Profile

(Provided by `XR_VERSION_1_1`)

Path: */interaction_profiles/htc/vive_cosmos_controller*

Valid for user paths:

- */user/hand/left*
- */user/hand/right*

This interaction profile represents the input sources and haptics on the Vive Cosmos Controller.

Supported component paths:

- On */user/hand/left* only:
  - *.../input/x/click*
  - *.../input/y/click*
  - *.../input/menu/click*
- On */user/hand/right* only:
  - *.../input/a/click*
  - *.../input/b/click*
  - *.../input/system/click* (**may** not be available for application use)

- *.../input/shoulder/click*
- *.../input/squeeze/click*
- *.../input/trigger/click*
- *.../input/trigger/value*
- *.../input/thumbstick/x*
- *.../input/thumbstick/y*
- *.../input/thumbstick/click*
- *.../input/thumbstick/touch*
- *.../input/grip/pose*
- *.../input/aim/pose*
- *.../output/haptic*



Note

When the runtime supports `XR_VERSION_1_1` and use of OpenXR 1.1 is requested by the application, this interaction profile **must** also support

- *.../input/grip_surface/pose*



Note

When the `XR_KHR_maintenance1` extension is available and enabled, this interaction profile **must** also support

- *.../input/grip_surface/pose*



Note

When the `XR_EXT_palm_pose` extension is available and enabled, this interaction profile **must** also support

- *.../input/palm_ext/pose*



Note

When the `XR_EXT_hand_interaction` extension is available and enabled, this interaction profile **must** also support

- *.../input/pinch_ext/pose*
- *.../input/poke_ext/pose*

6.4.9. HTC Vive Focus 3 Controller Profile

(Provided by `XR_VERSION_1_1`)

Path: */interaction_profiles/htc/vive_focus3_controller*

Valid for user paths:

- */user/hand/left*
- */user/hand/right*

This interaction profile represents the input sources and haptics on the Vive Focus 3 Controller.

- On */user/hand/left* only:
  - *.../input/x/click*
  - *.../input/y/click*
  - *.../input/menu/click*
- On */user/hand/right* only:
  - *.../input/a/click*
  - *.../input/b/click*
  - *.../input/system/click* (**may** not be available for application use)
- *.../input/squeeze/click*
- *.../input/squeeze/touch*
- *.../input/squeeze/value*
- *.../input/trigger/click*
- *.../input/trigger/touch*
- *.../input/trigger/value*
- *.../input/thumbstick/x*
- *.../input/thumbstick/y*
- *.../input/thumbstick/click*
- *.../input/thumbstick/touch*
- *.../input/thumbrest/touch*
- *.../input/grip/pose*
- *.../input/aim/pose*
- *.../output/haptic*



Note

When the runtime supports `XR_VERSION_1_1` and use of OpenXR 1.1 is requested by the application, this interaction profile **must** also support

- *.../input/grip_surface/pose*



Note

When the `XR_KHR_maintenance1` extension is available and enabled, this interaction profile **must** also support

- *.../input/grip_surface/pose*



Note

When the `XR_EXT_palm_pose` extension is available and enabled, this interaction profile **must** also support

- *.../input/palm_ext/pose*



Note

When the `XR_EXT_hand_interaction` extension is available and enabled, this interaction profile **must** also support

- *.../input/pinch_ext/pose*
- *.../input/poke_ext/pose*

6.4.10. HTC Vive Pro Profile

Path: */interaction_profiles/htc/vive_pro*

Valid for user paths:

- */user/head*

This interaction profile represents the input sources on the Vive Pro headset.

Supported component paths:

- *.../input/system/click* (**may** not be available for application use)
- *.../input/volume_up/click*
- *.../input/volume_down/click*
- *.../input/mute_mic/click*

6.4.11. Magic Leap 2 Controller Profile

(Provided by `XR_VERSION_1_1`)

Path: */interaction_profiles/ml/ml2_controller*

Valid for user paths:

- */user/hand/left*
- */user/hand/right*

This interaction profile represents the input sources and haptics on the Magic Leap 2 controller.

Supported component paths:

- *.../input/menu/click*
- *.../input/home/click* (**may** not be available for application use)
- *.../input/trigger/click*
- *.../input/trigger/value*
- *.../input/trackpad/y*
- *.../input/trackpad/x*
- *.../input/trackpad/click*
- *.../input/trackpad/force*
- *.../input/trackpad/touch*
- *.../input/aim/pose*
- *.../input/grip/pose*
- *.../input/shoulder/click*
- *.../output/haptic*



Note

When the runtime supports `XR_VERSION_1_1` and use of OpenXR 1.1 is requested by the application, this interaction profile **must** also support

- *.../input/grip_surface/pose*



Note

When the `XR_KHR_maintenance1` extension is available and enabled, this interaction profile **must** also support

- *.../input/grip_surface/pose*



Note

When the `XR_EXT_palm_pose` extension is available and enabled, this interaction profile **must** also support

- *.../input/palm_ext/pose*



Note

When the `XR_EXT_hand_interaction` extension is available and enabled, this interaction profile **must** also support

- *.../input/pinch_ext/pose*
- *.../input/poke_ext/pose*

6.4.12. Microsoft Mixed Reality Motion Controller Profile

Path: */interaction_profiles/microsoft/motion_controller*

Valid for user paths:

- */user/hand/left*
- */user/hand/right*

This interaction profile represents the input sources and haptics on the Microsoft Mixed Reality Controller.

Supported component paths:

- *.../input/menu/click*
- *.../input/squeeze/click*
- *.../input/trigger/value*
- *.../input/thumbstick/x*
- *.../input/thumbstick/y*
- *.../input/thumbstick/click*
- *.../input/trackpad/x*
- *.../input/trackpad/y*
- *.../input/trackpad/click*
- *.../input/trackpad/touch*
- *.../input/grip/pose*
- *.../input/aim/pose*
- *.../output/haptic*



Note

When the runtime supports `XR_VERSION_1_1` and use of OpenXR 1.1 is requested by the application, this interaction profile **must** also support

- *.../input/grip_surface/pose*



Note

When the `XR_KHR_maintenance1` extension is available and enabled, this interaction profile **must** also support

- *.../input/grip_surface/pose*



Note

When the `XR_EXT_palm_pose` extension is available and enabled, this interaction profile **must** also support

- *.../input/palm_ext/pose*



Note

When the `XR_EXT_hand_interaction` extension is available and enabled, this interaction profile **must** also support

- *.../input/pinch_ext/pose*
- *.../input/poke_ext/pose*

6.4.13. Microsoft Xbox Controller Profile

Path: */interaction_profiles/microsoft/xbox_controller*

Valid for user paths:

- */user/gamepad*

This interaction profile represents the input sources and haptics on the Microsoft Xbox Controller.

Supported component paths:

- *.../input/menu/click*
- *.../input/view/click*
- *.../input/a/click*
- *.../input/b/click*
- *.../input/x/click*

- *.../input/y/click*
- *.../input/dpad_down/click*
- *.../input/dpad_right/click*
- *.../input/dpad_up/click*
- *.../input/dpad_left/click*
- *.../input/shoulder_left/click*
- *.../input/shoulder_right/click*
- *.../input/thumbstick_left/click*
- *.../input/thumbstick_right/click*
- *.../input/trigger_left/value*
- *.../input/trigger_right/value*
- *.../input/thumbstick_left/x*
- *.../input/thumbstick_left/y*
- *.../input/thumbstick_right/x*
- *.../input/thumbstick_right/y*
- *.../output/haptic_left*
- *.../output/haptic_right*
- *.../output/haptic_left_trigger*
- *.../output/haptic_right_trigger*

6.4.14. Oculus Go Controller Profile

Path: */interaction_profiles/oculus/go_controller*

Valid for user paths:

- */user/hand/left*
- */user/hand/right*

This interaction profile represents the input sources on the Oculus Go controller.

Supported component paths:

- *.../input/system/click* (**may** not be available for application use)
- *.../input/trigger/click*
- *.../input/back/click*
- *.../input/trackpad/x*

- *.../input/trackpad/y*
- *.../input/trackpad/click*
- *.../input/trackpad/touch*
- *.../input/grip/pose*
- *.../input/aim/pose*



Note

When the runtime supports `XR_VERSION_1_1` and use of OpenXR 1.1 is requested by the application, this interaction profile **must** also support

- *.../input/grip_surface/pose*



Note

When the `XR_KHR_maintenance1` extension is available and enabled, this interaction profile **must** also support

- *.../input/grip_surface/pose*



Note

When the `XR_EXT_palm_pose` extension is available and enabled, this interaction profile **must** also support

- *.../input/palm_ext/pose*



Note

When the `XR_EXT_hand_interaction` extension is available and enabled, this interaction profile **must** also support

- *.../input/pinch_ext/pose*
- *.../input/poke_ext/pose*

6.4.15. Oculus Touch Controller Profile

Path: */interaction_profiles/oculus/touch_controller*

Valid for user paths:

- */user/hand/left*
- */user/hand/right*

This interaction profile represents the input sources and haptics on the Oculus Touch controller.

Supported component paths:

- On */user/hand/left* only:
  - *.../input/x/click*
  - *.../input/x/touch*
  - *.../input/y/click*
  - *.../input/y/touch*
  - *.../input/menu/click*
- On */user/hand/right* only:
  - *.../input/a/click*
  - *.../input/a/touch*
  - *.../input/b/click*
  - *.../input/b/touch*
  - *.../input/system/click* (**may** not be available for application use)
- *.../input/squeeze/value*
- *.../input/trigger/value*
- *.../input/trigger/touch*
- *.../input/trigger/proximity* (Provided by `XR_VERSION_1_1`)
- *.../input/thumb_resting_surfaces/proximity* (Provided by `XR_VERSION_1_1`)
- *.../input/thumbstick/x*
- *.../input/thumbstick/y*
- *.../input/thumbstick/click*
- *.../input/thumbstick/touch*
- *.../input/thumbrest/touch*
- *.../input/grip/pose*
- *.../input/aim/pose*
- *.../output/haptic*



Note

When the runtime supports `XR_VERSION_1_1` and use of OpenXR 1.1 is requested by the application, this interaction profile **must** also support

- *.../input/grip_surface/pose*



Note

When the `XR_KHR_maintenance1` extension is available and enabled, this interaction profile **must** also support

- *.../input/grip_surface/pose*



Note

When the `XR_EXT_palm_pose` extension is available and enabled, this interaction profile **must** also support

- *.../input/palm_ext/pose*



Note

When the `XR_EXT_hand_interaction` extension is available and enabled, this interaction profile **must** also support

- *.../input/pinch_ext/pose*
- *.../input/poke_ext/pose*

6.4.16. Meta Touch Pro Controller Profile

(Provided by `XR_VERSION_1_1`)

Path: */interaction_profiles/meta/touch_pro_controller*

Valid for user paths:

- */user/hand/left*
- */user/hand/right*

This interaction profile represents the input sources and haptics on the Meta Touch Pro controller.

Supported component paths:

- On */user/hand/left* only:
  - *.../input/x/click*
  - *.../input/x/touch*
  - *.../input/y/click*
  - *.../input/y/touch*
  - *.../input/menu/click*
- On */user/hand/right* only:
  - *.../input/a/click*

    - *.../input/a/touch*
    - *.../input/b/click*
    - *.../input/b/touch*
    - *.../input/system/click* (**may** not be available for application use)
- *.../input/squeeze/value*
- *.../input/trigger/value*
- *.../input/trigger/touch*
- *.../input/trigger/proximity*
- *.../input/trigger_curl/value*
- *.../input/trigger_slide/value*
- *.../input/thumb_resting_surfaces/proximity*
- *.../input/thumbstick/x*
- *.../input/thumbstick/y*
- *.../input/thumbstick/click*
- *.../input/thumbstick/touch*
- *.../input/thumbrest/touch*
- *.../input/thumbrest/force*
- *.../input/stylus/force*
- *.../input/grip/pose*
- *.../input/aim/pose*
- *.../output/haptic*
- *.../output/haptic_trigger*
- *.../output/haptic_thumb*



Note

When the runtime supports `XR_VERSION_1_1` and use of OpenXR 1.1 is requested by the application, this interaction profile **must** also support

- *.../input/grip_surface/pose*



Note

When the `XR_KHR_maintenance1` extension is available and enabled, this interaction profile **must** also support

- *.../input/grip_surface/pose*



Note

When the XR_EXT_palm_pose extension is available and enabled, this interaction profile **must** also support

- *.../input/palm_ext/pose*



Note

When the XR_EXT_hand_interaction extension is available and enabled, this interaction profile **must** also support

- *.../input/pinch_ext/pose*
- *.../input/poke_ext/pose*

6.4.17. Meta Touch Plus Controller Profile

(Provided by XR_VERSION_1_1)

Path: */interaction_profiles/meta/touch_plus_controller*

Valid for user paths:

- */user/hand/left*
- */user/hand/right*

This interaction profile represents the input sources and haptics on the Meta Touch Plus controller.

Supported component paths:

- On */user/hand/left* only:
  - *.../input/x/click*
  - *.../input/x/touch*
  - *.../input/y/click*
  - *.../input/y/touch*
  - *.../input/menu/click*
- On */user/hand/right* only:
  - *.../input/a/click*
  - *.../input/a/touch*
  - *.../input/b/click*
  - *.../input/b/touch*
  - *.../input/system/click* (**may** not be available for application use)

- *.../input/squeeze/value*
- *.../input/trigger/value*
- *.../input/trigger/touch*
- *.../input/trigger/force*
- *.../input/trigger/proximity*
- *.../input/trigger_curl/value*
- *.../input/trigger_slide/value*
- *.../input/thumb_resting_surfaces/proximity*
- *.../input/thumbstick/x*
- *.../input/thumbstick/y*
- *.../input/thumbstick/click*
- *.../input/thumbstick/touch*
- *.../input/thumbrest/touch*
- *.../input/grip/pose*
- *.../input/aim/pose*
- *.../output/haptic*



Note

When the runtime supports `XR_VERSION_1_1` and use of OpenXR 1.1 is requested by the application, this interaction profile **must** also support

- *.../input/grip_surface/pose*



Note

When the `XR_KHR_maintenance1` extension is available and enabled, this interaction profile **must** also support

- *.../input/grip_surface/pose*



Note

When the `XR_EXT_palm_pose` extension is available and enabled, this interaction profile **must** also support

- *.../input/palm_ext/pose*



Note

When the XR_EXT_hand_interaction extension is available and enabled, this interaction profile **must** also support

- *.../input/pinch_ext/pose*
- *.../input/poke_ext/pose*

6.4.18. Meta Touch Controller (Rift CV1) Profile

(Provided by XR_VERSION_1_1)

Path: */interaction_profiles/meta/touch_controller_rift_cv1*

Valid for user paths:

- */user/hand/left*
- */user/hand/right*

This interaction profile represents the input sources and haptics on the Oculus Touch controller and is a legacy profile added to specifically represent the controller shipped with the Rift CV1.

Supported component paths:

- On */user/hand/left* only:
  - *.../input/x/click*
  - *.../input/x/touch*
  - *.../input/y/click*
  - *.../input/y/touch*
  - *.../input/menu/click*
- On */user/hand/right* only:
  - *.../input/a/click*
  - *.../input/a/touch*
  - *.../input/b/click*
  - *.../input/b/touch*
  - *.../input/system/click* (**may** not be available for application use)
- *.../input/squeeze/value*
- *.../input/trigger/value*
- *.../input/trigger/touch*
- *.../input/trigger/proximity*

- *.../input/thumb_resting_surfaces/proximity*
- *.../input/thumbstick/x*
- *.../input/thumbstick/y*
- *.../input/thumbstick/click*
- *.../input/thumbstick/touch*
- *.../input/thumbrest/touch*
- *.../input/grip/pose*
- *.../input/aim/pose*
- *.../output/haptic*



Note

When the runtime supports `XR_VERSION_1_1` and use of OpenXR 1.1 is requested by the application, this interaction profile **must** also support

- *.../input/grip_surface/pose*



Note

When the `XR_KHR_maintenance1` extension is available and enabled, this interaction profile **must** also support

- *.../input/grip_surface/pose*



Note

When the `XR_EXT_palm_pose` extension is available and enabled, this interaction profile **must** also support

- *.../input/palm_ext/pose*



Note

When the `XR_EXT_hand_interaction` extension is available and enabled, this interaction profile **must** also support

- *.../input/pinch_ext/pose*
- *.../input/poke_ext/pose*

6.4.19. Meta Touch Controller (Rift S / Quest 1) Profile

(Provided by `XR_VERSION_1_1`)

Path: */interaction_profiles/meta/touch_controller_quest_1_rift_s*

Valid for user paths:

- */user/hand/left*
- */user/hand/right*

This interaction profile represents the input sources and haptics on the Oculus Touch controller and is a legacy profile added to specifically represent the controller shipped with the Rift S and Quest 1.

Supported component paths:

- On */user/hand/left* only:
  - *…/input/x/click*
  - *…/input/x/touch*
  - *…/input/y/click*
  - *…/input/y/touch*
  - *…/input/menu/click*
- On */user/hand/right* only:
  - *…/input/a/click*
  - *…/input/a/touch*
  - *…/input/b/click*
  - *…/input/b/touch*
  - *…/input/system/click* (**may** not be available for application use)
- *…/input/squeeze/value*
- *…/input/trigger/value*
- *…/input/trigger/touch*
- *…/input/trigger/proximity*
- *…/input/thumb_resting_surfaces/proximity*
- *…/input/thumbstick/x*
- *…/input/thumbstick/y*
- *…/input/thumbstick/click*
- *…/input/thumbstick/touch*
- *…/input/grip/pose*
- *…/input/aim/pose*
- *…/output/haptic*



Note

When the runtime supports `XR_VERSION_1_1` and use of OpenXR 1.1 is requested by the application, this interaction profile **must** also support

- *.../input/grip_surface/pose*



Note

When the `XR_KHR_maintenance1` extension is available and enabled, this interaction profile **must** also support

- *.../input/grip_surface/pose*



Note

When the `XR_EXT_palm_pose` extension is available and enabled, this interaction profile **must** also support

- *.../input/palm_ext/pose*



Note

When the `XR_EXT_hand_interaction` extension is available and enabled, this interaction profile **must** also support

- *.../input/pinch_ext/pose*
- *.../input/poke_ext/pose*

6.4.20. Meta Touch Controller (Quest 2) Profile

(Provided by `XR_VERSION_1_1`)

Path: */interaction_profiles/meta/touch_controller_quest_2*

Valid for user paths:

- */user/hand/left*
- */user/hand/right*

This interaction profile represents the input sources and haptics on the Oculus Touch controller and is a legacy profile added to specifically represent the controller shipped with the Quest 2.

Supported component paths:

- On */user/hand/left* only:
  - *.../input/x/click*

    - *.../input/x/touch*
    - *.../input/y/click*
    - *.../input/y/touch*
    - *.../input/menu/click*
- On */user/hand/right* only:
    - *.../input/a/click*
    - *.../input/a/touch*
    - *.../input/b/click*
    - *.../input/b/touch*
    - *.../input/system/click* (**may** not be available for application use)
- *.../input/squeeze/value*
- *.../input/trigger/value*
- *.../input/trigger/touch*
- *.../input/trigger/proximity*
- *.../input/thumb_resting_surfaces/proximity*
- *.../input/thumbstick/x*
- *.../input/thumbstick/y*
- *.../input/thumbstick/click*
- *.../input/thumbstick/touch*
- *.../input/thumbrest/touch*
- *.../input/grip/pose*
- *.../input/aim/pose*
- *.../output/haptic*



Note

When the runtime supports `XR_VERSION_1_1` and use of OpenXR 1.1 is requested by the application, this interaction profile **must** also support

- *.../input/grip_surface/pose*



Note

When the `XR_KHR_maintenance1` extension is available and enabled, this interaction profile **must** also support

- *.../input/grip_surface/pose*



Note

When the `XR_EXT_palm_pose` extension is available and enabled, this interaction profile **must** also support

- *.../input/palm_ext/pose*



Note

When the `XR_EXT_hand_interaction` extension is available and enabled, this interaction profile **must** also support

- *.../input/pinch_ext/pose*
- *.../input/poke_ext/pose*

6.4.21. Samsung Odyssey Controller Profile

(Provided by `XR_VERSION_1_1`)

Path: */interaction_profiles/samsung/odyssey_controller*

Valid for user paths:

- */user/hand/left*
- */user/hand/right*

This interaction profile represents the input sources and haptics on the Samsung Odyssey Controller. It is exactly the same, with the exception of the name of the interaction profile, as the Microsoft Mixed Reality Controller interaction profile. It enables the application to differentiate the newer form factor of motion controller released with the Samsung Odyssey headset. It enables the application to customize the appearance and experience of the controller differently from the original mixed reality motion controller.

Supported component paths:

- *.../input/menu/click*
- *.../input/squeeze/click*
- *.../input/trigger/value*
- *.../input/thumbstick/x*
- *.../input/thumbstick/y*
- *.../input/thumbstick/click*
- *.../input/trackpad/x*
- *.../input/trackpad/y*

- *.../input/trackpad/click*
- *.../input/trackpad/touch*
- *.../input/grip/pose*
- *.../input/aim/pose*
- *.../output/haptic*



Note

When the runtime supports `XR_VERSION_1_1` and use of OpenXR 1.1 is requested by the application, this interaction profile **must** also support

- *.../input/grip_surface/pose*



Note

When the `XR_KHR_maintenance1` extension is available and enabled, this interaction profile **must** also support

- *.../input/grip_surface/pose*



Note

When the `XR_EXT_palm_pose` extension is available and enabled, this interaction profile **must** also support

- *.../input/palm_ext/pose*



Note

When the `XR_EXT_hand_interaction` extension is available and enabled, this interaction profile **must** also support

- *.../input/pinch_ext/pose*
- *.../input/poke_ext/pose*

6.4.22. Valve Index Controller Profile

Path: */interaction_profiles/valve/index_controller*

Valid for user paths:

- */user/hand/left*
- */user/hand/right*

This interaction profile represents the input sources and haptics on the Valve Index controller.

Supported component paths:

- *.../input/system/click* (**may** not be available for application use)
- *.../input/system/touch* (**may** not be available for application use)
- *.../input/a/click*
- *.../input/a/touch*
- *.../input/b/click*
- *.../input/b/touch*
- *.../input/squeeze/value*
- *.../input/squeeze/force*
- *.../input/trigger/click*
- *.../input/trigger/value*
- *.../input/trigger/touch*
- *.../input/thumbstick/x*
- *.../input/thumbstick/y*
- *.../input/thumbstick/click*
- *.../input/thumbstick/touch*
- *.../input/trackpad/x*
- *.../input/trackpad/y*
- *.../input/trackpad/force*
- *.../input/trackpad/touch*
- *.../input/grip/pose*
- *.../input/aim/pose*
- *.../output/haptic*



Note

When the runtime supports `XR_VERSION_1_1` and use of OpenXR 1.1 is requested by the application, this interaction profile **must** also support

- *.../input/grip_surface/pose*



Note

When the `XR_KHR_maintenance1` extension is available and enabled, this interaction profile **must** also support

- *.../input/grip_surface/pose*



Note

When the `XR_EXT_palm_pose` extension is available and enabled, this interaction profile **must** also support

- *.../input/palm_ext/pose*



Note

When the `XR_EXT_hand_interaction` extension is available and enabled, this interaction profile **must** also support

- *.../input/pinch_ext/pose*
- *.../input/poke_ext/pose*

Chapter 7. Spaces

Across both virtual reality and augmented reality, XR applications have a core need to map the location of virtual objects to the corresponding real-world locations where they will be rendered. **Spaces** allow applications to explicitly create and specify the frames of reference in which they choose to track the real world, and then determine how those frames of reference move relative to one another over time.

```
// Provided by XR_VERSION_1_0
XR_DEFINE_HANDLE(XrSpace)
```

Spaces are represented by XrSpace handles, which the application creates and then uses in API calls. Whenever an application calls a function that returns coordinates, it provides an XrSpace to specify the frame of reference in which those coordinates will be expressed. Similarly, when providing coordinates to a function, the application specifies which XrSpace the runtime should use to interpret those coordinates.

OpenXR defines a set of well-known **reference spaces** that applications use to bootstrap their spatial reasoning. These reference spaces are: VIEW, LOCAL, LOCAL_FLOOR, and STAGE. Each reference space has a well-defined meaning, which establishes where its origin is positioned and how its axes are oriented.

Runtimes whose tracking systems improve their understanding of the world over time **may** track spaces independently. For example, even though a LOCAL space and a STAGE space each map their origin to a static position in the world, a runtime with an inside-out tracking system **may** introduce slight adjustments to the origin of each space on a continuous basis to keep each origin in place.

Beyond well-known reference spaces, runtimes expose other independently-tracked spaces, such as a pose action space that tracks the pose of a motion controller over time.

When one or both spaces are tracking a dynamic object, passing in an updated time to xrLocateSpace each frame will result in an updated relative pose. For example, the location of the left hand's pose action space in the STAGE reference space will change each frame as the user's hand moves relative to the stage's predefined origin on the floor. In other XR APIs, it is common to report the "pose" of an object relative to some presumed underlying global space. This API is careful to not explicitly define such an underlying global space, because it does not apply to all systems. Some systems will support no STAGE space, while others may support a STAGE space that switches between various physical stages with dynamic availability. To satisfy this wide variability, "poses" are always described as the relationship between two spaces.

Some devices improve their understanding of the world as the device is used. The location returned by xrLocateSpace in later frames **may** change over time, even for spaces that track static objects, as either the target space or base space adjusts its origin.

Composition layers submitted by the application include an XrSpace for the runtime to use to position that layer over time. Composition layers whose XrSpace is relative to the VIEW reference space are

implicitly "head-locked", even if they may not be "display-locked" for non-head-mounted form factors.

7.1. Reference Spaces

The XrReferenceSpaceType enumeration is defined as:

```
// Provided by XR_VERSION_1_0
typedef enum XrReferenceSpaceType {
    XR_REFERENCE_SPACE_TYPE_VIEW = 1,
    XR_REFERENCE_SPACE_TYPE_LOCAL = 2,
    XR_REFERENCE_SPACE_TYPE_STAGE = 3,
  // Provided by XR_VERSION_1_1
    XR_REFERENCE_SPACE_TYPE_LOCAL_FLOOR = 1000426000,
  // Provided by XR_MSFT_unbounded_reference_space
    XR_REFERENCE_SPACE_TYPE_UNBOUNDED_MSFT = 1000038000,
  // Provided by XR_VARJO_foveated_rendering
    XR_REFERENCE_SPACE_TYPE_COMBINED_EYE_VARJO = 1000121000,
  // Provided by XR_ML_localization_map
    XR_REFERENCE_SPACE_TYPE_LOCALIZATION_MAP_ML = 1000139000,
  // Provided by XR_ANDROID_unbounded_reference_space
    XR_REFERENCE_SPACE_TYPE_UNBOUNDED_ANDROID = 1000467000,
  // Provided by XR_EXT_stationary_reference_space
    XR_REFERENCE_SPACE_TYPE_STATIONARY_EXT = 1000742000,
  // Provided by XR_EXT_local_floor
    XR_REFERENCE_SPACE_TYPE_LOCAL_FLOOR_EXT = XR_REFERENCE_SPACE_TYPE_LOCAL_FLOOR,
    XR_REFERENCE_SPACE_TYPE_MAX_ENUM = 0x7FFFFFFF
} XrReferenceSpaceType;
```

Brief introductions to core reference space types follow. Each has full requirements in a subsequent section, linked from these descriptions.

Enumerant Descriptions

- XR_REFERENCE_SPACE_TYPE_VIEW. The VIEW reference space tracks the view origin used to generate view transforms for the primary viewer (or centroid of view origins if stereo), with +Y up, +X to the right, and -Z forward. This space points in the forward direction for the viewer without incorporating the user's eye orientation, and is not gravity-aligned.

  Runtimes **must** support VIEW reference space.

- XR_REFERENCE_SPACE_TYPE_LOCAL. The LOCAL reference space establishes a world-locked origin, gravity-aligned to exclude pitch and roll, with +Y up, +X to the right, and -Z forward. This space locks in both its initial position and orientation, which the runtime **may** define to be either the initial position at application launch or some other calibrated zero position.

  Runtimes **must** support LOCAL reference space.

- XR_REFERENCE_SPACE_TYPE_STAGE. The STAGE reference space is a runtime-defined flat, rectangular space that is empty and can be walked around on. The origin is on the floor at the center of the rectangle, with +Y up, and the X and Z axes aligned with the rectangle edges. The runtime **may** not be able to locate spaces relative to the STAGE reference space if the user has not yet defined one within the runtime-specific UI. Applications **can** use xrGetReferenceSpaceBoundsRect to determine the extents of the STAGE reference space's XZ bounds rectangle, if defined.

  Support for the STAGE reference space is **optional**.

- XR_REFERENCE_SPACE_TYPE_LOCAL_FLOOR (provided by XR_VERSION_1_1) Similar to LOCAL space, the LOCAL_FLOOR reference space establishes a world-locked origin, gravity-aligned to exclude pitch and roll, with +Y up, +X to the right, and -Z forward. However, the origin of this space is defined to be on an estimate of the floor level.

  Runtimes **must** support LOCAL_FLOOR reference space.

An XrSpace handle for a reference space is created using xrCreateReferenceSpace, by specifying the chosen reference space type and a pose within the natural reference frame defined for that reference space type.

Runtimes implement well-known reference spaces from XrReferenceSpaceType if they support tracking of that kind. Available reference space types are indicated by xrEnumerateReferenceSpaces. Note that other spaces can be created as well, such as pose action spaces created by xrCreateActionSpace, which are not enumerated by that API.

7.1.1. View Reference Space

The XR_REFERENCE_SPACE_TYPE_VIEW or VIEW reference space tracks the view origin used to generate view

transforms for the primary viewer (or centroid of view origins if stereo), with +Y up, +X to the right, and -Z forward. This space points in the forward direction for the viewer without incorporating the user's eye orientation, and is not gravity-aligned.

The `VIEW` space is primarily useful when projecting from the user's perspective into another space to obtain a targeting ray, or when rendering small head-locked content such as a reticle. Content rendered in the `VIEW` space will stay at a fixed point on head-mounted displays and may be uncomfortable to view if too large. To obtain the ideal view and projection transforms to use each frame for rendering world content, applications should call xrLocateViews instead of using this space.

7.1.2. Local Reference Space

The `XR_REFERENCE_SPACE_TYPE_LOCAL` or `LOCAL` reference space establishes a world-locked origin, gravity-aligned to exclude pitch and roll, with +Y up, +X to the right, and -Z forward. This space locks in both its initial position and orientation, which the runtime **may** define to be either the initial position at application launch or some other calibrated zero position.

When a user needs to recenter the `LOCAL` space, a runtime **may** offer some system-level recentering interaction that is transparent to the application, but which causes the current leveled head space to become the new `LOCAL` space. When such a recentering occurs, the runtime **must** queue the XrEventDataReferenceSpaceChangePending event, with the recentered `LOCAL` space origin only taking effect for xrLocateSpace or xrLocateViews calls whose `XrTime` parameter is greater than or equal to the XrEventDataReferenceSpaceChangePending::`changeTime` in that event.

When views, controllers or other spaces experience tracking loss relative to the `LOCAL` space, runtimes **should** continue to provide inferred or last-known `position` and `orientation` values. These inferred poses can, for example, be based on neck model updates, inertial dead reckoning, or a last-known position, so long as it is still reasonable for the application to use that pose. While a runtime is providing position data, it **must** continue to set `XR_SPACE_LOCATION_POSITION_VALID_BIT` and `XR_VIEW_STATE_POSITION_VALID_BIT` but it **can** clear `XR_SPACE_LOCATION_POSITION_TRACKED_BIT` and `XR_VIEW_STATE_POSITION_TRACKED_BIT` to indicate that the position is inferred or last-known in this way.

When tracking is recovered, runtimes **should** snap the pose of other spaces back into position relative to the original origin of `LOCAL` space.

7.1.3. Stage Reference Space

The `STAGE` reference space is a runtime-defined flat, rectangular space that is empty and can be walked around on. The origin is on the floor at the center of the rectangle, with +Y up, and the X and Z axes aligned with the rectangle edges. The runtime **may** not be able to locate spaces relative to the `STAGE` reference space if the user has not yet defined one within the runtime-specific UI. Applications **can** use xrGetReferenceSpaceBoundsRect to determine the extents of the `STAGE` reference space's XZ bounds rectangle, if defined.

The `STAGE` space is useful when an application needs to render **standing-scale** content (no bounds) or **room-scale** content (with bounds) that is relative to the physical floor.

When the user redefines the origin or bounds of the current STAGE space, or the runtime otherwise switches to a new STAGE space definition, the runtime **must** queue the XrEventDataReferenceSpaceChangePending event, with the new STAGE space origin only taking effect for xrLocateSpace or xrLocateViews calls whose XrTime parameter is greater than or equal to the XrEventDataReferenceSpaceChangePending::changeTime in that event.

When views, controllers, or other spaces experience tracking loss relative to the STAGE space, runtimes **should** continue to provide inferred or last-known position and orientation values. These inferred poses can, for example, be based on neck model updates, inertial dead reckoning, or a last-known position, so long as it is still reasonable for the application to use that pose. While a runtime is providing position data, it **must** continue to set XR_SPACE_LOCATION_POSITION_VALID_BIT and XR_VIEW_STATE_POSITION_VALID_BIT but it **can** clear XR_SPACE_LOCATION_POSITION_TRACKED_BIT and XR_VIEW_STATE_POSITION_TRACKED_BIT to indicate that the position is inferred or last-known in this way. When tracking is recovered, runtimes **should** snap the pose of other spaces back into position relative to the original origin of the STAGE space.

7.1.4. Local Floor Reference Space

Local floor reference space, indicated by XR_REFERENCE_SPACE_TYPE_LOCAL_FLOOR, is closely related to the LOCAL reference space. It always aligns with the LOCAL space, and matches it in X and Z position. However, unlike the LOCAL space, the LOCAL_FLOOR space has its Y axis origin on the runtime's best estimate of the floor level under the origin of the LOCAL space.

The location of the origin of the LOCAL_FLOOR space **must** match the LOCAL space in the X and Z coordinates but not in the Y coordinate.

The orientation of the LOCAL_FLOOR space **must** match the LOCAL space.

The runtime **must** establish the Y axis origin at its best estimate of the floor level under the origin of the LOCAL space space, subject to requirements under the following conditions to match the floor level of the STAGE space.

If all of the following conditions are true, the Y axis origin of the LOCAL_FLOOR space **must** match the Y axis origin of the STAGE space:

- the STAGE space is supported
- the location of the LOCAL space relative to the STAGE space has valid position (XR_SPACE_LOCATION_POSITION_VALID_BIT is set)
- bounds are available from xrGetReferenceSpaceBoundsRect for the STAGE space
- the position of the LOCAL space relative to the STAGE space is within the STAGE space XZ bounds

That is, if there is a stage with bounds, and if the local space and thus the local floor is logically within the stage, the local floor and the stage share the same floor level.

When the origin of the LOCAL space is changed in orientation or XZ position, the origin of the

LOCAL_FLOOR space **must** also change accordingly.

When a change in origin of the LOCAL_FLOOR space occurs, the runtime **must** queue the XrEventDataReferenceSpaceChangePending event, with the changed LOCAL_FLOOR space origin only taking effect for xrLocateSpace or xrLocateViews calls whose XrTime parameter is greater than or equal to the XrEventDataReferenceSpaceChangePending::changeTime in that event.

The xrGetReferenceSpaceBoundsRect function is defined as:

```
// Provided by XR_VERSION_1_0
XrResult xrGetReferenceSpaceBoundsRect(
    XrSession                                   session,
    XrReferenceSpaceType                        referenceSpaceType,
    XrExtent2Df*                                bounds);
```

Parameter Descriptions

- session is a handle to an XrSession previously created with xrCreateSession.
- referenceSpaceType is the reference space type whose bounds should be retrieved.
- bounds is the returned space extents.

XR systems **may** have limited real world spatial ranges in which users can freely move around while remaining tracked. Applications sometimes wish to query these boundaries and alter application behavior or content placement to ensure the user can complete the experience while remaining within the boundary. Applications **can** query this information using xrGetReferenceSpaceBoundsRect.

When called, xrGetReferenceSpaceBoundsRect **should** return the extents of a rectangle that is clear of obstacles down to the floor, allowing where the user can freely move while remaining tracked, if available for that reference space. The returned extent represents the dimensions of an axis-aligned bounding box where the XrExtent2Df::width and XrExtent2Df::height fields correspond to the X and Z axes of the provided space, with the extents centered at the origin of the space. Not all systems or spaces support boundaries. If a runtime is unable to provide bounds for a given space, XR_SPACE_BOUNDS_UNAVAILABLE **must** be returned and all fields of bounds **must** be set to 0.

The returned extents are expressed relative to the natural origin of the provided XrReferenceSpaceType and **must** not incorporate any origin offsets specified by the application during calls to xrCreateReferenceSpace.

The runtime **must** return XR_ERROR_REFERENCE_SPACE_UNSUPPORTED if the XrReferenceSpaceType passed in referenceSpaceType is not supported by this session.

When a runtime will begin operating with updated space bounds, the runtime **must** queue a corresponding XrEventDataReferenceSpaceChangePending event.

Valid Usage (Implicit)

- `session` **must** be a valid XrSession handle
- `referenceSpaceType` **must** be a valid XrReferenceSpaceType value
- `bounds` **must** be a pointer to an XrExtent2Df structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING
- XR_SPACE_BOUNDS_UNAVAILABLE

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_REFERENCE_SPACE_UNSUPPORTED
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The XrEventDataReferenceSpaceChangePending structure is defined as:

```
// Provided by XR_VERSION_1_0
typedef struct XrEventDataReferenceSpaceChangePending {
    XrStructureType         type;
    const void*             next;
    XrSession               session;
    XrReferenceSpaceType    referenceSpaceType;
    XrTime                  changeTime;
    XrBool32                poseValid;
    XrPosef                 poseInPreviousSpace;
} XrEventDataReferenceSpaceChangePending;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR.
- `session` is the XrSession for which the reference space is changing.
- `referenceSpaceType` is the XrReferenceSpaceType that is changing.
- `changeTime` is the target `XrTime` after which xrLocateSpace or xrLocateViews will return values that respect this change.
- `poseValid` is true if the runtime can determine the `poseInPreviousSpace` of the new space in the previous space before the change.
- `poseInPreviousSpace` is an XrPosef defining the position and orientation of the new reference space's natural origin within the natural reference frame of its previous space.

The XrEventDataReferenceSpaceChangePending event is sent to the application to notify it that the origin (and perhaps the bounds) of a reference space is changing. This may occur due to the user recentering the space explicitly, or the runtime otherwise switching to a different space definition.

The reference space change **must** only take effect for xrLocateSpace or xrLocateViews calls whose `XrTime` parameter is greater than or equal to the `changeTime` provided in that event. Runtimes **should** provide a `changeTime` to applications that allows for a deep render pipeline to present frames that are already in flight using the previous definition of the space. Runtimes **should** choose a `changeTime` that is midway between the XrFrameState::`predictedDisplayTime` of future frames to avoid threshold issues with applications that calculate future frame times using XrFrameState::`predictedDisplayPeriod`.

The `poseInPreviousSpace` provided here **must** only describe the change in the natural origin of the reference space and **must** not incorporate any origin offsets specified by the application during calls to xrCreateReferenceSpace. If the runtime does not know the location of the space's new origin relative to its previous origin, `poseValid` **must** be false, and the position and orientation of `poseInPreviousSpace` are undefined.

Valid Usage (Implicit)

- `type` **must** be `XR_TYPE_EVENT_DATA_REFERENCE_SPACE_CHANGE_PENDING`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

7.2. Action Spaces

An XrSpace handle for a pose action is created using xrCreateActionSpace, by specifying the chosen pose action and a pose within the action's natural reference frame.

Runtimes support suggested pose action bindings to well-known user paths with *.../pose* subpaths if they support tracking for that particular identifier.

Some example well-known pose action paths:

- */user/hand/left/input/grip*
- */user/hand/left/input/aim*
- */user/hand/right/input/grip*
- */user/hand/right/input/aim*

For definitions of these well-known pose device paths, see the discussion of device input subpaths in the Semantic Paths chapter.

7.2.1. Action Spaces Lifetime

XrSpace handles created for a pose action **must** be unlocatable unless the action set that contains the corresponding pose action was set as active via the most recent xrSyncActions call. If the underlying device that is active for the action changes, the device this space is tracking **must** only change to track the new device when xrSyncActions is called.

If xrLocateSpace is called with an unlocatable action space, the implementation **must** return no position or orientation and both `XR_SPACE_LOCATION_POSITION_VALID_BIT` and `XR_SPACE_LOCATION_ORIENTATION_VALID_BIT` **must** be unset. If XrSpaceVelocity is also supplied, `XR_SPACE_VELOCITY_LINEAR_VALID_BIT` and `XR_SPACE_VELOCITY_ANGULAR_VALID_BIT` **must** be unset. If xrLocateViews is called with an unlocatable action space, the implementation **must** return no position or orientation and both `XR_VIEW_STATE_POSITION_VALID_BIT` and `XR_VIEW_STATE_ORIENTATION_VALID_BIT` **must** be unset.

7.3. Space Lifecycle

There are a small set of core APIs that allow applications to reason about reference spaces, action spaces, and their relative locations.

7.3.1. xrEnumerateReferenceSpaces

The xrEnumerateReferenceSpaces function is defined as:

```
// Provided by XR_VERSION_1_0
XrResult xrEnumerateReferenceSpaces(
    XrSession                                   session,
    uint32_t                                    spaceCapacityInput,
    uint32_t*                                   spaceCountOutput,
    XrReferenceSpaceType*                       spaces);
```

Parameter Descriptions

- `session` is a handle to an XrSession previously created with xrCreateSession.
- `spaceCapacityInput` is the capacity of the `spaces` array, or 0 to indicate a request to retrieve the required capacity.
- `spaceCountOutput` is a pointer to the count of `spaces` written, or a pointer to the required capacity in the case that `spaceCapacityInput` is insufficient.
- `spaces` is a pointer to an application-allocated array that will be filled with the enumerant of each supported reference space. It **can** be `NULL` if `spaceCapacityInput` is 0.
- See the Buffer Size Parameters section for a detailed description of retrieving the required `spaces` size.

Enumerates the set of reference space types that this runtime supports for a given session. Runtimes **must** always return identical buffer contents from this enumeration for the lifetime of the session.

If a session enumerates support for a given reference space type, calls to xrCreateReferenceSpace **must** succeed for that session, with any transient unavailability of poses expressed later during calls to xrLocateSpace.

Valid Usage (Implicit)

- `session` **must** be a valid XrSession handle
- `spaceCountOutput` **must** be a pointer to a `uint32_t` value
- If `spaceCapacityInput` is not `0`, `spaces` **must** be a pointer to an array of `spaceCapacityInput` XrReferenceSpaceType values

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_SIZE_INSUFFICIENT
- XR_ERROR_VALIDATION_FAILURE

7.3.2. xrCreateReferenceSpace

The xrCreateReferenceSpace function is defined as:

```
// Provided by XR_VERSION_1_0
XrResult xrCreateReferenceSpace(
    XrSession                                   session,
    const XrReferenceSpaceCreateInfo*           createInfo,
    XrSpace*                                    space);
```

Parameter Descriptions

- session is a handle to an XrSession previously created with xrCreateSession.
- createInfo is the XrReferenceSpaceCreateInfo used to specify the space.
- space is the returned space handle.

Creates an XrSpace handle based on a chosen reference space. Application **can** provide an XrPosef to define the position and orientation of the new space's origin within the natural reference frame of the reference space.

Multiple XrSpace handles may exist simultaneously, up to some limit imposed by the runtime. The XrSpace handle **must** be eventually freed via the xrDestroySpace function.

The runtime **must** return XR_ERROR_REFERENCE_SPACE_UNSUPPORTED if the given reference space type is not supported by this session.

Valid Usage (Implicit)

- `session` **must** be a valid XrSession handle
- `createInfo` **must** be a pointer to a valid XrReferenceSpaceCreateInfo structure
- `space` **must** be a pointer to an XrSpace handle

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_POSE_INVALID
- XR_ERROR_REFERENCE_SPACE_UNSUPPORTED
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The XrReferenceSpaceCreateInfo structure is defined as:

```
// Provided by XR_VERSION_1_0
typedef struct XrReferenceSpaceCreateInfo {
    XrStructureType         type;
    const void*             next;
    XrReferenceSpaceType    referenceSpaceType;
    XrPosef                 poseInReferenceSpace;
} XrReferenceSpaceCreateInfo;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR.
- `referenceSpaceType` is the chosen XrReferenceSpaceType.
- `poseInReferenceSpace` is an XrPosef defining the position and orientation of the new space's origin within the natural reference frame of the reference space.

Valid Usage (Implicit)

- `type` **must** be `XR_TYPE_REFERENCE_SPACE_CREATE_INFO`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `referenceSpaceType` **must** be a valid XrReferenceSpaceType value

7.3.3. xrCreateActionSpace

The xrCreateActionSpace function is defined as:

```
// Provided by XR_VERSION_1_0
XrResult xrCreateActionSpace(
    XrSession                                   session,
    const XrActionSpaceCreateInfo*              createInfo,
    XrSpace*                                    space);
```

Parameter Descriptions

- `session` is the XrSession to create the action space in.
- `createInfo` is the XrActionSpaceCreateInfo used to specify the space.
- `space` is the returned space handle.

Creates an XrSpace handle based on a chosen pose action. Application **can** provide an XrPosef to define the position and orientation of the new space's origin within the natural reference frame of the action space.

Multiple XrSpace handles may exist simultaneously, up to some limit imposed by the runtime. The XrSpace handle must be eventually freed via the xrDestroySpace function or by destroying the parent XrSession handle. See Action Spaces Lifetime for details.

The runtime **must** return `XR_ERROR_ACTION_TYPE_MISMATCH` if the action provided in XrActionSpaceCreateInfo::`action` is not of type `XR_ACTION_TYPE_POSE_INPUT`.

Valid Usage (Implicit)

- `session` **must** be a valid XrSession handle
- `createInfo` **must** be a pointer to a valid XrActionSpaceCreateInfo structure
- `space` **must** be a pointer to an XrSpace handle

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_ACTION_TYPE_MISMATCH`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_LIMIT_REACHED`
- `XR_ERROR_OUT_OF_MEMORY`
- `XR_ERROR_PATH_INVALID`
- `XR_ERROR_PATH_UNSUPPORTED`
- `XR_ERROR_POSE_INVALID`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_VALIDATION_FAILURE`

The XrActionSpaceCreateInfo structure is defined as:

```
// Provided by XR_VERSION_1_0
typedef struct XrActionSpaceCreateInfo {
    XrStructureType    type;
    const void*        next;
    XrAction           action;
    XrPath             subactionPath;
    XrPosef            poseInActionSpace;
} XrActionSpaceCreateInfo;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR.
- `action` is a handle to a pose XrAction previously created with xrCreateAction.
- `subactionPath` is XR_NULL_PATH or an `XrPath` that was specified when the action was created. If `subactionPath` is a valid path not specified when the action was created the runtime **must** return `XR_ERROR_PATH_UNSUPPORTED`. If this parameter is set, the runtime **must** create a space that is relative to only that subaction's pose binding.
- `poseInActionSpace` is an XrPosef defining the position and orientation of the new space's origin within the natural reference frame of the pose action.

Valid Usage (Implicit)

- `type` **must** be `XR_TYPE_ACTION_SPACE_CREATE_INFO`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `action` **must** be a valid XrAction handle

7.3.4. xrDestroySpace

The xrDestroySpace function is defined as:

```
// Provided by XR_VERSION_1_0
XrResult xrDestroySpace(
    XrSpace                                     space);
```

Parameter Descriptions

- `space` is a handle to an XrSpace previously created by a function such as xrCreateReferenceSpace.

XrSpace handles are destroyed using xrDestroySpace. The runtime **may** still use this space if there are active dependencies (e.g, compositions in progress).

Valid Usage (Implicit)

- `space` **must** be a valid XrSpace handle

Thread Safety

- Access to `space`, and any child handles, **must** be externally synchronized

Return Codes

Success

- `XR_SUCCESS`

Failure

- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_RUNTIME_FAILURE`

7.4. Locating Spaces

Applications use the xrLocateSpace function to find the pose of an XrSpace's origin within a base XrSpace at a given historical or predicted time. If an application wants to know the velocity of the space's origin, it **can** chain an XrSpaceVelocity structure to the `next` pointer of the XrSpaceLocation structure when calling the xrLocateSpace function. Applications **should** inspect the output XrSpaceLocationFlagBits and XrSpaceVelocityFlagBits to determine the validity and tracking status of the components of the location.

7.4.1. xrLocateSpace

xrLocateSpace provides the physical location of a space in a base space at a specified time, if currently known by the runtime.

```
// Provided by XR_VERSION_1_0
XrResult xrLocateSpace(
    XrSpace                                     space,
    XrSpace                                     baseSpace,
    XrTime                                      time,
    XrSpaceLocation*                            location);
```

Parameter Descriptions

- `space` identifies the target space to locate.
- `baseSpace` identifies the underlying space in which to locate `space`.
- `time` is the time for which the location should be provided.
- `location` provides the location of `space` in `baseSpace`.

For a `time` in the past, the runtime **should** locate the spaces based on the runtime's most accurate current understanding of how the world was at that historical time.

For a `time` in the future, the runtime **should** locate the spaces based on the runtime's most up-to-date prediction of how the world will be at that future time.

The minimum valid range of values for `time` are described in Prediction Time Limits. For values of `time` outside this range, xrLocateSpace **may** return a location with no position and `XR_SPACE_LOCATION_POSITION_VALID_BIT` unset.

Some devices improve their understanding of the world as the device is used. The location returned by xrLocateSpace for a given `space`, `baseSpace` and `time` **may** change over time, even for spaces that track static objects, as one or both spaces adjust their origins.

During tracking loss of `space` relative to `baseSpace`, runtimes **should** continue to provide inferred or last-known XrPosef::`position` and XrPosef::`orientation` values. These inferred poses can, for example, be based on neck model updates, inertial dead reckoning, or a last-known position, so long as it is still reasonable for the application to use that pose. While a runtime is providing position data, it **must** continue to set `XR_SPACE_LOCATION_POSITION_VALID_BIT` but it **can** clear `XR_SPACE_LOCATION_POSITION_TRACKED_BIT` to indicate that the position is inferred or last-known in this way.

If the runtime has not yet observed even a last-known pose for how to locate `space` in `baseSpace` (e.g. one space is an action space bound to a motion controller that has not yet been detected, or the two spaces are in disconnected fragments of the runtime's tracked volume), the runtime **should** return a location with no position and `XR_SPACE_LOCATION_POSITION_VALID_BIT` unset.

The runtime **must** return a location with both `XR_SPACE_LOCATION_POSITION_VALID_BIT` and `XR_SPACE_LOCATION_POSITION_TRACKED_BIT` set when locating `space` and `baseSpace` if both spaces were

created relative to the same entity (e.g. two action spaces for the same action), even if the entity is currently untracked. The location in this case is the difference in the two spaces' application-specified transforms relative to that common entity.

During tracking loss, the runtime **should** return a location with XR_SPACE_LOCATION_POSITION_VALID_BIT and XR_SPACE_LOCATION_ORIENTATION_VALID_BIT set and XR_SPACE_LOCATION_POSITION_TRACKED_BIT and XR_SPACE_LOCATION_ORIENTATION_TRACKED_BIT unset for spaces tracking two static entities in the world when their relative pose is known to the runtime. This enables applications to continue to make use of the runtime's latest knowledge of the world.

If an XrSpaceVelocity structure is chained to the XrSpaceLocation::next pointer, and the velocity is observed or can be calculated by the runtime, the runtime **must** fill in the linear velocity of the origin of space within the reference frame of baseSpace and set the XR_SPACE_VELOCITY_LINEAR_VALID_BIT. Similarly, if an XrSpaceVelocity structure is chained to the XrSpaceLocation::next pointer, and the angular velocity is observed or can be calculated by the runtime, the runtime **must** fill in the angular velocity of the origin of space within the reference frame of baseSpace and set the XR_SPACE_VELOCITY_ANGULAR_VALID_BIT.

The following example code shows how an application can get both the location and velocity of a space within a base space using the xrLocateSpace function by chaining an XrSpaceVelocity to the next pointer of XrSpaceLocation and calling xrLocateSpace.

```
XrSpace space;      // previously initialized
XrSpace baseSpace;  // previously initialized
XrTime time;        // previously initialized

XrSpaceVelocity velocity {
    .type = XR_TYPE_SPACE_VELOCITY,
};
XrSpaceLocation location {
    .type = XR_TYPE_SPACE_LOCATION,
    .next = &velocity,
};
xrLocateSpace(space, baseSpace, time, &location);
```

Valid Usage (Implicit)

- space **must** be a valid XrSpace handle
- baseSpace **must** be a valid XrSpace handle
- location **must** be a pointer to an XrSpaceLocation structure
- Both of baseSpace and space **must** have been created, allocated, or retrieved from the same XrSession

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_TIME_INVALID
- XR_ERROR_VALIDATION_FAILURE

The XrSpaceLocation structure is defined as:

```
// Provided by XR_VERSION_1_0
typedef struct XrSpaceLocation {
    XrStructureType         type;
    void*                   next;
    XrSpaceLocationFlags    locationFlags;
    XrPosef                 pose;
} XrSpaceLocation;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain, such as XrSpaceVelocity.
- locationFlags is a bitfield, with bit masks defined in XrSpaceLocationFlagBits, to indicate which members contain valid data. If none of the bits are set, no other fields in this structure **should** be considered to be valid or meaningful.
- pose is an XrPosef defining the position and orientation of the origin of xrLocateSpace::space within the reference frame of xrLocateSpace::baseSpace.

Valid Usage (Implicit)

- `type` **must** be `XR_TYPE_SPACE_LOCATION`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain. See also: XrEyeGazeSampleTimeEXT, XrSpaceVelocity
- `locationFlags` **must** be `0` or a valid combination of XrSpaceLocationFlagBits values

The XrSpaceLocation::`locationFlags` member is of the following type, and contains a bitwise-OR of zero or more of the bits defined in XrSpaceLocationFlagBits.

```
// Provided by XR_VERSION_1_0
typedef XrFlags64 XrSpaceLocationFlags;
```

Valid bits for XrSpaceLocationFlags are defined by XrSpaceLocationFlagBits, which is specified as:

```
// Provided by XR_VERSION_1_0
// Flag bits for XrSpaceLocationFlags
static const XrSpaceLocationFlags XR_SPACE_LOCATION_ORIENTATION_VALID_BIT = 0x00000001;
static const XrSpaceLocationFlags XR_SPACE_LOCATION_POSITION_VALID_BIT = 0x00000002;
static const XrSpaceLocationFlags XR_SPACE_LOCATION_ORIENTATION_TRACKED_BIT = 0x00000004;
static const XrSpaceLocationFlags XR_SPACE_LOCATION_POSITION_TRACKED_BIT = 0x00000008;
```

The flag bits have the following meanings:

Flag Descriptions

- XR_SPACE_LOCATION_ORIENTATION_VALID_BIT indicates that the pose field's orientation field contains valid data. For a space location tracking a device with its own inertial tracking, XR_SPACE_LOCATION_ORIENTATION_TRACKED_BIT **should** remain set when this bit is set. Applications **must** not read the pose field's orientation if this flag is unset.
- XR_SPACE_LOCATION_POSITION_VALID_BIT indicates that the pose field's position field contains valid data. When a space location loses tracking, runtimes **should** continue to provide valid but untracked position values that are inferred or last-known, so long as it's still meaningful for the application to use that position, clearing XR_SPACE_LOCATION_POSITION_TRACKED_BIT until positional tracking is recovered. Applications **must** not read the pose field's position if this flag is unset.
- XR_SPACE_LOCATION_ORIENTATION_TRACKED_BIT indicates that the pose field's orientation field represents an actively tracked orientation. For a space location tracking a device with its own inertial tracking, this bit **should** remain set when XR_SPACE_LOCATION_ORIENTATION_VALID_BIT is set. For a space location tracking an object whose orientation is no longer known during tracking loss (e.g. an observed QR code), runtimes **should** continue to provide valid but untracked orientation values, so long as it's still meaningful for the application to use that orientation.
- XR_SPACE_LOCATION_POSITION_TRACKED_BIT indicates that the pose field's position field represents an actively tracked position. When a space location loses tracking, runtimes **should** continue to provide valid but untracked position values that are inferred or last-known, e.g. based on neck model updates, inertial dead reckoning, or a last-known position, so long as it's still meaningful for the application to use that position.

The XrSpaceVelocity structure is defined as:

```
// Provided by XR_VERSION_1_0
typedef struct XrSpaceVelocity {
    XrStructureType         type;
    void*                   next;
    XrSpaceVelocityFlags    velocityFlags;
    XrVector3f              linearVelocity;
    XrVector3f              angularVelocity;
} XrSpaceVelocity;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR.
- `velocityFlags` is a bitfield, with bit masks defined in XrSpaceVelocityFlagBits, to indicate which members contain valid data. If none of the bits are set, no other fields in this structure **should** be considered to be valid or meaningful.
- `linearVelocity` is the relative linear velocity of the origin of xrLocateSpace::`space` with respect to and expressed in the reference frame of xrLocateSpace::`baseSpace`, in units of meters per second.
- `angularVelocity` is the relative angular velocity of xrLocateSpace::`space` with respect to xrLocateSpace::`baseSpace`. The vector's direction is expressed in the reference frame of xrLocateSpace::`baseSpace` and is parallel to the rotational axis of xrLocateSpace::`space`. The vector's magnitude is the relative angular speed of xrLocateSpace::`space` in radians per second. The vector follows the right-hand rule for torque/rotation.

Valid Usage (Implicit)

- `type` **must** be `XR_TYPE_SPACE_VELOCITY`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `velocityFlags` **must** be `0` or a valid combination of XrSpaceVelocityFlagBits values

The XrSpaceVelocity::`velocityFlags` member is of the following type, and contains a bitwise-OR of zero or more of the bits defined in XrSpaceVelocityFlagBits.

```
// Provided by XR_VERSION_1_0
typedef XrFlags64 XrSpaceVelocityFlags;
```

Valid bits for XrSpaceVelocityFlags are defined by XrSpaceVelocityFlagBits, which is specified as:

```
// Provided by XR_VERSION_1_0
// Flag bits for XrSpaceVelocityFlags
static const XrSpaceVelocityFlags XR_SPACE_VELOCITY_LINEAR_VALID_BIT = 0x00000001;
static const XrSpaceVelocityFlags XR_SPACE_VELOCITY_ANGULAR_VALID_BIT = 0x00000002;
```

The flag bits have the following meanings:

Flag Descriptions

- `XR_SPACE_VELOCITY_LINEAR_VALID_BIT` — Indicates that the `linearVelocity` member contains valid data. Applications **must** not read the `linearVelocity` field if this flag is unset.
- `XR_SPACE_VELOCITY_ANGULAR_VALID_BIT` — Indicates that the `angularVelocity` member contains valid data. Applications **must** not read the `angularVelocity` field if this flag is unset.

7.4.2. Locate spaces

Applications **can** use xrLocateSpaces function to locate an array of spaces.

The xrLocateSpaces function is defined as:

```
// Provided by XR_VERSION_1_1
XrResult xrLocateSpaces(
    XrSession                                   session,
    const XrSpacesLocateInfo*                   locateInfo,
    XrSpaceLocations*                           spaceLocations);
```

Parameter Descriptions

- `session` is an XrSession handle previously created with xrCreateSession.
- `locateInfo` is a pointer to an XrSpacesLocateInfo that provides the input information to locate spaces.
- `spaceLocations` is a pointer to an XrSpaceLocations for the runtime to return the locations of the specified spaces in the base space.

xrLocateSpaces provides the physical location of one or more spaces in a base space at a specified time, if currently known by the runtime.

The XrSpacesLocateInfo::`time`, the XrSpacesLocateInfo::`baseSpace`, and each space in XrSpacesLocateInfo::`spaces`, in the `locateInfo` parameter, all follow the same specifics as the corresponding inputs to the xrLocateSpace function.

Valid Usage (Implicit)

- `session` **must** be a valid XrSession handle
- `locateInfo` **must** be a pointer to a valid XrSpacesLocateInfo structure
- `spaceLocations` **must** be a pointer to an XrSpaceLocations structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_SIZE_INSUFFICIENT
- XR_ERROR_TIME_INVALID
- XR_ERROR_VALIDATION_FAILURE

The XrSpacesLocateInfo structure is defined as:

```
// Provided by XR_VERSION_1_1
typedef struct XrSpacesLocateInfo {
    XrStructureType    type;
    const void*        next;
    XrSpace            baseSpace;
    XrTime             time;
    uint32_t           spaceCount;
    const XrSpace*     spaces;
} XrSpacesLocateInfo;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR.
- baseSpace identifies the underlying space in which to locate spaces.
- time is the time for which the location is requested.
- spaceCount is a uint32_t specifying the count of elements in the spaces array.
- spaces is an array of valid XrSpace handles to be located.

The time, the baseSpace, and each space in spaces all follow the same specifics as the corresponding inputs to the xrLocateSpace function.

The baseSpace and all of the XrSpace handles in the spaces array **must** be valid and share the same parent XrSession.

If the time is invalid, the xrLocateSpaces **must** return XR_ERROR_TIME_INVALID.

The spaceCount **must** be a positive number, i.e. the array spaces **must** not be empty. Otherwise, the runtime **must** return XR_ERROR_VALIDATION_FAILURE.

Valid Usage (Implicit)

- type **must** be XR_TYPE_SPACES_LOCATE_INFO
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- baseSpace **must** be a valid XrSpace handle
- spaces **must** be a pointer to an array of spaceCount valid XrSpace handles
- The spaceCount parameter **must** be greater than 0
- Both of baseSpace and the elements of spaces **must** have been created, allocated, or retrieved from the same XrSession

The XrSpaceLocations structure is defined as:

```
// Provided by XR_VERSION_1_1
typedef struct XrSpaceLocations {
    XrStructureType         type;
    void*                   next;
    uint32_t                locationCount;
    XrSpaceLocationData*    locations;
} XrSpaceLocations;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain, such as XrSpaceVelocities.
- locationCount is a uint32_t specifying the count of elements in the locations array.
- locations is an array of XrSpaceLocationData structures for the runtime to populate with the locations of the specified spaces in the XrSpacesLocateInfo::baseSpace at the specified XrSpacesLocateInfo::time.

The XrSpaceLocations structure contains an array of space locations in the member `locations`, to be used as output for xrLocateSpaces. The application **must** allocate this array to be populated with the function output. The `locationCount` value **must** be the same as XrSpacesLocateInfo::`spaceCount`, otherwise, the xrLocateSpaces function **must** return `XR_ERROR_VALIDATION_FAILURE`.

Valid Usage (Implicit)

- `type` **must** be `XR_TYPE_SPACE_LOCATIONS`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain. See also: XrSpaceVelocities
- `locations` **must** be a pointer to an array of `locationCount` XrSpaceLocationData structures
- The `locationCount` parameter **must** be greater than `0`

The XrSpaceLocationData structure is defined as:

```
// Provided by XR_VERSION_1_1
typedef struct XrSpaceLocationData {
    XrSpaceLocationFlags    locationFlags;
    XrPosef                 pose;
} XrSpaceLocationData;
```

Member Descriptions

- `locationFlags` is a bitfield, with bit masks defined in XrSpaceLocationFlagBits. It behaves the same as XrSpaceLocation::`locationFlags`.
- `pose` is an XrPosef that behaves the same as XrSpaceLocation::`pose`.

This is a single element of the array in XrSpaceLocations::`locations`, and is used to return the pose and location flags for a single space with respect to the specified base space from a call to xrLocateSpaces. It does not accept chained structures to allow for easier use in dynamically allocated container datatypes. Chained structures are possible with the XrSpaceLocations that describes an array of these elements.

7.4.3. Locate space velocities

Applications **can** request the velocities of spaces by chaining the XrSpaceVelocities structure to the next pointer of XrSpaceLocations when calling xrLocateSpaces.

The XrSpaceVelocities structure is defined as:

```
// Provided by XR_VERSION_1_1
typedef struct XrSpaceVelocities {
    XrStructureType         type;
    void*                   next;
    uint32_t                velocityCount;
    XrSpaceVelocityData*    velocities;
} XrSpaceVelocities;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `velocityCount` is a `uint32_t` specifying the count of elements in the `velocities` array.
- `velocities` is an array of XrSpaceVelocityData for the runtime to populate with the velocities of the specified spaces in the XrSpacesLocateInfo::`baseSpace` at the specified XrSpacesLocateInfo::`time`.

The `velocities` member contains an array of space velocities in the member `velocities`, to be used as output for xrLocateSpaces. The application **must** allocate this array to be populated with the function output. The `velocityCount` value **must** be the same as XrSpacesLocateInfo::`spaceCount`, otherwise, the xrLocateSpaces function **must** return `XR_ERROR_VALIDATION_FAILURE`.

Valid Usage (Implicit)

- `type` **must** be `XR_TYPE_SPACE_VELOCITIES`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `velocities` **must** be a pointer to an array of `velocityCount` XrSpaceVelocityData structures
- The `velocityCount` parameter **must** be greater than `0`

The XrSpaceVelocityData structure is defined as:

```
// Provided by XR_VERSION_1_1
typedef struct XrSpaceVelocityData {
    XrSpaceVelocityFlags    velocityFlags;
    XrVector3f              linearVelocity;
    XrVector3f              angularVelocity;
} XrSpaceVelocityData;
```

Member Descriptions

- `velocityFlags` is a bitfield, with bit values defined in XrSpaceVelocityFlagBits. It behaves the same as XrSpaceVelocity::`velocityFlags`.
- `linearVelocity` is an XrVector3f. It behaves the same as XrSpaceVelocity::`linearVelocity`.
- `angularVelocity` is an XrVector3f. It behaves the same as XrSpaceVelocity::`angularVelocity`.

This is a single element of the array in XrSpaceVelocities::`velocities`, and is used to return the linear and angular velocity and velocity flags for a single space with respect to the specified base space from a call to xrLocateSpaces. It does not accept chained structures to allow for easier use in dynamically allocated container datatypes.

7.4.4. Example code for xrLocateSpaces

The following example code shows how an application retrieves both the location and velocity of one or more spaces in a base space at a given time using the xrLocateSpaces function.

```
XrInstance instance; // previously initialized
XrSession session; // previously initialized
XrSpace baseSpace; // previously initialized
std::vector<XrSpace> spacesToLocate;  // previously initialized

// Prepare output buffers to receive data and get reused in frame loop.
std::vector<XrSpaceLocationData> locationBuffer(spacesToLocate.size());
std::vector<XrSpaceVelocityData> velocityBuffer(spacesToLocate.size());

// Get function pointer for xrLocateSpaces.
PFN_xrLocateSpaces xrLocateSpaces;
CHK_XR(xrGetInstanceProcAddr(instance, "xrLocateSpaces",
                             reinterpret_cast<PFN_xrVoidFunction*>(
                             &xrLocateSpaces)));

// application frame loop
while (1) {
    // Typically the time is the predicted display time returned from xrWaitFrame.
    XrTime displayTime; // previously initialized.

    XrSpacesLocateInfo locateInfo {
        .type = XR_TYPE_SPACES_LOCATE_INFO,
        .baseSpace = baseSpace,
        .time = displayTime,
        .spaceCount = static_cast<uint32_t>(spacesToLocate.size()),
        .spaces = spacesToLocate.data(),
    };
```

```
    XrSpaceVelocities velocities {
        .type = XR_TYPE_SPACE_VELOCITIES,
        .velocityCount = static_cast<uint32_t>(velocityBuffer.size()),
        .velocities = velocityBuffer.data(),
    };

    XrSpaceLocations locations {
        .type = XR_TYPE_SPACE_LOCATIONS,
        .next = &velocities,
        .locationCount = static_cast<uint32_t>(locationBuffer.size()),
        .locations = locationBuffer.data(),
    };

    CHK_XR(xrLocateSpaces(session, &locateInfo, &locations));

    for (uint32_t i = 0; i < spacesToLocate.size(); i++) {
        const auto positionAndOrientationTracked =
          XR_SPACE_LOCATION_POSITION_TRACKED_BIT |
XR_SPACE_LOCATION_ORIENTATION_TRACKED_BIT;
        const auto orientationOnlyTracked = XR_SPACE_LOCATION_ORIENTATION_TRACKED_BIT;

        if ((locationBuffer[i].locationFlags & positionAndOrientationTracked) ==
positionAndOrientationTracked) {
            // if the location is 6dof tracked
            do_something(locationBuffer[i].pose.position);
            do_something(locationBuffer[i].pose.orientation);

            const auto velocityValidBits =
              XR_SPACE_VELOCITY_LINEAR_VALID_BIT | XR_SPACE_VELOCITY_ANGULAR_VALID_BIT;
            if ((velocityBuffer[i].velocityFlags & velocityValidBits) ==
velocityValidBits) {
                do_something(velocityBuffer[i].linearVelocity);
                do_something(velocityBuffer[i].angularVelocity);
            }
        }
        else if ((locationBuffer[i].locationFlags & orientationOnlyTracked) ==
orientationOnlyTracked) {
            // if the location is 3dof tracked
            do_something(locationBuffer[i].pose.orientation);

            if ((velocityBuffer[i].velocityFlags & XR_SPACE_VELOCITY_ANGULAR_VALID_BIT)
== XR_SPACE_VELOCITY_ANGULAR_VALID_BIT) {
                do_something(velocityBuffer[i].angularVelocity);
            }
        }
    }
}
```

Chapter 8. View Configurations

A **view configuration** is a semantically meaningful set of one or more views for which an application can render images. A **primary view configuration** is a view configuration intended to be presented to the viewer interacting with the XR application. This distinction allows the later addition of additional views, for example views which are intended for spectators.

A typical head-mounted VR system has a view configuration with two views, while a typical phone-based AR system has a view configuration with a single view. A simple multi-wall projection-based (CAVE-like) VR system may have a view configuration with at least one view for each display surface (wall, floor, ceiling) in the room.

For any supported form factor, a system will support one or more primary view configurations. Supporting more than one primary view configuration can be useful if a system supports a special view configuration optimized for the hardware but also supports a more broadly used view configuration as a compatibility fallback.

View configurations are identified with an XrViewConfigurationType.

8.1. Primary View Configurations

```
// Provided by XR_VERSION_1_0
typedef enum XrViewConfigurationType {
    XR_VIEW_CONFIGURATION_TYPE_PRIMARY_MONO = 1,
    XR_VIEW_CONFIGURATION_TYPE_PRIMARY_STEREO = 2,
  // Provided by XR_VERSION_1_1
    XR_VIEW_CONFIGURATION_TYPE_PRIMARY_STEREO_WITH_FOVEATED_INSET = 1000037000,
  // Provided by XR_MSFT_first_person_observer
    XR_VIEW_CONFIGURATION_TYPE_SECONDARY_MONO_FIRST_PERSON_OBSERVER_MSFT = 1000054000,
  // Provided by XR_VARJO_quad_views
    XR_VIEW_CONFIGURATION_TYPE_PRIMARY_QUAD_VARJO =
XR_VIEW_CONFIGURATION_TYPE_PRIMARY_STEREO_WITH_FOVEATED_INSET,
    XR_VIEW_CONFIGURATION_TYPE_MAX_ENUM = 0x7FFFFFFF
} XrViewConfigurationType;
```

The application selects its primary view configuration type when calling xrBeginSession, and that configuration remains constant for the lifetime of the session, until xrEndSession is called.

The number of views and the semantic meaning of each view index within a given view configuration is well-defined, specified below for all core view configurations. The predefined primary view configuration types are:

Enumerant Descriptions

- XR_VIEW_CONFIGURATION_TYPE_PRIMARY_MONO. One view representing the form factor's one primary display. For example, an AR phone's screen. This configuration requires one projection in each XrCompositionLayerProjection layer.
- XR_VIEW_CONFIGURATION_TYPE_PRIMARY_STEREO. Two views representing the form factor's two primary displays, which map to a left-eye and right-eye view. This configuration requires two views in each XrCompositionLayerProjection layer. View index 0 **must** represent the left eye and view index 1 **must** represent the right eye.
- XR_VIEW_CONFIGURATION_TYPE_PRIMARY_STEREO_WITH_FOVEATED_INSET. Four views representing the form factor's primary stereo displays. This view configuration type represents a hardware independent way of providing foveated rendering. The view configuration adds two foveated inset views for the left and right eye separately to the already defined two views specified in the XR_VIEW_CONFIGURATION_TYPE_PRIMARY_STEREO view configuration. View index 0 **must** represent the left eye and view index 1 **must** represent the right eye as specified in XR_VIEW_CONFIGURATION_TYPE_PRIMARY_STEREO view configuration, and view index 2 **must** represent the left eye inset view and view index 3 **must** represent the right eye inset view. The new inset view 2 and view 3 **must**, after applying the pose and FoV projection to same plane, be contained within view 0 and 1 respectively. The inset views **may** have a higher resolution with respect to the same field of view as the corresponding wide FoV view for each eye. The runtime **may** blend between the views at the edges, so the application **must** not omit the inner field of view from being rendered in the outer view. The fov returned by xrLocateViews for each inset view relative to the corresponding outer stereo view **may** change at run-time, the pose for inset view and stereo view for each eye respectively **must** have the same values.



The benefits of the XR_VIEW_CONFIGURATION_TYPE_PRIMARY_STEREO_WITH_FOVEATED_INSET view configuration type can be demonstrated by looking at the rendered pixel count. For example, a Varjo Aero requires a pair of stereo views rendered at 4148 x 3556 (14.7 million pixels) to achieve a pixel density of 35 pixels per degree. By using four views, with an eye-tracked foveated inset covering about 1/9th of the full FoV and rendered with the same 35 pixels per degree and while the remaining views are dropped to 14 pixels per degree, the resolution of the inset is 1076 x 1076 (1.1 million pixels) and the resolution of the stereo views is 1660 x 1420 (2.3 million pixels). The total pixel count is 75% less with XR_VIEW_CONFIGURATION_TYPE_PRIMARY_STEREO_WITH_FOVEATED_INSET over the XR_VIEW_CONFIGURATION_TYPE_PRIMARY_STEREO view configuration type.

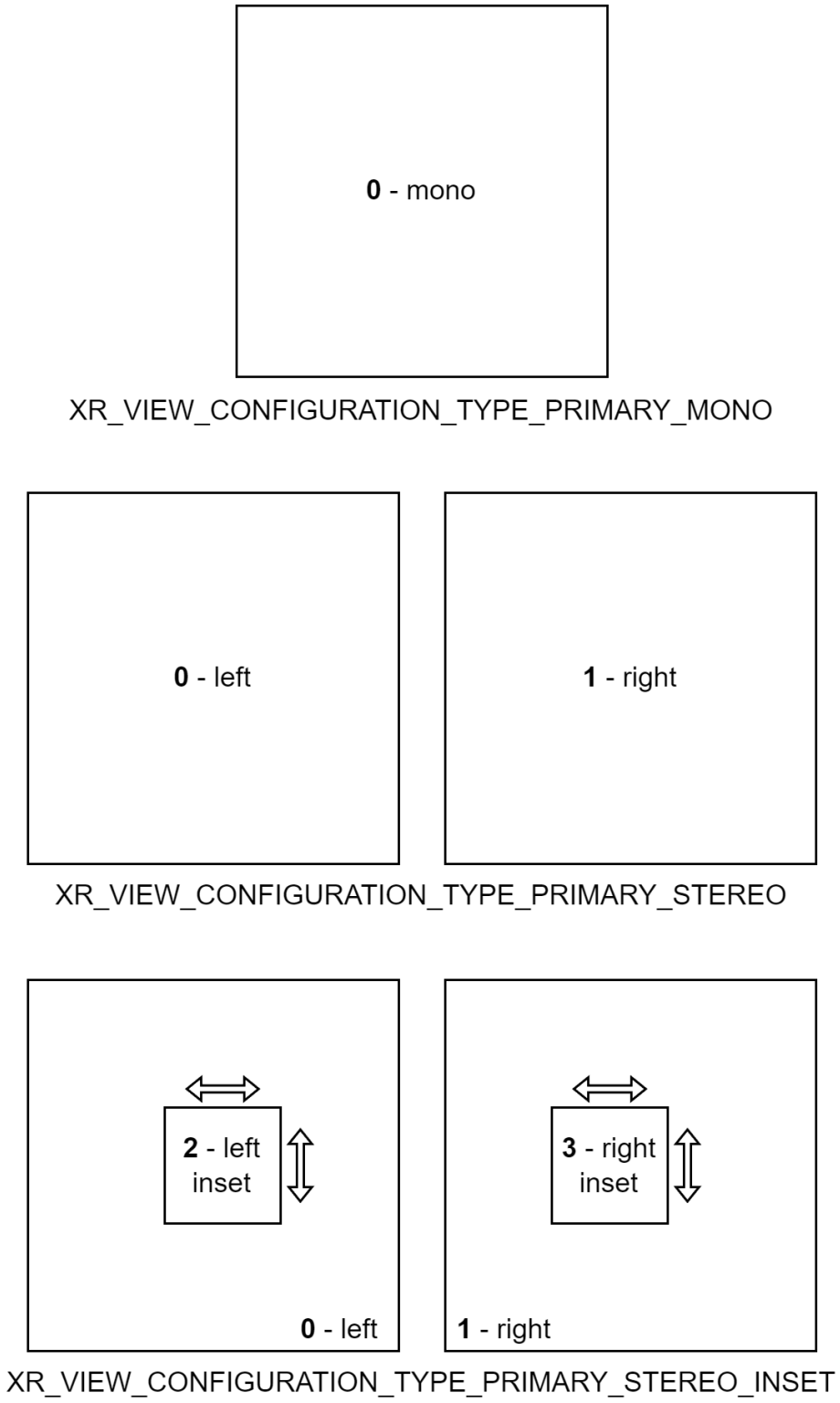


Figure 4. View configurations. The numbers in the figure is the view indices of the specific view.

8.2. View Configuration API

First an application needs to select which primary view configuration it wants to use. If it supports multiple configurations, an application **can** call xrEnumerateViewConfigurations before creating an

XrSession to get a list of the view configuration types supported for a given system.

The application **can** then call xrGetViewConfigurationProperties and xrEnumerateViewConfigurationViews to get detailed information about each view configuration type and its individual views.

8.2.1. xrEnumerateViewConfigurations

The xrEnumerateViewConfigurations function is defined as:

```
// Provided by XR_VERSION_1_0
XrResult xrEnumerateViewConfigurations(
    XrInstance                                  instance,
    XrSystemId                                  systemId,
    uint32_t                                    viewConfigurationTypeCapacityInput,
    uint32_t*                                   viewConfigurationTypeCountOutput,
    XrViewConfigurationType*                    viewConfigurationTypes);
```

Parameter Descriptions

- `instance` is the instance from which `systemId` was retrieved.
- `systemId` is the `XrSystemId` whose view configurations will be enumerated.
- `viewConfigurationTypeCapacityInput` is the capacity of the `viewConfigurationTypes` array, or 0 to indicate a request to retrieve the required capacity.
- `viewConfigurationTypeCountOutput` is a pointer to the count of `viewConfigurationTypes` written, or a pointer to the required capacity in the case that `viewConfigurationTypeCapacityInput` is insufficient.
- `viewConfigurationTypes` is a pointer to an array of XrViewConfigurationType values, but **can** be `NULL` if `viewConfigurationTypeCapacityInput` is 0.
- See the Buffer Size Parameters section for a detailed description of retrieving the required `viewConfigurationTypes` size.

xrEnumerateViewConfigurations enumerates the view configuration types supported by the `XrSystemId`. The supported set for that system **must** not change during the lifetime of its XrInstance. The returned list of primary view configurations **should** be in order from what the runtime considered highest to lowest user preference. Thus the first enumerated view configuration type **should** be the one the runtime prefers the application to use if possible.

Runtimes **must** always return identical buffer contents from this enumeration for the given `systemId` and for the lifetime of the instance.

Valid Usage (Implicit)

- `instance` **must** be a valid XrInstance handle
- `viewConfigurationTypeCountOutput` **must** be a pointer to a `uint32_t` value
- If `viewConfigurationTypeCapacityInput` is not `0`, `viewConfigurationTypes` **must** be a pointer to an array of `viewConfigurationTypeCapacityInput` XrViewConfigurationType values

Return Codes

Success

- `XR_SUCCESS`

Failure

- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SIZE_INSUFFICIENT`
- `XR_ERROR_SYSTEM_INVALID`
- `XR_ERROR_VALIDATION_FAILURE`

8.2.2. xrGetViewConfigurationProperties

The xrGetViewConfigurationProperties function is defined as:

```
// Provided by XR_VERSION_1_0
XrResult xrGetViewConfigurationProperties(
    XrInstance                                  instance,
    XrSystemId                                  systemId,
    XrViewConfigurationType                     viewConfigurationType,
    XrViewConfigurationProperties*              configurationProperties);
```

Parameter Descriptions

- `instance` is the instance from which `systemId` was retrieved.
- `systemId` is the `XrSystemId` whose view configuration is being queried.
- `viewConfigurationType` is the XrViewConfigurationType of the configuration to get.
- `configurationProperties` is a pointer to view configuration properties to return.

xrGetViewConfigurationProperties queries properties of an individual view configuration. Applications **must** use one of the supported view configuration types returned by xrEnumerateViewConfigurations. If `viewConfigurationType` is not supported by this XrInstance the runtime **must** return `XR_ERROR_VIEW_CONFIGURATION_TYPE_UNSUPPORTED`.

Valid Usage (Implicit)

- `instance` **must** be a valid XrInstance handle
- `viewConfigurationType` **must** be a valid XrViewConfigurationType value
- `configurationProperties` **must** be a pointer to an XrViewConfigurationProperties structure

Return Codes

Success

- `XR_SUCCESS`

Failure

- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SYSTEM_INVALID`
- `XR_ERROR_VALIDATION_FAILURE`
- `XR_ERROR_VIEW_CONFIGURATION_TYPE_UNSUPPORTED`

8.2.3. XrViewConfigurationProperties

The XrViewConfigurationProperties structure is defined as:

```
// Provided by XR_VERSION_1_0
typedef struct XrViewConfigurationProperties {
    XrStructureType            type;
    void*                      next;
    XrViewConfigurationType    viewConfigurationType;
    XrBool32                   fovMutable;
} XrViewConfigurationProperties;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR.
- `viewConfigurationType` is the XrViewConfigurationType of the configuration.
- `fovMutable` indicates if the view field of view can be modified by the application.

Valid Usage (Implicit)

- `type` **must** be `XR_TYPE_VIEW_CONFIGURATION_PROPERTIES`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `viewConfigurationType` **must** be a valid XrViewConfigurationType value

8.2.4. xrEnumerateViewConfigurationViews

The xrEnumerateViewConfigurationViews function is defined as:

```
// Provided by XR_VERSION_1_0
XrResult xrEnumerateViewConfigurationViews(
    XrInstance                                  instance,
    XrSystemId                                  systemId,
    XrViewConfigurationType                     viewConfigurationType,
    uint32_t                                    viewCapacityInput,
    uint32_t*                                   viewCountOutput,
    XrViewConfigurationView*                    views);
```

Parameter Descriptions

- `instance` is the instance from which `systemId` was retrieved.
- `systemId` is the `XrSystemId` whose view configuration is being queried.
- `viewConfigurationType` is the XrViewConfigurationType of the configuration to get.
- `viewCapacityInput` is the capacity of the `views` array, or 0 to indicate a request to retrieve the required capacity.
- `viewCountOutput` is a pointer to the count of `views` written, or a pointer to the required capacity in the case that `viewCapacityInput` is 0.
- `views` is a pointer to an array of XrViewConfigurationView values, but **can** be `NULL` if `viewCapacityInput` is 0.

Each XrViewConfigurationType defines the number of views associated with it. Applications can query more details of each view element using xrEnumerateViewConfigurationViews. If the supplied `viewConfigurationType` is not supported by this XrInstance and `XrSystemId`, the runtime **must** return `XR_ERROR_VIEW_CONFIGURATION_TYPE_UNSUPPORTED`.

Runtimes **must** always return identical buffer contents from this enumeration for the given `systemId` and `viewConfigurationType` for the lifetime of the instance.

Valid Usage (Implicit)

- `instance` **must** be a valid XrInstance handle
- `viewConfigurationType` **must** be a valid XrViewConfigurationType value
- `viewCountOutput` **must** be a pointer to a `uint32_t` value
- If `viewCapacityInput` is not `0`, `views` **must** be a pointer to an array of `viewCapacityInput` XrViewConfigurationView structures

Return Codes

Success

- XR_SUCCESS

Failure

- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SIZE_INSUFFICIENT
- XR_ERROR_SYSTEM_INVALID
- XR_ERROR_VALIDATION_FAILURE
- XR_ERROR_VIEW_CONFIGURATION_TYPE_UNSUPPORTED

8.2.5. XrViewConfigurationView

Each XrViewConfigurationView specifies properties related to rendering of an individual view within a view configuration.

The XrViewConfigurationView structure is defined as:

```
// Provided by XR_VERSION_1_0
typedef struct XrViewConfigurationView {
    XrStructureType    type;
    void*              next;
    uint32_t           recommendedImageRectWidth;
    uint32_t           maxImageRectWidth;
    uint32_t           recommendedImageRectHeight;
    uint32_t           maxImageRectHeight;
    uint32_t           recommendedSwapchainSampleCount;
    uint32_t           maxSwapchainSampleCount;
} XrViewConfigurationView;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR.
- `recommendedImageRectWidth` is the optimal width of XrSwapchainSubImage::`imageRect` to use when rendering this view into a swapchain.
- `maxImageRectWidth` is the maximum width of XrSwapchainSubImage::`imageRect` supported when rendering this view into a swapchain.
- `recommendedImageRectHeight` is the optimal height of XrSwapchainSubImage::`imageRect` to use when rendering this view into a swapchain.
- `maxImageRectHeight` is the maximum height of XrSwapchainSubImage::`imageRect` supported when rendering this view into a swapchain.
- `recommendedSwapchainSampleCount` is the recommended number of sub-data element samples to create for each swapchain image that will be rendered into for this view.
- `maxSwapchainSampleCount` is the maximum number of sub-data element samples supported for swapchain images that will be rendered into for this view.

See XrSwapchainSubImage for more information about XrSwapchainSubImage::`imageRect` values, and XrSwapchainCreateInfo for more information about creating swapchains appropriately sized to support those XrSwapchainSubImage::`imageRect` values.

The array of XrViewConfigurationView returned by the runtime **must** adhere to the rules defined in XrViewConfigurationType, such as the count and association to the left and right eyes.

Valid Usage (Implicit)

- `type` **must** be `XR_TYPE_VIEW_CONFIGURATION_VIEW`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain. See also: XrFoveatedViewConfigurationViewVARJO, XrFrameSynthesisConfigViewEXT, XrViewConfigurationDepthRangeEXT, XrViewConfigurationViewFovEPIC

8.3. Example View Configuration Code

```
XrInstance instance; // previously initialized
XrSystemId system;   // previously initialized
XrSession session;   // previously initialized
XrSpace sceneSpace;  // previously initialized
```

```
// Enumerate the view configurations paths.
uint32_t configurationCount;
CHK_XR(xrEnumerateViewConfigurations(instance, system, 0, &configurationCount, nullptr));

std::vector<XrViewConfigurationType> configurationTypes(configurationCount);
CHK_XR(xrEnumerateViewConfigurations(instance, system, configurationCount,
&configurationCount, configurationTypes.data()));

bool configFound = false;
XrViewConfigurationType viewConfig = XR_VIEW_CONFIGURATION_TYPE_MAX_ENUM;
for(uint32_t i = 0; i < configurationCount; ++i)
{
    if (configurationTypes[i] == XR_VIEW_CONFIGURATION_TYPE_PRIMARY_STEREO)
    {
        configFound = true;
        viewConfig = configurationTypes[i];
        break;  // Pick the first supported, i.e. preferred, view configuration.
    }
}

if (!configFound)
    return;   // Cannot support any view configuration of this system.

// Get detailed information of each view element.
uint32_t viewCount;
CHK_XR(xrEnumerateViewConfigurationViews(instance, system,
    viewConfig,
    0,
    &viewCount,
    nullptr));

std::vector<XrViewConfigurationView> configViews(viewCount, {
    .type = XR_TYPE_VIEW_CONFIGURATION_VIEW,
});
CHK_XR(xrEnumerateViewConfigurationViews(instance, system,
    viewConfig,
    viewCount,
    &viewCount,
    configViews.data()));

// Set the primary view configuration for the session.
XrSessionBeginInfo beginInfo {
    .type = XR_TYPE_SESSION_BEGIN_INFO,
    .primaryViewConfigurationType = viewConfig,
};
CHK_XR(xrBeginSession(session, &beginInfo));

// Allocate a buffer according to viewCount.
```

```
std::vector<XrView> views(viewCount, {
    .type = XR_TYPE_VIEW,
});

// Run a per-frame loop.
while (!quit)
{
    // Wait for a new frame.
    XrFrameWaitInfo frameWaitInfo {
        .type = XR_TYPE_FRAME_WAIT_INFO,
    };
    XrFrameState frameState {
        .type = XR_TYPE_FRAME_STATE,
    };
    CHK_XR(xrWaitFrame(session, &frameWaitInfo, &frameState));

    // Begin frame immediately before GPU work
    XrFrameBeginInfo frameBeginInfo {
        .type = XR_TYPE_FRAME_BEGIN_INFO,
    };
    CHK_XR(xrBeginFrame(session, &frameBeginInfo));

    std::vector<XrCompositionLayerBaseHeader*> layers;
    XrCompositionLayerProjectionView projViews[2] = { /*...*/ };
    XrCompositionLayerProjection layerProj {
        .type = XR_TYPE_COMPOSITION_LAYER_PROJECTION,
    };

    if (frameState.shouldRender) {
        XrViewLocateInfo viewLocateInfo {
            .type = XR_TYPE_VIEW_LOCATE_INFO,
            .viewConfigurationType = viewConfig,
            .displayTime = frameState.predictedDisplayTime,
            .space = sceneSpace,
        };

        XrViewState viewState{
            .type = XR_TYPE_VIEW_STATE,
        };
        XrView views[2] = { {.type = XR_TYPE_VIEW}, {.type = XR_TYPE_VIEW}};
        uint32_t viewCountOutput;
        CHK_XR(xrLocateViews(session, &viewLocateInfo, &viewState, configViews.size(),
&viewCountOutput, views));

        // ...
        // Use viewState and frameState for scene render, and fill in projViews[2]
        // ...
```

```
        // Assemble composition layers structure
        layerProj.layerFlags = XR_COMPOSITION_LAYER_BLEND_TEXTURE_SOURCE_ALPHA_BIT;
        layerProj.space = sceneSpace;
        layerProj.viewCount = 2;
        layerProj.views = projViews;
        layers.push_back(reinterpret_cast<XrCompositionLayerBaseHeader*>(&layerProj));
    }

    // End frame and submit layers, even if layers is empty due to shouldRender = false
    XrFrameEndInfo frameEndInfo{
        .type = XR_TYPE_FRAME_END_INFO,
        .displayTime = frameState.predictedDisplayTime,
        .environmentBlendMode = XR_ENVIRONMENT_BLEND_MODE_OPAQUE,
        .layerCount = static_cast<uint32_t>(layers.size()),
        .layers = layers.data(),
    };
    CHK_XR(xrEndFrame(session, &frameEndInfo));
}
```

Chapter 9. Session

```
// Provided by XR_VERSION_1_0
XR_DEFINE_HANDLE(XrSession)
```

A session represents an application's intention to display XR content to the user.

9.1. Session Lifecycle

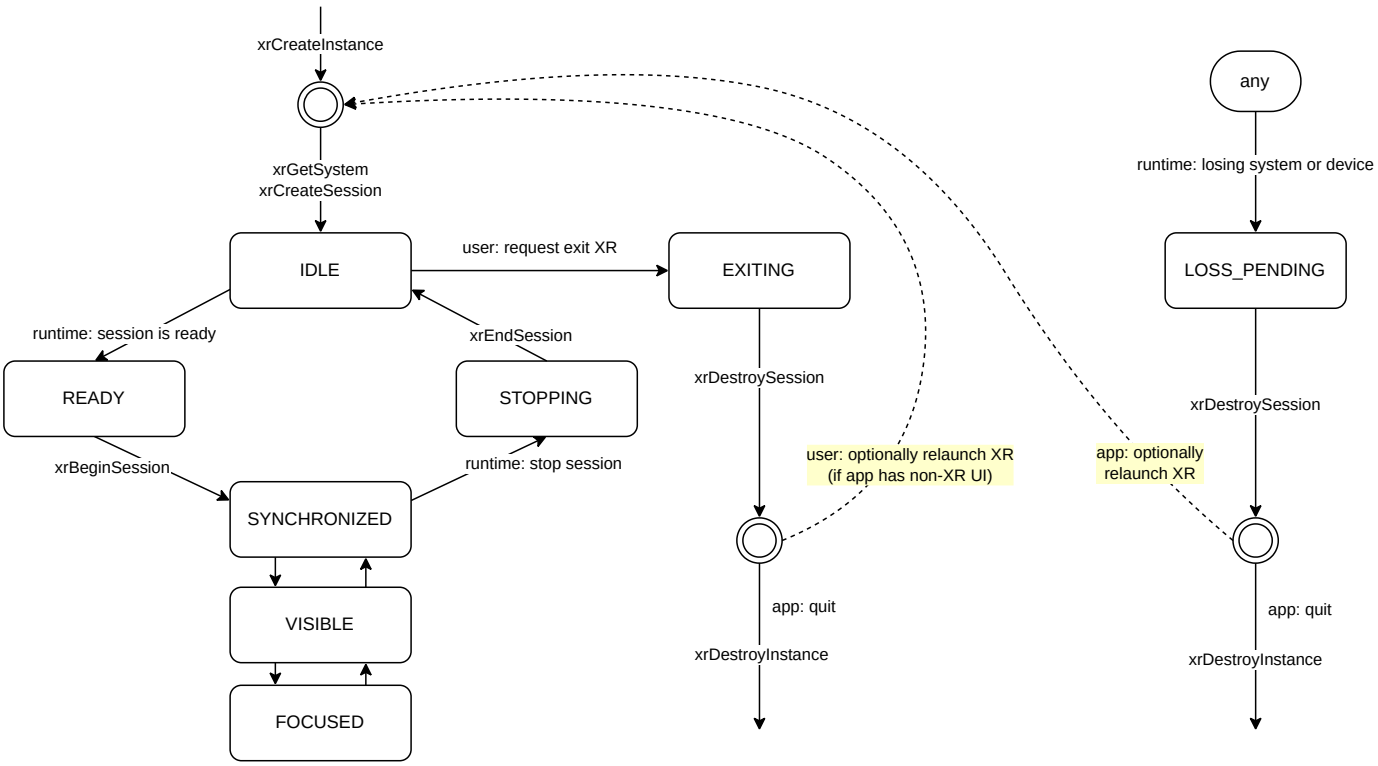


Figure 5. Session Lifecycle

A typical XR session coordinates the application and the runtime through session control functions and session state events.

1. The application creates a session by choosing a system and a graphics API and passing them into xrCreateSession. The newly created session is in the `XR_SESSION_STATE_IDLE` state.
2. The application can regularly call xrPollEvent to monitor for session state changes via XrEventDataSessionStateChanged events.
3. When the runtime determines that the system is ready to start transitioning to this session's XR content, the application receives a notification of session state change to `XR_SESSION_STATE_READY`. Once the application is also ready to proceed and display its XR content, it calls xrBeginSession and starts its frame loop, which

begins a running session.

4. While the session is running, the application is expected to continuously execute its frame loop by calling xrWaitFrame, xrBeginFrame and xrEndFrame each frame, establishing synchronization with the runtime. Once the runtime is synchronized with the application's frame loop and ready to display application's frames, the session moves into the `XR_SESSION_STATE_SYNCHRONIZED` state. In this state, the submitted frames will not be displayed or visible to the user yet.
5. When the runtime intends to display frames from the application, it notifies with `XR_SESSION_STATE_VISIBLE` state, and sets XrFrameState::`shouldRender` to `true` in xrWaitFrame. The application should render XR content and submit the composition layers to xrEndFrame.
6. When the runtime determines the application is eligible to receive XR inputs, e.g. motion controller or hand tracking inputs, it notifies with `XR_SESSION_STATE_FOCUSED` state. The application can expect to receive active action inputs.
7. When the runtime determines the application has lost XR input focus, it moves the session state from `XR_SESSION_STATE_FOCUSED` to `XR_SESSION_STATE_VISIBLE` state. The application may need to change its own internal state while input is unavailable. Since the session is still visible, the application needs to render and submit frames at full frame rate, but may wish to change visually to indicate its input suspended state. When the runtime returns XR focus back to the application, it moves the session state back to `XR_SESSION_STATE_FOCUSED`.
8. When the runtime needs to end a running session due to the user closing or switching the application, the runtime will change the session state through appropriate intermediate ones and finally to `XR_SESSION_STATE_STOPPING`. When the application receives the `XR_SESSION_STATE_STOPPING` event, it should stop its frame loop and then call xrEndSession to tell the runtime to stop the running session.
9. After xrEndSession, the runtime transitions the session state to `XR_SESSION_STATE_IDLE`. If the XR session is temporarily paused in the background, the runtime will keep the session state at `XR_SESSION_STATE_IDLE` and later transition the session state back to `XR_SESSION_STATE_READY` when the XR session is resumed. If the runtime determines that its use of this XR session has concluded, it will transition the session state from `XR_SESSION_STATE_IDLE` to `XR_SESSION_STATE_EXITING`.
10. When the application receives the `XR_SESSION_STATE_EXITING` event, it releases the resources related to the session and calls xrDestroySession.

A session is considered **running** after a successful call to xrBeginSession and remains running until any call is made to xrEndSession. Certain functions are only valid to call when a session is running, such as xrWaitFrame, or else the `XR_ERROR_SESSION_NOT_RUNNING` error **must** be returned by the runtime.

A session is considered **not running** before a successful call to xrBeginSession and becomes not

running again after any call is made to xrEndSession. Certain functions are only valid to call when a session is not running, such as xrBeginSession, or else the XR_ERROR_SESSION_RUNNING error **must** be returned by the runtime.

If an error is returned from xrBeginSession, the session remains in its current running or not running state. Calling xrEndSession always transitions a session to the not running state, regardless of any errors returned.

Only running sessions may become focused sessions that receive XR input. When a session is not running, the application **must** not submit frames. This is important because without a running session, the runtime no longer has to spend resources on sub-systems (tracking etc.) that are no longer needed by the application.

An application **must** call xrBeginSession when the session is in the XR_SESSION_STATE_READY state, or XR_ERROR_SESSION_NOT_READY will be returned; it **must** call xrEndSession when the session is in the XR_SESSION_STATE_STOPPING state, otherwise XR_ERROR_SESSION_NOT_STOPPING will be returned. This is to allow the runtimes to seamlessly transition from one application's session to another.

The application **can** call xrDestroySession at any time during the session life cycle, however, it **must** stop using the XrSession handle immediately in all threads and stop using any related resources. Therefore, it's typically undesirable to destroy a running session and instead it's recommended to wait for XR_SESSION_STATE_EXITING to destroy a session.

9.2. Session Creation

To present graphical content on an output device, OpenXR applications need to pick a graphics API which is supported by the runtime. Unextended OpenXR does not support any graphics APIs natively but provides a number of extensions of which each runtime can support any subset. These extensions can be activated during XrInstance create time.

During XrSession creation the application **must** provide information about which graphics API it intends to use by adding an XrGraphicsBinding* struct of one (and only one) of the enabled graphics API extensions to the next chain of XrSessionCreateInfo. The application **must** call the xrGet*GraphicsRequirements method (where * is a placeholder) provided by the chosen graphics API extension before attempting to create the session (for example, xrGetD3D11GraphicsRequirementsKHR xrGetD3D12GraphicsRequirementsKHR xrGetOpenGLGraphicsRequirementsKHR xrGetVulkanGraphicsRequirementsKHR xrGetVulkanGraphicsRequirements2KHR).

Unless specified differently in the graphics API extension, the application is responsible for creating a valid graphics device binding based on the requirements returned by xrGet*GraphicsRequirements methods (for details refer to the extension specification of the graphics API).

The xrCreateSession function is defined as:

```
// Provided by XR_VERSION_1_0
XrResult xrCreateSession(
    XrInstance                                  instance,
    const XrSessionCreateInfo*                  createInfo,
    XrSession*                                  session);
```

Parameter Descriptions

- `instance` is the instance from which XrSessionCreateInfo::`systemId` was retrieved.
- `createInfo` is a pointer to an XrSessionCreateInfo structure containing information about how to create the session.
- `session` is a pointer to a handle in which the created XrSession is returned.

Creates a session using the provided `createInfo` and returns a handle to that session. This session is created in the `XR_SESSION_STATE_IDLE` state, and a corresponding XrEventDataSessionStateChanged event to the `XR_SESSION_STATE_IDLE` state **must** be generated as the first such event for the new session.

The runtime **must** return `XR_ERROR_GRAPHICS_REQUIREMENTS_CALL_MISSING` (`XR_ERROR_VALIDATION_FAILURE` may be returned due to legacy behavior) on calls to xrCreateSession if a function named like `xrGet*GraphicsRequirements` has not been called for the same `instance` and XrSessionCreateInfo::`systemId`. (See graphics binding extensions for details.)

Valid Usage (Implicit)

- `instance` **must** be a valid XrInstance handle
- `createInfo` **must** be a pointer to a valid XrSessionCreateInfo structure
- `session` **must** be a pointer to an XrSession handle

Return Codes

Success

- XR_SUCCESS

Failure

- XR_ERROR_GRAPHICS_DEVICE_INVALID
- XR_ERROR_GRAPHICS_REQUIREMENTS_CALL_MISSING
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INITIALIZATION_FAILED
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SYSTEM_INVALID
- XR_ERROR_VALIDATION_FAILURE

The XrSessionCreateInfo structure is defined as:

```
// Provided by XR_VERSION_1_0
typedef struct XrSessionCreateInfo {
    XrStructureType         type;
    const void*             next;
    XrSessionCreateFlags    createFlags;
    XrSystemId              systemId;
} XrSessionCreateInfo;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR. Note that in most cases one graphics API extension specific struct needs to be in this next chain.
- createFlags identifies XrSessionCreateFlags that apply to the creation.
- systemId is the XrSystemId representing the system of devices to be used by this session.

Valid Usage

- `systemId` **must** be a valid `XrSystemId` or `XR_ERROR_SYSTEM_INVALID` **must** be returned.
- `next`, unless otherwise specified via an extension, **must** contain exactly one graphics API binding structure (a structure whose name begins with "XrGraphicsBinding") or `XR_ERROR_GRAPHICS_DEVICE_INVALID` **must** be returned.

Valid Usage (Implicit)

- `type` **must** be `XR_TYPE_SESSION_CREATE_INFO`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain. See also: XrGraphicsBindingD3D11KHR, XrGraphicsBindingD3D12KHR, XrGraphicsBindingEGLMNDX, XrGraphicsBindingMetalKHR, XrGraphicsBindingOpenGLESAndroidKHR, XrGraphicsBindingOpenGLWaylandKHR, XrGraphicsBindingOpenGLWin32KHR, XrGraphicsBindingOpenGLXcbKHR, XrGraphicsBindingOpenGLXlibKHR, XrGraphicsBindingVulkanKHR, XrHolographicWindowAttachmentMSFT, XrSessionCreateInfoOverlayEXTX
- `createFlags` **must** be `0`

The XrSessionCreateInfo::`createFlags` member is of the following type, and contains a bitwise-OR of zero or more of the bits defined in XrSessionCreateFlagBits.

```
// Provided by XR_VERSION_1_0
typedef XrFlags64 XrSessionCreateFlags;
```

Valid bits for XrSessionCreateFlags are defined by XrSessionCreateFlagBits.

```
// Provided by XR_VERSION_1_0
// Flag bits for XrSessionCreateFlags
```

There are currently no session creation flags. This is reserved for future use.

The xrDestroySession function is defined as.

```
// Provided by XR_VERSION_1_0
XrResult xrDestroySession(
    XrSession                                   session);
```

Parameter Descriptions

- `session` is the session to destroy.

XrSession handles are destroyed using xrDestroySession. When an XrSession is destroyed, all handles that are children of that XrSession are also destroyed.

The application is responsible for ensuring that it has no calls using `session` in progress when the session is destroyed.

xrDestroySession can be called when the session is in any session state.

Valid Usage (Implicit)

- `session` **must** be a valid XrSession handle

Thread Safety

- Access to `session`, and any child handles, **must** be externally synchronized

Return Codes

Success

- `XR_SUCCESS`

Failure

- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_RUNTIME_FAILURE`

9.3. Session Control

The xrBeginSession function is defined as:

```
// Provided by XR_VERSION_1_0
XrResult xrBeginSession(
    XrSession                                   session,
    const XrSessionBeginInfo*                   beginInfo);
```

Parameter Descriptions

- `session` is a valid XrSession handle.
- `beginInfo` is a pointer to an XrSessionBeginInfo structure.

When the application receives XrEventDataSessionStateChanged event with the `XR_SESSION_STATE_READY` state, the application **should** then call xrBeginSession to start rendering frames for display to the user.

After this function successfully returns, the session is considered to be running. The application **should** then start its frame loop consisting of some sequence of xrWaitFrame/xrBeginFrame /xrEndFrame calls.

If the session is already running when the application calls xrBeginSession, the runtime **must** return error `XR_ERROR_SESSION_RUNNING`. If the session is not running when the application calls xrBeginSession, but the session is not yet in the `XR_SESSION_STATE_READY` state, the runtime **must** return error `XR_ERROR_SESSION_NOT_READY`.

Note that a runtime **may** decide not to show the user any given frame from a session at any time, for example if the user has switched to a different application's running session. The application should check whether xrWaitFrame returns XrFrameState::`shouldRender` set to true before rendering a given frame to determine whether that frame will be visible to the user.

Runtime session frame state **must** start in a reset state when a session transitions to running so that no state is carried over from when the same session was previously running. Frame state in this context includes xrWaitFrame, xrBeginFrame, and xrEndFrame call order enforcement.

If XrSessionBeginInfo::`primaryViewConfigurationType` in `beginInfo` is not supported by the `XrSystemId` used to create the `session`, the runtime **must** return `XR_ERROR_VIEW_CONFIGURATION_TYPE_UNSUPPORTED`.

Valid Usage (Implicit)

- `session` **must** be a valid XrSession handle
- `beginInfo` **must** be a pointer to a valid XrSessionBeginInfo structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_SESSION_NOT_READY
- XR_ERROR_SESSION_RUNNING
- XR_ERROR_VALIDATION_FAILURE
- XR_ERROR_VIEW_CONFIGURATION_TYPE_UNSUPPORTED

The XrSessionBeginInfo structure is defined as:

```
// Provided by XR_VERSION_1_0
typedef struct XrSessionBeginInfo {
    XrStructureType            type;
    const void*                next;
    XrViewConfigurationType    primaryViewConfigurationType;
} XrSessionBeginInfo;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR.
- primaryViewConfigurationType is the XrViewConfigurationType to use during this session to provide images for the form factor's primary displays.

Valid Usage (Implicit)

- `type` **must** be `XR_TYPE_SESSION_BEGIN_INFO`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain. See also: XrSecondaryViewConfigurationSessionBeginInfoMSFT
- `primaryViewConfigurationType` **must** be a valid XrViewConfigurationType value

The xrEndSession function is defined as:

```
// Provided by XR_VERSION_1_0
XrResult xrEndSession(
    XrSession                                   session);
```

Parameter Descriptions

- `session` is a handle to a running XrSession.

When the application receives XrEventDataSessionStateChanged event with the `XR_SESSION_STATE_STOPPING` state, the application should stop its frame loop and then call xrEndSession to end the running session. This function signals to the runtime that the application will no longer call xrWaitFrame, xrBeginFrame or xrEndFrame from any thread allowing the runtime to safely transition the session to `XR_SESSION_STATE_IDLE`. The application **must** also avoid reading input state or sending haptic output after calling xrEndSession.

If the session is not running when the application calls xrEndSession, the runtime **must** return error `XR_ERROR_SESSION_NOT_RUNNING`. If the session is still running when the application calls xrEndSession, but the session is not yet in the `XR_SESSION_STATE_STOPPING` state, the runtime **must** return error `XR_ERROR_SESSION_NOT_STOPPING`.

If the application wishes to exit a running session, the application can call xrRequestExitSession so that the session transitions from `XR_SESSION_STATE_IDLE` to `XR_SESSION_STATE_EXITING`.

Valid Usage (Implicit)

- `session` **must** be a valid XrSession handle

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_SESSION_NOT_RUNNING
- XR_ERROR_SESSION_NOT_STOPPING
- XR_ERROR_VALIDATION_FAILURE

When an application wishes to exit a running session, it **can** call xrRequestExitSession, requesting that the runtime transition through the various intermediate session states including XR_SESSION_STATE_STOPPING to XR_SESSION_STATE_EXITING.

On platforms where an application's lifecycle is managed by the system, session state changes may be implicitly triggered by application lifecycle state changes. On such platforms, using platform-specific methods to alter application lifecycle state may be the preferred method of provoking session state changes. The behavior of xrRequestExitSession is not altered, however explicit session exit **may** not interact with the platform-specific application lifecycle.

The xrRequestExitSession function is defined as:

```
// Provided by XR_VERSION_1_0
XrResult xrRequestExitSession(
    XrSession                                   session);
```

Parameter Descriptions

- session is a handle to a running XrSession.

If session is not running when xrRequestExitSession is called, XR_ERROR_SESSION_NOT_RUNNING **must** be returned.

Valid Usage (Implicit)

- `session` **must** be a valid XrSession handle

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_SESSION_NOT_RUNNING`
- `XR_ERROR_VALIDATION_FAILURE`

9.4. Session States

While events can be expanded upon, there are a minimum set of lifecycle events which can occur which all OpenXR applications must be aware of. These events are detailed below.

9.4.1. XrEventDataSessionStateChanged

The XrEventDataSessionStateChanged structure is defined as:

```
// Provided by XR_VERSION_1_0
typedef struct XrEventDataSessionStateChanged {
    XrStructureType    type;
    const void*        next;
    XrSession          session;
    XrSessionState     state;
    XrTime             time;
} XrEventDataSessionStateChanged;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR.
- `session` is the XrSession which has changed state.
- `state` is the current XrSessionState of the `session`.
- `time` is an `XrTime` which indicates the time of the state change.

Receiving the XrEventDataSessionStateChanged event structure indicates that the application has changed lifecycle state.

Valid Usage (Implicit)

- `type` **must** be `XR_TYPE_EVENT_DATA_SESSION_STATE_CHANGED`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

The XrSessionState enumerates the possible session lifecycle states:

```
// Provided by XR_VERSION_1_0
typedef enum XrSessionState {
    XR_SESSION_STATE_UNKNOWN = 0,
    XR_SESSION_STATE_IDLE = 1,
    XR_SESSION_STATE_READY = 2,
    XR_SESSION_STATE_SYNCHRONIZED = 3,
    XR_SESSION_STATE_VISIBLE = 4,
    XR_SESSION_STATE_FOCUSED = 5,
    XR_SESSION_STATE_STOPPING = 6,
    XR_SESSION_STATE_LOSS_PENDING = 7,
    XR_SESSION_STATE_EXITING = 8,
    XR_SESSION_STATE_MAX_ENUM = 0x7FFFFFFF
} XrSessionState;
```

Enumerant Descriptions

- XR_SESSION_STATE_UNKNOWN. An unknown state. The runtime **must** not return this value in an XrEventDataSessionStateChanged event.
- XR_SESSION_STATE_IDLE. The initial state after calling xrCreateSession or returned to after calling xrEndSession.
- XR_SESSION_STATE_READY. The application is ready to call xrBeginSession and sync its frame loop with the runtime.
- XR_SESSION_STATE_SYNCHRONIZED. The application has synced its frame loop with the runtime but is not visible to the user.
- XR_SESSION_STATE_VISIBLE. The application has synced its frame loop with the runtime and is visible to the user but cannot receive XR input.
- XR_SESSION_STATE_FOCUSED. The application has synced its frame loop with the runtime, is visible to the user and can receive XR input.
- XR_SESSION_STATE_STOPPING. The application should exit its frame loop and call xrEndSession.
- XR_SESSION_STATE_LOSS_PENDING. The session is in the process of being lost. The application should destroy the current session and can optionally recreate it.
- XR_SESSION_STATE_EXITING. The application should end its XR experience and not automatically restart it.

The XR_SESSION_STATE_UNKNOWN state **must** not be returned by the runtime, and is only defined to avoid 0 being a valid state.

Receiving the XR_SESSION_STATE_IDLE state indicates that the runtime considers the session is idle. Applications in this state **should** minimize resource consumption but continue to call xrPollEvent at some reasonable cadence.

Receiving the XR_SESSION_STATE_READY state indicates that the runtime desires the application to prepare rendering resources, begin its session and synchronize its frame loop with the runtime.

The application does this by successfully calling xrBeginSession and then running its frame loop by calling xrWaitFrame, xrBeginFrame and xrEndFrame in a loop. If the runtime wishes to return the session to the XR_SESSION_STATE_IDLE state, it **must** wait until the application calls xrBeginSession. After returning from the xrBeginSession call, the runtime may then immediately transition forward through the XR_SESSION_STATE_SYNCHRONIZED state to the XR_SESSION_STATE_STOPPING state, to request that the application end this session. If the system supports a user engagement sensor and runtime is in XR_SESSION_STATE_IDLE state, the runtime **may** wait until the user starts engaging with the device before transitioning to the XR_SESSION_STATE_READY state.

Receiving the XR_SESSION_STATE_SYNCHRONIZED state indicates that the application has synchronized its frame loop with the runtime, but its frames are not visible to the user. The application **should** continue

running its frame loop by calling xrWaitFrame, xrBeginFrame and xrEndFrame, although it should avoid heavy GPU work so that other visible applications can take CPU and GPU precedence. The application can save resources here by skipping rendering and not submitting any composition layers until xrWaitFrame returns an XrFrameState with `shouldRender` set to true. A runtime **may** use this frame synchronization to facilitate seamless switching from a previous XR application to this application on a frame boundary.

Receiving the `XR_SESSION_STATE_VISIBLE` state indicates that the application has synchronized its frame loop with the runtime, and the session's frames will be visible to the user, but the session is not eligible to receive XR input. An application may be visible but not have focus, for example when the runtime is composing a modal pop-up on top of the application's rendered frames. The application **should** continue running its frame loop, rendering and submitting its composition layers, although it may wish to pause its experience, as users cannot interact with the application at this time. It is important for applications to continue rendering when visible, even when they do not have focus, so the user continues to see something reasonable underneath modal pop-ups. Runtimes **should** make input actions inactive while the application is unfocused, and applications should react to an inactive input action by skipping rendering of that action's input avatar (depictions of hands or other tracked objects controlled by the user).

Receiving the `XR_SESSION_STATE_FOCUSED` state indicates that the application has synchronized its frame loop with the runtime, the session's frames will be visible to the user, and the session is eligible to receive XR input. The runtime **should** only give one session XR input focus at any given time. The application **should** be running its frame loop, rendering and submitting composition layers, including input avatars (depictions of hands or other tracked objects controlled by the user) for any input actions that are active. The runtime **should** avoid rendering its own input avatars when an application is focused, unless input from a given source is being captured by the runtime at the moment.

Receiving the `XR_SESSION_STATE_STOPPING` state indicates that the runtime has determined that the application should halt its rendering loop. Applications **should** exit their rendering loop and call xrEndSession when in this state. A possible reason for this would be to minimize contention between multiple applications. If the system supports a user engagement sensor and the session is running, the runtime **may** transition to the `XR_SESSION_STATE_STOPPING` state when the user stops engaging with the device.

Receiving the `XR_SESSION_STATE_EXITING` state indicates the runtime wishes the application to terminate its XR experience, typically due to a user request via a runtime user interface. Applications **should** gracefully end their process when in this state if they do not have a non-XR user experience.

Receiving the `XR_SESSION_STATE_LOSS_PENDING` state indicates the runtime is no longer able to operate with the current session, for example due to the loss of a display hardware connection. An application **should** call xrDestroySession and **may** end its process or decide to poll xrGetSystem at some reasonable cadence to get a new `XrSystemId`, and re-initialize all graphics resources related to the new system, and then create a new session using xrCreateSession. After the event is queued, subsequent calls to functions that accept XrSession parameters **must** no longer return any success code other than `XR_SESSION_LOSS_PENDING` for the given XrSession handle. The `XR_SESSION_LOSS_PENDING` success result is returned for an unspecified grace period of time, and the functions that return it simulate success in

their behavior. If the runtime has no reasonable way to successfully complete a given function (e.g. xrCreateSwapchain) when a lost session is pending, or if the runtime is not able to provide the application a grace period, the runtime **may** return `XR_ERROR_SESSION_LOST`. Thereafter, functions which accept XrSession parameters for the lost session **may** return `XR_ERROR_SESSION_LOST` to indicate that the function failed and the given session was lost. The XrSession handle and child handles are henceforth unusable and **should** be destroyed by the application in order to immediately free up resources associated with those handles.

Chapter 10. Rendering

10.1. Swapchain Image Management

```
// Provided by XR_VERSION_1_0
XR_DEFINE_HANDLE(XrSwapchain)
```

Most XR applications present rendered images to the user. To allow this, the runtime provides collections of images organized in "swapchains" for the application to render into and submit. Note that these do not necessarily correspond to objects defined by any given graphics API named "swapchains". The runtime **must** allow applications to create multiple swapchains.

Swapchain image format support by the runtime is reported through use of the xrEnumerateSwapchainFormats function.

Swapchain images **can** be 2D or 2D Array.

Rendering operations involving composition of submitted layers are assumed to be internally performed by the runtime in linear color space. Images intended to be interpreted as being non-linear-encoded ("sRGB") **must** be created using an API-specific "sRGB" format (e.g. `DXGI_FORMAT_R8G8B8A8_UNORM_SRGB`, `GL_SRGB8_ALPHA8`, `VK_FORMAT_R8G8B8A8_SRGB`) to signal the need for sRGB-to-linear conversion (whether automatic or manual) when sampled by the runtime. All other formats will be treated as linear values.

OpenXR applications **should** avoid submitting linear encoded 8 bit color data (e.g. `DXGI_FORMAT_R8G8B8A8_UNORM`) whenever possible as it **may** result in color banding.



Note

For additional information, see:

Gritz, L. and d'Eon, E. 2007. The Importance of Being Linear. In: H. Nguyen, ed., *GPU Gems 3*. Addison-Wesley Professional. https://developer.nvidia.com/gpugems/gpugems3/part-iv-image-effects/chapter-24-importance-being-linear



Note

DXGI resources will be created with their associated TYPELESS format, but the runtime will use the application-specified format for reading the data.

The xrEnumerateSwapchainFormats function is defined as:

```
// Provided by XR_VERSION_1_0
XrResult xrEnumerateSwapchainFormats(
    XrSession                                   session,
    uint32_t                                    formatCapacityInput,
    uint32_t*                                   formatCountOutput,
    int64_t*                                    formats);
```

Parameter Descriptions

- `session` is the session that enumerates the supported formats.
- `formatCapacityInput` is the capacity of the `formats`, or 0 to retrieve the required capacity.
- `formatCountOutput` is a pointer to the count of `uint64_t` formats written, or a pointer to the required capacity in the case that `formatCapacityInput` is insufficient.
- `formats` is a pointer to an array of `int64_t` format IDs, but **can** be `NULL` if `formatCapacityInput` is `0`. The meanings of the format IDs are specific to the specified graphics API for the session.
- See the Buffer Size Parameters section for a detailed description of retrieving the required `formats` size.

xrEnumerateSwapchainFormats enumerates the texture formats supported by the current session. The type of formats returned are dependent on the graphics API specified by the graphics binding structure passed to xrCreateSession. For example, if a DirectX graphics API was specified, then the enumerated formats correspond to the DXGI formats, such as `DXGI_FORMAT_R8G8B8A8_UNORM_SRGB`. Texture formats **should** be in order from highest to lowest runtime preference. The application **should** use the highest preference format that it supports for optimal performance and quality.

Runtimes **should** support `R8G8B8A8` and `R8G8B8A8` formats with non-linear ("sRGB") encoding if possible.

With an OpenGL-based graphics API, the texture formats correspond to OpenGL internal formats.

With a Direct3D-based graphics API, xrEnumerateSwapchainFormats never returns typeless formats (e.g. `DXGI_FORMAT_R8G8B8A8_TYPELESS`). Only concrete formats are returned, and only concrete formats **may** be specified by applications for swapchain creation.

Runtimes **must** always return identical buffer contents from this enumeration for the lifetime of the session.

Valid Usage (Implicit)

- `session` **must** be a valid XrSession handle
- `formatCountOutput` **must** be a pointer to a `uint32_t` value
- If `formatCapacityInput` is not `0`, `formats` **must** be a pointer to an array of `formatCapacityInput` `int64_t` values

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_SIZE_INSUFFICIENT`
- `XR_ERROR_VALIDATION_FAILURE`

The xrCreateSwapchain function is defined as:

```
// Provided by XR_VERSION_1_0
XrResult xrCreateSwapchain(
    XrSession                                   session,
    const XrSwapchainCreateInfo*                createInfo,
    XrSwapchain*                                swapchain);
```

Parameter Descriptions

- `session` is the session that creates the image.
- `createInfo` is a pointer to an XrSwapchainCreateInfo structure containing parameters to be used to create the image.
- `swapchain` is a pointer to a handle in which the created XrSwapchain is returned.

Creates an XrSwapchain handle. The returned swapchain handle **may** be subsequently used in API

calls. Multiple XrSwapchain handles **may** exist simultaneously, up to some limit imposed by the runtime. The XrSwapchain handle **must** be eventually freed via the xrDestroySwapchain function. The runtime **must** return `XR_ERROR_SWAPCHAIN_FORMAT_UNSUPPORTED` if the image format specified in the XrSwapchainCreateInfo is unsupported. The runtime **must** return `XR_ERROR_FEATURE_UNSUPPORTED` if any bit of the create or usage flags specified in the XrSwapchainCreateInfo is unsupported.

Valid Usage (Implicit)

- `session` **must** be a valid XrSession handle
- `createInfo` **must** be a pointer to a valid XrSwapchainCreateInfo structure
- `swapchain` **must** be a pointer to an XrSwapchain handle

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_FEATURE_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_LIMIT_REACHED`
- `XR_ERROR_OUT_OF_MEMORY`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_SWAPCHAIN_FORMAT_UNSUPPORTED`
- `XR_ERROR_VALIDATION_FAILURE`

The XrSwapchainCreateInfo structure is defined as:

```
// Provided by XR_VERSION_1_0
typedef struct XrSwapchainCreateInfo {
    XrStructureType           type;
    const void*               next;
    XrSwapchainCreateFlags    createFlags;
    XrSwapchainUsageFlags     usageFlags;
    int64_t                   format;
    uint32_t                  sampleCount;
    uint32_t                  width;
    uint32_t                  height;
    uint32_t                  faceCount;
    uint32_t                  arraySize;
    uint32_t                  mipCount;
} XrSwapchainCreateInfo;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR.
- `createFlags` is a bitmask of XrSwapchainCreateFlagBits describing additional properties of the swapchain.
- `usageFlags` is a bitmask of XrSwapchainUsageFlagBits describing the intended usage of the swapchain's images. The usage flags define how the corresponding graphics API objects are created. A mismatch **may** result in swapchain images that do not support the application's usage.
- `format` is a graphics API-specific texture format identifier. For example, if the graphics API specified in xrCreateSession is Vulkan, then this format is a Vulkan format such as `VK_FORMAT_R8G8B8A8_SRGB`. The format identifies the format that the runtime will interpret the texture as upon submission. Valid formats are indicated by xrEnumerateSwapchainFormats.
- `sampleCount` is the number of sub-data element samples in the image, **must** not be `0` or greater than the graphics API's maximum limit.
- `width` is the width of the image, **must** not be `0` or greater than the graphics API's maximum limit.
- `height` is the height of the image, **must** not be `0` or greater than the graphics API's maximum limit.
- `faceCount` is the number of faces, which **must** be either `6` (for cubemaps) or `1`.
- `arraySize` is the number of array layers in the image or `1` for a 2D image, **must** not be `0` or greater than the graphics API's maximum limit.
- `mipCount` describes the number of levels of detail available for minified sampling of the image, **must** not be `0` or greater than the graphics API's maximum limit.

Valid Usage (Implicit)

- `type` **must** be `XR_TYPE_SWAPCHAIN_CREATE_INFO`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain. See also: XrAndroidSurfaceSwapchainCreateInfoFB, XrSecondaryViewConfigurationSwapchainCreateInfoMSFT, XrSwapchainCreateInfoColorSpaceSONY, XrSwapchainCreateInfoFoveationFB, XrVulkanSwapchainCreateInfoMETA, XrVulkanSwapchainFormatListCreateInfoKHR
- `createFlags` **must** be `0` or a valid combination of XrSwapchainCreateFlagBits values
- `usageFlags` **must** be `0` or a valid combination of XrSwapchainUsageFlagBits values

The XrSwapchainCreateInfo::createFlags member is of the following type, and contains a bitwise-OR of zero or more of the bits defined in XrSwapchainCreateFlagBits.

```
// Provided by XR_VERSION_1_0
typedef XrFlags64 XrSwapchainCreateFlags;
```

Valid bits for XrSwapchainCreateFlags are defined by XrSwapchainCreateFlagBits, which is specified as:

```
// Provided by XR_VERSION_1_0
// Flag bits for XrSwapchainCreateFlags
static const XrSwapchainCreateFlags XR_SWAPCHAIN_CREATE_PROTECTED_CONTENT_BIT =
0x00000001;
static const XrSwapchainCreateFlags XR_SWAPCHAIN_CREATE_STATIC_IMAGE_BIT = 0x00000002;
```

The flag bits have the following meanings:

Flag Descriptions

- XR_SWAPCHAIN_CREATE_PROTECTED_CONTENT_BIT indicates that the swapchain's images will be protected from CPU access, using a mechanism such as Vulkan protected memory.
- XR_SWAPCHAIN_CREATE_STATIC_IMAGE_BIT indicates that the application will acquire and release only one image to this swapchain over its entire lifetime. The runtime **must** allocate only one swapchain image.

A runtime **may** implement any of these, but is not required to. A runtime **must** return XR_ERROR_FEATURE_UNSUPPORTED from xrCreateSwapchain if an XrSwapchainCreateFlags bit is requested but not implemented.

XrSwapchainUsageFlags specify the intended usage of the swapchain images. The XrSwapchainCreateInfo::usageFlags member is of this type, and contains a bitwise-OR of one or more of the bits defined in XrSwapchainUsageFlagBits.

```
// Provided by XR_VERSION_1_0
typedef XrFlags64 XrSwapchainUsageFlags;
```

When images are created, the runtime needs to know how the images are used in a way that requires more information than simply the image format. The XrSwapchainCreateInfo passed to xrCreateSwapchain **must** match the intended usage.

Flags include:

```
// Provided by XR_VERSION_1_0
// Flag bits for XrSwapchainUsageFlags
static const XrSwapchainUsageFlags XR_SWAPCHAIN_USAGE_COLOR_ATTACHMENT_BIT = 0x00000001;
static const XrSwapchainUsageFlags XR_SWAPCHAIN_USAGE_DEPTH_STENCIL_ATTACHMENT_BIT =
0x00000002;
static const XrSwapchainUsageFlags XR_SWAPCHAIN_USAGE_UNORDERED_ACCESS_BIT = 0x00000004;
static const XrSwapchainUsageFlags XR_SWAPCHAIN_USAGE_TRANSFER_SRC_BIT = 0x00000008;
static const XrSwapchainUsageFlags XR_SWAPCHAIN_USAGE_TRANSFER_DST_BIT = 0x00000010;
static const XrSwapchainUsageFlags XR_SWAPCHAIN_USAGE_SAMPLED_BIT = 0x00000020;
static const XrSwapchainUsageFlags XR_SWAPCHAIN_USAGE_MUTABLE_FORMAT_BIT = 0x00000040;
  // Provided by XR_MND_swapchain_usage_input_attachment_bit
static const XrSwapchainUsageFlags XR_SWAPCHAIN_USAGE_INPUT_ATTACHMENT_BIT_MND =
0x00000080;
  // Provided by XR_KHR_swapchain_usage_input_attachment_bit
static const XrSwapchainUsageFlags XR_SWAPCHAIN_USAGE_INPUT_ATTACHMENT_BIT_KHR =
0x00000080;  // alias of XR_SWAPCHAIN_USAGE_INPUT_ATTACHMENT_BIT_MND
```

The flag bits have the following meanings:

Flag Descriptions

- XR_SWAPCHAIN_USAGE_COLOR_ATTACHMENT_BIT — Specifies that the image **may** be a color rendering target.
- XR_SWAPCHAIN_USAGE_DEPTH_STENCIL_ATTACHMENT_BIT — Specifies that the image **may** be a depth/stencil rendering target.
- XR_SWAPCHAIN_USAGE_UNORDERED_ACCESS_BIT — Specifies that the image **may** be accessed out of order and that access **may** be via atomic operations.
- XR_SWAPCHAIN_USAGE_TRANSFER_SRC_BIT — Specifies that the image **may** be used as the source of a transfer operation.
- XR_SWAPCHAIN_USAGE_TRANSFER_DST_BIT — Specifies that the image **may** be used as the destination of a transfer operation.
- XR_SWAPCHAIN_USAGE_SAMPLED_BIT — Specifies that the image **may** be sampled by a shader.
- XR_SWAPCHAIN_USAGE_MUTABLE_FORMAT_BIT — Specifies that the image **may** be reinterpreted as another image format.
- XR_SWAPCHAIN_USAGE_INPUT_ATTACHMENT_BIT_MND — Specifies that the image **may** be used as a input attachment. (Added by the XR_MND_swapchain_usage_input_attachment_bit extension)
- XR_SWAPCHAIN_USAGE_INPUT_ATTACHMENT_BIT_KHR — Specifies that the image **may** be used as a input attachment. (Added by the XR_KHR_swapchain_usage_input_attachment_bit extension)

The number of images in each swapchain is implementation-defined except in the case of a static

swapchain. To obtain the number of images actually allocated, call xrEnumerateSwapchainImages.

With a Direct3D-based graphics API, the swapchain returned by xrCreateSwapchain will be a typeless format if the requested format has a typeless analogue. Applications are required to reinterpret the swapchain as a compatible non-typeless type. Upon submitting such swapchains to the runtime, they are interpreted as the format specified by the application in the XrSwapchainCreateInfo.

Swapchains will be created with graphics API-specific flags appropriate to the type of underlying image and its usage.

Runtimes **must** honor underlying graphics API limits when creating resources.

xrEnumerateSwapchainFormats never returns typeless formats (e.g. DXGI_FORMAT_R8G8B8A8_TYPELESS). Only concrete formats are returned, and only concrete formats **may** be specified by applications for swapchain creation.

The xrDestroySwapchain function is defined as:

```
// Provided by XR_VERSION_1_0
XrResult xrDestroySwapchain(
    XrSwapchain                                 swapchain);
```

Parameter Descriptions

- swapchain is the swapchain to destroy.

All submitted graphics API commands that refer to swapchain **must** have completed execution before calling this function.

Note

Note that in some graphics APIs, this implies that the application needs to perform a CPU wait on a synchronization primitive or perform a queue flush before calling this function.

Runtimes **may** continue to utilize swapchain images after xrDestroySwapchain is called.

Valid Usage (Implicit)

- swapchain **must** be a valid XrSwapchain handle

Thread Safety

- Access to `swapchain`, and any child handles, **must** be externally synchronized

Return Codes

Success

- `XR_SUCCESS`

Failure

- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_RUNTIME_FAILURE`

Swapchain images are acquired, waited on, and released by index, but the number of images in a swapchain is implementation-defined. Additionally, rendering to images requires access to the underlying image primitive of the graphics API being used. Applications **may** query and cache the images at any time after swapchain creation.

The xrEnumerateSwapchainImages function is defined as:

```
// Provided by XR_VERSION_1_0
XrResult xrEnumerateSwapchainImages(
    XrSwapchain                                 swapchain,
    uint32_t                                    imageCapacityInput,
    uint32_t*                                   imageCountOutput,
    XrSwapchainImageBaseHeader*                 images);
```

Parameter Descriptions

- `swapchain` is the XrSwapchain to get images from.
- `imageCapacityInput` is the capacity of the `images` array, or 0 to indicate a request to retrieve the required capacity.
- `imageCountOutput` is a pointer to the count of `images` written, or a pointer to the required capacity in the case that `imageCapacityInput` is insufficient.
- `images` is a pointer to an array of graphics API-specific `XrSwapchainImage` structures, all of the same type, based on XrSwapchainImageBaseHeader. It **can** be `NULL` if `imageCapacityInput` is 0.
- See the Buffer Size Parameters section for a detailed description of retrieving the required `images` size.

Fills an array of graphics API-specific `XrSwapchainImage` structures. The resources **must** be constant and valid for the lifetime of the XrSwapchain.

Runtimes **must** always return identical buffer contents from this enumeration for the lifetime of the swapchain.

Note: `images` is a pointer to an array of structures of graphics API-specific type, not an array of structure pointers.

The pointer submitted as `images` will be treated as an array of the expected graphics API-specific type based on the graphics API used at session creation time. If the `type` member of any array element accessed in this way does not match the expected value, the runtime **must** return `XR_ERROR_VALIDATION_FAILURE`.



Note

Under a typical memory model, a runtime **must** treat the supplied pointer as an opaque blob beginning with XrSwapchainImageBaseHeader, until after it has verified the XrSwapchainImageBaseHeader::`type`.

Valid Usage (Implicit)

- `swapchain` **must** be a valid XrSwapchain handle
- `imageCountOutput` **must** be a pointer to a `uint32_t` value
- If `imageCapacityInput` is not `0`, `images` **must** be a pointer to an array of `imageCapacityInput` XrSwapchainImageBaseHeader-based structures. See also: XrSwapchainImageD3D11KHR, XrSwapchainImageD3D12KHR, XrSwapchainImageMetalKHR, XrSwapchainImageOpenGLESKHR, XrSwapchainImageOpenGLKHR, XrSwapchainImageVulkanKHR

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_SIZE_INSUFFICIENT
- XR_ERROR_VALIDATION_FAILURE

The XrSwapchainImageBaseHeader structure is defined as:

```
// Provided by XR_VERSION_1_0
typedef struct XrSwapchainImageBaseHeader {
    XrStructureType    type;
    void*              next;
} XrSwapchainImageBaseHeader;
```

Member Descriptions

- type is the XrStructureType of this structure. This base structure itself has no associated XrStructureType value.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR.

The XrSwapchainImageBaseHeader is a base structure that is extended by graphics API-specific XrSwapchainImage* child structures.

Valid Usage (Implicit)

- `type` **must** be one of the following XrStructureType values: `XR_TYPE_SWAPCHAIN_IMAGE_D3D11_KHR`, `XR_TYPE_SWAPCHAIN_IMAGE_D3D12_KHR`, `XR_TYPE_SWAPCHAIN_IMAGE_METAL_KHR`, `XR_TYPE_SWAPCHAIN_IMAGE_OPENGL_ES_KHR`, `XR_TYPE_SWAPCHAIN_IMAGE_OPENGL_KHR`, `XR_TYPE_SWAPCHAIN_IMAGE_VULKAN_KHR`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

Before an application builds graphics API command buffers that refer to an image in a swapchain, it **must** acquire the image from the swapchain. The acquire operation determines the index of the next image to be used in the swapchain. The order in which images are acquired is undefined. The runtime **must** allow the application to acquire more than one image from a single (non-static) swapchain at a time, for example if the application implements a multiple frame deep rendering pipeline.

The xrAcquireSwapchainImage function is defined as:

```
// Provided by XR_VERSION_1_0
XrResult xrAcquireSwapchainImage(
    XrSwapchain                                 swapchain,
    const XrSwapchainImageAcquireInfo*          acquireInfo,
    uint32_t*                                   index);
```

Parameter Descriptions

- `swapchain` is the swapchain from which to acquire an image.
- `acquireInfo` exists for extensibility purposes, it is `NULL` or a pointer to a valid XrSwapchainImageAcquireInfo.
- `index` is the returned image index that has been acquired.

Acquires the image corresponding to the `index` position in the array returned by xrEnumerateSwapchainImages. The runtime **must** return `XR_ERROR_CALL_ORDER_INVALID` if the next available index has already been acquired and not yet released with xrReleaseSwapchainImage. If the `swapchain` was created with the `XR_SWAPCHAIN_CREATE_STATIC_IMAGE_BIT` set in XrSwapchainCreateInfo::`createFlags`, this function **must** not have been previously called for this swapchain. The runtime **must** return `XR_ERROR_CALL_ORDER_INVALID` if a `swapchain` created with the `XR_SWAPCHAIN_CREATE_STATIC_IMAGE_BIT` set in XrSwapchainCreateInfo::`createFlags` and this function has been successfully called previously for this swapchain.

This function only provides the index of the swapchain image, for example for use in recording command buffers. It does not wait for the image to be usable by the application. The application **must** call xrWaitSwapchainImage for each "acquire" call before submitting graphics commands that write to

the image.

Valid Usage (Implicit)

- `swapchain` **must** be a valid XrSwapchain handle
- If `acquireInfo` is not `NULL`, `acquireInfo` **must** be a pointer to a valid XrSwapchainImageAcquireInfo structure
- `index` **must** be a pointer to a `uint32_t` value

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_CALL_ORDER_INVALID`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_VALIDATION_FAILURE`

The XrSwapchainImageAcquireInfo structure is defined as:

```
// Provided by XR_VERSION_1_0
typedef struct XrSwapchainImageAcquireInfo {
    XrStructureType    type;
    const void*        next;
} XrSwapchainImageAcquireInfo;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR.

Because this structure only exists to support extension-specific structures, xrAcquireSwapchainImage will accept a NULL argument for xrAcquireSwapchainImage::acquireInfo for applications that are not using any relevant extensions.

Valid Usage (Implicit)

- type **must** be XR_TYPE_SWAPCHAIN_IMAGE_ACQUIRE_INFO
- next **must** be NULL or a valid pointer to the next structure in a structure chain

The xrWaitSwapchainImage function is defined as:

```
// Provided by XR_VERSION_1_0
XrResult xrWaitSwapchainImage(
    XrSwapchain                                 swapchain,
    const XrSwapchainImageWaitInfo*             waitInfo);
```

Parameter Descriptions

- swapchain is the swapchain from which to wait for an image.
- waitInfo is a pointer to an XrSwapchainImageWaitInfo structure.

Before an application begins writing to a swapchain image, it **must** first wait on the image, to avoid writing to it before the compositor has finished reading from it. xrWaitSwapchainImage will implicitly wait on the oldest acquired swapchain image which has not yet been successfully waited on. Once a swapchain image has been successfully waited on without timeout, the app **must** release before waiting on the next acquired swapchain image.

This function **may** block for longer than the timeout specified in XrSwapchainImageWaitInfo due to scheduling or contention.

If the timeout expires without the image becoming available for writing, XR_TIMEOUT_EXPIRED **must** be returned. If xrWaitSwapchainImage returns XR_TIMEOUT_EXPIRED, the next call to xrWaitSwapchainImage will wait on the same image index again until the function succeeds with XR_SUCCESS. Note that this is not an error code; XR_SUCCEEDED(XR_TIMEOUT_EXPIRED) is true.

The runtime **must** eventually relinquish ownership of a swapchain image to the application and **must** not block indefinitely.

The runtime **must** return XR_ERROR_CALL_ORDER_INVALID if no image has been acquired by calling xrAcquireSwapchainImage.

Valid Usage (Implicit)

- `swapchain` **must** be a valid XrSwapchain handle
- `waitInfo` **must** be a pointer to a valid XrSwapchainImageWaitInfo structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING
- XR_TIMEOUT_EXPIRED

Failure

- XR_ERROR_CALL_ORDER_INVALID
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The XrSwapchainImageWaitInfo structure describes a swapchain image wait operation. It is defined as:

```
// Provided by XR_VERSION_1_0
typedef struct XrSwapchainImageWaitInfo {
    XrStructureType    type;
    const void*        next;
    XrDuration         timeout;
} XrSwapchainImageWaitInfo;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR.
- `timeout` indicates how many nanoseconds the call **may** block waiting for the image to become available for writing.

Valid Usage (Implicit)

- `type` **must** be `XR_TYPE_SWAPCHAIN_IMAGE_WAIT_INFO`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

Once an application is done submitting commands that reference the swapchain image, the application **must** release the swapchain image. xrReleaseSwapchainImage will implicitly release the oldest swapchain image which has been acquired. The swapchain image **must** have been successfully waited on without timeout before it is released. xrEndFrame will use the most recently released swapchain image. In each frame submitted to the compositor, only one image index from each swapchain will be used. Note that in case the swapchain contains 2D image arrays, one array is referenced per swapchain index and thus the whole image array **may** be used in one frame.

The xrReleaseSwapchainImage function is defined as:

```
// Provided by XR_VERSION_1_0
XrResult xrReleaseSwapchainImage(
    XrSwapchain                                 swapchain,
    const XrSwapchainImageReleaseInfo*          releaseInfo);
```

Parameter Descriptions

- `swapchain` is the XrSwapchain from which to release an image.
- `releaseInfo` exists for extensibility purposes, it is `NULL` or a pointer to a valid XrSwapchainImageReleaseInfo.

If the `swapchain` was created with the `XR_SWAPCHAIN_CREATE_STATIC_IMAGE_BIT` set in XrSwapchainCreateInfo::`createFlags` structure, this function **must** not have been previously called for this swapchain.

The runtime **must** return `XR_ERROR_CALL_ORDER_INVALID` if no image has been waited on by calling

xrWaitSwapchainImage.

Valid Usage (Implicit)

- `swapchain` **must** be a valid XrSwapchain handle
- If `releaseInfo` is not `NULL`, `releaseInfo` **must** be a pointer to a valid XrSwapchainImageReleaseInfo structure

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_CALL_ORDER_INVALID`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_VALIDATION_FAILURE`

The XrSwapchainImageReleaseInfo structure is defined as:

```
// Provided by XR_VERSION_1_0
typedef struct XrSwapchainImageReleaseInfo {
    XrStructureType    type;
    const void*        next;
} XrSwapchainImageReleaseInfo;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR.

Because this structure only exists to support extension-specific structures, xrReleaseSwapchainImage will accept a `NULL` argument for xrReleaseSwapchainImage::`releaseInfo` for applications that are not

using any relevant extensions.

Valid Usage (Implicit)

- `type` **must** be `XR_TYPE_SWAPCHAIN_IMAGE_RELEASE_INFO`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

10.2. View and Projection State

An application uses xrLocateViews to retrieve the viewer pose and projection parameters needed to render each view for use in a composition projection layer.

The xrLocateViews function is defined as:

```
// Provided by XR_VERSION_1_0
XrResult xrLocateViews(
    XrSession                                   session,
    const XrViewLocateInfo*                     viewLocateInfo,
    XrViewState*                                viewState,
    uint32_t                                    viewCapacityInput,
    uint32_t*                                   viewCountOutput,
    XrView*                                     views);
```

Parameter Descriptions

- `session` is a handle to the provided XrSession.
- `viewLocateInfo` is a pointer to a valid XrViewLocateInfo structure.
- `viewState` is the output structure with the viewer state information.
- `viewCapacityInput` is an input parameter which specifies the capacity of the `views` array. The required capacity **must** be same as defined by the corresponding XrViewConfigurationType.
- `viewCountOutput` is an output parameter which identifies the valid count of `views`.
- `views` is an array of XrView.
- See the Buffer Size Parameters section for a detailed description of retrieving the required `views` size.

The xrLocateViews function returns the view and projection info for a particular display time. This time is typically the target display time for a given frame. Repeatedly calling xrLocateViews with the same time **may** not necessarily return the same result. Instead the prediction gets increasingly accurate as the function is called closer to the given time for which a prediction is made. This allows an

application to get the predicted views as late as possible in its pipeline to get the least amount of latency and prediction error.

xrLocateViews returns an array of XrView elements, one for each view of the specified view configuration type, along with an XrViewState containing additional state data shared across all views. The eye each view corresponds to is statically defined in XrViewConfigurationType in case the application wants to apply eye-specific rendering traits. The XrViewState and XrView member data **may** change on subsequent calls to xrLocateViews, and so applications **must** not assume it to be constant.

If an application gives a `viewLocateInfo` with a XrViewLocateInfo::`viewConfigurationType` that was not passed in the session's call to xrBeginSession via the XrSessionBeginInfo::`primaryViewConfigurationType`, or enabled though an extension, then the runtime **must** return `XR_ERROR_VALIDATION_FAILURE`.

Valid Usage (Implicit)

- `session` **must** be a valid XrSession handle
- `viewLocateInfo` **must** be a pointer to a valid XrViewLocateInfo structure
- `viewState` **must** be a pointer to an XrViewState structure
- `viewCountOutput` **must** be a pointer to a `uint32_t` value
- If `viewCapacityInput` is not `0`, `views` **must** be a pointer to an array of `viewCapacityInput` XrView structures

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_SIZE_INSUFFICIENT
- XR_ERROR_TIME_INVALID
- XR_ERROR_VALIDATION_FAILURE
- XR_ERROR_VIEW_CONFIGURATION_TYPE_UNSUPPORTED

The XrViewLocateInfo structure is defined as:

```
// Provided by XR_VERSION_1_0
typedef struct XrViewLocateInfo {
    XrStructureType            type;
    const void*                next;
    XrViewConfigurationType    viewConfigurationType;
    XrTime                     displayTime;
    XrSpace                    space;
} XrViewLocateInfo;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain.
- viewConfigurationType is XrViewConfigurationType to query for.
- displayTime is the time for which the view poses are predicted.
- space is the XrSpace in which the pose in each XrView is expressed.

The XrViewLocateInfo structure contains the display time and space used to locate the view XrView structures.

The runtime **must** return error XR_ERROR_VIEW_CONFIGURATION_TYPE_UNSUPPORTED if the given viewConfigurationType is not one of the supported type reported by xrEnumerateViewConfigurations.

Valid Usage (Implicit)

- type **must** be XR_TYPE_VIEW_LOCATE_INFO
- next **must** be NULL or a valid pointer to the next structure in a structure chain. See also: XrViewLocateFoveatedRenderingVARJO
- viewConfigurationType **must** be a valid XrViewConfigurationType value
- space **must** be a valid XrSpace handle

The XrView structure is defined as:

```
// Provided by XR_VERSION_1_0
typedef struct XrView {
    XrStructureType    type;
    void*              next;
    XrPosef            pose;
    XrFovf             fov;
} XrView;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR.
- pose is an XrPosef defining the location and orientation of the view in the space specified by the xrLocateViews function.
- fov is the XrFovf for the four sides of the projection.

The XrView structure contains view pose and projection state necessary to render a single projection view in the view configuration.

Valid Usage (Implicit)

- type **must** be XR_TYPE_VIEW
- next **must** be NULL or a valid pointer to the next structure in a structure chain

The XrViewState structure is defined as:

```
// Provided by XR_VERSION_1_0
typedef struct XrViewState {
    XrStructureType     type;
    void*               next;
    XrViewStateFlags    viewStateFlags;
} XrViewState;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR.
- `viewStateFlags` is a bitmask of XrViewStateFlagBits indicating state for all views.

The XrViewState contains additional view state from xrLocateViews common to all views of the active view configuration.

Valid Usage (Implicit)

- `type` **must** be `XR_TYPE_VIEW_STATE`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `viewStateFlags` **must** be `0` or a valid combination of XrViewStateFlagBits values

The XrViewStateFlags specifies the validity and quality of the corresponding XrView array returned by xrLocateViews. The XrViewState::`viewStateFlags` member is of this type, and contains a bitwise-OR of zero or more of the bits defined in XrViewStateFlagBits.

```
// Provided by XR_VERSION_1_0
typedef XrFlags64 XrViewStateFlags;
```

Valid bits for XrViewStateFlags are defined by XrViewStateFlagBits, which is specified as:

```
// Provided by XR_VERSION_1_0
// Flag bits for XrViewStateFlags
static const XrViewStateFlags XR_VIEW_STATE_ORIENTATION_VALID_BIT = 0x00000001;
static const XrViewStateFlags XR_VIEW_STATE_POSITION_VALID_BIT = 0x00000002;
static const XrViewStateFlags XR_VIEW_STATE_ORIENTATION_TRACKED_BIT = 0x00000004;
static const XrViewStateFlags XR_VIEW_STATE_POSITION_TRACKED_BIT = 0x00000008;
```

The flag bits have the following meanings:

Flag Descriptions

- `XR_VIEW_STATE_ORIENTATION_VALID_BIT` indicates whether all XrView orientations contain valid data. Applications **must** not read any of the XrView `pose` `orientation` fields if this flag is unset. `XR_VIEW_STATE_ORIENTATION_TRACKED_BIT` **should** generally remain set when this bit is set for views on a tracked headset or handheld device.
- `XR_VIEW_STATE_POSITION_VALID_BIT` indicates whether all XrView positions contain valid data. Applications **must** not read any of the XrView::`pose` `position` fields if this flag is unset. When a view loses tracking, runtimes **should** continue to provide valid but untracked view `position` values that are inferred or last-known, so long as it's still meaningful for the application to render content using that position, clearing `XR_VIEW_STATE_POSITION_TRACKED_BIT` until tracking is recovered.
- `XR_VIEW_STATE_ORIENTATION_TRACKED_BIT` indicates whether all XrView orientations represent an actively tracked orientation. This bit **should** generally remain set when `XR_VIEW_STATE_ORIENTATION_VALID_BIT` is set for views on a tracked headset or handheld device.
- `XR_VIEW_STATE_POSITION_TRACKED_BIT` indicates whether all XrView positions represent an actively tracked position. When a view loses tracking, runtimes **should** continue to provide valid but untracked view `position` values that are inferred or last-known, e.g. based on neck model updates, inertial dead reckoning, or a last-known position, so long as it's still meaningful for the application to render content using that position.

10.3. Frame Synchronization

An application synchronizes its rendering loop to the runtime by calling xrWaitFrame.

The xrWaitFrame function is defined as:

```
// Provided by XR_VERSION_1_0
XrResult xrWaitFrame(
    XrSession                                   session,
    const XrFrameWaitInfo*                      frameWaitInfo,
    XrFrameState*                               frameState);
```

Parameter Descriptions

- `session` is a valid XrSession handle.
- `frameWaitInfo` exists for extensibility purposes, it is `NULL` or a pointer to a valid XrFrameWaitInfo.
- `frameState` is a pointer to an XrFrameState to populate, an output parameter.

xrWaitFrame throttles the application frame loop in order to synchronize application frame submissions with the display. xrWaitFrame returns a predicted display time for the next time that the runtime predicts a composited frame will be displayed. The runtime **may** affect this computation by changing the return values and throttling of xrWaitFrame in response to feedback from frame submission and completion times in xrEndFrame. A subsequent xrWaitFrame call **must** block until the previous frame has been begun with xrBeginFrame and **must** unblock independently of the corresponding call to xrEndFrame. Refer to xrBeginSession for details on how a transition to session running resets the frame function call order.

When less than one frame interval has passed since the previous return from xrWaitFrame, the runtime **should** block until the beginning of the next frame interval. If more than one frame interval has passed since the last return from xrWaitFrame, the runtime **may** return immediately or block until the beginning of the next frame interval.

In the case that an application has pipelined frame submissions, the application **should** compute the appropriate target display time using both the predicted display time and predicted display interval. The application **should** use the computed target display time when requesting space and view locations for rendering.

The XrFrameState::`predictedDisplayTime` returned by xrWaitFrame **must** be monotonically increasing.

The runtime **may** dynamically adjust the start time of the frame interval relative to the display hardware's refresh cycle to minimize graphics processor contention between the application and the compositor.

xrWaitFrame **must** be callable from any thread, including a different thread than xrBeginFrame /xrEndFrame are being called from.

Calling xrWaitFrame **must** be externally synchronized by the application, concurrent calls **may** result in undefined behavior.

The runtime **must** return `XR_ERROR_SESSION_NOT_RUNNING` if the `session` is not running.



Note

The engine simulation **should** advance based on the display time. Every stage in the engine pipeline **should** use the exact same display time for one particular application-generated frame. An accurate and consistent display time across all stages and threads in the engine pipeline is important to avoid object motion judder. If the application has multiple pipeline stages, the application **should** pass its computed display time through its pipeline, as xrWaitFrame **must** be called only once per frame.

Valid Usage (Implicit)

- `session` **must** be a valid XrSession handle
- If `frameWaitInfo` is not `NULL`, `frameWaitInfo` **must** be a pointer to a valid XrFrameWaitInfo structure
- `frameState` **must** be a pointer to an XrFrameState structure

Thread Safety

- Access to the `session` parameter by any other xrWaitFrame call **must** be externally synchronized

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_SESSION_NOT_RUNNING`
- `XR_ERROR_VALIDATION_FAILURE`

The XrFrameWaitInfo structure is defined as:

```
// Provided by XR_VERSION_1_0
typedef struct XrFrameWaitInfo {
    XrStructureType    type;
    const void*        next;
} XrFrameWaitInfo;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR.

Because this structure only exists to support extension-specific structures, xrWaitFrame **must** accept a `NULL` argument for xrWaitFrame::`frameWaitInfo` for applications that are not using any relevant extensions.

Valid Usage (Implicit)

- `type` **must** be `XR_TYPE_FRAME_WAIT_INFO`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

The XrFrameState structure is defined as:

```
// Provided by XR_VERSION_1_0
typedef struct XrFrameState {
    XrStructureType    type;
    void*              next;
    XrTime             predictedDisplayTime;
    XrDuration         predictedDisplayPeriod;
    XrBool32           shouldRender;
} XrFrameState;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR.
- `predictedDisplayTime` is the anticipated display `XrTime` for the next application-generated frame.
- `predictedDisplayPeriod` is the `XrDuration` of the display period for the next application-generated frame, for use in predicting display times beyond the next one.
- `shouldRender` is `XR_TRUE` if the application **should** render its layers as normal and submit them to xrEndFrame. When this value is `XR_FALSE`, the application **should** avoid heavy GPU work where possible, for example by skipping layer rendering and then omitting those layers when calling xrEndFrame.

XrFrameState describes the time at which the next frame will be displayed to the user. `predictedDisplayTime` **must** refer to the midpoint of the interval during which the frame is displayed. The runtime **may** report a different `predictedDisplayPeriod` from the hardware's refresh cycle.

For any frame where `shouldRender` is `XR_FALSE`, the application **should** avoid heavy GPU work for that frame, for example by not rendering its layers. This typically happens when the application is transitioning into or out of a running session, or when some system UI is fully covering the application at the moment. As long as the session is running, the application **should** keep running the frame loop to maintain the frame synchronization to the runtime, even if this requires calling xrEndFrame with all layers omitted.

Valid Usage (Implicit)

- `type` **must** be `XR_TYPE_FRAME_STATE`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain. See also: XrSecondaryViewConfigurationFrameStateMSFT

10.4. Frame Submission

Every application **must** call xrBeginFrame before calling xrEndFrame, and **should** call xrEndFrame before calling xrBeginFrame again. Calling xrEndFrame again without a prior call to xrBeginFrame **must** result in `XR_ERROR_CALL_ORDER_INVALID` being returned by xrEndFrame. An application **may** call xrBeginFrame again if the prior xrEndFrame fails or if the application wishes to discard an in-progress frame. A successful call to xrBeginFrame again with no intervening xrEndFrame call **must** result in the success code `XR_FRAME_DISCARDED` being returned from xrBeginFrame. In this case it is assumed that the xrBeginFrame refers to the next frame and the previously begun frame is forfeited by the application.

An application **may** call xrEndFrame without having called xrReleaseSwapchainImage since the previous call to xrEndFrame for any swapchain passed to xrEndFrame. Applications **should** call xrBeginFrame right before executing any graphics device work for a given frame, as opposed to calling it afterwards. The runtime **must** only compose frames whose xrBeginFrame and xrEndFrame both return success codes. While xrBeginFrame and xrEndFrame do not need to be called on the same thread, the application **must** handle synchronization if they are called on separate threads.

The xrBeginFrame function is defined as:

```
// Provided by XR_VERSION_1_0
XrResult xrBeginFrame(
    XrSession                                   session,
    const XrFrameBeginInfo*                     frameBeginInfo);
```

Parameter Descriptions

- `session` is a valid XrSession handle.
- `frameBeginInfo` exists for extensibility purposes, it is `NULL` or a pointer to a valid XrFrameBeginInfo.

xrBeginFrame is called prior to the start of frame rendering. The application **should** still call xrBeginFrame but omit rendering work for the frame if XrFrameState::`shouldRender` is `XR_FALSE`.

Runtimes **must** not perform frame synchronization or throttling through the xrBeginFrame function and **should** instead do so through xrWaitFrame.

The runtime **must** return the error code `XR_ERROR_CALL_ORDER_INVALID` if there was no corresponding successful call to xrWaitFrame. The runtime **must** return the success code `XR_FRAME_DISCARDED` if a prior xrBeginFrame has been called without an intervening call to xrEndFrame. Refer to xrBeginSession for details on how a transition to session running resets the frame function call order.

The runtime **must** return `XR_ERROR_SESSION_NOT_RUNNING` if the `session` is not running.

Valid Usage (Implicit)

- `session` **must** be a valid XrSession handle
- If `frameBeginInfo` is not `NULL`, `frameBeginInfo` **must** be a pointer to a valid XrFrameBeginInfo structure

Thread Safety

- Access to the `session` parameter by any other xrBeginFrame or xrEndFrame call **must** be externally synchronized

Return Codes

Success

- `XR_SUCCESS`
- `XR_FRAME_DISCARDED`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_CALL_ORDER_INVALID`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_SESSION_NOT_RUNNING`
- `XR_ERROR_VALIDATION_FAILURE`

The XrFrameBeginInfo structure is defined as:

```
// Provided by XR_VERSION_1_0
typedef struct XrFrameBeginInfo {
    XrStructureType    type;
    const void*        next;
} XrFrameBeginInfo;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR.

Because this structure only exists to support extension-specific structures, xrBeginFrame will accept a `NULL` argument for xrBeginFrame::`frameBeginInfo` for applications that are not using any relevant

extensions.

Valid Usage (Implicit)

- `type` **must** be `XR_TYPE_FRAME_BEGIN_INFO`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

The xrEndFrame function is defined as:

```
// Provided by XR_VERSION_1_0
XrResult xrEndFrame(
    XrSession                                   session,
    const XrFrameEndInfo*                       frameEndInfo);
```

Parameter Descriptions

- `session` is a valid XrSession handle.
- `frameEndInfo` is a pointer to a valid XrFrameEndInfo.

xrEndFrame **may** return immediately to the application. XrFrameEndInfo::`displayTime` **should** be computed using values returned by xrWaitFrame. The runtime **should** be robust against variations in the timing of calls to xrWaitFrame, since a pipelined system may call xrWaitFrame on a separate thread from xrBeginFrame and xrEndFrame without any synchronization guarantees.



Note

An accurate predicted display time is very important to avoid black pull-in by reprojection and to reduce motion judder in case the runtime does not implement a translational reprojection. Reprojection **should** never display images before the display refresh period they were predicted for, even if they are completed early, because this will cause motion judder just the same. In other words, the better the predicted display time, the less latency experienced by the user.

Every call to xrEndFrame **must** be preceded by a successful call to xrBeginFrame. Failure to do so **must** result in `XR_ERROR_CALL_ORDER_INVALID` being returned by xrEndFrame. Refer to xrBeginSession for details on how a transition to session running resets the frame function call order. XrFrameEndInfo **may** reference swapchains into which the application has rendered for this frame. From each XrSwapchain only one image index is implicitly referenced per frame, the one corresponding to the last call to xrReleaseSwapchainImage. However, a specific swapchain (and by extension a specific swapchain image index) **may** be referenced in XrFrameEndInfo multiple times. This **can** be used for example to render a side by side image into a single swapchain image and referencing it twice with differing image rectangles in different layers.

If no layers are provided then the display **must** be cleared.

`XR_ERROR_LAYER_INVALID` **must** be returned if an unknown, unsupported layer type, or `NULL` pointer is passed as one of the XrFrameEndInfo::`layers`.

`XR_ERROR_LAYER_INVALID` **must** be returned if a layer references a swapchain that has no released swapchain image.

`XR_ERROR_LAYER_LIMIT_EXCEEDED` **must** be returned if XrFrameEndInfo::`layerCount` exceeds XrSystemGraphicsProperties::maxLayerCount or if the runtime is unable to composite the specified layers due to resource constraints.

`XR_ERROR_SWAPCHAIN_RECT_INVALID` **must** be returned if XrFrameEndInfo::`layers` contains a composition layer which references pixels outside of the associated swapchain image or if negatively sized.

`XR_ERROR_ENVIRONMENT_BLEND_MODE_UNSUPPORTED` **must** be returned if and only if the XrFrameEndInfo::`environmentBlendMode` was not enumerated by xrEnumerateEnvironmentBlendModes for the XrInstance and `XrSystemId` used to create `session`.

`XR_ERROR_SESSION_NOT_RUNNING` **must** be returned if the `session` is not running.



Note

Applications should discard frames for which xrEndFrame returns a recoverable error over attempting to resubmit the frame with different frame parameters to provide a more consistent experience across different runtime implementations.

Valid Usage (Implicit)

- `session` **must** be a valid XrSession handle
- `frameEndInfo` **must** be a pointer to a valid XrFrameEndInfo structure

Thread Safety

- Access to the `session` parameter by any other xrBeginFrame or xrEndFrame call **must** be externally synchronized

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_CALL_ORDER_INVALID
- XR_ERROR_ENVIRONMENT_BLEND_MODE_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LAYER_INVALID
- XR_ERROR_LAYER_LIMIT_EXCEEDED
- XR_ERROR_POSE_INVALID
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_SESSION_NOT_RUNNING
- XR_ERROR_SWAPCHAIN_RECT_INVALID
- XR_ERROR_TIME_INVALID
- XR_ERROR_VALIDATION_FAILURE

The XrFrameEndInfo structure is defined as:

```
// Provided by XR_VERSION_1_0
typedef struct XrFrameEndInfo {
    XrStructureType                                 type;
    const void*                                     next;
    XrTime                                          displayTime;
    XrEnvironmentBlendMode                          environmentBlendMode;
    uint32_t                                        layerCount;
    const XrCompositionLayerBaseHeader* const*      layers;
} XrFrameEndInfo;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR.
- `displayTime` is the `XrTime` at which this frame **should** be displayed.
- `environmentBlendMode` is the XrEnvironmentBlendMode value representing the desired environment blend mode for this frame.
- `layerCount` is the number of composition layers in this frame. The maximum supported layer count is identified by XrSystemGraphicsProperties::maxLayerCount. If layerCount is greater than the maximum supported layer count then `XR_ERROR_LAYER_LIMIT_EXCEEDED` **must** be returned.
- `layers` is a pointer to an array of XrCompositionLayerBaseHeader pointers.

Valid Usage (Implicit)

- `type` **must** be `XR_TYPE_FRAME_END_INFO`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain. See also: XrFrameEndInfoML, XrGlobalDimmerFrameEndInfoML, XrLocalDimmingFrameEndInfoMETA, XrSecondaryViewConfigurationFrameEndInfoMSFT
- `environmentBlendMode` **must** be a valid XrEnvironmentBlendMode value
- If `layerCount` is not `0`, `layers` **must** be a pointer to an array of `layerCount` valid XrCompositionLayerBaseHeader-based structures. See also: XrCompositionLayerCubeKHR, XrCompositionLayerCylinderKHR, XrCompositionLayerEquirect2KHR, XrCompositionLayerEquirectKHR, XrCompositionLayerPassthroughANDROID, XrCompositionLayerPassthroughFB, XrCompositionLayerPassthroughHTC, XrCompositionLayerProjection, XrCompositionLayerQuad

All layers submitted to xrEndFrame will be presented to the primary view configuration of the running session.

10.5. Frame Rate

For every application-generated frame, the application **may** call xrEndFrame to submit the application-generated composition layers. In addition, the application **must** call xrWaitFrame when the application is ready to begin preparing the next set of frame layers. xrEndFrame **may** return immediately to the application, but xrWaitFrame **must** block for an amount of time that depends on throttling of the application by the runtime. The earliest the runtime will return from xrWaitFrame is when it determines that the application **should** start drawing the next frame.

10.6. Compositing

Composition layers are submitted by the application via the xrEndFrame call. All composition layers to be drawn **must** be submitted with every xrEndFrame call. A layer that is omitted in this call will not be drawn by the runtime layer compositor. All views associated with projection layers **must** be supplied, or XR_ERROR_VALIDATION_FAILURE **must** be returned by xrEndFrame.

Composition layers **must** be drawn in the same order as they are specified in via XrFrameEndInfo, with the 0th layer drawn first. Layers **must** be drawn with a "painter's algorithm," with each successive layer potentially overwriting the destination layers whether or not the new layers are virtually closer to the viewer.

10.6.1. Composition Layer Flags

XrCompositionLayerFlags specifies options for individual composition layers, and contains a bitwise-OR of zero or more of the bits defined in XrCompositionLayerFlagBits.

```
// Provided by XR_VERSION_1_0
typedef XrFlags64 XrCompositionLayerFlags;
```

Valid bits for XrCompositionLayerFlags are defined by XrCompositionLayerFlagBits, which is specified as:

```
// Provided by XR_VERSION_1_0
// Flag bits for XrCompositionLayerFlags
// XR_COMPOSITION_LAYER_CORRECT_CHROMATIC_ABERRATION_BIT is deprecated and should not be
used
static const XrCompositionLayerFlags
XR_COMPOSITION_LAYER_CORRECT_CHROMATIC_ABERRATION_BIT = 0x00000001;
static const XrCompositionLayerFlags XR_COMPOSITION_LAYER_BLEND_TEXTURE_SOURCE_ALPHA_BIT
= 0x00000002;
static const XrCompositionLayerFlags XR_COMPOSITION_LAYER_UNPREMULTIPLIED_ALPHA_BIT =
0x00000004;
  // Provided by XR_EXT_composition_layer_inverted_alpha
static const XrCompositionLayerFlags XR_COMPOSITION_LAYER_INVERTED_ALPHA_BIT_EXT =
0x00000008;
```

The flag bits have the following meanings:

Flag Descriptions

- XR_COMPOSITION_LAYER_CORRECT_CHROMATIC_ABERRATION_BIT *(deprecated — ignored)* — Enables chromatic aberration correction when not done by default. This flag has no effect on any known conformant runtime, and is officially deprecated in OpenXR 1.1.
- XR_COMPOSITION_LAYER_BLEND_TEXTURE_SOURCE_ALPHA_BIT — Enables the layer texture alpha channel.
- XR_COMPOSITION_LAYER_UNPREMULTIPLIED_ALPHA_BIT — Indicates the texture color channels have not been premultiplied by the texture alpha channel.
- XR_COMPOSITION_LAYER_INVERTED_ALPHA_BIT_EXT — Indicates that the texture alpha channel stores transparency instead of opacity, and is to be inverted before layer blending. (Added by the XR_EXT_composition_layer_inverted_alpha extension)

10.6.2. Composition Layer Blending

All types of composition layers are subject to blending with other layers. Blending of layers can be controlled by layer per-texel source alpha. Layer swapchain textures may contain an alpha channel, depending on the image format. If a submitted swapchain's texture format does not include an alpha channel or if the XR_COMPOSITION_LAYER_BLEND_TEXTURE_SOURCE_ALPHA_BIT is unset, then the layer alpha is initialized to one.

If the swapchain texture format color encoding is other than RGBA, it is converted to RGBA.

If the texture color channels are encoded without premultiplying by alpha, the XR_COMPOSITION_LAYER_UNPREMULTIPLIED_ALPHA_BIT **should** be set. The effect of this bit alters the layer color as follows:

```
LayerColor.RGB *= LayerColor.A
```

LayerColor is then clamped to a range of [0.0, 1.0].

The layer blending operation is defined as:

```
CompositeColor  = LayerColor + CompositeColor * (1 - LayerColor.A)
```

Before the first layer is composited, all components of CompositeColor are initialized to zero.

10.6.3. Composition Layer Types

Composition layers allow an application to offload the composition of the final image to a runtime-supplied compositor. This reduces the application's rendering complexity since details such as frame-

rate interpolation and distortion correction can be performed by the runtime. The core specification defines XrCompositionLayerProjection and XrCompositionLayerQuad layer types.

The projection layer type represents planar projected images rendered from the eye point of each eye using a perspective projection. This layer type is typically used to render the virtual world from the user's perspective.

The quad layer type describes a posable planar rectangle in the virtual world for displaying two-dimensional content. Quad layers can subtend a smaller portion of the display's field of view, allowing a better match between the resolutions of the XrSwapchain image and footprint of that image in the final composition. This improves legibility for user interface elements or heads-up displays and allows optimal sampling during any composition distortion corrections the runtime might employ.

The classes below describe the layer types in the layer composition system.

The XrCompositionLayerBaseHeader structure is defined as:

```
// Provided by XR_VERSION_1_0
typedef struct XrCompositionLayerBaseHeader {
    XrStructureType            type;
    const void*                next;
    XrCompositionLayerFlags    layerFlags;
    XrSpace                    space;
} XrCompositionLayerBaseHeader;
```

Member Descriptions

- `type` is the XrStructureType of this structure. This base structure itself has no associated XrStructureType value.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR.
- `layerFlags` is a bitmask of XrCompositionLayerFlagBits describing flags to apply to the layer.
- `space` is the XrSpace in which the layer will be kept stable over time.

All composition layer structures begin with the elements described in the XrCompositionLayerBaseHeader. The XrCompositionLayerBaseHeader structure is not intended to be directly used, but forms a basis for defining current and future structures containing composition layer information. The XrFrameEndInfo structure contains an array of pointers to these polymorphic header structures. All composition layer type pointers **must** be type-castable as an XrCompositionLayerBaseHeader pointer.

Valid Usage (Implicit)

- `type` **must** be one of the following XrStructureType values: `XR_TYPE_COMPOSITION_LAYER_CUBE_KHR`, `XR_TYPE_COMPOSITION_LAYER_CYLINDER_KHR`, `XR_TYPE_COMPOSITION_LAYER_EQUIRECT2_KHR`, `XR_TYPE_COMPOSITION_LAYER_EQUIRECT_KHR`, `XR_TYPE_COMPOSITION_LAYER_PASSTHROUGH_ANDROID`, `XR_TYPE_COMPOSITION_LAYER_PASSTHROUGH_FB`, `XR_TYPE_COMPOSITION_LAYER_PASSTHROUGH_HTC`, `XR_TYPE_COMPOSITION_LAYER_PROJECTION`, `XR_TYPE_COMPOSITION_LAYER_QUAD`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain. See also: XrCompositionLayerAlphaBlendFB, XrCompositionLayerColorScaleBiasKHR, XrCompositionLayerDepthTestFB, XrCompositionLayerImageLayoutFB, XrCompositionLayerSecureContentFB, XrCompositionLayerSettingsFB
- `layerFlags` **must** be `0` or a valid combination of XrCompositionLayerFlagBits values
- `space` **must** be a valid XrSpace handle

Many composition layer structures also contain one or more references to generic layer data stored in an XrSwapchainSubImage structure.

The XrSwapchainSubImage structure is defined as:

```
// Provided by XR_VERSION_1_0
typedef struct XrSwapchainSubImage {
    XrSwapchain    swapchain;
    XrRect2Di      imageRect;
    uint32_t       imageArrayIndex;
} XrSwapchainSubImage;
```

Member Descriptions

- `swapchain` is the XrSwapchain to be displayed.
- `imageRect` is an XrRect2Di representing the valid portion of the image to use, in pixels. It also implicitly defines the transform from normalized image coordinates into pixel coordinates. The coordinate origin depends on which graphics API is being used. See the graphics API extension details for more information on the coordinate origin definition. Note that the compositor **may** bleed in pixels from outside the bounds in some cases, for instance due to mipmapping.
- `imageArrayIndex` is the image array index, with 0 meaning the first or only array element.

Valid Usage (Implicit)

- `swapchain` **must** be a valid XrSwapchain handle

Runtimes **must** return `XR_ERROR_VALIDATION_FAILURE` if the XrSwapchainSubImage::`imageArrayIndex` is equal to or greater than the XrSwapchainCreateInfo::`arraySize` that the XrSwapchainSubImage ::`swapchain` was created with.

Projection Composition

The XrCompositionLayerProjection layer represents planar projected images rendered from the eye point of each eye using a standard perspective projection.

The XrCompositionLayerProjection structure is defined as:

```
// Provided by XR_VERSION_1_0
typedef struct XrCompositionLayerProjection {
    XrStructureType                             type;
    const void*                                 next;
    XrCompositionLayerFlags                     layerFlags;
    XrSpace                                     space;
    uint32_t                                    viewCount;
    const XrCompositionLayerProjectionView*     views;
} XrCompositionLayerProjection;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR.
- `layerFlags` is a bitmask of XrCompositionLayerFlagBits describing flags to apply to the layer.
- `space` is the XrSpace in which the `pose` of each XrCompositionLayerProjectionView is evaluated over time by the compositor.
- `viewCount` is the count of views in the `views` array. This **must** be equal to the number of view poses returned by xrLocateViews.
- `views` is the array of type XrCompositionLayerProjectionView containing each projection layer view.



Note

Because a runtime may reproject the layer over time, a projection layer should specify an XrSpace in which to maximize stability of the layer content. For example, a projection layer containing world-locked content should use an XrSpace which is also world-locked, such as the `LOCAL` or `STAGE` reference spaces. In the case that the projection layer should be head-locked, such as a heads up display, the `VIEW` reference space would provide the highest quality layer reprojection.

Valid Usage (Implicit)

- `type` **must** be `XR_TYPE_COMPOSITION_LAYER_PROJECTION`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain. See also: XrCompositionLayerDepthTestVARJO, XrCompositionLayerReprojectionInfoMSFT, XrCompositionLayerReprojectionPlaneOverrideMSFT
- `layerFlags` **must** be `0` or a valid combination of XrCompositionLayerFlagBits values
- `space` **must** be a valid XrSpace handle
- `views` **must** be a pointer to an array of `viewCount` valid XrCompositionLayerProjectionView structures
- The `viewCount` parameter **must** be greater than `0`

The XrCompositionLayerProjectionView structure is defined as:

```
// Provided by XR_VERSION_1_0
typedef struct XrCompositionLayerProjectionView {
    XrStructureType        type;
    const void*            next;
    XrPosef                pose;
    XrFovf                 fov;
    XrSwapchainSubImage    subImage;
} XrCompositionLayerProjectionView;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR.
- `pose` is an XrPosef defining the location and orientation of this projection element in the `space` of the corresponding XrCompositionLayerProjectionView.
- `fov` is the XrFovf for this projection element.
- `subImage` is the image layer XrSwapchainSubImage to use. The swapchain **must** have been created with a XrSwapchainCreateInfo::`faceCount` of 1.

The count and order of view poses submitted with XrCompositionLayerProjection **must** be the same order as that returned by xrLocateViews. The XrCompositionLayerProjectionView::`pose` and XrCompositionLayerProjectionView::`fov` **should** almost always derive from XrView::`pose` and XrView::`fov` as found in the xrLocateViews::`views` array. However, applications **may** submit an XrCompositionLayerProjectionView which has a different view or FOV than that from xrLocateViews. In this case, the runtime will map the view and FOV to the system display appropriately. In the case that two submitted views within a single layer overlap, they **must** be composited in view array order.

Valid Usage (Implicit)

- `type` **must** be `XR_TYPE_COMPOSITION_LAYER_PROJECTION_VIEW`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain. See also: XrCompositionLayerDepthInfoKHR, XrCompositionLayerSpaceWarpInfoFB, XrFrameSynthesisInfoEXT
- `subImage` **must** be a valid XrSwapchainSubImage structure

Quad Layer Composition

The XrCompositionLayerQuad structure defined as:

```
// Provided by XR_VERSION_1_0
typedef struct XrCompositionLayerQuad {
    XrStructureType            type;
    const void*                next;
    XrCompositionLayerFlags    layerFlags;
    XrSpace                    space;
    XrEyeVisibility            eyeVisibility;
    XrSwapchainSubImage        subImage;
    XrPosef                    pose;
    XrExtent2Df                size;
} XrCompositionLayerQuad;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR.
- `layerFlags` is a bitmask of XrCompositionLayerFlagBits describing flags to apply to the layer.
- `space` is the XrSpace in which the `pose` of the quad layer is evaluated over time.
- `eyeVisibility` is the XrEyeVisibility for this layer.
- `subImage` is the image layer XrSwapchainSubImage to use. The swapchain **must** have been created with a XrSwapchainCreateInfo::`faceCount` of 1.
- `pose` is an XrPosef defining the position and orientation of the quad in the reference frame of the `space`.
- `size` is the width and height of the quad in meters.

The XrCompositionLayerQuad layer is useful for user interface elements or 2D content rendered into the virtual world. The layer's XrSwapchainSubImage::swapchain image is applied to a quad in the virtual world space. Only front face of the quad surface is visible; the back face is not visible and **must** not be drawn by the runtime. A quad layer has no thickness; it is a two-dimensional object positioned and oriented in 3D space. The position of a quad refers to the center of the quad within the given XrSpace. The orientation of the quad refers to the orientation of the normal vector from the front face. The size of a quad refers to the quad's size in the x-y plane of the given XrSpace's coordinate system. A quad with a position of {0,0,0}, rotation of {0,0,0,1} (no rotation), and a size of {1,1} refers to a 1 meter x 1 meter quad centered at {0,0,0} with its front face normal vector coinciding with the +z axis.

Valid Usage (Implicit)

- `type` **must** be `XR_TYPE_COMPOSITION_LAYER_QUAD`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `layerFlags` **must** be `0` or a valid combination of XrCompositionLayerFlagBits values
- `space` **must** be a valid XrSpace handle
- `eyeVisibility` **must** be a valid XrEyeVisibility value
- `subImage` **must** be a valid XrSwapchainSubImage structure

The XrEyeVisibility enum selects which of the viewer's eyes to display a layer to:

```
// Provided by XR_VERSION_1_0
typedef enum XrEyeVisibility {
    XR_EYE_VISIBILITY_BOTH = 0,
    XR_EYE_VISIBILITY_LEFT = 1,
    XR_EYE_VISIBILITY_RIGHT = 2,
    XR_EYE_VISIBILITY_MAX_ENUM = 0x7FFFFFFF
} XrEyeVisibility;
```

Enumerant Descriptions

- `XR_EYE_VISIBILITY_BOTH` displays the layer to both eyes.
- `XR_EYE_VISIBILITY_LEFT` displays the layer to the viewer's physical left eye.
- `XR_EYE_VISIBILITY_RIGHT` displays the layer to the viewer's physical right eye.

10.6.4. Environment Blend Mode

After the compositor has blended and flattened all layers (including any layers added by the runtime itself), it will then present this image to the system's display. The composited image will then blend with the environment in one of three modes, based on the application's chosen **environment blend mode**. VR applications will generally choose the `XR_ENVIRONMENT_BLEND_MODE_OPAQUE` blend mode, while AR applications will generally choose either the `XR_ENVIRONMENT_BLEND_MODE_ADDITIVE` or `XR_ENVIRONMENT_BLEND_MODE_ALPHA_BLEND` mode.

The environment **may** be perceived in two ways. It could be the user's view of the physical world that exists beyond the displays, or it could be a synthetic environment including virtual components generated externally from the application. Alternatively, it could be a combination of both these elements.

Applications select their environment blend mode each frame as part of their call to xrEndFrame. The application can inspect the set of supported environment blend modes for a given system using xrEnumerateEnvironmentBlendModes, and prepare their assets and rendering techniques differently based on the blend mode they choose. For example, a black shadow rendered using the XR_ENVIRONMENT_BLEND_MODE_ADDITIVE blend mode will appear transparent, and so an application in that mode **may** render a glow as a grounding effect around the black shadow to ensure the shadow can be seen. Similarly, an application designed for XR_ENVIRONMENT_BLEND_MODE_OPAQUE or XR_ENVIRONMENT_BLEND_MODE_ADDITIVE rendering **may** choose to leave garbage in their alpha channel as a side effect of a rendering optimization, but this garbage would appear as visible display artifacts if the environment blend mode was instead XR_ENVIRONMENT_BLEND_MODE_ALPHA_BLEND.

Not all systems will support all environment blend modes. For example, a VR headset may not support the XR_ENVIRONMENT_BLEND_MODE_ADDITIVE or XR_ENVIRONMENT_BLEND_MODE_ALPHA_BLEND modes unless it has video passthrough, while an AR headset with an additive display may not support the XR_ENVIRONMENT_BLEND_MODE_OPAQUE or XR_ENVIRONMENT_BLEND_MODE_ALPHA_BLEND modes.

For devices that support video/optical passthrough or synthetic environments, they **may** support the XR_ENVIRONMENT_BLEND_MODE_ADDITIVE or XR_ENVIRONMENT_BLEND_MODE_ALPHA_BLEND modes. Selecting one of these modes would display the environment in the background, contingent upon the capability and status of the headsets.

For devices that can support multiple environment blend modes, such as AR phones with video passthrough, the runtime **may** optimize power consumption on the device in response to the environment blend mode that the application chooses each frame. For example, if an application on a video passthrough phone knows that it is currently rendering a 360-degree background covering all screen pixels, it can submit frames with an environment blend mode of XR_ENVIRONMENT_BLEND_MODE_OPAQUE, saving the runtime the cost of compositing a camera-based underlay of the physical world behind the application's layers.

The xrEnumerateEnvironmentBlendModes function is defined as:

```
// Provided by XR_VERSION_1_0
XrResult xrEnumerateEnvironmentBlendModes(
    XrInstance                                  instance,
    XrSystemId                                  systemId,
    XrViewConfigurationType                     viewConfigurationType,
    uint32_t                                    environmentBlendModeCapacityInput,
    uint32_t*                                   environmentBlendModeCountOutput,
    XrEnvironmentBlendMode*                     environmentBlendModes);
```

Parameter Descriptions

- `instance` is the instance from which `systemId` was retrieved.
- `systemId` is the `XrSystemId` whose environment blend modes will be enumerated.
- `viewConfigurationType` is the XrViewConfigurationType to enumerate.
- `environmentBlendModeCapacityInput` is the capacity of the `environmentBlendModes` array, or 0 to indicate a request to retrieve the required capacity.
- `environmentBlendModeCountOutput` is a pointer to the count of `environmentBlendModes` written, or a pointer to the required capacity in the case that `environmentBlendModeCapacityInput` is insufficient.
- `environmentBlendModes` is a pointer to an array of XrEnvironmentBlendMode values, but **can** be `NULL` if `environmentBlendModeCapacityInput` is 0.
- See the Buffer Size Parameters section for a detailed description of retrieving the required `environmentBlendModes` size.

Enumerates the set of environment blend modes that this runtime supports for a given view configuration of the system. Environment blend modes **should** be in order from highest to lowest runtime preference.

Runtimes **must** always return identical buffer contents from this enumeration for the given `systemId` and `viewConfigurationType` for the lifetime of the instance.

Valid Usage (Implicit)

- `instance` **must** be a valid XrInstance handle
- `viewConfigurationType` **must** be a valid XrViewConfigurationType value
- `environmentBlendModeCountOutput` **must** be a pointer to a `uint32_t` value
- If `environmentBlendModeCapacityInput` is not `0`, `environmentBlendModes` **must** be a pointer to an array of `environmentBlendModeCapacityInput` XrEnvironmentBlendMode values

Return Codes

Success

- XR_SUCCESS

Failure

- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SIZE_INSUFFICIENT
- XR_ERROR_SYSTEM_INVALID
- XR_ERROR_VALIDATION_FAILURE
- XR_ERROR_VIEW_CONFIGURATION_TYPE_UNSUPPORTED

The possible blend modes are specified by the XrEnvironmentBlendMode enumeration:

```
// Provided by XR_VERSION_1_0
typedef enum XrEnvironmentBlendMode {
    XR_ENVIRONMENT_BLEND_MODE_OPAQUE = 1,
    XR_ENVIRONMENT_BLEND_MODE_ADDITIVE = 2,
    XR_ENVIRONMENT_BLEND_MODE_ALPHA_BLEND = 3,
    XR_ENVIRONMENT_BLEND_MODE_MAX_ENUM = 0x7FFFFFFF
} XrEnvironmentBlendMode;
```

Enumerant Descriptions

- `XR_ENVIRONMENT_BLEND_MODE_OPAQUE`. The composition layers will be displayed with no view of the physical world behind them. The composited image will be interpreted as an RGB image, ignoring the composited alpha channel. This is the typical mode for VR experiences, although this mode can also be supported on devices that support video passthrough.
- `XR_ENVIRONMENT_BLEND_MODE_ADDITIVE`. The composition layers will be additively blended with the real world behind the display. The composited image will be interpreted as an RGB image, ignoring the composited alpha channel during the additive blending. This will cause black composited pixels to appear transparent. This is the typical mode for an AR experience on a see-through headset with an additive display, although this mode can also be supported on devices that support video passthrough.
- `XR_ENVIRONMENT_BLEND_MODE_ALPHA_BLEND`. The composition layers will be alpha-blended with the real world behind the display. The composited image will be interpreted as an RGBA image, with the composited alpha channel determining each pixel's level of blending with the real world behind the display. This is the typical mode for an AR experience on a phone or headset that supports video passthrough.

Chapter 11. Input and Haptics

11.1. Action Overview

OpenXR applications communicate with input devices using XrActions. Actions are created at initialization time and later used to request input device state, create action spaces, or control haptic events. Input action handles represent 'actions' that the application is interested in obtaining the state of, not direct input device hardware. For example, instead of the application directly querying the state of the A button when interacting with a menu, an OpenXR application instead creates a `menu_select` action at startup then asks OpenXR for the state of the action.

The application recommends that the action be assigned to a specific input source on the input device for a known interaction profile, but runtimes have the ability to choose a different control depending on user preference, input device availability, or any other reason. This abstraction ensures that applications can run on a wide variety of input hardware and maximize user accessibility.

Example usage:

```
XrInstance instance; // previously initialized
XrSession session; // previously initialized

// Create an action set
XrActionSetCreateInfo actionSetInfo {
    .type = XR_TYPE_ACTION_SET_CREATE_INFO,
    .actionSetName = "gameplay",
    .localizedActionSetName = "Gameplay",
    .priority = 0,
};
XrActionSet inGameActionSet {XR_NULL_HANDLE};
CHK_XR(xrCreateActionSet(instance, &actionSetInfo, &inGameActionSet));

// create a "teleport" input action
XrActionCreateInfo actioninfo {
    .type = XR_TYPE_ACTION_CREATE_INFO,
    .actionName = "teleport",
    .actionType = XR_ACTION_TYPE_BOOLEAN_INPUT,
    .localizedActionName = "Teleport",
};
XrAction teleportAction {XR_NULL_HANDLE};
CHK_XR(xrCreateAction(inGameActionSet, &actioninfo, &teleportAction));

// create a "player_hit" output action
XrActionCreateInfo hapticsactioninfo {
    .type = XR_TYPE_ACTION_CREATE_INFO,
    .actionName = "player_hit",
```

```
    .actionType = XR_ACTION_TYPE_VIBRATION_OUTPUT,
    .localizedActionName = "Player hit",
};
XrAction hapticsAction {XR_NULL_HANDLE};
CHK_XR(xrCreateAction(inGameActionSet, &hapticsactioninfo, &hapticsAction));

XrPath triggerClickPath {XR_NULL_PATH}, hapticPath {XR_NULL_PATH};
CHK_XR(xrStringToPath(instance, "/user/hand/right/input/trigger/click", &
triggerClickPath));
CHK_XR(xrStringToPath(instance, "/user/hand/right/output/haptic", &hapticPath))

XrPath interactionProfilePath {XR_NULL_PATH};
CHK_XR(xrStringToPath(instance, "/interaction_profiles/vendor_x/profile_x",
&interactionProfilePath));

XrActionSuggestedBinding bindings[2] = {
    {
        .action = teleportAction,
        .binding = triggerClickPath,
    },
    {
        .action = hapticsAction,
        .binding = hapticPath,
    },
};

XrInteractionProfileSuggestedBinding suggestedBindings {
    .type = XR_TYPE_INTERACTION_PROFILE_SUGGESTED_BINDING,
    .interactionProfile = interactionProfilePath,
    .countSuggestedBindings = 2,
    .suggestedBindings = bindings,
};
CHK_XR(xrSuggestInteractionProfileBindings(instance, &suggestedBindings));

XrSessionActionSetsAttachInfo attachInfo {
    .type = XR_TYPE_SESSION_ACTION_SETS_ATTACH_INFO,
    .countActionSets = 1,
    .actionSets = &inGameActionSet,
};
CHK_XR(xrAttachSessionActionSets(session, &attachInfo));

// application main loop
while (1)
{
    // sync action data
    XrActiveActionSet activeActionSet {
        .actionSet = inGameActionSet,
        .subactionPath = XR_NULL_PATH,
```

```
    };
    XrActionsSyncInfo syncInfo {
        .type = XR_TYPE_ACTIONS_SYNC_INFO,
        .countActiveActionSets = 1,
        .activeActionSets = &activeActionSet,
    };
    CHK_XR(xrSyncActions(session, &syncInfo));

    // query input action state
    XrActionStateBoolean teleportState {
        .type = XR_TYPE_ACTION_STATE_BOOLEAN,
    };
    XrActionStateGetInfo getInfo {
        .type = XR_TYPE_ACTION_STATE_GET_INFO,
       .action = teleportAction,
    };
    CHK_XR(xrGetActionStateBoolean(session, &getInfo, &teleportState));

    if (teleportState.changedSinceLastSync && teleportState.currentState)
    {
        // fire haptics using output action
        XrHapticVibration vibration {
            .type = XR_TYPE_HAPTIC_VIBRATION,
            .duration = 300000, // 300 ms
            .frequency = 3000,
            .amplitude = 0.5,
        };
        XrHapticActionInfo hapticActionInfo {
            .type = XR_TYPE_HAPTIC_ACTION_INFO,
            .action = hapticsAction,
        };
        CHK_XR(xrApplyHapticFeedback(session, &hapticActionInfo, (const
XrHapticBaseHeader*)&vibration));
    }
}
```

11.2. Action Sets

```
// Provided by XR_VERSION_1_0
XR_DEFINE_HANDLE(XrActionSet)
```

Action sets are application-defined collections of actions. They are attached to a given XrSession with a xrAttachSessionActionSets call. Enabled action sets are indicated by the application via xrSyncActions depending on the current application context.

For example, consider using one collection of actions that apply to controlling a character and another collection for navigating a menu system. When these actions are structured as two XrActionSet handles, the applicable action set is easy to specify according to application logic using a single function call.

Further, suppose some actions only apply when operating a vehicle as a character. This is intended to be modeled as another separate action set. While the user is operating a vehicle, the application enables both the character-control and vehicle action sets simultaneously in each xrSyncActions call.

Actions are passed a handle to their XrActionSet when they are created.

Action sets are created by calling xrCreateActionSet.

The xrCreateActionSet function is defined as:

```
// Provided by XR_VERSION_1_0
XrResult xrCreateActionSet(
    XrInstance                                  instance,
    const XrActionSetCreateInfo*                createInfo,
    XrActionSet*                                actionSet);
```

Parameter Descriptions

- `instance` is a handle to an XrInstance.
- `createInfo` is a pointer to a valid XrActionSetCreateInfo structure that defines the action set being created.
- `actionSet` is a pointer to an XrActionSet where the created action set is returned.

The xrCreateActionSet function creates an action set and returns a handle to the created action set.

Valid Usage (Implicit)

- `instance` **must** be a valid XrInstance handle
- `createInfo` **must** be a pointer to a valid XrActionSetCreateInfo structure
- `actionSet` **must** be a pointer to an XrActionSet handle

Return Codes

Success

- XR_SUCCESS

Failure

- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_LOCALIZED_NAME_DUPLICATED
- XR_ERROR_LOCALIZED_NAME_INVALID
- XR_ERROR_NAME_DUPLICATED
- XR_ERROR_NAME_INVALID
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_PATH_FORMAT_INVALID
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_VALIDATION_FAILURE

The XrActionSetCreateInfo structure is defined as:

```
// Provided by XR_VERSION_1_0
typedef struct XrActionSetCreateInfo {
    XrStructureType    type;
    const void*        next;
    char               actionSetName[XR_MAX_ACTION_SET_NAME_SIZE];
    char               localizedActionSetName[XR_MAX_LOCALIZED_ACTION_SET_NAME_SIZE];
    uint32_t           priority;
} XrActionSetCreateInfo;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR.
- `actionSetName` is an array containing a `NULL` terminated non-empty string with the name of this action set.
- `localizedActionSetName` is an array containing a `NULL` terminated `UTF`-8 string that can be presented to the user as a description of the action set. This string should be presented in the system's current active locale.
- `priority` defines which action sets' actions are active on a given input source when actions on multiple active action sets are bound to the same input source. Larger priority numbers take precedence over smaller priority numbers.

When multiple actions are bound to the same input source, the `priority` of each action set determines which bindings are suppressed. Runtimes **must** ignore input sources from action sets with a lower priority number if those specific input sources are also present in active actions within a higher priority action set. If multiple action sets with the same priority are bound to the same input source and that is the highest priority number, runtimes **must** process all those bindings at the same time.

Two actions are considered to be bound to the same input source if they use the same identifier and optional location path segments, even if they have different component segments.

When runtimes are ignoring bindings because of priority, they **must** treat the binding to that input source as though they do not exist. That means the `isActive` field **must** be `XR_FALSE` when retrieving action data, and that the runtime **must** not provide any visual, haptic, or other feedback related to the binding of that action to that input source. Other actions in the same action set which are bound to input sources that do not collide are not affected and are processed as normal.

If `actionSetName` or `localizedActionSetName` are empty strings, the runtime **must** return `XR_ERROR_NAME_INVALID` or `XR_ERROR_LOCALIZED_NAME_INVALID` respectively. If `actionSetName` or `localizedActionSetName` are duplicates of the corresponding field for any existing action set in the specified instance, the runtime **must** return `XR_ERROR_NAME_DUPLICATED` or `XR_ERROR_LOCALIZED_NAME_DUPLICATED` respectively. If the conflicting action set is destroyed, the conflicting field is no longer considered duplicated. If `actionSetName` contains characters which are not allowed in a single level of a well-formed path string, the runtime **must** return `XR_ERROR_PATH_FORMAT_INVALID`.

Valid Usage (Implicit)

- `type` **must** be `XR_TYPE_ACTION_SET_CREATE_INFO`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `actionSetName` **must** be a null-terminated UTF-8 string whose length is less than or equal to `XR_MAX_ACTION_SET_NAME_SIZE`
- `localizedActionSetName` **must** be a null-terminated UTF-8 string whose length is less than or equal to `XR_MAX_LOCALIZED_ACTION_SET_NAME_SIZE`

The xrDestroyActionSet function is defined as:

```
// Provided by XR_VERSION_1_0
XrResult xrDestroyActionSet(
    XrActionSet                                 actionSet);
```

Parameter Descriptions

- `actionSet` is the action set to destroy.

Action set handles **can** be destroyed by calling xrDestroyActionSet. When an action set handle is destroyed, all handles of actions in that action set are also destroyed.

The implementation **must** not free underlying resources for the action set while there are other valid handles that refer to those resources. The implementation **may** release resources for an action set when all of the action spaces for actions in that action set have been destroyed. See Action Spaces Lifetime for details.

Resources for all action sets in an instance **must** be freed when the instance containing those actions sets is destroyed.

Valid Usage (Implicit)

- `actionSet` **must** be a valid XrActionSet handle

Thread Safety

- Access to `actionSet`, and any child handles, **must** be externally synchronized

Return Codes

Success

- XR_SUCCESS

Failure

- XR_ERROR_HANDLE_INVALID
- XR_ERROR_RUNTIME_FAILURE

11.3. Creating Actions

```
// Provided by XR_VERSION_1_0
XR_DEFINE_HANDLE(XrAction)
```

Action handles are used to refer to individual actions when retrieving action data, creating action spaces, or sending haptic events.

The xrCreateAction function is defined as:

```
// Provided by XR_VERSION_1_0
XrResult xrCreateAction(
    XrActionSet                                 actionSet,
    const XrActionCreateInfo*                   createInfo,
    XrAction*                                   action);
```

Parameter Descriptions

- actionSet is a handle to an XrActionSet.
- createInfo is a pointer to a valid XrActionCreateInfo structure that defines the action being created.
- action is a pointer to an XrAction where the created action is returned.

xrCreateAction creates an action and returns its handle.

If actionSet has been included in a call to xrAttachSessionActionSets, the implementation **must** return XR_ERROR_ACTIONSETS_ALREADY_ATTACHED.

Valid Usage (Implicit)

- `actionSet` **must** be a valid XrActionSet handle
- `createInfo` **must** be a pointer to a valid XrActionCreateInfo structure
- `action` **must** be a pointer to an XrAction handle

Return Codes

Success

- `XR_SUCCESS`

Failure

- `XR_ERROR_ACTIONSETS_ALREADY_ATTACHED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_LIMIT_REACHED`
- `XR_ERROR_LOCALIZED_NAME_DUPLICATED`
- `XR_ERROR_LOCALIZED_NAME_INVALID`
- `XR_ERROR_NAME_DUPLICATED`
- `XR_ERROR_NAME_INVALID`
- `XR_ERROR_OUT_OF_MEMORY`
- `XR_ERROR_PATH_FORMAT_INVALID`
- `XR_ERROR_PATH_INVALID`
- `XR_ERROR_PATH_UNSUPPORTED`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_VALIDATION_FAILURE`

The XrActionCreateInfo structure is defined as:

```
// Provided by XR_VERSION_1_0
typedef struct XrActionCreateInfo {
    XrStructureType    type;
    const void*        next;
    char               actionName[XR_MAX_ACTION_NAME_SIZE];
    XrActionType       actionType;
    uint32_t           countSubactionPaths;
    const XrPath*      subactionPaths;
    char               localizedActionName[XR_MAX_LOCALIZED_ACTION_NAME_SIZE];
} XrActionCreateInfo;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR.
- `actionName` is an array containing a `NULL` terminated string with the name of this action.
- `actionType` is the XrActionType of the action to be created.
- `countSubactionPaths` is the number of elements in the `subactionPaths` array. If `subactionPaths` is NULL, this parameter must be 0.
- `subactionPaths` is an array of `XrPath` or `NULL`. If this array is specified, it contains one or more subaction paths that the application intends to query action state for.
- `localizedActionName` is an array containing a `NULL` terminated `UTF`-8 string that can be presented to the user as a description of the action. This string should be in the system's current active locale.

Subaction paths are a mechanism that enables applications to use the same action name and handle on multiple devices. Applications can query action state using subaction paths that differentiate data coming from each device. This allows the runtime to group logically equivalent actions together in system UI. For instance, an application could create a single `pick_up` action with the */user/hand/left* and */user/hand/right* subaction paths and use the subaction paths to independently query the state of `pick_up_with_left_hand` and `pick_up_with_right_hand`.

Applications **can** create actions with or without the `subactionPaths` set to a list of paths. If this list of paths is omitted (i.e. `subactionPaths` is set to `NULL`, and `countSubactionPaths` is set to `0`), the application is opting out of filtering action results by subaction paths and any call to get action data must also omit subaction paths.

If `subactionPaths` is specified and any of the following conditions are not satisfied, the runtime **must** return `XR_ERROR_PATH_UNSUPPORTED`:

- Each path provided is one of:
  - */user/head*
  - */user/hand/left*
  - */user/hand/right*
  - */user/gamepad*
- No path appears in the list more than once

Extensions **may** append additional top level user paths to the above list.



Note

Earlier revisions of the spec mentioned */user* but it could not be implemented as specified and was removed as errata.

The runtime **must** return `XR_ERROR_PATH_UNSUPPORTED` in the following circumstances:

- The application specified subaction paths at action creation and the application called `xrGetActionState*` or a haptic function with an empty subaction path array.
- The application called `xrGetActionState*` or a haptic function with a subaction path that was not specified when the action was created.

If `actionName` or `localizedActionName` are empty strings, the runtime **must** return `XR_ERROR_NAME_INVALID` or `XR_ERROR_LOCALIZED_NAME_INVALID` respectively. If `actionName` or `localizedActionName` are duplicates of the corresponding field for any existing action in the specified action set, the runtime **must** return `XR_ERROR_NAME_DUPLICATED` or `XR_ERROR_LOCALIZED_NAME_DUPLICATED` respectively. If the conflicting action is destroyed, the conflicting field is no longer considered duplicated. If `actionName` contains characters which are not allowed in a single level of a well-formed path string, the runtime **must** return `XR_ERROR_PATH_FORMAT_INVALID`.

Valid Usage (Implicit)

- `type` **must** be `XR_TYPE_ACTION_CREATE_INFO`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `actionName` **must** be a null-terminated UTF-8 string whose length is less than or equal to `XR_MAX_ACTION_NAME_SIZE`
- `actionType` **must** be a valid XrActionType value
- If `countSubactionPaths` is not `0`, `subactionPaths` **must** be a pointer to an array of `countSubactionPaths` valid `XrPath` values
- `localizedActionName` **must** be a null-terminated UTF-8 string whose length is less than or equal to `XR_MAX_LOCALIZED_ACTION_NAME_SIZE`

The XrActionType parameter takes one of the following values:

```
// Provided by XR_VERSION_1_0
typedef enum XrActionType {
    XR_ACTION_TYPE_BOOLEAN_INPUT = 1,
    XR_ACTION_TYPE_FLOAT_INPUT = 2,
    XR_ACTION_TYPE_VECTOR2F_INPUT = 3,
    XR_ACTION_TYPE_POSE_INPUT = 4,
    XR_ACTION_TYPE_VIBRATION_OUTPUT = 100,
    XR_ACTION_TYPE_MAX_ENUM = 0x7FFFFFFF
} XrActionType;
```

Enumerant Descriptions

- XR_ACTION_TYPE_BOOLEAN_INPUT. The action can be passed to xrGetActionStateBoolean to retrieve a boolean value.
- XR_ACTION_TYPE_FLOAT_INPUT. The action can be passed to xrGetActionStateFloat to retrieve a float value.
- XR_ACTION_TYPE_VECTOR2F_INPUT. The action can be passed to xrGetActionStateVector2f to retrieve a 2D float vector.
- XR_ACTION_TYPE_POSE_INPUT. The action can can be passed to xrCreateActionSpace to create a space.
- XR_ACTION_TYPE_VIBRATION_OUTPUT. The action can be passed to xrApplyHapticFeedback to send a haptic event to the runtime.

The xrDestroyAction function is defined as:

```
// Provided by XR_VERSION_1_0
XrResult xrDestroyAction(
    XrAction                                    action);
```

Parameter Descriptions

- action is the action to destroy.

Action handles **can** be destroyed by calling xrDestroyAction. Handles for actions that are part of an action set are automatically destroyed when the action set's handle is destroyed.

The implementation **must** not destroy the underlying resources for an action when xrDestroyAction is called. Those resources are still used to make action spaces locatable and when processing action

priority in xrSyncActions. Destroying the action handle removes the application's access to these resources, but has no other change on actions.

Resources for all actions in an instance **must** be freed when the instance containing those actions sets is destroyed.

Valid Usage (Implicit)

- `action` **must** be a valid XrAction handle

Thread Safety

- Access to `action`, and any child handles, **must** be externally synchronized

Return Codes

Success

- `XR_SUCCESS`

Failure

- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_RUNTIME_FAILURE`

11.3.1. Input Actions & Output Actions

Input actions are used to read sensors like buttons or joysticks while output actions are used for triggering haptics or motion platforms. The type of action created by xrCreateAction depends on the value of the XrActionType argument.

A given action can either be used for either input or output, but not both. Input actions are queried using one of the `xrGetActionState*` function calls, while output actions are set using the haptics calls. If either call is used with an action of the wrong type `XR_ERROR_ACTION_TYPE_MISMATCH` **must** be returned.

11.4. Suggested Bindings

Applications suggest bindings for their actions to runtimes so that raw input data is mapped appropriately to the application's actions. Suggested bindings also serve as a signal indicating the hardware that has been tested by the application developer. Applications **can** suggest bindings by calling xrSuggestInteractionProfileBindings for each interaction profile that the application is developed and tested with. If bindings are provided for an appropriate interaction profile, the runtime **may** select one and input will begin to flow. Interaction profile selection changes **must** only happen

when xrSyncActions is called. Applications **can** call xrGetCurrentInteractionProfile during on a running session to learn what the active interaction profile are for a top level user path. If this value ever changes, the runtime **must** send an XR_TYPE_EVENT_DATA_INTERACTION_PROFILE_CHANGED event to the application to indicate that the value should be queried again.

The bindings suggested by this system are only a hint to the runtime. Some runtimes **may** choose to use a different device binding depending on user preference, accessibility settings, or for any other reason. If the runtime is using the values provided by suggested bindings, it **must** make a best effort to convert the input value to the created action and apply certain rules to that use so that suggested bindings function in the same way across runtimes. If an input value cannot be converted to the type of the action, the value **must** be ignored and not contribute to the state of the action.

For actions created with XR_ACTION_TYPE_BOOLEAN_INPUT when the runtime is obeying suggested bindings: Boolean input sources **must** be bound directly to the action. If the path is to a scalar value, a threshold **must** be applied to the value and values over that threshold will be XR_TRUE. The runtime **should** use hysteresis when applying this threshold. The threshold and hysteresis range **may** vary from device to device or component to component and are left as an implementation detail. If the path refers to the parent of input values instead of to an input value itself, the runtime **must** use *.../example/path/click* instead of *.../example/path* if it is available. If a parent path does not have a *.../click* subpath, the runtime **must** use *.../value* and apply the same thresholding that would be applied to any scalar input. In any other situation the runtime **may** provide an alternate binding for the action or it will be unbound.

For actions created with XR_ACTION_TYPE_FLOAT_INPUT when the runtime is obeying suggested bindings: If the input value specified by the path is scalar, the input value **must** be bound directly to the float. If the path refers to the parent of input values instead of to an input value itself, the runtime **must** use *.../example/path/value* instead of *.../example/path* as the source of the value. If a parent path does not have a *.../value* subpath, the runtime **must** use *.../click*. If the input value is boolean, the runtime **must** supply 0.0 or 1.0 as a conversion of the boolean value. In any other situation, the runtime **may** provide an alternate binding for the action or it will be unbound.

For actions created with XR_ACTION_TYPE_VECTOR2F_INPUT when the runtime is obeying suggested bindings: The suggested binding path **must** refer to the parent of input values instead of to the input values themselves, and that parent path **must** contain subpaths *.../x* and *.../y*. *.../x* and *.../y* **must** be bound to 'x' and 'y' of the vector, respectively. In any other situation, the runtime **may** provide an alternate binding for the action or it will be unbound.

For actions created with XR_ACTION_TYPE_POSE_INPUT when the runtime is obeying suggested bindings: Pose input sources **must** be bound directly to the action. If the path refers to the parent of input values instead of to an input value itself, the runtime **must** use *.../example/path/pose* instead of *.../example/path* if it is available. In any other situation the runtime **may** provide an alternate binding for the action or it will be unbound.

The xrSuggestInteractionProfileBindings function is defined as:

```
// Provided by XR_VERSION_1_0
XrResult xrSuggestInteractionProfileBindings(
    XrInstance                                  instance,
    const XrInteractionProfileSuggestedBinding* suggestedBindings);
```

Parameter Descriptions

- `instance` is the XrInstance for which the application would like to set suggested bindings
- `suggestedBindings` is the XrInteractionProfileSuggestedBinding that the application would like to set

The xrSuggestInteractionProfileBindings function provides action bindings for a single interaction profile. The application **can** call xrSuggestInteractionProfileBindings once per interaction profile that it supports.

The application **can** provide any number of bindings for each action.

If the application successfully calls xrSuggestInteractionProfileBindings more than once for an interaction profile, the runtime **must** discard the previous suggested bindings and replace them with the new suggested bindings for that profile.

If the interaction profile path does not follow the structure defined in Interaction Profiles or suggested bindings contain paths that do not follow the format defined in Input subpaths (further described in XrActionSuggestedBinding), the runtime **must** return `XR_ERROR_PATH_UNSUPPORTED`. If the interaction profile path or binding path (top level */user* path plus input subpath) for any of the suggested bindings does not exist in the allowlist defined in Interaction Profile Paths, the runtime **must** return `XR_ERROR_PATH_UNSUPPORTED`. A runtime **must** accept every valid binding in the allowlist though it is free to ignore any of them.

If the action set for any action referenced in the `suggestedBindings` parameter has been included in a call to xrAttachSessionActionSets, the implementation **must** return `XR_ERROR_ACTIONSETS_ALREADY_ATTACHED`.

Valid Usage (Implicit)

- `instance` **must** be a valid XrInstance handle
- `suggestedBindings` **must** be a pointer to a valid XrInteractionProfileSuggestedBinding structure

Return Codes

Success

- XR_SUCCESS

Failure

- XR_ERROR_ACTIONSETS_ALREADY_ATTACHED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_PATH_INVALID
- XR_ERROR_PATH_UNSUPPORTED
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_VALIDATION_FAILURE

The XrInteractionProfileSuggestedBinding structure is defined as:

```
// Provided by XR_VERSION_1_0
typedef struct XrInteractionProfileSuggestedBinding {
    XrStructureType                    type;
    const void*                        next;
    XrPath                             interactionProfile;
    uint32_t                           countSuggestedBindings;
    const XrActionSuggestedBinding*    suggestedBindings;
} XrInteractionProfileSuggestedBinding;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR.
- interactionProfile is the XrPath of an interaction profile.
- countSuggestedBindings is the number of suggested bindings in the array pointed to by suggestedBindings.
- suggestedBindings is a pointer to an array of XrActionSuggestedBinding structures that define all of the application's suggested bindings for the specified interaction profile.

Valid Usage (Implicit)

- `type` **must** be `XR_TYPE_INTERACTION_PROFILE_SUGGESTED_BINDING`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain. See also: XrBindingModificationsKHR
- `suggestedBindings` **must** be a pointer to an array of `countSuggestedBindings` valid XrActionSuggestedBinding structures
- The `countSuggestedBindings` parameter **must** be greater than `0`

The XrActionSuggestedBinding structure is defined as:

```
// Provided by XR_VERSION_1_0
typedef struct XrActionSuggestedBinding {
    XrAction    action;
    XrPath      binding;
} XrActionSuggestedBinding;
```

Member Descriptions

- `action` is the XrAction handle for an action
- `binding` is the `XrPath` of a binding for the action specified in `action`. This "binding path" is any top level */user* path plus an applicable input subpath, for example */user/hand/right/input/trigger/click*. See Suggested Bindings for more details.

Valid Usage (Implicit)

- `action` **must** be a valid XrAction handle

The xrAttachSessionActionSets function is defined as:

```
// Provided by XR_VERSION_1_0
XrResult xrAttachSessionActionSets(
    XrSession                                   session,
    const XrSessionActionSetsAttachInfo*        attachInfo);
```

Parameter Descriptions

- `session` is the XrSession to attach the action sets to.
- `attachInfo` is the XrSessionActionSetsAttachInfo to provide information to attach action sets to the session.

xrAttachSessionActionSets attaches the XrActionSet handles in XrSessionActionSetsAttachInfo ::`actionSets` to the `session`. Action sets **must** be attached in order to be synchronized with xrSyncActions.

When an action set is attached to a session, that action set becomes immutable. See xrCreateAction and xrSuggestInteractionProfileBindings for details.

After action sets are attached to a session, if any unattached actions are passed to functions for the same session, then for those functions the runtime **must** return `XR_ERROR_ACTIONSET_NOT_ATTACHED`.

The runtime **must** return `XR_ERROR_ACTIONSETS_ALREADY_ATTACHED` if xrAttachSessionActionSets is called more than once for a given `session`.

Valid Usage (Implicit)

- `session` **must** be a valid XrSession handle
- `attachInfo` **must** be a pointer to a valid XrSessionActionSetsAttachInfo structure

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_ACTIONSETS_ALREADY_ATTACHED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_VALIDATION_FAILURE`

The XrSessionActionSetsAttachInfo structure is defined as:

```
// Provided by XR_VERSION_1_0
typedef struct XrSessionActionSetsAttachInfo {
    XrStructureType       type;
    const void*           next;
    uint32_t              countActionSets;
    const XrActionSet*    actionSets;
} XrSessionActionSetsAttachInfo;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR.
- `countActionSets` is an integer specifying the number of valid elements in the `actionSets` array.
- `actionSets` is a pointer to an array of one or more XrActionSet handles to be attached to the session.

Valid Usage (Implicit)

- `type` **must** be `XR_TYPE_SESSION_ACTION_SETS_ATTACH_INFO`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `actionSets` **must** be a pointer to an array of `countActionSets` valid XrActionSet handles
- The `countActionSets` parameter **must** be greater than `0`

11.5. Current Interaction Profile

The xrGetCurrentInteractionProfile function is defined as:

```
// Provided by XR_VERSION_1_0
XrResult xrGetCurrentInteractionProfile(
    XrSession                                   session,
    XrPath                                      topLevelUserPath,
    XrInteractionProfileState*                  interactionProfile);
```

Parameter Descriptions

- `session` is the XrSession for which the application would like to retrieve the current interaction profile.
- `topLevelUserPath` is the top level user path the application would like to retrieve the interaction profile for.
- `interactionProfile` is a pointer to an XrInteractionProfileState structure to receive the current interaction profile.

xrGetCurrentInteractionProfile retrieves the current interaction profile for a top level user path.

The runtime **must** return only interaction profiles for which the application has provided suggested bindings with xrSuggestInteractionProfileBindings or XR_NULL_PATH. The runtime **may** return interaction profiles that do not represent physically present hardware, for example if the runtime is using a known interaction profile to bind to hardware that the application is not aware of. The runtime **may** return an anticipated interaction profile, from the list of interaction profiles with suggested bindings (as supplied by the application through xrSuggestInteractionProfileBindings) for this top level */user* path, in the event that no controllers are active. Whether the runtime reports an interaction profile path or XR_NULL_PATH does **not** provide any signal to the application regarding presence or absence of a controller or other interaction method.

If xrAttachSessionActionSets has not yet been called for the `session`, the runtime **must** return `XR_ERROR_ACTIONSET_NOT_ATTACHED`. If `topLevelUserPath` is not one of the top level user paths described in Top level /user paths, the runtime **must** return `XR_ERROR_PATH_UNSUPPORTED`.

Valid Usage (Implicit)

- `session` **must** be a valid XrSession handle
- `interactionProfile` **must** be a pointer to an XrInteractionProfileState structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_ACTIONSET_NOT_ATTACHED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_PATH_INVALID
- XR_ERROR_PATH_UNSUPPORTED
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The XrInteractionProfileState structure is defined as:

```
// Provided by XR_VERSION_1_0
typedef struct XrInteractionProfileState {
    XrStructureType    type;
    void*              next;
    XrPath             interactionProfile;
} XrInteractionProfileState;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR.
- interactionProfile is the XrPath of the interaction profile path for the xrGetCurrentInteractionProfile::topLevelUserPath used to retrieve this state, or XR_NULL_PATH if there is no active interaction profile at that top level user path.

The runtime **must** only include interaction profiles that the application has provided bindings for via xrSuggestInteractionProfileBindings or XR_NULL_PATH. If the runtime is rebinding an interaction profile provided by the application to a device that the application did not provide bindings for, it **must** return the interaction profile path that it is emulating. If the runtime is unable to provide input

because it cannot emulate any of the application-provided interaction profiles, it **must** return XR_NULL_PATH.

Valid Usage (Implicit)

- `type` **must** be `XR_TYPE_INTERACTION_PROFILE_STATE`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain. See also: XrBatteryStateDisplayEXT

The XrEventDataInteractionProfileChanged structure is defined as:

```
// Provided by XR_VERSION_1_0
typedef struct XrEventDataInteractionProfileChanged {
    XrStructureType    type;
    const void*        next;
    XrSession          session;
} XrEventDataInteractionProfileChanged;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR.
- `session` is the XrSession for which at least one of the interaction profiles for a top level path has changed.

The XrEventDataInteractionProfileChanged event is queued to notify the application that the current interaction profile for one or more top level user paths has changed. This event **must** only be sent for interaction profiles that the application indicated its support for via xrSuggestInteractionProfileBindings. This event **must** only be queued for running sessions.

Upon receiving this event, an application **can** call xrGetCurrentInteractionProfile for each top level user path in use, if its behavior depends on the current interaction profile.

Valid Usage (Implicit)

- `type` **must** be `XR_TYPE_EVENT_DATA_INTERACTION_PROFILE_CHANGED`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

11.6. Reading Input Action State

The current state of an input action can be obtained by calling the `xrGetActionState*` function call that matches the XrActionType provided when the action was created. If a mismatched call is used to retrieve the state `XR_ERROR_ACTION_TYPE_MISMATCH` **must** be returned. `xrGetActionState*` calls for an action in an action set never bound to the session with xrAttachSessionActionSets **must** return `XR_ERROR_ACTIONSET_NOT_ATTACHED`.

The result of calls to `xrGetActionState*` for an XrAction and subaction path **must** not change between calls to xrSyncActions. When the combination of the parent XrActionSet and subaction path for an action is passed to xrSyncActions, the runtime **must** update the results from `xrGetActionState*` after this call with any changes to the state of the underlying hardware. When the parent action set and subaction path for an action is removed from or added to the list of active action sets passed to xrSyncActions, the runtime **must** update `isActive` to reflect the new active state after this call. In all cases the runtime **must** not change the results of `xrGetActionState*` calls between calls to xrSyncActions.

When `xrGetActionState*` or haptic output functions are called while the session is not focused, the runtime **must** set the `isActive` value to `XR_FALSE` and suppress all haptic output. Furthermore, the runtime should stop all in-progress haptic events when a session loses focus.

When retrieving action state, `lastChangeTime` **must** be set to the runtime's best estimate of when the physical state of the part of the device bound to that action last changed.

The `currentState` value is computed based on the current sync, combining the underlying input sources bound to the provided `subactionPaths` within this action.

The `changedSinceLastSync` value **must** be `XR_TRUE` if the computed `currentState` value differs from the `currentState` value that would have been computed as of the previous sync for the same `subactionPaths`. If there is no previous sync, or the action was not active for the previous sync, the `changedSinceLastSync` value **must** be set to `XR_FALSE`.

The `isActive` value **must** be `XR_TRUE` whenever an action is bound and a source is providing state data for the current sync. If the action is unbound or no source is present, the `isActive` value **must** be `XR_FALSE`. For any action which is inactive, the runtime **must** return zero (or `XR_FALSE`) for state, `XR_FALSE` for `changedSinceLastSync`, and `0` for `lastChangeTime`.

11.6.1. Resolving a single action bound to multiple inputs or outputs

It is often the case that a single action will be bound to multiple physical inputs simultaneously. In these circumstances, the runtime **must** resolve the ambiguity in that multiple binding as follows:

The current state value is selected based on the type of the action:

- Boolean actions - The current state **must** be the result of a boolean `OR` of all bound inputs
- Float actions - The current state **must** be the state of the input with the largest absolute value

- Vector2 actions - The current state **must** be the state of the input with the longest length
- Pose actions - The current state **must** be the state of a single pose source. The source of the pose **must** only be changed during a call to xrSyncAction. The runtime **should** only change the source in response to user actions, such as picking up a new controller, or external events, such as a controller running out of battery.
- Haptic actions - The runtime **must** send output events to all bound haptic devices

11.6.2. Structs to describe action and subaction paths

The XrActionStateGetInfo structure is used to provide action and subaction paths when calling `xrGetActionState*` function. It is defined as:

```
// Provided by XR_VERSION_1_0
typedef struct XrActionStateGetInfo {
    XrStructureType    type;
    const void*        next;
    XrAction           action;
    XrPath             subactionPath;
} XrActionStateGetInfo;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR.
- `action` is the XrAction being queried.
- `subactionPath` is the subaction path `XrPath` to query data from, or XR_NULL_PATH to specify all subaction paths. If the subaction path is specified, it is one of the subaction paths that were specified when the action was created. If the subaction path was not specified when the action was created, the runtime **must** return `XR_ERROR_PATH_UNSUPPORTED`. If this parameter is specified, the runtime **must** return data that originates only from the subaction paths specified.

See XrActionCreateInfo for a description of subaction paths, and the restrictions on their use.

Valid Usage (Implicit)

- `type` **must** be `XR_TYPE_ACTION_STATE_GET_INFO`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `action` **must** be a valid XrAction handle

The XrHapticActionInfo structure is used to provide action and subaction paths when calling `xr*HapticFeedback` function. It is defined as:

```
// Provided by XR_VERSION_1_0
typedef struct XrHapticActionInfo {
    XrStructureType    type;
    const void*        next;
    XrAction           action;
    XrPath             subactionPath;
} XrHapticActionInfo;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR.
- `action` is the XrAction handle for the desired output haptic action.
- `subactionPath` is the subaction path `XrPath` of the device to send the haptic event to, or XR_NULL_PATH to specify all subaction paths. If the subaction path is specified, it is one of the subaction paths that were specified when the action was created. If the subaction path was not specified when the action was created, the runtime **must** return `XR_ERROR_PATH_UNSUPPORTED`. If this parameter is specified, the runtime **must** trigger the haptic events only on the device from the subaction path.

See XrActionCreateInfo for a description of subaction paths, and the restrictions on their use.

Valid Usage (Implicit)

- `type` **must** be `XR_TYPE_HAPTIC_ACTION_INFO`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `action` **must** be a valid XrAction handle

11.6.3. Boolean Actions

xrGetActionStateBoolean retrieves the current state of a boolean action. It is defined as:

```
// Provided by XR_VERSION_1_0
XrResult xrGetActionStateBoolean(
    XrSession                                   session,
    const XrActionStateGetInfo*                 getInfo,
    XrActionStateBoolean*                       state);
```

Parameter Descriptions

- `session` is the XrSession to query.
- `getInfo` is a pointer to XrActionStateGetInfo to provide action and subaction paths information.
- `state` is a pointer to a valid XrActionStateBoolean into which the state will be placed.

Valid Usage (Implicit)

- `session` **must** be a valid XrSession handle
- `getInfo` **must** be a pointer to a valid XrActionStateGetInfo structure
- `state` **must** be a pointer to an XrActionStateBoolean structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_ACTIONSET_NOT_ATTACHED
- XR_ERROR_ACTION_TYPE_MISMATCH
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_PATH_INVALID
- XR_ERROR_PATH_UNSUPPORTED
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The XrActionStateBoolean structure is defined as:

```
// Provided by XR_VERSION_1_0
typedef struct XrActionStateBoolean {
    XrStructureType    type;
    void*              next;
    XrBool32           currentState;
    XrBool32           changedSinceLastSync;
    XrTime             lastChangeTime;
    XrBool32           isActive;
} XrActionStateBoolean;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR.
- `currentState` is the current state of the action.
- `changedSinceLastSync` is `XR_TRUE` if the value of `currentState` is different than it was before the most recent call to xrSyncActions. This parameter can be combined with `currentState` to detect rising and falling edges since the previous call to xrSyncActions. E.g. if both `changedSinceLastSync` and `currentState` are `XR_TRUE` then a rising edge (`XR_FALSE` to `XR_TRUE`) has taken place.
- `lastChangeTime` is the `XrTime` associated with the most recent change to this action's state.
- `isActive` is `XR_TRUE` if and only if there exists an input source that is contributing to the current state of this action.

When multiple input sources are bound to this action, the `currentState` follows the previously defined rule to resolve ambiguity.

Valid Usage (Implicit)

- `type` **must** be `XR_TYPE_ACTION_STATE_BOOLEAN`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

11.6.4. Scalar and Vector Actions

xrGetActionStateFloat retrieves the current state of a floating-point action. It is defined as:

```
// Provided by XR_VERSION_1_0
XrResult xrGetActionStateFloat(
    XrSession                                   session,
    const XrActionStateGetInfo*                 getInfo,
    XrActionStateFloat*                         state);
```

Parameter Descriptions

- `session` is the XrSession to query.
- `getInfo` is a pointer to XrActionStateGetInfo to provide action and subaction paths information.
- `state` is a pointer to a valid XrActionStateFloat into which the state will be placed.

Valid Usage (Implicit)

- `session` **must** be a valid XrSession handle
- `getInfo` **must** be a pointer to a valid XrActionStateGetInfo structure
- `state` **must** be a pointer to an XrActionStateFloat structure

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_ACTIONSET_NOT_ATTACHED`
- `XR_ERROR_ACTION_TYPE_MISMATCH`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_PATH_INVALID`
- `XR_ERROR_PATH_UNSUPPORTED`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_VALIDATION_FAILURE`

The XrActionStateFloat structure is defined as:

```
// Provided by XR_VERSION_1_0
typedef struct XrActionStateFloat {
    XrStructureType    type;
    void*              next;
    float              currentState;
    XrBool32           changedSinceLastSync;
    XrTime             lastChangeTime;
    XrBool32           isActive;
} XrActionStateFloat;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR.
- `currentState` is the current state of the Action.
- `changedSinceLastSync` is `XR_TRUE` if the value of `currentState` is different than it was before the most recent call to xrSyncActions.
- `lastChangeTime` is the `XrTime` associated with the most recent change to this action's state.
- `isActive` is `XR_TRUE` if and only if there exists an input source that is contributing to the current state of this action.

When multiple input sources are bound to this action, the `currentState` follows the previously defined rule to resolve ambiguity.

Valid Usage (Implicit)

- `type` **must** be `XR_TYPE_ACTION_STATE_FLOAT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

xrGetActionStateVector2f retrieves the current state of a two-dimensional vector action. It is defined as:

```
// Provided by XR_VERSION_1_0
XrResult xrGetActionStateVector2f(
    XrSession                                   session,
    const XrActionStateGetInfo*                 getInfo,
    XrActionStateVector2f*                      state);
```

Parameter Descriptions

- `session` is the XrSession to query.
- `getInfo` is a pointer to XrActionStateGetInfo to provide action and subaction paths information.
- `state` is a pointer to a valid XrActionStateVector2f into which the state will be placed.

Valid Usage (Implicit)

- `session` **must** be a valid XrSession handle
- `getInfo` **must** be a pointer to a valid XrActionStateGetInfo structure
- `state` **must** be a pointer to an XrActionStateVector2f structure

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_ACTIONSET_NOT_ATTACHED`
- `XR_ERROR_ACTION_TYPE_MISMATCH`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_PATH_INVALID`
- `XR_ERROR_PATH_UNSUPPORTED`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_VALIDATION_FAILURE`

The XrActionStateVector2f structure is defined as:

```
// Provided by XR_VERSION_1_0
typedef struct XrActionStateVector2f {
    XrStructureType    type;
    void*              next;
    XrVector2f         currentState;
    XrBool32           changedSinceLastSync;
    XrTime             lastChangeTime;
    XrBool32           isActive;
} XrActionStateVector2f;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR.
- `currentState` is the current XrVector2f state of the Action.
- `changedSinceLastSync` is `XR_TRUE` if the value of `currentState` is different than it was before the most recent call to xrSyncActions.
- `lastChangeTime` is the `XrTime` associated with the most recent change to this action's state.
- `isActive` is `XR_TRUE` if and only if there exists an input source that is contributing to the current state of this action.

When multiple input sources are bound to this action, the `currentState` follows the previously defined rule to resolve ambiguity.

Valid Usage (Implicit)

- `type` **must** be `XR_TYPE_ACTION_STATE_VECTOR2F`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

11.6.5. Pose Actions

The xrGetActionStatePose function is defined as:

```
// Provided by XR_VERSION_1_0
XrResult xrGetActionStatePose(
    XrSession                                   session,
    const XrActionStateGetInfo*                 getInfo,
    XrActionStatePose*                          state);
```

Parameter Descriptions

- `session` is the XrSession to query.
- `getInfo` is a pointer to XrActionStateGetInfo to provide action and subaction paths information.
- `state` is a pointer to a valid XrActionStatePose into which the state will be placed.

xrGetActionStatePose returns information about the binding and active state for the specified action. To determine the pose of this action at a historical or predicted time, the application **can** create an action space using xrCreateActionSpace. Then, after each sync, the application **can** locate the pose of this action space within a base space using xrLocateSpace.

Valid Usage (Implicit)

- `session` **must** be a valid XrSession handle
- `getInfo` **must** be a pointer to a valid XrActionStateGetInfo structure
- `state` **must** be a pointer to an XrActionStatePose structure

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_ACTIONSET_NOT_ATTACHED`
- `XR_ERROR_ACTION_TYPE_MISMATCH`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_PATH_INVALID`
- `XR_ERROR_PATH_UNSUPPORTED`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_VALIDATION_FAILURE`

The XrActionStatePose structure is defined as:

```
// Provided by XR_VERSION_1_0
typedef struct XrActionStatePose {
    XrStructureType    type;
    void*              next;
    XrBool32           isActive;
} XrActionStatePose;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR.
- `isActive` is `XR_TRUE` if and only if there exists an input source that is being tracked by this pose action.

A pose action **must** not be bound to multiple input sources, according to the previously defined rule.

Valid Usage (Implicit)

- `type` **must** be `XR_TYPE_ACTION_STATE_POSE`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

11.7. Output Actions and Haptics

Haptic feedback is sent to a device using the xrApplyHapticFeedback function. The `hapticEvent` points to a supported event structure. All event structures have in common that the first element is an XrHapticBaseHeader which can be used to determine the type of the haptic event.

Haptic feedback may be immediately halted for a haptic action using the xrStopHapticFeedback function.

Output action requests activate immediately and **must** not wait for the next call to xrSyncActions.

If a haptic event is sent to an action before a previous haptic event completes, the latest event will take precedence and the runtime **must** cancel all preceding incomplete haptic events on that action.

Output action requests **must** be discarded and have no effect on hardware if the application's session is not focused.

Output action requests for an action in an action set never attached to the session with xrAttachSessionActionSets **must** return `XR_ERROR_ACTIONSET_NOT_ATTACHED`.

The only haptics type supported by unextended OpenXR is XrHapticVibration.

The xrApplyHapticFeedback function is defined as:

```
// Provided by XR_VERSION_1_0
XrResult xrApplyHapticFeedback(
    XrSession                                   session,
    const XrHapticActionInfo*                   hapticActionInfo,
    const XrHapticBaseHeader*                   hapticFeedback);
```

Parameter Descriptions

- session is the XrSession to start outputting to.
- hapticActionInfo is a pointer to XrHapticActionInfo to provide action and subaction paths information.
- hapticFeedback is a pointer to a haptic event structure which starts with an XrHapticBaseHeader.

Triggers a haptic event through the specified action of type XR_ACTION_TYPE_VIBRATION_OUTPUT. The runtime **should** deliver this request to the appropriate device, but exactly which device, if any, this event is sent to is up to the runtime to decide. If an appropriate device is unavailable the runtime **may** ignore this request for haptic feedback.

If session is not focused, the runtime **must** return XR_SESSION_NOT_FOCUSED, and not trigger a haptic event.

If another haptic event from this session is currently happening on the device bound to this action, the runtime **must** interrupt that other event and replace it with the new one.

Valid Usage (Implicit)

- session **must** be a valid XrSession handle
- hapticActionInfo **must** be a pointer to a valid XrHapticActionInfo structure
- hapticFeedback **must** be a pointer to a valid XrHapticBaseHeader-based structure. See also: XrHapticAmplitudeEnvelopeVibrationFB, XrHapticParametricVibrationEXT, XrHapticPcmVibrationFB, XrHapticVibration

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING
- XR_SESSION_NOT_FOCUSED

Failure

- XR_ERROR_ACTIONSET_NOT_ATTACHED
- XR_ERROR_ACTION_TYPE_MISMATCH
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_PATH_INVALID
- XR_ERROR_PATH_UNSUPPORTED
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE
- XR_ERROR_FEATURE_UNSUPPORTED (if XR_EXT_haptic_parametric is enabled)

The XrHapticBaseHeader structure is defined as:

```
// Provided by XR_VERSION_1_0
typedef struct XrHapticBaseHeader {
    XrStructureType    type;
    const void*        next;
} XrHapticBaseHeader;
```

Member Descriptions

- type is the XrStructureType of this structure. This base structure itself has no associated XrStructureType value.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR.

Valid Usage (Implicit)

- `type` **must** be one of the following XrStructureType values: `XR_TYPE_HAPTIC_AMPLITUDE_ENVELOPE_VIBRATION_FB`, `XR_TYPE_HAPTIC_PARAMETRIC_VIBRATION_EXT`, `XR_TYPE_HAPTIC_PCM_VIBRATION_FB`, `XR_TYPE_HAPTIC_VIBRATION`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

The XrHapticVibration structure is defined as:

```
// Provided by XR_VERSION_1_0
typedef struct XrHapticVibration {
    XrStructureType    type;
    const void*        next;
    XrDuration         duration;
    float              frequency;
    float              amplitude;
} XrHapticVibration;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR.
- `duration` is the number of nanoseconds the vibration **should** last. If XR_MIN_HAPTIC_DURATION is specified, the runtime **must** produce a short haptics pulse of minimal supported duration for the haptic device.
- `frequency` is the frequency of the vibration in Hz. If XR_FREQUENCY_UNSPECIFIED is specified, it is left to the runtime to decide the optimal frequency value to use.
- `amplitude` is the amplitude of the vibration between 0.0 and 1.0.

The XrHapticVibration is used in calls to xrApplyHapticFeedback that trigger `vibration` output actions.

The `duration`, and `frequency` parameters **may** be clamped to implementation-dependent ranges.

Valid Usage (Implicit)

- `type` **must** be `XR_TYPE_HAPTIC_VIBRATION`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

XR_MIN_HAPTIC_DURATION is used to indicate to the runtime that a short haptic pulse of the minimal supported duration for the haptic device.

```
// Provided by XR_VERSION_1_0
#define XR_MIN_HAPTIC_DURATION -1
```

XR_FREQUENCY_UNSPECIFIED is used to indicate that the application wants the runtime to decide what the optimal frequency is for the haptic pulse.

```
// Provided by XR_VERSION_1_0
#define XR_FREQUENCY_UNSPECIFIED 0
```

The xrStopHapticFeedback function is defined as:

```
// Provided by XR_VERSION_1_0
XrResult xrStopHapticFeedback(
    XrSession                                   session,
    const XrHapticActionInfo*                   hapticActionInfo);
```

Parameter Descriptions

- `session` is the XrSession to stop outputting to.
- `hapticActionInfo` is a pointer to an XrHapticActionInfo to provide action and subaction path information.

If a haptic event from this XrAction is in progress, when this function is called the runtime **must** stop that event.

If `session` is not focused, the runtime **must** return `XR_SESSION_NOT_FOCUSED`.

Valid Usage (Implicit)

- `session` **must** be a valid XrSession handle
- `hapticActionInfo` **must** be a pointer to a valid XrHapticActionInfo structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING
- XR_SESSION_NOT_FOCUSED

Failure

- XR_ERROR_ACTIONSET_NOT_ATTACHED
- XR_ERROR_ACTION_TYPE_MISMATCH
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_PATH_INVALID
- XR_ERROR_PATH_UNSUPPORTED
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

11.8. Input Action State Synchronization

The xrSyncActions function is defined as:

```
// Provided by XR_VERSION_1_0
XrResult xrSyncActions(
    XrSession                                   session,
    const XrActionsSyncInfo*                    syncInfo);
```

Parameter Descriptions

- session is a handle to the XrSession that all provided action set handles belong to.
- syncInfo is an XrActionsSyncInfo providing information to synchronize action states.

xrSyncActions updates the current state of input actions. Repeated input action state queries between subsequent synchronization calls **must** return the same values. The XrActionSet structures referenced in the XrActionsSyncInfo::activeActionSets **must** have been previously attached to the session via xrAttachSessionActionSets. If any action sets not attached to this session are passed to xrSyncActions it **must** return XR_ERROR_ACTIONSET_NOT_ATTACHED. Subsets of the bound action sets **can** be synchronized in

order to control which actions are seen as active.

If `session` is not focused, the runtime **must** return `XR_SESSION_NOT_FOCUSED`, and all action states in the session **must** be inactive.

Valid Usage (Implicit)

- `session` **must** be a valid XrSession handle
- `syncInfo` **must** be a pointer to a valid XrActionsSyncInfo structure

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`
- `XR_SESSION_NOT_FOCUSED`

Failure

- `XR_ERROR_ACTIONSET_NOT_ATTACHED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_PATH_INVALID`
- `XR_ERROR_PATH_UNSUPPORTED`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_VALIDATION_FAILURE`

The XrActionsSyncInfo structure is defined as:

```
// Provided by XR_VERSION_1_0
typedef struct XrActionsSyncInfo {
    XrStructureType             type;
    const void*                 next;
    uint32_t                    countActiveActionSets;
    const XrActiveActionSet*    activeActionSets;
} XrActionsSyncInfo;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR.
- `countActiveActionSets` is an integer specifying the number of valid elements in the `activeActionSets` array.
- `activeActionSets` is `NULL` or a pointer to an array of one or more XrActiveActionSet structures that should be synchronized.

Valid Usage (Implicit)

- `type` **must** be `XR_TYPE_ACTIONS_SYNC_INFO`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain. See also: XrActiveActionSetPrioritiesEXT
- If `countActiveActionSets` is not `0`, `activeActionSets` **must** be a pointer to an array of `countActiveActionSets` valid XrActiveActionSet structures

The XrActiveActionSet structure is defined as:

```
// Provided by XR_VERSION_1_0
typedef struct XrActiveActionSet {
    XrActionSet    actionSet;
    XrPath         subactionPath;
} XrActiveActionSet;
```

Member Descriptions

- `actionSet` is the handle of the action set to activate.
- `subactionPath` is a subaction path that was declared when one or more actions in the action set was created or XR_NULL_PATH. If the application wants to activate the action set on more than one subaction path, it **can** include additional XrActiveActionSet structs with the other `subactionPath` values. Using XR_NULL_PATH as the value for `subactionPath`, acts as a wildcard for all subaction paths on the actions in the action set. If the subaction path was not specified on any of the actions in the actionSet when that action was created, the runtime **must** return `XR_ERROR_PATH_UNSUPPORTED`.

This structure defines a single active action set and subaction path combination. Applications **can**

provide a list of these structures to the xrSyncActions function.

Valid Usage (Implicit)

- `actionSet` **must** be a valid XrActionSet handle

11.9. Bound Sources

An application **can** use the xrEnumerateBoundSourcesForAction and xrGetInputSourceLocalizedName calls to prompt the user which physical inputs to use in order to perform an action. The bound `sources` are opaque `XrPath` values representing the physical controls that an action is bound to. An action **may** be bound to multiple sources at one time, for example an action named `hold` could be bound to both the X and A buttons.

Once the bound sources for an action are obtained, the application **can** gather additional information about it. xrGetInputSourceLocalizedName returns a localized human-readable string describing the bound physical control, e.g. 'A Button'.

The xrEnumerateBoundSourcesForAction function is defined as:

```
// Provided by XR_VERSION_1_0
XrResult xrEnumerateBoundSourcesForAction(
    XrSession                                   session,
    const XrBoundSourcesForActionEnumerateInfo* enumerateInfo,
    uint32_t                                    sourceCapacityInput,
    uint32_t*                                   sourceCountOutput,
    XrPath*                                     sources);
```

Parameter Descriptions

- `session` is the XrSession being queried.
- `enumerateInfo` is an XrBoundSourcesForActionEnumerateInfo providing the query information.
- `sourceCapacityInput` is the capacity of the array, or 0 to indicate a request to retrieve the required capacity.
- `sourceCountOutput` is a pointer to the count of `sources`, or a pointer to the required capacity in the case that `sourceCapacityInput` is insufficient.
- `sources` is a pointer to an application-allocated array that will be filled with the `XrPath` values for all bound sources. It **can** be `NULL` if `sourceCapacityInput` is 0.
- See the Buffer Size Parameters section for a detailed description of retrieving the required `sources` size.

If an action is unbound, xrEnumerateBoundSourcesForAction **must** assign `0` to the value pointed-to by `sourceCountOutput` and not modify the array.

xrEnumerateBoundSourcesForAction **must** return `XR_ERROR_ACTIONSET_NOT_ATTACHED` if passed an action in an action set never attached to the session with xrAttachSessionActionSets.

As bindings for actions do not change between calls to xrSyncActions, xrEnumerateBoundSourcesForAction **must** enumerate the same set of bound sources, or absence of bound sources, for a given query (defined by the `enumerateInfo` parameter) between any two calls to xrSyncActions.



Note

The `XrPath` bound sources returned by the runtime are opaque values and **should** not be inspected or persisted. They are only intended for use in conjunction with xrGetInputSourceLocalizedName.

Valid Usage (Implicit)

- `session` **must** be a valid XrSession handle
- `enumerateInfo` **must** be a pointer to a valid XrBoundSourcesForActionEnumerateInfo structure
- `sourceCountOutput` **must** be a pointer to a `uint32_t` value
- If `sourceCapacityInput` is not `0`, `sources` **must** be a pointer to an array of `sourceCapacityInput` `XrPath` values

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_ACTIONSET_NOT_ATTACHED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_PATH_INVALID
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_SIZE_INSUFFICIENT
- XR_ERROR_VALIDATION_FAILURE

The XrBoundSourcesForActionEnumerateInfo structure is defined as:

```
// Provided by XR_VERSION_1_0
typedef struct XrBoundSourcesForActionEnumerateInfo {
    XrStructureType    type;
    const void*        next;
    XrAction           action;
} XrBoundSourcesForActionEnumerateInfo;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR.
- action is the handle of the action to query.

Valid Usage (Implicit)

- `type` **must** be `XR_TYPE_BOUND_SOURCES_FOR_ACTION_ENUMERATE_INFO`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `action` **must** be a valid XrAction handle

The xrGetInputSourceLocalizedName function is defined as:

```
// Provided by XR_VERSION_1_0
XrResult xrGetInputSourceLocalizedName(
    XrSession                                   session,
    const XrInputSourceLocalizedNameGetInfo*    getInfo,
    uint32_t                                    bufferCapacityInput,
    uint32_t*                                   bufferCountOutput,
    char*                                       buffer);
```

Parameter Descriptions

- `session` is a handle to the XrSession associated with the action that reported this bound source.
- `getInfo` is an XrInputSourceLocalizedNameGetInfo providing the query information.
- `bufferCapacityInput` is the capacity of the `buffer`, or 0 to indicate a request to retrieve the required capacity.
- `bufferCountOutput` is a pointer to the count of name characters written to `buffer` (including the terminating `\0`), or a pointer to the required capacity in the case that `bufferCapacityInput` is insufficient.
- `buffer` is a pointer to an application-allocated buffer that will be filled with the bound source name. It **can** be `NULL` if `bufferCapacityInput` is 0.
- See the Buffer Size Parameters section for a detailed description of retrieving the required `buffer` size.

xrGetInputSourceLocalizedName returns a string for the bound source in the current system locale.

If xrAttachSessionActionSets has not yet been called for the session, the runtime **must** return `XR_ERROR_ACTIONSET_NOT_ATTACHED`.

Valid Usage (Implicit)

- `session` **must** be a valid XrSession handle
- `getInfo` **must** be a pointer to a valid XrInputSourceLocalizedNameGetInfo structure
- `bufferCountOutput` **must** be a pointer to a `uint32_t` value
- If `bufferCapacityInput` is not `0`, `buffer` **must** be a pointer to an array of `bufferCapacityInput` char values

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_ACTIONSET_NOT_ATTACHED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_PATH_INVALID`
- `XR_ERROR_PATH_UNSUPPORTED`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_SIZE_INSUFFICIENT`
- `XR_ERROR_VALIDATION_FAILURE`

The XrInputSourceLocalizedNameGetInfo structure is defined as:

```
// Provided by XR_VERSION_1_0
typedef struct XrInputSourceLocalizedNameGetInfo {
    XrStructureType                    type;
    const void*                        next;
    XrPath                             sourcePath;
    XrInputSourceLocalizedNameFlags    whichComponents;
} XrInputSourceLocalizedNameGetInfo;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR.
- `sourcePath` is an `XrPath` representing a bound source returned by xrEnumerateBoundSourcesForAction.
- `whichComponents` is any set of flags from XrInputSourceLocalizedNameFlagBits.

The result of passing an `XrPath` `sourcePath` **not** retrieved from xrEnumerateBoundSourcesForAction is not specified.

Valid Usage (Implicit)

- `type` **must** be `XR_TYPE_INPUT_SOURCE_LOCALIZED_NAME_GET_INFO`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `whichComponents` **must** be a valid combination of XrInputSourceLocalizedNameFlagBits values
- `whichComponents` **must** not be `0`

The XrInputSourceLocalizedNameGetInfo::`whichComponents` parameter is of the following type, and contains a bitwise-OR of one or more of the bits defined in XrInputSourceLocalizedNameFlagBits.

```
// Provided by XR_VERSION_1_0
typedef XrFlags64 XrInputSourceLocalizedNameFlags;
```

```
// Provided by XR_VERSION_1_0
// Flag bits for XrInputSourceLocalizedNameFlags
static const XrInputSourceLocalizedNameFlags XR_INPUT_SOURCE_LOCALIZED_NAME_USER_PATH_BIT
= 0x00000001;
static const XrInputSourceLocalizedNameFlags
XR_INPUT_SOURCE_LOCALIZED_NAME_INTERACTION_PROFILE_BIT = 0x00000002;
static const XrInputSourceLocalizedNameFlags XR_INPUT_SOURCE_LOCALIZED_NAME_COMPONENT_BIT
= 0x00000004;
```

The flag bits have the following meanings:

Flag Descriptions

- `XR_INPUT_SOURCE_LOCALIZED_NAME_USER_PATH_BIT` indicates that the runtime **must** include the user path portion of the string in the result, if available. E.g. `Left Hand`.
- `XR_INPUT_SOURCE_LOCALIZED_NAME_INTERACTION_PROFILE_BIT` indicates that the runtime **must** include the interaction profile portion of the string in the result, if available. E.g. `Vive Controller`.
- `XR_INPUT_SOURCE_LOCALIZED_NAME_COMPONENT_BIT` indicates that the runtime **must** include the input component portion of the string in the result, if available. E.g. `Trigger`.

Chapter 12. List of Current Extensions

- XR_KHR_android_create_instance
- XR_KHR_android_surface_swapchain
- XR_KHR_android_thread_settings
- XR_KHR_binding_modification
- XR_KHR_composition_layer_color_scale_bias
- XR_KHR_composition_layer_cube
- XR_KHR_composition_layer_cylinder
- XR_KHR_composition_layer_depth
- XR_KHR_composition_layer_equirect
- XR_KHR_composition_layer_equirect2
- XR_KHR_convert_timespec_time
- XR_KHR_D3D11_enable
- XR_KHR_D3D12_enable
- XR_KHR_extended_struct_name_lengths
- XR_KHR_generic_controller
- XR_KHR_loader_init
- XR_KHR_loader_init_android
- XR_KHR_metal_enable
- XR_KHR_opengl_enable
- XR_KHR_opengl_es_enable
- XR_KHR_swapchain_usage_input_attachment_bit
- XR_KHR_visibility_mask
- XR_KHR_vulkan_enable
- XR_KHR_vulkan_enable2
- XR_KHR_vulkan_swapchain_format_list
- XR_KHR_win32_convert_performance_counter_time
- XR_EXT_active_action_set_priority
- XR_EXT_composition_layer_inverted_alpha
- XR_EXT_conformance_automation
- XR_EXT_debug_utils

- XR_EXT_dpad_binding
- XR_EXT_eye_gaze_interaction
- XR_EXT_frame_synthesis
- XR_EXT_future
- XR_EXT_hand_interaction
- XR_EXT_hand_joints_motion_range
- XR_EXT_hand_tracking
- XR_EXT_hand_tracking_data_source
- XR_EXT_haptic_parametric
- XR_EXT_interaction_profile_battery_state_display
- XR_EXT_interaction_render_model
- XR_EXT_loader_init_properties
- XR_EXT_performance_settings
- XR_EXT_render_model
- XR_EXT_spatial_anchor
- XR_EXT_spatial_entity
- XR_EXT_spatial_marker_tracking
- XR_EXT_spatial_persistence
- XR_EXT_spatial_persistence_operations
- XR_EXT_spatial_plane_tracking
- XR_EXT_stationary_reference_space
- XR_EXT_thermal_query
- XR_EXT_user_presence
- XR_EXT_view_configuration_depth_range
- XR_EXT_view_configuration_views_change
- XR_EXT_win32_appcontainer_compatible
- XR_ALMALENCE_digital_lens_control
- XR_ANDROID_anchor_sharing_export
- XR_ANDROID_composition_layer_passthrough_mesh
- XR_ANDROID_eye_tracking
- XR_ANDROID_face_tracking
- XR_ANDROID_geospatial

- XR_ANDROID_geospatial_anchor
- XR_ANDROID_google_cloud_auth
- XR_ANDROID_light_estimation
- XR_ANDROID_mouse_interaction
- XR_ANDROID_passthrough_camera_state
- XR_ANDROID_performance_metrics
- XR_ANDROID_recommended_resolution
- XR_ANDROID_scene_meshing
- XR_ANDROID_spatial_anchor_space
- XR_ANDROID_spatial_component_subsumed_by
- XR_ANDROID_spatial_discovery_bounds
- XR_ANDROID_spatial_discovery_raycast
- XR_ANDROID_spatial_entity_bound_anchor
- XR_ANDROID_spatial_object_tracking
- XR_ANDROID_trackables_image
- XR_ANDROID_unbounded_reference_space
- XR_BD_body_tracking
- XR_BD_body_tracking_auxiliary_metrics
- XR_BD_dynamic_object_keyboard
- XR_BD_dynamic_object_mouse
- XR_BD_dynamic_object_tracking
- XR_BD_facial_simulation
- XR_BD_future_progress
- XR_BD_spatial_anchor
- XR_BD_spatial_anchor_sharing
- XR_BD_spatial_audio_rendering
- XR_BD_spatial_mesh
- XR_BD_spatial_plane
- XR_BD_spatial_scene
- XR_BD_spatial_sensing
- XR_BD_ultra_controller_interaction
- XR_EPIC_view_configuration_fov

- XR_FB_android_surface_swapchain_create
- XR_FB_body_tracking
- XR_FB_color_space
- XR_FB_composition_layer_alpha_blend
- XR_FB_composition_layer_depth_test
- XR_FB_composition_layer_image_layout
- XR_FB_composition_layer_secure_content
- XR_FB_composition_layer_settings
- XR_FB_display_refresh_rate
- XR_FB_eye_tracking_social
- XR_FB_face_tracking
- XR_FB_face_tracking2
- XR_FB_foveation
- XR_FB_foveation_configuration
- XR_FB_foveation_vulkan
- XR_FB_hand_tracking_aim
- XR_FB_hand_tracking_capsules
- XR_FB_hand_tracking_mesh
- XR_FB_haptic_amplitude_envelope
- XR_FB_haptic_pcm
- XR_FB_keyboard_tracking
- XR_FB_passthrough
- XR_FB_passthrough_keyboard_hands
- XR_FB_render_model
- XR_FB_scene
- XR_FB_scene_capture
- XR_FB_spatial_entity
- XR_FB_spatial_entity_container
- XR_FB_spatial_entity_query
- XR_FB_spatial_entity_sharing
- XR_FB_spatial_entity_storage
- XR_FB_spatial_entity_storage_batch

- XR_FB_spatial_entity_user
- XR_FB_swapchain_update_state
- XR_FB_swapchain_update_state_android_surface
- XR_FB_swapchain_update_state_opengl_es
- XR_FB_swapchain_update_state_vulkan
- XR_FB_triangle_mesh
- XR_HTC_anchor
- XR_HTC_body_tracking
- XR_HTC_facial_tracking
- XR_HTC_foveation
- XR_HTC_hand_interaction
- XR_HTC_passthrough
- XR_HTC_vive_wrist_tracker_interaction
- XR_HUAWEI_controller_interaction
- XR_LOGITECH_mx_ink_stylus_interaction
- XR_META_automatic_layer_filter
- XR_META_body_tracking_calibration
- XR_META_body_tracking_fidelity
- XR_META_body_tracking_full_body
- XR_META_boundary_visibility
- XR_META_colocation_discovery
- XR_META_detached_controllers
- XR_META_environment_depth
- XR_META_environment_raycast
- XR_META_face_tracking_visemes
- XR_META_foveation_eye_tracked
- XR_META_hand_tracking_frequency_hint
- XR_META_hand_tracking_microgestures
- XR_META_hand_tracking_unextrapolated_poses
- XR_META_hand_tracking_wide_motion_mode2
- XR_META_headset_id
- XR_META_local_dimming

- XR_META_passthrough_color_lut
- XR_META_passthrough_layer_resumed_event
- XR_META_passthrough_preferences
- XR_META_performance_metrics
- XR_META_recommended_layer_resolution
- XR_META_simultaneous_hands_and_controllers
- XR_META_spatial_entity_discovery
- XR_META_spatial_entity_group_sharing
- XR_META_spatial_entity_mesh
- XR_META_spatial_entity_persistence
- XR_META_spatial_entity_room_mesh
- XR_META_spatial_entity_semantic_label
- XR_META_spatial_entity_sharing
- XR_META_tile_properties_hint
- XR_META_virtual_keyboard
- XR_META_vulkan_swapchain_create_info
- XR_ML_compat
- XR_ML_facial_expression
- XR_ML_frame_end_info
- XR_ML_global_dimmer
- XR_ML_localization_map
- XR_ML_marker_understanding
- XR_ML_spatial_anchors
- XR_ML_spatial_anchors_storage
- XR_ML_system_notifications
- XR_ML_user_calibration
- XR_ML_view_configuration_depth_range_change
- XR_ML_world_mesh_detection
- XR_MND_headless
- XR_OCULUS_audio_device_guid
- XR_OCULUS_external_camera
- XR_OPPO_controller_interaction

- XR_QCOM_tracking_optimization_settings
- XR_SONY_swapchain_color_space
- XR_ULTRALEAP_hand_tracking_forearm
- XR_VALVE_analog_threshold
- XR_VARJO_composition_layer_depth_test
- XR_VARJO_environment_depth_estimation
- XR_VARJO_foveated_rendering
- XR_VARJO_marker_tracking
- XR_VARJO_view_offset
- XR_VARJO_xr4_controller_interaction
- XR_YVR_controller_interaction

12.1. XR_KHR_android_create_instance

Name String

`XR_KHR_android_create_instance`

Extension Type

Instance extension

Registered Extension Number

9

Revision

3

Ratification Status

Ratified

Extension and Version Dependencies

OpenXR 1.0

Last Modified Date

2019-07-17

IP Status

No known IP claims.

Contributors

Robert Menzel, NVIDIA
Martin Renschler, Qualcomm
Krzysztof Kosiński, Google

Overview

When the application creates an XrInstance object on Android systems, additional information from the application has to be provided to the XR runtime.

The Android XR runtime **must** return error `XR_ERROR_VALIDATION_FAILURE` if the additional information is not provided by the application or if the additional parameters are invalid.

New Object Types

New Flag Types

New Enum Constants

XrStructureType enumeration is extended with:

- XR_TYPE_INSTANCE_CREATE_INFO_ANDROID_KHR

New Enums

New Structures

The XrInstanceCreateInfoAndroidKHR structure is defined as:

```
// Provided by XR_KHR_android_create_instance
typedef struct XrInstanceCreateInfoAndroidKHR {
    XrStructureType    type;
    const void*        next;
    void*              applicationVM;
    void*              applicationActivity;
} XrInstanceCreateInfoAndroidKHR;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- applicationVM is a pointer to the JNI's opaque JavaVM structure, cast to a void pointer.
- applicationActivity is a JNI reference to an android.app.Activity that will drive the session lifecycle of this instance, cast to a void pointer.

XrInstanceCreateInfoAndroidKHR contains additional Android specific information needed when calling xrCreateInstance. The applicationVM field should be populated with the JavaVM structure received by the JNI_OnLoad function, while the applicationActivity field will typically contain a reference to a Java activity object received through an application-specific native method. The XrInstanceCreateInfoAndroidKHR structure **must** be provided in the next chain of the XrInstanceCreateInfo structure when calling xrCreateInstance.

Valid Usage (Implicit)

- The XR_KHR_android_create_instance extension **must** be enabled prior to using XrInstanceCreateInfoAndroidKHR
- type **must** be XR_TYPE_INSTANCE_CREATE_INFO_ANDROID_KHR
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- applicationVM **must** be a pointer value
- applicationActivity **must** be a pointer value

New Functions

Issues

Version History

- Revision 1, 2017-05-26 (Robert Menzel)
  - Initial draft
- Revision 2, 2019-01-24 (Martin Renschler)
  - Added error code, reformatted
- Revision 3, 2019-07-17 (Krzysztof Kosiński)
  - Non-substantive clarifications.

12.2. XR_KHR_android_surface_swapchain

Name String

`XR_KHR_android_surface_swapchain`

Extension Type

Instance extension

Registered Extension Number

5

Revision

4

Ratification Status

Ratified

Extension and Version Dependencies

OpenXR 1.0

Last Modified Date

2019-05-30

IP Status

No known IP claims.

Contributors

Krzysztof Kosiński, Google
Johannes van Waveren, Oculus
Martin Renschler, Qualcomm

Overview

A common activity in XR is to view an image stream. Image streams are often the result of camera previews or decoded video streams. On Android, the basic primitive representing the producer end of an image queue is the class `android.view.Surface`. This extension provides a special swapchain that uses an `android.view.Surface` as its producer end.

New Object Types

New Flag Types

New Enum Constants

New Enums

New Structures

New Functions

To create an XrSwapchain object and an Android Surface object call:

```
// Provided by XR_KHR_android_surface_swapchain
XrResult xrCreateSwapchainAndroidSurfaceKHR(
    XrSession                                   session,
    const XrSwapchainCreateInfo*                info,
    XrSwapchain*                                swapchain,
    jobject*                                    surface);
```

Parameter Descriptions

- `session` is an XrSession handle previously created with xrCreateSession.
- `info` is a pointer to an XrSwapchainCreateInfo structure.
- `swapchain` is a pointer to a handle in which the created XrSwapchain is returned.
- `surface` is a pointer to a `jobject` where the created Android Surface is returned.

xrCreateSwapchainAndroidSurfaceKHR creates an XrSwapchain object returned in `swapchain` and an Android Surface `jobject` returned in `surface`. The `jobject` **must** be valid to be passed back to Java code using JNI and **must** be valid to be used with ordinary Android APIs for submitting images to Surfaces. The returned XrSwapchain **must** be valid to be referenced in XrSwapchainSubImage structures to show content on the screen. The width and height passed in XrSwapchainCreateInfo **may** not be persistent throughout the life cycle of the created swapchain, since on Android, the size of the images is controlled by the producer and possibly changes at any time.

The only function that is allowed to be called on the XrSwapchain returned from this function is

xrDestroySwapchain. For example, calling any of the functions xrEnumerateSwapchainImages, xrAcquireSwapchainImage, xrWaitSwapchainImage or xrReleaseSwapchainImage is invalid.

When the application receives the XrEventDataSessionStateChanged event with the XR_SESSION_STATE_STOPPING state, it **must** ensure that no threads are writing to any of the Android surfaces created with this extension before calling xrEndSession. The effect of writing frames to the Surface when the session is in states other than XR_SESSION_STATE_VISIBLE or XR_SESSION_STATE_FOCUSED is undefined.

xrCreateSwapchainAndroidSurfaceKHR **must** return the same set of error codes as xrCreateSwapchain under the same circumstances, plus XR_ERROR_FUNCTION_UNSUPPORTED in case the function is not supported.

Valid Usage of XrSwapchainCreateInfo members

- The XrSwapchainCreateInfo::format, XrSwapchainCreateInfo::sampleCount, XrSwapchainCreateInfo::faceCount, XrSwapchainCreateInfo::arraySize and XrSwapchainCreateInfo::mipCount members of the structure passed as the info parameter **must** be zero.

Valid Usage (Implicit)

- The XR_KHR_android_surface_swapchain extension **must** be enabled prior to calling xrCreateSwapchainAndroidSurfaceKHR
- session **must** be a valid XrSession handle
- info **must** be a pointer to a valid XrSwapchainCreateInfo structure
- swapchain **must** be a pointer to an XrSwapchain handle
- surface **must** be a pointer to a jobject value

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_LIMIT_REACHED`
- `XR_ERROR_OUT_OF_MEMORY`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_VALIDATION_FAILURE`

Issues

Version History

- Revision 1, 2017-01-17 (Johannes van Waveren)
  - Initial draft
- Revision 2, 2017-10-30 (Kaye Mason)
  - Changed images to swapchains, used snippet includes. Added issue for Surfaces.
- Revision 3, 2018-05-16 (Krzysztof Kosiński)
  - Refactored to use Surface instead of SurfaceTexture.
- Revision 4, 2019-01-24 (Martin Renschler)
  - Refined the specification of the extension

12.3. XR_KHR_android_thread_settings

Name String

`XR_KHR_android_thread_settings`

Extension Type

Instance extension

Registered Extension Number

4

Revision

6

Ratification Status

Ratified

Extension and Version Dependencies

OpenXR 1.0

Last Modified Date

2023-12-04

IP Status

No known IP claims.

Contributors

Cass Everitt, Oculus
Johannes van Waveren, Oculus
Martin Renschler, Qualcomm
Krzysztof Kosiński, Google
Xiang Wei, Meta

Overview

For XR to be comfortable, it is important for applications to deliver frames quickly and consistently. In order to make sure the important application threads get their full share of time, these threads must be identified to the system, which will adjust their scheduling priority accordingly.

New Object Types

New Flag Types

New Enum Constants

XrResult enumeration is extended with:

- `XR_ERROR_ANDROID_THREAD_SETTINGS_ID_INVALID_KHR`
- `XR_ERROR_ANDROID_THREAD_SETTINGS_FAILURE_KHR`

New Enums

The possible thread types are specified by the XrAndroidThreadTypeKHR enumeration:

```
// Provided by XR_KHR_android_thread_settings
typedef enum XrAndroidThreadTypeKHR {
    XR_ANDROID_THREAD_TYPE_APPLICATION_MAIN_KHR = 1,
    XR_ANDROID_THREAD_TYPE_APPLICATION_WORKER_KHR = 2,
    XR_ANDROID_THREAD_TYPE_RENDERER_MAIN_KHR = 3,
    XR_ANDROID_THREAD_TYPE_RENDERER_WORKER_KHR = 4,
    XR_ANDROID_THREAD_TYPE_MAX_ENUM_KHR = 0x7FFFFFFF
} XrAndroidThreadTypeKHR;
```

Enumerants

- XR_ANDROID_THREAD_TYPE_APPLICATION_MAIN_KHR
  hints the XR runtime that the thread is doing time critical CPU tasks
- XR_ANDROID_THREAD_TYPE_APPLICATION_WORKER_KHR
  hints the XR runtime that the thread is doing background CPU tasks
- XR_ANDROID_THREAD_TYPE_RENDERER_MAIN_KHR
  hints the XR runtime that the thread is doing time critical graphics device tasks
- XR_ANDROID_THREAD_TYPE_RENDERER_WORKER_KHR
  hints the XR runtime that the thread is doing background graphics device tasks

New Structures

New Functions

To declare a thread to be of a certain XrAndroidThreadTypeKHR type call:

```
// Provided by XR_KHR_android_thread_settings
XrResult xrSetAndroidApplicationThreadKHR(
    XrSession                                   session,
    XrAndroidThreadTypeKHR                      threadType,
    uint32_t                                    threadId);
```

Parameter Descriptions

- `session` is a valid XrSession handle.
- `threadType` is a classification of the declared thread allowing the XR runtime to apply the relevant priority and attributes. If such settings fail, the runtime **must** return `XR_ERROR_ANDROID_THREAD_SETTINGS_FAILURE_KHR`.
- `threadId` is the kernel thread ID of the declared thread, as returned by `gettid()` or `android.os.process.myTid()`. If the thread ID is invalid, the runtime **must** return `XR_ERROR_ANDROID_THREAD_SETTINGS_ID_INVALID_KHR`.

xrSetAndroidApplicationThreadKHR allows to declare an XR-critical thread and to classify it.

Valid Usage (Implicit)

- The `XR_KHR_android_thread_settings` extension **must** be enabled prior to calling xrSetAndroidApplicationThreadKHR
- `session` **must** be a valid XrSession handle
- `threadType` **must** be a valid XrAndroidThreadTypeKHR value

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_ANDROID_THREAD_SETTINGS_FAILURE_KHR`
- `XR_ERROR_ANDROID_THREAD_SETTINGS_ID_INVALID_KHR`
- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_VALIDATION_FAILURE`

Version History

- Revision 1, 2017-01-17 (Johannes van Waveren)
  - Initial draft.
- Revision 2, 2017-10-31 (Armelle Laine)
  - Move the performance settings to EXT extension.
- Revision 3, 2018-12-20 (Paul Pedriana)
  - Revised the error code naming to use KHR and renamed xrSetApplicationThreadKHR → xrSetAndroidApplicationThreadKHR.
- Revision 4, 2019-01-24 (Martin Renschler)
  - Added enum specification, reformatting
- Revision 5, 2019-07-17 (Krzysztof Kosiński)
  - Clarify the type of thread identifier used by the extension.
- Revision 6, 2023-12-04 (Xiang Wei)
  - Revise/fix the hints of enum specification

12.4. XR_KHR_binding_modification

Name String

`XR_KHR_binding_modification`

Extension Type

Instance extension

Registered Extension Number

121

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

OpenXR 1.0

Last Modified Date

2020-07-29

IP Status

No known IP claims.

Contributors

Joe Ludwig, Valve

Contacts

Joe Ludwig, Valve

Overview

This extension adds an optional structure that can be included on the XrInteractionProfileSuggestedBinding::`next` chain passed to xrSuggestInteractionProfileBindings to specify additional information to modify default binding behavior.

This extension does not define any actual modification structs, but includes the list of modifications and the XrBindingModificationBaseHeaderKHR structure to allow other extensions to provide specific modifications.

New Object Types

New Flag Types

New Enum Constants

XrStructureType enumeration is extended with:

- `XR_TYPE_BINDING_MODIFICATIONS_KHR`

New Enums

New Structures

The XrBindingModificationsKHR structure is defined as:

```
// Provided by XR_KHR_binding_modification
typedef struct XrBindingModificationsKHR {
    XrStructureType                                 type;
    const void*                                     next;
    uint32_t                                        bindingModificationCount;
    const XrBindingModificationBaseHeaderKHR* const*    bindingModifications;
} XrBindingModificationsKHR;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `bindingModificationCount` is the number of binding modifications in the array pointed to by `bindingModifications`.
- `bindingModifications` is a pointer to an array of pointers to binding modification structures based on XrBindingModificationBaseHeaderKHR, that define all of the application's suggested binding modifications for the specified interaction profile.

Valid Usage (Implicit)

- The `XR_KHR_binding_modification` extension **must** be enabled prior to using XrBindingModificationsKHR
- `type` **must** be `XR_TYPE_BINDING_MODIFICATIONS_KHR`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- If `bindingModificationCount` is not `0`, `bindingModifications` **must** be a pointer to an array of `bindingModificationCount` valid XrBindingModificationBaseHeaderKHR-based structures. See also: XrInteractionProfileAnalogThresholdVALVE, XrInteractionProfileDpadBindingEXT

The XrBindingModificationBaseHeaderKHR structure is defined as:

```
// Provided by XR_KHR_binding_modification
typedef struct XrBindingModificationBaseHeaderKHR {
    XrStructureType    type;
    const void*        next;
} XrBindingModificationBaseHeaderKHR;
```

Member Descriptions

- `type` is the XrStructureType of this structure. This base structure itself has no associated XrStructureType value.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or in this extension.

The XrBindingModificationBaseHeaderKHR is a base structure is overridden by `XrBindingModification*` child structures.

Valid Usage (Implicit)

- The `XR_KHR_binding_modification` extension **must** be enabled prior to using XrBindingModificationBaseHeaderKHR
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

New Functions

Issues

Version History

- Revision 1, 2020-08-06 (Joe Ludwig)
  - Initial draft.

12.5. XR_KHR_composition_layer_color_scale_bias

Name String

`XR_KHR_composition_layer_color_scale_bias`

Extension Type

Instance extension

Registered Extension Number

35

Revision

5

Ratification Status

Ratified

Extension and Version Dependencies

OpenXR 1.0

Last Modified Date

2019-01-28

IP Status

No known IP claims.

Contributors

Paul Pedriana, Oculus

Cass Everitt, Oculus
Martin Renschler, Qualcomm

Overview

Color scale and bias are applied to a layer color during composition, after its conversion to premultiplied alpha representation.

If specified, `colorScale` and `colorBias` **must** be used to alter the LayerColor as follows:

- colorScale = max(vec4(0, 0, 0, 0), colorScale)
- LayerColor.RGB = LayerColor.A > 0 ? LayerColor.RGB / LayerColor.A : vec3(0, 0, 0)
- LayerColor = LayerColor * colorScale + colorBias
- LayerColor.RGB *= LayerColor.A

This extension specifies the XrCompositionLayerColorScaleBiasKHR structure, which, if present in the XrCompositionLayerBaseHeader::`next` chain, **must** be applied to the composition layer.

This extension does not define a new composition layer type, but rather it defines a transform that may be applied to the color derived from existing composition layer types.

New Object Types

New Flag Types

New Enum Constants

XrStructureType enumeration is extended with:

- `XR_TYPE_COMPOSITION_LAYER_COLOR_SCALE_BIAS_KHR`

New Enums

New Structures

The XrCompositionLayerColorScaleBiasKHR structure is defined as:

```
// Provided by XR_KHR_composition_layer_color_scale_bias
typedef struct XrCompositionLayerColorScaleBiasKHR {
    XrStructureType    type;
    const void*        next;
    XrColor4f          colorScale;
    XrColor4f          colorBias;
} XrCompositionLayerColorScaleBiasKHR;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `colorScale` is an XrColor4f which will modulate the color sourced from the images.
- `colorBias` is an XrColor4f which will offset the color sourced from the images.

XrCompositionLayerColorScaleBiasKHR contains the information needed to scale and bias the color of layer textures.

The XrCompositionLayerColorScaleBiasKHR structure **can** be applied by applications to composition layers by adding an instance of the struct to the XrCompositionLayerBaseHeader::`next` list.

Valid Usage (Implicit)

- The `XR_KHR_composition_layer_color_scale_bias` extension **must** be enabled prior to using XrCompositionLayerColorScaleBiasKHR
- `type` **must** be `XR_TYPE_COMPOSITION_LAYER_COLOR_SCALE_BIAS_KHR`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

New Functions

Issues

Version History

- Revision 1, 2017-09-13 (Paul Pedriana)
  - Initial implementation.
- Revision 2, 2019-01-24 (Martin Renschler)
  - Formatting, spec language changes
- Revision 3, 2019-01-28 (Paul Pedriana)
  - Revised math to remove premultiplied alpha before applying color scale and offset, then restoring.
- Revision 4, 2019-07-17 (Cass Everitt)
  - Non-substantive updates to the spec language and equations.
- Revision 5, 2020-05-20 (Cass Everitt)
  - Changed extension name, simplified language.

12.6. XR_KHR_composition_layer_cube

Name String

`XR_KHR_composition_layer_cube`

Extension Type

Instance extension

Registered Extension Number

7

Revision

8

Ratification Status

Ratified

Extension and Version Dependencies

OpenXR 1.0

Last Modified Date

2019-01-24

IP Status

No known IP claims.

Contributors

Johannes van Waveren, Oculus
Cass Everitt, Oculus
Paul Pedriana, Oculus
Gloria Kennickell, Oculus
Sam Martin, ARM
Kaye Mason, Google, Inc.
Martin Renschler, Qualcomm

Contacts

Cass Everitt, Oculus
Paul Pedriana, Oculus

Overview

This extension adds an additional layer type that enables direct sampling from cubemaps.

The cube layer is the natural layer type for hardware accelerated environment maps. Without updating the image source, the user can look all around, and the compositor can display what they are

looking at without intervention from the application.

New Object Types

New Flag Types

New Enum Constants

XrStructureType enumeration is extended with:

- `XR_TYPE_COMPOSITION_LAYER_CUBE_KHR`

New Enums

New Structures

The XrCompositionLayerCubeKHR structure is defined as:

```
// Provided by XR_KHR_composition_layer_cube
typedef struct XrCompositionLayerCubeKHR {
    XrStructureType            type;
    const void*                next;
    XrCompositionLayerFlags    layerFlags;
    XrSpace                    space;
    XrEyeVisibility            eyeVisibility;
    XrSwapchain                swapchain;
    uint32_t                   imageArrayIndex;
    XrQuaternionf              orientation;
} XrCompositionLayerCubeKHR;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `layerFlags` is any flags to apply to this layer.
- `space` is the XrSpace in which the `orientation` of the cube layer is evaluated over time.
- `eyeVisibility` is the eye represented by this layer.
- `swapchain` is the swapchain, which **must** have been created with a XrSwapchainCreateInfo ::`faceCount` of 6.
- `imageArrayIndex` is the image array index, with 0 meaning the first or only array element.
- `orientation` is the orientation of the environment map in the `space`.

XrCompositionLayerCubeKHR contains the information needed to render a cube map when calling xrEndFrame. XrCompositionLayerCubeKHR is an alias type for the base struct XrCompositionLayerBaseHeader used in XrFrameEndInfo.

Valid Usage (Implicit)

- The `XR_KHR_composition_layer_cube` extension **must** be enabled prior to using XrCompositionLayerCubeKHR
- `type` **must** be `XR_TYPE_COMPOSITION_LAYER_CUBE_KHR`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `layerFlags` **must** be `0` or a valid combination of XrCompositionLayerFlagBits values
- `space` **must** be a valid XrSpace handle
- `eyeVisibility` **must** be a valid XrEyeVisibility value
- `swapchain` **must** be a valid XrSwapchain handle
- Both of `space` and `swapchain` **must** have been created, allocated, or retrieved from the same XrSession

New Functions

Issues

Version History

- Revision 0, 2017-02-01 (Johannes van Waveren)
  - Initial draft.
- Revision 1, 2017-05-19 (Sam Martin)
  - Initial draft, moving the 3 layer types to an extension.
- Revision 2, 2017-08-30 (Paul Pedriana)
  - Updated the specification.
- Revision 3, 2017-10-12 (Cass Everitt)
  - Updated to reflect per-eye structs and the change to swapchains
- Revision 4, 2017-10-18 (Kaye Mason)
  - Update to flatten structs to remove per-eye arrays.
- Revision 5, 2017-12-05 (Paul Pedriana)
  - Updated to break out the cylinder and equirect features into separate extensions.
- Revision 6, 2017-12-07 (Paul Pedriana)
  - Updated to use transform components instead of transform matrices.

- Revision 7, 2017-12-07 (Paul Pedriana)
  - Updated to convert XrPosef to XrQuaternionf (there's no position component).
- Revision 8, 2019-01-24 (Martin Renschler)
  - Updated struct to use XrSwapchainSubImage, reformat and spec language changes, eye parameter description update

12.7. XR_KHR_composition_layer_cylinder

Name String

`XR_KHR_composition_layer_cylinder`

Extension Type

Instance extension

Registered Extension Number

18

Revision

4

Ratification Status

Ratified

Extension and Version Dependencies

OpenXR 1.0

Last Modified Date

2019-01-24

IP Status

No known IP claims.

Contributors

James Hughes, Oculus
Paul Pedriana, Oculus
Martin Renschler, Qualcomm

Contacts

Paul Pedriana, Oculus
Cass Everitt, Oculus

Overview

This extension adds an additional layer type where the XR runtime **must** map a texture stemming

from a swapchain onto the inside of a cylinder section. It can be imagined much the same way a curved television display looks to a viewer. This is not a projection type of layer but rather an object-in-world type of layer, similar to XrCompositionLayerQuad. Only the interior of the cylinder surface **must** be visible; the exterior of the cylinder is not visible and **must** not be drawn by the runtime.

The cylinder characteristics are specified by the following parameters:

```
XrPosef          pose;
float            radius;
float            centralAngle;
float            aspectRatio;
```

These can be understood via the following diagram, which is a top-down view of a horizontally oriented cylinder. The aspect ratio drives how tall the cylinder will appear based on the other parameters. Typically the aspectRatio would be set to be the aspect ratio of the texture being used, so that it looks the same within the cylinder as it does in 2D.

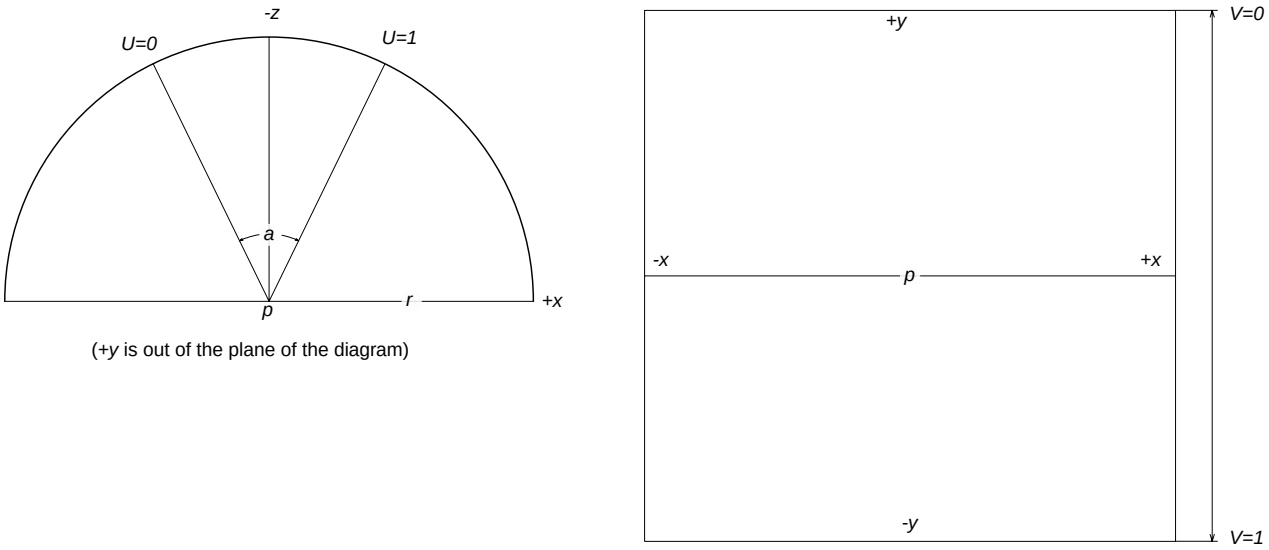


Figure 6. Cylinder Layer Parameters

- r — Radius
- a — Central angle in (0, 2π)
- p — Origin of pose transform
- U/V — UV coordinates

New Object Types

New Flag Types

New Enum Constants

XrStructureType enumeration is extended with:

- XR_TYPE_COMPOSITION_LAYER_CYLINDER_KHR

New Enums

New Structures

The XrCompositionLayerCylinderKHR structure is defined as:

```
// Provided by XR_KHR_composition_layer_cylinder
typedef struct XrCompositionLayerCylinderKHR {
    XrStructureType            type;
    const void*                next;
    XrCompositionLayerFlags    layerFlags;
    XrSpace                    space;
    XrEyeVisibility            eyeVisibility;
    XrSwapchainSubImage        subImage;
    XrPosef                    pose;
    float                      radius;
    float                      centralAngle;
    float                      aspectRatio;
} XrCompositionLayerCylinderKHR;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `layerFlags` specifies options for the layer.
- `space` is the XrSpace in which the `pose` of the cylinder layer is evaluated over time.
- `eyeVisibility` is the eye represented by this layer.
- `subImage` identifies the image XrSwapchainSubImage to use. The swapchain **must** have been created with a XrSwapchainCreateInfo::`faceCount` of 1.
- `pose` is an XrPosef defining the position and orientation of the center point of the view of the cylinder within the reference frame of the `space`.
- `radius` is the non-negative radius of the cylinder. Values of zero or floating point positive infinity are treated as an infinite cylinder.
- `centralAngle` is the angle of the visible section of the cylinder, based at 0 radians, in the range of [0, 2π). It grows symmetrically around the 0 radian angle.
- `aspectRatio` is the ratio of the visible cylinder section width / height. The height of the cylinder is given by: (cylinder radius × cylinder angle) / aspectRatio.

XrCompositionLayerCylinderKHR contains the information needed to render a texture onto a cylinder when calling xrEndFrame. XrCompositionLayerCylinderKHR is an alias type for the base struct XrCompositionLayerBaseHeader used in XrFrameEndInfo.

Valid Usage (Implicit)

- The `XR_KHR_composition_layer_cylinder` extension **must** be enabled prior to using XrCompositionLayerCylinderKHR
- `type` **must** be `XR_TYPE_COMPOSITION_LAYER_CYLINDER_KHR`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `layerFlags` **must** be `0` or a valid combination of XrCompositionLayerFlagBits values
- `space` **must** be a valid XrSpace handle
- `eyeVisibility` **must** be a valid XrEyeVisibility value
- `subImage` **must** be a valid XrSwapchainSubImage structure

New Functions

Issues

Version History

- Revision 1, 2017-05-19 (Paul Pedriana)
  - Initial version. This was originally part of a single extension which supported multiple such extension layer types.
- Revision 2, 2017-12-07 (Paul Pedriana)
  - Updated to use transform components instead of transform matrices.
- Revision 3, 2018-03-05 (Paul Pedriana)
  - Added improved documentation and brought the documentation in line with the existing core spec.
- Revision 4, 2019-01-24 (Martin Renschler)
  - Reformatted, spec language changes, eye parameter description update

12.8. XR_KHR_composition_layer_depth

Name String

`XR_KHR_composition_layer_depth`

Extension Type

Instance extension

Registered Extension Number

11

Revision

6

Ratification Status

Ratified

Extension and Version Dependencies

OpenXR 1.0

Last Modified Date

2019-01-24

IP Status

No known IP claims.

Contributors

Paul Pedriana, Oculus
Bryce Hutchings, Microsoft

Andreas Loeve Selvik, Arm
Martin Renschler, Qualcomm

Overview

This extension defines an extra layer type which allows applications to submit depth images along with color images in projection layers, i.e. XrCompositionLayerProjection.

The XR runtime **may** use this information to perform more accurate reprojections taking depth into account. Use of this extension does not affect the order of layer composition as described in Compositing.

New Object Types

New Flag Types

New Enum Constants

XrStructureType enumeration is extended with:

- `XR_TYPE_COMPOSITION_LAYER_DEPTH_INFO_KHR`

New Enums

New Structures

When submitting depth images along with projection layers, add the XrCompositionLayerDepthInfoKHR to the `next` chain for all XrCompositionLayerProjectionView structures in the given layer.

The XrCompositionLayerDepthInfoKHR structure is defined as:

```
// Provided by XR_KHR_composition_layer_depth
typedef struct XrCompositionLayerDepthInfoKHR {
    XrStructureType        type;
    const void*            next;
    XrSwapchainSubImage    subImage;
    float                  minDepth;
    float                  maxDepth;
    float                  nearZ;
    float                  farZ;
} XrCompositionLayerDepthInfoKHR;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `subImage` identifies the depth image XrSwapchainSubImage to be associated with the color swapchain. The swapchain **must** have been created with a XrSwapchainCreateInfo ::`faceCount` of 1.
- `minDepth` and `maxDepth` are the window space depths that correspond to the near and far frustum planes, respectively. `minDepth` must be less than `maxDepth`. `minDepth` and `maxDepth` must be in the range [0, 1].
- `nearZ` and `farZ` are the positive distances in meters to the near and far frustum planes, respectively. `nearZ` and `farZ` **must** not be equal. `nearZ` and `farZ` **must** be in the range (0, +infinity].



Note

The window space depth values `minDepth` and `maxDepth` are akin to the parameters of `glDepthRange` that specify the mapping from normalized device coordinates into window space.



Note

A reversed mapping of depth, such that points closer to the view have a window space depth that is greater than points further away can be achieved by making nearZ > farZ.

XrCompositionLayerDepthInfoKHR contains the information needed to associate depth with the color information in a projection layer. When submitting depth images along with projection layers, add the XrCompositionLayerDepthInfoKHR to the `next` chain for all XrCompositionLayerProjectionView structures in the given layer.

The homogeneous transform from view space z to window space depth is given by the following matrix, where a = minDepth, b = maxDepth, n = nearZ, and f = farZ.

$$\mathbf{T} = \begin{bmatrix} b-a & a \\ 0 & 1 \end{bmatrix} \begin{bmatrix} -\frac{f}{f-n} & -\frac{fn}{f-n} \\ -1 & 0 \end{bmatrix} = \begin{bmatrix} -\frac{bf-an}{f-n} & -\frac{fn(b-a)}{f-n} \\ -1 & 0 \end{bmatrix}$$

$$\mathbf{p}_w = \mathbf{T}\mathbf{p}_v$$

$$\mathbf{p}_w = \begin{bmatrix} z_w & w_w \end{bmatrix}^t, \text{ homogeneous window space depth}$$

$$\mathbf{p}_v = \begin{bmatrix} z_v & w_v \end{bmatrix}^t, \text{ homogeneous view space depth}$$

Figure 7. Homogeneous transform from view space to window space depth

Homogeneous values are constructed from real values by appending a w component with value 1.0.

General homogeneous values are projected back to real space by dividing by the w component.

Valid Usage (Implicit)

- The `XR_KHR_composition_layer_depth` extension **must** be enabled prior to using XrCompositionLayerDepthInfoKHR
- `type` **must** be `XR_TYPE_COMPOSITION_LAYER_DEPTH_INFO_KHR`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `subImage` **must** be a valid XrSwapchainSubImage structure

New Functions

Issues

1. Should the range of `minDepth` and `maxDepth` be constrained to [0,1]?

   RESOLVED: Yes.

   There is no compelling mathematical reason for this constraint, however, it does not impose any hardship currently, and the constraint could be relaxed in a future version of the extension if needed.

2. Should we require `minDepth` be less than `maxDepth`?

   RESOLVED: Yes.

   There is no compelling mathematical reason for this constraint, however, it does not impose any

hardship currently, and the constraint could be relaxed in a future version of the extension if needed. Reverse z mappings can be achieved by making `nearZ` > `farZ`.

3. Does this extension support view space depth images?

   RESOLVED: No.
   The formulation of the transform between view and window depths implies projected depth. A different extension would be needed to support a different interpretation of depth.

4. Is there any constraint on the resolution of the depth subimage?

   RESOLVED: No.

   The resolution of the depth image need not match that of the corresponding color image.

Version History

- Revision 1, 2017-08-18 (Paul Pedriana)
  - Initial proposal.
- Revision 2, 2017-10-30 (Kaye Mason)
  - Migration from Images to Swapchains.
- Revision 3, 2018-07-20 (Bryce Hutchings)
  - Support for swapchain texture arrays
- Revision 4, 2018-12-17 (Andreas Loeve Selvik)
  - depthImageRect in pixels instead of UVs
- Revision 5, 2019-01-24 (Martin Renschler)
  - changed depthSwapchain/depthImageRect/depthImageArrayIndex to XrSwapchainSubImage
  - reformat and spec language changes
  - removed vendor specific terminology
- Revision 6, 2022-02-16 (Cass Everitt)
  - Provide homogeneous transform as function of provided parameters

12.9. XR_KHR_composition_layer_equirect

Name String

`XR_KHR_composition_layer_equirect`

Extension Type

Instance extension

Registered Extension Number

19

Revision

3

Ratification Status

Ratified

Extension and Version Dependencies

OpenXR 1.0

Last Modified Date

2019-01-24

IP Status

No known IP claims.

Contributors

Johannes van Waveren, Oculus
Cass Everitt, Oculus
Paul Pedriana, Oculus
Gloria Kennickell, Oculus
Martin Renschler, Qualcomm

Contacts

Cass Everitt, Oculus
Paul Pedriana, Oculus

Overview

This extension adds an additional layer type where the XR runtime must map an equirectangular coded image stemming from a swapchain onto the inside of a sphere.

The equirect layer type provides most of the same benefits as a cubemap, but from an equirect 2D image source. This image source is appealing mostly because equirect environment maps are very common, and the highest quality you can get from them is by sampling them directly in the compositor.

This is not a projection type of layer but rather an object-in-world type of layer, similar to XrCompositionLayerQuad. Only the interior of the sphere surface **must** be visible; the exterior of the sphere is not visible and **must** not be drawn by the runtime.

New Object Types

New Flag Types

New Enum Constants

XrStructureType enumeration is extended with:

- XR_TYPE_COMPOSITION_LAYER_EQUIRECT_KHR

New Enums

New Structures

The XrCompositionLayerEquirectKHR structure is defined as:

```
// Provided by XR_KHR_composition_layer_equirect
typedef struct XrCompositionLayerEquirectKHR {
    XrStructureType            type;
    const void*                next;
    XrCompositionLayerFlags    layerFlags;
    XrSpace                    space;
    XrEyeVisibility            eyeVisibility;
    XrSwapchainSubImage        subImage;
    XrPosef                    pose;
    float                      radius;
    XrVector2f                 scale;
    XrVector2f                 bias;
} XrCompositionLayerEquirectKHR;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `layerFlags` specifies options for the layer.
- `space` is the XrSpace in which the `pose` of the equirect layer is evaluated over time.
- `eyeVisibility` is the eye represented by this layer.
- `subImage` identifies the image XrSwapchainSubImage to use. The swapchain **must** have been created with a XrSwapchainCreateInfo::`faceCount` of 1.
- `pose` is an XrPosef defining the position and orientation of the center point of the sphere onto which the equirect image data is mapped, relative to the reference frame of the `space`.
- `radius` is the non-negative radius of the sphere onto which the equirect image data is mapped. Values of zero or floating point positive infinity are treated as an infinite sphere.
- `scale` is an XrVector2f indicating a scale of the texture coordinates after the mapping to 2D.
- `bias` is an XrVector2f indicating a bias of the texture coordinates after the mapping to 2D.

XrCompositionLayerEquirectKHR contains the information needed to render an equirectangular image onto a sphere when calling xrEndFrame. XrCompositionLayerEquirectKHR is an alias type for the base struct XrCompositionLayerBaseHeader used in XrFrameEndInfo.

Valid Usage (Implicit)

- The `XR_KHR_composition_layer_equirect` extension **must** be enabled prior to using XrCompositionLayerEquirectKHR
- `type` **must** be `XR_TYPE_COMPOSITION_LAYER_EQUIRECT_KHR`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `layerFlags` **must** be `0` or a valid combination of XrCompositionLayerFlagBits values
- `space` **must** be a valid XrSpace handle
- `eyeVisibility` **must** be a valid XrEyeVisibility value
- `subImage` **must** be a valid XrSwapchainSubImage structure

New Functions

Issues

Version History

- Revision 1, 2017-05-19 (Paul Pedriana)
    - Initial version. This was originally part of a single extension which supported multiple such extension layer types.
- Revision 2, 2017-12-07 (Paul Pedriana)
    - Updated to use transform components instead of transform matrices.
- Revision 3, 2019-01-24 (Martin Renschler)
    - Reformatted, spec language changes, eye parameter description update

12.10. XR_KHR_composition_layer_equirect2

Name String

`XR_KHR_composition_layer_equirect2`

Extension Type

Instance extension

Registered Extension Number

92

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

OpenXR 1.0

Last Modified Date

2019-01-24

IP Status

No known IP claims.

Contributors

Johannes van Waveren, Oculus
Cass Everitt, Oculus
Paul Pedriana, Oculus
Gloria Kennickell, Oculus
Martin Renschler, Qualcomm

Contacts

Cass Everitt, Oculus

Overview

This extension adds an additional layer type where the XR runtime must map an equirectangular coded image stemming from a swapchain onto the inside of a sphere.

The equirect layer type provides most of the same benefits as a cubemap, but from an equirect 2D image source. This image source is appealing mostly because equirect environment maps are very common, and the highest quality you can get from them is by sampling them directly in the compositor.

This is not a projection type of layer but rather an object-in-world type of layer, similar to XrCompositionLayerQuad. Only the interior of the sphere surface **must** be visible; the exterior of the sphere is not visible and **must** not be drawn by the runtime.

This extension uses a different parameterization more in keeping with the formulation of KHR_composition_layer_cylinder but is functionally equivalent to KHR_composition_layer_equirect.

New Object Types

New Flag Types

New Enum Constants

XrStructureType enumeration is extended with:

- `XR_TYPE_COMPOSITION_LAYER_EQUIRECT2_KHR`

New Enums

New Structures

The XrCompositionLayerEquirect2KHR structure is defined as:

```
// Provided by XR_KHR_composition_layer_equirect2
typedef struct XrCompositionLayerEquirect2KHR {
    XrStructureType            type;
    const void*                next;
    XrCompositionLayerFlags    layerFlags;
    XrSpace                    space;
    XrEyeVisibility            eyeVisibility;
    XrSwapchainSubImage        subImage;
    XrPosef                    pose;
    float                      radius;
    float                      centralHorizontalAngle;
    float                      upperVerticalAngle;
    float                      lowerVerticalAngle;
} XrCompositionLayerEquirect2KHR;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `layerFlags` specifies options for the layer.
- `space` is the XrSpace in which the `pose` of the equirect layer is evaluated over time.
- `eyeVisibility` is the eye represented by this layer.
- `subImage` identifies the image XrSwapchainSubImage to use. The swapchain **must** have been created with a XrSwapchainCreateInfo::`faceCount` of 1.
- `pose` is an XrPosef defining the position and orientation of the center point of the sphere onto which the equirect image data is mapped, relative to the reference frame of the `space`.
- `radius` is the non-negative radius of the sphere onto which the equirect image data is mapped. Values of zero or floating point positive infinity are treated as an infinite sphere.
- `centralHorizontalAngle` defines the visible horizontal angle of the sphere, based at 0 radians, in the range of [0, 2π]. It grows symmetrically around the 0 radian angle.
- `upperVerticalAngle` defines the upper vertical angle of the visible portion of the sphere, in the range of [-π/2, π/2].
- `lowerVerticalAngle` defines the lower vertical angle of the visible portion of the sphere, in the range of [-π/2, π/2].

XrCompositionLayerEquirect2KHR contains the information needed to render an equirectangular image onto a sphere when calling xrEndFrame. XrCompositionLayerEquirect2KHR is an alias type for the base struct XrCompositionLayerBaseHeader used in XrFrameEndInfo.

Valid Usage (Implicit)

- The `XR_KHR_composition_layer_equirect2` extension **must** be enabled prior to using XrCompositionLayerEquirect2KHR
- `type` **must** be `XR_TYPE_COMPOSITION_LAYER_EQUIRECT2_KHR`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `layerFlags` **must** be `0` or a valid combination of XrCompositionLayerFlagBits values
- `space` **must** be a valid XrSpace handle
- `eyeVisibility` **must** be a valid XrEyeVisibility value
- `subImage` **must** be a valid XrSwapchainSubImage structure

New Functions

Issues

Version History

- Revision 1, 2020-05-08 (Cass Everitt)
  - Initial version.
  - Kept contributors from the original equirect extension.

12.11. XR_KHR_convert_timespec_time

Name String

`XR_KHR_convert_timespec_time`

Extension Type

Instance extension

Registered Extension Number

37

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

OpenXR 1.0

Last Modified Date

2019-01-24

IP Status

No known IP claims.

Contributors

Paul Pedriana, Oculus

Overview

This extension provides two functions for converting between timespec monotonic time and `XrTime`. The xrConvertTimespecTimeToTimeKHR function converts from timespec time to `XrTime`, while the xrConvertTimeToTimespecTimeKHR function converts `XrTime` to timespec monotonic time. The primary use case for this functionality is to be able to synchronize events between the local system and the OpenXR system.

New Object Types

New Flag Types

New Enum Constants

New Enums

New Structures

New Functions

To convert from timespec monotonic time to `XrTime`, call:

```
// Provided by XR_KHR_convert_timespec_time
XrResult xrConvertTimespecTimeToTimeKHR(
    XrInstance                                  instance,
    const struct timespec*                      timespecTime,
    XrTime*                                     time);
```

Parameter Descriptions

- `instance` is an XrInstance handle previously created with xrCreateInstance.
- `timespecTime` is a `timespec` obtained from `clock_gettime` with `CLOCK_MONOTONIC`.
- `time` is the resulting `XrTime` that is equivalent to the `timespecTime`.

The xrConvertTimespecTimeToTimeKHR function converts a time obtained by the `clock_gettime`

function to the equivalent XrTime.

If the output time cannot represent the input timespecTime, the runtime **must** return XR_ERROR_TIME_INVALID.

Valid Usage (Implicit)

- The XR_KHR_convert_timespec_time extension **must** be enabled prior to calling xrConvertTimespecTimeToTimeKHR
- instance **must** be a valid XrInstance handle
- timespecTime **must** be a pointer to a valid timespec value
- time **must** be a pointer to an XrTime value

Return Codes

Success

- XR_SUCCESS

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_TIME_INVALID
- XR_ERROR_VALIDATION_FAILURE

To convert from XrTime to timespec monotonic time, call:

```
// Provided by XR_KHR_convert_timespec_time
XrResult xrConvertTimeToTimespecTimeKHR(
    XrInstance                                  instance,
    XrTime                                      time,
    struct timespec*                            timespecTime);
```

Parameter Descriptions

- `instance` is an XrInstance handle previously created with xrCreateInstance.
- `time` is an `XrTime`.
- `timespecTime` is the resulting timespec time that is equivalent to a `timespec` obtained from `clock_gettime` with `CLOCK_MONOTONIC`.

The xrConvertTimeToTimespecTimeKHR function converts an `XrTime` to time as if generated by `clock_gettime`.

If the output `timespecTime` cannot represent the input `time`, the runtime **must** return `XR_ERROR_TIME_INVALID`.

Valid Usage (Implicit)

- The `XR_KHR_convert_timespec_time` extension **must** be enabled prior to calling xrConvertTimeToTimespecTimeKHR
- `instance` **must** be a valid XrInstance handle
- `timespecTime` **must** be a pointer to a `timespec` value

Return Codes

Success

- `XR_SUCCESS`

Failure

- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_TIME_INVALID`
- `XR_ERROR_VALIDATION_FAILURE`

Issues

Version History

- Revision 1, 2019-01-24 (Paul Pedriana)

- Initial draft

12.12. XR_KHR_D3D11_enable

Name String

`XR_KHR_D3D11_enable`

Extension Type

Instance extension

Registered Extension Number

28

Revision

11

Ratification Status

Ratified

Extension and Version Dependencies

OpenXR 1.0

Last Modified Date

2025-03-21

IP Status

No known IP claims.

Contributors

Bryce Hutchings, Microsoft
Paul Pedriana, Oculus
Mark Young, LunarG
Minmin Gong, Microsoft
Aaron Leiby, Valve

12.12.1. Overview

This extension enables the use of the Direct3D 11 (D3D11) graphics API in an OpenXR application. Without this extension, an OpenXR application **may** not be able to use any D3D11 swapchain images.

This extension provides the mechanisms necessary for an application to generate a valid XrGraphicsBindingD3D11KHR structure in order to create a D3D11-based XrSession. Note that during this process the application is responsible for creating all the required D3D11 objects, including a graphics device to be used for rendering. However, the runtime provides the D3D11 textures to render into. This extension provides mechanisms for the application to interact with those textures by calling

xrEnumerateSwapchainImages and providing XrSwapchainImageD3D11KHR structures to populate.

In order to expose the structures, types, and functions of this extension, the application source code **must** define XR_USE_GRAPHICS_API_D3D11 before including the OpenXR platform header `openxr_platform.h`, in all portions of the library or application that interact with the types, values, and functions it defines.

12.12.2. Get Graphics Requirements

Some computer systems have multiple graphics devices, each of which **may** have independent external display outputs. XR systems that connect to such computer systems are typically connected to a single graphics device. Applications need to know the graphics device associated with the XR system, so that rendering takes place on the correct graphics device.

The xrGetD3D11GraphicsRequirementsKHR function is defined as:

```
// Provided by XR_KHR_D3D11_enable
XrResult xrGetD3D11GraphicsRequirementsKHR(
    XrInstance                                  instance,
    XrSystemId                                  systemId,
    XrGraphicsRequirementsD3D11KHR*             graphicsRequirements);
```

Parameter Descriptions

- `instance` is an XrInstance handle previously created with xrCreateInstance.
- `systemId` is an `XrSystemId` handle for the system which will be used to create a session.
- `graphicsRequirements` is the XrGraphicsRequirementsD3D11KHR output structure.

This call retrieves the D3D11 feature level and graphics device for an instance and system. The xrGetD3D11GraphicsRequirementsKHR function identifies to the application the graphics device (Windows LUID) to be used and the minimum feature level to use. The runtime **must** return `XR_ERROR_GRAPHICS_REQUIREMENTS_CALL_MISSING` (`XR_ERROR_VALIDATION_FAILURE` **may** be returned due to legacy behavior) on calls to xrCreateSession if xrGetD3D11GraphicsRequirementsKHR has not been called for the same `instance` and `systemId`.

The LUID and feature level that xrGetD3D11GraphicsRequirementsKHR returns **must** be used to create the `ID3D11Device` that the application passes to xrCreateSession in the XrGraphicsBindingD3D11KHR.

Valid Usage (Implicit)

- The XR_KHR_D3D11_enable extension **must** be enabled prior to calling xrGetD3D11GraphicsRequirementsKHR
- `instance` **must** be a valid XrInstance handle
- `graphicsRequirements` **must** be a pointer to an XrGraphicsRequirementsD3D11KHR structure

Return Codes

Success

- XR_SUCCESS

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SYSTEM_INVALID
- XR_ERROR_VALIDATION_FAILURE

The XrGraphicsRequirementsD3D11KHR structure is defined as:

```
// Provided by XR_KHR_D3D11_enable
typedef struct XrGraphicsRequirementsD3D11KHR {
    XrStructureType      type;
    void*                next;
    LUID                 adapterLuid;
    D3D_FEATURE_LEVEL    minFeatureLevel;
} XrGraphicsRequirementsD3D11KHR;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `adapterLuid` identifies what graphics device needs to be used.
- `minFeatureLevel` is the minimum feature level for which to initialize the D3D11 device.

XrGraphicsRequirementsD3D11KHR is populated by xrGetD3D11GraphicsRequirementsKHR with the runtime's D3D11 API feature level and adapter requirements.

Valid Usage (Implicit)

- The `XR_KHR_D3D11_enable` extension **must** be enabled prior to using XrGraphicsRequirementsD3D11KHR
- `type` **must** be `XR_TYPE_GRAPHICS_REQUIREMENTS_D3D11_KHR`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `adapterLuid` **must** be a valid `LUID` value
- `minFeatureLevel` **must** be a valid `D3D_FEATURE_LEVEL` value

12.12.3. Graphics Binding Structure

The XrGraphicsBindingD3D11KHR structure is defined as:

```
// Provided by XR_KHR_D3D11_enable
typedef struct XrGraphicsBindingD3D11KHR {
    XrStructureType    type;
    const void*        next;
    ID3D11Device*      device;
} XrGraphicsBindingD3D11KHR;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `device` is a pointer to a valid `ID3D11Device` to use.

To create a D3D11-backed XrSession, the application provides a pointer to an XrGraphicsBindingD3D11KHR structure in the XrSessionCreateInfo::next chain when calling xrCreateSession. The D3D11 device specified in XrGraphicsBindingD3D11KHR::device **must** be created in accordance with the requirements retrieved through xrGetD3D11GraphicsRequirementsKHR, otherwise xrCreateSession **must** return XR_ERROR_GRAPHICS_DEVICE_INVALID.

Creating a session using this structure triggers several requirements on the runtime regarding swapchain images. See the specification of XrSwapchainImageD3D11KHR for details.

Valid Usage (Implicit)

- The XR_KHR_D3D11_enable extension **must** be enabled prior to using XrGraphicsBindingD3D11KHR
- type **must** be XR_TYPE_GRAPHICS_BINDING_D3D11_KHR
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- device **must** be a pointer to an ID3D11Device value

12.12.4. Swapchain Images

The XrSwapchainImageD3D11KHR structure is defined as:

```
// Provided by XR_KHR_D3D11_enable
typedef struct XrSwapchainImageD3D11KHR {
    XrStructureType    type;
    void*              next;
    ID3D11Texture2D*   texture;
} XrSwapchainImageD3D11KHR;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- texture is a pointer to a valid ID3D11Texture2D to use.

If a given session was created with XrGraphicsBindingD3D11KHR, the following conditions apply.

- Calls to xrEnumerateSwapchainImages on an XrSwapchain in that session **must** return an array of XrSwapchainImageD3D11KHR structures.
- Whenever an OpenXR function accepts an XrSwapchainImageBaseHeader pointer as a parameter

in that session, the runtime **must** also accept a pointer to an XrSwapchainImageD3D11KHR.

The OpenXR runtime **must** interpret the top-left corner of the swapchain image as the coordinate origin unless specified otherwise by extension functionality.

The OpenXR runtime **must** return a texture created in accordance with D3D11 Swapchain Flag Bits.

Valid Usage (Implicit)

- The `XR_KHR_D3D11_enable` extension **must** be enabled prior to using XrSwapchainImageD3D11KHR
- `type` **must** be `XR_TYPE_SWAPCHAIN_IMAGE_D3D11_KHR`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

12.12.5. D3D11 Swapchain Flag Bits

All valid XrSwapchainUsageFlags values passed in a session created using XrGraphicsBindingD3D11KHR **must** be interpreted as follows by the runtime, so that the returned swapchain images used by the application **may** be used as if they were created with the corresponding `D3D11_BIND_FLAG` flags. The runtime **may** set additional bind flags but **must** not restrict usage.

XrSwapchainUsageFlagBits	Corresponding D3D11 bind flag bits
`XR_SWAPCHAIN_USAGE_COLOR_ATTACHMENT_BIT`	`D3D11_BIND_RENDER_TARGET`
`XR_SWAPCHAIN_USAGE_DEPTH_STENCIL_ATTACHMENT_BIT`	`D3D11_BIND_DEPTH_STENCIL`
`XR_SWAPCHAIN_USAGE_UNORDERED_ACCESS_BIT`	`D3D11_BIND_UNORDERED_ACCESS`
`XR_SWAPCHAIN_USAGE_TRANSFER_SRC_BIT`	*ignored*
`XR_SWAPCHAIN_USAGE_TRANSFER_DST_BIT`	*ignored*
`XR_SWAPCHAIN_USAGE_SAMPLED_BIT`	`D3D11_BIND_SHADER_RESOURCE`
`XR_SWAPCHAIN_USAGE_MUTABLE_FORMAT_BIT`	*ignored*
`XR_SWAPCHAIN_USAGE_INPUT_ATTACHMENT_BIT_KHR` (Added by `XR_KHR_swapchain_usage_input_attachment_bit` and only available when that extension is enabled)	*ignored*

All D3D11 swapchain textures are created with `D3D11_USAGE_DEFAULT` usage.

12.12.6. New Commands

- xrGetD3D11GraphicsRequirementsKHR

12.12.7. New Structures

- XrGraphicsRequirementsD3D11KHR
- XrSwapchainImageD3D11KHR
- Extending XrSessionCreateInfo:
  - XrGraphicsBindingD3D11KHR

12.12.8. New Enum Constants

- XR_KHR_D3D11_ENABLE_EXTENSION_NAME
- XR_KHR_D3D11_enable_SPEC_VERSION
- Extending XrStructureType:
  - XR_TYPE_GRAPHICS_BINDING_D3D11_KHR
  - XR_TYPE_GRAPHICS_REQUIREMENTS_D3D11_KHR
  - XR_TYPE_SWAPCHAIN_IMAGE_D3D11_KHR

12.12.9. Version History

- Revision 1, 2018-05-07 (Mark Young)
  - Initial draft
- Revision 2, 2018-06-21 (Bryce Hutchings)
  - Split XR_KHR_D3D_enable into XR_KHR_D3D11_enable
  - Rename and expand xrGetD3DGraphicsDeviceKHR functionality to xrGetD3D11GraphicsRequirementsKHR
- Revision 3, 2018-11-15 (Paul Pedriana)
  - Specified the swapchain texture coordinate origin.
- Revision 4, 2018-11-16 (Minmin Gong)
  - Specified Y direction and Z range in clip space
- Revision 5, 2020-08-06 (Bryce Hutchings)
  - Added new XR_ERROR_GRAPHICS_REQUIREMENTS_CALL_MISSING error code
- Revision 8, 2021-09-09 (Bryce Hutchings)
  - Document mapping for XrSwapchainUsageFlags
- Revision 9, 2021-12-28 (Microsoft)
  - Added missing XR_ERROR_GRAPHICS_DEVICE_INVALID error condition
- Revision 10, 2025-03-07 (Rylie Pavlik, Collabora, Ltd.)
  - Re-organize, clarify, and make more uniform with other graphics binding extensions.

- Revision 11, 2025-03-21 (Aaron Leiby)
  - Removed clip space specification

12.13. XR_KHR_D3D12_enable

Name String

`XR_KHR_D3D12_enable`

Extension Type

Instance extension

Registered Extension Number

29

Revision

11

Ratification Status

Ratified

Extension and Version Dependencies

OpenXR 1.0

Last Modified Date

2025-03-21

IP Status

No known IP claims.

Contributors

Bryce Hutchings, Microsoft
Paul Pedriana, Oculus
Mark Young, LunarG
Minmin Gong, Microsoft
Dan Ginsburg, Valve
Aaron Leiby, Valve

12.13.1. Overview

This extension enables the use of the Direct3D 12 (D3D12) graphics API in an OpenXR application. Without this extension, an OpenXR application **may** not be able to use any D3D12 swapchain images.

This extension provides the mechanisms necessary for an application to generate a valid XrGraphicsBindingD3D12KHR structure in order to create a D3D12-based XrSession. Note that during

this process the application is responsible for creating all the required D3D12 objects, including a graphics device and queue to be used for rendering. However, the runtime provides the D3D12 images to render into. This extension provides mechanisms for the application to interact with those images by calling xrEnumerateSwapchainImages and providing XrSwapchainImageD3D12KHR structures to populate.

In order to expose the structures, types, and functions of this extension, the application source code **must** define XR_USE_GRAPHICS_API_D3D12 before including the OpenXR platform header `openxr_platform.h`, in all portions of the library or application that interact with the types, values, and functions it defines.

12.13.2. Get Graphics Requirements

Some computer systems have multiple graphics devices, each of which **may** have independent external display outputs. XR systems that connect to such computer systems are typically connected to a single graphics device. Applications need to know the graphics device associated with the XR system, so that rendering takes place on the correct graphics device.

The xrGetD3D12GraphicsRequirementsKHR function is defined as:

```
// Provided by XR_KHR_D3D12_enable
XrResult xrGetD3D12GraphicsRequirementsKHR(
    XrInstance                                  instance,
    XrSystemId                                  systemId,
    XrGraphicsRequirementsD3D12KHR*             graphicsRequirements);
```

Parameter Descriptions

- `instance` is an XrInstance handle previously created with xrCreateInstance.
- `systemId` is an `XrSystemId` handle for the system which will be used to create a session.
- `graphicsRequirements` is the XrGraphicsRequirementsD3D12KHR output structure.

This call retrieves the D3D12 feature level and graphics device for an instance and system. The xrGetD3D12GraphicsRequirementsKHR function identifies to the application the graphics device (Windows LUID) to be used and the minimum feature level to use. The runtime **must** return `XR_ERROR_GRAPHICS_REQUIREMENTS_CALL_MISSING` (`XR_ERROR_VALIDATION_FAILURE` **may** be returned due to legacy behavior) on calls to xrCreateSession if xrGetD3D12GraphicsRequirementsKHR has not been called for the same `instance` and `systemId`.

The LUID and feature level that xrGetD3D12GraphicsRequirementsKHR returns **must** be used to create the `ID3D12Device` that the application passes to xrCreateSession in the XrGraphicsBindingD3D12KHR.

Valid Usage (Implicit)

- The XR_KHR_D3D12_enable extension **must** be enabled prior to calling xrGetD3D12GraphicsRequirementsKHR
- `instance` **must** be a valid XrInstance handle
- `graphicsRequirements` **must** be a pointer to an XrGraphicsRequirementsD3D12KHR structure

Return Codes

Success

- XR_SUCCESS

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SYSTEM_INVALID
- XR_ERROR_VALIDATION_FAILURE

The XrGraphicsRequirementsD3D12KHR structure is defined as:

```
// Provided by XR_KHR_D3D12_enable
typedef struct XrGraphicsRequirementsD3D12KHR {
    XrStructureType      type;
    void*                next;
    LUID                 adapterLuid;
    D3D_FEATURE_LEVEL    minFeatureLevel;
} XrGraphicsRequirementsD3D12KHR;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `adapterLuid` identifies what graphics device needs to be used.
- `minFeatureLevel` is the minimum feature level for which to initialize the D3D12 device.

XrGraphicsRequirementsD3D12KHR is populated by xrGetD3D12GraphicsRequirementsKHR with the runtime's D3D12 API feature level and adapter requirements.

Valid Usage (Implicit)

- The `XR_KHR_D3D12_enable` extension **must** be enabled prior to using XrGraphicsRequirementsD3D12KHR
- `type` **must** be `XR_TYPE_GRAPHICS_REQUIREMENTS_D3D12_KHR`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `adapterLuid` **must** be a valid `LUID` value
- `minFeatureLevel` **must** be a valid `D3D_FEATURE_LEVEL` value

12.13.3. Graphics Binding Structure

The XrGraphicsBindingD3D12KHR structure is defined as:

```
// Provided by XR_KHR_D3D12_enable
typedef struct XrGraphicsBindingD3D12KHR {
    XrStructureType        type;
    const void*            next;
    ID3D12Device*          device;
    ID3D12CommandQueue*    queue;
} XrGraphicsBindingD3D12KHR;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `device` is a pointer to a valid `ID3D12Device` to use.
- `queue` is a pointer to a valid `ID3D12CommandQueue` to use.

To create a D3D12-backed XrSession, the application provides a pointer to an XrGraphicsBindingD3D12KHR structure in the XrSessionCreateInfo::`next` chain when calling xrCreateSession. The D3D12 device specified in XrGraphicsBindingD3D12KHR::`device` **must** be created in accordance with the requirements retrieved through xrGetD3D12GraphicsRequirementsKHR, otherwise xrCreateSession **must** return `XR_ERROR_GRAPHICS_DEVICE_INVALID`.

Creating a session using this structure triggers several requirements on the runtime regarding swapchain images. See the specification of XrSwapchainImageD3D12KHR for details.

Valid Usage (Implicit)

- The `XR_KHR_D3D12_enable` extension **must** be enabled prior to using XrGraphicsBindingD3D12KHR
- `type` **must** be `XR_TYPE_GRAPHICS_BINDING_D3D12_KHR`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `device` **must** be a pointer to an `ID3D12Device` value
- `queue` **must** be a pointer to an `ID3D12CommandQueue` value

12.13.4. Swapchain Images

The XrSwapchainImageD3D12KHR structure is defined as:

```
// Provided by XR_KHR_D3D12_enable
typedef struct XrSwapchainImageD3D12KHR {
    XrStructureType    type;
    void*              next;
    ID3D12Resource*    texture;
} XrSwapchainImageD3D12KHR;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `texture` is a pointer to a valid `ID3D12Texture2D` to use.

If a given session was created with XrGraphicsBindingD3D12KHR, the following conditions apply.

- Calls to xrEnumerateSwapchainImages on an XrSwapchain in that session **must** return an array of XrSwapchainImageD3D12KHR structures.
- Whenever an OpenXR function accepts an XrSwapchainImageBaseHeader pointer as a parameter in that session, the runtime **must** also accept a pointer to an XrSwapchainImageD3D12KHR.

The OpenXR runtime **must** interpret the top-left corner of the swapchain image as the coordinate origin unless specified otherwise by extension functionality.

The OpenXR runtime **must** return a texture created in accordance with D3D12 Swapchain Flag Bits.

The OpenXR runtime **must** manage image resource state in accordance with D3D12 Swapchain Image Resource State.

Valid Usage (Implicit)

- The `XR_KHR_D3D12_enable` extension **must** be enabled prior to using XrSwapchainImageD3D12KHR
- `type` **must** be `XR_TYPE_SWAPCHAIN_IMAGE_D3D12_KHR`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

12.13.5. D3D12 Swapchain Flag Bits

All valid XrSwapchainUsageFlags values passed in a session created using XrGraphicsBindingD3D12KHR **must** be interpreted as follows by the runtime, so that the returned swapchain images used by the application **may** be used as if they were created with the corresponding `D3D12_RESOURCE_FLAGS` flags and heap type. The runtime **may** set additional resource flags but **must** not restrict usage.

XrSwapchainUsageFlagBits	Corresponding D3D12 resource flag bits
`XR_SWAPCHAIN_USAGE_COLOR_ATTACHMENT_BIT`	`D3D12_RESOURCE_FLAG_ALLOW_RENDER_TARGET`
`XR_SWAPCHAIN_USAGE_DEPTH_STENCIL_ATTACHMENT_BIT`	`D3D12_RESOURCE_FLAG_ALLOW_DEPTH_STENCIL`
`XR_SWAPCHAIN_USAGE_UNORDERED_ACCESS_BIT`	`D3D12_RESOURCE_FLAG_ALLOW_UNORDERED_ACCESS`

XrSwapchainUsageFlagBits	Corresponding D3D12 resource flag bits
XR_SWAPCHAIN_USAGE_TRANSFER_SRC_BIT	*ignored*
XR_SWAPCHAIN_USAGE_TRANSFER_DST_BIT	*ignored*
XR_SWAPCHAIN_USAGE_SAMPLED_BIT **omitted**	D3D12_RESOURCE_FLAG_DENY_SHADER_RESOURCE
XR_SWAPCHAIN_USAGE_MUTABLE_FORMAT_BIT	*ignored*
XR_SWAPCHAIN_USAGE_INPUT_ATTACHMENT_BIT_KHR (Added by XR_KHR_swapchain_usage_input_attachment_bit and only available when that extension is enabled)	*ignored*

All D3D12 swapchain textures are created with D3D12_HEAP_TYPE_DEFAULT heap type.

12.13.6. D3D12 Swapchain Image Resource State

If an application waits on a swapchain image by calling xrWaitSwapchainImage in a session created using XrGraphicsBindingD3D12KHR, and that call returns XR_SUCCESS or XR_SESSION_LOSS_PENDING, then the OpenXR runtime **must** guarantee that the following conditions are true:

- The color rendering target image has a resource state match with D3D12_RESOURCE_STATE_RENDER_TARGET
- The depth rendering target image has a resource state match with D3D12_RESOURCE_STATE_DEPTH_WRITE
- The ID3D12CommandQueue specified in XrGraphicsBindingD3D12KHR is able to write to the image.

When an application releases a swapchain image by calling xrReleaseSwapchainImage in a session created using XrGraphicsBindingD3D12KHR, the OpenXR runtime **must** interpret the image as:

- Having a resource state match with D3D12_RESOURCE_STATE_RENDER_TARGET if the image is a color rendering target
- Having a resource state match with D3D12_RESOURCE_STATE_DEPTH_WRITE if the image is a depth rendering target
- Being available for read/write on the ID3D12CommandQueue specified in XrGraphicsBindingD3D12KHR.

The application is responsible for transitioning the swapchain image back to the resource state and queue availability that the OpenXR runtime requires. If the image is not in a resource state match with the above specifications the runtime **may** exhibit undefined behavior.

12.13.7. New Commands

- xrGetD3D12GraphicsRequirementsKHR

12.13.8. New Structures

- XrGraphicsRequirementsD3D12KHR
- XrSwapchainImageD3D12KHR
- Extending XrSessionCreateInfo:
  - XrGraphicsBindingD3D12KHR

12.13.9. New Enum Constants

- `XR_KHR_D3D12_ENABLE_EXTENSION_NAME`
- `XR_KHR_D3D12_enable_SPEC_VERSION`
- Extending XrStructureType:
  - `XR_TYPE_GRAPHICS_BINDING_D3D12_KHR`
  - `XR_TYPE_GRAPHICS_REQUIREMENTS_D3D12_KHR`
  - `XR_TYPE_SWAPCHAIN_IMAGE_D3D12_KHR`

12.13.10. Version History

- Revision 1, 2018-05-07 (Mark Young)
  - Initial draft
- Revision 2, 2018-06-21 (Bryce Hutchings)
  - Split `XR_KHR_D3D_enable` into `XR_KHR_D3D12_enable`
  - Rename and expand `xrGetD3DGraphicsDeviceKHR` functionality to `xrGetD3D12GraphicsRequirementsKHR`
- Revision 3, 2018-11-15 (Paul Pedriana)
  - Specified the swapchain texture coordinate origin.
- Revision 4, 2018-11-16 (Minmin Gong)
  - Specified Y direction and Z range in clip space
- Revision 5, 2019-01-29 (Dan Ginsburg)
  - Added swapchain image resource state details.
- Revision 6, 2020-03-18 (Minmin Gong)
  - Specified depth swapchain image resource state.
- Revision 7, 2020-08-06 (Bryce Hutchings)
  - Added new `XR_ERROR_GRAPHICS_REQUIREMENTS_CALL_MISSING` error code
- Revision 8, 2021-09-09 (Bryce Hutchings)
  - Document mapping for `XrSwapchainUsageFlags`

- Revision 9, 2021-12-28 (Microsoft)
  - Added missing `XR_ERROR_GRAPHICS_DEVICE_INVALID` error condition
- Revision 10, 2025-03-07 (Rylie Pavlik, Collabora, Ltd.)
  - Re-organize, clarify, and make more uniform with other graphics binding extensions.
- Revision 11, 2025-03-21 (Aaron Leiby)
  - Removed clip space specification

12.14. XR_KHR_extended_struct_name_lengths

Name String

`XR_KHR_extended_struct_name_lengths`

Extension Type

Instance extension

Registered Extension Number

149

Revision

2

Ratification Status

Ratified

Extension and Version Dependencies

OpenXR 1.0

Last Modified Date

2025-12-04

IP Status

No known IP claims.

Contributors

Matthew Langille, Meta Platforms
Andreas Selvik, Meta Platforms
Rylie Pavlik, Collabora, Ltd.

12.14.1. Overview

This extension extends the maximum struct name sizes and provides a new function to access these new extended names.

xrStructureTypeToString2KHR is provided to allow retrieving the string names of structure type enumerants with lengths that exceed the original limit of 63 bytes (64 bytes including the null terminator). xrStructureTypeToString2KHR returns name strings for structure type enumerants up to 255 bytes in length (256 bytes including the null terminator). An application **can** use xrStructureTypeToString2KHR as a drop-in replacement for xrStructureTypeToString, as it works with all structure type enumerants, regardless of string name length.

12.14.2. Retrieving Structure Type Enumerant Strings

If the original xrStructureTypeToString is used to retrieve string names for structure type enumerants with name lengths in excess of 63 bytes, its behavior is clarified as follows. xrStructureTypeToString **must** populate the buffer with the correct name, except that the string **must** be truncated at a codepoint boundary to fit within the available buffer. That is, the returned string **must** always be valid UTF-8.

The xrStructureTypeToString2KHR function is defined as:

```
// Provided by XR_KHR_extended_struct_name_lengths
XrResult xrStructureTypeToString2KHR(
    XrInstance                                  instance,
    XrStructureType                             value,
    char                                        buffer
[XR_MAX_STRUCTURE_NAME_SIZE_EXTENDED_KHR]);
```

Parameter Descriptions

- `instance` is the handle of the instance to ask for the string.
- `value` is the XrStructureType value to turn into a string.
- `buffer` is the buffer that will be used to return the string in.

Returns the name of the provided XrStructureType value by copying a valid null-terminated UTF-8 string into `buffer`.

In all cases the returned string **must** be one of:

Structure Type String Output Values

- The literal string defined for the provided numeric value in the core specification or extension. (e.g. the value of `XR_TYPE_INSTANCE_CREATE_INFO` results in the string `XR_TYPE_INSTANCE_CREATE_INFO`)
- `XR_UNKNOWN_STRUCTURE_TYPE_` concatenated with the structure type number expressed as a decimal number.

For structure type enumerants whose names fit within the original size limit of 63 bytes, xrStructureTypeToString2KHR **must** return the same resultant string as xrStructureTypeToString, up to the null terminator.

Valid Usage (Implicit)

- The `XR_KHR_extended_struct_name_lengths` extension **must** be enabled prior to calling xrStructureTypeToString2KHR
- `instance` **must** be a valid XrInstance handle
- `value` **must** be a valid XrStructureType value
- `buffer` **must** be a character array of length `XR_MAX_STRUCTURE_NAME_SIZE_EXTENDED_KHR`

Return Codes

Success

- `XR_SUCCESS`

Failure

- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_VALIDATION_FAILURE`

The `XR_MAX_STRUCTURE_NAME_SIZE_EXTENDED_KHR` enumerant defines the size of the buffer passed to xrStructureTypeToString2KHR.

```
#define XR_MAX_STRUCTURE_NAME_SIZE_EXTENDED_KHR 256
```

12.14.3. New Commands

- xrStructureTypeToString2KHR

12.14.4. New Enum Constants

- XR_KHR_EXTENDED_STRUCT_NAME_LENGTHS_EXTENSION_NAME
- XR_KHR_extended_struct_name_lengths_SPEC_VERSION
- XR_MAX_STRUCTURE_NAME_SIZE_EXTENDED_KHR

Version History

- Revision 1, 2024-02-29 (Matthew Langille)
  - Initial extension description
- Revision 2, 2025-12-04 (Kenny Vercaemer)
  - Fix structure name length specification

12.15. XR_KHR_generic_controller

Name String

XR_KHR_generic_controller

Extension Type

Instance extension

Registered Extension Number

712

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

OpenXR 1.0

API Interactions

- Interacts with XR_EXT_dpad_binding
- Interacts with XR_EXT_hand_interaction
- Interacts with XR_EXT_palm_pose

Contributors

Andreas Loeve Selvik, Meta Platforms
Bastiaan Olij, Godot Engine
Bryce Hutchings, Microsoft
John Kearney, Meta Platforms
Jules Blok, Epic Games
Nathan Nuber, Valve
Rylie Pavlik, Collabora
Lachlan Ford, Microsoft
Yin Li, Microsoft

12.15.1. Overview

This extension enables a new interaction profile for generic motion controllers. This new interaction profile provides button, trigger, squeeze, thumbstick, and haptic support for applications. Similarly to Khronos Simple Controller Profile, there is no hardware associated with the profile, and runtimes which support this profile **should** map the input paths provided to whatever the appropriate inputs are on the actual hardware.

12.15.2. New Interaction Profile Identifiers

- primary - A standalone button that is easier for the user to interact with than secondary button (`secondary`).
- secondary - A standalone button that is more difficult for the user to interact with than primary button (`primary`).

12.15.3. New Interaction Profile

Khronos Generic Controller Profile

Path: */interaction_profiles/khr/generic_controller*

Valid for user paths:

- */user/hand/left*
- */user/hand/right*

This interaction profile provides basic pose, button, thumbstick, trigger, and haptic support for applications which are able to use generic controller style input.

Unlike many interaction profiles, there is no specific hardware associated with the interaction profile, and runtimes which support this profile **should** map the input/output binding paths to whatever the appropriate inputs/outputs are on the actual hardware.

If there is a specific interaction profile associated with the motion controller in use, and the application

suggests bindings for that specific interaction profile and this Generic Controller Profile, the runtime **should** select the bindings suggested for the hardware specific interaction profile in preference to bindings suggested for this Generic Controller Profile.

Specifically, the Generic Controller Profile is designed to offer broad compatibility across motion controllers which offer generic controller style data but not to comprehensively cover any specific hardware.



Note

The intent of this interaction profile is to provide a fallback. It is still expected that the application will suggest bindings for all hardware based interaction profiles that the application has been tested with.

Some runtimes **must** select bindings suggested for this interaction profile in some conditions.

Specifically, if in some condition, a runtime obeying suggested bindings selects bindings suggested for one of the following interaction profiles:

- */interaction_profiles/oculus/touch_controller*
- */interaction_profiles/valve/index_controller*

Then, such a runtime **must** select suggested bindings for */interaction_profiles/khr/generic_controller* if bindings are suggested for neither of the above, nor for an interaction profile that maps more directly to the devices in use.

That is, if a runtime selects "touch_controller" or "index_controller" in some case, then it **must** select "generic_controller" in a similar situation.



Note

The intent of this language is to **guarantee** support for this interaction profile for runtimes implementing certain interaction profiles that are known to map well, but runtimes that do not typically remap any of these specific interaction profiles are encouraged to map this interaction profile onto their devices.

Supported component paths:

- *.../input/primary/click*
- *.../input/secondary/click*
- *.../input/thumbstick*
- *.../input/thumbstick/x*
- *.../input/thumbstick/y*
- *.../input/thumbstick/click*
- *.../input/squeeze/value*

- *.../input/trigger/value*
- *.../input/grip/pose*
- *.../input/grip_surface/pose*
- *.../input/aim/pose*
- *.../output/haptic*



Note

When the `XR_EXT_palm_pose` extension is available and enabled, this interaction profile **must** also support

- *.../input/palm_ext/pose*



Note

When the `XR_EXT_hand_interaction` extension is available and enabled, this interaction profile **must** also support

- *.../input/pinch_ext/pose*
- *.../input/poke_ext/pose*

Expected binding mappings

The runtime **may** use any appropriate hardware binding in this; however there are natural equivalences between the profiles:

Binding path for */interaction_profiles/khr/generic_controller*	**Equivalent binding path for** */interaction_profiles/oculus/touch_controller*
/user/hand/left/input/primary/click	*/user/hand/left/input/x/click*
/user/hand/left/input/secondary/click	*/user/hand/left/input/y/click*
/user/hand/left/input/thumbstick	*/user/hand/left/input/thumbstick*
/user/hand/left/input/thumbstick/x	*/user/hand/left/input/thumbstick/x*
/user/hand/left/input/thumbstick/y	*/user/hand/left/input/thumbstick/y*
/user/hand/left/input/thumbstick/click	*/user/hand/left/input/thumbstick/click*
/user/hand/left/input/squeeze/value	*/user/hand/left/input/squeeze/value*
/user/hand/left/input/trigger/value	*/user/hand/left/input/trigger/value*
/user/hand/left/input/grip/pose	*/user/hand/left/input/grip/pose*
/user/hand/left/input/grip_surface/pose	*/user/hand/left/input/grip_surface/pose*
/user/hand/left/input/aim/pose	*/user/hand/left/input/aim/pose*
/user/hand/left/output/haptic	*/user/hand/left/output/haptic*

Binding path for */interaction_profiles/khr/generic_controller*	**Equivalent binding path for** */interaction_profiles/oculus/touch_controller*
/user/hand/right/input/primary/click	*/user/hand/right/input/a/click*
/user/hand/right/input/secondary/click	*/user/hand/right/input/b/click*
/user/hand/right/input/thumbstick	*/user/hand/right/input/thumbstick*
/user/hand/right/input/thumbstick/x	*/user/hand/right/input/thumbstick/x*
/user/hand/right/input/thumbstick/y	*/user/hand/right/input/thumbstick/y*
/user/hand/right/input/thumbstick/click	*/user/hand/right/input/thumbstick/click*
/user/hand/right/input/squeeze/value	*/user/hand/right/input/squeeze/value*
/user/hand/right/input/trigger/value	*/user/hand/right/input/trigger/value*
/user/hand/right/input/grip/pose	*/user/hand/right/input/grip/pose*
/user/hand/right/input/grip_surface/pose	*/user/hand/right/input/grip_surface/pose*
/user/hand/right/input/aim/pose	*/user/hand/right/input/aim/pose*
/user/hand/right/output/haptic	*/user/hand/right/output/haptic*

The following binding paths for */interaction_profiles/oculus/touch_controller* lack a generic controller equivalent and therefore are omitted from the preceding table:

- */user/hand/left/input/trigger/proximity*
- */user/hand/left/input/thumb_resting_surfaces/proximity*
- */user/hand/left/input/menu/click*
- */user/hand/right/input/system/click*
- */user/hand/right/input/trigger/proximity*
- */user/hand/right/input/thumb_resting_surfaces/proximity*

Binding path for */interaction_profiles/khr/generic_controller*	**Equivalent binding path for** */interaction_profiles/valve/index_controller*
/user/hand/left/input/primary/click	*/user/hand/left/input/a/click*
/user/hand/left/input/secondary/click	*/user/hand/left/input/b/click*
/user/hand/left/input/thumbstick	*/user/hand/left/input/thumbstick*
/user/hand/left/input/thumbstick/x	*/user/hand/left/input/thumbstick/x*
/user/hand/left/input/thumbstick/y	*/user/hand/left/input/thumbstick/y*
/user/hand/left/input/thumbstick/click	*/user/hand/left/input/thumbstick/click*
/user/hand/left/input/squeeze/value	*/user/hand/left/input/squeeze/value*
/user/hand/left/input/trigger/value	*/user/hand/left/input/trigger/value*

Binding path for */interaction_profiles/khr/generic_controller*	**Equivalent binding path for** */interaction_profiles/valve/index_controller*
/user/hand/left/input/grip/pose	*/user/hand/left/input/grip/pose*
/user/hand/left/input/grip_surface/pose	*/user/hand/left/input/grip_surface/pose*
/user/hand/left/input/aim/pose	*/user/hand/left/input/aim/pose*
/user/hand/left/output/haptic	*/user/hand/left/output/haptic*
/user/hand/right/input/primary/click	*/user/hand/right/input/a/click*
/user/hand/right/input/secondary/click	*/user/hand/right/input/b/click*
/user/hand/right/input/thumbstick	*/user/hand/right/input/thumbstick*
/user/hand/right/input/thumbstick/x	*/user/hand/right/input/thumbstick/x*
/user/hand/right/input/thumbstick/y	*/user/hand/right/input/thumbstick/y*
/user/hand/right/input/thumbstick/click	*/user/hand/right/input/thumbstick/click*
/user/hand/right/input/squeeze/value	*/user/hand/right/input/squeeze/value*
/user/hand/right/input/trigger/value	*/user/hand/right/input/trigger/value*
/user/hand/right/input/grip/pose	*/user/hand/right/input/grip/pose*
/user/hand/right/input/grip_surface/pose	*/user/hand/right/input/grip_surface/pose*
/user/hand/right/input/aim/pose	*/user/hand/right/input/aim/pose*
/user/hand/right/output/haptic	*/user/hand/right/output/haptic*

The following binding paths for */interaction_profiles/valve/index_controller* lack a generic controller equivalent and therefore are omitted from the preceding table:

- */user/hand/left/input/system/click*
- */user/hand/left/input/system/touch*
- */user/hand/left/input/trackpad/x*
- */user/hand/left/input/trackpad/y*
- */user/hand/left/input/trackpad/force*
- */user/hand/left/input/trackpad/touch*
- */user/hand/right/input/system/click*
- */user/hand/right/input/system/touch*
- */user/hand/right/input/trackpad/x*
- */user/hand/right/input/trackpad/y*
- */user/hand/right/input/trackpad/force*
- */user/hand/right/input/trackpad/touch*

12.15.4. New Enum Constants

- `XR_KHR_GENERIC_CONTROLLER_EXTENSION_NAME`
- `XR_KHR_generic_controller_SPEC_VERSION`

12.15.5. Issues

- Should the specification mandate specific bindings for all hardware with existing interaction profiles?
  - No. This is an area where we expect that the runtime has better information that about hardware configuration, user preferences, etc than the application or the specification authors. Requiring specific binding behavior for the runtimes would be counter productive given that assumption.
- Why is the profile described as a Generic Controller Profile rather than some other name?
  - The data that is made available by the Generic Controller Profile can represent the data commonly made available by VR motion controllers but is not specific to any particular hardware.
- Should controller system buttons be added to this profile?
  - No. While interaction profiles make these system buttons available to applications, they are generally not likely to be bound for regular applications, instead being reserved for internal system usage.

12.15.6. Version History

- Revision 1, 2023-2025 (John Kearney)
  - Initial extension description

12.16. XR_KHR_loader_init

Name String

`XR_KHR_loader_init`

Extension Type

Instance extension

Registered Extension Number

89

Revision

2

Ratification Status

Ratified

Extension and Version Dependencies

OpenXR 1.0

Last Modified Date

2023-05-08

IP Status

No known IP claims.

Contributors

Cass Everitt, Facebook
Robert Blenkinsopp, Ultraleap

Overview

On some platforms, before loading can occur the loader must be initialized with platform-specific parameters.

Unlike other extensions, the presence of this extension is signaled by a successful call to xrGetInstanceProcAddr to retrieve the function pointer for xrInitializeLoaderKHR using XR_NULL_HANDLE as the `instance` parameter.

If this extension is supported, its use **may** be required on some platforms and the use of the xrInitializeLoaderKHR function **must** precede other OpenXR calls except xrGetInstanceProcAddr.

This function exists as part of the loader library that the application is using and the loader **must** pass calls to xrInitializeLoaderKHR to the active runtime, and all enabled API layers that expose a xrInitializeLoaderKHR function exposed either through their manifest, or through their implementation of xrGetInstanceProcAddr.

If the xrInitializeLoaderKHR function is discovered through the manifest, xrInitializeLoaderKHR will be called before `xrNegotiateLoaderRuntimeInterface` or `xrNegotiateLoaderApiLayerInterface` has been called on the runtime or layer respectively.

New Object Types

New Flag Types

New Enum Constants

New Enums

New Structures

The XrLoaderInitInfoBaseHeaderKHR structure is defined as:

```
// Provided by XR_KHR_loader_init
typedef struct XrLoaderInitInfoBaseHeaderKHR {
    XrStructureType    type;
    const void*        next;
} XrLoaderInitInfoBaseHeaderKHR;
```

Member Descriptions

- `type` is the XrStructureType of this structure. This base structure itself has no associated XrStructureType value.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.

Valid Usage (Implicit)

- The `XR_KHR_loader_init` extension **must** be enabled prior to using XrLoaderInitInfoBaseHeaderKHR
- `type` **must** be one of the following XrStructureType values: `XR_TYPE_LOADER_INIT_INFO_ANDROID_KHR`, `XR_TYPE_LOADER_INIT_INFO_PROPERTIES_EXT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain. See also: XrLoaderInitInfoPropertiesEXT

New Functions

To initialize an OpenXR loader with platform or implementation-specific parameters, call:

```
// Provided by XR_KHR_loader_init
XrResult xrInitializeLoaderKHR(
    const XrLoaderInitInfoBaseHeaderKHR*        loaderInitInfo);
```

Parameter Descriptions

- `loaderInitInfo` is a pointer to an XrLoaderInitInfoBaseHeaderKHR structure, which is a polymorphic type defined by other platform- or implementation-specific extensions.

Issues

Version History

- Revision 2, 2023-05-08 (Robert Blenkinsoppp)
  - Explicitly state that the call to xrInitializeLoaderKHR should be passed to the runtime and enabled API layers.
- Revision 1, 2020-05-07 (Cass Everitt)
  - Initial draft

12.17. XR_KHR_loader_init_android

Name String

`XR_KHR_loader_init_android`

Extension Type

Instance extension

Registered Extension Number

90

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

OpenXR 1.0
and
`XR_KHR_loader_init`

Last Modified Date

2020-05-07

IP Status

No known IP claims.

Contributors

Cass Everitt, Facebook

Overview

On Android, some loader implementations need the application to provide additional information on initialization. This extension defines the parameters needed by such implementations. If this is available on a given implementation, an application **must** make use of it.

On implementations where use of this is required, the following condition **must** apply:

- Whenever an OpenXR function accepts an XrLoaderInitInfoBaseHeaderKHR pointer, the runtime (and loader) **must** also accept a pointer to an XrLoaderInitInfoAndroidKHR.

New Object Types

New Flag Types

New Enum Constants

XrStructureType enumeration is extended with:

- `XR_TYPE_LOADER_INIT_INFO_ANDROID_KHR`

New Enums

New Structures

The XrLoaderInitInfoAndroidKHR structure is defined as:

```
// Provided by XR_KHR_loader_init_android
typedef struct XrLoaderInitInfoAndroidKHR {
    XrStructureType    type;
    const void*        next;
    void*              applicationVM;
    void*              applicationContext;
} XrLoaderInitInfoAndroidKHR;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `applicationVM` is a pointer to the JNI's opaque `JavaVM` structure, cast to a void pointer.
- `applicationContext` is a JNI reference to an `android.content.Context` associated with the application, cast to a void pointer.

Valid Usage (Implicit)

- The `XR_KHR_loader_init_android` extension **must** be enabled prior to using XrLoaderInitInfoAndroidKHR
- `type` **must** be `XR_TYPE_LOADER_INIT_INFO_ANDROID_KHR`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `applicationVM` **must** be a pointer value
- `applicationContext` **must** be a pointer value

New Functions

Issues

Version History

- Revision 1, 2020-05-07 (Cass Everitt)
  - Initial draft

12.18. XR_KHR_metal_enable

Name String

`XR_KHR_metal_enable`

Extension Type

Instance extension

Registered Extension Number

30

Revision

3

Ratification Status

Ratified

Extension and Version Dependencies

OpenXR 1.0

Last Modified Date

2025-03-21

IP Status

No known IP claims.

Contributors

Xiang Wei, Meta
Peter Kuhn, Unity
John Kearney, Meta
Andreas Selvik, Meta
Jakob Bornecrantz, Collabora
Rylie Pavlik, Collabora
Aaron Leiby, Valve

12.18.1. Overview

This extension enables the use of the Metal® graphics API in an OpenXR application. Without this extension, an OpenXR application **may** not be able to use any Metal swapchain images.

This extension provides the mechanisms necessary for an application to generate a valid XrGraphicsBindingMetalKHR structure in order to create a Metal-based XrSession. Note that during this process, the runtime is responsible for creating the Metal device for the application's drawing operations, and the application is responsible for creating all the required Metal objects from that, including a Metal command queue to be used for rendering. The runtime however will provide the Metal textures to render into in the form of a swapchain.

This extension also provides mechanisms for the application to interact with images acquired by calling xrEnumerateSwapchainImages.

In order to expose the structures, types, and functions of this extension, the application source code **must** define XR_USE_GRAPHICS_API_METAL before including the OpenXR platform header `openxr_platform.h`, in all portions of the library or application that interact with the types, values, and functions it defines.

12.18.2. Get Graphics Requirements

Some computer systems **may** have multiple graphics devices, each of which **may** have independent external display outputs. XR systems that connect to such computer systems are typically connected to a single graphics device. Applications need to know the graphics device associated with the XR system, so that rendering takes place on the correct graphics device.

To retrieve the Metal device that **can** be used in drawing operations, call:

```
// Provided by XR_KHR_metal_enable
XrResult xrGetMetalGraphicsRequirementsKHR(
    XrInstance                                  instance,
    XrSystemId                                  systemId,
    XrGraphicsRequirementsMetalKHR*             graphicsRequirements);
```

Parameter Descriptions

- `instance` is an XrInstance handle previously created with xrCreateInstance.
- `systemId` is an `XrSystemId` handle for the system which will be used to create a session.
- `graphicsRequirements` is the XrGraphicsRequirementsMetalKHR output structure.

The xrGetMetalGraphicsRequirementsKHR function identifies to the application the Metal device to be used in drawing operations. The runtime **must** return `XR_ERROR_GRAPHICS_REQUIREMENTS_CALL_MISSING` on calls to xrCreateSession if xrGetMetalGraphicsRequirementsKHR has not been called for the same `instance` and `systemId`.

The Metal device that xrGetMetalGraphicsRequirementsKHR returns **must** be used to create the Metal command queue that the application passes to xrCreateSession in the XrGraphicsBindingMetalKHR.

Valid Usage (Implicit)

- The `XR_KHR_metal_enable` extension **must** be enabled prior to calling xrGetMetalGraphicsRequirementsKHR
- `instance` **must** be a valid XrInstance handle
- `graphicsRequirements` **must** be a pointer to an XrGraphicsRequirementsMetalKHR structure

Return Codes

Success

- XR_SUCCESS

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SYSTEM_INVALID
- XR_ERROR_VALIDATION_FAILURE

The XrGraphicsRequirementsMetalKHR structure is defined as:

```
// Provided by XR_KHR_metal_enable
typedef struct XrGraphicsRequirementsMetalKHR {
    XrStructureType    type;
    void*              next;
    void*              metalDevice;
} XrGraphicsRequirementsMetalKHR;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- metalDevice is the Metal device that the application **must** use for drawing operations, which **may** be bridged casted to an Objective-C object that conforms to the MTLDevice protocol.

XrGraphicsRequirementsMetalKHR is populated by xrGetMetalGraphicsRequirementsKHR.

Valid Usage (Implicit)

- The XR_KHR_metal_enable extension **must** be enabled prior to using XrGraphicsRequirementsMetalKHR
- type **must** be XR_TYPE_GRAPHICS_REQUIREMENTS_METAL_KHR
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- metalDevice **must** be a pointer value

12.18.3. Graphics Binding Structure

The XrGraphicsBindingMetalKHR structure is defined as:

```
// Provided by XR_KHR_metal_enable
typedef struct XrGraphicsBindingMetalKHR {
    XrStructureType    type;
    const void*        next;
    void*              commandQueue;
} XrGraphicsBindingMetalKHR;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- commandQueue is a valid Metal command queue, which **must** be bridged casted from an Objective-C object that conforms to the MTLCommandQueue protocol.

To create a Metal-backed XrSession, the application provides a pointer to an XrGraphicsBindingMetalKHR in the XrSessionCreateInfo::next field of structure passed to xrCreateSession. The Metal command queue specified in XrGraphicsBindingMetalKHR::commandQueue **must** be created on the Metal device retrieved through XrGraphicsRequirementsMetalKHR ::metalDevice, otherwise xrCreateSession **must** return XR_ERROR_GRAPHICS_DEVICE_INVALID.

Creating a session using this structure triggers several requirements on the runtime regarding swapchain images. See the specification of XrSwapchainImageMetalKHR for details.

Valid Usage (Implicit)

- The `XR_KHR_metal_enable` extension **must** be enabled prior to using XrGraphicsBindingMetalKHR
- `type` **must** be `XR_TYPE_GRAPHICS_BINDING_METAL_KHR`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `commandQueue` **must** be a pointer value

12.18.4. Swapchain Images

The XrSwapchainImageMetalKHR structure is defined as:

```
// Provided by XR_KHR_metal_enable
typedef struct XrSwapchainImageMetalKHR {
    XrStructureType    type;
    void*              next;
    void*              texture;
} XrSwapchainImageMetalKHR;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `texture` is populated with a valid Metal texture to use, which **must** be able to be bridged casted to an Objective-C object that conforms to the `MTLTexture` protocol.

If a given session was created with XrGraphicsBindingMetalKHR, the following conditions apply.

- Calls to xrEnumerateSwapchainImages on an XrSwapchain in that session **must** return an array of XrSwapchainImageMetalKHR structures.
- Whenever an OpenXR function accepts an XrSwapchainImageBaseHeader pointer as a parameter in that session, the runtime **must** also accept a pointer to an XrSwapchainImageMetalKHR.

The OpenXR runtime **must** interpret the top-left corner of the swapchain image as the coordinate origin unless specified otherwise by extension functionality.

Valid Usage (Implicit)

- The XR_KHR_metal_enable extension **must** be enabled prior to using XrSwapchainImageMetalKHR
- type **must** be XR_TYPE_SWAPCHAIN_IMAGE_METAL_KHR
- next **must** be NULL or a valid pointer to the next structure in a structure chain

12.18.5. Metal Swapchain Flag Bits

All valid XrSwapchainUsageFlags values passed in a session created using XrGraphicsBindingMetalKHR **must** be interpreted as follows by the runtime, so that the returned swapchain images used by the application **may** be used as if they were created with the corresponding MTLTextureUsage flags. The runtime **may** set additional bind flags but **must** not restrict usage.

XrSwapchainUsageFlagBits	Corresponding MTLTextureUsage bits
XR_SWAPCHAIN_USAGE_COLOR_ATTACHMENT_BIT	MTLTextureUsageRenderTarget
XR_SWAPCHAIN_USAGE_DEPTH_STENCIL_ATTACHMENT_BIT	MTLTextureUsageRenderTarget
XR_SWAPCHAIN_USAGE_UNORDERED_ACCESS_BIT	MTLTextureUsageShaderRead | MTLTextureUsageShaderWrite
XR_SWAPCHAIN_USAGE_TRANSFER_SRC_BIT	*ignored*
XR_SWAPCHAIN_USAGE_TRANSFER_DST_BIT	*ignored*
XR_SWAPCHAIN_USAGE_SAMPLED_BIT	MTLTextureUsageShaderRead
XR_SWAPCHAIN_USAGE_MUTABLE_FORMAT_BIT	MTLTextureUsagePixelFormatView
XR_SWAPCHAIN_USAGE_INPUT_ATTACHMENT_BIT_KHR (Added by XR_KHR_swapchain_usage_input_attachment_bit and only available when that extension is enabled)	*ignored*

All Metal swapchain textures are created with MTLResourceStorageModePrivate resource option, and are accessible only by the GPU.

12.18.6. Issues

- How to manage the resource state of the Swapchain textures, etc?
  - The application uses the Metal device that is created by the runtime for the drawing operations. The runtime uses the same Metal device to create the swapchain images, and also create the synchronization events when necessary. On top of that, Metal tracks the write hazards and synchronizes the resources which are created from the same Metal device and directly bind to a pipeline. Please check this Apple documentation for more details: https://developer.apple.com/documentation/metal/resource_synchronization?language=objc

12.18.7. New Commands

- xrGetMetalGraphicsRequirementsKHR

12.18.8. New Structures

- XrGraphicsRequirementsMetalKHR
- XrSwapchainImageMetalKHR
- Extending XrSessionCreateInfo:
  - XrGraphicsBindingMetalKHR

12.18.9. New Enum Constants

- XR_KHR_METAL_ENABLE_EXTENSION_NAME
- XR_KHR_metal_enable_SPEC_VERSION
- Extending XrStructureType:
  - XR_TYPE_GRAPHICS_BINDING_METAL_KHR
  - XR_TYPE_GRAPHICS_REQUIREMENTS_METAL_KHR
  - XR_TYPE_SWAPCHAIN_IMAGE_METAL_KHR

12.18.10. Version History

- Revision 1, 2023-05-08 (Xiang Wei)
  - Initial draft
- Revision 2, 2025-03-07 (Rylie Pavlik, Collabora, Ltd.)
  - Re-organize, clarify, and make more uniform with other graphics binding extensions.
- Revision 3, 2025-03-21 (Aaron Leiby)
  - Removed clip space specification

12.19. XR_KHR_opengl_enable

Name String

XR_KHR_opengl_enable

Extension Type

Instance extension

Registered Extension Number

24

Revision

12

Ratification Status

Ratified

Extension and Version Dependencies

OpenXR 1.0

Last Modified Date

2025-10-16

IP Status

No known IP claims.

Contributors

Mark Young, LunarG
Bryce Hutchings, Microsoft
Paul Pedriana, Oculus
Minmin Gong, Microsoft
Robert Menzel, NVIDIA
Jakob Bornecrantz, Collabora
Paulo Gomes, Samsung Electronics
Aaron Leiby, Valve

12.19.1. Overview

This extension enables the use of the OpenGL graphics API in an OpenXR application. Without this extension, an OpenXR application **may** not be able to use any OpenGL swapchain images.

This extension provides the mechanisms necessary for an application to generate a valid `XrGraphicsBindingOpenGL*KHR` structure in order to create an OpenGL-based XrSession. Note that the application is responsible for creating an OpenGL context to be used for rendering. However, the runtime provides the OpenGL textures to render into. This extension provides mechanisms for the application to interact with those textures by calling xrEnumerateSwapchainImages and providing XrSwapchainImageOpenGLKHR structures to populate.

In order to expose the structures, types, and functions of this extension, the application source code **must** define XR_USE_GRAPHICS_API_OPENGL, as well as an appropriate window system define supported by this extension, before including the OpenXR platform header `openxr_platform.h`, in all portions of the library or application that interact with the types, values, and functions it defines. The window system defines currently supported by this extension are:

- XR_USE_PLATFORM_WIN32
- XR_USE_PLATFORM_XLIB

- XR_USE_PLATFORM_XCB
- XR_USE_PLATFORM_WAYLAND (deprecated)

Note that a runtime implementation of this extension is only required to support the structures introduced by this extension which correspond to the platform it is running on. For Wayland deprecation see Wayland Notes.

12.19.2. OpenGL Context and Threading

Note that the OpenGL context given to the call to xrCreateSession **must** not be bound in *another* thread by the application when calling the functions:

- xrCreateSession
- xrDestroySession
- xrBeginFrame
- xrEndFrame
- xrCreateSwapchain
- xrDestroySwapchain
- xrEnumerateSwapchainImages
- xrAcquireSwapchainImage
- xrWaitSwapchainImage
- xrReleaseSwapchainImage

However, it **may** be bound in the thread calling one of those functions. The runtime **must** not access the context from any other function. In particular the application **must** be able to call xrWaitFrame from a different thread than the rendering thread.

12.19.3. Get Graphics Requirements

The xrGetOpenGLGraphicsRequirementsKHR function is defined as:

```
// Provided by XR_KHR_opengl_enable
XrResult xrGetOpenGLGraphicsRequirementsKHR(
    XrInstance                                  instance,
    XrSystemId                                  systemId,
    XrGraphicsRequirementsOpenGLKHR*            graphicsRequirements);
```

Parameter Descriptions

- `instance` is an XrInstance handle previously created with xrCreateInstance.
- `systemId` is an `XrSystemId` handle for the system which will be used to create a session.
- `graphicsRequirements` is the XrGraphicsRequirementsOpenGLKHR output structure.

This call queries OpenGL API version requirements for an instance and system. The xrGetOpenGLGraphicsRequirementsKHR function identifies to the application the minimum OpenGL version requirement and the highest known tested OpenGL version. The runtime **must** return `XR_ERROR_GRAPHICS_REQUIREMENTS_CALL_MISSING` (`XR_ERROR_VALIDATION_FAILURE` **may** be returned due to legacy behavior) on calls to xrCreateSession if xrGetOpenGLGraphicsRequirementsKHR has not been called for the same `instance` and `systemId`.

Valid Usage (Implicit)

- The `XR_KHR_opengl_enable` extension **must** be enabled prior to calling xrGetOpenGLGraphicsRequirementsKHR
- `instance` **must** be a valid XrInstance handle
- `graphicsRequirements` **must** be a pointer to an XrGraphicsRequirementsOpenGLKHR structure

Return Codes

Success

- `XR_SUCCESS`

Failure

- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SYSTEM_INVALID`
- `XR_ERROR_VALIDATION_FAILURE`

The XrGraphicsRequirementsOpenGLKHR structure is defined as:

```
// Provided by XR_KHR_opengl_enable
typedef struct XrGraphicsRequirementsOpenGLKHR {
    XrStructureType    type;
    void*              next;
    XrVersion          minApiVersionSupported;
    XrVersion          maxApiVersionSupported;
} XrGraphicsRequirementsOpenGLKHR;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `minApiVersionSupported` is the minimum version of OpenGL that the runtime supports. Uses XR_MAKE_VERSION on major and minor API version, ignoring any patch version component.
- `maxApiVersionSupported` is the maximum version of OpenGL that the runtime has been tested on and is known to support. Newer OpenGL versions **may** work if they are compatible. Uses XR_MAKE_VERSION on major and minor API version, ignoring any patch version component.

XrGraphicsRequirementsOpenGLKHR is populated by xrGetOpenGLGraphicsRequirementsKHR with the runtime's OpenGL API version requirements.

Valid Usage (Implicit)

- The `XR_KHR_opengl_enable` extension **must** be enabled prior to using XrGraphicsRequirementsOpenGLKHR
- `type` **must** be `XR_TYPE_GRAPHICS_REQUIREMENTS_OPENGL_KHR`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

12.19.4. Graphics Binding Structure

These structures are only available when the corresponding `XR_USE_PLATFORM_` window system/platform macro is defined before including `openxr_platform.h`.

The XrGraphicsBindingOpenGLWin32KHR structure is defined as:

```
// Provided by XR_KHR_opengl_enable
typedef struct XrGraphicsBindingOpenGLWin32KHR {
    XrStructureType    type;
    const void*        next;
    HDC                hDC;
    HGLRC              hGLRC;
} XrGraphicsBindingOpenGLWin32KHR;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `hDC` is a valid Windows HW device context handle.
- `hGLRC` is a valid Windows OpenGL rendering context handle.

To create an OpenGL-backed XrSession on Microsoft Windows, the application provides a pointer to an XrGraphicsBindingOpenGLWin32KHR structure in the XrSessionCreateInfo::`next` chain when calling xrCreateSession. As no standardized way exists for OpenGL to create the graphics context on a specific GPU, the runtime **must** assume that the application uses the operating system's default GPU when this structure is supplied. If the GPU used by the runtime does not match the GPU on which the OpenGL context of the application was created, xrCreateSession **must** return `XR_ERROR_GRAPHICS_DEVICE_INVALID`.

Creating a session using this structure triggers several requirements on the runtime regarding swapchain images. See the specification of XrSwapchainImageOpenGLKHR for details.

By providing a context as `hGLRC`, the application becomes subject to restrictions on use of that context which effectively introduces additional external synchronization requirements on some OpenXR calls. See OpenGL Context and Threading for details.

The required window system configuration define to expose this structure type is XR_USE_PLATFORM_WIN32.

Valid Usage (Implicit)

- The XR_KHR_opengl_enable extension **must** be enabled prior to using XrGraphicsBindingOpenGLWin32KHR
- type **must** be XR_TYPE_GRAPHICS_BINDING_OPENGL_WIN32_KHR
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- hDC **must** be a valid HDC value
- hGLRC **must** be a valid HGLRC value

The XrGraphicsBindingOpenGLXlibKHR structure is defined as:

```
// Provided by XR_KHR_opengl_enable
typedef struct XrGraphicsBindingOpenGLXlibKHR {
    XrStructureType    type;
    const void*        next;
    Display*           xDisplay;
    uint32_t           visualid;
    GLXFBConfig        glxFBConfig;
    GLXDrawable        glxDrawable;
    GLXContext         glxContext;
} XrGraphicsBindingOpenGLXlibKHR;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- xDisplay is a valid X11 Display.
- visualid is a valid X11 visual identifier.
- glxFBConfig is a valid X11 OpenGL GLX GLXFBConfig.
- glxDrawable is a valid X11 OpenGL GLX GLXDrawable.
- glxContext is a valid X11 OpenGL GLX GLXContext.

To create an OpenGL-backed XrSession on any Linux/Unix platform that utilizes X11 and GLX, via the Xlib library, the application provides a pointer to an XrGraphicsBindingOpenGLXlibKHR in the XrSessionCreateInfo::next chain when calling xrCreateSession.

Creating a session using this structure triggers several requirements on the runtime regarding

swapchain images. See the specification of XrSwapchainImageOpenGLKHR for details.

By providing a GLX context as `glxContext`, the application becomes subject to restrictions on use of that context which effectively introduces additional external synchronization requirements on some OpenXR calls. See OpenGL Context and Threading for details.

The required window system configuration define to expose this structure type is XR_USE_PLATFORM_XLIB.

Valid Usage (Implicit)

- The `XR_KHR_opengl_enable` extension **must** be enabled prior to using XrGraphicsBindingOpenGLXlibKHR
- `type` **must** be `XR_TYPE_GRAPHICS_BINDING_OPENGL_XLIB_KHR`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `xDisplay` **must** be a pointer to a `Display` value
- `glxFBConfig` **must** be a valid `GLXFBConfig` value
- `glxDrawable` **must** be a valid `GLXDrawable` value
- `glxContext` **must** be a valid `GLXContext` value

The XrGraphicsBindingOpenGLXcbKHR structure is defined as:

```
// Provided by XR_KHR_opengl_enable
typedef struct XrGraphicsBindingOpenGLXcbKHR {
    XrStructureType       type;
    const void*           next;
    xcb_connection_t*     connection;
    uint32_t              screenNumber;
    xcb_glx_fbconfig_t    fbconfigid;
    xcb_visualid_t        visualid;
    xcb_glx_drawable_t    glxDrawable;
    xcb_glx_context_t     glxContext;
} XrGraphicsBindingOpenGLXcbKHR;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `connection` is a valid `xcb_connection_t`.
- `screenNumber` is an index indicating the screen to for rendering.
- `fbconfigid` is a valid XCB OpenGL GLX `xcb_glx_fbconfig_t`.
- `visualid` is a valid XCB OpenGL GLX `xcb_visualid_t`.
- `glxDrawable` is a valid XCB OpenGL GLX `xcb_glx_drawable_t`.
- `glxContext` is a valid XCB OpenGL GLX `xcb_glx_context_t`.

To create an OpenGL-backed XrSession on any Linux/Unix platform that utilizes X11 and GLX, via the Xlib library, the application provides a pointer to an XrGraphicsBindingOpenGLXcbKHR in the XrSessionCreateInfo::`next` chain when calling xrCreateSession.

Creating a session using this structure triggers several requirements on the runtime regarding swapchain images. See the specification of XrSwapchainImageOpenGLKHR for details.

By providing a GLX context as `glxContext`, the application becomes subject to restrictions on use of that context which effectively introduces additional external synchronization requirements on some OpenXR calls. See OpenGL Context and Threading for details.

The required window system configuration define to expose this structure type is XR_USE_PLATFORM_XCB.

Valid Usage (Implicit)

- The `XR_KHR_opengl_enable` extension **must** be enabled prior to using XrGraphicsBindingOpenGLXcbKHR
- `type` **must** be `XR_TYPE_GRAPHICS_BINDING_OPENGL_XCB_KHR`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `connection` **must** be a pointer to an `xcb_connection_t` value
- `fbconfigid` **must** be a valid `xcb_glx_fbconfig_t` value
- `visualid` **must** be a valid `xcb_visualid_t` value
- `glxDrawable` **must** be a valid `xcb_glx_drawable_t` value
- `glxContext` **must** be a valid `xcb_glx_context_t` value

The XrGraphicsBindingOpenGLWaylandKHR structure is defined as:

```
// Provided by XR_KHR_opengl_enable
typedef struct XrGraphicsBindingOpenGLWaylandKHR {
    XrStructureType       type;
    const void*           next;
    struct wl_display*    display;
} XrGraphicsBindingOpenGLWaylandKHR;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `display` is a valid Wayland `wl_display`.

Note that use of this structure is **deprecated** - see Wayland Notes.

To create an OpenGL-backed XrSession on any Linux/Unix platform that utilizes the Wayland protocol with its compositor, the application provides a pointer to an XrGraphicsBindingOpenGLWaylandKHR in the XrSessionCreateInfo::`next` chain when calling xrCreateSession.

Creating a session using this structure triggers several requirements on the runtime regarding swapchain images. See the specification of XrSwapchainImageOpenGLKHR for details.

The required window system configuration define to expose this structure type is XR_USE_PLATFORM_WAYLAND.

Valid Usage (Implicit)

- The `XR_KHR_opengl_enable` extension **must** be enabled prior to using XrGraphicsBindingOpenGLWaylandKHR
- `type` **must** be `XR_TYPE_GRAPHICS_BINDING_OPENGL_WAYLAND_KHR`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `display` **must** be a pointer to a `wl_display` value

12.19.5. Swapchain Images

The XrSwapchainImageOpenGLKHR structure is defined as:

```
// Provided by XR_KHR_opengl_enable
typedef struct XrSwapchainImageOpenGLKHR {
    XrStructureType    type;
    void*              next;
    uint32_t           image;
} XrSwapchainImageOpenGLKHR;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `image` is the OpenGL texture handle associated with this swapchain image.

If a given session was created with some `XrGraphicsBindingOpenGL*KHR` graphics binding structure, the following conditions apply.

- Calls to xrEnumerateSwapchainImages on an XrSwapchain in that session **must** return an array of XrSwapchainImageOpenGLKHR structures.
- Whenever an OpenXR function accepts an XrSwapchainImageBaseHeader pointer as a parameter in that session, the runtime **must** also accept a pointer to an XrSwapchainImageOpenGLKHR.

The OpenXR runtime **must** interpret the bottom-left corner of the swapchain image as the coordinate origin unless specified otherwise by extension functionality.

Valid Usage (Implicit)

- The `XR_KHR_opengl_enable` extension **must** be enabled prior to using XrSwapchainImageOpenGLKHR
- `type` **must** be `XR_TYPE_SWAPCHAIN_IMAGE_OPENGL_KHR`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

12.19.6. OpenGL Swapchain Flag Bits

All valid XrSwapchainUsageFlags values passed in a session created using XrGraphicsBindingOpenGLWin32KHR, XrGraphicsBindingOpenGLXlibKHR, XrGraphicsBindingOpenGLXcbKHR, or XrGraphicsBindingOpenGLWaylandKHR **should** be ignored as there is no mapping to OpenGL texture settings.



Note

In such a session, a runtime **may** use a supporting graphics API, such as Vulkan, to allocate images that are intended to alias with OpenGL textures, and be part of an XrSwapchain. A runtime which allocates the texture with a different graphics API **may** need to enable several usage flags on the underlying native texture resource to ensure compatibility with OpenGL.

12.19.7. New Commands

- xrGetOpenGLGraphicsRequirementsKHR

12.19.8. New Structures

- XrGraphicsRequirementsOpenGLKHR
- XrSwapchainImageOpenGLKHR
- Extending XrSessionCreateInfo:
  - XrGraphicsBindingOpenGLWaylandKHR
  - XrGraphicsBindingOpenGLWin32KHR
  - XrGraphicsBindingOpenGLXcbKHR
  - XrGraphicsBindingOpenGLXlibKHR

12.19.9. New Enum Constants

- `XR_KHR_OPENGL_ENABLE_EXTENSION_NAME`
- `XR_KHR_opengl_enable_SPEC_VERSION`
- Extending XrStructureType:
  - `XR_TYPE_GRAPHICS_BINDING_OPENGL_WAYLAND_KHR`
  - `XR_TYPE_GRAPHICS_BINDING_OPENGL_WIN32_KHR`
  - `XR_TYPE_GRAPHICS_BINDING_OPENGL_XCB_KHR`
  - `XR_TYPE_GRAPHICS_BINDING_OPENGL_XLIB_KHR`
  - `XR_TYPE_GRAPHICS_REQUIREMENTS_OPENGL_KHR`
  - `XR_TYPE_SWAPCHAIN_IMAGE_OPENGL_KHR`

12.19.10. Issues

- Why was Wayland deprecated?
  - See Wayland Notes.

12.19.11. Version History

- Revision 1, 2018-05-07 (Mark Young)
  - Initial draft
- Revision 2, 2018-06-21 (Bryce Hutchings)
  - Add new `xrGetOpenGLGraphicsRequirementsKHR`
- Revision 3, 2018-11-15 (Paul Pedriana)
  - Specified the swapchain texture coordinate origin.
- Revision 4, 2018-11-16 (Minmin Gong)
  - Specified Y direction and Z range in clip space
- Revision 5, 2019-01-25 (Robert Menzel)
  - Description updated
- Revision 6, 2019-07-02 (Robert Menzel)
  - Minor fixes
- Revision 7, 2019-07-08 (Rylie Pavlik)
  - Adjusted member name in XCB struct
- Revision 8, 2019-11-28 (Jakob Bornecrantz)
  - Added note about context not allowed to be current in a different thread.
- Revision 9, 2020-08-06 (Bryce Hutchings)
  - Added new `XR_ERROR_GRAPHICS_REQUIREMENTS_CALL_MISSING` error code
- Revision 10, 2021-08-31 (Paulo F. Gomes)
  - Document handling of `XrSwapchainUsageFlags`
- Revision 11, 2025-03-07 (Rylie Pavlik, Collabora, Ltd.)
  - Re-organize, clarify, and make more uniform with other graphics binding extensions.
- Revision 12, 2025-10-16 (Jakob Bornecrantz, NVIDIA and Aaron Leiby)
  - Removed clip space specification
  - Deprecate Wayland part of the extension.

12.20. XR_KHR_opengl_es_enable

Name String

`XR_KHR_opengl_es_enable`

Extension Type

Instance extension

Registered Extension Number

25

Revision

10

Ratification Status

Ratified

Extension and Version Dependencies

OpenXR 1.0

Last Modified Date

2025-03-21

IP Status

No known IP claims.

Contributors

Mark Young, LunarG
Bryce Hutchings, Microsoft
Paul Pedriana, Oculus
Minmin Gong, Microsoft
Robert Menzel, NVIDIA
Martin Renschler, Qualcomm
Paulo Gomes, Samsung Electronics
Aaron Leiby, Valve

12.20.1. Overview

This extension enables the use of the OpenGL ES graphics API in an OpenXR application. Without this extension, an OpenXR application **may** not be able to use any OpenGL ES swapchain images.

This extension provides the mechanisms necessary for an application to generate a valid XrGraphicsBindingOpenGLESAndroidKHR structure in order to create an OpenGL ES-based XrSession. The runtime needs the following OpenGL ES objects from the application in order to interact properly with the OpenGL ES driver: EGLDisplay, EGLConfig, and EGLContext. Although not theoretically Android-specific, the OpenGL ES extension is currently tailored for Android. Note that the application is responsible for creating an OpenGL ES context to be used for rendering. However, the runtime provides the OpenGL ES textures to render into. This extension provides mechanisms for the application to interact with those textures by calling xrEnumerateSwapchainImages and providing XrSwapchainImageOpenGLESKHR structures to populate.

In order to expose the structures, types, and functions of this extension, the application source code **must** define XR_USE_GRAPHICS_API_OPENGL_ES, as well as an appropriate window system define

supported by this extension, before including the OpenXR platform header `openxr_platform.h`, in all portions of the library or application that interact with the types, values, and functions it defines. The only window system define currently supported by this extension is:

- XR_USE_PLATFORM_ANDROID



Note

This extension does **not** specify requirements for when the supplied context is current in any thread, unlike `XR_KHR_opengl_enable`. This is an oversight in the design of this extension, and fixing it requires either breaking compatibility or a new extension.

12.20.2. Get Graphics Requirements

The xrGetOpenGLESGraphicsRequirementsKHR function is defined as:

```
// Provided by XR_KHR_opengl_es_enable
XrResult xrGetOpenGLESGraphicsRequirementsKHR(
    XrInstance                                  instance,
    XrSystemId                                  systemId,
    XrGraphicsRequirementsOpenGLESKHR*          graphicsRequirements);
```

Parameter Descriptions

- `instance` is an XrInstance handle previously created with xrCreateInstance.
- `systemId` is an `XrSystemId` handle for the system which will be used to create a session.
- `graphicsRequirements` is the XrGraphicsRequirementsOpenGLESKHR output structure.

This call queries OpenGL ES API version requirements for an instance and system. The xrGetOpenGLESGraphicsRequirementsKHR function identifies to the application the minimum OpenGL ES version requirement and the highest known tested OpenGL ES version. The runtime **must** return `XR_ERROR_GRAPHICS_REQUIREMENTS_CALL_MISSING` (`XR_ERROR_VALIDATION_FAILURE` **may** be returned due to legacy behavior) on calls to xrCreateSession if xrGetOpenGLESGraphicsRequirementsKHR has not been called for the same `instance` and `systemId`.

Valid Usage (Implicit)

- The XR_KHR_opengl_es_enable extension **must** be enabled prior to calling xrGetOpenGLESGraphicsRequirementsKHR
- instance **must** be a valid XrInstance handle
- graphicsRequirements **must** be a pointer to an XrGraphicsRequirementsOpenGLESKHR structure

Return Codes

Success

- XR_SUCCESS

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SYSTEM_INVALID
- XR_ERROR_VALIDATION_FAILURE

The XrGraphicsRequirementsOpenGLESKHR structure is defined as:

```
// Provided by XR_KHR_opengl_es_enable
typedef struct XrGraphicsRequirementsOpenGLESKHR {
    XrStructureType    type;
    void*              next;
    XrVersion          minApiVersionSupported;
    XrVersion          maxApiVersionSupported;
} XrGraphicsRequirementsOpenGLESKHR;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `minApiVersionSupported` is the minimum version of OpenGL ES that the runtime supports. Uses XR_MAKE_VERSION on major and minor API version, ignoring any patch version component.
- `maxApiVersionSupported` is the maximum version of OpenGL ES that the runtime has been tested on and is known to support. Newer OpenGL ES versions **may** work if they are compatible. Uses XR_MAKE_VERSION on major and minor API version, ignoring any patch version component.

XrGraphicsRequirementsOpenGLESKHR is populated by xrGetOpenGLESGraphicsRequirementsKHR with the runtime's OpenGL ES API version requirements.

Valid Usage (Implicit)

- The XR_KHR_opengl_es_enable extension **must** be enabled prior to using XrGraphicsRequirementsOpenGLESKHR
- `type` **must** be `XR_TYPE_GRAPHICS_REQUIREMENTS_OPENGL_ES_KHR`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

12.20.3. Graphics Binding Structure

These structures are only available when the corresponding `XR_USE_PLATFORM_` window system/platform macro is defined before including `openxr_platform.h`.

The XrGraphicsBindingOpenGLESAndroidKHR structure is defined as:

```
// Provided by XR_KHR_opengl_es_enable
typedef struct XrGraphicsBindingOpenGLESAndroidKHR {
    XrStructureType    type;
    const void*        next;
    EGLDisplay         display;
    EGLConfig          config;
    EGLContext         context;
} XrGraphicsBindingOpenGLESAndroidKHR;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `display` is a valid Android OpenGL ES `EGLDisplay`.
- `config` is a valid Android OpenGL ES `EGLConfig`.
- `context` is a valid Android OpenGL ES `EGLContext`.

To create an OpenGL ES-backed XrSession on Android, the application **can** provide a pointer to an XrGraphicsBindingOpenGLESAndroidKHR structure in the XrSessionCreateInfo::`next` chain when calling xrCreateSession.

Creating a session using this structure triggers several requirements on the runtime regarding swapchain images. See the specification of XrSwapchainImageOpenGLESKHR for details.

The required window system configuration define to expose this structure type is XR_USE_PLATFORM_ANDROID.

Valid Usage (Implicit)

- The `XR_KHR_opengl_es_enable` extension **must** be enabled prior to using XrGraphicsBindingOpenGLESAndroidKHR
- `type` **must** be `XR_TYPE_GRAPHICS_BINDING_OPENGL_ES_ANDROID_KHR`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `display` **must** be a valid `EGLDisplay` value
- `config` **must** be a valid `EGLConfig` value
- `context` **must** be a valid `EGLContext` value

12.20.4. Swapchain Images

The XrSwapchainImageOpenGLESKHR structure is defined as:

```
// Provided by XR_KHR_opengl_es_enable
typedef struct XrSwapchainImageOpenGLESKHR {
    XrStructureType    type;
    void*              next;
    uint32_t           image;
} XrSwapchainImageOpenGLESKHR;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `image` is an index indicating the current OpenGL ES swapchain image to use.

If a given session was created with some `XrGraphicsBindingOpenGLES*KHR` graphics binding structure, the following conditions apply.

- Calls to xrEnumerateSwapchainImages on an XrSwapchain in that session **must** return an array of XrSwapchainImageOpenGLESKHR structures.
- Whenever an OpenXR function accepts an XrSwapchainImageBaseHeader pointer as a parameter in that session, the runtime **must** also accept a pointer to an XrSwapchainImageOpenGLESKHR structure.

The OpenXR runtime **must** interpret the bottom-left corner of the swapchain image as the coordinate origin unless specified otherwise by extension functionality.

Valid Usage (Implicit)

- The `XR_KHR_opengl_es_enable` extension **must** be enabled prior to using XrSwapchainImageOpenGLESKHR
- `type` **must** be `XR_TYPE_SWAPCHAIN_IMAGE_OPENGL_ES_KHR`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

12.20.5. OpenGL ES Swapchain Flag Bits

All valid XrSwapchainUsageFlags values passed in a session created using XrGraphicsBindingOpenGLESAndroidKHR **should** be ignored as there is no mapping to OpenGL ES texture settings.



Note

In such a session, a runtime **may** use a supporting graphics API, such as Vulkan, to allocate images that are intended to alias with OpenGLES textures, and be part of an XrSwapchain. A runtime which allocates the texture with a different graphics API **may** need to enable several usage flags on the underlying native texture resource to ensure compatibility with OpenGL ES.

12.20.6. New Commands

- xrGetOpenGLESGraphicsRequirementsKHR

12.20.7. New Structures

- XrGraphicsRequirementsOpenGLESKHR
- XrSwapchainImageOpenGLESKHR
- Extending XrSessionCreateInfo:
  - XrGraphicsBindingOpenGLESAndroidKHR

12.20.8. New Enum Constants

- XR_KHR_OPENGL_ES_ENABLE_EXTENSION_NAME
- XR_KHR_opengl_es_enable_SPEC_VERSION
- Extending XrStructureType:
  - XR_TYPE_GRAPHICS_BINDING_OPENGL_ES_ANDROID_KHR
  - XR_TYPE_GRAPHICS_REQUIREMENTS_OPENGL_ES_KHR
  - XR_TYPE_SWAPCHAIN_IMAGE_OPENGL_ES_KHR

12.20.9. Version History

- Revision 1, 2018-05-07 (Mark Young)
  - Initial draft
- Revision 2, 2018-06-21 (Bryce Hutchings)
  - Add new xrGetOpenGLESGraphicsRequirementsKHR
- Revision 3, 2018-11-15 (Paul Pedriana)
  - Specified the swapchain texture coordinate origin.
- Revision 4, 2018-11-16 (Minmin Gong)
  - Specified Y direction and Z range in clip space
- Revision 5, 2019-01-25 (Robert Menzel)
  - Description updated
- Revision 6, 2019-07-12 (Martin Renschler)
  - Description updated
- Revision 7, 2020-08-06 (Bryce Hutchings)
  - Added new XR_ERROR_GRAPHICS_REQUIREMENTS_CALL_MISSING error code
- Revision 8, 2021-08-27 (Paulo F. Gomes)

    - Document handling of `XrSwapchainUsageFlags`
- Revision 9, 2025-03-07 (Rylie Pavlik, Collabora, Ltd.)
    - Re-organize, clarify, and make more uniform with other graphics binding extensions, and describe known design quirk.
- Revision 10, 2025-03-21 (Aaron Leiby)
    - Removed clip space specification

12.21. XR_KHR_swapchain_usage_input_attachment_bit

Name String

`XR_KHR_swapchain_usage_input_attachment_bit`

Extension Type

Instance extension

Registered Extension Number

166

Revision

3

Ratification Status

Ratified

Extension and Version Dependencies

OpenXR 1.0

Last Modified Date

2021-05-11

IP Status

No known IP claims.

Contributors

Jakob Bornecrantz, Collabora
Rylie Pavlik, Collabora

Overview

This extension enables an application to specify that swapchain images should be created in a way so that they can be used as input attachments. At the time of writing this bit only affects Vulkan swapchains.

New Object Types

New Flag Types

New Enum Constants

XrSwapchainUsageFlagBits enumeration is extended with:

- `XR_SWAPCHAIN_USAGE_INPUT_ATTACHMENT_BIT_KHR` - indicates that the image format **may** be used as an input attachment.

New Enums

New Structures

New Functions

Issues

Version History

- Revision 1, 2020-07-23 (Jakob Bornecrantz)
  - Initial draft
- Revision 2, 2020-07-24 (Jakob Bornecrantz)
  - Added note about only affecting Vulkan
  - Changed from MNDX to MND
- Revision 3, 2021-05-11 (Rylie Pavlik, Collabora, Ltd.)
  - Updated for promotion from MND to KHR

12.22. XR_KHR_visibility_mask

Name String

`XR_KHR_visibility_mask`

Extension Type

Instance extension

Registered Extension Number

32

Revision

2

Ratification Status

Ratified

Extension and Version Dependencies

OpenXR 1.0

Last Modified Date

2018-07-05

IP Status

No known IP claims.

Contributors

Paul Pedriana, Oculus
Alex Turner, Microsoft

Contacts

Paul Pedriana, Oculus

Overview

This extension support the providing of a per-view drawing mask for applications. The primary purpose of this is to enable performance improvements that result from avoiding drawing on areas that are not visible to the user. A common occurrence in head-mounted VR hardware is that the optical system's frustum does not intersect precisely with the rectangular display it is viewing. As a result, it may be that there are parts of the display that are not visible to the user, such as the corners of the display. In such cases it would be unnecessary for the application to draw into those parts.

New Object Types

New Flag Types

New Enum Constants

New Enums

XrVisibilityMaskTypeKHR identifies the different types of mask specification that is supported. The application **can** request a view mask in any of the formats identified by these types.

```
// Provided by XR_KHR_visibility_mask
typedef enum XrVisibilityMaskTypeKHR {
    XR_VISIBILITY_MASK_TYPE_HIDDEN_TRIANGLE_MESH_KHR = 1,
    XR_VISIBILITY_MASK_TYPE_VISIBLE_TRIANGLE_MESH_KHR = 2,
    XR_VISIBILITY_MASK_TYPE_LINE_LOOP_KHR = 3,
    XR_VISIBILITY_MASK_TYPE_MAX_ENUM_KHR = 0x7FFFFFFF
} XrVisibilityMaskTypeKHR;
```

Enumerant Descriptions

- XR_VISIBILITY_MASK_TYPE_HIDDEN_TRIANGLE_MESH_KHR refers to a two dimensional triangle mesh on the view surface which **should** not be drawn to by the application. XrVisibilityMaskKHR refers to a set of triangles identified by vertices and vertex indices. The index count will thus be a multiple of three. The triangle vertices will be returned in counter-clockwise order as viewed from the user perspective.
- XR_VISIBILITY_MASK_TYPE_VISIBLE_TRIANGLE_MESH_KHR refers to a two dimensional triangle mesh on the view surface which **should** be drawn to by the application. XrVisibilityMaskKHR refers to a set of triangles identified by vertices and vertex indices. The index count will thus be a multiple of three. The triangle vertices will be returned in counter-clockwise order as viewed from the user perspective.
- XR_VISIBILITY_MASK_TYPE_LINE_LOOP_KHR refers to a single multi-segmented line loop on the view surface which encompasses the view area which **should** be drawn by the application. It is the border that exists between the visible and hidden meshes identified by XR_VISIBILITY_MASK_TYPE_HIDDEN_TRIANGLE_MESH_KHR and XR_VISIBILITY_MASK_TYPE_VISIBLE_TRIANGLE_MESH_KHR. The line is counter-clockwise, contiguous, and non-self crossing, with the last point implicitly connecting to the first point. There is one vertex per point, the index count will equal the vertex count, and the indices will refer to the vertices.

New Structures

The XrVisibilityMaskKHR structure is an input/output struct which specifies the view mask.

```
// Provided by XR_KHR_visibility_mask
typedef struct XrVisibilityMaskKHR {
    XrStructureType    type;
    void*              next;
    uint32_t           vertexCapacityInput;
    uint32_t           vertexCountOutput;
    XrVector2f*        vertices;
    uint32_t           indexCapacityInput;
    uint32_t           indexCountOutput;
    uint32_t*          indices;
} XrVisibilityMaskKHR;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `vertexCapacityInput` is the capacity of the `vertices` array, or `0` to indicate a request to retrieve the required capacity.
- `vertexCountOutput` is filled in by the runtime with the count of vertices written or the required capacity in the case that `vertexCapacityInput` or `indexCapacityInput` is insufficient.
- `vertices` is an array of vertices filled in by the runtime that specifies mask coordinates in the z=-1 plane of the rendered view—i.e. one meter in front of the view. When rendering the mask for use in a projection layer, these vertices must be transformed by the application's projection matrix used for the respective XrCompositionLayerProjectionView.
- `indexCapacityInput` is the capacity of the `indices` array, or `0` to indicate a request to retrieve the required capacity.
- `indexCountOutput` is filled in by the runtime with the count of indices written or the required capacity in the case that `vertexCapacityInput` or `indexCapacityInput` is insufficient.
- `indices` is an array of indices filled in by the runtime, specifying the indices of the mask geometry in the `vertices` array.

Valid Usage (Implicit)

- The `XR_KHR_visibility_mask` extension **must** be enabled prior to using XrVisibilityMaskKHR
- `type` **must** be `XR_TYPE_VISIBILITY_MASK_KHR`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- If `vertexCapacityInput` is not `0`, `vertices` **must** be a pointer to an array of `vertexCapacityInput` XrVector2f structures
- If `indexCapacityInput` is not `0`, `indices` **must** be a pointer to an array of `indexCapacityInput` `uint32_t` values

The XrEventDataVisibilityMaskChangedKHR structure is defined as:

```
// Provided by XR_KHR_visibility_mask
typedef struct XrEventDataVisibilityMaskChangedKHR {
    XrStructureType            type;
    const void*                next;
    XrSession                  session;
    XrViewConfigurationType    viewConfigurationType;
    uint32_t                   viewIndex;
} XrEventDataVisibilityMaskChangedKHR;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `session` is the XrSession for which the view mask has changed.
- `viewConfigurationType` is the view configuration whose mask has changed.
- `viewIndex` is the individual view within the view configuration to which the change refers.

The XrEventDataVisibilityMaskChangedKHR structure is queued to indicate that a given visibility mask has changed. The application **should** respond to the event by calling xrGetVisibilityMaskKHR to retrieve the updated mask. This event is per-view, so if the masks for multiple views in a configuration change then multiple instances of this event will be queued for the application, one per view.

Valid Usage (Implicit)

- The `XR_KHR_visibility_mask` extension **must** be enabled prior to using XrEventDataVisibilityMaskChangedKHR
- `type` **must** be `XR_TYPE_EVENT_DATA_VISIBILITY_MASK_CHANGED_KHR`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

New Functions

The xrGetVisibilityMaskKHR function is defined as:

```
// Provided by XR_KHR_visibility_mask
XrResult xrGetVisibilityMaskKHR(
    XrSession                                   session,
    XrViewConfigurationType                     viewConfigurationType,
    uint32_t                                    viewIndex,
    XrVisibilityMaskTypeKHR                     visibilityMaskType,
    XrVisibilityMaskKHR*                        visibilityMask);
```

Parameter Descriptions

- `session` is an XrSession handle previously created with xrCreateSession.
- `viewConfigurationType` is the view configuration from which to retrieve mask information.
- `viewIndex` is the individual view within the view configuration from which to retrieve mask information.
- `visibilityMaskType` is the type of visibility mask requested.
- `visibilityMask` is an input/output struct which specifies the view mask.

xrGetVisibilityMaskKHR retrieves the view mask for a given view. This function follows the two-call idiom for filling multiple buffers in a struct. Specifically, if either XrVisibilityMaskKHR::`vertexCapacityInput` or XrVisibilityMaskKHR::`indexCapacityInput` is `0`, the runtime **must** respond as if both fields were set to `0`, returning the vertex count and index count through XrVisibilityMaskKHR::`vertexCountOutput` or XrVisibilityMaskKHR::`indexCountOutput` respectively. If a view mask for the specified view isn't available, the returned vertex and index counts **must** be `0`.

Valid Usage (Implicit)

- The `XR_KHR_visibility_mask` extension **must** be enabled prior to calling xrGetVisibilityMaskKHR
- `session` **must** be a valid XrSession handle
- `viewConfigurationType` **must** be a valid XrViewConfigurationType value
- `visibilityMaskType` **must** be a valid XrVisibilityMaskTypeKHR value
- `visibilityMask` **must** be a pointer to an XrVisibilityMaskKHR structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_SIZE_INSUFFICIENT
- XR_ERROR_VALIDATION_FAILURE
- XR_ERROR_VIEW_CONFIGURATION_TYPE_UNSUPPORTED

Issues

Version History

- Revision 1, 2018-07-05 (Paul Pedriana)
  - Initial version.
- Revision 2, 2019-07-15 (Alex Turner)
  - Adjust two-call idiom usage.

12.23. XR_KHR_vulkan_enable

Name String

XR_KHR_vulkan_enable

Extension Type

Instance extension

Registered Extension Number

26

Revision

10

Ratification Status

Ratified

Extension and Version Dependencies

OpenXR 1.0

Last Modified Date

2025-03-21

IP Status

No known IP claims.

Contributors

Mark Young, LunarG
Paul Pedriana, Oculus
Ed Hutchins, Oculus
Andres Rodriguez, Valve
Dan Ginsburg, Valve
Bryce Hutchings, Microsoft
Minmin Gong, Microsoft
Robert Menzel, NVIDIA
Paulo Gomes, Samsung Electronics
Aaron Leiby, Valve

12.23.1. Overview

This extension enables the use of the Vulkan graphics API in an OpenXR application. Without this extension, an OpenXR application **may** not be able to use any Vulkan swapchain images.

This extension provides the mechanisms necessary for an application to generate a valid XrGraphicsBindingVulkanKHR structure in order to create a Vulkan-based XrSession. Note that during this process the application is responsible for creating all the required Vulkan objects. However, the runtime provides the Vulkan images to render into. This extension provides mechanisms for the application to interact with those images by calling by calling xrEnumerateSwapchainImages.

In order to expose the structures, types, and functions of this extension, the application source code **must** define XR_USE_GRAPHICS_API_VULKAN before including the OpenXR platform header `openxr_platform.h`, in all portions of the library or application that interact with the types, values, and functions it defines.

12.23.2. Concurrency

Vulkan requires that concurrent access to a `VkQueue` from multiple threads be externally synchronized. Therefore, OpenXR functions that may access the `VkQueue` specified in the XrGraphicsBindingVulkanKHR must also be externally synchronized.

The list of OpenXR functions where the OpenXR runtime **may** access the VkQueue are:

- xrBeginFrame
- xrEndFrame
- xrAcquireSwapchainImage
- xrReleaseSwapchainImage

The runtime **must** not access the VkQueue in any OpenXR function that is not listed above or in an extension definition.

12.23.3. Initialization

Some of the requirements for creating a valid XrGraphicsBindingVulkanKHR include correct initialization of a VkInstance, VkPhysicalDevice, and VkDevice.

A runtime **may** require that the VkInstance be initialized to a specific Vulkan API version. Additionally, the runtime **may** require a set of instance extensions to be enabled in the VkInstance. These requirements can be queried by the application using xrGetVulkanGraphicsRequirementsKHR and xrGetVulkanInstanceExtensionsKHR, respectively.

Similarly, the runtime **may** require the VkDevice to have a set of device extensions enabled, which can be queried using xrGetVulkanDeviceExtensionsKHR.

In order to satisfy the VkPhysicalDevice requirements, the application can query xrGetVulkanGraphicsDeviceKHR to identify the correct VkPhysicalDevice.

Populating an XrGraphicsBindingVulkanKHR with a VkInstance, VkDevice, or VkPhysicalDevice that does not meet the requirements outlined by this extension **may** result in undefined behavior by the OpenXR runtime.

The API version, instance extension, device extension and physical device requirements only apply to the VkInstance, VkDevice, and VkPhysicalDevice objects which the application wishes to associate with an XrGraphicsBindingVulkanKHR.

The xrGetVulkanGraphicsRequirementsKHR function is defined as:

```
// Provided by XR_KHR_vulkan_enable
XrResult xrGetVulkanGraphicsRequirementsKHR(
    XrInstance                                  instance,
    XrSystemId                                  systemId,
    XrGraphicsRequirementsVulkanKHR*            graphicsRequirements);
```

Parameter Descriptions

- `instance` is an XrInstance handle previously created with xrCreateInstance.
- `systemId` is an `XrSystemId` handle for the system which will be used to create a session.
- `graphicsRequirements` is the XrGraphicsRequirementsVulkanKHR output structure.

The xrGetVulkanGraphicsRequirementsKHR function identifies to the application the minimum Vulkan version requirement and the highest known tested Vulkan version. The runtime **must** return `XR_ERROR_GRAPHICS_REQUIREMENTS_CALL_MISSING` (`XR_ERROR_VALIDATION_FAILURE` **may** be returned due to legacy behavior) on calls to xrCreateSession if xrGetVulkanGraphicsRequirementsKHR has not been called for the same `instance` and `systemId`.

Valid Usage (Implicit)

- The `XR_KHR_vulkan_enable` extension **must** be enabled prior to calling xrGetVulkanGraphicsRequirementsKHR
- `instance` **must** be a valid XrInstance handle
- `graphicsRequirements` **must** be a pointer to an XrGraphicsRequirementsVulkanKHR structure

Return Codes

Success

- `XR_SUCCESS`

Failure

- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SYSTEM_INVALID`
- `XR_ERROR_VALIDATION_FAILURE`

The XrGraphicsRequirementsVulkanKHR structure is defined as:

```
// Provided by XR_KHR_vulkan_enable
typedef struct XrGraphicsRequirementsVulkanKHR {
    XrStructureType    type;
    void*              next;
    XrVersion          minApiVersionSupported;
    XrVersion          maxApiVersionSupported;
} XrGraphicsRequirementsVulkanKHR;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `minApiVersionSupported` is the minimum Vulkan Instance API version that the runtime supports. Uses XR_MAKE_VERSION on major and minor API version, ignoring any patch version component.
- `maxApiVersionSupported` is the maximum Vulkan Instance API version that the runtime has been tested on and is known to support. Newer Vulkan Instance API versions might work if they are compatible. Uses XR_MAKE_VERSION on major and minor API version, ignoring any patch version component.

XrGraphicsRequirementsVulkanKHR is populated by xrGetVulkanGraphicsRequirementsKHR with the runtime's Vulkan API version requirements.

Valid Usage (Implicit)

- The `XR_KHR_vulkan_enable` extension **must** be enabled prior to using XrGraphicsRequirementsVulkanKHR
- `type` **must** be `XR_TYPE_GRAPHICS_REQUIREMENTS_VULKAN_KHR`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

Some computer systems have multiple graphics devices, each of which **may** have independent external display outputs. XR systems that connect to such graphics devices are typically connected to a single device. Applications need to know what graphics device the XR system is connected to so that they can use that graphics device to generate XR images.

To identify what graphics device needs to be used for an instance and system, call:

```
// Provided by XR_KHR_vulkan_enable
XrResult xrGetVulkanGraphicsDeviceKHR(
    XrInstance                                  instance,
    XrSystemId                                  systemId,
    VkInstance                                  vkInstance,
    VkPhysicalDevice*                           vkPhysicalDevice);
```

Parameter Descriptions

- `instance` is an XrInstance handle previously created with xrCreateInstance.
- `systemId` is an `XrSystemId` handle for the system which will be used to create a session.
- `vkInstance` is a valid Vulkan `VkInstance`.
- `vkPhysicalDevice` is a pointer to a `VkPhysicalDevice` value to populate.

xrGetVulkanGraphicsDeviceKHR function identifies to the application what graphics device (Vulkan `VkPhysicalDevice`) needs to be used. xrGetVulkanGraphicsDeviceKHR **must** be called prior to calling xrCreateSession, and the `VkPhysicalDevice` that xrGetVulkanGraphicsDeviceKHR returns should be passed to xrCreateSession in the XrGraphicsBindingVulkanKHR.

Valid Usage (Implicit)

- The `XR_KHR_vulkan_enable` extension **must** be enabled prior to calling xrGetVulkanGraphicsDeviceKHR
- `instance` **must** be a valid XrInstance handle
- `vkInstance` **must** be a valid `VkInstance` value
- `vkPhysicalDevice` **must** be a pointer to a `VkPhysicalDevice` value

Return Codes

Success

- XR_SUCCESS

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SYSTEM_INVALID
- XR_ERROR_VALIDATION_FAILURE

```
// Provided by XR_KHR_vulkan_enable
XrResult xrGetVulkanInstanceExtensionsKHR(
    XrInstance                                  instance,
    XrSystemId                                  systemId,
    uint32_t                                    bufferCapacityInput,
    uint32_t*                                   bufferCountOutput,
    char*                                       buffer);
```

Parameter Descriptions

- instance is an XrInstance handle previously created with xrCreateInstance.
- systemId is an XrSystemId handle for the system which will be used to create a session.
- bufferCapacityInput is the capacity of the buffer, or 0 to indicate a request to retrieve the required capacity.
- bufferCountOutput is a pointer to the count of characters written (including terminating \0), or a pointer to the required capacity in the case that bufferCapacityInput is insufficient.
- buffer is a pointer to an array of characters, but **can** be NULL if bufferCapacityInput is 0. The format of the output is a single space (ASCII 0x20) delimited string of extension names.
- See the Buffer Size Parameters section for a detailed description of retrieving the required buffer size.

Valid Usage (Implicit)

- The XR_KHR_vulkan_enable extension **must** be enabled prior to calling xrGetVulkanInstanceExtensionsKHR
- `instance` **must** be a valid XrInstance handle
- `bufferCountOutput` **must** be a pointer to a `uint32_t` value
- If `bufferCapacityInput` is not `0`, `buffer` **must** be a pointer to an array of `bufferCapacityInput` char values

Return Codes

Success

- XR_SUCCESS

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SIZE_INSUFFICIENT
- XR_ERROR_SYSTEM_INVALID
- XR_ERROR_VALIDATION_FAILURE

```
// Provided by XR_KHR_vulkan_enable
XrResult xrGetVulkanDeviceExtensionsKHR(
    XrInstance                                  instance,
    XrSystemId                                  systemId,
    uint32_t                                    bufferCapacityInput,
    uint32_t*                                   bufferCountOutput,
    char*                                       buffer);
```

Parameter Descriptions

- `instance` is an XrInstance handle previously created with xrCreateInstance.
- `systemId` is an `XrSystemId` handle for the system which will be used to create a session.
- `bufferCapacityInput` is the capacity of the `buffer`, or 0 to indicate a request to retrieve the required capacity.
- `bufferCountOutput` is a pointer to the count of characters written (including terminating `\0`), or a pointer to the required capacity in the case that `bufferCapacityInput` is insufficient.
- `buffer` is a pointer to an array of characters, but **can** be `NULL` if `bufferCapacityInput` is 0. The format of the output is a single space (ASCII `0x20`) delimited string of extension names.
- See the Buffer Size Parameters section for a detailed description of retrieving the required `buffer` size.

Valid Usage (Implicit)

- The `XR_KHR_vulkan_enable` extension **must** be enabled prior to calling xrGetVulkanDeviceExtensionsKHR
- `instance` **must** be a valid XrInstance handle
- `bufferCountOutput` **must** be a pointer to a `uint32_t` value
- If `bufferCapacityInput` is not `0`, `buffer` **must** be a pointer to an array of `bufferCapacityInput` char values

Return Codes

Success

- `XR_SUCCESS`

Failure

- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SIZE_INSUFFICIENT`
- `XR_ERROR_SYSTEM_INVALID`
- `XR_ERROR_VALIDATION_FAILURE`

12.23.4. Graphics Binding Structure

The XrGraphicsBindingVulkanKHR structure is defined as:

```
// Provided by XR_KHR_vulkan_enable
typedef struct XrGraphicsBindingVulkanKHR {
    XrStructureType     type;
    const void*         next;
    VkInstance          instance;
    VkPhysicalDevice    physicalDevice;
    VkDevice            device;
    uint32_t            queueFamilyIndex;
    uint32_t            queueIndex;
} XrGraphicsBindingVulkanKHR;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `instance` is a valid Vulkan `VkInstance`.
- `physicalDevice` is a valid Vulkan `VkPhysicalDevice`.
- `device` is a valid Vulkan `VkDevice`.
- `queueFamilyIndex` is a valid queue family index on `device`.
- `queueIndex` is a valid queue index on `device` to be used for synchronization.

To create a Vulkan-backed XrSession, the application provides a pointer to an XrGraphicsBindingVulkanKHR structure in the XrSessionCreateInfo::`next` chain when calling xrCreateSession.

Valid Usage

- `instance` **must** have enabled a Vulkan API version in the range specified by XrGraphicsBindingVulkanKHR
- `instance` **must** have enabled all the instance extensions specified by xrGetVulkanInstanceExtensionsKHR
- `physicalDevice` `VkPhysicalDevice` **must** match the device specified by xrGetVulkanGraphicsDeviceKHR
- `device` **must** have enabled all the device extensions specified by xrGetVulkanDeviceExtensionsKHR

Creating a session using this structure triggers several requirements on the runtime regarding swapchain images. See the specification of XrSwapchainImageVulkanKHR for details. The application **must** externally synchronize the queue referred to by this structure according to Concurrency.

Valid Usage (Implicit)

- The `XR_KHR_vulkan_enable` extension **must** be enabled prior to using XrGraphicsBindingVulkanKHR
- `type` **must** be `XR_TYPE_GRAPHICS_BINDING_VULKAN_KHR`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `instance` **must** be a valid `VkInstance` value
- `physicalDevice` **must** be a valid `VkPhysicalDevice` value
- `device` **must** be a valid `VkDevice` value

12.23.5. Swapchain Images

The XrSwapchainImageVulkanKHR structure is defined as:

```
// Provided by XR_KHR_vulkan_enable
typedef struct XrSwapchainImageVulkanKHR {
    XrStructureType    type;
    void*              next;
    VkImage            image;
} XrSwapchainImageVulkanKHR;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `image` is a valid Vulkan `VkImage` to use.

If a given session was created with XrGraphicsBindingVulkanKHR, the following conditions apply.

- Calls to xrEnumerateSwapchainImages on an XrSwapchain in that session **must** return an array of XrSwapchainImageVulkanKHR structures.
- Whenever an OpenXR function accepts an XrSwapchainImageBaseHeader pointer as a parameter in that session, the runtime **must** also accept a pointer to an XrSwapchainImageVulkanKHR.

The OpenXR runtime **must** interpret the top-left corner of the swapchain image as the coordinate origin unless specified otherwise by extension functionality.

The OpenXR runtime **must** return a texture created in accordance with Vulkan Swapchain Flag Bits.

The OpenXR runtime **must** manage image resource state in accordance with Vulkan Swapchain Image Layout.

Valid Usage (Implicit)

- The `XR_KHR_vulkan_enable` extension **must** be enabled prior to using XrSwapchainImageVulkanKHR
- `type` **must** be `XR_TYPE_SWAPCHAIN_IMAGE_VULKAN_KHR`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain. See also: XrSwapchainImageFoveationVulkanFB

12.23.6. Vulkan Swapchain Flag Bits

All XrSwapchainUsageFlags values passed in a session created using XrGraphicsBindingVulkanKHR **must** be interpreted as follows by the runtime, so that the returned swapchain images used by the application may be used as if they were created with at least the specified `VkImageUsageFlagBits` or `VkImageCreateFlagBits` set.

XrSwapchainUsageFlagBits	Corresponding Vulkan flag bit
`XR_SWAPCHAIN_USAGE_COLOR_ATTACHMENT_BIT`	`VK_IMAGE_USAGE_COLOR_ATTACHMENT_BIT`
`XR_SWAPCHAIN_USAGE_DEPTH_STENCIL_ATTACHMENT_BIT`	`VK_IMAGE_USAGE_DEPTH_STENCIL_ATTACHMENT_BIT`
`XR_SWAPCHAIN_USAGE_UNORDERED_ACCESS_BIT`	`VK_IMAGE_USAGE_STORAGE_BIT`

XrSwapchainUsageFlagBits	Corresponding Vulkan flag bit
XR_SWAPCHAIN_USAGE_TRANSFER_SRC_BIT	VK_IMAGE_USAGE_TRANSFER_SRC_BIT
XR_SWAPCHAIN_USAGE_TRANSFER_DST_BIT	VK_IMAGE_USAGE_TRANSFER_DST_BIT
XR_SWAPCHAIN_USAGE_SAMPLED_BIT	VK_IMAGE_USAGE_SAMPLED_BIT
XR_SWAPCHAIN_USAGE_MUTABLE_FORMAT_BIT	VK_IMAGE_CREATE_MUTABLE_FORMAT_BIT
XR_SWAPCHAIN_USAGE_INPUT_ATTACHMENT_BIT_KHR (Added by XR_KHR_swapchain_usage_input_attachment_bit and only available when that extension is enabled)	VK_IMAGE_USAGE_INPUT_ATTACHMENT_BIT
XR_SWAPCHAIN_USAGE_INPUT_ATTACHMENT_BIT_MND (Added by the now deprecated XR_MND_swapchain_usage_input_attachment_bit extension and only available when that extension is enabled)	VK_IMAGE_USAGE_INPUT_ATTACHMENT_BIT

12.23.7. Vulkan Swapchain Image Layout

If an application waits on a swapchain image by calling xrWaitSwapchainImage in a session created using XrGraphicsBindingVulkanKHR, and that call returns XR_SUCCESS or XR_SESSION_LOSS_PENDING, then the OpenXR runtime **must** guarantee that the following conditions are true, keeping in mind that the runtime **must** not access the VkQueue in xrWaitSwapchainImage:

- The image has a memory layout compatible with VK_IMAGE_LAYOUT_COLOR_ATTACHMENT_OPTIMAL for color images, or VK_IMAGE_LAYOUT_DEPTH_STENCIL_ATTACHMENT_OPTIMAL for depth images.
- The VkQueue specified in XrGraphicsBindingVulkanKHR has ownership of the image.

When an application releases a swapchain image by calling xrReleaseSwapchainImage, in a session created using XrGraphicsBindingVulkanKHR, the OpenXR runtime **must** interpret the image as:

- Having a memory layout compatible with VK_IMAGE_LAYOUT_COLOR_ATTACHMENT_OPTIMAL for color images, or VK_IMAGE_LAYOUT_DEPTH_STENCIL_ATTACHMENT_OPTIMAL for depth images.
- Being owned by the VkQueue specified in XrGraphicsBindingVulkanKHR.

The application is responsible for transitioning the swapchain image back to the image layout and queue ownership that the OpenXR runtime requires. If the image is not in a layout compatible with the above specifications the runtime **may** exhibit undefined behavior.

12.23.8. New Commands

- xrGetVulkanDeviceExtensionsKHR
- xrGetVulkanGraphicsDeviceKHR
- xrGetVulkanGraphicsRequirementsKHR

- xrGetVulkanInstanceExtensionsKHR

12.23.9. New Structures

- XrGraphicsRequirementsVulkanKHR
- XrSwapchainImageVulkanKHR
- Extending XrSessionCreateInfo:
  - XrGraphicsBindingVulkanKHR

12.23.10. New Enum Constants

- XR_KHR_VULKAN_ENABLE_EXTENSION_NAME
- XR_KHR_vulkan_enable_SPEC_VERSION
- Extending XrStructureType:
  - XR_TYPE_GRAPHICS_BINDING_VULKAN_KHR
  - XR_TYPE_GRAPHICS_REQUIREMENTS_VULKAN_KHR
  - XR_TYPE_SWAPCHAIN_IMAGE_VULKAN_KHR

12.23.11. Version History

- Revision 1, 2018-05-07 (Mark Young)
  - Initial draft
- Revision 2, 2018-06-21 (Bryce Hutchings)
  - Replace `session` parameter with `instance` and `systemId` parameters.
  - Move `xrGetVulkanDeviceExtensionsKHR`, `xrGetVulkanInstanceExtensionsKHR` and `xrGetVulkanGraphicsDeviceKHR` functions into this extension
  - Add new `XrGraphicsRequirementsVulkanKHR` function.
- Revision 3, 2018-11-15 (Paul Pedriana)
  - Specified the swapchain texture coordinate origin.
- Revision 4, 2018-11-16 (Minmin Gong)
  - Specified Y direction and Z range in clip space
- Revision 5, 2019-01-24 (Robert Menzel)
  - Description updated
- Revision 6, 2019-01-25 (Andres Rodriguez)
  - Reword sections of the spec to shift requirements on to the runtime instead of the app
- Revision 7, 2020-08-06 (Bryce Hutchings)

    - Added new `XR_ERROR_GRAPHICS_REQUIREMENTS_CALL_MISSING` error code
- Revision 8, 2021-01-21 (Rylie Pavlik, Collabora, Ltd.)
    - Document mapping for `XrSwapchainUsageFlags`
- Revision 9, 2025-03-07 (Rylie Pavlik, Collabora, Ltd.)
    - Re-organize, clarify, and make more uniform with other graphics binding extensions.
- Revision 10, 2025-03-21 (Aaron Leiby)
    - Removed clip space specification

12.24. XR_KHR_vulkan_enable2

Name String

`XR_KHR_vulkan_enable2`

Extension Type

Instance extension

Registered Extension Number

91

Revision

4

Ratification Status

Ratified

Extension and Version Dependencies

OpenXR 1.0

Last Modified Date

2025-03-21

IP Status

No known IP claims.

Contributors

Mark Young, LunarG
Paul Pedriana, Oculus
Ed Hutchins, Oculus
Andres Rodriguez, Valve
Dan Ginsburg, Valve
Bryce Hutchings, Microsoft
Minmin Gong, Microsoft

Robert Menzel, NVIDIA
Paulo Gomes, Samsung Electronics
Aaron Leiby, Valve

12.24.1. Overview

This extension enables the use of the Vulkan graphics API in an OpenXR application. Without this extension, an OpenXR application **may** not be able to use any Vulkan swapchain images.

This extension provides the mechanisms necessary for an application to generate a valid XrGraphicsBindingVulkan2KHR structure in order to create a Vulkan-based XrSession.

This extension also provides mechanisms for the application to interact with images acquired by calling xrEnumerateSwapchainImages.

In order to expose the structures, types, and functions of this extension, the application source code **must** define XR_USE_GRAPHICS_API_VULKAN before including the OpenXR platform header `openxr_platform.h`, in all portions of the library or application that interact with the types, values, and functions it defines.



Note

This extension is intended as an alternative to `XR_KHR_vulkan_enable`, and does not depend on it.

12.24.2. Initialization

When operating in Vulkan mode, the OpenXR runtime and the application will share the Vulkan queue described in the XrGraphicsBindingVulkan2KHR structure. This section of the document describes the mechanisms this extension exposes to ensure the shared Vulkan queue is compatible with the runtime and the application's requirements.

Vulkan Version Requirements

First, a compatible Vulkan version **must** be agreed upon. To query the runtime's Vulkan API version requirements an application will call:

```
// Provided by XR_KHR_vulkan_enable2
XrResult xrGetVulkanGraphicsRequirements2KHR(
    XrInstance                                  instance,
    XrSystemId                                  systemId,
    XrGraphicsRequirementsVulkanKHR*            graphicsRequirements);
```

The xrGetVulkanGraphicsRequirements2KHR function identifies to the application the runtime's minimum Vulkan version requirement and the highest known tested Vulkan version. xrGetVulkanGraphicsRequirements2KHR **must** be called prior to calling xrCreateSession. The runtime

must return XR_ERROR_GRAPHICS_REQUIREMENTS_CALL_MISSING on calls to xrCreateSession if xrGetVulkanGraphicsRequirements2KHR has not been called for the same instance and systemId.

Parameter Descriptions

- instance is an XrInstance handle previously created with xrCreateInstance.
- systemId is an XrSystemId handle for the system which will be used to create a session.
- graphicsRequirements is the XrGraphicsRequirementsVulkan2KHR output structure.

Valid Usage (Implicit)

- The XR_KHR_vulkan_enable2 extension **must** be enabled prior to calling xrGetVulkanGraphicsRequirements2KHR
- instance **must** be a valid XrInstance handle
- graphicsRequirements **must** be a pointer to an XrGraphicsRequirementsVulkanKHR structure

Return Codes

Success

- XR_SUCCESS

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SYSTEM_INVALID
- XR_ERROR_VALIDATION_FAILURE

The XrGraphicsRequirementsVulkan2KHR structure populated by xrGetVulkanGraphicsRequirements2KHR is defined as:

```
// Provided by XR_KHR_vulkan_enable2
// XrGraphicsRequirementsVulkan2KHR is an alias for XrGraphicsRequirementsVulkanKHR
typedef struct XrGraphicsRequirementsVulkanKHR {
    XrStructureType    type;
    void*              next;
    XrVersion          minApiVersionSupported;
    XrVersion          maxApiVersionSupported;
} XrGraphicsRequirementsVulkanKHR;

typedef XrGraphicsRequirementsVulkanKHR XrGraphicsRequirementsVulkan2KHR;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `minApiVersionSupported` is the minimum version of Vulkan that the runtime supports. Uses XR_MAKE_VERSION on major and minor API version, ignoring any patch version component.
- `maxApiVersionSupported` is the maximum version of Vulkan that the runtime has been tested on and is known to support. Newer Vulkan versions might work if they are compatible. Uses XR_MAKE_VERSION on major and minor API version, ignoring any patch version component.

Valid Usage (Implicit)

- The `XR_KHR_vulkan_enable2` extension **must** be enabled prior to using XrGraphicsRequirementsVulkan2KHR
- **Note:** XrGraphicsRequirementsVulkan2KHR is an alias for XrGraphicsRequirementsVulkanKHR, so the following items replicate the implicit valid usage for XrGraphicsRequirementsVulkanKHR
- `type` **must** be `XR_TYPE_GRAPHICS_REQUIREMENTS_VULKAN_KHR`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

Vulkan Instance Creation

Second, a compatible `VkInstance` **must** be created. The xrCreateVulkanInstanceKHR entry point is a wrapper around vkCreateInstance intended for this purpose. When called, the runtime **must** aggregate the requirements specified by the application with its own requirements and forward the `VkInstance` creation request to the `vkCreateInstance` function pointer returned by `pfnGetInstanceProcAddr`.

```
// Provided by XR_KHR_vulkan_enable2
XrResult xrCreateVulkanInstanceKHR(
    XrInstance                                  instance,
    const XrVulkanInstanceCreateInfoKHR*        createInfo,
    VkInstance*                                 vulkanInstance,
    VkResult*                                   vulkanResult);
```

Parameter Descriptions

- `instance` is an XrInstance handle previously created with xrCreateInstance.
- `createInfo` extensible input struct of type XrVulkanInstanceCreateInfoKHR
- `vulkanInstance` points to a `VkInstance` handle to populate with the new Vulkan instance.
- `vulkanResult` points to a `VkResult` to populate with the result of the `vkCreateInstance` operation as returned by XrVulkanInstanceCreateInfoKHR::`pfnGetInstanceProcAddr`.

Valid Usage (Implicit)

- The `XR_KHR_vulkan_enable2` extension **must** be enabled prior to calling xrCreateVulkanInstanceKHR
- `instance` **must** be a valid XrInstance handle
- `createInfo` **must** be a pointer to a valid XrVulkanInstanceCreateInfoKHR structure
- `vulkanInstance` **must** be a pointer to a `VkInstance` value
- `vulkanResult` **must** be a pointer to a `VkResult` value

Return Codes

Success

- XR_SUCCESS

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SYSTEM_INVALID
- XR_ERROR_VALIDATION_FAILURE

The XrVulkanInstanceCreateInfoKHR structure contains the input parameters to xrCreateVulkanInstanceKHR.

```
// Provided by XR_KHR_vulkan_enable2
typedef struct XrVulkanInstanceCreateInfoKHR {
    XrStructureType                 type;
    const void*                     next;
    XrSystemId                      systemId;
    XrVulkanInstanceCreateFlagsKHR  createFlags;
    PFN_vkGetInstanceProcAddr       pfnGetInstanceProcAddr;
    const VkInstanceCreateInfo*     vulkanCreateInfo;
    const VkAllocationCallbacks*    vulkanAllocator;
} XrVulkanInstanceCreateInfoKHR;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension
- `systemId` is an `XrSystemId` handle for the system which will be used to create a session.
- `createFlags` is a bitmask of XrVulkanInstanceCreateFlagBitsKHR
- `pfnGetInstanceProcAddr` is a function pointer to `vkGetInstanceProcAddr` or a compatible entry point.
- `vulkanCreateInfo` is the `VkInstanceCreateInfo` as specified by Vulkan.
- `vulkanAllocator` is the `VkAllocationCallbacks` as specified by Vulkan.

Valid Usage (Implicit)

- The `XR_KHR_vulkan_enable2` extension **must** be enabled prior to using XrVulkanInstanceCreateInfoKHR
- `type` **must** be `XR_TYPE_VULKAN_INSTANCE_CREATE_INFO_KHR`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `createFlags` **must** be `0`
- `pfnGetInstanceProcAddr` **must** be a valid `PFN_vkGetInstanceProcAddr` value
- `vulkanCreateInfo` **must** be a pointer to a valid `VkInstanceCreateInfo` value
- If `vulkanAllocator` is not `NULL`, `vulkanAllocator` **must** be a pointer to a valid `VkAllocationCallbacks` value

The XrVulkanInstanceCreateInfoKHR::`createFlags` member is of the following type, and contains a bitwise-OR of zero or more of the bits defined in XrVulkanInstanceCreateFlagBitsKHR.

```
// Provided by XR_KHR_vulkan_enable2
typedef XrFlags64 XrVulkanInstanceCreateFlagsKHR;
```

Valid bits for XrVulkanInstanceCreateFlagsKHR are defined by XrVulkanInstanceCreateFlagBitsKHR.

```
// Provided by XR_KHR_vulkan_enable2
// Flag bits for XrVulkanInstanceCreateFlagsKHR
```

There are currently no Vulkan instance creation flag bits defined. This is reserved for future use.

Physical Device Selection

Third, a VkPhysicalDevice **must** be chosen. Some computer systems may have multiple graphics devices, each of which may have independent external display outputs. The runtime **must** report a VkPhysicalDevice that is compatible with the OpenXR implementation when xrGetVulkanGraphicsDevice2KHR is invoked. The application will use this VkPhysicalDevice to interact with the OpenXR runtime.

```
// Provided by XR_KHR_vulkan_enable2
XrResult xrGetVulkanGraphicsDevice2KHR(
    XrInstance                                  instance,
    const XrVulkanGraphicsDeviceGetInfoKHR*     getInfo,
    VkPhysicalDevice*                           vulkanPhysicalDevice);
```

Parameter Descriptions

- instance is an XrInstance handle previously created with xrCreateInstance.
- getInfo extensible input struct of type XrVulkanGraphicsDeviceGetInfoKHR
- vulkanPhysicalDevice is a pointer to a VkPhysicalDevice handle to populate.

Valid Usage (Implicit)

- The XR_KHR_vulkan_enable2 extension **must** be enabled prior to calling xrGetVulkanGraphicsDevice2KHR
- instance **must** be a valid XrInstance handle
- getInfo **must** be a pointer to a valid XrVulkanGraphicsDeviceGetInfoKHR structure
- vulkanPhysicalDevice **must** be a pointer to a VkPhysicalDevice value

Return Codes

Success

- XR_SUCCESS

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SYSTEM_INVALID
- XR_ERROR_VALIDATION_FAILURE

The XrVulkanGraphicsDeviceGetInfoKHR structure contains the input parameters to xrCreateVulkanInstanceKHR.

```
// Provided by XR_KHR_vulkan_enable2
typedef struct XrVulkanGraphicsDeviceGetInfoKHR {
    XrStructureType    type;
    const void*        next;
    XrSystemId         systemId;
    VkInstance         vulkanInstance;
} XrVulkanGraphicsDeviceGetInfoKHR;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain.
- systemId is an XrSystemId handle for the system which will be used to create a session.
- vulkanInstance is a valid Vulkan VkInstance.

Valid Usage (Implicit)

- The XR_KHR_vulkan_enable2 extension **must** be enabled prior to using XrVulkanGraphicsDeviceGetInfoKHR
- type **must** be XR_TYPE_VULKAN_GRAPHICS_DEVICE_GET_INFO_KHR
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- vulkanInstance **must** be a valid VkInstance value

Vulkan Device Creation

Fourth, a compatible VkDevice **must** be created. The xrCreateVulkanDeviceKHR entry point is a wrapper around vkCreateDevice intended for this purpose. When called, the runtime **must** aggregate the requirements specified by the application with its own requirements and forward the VkDevice creation request to the vkCreateDevice function pointer returned by XrVulkanInstanceCreateInfoKHR ::pfnGetInstanceProcAddr.

```
// Provided by XR_KHR_vulkan_enable2
XrResult xrCreateVulkanDeviceKHR(
    XrInstance                                  instance,
    const XrVulkanDeviceCreateInfoKHR*          createInfo,
    VkDevice*                                   vulkanDevice,
    VkResult*                                   vulkanResult);
```

Parameter Descriptions

- instance is an XrInstance handle previously created with xrCreateInstance.
- createInfo extensible input struct of type XrCreateVulkanDeviceCreateInfoKHR
- vulkanDevice points to a VkDevice handle to populate with the new Vulkan device.
- vulkanResult points to a VkResult to populate with the result of the vkCreateDevice operation as returned by XrVulkanInstanceCreateInfoKHR::pfnGetInstanceProcAddr.

Valid Usage (Implicit)

- The XR_KHR_vulkan_enable2 extension **must** be enabled prior to calling xrCreateVulkanDeviceKHR
- `instance` **must** be a valid XrInstance handle
- `createInfo` **must** be a pointer to a valid XrVulkanDeviceCreateInfoKHR structure
- `vulkanDevice` **must** be a pointer to a `VkDevice` value
- `vulkanResult` **must** be a pointer to a `VkResult` value

Return Codes

Success

- XR_SUCCESS

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SYSTEM_INVALID
- XR_ERROR_VALIDATION_FAILURE

The XrVulkanDeviceCreateInfoKHR structure contains the input parameters to xrCreateVulkanDeviceKHR.

```
// Provided by XR_KHR_vulkan_enable2
typedef struct XrVulkanDeviceCreateInfoKHR {
    XrStructureType                 type;
    const void*                     next;
    XrSystemId                      systemId;
    XrVulkanDeviceCreateFlagsKHR    createFlags;
    PFN_vkGetInstanceProcAddr       pfnGetInstanceProcAddr;
    VkPhysicalDevice                vulkanPhysicalDevice;
    const VkDeviceCreateInfo*       vulkanCreateInfo;
    const VkAllocationCallbacks*    vulkanAllocator;
} XrVulkanDeviceCreateInfoKHR;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `systemId` is an `XrSystemId` handle for the system which will be used to create a session.
- `createFlags` is a bitmask of XrVulkanDeviceCreateFlagBitsKHR
- `pfnGetInstanceProcAddr` is a function pointer to `vkGetInstanceProcAddr` or a compatible entry point.
- `vulkanPhysicalDevice` **must** match xrGetVulkanGraphicsDeviceKHR.
- `vulkanCreateInfo` is the `VkDeviceCreateInfo` as specified by Vulkan.
- `vulkanAllocator` is the `VkAllocationCallbacks` as specified by Vulkan.

If the `vulkanPhysicalDevice` parameter does not match the output of xrGetVulkanGraphicsDeviceKHR, then the runtime **must** return `XR_ERROR_HANDLE_INVALID`.

Valid Usage (Implicit)

- The XR_KHR_vulkan_enable2 extension **must** be enabled prior to using XrVulkanDeviceCreateInfoKHR
- type **must** be XR_TYPE_VULKAN_DEVICE_CREATE_INFO_KHR
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- createFlags **must** be 0
- pfnGetInstanceProcAddr **must** be a valid PFN_vkGetInstanceProcAddr value
- vulkanPhysicalDevice **must** be a valid VkPhysicalDevice value
- vulkanCreateInfo **must** be a pointer to a valid VkDeviceCreateInfo value
- If vulkanAllocator is not NULL, vulkanAllocator **must** be a pointer to a valid VkAllocationCallbacks value

XrVulkanDeviceCreateFlagsKHR specify details of device creation. The XrVulkanDeviceCreateInfoKHR::createFlags member is of the following type, and contains a bitwise-OR of zero or more of the bits defined in XrVulkanDeviceCreateFlagBitsKHR.

```
// Provided by XR_KHR_vulkan_enable2
typedef XrFlags64 XrVulkanDeviceCreateFlagsKHR;
```

Valid bits for XrVulkanDeviceCreateFlagsKHR are defined by XrVulkanDeviceCreateFlagBitsKHR.

```
// Provided by XR_KHR_vulkan_enable2
// Flag bits for XrVulkanDeviceCreateFlagsKHR
```

There are currently no Vulkan device creation flag bits defined. This is reserved for future use.

Queue Selection

Last, the application selects a VkQueue from the VkDevice that has the VK_QUEUE_GRAPHICS_BIT set.



Note

The runtime may schedule work on the VkQueue specified in the binding, or it may schedule work on any hardware queue in a foreign logical device.

Vulkan Graphics Binding

When creating a Vulkan-backed XrSession, the application will chain a pointer to an XrGraphicsBindingVulkan2KHR to the XrSessionCreateInfo parameter of xrCreateSession. With the

data collected in the previous sections, the application now has all the necessary information to populate an XrGraphicsBindingVulkan2KHR structure for session creation.

```
// Provided by XR_KHR_vulkan_enable2
// XrGraphicsBindingVulkan2KHR is an alias for XrGraphicsBindingVulkanKHR
typedef struct XrGraphicsBindingVulkanKHR {
    XrStructureType     type;
    const void*         next;
    VkInstance          instance;
    VkPhysicalDevice    physicalDevice;
    VkDevice            device;
    uint32_t            queueFamilyIndex;
    uint32_t            queueIndex;
} XrGraphicsBindingVulkanKHR;

typedef XrGraphicsBindingVulkanKHR XrGraphicsBindingVulkan2KHR;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `instance` is a valid Vulkan `VkInstance`.
- `physicalDevice` is a valid Vulkan `VkPhysicalDevice`.
- `device` is a valid Vulkan `VkDevice`.
- `queueFamilyIndex` is a valid queue family index on `device`.
- `queueIndex` is a valid queue index on `device` to be used for synchronization.

Valid Usage

- `instance` **must** have enabled a Vulkan API version in the range specified by xrGetVulkanGraphicsRequirements2KHR
- `instance` **must** have been created using xrCreateVulkanInstanceKHR
- `physicalDevice` `VkPhysicalDevice` **must** match the device specified by xrGetVulkanGraphicsDevice2KHR
- `device` **must** have been created using xrCreateVulkanDeviceKHR

Valid Usage (Implicit)

- The XR_KHR_vulkan_enable2 extension **must** be enabled prior to using XrGraphicsBindingVulkan2KHR
- **Note:** XrGraphicsBindingVulkan2KHR is an alias for XrGraphicsBindingVulkanKHR, so the following items replicate the implicit valid usage for XrGraphicsBindingVulkanKHR
- type **must** be XR_TYPE_GRAPHICS_BINDING_VULKAN_KHR
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- instance **must** be a valid VkInstance value
- physicalDevice **must** be a valid VkPhysicalDevice value
- device **must** be a valid VkDevice value

Populating an XrGraphicsBindingVulkan2KHR structure with a member that does not meet the requirements outlined by this extension **may** result in undefined behavior by the OpenXR runtime.

The requirements outlined in this extension only apply to the VkInstance, VkDevice, VkPhysicalDevice and VkQueue objects which the application wishes to associate with an XrGraphicsBindingVulkan2KHR.

12.24.3. Concurrency

Vulkan requires that concurrent access to a VkQueue from multiple threads be externally synchronized. Therefore, OpenXR functions that may access the VkQueue specified in the XrGraphicsBindingVulkan2KHR **must** also be externally synchronized by the OpenXR application.

The list of OpenXR functions where the OpenXR runtime **may** access the VkQueue are:

- xrBeginFrame
- xrEndFrame
- xrAcquireSwapchainImage
- xrReleaseSwapchainImage

The runtime **must** not access the VkQueue in any OpenXR function that is not listed above or in an extension definition.

Failure by the application to synchronize access to VkQueue **may** result in undefined behavior in the OpenXR runtime.

12.24.4. Swapchain Interactions

Swapchain Images

When an application interacts with XrSwapchainImageBaseHeader structures in a Vulkan-backed XrSession, the application can interpret these to be XrSwapchainImageVulkan2KHR structures. These are defined as:

```
// Provided by XR_KHR_vulkan_enable2
// XrSwapchainImageVulkan2KHR is an alias for XrSwapchainImageVulkanKHR
typedef struct XrSwapchainImageVulkanKHR {
    XrStructureType    type;
    void*              next;
    VkImage            image;
} XrSwapchainImageVulkanKHR;

typedef XrSwapchainImageVulkanKHR XrSwapchainImageVulkan2KHR;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `image` is a valid Vulkan `VkImage` to use.

If a given session was created with XrGraphicsBindingVulkan2KHR, the following conditions apply.

- Calls to xrEnumerateSwapchainImages on an XrSwapchain in that session **must** return an array of XrSwapchainImageVulkan2KHR structures.
- Whenever an OpenXR function accepts an XrSwapchainImageBaseHeader pointer as a parameter in that session, the runtime **must** also accept a pointer to an XrSwapchainImageVulkan2KHR.

The OpenXR runtime **must** interpret the top-left corner of the swapchain image as the coordinate origin unless specified otherwise by extension functionality.

Valid Usage (Implicit)

- The XR_KHR_vulkan_enable2 extension **must** be enabled prior to using XrSwapchainImageVulkan2KHR
- **Note:** XrSwapchainImageVulkan2KHR is an alias for XrSwapchainImageVulkanKHR, so the following items replicate the implicit valid usage for XrSwapchainImageVulkanKHR
- type **must** be XR_TYPE_SWAPCHAIN_IMAGE_VULKAN_KHR
- next **must** be NULL or a valid pointer to the next structure in a structure chain. See also: XrSwapchainImageFoveationVulkanFB

Swapchain Image Layout

If an application waits on a swapchain image by calling xrWaitSwapchainImage in a session created using XrGraphicsBindingVulkan2KHR, and that call returns XR_SUCCESS or XR_SESSION_LOSS_PENDING, then the OpenXR runtime **must** guarantee that the following conditions are true, keeping in mind that the runtime **must** not access the VkQueue in xrWaitSwapchainImage:

- The image has a memory layout compatible with VK_IMAGE_LAYOUT_COLOR_ATTACHMENT_OPTIMAL for color images, or VK_IMAGE_LAYOUT_DEPTH_STENCIL_ATTACHMENT_OPTIMAL for depth images.
- The VkQueue specified in XrGraphicsBindingVulkan2KHR has ownership of the image.

When an application releases a swapchain image by calling xrReleaseSwapchainImage, in a session created using XrGraphicsBindingVulkan2KHR, the OpenXR runtime **must** interpret the image as:

- Having a memory layout compatible with VK_IMAGE_LAYOUT_COLOR_ATTACHMENT_OPTIMAL for color images, or VK_IMAGE_LAYOUT_DEPTH_STENCIL_ATTACHMENT_OPTIMAL for depth images.
- Being owned by the VkQueue specified in XrGraphicsBindingVulkan2KHR.
- Being referenced by command buffers submitted to the VkQueue specified in XrGraphicsBindingVulkan2KHR which have not yet completed execution.

The application is responsible for transitioning the swapchain image back to the image layout and queue ownership that the OpenXR runtime requires. If the image is not in a layout compatible with the above specifications the runtime **may** exhibit undefined behavior.

Swapchain Flag Bits

All XrSwapchainUsageFlags values passed in a session created using XrGraphicsBindingVulkan2KHR **must** be interpreted as follows by the runtime, so that the returned swapchain images used by the application may be used as if they were created with at least the specified VkImageUsageFlagBits or VkImageCreateFlagBits set.

XrSwapchainUsageFlagBits	Corresponding Vulkan flag bit
XR_SWAPCHAIN_USAGE_COLOR_ATTACHMENT_BIT	VK_IMAGE_USAGE_COLOR_ATTACHMENT_BIT
XR_SWAPCHAIN_USAGE_DEPTH_STENCIL_ATTACHMENT_BIT	VK_IMAGE_USAGE_DEPTH_STENCIL_ATTACHMENT_BIT
XR_SWAPCHAIN_USAGE_UNORDERED_ACCESS_BIT	VK_IMAGE_USAGE_STORAGE_BIT
XR_SWAPCHAIN_USAGE_TRANSFER_SRC_BIT	VK_IMAGE_USAGE_TRANSFER_SRC_BIT
XR_SWAPCHAIN_USAGE_TRANSFER_DST_BIT	VK_IMAGE_USAGE_TRANSFER_DST_BIT
XR_SWAPCHAIN_USAGE_SAMPLED_BIT	VK_IMAGE_USAGE_SAMPLED_BIT
XR_SWAPCHAIN_USAGE_MUTABLE_FORMAT_BIT	VK_IMAGE_CREATE_MUTABLE_FORMAT_BIT
XR_SWAPCHAIN_USAGE_INPUT_ATTACHMENT_BIT_KHR (Added by XR_KHR_swapchain_usage_input_attachment_bit and only available when that extension is enabled)	VK_IMAGE_USAGE_INPUT_ATTACHMENT_BIT
XR_SWAPCHAIN_USAGE_INPUT_ATTACHMENT_BIT_MND (Added by the now deprecated XR_MND_swapchain_usage_input_attachment_bit extension and only available when that extension is enabled)	VK_IMAGE_USAGE_INPUT_ATTACHMENT_BIT

12.24.5. Appendix

Questions

1. Should the xrCreateVulkanDeviceKHR and xrCreateVulkanInstanceKHR functions have an output parameter that returns the combined list of parameters used to create the Vulkan device/instance?
    - No. If the application is interested in capturing this data it can set the pfnGetInstanceProcAddr parameter to a local callback that captures the relevant information.

Quick Reference

New Enum Constants

XrStructureType enumeration is extended with:

- XR_TYPE_GRAPHICS_REQUIREMENTS_VULKAN2_KHR (alias of XR_TYPE_GRAPHICS_REQUIREMENTS_VULKAN_KHR)
- XR_TYPE_GRAPHICS_BINDING_VULKAN2_KHR (alias of XR_TYPE_GRAPHICS_BINDING_VULKAN_KHR)
- XR_TYPE_SWAPCHAIN_IMAGE_VULKAN2_KHR (alias of XR_TYPE_SWAPCHAIN_IMAGE_VULKAN_KHR)

New Structures

- XrVulkanInstanceCreateInfoKHR
- XrVulkanDeviceCreateInfoKHR

- XrVulkanGraphicsDeviceGetInfoKHR
- XrGraphicsBindingVulkan2KHR (alias of XrGraphicsBindingVulkanKHR)
- XrSwapchainImageVulkan2KHR (alias of XrSwapchainImageVulkanKHR)
- XrGraphicsRequirementsVulkan2KHR (alias of XrGraphicsRequirementsVulkanKHR)

New Functions

- xrCreateVulkanInstanceKHR
- xrCreateVulkanDeviceKHR
- xrGetVulkanGraphicsDevice2KHR
- xrGetVulkanGraphicsRequirements2KHR

Version History

- Revision 1, 2020-05-04 (Andres Rodriguez)
  - Initial draft
- Revision 2, 2021-01-21 (Rylie Pavlik, Collabora, Ltd.)
  - Document mapping for `XrSwapchainUsageFlags`
- Revision 3, 2025-03-07 (Rylie Pavlik, Collabora, Ltd.)
  - Clarify and make more uniform with other graphics binding extensions.
- Revision 4, 2025-03-21 (Aaron Leiby)
  - Removed clip space specification

12.25. XR_KHR_vulkan_swapchain_format_list

Name String

`XR_KHR_vulkan_swapchain_format_list`

Extension Type

Instance extension

Registered Extension Number

15

Revision

5

Ratification Status

Ratified

Extension and Version Dependencies

OpenXR 1.0
and
XR_KHR_vulkan_enable
or
OpenXR 1.0
and
XR_KHR_vulkan_enable2

Last Modified Date

2024-11-13

IP Status

No known IP claims.

Contributors

Paul Pedriana, Oculus
Dan Ginsburg, Valve
Jakob Bornecrantz, NVIDIA

Overview

Vulkan has the VK_KHR_image_format_list extension which allows applications to tell the vkCreateImage function which formats the application intends to use when VK_IMAGE_CREATE_MUTABLE_FORMAT_BIT is specified. This OpenXR extension exposes that Vulkan extension to OpenXR applications. In the same way that a Vulkan-based application can pass a VkImageFormatListCreateInfo struct to the vkCreateImage function, an OpenXR application can pass an identically configured XrVulkanSwapchainFormatListCreateInfoKHR structure to xrCreateSwapchain.

Applications using this extension to specify more than one swapchain format must create OpenXR swapchains with the XR_SWAPCHAIN_USAGE_MUTABLE_FORMAT_BIT bit set.

Runtimes implementing this extension **must** support the XR_KHR_vulkan_enable or the XR_KHR_vulkan_enable2 extension. When an application enables and uses XR_KHR_vulkan_enable2 as the graphics binding extension, the runtime **must** add VK_KHR_image_format_list to the list of extensions enabled in xrCreateVulkanDeviceKHR.

New Object Types

New Flag Types

New Enum Constants

XrStructureType enumeration is extended with:

```
    XR_TYPE_VULKAN_SWAPCHAIN_FORMAT_LIST_CREATE_INFO_KHR
```

New Enums

New Structures

```
// Provided by XR_KHR_vulkan_swapchain_format_list
typedef struct XrVulkanSwapchainFormatListCreateInfoKHR {
    XrStructureType    type;
    const void*        next;
    uint32_t           viewFormatCount;
    const VkFormat*    viewFormats;
} XrVulkanSwapchainFormatListCreateInfoKHR;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `viewFormatCount` is the number of view formats passed in `viewFormats`.
- `viewFormats` is an array of `VkFormat`.

Valid Usage (Implicit)

- The `XR_KHR_vulkan_swapchain_format_list` extension **must** be enabled prior to using XrVulkanSwapchainFormatListCreateInfoKHR
- `type` **must** be `XR_TYPE_VULKAN_SWAPCHAIN_FORMAT_LIST_CREATE_INFO_KHR`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- If `viewFormatCount` is not `0`, `viewFormats` **must** be a pointer to an array of `viewFormatCount` valid `VkFormat` values

New Functions

Issues

Version History

- Revision 1, 2017-09-13 (Paul Pedriana)
  - Initial proposal.

- Revision 2, 2018-06-21 (Bryce Hutchings)
  - Update reference of `XR_KHR_vulkan_extension_requirements` to `XR_KHR_vulkan_enable`
- Revision 3, 2020-01-01 (Andres Rodriguez)
  - Update for `XR_KHR_vulkan_enable2`
- Revision 4, 2021-01-21 (Rylie Pavlik, Collabora, Ltd.)
  - Fix reference to the mutable-format bit in Vulkan.
- Revision 5, 2024-11-13 (Jakob Bornecrantz, NVIDIA)
  - Fix correct Vulkan enable extension being referenced.
  - Clarify `XR_KHR_vulkan_enable2` being used by the app.

12.26. XR_KHR_win32_convert_performance_counter_time

Name String

`XR_KHR_win32_convert_performance_counter_time`

Extension Type

Instance extension

Registered Extension Number

36

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

OpenXR 1.0

Last Modified Date

2019-01-24

IP Status

No known IP claims.

Contributors

Paul Pedriana, Oculus
Bryce Hutchings, Microsoft

Overview

This extension provides two functions for converting between the Windows performance counter (QPC) time stamps and XrTime. The xrConvertWin32PerformanceCounterToTimeKHR function converts from Windows performance counter time stamps to XrTime, while the xrConvertTimeToWin32PerformanceCounterKHR function converts XrTime to Windows performance counter time stamps. The primary use case for this functionality is to be able to synchronize events between the local system and the OpenXR system.

New Object Types

New Flag Types

New Enum Constants

New Enums

New Structures

New Functions

To convert from a Windows performance counter time stamp to XrTime, call:

```
// Provided by XR_KHR_win32_convert_performance_counter_time
XrResult xrConvertWin32PerformanceCounterToTimeKHR(
    XrInstance                                  instance,
    const LARGE_INTEGER*                        performanceCounter,
    XrTime*                                     time);
```

Parameter Descriptions

- instance is an XrInstance handle previously created with xrCreateInstance.
- performanceCounter is a time returned by QueryPerformanceCounter.
- time is the resulting XrTime that is equivalent to the performanceCounter.

The xrConvertWin32PerformanceCounterToTimeKHR function converts a time stamp obtained by the QueryPerformanceCounter Windows function to the equivalent XrTime.

If the output time cannot represent the input performanceCounter, the runtime **must** return XR_ERROR_TIME_INVALID.

Valid Usage (Implicit)

- The XR_KHR_win32_convert_performance_counter_time extension **must** be enabled prior to calling xrConvertWin32PerformanceCounterToTimeKHR
- instance **must** be a valid XrInstance handle
- performanceCounter **must** be a pointer to a valid LARGE_INTEGER value
- time **must** be a pointer to an XrTime value

Return Codes

Success

- XR_SUCCESS

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_TIME_INVALID
- XR_ERROR_VALIDATION_FAILURE

To convert from XrTime to a Windows performance counter time stamp, call:

```
// Provided by XR_KHR_win32_convert_performance_counter_time
XrResult xrConvertTimeToWin32PerformanceCounterKHR(
    XrInstance                                  instance,
    XrTime                                      time,
    LARGE_INTEGER*                              performanceCounter);
```

Parameter Descriptions

- instance is an XrInstance handle previously created with xrCreateInstance.
- time is an XrTime.
- performanceCounter is the resulting Windows performance counter time stamp that is equivalent to the time.

The xrConvertTimeToWin32PerformanceCounterKHR function converts an XrTime to time as if

generated by the QueryPerformanceCounter Windows function.

If the output performanceCounter cannot represent the input time, the runtime **must** return XR_ERROR_TIME_INVALID.

Valid Usage (Implicit)

- The XR_KHR_win32_convert_performance_counter_time extension **must** be enabled prior to calling xrConvertTimeToWin32PerformanceCounterKHR
- instance **must** be a valid XrInstance handle
- performanceCounter **must** be a pointer to a LARGE_INTEGER value

Return Codes

Success

- XR_SUCCESS

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_TIME_INVALID
- XR_ERROR_VALIDATION_FAILURE

Issues

Version History

- Revision 1, 2019-01-24 (Paul Pedriana)
  - Initial draft

12.27. XR_EXT_active_action_set_priority

Name String

XR_EXT_active_action_set_priority

Extension Type

Instance extension

Registered Extension Number

374

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Last Modified Date

2022-08-19

IP Status

No known IP claims.

Contributors

Jules Blok, Epic Games
Lachlan Ford, Microsoft

Overview

The properties of an XrActionSet become immutable after it has been attached to a session. This currently includes the priority of the action set preventing the application from changing the priority number for the duration of the session.

Given that most runtimes do not actually require this number to be immutable this extension adds the ability to provide a different priority number for every XrActiveActionSet provided to xrSyncActions.

When updating the action state with xrSyncActions, the application **can** provide a pointer to an XrActiveActionSetPrioritiesEXT structure in the `next` chain of XrActionsSyncInfo. This structure contains an array of XrActiveActionSetPriorityEXT structures mapping active action sets to their priority numbers.

New Object Types

New Flag Types

New Enum Constants

XrStructureType enumeration is extended with:

- `XR_TYPE_ACTIVE_ACTION_SET_PRIORITIES_EXT`

New Enums

New Structures

The XrActiveActionSetPrioritiesEXT structure is defined as:

```
// Provided by XR_EXT_active_action_set_priority
typedef struct XrActiveActionSetPrioritiesEXT {
    XrStructureType                         type;
    const void*                             next;
    uint32_t                                actionSetPriorityCount;
    const XrActiveActionSetPriorityEXT*     actionSetPriorities;
} XrActiveActionSetPrioritiesEXT;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `actionSetPriorityCount` is an integer specifying the number of valid elements in the actionSetPriorities array.
- `actionSetPriorities` is a pointer to an array that maps action sets to their active priority numbers. If an action set is specified multiple times, the runtime **may** return `XR_ERROR_VALIDATION_FAILURE` from xrSyncActions.

Valid Usage (Implicit)

- The `XR_EXT_active_action_set_priority` extension **must** be enabled prior to using XrActiveActionSetPrioritiesEXT
- `type` **must** be `XR_TYPE_ACTIVE_ACTION_SET_PRIORITIES_EXT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `actionSetPriorities` **must** be a pointer to an array of `actionSetPriorityCount` valid XrActiveActionSetPriorityEXT structures
- The `actionSetPriorityCount` parameter **must** be greater than `0`

The runtime **must** ignore any priority numbers for action sets that were not specified as an active action set in the XrActionsSyncInfo structure as this would have no effect.

The priority numbers provided in XrActiveActionSetPriorityEXT **must** override the priority number of the active action set starting with the xrSyncActions call it is provided to, until the first subsequent call to xrSyncActions.

When a subsequent call is made to xrSyncActions where an active action set does not have a corresponding priority number specified in the XrActiveActionSetPriorityEXT structure the priority number for that action set **must** revert back to the priority number provided in XrActionSetCreateInfo when that action set was created.

The XrActiveActionSetPriorityEXT structure is defined as:

```
// Provided by XR_EXT_active_action_set_priority
typedef struct XrActiveActionSetPriorityEXT {
    XrActionSet    actionSet;
    uint32_t       priorityOverride;
} XrActiveActionSetPriorityEXT;
```

Member Descriptions

- `actionSet` is the handle of the XrActionSet to set the priority number for.
- `priorityOverride` is an integer specifying the priority of the action set while it is active.

Valid Usage (Implicit)

- The `XR_EXT_active_action_set_priority` extension **must** be enabled prior to using XrActiveActionSetPriorityEXT
- `actionSet` **must** be a valid XrActionSet handle

New Functions

Issues

- Can the same action set have a different priority on each subaction path?
  - No. To avoid additional complexity each action set can only be specified once in the array of priorities which does not include the subaction path.

Version History

- Revision 1, 2022-08-19 (Jules Blok)
  - Initial proposal.

12.28. XR_EXT_composition_layer_inverted_alpha

Name String

`XR_EXT_composition_layer_inverted_alpha`

Extension Type

Instance extension

Registered Extension Number

555

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Last Modified Date

2023-12-06

IP Status

No known IP claims.

Contributors

Xiang Wei, Meta
Jian Zhang, ByteDance
Erica Stella, Epic Games

12.28.1. Overview

This extension provides a flag which, when applied to a composition layer, inverts the interpretation of the alpha value in the layer's swapchain images. With this bit set, an alpha value of 1.0 represents fully transparent, and an alpha value of 0.0 represents fully opaque. This extension is primarily intended to allow applications using inverted alpha internally to submit composition layers with inverted alpha. Doing so using this extension over more general alternatives may result in less runtime overhead.

12.28.2. Modifications to Composition Layer Blending

Specifically, this extension supersedes some operations in Composition Layer Blending section, when `XR_COMPOSITION_LAYER_INVERTED_ALPHA_BIT_EXT` is set.

- If a submitted swapchain's texture format does not include an alpha channel or if the `XR_COMPOSITION_LAYER_BLEND_TEXTURE_SOURCE_ALPHA_BIT` is unset, then the layer alpha is initialized to zero, instead of one.
- After the LayerColor is fully initialized, the alpha component of LayerColor will then be inverted by the following formula, just before it might be further altered by the `XR_COMPOSITION_LAYER_UNPREMULTIPLIED_ALPHA_BIT`.

```
LayerColor.A = 1.0 - LayerColor.A
```

When this extension is enabled and XR_COMPOSITION_LAYER_INVERTED_ALPHA_BIT_EXT is set, the inversion of LayerColor.A **must** happen before any other modification to the LayerColor, including the operations that the XR_KHR_composition_layer_color_scale_bias extension may introduce.

12.28.3. New Enum Constants

- XR_EXT_COMPOSITION_LAYER_INVERTED_ALPHA_EXTENSION_NAME
- XR_EXT_composition_layer_inverted_alpha_SPEC_VERSION
- Extending XrCompositionLayerFlagBits:
  - XR_COMPOSITION_LAYER_INVERTED_ALPHA_BIT_EXT

Version History

- Revision 1, 2023-12-06 (Xiang Wei)
  - Initial extension description

12.29. XR_EXT_conformance_automation

Name String

XR_EXT_conformance_automation

Extension Type

Instance extension

Registered Extension Number

48

Revision

3

Ratification Status

Ratified

Extension and Version Dependencies

OpenXR 1.0

Last Modified Date

2021-04-14

IP Status

No known IP claims.

Contributors

Lachlan Ford, Microsoft
Rylie Pavlik, Collabora

Overview

The XR_EXT_conformance_automation allows conformance test and runtime developers to provide hints to the underlying runtime as to what input the test is expecting. This enables runtime authors to automate the testing of their runtime conformance. This is useful for achieving rapidly iterative runtime development whilst maintaining conformance for runtime releases.

This extension provides the following capabilities:

- The ability to toggle the active state of an input device.
- The ability to set the state of an input device button or other input component.
- The ability to set the location of the input device.

Applications **may** call these functions at any time. The runtime **must** do its best to honor the request of applications calling these functions, however it does not guarantee that any state change will be reflected immediately, at all, or with the exact value that was requested. Applications are thus advised to wait for the state change to be observable and to not assume that the value they requested will be the value observed. If any of the functions of this extension are called, control over input **must** be removed from the physical hardware of the system.

Warning

This extension is **not** intended for use by non-conformance-test applications. A runtime **may** require a runtime-specified configuration such as a "developer mode" to be enabled before reporting support for this extension or providing a non-stub implementation of it.

Do not use this functionality in a non-conformance-test application!

New Object Types

New Flag Types

New Enum Constants

New Enums

New Structures

New Functions

```
// Provided by XR_EXT_conformance_automation
XrResult xrSetInputDeviceActiveEXT(
    XrSession                                   session,
    XrPath                                      interactionProfile,
    XrPath                                      topLevelPath,
    XrBool32                                    isActive);
```

Parameter Descriptions

- `session` is the XrSession to set the input device state in.
- `interactionProfile` is the path representing the interaction profile of the input device (e.g. */interaction_profiles/khr/simple_controller*).
- `topLevelPath` is the path representing the input device (e.g. */user/hand/left*).
- `isActive` is the requested activation state of the input device.

Valid Usage

- `session` **must** be a valid session handle.
- `topLevelPath` **must** be a valid top level path.

Valid Usage (Implicit)

- The `XR_EXT_conformance_automation` extension **must** be enabled prior to calling xrSetInputDeviceActiveEXT
- `session` **must** be a valid XrSession handle

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_PATH_INVALID
- XR_ERROR_PATH_UNSUPPORTED
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

```
// Provided by XR_EXT_conformance_automation
XrResult xrSetInputDeviceStateBoolEXT(
    XrSession                                   session,
    XrPath                                      topLevelPath,
    XrPath                                      inputSourcePath,
    XrBool32                                    state);
```

Parameter Descriptions

- session is the XrSession to set the input device state in.
- topLevelPath is the path representing the input device (e.g. */user/hand/left*).
- inputSourcePath is the full path of the input component for which we wish to set the state for (e.g. */user/hand/left/input/select/click*).
- state is the requested boolean state of the input device.

Valid Usage

- `session` **must** be a valid session handle.
- `topLevelPath` **must** be a valid top level path.
- `inputSourcePath` **must** be a valid input source path.

Valid Usage (Implicit)

- The `XR_EXT_conformance_automation` extension **must** be enabled prior to calling xrSetInputDeviceStateBoolEXT
- `session` **must** be a valid XrSession handle

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_PATH_INVALID`
- `XR_ERROR_PATH_UNSUPPORTED`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_VALIDATION_FAILURE`

```
// Provided by XR_EXT_conformance_automation
XrResult xrSetInputDeviceStateFloatEXT(
    XrSession                                   session,
    XrPath                                      topLevelPath,
    XrPath                                      inputSourcePath,
    float                                       state);
```

Parameter Descriptions

- `session` is the XrSession to set the input device state in.
- `topLevelPath` is the path representing the input device (e.g. */user/hand/left*).
- `inputSourcePath` is the full path of the input component for which we wish to set the state for (e.g. */user/hand/left/input/trigger/value*).
- `state` is the requested float state of the input device.

Valid Usage

- `session` **must** be a valid session handle.
- `topLevelPath` **must** be a valid top level path.
- `inputSourcePath` **must** be a valid input source path.

Valid Usage (Implicit)

- The `XR_EXT_conformance_automation` extension **must** be enabled prior to calling xrSetInputDeviceStateFloatEXT
- `session` **must** be a valid XrSession handle

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_PATH_INVALID
- XR_ERROR_PATH_UNSUPPORTED
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

```
// Provided by XR_EXT_conformance_automation
XrResult xrSetInputDeviceStateVector2fEXT(
    XrSession                                   session,
    XrPath                                      topLevelPath,
    XrPath                                      inputSourcePath,
    XrVector2f                                  state);
```

Parameter Descriptions

- session is the XrSession to set the input device state in.
- topLevelPath is the path representing the input device (e.g. */user/hand/left*).
- inputSourcePath is the full path of the input component for which we wish to set the state for (e.g. */user/hand/left/input/thumbstick*).
- state is the requested two-dimensional state of the input device.

Valid Usage

- `session` **must** be a valid session handle.
- `topLevelPath` **must** be a valid top level path.
- `inputSourcePath` **must** be a valid input source path.

Valid Usage (Implicit)

- The `XR_EXT_conformance_automation` extension **must** be enabled prior to calling xrSetInputDeviceStateVector2fEXT
- `session` **must** be a valid XrSession handle

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_PATH_INVALID`
- `XR_ERROR_PATH_UNSUPPORTED`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_VALIDATION_FAILURE`

```
// Provided by XR_EXT_conformance_automation
XrResult xrSetInputDeviceLocationEXT(
    XrSession                                   session,
    XrPath                                      topLevelPath,
    XrPath                                      inputSourcePath,
    XrSpace                                     space,
    XrPosef                                     pose);
```

Parameter Descriptions

- `session` is the XrSession to set the input device state in.
- `topLevelPath` is the path representing the input device (e.g. */user/hand/left*).
- `inputSourcePath` is the full path of the input component for which we wish to set the pose for (e.g. */user/hand/left/input/grip/pose*).
- `space` is the XrSpace for the `pose`
- `pose` is the requested pose state of the input device.

Valid Usage

- `session` **must** be a valid session handle.
- `topLevelPath` **must** be a valid top level path.
- `inputSourcePath` **must** be a valid input source path.
- `space` **must** be a valid XrSpace.
- `pose` **must** be a valid XrPosef.

Valid Usage (Implicit)

- The `XR_EXT_conformance_automation` extension **must** be enabled prior to calling xrSetInputDeviceLocationEXT
- `session` **must** be a valid XrSession handle
- `space` **must** be a valid XrSpace handle
- `space` **must** have been created, allocated, or retrieved from `session`

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_PATH_INVALID
- XR_ERROR_PATH_UNSUPPORTED
- XR_ERROR_POSE_INVALID
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

New Function Pointers

Issues

None

Version History

- Revision 1, 2019-10-01 (Lachlan Ford)
  - Initial draft
- Revision 2, 2021-03-04 (Rylie Pavlik)
  - Correct errors in function parameter documentation.
- Revision 3, 2021-04-14 (Rylie Pavlik)
  - Fix missing error code

12.30. XR_EXT_debug_utils

Name String

XR_EXT_debug_utils

Extension Type

Instance extension

Registered Extension Number

20

Revision

5

Ratification Status

Ratified

Extension and Version Dependencies

OpenXR 1.0

Last Modified Date

2021-04-14

IP Status

No known IP claims.

Contributors

Mark Young, LunarG
Karl Schultz, LunarG
Rylie Pavlik, Collabora

Overview

Due to the nature of the OpenXR interface, there is very little error information available to the developer and application. By using the `XR_EXT_debug_utils` extension, developers **can** obtain more information. When combined with validation layers, even more detailed feedback on the application's use of OpenXR will be provided.

This extension provides the following capabilities:

- The ability to create a debug messenger which will pass along debug messages to an application supplied callback.
- The ability to identify specific OpenXR handles using a name to improve tracking.

12.30.1. Object Debug Annotation

It can be useful for an application to provide its own content relative to a specific OpenXR handle.

Object Naming

xrSetDebugUtilsObjectNameEXT allows application developers to associate user-defined information with OpenXR handles.

This is useful when paired with the callback that you register when creating an XrDebugUtilsMessengerEXT object. When properly used, debug messages will contain not only the corresponding object handle, but the associated object name as well.

An application can change the name associated with an object simply by calling xrSetDebugUtilsObjectNameEXT again with a new string. If the objectName member of the XrDebugUtilsObjectNameInfoEXT structure is an empty string, then any previously set name is removed.

12.30.2. Debug Messengers

OpenXR allows an application to register arbitrary number of callbacks with all the OpenXR components wishing to report debug information. Some callbacks **can** log the information to a file, others **can** cause a debug break point or any other behavior defined by the application. A primary producer of callback messages are the validation layers. If the extension is enabled, an application **can** register callbacks even when no validation layers are enabled. The OpenXR loader, other layers, and runtimes **may** also produce callback messages.

The debug messenger will provide detailed feedback on the application's use of OpenXR when events of interest occur. When an event of interest does occur, the debug messenger will submit a debug message to the debug callback that was provided during its creation. Additionally, the debug messenger is responsible with filtering out debug messages that the callback isn't interested in and will only provide desired debug messages.

12.30.3. Debug Message Categorization

Messages that are triggered by the debug messenger are categorized by their message type and severity. Additionally, each message has a string value identifying its `messageId`. These 3 bits of information can be used to filter out messages so you only receive reports on the messages you desire. In fact, during debug messenger creation, the severity and type flag values are provided to indicate what messages should be allowed to trigger the user's callback.

Message Type

The message type indicates the general category the message falls under. Currently we have the following message types:

Table 4. XR_EXT_debug_utils Message Type Flag Descriptions

Enum	Description
XR_DEBUG_UTILS_MESSAGE_TYPE_GENERAL_BIT_EXT	Specifies a general purpose event type. This is typically a non-validation, non-performance event.
XR_DEBUG_UTILS_MESSAGE_TYPE_VALIDATION_BIT_EXT	Specifies an event caused during a validation against the OpenXR specification that **may** indicate invalid OpenXR usage.
XR_DEBUG_UTILS_MESSAGE_TYPE_PERFORMANCE_BIT_EXT	Specifies a potentially non-optimal use of OpenXR.
XR_DEBUG_UTILS_MESSAGE_TYPE_CONFORMANCE_BIT_EXT	Specifies a non-conformant OpenXR result. This is typically caused by a layer or runtime returning non-conformant data.

A message may correspond to more than one type. For example, if a validation warning also could impact performance, then the message might be identified with both the XR_DEBUG_UTILS_MESSAGE_TYPE_VALIDATION_BIT_EXT and XR_DEBUG_UTILS_MESSAGE_TYPE_PERFORMANCE_BIT_EXT flag bits.

Message Severity

The severity of a message is a flag that indicates how important the message is using standard logging naming. The severity flag bit values are shown in the following table.

Table 5. XR_EXT_debug_utils Message Severity Flag Descriptions

Enum	Description
XR_DEBUG_UTILS_MESSAGE_SEVERITY_VERBOSE_BIT_EXT	Specifies the most verbose output indicating all diagnostic messages from the OpenXR loader, layers, and drivers should be captured.
XR_DEBUG_UTILS_MESSAGE_SEVERITY_INFO_BIT_EXT	Specifies an informational message such as resource details that might be handy when debugging an application.
XR_DEBUG_UTILS_MESSAGE_SEVERITY_WARNING_BIT_EXT	Specifies use of OpenXR that could be an application bug. Such cases **may** not be immediately harmful, such as providing too many swapchain images. Other cases **may** point to behavior that is almost certainly bad when unintended, such as using a swapchain image whose memory has not been filled. In general, if you see a warning but you know that the behavior is intended/desired, then simply ignore the warning.

Enum	Description
XR_DEBUG_UTILS_MESSAGE_SEVERITY_ERROR_BIT_EXT	Specifies an error that **may** cause undefined behavior, including an application crash.



Note

The values of XrDebugUtilsMessageSeverityFlagBitsEXT are sorted based on severity. The higher the flag value, the more severe the message. This allows for simple boolean operation comparisons when looking at XrDebugUtilsMessageSeverityFlagBitsEXT values.

Message IDs

The XrDebugUtilsMessengerCallbackDataEXT structure contains a `messageId` that may be a string identifying the message ID for the triggering debug message. This may be blank, or it may simply contain the name of an OpenXR component (like "OpenXR Loader"). However, when certain API layers or runtimes are used, especially the OpenXR core_validation API layer, then this value is intended to uniquely identify the message generated. If a certain warning/error message constantly fires, a user can simply look at the unique ID in their callback handler and manually filter it out.

For validation layers, this `messageId` value actually can be used to find the section of the OpenXR specification that the layer believes to have been violated. See the core_validation API Layer documentation for more information on how this can be done.

12.30.4. Session Labels

All OpenXR work is performed inside of an XrSession. There are times that it helps to label areas in your OpenXR session to allow easier debugging. This can be especially true if your application creates more than one session. There are two kinds of labels provided in this extension:

- Region labels
- Individual labels

To begin identifying a region using a debug label inside a session, you may use the xrSessionBeginDebugUtilsLabelRegionEXT function. Calls to xrSessionBeginDebugUtilsLabelRegionEXT may be nested allowing you to identify smaller and smaller labeled regions within your code. Using this, you can build a "call-stack" of sorts with labels since any logging callback will contain the list of all active session label regions.

To end the last session label region that was begun, you **must** call xrSessionEndDebugUtilsLabelRegionEXT. Each xrSessionBeginDebugUtilsLabelRegionEXT **must** have a matching xrSessionEndDebugUtilsLabelRegionEXT. All of a session's label regions **must** be closed before the xrDestroySession function is called for the given XrSession.

An individual debug label may be inserted at any time using xrSessionInsertDebugUtilsLabelEXT. The

xrSessionInsertDebugUtilsLabelEXT is used to indicate a particular location within the execution of the application's session functions. The next call to xrSessionInsertDebugUtilsLabelEXT, xrSessionBeginDebugUtilsLabelRegionEXT, or xrSessionEndDebugUtilsLabelRegionEXT overrides this value.

New Object Types

```
// Provided by XR_EXT_debug_utils
XR_DEFINE_HANDLE(XrDebugUtilsMessengerEXT)
```

XrDebugUtilsMessengerEXT represents a callback function and associated filters registered with the runtime.

New Flag Types

```
// Provided by XR_EXT_debug_utils
typedef XrFlags64 XrDebugUtilsMessageSeverityFlagsEXT;
```

```
// Provided by XR_EXT_debug_utils
// Flag bits for XrDebugUtilsMessageSeverityFlagsEXT
static const XrDebugUtilsMessageSeverityFlagsEXT
XR_DEBUG_UTILS_MESSAGE_SEVERITY_VERBOSE_BIT_EXT = 0x00000001;
static const XrDebugUtilsMessageSeverityFlagsEXT
XR_DEBUG_UTILS_MESSAGE_SEVERITY_INFO_BIT_EXT = 0x00000010;
static const XrDebugUtilsMessageSeverityFlagsEXT
XR_DEBUG_UTILS_MESSAGE_SEVERITY_WARNING_BIT_EXT = 0x00000100;
static const XrDebugUtilsMessageSeverityFlagsEXT
XR_DEBUG_UTILS_MESSAGE_SEVERITY_ERROR_BIT_EXT = 0x00001000;
```

```
// Provided by XR_EXT_debug_utils
typedef XrFlags64 XrDebugUtilsMessageTypeFlagsEXT;
```

```
// Provided by XR_EXT_debug_utils
// Flag bits for XrDebugUtilsMessageTypeFlagsEXT
static const XrDebugUtilsMessageTypeFlagsEXT XR_DEBUG_UTILS_MESSAGE_TYPE_GENERAL_BIT_EXT
= 0x00000001;
static const XrDebugUtilsMessageTypeFlagsEXT
XR_DEBUG_UTILS_MESSAGE_TYPE_VALIDATION_BIT_EXT = 0x00000002;
static const XrDebugUtilsMessageTypeFlagsEXT
XR_DEBUG_UTILS_MESSAGE_TYPE_PERFORMANCE_BIT_EXT = 0x00000004;
static const XrDebugUtilsMessageTypeFlagsEXT
XR_DEBUG_UTILS_MESSAGE_TYPE_CONFORMANCE_BIT_EXT = 0x00000008;
```

New Enum Constants

XrStructureType enumeration is extended with:

- XR_TYPE_DEBUG_UTILS_OBJECT_NAME_INFO_EXT
- XR_TYPE_DEBUG_UTILS_MESSENGER_CALLBACK_DATA_EXT
- XR_TYPE_DEBUG_UTILS_MESSENGER_CREATE_INFO_EXT
- XR_TYPE_DEBUG_UTILS_LABEL_EXT

New Enums

New Structures

```
// Provided by XR_EXT_debug_utils
typedef struct XrDebugUtilsObjectNameInfoEXT {
    XrStructureType    type;
    const void*        next;
    XrObjectType       objectType;
    uint64_t           objectHandle;
    const char*        objectName;
} XrDebugUtilsObjectNameInfoEXT;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `objectType` is an XrObjectType specifying the type of the object to be named.
- `objectHandle` is the object to be named.
- `objectName` is a `NULL` terminated UTF-8 string specifying the name to apply to objectHandle.

Valid Usage

- If `objectType` is `XR_OBJECT_TYPE_UNKNOWN`, `objectHandle` **must** not be XR_NULL_HANDLE
- If `objectType` is not `XR_OBJECT_TYPE_UNKNOWN`, `objectHandle` **must** be XR_NULL_HANDLE or an OpenXR handle of the type associated with `objectType`

Valid Usage (Implicit)

- The `XR_EXT_debug_utils` extension **must** be enabled prior to using XrDebugUtilsObjectNameInfoEXT
- `type` **must** be `XR_TYPE_DEBUG_UTILS_OBJECT_NAME_INFO_EXT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `objectType` **must** be a valid XrObjectType value
- If `objectName` is not `NULL`, `objectName` **must** be a null-terminated UTF-8 string

```
// Provided by XR_EXT_debug_utils
typedef struct XrDebugUtilsLabelEXT {
    XrStructureType    type;
    const void*        next;
    const char*        labelName;
} XrDebugUtilsLabelEXT;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `labelName` is a `NULL` terminated UTF-8 string specifying the label name.

Valid Usage (Implicit)

- The `XR_EXT_debug_utils` extension **must** be enabled prior to using XrDebugUtilsLabelEXT
- `type` **must** be `XR_TYPE_DEBUG_UTILS_LABEL_EXT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `labelName` **must** be a null-terminated UTF-8 string

```
// Provided by XR_EXT_debug_utils
typedef struct XrDebugUtilsMessengerCallbackDataEXT {
    XrStructureType                 type;
    const void*                     next;
    const char*                     messageId;
    const char*                     functionName;
    const char*                     message;
    uint32_t                        objectCount;
    XrDebugUtilsObjectNameInfoEXT*  objects;
    uint32_t                        sessionLabelCount;
    XrDebugUtilsLabelEXT*           sessionLabels;
} XrDebugUtilsMessengerCallbackDataEXT;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `messageId` is a `NULL` terminated string that identifies the message in a unique way. If the callback is triggered by a validation layer, this string corresponds the Valid Usage ID (VUID) that can be used to jump to the appropriate location in the OpenXR specification. This value **may** be `NULL` if no unique message identifier is associated with the message.
- `functionName` is a `NULL` terminated string that identifies the OpenXR function that was executing at the time the message callback was triggered. This value **may** be `NULL` in cases where it is difficult to determine the originating OpenXR function.
- `message` is a `NULL` terminated string detailing the trigger conditions.
- `objectCount` is a count of items contained in the `objects` array. This may be `0`.
- `objects` is `NULL` or a pointer to an array of XrDebugUtilsObjectNameInfoEXT objects related to the detected issue. The array is roughly in order of importance, but the 0th element is always guaranteed to be the most important object for this message.
- `sessionLabelCount` is a count of items contained in the `sessionLabels` array. This may be `0`.
- `sessionLabels` is `NULL` or a pointer to an array of XrDebugUtilsLabelEXT active in the current XrSession at the time the callback was triggered. Refer to Session Labels for more information.

Valid Usage (Implicit)

- The `XR_EXT_debug_utils` extension **must** be enabled prior to using XrDebugUtilsMessengerCallbackDataEXT
- `type` **must** be `XR_TYPE_DEBUG_UTILS_MESSENGER_CALLBACK_DATA_EXT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- If `messageId` is not `NULL`, `messageId` **must** be a null-terminated UTF-8 string
- If `functionName` is not `NULL`, `functionName` **must** be a null-terminated UTF-8 string
- `message` **must** be a null-terminated UTF-8 string

An XrDebugUtilsMessengerCallbackDataEXT is a messenger object that handles passing along debug messages to a provided debug callback.



Note

This structure should only be considered valid during the lifetime of the triggered callback.

The labels listed inside `sessionLabels` are organized in time order, with the most recently generated label appearing first, and the oldest label appearing last.

```
// Provided by XR_EXT_debug_utils
typedef struct XrDebugUtilsMessengerCreateInfoEXT {
    XrStructureType                         type;
    const void*                             next;
    XrDebugUtilsMessageSeverityFlagsEXT     messageSeverities;
    XrDebugUtilsMessageTypeFlagsEXT         messageTypes;
    PFN_xrDebugUtilsMessengerCallbackEXT    userCallback;
    void*                                   userData;
} XrDebugUtilsMessengerCreateInfoEXT;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `messageSeverities` is a bitmask of XrDebugUtilsMessageSeverityFlagBitsEXT specifying which severity of event(s) that will cause this callback to be called.
- `messageTypes` is a combination of XrDebugUtilsMessageTypeFlagBitsEXT specifying which type of event(s) will cause this callback to be called.
- `userCallback` is the application defined callback function to call.
- `userData` is arbitrary user data to be passed to the callback.

Valid Usage

- `userCallback` **must** be a valid PFN_xrDebugUtilsMessengerCallbackEXT

Valid Usage (Implicit)

- The XR_EXT_debug_utils extension **must** be enabled prior to using XrDebugUtilsMessengerCreateInfoEXT
- type **must** be XR_TYPE_DEBUG_UTILS_MESSENGER_CREATE_INFO_EXT
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- messageSeverities **must** be a valid combination of XrDebugUtilsMessageSeverityFlagBitsEXT values
- messageSeverities **must** not be 0
- messageTypes **must** be a valid combination of XrDebugUtilsMessageTypeFlagBitsEXT values
- messageTypes **must** not be 0
- userCallback **must** be a valid PFN_xrDebugUtilsMessengerCallbackEXT value

For each XrDebugUtilsMessengerEXT that is created the XrDebugUtilsMessengerCreateInfoEXT::messageSeverities and XrDebugUtilsMessengerCreateInfoEXT::messageTypes determine when that XrDebugUtilsMessengerCreateInfoEXT::userCallback is called. The process to determine if the user's userCallback is triggered when an event occurs is as follows:

- The runtime will perform a bitwise AND of the event's XrDebugUtilsMessageSeverityFlagBitsEXT with the XrDebugUtilsMessengerCreateInfoEXT::messageSeverities provided during creation of the XrDebugUtilsMessengerEXT object.
- If this results in 0, the message is skipped.
- The runtime will perform bitwise AND of the event's XrDebugUtilsMessageTypeFlagBitsEXT with the XrDebugUtilsMessengerCreateInfoEXT::messageTypes provided during the creation of the XrDebugUtilsMessengerEXT object.
- If this results in 0, the message is skipped.
- If the message of the current event is not skipped, the callback will be called with the message.

The callback will come directly from the component that detected the event, unless some other layer intercepts the calls for its own purposes (filter them in a different way, log to a system error log, etc.).

An application **can** receive multiple callbacks if multiple XrDebugUtilsMessengerEXT objects are created. A callback will always be executed in the same thread as the originating OpenXR call.



Note

A callback **can** be called from multiple threads simultaneously if the application is making OpenXR calls from multiple threads.

New Functions

```
// Provided by XR_EXT_debug_utils
XrResult xrSetDebugUtilsObjectNameEXT(
    XrInstance                                  instance,
    const XrDebugUtilsObjectNameInfoEXT*        nameInfo);
```

Parameter Descriptions

- `instance` is the XrInstance that the object was created under.
- `nameInfo` is a pointer to an instance of the XrDebugUtilsObjectNameInfoEXT structure specifying the parameters of the name to set on the object.

Valid Usage

- In the structure pointed to by `nameInfo`, XrDebugUtilsObjectNameInfoEXT::`objectType` **must** not be `XR_OBJECT_TYPE_UNKNOWN`
- In the structure pointed to by `nameInfo`, XrDebugUtilsObjectNameInfoEXT::`objectHandle` **must** not be XR_NULL_HANDLE

Valid Usage (Implicit)

- The `XR_EXT_debug_utils` extension **must** be enabled prior to calling xrSetDebugUtilsObjectNameEXT
- `instance` **must** be a valid XrInstance handle
- `nameInfo` **must** be a pointer to a valid XrDebugUtilsObjectNameInfoEXT structure

Thread Safety

- Access to the `objectHandle` member of the `nameInfo` parameter **must** be externally synchronized

Return Codes

Success

- XR_SUCCESS

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_VALIDATION_FAILURE

Applications **may** change the name associated with an object simply by calling xrSetDebugUtilsObjectNameEXT again with a new string. If XrDebugUtilsObjectNameInfoEXT ::objectName is an empty string, then any previously set name is removed.

```
// Provided by XR_EXT_debug_utils
XrResult xrCreateDebugUtilsMessengerEXT(
    XrInstance                                  instance,
    const XrDebugUtilsMessengerCreateInfoEXT*   createInfo,
    XrDebugUtilsMessengerEXT*                   messenger);
```

Parameter Descriptions

- instance is the instance the messenger will be used with.
- createInfo points to an XrDebugUtilsMessengerCreateInfoEXT structure, which contains the callback pointer as well as defines the conditions under which this messenger will trigger the callback.
- messenger is a pointer to which the created XrDebugUtilsMessengerEXT object is returned.

Valid Usage (Implicit)

- The XR_EXT_debug_utils extension **must** be enabled prior to calling xrCreateDebugUtilsMessengerEXT
- `instance` **must** be a valid XrInstance handle
- `createInfo` **must** be a pointer to a valid XrDebugUtilsMessengerCreateInfoEXT structure
- `messenger` **must** be a pointer to an XrDebugUtilsMessengerEXT handle

Thread Safety

- Access to `instance`, and any child handles, **must** be externally synchronized

Return Codes

Success

- XR_SUCCESS

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_VALIDATION_FAILURE

The application **must** ensure that xrCreateDebugUtilsMessengerEXT is not executed in parallel with any OpenXR function that is also called with `instance` or child of `instance`.

When an event of interest occurs a debug messenger calls its XrDebugUtilsMessengerCreateInfoEXT ::`userCallback` with a debug message from the producer of the event. Additionally, the debug messenger **must** filter out any debug messages that the application's callback is not interested in based on XrDebugUtilsMessengerCreateInfoEXT flags, as described below.

```
// Provided by XR_EXT_debug_utils
XrResult xrDestroyDebugUtilsMessengerEXT(
    XrDebugUtilsMessengerEXT                    messenger);
```

Parameter Descriptions

- `messenger` the XrDebugUtilsMessengerEXT object to destroy. `messenger` is an externally synchronized object and **must** not be used on more than one thread at a time. This means that xrDestroyDebugUtilsMessengerEXT **must** not be called when a callback is active.

Valid Usage (Implicit)

- The `XR_EXT_debug_utils` extension **must** be enabled prior to calling xrDestroyDebugUtilsMessengerEXT
- `messenger` **must** be a valid XrDebugUtilsMessengerEXT handle

Thread Safety

- Access to `messenger` **must** be externally synchronized
- Access to the XrInstance used to create `messenger`, and all of its child handles **must** be externally synchronized

Return Codes

Success

- `XR_SUCCESS`

Failure

- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_RUNTIME_FAILURE`

The application **must** ensure that xrDestroyDebugUtilsMessengerEXT is not executed in parallel with any OpenXR function that is also called with the `instance` or child of `instance` that it was created with.

```
// Provided by XR_EXT_debug_utils
XrResult xrSubmitDebugUtilsMessageEXT(
    XrInstance                                  instance,
    XrDebugUtilsMessageSeverityFlagsEXT         messageSeverity,
    XrDebugUtilsMessageTypeFlagsEXT             messageTypes,
    const XrDebugUtilsMessengerCallbackDataEXT* callbackData);
```

Parameter Descriptions

- `instance` is the debug stream's XrInstance.
- `messageSeverity` is a single bit value of XrDebugUtilsMessageSeverityFlagsEXT severity of this event/message.
- `messageTypes` is an XrDebugUtilsMessageTypeFlagsEXT bitmask of XrDebugUtilsMessageTypeFlagBitsEXT specifying which types of event to identify this message with.
- `callbackData` contains all the callback related data in the XrDebugUtilsMessengerCallbackDataEXT structure.

Valid Usage

- For each structure in XrDebugUtilsMessengerCallbackDataEXT::`objects`, the value of XrDebugUtilsObjectNameInfoEXT::`objectType` **must** not be `XR_OBJECT_TYPE_UNKNOWN`

Valid Usage (Implicit)

- The `XR_EXT_debug_utils` extension **must** be enabled prior to calling xrSubmitDebugUtilsMessageEXT
- `instance` **must** be a valid XrInstance handle
- `messageSeverity` **must** be a valid combination of XrDebugUtilsMessageSeverityFlagBitsEXT values
- `messageSeverity` **must** not be `0`
- `messageTypes` **must** be a valid combination of XrDebugUtilsMessageTypeFlagBitsEXT values
- `messageTypes` **must** not be `0`
- `callbackData` **must** be a pointer to a valid XrDebugUtilsMessengerCallbackDataEXT structure

Return Codes

Success

- XR_SUCCESS

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_VALIDATION_FAILURE

The application **can** also produce a debug message, and submit it into the OpenXR messaging system.

The call will propagate through the layers and generate callback(s) as indicated by the message's flags. The parameters are passed on to the callback in addition to the userData value that was defined at the time the messenger was created.

```
// Provided by XR_EXT_debug_utils
XrResult xrSessionBeginDebugUtilsLabelRegionEXT(
    XrSession                                   session,
    const XrDebugUtilsLabelEXT*                 labelInfo);
```

Parameter Descriptions

- session is the XrSession that a label region should be associated with.
- labelInfo is the XrDebugUtilsLabelEXT containing the label information for the region that should be begun.

Valid Usage (Implicit)

- The XR_EXT_debug_utils extension **must** be enabled prior to calling xrSessionBeginDebugUtilsLabelRegionEXT
- session **must** be a valid XrSession handle
- labelInfo **must** be a pointer to a valid XrDebugUtilsLabelEXT structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The xrSessionBeginDebugUtilsLabelRegionEXT function begins a label region within session.

```
// Provided by XR_EXT_debug_utils
XrResult xrSessionEndDebugUtilsLabelRegionEXT(
    XrSession                                   session);
```

Parameter Descriptions

- session is the XrSession that a label region should be associated with.

Valid Usage

- xrSessionEndDebugUtilsLabelRegionEXT **must** be called only after a matching xrSessionBeginDebugUtilsLabelRegionEXT.

Valid Usage (Implicit)

- The XR_EXT_debug_utils extension **must** be enabled prior to calling xrSessionEndDebugUtilsLabelRegionEXT
- session **must** be a valid XrSession handle

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

This function ends the last label region begun with the xrSessionBeginDebugUtilsLabelRegionEXT function within the same session.

```
// Provided by XR_EXT_debug_utils
XrResult xrSessionInsertDebugUtilsLabelEXT(
    XrSession                                   session,
    const XrDebugUtilsLabelEXT*                 labelInfo);
```

Parameter Descriptions

- session is the XrSession that a label region should be associated with.
- labelInfo is the XrDebugUtilsLabelEXT containing the label information for the region that should be begun.

Valid Usage (Implicit)

- The XR_EXT_debug_utils extension **must** be enabled prior to calling xrSessionInsertDebugUtilsLabelEXT
- session **must** be a valid XrSession handle
- labelInfo **must** be a pointer to a valid XrDebugUtilsLabelEXT structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The xrSessionInsertDebugUtilsLabelEXT function inserts an individual label within `session`. The individual labels are useful for different reasons based on the type of debugging scenario. When used with something active like a profiler or debugger, it identifies a single point of time. When used with logging, the individual label identifies that a particular location has been passed at the point the log message is triggered. Because of this usage, individual labels only exist in a log until the next call to any of the label functions:

- xrSessionBeginDebugUtilsLabelRegionEXT
- xrSessionEndDebugUtilsLabelRegionEXT
- xrSessionInsertDebugUtilsLabelEXT

New Function Pointers

```
// Provided by XR_EXT_debug_utils
typedef XrBool32 (XRAPI_PTR *PFN_xrDebugUtilsMessengerCallbackEXT)(
            XrDebugUtilsMessageSeverityFlagsEXT             messageSeverity,
            XrDebugUtilsMessageTypeFlagsEXT                 messageTypes,
            const XrDebugUtilsMessengerCallbackDataEXT*     callbackData,
            void*                                           userData);
```

Parameter Descriptions

- `messageSeverity` indicates the single bit value of XrDebugUtilsMessageSeverityFlagsEXT that triggered this callback.
- `messageTypes` indicates the XrDebugUtilsMessageTypeFlagsEXT specifying which types of event triggered this callback.
- `callbackData` contains all the callback related data in the XrDebugUtilsMessengerCallbackDataEXT structure.
- `userData` is the user data provided when the XrDebugUtilsMessengerEXT was created.

The callback **must** not call xrDestroyDebugUtilsMessengerEXT.

The callback returns an `XrBool32` that indicates to the calling layer the application's desire to abort the call. A value of `XR_TRUE` indicates that the application wants to abort this call. If the application returns `XR_FALSE`, the function **must** not be aborted. Applications **should** always return `XR_FALSE` so that they see the same behavior with and without validation layers enabled.

If the application returns `XR_TRUE` from its callback and the OpenXR call being aborted returns an XrResult, the layer will return `XR_ERROR_VALIDATION_FAILURE`.

The object pointed to by `callbackData` (and any pointers in it recursively) **must** be valid during the lifetime of the triggered callback. It **may** become invalid afterwards.

Examples

Example 1

XR_EXT_debug_utils allows an application to register multiple callbacks with any OpenXR component wishing to report debug information. Some callbacks may log the information to a file, others may cause a debug break point or other application defined behavior. An application **can** register callbacks even when no validation layers are enabled, but they will only be called for loader and, if implemented, driver events.

To capture events that occur while creating or destroying an instance an application **can** link an XrDebugUtilsMessengerCreateInfoEXT structure to the next element of the XrInstanceCreateInfo structure given to xrCreateInstance. This callback is only valid for the duration of the xrCreateInstance and the xrDestroyInstance call. Use xrCreateDebugUtilsMessengerEXT to create persistent callback objects.

Example uses: Create three callback objects. One will log errors and warnings to the debug console using Windows `OutputDebugString`. The second will cause the debugger to break at that callback when an error happens and the third will log warnings to stdout.

```
    extern XrInstance instance;  // previously initialized
```

```
    // Must call extension functions through a function pointer:
    PFN_xrCreateDebugUtilsMessengerEXT pfnCreateDebugUtilsMessengerEXT = nullptr;
    CHK_XR(xrGetInstanceProcAddr(instance, "xrCreateDebugUtilsMessengerEXT",
                                 reinterpret_cast<PFN_xrVoidFunction*>(
                                     &pfnCreateDebugUtilsMessengerEXT)));

    PFN_xrDestroyDebugUtilsMessengerEXT pfnDestroyDebugUtilsMessengerEXT = nullptr;
    CHK_XR(xrGetInstanceProcAddr(instance, "xrDestroyDebugUtilsMessengerEXT",
                                 reinterpret_cast<PFN_xrVoidFunction*>(
                                     &pfnDestroyDebugUtilsMessengerEXT)));

    XrDebugUtilsMessengerCreateInfoEXT callback1 {
        .type = XR_TYPE_DEBUG_UTILS_MESSENGER_CREATE_INFO_EXT,
        .next = nullptr,
        .messageSeverities = XR_DEBUG_UTILS_MESSAGE_SEVERITY_ERROR_BIT_EXT |
XR_DEBUG_UTILS_MESSAGE_SEVERITY_WARNING_BIT_EXT,
        .messageTypes = XR_DEBUG_UTILS_MESSAGE_TYPE_GENERAL_BIT_EXT |
XR_DEBUG_UTILS_MESSAGE_TYPE_VALIDATION_BIT_EXT,
        .userCallback = myOutputDebugString,
        .userData = nullptr,
    };
    XrDebugUtilsMessengerEXT messenger1 {XR_NULL_HANDLE};
    CHK_XR(pfnCreateDebugUtilsMessengerEXT(instance, &callback1, &messenger1));

    callback1.messageSeverities = XR_DEBUG_UTILS_MESSAGE_SEVERITY_ERROR_BIT_EXT;
    callback1.userCallback = myDebugBreak;
    callback1.userData = nullptr;
    XrDebugUtilsMessengerEXT messenger2 {XR_NULL_HANDLE};
    CHK_XR(pfnCreateDebugUtilsMessengerEXT(instance, &callback1, &messenger2));

    XrDebugUtilsMessengerCreateInfoEXT callback3 {
        .type = XR_TYPE_DEBUG_UTILS_MESSENGER_CREATE_INFO_EXT,
        .next = nullptr,
        .messageSeverities = XR_DEBUG_UTILS_MESSAGE_SEVERITY_WARNING_BIT_EXT,
        .messageTypes = XR_DEBUG_UTILS_MESSAGE_TYPE_GENERAL_BIT_EXT |
XR_DEBUG_UTILS_MESSAGE_TYPE_VALIDATION_BIT_EXT,
        .userCallback = myStdOutLogger,
        .userData = nullptr,
    };
    XrDebugUtilsMessengerEXT messenger3 {XR_NULL_HANDLE};
    CHK_XR(pfnCreateDebugUtilsMessengerEXT(instance, &callback3, &messenger3));

    // ...

    // Remove callbacks when cleaning up
    pfnDestroyDebugUtilsMessengerEXT(messenger1);
    pfnDestroyDebugUtilsMessengerEXT(messenger2);
```

```
    pfnDestroyDebugUtilsMessengerEXT(messenger3);
```

Example 2

Associate a name with an XrSpace, for easier debugging in external tools or with validation layers that can print a friendly name when referring to objects in error messages.

```
    extern XrInstance instance;  // previously initialized
    extern XrSpace space;        // previously initialized

    // Must call extension functions through a function pointer:
    PFN_xrSetDebugUtilsObjectNameEXT pfnSetDebugUtilsObjectNameEXT = nullptr;
    CHK_XR(xrGetInstanceProcAddr(instance, "xrSetDebugUtilsObjectNameEXT",
                                 reinterpret_cast<PFN_xrVoidFunction*>(
                                     &pfnSetDebugUtilsObjectNameEXT)));

    // Set a name on the space
    const XrDebugUtilsObjectNameInfoEXT spaceNameInfo {
        .type = XR_TYPE_DEBUG_UTILS_OBJECT_NAME_INFO_EXT,
        .next = nullptr,
        .objectType = XR_OBJECT_TYPE_SPACE,
        .objectHandle = (uint64_t)space,
        .objectName = "My Object-Specific Space",
    };

    pfnSetDebugUtilsObjectNameEXT(instance, &spaceNameInfo);

    // A subsequent error might print:
    //   Space "My Object-Specific Space" (0xc0dec0dedeadbeef) is used
    //   with an XrSession that is not it's parent.
```

Example 3

Labeling the workload with naming information so that any form of analysis can display a more usable visualization of where actions occur in the lifetime of a session.

```
    extern XrInstance instance;  // previously initialized
    extern XrSession session;    // previously initialized

    // Must call extension functions through a function pointer:

    PFN_xrSessionBeginDebugUtilsLabelRegionEXT pfnSessionBeginDebugUtilsLabelRegionEXT =
nullptr;
    CHK_XR(xrGetInstanceProcAddr(instance, "xrSessionBeginDebugUtilsLabelRegionEXT",
                                 reinterpret_cast<PFN_xrVoidFunction*>(
```

```
                                    &pfnSessionBeginDebugUtilsLabelRegionEXT)));

    PFN_xrSessionEndDebugUtilsLabelRegionEXT pfnSessionEndDebugUtilsLabelRegionEXT =
nullptr;
    CHK_XR(xrGetInstanceProcAddr(instance, "xrSessionEndDebugUtilsLabelRegionEXT",
                                 reinterpret_cast<PFN_xrVoidFunction*>(
                                    &pfnSessionEndDebugUtilsLabelRegionEXT)));

    PFN_xrSessionInsertDebugUtilsLabelEXT pfnSessionInsertDebugUtilsLabelEXT = nullptr;
    CHK_XR(xrGetInstanceProcAddr(instance, "xrSessionInsertDebugUtilsLabelEXT",
                                 reinterpret_cast<PFN_xrVoidFunction*>(
                                    &pfnSessionInsertDebugUtilsLabelEXT)));

    XrSessionBeginInfo session_begin_info {
        .type = XR_TYPE_SESSION_BEGIN_INFO,
        .next = nullptr,
        .primaryViewConfigurationType = XR_VIEW_CONFIGURATION_TYPE_PRIMARY_STEREO,
    };
    xrBeginSession(session, &session_begin_info);

    const XrDebugUtilsLabelEXT session_active_region_label {
        .type = XR_TYPE_DEBUG_UTILS_LABEL_EXT,
        .next = nullptr,
        .labelName = "Session active",
    };

    // Start an annotated region of calls under the 'Session Active' name
    pfnSessionBeginDebugUtilsLabelRegionEXT(session, &session_active_region_label);

    // Brackets added for clarity
    {
        XrDebugUtilsLabelEXT individual_label {
            .type = XR_TYPE_DEBUG_UTILS_LABEL_EXT,
            .next = nullptr,
            .labelName = "WaitFrame",
        };

        const char wait_frame_label[] = "WaitFrame";
        individual_label.labelName = wait_frame_label;
        pfnSessionInsertDebugUtilsLabelEXT(session, &individual_label);
        XrFrameWaitInfo wait_frame_info {
            .type = XR_TYPE_FRAME_WAIT_INFO,
        };
        XrFrameState frame_state {
            .type = XR_TYPE_FRAME_STATE,
            .next = nullptr,
        };
        xrWaitFrame(session, &wait_frame_info, &frame_state);
```

```
        // Do stuff 1

        const XrDebugUtilsLabelEXT session_frame_region_label {
            .type = XR_TYPE_DEBUG_UTILS_LABEL_EXT,
            .next = nullptr,
            .labelName = "Session Frame 123",
        };

        // Start an annotated region of calls under the 'Session Frame 123' name
        pfnSessionBeginDebugUtilsLabelRegionEXT(session, &session_frame_region_label);

        // Brackets added for clarity
        {

            const char begin_frame_label[] = "BeginFrame";
            individual_label.labelName = begin_frame_label;
            pfnSessionInsertDebugUtilsLabelEXT(session, &individual_label);

            XrFrameBeginInfo begin_frame_info {
                .type = XR_TYPE_FRAME_BEGIN_INFO,
            };
            xrBeginFrame(session, &begin_frame_info);

            // Do stuff 2

            const char end_frame_label[] = "EndFrame";
            individual_label.labelName = end_frame_label;
            pfnSessionInsertDebugUtilsLabelEXT(session, &individual_label);

            XrFrameEndInfo end_frame_info;  // initialization omitted for readability
            xrEndFrame(session, &end_frame_info);
        }

        // End the session/begun region started above
        // (in this case it's the "Session Frame 123" label)
        pfnSessionEndDebugUtilsLabelRegionEXT(session);
    }

    // End the session/begun region started above
    // (in this case it's the "Session Active" label)
    pfnSessionEndDebugUtilsLabelRegionEXT(session);
```

In the above example, if an error occurred in the `// Do stuff 1` section, then your debug utils callback would contain the following data in its `sessionLabels` array:

- `[0]` = `individual_label` with `labelName` = "WaitFrame"

- `[1]` = `session_active_region_label` with `labelName` = "Session active"

However, if an error occurred in the `// Do stuff 2` section, then your debug utils callback would contain the following data in its `sessionLabels` array:

- `[0]` = `individual_label` with `labelName` = "BeginFrame"
- `[1]` = `session_frame_region_label` with `labelName` = "Session Frame 123"
- `[2]` = `session_active_region_label` with `labelName` = "Session active"

You'll notice that "WaitFrame" is no longer available as soon as the next call to another function like xrSessionBeginDebugUtilsLabelRegionEXT.

Issues

None

Version History

- Revision 1, 2018-02-19 (Mark Young / Karl Schultz)
  - Initial draft, based on VK_EXT_debug_utils.
- Revision 2, 2018-11-16 (Mark Young)
  - Clean up some language based on changes going into the Vulkan VK_EXT_debug_utils extension by Peter Kraus (aka @krOoze).
  - Added session labels
- Revision 3, 2019-07-19 (Rylie Pavlik)
  - Update examples.
  - Improve formatting.
- Revision 4, 2021-04-04 (Rylie Pavlik)
  - Fix missing error code.
  - Improve formatting.
- Revision 5, 2023-07-25 (John Kearney, Meta)
  - `XrDebugUtilsMessengerCallbackDataEXT` parameters messageId and functionName to be optional.

12.31. XR_EXT_dpad_binding

Name String

`XR_EXT_dpad_binding`

Extension Type

Instance extension

Registered Extension Number

79

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

OpenXR 1.0
and
`XR_KHR_binding_modification`

Last Modified Date

2022-04-20

IP Status

No known IP claims.

Contributors

Joe Ludwig, Valve
Keith Bradner, Valve
Rune Berg, Valve
Nathan Nuber, Valve
Jakob Bornecrantz, Collabora
Rylie Pavlik, Collabora
Jules Blok, Epic Games

Overview

This extension allows the application to bind one or more digital actions to a trackpad or thumbstick as though it were a dpad by defining additional component paths to suggest bindings for. The behavior of this dpad-like mapping may be customized using XrInteractionProfileDpadBindingEXT.

Applications **must** also enable the `XR_KHR_binding_modification` extension that this builds on top of.

New Component Paths

When this extension is enabled, a runtime **must** accept otherwise-valid suggested bindings that refer to the following component paths added to certain existing input subpaths.

- For a given interaction profile,
  - For each input subpath valid in that interaction profile that has identifier *trackpad* but without a component specified (i.e. *.../input/trackpad* or *.../input/trackpad_*<location>), a runtime **must** accept the following components appended to that path in a suggested binding:

        - *.../dpad_up*
        - *.../dpad_down*
        - *.../dpad_left*
        - *.../dpad_right*
        - *.../dpad_center*
    - For each input subpath valid in that interaction profile that has identifier *thumbstick* but without a component specified (i.e. *.../input/thumbstick* or *.../input/thumbstick_*<location>), a runtime **must** accept the following components appended to that path in a suggested binding:
        - *.../dpad_up*
        - *.../dpad_down*
        - *.../dpad_left*
        - *.../dpad_right*

While a runtime **may** ignore accepted suggested bindings, and **may** use their contents as suggestions for automatic remapping when not obeying them, this extension defines interpretations the runtime **must** make in the case that a suggested binding using one of these paths is being obeyed.

An application **can** pass XrInteractionProfileDpadBindingEXT in the XrBindingModificationsKHR::`bindingModifications` array associated with a suggested binding to customize the behavior of this mapping in the case that suggested bindings are being obeyed, and to provide remapping hints in other cases. If no XrInteractionProfileDpadBindingEXT structure is present in XrBindingModificationsKHR::`bindingModifications` for a given action set and component-less input subpath, the runtime **must** behave as if one were passed with the following values:

- `forceThreshold` = 0.5
- `forceThresholdReleased` = 0.4
- `centerRegion` = 0.5
- `wedgeAngle` = ½ π
- `isSticky` = `XR_FALSE`
- `onHaptic` = `NULL`
- `offHaptic` = `NULL`

For the purposes of description, the (-1, 1) ranges of the *x* and *y* components of trackpad and thumbstick inputs are depicted in this extension as if their scale were equal between axes. However, this is **not** required by this extension: while their numeric scale is treated as equal, their physical scale **may** not be.

Each of the component paths defined by this extension behave as boolean inputs. The center component *.../dpad_center* (only present when the path identifier is *trackpad*) **must** not be active at the same time as any other dpad component. For the other components, zero, one, or (depending on the

wedgeAngle) two of them **may** be active at any time, though only adjacent components on a single logical dpad may be active simultaneously. For example, *.../dpad_down* and *.../dpad_left* are adjacent, and thus **may** be active simultaneously, while *.../dpad_up* and *.../dpad_down* are not adjacent and **must** not be active simultaneously.



Note

If wedgeAngle > ½ π, it is possible for two components referring to adjacent directions (excluding *.../dpad_center*) to be active at the same time, as the directional regions overlap. If wedgeAngle < ½ π, there are wedges between directional regions that correspond to no dpad component.

The following components are defined by possibly-overlapping truncated wedges pointing away from 0, 0 in *x*, *y* input space, with their angular size of XrInteractionProfileDpadBindingEXT::wedgeAngle centered around the indicated direction.

- *.../dpad_up*: direction (0, 1)
- *.../dpad_down*: direction (0, -1)
- *.../dpad_left*: direction (-1, 0)
- *.../dpad_right*: direction (1, 0)

Typical values for wedgeAngle are ½ π (or 90°) for regions that do not overlap or ¾ π (or 135°) for regions are evenly divided between the exclusive region for one cardinal direction and the overlap with neighboring regions.

Each of these regions are truncated by an arc to exclude the area within a radius of XrInteractionProfileDpadBindingEXT::centerRegion away from 0, 0. When used with an input path with an identifier of *trackpad*, the area within this radius corresponds to the *.../dpad_center* component. When used with an input path with an identifier of *thumbstick*, the area within this radius is a region where all dpad components **must** be inactive.

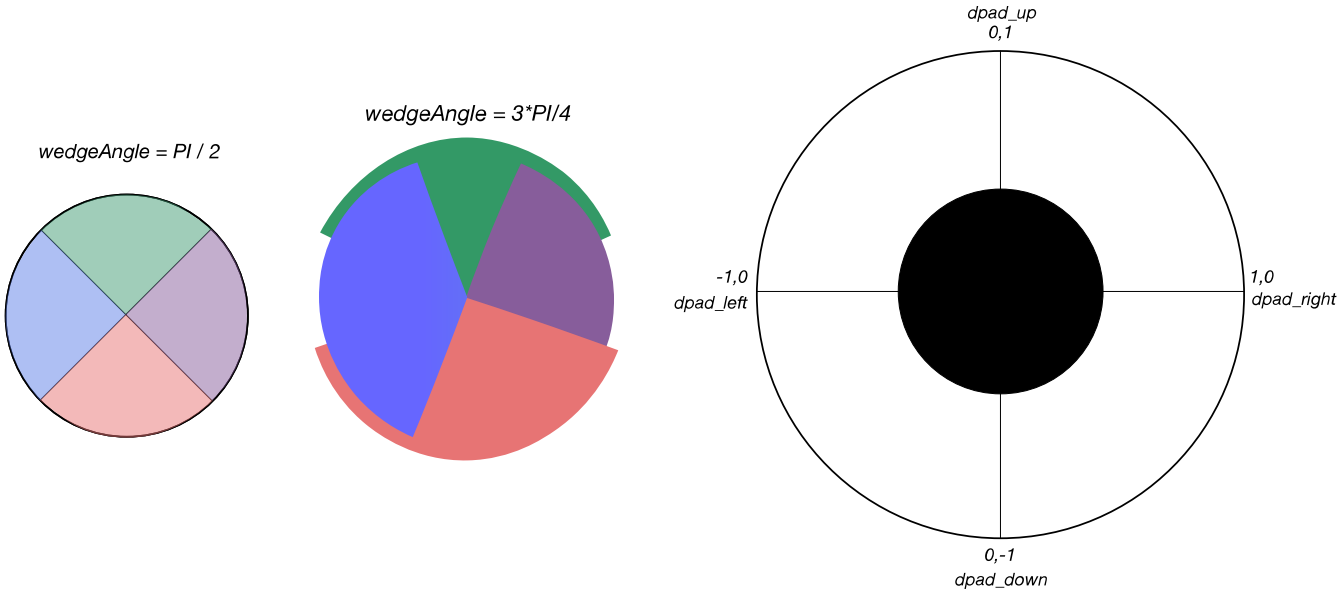


Figure 8. Wedge Angles

Behavior

For both the *trackpad* and *thumbstick* input identifiers, there are conditions that **must** be true for any dpad component to report active. If these conditions are true, the selection of which component or components are active, if any, takes place.

- Activation of a dpad component when appended to an input path with identifier *trackpad* on the values of the *.../x* and *.../y* components, as well as on an overall activation state. If the overall state is inactive, the runtime **must** treat all corresponding dpad components as inactive.
  - If the component *.../click* is also valid for the trackpad, the overall activation state is equal to the value of the *.../click*.
  - If the component *.../click* is **not** valid for the trackpad, but the component *.../force* **is** valid, the overall activation state depends on the value of that *.../force* component, as well as the previous overall activation state for hysteresis. The *.../force* component value hysteresis thresholds for overall activation are XrInteractionProfileDpadBindingEXT::`forceThreshold` and `forceThresholdReleased`. More explicitly:
    - If the previous overall state was inactive, the current overall state **must** be active if and only if the value of the *.../force* component is greater than or equal to `forceThreshold`.
    - If the previous overall state was active, the current state **must** be inactive if and only if the value of the *.../force* component is strictly less than `forceThresholdReleased`.
- Activation of a dpad component when appended to an input path with identifier *thumbstick* depends only on the value of the *.../x* and *.../y* components of that input.
  - If the thumbstick *x* and *y* values correspond to a deflection from center of less than `centerRegion`, all dpad components **must** be reported as inactive.

Hysteresis is desirable to avoid an unintentional, rapid toggling between the active and inactive state that can occur when the amount of force applied by the user is very close to the threshold at which the

input is considered active. Hysteresis is optional, and is achieved through a difference between `forceThreshold` and `forceThresholdReleased`.

When XrInteractionProfileDpadBindingEXT::`isSticky` is `XR_FALSE`, and the above logic indicates that some dpad component is active, a runtime obeying suggested bindings **must** select which dpad components to report as active based solely on the current *x*, *y* values.

If XrInteractionProfileDpadBindingEXT::`isSticky` is `XR_TRUE`, the region(s) to be made active **must** be latched when the above logic begins to indicate that some dpad component is active, and the *x* and *y* values are within at least one region. The latched region(s) **must** continue to be reported as active until the activation logic indicates that all dpad components **must** be inactive. The latched region(s) remain active even if the input leaves that region or enters another region.

The runtime **must** latch the *x* and *y* values, and thus the region or regions (in the case of overlapping dpad component wedges), when the sticky activation toggle becomes true. The latched regions **must** continue to be true until the input returns to the center region (for a thumbstick) or is released (for a trackpad). In this way, sticky dpads maintain their selected region across touch/click transitions.

Examples for isSticky == `XR_TRUE`

- Trackpad example: If the user clicks a trackpad in the *.../dpad_up* region, then (while clicked) slides their finger to the *.../dpad_down* region, *.../dpad_up* will remain true.
- Thumbstick example: If the user presses up on the thumbstick and activates the *.../dpad_up* region, then slides the thumbstick around to the *.../dpad_down* region without crossing the `centerRegion`, *.../dpad_up* is the virtual input that will be true.
- Thumbstick example: If the user presses up on the thumbstick and activates the *.../dpad_up* region, then slides the thumbstick directly down and through the region specified by `centerRegion` to *.../dpad_down*. Initially *.../dpad_up* will activate. Then when the thumbstick enters the `centerRegion` it will deactivate. Finally, when entering the *.../dpad_down* region *.../dpad_down* will activate.

New Structures

The XrInteractionProfileDpadBindingEXT structure is defined as:

```
// Provided by XR_EXT_dpad_binding
typedef struct XrInteractionProfileDpadBindingEXT {
    XrStructureType             type;
    const void*                 next;
    XrPath                      binding;
    XrActionSet                 actionSet;
    float                       forceThreshold;
    float                       forceThresholdReleased;
    float                       centerRegion;
    float                       wedgeAngle;
    XrBool32                    isSticky;
    const XrHapticBaseHeader*   onHaptic;
    const XrHapticBaseHeader*   offHaptic;
} XrInteractionProfileDpadBindingEXT;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `binding` is the input path used for the specified actions in the suggested binding list to be used as a dpad. E.g. path:/user/hand/right/input/thumbstick
- `actionSet` is the action set for which this dpad will be active. The implementation **must** use the parameters from this structure for any actions from this action set that are bound to one of the dpad subpaths for this input path.
- `forceThreshold` a number in the half-open range (0, 1] representing the force value threshold at or above which (≥) a dpad input will transition from inactive to active.
- `forceThresholdReleased` a number in the half-open range (0, 1] representing the force value threshold strictly below which (<) a dpad input will transition from active to inactive.
- `centerRegion` defines the center region of the thumbstick or trackpad. This is the radius, in the input value space, of a logically circular region in the center of the input, in the range (0, 1).
- `wedgeAngle` indicates the angle in radians of each direction region and is a value in the half-open range [0, π).
- `isSticky` indicates that the implementation will latch the first region that is activated and continue to indicate that the binding for that region is true until the user releases the input underlying the virtual dpad.
- `onHaptic` is the haptic output that the runtime **must** trigger when the binding changes from false to true. If this field is `NULL`, the runtime **must** not trigger any haptic output on the threshold. This field **can** point to any supported sub-type of XrHapticBaseHeader.
- `offHaptic` is the haptic output that the runtime **must** trigger when the binding changes from true to false. If this field is NULL, the runtime **must** not trigger any haptic output on the threshold. This field **can** point to any supported sub-type of XrHapticBaseHeader.

The XrInteractionProfileDpadBindingEXT structure is an input struct that defines how to use any two-axis input to provide dpad-like functionality to the application. The struct **must** be added for each input that should be treated as a dpad to the XrBindingModificationsKHR::`bindingModifications` array in the XrBindingModificationsKHR structure (See `XR_KHR_binding_modification` extension).

Runtimes are free to ignore any of the fields when not obeying the bindings, but **may** use it for automatic rebindings of actions.

The implementation **must** return `XR_ERROR_VALIDATION_FAILURE` from xrSuggestInteractionProfileBindings if any of the following are true:

- `forceThreshold` or `forceThresholdReleased` are outside the half-open range (0, 1]

- `forceThreshold` < `forceThresholdReleased`
- `centerRegion` is outside the exclusive range (0, 1)
- `wedgeAngle` outside the half-open range [0, π)

If more than one XrInteractionProfileDpadBindingEXT is provided for the same input identifier, including top level path (e.g. */user/hand/left/input/thumbstick*), and two or more of them specify the same actionset, the runtime **must** return `XR_ERROR_VALIDATION_FAILURE`. If the same input identifier, including top level path, is used for more than one action set, in addition to inputs being suppressed by higher priority action sets, haptic events from dpads are also suppressed.

For example, a Valve Index controller binding with a "Walking" action set can have a dpad on each of:

- left thumbstick
- right thumbstick
- left trackpad
- right trackpad

Another action set can also have a dpad active on each of those inputs, and they can have different settings. If both action sets are active, the higher priority one trumps the lower priority one, and the lower priority one is suppressed.

Valid Usage (Implicit)

- The `XR_EXT_dpad_binding` extension **must** be enabled prior to using XrInteractionProfileDpadBindingEXT
- `type` **must** be `XR_TYPE_INTERACTION_PROFILE_DPAD_BINDING_EXT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `actionSet` **must** be a valid XrActionSet handle
- If `onHaptic` is not `NULL`, `onHaptic` **must** be a pointer to a valid XrHapticBaseHeader-based structure. See also: XrHapticAmplitudeEnvelopeVibrationFB, XrHapticParametricVibrationEXT, XrHapticPcmVibrationFB, XrHapticVibration
- If `offHaptic` is not `NULL`, `offHaptic` **must** be a pointer to a valid XrHapticBaseHeader-based structure. See also: XrHapticAmplitudeEnvelopeVibrationFB, XrHapticParametricVibrationEXT, XrHapticPcmVibrationFB, XrHapticVibration

New Functions

Issues

- What if an interaction profile is added that contains a *trackpad* identifier, for which there is neither a *.../click* or a *.../force* component?

    - Equivalent logic would apply to whatever component is available to distinguish action from inaction.

- Is zero a valid wedge angle? Is π?
    - Yes, though it is mostly useless, as it makes the directional regions empty in size and thus impossible to activate. The user could only activate *.../dpad_center* on a *trackpad* identifier. π is not a valid wedge angle because that would imply being able to activate three adjacent directions, of which two must be opposite. In practice, the sensors underlying these inputs make it effectively impossible to input an exact floating point value.

Example

The following sample code shows how to create dpad bindings using this extension.

```
    // Create dpad paths
    XrPath pathThumbstick, pathDpadUp, pathDpadDown;
    xrStringToPath( pInstance, "/user/hand/left/input/thumbstick", &pathThumbstick);
    xrStringToPath( pInstance, "/user/hand/left/input/thumbstick/dpad_up", &pathDpadUp
);
    xrStringToPath( pInstance, "/user/hand/left/input/thumbstick/dpad_down",
&pathDpadDown );

    // Set dpad binding modifiers
    XrInteractionProfileDpadBindingEXT xrDpadModification {
XR_TYPE_INTERACTION_PROFILE_DPAD_BINDING_EXT };
    xrDpadModification.actionSet = xrActionSet_Main;
    xrDpadModification.binding = pathThumbstick;
    xrDpadModification.centerRegion = 0.25f;
    xrDpadModification.wedgeAngle = 2.0f;
    // A gap between these next two members creates hysteresis, to avoid rapid
toggling
    xrDpadModification.forceThreshold = 0.8f;
    xrDpadModification.forceThresholdReleased = 0.2f;

    // Add dpad binding modifiers to binding modifications vector
    std::vector< XrInteractionProfileDpadBindingEXT > vBindingModifs;
    vBindingModifs.push_back( xrDpadModification );

    std::vector< XrBindingModificationBaseHeaderKHR* > vBindingModifsBase;
    for ( XrInteractionProfileDpadBindingEXT &modif : vBindingModifs )
    {
        vBindingModifsBase.push_back( reinterpret_cast<
XrBindingModificationBaseHeaderKHR* >( &modif) );
    }

    XrBindingModificationsKHR xrBindingModifications {
XR_TYPE_BINDING_MODIFICATIONS_KHR };
```

```
    xrBindingModifications.bindingModifications = vBindingModifsBase.data();
    xrBindingModifications.bindingModificationCount = ( uint32_t )vBindingModifsBase
.size();

    // Set dpad input path as suggested binding for an action
    XrActionSuggestedBinding xrActionBindingTeleport, xrActionBindingMenu;

    xrActionBindingTeleport.action = xrAction_Teleport;
    xrActionBindingTeleport.binding = pathDpadUp;

    xrActionBindingMenu.action = xrAction_Menu;
    xrActionBindingMenu.binding = pathDpadDown;

    std::vector< XrActionSuggestedBinding > vActionBindings;
    vActionBindings.push_back( xrActionBindingTeleport );
    vActionBindings.push_back( xrActionBindingMenu );

    // Create interaction profile/controller path
    XrPath xrInteractionProfilePath;
    xrStringToPath( pInstance, "/interaction_profiles/valve/index_controller",
&xrInteractionProfilePath );

    // Set suggested binding to interaction profile
    XrInteractionProfileSuggestedBinding xrInteractionProfileSuggestedBinding {
XR_TYPE_INTERACTION_PROFILE_SUGGESTED_BINDING };
    xrInteractionProfileSuggestedBinding.interactionProfile =
xrInteractionProfilePath;
    xrInteractionProfileSuggestedBinding.suggestedBindings = vActionBindings.data();
    xrInteractionProfileSuggestedBinding.countSuggestedBindings = ( uint32_t
)vActionBindings.size();

    // Set binding modifications to interaction profile's suggested binding
    xrInteractionProfileSuggestedBinding.next = &xrBindingModifications;

    // Finally, suggest interaction profile bindings to runtime
    xrSuggestInteractionProfileBindings( pInstance,
&xrInteractionProfileSuggestedBinding );
```

Version History

- Revision 1, 2022-02-18 (Rune Berg)
  - Initial extension description

12.32. XR_EXT_eye_gaze_interaction

Name String

`XR_EXT_eye_gaze_interaction`

Extension Type

Instance extension

Registered Extension Number

31

Revision

2

Ratification Status

Ratified

Extension and Version Dependencies

OpenXR 1.0

Last Modified Date

2020-02-20

IP Status

No known IP claims.

Contributors

Denny Rönngren, Tobii
Yin Li, Microsoft
Alex Turner, Microsoft
Paul Pedriana, Oculus
Rémi Arnaud, Varjo
Blake Taylor, Magic Leap
Lachlan Ford, Microsoft
Cass Everitt, Oculus

Overview

This extension provides an `XrPath` for getting eye gaze input from an eye tracker to enable eye gaze interactions.

The intended use for this extension is to provide:

- system properties to inform if eye gaze interaction is supported by the current device.
- an `XrPath` for real time eye tracking that exposes an accurate and precise eye gaze pose to be used to enable eye gaze interactions.
- a structure XrEyeGazeSampleTimeEXT that allows for an application to retrieve more information

regarding the eye tracking samples.

With these building blocks, an application **can** discover if the XR runtime has access to an eye tracker, bind the eye gaze pose to the action system, determine if the eye tracker is actively tracking the users eye gaze, and use the eye gaze pose as an input signal to build eye gaze interactions.

12.32.1. Eye tracker

An eye tracker is a sensory device that tracks eyes and accurately maps what the user is looking at. The main purpose of this extension is to provide accurate and precise eye gaze for the application.

Eye tracking data is sensitive personal information and is closely linked to personal privacy and integrity. It is strongly recommended that applications that store or transfer eye tracking data always ask the user for active and specific acceptance to do so.

If a runtime supports a permission system to control application access to the eye tracker, then the runtime **must** set the `isActive` field to `XR_FALSE` on the supplied XrActionStatePose structure, and **must** clear `XR_SPACE_LOCATION_POSITION_TRACKED_BIT`, `XR_SPACE_LOCATION_POSITION_VALID_BIT`, `XR_SPACE_LOCATION_ORIENTATION_TRACKED_BIT` and `XR_SPACE_LOCATION_ORIENTATION_VALID_BIT` when locating using the tracked space until the application has been allowed access to the eye tracker. When the application access has been allowed, the runtime **may** set `isActive` on the supplied XrActionStatePose structure to `XR_TRUE` and **may** set `XR_SPACE_LOCATION_POSITION_TRACKED_BIT`, `XR_SPACE_LOCATION_POSITION_VALID_BIT` `XR_SPACE_LOCATION_ORIENTATION_TRACKED_BIT` and `XR_SPACE_LOCATION_ORIENTATION_VALID_BIT` when locating using the tracked space.

12.32.2. Device enumeration

When the eye gaze input extension is enabled, an application **may** pass in a XrSystemEyeGazeInteractionPropertiesEXT structure in the XrSystemProperties::`next` chain structure when calling xrGetSystemProperties to acquire information about the connected eye tracker.

The runtime **must** populate the XrSystemEyeGazeInteractionPropertiesEXT structure with the relevant information to the XrSystemProperties returned by the xrGetSystemProperties call.

```
// Provided by XR_EXT_eye_gaze_interaction
typedef struct XrSystemEyeGazeInteractionPropertiesEXT {
    XrStructureType    type;
    void*              next;
    XrBool32           supportsEyeGazeInteraction;
} XrSystemEyeGazeInteractionPropertiesEXT;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension. For more details about the structure chain, see the structure being extended (XrSystemProperties).
- `supportsEyeGazeInteraction` the runtime **must** set this value to `XR_TRUE` when eye gaze sufficient for use cases such as aiming or targeting is supported by the current device, otherwise the runtime **must** set this to `XR_FALSE`.

Regardless of the value of `supportsEyeGazeInteraction`, xrSuggestInteractionProfileBindings **must** accept eye gaze binding paths. However, if `supportsEyeGazeInteraction` is `XR_FALSE`, the runtime **should** never report an action as active if its only suggested binding is an eye gaze binding path.

Valid Usage (Implicit)

- The `XR_EXT_eye_gaze_interaction` extension **must** be enabled prior to using XrSystemEyeGazeInteractionPropertiesEXT
- `type` **must** be `XR_TYPE_SYSTEM_EYE_GAZE_INTERACTION_PROPERTIES_EXT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

12.32.3. Eye gaze input

This extension exposes a new interaction profile path */interaction_profiles/ext/eye_gaze_interaction* that is valid for the user path

- */user/eyes_ext*

with supported input subpath

- *.../input/gaze_ext/pose*

Note

The interaction profile path */interaction_profiles/ext/eye_gaze_interaction* defined here does not follow current rules for interaction profile names. If this extension were introduced today, it would be called */interaction_profiles/ext/eye_gaze_interaction_ext*, to allow for modifications when promoted to a KHR extension or the core specification.

The eye gaze pose is natively oriented with +Y up, +X to the right, and -Z forward and not gravity-aligned, similar to the `XR_REFERENCE_SPACE_TYPE_VIEW`. The eye gaze pose **may** originate from a point

positioned between the user's eyes. The tracking state of position and orientation (direction) is coupled: at any point in time, both the position and direction of the eye pose is either tracked or untracked. This means that the runtime **must** either set both XR_SPACE_LOCATION_POSITION_TRACKED_BIT and XR_SPACE_LOCATION_ORIENTATION_TRACKED_BIT or clear both XR_SPACE_LOCATION_POSITION_TRACKED_BIT and XR_SPACE_LOCATION_ORIENTATION_TRACKED_BIT.

When the runtime provides a "nominal" (high quality) eye gaze pose, the XR_SPACE_LOCATION_POSITION_TRACKED_BIT **must** be set if the eye otherwise has a fully-tracked pose relative to the other space. A runtime **can** provide a sub-nominal eye gaze pose but **must** then clear the XR_SPACE_LOCATION_POSITION_TRACKED_BIT. A nominal eye gaze pose is suitable for use cases such as aiming or targeting, while a sub-nominal eye gaze pose has degraded performance and **should** not be relied on for all input scenarios. Applications **should** be very careful when using sub-nominal eye gaze pose, since behavior varies considerably for different users and manufacturers, and some manufacturers **may** choose to not provide sub-nominal eye gaze pose at all.

With current technology, some eye trackers **may** need to undergo an explicit calibration routine to provide a nominal accurate and precise eye gaze pose. If the eye tracker is in an uncalibrated state when the first call to xrSyncActions is made with an eye gaze action enabled, then the runtime **should** request eye tracker calibration from the user if it has not yet been requested.

```
// Provided by XR_EXT_eye_gaze_interaction
typedef struct XrEyeGazeSampleTimeEXT {
    XrStructureType    type;
    void*              next;
    XrTime             time;
} XrEyeGazeSampleTimeEXT;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- time is when in time the eye gaze pose is expressed.

One particularity for eye trackers compared to most other spatial input is that the runtime **may** lack the capability to predict or interpolate eye gaze poses. Runtimes that are unable to predict or interpolate eye gaze poses **must** clamp the gaze pose requested in the xrLocateSpace call to the value nearest to time requested in the call. To allow for an application to reason about high accuracy eye tracking, the application **can** chain in an XrEyeGazeSampleTimeEXT to the next pointer of the XrSpaceLocation structure passed into the xrLocateSpace call. The runtime **must** set time in the XrEyeGazeSampleTimeEXT structure to the clamped, predicted, or interpolated time. The application **should** inspect the XrEyeGazeSampleTimeEXT::time field to understand when in time the pose is expressed. The time field **may** be in the future if a runtime is able to predict gaze poses. The runtime

must set the `time` field to 0 if the sample time is not available.

Valid Usage (Implicit)

- The `XR_EXT_eye_gaze_interaction` extension **must** be enabled prior to using XrEyeGazeSampleTimeEXT
- `type` **must** be `XR_TYPE_EYE_GAZE_SAMPLE_TIME_EXT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

12.32.4. Sample code

The following example code shows how to bind the eye pose to the action system.

```
extern XrInstance instance;
extern XrSession session;
extern XrPosef pose_identity;

// Create action set
XrActionSetCreateInfo actionSetInfo {
  .type = XR_TYPE_ACTION_SET_CREATE_INFO,
  .actionSetName = "gameplay",
  .localizedActionSetName = "Gameplay",
  .priority = 0,
};
XrActionSet gameplayActionSet {XR_NULL_HANDLE};
CHK_XR(xrCreateActionSet(instance, &actionSetInfo, &gameplayActionSet));

// Create user intent action
XrActionCreateInfo actionInfo {
  .type = XR_TYPE_ACTION_CREATE_INFO,
  .actionName = "user_intent",
  .actionType = XR_ACTION_TYPE_POSE_INPUT,
  .localizedActionName = "User Intent",
};
XrAction userIntentAction {XR_NULL_HANDLE};
CHK_XR(xrCreateAction(gameplayActionSet, &actionInfo, &userIntentAction));

// Create suggested bindings
XrPath eyeGazeInteractionProfilePath {XR_NULL_PATH};
CHK_XR(xrStringToPath(instance, "/interaction_profiles/ext/eye_gaze_interaction",
&eyeGazeInteractionProfilePath));

XrPath gazePosePath {XR_NULL_PATH};
CHK_XR(xrStringToPath(instance, "/user/eyes_ext/input/gaze_ext/pose", &gazePosePath));
```

```
XrActionSuggestedBinding bindings {
  .action = userIntentAction,
  .binding = gazePosePath,
};

XrInteractionProfileSuggestedBinding suggestedBindings{
  .type = XR_TYPE_INTERACTION_PROFILE_SUGGESTED_BINDING,
  .interactionProfile = eyeGazeInteractionProfilePath,
  .countSuggestedBindings = 1,
  .suggestedBindings = &bindings,
};
CHK_XR(xrSuggestInteractionProfileBindings(instance, &suggestedBindings));

XrSessionActionSetsAttachInfo attachInfo{
  .type = XR_TYPE_SESSION_ACTION_SETS_ATTACH_INFO,
  .countActionSets = 1,
  .actionSets = &gameplayActionSet,
};
CHK_XR(xrAttachSessionActionSets(session, &attachInfo));

XrActionSpaceCreateInfo createActionSpaceInfo{
  .type = XR_TYPE_ACTION_SPACE_CREATE_INFO,
  .action = userIntentAction,
  .poseInActionSpace = pose_identity,
};
XrSpace gazeActionSpace {XR_NULL_HANDLE};
CHK_XR(xrCreateActionSpace(session, &createActionSpaceInfo, &gazeActionSpace));

XrReferenceSpaceCreateInfo createReferenceSpaceInfo{
  .type = XR_TYPE_REFERENCE_SPACE_CREATE_INFO,
  .referenceSpaceType = XR_REFERENCE_SPACE_TYPE_LOCAL,
  .poseInReferenceSpace = pose_identity,
};
XrSpace localReferenceSpace {XR_NULL_HANDLE};
CHK_XR(xrCreateReferenceSpace(session, &createReferenceSpaceInfo, &localReferenceSpace));

while(true)
{
  XrActiveActionSet activeActionSet{
    .actionSet = gameplayActionSet,
    .subactionPath = XR_NULL_PATH,
  };
  XrTime time = 0;

  XrActionsSyncInfo syncInfo{
    .type = XR_TYPE_ACTIONS_SYNC_INFO,
    .countActiveActionSets = 1,
```

```
    .activeActionSets = &activeActionSet,
  };
  CHK_XR(xrSyncActions(session, &syncInfo));

  XrActionStatePose actionStatePose{
    .type = XR_TYPE_ACTION_STATE_POSE,
  };
  XrActionStateGetInfo getActionStateInfo{
    .type = XR_TYPE_ACTION_STATE_GET_INFO,
    .action = userIntentAction,
  };
  CHK_XR(xrGetActionStatePose(session, &getActionStateInfo, &actionStatePose));

  if(actionStatePose.isActive){
    XrEyeGazeSampleTimeEXT eyeGazeSampleTime{
      .type = XR_TYPE_EYE_GAZE_SAMPLE_TIME_EXT,
    };
    XrSpaceLocation gazeLocation{
      .type = XR_TYPE_SPACE_LOCATION,
      .next = &eyeGazeSampleTime,
    };
    CHK_XR(xrLocateSpace(gazeActionSpace, localReferenceSpace, time, &gazeLocation));

    // Do things
  }
}
```

Version History

- Revision 1, 2020-02-20 (Denny Rönngren)
  - Initial version
- Revision 2, 2022-05-27 (Bryce Hutchings)
  - Remove error-prone `XrEyeGazeSampleTimeEXT` validation requirement

12.33. XR_EXT_frame_synthesis

Name String

`XR_EXT_frame_synthesis`

Extension Type

Instance extension

Registered Extension Number

212

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Contributors

Jian Zhang, Meta Platforms
Neel Bedekar, Meta Platforms
Xiang Wei, Meta Platforms
Guodong Rong, Meta Platforms
Trevor Dasch, Meta Platforms
Rémi Arnaud, Varjo
Paulo Gomes, Samsung Electronics
Bryce Hutchings, Microsoft
Rylie Pavlik, Collabora
Shuai Liu, ByteDance

12.33.1. Overview

This extension provides support to enable frame synthesis on applications based on additional application provided data. Application generated motion vector images and depth images **may** be used by the runtime to do high quality frame extrapolation and reprojection to synthesize a new frame, providing a smooth experience even when the application is running below the FPS target.

This extension is designed to be independent of `XR_KHR_composition_layer_depth`, and both **may** be enabled and used at the same time, for different purposes. The XrFrameSynthesisInfoEXT ::`depthSubImage` **may** use depth data dedicated for frame synthesis, and its resolution **may** be lower than XrCompositionLayerDepthInfoKHR::`subImage`. See XrFrameSynthesisConfigViewEXT for the suggested resolution of `depthSubImage`.

12.33.2. Submit motion vector images and depth images

The XrFrameSynthesisInfoEXT structure is defined as:

```
// Provided by XR_EXT_frame_synthesis
typedef struct XrFrameSynthesisInfoEXT {
    XrStructureType                 type;
    const void*                     next;
    XrFrameSynthesisInfoFlagsEXT    layerFlags;
    XrSwapchainSubImage             motionVectorSubImage;
    XrVector4f                      motionVectorScale;
    XrVector4f                      motionVectorOffset;
    XrPosef                         appSpaceDeltaPose;
    XrSwapchainSubImage             depthSubImage;
    float                           minDepth;
    float                           maxDepth;
    float                           nearZ;
    float                           farZ;
} XrFrameSynthesisInfoEXT;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `layerFlags` is a bitmask of XrFrameSynthesisInfoFlagsEXT.
- `motionVectorSubImage` identifies the motion vector image XrSwapchainSubImage to be used for frame synthesis.
- `motionVectorScale` is an XrVector4f which scales the interpretation of the motion vector in `motionVectorSubImage`.
- `motionVectorOffset` is an XrVector4f which offsets the interpretation of the motion vector in `motionVectorSubImage`.
- `appSpaceDeltaPose` is the incremental application-applied transform, if any, that affects the view since the previous frame. Experiences of artificial locomotion (scripted movement, teleportation, etc.) involve the application transforming the whole XrCompositionLayerProjection::`space` from one application space pose to another pose between frames, affecting the view by more than the tracked movement between frames. The `appSpaceDeltaPose` **must** be identity when there is no XrCompositionLayerProjection::`space` transformation in the application. The XrPosef::`position` in `appSpaceDeltaPose` **must** be relative to the current application XrCompositionLayerProjection::`space`.
- `depthSubImage` identifies the depth image XrSwapchainSubImage to be used for frame synthesis.
- `minDepth` and `maxDepth` are the range of depth values in the `depthSubImage`, and **must** be in the range of [0.0,1.0]. This is akin to min and max values of OpenGL's `glDepthRange`, but with the requirement here that maxDepth ≥ minDepth.
- `nearZ` is the positive distance in meters of the `minDepth` value in the depth swapchain. Applications **may** use a `nearZ` that is greater than `farZ` to indicate depth values are reversed. `nearZ` **may** be infinite.
- `farZ` is the positive distance in meters of the `maxDepth` value in the depth swapchain. `farZ` **may** be infinite. The runtime **must** return error `XR_ERROR_VALIDATION_FAILURE` if `nearZ` == `farZ`

When submitting motion vector images and depth images along with projection layers, add an XrFrameSynthesisInfoEXT structure to the XrCompositionLayerProjectionView::`next` chain, for each XrCompositionLayerProjectionView structure in the given layer.

The runtime **must** interpret the motion vector data in the `motionVectorSubImage`'s RGB channels, modified by `motionVectorScale` and `motionVectorOffset` as follows: motionVector = $\text{motionVectorSubImage}_{rgb}$ * $\text{motionVectorScale}_{xyz}$ + $\text{motionVectorOffset}_{xyz}$. The components $\text{motionVectorSubImage}_{a}$, $\text{motionVectorScale}_{w}$ and $\text{motionVectorOffset}_{w}$ are ignored.

The motion vector represents the movement of a pixel since the XrFrameEndInfo::displayTime of the

previous frame until the XrFrameEndInfo::displayTime of the current frame. The runtime **may** use this information to extrapolate the rendered frame into a future frame.

The motion vector **must** derived from normalized device coordinate (NDC) space, which in this context uses Vulkan-style conventions: the NDC range is defined as [-1, -1, 0] to [1, 1, 1], different from OpenGL's NDC range. However, the motion vector itself is not constrained to this range; its values depend on the pixel's movement and may extend beyond the boundaries of the NDC space. For example, given that a pixel's NDC in the previous frame is PrevNDC, and CurrNDC in current frame, and that there is no scale or offset, then the motion vector value is "$(\text{CurrNDC} - \text{PrevNDC})_{xyz}$".

Valid Usage (Implicit)

- The `XR_EXT_frame_synthesis` extension **must** be enabled prior to using XrFrameSynthesisInfoEXT
- `type` **must** be `XR_TYPE_FRAME_SYNTHESIS_INFO_EXT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `layerFlags` **must** be `0` or a valid combination of XrFrameSynthesisInfoFlagBitsEXT values
- `motionVectorSubImage` **must** be a valid XrSwapchainSubImage structure
- `depthSubImage` **must** be a valid XrSwapchainSubImage structure



Note

There are many different ways to generate `motionVectorSubImage` and `depthSubImage`. For example, the application may render them in a lower resolution dedicated motion vector pass (see XrFrameSynthesisConfigViewEXT for recommended resolution), or render them in the main view pass with MRT (Multiple Render Targets). It is up to the application to decide the specific approach. Signed 16 bit float per pixel channel formats are recommended for `motionVectorSubImage`.

12.33.3. Frame synthesis flags

```
// Provided by XR_EXT_frame_synthesis
typedef XrFlags64 XrFrameSynthesisInfoFlagsEXT;
```

```
// Provided by XR_EXT_frame_synthesis
// Flag bits for XrFrameSynthesisInfoFlagsEXT
static const XrFrameSynthesisInfoFlagsEXT
XR_FRAME_SYNTHESIS_INFO_USE_2D_MOTION_VECTOR_BIT_EXT = 0x00000001;
static const XrFrameSynthesisInfoFlagsEXT
XR_FRAME_SYNTHESIS_INFO_REQUEST_RELAXED_FRAME_INTERVAL_BIT_EXT = 0x00000002;
```

Flag Descriptions

- `XR_FRAME_SYNTHESIS_INFO_USE_2D_MOTION_VECTOR_BIT_EXT` indicates 2D motion vector data is used.
- `XR_FRAME_SYNTHESIS_INFO_REQUEST_RELAXED_FRAME_INTERVAL_BIT_EXT` provides a hint to the runtime that the application prefers additional time to render each frame, with the trade-off that more frames would be synthesized by the runtime. This may be useful for applications that prefer to conserve CPU and GPU utilization (e.g. for increased battery life) or to increase the compute budget of each frame. The runtime **may** request frames at any interval, regardless of the value of this flag.

By default, 3D motion vector data is expected by the runtime, so $\text{motionVectorSubImage}_{rgb}$, $\text{motionVectorScale}_{xyz}$ and $\text{motionVectorOffset}_{xyz}$ are used, as described in XrFrameSynthesisInfoEXT.

When `XR_FRAME_SYNTHESIS_INFO_USE_2D_MOTION_VECTOR_BIT_EXT` is enabled on XrFrameSynthesisInfoEXT `layerFlags`, the runtime instead interprets the submitted motion vector image as 2D motion vector data, representing 2D pixel movement from the previous frame to the current frame. Pixels values are interpreted as follows for 2D motion vector data: $\text{motionVector} = \text{motionVectorSubImage}_{rg} * \text{motionVectorScale}_{xy} + \text{motionVectorOffset}_{xy}$. The components $\text{motionVectorSubImage}_{ba}$, $\text{motionVectorScale}_{zw}$ and $\text{motionVectorOffset}_{zw}$ are ignored. Using 2D instead of 3D motion vector data **may** decrease the quality of the synthesized frames.

12.33.4. Get recommended resolution

The XrFrameSynthesisConfigViewEXT structure is defined as:

```
// Provided by XR_EXT_frame_synthesis
typedef struct XrFrameSynthesisConfigViewEXT {
    XrStructureType    type;
    void*              next;
    uint32_t           recommendedMotionVectorImageRectWidth;
    uint32_t           recommendedMotionVectorImageRectHeight;
} XrFrameSynthesisConfigViewEXT;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `recommendedMotionVectorImageRectWidth`: recommended motion vector and depth image width.
- `recommendedMotionVectorImageRectHeight`: recommended motion vector and depth image height.

When this extension is enabled, an application **can** pass in an XrFrameSynthesisConfigViewEXT structure in the XrViewConfigurationView::`next` chain when calling xrEnumerateViewConfigurationViews to acquire information about the recommended motion vector image resolution.

Valid Usage (Implicit)

- The `XR_EXT_frame_synthesis` extension **must** be enabled prior to using XrFrameSynthesisConfigViewEXT
- `type` **must** be `XR_TYPE_FRAME_SYNTHESIS_CONFIG_VIEW_EXT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

12.33.5. New Structures

- Extending XrCompositionLayerProjectionView:
  - XrFrameSynthesisInfoEXT
- Extending XrViewConfigurationView:
  - XrFrameSynthesisConfigViewEXT

12.33.6. New Enums

- XrFrameSynthesisInfoFlagBitsEXT

12.33.7. New Bitmasks

- XrFrameSynthesisInfoFlagsEXT

12.33.8. New Enum Constants

- `XR_EXT_FRAME_SYNTHESIS_EXTENSION_NAME`
- `XR_EXT_frame_synthesis_SPEC_VERSION`

- Extending XrStructureType:
  - XR_TYPE_FRAME_SYNTHESIS_CONFIG_VIEW_EXT
  - XR_TYPE_FRAME_SYNTHESIS_INFO_EXT

Issues

Version History

- Revision 1, 2022-01-31 (Jian Zhang)
  - Initial extension description, converted from fb_space_warp
  - Collaborating with contributors to refine the extension interfaces.

12.34. XR_EXT_future

Name String

XR_EXT_future

Extension Type

Instance extension

Registered Extension Number

470

Revision

2

Ratification Status

Ratified

Extension and Version Dependencies

OpenXR 1.0

Contributors

Bryce Hutchings, Microsoft
Andreas Selvik, Meta
Ron Bessems, Magic Leap
Yin Li, Microsoft Corporation
Baolin Fu, ByteDance
Cass Everitt, Meta Platforms
Charlton Rodda, Collabora
Jakob Bornecrantz, NVIDIA
John Kearney, Meta Platforms
Jonathan Wright, Meta Platforms

Jun Yan, ByteDance
Junyi Wang, ByteDance
Karthik Kadappan, Magic Leap
Natalie Fleury, Meta Platforms
Nathan Nuber, Valve
Nikita Lutsenko, Meta Platforms
Robert Blenkinsopp, Ultraleap
Rylie Pavlik, Collabora
Tim Mowrer, Meta Platforms
Wenlin Mao, Meta Platforms
Will Fu, ByteDance
Zhipeng Liu, ByteDance

12.34.1. Overview

In XR systems there are certain operations that are long running and do not reasonably complete within a normal frame loop. This extension introduces the concept of a *future* which supports creation of asynchronous (async) functions for such long running operations. This extension does not include any asynchronous operations: it is expected that other extensions will use these *futures* and their associated conventions in this extension to define their asynchronous operations.

An `XrFutureEXT` represents the future result of an asynchronous operation, comprising an `XrResult` and possibly additional outputs. Long running operations immediately return an `XrFutureEXT` when started, letting the application poll the state of the future, and get the result once ready by calling a "complete"-function.

12.34.2. Getting a future

The `XrFutureEXT` basetype is defined as:

```
// Provided by XR_EXT_future
XR_DEFINE_OPAQUE_64(XrFutureEXT)
```

Asynchronous functions return an `XrFutureEXT` token as a placeholder for a value that will be returned later. An `XrFutureEXT` returned by a successful call to a function starting an asynchronous operation **should** normally start in the `XR_FUTURE_STATE_PENDING_EXT` state, but **may** skip directly to `XR_FUTURE_STATE_READY_EXT` if the result is immediately available.

The value `XR_NULL_FUTURE_EXT`, numerically equal to `0`, is never a valid `XrFutureEXT` value.

Note that an `XrFutureEXT` token is neither a handle nor an atom type (such as `XrPath`). It belongs to a new category and is defined as an opaque 64-bit value. See Future Scope for details on the scope and lifecycle of a future.

Style note: Functions that return an `XrFutureEXT` **should** be named with the suffix "Async", e.g.

xrPerformLongTaskAsync. This function **must** not set the XrFutureEXT to XR_NULL_FUTURE_EXT when the function returns XR_SUCCESS.

12.34.3. Waiting for a future to become ready

The xrPollFutureEXT function is defined as:

```
// Provided by XR_EXT_future
XrResult xrPollFutureEXT(
    XrInstance                                  instance,
    const XrFuturePollInfoEXT*                  pollInfo,
    XrFuturePollResultEXT*                      pollResult);
```

Parameter Descriptions

- instance is an XrInstance handle
- pollInfo is a pointer to an XrFuturePollInfoEXT structure.
- pollResult is a pointer to an XrFuturePollResultEXT structure to be populated on a successful call.

Applications **can** use this function to check the current state of a future, typically while waiting for the async operation to complete and the future to become "ready" to complete.



Note

Each XrFutureEXT value **must** be externally synchronized by the application when calling completion, polling, and cancellation functions, and when destroying the associated handle.

Valid Usage (Implicit)

- The XR_EXT_future extension **must** be enabled prior to calling xrPollFutureEXT
- instance **must** be a valid XrInstance handle
- pollInfo **must** be a pointer to a valid XrFuturePollInfoEXT structure
- pollResult **must** be a pointer to an XrFuturePollResultEXT structure

Return Codes

Success

- XR_SUCCESS

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_FUTURE_INVALID_EXT
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_VALIDATION_FAILURE

The XrFuturePollInfoEXT structure is defined as:

```
// Provided by XR_EXT_future
typedef struct XrFuturePollInfoEXT {
    XrStructureType    type;
    const void*        next;
    XrFutureEXT        future;
} XrFuturePollInfoEXT;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain.
- future is the XrFutureEXT future being polled.

An XrFuturePollInfoEXT structure is used to pass future to xrPollFutureEXT.

Valid Usage (Implicit)

- The XR_EXT_future extension **must** be enabled prior to using XrFuturePollInfoEXT
- type **must** be XR_TYPE_FUTURE_POLL_INFO_EXT
- next **must** be NULL or a valid pointer to the next structure in a structure chain

The XrFuturePollResultEXT structure is defined as:

```
// Provided by XR_EXT_future
typedef struct XrFuturePollResultEXT {
    XrStructureType     type;
    void*               next;
    XrFutureStateEXT    state;
} XrFuturePollResultEXT;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `state` is the XrFutureStateEXT of the `XrFutureEXT` passed to xrPollFutureEXT.

An XrFuturePollResultEXT structure is used to return the result of xrPollFutureEXT.

Valid Usage (Implicit)

- The `XR_EXT_future` extension **must** be enabled prior to using XrFuturePollResultEXT
- `type` **must** be `XR_TYPE_FUTURE_POLL_RESULT_EXT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain. See also: XrFuturePollResultProgressBD

12.34.4. Completing a Future

Extensions that provide async functions returning a future **should** also provide a matching completion function to "complete" the future in order to return the result of the asynchronous operation. This function **should** be named with the suffix "Complete" replacing the "Async" suffix, e.g. `xrPerformLongTaskComplete` is a suitable completion function name corresponding to `xrPerformLongTaskAsync`.

A completion function **must** populate a structure that **must** be based on XrFutureCompletionBaseHeaderEXT to return the result of the asynchronous operation. Such a structure **may** be `static_cast` to and from XrFutureCompletionBaseHeaderEXT, allowing generic handling of the asynchronous operation results as well as polymorphic output from such an operation. The XrResult returned from a completion function **must** not be used to return the result of the asynchronous operation. Instead, the XrResult returned from a completion function **must** indicate both whether the completion function was called correctly, and if the completion of the future succeeded.

For instance, a completion function returning `XR_ERROR_HANDLE_INVALID` means that a handle passed to the completion function was invalid, not that a handle associated with the asynchronous operation is

invalid. Note that XR_SUCCESS **should** be returned from the completion function even if the asynchronous operation itself was a failure; that failure is indicated in XrFutureCompletionBaseHeaderEXT::futureResult rather than the return value of the completion function.

When a completion function is called with a future that is in the XR_FUTURE_STATE_PENDING_EXT state, the runtime **must** return XR_ERROR_FUTURE_PENDING_EXT.

The XrResult of the asynchronous operation **must** be returned in the futureResult of the return structure extending XrFutureCompletionBaseHeaderEXT. Completion functions which only need to return an XrResult **may** populate the XrFutureCompletionEXT structure provided by this extension as their output structure.

If the asynchronous operation creates a handle, the runtime **must** not queue any events for that handle until the corresponding completion function has been called. For asynchronous operations that do not create a handle, the result of the asynchronous operation **may** be observable through other functions even before the completion function has been called, e.g. the runtime **may** mark an object as "started" when the XrFutureEXT is in ready state, without waiting for the completion function to be called. Alternatively, the runtime **may** use the invocation of the completion function to set those observable states, e.g. the runtime **may** mark an object as "started" only when the completion function is called. Extensions exposing asynchronous functions **should** clarify when the observable states of the asynchronous operation change.

Once a completion function is called on a future with a valid output structure and returns XR_SUCCESS, the future is considered **completed**, and therefore **invalidated**. Any usage of this future thereafter **must** return XR_ERROR_FUTURE_INVALID_EXT.

Passing a completed future to any function accepting futures **must** return XR_ERROR_FUTURE_INVALID_EXT.

The runtime **may** release any resources associated with an XrFutureEXT once the future has been completed or invalidated.



Note

Each XrFutureEXT value **must** be externally synchronized by the application when calling completion, polling, and cancellation functions, and when destroying the associated handle.

The XrFutureCompletionBaseHeaderEXT structure is defined as:

```
// Provided by XR_EXT_future
typedef struct XrFutureCompletionBaseHeaderEXT {
    XrStructureType    type;
    void*              next;
    XrResult           futureResult;
} XrFutureCompletionBaseHeaderEXT;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `futureResult` is XrResult of the async operation associated with future passed to the completion function.

XrFutureCompletionBaseHeaderEXT is a base header for the result of a future completion function.

Valid Usage (Implicit)

- The `XR_EXT_future` extension **must** be enabled prior to using XrFutureCompletionBaseHeaderEXT
- `type` **must** be one of the following XrStructureType values: `XR_TYPE_CREATE_SPATIAL_ANCHORS_COMPLETION_ML`, `XR_TYPE_CREATE_SPATIAL_CONTEXT_COMPLETION_EXT`, `XR_TYPE_CREATE_SPATIAL_DISCOVERY_SNAPSHOT_COMPLETION_EXT`, `XR_TYPE_CREATE_SPATIAL_PERSISTENCE_CONTEXT_COMPLETION_EXT`, `XR_TYPE_CREATE_TRACKABLE_IMAGE_DATABASE_COMPLETION_ANDROID`, `XR_TYPE_ENVIRONMENT_RAYCASTER_CREATE_COMPLETION_META`, `XR_TYPE_FUTURE_COMPLETION_EXT`, `XR_TYPE_PERSIST_SPATIAL_ENTITY_COMPLETION_EXT`, `XR_TYPE_SENSE_DATA_QUERY_COMPLETION_BD`, `XR_TYPE_SPATIAL_ANCHOR_CREATE_COMPLETION_BD`, `XR_TYPE_SPATIAL_ANCHORS_DELETE_COMPLETION_ML`, `XR_TYPE_SPATIAL_ANCHORS_PUBLISH_COMPLETION_ML`, `XR_TYPE_SPATIAL_ANCHORS_QUERY_COMPLETION_ML`, `XR_TYPE_SPATIAL_ANCHORS_UPDATE_EXPIRATION_COMPLETION_ML`, `XR_TYPE_SURFACE_ANCHOR_CREATE_COMPLETION_ANDROID`, `XR_TYPE_UNPERSIST_SPATIAL_ENTITY_COMPLETION_EXT`, `XR_TYPE_VPS_AVAILABILITY_CHECK_COMPLETION_ANDROID`, `XR_TYPE_WORLD_MESH_REQUEST_COMPLETION_ML`, `XR_TYPE_WORLD_MESH_STATE_REQUEST_COMPLETION_ML`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `futureResult` **must** be a valid XrResult value

The XrFutureCompletionEXT structure is defined as:

```
// Provided by XR_EXT_future
typedef struct XrFutureCompletionEXT {
    XrStructureType    type;
    void*              next;
    XrResult           futureResult;
} XrFutureCompletionEXT;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `futureResult` is XrResult of the async operation associated with future passed to the completion function.

This is a minimal implementation of XrFutureCompletionBaseHeaderEXT, containing only the fields present in the base header structure. It is intended for use by asynchronous operations that do not have other outputs or return values beyond an XrResult value, as the output parameter of their completion function.

Valid Usage (Implicit)

- The `XR_BD_spatial_sensing` extension **must** be enabled prior to using XrFutureCompletionEXT
- `type` **must** be `XR_TYPE_FUTURE_COMPLETION_EXT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `futureResult` **must** be a valid XrResult value

12.34.5. Two-Call Idiom in Asynchronous Operations

OpenXR uses a two-call idiom for interfaces that return arrays or buffers of variable size. Asynchronous operations returning such an array or buffer similarly use the structure style of that two-call idiom, with small modifications to the typical completion function conventions to account for this pattern.

For completion functions returning an array or buffer using the two-call idiom, the future **must** be marked as **completed** if the output array size is sufficient for all elements of the data and was thus populated by the completion function. If the output array size is not sufficient, the runtime **must** not mark the future as completed nor invalidated.

For an array of zero data elements, this means the first call to the two-call idiom completion function **must** mark the future as **completed** and invalidated, even if the array is a `NULL` pointer. If XrFutureCompletionBaseHeaderEXT::`futureResult` is a failure the runtime **must invalidate** the future after the first call, and any further usage of this future **must** return `XR_ERROR_FUTURE_INVALID_EXT`.

For non-zero output arrays where XrFutureCompletionBaseHeaderEXT::`futureResult` is not a failure, XrFutureCompletionBaseHeaderEXT::`futureResult` **must** be identical for both calls to the completion function.

This definition allows asynchronous operations to return dynamically sized outputs by using the two-

call idiom in a familiar way.

12.34.6. Cancelling a future

The xrCancelFutureEXT function is defined as:

```
// Provided by XR_EXT_future
XrResult xrCancelFutureEXT(
    XrInstance                                  instance,
    const XrFutureCancelInfoEXT*                cancelInfo);
```

Parameter Descriptions

- `instance` is an XrInstance handle
- `cancelInfo` is a pointer to an XrFutureCancelInfoEXT structure.

This function cancels the future and signals that the async operation is not required. After a future has been cancelled any functions using this future **must** return `XR_ERROR_FUTURE_INVALID_EXT`.

A runtime **may** stop the asynchronous operation associated with a future after an app has cancelled it.



Note

Each `XrFutureEXT` value **must** be externally synchronized by the application when calling completion, polling, and cancellation functions, or destroying the associated handle.

Valid Usage (Implicit)

- The `XR_EXT_future` extension **must** be enabled prior to calling xrCancelFutureEXT
- `instance` **must** be a valid XrInstance handle
- `cancelInfo` **must** be a pointer to a valid XrFutureCancelInfoEXT structure

Thread Safety

- Access to the `future` member of the `cancelInfo` parameter **must** be externally synchronized

Return Codes

Success

- XR_SUCCESS

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_FUTURE_INVALID_EXT
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_VALIDATION_FAILURE

The XrFutureCancelInfoEXT structure is defined as:

```
// Provided by XR_EXT_future
typedef struct XrFutureCancelInfoEXT {
    XrStructureType    type;
    const void*        next;
    XrFutureEXT        future;
} XrFutureCancelInfoEXT;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain.
- future is XrFutureEXT to cancel.

An XrFutureCancelInfoEXT describes which future to cancel.

Valid Usage (Implicit)

- The XR_EXT_future extension **must** be enabled prior to using XrFutureCancelInfoEXT
- type **must** be XR_TYPE_FUTURE_CANCEL_INFO_EXT
- next **must** be NULL or a valid pointer to the next structure in a structure chain

12.34.7. XrFutureEXT Lifecycle

The XrFutureStateEXT enumerates the possible future lifecycle states:

```
// Provided by XR_EXT_future
typedef enum XrFutureStateEXT {
    XR_FUTURE_STATE_PENDING_EXT = 1,
    XR_FUTURE_STATE_READY_EXT = 2,
    XR_FUTURE_STATE_MAX_ENUM_EXT = 0x7FFFFFFF
} XrFutureStateEXT;
```

Enumerant Descriptions

- XR_FUTURE_STATE_PENDING_EXT. The state of a future that is waiting for the async operation to conclude. This is typically the initial state of a future returned from an async function.
- XR_FUTURE_STATE_READY_EXT. The state of a future when the result of the async operation is ready. The application **can** retrieve the result by calling the associated completion function.

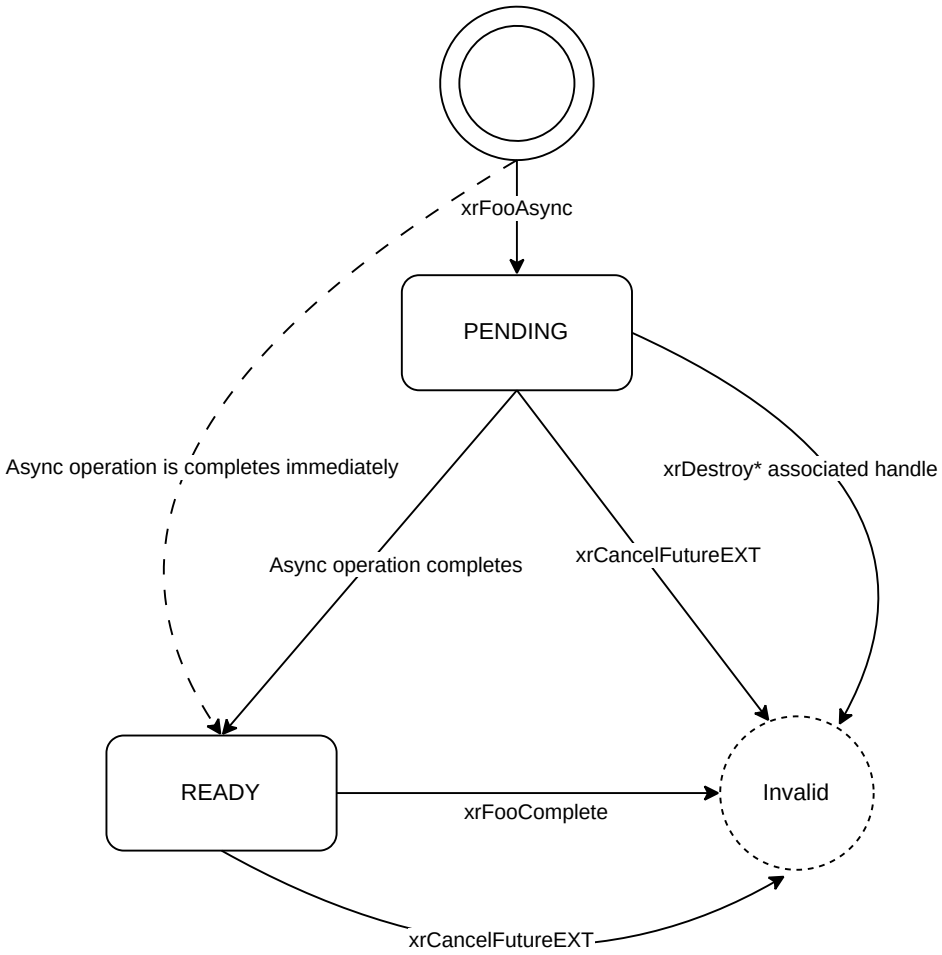


Figure 9. XrFutureEXT Lifecycle

A future that is not invalidated (or completed) **may** be in one of two states, `Pending` and `Ready`, represented by `XR_FUTURE_STATE_PENDING_EXT` and `XR_FUTURE_STATE_READY_EXT` respectively.

- When successfully returned from an async function the future starts out as `Pending`. In this state the future **may** be polled, but **must** not be passed to a completion function. Applications **should** wait for the future to become ready and keep polling the state of the future. If a pending future is passed to the associated completion function, it **must** return `XR_ERROR_FUTURE_PENDING_EXT`.
- Once the asynchronous operation succeeds or fails, the state of the future moves to `Ready`. In the ready state the future **may** be "Completed" with the `Complete` function. See Completing a Future.
- After being successfully completed, the future becomes invalidated if the completion function returns a success code, and in the case of two-call idioms, the array was not `NULL`.
- After a call to xrCancelFutureEXT, the future becomes invalidated immediately and any resources associated with it **may** be freed (including handles)
- When the associated handle is destroyed, the futures become invalidated. See Future Scope.

A future returned from an async function **must** be in either the state `XR_FUTURE_STATE_PENDING_EXT` or `XR_FUTURE_STATE_READY_EXT`. A runtime **may** skip the `Pending` state and go directly to `Ready` if the result is immediately available.

12.34.8. Future Scope

An `XrFutureEXT` is scoped to the "associated handle" of the future. The associated handle is the handle passed to the asynchronous operation that returns the `XrFutureEXT`. When the associated handle is destroyed, the runtime **must** invalidate the future and **may** free any associated resources.



Note

For example, for a hypothetical async function `xrGetFooAsync(Session session, XrFooGetInfo info, XrFutureEXT* future)` then `XrSession` is the associated handle, and if the app calls `xrDestroySession(…)` the returned future becomes invalid.

Likewise, for `xrRequestBar(BarGenerator barGenerator, XrBarGenerateInfo info, XrFutureEXT* future)`, the hypothetical `BarGenerator` is the associated handle that scopes the future.

12.34.9. Extension Guidelines for Asynchronous Functions

Extensions exposing asynchronous functions using `XR_EXT_future` **should** follow the following patterns:

1. Functions returning a future **should** use the suffix "Async", prior to an author/vendor tag if applicable. For example:
   - `xrGetFooAsync(…)`
   - `xrRequestBarAsyncKHR(…)`
   - `xrCreateObjectAsyncVENDOR(…)`

2. The name of the future out parameter **should** be `future`. For example:
    - `xrGetFooAsync(…, XrFutureEXT* future)`
    - `xrRequestBarAsyncKHR(…, XrFutureEXT* future)`
    - `xrCreateObjectAsyncVENDOR(…, XrFutureEXT* future)`
3. Functions completing a future **should** match the name of the function returning the future, but with "Complete" rather than "Async" as the suffix. This is a deviation from the normal pattern in OpenXR, if "complete" is considered to be the verb; however this provides for a useful sorting order keeping the "Async" and "Complete" functions adjacent, and fits the pattern of using suffixes for asynchronous functions. The completion function **must** use the same handle type as the corresponding async function and the runtime **must** return `XR_ERROR_HANDLE_INVALID` if the handle value passed to the completion function is different from the value passed to the async function that returned the future. For example:
    - `xrGetFooComplete(…)`
    - `xrRequestBarCompleteKHR(…)`,
    - `xrCreateObjectCompleteVENDOR(…)`
4. The output structure used in the "Complete" function **should** extend `XrFutureCompletionBaseHeaderEXT` (starting with `type`, `next`, and `futureResult` fields).
5. If an operation requires more than the basic `XrFutureCompletionEXT` output, the output structure populated by the "Complete" function **should** be named based on the function that returned the future, with the suffix "Completion". For example:
    - `xrGetFooComplete` populates `XrGetFooCompletion`
    - `xrRequestBarComplete` populates `XrRequestBarCompletionKHR`
    - `xrCreateObjectCompleteVENDOR` populates `XrCreateObjectCompletionVENDOR`
6. The `XrFutureEXT` parameter in the "Complete" function **should** be named `future`. For example:
    - `xrGetFooComplete(…, XrFutureEXT future)`
    - `xrRequestBarCompleteKHR(…, XrFutureEXT future)`
    - `xrCreateObjectCompleteVENDOR(…, XrFutureEXT future)`
7. The parameter with the completion structure **should** be named `completion`. e.g.
    - `xrGetFooComplete(…, XrFutureEXT future, XrGetFooCompletion* completion)`
    - `xrRequestBarCompleteKHR(…, XrFutureEXT future, XrRequestBarCompletionKHR* completion)`
    - `xrCreateObjectCompleteVENDOR(…, XrFutureEXT future, XrCreateObjectCompletionVENDOR* completion)`

12.34.10. Asynchronous function patterns

`xrCreate` functions

```
/*****************************/
/* Foo extension definition */
/*****************************/
typedef void *XrFoo; // Handle definition
typedef struct XrFooObjectCreateInfo {
  XrStructureType type;
  const void *next;
} XrFooObjectCreateInfo;
#define XR_TYPE_FOO_OBJECT_CREATE_INFO ((XrStructureType)1100092000U)

// extends struct XrFutureCompletionBaseHeader using "parentstruct"
typedef struct XrFooObjectCreateCompletionEXT {
  XrStructureType type;
  void *XR_MAY_ALIAS next;
  XrResult futureResult;
  XrFoo foo;
} XrFooObjectCreateCompletionEXT;
#define XR_TYPE_FOO_OBJECT_CREATE_COMPLETION ((XrStructureType)1100092001U)

typedef XrResult(XRAPI_PTR *PFN_xrCreateFooObjectAsync)(
    XrSession session, const XrFooObjectCreateInfo *createInfo,
    XrFutureEXT *future);
typedef XrResult(XRAPI_PTR *PFN_xrCreateFooObjectComplete)(
    XrSession session, XrFutureEXT future,
    XrFooObjectCreateCompletionEXT *completion);

/*************************/
/* End Foo definition    */
/*************************/

PFN_xrCreateFooObjectAsync xrCreateFooObjectAsync; // previously initialized
PFN_xrCreateFooObjectComplete
    xrCreateFooObjectComplete;       // previously initialized
PFN_xrPollFutureEXT xrPollFutureEXT; // previously initialized
XrInstance instance;                 // previously initialized
XrSession session;                   // previously initialized

XrFutureEXT futureFooObject {XR_NULL_FUTURE_EXT};
XrResult result;

XrFooObjectCreateInfo createInfo{
  .type = XR_TYPE_FOO_OBJECT_CREATE_INFO,
};
result = xrCreateFooObjectAsync(session, &createInfo, &futureFooObject);
CHK_XR(result);
```

```
bool keepLooping = true;
bool futureReady = false;
while (keepLooping) {
  XrFuturePollInfoEXT pollInfo{
    .type = XR_TYPE_FUTURE_POLL_INFO_EXT,
    .future = futureFooObject,
  };
  XrFuturePollResultEXT pollResult{
    .type = XR_TYPE_FUTURE_POLL_RESULT_EXT,
  };
  CHK_XR(xrPollFutureEXT(instance, &pollInfo, &pollResult));

  if (pollResult.state == XR_FUTURE_STATE_READY_EXT) {
    futureReady = true;
    keepLooping = false;
  } else {
    // sleep(10);
  }
}

if (futureReady) {
  XrFooObjectCreateCompletionEXT completion{
    .type = XR_TYPE_FOO_OBJECT_CREATE_COMPLETION,
  };
  result = xrCreateFooObjectComplete(session, futureFooObject, &completion);
  CHK_XR(result);                  // Result of the complete function
  CHK_XR(completion.futureResult); // Return code of the create function
  // completion.fooObject is now valid and may be used!
}
```

Two-call idiom

```
/*****************************/
/* Foo extension definition */
/*****************************/
typedef struct XrFooObjectCreateInfo {
  XrStructureType type;
  const void *next;
} XrFooObjectCreateInfo;
#define XR_TYPE_FOO_OBJECTS_CREATE_INFO ((XrStructureType)1100092002U)

// extends struct XrFutureCompletionBaseHeader using "parentstruct"
typedef struct XrFooObjectsCreateCompletionEXT {
  XrStructureType type;
  void *next;
```

```
  XrResult futureResult;
  uint32_t elementCapacityInput;
  uint32_t elementCapacityOutput;
  float *elements;
} XrFooObjectsCreateCompletionEXT;
#define XR_TYPE_FOO_OBJECTS_CREATE_COMPLETION ((XrStructureType)1100092003U)

typedef XrResult(XRAPI_PTR *PFN_xrCreateFooObjectsAsync)(
    XrSession session, const XrFooObjectCreateInfo *createInfo,
    XrFutureEXT *future);
typedef XrResult(XRAPI_PTR *PFN_xrCreateFooObjectsComplete)(
    XrSession session, XrFutureEXT future,
    XrFooObjectsCreateCompletionEXT *completion);

/*************************/
/* End Foo definition    */
/*************************/

PFN_xrCreateFooObjectsAsync xrCreateFooObjectsAsync; // previously initialized
PFN_xrCreateFooObjectsComplete
    xrCreateFooObjectsComplete;      // previously initialized
PFN_xrPollFutureEXT xrPollFutureEXT; // previously initialized
XrInstance instance;                 // previously initialized
XrSession session;                   // previously initialized

XrFutureEXT futureFooObjects;
XrResult result;

XrFooObjectCreateInfo createInfo{
  .type = XR_TYPE_FOO_OBJECTS_CREATE_INFO,
};
result = xrCreateFooObjectsAsync(session, &createInfo, &futureFooObjects);
CHK_XR(result);

bool keepLooping = true;
bool futureReady = false;
while (keepLooping) {
  XrFuturePollInfoEXT pollInfo{
    .type = XR_TYPE_FUTURE_POLL_INFO_EXT,
    .future = futureFooObjects,
  };
  XrFuturePollResultEXT pollResult{
    .type = XR_TYPE_FUTURE_POLL_RESULT_EXT,
  };
  CHK_XR(xrPollFutureEXT(instance, &pollInfo, &pollResult));

  if (pollResult.state == XR_FUTURE_STATE_READY_EXT) {
    futureReady = true;
```

```
    keepLooping = false;
  } else {
    // sleep(10);
  }
}

if (futureReady) {
  XrFooObjectsCreateCompletionEXT completion{
    .type = XR_TYPE_FOO_OBJECTS_CREATE_COMPLETION
  };
  result = xrCreateFooObjectsComplete(session, futureFooObjects, &completion);
  CHK_XR(result); // Result of the complete function
  CHK_XR(completion.futureResult);

  // If elementCapacityOutput is 0, then the future is now complete / invalid
  if (completion.elementCapacityOutput != 0) {
    std::vector<float> floatValues(completion.elementCapacityOutput);
    completion.elementCapacityInput = static_cast<uint32_t>(floatValues.size());
    completion.elements = floatValues.data();

    result = xrCreateFooObjectsComplete(session, futureFooObjects, &completion);
    CHK_XR(result); // Result of the complete function
  }
}

// completion.elements has now been filled with values by the runtime.
```

Sample code

```
/*****************************************/
/* Slow Foo extension definition */
/*****************************************/
// extends struct XrFutureCompletionBaseHeader using "parentstruct"
typedef struct XrSlowFooCompletionEXT {
  XrStructureType type;
  void *XR_MAY_ALIAS next;
  XrResult futureResult;
  float foo;
} XrSlowFooCompletionEXT;
#define XR_TYPE_SLOW_FOO_COMPLETION_EXT ((XrStructureType)1100092005U)

typedef struct XrSlowFooInfoEXT {
  XrStructureType type;
  void *XR_MAY_ALIAS next;
} XrSlowFooInfoEXT;
#define XR_TYPE_SLOW_FOO_INFO_EXT ((XrStructureType)1100092006U)
```

```
typedef XrResult(XRAPI_PTR *PFN_xrSlowFooAsyncEXT)(XrSession session,
                                                   XrSlowFooInfoEXT slowFooInfo,
                                                   XrFutureEXT *future);
typedef XrResult(XRAPI_PTR *PFN_xrSlowFooCompleteEXT)(
    XrSession session, XrFutureEXT future, XrSlowFooCompletionEXT *completion);

/*******************************************/
/* End Slow Foo extension definition */
/*******************************************/

class MyGame {
  void OnSlowFooRequest() {
    if (m_slowFooFuture == XR_NULL_FUTURE_EXT) {
      // Make initial request.
      XrSlowFooInfoEXT fooInfo{
        .type = XR_TYPE_SLOW_FOO_INFO_EXT,
      };
      XrResult result = xrSlowFooAsyncEXT(session, fooInfo, &m_slowFooFuture);
      CHK_XR(result);
    }
  }

  void OnGameTickOrSomeOtherReoccurringFunction() {

    // Check if a future is outstanding
    if (m_slowFooFuture == XR_NULL_FUTURE_EXT) {
      return;
    }

    // Poll for state of future
    XrFuturePollInfoEXT pollInfo{
      .type = XR_TYPE_FUTURE_POLL_INFO_EXT,
      .future = m_slowFooFuture,
    };
    XrFuturePollResultEXT pollResult{
      .type = XR_TYPE_FUTURE_POLL_RESULT_EXT,
    };
    CHK_XR(xrPollFutureEXT(instance, &pollInfo, &pollResult));

    if (pollResult.state == XR_FUTURE_STATE_READY_EXT) {
      // Complete the future, consuming the result
      XrSlowFooCompletionEXT completion{
        .type = XR_TYPE_SLOW_FOO_COMPLETION_EXT,
      };
      XrResult result = xrSlowFooCompleteEXT(session, m_slowFooFuture, &completion);
      // Check XrResult from the completion function
      CHK_XR(result);
```

```
      // Check XrResult from the async operation
      CHK_XR(completion.futureResult);
      m_fooValue = completion.foo;
      m_slowFooFuture = XR_NULL_FUTURE_EXT;
    }
  }

  XrFutureEXT m_slowFooFuture {XR_NULL_FUTURE_EXT};
  float m_fooValue{0.0f};

  PFN_xrSlowFooAsyncEXT xrSlowFooAsyncEXT;       // previously initialized
  PFN_xrSlowFooCompleteEXT xrSlowFooCompleteEXT; // previously initialized
  PFN_xrPollFutureEXT xrPollFutureEXT;           // previously initialized
  XrInstance instance;                           // previously initialized
  XrSession session;                             // previously initialized
};
```

Multi-threaded code

```
class MyThreadedGame {

  MyThreadedGame() {
    // Start the thread
    m_processThread = std::thread(&MyThreadedGame::ThreadFunction, this);
    StartSlowFooRequest();
  }

  ~MyThreadedGame() {
    // all functions using futures must be synchronized.
    CancelSlowFooRequestFuture();
    m_abort = true;
    m_processThread.join();
  }

  void StartSlowFooRequest() {
    std::unique_lock<std::mutex> lock(m_mutex);
    if (m_slowFooFuture == XR_NULL_FUTURE_EXT) {
      // Make initial request.
      XrSlowFooInfoEXT fooInfo{
        .type = XR_TYPE_SLOW_FOO_INFO_EXT,
      };
      XrResult result = xrSlowFooAsyncEXT(session, fooInfo, &m_slowFooFuture);
      CHK_XR(result);
    }
  }
```

```
void CancelSlowFooRequestFuture() {
  std::unique_lock<std::mutex> lock(m_mutex);
  if (m_slowFooFuture != XR_NULL_FUTURE_EXT) {
    XrFutureCancelInfoEXT cancel_info{
      .type = XR_TYPE_FUTURE_CANCEL_INFO_EXT,
      .future = m_slowFooFuture,
    };
    xrCancelFutureEXT(instance, &cancel_info);
    m_slowFooFuture = XR_NULL_FUTURE_EXT;
  }
}

void CheckFooRequestCompletion() {

  std::unique_lock<std::mutex> lock(m_mutex);
  // Check if a future is outstanding
  if (m_slowFooFuture == XR_NULL_FUTURE_EXT) {
    return;
  }

  // Poll for state of future
  XrFuturePollInfoEXT pollInfo{
    .type = XR_TYPE_FUTURE_POLL_INFO_EXT,
    .future = m_slowFooFuture,
  };
  XrFuturePollResultEXT pollResult{
    .type = XR_TYPE_FUTURE_POLL_RESULT_EXT,
  };
  CHK_XR(xrPollFutureEXT(instance, &pollInfo, &pollResult));

  if (pollResult.state == XR_FUTURE_STATE_READY_EXT) {
    // Complete the future, consuming the result
    XrSlowFooCompletionEXT completion{
      .type = XR_TYPE_SLOW_FOO_COMPLETION_EXT,
    };
    XrResult result =
        xrSlowFooCompleteEXT(session, m_slowFooFuture, &completion);
    // Check XrResult from the completion function
    CHK_XR(result);
    // Check XrResult from the async operation
    CHK_XR(completion.futureResult);
    m_fooValue = completion.foo;
    m_slowFooFuture = XR_NULL_FUTURE_EXT;

    // Do something with the foo value.
  }
}
```

```
  void ThreadFunction() {
    while (!m_abort) {
      // other logic here

      CheckFooRequestCompletion();

      // sleep if needed.
    }
  }

  XrFutureEXT m_slowFooFuture{XR_NULL_FUTURE_EXT};
  float m_fooValue{0.0f};
  bool m_abort{false};
  std::mutex m_mutex;
  std::thread m_processThread;
};
```

New Base Types

- XrFutureEXT

New Functions

- xrPollFutureEXT
- xrCancelFutureEXT

New Structures

- XrFutureCompletionEXT
- XrFutureCompletionBaseHeaderEXT
- XrFuturePollInfoEXT
- XrFuturePollResultEXT
- XrFutureCancelInfoEXT

New Enum Constants

- XR_NULL_FUTURE_EXT

XrStructureType enumeration is extended with:

- XR_TYPE_FUTURE_CANCEL_INFO_EXT
- XR_TYPE_FUTURE_POLL_INFO_EXT
- XR_TYPE_FUTURE_POLL_RESULT_EXT
- XR_TYPE_FUTURE_COMPLETION_EXT

XrResult enumeration is extended with:

- XR_ERROR_FUTURE_PENDING_EXT
- XR_ERROR_FUTURE_INVALID_EXT

Issues

- Should there be a state for completed functions that is separate from "invalid"?
  - Resolved.
  - Answer: No. This would force an implementing runtime to remember old futures forever. In order to allow implementations that delete all associated data about a future after completion, we cannot differentiate between a future that never existed and one that was completed. Similarly, invalidated/completed is not formally a "state" for futures in the final API.

Version History

- Revision 1, 2023-02-14 (Andreas Løve Selvik, Meta Platforms and Ron Bessems, Magic Leap)
  - Initial extension description
- Revision 2, 2025-04-17 (Nihav Jain, Google)
  - Clarify what the runtime may do in a completion function.

12.35. XR_EXT_hand_interaction

Name String

XR_EXT_hand_interaction

Extension Type

Instance extension

Registered Extension Number

303

Revision

2

Ratification Status

Ratified

Extension and Version Dependencies

OpenXR 1.0

API Interactions

- Interacts with XR_EXT_palm_pose

Contributors

Yin Li, Microsoft
Alex Turner, Microsoft
Casey Meekhof, Microsoft
Lachlan Ford, Microsoft
Eric Provencher, Unity Technologies
Bryan Dube, Unity Technologies
Peter Kuhn, Unity Technologies
Tanya Li, Unity Technologies
Jakob Bornecrantz, Collabora
Jonathan Wright, Meta Platforms
Federico Schliemann, Meta Platforms
Andreas Loeve Selvik, Meta Platforms
Nathan Nuber, Valve
Joe Ludwig, Valve
Rune Berg, Valve
Adam Harwood, Ultraleap
Robert Blenkinsopp, Ultraleap
Paulo Gomes, Samsung Electronics
Ron Bessems, Magic Leap
Bastiaan Olij, Godot Engine
John Kearney, Meta Platforms

12.35.1. Overview

This extension defines four commonly used action poses for all user hand interaction profiles including both hand tracking devices and motion controller devices.

This extension also introduces a new interaction profile specifically designed for hand tracking devices to input through the OpenXR action system. Though, for runtimes with controller inputs, the runtime **should** also provide this interaction profile through action mappings from the controller inputs, so that an application whose suggested action bindings solely depending on this hand interaction profile is usable on such runtimes as well.

12.35.2. Action poses for hand interactions

The following four action poses (i.e. "pinch," "poke," "aim," and "grip") enable a hand and finger interaction model, whether the tracking inputs are provided by a hand tracking device or a motion controller device.

The runtime **must** support all of the following action subpaths on all interaction profiles that are valid for the user paths of */user/hand/left* and */user/hand/right*, including those interaction profiles enabled through extensions.

- *.../input/aim/pose*

- *.../input/grip/pose*
- *.../input/pinch_ext/pose*
- *.../input/poke_ext/pose*

Aim pose

The *.../input/aim/pose* is designed for interacting with objects out of arm's reach. For example, using a virtual laser pointer to aim at a virtual button on the wall is an interaction suited to the "aim" pose.

This is the same "aim" pose defined in Standard pose identifiers. Every tracked controller profile already supports this pose.

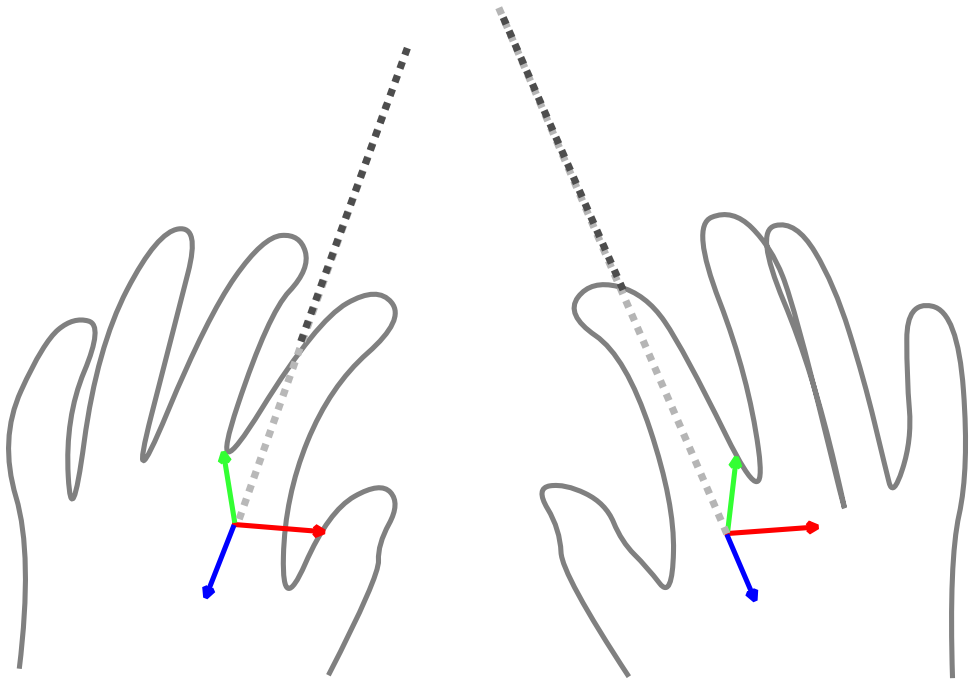


Figure 10. Example aim pose.

Position

The position of an "aim" pose is typically in front of the user's hand and moves together with the corresponding hand, so that the user is able to easily see the aiming ray cast to the target in the world and adjust for aim.

Orientation

The orientation of an "aim" pose is typically stabilized so that it is suitable to render an aiming ray emerging from the user's hand pointing into the world.

The -Z direction is the forward direction of the aiming gesture, that is, where the aiming ray is pointing at.

The +Y direction is a runtime defined direction based on the hand tracking device or ergonomics of the controller in the user's hand. It is typically pointing up in the world when the user is performing the

aiming gesture naturally forward with a hand or controller in front of the user body.

The +X direction is orthogonal to +Y and +Z using the right-hand rule.

When targeting an object out of arm's reach, the runtime **may** optimize the "aim" pose stability for pointing at a target, therefore the rotation of the "aim" pose **may** account for forearm or shoulder motion as well as hand rotation. Hence, the "aim" pose **may** not always rigidly attach to the user's hand rotation. If the application desires to rotate the targeted remote object in place, it **should** use the rotation of the "grip" pose instead of "aim" pose, as if the user is remotely holding the object and rotating it.

Grip pose

The *.../input/grip/pose* is designed for holding an object with a full hand grip gesture, for example, grasping and pushing a door's handle or holding and swinging a sword.

This is the same "grip" pose defined in Standard pose identifiers. Every tracked controller profile already supports this pose.

The runtime **should** optimize the "grip" pose orientation so that it stabilizes large virtual objects held in the user's hand.

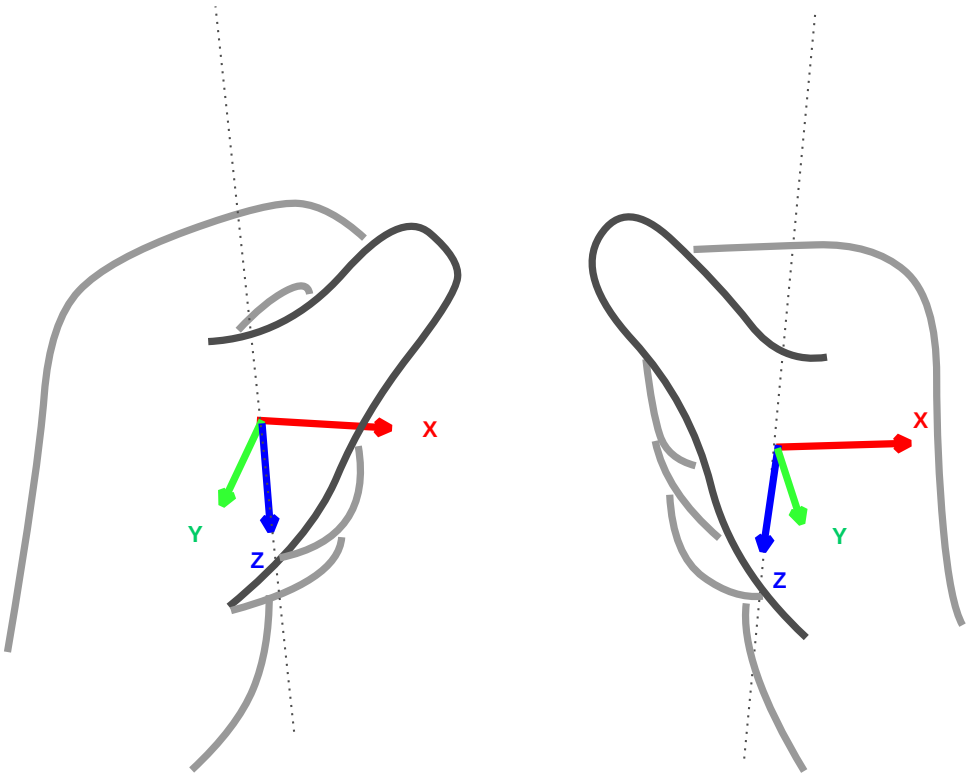


Figure 11. Example grip pose.

Position

The position of the "grip" pose is at the centroid of the user's palm when the user makes a fist or holds a tube-like object in the hand.

Orientation

The orientation of the "grip" pose **may** be used to render a virtual object held in the hand, for example, holding the grip of a virtual sword.

The Z axis of the grip pose goes through the center of the user's curled fingers when the user makes a fist or holds a controller, and the -Z direction (forward) goes from the little finger to the index finger.

When the user completely opens their hand to form a flat 5-finger pose and the palms face each other, the ray that is normal to the user's palms defines the X axis. The +X direction points away from the palm of the left hand and into the palm of the right hand. That is to say, in the described pose, the +X direction points to the user's right for both hands. To further illustrate: if the user is holding a stick by making a fist with each hand in front of the body and pointing the stick up, the +X direction points to the user's right for both hands.

The +Y direction is orthogonal to +Z and +X using the right-hand rule.

Pinch pose

The *.../input/pinch_ext/pose* is designed for interacting with a small object within arm's reach using a finger and thumb with a "pinch" gesture. For example, turning a key to open a lock or moving the knob on a slider control are interactions suited to the "pinch" pose.

The runtime **should** stabilize the "pinch" pose while the user is performing the "pinch" gesture.

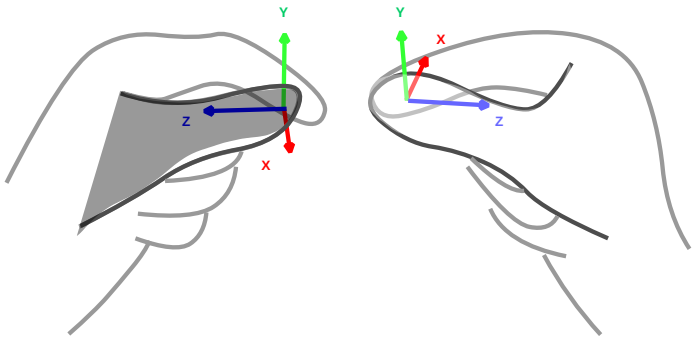


Figure 12. Example pinch pose.

Position

When the input is provided by a hand tracking device, the position of the "pinch" pose is typically where the index and thumb fingertips will touch each other for a "pinch" gesture.

The runtime **may** provide the "pinch" pose using any finger based on the current user's preference for accessibility support. An application typically designs the "pinch" pose interaction assuming the "pinch" is performed using the index finger and thumb.

When the input is provided by a motion controller device, the position of the "pinch" pose is typically based on a fixed offset from the grip pose in front of the controller, where the user **can** naturally interact with a small object. The runtime **should** avoid obstructing the "pinch" pose with the physical

profile of the motion controller.

Orientation

The "pinch" pose orientation **must** rotate together with the hand rotation.

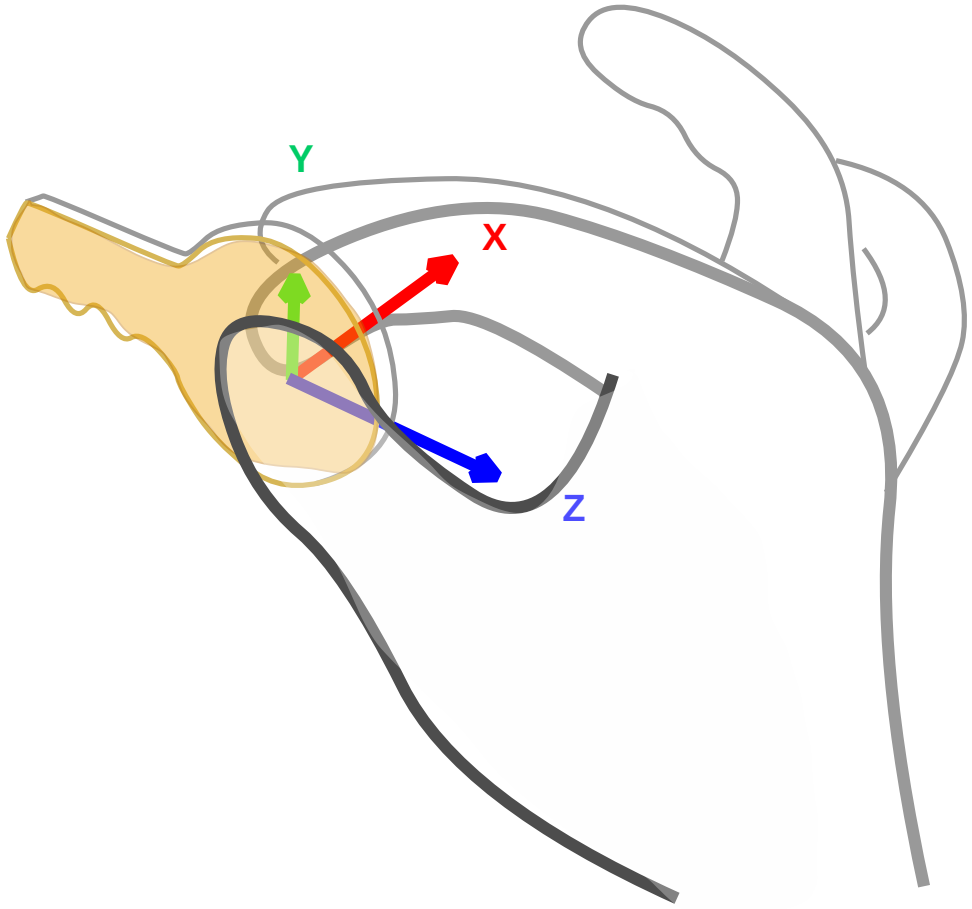


Figure 13. Example pinch orientation on right hand.

The "pinch" pose's orientation **may** be used to render a virtual object being held by a "pinch" gesture, for example, holding a key as illustrated in picture above.

If this virtual key is within a plane as illustrated in the above picture, the Y and Z axes of the "pinch" pose are within this plane.

The +Z axis is the backward direction of the "pinch" pose, typically the direction from the "pinch" position pointing to the mid point of thumb and finger proximal joints.

When the user puts both hands in front of the body at the same height, palms facing each other and fingers pointing forward, then performs a "pinch" gesture with both hands, the +Y direction for both hands **should** be roughly pointing up.

The X direction follows the right-hand rule using the Z and Y axes.

If the input is provided by a motion controller device, the orientation of the "pinch" pose is typically based on a fixed-rotation offset from the "grip" pose orientation that roughly follows the above

definition when the user is holding the controller naturally.

Poke pose

The *.../input/poke_ext/pose* is designed for interactions using a fingertip to touch and push a small object. For example, pressing a push button with a fingertip, swiping to scroll a browser view, or typing on a virtual keyboard are interactions suited to the "poke" pose.

The application **may** use the "poke" pose as a point to interact with virtual objects, and this pose is typically enough for simple interactions.

The application **may** also use a volumetric representation of a "poke" gesture using a sphere combined with the "poke" pose. The center of such a sphere is located the distance of one radius in the +Z direction of the "poke" pose, such that the "poke" pose falls on the surface of the sphere and the sphere models the shape of the fingertip.

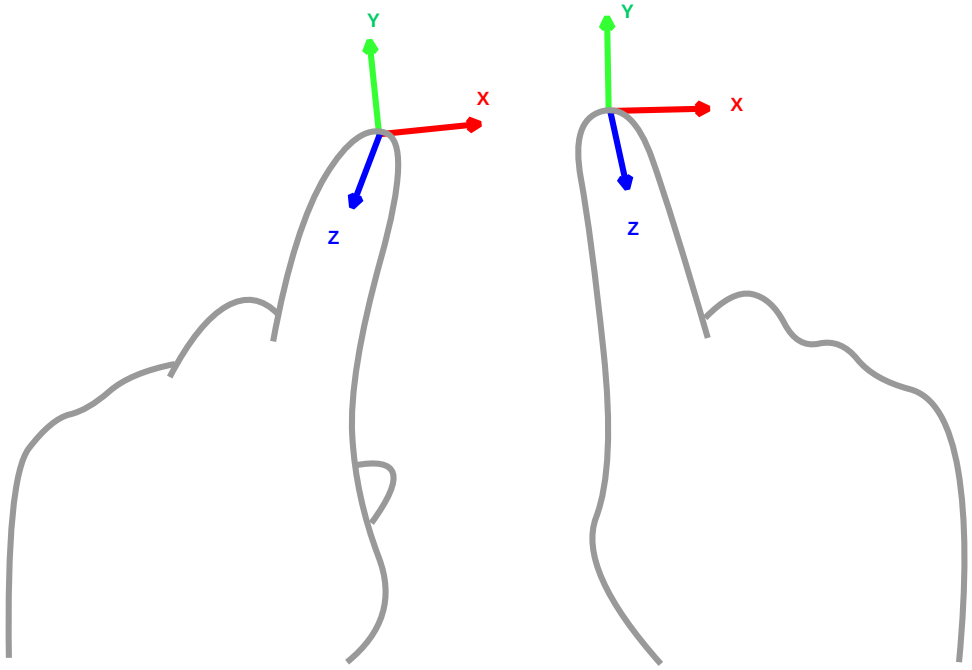


Figure 14. Example poke pose.

Position

When input is provided by a hand tracking device, the position of the "poke" pose is at the surface of the extended index fingertip. The runtime **may** provide the "poke" pose using other fingers for accessibility support.

When input is provided by a motion controller, the position of the "poke" pose is typically based on a fixed offset from the "grip" pose in front of the controller, where touching and pushing a small object feels natural using the controller. The runtime **should** avoid obstructing the "poke" pose with the physical profile of the motion controller.

Orientation

The +Y direction of the "poke" pose is the up direction in the world when the user is extending the index finger forward with palm facing down. When using a motion controller, +Y matches the up direction in the world when the user extends the index finger forward while holding the controller with palm facing down.

The +Z direction points from the fingertip towards the knuckle and parallel to the index finger distal bone, i.e. backwards when the user is holding a controller naturally in front of the body and pointing index finger forward.

The +X direction is orthogonal to +Y and +Z using the right-hand rule.

The "poke" pose **must** rotate together with the tip of the finger or the controller's "grip" pose.

12.35.3. The interaction profile for hand tracking devices

The hand interaction profile is designed for runtimes which provide hand inputs using hand tracking devices instead of controllers with triggers or buttons. This allows hand tracking devices to provide commonly used gestures and action poses to the OpenXR action system.

In addition to hand tracking devices, runtimes with controller inputs **should** also implement this interaction profile through action bindings, so that an application whose suggested action bindings solely depending on this hand interaction profile is usable on such runtimes as well.

Interaction profile path:

- */interaction_profiles/ext/hand_interaction_ext*

Valid for top level user path:

- */user/hand/left*
- */user/hand/right*

Supported component paths:

- *.../input/aim/pose*
- *.../input/grip/pose*
- *.../input/pinch_ext/pose*
- *.../input/poke_ext/pose*
- *.../input/pinch_ext/value*
- *.../input/pinch_ext/ready_ext*
- *.../input/aim_activate_ext/value*
- *.../input/aim_activate_ext/ready_ext*
- *.../input/grasp_ext/value*

- *.../input/grasp_ext/ready_ext*



Note

When the runtime supports `XR_VERSION_1_1` and use of OpenXR 1.1 is requested by the application, this interaction profile **must** also support

- *.../input/grip_surface/pose*



Note

When the `XR_KHR_maintenance1` extension is available and enabled, this interaction profile **must** also support

- *.../input/grip_surface/pose*



Note

When the `XR_EXT_palm_pose` extension is available and enabled, this interaction profile **must** also support

- *.../input/palm_ext/pose*

This interaction profile supports the above four action poses, as well as the following three groups of action inputs.

Pinch action

This interaction profile supports *.../input/pinch_ext/value* and *.../input/pinch_ext/ready_ext* actions.

The *.../input/pinch_ext/value* is a 1D analog input component indicating the extent which the user is bringing their finger and thumb together to perform a "pinch" gesture.

The *.../input/pinch_ext/value* **can** be used as either a boolean or float action type, where the value `XR_TRUE` or `1.0f` represents that the finger and thumb are touching each other.

The *.../input/pinch_ext/value* **must** be at value `0.0f` or `XR_FALSE` when the hand is in a natural and relaxed open state without the user making any extra effort.

The *.../input/pinch_ext/value* **should** be linear to the distance between the finger and thumb tips when they are in the range to change "pinch" value from 0 to 1.

The *.../input/pinch_ext/ready_ext* is a boolean input, where the value `XR_TRUE` indicates that the fingers used to perform the "pinch" gesture are properly tracked by the hand tracking device and the hand shape is observed to be ready to perform or is performing a "pinch" gesture.

The *.../input/pinch_ext/value* **must** be `0.0f` or `XR_FALSE` when the *.../input/pinch_ext/ready_ext* is `XR_FALSE`.

The runtime **may** drive the input of the "pinch" gesture using any finger with the thumb to support

accessibility.

Aim activate action

This interaction profile supports *.../input/aim_activate_ext/value* and *.../input/aim_activate_ext/ready_ext* actions.

The *.../input/aim_activate_ext/value* is a 1D analog input component indicating that the user activated the action on the target that the user is pointing at with the aim pose.

The "aim_activate" gesture is runtime defined, and it **should** be chosen so that the "aim" pose tracking is stable and usable for pointing at a distant target while the gesture is being performed.

The *.../input/aim_activate_ext/value* **can** be used as either a boolean or float action type, where the value `XR_TRUE` or `1.0f` represents that the aimed-at target is being fully interacted with.

The *.../input/aim_activate_ext/ready_ext* is a boolean input, where the value `XR_TRUE` indicates that the fingers to perform the "aim_activate" gesture are properly tracked by the hand tracking device and the hand shape is observed to be ready to perform or is performing an "aim_activate" gesture.

The *.../input/aim_activate_ext/value* **must** be `0.0f` or `XR_FALSE` when the *.../input/aim_activate_ext/ready_ext* is `XR_FALSE`.

Grasp action

This interaction profile supports *.../input/grasp_ext/value* action.

The *.../input/grasp_ext/value* is a 1D analog input component indicating that the user is making a fist.

The *.../input/grasp_ext/value* **can** be used as either a boolean or float action type, where the value `XR_TRUE` or `1.0f` represents that the fist is tightly closed.

The *.../input/grasp_ext/value* **must** be at value `0.0f` or `XR_FALSE` when the hand is in a natural and relaxed open state without the user making any extra effort.

The *.../input/grasp_ext/ready_ext* is a boolean input, where the value `XR_TRUE` indicates that the hand performing the grasp action is properly tracked by the hand tracking device and it is observed to be ready to perform or is performing the grasp action.

The *.../input/grasp_ext/value* **must** be `0.0f` or `XR_FALSE` when the *.../input/grasp_ext/ready_ext* is `XR_FALSE`.

Hand interaction gestures overlap

The values of the above "pinch", "grasp", and "aim_activate" input actions **may** not be mutually exclusive when the input is provided by a hand tracking device. The application **should** not assume these actions are distinctively activated as action inputs provided by buttons or triggers on a controller. The application **should** suggest action bindings considering the intent of the action and their paired action pose.

Using hand interaction profile with controllers

The runtimes with controller inputs **should** support the */interaction_profiles/ext/hand_interaction_ext* profile using input mapping, so that applications **can** solely rely on the */interaction_profiles/ext/hand_interaction_ext* profile to build XR experiences.

If the application desires to further customize the action poses with more flexible use of controller interaction profiles, the application **can** also provide action binding suggestions of controller profile using specific buttons or triggers to work together with the commonly used four action poses.



Typical usages of action poses with hand or controller profiles

- The *.../input/grip/pose* is typically used for holding a large object in the user's hand. When using a hand interaction profile, it is typically paired with *.../input/grasp_ext/value* for the user to directly manipulate an object held in a hand. When using a controller interaction profile, the "grip" pose is typically paired with a "squeeze" button or trigger that gives the user the sense of tightly holding an object.
- The *.../input/pinch_ext/pose* is typically used for directly manipulating a small object using the pinch gesture. When using a hand interaction profile, it is typically paired with the *.../input/pinch_ext/value* gesture. When using a controller interaction profile, it is typically paired with a trigger manipulated with the index finger, which typically requires curling the index finger and applying pressure with the fingertip.
- The *.../input/poke_ext/pose* is typically used for contact-based interactions using the motion of the hand or fingertip. It typically does not pair with other hand gestures or buttons on the controller. The application typically uses a sphere collider with the "poke" pose to visualize the pose and detect touch with a virtual object.
- The *.../input/aim/pose* is typically used for aiming at objects out of arm's reach. When using a hand interaction profile, it is typically paired with *.../input/aim_activate_ext/value* to optimize aiming ray stability while performing the gesture. When using a controller interaction profile, the "aim" pose is typically paired with a trigger or a button for aim and fire operations.
- Because controllers are typically mapping buttons or triggers for the above hand interaction values, they typically report `XR_TRUE` for their corresponding *.../ready_ext* action. This is because the buttons and triggers are always prepared and capable of receiving actions.

New Object Types

New Flag Types

New Enum Constants

New Enums

New Structures

New Functions

Issues

Version History

- Revision 1, 2021-08-06 (Yin Li)
  - Initial extension description
- Revision 2, 2025-08-20 (John Kearney, Meta)
  - Explicitly list support for grip_surface in extension definition.

12.36. XR_EXT_hand_joints_motion_range

Name String

`XR_EXT_hand_joints_motion_range`

Extension Type

Instance extension

Registered Extension Number

81

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

OpenXR 1.0
and
`XR_EXT_hand_tracking`

Last Modified Date

2021-04-15

IP Status

No known IP claims.

Contributors

Joe van den Heuvel, Valve
Rune Berg, Valve
Joe Ludwig, Valve
Jakob Bornecrantz, Collabora

Overview

This extension augments the XR_EXT_hand_tracking extension to enable applications to request that the XrHandJointLocationsEXT returned by xrLocateHandJointsEXT should return hand joint locations conforming to a range of motion specified by the application.

The application **must** enable the XR_EXT_hand_tracking extension in order to use this extension.

New Object Types

New Flag Types

New Enum Constants

New Enums

The XrHandJointsMotionRangeEXT describes the hand joints' range of motion returned by xrLocateHandJointsEXT.

Runtimes **must** support both XR_HAND_JOINTS_MOTION_RANGE_CONFORMING_TO_CONTROLLER_EXT and XR_HAND_JOINTS_MOTION_RANGE_UNOBSTRUCTED_EXT for each controller interaction profile that supports hand joint data.

```
// Provided by XR_EXT_hand_joints_motion_range
typedef enum XrHandJointsMotionRangeEXT {
    XR_HAND_JOINTS_MOTION_RANGE_UNOBSTRUCTED_EXT = 1,
    XR_HAND_JOINTS_MOTION_RANGE_CONFORMING_TO_CONTROLLER_EXT = 2,
    XR_HAND_JOINTS_MOTION_RANGE_MAX_ENUM_EXT = 0x7FFFFFFF
} XrHandJointsMotionRangeEXT;
```

Enumerant Descriptions

- XR_HAND_JOINTS_MOTION_RANGE_UNOBSTRUCTED_EXT This option refers to the range of motion of a human hand, without any obstructions. Input systems that obstruct the movement of the user's hand (e.g.: a held controller preventing the user from making a fist) or have only limited ability to track finger positions **must** use the information available to them to emulate an unobstructed range of motion.
- XR_HAND_JOINTS_MOTION_RANGE_CONFORMING_TO_CONTROLLER_EXT This option refers to the range of motion of the hand joints taking into account any physical limits imposed by the controller itself. This will tend to be the most accurate pose compared to the user's actual hand pose, but might not allow a closed fist for example.
  - If the current interaction profile represents a controller, or other device that obstructs the hand, the implementation **must** return joint locations conforming to the shape of that device. If the current interaction profile is being emulated by a different physical controller, the implementation **may** return joint locations conforming to the shape of either the current interaction profile or the actual physical controller.
  - If the current interaction profile does not represent a controller, the implementation **must** return joint locations based on the unobstructed joint locations.

New Structures

The XrHandJointsMotionRangeInfoEXT is a structure that an application **can** chain in XrHandJointsLocateInfoEXT to request the joint motion range specified by the handJointsMotionRange field.

Runtimes **must** return the appropriate joint locations depending on the handJointsMotionRange field and the currently active interaction profile.

```
// Provided by XR_EXT_hand_joints_motion_range
typedef struct XrHandJointsMotionRangeInfoEXT {
    XrStructureType               type;
    const void*                   next;
    XrHandJointsMotionRangeEXT    handJointsMotionRange;
} XrHandJointsMotionRangeInfoEXT;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `handJointsMotionRange` is an XrHandJointsMotionRangeEXT that defines the hand joint range of motion the application wants.

Valid Usage (Implicit)

- The `XR_EXT_hand_joints_motion_range` extension **must** be enabled prior to using XrHandJointsMotionRangeInfoEXT
- `type` **must** be `XR_TYPE_HAND_JOINTS_MOTION_RANGE_INFO_EXT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `handJointsMotionRange` **must** be a valid XrHandJointsMotionRangeEXT value

New Functions

Issues

Version History

- Revision 1, 2021-04-15 (Rune Berg)
  - Initial extension description

12.37. XR_EXT_hand_tracking

Name String

`XR_EXT_hand_tracking`

Extension Type

Instance extension

Registered Extension Number

52

Revision

4

Ratification Status

Ratified

Extension and Version Dependencies

OpenXR 1.0

Last Modified Date

2021-04-15

IP Status

No known IP claims.

Contributors

Yin Li, Microsoft
Lachlan Ford, Microsoft
Alex Turner, Microsoft
Bryce Hutchings, Microsoft
Cass Everitt, Oculus
Blake Taylor, Magic Leap
Joe van den Heuvel, Valve
Rune Berg, Valve
Valerie Benson, Ultraleap
Rylie Pavlik, Collabora

12.37.1. Overview

This extension enables applications to locate the individual joints of hand tracking inputs. It enables applications to render hands in XR experiences and interact with virtual objects using hand joints.

12.37.2. Inspect system capability

An application **can** inspect whether the system is capable of hand tracking input by extending the XrSystemProperties with XrSystemHandTrackingPropertiesEXT structure when calling xrGetSystemProperties.

```
// Provided by XR_EXT_hand_tracking
typedef struct XrSystemHandTrackingPropertiesEXT {
    XrStructureType    type;
    void*              next;
    XrBool32           supportsHandTracking;
} XrSystemHandTrackingPropertiesEXT;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension. For more details about the structure chain, see the structure being extended (XrSystemProperties).
- `supportsHandTracking` is an `XrBool32`, indicating if current system is capable of hand tracking input.

Valid Usage (Implicit)

- The `XR_EXT_hand_tracking` extension **must** be enabled prior to using XrSystemHandTrackingPropertiesEXT
- `type` **must** be `XR_TYPE_SYSTEM_HAND_TRACKING_PROPERTIES_EXT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

If a runtime returns `XR_FALSE` for `supportsHandTracking`, the runtime **must** return `XR_ERROR_FEATURE_UNSUPPORTED` from xrCreateHandTrackerEXT.

12.37.3. Create a hand tracker handle

The XrHandTrackerEXT handle represents the resources for hand tracking of the specific hand.

```
// Provided by XR_EXT_hand_tracking
XR_DEFINE_HANDLE(XrHandTrackerEXT)
```

An application creates separate XrHandTrackerEXT handles for left and right hands. This handle can be used to locate hand joints using xrLocateHandJointsEXT function.

A hand tracker provides joint locations with an unobstructed range of motion of an empty human hand.



Note

This behavior can be modified by the `XR_EXT_hand_joints_motion_range` extension

An application can create an XrHandTrackerEXT handle using xrCreateHandTrackerEXT function.

```
// Provided by XR_EXT_hand_tracking
XrResult xrCreateHandTrackerEXT(
    XrSession                                   session,
    const XrHandTrackerCreateInfoEXT*           createInfo,
    XrHandTrackerEXT*                           handTracker);
```

Parameter Descriptions

- `session` is an XrSession in which the hand tracker will be active.
- `createInfo` is the XrHandTrackerCreateInfoEXT used to specify the hand tracker.
- `handTracker` is the returned XrHandTrackerEXT handle.

Valid Usage (Implicit)

- The `XR_EXT_hand_tracking` extension **must** be enabled prior to calling xrCreateHandTrackerEXT
- `session` **must** be a valid XrSession handle
- `createInfo` **must** be a pointer to a valid XrHandTrackerCreateInfoEXT structure
- `handTracker` **must** be a pointer to an XrHandTrackerEXT handle

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FEATURE_UNSUPPORTED
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

If the system does not support hand tracking, runtime **must** return XR_ERROR_FEATURE_UNSUPPORTED from xrCreateHandTrackerEXT. In this case, the runtime **must** return XR_FALSE for XrSystemHandTrackingPropertiesEXT::supportsHandTracking when the function xrGetSystemProperties is called, so that the application **can** avoid creating a hand tracker.

The XrHandTrackerCreateInfoEXT structure describes the information to create an XrHandTrackerEXT handle.

```
// Provided by XR_EXT_hand_tracking
typedef struct XrHandTrackerCreateInfoEXT {
    XrStructureType      type;
    const void*          next;
    XrHandEXT            hand;
    XrHandJointSetEXT    handJointSet;
} XrHandTrackerCreateInfoEXT;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `hand` is an XrHandEXT which describes which hand the tracker is tracking.
- `handJointSet` is an XrHandJointSetEXT describe the set of hand joints to retrieve.

Valid Usage (Implicit)

- The `XR_EXT_hand_tracking` extension **must** be enabled prior to using XrHandTrackerCreateInfoEXT
- `type` **must** be `XR_TYPE_HAND_TRACKER_CREATE_INFO_EXT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain. See also: XrHandPoseTypeInfoMSFT, XrHandTrackingDataSourceInfoEXT
- `hand` **must** be a valid XrHandEXT value
- `handJointSet` **must** be a valid XrHandJointSetEXT value

The XrHandEXT describes which hand the XrHandTrackerEXT is tracking.

```
// Provided by XR_EXT_hand_tracking
typedef enum XrHandEXT {
    XR_HAND_LEFT_EXT = 1,
    XR_HAND_RIGHT_EXT = 2,
    XR_HAND_MAX_ENUM_EXT = 0x7FFFFFFF
} XrHandEXT;
```

Enumerant Descriptions

- `XR_HAND_LEFT_EXT` specifies the hand tracker will be tracking the user's left hand.
- `XR_HAND_RIGHT_EXT` specifies the hand tracker will be tracking the user's right hand.

The XrHandJointSetEXT enum describes the set of hand joints to track when creating an XrHandTrackerEXT.

```
// Provided by XR_EXT_hand_tracking
typedef enum XrHandJointSetEXT {
    XR_HAND_JOINT_SET_DEFAULT_EXT = 0,
  // Provided by XR_ULTRALEAP_hand_tracking_forearm
    XR_HAND_JOINT_SET_HAND_WITH_FOREARM_ULTRALEAP = 1000149000,
    XR_HAND_JOINT_SET_MAX_ENUM_EXT = 0x7FFFFFFF
} XrHandJointSetEXT;
```

Enumerant Descriptions

- XR_HAND_JOINT_SET_DEFAULT_EXT indicates that the created XrHandTrackerEXT tracks the set of hand joints described by XrHandJointEXT enum, i.e. the xrLocateHandJointsEXT function returns an array of joint locations with the count of XR_HAND_JOINT_COUNT_EXT and can be indexed using XrHandJointEXT.

xrDestroyHandTrackerEXT function releases the handTracker and the underlying resources when finished with hand tracking experiences.

```
// Provided by XR_EXT_hand_tracking
XrResult xrDestroyHandTrackerEXT(
    XrHandTrackerEXT                            handTracker);
```

Parameter Descriptions

- handTracker is an XrHandTrackerEXT previously created by xrCreateHandTrackerEXT.

Valid Usage (Implicit)

- The XR_EXT_hand_tracking extension **must** be enabled prior to calling xrDestroyHandTrackerEXT
- handTracker **must** be a valid XrHandTrackerEXT handle

Thread Safety

- Access to handTracker, and any child handles, **must** be externally synchronized

Return Codes

Success

- XR_SUCCESS

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_RUNTIME_FAILURE

12.37.4. Locate hand joints

The xrLocateHandJointsEXT function locates an array of hand joints to a base space at given time.

```
// Provided by XR_EXT_hand_tracking
XrResult xrLocateHandJointsEXT(
    XrHandTrackerEXT                            handTracker,
    const XrHandJointsLocateInfoEXT*            locateInfo,
    XrHandJointLocationsEXT*                    locations);
```

Parameter Descriptions

- handTracker is an XrHandTrackerEXT previously created by xrCreateHandTrackerEXT.
- locateInfo is a pointer to XrHandJointsLocateInfoEXT describing information to locate hand joints.
- locations is a pointer to XrHandJointLocationsEXT receiving the returned hand joint locations.

Valid Usage (Implicit)

- The XR_EXT_hand_tracking extension **must** be enabled prior to calling xrLocateHandJointsEXT
- handTracker **must** be a valid XrHandTrackerEXT handle
- locateInfo **must** be a pointer to a valid XrHandJointsLocateInfoEXT structure
- locations **must** be a pointer to an XrHandJointLocationsEXT structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_TIME_INVALID
- XR_ERROR_VALIDATION_FAILURE

The XrHandJointsLocateInfoEXT structure describes the information to locate hand joints.

```
// Provided by XR_EXT_hand_tracking
typedef struct XrHandJointsLocateInfoEXT {
    XrStructureType    type;
    const void*        next;
    XrSpace            baseSpace;
    XrTime             time;
} XrHandJointsLocateInfoEXT;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- baseSpace is an XrSpace within which the returned hand joint locations will be represented.
- time is an XrTime at which to locate the hand joints.

Valid Usage (Implicit)

- The XR_EXT_hand_tracking extension **must** be enabled prior to using XrHandJointsLocateInfoEXT
- type **must** be XR_TYPE_HAND_JOINTS_LOCATE_INFO_EXT
- next **must** be NULL or a valid pointer to the next structure in a structure chain. See also: XrHandJointsMotionRangeInfoEXT, XrHandTrackingUnextrapolatedPosesRequestMETA
- baseSpace **must** be a valid XrSpace handle

XrHandJointLocationsEXT structure returns the state of the hand joint locations.

```
// Provided by XR_EXT_hand_tracking
typedef struct XrHandJointLocationsEXT {
    XrStructureType            type;
    void*                      next;
    XrBool32                   isActive;
    uint32_t                   jointCount;
    XrHandJointLocationEXT*    jointLocations;
} XrHandJointLocationsEXT;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain, such as XrHandJointVelocitiesEXT.
- isActive is an XrBool32 indicating if the hand tracker is actively tracking.
- jointCount is a uint32_t describing the count of elements in jointLocations array.
- jointLocations is an array of XrHandJointLocationEXT receiving the returned hand joint locations.

The application **must** allocate the memory for the output array jointLocations that can contain at least jointCount of XrHandJointLocationEXT.

The application **must** set jointCount as described by the XrHandJointSetEXT when creating the XrHandTrackerEXT otherwise the runtime **must** return XR_ERROR_VALIDATION_FAILURE.

The runtime **must** return jointLocations representing the range of motion of a human hand, without any obstructions. Input systems that obstruct the movement of the user's hand (e.g.: a held controller preventing the user from making a fist) or that have only limited ability to track finger positions **must** use the information available to them to emulate an unobstructed range of motion.

The runtime **must** update the jointLocations array ordered so that the application can index elements using the corresponding hand joint enum (e.g. XrHandJointEXT) as described by XrHandJointSetEXT when creating the XrHandTrackerEXT. For example, when the XrHandTrackerEXT is created with XR_HAND_JOINT_SET_DEFAULT_EXT, the application **must** set the jointCount to XR_HAND_JOINT_COUNT_EXT, and the runtime **must** fill the jointLocations array ordered so that it may be indexed by the XrHandJointEXT enum.

If the returned isActive is true, the runtime **must** return all joint locations with both XR_SPACE_LOCATION_POSITION_VALID_BIT and XR_SPACE_LOCATION_ORIENTATION_VALID_BIT set. Although, in this case, some joint space locations **may** be untracked (i.e. XR_SPACE_LOCATION_POSITION_TRACKED_BIT or XR_SPACE_LOCATION_ORIENTATION_TRACKED_BIT is unset).

If the returned isActive is false, it indicates the hand tracker did not detect the hand input or the application lost input focus. In this case, the runtime **must** return all jointLocations with neither XR_SPACE_LOCATION_POSITION_VALID_BIT nor XR_SPACE_LOCATION_ORIENTATION_VALID_BIT set.

Valid Usage (Implicit)

- The XR_EXT_hand_tracking extension **must** be enabled prior to using XrHandJointLocationsEXT
- type **must** be XR_TYPE_HAND_JOINT_LOCATIONS_EXT
- next **must** be NULL or a valid pointer to the next structure in a structure chain. See also: XrHandJointVelocitiesEXT, XrHandTrackingAimStateFB, XrHandTrackingCapsulesStateFB, XrHandTrackingDataSourceStateEXT, XrHandTrackingScaleFB, XrHandTrackingUnextrapolatedPosesMETA
- jointLocations **must** be a pointer to an array of jointCount XrHandJointLocationEXT structures
- The jointCount parameter **must** be greater than 0

XrHandJointLocationEXT structure describes the position, orientation, and radius of a hand joint.

```
// Provided by XR_EXT_hand_tracking
typedef struct XrHandJointLocationEXT {
    XrSpaceLocationFlags    locationFlags;
    XrPosef                 pose;
    float                   radius;
} XrHandJointLocationEXT;
```

Member Descriptions

- `locationFlags` is a bitfield, with bit masks defined in XrSpaceLocationFlagBits, to indicate which members contain valid data. If none of the bits are set, no other fields in this structure **should** be considered to be valid or meaningful.
- `pose` is an XrPosef defining the position and orientation of the origin of a hand joint within the reference frame of the corresponding XrHandJointsLocateInfoEXT::`baseSpace`.
- `radius` is a `float` value radius of the corresponding joint in units of meters.

If the returned `locationFlags` has `XR_SPACE_LOCATION_POSITION_VALID_BIT` set, the returned radius **must** be a positive value.

If the returned `locationFlags` has `XR_SPACE_LOCATION_POSITION_VALID_BIT` unset, the returned radius value is undefined and should be avoided.

Valid Usage (Implicit)

- The `XR_EXT_hand_tracking` extension **must** be enabled prior to using XrHandJointLocationEXT
- `locationFlags` **must** be `0` or a valid combination of XrSpaceLocationFlagBits values

The application can chain an XrHandJointVelocitiesEXT structure to the `next` pointer of XrHandJointLocationsEXT when calling xrLocateHandJointsEXT to retrieve the hand joint velocities.

```
// Provided by XR_EXT_hand_tracking
typedef struct XrHandJointVelocitiesEXT {
    XrStructureType            type;
    void*                      next;
    uint32_t                   jointCount;
    XrHandJointVelocityEXT*    jointVelocities;
} XrHandJointVelocitiesEXT;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `jointCount` is a `uint32_t` describing the number of elements in `jointVelocities` array.
- `jointVelocities` is an array of XrHandJointVelocityEXT receiving the returned hand joint velocities.

The application **must** allocate the memory for the output array `jointVelocities` that can contain at least `jointCount` of XrHandJointVelocityEXT.

The application **must** input `jointCount` as described by the XrHandJointSetEXT when creating the XrHandTrackerEXT. Otherwise, the runtime **must** return `XR_ERROR_VALIDATION_FAILURE`.

The runtime **must** update the `jointVelocities` array in the order so that the application can index elements using the corresponding hand joint enum (e.g. XrHandJointEXT) as described by the XrHandJointSetEXT when creating the XrHandTrackerEXT. For example, when the XrHandTrackerEXT is created with `XR_HAND_JOINT_SET_DEFAULT_EXT`, the application **must** set the `jointCount` to XR_HAND_JOINT_COUNT_EXT, and the returned `jointVelocities` array **must** be ordered to be indexed by enum XrHandJointEXT enum.

If the returned XrHandJointLocationsEXT::`isActive` is false, it indicates the hand tracker did not detect a hand input or the application lost input focus. In this case, the runtime **must** return all `jointVelocities` with neither `XR_SPACE_VELOCITY_LINEAR_VALID_BIT` nor `XR_SPACE_VELOCITY_ANGULAR_VALID_BIT` set.

If an XrHandJointVelocitiesEXT structure is chained to XrHandJointLocationsEXT::`next`, the returned XrHandJointLocationsEXT::`isActive` is true, and the velocity is observed or can be calculated by the runtime, the runtime **must** fill in the linear velocity of each hand joint within the reference frame of XrHandJointsLocateInfoEXT::`baseSpace` and set the `XR_SPACE_VELOCITY_LINEAR_VALID_BIT`. Similarly, if an XrHandJointVelocitiesEXT structure is chained to XrHandJointLocationsEXT::`next`, the returned XrHandJointLocationsEXT::`isActive` is true, and the *angular velocity* is observed or can be calculated by the runtime, the runtime **must** fill in the angular velocity of each joint within the reference frame of XrHandJointsLocateInfoEXT::`baseSpace` and set the `XR_SPACE_VELOCITY_ANGULAR_VALID_BIT`.

Valid Usage (Implicit)

- The `XR_EXT_hand_tracking` extension **must** be enabled prior to using XrHandJointVelocitiesEXT
- `type` **must** be `XR_TYPE_HAND_JOINT_VELOCITIES_EXT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `jointVelocities` **must** be a pointer to an array of `jointCount` XrHandJointVelocityEXT structures
- The `jointCount` parameter **must** be greater than `0`

XrHandJointVelocityEXT structure describes the linear and angular velocity of a hand joint.

```
// Provided by XR_EXT_hand_tracking
typedef struct XrHandJointVelocityEXT {
    XrSpaceVelocityFlags    velocityFlags;
    XrVector3f              linearVelocity;
    XrVector3f              angularVelocity;
} XrHandJointVelocityEXT;
```

Member Descriptions

- `velocityFlags` is a bitfield, with bit masks defined in XrSpaceVelocityFlagBits, to indicate which members contain valid data. If none of the bits are set, no other fields in this structure **should** be considered to be valid or meaningful.
- `linearVelocity` is the relative linear velocity of the hand joint with respect to and expressed in the reference frame of the corresponding XrHandJointsLocateInfoEXT::`baseSpace`, in units of meters per second.
- `angularVelocity` is the relative angular velocity of the hand joint with respect to the corresponding XrHandJointsLocateInfoEXT::`baseSpace`. The vector's direction is expressed in the reference frame of the corresponding XrHandJointsLocateInfoEXT::`baseSpace` and is parallel to the rotational axis of the hand joint. The vector's magnitude is the relative angular speed of the hand joint in radians per second. The vector follows the right-hand rule for torque/rotation.

Valid Usage (Implicit)

- The `XR_EXT_hand_tracking` extension **must** be enabled prior to using XrHandJointVelocityEXT

12.37.5. Example code for locating hand joints

The following example code demonstrates how to locate all hand joints relative to a world space.

```
XrInstance instance; // previously initialized
XrSystemId systemId; // previously initialized
XrSession session;   // previously initialized
XrSpace worldSpace;  // previously initialized, e.g. from
                     // XR_REFERENCE_SPACE_TYPE_LOCAL

// Inspect hand tracking system properties
XrSystemHandTrackingPropertiesEXT handTrackingSystemProperties {
    .type = XR_TYPE_SYSTEM_HAND_TRACKING_PROPERTIES_EXT,
};
XrSystemProperties systemProperties {
```

```
    .type = XR_TYPE_SYSTEM_PROPERTIES,
    .next = &handTrackingSystemProperties,
};
CHK_XR(xrGetSystemProperties(instance, systemId, &systemProperties));
if (!handTrackingSystemProperties.supportsHandTracking) {
    // The system does not support hand tracking
    return;
}

// Get function pointer for xrCreateHandTrackerEXT
PFN_xrCreateHandTrackerEXT pfnCreateHandTrackerEXT = nullptr;
CHK_XR(xrGetInstanceProcAddr(instance, "xrCreateHandTrackerEXT",
                             reinterpret_cast<PFN_xrVoidFunction*>(
                             &pfnCreateHandTrackerEXT)));

// Create a hand tracker for left hand that tracks default set of hand joints.
XrHandTrackerEXT leftHandTracker {XR_NULL_HANDLE};
{
    XrHandTrackerCreateInfoEXT createInfo {
        .type = XR_TYPE_HAND_TRACKER_CREATE_INFO_EXT,
        .hand = XR_HAND_LEFT_EXT,
        .handJointSet = XR_HAND_JOINT_SET_DEFAULT_EXT,
    };
    CHK_XR(pfnCreateHandTrackerEXT(session, &createInfo, &leftHandTracker));
}

// Allocate buffers to receive joint location and velocity data before frame
// loop starts
XrHandJointLocationEXT jointLocations[XR_HAND_JOINT_COUNT_EXT];
XrHandJointVelocityEXT jointVelocities[XR_HAND_JOINT_COUNT_EXT];

XrHandJointVelocitiesEXT velocities {
    .type = XR_TYPE_HAND_JOINT_VELOCITIES_EXT,
    .jointCount = XR_HAND_JOINT_COUNT_EXT,
    .jointVelocities = jointVelocities,
};

XrHandJointLocationsEXT locations {
    .type = XR_TYPE_HAND_JOINT_LOCATIONS_EXT,
    .next = &velocities,
    .jointCount = XR_HAND_JOINT_COUNT_EXT,
    .jointLocations = jointLocations,
};

// Get function pointer for xrLocateHandJointsEXT
PFN_xrLocateHandJointsEXT pfnLocateHandJointsEXT = nullptr;
CHK_XR(xrGetInstanceProcAddr(instance, "xrLocateHandJointsEXT",
                             reinterpret_cast<PFN_xrVoidFunction*>(
```

```
                                &pfnLocateHandJointsEXT)));
while (1) {
    // ...
    // For every frame in frame loop
    // ...
    XrFrameState frameState; // previously returned from xrWaitFrame
    const XrTime time = frameState.predictedDisplayTime;

    XrHandJointsLocateInfoEXT locateInfo {
        .type = XR_TYPE_HAND_JOINTS_LOCATE_INFO_EXT,
        .baseSpace = worldSpace,
        .time = time,
    };

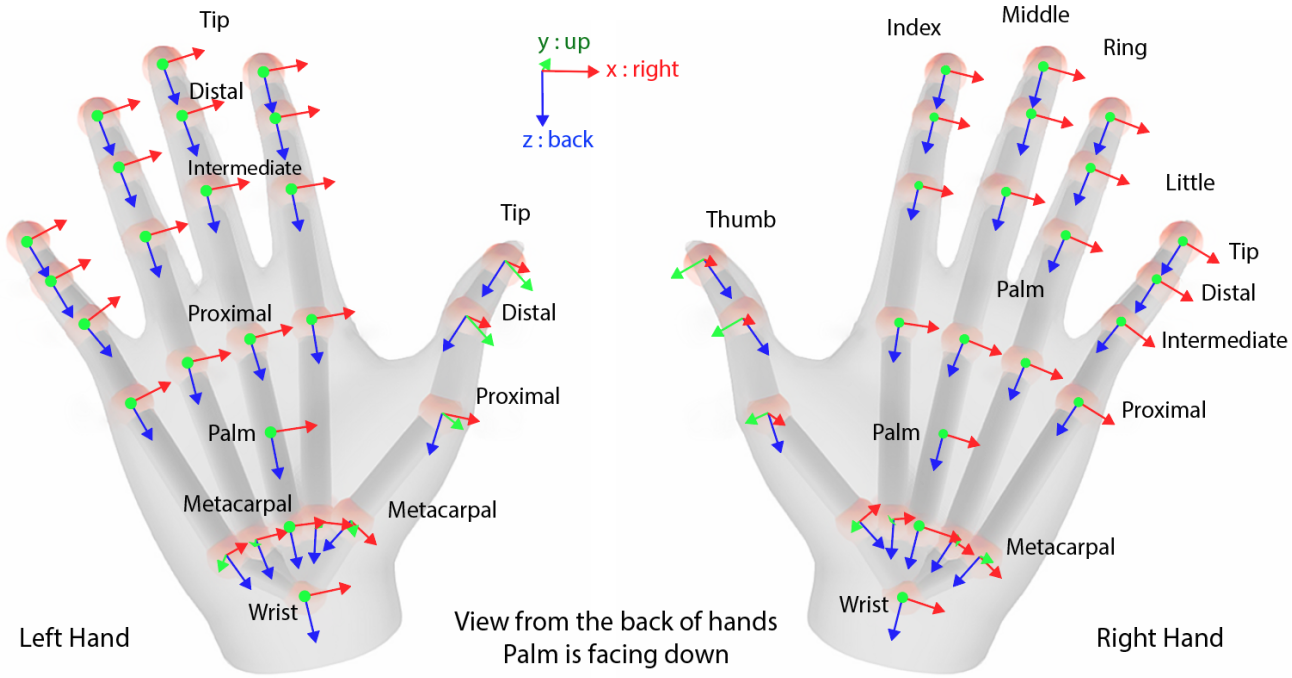
    CHK_XR(pfnLocateHandJointsEXT(leftHandTracker, &locateInfo, &locations));

    if (locations.isActive) {
        // The returned joint location array can be directly indexed with
        // XrHandJointEXT enum.
        const XrPosef &indexTipInWorld =
            jointLocations[XR_HAND_JOINT_INDEX_TIP_EXT].pose;
        const XrPosef &thumbTipInWorld =
            jointLocations[XR_HAND_JOINT_THUMB_TIP_EXT].pose;

        // using the returned radius and velocity of index finger tip.
        const float indexTipRadius =
            jointLocations[XR_HAND_JOINT_INDEX_TIP_EXT].radius;
        const XrHandJointVelocityEXT &indexTipVelocity =
            jointVelocities[XR_HAND_JOINT_INDEX_TIP_EXT];
    }
}
```

12.37.6. Conventions of hand joints

This extension defines 26 joints for hand tracking: 4 joints for the thumb finger, 5 joints for the other four fingers, and the wrist and palm of the hands.


Tip
Distal
Intermediate
Proximal
Palm
Metacarpal
Wrist
Left Hand
y : up
x : right
z : back
Tip
Distal
Proximal
Metacarpal
View from the back of hands
Palm is facing down
Index
Middle
Ring
Little
Thumb
Tip
Distal
Intermediate
Palm
Proximal
Metacarpal
Wrist
Right Hand

```
// Provided by XR_EXT_hand_tracking
typedef enum XrHandJointEXT {
    XR_HAND_JOINT_PALM_EXT = 0,
    XR_HAND_JOINT_WRIST_EXT = 1,
    XR_HAND_JOINT_THUMB_METACARPAL_EXT = 2,
    XR_HAND_JOINT_THUMB_PROXIMAL_EXT = 3,
    XR_HAND_JOINT_THUMB_DISTAL_EXT = 4,
    XR_HAND_JOINT_THUMB_TIP_EXT = 5,
    XR_HAND_JOINT_INDEX_METACARPAL_EXT = 6,
    XR_HAND_JOINT_INDEX_PROXIMAL_EXT = 7,
    XR_HAND_JOINT_INDEX_INTERMEDIATE_EXT = 8,
    XR_HAND_JOINT_INDEX_DISTAL_EXT = 9,
    XR_HAND_JOINT_INDEX_TIP_EXT = 10,
    XR_HAND_JOINT_MIDDLE_METACARPAL_EXT = 11,
    XR_HAND_JOINT_MIDDLE_PROXIMAL_EXT = 12,
    XR_HAND_JOINT_MIDDLE_INTERMEDIATE_EXT = 13,
    XR_HAND_JOINT_MIDDLE_DISTAL_EXT = 14,
    XR_HAND_JOINT_MIDDLE_TIP_EXT = 15,
    XR_HAND_JOINT_RING_METACARPAL_EXT = 16,
    XR_HAND_JOINT_RING_PROXIMAL_EXT = 17,
    XR_HAND_JOINT_RING_INTERMEDIATE_EXT = 18,
    XR_HAND_JOINT_RING_DISTAL_EXT = 19,
    XR_HAND_JOINT_RING_TIP_EXT = 20,
    XR_HAND_JOINT_LITTLE_METACARPAL_EXT = 21,
    XR_HAND_JOINT_LITTLE_PROXIMAL_EXT = 22,
    XR_HAND_JOINT_LITTLE_INTERMEDIATE_EXT = 23,
    XR_HAND_JOINT_LITTLE_DISTAL_EXT = 24,
    XR_HAND_JOINT_LITTLE_TIP_EXT = 25,
    XR_HAND_JOINT_MAX_ENUM_EXT = 0x7FFFFFFF
} XrHandJointEXT;
```

The finger joints, except the tips, are named after the corresponding bone at the further end of the bone from the finger tips. The joint's orientation is defined at a fully opened hand pose facing down as in the above picture.



Note

Many applications and game engines use names to identify joints rather than using indices. If possible, applications should use the joint name part of the XrHandJointEXT enum plus a hand identifier to help prevent joint name clashes (e.g. Index_Metacarpal_L, Thumb_Tip_R). Using consistent names increases the portability of assets between applications and engines. Including the hand in the identifier prevents ambiguity when both hands are used in the same skeleton, such as when they are combined with additional joints to form a full body skeleton.

The backward (+Z) direction is parallel to the corresponding bone and points away from the finger tip.

The up (+Y) direction is pointing out of the back of and perpendicular to the corresponding finger nail at the fully opened hand pose. The X direction is perpendicular to Y and Z and follows the right hand rule.

The wrist joint is located at the pivot point of the wrist which is location invariant when twisting hand without moving the forearm. The backward (+Z) direction is parallel to the line from wrist joint to middle finger metacarpal joint, and points away from the finger tips. The up (+Y) direction points out towards back of hand and perpendicular to the skin at wrist. The X direction is perpendicular to the Y and Z directions and follows the right hand rule.

The palm joint is located at the center of the middle finger's metacarpal bone. The backward (+Z) direction is parallel to the middle finger's metacarpal bone, and points away from the finger tips. The up (+Y) direction is perpendicular to palm surface and pointing towards the back of the hand. The X direction is perpendicular to the Y and Z directions and follows the right hand rule.

The radius of each joint is the distance from the joint to the skin in meters. The application can use a sphere at the joint location with joint radius for collision detection for interactions, such as pushing a virtual button using the index finger tip.

For example, suppose the radius of the palm joint is `r` then the app **can** offset `{0, -r, 0}` to palm joint location to get the surface of hand palm center, or offset `{0, r, 0}` to get the back surface of the hand.

Note that the palm joint for the hand tracking is not the same as *.../input/grip/pose* when hand tracking is provided by controller tracking. A "grip" pose is located at the center of the controller handle when user is holding a controller, outside of the user's hand. A "palm" pose is located at the center of middle finger metacarpal bone which is inside the user's hand.

```
// Provided by XR_EXT_hand_tracking
#define XR_HAND_JOINT_COUNT_EXT 26
```

XR_HAND_JOINT_COUNT_EXT defines the number of hand joint enumerants defined in XrHandJointEXT

New Object Types

- XrHandTrackerEXT

New Flag Types

New Enum Constants

- XR_HAND_JOINT_COUNT_EXT

XrObjectType enumeration is extended with:

- `XR_OBJECT_TYPE_HAND_TRACKER_EXT`

XrStructureType enumeration is extended with:

- `XR_TYPE_SYSTEM_HAND_TRACKING_PROPERTIES_EXT`
- `XR_TYPE_HAND_TRACKER_CREATE_INFO_EXT`
- `XR_TYPE_HAND_JOINTS_LOCATE_INFO_EXT`
- `XR_TYPE_HAND_JOINT_LOCATIONS_EXT`
- `XR_TYPE_HAND_JOINT_VELOCITIES_EXT`

New Enums

- XrHandEXT
- XrHandJointEXT
- XrHandJointSetEXT

New Structures

- XrSystemHandTrackingPropertiesEXT
- XrHandTrackerCreateInfoEXT
- XrHandJointsLocateInfoEXT
- XrHandJointLocationEXT
- XrHandJointVelocityEXT
- XrHandJointLocationsEXT
- XrHandJointVelocitiesEXT

New Functions

- xrCreateHandTrackerEXT
- xrDestroyHandTrackerEXT
- xrLocateHandJointsEXT

Issues

Version History

- Revision 1, 2019-09-16 (Yin LI)
  - Initial extension description
- Revision 2, 2020-04-20 (Yin LI)
  - Replace hand joint spaces to locate hand joints function.
- Revision 3, 2021-04-13 (Rylie Pavlik, Rune Berg)
  - Fix example code to properly use `xrGetInstanceProcAddr`.

    - Add recommended bone names
- Revision 4, 2021-04-15 (Rune Berg)
    - Clarify that use of this extension produces an unobstructed hand range of motion.

12.38. XR_EXT_hand_tracking_data_source

Name String

`XR_EXT_hand_tracking_data_source`

Extension Type

Instance extension

Registered Extension Number

429

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

OpenXR 1.0
and
`XR_EXT_hand_tracking`

Last Modified Date

2023-01-23

IP Status

No known IP claims.

Contributors

Jakob Bornecrantz, Collabora
John Kearney, Meta
Robert Memmott, Meta
Andreas Selvik, Meta
Yin Li, Microsoft
Robert Blenkinsopp, Ultraleap
Nathan Nuber, Valve

12.38.1. Overview

This extension augments the XR_EXT_hand_tracking extension.

Runtimes **may** support a variety of data sources for hand joint data for XR_EXT_hand_tracking, and some runtimes and devices **may** use joint data from multiple sources. This extension allows an application and the runtime to communicate about and make use of those data sources in a cooperative manner.

This extension allows the application to specify the data sources that it wants data from when creating a hand tracking handle, and allows the runtime to specify the currently active data source.

The application **must** enable the XR_EXT_hand_tracking extension in order to use this extension.

12.38.2. Data Sources

The XrHandTrackingDataSourceEXT enum describes a hand tracking data source when creating an XrHandTrackerEXT handle.

```
// Provided by XR_EXT_hand_tracking_data_source
typedef enum XrHandTrackingDataSourceEXT {
    XR_HAND_TRACKING_DATA_SOURCE_UNOBSTRUCTED_EXT = 1,
    XR_HAND_TRACKING_DATA_SOURCE_CONTROLLER_EXT = 2,
  // Provided by XR_META_hand_tracking_wide_motion_mode2
    XR_HAND_TRACKING_DATA_SOURCE_UNOBSTRUCTED_WIDE_MOTION_META = 1000695000,
    XR_HAND_TRACKING_DATA_SOURCE_MAX_ENUM_EXT = 0x7FFFFFFF
} XrHandTrackingDataSourceEXT;
```

The application **can** use XrHandTrackingDataSourceEXT with XrHandTrackingDataSourceInfoEXT when calling xrCreateHandTrackerEXT to tell the runtime all supported data sources for the application for the hand tracking inputs.

The application **can** use it with XrHandTrackingDataSourceStateEXT when calling xrLocateHandJointsEXT to inspect what data source the runtime used for the returned hand joint locations.

If the XR_EXT_hand_joints_motion_range extension is supported by the runtime and the data source is XR_HAND_TRACKING_DATA_SOURCE_CONTROLLER_EXT, then it is expected that application will use that extension when retrieving hand joint poses.

Enumerant Descriptions

- `XR_HAND_TRACKING_DATA_SOURCE_UNOBSTRUCTED_EXT` - This data source value indicates that the hand tracking data source supports using individual fingers and joints separately. Examples of such sources include optical hand tracking, data gloves, or motion capture devices.
- `XR_HAND_TRACKING_DATA_SOURCE_CONTROLLER_EXT` - This data source value indicates that the hand tracking data source is a motion controller. The runtime **must** not supply this data source if the controller providing the data is not actively held in the user's hand, but **may** still provide data if the runtime is unable to detect if the controller is not in the user's hand, or a user selected policy changes this behavior. Unless specified otherwise by another extension, data returned from `XR_HAND_TRACKING_DATA_SOURCE_CONTROLLER_EXT` **must** behave as `XR_HAND_JOINTS_MOTION_RANGE_UNOBSTRUCTED_EXT`.

12.38.3. Specifying Acceptable Data Source at Creation

The XrHandTrackingDataSourceInfoEXT structure is defined as:

```
// Provided by XR_EXT_hand_tracking_data_source
typedef struct XrHandTrackingDataSourceInfoEXT {
    XrStructureType                 type;
    const void*                     next;
    uint32_t                        requestedDataSourceCount;
    XrHandTrackingDataSourceEXT*    requestedDataSources;
} XrHandTrackingDataSourceInfoEXT;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `requestedDataSourceCount` is the number of elements in the `requestedDataSources` array.
- `requestedDataSources` is an array of XrHandTrackingDataSourceEXT that the application accepts.

The XrHandTrackingDataSourceInfoEXT is a structure that an application **can** chain to XrHandTrackerCreateInfoEXT::`next` to specify the hand tracking data sources that the application accepts.

Because the hand tracking device **may** change during a running session, the runtime **may** return a valid XrHandTrackerEXT handle even if there is no currently active hand tracking device or the active device does not safisty any or all data sources requested by the applications's call to

xrCreateHandTrackerEXT. The runtime **may** instead return XR_ERROR_FEATURE_UNSUPPORTED from xrCreateHandTrackerEXT, if for example the runtime believes it will never be able to satisfy the request.

If any value in requestedDataSources is duplicated, the runtime **must** return XR_ERROR_VALIDATION_FAILURE from the call to xrCreateHandTrackerEXT. If requestedDataSourceCount is 0, the runtime **must** return XR_ERROR_VALIDATION_FAILURE from the call to xrCreateHandTrackerEXT.

Valid Usage (Implicit)

- The XR_EXT_hand_tracking_data_source extension **must** be enabled prior to using XrHandTrackingDataSourceInfoEXT
- type **must** be XR_TYPE_HAND_TRACKING_DATA_SOURCE_INFO_EXT
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- If requestedDataSourceCount is not 0, requestedDataSources **must** be a pointer to an array of requestedDataSourceCount XrHandTrackingDataSourceEXT values

12.38.4. Getting Data Source With Data

The XrHandTrackingDataSourceStateEXT structure is defined as:

```
// Provided by XR_EXT_hand_tracking_data_source
typedef struct XrHandTrackingDataSourceStateEXT {
    XrStructureType                type;
    void*                          next;
    XrBool32                       isActive;
    XrHandTrackingDataSourceEXT    dataSource;
} XrHandTrackingDataSourceStateEXT;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain.
- isActive indicating there is an active data source
- dataSource indicating the data source that was used to generate the hand tracking joints.

XrHandTrackingDataSourceStateEXT is a structure that an application **can** chain to XrHandJointLocationsEXT::next when calling xrLocateHandJointsEXT to retrieve the data source of the currently active hand tracking device.

When the returned `isActive` is `XR_FALSE`, it indicates the currently active hand tracking device does not support any of the requested data sources. In these cases, the runtime **must** also return no valid tracking locations for hand joints from this xrLocateHandJointsEXT function.

If the tracker was not created with XrHandTrackingDataSourceInfoEXT chained to XrHandTrackerCreateInfoEXT::`next`, then the runtime **must** return `XR_ERROR_VALIDATION_FAILURE`, if XrHandTrackingDataSourceStateEXT is passed in the call to xrLocateHandJointsEXT.

If there is an active hand tracking device that is one of the specified XrHandTrackingDataSourceInfoEXT::`requestedDataSources`, the runtime **must** set `isActive` to `XR_TRUE`. When the runtime sets `isActive` to `XR_TRUE`, the runtime **must** set `dataSource` indicate the active data source. The runtime **must** return a `dataSource` that is a subset of the XrHandTrackingDataSourceInfoEXT::`requestedDataSources` when creating the corresponding hand tracker.

Valid Usage (Implicit)

- The `XR_EXT_hand_tracking_data_source` extension **must** be enabled prior to using XrHandTrackingDataSourceStateEXT
- `type` **must** be `XR_TYPE_HAND_TRACKING_DATA_SOURCE_STATE_EXT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

12.38.5. New Structures

- Extending XrHandJointLocationsEXT:
  - XrHandTrackingDataSourceStateEXT
- Extending XrHandTrackerCreateInfoEXT:
  - XrHandTrackingDataSourceInfoEXT

12.38.6. New Enums

- XrHandTrackingDataSourceEXT

12.38.7. New Enum Constants

- `XR_EXT_HAND_TRACKING_DATA_SOURCE_EXTENSION_NAME`
- `XR_EXT_hand_tracking_data_source_SPEC_VERSION`
- Extending XrStructureType:
  - `XR_TYPE_HAND_TRACKING_DATA_SOURCE_INFO_EXT`
  - `XR_TYPE_HAND_TRACKING_DATA_SOURCE_STATE_EXT`

12.38.8. Issues

1. Should this extension require `XR_HAND_JOINTS_MOTION_RANGE_CONFORMING_TO_CONTROLLER_EXT` if the data source is `XR_HAND_TRACKING_DATA_SOURCE_CONTROLLER_EXT` and `XR_EXT_hand_joints_motion_range` is not enabled?

   RESOLVED: Yes.

   It should not be required. We expect that a key use of the data from this extension will be replicating data hand tracking joint data for social purposes. For that use-case, the data returned in the style of `XR_HAND_JOINTS_MOTION_RANGE_UNOBSTRUCTED_EXT` is more appropriate.

   This is consistent with `XR_EXT_hand_tracking` extension which requires that the `jointLocations` represent `the range of motion of a human hand, without any obstructions`.

2. Should XrHandTrackingDataSourceInfoEXT include an `isActive` member or can it use `isActive` from XrHandJointLocationsEXT?

   RESOLVED: Yes.

   Yes; XrHandTrackingDataSourceInfoEXT needs to include the `isActive` member and cannot use the `isActive` from XrHandJointLocationsEXT as the meaning of these members is different.

   The `isActive` member of XrHandTrackingDataSourceStateEXT allows the runtime to describe if the tracking device is active. XrHandJointLocationsEXT::`isActive` describes if the tracking device is actively tracking. It is possible for a data source to be active but not actively tracking and we want to represent if the device is active in this extension.

12.38.9. Version History

- Revision 1, 2023-01-23 (John Kearney)
  - Initial extension description

12.39. XR_EXT_haptic_parametric

Name String

`XR_EXT_haptic_parametric`

Extension Type

Instance extension

Registered Extension Number

776

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Last Modified Date

2026-05-15

IP Status

No known IP claims.

Contributors

Thomas McGuire, Meta Platforms
Shuai Liu, ByteDance
Shanliang Xu, ByteDance

12.39.1. Overview

This extension enables applications to trigger haptic vibration events using a device-independent parametric haptic data format.

12.39.2. Parametric data format

The haptic event is described by three independent series of amplitude points, frequency points, and transients.

The amplitude and frequency points describe how the amplitude and frequency of the vibration change over time. The runtime applies linear interpolation between the amplitude and frequency points.

A transient is a short burst which has a strong and "clicky" characteristic. Transients are useful for adding a layer of distinct, discernible and emphasized points to the resulting vibration.

The runtime uses the parametric data to drive the haptic element. The runtime translates the parametric data into the data format needed to drive the haptic element in the best possible way, with graceful degradation for features not supported by the haptic element.

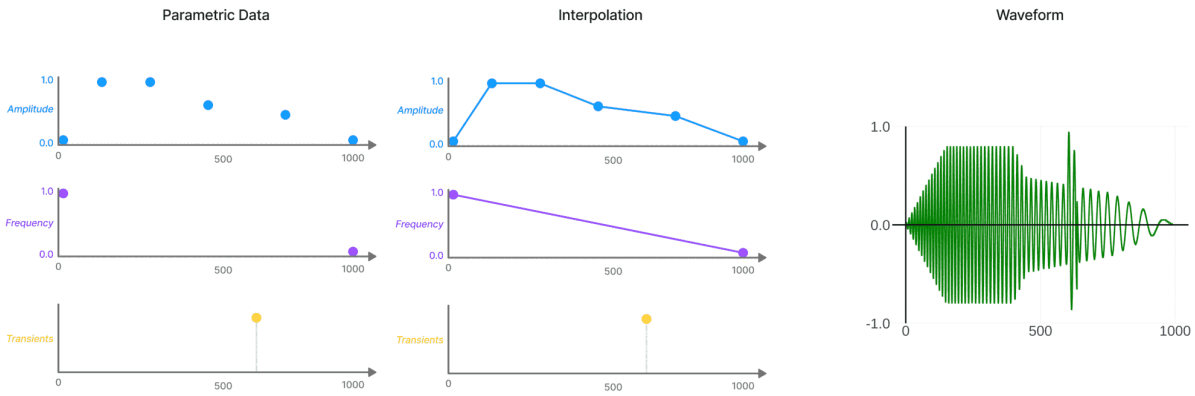


Figure 15. Example of translating parametric data to a PCM waveform for driving a VCM, using reserved headroom to make the transient stand out.

For example, a voice coil motor (VCM) haptic element is driven by pulse-code modulation (PCM) data, and the runtime translates the parametric data into PCM data by modulating a waveform. The amplitude and frequency of the waveform varies according to the amplitude and frequency points.

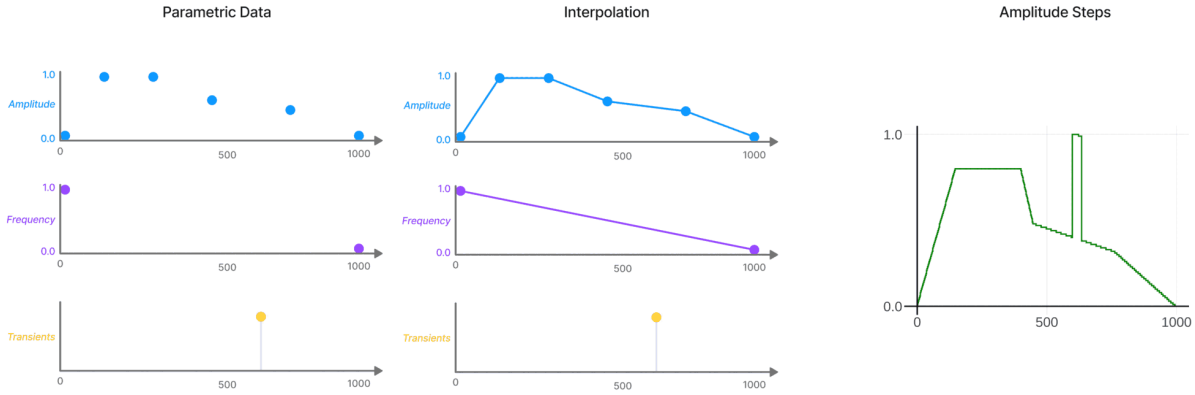


Figure 16. Example of translating parametric data to amplitude steps for driving an LRA, using reserved headroom to make the transient stand out.

For haptic elements that do not support varying the frequency of the vibration, like linear resonant actuators (LRAs), the runtime **must** ignore the frequency points and only change the vibration intensity according to the amplitude points.

The runtime **should** render a transient with dedicated system support if available. The runtime **must** make sure that a transient is perceivable when the transient's amplitude is higher than the amplitude defined by the amplitude points at the transient's time. For example the system might support briefly providing a non-nominal voltage spike (e.g. overdrive) to the actuator, which forces an abrupt change that is perceived as a clicky transient effect. Alternatively, the runtime **may** render a transient by reserving a headroom in the intensity used for the amplitude points, and use that headroom for the

transient, to make sure it stands out.

Amplitude and frequency values range from 0.0 to 1.0 (inclusive). The runtime maps the normalized amplitude range to the absolute intensity range supported by the haptic element. If the application specifies the absolute frequency range with the XrHapticParametricVibrationEXT::minFrequencyHz and XrHapticParametricVibrationEXT::maxFrequencyHz values, the runtime maps the normalized frequency range to the specified range. If the application does not specify the absolute frequency range, the runtime maps the normalized frequency range to the absolute frequency range supported by the haptic element.

The duration of the haptic event is defined by the time of the last amplitude point. When streaming, the haptic event consists of multiple haptic frames, and the last amplitude point is contained in the last haptic frame of the stream. Therefore the duration of the haptic event is not known upfront when streaming.

12.39.3. Inspect system capability

An application **can** inspect whether the system is capable of parametric haptics by extending the XrSystemProperties structure with XrSystemHapticParametricPropertiesEXT when calling xrGetSystemProperties.

The XrSystemHapticParametricPropertiesEXT structure is defined as:

```
// Provided by XR_EXT_haptic_parametric
typedef struct XrSystemHapticParametricPropertiesEXT {
    XrStructureType    type;
    void*              next;
    XrBool32           supportsParametricHaptics;
} XrSystemHapticParametricPropertiesEXT;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. For more details about the structure chain, see the structure being extended (XrSystemProperties).
- supportsParametricHaptics is an XrBool32, indicating if the current system is capable of parametric haptics.

If the system does not support parametric haptics:

- The runtime **must** return XR_FALSE for supportsParametricHaptics.
- The runtime **must** return XR_ERROR_FEATURE_UNSUPPORTED from xrHapticParametricGetPropertiesEXT.

- The runtime **must** not trigger a haptic event when the application calls xrApplyHapticFeedback with XrHapticParametricVibrationEXT, and **must** return XR_ERROR_FEATURE_UNSUPPORTED.

If the system supports parametric haptics:

- The runtime **must** return XR_TRUE for supportsParametricHaptics.
- The runtime **must** support haptic events of type XrHapticParametricVibrationEXT on all output component paths that are supported for haptic events of type XrHapticVibration.

Valid Usage (Implicit)

- The XR_EXT_haptic_parametric extension **must** be enabled prior to using XrSystemHapticParametricPropertiesEXT
- type **must** be XR_TYPE_SYSTEM_HAPTIC_PARAMETRIC_PROPERTIES_EXT
- next **must** be NULL or a valid pointer to the next structure in a structure chain

12.39.4. Trigger haptics

To trigger a parametric haptic event, create a XrHapticParametricVibrationEXT structure and call xrApplyHapticFeedback with it.

The XrHapticParametricVibrationEXT structure is defined as:

```
// Provided by XR_EXT_haptic_parametric
typedef struct XrHapticParametricVibrationEXT {
    XrStructureType                          type;
    const void*                              next;
    uint32_t                                 amplitudePointCount;
    const XrHapticParametricPointEXT*        amplitudePoints;
    uint32_t                                 frequencyPointCount;
    const XrHapticParametricPointEXT*        frequencyPoints;
    uint32_t                                 transientCount;
    const XrHapticParametricTransientEXT*    transients;
    float                                    minFrequencyHz;
    float                                    maxFrequencyHz;
    XrHapticParametricStreamFrameTypeEXT     streamFrameType;
} XrHapticParametricVibrationEXT;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `amplitudePointCount` is the number of points in `amplitudePoints`.
- `amplitudePoints` is an array of XrHapticParametricPointEXT, describing how the intensity of the vibration changes over time.
- `frequencyPointCount` is the number of points in `frequencyPoints`.
- `frequencyPoints` is an array of XrHapticParametricPointEXT, describing how the frequency of the vibration changes over time. If it is `NULL`, a vibration frequency of 0.5 is used for the entire haptic event.
- `transientCount` is the number of transients in `transients`.
- `transients` is an array of XrHapticParametricTransientEXT, describing the transients in the haptic event. If it is `NULL`, the haptic event has no transients.
- `minFrequencyHz` is the absolute frequency, in Hertz, that the normalized frequency of 0.0 maps to. If it is XR_FREQUENCY_UNSPECIFIED, the runtime maps 0.0 to the lowest frequency supported by the haptic element.
- `maxFrequencyHz` is the absolute frequency, in Hertz, that the normalized frequency of 1.0 maps to. If it is XR_FREQUENCY_UNSPECIFIED, the runtime maps 1.0 to the highest frequency supported by the haptic element.
- `streamFrameType` is the XrHapticParametricStreamFrameTypeEXT that describes if and how the data in this structure is part of a longer haptic stream.

This structure describes a parametric haptic event.

The runtime **must** return `XR_ERROR_VALIDATION_FAILURE` from xrApplyHapticFeedback if any of the following conditions are not met:

- When `streamFrameType` is `XR_HAPTIC_PARAMETRIC_STREAM_FRAME_TYPE_NONE_EXT` or `XR_HAPTIC_PARAMETRIC_STREAM_FRAME_TYPE_FIRST_FRAME_EXT`, the structure **must** contain at least two amplitude points. For all other values of `streamFrameType`, the structure **must** contain at least one amplitude point.
- The structure **must** not contain more than XR_HAPTIC_PARAMETRIC_MAX_POINTS_TRANSIENTS_EXT amplitude points, frequency points or transients, respectively. An application **can** trigger haptic events with more than XR_HAPTIC_PARAMETRIC_MAX_POINTS_TRANSIENTS_EXT amplitude points, frequency points or transients by calling xrApplyHapticFeedback multiple times in a sequence, see Streaming for details.

- When `streamFrameType` is `XR_HAPTIC_PARAMETRIC_STREAM_FRAME_TYPE_NONE_EXT` or `XR_HAPTIC_PARAMETRIC_STREAM_FRAME_TYPE_FIRST_FRAME_EXT`, the first amplitude and frequency point **must** be at time 0.
- When `streamFrameType` is `XR_HAPTIC_PARAMETRIC_STREAM_FRAME_TYPE_FIRST_FRAME_EXT`, the time of the last amplitude point in the frame **must** be greater than or equal to XrHapticParametricPropertiesEXT::`minimumFirstFrameDuration`.
- The time of the last frequency point and the last transient of the stream **must** be less than or equal to the time of the last amplitude point of the stream.
- When `streamFrameType` is `XR_HAPTIC_PARAMETRIC_STREAM_FRAME_TYPE_INTERMEDIATE_FRAME_EXT` or `XR_HAPTIC_PARAMETRIC_STREAM_FRAME_TYPE_LAST_FRAME_EXT`, a stream **must** already be running on the haptic element.
- `minFrequencyHz` and `maxFrequencyHz` **must** either be both XR_FREQUENCY_UNSPECIFIED, or both specified in the range XR_HAPTIC_PARAMETRIC_FREQUENCY_MIN_HZ_EXT - XR_HAPTIC_PARAMETRIC_FREQUENCY_MAX_HZ_EXT (inclusive).
- `maxFrequencyHz` **must** be equal to or larger than `minFrequencyHz`, or XR_FREQUENCY_UNSPECIFIED.
- When `streamFrameType` is `XR_HAPTIC_PARAMETRIC_STREAM_FRAME_TYPE_INTERMEDIATE_FRAME_EXT` or `XR_HAPTIC_PARAMETRIC_STREAM_FRAME_TYPE_LAST_FRAME_EXT`, `minFrequencyHz` and `maxFrequencyHz` **must** both be XR_FREQUENCY_UNSPECIFIED. The application **may** only set the absolute frequency range in the first haptic frame, and the runtime uses the range for the entire haptic event.
- Each amplitude point and frequency point **must** not have an earlier time than the one preceding it.
- Each transient **must** have a later time than the one preceding it.
- The amplitude and frequency of a point or transient **must** be between 0.0 and 1.0 (inclusive).

In addition to the restrictions above, the following is explicitly allowed:

- The first transient **may** be at any time, it does not need to be at time 0.
- The number of amplitude points, frequency points and transients **may** be completely independent of each other.
- Amplitude points, frequency points and transients with the same array index **may** be at a different times.
- There **may** be multiple amplitude points or frequency points at the same time. Applications **can** use this to model an abrupt jump in amplitude or frequency. The order of the points in the array determines the starting and ending values of the jump.

If the time of the last frequency point of the stream is earlier than the time of the last amplitude point of the stream, the runtime **must** continue and finish the haptic event with the frequency of the last frequency point. As per the validation rules, the opposite case, a stream in which the time of the last amplitude point of the stream is earlier than the time of the last frequency point of the stream, is not allowed, and the runtime **must** return `XR_ERROR_VALIDATION_FAILURE` from xrApplyHapticFeedback.

The application **may** call xrHapticParametricGetPropertiesEXT to determine the frequency range

supported by the haptic element. If the range specified by `minFrequencyHz` and `maxFrequencyHz` is outside of the supported range, the runtime **must** clamp the values to the supported range. The runtime then maps the normalized frequency range to the clamped absolute frequency range.

When `streamFrameType` is `XR_HAPTIC_PARAMETRIC_STREAM_FRAME_TYPE_NONE_EXT` or `XR_HAPTIC_PARAMETRIC_STREAM_FRAME_TYPE_FIRST_FRAME_EXT`, a new haptic event is started. In this case, the runtime **must** cancel any unfinished haptic events currently running on the haptic element, as usual for the core specification. This applies to all types of haptic events, including e.g. XrHapticPcmVibrationFB, XrHapticVibration and XrHapticParametricVibrationEXT.

When the call to xrApplyHapticFeedback fails with an error code, the runtime **must** ignore the new frame and discard the haptic data contained in it. The runtime **must** not cancel or otherwise modify an existing stream, if any, currently running on the haptic element. The runtime **must** not start a new haptic event.

Valid Usage (Implicit)

- The `XR_EXT_haptic_parametric` extension **must** be enabled prior to using XrHapticParametricVibrationEXT
- `type` **must** be `XR_TYPE_HAPTIC_PARAMETRIC_VIBRATION_EXT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `amplitudePoints` **must** be a pointer to an array of `amplitudePointCount` valid XrHapticParametricPointEXT structures
- If `frequencyPointCount` is not `0`, `frequencyPoints` **must** be a pointer to an array of `frequencyPointCount` valid XrHapticParametricPointEXT structures
- If `transientCount` is not `0`, `transients` **must** be a pointer to an array of `transientCount` valid XrHapticParametricTransientEXT structures
- `streamFrameType` **must** be a valid XrHapticParametricStreamFrameTypeEXT value
- The `amplitudePointCount` parameter **must** be greater than `0`

The XrHapticParametricPointEXT structure is defined as:

```
// Provided by XR_EXT_haptic_parametric
typedef struct XrHapticParametricPointEXT {
    XrDuration    time;
    float         value;
} XrHapticParametricPointEXT;
```

Member Descriptions

- `time` is the time of the amplitude or frequency point, as a duration in nanoseconds since the start of the haptic event.
- `value` is the normalized value of the vibration amplitude or frequency at this point in time, with a value between 0.0 and 1.0 (inclusive).

This structure describes an amplitude or frequency point.

Valid Usage (Implicit)

- The `XR_EXT_haptic_parametric` extension **must** be enabled prior to using XrHapticParametricPointEXT

The XrHapticParametricTransientEXT structure is defined as:

```
// Provided by XR_EXT_haptic_parametric
typedef struct XrHapticParametricTransientEXT {
    XrDuration    time;
    float         amplitude;
    float         frequency;
} XrHapticParametricTransientEXT;
```

Member Descriptions

- `time` is the time of the transient, as a duration in nanoseconds since the start of the haptic event.
- `amplitude` is the normalized amplitude of the transient, with a value between 0.0 and 1.0 (inclusive).
- `frequency` is the normalized frequency of the transient, with a value between 0.0 and 1.0 (inclusive).

This structure describes a transient, which is a brief and abrupt change of amplitude and frequency.

The intensity of a transient is set by the amplitude value, while the frequency value has an effect on how sharp a transient feels.

Valid Usage (Implicit)

- The XR_EXT_haptic_parametric extension **must** be enabled prior to using XrHapticParametricTransientEXT

```
// Provided by XR_EXT_haptic_parametric
#define XR_HAPTIC_PARAMETRIC_MAX_POINTS_TRANSIENTS_EXT 500
```

XR_HAPTIC_PARAMETRIC_MAX_POINTS_TRANSIENTS_EXT defines the maximum amount of amplitude points, frequency points or transients that the runtime accepts in a single call to xrApplyHapticFeedback.

```
// Provided by XR_EXT_haptic_parametric
#define XR_HAPTIC_PARAMETRIC_FREQUENCY_MIN_HZ_EXT 1
```

XR_HAPTIC_PARAMETRIC_FREQUENCY_MIN_HZ_EXT defines the minimum valid frequency value, in Hertz, for XrHapticParametricVibrationEXT::minFrequencyHz and XrHapticParametricVibrationEXT::maxFrequencyHz.

```
// Provided by XR_EXT_haptic_parametric
#define XR_HAPTIC_PARAMETRIC_FREQUENCY_MAX_HZ_EXT 1000
```

XR_HAPTIC_PARAMETRIC_FREQUENCY_MAX_HZ_EXT defines the maximum valid frequency value, in Hertz, for XrHapticParametricVibrationEXT::minFrequencyHz and XrHapticParametricVibrationEXT::maxFrequencyHz.

12.39.5. Streaming

An application **can** trigger a parametric haptic event in two ways:

- A single call to xrApplyHapticFeedback, in which the data for the entire haptic event is known in advance and submitted in one call.
  - In this case, XrHapticParametricVibrationEXT::streamFrameType is XR_HAPTIC_PARAMETRIC_STREAM_FRAME_TYPE_NONE_EXT.
- A sequence of calls over time to xrApplyHapticFeedback, in which the data of the entire haptic event is submitted piece by piece in multiple calls.
  - This is called *streaming*.
  - The haptic data submitted in one call is called a *haptic frame*.
  - In this case, XrHapticParametricVibrationEXT::streamFrameType is

`XR_HAPTIC_PARAMETRIC_STREAM_FRAME_TYPE_FIRST_FRAME_EXT` for the first call, `XR_HAPTIC_PARAMETRIC_STREAM_FRAME_TYPE_INTERMEDIATE_FRAME_EXT` for calls in the middle and `XR_HAPTIC_PARAMETRIC_STREAM_FRAME_TYPE_LAST_FRAME_EXT` for the last call of the stream sequence.

Streaming a haptic event is required when the entire haptic data is not yet known at the time of the first call, for example when the haptic data is generated on the fly. Streaming is also required when the haptic event has more amplitude points, frequency points or transients than the maximum supported in one call, for example for a long running haptic event.

Each haptic element has at most one stream running on it. The runtime **must** support running multiple independent streams on different haptic elements at the same time, e.g. one stream on the main haptic element of the device in the user's left hand and one stream on the trigger haptic element of the device in the user's right hand.

If an application wants to continue a stream to extend the haptic event, it **should** submit a new haptic frame before the runtime finishes playing back the previously submitted haptic frame. The application **can** extend the stream if the previously submitted frame is of type `XR_HAPTIC_PARAMETRIC_STREAM_FRAME_TYPE_FIRST_FRAME_EXT` or `XR_HAPTIC_PARAMETRIC_STREAM_FRAME_TYPE_INTERMEDIATE_FRAME_EXT`. If it is not, the runtime **must** return `XR_ERROR_VALIDATION_FAILURE` from xrApplyHapticFeedback, as per the validation rules.

If the application submits the new haptic frame too late and the runtime has already finished playing back the previously submitted haptic frame, the runtime **must** continue the vibration with the intensity of the last amplitude point and the frequency of the last frequency point, for exactly XR_HAPTIC_PARAMETRIC_VIBRATION_EXTEND_DURATION_EXT nanoseconds, or until the application submits a new haptic frame, or the application stops haptic feedback with xrStopHapticFeedback. If the application submits a new frame after the previously submitted haptic frame finished, the runtime **must** immediately change the intensity and frequency to the intensity and frequency described by the amplitude and frequency points of the newly submitted haptic frame at the current playback position. The runtime **must** ignore any amplitude points, frequency points or transients in the newly submitted haptic frame that are before the current playback position.

If the application does not submit a new haptic frame within XR_HAPTIC_PARAMETRIC_VIBRATION_EXTEND_DURATION_EXT nanoseconds after the previously submitted haptic frame finished, the runtime **must** stop the vibration. After this, the haptic event is still considered running, and the playback time is still advancing, but no vibration is triggered anymore. The runtime **must** still accept new haptic frames of type `XR_HAPTIC_PARAMETRIC_STREAM_FRAME_TYPE_INTERMEDIATE_FRAME_EXT` or `XR_HAPTIC_PARAMETRIC_STREAM_FRAME_TYPE_LAST_FRAME_EXT`. Once a new haptic frame is submitted, the runtime **must** resume the vibration.

To end a stream, the application **can** submit a haptic frame of type `XR_HAPTIC_PARAMETRIC_STREAM_FRAME_TYPE_LAST_FRAME_EXT` or call xrStopHapticFeedback. xrStopHapticFeedback stops the stream immediately, while `XR_HAPTIC_PARAMETRIC_STREAM_FRAME_TYPE_LAST_FRAME_EXT` stops the stream after the last haptic frame has finished playing.

A call to xrApplyHapticFeedback with XR_HAPTIC_PARAMETRIC_STREAM_FRAME_TYPE_FIRST_FRAME_EXT starts a new haptic event and a new stream. A call to xrApplyHapticFeedback with XR_HAPTIC_PARAMETRIC_STREAM_FRAME_TYPE_NONE_EXT starts a new haptic event, but not a stream.

The runtime **may** process the haptic data submitted by the application periodically, in batches. This affects the rate at which applications that generate haptic data on the fly **should** submit new haptic frames. The application **may** query this rate with xrHapticParametricGetPropertiesEXT. A faster rate than XrHapticParametricPropertiesEXT::idealFrameSubmissionRate would not be useful, as the runtime processes the haptic data at a slower rate anyway. A slower rate than XrHapticParametricPropertiesEXT::idealFrameSubmissionRate would increase latency between generating the haptic data and the data being played back.

When a stream starts, the runtime **may** need a certain minimum amount of haptic data upfront. The application **should** query this amount of data with xrHapticParametricGetPropertiesEXT. XrHapticParametricPropertiesEXT::minimumFirstFrameDuration describes how much haptic data the runtime needs in the first haptic frame. If the application submits less haptic data in the first frame, the runtime **must** return XR_ERROR_VALIDATION_FAILURE from xrApplyHapticFeedback, as per the validation rules.

The application **should** not use XrHapticParametricPropertiesEXT::minimumFirstFrameDuration to determine when to send the second frame of a stream, and **should** instead use XrHapticParametricPropertiesEXT::idealFrameSubmissionRate. As an example, a runtime **may** want to build up a buffer of haptic data in the first frame, and for that **may** return a value of XrHapticParametricPropertiesEXT::minimumFirstFrameDuration that is higher than XrHapticParametricPropertiesEXT::idealFrameSubmissionRate.

The time of amplitude points, frequency points and transients is defined as the duration since the start of the haptic event. In a stream, the haptic event starts with the first frame, which means that the stream maintains a uniform time base throughout the entire haptic event.

The XrHapticParametricStreamFrameTypeEXT enum is defined as:

```
// Provided by XR_EXT_haptic_parametric
typedef enum XrHapticParametricStreamFrameTypeEXT {
    XR_HAPTIC_PARAMETRIC_STREAM_FRAME_TYPE_NONE_EXT = 0,
    XR_HAPTIC_PARAMETRIC_STREAM_FRAME_TYPE_FIRST_FRAME_EXT = 1,
    XR_HAPTIC_PARAMETRIC_STREAM_FRAME_TYPE_INTERMEDIATE_FRAME_EXT = 2,
    XR_HAPTIC_PARAMETRIC_STREAM_FRAME_TYPE_LAST_FRAME_EXT = 3,
    XR_HAPTIC_PARAMETRIC_STREAM_FRAME_TYPE_MAX_ENUM_EXT = 0x7FFFFFFF
} XrHapticParametricStreamFrameTypeEXT;
```

This enum describes if and how the haptic data in XrHapticParametricVibrationEXT is part of a stream.

Enumerant Descriptions

- XR_HAPTIC_PARAMETRIC_STREAM_FRAME_TYPE_NONE_EXT The haptic data is not part of a stream.
- XR_HAPTIC_PARAMETRIC_STREAM_FRAME_TYPE_FIRST_FRAME_EXT The haptic data is the first haptic frame of the stream.
- XR_HAPTIC_PARAMETRIC_STREAM_FRAME_TYPE_INTERMEDIATE_FRAME_EXT The haptic data is a haptic frame in the middle of the stream.
- XR_HAPTIC_PARAMETRIC_STREAM_FRAME_TYPE_LAST_FRAME_EXT The haptic data is the last haptic frame of the stream.

```
// Provided by XR_EXT_haptic_parametric
#define XR_HAPTIC_PARAMETRIC_VIBRATION_EXTEND_DURATION_EXT 50000000
```

XR_HAPTIC_PARAMETRIC_VIBRATION_EXTEND_DURATION_EXT defines the duration, in nanoseconds, for which the runtime continues the vibration after the last submitted haptic frame of type XR_HAPTIC_PARAMETRIC_STREAM_FRAME_TYPE_FIRST_FRAME_EXT or XR_HAPTIC_PARAMETRIC_STREAM_FRAME_TYPE_INTERMEDIATE_FRAME_EXT has finished playing.

12.39.6. Properties

The xrHapticParametricGetPropertiesEXT function is defined as:

```
// Provided by XR_EXT_haptic_parametric
XrResult xrHapticParametricGetPropertiesEXT(
    XrSession                                   session,
    const XrHapticActionInfo*                   hapticActionInfo,
    XrHapticParametricPropertiesEXT*            parametricProperties);
```

Parameter Descriptions

- session is the specified XrSession.
- hapticActionInfo identifies the haptic element
- parametricProperties is a pointer to xrHapticParametricGetPropertiesEXT which is populated by the runtime.

xrHapticParametricGetPropertiesEXT retrieves various properties of a haptic element that relate to parametric haptics.

The action and subaction paths provided by hapticActionInfo identify the haptic element. If multiple

haptic elements are identified by `hapticActionInfo`, the runtime **must** select a single haptic element.

Valid Usage (Implicit)

- The `XR_EXT_haptic_parametric` extension **must** be enabled prior to calling xrHapticParametricGetPropertiesEXT
- `session` **must** be a valid XrSession handle
- `hapticActionInfo` **must** be a pointer to a valid XrHapticActionInfo structure
- `parametricProperties` **must** be a pointer to an XrHapticParametricPropertiesEXT structure

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_ACTIONSET_NOT_ATTACHED`
- `XR_ERROR_ACTION_TYPE_MISMATCH`
- `XR_ERROR_FEATURE_UNSUPPORTED`
- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_PATH_INVALID`
- `XR_ERROR_PATH_UNSUPPORTED`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_VALIDATION_FAILURE`

The XrHapticParametricPropertiesEXT structure is defined as:

```
// Provided by XR_EXT_haptic_parametric
typedef struct XrHapticParametricPropertiesEXT {
    XrStructureType    type;
    void*              next;
    XrDuration         idealFrameSubmissionRate;
    XrDuration         minimumFirstFrameDuration;
    float              minFrequencyHz;
    float              maxFrequencyHz;
} XrHapticParametricPropertiesEXT;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `idealFrameSubmissionRate` is the ideal rate the application **should** submit haptic frames with, in nanoseconds, for applications that generate haptic data on the fly.
- `minimumFirstFrameDuration` is the minimum amount of haptic data, in nanoseconds, the application **should** submit in the first haptic frame.
- `minFrequencyHz` is the minimum frequency, in Hertz, supported by the haptic element.
- `maxFrequencyHz` is the maximum frequency, in Hertz, supported by the haptic element.

This structure contains properties about streaming timing and supported frequency range specific to a haptic element.

The runtime **may** populate `idealFrameSubmissionRate` with 0 if it has no restrictions on the haptic frame submission rate. In this case, the application **may** freely choose any positive haptic frame submission rate.

The runtime **may** populate `minimumFirstFrameDuration` with 0 if it has no restrictions on the minimum first haptic frame duration. In this case, the application **may** freely choose the amount of haptic data in the first haptic frame.

`minFrequencyHz` and `maxFrequencyHz` specify the absolute frequency range supported by the haptic element. The runtime **may** populate both with XR_FREQUENCY_UNSPECIFIED if it does not have the information. Otherwise, `maxFrequencyHz` **must** be `minFrequencyHz` or higher, and both values **must** be between XR_HAPTIC_PARAMETRIC_FREQUENCY_MIN_HZ_EXT and XR_HAPTIC_PARAMETRIC_FREQUENCY_MAX_HZ_EXT (inclusive).

Valid Usage (Implicit)

- The `XR_EXT_haptic_parametric` extension **must** be enabled prior to using XrHapticParametricPropertiesEXT
- `type` **must** be `XR_TYPE_HAPTIC_PARAMETRIC_PROPERTIES_EXT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

12.39.7. Example code

The following example code shows how to trigger a one-second parametric haptic event that matches the illustration shown above. In the example, XR_FREQUENCY_UNSPECIFIED is used, so no absolute frequency range is specified. Streaming is not used in this example, as `XR_HAPTIC_PARAMETRIC_STREAM_FRAME_TYPE_NONE_EXT` is used as the haptic frame type.

```
XrSession session;                   // previously initialized
XrHapticActionInfo hapticActionInfo; // previously initialized

std::vector<XrHapticParametricPointEXT> amplitudePoints{{
  {
    .time = 0,
    .value = 0.0,
  },
  {
    .time = 100000000,
    .value = 1.0,
  },
  {
    .time = 270000000,
    .value = 1.0,
  },
  {
    .time = 480000000,
    .value = 0.7,
  },
  {
    .time = 750000000,
    .value = 0.6,
  },
  {
    .time = 1000000000,
    .value = 1.0,
  },
}};
std::vector<XrHapticParametricPointEXT> frequencyPoints{{
```

```
  {
    .time = 0,
    .value = 1.0,
  },
  {
    .time = 1000000000,
    .value = 0.0,
  },
}};
std::vector<XrHapticParametricTransientEXT> transients{{
  {
    .time = 600000000,
    .amplitude = 1.0,
    .frequency = 1.0,
  }
}};

XrHapticParametricVibrationEXT parametricVibration{
  .type = XR_TYPE_HAPTIC_PARAMETRIC_VIBRATION_EXT,
  .amplitudePointCount = static_cast<uint32_t>(amplitudePoints.size()),
  .amplitudePoints = amplitudePoints.data(),
  .frequencyPointCount = static_cast<uint32_t>(frequencyPoints.size()),
  .frequencyPoints = frequencyPoints.data(),
  .transientCount = static_cast<uint32_t>(transients.size()),
  .transients = transients.data(),
  .minFrequencyHz = XR_FREQUENCY_UNSPECIFIED,
  .maxFrequencyHz = XR_FREQUENCY_UNSPECIFIED,
  .streamFrameType = XR_HAPTIC_PARAMETRIC_STREAM_FRAME_TYPE_NONE_EXT,
};

CHK_XR(xrApplyHapticFeedback(session, &hapticActionInfo,
    reinterpret_cast<const XrHapticBaseHeader*>(&parametricVibration)));
```

The next example shows how to use streaming. The amplitude and frequency of the vibration is determined on the fly by calling the hypothetical `getDynamicAmplitude` and `getDynamicFrequency` functions.

The example submits haptic frames at the ideal frame submission rate recommended by the runtime. For this purpose, it sleeps between haptic frame submissions, which assumes a dedicated thread. Alternatively, it would also be possible to submit the haptic frames from any other thread at a different rate, as long as the application submits a new haptic frame before the runtime finishes playing back the previously submitted haptic frame. The example uses xrHapticParametricGetPropertiesEXT to query the ideal frame submission rate as well as the minimum first frame duration.

For the sake of brevity in the example code, the haptic event plays forever and is not stopped.

```
```

```
XrInstance instance;                // previously initialized
XrSession session;                  // previously initialized
XrHapticActionInfo hapticActionInfo; // previously initialized

float getDynamicAmplitude(void); // generates amplitude data on the fly
float getDynamicFrequency(void); // generates frequency data on the fly

PFN_xrHapticParametricGetPropertiesEXT xrHapticParametricGetPropertiesEXT = nullptr;
CHK_XR(xrGetInstanceProcAddr(
    instance, "xrHapticParametricGetPropertiesEXT",
    (PFN_xrVoidFunction*)(&xrHapticParametricGetPropertiesEXT)));

XrHapticParametricPropertiesEXT properties {
  .type = XR_TYPE_HAPTIC_PARAMETRIC_PROPERTIES_EXT,
};
CHK_XR(xrHapticParametricGetPropertiesEXT(session, &hapticActionInfo, &properties));
// The runtime may: populate properties with 0, use a reasonable default in that case
if (properties.idealFrameSubmissionRate == 0) {
    properties.idealFrameSubmissionRate = 50 * 1e6; // 50ms
}
if (properties.minimumFirstFrameDuration == 0) {
    properties.minimumFirstFrameDuration = 100 * 1e6; // 100ms
}

int64_t endOfFrameNs = properties.minimumFirstFrameDuration;
std::vector<XrHapticParametricPointEXT> amplitudePoints{{{0, 0.5}, {endOfFrameNs, 0.5}}};
std::vector<XrHapticParametricPointEXT> frequencyPoints{{{0, 0.5}, {endOfFrameNs, 0.5}}};

XrHapticParametricVibrationEXT parametricVibration{
  .type = XR_TYPE_HAPTIC_PARAMETRIC_VIBRATION_EXT,
  .amplitudePointCount = static_cast<uint32_t>(amplitudePoints.size()),
  .amplitudePoints = amplitudePoints.data(),
  .frequencyPointCount = static_cast<uint32_t>(frequencyPoints.size()),
  .frequencyPoints = frequencyPoints.data(),
  .minFrequencyHz = XR_FREQUENCY_UNSPECIFIED,
  .maxFrequencyHz = XR_FREQUENCY_UNSPECIFIED,
  .streamFrameType = XR_HAPTIC_PARAMETRIC_STREAM_FRAME_TYPE_FIRST_FRAME_EXT,
};
CHK_XR(xrApplyHapticFeedback(session, &hapticActionInfo,
    reinterpret_cast<const XrHapticBaseHeader*>(&parametricVibration)));

auto firstFrameTime = std::chrono::steady_clock::now();
for (int frameNum = 1;; frameNum++) {
    // To maintain accurate timing, it is not possible to simply use
    // "sleep_for(idealFrameSubmissionRate)". Due to scheduling imprecisions and the
    // execution time of the surrounding code, the timing would drift over time,
eventually
    // causing the application to submit haptic frames too late.
```

```
    std::this_thread::sleep_until(firstFrameTime +
        std::chrono::nanoseconds{properties.idealFrameSubmissionRate} * frameNum);
    endOfFrameNs += properties.idealFrameSubmissionRate;

    XrHapticParametricPointEXT amplitudePoint{endOfFrameNs, getDynamicAmplitude()};
    XrHapticParametricPointEXT frequencyPoint{endOfFrameNs, getDynamicFrequency()};

    XrHapticParametricVibrationEXT parametricVibrationInt{
      .type = XR_TYPE_HAPTIC_PARAMETRIC_VIBRATION_EXT,
      .amplitudePointCount = 1,
      .amplitudePoints = &amplitudePoint,
      .frequencyPointCount = 1,
      .frequencyPoints = &frequencyPoint,
      .minFrequencyHz = XR_FREQUENCY_UNSPECIFIED,
      .maxFrequencyHz = XR_FREQUENCY_UNSPECIFIED,
      .streamFrameType = XR_HAPTIC_PARAMETRIC_STREAM_FRAME_TYPE_INTERMEDIATE_FRAME_EXT,
    };

    CHK_XR(xrApplyHapticFeedback(session, &hapticActionInfo,
        reinterpret_cast<const XrHapticBaseHeader*>(&parametricVibrationInt)));
}
```

12.39.8. New Macros

- XR_HAPTIC_PARAMETRIC_FREQUENCY_MAX_HZ_EXT
- XR_HAPTIC_PARAMETRIC_FREQUENCY_MIN_HZ_EXT
- XR_HAPTIC_PARAMETRIC_MAX_POINTS_TRANSIENTS_EXT
- XR_HAPTIC_PARAMETRIC_VIBRATION_EXTEND_DURATION_EXT

12.39.9. New Commands

- xrHapticParametricGetPropertiesEXT

12.39.10. New Structures

- XrHapticParametricPointEXT
- XrHapticParametricPropertiesEXT
- XrHapticParametricTransientEXT
- XrHapticParametricVibrationEXT
- Extending XrSystemProperties:
  - XrSystemHapticParametricPropertiesEXT

12.39.11. New Enums

- XrHapticParametricStreamFrameTypeEXT

12.39.12. New Enum Constants

- XR_EXT_HAPTIC_PARAMETRIC_EXTENSION_NAME
- XR_EXT_haptic_parametric_SPEC_VERSION
- Extending XrStructureType:
  - XR_TYPE_HAPTIC_PARAMETRIC_PROPERTIES_EXT
  - XR_TYPE_HAPTIC_PARAMETRIC_VIBRATION_EXT
  - XR_TYPE_SYSTEM_HAPTIC_PARAMETRIC_PROPERTIES_EXT

12.39.13. Issues

12.39.14. Version History

- Revision 1, 2025-05-22 (Thomas McGuire, Meta Platforms)
  - Initial extension description

12.40. XR_EXT_interaction_profile_battery_state_display

Name String

XR_EXT_interaction_profile_battery_state_display

Extension Type

Instance extension

Registered Extension Number

837

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

OpenXR 1.0

Last Modified Date

2025-10-21

IP Status

No known IP claims.

Contributors

John Kearney, Meta
Daniel Willmott, Valve
Denny Rönngren, Varjo
Jakob Bornecrantz, NVIDIA
Nathan Nuber, Valve
Lachlan Ford, Google
Nihav Jain, Google

12.40.1. Overview

XR_EXT_interaction_profile_battery_state_display provides a mechanism for applications to query the runtime for battery state information for devices used with interaction profiles. This battery state information **should** only be used for display purposes and the runtime **should** ensure that this information matches what is used in built-in system user interfaces.

In cases where multiple physical devices provide input to the actions associated with a top level path, such as runtimes performing remapping for user customization or accessibility, those runtimes **should** make a best effort to present useful battery information in a single value.

Specifically, it is not expected for this information to be calibrated between runtimes or devices and **should** not be used to generate warnings or otherwise modify the application behavior.

12.40.2. Querying for battery state

The XrBatteryStateDisplayEXT structure is defined as:

```
// Provided by XR_EXT_interaction_profile_battery_state_display
typedef struct XrBatteryStateDisplayEXT {
    XrStructureType                       type;
    void*                                 next;
    XrBatteryStateDisplayStateFlagsEXT    stateFlags;
    float                                 batteryLevel;
} XrBatteryStateDisplayEXT;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `stateFlags` describes the validity of `batteryLevel` with bit masks defined in XrBatteryStateDisplayStateFlagBitsEXT
- `batteryLevel` is the level of the battery (from 0.0f to 1.0f) of the device connected by the associated entity or action.

To query the battery level, the application chains an XrBatteryStateDisplayEXT structure to the XrInteractionProfileState::`next` chain when calling xrGetCurrentInteractionProfile.

If xrGetCurrentInteractionProfile sets the XrInteractionProfileState::`interactionProfile` to XR_NULL_PATH, then the runtime **must** not set the `XR_BATTERY_STATE_DISPLAY_STATE_VALID_BIT_EXT` bit in `stateFlags`.

`XR_BATTERY_STATE_DISPLAY_STATE_VALID_BIT_EXT` indicates whether the `batteryLevel` contains valid data. The application **must** not read the `batteryLevel` if the `XR_BATTERY_STATE_DISPLAY_STATE_VALID_BIT_EXT` bit is not set in `stateFlags`.

Valid Usage (Implicit)

- The `XR_EXT_interaction_profile_battery_state_display` extension **must** be enabled prior to using XrBatteryStateDisplayEXT
- `type` **must** be `XR_TYPE_BATTERY_STATE_DISPLAY_EXT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `stateFlags` **must** be a valid combination of XrBatteryStateDisplayStateFlagBitsEXT values
- `stateFlags` **must** not be `0`

The XrBatteryStateDisplayEXT::`stateFlags` member is of the following type, and contains a bitwise-OR of zero or more of the bits defined in XrBatteryStateDisplayStateFlagBitsEXT.

```
// Provided by XR_EXT_interaction_profile_battery_state_display
typedef XrFlags64 XrBatteryStateDisplayStateFlagsEXT;
```

```
// Provided by XR_EXT_interaction_profile_battery_state_display
// Flag bits for XrBatteryStateDisplayStateFlagsEXT
static const XrBatteryStateDisplayStateFlagsEXT
XR_BATTERY_STATE_DISPLAY_STATE_VALID_BIT_EXT = 0x00000001;
static const XrBatteryStateDisplayStateFlagsEXT
XR_BATTERY_STATE_DISPLAY_STATE_CHARGING_BIT_EXT = 0x00000002;
static const XrBatteryStateDisplayStateFlagsEXT
XR_BATTERY_STATE_DISPLAY_STATE_PLUGGED_IN_BIT_EXT = 0x00000004;
static const XrBatteryStateDisplayStateFlagsEXT
XR_BATTERY_STATE_DISPLAY_STATE_NO_BATTERY_BIT_EXT = 0x00000008;
```

The flag bits have the following meanings:

Flag Descriptions

- `XR_BATTERY_STATE_DISPLAY_STATE_VALID_BIT_EXT` — Indicates validity of XrBatteryStateDisplayEXT::`batteryLevel`
- `XR_BATTERY_STATE_DISPLAY_STATE_CHARGING_BIT_EXT` — Indicates that the device associated with the interaction profile is charging.
- `XR_BATTERY_STATE_DISPLAY_STATE_PLUGGED_IN_BIT_EXT` — Indicates that the device associated with the interaction profile is plugged in to a power source.
- `XR_BATTERY_STATE_DISPLAY_STATE_NO_BATTERY_BIT_EXT` — Indicates that the device associated with the interaction profile does not have a battery power source.

12.40.3. Example

```
// previously initialized
extern XrSession session;
extern XrPath topLevelUserPath;

XrBatteryStateDisplayEXT batteryStateDisplay {
  .type = XR_TYPE_BATTERY_STATE_DISPLAY_EXT,
};
XrInteractionProfileState ipState {
  .type = XR_TYPE_INTERACTION_PROFILE_STATE,
  .next = &batteryStateDisplay,
};
CHK_XR(xrGetCurrentInteractionProfile(session, topLevelUserPath, &ipState));
if ((batteryStateDisplay.stateFlags & XR_BATTERY_STATE_DISPLAY_STATE_VALID_BIT_EXT) != 0)
{
  // use value of batteryStateDisplay.batteryLevel
}
```

12.40.4. New Structures

- Extending XrInteractionProfileState:
  - XrBatteryStateDisplayEXT

12.40.5. New Enums

- XrBatteryStateDisplayStateFlagBitsEXT

12.40.6. New Bitmasks

- XrBatteryStateDisplayStateFlagsEXT

12.40.7. New Enum Constants

- XR_EXT_INTERACTION_PROFILE_BATTERY_STATE_DISPLAY_EXTENSION_NAME
- XR_EXT_interaction_profile_battery_state_display_SPEC_VERSION
- Extending XrStructureType:
  - XR_TYPE_BATTERY_STATE_DISPLAY_EXT

12.40.8. Issues

- Can the application query the battery level displayed for a head-mounted device with this extension?
  - **Resolved. Not directly**. This extension only supports returning battery level displayed for top-level */user* paths with current interaction profiles. If the application suggests bindings for an

interaction profile for the head-mounted device (*/user/head*), and if the runtime makes that interaction profile active, then the runtime can return battery level for that top-level */user* path.

12.40.9. Version History

- Revision 1, 2025-10-21 (John Kearney)
  - Initial draft.

12.41. XR_EXT_interaction_render_model

Name String

`XR_EXT_interaction_render_model`

Extension Type

Instance extension

Registered Extension Number

302

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

`XR_EXT_render_model`

Contributors

Darryl Gough, Microsoft
Yin Li, Microsoft
Bryce Hutchings, Microsoft
Rylie Pavlik, Collabora
Joe Ludwig, Valve
Nathan Nuber, Valve
Dan Willmott, Valve
Jakob Bornecrantz, Collabora
Leonard Tsai, Meta Platforms
Paulo Gomes, Samsung Electronics
Lachlan Ford, Google
Wenlin Mao, Meta Platforms
Bastiaan Olij, Godot Engine

12.41.1. Overview

This extension allows an application to render realistic models representing the device or devices used by the user to interact. It is a generalized version of functionality that has been known elsewhere as "controller models", made generic by enumerating interaction-related render models without filtering them, and allowing association with a subaction path as a second lookup step.



Note

This extension is used for showing *the exact devices in use*. It returns models that may not have been available at the time of application creation, and thus not included in the suggested bindings. However, models returned from this extension must conform to the requirements in this spec, and thus are usable in a generic fashion. If you need to customize the models or otherwise *not* show the model as provided, please choose assets of your own to show based on the output of xrGetCurrentInteractionProfile rather than using this extension.

- **Do not** attempt to choose a model shipped with your software based on model ID or UUID from this extension.
- **Do not** attempt to detect the current hardware based on render model ID or UUID.
- The output must be considered fully dynamic, conforming only to the interface explicitly described in the specification. Failing to do so **will** result in compatibility problems for your application. The example code in this extension demonstrates intended generic application behavior and should be followed closely.

12.41.2. Getting Models

The design intent of this extension is to allow enumerating models *early* and keep enumerating them as long as their future use is possible. This is so that applications have time to load models, transcode textures, and otherwise prepare for rendering early in the session, and so that applications do not discard the results of that processing if it will be needed again. This large scope is only for enumerating the models in the first place, however: when those models are intended to be shown is more narrowly scoped and tightly specified, since it is less likely to be associated with a high computational startup cost.

The base `XR_EXT_render_model` extension delegates several design choices to dependent extensions, as described in Choices Delegated to Related Extensions. For models associated with this `XR_EXT_interaction_render_model` extension, the `XR_EXT_render_model` extension is specialized in the following ways, addressing those delegated choices and other important distinctions:

glTF extension behavior

For any render model ID retrieved from this extension, the runtime **must** support a glTF model without any required glTF extensions. Thus, the runtime **must** not return `XR_ERROR_RENDER_MODEL_GLTF_EXTENSION_REQUIRED_EXT` from xrCreateRenderModelEXT for any render model ID retrieved from this extension.

Alpha blending

Due to the difficulty and potential performance impact of implementing alpha blending correctly for multiple overlapping objects, applications are unlikely to be able to correctly render a model using alpha blending everywhere an interaction render model may appear. As such, the runtime **should** not set `alphaMode` to `BLEND` for any material in a render model associated with this extension. Materials with `alphaMode` set to `MASK` do not pose the same challenges of implementation and so are suitable for use if needed.

Animation

For any asset associated with this extension, the simple node-pose-visibility mechanism defined by `XR_EXT_render_model` in Animate Parts of a Render Model is used for animation.

External references

For any render model associated with this extension, the runtime **must** provide a glTF asset without any references to external buffers and textures outside of the GLB container. That is, all binary data **must** be embedded in the GLB binary chunk or as a Base64 `data:` URI.

Scenes

For any render model associated with this extension, the runtime **must** provide a glTF asset that contains 1 or more scene and defines the `scene` property to identify which scene to render.

Complexity and Optimization

The runtime **should** provide a glTF model optimized for real-time rendering use, with the expectation that the application **may** render all interaction render models every frame. Describing such optimization is beyond the scope of this specification.

Space location

Render models are located by a render model space, which does **not** correspond directly to any named pose.

> **Warning**
>
> The runtime **may** provide a lower quality glTF asset on some platforms when certain glTF extensions are not supported. Please consult platform specific documentation for best practices.

The xrEnumerateInteractionRenderModelIdsEXT function is defined as:

```
// Provided by XR_EXT_interaction_render_model
XrResult xrEnumerateInteractionRenderModelIdsEXT(
    XrSession                                       session,
    const XrInteractionRenderModelIdsEnumerateInfoEXT* getInfo,
    uint32_t                                        renderModelIdCapacityInput,
    uint32_t*                                       renderModelIdCountOutput,
    XrRenderModelIdEXT*                             renderModelIds);
```

Parameter Descriptions

- `session` is an XrSession in which the render model will be valid.
- `getInfo` exists for extensibility purposes, it is `NULL` or a pointer to a valid XrInteractionRenderModelIdsEnumerateInfoEXT structure.
- `renderModelIdCapacityInput` is the capacity of the `renderModelIds` array, or 0 to indicate a request to retrieve the required capacity.
- `renderModelIdCountOutput` is a pointer to the count of `renderModelIds` written, or a pointer to the required capacity in the case that `renderModelIdCapacityInput` is insufficient.
- `renderModelIds` is an array of `XrRenderModelIdEXT` that will be populated with IDs for devices that are associated with actions.
- See the Buffer Size Parameters section for a detailed description of retrieving the required `renderModelIds` size.

This function returns render model IDs associated with any device associated with actions, in any action set attached with `session` by xrAttachSessionActionSets. There is no specific meaning for array position. A runtime **may** return values in any order, although the enumerated array **must** remain constant between calls to xrSyncActions. An application **should** not assume any meaning based on array order. Note that a runtime **may** shuffle the order of IDs returned each time that the list changes, to aid application developers in avoiding accidental dependence on enumeration order.

An application **must** not assume any given size of this array based on suggested bindings: compatibility and user preference **may** result in more models being associated with actions than described in the suggested bindings. The runtime **may** return more models than the number of top level user paths in the suggested bindings due to user configuration and compatibility rebinding. The runtime **should** continue to return model IDs corresponding to any devices that has recently become inactive or disconnected, if they are reasonably expected to be used again soon, to minimize the need for applications to re-enumerate models and load assets. Similarly, the runtime **may** return model IDs for devices expected to be used, even if they are not yet connected or active.

The runtime **must** return render model IDs reflecting the actual hardware used, which **must** be independent of the currently active interaction profile. Accordingly, as long as the same actions within an XrInstance have suggested bindings, changing suggested bindings by adding or removing suggested

bindings for an interaction profile **must** not change the underlying assets. Furthermore, provided that identical actions within an XrInstance are associated with suggested bindings for a specified list of glTF extensions, the runtime **must** return an identical collection of render model asset UUIDs.

The application **can** monitor for the XrEventDataInteractionRenderModelsChangedEXT event to get notified when interaction render models need to be re-enumerated.

Changes to the collection of models enumerated (for example, due to device change) **must** only occur during a call to xrSyncActions. If the collection of models changes, the XrEventDataInteractionRenderModelsChangedEXT event **must** be queued during that call to xrSyncActions to signal the need for re-enumeration. This implies that a runtime **must** enumerate **no** models prior to the first call to xrSyncActions in a session.

Note that the UUIDs associated with the enumerated render model IDs for a given system and list of glTF extensions **may** change between instances due to runtime changes. Additionally, as with all atom types like `XrRenderModelIdEXT`, the enumerated render model ID values associated with a logical device **may** change between sessions as render model ID atoms inherently only have meaning within the single XrSession they are enumerated from.

If an `XrRenderModelIdEXT` was enumerated during a call to xrEnumerateInteractionRenderModelIdsEXT during the current session, but the set of interaction render models has now changed and that `XrRenderModelIdEXT` would not enumerated by a call to xrEnumerateInteractionRenderModelIdsEXT after that change, a call to xrCreateRenderModelEXT with that `XrRenderModelIdEXT` **must** return `XR_ERROR_RENDER_MODEL_ID_INVALID_EXT`. (Note that a change in the set of interaction render models only occurs during calls to xrSyncActions, and queues an XrEventDataInteractionRenderModelsChangedEXT event if it occurs.) That is, if an ID was previously enumerated with this function during the current session, but is no longer enumerated due to a change in interaction render models during an xrSyncActions call, it is no longer valid to create a XrRenderModelEXT from that `XrRenderModelIdEXT`.

Existing XrRenderModelEXT handles already created from an ID that is no longer enumerated remain valid, but "inactive" and effectively useless.

- Locating an associated render model space **must** report untracked/unlocatable, and therefore the model is not to be rendered.
- Calls to xrGetRenderModelStateEXT **may** stop providing updated data, as they are assumed to not be rendered and thus the model state is irrelevant.
- The runtime **may** return `XR_ERROR_RENDER_MODEL_ASSET_UNAVAILABLE_EXT` from xrCreateRenderModelAssetEXT if called with the cache UUID of that render model, if no other active render model uses the same asset UUID.

Runtimes **must** not enumerate a render model ID that they previously enumerated, then no longer enumerated. That is, if a render model ID is made inactive, it will never again become active. If the associated device returns, it will use a new render model ID.

A render model XrRenderModelEXT created from an `XrRenderModelIdEXT` enumerated by this function

must not be visible/locatable when located by xrCreateRenderModelSpaceEXT if the session state is not XR_SESSION_STATE_FOCUSED, to ensure render models are only being rendered once per frame.

If the session is not running, the runtime **must** return XR_ERROR_SESSION_NOT_RUNNING.

A render model XrRenderModelEXT created from an XrRenderModelIdEXT enumerated by this function **must** be locatable and visible if the corresponding device is locatable and there exists some action in any action set with which the render model is associated. This avoids having interaction render models disappear during corner cases of application interaction, e.g. when a "menu" button present on only one controller is the only active input. If an application wishes to only show models for which there are active actions, use the output of xrEnumerateRenderModelSubactionPathsEXT which enumerates subaction paths per model for the active action sets only.

Valid Usage (Implicit)

- The XR_EXT_interaction_render_model extension **must** be enabled prior to calling xrEnumerateInteractionRenderModelIdsEXT
- session **must** be a valid XrSession handle
- getInfo **must** be a pointer to a valid XrInteractionRenderModelIdsEnumerateInfoEXT structure
- renderModelIdCountOutput **must** be a pointer to a uint32_t value
- If renderModelIdCapacityInput is not 0, renderModelIds **must** be a pointer to an array of renderModelIdCapacityInput XrRenderModelIdEXT values

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_SESSION_NOT_RUNNING
- XR_ERROR_VALIDATION_FAILURE

The XrInteractionRenderModelIdsEnumerateInfoEXT structure is defined as:

```
// Provided by XR_EXT_interaction_render_model
typedef struct XrInteractionRenderModelIdsEnumerateInfoEXT {
    XrStructureType    type;
    const void*        next;
} XrInteractionRenderModelIdsEnumerateInfoEXT;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain.

XrInteractionRenderModelIdsEnumerateInfoEXT is an input structure for the xrEnumerateInteractionRenderModelIdsEXT function. XrInteractionRenderModelIdsEnumerateInfoEXT exists for future extensibility.

Valid Usage (Implicit)

- The XR_EXT_interaction_render_model extension **must** be enabled prior to using XrInteractionRenderModelIdsEnumerateInfoEXT
- type **must** be XR_TYPE_INTERACTION_RENDER_MODEL_IDS_ENUMERATE_INFO_EXT
- next **must** be NULL or a valid pointer to the next structure in a structure chain

The XrEventDataInteractionRenderModelsChangedEXT structure is an event defined as:

```
// Provided by XR_EXT_interaction_render_model
typedef struct XrEventDataInteractionRenderModelsChangedEXT {
    XrStructureType    type;
    const void*        next;
} XrEventDataInteractionRenderModelsChangedEXT;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain.

Receiving this event from xrPollEvent indicates that that the app **should** enumerate interaction render models (or re-enumerate them) using xrEnumerateInteractionRenderModelIdsEXT and the two-call idiom, because the list of IDs enumerated by it has changed. This event **must** only be queued by a call to xrSyncActions. For clarity, if an application has enabled this extension, this event **must** be emitted during the first xrSyncActions call if xrEnumerateInteractionRenderModelIdsEXT will enumerate any models, because it enumerates no models prior to the first xrSyncActions call.

Valid Usage (Implicit)

- The XR_EXT_interaction_render_model extension **must** be enabled prior to using XrEventDataInteractionRenderModelsChangedEXT
- type **must** be XR_TYPE_EVENT_DATA_INTERACTION_RENDER_MODELS_CHANGED_EXT
- next **must** be NULL or a valid pointer to the next structure in a structure chain

12.41.3. Associating Models with Active Action Set Subaction Paths

An application might wish to know which models are associated with a subaction path as used in suggested bindings, for example to adjust the shading to highlight a controller to use in user instructions. This operation is structured as enumerating the subaction paths for each render model to

encourage application logic that treats this data fully generally and handles common and less common configurations uniformly.

The xrEnumerateRenderModelSubactionPathsEXT function is defined as:

```
// Provided by XR_EXT_interaction_render_model
XrResult xrEnumerateRenderModelSubactionPathsEXT(
    XrRenderModelEXT                                renderModel,
    const XrInteractionRenderModelSubactionPathInfoEXT* info,
    uint32_t                                        pathCapacityInput,
    uint32_t*                                       pathCountOutput,
    XrPath*                                         paths);
```

Parameter Description

- renderModel is a render model handle created from an XrRenderModelIdEXT value retrieved from this extension.
- info exists for extensibility purposes, it is NULL or a pointer to a valid XrInteractionRenderModelSubactionPathInfoEXT structure.
- pathCapacityInput is the capacity of the paths array, or 0 to indicate a request to retrieve the required capacity.
- pathCountOutput is a pointer to the count of paths written, or a pointer to the required capacity in the case that pathCapacityInput is insufficient.
- paths is a pointer to an array of XrPath atoms, but **can** be NULL if pathCapacityInput is 0.
- See the Buffer Size Parameters section for a detailed description of retrieving the required paths size.

xrEnumerateRenderModelSubactionPathsEXT allows the application to associate an interaction-related render model with the associated subaction paths according to the exposed current interaction profile and active action sets.

If renderModel is valid but was not created from a render model ID from a call to xrEnumerateInteractionRenderModelIdsEXT earlier in the current session, the runtime **must** return XR_ERROR_NOT_INTERACTION_RENDER_MODEL_EXT.

The array enumerated by this function for a given render model **must** not change except during calls to xrSyncActions.

A given subaction path **must** be reported for a model if and only if both of the following are true:

- That path appears in the corresponding XrActionCreateInfo::subactionPaths for some action or actions associated with it in the active action sets.

- That path is used as a top-level user path for some suggested binding of at least one such action in the current interaction profile.

This paragraph describes implications and clarifications of the preceding requirement. If a given path is used as a top-level user path for a suggested binding to an action with no subaction paths specified, or without that specific subaction path specified, it is not sufficient to require enumerating that path. The runtime **must** only enumerate subaction paths that are included in the reported current interaction profile and mentioned in the corresponding suggested bindings, even if one of the models is logically better described by a path not used by the application. For example, a treadmill-like interaction device with its input mapped to actions suggested for left and right hands enumerates the paths */user/hand/left* and */user/hand/right* even though */user/treadmill* is defined in the specification. This also implies that a runtime **must** return no subaction paths prior to the first call to xrSyncActions in a session, or when the most recent call to xrSyncActions did not specify any active action sets. Additionally, the runtime **must** return no subaction paths when a given render model provides input only for actions that do not have a list of subaction paths specified in XrActionCreateInfo ::`subactionPaths`.

This function is intended for identifying models currently associated with any actions in an active action set, as well as identifying the subaction paths associated with the bound input. To identify which top-level */user* path is most closely associated with the overall pose of any given interaction render model, see xrGetRenderModelPoseTopLevelUserPathEXT. The description of that function contains a further discussion of the differences with this function.

Important: The order of values returned from this function is **not** meaningful, and the entire array **should** be iterated and treated uniformly by the application. An application **should** always be prepared for this function to return a list of any length, up to the total number of subaction paths used in suggested bindings. Most functionality in OpenXR is defined to operate as if the hardware corresponding to the current interaction profile were in use according to the suggested bindings. However, this function, and this extension in general, allows the application to access aspects of the user's actual input configuration, to provide accurate and realistic feedback to the user. Special care is required to ensure that application code using this function is maximally general.

Valid Usage (Implicit)

- The `XR_EXT_interaction_render_model` extension **must** be enabled prior to calling xrEnumerateRenderModelSubactionPathsEXT
- `renderModel` **must** be a valid XrRenderModelEXT handle
- If `info` is not `NULL`, `info` **must** be a pointer to a valid XrInteractionRenderModelSubactionPathInfoEXT structure
- `pathCountOutput` **must** be a pointer to a `uint32_t` value
- If `pathCapacityInput` is not `0`, `paths` **must** be a pointer to an array of `pathCapacityInput` `XrPath` values

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_NOT_INTERACTION_RENDER_MODEL_EXT
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The XrInteractionRenderModelSubactionPathInfoEXT structure is defined as:

```
// Provided by XR_EXT_interaction_render_model
typedef struct XrInteractionRenderModelSubactionPathInfoEXT {
    XrStructureType    type;
    const void*        next;
} XrInteractionRenderModelSubactionPathInfoEXT;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain.

XrInteractionRenderModelSubactionPathInfoEXT exists for future extensibility.

Valid Usage (Implicit)

- The `XR_EXT_interaction_render_model` extension **must** be enabled prior to using XrInteractionRenderModelSubactionPathInfoEXT
- `type` **must** be `XR_TYPE_INTERACTION_RENDER_MODEL_SUBACTION_PATH_INFO_EXT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

12.41.4. Query Pose-Related Top Level */user* Path for Model

Some applications need to know the top-level */user* path most closely associated with the overall pose of an interaction render model. This allows an application to adjust positioning of the render model where a render model retains its relative position to related poses and/or hand models.

An example use case is when rendering the controller, hand model, and elements related to poses for the player's right or left hand, while the player has moved their hand through a virtual wall. An application **may** choose to not render these elements at their tracked location but instead prevent movement through this obstruction. The application will want to adjust the position of these elements in equal measure.

The xrGetRenderModelPoseTopLevelUserPathEXT function is defined as:

```
// Provided by XR_EXT_interaction_render_model
XrResult xrGetRenderModelPoseTopLevelUserPathEXT(
    XrRenderModelEXT                                renderModel,
    const XrInteractionRenderModelTopLevelUserPathGetInfoEXT* info,
    XrPath*                                         topLevelUserPath);
```

Parameter Descriptions

- `renderModel` is an XrRenderModelEXT retrieved using the `XrRenderModelIdEXT` obtained through this extension.
- `info` is parameters affecting the results of this function, including an array of top-level */user* paths to limit results to.
- `topLevelUserPath` is a pointer to an `XrPath` to which the top level */user* path will be written.

This function returns the top level */user* path most closely associated with the pose of a given render model, if any, and if that path is present in the list passed in `info`.

A runtime **must** return: * the top level */user* path from the list in `info` that is **most closely** associated with the model pose as a physical reality (e.g. a device currently held in the user's left hand returns */user/hand/left*), if one exists. Note that this requirement does provide fallback behavior. That is, if a

model pose is related to more than one top level */user* path, the runtime returns the path from `info` with the closest association, even if it is less closely related than some other path **not** included in `info`. * XR_NULL_PATH if no such path can be determined (e.g. the corresponding device is currently not held by or attached to the user, or no path associated with the model pose was provided in `info`).

Note that unlike xrGetCurrentInteractionProfile, more than one model **may** report being most closely associated with a given top level */user* path. For example, a runtime **may** represent a single controller as two render models, or a user **may** have both a handheld device and a wrist-mounted tracker.

Changes to the top level */user* path state of each render model **must** only occur during a call to xrSyncActions.

If `renderModel` is valid but was not retrieved from a call to xrEnumerateInteractionRenderModelIdsEXT earlier in the current session, the runtime **must** return `XR_ERROR_NOT_INTERACTION_RENDER_MODEL_EXT`.

This function differs from xrEnumerateRenderModelSubactionPathsEXT by emphasizing poses and being broadly distinct from actions. xrGetRenderModelPoseTopLevelUserPathEXT focuses solely on poses related to a top level */user* path and returning only most applicable result. Contrast with xrEnumerateRenderModelSubactionPathsEXT, which reports all top level */user* paths **being used as subaction paths** that are associated with **actions in an active action set**. That function is meant more for e.g. highlighting models providing input, especially non-pose input, associated with a subaction path. For example, the right hand might have a pie menu related action set active, and an application could show the devices that can interact with that menu in a highlighted way, while dimming the other models.

Important: An application **should** always be prepared for this function to return any top-level */user* path in their list or XR_NULL_PATH for any of the interaction render models. Many systems will not report XR_NULL_PATH for any models, provided that both */user/hand/left* and */user/hand/right* are included on the list in `info`, but application code **must** be prepared to handle this and that code path **should** be tested manually. Most functionality in OpenXR is defined to operate as if the hardware corresponding to the current interaction profile were in use according to the suggested bindings. However, this function, and this extension in general, allows the application to access aspects of the user's actual input configuration, to provide accurate and realistic feedback to the user. Special care is required to ensure that application code using this function is maximally general.

Valid Usage (Implicit)

- The `XR_EXT_interaction_render_model` extension **must** be enabled prior to calling xrGetRenderModelPoseTopLevelUserPathEXT
- `renderModel` **must** be a valid XrRenderModelEXT handle
- `info` **must** be a pointer to a valid XrInteractionRenderModelTopLevelUserPathGetInfoEXT structure
- `topLevelUserPath` **must** be a pointer to an `XrPath` value

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_NOT_INTERACTION_RENDER_MODEL_EXT
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_PATH_INVALID
- XR_ERROR_PATH_UNSUPPORTED
- XR_ERROR_RENDER_MODEL_ID_INVALID_EXT
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The XrInteractionRenderModelTopLevelUserPathGetInfoEXT structure is defined as:

```
// Provided by XR_EXT_interaction_render_model
typedef struct XrInteractionRenderModelTopLevelUserPathGetInfoEXT {
    XrStructureType    type;
    const void*        next;
    uint32_t           topLevelUserPathCount;
    const XrPath*      topLevelUserPaths;
} XrInteractionRenderModelTopLevelUserPathGetInfoEXT;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain.
- topLevelUserPathCount is the number of elements in topLevelUserPaths.
- topLevelUserPaths is an array of unique XrPath values that correspond to valid top-level */user* paths.

If any elements in topLevelUserPaths are duplicated, the runtime **must** return XR_ERROR_VALIDATION_FAILURE from xrGetRenderModelPoseTopLevelUserPathEXT.

If any elements in topLevelUserPaths are not valid top level */user* paths, the runtime **must** return XR_ERROR_PATH_INVALID from xrGetRenderModelPoseTopLevelUserPathEXT.

Valid Usage (Implicit)

- The XR_EXT_interaction_render_model extension **must** be enabled prior to using XrInteractionRenderModelTopLevelUserPathGetInfoEXT
- type **must** be XR_TYPE_INTERACTION_RENDER_MODEL_TOP_LEVEL_USER_PATH_GET_INFO_EXT
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- topLevelUserPaths **must** be a pointer to an array of topLevelUserPathCount valid XrPath values
- The topLevelUserPathCount parameter **must** be greater than 0

12.41.5. Example

```
// previously initialized
extern XrInstance instance;
extern XrSession session;
extern XrSpace baseSpace;

// Get the function pointers for the extension's functions.

PFN_xrEnumerateInteractionRenderModelIdsEXT
    pfnEnumerateInteractionRenderModelIdsEXT;
CHK_XR(xrGetInstanceProcAddr(instance,
                             "xrEnumerateInteractionRenderModelIdsEXT",
                             reinterpret_cast<PFN_xrVoidFunction *>(
                                 &pfnEnumerateInteractionRenderModelIdsEXT)));

// And the XR_EXT_render_model functions
PFN_xrCreateRenderModelEXT pfnCreateRenderModelEXT;
CHK_XR(xrGetInstanceProcAddr(
    instance, "xrCreateRenderModelEXT",
    reinterpret_cast<PFN_xrVoidFunction *>(&pfnCreateRenderModelEXT)));

PFN_xrDestroyRenderModelEXT pfnDestroyRenderModelEXT;
CHK_XR(xrGetInstanceProcAddr(
    instance, "xrDestroyRenderModelEXT",
    reinterpret_cast<PFN_xrVoidFunction *>(&pfnDestroyRenderModelEXT)));

PFN_xrGetRenderModelPropertiesEXT pfnGetRenderModelPropertiesEXT;
```

```
CHK_XR(xrGetInstanceProcAddr(
    instance, "xrGetRenderModelPropertiesEXT",
    reinterpret_cast<PFN_xrVoidFunction *>(&pfnGetRenderModelPropertiesEXT)));

PFN_xrCreateRenderModelSpaceEXT pfnCreateRenderModelSpaceEXT;
CHK_XR(xrGetInstanceProcAddr(
    instance, "xrCreateRenderModelSpaceEXT",
    reinterpret_cast<PFN_xrVoidFunction *>(&pfnCreateRenderModelSpaceEXT)));

PFN_xrCreateRenderModelAssetEXT pfnCreateRenderModelAssetEXT;
CHK_XR(xrGetInstanceProcAddr(
    instance, "xrCreateRenderModelAssetEXT",
    reinterpret_cast<PFN_xrVoidFunction *>(&pfnCreateRenderModelAssetEXT)));

PFN_xrDestroyRenderModelAssetEXT pfnDestroyRenderModelAssetEXT;
CHK_XR(xrGetInstanceProcAddr(
    instance, "xrDestroyRenderModelAssetEXT",
    reinterpret_cast<PFN_xrVoidFunction *>(&pfnDestroyRenderModelAssetEXT)));

PFN_xrGetRenderModelAssetDataEXT pfnGetRenderModelAssetDataEXT;
CHK_XR(xrGetInstanceProcAddr(
    instance, "xrGetRenderModelAssetDataEXT",
    reinterpret_cast<PFN_xrVoidFunction *>(&pfnGetRenderModelAssetDataEXT)));

PFN_xrGetRenderModelAssetPropertiesEXT pfnGetRenderModelAssetPropertiesEXT;
CHK_XR(xrGetInstanceProcAddr(instance, "xrGetRenderModelAssetPropertiesEXT",
                             reinterpret_cast<PFN_xrVoidFunction *>(
                                 &pfnGetRenderModelAssetPropertiesEXT)));

PFN_xrGetRenderModelStateEXT pfnGetRenderModelStateEXT;
CHK_XR(xrGetInstanceProcAddr(
    instance, "xrGetRenderModelStateEXT",
    reinterpret_cast<PFN_xrVoidFunction *>(&pfnGetRenderModelStateEXT)));

XrPath rightHandPath;
CHK_XR(xrStringToPath(instance, "/user/hand/right", &rightHandPath));

// Enumerate the render model IDs
XrInteractionRenderModelIdsEnumerateInfoEXT renderModelGetInfo {
    .type = XR_TYPE_INTERACTION_RENDER_MODEL_IDS_ENUMERATE_INFO_EXT,
};

uint32_t numModels{0};
CHK_XR(pfnEnumerateInteractionRenderModelIdsEXT(session, NULL, 0, &numModels,
                                                NULL));
std::vector<XrRenderModelIdEXT> interactionModelIds{XR_NULL_PATH, numModels};
CHK_XR(pfnEnumerateInteractionRenderModelIdsEXT(session, NULL, numModels,
                                                &numModels,
```

```
                                                interactionModelIds.data()));

// Create render model handles
// The names of glTF extensions that the application is capable of supporting.
// The returned glTF model may have any or all of these extensions listed in
// the "extensionsRequired" array.
// Pass only the extensions that your app/engine are capable of supporting.
std::vector<const char *> appSupportedGltfExtensions{"KHR_texture_basisu",
                                                     "KHR_materials_specular"};

std::vector<XrRenderModelEXT> interactionModels;
for (XrRenderModelIdEXT id : interactionModelIds) {
  XrRenderModelEXT renderModel {XR_NULL_HANDLE};
  XrRenderModelCreateInfoEXT renderModelCreateInfo {
    .type = XR_TYPE_RENDER_MODEL_CREATE_INFO_EXT,
    .renderModelId = id,
    .gltfExtensionCount = static_cast<uint32_t>(appSupportedGltfExtensions.size()),
    .gltfExtensions = appSupportedGltfExtensions.data(),
  };
  CHK_XR(pfnCreateRenderModelEXT(session, &renderModelCreateInfo, &renderModel));
  interactionModels.push_back(renderModel);
}

std::vector<XrSpace> modelSpaces;
std::vector<XrRenderModelPropertiesEXT> modelProperties;
for (XrRenderModelEXT renderModel : interactionModels) {
  // Create a space for locating the render model.
  XrRenderModelSpaceCreateInfoEXT spaceCreateInfo {
    .type = XR_TYPE_RENDER_MODEL_SPACE_CREATE_INFO_EXT,
    .renderModel = renderModel,
  };
  XrSpace modelSpace {XR_NULL_HANDLE};
  CHK_XR(pfnCreateRenderModelSpaceEXT(session, &spaceCreateInfo, &modelSpace));
  modelSpaces.push_back(modelSpace);

  // Get the model properties: UUID and number of animatable nodes
  XrRenderModelPropertiesGetInfoEXT propertiesGetInfo {
    .type = XR_TYPE_RENDER_MODEL_PROPERTIES_GET_INFO_EXT,
  };
  XrRenderModelPropertiesEXT properties {
    .type = XR_TYPE_RENDER_MODEL_PROPERTIES_EXT,
  };
  CHK_XR(pfnGetRenderModelPropertiesEXT(renderModel, &propertiesGetInfo,
                                        &properties));

  modelProperties.push_back(properties);
  {
    // Create the asset handle to request the data.
```

```
    XrRenderModelAssetCreateInfoEXT assetCreateInfo {
      .type = XR_TYPE_RENDER_MODEL_ASSET_CREATE_INFO_EXT,
      .cacheId = properties.cacheId,
    };
    XrRenderModelAssetEXT asset {XR_NULL_HANDLE};
    CHK_XR(pfnCreateRenderModelAssetEXT(session, &assetCreateInfo, &asset));

    // Copy the binary glTF (GLB) asset data using two-call idiom.
    XrRenderModelAssetDataGetInfoEXT assetGetInfo {
      .type = XR_TYPE_RENDER_MODEL_ASSET_DATA_GET_INFO_EXT,
    };
    XrRenderModelAssetDataEXT assetData {
      .type = XR_TYPE_RENDER_MODEL_ASSET_DATA_EXT,
    };
    CHK_XR(pfnGetRenderModelAssetDataEXT(asset, &assetGetInfo, &assetData));
    std::vector<uint8_t> glbData(assetData.bufferCountOutput);
    assetData.bufferCapacityInput = (uint32_t)glbData.size();
    assetData.buffer = glbData.data();
    CHK_XR(pfnGetRenderModelAssetDataEXT(asset, &assetGetInfo, &assetData));

    // Parsing the binary glTF data is outside the scope of this extension,
    // but do it here.

  // Get the unique names of the animatable nodes
    XrRenderModelAssetPropertiesGetInfoEXT assetPropertiesGetInfo {
      .type = XR_TYPE_RENDER_MODEL_ASSET_PROPERTIES_GET_INFO_EXT,
    };
    std::vector<XrRenderModelAssetNodePropertiesEXT> nodeProperties(
        properties.animatableNodeCount);
    XrRenderModelAssetPropertiesEXT assetProperties {
      .type = XR_TYPE_RENDER_MODEL_ASSET_PROPERTIES_EXT,
      .nodePropertyCount = (uint32_t)nodeProperties.size(),
      .nodeProperties = nodeProperties.data(),
    };
    CHK_XR(pfnGetRenderModelAssetPropertiesEXT(asset, &assetPropertiesGetInfo,
                                               &assetProperties));

    // Once the glTF data has been handled, we no longer need the
    // XrRenderModelAssetEXT handle.
    CHK_XR(pfnDestroyRenderModelAssetEXT(asset));

    // Save the list of nodes for rendering. The order of the array matters.
    // The application will store some sort of "reference" to a node for
    // each element, using the node name (in nodeProperties) to find it here.
    // This code is not shown because it will depend on how your
    // application represents glTF assets, so add your own here.
  }
}
```

```
// Each frame the application's work for each model includes
// reading the state of the animatable nodes
// and then adjusting the pose or visibility of the node.

// Initialized from xrWaitFrame output
XrTime predictedDisplayTime;

for (size_t modelIndex = 0; modelIndex < interactionModels.size();
     ++modelIndex) {
  XrRenderModelEXT renderModel = interactionModels[modelIndex];
  const XrRenderModelPropertiesEXT& properties = modelProperties[modelIndex];
  XrSpace modelSpace = modelSpaces[modelIndex];

  // Use xrLocateSpace to locate the model's space
  XrSpaceLocation modelLocation {
    .type = XR_TYPE_SPACE_LOCATION,
  };
  CHK_XR(xrLocateSpace(modelSpace, baseSpace, predictedDisplayTime, &modelLocation));

  bool orientationTracked = (modelLocation.locationFlags &
      XR_SPACE_LOCATION_ORIENTATION_TRACKED_BIT) != 0;
  bool positionTracked = (modelLocation.locationFlags &
      XR_SPACE_LOCATION_POSITION_TRACKED_BIT) != 0;

  if (!orientationTracked || !positionTracked) {
    // Only render if the model space is tracked,
    // and if the session state is appropriate, if applicable.
    // (e.g. interaction models are only to be rendered when FOCUSED)

    // Flag this model as not-rendered-this-frame in your app-specific way here.

    continue;
  }

  XrRenderModelStateGetInfoEXT stateGetInfo {
    .type = XR_TYPE_RENDER_MODEL_STATE_GET_INFO_EXT,
    .displayTime = predictedDisplayTime,
  };

  // In practice, you do not want to re-allocate this array of
  // node state every frame, but it is clearer for illustration.
  // We know the number of elements from the model properties,
  // and we used the names from the asset handle to find and retain
  // our app-specific references to those nodes in the model.
  std::vector<XrRenderModelNodeStateEXT> nodeStates(
      properties.animatableNodeCount);
```

```
  XrRenderModelStateEXT state {
    .type = XR_TYPE_RENDER_MODEL_STATE_EXT,
    .nodeStateCount = static_cast<uint32_t>(nodeStates.size()),
    .nodeStates = nodeStates.data(),
  };
  // xrGetRenderModelStateEXT does not use the two-call idiom. The size is
  // determined by xrGetRenderModelAssetPropertiesEXT.
  CHK_XR(pfnGetRenderModelStateEXT(renderModel, &stateGetInfo, &state));

  for (size_t i = 0; i < nodeStates.size(); ++i) {
    // Use nodeStates[i].isVisible and nodeStates[i].nodePose to update the
    // node's visibility or pose.
    // nodeStates[i] refers to the node identified by name in nodeProperties[i]
  }

  // Your app now has the overall transform and all node transforms/status here.
}
```

As a demonstration of xrEnumerateRenderModelSubactionPathsEXT, the following additional code assumes that the application would like to modify rendering (e.g. highlight) for devices that provide input to a given subaction path, such as to emphasize which device is controlling a currently-active teleport targeting.

```
// previously initialized
extern XrInstance instance;

// as populated in the preceding sample
std::vector<XrRenderModelEXT> interactionModels;

// Get the function pointers for the extension's functions.
PFN_xrEnumerateRenderModelSubactionPathsEXT
    pfnEnumerateRenderModelSubactionPathsEXT;
CHK_XR(xrGetInstanceProcAddr(instance,
                             "xrEnumerateRenderModelSubactionPathsEXT",
                             reinterpret_cast<PFN_xrVoidFunction *>(
                                 &pfnEnumerateRenderModelSubactionPathsEXT)));

// During each frame when an application wishes to treat render models
// associated with some subaction path differently, it performs the following.

// Previously initialized
XrPath subactionPathToHighlight;

// Reused for each model because results are temporary
std::vector<XrPath> paths;
for (size_t modelIndex = 0; modelIndex < interactionModels.size();
```

```
     ++modelIndex) {
  XrRenderModelEXT renderModel = interactionModels[modelIndex];

  // Two-call idiom for subaction paths
  uint32_t count;
  CHK_XR(pfnEnumerateRenderModelSubactionPathsEXT(renderModel, nullptr,
                                                  0, &count, nullptr));
  paths.resize(count, XR_NULL_PATH);
  CHK_XR(pfnEnumerateRenderModelSubactionPathsEXT(renderModel, nullptr,
                                                  (uint32_t)paths.size(),
                                                  &count, paths.data()));

  // Determine if our desired subaction path is in the collection.
  bool foundHighlightPath = (paths.end() !=
                             std::find(paths.begin(),
                                       paths.end(),
                                       subactionPathToHighlight));
  if (foundHighlightPath) {
    // Highlight this model: it is providing input for
    // actions on subactionPathToHighlight
  } else {
    // Render normally: no input from this model is
    // associated with subactionPathToHighlight
  }
}
```

12.41.6. New Commands

- xrEnumerateInteractionRenderModelIdsEXT
- xrEnumerateRenderModelSubactionPathsEXT
- xrGetRenderModelPoseTopLevelUserPathEXT

12.41.7. New Structures

- XrEventDataInteractionRenderModelsChangedEXT
- XrInteractionRenderModelIdsEnumerateInfoEXT
- XrInteractionRenderModelSubactionPathInfoEXT
- XrInteractionRenderModelTopLevelUserPathGetInfoEXT

12.41.8. New Enum Constants

- XR_EXT_INTERACTION_RENDER_MODEL_EXTENSION_NAME
- XR_EXT_interaction_render_model_SPEC_VERSION

- Extending XrResult:
  - XR_ERROR_NOT_INTERACTION_RENDER_MODEL_EXT
- Extending XrStructureType:
  - XR_TYPE_EVENT_DATA_INTERACTION_RENDER_MODELS_CHANGED_EXT
  - XR_TYPE_INTERACTION_RENDER_MODEL_IDS_ENUMERATE_INFO_EXT
  - XR_TYPE_INTERACTION_RENDER_MODEL_SUBACTION_PATH_INFO_EXT
  - XR_TYPE_INTERACTION_RENDER_MODEL_TOP_LEVEL_USER_PATH_GET_INFO_EXT

12.41.9. Issues

- Should we enumerate models per subaction path? per action? or overall?
  - We enumerate all models to normalize looping over an array of models of arbitrary length, to avoid fragility when more than one device is providing input for a single subaction path due to rebinding. (Application authors are likely to assume one model per subaction path unless the API is structured to avoid that assumption.)
- Given enumeration of models first, what action-related data is safe to expose to the application without introducing untested code paths used only in case of rebinding?
  - Enumerating subaction paths for a model is not a problem: the runtime only returns subaction paths submitted by the app (so no untested code paths), and the mistaken assumption that only one subaction path is returned is less dangerous than assuming a number of models: the association with subaction paths is likely primarily for highlighting, etc. Incorrect processing of this data by the application produces a less-optimal experience, but does not result in any crash or incompatibility.
- Can the application associate individual actions with models or nodes in them?
  - This is out of scope for this extension and will be provided in a follow-up. It requires more design work to achieve the working group goals.
- Should the main function only enumerate models associated with currently-bound and active actions?
  - No, this will change for each active action set change, requiring frequent re-enumeration of models. If an application wants to display only models associated with a bound and active action, it **can** use the results of xrEnumerateRenderModelSubactionPathsEXT to identify them, and no event is needed as the application controls calling xrSyncActions. The current design instead enumerates models associated with the union of all actions attached to the session.
- Does the asset corresponding to a render model ID change when the user switches devices, or should it trigger an event prompting the runtime to enumerate a new render model ID for the new device?
  - An event triggers fresh enumeration retrieving a new render model ID, to keep one render model ID closely associated with a physical device rather than with a role or the inputs driven by it. A different type of controller is a new model ID and not just an updated asset for an

existing one. Additionally, the UUID and asset for a given render model ID and list of extensions in a session is now defined to be immutable.

- Does xrEnumerateRenderModelSubactionPathsEXT enumerate subaction paths in any specific order?
  - No, the order is explicitly defined to have no meaning. An application that uses xrEnumerateRenderModelSubactionPathsEXT **should** assume there may be multiple values in this list, even though there **may** be only one in some cases, and treat the list returned from xrEnumerateRenderModelSubactionPathsEXT as a set. An application **should** process all values in that list equally: e.g. if looking to highlight "right hand" devices, apply a highlight shader to all render models that contain */user/hand/right* in their list from xrEnumerateRenderModelSubactionPathsEXT no matter where it appears in the output.
- Can the active assets for hardware change between sessions or only instances?
  - Assets for devices must remain fixed within a given instance. This is primarily unneeded implementation freedom that is restricted so that the conformance test suite can enforce the requirement that suggested bindings for additional interaction profiles, as long as they do not change the collection of bound actions, do not change the assets. It is very important for the purpose and usability of this extension that it returns assets related to the real hardware in use, which means it **must** be unaffected by the interaction profile system. We cannot test automatically whether the hardware looks like the model, but if we require that the underlying assets are fixed across sessions within an instance, we can check that the UUID does not change based on the suggested bindings for a given session.
- What device render models are enumerated? Options include A: only devices for actions in the active action sets, B: devices associated with any action in any action set, C: any devices the user may interact with even if they do not have an associated action
  - Option B is selected. Option A (only devices for the currently active action sets) may mean that the set of enumerated devices changes frequently, if not all action sets contain actions being supplied by every device. This could lead applications to have a more robust lifecycle for interaction render models, and well tested code paths for setup and teardown, but it also could result in a lot of extra overhead from this setup and teardown. Option B would enumerate a larger group of models, though not all of them would necessarily be applicable at all times. (The runtime could report not-applicable ones as not locatable when no actions are active.) Option C would show devices that are not necessarily intended for interaction (things like cameras and base stations), which was determined to be out of scope for this extension, though may be added by additional extensions with chained structure modifying this functionality.
- Should interaction render models remain locatable even when they do not have any active actions associated with them?
  - Yes. If applications want to further filter which models to display, this is possible by enumerating subaction paths in the active action set for each model, and omitting those that enumerate no subaction paths.

12.41.10. Version History

- Revision 1, 2021-12-21 (Yin Li)
  - Initial extension description

12.42. XR_EXT_loader_init_properties

Name String

`XR_EXT_loader_init_properties`

Extension Type

Instance extension

Registered Extension Number

839

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Last Modified Date

2024-11-20

IP Status

No known IP claims.

Contributors

Bryce Hutchings, Microsoft Corporation
Jakob Bornecrantz, NVIDIA
Rafal Karp, NVIDIA

12.42.1. Overview

This extension allows the application to safely pass custom name-value pair properties, which act as environment variable overrides, directly to the OpenXR loader in XrLoaderInitInfoPropertiesEXT by calling xrInitializeLoaderKHR. This extension introduces the concept of a internal database of name-value pairs that the loader stores internally. The loader queries this database first and when looking up environment variables as outlined in the OpenXR Loader - Design and Operation document, and if finding a name-value pair uses this instead of querying the platform. The application uses the

XrLoaderInitInfoPropertiesEXT structure passed into the xrInitializeLoaderKHR function to populate these pairs.

12.42.2. Loader Initialization Structure

The XrLoaderInitInfoPropertiesEXT structure is defined as:

```
// Provided by XR_EXT_loader_init_properties
typedef struct XrLoaderInitInfoPropertiesEXT {
    XrStructureType                         type;
    const void*                             next;
    uint32_t                                propertyValueCount;
    const XrLoaderInitPropertyValueEXT*     propertyValues;
} XrLoaderInitInfoPropertiesEXT;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `propertyValueCount` is a `uint32_t` specifying the count of elements in the `propertyValues` array.
- `propertyValues` is an array of XrLoaderInitPropertyValueEXT structures containing the name-value pairs for loader properties.

This structure is either provided directly as the parameter to xrInitializeLoaderKHR or on that parameter's `next` chain, depending on whether your platform requires another structure to be passed to xrInitializeLoaderKHR. It contains an array of string property name-value pairs, intended primarily to be used by the loader.

The ordering of properties does not matter because the loader rejects any duplicate entries.

The loader **must** return `XR_ERROR_VALIDATION_FAILURE` if there are duplicate entries of property names in the given array. The loader **must** obey the properties in the internal database and use them to override environment variables. The loader **must** start the internal database of property pairs as empty. The loader **must** must clear the internal database of property pairs when xrInitializeLoaderKHR is called with XrLoaderInitInfoPropertiesEXT passed in an allowed way. (That is, the loader obeys only the properties passed in the most recent successful xrInitializeLoaderKHR call that included a XrLoaderInitInfoPropertiesEXT structure.)

It is unspecified what happens to the properties if the loader is unloaded then reloaded through dynamic library loading, due to differences in platforms beyond the control of this extension. Unlike platform functions to query environment variables, whose case sensitivity varies from platform to platform, the loader **must** always treat lookups of properties as case-sensitive in the internal database.

This extension describes a behavior of the OpenXR loader and not any behavior of a loaded runtime. As such any properties supplied by the application **must** not be exposed as environment variables and **must** not affect any other component of the system. Future loader implementations and loader specifications **may** add mechanisms for the runtime or OpenXR layers to query the properties but as the writing of this spec no such functionality exists.

Valid Usage (Implicit)

- The `XR_EXT_loader_init_properties` extension **must** be enabled prior to using XrLoaderInitInfoPropertiesEXT
- `type` **must** be `XR_TYPE_LOADER_INIT_INFO_PROPERTIES_EXT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `propertyValues` **must** be a pointer to an array of `propertyValueCount` valid XrLoaderInitPropertyValueEXT structures
- The `propertyValueCount` parameter **must** be greater than `0`

The XrLoaderInitPropertyValueEXT structure is defined as:

```
// Provided by XR_EXT_loader_init_properties
typedef struct XrLoaderInitPropertyValueEXT {
    const char*    name;
    const char*    value;
} XrLoaderInitPropertyValueEXT;
```

Member Descriptions

- `name` is the name of the given property.
- `value` is the value of the given property.

This structure contains a single property name-value pair passed to loader initialization. The XrLoaderInitInfoPropertiesEXT structure is used to pass an array of these structures to the OpenXR loader, and the runtime upon loading, via xrInitializeLoaderKHR.

Rules for the value pair strings:

- The loader **must** accept any valid UTF-8 string, including those that only contain whitespace, for both `name` and `value` except otherwise covered by a rule in this list. Whitespace-only strings are accepted by this extension because whitespace only paths are valid on some platforms.
- The loader **must** return `XR_ERROR_VALIDATION_FAILURE` if `name` or `value` is NULL.
- The loader **must** return `XR_ERROR_VALIDATION_FAILURE` if `name` is a zero length string.

- The loader **must** treat a zero length `value` as if the environmental variable was unset in a platform specific way.
- The loader **may** impose implementation or platform specific limitation on string lengths and/or total string memory usage and **must** return `XR_ERROR_LIMIT_REACHED` when hit.
- The loader **may** impose length limits on `name` and/or `value` strings and **must** return `XR_ERROR_LIMIT_REACHED` when hit. Because they might be imposed by the platform, the limits are unspecified.
- The loader **must** not reference the memory pointed to by the given strings beyond returning from this call, this implies that the loader will copy the strings into an internal database.

Valid Usage (Implicit)

- The `XR_EXT_loader_init_properties` extension **must** be enabled prior to using XrLoaderInitPropertyValueEXT
- `name` **must** be a null-terminated UTF-8 string
- `value` **must** be a null-terminated UTF-8 string

12.42.3. New Structures

- XrLoaderInitPropertyValueEXT
- Extending XrLoaderInitInfoBaseHeaderKHR:
  - XrLoaderInitInfoPropertiesEXT

12.42.4. New Enum Constants

- `XR_EXT_LOADER_INIT_PROPERTIES_EXTENSION_NAME`
- `XR_EXT_loader_init_properties_SPEC_VERSION`
- Extending XrStructureType:
  - `XR_TYPE_LOADER_INIT_INFO_PROPERTIES_EXT`

12.42.5. Issues

- Does this extension mandate when the loader reads the environment variables?
  - Resolved
  - Answer: No. To the extent that this extension is able, it does not mandate when a specific variable is read by the loader. But to be usable this extension needs to impose some rules on the OpenXR loader, but tries to defer to the base or OpenXR loader specification.

12.42.6. Version History

- Revision 1, 2024-11-20 (Bryce Hutchings & Rafal Karp)
    - Initial draft

12.43. XR_EXT_performance_settings

Name String

`XR_EXT_performance_settings`

Extension Type

Instance extension

Registered Extension Number

16

Revision

4

Ratification Status

Ratified

Extension and Version Dependencies

OpenXR 1.0

Last Modified Date

2021-04-14

IP Status

No known IP claims.

Contributors

Armelle Laine, Qualcomm Technologies Inc, on behalf of Qualcomm Innovation Center, Inc
Rylie Pavlik, Collabora

12.43.1. Overview

This extension defines an API for the application to give performance hints to the runtime and for the runtime to send performance related notifications back to the application. This allows both sides to dial in a suitable compromise between needed CPU and GPU performance, thermal sustainability and a consistent good user experience throughout the session.

The goal is to render frames consistently, in time, under varying system load without consuming more energy than necessary.

In summary, the APIs allow:

- setting performance level hints
- receiving performance related notifications

12.43.2. Setting Performance Levels Hints

Performance level hint definition

The XR performance level hints for a given hardware system are expressed as a level XrPerfSettingsLevelEXT for each of the XR-critical processing domains XrPerfSettingsDomainEXT (currently defined is a CPU and a GPU domain):

```
// Provided by XR_EXT_performance_settings, XR_EXT_thermal_query
typedef enum XrPerfSettingsDomainEXT {
    XR_PERF_SETTINGS_DOMAIN_CPU_EXT = 1,
    XR_PERF_SETTINGS_DOMAIN_GPU_EXT = 2,
    XR_PERF_SETTINGS_DOMAIN_MAX_ENUM_EXT = 0x7FFFFFFF
} XrPerfSettingsDomainEXT;
```

```
// Provided by XR_EXT_performance_settings
typedef enum XrPerfSettingsLevelEXT {
    XR_PERF_SETTINGS_LEVEL_POWER_SAVINGS_EXT = 0,
    XR_PERF_SETTINGS_LEVEL_SUSTAINED_LOW_EXT = 25,
    XR_PERF_SETTINGS_LEVEL_SUSTAINED_HIGH_EXT = 50,
    XR_PERF_SETTINGS_LEVEL_BOOST_EXT = 75,
    XR_PERF_SETTINGS_LEVEL_MAX_ENUM_EXT = 0x7FFFFFFF
} XrPerfSettingsLevelEXT;
```

This extension defines platform-independent level hints:

- XR_PERF_SETTINGS_LEVEL_POWER_SAVINGS_EXT is used by the application to indicate that it enters a non-XR section (head-locked / static screen), during which power savings are to be prioritized. Consistent XR compositing, consistent frame rendering, and low latency are not needed.
- XR_PERF_SETTINGS_LEVEL_SUSTAINED_LOW_EXT is used by the application to indicate that it enters a low and stable complexity section, during which reducing power is more important than occasional late rendering frames. With such a hint, the XR Runtime still strives for consistent XR compositing (no tearing) within a thermally sustainable range(*), but is allowed to take measures to reduce power, such as increasing latencies or reducing headroom.
- XR_PERF_SETTINGS_LEVEL_SUSTAINED_HIGH_EXT is used by the application to indicate that it enters a high or dynamic complexity section, during which the XR Runtime strives for consistent XR compositing and frame rendering within a thermally sustainable range(*).

- XR_PERF_SETTINGS_LEVEL_BOOST_EXT is used to indicate that the application enters a section with very high complexity, during which the XR Runtime is allowed to step up beyond the thermally sustainable range. As not thermally sustainable, this level is meant to be used for short-term durations (< 30 seconds).

(*) If the application chooses one of the two sustainable levels (XR_PERF_SETTINGS_LEVEL_SUSTAINED_LOW_EXT or XR_PERF_SETTINGS_LEVEL_SUSTAINED_HIGH_EXT), the device may still run into thermal limits under non-nominal circumstances (high room temperature, additional background loads, extended device operation) and therefore the application should also in the sustainable modes be prepared to react to performance notifications (in particular XR_PERF_SETTINGS_NOTIF_LEVEL_WARNING_EXT and XR_PERF_SETTINGS_NOTIF_LEVEL_IMPAIRED_EXT in the thermal sub-domain, see Notification level definition).

The XR Runtime shall select XR_PERF_SETTINGS_LEVEL_SUSTAINED_HIGH_EXT as the default hint if the application does not provide any. The function to call for setting performance level hints is xrPerfSettingsSetPerformanceLevelEXT.

```
// Provided by XR_EXT_performance_settings
XrResult xrPerfSettingsSetPerformanceLevelEXT(
    XrSession                                   session,
    XrPerfSettingsDomainEXT                     domain,
    XrPerfSettingsLevelEXT                      level);
```

Example of using the short-term boost level hint

For a limited amount of time, both the Mobile and PC systems can provide a higher level of performance than is thermally sustainable. It is desirable to make this extra computational power available for short complex scenes, then go back to a sustainable lower level. This section describes means for the application developer to apply settings directing the runtime to boost performance for a short-term duration.

The application developer must pay attention to keep these boost periods very short and carefully monitor the side effects, which may vary a lot between different hardware systems.

Sample code for temporarily boosting the performance

```
extern XrInstance instance; ①
extern XrSession session;

// Get function pointer for xrPerfSettingsSetPerformanceLevelEXT
PFN_xrPerfSettingsSetPerformanceLevelEXT pfnPerfSettingsSetPerformanceLevelEXT;
CHK_XR(xrGetInstanceProcAddr(instance, "xrPerfSettingsSetPerformanceLevelEXT",
                             (PFN_xrVoidFunction*)(
                             &pfnPerfSettingsSetPerformanceLevelEXT)));

// before entering the high complexity section
pfnPerfSettingsSetPerformanceLevelEXT(session, XR_PERF_SETTINGS_DOMAIN_CPU_EXT,
XR_PERF_SETTINGS_LEVEL_BOOST_EXT); ②
pfnPerfSettingsSetPerformanceLevelEXT(session, XR_PERF_SETTINGS_DOMAIN_GPU_EXT,
XR_PERF_SETTINGS_LEVEL_BOOST_EXT);

// entering the high complexity section
// ... running
// end of the high complexity section

pfnPerfSettingsSetPerformanceLevelEXT(session, XR_PERF_SETTINGS_DOMAIN_CPU_EXT,
XR_PERF_SETTINGS_LEVEL_SUSTAINED_HIGH_EXT); ③
pfnPerfSettingsSetPerformanceLevelEXT(session, XR_PERF_SETTINGS_DOMAIN_GPU_EXT,
XR_PERF_SETTINGS_LEVEL_SUSTAINED_HIGH_EXT);
```

① we assume that `instance` and `session` are initialized and their handles are available

② setting performance level to `XR_PERF_SETTINGS_LEVEL_BOOST_EXT` on both CPU and GPU domains

③ going back to the sustainable `XR_PERF_SETTINGS_LEVEL_SUSTAINED_HIGH_EXT`

Example of using the sustained low level hint for the CPU domain

power reduction sample code

```
extern XrInstance instance; ①
extern XrSession session;

// Get function pointer for xrPerfSettingsSetPerformanceLevelEXT
PFN_xrPerfSettingsSetPerformanceLevelEXT pfnPerfSettingsSetPerformanceLevelEXT;
CHK_XR(xrGetInstanceProcAddr(instance, "xrPerfSettingsSetPerformanceLevelEXT",
                             (PFN_xrVoidFunction*)(
                             &pfnPerfSettingsSetPerformanceLevelEXT)));

// before entering a low CPU complexity section
pfnPerfSettingsSetPerformanceLevelEXT(session, XR_PERF_SETTINGS_DOMAIN_CPU_EXT,
XR_PERF_SETTINGS_LEVEL_SUSTAINED_LOW_EXT);
pfnPerfSettingsSetPerformanceLevelEXT(session, XR_PERF_SETTINGS_DOMAIN_GPU_EXT,
XR_PERF_SETTINGS_LEVEL_SUSTAINED_HIGH_EXT); ②

// entering the low complexity section
// ... running
// end of the low complexity section

pfnPerfSettingsSetPerformanceLevelEXT(session, XR_PERF_SETTINGS_DOMAIN_CPU_EXT,
XR_PERF_SETTINGS_LEVEL_SUSTAINED_HIGH_EXT); ③
```

① we assume that `instance` and `session` are initialized and their handles are available

② the developer may choose to only reduce CPU domain and keep the GPU domain at `XR_PERF_SETTINGS_LEVEL_SUSTAINED_HIGH_EXT`

③ going back to the sustainable `XR_PERF_SETTINGS_LEVEL_SUSTAINED_HIGH_EXT` for CPU

12.43.3. Receiving Performance Related Notifications

The XR runtime shall provide performance related notifications to the application in the following situations:

- the compositing performance within the runtime has reached a new level, either improved or degraded from the previous one (`subDomain` is set to `XR_PERF_SETTINGS_SUB_DOMAIN_COMPOSITING_EXT`)
- the application rendering performance has reached a new level, either improved or degraded from the previous one (`subDomain` is set to `XR_PERF_SETTINGS_SUB_DOMAIN_RENDERING_EXT`)
- the temperature of the device has reached a new level, either improved or degraded from the previous one (`subDomain` is set to `XR_PERF_SETTINGS_SUB_DOMAIN_THERMAL_EXT`).

When degradation is observed, the application **should** take measures reducing its workload, helping the compositing or rendering `subDomain` to meet their deadlines, or the thermal `subDomain` to avoid or stop throttling. When improvement is observed, the application can potentially rollback some of its mitigations.

```
// Provided by XR_EXT_performance_settings
typedef struct XrEventDataPerfSettingsEXT {
    XrStructureType                         type;
    const void*                             next;
    XrPerfSettingsDomainEXT                 domain;
    XrPerfSettingsSubDomainEXT              subDomain;
    XrPerfSettingsNotificationLevelEXT      fromLevel;
    XrPerfSettingsNotificationLevelEXT      toLevel;
} XrEventDataPerfSettingsEXT;
```

```
// Provided by XR_EXT_performance_settings
typedef enum XrPerfSettingsSubDomainEXT {
    XR_PERF_SETTINGS_SUB_DOMAIN_COMPOSITING_EXT = 1,
    XR_PERF_SETTINGS_SUB_DOMAIN_RENDERING_EXT = 2,
    XR_PERF_SETTINGS_SUB_DOMAIN_THERMAL_EXT = 3,
    XR_PERF_SETTINGS_SUB_DOMAIN_MAX_ENUM_EXT = 0x7FFFFFFF
} XrPerfSettingsSubDomainEXT;
```

Compositing Sub-Domain

One of the major functions the runtime shall provide is the timely compositing of the submitted layers in the background. The runtime has to share the CPU and GPU system resources for this operation with the application. Since this is extremely time sensitive - the head room is only a few milliseconds - the runtime may have to ask the application via notifications to cooperate and relinquish some usage of the indicated resource (CPU or GPU domain). Performance issues in this area that the runtime notices are notified to the application with the `subDomain` set to `XR_PERF_SETTINGS_SUB_DOMAIN_COMPOSITING_EXT`.

Rendering Sub-Domain

The application submits rendered layers to the runtime for compositing. Performance issues in this area that the runtime notices (i.e. missing submission deadlines) are notified to the application with the `subDomain` set to `XR_PERF_SETTINGS_SUB_DOMAIN_RENDERING_EXT`.

Thermal Sub-Domain

XR applications run at a high-performance level during long periods of time, across a game or an entire movie session. As form factors shrink, especially on mobile solutions, the risk of reaching die thermal runaway or reaching the limits on skin and battery temperatures increases. When thermal limits are reached, the device mitigates the heat generation leading to severe performance reductions, which greatly affects user experience (dropped frames, high latency).

Better than dropping frames when it is too late, pro-active measures from the application should be encouraged.

The performance notification with the subDomain set to XR_PERF_SETTINGS_SUB_DOMAIN_THERMAL_EXT provides an early warning allowing the application to take mitigation actions.

Notification level definition

The levels are defined as follows:

```
// Provided by XR_EXT_performance_settings, XR_EXT_thermal_query
typedef enum XrPerfSettingsNotificationLevelEXT {
    XR_PERF_SETTINGS_NOTIF_LEVEL_NORMAL_EXT = 0,
    XR_PERF_SETTINGS_NOTIF_LEVEL_WARNING_EXT = 25,
    XR_PERF_SETTINGS_NOTIF_LEVEL_IMPAIRED_EXT = 75,
    XR_PERF_SETTINGS_NOTIFICATION_LEVEL_MAX_ENUM_EXT = 0x7FFFFFFF
} XrPerfSettingsNotificationLevelEXT;
```

- XR_PERF_SETTINGS_NOTIF_LEVEL_NORMAL_EXT notifies that the sub-domain has reached a level where no further actions other than currently applied are necessary.
- XR_PERF_SETTINGS_NOTIF_LEVEL_WARNING_EXT notifies that the sub-domain has reached an early warning level where the application should start proactive mitigation actions with the goal to return to the XR_PERF_SETTINGS_NOTIF_LEVEL_NORMAL_EXT level.
- XR_PERF_SETTINGS_NOTIF_LEVEL_IMPAIRED_EXT notifies that the sub-domain has reached a critical level with significant performance degradation. The application should take drastic mitigation action.

The above definitions summarize the broad interpretation of the notification levels, however sub-domain specific definitions of each level and their transitions are specified below:

- XR_PERF_SETTINGS_NOTIF_LEVEL_NORMAL_EXT
  - For the compositing sub-domain, XR_PERF_SETTINGS_NOTIF_LEVEL_NORMAL_EXT indicates that the composition headroom is consistently being met with sufficient margin.
    Getting into XR_PERF_SETTINGS_NOTIF_LEVEL_NORMAL_EXT from XR_PERF_SETTINGS_NOTIF_LEVEL_WARNING_EXT indicates that the composition headroom was consistently **met with sufficient margin during a sufficient time period**.
  - For the rendering sub-domain, XR_PERF_SETTINGS_NOTIF_LEVEL_NORMAL_EXT indicates that frames are being submitted in time to be used by the compositor.
    Getting into XR_PERF_SETTINGS_NOTIF_LEVEL_NORMAL_EXT from XR_PERF_SETTINGS_NOTIF_LEVEL_WARNING_EXT indicates that **during a sufficient time period**, **none** of the due layers was **too late** to be picked up by the compositor.
  - For the thermal sub-domain, XR_PERF_SETTINGS_NOTIF_LEVEL_NORMAL_EXT indicates that the current load should be sustainable in the near future.
    Getting into XR_PERF_SETTINGS_NOTIF_LEVEL_NORMAL_EXT from XR_PERF_SETTINGS_NOTIF_LEVEL_WARNING_EXT indicates that the runtime does not presuppose any further temperature mitigation action on the application side, other than the current ones.
- XR_PERF_SETTINGS_NOTIF_LEVEL_WARNING_EXT

- For the compositing sub-domain, `XR_PERF_SETTINGS_NOTIF_LEVEL_WARNING_EXT` indicates that the compositing headroom of the current frame was met but the margin is considered insufficient by the runtime, and the application **should** reduce its workload in the notified domain to solve this problem.
  Getting into `XR_PERF_SETTINGS_NOTIF_LEVEL_WARNING_EXT` from `XR_PERF_SETTINGS_NOTIF_LEVEL_IMPAIRED_EXT` indicates that the compositing deadline was **not missed during a sufficient time period**.
- For the rendering sub-domain, `XR_PERF_SETTINGS_NOTIF_LEVEL_WARNING_EXT` indicates that at least one layer is regularly late to be picked up by the compositor, resulting in a degraded user experience, and that the application should take action to consistently provide frames in a more timely manner.
  Getting into `XR_PERF_SETTINGS_NOTIF_LEVEL_WARNING_EXT` from `XR_PERF_SETTINGS_NOTIF_LEVEL_IMPAIRED_EXT` indicates that **the runtime has stopped any of its own independent actions** which are tied to the `XR_PERF_SETTINGS_NOTIF_LEVEL_IMPAIRED_EXT` level.
- For the thermal sub-domain, the `XR_PERF_SETTINGS_NOTIF_LEVEL_WARNING_EXT` indicates that the runtime expects the device to overheat under the current load, and that the application should take mitigating action in order to prevent thermal throttling.
  Getting into `XR_PERF_SETTINGS_NOTIF_LEVEL_WARNING_EXT` from `XR_PERF_SETTINGS_NOTIF_LEVEL_IMPAIRED_EXT` indicates that **the underlying system thermal throttling has stopped**.

- `XR_PERF_SETTINGS_NOTIF_LEVEL_IMPAIRED_EXT`
  - For the compositing sub-domain, `XR_PERF_SETTINGS_NOTIF_LEVEL_IMPAIRED_EXT` indicates that composition can no longer be maintained under the current workload. The runtime may take independent action that will interfere with the application (e.g. limiting the framerate, ignoring submitted layers, or shutting down the application) in order to correct this problem.
  - For the rendering sub-domain, `XR_PERF_SETTINGS_NOTIF_LEVEL_IMPAIRED_EXT` indicates that at least one layer is **too often** late to be picked up by the compositor, and consequently the runtime may take independent action that will interfere with the application (e.g. informing the user that the application is not responding, displaying a tracking environment in order to maintain user orientation).
  - For the thermal sub-domain, `XR_PERF_SETTINGS_NOTIF_LEVEL_IMPAIRED_EXT` indicates that the **underlying system is taking measures, such as thermal throttling** to reduce the temperature, impacting the XR experience..

Leaving `XR_PERF_SETTINGS_NOTIF_LEVEL_IMPAIRED_EXT` indicates that any mitigating actions by the runtime (e.g. down-clocking the device to stay within thermal limits) have ended.

Performance Settings API Reference

xrPerfSettingsSetPerformanceLevelEXT

```
// Provided by XR_EXT_performance_settings
XrResult xrPerfSettingsSetPerformanceLevelEXT(
    XrSession                                   session,
    XrPerfSettingsDomainEXT                     domain,
    XrPerfSettingsLevelEXT                      level);
```

Parameter Descriptions

- `session` is a valid XrSession handle.
- `domain`: the processing domain for which the level hint is applied
- `level`: the level hint to be applied

Valid Usage (Implicit)

- The `XR_EXT_performance_settings` extension **must** be enabled prior to calling xrPerfSettingsSetPerformanceLevelEXT
- `session` **must** be a valid XrSession handle
- `domain` **must** be a valid XrPerfSettingsDomainEXT value
- `level` **must** be a valid XrPerfSettingsLevelEXT value

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_VALIDATION_FAILURE`

Refer to Performance level hint definition for the definition of the level enumerations.

XrEventDataPerformanceSettingsEXT

The XrEventDataPerfSettingsEXT structure is defined as:

```
// Provided by XR_EXT_performance_settings
typedef struct XrEventDataPerfSettingsEXT {
    XrStructureType                         type;
    const void*                             next;
    XrPerfSettingsDomainEXT                 domain;
    XrPerfSettingsSubDomainEXT              subDomain;
    XrPerfSettingsNotificationLevelEXT      fromLevel;
    XrPerfSettingsNotificationLevelEXT      toLevel;
} XrEventDataPerfSettingsEXT;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `domain` : processing domain in which a threshold has been crossed
- `subDomain` : system area in which a threshold has been crossed
- `fromLevel` : enumerated notification level which has been exited
- `toLevel` : enumerated notification level which has been entered

Valid Usage (Implicit)

- The `XR_EXT_performance_settings` extension **must** be enabled prior to using XrEventDataPerfSettingsEXT
- `type` **must** be `XR_TYPE_EVENT_DATA_PERF_SETTINGS_EXT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

```
// Provided by XR_EXT_performance_settings, XR_EXT_thermal_query
typedef enum XrPerfSettingsDomainEXT {
    XR_PERF_SETTINGS_DOMAIN_CPU_EXT = 1,
    XR_PERF_SETTINGS_DOMAIN_GPU_EXT = 2,
    XR_PERF_SETTINGS_DOMAIN_MAX_ENUM_EXT = 0x7FFFFFFF
} XrPerfSettingsDomainEXT;
```

```
// Provided by XR_EXT_performance_settings
typedef enum XrPerfSettingsSubDomainEXT {
    XR_PERF_SETTINGS_SUB_DOMAIN_COMPOSITING_EXT = 1,
    XR_PERF_SETTINGS_SUB_DOMAIN_RENDERING_EXT = 2,
    XR_PERF_SETTINGS_SUB_DOMAIN_THERMAL_EXT = 3,
    XR_PERF_SETTINGS_SUB_DOMAIN_MAX_ENUM_EXT = 0x7FFFFFFF
} XrPerfSettingsSubDomainEXT;
```

```
// Provided by XR_EXT_performance_settings, XR_EXT_thermal_query
typedef enum XrPerfSettingsNotificationLevelEXT {
    XR_PERF_SETTINGS_NOTIF_LEVEL_NORMAL_EXT = 0,
    XR_PERF_SETTINGS_NOTIF_LEVEL_WARNING_EXT = 25,
    XR_PERF_SETTINGS_NOTIF_LEVEL_IMPAIRED_EXT = 75,
    XR_PERF_SETTINGS_NOTIFICATION_LEVEL_MAX_ENUM_EXT = 0x7FFFFFFF
} XrPerfSettingsNotificationLevelEXT;
```

Version History

- Revision 1, 2017-11-30 (Armelle Laine)
- Revision 2, 2021-04-13 (Rylie Pavlik)
  - Correctly show function pointer retrieval in sample code
  - Fix sample code callouts
- Revision 3, 2021-04-14 (Rylie Pavlik)
  - Fix missing error code
- Revision 4, 2022-10-26 (Rylie Pavlik)
  - Update XML markup to correct the generated valid usage

12.44. XR_EXT_render_model

Name String

`XR_EXT_render_model`

Extension Type

Instance extension

Registered Extension Number

301

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

OpenXR 1.1
or
`XR_EXT_uuid`

Contributors

Darryl Gough, Microsoft
Yin Li, Microsoft
Bryce Hutchings, Microsoft
Joe Ludwig, Valve
Nathan Nuber, Valve
Rylie Pavlik, Collabora
Wenlin Mao, Meta Platforms
Dan Willmott, Valve
Jakob Bornecrantz, Collabora
Leonard Tsai, Meta Platforms
Paulo Gomes, Samsung Electronics
Lachlan Ford, Google

12.44.1. Overview

This extension enables the application to retrieve a glTF 2.0 render model asset from a runtime and animate parts of the model. Other extensions depending on this one specify how to obtain a render model ID, and **may** specify further restrictions on glTF assets associated with IDs they produce.

Note

An OpenXR application typically uses a render model as follows:

1. Obtain an `XrRenderModelIdEXT`, valid for an XrSession, from the runtime using another extension.
2. Create an XrRenderModelEXT handle using the render model ID and the corresponding session, using xrCreateRenderModelEXT.
3. Retrieve the UUID of the model asset data from the render model handle using the xrGetRenderModelPropertiesEXT function.
4. Create an XrRenderModelAssetEXT handle using the UUID, to request that the runtime make the asset data available.
   - Reading and processing the glTF asset data can be a slow operation, possibly involving storage or network access, and the application should avoid doing the work in latency sensitive threads. The XrRenderModelPropertiesEXT ::`cacheId` is an XrUuidEXT type to identify the asset data, and the application can use this UUID as a key to cache the glTF, node names, and processed data within or across multiple XR sessions. This cache can be kept locally but is not be distributed with an application.



5. Use the two-call idiom function xrGetRenderModelAssetDataEXT to get the glTF asset, and get the names of animatable nodes with xrGetRenderModelAssetPropertiesEXT. Once this is complete, xrDestroyRenderModelAssetEXT can be called.
6. Process the glTF asset as your application requires, and populate an array of references to nodes in the glTF tree by finding the nodes with the names returned from xrGetRenderModelAssetPropertiesEXT.
7. Create an XrSpace to locate the model, using xrCreateRenderModelSpaceEXT.
8. During the render loop, repeatedly do the following:
   - Use the xrLocateSpace function on the render model space, to know where to render the origin of the glTF model and whether to render it at all. Do not render a given render model if position and orientation are not tracked.
   - Use xrGetRenderModelPropertiesEXT to retrieve the pose and visibility of each animatable node on the glTF asset, which were found in the asset in step 6. This results in animating the render model according to runtime-provided parameters, such as physical state like the tilt of a thumbstick on a controller.

12.44.2. Choices Delegated to Related Extensions

This extension is permissive in its design to accommodate a variety of use cases for runtime-provided, application-rendered glTF assets. Extensions that build on this one are encouraged to further specify render model properties for render models associated with them. Be aware that the required behavior

of functions in this extension depend on the extension from which a given XrRenderModelIdEXT was retrieved. Some aspects for other extensions to specify include:

glTF Extension Behavior

Whether a runtime **must** support providing an asset with no required glTF extensions (and thus not return XR_ERROR_RENDER_MODEL_GLTF_EXTENSION_REQUIRED_EXT from xrCreateRenderModelAssetEXT for its models), or whether the runtime **may** return XR_ERROR_RENDER_MODEL_GLTF_EXTENSION_REQUIRED_EXT if specific glTF extensions are not supported. (If possible, indicate which extensions **may** be considered mandatory.)

Alpha Blending

What values for alphaMode are permissible in materials used by a render model asset. Some use cases are highly interactive and thus **must** not use alpha mode of BLEND to avoid mandating order independent transparency processing between application content and render models.

Animation

How any animation is performed: whether the simple node-pose-visibility mechanism described in this extension is used for animation, and/or whether and how standard glTF animations are used.

External References

Whether external references for buffers and textures are permitted.

Scenes

Whether the asset **may** contain more than one scene without specifying a default scene, and if so, how to select the scene to render. Alternately, the number of scenes the asset **may** contain, and that the property value for default scene **must** be defined. (An extension is encouraged to require the presence of the scene property except in cases where the extension provides a way to explicitly compute which scene to use.)

Complexity and Optimization

What hard limits exist for models associated with an extension, if any; any guidelines for asset size, complexity, and feature usage; and what type of usage to optimize assets for.

12.44.3. Render Model ID Atom and Handle

```
// Provided by XR_EXT_render_model
XR_DEFINE_ATOM(XrRenderModelIdEXT)
```

The render model ID is used to create an XrRenderModelEXT handle. Like other atom types in OpenXR, the ID **should** not correspond to consuming noticeable resources in the runtime, it has no explicit lifetime of its own, and it has no persistence nor identity beyond the lifetime of the XrSession handle it is retrieved from. Once the XrRenderModelEXT handle is created from the ID, the runtime **may** start to consume resources to load and track the state of the render model.

The application **can** use a valid XrRenderModelIdEXT to create an XrRenderModelEXT handle.

The value XR_NULL_RENDER_MODEL_ID_EXT, equal to 0, is defined to be an invalid XrRenderModelIdEXT value.

The application **can** use a valid XrRenderModelIdEXT to create an XrRenderModelEXT handle.

This XR_EXT_render_model extension does not specify how to obtain a valid XrRenderModelIdEXT. The application **can** obtain a valid ID through other extensions that depend on this one.

Be aware that there is a potential pitfall when creating a dependent extension, if the set of render models it enumerates has any in common with the set of render models enumerated by another (existing) dependent extension. To avoid unexpected application behavior when the same XrRenderModelIdEXT is enumerated from two separate functions, it is recommended to do one of the following:

- **Extend the existing enumeration function** through extending an input structure chain, rather than creating a new enumeration function.
- **Forbid simultaneous use** of those two extensions in your new extension.

```
#define XR_NULL_RENDER_MODEL_ID_EXT       0
```

The ID XR_NULL_RENDER_MODEL_ID_EXT **cannot** be used to create an XrRenderModelEXT handle, and is considered by definition to be an invalid render model ID.

```
// Provided by XR_EXT_render_model
XR_DEFINE_HANDLE(XrRenderModelEXT)
```

The XrRenderModelEXT handle represents the resources to load and track the state of a render model, states of animatable parts, and a set of glTF extensions that the application is prepared to handle in a corresponding asset.

It does not directly represent the model's data, however. See XrRenderModelAssetEXT for the handle representing the data for a render model asset, including names of animatable nodes.

An application **can** create an XrRenderModelEXT handle using the xrCreateRenderModelEXT function.

```
// Provided by XR_EXT_render_model
XrResult xrCreateRenderModelEXT(
    XrSession                                   session,
    const XrRenderModelCreateInfoEXT*           createInfo,
    XrRenderModelEXT*                           renderModel);
```

Parameter Descriptions

- `session` is an XrSession in which the render model handle will be valid.
- `createInfo` is the XrRenderModelCreateInfoEXT used to specify the render model.
- `renderModel` is the returned XrRenderModelEXT handle.

If, when attempting to create the handle, the session does not support any render model of the given render model ID requiring only glTF extensions from the supplied glTF extension list (in XrRenderModelCreateInfoEXT::`gltfExtensions`), the runtime **must** return `XR_ERROR_RENDER_MODEL_GLTF_EXTENSION_REQUIRED_EXT`.

Valid Usage (Implicit)

- The `XR_EXT_render_model` extension **must** be enabled prior to calling xrCreateRenderModelEXT
- `session` **must** be a valid XrSession handle
- `createInfo` **must** be a pointer to a valid XrRenderModelCreateInfoEXT structure
- `renderModel` **must** be a pointer to an XrRenderModelEXT handle

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_LIMIT_REACHED`
- `XR_ERROR_OUT_OF_MEMORY`
- `XR_ERROR_RENDER_MODEL_ID_INVALID_EXT`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_VALIDATION_FAILURE`

The XrRenderModelCreateInfoEXT structure is defined as:

```
// Provided by XR_EXT_render_model
typedef struct XrRenderModelCreateInfoEXT {
    XrStructureType       type;
    const void*           next;
    XrRenderModelIdEXT    renderModelId;
    uint32_t              gltfExtensionCount;
    const char* const*    gltfExtensions;
} XrRenderModelCreateInfoEXT;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `renderModelId` is an `XrRenderModelIdEXT` to identify the render model to be created.
- `gltfExtensionCount` is the count of strings in the `gltfExtensions` array. It **must** be `0` if `gltfExtensions` is `NULL`.
- `gltfExtensions` is an **optional** array of strings that represents a list of glTF extensions the application supports.

The XrRenderModelCreateInfoEXT structure describes the information necessary to create an XrRenderModelEXT handle.

The input `renderModelId` value **must** be obtained from the same XrSession used in xrCreateRenderModelEXT. If the `renderModelId` value does not match one retrieved from the relevant XrSession, the runtime **must** return error `XR_ERROR_RENDER_MODEL_ID_INVALID_EXT`.

Note: There is a chance that a `renderModelId` value incorrectly retained from another session **may** have the same numerical value as one retrieved from the current XrSession. In such instances, the runtime is unable to distinguish between the two IDs. As a result, the runtime **may** mistakenly accept the ID and return a success code, even though it represents an invalid usage. Applications **should** be prepared to handle unexpected behaviors or outcomes stemming from this scenario.

The application **can** create multiple XrRenderModelEXT handles using the same ID. The runtime **must** return the same render model states and asset UUID to these handles if they also share the same list of extensions, since they are sharing the same underlying render model ID. If the list of extensions differs, the runtime **may** expose a different number of animatable nodes, different asset data and UUID, etc.

The runtime **must** return `XR_ERROR_RENDER_MODEL_GLTF_EXTENSION_REQUIRED_EXT` if the runtime is unable

to return a glTF asset that only requires extensions found in the application's list of supported glTF extensions.

Related extensions **may** require the application to support certain glTF extensions, in which case this error code indicates a failure to satisfy the requirement.

Alternately, related extensions **may** require the runtime to support providing base glTF assets without any required glTF extensions, in which case this error **must** not be returned by xrCreateRenderModelEXT in association with render model IDs retrieved from such extensions. See Delegated Choice: glTF Extension Behavior.

The order of `gltfExtensions` array represents the preferences from the application when multiple extensions are specified. The runtime **may** select or modify the retrieved glTF assets based on this array of extensions to optimize the glTF asset for this application.

Successful creation of this handle implies that the runtime is ready to report a fixed number and sequence of animatable node states for an asset satisfying the application's criteria, and that asset data, with node names, meeting the criteria **may** be available during this session. The asset data and node names **may** still be unavailable at the time the XrRenderModelEXT handle is returned.

Valid Usage (Implicit)

- The `XR_EXT_render_model` extension **must** be enabled prior to using XrRenderModelCreateInfoEXT
- `type` **must** be `XR_TYPE_RENDER_MODEL_CREATE_INFO_EXT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- If `gltfExtensionCount` is not `0`, `gltfExtensions` **must** be a pointer to an array of `gltfExtensionCount` null-terminated UTF-8 strings

The xrDestroyRenderModelEXT function is defined as:

```
// Provided by XR_EXT_render_model
XrResult xrDestroyRenderModelEXT(
    XrRenderModelEXT                            renderModel);
```

Parameter Descriptions

- `renderModel` is an XrRenderModelEXT previously created by xrCreateRenderModelEXT.

xrDestroyRenderModelEXT function releases the XrRenderModelEXT handle and the underlying resources when finished with the render model tracking and animation.

Although any associated XrSpace handles created by xrCreateRenderModelSpaceEXT are not destroyed upon calling xrDestroyRenderModelEXT because the space is a child of the session handle, any render model spaces created from a now-destroyed render model handle **must** no longer return any XrSpaceLocationFlagBits or XrSpaceVelocityFlagBits set in XrSpaceLocation::`locationFlags` or XrSpaceVelocity::`velocityFlags`, respectively. That is, a space created from a render model handle that is now destroyed becomes no longer locatable.

Valid Usage (Implicit)

- The XR_EXT_render_model extension **must** be enabled prior to calling xrDestroyRenderModelEXT
- `renderModel` **must** be a valid XrRenderModelEXT handle

Thread Safety

- Access to `renderModel`, and any child handles, **must** be externally synchronized

Return Codes

Success

- XR_SUCCESS

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_RUNTIME_FAILURE

12.44.4. Get Render Model Properties

The xrGetRenderModelPropertiesEXT function is defined as:

```
// Provided by XR_EXT_render_model
XrResult xrGetRenderModelPropertiesEXT(
    XrRenderModelEXT                            renderModel,
    const XrRenderModelPropertiesGetInfoEXT*    getInfo,
    XrRenderModelPropertiesEXT*                 properties);
```

Parameter Descriptions

- `renderModel` is an XrRenderModelEXT previously created by xrCreateRenderModelEXT.
- `getInfo` exists for extensibility purposes, it is `NULL` or a pointer to a valid XrRenderModelPropertiesGetInfoEXT.
- `properties` is a pointer to an XrRenderModelPropertiesEXT structure.

The properties of an XrRenderModelEXT handle are immutable and **must** not change for the lifetime of the handle.

Valid Usage (Implicit)

- The `XR_EXT_render_model` extension **must** be enabled prior to calling xrGetRenderModelPropertiesEXT
- `renderModel` **must** be a valid XrRenderModelEXT handle
- If `getInfo` is not `NULL`, `getInfo` **must** be a pointer to a valid XrRenderModelPropertiesGetInfoEXT structure
- `properties` **must** be a pointer to an XrRenderModelPropertiesEXT structure

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_OUT_OF_MEMORY`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_VALIDATION_FAILURE`

XrRenderModelPropertiesGetInfoEXT is an input structure for xrGetRenderModelPropertiesEXT.

```
// Provided by XR_EXT_render_model
typedef struct XrRenderModelPropertiesGetInfoEXT {
    XrStructureType    type;
    const void*        next;
} XrRenderModelPropertiesGetInfoEXT;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.

Valid Usage (Implicit)

- The `XR_EXT_render_model` extension **must** be enabled prior to using XrRenderModelPropertiesGetInfoEXT
- `type` **must** be `XR_TYPE_RENDER_MODEL_PROPERTIES_GET_INFO_EXT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

The XrRenderModelPropertiesEXT structure is defined as:

```
// Provided by XR_EXT_render_model
typedef struct XrRenderModelPropertiesEXT {
    XrStructureType    type;
    void*              next;
    XrUuidEXT          cacheId;
    uint32_t           animatableNodeCount;
} XrRenderModelPropertiesEXT;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `cacheId` is an XrUuidEXT to uniquely identify a render model asset using only extensions listed by the application at XrRenderModelEXT creation.
- `animatableNodeCount` is the number of uniquely-named nodes that are animated using state tracked by the XrRenderModelEXT.

The XrRenderModelPropertiesEXT structure is an output structure for

xrGetRenderModelPropertiesEXT.

Applications **may** use `cacheId` to avoid loading the exact same render model asset twice when two or more XrRenderModelEXT handles use the same glTF asset. Applications **may** also use `cacheId` to cache preprocessed render model asset data (and the associated `animatableNodeCount` node names) between sessions: it is a persistent UUID, unlike the associated XrRenderModelEXT handle or `XrRenderModelIdEXT` atom. Note that runtimes **may** return a different UUID for a given logical entity (e.g. hardware) in another session.

Within the corresponding XrSession, the association between an `XrRenderModelIdEXT` value, the glTF extensions required by the underlying model based on the contents of the XrRenderModelCreateInfoEXT::`gltfExtensions` array, and the `cacheId`, is constant. A UUID `cacheId` corresponds to a unique binary asset, with a constant `animatableNodeCount`, and is a function of the render model ID and the required glTF extensions selected based on the supported glTF extension contents reported by the application. The runtime **must** set `cacheId` to a valid UUID value and subsequent valid calls to xrGetRenderModelPropertiesEXT with the same XrRenderModelEXT and XrRenderModelPropertiesGetInfoEXT values **must** return the same values for `cacheId` while that ID remains valid to use.

Valid Usage (Implicit)

- The `XR_EXT_render_model` extension **must** be enabled prior to using XrRenderModelPropertiesEXT
- `type` **must** be `XR_TYPE_RENDER_MODEL_PROPERTIES_EXT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

12.44.5. Locate a Render Model in Space

The application **can** locate a render model by first creating an XrSpace handle from an XrRenderModelEXT handle.

The xrCreateRenderModelSpaceEXT function is defined as:

```
// Provided by XR_EXT_render_model
XrResult xrCreateRenderModelSpaceEXT(
    XrSession                                   session,
    const XrRenderModelSpaceCreateInfoEXT*      createInfo,
    XrSpace*                                    space);
```

Parameter Descriptions

- `session` is the parent XrSession handle of your XrRenderModelEXT handle, which will become the parent of the created XrSpace handle as well.
- `createInfo` is the XrRenderModelSpaceCreateInfoEXT used to specify the space.
- `space` is the returned XrSpace handle of the new render model space.

The application **can** create an XrSpace handle that tracks a render model using xrCreateRenderModelSpaceEXT.

The origin of the underlying render model space is defined to be the origin of the glTF model.

Applications **can** use xrLocateSpace to locate the space created this way in a desired base space, as with all other varieties of XrSpace handles. Unless otherwise specified by a related extension, the pose and locatability of a render model space have no fixed relationship with any other object or space, and **should** be used only to transform the associated model for rendering. If a render model space is not both position and orientation `TRACKED` when location is queried for a time equal to the intended display time, this indicates that the application is intended to *not* render that model in that frame, unless otherwise specified by a related extension. This is used in lieu of an explicit visibility state flag.

Valid Usage (Implicit)

- The `XR_EXT_render_model` extension **must** be enabled prior to calling xrCreateRenderModelSpaceEXT
- `session` **must** be a valid XrSession handle
- `createInfo` **must** be a pointer to a valid XrRenderModelSpaceCreateInfoEXT structure
- `space` **must** be a pointer to an XrSpace handle

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The XrRenderModelSpaceCreateInfoEXT structure is defined as:

```
// Provided by XR_EXT_render_model
typedef struct XrRenderModelSpaceCreateInfoEXT {
    XrStructureType     type;
    const void*         next;
    XrRenderModelEXT    renderModel;
} XrRenderModelSpaceCreateInfoEXT;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain.
- renderModel is an XrRenderModelEXT previously created by xrCreateRenderModelEXT.

XrRenderModelSpaceCreateInfoEXT is an input structure for xrCreateRenderModelSpaceEXT.

Valid Usage (Implicit)

- The XR_EXT_render_model extension **must** be enabled prior to using XrRenderModelSpaceCreateInfoEXT
- type **must** be XR_TYPE_RENDER_MODEL_SPACE_CREATE_INFO_EXT
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- renderModel **must** be a valid XrRenderModelEXT handle

12.44.6. Create Render Model Asset Handle

```
// Provided by XR_EXT_render_model
XR_DEFINE_HANDLE(XrRenderModelAssetEXT)
```

The XrRenderModelAssetEXT handle represents the in-runtime memory buffer for a glTF 2.0 render model asset, and the node names in that asset that correspond to the state array elements tracked by XrRenderModelEXT. The application **may** destroy the asset handle when it has finished retrieving the binary data and name array into its own memory, that is, after successful application of the two-call idiom with two calls to xrGetRenderModelAssetDataEXT.

The xrCreateRenderModelAssetEXT function is defined as:

```
// Provided by XR_EXT_render_model
XrResult xrCreateRenderModelAssetEXT(
    XrSession                                   session,
    const XrRenderModelAssetCreateInfoEXT*      createInfo,
    XrRenderModelAssetEXT*                      asset);
```

Parameter Descriptions

- session is an XrSession handle from which a render model ID and its corresponding UUID have been retrieved.
- createInfo is a pointer to an XrRenderModelAssetCreateInfoEXT structure.
- asset is the returned XrRenderModelAssetEXT handle.

An application **can** create an XrRenderModelAssetEXT handle using the xrCreateRenderModelAssetEXT function. The application **must** only call xrCreateRenderModelAssetEXT with a UUID specified by parameter createInfo member XrRenderModelAssetCreateInfoEXT::cacheId that has been retrieved by calling xrGetRenderModelPropertiesEXT on a render model associated with the current session. If the

application passes a UUID not retrieved in this way (for example, passing a UUID received from a previous session), the runtime **must** return `XR_ERROR_RENDER_MODEL_ASSET_UNAVAILABLE_EXT`. This implies that the runtime **must** track which UUIDs it has returned to the application in a given session to validate the input to this function. If this function returns successfully, the runtime **must** have the asset data and node names in memory for immediate return to the application in a subsequent use of xrGetRenderModelAssetDataEXT.

The runtime **may** return `XR_ERROR_RENDER_MODEL_ASSET_UNAVAILABLE_EXT` if the asset data has become unavailable for external reasons after the creation of the relevant XrRenderModelEXT.

A valid asset handle enables the application to retrieve the data for the glTF asset of the render model and the names of animatable nodes. For a valid XrRenderModelPropertiesEXT::`cacheId`, the runtime **must** return the same glTF asset data, even between different sessions, if the cache ID is returned from both sessions. Therefore, the application **may** rely on the XrRenderModelPropertiesEXT::`cacheId` to cache the glTF asset data and the processed derived data from the asset, as well as the names of animatable nodes, for reuse across sessions. An application **may** choose to use the UUID as a key to cache data associated with the asset, but is not the asset data itself, however it is invalid to call xrCreateRenderModelAssetEXT using a cached UUID before it is available from the current session. An application **must** not use a cached UUID to retrieve asset data from the runtime without ensuring it is retrievable from the current session (and identifying the semantic use of the model) by calling xrGetRenderModelPropertiesEXT.

Valid Usage (Implicit)

- The `XR_EXT_render_model` extension **must** be enabled prior to calling xrCreateRenderModelAssetEXT
- `session` **must** be a valid XrSession handle
- `createInfo` **must** be a pointer to a valid XrRenderModelAssetCreateInfoEXT structure
- `asset` **must** be a pointer to an XrRenderModelAssetEXT handle

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RENDER_MODEL_ASSET_UNAVAILABLE_EXT
- XR_ERROR_RENDER_MODEL_GLTF_EXTENSION_REQUIRED_EXT
- XR_ERROR_RENDER_MODEL_ID_INVALID_EXT
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The XrRenderModelAssetCreateInfoEXT structure is defined as:

```
// Provided by XR_EXT_render_model
typedef struct XrRenderModelAssetCreateInfoEXT {
    XrStructureType    type;
    const void*        next;
    XrUuidEXT          cacheId;
} XrRenderModelAssetCreateInfoEXT;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- cacheId is an XrUuidEXT to uniquely identify a render model asset.

The XrRenderModelAssetCreateInfoEXT structure contains the information to create an XrRenderModelAssetEXT handle.

The UUID `cacheId` **must** match the XrRenderModelPropertiesEXT::`cacheId` from some previous call to xrGetRenderModelPropertiesEXT in the current session.

Valid Usage (Implicit)

- The `XR_EXT_render_model` extension **must** be enabled prior to using XrRenderModelAssetCreateInfoEXT
- `type` **must** be `XR_TYPE_RENDER_MODEL_ASSET_CREATE_INFO_EXT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

The xrDestroyRenderModelAssetEXT function is defined as:

```
// Provided by XR_EXT_render_model
XrResult xrDestroyRenderModelAssetEXT(
    XrRenderModelAssetEXT                       asset);
```

Parameter Descriptions

- `asset` is an XrRenderModelAssetEXT previously created by xrCreateRenderModelAssetEXT.

The xrDestroyRenderModelAssetEXT function releases the XrRenderModelAssetEXT handle and the underlying resources for the glTF asset data and names of animatable nodes.

For clarity, a call to xrDestroyRenderModelAssetEXT does *not* stop the ability to locate a render model space, nor the ability to retrieve animatable node states. The asset handle refers only to the asset data and list of animatable node names in memory for transfer to the application.

Valid Usage (Implicit)

- The `XR_EXT_render_model` extension **must** be enabled prior to calling xrDestroyRenderModelAssetEXT
- `asset` **must** be a valid XrRenderModelAssetEXT handle

Thread Safety

- Access to `asset`, and any child handles, **must** be externally synchronized

Return Codes

Success

- XR_SUCCESS

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_RUNTIME_FAILURE

12.44.7. Retrieve Render Model Asset Data

The xrGetRenderModelAssetDataEXT function is defined as:

```
// Provided by XR_EXT_render_model
XrResult xrGetRenderModelAssetDataEXT(
    XrRenderModelAssetEXT                       asset,
    const XrRenderModelAssetDataGetInfoEXT*     getInfo,
    XrRenderModelAssetDataEXT*                  buffer);
```

Parameter Descriptions

- asset is an XrRenderModelAssetEXT previously created by xrCreateRenderModelAssetEXT.
- getInfo exists for extensibility purposes, it is NULL or a pointer to a valid XrRenderModelAssetDataGetInfoEXT.
- buffer is a pointer to an XrRenderModelAssetDataEXT structure for retrieving the glTF binary data.

The application **can** use the xrGetRenderModelAssetDataEXT function to populate application-allocated memory with the glTF 2.0 binary data and animatable node names of a render model asset. The application uses a two-call idiom with xrGetRenderModelAssetDataEXT to allocate the memory required for the binary asset data.

The binary data copied by the xrGetRenderModelAssetDataEXT function **must** conform to the glTF 2.0 binary format (GLB) and **must** contain a valid glTF 2.0 asset that passes validation.

Note

Runtimes are strongly encouraged to use a tool such as glTF Validator to help detect issues that make their models invalid, and thus make their implementation of this extension non-conformant. Passing validation with such a tool is *necessary*, but not *sufficient*, to prove that an asset is valid in the context of this extension.

The glTF asset data returned from this function **must** not change during the lifetime of the corresponding XrRenderModelAssetEXT handle. Further, the runtime **must** return the same glTF binary data for any XrRenderModelAssetEXT handles created using the same XrUuidEXT XrRenderModelPropertiesEXT::`cacheId`.

The application **may** call xrDestroyRenderModelAssetEXT after successfully populating the buffer with this call, and similar successful use of xrGetRenderModelAssetPropertiesEXT, as the only purpose of this handle is to manage the lifetime of the loaded glTF asset (copied into application-allocated memory by this call) and animatable node names (copied into application-allocated memory by xrGetRenderModelAssetPropertiesEXT) within the runtime.

Valid Usage (Implicit)

- The `XR_EXT_render_model` extension **must** be enabled prior to calling xrGetRenderModelAssetDataEXT
- `asset` **must** be a valid XrRenderModelAssetEXT handle
- If `getInfo` is not `NULL`, `getInfo` **must** be a pointer to a valid XrRenderModelAssetDataGetInfoEXT structure
- `buffer` **must** be a pointer to an XrRenderModelAssetDataEXT structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_SIZE_INSUFFICIENT
- XR_ERROR_VALIDATION_FAILURE

The XrRenderModelAssetDataGetInfoEXT structure is defined as:

```
// Provided by XR_EXT_render_model
typedef struct XrRenderModelAssetDataGetInfoEXT {
    XrStructureType    type;
    const void*        next;
} XrRenderModelAssetDataGetInfoEXT;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain.

XrRenderModelAssetDataGetInfoEXT is an input structure for xrGetRenderModelAssetDataEXT, defined for the purpose of future extension.

Valid Usage (Implicit)

- The XR_EXT_render_model extension **must** be enabled prior to using XrRenderModelAssetDataGetInfoEXT
- type **must** be XR_TYPE_RENDER_MODEL_ASSET_DATA_GET_INFO_EXT
- next **must** be NULL or a valid pointer to the next structure in a structure chain

The XrRenderModelAssetDataEXT structure is defined as:

```
// Provided by XR_EXT_render_model
typedef struct XrRenderModelAssetDataEXT {
    XrStructureType    type;
    void*              next;
    uint32_t           bufferCapacityInput;
    uint32_t           bufferCountOutput;
    uint8_t*           buffer;
} XrRenderModelAssetDataEXT;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain.
- bufferCapacityInput is the capacity of the array, or 0 to indicate a request to retrieve the required capacity.
- bufferCountOutput is the count of uint8_t written to buffer, or the required capacity in the case that bufferCapacityInput is 0.
- buffer is a pointer to an application-allocated byte array that will be filled with the render model asset binary data.
- See the Buffer Size Parameters section for a detailed description of retrieving the required buffer size.

XrRenderModelAssetDataEXT is an input/output structure for xrGetRenderModelAssetDataEXT.

Valid Usage (Implicit)

- The `XR_EXT_render_model` extension **must** be enabled prior to using XrRenderModelAssetDataEXT
- `type` **must** be `XR_TYPE_RENDER_MODEL_ASSET_DATA_EXT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- If `bufferCapacityInput` is not `0`, `buffer` **must** be a pointer to an array of `bufferCapacityInput` `uint8_t` values

12.44.8. Animate Parts of a Render Model

The application **can** animate parts of the glTF model using data from the runtime by retrieving and updating the XrPosef offset and visibility state of certain glTF nodes identified by unique names. The requirements for interpretation of the pose and visibility state in an application renderer are described in XrRenderModelNodeStateEXT.

The number of animatable nodes is a property of the XrRenderModelEXT, and are retrieved with xrGetRenderModelPropertiesEXT as previously described. The identities of those animatable nodes are properties of the render model asset, and are retrieved with xrGetRenderModelAssetPropertiesEXT.

The xrGetRenderModelAssetPropertiesEXT function is defined as:

```
// Provided by XR_EXT_render_model
XrResult xrGetRenderModelAssetPropertiesEXT(
    XrRenderModelAssetEXT                       asset,
    const XrRenderModelAssetPropertiesGetInfoEXT* getInfo,
    XrRenderModelAssetPropertiesEXT*            properties);
```

Parameter Descriptions

- `asset` is an XrRenderModelAssetEXT previously created by xrCreateRenderModelAssetEXT.
- `getInfo` exists for extensibility purposes, it is `NULL` or a pointer to a valid XrRenderModelAssetPropertiesGetInfoEXT.
- `properties` is a pointer to an XrRenderModelAssetPropertiesEXT structure for the render model asset properties.

The application **can** use the xrGetRenderModelAssetPropertiesEXT function to get the array of animatable node names in the glTF asset.

The runtime **must** return node names in `properties` member XrRenderModelAssetPropertiesEXT

::nodeProperties that are unique within the corresponding glTF asset.

The application **must** allocate an array of XrRenderModelAssetNodePropertiesEXT within properties, of size XrRenderModelAssetPropertiesEXT::nodePropertyCount, which **must** be equal to XrRenderModelPropertiesEXT::animatableNodeCount. If XrRenderModelAssetPropertiesEXT::nodePropertyCount is not equal to XrRenderModelPropertiesEXT::animatableNodeCount as populated by xrGetRenderModelPropertiesEXT, the runtime **must** return XR_ERROR_VALIDATION_FAILURE from xrGetRenderModelAssetPropertiesEXT. Because the number of animatable nodes is fixed per render model handle and retrievable with xrGetRenderModelPropertiesEXT, the two-call idiom for buffer sizing and allocation is not needed in this case.

The application **may** call xrDestroyRenderModelAssetEXT after successfully populating the buffer with this call, and similar successful use of xrGetRenderModelAssetDataEXT, as the only purpose of this handle is to manage the lifetime of the animatable node names (copied into application-allocated memory by this call) the loaded glTF asset (copied into application-allocated memory by xrGetRenderModelAssetDataEXT) within the runtime.

Valid Usage (Implicit)

- The XR_EXT_render_model extension **must** be enabled prior to calling xrGetRenderModelAssetPropertiesEXT
- asset **must** be a valid XrRenderModelAssetEXT handle
- If getInfo is not NULL, getInfo **must** be a pointer to a valid XrRenderModelAssetPropertiesGetInfoEXT structure
- properties **must** be a pointer to an XrRenderModelAssetPropertiesEXT structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The xrGetRenderModelAssetPropertiesEXT call takes an **optional** getInfo parameter for extensibility.

The XrRenderModelAssetPropertiesGetInfoEXT structure is defined as:

```
// Provided by XR_EXT_render_model
typedef struct XrRenderModelAssetPropertiesGetInfoEXT {
    XrStructureType    type;
    const void*        next;
} XrRenderModelAssetPropertiesGetInfoEXT;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain.

This structure exists for extensibility purposes.

Valid Usage (Implicit)

- The XR_EXT_render_model extension **must** be enabled prior to using XrRenderModelAssetPropertiesGetInfoEXT
- type **must** be XR_TYPE_RENDER_MODEL_ASSET_PROPERTIES_GET_INFO_EXT
- next **must** be NULL or a valid pointer to the next structure in a structure chain

The xrGetRenderModelAssetPropertiesEXT call populates a XrRenderModelAssetPropertiesEXT supplied by the application, including an application-allocated array for the animatable node properties.

The XrRenderModelAssetPropertiesEXT structure is defined as:

```
// Provided by XR_EXT_render_model
typedef struct XrRenderModelAssetPropertiesEXT {
    XrStructureType                         type;
    void*                                   next;
    uint32_t                                nodePropertyCount;
    XrRenderModelAssetNodePropertiesEXT*    nodeProperties;
} XrRenderModelAssetPropertiesEXT;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain.
- nodePropertyCount is the number of elements in nodeProperties.
- nodeProperties is a pointer to an application-allocated array of XrRenderModelAssetNodePropertiesEXT that will be filled with the render model asset node properties.

The count XrRenderModelAssetPropertiesEXT::nodePropertyCount **must** be equal to XrRenderModelPropertiesEXT::animatableNodeCount. If XrRenderModelAssetPropertiesEXT::nodePropertyCount is not equal to XrRenderModelPropertiesEXT::animatableNodeCount as populated by xrGetRenderModelPropertiesEXT, the runtime **must** return XR_ERROR_VALIDATION_FAILURE from xrGetRenderModelAssetPropertiesEXT.

The node names in the nodeProperties array define the identities of the animatable nodes. Order is significant, in that node states retrieved repeatedly during rendering form a parallel associated array.

Because the number of animatable nodes is fixed per render model handle and retrievable with xrGetRenderModelPropertiesEXT, the two-call idiom for buffer sizing and allocation is not needed in

this case.

Valid Usage (Implicit)

- The XR_EXT_render_model extension **must** be enabled prior to using XrRenderModelAssetPropertiesEXT
- type **must** be XR_TYPE_RENDER_MODEL_ASSET_PROPERTIES_EXT
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- nodeProperties **must** be a pointer to an array of nodePropertyCount XrRenderModelAssetNodePropertiesEXT structures
- The nodePropertyCount parameter **must** be greater than 0

The XrRenderModelAssetNodePropertiesEXT structure is defined as:

```
// Provided by XR_EXT_render_model
typedef struct XrRenderModelAssetNodePropertiesEXT {
    char    uniqueName[XR_MAX_RENDER_MODEL_ASSET_NODE_NAME_SIZE_EXT];
} XrRenderModelAssetNodePropertiesEXT;
```

Member Descriptions

- uniqueName is a unique string name of a node in the glTF asset.

The string returned in uniqueName **must** be the name of exactly one node in the glTF asset. Any given name **must** appear no more than once in the XrRenderModelAssetPropertiesEXT::nodeProperties for a given XrRenderModelAssetEXT.

Valid Usage (Implicit)

- The XR_EXT_render_model extension **must** be enabled prior to using XrRenderModelAssetNodePropertiesEXT
- uniqueName **must** be a null-terminated UTF-8 string whose length is less than or equal to XR_MAX_RENDER_MODEL_ASSET_NODE_NAME_SIZE_EXT

The xrGetRenderModelStateEXT function reads the current state of the animatable nodes in the render model.

```
// Provided by XR_EXT_render_model
XrResult xrGetRenderModelStateEXT(
    XrRenderModelEXT                            renderModel,
    const XrRenderModelStateGetInfoEXT*         getInfo,
    XrRenderModelStateEXT*                      state);
```

Parameter Descriptions

- `renderModel` is an XrRenderModelEXT previously created by xrCreateRenderModelEXT.
- `getInfo` is a pointer to a valid XrRenderModelStateGetInfoEXT specifying parameters affecting the state to retrieve.
- `state` is a pointer to an XrRenderModelStateEXT structure.

The order of the elements in XrRenderModelStateEXT::`nodeStates` in `state` is the same as the order of node names returned by the xrGetRenderModelAssetPropertiesEXT function. The corresponding index in XrRenderModelStateEXT::`nodeStates` is the same as the index in XrRenderModelAssetPropertiesEXT::`nodeProperties`. The number of states is XrRenderModelPropertiesEXT::`animatableNodeCount`.

The runtime **must** return `XR_ERROR_VALIDATION_FAILURE` if XrRenderModelStateEXT::`nodeStateCount` is not equal to XrRenderModelPropertiesEXT::`animatableNodeCount`.

Valid Usage (Implicit)

- The `XR_EXT_render_model` extension **must** be enabled prior to calling xrGetRenderModelStateEXT
- `renderModel` **must** be a valid XrRenderModelEXT handle
- `getInfo` **must** be a pointer to a valid XrRenderModelStateGetInfoEXT structure
- `state` **must** be a pointer to an XrRenderModelStateEXT structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_TIME_INVALID
- XR_ERROR_VALIDATION_FAILURE

The XrRenderModelStateGetInfoEXT structure is defined as:

```
// Provided by XR_EXT_render_model
typedef struct XrRenderModelStateGetInfoEXT {
    XrStructureType    type;
    const void*        next;
    XrTime             displayTime;
} XrRenderModelStateGetInfoEXT;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain.
- displayTime is the anticipated XrTime to query state for, and for which the model will be rendered and displayed.

When retrieving model state for a given frame, displayTime **should** be set to the time value intended to be passed as XrFrameEndInfo::displayTime. See xrEndFrame for information on how to compute this value.

Valid Usage (Implicit)

- The XR_EXT_render_model extension **must** be enabled prior to using XrRenderModelStateGetInfoEXT
- type **must** be XR_TYPE_RENDER_MODEL_STATE_GET_INFO_EXT
- next **must** be NULL or a valid pointer to the next structure in a structure chain

The XrRenderModelStateEXT structure is defined as:

```
// Provided by XR_EXT_render_model
typedef struct XrRenderModelStateEXT {
    XrStructureType                type;
    void*                          next;
    uint32_t                       nodeStateCount;
    XrRenderModelNodeStateEXT*     nodeStates;
} XrRenderModelStateEXT;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain.
- nodeStateCount is the count of XrRenderModelNodeStateEXT structures in the nodeStates array.
- nodeStates is a pointer to an application-allocated array of XrRenderModelNodeStateEXT structures that will be filled with the states of the nodes.

Valid Usage (Implicit)

- The XR_EXT_render_model extension **must** be enabled prior to using XrRenderModelStateEXT
- type **must** be XR_TYPE_RENDER_MODEL_STATE_EXT
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- If nodeStateCount is not 0, nodeStates **must** be a pointer to an array of nodeStateCount XrRenderModelNodeStateEXT structures

The XrRenderModelNodeStateEXT structure is defined as:

```
// Provided by XR_EXT_render_model
typedef struct XrRenderModelNodeStateEXT {
    XrPosef     nodePose;
    XrBool32    isVisible;
} XrRenderModelNodeStateEXT;
```

Member Descriptions

- `nodePose` is an XrPosef of the node in its parent node space.
- `isVisible` is an `XrBool32` which indicates if the node is visible or not.

This structure is populated with state for a single animatable node in an XrRenderModelEXT.

For any animatable node *N*, if an ancestor node *M* is also animatable, and `isVisible` is `XR_FALSE` for node *M*, then `isVisible` **must** be `XR_FALSE` for node *N* as well. That is, being not-visible is recursive. An application **should** interpret all descendant nodes of an animatable node with `isVisible` = `XR_FALSE` to also not be visible (to similarly interpret being not-visible as recursive).

The pose `nodePose` locates the associated animatable node, and all descendants, relative to that animatable node's parent, replacing the animatable node's transform, if any was supplied as `matrix` or `translation`/`rotation`/`scale` properties in the glTF asset. The application **should** apply this `nodePose` to the associated node, as well as to all descendant nodes per the glTF specification. That is, the `nodePose` replaces, instead of composes with, the asset-specified transform.

Where one animatable node *M* is a descendant of another animatable node *N*, the application **should** transform the descendant node *M* and its descendants by the composition of the `nodePose` for both *M* and *N*. That is, `nodePose` **should** be interpreted by the application to respect the hierarchy in the glTF asset, and compose with other animatable node poses, as well as transformations supplied in the glTF asset on non-animatable nodes.

For clarity, given a model for which the runtime returns a `nodePose` equal to the original transform in the asset for all animatable nodes, the resulting rendered model **should** be rendered the same as the unmodified glTF asset. This implies that for ease of use, runtimes **may** consider structuring their assets such that animatable nodes have no (or identity) transformation specified in the glTF asset, such that `nodePose` of identity for all animatable nodes produces an rendered model in its neutral, original state.

Valid Usage (Implicit)

- The `XR_EXT_render_model` extension **must** be enabled prior to using XrRenderModelNodeStateEXT

12.44.9. Example

```
// previously initialized
extern XrInstance instance;
extern XrSession session;
extern XrSpace baseSpace;

// retrieved from another extension that builds on this one
extern XrRenderModelIdEXT renderModelId;

// Get the function pointers for the extension's functions.
PFN_xrCreateRenderModelEXT pfnCreateRenderModelEXT = nullptr;
CHK_XR(xrGetInstanceProcAddr(
    instance, "xrCreateRenderModelEXT",
    reinterpret_cast<PFN_xrVoidFunction *>(&pfnCreateRenderModelEXT)));

PFN_xrDestroyRenderModelEXT pfnDestroyRenderModelEXT = nullptr;
CHK_XR(xrGetInstanceProcAddr(
    instance, "xrDestroyRenderModelEXT",
    reinterpret_cast<PFN_xrVoidFunction *>(&pfnDestroyRenderModelEXT)));

PFN_xrGetRenderModelPropertiesEXT pfnGetRenderModelPropertiesEXT = nullptr;
CHK_XR(xrGetInstanceProcAddr(
    instance, "xrGetRenderModelPropertiesEXT",
    reinterpret_cast<PFN_xrVoidFunction *>(&pfnGetRenderModelPropertiesEXT)));

PFN_xrCreateRenderModelSpaceEXT pfnCreateRenderModelSpaceEXT = nullptr;
CHK_XR(xrGetInstanceProcAddr(
    instance, "xrCreateRenderModelSpaceEXT",
    reinterpret_cast<PFN_xrVoidFunction *>(&pfnCreateRenderModelSpaceEXT)));

PFN_xrCreateRenderModelAssetEXT pfnCreateRenderModelAssetEXT = nullptr;
CHK_XR(xrGetInstanceProcAddr(
    instance, "xrCreateRenderModelAssetEXT",
    reinterpret_cast<PFN_xrVoidFunction *>(&pfnCreateRenderModelAssetEXT)));

PFN_xrDestroyRenderModelAssetEXT pfnDestroyRenderModelAssetEXT = nullptr;
CHK_XR(xrGetInstanceProcAddr(
    instance, "xrDestroyRenderModelAssetEXT",
    reinterpret_cast<PFN_xrVoidFunction *>(&pfnDestroyRenderModelAssetEXT)));

PFN_xrGetRenderModelAssetDataEXT pfnGetRenderModelAssetDataEXT = nullptr;
CHK_XR(xrGetInstanceProcAddr(
    instance, "xrGetRenderModelAssetDataEXT",
    reinterpret_cast<PFN_xrVoidFunction *>(&pfnGetRenderModelAssetDataEXT)));

PFN_xrGetRenderModelAssetPropertiesEXT pfnGetRenderModelAssetPropertiesEXT;
```

```
CHK_XR(xrGetInstanceProcAddr(instance, "xrGetRenderModelAssetPropertiesEXT",
                             reinterpret_cast<PFN_xrVoidFunction *>(
                                 &pfnGetRenderModelAssetPropertiesEXT)));

PFN_xrGetRenderModelStateEXT pfnGetRenderModelStateEXT = nullptr;
CHK_XR(xrGetInstanceProcAddr(
    instance, "xrGetRenderModelStateEXT",
    reinterpret_cast<PFN_xrVoidFunction *>(&pfnGetRenderModelStateEXT)));

// Create render model handles
// The names of glTF extensions that the application is capable of supporting.
// The returned glTF model is allowed to have have any or all of these extensions
// listed in the "extensionsRequired" array.
// Pass only the extensions that your app/engine are capable of supporting.
std::vector<const char *> appSupportedGltfExtensions{"KHR_texture_basisu",
                                                     "KHR_materials_specular"};

XrRenderModelEXT renderModel {XR_NULL_HANDLE};
XrRenderModelCreateInfoEXT renderModelCreateInfo {
    .type = XR_TYPE_RENDER_MODEL_CREATE_INFO_EXT,
    .renderModelId = renderModelId,
    .gltfExtensionCount = static_cast<uint32_t>(appSupportedGltfExtensions.size()),
    .gltfExtensions = appSupportedGltfExtensions.data(),
};
CHK_XR(
    pfnCreateRenderModelEXT(session, &renderModelCreateInfo, &renderModel));


// Create a space for locating the render model.
XrRenderModelSpaceCreateInfoEXT spaceCreateInfo {
    .type = XR_TYPE_RENDER_MODEL_SPACE_CREATE_INFO_EXT,
    .renderModel = renderModel,
};
XrSpace modelSpace {XR_NULL_HANDLE};
CHK_XR(pfnCreateRenderModelSpaceEXT(session, &spaceCreateInfo, &modelSpace));

// Get the model properties: UUID and number of animatable nodes
XrRenderModelPropertiesGetInfoEXT propertiesGetInfo {
    .type = XR_TYPE_RENDER_MODEL_PROPERTIES_GET_INFO_EXT,
};
XrRenderModelPropertiesEXT properties {
    .type = XR_TYPE_RENDER_MODEL_PROPERTIES_EXT,
};
CHK_XR(pfnGetRenderModelPropertiesEXT(renderModel, &propertiesGetInfo, &properties));

{
  // Create the asset handle to request the data.
  XrRenderModelAssetCreateInfoEXT assetCreateInfo {
```

```
        .type = XR_TYPE_RENDER_MODEL_ASSET_CREATE_INFO_EXT,
        .cacheId = properties.cacheId,
    };
    XrRenderModelAssetEXT asset {XR_NULL_HANDLE};
    CHK_XR(pfnCreateRenderModelAssetEXT(session, &assetCreateInfo, &asset));

    // Copy the binary glTF (GLB) asset data using two-call idiom.
    XrRenderModelAssetDataGetInfoEXT assetGetInfo {
        .type = XR_TYPE_RENDER_MODEL_ASSET_DATA_GET_INFO_EXT,
    };
    XrRenderModelAssetDataEXT assetData {
        .type = XR_TYPE_RENDER_MODEL_ASSET_DATA_EXT,
    };
    CHK_XR(pfnGetRenderModelAssetDataEXT(asset, &assetGetInfo, &assetData));
    std::vector<uint8_t> glbData(assetData.bufferCountOutput);
    assetData.bufferCapacityInput = (uint32_t)glbData.size();
    assetData.buffer = glbData.data();
    CHK_XR(pfnGetRenderModelAssetDataEXT(asset, &assetGetInfo, &assetData));

    // Parsing the binary glTF data is outside the scope of this extension,
    // but do it here.

    // Get the unique names of the animatable nodes
    XrRenderModelAssetPropertiesGetInfoEXT assetPropertiesGetInfo {
        .type = XR_TYPE_RENDER_MODEL_ASSET_PROPERTIES_GET_INFO_EXT,
    };
    XrRenderModelAssetPropertiesEXT assetProperties {
        .type = XR_TYPE_RENDER_MODEL_ASSET_PROPERTIES_EXT,
    };
    std::vector<XrRenderModelAssetNodePropertiesEXT> nodeProperties(
        properties.animatableNodeCount);
    assetProperties.nodePropertyCount = (uint32_t)nodeProperties.size();
    assetProperties.nodeProperties = nodeProperties.data();
    CHK_XR(pfnGetRenderModelAssetPropertiesEXT(asset, &assetPropertiesGetInfo,
                                               &assetProperties));

    // Once the glTF data has been handled, we no longer need the
    // XrRenderModelAssetEXT handle.
    CHK_XR(pfnDestroyRenderModelAssetEXT(asset));


    // Save the list of nodes for rendering. The order of the array matters.
    // The application will store some sort of "reference" to a node for
    // each element, using the node name (in nodeProperties) to find it here.
    // This code is not shown because it will depend on how your
    // application represents glTF assets, so add your own here.
}
```

```
// Each frame the application's work for each model includes
// reading the state of the animatable nodes
// and then adjusting the pose or visibility of the node.

// Initialized from xrWaitFrame output
XrTime predictedDisplayTime;

// Use xrLocateSpace to locate the model's space
XrSpaceLocation modelLocation {
    .type = XR_TYPE_SPACE_LOCATION,
};
CHK_XR(xrLocateSpace(modelSpace, baseSpace, predictedDisplayTime, &modelLocation));

bool orientationTracked = (modelLocation.locationFlags &
    XR_SPACE_LOCATION_ORIENTATION_TRACKED_BIT) != 0;
bool positionTracked = (modelLocation.locationFlags &
    XR_SPACE_LOCATION_POSITION_TRACKED_BIT) != 0;

if (orientationTracked && positionTracked) {
  // Only render if the model space is tracked,
  // and if the session state is appropriate, if applicable.
  // (e.g. interaction models are only to be rendered when FOCUSED)

  XrRenderModelStateGetInfoEXT stateGetInfo {
      .type = XR_TYPE_RENDER_MODEL_STATE_GET_INFO_EXT,
      .displayTime = predictedDisplayTime,
  };

  // In practice, you do not want to re-allocate this array of
  // node state every frame, but it is clearer for illustration.
  // We know the number of elements from the model properties,
  // and we used the names from the asset handle to find and retain
  // our app-specific references to those nodes in the model.
  std::vector<XrRenderModelNodeStateEXT> nodeStates(
      properties.animatableNodeCount);
  XrRenderModelStateEXT state {
      .type = XR_TYPE_RENDER_MODEL_STATE_EXT,
      .nodeStateCount = static_cast<uint32_t>(nodeStates.size()),
      .nodeStates = nodeStates.data(),
  };
  // xrGetRenderModelStateEXT does not use the two-call idiom. The size is
  // determined by xrGetRenderModelAssetPropertiesEXT.
  CHK_XR(pfnGetRenderModelStateEXT(renderModel, &stateGetInfo, &state));

  for (size_t i = 0; i < nodeStates.size(); ++i) {
    // Use nodeStates[i].isVisible and nodeStates[i].nodePose to update the
    // node's visibility or pose.
```

```
    // nodeStates[i] refers to the node identified by name in nodeProperties[i]
  }
} else {
  // do not render any of the model if the space not locatable
}
```

12.44.10. New Base Types

- XrRenderModelIdEXT

12.44.11. New Object Types

- XrRenderModelAssetEXT
- XrRenderModelEXT

12.44.12. New Commands

- xrCreateRenderModelAssetEXT
- xrCreateRenderModelEXT
- xrCreateRenderModelSpaceEXT
- xrDestroyRenderModelAssetEXT
- xrDestroyRenderModelEXT
- xrGetRenderModelAssetDataEXT
- xrGetRenderModelAssetPropertiesEXT
- xrGetRenderModelPropertiesEXT
- xrGetRenderModelStateEXT

12.44.13. New Structures

- XrRenderModelAssetCreateInfoEXT
- XrRenderModelAssetDataEXT
- XrRenderModelAssetDataGetInfoEXT
- XrRenderModelAssetNodePropertiesEXT
- XrRenderModelAssetPropertiesEXT
- XrRenderModelAssetPropertiesGetInfoEXT
- XrRenderModelCreateInfoEXT
- XrRenderModelNodeStateEXT
- XrRenderModelPropertiesEXT

- XrRenderModelPropertiesGetInfoEXT
- XrRenderModelSpaceCreateInfoEXT
- XrRenderModelStateEXT
- XrRenderModelStateGetInfoEXT

12.44.14. New Enum Constants

- XR_EXT_RENDER_MODEL_EXTENSION_NAME
- XR_EXT_render_model_SPEC_VERSION
- XR_MAX_RENDER_MODEL_ASSET_NODE_NAME_SIZE_EXT
- XR_NULL_RENDER_MODEL_ID_EXT
- Extending XrObjectType:
  - XR_OBJECT_TYPE_RENDER_MODEL_ASSET_EXT
  - XR_OBJECT_TYPE_RENDER_MODEL_EXT
- Extending XrResult:
  - XR_ERROR_RENDER_MODEL_ASSET_UNAVAILABLE_EXT
  - XR_ERROR_RENDER_MODEL_GLTF_EXTENSION_REQUIRED_EXT
  - XR_ERROR_RENDER_MODEL_ID_INVALID_EXT
- Extending XrStructureType:
  - XR_TYPE_RENDER_MODEL_ASSET_CREATE_INFO_EXT
  - XR_TYPE_RENDER_MODEL_ASSET_DATA_EXT
  - XR_TYPE_RENDER_MODEL_ASSET_DATA_GET_INFO_EXT
  - XR_TYPE_RENDER_MODEL_ASSET_PROPERTIES_EXT
  - XR_TYPE_RENDER_MODEL_ASSET_PROPERTIES_GET_INFO_EXT
  - XR_TYPE_RENDER_MODEL_CREATE_INFO_EXT
  - XR_TYPE_RENDER_MODEL_PROPERTIES_EXT
  - XR_TYPE_RENDER_MODEL_PROPERTIES_GET_INFO_EXT
  - XR_TYPE_RENDER_MODEL_SPACE_CREATE_INFO_EXT
  - XR_TYPE_RENDER_MODEL_STATE_EXT
  - XR_TYPE_RENDER_MODEL_STATE_GET_INFO_EXT

12.44.15. Issues

- Is there any restriction on unique node names in a retrieved asset?
  - **Resolved. Yes**: any node name intended by the runtime to be used by the application, such as

through the transform/visibility animation capability in this extension, **must** be unique in that glTF file: see XrRenderModelAssetNodePropertiesEXT. The working group has verified that this is the intended way for glTF nodes to be referred to, not by index or any other method. Node names not for use by the application do not need to be unique. Node names used for animation **must** also fit in the fixed size buffer in XrRenderModelAssetNodePropertiesEXT

- Is visibility of nodes in the provided animation system recursive?
  - **Resolved. Partially recursive**: If an animatable node is not visible, and it is the ancestor of another animatable node, its descendant node is also reported as not visible. The base glTF specification does not have a concept of visibility in this way, so the semantics of it are left for this specification to define. See XrRenderModelNodeStateEXT for detail.
- What values are valid for XrRenderModelStateGetInfoEXT::`displayTime`?
  - **Not fully resolved**. It **must** be valid to use the XrFrameState::`predictedDisplayTime` returned from the most recent xrWaitFrame call. For the sake of pipelined rendering engines, XrFrameState::`predictedDisplayTime` XrFrameState::`predictedDisplayPeriod` **must** also be considered valid. Because the purpose of these calls is solely for rendering, it is unclear if any time earlier than the most recent predicted display time makes sense to support. It is also unclear how far in the future runtimes support. Additionally, depending on the purpose of a given render model, the runtime **may** not have any useful method to predict future states beyond using the most recently measured physical state.
- Do animation transforms replace transforms provided in the glTF file, or compose with them? If they compose, in what order do they compose?
  - **Resolved. They replace the transforms.** For simplicity and performance, the node state transforms are specified to replace any transformation as supplied as `matrix` or `translation`/`rotation`/`scale` properties in the glTF asset. Composing automatically gives the useful property that having all node states contain identity is equivalent to rendering without any animation ability at all, providing a way to check rendering. However, if "compose" were selected as the specified behavior, and some runtimes "baked" transforms into their node vertices (producing an asset with no transforms) while others did not, this would have presented a trap for application developers who might not realize they are supposed to honor both the glTF-provided transform as well as the node state transform. Additionally, consensus among the Working Group appeared to be strongly in favor of the "replace" option.

12.44.16. Version History

- Revision 1, 2025-06-06 (Yin Li et al)
  - Initial extension description

12.45. XR_EXT_spatial_anchor

Name String

`XR_EXT_spatial_anchor`

Extension Type

Instance extension

Registered Extension Number

763

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

`XR_EXT_spatial_entity`

Contributors

Nihav Jain, Google
Natalie Fleury, Meta
Yuichi Taguchi, Meta
Ron Bessems, Meta
Yin Li, Microsoft
Jimmy Alamparambil, ByteDance
Zhipeng Liu, ByteDance
Jun Yan, ByteDance

12.45.1. Overview

This extension builds on `XR_EXT_spatial_entity` and allows applications to create spatial anchors, which are arbitrary points in the user's physical environment that will then be tracked by the runtime. The runtime **should** then adjust the position and orientation of the anchor's origin over time as needed, independent of all other spaces & anchors, to ensure that it maintains its original mapping to the real world.

An anchor that tracks a given position and orientation within an XrSpatialContextEXT is represented as a spatial entity with (or "that has") the `XR_SPATIAL_COMPONENT_TYPE_ANCHOR_EXT` component.

12.45.2. Benefit of using anchors

As the runtime's understanding of the user's physical environment updates throughout the lifetime of an XrSpatialContextEXT, virtual objects **may** appear to drift away from where they were placed by the application, which impacts the application's realism and the quality of the user's experience. By creating an anchor close to where a virtual object is placed, and then always rendering that virtual object relative to its anchor, an application **can** ensure that each virtual object appears to stay at the same position and orientation in the physical environment. Also, unlike certain reference spaces, anchors are unaffected by system-level recentering.

12.45.3. Runtime support

If the runtime supports spatial anchors, it **must** indicate this by enumerating XR_SPATIAL_CAPABILITY_ANCHOR_EXT in xrEnumerateSpatialCapabilitiesEXT.

12.45.4. Configuration

The XrSpatialCapabilityConfigurationAnchorEXT structure is defined as:

```
// Provided by XR_EXT_spatial_anchor
typedef struct XrSpatialCapabilityConfigurationAnchorEXT {
    XrStructureType                     type;
    const void*                         next;
    XrSpatialCapabilityEXT              capability;
    uint32_t                            enabledComponentCount;
    const XrSpatialComponentTypeEXT*    enabledComponents;
} XrSpatialCapabilityConfigurationAnchorEXT;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain.
- capability is an XrSpatialCapabilityEXT.
- enabledComponentCount is a uint32_t describing the count of elements in the enabledComponents array.
- enabledComponents is a pointer to an array of XrSpatialComponentTypeEXT.

Applications **can** enable the XR_SPATIAL_CAPABILITY_ANCHOR_EXT spatial capability by including a pointer to an XrSpatialCapabilityConfigurationAnchorEXT structure in XrSpatialContextCreateInfoEXT::capabilityConfigs.

The runtime **must** return XR_ERROR_VALIDATION_FAILURE if capability is not XR_SPATIAL_CAPABILITY_ANCHOR_EXT.

Valid Usage (Implicit)

- The XR_EXT_spatial_anchor extension **must** be enabled prior to using XrSpatialCapabilityConfigurationAnchorEXT
- type **must** be XR_TYPE_SPATIAL_CAPABILITY_CONFIGURATION_ANCHOR_EXT
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- capability **must** be a valid XrSpatialCapabilityEXT value
- enabledComponents **must** be a pointer to an array of enabledComponentCount valid XrSpatialComponentTypeEXT values
- The enabledComponentCount parameter **must** be greater than 0

12.45.5. Guaranteed Components

A runtime that supports XR_SPATIAL_CAPABILITY_ANCHOR_EXT **must** provide the following spatial components as guaranteed components of all entities created or discovered by this capability and **must** enumerate them in xrEnumerateSpatialCapabilityComponentTypesEXT:

- XR_SPATIAL_COMPONENT_TYPE_ANCHOR_EXT

Anchor Component

Component Data

The XR_SPATIAL_COMPONENT_TYPE_ANCHOR_EXT uses XrPosef for its data which provides the position and orientation of the anchor.

Component List Structure to Query Data

The XrSpatialComponentAnchorListEXT structure is defined as:

```
// Provided by XR_EXT_spatial_anchor
typedef struct XrSpatialComponentAnchorListEXT {
    XrStructureType    type;
    void*              next;
    uint32_t           locationCount;
    XrPosef*           locations;
} XrSpatialComponentAnchorListEXT;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `locationCount` is a `uint32_t` describing the count of elements in the `locations` array.
- `locations` is an array of XrPosef.

The runtime **must** return `XR_ERROR_VALIDATION_FAILURE` from xrQuerySpatialComponentDataEXT if XrSpatialComponentAnchorListEXT is in the XrSpatialComponentDataQueryResultEXT::`next` chain but `XR_SPATIAL_COMPONENT_TYPE_ANCHOR_EXT` is not included in XrSpatialComponentDataQueryConditionEXT ::`componentTypes`.

The runtime **must** return `XR_ERROR_SIZE_INSUFFICIENT` from xrQuerySpatialComponentDataEXT if `locationCount` is less than XrSpatialComponentDataQueryResultEXT::`entityIdCountOutput`.

Valid Usage (Implicit)

- The `XR_EXT_spatial_anchor` extension **must** be enabled prior to using XrSpatialComponentAnchorListEXT
- `type` **must** be `XR_TYPE_SPATIAL_COMPONENT_ANCHOR_LIST_EXT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `locations` **must** be a pointer to an array of `locationCount` XrPosef structures
- The `locationCount` parameter **must** be greater than `0`

Configuration

If `XR_SPATIAL_COMPONENT_TYPE_ANCHOR_EXT` is enumerated in XrSpatialCapabilityComponentTypesEXT ::`componentTypes` for some capability, an application **can** enable it by including the enumerant in the XrSpatialCapabilityConfigurationBaseHeaderEXT::`enabledComponents` list of the XrSpatialCapabilityConfigurationBaseHeaderEXT derived structure of the capability that supports this component.

This component does not require any special configuration to be included in the XrSpatialCapabilityConfigurationBaseHeaderEXT::`next` chain.

12.45.6. Creating a Spatial Anchor

The xrCreateSpatialAnchorEXT function is defined as:

```
// Provided by XR_EXT_spatial_anchor
XrResult xrCreateSpatialAnchorEXT(
    XrSpatialContextEXT                         spatialContext,
    const XrSpatialAnchorCreateInfoEXT*         createInfo,
    XrSpatialEntityIdEXT*                       anchorEntityId,
    XrSpatialEntityEXT*                         anchorEntity);
```

Parameter Descriptions

- `spatialContext` is an XrSpatialContextEXT previously created using xrCreateSpatialContextAsyncEXT.
- `createInfo` is a pointer to an XrSpatialAnchorCreateInfoEXT.
- `anchorEntityId` is a pointer to an `XrSpatialEntityIdEXT` in which the anchor entity's ID is returned.
- `anchorEntity` is a pointer to an XrSpatialEntityEXT in which the anchor entity's handle is returned.

The application **can** create a spatial anchor by using xrCreateSpatialAnchorEXT.

To get updated component data for an anchor, pass the value populated in `anchorEntity` into the XrSpatialUpdateSnapshotCreateInfoEXT::`entities` when creating a snapshot. The application **can** use `anchorEntityId` to uniquely identify this anchor in the XrSpatialComponentDataQueryResultEXT ::`entityIds` array when using xrQuerySpatialComponentDataEXT.

The runtime **must** return `XR_ERROR_VALIDATION_FAILURE` from xrCreateSpatialAnchorEXT if `XR_SPATIAL_CAPABILITY_ANCHOR_EXT` was not configured for `spatialContext`. See Configuration for how to configure an XrSpatialContextEXT for the `XR_SPATIAL_CAPABILITY_ANCHOR_EXT` capability.

The anchor represented by `anchorEntity` is only valid for the lifetime of `spatialContext`, or until the application calls xrDestroySpatialEntityEXT on it, whichever comes first. Other extensions **may** offer functions to persist this newly created anchor across multiple XrSession or to share it across process boundaries with other applications.

A newly created anchor, until destroyed, **must** be discoverable in its parent spatial context. This means that the runtime **must** include `anchorEntityId` in the snapshot created using xrCreateSpatialDiscoverySnapshotAsyncEXT for `spatialContext` if the anchor matches the discovery criteria set in XrSpatialDiscoverySnapshotCreateInfoEXT. The newly created anchor **may** also be discoverable in other spatial contexts configured with `XR_SPATIAL_CAPABILITY_ANCHOR_EXT`, although with a different `XrSpatialEntityIdEXT` since a particular `XrSpatialEntityIdEXT` is unique to its XrSpatialContextEXT.

Valid Usage (Implicit)

- The XR_EXT_spatial_anchor extension **must** be enabled prior to calling xrCreateSpatialAnchorEXT
- spatialContext **must** be a valid XrSpatialContextEXT handle
- createInfo **must** be a pointer to a valid XrSpatialAnchorCreateInfoEXT structure
- anchorEntityId **must** be a pointer to an XrSpatialEntityIdEXT value
- anchorEntity **must** be a pointer to an XrSpatialEntityEXT handle

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_POSE_INVALID
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_TIME_INVALID
- XR_ERROR_VALIDATION_FAILURE
- XR_ERROR_SPATIAL_ANCHOR_ATTACHABLE_COMPONENT_NOT_FOUND_ANDROID (if XR_ANDROID_spatial_entity_bound_anchor is enabled)
- XR_ERROR_SPATIAL_ENTITY_ID_INVALID_EXT (if XR_ANDROID_spatial_entity_bound_anchor is enabled)

The XrSpatialAnchorCreateInfoEXT structure is defined as:

```
// Provided by XR_EXT_spatial_anchor
typedef struct XrSpatialAnchorCreateInfoEXT {
    XrStructureType    type;
    const void*        next;
    XrSpace            baseSpace;
    XrTime             time;
    XrPosef            pose;
} XrSpatialAnchorCreateInfoEXT;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `baseSpace` is the XrSpace in which `pose` is applied.
- `time` is the `XrTime` at which `baseSpace` is located (and `pose` is applied).
- `pose` is the location for the anchor entity.

Valid Usage (Implicit)

- The `XR_EXT_spatial_anchor` extension **must** be enabled prior to using XrSpatialAnchorCreateInfoEXT
- `type` **must** be `XR_TYPE_SPATIAL_ANCHOR_CREATE_INFO_EXT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain. See also: XrSpatialAnchorParentANDROID
- `baseSpace` **must** be a valid XrSpace handle

12.45.7. Query Anchor Pose

After the anchor is created, the runtime **should** then adjust its position and orientation over time relative to other spaces in order to maintain the best possible alignment to its original real-world location, even if that changes the anchor's relationship to the original XrSpatialAnchorCreateInfoEXT ::`baseSpace` used to initialize it.

The application **can** use xrCreateSpatialUpdateSnapshotEXT with the anchor's XrSpatialEntityEXT to create a new XrSpatialSnapshotEXT and then query the `XR_SPATIAL_COMPONENT_TYPE_ANCHOR_EXT` component from that snapshot using xrQuerySpatialComponentDataEXT. The application **can** add XrSpatialComponentAnchorListEXT to XrSpatialComponentDataQueryResultEXT::`next` to retrieve the latest location data for the anchors.

The runtime **may** set the tracking state of a newly created anchor to `XR_SPATIAL_ENTITY_TRACKING_STATE_PAUSED_EXT`. The application **must** only read the anchor entity's state provided in XrSpatialComponentDataQueryResultEXT::`entityStates` and the entity's anchor component data if the tracking state is `XR_SPATIAL_ENTITY_TRACKING_STATE_TRACKING_EXT`.

12.45.8. Guidelines For Using Anchors

- Each anchor's pose adjusts independent of any other anchor or space. Separately anchored virtual objects **may** shift or rotate relative to each other, breaking the spatial hierarchy in cases where these virtual objects are expected to stay in place relative to each other. For such cases, the application **should** reuse the same anchor for all virtual objects that do not move relative to each other.
- Application **should** destroy any XrSpatialEntityEXT handles for anchors that are no longer being used in order to free up the resources the runtime **may** be using to track those anchors.

12.45.9. Example Code

Configure Anchor Capability

The following example demonstrates how to configure the anchor capability when creating a spatial context.

```
// Create a spatial spatial context
XrSpatialContextEXT spatialContext {XR_NULL_HANDLE};
{

  std::vector<XrSpatialComponentTypeEXT> enabledComponents = {
    XR_SPATIAL_COMPONENT_TYPE_ANCHOR_EXT,
  };

  XrSpatialCapabilityConfigurationAnchorEXT anchorConfig {
    .type = XR_TYPE_SPATIAL_CAPABILITY_CONFIGURATION_ANCHOR_EXT,
    .capability = XR_SPATIAL_CAPABILITY_ANCHOR_EXT,
    .enabledComponentCount = static_cast<uint32_t>(enabledComponents.size()),
    .enabledComponents = enabledComponents.data(),
  };

  std::array<XrSpatialCapabilityConfigurationBaseHeaderEXT*, 1> capabilityConfigs = {
    reinterpret_cast<XrSpatialCapabilityConfigurationBaseHeaderEXT*>(&anchorConfig),
  };

  XrSpatialContextCreateInfoEXT spatialContextCreateInfo {
    .type = XR_TYPE_SPATIAL_CONTEXT_CREATE_INFO_EXT,
    .capabilityConfigCount = capabilityConfigs.size(),
    .capabilityConfigs = capabilityConfigs.data(),
  };
  XrFutureEXT createContextFuture {XR_NULL_FUTURE_EXT};
  CHK_XR(xrCreateSpatialContextAsyncEXT(session, &spatialContextCreateInfo,
&createContextFuture));

  waitUntilReady(createContextFuture);

  XrCreateSpatialContextCompletionEXT completion {
    .type = XR_TYPE_CREATE_SPATIAL_CONTEXT_COMPLETION_EXT,
  };
  CHK_XR(xrCreateSpatialContextCompleteEXT(session, createContextFuture, &completion));
  if (completion.futureResult != XR_SUCCESS) {
    return;
  }

  spatialContext = completion.spatialContext;
}

// ...
// Create spatial anchors and get their latest pose in the frame loop.
// ...

CHK_XR(xrDestroySpatialContextEXT(spatialContext));
```

Create Spatial Anchor & Get Its Location

The following example demonstrates how to create a spatial anchor & gets its pose every frame.

```
XrSpatialAnchorCreateInfoEXT createInfo {
  .type = XR_TYPE_SPATIAL_ANCHOR_CREATE_INFO_EXT,
  .baseSpace = localSpace,
  .time = predictedDisplayTime,
  .pose = {{0, 0, 0, 1}, {1, 1, 1}},
};

XrSpatialEntityIdEXT spatialAnchorEntityId;
XrSpatialEntityEXT spatialAnchorEntity {XR_NULL_HANDLE};
CHK_XR(xrCreateSpatialAnchorEXT(spatialContext, &createInfo, &spatialAnchorEntityId,
&spatialAnchorEntity));

auto updateAnchorLocation = [&](XrTime time) {
  // We want to get updated data for all components of the entities, so skip specifying
componentTypes.
  XrSpatialUpdateSnapshotCreateInfoEXT snapshotCreateInfo {
    .type = XR_TYPE_SPATIAL_UPDATE_SNAPSHOT_CREATE_INFO_EXT,
    .entityCount = 1,
    .entities = &spatialAnchorEntity,
    .baseSpace = localSpace,
    .time = time,
  };

  XrSpatialSnapshotEXT snapshot {XR_NULL_HANDLE};
  CHK_XR(xrCreateSpatialUpdateSnapshotEXT(spatialContext, &snapshotCreateInfo, &
snapshot));

  // Query for the entities that have the anchor component on them.
  std::array<XrSpatialComponentTypeEXT, 1> componentsToQuery
{XR_SPATIAL_COMPONENT_TYPE_ANCHOR_EXT};
  XrSpatialComponentDataQueryConditionEXT queryCond {
    .type = XR_TYPE_SPATIAL_COMPONENT_DATA_QUERY_CONDITION_EXT,
    .componentTypeCount = componentsToQuery.size(),
    .componentTypes = componentsToQuery.data(),
  };

  XrSpatialComponentDataQueryResultEXT queryResult {
    .type = XR_TYPE_SPATIAL_COMPONENT_DATA_QUERY_RESULT_EXT,
  };
  CHK_XR(xrQuerySpatialComponentDataEXT(snapshot, &queryCond, &queryResult));

  std::vector<XrSpatialEntityIdEXT> entityIds(queryResult.entityIdCountOutput);
  std::vector<XrSpatialEntityTrackingStateEXT> entityStates(queryResult
```

```
.entityIdCountOutput);
  queryResult.entityIdCapacityInput = entityIds.size();
  queryResult.entityIds = entityIds.data();
  queryResult.entityStateCapacityInput = entityStates.size();
  queryResult.entityStates = entityStates.data();

  // query for the pose data
  std::vector<XrPosef> locations(queryResult.entityIdCountOutput);
  XrSpatialComponentAnchorListEXT locationList {
    .type = XR_TYPE_SPATIAL_COMPONENT_ANCHOR_LIST_EXT,
    .locationCount = static_cast<uint32_t>(locations.size()),
    .locations = locations.data(),
  };
  queryResult.next = &locationList;

  CHK_XR(xrQuerySpatialComponentDataEXT(snapshot, &queryCond, &queryResult));

  for (int32_t i = 0; i < queryResult.entityIdCountOutput; ++i) {
    if (entityStates[i] == XR_SPATIAL_ENTITY_TRACKING_STATE_TRACKING_EXT) {
      // Pose for entity entityIds[i] is locations[i].
    }
  }

  CHK_XR(xrDestroySpatialSnapshotEXT(snapshot));
};


while (1) {
  // ...
  // For every frame in frame loop
  // ...

  XrFrameState frameState;  // previously returned from xrWaitFrame
  const XrTime time = frameState.predictedDisplayTime;

  updateAnchorLocation(time);

  // ...
  // Finish frame loop
  // ...
}

CHK_XR(xrDestroySpatialEntityEXT(spatialAnchorEntity));
```

12.45.10. New Commands

- xrCreateSpatialAnchorEXT

12.45.11. New Structures

- XrSpatialAnchorCreateInfoEXT
- XrSpatialCapabilityConfigurationAnchorEXT
- Extending XrSpatialComponentDataQueryResultEXT:
  - XrSpatialComponentAnchorListEXT

12.45.12. New Enum Constants

- `XR_EXT_SPATIAL_ANCHOR_EXTENSION_NAME`
- `XR_EXT_spatial_anchor_SPEC_VERSION`
- Extending XrSpatialCapabilityEXT:
  - `XR_SPATIAL_CAPABILITY_ANCHOR_EXT`
- Extending XrSpatialComponentTypeEXT:
  - `XR_SPATIAL_COMPONENT_TYPE_ANCHOR_EXT`
- Extending XrStructureType:
  - `XR_TYPE_SPATIAL_ANCHOR_CREATE_INFO_EXT`
  - `XR_TYPE_SPATIAL_CAPABILITY_CONFIGURATION_ANCHOR_EXT`
  - `XR_TYPE_SPATIAL_COMPONENT_ANCHOR_LIST_EXT`

12.45.13. Issues

- Why does xrCreateSpatialAnchorEXT output an entity ID as well as an entity handle?
  - Resolved
  - Answer: The xrCreateSpatialAnchorEXT function could very well have provided just the entity id as the output and applications could create an entity handle for that id using xrCreateSpatialEntityFromIdEXT. However, given the typical usage of an anchor where applications query the anchor pose every frame, it becomes a good candidate to be used in an "update snapshot", which requires entity handles as input. Anticipating this typical usecase, xrCreateSpatialAnchorEXT performs xrCreateSpatialEntityFromIdEXT on behalf of the application and provides it with the entity handle to use with xrCreateSpatialUpdateSnapshotEXT.

12.45.14. Version History

- Revision 1, 2024-07-10 (Nihav Jain, Google)
  - Initial extension description

12.46. XR_EXT_spatial_entity

Name String

`XR_EXT_spatial_entity`

Extension Type

Instance extension

Registered Extension Number

741

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

`XR_EXT_future`

Contributors

Nihav Jain, Google
Jared Finder, Google
Natalie Fleury, Meta
Yuichi Taguchi, Meta
Ron Bessems, Meta
Yin Li, Microsoft
Karthik Kadappan, Magic Leap
Jimmy Alamparambil, ByteDance
Zhipeng Liu, ByteDance
Jun Yan, ByteDance

12.46.1. Overview

This extension introduces the concepts and foundations for scene understanding and spatial reasoning in OpenXR. This unifies several related but distinct areas of functionality, which are enumerated, configured, and interacted with in a broadly uniform way as defined by this extension. As this extension lacks concrete definitions of any one of these functional areas, the formal specification text tends to be somewhat abstract. Examples included in this extension specification text refers at times to functionality defined in a forthcoming or hypothetical related extension for the purpose of illustration, without inherently limiting or specifying such additional functionality.

The broad pieces of this extension are the following:

- Spatial entities: The functionality is centered around *entities*, which provide very little functionality

on their own.

- Spatial components: These entities have *components* associated with them that provide data and behaviors.
- Spatial component types: Each spatial component is of a specific *component type*, and any given *entity* has at most a single *component* of any given *component type*.
- Spatial context: All spatial entity interaction occurs in a *context* after an initialization and configuration phase.
- Spatial capabilities: Spatial entity manipulation is broadly provided by *capabilities*. A capability is some unit of functionality, for example (without limitation) application-defined anchors, plane detection, or image tracking. Each *capability* is typically defined in a separate extension (enabled at instance creation as usual) and is enabled for a specific *context* at the time of creation.
- Each *capability* is associated with a set of *component types* for which *components* are present on every entity exposed by that *capability*. The extension defining a *capability* specifies which *component types* are mandatory for the capability ("guaranteed"), while that same extension or others **may** specify **optional** *component types* provided by some potential implementations. Any number of *capabilities* might provide entities with components of a given *component type*, which are uniformly usable no matter the *capability* that produced it.
- Spatial capability features: Further, some capabilities require configuration, and thus are parameterized by *capability features*.

This extension provides a mechanism for enumerating the *components* provided by each *capability* supported on the current system, both the mandatory and any **optional** *components*.

As some implementations **may** require different degrees of parameterization for *capabilities*, this extension provides a mechanism for enumerating the supported *capability features* associated with a given *capability* in the current system.

This extension also defines several common *components* expected to be used across a wide range of *capabilities*.

12.46.2. Spatial Entity

Spatial entities are entities that exist in some space, that have various associated data organized into components. They **may** be any of the following:

- Physical (e.g. planar surfaces like walls and floors, objects like chairs and bookcases, etc.)
- Virtual (e.g. content placed and shared by another application or user),
- App-defined (e.g. application marking an area as the "living room" or "kitchen", or marking a point as the location to place the TV etc.)

Things which are exposed via the action system, like controllers or eye gaze, are not intended to be modeled as spatial entities.

Spatial entities in OpenXR are modeled as an *Entity-Component system*. Each spatial entity has a set of components, and each component provides a unique set of data and behaviors for that entity.

Spatial entities are represented by either an `XrSpatialEntityIdEXT` atom or an XrSpatialEntityEXT handle, details of which are provided in the Spatial Entity Representations section.

12.46.3. Spatial Component Types

A spatial entity has one or more components which provide data or behaviors for that entity. See Common Components for some common components defined by this extension.

```
// Provided by XR_EXT_spatial_entity
typedef enum XrSpatialComponentTypeEXT {
    XR_SPATIAL_COMPONENT_TYPE_BOUNDED_2D_EXT = 1,
    XR_SPATIAL_COMPONENT_TYPE_BOUNDED_3D_EXT = 2,
    XR_SPATIAL_COMPONENT_TYPE_PARENT_EXT = 3,
    XR_SPATIAL_COMPONENT_TYPE_MESH_3D_EXT = 4,
  // Provided by XR_EXT_spatial_plane_tracking
    XR_SPATIAL_COMPONENT_TYPE_PLANE_ALIGNMENT_EXT = 1000741000,
  // Provided by XR_EXT_spatial_plane_tracking
    XR_SPATIAL_COMPONENT_TYPE_MESH_2D_EXT = 1000741001,
  // Provided by XR_EXT_spatial_plane_tracking
    XR_SPATIAL_COMPONENT_TYPE_POLYGON_2D_EXT = 1000741002,
  // Provided by XR_EXT_spatial_plane_tracking
    XR_SPATIAL_COMPONENT_TYPE_PLANE_SEMANTIC_LABEL_EXT = 1000741003,
  // Provided by XR_EXT_spatial_marker_tracking
    XR_SPATIAL_COMPONENT_TYPE_MARKER_EXT = 1000743000,
  // Provided by XR_EXT_spatial_anchor
    XR_SPATIAL_COMPONENT_TYPE_ANCHOR_EXT = 1000762000,
  // Provided by XR_EXT_spatial_persistence
    XR_SPATIAL_COMPONENT_TYPE_PERSISTENCE_EXT = 1000763000,
  // Provided by XR_ANDROID_spatial_object_tracking
    XR_SPATIAL_COMPONENT_TYPE_OBJECT_SEMANTIC_LABEL_ANDROID = 1000785000,
  // Provided by XR_ANDROID_spatial_discovery_raycast
    XR_SPATIAL_COMPONENT_TYPE_RAYCAST_RESULT_ANDROID = 1000786000,
  // Provided by XR_ANDROID_spatial_component_subsumed_by
    XR_SPATIAL_COMPONENT_TYPE_SUBSUMED_BY_ANDROID = 1000791000,
    XR_SPATIAL_COMPONENT_TYPE_MAX_ENUM_EXT = 0x7FFFFFFF
} XrSpatialComponentTypeEXT;
```

The XrSpatialComponentTypeEXT enumeration identifies the different types of components that the runtime **may** support.

Not all component types listed are provided by this extension on its own: some require additional extensions to be enabled at instance creation time, as documented.

The enumerants have the following values:

Enum	Description
XR_SPATIAL_COMPONENT_TYPE_BOUNDED_2D_EXT	Component that provides the 2D bounds for a spatial entity. Corresponding list structure is XrSpatialComponentBounded2DListEXT; Corresponding data structure is XrSpatialBounded2DDataEXT
XR_SPATIAL_COMPONENT_TYPE_BOUNDED_3D_EXT	Component that provides the 3D bounds for a spatial entity. Corresponding list structure is XrSpatialComponentBounded3DListEXT; Corresponding data structure is XrBoxf
XR_SPATIAL_COMPONENT_TYPE_PARENT_EXT	Component that provides the XrSpatialEntityIdEXT of the parent for a spatial entity. Corresponding list structure is XrSpatialComponentParentListEXT; Corresponding data structure is XrSpatialEntityIdEXT
XR_SPATIAL_COMPONENT_TYPE_MESH_3D_EXT	Component that provides a 3D mesh for a spatial entity. Corresponding list structure is XrSpatialComponentMesh3DListEXT; Corresponding data structure is XrSpatialMeshDataEXT
XR_SPATIAL_COMPONENT_TYPE_PLANE_ALIGNMENT_EXT	Component that provides the plane alignment enum for a spatial entity. Corresponding list structure is XrSpatialComponentPlaneAlignmentListEXT; Corresponding data structure is XrSpatialPlaneAlignmentEXT (Added by the XR_EXT_spatial_plane_tracking extension)
XR_SPATIAL_COMPONENT_TYPE_MESH_2D_EXT	Component that provides a 2D mesh for a spatial entity. Corresponding list structure is XrSpatialComponentMesh2DListEXT; Corresponding data structure is XrSpatialMeshDataEXT (Added by the XR_EXT_spatial_plane_tracking extension)
XR_SPATIAL_COMPONENT_TYPE_POLYGON_2D_EXT	Component that provides a 2D boundary polygon for a spatial entity. Corresponding list structure is XrSpatialComponentPolygon2DListEXT; Corresponding data structure is XrSpatialPolygon2DDataEXT (Added by the XR_EXT_spatial_plane_tracking extension)

Enum	Description
XR_SPATIAL_COMPONENT_TYPE_PLANE_SEMANTIC_LABEL_EXT	Component that provides a semantic label for a plane. Corresponding list structure is XrSpatialComponentPlaneSemanticLabelListEXT; Corresponding data structure is XrSpatialPlaneSemanticLabelEXT (Added by the XR_EXT_spatial_plane_tracking extension)
XR_SPATIAL_COMPONENT_TYPE_MARKER_EXT	A component describing the marker type, id and location. Corresponding list structure is XrSpatialComponentMarkerListEXT; Corresponding data structure is XrSpatialMarkerDataEXT (Added by the XR_EXT_spatial_marker_tracking extension)
XR_SPATIAL_COMPONENT_TYPE_ANCHOR_EXT	Component that provides the location for an anchor. Corresponding list structure is XrSpatialComponentAnchorListEXT; Corresponding data structure is XrPosef (Added by the XR_EXT_spatial_anchor extension)
XR_SPATIAL_COMPONENT_TYPE_PERSISTENCE_EXT	Component that provides the persisted UUID for a spatial entity. Corresponding list structure is XrSpatialComponentPersistenceListEXT; Corresponding data structure is XrSpatialPersistenceDataEXT (Added by the XR_EXT_spatial_persistence extension)
XR_SPATIAL_COMPONENT_TYPE_OBJECT_SEMANTIC_LABEL_ANDROID	Component that provides a semantic label for a object; Corresponding list structure is XrSpatialComponentObjectSemanticLabelListANDROID; Corresponding data structure is XrSpatialObjectSemanticLabelANDROID (Added by the XR_ANDROID_spatial_object_tracking extension)
XR_SPATIAL_COMPONENT_TYPE_RAYCAST_RESULT_ANDROID	Component that provides the pose of a raycast hit on an entity. Corresponding list structure is XrSpatialComponentRaycastResultListANDROID; Corresponding data structure is XrSpatialRaycastResultDataANDROID (Added by the XR_ANDROID_spatial_discovery_raycast extension)

Enum	Description
XR_SPATIAL_COMPONENT_TYPE_SUBSUMED_BY_ANDROID	Component that provides entity ID of the entity subsuming the attached entity. Corresponding list structure is XrSpatialComponentSubsumedByListANDROID; Corresponding data structure is XrSpatialEntityIdEXT (Added by the XR_ANDROID_spatial_component_subsumed_by extension)

12.46.4. Spatial Capabilities and Setup

Spatial capabilities define a runtime's abilities to discover entities that have a guaranteed set of components on them. Applications enable the components of a spatial capability when creating the XrSpatialContextEXT, and the runtime in turn **must** provide only the enabled components on discovered entities. e.g. If a runtime reports that one of the components for a given capability is "semantic labels", it means the application **can** enable semantic labels via the configuration for that capability and the runtime **must** only provide the semantic label component if it is configured.

```
// Provided by XR_EXT_spatial_entity
typedef enum XrSpatialCapabilityEXT {
  // Provided by XR_EXT_spatial_plane_tracking
    XR_SPATIAL_CAPABILITY_PLANE_TRACKING_EXT = 1000741000,
  // Provided by XR_EXT_spatial_marker_tracking
    XR_SPATIAL_CAPABILITY_MARKER_TRACKING_QR_CODE_EXT = 1000743000,
  // Provided by XR_EXT_spatial_marker_tracking
    XR_SPATIAL_CAPABILITY_MARKER_TRACKING_MICRO_QR_CODE_EXT = 1000743001,
  // Provided by XR_EXT_spatial_marker_tracking
    XR_SPATIAL_CAPABILITY_MARKER_TRACKING_ARUCO_MARKER_EXT = 1000743002,
  // Provided by XR_EXT_spatial_marker_tracking
    XR_SPATIAL_CAPABILITY_MARKER_TRACKING_APRIL_TAG_EXT = 1000743003,
  // Provided by XR_EXT_spatial_anchor
    XR_SPATIAL_CAPABILITY_ANCHOR_EXT = 1000762000,
  // Provided by XR_ANDROID_spatial_object_tracking
    XR_SPATIAL_CAPABILITY_OBJECT_TRACKING_ANDROID = 1000785000,
  // Provided by XR_ANDROID_spatial_discovery_raycast
    XR_SPATIAL_CAPABILITY_DEPTH_RAYCAST_ANDROID = 1000786000,
    XR_SPATIAL_CAPABILITY_MAX_ENUM_EXT = 0x7FFFFFFF
} XrSpatialCapabilityEXT;
```

The XrSpatialCapabilityEXT enumeration identifies the different types of capabilities that the runtime **may** support.

The enumerants have the following values:

Enum	Description
XR_SPATIAL_CAPABILITY_PLANE_TRACKING_EXT	Plane tracking (Added by the XR_EXT_spatial_plane_tracking extension)
XR_SPATIAL_CAPABILITY_MARKER_TRACKING_QR_CODE_EXT	Capability to be able to detect and track QR codes. (Added by the XR_EXT_spatial_marker_tracking extension)
XR_SPATIAL_CAPABILITY_MARKER_TRACKING_MICRO_QR_CODE_EXT	Capability to be able to detect and track Micro QR codes. (Added by the XR_EXT_spatial_marker_tracking extension)
XR_SPATIAL_CAPABILITY_MARKER_TRACKING_ARUCO_MARKER_EXT	Capability to be able to detect and track Aruco Markers. (Added by the XR_EXT_spatial_marker_tracking extension)
XR_SPATIAL_CAPABILITY_MARKER_TRACKING_APRIL_TAG_EXT	Capability to be able to detect and track AprilTags. (Added by the XR_EXT_spatial_marker_tracking extension)
XR_SPATIAL_CAPABILITY_ANCHOR_EXT	Capability to be able to create spatial anchors (Added by the XR_EXT_spatial_anchor extension)
XR_SPATIAL_CAPABILITY_OBJECT_TRACKING_ANDROID	Object tracking (Added by the XR_ANDROID_spatial_object_tracking extension)
XR_SPATIAL_CAPABILITY_DEPTH_RAYCAST_ANDROID	Raycast against depth buffer (Added by the XR_ANDROID_spatial_discovery_raycast extension)

The xrEnumerateSpatialCapabilitiesEXT function is defined as:

```
// Provided by XR_EXT_spatial_entity
XrResult xrEnumerateSpatialCapabilitiesEXT(
    XrInstance                                  instance,
    XrSystemId                                  systemId,
    uint32_t                                    capabilityCapacityInput,
    uint32_t*                                   capabilityCountOutput,
    XrSpatialCapabilityEXT*                     capabilities);
```

Parameter Descriptions

- `instance` is a handle to an XrInstance.
- `systemId` is the `XrSystemId` whose spatial capabilities will be enumerated.
- `capabilityCapacityInput` is the capacity of the `capabilities` array, or 0 to indicate a request to retrieve the required capacity.
- `capabilityCountOutput` is the number of capabilities, or the required capacity in the case that `capabilityCapacityInput` is insufficient.
- `capabilities` is an array of XrSpatialCapabilityEXT. It **can** be `NULL` if `capabilityCapacityInput` is 0.
- See the Buffer Size Parameters section for a detailed description of retrieving the required `capabilities` size.

The application **can** enumerate the list of spatial capabilities supported by a given `XrSystemId` using xrEnumerateSpatialCapabilitiesEXT.

The runtime **must** not enumerate the spatial capabilities whose extension is not enabled for `instance`.

Valid Usage (Implicit)

- The `XR_EXT_spatial_entity` extension **must** be enabled prior to calling xrEnumerateSpatialCapabilitiesEXT
- `instance` **must** be a valid XrInstance handle
- `capabilityCountOutput` **must** be a pointer to a `uint32_t` value
- If `capabilityCapacityInput` is not `0`, `capabilities` **must** be a pointer to an array of `capabilityCapacityInput` XrSpatialCapabilityEXT values

Return Codes

Success

- XR_SUCCESS

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SIZE_INSUFFICIENT
- XR_ERROR_SYSTEM_INVALID
- XR_ERROR_VALIDATION_FAILURE

The xrEnumerateSpatialCapabilityComponentTypesEXT function is defined as:

```
// Provided by XR_EXT_spatial_entity
XrResult xrEnumerateSpatialCapabilityComponentTypesEXT(
    XrInstance                                  instance,
    XrSystemId                                  systemId,
    XrSpatialCapabilityEXT                      capability,
    XrSpatialCapabilityComponentTypesEXT*       capabilityComponents);
```

Parameter Descriptions

- instance is a handle to an XrInstance.
- systemId is the XrSystemId whose spatial capability components will be enumerated.
- capability is the XrSpatialCapabilityEXT for which the components will be enumerated.
- capabilityComponents is a pointer to an XrSpatialCapabilityComponentTypesEXT, which is an input/output structure in which an application-allocated array is populated.

This function enumerates the component types that the given capability provides on its entities in the system as configured.

The application **can** use the component types enumerated in XrSpatialCapabilityComponentTypesEXT ::componentTypes to understand the full set of components that the systemId supports for capability and **can** use this list to determine what a valid configuration for capability is when creating an XrSpatialContextEXT for it.

The runtime **must** return XR_ERROR_SPATIAL_CAPABILITY_UNSUPPORTED_EXT if capability is not enumerated by xrEnumerateSpatialCapabilitiesEXT.

The runtime **must** not enumerate the spatial component types whose extension is not enabled for instance.

Valid Usage (Implicit)

- The XR_EXT_spatial_entity extension **must** be enabled prior to calling xrEnumerateSpatialCapabilityComponentTypesEXT
- instance **must** be a valid XrInstance handle
- capability **must** be a valid XrSpatialCapabilityEXT value
- capabilityComponents **must** be a pointer to an XrSpatialCapabilityComponentTypesEXT structure

Return Codes

Success

- XR_SUCCESS

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SIZE_INSUFFICIENT
- XR_ERROR_SPATIAL_CAPABILITY_UNSUPPORTED_EXT
- XR_ERROR_SYSTEM_INVALID
- XR_ERROR_VALIDATION_FAILURE

The XrSpatialCapabilityComponentTypesEXT structure is defined as:

```
// Provided by XR_EXT_spatial_entity
typedef struct XrSpatialCapabilityComponentTypesEXT {
    XrStructureType               type;
    void*                         next;
    uint32_t                      componentTypeCapacityInput;
    uint32_t                      componentTypeCountOutput;
    XrSpatialComponentTypeEXT*    componentTypes;
} XrSpatialCapabilityComponentTypesEXT;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `componentTypeCapacityInput` is the capacity of the array, or 0 to indicate a request to retrieve the required capacity.
- `componentTypeCountOutput` is the number of component types, or the required capacity in the case that `componentTypeCapacityInput` is insufficient.
- `componentTypes` is an array of XrSpatialComponentTypeEXT. It **can** be `NULL` if `componentTypeCapacityInput` is 0.
- See the Buffer Size Parameters section for a detailed description of retrieving the required `componentTypes` size.

Valid Usage (Implicit)

- The `XR_EXT_spatial_entity` extension **must** be enabled prior to using XrSpatialCapabilityComponentTypesEXT
- `type` **must** be `XR_TYPE_SPATIAL_CAPABILITY_COMPONENT_TYPES_EXT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- If `componentTypeCapacityInput` is not `0`, `componentTypes` **must** be a pointer to an array of `componentTypeCapacityInput` XrSpatialComponentTypeEXT values

12.46.5. Spatial capability features

```
// Provided by XR_EXT_spatial_entity
typedef enum XrSpatialCapabilityFeatureEXT {
  // Provided by XR_EXT_spatial_marker_tracking
    XR_SPATIAL_CAPABILITY_FEATURE_MARKER_TRACKING_FIXED_SIZE_MARKERS_EXT = 1000743000,
  // Provided by XR_EXT_spatial_marker_tracking
    XR_SPATIAL_CAPABILITY_FEATURE_MARKER_TRACKING_STATIC_MARKERS_EXT = 1000743001,
  // Provided by XR_ANDROID_spatial_discovery_bounds
    XR_SPATIAL_CAPABILITY_FEATURE_SPHERE_BOUNDS_FILTER_ANDROID = 1000761000,
  // Provided by XR_ANDROID_spatial_discovery_bounds
    XR_SPATIAL_CAPABILITY_FEATURE_BOX_BOUNDS_FILTER_ANDROID = 1000761001,
  // Provided by XR_ANDROID_spatial_discovery_bounds
    XR_SPATIAL_CAPABILITY_FEATURE_FRUSTUM_BOUNDS_FILTER_ANDROID = 1000761002,
    XR_SPATIAL_CAPABILITY_FEATURE_MAX_ENUM_EXT = 0x7FFFFFFF
} XrSpatialCapabilityFeatureEXT;
```

Some capabilities have parameters exposed to the application to configure how the component data is computed by the runtime. These dimensions of parameterization/configurability are known as capability features. E.g. for an image tracking capability, a runtime **may** support a feature for the application to specify whether the tracked images are stationary or not.

Providing this information to the runtime via a configuration structure **must** not change the set of component types present on the associated entities, e.g. on the tracked image. However, the runtime **may** be able to optimize e.g. the tracking abilities of the image tracking capability and provide a better experience to the application.

Such features are represented by XrSpatialCapabilityFeatureEXT and the application enumerates them by using xrEnumerateSpatialCapabilityFeaturesEXT.

Each capability feature has a corresponding configuration structure to enable it. Such configuration structures **must** be chained to XrSpatialCapabilityConfigurationBaseHeaderEXT::`next` of the corresponding capability.

The enumerants have the following values:

Enum	Description
`XR_SPATIAL_CAPABILITY_FEATURE_MARKER_TRACKING_FIXED_SIZE_MARKERS_EXT`	Capability feature to allow applications to specify the size for the markers. Corresponding config structure is XrSpatialMarkerSizeEXT (Added by the `XR_EXT_spatial_marker_tracking` extension)
`XR_SPATIAL_CAPABILITY_FEATURE_MARKER_TRACKING_STATIC_MARKERS_EXT`	Capability feature to allow applications to specify if markers are static. Corresponding config structure is XrSpatialMarkerStaticOptimizationEXT (Added by the `XR_EXT_spatial_marker_tracking` extension)

Enum	Description
XR_SPATIAL_CAPABILITY_FEATURE_SPHERE_BOUNDS_FILTER_ANDROID	The runtime supports XrSpatialBoundsSpherefANDROID filter for XrSpatialDiscoverySnapshotCreateInfoEXT; This does not require any configuration structure to be included during spatial context creation. (Added by the XR_ANDROID_spatial_discovery_bounds extension)
XR_SPATIAL_CAPABILITY_FEATURE_BOX_BOUNDS_FILTER_ANDROID	The runtime supports XrSpatialBoundsBoxfANDROID filter for XrSpatialDiscoverySnapshotCreateInfoEXT; This does not require any configuration structure to be included during spatial context creation. (Added by the XR_ANDROID_spatial_discovery_bounds extension)
XR_SPATIAL_CAPABILITY_FEATURE_FRUSTUM_BOUNDS_FILTER_ANDROID	The runtime supports XrSpatialBoundsFrustumfANDROID filter for XrSpatialDiscoverySnapshotCreateInfoEXT; This does not require any configuration structure to be included during spatial context creation. (Added by the XR_ANDROID_spatial_discovery_bounds extension)

The xrEnumerateSpatialCapabilityFeaturesEXT function is defines as:

```
// Provided by XR_EXT_spatial_entity
XrResult xrEnumerateSpatialCapabilityFeaturesEXT(
    XrInstance                                  instance,
    XrSystemId                                  systemId,
    XrSpatialCapabilityEXT                      capability,
    uint32_t                                    capabilityFeatureCapacityInput,
    uint32_t*                                   capabilityFeatureCountOutput,
    XrSpatialCapabilityFeatureEXT*              capabilityFeatures);
```

Parameter Descriptions

- `instance` is a handle to an XrInstance.
- `systemId` is the `XrSystemId` whose spatial capability features will be enumerated.
- `capability` is the XrSpatialCapabilityEXT for which the features will be enumerated.
- `capabilityFeatureCapacityInput` is the capacity of the array, or 0 to indicate a request to retrieve the required capacity.
- `capabilityFeatureCountOutput` is the number of features, or the required capacity in the case that `capabilityFeatureCapacityInput` is insufficient.
- `capabilityFeatures` is an array of XrSpatialCapabilityFeatureEXT. It **can** be `NULL` if `capabilityFeatureCapacityInput` is 0.
- See the Buffer Size Parameters section for a detailed description of retrieving the required `capabilityFeatures` size.

The application discovers the features supported by a given system for a XrSpatialCapabilityEXT by using xrEnumerateSpatialCapabilityFeaturesEXT.

For capabilities that have features exposed, the application selects the feature or features to enable and provides the corresponding configuration structure in the next chain of the capability configuration structures in XrSpatialContextCreateInfoEXT::`capabilityConfigs`.

If `capability` is not a capability enumerated by xrEnumerateSpatialCapabilitiesEXT, the runtime **must** return `XR_ERROR_SPATIAL_CAPABILITY_UNSUPPORTED_EXT`.

The runtime **must** not enumerate the spatial capability features whose extension is not enabled for `instance`.

Valid Usage (Implicit)

- The `XR_EXT_spatial_entity` extension **must** be enabled prior to calling xrEnumerateSpatialCapabilityFeaturesEXT
- `instance` **must** be a valid XrInstance handle
- `capability` **must** be a valid XrSpatialCapabilityEXT value
- `capabilityFeatureCountOutput` **must** be a pointer to a `uint32_t` value
- If `capabilityFeatureCapacityInput` is not `0`, `capabilityFeatures` **must** be a pointer to an array of `capabilityFeatureCapacityInput` XrSpatialCapabilityFeatureEXT values

Return Codes

Success

- XR_SUCCESS

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SIZE_INSUFFICIENT
- XR_ERROR_SPATIAL_CAPABILITY_UNSUPPORTED_EXT
- XR_ERROR_SYSTEM_INVALID
- XR_ERROR_VALIDATION_FAILURE

12.46.6. Spatial Context

Create a spatial context

```
// Provided by XR_EXT_spatial_entity
XR_DEFINE_HANDLE(XrSpatialContextEXT)
```

The XrSpatialContextEXT handle represents the resources for discovering and updating some number of spatial entities in the environment of the user. Application **can** use this handle to discover and update spatial entities using other functions in this extension.

The xrCreateSpatialContextAsyncEXT function is defined as:

```
// Provided by XR_EXT_spatial_entity
XrResult xrCreateSpatialContextAsyncEXT(
    XrSession                                   session,
    const XrSpatialContextCreateInfoEXT*        createInfo,
    XrFutureEXT*                                future);
```

Parameter Descriptions

- `session` is an XrSession in which the spatial context will be active.
- `createInfo` is the XrSpatialContextCreateInfoEXT used to specify the spatial context parameters.
- `future` is a pointer to an `XrFutureEXT`.

The application **can** create an XrSpatialContextEXT handle by:

- Providing XrSpatialCapabilityConfigurationBaseHeaderEXT derived structures in XrSpatialContextCreateInfoEXT::`capabilityConfigs` to enable capabilities and enable components for that capability.
- Configuring the capabilities themselves with the corresponding configuration structures of its XrSpatialCapabilityFeatureEXT.

The runtime **must** return `XR_ERROR_SPATIAL_CAPABILITY_CONFIGURATION_INVALID_EXT` if XrSpatialContextCreateInfoEXT::capabilityConfigCount is 0. A spatial context handle needs at least one capability.

The runtime **must** return `XR_ERROR_SPATIAL_CAPABILITY_UNSUPPORTED_EXT` if any capability in the XrSpatialContextCreateInfoEXT::`capabilityConfigs` array is not enumerated by xrEnumerateSpatialCapabilitiesEXT.

The runtime **must** return `XR_ERROR_SPATIAL_CAPABILITY_CONFIGURATION_INVALID_EXT` if any XrSpatialCapabilityConfigurationBaseHeaderEXT::`enabledComponentCount` in XrSpatialContextCreateInfoEXT::`capabilityConfigs` is 0. A capability configuration is incomplete without a list of component types to enable for that capability.

The runtime **must** return `XR_ERROR_SPATIAL_COMPONENT_UNSUPPORTED_FOR_CAPABILITY_EXT` if any component type listed in XrSpatialCapabilityConfigurationBaseHeaderEXT::`enabledComponents` is not enumerated for XrSpatialCapabilityConfigurationBaseHeaderEXT::`capability` in xrEnumerateSpatialCapabilityComponentTypesEXT.

If any of the structures in the next chain of XrSpatialContextCreateInfoEXT::`capabilityConfigs` corresponds to an XrSpatialCapabilityFeatureEXT that is not enumerated for that capability in xrEnumerateSpatialCapabilityFeaturesEXT, the runtime **must** ignore that XrSpatialCapabilityFeatureEXT structure.

The runtime **must** return `XR_ERROR_SPATIAL_CAPABILITY_CONFIGURATION_INVALID_EXT` if XrSpatialContextCreateInfoEXT::`capabilityConfigs` contains multiple structures with the same XrSpatialCapabilityConfigurationBaseHeaderEXT::`capability`.

To ensure optimal use of system resources, the runtime **may** use the configurations provided in XrSpatialContextCreateInfoEXT array to prepare itself for spatial requests to come in. For example, a

runtime that supports plane tracking capability **may** only begin its plane tracking pipeline if a spatial context handle containing the plane tracking capability is created by the application. If the configured capabilities have a long warm-up time, calls to xrCreateSpatialDiscoverySnapshotAsyncEXT **may** result in an empty snapshot. Application **can** wait for XrEventDataSpatialDiscoveryRecommendedEXT before using xrCreateSpatialDiscoverySnapshotAsyncEXT to be sure that the underlying tracking services have warmed up.

If a runtime enforces a permission system to control application access to the spatial capabilities being configured for the XrSpatialContextEXT, then the runtime **must** return `XR_ERROR_PERMISSION_INSUFFICIENT` if those permissions have not been granted to this application.

This function starts an asynchronous operation and creates a corresponding `XrFutureEXT`, usable with xrPollFutureEXT and related functions. The return value of this function only indicates whether the parameters were acceptable to schedule the asynchronous operation. The corresponding completion function is xrCreateSpatialContextCompleteEXT, usable when a future from this function is in the READY state, with outputs populated by that function in the completion structure XrCreateSpatialContextCompletionEXT.

Valid Usage (Implicit)

- The `XR_EXT_spatial_entity` extension **must** be enabled prior to calling xrCreateSpatialContextAsyncEXT
- `session` **must** be a valid XrSession handle
- `createInfo` **must** be a pointer to a valid XrSpatialContextCreateInfoEXT structure
- `future` **must** be a pointer to an `XrFutureEXT` value

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_PERMISSION_INSUFFICIENT
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_SPATIAL_CAPABILITY_CONFIGURATION_INVALID_EXT
- XR_ERROR_SPATIAL_CAPABILITY_UNSUPPORTED_EXT
- XR_ERROR_SPATIAL_COMPONENT_UNSUPPORTED_FOR_CAPABILITY_EXT
- XR_ERROR_VALIDATION_FAILURE

The XrSpatialContextCreateInfoEXT structure is defined as:

```
// Provided by XR_EXT_spatial_entity
typedef struct XrSpatialContextCreateInfoEXT {
    XrStructureType                                         type;
    const void*                                             next;
    uint32_t                                                capabilityConfigCount;
    const XrSpatialCapabilityConfigurationBaseHeaderEXT* const*    capabilityConfigs;
} XrSpatialContextCreateInfoEXT;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `capabilityConfigCount` is a `uint32_t` describing the count of elements in the `capabilityConfigs` array.
- `capabilityConfigs` is a pointer to an array of XrSpatialCapabilityConfigurationBaseHeaderEXT pointers.

The XrSpatialContextCreateInfoEXT structure describes the information to create an XrSpatialContextEXT handle.

Valid Usage (Implicit)

- The `XR_EXT_spatial_entity` extension **must** be enabled prior to using XrSpatialContextCreateInfoEXT
- `type` **must** be `XR_TYPE_SPATIAL_CONTEXT_CREATE_INFO_EXT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain. See also: XrSpatialContextPersistenceConfigEXT
- `capabilityConfigs` **must** be a pointer to an array of `capabilityConfigCount` valid XrSpatialCapabilityConfigurationBaseHeaderEXT-based structures. See also: XrSpatialCapabilityConfigurationAnchorEXT, XrSpatialCapabilityConfigurationAprilTagEXT, XrSpatialCapabilityConfigurationArucoMarkerEXT, XrSpatialCapabilityConfigurationDepthRaycastANDROID, XrSpatialCapabilityConfigurationMicroQrCodeEXT, XrSpatialCapabilityConfigurationObjectTrackingANDROID, XrSpatialCapabilityConfigurationPlaneTrackingEXT, XrSpatialCapabilityConfigurationQrCodeEXT
- The `capabilityConfigCount` parameter **must** be greater than `0`

The XrSpatialCapabilityConfigurationBaseHeaderEXT structure is defined as:

```
// Provided by XR_EXT_spatial_entity
typedef struct XrSpatialCapabilityConfigurationBaseHeaderEXT {
    XrStructureType                     type;
    const void*                         next;
    XrSpatialCapabilityEXT              capability;
    uint32_t                            enabledComponentCount;
    const XrSpatialComponentTypeEXT*    enabledComponents;
} XrSpatialCapabilityConfigurationBaseHeaderEXT;
```

This structure is not directly used in the API but instead its child structures **can** be used with XrSpatialContextCreateInfoEXT to configure spatial capabilities.

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `capability` is an XrSpatialCapabilityEXT.
- `enabledComponentCount` is a `uint32_t` describing the count of elements in the `enabledComponents` array.
- `enabledComponents` is a pointer to an array of XrSpatialComponentTypeEXT.

The runtime **must** return `XR_ERROR_SPATIAL_CAPABILITY_UNSUPPORTED_EXT` if `capability` is not enumerated by xrEnumerateSpatialCapabilitiesEXT. The runtime **must** return `XR_ERROR_SPATIAL_COMPONENT_UNSUPPORTED_FOR_CAPABILITY_EXT` if any component type listed in `enabledComponents` is not enumerated for `capability` in xrEnumerateSpatialCapabilityComponentTypesEXT.

Valid Usage (Implicit)

- The `XR_EXT_spatial_entity` extension **must** be enabled prior to using XrSpatialCapabilityConfigurationBaseHeaderEXT
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `capability` **must** be a valid XrSpatialCapabilityEXT value
- `enabledComponents` **must** be a pointer to an array of `enabledComponentCount` valid XrSpatialComponentTypeEXT values
- The `enabledComponentCount` parameter **must** be greater than `0`

The xrCreateSpatialContextCompleteEXT function is defined as:

```
// Provided by XR_EXT_spatial_entity
XrResult xrCreateSpatialContextCompleteEXT(
    XrSession                                   session,
    XrFutureEXT                                 future,
    XrCreateSpatialContextCompletionEXT*        completion);
```

Parameter Descriptions

- `session` is the XrSession previously passed to xrCreateSpatialContextAsyncEXT::`session`.
- `future` is the `XrFutureEXT` received from xrCreateSpatialContextAsyncEXT.
- `completion` is a pointer to an XrCreateSpatialContextCompletionEXT.

xrCreateSpatialContextCompleteEXT completes the asynchronous operation started by xrCreateSpatialContextAsyncEXT. The runtime **must** return `XR_ERROR_FUTURE_PENDING_EXT` if `future` is not in ready state. The runtime **must** return `XR_ERROR_FUTURE_INVALID_EXT` if `future` has already been completed or cancelled.

Valid Usage (Implicit)

- The `XR_EXT_spatial_entity` extension **must** be enabled prior to calling xrCreateSpatialContextCompleteEXT
- `session` **must** be a valid XrSession handle
- `completion` **must** be a pointer to an XrCreateSpatialContextCompletionEXT structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_FUTURE_INVALID_EXT
- XR_ERROR_FUTURE_PENDING_EXT
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The XrCreateSpatialContextCompletionEXT structure is defined as:

```
// Provided by XR_EXT_spatial_entity
typedef struct XrCreateSpatialContextCompletionEXT {
    XrStructureType        type;
    void*                  next;
    XrResult               futureResult;
    XrSpatialContextEXT    spatialContext;
} XrCreateSpatialContextCompletionEXT;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain.
- futureResult is the XrResult of the spatial context creation operation.
- spatialContext is an XrSpatialContextEXT created using the data and configuration in xrCreateSpatialContextAsyncEXT::createInfo.

Future Return Codes

futureResult values:

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_SESSION_LOST
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_SPATIAL_CAPABILITY_CONFIGURATION_INVALID_EXT

If futureResult is a success code, spatialContext **must** be valid. If spatialContext is valid, it remains so only within the lifecycle of xrCreateSpatialContextAsyncEXT::session or until the application destroys the spatialContext with xrDestroySpatialContextEXT, whichever comes first.

Valid Usage (Implicit)

- The XR_EXT_spatial_entity extension **must** be enabled prior to using XrCreateSpatialContextCompletionEXT
- type **must** be XR_TYPE_CREATE_SPATIAL_CONTEXT_COMPLETION_EXT
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- futureResult **must** be a valid XrResult value
- If spatialContext is not XR_NULL_HANDLE, spatialContext **must** be a valid XrSpatialContextEXT handle

Destroy the spatial context

The xrDestroySpatialContextEXT function is defined as:

```
// Provided by XR_EXT_spatial_entity
XrResult xrDestroySpatialContextEXT(
    XrSpatialContextEXT                         spatialContext);
```

Parameter Descriptions

- `spatialContext` is an XrSpatialContextEXT previously created by xrCreateSpatialContextAsyncEXT.

The application **can** call xrDestroySpatialContextEXT function to release the `spatialContext` handle and the underlying resources when finished with spatial entity discovery and update tasks. If there is no other valid XrSpatialContextEXT that was created with the same spatial capabilities as `spatialContext`, this call serves as a suggestion to the runtime to disable the tracking services required for those capabilities to save system resources.

Valid Usage (Implicit)

- The `XR_EXT_spatial_entity` extension **must** be enabled prior to calling xrDestroySpatialContextEXT
- `spatialContext` **must** be a valid XrSpatialContextEXT handle

Thread Safety

- Access to `spatialContext`, and any child handles, **must** be externally synchronized

Return Codes

Success

- `XR_SUCCESS`

Failure

- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_RUNTIME_FAILURE`

12.46.7. Spatial Entity Representations

Spatial Entity ID

```
// Provided by XR_EXT_spatial_entity
XR_DEFINE_ATOM(XrSpatialEntityIdEXT)
```

XrSpatialEntityIdEXT is used to represent any kind of entity discovered by the runtime in the spatial environment of the user. An XrSpatialEntityIdEXT is valid for the XrSpatialContextEXT in which it is discovered, and the runtime **must** not reuse the same XrSpatialEntityIdEXT for different entities within the same XrSpatialContextEXT. Also, the runtime **must** not reuse the same XrSpatialEntityIdEXT across multiple XrSpatialContextEXT within the same XrSession regardless of whether it represents the same entity or different ones.

```
// Provided by XR_EXT_spatial_entity
#define XR_NULL_SPATIAL_ENTITY_ID_EXT 0
```

XR_NULL_SPATIAL_ENTITY_ID_EXT is a reserved value representing an invalid XrSpatialEntityIdEXT. It **may** be passed to and returned from API functions only when specifically allowed.

Spatial Entity Handle

```
// Provided by XR_EXT_spatial_entity
XR_DEFINE_HANDLE(XrSpatialEntityEXT)
```

The XrSpatialEntityEXT handle represents a spatial entity. An application **can** create such a handle to express its interest in a specific entity to the runtime.

Create Spatial Entity Handle from ID

The xrCreateSpatialEntityFromIdEXT function is defined as:

```
// Provided by XR_EXT_spatial_entity
XrResult xrCreateSpatialEntityFromIdEXT(
    XrSpatialContextEXT                         spatialContext,
    const XrSpatialEntityFromIdCreateInfoEXT*   createInfo,
    XrSpatialEntityEXT*                         spatialEntity);
```

Parameter Descriptions

- spatialContext is an XrSpatialContextEXT previously created using xrCreateSpatialContextAsyncEXT.
- createInfo is a pointer to XrSpatialEntityFromIdCreateInfoEXT structure.
- spatialEntity is the returned XrSpatialEntityEXT handle.

The application **can** use xrCreateSpatialEntityFromIdEXT to create an XrSpatialEntityEXT handle which is a reference to an entity that exists in the user's environment.

The runtime **must** return XR_ERROR_SPATIAL_ENTITY_ID_INVALID_EXT if XrSpatialEntityFromIdCreateInfoEXT::entityId is not a valid ID for spatialContext.

Valid Usage (Implicit)

- The XR_EXT_spatial_entity extension **must** be enabled prior to calling xrCreateSpatialEntityFromIdEXT
- spatialContext **must** be a valid XrSpatialContextEXT handle
- createInfo **must** be a pointer to a valid XrSpatialEntityFromIdCreateInfoEXT structure
- spatialEntity **must** be a pointer to an XrSpatialEntityEXT handle

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_SPATIAL_ENTITY_ID_INVALID_EXT
- XR_ERROR_VALIDATION_FAILURE

The XrSpatialEntityFromIdCreateInfoEXT structure is defined as:

```
// Provided by XR_EXT_spatial_entity
typedef struct XrSpatialEntityFromIdCreateInfoEXT {
    XrStructureType         type;
    const void*             next;
    XrSpatialEntityIdEXT    entityId;
} XrSpatialEntityFromIdCreateInfoEXT;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `entityId` is the `XrSpatialEntityIdEXT` of the entity that the application wants to create a handle for.

Valid Usage (Implicit)

- The `XR_EXT_spatial_entity` extension **must** be enabled prior to using XrSpatialEntityFromIdCreateInfoEXT
- `type` **must** be `XR_TYPE_SPATIAL_ENTITY_FROM_ID_CREATE_INFO_EXT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

Destroy Spatial Entity Handle

The xrDestroySpatialEntityEXT function is defined as:

```
// Provided by XR_EXT_spatial_entity
XrResult xrDestroySpatialEntityEXT(
    XrSpatialEntityEXT                          spatialEntity);
```

Parameter Descriptions

- `spatialEntity` is a handle to an XrSpatialEntityEXT previously created by a function such as xrCreateSpatialEntityFromIdEXT.

The application **can** use xrDestroySpatialEntityEXT to release the `spatialEntity` handle when it is no longer interested in the entity referenced by this handle.

Valid Usage (Implicit)

- The `XR_EXT_spatial_entity` extension **must** be enabled prior to calling xrDestroySpatialEntityEXT
- `spatialEntity` **must** be a valid XrSpatialEntityEXT handle

Thread Safety

- Access to `spatialEntity`, and any child handles, **must** be externally synchronized

Return Codes

Success

- `XR_SUCCESS`

Failure

- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_RUNTIME_FAILURE`

12.46.8. Spatial Snapshot

```
// Provided by XR_EXT_spatial_entity
XR_DEFINE_HANDLE(XrSpatialSnapshotEXT)
```

The application **can** create spatial snapshots for the purpose of discovering spatial entities or for updating its information about known spatial entities. The XrSpatialSnapshotEXT handle represents the immutable data for the discovered or updated spatial entities and a subset of their components as selected by the application. The spatial snapshot represents a coherent view of the entities and their component data. Once a snapshot is created, the snapshot's data **must** remain constant while the snapshot is valid.

The application **can** create any number of snapshots it wants but **must** be mindful of the memory being allocated for each new snapshot and **must** destroy the snapshots once it no longer needs them.

Create discovery snapshot

The xrCreateSpatialDiscoverySnapshotAsyncEXT function is defined as:

```
// Provided by XR_EXT_spatial_entity
XrResult xrCreateSpatialDiscoverySnapshotAsyncEXT(
    XrSpatialContextEXT                           spatialContext,
    const XrSpatialDiscoverySnapshotCreateInfoEXT* createInfo,
    XrFutureEXT*                                  future);
```

Parameter Descriptions

- `spatialContext` is an XrSpatialContextEXT previously created by using xrCreateSpatialContextAsyncEXT.
- `createInfo` is a pointer to an XrSpatialDiscoverySnapshotCreateInfoEXT.
- `future` is a pointer to an `XrFutureEXT`.

The application **can** discover spatial entities by creating a discovery snapshot by using xrCreateSpatialDiscoverySnapshotAsyncEXT.

This function starts an asynchronous operation and creates a corresponding `XrFutureEXT`, usable with xrPollFutureEXT and related functions. The return value of this function only indicates whether the parameters were acceptable to schedule the asynchronous operation. The corresponding completion function is xrCreateSpatialDiscoverySnapshotCompleteEXT, usable when a future from this function is in the READY state, with outputs populated by that function in the completion structure XrCreateSpatialDiscoverySnapshotCompletionEXT.

The application **can** submit multiple discovery snapshot creation requests without needing to wait for the previous one to be completed. The runtime **may** process and complete the snapshot creation in any order. The runtime **may** delay the completion of the discovery snapshot creation to throttle the application if it needs to reduce the use of system resources due to power, thermal or other policies of the device.

The application **can** use XrSpatialDiscoverySnapshotCreateInfoEXT::`componentTypes` to filter the list of entities and the components whose data the runtime **must** include in the snapshot. If the application provides a valid list of spatial component types in XrSpatialDiscoverySnapshotCreateInfoEXT::`componentTypes`, then the runtime **must** only include spatial entities in the snapshot that have at least one of the components provided in XrSpatialDiscoverySnapshotCreateInfoEXT::`componentTypes`. Also, the runtime **must** only include data for only those components in the snapshot.

The runtime **must** return `XR_ERROR_SPATIAL_COMPONENT_NOT_ENABLED_EXT` if any of the XrSpatialComponentTypeEXT in XrSpatialDiscoverySnapshotCreateInfoEXT::`componentTypes` are not enabled for the spatial capabilities passed to XrSpatialContextCreateInfoEXT::`capabilityConfigs` when creating `spatialContext`.

If the application does not provide a list of spatial component types in XrSpatialDiscoverySnapshotCreateInfoEXT::`componentTypes`, the runtime **must** include all the spatial entities in the snapshot that have the set of components which are enumerated in XrSpatialCapabilityConfigurationBaseHeaderEXT::enabledComponents for the capabilities configured for `spatialContext`. The runtime **must** include the data for all the enabled components of the capabilities configured for `spatialContext`.

If XrEventDataReferenceSpaceChangePending is queued before the completion of `future`, and XrEventDataReferenceSpaceChangePending::`poseValid` is false, then the runtime **may** either create an

XrSpatialSnapshotEXT that has no entities in it or set the XrSpatialEntityTrackingStateEXT of the entities that are no longer locatable in XrCreateSpatialDiscoverySnapshotCompletionInfoEXT::baseSpace at XrCreateSpatialDiscoverySnapshotCompletionInfoEXT::time to XR_SPATIAL_ENTITY_TRACKING_STATE_PAUSED_EXT or XR_SPATIAL_ENTITY_TRACKING_STATE_STOPPED_EXT. The runtime **must** not set XrCreateSpatialContextCompletionEXT::futureResult to an error code because of XrEventDataReferenceSpaceChangePending.

Valid Usage (Implicit)

- The XR_EXT_spatial_entity extension **must** be enabled prior to calling xrCreateSpatialDiscoverySnapshotAsyncEXT
- spatialContext **must** be a valid XrSpatialContextEXT handle
- createInfo **must** be a pointer to a valid XrSpatialDiscoverySnapshotCreateInfoEXT structure
- future **must** be a pointer to an XrFutureEXT value

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_SPATIAL_COMPONENT_NOT_ENABLED_EXT
- XR_ERROR_VALIDATION_FAILURE

The XrSpatialDiscoverySnapshotCreateInfoEXT structure is defined as:

```
// Provided by XR_EXT_spatial_entity
typedef struct XrSpatialDiscoverySnapshotCreateInfoEXT {
    XrStructureType                    type;
    const void*                        next;
    uint32_t                           componentTypeCount;
    const XrSpatialComponentTypeEXT*   componentTypes;
} XrSpatialDiscoverySnapshotCreateInfoEXT;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `componentTypeCount` is a `uint32_t` describing the count of elements in the `componentTypes` array.
- `componentTypes` is an array of XrSpatialComponentTypeEXT.

The XrSpatialDiscoverySnapshotCreateInfoEXT structure describes the information to create an XrSpatialSnapshotEXT handle when discovering spatial entities.

Valid Usage (Implicit)

- The `XR_EXT_spatial_entity` extension **must** be enabled prior to using XrSpatialDiscoverySnapshotCreateInfoEXT
- `type` **must** be `XR_TYPE_SPATIAL_DISCOVERY_SNAPSHOT_CREATE_INFO_EXT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain. See also: XrSpatialBoundsBoxfANDROID, XrSpatialBoundsFrustumfANDROID, XrSpatialBoundsSpherefANDROID, XrSpatialDiscoveryPersistenceUuidFilterEXT, XrSpatialDiscoveryUniqueEntitiesFilterANDROID, XrSpatialFilterTrackingStateEXT, XrSpatialRaycastInfoANDROID
- If `componentTypeCount` is not `0`, `componentTypes` **must** be a pointer to an array of `componentTypeCount` valid XrSpatialComponentTypeEXT values

The xrCreateSpatialDiscoverySnapshotCompleteEXT function is defined as:

```
// Provided by XR_EXT_spatial_entity
XrResult xrCreateSpatialDiscoverySnapshotCompleteEXT(
    XrSpatialContextEXT                         spatialContext,
    const XrCreateSpatialDiscoverySnapshotCompletionInfoEXT*
createSnapshotCompletionInfo,
    XrCreateSpatialDiscoverySnapshotCompletionEXT* completion);
```

Parameter Descriptions

- `spatialContext` is the XrSpatialContextEXT previously passed to xrCreateSpatialDiscoverySnapshotAsyncEXT::`spatialContext`.
- `createSnapshotCompletionInfo` is a pointer to an XrCreateSpatialDiscoverySnapshotCompletionInfoEXT which provides info about the discovery snapshot creation request completion.
- `completion` is a pointer to an XrCreateSpatialDiscoverySnapshotCompletionEXT.

xrCreateSpatialDiscoverySnapshotCompleteEXT completes the asynchronous operation started by xrCreateSpatialDiscoverySnapshotAsyncEXT. The runtime **must** return `XR_ERROR_FUTURE_PENDING_EXT` if XrCreateSpatialDiscoverySnapshotCompletionInfoEXT::`future` is not in ready state. The runtime **must** return `XR_ERROR_FUTURE_INVALID_EXT` if XrCreateSpatialDiscoverySnapshotCompletionInfoEXT::`future` has already been completed or cancelled.

Valid Usage (Implicit)

- The `XR_EXT_spatial_entity` extension **must** be enabled prior to calling xrCreateSpatialDiscoverySnapshotCompleteEXT
- `spatialContext` **must** be a valid XrSpatialContextEXT handle
- `createSnapshotCompletionInfo` **must** be a pointer to a valid XrCreateSpatialDiscoverySnapshotCompletionInfoEXT structure
- `completion` **must** be a pointer to an XrCreateSpatialDiscoverySnapshotCompletionEXT structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_FUTURE_INVALID_EXT
- XR_ERROR_FUTURE_PENDING_EXT
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_TIME_INVALID
- XR_ERROR_VALIDATION_FAILURE

The XrCreateSpatialDiscoverySnapshotCompletionInfoEXT structure is defined as:

```
// Provided by XR_EXT_spatial_entity
typedef struct XrCreateSpatialDiscoverySnapshotCompletionInfoEXT {
    XrStructureType    type;
    const void*        next;
    XrSpace            baseSpace;
    XrTime             time;
    XrFutureEXT        future;
} XrCreateSpatialDiscoverySnapshotCompletionInfoEXT;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `baseSpace` is the XrSpace in which all the locations of the discovery XrSpatialSnapshotEXT will be located.
- `time` is the `XrTime` at which all the locations of the discovery XrSpatialSnapshotEXT will be located.
- `future` is the `XrFutureEXT` received from xrCreateSpatialDiscoverySnapshotAsyncEXT

The locations in the various component data included in the created snapshot will be represented in `baseSpace`, located at `time`.

Valid Usage (Implicit)

- The `XR_EXT_spatial_entity` extension **must** be enabled prior to using XrCreateSpatialDiscoverySnapshotCompletionInfoEXT
- `type` **must** be `XR_TYPE_CREATE_SPATIAL_DISCOVERY_SNAPSHOT_COMPLETION_INFO_EXT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `baseSpace` **must** be a valid XrSpace handle

The XrCreateSpatialDiscoverySnapshotCompletionEXT structure is defined as:

```
// Provided by XR_EXT_spatial_entity
typedef struct XrCreateSpatialDiscoverySnapshotCompletionEXT {
    XrStructureType         type;
    void*                   next;
    XrResult                futureResult;
    XrSpatialSnapshotEXT    snapshot;
} XrCreateSpatialDiscoverySnapshotCompletionEXT;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `futureResult` is the XrResult of the spatial discovery snapshot creation operation.
- `snapshot` is an XrSpatialSnapshotEXT which **can** be used to query the component data of the discovered spatial entities.

Future Return Codes

`futureResult` values:

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_OUT_OF_MEMORY`
- `XR_ERROR_LIMIT_REACHED`

Valid Usage (Implicit)

- The `XR_EXT_spatial_entity` extension **must** be enabled prior to using XrCreateSpatialDiscoverySnapshotCompletionEXT
- `type` **must** be `XR_TYPE_CREATE_SPATIAL_DISCOVERY_SNAPSHOT_COMPLETION_EXT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `futureResult` **must** be a valid XrResult value
- If `snapshot` is not XR_NULL_HANDLE, `snapshot` **must** be a valid XrSpatialSnapshotEXT handle

Discovery Recommendation Event

The XrEventDataSpatialDiscoveryRecommendedEXT structure is defined as:

```
// Provided by XR_EXT_spatial_entity
typedef struct XrEventDataSpatialDiscoveryRecommendedEXT {
    XrStructureType        type;
    const void*            next;
    XrSpatialContextEXT    spatialContext;
} XrEventDataSpatialDiscoveryRecommendedEXT;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `spatialContext` is the XrSpatialContextEXT for which discovery is being recommended by the runtime.

The application **can** retrieve this event by using xrPollEvent. The application **can** avoid excessive calls to xrCreateSpatialDiscoverySnapshotAsyncEXT to discover spatial entities by waiting for this event. If the application creates multiple discovery snapshots with the same XrSpatialDiscoverySnapshotCreateInfoEXT between two XrEventDataSpatialDiscoveryRecommendedEXT events, the resultant snapshots **may** contain the same entities and therefore the snapshot creation and data queries would be wasteful.

Waiting for this event to create a new discovery snapshot ensures that the application is not overloading the system with discovery requests for which the runtime **may** not return any new data and helps avoid the risk of overusing the system resources, and getting throttled due to power or thermal policies of the device. This also helps create parity between runtimes that are discovering spatial entities on the fly with live tracking and runtimes which are providing spatial entities off of a previously recorded state (where the runtime **may** queue the discovery recommendation event only once for each XrSpatialContextEXT).

The runtime **must** not queue this event for notifying the application about changes or adjustments made to the component data of existing spatial entities. The application **can** use the xrCreateSpatialUpdateSnapshotEXT to keep track of component data updates for the spatial entities it is interested in.

A runtime **may** queue a discovery recommendation event without waiting for the application to first call xrCreateSpatialDiscoverySnapshotAsyncEXT. For example, a runtime **may** base the decision of queueing the discovery recommendation event on the configuration of the XrSpatialContextEXT, its own understanding of the environment around the user (discovery of new entities or loss of existing ones), or for hinting an appropriate discovery request cadence to the application so as not to overload the system resources. The runtime **may** choose to never queue this event for an XrSpatialContextEXT if no entities are found in the user's environment throughout the lifetime of that XrSpatialContextEXT.

The runtime **must** not queue this event for a given `spatialContext` until the application completes its

creation by using xrCreateSpatialContextCompleteEXT.

After the application calls xrDestroySpatialContextEXT, the runtime **must** not queue any more discovery recommendation events for that spatial context nor return any such events for that context from xrPollEvent.

Valid Usage (Implicit)

- The XR_EXT_spatial_entity extension **must** be enabled prior to using XrEventDataSpatialDiscoveryRecommendedEXT
- `type` **must** be `XR_TYPE_EVENT_DATA_SPATIAL_DISCOVERY_RECOMMENDED_EXT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

Query Component Data

The xrQuerySpatialComponentDataEXT function is defined as:

```
// Provided by XR_EXT_spatial_entity
XrResult xrQuerySpatialComponentDataEXT(
    XrSpatialSnapshotEXT                          snapshot,
    const XrSpatialComponentDataQueryConditionEXT* queryCondition,
    XrSpatialComponentDataQueryResultEXT*         queryResult);
```

Parameter Descriptions

- `snapshot` is an XrSpatialSnapshotEXT previously provided by xrCreateSpatialDiscoverySnapshotCompleteEXT or xrCreateSpatialUpdateSnapshotEXT.
- `queryCondition` is a pointer to an XrSpatialComponentDataQueryConditionEXT structure.
- `queryResult` is a pointer to an XrSpatialComponentDataQueryResultEXT structure.

The application **can** use xrQuerySpatialComponentDataEXT to query the component data of the entities in the snapshot by attaching a list structure to XrSpatialComponentDataQueryResultEXT::`next` corresponding to each XrSpatialComponentTypeEXT in XrSpatialComponentDataQueryConditionEXT::`componentTypes`.

If the application attaches a list structure to XrSpatialComponentDataQueryResultEXT::`next` that does not correspond to any of the components listed in XrSpatialComponentDataQueryConditionEXT::`componentTypes`, the runtime **must** return `XR_ERROR_VALIDATION_FAILURE`.

The application **can** choose to attach the list structures corresponding to only a subset of components listed in XrSpatialComponentDataQueryConditionEXT::`componentTypes`. The application **can** choose to

omit the list structures altogether if it only wishes to know the ids and tracking state of the spatial entities that satisfy the `queryCondition`. The runtime **must** not treat the absence of list structures from the XrSpatialComponentDataQueryResultEXT::`next` chain as a failure.

If XrEventDataReferenceSpaceChangePending is queued and XrEventDataReferenceSpaceChangePending::`changeTime` elapsed while the application is querying component data from an XrSpatialSnapshotEXT, the application **may** use the event data to adjust the poses accordingly.

The runtime **must** populate XrSpatialComponentDataQueryResultEXT::`entityIds` only with entities that have all the components specified in XrSpatialComponentDataQueryConditionEXT::`componentTypes`. If XrSpatialComponentDataQueryConditionEXT::`componentTypeCount` is 0, the runtime **must** populate `queryResult` with all the entities (and their tracking states) that are in the snapshot. If additional query conditions are added to XrSpatialComponentDataQueryConditionEXT::`next`, the runtime **must** treat those as an "AND" with the component types availability i.e. the runtime **must** populate XrSpatialComponentDataQueryResultEXT::entityIds only with entities that satisfy all of the provided conditions. The runtime **must** populate the component data in the list structures in the same order as the entities in XrSpatialComponentDataQueryResultEXT::`entityIds` i.e. the component data at a given index in the list structure array **must** correspond to the entity at the same index.

If the tracking state for an entity is not `XR_SPATIAL_ENTITY_TRACKING_STATE_TRACKING_EXT`, the runtime **must** not change the data at the index corresponding to that entity in the array contained in the list structures attached to XrSpatialComponentDataQueryResultEXT.

Valid Usage (Implicit)

- The `XR_EXT_spatial_entity` extension **must** be enabled prior to calling xrQuerySpatialComponentDataEXT
- `snapshot` **must** be a valid XrSpatialSnapshotEXT handle
- `queryCondition` **must** be a pointer to a valid XrSpatialComponentDataQueryConditionEXT structure
- `queryResult` **must** be a pointer to an XrSpatialComponentDataQueryResultEXT structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_SIZE_INSUFFICIENT
- XR_ERROR_VALIDATION_FAILURE

As an example the application creates an XrSpatialSnapshotEXT which contains 5 entities, where -

- Entity 1 and 2 have components XR_SPATIAL_COMPONENT_TYPE_BOUNDED_2D_EXT and XR_SPATIAL_COMPONENT_TYPE_PARENT_EXT
- Entity 3 and 4 have components XR_SPATIAL_COMPONENT_TYPE_BOUNDED_3D_EXT and XR_SPATIAL_COMPONENT_TYPE_MESH_3D_EXT
- Entity 5 has components XR_SPATIAL_COMPONENT_TYPE_BOUNDED_2D_EXT and XR_SPATIAL_COMPONENT_TYPE_MESH_3D_EXT.

XrSpatialEntityIdEXT	1	2	3	4	5
Bounded2D	Yes	Yes	No	No	Yes
Parent	Yes	Yes	No	No	No
Mesh3D	No	No	Yes	Yes	Yes
Bounded3D	No	No	Yes	Yes	No

Text is not SVG - cannot display

Figure 17. Example snapshot

xrQuerySpatialComponentDataEXT on the above snapshot with XR_SPATIAL_COMPONENT_TYPE_BOUNDED_2D_EXT listed in the query condition will result in entity #1, #2, and #5 being returned to the application and the application **can** attach an array of

XrSpatialBounded2DDataEXT as part of the XrSpatialComponentBounded2DListEXT structure to the next chain of XrSpatialComponentDataQueryResultEXT to get the bounded2D data.

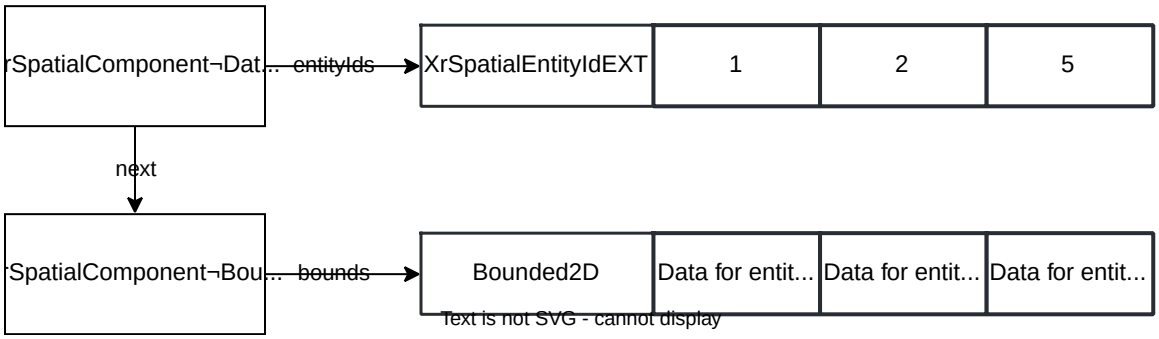


Figure 18. Example query result

xrQuerySpatialComponentDataEXT on the above snapshot with XR_SPATIAL_COMPONENT_TYPE_BOUNDED_3D_EXT and XR_SPATIAL_COMPONENT_TYPE_MESH_3D_EXT components listed in the query condition will result in entity #3 and #4 being returned to the application and the application **can** attach arrays of XrBoxf and XrSpatialMeshDataEXT as part of the XrSpatialComponentBounded3DListEXT and XrSpatialComponentMesh3DListEXT structures respectively to the next chain of XrSpatialComponentDataQueryResultEXT to get the component data.

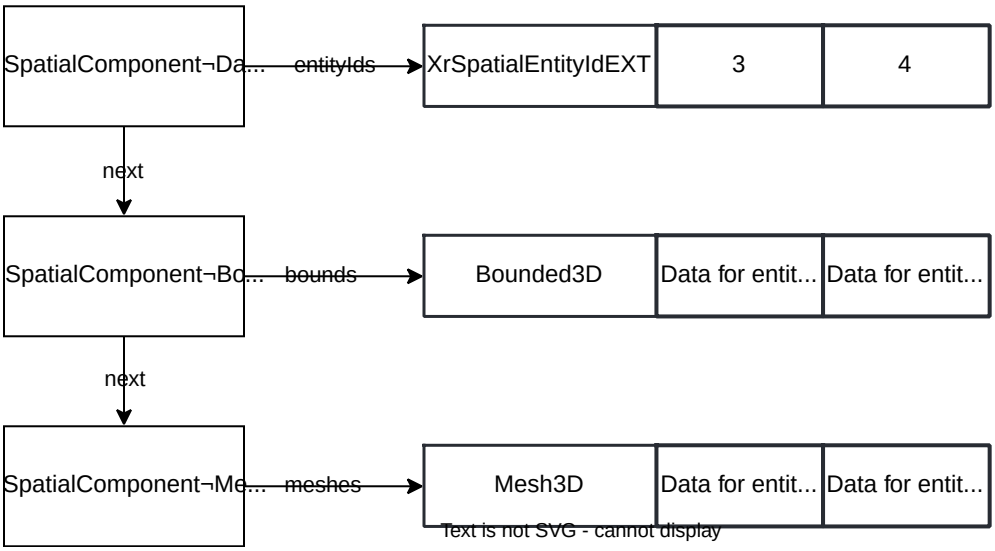


Figure 19. Example query result

The XrSpatialComponentDataQueryConditionEXT structure is defined as:

```
// Provided by XR_EXT_spatial_entity
typedef struct XrSpatialComponentDataQueryConditionEXT {
    XrStructureType                    type;
    const void*                        next;
    uint32_t                           componentTypeCount;
    const XrSpatialComponentTypeEXT*   componentTypes;
} XrSpatialComponentDataQueryConditionEXT;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `componentTypeCount` is a `uint32_t` describing the count of elements in the `componentTypes` array.
- `componentTypes` is an array of XrSpatialComponentTypeEXT for which to get the data from the snapshot.

Valid Usage (Implicit)

- The `XR_EXT_spatial_entity` extension **must** be enabled prior to using XrSpatialComponentDataQueryConditionEXT
- `type` **must** be `XR_TYPE_SPATIAL_COMPONENT_DATA_QUERY_CONDITION_EXT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain. See also: XrSpatialDiscoveryPersistenceUuidFilterEXT, XrSpatialFilterTrackingStateEXT
- If `componentTypeCount` is not `0`, `componentTypes` **must** be a pointer to an array of `componentTypeCount` valid XrSpatialComponentTypeEXT values

The XrSpatialComponentDataQueryResultEXT structure is defined as:

```
// Provided by XR_EXT_spatial_entity
typedef struct XrSpatialComponentDataQueryResultEXT {
    XrStructureType                     type;
    void*                               next;
    uint32_t                            entityIdCapacityInput;
    uint32_t                            entityIdCountOutput;
    XrSpatialEntityIdEXT*               entityIds;
    uint32_t                            entityStateCapacityInput;
    uint32_t                            entityStateCountOutput;
    XrSpatialEntityTrackingStateEXT*    entityStates;
} XrSpatialComponentDataQueryResultEXT;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `entityIdCapacityInput` is the capacity of the `entityIds` array, or 0 to indicate a request to retrieve the required capacity.
- `entityIdCountOutput` is the number of `XrSpatialEntityIdEXT` in `entityIds`, or the required capacity in the case that `entityIdCapacityInput` is insufficient.
- `entityIds` is an array of `XrSpatialEntityIdEXT`. It **can** be `NULL` if `entityIdCapacityInput` is 0.
- `entityStateCapacityInput` is the capacity of the `entityStates` array, or 0 to indicate a request to retrieve the required capacity.
- `entityStateCountOutput` is the number of XrSpatialEntityTrackingStateEXT in `entityStates`, or the required capacity in the case that `entityStateCapacityInput` is insufficient. This **must** always be the same as `entityIdCountOutput`.
- `entityStates` is an array of XrSpatialEntityTrackingStateEXT. It **can** be `NULL` if `entityStateCapacityInput` is 0.
- See the Buffer Size Parameters section for a detailed description of retrieving the required `entityIds` size.

An application **can** use the `entityIds` with xrCreateSpatialEntityFromIdEXT to create XrSpatialEntityEXT handles for the entities it is interested in getting regular updates for. The application **can** then use these XrSpatialEntityEXT handles with xrCreateSpatialUpdateSnapshotEXT to create an update snapshot that has the runtime's latest known data of the components for the provided entities.

Valid Usage (Implicit)

- The XR_EXT_spatial_entity extension **must** be enabled prior to using XrSpatialComponentDataQueryResultEXT
- type **must** be XR_TYPE_SPATIAL_COMPONENT_DATA_QUERY_RESULT_EXT
- next **must** be NULL or a valid pointer to the next structure in a structure chain. See also: XrSpatialComponentAnchorListEXT, XrSpatialComponentBounded2DListEXT, XrSpatialComponentBounded3DListEXT, XrSpatialComponentMarkerListEXT, XrSpatialComponentMesh2DListEXT, XrSpatialComponentMesh3DListEXT, XrSpatialComponentObjectSemanticLabelListANDROID, XrSpatialComponentParentListEXT, XrSpatialComponentPersistenceListEXT, XrSpatialComponentPlaneAlignmentListEXT, XrSpatialComponentPlaneSemanticLabelListEXT, XrSpatialComponentPolygon2DListEXT, XrSpatialComponentRaycastResultListANDROID, XrSpatialComponentSubsumedByListANDROID
- If entityIdCapacityInput is not 0, entityIds **must** be a pointer to an array of entityIdCapacityInput XrSpatialEntityIdEXT values
- If entityStateCapacityInput is not 0, entityStates **must** be a pointer to an array of entityStateCapacityInput XrSpatialEntityTrackingStateEXT values

```
// Provided by XR_EXT_spatial_entity
typedef enum XrSpatialEntityTrackingStateEXT {
    XR_SPATIAL_ENTITY_TRACKING_STATE_STOPPED_EXT = 1,
    XR_SPATIAL_ENTITY_TRACKING_STATE_PAUSED_EXT = 2,
    XR_SPATIAL_ENTITY_TRACKING_STATE_TRACKING_EXT = 3,
    XR_SPATIAL_ENTITY_TRACKING_STATE_MAX_ENUM_EXT = 0x7FFFFFFF
} XrSpatialEntityTrackingStateEXT;
```

The XrSpatialEntityTrackingStateEXT enumerates the possible spatial entity tracking states:

The enums have the following meanings:

Enum	Description
XR_SPATIAL_ENTITY_TRACKING_STATE_STOPPED_EXT	The runtime has stopped tracking this entity and will never resume tracking it.
XR_SPATIAL_ENTITY_TRACKING_STATE_PAUSED_EXT	The runtime has paused tracking this entity but **may** resume tracking it in the future.
XR_SPATIAL_ENTITY_TRACKING_STATE_TRACKING_EXT	The runtime is currently tracking this entity and its component data is valid.

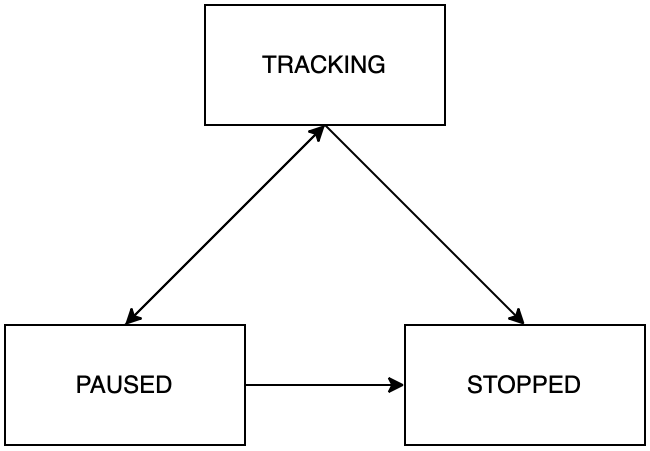


Figure 20. XrSpatialEntityTrackingStateEXT

- The runtime **may** change the state of the spatial entity from `XR_SPATIAL_ENTITY_TRACKING_STATE_TRACKING_EXT` to `XR_SPATIAL_ENTITY_TRACKING_STATE_PAUSED_EXT` if it suspends the tracking of that spatial entity but has the possibility of resuming its tracking in the future. Some examples of when the runtime **may** do this include (but not limited to) if the application loses input focus; or if the given spatial entity is too far from the user to be accurately tracked; or if there are too many entities being tracked and the runtime wants to reduce the cost of tracking. XrSpatialEntityTrackingStateEXT helps the application insulate itself from the different tracking policies of each runtime.
- The runtime **may** change the state of an entity from `XR_SPATIAL_ENTITY_TRACKING_STATE_PAUSED_EXT` to `XR_SPATIAL_ENTITY_TRACKING_STATE_TRACKING_EXT` or `XR_SPATIAL_ENTITY_TRACKING_STATE_STOPPED_EXT`.
- The runtime **must** change the state of the spatial entity from `XR_SPATIAL_ENTITY_TRACKING_STATE_TRACKING_EXT` or `XR_SPATIAL_ENTITY_TRACKING_STATE_PAUSED_EXT` to `XR_SPATIAL_ENTITY_TRACKING_STATE_STOPPED_EXT` if the spatial entity is lost and its tracking will never be recovered or resumed. An example of such a case would be if the device loses tracking, restarts its tracking session but is unable to relocalize in its environment, and therefore treats discovered entities of this tracking session as new entities.
- Once the tracking state of an entity is set to `XR_SPATIAL_ENTITY_TRACKING_STATE_STOPPED_EXT`, the runtime **must** never change it any other state.
- When querying the component data of a spatial entity using xrQuerySpatialComponentDataEXT, the runtime **must** set valid data in the contents of the buffers provided by the application in the next chain of XrSpatialComponentDataQueryResultEXT if the entity state is `XR_SPATIAL_ENTITY_TRACKING_STATE_TRACKING_EXT`. If the entity state is `XR_SPATIAL_ENTITY_TRACKING_STATE_STOPPED_EXT` or `XR_SPATIAL_ENTITY_TRACKING_STATE_PAUSED_EXT`, the runtime **must** not change the content of the buffers.

Two-call idiom for component data

The XrSpatialBufferEXT structure is defined as:

```
// Provided by XR_EXT_spatial_entity
typedef struct XrSpatialBufferEXT {
    XrSpatialBufferIdEXT      bufferId;
    XrSpatialBufferTypeEXT    bufferType;
} XrSpatialBufferEXT;
```

Member Descriptions

- `bufferId` the `XrSpatialBufferIdEXT` of the buffer data.
- `bufferType` is the `XrSpatialBufferTypeEXT` to indicate the type of data in `bufferId`. The application **can** use `bufferType` to determine which function to use to retrieve the actual data of the buffer.

Some spatial components have variable-sized data and therefore require using the two-call idiom to retrieve their data. In such cases, the spatial component data structure provides an `XrSpatialBufferEXT` for each variable sized buffer needed in that component's data.

For the same `bufferId`, the runtime **must** provide the same data from one component data query to another, even across one snapshot to another. A different `bufferId` between component data query calls indicates to the application that the data for that component **may** have changed.

Valid Usage (Implicit)

- The `XR_EXT_spatial_entity` extension **must** be enabled prior to using `XrSpatialBufferEXT`
- `bufferType` **must** be a valid `XrSpatialBufferTypeEXT` value

```
// Provided by XR_EXT_spatial_entity
XR_DEFINE_ATOM(XrSpatialBufferIdEXT)
```

`XrSpatialBufferIdEXT` is used to represent any kind of variable sized data for a spatial component.

The runtime **must** keep the `XrSpatialBufferIdEXT` and its data in memory for at least the lifecycle of the `XrSpatialSnapshotEXT` that contains it. The runtime **may** keep the `XrSpatialBufferIdEXT` and its data in memory for longer than the lifecycle of the `XrSpatialSnapshotEXT` in order to return the same ID as part of snapshots created later on by the application. For the same `XrSpatialBufferIdEXT`, the runtime **must** always return the same data via the appropriate `xrGetSpatialBuffer*` function.

```
// Provided by XR_EXT_spatial_entity
typedef enum XrSpatialBufferTypeEXT {
    XR_SPATIAL_BUFFER_TYPE_UNKNOWN_EXT = 0,
    XR_SPATIAL_BUFFER_TYPE_STRING_EXT = 1,
    XR_SPATIAL_BUFFER_TYPE_UINT8_EXT = 2,
    XR_SPATIAL_BUFFER_TYPE_UINT16_EXT = 3,
    XR_SPATIAL_BUFFER_TYPE_UINT32_EXT = 4,
    XR_SPATIAL_BUFFER_TYPE_FLOAT_EXT = 5,
    XR_SPATIAL_BUFFER_TYPE_VECTOR2F_EXT = 6,
    XR_SPATIAL_BUFFER_TYPE_VECTOR3F_EXT = 7,
    XR_SPATIAL_BUFFER_TYPE_MAX_ENUM_EXT = 0x7FFFFFFF
} XrSpatialBufferTypeEXT;
```

The XrSpatialBufferTypeEXT enumeration identifies the different data types of the buffer represented XrSpatialBufferIdEXT.

Enumerant Descriptions

- XR_SPATIAL_BUFFER_TYPE_STRING_EXT. The XrSpatialBufferIdEXT **can** be passed to xrGetSpatialBufferStringEXT to retrieve a string buffer.
- XR_SPATIAL_BUFFER_TYPE_UINT8_EXT. The XrSpatialBufferIdEXT **can** be passed to xrGetSpatialBufferUint8EXT to retrieve a uint8_t buffer.
- XR_SPATIAL_BUFFER_TYPE_UINT16_EXT. The XrSpatialBufferIdEXT **can** be passed to xrGetSpatialBufferUint16EXT to retrieve a uint16_t buffer.
- XR_SPATIAL_BUFFER_TYPE_UINT32_EXT. The XrSpatialBufferIdEXT **can** be passed to xrGetSpatialBufferUint32EXT to retrieve a uint32_t buffer.
- XR_SPATIAL_BUFFER_TYPE_FLOAT_EXT. The XrSpatialBufferIdEXT **can** be passed to xrGetSpatialBufferFloatEXT to retrieve a float buffer.
- XR_SPATIAL_BUFFER_TYPE_VECTOR2F_EXT. The XrSpatialBufferIdEXT **can** be passed to xrGetSpatialBufferVector2fEXT to retrieve an XrVector2f buffer.
- XR_SPATIAL_BUFFER_TYPE_VECTOR3F_EXT. The XrSpatialBufferIdEXT **can** be passed to xrGetSpatialBufferVector3fEXT to retrieve an XrVector3f buffer.

The xrGetSpatialBufferStringEXT function is defined as:

```
// Provided by XR_EXT_spatial_entity
XrResult xrGetSpatialBufferStringEXT(
    XrSpatialSnapshotEXT                        snapshot,
    const XrSpatialBufferGetInfoEXT*            info,
    uint32_t                                    bufferCapacityInput,
    uint32_t*                                   bufferCountOutput,
    char*                                       buffer);
```

Parameter Descriptions

- `snapshot` is a handle to an XrSpatialSnapshotEXT.
- `info` holds the information on the buffer to query.
- `bufferCapacityInput` is the capacity of the array, or 0 to indicate a request to retrieve the required capacity.
- `bufferCountOutput` is the number of characters, or the required capacity in the case that `bufferCapacityInput` is insufficient.
- `buffer` is an array of `char`. It **can** be `NULL` if `bufferCapacityInput` is 0.
- See the Buffer Size Parameters section for a detailed description of retrieving the required `buffer` size.

The application **can** get the data for an XrSpatialBufferEXT provided by a component, where XrSpatialBufferEXT::`bufferType` is `XR_SPATIAL_BUFFER_TYPE_STRING_EXT` by using xrGetSpatialBufferStringEXT.

The runtime **must** return `XR_ERROR_VALIDATION_FAILURE` if the XrSpatialBufferTypeEXT for XrSpatialBufferGetInfoEXT::`bufferId` is not `XR_SPATIAL_BUFFER_TYPE_STRING_EXT`.

The runtime **must** return `XR_ERROR_SPATIAL_BUFFER_ID_INVALID_EXT` if XrSpatialBufferGetInfoEXT::`bufferId` does not belong to `snapshot`.

`buffer` filled by the runtime **must** be a null-terminated UTF-8 string.

Valid Usage (Implicit)

- The XR_EXT_spatial_entity extension **must** be enabled prior to calling xrGetSpatialBufferStringEXT
- `snapshot` **must** be a valid XrSpatialSnapshotEXT handle
- `info` **must** be a pointer to a valid XrSpatialBufferGetInfoEXT structure
- `bufferCountOutput` **must** be a pointer to a `uint32_t` value
- If `bufferCapacityInput` is not `0`, `buffer` **must** be a pointer to an array of `bufferCapacityInput` char values

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_SIZE_INSUFFICIENT
- XR_ERROR_SPATIAL_BUFFER_ID_INVALID_EXT
- XR_ERROR_VALIDATION_FAILURE

The xrGetSpatialBufferUint8EXT function is defined as:

```
// Provided by XR_EXT_spatial_entity
XrResult xrGetSpatialBufferUint8EXT(
    XrSpatialSnapshotEXT                        snapshot,
    const XrSpatialBufferGetInfoEXT*            info,
    uint32_t                                    bufferCapacityInput,
    uint32_t*                                   bufferCountOutput,
    uint8_t*                                    buffer);
```

Parameter Descriptions

- `snapshot` is a handle to an XrSpatialSnapshotEXT.
- `info` holds the information on the buffer to query.
- `bufferCapacityInput` is the capacity of the array, or 0 to indicate a request to retrieve the required capacity.
- `bufferCountOutput` is the number of elements in the `buffer` array, or the required capacity in the case that `bufferCapacityInput` is insufficient.
- `buffer` is an array of `uint8_t`. It **can** be `NULL` if `bufferCapacityInput` is 0.
- See the Buffer Size Parameters section for a detailed description of retrieving the required `buffer` size.

The application **can** get the data for an XrSpatialBufferEXT provided by a component, where XrSpatialBufferEXT::`bufferType` is `XR_SPATIAL_BUFFER_TYPE_UINT8_EXT` by using xrGetSpatialBufferUint8EXT.

The runtime **must** return `XR_ERROR_VALIDATION_FAILURE` if the XrSpatialBufferTypeEXT for XrSpatialBufferGetInfoEXT::`bufferId` is not `XR_SPATIAL_BUFFER_TYPE_UINT8_EXT`.

The runtime **must** return `XR_ERROR_SPATIAL_BUFFER_ID_INVALID_EXT` if XrSpatialBufferGetInfoEXT ::`bufferId` does not belong to `snapshot`.

Valid Usage (Implicit)

- The `XR_EXT_spatial_entity` extension **must** be enabled prior to calling xrGetSpatialBufferUint8EXT
- `snapshot` **must** be a valid XrSpatialSnapshotEXT handle
- `info` **must** be a pointer to a valid XrSpatialBufferGetInfoEXT structure
- `bufferCountOutput` **must** be a pointer to a `uint32_t` value
- If `bufferCapacityInput` is not `0`, `buffer` **must** be a pointer to an array of `bufferCapacityInput` `uint8_t` values

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_SIZE_INSUFFICIENT
- XR_ERROR_SPATIAL_BUFFER_ID_INVALID_EXT
- XR_ERROR_VALIDATION_FAILURE

The xrGetSpatialBufferUint16EXT function is defined as:

```
// Provided by XR_EXT_spatial_entity
XrResult xrGetSpatialBufferUint16EXT(
    XrSpatialSnapshotEXT                        snapshot,
    const XrSpatialBufferGetInfoEXT*            info,
    uint32_t                                    bufferCapacityInput,
    uint32_t*                                   bufferCountOutput,
    uint16_t*                                   buffer);
```

Parameter Descriptions

- `snapshot` is a handle to an XrSpatialSnapshotEXT.
- `info` holds the information on the buffer to query.
- `bufferCapacityInput` is the capacity of the array, or 0 to indicate a request to retrieve the required capacity.
- `bufferCountOutput` is the number of elements in the `buffer` array, or the required capacity in the case that `bufferCapacityInput` is insufficient.
- `buffer` is an array of `uint16_t`. It **can** be `NULL` if `bufferCapacityInput` is 0.
- See the Buffer Size Parameters section for a detailed description of retrieving the required `buffer` size.

The application **can** get the data for an XrSpatialBufferEXT provided by a component, where XrSpatialBufferEXT::`bufferType` is `XR_SPATIAL_BUFFER_TYPE_UINT16_EXT` by using xrGetSpatialBufferUint16EXT.

The runtime **must** return `XR_ERROR_VALIDATION_FAILURE` if the XrSpatialBufferTypeEXT for XrSpatialBufferGetInfoEXT::`bufferId` is not `XR_SPATIAL_BUFFER_TYPE_UINT16_EXT`.

The runtime **must** return `XR_ERROR_SPATIAL_BUFFER_ID_INVALID_EXT` if XrSpatialBufferGetInfoEXT ::`bufferId` does not belong to `snapshot`.

Valid Usage (Implicit)

- The `XR_EXT_spatial_entity` extension **must** be enabled prior to calling xrGetSpatialBufferUint16EXT
- `snapshot` **must** be a valid XrSpatialSnapshotEXT handle
- `info` **must** be a pointer to a valid XrSpatialBufferGetInfoEXT structure
- `bufferCountOutput` **must** be a pointer to a `uint32_t` value
- If `bufferCapacityInput` is not `0`, `buffer` **must** be a pointer to an array of `bufferCapacityInput` `uint16_t` values

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_SIZE_INSUFFICIENT
- XR_ERROR_SPATIAL_BUFFER_ID_INVALID_EXT
- XR_ERROR_VALIDATION_FAILURE

The xrGetSpatialBufferUint32EXT function is defined as:

```
// Provided by XR_EXT_spatial_entity
XrResult xrGetSpatialBufferUint32EXT(
    XrSpatialSnapshotEXT                        snapshot,
    const XrSpatialBufferGetInfoEXT*            info,
    uint32_t                                    bufferCapacityInput,
    uint32_t*                                   bufferCountOutput,
    uint32_t*                                   buffer);
```

Parameter Descriptions

- `snapshot` is a handle to an XrSpatialSnapshotEXT.
- `info` holds the information on the buffer to query.
- `bufferCapacityInput` is the capacity of the array, or 0 to indicate a request to retrieve the required capacity.
- `bufferCountOutput` is the number of elements in the `buffer` array, or the required capacity in the case that `bufferCapacityInput` is insufficient.
- `buffer` is an array of `uint32_t`. It **can** be `NULL` if `bufferCapacityInput` is 0.
- See the Buffer Size Parameters section for a detailed description of retrieving the required `buffer` size.

The application **can** get the data for an XrSpatialBufferEXT provided by a component, where XrSpatialBufferEXT::`bufferType` is `XR_SPATIAL_BUFFER_TYPE_UINT32_EXT` by using xrGetSpatialBufferUint32EXT.

The runtime **must** return `XR_ERROR_VALIDATION_FAILURE` if the XrSpatialBufferTypeEXT for XrSpatialBufferGetInfoEXT::`bufferId` is not `XR_SPATIAL_BUFFER_TYPE_UINT32_EXT`.

The runtime **must** return `XR_ERROR_SPATIAL_BUFFER_ID_INVALID_EXT` if XrSpatialBufferGetInfoEXT ::`bufferId` does not belong to `snapshot`.

Valid Usage (Implicit)

- The `XR_EXT_spatial_entity` extension **must** be enabled prior to calling xrGetSpatialBufferUint32EXT
- `snapshot` **must** be a valid XrSpatialSnapshotEXT handle
- `info` **must** be a pointer to a valid XrSpatialBufferGetInfoEXT structure
- `bufferCountOutput` **must** be a pointer to a `uint32_t` value
- If `bufferCapacityInput` is not `0`, `buffer` **must** be a pointer to an array of `bufferCapacityInput` `uint32_t` values

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_SIZE_INSUFFICIENT
- XR_ERROR_SPATIAL_BUFFER_ID_INVALID_EXT
- XR_ERROR_VALIDATION_FAILURE

The xrGetSpatialBufferFloatEXT function is defined as:

```
// Provided by XR_EXT_spatial_entity
XrResult xrGetSpatialBufferFloatEXT(
    XrSpatialSnapshotEXT                        snapshot,
    const XrSpatialBufferGetInfoEXT*            info,
    uint32_t                                    bufferCapacityInput,
    uint32_t*                                   bufferCountOutput,
    float*                                      buffer);
```

Parameter Descriptions

- `snapshot` is a handle to an XrSpatialSnapshotEXT.
- `info` holds the information on the buffer to query.
- `bufferCapacityInput` is the capacity of the array, or 0 to indicate a request to retrieve the required capacity.
- `bufferCountOutput` is the number of elements in the `buffer` array, or the required capacity in the case that `bufferCapacityInput` is insufficient.
- `buffer` is an array of `float`. It **can** be `NULL` if `bufferCapacityInput` is 0.
- See the Buffer Size Parameters section for a detailed description of retrieving the required `buffer` size.

The application **can** get the data for an XrSpatialBufferEXT provided by a component, where XrSpatialBufferEXT::`bufferType` is `XR_SPATIAL_BUFFER_TYPE_FLOAT_EXT` by using xrGetSpatialBufferFloatEXT.

The runtime **must** return `XR_ERROR_VALIDATION_FAILURE` if the XrSpatialBufferTypeEXT for XrSpatialBufferGetInfoEXT::`bufferId` is not `XR_SPATIAL_BUFFER_TYPE_FLOAT_EXT`.

The runtime **must** return `XR_ERROR_SPATIAL_BUFFER_ID_INVALID_EXT` if XrSpatialBufferGetInfoEXT ::`bufferId` does not belong to `snapshot`.

Valid Usage (Implicit)

- The `XR_EXT_spatial_entity` extension **must** be enabled prior to calling xrGetSpatialBufferFloatEXT
- `snapshot` **must** be a valid XrSpatialSnapshotEXT handle
- `info` **must** be a pointer to a valid XrSpatialBufferGetInfoEXT structure
- `bufferCountOutput` **must** be a pointer to a `uint32_t` value
- If `bufferCapacityInput` is not `0`, `buffer` **must** be a pointer to an array of `bufferCapacityInput` `float` values

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_SIZE_INSUFFICIENT
- XR_ERROR_SPATIAL_BUFFER_ID_INVALID_EXT
- XR_ERROR_VALIDATION_FAILURE

The xrGetSpatialBufferVector2fEXT function is defined as:

```
// Provided by XR_EXT_spatial_entity
XrResult xrGetSpatialBufferVector2fEXT(
    XrSpatialSnapshotEXT                        snapshot,
    const XrSpatialBufferGetInfoEXT*            info,
    uint32_t                                    bufferCapacityInput,
    uint32_t*                                   bufferCountOutput,
    XrVector2f*                                 buffer);
```

Parameter Descriptions

- `snapshot` is a handle to an XrSpatialSnapshotEXT.
- `info` holds the information on the buffer to query.
- `bufferCapacityInput` is the capacity of the array, or 0 to indicate a request to retrieve the required capacity.
- `bufferCountOutput` is the number of elements in the `buffer` array, or the required capacity in the case that `bufferCapacityInput` is insufficient.
- `buffer` is an array of XrVector2f. It **can** be `NULL` if `bufferCapacityInput` is 0.
- See the Buffer Size Parameters section for a detailed description of retrieving the required `buffer` size.

The application **can** get the data for an XrSpatialBufferEXT provided by a component, where XrSpatialBufferEXT::`bufferType` is `XR_SPATIAL_BUFFER_TYPE_VECTOR2F_EXT` by using xrGetSpatialBufferVector2fEXT.

The runtime **must** return `XR_ERROR_VALIDATION_FAILURE` if the XrSpatialBufferTypeEXT for XrSpatialBufferGetInfoEXT::`bufferId` is not `XR_SPATIAL_BUFFER_TYPE_VECTOR2F_EXT`.

The runtime **must** return `XR_ERROR_SPATIAL_BUFFER_ID_INVALID_EXT` if XrSpatialBufferGetInfoEXT ::`bufferId` does not belong to `snapshot`.

Valid Usage (Implicit)

- The `XR_EXT_spatial_entity` extension **must** be enabled prior to calling xrGetSpatialBufferVector2fEXT
- `snapshot` **must** be a valid XrSpatialSnapshotEXT handle
- `info` **must** be a pointer to a valid XrSpatialBufferGetInfoEXT structure
- `bufferCountOutput` **must** be a pointer to a `uint32_t` value
- If `bufferCapacityInput` is not `0`, `buffer` **must** be a pointer to an array of `bufferCapacityInput` XrVector2f structures

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_SIZE_INSUFFICIENT
- XR_ERROR_SPATIAL_BUFFER_ID_INVALID_EXT
- XR_ERROR_VALIDATION_FAILURE

The xrGetSpatialBufferVector3fEXT function is defined as:

```
// Provided by XR_EXT_spatial_entity
XrResult xrGetSpatialBufferVector3fEXT(
    XrSpatialSnapshotEXT                        snapshot,
    const XrSpatialBufferGetInfoEXT*            info,
    uint32_t                                    bufferCapacityInput,
    uint32_t*                                   bufferCountOutput,
    XrVector3f*                                 buffer);
```

Parameter Descriptions

- `snapshot` is a handle to an XrSpatialSnapshotEXT.
- `info` holds the information on the buffer to query.
- `bufferCapacityInput` is the capacity of the array, or 0 to indicate a request to retrieve the required capacity.
- `bufferCountOutput` is the number of elements in the `buffer` array, or the required capacity in the case that `bufferCapacityInput` is insufficient.
- `buffer` is an array of XrVector3f. It **can** be `NULL` if `bufferCapacityInput` is 0.
- See the Buffer Size Parameters section for a detailed description of retrieving the required `buffer` size.

The application **can** get the data for an XrSpatialBufferEXT provided by a component, where XrSpatialBufferEXT::`bufferType` is `XR_SPATIAL_BUFFER_TYPE_VECTOR3F_EXT` by using xrGetSpatialBufferVector3fEXT.

The runtime **must** return `XR_ERROR_VALIDATION_FAILURE` if the XrSpatialBufferTypeEXT for XrSpatialBufferGetInfoEXT::`bufferId` is not `XR_SPATIAL_BUFFER_TYPE_VECTOR3F_EXT`.

The runtime **must** return `XR_ERROR_SPATIAL_BUFFER_ID_INVALID_EXT` if XrSpatialBufferGetInfoEXT ::`bufferId` does not belong to `snapshot`.

Valid Usage (Implicit)

- The `XR_EXT_spatial_entity` extension **must** be enabled prior to calling xrGetSpatialBufferVector3fEXT
- `snapshot` **must** be a valid XrSpatialSnapshotEXT handle
- `info` **must** be a pointer to a valid XrSpatialBufferGetInfoEXT structure
- `bufferCountOutput` **must** be a pointer to a `uint32_t` value
- If `bufferCapacityInput` is not `0`, `buffer` **must** be a pointer to an array of `bufferCapacityInput` XrVector3f structures

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_SIZE_INSUFFICIENT
- XR_ERROR_SPATIAL_BUFFER_ID_INVALID_EXT
- XR_ERROR_VALIDATION_FAILURE

The XrSpatialBufferGetInfoEXT structure is defined as:

```
// Provided by XR_EXT_spatial_entity
typedef struct XrSpatialBufferGetInfoEXT {
    XrStructureType         type;
    const void*             next;
    XrSpatialBufferIdEXT    bufferId;
} XrSpatialBufferGetInfoEXT;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain.
- bufferId an XrSpatialBufferIdEXT for the buffer whose data to retrieve.

Valid Usage (Implicit)

- The XR_EXT_spatial_entity extension **must** be enabled prior to using XrSpatialBufferGetInfoEXT
- type **must** be XR_TYPE_SPATIAL_BUFFER_GET_INFO_EXT
- next **must** be NULL or a valid pointer to the next structure in a structure chain

```
// Provided by XR_EXT_spatial_entity
#define XR_NULL_SPATIAL_BUFFER_ID_EXT 0
```

XR_NULL_SPATIAL_BUFFER_ID_EXT is a reserved value representing an invalid XrSpatialBufferIdEXT. It **may** be passed to and returned from API functions only when specifically allowed.

Create Update Snapshot

The xrCreateSpatialUpdateSnapshotEXT function is defined as:

```
// Provided by XR_EXT_spatial_entity
XrResult xrCreateSpatialUpdateSnapshotEXT(
    XrSpatialContextEXT                          spatialContext,
    const XrSpatialUpdateSnapshotCreateInfoEXT* createInfo,
    XrSpatialSnapshotEXT*                        snapshot);
```

Parameter Descriptions

- spatialContext is an XrSpatialContextEXT previously created using xrCreateSpatialContextAsyncEXT.
- createInfo is a pointer to an XrSpatialUpdateSnapshotCreateInfoEXT.
- snapshot is a pointer to the result XrSpatialSnapshotEXT.

The application **can** use xrCreateSpatialUpdateSnapshotEXT to create a snapshot and get the latest component data for specific entities as known by the runtime. Applications **can** provide the XrSpatialEntityEXT handles and the component types they are interested in when creating the snapshot.

The application **can** use XrSpatialUpdateSnapshotCreateInfoEXT::componentTypes to filter the list of components whose data **must** be included in the snapshot. If the application provides a valid list of spatial component types in XrSpatialUpdateSnapshotCreateInfoEXT::componentTypes, then the runtime **must** only include spatial entities in the snapshot that have at least one of the components provided in XrSpatialUpdateSnapshotCreateInfoEXT::componentTypes. Also, the runtime **must** only include data for

those components in the snapshot.

The runtime **must** return XR_ERROR_SPATIAL_COMPONENT_NOT_ENABLED_EXT if any of the XrSpatialComponentTypeEXT in XrSpatialUpdateSnapshotCreateInfoEXT::componentTypes are not enabled for the spatial capabilities passed to XrSpatialContextCreateInfoEXT::capabilityConfigs when creating spatialContext.

If the application does not provide a list of spatial component types in XrSpatialUpdateSnapshotCreateInfoEXT::componentTypes, the runtime **must** include all the spatial entities listed in XrSpatialUpdateSnapshotCreateInfoEXT::entities in the snapshot and it **must** include the data for all the enabled components of the capabilities configured for spatialContext.

The runtime **must** not include spatial entities that are not listed in XrSpatialUpdateSnapshotCreateInfoEXT::entities in snapshot.

The application **can** create any number of snapshots it wants but **must** be mindful of the memory being allocated for each new snapshot and **must** destroy the snapshots once it no longer needs them.

Valid Usage (Implicit)

- The XR_EXT_spatial_entity extension **must** be enabled prior to calling xrCreateSpatialUpdateSnapshotEXT
- spatialContext **must** be a valid XrSpatialContextEXT handle
- createInfo **must** be a pointer to a valid XrSpatialUpdateSnapshotCreateInfoEXT structure
- snapshot **must** be a pointer to an XrSpatialSnapshotEXT handle

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_SPATIAL_COMPONENT_NOT_ENABLED_EXT
- XR_ERROR_TIME_INVALID
- XR_ERROR_VALIDATION_FAILURE

The XrSpatialUpdateSnapshotCreateInfoEXT structure is defined as:

```
// Provided by XR_EXT_spatial_entity
typedef struct XrSpatialUpdateSnapshotCreateInfoEXT {
    XrStructureType                     type;
    const void*                         next;
    uint32_t                            entityCount;
    const XrSpatialEntityEXT*           entities;
    uint32_t                            componentTypeCount;
    const XrSpatialComponentTypeEXT*    componentTypes;
    XrSpace                             baseSpace;
    XrTime                              time;
} XrSpatialUpdateSnapshotCreateInfoEXT;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `entityCount` is a `uint32_t` describing the count of elements in the `entities` array.
- `entities` is an array of XrSpatialEntityEXT for which the runtime **must** include the component data in the snapshot.
- `componentTypeCount` is a `uint32_t` describing the count of elements in the `componentTypes` array.
- `componentTypes` is an array of XrSpatialComponentTypeEXT for which the runtime **must** include the data in the snapshot.
- `baseSpace` is the XrSpace relative to which all the locations of the update XrSpatialSnapshotEXT will be located.
- `time` is the `XrTime` at which all the locations of the update XrSpatialSnapshotEXT will be located.

Valid Usage (Implicit)

- The `XR_EXT_spatial_entity` extension **must** be enabled prior to using XrSpatialUpdateSnapshotCreateInfoEXT
- `type` **must** be `XR_TYPE_SPATIAL_UPDATE_SNAPSHOT_CREATE_INFO_EXT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `entities` **must** be a pointer to an array of `entityCount` valid XrSpatialEntityEXT handles
- If `componentTypeCount` is not `0`, `componentTypes` **must** be a pointer to an array of `componentTypeCount` valid XrSpatialComponentTypeEXT values
- `baseSpace` **must** be a valid XrSpace handle
- The `entityCount` parameter **must** be greater than `0`
- Both of `baseSpace` and the elements of `entities` **must** have been created, allocated, or retrieved from the same XrSession

Destroy snapshot

The xrDestroySpatialSnapshotEXT function is defined as:

```
// Provided by XR_EXT_spatial_entity
XrResult xrDestroySpatialSnapshotEXT(
    XrSpatialSnapshotEXT                        snapshot);
```

Parameter Descriptions

- `snapshot` is an XrSpatialSnapshotEXT previously provided by xrCreateSpatialDiscoverySnapshotCompleteEXT or xrCreateSpatialUpdateSnapshotEXT.

The application **can** call xrDestroySpatialSnapshotEXT to destroy the XrSpatialSnapshotEXT handle and the resources associated with it.

Valid Usage (Implicit)

- The XR_EXT_spatial_entity extension **must** be enabled prior to calling xrDestroySpatialSnapshotEXT
- `snapshot` **must** be a valid XrSpatialSnapshotEXT handle

Thread Safety

- Access to `snapshot`, and any child handles, **must** be externally synchronized

Return Codes

Success

- XR_SUCCESS

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_RUNTIME_FAILURE

12.46.9. Common Components

Bounded 2D

Component data

The XrSpatialBounded2DDataEXT structure is defined as:

```
// Provided by XR_EXT_spatial_entity
typedef struct XrSpatialBounded2DDataEXT {
    XrPosef        center;
    XrExtent2Df    extents;
} XrSpatialBounded2DDataEXT;
```

Member Descriptions

- `center` is an XrPosef defining the geometric center of the bounded 2D component.
- `extents` is extents of the bounded 2D component along the x-axis (extents.width), y-axis (extents.height), centered on `center`.

The `extents` of the `XR_SPATIAL_COMPONENT_TYPE_BOUNDED_2D_EXT` refer to the entity's size in the x-y plane of the plane's coordinate system. A plane with a position of {0, 0, 0}, rotation of {0, 0, 0, 1} (no rotation), and an extent of {1, 1} refers to a 1 meter x 1 meter plane centered at {0, 0, 0} with its front face normal vector pointing towards the +Z direction in the component's space.

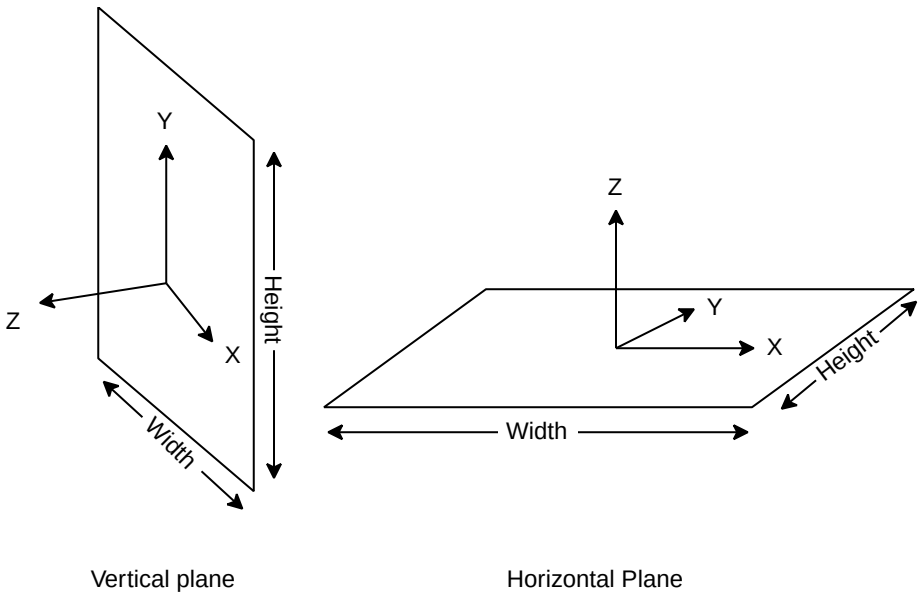


Figure 21. Bounded2D Component Coordinate System



Note

OpenXR uses an X-Y plane with +Z as the plane normal but other APIs **may** use an X-Z plane with +Y as the plane normal. The X-Y plane **can** be converted to an X-Z plane by rotating -π/2 radians around the +X axis.

Valid Usage (Implicit)

- The XR_EXT_spatial_entity extension **must** be enabled prior to using XrSpatialBounded2DDataEXT

Component list structure to query data

The XrSpatialComponentBounded2DListEXT structure is defined as:

```
// Provided by XR_EXT_spatial_entity
typedef struct XrSpatialComponentBounded2DListEXT {
    XrStructureType               type;
    void*                         next;
    uint32_t                      boundCount;
    XrSpatialBounded2DDataEXT*    bounds;
} XrSpatialComponentBounded2DListEXT;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- boundCount is a uint32_t describing the count of elements in the bounds array.
- bounds is an array of XrSpatialBounded2DDataEXT.

The runtime **must** return XR_ERROR_VALIDATION_FAILURE from xrQuerySpatialComponentDataEXT if XrSpatialComponentBounded2DListEXT is in the next chain of XrSpatialComponentDataQueryResultEXT::next but XrSpatialComponentDataQueryConditionEXT::componentTypeCount is not zero and XR_SPATIAL_COMPONENT_TYPE_BOUNDED_2D_EXT is not included in XrSpatialComponentDataQueryConditionEXT::componentTypes.

The runtime **must** return XR_ERROR_SIZE_INSUFFICIENT from xrQuerySpatialComponentDataEXT if boundCount is less than XrSpatialComponentDataQueryResultEXT::entityIdCountOutput.

If xrQuerySpatialComponentDataEXT::snapshot was created from xrCreateSpatialDiscoverySnapshotCompleteEXT, then the runtime **must** provide XrSpatialBounded2DDataEXT::center in XrCreateSpatialDiscoverySnapshotCompletionInfoEXT::baseSpace and XrCreateSpatialDiscoverySnapshotCompletionInfoEXT::time.

If xrQuerySpatialComponentDataEXT::snapshot was created from xrCreateSpatialUpdateSnapshotEXT, then the runtime **must** provide XrSpatialBounded2DDataEXT::center in

XrSpatialUpdateSnapshotCreateInfoEXT::baseSpace and XrSpatialUpdateSnapshotCreateInfoEXT::time.

Valid Usage (Implicit)

- The XR_EXT_spatial_entity extension **must** be enabled prior to using XrSpatialComponentBounded2DListEXT
- type **must** be XR_TYPE_SPATIAL_COMPONENT_BOUNDED_2D_LIST_EXT
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- bounds **must** be a pointer to an array of boundCount XrSpatialBounded2DDataEXT structures
- The boundCount parameter **must** be greater than 0

Configuration

If XR_SPATIAL_COMPONENT_TYPE_BOUNDED_2D_EXT is enumerated in XrSpatialCapabilityComponentTypesEXT::componentTypes for some capability, the application **can** enable it by including the enum value in the XrSpatialCapabilityConfigurationBaseHeaderEXT::enabledComponents list. This component does not require any special configuration to be included in the next chain of XrSpatialCapabilityConfigurationBaseHeaderEXT.

Bounded 3D

Component data

XR_SPATIAL_COMPONENT_TYPE_BOUNDED_3D_EXT uses XrBoxf for its data.

Component list structure to query data

The XrSpatialComponentBounded3DListEXT structure is defined as:

```
// Provided by XR_EXT_spatial_entity
typedef struct XrSpatialComponentBounded3DListEXT {
    XrStructureType    type;
    void*              next;
    uint32_t           boundCount;
    XrBoxf*            bounds;
} XrSpatialComponentBounded3DListEXT;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `boundCount` is a `uint32_t` describing the count of elements in the `bounds` array.
- `bounds` is an array of XrBoxf.

The runtime **must** return `XR_ERROR_VALIDATION_FAILURE` from xrQuerySpatialComponentDataEXT if XrSpatialComponentBounded3DListEXT is in the next chain of XrSpatialComponentDataQueryResultEXT::`next` but XrSpatialComponentDataQueryConditionEXT::componentTypeCount is not zero and `XR_SPATIAL_COMPONENT_TYPE_BOUNDED_3D_EXT` is not included in XrSpatialComponentDataQueryConditionEXT::`componentTypes`.

The runtime **must** return `XR_ERROR_SIZE_INSUFFICIENT` from xrQuerySpatialComponentDataEXT if `boundCount` is less than XrSpatialComponentDataQueryResultEXT::`entityIdCountOutput`.

If xrQuerySpatialComponentDataEXT::`snapshot` was created from xrCreateSpatialDiscoverySnapshotCompleteEXT, then the runtime **must** provide XrBoxf::`center` in XrCreateSpatialDiscoverySnapshotCompletionInfoEXT::`baseSpace` at XrCreateSpatialDiscoverySnapshotCompletionInfoEXT::`time`.

If xrQuerySpatialComponentDataEXT::`snapshot` was created from xrCreateSpatialUpdateSnapshotEXT, then the runtime **must** provide XrBoxf::`center` in XrSpatialUpdateSnapshotCreateInfoEXT::`baseSpace` at XrSpatialUpdateSnapshotCreateInfoEXT::`time`.

Valid Usage (Implicit)

- The `XR_EXT_spatial_entity` extension **must** be enabled prior to using XrSpatialComponentBounded3DListEXT
- `type` **must** be `XR_TYPE_SPATIAL_COMPONENT_BOUNDED_3D_LIST_EXT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `bounds` **must** be a pointer to an array of `boundCount` XrBoxf structures
- The `boundCount` parameter **must** be greater than `0`

Configuration

If `XR_SPATIAL_COMPONENT_TYPE_BOUNDED_3D_EXT` is enumerated in XrSpatialCapabilityComponentTypesEXT::`componentTypes` for some capability, the application **can** enable it by including the enum in the XrSpatialCapabilityConfigurationBaseHeaderEXT

::enabledComponents list. This component does not require any special configuration to be included in the next chain of XrSpatialCapabilityConfigurationBaseHeaderEXT.

Parent

Component data

XR_SPATIAL_COMPONENT_TYPE_PARENT_EXT uses XrSpatialEntityIdEXT for its data.

Component list structure to query data

The XrSpatialComponentParentListEXT structure is defined as:

```
// Provided by XR_EXT_spatial_entity
typedef struct XrSpatialComponentParentListEXT {
    XrStructureType          type;
    void*                    next;
    uint32_t                 parentCount;
    XrSpatialEntityIdEXT*    parents;
} XrSpatialComponentParentListEXT;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- parentCount is a uint32_t describing the count of elements in the parents array.
- parents is an array of XrSpatialEntityIdEXT.

The runtime **must** return XR_ERROR_VALIDATION_FAILURE from xrQuerySpatialComponentDataEXT if XrSpatialComponentParentListEXT is in the next chain of XrSpatialComponentDataQueryResultEXT::next but XrSpatialComponentDataQueryConditionEXT::componentTypeCount is not zero and XR_SPATIAL_COMPONENT_TYPE_PARENT_EXT is not included in XrSpatialComponentDataQueryConditionEXT::componentTypes.

The runtime **must** return XR_ERROR_SIZE_INSUFFICIENT from xrQuerySpatialComponentDataEXT if parentCount is less than XrSpatialComponentDataQueryResultEXT::entityIdCountOutput.

Valid Usage (Implicit)

- The XR_EXT_spatial_entity extension **must** be enabled prior to using XrSpatialComponentParentListEXT
- type **must** be XR_TYPE_SPATIAL_COMPONENT_PARENT_LIST_EXT
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- parents **must** be a pointer to an array of parentCount XrSpatialEntityIdEXT values
- The parentCount parameter **must** be greater than 0

Configuration

If XR_SPATIAL_COMPONENT_TYPE_PARENT_EXT is enumerated in XrSpatialCapabilityComponentTypesEXT::componentTypes for some capability, the application **can** enable it by including the enum in the XrSpatialCapabilityConfigurationBaseHeaderEXT::enabledComponents list. This component does not require any special configuration to be included in the next chain of XrSpatialCapabilityConfigurationBaseHeaderEXT.

Mesh 3D

Component data

The XrSpatialMeshDataEXT structure is defined as:

```
// Provided by XR_EXT_spatial_entity
typedef struct XrSpatialMeshDataEXT {
    XrPosef               origin;
    XrSpatialBufferEXT    vertexBuffer;
    XrSpatialBufferEXT    indexBuffer;
} XrSpatialMeshDataEXT;
```

Member Descriptions

- `origin` is an XrPosef defining the origin of the mesh. All vertices of the mesh **must** be relative to this origin.
- `vertexBuffer` is an XrSpatialBufferEXT that provides the ID for a buffer that represents the vertex buffer of the entity this component is on. The position of vertices **must** be relative to `origin`.
- `indexBuffer` is an XrSpatialBufferEXT that provides the ID for a buffer that represents an array of triangle indices, specifying the indices of the mesh vertices in the `vertexBuffer`. The triangle indices **must** be returned in counter-clockwise order and three indices denote one triangle.

The component type using XrSpatialMeshDataEXT **must** specify the XrSpatialBufferTypeEXT of the `vertexBuffer` and `indexBuffer`.

Valid Usage (Implicit)

- The `XR_EXT_spatial_entity` extension **must** be enabled prior to using XrSpatialMeshDataEXT
- `vertexBuffer` **must** be a valid XrSpatialBufferEXT structure
- `indexBuffer` **must** be a valid XrSpatialBufferEXT structure

Component list structure to query data

The XrSpatialComponentMesh3DListEXT structure is defined as:

```
// Provided by XR_EXT_spatial_entity
typedef struct XrSpatialComponentMesh3DListEXT {
    XrStructureType          type;
    void*                    next;
    uint32_t                 meshCount;
    XrSpatialMeshDataEXT*    meshes;
} XrSpatialComponentMesh3DListEXT;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `meshCount` is a `uint32_t` describing the count of elements in the `meshes` array.
- `meshes` is an array of XrSpatialMeshDataEXT.

The application **can** query the mesh 3D component of the spatial entities in an XrSpatialSnapshotEXT by adding `XR_SPATIAL_COMPONENT_TYPE_MESH_3D_EXT` in XrSpatialComponentDataQueryConditionEXT::`componentTypes` and adding XrSpatialComponentMesh3DListEXT to the next pointer chain of XrSpatialComponentDataQueryResultEXT.

The runtime **must** return `XR_ERROR_VALIDATION_FAILURE` from xrQuerySpatialComponentDataEXT if XrSpatialComponentMesh3DListEXT is in the next chain of XrSpatialComponentDataQueryResultEXT::`next` but XrSpatialComponentDataQueryConditionEXT::componentTypeCount is not zero and `XR_SPATIAL_COMPONENT_TYPE_MESH_3D_EXT` is not included in XrSpatialComponentDataQueryConditionEXT::`componentTypes`.

The runtime **must** return `XR_ERROR_SIZE_INSUFFICIENT` from xrQuerySpatialComponentDataEXT if `meshCount` is less than XrSpatialComponentDataQueryResultEXT::`entityIdCountOutput`.

For the XrSpatialMeshDataEXT filled out by the runtime in the `meshes` array, the XrSpatialBufferEXT::`bufferType` for XrSpatialMeshDataEXT::`vertexBuffer` **must** be `XR_SPATIAL_BUFFER_TYPE_VECTOR3F_EXT` and XrSpatialBufferEXT::`bufferType` for XrSpatialMeshDataEXT::`indexBuffer` **must** be `XR_SPATIAL_BUFFER_TYPE_UINT32_EXT`.

Valid Usage (Implicit)

- The `XR_EXT_spatial_entity` extension **must** be enabled prior to using XrSpatialComponentMesh3DListEXT
- `type` **must** be `XR_TYPE_SPATIAL_COMPONENT_MESH_3D_LIST_EXT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `meshes` **must** be a pointer to an array of `meshCount` XrSpatialMeshDataEXT structures
- The `meshCount` parameter **must** be greater than `0`

Configuration

If `XR_SPATIAL_COMPONENT_TYPE_MESH_3D_EXT` is enumerated in XrSpatialCapabilityComponentTypesEXT::`componentTypes` for some capability, the application **can** enable it by including the enum in the XrSpatialCapabilityConfigurationBaseHeaderEXT::`enabledComponents` list of the XrSpatialCapabilityConfigurationBaseHeaderEXT derived structure of the capability that supports this

component.

This component does not require any special configuration to be included in the next chain of XrSpatialCapabilityConfigurationBaseHeaderEXT.

12.46.10. Tracking state filters

The XrSpatialFilterTrackingStateEXT structure is defined as:

```
// Provided by XR_EXT_spatial_entity
typedef struct XrSpatialFilterTrackingStateEXT {
    XrStructureType                    type;
    const void*                        next;
    XrSpatialEntityTrackingStateEXT    trackingState;
} XrSpatialFilterTrackingStateEXT;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `trackingState` is the XrSpatialEntityTrackingStateEXT for which the application wants to apply the filter.

The application **can** use XrSpatialFilterTrackingStateEXT in the next chain of XrSpatialDiscoverySnapshotCreateInfoEXT to scope the discovery to only those entities whose tracking state is `trackingState`.

The application **can** use XrSpatialFilterTrackingStateEXT in the next chain of XrSpatialComponentDataQueryConditionEXT to scope the component data query from a snapshot only to entities whose tracking state is `trackingState`.

Valid Usage (Implicit)

- The `XR_EXT_spatial_entity` extension **must** be enabled prior to using XrSpatialFilterTrackingStateEXT
- `type` **must** be `XR_TYPE_SPATIAL_FILTER_TRACKING_STATE_EXT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `trackingState` **must** be a valid XrSpatialEntityTrackingStateEXT value

12.46.11. Example code

Application Usage

Applications typically use the spatial entity extension in the following pattern:

- An application first enumerates the spatial capabilities of the system using xrEnumerateSpatialCapabilitiesEXT. It then inspects the returned array of XrSpatialCapabilityEXT and enumerates the components and features supported for each of those capabilities by using xrEnumerateSpatialCapabilityComponentTypesEXT and xrEnumerateSpatialCapabilityFeaturesEXT respectively. This gives the application a full picture of the components that it **can** enable and the configurations the capability accepts.
- The application then creates one or many XrSpatialContextEXT handles with specific spatial capability configurations, wherein the configurations enable & configure a specific capability in the spatial context, and enable & configure components for those capabilities.
- For each XrSpatialContextEXT, the application waits to receive XrEventDataSpatialDiscoveryRecommendedEXT events for that XrSpatialContextEXT before using xrCreateSpatialDiscoverySnapshotAsyncEXT to discover spatial entities. Once this async operation is complete, the application receives a XrSpatialSnapshotEXT handle.
- The application queries for the entities and the component data included in this XrSpatialSnapshotEXT by using xrQuerySpatialComponentDataEXT. The application reads the latest component data of the queried entities from structures attached to the next chain of XrSpatialComponentDataQueryResultEXT if the entity state is `XR_SPATIAL_ENTITY_TRACKING_STATE_TRACKING_EXT`.
- If there are specific entities that the application identifies as interesting and wants to get updates for over time, it creates XrSpatialEntityEXT handles for those entities by using xrCreateSpatialEntityFromIdEXT. The application gets updates for such interesting entities by using xrCreateSpatialUpdateSnapshotEXT and use the same xrQuerySpatialComponentDataEXT function on the newly created XrSpatialSnapshotEXT to get the latest component data for those entities.

Discover spatial entities & query component data

The following example code demonstrates how to discover spatial entities for capability "Foo" query its component data.

```
/*****************************/
/* Capability definition    */
/*****************************/
// Foo capability has the following components -
// - XR_SPATIAL_COMPONENT_TYPE_BOUNDED_2D_EXT
#define XR_SPATIAL_CAPABILITY_FOO ((XrSpatialCapabilityEXT)1000740000U)

#define XR_TYPE_SPATIAL_CAPABILITY_CONFIGURATION_FOO_EXT ((XrStructureType)1000740000U)
// Derives from XrSpatialCapabilityConfigurationBaseHeaderEXT
```

```
typedef struct XrSpatialCapabilityConfigurationFooEXT {
  XrStructureType                     type;
  const void* XR_MAY_ALIAS            next;
  XrSpatialCapabilityEXT              capability;
  uint32_t                            enabledComponentCount;
  const XrSpatialComponentTypeEXT*    enabledComponents;
} XrSpatialCapabilityConfigurationFooEXT;

/*******************************/
/* End capability definition  */
/*******************************/

auto waitUntilReady = [](XrFutureEXT future) {
  XrFuturePollInfoEXT pollInfo {
    .type = XR_TYPE_FUTURE_POLL_INFO_EXT,
    .future = future,
  };
  XrFuturePollResultEXT pollResult {
    .type = XR_TYPE_FUTURE_POLL_RESULT_EXT,
  };
  do {
    // sleep(1);
    CHK_XR(xrPollFutureEXT(instance, &pollInfo, &pollResult));
  } while (pollResult.state != XR_FUTURE_STATE_READY_EXT);
};

// Create a spatial spatial context
XrSpatialContextEXT spatialContext{};
{
  const std::array<XrSpatialComponentTypeEXT, 1> enabledComponents = {
    XR_SPATIAL_COMPONENT_TYPE_BOUNDED_2D_EXT,
  };

  // Configure Foo capability for the spatial context
  XrSpatialCapabilityConfigurationFooEXT fooConfig {
    .type = XR_TYPE_SPATIAL_CAPABILITY_CONFIGURATION_FOO_EXT,
    .capability = XR_SPATIAL_CAPABILITY_FOO,
    .enabledComponentCount = static_cast<uint32_t>(enabledComponents.size()),
    .enabledComponents = enabledComponents.data(),
  };

  std::vector<XrSpatialCapabilityConfigurationBaseHeaderEXT*> capabilityConfigs;
  capabilityConfigs.push_back(reinterpret_cast
<XrSpatialCapabilityConfigurationBaseHeaderEXT*>(&fooConfig));

  XrSpatialContextCreateInfoEXT spatialContextCreateInfo {
    .type = XR_TYPE_SPATIAL_CONTEXT_CREATE_INFO_EXT,
    .capabilityConfigCount = static_cast<uint32_t>(capabilityConfigs.size()),
```

```
    .capabilityConfigs = capabilityConfigs.data(),
  };
  XrFutureEXT createContextFuture {XR_NULL_FUTURE_EXT};
  CHK_XR(xrCreateSpatialContextAsyncEXT(session, &spatialContextCreateInfo,
&createContextFuture));

  waitUntilReady(createContextFuture);

  XrCreateSpatialContextCompletionEXT completion {
    .type = XR_TYPE_CREATE_SPATIAL_CONTEXT_COMPLETION_EXT,
  };
  CHK_XR(xrCreateSpatialContextCompleteEXT(session, createContextFuture, &completion));
  if (completion.futureResult != XR_SUCCESS) {
    return;
  }

  spatialContext = completion.spatialContext;
}

auto discoverSpatialEntities = [&](XrSpatialContextEXT spatialContext, XrTime time) {
  // We want to look for entities that have the following components.
  std::array<XrSpatialComponentTypeEXT, 1> snapshotComponents
{XR_SPATIAL_COMPONENT_TYPE_BOUNDED_2D_EXT};

  XrSpatialDiscoverySnapshotCreateInfoEXT snapshotCreateInfo {
    .type = XR_TYPE_SPATIAL_DISCOVERY_SNAPSHOT_CREATE_INFO_EXT,
    .componentTypeCount = snapshotComponents.size(),
    .componentTypes = snapshotComponents.data(),
  };
  XrFutureEXT future {XR_NULL_FUTURE_EXT};
  CHK_XR(xrCreateSpatialDiscoverySnapshotAsyncEXT(spatialContext, &snapshotCreateInfo,
&future));

  waitUntilReady(future);

  XrCreateSpatialDiscoverySnapshotCompletionInfoEXT completionInfo {
    .type = XR_TYPE_CREATE_SPATIAL_DISCOVERY_SNAPSHOT_COMPLETION_INFO_EXT,
    .baseSpace = localSpace,
    .time = time,
    .future = future,
  };

  XrCreateSpatialDiscoverySnapshotCompletionEXT completion {
    .type = XR_TYPE_CREATE_SPATIAL_DISCOVERY_SNAPSHOT_COMPLETION_EXT,
  };
  CHK_XR(xrCreateSpatialDiscoverySnapshotCompleteEXT(spatialContext, &completionInfo,
&completion));
  if (completion.futureResult == XR_SUCCESS) {
```

```
    // Query for the bounded2d component data
    XrSpatialComponentTypeEXT componentToQuery =
XR_SPATIAL_COMPONENT_TYPE_BOUNDED_2D_EXT;
    XrSpatialComponentDataQueryConditionEXT queryCond {
      .type = XR_TYPE_SPATIAL_COMPONENT_DATA_QUERY_CONDITION_EXT,
      .componentTypes = &componentToQuery,
    };

    XrSpatialComponentDataQueryResultEXT queryResult {
      .type = XR_TYPE_SPATIAL_COMPONENT_DATA_QUERY_RESULT_EXT,
      .entityIdCapacityInput = 0,
      .entityIds = nullptr,
      .entityStateCapacityInput = 0,
      .entityStates = nullptr,
    };
    CHK_XR(xrQuerySpatialComponentDataEXT(completion.snapshot, &queryCond, &
queryResult));

    std::vector<XrSpatialEntityIdEXT> entityIds(queryResult.entityIdCountOutput);
    std::vector<XrSpatialEntityTrackingStateEXT> entityStates(queryResult
.entityStateCountOutput);
    queryResult.entityIdCapacityInput = entityIds.size();
    queryResult.entityIds = entityIds.data();
    queryResult.entityStateCapacityInput = entityStates.size();
    queryResult.entityStates = entityStates.data();

    std::vector<XrSpatialBounded2DDataEXT> bounded2d(queryResult.entityIdCountOutput);
    XrSpatialComponentBounded2DListEXT boundsList {
      .type = XR_TYPE_SPATIAL_COMPONENT_BOUNDED_2D_LIST_EXT,
      .boundCount = static_cast<uint32_t>(bounded2d.size()),
      .bounds = bounded2d.data(),
    };
    queryResult.next = &boundsList;

    CHK_XR(xrQuerySpatialComponentDataEXT(completion.snapshot, &queryCond, &
queryResult));

    for (int32_t i = 0; i < queryResult.entityIdCountOutput; ++i) {
      if (entityStates[i] == XR_SPATIAL_ENTITY_TRACKING_STATE_TRACKING_EXT) {
        // 2D extents for entity entityIds[i] is bounded2d[i].extents.
      }
    }

    CHK_XR(xrDestroySpatialSnapshotEXT(completion.snapshot));
  }
};

while (1) {
```

```
  // ...
  // For every frame in frame loop
  // ...

  XrFrameState frameState;  // previously returned from xrWaitFrame
  const XrTime time = frameState.predictedDisplayTime;

  // Poll for the XR_TYPE_EVENT_DATA_SPATIAL_DISCOVERY_RECOMMENDED_EXT event
  XrEventDataBuffer event = {
    .type = XR_TYPE_EVENT_DATA_BUFFER,
  };
  XrResult result = xrPollEvent(instance, &event);
  if (result == XR_SUCCESS) {
      switch (event.type) {
          case XR_TYPE_EVENT_DATA_SPATIAL_DISCOVERY_RECOMMENDED_EXT: {
              const XrEventDataSpatialDiscoveryRecommendedEXT& eventdata =
                  *reinterpret_cast<XrEventDataSpatialDiscoveryRecommendedEXT*>(&event);
              // Discover spatial entities for the context that we recceived the
"discovery
              // recommended" event for.
              discoverSpatialEntities(eventdata.spatialContext, time);
              break;
          }
      }
  }

  // ...
  // Finish frame loop
  // ...
}

CHK_XR(xrDestroySpatialContextEXT(spatialContext));
```

Query buffer data

The following example code demonstrates how to get the data of a component that provides an XrSpatialBufferEXT.

```
/*****************************/
/* Component definition     */
/*****************************/
// Foo component that provides an XrVector3f buffer
#define XR_SPATIAL_COMPONENT_TYPE_FOO_EXT ((XrSpatialComponentTypeEXT)1000740000U)

#define XR_TYPE_SPATIAL_COMPONENT_FOO_LIST_EXT ((XrStructureType)1000740000U)
```

```
// XrSpatialComponentFooListEXT extends XrSpatialComponentDataQueryResultEXT
typedef struct XrSpatialComponentFooListEXT {
    XrStructureType                   type;
    void* XR_MAY_ALIAS                next;
    uint32_t                          fooCount;
    XrSpatialBufferEXT*               foo;
} XrSpatialComponentFooListEXT;

/*******************************/
/* End Component definition  */
/*******************************/

// Query for the foo component data
XrSpatialComponentTypeEXT componentToQuery = XR_SPATIAL_COMPONENT_TYPE_FOO_EXT;
XrSpatialComponentDataQueryConditionEXT queryCond{
  .type = XR_TYPE_SPATIAL_COMPONENT_DATA_QUERY_CONDITION_EXT,
  .componentTypeCount = 1,
  .componentTypes = &componentToQuery,
};

XrSpatialComponentDataQueryResultEXT queryResult{
  .type = XR_TYPE_SPATIAL_COMPONENT_DATA_QUERY_RESULT_EXT,
};
CHK_XR(xrQuerySpatialComponentDataEXT(snapshot, &queryCond, &queryResult));

std::vector<XrSpatialEntityIdEXT> entityIds(queryResult.entityIdCountOutput);
queryResult.entityIdCapacityInput = entityIds.size();
queryResult.entityIds = entityIds.data();

std::vector<XrSpatialBufferEXT> fooBuffers(queryResult.entityIdCountOutput);
XrSpatialComponentFooListEXT fooList {
  .type = XR_TYPE_SPATIAL_COMPONENT_FOO_LIST_EXT,
  .fooCount = static_cast<uint32_t>(fooBuffers.size()),
  .foo = fooBuffers.data(),
};
queryResult.next = &fooList;

CHK_XR(xrQuerySpatialComponentDataEXT(snapshot, &queryCond, &queryResult));

for (int32_t i = 0; i < queryResult.entityIdCountOutput; ++i) {
  // foo component data for entity entityIds[i]
  if (fooBuffers[i].bufferType == XR_SPATIAL_BUFFER_TYPE_VECTOR3F_EXT) {
    XrSpatialBufferGetInfoEXT getInfo {
      .type = XR_TYPE_SPATIAL_BUFFER_GET_INFO_EXT,
      .bufferId = fooBuffers[i].bufferId,
    };
    uint32_t bufferCountOutput = 0;
    CHK_XR(xrGetSpatialBufferVector3fEXT(snapshot, &getInfo, 0, &bufferCountOutput,
```

```
nullptr));
    std::vector<XrVector3f> vertexBuffer(bufferCountOutput);
    CHK_XR(xrGetSpatialBufferVector3fEXT(snapshot, &getInfo, bufferCountOutput,
&bufferCountOutput, vertexBuffer.data()));

    // XrVertex3f buffer for entity entityIds[i] is now available in vertexBuffer vector.
  }
}
```

12.46.12. Extension guidelines

- If an extension defines a new XrSpatialComponentTypeEXT which provides additional data about a spatial entity,
  - The extension **must** also define a list structure for that component which allows the application to pass an array to the runtime to fill out with the data for each of the spatial entities that satisfy xrQuerySpatialComponentDataEXT::`queryCondition`. Examples of such list structures are XrSpatialComponentParentListEXT and XrSpatialComponentBounded2DListEXT. If the component data size is variable and requires the application to use the two-call idiom to query it, the component data **should** provide an XrSpatialBufferEXT for each variable-sized data in the list structure and for each such buffer it **must** specify the XrSpatialBufferTypeEXT. Applications **can** then query the buffer data using functions defined in Two-call idiom for component data. An example of such a structure is XrSpatialMeshDataEXT which is included in XrSpatialComponentMesh3DListEXT.
  - The extension **can** also provide structures that the application **can** chain to XrSpatialComponentDataQueryConditionEXT::`next` to provide additional filters for the query pertaining to the data of this component.
- Extensions **can** define structures that extend XrSpatialDiscoverySnapshotCreateInfoEXT to provide additional filters for discovery. The filters for creating the snapshot **must** not affect the configuration of the spatial context, but instead provide hints to the runtime on what entities and data to include in the snapshot as tracked by the current configuration of the spatial context (and therefore the current configuration of the underlying services).
- If an extension defines a new XrSpatialCapabilityEXT,
  - The extension **should** also specify the list of XrSpatialComponentTypeEXT that the runtimes **must** provide on entities for that capability.
  - The extension **must** also provide structures derived from XrSpatialCapabilityConfigurationBaseHeaderEXT that allow the configuration of that capability.
- If an extension defines a new XrSpatialCapabilityFeatureEXT,
  - The extension **must** also define a corresponding configuration structure that **can** be chained to the next pointer of XrSpatialCapabilityConfigurationBaseHeaderEXT, that the application **can** use to enable the feature when creating an XrSpatialContextEXT.
- An extension defining a new XrSpatialCapabilityEXT **should** follow this template for the

specification:

  - Overview
  - Runtime support
  - Configuration
  - Guaranteed components
  - Example Code

- An extension defining a new XrSpatialComponentTypeEXT **should** follow this template for the specification:

  - Component data
  - Component list structure to query data
  - Configuration

- When writing an api that provides the application with a `XrSpatialBufferIdEXT`, it **must** be accompanied by a XrSpatialBufferTypeEXT to inform the application of the data type of the buffer, so that the application **can** use an appropriate `xrGetSpatialBuffer*` function to retrieve the actual contents of the buffer.

Semantic Label Design & Extension Guidelines

While this extension does not define any component for conveying the semantic label information of an entity, it lays down the design rules and guidelines for any extensions that do define such a component. In the following text, "**should**" and "**must**" constrain the authors of such functionality, rather than constraining a runtime implementation or an application using the API.

- If an extension is defining semantic labels for any XrSpatialCapabilityEXT, it **should** do so by defining a new XrSpatialComponentTypeEXT as well as a new enumeration with a set of labels (a "semantic label set") to be the data for that component.
- An enumeration representing a semantic label set **must** have a value to indicate that the entity is not categorized into any of the labels of this set. The name of this enumerant **must** include 'UNCATEGORIZED', e.g. `XR_SPATIAL_PLANE_SEMANTIC_LABEL_UNCATEGORIZED_EXT`. This value indicates to applications that the runtime does recognize this entity, but that there are other semantic label sets where this entity is categorized.
- Enumeration of this newly defined XrSpatialComponentTypeEXT in xrEnumerateSpatialCapabilityComponentTypesEXT implies that the runtime supports the semantic label set (and therefore the labels defined in the set) corresponding to that component type.
- Applications express their desire to discover entities that have labels belonging to this semantic label set by including the corresponding XrSpatialComponentTypeEXT in the XrSpatialCapabilityConfigurationBaseHeaderEXT::`enabledComponents` array of the configuration struct for the capability that the component type is enumerated for.
- It is possible that a single spatial entity has multiple components on it, each providing it with a label from their own semantic label set. Consider the following example with some hypothetical

semantic label sets - XrSpatialObjectSemanticLabel, which might have values like chair, sofa, table etc., XrSpatialSofaBrand, which might have values like IKEA, Wayfair, etc., and XrSpatialIkeaSofa, which might have specific IKEA model names of sofas. In this scenario, the same sofa entity has all 3 components (labels from all 3 semantic label sets) on it, each giving a different granularity of information about the entity.

- If an extension is defining a new XrSpatialComponentTypeEXT which provides a new semantic label set containing multiple labels in addition to the UNCATEGORIZED label, the extension **should** consider defining an XrSpatialCapabilityFeatureEXT which allows applications to activate only a subset of the labels defined in the semantic label set corresponding to that component type. This subset, provided in the next chain of the capability configuration, acts as a hint for runtimes to optimize their tracking pipelines and only track the subset provided. An example of such a feature struct is provided below.
- Extensions **must** not add labels to existing semantic label set enumeration. They **must** instead define a new enumeration which contains equivalent enumerants with identical semantics and with values equal to those from the other set, in addition to the newly added enumerants. The specification of the newly defined semantic label set **must** also establish the mapping between the old label set and the new one. See the Spatial Semantic Label Extension example below.

Example Spatial Feature Definition for Semantic Label Subset

```
// New example semantic label set.
typedef enum XrSpatialExampleSemanticLabelEXT {
  // ... label values
} XrSpatialExampleSemanticLabelEXT;

/*
typedef enum XrSpatialCapabilityFeatureEXT {
  // .. existing values

  // new spatial feature value
  XR_SPATIAL_CAPABILITY_FEATURE_EXAMPLE_SEMANTIC_LABEL_SUBSET_EXT

  // .. existing values
} XrSpatialCapabilityFeatureEXT;
*/

// New spatial feaure configuration structure
typedef struct XrSpatialFeatureExampleSemanticLabelSubsetEXT {
    XrStructureType                         type;
    const void* XR_MAY_ALIAS                next;
    uint32_t                                semanticLabelCount;
    const XrSpatialExampleSemanticLabelEXT*     semanticLabels;
} XrSpatialFeatureExampleSemanticLabelSubsetEXT;
```

Example Spatial Semantic Label Extension

```
typedef enum XrSpatialExamplePlaneSemanticLabelExtensionEXT {
    XR_SPATIAL_EXAMPLE_PLANE_SEMANTIC_LABEL_EXTENSION_UNCATEGORIZED_EXT = 1,
    XR_SPATIAL_EXAMPLE_PLANE_SEMANTIC_LABEL_EXTENSION_FLOOR_EXT = 2,
    XR_SPATIAL_EXAMPLE_PLANE_SEMANTIC_LABEL_EXTENSION_WALL_EXT = 3,
    XR_SPATIAL_EXAMPLE_PLANE_SEMANTIC_LABEL_EXTENSION_CEILING_EXT = 4,
    XR_SPATIAL_EXAMPLE_PLANE_SEMANTIC_LABEL_EXTENSION_TABLE_EXT = 5,
    XR_SPATIAL_EXAMPLE_PLANE_SEMANTIC_LABEL_EXTENSION_DOOR_EXT = 6,
    XR_SPATIAL_EXAMPLE_PLANE_SEMANTIC_LABEL_EXTENSION_WINDOW_EXT = 7,
    XR_SPATIAL_EXAMPLE_PLANE_SEMANTIC_LABEL_EXTENSION_MAX_ENUM_EXT = 0x7FFFFFFF
} XrSpatialExamplePlaneSemanticLabelExtensionEXT;
```

Consider the above example where an extension defines an enumeration XrSpatialExamplePlaneSemanticLabelExtensionEXT in order to extend the existing XrSpatialPlaneSemanticLabelEXT with values to represent doors and windows. The specification for the new enumeration **must** include text of the following form:

```
For all non-UNCATEGORIZED label enumerants in elink:XrSpatialPlaneSemanticLabelEXT,
the runtime must: return that enumerant if and only if it returns the corresponding
label enumerant from elink:XrSpatialExamplePlaneSemanticLabelExtensionEXT with the same
value
i.e.
the ename:XR_SPATIAL_COMPONENT_TYPE_PLANE_SEMANTIC_LABEL_EXT value for that entity must:
correspond with the
ename:XR_SPATIAL_COMPONENT_TYPE_EXAMPLE_PLANE_SEMANTIC_LABEL_EXTENSION_EXT values
according to the following table:

[width="100%",options="header"]
|====
| XrSpatialExamplePlaneSemanticLabelExtensionEXT                |
XrSpatialPlaneSemanticLabelEXT
| XR_SPATIAL_EXAMPLE_PLANE_SEMANTIC_LABEL_EXTENSION_FLOOR_EXT   |
XR_SPATIAL_PLANE_SEMANTIC_LABEL_FLOOR_EXT
| XR_SPATIAL_EXAMPLE_PLANE_SEMANTIC_LABEL_EXTENSION_WALL_EXT    |
XR_SPATIAL_PLANE_SEMANTIC_LABEL_WALL_EXT
| XR_SPATIAL_EXAMPLE_PLANE_SEMANTIC_LABEL_EXTENSION_CEILING_EXT |
XR_SPATIAL_PLANE_SEMANTIC_LABEL_CEILING_EXT
| XR_SPATIAL_EXAMPLE_PLANE_SEMANTIC_LABEL_EXTENSION_TABLE_EXT   |
XR_SPATIAL_PLANE_SEMANTIC_LABEL_TABLE_EXT
| XR_SPATIAL_EXAMPLE_PLANE_SEMANTIC_LABEL_EXTENSION_DOOR_EXT    |
XR_SPATIAL_PLANE_SEMANTIC_LABEL_UNCATEGORIZED_EXT
| XR_SPATIAL_EXAMPLE_PLANE_SEMANTIC_LABEL_EXTENSION_WINDOW_EXT  |
XR_SPATIAL_PLANE_SEMANTIC_LABEL_UNCATEGORIZED_EXT
|====
```

12.46.13. New Macros

- XR_NULL_SPATIAL_BUFFER_ID_EXT
- XR_NULL_SPATIAL_ENTITY_ID_EXT

12.46.14. New Base Types

- `XrSpatialBufferIdEXT`
- `XrSpatialEntityIdEXT`

12.46.15. New Object Types

- XrSpatialContextEXT
- XrSpatialEntityEXT
- XrSpatialSnapshotEXT

12.46.16. New Commands

- xrCreateSpatialContextAsyncEXT
- xrCreateSpatialContextCompleteEXT
- xrCreateSpatialDiscoverySnapshotAsyncEXT
- xrCreateSpatialDiscoverySnapshotCompleteEXT
- xrCreateSpatialEntityFromIdEXT
- xrCreateSpatialUpdateSnapshotEXT
- xrDestroySpatialContextEXT
- xrDestroySpatialEntityEXT
- xrDestroySpatialSnapshotEXT
- xrEnumerateSpatialCapabilitiesEXT
- xrEnumerateSpatialCapabilityComponentTypesEXT
- xrEnumerateSpatialCapabilityFeaturesEXT
- xrGetSpatialBufferFloatEXT
- xrGetSpatialBufferStringEXT
- xrGetSpatialBufferUint16EXT
- xrGetSpatialBufferUint32EXT
- xrGetSpatialBufferUint8EXT
- xrGetSpatialBufferVector2fEXT
- xrGetSpatialBufferVector3fEXT

- xrQuerySpatialComponentDataEXT

12.46.17. New Structures

- XrCreateSpatialContextCompletionEXT
- XrCreateSpatialDiscoverySnapshotCompletionEXT
- XrCreateSpatialDiscoverySnapshotCompletionInfoEXT
- XrEventDataSpatialDiscoveryRecommendedEXT
- XrSpatialBounded2DDataEXT
- XrSpatialBufferEXT
- XrSpatialBufferGetInfoEXT
- XrSpatialCapabilityComponentTypesEXT
- XrSpatialCapabilityConfigurationBaseHeaderEXT
- XrSpatialComponentDataQueryConditionEXT
- XrSpatialComponentDataQueryResultEXT
- XrSpatialContextCreateInfoEXT
- XrSpatialDiscoverySnapshotCreateInfoEXT
- XrSpatialEntityFromIdCreateInfoEXT
- XrSpatialMeshDataEXT
- XrSpatialUpdateSnapshotCreateInfoEXT
- Extending XrSpatialComponentDataQueryResultEXT:
  - XrSpatialComponentBounded2DListEXT
  - XrSpatialComponentBounded3DListEXT
  - XrSpatialComponentMesh3DListEXT
  - XrSpatialComponentParentListEXT
- Extending XrSpatialDiscoverySnapshotCreateInfoEXT, XrSpatialComponentDataQueryConditionEXT:
  - XrSpatialFilterTrackingStateEXT

12.46.18. New Enums

- XrSpatialBufferTypeEXT
- XrSpatialCapabilityEXT
- XrSpatialCapabilityFeatureEXT
- XrSpatialComponentTypeEXT

- XrSpatialEntityTrackingStateEXT

12.46.19. New Enum Constants

- XR_EXT_SPATIAL_ENTITY_EXTENSION_NAME
- XR_EXT_spatial_entity_SPEC_VERSION
- Extending XrObjectType:
  - XR_OBJECT_TYPE_SPATIAL_CONTEXT_EXT
  - XR_OBJECT_TYPE_SPATIAL_ENTITY_EXT
  - XR_OBJECT_TYPE_SPATIAL_SNAPSHOT_EXT
- Extending XrResult:
  - XR_ERROR_SPATIAL_BUFFER_ID_INVALID_EXT
  - XR_ERROR_SPATIAL_CAPABILITY_CONFIGURATION_INVALID_EXT
  - XR_ERROR_SPATIAL_CAPABILITY_UNSUPPORTED_EXT
  - XR_ERROR_SPATIAL_COMPONENT_NOT_ENABLED_EXT
  - XR_ERROR_SPATIAL_COMPONENT_UNSUPPORTED_FOR_CAPABILITY_EXT
  - XR_ERROR_SPATIAL_ENTITY_ID_INVALID_EXT
- Extending XrStructureType:
  - XR_TYPE_CREATE_SPATIAL_CONTEXT_COMPLETION_EXT
  - XR_TYPE_CREATE_SPATIAL_DISCOVERY_SNAPSHOT_COMPLETION_EXT
  - XR_TYPE_CREATE_SPATIAL_DISCOVERY_SNAPSHOT_COMPLETION_INFO_EXT
  - XR_TYPE_EVENT_DATA_SPATIAL_DISCOVERY_RECOMMENDED_EXT
  - XR_TYPE_SPATIAL_BUFFER_GET_INFO_EXT
  - XR_TYPE_SPATIAL_CAPABILITY_COMPONENT_TYPES_EXT
  - XR_TYPE_SPATIAL_COMPONENT_BOUNDED_2D_LIST_EXT
  - XR_TYPE_SPATIAL_COMPONENT_BOUNDED_3D_LIST_EXT
  - XR_TYPE_SPATIAL_COMPONENT_DATA_QUERY_CONDITION_EXT
  - XR_TYPE_SPATIAL_COMPONENT_DATA_QUERY_RESULT_EXT
  - XR_TYPE_SPATIAL_COMPONENT_MESH_3D_LIST_EXT
  - XR_TYPE_SPATIAL_COMPONENT_PARENT_LIST_EXT
  - XR_TYPE_SPATIAL_CONTEXT_CREATE_INFO_EXT
  - XR_TYPE_SPATIAL_DISCOVERY_SNAPSHOT_CREATE_INFO_EXT
  - XR_TYPE_SPATIAL_ENTITY_FROM_ID_CREATE_INFO_EXT
  - XR_TYPE_SPATIAL_FILTER_TRACKING_STATE_EXT

- `XR_TYPE_SPATIAL_UPDATE_SNAPSHOT_CREATE_INFO_EXT`

12.46.20. Issues

- Does a single entity always derive from solely a single capability?
  - Resolved
  - Answer: No. It is completely upto the runtime based on its own tracking capabilities and how it wants to represent a detected entity. The spec does not prescribe any particular representation of spatial entity except for the guaranteed components of a given capability to set a minimum expectation. A runtime **may** be able to merge entities detected by separate capabilities and represent them as a single entity with the guaranteed components of all the capabilities that helped identify it. An example of this could be that tables can be detected by both a plane tracking capability and an object tracking capability, with plane tracking providing the `XR_SPATIAL_COMPONENT_TYPE_BOUNDED_2D_EXT` component on the entity and object tracking providing `XR_SPATIAL_COMPONENT_TYPE_BOUNDED_3D_EXT`. A certain runtime **may** provide the table as 2 separate entities, each with their own set of guaranteed components, while certain runtimes **may** provide just 1 entity to represent the table, and have both `XR_SPATIAL_COMPONENT_TYPE_BOUNDED_2D_EXT` and `XR_SPATIAL_COMPONENT_TYPE_BOUNDED_3D_EXT` on the same entity. What is important to note here is that a given spatial entity can have at most a single *component* of any given *component type*. Therefore, if the component data produced by the different capabilities conflicts for a certain entity, the runtime **must** represent them as 2 separate entities.

12.46.21. Version History

- Revision 1, 2024-04-12 (Nihav Jain, Google)
  - Initial extension description

12.47. XR_EXT_spatial_marker_tracking

Name String

`XR_EXT_spatial_marker_tracking`

Extension Type

Instance extension

Registered Extension Number

744

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

XR_EXT_spatial_entity

Contributors

Ron Bessems, Meta
Nihav Jain, Google
Natalie Fleury, Meta
Yuichi Taguchi, Meta
Yin Li, Microsoft
Jimmy Alamparambil, ByteDance
Zhipeng Liu, ByteDance
Jun Yan, ByteDance

12.47.1. Overview

This extension builds on XR_EXT_spatial_entity and allows applications to detect and track markers in their environment. Markers are 2D codes which **may** include QR Codes, Micro QR Codes, ArUco markers, or AprilTags.

A tracked marker is represented as a spatial entity with (or "that has") the following components:

- XR_SPATIAL_COMPONENT_TYPE_MARKER_EXT
- XR_SPATIAL_COMPONENT_TYPE_BOUNDED_2D_EXT

12.47.2. Runtime support

A runtime **must** advertise its support for the various marker tracking capabilities using xrEnumerateSpatialCapabilitiesEXT by listing any of the following capabilities:

- XR_SPATIAL_CAPABILITY_MARKER_TRACKING_QR_CODE_EXT
- XR_SPATIAL_CAPABILITY_MARKER_TRACKING_MICRO_QR_CODE_EXT
- XR_SPATIAL_CAPABILITY_MARKER_TRACKING_ARUCO_MARKER_EXT
- XR_SPATIAL_CAPABILITY_MARKER_TRACKING_APRIL_TAG_EXT

12.47.3. Configuration

To enable detection of a marker type the application **must** pass the corresponding configuration structure to xrCreateSpatialContextAsyncEXT.

Marker Type Configurations

QR codes

The XrSpatialCapabilityConfigurationQrCodeEXT structure is defined as:

```
// Provided by XR_EXT_spatial_marker_tracking
typedef struct XrSpatialCapabilityConfigurationQrCodeEXT {
    XrStructureType                     type;
    const void*                         next;
    XrSpatialCapabilityEXT              capability;
    uint32_t                            enabledComponentCount;
    const XrSpatialComponentTypeEXT*    enabledComponents;
} XrSpatialCapabilityConfigurationQrCodeEXT;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `capability` **must** be `XR_SPATIAL_CAPABILITY_MARKER_TRACKING_QR_CODE_EXT`.
- `enabledComponentCount` is a `uint32_t` with the number of elements in `enabledComponents`.
- `enabledComponents` is a pointer to an array of components to enable for this capability.

If QR codes are supported, the runtime **must** enable QR Code tracking when an XrSpatialCapabilityConfigurationQrCodeEXT structure is passed in XrSpatialContextCreateInfoEXT ::`capabilityConfigs` when calling xrCreateSpatialContextAsyncEXT.

The runtime **must** return `XR_ERROR_SPATIAL_CAPABILITY_UNSUPPORTED_EXT` if `XR_SPATIAL_CAPABILITY_MARKER_TRACKING_QR_CODE_EXT` is not enumerated by xrEnumerateSpatialCapabilitiesEXT.

Valid Usage (Implicit)

- The XR_EXT_spatial_marker_tracking extension **must** be enabled prior to using XrSpatialCapabilityConfigurationQrCodeEXT
- type **must** be XR_TYPE_SPATIAL_CAPABILITY_CONFIGURATION_QR_CODE_EXT
- next **must** be NULL or a valid pointer to the next structure in a structure chain. See also: XrSpatialMarkerSizeEXT, XrSpatialMarkerStaticOptimizationEXT
- capability **must** be a valid XrSpatialCapabilityEXT value
- enabledComponents **must** be a pointer to an array of enabledComponentCount valid XrSpatialComponentTypeEXT values
- The enabledComponentCount parameter **must** be greater than 0

Micro QR codes

The XrSpatialCapabilityConfigurationMicroQrCodeEXT structure is defined as:

```
// Provided by XR_EXT_spatial_marker_tracking
typedef struct XrSpatialCapabilityConfigurationMicroQrCodeEXT {
    XrStructureType                     type;
    const void*                         next;
    XrSpatialCapabilityEXT              capability;
    uint32_t                            enabledComponentCount;
    const XrSpatialComponentTypeEXT*    enabledComponents;
} XrSpatialCapabilityConfigurationMicroQrCodeEXT;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain.
- capability **must** be XR_SPATIAL_CAPABILITY_MARKER_TRACKING_MICRO_QR_CODE_EXT.
- enabledComponentCount is a uint32_t with the number of elements in enabledComponents.
- enabledComponents is a pointer to an array of components to enable for this capability.

If Micro QR codes are supported, the runtime **must** enable Micro QR Code tracking when an XrSpatialCapabilityConfigurationMicroQrCodeEXT structure is passed in XrSpatialContextCreateInfoEXT::capabilityConfigs when calling xrCreateSpatialContextAsyncEXT.

The runtime **must** return XR_ERROR_SPATIAL_CAPABILITY_UNSUPPORTED_EXT if XR_SPATIAL_CAPABILITY_MARKER_TRACKING_MICRO_QR_CODE_EXT is not enumerated by

xrEnumerateSpatialCapabilitiesEXT.

Valid Usage (Implicit)

- The XR_EXT_spatial_marker_tracking extension **must** be enabled prior to using XrSpatialCapabilityConfigurationMicroQrCodeEXT
- type **must** be XR_TYPE_SPATIAL_CAPABILITY_CONFIGURATION_MICRO_QR_CODE_EXT
- next **must** be NULL or a valid pointer to the next structure in a structure chain. See also: XrSpatialMarkerSizeEXT, XrSpatialMarkerStaticOptimizationEXT
- capability **must** be a valid XrSpatialCapabilityEXT value
- enabledComponents **must** be a pointer to an array of enabledComponentCount valid XrSpatialComponentTypeEXT values
- The enabledComponentCount parameter **must** be greater than 0

ArUco Markers

The XrSpatialCapabilityConfigurationArucoMarkerEXT structure is defined as:

```
// Provided by XR_EXT_spatial_marker_tracking
typedef struct XrSpatialCapabilityConfigurationArucoMarkerEXT {
    XrStructureType                     type;
    const void*                         next;
    XrSpatialCapabilityEXT              capability;
    uint32_t                            enabledComponentCount;
    const XrSpatialComponentTypeEXT*    enabledComponents;
    XrSpatialMarkerArucoDictEXT         arUcoDict;
} XrSpatialCapabilityConfigurationArucoMarkerEXT;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain.
- capability **must** be XR_SPATIAL_CAPABILITY_MARKER_TRACKING_ARUCO_MARKER_EXT.
- enabledComponentCount is a uint32_t with the number of elements in enabledComponents.
- enabledComponents is a pointer to an array of components to enable for this capability.
- arUcoDict is the marker dictionary to detect.

If ArUco markers are supported, the runtime **must** enable ArUco marker tracking when an XrSpatialCapabilityConfigurationArucoMarkerEXT structure is passed in

XrSpatialContextCreateInfoEXT::capabilityConfigs when calling xrCreateSpatialContextAsyncEXT.

The runtime **must** return XR_ERROR_SPATIAL_CAPABILITY_UNSUPPORTED_EXT from xrCreateSpatialContextAsyncEXT if an XrSpatialCapabilityConfigurationArucoMarkerEXT structure is in XrSpatialContextCreateInfoEXT::capabilityConfigs but XR_SPATIAL_CAPABILITY_MARKER_TRACKING_ARUCO_MARKER_EXT is not enumerated by xrEnumerateSpatialCapabilitiesEXT.

Valid Usage (Implicit)

- The XR_EXT_spatial_marker_tracking extension **must** be enabled prior to using XrSpatialCapabilityConfigurationArucoMarkerEXT
- type **must** be XR_TYPE_SPATIAL_CAPABILITY_CONFIGURATION_ARUCO_MARKER_EXT
- next **must** be NULL or a valid pointer to the next structure in a structure chain. See also: XrSpatialMarkerSizeEXT, XrSpatialMarkerStaticOptimizationEXT
- capability **must** be a valid XrSpatialCapabilityEXT value
- enabledComponents **must** be a pointer to an array of enabledComponentCount valid XrSpatialComponentTypeEXT values
- arUcoDict **must** be a valid XrSpatialMarkerArucoDictEXT value
- The enabledComponentCount parameter **must** be greater than 0

The XrSpatialMarkerArucoDictEXT enumeration is defined as:

```
// Provided by XR_EXT_spatial_marker_tracking
typedef enum XrSpatialMarkerArucoDictEXT {
    XR_SPATIAL_MARKER_ARUCO_DICT_4X4_50_EXT = 1,
    XR_SPATIAL_MARKER_ARUCO_DICT_4X4_100_EXT = 2,
    XR_SPATIAL_MARKER_ARUCO_DICT_4X4_250_EXT = 3,
    XR_SPATIAL_MARKER_ARUCO_DICT_4X4_1000_EXT = 4,
    XR_SPATIAL_MARKER_ARUCO_DICT_5X5_50_EXT = 5,
    XR_SPATIAL_MARKER_ARUCO_DICT_5X5_100_EXT = 6,
    XR_SPATIAL_MARKER_ARUCO_DICT_5X5_250_EXT = 7,
    XR_SPATIAL_MARKER_ARUCO_DICT_5X5_1000_EXT = 8,
    XR_SPATIAL_MARKER_ARUCO_DICT_6X6_50_EXT = 9,
    XR_SPATIAL_MARKER_ARUCO_DICT_6X6_100_EXT = 10,
    XR_SPATIAL_MARKER_ARUCO_DICT_6X6_250_EXT = 11,
    XR_SPATIAL_MARKER_ARUCO_DICT_6X6_1000_EXT = 12,
    XR_SPATIAL_MARKER_ARUCO_DICT_7X7_50_EXT = 13,
    XR_SPATIAL_MARKER_ARUCO_DICT_7X7_100_EXT = 14,
    XR_SPATIAL_MARKER_ARUCO_DICT_7X7_250_EXT = 15,
    XR_SPATIAL_MARKER_ARUCO_DICT_7X7_1000_EXT = 16,
    XR_SPATIAL_MARKER_ARUCO_DICT_MAX_ENUM_EXT = 0x7FFFFFFF
} XrSpatialMarkerArucoDictEXT;
```

Supported predefined ArUco dictionary:

Enumerant Descriptions

- `XR_SPATIAL_MARKER_ARUCO_DICT_4X4_50_EXT` — 4 by 4 pixel Aruco marker dictionary with 50 IDs.
- `XR_SPATIAL_MARKER_ARUCO_DICT_4X4_100_EXT` — 4 by 4 pixel Aruco marker dictionary with 100 IDs.
- `XR_SPATIAL_MARKER_ARUCO_DICT_4X4_250_EXT` — 4 by 4 pixel Aruco marker dictionary with 250 IDs.
- `XR_SPATIAL_MARKER_ARUCO_DICT_4X4_1000_EXT` — 4 by 4 pixel Aruco marker dictionary with 1000 IDs.
- `XR_SPATIAL_MARKER_ARUCO_DICT_5X5_50_EXT` — 5 by 5 pixel Aruco marker dictionary with 50 IDs.
- `XR_SPATIAL_MARKER_ARUCO_DICT_5X5_100_EXT` — 5 by 5 pixel Aruco marker dictionary with 100 IDs.
- `XR_SPATIAL_MARKER_ARUCO_DICT_5X5_250_EXT` — 5 by 5 pixel Aruco marker dictionary with 250 IDs.
- `XR_SPATIAL_MARKER_ARUCO_DICT_5X5_1000_EXT` — 5 by 5 pixel Aruco marker dictionary with 1000 IDs.
- `XR_SPATIAL_MARKER_ARUCO_DICT_6X6_50_EXT` — 6 by 6 pixel Aruco marker dictionary with 50 IDs.
- `XR_SPATIAL_MARKER_ARUCO_DICT_6X6_100_EXT` — 6 by 6 pixel Aruco marker dictionary with 100 IDs.
- `XR_SPATIAL_MARKER_ARUCO_DICT_6X6_250_EXT` — 6 by 6 pixel Aruco marker dictionary with 250 IDs.
- `XR_SPATIAL_MARKER_ARUCO_DICT_6X6_1000_EXT` — 6 by 6 pixel Aruco marker dictionary with 1000 IDs.
- `XR_SPATIAL_MARKER_ARUCO_DICT_7X7_50_EXT` — 7 by 7 pixel Aruco marker dictionary with 50 IDs.
- `XR_SPATIAL_MARKER_ARUCO_DICT_7X7_100_EXT` — 7 by 7 pixel Aruco marker dictionary with 100 IDs.
- `XR_SPATIAL_MARKER_ARUCO_DICT_7X7_250_EXT` — 7 by 7 pixel Aruco marker dictionary with 250 IDs.
- `XR_SPATIAL_MARKER_ARUCO_DICT_7X7_1000_EXT` — 7 by 7 pixel Aruco marker dictionary with 1000 IDs.

AprilTags

The XrSpatialCapabilityConfigurationAprilTagEXT structure is defined as:

```
// Provided by XR_EXT_spatial_marker_tracking
typedef struct XrSpatialCapabilityConfigurationAprilTagEXT {
    XrStructureType                     type;
    const void*                         next;
    XrSpatialCapabilityEXT              capability;
    uint32_t                            enabledComponentCount;
    const XrSpatialComponentTypeEXT*    enabledComponents;
    XrSpatialMarkerAprilTagDictEXT      aprilDict;
} XrSpatialCapabilityConfigurationAprilTagEXT;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `capability` **must** be `XR_SPATIAL_CAPABILITY_MARKER_TRACKING_APRIL_TAG_EXT`.
- `enabledComponentCount` is a `uint32_t` with the number of elements in `enabledComponents`.
- `enabledComponents` is a pointer to an array of components to enable for this capability.
- `aprilDict` is the marker dictionary to detect.

If AprilTags are supported, the runtime **must** enable AprilTag tracking when an XrSpatialCapabilityConfigurationAprilTagEXT structure is passed in XrSpatialContextCreateInfoEXT ::`capabilityConfigs` when calling xrCreateSpatialContextAsyncEXT.

The runtime **must** return `XR_ERROR_SPATIAL_CAPABILITY_UNSUPPORTED_EXT` from xrCreateSpatialContextAsyncEXT if an XrSpatialCapabilityConfigurationAprilTagEXT structure is in XrSpatialContextCreateInfoEXT::`capabilityConfigs` but `XR_SPATIAL_CAPABILITY_MARKER_TRACKING_APRIL_TAG_EXT` is not enumerated by xrEnumerateSpatialCapabilitiesEXT.

Valid Usage (Implicit)

- The `XR_EXT_spatial_marker_tracking` extension **must** be enabled prior to using XrSpatialCapabilityConfigurationAprilTagEXT
- `type` **must** be `XR_TYPE_SPATIAL_CAPABILITY_CONFIGURATION_APRIL_TAG_EXT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain. See also: XrSpatialMarkerSizeEXT, XrSpatialMarkerStaticOptimizationEXT
- `capability` **must** be a valid XrSpatialCapabilityEXT value
- `enabledComponents` **must** be a pointer to an array of `enabledComponentCount` valid XrSpatialComponentTypeEXT values
- `aprilDict` **must** be a valid XrSpatialMarkerAprilTagDictEXT value
- The `enabledComponentCount` parameter **must** be greater than `0`

The XrSpatialMarkerAprilTagDictEXT enumeration is defined as:

```
// Provided by XR_EXT_spatial_marker_tracking
typedef enum XrSpatialMarkerAprilTagDictEXT {
    XR_SPATIAL_MARKER_APRIL_TAG_DICT_16H5_EXT = 1,
    XR_SPATIAL_MARKER_APRIL_TAG_DICT_25H9_EXT = 2,
    XR_SPATIAL_MARKER_APRIL_TAG_DICT_36H10_EXT = 3,
    XR_SPATIAL_MARKER_APRIL_TAG_DICT_36H11_EXT = 4,
    XR_SPATIAL_MARKER_APRIL_TAG_DICT_MAX_ENUM_EXT = 0x7FFFFFFF
} XrSpatialMarkerAprilTagDictEXT;
```

Supported predefined AprilTag dictionary:

Enumerant Descriptions

- `XR_SPATIAL_MARKER_APRIL_TAG_DICT_16H5_EXT` — 4 by 4 bits, minimum Hamming distance between any two codes = 5, 30 codes.
- `XR_SPATIAL_MARKER_APRIL_TAG_DICT_25H9_EXT` — 5 by 5 bits, minimum Hamming distance between any two codes = 9, 35 codes.
- `XR_SPATIAL_MARKER_APRIL_TAG_DICT_36H10_EXT` — 6 by 6 bits, minimum Hamming distance between any two codes = 10, 2320 codes.
- `XR_SPATIAL_MARKER_APRIL_TAG_DICT_36H11_EXT` — 6 by 6 bits, minimum Hamming distance between any two codes = 11, 587 codes.

Optional Marker Configurations

Applications **should** call xrEnumerateSpatialCapabilityFeaturesEXT to get the list of supported optional features.

See XrSpatialCapabilityFeatureEXT for a complete list of all spatial capability features supported by any extension.

Marker Size

The XrSpatialMarkerSizeEXT structure is defined as:

```
// Provided by XR_EXT_spatial_marker_tracking
typedef struct XrSpatialMarkerSizeEXT {
    XrStructureType    type;
    const void*        next;
    float              markerSideLength;
} XrSpatialMarkerSizeEXT;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `markerSideLength` is the size in meters of all markers.

If `XR_SPATIAL_CAPABILITY_FEATURE_MARKER_TRACKING_FIXED_SIZE_MARKERS_EXT` is enumerated by xrEnumerateSpatialCapabilityFeaturesEXT for a certain capability, and if the application chains XrSpatialMarkerSizeEXT to the corresponding configuration structure of that capability, the runtime **must** assume that all markers detected have width and height of `markerSideLength`. Providing this information to the runtime allows the runtime to return a more accurate pose and size. This structure **must** be linked into the `next` chain of XrSpatialCapabilityConfigurationQrCodeEXT, XrSpatialCapabilityConfigurationMicroQrCodeEXT, XrSpatialCapabilityConfigurationArucoMarkerEXT, or XrSpatialCapabilityConfigurationAprilTagEXT.

Valid Usage (Implicit)

- The `XR_EXT_spatial_marker_tracking` extension **must** be enabled prior to using XrSpatialMarkerSizeEXT
- `type` **must** be `XR_TYPE_SPATIAL_MARKER_SIZE_EXT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

Static Marker Optimization

The XrSpatialMarkerStaticOptimizationEXT structure is defined as:

```
// Provided by XR_EXT_spatial_marker_tracking
typedef struct XrSpatialMarkerStaticOptimizationEXT {
    XrStructureType    type;
    const void*        next;
    XrBool32           optimizeForStaticMarker;
} XrSpatialMarkerStaticOptimizationEXT;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `optimizeForStaticMarker` indicates if all markers in the space are expected to not move.

If `XR_SPATIAL_CAPABILITY_FEATURE_MARKER_TRACKING_STATIC_MARKERS_EXT` is enumerated by xrEnumerateSpatialCapabilityFeaturesEXT for a certain capability, and if the application chains XrSpatialMarkerStaticOptimizationEXT to the corresponding configuration structure of that capability, the runtime **must** assume that all markers detected are static if `optimizeForStaticMarker` is set to `XR_TRUE`. This allows the runtime to generate a more accurate pose and size. This structure **must** be linked into the `next` chain of XrSpatialCapabilityConfigurationQrCodeEXT, XrSpatialCapabilityConfigurationMicroQrCodeEXT, XrSpatialCapabilityConfigurationArucoMarkerEXT, or XrSpatialCapabilityConfigurationAprilTagEXT.

Valid Usage (Implicit)

- The `XR_EXT_spatial_marker_tracking` extension **must** be enabled prior to using XrSpatialMarkerStaticOptimizationEXT
- `type` **must** be `XR_TYPE_SPATIAL_MARKER_STATIC_OPTIMIZATION_EXT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

12.47.4. Guaranteed Components

A runtime that supports `XR_SPATIAL_CAPABILITY_MARKER_TRACKING_QR_CODE_EXT`, `XR_SPATIAL_CAPABILITY_MARKER_TRACKING_MICRO_QR_CODE_EXT`, `XR_SPATIAL_CAPABILITY_MARKER_TRACKING_ARUCO_MARKER_EXT`, or `XR_SPATIAL_CAPABILITY_MARKER_TRACKING_APRIL_TAG_EXT` **must** provide the following spatial components as guaranteed components of all entities discovered by those capabilities, and **must** enumerate them in xrEnumerateSpatialCapabilityComponentTypesEXT:

- XR_SPATIAL_COMPONENT_TYPE_MARKER_EXT
- XR_SPATIAL_COMPONENT_TYPE_BOUNDED_2D_EXT

Marker Component

Component data

The XrSpatialMarkerDataEXT structure is defined as:

```
// Provided by XR_EXT_spatial_marker_tracking
typedef struct XrSpatialMarkerDataEXT {
    XrSpatialCapabilityEXT    capability;
    uint32_t                  markerId;
    XrSpatialBufferEXT        data;
} XrSpatialMarkerDataEXT;
```

Member Descriptions

- capability is the XrSpatialCapabilityEXT that detected the marker.
- markerId is the encoded identifier from the marker. For ArUco markers and AprilTag this field **must** be valid and filled with the encoded ID. For QR codes this field **must** be zero.
- data is the buffer ID and type of additional information contained in the marker.

XR_SPATIAL_CAPABILITY_MARKER_TRACKING_QR_CODE_EXT and XR_SPATIAL_CAPABILITY_MARKER_TRACKING_MICRO_QR_CODE_EXT support extra data. If capability is one of these -

- If the runtime has successfully decoded the data for the marker, it **must** set the data buffer type to either XR_SPATIAL_BUFFER_TYPE_UINT8_EXT or XR_SPATIAL_BUFFER_TYPE_STRING_EXT, depending on the data in the marker. The runtime **must** also set a valid buffer ID in data which the application **can** use with the appropriate xrGetSpatialBuffer* function to get the data.
- If the runtime has not yet decoded the data of the marker, it **must** set data buffer ID to XR_NULL_SPATIAL_BUFFER_ID_EXT and the buffer type to XR_SPATIAL_BUFFER_TYPE_UNKNOWN_EXT.

XR_SPATIAL_CAPABILITY_MARKER_TRACKING_ARUCO_MARKER_EXT and XR_SPATIAL_CAPABILITY_MARKER_TRACKING_APRIL_TAG_EXT do not support extra data and the runtime **must** set the buffer ID of data to XR_NULL_SPATIAL_BUFFER_ID_EXT.

Valid Usage (Implicit)

- The `XR_EXT_spatial_marker_tracking` extension **must** be enabled prior to using XrSpatialMarkerDataEXT
- `capability` **must** be a valid XrSpatialCapabilityEXT value
- `data` **must** be a valid XrSpatialBufferEXT structure

Component list structure to query data

The XrSpatialComponentMarkerListEXT structure is defined as:

```
// Provided by XR_EXT_spatial_marker_tracking
typedef struct XrSpatialComponentMarkerListEXT {
    XrStructureType           type;
    void*                     next;
    uint32_t                  markerCount;
    XrSpatialMarkerDataEXT*   markers;
} XrSpatialComponentMarkerListEXT;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `markerCount` is the number of elements in the `markers` member.
- `markers` is an array of XrSpatialMarkerDataEXT.

The application **can** query the marker component of the spatial entities in an XrSpatialSnapshotEXT by adding `XR_SPATIAL_COMPONENT_TYPE_MARKER_EXT` in XrSpatialComponentDataQueryConditionEXT::`componentTypes` and adding XrSpatialComponentMarkerListEXT to the next pointer chain of XrSpatialComponentDataQueryResultEXT.

The runtime **must** return `XR_ERROR_VALIDATION_FAILURE` from xrQuerySpatialComponentDataEXT if XrSpatialComponentMarkerListEXT is in the next chain of XrSpatialComponentDataQueryResultEXT::`next` but `XR_SPATIAL_COMPONENT_TYPE_MARKER_EXT` is not included in XrSpatialComponentDataQueryConditionEXT::`componentTypes`.

The runtime **must** return `XR_ERROR_SIZE_INSUFFICIENT` from xrQuerySpatialComponentDataEXT if `markerCount` is less than XrSpatialComponentDataQueryResultEXT::`entityIdCountOutput`.

Valid Usage (Implicit)

- The XR_EXT_spatial_marker_tracking extension **must** be enabled prior to using XrSpatialComponentMarkerListEXT
- type **must** be XR_TYPE_SPATIAL_COMPONENT_MARKER_LIST_EXT
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- markers **must** be a pointer to an array of markerCount XrSpatialMarkerDataEXT structures
- The markerCount parameter **must** be greater than 0

Configuration

If XR_SPATIAL_COMPONENT_TYPE_MARKER_EXT is enumerated in XrSpatialCapabilityComponentTypesEXT::componentTypes for some capability, an application **can** enable it by including the enumerant in the XrSpatialCapabilityConfigurationBaseHeaderEXT::enabledComponents list of the XrSpatialCapabilityConfigurationBaseHeaderEXT derived structure of the capability that supports this component.

This component does not require any special configuration to be included in the XrSpatialCapabilityConfigurationBaseHeaderEXT::next chain.

Bounded 2D Component

The bounded 2D component provides the center and extents of the marker represented by the entity it is on. See Bounded 2D for more details about the bounded 2D component.

The XrSpatialBounded2DDataEXT::center **must** point to the center of the marker. When looking at the front face of the marker, the X-axis **must** point to the right, and the Y-axis **must** point to the top of the marker. The runtime **must** follow the right-handed coordinate system convention thus the Z-axis comes out of the front face of the marker. This means that a marker with a position of {0, 0, 0}, rotation of {0, 0, 0, 1} (no rotation), and an extent of {1, 1} refers to a 1 meter x 1 meter marker centered at {0, 0, 0} with its front face normal vector pointing towards the +Z direction in the component's space.

A representation of the orientation of the marker is shown below.

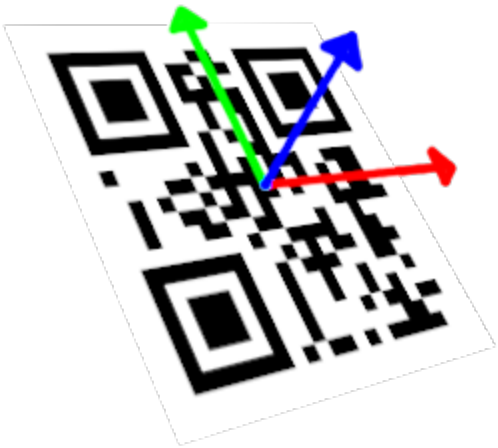


Figure 22. QR code marker with axis

12.47.5. Test Codes

The following codes **must** have their X-Y plane inside the document and the Z-axis pointing at the viewer. The axis origin **must** appear at the center of each marker. The X-axis **must** point to the right, the Y-axis **must** point to the top of the document.

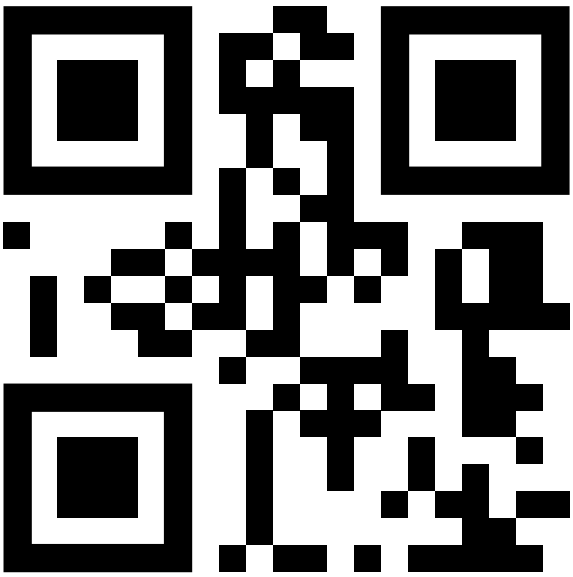


Figure 23. QR code with text 'OpenXR'

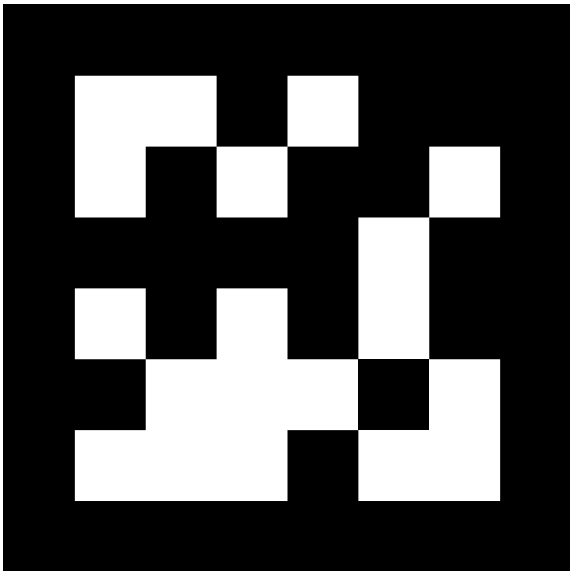


Figure 24. AprilTag `XR_SPATIAL_MARKER_APRIL_TAG_DICT_36H11_EXT` *with ID 42*

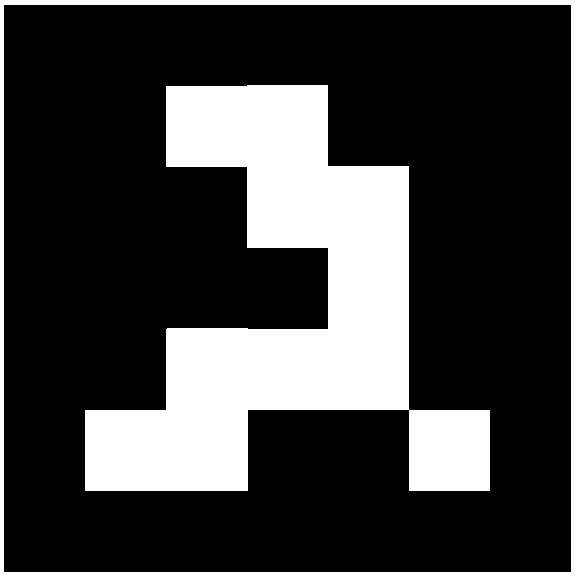


Figure 25. ArUco marker `XR_SPATIAL_MARKER_ARUCO_DICT_5X5_50_EXT` *with ID 43*

12.47.6. Example Code

Configure QR Code Tracking Capability

The following example code demonstrates how to configure the QR code tracking capability when creating a spatial context.

```
// Check if marker tracking capability is supported
uint32_t capabilityCount;
CHK_XR(xrEnumerateSpatialCapabilitiesEXT(instance, systemId, 0, &capabilityCount,
nullptr));
std::vector<XrSpatialCapabilityEXT> capabilities(capabilityCount);
CHK_XR(xrEnumerateSpatialCapabilitiesEXT(instance, systemId, capabilityCount,
&capabilityCount, capabilities.data()));
```

```
if (std::find(capabilities.begin(), capabilities.end(),
XR_SPATIAL_CAPABILITY_MARKER_TRACKING_QR_CODE_EXT) == capabilities.end()) {
  return;
}

uint32_t featureCount = 0;
CHK_XR(xrEnumerateSpatialCapabilityFeaturesEXT(instance, systemId,
XR_SPATIAL_CAPABILITY_MARKER_TRACKING_QR_CODE_EXT, 0, &featureCount, nullptr));
std::vector<XrSpatialCapabilityFeatureEXT> capabilityFeatures(featureCount);
CHK_XR(xrEnumerateSpatialCapabilityFeaturesEXT(instance, systemId,
XR_SPATIAL_CAPABILITY_MARKER_TRACKING_QR_CODE_EXT, featureCount, &featureCount,
capabilityFeatures.data()));

bool supportsFixedMarkerSize = std::find(capabilityFeatures.begin(), capabilityFeatures
.end(), XR_SPATIAL_CAPABILITY_FEATURE_MARKER_TRACKING_FIXED_SIZE_MARKERS_EXT) !=
capabilityFeatures.end();

// Create a spatial context
XrSpatialContextEXT spatialContext{};

// Enable the 2 guaranteed components of the qr code tracking capability
std::vector<XrSpatialComponentTypeEXT> enabledComponents = {
  XR_SPATIAL_COMPONENT_TYPE_BOUNDED_2D_EXT,
  XR_SPATIAL_COMPONENT_TYPE_MARKER_EXT,
};
XrSpatialCapabilityConfigurationQrCodeEXT markerConfiguration {
  .type = XR_TYPE_SPATIAL_CAPABILITY_CONFIGURATION_QR_CODE_EXT,
  .capability = XR_SPATIAL_CAPABILITY_MARKER_TRACKING_QR_CODE_EXT,
  .enabledComponentCount = static_cast<uint32_t>(enabledComponents.size()),
  .enabledComponents = enabledComponents.data(),
};

// only chained if features.markerSideLength is true.
XrSpatialMarkerSizeEXT markerSize {
  .type = XR_TYPE_SPATIAL_MARKER_SIZE_EXT,
  .markerSideLength = 0.10f,
};
if (supportsFixedMarkerSize) {
  markerConfiguration.next = &markerSize;
}

std::array<XrSpatialCapabilityConfigurationBaseHeaderEXT*, 1> capabilityConfigs = {
  reinterpret_cast<XrSpatialCapabilityConfigurationBaseHeaderEXT*>(&markerConfiguration),
};

XrSpatialContextCreateInfoEXT spatialContextCreateInfo {
  .type = XR_TYPE_SPATIAL_CONTEXT_CREATE_INFO_EXT,
```

```
  .capabilityConfigCount = capabilityConfigs.size(),
  .capabilityConfigs = capabilityConfigs.data(),
};
XrFutureEXT createContextFuture {XR_NULL_FUTURE_EXT};
CHK_XR(xrCreateSpatialContextAsyncEXT(session, &spatialContextCreateInfo,
&createContextFuture));

waitUntilReady(createContextFuture);

XrCreateSpatialContextCompletionEXT completion {
  .type = XR_TYPE_CREATE_SPATIAL_CONTEXT_COMPLETION_EXT,
};
CHK_XR(xrCreateSpatialContextCompleteEXT(session, createContextFuture, &completion));
if (completion.futureResult != XR_SUCCESS) {
  return;
}

spatialContext = completion.spatialContext;

// ...
// Discovery entities with the spatial context
// ...

CHK_XR(xrDestroySpatialContextEXT(spatialContext));
```

Discover Spatial Entities & Query Component Data

The following example code demonstrates how to discover spatial entities for a context configured with XR_SPATIAL_CAPABILITY_MARKER_TRACKING_QR_CODE_EXT and query its component data.

```
XrFutureEXT future = XR_NULL_FUTURE_EXT;

// We want to look for entities that have the following components.
std::vector<XrSpatialComponentTypeEXT> snapshotComponents = {
  XR_SPATIAL_COMPONENT_TYPE_BOUNDED_2D_EXT,
  XR_SPATIAL_COMPONENT_TYPE_MARKER_EXT,
};

auto discoverSpatialEntities = [&](XrSpatialContextEXT spatialContext, XrTime time) {
  XrSpatialDiscoverySnapshotCreateInfoEXT snapshotCreateInfo {
    .type = XR_TYPE_SPATIAL_DISCOVERY_SNAPSHOT_CREATE_INFO_EXT,
    .componentTypeCount = static_cast<uint32_t>(snapshotComponents.size()),
    .componentTypes = snapshotComponents.data(),
  };
  CHK_XR(xrCreateSpatialDiscoverySnapshotAsyncEXT(spatialContext, &snapshotCreateInfo,
&future));
```

```
  waitUntilReady(future);

  XrCreateSpatialDiscoverySnapshotCompletionInfoEXT completionInfo {
    .type = XR_TYPE_CREATE_SPATIAL_DISCOVERY_SNAPSHOT_COMPLETION_INFO_EXT,
    .baseSpace = localSpace,
    .time = time,
    .future = future,
  };

  XrCreateSpatialDiscoverySnapshotCompletionEXT completion {
    .type = XR_TYPE_CREATE_SPATIAL_DISCOVERY_SNAPSHOT_COMPLETION_EXT,
  };
  CHK_XR(xrCreateSpatialDiscoverySnapshotCompleteEXT(spatialContext, &completionInfo,
&completion));
  if (completion.futureResult == XR_SUCCESS) {

    // Query for the bounded2D and marker component data
    XrSpatialComponentDataQueryConditionEXT queryCond {
      .type = XR_TYPE_SPATIAL_COMPONENT_DATA_QUERY_CONDITION_EXT,
      .componentTypeCount = static_cast<uint32_t>(snapshotComponents.size()),
      .componentTypes = snapshotComponents.data(),
    };

    XrSpatialComponentDataQueryResultEXT queryResult {
      .type = XR_TYPE_SPATIAL_COMPONENT_DATA_QUERY_RESULT_EXT,
    };
    CHK_XR(xrQuerySpatialComponentDataEXT(completion.snapshot, &queryCond, &
queryResult));

    std::vector<XrSpatialEntityIdEXT> entityIds(queryResult.entityIdCountOutput);
    std::vector<XrSpatialEntityTrackingStateEXT> entityStates(queryResult
.entityIdCountOutput);
    queryResult.entityIdCapacityInput = entityIds.size();
    queryResult.entityIds = entityIds.data();
    queryResult.entityStateCapacityInput = entityStates.size();
    queryResult.entityStates = entityStates.data();

    std::vector<XrSpatialBounded2DDataEXT> bounded2D(queryResult.entityIdCountOutput);
    XrSpatialComponentBounded2DListEXT bounded2DList {
      .type = XR_TYPE_SPATIAL_COMPONENT_BOUNDED_2D_LIST_EXT,
      .boundCount = static_cast<uint32_t>(bounded2D.size()),
      .bounds = bounded2D.data(),
    };
    queryResult.next = &bounded2DList;

    std::vector<XrSpatialMarkerDataEXT> markers(queryResult.entityIdCountOutput);
    XrSpatialComponentMarkerListEXT markerList {
```

```
      .type = XR_TYPE_SPATIAL_COMPONENT_MARKER_LIST_EXT,
      .markerCount = static_cast<uint32_t>(markers.size()),
      .markers = markers.data(),
    };
    bounded2DList.next = &markerList;

    CHK_XR(xrQuerySpatialComponentDataEXT(completion.snapshot, &queryCond, &
queryResult));

    for (int32_t i = 0; i < queryResult.entityIdCountOutput; ++i) {
      if (entityStates[i] != XR_SPATIAL_ENTITY_TRACKING_STATE_TRACKING_EXT) {
        continue;
      }

      // 2D bounds for entity entityIds[i] is bounded2D[i].extents centered on
bounded2D[i].center.

      if (markers[i].capability == XR_SPATIAL_CAPABILITY_MARKER_TRACKING_QR_CODE_EXT) {
        // Check if marker data has been decoded.
        if (markers[i].data.bufferId != XR_NULL_SPATIAL_BUFFER_ID_EXT) {
          if (markers[i].data.bufferType == XR_SPATIAL_BUFFER_TYPE_STRING_EXT) {
            // Qr Code data can be queried using
            // XrSpatialBufferGetInfoEXT getInfo {
            //   .type = XR_TYPE_SPATIAL_BUFFER_GET_INFO_EXT,
            //   .bufferId = markers[i].data.bufferId,
            // };
            // xrGetSpatialBufferStringEXT(completion.snapshot, &getInfo, ...)
          } else if (markers[i].data.bufferType == XR_SPATIAL_BUFFER_TYPE_UINT8_EXT) {
            // Qr Code data can be queried using
            // XrSpatialBufferGetInfoEXT getInfo {
            //   .type = XR_TYPE_SPATIAL_BUFFER_GET_INFO_EXT,
            //   .bufferId = markers[i].data.bufferId,
            // };
            // xrGetSpatialBufferUint8(completion.snapshot, &getInfo, ...)
          }
        }
      }
    }

    CHK_XR(xrDestroySpatialSnapshotEXT(completion.snapshot));
  }
};

while (1) {
  // ...
  // For every frame in frame loop
  // ...
```

```
  XrFrameState frameState;  // previously returned from xrWaitFrame
  const XrTime time = frameState.predictedDisplayTime;

  // Poll for the XR_TYPE_EVENT_DATA_SPATIAL_DISCOVERY_RECOMMENDED_EXT event
  XrEventDataBuffer event {
    .type = XR_TYPE_EVENT_DATA_BUFFER,
  };
  XrResult result = xrPollEvent(instance, &event);
  if (result == XR_SUCCESS) {
      switch (event.type) {
          case XR_TYPE_EVENT_DATA_SPATIAL_DISCOVERY_RECOMMENDED_EXT: {
              const XrEventDataSpatialDiscoveryRecommendedEXT& eventdata =
                  *reinterpret_cast<XrEventDataSpatialDiscoveryRecommendedEXT*>(&event);
              // Discover spatial entities for the context that we received the
"discovery
              // recommended" event for.
              discoverSpatialEntities(eventdata.spatialContext, time);
              break;
          }
      }
  }

  // ...
  // Finish frame loop
  // ...
}
```

12.47.7. New Structures

- XrSpatialCapabilityConfigurationAprilTagEXT
- XrSpatialCapabilityConfigurationArucoMarkerEXT
- XrSpatialCapabilityConfigurationMicroQrCodeEXT
- XrSpatialCapabilityConfigurationQrCodeEXT
- XrSpatialMarkerDataEXT
- Extending XrSpatialCapabilityConfigurationArucoMarkerEXT, XrSpatialCapabilityConfigurationAprilTagEXT, XrSpatialCapabilityConfigurationQrCodeEXT, XrSpatialCapabilityConfigurationMicroQrCodeEXT:
  - XrSpatialMarkerSizeEXT
  - XrSpatialMarkerStaticOptimizationEXT
- Extending XrSpatialComponentDataQueryResultEXT:
  - XrSpatialComponentMarkerListEXT

12.47.8. New Enums

- XrSpatialMarkerAprilTagDictEXT
- XrSpatialMarkerArucoDictEXT

12.47.9. New Enum Constants

- XR_EXT_SPATIAL_MARKER_TRACKING_EXTENSION_NAME
- XR_EXT_spatial_marker_tracking_SPEC_VERSION
- Extending XrSpatialCapabilityEXT:
  - XR_SPATIAL_CAPABILITY_MARKER_TRACKING_APRIL_TAG_EXT
  - XR_SPATIAL_CAPABILITY_MARKER_TRACKING_ARUCO_MARKER_EXT
  - XR_SPATIAL_CAPABILITY_MARKER_TRACKING_MICRO_QR_CODE_EXT
  - XR_SPATIAL_CAPABILITY_MARKER_TRACKING_QR_CODE_EXT
- Extending XrSpatialCapabilityFeatureEXT:
  - XR_SPATIAL_CAPABILITY_FEATURE_MARKER_TRACKING_FIXED_SIZE_MARKERS_EXT
  - XR_SPATIAL_CAPABILITY_FEATURE_MARKER_TRACKING_STATIC_MARKERS_EXT
- Extending XrSpatialComponentTypeEXT:
  - XR_SPATIAL_COMPONENT_TYPE_MARKER_EXT
- Extending XrStructureType:
  - XR_TYPE_SPATIAL_CAPABILITY_CONFIGURATION_APRIL_TAG_EXT
  - XR_TYPE_SPATIAL_CAPABILITY_CONFIGURATION_ARUCO_MARKER_EXT
  - XR_TYPE_SPATIAL_CAPABILITY_CONFIGURATION_MICRO_QR_CODE_EXT
  - XR_TYPE_SPATIAL_CAPABILITY_CONFIGURATION_QR_CODE_EXT
  - XR_TYPE_SPATIAL_COMPONENT_MARKER_LIST_EXT
  - XR_TYPE_SPATIAL_MARKER_SIZE_EXT
  - XR_TYPE_SPATIAL_MARKER_STATIC_OPTIMIZATION_EXT

12.47.10. Version History

- Revision 1, 2024-07-29 (Ron Bessems, Meta)
  - Initial extension description

12.48. XR_EXT_spatial_persistence

Name String

`XR_EXT_spatial_persistence`

Extension Type

Instance extension

Registered Extension Number

764

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

`XR_EXT_spatial_entity`
and
`XR_EXT_future`

Contributors

Nihav Jain, Google
Jared Finder, Google
Natalie Fleury, Meta
Yuichi Taguchi, Meta
Ron Bessems, Meta
Yin Li, Microsoft
Jimmy Alamparambil, ByteDance
Zhipeng Liu, ByteDance
Jun Yan, ByteDance

12.48.1. Overview

This extension allows applications to discover and correlate spatial entities across application sessions, OpenXR sessions and multiple OpenXR spatial contexts within a session. The `XR_EXT_spatial_entity` extension established that an entity within an XrSpatialContextEXT is represented by an `XrSpatialEntityIdEXT`. This extension extends on that concept by establishing that an entity, if persisted, is represented by an XrUuid across application and OpenXR sessions i.e. an application **can** use the XrUuid provided by this extension to identify an entity across sessions. This extension also provides useful overlaps with the `XR_EXT_spatial_entity` extension to discover persisted entities in the user's environment and the ability to query their component data.

12.48.2. Spatial Persistence Context

Create a spatial persistence context

```
// Provided by XR_EXT_spatial_persistence
XR_DEFINE_HANDLE(XrSpatialPersistenceContextEXT)
```

The XrSpatialPersistenceContextEXT handle represents the connection to a persistent spatial entity storage.

The xrCreateSpatialPersistenceContextAsyncEXT function is defined as:

```
// Provided by XR_EXT_spatial_persistence
XrResult xrCreateSpatialPersistenceContextAsyncEXT(
    XrSession                                       session,
    const XrSpatialPersistenceContextCreateInfoEXT* createInfo,
    XrFutureEXT*                                    future);
```

Parameter Descriptions

- `session` is an XrSession in which the spatial persistence context will be active.
- `createInfo` is the XrSpatialPersistenceContextCreateInfoEXT used to specify the spatial persistence context parameters.
- `future` is a pointer to an `XrFutureEXT`.

An application **can** create an XrSpatialPersistenceContextEXT handle using the xrCreateSpatialPersistenceContextAsyncEXT function and configure the scope of the persistence context in `createInfo`.

The runtime **must** return `XR_ERROR_SPATIAL_PERSISTENCE_SCOPE_UNSUPPORTED_EXT` if XrSpatialPersistenceContextCreateInfoEXT::`scope` is not enumerated by xrEnumerateSpatialPersistenceScopesEXT.

If a runtime enforces a permission system to control application access to the persistence storage represented by XrSpatialPersistenceContextEXT, then the runtime **must** return `XR_ERROR_PERMISSION_INSUFFICIENT` if those permissions have not been granted to this application.

Valid Usage (Implicit)

- The XR_EXT_spatial_persistence extension **must** be enabled prior to calling xrCreateSpatialPersistenceContextAsyncEXT
- `session` **must** be a valid XrSession handle
- `createInfo` **must** be a pointer to a valid XrSpatialPersistenceContextCreateInfoEXT structure
- `future` **must** be a pointer to an `XrFutureEXT` value

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_SPATIAL_PERSISTENCE_SCOPE_UNSUPPORTED_EXT
- XR_ERROR_VALIDATION_FAILURE

The XrSpatialPersistenceContextCreateInfoEXT structure is defined as:

```
// Provided by XR_EXT_spatial_persistence
typedef struct XrSpatialPersistenceContextCreateInfoEXT {
    XrStructureType                type;
    const void*                    next;
    XrSpatialPersistenceScopeEXT    scope;
} XrSpatialPersistenceContextCreateInfoEXT;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `scope` is an XrSpatialPersistenceScopeEXT defining the storage scope of the persistence context.

The XrSpatialPersistenceContextCreateInfoEXT structure describes the information to create an XrSpatialPersistenceContextEXT handle.

Valid Usage (Implicit)

- The `XR_EXT_spatial_persistence` extension **must** be enabled prior to using XrSpatialPersistenceContextCreateInfoEXT
- `type` **must** be `XR_TYPE_SPATIAL_PERSISTENCE_CONTEXT_CREATE_INFO_EXT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `scope` **must** be a valid XrSpatialPersistenceScopeEXT value

```
// Provided by XR_EXT_spatial_persistence
typedef enum XrSpatialPersistenceScopeEXT {
    XR_SPATIAL_PERSISTENCE_SCOPE_SYSTEM_MANAGED_EXT = 1,
  // Provided by XR_EXT_spatial_persistence_operations
    XR_SPATIAL_PERSISTENCE_SCOPE_LOCAL_ANCHORS_EXT = 1000781000,
    XR_SPATIAL_PERSISTENCE_SCOPE_MAX_ENUM_EXT = 0x7FFFFFFF
} XrSpatialPersistenceScopeEXT;
```

The XrSpatialPersistenceScopeEXT enumeration identifies the different types of persistence context scopes.

The enums have the following meanings:

Enum	Description
`XR_SPATIAL_PERSISTENCE_SCOPE_SYSTEM_MANAGED_EXT`	Provides the application with read-only access (i.e. application cannot modify the store associated with this scope) to spatial entities persisted and managed by the system. The application **can** use the UUID in the persistence component for this scope to correlate entities across spatial contexts and device reboots.

Enum	Description
XR_SPATIAL_PERSISTENCE_SCOPE_LOCAL_ANCHORS_EXT	Persistence operations and data access is limited to spatial anchors, on the same device, for the same user and same app (Added by the XR_EXT_spatial_persistence_operations extension)

The xrEnumerateSpatialPersistenceScopesEXT function is defined as:

```
// Provided by XR_EXT_spatial_persistence
XrResult xrEnumerateSpatialPersistenceScopesEXT(
    XrInstance                                  instance,
    XrSystemId                                  systemId,
    uint32_t                                    persistenceScopeCapacityInput,
    uint32_t*                                   persistenceScopeCountOutput,
    XrSpatialPersistenceScopeEXT*               persistenceScopes);
```

Parameter Descriptions

- instance is a handle to an XrInstance.
- systemId is the XrSystemId whose spatial persistence scopes will be enumerated.
- persistenceScopeCapacityInput is the capacity of the persistenceScopes array, or 0 to indicate a request to retrieve the required capacity.
- persistenceScopeCountOutput is the number of persistence scopes, or the required capacity in the case that persistenceScopeCapacityInput is insufficient.
- persistenceScopes is an array of XrSpatialPersistenceScopeEXT. It **can** be NULL if persistenceScopeCapacityInput is 0.
- See Buffer Size Parameters chapter for a detailed description of retrieving the required persistenceScopes size.

The application **can** enumerate the list of spatial persistence scopes supported by a given XrSystemId using xrEnumerateSpatialPersistenceScopesEXT.

Valid Usage (Implicit)

- The XR_EXT_spatial_persistence extension **must** be enabled prior to calling xrEnumerateSpatialPersistenceScopesEXT
- instance **must** be a valid XrInstance handle
- persistenceScopeCountOutput **must** be a pointer to a uint32_t value
- If persistenceScopeCapacityInput is not 0, persistenceScopes **must** be a pointer to an array of persistenceScopeCapacityInput XrSpatialPersistenceScopeEXT values

Return Codes

Success

- XR_SUCCESS

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SIZE_INSUFFICIENT
- XR_ERROR_SYSTEM_INVALID
- XR_ERROR_VALIDATION_FAILURE

The xrCreateSpatialPersistenceContextCompleteEXT function is defined as:

```
// Provided by XR_EXT_spatial_persistence
XrResult xrCreateSpatialPersistenceContextCompleteEXT(
    XrSession                                       session,
    XrFutureEXT                                     future,
    XrCreateSpatialPersistenceContextCompletionEXT* completion);
```

Parameter Descriptions

- `session` is the XrSession previously passed to xrCreateSpatialPersistenceContextAsyncEXT ::`session`.
- `future` is the `XrFutureEXT` received from xrCreateSpatialPersistenceContextAsyncEXT.
- `completion` is a pointer to an XrCreateSpatialPersistenceContextCompletionEXT.

xrCreateSpatialPersistenceContextCompleteEXT completes the asynchronous operation started by xrCreateSpatialPersistenceContextAsyncEXT. The runtime **must** return `XR_ERROR_FUTURE_PENDING_EXT` if `future` is not in ready state. The runtime **must** return `XR_ERROR_FUTURE_INVALID_EXT` if `future` has already been completed or cancelled.

Valid Usage (Implicit)

- The `XR_EXT_spatial_persistence` extension **must** be enabled prior to calling xrCreateSpatialPersistenceContextCompleteEXT
- `session` **must** be a valid XrSession handle
- `completion` **must** be a pointer to an XrCreateSpatialPersistenceContextCompletionEXT structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_FUTURE_INVALID_EXT
- XR_ERROR_FUTURE_PENDING_EXT
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The XrCreateSpatialPersistenceContextCompletionEXT structure is defined as:

```
// Provided by XR_EXT_spatial_persistence
typedef struct XrCreateSpatialPersistenceContextCompletionEXT {
    XrStructureType                         type;
    void*                                   next;
    XrResult                                futureResult;
    XrSpatialPersistenceContextResultEXT    createResult;
    XrSpatialPersistenceContextEXT          persistenceContext;
} XrCreateSpatialPersistenceContextCompletionEXT;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `futureResult` is the XrResult of the spatial persistence context creation operation.
- `createResult` is the XrSpatialPersistenceContextResultEXT of the spatial persistence context creation operation.
- `persistenceContext` is an XrSpatialPersistenceContextEXT created using the data and configuration in xrCreateSpatialPersistenceContextAsyncEXT::`createInfo`.

Future Return Codes

`futureResult` values:

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_OUT_OF_MEMORY`
- `XR_ERROR_LIMIT_REACHED`
- `XR_ERROR_PERMISSION_INSUFFICIENT`

If `futureResult` and `createResult` are both success codes, `persistenceContext` **must** be valid. If `persistenceContext` is valid, it **must** remain so within the lifecycle of xrCreateSpatialPersistenceContextAsyncEXT::`session` or until the application uses xrDestroySpatialPersistenceContextEXT with `persistenceContext`, whichever comes first.

The runtime **must** set `createResult` only if `futureResult` is a success code.

Valid Usage (Implicit)

- The XR_EXT_spatial_persistence extension **must** be enabled prior to using XrCreateSpatialPersistenceContextCompletionEXT
- type **must** be XR_TYPE_CREATE_SPATIAL_PERSISTENCE_CONTEXT_COMPLETION_EXT
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- futureResult **must** be a valid XrResult value
- createResult **must** be a valid XrSpatialPersistenceContextResultEXT value
- persistenceContext **must** be a valid XrSpatialPersistenceContextEXT handle

```
// Provided by XR_EXT_spatial_persistence
typedef enum XrSpatialPersistenceContextResultEXT {
    XR_SPATIAL_PERSISTENCE_CONTEXT_RESULT_SUCCESS_EXT = 0,
  // Provided by XR_EXT_spatial_persistence_operations
    XR_SPATIAL_PERSISTENCE_CONTEXT_RESULT_ENTITY_NOT_TRACKING_EXT = -1000781001,
  // Provided by XR_EXT_spatial_persistence_operations
    XR_SPATIAL_PERSISTENCE_CONTEXT_RESULT_PERSIST_UUID_NOT_FOUND_EXT = -1000781002,
    XR_SPATIAL_PERSISTENCE_CONTEXT_RESULT_MAX_ENUM_EXT = 0x7FFFFFFF
} XrSpatialPersistenceContextResultEXT;
```

The XrSpatialPersistenceContextResultEXT enumeration identifies the different types of result codes for a persistence operation. Failures during persistence operations are not always in control of the application and this enumeration is used for conveying such cases. Similar to XrResult, success codes in the XrSpatialPersistenceContextResultEXT enumeration are non-negative values, and failure codes are negative values.

The enums have the following meanings:

Enum	Description
XR_SPATIAL_PERSISTENCE_CONTEXT_RESULT_SUCCESS_EXT	The persistence context operation was a success.
XR_SPATIAL_PERSISTENCE_CONTEXT_RESULT_ENTITY_NOT_TRACKING_EXT	The persistence operation failed because the entity could not be tracked by the runtime. (Added by the XR_EXT_spatial_persistence_operations extension)
XR_SPATIAL_PERSISTENCE_CONTEXT_RESULT_PERSIST_UUID_NOT_FOUND_EXT	The provided persist UUID was not found in the storage. (Added by the XR_EXT_spatial_persistence_operations extension)

Destroy the spatial persistence context

The xrDestroySpatialPersistenceContextEXT function is defined as:

```
// Provided by XR_EXT_spatial_persistence
XrResult xrDestroySpatialPersistenceContextEXT(
    XrSpatialPersistenceContextEXT              persistenceContext);
```

Parameter Descriptions

- `persistenceContext` is an XrSpatialPersistenceContextEXT previously created by xrCreateSpatialPersistenceContextAsyncEXT.

The application **can** use xrDestroySpatialPersistenceContextEXT to release the `persistenceContext` handle when it is finished with spatial persistence tasks.

The runtime **must** not destroy the underlying resources for `persistenceContext` when xrDestroySpatialPersistenceContextEXT is called if there are any valid XrSpatialContextEXT handles that `persistenceContext` was linked to via XrSpatialContextPersistenceConfigEXT. This is because the persistence context's resources are still used by the spatial context for discovering persisted entities. Destroying the persistence context handle in such a situation only removes the application's access to these resources.

The resources for a destroyed XrSpatialPersistenceContextEXT **must** be freed when all the XrSpatialContextEXT handles the persistence context was linked to are destroyed.

Valid Usage (Implicit)

- The `XR_EXT_spatial_persistence` extension **must** be enabled prior to calling xrDestroySpatialPersistenceContextEXT
- `persistenceContext` **must** be a valid XrSpatialPersistenceContextEXT handle

Thread Safety

- Access to `persistenceContext`, and any child handles, **must** be externally synchronized

Return Codes

Success

- XR_SUCCESS

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_RUNTIME_FAILURE

12.48.3. Discover persisted entities

Persistence component

Persisted spatial entities have the persistence component on them which the runtime **must** include in the discovery and update snapshots if XR_SPATIAL_COMPONENT_TYPE_PERSISTENCE_EXT is enabled during the creation of XrSpatialContextEXT and included in XrSpatialDiscoverySnapshotCreateInfoEXT::componentTypes or XrSpatialUpdateSnapshotCreateInfoEXT::componentTypes.

Component Data

The XrSpatialPersistenceDataEXT structure is defined as:

```
// Provided by XR_EXT_spatial_persistence
typedef struct XrSpatialPersistenceDataEXT {
    XrUuid                          persistUuid;
    XrSpatialPersistenceStateEXT    persistState;
} XrSpatialPersistenceDataEXT;
```

Member Descriptions

- persistUuid is an XrUuid.
- persistState is an XrSpatialPersistenceStateEXT representing the persistence state of persistUuid

Valid Usage (Implicit)

- The XR_EXT_spatial_persistence extension **must** be enabled prior to using XrSpatialPersistenceDataEXT
- persistState **must** be a valid XrSpatialPersistenceStateEXT value

```
// Provided by XR_EXT_spatial_persistence
typedef enum XrSpatialPersistenceStateEXT {
    XR_SPATIAL_PERSISTENCE_STATE_LOADED_EXT = 1,
    XR_SPATIAL_PERSISTENCE_STATE_NOT_FOUND_EXT = 2,
    XR_SPATIAL_PERSISTENCE_STATE_MAX_ENUM_EXT = 0x7FFFFFFF
} XrSpatialPersistenceStateEXT;
```

The XrSpatialPersistenceStateEXT enumeration identifies the different states of the persisted uuid.

The enums have the following meanings:

Enum	Description
XR_SPATIAL_PERSISTENCE_STATE_LOADED_EXT	The persisted UUID has been successfully loaded from the storage.
XR_SPATIAL_PERSISTENCE_STATE_NOT_FOUND_EXT	The persisted UUID was not found in the storage and was either removed from it or never was in it.

Component list structure to query data

The XrSpatialComponentPersistenceListEXT structure is defined as:

```
// Provided by XR_EXT_spatial_persistence
typedef struct XrSpatialComponentPersistenceListEXT {
    XrStructureType                 type;
    void*                           next;
    uint32_t                        persistDataCount;
    XrSpatialPersistenceDataEXT*    persistData;
} XrSpatialComponentPersistenceListEXT;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `persistDataCount` is a `uint32_t` describing the count of elements in the `persistData` array.
- `persistData` is an array of XrSpatialPersistenceDataEXT.

The runtime **must** return `XR_ERROR_VALIDATION_FAILURE` from xrQuerySpatialComponentDataEXT if XrSpatialComponentPersistenceListEXT is in the next chain of XrSpatialComponentDataQueryResultEXT::`next` but `XR_SPATIAL_COMPONENT_TYPE_PERSISTENCE_EXT` is not included in XrSpatialComponentDataQueryConditionEXT::`componentTypes`.

The runtime **must** return `XR_ERROR_SIZE_INSUFFICIENT` from xrQuerySpatialComponentDataEXT if `persistDataCount` is less than XrSpatialComponentDataQueryResultEXT::`entityIdCountOutput`.

Unlike the other components, the runtime **must** set the data for `XR_SPATIAL_COMPONENT_TYPE_PERSISTENCE_EXT` regardless of the XrSpatialEntityTrackingStateEXT.

Valid Usage (Implicit)

- The `XR_EXT_spatial_persistence` extension **must** be enabled prior to using XrSpatialComponentPersistenceListEXT
- `type` **must** be `XR_TYPE_SPATIAL_COMPONENT_PERSISTENCE_LIST_EXT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `persistData` **must** be a pointer to an array of `persistDataCount` XrSpatialPersistenceDataEXT structures
- The `persistDataCount` parameter **must** be greater than `0`

Configuration

If `XR_SPATIAL_COMPONENT_TYPE_PERSISTENCE_EXT` is enumerated in XrSpatialCapabilityComponentTypesEXT::`componentTypes` for some capability, the application **can** enable it by including the enum in the XrSpatialCapabilityConfigurationBaseHeaderEXT::`enabledComponents` list. This component does not require any special configuration to be included in the next chain of XrSpatialCapabilityConfigurationBaseHeaderEXT. If the application is including `XR_SPATIAL_COMPONENT_TYPE_PERSISTENCE_EXT` in the enabled component list, it **must** also include XrSpatialContextPersistenceConfigEXT in the next chain of XrSpatialContextCreateInfoEXT otherwise the runtime **must** return `XR_ERROR_SPATIAL_CAPABILITY_CONFIGURATION_INVALID_EXT` from xrCreateSpatialContextAsyncEXT.

Configure spatial context with persistence contexts

The XrSpatialContextPersistenceConfigEXT structure is defined as:

```
// Provided by XR_EXT_spatial_persistence
typedef struct XrSpatialContextPersistenceConfigEXT {
    XrStructureType                           type;
    const void*                               next;
    uint32_t                                  persistenceContextCount;
    const XrSpatialPersistenceContextEXT*     persistenceContexts;
} XrSpatialContextPersistenceConfigEXT;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `persistenceContextCount` is a `uint32_t` describing the count of elements in the `persistenceContexts` array.
- `persistenceContexts` is an an array of XrSpatialPersistenceContextEXT.

An application **can** add XrSpatialContextPersistenceConfigEXT to the next chain of XrSpatialContextCreateInfoEXT. This will configure the created XrSpatialContextEXT with `persistenceContexts` and allow the application to discover the spatial entities persisted in the storage represented by the XrSpatialPersistenceContextEXT handles in `persistenceContexts`.

Valid Usage (Implicit)

- The `XR_EXT_spatial_persistence` extension **must** be enabled prior to using XrSpatialContextPersistenceConfigEXT
- `type` **must** be `XR_TYPE_SPATIAL_CONTEXT_PERSISTENCE_CONFIG_EXT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `persistenceContexts` **must** be a pointer to an array of `persistenceContextCount` valid XrSpatialPersistenceContextEXT handles
- The `persistenceContextCount` parameter **must** be greater than `0`

Create discovery snapshot

Discover entities with specific UUIDs

The XrSpatialDiscoveryPersistenceUuidFilterEXT structure is defined as:

```
// Provided by XR_EXT_spatial_persistence
typedef struct XrSpatialDiscoveryPersistenceUuidFilterEXT {
    XrStructureType    type;
    const void*        next;
    uint32_t           persistedUuidCount;
    const XrUuid*      persistedUuids;
} XrSpatialDiscoveryPersistenceUuidFilterEXT;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `persistedUuidCount` is a `uint32_t` describing the count of elements in the `persistedUuids` array
- `persistedUuids` is an array of XrUuid.

The application **can** use XrSpatialDiscoveryPersistenceUuidFilterEXT in the next chain of XrSpatialDiscoverySnapshotCreateInfoEXT to scope the discovery operation to just the entities whose persisted UUIDs are in the set of the UUIDs provided in `persistedUuids`.

If the application adds XrSpatialDiscoveryPersistenceUuidFilterEXT in the next chain of XrSpatialDiscoverySnapshotCreateInfoEXT but the xrCreateSpatialDiscoverySnapshotAsyncEXT ::`spatialContext` was not configured with any XrSpatialPersistenceContextEXT using XrSpatialContextPersistenceConfigEXT, the runtime **must** return `XR_ERROR_VALIDATION_FAILURE` from xrCreateSpatialDiscoverySnapshotAsyncEXT.

The runtime **must** treat the XrSpatialDiscoveryPersistenceUuidFilterEXT filter as an 'AND' condition with any other filters provided in XrSpatialDiscoverySnapshotCreateInfoEXT or its next chain. The runtime **must** treat the `persistedUuids` array itself as an 'OR' condition i.e. filter for entities that have any of the UUIDs provided in that array.

The runtime **must** include one entry in the created snapshot for each of the UUIDs in `persistedUuids` for which it was able to determine the XrSpatialPersistenceStateEXT state at this time.

- If the runtime has successfully found the UUID in its storage, then -
  - The runtime **must** set the XrSpatialPersistenceStateEXT in the XrSpatialPersistenceDataEXT of this entity to `XR_SPATIAL_PERSISTENCE_STATE_LOADED_EXT`.
  - The runtime **must** include a valid `XrSpatialEntityIdEXT` for this entity in the created snapshot.

    - The runtime **must** set the XrSpatialEntityTrackingStateEXT of that entity to XR_SPATIAL_ENTITY_TRACKING_STATE_TRACKING_EXT if it is actively tracking the entity and has valid data for its components. Otherwise, the runtime **must** set the XrSpatialEntityTrackingStateEXT to XR_SPATIAL_ENTITY_TRACKING_STATE_PAUSED_EXT.
- If the runtime has determined that the UUID is not present in its storage (regardless of whether that UUID was never in the storage or has was present once but has since been unpersisted), then -
    - The runtime **must** set the XrSpatialPersistenceStateEXT in the XrSpatialPersistenceDataEXT of this entity to XR_SPATIAL_PERSISTENCE_STATE_NOT_FOUND_EXT to indicate to the application that this UUID is no longer present in the storage.
    - The runtime **must** set the XrSpatialEntityIdEXT for this entity in the created snapshot to XR_NULL_SPATIAL_ENTITY_ID_EXT.
    - The runtime **must** set the XrSpatialEntityTrackingStateEXT of that entity to XR_SPATIAL_ENTITY_TRACKING_STATE_STOPPED_EXT to indicate to the application that this entity will never be tracked.
- If the runtime was not able to determine if the UUID is present in its storage or not, it **must** not include it in the snapshot.

The application **can** also use XrSpatialDiscoveryPersistenceUuidFilterEXT in the next chain of XrSpatialComponentDataQueryConditionEXT to query for entities of specific UUIDs in existing snapshots. When used with XrSpatialComponentDataQueryConditionEXT, if XrSpatialDiscoveryPersistenceUuidFilterEXT::persistedUuids contains any XrUuid that is not in the XrSpatialSnapshotEXT, the runtime **must** not include an entry for that XrUuid in the query result. Also, the order (sequence) of entities in the query result **may** not match the order of UUIDs provided in XrSpatialDiscoveryPersistenceUuidFilterEXT::persistedUuids. Application **should** include XR_SPATIAL_COMPONENT_TYPE_PERSISTENCE_EXT in XrSpatialComponentDataQueryConditionEXT::componentTypes and XrSpatialComponentPersistenceListEXT in the next chain of XrSpatialComponentDataQueryResultEXT and then check the XrSpatialPersistenceDataEXT::persistUuid of the each query result index to understand which UUID the current result index corresponds to.

Valid Usage (Implicit)

- The XR_EXT_spatial_persistence extension **must** be enabled prior to using XrSpatialDiscoveryPersistenceUuidFilterEXT
- type **must** be XR_TYPE_SPATIAL_DISCOVERY_PERSISTENCE_UUID_FILTER_EXT
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- persistedUuids **must** be a pointer to an array of persistedUuidCount XrUuid structures
- The persistedUuidCount parameter **must** be greater than 0

Discover all persisted entities

If the application uses xrCreateSpatialDiscoverySnapshotAsyncEXT without XrSpatialDiscoveryPersistenceUuidFilterEXT and with an XrSpatialContextEXT which has been configured with an XrSpatialPersistenceContextEXT, then the runtime **must** include those entities in the created snapshot that are persisted in the storage represented by XrSpatialPersistenceContextEXT and satisfy the filters provided in XrSpatialDiscoverySnapshotCreateInfoEXT. For those entities -

- The runtime **must** set the XrSpatialPersistenceStateEXT in the XrSpatialPersistenceDataEXT of this entity to `XR_SPATIAL_PERSISTENCE_STATE_LOADED_EXT`.
- The runtime **must** include a valid `XrSpatialEntityIdEXT` for this entity in the created snapshot.
- The runtime **must** set the XrSpatialEntityTrackingStateEXT of that entity to `XR_SPATIAL_ENTITY_TRACKING_STATE_TRACKING_EXT` if it is actively tracking the entity and has valid data for its components. Otherwise, the runtime **must** set the XrSpatialEntityTrackingStateEXT to `XR_SPATIAL_ENTITY_TRACKING_STATE_PAUSED_EXT`.

12.48.4. New Object Types

- XrSpatialPersistenceContextEXT

12.48.5. New Commands

- xrCreateSpatialPersistenceContextAsyncEXT
- xrCreateSpatialPersistenceContextCompleteEXT
- xrDestroySpatialPersistenceContextEXT
- xrEnumerateSpatialPersistenceScopesEXT

12.48.6. New Structures

- XrCreateSpatialPersistenceContextCompletionEXT
- XrSpatialPersistenceContextCreateInfoEXT
- XrSpatialPersistenceDataEXT
- Extending XrSpatialComponentDataQueryResultEXT:
  - XrSpatialComponentPersistenceListEXT
- Extending XrSpatialContextCreateInfoEXT:
  - XrSpatialContextPersistenceConfigEXT
- Extending XrSpatialDiscoverySnapshotCreateInfoEXT, XrSpatialComponentDataQueryConditionEXT:
  - XrSpatialDiscoveryPersistenceUuidFilterEXT

12.48.7. New Enums

- XrSpatialPersistenceContextResultEXT
- XrSpatialPersistenceScopeEXT
- XrSpatialPersistenceStateEXT

12.48.8. New Enum Constants

- XR_EXT_SPATIAL_PERSISTENCE_EXTENSION_NAME
- XR_EXT_spatial_persistence_SPEC_VERSION
- Extending XrObjectType:
  - XR_OBJECT_TYPE_SPATIAL_PERSISTENCE_CONTEXT_EXT
- Extending XrResult:
  - XR_ERROR_SPATIAL_PERSISTENCE_SCOPE_UNSUPPORTED_EXT
- Extending XrSpatialComponentTypeEXT:
  - XR_SPATIAL_COMPONENT_TYPE_PERSISTENCE_EXT
- Extending XrStructureType:
  - XR_TYPE_CREATE_SPATIAL_PERSISTENCE_CONTEXT_COMPLETION_EXT
  - XR_TYPE_SPATIAL_COMPONENT_PERSISTENCE_LIST_EXT
  - XR_TYPE_SPATIAL_CONTEXT_PERSISTENCE_CONFIG_EXT
  - XR_TYPE_SPATIAL_DISCOVERY_PERSISTENCE_UUID_FILTER_EXT
  - XR_TYPE_SPATIAL_PERSISTENCE_CONTEXT_CREATE_INFO_EXT

12.48.9. Example Code

Create Persistence Context

```
// Check if the required persistence scope supported
uint32_t scopeCount = 0;
CHK_XR(xrEnumerateSpatialPersistenceScopesEXT(instance, systemId, 0, &scopeCount,
nullptr));
std::vector<XrSpatialPersistenceScopeEXT> persistenceScopes(scopeCount);
CHK_XR(xrEnumerateSpatialPersistenceScopesEXT(instance, systemId, scopeCount, &
scopeCount, persistenceScopes.data()));

if (std::find(persistenceScopes.begin(), persistenceScopes.end(),
XR_SPATIAL_PERSISTENCE_SCOPE_SYSTEM_MANAGED_EXT) == persistenceScopes.end()) {
  return;
}

XrSpatialPersistenceContextEXT persistenceContext {XR_NULL_HANDLE};

XrSpatialPersistenceContextCreateInfoEXT persistenceContextCreateInfo{
  .type = XR_TYPE_SPATIAL_PERSISTENCE_CONTEXT_CREATE_INFO_EXT,
  .scope = XR_SPATIAL_PERSISTENCE_SCOPE_SYSTEM_MANAGED_EXT,
};
XrFutureEXT createContextFuture {XR_NULL_FUTURE_EXT};
CHK_XR(xrCreateSpatialPersistenceContextAsyncEXT(session, &persistenceContextCreateInfo,
&createContextFuture));

waitUntilReady(createContextFuture);

XrCreateSpatialPersistenceContextCompletionEXT completion{
  .type = XR_TYPE_CREATE_SPATIAL_PERSISTENCE_CONTEXT_COMPLETION_EXT,
};
CHK_XR(xrCreateSpatialPersistenceContextCompleteEXT(session, createContextFuture,
&completion));
if (completion.futureResult != XR_SUCCESS || completion.createResult !=
XR_SPATIAL_PERSISTENCE_CONTEXT_RESULT_SUCCESS_EXT) {
  return;
}

persistenceContext = completion.persistenceContext;

// ...
// Connect persistence context to a spatial context and discover persisted entities.
// ...

CHK_XR(xrDestroySpatialPersistenceContextEXT(persistenceContext));
```

Connect Persistence Context to a Spatial Context

```
// Note: Anchor capability is just used as an example here. Persistence can be
```

```
// supported by other capabilities too. xrEnumerateSpatialCapabilityComponentTypesEXT()
can
// be used to check if a certain capability supports persistence.
if (!isSpatialCapabilitySupported(instance, systemId, XR_SPATIAL_CAPABILITY_ANCHOR_EXT))
{
  return;
}

const bool supportsPersistenceComponent = isSpatialComponentSupported(instance, systemId,
XR_SPATIAL_CAPABILITY_ANCHOR_EXT, XR_SPATIAL_COMPONENT_TYPE_PERSISTENCE_EXT);

// Create a spatial spatial context
XrSpatialContextEXT spatialContext{};
{

  std::vector<XrSpatialComponentTypeEXT> enabledComponents = {
    XR_SPATIAL_COMPONENT_TYPE_ANCHOR_EXT,
  };

  if (supportsPersistenceComponent) {
    enabledComponents.push_back(XR_SPATIAL_COMPONENT_TYPE_PERSISTENCE_EXT);
  }

  XrSpatialCapabilityConfigurationAnchorEXT anchorConfig {
    .type = XR_TYPE_SPATIAL_CAPABILITY_CONFIGURATION_ANCHOR_EXT,
    .capability = XR_SPATIAL_CAPABILITY_ANCHOR_EXT,
    .enabledComponentCount = static_cast<uint32_t>(enabledComponents.size()),
    .enabledComponents = enabledComponents.data(),
  };

  std::array<XrSpatialCapabilityConfigurationBaseHeaderEXT*, 1> capabilityConfigs = {
    reinterpret_cast<XrSpatialCapabilityConfigurationBaseHeaderEXT*>(&anchorConfig),
  };

  XrSpatialContextCreateInfoEXT spatialContextCreateInfo{
    .type = XR_TYPE_SPATIAL_CONTEXT_CREATE_INFO_EXT,
    .capabilityConfigCount = capabilityConfigs.size(),
    .capabilityConfigs = capabilityConfigs.data(),
  };

  XrSpatialContextPersistenceConfigEXT persistenceConfig{
    .type = XR_TYPE_SPATIAL_CONTEXT_PERSISTENCE_CONFIG_EXT,
    .persistenceContextCount = 1,
    .persistenceContexts = &persistenceContext,
  };

  if (supportsPersistenceComponent) {
    spatialContextCreateInfo.next = &persistenceConfig;
```

```
  }

  XrFutureEXT createContextFuture {XR_NULL_FUTURE_EXT};
  CHK_XR(xrCreateSpatialContextAsyncEXT(session, &spatialContextCreateInfo,
&createContextFuture));

  waitUntilReady(createContextFuture);

  XrCreateSpatialContextCompletionEXT completion{
    .type = XR_TYPE_CREATE_SPATIAL_CONTEXT_COMPLETION_EXT,
  };
  CHK_XR(xrCreateSpatialContextCompleteEXT(session, createContextFuture, &completion));
  if (completion.futureResult != XR_SUCCESS) {
    return;
  }

  spatialContext = completion.spatialContext;
}

// ...
// Discover persisted anchors.
// ...

CHK_XR(xrDestroySpatialContextEXT(spatialContext));
```

Discover all persisted entities

```
XrFutureEXT future = XR_NULL_FUTURE_EXT;

// We want to look for entities that have the following components.
std::vector<XrSpatialComponentTypeEXT> snapshotComponents = {
  XR_SPATIAL_COMPONENT_TYPE_PERSISTENCE_EXT,
};

auto discoverSpatialEntities = [&](XrSpatialContextEXT spatialContext, XrTime time) {
  XrSpatialDiscoverySnapshotCreateInfoEXT snapshotCreateInfo {
    .type = XR_TYPE_SPATIAL_DISCOVERY_SNAPSHOT_CREATE_INFO_EXT,
    .componentTypeCount = static_cast<uint32_t>(snapshotComponents.size()),
    .componentTypes = snapshotComponents.data(),
  };
  CHK_XR(xrCreateSpatialDiscoverySnapshotAsyncEXT(spatialContext, &snapshotCreateInfo,
&future));

  waitUntilReady(future);

  XrCreateSpatialDiscoverySnapshotCompletionInfoEXT completionInfo {
```

```
    .type = XR_TYPE_CREATE_SPATIAL_DISCOVERY_SNAPSHOT_COMPLETION_INFO_EXT,
    .baseSpace = localSpace,
    .time = time,
    .future = future,
  };

  XrCreateSpatialDiscoverySnapshotCompletionEXT completion {
    .type = XR_TYPE_CREATE_SPATIAL_DISCOVERY_SNAPSHOT_COMPLETION_EXT,
  };
  CHK_XR(xrCreateSpatialDiscoverySnapshotCompleteEXT(spatialContext, &completionInfo,
&completion));
  if (completion.futureResult == XR_SUCCESS) {

    // Query for the semantic label component data
    XrSpatialComponentDataQueryConditionEXT queryCond {
      .type = XR_TYPE_SPATIAL_COMPONENT_DATA_QUERY_CONDITION_EXT,
      .componentTypeCount = static_cast<uint32_t>(snapshotComponents.size()),
      .componentTypes = snapshotComponents.data(),
    };

    XrSpatialComponentDataQueryResultEXT queryResult {
      .type = XR_TYPE_SPATIAL_COMPONENT_DATA_QUERY_RESULT_EXT,
    };
    CHK_XR(xrQuerySpatialComponentDataEXT(completion.snapshot, &queryCond, &
queryResult));

    std::vector<XrSpatialEntityIdEXT> entityIds(queryResult.entityIdCountOutput);
    std::vector<XrSpatialEntityTrackingStateEXT> entityStates(queryResult
.entityIdCountOutput);
    queryResult.entityIdCapacityInput = entityIds.size();
    queryResult.entityIds = entityIds.data();
    queryResult.entityStateCapacityInput = entityStates.size();
    queryResult.entityStates = entityStates.data();

    std::vector<XrSpatialPersistenceDataEXT> persistenceData(queryResult
.entityIdCountOutput);
    XrSpatialComponentPersistenceListEXT persistenceDataList {
      .type = XR_TYPE_SPATIAL_COMPONENT_PERSISTENCE_LIST_EXT,
      .persistDataCount = static_cast<uint32_t>(persistenceData.size()),
      .persistData = persistenceData.data(),
    };
    queryResult.next = &persistenceDataList;

    CHK_XR(xrQuerySpatialComponentDataEXT(completion.snapshot, &queryCond, &
queryResult));

    for (int32_t i = 0; i < queryResult.entityIdCountOutput; ++i) {
      // persistenceData[i].persistUuid is the UUID of the persisted entity whose entity
```

```
ID is entityIds[i].
      // The persistenceData array is essentially the uuids persisted in the scope that
the current
      // XrSpatialPersistenceContextEXT is configured with.
    }

    CHK_XR(xrDestroySpatialSnapshotEXT(completion.snapshot));
  }
};

while (1) {
  // ...
  // For every frame in frame loop
  // ...

  XrFrameState frameState;  // previously returned from xrWaitFrame
  const XrTime time = frameState.predictedDisplayTime;

  // Poll for the XR_TYPE_EVENT_DATA_SPATIAL_DISCOVERY_RECOMMENDED_EXT event
  XrEventDataBuffer event {
    .type = XR_TYPE_EVENT_DATA_BUFFER,
  };
  XrResult result = xrPollEvent(instance, &event);
  if (result == XR_SUCCESS) {
      switch (event.type) {
          case XR_TYPE_EVENT_DATA_SPATIAL_DISCOVERY_RECOMMENDED_EXT: {
              const XrEventDataSpatialDiscoveryRecommendedEXT& eventdata =
                  *reinterpret_cast<XrEventDataSpatialDiscoveryRecommendedEXT*>(&event);
              // Discover spatial entities for the context that we received the
"discovery
              // recommended" event for.
              discoverSpatialEntities(eventdata.spatialContext, time);
              break;
          }
      }
  }

  // ...
  // Finish frame loop
  // ...
}
```

Discover entities with specific UUIDs

```
XrFutureEXT future {XR_NULL_FUTURE_EXT};
```

```
// Load up the uuids that the app has stored on its own i.e. the uuids it is interested
in.
std::vector<XrUuid> uuidsStoredByApp = loadPersistedUuids();

// We want to look for entities that have the following components.
std::vector<XrSpatialComponentTypeEXT> snapshotComponents = {
  XR_SPATIAL_COMPONENT_TYPE_PERSISTENCE_EXT,
};

auto discoverSpatialEntities = [&](XrSpatialContextEXT spatialContext, XrTime time) {
  XrSpatialDiscoveryPersistenceUuidFilterEXT persistenceFilter {
    .type = XR_TYPE_SPATIAL_DISCOVERY_PERSISTENCE_UUID_FILTER_EXT,
    .persistedUuidCount = static_cast<uint32_t>(uuidsStoredByApp.size()),
    .persistedUuids = uuidsStoredByApp.data(),
  };

  XrSpatialDiscoverySnapshotCreateInfoEXT snapshotCreateInfo {
    .type = XR_TYPE_SPATIAL_DISCOVERY_SNAPSHOT_CREATE_INFO_EXT,
    .next = &persistenceFilter,
    .componentTypeCount = static_cast<uint32_t>(snapshotComponents.size()),
    .componentTypes = snapshotComponents.data(),
  };

  CHK_XR(xrCreateSpatialDiscoverySnapshotAsyncEXT(spatialContext, &snapshotCreateInfo,
&future));

  waitUntilReady(future);

  XrCreateSpatialDiscoverySnapshotCompletionInfoEXT completionInfo {
    .type = XR_TYPE_CREATE_SPATIAL_DISCOVERY_SNAPSHOT_COMPLETION_INFO_EXT,
    .baseSpace = localSpace,
    .time = time,
    .future = future,
  };

  XrCreateSpatialDiscoverySnapshotCompletionEXT completion {
    .type = XR_TYPE_CREATE_SPATIAL_DISCOVERY_SNAPSHOT_COMPLETION_EXT,
  };
  CHK_XR(xrCreateSpatialDiscoverySnapshotCompleteEXT(spatialContext, &completionInfo,
&completion));
  if (completion.futureResult == XR_SUCCESS) {

    // Query for the semantic label component data
    XrSpatialComponentDataQueryConditionEXT queryCond {
      .type = XR_TYPE_SPATIAL_COMPONENT_DATA_QUERY_CONDITION_EXT,
      .componentTypeCount = static_cast<uint32_t>(snapshotComponents.size()),
      .componentTypes = snapshotComponents.data(),
    };
```

```
    XrSpatialComponentDataQueryResultEXT queryResult {
      .type = XR_TYPE_SPATIAL_COMPONENT_DATA_QUERY_RESULT_EXT
    };
    CHK_XR(xrQuerySpatialComponentDataEXT(completion.snapshot, &queryCond, &
queryResult));

    std::vector<XrSpatialEntityIdEXT> entityIds(queryResult.entityIdCountOutput);
    std::vector<XrSpatialEntityTrackingStateEXT> entityStates(queryResult
.entityIdCountOutput);
    queryResult.entityIdCapacityInput = entityIds.size();
    queryResult.entityIds = entityIds.data();
    queryResult.entityStateCapacityInput = entityStates.size();
    queryResult.entityStates = entityStates.data();

    std::vector<XrSpatialPersistenceDataEXT> persistenceData(queryResult
.entityIdCountOutput);
    XrSpatialComponentPersistenceListEXT persistenceDataList{
      .type = XR_TYPE_SPATIAL_COMPONENT_PERSISTENCE_LIST_EXT,
      .persistDataCount = static_cast<uint32_t>(persistenceData.size()),
      .persistData = persistenceData.data(),
    };
    queryResult.next = &persistenceDataList;

    CHK_XR(xrQuerySpatialComponentDataEXT(completion.snapshot, &queryCond, &
queryResult));

    for (int32_t i = 0; i < queryResult.entityIdCountOutput; ++i) {
      if (persistenceData[i].persistState == XR_SPATIAL_PERSISTENCE_STATE_LOADED_EXT) {
        // persistenceData[i].persistUuid, requested by the app, is present in the
persistence scope
        // and its corresponding entity ID and state are entityIds[i] & entityStates[i]
respectively.
      } else if (persistenceData[i].persistState ==
XR_SPATIAL_PERSISTENCE_STATE_NOT_FOUND_EXT) {
        // persistenceData[i].persistUuid, requested by the app, is NOT present in the
persistence scope
        // and its corresponding entity ID (entityIds[i]) would be
XR_NULL_SPATIAL_ENTITY_ID_EXT
        // and tracking state (entityStates[i]) would be
XR_SPATIAL_ENTITY_TRACKING_STATE_STOPPED_EXT.
      }
    }

    CHK_XR(xrDestroySpatialSnapshotEXT(completion.snapshot));
  }
};
```

```
while (1) {
  // ...
  // For every frame in frame loop
  // ...

  XrFrameState frameState;  // previously returned from xrWaitFrame
  const XrTime time = frameState.predictedDisplayTime;

  // Poll for the XR_TYPE_EVENT_DATA_SPATIAL_DISCOVERY_RECOMMENDED_EXT event
  XrEventDataBuffer event {
    .type = XR_TYPE_EVENT_DATA_BUFFER,
  };
  XrResult result = xrPollEvent(instance, &event);
  if (result == XR_SUCCESS) {
      switch (event.type) {
          case XR_TYPE_EVENT_DATA_SPATIAL_DISCOVERY_RECOMMENDED_EXT: {
              const XrEventDataSpatialDiscoveryRecommendedEXT& eventdata =
                  *reinterpret_cast<XrEventDataSpatialDiscoveryRecommendedEXT*>(&event);
              // Discover spatial entities for the context that we received the
"discovery
              // recommended" event for.
              discoverSpatialEntities(eventdata.spatialContext, time);
              break;
          }
      }
  }

  // ...
  // Finish frame loop
  // ...
}
```

12.48.10. Issues

12.48.11. Version History

- Revision 1, 2024-08-29 (Nihav Jain, Google)
  - Initial extension description

12.49. XR_EXT_spatial_persistence_operations

Name String

`XR_EXT_spatial_persistence_operations`

Extension Type

Instance extension

Registered Extension Number

782

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

XR_EXT_spatial_persistence
and
XR_EXT_future

Contributors

Nihav Jain, Google
Jared Finder, Google
Natalie Fleury, Meta
Yuichi Taguchi, Meta
Ron Bessems, Meta
Yin Li, Microsoft
Jimmy Alamparambil, ByteDance
Zhipeng Liu, ByteDance
Jun Yan, ByteDance

12.49.1. Overview

While XR_EXT_spatial_persistence allows applications to discover persisted entities, this extension allows applications to persist and unpersist spatial entities.

12.49.2. Persist spatial entities

The xrPersistSpatialEntityAsyncEXT function is defined as:

```
// Provided by XR_EXT_spatial_persistence_operations
XrResult xrPersistSpatialEntityAsyncEXT(
    XrSpatialPersistenceContextEXT              persistenceContext,
    const XrSpatialEntityPersistInfoEXT*        persistInfo,
    XrFutureEXT*                                future);
```

Parameter Descriptions

- `persistenceContext` is an XrSpatialPersistenceContextEXT previously created using xrCreateSpatialPersistenceContextAsyncEXT.
- `persistInfo` is the XrSpatialEntityPersistInfoEXT used to specify the persist operation parameters.
- `future` is a pointer to an `XrFutureEXT`.

An application **can** persist a spatial entity using the xrPersistSpatialEntityAsyncEXT function.

The runtime **must** return `XR_ERROR_SPATIAL_ENTITY_ID_INVALID_EXT` if XrSpatialEntityPersistInfoEXT::`spatialEntityId` does not belong to XrSpatialEntityPersistInfoEXT::`spatialContext`.

The runtime **must** return `XR_ERROR_PERMISSION_INSUFFICIENT` if the XrSpatialPersistenceScopeEXT that `persistenceContext` was configured with is a read-only scope and does not allow applications to modify the storage represented by it. An example of this would be if `persistenceContext` was created with `XR_SPATIAL_PERSISTENCE_SCOPE_SYSTEM_MANAGED_EXT` and the application uses xrPersistSpatialEntityAsyncEXT with that `persistenceContext`.

The runtime **must** return `XR_ERROR_SPATIAL_PERSISTENCE_SCOPE_INCOMPATIBLE_EXT` if the XrSpatialPersistenceScopeEXT that `persistenceContext` was configured does allow the application to persist entities of its choice in the storage but XrSpatialEntityPersistInfoEXT::`spatialEntityId` is not covered in the configured scope. An example of this would be if the persistence context scope is set to `XR_SPATIAL_PERSISTENCE_SCOPE_LOCAL_ANCHORS_EXT`, and XrSpatialEntityPersistInfoEXT::`spatialEntityId` does not represent an anchor.

The runtime **must** not return an error if XrSpatialEntityPersistInfoEXT::`spatialContext` was not configured with `persistenceContext` using XrSpatialContextPersistenceConfigEXT. Using xrPersistSpatialEntityAsyncEXT does not require that `persistenceContext` be connected with the spatial context.

This function starts an asynchronous operation and creates a corresponding `XrFutureEXT`, usable with xrPollFutureEXT and related functions. The return value of this function only indicates whether the parameters were acceptable to schedule the asynchronous operation. The corresponding completion function is xrPersistSpatialEntityCompleteEXT, usable when a future from this function is in the READY state, with outputs populated by that function in the completion structure XrPersistSpatialEntityCompletionEXT.

If the XrSpatialEntityTrackingStateEXT of XrSpatialEntityPersistInfoEXT::`spatialEntityId` is not `XR_SPATIAL_ENTITY_TRACKING_STATE_TRACKING_EXT` when xrPersistSpatialEntityAsyncEXT is called, the runtime **must** not return an error from this function or set XrPersistSpatialEntityCompletionEXT::`futureResult` to an error code to indicate this. The runtime **may** either set `future` to the READY state immediately and set XrPersistSpatialEntityCompletionEXT::`persistResult` to `XR_SPATIAL_PERSISTENCE_CONTEXT_RESULT_ENTITY_NOT_TRACKING_EXT` to indicate the lack of tracking state,

or wait for the entity to get into tracking state as part of the async operation and set XrPersistSpatialEntityCompletionEXT::persistResult to XR_SPATIAL_PERSISTENCE_CONTEXT_RESULT_ENTITY_NOT_TRACKING_EXT if the entity does not get into tracking state until a runtime determined timeout. A common usage pattern of applications is to create a spatial anchor using xrCreateSpatialAnchorEXT and then immediately request to persist the newly created spatial anchor using xrPersistSpatialEntityAsyncEXT. XR_EXT_spatial_anchor states that the tracking state of an anchor **may** not be XR_SPATIAL_ENTITY_TRACKING_STATE_TRACKING_EXT immediately upon its creation. For such cases, the runtime **should** wait for the anchor to get into tracking state as part of the persist async operation instead of immediately setting future to the READY state and fail the operation with XR_SPATIAL_PERSISTENCE_CONTEXT_RESULT_ENTITY_NOT_TRACKING_EXT only if the anchor does not get into tracking state within a runtime determined timeout.

If the spatial entity represented by XrSpatialEntityPersistInfoEXT::spatialEntityId has already been persisted in the scope associated with persistenceContext, the runtime **must** not treat that as an error but instead complete the async operation successfully and provide the appropriate persist UUID to the application.

Valid Usage (Implicit)

- The XR_EXT_spatial_persistence_operations extension **must** be enabled prior to calling xrPersistSpatialEntityAsyncEXT
- persistenceContext **must** be a valid XrSpatialPersistenceContextEXT handle
- persistInfo **must** be a pointer to a valid XrSpatialEntityPersistInfoEXT structure
- future **must** be a pointer to an XrFutureEXT value

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_PERMISSION_INSUFFICIENT
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_SPATIAL_ENTITY_ID_INVALID_EXT
- XR_ERROR_SPATIAL_PERSISTENCE_SCOPE_INCOMPATIBLE_EXT
- XR_ERROR_VALIDATION_FAILURE

The XrSpatialEntityPersistInfoEXT structure is defined as:

```
// Provided by XR_EXT_spatial_persistence_operations
typedef struct XrSpatialEntityPersistInfoEXT {
    XrStructureType         type;
    const void*             next;
    XrSpatialContextEXT     spatialContext;
    XrSpatialEntityIdEXT    spatialEntityId;
} XrSpatialEntityPersistInfoEXT;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- spatialContext is an XrSpatialContextEXT to which spatialEntityId belongs.
- spatialEntityId is an XrSpatialEntityIdEXT of the spatial entity that is to be persisted.

The XrSpatialEntityPersistInfoEXT structure describes the information to persist a spatial entity represented by `spatialEntityId` in an XrSpatialPersistenceContextEXT.

Valid Usage (Implicit)

- The `XR_EXT_spatial_persistence_operations` extension **must** be enabled prior to using XrSpatialEntityPersistInfoEXT
- `type` **must** be `XR_TYPE_SPATIAL_ENTITY_PERSIST_INFO_EXT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `spatialContext` **must** be a valid XrSpatialContextEXT handle

The xrPersistSpatialEntityCompleteEXT function is defined as:

```
// Provided by XR_EXT_spatial_persistence_operations
XrResult xrPersistSpatialEntityCompleteEXT(
    XrSpatialPersistenceContextEXT              persistenceContext,
    XrFutureEXT                                 future,
    XrPersistSpatialEntityCompletionEXT*        completion);
```

Parameter Descriptions

- `persistenceContext` is the XrSpatialPersistenceContextEXT previously passed to xrPersistSpatialEntityAsyncEXT::`persistenceContext`.
- `future` is the `XrFutureEXT` received from xrPersistSpatialEntityAsyncEXT.
- `completion` is a pointer to an XrPersistSpatialEntityCompletionEXT.

xrPersistSpatialEntityCompleteEXT completes the asynchronous operation started by xrPersistSpatialEntityAsyncEXT. The runtime **must** return `XR_ERROR_FUTURE_PENDING_EXT` if `future` is not in READY state. The runtime **must** return `XR_ERROR_FUTURE_INVALID_EXT` if `future` has already been completed or cancelled.

This is the completion function corresponding to the operation started by xrPersistSpatialEntityAsyncEXT. Do not call until the `future` is READY.

If XrPersistSpatialEntityCompletionEXT::`persistUuid` is a UUID that has already been provided to the application either via a previous successful completion of xrPersistSpatialEntityAsyncEXT or by discovering existing persisted entities, then the XrSpatialEntityPersistInfoEXT::`spatialEntityId` **must** represent the same entity as the one the UUID was originally provided for.

Valid Usage (Implicit)

- The XR_EXT_spatial_persistence_operations extension **must** be enabled prior to calling xrPersistSpatialEntityCompleteEXT
- `persistenceContext` **must** be a valid XrSpatialPersistenceContextEXT handle
- `completion` **must** be a pointer to an XrPersistSpatialEntityCompletionEXT structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_FUTURE_INVALID_EXT
- XR_ERROR_FUTURE_PENDING_EXT
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The XrPersistSpatialEntityCompletionEXT structure is defined as:

```
// Provided by XR_EXT_spatial_persistence_operations
typedef struct XrPersistSpatialEntityCompletionEXT {
    XrStructureType                         type;
    void*                                   next;
    XrResult                                futureResult;
    XrSpatialPersistenceContextResultEXT    persistResult;
    XrUuid                                  persistUuid;
} XrPersistSpatialEntityCompletionEXT;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `futureResult` is the XrResult of the spatial entity persist operation.
- `persistResult` is the XrSpatialPersistenceContextResultEXT of the spatial entity persist operation.
- `persistUuid` is an XrUuid that the application **can** use to identify the persisted spatial entity across sessions.

Future Return Codes

`futureResult` values:

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_OUT_OF_MEMORY`
- `XR_ERROR_LIMIT_REACHED`

If `futureResult` and `persistResult` are both success codes, `persistUuid` **must** be valid and the application **can** use it to identify the persisted spatial entity across sessions.

The runtime **must** set `persistResult` to `XR_SPATIAL_PERSISTENCE_CONTEXT_RESULT_ENTITY_NOT_TRACKING_EXT` if it lost tracking of the entity represented by XrSpatialEntityPersistInfoEXT::`spatialEntityId` before it could be successfully persisted.

The runtime **must** set `persistResult` only if `futureResult` is a success code.

Valid Usage (Implicit)

- The XR_EXT_spatial_persistence_operations extension **must** be enabled prior to using XrPersistSpatialEntityCompletionEXT
- type **must** be XR_TYPE_PERSIST_SPATIAL_ENTITY_COMPLETION_EXT
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- futureResult **must** be a valid XrResult value
- persistResult **must** be a valid XrSpatialPersistenceContextResultEXT value

12.49.3. Unpersist spatial entities

The xrUnpersistSpatialEntityAsyncEXT function is defined as:

```
// Provided by XR_EXT_spatial_persistence_operations
XrResult xrUnpersistSpatialEntityAsyncEXT(
    XrSpatialPersistenceContextEXT              persistenceContext,
    const XrSpatialEntityUnpersistInfoEXT*      unpersistInfo,
    XrFutureEXT*                                future);
```

Parameter Descriptions

- persistenceContext is an XrSpatialPersistenceContextEXT previously created using xrCreateSpatialPersistenceContextAsyncEXT.
- unpersistInfo is the XrSpatialEntityUnpersistInfoEXT used to specify the unpersist operation parameters.
- future is a pointer to an XrFutureEXT.

An application **can** unpersist a spatial entity using the xrUnpersistSpatialEntityAsyncEXT function.

The runtime **must** return XR_ERROR_PERMISSION_INSUFFICIENT if the XrSpatialPersistenceScopeEXT that persistenceContext was configured with is a read-only scope and does not allow applications to modify the storage represented by it. An example of this would be if persistenceContext was created with XR_SPATIAL_PERSISTENCE_SCOPE_SYSTEM_MANAGED_EXT and application uses xrUnpersistSpatialEntityAsyncEXT with that persistenceContext.

This function starts an asynchronous operation and creates a corresponding XrFutureEXT, usable with xrPollFutureEXT and related functions. The return value of this function only indicates whether the parameters were acceptable to schedule the asynchronous operation. The corresponding completion function is xrUnpersistSpatialEntityCompleteEXT, usable when a future from this function is in the READY state, with outputs populated by that function in the completion structure

XrUnpersistSpatialEntityCompletionEXT.

Valid Usage (Implicit)

- The XR_EXT_spatial_persistence_operations extension **must** be enabled prior to calling xrUnpersistSpatialEntityAsyncEXT
- `persistenceContext` **must** be a valid XrSpatialPersistenceContextEXT handle
- `unpersistInfo` **must** be a pointer to a valid XrSpatialEntityUnpersistInfoEXT structure
- `future` **must** be a pointer to an `XrFutureEXT` value

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_PERMISSION_INSUFFICIENT
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The XrSpatialEntityUnpersistInfoEXT structure is defined as:

```
// Provided by XR_EXT_spatial_persistence_operations
typedef struct XrSpatialEntityUnpersistInfoEXT {
    XrStructureType    type;
    const void*        next;
    XrUuid             persistUuid;
} XrSpatialEntityUnpersistInfoEXT;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `persistUuid` is an XrUuid representing the spatial entity that is to be unpersisted.

The XrSpatialEntityUnpersistInfoEXT structure describes the information to unpersist a spatial entity previously persisted using xrPersistSpatialEntityAsyncEXT.

Valid Usage (Implicit)

- The `XR_EXT_spatial_persistence_operations` extension **must** be enabled prior to using XrSpatialEntityUnpersistInfoEXT
- `type` **must** be `XR_TYPE_SPATIAL_ENTITY_UNPERSIST_INFO_EXT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

The xrUnpersistSpatialEntityCompleteEXT function is defined as:

```
// Provided by XR_EXT_spatial_persistence_operations
XrResult xrUnpersistSpatialEntityCompleteEXT(
    XrSpatialPersistenceContextEXT              persistenceContext,
    XrFutureEXT                                 future,
    XrUnpersistSpatialEntityCompletionEXT*      completion);
```

Parameter Descriptions

- `persistenceContext` is the XrSpatialPersistenceContextEXT previously passed to xrUnpersistSpatialEntityAsyncEXT::`persistenceContext`.
- `future` is the `XrFutureEXT` received from xrUnpersistSpatialEntityAsyncEXT.
- `completion` is a pointer to an XrUnpersistSpatialEntityCompletionEXT.

xrUnpersistSpatialEntityCompleteEXT completes the asynchronous operation started by xrUnpersistSpatialEntityAsyncEXT. The runtime **must** return `XR_ERROR_FUTURE_PENDING_EXT` if `future` is not in READY state. The runtime **must** return `XR_ERROR_FUTURE_INVALID_EXT` if `future` has already been completed or cancelled.

This is the completion function corresponding to xrUnpersistSpatialEntityAsyncEXT. It completes the asynchronous operation and returns the results. Do not call until the `future` is READY.

Valid Usage (Implicit)

- The XR_EXT_spatial_persistence_operations extension **must** be enabled prior to calling xrUnpersistSpatialEntityCompleteEXT
- `persistenceContext` **must** be a valid XrSpatialPersistenceContextEXT handle
- `completion` **must** be a pointer to an XrUnpersistSpatialEntityCompletionEXT structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_FUTURE_INVALID_EXT
- XR_ERROR_FUTURE_PENDING_EXT
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The XrUnpersistSpatialEntityCompletionEXT structure is defined as:

```
// Provided by XR_EXT_spatial_persistence_operations
typedef struct XrUnpersistSpatialEntityCompletionEXT {
    XrStructureType                         type;
    void*                                   next;
    XrResult                                futureResult;
    XrSpatialPersistenceContextResultEXT    unpersistResult;
} XrUnpersistSpatialEntityCompletionEXT;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `futureResult` is the XrResult of the spatial entity unpersist operation.
- `unpersistResult` is the XrSpatialPersistenceContextResultEXT of the spatial entity unpersist operation.

Future Return Codes

`futureResult` values:

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_OUT_OF_MEMORY`
- `XR_ERROR_LIMIT_REACHED`

The runtime **must** set `unpersistResult` only if `futureResult` is a success code.

If XrSpatialEntityUnpersistInfoEXT::`persistUuid` is not found in the storage represented by xrUnpersistSpatialEntityCompleteEXT::`persistenceContext`, then the runtime **must** set `unpersistResult` to `XR_SPATIAL_PERSISTENCE_CONTEXT_RESULT_PERSIST_UUID_NOT_FOUND_EXT`.

Valid Usage (Implicit)

- The `XR_EXT_spatial_persistence_operations` extension **must** be enabled prior to using XrUnpersistSpatialEntityCompletionEXT
- `type` **must** be `XR_TYPE_UNPERSIST_SPATIAL_ENTITY_COMPLETION_EXT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `futureResult` **must** be a valid XrResult value
- `unpersistResult` **must** be a valid XrSpatialPersistenceContextResultEXT value

12.49.4. Anchor Persistence Local Scope

If the runtime supports persistence for spatial anchors, and stores them on the same device, for the same user and application that originally created it, it **must** indicate this by enumerating `XR_SPATIAL_PERSISTENCE_SCOPE_LOCAL_ANCHORS_EXT` in xrEnumerateSpatialPersistenceScopesEXT.

If a runtime enumerates `XR_SPATIAL_PERSISTENCE_SCOPE_LOCAL_ANCHORS_EXT` in xrEnumerateSpatialPersistenceScopesEXT, the runtime **must** also enumerate `XR_SPATIAL_CAPABILITY_ANCHOR_EXT` in xrEnumerateSpatialCapabilitiesEXT and `XR_SPATIAL_COMPONENT_TYPE_PERSISTENCE_EXT` in xrEnumerateSpatialCapabilityComponentTypesEXT for `XR_SPATIAL_CAPABILITY_ANCHOR_EXT`.

12.49.5. New Commands

- xrPersistSpatialEntityAsyncEXT
- xrPersistSpatialEntityCompleteEXT
- xrUnpersistSpatialEntityAsyncEXT
- xrUnpersistSpatialEntityCompleteEXT

12.49.6. New Structures

- XrPersistSpatialEntityCompletionEXT
- XrSpatialEntityPersistInfoEXT
- XrSpatialEntityUnpersistInfoEXT
- XrUnpersistSpatialEntityCompletionEXT

12.49.7. New Enum Constants

- `XR_EXT_SPATIAL_PERSISTENCE_OPERATIONS_EXTENSION_NAME`
- `XR_EXT_spatial_persistence_operations_SPEC_VERSION`
- Extending XrResult:
  - `XR_ERROR_SPATIAL_PERSISTENCE_SCOPE_INCOMPATIBLE_EXT`
- Extending XrSpatialPersistenceContextResultEXT:
  - `XR_SPATIAL_PERSISTENCE_CONTEXT_RESULT_ENTITY_NOT_TRACKING_EXT`
  - `XR_SPATIAL_PERSISTENCE_CONTEXT_RESULT_PERSIST_UUID_NOT_FOUND_EXT`
- Extending XrSpatialPersistenceScopeEXT:
  - `XR_SPATIAL_PERSISTENCE_SCOPE_LOCAL_ANCHORS_EXT`
- Extending XrStructureType:
  - `XR_TYPE_PERSIST_SPATIAL_ENTITY_COMPLETION_EXT`

- XR_TYPE_SPATIAL_ENTITY_PERSIST_INFO_EXT
- XR_TYPE_SPATIAL_ENTITY_UNPERSIST_INFO_EXT
- XR_TYPE_UNPERSIST_SPATIAL_ENTITY_COMPLETION_EXT

12.49.8. Issues

12.49.9. Version History

- Revision 1, 2024-08-29 (Nihav Jain, Google)
  - Initial extension description

12.50. XR_EXT_spatial_plane_tracking

Name String

XR_EXT_spatial_plane_tracking

Extension Type

Instance extension

Registered Extension Number

742

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

XR_EXT_spatial_entity

Contributors

Nihav Jain, Google
Natalie Fleury, Meta
Yuichi Taguchi, Meta
Ron Bessems, Meta
Yin Li, Microsoft
Jimmy Alamparambil, ByteDance
Zhipeng Liu, ByteDance
Jun Yan, ByteDance

12.50.1. Overview

This extension builds on XR_EXT_spatial_entity and defines the plane tracking spatial capability for the spatial entity framework.

12.50.2. Runtime Support

If the runtime supports plane tracking, it **must** indicate this by enumerating XR_SPATIAL_CAPABILITY_PLANE_TRACKING_EXT in xrEnumerateSpatialCapabilitiesEXT.

12.50.3. Configuration

The XrSpatialCapabilityConfigurationPlaneTrackingEXT structure is defined as:

```
// Provided by XR_EXT_spatial_plane_tracking
typedef struct XrSpatialCapabilityConfigurationPlaneTrackingEXT {
    XrStructureType                     type;
    const void*                         next;
    XrSpatialCapabilityEXT              capability;
    uint32_t                            enabledComponentCount;
    const XrSpatialComponentTypeEXT*    enabledComponents;
} XrSpatialCapabilityConfigurationPlaneTrackingEXT;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain.
- capability is an XrSpatialCapabilityEXT and **must** be XR_SPATIAL_CAPABILITY_PLANE_TRACKING_EXT.
- enabledComponentCount is a uint32_t describing the count of elements in the enabledComponents array.
- enabledComponents is a pointer to an array of XrSpatialComponentTypeEXT.

Applications **can** enable the XR_SPATIAL_CAPABILITY_PLANE_TRACKING_EXT spatial capability by including a pointer to an XrSpatialCapabilityConfigurationPlaneTrackingEXT structure in XrSpatialContextCreateInfoEXT::capabilityConfigs.

The runtime **must** return XR_ERROR_VALIDATION_FAILURE if capability is not XR_SPATIAL_CAPABILITY_PLANE_TRACKING_EXT.

Valid Usage (Implicit)

- The `XR_EXT_spatial_plane_tracking` extension **must** be enabled prior to using XrSpatialCapabilityConfigurationPlaneTrackingEXT
- `type` **must** be `XR_TYPE_SPATIAL_CAPABILITY_CONFIGURATION_PLANE_TRACKING_EXT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `capability` **must** be a valid XrSpatialCapabilityEXT value
- `enabledComponents` **must** be a pointer to an array of `enabledComponentCount` valid XrSpatialComponentTypeEXT values
- The `enabledComponentCount` parameter **must** be greater than `0`

12.50.4. Guaranteed Components

A runtime that supports `XR_SPATIAL_CAPABILITY_PLANE_TRACKING_EXT` **must** provide the following spatial components as guaranteed components of all entities discovered by this capability and **must** enumerate them in xrEnumerateSpatialCapabilityComponentTypesEXT:

- `XR_SPATIAL_COMPONENT_TYPE_BOUNDED_2D_EXT`
- `XR_SPATIAL_COMPONENT_TYPE_PLANE_ALIGNMENT_EXT`

Bounded 2D Component

The bounded 2D component provides the center and extents of the plane represented by the entity it is on. See Bounded 2D for more details.

Plane Alignment Component

Component data

```
// Provided by XR_EXT_spatial_plane_tracking
typedef enum XrSpatialPlaneAlignmentEXT {
    XR_SPATIAL_PLANE_ALIGNMENT_HORIZONTAL_UPWARD_EXT = 0,
    XR_SPATIAL_PLANE_ALIGNMENT_HORIZONTAL_DOWNWARD_EXT = 1,
    XR_SPATIAL_PLANE_ALIGNMENT_VERTICAL_EXT = 2,
    XR_SPATIAL_PLANE_ALIGNMENT_ARBITRARY_EXT = 3,
    XR_SPATIAL_PLANE_ALIGNMENT_MAX_ENUM_EXT = 0x7FFFFFFF
} XrSpatialPlaneAlignmentEXT;
```

The XrSpatialPlaneAlignmentEXT enumeration describes the alignment of the plane associated with the spatial entity with an `XR_SPATIAL_COMPONENT_TYPE_PLANE_ALIGNMENT_EXT` component.

The enumeration values have the following meanings:

Enum	Description
XR_SPATIAL_PLANE_ALIGNMENT_HORIZONTAL_UPWARD_EXT	The entity is horizontal and faces upward (e.g. floor).
XR_SPATIAL_PLANE_ALIGNMENT_HORIZONTAL_DOWNWARD_EXT	The entity is horizontal and faces downward (e.g. ceiling).
XR_SPATIAL_PLANE_ALIGNMENT_VERTICAL_EXT	The entity is vertical (e.g. wall).
XR_SPATIAL_PLANE_ALIGNMENT_ARBITRARY_EXT	The entity has an arbitrary, non-vertical and non-horizontal orientation.

Component list structure to query data

The XrSpatialComponentPlaneAlignmentListEXT structure is defined as:

```
// Provided by XR_EXT_spatial_plane_tracking
typedef struct XrSpatialComponentPlaneAlignmentListEXT {
    XrStructureType                 type;
    void*                           next;
    uint32_t                        planeAlignmentCount;
    XrSpatialPlaneAlignmentEXT*     planeAlignments;
} XrSpatialComponentPlaneAlignmentListEXT;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain.
- planeAlignmentCount is a uint32_t describing the count of elements in the planeAlignments array.
- planeAlignments is an array of XrSpatialPlaneAlignmentEXT.

To query the plane alignment component of the spatial entities in an XrSpatialSnapshotEXT, include XR_SPATIAL_COMPONENT_TYPE_PLANE_ALIGNMENT_EXT in XrSpatialComponentDataQueryConditionEXT::componentTypes and add XrSpatialComponentPlaneAlignmentListEXT to the XrSpatialComponentDataQueryResultEXT::next chain.

The runtime **must** return XR_ERROR_VALIDATION_FAILURE from xrQuerySpatialComponentDataEXT if XrSpatialComponentPlaneAlignmentListEXT is in the XrSpatialComponentDataQueryResultEXT::next chain but XR_SPATIAL_COMPONENT_TYPE_PLANE_ALIGNMENT_EXT is not included in XrSpatialComponentDataQueryConditionEXT::componentTypes.

The runtime **must** return XR_ERROR_SIZE_INSUFFICIENT from xrQuerySpatialComponentDataEXT if planeAlignmentCount is less than XrSpatialComponentDataQueryResultEXT::entityIdCountOutput.

Valid Usage (Implicit)

- The XR_EXT_spatial_plane_tracking extension **must** be enabled prior to using XrSpatialComponentPlaneAlignmentListEXT
- type **must** be XR_TYPE_SPATIAL_COMPONENT_PLANE_ALIGNMENT_LIST_EXT
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- planeAlignments **must** be a pointer to an array of planeAlignmentCount XrSpatialPlaneAlignmentEXT values
- The planeAlignmentCount parameter **must** be greater than 0

Configuration

If XR_SPATIAL_COMPONENT_TYPE_PLANE_ALIGNMENT_EXT is enumerated in XrSpatialCapabilityComponentTypesEXT::componentTypes for some capability, an application **can** enable it by including the enumerant in the XrSpatialCapabilityConfigurationBaseHeaderEXT::enabledComponents list of the XrSpatialCapabilityConfigurationBaseHeaderEXT derived structure of the capability that supports this component.

This component does not require any special configuration to be included in the XrSpatialCapabilityConfigurationBaseHeaderEXT::next chain.

12.50.5. Optional Components

A runtime that supports XR_SPATIAL_CAPABILITY_PLANE_TRACKING_EXT **may** support other spatial components in addition to the ones listed in the Guaranteed Components section. An application uses xrEnumerateSpatialCapabilityComponentTypesEXT to get the full list of components that a runtime supports, then configures the ones it is interested in when creating the spatial context.

Mesh 2D Component

Component data

XR_SPATIAL_COMPONENT_TYPE_MESH_2D_EXT uses the XrSpatialMeshDataEXT structure for its data.

Component list structure to query data

The XrSpatialComponentMesh2DListEXT structure is defined as:

```
// Provided by XR_EXT_spatial_plane_tracking
typedef struct XrSpatialComponentMesh2DListEXT {
    XrStructureType          type;
    void*                    next;
    uint32_t                 meshCount;
    XrSpatialMeshDataEXT*    meshes;
} XrSpatialComponentMesh2DListEXT;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `meshCount` is a `uint32_t` describing the count of elements in the `meshes` array.
- `meshes` is an array of XrSpatialMeshDataEXT.

To query the mesh 2D component of the spatial entities in an XrSpatialSnapshotEXT, include `XR_SPATIAL_COMPONENT_TYPE_MESH_2D_EXT` in XrSpatialComponentDataQueryConditionEXT::`componentTypes` and add XrSpatialComponentMesh2DListEXT to the XrSpatialComponentDataQueryResultEXT::`next` chain.

The runtime **must** return `XR_ERROR_VALIDATION_FAILURE` from xrQuerySpatialComponentDataEXT if XrSpatialComponentMesh2DListEXT is in the XrSpatialComponentDataQueryResultEXT::`next` chain but `XR_SPATIAL_COMPONENT_TYPE_MESH_2D_EXT` is not included in XrSpatialComponentDataQueryConditionEXT::`componentTypes`.

The runtime **must** return `XR_ERROR_SIZE_INSUFFICIENT` from xrQuerySpatialComponentDataEXT if `meshCount` is less than XrSpatialComponentDataQueryResultEXT::`entityIdCountOutput`.

For the XrSpatialMeshDataEXT populated by the runtime in the `meshes` array, the XrSpatialBufferEXT::`bufferType` for XrSpatialMeshDataEXT::`vertexBuffer` **must** be `XR_SPATIAL_BUFFER_TYPE_VECTOR2F_EXT` and XrSpatialBufferEXT::`bufferType` for XrSpatialMeshDataEXT::`indexBuffer` **must** be `XR_SPATIAL_BUFFER_TYPE_UINT16_EXT`.

Valid Usage (Implicit)

- The XR_EXT_spatial_plane_tracking extension **must** be enabled prior to using XrSpatialComponentMesh2DListEXT
- type **must** be XR_TYPE_SPATIAL_COMPONENT_MESH_2D_LIST_EXT
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- meshes **must** be a pointer to an array of meshCount XrSpatialMeshDataEXT structures
- The meshCount parameter **must** be greater than 0

Configuration

If XR_SPATIAL_COMPONENT_TYPE_MESH_2D_EXT is enumerated in XrSpatialCapabilityComponentTypesEXT::componentTypes for some capability, an application **can** enable it by including the enumerant in the XrSpatialCapabilityConfigurationBaseHeaderEXT::enabledComponents list of the XrSpatialCapabilityConfigurationBaseHeaderEXT derived structure of the capability that supports this component.

This component does not require any special configuration to be included in the XrSpatialCapabilityConfigurationBaseHeaderEXT::next chain.

Polygon 2D Component

Component Data

The XrSpatialPolygon2DDataEXT structure is defined as:

```
// Provided by XR_EXT_spatial_plane_tracking
typedef struct XrSpatialPolygon2DDataEXT {
    XrPosef               origin;
    XrSpatialBufferEXT    vertexBuffer;
} XrSpatialPolygon2DDataEXT;
```

Member Descriptions

- origin is an XrPosef defining the origin of the polygon. All vertices of the polygon are relative to this origin in the X-Y plane.
- vertexBuffer is an XrSpatialBufferEXT that provides the ID for a buffer of type XR_SPATIAL_BUFFER_TYPE_VECTOR2F_EXT and represents the vertex buffer of the entity this component is on. The vertices **must** be returned in counter-clockwise order. The polygon represented by these vertices **must** not be self-intersecting and **may** be concave.

XrSpatialBufferEXT::bufferType for vertexBuffer **must** be XR_SPATIAL_BUFFER_TYPE_VECTOR2F_EXT.

Valid Usage (Implicit)

- The XR_EXT_spatial_plane_tracking extension **must** be enabled prior to using XrSpatialPolygon2DDataEXT
- vertexBuffer **must** be a valid XrSpatialBufferEXT structure

Component list structure to query data

The XrSpatialComponentPolygon2DListEXT structure is defined as:

```
// Provided by XR_EXT_spatial_plane_tracking
typedef struct XrSpatialComponentPolygon2DListEXT {
    XrStructureType               type;
    void*                         next;
    uint32_t                      polygonCount;
    XrSpatialPolygon2DDataEXT*    polygons;
} XrSpatialComponentPolygon2DListEXT;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain.
- polygonCount is a uint32_t describing the count of elements in the polygons array.
- polygons is an array of XrSpatialPolygon2DDataEXT.

To query the polygon 2D component of the spatial entities in an XrSpatialSnapshotEXT, include XR_SPATIAL_COMPONENT_TYPE_POLYGON_2D_EXT in XrSpatialComponentDataQueryConditionEXT::componentTypes and add XrSpatialComponentPolygon2DListEXT to the XrSpatialComponentDataQueryResultEXT::next chain.

The runtime **must** return XR_ERROR_VALIDATION_FAILURE from xrQuerySpatialComponentDataEXT if XrSpatialComponentPolygon2DListEXT is in the XrSpatialComponentDataQueryResultEXT::next chain but XR_SPATIAL_COMPONENT_TYPE_POLYGON_2D_EXT is not included in XrSpatialComponentDataQueryConditionEXT::componentTypes.

The runtime **must** return XR_ERROR_SIZE_INSUFFICIENT from xrQuerySpatialComponentDataEXT if polygonCount is less than XrSpatialComponentDataQueryResultEXT::entityIdCountOutput.

Valid Usage (Implicit)

- The XR_EXT_spatial_plane_tracking extension **must** be enabled prior to using XrSpatialComponentPolygon2DListEXT
- type **must** be XR_TYPE_SPATIAL_COMPONENT_POLYGON_2D_LIST_EXT
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- polygons **must** be a pointer to an array of polygonCount XrSpatialPolygon2DDataEXT structures
- The polygonCount parameter **must** be greater than 0

Configuration

If XR_SPATIAL_COMPONENT_TYPE_POLYGON_2D_EXT is enumerated in XrSpatialCapabilityComponentTypesEXT::componentTypes for some capability, an application **can** enable it by including the enumerant in the XrSpatialCapabilityConfigurationBaseHeaderEXT ::enabledComponents list of the XrSpatialCapabilityConfigurationBaseHeaderEXT derived structure of the capability that supports this component.

This component does not require any special configuration to be included in the XrSpatialCapabilityConfigurationBaseHeaderEXT::next chain.

Plane Semantic Label

Component Data

```
// Provided by XR_EXT_spatial_plane_tracking
typedef enum XrSpatialPlaneSemanticLabelEXT {
    XR_SPATIAL_PLANE_SEMANTIC_LABEL_UNCATEGORIZED_EXT = 1,
    XR_SPATIAL_PLANE_SEMANTIC_LABEL_FLOOR_EXT = 2,
    XR_SPATIAL_PLANE_SEMANTIC_LABEL_WALL_EXT = 3,
    XR_SPATIAL_PLANE_SEMANTIC_LABEL_CEILING_EXT = 4,
    XR_SPATIAL_PLANE_SEMANTIC_LABEL_TABLE_EXT = 5,
    XR_SPATIAL_PLANE_SEMANTIC_LABEL_MAX_ENUM_EXT = 0x7FFFFFFF
} XrSpatialPlaneSemanticLabelEXT;
```

The XrSpatialPlaneSemanticLabelEXT enumeration describes a set of semantic labels for planes.

Enum	Description
XR_SPATIAL_PLANE_SEMANTIC_LABEL_UNCATEGORIZED_EXT	The runtime was unable to classify this entity.
XR_SPATIAL_PLANE_SEMANTIC_LABEL_FLOOR_EXT	The entity is a floor.

Enum	Description
XR_SPATIAL_PLANE_SEMANTIC_LABEL_WALL_EXT	The entity is a wall.
XR_SPATIAL_PLANE_SEMANTIC_LABEL_CEILING_EXT	The entity is a ceiling.
XR_SPATIAL_PLANE_SEMANTIC_LABEL_TABLE_EXT	The entity is a table.

Component List Structure to Query Data

The XrSpatialComponentPlaneSemanticLabelListEXT structure is defined as:

```
// Provided by XR_EXT_spatial_plane_tracking
typedef struct XrSpatialComponentPlaneSemanticLabelListEXT {
    XrStructureType                     type;
    void*                               next;
    uint32_t                            semanticLabelCount;
    XrSpatialPlaneSemanticLabelEXT*     semanticLabels;
} XrSpatialComponentPlaneSemanticLabelListEXT;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain.
- semanticLabelCount is a uint32_t describing the count of elements in the semanticLabels array.
- semanticLabels is an array of XrSpatialPlaneSemanticLabelEXT.

To query the plane semantic label component of the spatial entities in an XrSpatialSnapshotEXT, include XR_SPATIAL_COMPONENT_TYPE_PLANE_SEMANTIC_LABEL_EXT in XrSpatialComponentDataQueryConditionEXT::componentTypes and add XrSpatialComponentPlaneSemanticLabelListEXT to the XrSpatialComponentDataQueryResultEXT::next chain.

The runtime **must** return XR_ERROR_VALIDATION_FAILURE from xrQuerySpatialComponentDataEXT if XrSpatialComponentPlaneSemanticLabelListEXT is in the next chain of XrSpatialComponentDataQueryResultEXT::next but XR_SPATIAL_COMPONENT_TYPE_PLANE_SEMANTIC_LABEL_EXT is not included in XrSpatialComponentDataQueryConditionEXT::componentTypes.

The runtime **must** return XR_ERROR_SIZE_INSUFFICIENT from xrQuerySpatialComponentDataEXT if semanticLabelCount is less than XrSpatialComponentDataQueryResultEXT::entityIdCountOutput.

Valid Usage (Implicit)

- The XR_EXT_spatial_plane_tracking extension **must** be enabled prior to using XrSpatialComponentPlaneSemanticLabelListEXT
- type **must** be XR_TYPE_SPATIAL_COMPONENT_PLANE_SEMANTIC_LABEL_LIST_EXT
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- semanticLabels **must** be a pointer to an array of semanticLabelCount XrSpatialPlaneSemanticLabelEXT values
- The semanticLabelCount parameter **must** be greater than 0

Configuration

If XR_SPATIAL_COMPONENT_TYPE_PLANE_SEMANTIC_LABEL_EXT is enumerated in XrSpatialCapabilityComponentTypesEXT::componentTypes for some capability, an application **can** enable it by including the enumerant in the XrSpatialCapabilityConfigurationBaseHeaderEXT::enabledComponents list of the XrSpatialCapabilityConfigurationBaseHeaderEXT derived structure of the capability that supports this component.

This component does not require any special configuration to be included in the XrSpatialCapabilityConfigurationBaseHeaderEXT::next chain.

12.50.6. Example Code

Configure Plane Tracking Capability

The following example code demonstrates how to configure plane tracking capability when creating a spatial context.

```
// Check if plane tracking capability is supported
uint32_t capabilityCount = 0;
CHK_XR(xrEnumerateSpatialCapabilitiesEXT(instance, systemId, 0, &capabilityCount,
nullptr));
std::vector<XrSpatialCapabilityEXT> capabilities(capabilityCount);
CHK_XR(xrEnumerateSpatialCapabilitiesEXT(instance, systemId, capabilityCount,
&capabilityCount, capabilities.data()));

if (std::find(capabilities.begin(), capabilities.end(),
XR_SPATIAL_CAPABILITY_PLANE_TRACKING_EXT) == capabilities.end()) {
  return;
}

// Enumerate supported components for plane tracking capability
XrSpatialCapabilityComponentTypesEXT planeComponents{
```

```
  .type = XR_TYPE_SPATIAL_CAPABILITY_COMPONENT_TYPES_EXT,
};
CHK_XR(xrEnumerateSpatialCapabilityComponentTypesEXT(instance, systemId,
XR_SPATIAL_CAPABILITY_PLANE_TRACKING_EXT, &planeComponents));
std::vector<XrSpatialComponentTypeEXT> planeCapabilityComponents(planeComponents
.componentTypeCountOutput);
planeComponents.componentTypeCapacityInput = planeCapabilityComponents.size();
planeComponents.componentTypes = planeCapabilityComponents.data();
CHK_XR(xrEnumerateSpatialCapabilityComponentTypesEXT(instance, systemId,
XR_SPATIAL_CAPABILITY_PLANE_TRACKING_EXT, &planeComponents));

// Check if polygon 2D and plane semantic labels optional components are supported
const auto supportsComponent = [&planeCapabilityComponents](XrSpatialComponentTypeEXT
component) {
  return std::find(planeCapabilityComponents.begin(), planeCapabilityComponents.end(),
component) != planeCapabilityComponents.end();
};

const bool supportsPolygon2DComponent = supportsComponent
(XR_SPATIAL_COMPONENT_TYPE_POLYGON_2D_EXT);
const bool supportsSemanticLabelComponent = supportsComponent
(XR_SPATIAL_COMPONENT_TYPE_PLANE_SEMANTIC_LABEL_EXT);

// Create a spatial context
XrSpatialContextEXT spatialContext{};

// Enable the 2 guaranteed components of the plane tracking capability
std::vector<XrSpatialComponentTypeEXT> enabledComponents = {
  XR_SPATIAL_COMPONENT_TYPE_BOUNDED_2D_EXT,
  XR_SPATIAL_COMPONENT_TYPE_PLANE_ALIGNMENT_EXT,
};
// Optionally enable polygon2D if it is supported
if (supportsPolygon2DComponent) {
  enabledComponents.push_back(XR_SPATIAL_COMPONENT_TYPE_POLYGON_2D_EXT);
}
// Optionally enable semantic labels if it is supported
if (supportsSemanticLabelComponent) {
  enabledComponents.push_back(XR_SPATIAL_COMPONENT_TYPE_PLANE_SEMANTIC_LABEL_EXT);
}

XrSpatialCapabilityConfigurationPlaneTrackingEXT planeConfig{
  .type = XR_TYPE_SPATIAL_CAPABILITY_CONFIGURATION_PLANE_TRACKING_EXT,
  .capability = XR_SPATIAL_CAPABILITY_PLANE_TRACKING_EXT,
  .enabledComponentCount = static_cast<uint32_t>(enabledComponents.size()),
  .enabledComponents = enabledComponents.data(),
};

std::array<XrSpatialCapabilityConfigurationBaseHeaderEXT*, 1> capabilityConfigs = {
```

```
  reinterpret_cast<XrSpatialCapabilityConfigurationBaseHeaderEXT*>(&planeConfig),
};

XrSpatialContextCreateInfoEXT spatialContextCreateInfo{
  .type = XR_TYPE_SPATIAL_CONTEXT_CREATE_INFO_EXT,
  .capabilityConfigCount = capabilityConfigs.size(),
  .capabilityConfigs = capabilityConfigs.data(),
};
XrFutureEXT createContextFuture {XR_NULL_FUTURE_EXT};
CHK_XR(xrCreateSpatialContextAsyncEXT(session, &spatialContextCreateInfo,
&createContextFuture));

waitUntilReady(createContextFuture);

XrCreateSpatialContextCompletionEXT completion{
  .type = XR_TYPE_CREATE_SPATIAL_CONTEXT_COMPLETION_EXT,
};
CHK_XR(xrCreateSpatialContextCompleteEXT(session, createContextFuture, &completion));
if (completion.futureResult != XR_SUCCESS) {
  return;
}

spatialContext = completion.spatialContext;

// ...
// Discover entities with the spatial context
// ...

CHK_XR(xrDestroySpatialContextEXT(spatialContext));
```

Discover Spatial Entities & Query Component Data

The following example code demonstrates how to discover spatial entities for a context configured with XR_SPATIAL_CAPABILITY_PLANE_TRACKING_EXT and query its component data.

```
XrFutureEXT future = XR_NULL_FUTURE_EXT;

// We want to look for entities that have the following components.
std::vector<XrSpatialComponentTypeEXT> snapshotComponents = {
  XR_SPATIAL_COMPONENT_TYPE_BOUNDED_2D_EXT,
  XR_SPATIAL_COMPONENT_TYPE_PLANE_ALIGNMENT_EXT,
};
if (supportsPolygon2DComponent) {
  snapshotComponents.push_back(XR_SPATIAL_COMPONENT_TYPE_POLYGON_2D_EXT);
}
if (supportsSemanticLabelComponent) {
```

```
  snapshotComponents.push_back(XR_SPATIAL_COMPONENT_TYPE_PLANE_SEMANTIC_LABEL_EXT);
}

auto discoverSpatialEntities = [&](XrSpatialContextEXT spatialContext, XrTime time) {
  XrSpatialDiscoverySnapshotCreateInfoEXT snapshotCreateInfo {
    .type = XR_TYPE_SPATIAL_DISCOVERY_SNAPSHOT_CREATE_INFO_EXT,
    .componentTypeCount = static_cast<uint32_t>(snapshotComponents.size()),
    .componentTypes = snapshotComponents.data(),
  };
  CHK_XR(xrCreateSpatialDiscoverySnapshotAsyncEXT(spatialContext, &snapshotCreateInfo,
&future));

  waitUntilReady(future);

  XrCreateSpatialDiscoverySnapshotCompletionInfoEXT completionInfo {
    .type = XR_TYPE_CREATE_SPATIAL_DISCOVERY_SNAPSHOT_COMPLETION_INFO_EXT,
    .baseSpace = localSpace,
    .time = time,
    .future = future,
  };

  XrCreateSpatialDiscoverySnapshotCompletionEXT completion {
    .type = XR_TYPE_CREATE_SPATIAL_DISCOVERY_SNAPSHOT_COMPLETION_EXT,
  };
  CHK_XR(xrCreateSpatialDiscoverySnapshotCompleteEXT(spatialContext, &completionInfo,
&completion));
  if (completion.futureResult == XR_SUCCESS) {

    // Query for the semantic label component data
    XrSpatialComponentDataQueryConditionEXT queryCond {
      .type = XR_TYPE_SPATIAL_COMPONENT_DATA_QUERY_CONDITION_EXT,
      .componentTypeCount = static_cast<uint32_t>(snapshotComponents.size()),
      .componentTypes = snapshotComponents.data(),
    };

    XrSpatialComponentDataQueryResultEXT queryResult {
      .type = XR_TYPE_SPATIAL_COMPONENT_DATA_QUERY_RESULT_EXT,
    };
    CHK_XR(xrQuerySpatialComponentDataEXT(completion.snapshot, &queryCond, &
queryResult));

    std::vector<XrSpatialEntityIdEXT> entityIds(queryResult.entityIdCountOutput);
    std::vector<XrSpatialEntityTrackingStateEXT> entityStates(queryResult
.entityIdCountOutput);
    queryResult.entityIdCapacityInput = entityIds.size();
    queryResult.entityIds = entityIds.data();
    queryResult.entityStateCapacityInput = entityStates.size();
    queryResult.entityStates = entityStates.data();
```

```
    std::vector<XrSpatialBounded2DDataEXT> bounded2D(queryResult.entityIdCountOutput);
    XrSpatialComponentBounded2DListEXT bounded2DList {
      .type = XR_TYPE_SPATIAL_COMPONENT_BOUNDED_2D_LIST_EXT,
      .boundCount = static_cast<uint32_t>(bounded2D.size()),
      .bounds = bounded2D.data(),
    };
    queryResult.next = &bounded2DList;

    std::vector<XrSpatialPolygon2DDataEXT> polygons;
    XrSpatialComponentPolygon2DListEXT polygonList {
      .type = XR_TYPE_SPATIAL_COMPONENT_POLYGON_2D_LIST_EXT,
    };
    if (supportsPolygon2DComponent) {
      polygons.resize(queryResult.entityIdCountOutput);
      polygonList.polygonCount = polygons.size();
      polygonList.polygons = polygons.data();
      polygonList.next = queryResult.next;
      queryResult.next = &polygonList;
    }

    std::vector<XrSpatialPlaneSemanticLabelEXT> semanticLabels;
    XrSpatialComponentPlaneSemanticLabelListEXT semanticLabelsList {
      .type = XR_TYPE_SPATIAL_COMPONENT_PLANE_SEMANTIC_LABEL_LIST_EXT,
    };
    if (supportsSemanticLabelComponent) {
      semanticLabels.resize(queryResult.entityIdCountOutput);
      semanticLabelsList.semanticLabelCount = semanticLabels.size();
      semanticLabelsList.semanticLabels = semanticLabels.data();
      semanticLabelsList.next = queryResult.next;
      queryResult.next = &semanticLabelsList;
    }

    CHK_XR(xrQuerySpatialComponentDataEXT(completion.snapshot, &queryCond, &
queryResult));

    for (int32_t i = 0; i < queryResult.entityIdCountOutput; ++i) {
      if (entityStates[i] != XR_SPATIAL_ENTITY_TRACKING_STATE_TRACKING_EXT) {
        continue;
      }

      // 2D bounds for entity entityIds[i] is bounded2D[i].extents centered on
bounded2D[i].center.

      if (supportsPolygon2DComponent) {
        // 2D polygon for entity entityIds[i] is the buffer represented by
polygons[i].bufferId.
        // Application uses flink:xrGetSpatialBufferVector2fEXT to get the buffer data.
```

```
      }

      if (supportsSemanticLabelComponent) {
        // semantic label for entity entityIds[i] is semanticLabels[i].
      }
    }

    CHK_XR(xrDestroySpatialSnapshotEXT(completion.snapshot));
  }
};

while (1) {
  // ...
  // For every frame in frame loop
  // ...

  XrFrameState frameState;  // previously returned from xrWaitFrame
  const XrTime time = frameState.predictedDisplayTime;

  // Poll for the XR_TYPE_EVENT_DATA_SPATIAL_DISCOVERY_RECOMMENDED_EXT event
  XrEventDataBuffer event {
    .type = XR_TYPE_EVENT_DATA_BUFFER,
  };
  XrResult result = xrPollEvent(instance, &event);
  if (result == XR_SUCCESS) {
      switch (event.type) {
          case XR_TYPE_EVENT_DATA_SPATIAL_DISCOVERY_RECOMMENDED_EXT: {
              const XrEventDataSpatialDiscoveryRecommendedEXT& eventdata =
                  *reinterpret_cast<XrEventDataSpatialDiscoveryRecommendedEXT*>(&event);
              // Discover spatial entities for the context that we received the
"discovery
              // recommended" event for.
              discoverSpatialEntities(eventdata.spatialContext, time);
              break;
          }
      }
  }

  // ...
  // Finish frame loop
  // ...
}
```

12.50.7. New Structures

- XrSpatialCapabilityConfigurationPlaneTrackingEXT

- XrSpatialPolygon2DDataEXT
- Extending XrSpatialComponentDataQueryResultEXT:
  - XrSpatialComponentMesh2DListEXT
  - XrSpatialComponentPlaneAlignmentListEXT
  - XrSpatialComponentPlaneSemanticLabelListEXT
  - XrSpatialComponentPolygon2DListEXT

12.50.8. New Enums

- XrSpatialPlaneAlignmentEXT
- XrSpatialPlaneSemanticLabelEXT

12.50.9. New Enum Constants

- `XR_EXT_SPATIAL_PLANE_TRACKING_EXTENSION_NAME`
- `XR_EXT_spatial_plane_tracking_SPEC_VERSION`
- Extending XrSpatialCapabilityEXT:
  - `XR_SPATIAL_CAPABILITY_PLANE_TRACKING_EXT`
- Extending XrSpatialComponentTypeEXT:
  - `XR_SPATIAL_COMPONENT_TYPE_MESH_2D_EXT`
  - `XR_SPATIAL_COMPONENT_TYPE_PLANE_ALIGNMENT_EXT`
  - `XR_SPATIAL_COMPONENT_TYPE_PLANE_SEMANTIC_LABEL_EXT`
  - `XR_SPATIAL_COMPONENT_TYPE_POLYGON_2D_EXT`
- Extending XrStructureType:
  - `XR_TYPE_SPATIAL_CAPABILITY_CONFIGURATION_PLANE_TRACKING_EXT`
  - `XR_TYPE_SPATIAL_COMPONENT_MESH_2D_LIST_EXT`
  - `XR_TYPE_SPATIAL_COMPONENT_PLANE_ALIGNMENT_LIST_EXT`
  - `XR_TYPE_SPATIAL_COMPONENT_PLANE_SEMANTIC_LABEL_LIST_EXT`
  - `XR_TYPE_SPATIAL_COMPONENT_POLYGON_2D_LIST_EXT`

12.50.10. Issues

12.50.11. Version History

- Revision 1, 2024-07-02 (Nihav Jain, Google)
  - Initial extension description

12.51. XR_EXT_stationary_reference_space

Name String

`XR_EXT_stationary_reference_space`

Extension Type

Instance extension

Registered Extension Number

743

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0
and
`XR_EXT_uuid`

IP Status

No known IP claims.

Contributors

Yin Li, Microsoft
Andreas Selvik, Meta
Yuichi Taguchi, Meta
Nathan Nuber, Valve
Jakob Bornecrantz, NVIDIA
Jared Finder, Google

12.51.1. Overview

There are 3 primary motivations for applications to use a STATIONARY reference space.

1. Use a reference space that is related to the physical world and is unaffected by the "recenter" operation or user-defined room boundary.
2. Use a reference space that can regain its location in the physical world after tracking is lost, after the headset is removed and put back on, or after the device is suspended and resumed from low power mode.
3. Use a reference space that persists its origin location in the physical world across app restarts or device reboots.

12.51.2. Conventions

When this extension is not enabled, a runtime **must** not return XR_REFERENCE_SPACE_TYPE_STATIONARY_EXT from xrEnumerateReferenceSpaces.

When this extension is enabled, a runtime **may** return XR_REFERENCE_SPACE_TYPE_STATIONARY_EXT from xrEnumerateReferenceSpaces. When it is not returned, the runtime indicates the current system does not support remembering the space location in the physical world according to this extension's requirements. If it is not supported, the runtime **must** return the XR_ERROR_REFERENCE_SPACE_UNSUPPORTED error from the xrCreateReferenceSpace function when called with XR_REFERENCE_SPACE_TYPE_STATIONARY_EXT.

A STATIONARY reference space (XR_REFERENCE_SPACE_TYPE_STATIONARY_EXT) establishes a world-locked origin, gravity-aligned to exclude pitch and roll, with +Y pointing up.

Unlike LOCAL space or LOCAL_FLOOR space, the STATIONARY reference space does not prescribe the forward direction, nor the Y height at eye level or at floor level. Applications **must** not rely on any conventions regarding the height or forward direction of the STATIONARY reference space, because the runtime **may** choose the best stationary space origin in the scene which might be unrelated to the current user or the height of the floor level.

The runtime **must** not relate a system-level "recenter" operation to the STATIONARY reference space, nor raise an XrEventDataReferenceSpaceChangePending event for STATIONARY reference space only because the user did a recenter operation.

Unlike STAGE space, the origin of the STATIONARY space is not impacted by the geometry of the user-defined room boundary, if there is any. The origin of STATIONARY space **must** not move only because the user redefines the room boundary.



Note

- Applications can use the LOCAL_FLOOR space location in STATIONARY reference space to obtain the floor height.
- Applications can use the VIEW space location in STATIONARY reference space to compute the user's eye level height and forward direction at a specified time.
- Applications can use the LOCAL space location in STATIONARY reference space to reason about the recenter operation, or place the UI at a comfortable position in front of the user according to recentering.

The STATIONARY reference space enables the user to move freely through a complex environment, often many meters from where they started. The runtime **may** optimize for coordinate system stability near the user by slightly adjusting the STATIONARY reference space origin over time while the user is walking around. The runtime **must** not queue the XrEventDataReferenceSpaceChangePending event for such minor adjustments. The runtime **must** maintain an accurate location of the space origin when the user is returning near the space origin.

12.51.3. Relocation

The runtime **must** make a best effort to remember the location of the STATIONARY space origin in the physical world under the following conditions and try to relocate it after regaining tracking:

- When the tracking system loses tracking and later regains it.
- When the device is removed from the user's head and later put back on.
- When the session state is suspended to the IDLE state and later resumed to RUNNING.
- When the session is destroyed and later a new session is created.
- When the application is shut down and later relaunched.
- When the device is powered off and later turned on, and the application resumes.

The runtime **may** be unable to relocate the STATIONARY space; for example when the lighting conditions or the objects in the environment change significantly or when the user enters a different room. The runtime **may** determine that the previous STATIONARY space cannot be relocated and move on to the next generation of the STATIONARY space at a new location. When this happens, the runtime **must** queue an XrEventDataReferenceSpaceChangePending event for the STATIONARY space to notify applications about the change of the reference space origin. The runtime **must** also generate a new distinct generationId returned by xrGetStationaryReferenceSpaceGenerationIdEXT to indicate that a new generation of the STATIONARY space origin is created. In this event, the XrEventDataReferenceSpaceChangePending::poseValid **must** be XR_FALSE, because the runtime cannot establish the relation between the new space and the old one.



Note

A runtime **may** queue space change events in any order when multiple types of reference spaces change simultaneously. This is to allow runtimes to have different delays for different types of reference spaces since how they are implemented can vary between runtimes. Portable applications cannot assume a particular order for space change events.

12.51.4. Stationary space generation ID

The xrGetStationaryReferenceSpaceGenerationIdEXT function is defined as:

```
// Provided by XR_EXT_stationary_reference_space
XrResult xrGetStationaryReferenceSpaceGenerationIdEXT(
    XrSession                                       session,
    const XrStationaryReferenceSpaceGenerationIdGetInfoEXT* getInfo,
    XrStationaryReferenceSpaceGenerationIdResultEXT* generationIdResult);
```

Parameter Descriptions

- `session` is a handle to an XrSession previously created with xrCreateSession.
- `getInfo` is `NULL` or a pointer to XrStationaryReferenceSpaceGenerationIdGetInfoEXT to provide information for getting the generation ID of the `STATIONARY` reference space.
- `generationIdResult` is a pointer to XrStationaryReferenceSpaceGenerationIdResultEXT to populate with the current ID for the `STATIONARY` reference space.

During a running session, the application **can** call xrGetStationaryReferenceSpaceGenerationIdEXT to get the XrUuid corresponding to the current `STATIONARY` reference space.

Valid Usage (Implicit)

- The `XR_EXT_stationary_reference_space` extension **must** be enabled prior to calling xrGetStationaryReferenceSpaceGenerationIdEXT
- `session` **must** be a valid XrSession handle
- If `getInfo` is not `NULL`, `getInfo` **must** be a pointer to a valid XrStationaryReferenceSpaceGenerationIdGetInfoEXT structure
- `generationIdResult` **must** be a pointer to an XrStationaryReferenceSpaceGenerationIdResultEXT structure

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_FEATURE_UNSUPPORTED`
- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_SESSION_NOT_RUNNING`
- `XR_ERROR_VALIDATION_FAILURE`

The XrStationaryReferenceSpaceGenerationIdGetInfoEXT structure is defined as:

```
// Provided by XR_EXT_stationary_reference_space
typedef struct XrStationaryReferenceSpaceGenerationIdGetInfoEXT {
    XrStructureType    type;
    const void*        next;
} XrStationaryReferenceSpaceGenerationIdGetInfoEXT;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.

XrStationaryReferenceSpaceGenerationIdGetInfoEXT is an input structure for the xrGetStationaryReferenceSpaceGenerationIdEXT function.

XrStationaryReferenceSpaceGenerationIdGetInfoEXT exists for future extensibility.

Valid Usage (Implicit)

- The `XR_EXT_stationary_reference_space` extension **must** be enabled prior to using XrStationaryReferenceSpaceGenerationIdGetInfoEXT
- `type` **must** be `XR_TYPE_STATIONARY_REFERENCE_SPACE_GENERATION_ID_GET_INFO_EXT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

The XrStationaryReferenceSpaceGenerationIdResultEXT structure is defined as:

```
// Provided by XR_EXT_stationary_reference_space
typedef struct XrStationaryReferenceSpaceGenerationIdResultEXT {
    XrStructureType    type;
    void*              next;
    XrUuid             generationId;
} XrStationaryReferenceSpaceGenerationIdResultEXT;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `generationId` is an XrUuid that contains the returned space generation ID.

The XrStationaryReferenceSpaceGenerationIdResultEXT structure returns the `STATIONARY` reference space generation ID.

All `STATIONARY` reference XrSpace handles share the same UUID in a given session.

The runtime **must** not return an empty UUID, i.e. a UUID with all zeros in its bytes. The empty UUID typically reserved to represent an invalid UUID value, and the generation ID of `STATIONARY` space **must** never be invalid.

To detect whether the `STATIONARY` reference space origin has changed during a period in which the session was not running, an application gets the `generationId` after a successful xrBeginSession and compares the XrUuid with a `generationId` from a previous session.

For this use case the application is responsible for remembering the ID from a previous session.

The equality of `generationId` is a reliable indicator for the application to determine the equality of the `STATIONARY` space origin across multiple sessions, application restarts, or device restarts.

However, this does not imply that when `generationId` is different, the origin is not at the same location or in close proximity. It also does not imply that the same origin always returns the same ID.



Note

If the application is interested in identifying the space the user is in, it should use the `XR_EXT_spatial_persistence` and `XR_EXT_spatial_persistence_operations` extensions instead.

Because the generation ID of the `STATIONARY` reference space cannot be used to identify a space, the runtime **must** not require the application to request any permissions to use the stationary reference space.

For systems that are not capable of relocating the `STATIONARY` space across sessions, for example because the tracking system does not support relocalization or the platform blocks location persistence due to privacy requirements, the runtime **may** return a different generation ID for each session, and the `STATIONARY` space origin **may** be at a different location in the physical world across sessions.

The runtime **may** or **may** not use the same `generationId` for `STATIONARY` space across applications.

When the runtime loses tracking or before relocating the space origin while starting a new session or

application, it **may** attempt to relocate the space origin for reasonable duration. During this relocation phase, the runtime **must** return the previous `generationId` for the `STATIONARY` space, not raise the XrEventDataReferenceSpaceChangePending event, and return untracked data for all space locate functions when using the `STATIONARY` space, such as xrLocateSpace or xrLocateViews.

When the runtime changes the placement of the `STATIONARY` space origin in the physical world, typically after a reasonable timeout during which it attempted but failed to relocate the previous space origin, or when the runtime is certain that the user has moved to a different physical space, it **must** raise an XrEventDataReferenceSpaceChangePending event with XrEventDataReferenceSpaceChangePending::`poseValid` set to `XR_FALSE`, indicating that the new space and the old space are unrelated. The application then queries the xrGetStationaryReferenceSpaceGenerationIdEXT function to obtain the changed XrUuid if desired.

The runtime **must** not reuse the same XrUuid once the corresponding origin of the `STATIONARY` reference space is changed, i.e. the returned XrUuid **must** be unique once it is changed and will never repeat.

The runtime **must** generate a new UUID every time a new `generationId` is created, typically using the `uuid_generate*` function on Linux or the `CoCreateGuid` function on Windows.

The runtime **must** not reuse any UUID from the underlying spatial anchor or spatial entity system.

Valid Usage (Implicit)

- The `XR_EXT_stationary_reference_space` extension **must** be enabled prior to using XrStationaryReferenceSpaceGenerationIdResultEXT
- `type` **must** be `XR_TYPE_STATIONARY_REFERENCE_SPACE_GENERATION_ID_RESULT_EXT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

12.51.5. New Commands

- xrGetStationaryReferenceSpaceGenerationIdEXT

12.51.6. New Structures

- XrStationaryReferenceSpaceGenerationIdGetInfoEXT
- XrStationaryReferenceSpaceGenerationIdResultEXT

12.51.7. New Enum Constants

- `XR_EXT_STATIONARY_REFERENCE_SPACE_EXTENSION_NAME`
- `XR_EXT_stationary_reference_space_SPEC_VERSION`
- Extending XrReferenceSpaceType:

    - XR_REFERENCE_SPACE_TYPE_STATIONARY_EXT
- Extending XrStructureType:
    - XR_TYPE_STATIONARY_REFERENCE_SPACE_GENERATION_ID_GET_INFO_EXT
    - XR_TYPE_STATIONARY_REFERENCE_SPACE_GENERATION_ID_RESULT_EXT

12.51.8. Issues

12.51.9. Version History

- Revision 1, 2026-04-22 (Yin Li)
    - Initial extension description

12.52. XR_EXT_thermal_query

Name String

XR_EXT_thermal_query

Extension Type

Instance extension

Registered Extension Number

17

Revision

2

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Last Modified Date

2021-04-14

IP Status

No known IP claims.

Contributors

Armelle Laine, Qualcomm Technologies Inc, on behalf of Qualcomm Innovation Center, Inc

12.52.1. Overview

This extension provides an API to query a domain's current thermal warning level and current thermal trend.

12.52.2. Querying the current thermal level and trend

This query allows to determine the extent and urgency of the needed workload reduction and to verify that the mitigation measures efficiently reduce the temperature.
This query allows the application to retrieve the current `notificationLevel`, allowing to quickly verify whether the underlying system's thermal throttling is still in effect.
It also provides the application with the remaining temperature headroom (`tempHeadroom`) until thermal throttling occurs, and the current rate of change (`tempSlope`).
The most critical temperature of the domain is the one which is currently most likely to be relevant for thermal throttling.

To query the status of a given domain:

```
// Provided by XR_EXT_thermal_query
XrResult xrThermalGetTemperatureTrendEXT(
    XrSession                                   session,
    XrPerfSettingsDomainEXT                     domain,
    XrPerfSettingsNotificationLevelEXT*         notificationLevel,
    float*                                      tempHeadroom,
    float*                                      tempSlope);
```

```
// Provided by XR_EXT_performance_settings, XR_EXT_thermal_query
typedef enum XrPerfSettingsDomainEXT {
    XR_PERF_SETTINGS_DOMAIN_CPU_EXT = 1,
    XR_PERF_SETTINGS_DOMAIN_GPU_EXT = 2,
    XR_PERF_SETTINGS_DOMAIN_MAX_ENUM_EXT = 0x7FFFFFFF
} XrPerfSettingsDomainEXT;
```

```
// Provided by XR_EXT_performance_settings, XR_EXT_thermal_query
typedef enum XrPerfSettingsNotificationLevelEXT {
    XR_PERF_SETTINGS_NOTIF_LEVEL_NORMAL_EXT = 0,
    XR_PERF_SETTINGS_NOTIF_LEVEL_WARNING_EXT = 25,
    XR_PERF_SETTINGS_NOTIF_LEVEL_IMPAIRED_EXT = 75,
    XR_PERF_SETTINGS_NOTIFICATION_LEVEL_MAX_ENUM_EXT = 0x7FFFFFFF
} XrPerfSettingsNotificationLevelEXT;
```

For the definition of the notification levels, see Notification level definition.

12.52.3. Thermal Query API Reference

xrThermalGetTemperatureTrendEXT

```
// Provided by XR_EXT_thermal_query
XrResult xrThermalGetTemperatureTrendEXT(
    XrSession                                   session,
    XrPerfSettingsDomainEXT                     domain,
    XrPerfSettingsNotificationLevelEXT*         notificationLevel,
    float*                                      tempHeadroom,
    float*                                      tempSlope);
```

Allows to query the current temperature warning level of a domain, the remaining headroom and the trend.

Parameter Descriptions

- `session` is a valid XrSession handle.
- `domain` : the processing domain
- `notificationLevel` : the current warning level
- `tempHeadroom` : temperature headroom in degrees Celsius, expressing how far the most-critical temperature of the domain is from its thermal throttling threshold temperature.
- `tempSlope` : the current trend in degrees Celsius per second of the most critical temperature of the domain.

Valid Usage (Implicit)

- The `XR_EXT_thermal_query` extension **must** be enabled prior to calling xrThermalGetTemperatureTrendEXT
- `session` **must** be a valid XrSession handle
- `domain` **must** be a valid XrPerfSettingsDomainEXT value
- `notificationLevel` **must** be a pointer to an XrPerfSettingsNotificationLevelEXT value
- `tempHeadroom` **must** be a pointer to a `float` value
- `tempSlope` **must** be a pointer to a `float` value

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

```
// Provided by XR_EXT_performance_settings, XR_EXT_thermal_query
typedef enum XrPerfSettingsDomainEXT {
    XR_PERF_SETTINGS_DOMAIN_CPU_EXT = 1,
    XR_PERF_SETTINGS_DOMAIN_GPU_EXT = 2,
    XR_PERF_SETTINGS_DOMAIN_MAX_ENUM_EXT = 0x7FFFFFFF
} XrPerfSettingsDomainEXT;
```

```
// Provided by XR_EXT_performance_settings, XR_EXT_thermal_query
typedef enum XrPerfSettingsNotificationLevelEXT {
    XR_PERF_SETTINGS_NOTIF_LEVEL_NORMAL_EXT = 0,
    XR_PERF_SETTINGS_NOTIF_LEVEL_WARNING_EXT = 25,
    XR_PERF_SETTINGS_NOTIF_LEVEL_IMPAIRED_EXT = 75,
    XR_PERF_SETTINGS_NOTIFICATION_LEVEL_MAX_ENUM_EXT = 0x7FFFFFFF
} XrPerfSettingsNotificationLevelEXT;
```

Version History

- Revision 1, 2017-11-30 (Armelle Laine)
- Revision 2, 2021-04-14 (Rylie Pavlik, Collabora, Ltd.)
  - Fix missing error code

12.53. XR_EXT_user_presence

Name String

`XR_EXT_user_presence`

Extension Type

Instance extension

Registered Extension Number

471

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Last Modified Date

2023-04-22

IP Status

No known IP claims.

Contributors

Yin Li, Microsoft
Bryce Hutchings, Microsoft
John Kearney, Meta Platforms
Andreas Loeve Selvik, Meta Platforms
Peter Kuhn, Unity Technologies
Jakob Bornecrantz, Collabora

12.53.1. Overview

This extension introduces a new event to notify when the system detected the change of user presence, such as when the user has taken off or put on an XR headset.

This event is typically used by an XR applications with non-XR experiences outside of the XR headset. For instance, some applications pause the game logic or video playback until the user puts on the headset, displaying an instructional message to the user in the mirror window on the desktop PC monitor. As another example, the application might use this event to disable a head-tracking driven avatar in an online meeting when the user has taken off the headset.

The user presence is fundamentally decoupled from the session lifecycle. Although the core spec for XrSessionState hinted potential correlation between the session state and user presence, in practice,

such a connection **may** not consistently hold across various runtimes. Application **should** avoid relying on assumptions regarding these relationships between session state and user presence, instead, they should utilize this extension to reliably obtain user presence information.

12.53.2. System Supports User Presence

The XrSystemUserPresencePropertiesEXT structure is defined as:

```
// Provided by XR_EXT_user_presence
typedef struct XrSystemUserPresencePropertiesEXT {
    XrStructureType    type;
    void*              next;
    XrBool32           supportsUserPresence;
} XrSystemUserPresencePropertiesEXT;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension. For more details about the structure chain, see the structure being extended (XrSystemProperties).
- `supportsUserPresence` is an `XrBool32` value that indicates whether the system supports user presence sensing.

The application **can** use the XrSystemUserPresencePropertiesEXT event in xrGetSystemProperties to detect if the given system supports the sensing of user presence.

If the system does not support user presence sensing, the runtime **must** return `XR_FALSE` for `supportsUserPresence` and **must** not queue the XrEventDataUserPresenceChangedEXT event for any session on this system.

In this case, an application typically assumes that the user is always present, as the runtime is unable to detect changes in user presence.

Valid Usage (Implicit)

- The `XR_EXT_user_presence` extension **must** be enabled prior to using XrSystemUserPresencePropertiesEXT
- `type` **must** be `XR_TYPE_SYSTEM_USER_PRESENCE_PROPERTIES_EXT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

12.53.3. User Presence Changed Event

The XrEventDataUserPresenceChangedEXT structure is defined as:

```
// Provided by XR_EXT_user_presence
typedef struct XrEventDataUserPresenceChangedEXT {
    XrStructureType    type;
    const void*        next;
    XrSession          session;
    XrBool32           isUserPresent;
} XrEventDataUserPresenceChangedEXT;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `session` is the XrSession that is receiving the notification.
- `isUserPresent` is an `XrBool32` value for new state of user presence after the change.

The XrEventDataUserPresenceChangedEXT event is queued for retrieval using xrPollEvent when the user presence is changed, as well as when a session starts running.

Receiving XrEventDataUserPresenceChangedEXT with the `isUserPresent` is `XR_TRUE` indicates that the system has detected the presence of a user in the XR experience. For example, this may indicate that the user has put on the headset, or has entered the tracking area of a non-head-worn XR system.

Receiving XrEventDataUserPresenceChangedEXT with the `isUserPresent` is `XR_FALSE` indicates that the system has detected the absence of a user in the XR experience. For example, this may indicate that the user has removed the headset or has stepped away from the tracking area of a non-head-worn XR system.

The runtime **must** queue this event upon a successful call to the xrBeginSession function, regardless of the value of `isUserPresent`, so that the application can be in sync on the state when a session begins running.

The runtime **must** return a valid XrSession handle for a running session.

After the application calls xrEndSession, a running session is ended and the runtime **must** not enqueue any more user presence events. Therefore, the application will no longer observe any changes of the `isUserPresent` until another running session.



Note

This extension does not require any specific correlation between user presence state and session state except that the XrEventDataUserPresenceChangedEXT event can not be observed without a running session. A runtime may choose to correlate the two states or keep them independent.

Valid Usage (Implicit)

- The `XR_EXT_user_presence` extension **must** be enabled prior to using XrEventDataUserPresenceChangedEXT
- `type` **must** be `XR_TYPE_EVENT_DATA_USER_PRESENCE_CHANGED_EXT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

Example 2. Proper Method for Receiving OpenXR Event Data

```
XrInstance instance;  // previously initialized
XrSystemId systemId;  // previously initialized
XrSession session;  // previously initialized

XrSystemUserPresencePropertiesEXT userPresenceProperties {
    .type = XR_TYPE_SYSTEM_USER_PRESENCE_PROPERTIES_EXT,
};
XrSystemProperties systemProperties{
    .type = XR_TYPE_SYSTEM_PROPERTIES,
    .next = &userPresenceProperties,
};
CHK_XR(xrGetSystemProperties(instance, systemId, &systemProperties));
bool supportsUserPresence = userPresenceProperties.supportsUserPresence;

// When either the extension is not supported or the system does not support the
sensor,
// the application typically assumes user always present, and initialize the
isUserPresent
// to true before xrBeginSession and reset it to false after xrEndSession.
bool isUserPresent = true;

// Initialize an event buffer to hold the output.
XrEventDataBuffer event {
    .type = XR_TYPE_EVENT_DATA_BUFFER,
};
XrResult result = xrPollEvent(instance, &event);
if (result == XR_SUCCESS) {
    switch (event.type) {
        case XR_TYPE_EVENT_DATA_SESSION_STATE_CHANGED: {
            const XrEventDataSessionStateChanged& eventdata =
                *reinterpret_cast<XrEventDataSessionStateChanged*>(&event);
            XrSessionState sessionState = eventdata.state;
            switch(sessionState)
            {
              case XR_SESSION_STATE_READY: {
                  isUserPresent = true;
                  XrSessionBeginInfo beginInfo {
                      .type = XR_TYPE_SESSION_BEGIN_INFO,
                  };
                  CHK_XR(xrBeginSession(session, &beginInfo));
                  break;
              }
              case XR_SESSION_STATE_STOPPING:{
                  CHK_XR(xrEndSession(session));
                  isUserPresent = false;
```

```
                break;
              }
            }
            break;
        }
        case XR_TYPE_EVENT_DATA_USER_PRESENCE_CHANGED_EXT: {
            const XrEventDataUserPresenceChangedEXT& eventdata =
                *reinterpret_cast<XrEventDataUserPresenceChangedEXT*>(&event);
            isUserPresent = eventdata.isUserPresent;
            // do_something(isUserPresent);
            break;
        }
    }
}
```

New Object Types

New Flag Types

New Enum Constants

XrStructureType enumeration is extended with:

- `XR_TYPE_EVENT_DATA_USER_PRESENCE_CHANGED_EXT`
- `XR_TYPE_SYSTEM_USER_PRESENCE_PROPERTIES_EXT`

New Enums

New Structures

- XrSystemUserPresencePropertiesEXT
- XrEventDataUserPresenceChangedEXT

New Functions

Issues

Version History

- Revision 1, 2023-04-22 (Yin Li)
  - Initial extension description

12.54. XR_EXT_view_configuration_depth_range

Name String

`XR_EXT_view_configuration_depth_range`

Extension Type

Instance extension

Registered Extension Number

47

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

OpenXR 1.0

Last Modified Date

2019-08-16

IP Status

No known IP claims.

Contributors

Blake Taylor, Magic Leap
Gilles Cadet, Magic Leap
Michael Liebenow, Magic Leap
Supreet Suresh, Magic Leap
Alex Turner, Microsoft
Bryce Hutchings, Microsoft
Yin Li, Microsoft

Overview

For XR systems there may exist a per view recommended min/max depth range at which content should be rendered into the virtual world. The depth range may be driven by several factors, including user comfort, or fundamental capabilities of the system.

Displaying rendered content outside the recommended min/max depth range would violate the system requirements for a properly integrated application, and can result in a poor user experience due to observed visual artifacts, visual discomfort, or fatigue. The near/far depth values will fall in the range of (0..+infinity] where max(`recommendedNearZ`, `minNearZ`) < min(`recommendedFarZ`, `maxFarZ`). Infinity is defined matching the standard library definition such that std::isinf will return true for a returned infinite value.

In order to provide the application with the appropriate depth range at which to render content for each XrViewConfigurationView, this extension provides additional view configuration information, as defined by XrViewConfigurationDepthRangeEXT, to inform the application of the min/max recommended and absolute distances at which content should be rendered for that view.

New Object Types

New Flag Types

New Enum Constants

XrStructureType enumeration is extended with:

- XR_TYPE_VIEW_CONFIGURATION_DEPTH_RANGE_EXT

New Enums

New Structures

The XrViewConfigurationDepthRangeEXT structure is defined as:

```
// Provided by XR_EXT_view_configuration_depth_range
typedef struct XrViewConfigurationDepthRangeEXT {
    XrStructureType    type;
    void*              next;
    float              recommendedNearZ;
    float              minNearZ;
    float              recommendedFarZ;
    float              maxFarZ;
} XrViewConfigurationDepthRangeEXT;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `recommendedNearZ` is the recommended minimum positive distance in meters that content should be rendered for the view to achieve the best user experience.
- `minNearZ` is the absolute minimum positive distance in meters that content should be rendered for the view.
- `recommendedFarZ` is the recommended maximum positive distance in meters that content should be rendered for the view to achieve the best user experience.
- `maxFarZ` is the absolute maximum positive distance in meters that content should be rendered for the view.

When enumerating the view configurations with xrEnumerateViewConfigurationViews, the application **can** provide a pointer to an XrViewConfigurationDepthRangeEXT in the `next` chain of XrViewConfigurationView.

Valid Usage (Implicit)

- The `XR_EXT_view_configuration_depth_range` extension **must** be enabled prior to using XrViewConfigurationDepthRangeEXT
- `type` **must** be `XR_TYPE_VIEW_CONFIGURATION_DEPTH_RANGE_EXT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

New Functions

Issues

Version History

- Revision 1, 2019-10-01 (Blake Taylor)
  - Initial proposal.

12.55. XR_EXT_view_configuration_views_change

Name String

`XR_EXT_view_configuration_views_change`

Extension Type

Instance extension

Registered Extension Number

840

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Last Modified Date

2025-12-11

IP Status

No known IP claims.

Contributors

Paul Pedriana, Oculus
Rune Berg, Valve
Aidan Howie, Holo-Light GmbH
Robert Menzel, NVIDIA
Yin Li, Microsoft
Rylie Pavlik, Collabora
Denny Rönngren, Varjo
Jakob Bornecrantz, NVIDIA
Bastiaan Olij, Godot Engine
Nihav Jain, Google,
Aaron Leiby, Valve Corporation

12.55.1. Overview

This extension enables runtimes to notify applications that the recommended view configuration values (e.g. XrViewConfigurationView::`recommendedImageRectWidth`) have changed. The runtime **may** notify the application that the view configuration values have changed via the XrEventDataViewConfigurationViewsChangedEXT event.

The runtime **must** not use this event for frequent (at a rate faster than 1Hz per view configuration) adjustments of the resolution, e.g. to adjust the viewport based on current performance. This is to avoid expensive reallocations of swapchain images by the application at a quick rate.

The updated recommendation can be found by calling xrEnumerateViewConfigurationViews for the XrViewConfigurationType returned in XrEventDataViewConfigurationViewsChangedEXT. The application **should** reallocate the swapchain being rendering to, by calling xrCreateSwapchain. The application **may** choose to ignore the updated recommendations, however this may result in undesired compositing.

12.55.2. Changes to xrEnumerateViewConfigurationViews behavior

This extension modifies the behavior specified in xrEnumerateViewConfigurationViews in the following way:

- The runtime **may** return non-identical buffer contents from the xrEnumerateViewConfigurationViews enumeration for the given `systemId` and `viewConfigurationType` for the lifetime of the instance. The runtime **must** only change the content of the recommended values within XrViewConfigurationView, e.g. `recommendedImageRectWidth`.

12.55.3. Views Change Event

The XrEventDataViewConfigurationViewsChangedEXT structure is defined as:

```
// Provided by XR_EXT_view_configuration_views_change
typedef struct XrEventDataViewConfigurationViewsChangedEXT {
    XrStructureType            type;
    const void*                next;
    XrSystemId                 systemId;
    XrViewConfigurationType    viewConfigurationType;
} XrEventDataViewConfigurationViewsChangedEXT;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `systemId` is an `XrSystemId` that is notifying that the XrViewConfigurationView viewport configuration has changed.
- `viewConfigurationType` is the viewport configuration type that the runtime has new recommended XrViewConfigurationView values for. This may happen for example if the user has changed resolution settings in the runtime.

Valid Usage (Implicit)

- The `XR_EXT_view_configuration_views_change` extension **must** be enabled prior to using XrEventDataViewConfigurationViewsChangedEXT
- `type` **must** be `XR_TYPE_EVENT_DATA_VIEW_CONFIGURATION_VIEWS_CHANGED_EXT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

12.55.4. New Structures

- XrEventDataViewConfigurationViewsChangedEXT

12.55.5. New Enum Constants

- `XR_EXT_VIEW_CONFIGURATION_VIEWS_CHANGE_EXTENSION_NAME`
- `XR_EXT_view_configuration_views_change_SPEC_VERSION`
- Extending XrStructureType:
  - `XR_TYPE_EVENT_DATA_VIEW_CONFIGURATION_VIEWS_CHANGED_EXT`

12.55.6. Issues

12.55.7. Version History

- Revision 1, 2025-12-11
  - Initial version, based on early draft changes.

12.56. XR_EXT_win32_appcontainer_compatible

Name String

`XR_EXT_win32_appcontainer_compatible`

Extension Type

Instance extension

Registered Extension Number

58

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Last Modified Date

2019-12-16

IP Status

No known IP claims.

Contributors

Yin Li, Microsoft
Alex Turner, Microsoft
Lachlan Ford, Microsoft

Overview

To minimize opportunities for malicious manipulation, a common practice on the Windows OS is to isolate the application process in an AppContainer execution environment. In order for a runtime to work properly in such an application process, the runtime **must** properly set ACL to device resources and cross process resources.

An application running in an AppContainer process **can** request for a runtime to enable such AppContainer compatibility by adding `XR_EXT_WIN32_APPCONTAINER_COMPATIBLE_EXTENSION_NAME` to `enabledExtensionNames` of XrInstanceCreateInfo when calling xrCreateInstance. If the runtime is not capable of running properly within the AppContainer execution environment, it **must** return `XR_ERROR_EXTENSION_NOT_PRESENT`.

If the runtime supports this extension, it **can** further inspect the capability based on the connected device. If the XR system cannot support an AppContainer execution environment, the runtime **must** return `XR_ERROR_FORM_FACTOR_UNAVAILABLE` when the application calls xrGetSystem.

If the call to xrGetSystem successfully returned with a valid `XrSystemId`, the application **can** rely on the runtime working properly in the AppContainer execution environment.

New Object Types

New Flag Types

New Enum Constants

New Enums

New Structures

New Functions

Issues

Version History

- Revision 1, 2019-12-16 (Yin Li)
    - Initial proposal.

12.57. XR_ALMALENCE_digital_lens_control

Name String

`XR_ALMALENCE_digital_lens_control`

Extension Type

Instance extension

Registered Extension Number

197

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Last Modified Date

2021-11-08

IP Status

No known IP claims.

Contributors

Ivan Chupakhin, Almalence Inc.
Dmitry Shmunk, Almalence Inc.

Overview

Digital Lens for VR (DLVR) is a computational lens aberration correction technology enabling high resolution, visual clarity and fidelity in VR head mounted displays. The Digital Lens allows to overcome two fundamental factors limiting VR picture quality, size constraints and presence of a moving optical element — the eye pupil.

Features:

- Complete removal of lateral chromatic aberrations, across the entire FoV, at all gaze directions.

- Correction of longitudinal chromatic aberrations, lens blur and higher order aberrations.
- Increase of visible resolution.
- Enhancement of edge contrast (otherwise degraded due to lens smear).
- Enables high quality at wide FoV.

For OpenXR runtimes DLVR is implemented as implicit API Layer distributed by Almalence Inc. as installable package. DLVR utilize eye tracking data (eye pupil coordinates and gaze direction) to produce corrections of render frames. As long as current core OpenXR API does not expose an eye tracking data, DLVR API Layer relies on 3rd-party eye tracking runtimes.

List of supported eye tracking devices:

- *Tobii_VR4_CARBON_P1* (HP Reverb G2 Omnicept Edition)
- *Tobii_VR4_U2_P2* (HTC Vive Pro Eye)

This extension enables the handling of the Digital Lens for VR API Layer by calling xrSetDigitalLensControlALMALENCE.

New Object Types

New Flag Types

```
// Provided by XR_ALMALENCE_digital_lens_control
typedef XrFlags64 XrDigitalLensControlFlagsALMALENCE;
```

```
// Provided by XR_ALMALENCE_digital_lens_control
// Flag bits for XrDigitalLensControlFlagsALMALENCE
static const XrDigitalLensControlFlagsALMALENCE
XR_DIGITAL_LENS_CONTROL_PROCESSING_DISABLE_BIT_ALMALENCE = 0x00000001;
```

Flag Descriptions

- XR_DIGITAL_LENS_CONTROL_PROCESSING_DISABLE_BIT_ALMALENCE — disables Digital Lens processing of render textures

New Enum Constants

XrStructureType enumeration is extended with:

- XR_TYPE_DIGITAL_LENS_CONTROL_ALMALENCE

New Enums

New Structures

The XrDigitalLensControlALMALENCE structure is defined as:

```
// Provided by XR_ALMALENCE_digital_lens_control
typedef struct XrDigitalLensControlALMALENCE {
    XrStructureType                       type;
    const void*                           next;
    XrDigitalLensControlFlagsALMALENCE    flags;
} XrDigitalLensControlALMALENCE;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` must be `NULL`. No such structures are defined in core OpenXR or this extension.
- `flags` is a bitmask of XrDigitalLensControlFlagBitsALMALENCE indicating various characteristics desired for the Digital Lens.

Valid Usage (Implicit)

- The `XR_ALMALENCE_digital_lens_control` extension **must** be enabled prior to using XrDigitalLensControlALMALENCE
- `type` **must** be `XR_TYPE_DIGITAL_LENS_CONTROL_ALMALENCE`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `flags` **must** be a valid combination of XrDigitalLensControlFlagBitsALMALENCE values
- `flags` **must** not be `0`

New Functions

The xrSetDigitalLensControlALMALENCE function is defined as:

```
// Provided by XR_ALMALENCE_digital_lens_control
XrResult xrSetDigitalLensControlALMALENCE(
    XrSession                                   session,
    const XrDigitalLensControlALMALENCE*        digitalLensControl);
```

Parameter Descriptions

- `session` is a handle to a running XrSession.
- `digitalLensControl` is the XrDigitalLensControlALMALENCE that contains desired characteristics for the Digital Lens

xrSetDigitalLensControlALMALENCE handles state of Digital Lens API Layer

Valid Usage (Implicit)

- The `XR_ALMALENCE_digital_lens_control` extension **must** be enabled prior to calling xrSetDigitalLensControlALMALENCE
- `session` **must** be a valid XrSession handle
- `digitalLensControl` **must** be a pointer to a valid XrDigitalLensControlALMALENCE structure

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`

Issues

Version History

- Revision 1, 2021-11-08 (Ivan Chupakhin)
  - Initial draft

12.58. XR_ANDROID_anchor_sharing_export

Name String

`XR_ANDROID_anchor_sharing_export`

Extension Type

Instance extension

Registered Extension Number

702

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Last Modified Date

2025-07-23

IP Status

No known IP claims.

Contributors

Nihav Jain, Google
Levana Chen, Google
Spencer Quin, Google
Kenny Vercaemer, Google

12.58.1. Overview

Anchors created by an application via `XR_ANDROID_trackables`, `XR_EXT_spatial_anchor`, and `XR_ANDROID_spatial_anchor_space` are tied to a specific session and are thus not visible to other applications. This extension allows applications to share anchors with other running applications.

This extension does not define any functions that make use of shared anchors. Other extensions **may** define such functions.



Permissions

Android applications **must** have the android.permission.SCENE_UNDERSTANDING_COARSE permission listed in their manifest as this extension depends on `XR_ANDROID_trackables` and exposes the geometry of the environment. The android.permission.SCENE_UNDERSTANDING_COARSE permission is considered a dangerous permission.

(protection level: dangerous)

12.58.2. Inspect system capability

The XrSystemAnchorSharingExportPropertiesANDROID structure is defined as:

```
// Provided by XR_ANDROID_anchor_sharing_export
typedef struct XrSystemAnchorSharingExportPropertiesANDROID {
    XrStructureType    type;
    void*              next;
    XrBool32           supportsAnchorSharingExport;
} XrSystemAnchorSharingExportPropertiesANDROID;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `supportsAnchorSharingExport` is an `XrBool32` indicating if current system supports anchor sharing.

An application **can** inspect whether the system is capable of sharing anchors by chaining an XrSystemAnchorSharingExportPropertiesANDROID structure to the XrSystemProperties::`next` chain when calling xrGetSystemProperties.

If and only if a runtime returns `XR_FALSE` for `supportsAnchorSharingExport`, the runtime **must** return `XR_ERROR_FEATURE_UNSUPPORTED` from xrShareAnchorANDROID and xrUnshareAnchorANDROID.

Valid Usage (Implicit)

- The XR_ANDROID_anchor_sharing_export extension **must** be enabled prior to using XrSystemAnchorSharingExportPropertiesANDROID
- type **must** be XR_TYPE_SYSTEM_ANCHOR_SHARING_EXPORT_PROPERTIES_ANDROID
- next **must** be NULL or a valid pointer to the next structure in a structure chain

12.58.3. Share an anchor

The xrShareAnchorANDROID function is defined as:

```
// Provided by XR_ANDROID_anchor_sharing_export
XrResult xrShareAnchorANDROID(
    XrSession                                   session,
    const XrAnchorSharingInfoANDROID*           sharingInfo,
    XrAnchorSharingTokenANDROID*                anchorToken);
```

Parameter Descriptions

- session is the XrSession that owns the anchor.
- sharingInfo is the XrAnchorSharingInfoANDROID with the information about the anchor to share.
- anchorToken is a pointer to the XrAnchorSharingTokenANDROID in which the runtime returns the anchor sharing token.

The xrShareAnchorANDROID function returns a share token to the application. The application **can** share this token with other applications to give them access to the pose data of XrAnchorSharingInfoANDROID::anchor.

The runtime **must** return XR_ERROR_ANCHOR_NOT_OWNED_BY_CALLER_ANDROID if XrAnchorSharingInfoANDROID::anchor was not originally created by the application that calls xrShareAnchorANDROID.

If the application calls xrShareAnchorANDROID for the same XrAnchorSharingInfoANDROID::anchor multiple times without unsharing it first, the runtime **must** return the same share token each time.

If the application calls xrShareAnchorANDROID for an anchor that was previously shared but then unshared, the runtime **may** return a share token that is different from the previous sharing scope. The lifetime of the anchorToken share token is limited to the application controlled share-unshare scope.

Valid Usage (Implicit)

- The XR_ANDROID_anchor_sharing_export extension **must** be enabled prior to calling xrShareAnchorANDROID
- `session` **must** be a valid XrSession handle
- `sharingInfo` **must** be a pointer to a valid XrAnchorSharingInfoANDROID structure
- `anchorToken` **must** be a pointer to an XrAnchorSharingTokenANDROID structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_ANCHOR_NOT_OWNED_BY_CALLER_ANDROID
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The XrAnchorSharingInfoANDROID structure is defined as:

```
// Provided by XR_ANDROID_anchor_sharing_export
typedef struct XrAnchorSharingInfoANDROID {
    XrStructureType    type;
    const void*        next;
    XrSpace            anchor;
} XrAnchorSharingInfoANDROID;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `anchor` is the XrSpace of the anchor to share.

XrAnchorSharingInfoANDROID::`anchor` **must** be a valid XrSpace created previously by calling xrCreateAnchorSpaceANDROID, xrCreatePersistedAnchorSpaceANDROID, or xrCreateSpatialAnchorSpaceANDROID.

Valid Usage (Implicit)

- The `XR_ANDROID_anchor_sharing_export` extension **must** be enabled prior to using XrAnchorSharingInfoANDROID
- `type` **must** be `XR_TYPE_ANCHOR_SHARING_INFO_ANDROID`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `anchor` **must** be a valid XrSpace handle

The XrAnchorSharingTokenANDROID structure is defined as:

```
// Provided by XR_ANDROID_anchor_sharing_export
typedef struct XrAnchorSharingTokenANDROID {
    XrStructureType     type;
    void*               next;
    struct AIBinder*    token;
} XrAnchorSharingTokenANDROID;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `token` is the Android binder reference that the runtime returns to the application.

Other extensions **may** define functions that use the XrAnchorSharingTokenANDROID::`token` to access the anchor from other sessions.

Valid Usage (Implicit)

- The XR_ANDROID_anchor_sharing_export extension **must** be enabled prior to using XrAnchorSharingTokenANDROID
- type **must** be XR_TYPE_ANCHOR_SHARING_TOKEN_ANDROID
- next **must** be NULL or a valid pointer to the next structure in a structure chain

12.58.4. Unshare an anchor

The xrUnshareAnchorANDROID function is defined as:

```
// Provided by XR_ANDROID_anchor_sharing_export
XrResult xrUnshareAnchorANDROID(
    XrSession                                   session,
    XrSpace                                     anchor);
```

Parameter Descriptions

- session is the XrSession that owns the anchor.
- anchor is the XrSpace of the anchor to unshare.

The xrUnshareAnchorANDROID function invalidates previously shared anchors. This means that future uses of the anchor sharing token obtained when xrShareAnchorANDROID was called **must** fail until the anchor is shared again via xrShareAnchorANDROID. The runtime **must** also invalidate previous imports of the anchor sharing token and **must** not set XR_SPACE_LOCATION_POSITION_TRACKED_BIT or XR_SPACE_LOCATION_ORIENTATION_TRACKED_BIT for the imported anchor's XrSpace. This means that a previously imported anchor **must** stop tracking if the original anchor is unshared by its parent session.

The runtime **must** return XR_ERROR_ANCHOR_NOT_OWNED_BY_CALLER_ANDROID if the anchor is not owned by the same XrSession that is the parent of XrAnchorSharingInfoANDROID::anchor.

An anchor that is not unshared explicitly by calling xrUnshareAnchorANDROID **must** be implicitly unshared by the runtime when the anchor is destroyed via xrDestroySpace, or when the parent XrSession is destroyed, including when the application that shared the anchor quits.

Valid Usage (Implicit)

- The `XR_ANDROID_anchor_sharing_export` extension **must** be enabled prior to calling xrUnshareAnchorANDROID
- `session` **must** be a valid XrSession handle
- `anchor` **must** be a valid XrSpace handle
- `anchor` **must** have been created, allocated, or retrieved from `session`

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_ANCHOR_NOT_OWNED_BY_CALLER_ANDROID`
- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_OUT_OF_MEMORY`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_VALIDATION_FAILURE`

12.58.5. Example code for anchor sharing export

The following example code demonstrates how to share and unshare an anchor with other applications.

```
#if defined(XR_USE_PLATFORM_ANDROID)
XrSession session; // previously initialized
XrSpace anchor; // created via xrCreateAnchorSpaceANDROID() or
xrCreatePersistedAnchorSpaceANDROID()

// The function pointers are previously initialized using xrGetInstanceProcAddr.
PFN_xrShareAnchorANDROID xrShareAnchorANDROID; // previously initialized
PFN_xrUnshareAnchorANDROID xrUnshareAnchorANDROID; // previously initialized

XrAnchorSharingInfoANDROID sharingInfo {
  .type = XR_TYPE_ANCHOR_SHARING_INFO_ANDROID,
  .next = nullptr,
  .anchor = anchor,
};

AIBinder* anchorToken;
CHK_XR(xrShareAnchorANDROID(
    session,
  &sharingInfo,
  &anchorToken
));

// ... share anchorToken with other processes ...

// Once app no longer wants to share the anchor
CHK_XR(xrUnshareAnchorANDROID(session, anchor));

// Once app is done using the anchor. This will also unshare
// the anchor if the anchor has not been unshared explicitly before.
CHK_XR(xrDestroySpace(anchor));
#endif
```

12.58.6. New Commands

- xrShareAnchorANDROID
- xrUnshareAnchorANDROID

12.58.7. New Structures

- XrAnchorSharingInfoANDROID
- XrAnchorSharingTokenANDROID
- Extending XrSystemProperties:
  - XrSystemAnchorSharingExportPropertiesANDROID

12.58.8. New Enum Constants

- XR_ANDROID_ANCHOR_SHARING_EXPORT_EXTENSION_NAME
- XR_ANDROID_anchor_sharing_export_SPEC_VERSION
- Extending XrResult:
  - XR_ERROR_ANCHOR_NOT_OWNED_BY_CALLER_ANDROID
- Extending XrStructureType:
  - XR_TYPE_ANCHOR_SHARING_INFO_ANDROID
  - XR_TYPE_ANCHOR_SHARING_TOKEN_ANDROID
  - XR_TYPE_SYSTEM_ANCHOR_SHARING_EXPORT_PROPERTIES_ANDROID

12.58.9. Issues

12.58.10. Version History

- Revision 1, 2025-07-23 (Kenny Vercaemer)
  - Initial extension description

12.59. XR_ANDROID_composition_layer_passthrough_mesh

Name String

XR_ANDROID_composition_layer_passthrough_mesh

Extension Type

Instance extension

Registered Extension Number

463

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Last Modified Date

2024-09-18

IP Status

No known IP claims.

Contributors

Grant Yoshida, Google
Kevin Moule, Google
Vasiliy Baranov, Google
Peter Chen, Google
Levana Chen, Google

12.59.1. Overview

For devices that support multiple environment blend modes, the runtime **may** provide passthrough configurations to show a user their physical environment from an immersive view.

This extension enables applications to project passthrough textures onto arbitrary geometry through an additional composition layer XrCompositionLayerPassthroughANDROID.

The passthrough layer characteristics are specified by the following parameters, in which the projection is represented by XrPassthroughLayerANDROID.

```
    XrPosef                     pose;
    XrVector3f                  scale;
    float                       opacity;
    XrPassthroughLayerANDROID   layer;
```

For full screen passthrough, applications **should** prefer Environment Blend Mode with available blend modes.

12.59.2. Inspect system capability

The XrSystemPassthroughLayerPropertiesANDROID structure is defined as:

```
// Provided by XR_ANDROID_composition_layer_passthrough_mesh
typedef struct XrSystemPassthroughLayerPropertiesANDROID {
    XrStructureType    type;
    void*              next;
    XrBool32           supportsPassthroughLayer;
    uint32_t           maxMeshIndexCount;
    uint32_t           maxMeshVertexCount;
} XrSystemPassthroughLayerPropertiesANDROID;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `supportsPassthroughLayer` is an `XrBool32`, indicating if the current system supports composition layer passthrough mesh.
- `maxMeshIndexCount` is a `uint32_t` returns the maximum count of indices that will be accepted for a passthrough mesh.
- `maxMeshVertexCount` is a `uint32_t` returns the maximum count of vertices that will be accepted for a passthrough mesh.

An application inspects whether the system is capable of composition layer passthrough mesh by chaining an XrSystemPassthroughLayerPropertiesANDROID structure to the XrSystemProperties when calling xrGetSystemProperties.

If a runtime returns `XR_FALSE` for `supportsPassthroughLayer`, the system does not support composition layer passthrough mesh, and therefore **must** return `XR_ERROR_FEATURE_UNSUPPORTED` from xrCreatePassthroughLayerANDROID. The application **should** avoid using composition layer passthrough mesh when `supportsPassthroughLayer` is `XR_FALSE`.

If a runtime returns `XR_TRUE` for `supportsPassthroughLayer`, the system supports composition layer passthrough mesh. In this case, the runtime **must** return a non-zero number for `maxMeshIndexCount` and `maxMeshVertexCount`. An application **must** not exceed `maxMeshIndexCount` and `maxMeshVertexCount` as the maximum values to set passthrough meshes when calling xrCreatePassthroughLayerANDROID and xrSetPassthroughLayerMeshANDROID, otherwise the runtime **must** return `XR_ERROR_MESH_DATA_LIMIT_EXCEEDED_ANDROID` to indicate the mesh data exceeds the supported limit.

Valid Usage (Implicit)

- The `XR_ANDROID_composition_layer_passthrough_mesh` extension **must** be enabled prior to using XrSystemPassthroughLayerPropertiesANDROID
- `type` **must** be `XR_TYPE_SYSTEM_PASSTHROUGH_LAYER_PROPERTIES_ANDROID`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

12.59.3. Passthrough Layer Composition

The XrCompositionLayerPassthroughANDROID structure is defined as:

```
// Provided by XR_ANDROID_composition_layer_passthrough_mesh
typedef struct XrCompositionLayerPassthroughANDROID {
    XrStructureType              type;
    const void*                  next;
    XrCompositionLayerFlags      layerFlags;
    XrSpace                      space;
    XrPosef                      pose;
    XrVector3f                   scale;
    float                        opacity;
    XrPassthroughLayerANDROID    layer;
} XrCompositionLayerPassthroughANDROID;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `layerFlags` is a bitmask of XrCompositionLayerFlags describing flags to apply to the layer.
- `space` is the XrSpace in which the `pose` of the layer mesh is evaluated over time.
- `pose` is an XrPosef defining the position and orientation of the layer mesh in the reference frame of the `space`.
- `scale` is an XrVector3f defining the scale of the layer mesh.
- `opacity` is a `float` defining the opacity of the passthrough texture in range [0, 1].
- `layer` is an XrPassthroughLayerANDROID previously created by xrCreatePassthroughLayerANDROID.

The XrCompositionLayerPassthroughANDROID contains the information needed to render a passthrough texture onto a triangle mesh when calling xrEndFrame. XrCompositionLayerPassthroughANDROID is an alias type for the base struct XrCompositionLayerBaseHeader used in XrFrameEndInfo.

When the application submits an XrCompositionLayerPassthroughANDROID structure with the created `layer` and the corresponding meshes provided by XrPassthroughLayerMeshANDROID in the desired layer order, the runtime composites a passthrough layer into the final frame output if passthrough data is ready.

The runtime **may** composite black pixels in place of the passthrough image if the data is not yet available for a small period of time.

Valid Usage (Implicit)

- The XR_ANDROID_composition_layer_passthrough_mesh extension **must** be enabled prior to using XrCompositionLayerPassthroughANDROID
- type **must** be XR_TYPE_COMPOSITION_LAYER_PASSTHROUGH_ANDROID
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- layerFlags **must** be 0 or a valid combination of XrCompositionLayerFlagBits values
- space **must** be a valid XrSpace handle
- layer **must** be a valid XrPassthroughLayerANDROID handle
- Both of layer and space **must** have been created, allocated, or retrieved from the same XrSession

12.59.4. Create a passthrough layer handle

The XrPassthroughLayerANDROID handle represents a passthrough layer which defines the behavior of XrCompositionLayerPassthroughANDROID.

```
// Provided by XR_ANDROID_composition_layer_passthrough_mesh
XR_DEFINE_HANDLE(XrPassthroughLayerANDROID)
```

The xrCreatePassthroughLayerANDROID function is defined as:

```
// Provided by XR_ANDROID_composition_layer_passthrough_mesh
XrResult xrCreatePassthroughLayerANDROID(
    XrSession                                   session,
    const XrPassthroughLayerCreateInfoANDROID*  createInfo,
    XrPassthroughLayerANDROID*                  layer);
```

Parameter Descriptions

- session is an XrSession in which the passthrough layer will be created for.
- createInfo is a pointer to an XrPassthroughLayerCreateInfoANDROID structure specifying initial passthrough layer parameters.
- layer is a pointer to a handle in which the created XrPassthroughLayerANDROID is returned.

An application creates an XrPassthroughLayerANDROID handle by calling xrCreatePassthroughLayerANDROID.

The application **should** specify the maximum number of passthrough mesh indices it will submit for this layer in XrPassthroughLayerCreateInfoANDROID::`vertexCapacity` and XrPassthroughLayerCreateInfoANDROID::`indexCapacity`, and these values **should** be less than or equal to the maximum values returned by XrSystemPassthroughLayerPropertiesANDROID::`maxMeshIndexCount` and XrSystemPassthroughLayerPropertiesANDROID::`maxMeshVertexCount` when calling xrGetSystemProperties.

If XrPassthroughLayerCreateInfoANDROID::`indexCapacity` passed in `createInfo` is greater than XrSystemPassthroughLayerPropertiesANDROID::`maxMeshIndexCount`, the runtime **must** return the `XR_ERROR_MESH_DATA_LIMIT_EXCEEDED_ANDROID` error when calling xrCreatePassthroughLayerANDROID.

The XrPassthroughLayerANDROID handle **must** be eventually freed via the xrDestroyPassthroughLayerANDROID function.

Valid Usage (Implicit)

- The `XR_ANDROID_composition_layer_passthrough_mesh` extension **must** be enabled prior to calling xrCreatePassthroughLayerANDROID
- `session` **must** be a valid XrSession handle
- `createInfo` **must** be a pointer to a valid XrPassthroughLayerCreateInfoANDROID structure
- `layer` **must** be a pointer to an XrPassthroughLayerANDROID handle

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_MESH_DATA_LIMIT_EXCEEDED_ANDROID
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_SIZE_INSUFFICIENT
- XR_ERROR_VALIDATION_FAILURE

The XrPassthroughLayerCreateInfoANDROID structure is defined as:

```
// Provided by XR_ANDROID_composition_layer_passthrough_mesh
typedef struct XrPassthroughLayerCreateInfoANDROID {
    XrStructureType    type;
    const void*        next;
    uint32_t           vertexCapacity;
    uint32_t           indexCapacity;
} XrPassthroughLayerCreateInfoANDROID;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is NULL or a pointer to the next structure in a structure chain. XrPassthroughLayerMeshANDROID **can** be provided in the next chain to specify an initial mesh for the passthrough layer when calling xrCreatePassthroughLayerANDROID.
- `vertexCapacity` is an `uint32_t` representing the **maximum** capacity of the vertex buffer for this layer's mesh, or `0` if unspecified. If specified, the XrPassthroughLayerMeshANDROID ::`vertexCount` of any mesh set for this layer **must** be less than or equal to the `vertexCapacity`.
- `indexCapacity` is an `uint32_t` representing the **maximum** capacity of the index buffer for this layer's mesh, or `0` if unspecified. If specified, the XrPassthroughLayerMeshANDROID ::`indexCount` of any mesh set for this layer **must** be less than or equal to the `indexCapacity`.

Valid Usage (Implicit)

- The `XR_ANDROID_composition_layer_passthrough_mesh` extension **must** be enabled prior to using XrPassthroughLayerCreateInfoANDROID
- `type` **must** be `XR_TYPE_PASSTHROUGH_LAYER_CREATE_INFO_ANDROID`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain. See also: XrPassthroughLayerMeshANDROID

The xrDestroyPassthroughLayerANDROID function is defined as:

```
// Provided by XR_ANDROID_composition_layer_passthrough_mesh
XrResult xrDestroyPassthroughLayerANDROID(
    XrPassthroughLayerANDROID                   layer);
```

Parameter Descriptions

- `layer` is the XrPassthroughLayerANDROID to be destroyed.

An application releases the passthrough layer and the underlying resources by calling the xrDestroyPassthroughLayerANDROID function.

Valid Usage (Implicit)

- The XR_ANDROID_composition_layer_passthrough_mesh extension **must** be enabled prior to calling xrDestroyPassthroughLayerANDROID
- layer **must** be a valid XrPassthroughLayerANDROID handle

Thread Safety

- Access to layer, and any child handles, **must** be externally synchronized

Return Codes

Success

- XR_SUCCESS

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_RUNTIME_FAILURE

12.59.5. Set passthrough layer mesh

The xrSetPassthroughLayerMeshANDROID function is defined as:

```
// Provided by XR_ANDROID_composition_layer_passthrough_mesh
XrResult xrSetPassthroughLayerMeshANDROID(
    XrPassthroughLayerANDROID                   layer,
    const XrPassthroughLayerMeshANDROID*        mesh);
```

Parameter Descriptions

- layer is an XrPassthroughLayerANDROID handle in which to update with the given mesh.
- mesh is a pointer to an XrPassthroughLayerMeshANDROID structure specifying the information of the mesh.

An application sets the mesh for a passthrough layer by calling the xrSetPassthroughLayerMeshANDROID function.

The application **should** specify the number of passthrough mesh indices in the XrPassthroughLayerMeshANDROID::`vertexCount` and XrPassthroughLayerMeshANDROID::`indexCount` less than or equal to the maximum values returned by XrSystemPassthroughLayerPropertiesANDROID::`maxMeshIndexCount` and XrSystemPassthroughLayerPropertiesANDROID::`maxMeshVertexCount` when calling xrGetSystemProperties. The runtime **must** return `XR_ERROR_MESH_DATA_LIMIT_EXCEEDED_ANDROID` error on xrSetPassthroughLayerMeshANDROID if the count of mesh indices or vertices given by `mesh` is greater than the maximum values.

If XrPassthroughLayerCreateInfoANDROID::`vertexCapacity` or XrPassthroughLayerCreateInfoANDROID::`indexCapacity` are specified when creating the `layer` via xrCreatePassthroughLayerANDROID, the runtime **must** return `XR_ERROR_SIZE_INSUFFICIENT` error on xrSetPassthroughLayerMeshANDROID if the count of mesh vertices or indices defined by `mesh` is greater than the respective application specified capacity.

Valid Usage (Implicit)

- The `XR_ANDROID_composition_layer_passthrough_mesh` extension **must** be enabled prior to calling xrSetPassthroughLayerMeshANDROID
- `layer` **must** be a valid XrPassthroughLayerANDROID handle
- `mesh` **must** be a pointer to a valid XrPassthroughLayerMeshANDROID structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_MESH_DATA_LIMIT_EXCEEDED_ANDROID
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_SIZE_INSUFFICIENT
- XR_ERROR_VALIDATION_FAILURE

The XrPassthroughLayerMeshANDROID structure is defined as:

```
// Provided by XR_ANDROID_composition_layer_passthrough_mesh
typedef struct XrPassthroughLayerMeshANDROID {
    XrStructureType          type;
    const void*              next;
    XrWindingOrderANDROID    windingOrder;
    uint32_t                 vertexCount;
    const XrVector3f*        vertices;
    uint32_t                 indexCount;
    const uint16_t*          indices;
} XrPassthroughLayerMeshANDROID;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `windingOrder` is the XrWindingOrderANDROID of the mesh triangles. The runtime **must** use it for backface culling when rendering the mesh.
- `vertexCount` is an `uint32_t` representing the number of vertices in the mesh. When XrPassthroughLayerCreateInfoANDROID::`vertexCapacity` is specified, the `vertexCount` **must** be less than or equal to the XrPassthroughLayerCreateInfoANDROID::`vertexCapacity`.
- `vertices` is a pointer to an array of `vertexCount` XrVector3f which contains the vertex positions of the triangle mesh.
- `indexCount` is an `uint32_t` representing the number of indices in the triangle mesh. The last `indexCount % 3` indices, if any, will not be drawn. When XrPassthroughLayerCreateInfoANDROID::`indexCapacity` is specified, the `indexCount` **must** be less than or equal to the XrPassthroughLayerCreateInfoANDROID::`indexCapacity`.
- `indices` is a pointer to an array of `indexCount` `uint16_t` which contains the indices of the triangle mesh.

Valid Usage (Implicit)

- The `XR_ANDROID_composition_layer_passthrough_mesh` extension **must** be enabled prior to using XrPassthroughLayerMeshANDROID
- `type` **must** be `XR_TYPE_PASSTHROUGH_LAYER_MESH_ANDROID`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `windingOrder` **must** be a valid XrWindingOrderANDROID value
- If `vertexCount` is not `0`, `vertices` **must** be a pointer to an array of `vertexCount` XrVector3f structures
- If `indexCount` is not `0`, `indices` **must** be a pointer to an array of `indexCount` `uint16_t` values

The XrWindingOrderANDROID enumeration identifies the winding order of a mesh's triangles, used by the runtime for backface culling when rendering the mesh of the passthrough layer.

```
// Provided by XR_ANDROID_composition_layer_passthrough_mesh
typedef enum XrWindingOrderANDROID {
    XR_WINDING_ORDER_UNKNOWN_ANDROID = 0,
    XR_WINDING_ORDER_CW_ANDROID = 1,
    XR_WINDING_ORDER_CCW_ANDROID = 2,
    XR_WINDING_ORDER_MAX_ENUM_ANDROID = 0x7FFFFFFF
} XrWindingOrderANDROID;
```

Enumerant Descriptions

- XR_WINDING_ORDER_UNKNOWN_ANDROID — Winding order of the mesh's triangles is not known.
- XR_WINDING_ORDER_CW_ANDROID — Winding order of the mesh's triangles is clockwise.
- XR_WINDING_ORDER_CCW_ANDROID — Winding order of the mesh's triangles is counter-clockwise.

12.59.6. Example code for passthrough layer composition

The following example code demonstrates how to create a passthrough layer and use it in compositing.

```
XrInstance instance; // previously initialized
XrSystemId systemId; // previously initialized
XrSession session; // previously initialized
XrSpace space; // previously initialized

// The function pointers are previously initialized using xrGetInstanceProcAddr.
PFN_xrCreatePassthroughLayerANDROID xrCreatePassthroughLayerANDROID; // previously
initialized
PFN_xrDestroyPassthroughLayerANDROID xrDestroyPassthroughLayerANDROID; // previously
initialized
PFN_xrSetPassthroughLayerMeshANDROID xrSetPassthroughLayerMeshANDROID; // previously
initialized

// Inspect passthrough mesh system properties
XrSystemPassthroughLayerPropertiesANDROID passthroughLayerSystemProperties {
  .type = XR_TYPE_SYSTEM_PASSTHROUGH_LAYER_PROPERTIES_ANDROID,
};
XrSystemProperties systemProperties {
  .type = XR_TYPE_SYSTEM_PROPERTIES,
  .next = &passthroughLayerSystemProperties,
};
CHK_XR(xrGetSystemProperties(instance, systemId, &systemProperties));
if (!passthroughLayerSystemProperties.supportsPassthroughLayer) {
    // the system does not support composite layer passthrough mesh.
    return;
```

```
}

// The initial mesh for the layer.
uint16_t initialIndices[] = {
  0, 1, 2,
  0, 2, 3
};
XrVector3f initialVertices[] = {
  { 0, 0, 0 }, { 0, 1, 0 }, { 1, 1, 0 }, { 1, 0, 0 }
};
XrPassthroughLayerMeshANDROID mesh {
  .type = XR_TYPE_PASSTHROUGH_LAYER_MESH_ANDROID,
  .windingOrder = XR_WINDING_ORDER_CW_ANDROID,
  .vertexCount = 4,
  .vertices = initialVertices,
  .indexCount = 6,
  .indices = initialIndices
};

// Create the layer. Layers are expected to persist across frames.
XrPassthroughLayerCreateInfoANDROID create_info {
  .type = XR_TYPE_PASSTHROUGH_LAYER_CREATE_INFO_ANDROID,
  .next = &mesh,
  .vertexCapacity = 0,
  .indexCapacity = 0,
};
XrPassthroughLayerANDROID layer;
CHK_XR(xrCreatePassthroughLayerANDROID(session, &create_info, &layer));

// Create a composition layer. Composition layers are submitted per frame.
XrCompositionLayerPassthroughANDROID passthrough_layer {
  .type = XR_TYPE_COMPOSITION_LAYER_PASSTHROUGH_ANDROID,
  .next = nullptr,
  .layerFlags = 0,
  .space = space,
  .pose = {
    .orientation = { 0.0f, 0.0f, 0.0f, 1.0f },
    .position = { 0.0f, 0.0f, 0.0f }
  },
  .scale = { 1.0f, 1.0f, 1.0f },
  .opacity = 1.0f,
  .layer = layer
};

while (1) {
    // ...
    // For every frame in frame loop
    // ...
```

```
    // Submit composition layer in xrEndFrame.
    std::vector<XrCompositionLayerBaseHeader*> layers = {
        // ...,
        (XrCompositionLayerBaseHeader*)&passthrough_layer
        // ...
    };
    XrFrameEndInfo end_frame_info {
      .type = XR_TYPE_FRAME_END_INFO,
      .next = nullptr,
      .layerCount = static_cast<uint32_t>(layers.size()),
      .layers = layers.data(),
    };
    CHK_XR(xrEndFrame(session, &end_frame_info));

    // Update the layer. Results will be seen the next time a passthrough composition
    // layer is submitted.
    mesh.indexCount = 9;
    const uint16_t new_index_buffer[] = {
        0, 1, 2,
        0, 2, 3,
        0, 1, 2
    };
    mesh.indices = &new_index_buffer[0];
    CHK_XR(xrSetPassthroughLayerMeshANDROID(layer, &mesh));

    // ...
    // Finish frame loop
    // ...
}

// Clean up.
CHK_XR(xrDestroyPassthroughLayerANDROID(layer));
```

12.59.7. New Object Types

- XrPassthroughLayerANDROID

12.59.8. New Commands

- xrCreatePassthroughLayerANDROID
- xrDestroyPassthroughLayerANDROID
- xrSetPassthroughLayerMeshANDROID

12.59.9. New Structures

- XrCompositionLayerPassthroughANDROID
- XrPassthroughLayerCreateInfoANDROID
- Extending XrPassthroughLayerCreateInfoANDROID:
  - XrPassthroughLayerMeshANDROID
- Extending XrSystemProperties:
  - XrSystemPassthroughLayerPropertiesANDROID

12.59.10. New Enums

- XrWindingOrderANDROID

12.59.11. New Enum Constants

- `XR_ANDROID_COMPOSITION_LAYER_PASSTHROUGH_MESH_EXTENSION_NAME`
- `XR_ANDROID_composition_layer_passthrough_mesh_SPEC_VERSION`
- Extending XrObjectType:
  - `XR_OBJECT_TYPE_PASSTHROUGH_LAYER_ANDROID`
- Extending XrResult:
  - `XR_ERROR_MESH_DATA_LIMIT_EXCEEDED_ANDROID`
- Extending XrStructureType:
  - `XR_TYPE_COMPOSITION_LAYER_PASSTHROUGH_ANDROID`
  - `XR_TYPE_PASSTHROUGH_LAYER_CREATE_INFO_ANDROID`
  - `XR_TYPE_PASSTHROUGH_LAYER_MESH_ANDROID`
  - `XR_TYPE_SYSTEM_PASSTHROUGH_LAYER_PROPERTIES_ANDROID`

12.59.12. Issues

12.59.13. Version History

- Revision 1, 2024-09-11 (Levana Chen)
  - Initial extension description

12.60. XR_ANDROID_eye_tracking

Name String

`XR_ANDROID_eye_tracking`

Extension Type

Instance extension

Registered Extension Number

457

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Last Modified Date

2025-01-17

IP Status

No known IP claims.

Contributors

Spencer Quin, Google
Jared Finder, Google
Levana Chen, Google
Kenny Vercaemer, Google
Prasanthi Gurumurthy, Google
Nihav Jain, Google

12.60.1. Overview

This extension enables applications to obtain position and orientation of the user's eyes, as well as eye tracking status.

Eye tracking data is provided in two modes: coarse and fine. Coarse tracking provides a coarse estimate of the user's eyes, while fine tracking provides a more accurate estimate. Coarse tracking is meant for applications that want to provide a basic avatar-like representation, while fine tracking is meant for more precise applications.

For interaction, `XR_EXT_eye_gaze_interaction` **should** be used.



Permissions

Android applications **must** have the android.permission.EYE_TRACKING_COARSE or the android.permission.EYE_TRACKING_FINE permission listed in their manifest. These permissions are considered dangerous permissions. The application **must** request the permission at runtime to use these functions:

- xrCreateEyeTrackerANDROID (at least one of the permissions)
- xrGetCoarseTrackingEyesInfoANDROID (android.permission.EYE_TRACKING_COARSE)
- xrGetFineTrackingEyesInfoANDROID (android.permission.EYE_TRACKING_FINE)

(protection level: dangerous)

12.60.2. Inspect system capability

The XrSystemEyeTrackingPropertiesANDROID structure is defined as:

```
// Provided by XR_ANDROID_eye_tracking
typedef struct XrSystemEyeTrackingPropertiesANDROID {
    XrStructureType    type;
    void*              next;
    XrBool32           supportsEyeTracking;
} XrSystemEyeTrackingPropertiesANDROID;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `supportsEyeTracking` is an `XrBool32` indicating if current system supports eye tracking.

An application **can** inspect whether the system is capable of eye tracking by extending the XrSystemProperties with XrSystemEyeTrackingPropertiesANDROID structure when calling xrGetSystemProperties.

If and only if a runtime returns `XR_FALSE` for `supportsEyeTracking`, the runtime **must** return `XR_ERROR_FEATURE_UNSUPPORTED` from xrCreateEyeTrackerANDROID.

Valid Usage (Implicit)

- The XR_ANDROID_eye_tracking extension **must** be enabled prior to using XrSystemEyeTrackingPropertiesANDROID
- type **must** be XR_TYPE_SYSTEM_EYE_TRACKING_PROPERTIES_ANDROID
- next **must** be NULL or a valid pointer to the next structure in a structure chain

12.60.3. Creating an eye tracker handle

```
// Provided by XR_ANDROID_eye_tracking
XR_DEFINE_HANDLE(XrEyeTrackerANDROID)
```

The XrEyeTrackerANDROID handle represents an eye tracker for tracking eyes and accurately maps where the user is looking.

Eye tracking data is sensitive personal information and is closely linked to personal privacy and integrity. It is strongly recommended that applications that store or transfer eye tracking data always ask the user for active and specific acceptance to do so.

The application uses this handle to access eye tracking data using other functions in this extension.

Eye tracking provides eye pose and status representation in the scene.

The xrCreateEyeTrackerANDROID function is defined as:

```
// Provided by XR_ANDROID_eye_tracking
XrResult xrCreateEyeTrackerANDROID(
    XrSession                                   session,
    const XrEyeTrackerCreateInfoANDROID*        createInfo,
    XrEyeTrackerANDROID*                        eyeTracker);
```

Parameter Descriptions

- session is an XrSession handle in which the eye tracking will be active.
- createInfo is the XrEyeTrackerCreateInfoANDROID used to specify the eye tracking.
- eyeTracker is the returned XrEyeTrackerANDROID handle.

An application creates an XrEyeTrackerANDROID handle using the xrCreateEyeTrackerANDROID function.

If the system does not support eye tracking, the runtime **must** return XR_ERROR_FEATURE_UNSUPPORTED from xrCreateEyeTrackerANDROID.

Valid Usage (Implicit)

- The XR_ANDROID_eye_tracking extension **must** be enabled prior to calling xrCreateEyeTrackerANDROID
- session **must** be a valid XrSession handle
- createInfo **must** be a pointer to a valid XrEyeTrackerCreateInfoANDROID structure
- eyeTracker **must** be a pointer to an XrEyeTrackerANDROID handle

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FEATURE_UNSUPPORTED
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The XrEyeTrackerCreateInfoANDROID structure is defined as:

```
// Provided by XR_ANDROID_eye_tracking
typedef struct XrEyeTrackerCreateInfoANDROID {
    XrStructureType    type;
    const void*        next;
} XrEyeTrackerCreateInfoANDROID;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.

The XrEyeTrackerCreateInfoANDROID structure describes the information to create an XrEyeTrackerANDROID handle.

Valid Usage (Implicit)

- The `XR_ANDROID_eye_tracking` extension **must** be enabled prior to using XrEyeTrackerCreateInfoANDROID
- `type` **must** be `XR_TYPE_EYE_TRACKER_CREATE_INFO_ANDROID`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

The xrDestroyEyeTrackerANDROID function is defined as:

```
// Provided by XR_ANDROID_eye_tracking
XrResult xrDestroyEyeTrackerANDROID(
    XrEyeTrackerANDROID                         eyeTracker);
```

Parameter Descriptions

- `eyeTracker` is an XrEyeTrackerANDROID previously created by xrCreateEyeTrackerANDROID.

xrDestroyEyeTrackerANDROID function releases the `eyeTracker` and the underlying resources when finished with eye tracking experiences.

Valid Usage (Implicit)

- The `XR_ANDROID_eye_tracking` extension **must** be enabled prior to calling xrDestroyEyeTrackerANDROID
- `eyeTracker` **must** be a valid XrEyeTrackerANDROID handle

Thread Safety

- Access to `eyeTracker`, and any child handles, **must** be externally synchronized

Return Codes

Success

- `XR_SUCCESS`

Failure

- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_RUNTIME_FAILURE`

12.60.4. Getting eyes information

The xrGetCoarseTrackingEyesInfoANDROID function is defined as:

```
// Provided by XR_ANDROID_eye_tracking
XrResult xrGetCoarseTrackingEyesInfoANDROID(
    XrEyeTrackerANDROID                         eyeTracker,
    const XrEyesGetInfoANDROID*                 getInfo,
    XrEyesANDROID*                              eyesOutput);
```

Parameter Descriptions

- `eyeTracker` is an XrEyeTrackerANDROID previously created by xrCreateEyeTrackerANDROID.
- `getInfo` is a pointer to XrEyesGetInfoANDROID used to provide info for querying eye poses and states.
- `eyesOutput` is a pointer to XrEyesANDROID that contains the returned eyes information including poses and states.

xrGetCoarseTrackingEyesInfoANDROID function gets the information for eye states and poses in a way that preserves user privacy.

The runtime **must** return `XR_ERROR_PERMISSION_INSUFFICIENT` if the application does not have the android.permission.EYE_TRACKING_COARSE permission.

The eye poses are expressed in XrEyesGetInfoANDROID::`baseSpace` at XrEyesGetInfoANDROID::`time`.

At any point of time both the position and direction of the eye pose is tracked or untracked. This means that the runtime **must** set both `XR_SPACE_LOCATION_POSITION_TRACKED_BIT` and `XR_SPACE_LOCATION_ORIENTATION_TRACKED_BIT` or clear both `XR_SPACE_LOCATION_POSITION_TRACKED_BIT` and `XR_SPACE_LOCATION_ORIENTATION_TRACKED_BIT` on the supplied XrEyesANDROID::`eyes`, and set XrEyesANDROID::`mode` to indicate the tracking states.

Valid Usage (Implicit)

- The `XR_ANDROID_eye_tracking` extension **must** be enabled prior to calling xrGetCoarseTrackingEyesInfoANDROID
- `eyeTracker` **must** be a valid XrEyeTrackerANDROID handle
- `getInfo` **must** be a pointer to a valid XrEyesGetInfoANDROID structure
- `eyesOutput` **must** be a pointer to an XrEyesANDROID structure

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_LIMIT_REACHED`
- `XR_ERROR_OUT_OF_MEMORY`
- `XR_ERROR_PERMISSION_INSUFFICIENT`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_TIME_INVALID`
- `XR_ERROR_VALIDATION_FAILURE`

The xrGetFineTrackingEyesInfoANDROID function is defined as:

```
// Provided by XR_ANDROID_eye_tracking
XrResult xrGetFineTrackingEyesInfoANDROID(
    XrEyeTrackerANDROID                         eyeTracker,
    const XrEyesGetInfoANDROID*                 getInfo,
    XrEyesANDROID*                              eyesOutput);
```

Parameter Descriptions

- `eyeTracker` is an XrEyeTrackerANDROID previously created by xrCreateEyeTrackerANDROID.
- `getInfo` is a pointer to XrEyesGetInfoANDROID used to specify what output is required.
- `eyesOutput` is a pointer to XrEyesANDROID that contains the returned eyes information including poses and states.

xrGetFineTrackingEyesInfoANDROID function gets the information for eye states and poses with higher precision than xrGetCoarseTrackingEyesInfoANDROID.

The runtime **must** return `XR_ERROR_PERMISSION_INSUFFICIENT` if the application does not have the android.permission.EYE_TRACKING_FINE permission.

The eyes information is resolved and relative to the base space at the time of the call to xrGetFineTrackingEyesInfoANDROID using XrEyesGetInfoANDROID::`time`, XrEyesGetInfoANDROID::`baseSpace`. The runtime **must** resolve the location defined by XrEyesGetInfoANDROID::`baseSpace` at the time of the call.

At any point of time both the position and direction of the eye pose is tracked or untracked. This means that the runtime **must** set both `XR_SPACE_LOCATION_POSITION_TRACKED_BIT` and `XR_SPACE_LOCATION_ORIENTATION_TRACKED_BIT` or clear both `XR_SPACE_LOCATION_POSITION_TRACKED_BIT` and `XR_SPACE_LOCATION_ORIENTATION_TRACKED_BIT` on the supplied XrEyesANDROID::`eyes`, and set XrEyesANDROID::`mode` to indicate the tracking states.

Valid Usage (Implicit)

- The `XR_ANDROID_eye_tracking` extension **must** be enabled prior to calling xrGetFineTrackingEyesInfoANDROID
- `eyeTracker` **must** be a valid XrEyeTrackerANDROID handle
- `getInfo` **must** be a pointer to a valid XrEyesGetInfoANDROID structure
- `eyesOutput` **must** be a pointer to an XrEyesANDROID structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_PERMISSION_INSUFFICIENT
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_TIME_INVALID
- XR_ERROR_VALIDATION_FAILURE

XrEyesGetInfoANDROID structure contains the information required to retrieve eye poses and states.

```
// Provided by XR_ANDROID_eye_tracking
typedef struct XrEyesGetInfoANDROID {
    XrStructureType    type;
    const void*        next;
    XrTime             time;
    XrSpace            baseSpace;
} XrEyesGetInfoANDROID;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- time is the XrTime at which to evaluate the coordinates relative to the baseSpace.
- baseSpace is the XrSpace in which to return the eye poses.

Valid Usage (Implicit)

- The XR_ANDROID_eye_tracking extension **must** be enabled prior to using XrEyesGetInfoANDROID
- type **must** be XR_TYPE_EYES_GET_INFO_ANDROID
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- baseSpace **must** be a valid XrSpace handle

The XrEyesANDROID structure contains information about the tracked eyes.

```
// Provided by XR_ANDROID_eye_tracking
typedef struct XrEyesANDROID {
    XrStructureType             type;
    void*                       next;
    XrEyeANDROID                eyes[XR_EYE_MAX_ANDROID];
    XrEyeTrackingModeANDROID    mode;
} XrEyesANDROID;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- eyes is an array of XrEyeANDROID, of size XR_EYE_MAX_ANDROID, for the left and right eyes as indexed by XrEyeIndexANDROID.
- mode is the XrEyeTrackingModeANDROID indicating which of the eyes are currently tracking.

Valid Usage (Implicit)

- The XR_ANDROID_eye_tracking extension **must** be enabled prior to using XrEyesANDROID
- type **must** be XR_TYPE_EYES_ANDROID
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- Any given element of eyes **must** be a valid XrEyeANDROID structure
- mode **must** be a valid XrEyeTrackingModeANDROID value

The XrEyeANDROID structure describes the state, position and orientation of an eye.

```
// Provided by XR_ANDROID_eye_tracking
typedef struct XrEyeANDROID {
    XrEyeStateANDROID    eyeState;
    XrPosef              eyePose;
} XrEyeANDROID;
```

Member Descriptions

- `eyeState` is the XrEyeStateANDROID of an eye.
- `eyePose` is an XrPosef defining the position and orientation of the pupil of an eye within the reference frame of the corresponding XrEyesGetInfoANDROID::`baseSpace`. An identity orientation here represents a coordinate axes with +Z into the user's eyes, +X to the right and +Y up.

Valid Usage (Implicit)

- The `XR_ANDROID_eye_tracking` extension **must** be enabled prior to using XrEyeANDROID
- `eyeState` **must** be a valid XrEyeStateANDROID value

The XrEyeStateANDROID enumeration identifies the different state of tracked eyes.

```
// Provided by XR_ANDROID_eye_tracking
typedef enum XrEyeStateANDROID {
    XR_EYE_STATE_INVALID_ANDROID = 0,
    XR_EYE_STATE_GAZING_ANDROID = 1,
    XR_EYE_STATE_SHUT_ANDROID = 2,
    XR_EYE_STATE_MAX_ENUM_ANDROID = 0x7FFFFFFF
} XrEyeStateANDROID;
```

The enums have the following meanings:

Enum	Description
`XR_EYE_STATE_INVALID_ANDROID`	Indicates that the eye is in an error state or not present.
`XR_EYE_STATE_GAZING_ANDROID`	Indicates that the eye is currently gazing.
`XR_EYE_STATE_SHUT_ANDROID`	Indicates that the eye is currently shut due to a wink or a blink.

The XrEyeIndexANDROID enumeration identifies the index of the left or right eyes.

```
// Provided by XR_ANDROID_eye_tracking
typedef enum XrEyeIndexANDROID {
    XR_EYE_INDEX_LEFT_ANDROID = 0,
    XR_EYE_INDEX_RIGHT_ANDROID = 1,
    XR_EYE_INDEX_MAX_ENUM_ANDROID = 0x7FFFFFFF
} XrEyeIndexANDROID;
```

The enums have the following meanings:

Enum	Description
XR_EYE_INDEX_LEFT_ANDROID	Left eye.
XR_EYE_INDEX_RIGHT_ANDROID	Right eye.

The XrEyeTrackingModeANDROID enumeration identifies the different state of tracked eyes.

```
// Provided by XR_ANDROID_eye_tracking
typedef enum XrEyeTrackingModeANDROID {
    XR_EYE_TRACKING_MODE_NOT_TRACKING_ANDROID = 0,
    XR_EYE_TRACKING_MODE_RIGHT_ANDROID = 1,
    XR_EYE_TRACKING_MODE_LEFT_ANDROID = 2,
    XR_EYE_TRACKING_MODE_BOTH_ANDROID = 3,
    XR_EYE_TRACKING_MODE_MAX_ENUM_ANDROID = 0x7FFFFFFF
} XrEyeTrackingModeANDROID;
```

The enums have the following meanings:

Enum	Description
XR_EYE_TRACKING_MODE_NOT_TRACKING_ANDROID	Indicates that eye tracking is not currently active.
XR_EYE_TRACKING_MODE_RIGHT_ANDROID	Indicates that only the right eye is tracking.
XR_EYE_TRACKING_MODE_LEFT_ANDROID	Indicates that only the left eye is tracking.
XR_EYE_TRACKING_MODE_BOTH_ANDROID	Indicates that both the left and right eyes are tracking.

```
#define XR_EYE_MAX_ANDROID              2
```

XR_EYE_MAX_ANDROID is the maximum number of eyes that the XrEyeANDROID data is reported for.

12.60.5. Example code for eye tracking

The following example code demonstrates how to get eyes information relative to a view space.

```
XrSession session; // previously initialized, e.g. created at app startup.
XrSpace viewSpace; // space created for XR_REFERENCE_SPACE_TYPE_VIEW.

// The function pointers are previously initialized using xrGetInstanceProcAddr.
PFN_xrCreateEyeTrackerANDROID xrCreateEyeTrackerANDROID; // previously initialized
PFN_xrDestroyEyeTrackerANDROID xrDestroyEyeTrackerANDROID; // previously initialized
PFN_xrGetCoarseTrackingEyesInfoANDROID xrGetCoarseTrackingEyesInfoANDROID; // previously
initialized
PFN_xrGetFineTrackingEyesInfoANDROID xrGetFineTrackingEyesInfoANDROID; // previously
initialized

// This will use the XrSession that is bound to the eye tracker done at time of creation.
XrEyeTrackerANDROID eyeTracker;
XrEyeTrackerCreateInfoANDROID createInfo{
    .type = XR_TYPE_EYE_TRACKER_CREATE_INFO_ANDROID,
    .next = nullptr};
CHK_XR(xrCreateEyeTrackerANDROID(session, &createInfo, &eyeTracker));

while (1) {
    // ...
    // For every frame in frame loop
    // ...

    XrFrameState frameState;  // previously returned from xrWaitFrame
    const XrTime time = frameState.predictedDisplayTime;
    XrEyesANDROID fineEyesInfo{.type = XR_TYPE_EYES_ANDROID,
                               .next = nullptr,
                               .mode = XR_EYE_TRACKING_MODE_BOTH_ANDROID};
    XrEyesANDROID coarseEyesInfo{.type = XR_TYPE_EYES_ANDROID,
                                 .next = nullptr,
                                 .mode = XR_EYE_TRACKING_MODE_BOTH_ANDROID};
    XrEyesGetInfoANDROID eyesGetInfo{.type = XR_TYPE_EYES_GET_INFO_ANDROID,
                                     .next = nullptr,
                                     .time = time,
                                     .baseSpace = viewSpace};
    CHK_XR(xrGetCoarseTrackingEyesInfoANDROID(eyeTracker, &eyesGetInfo, &
coarseEyesInfo));
    CHK_XR(xrGetFineTrackingEyesInfoANDROID(eyeTracker, &eyesGetInfo, &fineEyesInfo));

    // eyes tracking information is now available:
    // drawLeftEye(eyesInfo.eyes[XR_EYE_INDEX_LEFT_ANDROID].eyePose);
    // drawRightEye(eyesInfo.eyes[XR_EYE_INDEX_RIGHT_ANDROID].eyePose);

    // ...
    // Finish frame loop
    // ...
}
```

```
// after usage
CHK_XR(xrDestroyEyeTrackerANDROID(eyeTracker));
```

12.60.6. New Object Types

- XrEyeTrackerANDROID

12.60.7. New Commands

- xrCreateEyeTrackerANDROID
- xrDestroyEyeTrackerANDROID
- xrGetCoarseTrackingEyesInfoANDROID
- xrGetFineTrackingEyesInfoANDROID

12.60.8. New Structures

- XrEyeANDROID
- XrEyeTrackerCreateInfoANDROID
- XrEyesANDROID
- XrEyesGetInfoANDROID
- Extending XrSystemProperties:
  - XrSystemEyeTrackingPropertiesANDROID

12.60.9. New Enums

- XrEyeIndexANDROID
- XrEyeStateANDROID
- XrEyeTrackingModeANDROID

12.60.10. New Enum Constants

- XR_ANDROID_EYE_TRACKING_EXTENSION_NAME
- XR_ANDROID_eye_tracking_SPEC_VERSION
- XR_EYE_MAX_ANDROID
- Extending XrObjectType:
  - XR_OBJECT_TYPE_EYE_TRACKER_ANDROID
- Extending XrStructureType:
  - XR_TYPE_EYES_ANDROID

    - XR_TYPE_EYES_GET_INFO_ANDROID
    - XR_TYPE_EYE_TRACKER_CREATE_INFO_ANDROID
    - XR_TYPE_SYSTEM_EYE_TRACKING_PROPERTIES_ANDROID

12.60.11. Issues

12.60.12. Version History

- Revision 1, 2025-01-17 (Kenny Vercaemer)
    - Initial extension description

12.61. XR_ANDROID_face_tracking

Name String

XR_ANDROID_face_tracking

Extension Type

Instance extension

Registered Extension Number

459

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Last Modified Date

2025-06-04

IP Status

No known IP claims.

Contributors

Spencer Quin, Google
Jared Finder, Google
Levana Chen, Google

12.61.1. Overview

This extension enables applications to get weights of blend shapes and render facial expressions in XR experiences.

This extension is intended to provide the information needed to create realistic avatars and expressive representations of users in virtual space. The runtime **may** provide additional face calibration to help with face tracking experiences. The application **can** check calibration activation before getting weights of blend shapes.

12.61.2. Face Tracker

Face tracking data is sensitive personal information and is closely linked to personal privacy and integrity. It is strongly recommended that applications storing or transferring face tracking data always ask the user for active and specific acceptance to do so.



Permissions

Android applications **must** have the android.permission.FACE_TRACKING permission listed in their manifest. The android.permission.FACE_TRACKING permission is considered a dangerous permission. The application **must** request the permission at runtime to use these functions:

- xrCreateFaceTrackerANDROID

(protection level: dangerous)

The runtime **must** support a permission system to control application access to face tracking.

- The runtime **must** return `XR_ERROR_PERMISSION_INSUFFICIENT` when creating an active face tracker until the application has been granted permission for the face tracker.
- When the application access has been granted permission, the runtime **may** set XrFaceStateANDROID::`isValid` `XR_TRUE` when getting face states via xrGetFaceStateANDROID.

12.61.3. Inspect system capability

The XrSystemFaceTrackingPropertiesANDROID structure is defined as:

```
// Provided by XR_ANDROID_face_tracking
typedef struct XrSystemFaceTrackingPropertiesANDROID {
    XrStructureType    type;
    void*              next;
    XrBool32           supportsFaceTracking;
} XrSystemFaceTrackingPropertiesANDROID;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension. For more details about the structure chain, see the structure being extended (XrSystemProperties).
- `supportsFaceTracking` is an `XrBool32` indicating if current system supports face tracking.



Permissions

Android applications **must** have the android.permission.FACE_TRACKING permission listed in their manifest. The android.permission.FACE_TRACKING permission is considered a dangerous permission. The application **must** request the permission at runtime to use these functions:

- xrCreateFaceTrackerANDROID

(protection level: dangerous)

An application inspects whether the system is capable of face tracking by extending the XrSystemProperties with an XrSystemFaceTrackingPropertiesANDROID structure when calling xrGetSystemProperties.

If and only if a runtime returns `XR_FALSE` for `supportsFaceTracking`, the runtime **must** return `XR_ERROR_FEATURE_UNSUPPORTED` from xrCreateFaceTrackerANDROID.

Valid Usage (Implicit)

- The `XR_ANDROID_face_tracking` extension **must** be enabled prior to using XrSystemFaceTrackingPropertiesANDROID
- `type` **must** be `XR_TYPE_SYSTEM_FACE_TRACKING_PROPERTIES_ANDROID`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

12.61.4. Create a face tracker handle

```
// Provided by XR_ANDROID_face_tracking
XR_DEFINE_HANDLE(XrFaceTrackerANDROID)
```

The XrFaceTrackerANDROID handle represents a face tracker for face tracking.

The application uses this handle to access face tracking data using other functions in this extension.

The xrCreateFaceTrackerANDROID function is defined as:

```
// Provided by XR_ANDROID_face_tracking
XrResult xrCreateFaceTrackerANDROID(
    XrSession                                   session,
    const XrFaceTrackerCreateInfoANDROID*       createInfo,
    XrFaceTrackerANDROID*                       faceTracker);
```

Parameter Descriptions

- `session` is an XrSession handle in which the face tracker will be active.
- `createInfo` is the XrFaceTrackerCreateInfoANDROID used to specify the face tracker.
- `faceTracker` is the returned XrFaceTrackerANDROID handle.



Permissions

Android applications **must** have the android.permission.FACE_TRACKING permission listed in their manifest. The android.permission.FACE_TRACKING permission is considered a dangerous permission. The application **must** request the permission at runtime to use these functions:

- xrCreateFaceTrackerANDROID

(protection level: dangerous)

An application creates an XrFaceTrackerANDROID handle by calling the xrCreateFaceTrackerANDROID function.

If the system does not support face tracking, the runtime **must** return `XR_ERROR_FEATURE_UNSUPPORTED` from xrCreateFaceTrackerANDROID.

Valid Usage (Implicit)

- The `XR_ANDROID_face_tracking` extension **must** be enabled prior to calling xrCreateFaceTrackerANDROID
- `session` **must** be a valid XrSession handle
- `createInfo` **must** be a pointer to a valid XrFaceTrackerCreateInfoANDROID structure
- `faceTracker` **must** be a pointer to an XrFaceTrackerANDROID handle

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FEATURE_UNSUPPORTED
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The XrFaceTrackerCreateInfoANDROID structure is described as follows:

```
// Provided by XR_ANDROID_face_tracking
typedef struct XrFaceTrackerCreateInfoANDROID {
    XrStructureType    type;
    const void*        next;
} XrFaceTrackerCreateInfoANDROID;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.

The XrFaceTrackerCreateInfoANDROID structure describes the information to create an XrFaceTrackerANDROID handle.

Valid Usage (Implicit)

- The XR_ANDROID_face_tracking extension **must** be enabled prior to using XrFaceTrackerCreateInfoANDROID
- `type` **must** be `XR_TYPE_FACE_TRACKER_CREATE_INFO_ANDROID`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

The xrDestroyFaceTrackerANDROID function is defined as:

```
// Provided by XR_ANDROID_face_tracking
XrResult xrDestroyFaceTrackerANDROID(
    XrFaceTrackerANDROID                        faceTracker);
```

Parameter Descriptions

- `faceTracker` is an XrFaceTrackerANDROID previously created by xrCreateFaceTrackerANDROID.

When the application's face tracking experience is over it calls the xrDestroyFaceTrackerANDROID function to release the `faceTracker` and underlying resources.

Valid Usage (Implicit)

- The XR_ANDROID_face_tracking extension **must** be enabled prior to calling xrDestroyFaceTrackerANDROID
- `faceTracker` **must** be a valid XrFaceTrackerANDROID handle

Thread Safety

- Access to `faceTracker`, and any child handles, **must** be externally synchronized

Return Codes

Success

- XR_SUCCESS

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_RUNTIME_FAILURE

12.61.5. Check face calibration

The xrGetFaceCalibrationStateANDROID function is defined as:

```
// Provided by XR_ANDROID_face_tracking
XrResult xrGetFaceCalibrationStateANDROID(
    XrFaceTrackerANDROID                        faceTracker,
    XrBool32*                                   faceIsCalibratedOutput);
```

Parameter Descriptions

- faceTracker is an XrFaceTrackerANDROID previously created by xrCreateFaceTrackerANDROID.
- faceIsCalibratedOutput indicates if the face tracker has been calibrated or not.

An application checks the face calibration state by calling the xrGetFaceCalibrationStateANDROID function.

When the tracking service is still under initialization, the runtime **may** return XR_ERROR_SERVICE_NOT_READY_ANDROID from xrGetFaceCalibrationStateANDROID to indicate that the application **can** retry later.

If the system does not support face calibration, the runtime **must** return XR_ERROR_FEATURE_UNSUPPORTED from xrGetFaceCalibrationStateANDROID. Otherwise, the runtime **may** set faceIsCalibratedOutput to XR_TRUE to reflect the face calibration state.

Valid Usage (Implicit)

- The XR_ANDROID_face_tracking extension **must** be enabled prior to calling xrGetFaceCalibrationStateANDROID
- faceTracker **must** be a valid XrFaceTrackerANDROID handle
- faceIsCalibratedOutput **must** be a pointer to an XrBool32 value

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

12.61.6. Get facial expressions

The xrGetFaceStateANDROID function returns blend shapes of facial expressions at a given time.

```
// Provided by XR_ANDROID_face_tracking
XrResult xrGetFaceStateANDROID(
    XrFaceTrackerANDROID                        faceTracker,
    const XrFaceStateGetInfoANDROID*            getInfo,
    XrFaceStateANDROID*                         faceStateOutput);
```

Parameter Descriptions

- `faceTracker` is an XrFaceTrackerANDROID previously created by xrCreateFaceTrackerANDROID.
- `getInfo` is a pointer to XrFaceStateGetInfoANDROID describing information to obtain face expressions.
- `faceStateOutput` is a pointer to XrFaceStateANDROID receiving the returned face tracking state and facial expressions.

Valid Usage (Implicit)

- The `XR_ANDROID_face_tracking` extension **must** be enabled prior to calling xrGetFaceStateANDROID
- `faceTracker` **must** be a valid XrFaceTrackerANDROID handle
- `getInfo` **must** be a pointer to a valid XrFaceStateGetInfoANDROID structure
- `faceStateOutput` **must** be a pointer to an XrFaceStateANDROID structure

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_LIMIT_REACHED`
- `XR_ERROR_OUT_OF_MEMORY`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SERVICE_NOT_READY_ANDROID`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_SIZE_INSUFFICIENT`
- `XR_ERROR_TIME_INVALID`
- `XR_ERROR_VALIDATION_FAILURE`

The XrFaceStateGetInfoANDROID structure describes the information to obtain facial expressions.

```
// Provided by XR_ANDROID_face_tracking
typedef struct XrFaceStateGetInfoANDROID {
    XrStructureType    type;
    const void*        next;
    XrTime             time;
} XrFaceStateGetInfoANDROID;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `time` is an `XrTime` at which the facial expressions are requested.

Applications **should** request a time equal to the predicted display time for the rendered frame. The runtime will employ appropriate modeling to provide expressions for this time.

Valid Usage (Implicit)

- The `XR_ANDROID_face_tracking` extension **must** be enabled prior to using XrFaceStateGetInfoANDROID
- `type` **must** be `XR_TYPE_FACE_STATE_GET_INFO_ANDROID`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

XrFaceStateANDROID structure returns the face tracking state and facial expressions.

```
// Provided by XR_ANDROID_face_tracking
typedef struct XrFaceStateANDROID {
    XrStructureType               type;
    void*                         next;
    uint32_t                      parametersCapacityInput;
    uint32_t                      parametersCountOutput;
    float*                        parameters;
    XrFaceTrackingStateANDROID    faceTrackingState;
    XrTime                        sampleTime;
    XrBool32                      isValid;
    uint32_t                      regionConfidencesCapacityInput;
    uint32_t                      regionConfidencesCountOutput;
    float*                        regionConfidences;
} XrFaceStateANDROID;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `parametersCapacityInput` is a `uint32_t` describing the capacity of the `parameters` array, or 0 to indicate a request to retrieve the required capacity.
- `parametersCountOutput` is a `uint32_t` describing the number of `parameters`, or the required capacity in the case that `parametersCapacityInput` is insufficient.
- `parameters` is a pointer to an application-allocated array of `float` that will be filled with weights of facial expression blend shapes.
- `faceTrackingState` is the XrFaceTrackingStateANDROID of validity status of face tracking.
- `sampleTime` is an `XrTime` time at which the returned expressions are tracked or extrapolated to. It equals the time at which the expression weights were requested if the extrapolation at the time was successful.
- `isValid` indicates if the data is valid even if it is not from the current frame.
- `regionConfidencesCapacityInput` is a `uint32_t` describing the capacity of the `regionConfidences` array, or 0 to indicate a request to retrieve the required capacity.
- `regionConfidencesCountOutput` is a `uint32_t` describing the number of `regionConfidences`, or the required capacity in the case that `regionConfidencesCapacityInput` is insufficient.
- `regionConfidences` is a pointer to an application-allocated array of XrFaceConfidenceRegionsANDROID that will be filled with confidences of facial expression regions.
- See the Buffer Size Parameters section for a detailed description of retrieving the required `parameters` size.

The application **can** set `parametersCapacityInput` to `XR_FACE_PARAMETER_COUNT_ANDROID` to get facial expressions which are indexed by XrFaceParameterIndicesANDROID.

The runtime **must** return `parameters` representing the weights of blend shapes of current facial expressions.

The runtime **must** update the `parameters` array ordered so that the application **can** index elements using the corresponding facial expression enum (e.g. XrFaceParameterIndicesANDROID).

Valid Usage (Implicit)

- The XR_ANDROID_face_tracking extension **must** be enabled prior to using XrFaceStateANDROID
- type **must** be XR_TYPE_FACE_STATE_ANDROID
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- If parametersCapacityInput is not 0, parameters **must** be a pointer to an array of parametersCapacityInput float values
- faceTrackingState **must** be a valid XrFaceTrackingStateANDROID value
- If regionConfidencesCapacityInput is not 0, regionConfidences **must** be a pointer to an array of regionConfidencesCapacityInput float values

The XrFaceTrackingStateANDROID enumeration identifies the different states of the face tracker.

```
// Provided by XR_ANDROID_face_tracking
typedef enum XrFaceTrackingStateANDROID {
    XR_FACE_TRACKING_STATE_PAUSED_ANDROID = 0,
    XR_FACE_TRACKING_STATE_STOPPED_ANDROID = 1,
    XR_FACE_TRACKING_STATE_TRACKING_ANDROID = 2,
    XR_FACE_TRACKING_STATE_MAX_ENUM_ANDROID = 0x7FFFFFFF
} XrFaceTrackingStateANDROID;
```

The enumerants have the following meanings:

Enum	Description
XR_FACE_TRACKING_STATE_PAUSED_ANDROID	Indicates that face tracking is paused but may be resumed in the future.
XR_FACE_TRACKING_STATE_STOPPED_ANDROID	Tracking has stopped but the client still has an active face tracker.
XR_FACE_TRACKING_STATE_TRACKING_ANDROID	The face is currently tracked and its pose is current.

The XrFaceConfidenceRegionsANDROID enumeration identifies the different confidence regions of the face.

```
// Provided by XR_ANDROID_face_tracking
typedef enum XrFaceConfidenceRegionsANDROID {
    XR_FACE_CONFIDENCE_REGIONS_LOWER_ANDROID = 0,
    XR_FACE_CONFIDENCE_REGIONS_LEFT_UPPER_ANDROID = 1,
    XR_FACE_CONFIDENCE_REGIONS_RIGHT_UPPER_ANDROID = 2,
    XR_FACE_CONFIDENCE_REGIONS_MAX_ENUM_ANDROID = 0x7FFFFFFF
} XrFaceConfidenceRegionsANDROID;
```

The enumerants have the following meanings:

Enum	Description
XR_FACE_CONFIDENCE_REGIONS_LOWER_ANDROID	Confidence corresponding to the lower region.
XR_FACE_CONFIDENCE_REGIONS_LEFT_UPPER_ANDROID	Confidence corresponding to the left upper region.
XR_FACE_CONFIDENCE_REGIONS_RIGHT_UPPER_ANDROID	Confidence corresponding to the right upper region.

12.61.7. Conventions of blend shapes

This extension defines 68 blend shapes, i.e. XR_FACE_PARAMETER_COUNT_ANDROID, for the reduced G-Nome format. Each parameter in this enum is an index into a blend shape array whose values are type of float and the runtime **must** normalize to 1 - 0.

```
// Provided by XR_ANDROID_face_tracking
typedef enum XrFaceParameterIndicesANDROID {
    XR_FACE_PARAMETER_INDICES_BROW_LOWERER_L_ANDROID = 0,
    XR_FACE_PARAMETER_INDICES_BROW_LOWERER_R_ANDROID = 1,
    XR_FACE_PARAMETER_INDICES_CHEEK_PUFF_L_ANDROID = 2,
    XR_FACE_PARAMETER_INDICES_CHEEK_PUFF_R_ANDROID = 3,
    XR_FACE_PARAMETER_INDICES_CHEEK_RAISER_L_ANDROID = 4,
    XR_FACE_PARAMETER_INDICES_CHEEK_RAISER_R_ANDROID = 5,
    XR_FACE_PARAMETER_INDICES_CHEEK_SUCK_L_ANDROID = 6,
    XR_FACE_PARAMETER_INDICES_CHEEK_SUCK_R_ANDROID = 7,
    XR_FACE_PARAMETER_INDICES_CHIN_RAISER_B_ANDROID = 8,
    XR_FACE_PARAMETER_INDICES_CHIN_RAISER_T_ANDROID = 9,
    XR_FACE_PARAMETER_INDICES_DIMPLER_L_ANDROID = 10,
    XR_FACE_PARAMETER_INDICES_DIMPLER_R_ANDROID = 11,
    XR_FACE_PARAMETER_INDICES_EYES_CLOSED_L_ANDROID = 12,
    XR_FACE_PARAMETER_INDICES_EYES_CLOSED_R_ANDROID = 13,
    XR_FACE_PARAMETER_INDICES_EYES_LOOK_DOWN_L_ANDROID = 14,
    XR_FACE_PARAMETER_INDICES_EYES_LOOK_DOWN_R_ANDROID = 15,
    XR_FACE_PARAMETER_INDICES_EYES_LOOK_LEFT_L_ANDROID = 16,
    XR_FACE_PARAMETER_INDICES_EYES_LOOK_LEFT_R_ANDROID = 17,
    XR_FACE_PARAMETER_INDICES_EYES_LOOK_RIGHT_L_ANDROID = 18,
    XR_FACE_PARAMETER_INDICES_EYES_LOOK_RIGHT_R_ANDROID = 19,
```

```
    XR_FACE_PARAMETER_INDICES_EYES_LOOK_UP_L_ANDROID = 20,
    XR_FACE_PARAMETER_INDICES_EYES_LOOK_UP_R_ANDROID = 21,
    XR_FACE_PARAMETER_INDICES_INNER_BROW_RAISER_L_ANDROID = 22,
    XR_FACE_PARAMETER_INDICES_INNER_BROW_RAISER_R_ANDROID = 23,
    XR_FACE_PARAMETER_INDICES_JAW_DROP_ANDROID = 24,
    XR_FACE_PARAMETER_INDICES_JAW_SIDEWAYS_LEFT_ANDROID = 25,
    XR_FACE_PARAMETER_INDICES_JAW_SIDEWAYS_RIGHT_ANDROID = 26,
    XR_FACE_PARAMETER_INDICES_JAW_THRUST_ANDROID = 27,
    XR_FACE_PARAMETER_INDICES_LID_TIGHTENER_L_ANDROID = 28,
    XR_FACE_PARAMETER_INDICES_LID_TIGHTENER_R_ANDROID = 29,
    XR_FACE_PARAMETER_INDICES_LIP_CORNER_DEPRESSOR_L_ANDROID = 30,
    XR_FACE_PARAMETER_INDICES_LIP_CORNER_DEPRESSOR_R_ANDROID = 31,
    XR_FACE_PARAMETER_INDICES_LIP_CORNER_PULLER_L_ANDROID = 32,
    XR_FACE_PARAMETER_INDICES_LIP_CORNER_PULLER_R_ANDROID = 33,
    XR_FACE_PARAMETER_INDICES_LIP_FUNNELER_LB_ANDROID = 34,
    XR_FACE_PARAMETER_INDICES_LIP_FUNNELER_LT_ANDROID = 35,
    XR_FACE_PARAMETER_INDICES_LIP_FUNNELER_RB_ANDROID = 36,
    XR_FACE_PARAMETER_INDICES_LIP_FUNNELER_RT_ANDROID = 37,
    XR_FACE_PARAMETER_INDICES_LIP_PRESSOR_L_ANDROID = 38,
    XR_FACE_PARAMETER_INDICES_LIP_PRESSOR_R_ANDROID = 39,
    XR_FACE_PARAMETER_INDICES_LIP_PUCKER_L_ANDROID = 40,
    XR_FACE_PARAMETER_INDICES_LIP_PUCKER_R_ANDROID = 41,
    XR_FACE_PARAMETER_INDICES_LIP_STRETCHER_L_ANDROID = 42,
    XR_FACE_PARAMETER_INDICES_LIP_STRETCHER_R_ANDROID = 43,
    XR_FACE_PARAMETER_INDICES_LIP_SUCK_LB_ANDROID = 44,
    XR_FACE_PARAMETER_INDICES_LIP_SUCK_LT_ANDROID = 45,
    XR_FACE_PARAMETER_INDICES_LIP_SUCK_RB_ANDROID = 46,
    XR_FACE_PARAMETER_INDICES_LIP_SUCK_RT_ANDROID = 47,
    XR_FACE_PARAMETER_INDICES_LIP_TIGHTENER_L_ANDROID = 48,
    XR_FACE_PARAMETER_INDICES_LIP_TIGHTENER_R_ANDROID = 49,
    XR_FACE_PARAMETER_INDICES_LIPS_TOWARD_ANDROID = 50,
    XR_FACE_PARAMETER_INDICES_LOWER_LIP_DEPRESSOR_L_ANDROID = 51,
    XR_FACE_PARAMETER_INDICES_LOWER_LIP_DEPRESSOR_R_ANDROID = 52,
    XR_FACE_PARAMETER_INDICES_MOUTH_LEFT_ANDROID = 53,
    XR_FACE_PARAMETER_INDICES_MOUTH_RIGHT_ANDROID = 54,
    XR_FACE_PARAMETER_INDICES_NOSE_WRINKLER_L_ANDROID = 55,
    XR_FACE_PARAMETER_INDICES_NOSE_WRINKLER_R_ANDROID = 56,
    XR_FACE_PARAMETER_INDICES_OUTER_BROW_RAISER_L_ANDROID = 57,
    XR_FACE_PARAMETER_INDICES_OUTER_BROW_RAISER_R_ANDROID = 58,
    XR_FACE_PARAMETER_INDICES_UPPER_LID_RAISER_L_ANDROID = 59,
    XR_FACE_PARAMETER_INDICES_UPPER_LID_RAISER_R_ANDROID = 60,
    XR_FACE_PARAMETER_INDICES_UPPER_LIP_RAISER_L_ANDROID = 61,
    XR_FACE_PARAMETER_INDICES_UPPER_LIP_RAISER_R_ANDROID = 62,
    XR_FACE_PARAMETER_INDICES_TONGUE_OUT_ANDROID = 63,
    XR_FACE_PARAMETER_INDICES_TONGUE_LEFT_ANDROID = 64,
    XR_FACE_PARAMETER_INDICES_TONGUE_RIGHT_ANDROID = 65,
    XR_FACE_PARAMETER_INDICES_TONGUE_UP_ANDROID = 66,
    XR_FACE_PARAMETER_INDICES_TONGUE_DOWN_ANDROID = 67,
```

```
    XR_FACE_PARAMETER_INDICES_MAX_ENUM_ANDROID = 0x7FFFFFFF
} XrFaceParameterIndicesANDROID;
```

Enum	Description
XR_FACE_PARAMETER_INDICES_BROW_LOWERER_L_ANDROID	The left brow lowerer blendshape parameter.
XR_FACE_PARAMETER_INDICES_BROW_LOWERER_R_ANDROID	The right brow lowerer blendshape parameter.
XR_FACE_PARAMETER_INDICES_CHEEK_PUFF_L_ANDROID	The left cheek puff blendshape parameter.
XR_FACE_PARAMETER_INDICES_CHEEK_PUFF_R_ANDROID	The right cheek puff blendshape parameter.
XR_FACE_PARAMETER_INDICES_CHEEK_RAISER_L_ANDROID	The left cheek raiser blendshape parameter.
XR_FACE_PARAMETER_INDICES_CHEEK_RAISER_R_ANDROID	The right cheek raiser blendshape parameter.
XR_FACE_PARAMETER_INDICES_CHEEK_SUCK_L_ANDROID	The left cheek suck blendshape parameter.
XR_FACE_PARAMETER_INDICES_CHEEK_SUCK_R_ANDROID	The right cheek suck blendshape parameter.
XR_FACE_PARAMETER_INDICES_CHIN_RAISER_B_ANDROID	The bottom chin raiser blendshape parameter.
XR_FACE_PARAMETER_INDICES_CHIN_RAISER_T_ANDROID	The top chin raiser blendshape parameter.
XR_FACE_PARAMETER_INDICES_DIMPLER_L_ANDROID	The left dimpler blendshape parameter.
XR_FACE_PARAMETER_INDICES_DIMPLER_R_ANDROID	The right dimpler lowerer blendshape parameter.
XR_FACE_PARAMETER_INDICES_EYES_CLOSED_L_ANDROID	The left eyes closed blendshape parameter.
XR_FACE_PARAMETER_INDICES_EYES_CLOSED_R_ANDROID	The right eyes closed blendshape parameter.
XR_FACE_PARAMETER_INDICES_EYES_LOOK_DOWN_L_ANDROID	The left eyes look down blendshape parameter.
XR_FACE_PARAMETER_INDICES_EYES_LOOK_DOWN_R_ANDROID	The right eyes look down blendshape parameter.
XR_FACE_PARAMETER_INDICES_EYES_LOOK_LEFT_L_ANDROID	The left look left blendshape parameter.
XR_FACE_PARAMETER_INDICES_EYES_LOOK_LEFT_R_ANDROID	The left look right blendshape parameter.
XR_FACE_PARAMETER_INDICES_EYES_LOOK_RIGHT_L_ANDROID	The right look left blendshape parameter.
XR_FACE_PARAMETER_INDICES_EYES_LOOK_RIGHT_R_ANDROID	The right look right blendshape parameter.
XR_FACE_PARAMETER_INDICES_EYES_LOOK_UP_L_ANDROID	The left eyes look up blendshape parameter.
XR_FACE_PARAMETER_INDICES_EYES_LOOK_UP_R_ANDROID	The right eyes look up blendshape parameter.
XR_FACE_PARAMETER_INDICES_INNER_BROW_RAISER_L_ANDROID	The left inner brow raiser blendshape parameter.
XR_FACE_PARAMETER_INDICES_INNER_BROW_RAISER_R_ANDROID	The right inner brow raiser blendshape parameter.
XR_FACE_PARAMETER_INDICES_JAW_DROP_ANDROID	The jaw drop blendshape parameter.

Enum	Description
XR_FACE_PARAMETER_INDICES_JAW_SIDEWAYS_LEFT_ANDROID	The left jaw sideways blendshape parameter.
XR_FACE_PARAMETER_INDICES_JAW_SIDEWAYS_RIGHT_ANDROID	The right jaw sideways blendshape parameter.
XR_FACE_PARAMETER_INDICES_JAW_THRUST_ANDROID	The jaw thrust blendshape parameter.
XR_FACE_PARAMETER_INDICES_LID_TIGHTENER_L_ANDROID	The left lid tightener blendshape parameter.
XR_FACE_PARAMETER_INDICES_LID_TIGHTENER_R_ANDROID	The right lid tightener blendshape parameter.
XR_FACE_PARAMETER_INDICES_LIP_CORNER_DEPRESSOR_L_ANDROID	The left corner lip depressor blendshape parameter.
XR_FACE_PARAMETER_INDICES_LIP_CORNER_DEPRESSOR_R_ANDROID	The right corner lip depressor blendshape parameter.
XR_FACE_PARAMETER_INDICES_LIP_CORNER_PULLER_L_ANDROID	The left corner lip puller blendshape parameter.
XR_FACE_PARAMETER_INDICES_LIP_CORNER_PULLER_R_ANDROID	The right corner lip puller blendshape parameter.
XR_FACE_PARAMETER_INDICES_LIP_FUNNELER_LB_ANDROID	The left bottom lip funneler blendshape parameter.
XR_FACE_PARAMETER_INDICES_LIP_FUNNELER_LT_ANDROID	The left top lip funneler blendshape parameter.
XR_FACE_PARAMETER_INDICES_LIP_FUNNELER_RB_ANDROID	The right bottom lip funneler blendshape parameter.
XR_FACE_PARAMETER_INDICES_LIP_FUNNELER_RT_ANDROID	The right top lip funneler blendshape parameter.
XR_FACE_PARAMETER_INDICES_LIP_PRESSOR_L_ANDROID	The left lip pressor blendshape parameter.
XR_FACE_PARAMETER_INDICES_LIP_PRESSOR_R_ANDROID	The right lip pressor blendshape parameter.
XR_FACE_PARAMETER_INDICES_LIP_PUCKER_L_ANDROID	The left lip pucker blendshape parameter.
XR_FACE_PARAMETER_INDICES_LIP_PUCKER_R_ANDROID	The right lip pucker blendshape parameter.
XR_FACE_PARAMETER_INDICES_LIP_STRETCHER_L_ANDROID	The left lip stretcher blendshape parameter.
XR_FACE_PARAMETER_INDICES_LIP_STRETCHER_R_ANDROID	The right lip stretcher blendshape parameter.
XR_FACE_PARAMETER_INDICES_LIP_SUCK_LB_ANDROID	The left bottom lip suck blendshape parameter.
XR_FACE_PARAMETER_INDICES_LIP_SUCK_LT_ANDROID	The left top lip suck blendshape parameter.
XR_FACE_PARAMETER_INDICES_LIP_SUCK_RB_ANDROID	The right bottom lip suck blendshape parameter.
XR_FACE_PARAMETER_INDICES_LIP_SUCK_RT_ANDROID	The right top lip suck blendshape parameter.

Enum	Description
XR_FACE_PARAMETER_INDICES_LIP_TIGHTENER_L_ANDROID	The left lip tightener blendshape parameter.
XR_FACE_PARAMETER_INDICES_LIP_TIGHTENER_R_ANDROID	The right lip tightener blendshape parameter.
XR_FACE_PARAMETER_INDICES_LIPS_TOWARD_ANDROID	The lips toward blendshape parameter.
XR_FACE_PARAMETER_INDICES_LOWER_LIP_DEPRESSOR_L_ANDROID	The left lower lip depressor blendshape parameter.
XR_FACE_PARAMETER_INDICES_LOWER_LIP_DEPRESSOR_R_ANDROID	The right lower lip depressor blendshape parameter.
XR_FACE_PARAMETER_INDICES_MOUTH_LEFT_ANDROID	The mouth move left blendshape parameter.
XR_FACE_PARAMETER_INDICES_MOUTH_RIGHT_ANDROID	The mouth move right blendshape parameter.
XR_FACE_PARAMETER_INDICES_NOSE_WRINKLER_L_ANDROID	The left nose wrinkler blendshape parameter.
XR_FACE_PARAMETER_INDICES_NOSE_WRINKLER_R_ANDROID	The right nose wrinkler blendshape parameter.
XR_FACE_PARAMETER_INDICES_OUTER_BROW_RAISER_L_ANDROID	The left outer brow raiser blendshape parameter.
XR_FACE_PARAMETER_INDICES_OUTER_BROW_RAISER_R_ANDROID	The right outer brow raiser blendshape parameter.
XR_FACE_PARAMETER_INDICES_UPPER_LID_RAISER_L_ANDROID	The left lid raiser blendshape parameter.
XR_FACE_PARAMETER_INDICES_UPPER_LID_RAISER_R_ANDROID	The right lid raiser blendshape parameter.
XR_FACE_PARAMETER_INDICES_UPPER_LIP_RAISER_L_ANDROID	The left lip raiser blendshape parameter.
XR_FACE_PARAMETER_INDICES_UPPER_LIP_RAISER_R_ANDROID	The right lip raiser blendshape parameter.
XR_FACE_PARAMETER_INDICES_TONGUE_OUT_ANDROID	The tongue out blendshape parameter.
XR_FACE_PARAMETER_INDICES_TONGUE_LEFT_ANDROID	The tongue left puller blendshape parameter.
XR_FACE_PARAMETER_INDICES_TONGUE_RIGHT_ANDROID	The right right puller blendshape parameter.
XR_FACE_PARAMETER_INDICES_TONGUE_UP_ANDROID	The right up puller blendshape parameter.
XR_FACE_PARAMETER_INDICES_TONGUE_DOWN_ANDROID	The right down puller blendshape parameter.

```
#define XR_FACE_PARAMETER_COUNT_ANDROID   68
```

XR_FACE_PARAMETER_COUNT_ANDROID is the number of blend shapes for which weights are reported in XrFaceStateANDROID::parameters, indexed by XrFaceParameterIndicesANDROID.

```
#define XR_FACE_REGION_CONFIDENCE_COUNT_ANDROID 3
```

XR_FACE_REGION_CONFIDENCE_COUNT_ANDROID is the number of regions of the face for which confidence is reported by XrFaceStateANDROID::regionConfidences, indexed by XrFaceConfidenceRegionsANDROID.

12.61.8. Example code for face tracking

The following example code demonstrates how to get all weights for facial expression blend shapes.

```
XrInstance instance; // previously initialized
XrSystemId systemId; // previously initialized
XrSession session; // previously initialized, e.g. created at app startup.

// The function pointers are previously initialized using xrGetInstanceProcAddr.
PFN_xrCreateFaceTrackerANDROID xrCreateFaceTrackerANDROID; // previously initialized
PFN_xrDestroyFaceTrackerANDROID xrDestroyFaceTrackerANDROID; // previously initialized
PFN_xrGetFaceStateANDROID xrGetFaceStateANDROID; // previously initialized
PFN_xrGetFaceCalibrationStateANDROID xrGetFaceCalibrationStateANDROID; // previously
initialized

// Inspect system capability
XrSystemFaceTrackingPropertiesANDROID faceTrackingProperties {
  .type = XR_TYPE_SYSTEM_FACE_TRACKING_PROPERTIES_ANDROID,
};
XrSystemProperties properties {
  .type = XR_TYPE_SYSTEM_PROPERTIES,
  .next = &faceTrackingProperties,
};
CHK_XR(xrGetSystemProperties(instance, systemId, &properties));
if (!faceTrackingProperties.supportsFaceTracking) {
  // face tracking is not supported.
  return;
}

XrFaceTrackerANDROID faceTracker;
XrFaceTrackerCreateInfoANDROID createInfo{
  .type = XR_TYPE_FACE_TRACKER_CREATE_INFO_ANDROID,
  .next = nullptr,
};
CHK_XR(xrCreateFaceTrackerANDROID(session, &createInfo, &faceTracker));

// If the system supports face calibration:
XrBool32 isCalibrated;
CHK_XR(xrGetFaceCalibrationStateANDROID(faceTracker, &isCalibrated));
if (!isCalibrated) {
```

```
    // Redirect the user to system calibration setting.
}

float faceExpressionParameters[XR_FACE_PARAMETER_COUNT_ANDROID];
XrFaceStateANDROID faceState {
  .type = XR_TYPE_FACE_STATE_ANDROID,
  .next = nullptr,
  .parametersCapacityInput = XR_FACE_PARAMETER_COUNT_ANDROID,
  .parameters = faceExpressionParameters,
};

while (1) {
    // ...
    // For every frame in the frame loop
    // ...
    XrFrameState frameState; // previously returned from xrWaitFrame

    XrFaceStateGetInfoANDROID faceGetInfo {
      .type = XR_TYPE_FACE_STATE_GET_INFO_ANDROID,
      .next = nullptr,
      .time = frameState.predictedDisplayTime,
    };

    CHK_XR(xrGetFaceStateANDROID(faceTracker, &faceGetInfo, &faceState));
    if (faceState.isValid) {
        for (uint32_t i = 0; i < XR_FACE_PARAMETER_COUNT_ANDROID; ++i) {
            // parameters[i] contains a weight of specific blend shape
        }
    }
}

// after usage
CHK_XR(xrDestroyFaceTrackerANDROID(faceTracker));
```

12.61.9. New Object Types

- XrFaceTrackerANDROID

12.61.10. New Commands

- xrCreateFaceTrackerANDROID
- xrDestroyFaceTrackerANDROID
- xrGetFaceCalibrationStateANDROID
- xrGetFaceStateANDROID

12.61.11. New Structures

- XrFaceStateANDROID
- XrFaceStateGetInfoANDROID
- XrFaceTrackerCreateInfoANDROID
- Extending XrSystemProperties:
  - XrSystemFaceTrackingPropertiesANDROID

12.61.12. New Enums

- XrFaceConfidenceRegionsANDROID
- XrFaceParameterIndicesANDROID
- XrFaceTrackingStateANDROID

12.61.13. New Enum Constants

- XR_ANDROID_FACE_TRACKING_EXTENSION_NAME
- XR_ANDROID_face_tracking_SPEC_VERSION
- XR_FACE_PARAMETER_COUNT_ANDROID
- XR_FACE_REGION_CONFIDENCE_COUNT_ANDROID
- Extending XrObjectType:
  - XR_OBJECT_TYPE_FACE_TRACKER_ANDROID
- Extending XrResult:
  - XR_ERROR_SERVICE_NOT_READY_ANDROID
- Extending XrStructureType:
  - XR_TYPE_FACE_STATE_ANDROID
  - XR_TYPE_FACE_STATE_GET_INFO_ANDROID
  - XR_TYPE_FACE_TRACKER_CREATE_INFO_ANDROID
  - XR_TYPE_SYSTEM_FACE_TRACKING_PROPERTIES_ANDROID

12.61.14. Issues

12.61.15. Version History

- Revision 1, 2024-09-05 (Levana Chen)
  - Initial extension description

12.62. XR_ANDROID_geospatial

Name String

`XR_ANDROID_geospatial`

Extension Type

Instance extension

Registered Extension Number

790

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

`XR_EXT_future`

Last Modified Date

2025-12-18

IP Status

No known IP claims.

Contributors

John Ullman, Google
Ben King, Google
Nihav Jain, Google
Jared Finder, Google

12.62.1. Overview

This extension provides the Geospatial tracking for Google's Geospatial API, which provides precise geo-location and orientation, and allows the application to place content with respect to the Earth. It does this using a combination of motion tracking, GPS and other sensors, and Google's Visual Positioning System (VPS). VPS compares images from a device's camera to Street View imagery in order to determine a precise location and orientation. The Geospatial API often provides sub-meter positional accuracy (orders of magnitude better than GPS) and sub-degree orientation accuracy.

To successfully use the APIs in this extension (other than checking for extension support), the application **must** have successfully set authentication credentials via some mechanism, such as `XR_ANDROID_google_cloud_auth`. See the auth extension's documentation for more details about setup and error results.



Permissions

Android applications **must** have the `android.permission.ACCESS_FINE_LOCATION` permission listed in their manifest to use this extension. The `android.permission.ACCESS_FINE_LOCATION` permission is considered a dangerous permission. The application **must** request the permission at runtime to use these functions:

- xrCreateGeospatialTrackerANDROID

(protection level: dangerous)

12.62.2. Inspect system capability

The XrSystemGeospatialPropertiesANDROID structure is defined as:

```
// Provided by XR_ANDROID_geospatial
typedef struct XrSystemGeospatialPropertiesANDROID {
    XrStructureType    type;
    void*              next;
    XrBool32           supportsGeospatial;
} XrSystemGeospatialPropertiesANDROID;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension. For more details about the structure chain, see the structure being extended (XrSystemProperties).
- `supportsGeospatial` is an `XrBool32`, indicating if the current system supports geospatial features.

An application **can** inspect whether the system supports geospatial features by chaining an XrSystemGeospatialPropertiesANDROID structure to the XrSystemProperties when calling xrGetSystemProperties.

If a runtime returns `XR_FALSE` for `supportsGeospatial`, the system does not support geospatial features, and therefore **must** return `XR_ERROR_FEATURE_UNSUPPORTED` from xrCreateGeospatialTrackerANDROID. The application **should** avoid using geospatial functionality when `supportsGeospatial` is `XR_FALSE`.

Valid Usage (Implicit)

- The XR_ANDROID_geospatial extension **must** be enabled prior to using XrSystemGeospatialPropertiesANDROID
- type **must** be XR_TYPE_SYSTEM_GEOSPATIAL_PROPERTIES_ANDROID
- next **must** be NULL or a valid pointer to the next structure in a structure chain

12.62.3. Create a Geospatial Tracker handle

```
// Provided by XR_ANDROID_geospatial
XR_DEFINE_HANDLE(XrGeospatialTrackerANDROID)
```

The xrCreateGeospatialTrackerANDROID function is defined as:

```
// Provided by XR_ANDROID_geospatial
XrResult xrCreateGeospatialTrackerANDROID(
    XrSession                                   session,
    const XrGeospatialTrackerCreateInfoANDROID* createInfo,
    XrGeospatialTrackerANDROID*                 geospatialTrackerOutput);
```

Parameter Descriptions

- session is an XrSession in which the geospatial tracker will be active.
- createInfo is a pointer to an XrGeospatialTrackerCreateInfoANDROID structure specifying initial geospatial tracker parameters.
- geospatialTrackerOutput is a pointer to a handle in which the created XrGeospatialTrackerANDROID is returned.

An application **can** create an XrGeospatialTrackerANDROID handle by calling xrCreateGeospatialTrackerANDROID. The returned XrGeospatialTrackerANDROID handle **can** be subsequently used in API calls. If the application has not obtained the required permissions, the runtime **must** return XR_ERROR_PERMISSION_INSUFFICIENT. Only one XrGeospatialTrackerANDROID **can** exist at a time for a particular XrSession. The application **must** ensure that any previous XrGeospatialTrackerANDROID objects have been destroyed before calling this function again, otherwise the runtime **must** return XR_ERROR_LIMIT_REACHED. If the tracker is successfully created, it will initially enter the state XR_GEOSPATIAL_TRACKER_STATE_STOPPED_ANDROID, and the application **must** wait until the state changes to XR_GEOSPATIAL_TRACKER_STATE_RUNNING_ANDROID before using the tracker. See XrEventDataGeospatialTrackerStateChangedANDROID. If the application passes an XrGeospatialTrackerANDROID which is not in the state XR_GEOSPATIAL_TRACKER_STATE_RUNNING_ANDROID to

a function which requires it, the runtime **must** return XR_ERROR_GEOSPATIAL_TRACKER_NOT_RUNNING_ANDROID.

If geospatialTrackerOutput is valid, it remains so only within the lifecycle of session or until the application destroys it using xrDestroyGeospatialTrackerANDROID, whichever comes first.

Valid Usage (Implicit)

- The XR_ANDROID_geospatial extension **must** be enabled prior to calling xrCreateGeospatialTrackerANDROID
- session **must** be a valid XrSession handle
- createInfo **must** be a pointer to a valid XrGeospatialTrackerCreateInfoANDROID structure
- geospatialTrackerOutput **must** be a pointer to an XrGeospatialTrackerANDROID handle

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_PERMISSION_INSUFFICIENT
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The XrGeospatialTrackerCreateInfoANDROID structure is defined as:

```
// Provided by XR_ANDROID_geospatial
typedef struct XrGeospatialTrackerCreateInfoANDROID {
    XrStructureType    type;
    const void*        next;
} XrGeospatialTrackerCreateInfoANDROID;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.

Valid Usage (Implicit)

- The `XR_ANDROID_geospatial` extension **must** be enabled prior to using XrGeospatialTrackerCreateInfoANDROID
- `type` **must** be `XR_TYPE_GEOSPATIAL_TRACKER_CREATE_INFO_ANDROID`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain. See also: XrGeospatialTrackerAnchorTrackingInfoANDROID

The xrDestroyGeospatialTrackerANDROID function is defined as:

```
// Provided by XR_ANDROID_geospatial
XrResult xrDestroyGeospatialTrackerANDROID(
    XrGeospatialTrackerANDROID                  geospatialTracker);
```

Parameter Descriptions

- `geospatialTracker` is the XrGeospatialTrackerANDROID to be destroyed.

An application **can** use xrDestroyGeospatialTrackerANDROID function to release the geospatial tracker and the underlying resources.

Valid Usage (Implicit)

- The `XR_ANDROID_geospatial` extension **must** be enabled prior to calling xrDestroyGeospatialTrackerANDROID
- `geospatialTracker` **must** be a valid XrGeospatialTrackerANDROID handle

Thread Safety

- Access to `geospatialTracker`, and any child handles, **must** be externally synchronized

Return Codes

Success

- `XR_SUCCESS`

Failure

- `XR_ERROR_CALL_ORDER_INVALID`
- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_RUNTIME_FAILURE`

12.62.4. Geospatial Tracker State

The XrGeospatialTrackerStateANDROID enumeration is defined as:

```
// Provided by XR_ANDROID_geospatial
typedef enum XrGeospatialTrackerStateANDROID {
    XR_GEOSPATIAL_TRACKER_STATE_STOPPED_ANDROID = 0,
    XR_GEOSPATIAL_TRACKER_STATE_RUNNING_ANDROID = 1,
    XR_GEOSPATIAL_TRACKER_STATE_INITIALIZATION_FAILED_ANDROID = 2,
    XR_GEOSPATIAL_TRACKER_STATE_MAX_ENUM_ANDROID = 0x7FFFFFFF
} XrGeospatialTrackerStateANDROID;
```

The XrGeospatialTrackerStateANDROID enumeration identifies the different states of a geospatial tracker.

The enumerants have the following values:

Enum	Description
`XR_GEOSPATIAL_TRACKER_STATE_STOPPED_ANDROID`	The Geospatial Tracker is not running.
`XR_GEOSPATIAL_TRACKER_STATE_RUNNING_ANDROID`	The Geospatial tracker is running and usable.
`XR_GEOSPATIAL_TRACKER_STATE_INITIALIZATION_FAILED_ANDROID`	The Geospatial Tracker failed to initialize, and will never be usable.

The XrEventDataGeospatialTrackerStateChangedANDROID structure is defined as:

```
// Provided by XR_ANDROID_geospatial
typedef struct XrEventDataGeospatialTrackerStateChangedANDROID {
    XrStructureType                    type;
    const void*                        next;
    XrGeospatialTrackerANDROID         geospatialTracker;
    XrGeospatialTrackerStateANDROID    state;
    XrResult                           initializationResult;
    XrTime                             time;
} XrEventDataGeospatialTrackerStateChangedANDROID;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `geospatialTracker` is the XrGeospatialTrackerANDROID whose state has changed.
- `state` is the new XrGeospatialTrackerStateANDROID.
- `initializationResult` is the error result if `state` is `XR_GEOSPATIAL_TRACKER_STATE_INITIALIZATION_FAILED_ANDROID`, otherwise `XR_SUCCESS`.
- `time` is the `XrTime` at which the state change occurred.

The XrEventDataGeospatialTrackerStateChangedANDROID structure is sent when the geospatial tracker state changes. If the application has a valid XrGeospatialTrackerANDROID, it **should** poll for this event. The first event received for a tracker will have `state` `XR_GEOSPATIAL_TRACKER_STATE_STOPPED_ANDROID`. After a runtime-determined amount of time, the state **must** change to either `XR_GEOSPATIAL_TRACKER_STATE_RUNNING_ANDROID` or `XR_GEOSPATIAL_TRACKER_STATE_INITIALIZATION_FAILED_ANDROID`. This transition will take an arbitrary amount of time. If `state` changes to `XR_GEOSPATIAL_TRACKER_STATE_INITIALIZATION_FAILED_ANDROID`, it **must** be the last event received for this tracker, and the `initializationResult` field will hold the error code. It **may** take several seconds for an error to occur. The application **should** destroy the tracker in this case. If `state` changes to `XR_GEOSPATIAL_TRACKER_STATE_STOPPED_ANDROID`, all previously created geospatial anchors **must** permanently stop tracking and **should** be destroyed by the application. The state **may** change back and forth between `XR_GEOSPATIAL_TRACKER_STATE_RUNNING_ANDROID` and `XR_GEOSPATIAL_TRACKER_STATE_STOPPED_ANDROID` arbitrarily many times.

Valid Usage (Implicit)

- The XR_ANDROID_geospatial extension **must** be enabled prior to using XrEventDataGeospatialTrackerStateChangedANDROID
- `type` **must** be `XR_TYPE_EVENT_DATA_GEOSPATIAL_TRACKER_STATE_CHANGED_ANDROID`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

12.62.5. Geospatial Pose

The XrGeospatialPoseANDROID structure is defined as:

```
// Provided by XR_ANDROID_geospatial
typedef struct XrGeospatialPoseANDROID {
    XrQuaternionf    eastUpSouthOrientation;
    double           latitude;
    double           longitude;
    double           altitude;
} XrGeospatialPoseANDROID;
```

Member Descriptions

- `eastUpSouthOrientation` is an XrQuaternionf defining the orientation with respect to a coordinate system where +X=East, +Y=Up and +Z=South.
- `latitude` is the latitude in degrees, between -90 and +90.
- `longitude` is the longitude in degrees, between -180 and +180.
- `altitude` is the altitude in meters above the WGS84 ellipsoid.

The XrGeospatialPoseANDROID structure represents a position and orientation relative to the Earth using the WGS84 ellipsoid.

Valid Usage (Implicit)

- The XR_ANDROID_geospatial extension **must** be enabled prior to using XrGeospatialPoseANDROID
- `latitude` **must** be a valid `double` value
- `longitude` **must** be a valid `double` value
- `altitude` **must** be a valid `double` value

The XrGeospatialPoseFlagBitsANDROID enumeration is defined as:

```
// Provided by XR_ANDROID_geospatial
// Flag bits for XrGeospatialPoseFlagsANDROID
static const XrGeospatialPoseFlagsANDROID
XR_GEOSPATIAL_POSE_ORIENTATION_VALID_BIT_ANDROID = 0x00000001;
static const XrGeospatialPoseFlagsANDROID XR_GEOSPATIAL_POSE_POSITION_VALID_BIT_ANDROID =
0x00000002;
```

The XrGeospatialPoseFlagBitsANDROID enumeration specifies flags for geospatial poses.

The flag bits have the following meanings:

Flag Descriptions

- XR_GEOSPATIAL_POSE_ORIENTATION_VALID_BIT_ANDROID — Indicates that the orientation member contains valid data
- XR_GEOSPATIAL_POSE_POSITION_VALID_BIT_ANDROID — Indicates that the position member contains valid data

The XrGeospatialPoseFlagsANDROID type is a bitmask of XrGeospatialPoseFlagBitsANDROID.

```
// Provided by XR_ANDROID_geospatial
typedef XrFlags64 XrGeospatialPoseFlagsANDROID;
```

12.62.6. Converting XrPosef to Geospatial Pose

The xrLocateGeospatialPoseFromPoseANDROID function is defined as:

```
// Provided by XR_ANDROID_geospatial
XrResult xrLocateGeospatialPoseFromPoseANDROID(
    XrGeospatialTrackerANDROID                  geospatialTracker,
    const XrGeospatialPoseFromPoseLocateInfoANDROID* locateInfo,
    XrGeospatialPoseResultANDROID*              geospatialPoseResult);
```

Parameter Descriptions

- `geospatialTracker` is the XrGeospatialTrackerANDROID to use for conversion.
- `locateInfo` is a pointer to XrGeospatialPoseFromPoseLocateInfoANDROID containing query parameters.
- `geospatialPoseResult` is a pointer to XrGeospatialPoseResultANDROID that receives the result.

The xrLocateGeospatialPoseFromPoseANDROID function converts a pose in XrGeospatialPoseFromPoseLocateInfoANDROID::`space` to a geospatial pose. If the state of `geospatialTracker` is not `XR_GEOSPATIAL_TRACKER_STATE_RUNNING_ANDROID`, the runtime **must** return `XR_ERROR_GEOSPATIAL_TRACKER_NOT_RUNNING_ANDROID`. If the function returns `XR_SUCCESS`, the XrGeospatialPoseResultANDROID::`poseFlags` field of `geospatialPoseResult` determines which output fields are valid. If `XR_GEOSPATIAL_POSE_POSITION_VALID_BIT_ANDROID` is not set in XrGeospatialPoseResultANDROID::`poseFlags`, the application **must** not read the fields XrGeospatialPoseANDROID::`latitude`, XrGeospatialPoseANDROID::`longitude`, XrGeospatialPoseANDROID::`altitude`, XrGeospatialPoseResultANDROID::`horizontalAccuracy` or XrGeospatialPoseResultANDROID::`verticalAccuracy` in XrGeospatialPoseResultANDROID. If `XR_GEOSPATIAL_POSE_ORIENTATION_VALID_BIT_ANDROID` is not set in XrGeospatialPoseResultANDROID::`poseFlags`, the application **must** not read XrGeospatialPoseANDROID::`eastUpSouthOrientation` or XrGeospatialPoseResultANDROID::`orientationYawAccuracy`. If `XR_GEOSPATIAL_POSE_POSITION_VALID_BIT_ANDROID` is not set, `XR_GEOSPATIAL_POSE_ORIENTATION_VALID_BIT_ANDROID` **must** also not be set.

If the accuracy is lower than expected, this **may** be an indication that the device is not utilizing VPS localization. The application **can** instruct the user to point their device at signs and buildings to improve localization.

Valid Usage (Implicit)

- The `XR_ANDROID_geospatial` extension **must** be enabled prior to calling xrLocateGeospatialPoseFromPoseANDROID
- `geospatialTracker` **must** be a valid XrGeospatialTrackerANDROID handle
- `locateInfo` **must** be a pointer to a valid XrGeospatialPoseFromPoseLocateInfoANDROID structure
- `geospatialPoseResult` **must** be a pointer to an XrGeospatialPoseResultANDROID structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_GEOSPATIAL_TRACKER_NOT_RUNNING_ANDROID
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_POSE_INVALID
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_TIME_INVALID
- XR_ERROR_VALIDATION_FAILURE

The XrGeospatialPoseFromPoseLocateInfoANDROID structure is defined as:

```
// Provided by XR_ANDROID_geospatial
typedef struct XrGeospatialPoseFromPoseLocateInfoANDROID {
    XrStructureType    type;
    const void*        next;
    XrSpace            space;
    XrTime             time;
    XrPosef            pose;
} XrGeospatialPoseFromPoseLocateInfoANDROID;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain.
- space is the XrSpace in which pose is defined.
- time is the XrTime at which to evaluate pose.
- pose is the XrPosef in space to convert to a geospatial pose.

Valid Usage (Implicit)

- The XR_ANDROID_geospatial extension **must** be enabled prior to using XrGeospatialPoseFromPoseLocateInfoANDROID
- type **must** be XR_TYPE_GEOSPATIAL_POSE_FROM_POSE_LOCATE_INFO_ANDROID
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- space **must** be a valid XrSpace handle

The XrGeospatialPoseResultANDROID structure is defined as:

```
// Provided by XR_ANDROID_geospatial
typedef struct XrGeospatialPoseResultANDROID {
    XrStructureType                 type;
    void*                           next;
    XrGeospatialPoseFlagsANDROID    poseFlags;
    XrGeospatialPoseANDROID         geospatialPose;
    double                          horizontalAccuracy;
    double                          verticalAccuracy;
    double                          orientationYawAccuracy;
} XrGeospatialPoseResultANDROID;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain.
- poseFlags is a bitmask of XrGeospatialPoseFlagsANDROID indicating validity of pose components.
- geospatialPose is the resulting XrGeospatialPoseANDROID.
- horizontalAccuracy is the estimated horizontal accuracy of the Geospatial pose position, defined as the radius in meters of the circle of 68% confidence level around the given latitude and longitude.
- verticalAccuracy is the estimated vertical accuracy of the Geospatial pose position, defined as the distance in meters of 68% confidence level around the given altitude. In other words, there is a 68% chance that the true altitude is in the range [XrGeospatialPoseANDROID::altitude - verticalAccuracy, XrGeospatialPoseANDROID::altitude \+ verticalAccuracy].
- orientationYawAccuracy is the estimated yaw accuracy of the Geospatial pose orientation, defined as the radius in degrees of 68% confidence level around the given orientation.

Valid Usage (Implicit)

- The XR_ANDROID_geospatial extension **must** be enabled prior to using XrGeospatialPoseResultANDROID
- type **must** be XR_TYPE_GEOSPATIAL_POSE_RESULT_ANDROID
- next **must** be NULL or a valid pointer to the next structure in a structure chain

12.62.7. Converting a Geospatial Pose to an XrPosef

The xrLocateGeospatialPoseANDROID function is defined as:

```
// Provided by XR_ANDROID_geospatial
XrResult xrLocateGeospatialPoseANDROID(
    XrGeospatialTrackerANDROID                  geospatialTracker,
    const XrGeospatialPoseLocateInfoANDROID*    locateInfo,
    XrSpaceLocation*                            location);
```

Parameter Descriptions

- geospatialTracker is the XrGeospatialTrackerANDROID to use for conversion.
- locateInfo is a pointer to XrGeospatialPoseLocateInfoANDROID containing query parameters.
- location is a pointer to XrSpaceLocation that receives the resulting pose.

The xrLocateGeospatialPoseANDROID function converts a geospatial pose to an XrSpaceLocation. If the XrGeospatialTrackerANDROID is not running, the runtime **must** return XR_ERROR_GEOSPATIAL_TRACKER_NOT_RUNNING_ANDROID. If the function returns XR_SUCCESS, the XrSpaceLocation::locationFlags field of location determines which output fields are valid.

Valid Usage (Implicit)

- The XR_ANDROID_geospatial extension **must** be enabled prior to calling xrLocateGeospatialPoseANDROID
- geospatialTracker **must** be a valid XrGeospatialTrackerANDROID handle
- locateInfo **must** be a pointer to a valid XrGeospatialPoseLocateInfoANDROID structure
- location **must** be a pointer to an XrSpaceLocation structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_GEOSPATIAL_COORDINATES_INVALID_ANDROID
- XR_ERROR_GEOSPATIAL_TRACKER_NOT_RUNNING_ANDROID
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_POSE_INVALID
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_TIME_INVALID
- XR_ERROR_VALIDATION_FAILURE

The XrGeospatialPoseLocateInfoANDROID structure is defined as:

```
// Provided by XR_ANDROID_geospatial
typedef struct XrGeospatialPoseLocateInfoANDROID {
    XrStructureType            type;
    const void*                next;
    XrSpace                    space;
    XrTime                     time;
    XrGeospatialPoseANDROID    geospatialPose;
} XrGeospatialPoseLocateInfoANDROID;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain.
- space is the XrSpace in which the resulting pose will be represented.
- time is the XrTime at which to locate the pose.
- geospatialPose is the XrGeospatialPoseANDROID to convert.

Valid Usage (Implicit)

- The XR_ANDROID_geospatial extension **must** be enabled prior to using XrGeospatialPoseLocateInfoANDROID
- `type` **must** be `XR_TYPE_GEOSPATIAL_POSE_LOCATE_INFO_ANDROID`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `space` **must** be a valid XrSpace handle
- `geospatialPose` **must** be a valid XrGeospatialPoseANDROID structure

12.62.8. VPS Availability

The XrVPSAvailabilityANDROID enumeration is defined as:

```
// Provided by XR_ANDROID_geospatial
typedef enum XrVPSAvailabilityANDROID {
    XR_VPS_AVAILABILITY_UNAVAILABLE_ANDROID = 1,
    XR_VPS_AVAILABILITY_AVAILABLE_ANDROID = 2,
    XR_VPSAVAILABILITY_MAX_ENUM_ANDROID = 0x7FFFFFFF
} XrVPSAvailabilityANDROID;
```

The XrVPSAvailabilityANDROID enumeration indicates VPS availability.

The enumerants have the following values:

Enum	Description
XR_VPS_AVAILABILITY_UNAVAILABLE_ANDROID	VPS is not available near the given location.
XR_VPS_AVAILABILITY_AVAILABLE_ANDROID	VPS is available near the given location.

The xrCheckVpsAvailabilityAsyncANDROID function is defined as:

```
// Provided by XR_ANDROID_geospatial
XrResult xrCheckVpsAvailabilityAsyncANDROID(
    XrSession                                   session,
    double                                      latitude,
    double                                      longitude,
    XrFutureEXT*                                future);
```

Parameter Descriptions

- `session` is the XrSession to use for the check.
- `latitude` is the latitude in degrees.
- `longitude` is the longitude in degrees.
- `future` is a pointer to `XrFutureEXT` that will hold the result of the asynchronous operation.

Visual Positioning Service (VPS) availability indicates whether VPS **can** be used to improve geospatial accuracy at a given location.

The xrCheckVpsAvailabilityAsyncANDROID function begins an asynchronous check for VPS availability at a given location. The application does not need an XrGeospatialTrackerANDROID to call this function, and **can** use the result of this operation to decide whether to create one. If the application has not obtained the required permissions, the runtime **must** return `XR_ERROR_PERMISSION_INSUFFICIENT`.

Valid Usage (Implicit)

- The `XR_ANDROID_geospatial` extension **must** be enabled prior to calling xrCheckVpsAvailabilityAsyncANDROID
- `session` **must** be a valid XrSession handle
- `latitude` **must** be a valid `double` value
- `longitude` **must** be a valid `double` value
- `future` **must** be a pointer to an `XrFutureEXT` value

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_GEOSPATIAL_COORDINATES_INVALID_ANDROID
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The xrCheckVpsAvailabilityCompleteANDROID function is defined as:

```
// Provided by XR_ANDROID_geospatial
XrResult xrCheckVpsAvailabilityCompleteANDROID(
    XrSession                                   session,
    XrFutureEXT                                 future,
    XrVPSAvailabilityCheckCompletionANDROID*    completion);
```

Parameter Descriptions

- session is the XrSession used for the check.
- future is the XrFutureEXT returned by xrCheckVpsAvailabilityAsyncANDROID.
- completion is a pointer to XrVPSAvailabilityCheckCompletionANDROID that receives the result.

Valid Usage (Implicit)

- The XR_ANDROID_geospatial extension **must** be enabled prior to calling xrCheckVpsAvailabilityCompleteANDROID
- session **must** be a valid XrSession handle
- completion **must** be a pointer to an XrVPSAvailabilityCheckCompletionANDROID structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_FUTURE_INVALID_EXT
- XR_ERROR_FUTURE_PENDING_EXT
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The XrVPSAvailabilityCheckCompletionANDROID structure is defined as:

```
// Provided by XR_ANDROID_geospatial
typedef struct XrVPSAvailabilityCheckCompletionANDROID {
    XrStructureType             type;
    void*                       next;
    XrResult                    futureResult;
    XrVPSAvailabilityANDROID    availability;
} XrVPSAvailabilityCheckCompletionANDROID;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain.
- futureResult is the XrResult of the check operation. If futureResult is XR_ERROR_GEOSPATIAL_CLOUD_AUTH_FAILED_ANDROID, a structure in the next chain **may** provide more information about the failure.
- availability is the resulting XrVPSAvailabilityANDROID. The application **must** not read this field unless futureResult is XR_SUCCESS.

Future Return Codes

`futureResult` values:

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_GEOSPATIAL_CLOUD_AUTH_FAILED_ANDROID`

Valid Usage (Implicit)

- The `XR_ANDROID_geospatial` extension **must** be enabled prior to using XrVPSAvailabilityCheckCompletionANDROID
- `type` **must** be `XR_TYPE_VPS_AVAILABILITY_CHECK_COMPLETION_ANDROID`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `futureResult` **must** be a valid XrResult value
- `availability` **must** be a valid XrVPSAvailabilityANDROID value

12.62.9. Example

Setup Geospatial Tracker

```
PFN_xrCheckVpsAvailabilityAsyncANDROID xrCheckVpsAvailabilityAsyncANDROID;
PFN_xrPollFutureEXT xrPollFutureEXT;
PFN_xrCheckVpsAvailabilityCompleteANDROID xrCheckVpsAvailabilityCompleteANDROID;
PFN_xrCreateGeospatialTrackerANDROID xrCreateGeospatialTrackerANDROID;
XrInstance instance = XR_NULL_HANDLE;
XrSystemId systemId = XR_NULL_SYSTEM_ID;
XrSession session = XR_NULL_HANDLE;
double lat = 37.422, lng = -122.084;

// Check for support.
XrSystemGeospatialPropertiesANDROID geospatialSystemProperties{
  .type = XR_TYPE_SYSTEM_GEOSPATIAL_PROPERTIES_ANDROID,
```

```
};
XrSystemProperties systemProperties{
  .type = XR_TYPE_SYSTEM_PROPERTIES,
  .next = &geospatialSystemProperties,
};
CHK_XR(xrGetSystemProperties(instance, systemId, &systemProperties));
if (!geospatialSystemProperties.supportsGeospatial) {
  return;
}

// Check VPS Availability.
XrFutureEXT future {XR_NULL_FUTURE_EXT};
CHK_XR(xrCheckVpsAvailabilityAsyncANDROID(session, lat, lng, &future));

XrFuturePollInfoEXT pollInfo{
  .type = XR_TYPE_FUTURE_POLL_INFO_EXT,
  .future = future
};
XrFuturePollResultEXT pollResult{
  .type = XR_TYPE_FUTURE_POLL_RESULT_EXT,
  .state = XR_FUTURE_STATE_PENDING_EXT,
};

while (pollResult.state == XR_FUTURE_STATE_PENDING_EXT) {
  // Do in render loop/state loop.
  CHK_XR(xrPollFutureEXT(instance, &pollInfo, &pollResult));
}

XrVPSAvailabilityCheckCompletionANDROID vpsCompletion{
  .type = XR_TYPE_VPS_AVAILABILITY_CHECK_COMPLETION_ANDROID,
};
CHK_XR(xrCheckVpsAvailabilityCompleteANDROID(session, future, &vpsCompletion));
if (vpsCompletion.futureResult == XR_SUCCESS) {
  if (vpsCompletion.availability == XR_VPS_AVAILABILITY_UNAVAILABLE_ANDROID) {
    // Visual Positioning Service is not available. Accuracy of positions and
    // orientations from Geospatial APIs are expected to be lower at this location.
  } else {
    // Visual Positioning Service is available. Higher accuracy of position and
    // orientation is achievable at this location.
  }
}

// Create Geospatial Tracker.
XrGeospatialTrackerCreateInfoANDROID createInfo{
  .type = XR_TYPE_GEOSPATIAL_TRACKER_CREATE_INFO_ANDROID,
};
XrGeospatialTrackerANDROID geospatialTracker {XR_NULL_HANDLE};
CHK_XR(xrCreateGeospatialTrackerANDROID(session, &createInfo, &geospatialTracker));
```

```
// In application main event loop:
while (true) {
  XrEventDataBuffer event = {
    .type = XR_TYPE_EVENT_DATA_BUFFER,
  };
  if (xrPollEvent(instance, &event) != XR_SUCCESS) {
    continue;
  }
  switch (event.type) {
    case XR_TYPE_EVENT_DATA_GEOSPATIAL_TRACKER_STATE_CHANGED_ANDROID:
      const XrEventDataGeospatialTrackerStateChangedANDROID& eventData =
          *reinterpret_cast<XrEventDataGeospatialTrackerStateChangedANDROID*>(&event);
      switch (eventData.state) {
        case XR_GEOSPATIAL_TRACKER_STATE_STOPPED_ANDROID:
          // Destroy existing anchors, if any.
          break;
        case XR_GEOSPATIAL_TRACKER_STATE_RUNNING_ANDROID:
          // Start adding content.
          break;
        case XR_GEOSPATIAL_TRACKER_STATE_INITIALIZATION_FAILED_ANDROID:
          // Handle eventData.initializationResult error result.
          break;
      }
  }
}
```

Call Geospatial Pose APIs

```
// previously initialized
PFN_xrLocateGeospatialPoseFromPoseANDROID xrLocateGeospatialPoseFromPoseANDROID;
PFN_xrLocateGeospatialPoseANDROID xrLocateGeospatialPoseANDROID;
XrTime nextFrameTimestamp;

XrGeospatialTrackerANDROID geospatialTracker {XR_NULL_HANDLE};

// Get pose from view space.
XrSpace viewSpace {XR_NULL_HANDLE};
XrPosef identityPose = {{0,0,0,1},{0,0,0}};
XrGeospatialPoseFromPoseLocateInfoANDROID poseGetInfo{
  .type = XR_TYPE_GEOSPATIAL_POSE_FROM_POSE_LOCATE_INFO_ANDROID,
  .space = viewSpace,
  .time = nextFrameTimestamp,
  .pose = identityPose,
};
XrGeospatialPoseResultANDROID poseResult{
  .type = XR_TYPE_GEOSPATIAL_POSE_RESULT_ANDROID,
};
CHK_XR(xrLocateGeospatialPoseFromPoseANDROID(geospatialTracker, &poseGetInfo, &
poseResult));
if ((poseResult.poseFlags & XR_GEOSPATIAL_POSE_ORIENTATION_VALID_BIT_ANDROID) &&
    (poseResult.poseFlags & XR_GEOSPATIAL_POSE_POSITION_VALID_BIT_ANDROID)) {
  // poseResult.geospatialPose is valid.
}

// Convert Geospatial pose to an XrSpaceLocation.
XrGeospatialPoseLocateInfoANDROID poseLocateInfo{
  .type = XR_TYPE_GEOSPATIAL_POSE_LOCATE_INFO_ANDROID,
  .space = viewSpace,
  .time = nextFrameTimestamp,
  .geospatialPose = poseResult.geospatialPose,
};
XrSpaceLocation location{
  .type = XR_TYPE_SPACE_LOCATION,
};
CHK_XR(xrLocateGeospatialPoseANDROID(geospatialTracker, &poseLocateInfo, &location));
if ((location.locationFlags & XR_SPACE_LOCATION_ORIENTATION_VALID_BIT) &&
    (location.locationFlags & XR_SPACE_LOCATION_POSITION_VALID_BIT)) {
  // location.pose is valid.
}
```

12.62.10. New Object Types

- XrGeospatialTrackerANDROID

12.62.11. New Commands

- xrCheckVpsAvailabilityAsyncANDROID
- xrCheckVpsAvailabilityCompleteANDROID
- xrCreateGeospatialTrackerANDROID
- xrDestroyGeospatialTrackerANDROID
- xrLocateGeospatialPoseANDROID
- xrLocateGeospatialPoseFromPoseANDROID

12.62.12. New Structures

- XrEventDataGeospatialTrackerStateChangedANDROID
- XrGeospatialPoseANDROID
- XrGeospatialPoseFromPoseLocateInfoANDROID
- XrGeospatialPoseLocateInfoANDROID
- XrGeospatialPoseResultANDROID
- XrGeospatialTrackerCreateInfoANDROID
- XrVPSAvailabilityCheckCompletionANDROID
- Extending XrSystemProperties:
  - XrSystemGeospatialPropertiesANDROID

12.62.13. New Enums

- XrGeospatialPoseFlagBitsANDROID
- XrGeospatialTrackerStateANDROID
- XrVPSAvailabilityANDROID

12.62.14. New Bitmasks

- XrGeospatialPoseFlagsANDROID

12.62.15. New Enum Constants

- `XR_ANDROID_GEOSPATIAL_EXTENSION_NAME`
- `XR_ANDROID_geospatial_SPEC_VERSION`
- Extending XrObjectType:
  - `XR_OBJECT_TYPE_GEOSPATIAL_TRACKER_ANDROID`
- Extending XrResult:

    - XR_ERROR_GEOSPATIAL_CLOUD_AUTH_FAILED_ANDROID
    - XR_ERROR_GEOSPATIAL_COORDINATES_INVALID_ANDROID
    - XR_ERROR_GEOSPATIAL_TRACKER_NOT_RUNNING_ANDROID
- Extending XrStructureType:
    - XR_TYPE_EVENT_DATA_GEOSPATIAL_TRACKER_STATE_CHANGED_ANDROID
    - XR_TYPE_GEOSPATIAL_POSE_FROM_POSE_LOCATE_INFO_ANDROID
    - XR_TYPE_GEOSPATIAL_POSE_LOCATE_INFO_ANDROID
    - XR_TYPE_GEOSPATIAL_POSE_RESULT_ANDROID
    - XR_TYPE_GEOSPATIAL_TRACKER_CREATE_INFO_ANDROID
    - XR_TYPE_SYSTEM_GEOSPATIAL_PROPERTIES_ANDROID
    - XR_TYPE_VPS_AVAILABILITY_CHECK_COMPLETION_ANDROID

Issues

Version History

- Revision 1, 2025-12-18 (Ben King)
    - Initial extension description.

12.63. XR_ANDROID_geospatial_anchor

Name String

XR_ANDROID_geospatial_anchor

Extension Type

Instance extension

Registered Extension Number

798

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

XR_ANDROID_geospatial
and
XR_EXT_future
and

XR_EXT_spatial_entity
and
XR_EXT_spatial_anchor

Last Modified Date

2025-10-30

IP Status

No known IP claims.

Contributors

John Ullman, Google
Ben King, Google
Nihav Jain, Google
Jared Finder, Google

12.63.1. Overview

This extension provides Geospatial Anchors and Surface Anchors that build on the base Geospatial extension. Geospatial Anchors are anchors that are positioned in space relative to the Earth at a given latitude, longitude and altitude. Surface Anchors are Earth-relative anchors which are placed at a given latitude, longitude, and altitude relative to a surface as known by the Visual Positioning Service. As the runtime's accuracy of its position relative to the Earth improves, the Anchor's pose will adjust accordingly.

12.63.2. System Capability for Geospatial Anchors

The XrSystemGeospatialAnchorPropertiesANDROID structure is defined as:

```
// Provided by XR_ANDROID_geospatial_anchor
typedef struct XrSystemGeospatialAnchorPropertiesANDROID {
    XrStructureType    type;
    void*              next;
    uint32_t           maxSurfaceAnchorCount;
} XrSystemGeospatialAnchorPropertiesANDROID;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `maxSurfaceAnchorCount` is a `uint32_t` indicating the maximum number of surface anchors that **can** be created.

An application **can** inspect geospatial anchor capabilities of the system by chaining an XrSystemGeospatialAnchorPropertiesANDROID structure to the XrSystemProperties when calling xrGetSystemProperties.

If XrSystemGeospatialPropertiesANDROID::`supportsGeospatial` is `XR_TRUE`, then `maxSurfaceAnchorCount` indicates the maximum number of surface anchors the runtime supports. The limit **must** be greater than 0 in this case.

If XrSystemGeospatialPropertiesANDROID::`supportsGeospatial` is not `XR_TRUE`, then `maxSurfaceAnchorCount` **must** be 0.

Valid Usage (Implicit)

- The `XR_ANDROID_geospatial_anchor` extension **must** be enabled prior to using XrSystemGeospatialAnchorPropertiesANDROID
- `type` **must** be `XR_TYPE_SYSTEM_GEOSPATIAL_ANCHOR_PROPERTIES_ANDROID`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

12.63.3. Plane Tracking for Anchors

The XrGeospatialTrackerAnchorTrackingInfoANDROID structure is defined as:

```
// Provided by XR_ANDROID_geospatial_anchor
typedef struct XrGeospatialTrackerAnchorTrackingInfoANDROID {
    XrStructureType    type;
    const void*        next;
    XrBool32           shouldTrackPlanes;
} XrGeospatialTrackerAnchorTrackingInfoANDROID;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `shouldTrackPlanes` is an `XrBool32` indicating if the geospatial tracker will track planes to improve positioning of surface-locked anchors.

Plane tracking **can** be enabled to improve positioning of surface-locked anchors.

If the application wants to enable plane tracking to improve surface anchor poses, it **can** chain an XrGeospatialTrackerAnchorTrackingInfoANDROID structure to XrGeospatialTrackerCreateInfoANDROID when calling xrCreateGeospatialTrackerANDROID.

Valid Usage (Implicit)

- The `XR_ANDROID_geospatial_anchor` extension **must** be enabled prior to using XrGeospatialTrackerAnchorTrackingInfoANDROID
- `type` **must** be `XR_TYPE_GEOSPATIAL_TRACKER_ANCHOR_TRACKING_INFO_ANDROID`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

12.63.4. Geospatial Anchors

The xrCreateGeospatialAnchorANDROID function is defined as:

```
// Provided by XR_ANDROID_geospatial_anchor
XrResult xrCreateGeospatialAnchorANDROID(
    XrSpatialContextEXT                         spatialContext,
    const XrGeospatialAnchorCreateInfoANDROID*  createInfo,
    XrSpatialEntityIdEXT*                       anchorEntityId);
```

Parameter Descriptions

- `spatialContext` is the XrSpatialContextEXT to create the anchor in. The context **must** be configured for `XR_SPATIAL_CAPABILITY_ANCHOR_EXT` otherwise the runtime **must** return `XR_ERROR_VALIDATION_FAILURE`.
- `createInfo` is a pointer to XrGeospatialAnchorCreateInfoANDROID containing anchor creation parameters.
- `anchorEntityId` is a pointer to `XrSpatialEntityIdEXT` in which the anchor entity ID is returned.

If the XrGeospatialTrackerANDROID specified in XrGeospatialAnchorCreateInfoANDROID ::geospatialTracker is not in the XR_GEOSPATIAL_TRACKER_STATE_RUNNING_ANDROID state, the runtime **must** return XR_ERROR_GEOSPATIAL_TRACKER_NOT_RUNNING_ANDROID.

Valid Usage (Implicit)

- The XR_ANDROID_geospatial_anchor extension **must** be enabled prior to calling xrCreateGeospatialAnchorANDROID
- spatialContext **must** be a valid XrSpatialContextEXT handle
- createInfo **must** be a pointer to a valid XrGeospatialAnchorCreateInfoANDROID structure
- anchorEntityId **must** be a pointer to an XrSpatialEntityIdEXT value

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_GEOSPATIAL_COORDINATES_INVALID_ANDROID
- XR_ERROR_GEOSPATIAL_TRACKER_NOT_RUNNING_ANDROID
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_POSE_INVALID
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The XrGeospatialAnchorCreateInfoANDROID structure is defined as:

```
// Provided by XR_ANDROID_geospatial_anchor
typedef struct XrGeospatialAnchorCreateInfoANDROID {
    XrStructureType               type;
    const void*                   next;
    XrGeospatialTrackerANDROID    geospatialTracker;
    XrGeospatialPoseANDROID       geospatialPose;
} XrGeospatialAnchorCreateInfoANDROID;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `geospatialTracker` is the XrGeospatialTrackerANDROID to use for creating the anchor.
- `geospatialPose` is a pointer to XrGeospatialPoseANDROID defining the anchor's location and orientation.

Valid Usage (Implicit)

- The `XR_ANDROID_geospatial_anchor` extension **must** be enabled prior to using XrGeospatialAnchorCreateInfoANDROID
- `type` **must** be `XR_TYPE_GEOSPATIAL_ANCHOR_CREATE_INFO_ANDROID`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `geospatialTracker` **must** be a valid XrGeospatialTrackerANDROID handle
- `geospatialPose` **must** be a valid XrGeospatialPoseANDROID structure

12.63.5. Surface Anchors

Surface anchors are anchors placed relative to a surface, such as terrain or rooftops.

The XrSurfaceAnchorTypeANDROID enumeration is defined as:

```
// Provided by XR_ANDROID_geospatial_anchor
typedef enum XrSurfaceAnchorTypeANDROID {
    XR_SURFACE_ANCHOR_TYPE_TERRAIN_ANDROID = 1,
    XR_SURFACE_ANCHOR_TYPE_ROOFTOP_ANDROID = 2,
    XR_SURFACE_ANCHOR_TYPE_MAX_ENUM_ANDROID = 0x7FFFFFFF
} XrSurfaceAnchorTypeANDROID;
```

The XrSurfaceAnchorTypeANDROID enumeration specifies the type of surface an anchor is relative to.

The enumerants have the following values:

Enum	Description
XR_SURFACE_ANCHOR_TYPE_TERRAIN_ANDROID	Type of an anchor placed relative to the ground.
XR_SURFACE_ANCHOR_TYPE_ROOFTOP_ANDROID	Type of an anchor placed relative to the rooftop, or ground where there is no building.

The xrCreateSurfaceAnchorAsyncANDROID function is defined as:

```
// Provided by XR_ANDROID_geospatial_anchor
XrResult xrCreateSurfaceAnchorAsyncANDROID(
    XrSpatialContextEXT                         spatialContext,
    const XrSurfaceAnchorCreateInfoANDROID*     createInfo,
    XrFutureEXT*                                future);
```

Parameter Descriptions

- spatialContext is the XrSpatialContextEXT to create the anchor in.
- createInfo is a pointer to XrSurfaceAnchorCreateInfoANDROID containing anchor creation parameters.
- future is a pointer to XrFutureEXT that will hold the result of the asynchronous operation.

The xrCreateSurfaceAnchorAsyncANDROID function begins an asynchronous operation to create a surface anchor. Unlike standard geospatial anchors, the runtime **may** fetch terrain data to determine the correct altitude. The runtime **must** return XR_ERROR_LIMIT_REACHED if the application attempts to create more than XrSystemGeospatialAnchorPropertiesANDROID::maxSurfaceAnchorCount surface anchors at a time. If the XrGeospatialTrackerANDROID specified in XrSurfaceAnchorCreateInfoANDROID::geospatialTracker is not in the XR_GEOSPATIAL_TRACKER_STATE_RUNNING_ANDROID state, the runtime **must** return XR_ERROR_GEOSPATIAL_TRACKER_NOT_RUNNING_ANDROID. The operation **may** fail asynchronously with result XR_ERROR_SURFACE_ANCHOR_LOCATION_UNSUPPORTED_ANDROID if there is no surface data for the given location. The runtime **must** return XR_ERROR_VALIDATION_FAILURE if XR_SPATIAL_CAPABILITY_ANCHOR_EXT was not configured for spatialContext.

Valid Usage (Implicit)

- The XR_ANDROID_geospatial_anchor extension **must** be enabled prior to calling xrCreateSurfaceAnchorAsyncANDROID
- spatialContext **must** be a valid XrSpatialContextEXT handle
- createInfo **must** be a pointer to a valid XrSurfaceAnchorCreateInfoANDROID structure
- future **must** be a pointer to an XrFutureEXT value

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_GEOSPATIAL_COORDINATES_INVALID_ANDROID
- XR_ERROR_GEOSPATIAL_TRACKER_NOT_RUNNING_ANDROID
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_POSE_INVALID
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The XrSurfaceAnchorCreateInfoANDROID structure is defined as:

```
// Provided by XR_ANDROID_geospatial_anchor
typedef struct XrSurfaceAnchorCreateInfoANDROID {
    XrStructureType               type;
    const void*                   next;
    XrGeospatialTrackerANDROID    geospatialTracker;
    XrSurfaceAnchorTypeANDROID    surfaceAnchorType;
    XrQuaternionf                 eastUpSouthOrientation;
    double                        latitude;
    double                        longitude;
    double                        altitudeRelativeToSurface;
} XrSurfaceAnchorCreateInfoANDROID;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `geospatialTracker` is the XrGeospatialTrackerANDROID to use.
- `surfaceAnchorType` is the XrSurfaceAnchorTypeANDROID for the anchor.
- `eastUpSouthOrientation` is the orientation with respect to a coordinate system where +X=East, +Y=Up and +Z=South.
- `latitude` is the latitude in degrees, between -89.9 and +89.9.
- `longitude` is the longitude in degrees, between -180 and +180.
- `altitudeRelativeToSurface` is the altitude in meters relative to the surface specified by `surfaceAnchorType`.

Valid Usage (Implicit)

- The `XR_ANDROID_geospatial_anchor` extension **must** be enabled prior to using XrSurfaceAnchorCreateInfoANDROID
- `type` **must** be `XR_TYPE_SURFACE_ANCHOR_CREATE_INFO_ANDROID`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `geospatialTracker` **must** be a valid XrGeospatialTrackerANDROID handle
- `surfaceAnchorType` **must** be a valid XrSurfaceAnchorTypeANDROID value
- `latitude` **must** be a valid `double` value
- `longitude` **must** be a valid `double` value
- `altitudeRelativeToSurface` **must** be a valid `double` value

The xrCreateSurfaceAnchorCompleteANDROID function is defined as:

```
// Provided by XR_ANDROID_geospatial_anchor
XrResult xrCreateSurfaceAnchorCompleteANDROID(
    XrSpatialContextEXT                         spatialContext,
    XrFutureEXT                                 future,
    XrSurfaceAnchorCreateCompletionANDROID*     completion);
```

Parameter Descriptions

- `spatialContext` is the XrSpatialContextEXT used for creation.
- `future` is the `XrFutureEXT` returned by xrCreateSurfaceAnchorAsyncANDROID.
- `completion` is a pointer to XrSurfaceAnchorCreateCompletionANDROID that receives the result.

Valid Usage (Implicit)

- The `XR_ANDROID_geospatial_anchor` extension **must** be enabled prior to calling xrCreateSurfaceAnchorCompleteANDROID
- `spatialContext` **must** be a valid XrSpatialContextEXT handle
- `completion` **must** be a pointer to an XrSurfaceAnchorCreateCompletionANDROID structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_FUTURE_INVALID_EXT
- XR_ERROR_FUTURE_PENDING_EXT
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The XrSurfaceAnchorCreateCompletionANDROID structure holds the result of an asynchronous surface anchor creation.

```
// Provided by XR_ANDROID_geospatial_anchor
typedef struct XrSurfaceAnchorCreateCompletionANDROID {
    XrStructureType         type;
    void*                   next;
    XrResult                futureResult;
    XrSpatialEntityIdEXT    anchorEntityId;
} XrSurfaceAnchorCreateCompletionANDROID;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `futureResult` is the XrResult of the creation operation. If `futureResult` is `XR_ERROR_GEOSPATIAL_CLOUD_AUTH_FAILED_ANDROID`, a structure in the `next` chain **may** provide more information about the failure.
- `anchorEntityId` is the `XrSpatialEntityIdEXT` of the created anchor, or XR_NULL_SPATIAL_ENTITY_ID_EXT if `futureResult` is not `XR_SUCCESS`.

Future Return Codes

`futureResult` values:

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_OUT_OF_MEMORY`
- `XR_ERROR_LIMIT_REACHED`
- `XR_ERROR_GEOSPATIAL_TRACKER_NOT_RUNNING_ANDROID`
- `XR_ERROR_SURFACE_ANCHOR_LOCATION_UNSUPPORTED_ANDROID`
- `XR_ERROR_GEOSPATIAL_CLOUD_AUTH_FAILED_ANDROID`

Valid Usage (Implicit)

- The `XR_ANDROID_geospatial_anchor` extension **must** be enabled prior to using XrSurfaceAnchorCreateCompletionANDROID
- `type` **must** be `XR_TYPE_SURFACE_ANCHOR_CREATE_COMPLETION_ANDROID`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `futureResult` **must** be a valid XrResult value

12.63.6. Example

Create Terrain Anchor

```
PFN_xrCreateSpatialContextAsyncEXT xrCreateSpatialContextAsyncEXT;
PFN_xrCreateSpatialContextCompleteEXT xrCreateSpatialContextCompleteEXT;
PFN_xrCreateSurfaceAnchorAsyncANDROID xrCreateSurfaceAnchorAsyncANDROID;
PFN_xrPollFutureEXT xrPollFutureEXT;
PFN_xrCreateSurfaceAnchorCompleteANDROID xrCreateSurfaceAnchorCompleteANDROID;
XrInstance instance;
XrSession session;
XrGeospatialTrackerANDROID geospatialTracker;

// Create a spatial context
XrSpatialContextEXT spatialContext{};
{
  std::vector<XrSpatialComponentTypeEXT> enabledComponents = {
    XR_SPATIAL_COMPONENT_TYPE_ANCHOR_EXT,
  };

  XrSpatialCapabilityConfigurationAnchorEXT anchorConfig {
    .type = XR_TYPE_SPATIAL_CAPABILITY_CONFIGURATION_ANCHOR_EXT,
    .capability = XR_SPATIAL_CAPABILITY_ANCHOR_EXT,
    .enabledComponentCount = static_cast<uint32_t>(enabledComponents.size()),
    .enabledComponents = enabledComponents.data(),
  };

  std::array<XrSpatialCapabilityConfigurationBaseHeaderEXT*, 1> capabilityConfigs = {
    reinterpret_cast<XrSpatialCapabilityConfigurationBaseHeaderEXT*>(&anchorConfig),
  };

  XrSpatialContextCreateInfoEXT spatialContextCreateInfo {
    .type = XR_TYPE_SPATIAL_CONTEXT_CREATE_INFO_EXT,
    .capabilityConfigCount = capabilityConfigs.size(),
    .capabilityConfigs = capabilityConfigs.data(),
  };
  XrFutureEXT createContextFuture {XR_NULL_FUTURE_EXT};
  CHK_XR(xrCreateSpatialContextAsyncEXT(session, &spatialContextCreateInfo,
&createContextFuture));

  // ... wait until future is ready ...

  XrCreateSpatialContextCompletionEXT contextCompletion{
    .type = XR_TYPE_CREATE_SPATIAL_CONTEXT_COMPLETION_EXT,
  };
  CHK_XR(xrCreateSpatialContextCompleteEXT(session, createContextFuture,
&contextCompletion));
```

```
  if (contextCompletion.futureResult != XR_SUCCESS) {
    return;
  }

  spatialContext = contextCompletion.spatialContext;
}

XrSurfaceAnchorCreateInfoANDROID anchorCreateInfo {
  .type = XR_TYPE_SURFACE_ANCHOR_CREATE_INFO_ANDROID,
  .geospatialTracker = geospatialTracker,
  .surfaceAnchorType = XR_SURFACE_ANCHOR_TYPE_TERRAIN_ANDROID,
  .eastUpSouthOrientation = {0, 0, 0, 1},
  .latitude = 37.7749,
  .longitude = -122.4194,
  .altitudeRelativeToSurface = 0,
};
XrFutureEXT anchorFuture {XR_NULL_FUTURE_EXT};
CHK_XR(xrCreateSurfaceAnchorAsyncANDROID(spatialContext, &anchorCreateInfo, &
anchorFuture));

XrFuturePollInfoEXT anchorPollInfo{
  .type = XR_TYPE_FUTURE_POLL_INFO_EXT,
  .future = anchorFuture,
};
XrFuturePollResultEXT anchorPollResult{
  .type = XR_TYPE_FUTURE_POLL_RESULT_EXT,
  .state = XR_FUTURE_STATE_PENDING_EXT,
};
while (anchorPollResult.state == XR_FUTURE_STATE_PENDING_EXT) {
  // Do in render loop/state loop.
  CHK_XR(xrPollFutureEXT(instance, &anchorPollInfo, &anchorPollResult));
}

XrSurfaceAnchorCreateCompletionANDROID anchorCompletion{
  .type = XR_TYPE_SURFACE_ANCHOR_CREATE_COMPLETION_ANDROID,
};
CHK_XR(xrCreateSurfaceAnchorCompleteANDROID(spatialContext, anchorFuture,
&anchorCompletion));
if (anchorCompletion.futureResult == XR_SUCCESS) {
  // Use completion.anchorEntityId.
  XrSpatialEntityIdEXT anchorId = anchorCompletion.anchorEntityId;

  // Query in UpdateSnapshot.
}
```

12.63.7. New Commands

- xrCreateGeospatialAnchorANDROID
- xrCreateSurfaceAnchorAsyncANDROID
- xrCreateSurfaceAnchorCompleteANDROID

12.63.8. New Structures

- XrGeospatialAnchorCreateInfoANDROID
- XrSurfaceAnchorCreateCompletionANDROID
- XrSurfaceAnchorCreateInfoANDROID
- Extending XrGeospatialTrackerCreateInfoANDROID:
  - XrGeospatialTrackerAnchorTrackingInfoANDROID
- Extending XrSystemProperties:
  - XrSystemGeospatialAnchorPropertiesANDROID

12.63.9. New Enums

- XrSurfaceAnchorTypeANDROID

12.63.10. New Enum Constants

- XR_ANDROID_GEOSPATIAL_ANCHOR_EXTENSION_NAME
- XR_ANDROID_geospatial_anchor_SPEC_VERSION
- Extending XrResult:
  - XR_ERROR_SURFACE_ANCHOR_LOCATION_UNSUPPORTED_ANDROID
- Extending XrStructureType:
  - XR_TYPE_GEOSPATIAL_ANCHOR_CREATE_INFO_ANDROID
  - XR_TYPE_GEOSPATIAL_TRACKER_ANCHOR_TRACKING_INFO_ANDROID
  - XR_TYPE_SURFACE_ANCHOR_CREATE_COMPLETION_ANDROID
  - XR_TYPE_SURFACE_ANCHOR_CREATE_INFO_ANDROID
  - XR_TYPE_SYSTEM_GEOSPATIAL_ANCHOR_PROPERTIES_ANDROID

12.63.11. Issues

12.63.12. Version History

- Revision 1, 2025-10-30 (Ben King)

- Initial draft.

12.64. XR_ANDROID_google_cloud_auth

Name String

`XR_ANDROID_google_cloud_auth`

Extension Type

Instance extension

Registered Extension Number

788

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

`XR_EXT_future`

Last Modified Date

2025-12-18

IP Status

No known IP claims.

Contributors

John Ullman, Google
Ben King, Google
Nihav Jain, Google
Jared Finder, Google

12.64.1. Overview

This extension enables the use of Google Cloud-based extensions by allowing the application to provide auth credentials for Google Cloud APIs. The developer **must** use the Google Cloud Console (https://console.cloud.google.com/) to create a Google Cloud project for the application. This extension requires the extension `XR_EXT_future`.

During development, the application **can** debug issues with their Google Cloud setup using the extension `XR_EXT_debug_utils`. If the application has a Debug Messenger, and a potentially actionable error occurs in the runtime when sending a request to Google Cloud on behalf of the application, the runtime invokes the messenger's callback with an error message. In this case, the developer **should**

consult the error message and all available documentation, and then ensure that the application and Google Cloud project are properly setup to use Google Cloud APIs. The runtime **must** use message type XR_DEBUG_UTILS_MESSAGE_TYPE_GENERAL_BIT_EXT, severity XR_DEBUG_UTILS_MESSAGE_SEVERITY_ERROR_BIT_EXT, and messageId "GoogleCloudError".

12.64.2. Authentication

The XrGoogleCloudAuthInfoBaseHeaderANDROID structure is defined as:

```
// Provided by XR_ANDROID_google_cloud_auth
typedef struct XrGoogleCloudAuthInfoBaseHeaderANDROID {
    XrStructureType    type;
    const void*        next;
} XrGoogleCloudAuthInfoBaseHeaderANDROID;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain.

This structure is not intended to be explicitly constructed but is extended by other structures that **can** be used with xrSetGoogleCloudAuthAsyncANDROID to provide authentication credentials.

Valid Usage (Implicit)

- The XR_ANDROID_google_cloud_auth extension **must** be enabled prior to using XrGoogleCloudAuthInfoBaseHeaderANDROID
- type **must** be one of the following XrStructureType values: XR_TYPE_GOOGLE_CLOUD_AUTH_INFO_API_KEY_ANDROID, XR_TYPE_GOOGLE_CLOUD_AUTH_INFO_KEYLESS_ANDROID, XR_TYPE_GOOGLE_CLOUD_AUTH_INFO_TOKEN_ANDROID
- next **must** be NULL or a valid pointer to the next structure in a structure chain. See also: XrGoogleCloudAuthErrorResultANDROID

The XrGoogleCloudAuthInfoApiKeyANDROID structure is defined as:

```
// Provided by XR_ANDROID_google_cloud_auth
typedef struct XrGoogleCloudAuthInfoApiKeyANDROID {
    XrStructureType    type;
    const void*        next;
    const char*        apiKey;
} XrGoogleCloudAuthInfoApiKeyANDROID;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `apiKey` is a pointer to a string representing the API key.

When this structure is passed to xrSetGoogleCloudAuthAsyncANDROID, the `apiKey` member **must** be a nonempty ASCII string with no spaces or control characters, otherwise the runtime **must** return `XR_ERROR_VALIDATION_FAILURE`.

`apiKey` **must** further satisfy the following conditions:

- It **must** be a valid API Key generated for your Google Cloud project.
- Your Google Cloud project **must** enable the relevant Google Cloud APIs (specified by the depending extensions).
- If the API Key has restrictions, the restrictions **must** allow the relevant Google Cloud APIs and your application.

Otherwise, the call to xrSetGoogleCloudAuthAsyncANDROID will succeed but all calls to functions that depend on cloud authorization will act as documented for cloud failures in the extension that defines those functions. If such a function reports a failure and the application chains XrGoogleCloudAuthErrorResultANDROID to the output parameter of that function, the runtime **must** set XrGoogleCloudAuthErrorResultANDROID::`error` to `XR_GOOGLE_CLOUD_AUTH_ERROR_ANDROID` to indicate this error.

The asynchronous operation will complete with `XR_SUCCESS` once the API key is validated for formatting and stored by the runtime.

The application does not need to keep `apiKey` alive after the function returns. The runtime **must** make a copy of the `apiKey` for its asynchronous processing.

Valid Usage (Implicit)

- The XR_ANDROID_google_cloud_auth extension **must** be enabled prior to using XrGoogleCloudAuthInfoApiKeyANDROID
- type **must** be XR_TYPE_GOOGLE_CLOUD_AUTH_INFO_API_KEY_ANDROID
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- apiKey **must** be a null-terminated UTF-8 string

The XrGoogleCloudAuthInfoTokenANDROID structure is defined as:

```
// Provided by XR_ANDROID_google_cloud_auth
typedef struct XrGoogleCloudAuthInfoTokenANDROID {
    XrStructureType    type;
    const void*        next;
    const char*        authToken;
} XrGoogleCloudAuthInfoTokenANDROID;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain.
- authToken is a pointer to a string representing the auth token.

When this structure is passed to xrSetGoogleCloudAuthAsyncANDROID, the authToken member **must** be a nonempty ASCII string with no spaces or control characters, otherwise the runtime **must** return XR_ERROR_VALIDATION_FAILURE. authToken **must** further satisfy the following conditions:

- It **must** be a valid and unexpired credential generated for your Google Cloud project.
- Your Google Cloud project **must** enable the relevant Google Cloud APIs (specified by the depending extensions).
- The credential **must** be one of:
  - An OAuth2 access token with the relevant scopes, generated by signing into a Google account with your application, OR
  - A Signed JWT token with the relevant claims, generated by a Service Account from your Google Cloud project.

The requirements in each case are specified by the depending extensions. Otherwise, the call to xrSetGoogleCloudAuthAsyncANDROID will succeed but all calls to functions that depend on cloud authorization will act as documented for cloud failures in the extension that defines those functions. If

such a function reports a failure and the application chains XrGoogleCloudAuthErrorResultANDROID to the output parameter of that function, the runtime **must** set XrGoogleCloudAuthErrorResultANDROID::`error` to `XR_GOOGLE_CLOUD_AUTH_ERROR_ANDROID` to indicate this error.

The application **must** proactively pass in a new token via xrSetGoogleCloudAuthAsyncANDROID before the old token expires otherwise the runtime **must** report `XR_GOOGLE_CLOUD_AUTH_ERROR_ANDROID` via XrGoogleCloudAuthErrorResultANDROID from the functions that require Google Cloud authentication. The runtime **must** use the latest token passed in by the application when starting a new network request.

The asynchronous operation will complete with `XR_SUCCESS` once the token is validated for formatting and stored by the runtime.

The application does not need to keep `authToken` alive after the function returns. The runtime **must** make a copy of the `authToken` for its asynchronous processing.

Valid Usage (Implicit)

- The `XR_ANDROID_google_cloud_auth` extension **must** be enabled prior to using XrGoogleCloudAuthInfoTokenANDROID
- `type` **must** be `XR_TYPE_GOOGLE_CLOUD_AUTH_INFO_TOKEN_ANDROID`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `authToken` **must** be a null-terminated UTF-8 string

The XrGoogleCloudAuthInfoKeylessANDROID structure is defined as:

```
// Provided by XR_ANDROID_google_cloud_auth
typedef struct XrGoogleCloudAuthInfoKeylessANDROID {
    XrStructureType    type;
    const void*        next;
} XrGoogleCloudAuthInfoKeylessANDROID;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.

When this structure is passed to xrSetGoogleCloudAuthAsyncANDROID, the runtime **may** generate credentials dynamically (e.g. on Android, the runtime communicates with Google Play Services from the application process). If Keyless Auth is not properly setup, the runtime **may** return

XR_ERROR_KEYLESS_AUTH_NOT_SETUP_ANDROID synchronously.

The asynchronous operation will perform the necessary network requests to fetch credentials. The result of this operation will be returned in the XrFutureCompletionEXT::futureResult from xrSetGoogleCloudAuthCompleteANDROID. If the future result is XR_SUCCESS, Keyless Auth credentials were successfully applied. If the future result is XR_ERROR_KEYLESS_AUTH_FAILED_ANDROID, the application **may** try again later.

Valid Usage (Implicit)

- The XR_ANDROID_google_cloud_auth extension **must** be enabled prior to using XrGoogleCloudAuthInfoKeylessANDROID
- type **must** be XR_TYPE_GOOGLE_CLOUD_AUTH_INFO_KEYLESS_ANDROID
- next **must** be NULL or a valid pointer to the next structure in a structure chain

The xrSetGoogleCloudAuthAsyncANDROID function is defined as:

```
// Provided by XR_ANDROID_google_cloud_auth
XrResult xrSetGoogleCloudAuthAsyncANDROID(
    XrSession                                       session,
    const XrGoogleCloudAuthInfoBaseHeaderANDROID* authInfo,
    XrFutureEXT*                                    future);
```

Parameter Descriptions

- session is the XrSession that will use the credentials.
- authInfo is a pointer to a struct specifying the authentication method and parameters, with its XrGoogleCloudAuthInfoBaseHeaderANDROID::type field indicating the specific struct.
- future is a pointer to an XrFutureEXT handle in which the created future is returned, or XR_NULL_HANDLE if the function did not return XR_SUCCESS.

Set the credentials used to authenticate with Google Cloud. This operation is asynchronous. Call xrPollFutureEXT to check the ready state on the future. Once the future is in the ready state, call xrSetGoogleCloudAuthCompleteANDROID to retrieve the result.

The authInfo parameter **must** be a pointer to a struct whose XrGoogleCloudAuthInfoBaseHeaderANDROID::type member identifies the authentication method to be used, and which extends XrGoogleCloudAuthInfoBaseHeaderANDROID (e.g. XrGoogleCloudAuthInfoApiKeyANDROID).

The specific requirements, behaviors, and error conditions (both synchronous and asynchronous) for

each authentication method are described in the documentation for their respective data structs.

Valid Usage (Implicit)

- The `XR_ANDROID_google_cloud_auth` extension **must** be enabled prior to calling xrSetGoogleCloudAuthAsyncANDROID
- `session` **must** be a valid XrSession handle
- `authInfo` **must** be a pointer to a valid XrGoogleCloudAuthInfoBaseHeaderANDROID-based structure. See also: XrGoogleCloudAuthInfoApiKeyANDROID, XrGoogleCloudAuthInfoKeylessANDROID, XrGoogleCloudAuthInfoTokenANDROID
- `future` **must** be a pointer to an `XrFutureEXT` value

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_KEYLESS_AUTH_NOT_SETUP_ANDROID`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_VALIDATION_FAILURE`

The xrSetGoogleCloudAuthCompleteANDROID function is defined as:

```
// Provided by XR_ANDROID_google_cloud_auth
XrResult xrSetGoogleCloudAuthCompleteANDROID(
    XrSession                                   session,
    XrFutureEXT                                 future,
    XrFutureCompletionEXT*                      completion);
```

Parameter Descriptions

- `session` is the XrSession used to start the operation.
- `future` is the `XrFutureEXT` to complete.
- `completion` is a pointer to an XrFutureCompletionEXT filled in by the runtime.

Future Return Codes

XrFutureCompletionEXT::`futureResult` values:

Success

- `XR_SUCCESS`: Credentials were successfully applied.

Failure

- `XR_ERROR_KEYLESS_AUTH_FAILED_ANDROID`: Keyless Auth failed. The application **can** try again later. The application or developer **may** also check certain preconditions for success (e.g. on Android, the device **must** have an up-to-date installation of Google Play Services and these services **must** be functioning properly).

Valid Usage (Implicit)

- The `XR_ANDROID_google_cloud_auth` extension **must** be enabled prior to calling xrSetGoogleCloudAuthCompleteANDROID
- `session` **must** be a valid XrSession handle
- `completion` **must** be a pointer to an XrFutureCompletionEXT structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_FUTURE_INVALID_EXT
- XR_ERROR_FUTURE_PENDING_EXT
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

12.64.3. Google Cloud Auth Errors

The XrGoogleCloudAuthErrorANDROID enumeration is defined as:

```
// Provided by XR_ANDROID_google_cloud_auth
typedef enum XrGoogleCloudAuthErrorANDROID {
    XR_GOOGLE_CLOUD_AUTH_ERROR_NONE_ANDROID = 0,
    XR_GOOGLE_CLOUD_AUTH_ERROR_QUOTA_EXCEEDED_ANDROID = -1,
    XR_GOOGLE_CLOUD_AUTH_ERROR_UNREACHABLE_ANDROID = -2,
    XR_GOOGLE_CLOUD_AUTH_ERROR_ANDROID = -3,
    XR_GOOGLE_CLOUD_AUTH_ERROR_MAX_ENUM_ANDROID = 0x7FFFFFFF
} XrGoogleCloudAuthErrorANDROID;
```

The enumerants have the following values:

Enum	Description
XR_GOOGLE_CLOUD_AUTH_ERROR_NONE_ANDROID	No error occurred when invoking a Google Cloud API.
XR_GOOGLE_CLOUD_AUTH_ERROR_QUOTA_EXCEEDED_ANDROID	Quota exceeded when invoking a Google Cloud API.
XR_GOOGLE_CLOUD_AUTH_ERROR_UNREACHABLE_ANDROID	Failed to reach a Google Cloud API, possibly due to network connectivity issues or server availability.

Enum	Description
XR_GOOGLE_CLOUD_AUTH_ERROR_ANDROID	An auth error occurred when invoking a Google Cloud API.

The XrGoogleCloudAuthErrorResultANDROID structure is defined as:

```
// Provided by XR_ANDROID_google_cloud_auth
typedef struct XrGoogleCloudAuthErrorResultANDROID {
    XrStructureType                 type;
    void*                           next;
    XrGoogleCloudAuthErrorANDROID   error;
} XrGoogleCloudAuthErrorResultANDROID;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain.
- error is the XrGoogleCloudAuthErrorANDROID detailing the cause of the error of an operation accessing Google Cloud.

If an operation fails due to Google Cloud Authentication, this struct **may** be chained to the operational result struct to provide more information about the error.

Valid Usage (Implicit)

- The XR_ANDROID_google_cloud_auth extension **must** be enabled prior to using XrGoogleCloudAuthErrorResultANDROID
- type **must** be XR_TYPE_GOOGLE_CLOUD_AUTH_ERROR_RESULT_ANDROID
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- error **must** be a valid XrGoogleCloudAuthErrorANDROID value

12.64.4. Example code

```
XrSession session; // previously initialized
XrInstance instance; // previously initialized
XrFutureEXT future {XR_NULL_FUTURE_EXT};

// The function pointers are previously initialized using
// xrGetInstanceProcAddr.
```

```
PFN_xrPollFutureEXT xrPollFutureEXT; // previously initialized
PFN_xrSetGoogleCloudAuthAsyncANDROID xrSetGoogleCloudAuthAsyncANDROID; // previously
initialized
PFN_xrSetGoogleCloudAuthCompleteANDROID xrSetGoogleCloudAuthCompleteANDROID; //
previously initialized

auto waitUntilReady = [&](XrFutureEXT future) {
  XrFuturePollInfoEXT pollInfo {
    .type = XR_TYPE_FUTURE_POLL_INFO_EXT,
    .future = future,
  };
  XrFuturePollResultEXT pollResult {
    .type = XR_TYPE_FUTURE_POLL_RESULT_EXT,
  };
  do {
    // sleep(1);
    xrPollFutureEXT(instance, &pollInfo, &pollResult);
  } while (pollResult.state != XR_FUTURE_STATE_READY_EXT);
};

// Set Google Cloud auth via API key.
XrGoogleCloudAuthInfoApiKeyANDROID authApiKey{
  .type = XR_TYPE_GOOGLE_CLOUD_AUTH_INFO_API_KEY_ANDROID,
  .apiKey = "MYAPIKEY",
};
XrResult result = xrSetGoogleCloudAuthAsyncANDROID(
    session, reinterpret_cast<XrGoogleCloudAuthInfoBaseHeaderANDROID*>(&authApiKey),
&future);

// Or, set Google Cloud auth via auth token:
XrGoogleCloudAuthInfoTokenANDROID authToken{
  .type = XR_TYPE_GOOGLE_CLOUD_AUTH_INFO_TOKEN_ANDROID,
  .authToken = "MYAUTHTOKEN",
};
result = xrSetGoogleCloudAuthAsyncANDROID(
    session, reinterpret_cast<XrGoogleCloudAuthInfoBaseHeaderANDROID*>(&authToken),
&future);

// Or, set Google Cloud auth via keyless auth:
XrGoogleCloudAuthInfoKeylessANDROID authKeyless {
  .type = XR_TYPE_GOOGLE_CLOUD_AUTH_INFO_KEYLESS_ANDROID,
};
result = xrSetGoogleCloudAuthAsyncANDROID(
    session, reinterpret_cast<XrGoogleCloudAuthInfoBaseHeaderANDROID*>(&authKeyless),
    &future);

// Check the result of the auth setup.
if (result == XR_ERROR_VALIDATION_FAILURE) {
```

```
  // The credentials were invalid.
} else if (result == XR_ERROR_KEYLESS_AUTH_NOT_SETUP_ANDROID) {
  // Keyless auth was not properly setup.
} else if (result == XR_SUCCESS) {
  waitUntilReady(future);
  XrFutureCompletionEXT completion{
    .type = XR_TYPE_FUTURE_COMPLETION_EXT,
  };
  xrSetGoogleCloudAuthCompleteANDROID(session, future, &completion);

  if (completion.futureResult == XR_SUCCESS) {
    // Credentials were successfully applied.
  } else if (completion.futureResult == XR_ERROR_KEYLESS_AUTH_FAILED_ANDROID) {
    // An error occurred when setting keyless auth credentials. OK to retry.
  }
}
```

12.64.5. New Commands

- xrSetGoogleCloudAuthAsyncANDROID
- xrSetGoogleCloudAuthCompleteANDROID

12.64.6. New Structures

- XrGoogleCloudAuthInfoApiKeyANDROID
- XrGoogleCloudAuthInfoBaseHeaderANDROID
- XrGoogleCloudAuthInfoKeylessANDROID
- XrGoogleCloudAuthInfoTokenANDROID
- Extending XrGoogleCloudAuthInfoBaseHeaderANDROID:
  - XrGoogleCloudAuthErrorResultANDROID

12.64.7. New Enums

- XrGoogleCloudAuthErrorANDROID

12.64.8. New Enum Constants

- `XR_ANDROID_GOOGLE_CLOUD_AUTH_EXTENSION_NAME`
- `XR_ANDROID_google_cloud_auth_SPEC_VERSION`
- Extending XrResult:
  - `XR_ERROR_KEYLESS_AUTH_FAILED_ANDROID`
  - `XR_ERROR_KEYLESS_AUTH_NOT_SETUP_ANDROID`

- Extending XrStructureType:
  - XR_TYPE_GOOGLE_CLOUD_AUTH_ERROR_RESULT_ANDROID
  - XR_TYPE_GOOGLE_CLOUD_AUTH_INFO_API_KEY_ANDROID
  - XR_TYPE_GOOGLE_CLOUD_AUTH_INFO_KEYLESS_ANDROID
  - XR_TYPE_GOOGLE_CLOUD_AUTH_INFO_TOKEN_ANDROID

12.64.9. Issues

- Revision 1, 2025-12-18 (Ben King)
  - Initial extension description.

12.65. XR_ANDROID_light_estimation

Name String

XR_ANDROID_light_estimation

Extension Type

Instance extension

Registered Extension Number

701

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Last Modified Date

2025-03-13

IP Status

No known IP claims.

Contributors

Jared Finder, Google
Cairn Overturf, Google
Spencer Quin, Google
Levana Chen, Google
Nihav Jain, Google

Salar Khan, Google
Scott Chung, Google

12.65.1. Overview

This extension allows the application to request data representing the lighting of the real-world environment around the headset. The application **can** use this information to light virtual objects under the same conditions as the real-world scene they're placed in.



Permissions

Android applications **must** have the android.permission.SCENE_UNDERSTANDING_COARSE permission listed in their manifest as this extension exposes lighting information about the environment. The android.permission.SCENE_UNDERSTANDING_COARSE permission is considered a dangerous permission. The application **must** request the permission at runtime to use these functions:

- xrCreateLightEstimatorANDROID

(protection level: dangerous)

12.65.2. Inspect system capability

The XrSystemLightEstimationPropertiesANDROID structure is defined as:

```
// Provided by XR_ANDROID_light_estimation
typedef struct XrSystemLightEstimationPropertiesANDROID {
    XrStructureType    type;
    void*              next;
    XrBool32           supportsLightEstimation;
} XrSystemLightEstimationPropertiesANDROID;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `supportsLightEstimation` is an `XrBool32`, indicating if the current system supports light estimation.

An application **can** inspect whether the system is capable of supporting light estimation by extending the XrSystemProperties with XrSystemLightEstimationPropertiesANDROID structure when calling

xrGetSystemProperties.

If and only if a runtime returns XR_FALSE for supportsLightEstimation, the runtime **must** return XR_ERROR_FEATURE_UNSUPPORTED from xrCreateLightEstimatorANDROID.

Valid Usage (Implicit)

- The XR_ANDROID_light_estimation extension **must** be enabled prior to using XrSystemLightEstimationPropertiesANDROID
- type **must** be XR_TYPE_SYSTEM_LIGHT_ESTIMATION_PROPERTIES_ANDROID
- next **must** be NULL or a valid pointer to the next structure in a structure chain

12.65.3. Create a light estimator handle

```
// Provided by XR_ANDROID_light_estimation
XR_DEFINE_HANDLE(XrLightEstimatorANDROID)
```

The light estimation data is generated by the runtime and shared with the application using the XrLightEstimatorANDROID handle.

This handle **can** be used to access light estimation information using other functions in this extension.

The xrCreateLightEstimatorANDROID function is defined as:

```
// Provided by XR_ANDROID_light_estimation
XrResult xrCreateLightEstimatorANDROID(
    XrSession                                   session,
    XrLightEstimatorCreateInfoANDROID*          createInfo,
    XrLightEstimatorANDROID*                    outHandle);
```

Parameter Descriptions

- session is the XrSession that creates the light estimator.
- createInfo is a pointer to an XrLightEstimatorCreateInfoANDROID structure containing parameters to be used to create the light estimator.
- outHandle is a pointer to a handle in which the created XrLightEstimatorANDROID is returned.

The application uses the xrCreateLightEstimatorANDROID function to create a light estimator.

- The runtime **must** return `XR_ERROR_FEATURE_UNSUPPORTED` if the system does not support light estimation.
- The runtime **must** return `XR_ERROR_PERMISSION_INSUFFICIENT` if the required permissions have not been granted to the calling app.

If an application wants to indicate to the runtime that it is done accessing light estimation data it **must** destroy the handle via xrDestroyLightEstimatorANDROID.

Valid Usage (Implicit)

- The `XR_ANDROID_light_estimation` extension **must** be enabled prior to calling xrCreateLightEstimatorANDROID
- `session` **must** be a valid XrSession handle
- `createInfo` **must** be a pointer to an XrLightEstimatorCreateInfoANDROID structure
- `outHandle` **must** be a pointer to an XrLightEstimatorANDROID handle

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_LIMIT_REACHED`
- `XR_ERROR_OUT_OF_MEMORY`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`

The XrLightEstimatorCreateInfoANDROID structure describes the information to create an XrLightEstimatorANDROID handle.

```
// Provided by XR_ANDROID_light_estimation
typedef struct XrLightEstimatorCreateInfoANDROID {
    XrStructureType    type;
    const void*        next;
} XrLightEstimatorCreateInfoANDROID;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.

Valid Usage (Implicit)

- The `XR_ANDROID_light_estimation` extension **must** be enabled prior to using XrLightEstimatorCreateInfoANDROID
- `type` **must** be `XR_TYPE_LIGHT_ESTIMATOR_CREATE_INFO_ANDROID`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

The xrDestroyLightEstimatorANDROID function is defined as:

```
// Provided by XR_ANDROID_light_estimation
XrResult xrDestroyLightEstimatorANDROID(
    XrLightEstimatorANDROID                     estimator);
```

Parameter Descriptions

- `estimator` is an XrLightEstimatorANDROID previously created by xrCreateLightEstimatorANDROID.

The xrDestroyLightEstimatorANDROID function releases the `estimator` and any underlying resources.

Valid Usage (Implicit)

- The `XR_ANDROID_light_estimation` extension **must** be enabled prior to calling xrDestroyLightEstimatorANDROID
- `estimator` **must** be a valid XrLightEstimatorANDROID handle

Thread Safety

- Access to `estimator`, and any child handles, **must** be externally synchronized

Return Codes

Success

- `XR_SUCCESS`

Failure

- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_RUNTIME_FAILURE`

12.65.4. Access light estimation data

The xrGetLightEstimateANDROID function is defined as:

```
// Provided by XR_ANDROID_light_estimation
XrResult xrGetLightEstimateANDROID(
    XrLightEstimatorANDROID                     estimator,
    const XrLightEstimateGetInfoANDROID*        input,
    XrLightEstimateANDROID*                     output);
```

Parameter Descriptions

- `estimator` is a handle to an XrLightEstimatorANDROID previously created with xrCreateLightEstimatorANDROID.
- `input` is a pointer to an XrLightEstimateGetInfoANDROID structure.
- `output` is a pointer to an XrLightEstimateANDROID structure.

Valid Usage (Implicit)

- The `XR_ANDROID_light_estimation` extension **must** be enabled prior to calling xrGetLightEstimateANDROID
- `estimator` **must** be a valid XrLightEstimatorANDROID handle
- `input` **must** be a pointer to a valid XrLightEstimateGetInfoANDROID structure
- `output` **must** be a pointer to an XrLightEstimateANDROID structure

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_OUT_OF_MEMORY`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_TIME_INVALID`
- `XR_ERROR_VALIDATION_FAILURE`

The XrLightEstimateGetInfoANDROID structure is defined as:

```
// Provided by XR_ANDROID_light_estimation
typedef struct XrLightEstimateGetInfoANDROID {
    XrStructureType    type;
    const void*        next;
    XrSpace            space;
    XrTime             time;
} XrLightEstimateGetInfoANDROID;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `space` is the XrSpace defining the reference space in which the returned light direction and spherical harmonics are expressed in.
- `time` is the `XrTime` that describes the time at which the application wishes to query the light estimation.

Valid Usage (Implicit)

- The `XR_ANDROID_light_estimation` extension **must** be enabled prior to using XrLightEstimateGetInfoANDROID
- `type` **must** be `XR_TYPE_LIGHT_ESTIMATE_GET_INFO_ANDROID`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `space` **must** be a valid XrSpace handle

The XrLightEstimateANDROID structure contains light estimation data:

```
// Provided by XR_ANDROID_light_estimation
typedef struct XrLightEstimateANDROID {
    XrStructureType                type;
    void*                          next;
    XrLightEstimateStateANDROID    state;
    XrTime                         lastUpdatedTime;
} XrLightEstimateANDROID;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. Valid structures include XrAmbientLightANDROID, XrSphericalHarmonicsANDROID, and XrDirectionalLightANDROID.
- `state` is the XrLightEstimateStateANDROID representing the state of the light estimate.
- `lastUpdatedTime` is the `XrTime` that represents when the estimate was last calculated.

For every light estimate struct if the `state` is `XR_LIGHT_ESTIMATE_STATE_INVALID_ANDROID`, then all other

fields in the struct are arbitrary, meaning their values are undefined.

To obtain light estimate information for ambient light, spherical harmonics and the primary directional light, applications **can** chain instances of the following structures, XrAmbientLightANDROID, XrSphericalHarmonicsANDROID, and XrDirectionalLightANDROID respectively to XrLightEstimateANDROID::`next`.

Valid Usage (Implicit)

- The `XR_ANDROID_light_estimation` extension **must** be enabled prior to using XrLightEstimateANDROID
- `type` **must** be `XR_TYPE_LIGHT_ESTIMATE_ANDROID`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain. See also: XrAmbientLightANDROID, XrDirectionalLightANDROID, XrSphericalHarmonicsANDROID
- `state` **must** be a valid XrLightEstimateStateANDROID value

The XrAmbientLightANDROID structure contains light estimation data about the ambient light in the scene.

```
// Provided by XR_ANDROID_light_estimation
typedef struct XrAmbientLightANDROID {
    XrStructureType                type;
    void*                          next;
    XrLightEstimateStateANDROID    state;
    XrVector3f                     intensity;
    XrVector3f                     colorCorrection;
} XrAmbientLightANDROID;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. Valid structures include XrSphericalHarmonicsANDROID and XrDirectionalLightANDROID.
- `state` is the XrLightEstimateStateANDROID representing the state of the light estimate.
- `intensity` is an XrVector3f representing the intensity of the ambient light. Each component of the vector corresponds to the red, green, and blue channels respectively.
- `colorCorrection` is an XrVector3f with values in gamma space. If rendering in gamma color space, multiply them component-wise against the final calculated color after rendering. If rendering in linear space, first convert the values to linear space by rising to the power 2.2, then multiply component-wise against the final calculated color after rendering.

The purpose of `colorCorrection` is to make a scene appear natural and blend with the real world.

Valid Usage (Implicit)

- The `XR_ANDROID_light_estimation` extension **must** be enabled prior to using XrAmbientLightANDROID
- `type` **must** be `XR_TYPE_AMBIENT_LIGHT_ANDROID`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `state` **must** be a valid XrLightEstimateStateANDROID value

The XrSphericalHarmonicsANDROID structure contains spherical harmonics representing the lighting of the scene.

```
// Provided by XR_ANDROID_light_estimation
typedef struct XrSphericalHarmonicsANDROID {
    XrStructureType                    type;
    void*                              next;
    XrLightEstimateStateANDROID        state;
    XrSphericalHarmonicsKindANDROID    kind;
    float                              coefficients[9][3];
} XrSphericalHarmonicsANDROID;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. Valid structures include XrAmbientLightANDROID and XrDirectionalLightANDROID.
- `state` is the XrLightEstimateStateANDROID representing the state of the light estimate.
- `kind` is the XrSphericalHarmonicsKindANDROID requested by the application.
- `coefficients` is a two-dimensional `float` array with dimensions 9 rows and 3 columns. The 3 columns corresponding to color channels red, green and blue respectively. Each channel has 9 spherical harmonic coefficients.

The `coefficients` are to be used in a special function which takes a direction as an input and outputs the light color coming from that direction. More detail available in R. Ramamoorthi and P. Hanrahan, "An efficient representation for irradiance environment maps," in *SIGGRAPH '01: Proceedings of the 28th annual conference on Computer graphics and interactive techniques*, ACM, Aug. 2001, pp. 497-500. doi:10.1145/383259.383317.

Valid Usage (Implicit)

- The `XR_ANDROID_light_estimation` extension **must** be enabled prior to using XrSphericalHarmonicsANDROID
- `type` **must** be `XR_TYPE_SPHERICAL_HARMONICS_ANDROID`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `state` **must** be a valid XrLightEstimateStateANDROID value
- `kind` **must** be a valid XrSphericalHarmonicsKindANDROID value

The XrSphericalHarmonicsKindANDROID enumeration identifies to the runtime, what type of spherical harmonics the application is requesting.

```
// Provided by XR_ANDROID_light_estimation
typedef enum XrSphericalHarmonicsKindANDROID {
    XR_SPHERICAL_HARMONICS_KIND_TOTAL_ANDROID = 0,
    XR_SPHERICAL_HARMONICS_KIND_AMBIENT_ANDROID = 1,
    XR_SPHERICAL_HARMONICS_KIND_MAX_ENUM_ANDROID = 0x7FFFFFFF
} XrSphericalHarmonicsKindANDROID;
```

The enumeration values have the following meanings:

Enum	Description
XR_SPHERICAL_HARMONICS_KIND_TOTAL_ANDROID	The spherical harmonic coefficients represent the environment light radiance function, including the contribution from the main light.
XR_SPHERICAL_HARMONICS_KIND_AMBIENT_ANDROID	The spherical harmonic coefficients represent the environment light radiance function, excluding the contribution from the main light.

The XrDirectionalLightANDROID structure contains light estimation data.

```
// Provided by XR_ANDROID_light_estimation
typedef struct XrDirectionalLightANDROID {
    XrStructureType                type;
    void*                          next;
    XrLightEstimateStateANDROID    state;
    XrVector3f                     intensity;
    XrVector3f                     direction;
} XrDirectionalLightANDROID;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. Valid structures include XrAmbientLightANDROID, XrSphericalHarmonicsANDROID, and XrDirectionalLightANDROID.
- state is the XrLightEstimateStateANDROID representing the state of the light estimate.
- intensity is an XrVector3f representing the intensity of the directional light. Each component of the vector corresponds to the red, green, and blue channels.
- direction is an XrVector3f representing the light direction.

Valid Usage (Implicit)

- The XR_ANDROID_light_estimation extension **must** be enabled prior to using XrDirectionalLightANDROID
- type **must** be XR_TYPE_DIRECTIONAL_LIGHT_ANDROID
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- state **must** be a valid XrLightEstimateStateANDROID value

The XrLightEstimateStateANDROID enumeration indicates the state of the light estimate returned by the runtime.

```
// Provided by XR_ANDROID_light_estimation
typedef enum XrLightEstimateStateANDROID {
    XR_LIGHT_ESTIMATE_STATE_VALID_ANDROID = 0,
    XR_LIGHT_ESTIMATE_STATE_INVALID_ANDROID = 1,
    XR_LIGHT_ESTIMATE_STATE_MAX_ENUM_ANDROID = 0x7FFFFFFF
} XrLightEstimateStateANDROID;
```

The enumeration values have the following meanings:

Enum	Description
XR_LIGHT_ESTIMATE_STATE_VALID_ANDROID	The light estimate is valid
XR_LIGHT_ESTIMATE_STATE_INVALID_ANDROID	The light estimate is invalid

12.65.5. Example code for light estimation

The following example code demonstrates how to get all possible light estimation quantities from the runtime

```
XrSession session;  // Created at app startup
XrSpace appSpace;   // Created previously.
PFN_xrCreateLightEstimatorANDROID xrCreateLightEstimatorANDROID; // Created previously.
PFN_xrDestroyLightEstimatorANDROID xrDestroyLightEstimatorANDROID; // Created previously.
PFN_xrGetLightEstimateANDROID xrGetLightEstimateANDROID; // Created previously.

XrLightEstimatorANDROID estimator;
XrLightEstimatorCreateInfoANDROID createInfo = {
    .type = XR_TYPE_LIGHT_ESTIMATOR_CREATE_INFO_ANDROID};
CHK_XR(xrCreateLightEstimatorANDROID(session, &createInfo, &estimator));

// Every frame
XrTime updateTime;  // Time used for the current frame's simulation update.

XrLightEstimateGetInfoANDROID info = {
    .type = XR_TYPE_LIGHT_ESTIMATE_GET_INFO_ANDROID,
    .space = appSpace,
    .time = updateTime,
};

XrDirectionalLightANDROID directionalLight = {
    .type = XR_TYPE_DIRECTIONAL_LIGHT_ANDROID,
};
```

```
XrSphericalHarmonicsANDROID sphericalHarmonics = {
    .type = XR_TYPE_SPHERICAL_HARMONICS_ANDROID,
    .next = &directionalLight,
};

// Querying both TOTAL or AMBIENT spherical harmonics in one call will result in an error
because chaining two structs with the same type is not allowed
bool useTotalSH; // initialized previously
if (useTotalSH) {
  sphericalHarmonics.kind = XR_SPHERICAL_HARMONICS_KIND_TOTAL_ANDROID;
} else {
  sphericalHarmonics.kind = XR_SPHERICAL_HARMONICS_KIND_AMBIENT_ANDROID;
}

XrAmbientLightANDROID ambientLight = {
    .type = XR_TYPE_AMBIENT_LIGHT_ANDROID,
    .next = &sphericalHarmonics,
};

XrLightEstimateANDROID estimate = {
    .type = XR_TYPE_LIGHT_ESTIMATE_ANDROID,
    .next = &ambientLight,
};

XrResult result = xrGetLightEstimateANDROID(estimator, &info, &estimate);
if (result == XR_SUCCESS &&
    estimate.state == XR_LIGHT_ESTIMATE_STATE_VALID_ANDROID) {
  // use directionalLight, totalSh, ambientSh, ambientLight if each
  // struct has a valid state field
}

// When you want to disable light estimation
CHK_XR(xrDestroyLightEstimatorANDROID(estimator));
```

12.65.6. New Object Types

- XrLightEstimatorANDROID

12.65.7. New Commands

- xrCreateLightEstimatorANDROID
- xrDestroyLightEstimatorANDROID
- xrGetLightEstimateANDROID

12.65.8. New Structures

- XrLightEstimateANDROID
- XrLightEstimateGetInfoANDROID
- XrLightEstimatorCreateInfoANDROID
- Extending XrLightEstimateANDROID:
  - XrAmbientLightANDROID
  - XrDirectionalLightANDROID
  - XrSphericalHarmonicsANDROID
- Extending XrSystemProperties:
  - XrSystemLightEstimationPropertiesANDROID

12.65.9. New Enums

- XrLightEstimateStateANDROID
- XrSphericalHarmonicsKindANDROID

12.65.10. New Enum Constants

- `XR_ANDROID_LIGHT_ESTIMATION_EXTENSION_NAME`
- `XR_ANDROID_light_estimation_SPEC_VERSION`
- Extending XrObjectType:
  - `XR_OBJECT_TYPE_LIGHT_ESTIMATOR_ANDROID`
- Extending XrStructureType:
  - `XR_TYPE_AMBIENT_LIGHT_ANDROID`
  - `XR_TYPE_DIRECTIONAL_LIGHT_ANDROID`
  - `XR_TYPE_LIGHT_ESTIMATE_ANDROID`
  - `XR_TYPE_LIGHT_ESTIMATE_GET_INFO_ANDROID`
  - `XR_TYPE_LIGHT_ESTIMATOR_CREATE_INFO_ANDROID`
  - `XR_TYPE_SPHERICAL_HARMONICS_ANDROID`
  - `XR_TYPE_SYSTEM_LIGHT_ESTIMATION_PROPERTIES_ANDROID`

12.65.11. Issues

12.65.12. Version History

- Revision 1, 2024-09-16 (Cairn Overturf)

- Initial extension description

12.66. XR_ANDROID_mouse_interaction

Name String

`XR_ANDROID_mouse_interaction`

Extension Type

Instance extension

Registered Extension Number

705

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Last Modified Date

2024-09-06

IP Status

No known IP claims.

Contributors

Sharayu Shenoy, Google
Chiara Coetzee, Google
Levana Chen, Google
Spencer Quin, Google

12.66.1. Overview

This extension provides an `XrPath` for getting mouse input and defines one commonly used action pose for user mouse profiles including both mouse devices and trackpad devices.

This extension also introduces a new interaction profile specifically designed for mouse devices to input through the OpenXR action system.

12.66.2. Action pose for mouse interactions

The following action pose (i.e. "aim") enables a 3D pointer ray, whether the tracking inputs are provided by a mouse device or a trackpad device.

The runtime **must** support the following action subpath on all interaction profiles that are valid for the user path of */user/mouse*, including those interaction profiles enabled through extensions.

- *.../input/aim/pose*

Aim pose

The *.../input/aim/pose* is designed for interacting with objects through a mouse pointer in 3D space. For example, using a virtual laser pointer to aim at a virtual button on the wall is an interaction suited to the "aim" pose.

This is the same "aim" pose defined in Standard pose identifiers. Every tracked controller profile already supports this pose.

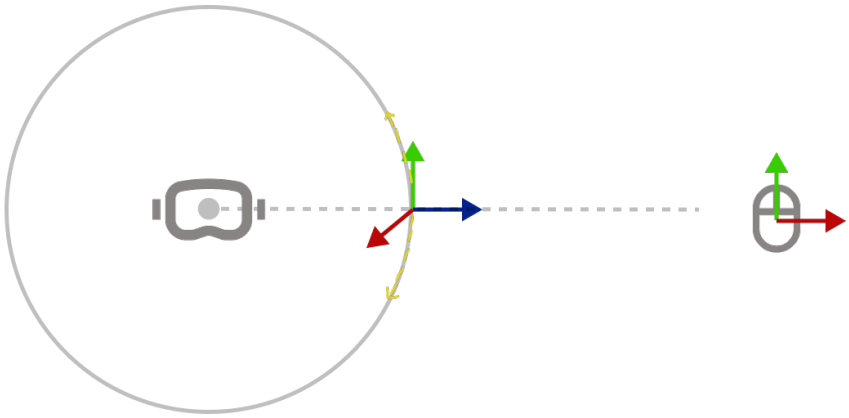


Figure 26. Example aim pose.

Position

The position of the "aim" pose is typically the user's head, that is, the same position of the head when a mouse movement is detected. The aim pose for the mouse is calculated when there is a mouse movement. The runtime **should** keep the last known aim position until the next mouse movement.

Orientation

The orientation of the "aim" pose **may** be used to render a 3D pointer ray to interact with a virtual object, for example, clicking a menu button on the wall.

The aim pose for the mouse is calculated when there is a mouse movement. The runtime **should** keep the last known aim orientation until the next mouse movement.

The -Z direction is the forward direction of the aiming gesture, that is, where the aiming ray is pointing at.

The relative X,Y movement of the mouse is used to compute the relative movement of the mouse along a sphere around the user's head.

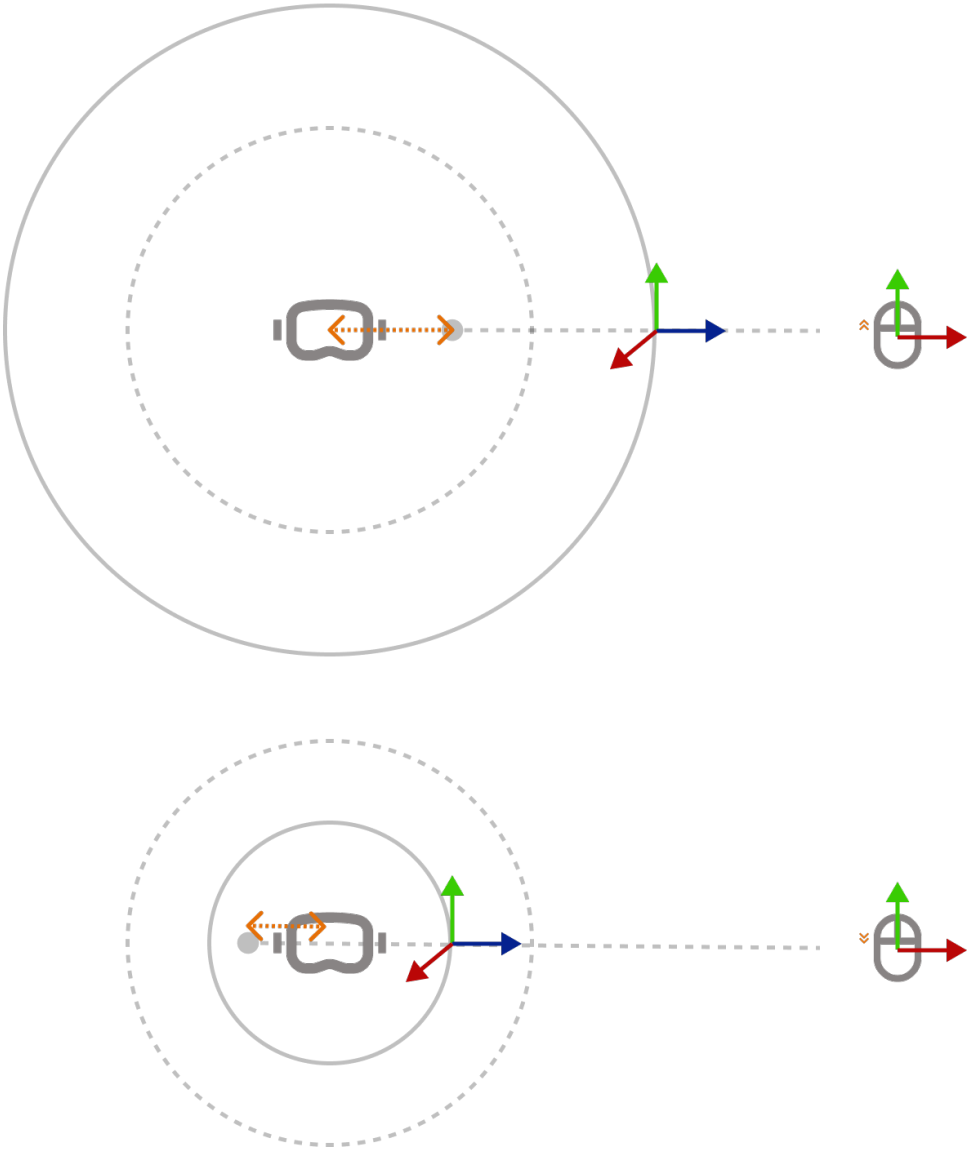


Figure 27. Example aim pose.

The runtime **should** support depth movement using primary click and scroll. When an action to move in depth is in process, the position is offset from the head position in positive or negative Z-direction along the ray depending on the positive or negative scroll value. The runtime **should** stabilize the depth movement within the sphere around the user's head.

12.66.3. The interaction profile for mouse devices

The mouse interaction profile is designed for runtimes which provide mouse inputs using mouse devices or trackpad devices with buttons and scroll. This allows mouse devices and trackpad devices to provide commonly used clicks, scroll and action pose to the OpenXR action system.

Interaction profile path:

- */interaction_profiles/android/mouse_interaction_android*

Valid for top level user path:

- */user/mouse*

Supported component paths:

- *.../input/aim/pose*
- *.../input/select/click*
- *.../input/secondary_android/click*
- *.../input/tertiary_android/click*
- *.../input/scroll_android/value*

This interaction profile supports the above action pose, as well as the following two groups of action inputs.

Click action

This interaction profile supports *.../input/select/click*, *.../input/secondary_android/click* and /input/tertiary_android/click actions.

The *.../input/select/click* is a boolean input, where the value `XR_TRUE` indicates that the primary button on the mouse or trackpad is pressed.

The *.../input/secondary_android/click* is a boolean input, where the value `XR_TRUE` indicates that the secondary button on the mouse or trackpad is pressed.

The *.../input/tertiary_android/click* is a boolean input, where the value `XR_TRUE` indicates that the mouse scroll or the tertiary button is pressed.

Scroll action

This interaction profile supports *.../input/scroll_android/value* actions.

The *.../input/scroll_android/value* is a 2D input component varying from -1 to 1 to map to scroll left and right using the X component and scroll down and up using the Y component.

12.66.4. New Enum Constants

- `XR_ANDROID_MOUSE_INTERACTION_EXTENSION_NAME`
- `XR_ANDROID_mouse_interaction_SPEC_VERSION`

12.66.5. Issues

12.66.6. Version History

- Revision 1, 2024-08-29 (Levana Chen)
  - Initial extension description

12.67. XR_ANDROID_passthrough_camera_state

Name String

`XR_ANDROID_passthrough_camera_state`

Extension Type

Instance extension

Registered Extension Number

461

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Last Modified Date

2025-07-23

Contributors

Spencer Quin, Google
Jared Finder, Google
Kevin Moule, Google
Nihav Jain, Google

12.67.1. Overview

Passthrough cameras **may** take time to start up and are not immediately available. This extension lets applications know the current state of the passthrough camera.

12.67.2. Inspect system capability

The XrSystemPassthroughCameraStatePropertiesANDROID structure is defined as:

```
// Provided by XR_ANDROID_passthrough_camera_state
typedef struct XrSystemPassthroughCameraStatePropertiesANDROID {
    XrStructureType    type;
    void*              next;
    XrBool32           supportsPassthroughCameraState;
} XrSystemPassthroughCameraStatePropertiesANDROID;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension. For more details about the structure chain, see the structure being extended (XrSystemProperties).
- `supportsPassthroughCameraState` is an `XrBool32` indicating if current system supports passthrough camera state.

Applications inspect whether the system is capable of querying the passthrough camera state by extending the XrSystemProperties with an XrSystemPassthroughCameraStatePropertiesANDROID structure when calling xrGetSystemProperties.

If and only if a runtime returns `XR_FALSE` for `supportsPassthroughCameraState`, the runtime **must** return `XR_ERROR_FEATURE_UNSUPPORTED` from xrGetPassthroughCameraStateANDROID.

Valid Usage (Implicit)

- The `XR_ANDROID_passthrough_camera_state` extension **must** be enabled prior to using XrSystemPassthroughCameraStatePropertiesANDROID
- `type` **must** be `XR_TYPE_SYSTEM_PASSTHROUGH_CAMERA_STATE_PROPERTIES_ANDROID`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

12.67.3. Get the current passthrough camera state

```
// Provided by XR_ANDROID_passthrough_camera_state
XrResult xrGetPassthroughCameraStateANDROID(
    XrSession                                   session,
    const XrPassthroughCameraStateGetInfoANDROID* getInfo,
    XrPassthroughCameraStateANDROID*            cameraStateOutput);
```

Parameter Descriptions

- `session` is an XrSession handle previously created with xrCreateSession.
- `getInfo` is any info that will affect how camera state is determined.
- `cameraStateOutput` is the current state of the camera.

xrGetPassthroughCameraStateANDROID retrieves the current state of the passthrough camera.

Valid Usage (Implicit)

- The `XR_ANDROID_passthrough_camera_state` extension **must** be enabled prior to calling xrGetPassthroughCameraStateANDROID
- `session` **must** be a valid XrSession handle
- `getInfo` **must** be a pointer to a valid XrPassthroughCameraStateGetInfoANDROID structure
- `cameraStateOutput` **must** be a pointer to an XrPassthroughCameraStateANDROID value

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_VALIDATION_FAILURE`

The XrPassthroughCameraStateGetInfoANDROID structure is an input struct which specifies the camera state request parameters.

```
// Provided by XR_ANDROID_passthrough_camera_state
typedef struct XrPassthroughCameraStateGetInfoANDROID {
    XrStructureType    type;
    const void*        next;
} XrPassthroughCameraStateGetInfoANDROID;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.

Valid Usage (Implicit)

- The `XR_ANDROID_passthrough_camera_state` extension **must** be enabled prior to using XrPassthroughCameraStateGetInfoANDROID
- `type` **must** be `XR_TYPE_PASSTHROUGH_CAMERA_STATE_GET_INFO_ANDROID`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

The XrPassthroughCameraStateANDROID enumeration identifies the different possible states of the passthrough camera.

```
// Provided by XR_ANDROID_passthrough_camera_state
typedef enum XrPassthroughCameraStateANDROID {
    XR_PASSTHROUGH_CAMERA_STATE_DISABLED_ANDROID = 0,
    XR_PASSTHROUGH_CAMERA_STATE_INITIALIZING_ANDROID = 1,
    XR_PASSTHROUGH_CAMERA_STATE_READY_ANDROID = 2,
    XR_PASSTHROUGH_CAMERA_STATE_ERROR_ANDROID = 3,
    XR_PASSTHROUGH_CAMERA_STATE_MAX_ENUM_ANDROID = 0x7FFFFFFF
} XrPassthroughCameraStateANDROID;
```

Enum	Description
`XR_PASSTHROUGH_CAMERA_STATE_DISABLED_ANDROID`	The camera has been disabled by an app, the system or the user.
`XR_PASSTHROUGH_CAMERA_STATE_INITIALIZING_ANDROID`	The camera is still coming online and not yet ready to use. The runtime **may** render a black background where the passthrough video is supposed to appear.
`XR_PASSTHROUGH_CAMERA_STATE_READY_ANDROID`	The camera is ready to use.

Enum	Description
XR_PASSTHROUGH_CAMERA_STATE_ERROR_ANDROID	The camera is in an unrecoverable error state.

12.67.4. New Commands

- xrGetPassthroughCameraStateANDROID

12.67.5. New Structures

- XrPassthroughCameraStateGetInfoANDROID
- Extending XrSystemProperties:
  - XrSystemPassthroughCameraStatePropertiesANDROID

12.67.6. New Enums

- XrPassthroughCameraStateANDROID

12.67.7. New Enum Constants

- XR_ANDROID_PASSTHROUGH_CAMERA_STATE_EXTENSION_NAME
- XR_ANDROID_passthrough_camera_state_SPEC_VERSION
- Extending XrStructureType:
  - XR_TYPE_PASSTHROUGH_CAMERA_STATE_GET_INFO_ANDROID
  - XR_TYPE_SYSTEM_PASSTHROUGH_CAMERA_STATE_PROPERTIES_ANDROID

12.67.8. Issues

12.67.9. Version History

- Revision 1, 2025-07-23 (Spencer Quin)
  - Initial version.

12.68. XR_ANDROID_performance_metrics

Name String

XR_ANDROID_performance_metrics

Extension Type

Instance extension

Registered Extension Number

466

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Last Modified Date

2024-09-06

IP Status

No known IP claims.

Contributors

Dmitry Kotov, Google
Levana Chen, Google
Jared Finder, Google
Spencer Quin, Google

12.68.1. Overview

This extension provides APIs to enumerate and query various performance metrics counters of the current XR device, compositor and XR application.

Its intended use is for developers to perform performance analysis and do targeted optimization to the XR application using the performance metrics counters being collected. The application **should** not change its behavior based on the counter reads.

The performance metrics counters are organized into predefined `XrPath` values, under the root path */perfmetrics_android*. Applications query the available counters through xrEnumeratePerformanceMetricsCounterPathsANDROID.

The following performance metrics counter paths **may** be provided on Android devices:

- */perfmetrics_android/app/cpu_frametime* (milliseconds, float) - time spent by the application to process a frame, averaged over a sliding window.
- */perfmetrics_android/app/gpu_frametime* (milliseconds, float) - time spent to complete the GPU work submitted by the application per frame, averaged over a sliding window.
    - A high wait time **may** mean that the GPU was busy with other tasks, not necessarily that this client is doing too much GPU work.

    - The GPU wait time **may** be zero if rendering was already complete when checked by the compositor.
- */perfmetrics_android/app/cpu_utilization* (percentage, float) - total app CPU utilization rate averaged over time.
    - It **may** be higher than 100% on multi-core processors.
- */perfmetrics_android/app/gpu_utilization* (percentage) - app total GPU utilization rate averaged over time.
- */perfmetrics_android/app/motion_to_photon_latency* (milliseconds, float) - time spent from user-initiated motion event to corresponding physical image update on the display.
- */perfmetrics_android/compositor/cpu_frametime* (milliseconds, float) - time spent by the compositor to process a frame, averaged over a sliding window.
- */perfmetrics_android/compositor/gpu_frametime* (milliseconds, float) - time spent to complete the GPU work submitted by the compositor per frame, averaged over a sliding window.
- */perfmetrics_android/compositor/dropped_frame_count* (integer) - total number of dropped frames from all apps.
- */perfmetrics_android/compositor/frames_per_second* (float) - number of compositor frames drawn on device per second.
- */perfmetrics_android/device/cpu_utilization_average* (percentage, float) - device CPU utilization rate averaged across all cores and averaged over time.
- */perfmetrics_android/device/cpu_utilization_worst* (percentage, float) - device CPU utilization rate of worst performing core averaged over time.
- */perfmetrics_android/device/cpu0_utilization* through */perfmetrics_android/device/cpuX_utilization* (percentage, float, X is the number of CPU cores minus one) - device CPU utilization rate per CPU core averaged over time.
- */perfmetrics_android/device/cpu_frequency* (MHz, float) - device CPU frequency averaged across all cores and averaged over time.
- */perfmetrics_android/device/gpu_utilization* (percentage, float) - device GPU utilization rate averaged over time.

To enable the performance metrics system for a session, applications call xrSetPerformanceMetricsStateANDROID.

Applications use xrQueryPerformanceMetricsCounterANDROID to query a performance metrics counter on a session that has the performance metrics system enabled, or xrGetPerformanceMetricsStateANDROID to query if the performance metrics system is enabled.

Note that the measurement intervals of individual performance metrics counters are defined by the OpenXR runtime. The application **should** not make assumptions or change its behavior at runtime by measuring them.

12.68.2. Enumerate supported metrics counter paths

The xrEnumeratePerformanceMetricsCounterPathsANDROID function is defined as:

```
// Provided by XR_ANDROID_performance_metrics
XrResult xrEnumeratePerformanceMetricsCounterPathsANDROID(
    XrInstance                                  instance,
    uint32_t                                    counterPathCapacityInput,
    uint32_t*                                   counterPathCountOutput,
    XrPath*                                     counterPaths);
```

Parameter Descriptions

- `instance` is an XrInstance handle previously created with xrCreateInstance.
- `counterPathCapacityInput` is the capacity of the `counterPaths` array, or 0 to indicate a request to retrieve the required capacity.
- `counterPathCountOutput` is filled in by the runtime with the count of `counterPaths` written or the required capacity in the case that `counterPathCapacityInput` is insufficient.
- `counterPaths` is an array of `XrPath` filled in by the runtime which contains all the available performance metrics counters, but **can** be `NULL` if `counterPathCapacityInput` is `0`.
- See the Buffer Size Parameters section for a detailed description of retrieving the required `counterPaths` size.

This function enumerates all performance metrics counter paths that are supported by the runtime.

Valid Usage (Implicit)

- The `XR_ANDROID_performance_metrics` extension **must** be enabled prior to calling xrEnumeratePerformanceMetricsCounterPathsANDROID
- `instance` **must** be a valid XrInstance handle
- `counterPathCountOutput` **must** be a pointer to a `uint32_t` value
- If `counterPathCapacityInput` is not `0`, `counterPaths` **must** be a pointer to an array of `counterPathCapacityInput` `XrPath` values

Return Codes

Success

- XR_SUCCESS

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SIZE_INSUFFICIENT
- XR_ERROR_VALIDATION_FAILURE

12.68.3. Set and query performance metrics system state

The xrSetPerformanceMetricsStateANDROID function is defined as:

```
// Provided by XR_ANDROID_performance_metrics
XrResult xrSetPerformanceMetricsStateANDROID(
    XrSession                                   session,
    const XrPerformanceMetricsStateANDROID*     state);
```

Parameter Descriptions

- session is an XrSession handle previously created with xrCreateSession.
- state is a pointer to an XrPerformanceMetricsStateANDROID structure.

The xrSetPerformanceMetricsStateANDROID function enables or disables the performance metrics system.

Valid Usage (Implicit)

- The XR_ANDROID_performance_metrics extension **must** be enabled prior to calling xrSetPerformanceMetricsStateANDROID
- session **must** be a valid XrSession handle
- state **must** be a pointer to a valid XrPerformanceMetricsStateANDROID structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The XrPerformanceMetricsStateANDROID structure is defined as:

```
// Provided by XR_ANDROID_performance_metrics
typedef struct XrPerformanceMetricsStateANDROID {
    XrStructureType    type;
    void*              next;
    XrBool32           enabled;
} XrPerformanceMetricsStateANDROID;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- enabled is set to XR_TRUE to indicate the performance metrics system is enabled, XR_FALSE otherwise, when getting state. When setting state, set to XR_TRUE to enable the performance metrics system and XR_FALSE to disable it.

XrPerformanceMetricsStateANDROID is provided as input when calling xrSetPerformanceMetricsStateANDROID to enable or disable the performance metrics system.

XrPerformanceMetricsStateANDROID is populated as an output parameter when calling xrGetPerformanceMetricsStateANDROID to query if the performance metrics system is enabled.

Valid Usage (Implicit)

- The XR_ANDROID_performance_metrics extension **must** be enabled prior to using XrPerformanceMetricsStateANDROID
- `type` **must** be `XR_TYPE_PERFORMANCE_METRICS_STATE_ANDROID`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

The xrGetPerformanceMetricsStateANDROID function is defined as:

```
// Provided by XR_ANDROID_performance_metrics
XrResult xrGetPerformanceMetricsStateANDROID(
    XrSession                                   session,
    XrPerformanceMetricsStateANDROID*           state);
```

Parameter Descriptions

- `session` is an XrSession handle previously created with xrCreateSession.
- `state` is a pointer to an XrPerformanceMetricsStateANDROID structure.

The xrGetPerformanceMetricsStateANDROID function gets the current state of the performance metrics system.

Valid Usage (Implicit)

- The XR_ANDROID_performance_metrics extension **must** be enabled prior to calling xrGetPerformanceMetricsStateANDROID
- `session` **must** be a valid XrSession handle
- `state` **must** be a pointer to an XrPerformanceMetricsStateANDROID structure

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_VALIDATION_FAILURE`

12.68.4. Query performance counters

The xrQueryPerformanceMetricsCounterANDROID function is defined as:

```
// Provided by XR_ANDROID_performance_metrics
XrResult xrQueryPerformanceMetricsCounterANDROID(
    XrSession                                   session,
    XrPath                                      counterPath,
    XrPerformanceMetricsCounterANDROID*         counter);
```

Parameter Descriptions

- `session` is an XrSession handle previously created with xrCreateSession.
- `counterPath` is a valid performance metrics counter path.
- `counter` is a pointer to an XrPerformanceMetricsCounterANDROID structure.

The xrQueryPerformanceMetricsCounterANDROID function queries a performance metrics counter.

The application **must** enable the performance metrics system (by calling xrSetPerformanceMetricsStateANDROID) before querying metrics using xrQueryPerformanceMetricsCounterANDROID. If the performance metrics system has not been enabled before calling xrQueryPerformanceMetricsCounterANDROID, the runtime **must** return `XR_ERROR_VALIDATION_FAILURE`.

If `counterPath` is not in the list returned by xrEnumeratePerformanceMetricsCounterPathsANDROID,

the runtime **must** return XR_ERROR_PATH_UNSUPPORTED.

Valid Usage (Implicit)

- The XR_ANDROID_performance_metrics extension **must** be enabled prior to calling xrQueryPerformanceMetricsCounterANDROID
- session **must** be a valid XrSession handle
- counter **must** be a pointer to an XrPerformanceMetricsCounterANDROID structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_PATH_INVALID
- XR_ERROR_PATH_UNSUPPORTED
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The XrPerformanceMetricsCounterANDROID structure is defined as:

```
// Provided by XR_ANDROID_performance_metrics
typedef struct XrPerformanceMetricsCounterANDROID {
    XrStructureType                           type;
    void*                                     next;
    XrPerformanceMetricsCounterFlagsANDROID   counterFlags;
    XrPerformanceMetricsCounterUnitANDROID    counterUnit;
    uint32_t                                  uintValue;
    float                                     floatValue;
} XrPerformanceMetricsCounterANDROID;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `counterFlags` is a bitmask of XrPerformanceMetricsCounterFlagBitsANDROID describing the validity of value members.
- `counterUnit` is an enum of XrPerformanceMetricsCounterUnitANDROID describing the measurement unit.
- `uintValue` is the counter value in `uint32_t` format. It is valid if `counterFlags` contains `XR_PERFORMANCE_METRICS_COUNTER_UINT_VALUE_VALID_BIT_ANDROID`.
- `floatValue` is the counter value in `float` format. It is valid if `counterFlags` contains `XR_PERFORMANCE_METRICS_COUNTER_FLOAT_VALUE_VALID_BIT_ANDROID`.

XrPerformanceMetricsCounterANDROID is populated by calling xrQueryPerformanceMetricsCounterANDROID to query real-time performance metrics counter information.

Valid Usage (Implicit)

- The `XR_ANDROID_performance_metrics` extension **must** be enabled prior to using XrPerformanceMetricsCounterANDROID
- `type` **must** be `XR_TYPE_PERFORMANCE_METRICS_COUNTER_ANDROID`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

```
// Provided by XR_ANDROID_performance_metrics
typedef XrFlags64 XrPerformanceMetricsCounterFlagsANDROID;
```

```
// Provided by XR_ANDROID_performance_metrics
// Flag bits for XrPerformanceMetricsCounterFlagsANDROID
static const XrPerformanceMetricsCounterFlagsANDROID
XR_PERFORMANCE_METRICS_COUNTER_ANY_VALUE_VALID_BIT_ANDROID = 0x00000001;
static const XrPerformanceMetricsCounterFlagsANDROID
XR_PERFORMANCE_METRICS_COUNTER_UINT_VALUE_VALID_BIT_ANDROID = 0x00000002;
static const XrPerformanceMetricsCounterFlagsANDROID
XR_PERFORMANCE_METRICS_COUNTER_FLOAT_VALUE_VALID_BIT_ANDROID = 0x00000004;
```

Flag Descriptions

- XR_PERFORMANCE_METRICS_COUNTER_ANY_VALUE_VALID_BIT_ANDROID — Indicates any of the values in XrPerformanceMetricsCounterANDROID is valid.
- XR_PERFORMANCE_METRICS_COUNTER_UINT_VALUE_VALID_BIT_ANDROID — Indicates the uintValue in XrPerformanceMetricsCounterANDROID is valid.
- XR_PERFORMANCE_METRICS_COUNTER_FLOAT_VALUE_VALID_BIT_ANDROID — Indicates the floatValue in XrPerformanceMetricsCounterANDROID is valid.

```
// Provided by XR_ANDROID_performance_metrics
typedef enum XrPerformanceMetricsCounterUnitANDROID {
    XR_PERFORMANCE_METRICS_COUNTER_UNIT_GENERIC_ANDROID = 0,
    XR_PERFORMANCE_METRICS_COUNTER_UNIT_PERCENTAGE_ANDROID = 1,
    XR_PERFORMANCE_METRICS_COUNTER_UNIT_MILLISECONDS_ANDROID = 2,
    XR_PERFORMANCE_METRICS_COUNTER_UNIT_BYTES_ANDROID = 3,
    XR_PERFORMANCE_METRICS_COUNTER_UNIT_HERTZ_ANDROID = 4,
    XR_PERFORMANCE_METRICS_COUNTER_UNIT_MAX_ENUM_ANDROID = 0x7FFFFFFF
} XrPerformanceMetricsCounterUnitANDROID;
```

Enum	Description
XR_PERFORMANCE_METRICS_COUNTER_UNIT_GENERIC_ANDROID	The performance counter unit is generic (unspecified).
XR_PERFORMANCE_METRICS_COUNTER_UNIT_PERCENTAGE_ANDROID	The performance counter unit is percentage (%).
XR_PERFORMANCE_METRICS_COUNTER_UNIT_MILLISECONDS_ANDROID	The performance counter unit is millisecond.
XR_PERFORMANCE_METRICS_COUNTER_UNIT_BYTES_ANDROID	The performance counter unit is byte.
XR_PERFORMANCE_METRICS_COUNTER_UNIT_HERTZ_ANDROID	The performance counter unit is hertz (Hz).

12.68.5. New Commands

- xrEnumeratePerformanceMetricsCounterPathsANDROID
- xrGetPerformanceMetricsStateANDROID
- xrQueryPerformanceMetricsCounterANDROID
- xrSetPerformanceMetricsStateANDROID

12.68.6. New Structures

- XrPerformanceMetricsCounterANDROID
- XrPerformanceMetricsStateANDROID

12.68.7. New Enums

- XrPerformanceMetricsCounterFlagBitsANDROID
- XrPerformanceMetricsCounterUnitANDROID

12.68.8. New Bitmasks

- XrPerformanceMetricsCounterFlagsANDROID

12.68.9. New Enum Constants

- `XR_ANDROID_PERFORMANCE_METRICS_EXTENSION_NAME`
- `XR_ANDROID_performance_metrics_SPEC_VERSION`
- Extending XrStructureType:
  - `XR_TYPE_PERFORMANCE_METRICS_COUNTER_ANDROID`
  - `XR_TYPE_PERFORMANCE_METRICS_STATE_ANDROID`

12.68.10. Issues

12.68.11. Version History

- Revision 1, 2024-08-28 (Levana Chen)
  - Initial extension description

12.69. XR_ANDROID_recommended_resolution

Name String

`XR_ANDROID_recommended_resolution`

Extension Type

Instance extension

Registered Extension Number

462

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

IP Status

No known IP claims.

Contributors

Trevor Debrechtabel, Google
Spencer Quin, Google
Lachlan Ford, Google
Vasiliy Baranov, Google

12.69.1. Overview

This extension allows the runtime to notify the application when the recommended resolution changes, based on current system performance, device thermals, or other factors.

This extension modifies the specification in the following way:

- The runtime **may** return non-identical buffer contents from the xrEnumerateViewConfigurationViews enumeration for the given `systemId` and `viewConfigurationType` for the lifetime of the instance.

The XrEventDataRecommendedResolutionChangedANDROID structure is defined as:

```
// Provided by XR_ANDROID_recommended_resolution
typedef struct XrEventDataRecommendedResolutionChangedANDROID {
    XrStructureType    type;
    const void*        next;
} XrEventDataRecommendedResolutionChangedANDROID;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.

Receiving the XrEventDataRecommendedResolutionChangedANDROID event structure indicates that the recommended resolution has changed. The application **should** query the runtime for the new recommended resolution using xrEnumerateViewConfigurationViews.

Valid Usage (Implicit)

- The `XR_ANDROID_recommended_resolution` extension **must** be enabled prior to using XrEventDataRecommendedResolutionChangedANDROID
- `type` **must** be `XR_TYPE_EVENT_DATA_RECOMMENDED_RESOLUTION_CHANGED_ANDROID`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

12.69.2. Example code

The following example code demonstrates how to listen for recommended resolution change events.

```
// Created at app startup time.
XrInstance instance;
XrSystemId systemId; // Previously initialized.
uint32_t viewCountOutput; // Previously initialized.

// View configuration type the application uses.
XrViewConfigurationType viewConfigType;

// Poll events for recommended resolution changes.
XrEventDataBuffer event = {
  .type = XR_TYPE_EVENT_DATA_BUFFER,
};
XrResult result = xrPollEvent(instance, &event);
if (result == XR_SUCCESS) {
  switch (event.type) {
    case XR_TYPE_EVENT_DATA_RECOMMENDED_RESOLUTION_CHANGED_ANDROID: {
        uint32_t viewCapacityInput = viewCountOutput;
        std::vector<XrViewConfigurationView> views(viewCapacityInput);
        result = xrEnumerateViewConfigurationViews(instance, systemId,
          viewConfigType, viewCapacityInput, &viewCountOutput, views.data());
        if(!XR_SUCCEEDED(result)) {
          // Handle error
        }

        // New recommended resolution is found in
        // views.recommendedImageRectWidth and views.recommendedImageRectHeight
        // Change the resolution for the viewConfigType
      break;
    }
    default:
      break;
  }
}
```

12.69.3. Version History

- Revision 1, 2025-04-04 (Kenny Vercaemer)
  - Initial extension description

12.70. XR_ANDROID_scene_meshing

Name String

`XR_ANDROID_scene_meshing`

Extension Type

Instance extension

Registered Extension Number

719

Revision

3

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Last Modified Date

2025-05-15

IP Status

No known IP claims.

Contributors

Spencer Quin, Google
Jared Finder, Google
Antonio Fontan, Google
Cairn Overturf, Google
Nihav Jain, Google
Salar Khan, Google
Sebastian Klose, Google
Jürgen Sturm, Google
Vinny DaSilva, Google
Ricardo Campbell, Google

12.70.1. Overview

This extension is intended to provide meshes that approximately represent the physical objects in the user's environment.

Applications **can** use this extension to visualize the scene and to let virtual objects interact with physical objects such as collisions.

The scene mesh data is generated by the runtime and shared with the application using XrSceneMeshingTrackerANDROID and XrSceneMeshSnapshotANDROID. Applications **can** access the values in scene submesh information through XrSceneSubmeshStateANDROID returned from xrGetAllSubmeshStatesANDROID. Applications **can** access the values in scene submesh data through

XrSceneSubmeshDataANDROID returned from xrGetSubmeshDataANDROID.

Scene meshing data is sensitive personal information and is closely linked to personal privacy and integrity. It is strongly recommended that applications that store or transfer scene meshing data always ask the user for active and specific acceptance to do so.

Permissions

Android applications **must** have the android.permission.SCENE_UNDERSTANDING_FINE permission listed in their manifest. The android.permission.SCENE_UNDERSTANDING_FINE permission is considered a sensitive permission as it allows the runtime to study the user environment.

The application **must** request the permission at runtime to use these functions:



- xrCreateSceneMeshingTrackerANDROID
- xrDestroySceneMeshingTrackerANDROID
- xrCreateSceneMeshSnapshotANDROID
- xrDestroySceneMeshSnapshotANDROID
- xrGetAllSubmeshStatesANDROID
- xrGetSubmeshDataANDROID

(protection level: dangerous)

12.70.2. Inspect system capability

The XrSystemSceneMeshingPropertiesANDROID structure is defined as:

```
// Provided by XR_ANDROID_scene_meshing
typedef struct XrSystemSceneMeshingPropertiesANDROID {
    XrStructureType    type;
    void*              next;
    XrBool32           supportsSceneMeshing;
} XrSystemSceneMeshingPropertiesANDROID;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `supportsSceneMeshing` is an `XrBool32`, indicating if the system supports scene meshing.

An application **can** inspect whether the system is capable of scene meshing by chaining an XrSystemSceneMeshingPropertiesANDROID structure to the XrSystemProperties when calling xrGetSystemProperties.

If a runtime returns XR_FALSE for supportsSceneMeshing, the system does not support scene meshing, and therefore **must** return XR_ERROR_FEATURE_UNSUPPORTED from xrCreateSceneMeshingTrackerANDROID. The application **should** avoid using scene meshing functionality when supportsSceneMeshing is XR_FALSE.

If a runtime returns XR_TRUE for supportsSceneMeshing, the system supports scene meshing.

Valid Usage (Implicit)

- The XR_ANDROID_scene_meshing extension **must** be enabled prior to using XrSystemSceneMeshingPropertiesANDROID
- type **must** be XR_TYPE_SYSTEM_SCENE_MESHING_PROPERTIES_ANDROID
- next **must** be NULL or a valid pointer to the next structure in a structure chain

The XrSceneMeshSemanticLabelSetANDROID enumeration describes the semantic label sets for scene meshing. Each value in this enum represents another enum that contains the semantic labels. For example, the XR_SCENE_MESH_SEMANTIC_LABEL_SET_DEFAULT_ANDROID value represents the set XrSceneMeshSemanticLabelANDROID.

```
// Provided by XR_ANDROID_scene_meshing
typedef enum XrSceneMeshSemanticLabelSetANDROID {
    XR_SCENE_MESH_SEMANTIC_LABEL_SET_NONE_ANDROID = 0,
    XR_SCENE_MESH_SEMANTIC_LABEL_SET_DEFAULT_ANDROID = 1,
    XR_SCENE_MESH_SEMANTIC_LABEL_SET_MAX_ENUM_ANDROID = 0x7FFFFFFF
} XrSceneMeshSemanticLabelSetANDROID;
```

The enumeration values have the following meanings:

Enum	Description
XR_SCENE_MESH_SEMANTIC_LABEL_SET_NONE_ANDROID	This semantic label set represents absence of semantic labels.
XR_SCENE_MESH_SEMANTIC_LABEL_SET_DEFAULT_ANDROID	This semantic label set represents XrSceneMeshSemanticLabelANDROID.

The xrEnumerateSupportedSemanticLabelSetsANDROID function is defined as:

```
// Provided by XR_ANDROID_scene_meshing
XrResult xrEnumerateSupportedSemanticLabelSetsANDROID(
    XrInstance                                  instance,
    XrSystemId                                  systemId,
    uint32_t                                    supportedSemanticLabelSetsInputCapacity,
    uint32_t*                                   supportedSemanticLabelSetsOutputCount,
    XrSceneMeshSemanticLabelSetANDROID*         supportedSemanticLabelSets);
```

Parameter Descriptions

- `instance` is an XrInstance created by the application.
- `systemId` is an `XrSystemId` retried from xrGetSystem.
- `supportedSemanticLabelSetsInputCapacity` is the capacity of `supportedSemanticLabelSets`, or 0 to retrieve the required capacity.
- `supportedSemanticLabelSetsOutputCount` is a pointer to the count of `supportedSemanticLabelSets` written, or a pointer to the required capacity in the case that `supportedSemanticLabelSetsInputCapacity` is insufficient.
- `supportedSemanticLabelSets` is a pointer to an array of XrSceneMeshSemanticLabelSetANDROID. It **can** be `NULL` if `supportedSemanticLabelSetsInputCapacity` is 0.
- See Buffer Size Parameters chapter for a detailed description of retrieving the required `supportedSemanticLabelSets` size.

The application **can** enumerate semantic label sets supported by the system using xrEnumerateSupportedSemanticLabelSetsANDROID function.

Every system **must** at least support `XR_SCENE_MESH_SEMANTIC_LABEL_SET_NONE_ANDROID` because this enum value represents no semantic label set and **can** be used in the case the application does not need vertex semantics.

Valid Usage (Implicit)

- The `XR_ANDROID_scene_meshing` extension **must** be enabled prior to calling xrEnumerateSupportedSemanticLabelSetsANDROID
- `instance` **must** be a valid XrInstance handle
- `supportedSemanticLabelSetsOutputCount` **must** be a pointer to a `uint32_t` value
- If `supportedSemanticLabelSetsInputCapacity` is not `0`, `supportedSemanticLabelSets` **must** be a pointer to an array of `supportedSemanticLabelSetsInputCapacity` XrSceneMeshSemanticLabelSetANDROID values

Return Codes

Success

- XR_SUCCESS

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SIZE_INSUFFICIENT
- XR_ERROR_SYSTEM_INVALID
- XR_ERROR_VALIDATION_FAILURE

12.70.3. Create a scene meshing tracker handle

```
// Provided by XR_ANDROID_scene_meshing
XR_DEFINE_HANDLE(XrSceneMeshingTrackerANDROID)
```

The XrSceneMeshingTrackerANDROID handle represents a scene meshing tracker for scene meshing and managing the related resources.

This handle **can** be used to create a scene mesh snapshot using xrCreateSceneMeshSnapshotANDROID in this extension.

The xrCreateSceneMeshingTrackerANDROID function is defined as:

```
// Provided by XR_ANDROID_scene_meshing
XrResult xrCreateSceneMeshingTrackerANDROID(
    XrSession                                       session,
    const XrSceneMeshingTrackerCreateInfoANDROID* createInfo,
    XrSceneMeshingTrackerANDROID*                   tracker);
```

Parameter Descriptions

- `session` is an XrSession in which the scene meshing tracker will be active.
- `createInfo` is a pointer to an XrSceneMeshingTrackerCreateInfoANDROID structure used to describe the scene meshing tracker to be created.
- `tracker` is the returned XrSceneMeshingTrackerANDROID handle.

An application **can** create an XrSceneMeshingTrackerANDROID handle using the xrCreateSceneMeshingTrackerANDROID function.

If the system does not support scene meshing, the runtime **must** return `XR_ERROR_FEATURE_UNSUPPORTED` from xrCreateSceneMeshingTrackerANDROID. In this case, the runtime **must** return `XR_FALSE` for XrSystemSceneMeshingPropertiesANDROID::`supportsSceneMeshing` when the xrGetSystemProperties function is called, so that the application **can** avoid creating a scene meshing tracker.

The application **must** only use the semantic label sets that are supported by the system in XrSceneMeshingTrackerCreateInfoANDROID::`semanticLabelSet` when creating a scene meshing tracker. If the application uses an unsupported scene mesh vertex semantic label set, the runtime **must** return `XR_ERROR_FEATURE_UNSUPPORTED` from xrCreateSceneMeshingTrackerANDROID.

The application **can** enumerate the list of supported semantic label sets with the xrEnumerateSupportedSemanticLabelSetsANDROID function.

The xrCreateSceneMeshingTrackerANDROID handle owns all the resources for scene meshing. Once finished with scene meshing experience, the application **must** destroy the handle via the xrDestroySceneMeshingTrackerANDROID function.

Valid Usage (Implicit)

- The `XR_ANDROID_scene_meshing` extension **must** be enabled prior to calling xrCreateSceneMeshingTrackerANDROID
- `session` **must** be a valid XrSession handle
- `createInfo` **must** be a pointer to a valid XrSceneMeshingTrackerCreateInfoANDROID structure
- `tracker` **must** be a pointer to an XrSceneMeshingTrackerANDROID handle

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FEATURE_UNSUPPORTED
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The XrSceneMeshingTrackerCreateInfoANDROID structure is defined as:

```
// Provided by XR_ANDROID_scene_meshing
typedef struct XrSceneMeshingTrackerCreateInfoANDROID {
    XrStructureType                       type;
    const void*                           next;
    XrSceneMeshSemanticLabelSetANDROID    semanticLabelSet;
    XrBool32                              enableNormals;
} XrSceneMeshingTrackerCreateInfoANDROID;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `semanticLabelSet` is an XrSceneMeshSemanticLabelSetANDROID used to specify the semantic label set to be used for the scene meshing. If this is set to `XR_SCENE_MESH_SEMANTIC_LABEL_SET_NONE_ANDROID`, the runtime will ignore the XrSceneSubmeshDataANDROID::`vertexSemantics` buffer field.
- `enableNormals` is an `XrBool32` used to specify whether to include vertex normals for the mesh vertices in the scene mesh buffer when getting the scene mesh data.

The XrSceneMeshingTrackerCreateInfoANDROID structure describes the information to create an XrSceneMeshingTrackerANDROID handle.

Valid Usage (Implicit)

- The `XR_ANDROID_scene_meshing` extension **must** be enabled prior to using XrSceneMeshingTrackerCreateInfoANDROID
- `type` **must** be `XR_TYPE_SCENE_MESHING_TRACKER_CREATE_INFO_ANDROID`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `semanticLabelSet` **must** be a valid XrSceneMeshSemanticLabelSetANDROID value

The xrDestroySceneMeshingTrackerANDROID function is defined as:

```
// Provided by XR_ANDROID_scene_meshing
XrResult xrDestroySceneMeshingTrackerANDROID(
    XrSceneMeshingTrackerANDROID                tracker);
```

Parameter Descriptions

- `tracker` is an XrSceneMeshingTrackerANDROID previously created by xrCreateSceneMeshingTrackerANDROID.

The xrDestroySceneMeshingTrackerANDROID function releases the `tracker` and the underlying resources when finished with scene meshing experiences.

Valid Usage (Implicit)

- The XR_ANDROID_scene_meshing extension **must** be enabled prior to calling xrDestroySceneMeshingTrackerANDROID
- `tracker` **must** be a valid XrSceneMeshingTrackerANDROID handle

Thread Safety

- Access to `tracker`, and any child handles, **must** be externally synchronized

Return Codes

Success

- XR_SUCCESS

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_RUNTIME_FAILURE

12.70.4. Create a scene mesh snapshot handle

```
// Provided by XR_ANDROID_scene_meshing
XR_DEFINE_HANDLE(XrSceneMeshSnapshotANDROID)
```

The XrSceneMeshSnapshotANDROID handle represents a scene mesh snapshot. This snapshot is created by using xrCreateSceneMeshSnapshotANDROID which takes a snapshot of the scene mesh data at the time of the function call.

This handle **can** be used to retrieve submesh information and data using xrGetAllSubmeshStatesANDROID and xrGetSubmeshDataANDROID.

The xrCreateSceneMeshSnapshotANDROID function is defined as:

```
// Provided by XR_ANDROID_scene_meshing
XrResult xrCreateSceneMeshSnapshotANDROID(
    XrSceneMeshingTrackerANDROID                tracker,
    const XrSceneMeshSnapshotCreateInfoANDROID* createInfo,
    XrSceneMeshSnapshotCreationResultANDROID*   outSnapshotCreationResult);
```

Parameter Descriptions

- `tracker` is an XrSceneMeshingTrackerANDROID handle previously created with xrCreateSceneMeshingTrackerANDROID.
- `createInfo` is pointer to an XrSceneMeshSnapshotCreateInfoANDROID structure which contains necessary information required to create the scene mesh snapshot.
- `outSnapshotCreationResult` is a pointer to an XrSceneMeshSnapshotCreationResultANDROID object which is populated by the runtime with the snapshot creation result.

The application **can** use the xrCreateSceneMeshSnapshotANDROID function to create a scene mesh snapshot from the scene meshing tracker. This function returns an XrSceneMeshSnapshotANDROID handle along with XrSceneMeshTrackingStateANDROID wrapped in an XrSceneMeshSnapshotCreationResultANDROID structure. The handle is technically a snapshot of the scene mesh data at the time of creation. This handle **can** be used to query the scene mesh info and data using xrGetAllSubmeshStatesANDROID and xrGetSubmeshDataANDROID respectively.

Valid Usage (Implicit)

- The `XR_ANDROID_scene_meshing` extension **must** be enabled prior to calling xrCreateSceneMeshSnapshotANDROID
- `tracker` **must** be a valid XrSceneMeshingTrackerANDROID handle
- `createInfo` **must** be a pointer to a valid XrSceneMeshSnapshotCreateInfoANDROID structure
- `outSnapshotCreationResult` **must** be a pointer to an XrSceneMeshSnapshotCreationResultANDROID structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_POSE_INVALID
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_TIME_INVALID

The XrSceneMeshSnapshotCreateInfoANDROID structure is defined as:

```
// Provided by XR_ANDROID_scene_meshing
typedef struct XrSceneMeshSnapshotCreateInfoANDROID {
    XrStructureType    type;
    const void*        next;
    XrSpace            baseSpace;
    XrTime             time;
    XrBoxf             boundingBox;
} XrSceneMeshSnapshotCreateInfoANDROID;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `baseSpace` is an XrSpace within which the scene submesh pose info is represented.
- `time` is the `XrTime` that describes the time when the scene mesh will be processed or displayed, typically XrFrameState::`predictedDisplayTime`.
- `boundingBox` is an XrBoxf describing the bounding box within which to acquire the scene mesh.

The XrSceneMeshSnapshotCreateInfoANDROID structure describes the information to create an XrSceneMeshSnapshotANDROID handle

Valid Usage (Implicit)

- The `XR_ANDROID_scene_meshing` extension **must** be enabled prior to using XrSceneMeshSnapshotCreateInfoANDROID
- `type` **must** be `XR_TYPE_SCENE_MESH_SNAPSHOT_CREATE_INFO_ANDROID`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `baseSpace` **must** be a valid XrSpace handle

The XrSceneMeshSnapshotCreationResultANDROID structure is defined as:

```
// Provided by XR_ANDROID_scene_meshing
typedef struct XrSceneMeshSnapshotCreationResultANDROID {
    XrStructureType                    type;
    const void*                        next;
    XrSceneMeshSnapshotANDROID         snapshot;
    XrSceneMeshTrackingStateANDROID    trackingState;
} XrSceneMeshSnapshotCreationResultANDROID;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `snapshot` is an XrSceneMeshSnapshotANDROID handle created by the runtime.
- `trackingState` is an XrSceneMeshTrackingStateANDROID that describes the state of the scene meshing tracker at the time of the snapshot creation.

The XrSceneMeshSnapshotCreationResultANDROID structure stores the result of a scene mesh snapshot creation returned from xrCreateSceneMeshSnapshotANDROID.

Valid Usage (Implicit)

- The `XR_ANDROID_scene_meshing` extension **must** be enabled prior to using XrSceneMeshSnapshotCreationResultANDROID
- `type` **must** be `XR_TYPE_SCENE_MESH_SNAPSHOT_CREATION_RESULT_ANDROID`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `snapshot` **must** be a valid XrSceneMeshSnapshotANDROID handle
- `trackingState` **must** be a valid XrSceneMeshTrackingStateANDROID value

The XrSceneMeshTrackingStateANDROID enumeration describes the tracking state for a scene meshing tracker. Each value in this enum represents a state of the scene meshing tracker.

```
// Provided by XR_ANDROID_scene_meshing
typedef enum XrSceneMeshTrackingStateANDROID {
    XR_SCENE_MESH_TRACKING_STATE_INITIALIZING_ANDROID = 0,
    XR_SCENE_MESH_TRACKING_STATE_TRACKING_ANDROID = 1,
    XR_SCENE_MESH_TRACKING_STATE_WAITING_ANDROID = 2,
    XR_SCENE_MESH_TRACKING_STATE_ERROR_ANDROID = 3,
    XR_SCENE_MESH_TRACKING_STATE_MAX_ENUM_ANDROID = 0x7FFFFFFF
} XrSceneMeshTrackingStateANDROID;
```

The enumeration values have the following meanings:

Enum	Description
`XR_SCENE_MESH_TRACKING_STATE_INITIALIZING_ANDROID`	The tracker is not yet ready to provide mesh data.
`XR_SCENE_MESH_TRACKING_STATE_TRACKING_ANDROID`	The tracker is actively tracking.

Enum	Description
XR_SCENE_MESH_TRACKING_STATE_WAITING_ANDROID	The tracker is waiting for valid measurements to integrate since the last mesh update.
XR_SCENE_MESH_TRACKING_STATE_ERROR_ANDROID	The tracker has not received valid measurements for multiple cycles and is in an error state.

This state serves only as additional information to the application. In the absence of other sources of errors, the runtime **must** continue to allow the application to create snapshots and retrieve data from them regardless of the current XrSceneMeshTrackingStateANDROID of the tracker. If the state is not XR_SCENE_MESH_TRACKING_STATE_TRACKING_ANDROID, the snapshots **may** not be up to date or empty.

12.70.5. Acquiring scene mesh data from the scene mesh snapshot

The xrGetAllSubmeshStatesANDROID function is defined as:

```
// Provided by XR_ANDROID_scene_meshing
XrResult xrGetAllSubmeshStatesANDROID(
    XrSceneMeshSnapshotANDROID                  snapshot,
    uint32_t                                    submeshStateCapacityInput,
    uint32_t*                                   submeshStateCountOutput,
    XrSceneSubmeshStateANDROID*                 submeshStates);
```

Parameter Descriptions

- snapshot is an XrSceneMeshSnapshotANDROID handle previously created with xrCreateSceneMeshSnapshotANDROID.
- submeshStateCapacityInput is the capacity of submeshStates, or 0 to retrieve the required capacity.
- submeshStateCountOutput is a pointer to the count of submeshStates written, or a pointer to the required capacity in the case that submeshStateCapacityInput is insufficient.
- submeshStates is a pointer to an array of XrSceneSubmeshStateANDROID structures to which the runtime will output the state of all submeshes in the scene mesh. It **can** be NULL if submeshStateCapacityInput is 0.
- See Buffer Size Parameters chapter for a detailed description of retrieving the required submeshStates size.

The application **can** use the xrGetAllSubmeshStatesANDROID function to get the state of all submeshes in the scene mesh. This contains basic information information to allow the application to select submeshes for which the application wants data for.

Valid Usage (Implicit)

- The XR_ANDROID_scene_meshing extension **must** be enabled prior to calling xrGetAllSubmeshStatesANDROID
- snapshot **must** be a valid XrSceneMeshSnapshotANDROID handle
- submeshStateCountOutput **must** be a pointer to a uint32_t value
- If submeshStateCapacityInput is not 0, submeshStates **must** be a pointer to an array of submeshStateCapacityInput XrSceneSubmeshStateANDROID structures

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_SIZE_INSUFFICIENT
- XR_ERROR_VALIDATION_FAILURE

The XrSceneSubmeshStateANDROID structure is defined as:

```
// Provided by XR_ANDROID_scene_meshing
typedef struct XrSceneSubmeshStateANDROID {
    XrStructureType    type;
    void*              next;
    XrUuid             submeshId;
    XrTime             lastUpdatedTime;
    XrPosef            submeshPoseInBaseSpace;
    XrExtent3Df        bounds;
} XrSceneSubmeshStateANDROID;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `submeshId` is an XrUuid identifying the submesh.
- `lastUpdatedTime` is an `XrTime` representing the time when the submesh was last updated.
- `submeshPoseInBaseSpace` is an XrPosef representing the pose of the submesh in the base space used to create the scene mesh snapshot. The pose is also the center of the submesh bounding box.
- `bounds` is an XrExtent3Df describing the dimensions of the bounding box that encloses the submesh. This bounds is defined in the base space that was used to create the scene mesh handle.

The runtime makes best effort to map `submeshId` with the region of the physical space occupied by the submesh. The Runtimes **may** at any time lose this mapping and assign a new UUID to any subset of the detected submeshes. In particular runtimes are not required to reuse the same UUIDs for the same meshes in subsequent application instances.

Valid Usage (Implicit)

- The `XR_ANDROID_scene_meshing` extension **must** be enabled prior to using XrSceneSubmeshStateANDROID
- `type` **must** be `XR_TYPE_SCENE_SUBMESH_STATE_ANDROID`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

The xrGetSubmeshDataANDROID function is defined as:

```
// Provided by XR_ANDROID_scene_meshing
XrResult xrGetSubmeshDataANDROID(
    XrSceneMeshSnapshotANDROID                  snapshot,
    uint32_t                                    submeshDataCount,
    XrSceneSubmeshDataANDROID*                  inoutSubmeshData);
```

Parameter Descriptions

- `snapshot` is an XrSceneMeshSnapshotANDROID handle previously created with xrCreateSceneMeshSnapshotANDROID.
- `submeshDataCount` is the length of `inoutSubmeshData`.
- `inoutSubmeshData` is an application allocated array of XrSceneSubmeshDataANDROID in which the runtime populates each element with the mesh data associated with the submesh id passed in XrSceneSubmeshDataANDROID::`submeshId`.

The application **can** use the xrGetSubmeshDataANDROID function to get batches of submeshes, or all submeshes at once, by passing a list of XrSceneSubmeshDataANDROID with each XrSceneSubmeshDataANDROID::`submeshId` set to the respective id of the submesh to retrieve.

The XrSceneSubmeshDataANDROID::`submeshId` **must** be unique for every element in `inoutSubmeshData` and retrievable by xrGetAllSubmeshStatesANDROID for the same `snapshot`. If the submesh Id is invalid or more than one elements in `inoutSubmeshData` have the same submesh id, then the runtime **must** return `XR_ERROR_VALIDATION_FAILURE`.

`submeshDataCount` **must** be equal or less than the value returned by xrGetAllSubmeshStatesANDROID ::`submeshStateCountOutput`, otherwise the runtime **must** return `XR_ERROR_VALIDATION_FAILURE`.

This **can** be used in a 2-call idiom to query the vertex and index count for all of the selected submeshes.

Valid Usage (Implicit)

- The `XR_ANDROID_scene_meshing` extension **must** be enabled prior to calling xrGetSubmeshDataANDROID
- `snapshot` **must** be a valid XrSceneMeshSnapshotANDROID handle
- `inoutSubmeshData` **must** be a pointer to an array of `submeshDataCount` XrSceneSubmeshDataANDROID structures
- The `submeshDataCount` parameter **must** be greater than `0`

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_SIZE_INSUFFICIENT
- XR_ERROR_VALIDATION_FAILURE

The XrSceneSubmeshDataANDROID structure is defined as:

```
// Provided by XR_ANDROID_scene_meshing
typedef struct XrSceneSubmeshDataANDROID {
    XrStructureType    type;
    const void*        next;
    XrUuid             submeshId;
    uint32_t           vertexCapacityInput;
    uint32_t           vertexCountOutput;
    XrVector3f*        vertexPositions;
    XrVector3f*        vertexNormals;
    uint8_t*           vertexSemantics;
    uint32_t           indexCapacityInput;
    uint32_t           indexCountOutput;
    uint32_t*          indices;
} XrSceneSubmeshDataANDROID;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `submeshId` is an XrUuid identifying the submesh.
- `vertexCapacityInput` is the capacity of `vertexPositions`, `vertexNormals` and `vertexSemantics`, or 0 to retrieve the required capacity.
- `vertexCountOutput` is a pointer to the count of `vertexPositions`, `vertexNormals`, and `vertexSemantics` written, or a pointer to the required capacity in the case that `vertexCapacityInput` is insufficient.
- `vertexPositions` is an array of XrVector3f to which the runtime will output the vertex positions of the submesh.
- `vertexNormals` is an array of XrVector3f to which the runtime will output the vertex normals of the submesh. This field **can** be left `NULL` if the application created a tracker with normals disabled.
- `vertexSemantics` is an array of `uint8_t` to which the runtime will output the vertex semantics of the submesh. This field **can** be left `NULL` if the application created a tracker with `XR_SCENE_MESH_SEMANTIC_LABEL_SET_NONE_ANDROID` semantic label set.
- `indexCapacityInput` is the capacity of `indices`, or 0 to retrieve the required capacity.
- `indexCountOutput` is a pointer to the count of `indices` written, or a pointer to the required capacity in the case that `vertexCapacityInput` is insufficient.
- `indices` is an array of `uint32_t` to which the runtime will output the indices of the submesh.
- See Buffer Size Parameters chapter for a detailed description of retrieving the required `vertexPositions`, `vertexNormals`, `vertexSemantics`, and `indices` size.

Valid Usage (Implicit)

- The XR_ANDROID_scene_meshing extension **must** be enabled prior to using XrSceneSubmeshDataANDROID
- type **must** be XR_TYPE_SCENE_SUBMESH_DATA_ANDROID
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- If vertexCapacityInput is not 0, vertexPositions **must** be a pointer to an array of vertexCapacityInput XrVector3f structures
- If vertexCapacityInput is not 0, vertexNormals **must** be a pointer to an array of vertexCapacityInput XrVector3f structures
- If vertexCapacityInput is not 0, vertexSemantics **must** be a pointer to an array of vertexCapacityInput uint8_t values
- If indexCapacityInput is not 0, indices **must** be a pointer to an array of indexCapacityInput uint32_t values

The XrSceneMeshSemanticLabelANDROID enumeration is the default set of semantic labels that **can** be used to label mesh vertices to describe which physical environment surface the mesh vertex is on. This enum set is represented by the XR_SCENE_MESH_SEMANTIC_LABEL_SET_DEFAULT_ANDROID value in XrSceneMeshSemanticLabelSetANDROID.

```
// Provided by XR_ANDROID_scene_meshing
typedef enum XrSceneMeshSemanticLabelANDROID {
    XR_SCENE_MESH_SEMANTIC_LABEL_OTHER_ANDROID = 0,
    XR_SCENE_MESH_SEMANTIC_LABEL_FLOOR_ANDROID = 1,
    XR_SCENE_MESH_SEMANTIC_LABEL_CEILING_ANDROID = 2,
    XR_SCENE_MESH_SEMANTIC_LABEL_WALL_ANDROID = 3,
    XR_SCENE_MESH_SEMANTIC_LABEL_TABLE_ANDROID = 4,
    XR_SCENE_MESH_SEMANTIC_LABEL_MAX_ENUM_ANDROID = 0x7FFFFFFF
} XrSceneMeshSemanticLabelANDROID;
```

The enums have the following meanings:

Enum	Description
XR_SCENE_MESH_SEMANTIC_LABEL_OTHER_ANDROID	This semantic label indicates that the corresponding mesh element represents an unknown object.
XR_SCENE_MESH_SEMANTIC_LABEL_FLOOR_ANDROID	This semantic label indicates that the corresponding mesh element represents a floor.

Enum	Description
XR_SCENE_MESH_SEMANTIC_LABEL_CEILING_ANDROID	This semantic label indicates that the corresponding mesh element represents a ceiling.
XR_SCENE_MESH_SEMANTIC_LABEL_WALL_ANDROID	This semantic label indicates that the corresponding mesh element represents a wall.
XR_SCENE_MESH_SEMANTIC_LABEL_TABLE_ANDROID	This semantic label indicates that the corresponding mesh element represents a table.

12.70.6. Destroy a scene mesh snapshot

The xrDestroySceneMeshSnapshotANDROID function is defined as:

```
// Provided by XR_ANDROID_scene_meshing
XrResult xrDestroySceneMeshSnapshotANDROID(
    XrSceneMeshSnapshotANDROID                  snapshot);
```

Parameter Descriptions

- snapshot is an XrSceneMeshSnapshotANDROID handle previously created with xrCreateSceneMeshSnapshotANDROID that will be destroyed by this function.

The application **can** use the xrDestroySceneMeshSnapshotANDROID function to destroy a scene mesh snapshot. Once the handle is destroyed, the application **can** no longer use it to get the submesh info or data. The handle is automatically destroyed when the tracker is destroyed because the tracker handle is the parent of the scene mesh snapshot handle.

Valid Usage (Implicit)

- The XR_ANDROID_scene_meshing extension **must** be enabled prior to calling xrDestroySceneMeshSnapshotANDROID
- snapshot **must** be a valid XrSceneMeshSnapshotANDROID handle

Thread Safety

- Access to snapshot, and any child handles, **must** be externally synchronized

Return Codes

Success

- XR_SUCCESS

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_RUNTIME_FAILURE

12.70.7. Example code for scene meshing

Following example code demonstrates how to access scene mesh data for rendering.

```
XrInstance instance;  // Created at app startup
XrSystemId systemId;  // Received from xrGetSystem() at app startup
XrSession session;    // Created at app startup
XrSpace appPlaySpace; // Created at app startup

// The function pointers are previously initialized using xrGetInstanceProcAddr.
PFN_xrCreateSceneMeshingTrackerANDROID xrCreateSceneMeshingTrackerANDROID;
PFN_xrDestroySceneMeshingTrackerANDROID xrDestroySceneMeshingTrackerANDROID;
PFN_xrEnumerateSupportedSemanticLabelSetsANDROID
xrEnumerateSupportedSemanticLabelSetsANDROID;
PFN_xrCreateSceneMeshSnapshotANDROID xrCreateSceneMeshSnapshotANDROID;
PFN_xrDestroySceneMeshSnapshotANDROID xrDestroySceneMeshSnapshotANDROID;
PFN_xrGetAllSubmeshStatesANDROID xrGetAllSubmeshStatesANDROID;
PFN_xrGetSubmeshDataANDROID xrGetSubmeshDataANDROID;

// Inspect system capability
XrSystemSceneMeshingPropertiesANDROID sceneMeshingProps = {
  .type = XR_TYPE_SYSTEM_SCENE_MESHING_PROPERTIES_ANDROID,
};
XrSystemProperties sysProps = {
  .type = XR_TYPE_SYSTEM_PROPERTIES,
  .next = &sceneMeshingProps
};
CHK_XR(xrGetSystemProperties(instance, systemId, &sysProps));
if (!sceneMeshingProps.supportsSceneMeshing) {
    // scene meshing is not supported.
    return;
}

uint32_t supportedsemanticLabelSetsCount = 0;
```

```
xrEnumerateSupportedSemanticLabelSetsANDROID(
  instance, systemId, 0, &supportedsemanticLabelSetsCount, nullptr);
std::vector<XrSceneMeshSemanticLabelSetANDROID> supportedSemanticLabelSets(
  supportedsemanticLabelSetsCount);
xrEnumerateSupportedSemanticLabelSetsANDROID(
  instance, systemId, supportedsemanticLabelSetsCount,
  &supportedsemanticLabelSetsCount, supportedSemanticLabelSets.data());

XrSceneMeshSemanticLabelSetANDROID semanticLabelSet =
XR_SCENE_MESH_SEMANTIC_LABEL_SET_NONE_ANDROID;
// Check if system supports XR_SCENE_MESH_SEMANTIC_LABEL_SET_DEFAULT_ANDROID
if (std::find(supportedSemanticLabelSets.begin(), supportedSemanticLabelSets.end(),
              XR_SCENE_MESH_SEMANTIC_LABEL_SET_DEFAULT_ANDROID) !=
    supportedSemanticLabelSets.end()) {
  semanticLabelSet = XR_SCENE_MESH_SEMANTIC_LABEL_SET_DEFAULT_ANDROID;
}

XrSceneMeshingTrackerCreateInfoANDROID trackerCreateInfo = {
  .type = XR_TYPE_SCENE_MESHING_TRACKER_CREATE_INFO_ANDROID,
  .semanticLabelSet = semanticLabelSet,
  .enableNormals = XR_TRUE
};

XrSceneMeshingTrackerANDROID tracker = XR_NULL_HANDLE;
CHK_XR(xrCreateSceneMeshingTrackerANDROID(session, &trackerCreateInfo, &tracker));

XrSceneMeshSnapshotCreationResultANDROID snapshotResult {
  .type = XR_TYPE_SCENE_MESH_SNAPSHOT_CREATION_RESULT_ANDROID,
  .snapshot = XR_NULL_HANDLE
};
XrSceneMeshSnapshotANDROID& snapshot = snapshotResult.snapshot;
XrSceneMeshSnapshotCreateInfoANDROID createInfo {
  .type = XR_TYPE_SCENE_MESH_SNAPSHOT_CREATE_INFO_ANDROID
};

// app update loop
while (true) {
  // ...
  // For every frame in frame loop
  // ...

  XrFrameState frameState; // previously returned from xrWaitFrame
  const XrTime time = frameState.predictedDisplayTime;
  XrBoxf box; // populated with the bounding box of the camera frustum

  // ...

  createInfo.baseSpace = appPlaySpace;
```

```
  createInfo.time = time;
  createInfo.boundingBox = box;

  // Grab the scene mesh snapshot. This way the app can: know all of the submesh infos,
  // choose the appropriate submeshes for which to get data, allocate the necessary
  // buffer for those submeshes, and then get the data.
  CHK_XR(xrCreateSceneMeshSnapshotANDROID(tracker, &createInfo, &snapshotResult));

  // check the tracking state
  if (snapshotResult.trackingState == XR_SCENE_MESH_TRACKING_STATE_ERROR_ANDROID) {
    // unrecoverable error. Exit the app.
    if (snapshot != XR_NULL_HANDLE) {
      CHK_XR(xrDestroySceneMeshSnapshotANDROID(snapshot));
      snapshot = XR_NULL_HANDLE;
    }
    break;
  } else if (snapshotResult.trackingState !=
XR_SCENE_MESH_TRACKING_STATE_TRACKING_ANDROID) {
    // The tracker is not tracking. Try again later.
    if (snapshot != XR_NULL_HANDLE) {
      CHK_XR(xrDestroySceneMeshSnapshotANDROID(snapshot));
      snapshot = XR_NULL_HANDLE;
    }
    continue;
  }

  // 2-call idiom for getting submesh states
  uint32_t submeshCount = 0;
  CHK_XR(xrGetAllSubmeshStatesANDROID(snapshot, 0, &submeshCount, nullptr));
  std::vector<XrSceneSubmeshStateANDROID> states(submeshCount, {
    .type = XR_TYPE_SCENE_SUBMESH_STATE_ANDROID,
  });
  CHK_XR(xrGetAllSubmeshStatesANDROID(snapshot, submeshCount, &submeshCount, states.
data()));

  // To store the poses for the selected submeshes
  std::vector<XrPosef> submeshesPoses;
  submeshesPoses.reserve(submeshCount);

  std::vector<XrSceneSubmeshDataANDROID> submeshes;
  submeshes.reserve(submeshCount);
  // Iterate the states and determine which submeshes the app wants data for
  for (XrSceneSubmeshStateANDROID& state : states) {
    // Modify this bool as necessary by looking at states. Maybe state.lastUpdatedTime
    // is below a threshold or maybe the bounding box does not intersect with camera view
    // frustum (The submesh will be culled).
    bool needed;
    if (needed) {
```

```
      // Add the selected submesh to the submeshes list. Set the capacity
      // zero for now since the size of the buffer will be determined later.
      XrSceneSubmeshDataANDROID submesh {
        .type = XR_TYPE_SCENE_SUBMESH_DATA_ANDROID,
        .submeshId = state.submeshId,
        .vertexCapacityInput = 0,
        .vertexCountOutput = 0,
        .vertexPositions = nullptr,
        .vertexNormals = nullptr,
        .vertexSemantics = nullptr,
        .indexCapacityInput = 0,
        .indexCountOutput = 0,
        .indices = nullptr,
      };
      submeshes.push_back(submesh);
      submeshesPoses.push_back(state.submeshPoseInBaseSpace);
    }
  }

  // Grab the data for the selected submeshes using the 2-call idiom.
  CHK_XR(xrGetSubmeshDataANDROID(snapshot, submeshes.size(), submeshes.data()));
  for (XrSceneSubmeshDataANDROID& submesh : submeshes) {
    submesh.vertexCapacityInput = submesh.vertexCountOutput;
    submesh.vertexCountOutput = 0;
    submesh.vertexPositions = new XrVector3f[submesh.vertexCapacityInput];
    submesh.vertexNormals = new XrVector3f[submesh.vertexCapacityInput];
    submesh.vertexSemantics = new uint8_t[submesh.vertexCapacityInput];
    submesh.indexCapacityInput = submesh.indexCountOutput;
    submesh.indexCountOutput = 0;
    submesh.indices = new uint32_t[submesh.indexCapacityInput];
  }
  CHK_XR(xrGetSubmeshDataANDROID(snapshot, submeshes.size(), submeshes.data()));

  // Destroy the scene mesh snapshot since we have finally grabbed the submeshes data. In
  // next iteration app can: create a new one to get latest mesh data
  CHK_XR(xrDestroySceneMeshSnapshotANDROID(snapshot));
  snapshot = XR_NULL_HANDLE;

  // Iterate the submeshes and get the vertex positions, vertex normals and
  // vertex semantics data for the selected submeshes
  for (uint32_t i = 0; i < submeshes.size(); i++) {
    XrSceneSubmeshDataANDROID& data = submeshes[i];
    XrVector3f* vertexPositions = data.vertexPositions;
    XrVector3f* vertexNormals = data.vertexNormals;
    XrSceneMeshSemanticLabelANDROID* vertexSemantics =
      new XrSceneMeshSemanticLabelANDROID[data.vertexCountOutput];
    for (uint32_t j = 0; j < data.vertexCountOutput; j++) {
      vertexSemantics[j] =
```

```
            static_cast<XrSceneMeshSemanticLabelANDROID>(data.vertexSemantics[j]);
        }
        XrPosef pose = submeshesPoses[i];

        // *** Use the data as per needs ***

        // Release the allocated memory for the data buffers when done using
        delete [] vertexSemantics;
        delete [] data.vertexPositions;
        delete [] data.vertexNormals;
        delete [] data.vertexSemantics;
        delete [] data.indices;
      }

      // ...
      // Finish frame loop
      // ...
    }

    CHK_XR(xrDestroySceneMeshingTrackerANDROID(tracker));
```

12.70.8. New Object Types

- XrSceneMeshSnapshotANDROID
- XrSceneMeshingTrackerANDROID

12.70.9. New Commands

- xrCreateSceneMeshSnapshotANDROID
- xrCreateSceneMeshingTrackerANDROID
- xrDestroySceneMeshSnapshotANDROID
- xrDestroySceneMeshingTrackerANDROID
- xrEnumerateSupportedSemanticLabelSetsANDROID
- xrGetAllSubmeshStatesANDROID
- xrGetSubmeshDataANDROID

12.70.10. New Structures

- XrSceneMeshSnapshotCreateInfoANDROID
- XrSceneMeshSnapshotCreationResultANDROID
- XrSceneMeshingTrackerCreateInfoANDROID
- XrSceneSubmeshDataANDROID

- XrSceneSubmeshStateANDROID
- Extending XrSystemProperties:
  - XrSystemSceneMeshingPropertiesANDROID

12.70.11. New Enums

- XrSceneMeshSemanticLabelANDROID
- XrSceneMeshSemanticLabelSetANDROID
- XrSceneMeshTrackingStateANDROID

12.70.12. New Enum Constants

- `XR_ANDROID_SCENE_MESHING_EXTENSION_NAME`
- `XR_ANDROID_scene_meshing_SPEC_VERSION`
- Extending XrObjectType:
  - `XR_OBJECT_TYPE_SCENE_MESHING_TRACKER_ANDROID`
  - `XR_OBJECT_TYPE_SCENE_MESH_SNAPSHOT_ANDROID`
- Extending XrStructureType:
  - `XR_TYPE_SCENE_MESHING_TRACKER_CREATE_INFO_ANDROID`
  - `XR_TYPE_SCENE_MESH_SNAPSHOT_CREATE_INFO_ANDROID`
  - `XR_TYPE_SCENE_MESH_SNAPSHOT_CREATION_RESULT_ANDROID`
  - `XR_TYPE_SCENE_SUBMESH_DATA_ANDROID`
  - `XR_TYPE_SCENE_SUBMESH_STATE_ANDROID`
  - `XR_TYPE_SYSTEM_SCENE_MESHING_PROPERTIES_ANDROID`

12.70.13. Issues

12.70.14. Version History

- Revision 3, 2025-05-15 (Salar Khan)
  - Initial extension description

12.71. XR_ANDROID_spatial_anchor_space

Name String

`XR_ANDROID_spatial_anchor_space`

Extension Type

Instance extension

Registered Extension Number

796

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

`XR_EXT_spatial_anchor`

Last Modified Date

2025-09-15

IP Status

No known IP claims.

Contributors

YuSheng Chang, Google
Kyle Chen, Google
Levana Chen, Google
Nihav Jain, Google
Spencer Quin, Google

12.71.1. Overview

The xrCreateSpatialAnchorEXT function defined in the `XR_EXT_spatial_anchor` extension allows applications to create spatial anchors with an XrSpatialEntityEXT handle to represent it. In contrast, this extension creates a spatial anchor that are represented by an XrSpace handle instead. This is useful for cases where the application wants to:

- Use xrLocateSpaces to get the location of the anchor.
- Locate other spaces relative to the anchor.
- Export the anchor using `XR_ANDROID_anchor_sharing_export`.
- Application has placed anchors and now only cares about the location of the anchors (to attach virtual content to), and does not care about any other component of any other entities discovered via the XrSpatialContextEXT used to create the anchor, thereby allowing the application to destroy the spatial context.

Anchors created with this extension **cannot** be persisted using the

XR_ANDROID_device_anchor_persistence extension and the application **must** instead use XR_EXT_spatial_persistence_operations. The runtime **must** return XR_ERROR_VALIDATION_FAILURE from xrPersistAnchorANDROID if XrPersistedAnchorSpaceInfoANDROID::anchor is created using xrCreateSpatialAnchorSpaceANDROID or xrCreateSpatialAnchorSpaceFromIdANDROID.

12.71.2. Create XrSpace for a Spatial Anchor

Create a spatial anchor and space

The xrCreateSpatialAnchorSpaceANDROID function is defined as:

```
// Provided by XR_ANDROID_spatial_anchor_space
XrResult xrCreateSpatialAnchorSpaceANDROID(
    XrSession                                   session,
    XrSpatialContextEXT                         spatialContext,
    const XrSpatialAnchorCreateInfoEXT*         createInfo,
    XrSpatialEntityIdEXT*                       anchorEntityId,
    XrSpace*                                    anchorSpace);
```

Parameter Descriptions

- session is a handle to an XrSession previously created with xrCreateSession
- spatialContext is an XrSpatialContextEXT previously created using xrCreateSpatialContextAsyncEXT.
- createInfo is a pointer to an XrSpatialAnchorCreateInfoEXT.
- anchorEntityId is a pointer to an XrSpatialEntityIdEXT.
- anchorSpace is a pointer to an XrSpace.

The xrCreateSpatialAnchorSpaceANDROID function creates an anchor that outputs an XrSpatialEntityIdEXT and an XrSpace handle. xrCreateSpatialAnchorSpaceANDROID has the same validation requirements as xrCreateSpatialAnchorEXT.

Runtime **must** return XR_ERROR_VALIDATION_FAILURE if the XrSpatialContextEXT passed to xrCreateSpatialAnchorSpaceANDROID was not configured with XR_SPATIAL_CAPABILITY_ANCHOR_EXT.

With respect to the discoverability of the created anchor by xrCreateSpatialAnchorSpaceANDROID, the last paragraph of the xrCreateSpatialAnchorEXT spec applies.

Valid Usage (Implicit)

- The XR_ANDROID_spatial_anchor_space extension **must** be enabled prior to calling xrCreateSpatialAnchorSpaceANDROID
- session **must** be a valid XrSession handle
- spatialContext **must** be a valid XrSpatialContextEXT handle
- createInfo **must** be a pointer to a valid XrSpatialAnchorCreateInfoEXT structure
- anchorEntityId **must** be a pointer to an XrSpatialEntityIdEXT value
- anchorSpace **must** be a pointer to an XrSpace handle
- spatialContext **must** have been created, allocated, or retrieved from session

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_POSE_INVALID
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_SPATIAL_ANCHOR_ATTACHABLE_COMPONENT_NOT_FOUND_ANDROID
- XR_ERROR_SPATIAL_ENTITY_ID_INVALID_EXT
- XR_ERROR_TIME_INVALID
- XR_ERROR_VALIDATION_FAILURE

Create XrSpace from an existing spatial anchor

The xrCreateSpatialAnchorSpaceFromIdANDROID function is defined as:

```
// Provided by XR_ANDROID_spatial_anchor_space
XrResult xrCreateSpatialAnchorSpaceFromIdANDROID(
    XrSession                                       session,
    XrSpatialContextEXT                             spatialContext,
    const XrSpatialAnchorSpaceFromIdCreateInfoANDROID* createInfo,
    XrSpace*                                        anchorSpace);
```

Parameter Descriptions

- `session` is a handle to an XrSession previously created with xrCreateSession
- `spatialContext` is an XrSpatialContextEXT previously created using xrCreateSpatialContextAsyncEXT.
- `createInfo` is a pointer to an XrSpatialAnchorSpaceFromIdCreateInfoANDROID.
- `anchorSpace` is a pointer to an XrSpace.

In scenarios where spatial anchor entities are preexisting (e.g. due to persistence), applications **can** create an XrSpace from these spatial anchors by supplying their spatial entity IDs.

The xrCreateSpatialAnchorSpaceFromIdANDROID function creates an XrSpace handle by providing an `XrSpatialEntityIdEXT` in the XrSpatialAnchorSpaceFromIdCreateInfoANDROID::entityId field.

Runtime **must** return `XR_ERROR_VALIDATION_FAILURE` if the XrSpatialContextEXT passed to xrCreateSpatialAnchorSpaceFromIdANDROID was not configured with `XR_SPATIAL_CAPABILITY_ANCHOR_EXT`.

Valid Usage (Implicit)

- The `XR_ANDROID_spatial_anchor_space` extension **must** be enabled prior to calling xrCreateSpatialAnchorSpaceFromIdANDROID
- `session` **must** be a valid XrSession handle
- `spatialContext` **must** be a valid XrSpatialContextEXT handle
- `createInfo` **must** be a pointer to a valid XrSpatialAnchorSpaceFromIdCreateInfoANDROID structure
- `anchorSpace` **must** be a pointer to an XrSpace handle
- `spatialContext` **must** have been created, allocated, or retrieved from `session`

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_SPATIAL_ANCHOR_ENTITY_ID_INVALID_ANDROID
- XR_ERROR_SPATIAL_ENTITY_ID_INVALID_EXT
- XR_ERROR_VALIDATION_FAILURE

The XrSpatialAnchorSpaceFromIdCreateInfoANDROID structure is defined as:

```
// Provided by XR_ANDROID_spatial_anchor_space
typedef struct XrSpatialAnchorSpaceFromIdCreateInfoANDROID {
    XrStructureType         type;
    const void*             next;
    XrSpatialEntityIdEXT    anchorEntityId;
} XrSpatialAnchorSpaceFromIdCreateInfoANDROID;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain.
- anchorEntityId is the XrSpatialEntityIdEXT of the existing spatial anchor from which to create the XrSpace.

The runtime **must** return XR_ERROR_SPATIAL_ENTITY_ID_INVALID_EXT if XrSpatialAnchorSpaceFromIdCreateInfoANDROID::entityId is not a valid ID for spatialContext.

The runtime **must** return XR_ERROR_SPATIAL_ANCHOR_ENTITY_ID_INVALID_ANDROID if XrSpatialAnchorSpaceFromIdCreateInfoANDROID::entityId is not a valid entity ID for an anchor.

Valid Usage (Implicit)

- The XR_ANDROID_spatial_anchor_space extension **must** be enabled prior to using XrSpatialAnchorSpaceFromIdCreateInfoANDROID
- type **must** be XR_TYPE_SPATIAL_ANCHOR_SPACE_FROM_ID_CREATE_INFO_ANDROID
- next **must** be NULL or a valid pointer to the next structure in a structure chain

12.71.3. Anchor Lifecycle

The parent of an XrSpace handle is an XrSession. This implies that an XrSpace handle representing the anchor remains valid even after xrCreateSpatialAnchorSpaceFromIdANDROID::spatialContext or xrCreateSpatialAnchorSpaceANDROID::spatialContext are destroyed. Creating an XrSpace to represent a spatial entity anchor allows the application to destroy the XrSpatialContextEXT that was configured with XR_SPATIAL_CAPABILITY_ANCHOR_EXT while still continuing to get the latest pose of the anchor using its XrSpace with xrLocateSpaces i.e. the runtime **must** continue to track the anchor until all XrSpace handles that represent it have been destroyed.

12.71.4. Example code

Create spatial anchor as an XrSpace

The following example code demonstrates how to create a spatial anchor and return it as an XrSpace.

```
XrSpatialEntityIdEXT spatialAnchorEntityId;
XrSpace spatialAnchorSpace {XR_NULL_HANDLE}; // XrSpace handle of anchor

// Spatial anchor create info
XrSpatialAnchorCreateInfoEXT createInfo{
  .type = XR_TYPE_SPATIAL_ANCHOR_CREATE_INFO_EXT,
  .baseSpace = localSpace,
  .time = predictedDisplayTime,
  .pose = {{0, 0, 0, 1}, {1, 1, 1}},
};

// Create spatial anchor using the new API
CHK_XR(xrCreateSpatialAnchorSpaceANDROID(session, spatialContext, &createInfo,
&spatialAnchorEntityId, &spatialAnchorSpace));

// ...
```

Create XrSpace from anchor entity ID

The following example code demonstrates how to create an XrSpace from a preexisting anchor entity ID.

```
XrSpace anchorSpace = XR_NULL_HANDLE;

// Spatial anchor from XrSpace create info
XrSpatialAnchorSpaceFromIdCreateInfoANDROID createInfoFromId{
  .type = XR_TYPE_SPATIAL_ANCHOR_SPACE_FROM_ID_CREATE_INFO_ANDROID,
  .anchorEntityId = spatialAnchorEntityId,
};

// Create XrSpace handle from spatial anchor ID
CHK_XR(xrCreateSpatialAnchorSpaceFromIdANDROID(session, spatialContext, &
createInfoFromId, &anchorSpace));

// ...
```

12.71.5. New Commands

- xrCreateSpatialAnchorSpaceANDROID
- xrCreateSpatialAnchorSpaceFromIdANDROID

12.71.6. New Structures

- XrSpatialAnchorSpaceFromIdCreateInfoANDROID

12.71.7. New Enum Constants

- XR_ANDROID_SPATIAL_ANCHOR_SPACE_EXTENSION_NAME
- XR_ANDROID_spatial_anchor_space_SPEC_VERSION
- Extending XrResult:
  - XR_ERROR_SPATIAL_ANCHOR_ENTITY_ID_INVALID_ANDROID
- Extending XrStructureType:
  - XR_TYPE_SPATIAL_ANCHOR_SPACE_FROM_ID_CREATE_INFO_ANDROID

12.71.8. Issues

12.71.9. Version History

- Revision 1, 2025-09-15 (YuSheng Chang, Kyle Chen)

- Initial extension description.

12.72. XR_ANDROID_spatial_component_subsumed_by

Name String

`XR_ANDROID_spatial_component_subsumed_by`

Extension Type

Instance extension

Registered Extension Number

792

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

`XR_EXT_spatial_entity`
and
`XR_EXT_spatial_plane_tracking`

Last Modified Date

2025-08-19

IP Status

No known IP claims.

Contributors

Brian Chen, Google
Kyle Chen, Google
Levana Chen, Google
Nihav Jain, Google
Spencer Quin, Google

12.72.1. Overview

This extension builds on `XR_EXT_spatial_entity` and provides a new component for `XR_SPATIAL_CAPABILITY_PLANE_TRACKING_EXT` to expose the `XrSpatialEntityIdEXT` that subsumes the current entity.

When the runtime has acquired enough environment information to detect that 2 tracked planes are actually the same, the `XR_SPATIAL_COMPONENT_TYPE_SUBSUMED_BY_ANDROID` component with the ID of one of

the planes is attached to the other. From then on, the application only needs to process the entity that does not have the XR_SPATIAL_COMPONENT_TYPE_SUBSUMED_BY_ANDROID component.

This extension also introduces a new filter, which the application **can** chain to XrSpatialDiscoverySnapshotCreateInfoEXT to filter out any entities that have the XR_SPATIAL_COMPONENT_TYPE_SUBSUMED_BY_ANDROID component.



Permissions

Android applications **must** have the android.permission.SCENE_UNDERSTANDING_COARSE permission listed in their manifest as this extension tracks planes in the environment. The android.permission.SCENE_UNDERSTANDING_COARSE permission is considered a dangerous permission.

(protection level: dangerous)

12.72.2. Runtime Support

If the runtime is capable of subsuming one plane into another, it **must** indicate this by enumerating XR_SPATIAL_COMPONENT_TYPE_SUBSUMED_BY_ANDROID as a supported component for XR_SPATIAL_CAPABILITY_PLANE_TRACKING_EXT capability in xrEnumerateSpatialCapabilityComponentTypesEXT.

All the component data of the XR_SPATIAL_COMPONENT_TYPE_SUBSUMED_BY_ANDROID attached entity **must** be identical to the entity that subsumes it.

12.72.3. Subsumed By Component

Component data

XR_SPATIAL_COMPONENT_TYPE_SUBSUMED_BY_ANDROID uses the XrSpatialEntityIdEXT structure for its data, which represents the ID of the subsuming entity.

Component list struct to query data

The XrSpatialComponentSubsumedByListANDROID structure is defined as:

```
// Provided by XR_ANDROID_spatial_component_subsumed_by
typedef struct XrSpatialComponentSubsumedByListANDROID {
    XrStructureType          type;
    void*                    next;
    uint32_t                 subsumedUniqueIdCount;
    XrSpatialEntityIdEXT*    subsumedUniqueIds;
} XrSpatialComponentSubsumedByListANDROID;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `subsumedUniqueIdCount` is a `uint32_t` describing the count of elements in the `subsumedUniqueIds` array.
- `subsumedUniqueIds` is an array of `XrSpatialEntityIdEXT`.

The application **can** query the `XR_SPATIAL_COMPONENT_TYPE_SUBSUMED_BY_ANDROID` component of the spatial entities in an XrSpatialSnapshotEXT by adding `XR_TYPE_SPATIAL_COMPONENT_SUBSUMED_BY_LIST_ANDROID` to the next chain of the XrSpatialComponentDataQueryResultEXT.

The runtime **must** return `XR_ERROR_VALIDATION_FAILURE` from xrQuerySpatialComponentDataEXT if `XR_TYPE_SPATIAL_COMPONENT_SUBSUMED_BY_LIST_ANDROID` is in the next chain of XrSpatialComponentDataQueryResultEXT::`next` but `XR_SPATIAL_COMPONENT_TYPE_SUBSUMED_BY_ANDROID` is not included in XrSpatialComponentDataQueryConditionEXT::`componentTypes`.

The runtime **must** return `XR_ERROR_SIZE_INSUFFICIENT` from xrQuerySpatialComponentDataEXT if `subsumedUniqueIdCount` is less than XrSpatialComponentDataQueryResultEXT::`entityIdCountOutput`.

Valid Usage (Implicit)

- The `XR_ANDROID_spatial_component_subsumed_by` extension **must** be enabled prior to using XrSpatialComponentSubsumedByListANDROID
- `type` **must** be `XR_TYPE_SPATIAL_COMPONENT_SUBSUMED_BY_LIST_ANDROID`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `subsumedUniqueIds` **must** be a pointer to an array of `subsumedUniqueIdCount` `XrSpatialEntityIdEXT` values
- The `subsumedUniqueIdCount` parameter **must** be greater than `0`

Configuration

If `XR_SPATIAL_COMPONENT_TYPE_SUBSUMED_BY_ANDROID` is enumerated in XrSpatialCapabilityComponentTypesEXT::`componentTypes` for `XR_SPATIAL_CAPABILITY_PLANE_TRACKING_EXT` capability, the application **can** enable it by including the enum in the XrSpatialCapabilityConfigurationBaseHeaderEXT::`enabledComponents` list of the XrSpatialCapabilityConfigurationBaseHeaderEXT derived structure of the capability that supports this component.

Filter Subsumed Entities

The XrSpatialDiscoveryUniqueEntitiesFilterANDROID structure is defined as:

```
// Provided by XR_ANDROID_spatial_component_subsumed_by
typedef struct XrSpatialDiscoveryUniqueEntitiesFilterANDROID {
    XrStructureType    type;
    const void*        next;
} XrSpatialDiscoveryUniqueEntitiesFilterANDROID;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.

The application **can** include XrSpatialDiscoveryUniqueEntitiesFilterANDROID in the `next` chain of XrSpatialDiscoverySnapshotCreateInfoEXT to get a snapshot with entities that are not subsumed by another entity.

If applications chain XrSpatialDiscoveryUniqueEntitiesFilterANDROID to XrSpatialDiscoverySnapshotCreateInfoEXT while including `XR_SPATIAL_COMPONENT_TYPE_SUBSUMED_BY_ANDROID` component in the XrSpatialDiscoverySnapshotCreateInfoEXT::`componentTypes`, the runtime **must** return `XR_ERROR_VALIDATION_FAILURE`.

If application chains XrSpatialDiscoveryUniqueEntitiesFilterANDROID to XrSpatialDiscoverySnapshotCreateInfoEXT but does not list any components in XrSpatialDiscoverySnapshotCreateInfoEXT::`componentTypes`, the runtime **must** include all the spatial entities in the snapshot that have the set of components which are enumerated in XrSpatialCapabilityConfigurationBaseHeaderEXT::`enabledComponents` for the capabilities configured for `spatialContext`, except entities that have the `XR_SPATIAL_COMPONENT_TYPE_SUBSUMED_BY_ANDROID` component.

Valid Usage (Implicit)

- The `XR_ANDROID_spatial_component_subsumed_by` extension **must** be enabled prior to using XrSpatialDiscoveryUniqueEntitiesFilterANDROID
- `type` **must** be `XR_TYPE_SPATIAL_DISCOVERY_UNIQUE_ENTITIES_FILTER_ANDROID`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

12.72.4. Example code

Configure Plane Tracking Capability

The following example code demonstrates how to create a spatial context with XR_SPATIAL_CAPABILITY_PLANE_TRACKING_EXT capability that supports XR_SPATIAL_COMPONENT_TYPE_SUBSUMED_BY_ANDROID.

```
// Check runtime supported capabilities
uint32_t capabilityCount = 0;
CHK_XR(xrEnumerateSpatialCapabilitiesEXT(instance, systemId, 0, &capabilityCount,
nullptr));
std::vector<XrSpatialCapabilityEXT> capabilities(capabilityCount);
CHK_XR(xrEnumerateSpatialCapabilitiesEXT(instance, systemId, capabilityCount,
&capabilityCount, capabilities.data()));

if (std::find(capabilities.begin(), capabilities.end(),
XR_SPATIAL_CAPABILITY_PLANE_TRACKING_EXT) == capabilities.end()) {
  return;
}

// Enumerate supported components for plane tracking capability
XrSpatialCapabilityComponentTypesEXT planeComponents{
  .type = XR_TYPE_SPATIAL_CAPABILITY_COMPONENT_TYPES_EXT,
};
CHK_XR(xrEnumerateSpatialCapabilityComponentTypesEXT(instance, systemId,
XR_SPATIAL_CAPABILITY_PLANE_TRACKING_EXT, &planeComponents));
std::vector<XrSpatialComponentTypeEXT> planeCapabilityComponents(planeComponents
.componentTypeCountOutput);
planeComponents.componentTypeCapacityInput = planeCapabilityComponents.size();
planeComponents.componentTypes = planeCapabilityComponents.data();
CHK_XR(xrEnumerateSpatialCapabilityComponentTypesEXT(instance, systemId,
XR_SPATIAL_CAPABILITY_PLANE_TRACKING_EXT, &planeComponents));

const auto supportsComponent = [&planeCapabilityComponents](XrSpatialComponentTypeEXT
component) {
  return std::find(planeCapabilityComponents.begin(), planeCapabilityComponents.end(),
component) != planeCapabilityComponents.end();
};


std::vector<XrSpatialComponentTypeEXT> planeTrackingComponents {
  XR_SPATIAL_COMPONENT_TYPE_BOUNDED_2D_EXT,
  XR_SPATIAL_COMPONENT_TYPE_PLANE_ALIGNMENT_EXT,
};

if (supportsComponent(XR_SPATIAL_COMPONENT_TYPE_SUBSUMED_BY_ANDROID)) {
```

```
  planeTrackingComponents.push_back(XR_SPATIAL_COMPONENT_TYPE_SUBSUMED_BY_ANDROID);
}

// Create capability config for plane tracking
XrSpatialCapabilityConfigurationPlaneTrackingEXT planeTrackingConfig {
  .type = XR_TYPE_SPATIAL_CAPABILITY_CONFIGURATION_PLANE_TRACKING_EXT,
  .next = nullptr,
  .capability = XR_SPATIAL_CAPABILITY_PLANE_TRACKING_EXT,
  .enabledComponentCount = (uint32_t)planeTrackingComponents.size(),
  .enabledComponents = planeTrackingComponents.data(),
};

// Create spatial context
std::vector<const XrSpatialCapabilityConfigurationBaseHeaderEXT*> capabilityConfigs;
capabilityConfigs.push_back(reinterpret_cast<const
XrSpatialCapabilityConfigurationBaseHeaderEXT*>(&planeTrackingConfig));

XrSpatialContextCreateInfoEXT contextCreateInfo {
  .type = XR_TYPE_SPATIAL_CONTEXT_CREATE_INFO_EXT,
  .next = nullptr,
  .capabilityConfigCount = (uint32_t)capabilityConfigs.size(),
  .capabilityConfigs = capabilityConfigs.data(),
};

CHK_XR(xrCreateSpatialContextAsyncEXT(session, &contextCreateInfo, &future))

// Completes creating spatial context
XrCreateSpatialContextCompletionEXT contextCompletion{
XR_TYPE_CREATE_SPATIAL_CONTEXT_COMPLETION_EXT};

CHK_XR(xrCreateSpatialContextCompleteEXT(session, future, &contextCompletion))
```

Query Component Data

The following example code demonstrates how to query `XR_SPATIAL_COMPONENT_TYPE_SUBSUMED_BY_ANDROID` component data from context configured with `XR_SPATIAL_CAPABILITY_PLANE_TRACKING_EXT`.

```
// previously created
XrSpatialSnapshotEXT snapshot;

// Query subsumed_by components
std::array<XrSpatialComponentTypeEXT, 1> enabledComponents = {
  XR_SPATIAL_COMPONENT_TYPE_SUBSUMED_BY_ANDROID
};

XrSpatialComponentDataQueryConditionEXT queryCond {
  .type = XR_TYPE_SPATIAL_COMPONENT_DATA_QUERY_CONDITION_EXT,
  .componentTypeCount = 1,
  .componentTypes     = enabledComponents.data(),
};

XrSpatialComponentDataQueryResultEXT queryResult {
  .type = XR_TYPE_SPATIAL_COMPONENT_DATA_QUERY_RESULT_EXT,
};

CHK_XR(xrQuerySpatialComponentDataEXT(
           snapshot, &queryCond, &queryResult));

// Query again with allocated memory
std::vector<XrSpatialEntityIdEXT> subsumedUniqueIds;
subsumedUniqueIds.resize(queryResult.entityIdCountOutput);
XrSpatialComponentSubsumedByListANDROID subsumedByList {
  .type = XR_TYPE_SPATIAL_COMPONENT_SUBSUMED_BY_LIST_ANDROID,
  .subsumedUniqueIdCount = static_cast<uint32_t>(subsumedUniqueIds.size()),
  .subsumedUniqueIds = subsumedUniqueIds.data(),
};

queryResult.next = &subsumedByList;
CHK_XR(xrQuerySpatialComponentDataEXT(
           snapshot, &queryCond, &queryResult));

for (uint32_t i = 0; i < queryResult.entityIdCountOutput; ++i) {
  // Plane was subsumed, remove it from processing logic
  // removeEntityFromProcessingLogic(queryResult.entityIds[i], /*replaceWith=*/
subsumedByList.subsumedUniqueIds[i]);
}

// Cleanup
xrDestroySpatialSnapshotEXT(snapshot);
```

Filter Out Subsumed Entities

The following example code demonstrates how to filter out entities with

XR_SPATIAL_COMPONENT_TYPE_SUBSUMED_BY_ANDROID component attached from discovery snapshot using the filter, as well as query the entity id of the subsuming entities.

```
// Init filter
XrSpatialDiscoveryUniqueEntitiesFilterANDROID filter {
  .type = XR_TYPE_SPATIAL_DISCOVERY_UNIQUE_ENTITIES_FILTER_ANDROID,
};

// Chain filter to the snapshot create info
// WARNING: Chaining the filter while include subsumed_by component in the
// componentTypes is invalid
XrSpatialDiscoverySnapshotCreateInfoEXT discoverySnapshotCreateInfo {
  .type = XR_TYPE_SPATIAL_DISCOVERY_SNAPSHOT_CREATE_INFO_EXT,
  .next = &filter
};

XrFutureEXT future {XR_NULL_FUTURE_EXT};
CHK_XR(xrCreateSpatialDiscoverySnapshotAsyncEXT(spatialContext,
&discoverySnapshotCreateInfo, &future))

waitUntilReady(future);

// Complete async operation.
XrCreateSpatialDiscoverySnapshotCompletionInfoEXT
  createSnapshotCompletionInfo {
    .type   = XR_TYPE_CREATE_SPATIAL_DISCOVERY_SNAPSHOT_COMPLETION_INFO_EXT,
    .baseSpace = space,
    .time      = updateTime,
    .future    = future,
  };

XrCreateSpatialDiscoverySnapshotCompletionEXT completion {
  .type = XR_TYPE_CREATE_SPATIAL_DISCOVERY_SNAPSHOT_COMPLETION_EXT,
};

CHK_XR(xrCreateSpatialDiscoverySnapshotCompleteEXT(
           spatialContext, &createSnapshotCompletionInfo,
           &completion));

if(completion.futureResult != XR_SUCCESS) return;

// Subsumed entities has already been filtered out in this snapshot,
// now query the various components of the entities.

// Cleanup
xrDestroySpatialSnapshotEXT(completion.snapshot);
```

12.72.5. New Structures

- Extending XrSpatialComponentDataQueryResultEXT:
  - XrSpatialComponentSubsumedByListANDROID
- Extending XrSpatialDiscoverySnapshotCreateInfoEXT:
  - XrSpatialDiscoveryUniqueEntitiesFilterANDROID

12.72.6. New Enum Constants

- `XR_ANDROID_SPATIAL_COMPONENT_SUBSUMED_BY_EXTENSION_NAME`
- `XR_ANDROID_spatial_component_subsumed_by_SPEC_VERSION`
- Extending XrSpatialComponentTypeEXT:
  - `XR_SPATIAL_COMPONENT_TYPE_SUBSUMED_BY_ANDROID`
- Extending XrStructureType:
  - `XR_TYPE_SPATIAL_COMPONENT_SUBSUMED_BY_LIST_ANDROID`
  - `XR_TYPE_SPATIAL_DISCOVERY_UNIQUE_ENTITIES_FILTER_ANDROID`

12.72.7. Issues

12.72.8. Version History

- Revision 1, 2025-11-19 (Brian Chen)
  - Initial extension description.

12.73. XR_ANDROID_spatial_discovery_bounds

Name String

`XR_ANDROID_spatial_discovery_bounds`

Extension Type

Instance extension

Registered Extension Number

762

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

XR_EXT_spatial_entity

Contributors

Nihav Jain, Google
Natalie Fleury, Meta
Yuichi Taguchi, Meta
Yin Li, Microsoft
Ron Bessems, Meta
Karthik Kadappan, Magic Leap
Jimmy Alamparambil, ByteDance
Zhipeng Liu, ByteDance
Jun Yan, ByteDance

12.73.1. Overview

This extension adds definitions for spatial bounds that allows applications to filter discovered spatial entities when calling xrCreateSpatialDiscoverySnapshotAsyncEXT.

12.73.2. Spatial Bounds

The XrSpatialBoundsSpherefANDROID structure is defined as:

```
// Provided by XR_ANDROID_spatial_discovery_bounds
typedef struct XrSpatialBoundsSpherefANDROID {
    XrStructureType    type;
    const void*        next;
    XrSpace            space;
    XrTime             time;
    XrSpheref          sphere;
} XrSpatialBoundsSpherefANDROID;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain.
- space is the XrSpace that any poses for the bounds are defined in.
- time is an XrTime at which the poses for the bounds are defined.
- sphere is an XrSpheref which describes the spherical volume within which the runtime **must** constrain its discovery of spatial entities.

Spatial capabilities that support XR_SPATIAL_CAPABILITY_FEATURE_SPHERE_BOUNDS_FILTER_ANDROID (as

enumerated by xrEnumerateSpatialCapabilityFeaturesEXT) allow the application to restrict the discovery to spatial entities whose location or bounds are within or intersecting with sphere by chaining XrSpatialBoundsSpherefANDROID to XrSpatialDiscoverySnapshotCreateInfoEXT::next. For a discovery snapshot in which the applicable capability supports XR_SPATIAL_CAPABILITY_FEATURE_SPHERE_BOUNDS_FILTER_ANDROID, if this structure is present in the next chain, the runtime **must** not include any spatial entities whose location or bounds are entirely outside of sphere.

It is important to note that the inclusion of XrSpatialBoundsSpherefANDROID in the next chain of XrSpatialDiscoverySnapshotCreateInfoEXT acts as an 'AND' condition with the other filters provided. Entities coming from spatial capabilities that do not support XR_SPATIAL_CAPABILITY_FEATURE_SPHERE_BOUNDS_FILTER_ANDROID **must** not be included in the discovery snapshot. This behavior is different from the rest of the OpenXR specification where unrecognized or unsupported structures in the next pointer chain are ignored by the runtime. Also, as described in the Next Chain Structure Uniqueness section, the runtime will only process the first occurrence of the XrSpatialBoundsSpherefANDROID structure in the next chain.

Valid Usage (Implicit)

- The XR_ANDROID_spatial_discovery_bounds extension **must** be enabled prior to using XrSpatialBoundsSpherefANDROID
- type **must** be XR_TYPE_SPATIAL_BOUNDS_SPHEREF_ANDROID
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- space **must** be a valid XrSpace handle

The XrSpatialBoundsBoxfANDROID structure is defined as:

```
// Provided by XR_ANDROID_spatial_discovery_bounds
typedef struct XrSpatialBoundsBoxfANDROID {
    XrStructureType    type;
    const void*        next;
    XrSpace            space;
    XrTime             time;
    XrBoxf             box;
} XrSpatialBoundsBoxfANDROID;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `space` is the XrSpace that any poses for the bounds are defined in.
- `time` is an `XrTime` at which the poses for the bounds are defined.
- `box` is an XrBoxf which describes the box volume within which the runtime **must** constrain its discovery of spatial entities.

Spatial capabilities that support `XR_SPATIAL_CAPABILITY_FEATURE_BOX_BOUNDS_FILTER_ANDROID` (as enumerated by xrEnumerateSpatialCapabilityFeaturesEXT) allow the application to restrict the discovery to spatial entities whose location or bounds are within or intersecting with `box` by chaining XrSpatialBoundsBoxfANDROID to XrSpatialDiscoverySnapshotCreateInfoEXT::`next`. For a discovery snapshot in which the applicable capability supports `XR_SPATIAL_CAPABILITY_FEATURE_BOX_BOUNDS_FILTER_ANDROID`, if this structure is present in the `next` chain, the runtime **must** not include any spatial entities whose location or bounds are entirely outside of `box`.

It is important to note that the inclusion of XrSpatialBoundsBoxfANDROID in the next chain of XrSpatialDiscoverySnapshotCreateInfoEXT acts as an 'AND' condition with the other filters provided. Entities coming from spatial capabilities that do not support `XR_SPATIAL_CAPABILITY_FEATURE_BOX_BOUNDS_FILTER_ANDROID` **must** not be included in the discovery snapshot. This behavior is different from the rest of the OpenXR specification where unrecognized or unsupported structures in the next pointer chain are ignored by the runtime. Also, as described in the Next Chain Structure Uniqueness section, the runtime will only process the first occurrence of the XrSpatialBoundsBoxfANDROID structure in the next chain.

Valid Usage (Implicit)

- The `XR_ANDROID_spatial_discovery_bounds` extension **must** be enabled prior to using XrSpatialBoundsBoxfANDROID
- `type` **must** be `XR_TYPE_SPATIAL_BOUNDS_BOXF_ANDROID`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `space` **must** be a valid XrSpace handle

The XrSpatialBoundsFrustumfANDROID structure is defined as:

```
// Provided by XR_ANDROID_spatial_discovery_bounds
typedef struct XrSpatialBoundsFrustumfANDROID {
    XrStructureType    type;
    const void*        next;
    XrSpace            space;
    XrTime             time;
    XrFrustumf         frustum;
} XrSpatialBoundsFrustumfANDROID;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `space` is the XrSpace that any poses for the bounds are defined in.
- `time` is an `XrTime` at which the poses for the bounds are defined.
- `frustum` is an XrFrustumf which describes the frustum volume within which the runtime **must** constrain its discovery of spatial entities.

Spatial capabilities that support `XR_SPATIAL_CAPABILITY_FEATURE_FRUSTUM_BOUNDS_FILTER_ANDROID` (as enumerated by xrEnumerateSpatialCapabilityFeaturesEXT) allow the application to restrict the discovery to spatial entities whose location or bounds are within or intersecting with `frustum` by chaining XrSpatialBoundsFrustumfANDROID to XrSpatialDiscoverySnapshotCreateInfoEXT::`next`. For a discovery snapshot in which the applicable capability supports `XR_SPATIAL_CAPABILITY_FEATURE_FRUSTUM_BOUNDS_FILTER_ANDROID`, if this structure is present in the `next` chain, the runtime **must** not include any spatial entities whose location or bounds are entirely outside of `frustum`.

It is important to note that the inclusion of XrSpatialBoundsFrustumfANDROID in the next chain of XrSpatialDiscoverySnapshotCreateInfoEXT acts as an 'AND' condition with the other filters provided. Entities coming from spatial capabilities that do not support `XR_SPATIAL_CAPABILITY_FEATURE_FRUSTUM_BOUNDS_FILTER_ANDROID` **must** not be included in the discovery snapshot. This behavior is different from the rest of the OpenXR specification where unrecognized or unsupported structures in the next pointer chain are ignored by the runtime. Also, as described in the Next Chain Structure Uniqueness section, the runtime will only process the first occurrence of the XrSpatialBoundsFrustumfANDROID structure in the next chain.

Valid Usage (Implicit)

- The XR_ANDROID_spatial_discovery_bounds extension **must** be enabled prior to using XrSpatialBoundsFrustumfANDROID
- type **must** be XR_TYPE_SPATIAL_BOUNDS_FRUSTUMF_ANDROID
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- space **must** be a valid XrSpace handle

12.73.3. Example

The following code demonstrates how to use spatial bounds to filter the discovery of spatial entities.

```
// previously initialized & configured with the XR_SPATIAL_CAPABILITY_PLANE_TRACKING_EXT
capability.
XrSpatialContextEXT spatialContext;

// Enumerate supported features for XR_SPATIAL_CAPABILITY_PLANE_TRACKING_EXT capability
uint32_t featureCount = 0;
CHK_XR(xrEnumerateSpatialCapabilityFeaturesEXT(instance, systemId,
XR_SPATIAL_CAPABILITY_PLANE_TRACKING_EXT, 0, &featureCount, nullptr));
std::vector<XrSpatialCapabilityFeatureEXT> fooFeatures(featureCount);
CHK_XR(xrEnumerateSpatialCapabilityFeaturesEXT(instance, systemId,
XR_SPATIAL_CAPABILITY_PLANE_TRACKING_EXT, featureCount, &featureCount, fooFeatures.
data()));

// Check if spatial sphere bounds filter is supported
const auto supportsFeature = [&fooFeatures](XrSpatialCapabilityFeatureEXT feature) {
  return std::find(fooFeatures.begin(), fooFeatures.end(), feature) != fooFeatures.end();
};

const bool supportsSphereBoundsFilter = supportsFeature
(XR_SPATIAL_CAPABILITY_FEATURE_SPHERE_BOUNDS_FILTER_ANDROID);

// We want to look for entities in a sphere around the head with a radius of 2m.
XrSpatialBoundsSpherefANDROID sphereBounds {
  .type = XR_TYPE_SPATIAL_BOUNDS_SPHEREF_ANDROID,
  .space = viewSpace,
  .time = predictedDisplayTime,
  .sphere = {
    .center = {
      .orientation = {
        .w = 1.0f,
      },
    },
```

```
    .radius = 2.0f,
  },
};

// We want to look for entities that have the following components.
std::array<XrSpatialComponentTypeEXT, 1> snapshotComponents
{XR_SPATIAL_COMPONENT_TYPE_BOUNDED_2D_EXT};

XrSpatialDiscoverySnapshotCreateInfoEXT snapshotCreateInfo{
  .type = XR_TYPE_SPATIAL_DISCOVERY_SNAPSHOT_CREATE_INFO_EXT,
  .next = supportsSphereBoundsFilter ? &sphereBounds : nullptr,
  .componentTypeCount = snapshotComponents.size(),
  .componentTypes = snapshotComponents.data(),
};

XrFutureEXT future = XR_NULL_FUTURE_EXT;
CHK_XR(xrCreateSpatialDiscoverySnapshotAsyncEXT(spatialContext, &snapshotCreateInfo,
&future));
```

12.73.4. New Structures

- Extending XrSpatialDiscoverySnapshotCreateInfoEXT:
  - XrSpatialBoundsBoxfANDROID
  - XrSpatialBoundsFrustumfANDROID
  - XrSpatialBoundsSpherefANDROID

12.73.5. New Enum Constants

- XR_ANDROID_SPATIAL_DISCOVERY_BOUNDS_EXTENSION_NAME
- XR_ANDROID_spatial_discovery_bounds_SPEC_VERSION
- Extending XrSpatialCapabilityFeatureEXT:
  - XR_SPATIAL_CAPABILITY_FEATURE_BOX_BOUNDS_FILTER_ANDROID
  - XR_SPATIAL_CAPABILITY_FEATURE_FRUSTUM_BOUNDS_FILTER_ANDROID
  - XR_SPATIAL_CAPABILITY_FEATURE_SPHERE_BOUNDS_FILTER_ANDROID
- Extending XrStructureType:
  - XR_TYPE_SPATIAL_BOUNDS_BOXF_ANDROID
  - XR_TYPE_SPATIAL_BOUNDS_FRUSTUMF_ANDROID
  - XR_TYPE_SPATIAL_BOUNDS_SPHEREF_ANDROID

12.73.6. Issues

- Why is the support for spatial bounds dependent on an XrSpatialCapabilityFeatureEXT definition?
  - Resolved
  - Answer: Not every spatial capability is capable of supporting spatial bounds. An example of this is a hypothetical global container entity that does not have any spatial position or bounds of its own but merely helps collect and organize the other entities inside it. The XrSpatialCapabilityFeatureEXT values enumerated by the capability helps inform the app about the support for spatial bounds filtering.

12.73.7. Version History

- Revision 1, 2025-05-03 (Nihav Jain, Google)
  - Initial extension description

12.74. XR_ANDROID_spatial_discovery_raycast

Name String

`XR_ANDROID_spatial_discovery_raycast`

Extension Type

Instance extension

Registered Extension Number

787

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

`XR_EXT_spatial_entity`

Last Modified Date

2025-08-12

IP Status

No known IP claims.

Contributors

Brian Chen, Google

Levana Chen, Google
Kyle Chen, Google
Nihav Jain, Google
Spencer Quin, Google
Natalie Fleury, Meta
Yuichi Taguchi, Meta
Jimmy Alamparambil, ByteDance
Zhipeng Liu, ByteDance

12.74.1. Overview

This extension builds on XR_EXT_spatial_entity and provides raycast functionality for existing spatial capabilities, such as XR_SPATIAL_CAPABILITY_PLANE_TRACKING_EXT, to discover spatial entities.

This extension also introduces a new spatial capability for raycasting against depth buffer.

Permissions

Required permissions depending on the capabilities the Android application uses. If XR_SPATIAL_CAPABILITY_DEPTH_RAYCAST_ANDROID is used, the application **must** have android.permission.SCENE_UNDERSTANDING_FINE permission. If XR_SPATIAL_CAPABILITY_PLANE_TRACKING_EXT is used, the application **must** have android.permission.SCENE_UNDERSTANDING_COARSE permission. The android.permission.SCENE_UNDERSTANDING_FINE also implies the android.permission.SCENE_UNDERSTANDING_COARSE permission, if the application uses both capabilities, the application does not need to request both permissions separately.

Both android.permission.SCENE_UNDERSTANDING_COARSE and android.permission.SCENE_UNDERSTANDING_FINE are considered dangerous permissions.

(protection level: dangerous)

12.74.2. Runtime Support

If the runtime supports discovery of spatial entities via raycasting, it **must** indicate this by enumerating XR_SPATIAL_COMPONENT_TYPE_RAYCAST_RESULT_ANDROID in xrEnumerateSpatialCapabilityComponentTypesEXT for the capabilities that support discovery by raycast.

If the runtime supports raycasting against the depth buffer, it **must** enumerate XR_SPATIAL_CAPABILITY_DEPTH_RAYCAST_ANDROID in xrEnumerateSpatialCapabilitiesEXT.

12.74.3. Discover spatial entities by raycasting

The XrSpatialRaycastInfoANDROID structure is defined as:

```
// Provided by XR_ANDROID_spatial_discovery_raycast
typedef struct XrSpatialRaycastInfoANDROID {
    XrStructureType    type;
    const void*        next;
    XrSpace            space;
    XrTime             time;
    XrVector3f         origin;
    XrVector3f         direction;
    float              maxDistance;
} XrSpatialRaycastInfoANDROID;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `space` is the XrSpace in which the pose of XrSpatialRaycastResultDataANDROID will be located.
- `time` is an `XrTime` at which the pose of XrSpatialRaycastResultDataANDROID defined.
- `origin` is an XrVector3f which describes the origin of the ray.
- `direction` is an XrVector3f which describes the direction of the ray.
- `maxDistance` is the distance in meters that application **can** optionally specify to limit the discovery of entities that are within the specified distance from the `origin`, along `direction`.

An application **can** include XrSpatialRaycastInfoANDROID in the XrSpatialDiscoverySnapshotCreateInfoEXT::`next` chain to discover spatial entities that intersect with the ray defined in this struct.

If an application uses XrSpatialRaycastInfoANDROID with an XrSpatialContextEXT that was configured without enabling `XR_SPATIAL_COMPONENT_TYPE_RAYCAST_RESULT_ANDROID` for any capability, the runtime **must** return `XR_ERROR_SPATIAL_COMPONENT_NOT_ENABLED_EXT` from xrCreateSpatialDiscoverySnapshotAsyncEXT or xrCreateSpatialRaycastSnapshotANDROID.

The runtime **must** include the component data and ids of only those spatial entities in XrSpatialSnapshotEXT that come from capabilities that support the `XR_SPATIAL_COMPONENT_TYPE_RAYCAST_RESULT_ANDROID` component.

If an application sets `maxDistance` to a positive value, the runtime **must** not include any hit spatial entities in the snapshot that are more than `maxDistance` away from the panme:origin along `direction`. If

maxDistance is set to 0 or a negative value, the runtime **must** consider the ray unbounded.

Valid Usage (Implicit)

- The XR_ANDROID_spatial_discovery_raycast extension **must** be enabled prior to using XrSpatialRaycastInfoANDROID
- type **must** be XR_TYPE_SPATIAL_RAYCAST_INFO_ANDROID
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- space **must** be a valid XrSpace handle

12.74.4. Raycast Result Component

The XrSpatialRaycastResultDataANDROID structure is defined as:

```
// Provided by XR_ANDROID_spatial_discovery_raycast
typedef struct XrSpatialRaycastResultDataANDROID {
    XrPosef    hitPose;
    float      distanceSquared;
} XrSpatialRaycastResultDataANDROID;
```

Member Descriptions

- hitPose is an XrPosef defining the point where the ray intersects with the spatial entity.
- distanceSquared is the squared of the distance from origin of the ray to the intersection point.

The XR_SPATIAL_COMPONENT_TYPE_RAYCAST_RESULT_ANDROID component will only be present in an XrSpatialSnapshotEXT created from xrCreateSpatialDiscoverySnapshotAsyncEXT where XrSpatialRaycastInfoANDROID is included in the XrSpatialDiscoverySnapshotCreateInfoEXT::next chain or in an XrSpatialSnapshotEXT created from xrCreateSpatialRaycastSnapshotANDROID.

The runtime **must** not include this component in any XrSpatialSnapshotEXT that is created using xrCreateSpatialUpdateSnapshotEXT.

The runtime **must** return XR_ERROR_VALIDATION_FAILURE from xrCreateSpatialUpdateSnapshotEXT if the application includes XR_SPATIAL_COMPONENT_TYPE_RAYCAST_RESULT_ANDROID in XrSpatialUpdateSnapshotCreateInfoEXT::componentTypes.

If XR_SPATIAL_COMPONENT_TYPE_RAYCAST_RESULT_ANDROID is enabled only for certain capabilities, the runtime **must** include only spatial entities provided by those capabilities in the snapshot. For example, if an object and a plane lines up in the same direction and XR_SPATIAL_COMPONENT_TYPE_RAYCAST_RESULT_ANDROID is enabled only for

XR_SPATIAL_CAPABILITY_PLANE_TRACKING_EXT, only the plane will be discovered.

The hitPose of the XR_SPATIAL_COMPONENT_TYPE_RAYCAST_RESULT_ANDROID will be located in XrCreateSpatialDiscoverySnapshotCompletionInfoEXT::baseSpace at XrCreateSpatialDiscoverySnapshotCompletionInfoEXT::time. The positive Z axis is the normal of the hit surface, and the negative Y axis roughly points toward the ray origin.

Valid Usage (Implicit)

- The XR_ANDROID_spatial_discovery_raycast extension **must** be enabled prior to using XrSpatialRaycastResultDataANDROID

Component list struct to query data

The XrSpatialComponentRaycastResultListANDROID structure is defined as:

```
// Provided by XR_ANDROID_spatial_discovery_raycast
typedef struct XrSpatialComponentRaycastResultListANDROID {
    XrStructureType                       type;
    void*                                 next;
    uint32_t                              raycastResultCount;
    XrSpatialRaycastResultDataANDROID*    raycastResults;
} XrSpatialComponentRaycastResultListANDROID;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain.
- raycastResultCount is a uint32_t describing the count of elements in the raycastResults array.
- raycastResults is an array of XrSpatialRaycastResultDataANDROID.

To query the raycast result component of the spatial entities in an XrSpatialSnapshotEXT, include XR_TYPE_SPATIAL_COMPONENT_RAYCAST_RESULT_LIST_ANDROID in XrSpatialComponentDataQueryConditionEXT::componentTypes and add XrSpatialComponentRaycastResultListANDROID to the XrSpatialComponentDataQueryResultEXT::next chain.

The runtime **must** return XR_ERROR_VALIDATION_FAILURE from xrQuerySpatialComponentDataEXT if XrSpatialComponentRaycastResultListANDROID is in the XrSpatialComponentDataQueryResultEXT::next chain but XR_SPATIAL_COMPONENT_TYPE_RAYCAST_RESULT_ANDROID is not included in XrSpatialComponentDataQueryConditionEXT::componentTypes.

The runtime **must** return XR_ERROR_SIZE_INSUFFICIENT from xrQuerySpatialComponentDataEXT if raycastResultCount is less than XrSpatialComponentDataQueryResultEXT::entityIdCountOutput.

The returned raycastResults are unordered. An application **can** use XrSpatialRaycastResultDataANDROID::distanceSquared to sort the results by distance.

Valid Usage (Implicit)

- The XR_ANDROID_spatial_discovery_raycast extension **must** be enabled prior to using XrSpatialComponentRaycastResultListANDROID
- type **must** be XR_TYPE_SPATIAL_COMPONENT_RAYCAST_RESULT_LIST_ANDROID
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- raycastResults **must** be a pointer to an array of raycastResultCount XrSpatialRaycastResultDataANDROID structures
- The raycastResultCount parameter **must** be greater than 0

Configuration

If XR_SPATIAL_COMPONENT_TYPE_RAYCAST_RESULT_ANDROID is enumerated in XrSpatialCapabilityComponentTypesEXT::componentTypes for some capability, the application **can** enable it by including the enum value in the XrSpatialCapabilityConfigurationBaseHeaderEXT::enabledComponents list. This component does not require any special configuration to be included in the next chain of XrSpatialCapabilityConfigurationBaseHeaderEXT.

The runtime **must** return XR_ERROR_VALIDATION_FAILURE from xrCreateSpatialUpdateSnapshotEXT if the application includes XR_SPATIAL_COMPONENT_TYPE_RAYCAST_RESULT_ANDROID in XrSpatialUpdateSnapshotCreateInfoEXT::componentTypes. This component is specific to discovery snapshots.

12.74.5. Depth Raycast Spatial Capability

Configuration

The XrSpatialCapabilityConfigurationDepthRaycastANDROID structure is defined as:

```
// Provided by XR_ANDROID_spatial_discovery_raycast
typedef struct XrSpatialCapabilityConfigurationDepthRaycastANDROID {
    XrStructureType                     type;
    const void*                         next;
    XrSpatialCapabilityEXT              capability;
    uint32_t                            enabledComponentCount;
    const XrSpatialComponentTypeEXT*    enabledComponents;
} XrSpatialCapabilityConfigurationDepthRaycastANDROID;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `capability` is an XrSpatialCapabilityEXT and **must** be `XR_SPATIAL_CAPABILITY_DEPTH_RAYCAST_ANDROID`.
- `enabledComponentCount` is a `uint32_t` describing the count of elements in the `enabledComponents` array. It **must** be greater than 0.
- `enabledComponents` is a pointer to an array of XrSpatialComponentTypeEXT.

Anpplications **can** enable the `XR_SPATIAL_CAPABILITY_DEPTH_RAYCAST_ANDROID` spatial capability by including a pointer to an XrSpatialCapabilityConfigurationDepthRaycastANDROID structure in XrSpatialContextCreateInfoEXT::`capabilityConfigs`.

The runtime **must** return `XR_ERROR_VALIDATION_FAILURE` if `capability` is not `XR_SPATIAL_CAPABILITY_DEPTH_RAYCAST_ANDROID`.

The ID of entities discovered via this capability is always XR_NULL_SPATIAL_ENTITY_ID_EXT. This also means that application **can** not create an XrSpatialEntityEXT handle for these entities, and thus they **can** not be included in XrSpatialUpdateSnapshotCreateInfoEXT::`entities` to create a update snapshot

Valid Usage (Implicit)

- The XR_ANDROID_spatial_discovery_raycast extension **must** be enabled prior to using XrSpatialCapabilityConfigurationDepthRaycastANDROID
- type **must** be XR_TYPE_SPATIAL_CAPABILITY_CONFIGURATION_DEPTH_RAYCAST_ANDROID
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- capability **must** be a valid XrSpatialCapabilityEXT value
- enabledComponents **must** be a pointer to an array of enabledComponentCount valid XrSpatialComponentTypeEXT values
- The enabledComponentCount parameter **must** be greater than 0

Guaranteed Components

A runtime that supports XR_SPATIAL_CAPABILITY_DEPTH_RAYCAST_ANDROID, **must** support the following spatial components as guaranteed components of the entities discovered by this capability and **must** enumerate them in xrEnumerateSpatialCapabilityComponentTypesEXT -

- XR_SPATIAL_COMPONENT_TYPE_RAYCAST_RESULT_ANDROID

12.74.6. Synchronous Raycast

The xrCreateSpatialRaycastSnapshotANDROID function is defined as:

```
// Provided by XR_ANDROID_spatial_discovery_raycast
XrResult xrCreateSpatialRaycastSnapshotANDROID(
    XrSpatialContextEXT                          spatialContext,
    const XrSpatialRaycastSnapshotCreateInfoANDROID* createInfo,
    XrSpatialSnapshotEXT*                        snapshot);
```

Parameter Descriptions

- spatialContext is an XrSpatialContextEXT previously created by using xrCreateSpatialContextAsyncEXT.
- createInfo is a pointer to an XrSpatialRaycastSnapshotCreateInfoANDROID.
- snapshot is a pointer to an XrSpatialSnapshotEXT.

While xrCreateSpatialDiscoverySnapshotAsyncEXT is sufficient for most use cases, it is considered slow for applications that require instant raycast results. xrCreateSpatialRaycastSnapshotANDROID starts a synchronous raycasting to create a snapshot, which allows applications to start querying

raycast results immediately.

The runtime **must** return `XR_ERROR_SPATIAL_COMPONENT_NOT_ENABLED_EXT` if any of the XrSpatialComponentTypeEXT in XrSpatialRaycastSnapshotCreateInfoANDROID::`componentTypes` are not enabled for the spatial capabilities passed to XrSpatialContextCreateInfoEXT::`capabilityConfigs` when creating `spatialContext`.

If the application does not provide a list of spatial component types in XrSpatialRaycastSnapshotCreateInfoANDROID::`componentTypes`, the runtime **must** include all the spatial entities in the snapshot that have the set of components which are enumerated in XrSpatialCapabilityConfigurationBaseHeaderEXT::enabledComponents for raycast supported capabilities configured for `spatialContext`. The runtime **must** include the data for all the enabled components of the raycast supported capabilities configured for `spatialContext`.

Valid Usage (Implicit)

- The `XR_ANDROID_spatial_discovery_raycast` extension **must** be enabled prior to calling xrCreateSpatialRaycastSnapshotANDROID
- `spatialContext` **must** be a valid XrSpatialContextEXT handle
- `createInfo` **must** be a pointer to a valid XrSpatialRaycastSnapshotCreateInfoANDROID structure
- `snapshot` **must** be a pointer to an XrSpatialSnapshotEXT handle

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_SPATIAL_COMPONENT_NOT_ENABLED_EXT
- XR_ERROR_TIME_INVALID
- XR_ERROR_VALIDATION_FAILURE

The XrSpatialRaycastSnapshotCreateInfoANDROID structure is defined as:

```
// Provided by XR_ANDROID_spatial_discovery_raycast
typedef struct XrSpatialRaycastSnapshotCreateInfoANDROID {
    XrStructureType                       type;
    const void*                           next;
    uint32_t                              componentTypeCount;
    const XrSpatialComponentTypeEXT*      componentTypes;
    const XrSpatialRaycastInfoANDROID*    raycastInfo;
} XrSpatialRaycastSnapshotCreateInfoANDROID;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `componentTypeCount` is a `uint32_t` describing the count of elements in the `componentTypes` array.
- `componentTypes` is an array of XrSpatialComponentTypeEXT.
- `raycastInfo` is the XrSpatialRaycastInfoANDROID structure describing the information of a ray.

Valid Usage (Implicit)

- The `XR_ANDROID_spatial_discovery_raycast` extension **must** be enabled prior to using XrSpatialRaycastSnapshotCreateInfoANDROID
- `type` **must** be `XR_TYPE_SPATIAL_RAYCAST_SNAPSHOT_CREATE_INFO_ANDROID`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- If `componentTypeCount` is not `0`, `componentTypes` **must** be a pointer to an array of `componentTypeCount` valid XrSpatialComponentTypeEXT values
- `raycastInfo` **must** be a pointer to a valid XrSpatialRaycastInfoANDROID structure

12.74.7. Example code

Configure Depth Raycast and Plane Tracking Capability

The following example code demonstrates how to create a spatial context with capabilities that support raycast functionality. For this example, `XR_SPATIAL_CAPABILITY_DEPTH_RAYCAST_ANDROID` and `XR_SPATIAL_CAPABILITY_PLANE_TRACKING_EXT` are used.

```
// Check runtime supported capabilities
uint32_t capabilityCount;
CHK_XR(xrEnumerateSpatialCapabilitiesEXT(instance, systemId, 0, &capabilityCount,
nullptr));
std::vector<XrSpatialCapabilityEXT> capabilities(capabilityCount);
CHK_XR(xrEnumerateSpatialCapabilitiesEXT(instance, systemId, capabilityCount,
&capabilityCount, capabilities.data()));

if (std::find(capabilities.begin(), capabilities.end(),
XR_SPATIAL_CAPABILITY_DEPTH_RAYCAST_ANDROID) == capabilities.end()) {
  return;
}
```

```
if (std::find(capabilities.begin(), capabilities.end(),
XR_SPATIAL_CAPABILITY_PLANE_TRACKING_EXT) == capabilities.end()) {
  return;
}

// Create capability config for depth raycasting
std::vector<XrSpatialComponentTypeEXT> depthRaycastComponents {
  XR_SPATIAL_COMPONENT_TYPE_RAYCAST_RESULT_ANDROID
};

// Create capability config for depth raycasting
XrSpatialCapabilityConfigurationDepthRaycastANDROID depthRaycastConfig {
  .type = XR_TYPE_SPATIAL_CAPABILITY_CONFIGURATION_DEPTH_RAYCAST_ANDROID,
  .next = nullptr,
  .capability = XR_SPATIAL_CAPABILITY_DEPTH_RAYCAST_ANDROID,
  .enabledComponentCount = (uint32_t)depthRaycastComponents.size(),
  .enabledComponents = depthRaycastComponents.data(),
};

// Enumerate runtime supported components for plane tracking capability
XrSpatialCapabilityComponentTypesEXT capabilityComponentTypes {
  XR_TYPE_SPATIAL_CAPABILITY_COMPONENT_TYPES_EXT
};

bool isRaycastSupportedViaPlaneTracking;
std::vector<XrSpatialComponentTypeEXT> planeTrackingComponents {
  XR_SPATIAL_COMPONENT_TYPE_BOUNDED_2D_EXT,
  XR_SPATIAL_COMPONENT_TYPE_PLANE_ALIGNMENT_EXT,
};

CHK_XR(xrEnumerateSpatialCapabilityComponentTypesEXT(instance, systemId,
       XR_SPATIAL_CAPABILITY_PLANE_TRACKING_EXT, &capabilityComponentTypes));

planeTrackingComponents.resize(capabilityComponentTypes.componentTypeCountOutput);
capabilityComponentTypes.componentTypeCapacityInput = (uint32_t)planeTrackingComponents
.size();
capabilityComponentTypes.componentTypes = planeTrackingComponents.data();

CHK_XR(xrEnumerateSpatialCapabilityComponentTypesEXT(instance, systemId,
       XR_SPATIAL_CAPABILITY_PLANE_TRACKING_EXT, &capabilityComponentTypes));

// Check if raycast is supported by plane tracking
isRaycastSupportedViaPlaneTracking = std::find(planeTrackingComponents.begin(),
planeTrackingComponents.end(), XR_SPATIAL_COMPONENT_TYPE_RAYCAST_RESULT_ANDROID)
!= planeTrackingComponents.end();

// Create capability config for plane tracking
```

```
XrSpatialCapabilityConfigurationPlaneTrackingEXT planeTrackingConfig {
  .type = XR_TYPE_SPATIAL_CAPABILITY_CONFIGURATION_PLANE_TRACKING_EXT,
  .next = nullptr,
  .capability = XR_SPATIAL_CAPABILITY_PLANE_TRACKING_EXT,
  .enabledComponentCount = (uint32_t)planeTrackingComponents.size(),
  .enabledComponents = planeTrackingComponents.data(),
};

// Create spatial context
// Types of raycast is determined by configs included in XrSpatialContextCreateInfoEXT
// For this example, both plane raycast & depth raycast are performed
std::vector<const XrSpatialCapabilityConfigurationBaseHeaderEXT*> capabilityConfigs;
capabilityConfigs.push_back(reinterpret_cast<const
XrSpatialCapabilityConfigurationBaseHeaderEXT*>(&depthRaycastConfig));
if(isRaycastSupportedViaPlaneTracking) capabilityConfigs.push_back(reinterpret_cast<const
XrSpatialCapabilityConfigurationBaseHeaderEXT*>(&planeTrackingConfig));

XrSpatialContextCreateInfoEXT contextCreateInfo {
  .type = XR_TYPE_SPATIAL_CONTEXT_CREATE_INFO_EXT,
  .next = nullptr,
  .capabilityConfigCount = (uint32_t)capabilityConfigs.size(),
  .capabilityConfigs = capabilityConfigs.data(),
};

CHK_XR(xrCreateSpatialContextAsyncEXT(session, &contextCreateInfo, &future))

// Completes creating spatial context
XrCreateSpatialContextCompletionEXT contextCompletion{
XR_TYPE_CREATE_SPATIAL_CONTEXT_COMPLETION_EXT};

CHK_XR(xrCreateSpatialContextCompleteEXT(session, future, &contextCompletion))
```

Discover Spatial Entities & Query Component Data

The following example code demonstrates how to discover spatial entities for a context configured with `XR_SPATIAL_CAPABILITY_DEPTH_RAYCAST_ANDROID` & `XR_SPATIAL_CAPABILITY_PLANE_TRACKING_EXT` and query its component data.

```
// Fill in the raycast info.
XrSpatialRaycastInfoANDROID raycastInfo {
  .type        = XR_TYPE_SPATIAL_RAYCAST_INFO_ANDROID,
  .next        = nullptr,
  .space       = viewSpace,
  .time        = updateTime,
  .origin      = {0.0f, 0.0f, 0.0f},
  .direction   = {0.0f, 0.0f,-1.0f},
```

```
  .maxDistance = 0.0f  // ray is unbounded
};

// Next pointing to the raycast info.
XrSpatialDiscoverySnapshotCreateInfoEXT discoverySnapshotCreateInfo {
  .type = XR_TYPE_SPATIAL_DISCOVERY_SNAPSHOT_CREATE_INFO_EXT,
  .next = &raycastInfo
};

// Create snapshot
CHK_XR(xrCreateSpatialDiscoverySnapshotAsyncEXT (
spatialContext, &discoverySnapshotCreateInfo, &future));

waitUntilReady(future);

// Complete async operation.
XrCreateSpatialDiscoverySnapshotCompletionInfoEXT
  createSnapshotCompletionInfo {
    .type      = XR_TYPE_CREATE_SPATIAL_DISCOVERY_SNAPSHOT_COMPLETION_INFO_EXT,
    .baseSpace = viewSpace,
    .time      = updateTime,
    .future    = future,
  };

XrCreateSpatialDiscoverySnapshotCompletionEXT completion {
  XR_TYPE_CREATE_SPATIAL_DISCOVERY_SNAPSHOT_COMPLETION_EXT};

CHK_XR(xrCreateSpatialDiscoverySnapshotCompleteEXT(
           spatialContext, &createSnapshotCompletionInfo,
           &completion));

if(completion.futureResult != XR_SUCCESS) return;

// Query hit result components
std::array<XrSpatialComponentTypeEXT, 1> enabledComponents = {
  XR_SPATIAL_COMPONENT_TYPE_RAYCAST_RESULT_ANDROID
};

XrSpatialComponentDataQueryConditionEXT queryCond {
  .type = XR_TYPE_SPATIAL_COMPONENT_DATA_QUERY_CONDITION_EXT,
  .componentTypeCount = 1,
  .componentTypes     = enabledComponents.data(),
};

XrSpatialComponentDataQueryResultEXT queryResult {
  .type = XR_TYPE_SPATIAL_COMPONENT_DATA_QUERY_RESULT_EXT,
};
```

```
XrResult result = xrQuerySpatialComponentDataEXT(
           completion.snapshot, &queryCond, &queryResult);

// Check query result.
if(result != XR_SUCCESS) return;

// Query again for the hit result list.
std::vector<XrSpatialRaycastResultDataANDROID> raycastResults;
raycastResults.resize(queryResult.entityIdCountOutput);
XrSpatialComponentRaycastResultListANDROID raycastResultList {
  .type               = XR_TYPE_SPATIAL_COMPONENT_RAYCAST_RESULT_LIST_ANDROID,
  .raycastResultCount = (uint32_t)raycastResults.size(),
  .raycastResults     = raycastResults.data(),
};

queryResult.next = &raycastResultList;
result = xrQuerySpatialComponentDataEXT(completion.snapshot, &queryCond, &queryResult);

// Check query result again.
if(result != XR_SUCCESS) return;

std::vector<XrSpatialEntityEXT> hitEntities;
std::vector<XrPosef> depthHitPoses;
for(uint32_t i = 0; i < queryResult.entityIdCountOutput; ++i) {
    // access raycastResults[i] for id & pose
    XrSpatialEntityIdEXT   id   = queryResult.entityIds[i];
    XrPosef                pose = raycastResults[i].hitPose;

    if (id != XR_NULL_SPATIAL_ENTITY_ID_EXT) {
        // hit entities (e.g. planes)
    // get entity handle via entity id
    XrSpatialEntityFromIdCreateInfoEXT entityCreateInfo {
      .type = XR_TYPE_SPATIAL_ENTITY_FROM_ID_CREATE_INFO_EXT,
      .entityId = id,
    };

    XrSpatialEntityEXT entity = XR_NULL_HANDLE;
    xrCreateSpatialEntityFromIdEXT(spatialContext, &entityCreateInfo, &entity);

    hitEntities.push_back(entity);
  } else {
    // hit depth
    depthHitPoses.push_back(pose);
  }
}

// Cleanup
xrDestroySpatialSnapshotEXT(completion.snapshot);
```

Create Raycast Snapshot

The following example code demonstrates how to create synchronous raycast snapshot.

```
// fill in create info
XrSpatialRaycastSnapshotCreateInfoANDROID createInfo {
  .type = XR_TYPE_SPATIAL_RAYCAST_SNAPSHOT_CREATE_INFO_ANDROID,
  .componentTypeCount = 0,   // include all hit spatial entities
  .componentTypes = nullptr,
  .raycastInfo = &raycastInfo
};

// create raycast snapshot
XrSpatialSnapshotEXT raycastSnapshot = XR_NULL_HANDLE;
CHK_XR(xrCreateSpatialRaycastSnapshotANDROID(
  spatialContext, &createInfo, &raycastSnapshot));

// ...
// query component data as previous example
// ...

// Cleanup
xrDestroySpatialSnapshotEXT(raycastSnapshot);
```

12.74.8. New Commands

- xrCreateSpatialRaycastSnapshotANDROID

12.74.9. New Structures

- XrSpatialCapabilityConfigurationDepthRaycastANDROID
- XrSpatialRaycastResultDataANDROID
- XrSpatialRaycastSnapshotCreateInfoANDROID
- Extending XrSpatialComponentDataQueryResultEXT:
  - XrSpatialComponentRaycastResultListANDROID
- Extending XrSpatialDiscoverySnapshotCreateInfoEXT:
  - XrSpatialRaycastInfoANDROID

12.74.10. New Enum Constants

- XR_ANDROID_SPATIAL_DISCOVERY_RAYCAST_EXTENSION_NAME
- XR_ANDROID_spatial_discovery_raycast_SPEC_VERSION

- Extending XrSpatialCapabilityEXT:
  - XR_SPATIAL_CAPABILITY_DEPTH_RAYCAST_ANDROID
- Extending XrSpatialComponentTypeEXT:
  - XR_SPATIAL_COMPONENT_TYPE_RAYCAST_RESULT_ANDROID
- Extending XrStructureType:
  - XR_TYPE_SPATIAL_CAPABILITY_CONFIGURATION_DEPTH_RAYCAST_ANDROID
  - XR_TYPE_SPATIAL_COMPONENT_RAYCAST_RESULT_LIST_ANDROID
  - XR_TYPE_SPATIAL_RAYCAST_INFO_ANDROID
  - XR_TYPE_SPATIAL_RAYCAST_SNAPSHOT_CREATE_INFO_ANDROID

12.74.11. Issues

12.74.12. Version History

- Revision 1, 2025-07-15 (Brian Chen)
  - Initial extension description.

12.75. XR_ANDROID_spatial_entity_bound_anchor

Name String

XR_ANDROID_spatial_entity_bound_anchor

Extension Type

Instance extension

Registered Extension Number

791

Revision

2

Ratification Status

Not ratified

Extension and Version Dependencies

XR_EXT_spatial_anchor

Last Modified Date

2025-08-18

IP Status

No known IP claims.

Contributors

YuSheng Chang, Google
Kyle Chen, Google
Nihav Jain, Google
Levana Chen, Google
Spencer Quin, Google

12.75.1. Overview

This extension allows applications to create and attach anchors to spatial entities, which are called "entity-bound anchors" in this extension.

An entity-bound anchor is represented as a spatial entity with (or "that has") the `XR_SPATIAL_COMPONENT_TYPE_ANCHOR_EXT` component and the `XR_SPATIAL_COMPONENT_TYPE_PARENT_EXT` component. The `XR_SPATIAL_COMPONENT_TYPE_PARENT_EXT` component stores the `XrSpatialEntityIdEXT` of the parent entity that the anchor is attached to.

The pose of an entity-bound anchor is always represented by a fixed offset from its parent entity, which is thought of as the "base" of the anchor. For example, imagine a user has a virtual picture frame attached to a wall, the user uses an entity-bound anchor to represent the virtual picture frame, and attaches it to the wall entity. With that, the relative position between the wall and the picture frame is always consistent as the tracking estimates of the physical wall improve.

12.75.2. Runtime Support

A runtime **must** support at least one spatial tracking extension, e.g. `XR_EXT_spatial_plane_tracking`. If the runtime supports spatial entity bound anchor, it **must** provide at least one attachable component by enumerating the xrEnumerateSpatialAnchorAttachableComponentsANDROID function. The application **can** enumerate the attachable components by using xrEnumerateSpatialAnchorAttachableComponentsANDROID.

The xrEnumerateSpatialAnchorAttachableComponentsANDROID function is defined as:

```
// Provided by XR_ANDROID_spatial_entity_bound_anchor
XrResult xrEnumerateSpatialAnchorAttachableComponentsANDROID(
    XrInstance                                  instance,
    XrSystemId                                  systemId,
    uint32_t                                    attachableComponentCapacityInput,
    uint32_t*                                   attachableComponentCountOutput,
    XrSpatialComponentTypeEXT*                  attachableComponents);
```

Parameter Descriptions

- `instance` is a handle to an XrInstance.
- `systemId` is the `XrSystemId` whose spatial persistence stores will be enumerated.
- `attachableComponentCapacityInput` is the capacity of the `attachableComponents` array, or 0 to indicate a request to retrieve the required capacity.
- `attachableComponentCountOutput` is the number of attachable components, or the required capacity in the case that `attachableComponentCapacityInput` is insufficient.
- `attachableComponents` is an array of XrSpatialComponentTypeEXT. It **can** be `NULL` if `attachableComponentCapacityInput` is 0.
- See Buffer Size Parameters chapter for a detailed description of retrieving the required `attachableComponents` size.

Runtimes **must** always return identical buffer contents from this enumeration for the given `systemId` for the lifetime of the instance.

Valid Usage (Implicit)

- The `XR_ANDROID_spatial_entity_bound_anchor` extension **must** be enabled prior to calling xrEnumerateSpatialAnchorAttachableComponentsANDROID
- `instance` **must** be a valid XrInstance handle
- `attachableComponentCountOutput` **must** be a pointer to a `uint32_t` value
- If `attachableComponentCapacityInput` is not `0`, `attachableComponents` **must** be a pointer to an array of `attachableComponentCapacityInput` XrSpatialComponentTypeEXT values

Return Codes

Success

- XR_SUCCESS

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SIZE_INSUFFICIENT
- XR_ERROR_SYSTEM_INVALID
- XR_ERROR_VALIDATION_FAILURE

12.75.3. Creating a spatial entity-bound anchor

Applications use the xrCreateSpatialAnchorEXT function to create an anchor. If an application wants to create an entity-bound anchor that is attached to a spatial entity, it **can** chain an XrSpatialAnchorParentANDROID structure to the next pointer of the XrSpatialAnchorCreateInfoEXT structure when calling the xrCreateSpatialAnchorEXT function.

The XrSpatialAnchorParentANDROID structure is defined as:

```
// Provided by XR_ANDROID_spatial_entity_bound_anchor
typedef struct XrSpatialAnchorParentANDROID {
    XrStructureType         type;
    const void*             next;
    XrSpatialEntityIdEXT    parentId;
} XrSpatialAnchorParentANDROID;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain.
- parentId is the XrSpatialEntityIdEXT of the entity that the anchor will be attached to.

The runtime **must** guaranteed that the distance between the parent entity and the anchor is always consistent, where the distance is the anchor's pose to the nearest surface of the parent entity along the surface's normal. The pose of the anchor is updated based on the parent entity's position and the

distance to the parent entity's surface regardless of how many attachable components the parent entity has.

The runtime **must** return XR_ERROR_SPATIAL_ENTITY_ID_INVALID_EXT from xrCreateSpatialAnchorEXT if XrSpatialAnchorParentANDROID::parentId is not a valid ID for xrCreateSpatialAnchorEXT::spatialContext.

The runtime **must** return XR_ERROR_SPATIAL_ANCHOR_ATTACHABLE_COMPONENT_NOT_FOUND_ANDROID from xrCreateSpatialAnchorEXT if none of the components enumerated by xrEnumerateSpatialAnchorAttachableComponentsANDROID are on the parent entity.

Valid Usage (Implicit)

- The XR_ANDROID_spatial_entity_bound_anchor extension **must** be enabled prior to using XrSpatialAnchorParentANDROID
- type **must** be XR_TYPE_SPATIAL_ANCHOR_PARENT_ANDROID
- next **must** be NULL or a valid pointer to the next structure in a structure chain

12.75.4. Example code

Create spatial entity-bound anchor

The following example code demonstrates how to create an entity-bound anchor and attach it to a spatial plane tracking entity.

```
XrFutureEXT future {XR_NULL_FUTURE_EXT};

std::vector<XrSpatialEntityEXT> entityBoundAnchorEntities;

// We want to look for entities that have the plane tracking components.
std::vector<XrSpatialComponentTypeEXT> snapshotComponents = {
  XR_SPATIAL_COMPONENT_TYPE_BOUNDED_2D_EXT,
  XR_SPATIAL_COMPONENT_TYPE_PLANE_ALIGNMENT_EXT,
};

auto discoverSpatialEntities = [&](XrSpatialContextEXT spatialContext, XrTime time) {
  XrSpatialDiscoverySnapshotCreateInfoEXT snapshotCreateInfo{
    .type = XR_TYPE_SPATIAL_DISCOVERY_SNAPSHOT_CREATE_INFO_EXT,
    .componentTypeCount = static_cast<uint32_t>(snapshotComponents.size()),
    .componentTypes = snapshotComponents.data(),
  };
  CHK_XR(xrCreateSpatialDiscoverySnapshotAsyncEXT(spatialContext, &snapshotCreateInfo,
&future));
```

```
  waitUntilReady(future);

  XrCreateSpatialDiscoverySnapshotCompletionInfoEXT completionInfo{
    .type = XR_TYPE_CREATE_SPATIAL_DISCOVERY_SNAPSHOT_COMPLETION_INFO_EXT,
    .baseSpace = localSpace,
    .time = time,
    .future = future,
  };

  XrCreateSpatialDiscoverySnapshotCompletionEXT completion{
    .type = XR_TYPE_CREATE_SPATIAL_DISCOVERY_SNAPSHOT_COMPLETION_EXT,
  };
  CHK_XR(xrCreateSpatialDiscoverySnapshotCompleteEXT(spatialContext, &completionInfo,
&completion));
  if (completion.futureResult == XR_SUCCESS) {

    XrSpatialComponentDataQueryConditionEXT queryCond{
      .type = XR_TYPE_SPATIAL_COMPONENT_DATA_QUERY_CONDITION_EXT,
      .componentTypeCount = static_cast<uint32_t>(snapshotComponents.size()),
      .componentTypes = snapshotComponents.data(),
    };

    XrSpatialComponentDataQueryResultEXT queryResult{
      .type = XR_TYPE_SPATIAL_COMPONENT_DATA_QUERY_RESULT_EXT,
    };
    CHK_XR(xrQuerySpatialComponentDataEXT(completion.snapshot, &queryCond, &
queryResult));

    std::vector<XrSpatialEntityIdEXT> entityIds(queryResult.entityIdCountOutput);
    std::vector<XrSpatialEntityTrackingStateEXT> entityStates(queryResult
.entityIdCountOutput);
    queryResult.entityIdCapacityInput = entityIds.size();
    queryResult.entityIds = entityIds.data();
    queryResult.entityStateCapacityInput = entityStates.size();
    queryResult.entityStates = entityStates.data();

    std::vector<XrSpatialBounded2DDataEXT> bounded2D(queryResult.entityIdCountOutput);
    XrSpatialComponentBounded2DListEXT bounded2DList{
      .type = XR_TYPE_SPATIAL_COMPONENT_BOUNDED_2D_LIST_EXT,
      .boundCount = static_cast<uint32_t>(bounded2D.size()),
      .bounds = bounded2D.data(),
    };
    queryResult.next = &bounded2DList;

    CHK_XR(xrQuerySpatialComponentDataEXT(completion.snapshot, &queryCond, &
queryResult));

    entityBoundAnchorEntities.reserve(queryResult.entityIdCountOutput);
```

```
    // Create anchors attached to the plane entities
    for (int32_t i = 0; i < queryResult.entityIdCountOutput; ++i) {
      if (entityStates[i] != XR_SPATIAL_ENTITY_TRACKING_STATE_TRACKING_EXT) {
        continue;
      }

      // Parent ID info the chained to spatial anchor create info next
      XrSpatialAnchorParentANDROID parentIdCreateInfo{
        .type = XR_TYPE_SPATIAL_ANCHOR_PARENT_ANDROID,
        .parentId = entityIds[i],
      };

      // spatial anchor create info
      XrSpatialAnchorCreateInfoEXT createInfo{
        .type = XR_TYPE_SPATIAL_ANCHOR_CREATE_INFO_EXT,
        // assign Parent ID to anchor create info next, and the pose to the bounded2D
center
        .next = &parentIdCreateInfo,
        .baseSpace = localSpace,
        .time = time,
        .pose = bounded2D[i].center,
      };

      XrSpatialEntityIdEXT entityBoundAnchorEntityId {XR_NULL_SPATIAL_ENTITY_ID_EXT};
      XrSpatialEntityEXT entityBoundAnchorEntity {XR_NULL_HANDLE};
      CHK_XR(xrCreateSpatialAnchorEXT(spatialContext, &createInfo,
&entityBoundAnchorEntityId, &entityBoundAnchorEntity));

      entityBoundAnchorEntities.push_back(entityBoundAnchorEntity);
    }

    CHK_XR(xrDestroySpatialSnapshotEXT(completion.snapshot));
  }
};

while (1) {
  // ...
  // For every frame in frame loop
  // ...

  XrFrameState frameState;  // previously returned from xrWaitFrame
  const XrTime time = frameState.predictedDisplayTime;

  // Poll for the XR_TYPE_EVENT_DATA_SPATIAL_DISCOVERY_RECOMMENDED_EXT event
  XrEventDataBuffer event = {
    .type = XR_TYPE_EVENT_DATA_BUFFER,
  };
```

```
  XrResult result = xrPollEvent(instance, &event);
  if (result == XR_SUCCESS) {
      if (event.type == XR_TYPE_EVENT_DATA_SPATIAL_DISCOVERY_RECOMMENDED_EXT) {
              const XrEventDataSpatialDiscoveryRecommendedEXT& eventdata =
                  *reinterpret_cast<XrEventDataSpatialDiscoveryRecommendedEXT*>(&event);
              // Discover spatial entities for the context that we received the
"discovery
              // recommended" event for.
              discoverSpatialEntities(eventdata.spatialContext, time);
              break;
      }
  }

  // ...
  // Finish frame loop
  // ...
}
```

Get entity-bound anchor pose and parent entity ID

The following example code demonstrates how to get the pose of an entity-bound anchor and the ID of its parent entity.

```
std::vector<XrSpatialEntityEXT> entities;

auto updateEntityBoundAnchorInfo = [&](XrSpatialContextEXT spatialContext, XrTime time) {
    // We want to get updated data for all components of the entities, so skip specifying
componentTypes.
    XrSpatialUpdateSnapshotCreateInfoEXT snapshotCreateInfo{
      .type = XR_TYPE_SPATIAL_UPDATE_SNAPSHOT_CREATE_INFO_EXT,
      .entityCount = static_cast<uint32_t>(entities.size()),
      .entities = entities.data(),
      .baseSpace = localSpace,
      .time = time,
    };

    XrSpatialSnapshotEXT snapshot {XR_NULL_HANDLE};
    CHK_XR(xrCreateSpatialUpdateSnapshotEXT(spatialContext, &snapshotCreateInfo,
&snapshot));

    // Query for the entities that have the anchor component and parent component on
them.
    std::array<XrSpatialComponentTypeEXT, 2> componentsToQuery
{XR_SPATIAL_COMPONENT_TYPE_ANCHOR_EXT, XR_SPATIAL_COMPONENT_TYPE_PARENT_EXT};
    XrSpatialComponentDataQueryConditionEXT queryCond{
      .type = XR_TYPE_SPATIAL_COMPONENT_DATA_QUERY_CONDITION_EXT,
```

```
      .componentTypeCount = componentsToQuery.size(),
      .componentTypes = componentsToQuery.data(),
    };

    XrSpatialComponentDataQueryResultEXT queryResult{
      .type = XR_TYPE_SPATIAL_COMPONENT_DATA_QUERY_RESULT_EXT,
    };
    CHK_XR(xrQuerySpatialComponentDataEXT(snapshot, &queryCond, &queryResult));

    std::vector<XrSpatialEntityIdEXT> entityIds(queryResult.entityIdCountOutput);
    std::vector<XrSpatialEntityTrackingStateEXT> entityStates(queryResult
.entityIdCountOutput);
    queryResult.entityIdCapacityInput = entityIds.size();
    queryResult.entityIds = entityIds.data();
    queryResult.entityStateCapacityInput = entityStates.size();
    queryResult.entityStates = entityStates.data();

    // query for the pose data
    std::vector<XrPosef> locations(queryResult.entityIdCountOutput);
    XrSpatialComponentAnchorListEXT locationList{
      .type = XR_TYPE_SPATIAL_COMPONENT_ANCHOR_LIST_EXT,
      .locationCount = static_cast<uint32_t>(locations.size()),
      .locations = locations.data(),
    };
    queryResult.next = &locationList;

    // query for the parent entity ID data
    std::vector<XrSpatialEntityIdEXT> parentIds(queryResult.entityIdCountOutput);
    XrSpatialComponentParentListEXT parentList{
      .type = XR_TYPE_SPATIAL_COMPONENT_PARENT_LIST_EXT,
      .parentCount = static_cast<uint32_t>(parentIds.size()),
      .parents = parentIds.data(),
    };
    queryResult.next = &parentList;

    CHK_XR(xrQuerySpatialComponentDataEXT(snapshot, &queryCond, &queryResult));

    for (int32_t i = 0; i < queryResult.entityIdCountOutput; ++i) {
      if (entityStates[i] == XR_SPATIAL_ENTITY_TRACKING_STATE_TRACKING_EXT) {
        // Pose for entity entityIds[i] is locations[i].
        // Parent entity ID for entity entityIds[i] is parentIds[i].
      }
    }

    CHK_XR(xrDestroySpatialSnapshotEXT(snapshot));
};

while (1) {
```

```
  // ...
  // For every frame in frame loop
  // ...

  XrFrameState frameState;  // previously returned from xrWaitFrame
  const XrTime time = frameState.predictedDisplayTime;

  updateEntityBoundAnchorInfo(spatialContext, time);

  // ...
  // Finish frame loop
  // ...
}
```

Enumerate attachable Components and check entity bound anchor capability

The following example code demonstrates how to enumerate the attachable components and check if the runtime supports entity bound anchor capability.

```
// Check spatial capability
uint32_t capabilityCount = 0;
CHK_XR(xrEnumerateSpatialCapabilitiesEXT(instance, systemId, 0, &capabilityCount,
nullptr));
std::vector<XrSpatialCapabilityEXT> capabilities(capabilityCount);
CHK_XR(xrEnumerateSpatialCapabilitiesEXT(instance, systemId, capabilityCount,
&capabilityCount, capabilities.data()));

// Check if anchor capability is supported
if (std::find(capabilities.begin(), capabilities.end(), XR_SPATIAL_CAPABILITY_ANCHOR_EXT)
== capabilities.end()) {
  return;
}

// Check if plane tracking capability is supported
if (std::find(capabilities.begin(), capabilities.end(),
XR_SPATIAL_CAPABILITY_PLANE_TRACKING_EXT) == capabilities.end()) {
  return;
}

// The supported spatial tracking components
std::vector<XrSpatialComponentTypeEXT> spatialTrackingCapabilityComponents;

// Enumerate supported components for plane tracking capability
XrSpatialCapabilityComponentTypesEXT planeComponents{
  .type = XR_TYPE_SPATIAL_CAPABILITY_COMPONENT_TYPES_EXT,
};
```

```
CHK_XR(xrEnumerateSpatialCapabilityComponentTypesEXT(instance, systemId,
XR_SPATIAL_CAPABILITY_PLANE_TRACKING_EXT, &planeComponents));
std::vector<XrSpatialComponentTypeEXT> planeCapabilityComponents(planeComponents
.componentTypeCountOutput);
planeComponents.componentTypes = planeCapabilityComponents.data();
CHK_XR(xrEnumerateSpatialCapabilityComponentTypesEXT(instance, systemId,
XR_SPATIAL_CAPABILITY_PLANE_TRACKING_EXT, &planeComponents));

// Add plane supported components to spatial tracking supported components
spatialTrackingCapabilityComponents.insert(spatialTrackingCapabilityComponents.end(),
planeCapabilityComponents.begin(), planeCapabilityComponents.end());

// Enumerate supported attachable components for anchor
uint32_t attachableComponentCount = 0;
CHK_XR(xrEnumerateSpatialAnchorAttachableComponentsANDROID(instance, systemId, 0,
&attachableComponentCount, nullptr));
std::vector<XrSpatialComponentTypeEXT> attachableComponents(attachableComponentCount);
CHK_XR(xrEnumerateSpatialAnchorAttachableComponentsANDROID(instance, systemId,
attachableComponentCount, &attachableComponentCount, attachableComponents.data()));

// Check if at least one spatial tracking component is supported
const auto supportsComponent = [&spatialTrackingCapabilityComponents
](XrSpatialComponentTypeEXT component) {
  return std::find(spatialTrackingCapabilityComponents.begin(),
spatialTrackingCapabilityComponents.end(), component) !=
spatialTrackingCapabilityComponents.end();
};

bool atLeastOneComponentSupported = false;
for (int32_t i = 0; i < attachableComponentCount; ++i) {
  if(supportsComponent(attachableComponents[i])) {
    atLeastOneComponentSupported = true;
    break;
  }
}

// No spatial tracking component supported for anchor attachment
if(!atLeastOneComponentSupported) return;

// ...
// Create spatial entity anchors and get their latest pose in the frame loop.
// ...
```

12.75.5. New Commands

- xrEnumerateSpatialAnchorAttachableComponentsANDROID

12.75.6. New Structures

- Extending XrSpatialAnchorCreateInfoEXT:
  - XrSpatialAnchorParentANDROID

12.75.7. New Enum Constants

- XR_ANDROID_SPATIAL_ENTITY_BOUND_ANCHOR_EXTENSION_NAME
- XR_ANDROID_spatial_entity_bound_anchor_SPEC_VERSION
- Extending XrResult:
  - XR_ERROR_SPATIAL_ANCHOR_ATTACHABLE_COMPONENT_NOT_FOUND_ANDROID
- Extending XrStructureType:
  - XR_TYPE_SPATIAL_ANCHOR_PARENT_ANDROID

12.75.8. Issues

12.75.9. Version History

- Revision 1, 2025-08-18 (YuSheng Chang)
  - Initial extension description.
- Revision 2, 2025-12-16 (Kyle Chen)
  - Add more prescriptive language to certain behaviors of the API.
  - Add the example code of xrEnumerateSpatialAnchorAttachableComponentsANDROID.

12.76. XR_ANDROID_spatial_object_tracking

Name String

XR_ANDROID_spatial_object_tracking

Extension Type

Instance extension

Registered Extension Number

786

Revision

2

Ratification Status

Not ratified

Extension and Version Dependencies

XR_EXT_spatial_entity

Last Modified Date

2025-06-25

IP Status

No known IP claims.

Contributors

Kyle Chen, Google
Nihav Jain, Google
Levana Chen, Google
Spencer Quin, Google

12.76.1. Overview

This extension builds on XR_EXT_spatial_entity and defines a spatial capability for tracking a set of predefined physical objects without requiring an input dataset from the application. For example, the system **can** track keyboards, mice, and other objects in the environment, and then report the poses and extents of these objects to the application.



Permissions

Android applications **must** have the android.permission.SCENE_UNDERSTANDING_COARSE permission listed in their manifest as this extension depends on the geometry of the environment. The android.permission.SCENE_UNDERSTANDING_COARSE permission is considered a dangerous permission.

(protection level: dangerous)

12.76.2. Runtime Support

If the runtime supports object tracking, it **must** indicate this by enumerating XR_SPATIAL_CAPABILITY_OBJECT_TRACKING_ANDROID in xrEnumerateSpatialCapabilitiesEXT.

12.76.3. Configuration

The XrSpatialCapabilityConfigurationObjectTrackingANDROID structure is defined as:

```
// Provided by XR_ANDROID_spatial_object_tracking
typedef struct XrSpatialCapabilityConfigurationObjectTrackingANDROID {
    XrStructureType                                 type;
    const void*                                     next;
    XrSpatialCapabilityEXT                          capability;
    uint32_t                                        enabledComponentCount;
    const XrSpatialComponentTypeEXT*                enabledComponents;
    uint32_t                                        activeSemanticLabelCount;
    const XrSpatialObjectSemanticLabelANDROID*      activeSemanticLabels;
} XrSpatialCapabilityConfigurationObjectTrackingANDROID;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `capability` is an XrSpatialCapabilityEXT and **must** be `XR_SPATIAL_CAPABILITY_OBJECT_TRACKING_ANDROID`.
- `enabledComponentCount` is a `uint32_t` describing the count of elements in the `enabledComponents` array. It **must** be greater than 0.
- `enabledComponents` is a pointer to an array of XrSpatialComponentTypeEXT.
- `activeSemanticLabelCount` is a `uint32_t` describing the count of elements in the `activeSemanticLabels` array. It **can** be 0, which means all semantic labels are active, or it **can** be greater than 0, in which case the `activeSemanticLabels` array is used to determine which semantic labels are active.
- `activeSemanticLabels` is a pointer to the array of XrSpatialObjectSemanticLabelANDROID indicating the active objects in tracking. It **must** be null if `activeSemanticLabelCount` is 0. And it **must** be non-null if `activeSemanticLabelCount` is greater than 0, which means that it is used to determine which semantic labels are active.

Applications **can** enable the `XR_SPATIAL_CAPABILITY_OBJECT_TRACKING_ANDROID` spatial capability by adding a pointer to the XrSpatialCapabilityConfigurationObjectTrackingANDROID structure in XrSpatialContextCreateInfoEXT::`capabilityConfigs`.

The runtime **must** return `XR_ERROR_VALIDATION_FAILURE` if `capability` is not `XR_SPATIAL_CAPABILITY_OBJECT_TRACKING_ANDROID`.

Applications **can** specify the desired semantic labels in `activeSemanticLabels` which they are in favor to track. If there is no semantic labels specified in `activeSemanticLabels` and `activeSemanticLabelCount` is 0, all semantic labels are active to be tracked by default.

Valid Usage (Implicit)

- The XR_ANDROID_spatial_object_tracking extension **must** be enabled prior to using XrSpatialCapabilityConfigurationObjectTrackingANDROID
- type **must** be XR_TYPE_SPATIAL_CAPABILITY_CONFIGURATION_OBJECT_TRACKING_ANDROID
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- capability **must** be a valid XrSpatialCapabilityEXT value
- enabledComponents **must** be a pointer to an array of enabledComponentCount valid XrSpatialComponentTypeEXT values
- activeSemanticLabels **must** be a pointer to an array of activeSemanticLabelCount valid XrSpatialObjectSemanticLabelANDROID values
- The enabledComponentCount parameter **must** be greater than 0
- The activeSemanticLabelCount parameter **must** be greater than 0

12.76.4. Guaranteed Components

A runtime that supports XR_SPATIAL_CAPABILITY_OBJECT_TRACKING_ANDROID **must** support the following spatial components as guaranteed components of the entities discovered by this capability and **must** enumerate them in xrEnumerateSpatialCapabilityComponentTypesEXT -

- XR_SPATIAL_COMPONENT_TYPE_BOUNDED_3D_EXT
- XR_SPATIAL_COMPONENT_TYPE_OBJECT_SEMANTIC_LABEL_ANDROID An object tracking capability is meaningless without a semantic label component. While this extension defines the XR_SPATIAL_COMPONENT_TYPE_OBJECT_SEMANTIC_LABEL_ANDROID as one of the possible semantic label components for runtimes to support, other extensions **may** define new component types to provide different classes of semantic labels that describe a tracked object.

Bounded 3D Component

The bounded 3D component provides the center and extents of the object represented by the entity it is on. See Bounded 3D for more details.

Object Semantic Label Component

Component Data

```
// Provided by XR_ANDROID_spatial_object_tracking
typedef enum XrSpatialObjectSemanticLabelANDROID {
    XR_SPATIAL_OBJECT_SEMANTIC_LABEL_UNCATEGORIZED_ANDROID = 0,
    XR_SPATIAL_OBJECT_SEMANTIC_LABEL_KEYBOARD_ANDROID = 1,
    XR_SPATIAL_OBJECT_SEMANTIC_LABEL_MOUSE_ANDROID = 2,
    XR_SPATIAL_OBJECT_SEMANTIC_LABEL_LAPTOP_BASE_ANDROID = 3,
    XR_SPATIAL_OBJECT_SEMANTIC_LABEL_MAX_ENUM_ANDROID = 0x7FFFFFFF
} XrSpatialObjectSemanticLabelANDROID;
```

The XrSpatialObjectSemanticLabelANDROID enumeration describes a set of semantic labels for a object.

Enum	Description
XR_SPATIAL_OBJECT_SEMANTIC_LABEL_UNCATEGORIZED_ANDROID	The runtime was unable to classify this entity.
XR_SPATIAL_OBJECT_SEMANTIC_LABEL_KEYBOARD_ANDROID	The entity is a keyboard.
XR_SPATIAL_OBJECT_SEMANTIC_LABEL_MOUSE_ANDROID	The entity is a mouse.
XR_SPATIAL_OBJECT_SEMANTIC_LABEL_LAPTOP_BASE_ANDROID	The entity is a laptop.

Spatial object orientation

The orientation of the objects is shown in the following image (with xyz = rgb) originating at the bottom center of the objects.

The 3D orientation of the mouse is aligned to the 3D orientation of the supporting plane. The extents enclose the keyboard and mouse completely and the base of the laptop.

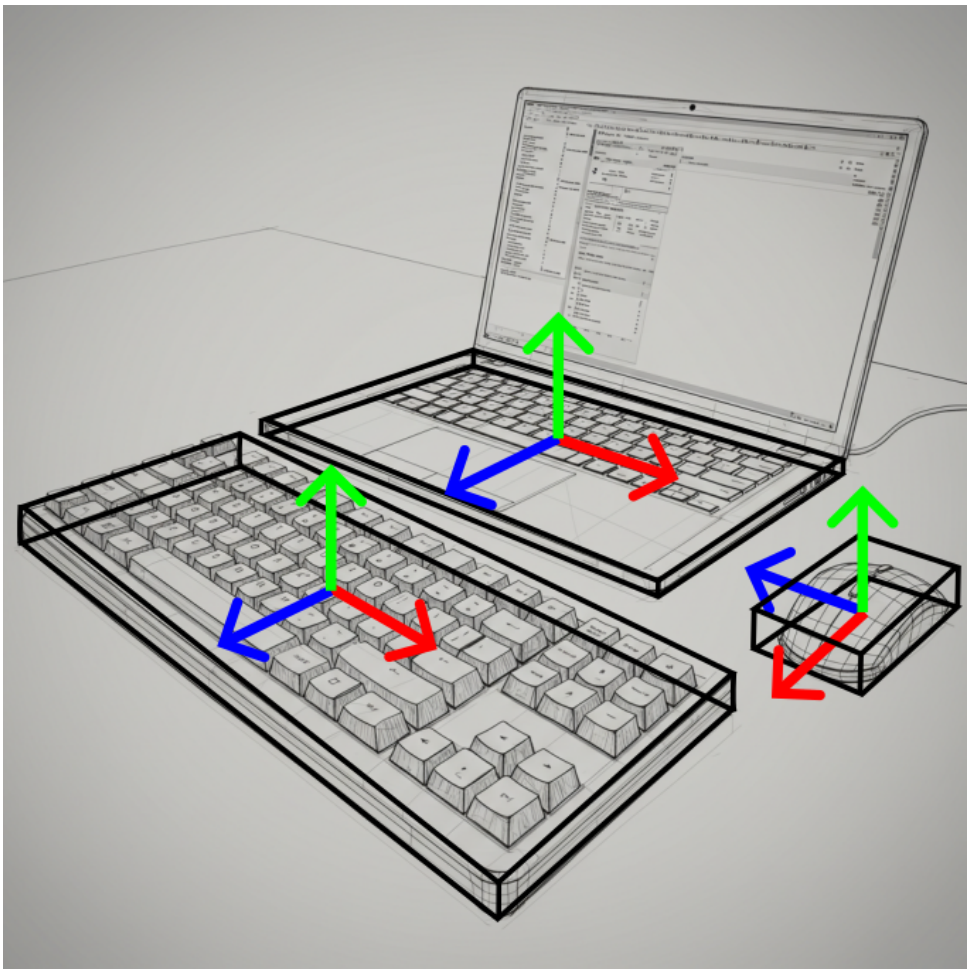


Figure 28. Example object orientation.

Component list struct to query data

The XrSpatialComponentObjectSemanticLabelListANDROID structure is defined as:

```
// Provided by XR_ANDROID_spatial_object_tracking
typedef struct XrSpatialComponentObjectSemanticLabelListANDROID {
    XrStructureType                          type;
    void*                                    next;
    uint32_t                                 semanticLabelCount;
    XrSpatialObjectSemanticLabelANDROID*     semanticLabels;
} XrSpatialComponentObjectSemanticLabelListANDROID;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `semanticLabelCount` is a `uint32_t` describing the count of elements in the `semanticLabels` array.
- `semanticLabels` is an array of XrSpatialObjectSemanticLabelANDROID.

The application **can** query the semantic label component of the spatial entities in an XrSpatialSnapshotEXT by adding `XR_TYPE_SPATIAL_COMPONENT_OBJECT_SEMANTIC_LABEL_LIST_ANDROID` in XrSpatialComponentDataQueryConditionEXT::`componentTypes` and adding XrSpatialComponentObjectSemanticLabelListANDROID to the next pointer chain of XrSpatialComponentDataQueryResultEXT.

The runtime **must** return `XR_ERROR_VALIDATION_FAILURE` from xrQuerySpatialComponentDataEXT if XrSpatialComponentObjectSemanticLabelListANDROID is in the next chain of XrSpatialComponentDataQueryResultEXT::`next` but `XR_SPATIAL_COMPONENT_TYPE_OBJECT_SEMANTIC_LABEL_ANDROID` is not included in XrSpatialComponentDataQueryConditionEXT::`componentTypes`.

The runtime **must** return `XR_ERROR_SIZE_INSUFFICIENT` from xrQuerySpatialComponentDataEXT if `semanticLabelCount` is less than XrSpatialComponentDataQueryResultEXT::`entityIdCountOutput`.

Valid Usage (Implicit)

- The `XR_ANDROID_spatial_object_tracking` extension **must** be enabled prior to using XrSpatialComponentObjectSemanticLabelListANDROID
- `type` **must** be `XR_TYPE_SPATIAL_COMPONENT_OBJECT_SEMANTIC_LABEL_LIST_ANDROID`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `semanticLabels` **must** be a pointer to an array of `semanticLabelCount` XrSpatialObjectSemanticLabelANDROID values
- The `semanticLabelCount` parameter **must** be greater than `0`

Configuration

If `XR_SPATIAL_COMPONENT_TYPE_OBJECT_SEMANTIC_LABEL_ANDROID` is enumerated in XrSpatialCapabilityComponentTypesEXT::`componentTypes` for some capability, application **can** enable it by including the enum in the XrSpatialCapabilityConfigurationBaseHeaderEXT::`enabledComponents` list of the XrSpatialCapabilityConfigurationBaseHeaderEXT derived structure of the capability that supports this component.

This component does not require any special configuration to be included in the next chain of XrSpatialCapabilityConfigurationBaseHeaderEXT.

12.76.5. Example code

Configure Object Tracking Capability

The following example code demonstrates how to configure object tracking capability when creating a spatial context.

```
// Check if object tracking capability is supported
uint32_t capabilityCount = 0;
CHK_XR(xrEnumerateSpatialCapabilitiesEXT(instance, systemId, 0, &capabilityCount,
nullptr));
std::vector<XrSpatialCapabilityEXT> capabilities(capabilityCount);
CHK_XR(xrEnumerateSpatialCapabilitiesEXT(instance, systemId, capabilityCount,
&capabilityCount, capabilities.data()));

if (std::find(capabilities.begin(), capabilities.end(),
XR_SPATIAL_CAPABILITY_OBJECT_TRACKING_ANDROID) == capabilities.end()) {
  return;
}

// Enumerate supported components for object tracking capability
XrSpatialCapabilityComponentTypesEXT objectComponents{
  .type = XR_TYPE_SPATIAL_CAPABILITY_COMPONENT_TYPES_EXT,
};
CHK_XR(xrEnumerateSpatialCapabilityComponentTypesEXT(instance, systemId,
XR_SPATIAL_CAPABILITY_OBJECT_TRACKING_ANDROID, &objectComponents));
std::vector<XrSpatialComponentTypeEXT> objectCapabilityComponents(objectComponents
.componentTypeCountOutput);
objectComponents.componentTypeCapacityInput = objectCapabilityComponents.size();
objectComponents.componentTypes = objectCapabilityComponents.data();
CHK_XR(xrEnumerateSpatialCapabilityComponentTypesEXT(instance, systemId,
XR_SPATIAL_CAPABILITY_OBJECT_TRACKING_ANDROID, &objectComponents));

// Create a spatial context
XrSpatialContextEXT spatialContext{XR_NULL_HANDLE};

// Enable the 2 guaranteed components (bounded 3D, semantic label) of the object tracking
capability
std::vector<XrSpatialComponentTypeEXT> enabledComponents = {
  XR_SPATIAL_COMPONENT_TYPE_BOUNDED_3D_EXT,
  XR_SPATIAL_COMPONENT_TYPE_OBJECT_SEMANTIC_LABEL_ANDROID,
};

// Specify some semantic labels to be active and tracked
```

```
std::vector<XrSpatialObjectSemanticLabelANDROID> activeSemanticLabels =
{
    XR_SPATIAL_OBJECT_SEMANTIC_LABEL_KEYBOARD_ANDROID,
    XR_SPATIAL_OBJECT_SEMANTIC_LABEL_MOUSE_ANDROID,
};

XrSpatialCapabilityConfigurationObjectTrackingANDROID objectTrackingConfig{
  .type = XR_TYPE_SPATIAL_CAPABILITY_CONFIGURATION_OBJECT_TRACKING_ANDROID,
  .capability = XR_SPATIAL_CAPABILITY_OBJECT_TRACKING_ANDROID,
  .enabledComponentCount = static_cast<uint32_t>(enabledComponents.size()),
  .enabledComponents = enabledComponents.data(),
  // Specify the count of active semantic labels and the pointer to the array of
  // active semantic labels.
  // If the `activeSemanticLabelCount` is 0 and `activeSemanticLabels` is NULL,
  // all labels will be set to active by default.
  .activeSemanticLabelCount = static_cast<uint32_t>(activeSemanticLabels.size()),
  .activeSemanticLabels = activeSemanticLabels.data(),
};

std::array<XrSpatialCapabilityConfigurationBaseHeaderEXT*, 1> capabilityConfigs = {
  reinterpret_cast<XrSpatialCapabilityConfigurationBaseHeaderEXT*>(&
objectTrackingConfig),
};

XrSpatialContextCreateInfoEXT spatialContextCreateInfo{
  .type = XR_TYPE_SPATIAL_CONTEXT_CREATE_INFO_EXT,
  .capabilityConfigCount = capabilityConfigs.size(),
  .capabilityConfigs = capabilityConfigs.data(),
};
XrFutureEXT createContextFuture {XR_NULL_FUTURE_EXT};
CHK_XR(xrCreateSpatialContextAsyncEXT(session, &spatialContextCreateInfo,
&createContextFuture));

waitUntilReady(createContextFuture);

XrCreateSpatialContextCompletionEXT completion{
  .type = XR_TYPE_CREATE_SPATIAL_CONTEXT_COMPLETION_EXT,
};
CHK_XR(xrCreateSpatialContextCompleteEXT(session, createContextFuture, &completion));
if (completion.futureResult != XR_SUCCESS) {
  return;
}

spatialContext = completion.spatialContext;

// ...
// Discover entities with the spatial context
// ...
```

```
CHK_XR(xrDestroySpatialContextEXT(spatialContext));
```

Discover Spatial Entities & Query Component Data

The following example code demonstrates how to discover spatial entities for a context configured with XR_SPATIAL_CAPABILITY_OBJECT_TRACKING_ANDROID and query its component data.

```
XrFutureEXT future = XR_NULL_FUTURE_EXT;

// We want to look for entities that have the following components.
std::vector<XrSpatialComponentTypeEXT> snapshotComponents = {
  XR_SPATIAL_COMPONENT_TYPE_BOUNDED_3D_EXT,
  XR_SPATIAL_COMPONENT_TYPE_OBJECT_SEMANTIC_LABEL_ANDROID,
};

auto discoverSpatialEntities = [&](XrSpatialContextEXT spatialContext, XrTime time) {
  XrSpatialDiscoverySnapshotCreateInfoEXT snapshotCreateInfo{
    .type = XR_TYPE_SPATIAL_DISCOVERY_SNAPSHOT_CREATE_INFO_EXT,
    .componentTypeCount = static_cast<uint32_t>(snapshotComponents.size()),
    .componentTypes = snapshotComponents.data(),
  };
  CHK_XR(xrCreateSpatialDiscoverySnapshotAsyncEXT(spatialContext, &snapshotCreateInfo,
&future));

  waitUntilReady(future);

  XrCreateSpatialDiscoverySnapshotCompletionInfoEXT completionInfo{
    .type = XR_TYPE_CREATE_SPATIAL_DISCOVERY_SNAPSHOT_COMPLETION_INFO_EXT,
    .baseSpace = localSpace,
    .time = time,
    .future = future,
  };

  XrCreateSpatialDiscoverySnapshotCompletionEXT completion{
    .type = XR_TYPE_CREATE_SPATIAL_DISCOVERY_SNAPSHOT_COMPLETION_EXT,
  };
  CHK_XR(xrCreateSpatialDiscoverySnapshotCompleteEXT(spatialContext, &completionInfo,
&completion));
  if (completion.futureResult == XR_SUCCESS) {

    // Query for the semantic label component data
    XrSpatialComponentDataQueryConditionEXT queryCond{
      .type = XR_TYPE_SPATIAL_COMPONENT_DATA_QUERY_CONDITION_EXT,
      .componentTypeCount = static_cast<uint32_t>(snapshotComponents.size()),
      .componentTypes = snapshotComponents.data(),
```

```
    };

    XrSpatialComponentDataQueryResultEXT queryResult{
      .type = XR_TYPE_SPATIAL_COMPONENT_DATA_QUERY_RESULT_EXT,
    };
    CHK_XR(xrQuerySpatialComponentDataEXT(completion.snapshot, &queryCond, &
queryResult));

    std::vector<XrSpatialEntityIdEXT> entityIds(queryResult.entityIdCountOutput);
    std::vector<XrSpatialEntityTrackingStateEXT> entityStates(queryResult
.entityIdCountOutput);
    queryResult.entityIdCapacityInput = entityIds.size();
    queryResult.entityIds = entityIds.data();
    queryResult.entityStateCapacityInput = entityStates.size();
    queryResult.entityStates = entityStates.data();

    std::vector<XrBoxf> bounded3D(queryResult.entityIdCountOutput);
    XrSpatialComponentBounded3DListEXT bounded3DList{
      .type = XR_TYPE_SPATIAL_COMPONENT_BOUNDED_3D_LIST_EXT,
      .boundCount = static_cast<uint32_t>(bounded3D.size()),
      .bounds = bounded3D.data(),
    };
    queryResult.next = &bounded3DList;

    std::vector<XrSpatialObjectSemanticLabelANDROID> semanticLabels(queryResult
.entityIdCountOutput);
    XrSpatialComponentObjectSemanticLabelListANDROID semanticLabelsList{
      .type = XR_TYPE_SPATIAL_COMPONENT_OBJECT_SEMANTIC_LABEL_LIST_ANDROID,
      .semanticLabelCount = static_cast<uint32_t>(semanticLabels.size()),
      .semanticLabels = semanticLabels.data(),
    };
    queryResult.next = &semanticLabelsList;

    CHK_XR(xrQuerySpatialComponentDataEXT(completion.snapshot, &queryCond, &
queryResult));

    for (int32_t i = 0; i < queryResult.entityIdCountOutput; ++i) {
      if (entityStates[i] != XR_SPATIAL_ENTITY_TRACKING_STATE_TRACKING_EXT) {
        continue;
      }

      // 3D bounds for entity entityIds[i] is bounded3D[i].extents centered on
bounded3D[i].center.

      // semantic label for entity entityIds[i] is semanticLabels[i].

    }
```

```
    CHK_XR(xrDestroySpatialSnapshotEXT(completion.snapshot));
  }
};

while (1) {
  // ...
  // For every frame in frame loop
  // ...

  XrFrameState frameState;  // previously returned from xrWaitFrame
  const XrTime time = frameState.predictedDisplayTime;

  // Poll for the XR_TYPE_EVENT_DATA_SPATIAL_DISCOVERY_RECOMMENDED_EXT event
  XrEventDataBuffer event = {
    .type = XR_TYPE_EVENT_DATA_BUFFER,
  };
  XrResult result = xrPollEvent(instance, &event);
  if (result == XR_SUCCESS) {
    if (event.type == XR_TYPE_EVENT_DATA_SPATIAL_DISCOVERY_RECOMMENDED_EXT) {
      const XrEventDataSpatialDiscoveryRecommendedEXT& eventdata =
          *reinterpret_cast<XrEventDataSpatialDiscoveryRecommendedEXT*>(&event);
      // Discover spatial entities for the context that we received the "discovery
      // recommended" event for.
      discoverSpatialEntities(eventdata.spatialContext, time);
    }
  }

  // ...
  // Finish frame loop
  // ...
}
```

12.76.6. New Structures

- XrSpatialCapabilityConfigurationObjectTrackingANDROID
- Extending XrSpatialComponentDataQueryResultEXT:
  - XrSpatialComponentObjectSemanticLabelListANDROID

12.76.7. New Enums

- XrSpatialObjectSemanticLabelANDROID

12.76.8. New Enum Constants

- XR_ANDROID_SPATIAL_OBJECT_TRACKING_EXTENSION_NAME

- XR_ANDROID_spatial_object_tracking_SPEC_VERSION
- Extending XrSpatialCapabilityEXT:
  - XR_SPATIAL_CAPABILITY_OBJECT_TRACKING_ANDROID
- Extending XrSpatialComponentTypeEXT:
  - XR_SPATIAL_COMPONENT_TYPE_OBJECT_SEMANTIC_LABEL_ANDROID
- Extending XrStructureType:
  - XR_TYPE_SPATIAL_CAPABILITY_CONFIGURATION_OBJECT_TRACKING_ANDROID
  - XR_TYPE_SPATIAL_COMPONENT_OBJECT_SEMANTIC_LABEL_LIST_ANDROID

12.76.9. Issues

12.76.10. Version History

- Revision 1, 2025-06-26 (Kyle Chen)
  - Initial extension description.
- Revision 2, 2025-12-04 (Kyle Chen)
  - Update the example format to new style.

12.77. XR_ANDROID_trackables_image

Name String

XR_ANDROID_trackables_image

Extension Type

Instance extension

Registered Extension Number

710

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

XR_EXT_future
and
XR_ANDROID_trackables

Last Modified Date

2025-04-08

IP Status

No known IP claims.

Contributors

Christopher Doer, Google
Levana Chen, Google
Jared Finder, Google
Spencer Quin, Google
Nihav Jain, Google
Diego Tipaldi, Google
Daniel Guttenberg, Qualcomm
Mark Vadasi, Qualcomm
Markus Birkner, Qualcomm
Maximilian Mayer, Qualcomm

12.77.1. Overview

This extension enables tracking of planar images as specified by sets of input reference images.



Permissions

Android applications **must** have the android.permission.SCENE_UNDERSTANDING_COARSE permission listed in their manifest as this extension depends on `XR_ANDROID_trackables` and exposes the geometry of the environment. The android.permission.SCENE_UNDERSTANDING_COARSE permission is considered a dangerous permission.

(protection level: dangerous)

12.77.2. Inspect system capability

The XrSystemImageTrackingPropertiesANDROID structure is defined as:

```
// Provided by XR_ANDROID_trackables_image
typedef struct XrSystemImageTrackingPropertiesANDROID {
    XrStructureType    type;
    void*              next;
    XrBool32           supportsImageTracking;
    XrBool32           supportsPhysicalSizeEstimation;
    uint32_t           maxTrackedImageCount;
    uint32_t           maxLoadedImageCount;
} XrSystemImageTrackingPropertiesANDROID;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension. For more details about the structure chain, see the structure being extended (XrSystemProperties).
- `supportsImageTracking` is an `XrBool32` indicating if current system provides image tracking capability.
- `supportsPhysicalSizeEstimation` is an `XrBool32` indicating if current system provides image size estimation.
- `maxTrackedImageCount` is the total maximum number of images that **can** be tracked at the same time.
- `maxLoadedImageCount` is the total maximum number of reference images that **can** be loaded across all databases.

An application **can** inspect whether the system is capable of image tracking by extending the XrSystemProperties with XrSystemImageTrackingPropertiesANDROID structure when calling xrGetSystemProperties. The runtime **must** return `XR_ERROR_FEATURE_UNSUPPORTED` for image tracker creation if and only if `supportsImageTracking` is `XR_FALSE`.

If a runtime supports image tracking, it **must** support `maxTrackedImageCount` tracked images at any given time.

If a runtime supports image tracking, it **must** support `maxLoadedImageCount` loaded images at any given time.

If a runtime supports image size estimation, the application **can** set XrTrackableImageDatabaseEntryANDROID::`physicalWidth` `0` to indicate the usage of size estimation. Otherwise, the application **must** set XrTrackableImageDatabaseEntryANDROID::`physicalWidth` to a positive value or `XR_ERROR_VALIDATION_FAILURE` will be returned.

Valid Usage (Implicit)

- The XR_ANDROID_trackables_image extension **must** be enabled prior to using XrSystemImageTrackingPropertiesANDROID
- type **must** be XR_TYPE_SYSTEM_IMAGE_TRACKING_PROPERTIES_ANDROID
- next **must** be NULL or a valid pointer to the next structure in a structure chain

12.77.3. Creating databases

The application **can** create an XrTrackableImageDatabaseANDROID handle by creating one or more XrTrackableImageDatabaseEntryANDROID structures and passing them to the xrCreateTrackableImageDatabaseAsyncANDROID function through an XrTrackableImageDatabaseCreateInfoANDROID structure.

The application **must** provide at least one XrTrackableImageDatabaseEntryANDROID when creating an XrTrackableImageDatabaseANDROID handle.

An XrTrackableImageDatabaseANDROID is a handle that represents a set processed reference images that **can** be discovered and tracked in the environment.

```
// Provided by XR_ANDROID_trackables_image
XR_DEFINE_HANDLE(XrTrackableImageDatabaseANDROID)
```

The XrTrackableImageDatabaseEntryANDROID structure is defined as:

```
// Provided by XR_ANDROID_trackables_image
typedef struct XrTrackableImageDatabaseEntryANDROID {
    XrStructureType                         type;
    const void*                             next;
    XrTrackableImageTrackingModeANDROID     trackingMode;
    float                                   physicalWidth;
    uint32_t                                imageWidth;
    uint32_t                                imageHeight;
    XrTrackableImageFormatANDROID           format;
    uint32_t                                bufferSize;
    const uint8_t*                          buffer;
} XrTrackableImageDatabaseEntryANDROID;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `trackingMode` is an XrTrackableImageTrackingModeANDROID indicating the desired mode for tracking.
- `physicalWidth` indicates the width of the image in meters. If zero, the image size will be estimated online.
- `imageWidth` indicates the width of the image in pixels.
- `imageHeight` indicates the height of the image in pixels.
- `format` is an XrTrackableImageFormatANDROID indicating the format of the image data in `buffer`.
- `bufferSize` indicates the byte length of `buffer`.
- `buffer` is the `uint8_t` buffer containing the reference image pixel data. The contents of `buffer` **must** be valid for the duration of the database creation async operation, which is started by xrCreateTrackableImageDatabaseAsyncANDROID and completed by xrCreateTrackableImageDatabaseCompleteANDROID.

The application **may** set `physicalWidth` to 0 to request online size estimation if XrSystemImageTrackingPropertiesANDROID::`supportsPhysicalSizeEstimation` is `XR_TRUE`.

The runtime **may** return `XR_ERROR_VALIDATION_FAILURE` from xrCreateTrackableImageDatabaseAsyncANDROID if `bufferSize` does not match the expected size based on the entry's `imageWidth`, `imageHeight` and `format`.

Valid Usage (Implicit)

- The `XR_ANDROID_trackables_image` extension **must** be enabled prior to using XrTrackableImageDatabaseEntryANDROID
- `type` **must** be `XR_TYPE_TRACKABLE_IMAGE_DATABASE_ENTRY_ANDROID`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `trackingMode` **must** be a valid XrTrackableImageTrackingModeANDROID value
- `format` **must** be a valid XrTrackableImageFormatANDROID value
- `buffer` **must** be a pointer to an array of `bufferSize` `uint8_t` values
- The `bufferSize` parameter **must** be greater than `0`

The XrTrackableImageDatabaseCreateInfoANDROID structure is defined as:

```
// Provided by XR_ANDROID_trackables_image
typedef struct XrTrackableImageDatabaseCreateInfoANDROID {
    XrStructureType                                 type;
    const void*                                     next;
    uint32_t                                        entryCount;
    const XrTrackableImageDatabaseEntryANDROID*     entries;
} XrTrackableImageDatabaseCreateInfoANDROID;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `entryCount` is a `uint32_t` specifying the count of elements in the `entries` array.
- `entries` is an array of XrTrackableImageDatabaseEntryANDROID structures.

Valid Usage (Implicit)

- The `XR_ANDROID_trackables_image` extension **must** be enabled prior to using XrTrackableImageDatabaseCreateInfoANDROID
- `type` **must** be `XR_TYPE_TRACKABLE_IMAGE_DATABASE_CREATE_INFO_ANDROID`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `entries` **must** be a pointer to an array of `entryCount` valid XrTrackableImageDatabaseEntryANDROID structures
- The `entryCount` parameter **must** be greater than `0`

The XrCreateTrackableImageDatabaseCompletionANDROID structure is defined as:

```
// Provided by XR_ANDROID_trackables_image
typedef struct XrCreateTrackableImageDatabaseCompletionANDROID {
    XrStructureType                     type;
    void*                               next;
    XrResult                            futureResult;
    XrTrackableImageDatabaseANDROID     database;
} XrCreateTrackableImageDatabaseCompletionANDROID;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `futureResult` is the XrResult of the asynchronous operation.
- `database` is the created XrTrackableImageDatabaseANDROID handle.

Future Return Codes

`futureResult` values:

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_OUT_OF_MEMORY`
- `XR_ERROR_LIMIT_REACHED`

Valid Usage (Implicit)

- The `XR_ANDROID_trackables_image` extension **must** be enabled prior to using XrCreateTrackableImageDatabaseCompletionANDROID
- `type` **must** be `XR_TYPE_CREATE_TRACKABLE_IMAGE_DATABASE_COMPLETION_ANDROID`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `futureResult` **must** be a valid XrResult value
- `database` **must** be a valid XrTrackableImageDatabaseANDROID handle

The xrCreateTrackableImageDatabaseAsyncANDROID function is defined as:

```
// Provided by XR_ANDROID_trackables_image
XrResult xrCreateTrackableImageDatabaseAsyncANDROID(
    XrSession                                       session,
    const XrTrackableImageDatabaseCreateInfoANDROID* createInfo,
    XrFutureEXT*                                    future);
```

Parameter Descriptions

- `session` is a handle to an XrSession previously created with xrCreateSession.
- `createInfo` is a pointer to an XrTrackableImageDatabaseCreateInfoANDROID structure.
- `future` is a pointer to the created `XrFutureEXT`.

Valid Usage (Implicit)

- The `XR_ANDROID_trackables_image` extension **must** be enabled prior to calling xrCreateTrackableImageDatabaseAsyncANDROID
- `session` **must** be a valid XrSession handle
- `createInfo` **must** be a pointer to a valid XrTrackableImageDatabaseCreateInfoANDROID structure
- `future` **must** be a pointer to an `XrFutureEXT` value

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FEATURE_UNSUPPORTED
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_IMAGE_FORMAT_UNSUPPORTED_ANDROID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The xrCreateTrackableImageDatabaseCompleteANDROID function is defined as:

```
// Provided by XR_ANDROID_trackables_image
XrResult xrCreateTrackableImageDatabaseCompleteANDROID(
    XrSession                                       session,
    XrFutureEXT                                     future,
    XrCreateTrackableImageDatabaseCompletionANDROID* completion);
```

Parameter Descriptions

- session is a handle to an XrSession previously created with xrCreateSession.
- future is the XrFutureEXT of the future to complete.
- completion is a pointer to an XrCreateTrackableImageDatabaseCompletionANDROID structure containing the result of the operation.

Valid Usage (Implicit)

- The XR_ANDROID_trackables_image extension **must** be enabled prior to calling xrCreateTrackableImageDatabaseCompleteANDROID
- `session` **must** be a valid XrSession handle
- `completion` **must** be a pointer to an XrCreateTrackableImageDatabaseCompletionANDROID structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_FUTURE_INVALID_EXT
- XR_ERROR_FUTURE_PENDING_EXT
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The xrDestroyTrackableImageDatabaseANDROID function is defined as:

```
// Provided by XR_ANDROID_trackables_image
XrResult xrDestroyTrackableImageDatabaseANDROID(
    XrTrackableImageDatabaseANDROID             database);
```

Parameter Descriptions

- `database` is the XrTrackableImageDatabaseANDROID handle to destroy.

Valid Usage (Implicit)

- The XR_ANDROID_trackables_image extension **must** be enabled prior to calling xrDestroyTrackableImageDatabaseANDROID
- database **must** be a valid XrTrackableImageDatabaseANDROID handle

Thread Safety

- Access to database, and any child handles, **must** be externally synchronized

Return Codes

Success

- XR_SUCCESS

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_RUNTIME_FAILURE

12.77.4. Tracking images

This extension adds XR_TRACKABLE_TYPE_IMAGE_ANDROID to XrTrackableTypeANDROID.

The application **can** create an XrTrackableTrackerANDROID by calling xrCreateTrackableTrackerANDROID and specifying XR_TRACKABLE_TYPE_IMAGE_ANDROID as the trackable type in XrTrackableTrackerCreateInfoANDROID::trackableType to track images.

The runtime **must** return XR_ERROR_FEATURE_UNSUPPORTED if XrTrackableTrackerCreateInfoANDROID::trackableType is XR_TRACKABLE_TYPE_IMAGE_ANDROID and XrSystemImageTrackingPropertiesANDROID::supportsImageTracking returns XR_FALSE via xrGetSystemProperties.

The XrTrackableImageConfigurationANDROID structure is defined as:

```
// Provided by XR_ANDROID_trackables_image
typedef struct XrTrackableImageConfigurationANDROID {
    XrStructureType                           type;
    const void*                               next;
    uint32_t                                  databaseCount;
    const XrTrackableImageDatabaseANDROID*    databases;
} XrTrackableImageConfigurationANDROID;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `databaseCount` is a `uint32_t` that specifies the number of elements in `databases`
- `databases` is an array of XrTrackableImageDatabaseANDROID that specifies the databases to create the tracker with.

The application **must** set a valid configuration by adding a XrTrackableImageConfigurationANDROID to the `next` chain of XrTrackableTrackerCreateInfoANDROID. Otherwise, the runtime **must** return `XR_ERROR_VALIDATION_FAILURE`.

The application **must** provide at least one XrTrackableImageDatabaseANDROID structure to create the tracker with.

Valid Usage (Implicit)

- The `XR_ANDROID_trackables_image` extension **must** be enabled prior to using XrTrackableImageConfigurationANDROID
- `type` **must** be `XR_TYPE_TRACKABLE_IMAGE_CONFIGURATION_ANDROID`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `databases` **must** be a pointer to an array of `databaseCount` valid XrTrackableImageDatabaseANDROID handles
- The `databaseCount` parameter **must** be greater than `0`

The XrTrackableImageTrackingModeANDROID enum describes the supported tracking modes of images.

```
// Provided by XR_ANDROID_trackables_image
typedef enum XrTrackableImageTrackingModeANDROID {
    XR_TRACKABLE_IMAGE_TRACKING_MODE_DYNAMIC_ANDROID = 1,
    XR_TRACKABLE_IMAGE_TRACKING_MODE_STATIC_ANDROID = 2,
    XR_TRACKABLE_IMAGE_TRACKING_MODE_MAX_ENUM_ANDROID = 0x7FFFFFFF
} XrTrackableImageTrackingModeANDROID;
```

Enumerant Descriptions

- XR_TRACKABLE_IMAGE_TRACKING_MODE_DYNAMIC_ANDROID — This mode has the highest accuracy and allows low latency tracking of moving images. It has also has the highest power consumption.
- XR_TRACKABLE_IMAGE_TRACKING_MODE_STATIC_ANDROID — This mode should be used for images that are known to be static or semi-static. This mode leads to less power consumption in comparison to the dynamic mode. If a static image is being moved, it will be updated with a much higher latency than using the dynamic mode.

The XrTrackableImageFormatANDROID enum describes the supported formats of images.

```
// Provided by XR_ANDROID_trackables_image
typedef enum XrTrackableImageFormatANDROID {
    XR_TRACKABLE_IMAGE_FORMAT_R8G8B8A8_ANDROID = 1,
    XR_TRACKABLE_IMAGE_FORMAT_MAX_ENUM_ANDROID = 0x7FFFFFFF
} XrTrackableImageFormatANDROID;
```

Enumerant Descriptions

- XR_TRACKABLE_IMAGE_FORMAT_R8G8B8A8_ANDROID — RGBA image format with 8 bits per channel color and transparency data.

The xrAddTrackableImageDatabaseANDROID function is defined as:

```
// Provided by XR_ANDROID_trackables_image
XrResult xrAddTrackableImageDatabaseANDROID(
    XrTrackableTrackerANDROID                   tracker,
    XrTrackableImageDatabaseANDROID             database);
```

Parameter Descriptions

- `tracker` is the XrTrackableTrackerANDROID handle to add the `database` to.
- `database` is the XrTrackableImageDatabaseANDROID handle to add to the `tracker`.

When an XrTrackableImageDatabaseANDROID is added to a tracker, the reference images of that database **must** be considered for detection and tracking in addition to any other databases that have been added previously with either xrAddTrackableImageDatabaseANDROID or through the XrTrackableImageConfigurationANDROID structure when initially creating the tracker.

Valid Usage (Implicit)

- The `XR_ANDROID_trackables_image` extension **must** be enabled prior to calling xrAddTrackableImageDatabaseANDROID
- `tracker` **must** be a valid XrTrackableTrackerANDROID handle
- `database` **must** be a valid XrTrackableImageDatabaseANDROID handle
- Both of `database` and `tracker` **must** have been created, allocated, or retrieved from the same XrSession

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_LIMIT_REACHED`
- `XR_ERROR_OUT_OF_MEMORY`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_VALIDATION_FAILURE`

The xrRemoveTrackableImageDatabaseANDROID function is defined as:

```
// Provided by XR_ANDROID_trackables_image
XrResult xrRemoveTrackableImageDatabaseANDROID(
    XrTrackableTrackerANDROID                   tracker,
    XrTrackableImageDatabaseANDROID             database);
```

Parameter Descriptions

- `tracker` is the XrTrackableTrackerANDROID handle to remove the `database` from.
- `database` is the XrTrackableImageDatabaseANDROID handle to remove from the `tracker`.

When an XrTrackableImageDatabaseANDROID is removed from an XrTrackableTrackerANDROID, the XrTrackableImageDatabaseEntryANDROID structures of that database **must** no longer be considered for detection and tracking. Any actively tracked entries of that database **must** no longer be reported. The removed XrTrackableImageDatabaseANDROID handle **must** not be implicitly destroyed as part of this operation.

Valid Usage (Implicit)

- The `XR_ANDROID_trackables_image` extension **must** be enabled prior to calling xrRemoveTrackableImageDatabaseANDROID
- `tracker` **must** be a valid XrTrackableTrackerANDROID handle
- `database` **must** be a valid XrTrackableImageDatabaseANDROID handle
- Both of `database` and `tracker` **must** have been created, allocated, or retrieved from the same XrSession

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

12.77.5. Getting images

The xrGetTrackableImageANDROID function is defined as:

```
// Provided by XR_ANDROID_trackables_image
XrResult xrGetTrackableImageANDROID(
    XrTrackableTrackerANDROID                   tracker,
    const XrTrackableGetInfoANDROID*            getInfo,
    XrTrackableImageANDROID*                    trackable);
```

Parameter Descriptions

- tracker is the XrTrackableTrackerANDROID to query.
- getInfo is the XrTrackableGetInfoANDROID with the information used to get the trackable QR code.
- trackable is a pointer to the XrTrackableImageANDROID structure in which the trackable image is returned.

The runtime **must** return XR_ERROR_MISMATCHING_TRACKABLE_TYPE_ANDROID if the trackable type of the XrTrackableANDROID is not XR_TRACKABLE_TYPE_IMAGE_ANDROID, or if the trackable type of the XrTrackableTrackerANDROID is not XR_TRACKABLE_TYPE_IMAGE_ANDROID.

Valid Usage (Implicit)

- The XR_ANDROID_trackables_image extension **must** be enabled prior to calling xrGetTrackableImageANDROID
- `tracker` **must** be a valid XrTrackableTrackerANDROID handle
- `getInfo` **must** be a pointer to a valid XrTrackableGetInfoANDROID structure
- `trackable` **must** be a pointer to an XrTrackableImageANDROID structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_MISMATCHING_TRACKABLE_TYPE_ANDROID
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_TIME_INVALID
- XR_ERROR_VALIDATION_FAILURE

The XrTrackableImageANDROID structure is defined as:

```
// Provided by XR_ANDROID_trackables_image
typedef struct XrTrackableImageANDROID {
    XrStructureType                   type;
    const void*                       next;
    XrTrackingStateANDROID            trackingState;
    XrTime                            lastUpdatedTime;
    XrTrackableImageDatabaseANDROID   database;
    uint32_t                          databaseEntryIndex;
    XrPosef                           centerPose;
    XrExtent2Df                       extents;
} XrTrackableImageANDROID;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `trackingState` is the XrTrackingStateANDROID of the image.
- `lastUpdatedTime` is the `XrTime` of the last update of the image.
- `database` is the XrTrackableImageDatabaseANDROID handle this image was tracked from.
- `databaseEntryIndex` is the index that maps into the XrTrackableImageDatabaseCreateInfoANDROID::`entries` array of `database`.
- `centerPose` is the XrPosef of the image located in XrTrackableGetInfoANDROID::`baseSpace`. The image lies in the XZ plane with X pointing to the right of the image and Z pointing to its bottom.
- `extents` is the XrExtent2Df dimensions of the image. The boundary of the bounding box is at points: `centerPose` +/- (`extents` / 2).

Valid Usage (Implicit)

- The `XR_ANDROID_trackables_image` extension **must** be enabled prior to using XrTrackableImageANDROID
- `type` **must** be `XR_TYPE_TRACKABLE_IMAGE_ANDROID`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `trackingState` **must** be a valid XrTrackingStateANDROID value
- `database` **must** be a valid XrTrackableImageDatabaseANDROID handle

12.77.6. Handling failures

The application **must** poll for the XrEventDataImageTrackingLostANDROID event using xrPollEvent and **must** not ignore it.

The XrEventDataImageTrackingLostANDROID structure is defined as:

```
// Provided by XR_ANDROID_trackables_image
typedef struct XrEventDataImageTrackingLostANDROID {
    XrStructureType    type;
    const void*        next;
    XrTime             time;
} XrEventDataImageTrackingLostANDROID;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `time` `XrTime`

Receiving the XrEventDataImageTrackingLostANDROID event indicates that image tracking has suffered and internal failure causing existing resources to be invalidated. The application **must** destroy all XrTrackableImageDatabaseANDROID handles and re-create them if it wishes to continue image tracking. The application **must** also destroy all image tracking related XrTrackableTrackerANDROID handles and re-create them if it wishes to continue image tracking.

Valid Usage (Implicit)

- The `XR_ANDROID_trackables_image` extension **must** be enabled prior to using XrEventDataImageTrackingLostANDROID
- `type` **must** be `XR_TYPE_EVENT_DATA_IMAGE_TRACKING_LOST_ANDROID`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

12.77.7. Example code for getting trackable images

The following example code demonstrates how to get trackable images.

```
XrInstance instance;  // Previously initialized.
XrSession session;    // Previously initialized.
XrSystemId systemId;  // Previously initialized.

// The function pointers are previously initialized using xrGetInstanceProcAddr.
PFN_xrGetSystemProperties xrGetSystemProperties;                    // Previously
initialized.
PFN_xrPollFutureEXT xrPollFutureEXT;                                // Previously
initialized.
```

```
PFN_xrCreateTrackableTrackerANDROID xrCreateTrackableTrackerANDROID;   // Previously
initialized.
PFN_xrGetAllTrackablesANDROID xrGetAllTrackablesANDROID;               // Previously
initialized.
PFN_xrDestroyTrackableTrackerANDROID xrDestroyTrackableTrackerANDROID; // Previously
initialized.

PFN_xrGetTrackableImageANDROID xrGetTrackableImageANDROID;
// Previously initialized.
PFN_xrCreateTrackableImageDatabaseAsyncANDROID
xrCreateTrackableImageDatabaseAsyncANDROID;        // Previously initialized.
PFN_xrCreateTrackableImageDatabaseCompleteANDROID
xrCreateTrackableImageDatabaseCompleteANDROID;  // Previously initialized.
PFN_xrDestroyTrackableImageDatabaseANDROID xrDestroyTrackableImageDatabaseANDROID;
// Previously initialized.
PFN_xrAddTrackableImageDatabaseANDROID xrAddTrackableImageDatabaseANDROID;
// Previously initialized.
PFN_xrRemoveTrackableImageDatabaseANDROID xrRemoveTrackableImageDatabaseANDROID;
// Previously initialized.

XrTime updateTime; // Time used for the current frame's simulation update.
XrSpace appSpace;  // Space created for XR_REFERENCE_SPACE_TYPE_LOCAL.

// Inspect system capability
XrSystemImageTrackingPropertiesANDROID imageProperty {
  .type = XR_TYPE_SYSTEM_IMAGE_TRACKING_PROPERTIES_ANDROID,
  .next = nullptr,
};
XrSystemProperties systemProperties {
  .type = XR_TYPE_SYSTEM_PROPERTIES,
  .next = &imageProperty,
};
CHK_XR(xrGetSystemProperties(instance, systemId, &systemProperties));
if (!imageProperty.supportsImageTracking) {
    // image tracking is not supported.
    return;
}

uint8_t* imageBuffer; // Load the image buffer.
uint32_t imageBufferSize; // Get the image buffer size.

XrTrackableImageDatabaseEntryANDROID imageDatabaseEntries[1] = {
  {
    .type = XR_TYPE_TRACKABLE_IMAGE_DATABASE_ENTRY_ANDROID,
    .next = nullptr,
    .trackingMode = XR_TRACKABLE_IMAGE_TRACKING_MODE_STATIC_ANDROID,
    .physicalWidth = 0.1f, // The width of the image in meters.
    .imageWidth = 640,
```

```
    .imageHeight = 480,
    .format = XR_TRACKABLE_IMAGE_FORMAT_R8G8B8A8_ANDROID,
    .bufferSize = imageBufferSize, // RGBA buffer size in bytes.
    .buffer = imageBuffer, // RGBA data.
  }
};

XrTrackableImageDatabaseCreateInfoANDROID imageDatabaseCreateInfo {
  .type = XR_TYPE_TRACKABLE_IMAGE_DATABASE_CREATE_INFO_ANDROID,
  .next = nullptr,
  .entryCount = 1,
  .entries = imageDatabaseEntries
};

XrFutureEXT imageDatabaseFuture;
CHK_XR(xrCreateTrackableImageDatabaseAsyncANDROID(session, &imageDatabaseCreateInfo,
&imageDatabaseFuture));

bool keepLooping = true;
bool futureReady = false;
while (keepLooping) {
  XrFuturePollInfoEXT pollInfo{
    .type = XR_TYPE_FUTURE_POLL_INFO_EXT,
    .future = imageDatabaseFuture,
  };
  XrFuturePollResultEXT pollResult{
    .type = XR_TYPE_FUTURE_POLL_RESULT_EXT,
  };
  CHK_XR(xrPollFutureEXT(instance, &pollInfo, &pollResult));

  if (pollResult.state == XR_FUTURE_STATE_READY_EXT) {
    futureReady = true;
    keepLooping = false;
  } else {
    // Throttle the loop to not fully expend this CPU core.
    std::this_thread::yield();
  }
}

XrTrackableImageDatabaseANDROID imageDatabase;

if (futureReady) {
  XrCreateTrackableImageDatabaseCompletionANDROID imageDatabaseCompletion {
    .type = XR_TYPE_CREATE_TRACKABLE_IMAGE_DATABASE_COMPLETION_ANDROID,
    .next = nullptr,
  };

  CHK_XR(xrCreateTrackableImageDatabaseCompleteANDROID(session, imageDatabaseFuture,
```

```
&imageDatabaseCompletion));
  CHK_XR(imageDatabaseCompletion.futureResult);
  imageDatabase = imageDatabaseCompletion.database;
}

XrTrackableImageConfigurationANDROID imageConfig {
  .type = XR_TYPE_TRACKABLE_IMAGE_CONFIGURATION_ANDROID,
  .next = nullptr,
  .databaseCount = 1,
  .databases = &imageDatabase
};

XrTrackableTrackerCreateInfoANDROID createInfo {
  .type = XR_TYPE_TRACKABLE_TRACKER_CREATE_INFO_ANDROID,
  .next = &imageConfig,
  .trackableType = XR_TRACKABLE_TYPE_IMAGE_ANDROID
};

XrTrackableTrackerANDROID imageTrackableTracker;
CHK_XR(xrCreateTrackableTrackerANDROID(session, &createInfo, &imageTrackableTracker));

XrTrackableImageDatabaseANDROID anotherImageDatabase; // Load another database.

// ... dynamically add it to the existing tracker
CHK_XR(xrAddTrackableImageDatabaseANDROID(imageTrackableTracker, anotherImageDatabase));

while (1) {
  uint32_t trackableCountOutput = 0;

  CHK_XR(xrGetAllTrackablesANDROID(imageTrackableTracker, 0, &trackableCountOutput,
nullptr));

  std::vector<XrTrackableANDROID> allImageTrackables;
  allImageTrackables.resize(trackableCountOutput);

  CHK_XR(xrGetAllTrackablesANDROID(imageTrackableTracker, 0, &trackableCountOutput,
allImageTrackables.data()));

  for (XrTrackableANDROID trackable : allImageTrackables) {
    XrTrackableGetInfoANDROID imageGetInfo {
      .type = XR_TYPE_TRACKABLE_GET_INFO_ANDROID,
      .next = nullptr,
      .trackable = trackable,
      .baseSpace = appSpace,
      .time = updateTime
    };

    XrTrackableImageANDROID trackableImage{
```

```
      .type = XR_TYPE_TRACKABLE_IMAGE_ANDROID,
    };
    CHK_XR(xrGetTrackableImageANDROID(imageTrackableTracker, &imageGetInfo,
&trackableImage));

    // Use XrTrackableImageANDROID data.
    (void)trackableImage.trackingState;
    (void)trackableImage.lastUpdatedTime;
    (void)trackableImage.centerPose;
    (void)trackableImage.extents;

    if (trackableImage.database == imageDatabase && trackableImage.databaseEntryIndex ==
0) {
      // Knowing which image the index of 0 maps to, use the specific image database
      // entry (e.g. rendering A for image A).
    }
    // indices 1+N comparisons for another specific image database entry.
  }

  // Throttle the loop to not fully expend this CPU core.
  std::this_thread::yield();
}

// Remove image database from an existing tracker to stop tracking the images
// of that specific database. To resume tracking of those images re-add the
// database at a later point.
CHK_XR(xrRemoveTrackableImageDatabaseANDROID(imageTrackableTracker,
anotherImageDatabase));

// Destroy the image tracker to stop image tracking completely. Re-creating the
// image tracker with existing image databases will restart image tracking.
CHK_XR(xrDestroyTrackableTrackerANDROID(imageTrackableTracker));

// Destroy image databases to unload the associated resources. Re-creatingd
// databases requires going through the asynchronous creation procedure again.
CHK_XR(xrDestroyTrackableImageDatabaseANDROID(anotherImageDatabase));
CHK_XR(xrDestroyTrackableImageDatabaseANDROID(imageDatabase));
```

12.77.8. Example code for managing image databases at runtime

The following example code demonstrates how to modify the set of images being tracked.

```
XrSession session; // Previously initialized.

// The function pointers are previously initialized using xrGetInstanceProcAddr.
PFN_xrCreateTrackableTrackerANDROID xrCreateTrackableTrackerANDROID;   // Previously
```

```
initialized.
PFN_xrDestroyTrackableTrackerANDROID xrDestroyTrackableTrackerANDROID; // Previously
initialized.

PFN_xrDestroyTrackableImageDatabaseANDROID xrDestroyTrackableImageDatabaseANDROID;
// Previously initialized.
PFN_xrAddTrackableImageDatabaseANDROID xrAddTrackableImageDatabaseANDROID;
// Previously initialized.
PFN_xrRemoveTrackableImageDatabaseANDROID xrRemoveTrackableImageDatabaseANDROID;
// Previously initialized.

// See previous C++ sample for database and tracker initialization.
XrTrackableImageDatabaseANDROID imageDatabases[2]; // Previously initialized.
XrTrackableImageDatabaseANDROID anotherImageDatabase; // Previously initialized.

// Create the image tracker config with two input databases to track.
XrTrackableImageConfigurationANDROID imageConfig {
  .type = XR_TYPE_TRACKABLE_IMAGE_CONFIGURATION_ANDROID,
  .next = nullptr,
  .databaseCount = 2,
  .databases = imageDatabases
};

XrTrackableTrackerCreateInfoANDROID createInfo {
  .type = XR_TYPE_TRACKABLE_TRACKER_CREATE_INFO_ANDROID,
  .next = &imageConfig,
  .trackableType = XR_TRACKABLE_TYPE_IMAGE_ANDROID
};

XrTrackableTrackerANDROID imageTrackableTracker;
CHK_XR(xrCreateTrackableTrackerANDROID(session, &createInfo, &imageTrackableTracker));

// The tracker currently tracks the images of the two databases 'imageDatabases[0]' and
// 'imageDatabases[1]' supplied through 'imageConfig'.

CHK_XR(xrRemoveTrackableImageDatabaseANDROID(imageTrackableTracker, imageDatabases[1]));

// The tracker currently tracks 'imageDatabases[0]', but no longer tracks
// 'imageDatabases[1]'.
// The 'imageDatabases[1]' database handle is still valid.

CHK_XR(xrAddTrackableImageDatabaseANDROID(imageTrackableTracker, imageDatabases[1]));

// The tracker currently tracks 'imageDatabases[0]' and 'imageDatabases[1]'.

CHK_XR(xrDestroyTrackableImageDatabaseANDROID(imageDatabases[1]));

// The tracker currently tracks 'imageDatabases[0]', but no longer tracks
```

```
// 'imageDatabases[1]'.
// The 'imageDatabases[1]' database handle is no longer valid and the corresponding
// resources have been released internally. The database needs to be re-initialized
// and re-added to resume tracking of 'imageDatabases[0]'.

CHK_XR(xrAddTrackableImageDatabaseANDROID(imageTrackableTracker, anotherImageDatabase));

// The tracker currently tracks 'imageDatabases[0]' and 'anotherImageDatabase'.

CHK_XR(xrDestroyTrackableTrackerANDROID(imageTrackableTracker));

// The 'imageTrackableTracker' tracker handle is invalid and image tracking has been
// stopped.
// The 'imageDatabases[0]' and 'anotherImageDatabase' database handles are still valid.

// Create another the image tracker config to re-create the image tracker.
XrTrackableImageConfigurationANDROID anotherImageConfig {
  .type = XR_TYPE_TRACKABLE_IMAGE_CONFIGURATION_ANDROID,
  .next = nullptr,
  .databaseCount = 1,
  .databases = &anotherImageDatabase
};

XrTrackableTrackerCreateInfoANDROID anotherCreateInfo {
  .type = XR_TYPE_TRACKABLE_TRACKER_CREATE_INFO_ANDROID,
  .next = &anotherImageConfig,
  .trackableType = XR_TRACKABLE_TYPE_IMAGE_ANDROID
};

CHK_XR(xrCreateTrackableTrackerANDROID(session, &anotherCreateInfo,
&imageTrackableTracker));

// The tracker handle has been re-initialized and image tracking has been started again.
// The tracker currently tracks 'anotherImageDatabase'.
// The 'imageDatabases[0]' database handle is still valid, but not currently tracked.
```

12.77.9. Example code for reacting to image tracking failures

The following example code demonstrates how to handle failure by polling for the XrEventDataImageTrackingLostANDROID event.

```
XrInstance instance; // Previously initialized.
XrSession session; // Previously initialized.

// The function pointers are previously initialized using xrGetInstanceProcAddr.
PFN_xrPollFutureEXT xrPollFutureEXT;                                    // Previously
```

```
initialized.
PFN_xrCreateTrackableTrackerANDROID xrCreateTrackableTrackerANDROID;   // Previously
initialized.
PFN_xrDestroyTrackableTrackerANDROID xrDestroyTrackableTrackerANDROID; // Previously
initialized.

PFN_xrCreateTrackableImageDatabaseAsyncANDROID
xrCreateTrackableImageDatabaseAsyncANDROID;        // Previously initialized.
PFN_xrCreateTrackableImageDatabaseCompleteANDROID
xrCreateTrackableImageDatabaseCompleteANDROID;  // Previously initialized.
PFN_xrDestroyTrackableImageDatabaseANDROID xrDestroyTrackableImageDatabaseANDROID;
// Previously initialized.

XrTrackableImageDatabaseEntryANDROID imageDatabaseEntries[1]; // Previously initialized.
XrTrackableImageDatabaseANDROID imageDatabase; // Previously initialized.
XrTrackableTrackerANDROID imageTrackableTracker; // Previously initialized.

// Initialize an event buffer to hold the output.
XrEventDataBuffer event = {
  .type = XR_TYPE_EVENT_DATA_BUFFER,
};
XrResult result = xrPollEvent(instance, &event);
if (result == XR_SUCCESS) {
  switch (event.type) {
    case XR_TYPE_EVENT_DATA_IMAGE_TRACKING_LOST_ANDROID: {
      const XrEventDataImageTrackingLostANDROID& eventdata =
        *reinterpret_cast<XrEventDataImageTrackingLostANDROID*>(&event);

      // All existing databases and trackers need to be destroyed.
      CHK_XR(xrDestroyTrackableTrackerANDROID(imageTrackableTracker));
      CHK_XR(xrDestroyTrackableImageDatabaseANDROID(imageDatabase));

      // To resume image tracking, the database(s) and the tracker need to be re-created.

      XrTrackableImageDatabaseCreateInfoANDROID imageDatabaseCreateInfo {
        .type = XR_TYPE_TRACKABLE_IMAGE_DATABASE_CREATE_INFO_ANDROID,
        .next = nullptr,
        .entryCount = 1,
        .entries = imageDatabaseEntries
      };

      XrFutureEXT imageDatabaseFuture;
      CHK_XR(xrCreateTrackableImageDatabaseAsyncANDROID(session, &
imageDatabaseCreateInfo, &imageDatabaseFuture));

      while (true) {
        XrFuturePollInfoEXT pollInfo{
          .type = XR_TYPE_FUTURE_POLL_INFO_EXT,
```

```
          .future = imageDatabaseFuture,
        };
        XrFuturePollResultEXT pollResult{
          .type = XR_TYPE_FUTURE_POLL_RESULT_EXT,
        };
        CHK_XR(xrPollFutureEXT(instance, &pollInfo, &pollResult));

        if (pollResult.state == XR_FUTURE_STATE_READY_EXT) {
          break;
        } else {
          // Throttle the loop to not fully expend this CPU core.
          std::this_thread::yield();
        }
      }

      XrCreateTrackableImageDatabaseCompletionANDROID imageDatabaseCompletion {
        .type = XR_TYPE_CREATE_TRACKABLE_IMAGE_DATABASE_COMPLETION_ANDROID,
        .next = nullptr,
      };

      CHK_XR(xrCreateTrackableImageDatabaseCompleteANDROID(session, imageDatabaseFuture,
&imageDatabaseCompletion));
      CHK_XR(imageDatabaseCompletion.futureResult);
      imageDatabase = imageDatabaseCompletion.database;

      XrTrackableImageConfigurationANDROID imageConfig {
       .type = XR_TYPE_TRACKABLE_IMAGE_CONFIGURATION_ANDROID,
       .next = nullptr,
       .databaseCount = 1,
       .databases = &imageDatabase
      };

      XrTrackableTrackerCreateInfoANDROID createInfo {
        .type = XR_TYPE_TRACKABLE_TRACKER_CREATE_INFO_ANDROID,
        .next = &imageConfig,
        .trackableType = XR_TRACKABLE_TYPE_IMAGE_ANDROID
      };

      XrTrackableTrackerANDROID imageTrackableTracker;
      CHK_XR(xrCreateTrackableTrackerANDROID(session, &createInfo, &
imageTrackableTracker));

      break;
    }
  }
}
```

12.77.10. New Object Types

- XrTrackableImageDatabaseANDROID

12.77.11. New Commands

- xrAddTrackableImageDatabaseANDROID
- xrCreateTrackableImageDatabaseAsyncANDROID
- xrCreateTrackableImageDatabaseCompleteANDROID
- xrDestroyTrackableImageDatabaseANDROID
- xrGetTrackableImageANDROID
- xrRemoveTrackableImageDatabaseANDROID

12.77.12. New Structures

- XrCreateTrackableImageDatabaseCompletionANDROID
- XrEventDataImageTrackingLostANDROID
- XrTrackableImageANDROID
- XrTrackableImageConfigurationANDROID
- XrTrackableImageDatabaseCreateInfoANDROID
- XrTrackableImageDatabaseEntryANDROID
- Extending XrSystemProperties:
  - XrSystemImageTrackingPropertiesANDROID

12.77.13. New Enums

- XrTrackableImageFormatANDROID
- XrTrackableImageTrackingModeANDROID

12.77.14. New Enum Constants

- `XR_ANDROID_TRACKABLES_IMAGE_EXTENSION_NAME`
- `XR_ANDROID_trackables_image_SPEC_VERSION`
- Extending XrObjectType:
  - `XR_OBJECT_TYPE_TRACKABLE_IMAGE_DATABASE_ANDROID`
- Extending XrResult:
  - `XR_ERROR_IMAGE_FORMAT_UNSUPPORTED_ANDROID`
- Extending XrStructureType:

    - XR_TYPE_CREATE_TRACKABLE_IMAGE_DATABASE_COMPLETION_ANDROID
    - XR_TYPE_EVENT_DATA_IMAGE_TRACKING_LOST_ANDROID
    - XR_TYPE_SYSTEM_IMAGE_TRACKING_PROPERTIES_ANDROID
    - XR_TYPE_TRACKABLE_IMAGE_ANDROID
    - XR_TYPE_TRACKABLE_IMAGE_CONFIGURATION_ANDROID
    - XR_TYPE_TRACKABLE_IMAGE_DATABASE_CREATE_INFO_ANDROID
    - XR_TYPE_TRACKABLE_IMAGE_DATABASE_ENTRY_ANDROID
- Extending XrTrackableTypeANDROID:
    - XR_TRACKABLE_TYPE_IMAGE_ANDROID

12.77.15. Issues

12.77.16. Version History

- Revision 1, 2025-04-08 (Daniel Guttenberg)
    - Initial extension description.

12.78. XR_ANDROID_unbounded_reference_space

Name String

XR_ANDROID_unbounded_reference_space

Extension Type

Instance extension

Registered Extension Number

468

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Last Modified Date

2024-09-12

IP Status

No known IP claims.

Contributors

Spencer Quin, Google
Jared Finder, Google
Fengtao Fan, Google
Lachlan Ford, Google
Nihav Jain, Google
Levana Chen, Google

12.78.1. Overview

This extension allows applications to create an XR_REFERENCE_SPACE_TYPE_UNBOUNDED_ANDROID reference space. This reference space enables the viewer to move freely through a complex environment, often many meters from where they started, while always optimizing for coordinate system stability near the viewer. As the device senses more of its environment to build a better scene understanding, the origin of the reference space **may** drift with **huge adjustments** as necessary to maintain device tracking.

Runtimes that support the XR_REFERENCE_SPACE_TYPE_UNBOUNDED_ANDROID reference space **must** enumerate it when applications call xrEnumerateReferenceSpaces if and only if this extension is enabled.

To create an XR_REFERENCE_SPACE_TYPE_UNBOUNDED_ANDROID reference space, the application sets XrReferenceSpaceCreateInfo::referenceSpaceType to XR_REFERENCE_SPACE_TYPE_UNBOUNDED_ANDROID when passing it to xrCreateReferenceSpace.

The XR_REFERENCE_SPACE_TYPE_UNBOUNDED_ANDROID reference space is meant to be used in conjunction with anchor spaces, in order to provide stable world tracking information to the application regardless of the viewer's movement through the environment and possible recenter events.

Possible use cases include tracking of world elements across multiple rooms of a building or tracking of world elements in large outdoor environments.

The XR_REFERENCE_SPACE_TYPE_UNBOUNDED_ANDROID reference space establishes a world-locked origin of the headset's position when the device tracking starts. It is gravity-aligned to exclude pitch and roll, with +X to the right, +Y up, and -Z forward.

XR_REFERENCE_SPACE_TYPE_UNBOUNDED_ANDROID space is useful when an application needs to render **world-scale** content that spans beyond the bounds of a single STAGE, for example, an entire floor or multiple floors of a building.

An XR_REFERENCE_SPACE_TYPE_UNBOUNDED_ANDROID space maintains stability near the viewer by adjusting its origin over time. The runtime **may** make **slight** and **huge** adjustments as necessary to maintain device tracking.

- The runtime **must** not queue the XrEventDataReferenceSpaceChangePending event in response to **minor adjustments**.
- The runtime **must** queue the XrEventDataReferenceSpaceChangePending event in response to **huge adjustments**. For example, the pose in XR_REFERENCE_SPACE_TYPE_UNBOUNDED_ANDROID space is reset due to a tracking loss and the tracking is re-established on a disconnected estimate of the world (a "new map").
- The system is constantly updating its understanding of the world and adjusting device tracking. If an application requires a persisted location regardless of tracking resets, an anchor **can** be used in this case.

The application **should** use anchor spaces and xrLocateViews to get the world locked position of elements tracked around the user.

Reference space change event

If the runtime queues an XrEventDataReferenceSpaceChangePending for XR_REFERENCE_SPACE_TYPE_UNBOUNDED_ANDROID, the application **can** use the parameters of XrEventDataReferenceSpaceChangePending to understand the relocalization event:

- XrEventDataReferenceSpaceChangePending::changeTime represents the XrTime at which relocalization completed.
- XrEventDataReferenceSpaceChangePending::poseValid is false if the new tracking session is disconnected from the previous one or true after if the new tracking session was able to relocalize and connect with the previous session.
- XrEventDataReferenceSpaceChangePending::poseInPreviousSpace is not valid when XrEventDataReferenceSpaceChangePending::poseValid is false.

When tracking relative to the XR_REFERENCE_SPACE_TYPE_UNBOUNDED_ANDROID space is recovered after a loss, runtimes **should** snap the pose of other spaces back into position relative to the XR_REFERENCE_SPACE_TYPE_UNBOUNDED_ANDROID space's original origin. However, if tracking recovers into a new tracking volume in which the original origin is unable to be located, the runtime **may** recenter the origin arbitrarily, for example moving the origin to coincide with the viewer. If such recentering occurs, the runtime **must** queue the XrEventDataReferenceSpaceChangePending event with poseValid set to false.

12.78.2. Example code for creating an unbounded reference space

The following example code demonstrates how to create an XR_REFERENCE_SPACE_TYPE_UNBOUNDED_ANDROID XrSpace.

```
XrInstance instance; // previously initialized
XrSession session; // previously initialized
XrPosef pose; // previously initialized

// Use the new reference space type in the create info struct
XrReferenceSpaceCreateInfo createInfo {
  .type = XR_TYPE_REFERENCE_SPACE_CREATE_INFO,
  .next = nullptr,
  .referenceSpaceType = XR_REFERENCE_SPACE_TYPE_UNBOUNDED_ANDROID,
  .poseInReferenceSpace = pose,
};
XrSpace referenceSpace;
CHK_XR(xrCreateReferenceSpace(session, &createInfo, &referenceSpace));

// After usage
CHK_XR(xrDestroySpace(referenceSpace));
```

12.78.3. New Enum Constants

- XR_ANDROID_UNBOUNDED_REFERENCE_SPACE_EXTENSION_NAME
- XR_ANDROID_unbounded_reference_space_SPEC_VERSION
- Extending XrReferenceSpaceType:
  - XR_REFERENCE_SPACE_TYPE_UNBOUNDED_ANDROID

12.78.4. Issues

12.78.5. Version History

- Revision 1, 2024-09-12 (Levana Chen)
  - Initial extension description

12.79. XR_BD_body_tracking

Name String

XR_BD_body_tracking

Extension Type

Instance extension

Registered Extension Number

386

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Last Modified Date

2023-06-05

IP Status

No known IP claims.

Contributors

Shanliang Xu, ByteDance
Chenxi Bao, ByteDance
Shuai Liu, ByteDance
Jun Yan, ByteDance

12.79.1. Overview

This extension enables applications to locate the individual body joints that represent the position of the user. It enables applications to render the whole body in XR experiences.

12.79.2. Inspect system capability

The XrSystemBodyTrackingPropertiesBD structure is defined as:

```
// Provided by XR_BD_body_tracking
typedef struct XrSystemBodyTrackingPropertiesBD {
    XrStructureType    type;
    void*              next;
    XrBool32           supportsBodyTracking;
} XrSystemBodyTrackingPropertiesBD;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. For more details about the structure chain, see the structure being extended (XrSystemProperties).
- `supportsBodyTracking` is an `XrBool32`, indicating if current system is capable of receiving body tracking input.

An application **can** inspect whether the system is capable of body tracking by extending the XrSystemProperties with XrSystemBodyTrackingPropertiesBD structure when calling xrGetSystemProperties.

If a runtime returns `XR_FALSE` for `supportsBodyTracking`, the runtime **must** return `XR_ERROR_FEATURE_UNSUPPORTED` from xrCreateBodyTrackerBD.

Valid Usage (Implicit)

- The `XR_BD_body_tracking` extension **must** be enabled prior to using XrSystemBodyTrackingPropertiesBD
- `type` **must** be `XR_TYPE_SYSTEM_BODY_TRACKING_PROPERTIES_BD`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

12.79.3. Create a body tracker handle

The XrBodyTrackerBD handle represents the resources for body tracking.

```
// Provided by XR_BD_body_tracking
XR_DEFINE_HANDLE(XrBodyTrackerBD)
```

This handle **can** be used to locate body joints using xrLocateBodyJointsBD function.

A body tracker provides joint locations that track human body motion.

The xrCreateBodyTrackerBD function is defined as:

```
// Provided by XR_BD_body_tracking
XrResult xrCreateBodyTrackerBD(
    XrSession                                   session,
    const XrBodyTrackerCreateInfoBD*            createInfo,
    XrBodyTrackerBD*                            bodyTracker);
```

Parameter Descriptions

- `session` is an XrSession in which the body tracker will be active.
- `createInfo` is the XrBodyTrackerCreateInfoBD used to specify the body tracker.
- `bodyTracker` is the returned XrBodyTrackerBD handle.

An application **can** create an XrBodyTrackerBD handle using xrCreateBodyTrackerBD function.

If the system does not support body tracking, the runtime **must** return `XR_ERROR_FEATURE_UNSUPPORTED` from xrCreateBodyTrackerBD. In this case, the runtime **must** return `XR_FALSE` for XrSystemBodyTrackingPropertiesBD::`supportsBodyTracking` when the function xrGetSystemProperties is called, so that the application avoids creating a body tracker.

If an invalid value of XrBodyTrackerCreateInfoBD::`jointSet` is passed in `createInfo`, the runtime **must** return `XR_ERROR_VALIDATION_FAILURE`.

Valid Usage (Implicit)

- The `XR_BD_body_tracking` extension **must** be enabled prior to calling xrCreateBodyTrackerBD
- `session` **must** be a valid XrSession handle
- `createInfo` **must** be a pointer to a valid XrBodyTrackerCreateInfoBD structure
- `bodyTracker` **must** be a pointer to an XrBodyTrackerBD handle

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FEATURE_UNSUPPORTED
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The XrBodyTrackerCreateInfoBD structure is defined as:

```
// Provided by XR_BD_body_tracking
typedef struct XrBodyTrackerCreateInfoBD {
    XrStructureType     type;
    const void*         next;
    XrBodyJointSetBD    jointSet;
} XrBodyTrackerCreateInfoBD;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- jointSet is an XrBodyJointSetBD that describes the set of body joints to retrieve.

The XrBodyTrackerCreateInfoBD structure describes the information to create an XrBodyTrackerBD handle.

Valid Usage (Implicit)

- The XR_BD_body_tracking extension **must** be enabled prior to using XrBodyTrackerCreateInfoBD
- type **must** be XR_TYPE_BODY_TRACKER_CREATE_INFO_BD
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- jointSet **must** be a valid XrBodyJointSetBD value

The XrBodyJointSetBD enum describes the set of body joints to track when creating an XrBodyTrackerBD.

```
// Provided by XR_BD_body_tracking
typedef enum XrBodyJointSetBD {
    XR_BODY_JOINT_SET_BODY_WITHOUT_ARM_BD = 1,
    XR_BODY_JOINT_SET_FULL_BODY_JOINTS_BD = 2,
    XR_BODY_JOINT_SET_MAX_ENUM_BD = 0x7FFFFFFF
} XrBodyJointSetBD;
```

The joint sets have the following meaning.

Enumerant Descriptions

- XR_BODY_JOINT_SET_BODY_WITHOUT_ARM_BD — Contains joints in XrBodyJointBD in the range XR_BODY_JOINT_PELVIS_BD through XR_BODY_JOINT_HEAD_BD inclusive, a total of XR_BODY_JOINT_WITHOUT_ARM_COUNT_BD joints.
- XR_BODY_JOINT_SET_FULL_BODY_JOINTS_BD — Contains joints in XrBodyJointBD in the range XR_BODY_JOINT_PELVIS_BD through XR_BODY_JOINT_RIGHT_HAND_BD inclusive, a total of XR_BODY_JOINT_COUNT_BD joints.

The xrDestroyBodyTrackerBD function is defined as:

```
// Provided by XR_BD_body_tracking
XrResult xrDestroyBodyTrackerBD(
    XrBodyTrackerBD                             bodyTracker);
```

Parameter Descriptions

- bodyTracker is an XrBodyTrackerBD previously created by xrCreateBodyTrackerBD.

xrDestroyBodyTrackerBD function releases the bodyTracker and the underlying resources when the body tracking experience is over.

Valid Usage (Implicit)

- The XR_BD_body_tracking extension **must** be enabled prior to calling xrDestroyBodyTrackerBD
- bodyTracker **must** be a valid XrBodyTrackerBD handle

Thread Safety

- Access to bodyTracker, and any child handles, **must** be externally synchronized

Return Codes

Success

- XR_SUCCESS

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_RUNTIME_FAILURE

12.79.4. Locate body joints

The xrLocateBodyJointsBD function is defined as:

```
// Provided by XR_BD_body_tracking
XrResult xrLocateBodyJointsBD(
    XrBodyTrackerBD                             bodyTracker,
    const XrBodyJointsLocateInfoBD*             locateInfo,
    XrBodyJointLocationsBD*                     locations);
```

Parameter Descriptions

- `bodyTracker` is an XrBodyTrackerBD previously created by xrCreateBodyTrackerBD.
- `locateInfo` is a pointer to XrBodyJointsLocateInfoBD describing information to locate body joints.
- `locations` is a pointer to XrBodyJointLocationsBD receiving the returned body joint locations.

The xrLocateBodyJointsBD function locates an array of body joints relative to a base space at a given time.

Valid Usage (Implicit)

- The `XR_BD_body_tracking` extension **must** be enabled prior to calling xrLocateBodyJointsBD
- `bodyTracker` **must** be a valid XrBodyTrackerBD handle
- `locateInfo` **must** be a pointer to a valid XrBodyJointsLocateInfoBD structure
- `locations` **must** be a pointer to an XrBodyJointLocationsBD structure

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_TIME_INVALID`
- `XR_ERROR_VALIDATION_FAILURE`

The XrBodyJointsLocateInfoBD structure is defined as:

```
// Provided by XR_BD_body_tracking
typedef struct XrBodyJointsLocateInfoBD {
    XrStructureType    type;
    const void*        next;
    XrSpace            baseSpace;
    XrTime             time;
} XrBodyJointsLocateInfoBD;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `baseSpace` is an XrSpace within which the returned body joint locations will be represented.
- `time` is an `XrTime` at which to locate the body joints.

The XrBodyJointsLocateInfoBD structure describes the information to locate body joints. Callers **should** request a time equal to the predicted display time for the rendered frame. The system will employ appropriate modeling to support body tracking at this time.

Valid Usage (Implicit)

- The `XR_BD_body_tracking` extension **must** be enabled prior to using XrBodyJointsLocateInfoBD
- `type` **must** be `XR_TYPE_BODY_JOINTS_LOCATE_INFO_BD`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `baseSpace` **must** be a valid XrSpace handle

The XrBodyJointLocationsBD structure is defined as:

```
// Provided by XR_BD_body_tracking
typedef struct XrBodyJointLocationsBD {
    XrStructureType           type;
    void*                     next;
    XrBool32                  allJointPosesTracked;
    uint32_t                  jointLocationCount;
    XrBodyJointLocationBD*    jointLocations;
} XrBodyJointLocationsBD;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `allJointPosesTracked` is an `XrBool32` indicating if all joint poses (position and orientation) are valid and tracked
- `jointLocationCount` is a `uint32_t` describing the count of elements in `jointLocations` array.
- `jointLocations` is an application-allocated array of XrBodyJointLocationBD that will be filled with joint locations.

XrBodyJointLocationsBD structure returns the state of the body joint locations.

The application **must** allocate `jointLocations` with enough elements for all joints of the chosen joint set, and set `jointLocationCount` according.

The runtime **must** populate elements of the application-allocated `jointLocations` array representing human body motion.

The runtime **must** populate the `jointLocations` array with elements indexed using the corresponding body joint enumeration (e.g. XrBodyJointBD). The index enumeration is determined by the XrBodyJointSetBD value used when creating the XrBodyTrackerBD. For example, when the XrBodyTrackerBD is created with `XR_BODY_JOINT_SET_FULL_BODY_JOINTS_BD`, the application **must** set the `jointLocationCount` to XR_BODY_JOINT_COUNT_BD, allocating at least that many elements in `jointLocations`, and the runtime **must** populate the `jointLocations` array indexed by the XrBodyJointBD enumeration.

If the value of `jointLocationCount` does not equal the joint count value associated with the XrBodyJointSetBD value used when creating the XrBodyTrackerBD, or if `jointLocations` is `NULL`, the runtime **must** return `XR_ERROR_VALIDATION_FAILURE`.

If the returned `allJointPosesTracked` is true, the runtime **must** return all joint locations with `XR_SPACE_LOCATION_POSITION_VALID_BIT`, `XR_SPACE_LOCATION_ORIENTATION_VALID_BIT`, `XR_SPACE_LOCATION_POSITION_TRACKED_BIT`, `XR_SPACE_LOCATION_ORIENTATION_TRACKED_BIT` set.

If the returned `allJointPosesTracked` is false, it indicates that for some joint(s), the body input is not detected or tracked.

Valid Usage (Implicit)

- The XR_BD_body_tracking extension **must** be enabled prior to using XrBodyJointLocationsBD
- type **must** be XR_TYPE_BODY_JOINT_LOCATIONS_BD
- next **must** be NULL or a valid pointer to the next structure in a structure chain. See also: XrBodyJointAccelerationsBD, XrBodyJointVelocitiesBD, XrBodyTrackingPostureDataBD
- jointLocations **must** be a pointer to an array of jointLocationCount XrBodyJointLocationBD structures
- The jointLocationCount parameter **must** be greater than 0

The XrBodyJointLocationBD structure is defined as:

```
// Provided by XR_BD_body_tracking
typedef struct XrBodyJointLocationBD {
    XrSpaceLocationFlags    locationFlags;
    XrPosef                 pose;
} XrBodyJointLocationBD;
```

Member Descriptions

- locationFlags is a bitfield, with bit masks defined in XrSpaceLocationFlagBits, to indicate which members contain valid data.
- pose is an XrPosef defining the position and orientation of the origin of a body joint within the reference frame of the corresponding XrBodyJointsLocateInfoBD::baseSpace.

XrBodyJointLocationBD structure describes the position and orientation of a body joint.

Valid Usage (Implicit)

- The XR_BD_body_tracking extension **must** be enabled prior to using XrBodyJointLocationBD
- locationFlags **must** be 0 or a valid combination of XrSpaceLocationFlagBits values

12.79.5. Example code for locating body joints

The following example code demonstrates how to locate all body joints relative to a base space.

```
XrInstance instance; // previously initialized
XrSystemId systemId; // previously initialized
```

```
XrSession session;   // previously initialized
XrSpace baseSpace;  // previously initialized, e.g. from
                     // XR_REFERENCE_SPACE_TYPE_LOCAL

// Inspect body tracking system properties
XrSystemBodyTrackingPropertiesBD bodyTrackingSystemProperties {
    .type = XR_TYPE_SYSTEM_BODY_TRACKING_PROPERTIES_BD,
};
XrSystemProperties systemProperties {
    .type = XR_TYPE_SYSTEM_PROPERTIES,
    .next = &bodyTrackingSystemProperties,
};
CHK_XR(xrGetSystemProperties(instance, systemId, &systemProperties));
if (!bodyTrackingSystemProperties.supportsBodyTracking) {
    // The system does not support body tracking
    return;
}

// Get function pointer for xrCreateBodyTrackerBD
PFN_xrCreateBodyTrackerBD pfnCreateBodyTrackerBD = nullptr;
CHK_XR(xrGetInstanceProcAddr(instance, "xrCreateBodyTrackerBD",
                             reinterpret_cast<PFN_xrVoidFunction*>(
                             &pfnCreateBodyTrackerBD)));

// Create a body tracker that tracks default set of body joints.
XrBodyTrackerBD bodyTracker{XR_NULL_HANDLE};
{
    XrBodyTrackerCreateInfoBD createInfo {
        .type = XR_TYPE_BODY_TRACKER_CREATE_INFO_BD,
        .jointSet = XR_BODY_JOINT_SET_FULL_BODY_JOINTS_BD,
    };
    CHK_XR(pfnCreateBodyTrackerBD(session, &createInfo, &bodyTracker));
}

// Allocate buffers to receive joint location data before frame
// loop starts.
XrBodyJointLocationBD jointLocations[XR_BODY_JOINT_COUNT_BD];
XrBodyJointLocationsBD locations {
    .type = XR_TYPE_BODY_JOINT_LOCATIONS_BD,
    .jointLocationCount = XR_BODY_JOINT_COUNT_BD,
    .jointLocations = jointLocations,
};

// Get function pointer for xrLocateBodyJointsBD.
PFN_xrLocateBodyJointsBD pfnLocateBodyJointsBD = nullptr;
CHK_XR(xrGetInstanceProcAddr(instance, "xrLocateBodyJointsBD",
                             reinterpret_cast<PFN_xrVoidFunction*>(
                             &pfnLocateBodyJointsBD)));
```

```
while (1) {
    // ...
    // For every frame in the frame loop
    // ...
    XrFrameState frameState; // previously returned from xrWaitFrame
    const XrTime time = frameState.predictedDisplayTime;

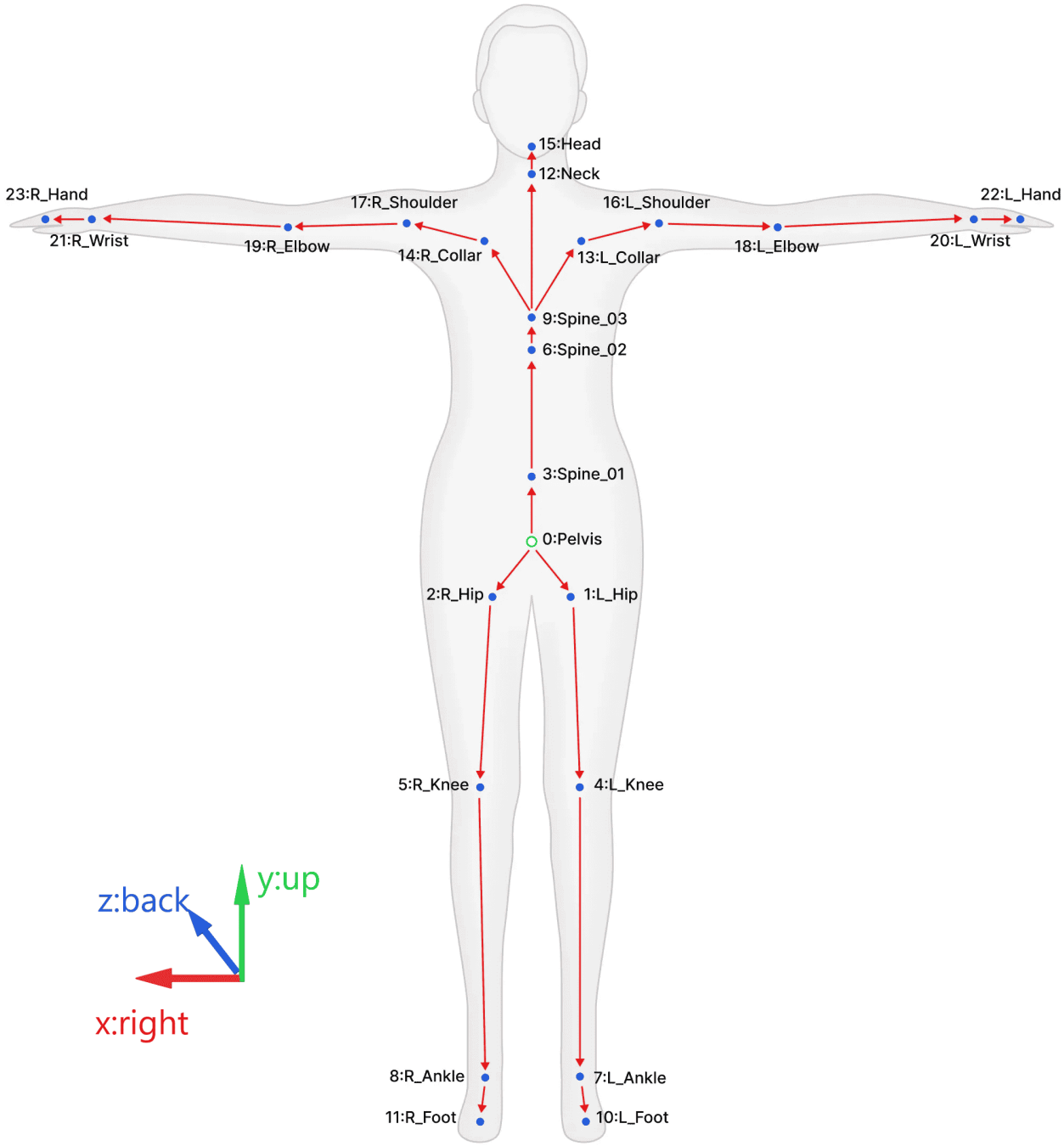
    XrBodyJointsLocateInfoBD locateInfo {
        .type = XR_TYPE_BODY_JOINTS_LOCATE_INFO_BD,
        .baseSpace = baseSpace,
        .time = time,
    };

    CHK_XR(pfnLocateBodyJointsBD(bodyTracker, &locateInfo, &locations));

    if (locations.allJointPosesTracked) {
        // The returned joint location array is directly indexed with
        // XrBodyJointBD enum.
        const XrPosef pose = jointLocations[XR_BODY_JOINT_LEFT_HAND_BD].pose;
    }
}
```

12.79.6. Conventions of body joints

This extension defines 24 body joints. The body joints are sorted starting from the XR_BODY_JOINT_PELVIS_BD and then incrementing in a spiral manner.


15:Head
12:Neck
23:R_Hand
17:R_Shoulder
16:L_Shoulder
22:L_Hand
21:R_Wrist
19:R_Elbow
14:R_Collar
13:L_Collar
18:L_Elbow
20:L_Wrist
9:Spine_03
6:Spine_02
3:Spine_01
0:Pelvis
2:R_Hip
1:L_Hip
5:R_Knee
4:L_Knee
z:back
y:up
x:right
8:R_Ankle
7:L_Ankle
11:R_Foot
10:L_Foot

```
// Provided by XR_BD_body_tracking
typedef enum XrBodyJointBD {
    XR_BODY_JOINT_PELVIS_BD = 0,
    XR_BODY_JOINT_LEFT_HIP_BD = 1,
    XR_BODY_JOINT_RIGHT_HIP_BD = 2,
    XR_BODY_JOINT_SPINE1_BD = 3,
    XR_BODY_JOINT_LEFT_KNEE_BD = 4,
    XR_BODY_JOINT_RIGHT_KNEE_BD = 5,
    XR_BODY_JOINT_SPINE2_BD = 6,
    XR_BODY_JOINT_LEFT_ANKLE_BD = 7,
    XR_BODY_JOINT_RIGHT_ANKLE_BD = 8,
    XR_BODY_JOINT_SPINE3_BD = 9,
    XR_BODY_JOINT_LEFT_FOOT_BD = 10,
    XR_BODY_JOINT_RIGHT_FOOT_BD = 11,
    XR_BODY_JOINT_NECK_BD = 12,
    XR_BODY_JOINT_LEFT_COLLAR_BD = 13,
    XR_BODY_JOINT_RIGHT_COLLAR_BD = 14,
    XR_BODY_JOINT_HEAD_BD = 15,
    XR_BODY_JOINT_LEFT_SHOULDER_BD = 16,
    XR_BODY_JOINT_RIGHT_SHOULDER_BD = 17,
    XR_BODY_JOINT_LEFT_ELBOW_BD = 18,
    XR_BODY_JOINT_RIGHT_ELBOW_BD = 19,
    XR_BODY_JOINT_LEFT_WRIST_BD = 20,
    XR_BODY_JOINT_RIGHT_WRIST_BD = 21,
    XR_BODY_JOINT_LEFT_HAND_BD = 22,
    XR_BODY_JOINT_RIGHT_HAND_BD = 23,
    XR_BODY_JOINT_MAX_ENUM_BD = 0x7FFFFFFF
} XrBodyJointBD;
```

Put body in a T-shape as above. The backward (+Z) direction is perpendicular to body surface and pointing towards the back of the body. The right (+X) direction points to the right side of the body. The Y direction is perpendicular to the X and Z directions and follows the right hand rule.

```
// Provided by XR_BD_body_tracking
#define XR_BODY_JOINT_COUNT_BD 24
```

XR_BODY_JOINT_COUNT_BD defines the number of body joint enumerants defined in the full enumeration XrBodyJointBD. This corresponds to the joint set XR_BODY_JOINT_SET_FULL_BODY_JOINTS_BD (in XrBodyJointSetBD).

```
// Provided by XR_BD_body_tracking
#define XR_BODY_JOINT_WITHOUT_ARM_COUNT_BD 16
```

XR_BODY_JOINT_WITHOUT_ARM_COUNT_BD defines the number of body joints in the joint set

XR_BODY_JOINT_SET_BODY_WITHOUT_ARM_BD (in XrBodyJointSetBD), which excludes the arms. This count includes joints indexed by XrBodyJointBD in the range XR_BODY_JOINT_PELVIS_BD through XR_BODY_JOINT_HEAD_BD inclusive.

12.79.7. New Macros

- XR_BODY_JOINT_COUNT_BD
- XR_BODY_JOINT_WITHOUT_ARM_COUNT_BD

12.79.8. New Object Types

- XrBodyTrackerBD

12.79.9. New Commands

- xrCreateBodyTrackerBD
- xrDestroyBodyTrackerBD
- xrLocateBodyJointsBD

12.79.10. New Structures

- XrBodyJointLocationBD
- XrBodyJointLocationsBD
- XrBodyJointsLocateInfoBD
- XrBodyTrackerCreateInfoBD
- Extending XrSystemProperties:
  - XrSystemBodyTrackingPropertiesBD

12.79.11. New Enums

- XrBodyJointBD
- XrBodyJointSetBD

12.79.12. New Enum Constants

- XR_BD_BODY_TRACKING_EXTENSION_NAME
- XR_BD_body_tracking_SPEC_VERSION
- Extending XrObjectType:
  - XR_OBJECT_TYPE_BODY_TRACKER_BD
- Extending XrStructureType:

- XR_TYPE_BODY_JOINTS_LOCATE_INFO_BD
- XR_TYPE_BODY_JOINT_LOCATIONS_BD
- XR_TYPE_BODY_TRACKER_CREATE_INFO_BD
- XR_TYPE_SYSTEM_BODY_TRACKING_PROPERTIES_BD

12.79.13. Version History

- Revision 1, 2023-06-05 (Shanliang Xu)
  - Initial extension description

12.80. XR_BD_body_tracking_auxiliary_metrics

Name String

XR_BD_body_tracking_auxiliary_metrics

Extension Type

Instance extension

Registered Extension Number

396

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Last Modified Date

2025-3-14

IP Status

No known IP claims.

Contributors

Shanliang Xu, Bytedance
Shuai Liu, Bytedance
Chenxi Bao, Bytedance
Chen Han, Bytedance

12.80.1. Overview

This extension enables applications to start the system's body tracking calibration process, and to obtain additional status values and information about body tracking.

Applications **should** also enable the `XR_BD_body_tracking` extension to get more data associated with body tracking.

12.80.2. Start body tracking calibration

The xrStartBodyTrackingCalibrationAppBD function is defined as:

```
// Provided by XR_BD_body_tracking_auxiliary_metrics
XrResult xrStartBodyTrackingCalibrationAppBD(
    XrSession                                   session);
```

Parameter Descriptions

- `session` is an XrSession in which the body tracker will be active.

The application **can** invoke the body tracking calibration app at any time during any session's lifecycle.

The resulting body tracking calibration is managed in an implementation defined way by the system.

Typically there is one current calibration applied globally to the entire system.

Valid Usage (Implicit)

- The `XR_BD_body_tracking_auxiliary_metrics` extension **must** be enabled prior to calling xrStartBodyTrackingCalibrationAppBD
- `session` **must** be a valid XrSession handle

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FEATURE_UNSUPPORTED
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

12.80.3. Get body tracking state

The xrGetBodyTrackingStateBD function is defined as:

```
// Provided by XR_BD_body_tracking_auxiliary_metrics
XrResult xrGetBodyTrackingStateBD(
    XrSession                                   session,
    XrBodyTrackingStateBD*                      state);
```

Parameter Descriptions

- session is an XrSession in which the body tracker will be active.
- state is a pointer to XrBodyTrackingStateBD describing the state of body tracking.

Valid Usage (Implicit)

- The XR_BD_body_tracking_auxiliary_metrics extension **must** be enabled prior to calling xrGetBodyTrackingStateBD
- session **must** be a valid XrSession handle
- state **must** be a pointer to an XrBodyTrackingStateBD structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FEATURE_UNSUPPORTED
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The XrBodyTrackingStateBD structure is defined as:

```
// Provided by XR_BD_body_tracking_auxiliary_metrics
typedef struct XrBodyTrackingStateBD {
    XrStructureType            type;
    void*                      next;
    XrBodyTrackingStatusBD     status;
    XrBodyTrackingMessageBD    message;
} XrBodyTrackingStateBD;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- status is the XrBodyTrackingStatusBD.
- message is the XrBodyTrackingMessageBD.

Valid Usage (Implicit)

- The `XR_BD_body_tracking_auxiliary_metrics` extension **must** be enabled prior to using XrBodyTrackingStateBD
- `type` **must** be `XR_TYPE_BODY_TRACKING_STATE_BD`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `status` **must** be a valid XrBodyTrackingStatusBD value
- `message` **must** be a valid XrBodyTrackingMessageBD value

The XrBodyTrackingStatusBD enum describes the body tracking status values.

```
// Provided by XR_BD_body_tracking_auxiliary_metrics
typedef enum XrBodyTrackingStatusBD {
    XR_BODY_TRACKING_STATUS_INVALID_BD = 0,
    XR_BODY_TRACKING_STATUS_VALID_BD = 1,
    XR_BODY_TRACKING_STATUS_LIMITED_BD = 2,
    XR_BODY_TRACKING_STATUS_MAX_ENUM_BD = 0x7FFFFFFF
} XrBodyTrackingStatusBD;
```

Enum	Description
`XR_BODY_TRACKING_STATUS_INVALID_BD`	Body tracking status is invalid
`XR_BODY_TRACKING_STATUS_VALID_BD`	Body tracking status is valid
`XR_BODY_TRACKING_STATUS_LIMITED_BD`	Body tracking status is limited

The XrBodyTrackingMessageBD enum describes the body tracking status messages.

```
// Provided by XR_BD_body_tracking_auxiliary_metrics
typedef enum XrBodyTrackingMessageBD {
    XR_BODY_TRACKING_MESSAGE_NO_ERROR_BD = 0,
    XR_BODY_TRACKING_MESSAGE_TRACKER_NOT_CALIBRATED_BD = 1,
    XR_BODY_TRACKING_MESSAGE_TRACKER_NUM_NOT_ENOUGH_BD = 2,
    XR_BODY_TRACKING_MESSAGE_TRACKER_STATE_NOT_SATISFIED_BD = 3,
    XR_BODY_TRACKING_MESSAGE_TRACKER_PERSISTENT_INVISIBILITY_BD = 4,
    XR_BODY_TRACKING_MESSAGE_TRACKER_DATA_ERROR_BD = 5,
    XR_BODY_TRACKING_MESSAGE_USER_CHANGE_BD = 6,
    XR_BODY_TRACKING_MESSAGE_TRACKING_POSE_ERROR_BD = 7,
    XR_BODY_TRACKING_MESSAGE_MAX_ENUM_BD = 0x7FFFFFFF
} XrBodyTrackingMessageBD;
```

Enum	Description
XR_BODY_TRACKING_MESSAGE_NO_ERROR_BD	No error
XR_BODY_TRACKING_MESSAGE_TRACKER_NOT_CALIBRATED_BD	Body tracker is not calibrated
XR_BODY_TRACKING_MESSAGE_TRACKER_NUM_NOT_ENOUGH_BD	The number of connected trackers is insufficient.
XR_BODY_TRACKING_MESSAGE_TRACKER_STATE_NOT_SATISFIED_BD	The tracker is not in normal tracking mode, e.g., it is being charged.
XR_BODY_TRACKING_MESSAGE_TRACKER_PERSISTENT_INVISIBILITY_BD	The tracker remains persistently undetected or invisible.
XR_BODY_TRACKING_MESSAGE_TRACKER_DATA_ERROR_BD	There are errors in the data generated by the tracker.
XR_BODY_TRACKING_MESSAGE_USER_CHANGE_BD	The user of the tracker may have been changed.
XR_BODY_TRACKING_MESSAGE_TRACKING_POSE_ERROR_BD	The body tracking pose is abnormal, e.g. the head is twisted.

12.80.4. Get more body tracking data

The XrBodyTrackingPostureDataBD structure is defined as:

```
// Provided by XR_BD_body_tracking_auxiliary_metrics
typedef struct XrBodyTrackingPostureDataBD {
    XrStructureType             type;
    void*                       next;
    uint32_t                    postureCount;
    XrBodyTrackingPostureBD*    postureData;
} XrBodyTrackingPostureDataBD;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `postureCount` is a `uint32_t` describing the element count in the `postureData` array.
- `postureData` is an application-allocated array of XrBodyTrackingPostureBD describing the availability and state of other signals.

XrBodyTrackingPostureDataBD **can** be provided in the `next` chain of XrBodyJointLocationsBD when calling xrLocateBodyJointsBD to get the specific posture state on the body joints.

Valid Usage (Implicit)

- The XR_BD_body_tracking_auxiliary_metrics extension **must** be enabled prior to using XrBodyTrackingPostureDataBD
- type **must** be XR_TYPE_BODY_TRACKING_POSTURE_DATA_BD
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- postureData **must** be a pointer to an array of postureCount XrBodyTrackingPostureBD values
- The postureCount parameter **must** be greater than 0

```
// Provided by XR_BD_body_tracking_auxiliary_metrics
typedef enum XrBodyTrackingPostureBD {
    XR_BODY_TRACKING_POSTURE_STOMP_BD = 1,
    XR_BODY_TRACKING_POSTURE_STATIC_BD = 2,
    XR_BODY_TRACKING_POSTURE_MAX_ENUM_BD = 0x7FFFFFFF
} XrBodyTrackingPostureBD;
```

Enum	Description
XR_BODY_TRACKING_POSTURE_STOMP_BD	Represents a stomp posture.
XR_BODY_TRACKING_POSTURE_STATIC_BD	Represents a static posture where the tracked user maintains a fixed pose.

Currently XrBodyTrackingPostureBD is applicable to XR_BODY_JOINT_LEFT_ANKLE_BD and XR_BODY_JOINT_RIGHT_ANKLE_BD (in XrBodyJointBD).

The runtime **must** populate the data corresponding to the indices of XR_BODY_JOINT_LEFT_ANKLE_BD and XR_BODY_JOINT_RIGHT_ANKLE_BD with valid data and set all others to 0.

The XrBodyJointVelocitiesBD structure is defined as:

```
// Provided by XR_BD_body_tracking_auxiliary_metrics
typedef struct XrBodyJointVelocitiesBD {
    XrStructureType           type;
    void*                     next;
    uint32_t                  velocityCount;
    XrBodyJointVelocityBD*    velocities;
} XrBodyJointVelocitiesBD;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `velocityCount` is a `uint32_t` describing the count of elements in `velocities` array.
- `velocities` is an application-allocated array of XrBodyJointVelocityBD to be filled with body joints velocities.

XrBodyJointVelocitiesBD **can** be provided in the `next` chain of XrBodyJointLocationsBD when calling xrLocateBodyJointsBD to get an array of body joints velocities.

Valid Usage (Implicit)

- The `XR_BD_body_tracking_auxiliary_metrics` extension **must** be enabled prior to using XrBodyJointVelocitiesBD
- `type` **must** be `XR_TYPE_BODY_JOINT_VELOCITIES_BD`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `velocities` **must** be a pointer to an array of `velocityCount` XrBodyJointVelocityBD structures
- The `velocityCount` parameter **must** be greater than `0`

```
// Provided by XR_BD_body_tracking_auxiliary_metrics
typedef struct XrBodyJointVelocityBD {
    XrSpaceVelocityFlags    velocityFlags;
    XrVector3f              linearVelocity;
    XrVector3f              angularVelocity;
} XrBodyJointVelocityBD;
```

The XrBodyJointAccelerationsBD structure is defined as:

```
// Provided by XR_BD_body_tracking_auxiliary_metrics
typedef struct XrBodyJointAccelerationsBD {
    XrStructureType                 type;
    void*                           next;
    uint32_t                        accelerationCount;
    XrBodyJointAccelerationBD*      accelerations;
} XrBodyJointAccelerationsBD;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `accelerationCount` is a `uint32_t` describing the count of elements in `accelerations` array.
- `accelerations` is an application-allocated array of XrBodyJointAccelerationBD describing body joints accelerations.

XrBodyJointAccelerationsBD **can** be provided in the `next` chain of XrBodyJointLocationsBD when calling xrLocateBodyJointsBD to populate an array of body joints accelerations.

Valid Usage (Implicit)

- The `XR_BD_body_tracking_auxiliary_metrics` extension **must** be enabled prior to using XrBodyJointAccelerationsBD
- `type` **must** be `XR_TYPE_BODY_JOINT_ACCELERATIONS_BD`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `accelerations` **must** be a pointer to an array of `accelerationCount` XrBodyJointAccelerationBD structures
- The `accelerationCount` parameter **must** be greater than `0`

The XrBodyJointAccelerationBD structure is defined as:

Member Descriptions

- `accelerationFlags` is a bitfield, with bit masks defined in XrSpaceAccelerationFlagBitsBD, to indicate which members contain valid data. If none of the bits are set, no other fields in this structure **should** be considered to be valid or meaningful.
- `linearAcceleration` is the linear acceleration of XrBodyJointLocationsBD::`jointLocations` with respect to and expressed in the reference frame of XrBodyJointsLocateInfoBD::`baseSpace`.
- `angularAcceleration` is the relative angular acceleration of XrBodyJointLocationsBD::`jointLocations` with respect to XrBodyJointsLocateInfoBD::`baseSpace`.

```
// Provided by XR_BD_body_tracking_auxiliary_metrics
typedef struct XrBodyJointAccelerationBD {
    XrSpaceAccelerationFlagsBD    accelerationFlags;
    XrVector3f                    linearAcceleration;
    XrVector3f                    angularAcceleration;
} XrBodyJointAccelerationBD;
```

```
// Provided by XR_BD_body_tracking_auxiliary_metrics
typedef XrFlags64 XrSpaceAccelerationFlagsBD;
```

```
// Provided by XR_BD_body_tracking_auxiliary_metrics
// Flag bits for XrSpaceAccelerationFlagsBD
static const XrSpaceAccelerationFlagsBD XR_SPACE_ACCELERATION_LINEAR_VALID_BIT_BD =
0x00000001;
static const XrSpaceAccelerationFlagsBD XR_SPACE_ACCELERATION_ANGULAR_VALID_BIT_BD =
0x00000002;
```

Flag Descriptions

- XR_SPACE_ACCELERATION_LINEAR_VALID_BIT_BD — Linear acceleration.
- XR_SPACE_ACCELERATION_ANGULAR_VALID_BIT_BD — Angular acceleration.

The output results of velocity and acceleration data for each body joint are not necessarily the same.

Not every body joint necessarily provide valid values for linear velocity, angular velocity, linear acceleration, and angular acceleration.

12.80.5. New Commands

- xrGetBodyTrackingStateBD
- xrStartBodyTrackingCalibrationAppBD

12.80.6. New Structures

- XrBodyJointAccelerationBD
- XrBodyJointVelocityBD
- XrBodyTrackingStateBD
- Extending XrBodyJointLocationsBD:
  - XrBodyJointAccelerationsBD

    - XrBodyJointVelocitiesBD
    - XrBodyTrackingPostureDataBD

12.80.7. New Enums

- XrBodyTrackingMessageBD
- XrBodyTrackingPostureBD
- XrBodyTrackingStatusBD
- XrSpaceAccelerationFlagBitsBD

12.80.8. New Bitmasks

- XrSpaceAccelerationFlagsBD

12.80.9. New Enum Constants

- `XR_BD_BODY_TRACKING_AUXILIARY_METRICS_EXTENSION_NAME`
- `XR_BD_body_tracking_auxiliary_metrics_SPEC_VERSION`
- Extending XrStructureType:
    - `XR_TYPE_BODY_JOINT_ACCELERATIONS_BD`
    - `XR_TYPE_BODY_JOINT_VELOCITIES_BD`
    - `XR_TYPE_BODY_TRACKING_POSTURE_DATA_BD`
    - `XR_TYPE_BODY_TRACKING_STATE_BD`

12.80.10. Version History

- Revision 1, 2025-3-14 (Shanliang Xu)
    - Initial extension description

12.81. XR_BD_dynamic_object_keyboard

Name String

`XR_BD_dynamic_object_keyboard`

Extension Type

Instance extension

Registered Extension Number

748

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

XR_BD_dynamic_object_tracking

Last Modified Date

2025-08-05

IP Status

No known IP claims.

Contributors

Zijian Wang, ByteDance
Zhipeng Liu, ByteDance
Shuai Liu, ByteDance

12.81.1. Overview

The XR_BD_dynamic_object_keyboard extension enables the keyboard to be tracked as a type of dynamic object by the XR_BD_dynamic_object_tracking extension.

12.81.2. Inspect System Capability

The XrSystemDynamicObjectKeyboardPropertiesBD structure is defined as:

```
// Provided by XR_BD_dynamic_object_keyboard
typedef struct XrSystemDynamicObjectKeyboardPropertiesBD {
    XrStructureType    type;
    void*              next;
    XrBool32           supportsDynamicObjectKeyboard;
} XrSystemDynamicObjectKeyboardPropertiesBD;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `supportsDynamicObjectKeyboard` is an `XrBool32`, with `XR_TRUE` indicating that the system supports tracking keyboard, and `XR_FALSE` indicating that the system does not support tracking keyboard.

An application **can** inspect whether the system is capable of tracking keyboards by chaining an XrSystemDynamicObjectKeyboardPropertiesBD structure to the XrSystemProperties::`next` chain when calling xrGetSystemProperties.

If XrSystemSpatialSensingPropertiesBD::`supportsSpatialSensing` is `XR_FALSE` or XrSystemDynamicObjectTrackingPropertiesBD::`supportsDynamicObjectTracking` is `XR_FALSE`, then `supportsDynamicObjectKeyboard` **must** also be `XR_FALSE`.

If a runtime returns `XR_TRUE` for `supportsDynamicObjectKeyboard`, the system supports dynamic keyboard tracking. This implies that XrSystemDynamicObjectTrackingPropertiesBD::`supportsDynamicObjectTracking` and XrSystemSpatialSensingPropertiesBD::`supportsSpatialSensing` **must** also be `XR_TRUE`.

Note that `supportsDynamicObjectKeyboard` **may** be `XR_TRUE`, even if the application has not obtained the permission required by the system.

Evaluation of permissions takes place later, in the asynchronous operation started by xrStartSenseDataProviderAsyncBD.

Valid Usage (Implicit)

- The `XR_BD_dynamic_object_keyboard` extension **must** be enabled prior to using XrSystemDynamicObjectKeyboardPropertiesBD
- `type` **must** be `XR_TYPE_SYSTEM_DYNAMIC_OBJECT_KEYBOARD_PROPERTIES_BD`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

12.81.3. The Keyboard Dynamic Object

Applications **can** obtain component data through xrGetSpatialEntityComponentDataBD from the spatial entity of a keyboard, such as location, semantic label, or bounding box.

Applications **can** also create an XrSpace for the spatial entity of a keyboard and obtain the location through xrLocateSpace or xrLocateSpaces.

See more details in Locate Dynamic Object.

The origin of the keyboard space is located at the geometric center point inside the bounding box, and the positive Z axis extends outward normal to the typing face, and the X-Y plane of the keyboard matches the intended viewing orientation when typing. On a QWERTY keyboard, the positive Y direction extends from the center of the keyboard to the row of number keys, and the positive X direction extends from the center of the keyboard toward the side with the return or enter key.

12.81.4. New Structures

- Extending XrSystemProperties:
  - XrSystemDynamicObjectKeyboardPropertiesBD

12.81.5. New Enum Constants

- `XR_BD_DYNAMIC_OBJECT_KEYBOARD_EXTENSION_NAME`
- `XR_BD_dynamic_object_keyboard_SPEC_VERSION`
- Extending XrDynamicObjectTypeBD:
  - `XR_DYNAMIC_OBJECT_TYPE_KEYBOARD_BD`
- Extending XrStructureType:
  - `XR_TYPE_SYSTEM_DYNAMIC_OBJECT_KEYBOARD_PROPERTIES_BD`

12.81.6. Issues

12.81.7. Version History

- Revision 1, 2025-08-05 (Zijian Wang)
  - Initial extension description

12.82. XR_BD_dynamic_object_mouse

Name String

`XR_BD_dynamic_object_mouse`

Extension Type

Instance extension

Registered Extension Number

749

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

XR_BD_dynamic_object_tracking

Last Modified Date

2025-08-05

IP Status

No known IP claims.

Contributors

Zijian Wang, ByteDance
Zhipeng Liu, ByteDance
Shuai Liu, ByteDance

12.82.1. Overview

The XR_BD_dynamic_object_mouse extension enables a computer mouse to be tracked as a type of dynamic object by the XR_BD_dynamic_object_tracking extension.

12.82.2. Inspect System Capability

The XrSystemDynamicObjectMousePropertiesBD structure is defined as:

```
// Provided by XR_BD_dynamic_object_mouse
typedef struct XrSystemDynamicObjectMousePropertiesBD {
    XrStructureType    type;
    void*              next;
    XrBool32           supportsDynamicObjectMouse;
} XrSystemDynamicObjectMousePropertiesBD;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain.
- supportsDynamicObjectMouse is an XrBool32, with XR_TRUE indicating that the system supports tracking a computer mouse, and XR_FALSE indicating that the system does not support tracking a computer mouse.

An application **can** inspect whether the system is capable of tracking a computer mouse by chaining an

XrSystemDynamicObjectMousePropertiesBD structure to the XrSystemProperties::next chain when calling xrGetSystemProperties.

If XrSystemSpatialSensingPropertiesBD::supportsSpatialSensing is XR_FALSE or XrSystemDynamicObjectTrackingPropertiesBD::supportsDynamicObjectTracking is XR_FALSE, then supportsDynamicObjectMouse **must** also be XR_FALSE.

If a runtime returns XR_TRUE for supportsDynamicObjectMouse, the system supports dynamic computer mouse tracking. This implies that XrSystemDynamicObjectTrackingPropertiesBD::supportsDynamicObjectTracking and XrSystemSpatialSensingPropertiesBD::supportsSpatialSensing **must** also be XR_TRUE.

Note that supportsDynamicObjectMouse **may** be XR_TRUE, even if the application has not obtained the permission required by the system.

Evaluation of permissions takes place later, in the asynchronous operation started by xrStartSenseDataProviderAsyncBD.

Valid Usage (Implicit)

- The XR_BD_dynamic_object_mouse extension **must** be enabled prior to using XrSystemDynamicObjectMousePropertiesBD
- type **must** be XR_TYPE_SYSTEM_DYNAMIC_OBJECT_MOUSE_PROPERTIES_BD
- next **must** be NULL or a valid pointer to the next structure in a structure chain

12.82.3. The Mouse Dynamic Object

Applications **can** obtain component data through xrGetSpatialEntityComponentDataBD from the spatial entity of a computer mouse, such as location, semantic label, or bounding sphere.

The origin of the mouse space is located at the center of the sphere, and no direction is distinguished. Applications **can** create an XrSpace for the spatial entity of the mouse and obtain location through xrLocateSpace or xrLocateSpaces.

See more details in Locate Dynamic Object.

12.82.4. New Structures

- Extending XrSystemProperties:
  - XrSystemDynamicObjectMousePropertiesBD

12.82.5. New Enum Constants

- XR_BD_DYNAMIC_OBJECT_MOUSE_EXTENSION_NAME

- XR_BD_dynamic_object_mouse_SPEC_VERSION
- Extending XrDynamicObjectTypeBD:
  - XR_DYNAMIC_OBJECT_TYPE_MOUSE_BD
- Extending XrStructureType:
  - XR_TYPE_SYSTEM_DYNAMIC_OBJECT_MOUSE_PROPERTIES_BD

12.82.6. Issues

12.82.7. Version History

- Revision 1, 2025-08-05 (Zijian Wang)
  - Initial extension description

12.83. XR_BD_dynamic_object_tracking

Name String

XR_BD_dynamic_object_tracking

Extension Type

Instance extension

Registered Extension Number

747

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

XR_BD_spatial_sensing

Last Modified Date

2025-08-05

IP Status

No known IP claims.

Contributors

Zijian Wang, ByteDance
Zhipeng Liu, ByteDance

Shuai Liu, ByteDance

12.83.1. Overview

This extension allows applications to track different types of dynamic objects in the physical environment, such as keyboard and mouse. The tracked objects are represented by spatial entities (from XR_BD_spatial_sensing), and information is obtained in the form of components, such as the position, orientation, and bounding box of a dynamic object. Each dynamic object is continuously tracked by the runtime and its information is queried as defined in XR_BD_spatial_sensing.

This extension requires XR_BD_spatial_sensing to be enabled.

12.83.2. Permission

A runtime on an AOSP-based platform **must** verify that applications have the com.picovr.permission.SPATIAL_DATA permission both listed in their manifest and granted to use dynamic object tracking. Without it, runtime **must** set XrFutureCompletionEXT::futureResult to XR_ERROR_PERMISSION_INSUFFICIENT when the xrStartSenseDataProviderCompleteBD is called.

This is an auto-requested permission: if it is listed in the manifest but not yet granted or denied, the runtime **must** prompt the user to grant or deny the permission when xrCreateSenseDataProviderBD is called with a provider type that requires it.

This permission is also used by XR_BD_spatial_anchor.

This permission is also used by XR_BD_spatial_mesh.

This permission is also used by XR_BD_spatial_scene.

This permission is also used by XR_BD_spatial_plane.

12.83.3. Inspect System Capability

The XrSystemDynamicObjectTrackingPropertiesBD structure is defined as:

```
// Provided by XR_BD_dynamic_object_tracking
typedef struct XrSystemDynamicObjectTrackingPropertiesBD {
    XrStructureType    type;
    void*              next;
    XrBool32           supportsDynamicObjectTracking;
} XrSystemDynamicObjectTrackingPropertiesBD;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `supportsDynamicObjectTracking` is an `XrBool32`, indicating if the current system is capable of dynamic object tracking.

An application **can** inspect whether the system is capable of dynamic object tracking by chaining an XrSystemDynamicObjectTrackingPropertiesBD structure to the XrSystemProperties::`next` chain when calling xrGetSystemProperties.

If a runtime returns `XR_FALSE` for `supportsDynamicObjectTracking`, the system does not support dynamic object tracking, and the runtime **must** return `XR_ERROR_FEATURE_UNSUPPORTED` from xrCreateSenseDataProviderBD when passing the XrSenseDataProviderTypeBD value `XR_SENSE_DATA_PROVIDER_TYPE_DYNAMIC_OBJECT_BD`.

If XrSystemSpatialSensingPropertiesBD::`supportsSpatialSensing` is `XR_FALSE`, then `supportsDynamicObjectTracking` **must** also be `XR_FALSE`.

If a runtime returns `XR_TRUE` for `supportsDynamicObjectTracking`, the system supports dynamic object tracking. This implies that XrSystemSpatialSensingPropertiesBD::`supportsSpatialSensing` **must** also be `XR_TRUE`.

Note that `supportsDynamicObjectTracking` **may** be `XR_TRUE` even if running on an Android-based platform and the application does not have the required com.picovr.permission.SPATIAL_DATA permission both declared in the manifest and granted at runtime. Evaluation of permissions takes place later, in the asynchronous operation started by xrStartSenseDataProviderAsyncBD.

Valid Usage (Implicit)

- The `XR_BD_dynamic_object_tracking` extension **must** be enabled prior to using XrSystemDynamicObjectTrackingPropertiesBD
- `type` **must** be `XR_TYPE_SYSTEM_DYNAMIC_OBJECT_TRACKING_PROPERTIES_BD`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

12.83.4. Create Dynamic Object Provider

The XrSenseDataProviderCreateInfoDynamicObjectBD structure is defined as:

```
// Provided by XR_BD_dynamic_object_tracking
typedef struct XrSenseDataProviderCreateInfoDynamicObjectBD {
    XrStructureType                 type;
    const void*                     next;
    uint32_t                        trackingTypeCount;
    const XrDynamicObjectTypeBD*    trackingTypes;
} XrSenseDataProviderCreateInfoDynamicObjectBD;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `trackingTypeCount` is the number of XrDynamicObjectTypeBD elements in `trackingTypes`.
- `trackingTypes` is a pointer to the XrDynamicObjectTypeBD array.

The XrSenseDataProviderCreateInfoDynamicObjectBD structure defines the information required for dynamic object tracking, including a list of trackable types and all the details required to track them.

An application creates an XrSenseDataProviderBD handle representing a dynamic object tracking provider by calling xrCreateSenseDataProviderBD, setting XrSenseDataProviderCreateInfoBD::`providerType` to `XR_SENSE_DATA_PROVIDER_TYPE_DYNAMIC_OBJECT_BD`, and chaining XrSenseDataProviderCreateInfoDynamicObjectBD to XrSenseDataProviderCreateInfoBD::`next`.

Both the XrSenseDataProviderTypeBD value and the chained XrSenseDataProviderCreateInfoDynamicObjectBD structure are required. If XrSenseDataProviderCreateInfoBD::`providerType` is `XR_SENSE_DATA_PROVIDER_TYPE_DYNAMIC_OBJECT_BD` but XrSenseDataProviderCreateInfoDynamicObjectBD is not in the `next` chain, the runtime **must** return `XR_ERROR_VALIDATION_FAILURE`.

Valid Usage (Implicit)

- The `XR_BD_dynamic_object_tracking` extension **must** be enabled prior to using XrSenseDataProviderCreateInfoDynamicObjectBD
- `type` **must** be `XR_TYPE_SENSE_DATA_PROVIDER_CREATE_INFO_DYNAMIC_OBJECT_BD`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `trackingTypes` **must** be a pointer to an array of `trackingTypeCount` valid XrDynamicObjectTypeBD values
- The `trackingTypeCount` parameter **must** be greater than `0`

XrDynamicObjectTypeBD represents the dynamic object categories supported by the runtime for

detection and **can** be set when creating the XrSenseDataProviderBD.

The XrDynamicObjectTypeBD enumeration is defined as:

```
// Provided by XR_BD_dynamic_object_tracking
typedef enum XrDynamicObjectTypeBD {
    XR_DYNAMIC_OBJECT_TYPE_UNKNOWN_BD = 0,
  // Provided by XR_BD_dynamic_object_keyboard
    XR_DYNAMIC_OBJECT_TYPE_KEYBOARD_BD = 1000747000,
  // Provided by XR_BD_dynamic_object_mouse
    XR_DYNAMIC_OBJECT_TYPE_MOUSE_BD = 1000748000,
    XR_DYNAMIC_OBJECT_TYPE_MAX_ENUM_BD = 0x7FFFFFFF
} XrDynamicObjectTypeBD;
```

Enum	Description
XR_DYNAMIC_OBJECT_TYPE_UNKNOWN_BD	Represents an unknown object type.
XR_DYNAMIC_OBJECT_TYPE_KEYBOARD_BD	Represents a keyboard (Added by the XR_BD_dynamic_object_keyboard extension)
XR_DYNAMIC_OBJECT_TYPE_MOUSE_BD	Represents a mouse (Added by the XR_BD_dynamic_object_mouse extension)

12.83.5. Start Dynamic Object Tracking Provider

Applications start the dynamic object tracking data provider by calling xrStartSenseDataProviderAsyncBD after it has been successfully created. To check the data provider state, call xrGetSenseDataProviderStateBD.

Subsequent application operations using this provider handle **must** not be performed unless the data provider state is XR_SENSE_DATA_PROVIDER_STATE_RUNNING_BD. If the data provider state is not XR_SENSE_DATA_PROVIDER_STATE_RUNNING_BD and the application needs to use the provider, the application **must** take appropriate action and try to call xrStartSenseDataProviderAsyncBD again before using the handle.

Detailed definitions and usage details are described in XR_BD_spatial_sensing.

12.83.6. Query Dynamic Object Data

Applications query the latest detected dynamic objects from the data provider by calling xrQuerySenseDataAsyncBD. The runtime generates a snapshot of the dynamic object, from which the application **can** obtain detailed information. The dynamic object information is presented in the form of spatial entities which the application queries by calling xrGetQueriedSenseDataBD. Use xrEnumerateSpatialEntityComponentTypesBD to get the types of components contained within these entities, such as location, semantics, and bounding boxes. To further retrieve component information, call xrGetSpatialEntityComponentDataBD.

The XrSpatialEntityComponentDataDynamicObjectBD structure describes dynamic object detail data. This structure **may** be attached to a spatial entity via the XR_SPATIAL_ENTITY_COMPONENT_TYPE_DYNAMIC_OBJECT_BD component type.

When the runtime's understanding of the detected dynamic objects changes, the runtime **must** queue an XrEventDataSenseDataUpdatedBD event with the handle of the dynamic object tracking provider.

Receiving this event signals to the application that updated dynamic object data is available to query from the sense data provider. A new query request for this sense data provider will get the latest, updated data. This is the recommended way to get updated data.

The application **may** also initiate a new query request at any time without receiving this event, but the queried data **may** be unmodified.

All the functions to get dynamic object info are defined in XR_BD_spatial_sensing.

The XrSpatialEntityComponentDataDynamicObjectBD structure is defined as:

```
// Provided by XR_BD_dynamic_object_tracking
typedef struct XrSpatialEntityComponentDataDynamicObjectBD {
    XrStructureType          type;
    void*                    next;
    XrDynamicObjectDataBD    data;
} XrSpatialEntityComponentDataDynamicObjectBD;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain.
- data is the XrDynamicObjectDataBD of the dynamic object.

XrSpatialEntityComponentDataDynamicObjectBD is an output struct for getting the dynamic object component data from an entity in the snapshot. This corresponds to an XrSpatialEntityComponentTypeBD value of XR_SPATIAL_ENTITY_COMPONENT_TYPE_DYNAMIC_OBJECT_BD.

All spatial entities describing dynamic objects **must** have the XR_SPATIAL_ENTITY_COMPONENT_TYPE_DYNAMIC_OBJECT_BD component enabled by the runtime.

Valid Usage (Implicit)

- The XR_BD_dynamic_object_tracking extension **must** be enabled prior to using XrSpatialEntityComponentDataDynamicObjectBD
- type **must** be XR_TYPE_SPATIAL_ENTITY_COMPONENT_DATA_DYNAMIC_OBJECT_BD
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- data **must** be a valid XrDynamicObjectDataBD structure

The XrDynamicObjectDataBD structure is defined as:

```
// Provided by XR_BD_dynamic_object_tracking
typedef struct XrDynamicObjectDataBD {
    XrStructureType          type;
    void*                    next;
    XrDynamicObjectTypeBD    objectType;
} XrDynamicObjectDataBD;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain.
- objectType is the XrDynamicObjectTypeBD of the dynamic object.

Valid Usage (Implicit)

- The XR_BD_dynamic_object_tracking extension **must** be enabled prior to using XrDynamicObjectDataBD
- type **must** be XR_TYPE_DYNAMIC_OBJECT_DATA_BD
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- objectType **must** be a valid XrDynamicObjectTypeBD value

12.83.7. Dynamic Object Type Filter

The XrSenseDataFilterDynamicObjectTypeBD structure is defined as:

```
// Provided by XR_BD_dynamic_object_tracking
typedef struct XrSenseDataFilterDynamicObjectTypeBD {
    XrStructureType                 type;
    const void*                     next;
    uint32_t                        typeCount;
    const XrDynamicObjectTypeBD*    types;
} XrSenseDataFilterDynamicObjectTypeBD;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `typeCount` is the number of XrDynamicObjectTypeBD elements in `types`.
- `types` is a pointer to an array of XrDynamicObjectTypeBD.

The XrSenseDataFilterDynamicObjectTypeBD structure contains a list of dynamic object types. When the application passes this filter, only sense data that matches the criteria in the filter is included in the result returned. The runtime **must** not include sense data that does not match the provided filter.

When the application initiates a query request, it **can** chain an XrSenseDataFilterDynamicObjectTypeBD structure to XrSenseDataQueryInfoBD::`next` to filter dynamic object data.

Valid Usage (Implicit)

- The `XR_BD_dynamic_object_tracking` extension **must** be enabled prior to using XrSenseDataFilterDynamicObjectTypeBD
- `type` **must** be `XR_TYPE_SENSE_DATA_FILTER_DYNAMIC_OBJECT_TYPE_BD`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `types` **must** be a pointer to an array of `typeCount` valid XrDynamicObjectTypeBD values
- The `typeCount` parameter **must** be greater than `0`

12.83.8. Locate Dynamic Object

Locating a dynamic object relative to a base space is performed by creating anchor objects, then through use of an XrSpace handle created from the anchor and functions like xrLocateSpace and xrLocateSpaces. To locate a dynamic object in a base space, first create an XrAnchorBD from a dynamic object spatial entity by calling xrCreateSpatialEntityAnchorBD. Then use this anchor handle to create an XrSpace by calling xrCreateAnchorSpaceBD, and obtain the location of the dynamic object

by calling xrLocateSpace and xrLocateSpaces.

The function and related behaviors are defined in XR_BD_spatial_sensing.

12.83.9. New Structures

- XrDynamicObjectDataBD
- Extending XrSenseDataProviderCreateInfoBD:
  - XrSenseDataProviderCreateInfoDynamicObjectBD
- Extending XrSenseDataQueryInfoBD:
  - XrSenseDataFilterDynamicObjectTypeBD
- Extending XrSpatialEntityComponentDataBaseHeaderBD:
  - XrSpatialEntityComponentDataDynamicObjectBD
- Extending XrSystemProperties:
  - XrSystemDynamicObjectTrackingPropertiesBD

12.83.10. New Enums

- XrDynamicObjectTypeBD

12.83.11. New Enum Constants

- XR_BD_DYNAMIC_OBJECT_TRACKING_EXTENSION_NAME
- XR_BD_dynamic_object_tracking_SPEC_VERSION
- Extending XrSenseDataProviderTypeBD:
  - XR_SENSE_DATA_PROVIDER_TYPE_DYNAMIC_OBJECT_BD
- Extending XrSpatialEntityComponentTypeBD:
  - XR_SPATIAL_ENTITY_COMPONENT_TYPE_DYNAMIC_OBJECT_BD
- Extending XrStructureType:
  - XR_TYPE_DYNAMIC_OBJECT_DATA_BD
  - XR_TYPE_SENSE_DATA_FILTER_DYNAMIC_OBJECT_TYPE_BD
  - XR_TYPE_SENSE_DATA_PROVIDER_CREATE_INFO_DYNAMIC_OBJECT_BD
  - XR_TYPE_SPATIAL_ENTITY_COMPONENT_DATA_DYNAMIC_OBJECT_BD
  - XR_TYPE_SYSTEM_DYNAMIC_OBJECT_TRACKING_PROPERTIES_BD

Issues

Version History

- Revision 1, 2025-08-05 (Zijian Wang)
  - Initial extension description

12.84. XR_BD_facial_simulation

Name String

`XR_BD_facial_simulation`

Extension Type

Instance extension

Registered Extension Number

387

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Last Modified Date

2024-10-17

IP Status

No known IP claims.

Contributors

Shanliang Xu, ByteDance
Chenxi Bao, ByteDance
Shuai Liu, ByteDance
Junhuan Peng, ByteDance
Zhicheng Chen, ByteDance

12.84.1. Overview

This extension allows applications to get weights of blend shapes which express the users' face movements.

This extension defines several modes in XrFacialSimulationModeBD. These modes correspond to different output types of blend shapes, such as the XrFaceExpressionBD or XrLipExpressionBD, or even

a combination of them.

Because not all modes involve direct use of private data like images, and some modes are closer to an audio-driven estimation of facial expression, this extension is referred to as Facial Expression Simulation instead of Facial Expression Tracking.



Permissions

Android applications **must** have the com.picovr.permission.FACE_TRACKING permission and the android.permission.RECORD_AUDIO listed in their manifest and granted to use this extension, otherwise xrCreateFaceTrackerBD function will return a XR_ERROR_PERMISSION_INSUFFICIENT error. (protection level: dangerous)

12.84.2. Inspect system capability

The XrSystemFacialSimulationPropertiesBD structure is defined as:

```
// Provided by XR_BD_facial_simulation
typedef struct XrSystemFacialSimulationPropertiesBD {
    XrStructureType    type;
    void*              next;
    XrBool32           supportsFaceTracking;
} XrSystemFacialSimulationPropertiesBD;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. For more details about the structure chain, see the structure being extended (XrSystemProperties).
- `supportsFaceTracking` is an `XrBool32`, indicating if the current system is capable of receiving face tracking input.

An application **can** inspect whether the system is capable of providing face tracking input by extending the XrSystemProperties::`next` chain with an XrSystemFacialSimulationPropertiesBD structure when calling xrGetSystemProperties.

If a runtime returns `XR_FALSE` for `supportsFaceTracking`, the runtime **must** return `XR_ERROR_FEATURE_UNSUPPORTED` from xrCreateFaceTrackerBD.

Valid Usage (Implicit)

- The `XR_BD_facial_simulation` extension **must** be enabled prior to using XrSystemFacialSimulationPropertiesBD
- `type` **must** be `XR_TYPE_SYSTEM_FACIAL_SIMULATION_PROPERTIES_BD`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

12.84.3. Enumerating tracking modes

The application **can** enumerate the supported modes by calling the xrEnumerateFacialSimulationModesBD function. The list of available modes depends on the internal facial simulation algorithm.

The xrEnumerateFacialSimulationModesBD function is defined as:

```
// Provided by XR_BD_facial_simulation
XrResult xrEnumerateFacialSimulationModesBD(
    XrSession                                   session,
    uint32_t                                    modeCapacityInput,
    uint32_t*                                   modeCountOutput,
    XrFacialSimulationModeBD*                   modes);
```

Parameter Descriptions

- `session` is an XrSession previously created with flinl:xrCreateSession.
- `modeCapacityInput` is the capacity of the `modes` array, or `0` to retrieve the required capacity.
- `modeCountOutput` is a pointer to the count of `modes` written, or a pointer to the required capacity in the case that `modeCapacityInput` is insufficient.
- `modes` is a pointer to an array of XrFacialSimulationModeBD values, but **can** be `NULL` if `modeCapacityInput` is `0`.
- See the Buffer Size Parameters section for a detailed description of retrieving the required `modes` size.

Valid Usage (Implicit)

- The XR_BD_facial_simulation extension **must** be enabled prior to calling xrEnumerateFacialSimulationModesBD
- session **must** be a valid XrSession handle
- modeCountOutput **must** be a pointer to a uint32_t value
- If modeCapacityInput is not 0, modes **must** be a pointer to an array of modeCapacityInput XrFacialSimulationModeBD values

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_SIZE_INSUFFICIENT
- XR_ERROR_VALIDATION_FAILURE

This extension provides four modes to express facial blend shapes. Some modes just need voice input for audio-driven facial expression simulation. Other modes require image input, making use of face tracking.

The XrFacialSimulationModeBD enumeration describes the modes of facial simulation and tracking.

```
// Provided by XR_BD_facial_simulation
typedef enum XrFacialSimulationModeBD {
    XR_FACIAL_SIMULATION_MODE_DEFAULT_BD = 0,
    XR_FACIAL_SIMULATION_MODE_COMBINED_AUDIO_BD = 1,
    XR_FACIAL_SIMULATION_MODE_COMBINED_AUDIO_WITH_LIP_BD = 2,
    XR_FACIAL_SIMULATION_MODE_ONLY_AUDIO_WITH_LIP_BD = 3,
    XR_FACIAL_SIMULATION_MODE_MAX_ENUM_BD = 0x7FFFFFFF
} XrFacialSimulationModeBD;
```

Enumerant Descriptions

- `XR_FACIAL_SIMULATION_MODE_DEFAULT_BD` — When calling xrGetFacialSimulationDataBD, the output contains the XrFacialSimulationDataBD only.
- `XR_FACIAL_SIMULATION_MODE_COMBINED_AUDIO_BD` — Results in the same output as the `Default` mode, furthermore, runtime can provide a precise result by using the audio input.
- `XR_FACIAL_SIMULATION_MODE_COMBINED_AUDIO_WITH_LIP_BD` — Separates the result into two parts, one is XrFacialSimulationDataBD, the other is XrLipExpressionDataBD.
- `XR_FACIAL_SIMULATION_MODE_ONLY_AUDIO_WITH_LIP_BD` — Uses only audio input to produce the output result as XrLipExpressionDataBD.

When using `XR_FACIAL_SIMULATION_MODE_DEFAULT_BD` and `XR_FACIAL_SIMULATION_MODE_COMBINED_AUDIO_BD`, only XrFacialSimulationDataBD is populated in the output.

When using `XR_FACIAL_SIMULATION_MODE_COMBINED_AUDIO_WITH_LIP_BD` the application **must** chain an XrLipExpressionDataBD to XrFacialSimulationDataBD::`next` when calling xrGetFacialSimulationDataBD.

When using `XR_FACIAL_SIMULATION_MODE_ONLY_AUDIO_WITH_LIP_BD` the runtime **may** generate predicted animations in face expression in XrFacialSimulationDataBD and XrLipExpressionDataBD structures when calling xrGetFacialSimulationDataBD, to make the avatar look more natural.

12.84.4. Create a face tracker handle

The XrFaceTrackerBD handle represents the resources for face tracking.

```
// Provided by XR_BD_facial_simulation
 XR_DEFINE_HANDLE(XrFaceTrackerBD)
```

This handle is used to obtain blend shapes using the xrGetFacialSimulationDataBD function.

The xrCreateFaceTrackerBD function is defined as:

```
// Provided by XR_BD_facial_simulation
XrResult xrCreateFaceTrackerBD(
    XrSession                                   session,
    const XrFaceTrackerCreateInfoBD*            createInfo,
    XrFaceTrackerBD*                            tracker);
```

Parameter Descriptions

- `session` is an XrSession in which the face tracker will be active.
- `createInfo` is the XrFaceTrackerCreateInfoBD used to configure the face tracker.
- `tracker` is the returned XrFaceTrackerBD handle.

An application **can** create an XrFaceTrackerBD handle by calling the xrCreateFaceTrackerBD function, specifying a supported mode from those defined in XrFacialSimulationModeBD.

If the runtime does not support the specified mode provided in `createInfo` in XrFaceTrackerCreateInfoBD, the runtime **must** return `XR_ERROR_FEATURE_UNSUPPORTED` from xrCreateFaceTrackerBD. An application **can** get supported modes by calling the xrEnumerateFacialSimulationModesBD function.

Valid Usage (Implicit)

- The `XR_BD_facial_simulation` extension **must** be enabled prior to calling xrCreateFaceTrackerBD
- `session` **must** be a valid XrSession handle
- `createInfo` **must** be a pointer to a valid XrFaceTrackerCreateInfoBD structure
- `tracker` **must** be a pointer to an XrFaceTrackerBD handle

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FEATURE_UNSUPPORTED
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_PERMISSION_INSUFFICIENT
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The XrFaceTrackerCreateInfoBD structure is defined as:

```
// Provided by XR_BD_facial_simulation
typedef struct XrFaceTrackerCreateInfoBD {
    XrStructureType             type;
    const void*                 next;
    XrFacialSimulationModeBD    mode;
} XrFaceTrackerCreateInfoBD;
```

Member Description

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- mode is an XrFacialSimulationModeBD, indicating the mode that the tracker will use.

Valid Usage (Implicit)

- The XR_BD_facial_simulation extension **must** be enabled prior to using XrFaceTrackerCreateInfoBD
- type **must** be XR_TYPE_FACE_TRACKER_CREATE_INFO_BD
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- mode **must** be a valid XrFacialSimulationModeBD value

12.84.5. Destroy a face tracker handle

The xrDestroyFaceTrackerBD function is defined as:

```
// Provided by XR_BD_facial_simulation
XrResult xrDestroyFaceTrackerBD(
    XrFaceTrackerBD                             tracker);
```

Parameter Descriptions

- tracker is an XrFaceTrackerBD previously created by xrCreateFaceTrackerBD.

When facial expression tracking is no longer needed, release the tracker and the resources associated with the tracker through the xrDestroyFaceTrackerBD function.

Valid Usage (Implicit)

- The XR_BD_facial_simulation extension **must** be enabled prior to calling xrDestroyFaceTrackerBD
- tracker **must** be a valid XrFaceTrackerBD handle

Thread Safety

- Access to tracker, and any child handles, **must** be externally synchronized

Return Codes

Success

- XR_SUCCESS

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_RUNTIME_FAILURE

12.84.6. Get facial expressions

The xrGetFacialSimulationDataBD function is defined as:

```
// Provided by XR_BD_facial_simulation
XrResult xrGetFacialSimulationDataBD(
    XrFaceTrackerBD                             tracker,
    const XrFacialSimulationDataGetInfoBD*      info,
    XrFacialSimulationDataBD*                   facialData);
```

Parameter Descriptions

- tracker is the face tracker created by xrCreateFaceTrackerBD
- info is a pointer to XrFacialSimulationDataGetInfoBD describing information to get face expression.
- facialData is a pointer to XrFacialSimulationDataBD receiving the returned facial expression.

The xrGetFacialSimulationDataBD function returns the blend shapes of facial expression.

Valid Usage (Implicit)

- The XR_BD_facial_simulation extension **must** be enabled prior to calling xrGetFacialSimulationDataBD
- tracker **must** be a valid XrFaceTrackerBD handle
- info **must** be a pointer to a valid XrFacialSimulationDataGetInfoBD structure
- facialData **must** be a pointer to an XrFacialSimulationDataBD structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_TIME_INVALID
- XR_ERROR_VALIDATION_FAILURE

The XrFacialSimulationDataGetInfoBD structure is defined as:

```
// Provided by XR_BD_facial_simulation
typedef struct XrFacialSimulationDataGetInfoBD {
    XrStructureType    type;
    const void*        next;
    XrTime             time;
} XrFacialSimulationDataGetInfoBD;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain.
- time is an XrTime at which the facial expression weights are requested.

The XrFacialSimulationDataGetInfoBD structure describes the information to get facial expression. Callers **should** request a time equal to the predicted display time for the rendered frame. The system **should** employ appropriate modeling to provide expressions for this time.

Valid Usage (Implicit)

- The XR_BD_facial_simulation extension **must** be enabled prior to using XrFacialSimulationDataGetInfoBD
- type **must** be XR_TYPE_FACIAL_SIMULATION_DATA_GET_INFO_BD
- next **must** be NULL or a valid pointer to the next structure in a structure chain

The XrFacialSimulationDataBD structure is defined as:

```
// Provided by XR_BD_facial_simulation
typedef struct XrFacialSimulationDataBD {
    XrStructureType    type;
    void*              next;
    uint32_t           faceExpressionWeightCount;
    float*             faceExpressionWeights;
    XrBool32           isUpperFaceDataValid;
    XrBool32           isLowerFaceDataValid;
    XrTime             time;
} XrFacialSimulationDataBD;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain.
- faceExpressionWeightCount is a uint32_t describing the count of elements in faceExpressionWeights array.
- faceExpressionWeights is a array of float that will be filled with weights of facial expression blend shapes.
- isUpperFaceDataValid is an XrBool32 representing the validity of upper face, generally the eyes and its surrounding area
- isLowerFaceDataValid is an XrBool32 representing the validity of lower face, generally the mouth and its surrounding area
- time is the XrTime time at which the returned expression weights were tracked

The XrFacialSimulationDataBD structure contains the facial expression weight. See XrFacialSimulationModeBD for details and for how this interacts with lip expression tracking.

The runtime **must** fill the faceExpressionWeights array ordered so that it is indexed using the corresponding facial expression enum XrFaceExpressionBD.

An application **must** preallocate the output faceExpressionWeights array that can contain at least faceExpressionWeightCount of float. The value of faceExpressionWeightCount **must** be XR_FACE_EXPRESSION_COUNT_BD.

Valid Usage (Implicit)

- The XR_BD_facial_simulation extension **must** be enabled prior to using XrFacialSimulationDataBD
- type **must** be XR_TYPE_FACIAL_SIMULATION_DATA_BD
- next **must** be NULL or a valid pointer to the next structure in a structure chain. See also: XrLipExpressionDataBD
- faceExpressionWeights **must** be a pointer to an array of faceExpressionWeightCount float values
- The faceExpressionWeightCount parameter **must** be greater than 0

The XrLipExpressionDataBD structure is defined as:

```
// Provided by XR_BD_facial_simulation
typedef struct XrLipExpressionDataBD {
    XrStructureType    type;
    void*              next;
    uint32_t           lipsyncExpressionWeightCount;
    float*             lipsyncExpressionWeights;
} XrLipExpressionDataBD;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- lipsyncExpressionWeightCount is a uint32_t describing the count of elements in lipsyncExpressionWeights array.
- lipsyncExpressionWeights is a array of float that will be filled with weights of lip expression blend shapes.

The XrLipExpressionDataBD structure contains the lip expression blend weights. See XrFacialSimulationModeBD for details and for how this interacts with facial expression tracking.

If an application creates XrFaceTrackerBD by calling xrCreateFaceTrackerBD using the mode XR_FACIAL_SIMULATION_MODE_COMBINED_AUDIO_WITH_LIP_BD or the mode

XR_FACIAL_SIMULATION_MODE_ONLY_AUDIO_WITH_LIP_BD, meaning the application wants to get the lip expression in XrLipExpressionBD, the application **must** chain an XrLipExpressionDataBD to XrFacialSimulationDataBD::next when calling xrGetFacialSimulationDataBD. If no structure containing lip data is passed, the structure on this chain will be ignored.

The runtime **must** return lipsyncExpressionWeights representing the weights of the blend shapes of the current lip expression.

The runtime **must** update the lipsyncExpressionWeights array ordered so that it is indexed using the corresponding lip expression enum XrLipExpressionBD.

An application **must** preallocate the output lipsyncExpressionWeights array that can contain at least lipsyncExpressionWeightCount of float. lipsyncExpressionWeightCount **must** be XR_LIP_EXPRESSION_COUNT_BD.

Valid Usage (Implicit)

- The XR_BD_facial_simulation extension **must** be enabled prior to using XrLipExpressionDataBD
- type **must** be XR_TYPE_LIP_EXPRESSION_DATA_BD
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- lipsyncExpressionWeights **must** be a pointer to an array of lipsyncExpressionWeightCount float values
- The lipsyncExpressionWeightCount parameter **must** be greater than 0

```
// Provided by XR_BD_facial_simulation
#define XR_FACE_EXPRESSION_COUNT_BD 52
```

XR_FACE_EXPRESSION_COUNT_BD is the number of blend shapes of XrFaceExpressionBD.

```
// Provided by XR_BD_facial_simulation
#define XR_LIP_EXPRESSION_COUNT_BD 20
```

XR_LIP_EXPRESSION_COUNT_BD is the number of blend shapes of XrLipExpressionBD.

12.84.7. Set/Get facial simulation mode

Application **can** query the mode of the tracker which has been created by calling xrGetFacialSimulationModeBD, and change the tracker into another mode by calling xrSetFacialSimulationModeBD.

The xrGetFacialSimulationModeBD function is defined as:

```
// Provided by XR_BD_facial_simulation
XrResult xrGetFacialSimulationModeBD(
    XrFaceTrackerBD                             tracker,
    XrFacialSimulationModeBD*                   mode);
```

Parameter Descriptions

- `tracker` is an XrFaceTrackerBD that has been created by xrCreateFaceTrackerBD
- `mode` is a pointer to an XrFacialSimulationModeBD value.

Valid Usage (Implicit)

- The `XR_BD_facial_simulation` extension **must** be enabled prior to calling xrGetFacialSimulationModeBD
- `tracker` **must** be a valid XrFaceTrackerBD handle
- `mode` **must** be a pointer to an XrFacialSimulationModeBD value

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_VALIDATION_FAILURE`

The xrSetFacialSimulationModeBD function is defined as:

```
// Provided by XR_BD_facial_simulation
XrResult xrSetFacialSimulationModeBD(
    XrFaceTrackerBD                             tracker,
    XrFacialSimulationModeBD                    mode);
```

Parameter Descriptions

- `tracker` is an XrFaceTrackerBD that has been created by xrCreateFaceTrackerBD
- `mode` is a value defined in xrEnumerateFacialSimulationModesBD that the application wants to assign to `tracker`

The runtime **must** return `XR_ERROR_FEATURE_UNSUPPORTED` if the runtime does not support the new mode.

Valid Usage (Implicit)

- The `XR_BD_facial_simulation` extension **must** be enabled prior to calling xrSetFacialSimulationModeBD
- `tracker` **must** be a valid XrFaceTrackerBD handle
- `mode` **must** be a valid XrFacialSimulationModeBD value

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_VALIDATION_FAILURE`

12.84.8. Example code for getting facial expression and lip expression

The code below shows how to use this extension.

```
XrInstance instance; // previously initialized
XrSystemId systemId; // previously initialized
XrSession session;   // previously initialized

XrSystemFacialSimulationPropertiesBD facialSimulationProperties {
    .type = XR_TYPE_SYSTEM_FACIAL_SIMULATION_PROPERTIES_BD,
};
XrSystemProperties sysProperties {
    .type = XR_TYPE_SYSTEM_PROPERTIES,
    .next = &facialSimulationProperties,
};
CHK_XR(xrGetSystemProperties(instance, systemId, &sysProperties));

if (facialSimulationProperties.supportsFaceTracking == false) {
  return;
}

uint32_t modeCount = 0;
CHK_XR(xrEnumerateFacialSimulationModesBD(session, 0, &modeCount, nullptr));
std::vector<XrFacialSimulationModeBD> facialSimulationModes(modeCount);
CHK_XR(xrEnumerateFacialSimulationModesBD(session, modeCount, &modeCount,
                                          facialSimulationModes.data()));

XrFaceTrackerBD faceTracker{XR_NULL_HANDLE};
XrFaceTrackerCreateInfoBD info {
    .type = XR_TYPE_FACE_TRACKER_CREATE_INFO_BD,
    .mode = XR_FACIAL_SIMULATION_MODE_DEFAULT_BD,
};
CHK_XR(xrCreateFaceTrackerBD(session, &info, &faceTracker));

XrFrameState frameState; // previously returned from xrWaitFrame
const XrTime time = frameState.predictedDisplayTime;

XrFacialSimulationDataGetInfoBD infoBd {
  .type = XR_TYPE_FACIAL_SIMULATION_DATA_GET_INFO_BD,
  .time = time,
};

XrFacialSimulationDataGetInfoBD getInfo {
    .type = XR_TYPE_FACIAL_SIMULATION_DATA_GET_INFO_BD,
    .time = time,
};
std::vector<float> faceExpressionWeights(XR_FACE_EXPRESSION_COUNT_BD);
XrFacialSimulationDataBD facedata {
    .type = XR_TYPE_FACIAL_SIMULATION_DATA_BD,
    .faceExpressionWeightCount = XR_FACE_EXPRESSION_COUNT_BD,
    .faceExpressionWeights = faceExpressionWeights.data(),
```

```
};

CHK_XR(xrGetFacialSimulationDataBD(faceTracker, &getInfo, &facedata));
assert(facedata.isUpperFaceDataValid == true);
assert(facedata.isLowerFaceDataValid == true);

for (auto blendWeight : faceExpressionWeights) {
  std::cout << blendWeight << '\n'; // print each data of face expression
}

// change mode from default to COMBINED_AUDIO_WITH_LIP
CHK_XR(xrSetFacialSimulationModeBD(
    faceTracker, XR_FACIAL_SIMULATION_MODE_COMBINED_AUDIO_WITH_LIP_BD));

std::vector<float> lipExpressionWeights(XR_LIP_EXPRESSION_COUNT_BD);
XrLipExpressionDataBD lipdata {
    .type = XR_TYPE_LIP_EXPRESSION_DATA_BD,
    .lipsyncExpressionWeightCount = XR_LIP_EXPRESSION_COUNT_BD,
    .lipsyncExpressionWeights = lipExpressionWeights.data(),
};

facedata.next = &lipdata;
CHK_XR(xrGetFacialSimulationDataBD(faceTracker, &getInfo, &facedata));

for (auto blendWeight : lipExpressionWeights) {
  std::cout << blendWeight << '\n'; // print each data of lip expression
}

XrFacialSimulationModeBD currentMode;
CHK_XR(xrGetFacialSimulationModeBD(faceTracker, &currentMode));
assert(currentMode == XR_FACIAL_SIMULATION_MODE_COMBINED_AUDIO_WITH_LIP_BD);

CHK_XR(xrDestroyFaceTrackerBD(faceTracker));
```

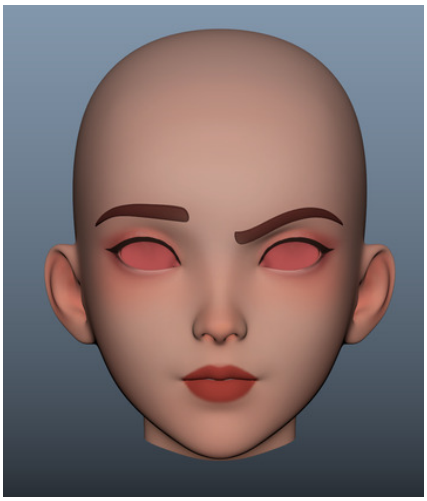
12.84.9. Conventions of blend shapes

This extension defines 52 blend shapes for facial expression and 20 blend shapes for lip expression.

```
// Provided by XR_BD_facial_simulation
typedef enum XrFaceExpressionBD {
    XR_FACE_EXPRESSION_BROW_DROP_L_BD = 0,
    XR_FACE_EXPRESSION_BROW_DROP_R_BD = 1,
    XR_FACE_EXPRESSION_BROW_INNER_UPWARDS_BD = 2,
    XR_FACE_EXPRESSION_BROW_OUTER_UPWARDS_L_BD = 3,
    XR_FACE_EXPRESSION_BROW_OUTER_UPWARDS_R_BD = 4,
    XR_FACE_EXPRESSION_EYE_BLINK_L_BD = 5,
```

```
    XR_FACE_EXPRESSION_EYE_LOOK_DROP_L_BD = 6,
    XR_FACE_EXPRESSION_EYE_LOOK_IN_L_BD = 7,
    XR_FACE_EXPRESSION_EYE_LOOK_OUT_L_BD = 8,
    XR_FACE_EXPRESSION_EYE_LOOK_UPWARDS_L_BD = 9,
    XR_FACE_EXPRESSION_EYE_LOOK_SQUINT_L_BD = 10,
    XR_FACE_EXPRESSION_EYE_LOOK_WIDE_L_BD = 11,
    XR_FACE_EXPRESSION_EYE_BLINK_R_BD = 12,
    XR_FACE_EXPRESSION_EYE_LOOK_DROP_R_BD = 13,
    XR_FACE_EXPRESSION_EYE_LOOK_IN_R_BD = 14,
    XR_FACE_EXPRESSION_EYE_LOOK_OUT_R_BD = 15,
    XR_FACE_EXPRESSION_EYE_LOOK_UPWARDS_R_BD = 16,
    XR_FACE_EXPRESSION_EYE_LOOK_SQUINT_R_BD = 17,
    XR_FACE_EXPRESSION_EYE_LOOK_WIDE_R_BD = 18,
    XR_FACE_EXPRESSION_NOSE_SNEER_L_BD = 19,
    XR_FACE_EXPRESSION_NOSE_SNEER_R_BD = 20,
    XR_FACE_EXPRESSION_CHEEK_PUFF_BD = 21,
    XR_FACE_EXPRESSION_CHEEK_SQUINT_L_BD = 22,
    XR_FACE_EXPRESSION_CHEEK_SQUINT_R_BD = 23,
    XR_FACE_EXPRESSION_MOUTH_CLOSE_BD = 24,
    XR_FACE_EXPRESSION_MOUTH_FUNNEL_BD = 25,
    XR_FACE_EXPRESSION_MOUTH_PUCKER_BD = 26,
    XR_FACE_EXPRESSION_MOUTH_L_BD = 27,
    XR_FACE_EXPRESSION_MOUTH_R_BD = 28,
    XR_FACE_EXPRESSION_MOUTH_SMILE_L_BD = 29,
    XR_FACE_EXPRESSION_MOUTH_SMILE_R_BD = 30,
    XR_FACE_EXPRESSION_MOUTH_FROWN_L_BD = 31,
    XR_FACE_EXPRESSION_MOUTH_FROWN_R_BD = 32,
    XR_FACE_EXPRESSION_MOUTH_DIMPLE_L_BD = 33,
    XR_FACE_EXPRESSION_MOUTH_DIMPLE_R_BD = 34,
    XR_FACE_EXPRESSION_MOUTH_STRETCH_L_BD = 35,
    XR_FACE_EXPRESSION_MOUTH_STRETCH_R_BD = 36,
    XR_FACE_EXPRESSION_MOUTH_ROLL_LOWER_BD = 37,
    XR_FACE_EXPRESSION_MOUTH_ROLL_UPPER_BD = 38,
    XR_FACE_EXPRESSION_MOUTH_SHRUG_LOWER_BD = 39,
    XR_FACE_EXPRESSION_MOUTH_SHRUG_UPPER_BD = 40,
    XR_FACE_EXPRESSION_MOUTH_PRESS_L_BD = 41,
    XR_FACE_EXPRESSION_MOUTH_PRESS_R_BD = 42,
    XR_FACE_EXPRESSION_MOUTH_LOWER_DROP_L_BD = 43,
    XR_FACE_EXPRESSION_MOUTH_LOWER_DROP_R_BD = 44,
    XR_FACE_EXPRESSION_MOUTH_UPPER_UPWARDS_L_BD = 45,
    XR_FACE_EXPRESSION_MOUTH_UPPER_UPWARDS_R_BD = 46,
    XR_FACE_EXPRESSION_JAW_FORWARD_BD = 47,
    XR_FACE_EXPRESSION_JAW_L_BD = 48,
    XR_FACE_EXPRESSION_JAW_R_BD = 49,
    XR_FACE_EXPRESSION_JAW_OPEN_BD = 50,
    XR_FACE_EXPRESSION_TONGUE_OUT_BD = 51,
    XR_FACE_EXPRESSION_MAX_ENUM_BD = 0x7FFFFFFF
} XrFaceExpressionBD;
```

XR_FACE_EXPRESSION_BROW_DROP_L_BD



Description
The downward movement of the outer portion of the left eyebrow.

XR_FACE_EXPRESSION_BROW_DROP_R_BD



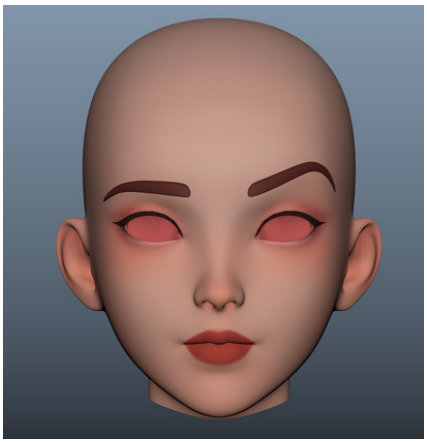
Description
The downward movement of the outer portion of the right eyebrow.

XR_FACE_EXPRESSION_BROW_INNER_UPWARDS_BD



Description
The upward movement of the inner portion of both eyebrows.

XR_FACE_EXPRESSION_BROW_OUTER_UPWARDS_L_BD



Description
The upward movement of the outer portion of the left eyebrow.

XR_FACE_EXPRESSION_BROW_OUTER_UPWARDS_R_BD



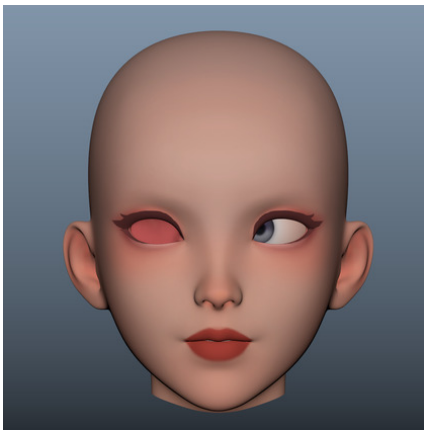
Description
The upward movement of the outer portion of the right eyebrow.

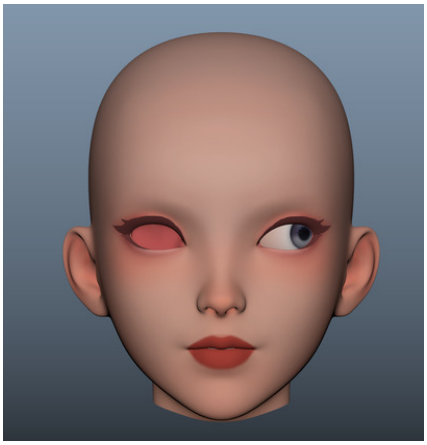
XR_FACE_EXPRESSION_EYE_BLINK_L_BD



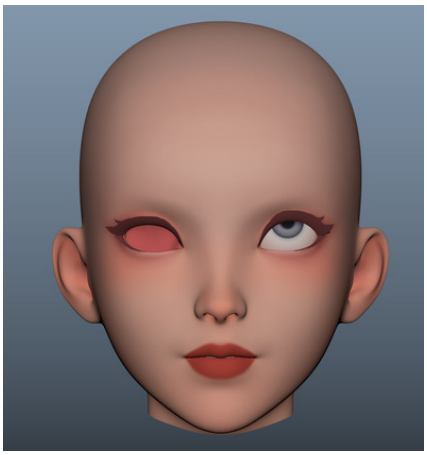
Description
The closure of the eyelids over the left eye.

XR_FACE_EXPRESSION_EYE_LOOK_DROP_L_BD	
	**Description** The movement of the left eyelids consistent with a downward gaze.

XR_FACE_EXPRESSION_EYE_LOOK_IN_L_BD	
	**Description** The movement of the left eyelids consistent with a rightward gaze.

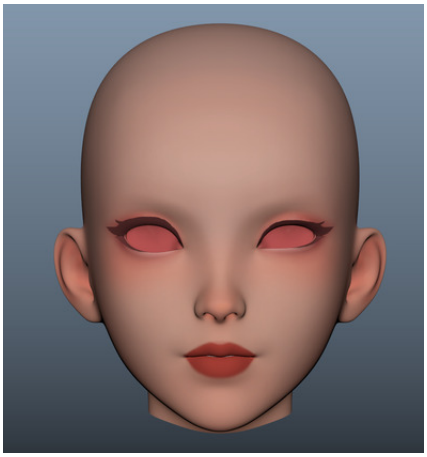
XR_FACE_EXPRESSION_EYE_LOOK_OUT_L_BD	
	**Description** The movement of the left eyelids consistent with a leftward gaze.

XR_FACE_EXPRESSION_EYE_LOOK_UPWARDS_L_BD



Description
The movement of the left eyelids consistent with an upward gaze.

XR_FACE_EXPRESSION_EYE_LOOK_SQUINT_L_BD



Description
The contraction of the face around the left eye.

XR_FACE_EXPRESSION_EYE_LOOK_WIDE_L_BD



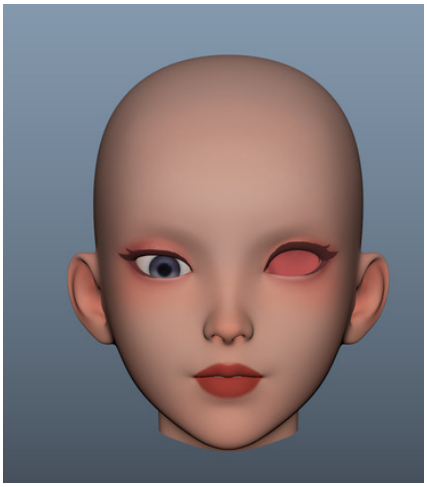
Description
The widening of the eyelids around the left eye.

`XR_FACE_EXPRESSION_EYE_BLINK_R_BD`



Description
The closure of the eyelids over the right eye.

`XR_FACE_EXPRESSION_EYE_LOOK_DROP_R_BD`



Description
The movement of the right eyelids consistent with a downward gaze.

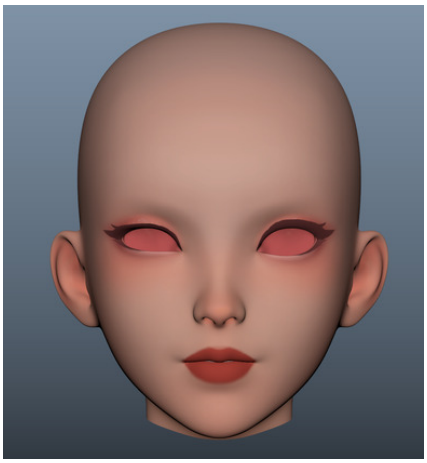
`XR_FACE_EXPRESSION_EYE_LOOK_IN_R_BD`



Description
The movement of the right eyelids consistent with a leftward gaze.

XR_FACE_EXPRESSION_EYE_LOOK_OUT_R_BD	
	**Description** The movement of the right eyelids consistent with a rightward gaze.

XR_FACE_EXPRESSION_EYE_LOOK_UPWARDS_R_BD	
	**Description** The movement of the right eyelids consistent with an upward gaze.

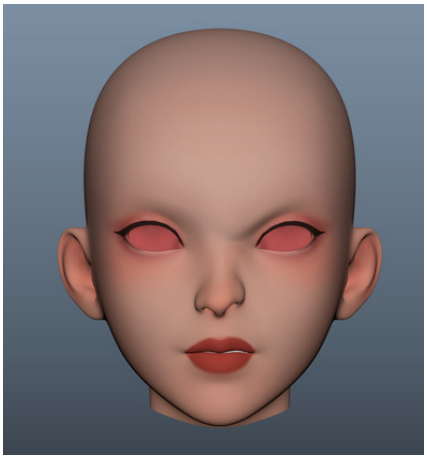
XR_FACE_EXPRESSION_EYE_LOOK_SQUINT_R_BD	
	**Description** The contraction of the face around the right eye.

`XR_FACE_EXPRESSION_EYE_LOOK_WIDE_R_BD`



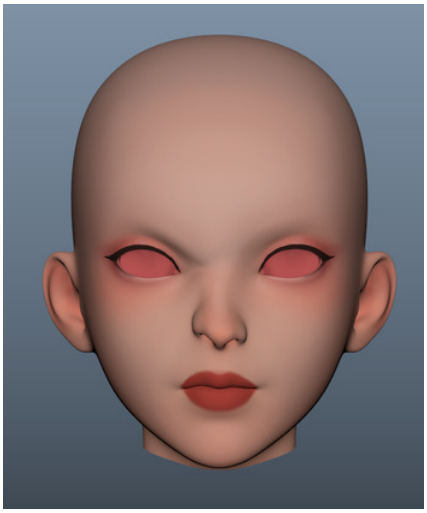
Description
The widening of the eyelids around the right eye.

`XR_FACE_EXPRESSION_NOSE_SNEER_L_BD`



Description
The raising of the left side of the nose around the nostril.

`XR_FACE_EXPRESSION_NOSE_SNEER_R_BD`



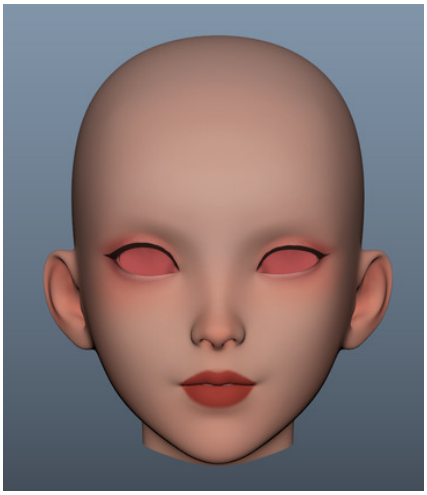
Description
The raising of the right side of the nose around the nostril.

XR_FACE_EXPRESSION_CHEEK_PUFF_BD



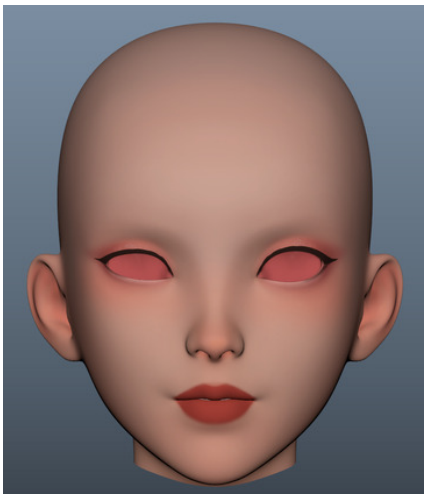
Description
The outward movement of both cheeks.

XR_FACE_EXPRESSION_CHEEK_SQUINT_L_BD



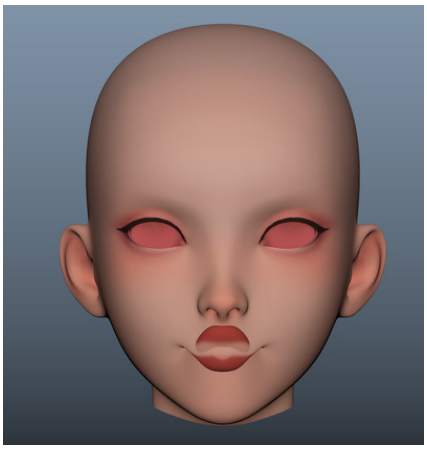
Description
The upward movement of the cheek around and below the left eye, which is usually used to make a smiling expression.

XR_FACE_EXPRESSION_CHEEK_SQUINT_R_BD



Description
The upward movement of the cheek around and below the right eye, which is usually used to make a smiling expression.

XR_FACE_EXPRESSION_MOUTH_CLOSE_BD



Description
The closure of the lips independent of jaw position.

XR_FACE_EXPRESSION_MOUTH_FUNNEL_BD



Description
The contraction of both lips into an open shape.

XR_FACE_EXPRESSION_MOUTH_PUCKER_BD



Description
The contraction and compression of both closed lips.

`XR_FACE_EXPRESSION_MOUTH_L_BD`



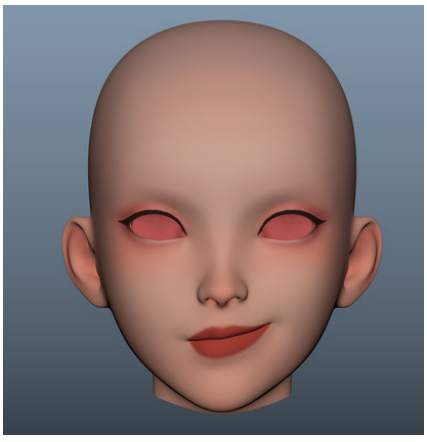
Description
The leftward movement of both lips together.

`XR_FACE_EXPRESSION_MOUTH_R_BD`

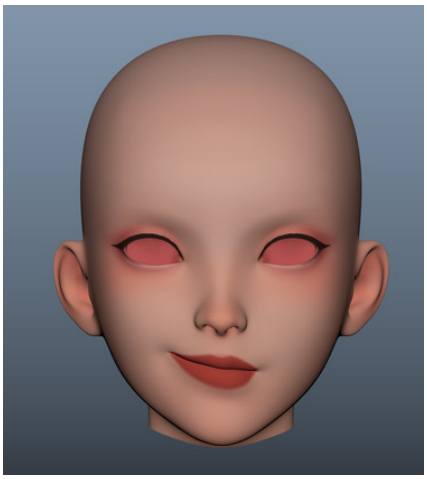


Description
The rightward movement of both lips together.

`XR_FACE_EXPRESSION_MOUTH_SMILE_L_BD`



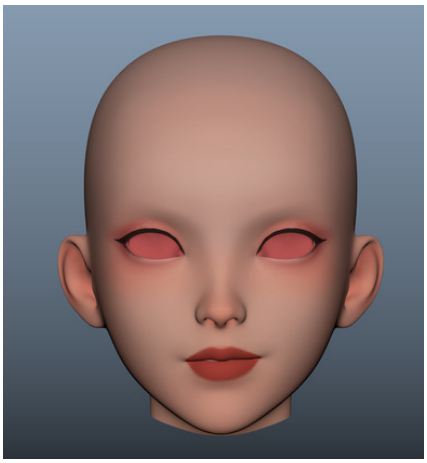
Description
The upward movement of the left corner of the mouth.

XR_FACE_EXPRESSION_MOUTH_SMILE_R_BD	
	**Description** The upward movement of the right corner of the mouth.

XR_FACE_EXPRESSION_MOUTH_FROWN_L_BD	
	**Description** The downward movement of the left corner of the mouth.

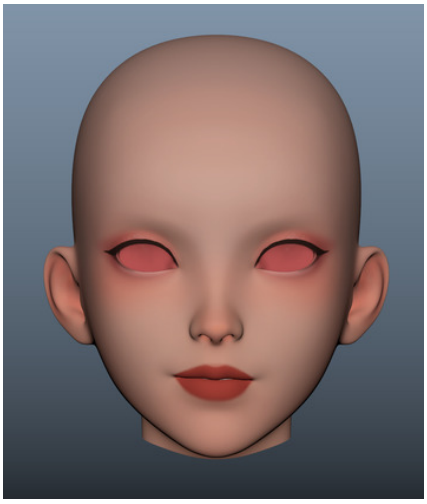
XR_FACE_EXPRESSION_MOUTH_FROWN_R_BD	
	**Description** The downward movement of the right corner of the mouth.

XR_FACE_EXPRESSION_MOUTH_DIMPLE_L_BD



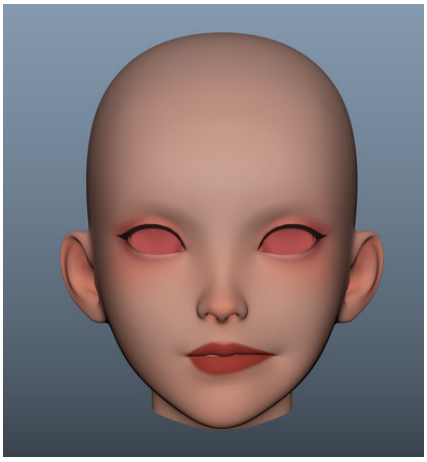
Description
The backward movement of the left corner of the mouth.

XR_FACE_EXPRESSION_MOUTH_DIMPLE_R_BD



Description
The backward movement of the right corner of the mouth.

XR_FACE_EXPRESSION_MOUTH_STRETCH_L_BD



Description
The leftward movement of the left corner of the mouth.

XR_FACE_EXPRESSION_MOUTH_STRETCH_R_BD



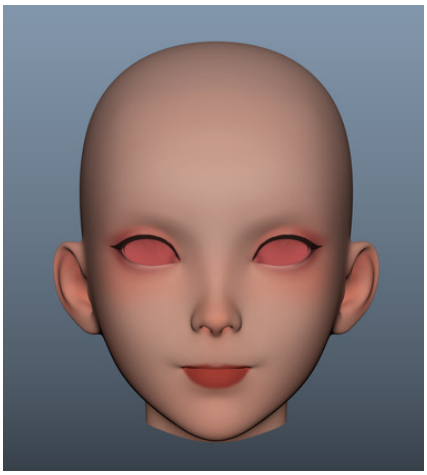
Description
The rightward movement of the right corner of the mouth.

XR_FACE_EXPRESSION_MOUTH_ROLL_LOWER_BD



Description
The movement of the lower lip toward the inside of the mouth.

XR_FACE_EXPRESSION_MOUTH_ROLL_UPPER_BD



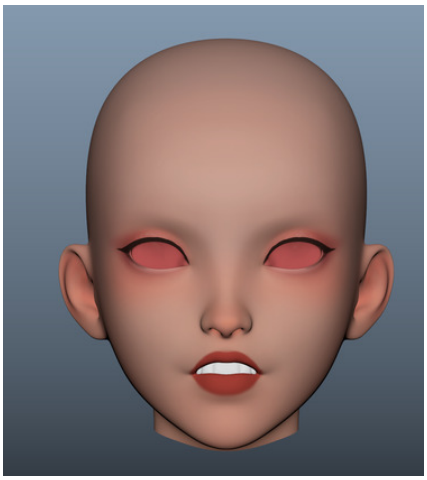
Description
The movement of the upper lip toward the inside of the mouth.

XR_FACE_EXPRESSION_MOUTH_SHRUG_LOWER_BD



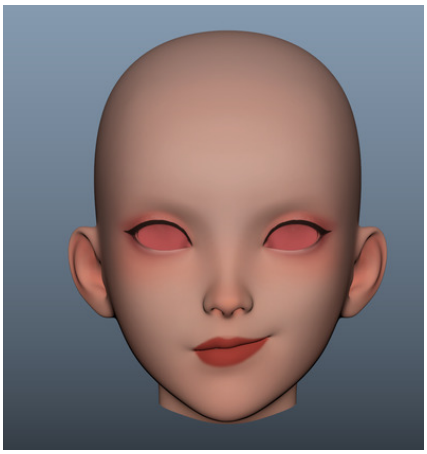
Description
The outward movement of the lower lip.

XR_FACE_EXPRESSION_MOUTH_SHRUG_UPPER_BD



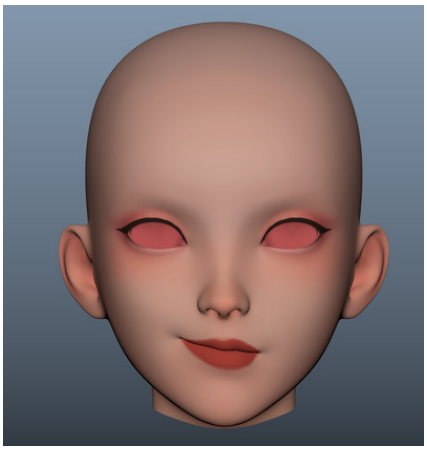
Description
The outward movement of the upper lip.

XR_FACE_EXPRESSION_MOUTH_PRESS_L_BD



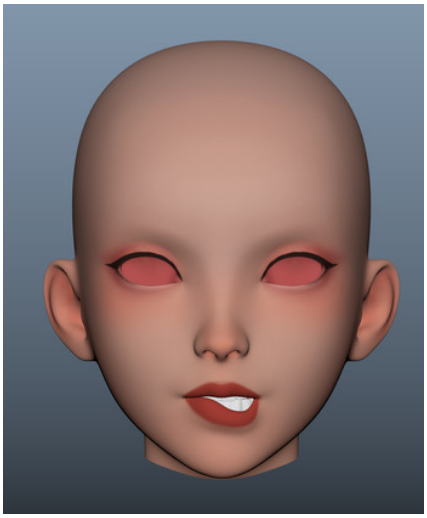
Description
The upward compression of the lower lip on the left side.

`XR_FACE_EXPRESSION_MOUTH_PRESS_R_BD`



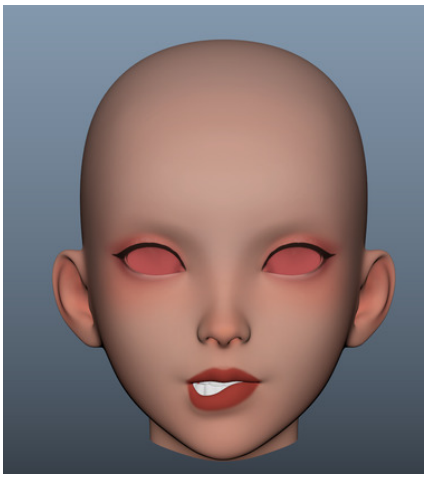
Description
The upward compression of the lower lip on the right side.

`XR_FACE_EXPRESSION_MOUTH_LOWER_DROP_L_BD`



Description
The upward compression of the lower lip on the left side.

`XR_FACE_EXPRESSION_MOUTH_LOWER_DROP_R_BD`



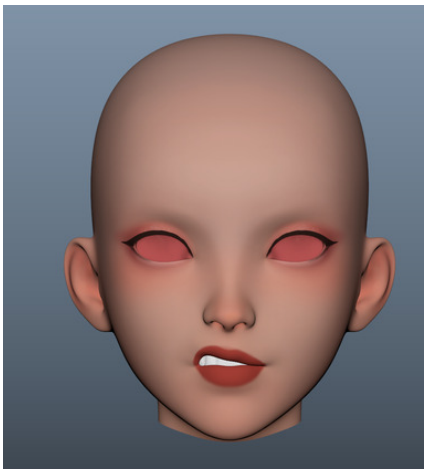
Description
The downward movement of the lower lip on the right side.

XR_FACE_EXPRESSION_MOUTH_UPPER_UPWARDS_L_BD



Description
The upward movement of the upper lip on the left side.

XR_FACE_EXPRESSION_MOUTH_UPPER_UPWARDS_R_BD



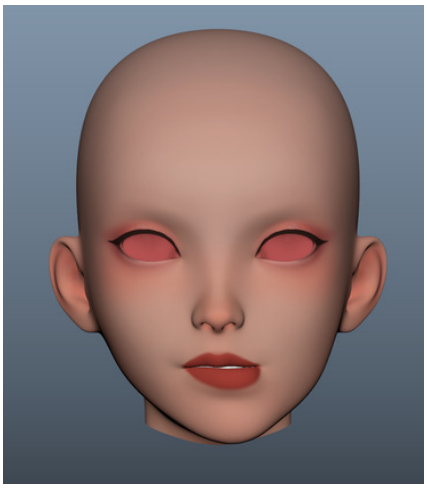
Description
The upward movement of the upper lip on the right side.

XR_FACE_EXPRESSION_JAW_FORWARD_BD



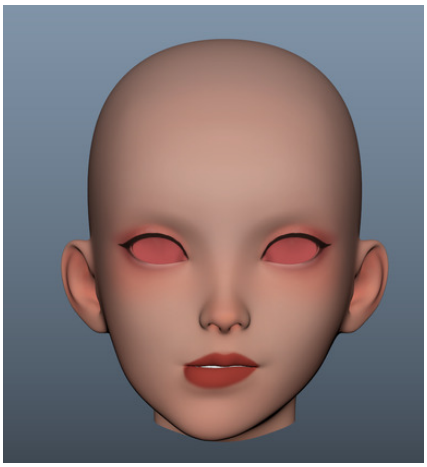
Description
The forward movement of the lower jaw.

XR_FACE_EXPRESSION_JAW_L_BD



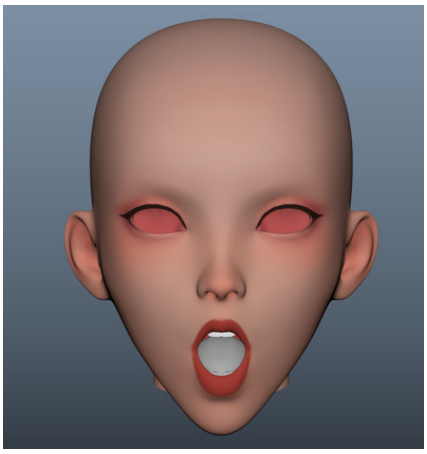
Description
The leftward movement of the lower jaw.

XR_FACE_EXPRESSION_JAW_R_BD

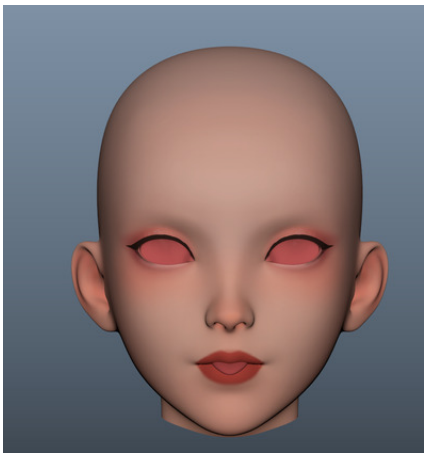


Description
The rightward movement of the lower jaw.

XR_FACE_EXPRESSION_JAW_OPEN_BD

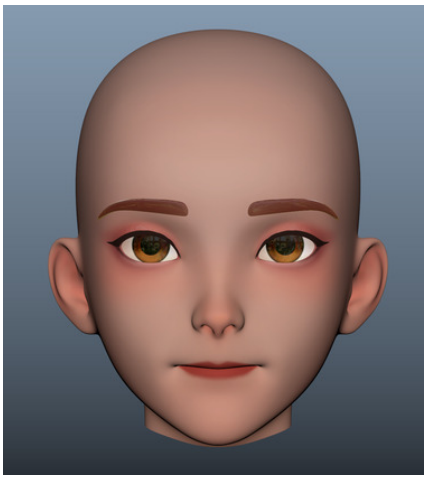


Description
The opening of the lower jaw.

XR_FACE_EXPRESSION_TONGUE_OUT_BD	
	**Description** The extension of the tongue.

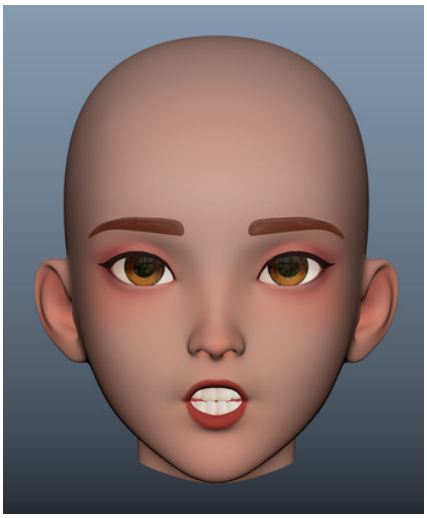
```
// Provided by XR_BD_facial_simulation
typedef enum XrLipExpressionBD {
    XR_LIP_EXPRESSION_PP_BD = 0,
    XR_LIP_EXPRESSION_CH_BD = 1,
    XR_LIP_EXPRESSION_LO_BD = 2,
    XR_LIP_EXPRESSION_O_BD = 3,
    XR_LIP_EXPRESSION_I_BD = 4,
    XR_LIP_EXPRESSION_LU_BD = 5,
    XR_LIP_EXPRESSION_RR_BD = 6,
    XR_LIP_EXPRESSION_XX_BD = 7,
    XR_LIP_EXPRESSION_LAA_BD = 8,
    XR_LIP_EXPRESSION_LI_BD = 9,
    XR_LIP_EXPRESSION_FF_BD = 10,
    XR_LIP_EXPRESSION_U_BD = 11,
    XR_LIP_EXPRESSION_TH_BD = 12,
    XR_LIP_EXPRESSION_LKK_BD = 13,
    XR_LIP_EXPRESSION_SS_BD = 14,
    XR_LIP_EXPRESSION_LE_BD = 15,
    XR_LIP_EXPRESSION_DD_BD = 16,
    XR_LIP_EXPRESSION_E_BD = 17,
    XR_LIP_EXPRESSION_LNN_BD = 18,
    XR_LIP_EXPRESSION_SIL_BD = 19,
    XR_LIP_EXPRESSION_MAX_ENUM_BD = 0x7FFFFFFF
} XrLipExpressionBD;
```

XR_LIP_EXPRESSION_PP_BD



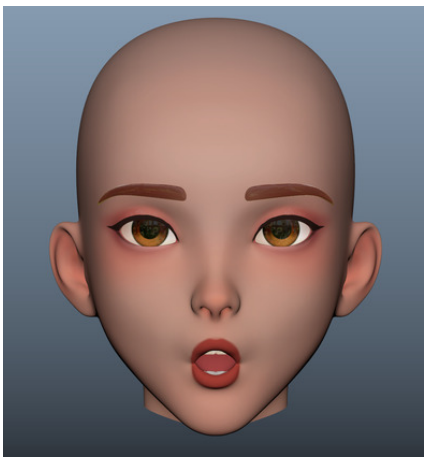
Description
p, b, m.

XR_LIP_EXPRESSION_CH_BD



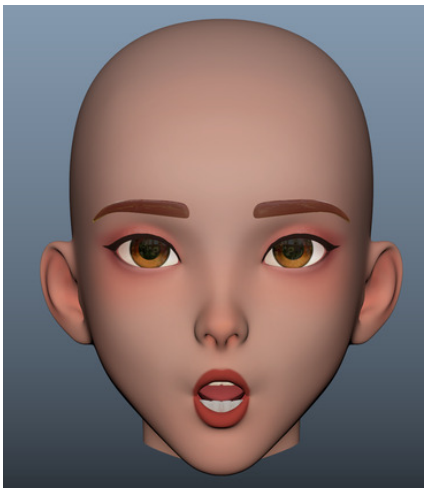
Description
dZ, tS, S.

XR_LIP_EXPRESSION_LO_BD



Description
aw.

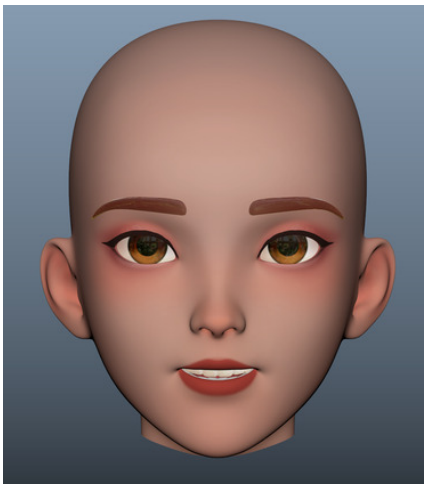
XR_LIP_EXPRESSION_O_BD



Description

ow.

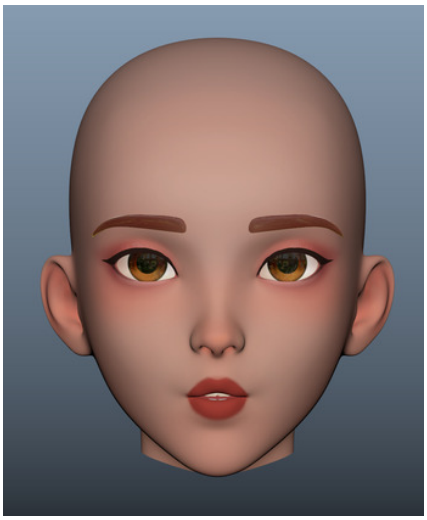
XR_LIP_EXPRESSION_I_BD



Description

ih (stressed).

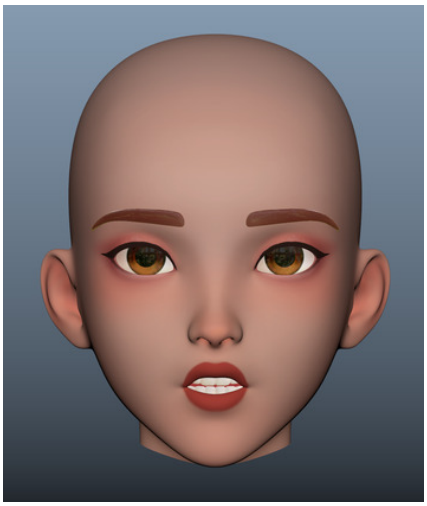
XR_LIP_EXPRESSION_LU_BD



Description

-.

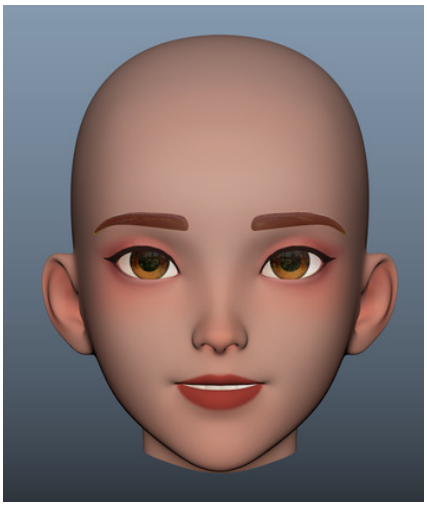
XR_LIP_EXPRESSION_RR_BD



Description

r.

XR_LIP_EXPRESSION_XX_BD



Description

jh.

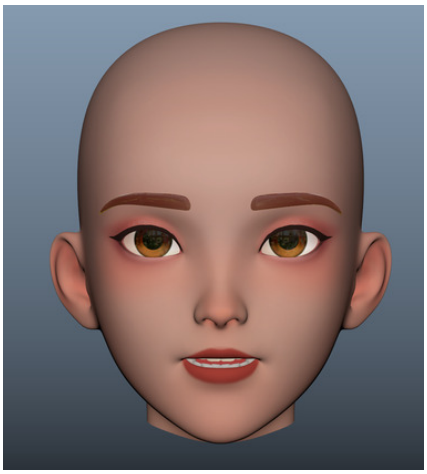
XR_LIP_EXPRESSION_LAA_BD



Description

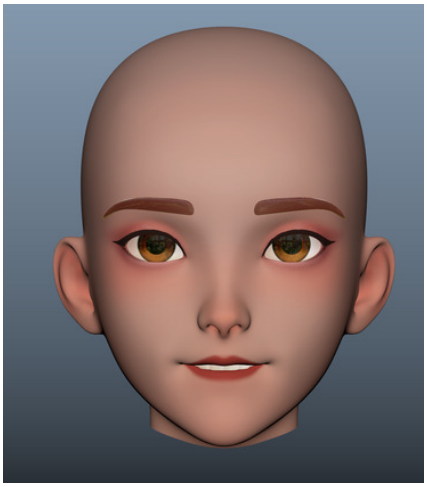
□:.

XR_LIP_EXPRESSION_LI_BD



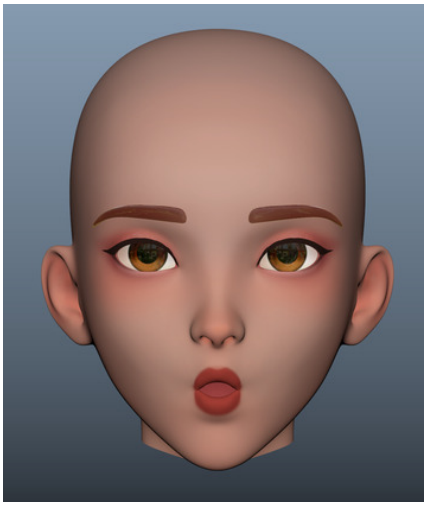
Description
ih (unstressed).

XR_LIP_EXPRESSION_FF_BD



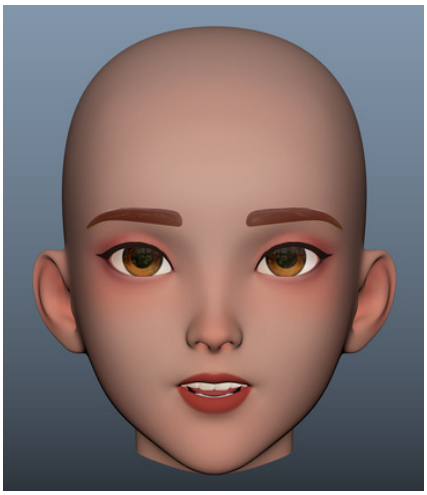
Description
f, v.

XR_LIP_EXPRESSION_U_BD



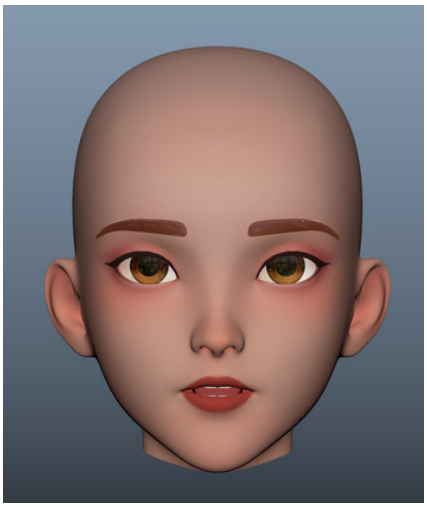
Description
ou.

`XR_LIP_EXPRESSION_TH_BD`



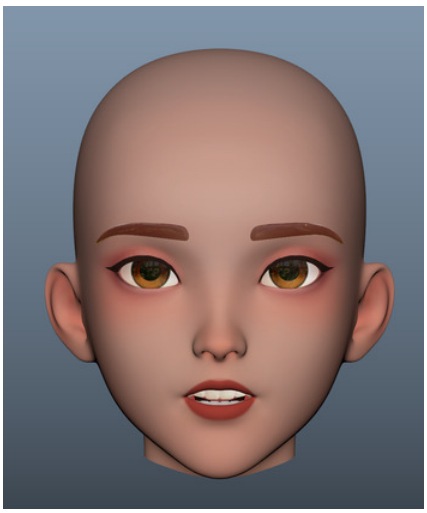
Description
th.

`XR_LIP_EXPRESSION_LKK_BD`



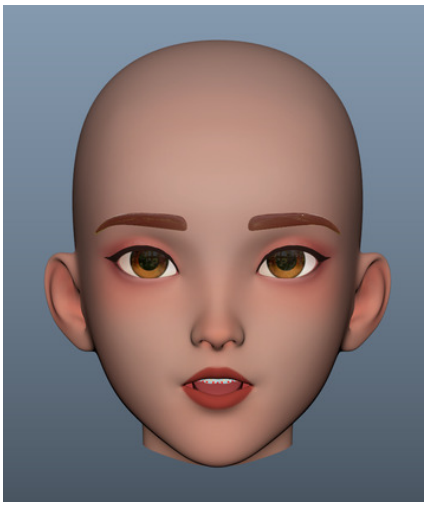
Description
g, h, k.

`XR_LIP_EXPRESSION_SS_BD`



Description
z, s.

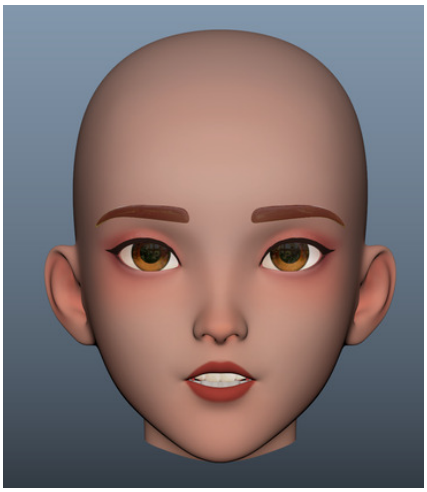
XR_LIP_EXPRESSION_LE_BD



Description

-.

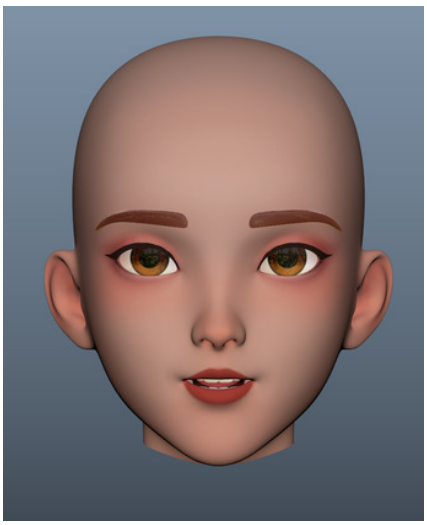
XR_LIP_EXPRESSION_DD_BD



Description

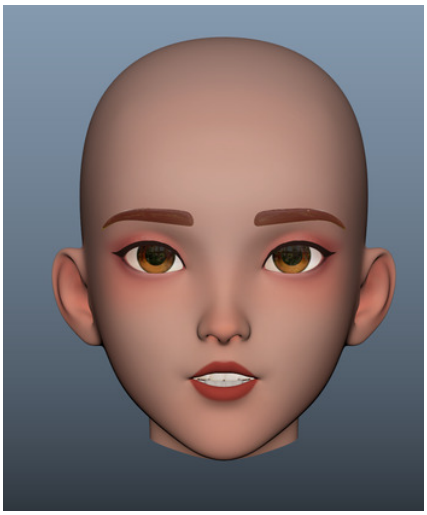
d, t.

XR_LIP_EXPRESSION_E_BD



Description

e.

XR_LIP_EXPRESSION_LNN_BD	
	**Description** n, l.

12.84.10. New Macros

- XR_FACE_EXPRESSION_COUNT_BD
- XR_LIP_EXPRESSION_COUNT_BD

12.84.11. New Object Types

- XrFaceTrackerBD

12.84.12. New Commands

- xrCreateFaceTrackerBD
- xrDestroyFaceTrackerBD
- xrEnumerateFacialSimulationModesBD
- xrGetFacialSimulationDataBD
- xrGetFacialSimulationModeBD
- xrSetFacialSimulationModeBD

12.84.13. New Structures

- XrFaceTrackerCreateInfoBD
- XrFacialSimulationDataBD
- XrFacialSimulationDataGetInfoBD
- Extending XrFacialSimulationDataBD:
  - XrLipExpressionDataBD

- Extending XrSystemProperties:
  - XrSystemFacialSimulationPropertiesBD

12.84.14. New Enums

- XrFaceExpressionBD
- XrFacialSimulationModeBD
- XrLipExpressionBD

12.84.15. New Enum Constants

- XR_BD_FACIAL_SIMULATION_EXTENSION_NAME
- XR_BD_facial_simulation_SPEC_VERSION
- Extending XrObjectType:
  - XR_OBJECT_TYPE_FACE_TRACKER_BD
- Extending XrStructureType:
  - XR_TYPE_FACE_TRACKER_CREATE_INFO_BD
  - XR_TYPE_FACIAL_SIMULATION_DATA_BD
  - XR_TYPE_FACIAL_SIMULATION_DATA_GET_INFO_BD
  - XR_TYPE_LIP_EXPRESSION_DATA_BD
  - XR_TYPE_SYSTEM_FACIAL_SIMULATION_PROPERTIES_BD

12.84.16. Version History

- Revision 1, 2024-10-17 (Shanliang Xu)
  - Initial extension description

12.85. XR_BD_future_progress

Name String

XR_BD_future_progress

Extension Type

Instance extension

Registered Extension Number

395

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0
and
XR_EXT_future

Last Modified Date

2025-02-25

IP Status

No known IP claims.

Contributors

Zhipeng Liu, Bytedance
Ya Huang, Bytedance

12.85.1. Overview

Asynchronous operations **may** take a long time. If some progress hints can be provided before the asynchronous operation is completed, it will significantly improve the user experience. This extension allows an application to get the rough progress percentage of an asynchronous operation before it is completed. This extension requires XR_EXT_future to be enabled.

12.85.2. Get Future Progress

The XrFuturePollResultProgressBD structure is defined as:

```
// Provided by XR_BD_future_progress
typedef struct XrFuturePollResultProgressBD {
    XrStructureType    type;
    void*              next;
    XrBool32           isSupported;
    uint32_t           progressPercentage;
} XrFuturePollResultProgressBD;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `isSupported` is a boolean value indicating whether reporting progress is supported for this future.
- `progressPercentage` is the progress percentage of the future.

As defined in `XR_EXT_future`, an `XrFutureEXT` is returned by a successful call to an asynchronous function.

Applications **can** use xrPollFutureEXT to check the current state of a future, typically while waiting for the async operation to complete and the future to become "ready" to complete. An XrFuturePollResultEXT structure is used to return the result of xrPollFutureEXT.

With this extension, the application **can** chain an XrFuturePollResultProgressBD to XrFuturePollResultEXT::`next` to get a rough progress percentage of the asynchronous operation.

If the runtime does not support reporting progress for a specific future, it **must** set `isSupported` to false. And in this case, the application **should** ignore the value of `progressPercentage`. If the runtime supports reporting progress for the specific future, it **must** set `isSupported` to true. The `progressPercentage` is only valid when the future is in either the `XR_FUTURE_STATE_PENDING_EXT` or `XR_FUTURE_STATE_READY_EXT` state.

The runtime **must** not set `progressPercentage` to a value less than 0 or greater than 100. The runtime **must** set `progressPercentage` to 100 if the future is in the state `XR_FUTURE_STATE_READY_EXT`.

Valid Usage (Implicit)

- The `XR_BD_future_progress` extension **must** be enabled prior to using XrFuturePollResultProgressBD
- `type` **must** be `XR_TYPE_FUTURE_POLL_RESULT_PROGRESS_BD`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

12.85.3. New Structures

- Extending XrFuturePollResultEXT:
  - XrFuturePollResultProgressBD

12.85.4. New Enum Constants

- `XR_BD_FUTURE_PROGRESS_EXTENSION_NAME`
- `XR_BD_future_progress_SPEC_VERSION`
- Extending XrStructureType:
  - `XR_TYPE_FUTURE_POLL_RESULT_PROGRESS_BD`

12.85.5. Issues

12.85.6. Version History

- Revision 1, 2025-02-25 (Zhipeng Liu)
  - Initial extension description

12.86. XR_BD_spatial_anchor

Name String

`XR_BD_spatial_anchor`

Extension Type

Instance extension

Registered Extension Number

391

Revision

2

Ratification Status

Not ratified

Extension and Version Dependencies

`XR_BD_spatial_sensing`

Last Modified Date

2025-06-10

IP Status

No known IP claims.

Contributors

Zhipeng Liu, ByteDance
Zijian Wang, ByteDance

Zhao Li, ByteDance
Jun Yan, ByteDance
Jimmy Alamparambil, ByteDance

12.86.1. Overview

This extension allows an application to create and destroy a spatial anchor, representing an arbitrary point of interest in the user's physical environment. The position and orientation of each spatial anchor will be tracked by the runtime over time. The runtime adjusts the position and orientation of the spatial anchor over time as needed, to ensure that it maintains its original mapping to the real world.

This extension also allows an application to persist or unpersist spatial anchors. Persisting a spatial anchor means saving it to persistent storage, and the runtime **can** reload and locate it in a subsequent session. Unpersisting an anchor means erasing it from persistent storage, which causes it to not appear in snapshots or queried data in subsequent sessions, but it still appears in snapshots and queried data in the same session. Both the persisting and unpersisting operations **must** not affect the existing spatial anchor handles in the same session. This allows spatial anchors to be shared and localized across application sessions on the device for the same application. This extension requires `XR_BD_spatial_sensing` to be enabled.

12.86.2. Permission

A runtime on an AOSP-based platform **must** verify that applications have the `com.picovr.permission.SPATIAL_DATA` permission both listed in their manifest and granted to use spatial anchor. Without it, runtime **must** set XrFutureCompletionEXT::`futureResult` to `XR_ERROR_PERMISSION_INSUFFICIENT` when the xrStartSenseDataProviderCompleteBD is called.

This is an auto-requested permission: if it is listed in the manifest but not yet granted or denied, the runtime **must** prompt the user to grant or deny the permission when xrCreateSenseDataProviderBD is called with a provider type that requires it.

This permission is also used by `XR_BD_spatial_mesh`.

This permission is also used by `XR_BD_spatial_scene`.

This permission is also used by `XR_BD_spatial_plane`.

12.86.3. Inspect System Capability

The XrSystemSpatialAnchorPropertiesBD structure is defined as:

```
// Provided by XR_BD_spatial_anchor
typedef struct XrSystemSpatialAnchorPropertiesBD {
    XrStructureType    type;
    void*              next;
    XrBool32           supportsSpatialAnchor;
} XrSystemSpatialAnchorPropertiesBD;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. For more details about the structure chain, see the structure being extended (XrSystemProperties).
- `supportsSpatialAnchor` is an `XrBool32`, indicating if the current system is capable of creating and storing arbitrary spatial anchors.

An application **can** inspect whether the system is capable of application created spatial anchor by chaining an XrSystemSpatialAnchorPropertiesBD structure to the XrSystemProperties::`next` chain when calling xrGetSystemProperties.

If a runtime returns `XR_FALSE` for `supportsSpatialAnchor`, the system does not support creating arbitrary spatial anchors, and **must** return `XR_ERROR_FEATURE_UNSUPPORTED` from xrCreateSpatialAnchorAsyncBD, as well as from xrCreateSenseDataProviderBD when passing the XrSenseDataProviderTypeBD value `XR_SENSE_DATA_PROVIDER_TYPE_ANCHOR_BD`.

The application **should** avoid using spatial anchor functionality when `supportsSpatialAnchor` is `XR_FALSE`.

If XrSystemSpatialSensingPropertiesBD::`supportsSpatialSensing` is `XR_FALSE`, then `supportsSpatialAnchor` **must** also be `XR_FALSE`.

If a runtime returns `XR_TRUE` for `supportsSpatialAnchor`, the system supports creating and storing arbitrary spatial anchors. This implies that XrSystemSpatialSensingPropertiesBD ::`supportsSpatialSensing` **must** also be `XR_TRUE`.

Note that `supportsSpatialAnchor` **may** be `XR_TRUE` even if running on an Android-based platform and the application does not have the required com.picovr.permission.SPATIAL_DATA permission both declared in the manifest and granted at runtime. Evaluation of permissions takes place later, in the asynchronous operation started by xrStartSenseDataProviderAsyncBD.

Valid Usage (Implicit)

- The XR_BD_spatial_anchor extension **must** be enabled prior to using XrSystemSpatialAnchorPropertiesBD
- type **must** be XR_TYPE_SYSTEM_SPATIAL_ANCHOR_PROPERTIES_BD
- next **must** be NULL or a valid pointer to the next structure in a structure chain

12.86.4. Create Spatial Anchor Provider

An application creates an XrSenseDataProviderBD handle representing a spatial anchor provider by calling xrCreateSenseDataProviderBD and setting XrSenseDataProviderCreateInfoBD::providerType equal to the XrSenseDataProviderTypeBD value XR_SENSE_DATA_PROVIDER_TYPE_ANCHOR_BD.

An application uses such a provider to create or query spatial anchors.

This provider type does not define any configuration and does not require a chained structure.

12.86.5. Start Spatial Anchor Provider

Applications start the spatial anchor data provider by calling xrStartSenseDataProviderAsyncBD after it is successfully created. To check the data provider state, call xrGetSenseDataProviderStateBD.

Subsequent application operations using this provider handle **must** not be performed unless the data provider state is XR_SENSE_DATA_PROVIDER_STATE_RUNNING_BD. If the data provider state is not XR_SENSE_DATA_PROVIDER_STATE_RUNNING_BD and the application needs to use the provider, the application **must** take appropriate action and try to call xrStartSenseDataProviderAsyncBD again before using the handle.

Detailed definitions and usage details are described in XR_BD_spatial_sensing.

Upon start, the provider immediately begins trying to load and locate any previously persisted spatial anchors. See Query Spatial Anchor and xrPersistSpatialAnchorAsyncBD for information on how to work with previously-persisted anchors.

12.86.6. Create Spatial Anchor

The xrCreateSpatialAnchorAsyncBD function is defined as:

```
// Provided by XR_BD_spatial_anchor
XrResult xrCreateSpatialAnchorAsyncBD(
    XrSenseDataProviderBD                       provider,
    const XrSpatialAnchorCreateInfoBD*          info,
    XrFutureEXT*                                future);
```

Parameter Descriptions

- `provider` is the previously created XrSenseDataProviderBD for spatial anchor.
- `info` is a pointer to the XrSpatialAnchorCreateInfoBD .
- `future` is a pointer to the `XrFutureEXT`.

An application creates a spatial anchor by calling this function.

This function starts an asynchronous operation and creates a corresponding `XrFutureEXT`, usable with xrPollFutureEXT and related functions. The return value of this function only indicates whether the parameters were acceptable to schedule the asynchronous operation. The corresponding completion function is xrCreateSpatialAnchorCompleteBD, usable when a future from this function is in the READY state, with outputs populated by that function in the completion structure XrSpatialAnchorCreateCompletionBD.

If the `provider` was not created with the XrSenseDataProviderTypeBD value `XR_SENSE_DATA_PROVIDER_TYPE_ANCHOR_BD`, the function xrCreateSpatialAnchorAsyncBD **must** return `XR_ERROR_VALIDATION_FAILURE`.

Valid Usage (Implicit)

- The `XR_BD_spatial_anchor` extension **must** be enabled prior to calling xrCreateSpatialAnchorAsyncBD
- `provider` **must** be a valid XrSenseDataProviderBD handle
- `info` **must** be a pointer to a valid XrSpatialAnchorCreateInfoBD structure
- `future` **must** be a pointer to an `XrFutureEXT` value

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_CALL_ORDER_INVALID
- XR_ERROR_FEATURE_UNSUPPORTED
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_POSE_INVALID
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_SPATIAL_SENSING_SERVICE_UNAVAILABLE_BD
- XR_ERROR_TIME_INVALID
- XR_ERROR_VALIDATION_FAILURE

The XrSpatialAnchorCreateInfoBD structure is defined as:

```
// Provided by XR_BD_spatial_anchor
typedef struct XrSpatialAnchorCreateInfoBD {
    XrStructureType    type;
    const void*        next;
    XrSpace            space;
    XrPosef            pose;
    XrTime             time;
} XrSpatialAnchorCreateInfoBD;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `space` is a handle to the XrSpace in which `pose` is specified.
- `pose` is the XrPosef within `space` at `time` that specifies the point in the real world to create the new spatial anchor.
- `time` is the `XrTime` at which `pose` will be evaluated within `space`.

Valid Usage (Implicit)

- The `XR_BD_spatial_anchor` extension **must** be enabled prior to using XrSpatialAnchorCreateInfoBD
- `type` **must** be `XR_TYPE_SPATIAL_ANCHOR_CREATE_INFO_BD`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `space` **must** be a valid XrSpace handle

The xrCreateSpatialAnchorCompleteBD function is defined as:

```
// Provided by XR_BD_spatial_anchor
XrResult xrCreateSpatialAnchorCompleteBD(
    XrSenseDataProviderBD                       provider,
    XrFutureEXT                                 future,
    XrSpatialAnchorCreateCompletionBD*          completion);
```

Parameter Descriptions

- `provider` is the XrSenseDataProviderBD.
- `future` is the `XrFutureEXT`.
- `completion` is a pointer to the XrSpatialAnchorCreateCompletionBD.

The application obtains the spatial anchor create result using xrCreateSpatialAnchorCompleteBD, after the future is ready.

This is the completion function corresponding to xrCreateSpatialAnchorAsyncBD. It completes the asynchronous operation and returns the results. Do not call until the `future` is READY.

Valid Usage (Implicit)

- The `XR_BD_spatial_anchor` extension **must** be enabled prior to calling xrCreateSpatialAnchorCompleteBD
- `provider` **must** be a valid XrSenseDataProviderBD handle
- `completion` **must** be a pointer to an XrSpatialAnchorCreateCompletionBD structure

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_FEATURE_UNSUPPORTED`
- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_FUTURE_INVALID_EXT`
- `XR_ERROR_FUTURE_PENDING_EXT`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_LIMIT_REACHED`
- `XR_ERROR_OUT_OF_MEMORY`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_SPATIAL_SENSING_SERVICE_UNAVAILABLE_BD`
- `XR_ERROR_VALIDATION_FAILURE`

The XrSpatialAnchorCreateCompletionBD structure is defined as:

```
// Provided by XR_BD_spatial_anchor
typedef struct XrSpatialAnchorCreateCompletionBD {
    XrStructureType    type;
    void*              next;
    XrResult           futureResult;
    XrUuidEXT          uuid;
    XrAnchorBD         anchor;
} XrSpatialAnchorCreateCompletionBD;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `futureResult` is the result of the future.
- `uuid` is the XrUuidEXT of the created anchor.
- `anchor` is the handle of the created spatial anchor.

This is the completion data structure associated with the asynchronous operation started by xrCreateSpatialAnchorAsyncBD and completed by xrCreateSpatialAnchorCompleteBD.

It is populated by a valid call to xrCreateSpatialAnchorCompleteBD on a corresponding, READY future.

Future Return Codes

`futureResult` values:

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_OUT_OF_MEMORY`
- `XR_ERROR_LIMIT_REACHED`

Valid Usage (Implicit)

- The XR_BD_spatial_anchor extension **must** be enabled prior to using XrSpatialAnchorCreateCompletionBD
- type **must** be XR_TYPE_SPATIAL_ANCHOR_CREATE_COMPLETION_BD
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- futureResult **must** be a valid XrResult value
- anchor **must** be a valid XrAnchorBD handle

12.86.7. Locate Spatial Anchor

Locating an anchor relative to a base space is performed similarly to locating other spatial objects: through use of an XrSpace handle and functions like xrLocateSpace and xrLocateSpaces. To locate an anchor in a base space, first create an XrSpace handle for that anchor using xrCreateAnchorSpaceBD. The function and related behaviors are defined in XR_BD_spatial_sensing.

12.86.8. Persist Spatial Anchor

The xrPersistSpatialAnchorAsyncBD function is defined as:

```
// Provided by XR_BD_spatial_anchor
XrResult xrPersistSpatialAnchorAsyncBD(
    XrSenseDataProviderBD                       provider,
    const XrSpatialAnchorPersistInfoBD*         info,
    XrFutureEXT*                                future);
```

Parameter Descriptions

- provider is the previously created XrSenseDataProviderBD for spatial anchor.
- info is a pointer to the XrSpatialAnchorPersistInfoBD.
- future is a pointer to the XrFutureEXT.

To persist a spatial anchor, call xrPersistSpatialAnchorAsyncBD.

Persisting a spatial anchor means storing it to a persistent storage, allowing it to be reloaded and located in subsequent sessions by the same application.

This function starts an asynchronous operation and creates a corresponding XrFutureEXT, usable with xrPollFutureEXT and related functions. The return value of this function only indicates whether the parameters were acceptable to schedule the asynchronous operation. The corresponding completion

function is xrPersistSpatialAnchorCompleteBD, usable when a future from this function is in the READY state, with outputs populated by that function in the completion structure XrFutureCompletionEXT.

If a previously persisted spatial anchor is successfully loaded and located by the runtime after starting a spatial anchor provider, the runtime **must** queue an XrEventDataSenseDataUpdatedBD event.

In subsequent sessions, to retrieve the previously persisted spatial anchor or anchors, after receiving the XrEventDataSenseDataUpdatedBD event, use the normal practice of creating a snapshot and getting the queried sense data (call xrQuerySenseDataAsyncBD, xrQuerySenseDataCompleteBD, xrGetQueriedSenseDataBD per the specification). Tracking data is accessible by calling xrCreateSpatialEntityAnchorBD and xrCreateAnchorSpaceBD as specified. All of these functions are defined in `XR_BD_spatial_sensing`.

If the XrAnchorBD is already persisted, calling this function does not change the persistence status. In this case, the runtime **must** still set the return value of xrPersistSpatialAnchorAsyncBD to `XR_SUCCESS`. The runtime **must** set the corresponding future result returned by xrPersistSpatialAnchorCompleteBD to `XR_SUCCESS`.

Valid Usage (Implicit)

- The `XR_BD_spatial_anchor` extension **must** be enabled prior to calling xrPersistSpatialAnchorAsyncBD
- `provider` **must** be a valid XrSenseDataProviderBD handle
- `info` **must** be a pointer to a valid XrSpatialAnchorPersistInfoBD structure
- `future` **must** be a pointer to an `XrFutureEXT` value

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FEATURE_UNSUPPORTED
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_SPATIAL_SENSING_SERVICE_UNAVAILABLE_BD
- XR_ERROR_VALIDATION_FAILURE

The XrSpatialAnchorPersistInfoBD structure is defined as:

```
// Provided by XR_BD_spatial_anchor
typedef struct XrSpatialAnchorPersistInfoBD {
    XrStructureType            type;
    const void*                next;
    XrPersistenceLocationBD    location;
    XrAnchorBD                 anchor;
} XrSpatialAnchorPersistInfoBD;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- location is the XrPersistenceLocationBD to persist the spatial anchor.
- anchor is the XrAnchorBD handle of the spatial anchor.

Valid Usage (Implicit)

- The XR_BD_spatial_anchor extension **must** be enabled prior to using XrSpatialAnchorPersistInfoBD
- type **must** be XR_TYPE_SPATIAL_ANCHOR_PERSIST_INFO_BD
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- location **must** be a valid XrPersistenceLocationBD value
- anchor **must** be a valid XrAnchorBD handle

The XrPersistenceLocationBD enumeration identifies the different persistence locations.

```
// Provided by XR_BD_spatial_anchor
typedef enum XrPersistenceLocationBD {
    XR_PERSISTENCE_LOCATION_LOCAL_BD = 0,
    XR_PERSISTENCE_LOCATION_MAX_ENUM_BD = 0x7FFFFFFF
} XrPersistenceLocationBD;
```

Enum	Description
XR_PERSISTENCE_LOCATION_LOCAL_BD	The persistence storage location is local to the device.

The xrPersistSpatialAnchorCompleteBD function is defined as:

```
// Provided by XR_BD_spatial_anchor
XrResult xrPersistSpatialAnchorCompleteBD(
    XrSenseDataProviderBD                       provider,
    XrFutureEXT                                 future,
    XrFutureCompletionEXT*                      completion);
```

Parameter Descriptions

- provider is the XrSenseDataProviderBD.
- future is the XrFutureEXT.
- completion is a pointer to the XrFutureCompletionEXT to populate.

To complete persisting an anchor and retrieve the result, call xrPersistSpatialAnchorCompleteBD.

This is the completion function corresponding to xrPersistSpatialAnchorAsyncBD. It completes the

asynchronous operation and returns the results. Do not call until the `future` is READY.

Future Return Codes

XrFutureCompletionEXT::`futureResult` values:

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_OUT_OF_MEMORY`
- `XR_ERROR_LIMIT_REACHED`

Valid Usage (Implicit)

- The `XR_BD_spatial_anchor` extension **must** be enabled prior to calling xrPersistSpatialAnchorCompleteBD
- `provider` **must** be a valid XrSenseDataProviderBD handle
- `completion` **must** be a pointer to an XrFutureCompletionEXT structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FEATURE_UNSUPPORTED
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_FUTURE_INVALID_EXT
- XR_ERROR_FUTURE_PENDING_EXT
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_SPATIAL_SENSING_SERVICE_UNAVAILABLE_BD
- XR_ERROR_VALIDATION_FAILURE

12.86.9. Unpersist Spatial Anchor

The xrUnpersistSpatialAnchorAsyncBD function is defined as:

```
// Provided by XR_BD_spatial_anchor
XrResult xrUnpersistSpatialAnchorAsyncBD(
    XrSenseDataProviderBD                       provider,
    const XrSpatialAnchorUnpersistInfoBD*       info,
    XrFutureEXT*                                future);
```

Parameter Descriptions

- provider is the previously persisted XrSenseDataProviderBD for spatial anchor.
- info is a pointer to the XrSpatialAnchorUnpersistInfoBD.
- future is a pointer to the XrFutureEXT.

To unpersist a spatial anchor, call xrUnpersistSpatialAnchorAsyncBD.

Unpersisting an anchor means erasing it from persistent storage, which causes it to not appear in

snapshots or queried data in subsequent sessions. However, it **must** still appear in snapshots and queried data in the same session.

This function starts an asynchronous operation and creates a corresponding XrFutureEXT, usable with xrPollFutureEXT and related functions. The return value of this function only indicates whether the parameters were acceptable to schedule the asynchronous operation. The corresponding completion function is xrUnpersistSpatialAnchorCompleteBD, usable when a future from this function is in the READY state, with outputs populated by that function in the completion structure XrFutureCompletionEXT.

Valid Usage (Implicit)

- The XR_BD_spatial_anchor extension **must** be enabled prior to calling xrUnpersistSpatialAnchorAsyncBD
- `provider` **must** be a valid XrSenseDataProviderBD handle
- `info` **must** be a pointer to a valid XrSpatialAnchorUnpersistInfoBD structure
- `future` **must** be a pointer to an XrFutureEXT value

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FEATURE_UNSUPPORTED
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_SPATIAL_SENSING_SERVICE_UNAVAILABLE_BD
- XR_ERROR_VALIDATION_FAILURE

The XrSpatialAnchorUnpersistInfoBD structure is defined as:

```
// Provided by XR_BD_spatial_anchor
typedef struct XrSpatialAnchorUnpersistInfoBD {
    XrStructureType            type;
    const void*                next;
    XrPersistenceLocationBD    location;
    XrAnchorBD                 anchor;
} XrSpatialAnchorUnpersistInfoBD;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `location` is the XrPersistenceLocationBD to unpersist the spatial anchor.
- `anchor` is the handle:XrAnchorBD handle of the spatial anchor.

Valid Usage (Implicit)

- The `XR_BD_spatial_anchor` extension **must** be enabled prior to using XrSpatialAnchorUnpersistInfoBD
- `type` **must** be `XR_TYPE_SPATIAL_ANCHOR_UNPERSIST_INFO_BD`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `location` **must** be a valid XrPersistenceLocationBD value
- `anchor` **must** be a valid XrAnchorBD handle

The xrUnpersistSpatialAnchorCompleteBD function is defined as:

```
// Provided by XR_BD_spatial_anchor
XrResult xrUnpersistSpatialAnchorCompleteBD(
    XrSenseDataProviderBD                   provider,
    XrFutureEXT                             future,
    XrFutureCompletionEXT*                  completion);
```

Parameter Descriptions

- `provider` is the XrSenseDataProviderBD.
- `future` is the `XrFutureEXT`.
- `completion` is a pointer to the XrFutureCompletionEXT.

The application obtains the spatial anchor unpersist result using xrUnpersistSpatialAnchorCompleteBD.

This is the completion function corresponding to xrUnpersistSpatialAnchorAsyncBD. It completes the asynchronous operation and returns the results. Do not call until the `future` is READY.

Future Return Codes

XrFutureCompletionEXT::`futureResult` values:

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_SPATIAL_ANCHOR_NOT_FOUND_BD`

The runtime **must** set XrFutureCompletionEXT::`futureResult` to `XR_ERROR_SPATIAL_ANCHOR_NOT_FOUND_BD` if the spatial anchor is not found in XrSpatialAnchorUnpersistInfoBD::`location`.

Valid Usage (Implicit)

- The `XR_BD_spatial_anchor` extension **must** be enabled prior to calling xrUnpersistSpatialAnchorCompleteBD
- `provider` **must** be a valid XrSenseDataProviderBD handle
- `completion` **must** be a pointer to an XrFutureCompletionEXT structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FEATURE_UNSUPPORTED
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_FUTURE_INVALID_EXT
- XR_ERROR_FUTURE_PENDING_EXT
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_SPATIAL_SENSING_SERVICE_UNAVAILABLE_BD
- XR_ERROR_VALIDATION_FAILURE

12.86.10. Query Spatial Anchor

If spatial anchors were previously persisted, they **may** be detected and located again by the runtime when the user is in the same physical space where they are persisted. For details on operations made available by persisting an anchor, see xrPersistSpatialAnchorAsyncBD.

12.86.11. New Commands

- xrCreateSpatialAnchorAsyncBD
- xrCreateSpatialAnchorCompleteBD
- xrPersistSpatialAnchorAsyncBD
- xrPersistSpatialAnchorCompleteBD
- xrUnpersistSpatialAnchorAsyncBD
- xrUnpersistSpatialAnchorCompleteBD

12.86.12. New Structures

- XrSpatialAnchorCreateCompletionBD
- XrSpatialAnchorCreateInfoBD

- XrSpatialAnchorPersistInfoBD
- XrSpatialAnchorUnpersistInfoBD
- Extending XrSystemProperties:
  - XrSystemSpatialAnchorPropertiesBD

12.86.13. New Enums

- XrPersistenceLocationBD

12.86.14. New Enum Constants

- XR_BD_SPATIAL_ANCHOR_EXTENSION_NAME
- XR_BD_spatial_anchor_SPEC_VERSION
- Extending XrResult:
  - XR_ERROR_SPATIAL_ANCHOR_NOT_FOUND_BD
- Extending XrSenseDataProviderTypeBD:
  - XR_SENSE_DATA_PROVIDER_TYPE_ANCHOR_BD
- Extending XrStructureType:
  - XR_TYPE_SPATIAL_ANCHOR_CREATE_COMPLETION_BD
  - XR_TYPE_SPATIAL_ANCHOR_CREATE_INFO_BD
  - XR_TYPE_SPATIAL_ANCHOR_PERSIST_INFO_BD
  - XR_TYPE_SPATIAL_ANCHOR_UNPERSIST_INFO_BD
  - XR_TYPE_SYSTEM_SPATIAL_ANCHOR_PROPERTIES_BD

12.86.15. Issues

12.86.16. Version History

- Revision 1, 2024-05-06 (Zhipeng Liu)
  - Initial extension description
- Revision 2, 2025-06-10 (Zhipeng Liu)
  - Move anchor to the last parameter of XrSpatialAnchorCreateCompletionBD.

12.87. XR_BD_spatial_anchor_sharing

Name String

XR_BD_spatial_anchor_sharing

Extension Type

Instance extension

Registered Extension Number

392

Revision

2

Ratification Status

Not ratified

Extension and Version Dependencies

`XR_BD_spatial_anchor`

Last Modified Date

2025-06-02

IP Status

No known IP claims.

Contributors

Zhipeng Liu, ByteDance
Ya Huang, ByteDance
Xiangxin Liu, ByteDance
Jun Yan, ByteDance
Jimmy Alamparambil, ByteDance

12.87.1. Overview

This extension extends `XR_BD_spatial_anchor` and allows applications to share spatial anchors between different sessions on different devices. This extension requires `XR_BD_spatial_anchor` to be enabled.

12.87.2. Inspect System Capability

```
// Provided by XR_BD_spatial_anchor_sharing
typedef struct XrSystemSpatialAnchorSharingPropertiesBD {
    XrStructureType    type;
    void*              next;
    XrBool32           supportsSpatialAnchorSharing;
} XrSystemSpatialAnchorSharingPropertiesBD;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. For more details about the structure chain, see the structure being extended (XrSystemProperties).
- `supportsSpatialAnchorSharing` is an `XrBool32`, indicating if the current system is capable of spatial anchor sharing.

An application **can** inspect whether the system is capable of spatial anchor sharing by chaining an XrSystemSpatialAnchorSharingPropertiesBD structure to the XrSystemProperties::`next` chain when calling xrGetSystemProperties.

If a runtime returns `XR_FALSE` for `supportsSpatialAnchorSharing`, the system does not support spatial anchor sharing. The application **should** avoid using spatial anchor sharing functionality when `supportsSpatialAnchorSharing` is `XR_FALSE`. If XrSystemSpatialSensingPropertiesBD::`supportsSpatialSensing` is `XR_FALSE`, then `supportsSpatialAnchorSharing` **must** also be `XR_FALSE`. Similarly, if XrSystemSpatialAnchorPropertiesBD::`supportsSpatialAnchor` is `XR_FALSE`, then `supportsSpatialAnchorSharing` **must** also be `XR_FALSE`.

If a runtime returns `XR_FALSE` for `supportsSpatialAnchorSharing`, the runtime **must** return `XR_ERROR_FEATURE_UNSUPPORTED` for all the functions defined in this extension.

If a runtime returns `XR_TRUE` for `supportsSpatialAnchorSharing`, the system supports spatial anchor sharing. This implies that XrSystemSpatialSensingPropertiesBD::`supportsSpatialSensing` and XrSystemSpatialAnchorPropertiesBD::`supportsSpatialAnchor` **must** also be also `XR_TRUE`.

Valid Usage (Implicit)

- The `XR_BD_spatial_anchor_sharing` extension **must** be enabled prior to using XrSystemSpatialAnchorSharingPropertiesBD
- `type` **must** be `XR_TYPE_SYSTEM_SPATIAL_ANCHOR_SHARING_PROPERTIES_BD`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

12.87.3. Create Spatial Anchor Provider

See Create Spatial Anchor Provider in `XR_BD_spatial_anchor` for information on creating a provider.

12.87.4. Share Spatial Anchor

The xrShareSpatialAnchorAsyncBD function is defined as:

```
// Provided by XR_BD_spatial_anchor_sharing
XrResult xrShareSpatialAnchorAsyncBD(
    XrSenseDataProviderBD                       provider,
    const XrSpatialAnchorShareInfoBD*           info,
    XrFutureEXT*                                future);
```

Parameter Descriptions

- `provider` is the previously created XrSenseDataProviderBD for spatial anchor.
- `info` is a pointer to the XrSpatialAnchorShareInfoBD .
- `future` is a pointer to the `XrFutureEXT`.

To share a spatial anchor, call xrShareSpatialAnchorAsyncBD.

This function starts an asynchronous operation and creates a corresponding `XrFutureEXT`, usable with xrPollFutureEXT and related functions. The return value of this function only indicates whether the parameters were acceptable to schedule the asynchronous operation. The corresponding completion function is xrShareSpatialAnchorCompleteBD, usable when a future from this function is in the READY state, with outputs populated by that function in the completion structure XrFutureCompletionEXT.

Shared spatial anchors are shared with other sessions and devices using a runtime defined method such as a cloud storage.

Valid Usage (Implicit)

- The `XR_BD_spatial_anchor_sharing` extension **must** be enabled prior to calling xrShareSpatialAnchorAsyncBD
- `provider` **must** be a valid XrSenseDataProviderBD handle
- `info` **must** be a pointer to a valid XrSpatialAnchorShareInfoBD structure
- `future` **must** be a pointer to an `XrFutureEXT` value

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FEATURE_UNSUPPORTED
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_SPATIAL_SENSING_SERVICE_UNAVAILABLE_BD
- XR_ERROR_VALIDATION_FAILURE

The XrSpatialAnchorShareInfoBD structure is defined as:

```
// Provided by XR_BD_spatial_anchor_sharing
typedef struct XrSpatialAnchorShareInfoBD {
    XrStructureType    type;
    const void*        next;
    XrAnchorBD         anchor;
} XrSpatialAnchorShareInfoBD;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- anchor is the XrAnchorBD handle of the spatial anchor to be shared.

Valid Usage (Implicit)

- The `XR_BD_spatial_anchor_sharing` extension **must** be enabled prior to using XrSpatialAnchorShareInfoBD
- `type` **must** be `XR_TYPE_SPATIAL_ANCHOR_SHARE_INFO_BD`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `anchor` **must** be a valid XrAnchorBD handle

The xrShareSpatialAnchorCompleteBD function is defined as:

```
// Provided by XR_BD_spatial_anchor_sharing
XrResult xrShareSpatialAnchorCompleteBD(
    XrSenseDataProviderBD                       provider,
    XrFutureEXT                                 future,
    XrFutureCompletionEXT*                      completion);
```

Parameter Descriptions

- `provider` is the XrSenseDataProviderBD.
- `future` is the `XrFutureEXT`.
- `completion` is a pointer to the XrFutureCompletionEXT.

The application obtains the spatial anchor sharing result using xrShareSpatialAnchorCompleteBD.

This is the completion function corresponding to the operation started by xrShareSpatialAnchorAsyncBD. Do not call until the `future` is READY.

The XrFutureCompletionEXT structure is defined in `XR_EXT_future`.

Future Return Codes

XrFutureCompletionEXT::`futureResult` values:

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_PERMISSION_INSUFFICIENT`

Valid Usage (Implicit)

- The `XR_BD_spatial_anchor_sharing` extension **must** be enabled prior to calling xrShareSpatialAnchorCompleteBD
- `provider` **must** be a valid XrSenseDataProviderBD handle
- `completion` **must** be a pointer to an XrFutureCompletionEXT structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FEATURE_UNSUPPORTED
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_FUTURE_INVALID_EXT
- XR_ERROR_FUTURE_PENDING_EXT
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_SPATIAL_SENSING_SERVICE_UNAVAILABLE_BD
- XR_ERROR_VALIDATION_FAILURE

12.87.5. Download Shared Spatial Anchor

The xrDownloadSharedSpatialAnchorAsyncBD function is defined as:

```
// Provided by XR_BD_spatial_anchor_sharing
XrResult xrDownloadSharedSpatialAnchorAsyncBD(
    XrSenseDataProviderBD                       provider,
    const XrSharedSpatialAnchorDownloadInfoBD*  info,
    XrFutureEXT*                                future);
```

Parameter Descriptions

- provider is the previously created XrSenseDataProviderBD for spatial anchor.
- info is a pointer to the XrSharedSpatialAnchorDownloadInfoBD .
- future is a pointer to the XrFutureEXT.

In order to access the shared spatial anchor, the application first downloads the anchor. To download a shared spatial anchor, call xrDownloadSharedSpatialAnchorAsyncBD.

The spatial anchor will be downloaded to the local device and runtime will then locate it.

This function starts an asynchronous operation and creates a corresponding XrFutureEXT, usable with xrPollFutureEXT and related functions. The return value of this function only indicates whether the parameters were acceptable to schedule the asynchronous operation. The corresponding completion function is xrDownloadSharedSpatialAnchorCompleteBD, usable when a future from this function is in the READY state, with outputs populated by that function in the completion structure XrFutureCompletionEXT.

Valid Usage (Implicit)

- The XR_BD_spatial_anchor_sharing extension **must** be enabled prior to calling xrDownloadSharedSpatialAnchorAsyncBD
- provider **must** be a valid XrSenseDataProviderBD handle
- info **must** be a pointer to a valid XrSharedSpatialAnchorDownloadInfoBD structure
- future **must** be a pointer to an XrFutureEXT value

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FEATURE_UNSUPPORTED
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_SPATIAL_SENSING_SERVICE_UNAVAILABLE_BD
- XR_ERROR_VALIDATION_FAILURE

The XrSharedSpatialAnchorDownloadInfoBD structure is defined as:

```
// Provided by XR_BD_spatial_anchor_sharing
typedef struct XrSharedSpatialAnchorDownloadInfoBD {
    XrStructureType    type;
    const void*        next;
    XrUuidEXT          uuid;
} XrSharedSpatialAnchorDownloadInfoBD;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `uuid` is the XrUuidEXT of the spatial anchor to be downloaded.

Valid Usage (Implicit)

- The `XR_BD_spatial_anchor_sharing` extension **must** be enabled prior to using XrSharedSpatialAnchorDownloadInfoBD
- `type` **must** be `XR_TYPE_SHARED_SPATIAL_ANCHOR_DOWNLOAD_INFO_BD`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

The xrDownloadSharedSpatialAnchorCompleteBD function is defined as:

```
// Provided by XR_BD_spatial_anchor_sharing
XrResult xrDownloadSharedSpatialAnchorCompleteBD(
    XrSenseDataProviderBD                       provider,
    XrFutureEXT                                 future,
    XrFutureCompletionEXT*                      completion);
```

Parameter Descriptions

- `provider` is the XrSenseDataProviderBD.
- `future` is the `XrFutureEXT`.
- `completion` is a pointer to the XrFutureCompletionEXT.

The application obtains the spatial anchor download result using xrDownloadSharedSpatialAnchorCompleteBD.

This is the completion function corresponding to the operation started by xrDownloadSharedSpatialAnchorAsyncBD. Do not call until the `future` is READY.

The XrFutureCompletionEXT structure is defined in `XR_EXT_future`.

Future Return Codes

XrFutureCompletionEXT::`futureResult` values:

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_PERMISSION_INSUFFICIENT`

If the spatial anchor is downloaded to the local device, the runtime **must** set the XrFutureCompletionEXT::`futureResult` value to `XR_SUCCESS`. This indicates that the spatial anchor has been successfully downloaded to the local device and located. To obtain the spatial anchors from the spatial anchor data provider, call xrQuerySenseDataAsyncBD, xrQuerySenseDataCompleteBD, xrGetQueriedSenseDataBD, and xrCreateSpatialEntityAnchorBD. All of these functions are defined in `XR_BD_spatial_sensing`.



Note

It is implementation defined whether the runtime implements permission controls, in which case it **may** set XrFutureCompletionEXT::`futureResult` to `XR_ERROR_PERMISSION_INSUFFICIENT` when attempting to download an anchor without permission.

Valid Usage (Implicit)

- The `XR_BD_spatial_anchor_sharing` extension **must** be enabled prior to calling xrDownloadSharedSpatialAnchorCompleteBD
- `provider` **must** be a valid XrSenseDataProviderBD handle
- `completion` **must** be a pointer to an XrFutureCompletionEXT structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FEATURE_UNSUPPORTED
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_FUTURE_INVALID_EXT
- XR_ERROR_FUTURE_PENDING_EXT
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_SPATIAL_SENSING_SERVICE_UNAVAILABLE_BD
- XR_ERROR_VALIDATION_FAILURE

12.87.6. New Commands

- xrDownloadSharedSpatialAnchorAsyncBD
- xrDownloadSharedSpatialAnchorCompleteBD
- xrShareSpatialAnchorAsyncBD
- xrShareSpatialAnchorCompleteBD

12.87.7. New Structures

- XrSharedSpatialAnchorDownloadInfoBD
- XrSpatialAnchorShareInfoBD
- Extending XrSystemProperties:
  - XrSystemSpatialAnchorSharingPropertiesBD

12.87.8. New Enum Constants

- XR_BD_SPATIAL_ANCHOR_SHARING_EXTENSION_NAME
- XR_BD_spatial_anchor_sharing_SPEC_VERSION

- Extending XrResult:
  - XR_ERROR_SPATIAL_ANCHOR_SHARING_AUTHENTICATION_FAILURE_BD
  - XR_ERROR_SPATIAL_ANCHOR_SHARING_LOCALIZATION_FAIL_BD
  - XR_ERROR_SPATIAL_ANCHOR_SHARING_MAP_INSUFFICIENT_BD
  - XR_ERROR_SPATIAL_ANCHOR_SHARING_NETWORK_FAILURE_BD
  - XR_ERROR_SPATIAL_ANCHOR_SHARING_NETWORK_TIMEOUT_BD
- Extending XrStructureType:
  - XR_TYPE_SHARED_SPATIAL_ANCHOR_DOWNLOAD_INFO_BD
  - XR_TYPE_SPATIAL_ANCHOR_SHARE_INFO_BD
  - XR_TYPE_SYSTEM_SPATIAL_ANCHOR_SHARING_PROPERTIES_BD

12.87.9. Issues

12.87.10. Version History

- Revision 1, 2024-05-06 (Zhipeng Liu)
  - Initial extension description
- Revision 2, 2025-06-02 (Zhipeng Liu)
  - Update specification of xrDownloadSharedSpatialAnchorCompleteBD.

12.88. XR_BD_spatial_audio_rendering

Name String

XR_BD_spatial_audio_rendering

Extension Type

Instance extension

Registered Extension Number

410

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Contributors

Xuzhou Ye, ByteDance
Shuai Liu, ByteDance

12.88.1. Overview

This extension allows applications to render spatial audio in a 3D space. It allows applications to create sound scenes, and submit raw audio data which is then processed and rendered by the system.

12.88.2. Create Spatial Audio Renderer

```
// Provided by XR_BD_spatial_audio_rendering
XR_DEFINE_HANDLE(XrSpatialAudioRendererBD)
```

An XrSpatialAudioRendererBD handle represents a spatial audio renderer instance, which is created by calling xrCreateSpatialAudioRendererBD.

Before creating an XrSpatialAudioRendererBD, there are two enumeration functions to call to negotiate mutually acceptable parameters between the runtime and the application: xrEnumerateSupportedAudioSampleRateBD and xrQueryFramesPerBufferRangeBD.

The xrEnumerateSupportedAudioSampleRateBD function is defined as:

```
// Provided by XR_BD_spatial_audio_rendering
XrResult xrEnumerateSupportedAudioSampleRateBD(
    XrSession                                   session,
    uint32_t                                    sampleRateCapacityInput,
    uint32_t*                                   sampleRateCountOutput,
    XrAudioSampleRateBD*                        sampleRates);
```

Parameter Descriptions

- `session` is the XrSession for which to enumerate supported audio sample rates.
- `sampleRateCapacityInput` is the capacity of the `sampleRates` array, or 0 to indicate a request to retrieve the required capacity.
- `sampleRateCountOutput` is a pointer to the count of `sampleRates` written, or a pointer to the required capacity in the case that `sampleRateCapacityInput` is insufficient.
- `sampleRates` is a pointer to an array of XrAudioSampleRateBD structures, but **can** be `NULL` if `sampleRateCapacityInput` is 0.
- See the Buffer Size Parameters section for a detailed description of retrieving the required `sampleRates` size.

Before creating the XrSpatialAudioRendererBD, applications **should** call xrEnumerateSupportedAudioSampleRateBD to enumerate supported audio sample rates. Applications **must** set XrSpatialAudioRendererCreateInfoBD::`sampleRate` to one of the supported sample rates, otherwise the runtime **must** return `XR_ERROR_VALIDATION_FAILURE` from xrCreateSpatialAudioRendererBD.

The values enumerated by this function **must** not change during the lifetime of an XrInstance.

Valid Usage (Implicit)

- The `XR_BD_spatial_audio_rendering` extension **must** be enabled prior to calling xrEnumerateSupportedAudioSampleRateBD
- `session` **must** be a valid XrSession handle
- `sampleRateCountOutput` **must** be a pointer to a `uint32_t` value
- If `sampleRateCapacityInput` is not `0`, `sampleRates` **must** be a pointer to an array of `sampleRateCapacityInput` XrAudioSampleRateBD values

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_VALIDATION_FAILURE`

The XrAudioSampleRateBD enumeration describes an audio sample rate.

```
// Provided by XR_BD_spatial_audio_rendering
typedef enum XrAudioSampleRateBD {
    XR_AUDIO_SAMPLE_RATE_192000_HZ_BD = 1,
    XR_AUDIO_SAMPLE_RATE_96000_HZ_BD = 2,
    XR_AUDIO_SAMPLE_RATE_48000_HZ_BD = 3,
    XR_AUDIO_SAMPLE_RATE_44100_HZ_BD = 4,
    XR_AUDIO_SAMPLE_RATE_32000_HZ_BD = 5,
    XR_AUDIO_SAMPLE_RATE_24000_HZ_BD = 6,
    XR_AUDIO_SAMPLE_RATE_22050_HZ_BD = 7,
    XR_AUDIO_SAMPLE_RATE_16000_HZ_BD = 8,
    XR_AUDIO_SAMPLE_RATE_12000_HZ_BD = 9,
    XR_AUDIO_SAMPLE_RATE_11025_HZ_BD = 10,
    XR_AUDIO_SAMPLE_RATE_8000_HZ_BD = 11,
    XR_AUDIO_SAMPLE_RATE_MAX_ENUM_BD = 0x7FFFFFFF
} XrAudioSampleRateBD;
```

Enum	Description
XR_AUDIO_SAMPLE_RATE_192000_HZ_BD	192000 Hz
XR_AUDIO_SAMPLE_RATE_96000_HZ_BD	96000 Hz
XR_AUDIO_SAMPLE_RATE_48000_HZ_BD	48000 Hz
XR_AUDIO_SAMPLE_RATE_44100_HZ_BD	44100 Hz
XR_AUDIO_SAMPLE_RATE_32000_HZ_BD	32000 Hz
XR_AUDIO_SAMPLE_RATE_24000_HZ_BD	24000 Hz
XR_AUDIO_SAMPLE_RATE_22050_HZ_BD	22050 Hz
XR_AUDIO_SAMPLE_RATE_16000_HZ_BD	16000 Hz
XR_AUDIO_SAMPLE_RATE_12000_HZ_BD	12000 Hz
XR_AUDIO_SAMPLE_RATE_11025_HZ_BD	11025 Hz
XR_AUDIO_SAMPLE_RATE_8000_HZ_BD	8000 Hz

The xrQueryFramesPerBufferRangeBD function is defined as:

```
// Provided by XR_BD_spatial_audio_rendering
XrResult xrQueryFramesPerBufferRangeBD(
    XrSession                                   session,
    XrAudioSampleRateBD                         sampleRate,
    uint32_t*                                   min,
    uint32_t*                                   max);
```

Parameter Descriptions

- `session` is the XrSession to query supported frames per buffer range.
- `sampleRate` is the sample rate for which the supported frames per buffer range is queried.
- `min` is a pointer to an uint32_t that receives the minimum supported frames per buffer.
- `max` is a pointer to an uint32_t that receives the maximum supported frames per buffer.

Before creating the XrSpatialAudioRendererBD, applications **should** call xrQueryFramesPerBufferRangeBD to query supported frames per buffer range. Applications **must** set XrSpatialAudioRendererCreateInfoBD::`framesPerBuffer` to a value within the supported range, otherwise the runtime **must** return `XR_ERROR_VALIDATION_FAILURE` from xrCreateSpatialAudioRendererBD.

The values enumerated by this function **must** not change during the lifetime of an XrInstance.

Valid Usage (Implicit)

- The `XR_BD_spatial_audio_rendering` extension **must** be enabled prior to calling xrQueryFramesPerBufferRangeBD
- `session` **must** be a valid XrSession handle
- `sampleRate` **must** be a valid XrAudioSampleRateBD value
- `min` **must** be a pointer to a `uint32_t` value
- `max` **must** be a pointer to a `uint32_t` value

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The xrCreateSpatialAudioRendererBD function is defined as:

```
// Provided by XR_BD_spatial_audio_rendering
XrResult xrCreateSpatialAudioRendererBD(
    XrSession                                   session,
    const XrSpatialAudioRendererCreateInfoBD*   createInfo,
    XrSpatialAudioRendererBD*                   renderer);
```

Parameter Descriptions

- session is the XrSession to create the audio renderer in.
- createInfo is a pointer to the XrSpatialAudioRendererCreateInfoBD structure used to create and initialize the spatial audio renderer instance.
- renderer is a pointer to an XrSpatialAudioRendererBD that receives the handle of the created spatial audio renderer instance.

To create an XrSpatialAudioRendererBD, applications call xrCreateSpatialAudioRendererBD.

Runtimes **must** allow at most one XrSpatialAudioRendererBD to exist within a session, and **must** return XR_ERROR_LIMIT_REACHED if an application calls xrCreateSpatialAudioRendererBD to create another XrSpatialAudioRendererBD while another one already exists in the same session.

An application **may** create a new XrSpatialAudioRendererBD after the previous one has been destroyed by calling xrDestroySpatialAudioRendererBD.

Valid Usage (Implicit)

- The XR_BD_spatial_audio_rendering extension **must** be enabled prior to calling xrCreateSpatialAudioRendererBD
- session **must** be a valid XrSession handle
- createInfo **must** be a pointer to a valid XrSpatialAudioRendererCreateInfoBD structure
- renderer **must** be a pointer to an XrSpatialAudioRendererBD handle

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The XrSpatialAudioRendererCreateInfoBD structure is defined as:

```
// Provided by XR_BD_spatial_audio_rendering
typedef struct XrSpatialAudioRendererCreateInfoBD {
    XrStructureType        type;
    const void*            next;
    uint32_t               framesPerBuffer;
    XrAudioSampleRateBD    sampleRate;
} XrSpatialAudioRendererCreateInfoBD;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `framesPerBuffer` is the number of frames contained in each audio data buffer submitted by the application. Each frame consists of one sample for each channel. Applications **should** call xrQueryFramesPerBufferRangeBD to get the supported range of `framesPerBuffer`, and **must** set the `framesPerBuffer` to a value within the supported range.
- `sampleRate` is the number of audio frames per second of the audio data submitted by the application. Applications **should** call xrEnumerateSupportedAudioSampleRateBD to get the supported sample rates, and **must** set the `sampleRate` as one of the supported sample rates.

The XrSpatialAudioRendererCreateInfoBD structure describes the audio configuration the application will use for the submitted audio data. The values of `framesPerBuffer` and `sampleRate` **must** be a supported value according to their respective functions xrEnumerateSupportedAudioSampleRateBD and xrQueryFramesPerBufferRangeBD. If the specified values are not supported by the runtime, xrCreateSpatialAudioRendererBD **must** return `XR_ERROR_VALIDATION_FAILURE`.

Valid Usage (Implicit)

- The `XR_BD_spatial_audio_rendering` extension **must** be enabled prior to using XrSpatialAudioRendererCreateInfoBD
- `type` **must** be `XR_TYPE_SPATIAL_AUDIO_RENDERER_CREATE_INFO_BD`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `sampleRate` **must** be a valid XrAudioSampleRateBD value

The xrDestroySpatialAudioRendererBD function is defined as:

```
// Provided by XR_BD_spatial_audio_rendering
XrResult xrDestroySpatialAudioRendererBD(
    XrSpatialAudioRendererBD                    renderer);
```

Parameter Descriptions

- `renderer` is the XrSpatialAudioRendererBD to destroy.

Applications **should** destroy the spatial audio renderer if it is no longer required. When applications

call xrDestroySpatialAudioRendererBD, the runtime also destroys the sound scene and **may** free the associated resources. Destroying the audio renderer stops all audio playback associated with it, if any.

Valid Usage (Implicit)

- The XR_BD_spatial_audio_rendering extension **must** be enabled prior to calling xrDestroySpatialAudioRendererBD
- `renderer` **must** be a valid XrSpatialAudioRendererBD handle

Thread Safety

- Access to `renderer`, and any child handles, **must** be externally synchronized

Return Codes

Success

- XR_SUCCESS

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_RUNTIME_FAILURE

12.88.3. Audience

With this extension, the application does not configure an audience explicitly. The runtime always renders audio for the user's position and orientation, which correlate with XR_REFERENCE_SPACE_TYPE_VIEW (the "audience" in audio terms).

Note that this section uses the term **audience** descriptively; readers familiar with audio middleware may know this concept under that name. No additional audience handle or configuration is introduced by this extension.

When the tracking quality of the audience changes (VIEW reference space), spatial audio is rendered in the following way:

- While the VIEW reference space pose is valid, but not tracked (i.e. xrLocateSpace reports its location with XR_SPACE_LOCATION_POSITION_VALID_BIT and XR_SPACE_LOCATION_ORIENTATION_VALID_BIT set, but XR_SPACE_LOCATION_POSITION_TRACKED_BIT and/or XR_VIEW_STATE_POSITION_TRACKED_BIT not set), the runtime **must** continue full spatial audio rendering using the reported pose state without introducing discontinuities.

- While the VIEW reference space pose is neither valid nor tracked (i.e. xrLocateSpace reports its location with at least one of XR_SPACE_LOCATION_POSITION_VALID_BIT and XR_SPACE_LOCATION_ORIENTATION_VALID_BIT unset), the runtime **should** fall back to use the most recent valid pose of the view space.

12.88.4. Create Sound Scene and Sound Sources

To create a realistic spatial audio effect, the application creates a sound scene based on the virtual environment in which the audio is intended to be rendered. This scene includes application submitted information about sound sources (sound fields and sound objects) within the scene. The runtime calculates and renders spatial audio based on this scene.

There are two types of sound sources:

- Sound Object - A sound object is a point in 3D space; it has a position and a volume. For example a shotgun, a car engine, or a bird would be modeled as a sound object.
- Sound Field - A sound field is generally a background sound; it has a position and orientation.

Applications create sound sources in the scene by calling xrCreateSoundFieldBD and xrCreateSoundObjectBD.

Create Sound Fields

```
// Provided by XR_BD_spatial_audio_rendering
XR_DEFINE_HANDLE(XrSoundFieldBD)
```

An XrSoundFieldBD handle represents a sound field.

The xrCreateSoundFieldBD function is defined as:

```
// Provided by XR_BD_spatial_audio_rendering
XrResult xrCreateSoundFieldBD(
    XrSpatialAudioRendererBD                    renderer,
    const XrSoundFieldConfigBD*                 config,
    XrSoundFieldBD*                             soundField);
```

Parameter Descriptions

- renderer is the XrSpatialAudioRendererBD used for spatial audio rendering.
- config is the XrSoundFieldConfigBD that defines the properties of the sound field.
- soundField is a pointer to a handle in which the xrCreateSoundFieldBD is returned.

When creating a sound field in the sound scene rendered by the spatial audio renderer, applications **must** chain a structure that indicates the channel definition of the sound field to XrSoundFieldConfigBD::`next`.

Channel definitions specified by this extension are:

- Channel-based surround sound (for example 5.1): chain XrSoundFieldChannelDefinitionSurroundBD.
- AmbiX format (ACN channel order and SN3D normalization): chain XrSoundFieldChannelDefinitionAmbixBD.
- FuMA format (Furse-Malham channel order and maxN normalization): chain XrSoundFieldChannelDefinitionFumaBD.

If the application does not chain any structure specifying the sound format to XrSoundFieldConfigBD::`next`, the runtime **must** return `XR_ERROR_VALIDATION_FAILURE`.

If multiple sound field config structures are chained, the runtime ignores all but one of those structs. To avoid unexpected results, applications **should** only chain one sound field definition structure to indicate the channel definition of the sound field.

Valid Usage (Implicit)

- The `XR_BD_spatial_audio_rendering` extension **must** be enabled prior to calling xrCreateSoundFieldBD
- `renderer` **must** be a valid XrSpatialAudioRendererBD handle
- `config` **must** be a pointer to a valid XrSoundFieldConfigBD structure
- `soundField` **must** be a pointer to an XrSoundFieldBD handle

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_POSE_INVALID
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The XrSoundFieldConfigBD structure is defined as:

```
// Provided by XR_BD_spatial_audio_rendering
typedef struct XrSoundFieldConfigBD {
     XrStructureType    type;
    const void*         next;
    XrBool32            enabled;
    XrQuaternionf       orientation;
    XrSpace             baseSpace;
    float               mainVolume;
    float               lfeGain;
} XrSoundFieldConfigBD;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `enabled` is an `XrBool32`, indicating whether the sound field is enabled.
- `orientation` is the XrQuaternionf of the sound field. It defines the orientation of the sound field in the `baseSpace`.
- `baseSpace` is an XrSpace which the `orientation` is defined in.
- `mainVolume` is the main volume of the sound field. It acts as a linear gain attenuation of the input signal in the range (0, 1).
- `lfeGain` is the LFE (low-frequency effects) gain of the sound field. It acts as a linear gain attenuation of the input signal in the range (0, 1) on top of the main volume.

The behavior below specifies how the runtime renders audio when tracking quality changes for the `baseSpace`.

- While the `baseSpace` pose is valid, but not tracked (i.e. xrLocateSpace reports its location with `XR_SPACE_LOCATION_POSITION_VALID_BIT` and `XR_SPACE_LOCATION_ORIENTATION_VALID_BIT` set, but not `XR_SPACE_LOCATION_POSITION_TRACKED_BIT` and `XR_VIEW_STATE_POSITION_TRACKED_BIT`), the runtime **must** keep rendering the sound field using the most recent valid pose of that space.
- While the `baseSpace` pose is neither valid nor tracked (i.e. xrLocateSpace reports its location with either `XR_SPACE_LOCATION_POSITION_VALID_BIT` or `XR_SPACE_LOCATION_ORIENTATION_VALID_BIT` unset), the runtime **should** keep rendering the sound field using the most recent valid position and orientation of that space relative to its parent.

Valid Usage (Implicit)

- The `XR_BD_spatial_audio_rendering` extension **must** be enabled prior to using XrSoundFieldConfigBD
- `type` **must** be `XR_TYPE_SOUND_FIELD_CONFIG_BD`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain. See also: XrSoundFieldChannelDefinitionAmbixBD, XrSoundFieldChannelDefinitionFumaBD, XrSoundFieldChannelDefinitionSurroundBD
- `baseSpace` **must** be a valid XrSpace handle

Channel-based Surround Configuration

The XrSoundFieldChannelDefinitionSurroundBD structure is defined as:

```
// Provided by XR_BD_spatial_audio_rendering
typedef struct XrSoundFieldChannelDefinitionSurroundBD {
    XrStructureType                       type;
    const void*                           next;
    XrSoundFieldChannelMaskSurroundBD     channelMask;
} XrSoundFieldChannelDefinitionSurroundBD;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `channelMask` is the channel mask of the sound field.

Valid Usage (Implicit)

- The `XR_BD_spatial_audio_rendering` extension **must** be enabled prior to using XrSoundFieldChannelDefinitionSurroundBD
- `type` **must** be `XR_TYPE_SOUND_FIELD_CHANNEL_DEFINITION_SURROUND_BD`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `channelMask` **must** be a valid XrSoundFieldChannelMaskSurroundBD value

The XrSoundFieldChannelMaskSurroundBD describes the channel mask of a surround sound field. Single bit values for each single channel are included, but applications typically select a value (or combination) that includes the word `SETUP`, which describes a predefined "composed" configuration.

```
// Provided by XR_BD_spatial_audio_rendering
typedef enum XrSoundFieldChannelMaskSurroundBD {
    XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_FRONT_LEFT_BD = 1,
    XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_FRONT_RIGHT_BD = 2,
    XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_FRONT_CENTER_BD = 4,
    XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_LOW_FREQUENCY_BD = 8,
    XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_BACK_LEFT_BD = 16,
    XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_BACK_RIGHT_BD = 32,
    XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_SIDE_LEFT_BD = 64,
    XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_SIDE_RIGHT_BD = 128,
    XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_TOP_FRONT_LEFT_BD = 256,
    XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_TOP_FRONT_RIGHT_BD = 512,
    XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_TOP_BACK_LEFT_BD = 1024,
    XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_TOP_BACK_RIGHT_BD = 2048,
    XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_BACK_CENTER_BD = 4096,
    XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_TOP_SIDE_LEFT_BD = 8192,
    XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_TOP_SIDE_RIGHT_BD = 16384,
    XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_SETUP_STEREO_BD = 3,
    XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_SETUP_2_1_BD = 11,
    XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_SETUP_3_0_BD = 7,
    XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_SETUP_4_0_BD = 4099,
    XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_SETUP_BACK_SURROUND_BD = 48,
    XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_SETUP_QUAD_BD = 51,
    XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_SETUP_3_1_BD = 15,
    XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_SETUP_5_0_BD = 55,
    XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_SETUP_SIDE_SURROUND_BD = 192,
    XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_SETUP_5_0_SIDE_BD = 199,
    XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_SETUP_5_1_BD = 63,
    XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_SETUP_5_1_SIDE_BD = 207,
    XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_SETUP_7_0_BD = 247,
    XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_SETUP_7_1_BD = 255,
    XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_SETUP_5_1_2_BD = 831,
    XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_SETUP_5_1_4_BD = 3903,
    XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_SETUP_7_1_2_BD = 24831,
    XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_SETUP_7_1_4_BD = 4095,
    XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_MAX_ENUM_BD = 0x7FFFFFFF
} XrSoundFieldChannelMaskSurroundBD;
```

Enumerant Descriptions

- `XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_FRONT_LEFT_BD` — Bitmask of ear-level front-left speaker
- `XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_FRONT_RIGHT_BD` — Bitmask of ear-level front-right speaker
- `XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_FRONT_CENTER_BD` — Bitmask of ear-level front-center speaker
- `XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_LOW_FREQUENCY_BD` — Bitmask of LFE speaker
- `XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_BACK_LEFT_BD` — Bitmask of ear-level back-left speaker
- `XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_BACK_RIGHT_BD` — Bitmask of ear-level back-right speaker
- `XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_SIDE_LEFT_BD` — Bitmask of ear-level side-left speaker
- `XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_SIDE_RIGHT_BD` — Bitmask of ear-level side-right speaker
- `XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_TOP_FRONT_LEFT_BD` — Bitmask of height-channel front-left speaker
- `XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_TOP_FRONT_RIGHT_BD` — Bitmask of height-channel front-right speaker
- `XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_TOP_BACK_LEFT_BD` — Bitmask of height-channel back-left speaker
- `XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_TOP_BACK_RIGHT_BD` — Bitmask of height-channel back-right speaker
- `XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_BACK_CENTER_BD` — Bitmask of ear-level back-center speaker
- `XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_TOP_SIDE_LEFT_BD` — Bitmask of height-channel side-left speaker
- `XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_TOP_SIDE_RIGHT_BD` — Bitmask of height-channel side-right speaker
- `XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_SETUP_STEREO_BD` — Composition of `XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_FRONT_LEFT_BD` and `XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_FRONT_RIGHT_BD`
- `XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_SETUP_2_1_BD` — Composition of `XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_SETUP_STEREO_BD` and `XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_LOW_FREQUENCY_BD`
- `XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_SETUP_3_0_BD` — Composition of `XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_SETUP_STEREO_BD` and `XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_FRONT_CENTER_BD`

- `XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_SETUP_4_0_BD` — Composition of
  `XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_SETUP_3_0_BD` and
  `XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_BACK_CENTER_BD`
- `XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_SETUP_BACK_SURROUND_BD` — Composition of
  `XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_BACK_LEFT_BD` and
  `XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_BACK_RIGHT_BD`
- `XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_SETUP_QUAD_BD` — Composition of
  `XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_SETUP_STEREO_BD` and
  `XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_SETUP_BACK_SURROUND_BD`
- `XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_SETUP_3_1_BD` — Composition of
  `XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_SETUP_3_0_BD` and
  `XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_LOW_FREQUENCY_BD`
- `XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_SETUP_5_0_BD` — Composition of
  `XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_SETUP_3_0_BD` and
  `XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_SETUP_BACK_SURROUND_BD`
- `XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_SETUP_SIDE_SURROUND_BD` — Composition of
  `XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_SIDE_LEFT_BD` and
  `XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_SIDE_RIGHT_BD`
- `XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_SETUP_5_0_SIDE_BD` — Composition of
  `XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_SETUP_3_0_BD` and
  `XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_SETUP_SIDE_SURROUND_BD`
- `XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_SETUP_5_1_BD` — Composition of
  `XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_SETUP_5_0_BD` and
  `XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_LOW_FREQUENCY_BD`
- `XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_SETUP_5_1_SIDE_BD` — Composition of
  `XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_SETUP_5_1_BD` and
  `XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_SETUP_SIDE_SURROUND_BD`
- `XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_SETUP_7_0_BD` — Composition of
  `XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_SETUP_5_0_BD` and
  `XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_SETUP_SIDE_SURROUND_BD`
- `XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_SETUP_7_1_BD` — Composition of
  `XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_SETUP_7_0_BD` and
  `XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_LOW_FREQUENCY_BD`
- `XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_SETUP_5_1_2_BD` — Composition of
  `XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_SETUP_5_1_BD` and
  `XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_TOP_FRONT_LEFT_BD` and
  `XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_TOP_FRONT_RIGHT_BD`
- `XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_SETUP_5_1_4_BD` — Composition of
  `XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_SETUP_5_1_BD` and
  `XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_TOP_FRONT_LEFT_BD` and
  `XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_TOP_FRONT_RIGHT_BD` and
  `XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_TOP_BACK_LEFT_BD` and
  `XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_TOP_BACK_RIGHT_BD`

- XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_SETUP_7_1_2_BD — Composition of XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_SETUP_7_1_BD and XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_TOP_SIDE_LEFT_BD and XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_TOP_SIDE_RIGHT_BD
- XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_SETUP_7_1_4_BD — Composition of XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_SETUP_7_1_BD and XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_TOP_FRONT_LEFT_BD and XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_TOP_FRONT_RIGHT_BD and XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_TOP_BACK_LEFT_BD and XR_SOUND_FIELD_CHANNEL_MASK_SURROUND_TOP_BACK_RIGHT_BD

AmbiX Configuration

The XrSoundFieldChannelDefinitionAmbixBD structure is defined as:

```
// Provided by XR_BD_spatial_audio_rendering
typedef struct XrSoundFieldChannelDefinitionAmbixBD {
    XrStructureType                   type;
    const void*                       next;
    XrSoundFieldChannelMaskAmbixBD    channelMask;
} XrSoundFieldChannelDefinitionAmbixBD;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain.
- channelMask is the channel mask of the sound field.

Valid Usage (Implicit)

- The XR_BD_spatial_audio_rendering extension **must** be enabled prior to using XrSoundFieldChannelDefinitionAmbixBD
- type **must** be XR_TYPE_SOUND_FIELD_CHANNEL_DEFINITION_AMBIX_BD
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- channelMask **must** be a valid XrSoundFieldChannelMaskAmbixBD value

The XrSoundFieldChannelMaskAmbixBD describes the channel mask of an AmbiX sound field.

```
// Provided by XR_BD_spatial_audio_rendering
typedef enum XrSoundFieldChannelMaskAmbixBD {
    XR_SOUND_FIELD_CHANNEL_MASK_AMBIX_1ST_ORDER_BD = 1,
    XR_SOUND_FIELD_CHANNEL_MASK_AMBIX_2ND_ORDER_BD = 2,
    XR_SOUND_FIELD_CHANNEL_MASK_AMBIX_3RD_ORDER_BD = 3,
    XR_SOUND_FIELD_CHANNEL_MASK_AMBIX_4TH_ORDER_BD = 4,
    XR_SOUND_FIELD_CHANNEL_MASK_AMBIX_5TH_ORDER_BD = 5,
    XR_SOUND_FIELD_CHANNEL_MASK_AMBIX_6TH_ORDER_BD = 6,
    XR_SOUND_FIELD_CHANNEL_MASK_AMBIX_7TH_ORDER_BD = 7,
    XR_SOUND_FIELD_CHANNEL_MASK_AMBIX_MAX_ENUM_BD = 0x7FFFFFFF
} XrSoundFieldChannelMaskAmbixBD;
```

Enumerant Descriptions

- XR_SOUND_FIELD_CHANNEL_MASK_AMBIX_1ST_ORDER_BD — Indicates 1st-order ambisonics in AmbiX format, which uses ACN (Ambisonic Channel Number) channel ordering and SN3D normalization
- XR_SOUND_FIELD_CHANNEL_MASK_AMBIX_2ND_ORDER_BD — Indicates 2nd-order ambisonics in AmbiX format, which uses ACN (Ambisonic Channel Number) channel ordering and SN3D normalization
- XR_SOUND_FIELD_CHANNEL_MASK_AMBIX_3RD_ORDER_BD — Indicates 3rd-order ambisonics in AmbiX format, which uses ACN (Ambisonic Channel Number) channel ordering and SN3D normalization
- XR_SOUND_FIELD_CHANNEL_MASK_AMBIX_4TH_ORDER_BD — Indicates 4th-order ambisonics in AmbiX format, which uses ACN (Ambisonic Channel Number) channel ordering and SN3D normalization
- XR_SOUND_FIELD_CHANNEL_MASK_AMBIX_5TH_ORDER_BD — Indicates 5th-order-ambisonics in AmbiX format, which uses ACN (Ambisonic Channel Number) channel ordering and SN3D normalization
- XR_SOUND_FIELD_CHANNEL_MASK_AMBIX_6TH_ORDER_BD — Indicates 6th-order ambisonics in AmbiX format, which uses ACN (Ambisonic Channel Number) channel ordering and SN3D normalization
- XR_SOUND_FIELD_CHANNEL_MASK_AMBIX_7TH_ORDER_BD — Indicates 7th-order ambisonics in AmbiX format, which uses ACN (Ambisonic Channel Number) channel ordering and SN3D normalization

FuMA Configuration

The XrSoundFieldChannelDefinitionFumaBD structure is defined as:

```
// Provided by XR_BD_spatial_audio_rendering
typedef struct XrSoundFieldChannelDefinitionFumaBD {
    XrStructureType                 type;
    const void*                     next;
    XrSoundFieldChannelMaskFumaBD   channelMask;
} XrSoundFieldChannelDefinitionFumaBD;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `channelMask` is the channel mask of the sound field.

Valid Usage (Implicit)

- The `XR_BD_spatial_audio_rendering` extension **must** be enabled prior to using XrSoundFieldChannelDefinitionFumaBD
- `type` **must** be `XR_TYPE_SOUND_FIELD_CHANNEL_DEFINITION_FUMA_BD`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `channelMask` **must** be a valid XrSoundFieldChannelMaskFumaBD value

The XrSoundFieldChannelMaskFumaBD describes the channel mask of a FuMA sound field.

```
// Provided by XR_BD_spatial_audio_rendering
typedef enum XrSoundFieldChannelMaskFumaBD {
    XR_SOUND_FIELD_CHANNEL_MASK_FUMA_1ST_ORDER_BD = 1,
    XR_SOUND_FIELD_CHANNEL_MASK_FUMA_MAX_ENUM_BD = 0x7FFFFFFF
} XrSoundFieldChannelMaskFumaBD;
```

Enumerant Descriptions

- `XR_SOUND_FIELD_CHANNEL_MASK_FUMA_1ST_ORDER_BD` — Indicates the 1st-order ambisonic in FuMA format, which refers to Furse-Malham channel order and maxN normalization

Update Sound Field Config

The xrUpdateSoundFieldConfigBD function is defined as:

```
// Provided by XR_BD_spatial_audio_rendering
XrResult xrUpdateSoundFieldConfigBD(
    XrSoundFieldBD                              soundField,
    const XrSoundFieldConfigBD*                 config,
    XrSoundFieldFlagsBD                         flags);
```

Parameter Descriptions

- `soundField` is the XrSoundFieldBD to be updated.
- `config` is the XrSoundFieldConfigBD that defines the configuration of the sound field.
- `flags` is the bit mask (bitwise `OR` of XrSoundFieldFlagsBD) of the sound field configuration to change.

When updating the configuration of a submitted sound field, applications set `flags` to specify which properties to update. The runtime **must** not update a property if the corresponding flag bit is not set. The runtime **must** not read the parameters of XrSoundFieldConfigBD if the corresponding bits are not set; this means that the application **may** pass invalid or uninitialized fields for parameters it does not intend to update.

Valid Usage (Implicit)

- The `XR_BD_spatial_audio_rendering` extension **must** be enabled prior to calling xrUpdateSoundFieldConfigBD
- `soundField` **must** be a valid XrSoundFieldBD handle
- If `config` is not `NULL`, `config` **must** be a pointer to a valid XrSoundFieldConfigBD structure
- `flags` **must** be a valid combination of XrSoundFieldFlagBitsBD values
- `flags` **must** not be `0`

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_POSE_INVALID
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

```
// Provided by XR_BD_spatial_audio_rendering
typedef XrFlags64 XrSoundFieldFlagsBD;
```

```
// Provided by XR_BD_spatial_audio_rendering
// Flag bits for XrSoundFieldFlagsBD
static const XrSoundFieldFlagsBD XR_SOUND_FIELD_ENABLED_BIT_BD = 0x00000001;
static const XrSoundFieldFlagsBD XR_SOUND_FIELD_ORIENTATION_BIT_BD = 0x00000002;
static const XrSoundFieldFlagsBD XR_SOUND_FIELD_MAIN_VOLUME_BIT_BD = 0x00000004;
static const XrSoundFieldFlagsBD XR_SOUND_FIELD_LFE_GAIN_BIT_BD = 0x00000008;
```

Flag Descriptions

- XR_SOUND_FIELD_ENABLED_BIT_BD — to indicate an update of XrSoundFieldConfigBD::enabled
- XR_SOUND_FIELD_ORIENTATION_BIT_BD — to indicate an update of XrSoundFieldConfigBD::pose and/or XrSoundFieldConfigBD::baseSpace
- XR_SOUND_FIELD_MAIN_VOLUME_BIT_BD — to indicate an update of XrSoundFieldConfigBD::mainVolume
- XR_SOUND_FIELD_LFE_GAIN_BIT_BD — to indicate an update of XrSoundFieldConfigBD::lfeGain

Destroy Sound Field

The xrDestroySoundFieldBD function is defined as:

```
// Provided by XR_BD_spatial_audio_rendering
XrResult xrDestroySoundFieldBD(
    XrSoundFieldBD                              soundField);
```

Parameter Descriptions

- `soundField` is the handle of the sound field to be removed.

When an application removes a sound field with xrDestroySoundFieldBD, its corresponding sound is not rendered anymore.

Valid Usage (Implicit)

- The `XR_BD_spatial_audio_rendering` extension **must** be enabled prior to calling xrDestroySoundFieldBD
- `soundField` **must** be a valid XrSoundFieldBD handle

Thread Safety

- Access to `soundField`, and any child handles, **must** be externally synchronized

Return Codes

Success

- `XR_SUCCESS`

Failure

- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_RUNTIME_FAILURE`

Create Sound Objects

```
// Provided by XR_BD_spatial_audio_rendering
XR_DEFINE_HANDLE(XrSoundObjectBD)
```

An XrSoundObjectBD handle represents a sound object.

The xrCreateSoundObjectBD function is defined as:

```
// Provided by XR_BD_spatial_audio_rendering
XrResult xrCreateSoundObjectBD(
    XrSpatialAudioRendererBD                    renderer,
    const XrSoundObjectConfigBD*                config,
    XrSoundObjectBD*                            soundObject);
```

Parameter Descriptions

- `renderer` is the XrSpatialAudioRendererBD used for spatial audio rendering.
- `config` is the XrSoundObjectConfigBD that defines the configuration of the sound object.
- `soundObject` is a pointer to a handle in which the xrCreateSoundObjectBD returns

Applications add a sound object to the sound scene rendered by the spatial audio renderer with this function.

Valid Usage (Implicit)

- The `XR_BD_spatial_audio_rendering` extension **must** be enabled prior to calling xrCreateSoundObjectBD
- `renderer` **must** be a valid XrSpatialAudioRendererBD handle
- `config` **must** be a pointer to a valid XrSoundObjectConfigBD structure
- `soundObject` **must** be a pointer to an XrSoundObjectBD handle

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_POSE_INVALID
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The XrSoundObjectConfigBD structure is defined as:

```
// Provided by XR_BD_spatial_audio_rendering
typedef struct XrSoundObjectConfigBD {
    XrStructureType                                 type;
    const void*                                     next;
    XrBool32                                        enabled;
    XrPosef                                         pose;
    XrSpace                                         baseSpace;
    float                                           mainVolume;
    float                                           reflectionGain;
    XrBool32                                        enableDoppler;
    const XrSoundObjectDistanceAttenuationBD*       directSoundAttenuation;
    const XrSoundObjectDistanceAttenuationBD*       indirectSoundAttenuation;
} XrSoundObjectConfigBD;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `enabled` is an `XrBool32`, indicating whether the sound object is enabled.
- `pose` is an XrPosef of the sound object.
- `baseSpace` is an XrSpace which the `pose` is defined in.
- `mainVolume` is the main volume of the sound object. It acts as a linear gain attenuation of the input signal in the range (0, 1).
- `reflectionGain` is the reflection gain of the sound object. It acts as a linear gain attenuation of the input signal in the range (0, 1) on top of the main volume.
- `enableDoppler` is an `XrBool32`, indicating whether the Doppler effect is enabled.
- `directSoundAttenuation` is the XrSoundObjectDistanceAttenuationBD of the direct sound of the sound object, propagating along a straight line from the source to the listener.
- `indirectSoundAttenuation` is the XrSoundObjectDistanceAttenuationBD of the indirect sound of the sound object, which is any other propagation that is not considered "direct."

Applications **may** specify the directivity and/or the shape of a sound object:

- To describe the sound object's shape, applications chain XrSoundObjectShapeSphereBD to XrSoundObjectConfigBD::`next`
- To describe the directivity of a sound object by a cardioid polar pattern, applications chain XrSoundObjectDirectivityCardioidBD to XrSoundObjectConfigBD::`next`.

If the application does not chain XrSoundObjectShapeSphereBD, the sound object is defined as an infinitesimally small point. If the application does not change XrSoundObjectDirectivityCardioidBD, the sound object is defined to be omnidirectional (uniform sound emission intensity over all directions).

The behavior below specifies how the runtime renders audio when tracking quality changes for the `baseSpace`.

- While the `baseSpace` pose is valid, but not tracked (i.e. xrLocateSpace reports its location with `XR_SPACE_LOCATION_POSITION_VALID_BIT` and `XR_SPACE_LOCATION_ORIENTATION_VALID_BIT` set, but not `XR_SPACE_LOCATION_POSITION_TRACKED_BIT` and `XR_VIEW_STATE_POSITION_TRACKED_BIT`), the runtime **must** keep rendering the sound object using the most recent valid pose of that space.
- While the `baseSpace` pose is neither valid nor tracked (i.e. xrLocateSpace reports its location with `XR_SPACE_LOCATION_POSITION_VALID_BIT` and `XR_SPACE_LOCATION_ORIENTATION_VALID_BIT` unset), the runtime **should** keep rendering the sound object using the most recent valid pose of that space relative to its parent.

Valid Usage (Implicit)

- The XR_BD_spatial_audio_rendering extension **must** be enabled prior to using XrSoundObjectConfigBD
- type **must** be XR_TYPE_SOUND_OBJECT_CONFIG_BD
- next **must** be NULL or a valid pointer to the next structure in a structure chain. See also: XrSoundObjectDirectivityCardioidBD, XrSoundObjectShapeSphereBD
- baseSpace **must** be a valid XrSpace handle
- directSoundAttenuation **must** be a pointer to a valid XrSoundObjectDistanceAttenuationBD structure
- indirectSoundAttenuation **must** be a pointer to a valid XrSoundObjectDistanceAttenuationBD structure

The XrSoundObjectDistanceAttenuationBD structure is defined as:

```
// Provided by XR_BD_spatial_audio_rendering
typedef struct XrSoundObjectDistanceAttenuationBD {
    XrStructureType                              type;
    const void*                                  next;
    XrSoundObjectDistanceAttenuationTypeBD       distanceAttenuationType;
    float                                        minAttenuationRange;
    float                                        maxAttenuationRange;
    float                                        referenceDistance;
    float                                        rolloffFactor;
    XrSoundObjectDistanceAttenuationCurveBD*     customDistanceAttenuationCurve;
} XrSoundObjectDistanceAttenuationBD;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `distanceAttenuationType` is the type of this distance attenuation.
- `minAttenuationRange` is the minimum distance of attenuation in meters. Any distance closer than this value will not cause less attenuation.
- `maxAttenuationRange` is the maximum distance of attenuation in meters. Any distance further than this value will not cause more attenuation.
- `referenceDistance` is the propagation distance in which sound is neither attenuated nor gained. It is ignored if `distanceAttenuationType` is not `XR_SOUND_OBJECT_DISTANCE_ATTENUATION_TYPE_ROLLOFF_BD`.
- `rolloffFactor` is the intensity of distance attenuation, which **must** be larger or equal to 0. It is ignored if `distanceAttenuationType` is not `XR_SOUND_OBJECT_DISTANCE_ATTENUATION_TYPE_ROLLOFF_BD`.
- `customDistanceAttenuationCurve` is a pointer to an XrSoundObjectDistanceAttenuationCurveBD structure defining an attenuation curve for a customized distance attenuation. It is ignored if `distanceAttenuationType` is not `XR_SOUND_OBJECT_DISTANCE_ATTENUATION_TYPE_CUSTOMIZED_BD`

When `distanceAttenuationType` is `XR_SOUND_OBJECT_DISTANCE_ATTENUATION_TYPE_NONE_BD`, the runtime **must** not apply attenuation based on sound propagation distance, which means: attenuation(distance) = 1

When `distanceAttenuationType` is `XR_SOUND_OBJECT_DISTANCE_ATTENUATION_TYPE_INVERSE_SQUARE_BD`, the runtime **must** use the following formulas to calculate the distance attenuation:

- attenuation(d) = 1, $d < r_{min}$
- attenuation(d) = $1/\sqrt{(1 + (r_{max} - r_{min})^2)}$, $d > r_{max}$
- attenuation(d) = $1/\sqrt{(1 + (d - r_{min})^2)}$, otherwise

where:

- d is the propagation distance of sound.
- r_{min} is `minAttenuationRange`.
- r_{max} is `maxAttenuationRange`.

When `distanceAttenuationType` is `XR_SOUND_OBJECT_DISTANCE_ATTENUATION_TYPE_ROLLOFF_BD`, the runtime **must** use the following formulas to calculate the distance attenuation:

- attenuation(d) = $(r_{min}/d_{ref})^{\rho}$, $d < r_{min}$
- attenuation(d) = $(r_{max}/d_{ref})^{\rho}$, $d > r_{max}$
- attenuation(d) = $(d/d_{ref})^{\rho}$, otherwise

where:

- d is the propagation distance of sound.
- r_{min} is `minAttenuationRange`.
- r_{max} is `maxAttenuationRange`.
- d_{ref} is `referenceDistance`.
- ρ is `rolloffFactor`.

When `distanceAttenuationType` is `XR_SOUND_OBJECT_DISTANCE_ATTENUATION_TYPE_CUSTOMIZED_BD`, the application **must** set `customDistanceAttenuationCurve` to an XrSoundObjectDistanceAttenuationCurveBD structure defining the distance attenuation curve function to calculate the distance attenuation.

If `minAttenuationRange` is greater than `maxAttenuationRange`, the runtime **must** return `XR_ERROR_VALIDATION_FAILURE`.

Valid Usage (Implicit)

- The `XR_BD_spatial_audio_rendering` extension **must** be enabled prior to using XrSoundObjectDistanceAttenuationBD
- `type` **must** be `XR_TYPE_SOUND_OBJECT_DISTANCE_ATTENUATION_BD`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `distanceAttenuationType` **must** be a valid XrSoundObjectDistanceAttenuationTypeBD value
- If `customDistanceAttenuationCurve` is not `NULL`, `customDistanceAttenuationCurve` **must** be a pointer to an XrSoundObjectDistanceAttenuationCurveBD structure

The XrSoundObjectDistanceAttenuationTypeBD enumeration describes the type of distance attenuation.

```
// Provided by XR_BD_spatial_audio_rendering
typedef enum XrSoundObjectDistanceAttenuationTypeBD {
    XR_SOUND_OBJECT_DISTANCE_ATTENUATION_TYPE_NONE_BD = 0,
    XR_SOUND_OBJECT_DISTANCE_ATTENUATION_TYPE_INVERSE_SQUARE_BD = 1,
    XR_SOUND_OBJECT_DISTANCE_ATTENUATION_TYPE_ROLLOFF_BD = 2,
    XR_SOUND_OBJECT_DISTANCE_ATTENUATION_TYPE_CUSTOMIZED_BD = 100,
    XR_SOUND_OBJECT_DISTANCE_ATTENUATION_TYPE_MAX_ENUM_BD = 0x7FFFFFFF
} XrSoundObjectDistanceAttenuationTypeBD;
```

Enumerant Descriptions

- XR_SOUND_OBJECT_DISTANCE_ATTENUATION_TYPE_NONE_BD — Indicates that no attenuation is applied when source-listener distance varies.
- XR_SOUND_OBJECT_DISTANCE_ATTENUATION_TYPE_INVERSE_SQUARE_BD — Indicates that attenuation scales proportional to the inverse-squared distance of source-listener distance.
- XR_SOUND_OBJECT_DISTANCE_ATTENUATION_TYPE_ROLLOFF_BD — Indicates that attenuation equals pow(distance/refer_distance, -rolloff_factor), in which distance is clamped by minAttenuationRange and maxAttenuationRange.
- XR_SOUND_OBJECT_DISTANCE_ATTENUATION_TYPE_CUSTOMIZED_BD — Indicates that the distance attenuation is customized.

The XrSoundObjectDistanceAttenuationCurveBD structure is defined as:

```
// Provided by XR_BD_spatial_audio_rendering
typedef struct XrSoundObjectDistanceAttenuationCurveBD {
    XrStructureType               type;
    const void*                   next;
    uint32_t                      curvePointCount;
    XrAttenuationCurvePointBD*    curvePoints;
} XrSoundObjectDistanceAttenuationCurveBD;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain.
- curvePointCount is the number of points in the curve.
- curvePoints is a pointer to an array of XrAttenuationCurvePointBD.

If the propagation distance falls between two points in the curve, the runtime **must** calculate attenuation by linear interpolation.

If the propagation distance is less than the first point in the curve, the runtime **must** use the attenuation of the first point.

If the propagation distance is greater than the last point in the curve, the runtime **must** use the attenuation of the last point.

Valid Usage (Implicit)

- The XR_BD_spatial_audio_rendering extension **must** be enabled prior to using XrSoundObjectDistanceAttenuationCurveBD
- type **must** be XR_TYPE_SOUND_OBJECT_DISTANCE_ATTENUATION_CURVE_BD
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- curvePoints **must** be a pointer to an array of curvePointCount XrAttenuationCurvePointBD structures
- The curvePointCount parameter **must** be greater than 0

The XrAttenuationCurvePointBD structure is defined as:

```
// Provided by XR_BD_spatial_audio_rendering
typedef struct XrAttenuationCurvePointBD {
    float    distance;
    float    gain;
} XrAttenuationCurvePointBD;
```

Member Descriptions

- distance is the propagation distance of the sound wave.
- gain is the linear attenuation gain within the range of (0, 1), where 0 means complete attenuation (silence) and 1 means no attenuation.

Valid Usage (Implicit)

- The XR_BD_spatial_audio_rendering extension **must** be enabled prior to using XrAttenuationCurvePointBD

The XrSoundObjectShapeSphereBD structure is defined as:

```
// Provided by XR_BD_spatial_audio_rendering
typedef struct XrSoundObjectShapeSphereBD {
    XrStructureType    type;
    const void*        next;
    float              radius;
} XrSoundObjectShapeSphereBD;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `radius` is the radius of the sphere in meters.

To describe the sound object's shape as defined by a sphere, applications **may** chain XrSoundObjectShapeSphereBD to the XrSoundObjectConfigBD::`next` chain.

Valid Usage (Implicit)

- The `XR_BD_spatial_audio_rendering` extension **must** be enabled prior to using XrSoundObjectShapeSphereBD
- `type` **must** be `XR_TYPE_SOUND_OBJECT_SHAPE_SPHERE_BD`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

The XrSoundObjectDirectivityCardioidBD structure is defined as:

```
// Provided by XR_BD_spatial_audio_rendering
typedef struct XrSoundObjectDirectivityCardioidBD {
    XrStructureType    type;
    const void*        next;
    float              alpha;
    float              order;
} XrSoundObjectDirectivityCardioidBD;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `alpha` describes the shape of the cardioid polar pattern.
- `order` is the sharpness of the cardioid polar pattern.

To describe the directivity of a sound object by a cardioid polar pattern, applications **may** chain XrSoundObjectDirectivityCardioidBD to the XrSoundObjectConfigBD::`next` chain.

Valid Usage (Implicit)

- The `XR_BD_spatial_audio_rendering` extension **must** be enabled prior to using XrSoundObjectDirectivityCardioidBD
- `type` **must** be `XR_TYPE_SOUND_OBJECT_DIRECTIVITY_CARDIOID_BD`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

Update Sound Object Config

The xrUpdateSoundObjectConfigBD function is defined as:

```
// Provided by XR_BD_spatial_audio_rendering
XrResult xrUpdateSoundObjectConfigBD(
    XrSoundObjectBD                             soundObject,
    const XrSoundObjectConfigBD*                config,
    XrSoundObjectFlagsBD                        flags);
```

Parameter Descriptions

- `soundObject` is the XrSoundObjectBD
- `config` is the XrSoundObjectConfigBD that defines the configuration of the sound object.
- `flags` is the bit mask (bitwise `OR` of XrSoundObjectFlagsBD) of the sound object properties to change.

When updating the configuration of a submitted sound field, applications use `flags` to specify which properties to update. The runtime **must** not update a property if the corresponding flag bit is not set. The runtime **must** not read the parameters of XrSoundObjectConfigBD if the corresponding bits are not set, this means that the application **may** pass invalid or uninitialized fields for parameters it does not intend to update.

Valid Usage (Implicit)

- The `XR_BD_spatial_audio_rendering` extension **must** be enabled prior to calling xrUpdateSoundObjectConfigBD
- `soundObject` **must** be a valid XrSoundObjectBD handle
- `config` **must** be a pointer to a valid XrSoundObjectConfigBD structure
- `flags` **must** be a valid combination of XrSoundObjectFlagBitsBD values
- `flags` **must** not be `0`

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_POSE_INVALID
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

```
// Provided by XR_BD_spatial_audio_rendering
typedef XrFlags64 XrSoundObjectFlagsBD;
```

```
// Provided by XR_BD_spatial_audio_rendering
// Flag bits for XrSoundObjectFlagsBD
static const XrSoundObjectFlagsBD XR_SOUND_OBJECT_ENABLED_BIT_BD = 0x00000001;
static const XrSoundObjectFlagsBD XR_SOUND_OBJECT_POSE_BIT_BD = 0x00000002;
static const XrSoundObjectFlagsBD XR_SOUND_OBJECT_DIRECTIVITY_BIT_BD = 0x00000004;
static const XrSoundObjectFlagsBD XR_SOUND_OBJECT_SHAPE_BIT_BD = 0x00000008;
static const XrSoundObjectFlagsBD XR_SOUND_OBJECT_MAIN_VOLUME_BIT_BD = 0x00000010;
static const XrSoundObjectFlagsBD XR_SOUND_OBJECT_REFLECTION_GAIN_BIT_BD = 0x00000020;
static const XrSoundObjectFlagsBD XR_SOUND_OBJECT_ENABLE_DOPPLER_BIT_BD = 0x00000040;
static const XrSoundObjectFlagsBD XR_SOUND_OBJECT_DIRECT_SOUND_ATTENUATION_BIT_BD =
0x00000080;
static const XrSoundObjectFlagsBD XR_SOUND_OBJECT_INDIRECT_SOUND_ATTENUATION_BIT_BD =
0x00000100;
```

The flag bits have the following meanings:

Flag Descriptions

- XR_SOUND_OBJECT_ENABLED_BIT_BD — to indicate an update of XrSoundObjectConfigBD::enabled
- XR_SOUND_OBJECT_POSE_BIT_BD — to indicate an update of XrSoundObjectConfigBD::pose
- XR_SOUND_OBJECT_DIRECTIVITY_BIT_BD — to indicate an update of the directivity using an XrSoundObjectDirectivityCardioidBD chained to XrSoundObjectConfigBD::next
- XR_SOUND_OBJECT_SHAPE_BIT_BD — to indicate an update of the shape using an XrSoundObjectShapeSphereBD chained to XrSoundObjectConfigBD::next
- XR_SOUND_OBJECT_MAIN_VOLUME_BIT_BD — to indicate an update of XrSoundObjectConfigBD::mainVolume
- XR_SOUND_OBJECT_REFLECTION_GAIN_BIT_BD — to indicate an update of XrSoundObjectConfigBD::reflectionGain
- XR_SOUND_OBJECT_ENABLE_DOPPLER_BIT_BD — to indicate an update of XrSoundObjectConfigBD::enableDoppler
- XR_SOUND_OBJECT_DIRECT_SOUND_ATTENUATION_BIT_BD — to indicate an update of XrSoundObjectConfigBD::directSoundAttenuation
- XR_SOUND_OBJECT_INDIRECT_SOUND_ATTENUATION_BIT_BD — to indicate an update of XrSoundObjectConfigBD::indirectSoundAttenuation

Destroy Sound Object

The xrDestroySoundObjectBD function is defined as:

```
// Provided by XR_BD_spatial_audio_rendering
XrResult xrDestroySoundObjectBD(
    XrSoundObjectBD                             soundObject);
```

Parameter Descriptions

- soundObject is the XrSoundObjectBD to be destroyed.

When an application removes a sound object with xrDestroySoundObjectBD, its corresponding sound is not rendered anymore.

Valid Usage (Implicit)

- The XR_BD_spatial_audio_rendering extension **must** be enabled prior to calling xrDestroySoundObjectBD
- `soundObject` **must** be a valid XrSoundObjectBD handle

Thread Safety

- Access to `soundObject`, and any child handles, **must** be externally synchronized

Return Codes

Success

- XR_SUCCESS

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_RUNTIME_FAILURE

Add Sound Obstacles to the Scene

Applications **may** create sound obstacles in the scene that simulate real-world or virtual objects that reflect or diffract sound, such as doors, walls, carpets, etc.

```
// Provided by XR_BD_spatial_audio_rendering
XR_DEFINE_HANDLE(XrSoundObstacleMaterialBD)
```

The XrSoundObstacleMaterialBD handle represents a sound obstacle material.

The xrCreateSoundObstacleMaterialBD function is defined as:

```
// Provided by XR_BD_spatial_audio_rendering
XrResult xrCreateSoundObstacleMaterialBD(
    XrSpatialAudioRendererBD                renderer,
    const XrSoundObstacleMaterialConfigBD*  config,
    XrSoundObstacleMaterialBD*              material);
```

Parameter Descriptions

- `renderer` is the XrSpatialAudioRendererBD handle that the sound obstacle material is created for.
- `config` is the XrSoundObstacleMaterialBD that defines the configuration of the sound obstacle material.
- `material` is a pointer to XrSoundObstacleMaterialBD that xrCreateSoundObstacleMaterialBD returned.

A valid XrSoundObstacleMaterialBD handle **can** be used to configure the material properties of the sound obstacle when creating or updating a sound obstacle.

Note that a successful creation of an XrSoundObstacleMaterialBD on its own does not affect the virtual sound propagation, until the created XrSoundObstacleMaterialBD is used to create or update sound obstacles using xrCreateSoundObstacleBD or xrUpdateSoundObstacleConfigBD.

Valid Usage (Implicit)

- The `XR_BD_spatial_audio_rendering` extension **must** be enabled prior to calling xrCreateSoundObstacleMaterialBD
- `renderer` **must** be a valid XrSpatialAudioRendererBD handle
- `config` **must** be a pointer to a valid XrSoundObstacleMaterialConfigBD structure
- `material` **must** be a pointer to an XrSoundObstacleMaterialBD handle

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The XrSoundObstacleMaterialConfigBD structure is defined as:

```
// Provided by XR_BD_spatial_audio_rendering
typedef struct XrSoundObstacleMaterialConfigBD {
    XrStructureType                 type;
    const void*                     next;
    XrSoundObstacleMaterialTypeBD   materialType;
    uint32_t                        bandCount;
    float*                          bandFrequencies;
    float*                          bandAbsorptions;
    float*                          bandScatterings;
    float*                          bandTransmissions;
} XrSoundObstacleMaterialConfigBD;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `materialType` is the XrSoundObstacleMaterialTypeBD of sound obstacle.
- `bandCount` is the number of bands for which frequencies, absorptions, scatterings, and transmissions are defined below. This field and its associated arrays in subsequent fields are ignored if `materialType` is not `XR_SOUND_OBSTACLE_MATERIAL_TYPE_CUSTOM_BD`.
- `bandFrequencies` is an array of `bandCount` frequency centers of the frequency bands for custom materials.
- `bandAbsorptions` is an array of `bandCount` absorption values: the ratio of absorbed sound energy of the sound obstacle when a reflection happens, in the range (0, 1) for custom materials.
- `bandScatterings` is an array of `bandCount` scattering values: the ratio of scattered sound energy of the sound obstacle when a reflection happens, in the range (0, 1) for custom materials.
- `bandTransmissions` is an array of `bandCount` transmission values: the ratio of transmitted sound energy of the sound obstacle when a reflection happens, in the range (0, 1) for custom materials.

Applications use XrSoundObstacleMaterialConfigBD to configure the material properties of an XrSoundObstacleMaterialBD.

Valid Usage (Implicit)

- The `XR_BD_spatial_audio_rendering` extension **must** be enabled prior to using XrSoundObstacleMaterialConfigBD
- `type` **must** be `XR_TYPE_SOUND_OBSTACLE_MATERIAL_CONFIG_BD`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `materialType` **must** be a valid XrSoundObstacleMaterialTypeBD value
- `bandFrequencies` **must** be a pointer to an array of `bandCount` `float` values
- `bandAbsorptions` **must** be a pointer to an array of `bandCount` `float` values
- `bandScatterings` **must** be a pointer to an array of `bandCount` `float` values
- `bandTransmissions` **must** be a pointer to an array of `bandCount` `float` values
- The `bandCount` parameter **must** be greater than `0`

The XrSoundObstacleMaterialTypeBD enumeration describes the sound obstacle material type.

```
// Provided by XR_BD_spatial_audio_rendering
typedef enum XrSoundObstacleMaterialTypeBD {
    XR_SOUND_OBSTACLE_MATERIAL_TYPE_ACOUSTIC_TILE_BD = 0,
    XR_SOUND_OBSTACLE_MATERIAL_TYPE_BRICK_BD = 1,
    XR_SOUND_OBSTACLE_MATERIAL_TYPE_BRICK_PAINTED_BD = 2,
    XR_SOUND_OBSTACLE_MATERIAL_TYPE_CARPET_BD = 3,
    XR_SOUND_OBSTACLE_MATERIAL_TYPE_CARPET_HEAVY_BD = 4,
    XR_SOUND_OBSTACLE_MATERIAL_TYPE_CARPET_HEAVY_PADDED_BD = 5,
    XR_SOUND_OBSTACLE_MATERIAL_TYPE_CERAMIC_TILE_BD = 6,
    XR_SOUND_OBSTACLE_MATERIAL_TYPE_CONCRETE_BD = 7,
    XR_SOUND_OBSTACLE_MATERIAL_TYPE_CONCRETE_ROUGH_BD = 8,
    XR_SOUND_OBSTACLE_MATERIAL_TYPE_CONCRETE_BLOCK_BD = 9,
    XR_SOUND_OBSTACLE_MATERIAL_TYPE_CONCRETE_BLOCK_PAINTED_BD = 10,
    XR_SOUND_OBSTACLE_MATERIAL_TYPE_CURTAIN_BD = 11,
    XR_SOUND_OBSTACLE_MATERIAL_TYPE_FOLIAGE_BD = 12,
    XR_SOUND_OBSTACLE_MATERIAL_TYPE_GLASS_BD = 13,
    XR_SOUND_OBSTACLE_MATERIAL_TYPE_GLASS_HEAVY_BD = 14,
    XR_SOUND_OBSTACLE_MATERIAL_TYPE_GRASS_BD = 15,
    XR_SOUND_OBSTACLE_MATERIAL_TYPE_GRAVEL_BD = 16,
    XR_SOUND_OBSTACLE_MATERIAL_TYPE_GYPSUM_BOARD_BD = 17,
    XR_SOUND_OBSTACLE_MATERIAL_TYPE_PLASTER_ON_BRICK_BD = 18,
    XR_SOUND_OBSTACLE_MATERIAL_TYPE_PLASTER_ON_CONCRETE_BLOCK_BD = 19,
    XR_SOUND_OBSTACLE_MATERIAL_TYPE_SOIL_BD = 20,
    XR_SOUND_OBSTACLE_MATERIAL_TYPE_SOUND_PROOF_BD = 21,
    XR_SOUND_OBSTACLE_MATERIAL_TYPE_SNOW_BD = 22,
    XR_SOUND_OBSTACLE_MATERIAL_TYPE_STEEL_BD = 23,
    XR_SOUND_OBSTACLE_MATERIAL_TYPE_WATER_BD = 24,
    XR_SOUND_OBSTACLE_MATERIAL_TYPE_WOOD_THIN_BD = 25,
    XR_SOUND_OBSTACLE_MATERIAL_TYPE_WOOD_THICK_BD = 26,
    XR_SOUND_OBSTACLE_MATERIAL_TYPE_WOOD_FLOOR_BD = 27,
    XR_SOUND_OBSTACLE_MATERIAL_TYPE_WOOD_ON_CONCRETE_BD = 28,
    XR_SOUND_OBSTACLE_MATERIAL_TYPE_CUSTOM_BD = 29,
    XR_SOUND_OBSTACLE_MATERIAL_TYPE_MAX_ENUM_BD = 0x7FFFFFFF
} XrSoundObstacleMaterialTypeBD;
```

Enum	Description
XR_SOUND_OBSTACLE_MATERIAL_TYPE_ACOUSTIC_TILE_BD	indicates that the material is acoustic tile
XR_SOUND_OBSTACLE_MATERIAL_TYPE_BRICK_BD	indicates that the material resembles brick
XR_SOUND_OBSTACLE_MATERIAL_TYPE_BRICK_PAINTED_BD	indicates that the material resembles painted brick
XR_SOUND_OBSTACLE_MATERIAL_TYPE_CARPET_BD	indicates that the material resembles carpet
XR_SOUND_OBSTACLE_MATERIAL_TYPE_CARPET_HEAVY_BD	indicates that the material resembles heavy carpet

Enum	Description
XR_SOUND_OBSTACLE_MATERIAL_TYPE_CARPET_HEAVY_PADDED_BD	indicates that the material resembles heavily padded carpet
XR_SOUND_OBSTACLE_MATERIAL_TYPE_CERAMIC_TILE_BD	indicates that the material resembles ceramic tile
XR_SOUND_OBSTACLE_MATERIAL_TYPE_CONCRETE_BD	indicates that the material resembles concrete
XR_SOUND_OBSTACLE_MATERIAL_TYPE_CONCRETE_ROUGH_BD	indicates that the material resembles rough concrete
XR_SOUND_OBSTACLE_MATERIAL_TYPE_CONCRETE_BLOCK_BD	indicates that the material resembles concrete block
XR_SOUND_OBSTACLE_MATERIAL_TYPE_CONCRETE_BLOCK_PAINTED_BD	indicates that the material resembles painted concrete block
XR_SOUND_OBSTACLE_MATERIAL_TYPE_CURTAIN_BD	indicates that the material resembles curtain
XR_SOUND_OBSTACLE_MATERIAL_TYPE_FOLIAGE_BD	indicates that the material resembles foliage
XR_SOUND_OBSTACLE_MATERIAL_TYPE_GLASS_BD	indicates that the material resembles glass
XR_SOUND_OBSTACLE_MATERIAL_TYPE_GLASS_HEAVY_BD	indicates that the material resembles heavy glass
XR_SOUND_OBSTACLE_MATERIAL_TYPE_GRASS_BD	indicates that the material resembles grass
XR_SOUND_OBSTACLE_MATERIAL_TYPE_GRAVEL_BD	indicates that the material resembles gravel
XR_SOUND_OBSTACLE_MATERIAL_TYPE_GYPSUM_BOARD_BD	indicates that the material resembles gypsum board
XR_SOUND_OBSTACLE_MATERIAL_TYPE_PLASTER_ON_BRICK_BD	indicates that the material resembles plaster on brick
XR_SOUND_OBSTACLE_MATERIAL_TYPE_PLASTER_ON_CONCRETE_BLOCK_BD	indicates that the material resembles plaster on concrete block
XR_SOUND_OBSTACLE_MATERIAL_TYPE_SOIL_BD	indicates that the material resembles soil
XR_SOUND_OBSTACLE_MATERIAL_TYPE_SOUND_PROOF_BD	indicates that the material resembles sound proof
XR_SOUND_OBSTACLE_MATERIAL_TYPE_SNOW_BD	indicates that the material resembles snow
XR_SOUND_OBSTACLE_MATERIAL_TYPE_STEEL_BD	indicates that the material resembles steel
XR_SOUND_OBSTACLE_MATERIAL_TYPE_WATER_BD	indicates that the material resembles water
XR_SOUND_OBSTACLE_MATERIAL_TYPE_WOOD_THIN_BD	indicates that the material resembles thin wood
XR_SOUND_OBSTACLE_MATERIAL_TYPE_WOOD_THICK_BD	indicates that the material resembles thick wood
XR_SOUND_OBSTACLE_MATERIAL_TYPE_WOOD_FLOOR_BD	indicates that the material resembles floor
XR_SOUND_OBSTACLE_MATERIAL_TYPE_WOOD_ON_CONCRETE_BD	indicates that the material resembles wood on concrete

Enum	Description
XR_SOUND_OBSTACLE_MATERIAL_TYPE_CUSTOM_BD	Customized material described by detailed acoustic surface parameters, for example, absorption, scattering, and transmission.

The xrUpdateSoundObstacleMaterialConfigBD function is defined as:

```
// Provided by XR_BD_spatial_audio_rendering
XrResult xrUpdateSoundObstacleMaterialConfigBD(
    XrSoundObstacleMaterialBD                   material,
    const XrSoundObstacleMaterialConfigBD*      config);
```

Parameter Descriptions

- `material` is the XrSoundObstacleMaterialBD to be updated.
- `config` is the XrSoundObstacleMaterialBD that defines the configuration of the sound obstacle material.

This function **can** be used to update the configuration of a created XrSoundObstacleMaterialBD.

Note that a successful update of an XrSoundObstacleMaterialBD on its own does not affect the virtual sound propagation, until the updated XrSoundObstacleMaterialBD is used to create or update sound obstacles using xrCreateSoundObstacleBD or xrUpdateSoundObstacleConfigBD. This is true even if XrSoundObstacleMaterialBD was referenced by an active sound obstacle.

Valid Usage (Implicit)

- The `XR_BD_spatial_audio_rendering` extension **must** be enabled prior to calling xrUpdateSoundObstacleMaterialConfigBD
- `material` **must** be a valid XrSoundObstacleMaterialBD handle
- `config` **must** be a pointer to a valid XrSoundObstacleMaterialConfigBD structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The xrDestroySoundObstacleMaterialBD function is defined as:

```
// Provided by XR_BD_spatial_audio_rendering
XrResult xrDestroySoundObstacleMaterialBD(
    XrSoundObstacleMaterialBD                   material);
```

Parameter Descriptions

- material is the XrSoundObstacleMaterialBD to be destroyed.

This function **can** be used to destroy an existing XrSoundObstacleMaterialBD.

Note that destroying an XrSoundObstacleMaterialBD does not affect the virtual sound propagation, even if it was referenced by an active sound obstacle.

Valid Usage (Implicit)

- The XR_BD_spatial_audio_rendering extension **must** be enabled prior to calling xrDestroySoundObstacleMaterialBD
- material **must** be a valid XrSoundObstacleMaterialBD handle

Thread Safety

- Access to `material`, and any child handles, **must** be externally synchronized

Return Codes

Success

- `XR_SUCCESS`

Failure

- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_RUNTIME_FAILURE`

```
// Provided by XR_BD_spatial_audio_rendering
XR_DEFINE_HANDLE(XrSoundObstacleBD)
```

The XrSoundObstacleBD handle represents a sound obstacle.

Sound obstacles in the environment are represented by XrSoundObstacleBD handles, and each XrSoundObstacleBD has material properties affecting sound reflection and diffraction characteristics.

The runtime **must** calculate and render sound according to all the sound fields, sound objects, and sound obstacles in the scene.

The xrCreateSoundObstacleBD function is defined as:

```
// Provided by XR_BD_spatial_audio_rendering
XrResult xrCreateSoundObstacleBD(
    XrSpatialAudioRendererBD                    renderer,
    const XrSoundObstacleConfigBD*              config,
    const XrSoundTriangleMeshBD*                mesh,
    XrSoundObstacleBD*                          soundObstacle);
```

Parameter Descriptions

- `renderer` is the XrSpatialAudioRendererBD used for spatial audio rendering.
- `config` is the XrSoundObstacleConfigBD that defines the configuration of the sound obstacle, including pose and material information.
- `mesh` is the XrSoundTriangleMeshBD that defines the shape of the sound obstacle.
- `soundObstacle` is a pointer to XrSoundObstacleBD that xrCreateSoundObstacleBD returned.

When an application creates a sound obstacle in the spatial audio renderer, virtual sound propagation is influenced by this acoustic mesh.

Valid Usage (Implicit)

- The `XR_BD_spatial_audio_rendering` extension **must** be enabled prior to calling xrCreateSoundObstacleBD
- `renderer` **must** be a valid XrSpatialAudioRendererBD handle
- `config` **must** be a pointer to a valid XrSoundObstacleConfigBD structure
- `mesh` **must** be a pointer to a valid XrSoundTriangleMeshBD structure
- `soundObstacle` **must** be a pointer to an XrSoundObstacleBD handle

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_POSE_INVALID
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The XrSoundObstacleConfigBD structure is defined as:

```
// Provided by XR_BD_spatial_audio_rendering
typedef struct XrSoundObstacleConfigBD {
    XrStructureType               type;
    const void*                   next;
    XrBool32                      enabled;
    XrPosef                       pose;
    XrSpace                       baseSpace;
    uint32_t                      materialCount;
    XrSoundObstacleMaterialBD*    materials;
} XrSoundObstacleConfigBD;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `enabled` is an `XrBool32`, indicating whether the sound obstacle is enabled.
- `pose` is an XrPosef of the acoustic mesh.
- `baseSpace` is an XrSpace which the `pose` is defined in.
- `materialCount` is the number of materials in the `materials` array.
- `materials` is an array of `materialCount` XrSoundObstacleMaterialBD that defines the material properties for each vertex of the sound obstacle.

Applications use XrSoundObstacleConfigBD to configure an XrSoundObstacleBD.

If this config is used to create an XrSoundObstacleBD, `materialCount` **must** be equal to the associated XrSoundTriangleMeshBD::`vertexCount`, otherwise the runtime **must** return `XR_ERROR_VALIDATION_FAILURE`.

If this config is used to update both shape and materials of an existing XrSoundObstacleBD, `materialCount` **must** be equal to the associated XrSoundTriangleMeshBD::`vertexCount`, otherwise the runtime **must** return `XR_ERROR_VALIDATION_FAILURE`.

If this config is used to update only materials of an existing XrSoundObstacleBD, `materialCount` **must** be equal to the vertices count of this XrSoundTriangleMeshBD, otherwise the runtime **must** return `XR_ERROR_VALIDATION_FAILURE`.

The runtime **must** not use the sound obstacle for sound calculations if `enabled` is set to `XR_FALSE`.

Valid Usage (Implicit)

- The `XR_BD_spatial_audio_rendering` extension **must** be enabled prior to using XrSoundObstacleConfigBD
- `type` **must** be `XR_TYPE_SOUND_OBSTACLE_CONFIG_BD`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `baseSpace` **must** be a valid XrSpace handle
- `materials` **must** be a pointer to an array of `materialCount` XrSoundObstacleMaterialBD handles
- The `materialCount` parameter **must** be greater than `0`

The XrSoundTriangleMeshBD structure is described as:

```
// Provided by XR_BD_spatial_audio_rendering
typedef struct XrSoundTriangleMeshBD {
    XrStructureType    type;
    const void*        next;
    uint32_t           vertexCount;
    XrVector3f*        vertices;
    uint32_t           indexCount;
    uint32_t*          indices;
} XrSoundTriangleMeshBD;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `vertexCount` is the number of vertices in the triangle mesh.
- `vertices` is a pointer to an array of XrVector3f vertices.
- `indexCount` is the number of indices in the triangle mesh.
- `indices` is a pointer to an array of indices that defines the triangles.

The XrSoundTriangleMeshBD structure describes the shape of a sound obstacle. The `vertices` are in counter-clockwise order as viewed from the user perspective. Only the front faces are considered when rendering the sound. The `indices` array defines the topology of the triangle mesh. Each triplet of consecutive elements points to three vertices in the vertices array and thus form a triangle.

Valid Usage (Implicit)

- The `XR_BD_spatial_audio_rendering` extension **must** be enabled prior to using XrSoundTriangleMeshBD
- `type` **must** be `XR_TYPE_SOUND_TRIANGLE_MESH_BD`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `vertices` **must** be a pointer to an array of `vertexCount` XrVector3f structures
- `indices` **must** be a pointer to an array of `indexCount` `uint32_t` values
- The `vertexCount` parameter **must** be greater than `0`
- The `indexCount` parameter **must** be greater than `0`

Update Sound Obstacle Config

The xrUpdateSoundObstacleConfigBD function is defined as:

```
// Provided by XR_BD_spatial_audio_rendering
XrResult xrUpdateSoundObstacleConfigBD(
    XrSoundObstacleBD                           soundObstacle,
    const XrSoundObstacleConfigBD*              config,
    const XrSoundTriangleMeshBD*                mesh,
    XrSoundObstacleFlagsBD                      flags);
```

Parameter Descriptions

- `soundObstacle` is the XrSoundObstacleBD to be updated.
- `config` is the XrSoundObstacleConfigBD that defines the configuration of the sound obstacle, including pose and material information.
- `mesh` is the XrSoundTriangleMeshBD that describes the shape of the sound obstacle.
- `flags` is the bit mask (bitwise `OR` of XrSoundObstacleFlagsBD) of the sound obstacle properties to change, including pose and material information.

When updating the configuration of a submitted sound obstacle, applications use `flags` to specify which properties to update. The runtime **must** not update a property if the corresponding flag bit is not set. The runtime **must** not read the parameters of XrSoundObstacleConfigBD if the corresponding bits are not set; this means that the application **may** pass invalid or uninitialized fields for parameters it does not intend to update.

If `XR_SOUND_OBSTACLE_MESH_BIT_BD` is set in `flags`, `mesh` **must** not be `NULL`, otherwise, the runtime **must** return `XR_ERROR_VALIDATION_FAILURE`.

If any other bit is set, `config` **must** not be `NULL`, otherwise, the runtime **must** return `XR_ERROR_VALIDATION_FAILURE`.

Valid Usage (Implicit)

- The XR_BD_spatial_audio_rendering extension **must** be enabled prior to calling xrUpdateSoundObstacleConfigBD
- soundObstacle **must** be a valid XrSoundObstacleBD handle
- If config is not NULL, config **must** be a pointer to a valid XrSoundObstacleConfigBD structure
- If mesh is not NULL, mesh **must** be a pointer to a valid XrSoundTriangleMeshBD structure
- flags **must** be a valid combination of XrSoundObstacleFlagBitsBD values
- flags **must** not be 0

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_POSE_INVALID
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

```
// Provided by XR_BD_spatial_audio_rendering
typedef XrFlags64 XrSoundObstacleFlagsBD;
```

```
// Provided by XR_BD_spatial_audio_rendering
// Flag bits for XrSoundObstacleFlagsBD
static const XrSoundObstacleFlagsBD XR_SOUND_OBSTACLE_ENABLED_BIT_BD = 0x00000001;
static const XrSoundObstacleFlagsBD XR_SOUND_OBSTACLE_POSE_BIT_BD = 0x00000002;
static const XrSoundObstacleFlagsBD XR_SOUND_OBSTACLE_MESH_BIT_BD = 0x00000004;
static const XrSoundObstacleFlagsBD XR_SOUND_OBSTACLE_MATERIALS_BIT_BD = 0x00000008;
```

The flag bits have the following meanings:

Flag Descriptions

- XR_SOUND_OBSTACLE_ENABLED_BIT_BD — to indicate an update of XrSoundObstacleConfigBD::enabled
- XR_SOUND_OBSTACLE_POSE_BIT_BD — to indicate an update of XrSoundObstacleConfigBD::pose
- XR_SOUND_OBSTACLE_MESH_BIT_BD — to indicate an update of XrSoundObstacleConfigBD::mesh
- XR_SOUND_OBSTACLE_MATERIALS_BIT_BD — to indicate an update of XrSoundObstacleConfigBD::materials

The xrDestroySoundObstacleBD function is defined as:

```
// Provided by XR_BD_spatial_audio_rendering
XrResult xrDestroySoundObstacleBD(
    XrSoundObstacleBD                           soundObstacle);
```

Parameter Descriptions

- soundObstacle is the XrSoundObstacleBD to be removed.

Applications use this function to remove a submitted sound obstacle.

To temporarily disable an obstacle, set the respective XrSoundObstacleConfigBD::enabled to XR_FALSE instead with a call to xrUpdateSoundObstacleConfigBD.

Valid Usage (Implicit)

- The XR_BD_spatial_audio_rendering extension **must** be enabled prior to calling xrDestroySoundObstacleBD
- soundObstacle **must** be a valid XrSoundObstacleBD handle

Thread Safety

- Access to soundObstacle, and any child handles, **must** be externally synchronized

Return Codes

Success

- `XR_SUCCESS`

Failure

- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_RUNTIME_FAILURE`

12.88.5. Submit Audio Buffer

Applications **should** submit an audio buffer for each sound object and sound field in each audio period.

Because xrWaitAudioPeriodBD is a blocking function, applications typically submit the audio buffer within a dedicated audio thread.

Audio Period Synchronization

An audio period refers to a continuous block of time during which the application provides new input audio data and the runtime processes it for spatial rendering. Each audio period corresponds to one buffer of audio frames defined by XrSpatialAudioRendererCreateInfoBD::`framesPerBuffer`.

When an application waits for the next audio period using xrWaitAudioPeriodBD, it is effectively synchronizing with the runtime's internal audio rendering cycle. xrWaitAudioPeriodBD blocks its calling thread until the next audio period begins, or the specified timeout expires.

During each audio period:

- The application waits for the runtime to request new data by calling xrWaitAudioPeriodBD.
- The application submits new audio data for all sound fields and sound objects by calling xrSubmitSoundFieldBufferBD and/or xrSubmitSoundObjectBufferBD.
- After all submissions are completed, the application signals the end of submissions for this period with xrEndAudioPeriodBD.

This mechanism ensures that input buffers are consumed at the correct temporal cadence, matching the renderer's internal processing rate and preventing underruns or latency accumulation.

Thus, an application **must** follow the following call order to submit the audio buffers during each audio period:

1. Call xrWaitAudioPeriodBD to wait for the next audio period to begin.

2. Call xrSubmitSoundObjectBufferBD and/or xrSubmitSoundFieldBufferBD to submit audio buffers for sound objects and sound fields, if it submits buffers in this audio period.
3. Call xrEndAudioPeriodBD to notify the runtime of the end of buffer submissions for the current audio period.

If the application violates this call order, the runtime **must** return XR_ERROR_CALL_ORDER_INVALID for out-of-order calls.

The xrWaitAudioPeriodBD function is defined as:

```
// Provided by XR_BD_spatial_audio_rendering
XrResult xrWaitAudioPeriodBD(
    XrSpatialAudioRendererBD                    renderer,
    XrDuration                                  timeout);
```

Parameter Descriptions

- renderer is the XrSpatialAudioRendererBD which notifies the beginning of the next audio period.
- timeout is an XrDuration that specifies the maximum duration to wait for the next audio period in nanoseconds.

This function blocks the calling thread until the next audio period begins or the specified timeout expires.

The runtime **must** only unblock the calling thread after the previously submitted audio buffers are processed and before the previous output audio is done playing, so that output audio is gapless.

The runtime **should** unblock the calling thread as soon as possible after the above criteria is met.

If a previous call to xrEndAudioPeriodBD has not been successful, this function **must** return XR_ERROR_CALL_ORDER_INVALID.

timeout **must** be greater than zero, otherwise the runtime **must** return XR_ERROR_VALIDATION_FAILURE.

This function **must** return XR_TIMEOUT_EXPIRED if the specified timeout expires before the next audio period begins.

Calling this function **must** be externally synchronized by the application, concurrent calls **may** result in undefined behavior.

Valid Usage (Implicit)

- The XR_BD_spatial_audio_rendering extension **must** be enabled prior to calling xrWaitAudioPeriodBD
- renderer **must** be a valid XrSpatialAudioRendererBD handle

Thread Safety

- Access to the renderer parameter by any other xrWaitAudioPeriodBD or xrEndAudioPeriodBD call, soundField parameter by any other xrSubmitSoundFieldBufferBD call (if the soundField is a child of the renderer), and soundObject parameter by any other xrSubmitSoundObjectBufferBD call (if the soundObject is a child of the renderer) **must** be externally synchronized

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING
- XR_TIMEOUT_EXPIRED

Failure

- XR_ERROR_CALL_ORDER_INVALID
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST

The xrEndAudioPeriodBD function is defined as:

```
// Provided by XR_BD_spatial_audio_rendering
XrResult xrEndAudioPeriodBD(
    XrSpatialAudioRendererBD                    renderer);
```

Parameter Descriptions

- `renderer` is the XrSpatialAudioRendererBD to be notified with the end of the current audio period.

This function notifies the runtime of the end of the current audio period, which means that the application finished submitting all audio buffers for the current audio period and the runtime **must** start further processing them for spatial rendering.

This function **must** only be called after a successful and not timed out call to xrWaitAudioPeriodBD, otherwise the runtime **must** return `XR_ERROR_CALL_ORDER_INVALID`.

Calling this function **must** be externally synchronized by the application, concurrent calls **may** result in undefined behavior.

Valid Usage (Implicit)

- The `XR_BD_spatial_audio_rendering` extension **must** be enabled prior to calling xrEndAudioPeriodBD
- `renderer` **must** be a valid XrSpatialAudioRendererBD handle

Thread Safety

- Access to the `renderer` parameter by any other xrWaitAudioPeriodBD or xrEndAudioPeriodBD call, `soundField` parameter by any other xrSubmitSoundFieldBufferBD call (if the `soundField` is a child of the `renderer`), and `soundObject` parameter by any other xrSubmitSoundObjectBufferBD call (if the `soundObject` is a child of the `renderer`) **must** be externally synchronized

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_CALL_ORDER_INVALID
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST

Audio Buffer Submission

Within each audio period, applications **should** submit the audio buffer for each sound object and sound field.

```
// Provided by XR_BD_spatial_audio_rendering
typedef struct XrAudioBufferBD {
    XrStructureType                 type;
    const void*                     next;
    XrAudioBufferChannelLayoutBD    channelLayout;
    uint32_t                        bufferChannels;
    uint32_t                        bufferLength;
    float*                          buffer;
} XrAudioBufferBD;
```

Parameter Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `channelLayout` is the XrAudioBufferChannelLayoutBD that specifies the memory layout of the audio data (for example interleaved or planar).
- `bufferChannels` is the number of channels in the audio `buffer`.
- `bufferLength` is the number of floating-point elements in `buffer`. It **must** be equal to the product of `bufferChannels` and the XrSpatialAudioRendererCreateInfoBD::`framesPerBuffer` used when creating the spatial audio renderer.
- `buffer` is a pointer to a floating-point array containing the audio data.

The audio data in `buffer` is stored in little-endian byte order.

`bufferLength` and `bufferChannels` **must** be compatible with the values specified in XrSpatialAudioRendererCreateInfoBD when creating the renderer. In particular, the application **must** set `bufferLength` to `bufferChannels` * XrSpatialAudioRendererCreateInfoBD::`framesPerBuffer`, otherwise the runtime **must** return `XR_ERROR_VALIDATION_FAILURE`.

The application **must** ensure that the audio signal stored in this XrAudioBufferBD is sampled at a sample rate of XrSpatialAudioRendererCreateInfoBD::`sampleRate`. Supplying data from a mismatched sample rate results in unspecified behavior, commonly perceived as tempo/pitch shifts and spectral coloration.

Valid Usage (Implicit)

- The `XR_BD_spatial_audio_rendering` extension **must** be enabled prior to using XrAudioBufferBD
- `type` **must** be `XR_TYPE_AUDIO_BUFFER_BD`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `channelLayout` **must** be a valid XrAudioBufferChannelLayoutBD value
- `buffer` **must** be a pointer to an array of `bufferLength` `float` values
- The `bufferLength` parameter **must** be greater than `0`

The XrAudioBufferChannelLayoutBD enumeration describes the channel layout of audio buffers.

```
// Provided by XR_BD_spatial_audio_rendering
typedef enum XrAudioBufferChannelLayoutBD {
    XR_AUDIO_BUFFER_CHANNEL_LAYOUT_INTERLEAVED_BD = 0,
    XR_AUDIO_BUFFER_CHANNEL_LAYOUT_PLANAR_BD = 1,
    XR_AUDIO_BUFFER_CHANNEL_LAYOUT_MAX_ENUM_BD = 0x7FFFFFFF
} XrAudioBufferChannelLayoutBD;
```

Enum	Description
XR_AUDIO_BUFFER_CHANNEL_LAYOUT_INTERLEAVED_BD	indicates that audio samples from all channels are stored in an interleaved layout, where successive frames contain one sample per channel (e.g. LRLRLR for stereo)
XR_AUDIO_BUFFER_CHANNEL_LAYOUT_PLANAR_BD	indicates that audio samples are stored in a planar layout, where all samples for one channel are contiguous in memory (e.g. LLL...RRR... for stereo)

The xrSubmitSoundFieldBufferBD function is defined as:

```
// Provided by XR_BD_spatial_audio_rendering
XrResult xrSubmitSoundFieldBufferBD(
    XrSoundFieldBD                              soundField,
    const XrAudioBufferBD*                      buffer);
```

Parameter Descriptions

- soundField is the XrSoundFieldBD to input audio signal.
- buffer is a pointer to an XrAudioBufferBD structure that contains the input audio signal.

This function submits an audio buffer to a sound field XrSoundFieldBD. The corresponding function for a sound object XrSoundObjectBD is xrSubmitSoundObjectBufferBD.

This function **must** only be called during an audio period, which is the time period after a successful and not timed out call to xrWaitAudioPeriodBD and before a successful call to xrEndAudioPeriodBD, otherwise the runtime **must** return XR_ERROR_CALL_ORDER_INVALID.

For each sound field, this function **must** not be called more than once per audio period, otherwise the runtime **must** return XR_ERROR_CALL_ORDER_INVALID.

The buffer **must**

- Have an XrAudioBufferBD::bufferChannels greater than 0 and match the channel count defined by

the channel mask and configuration used to create the XrSoundFieldBD.

- Have an XrAudioBufferBD::bufferLength that, when divided by the XrAudioBufferBD::bufferChannels, equals the XrSpatialAudioRendererCreateInfoBD::framesPerBuffer specified at the creation of the spatial audio renderer with xrCreateSpatialAudioRendererBD.

If any of the above conditions are not met, the runtime **must** return XR_ERROR_VALIDATION_FAILURE.

When valid audio is submitted, the runtime **must** correctly process and render the spatial audio.

Applications **must** not call this function concurrently with xrWaitAudioPeriodBD or xrEndAudioPeriodBD. Doing so **may** result in undefined behavior.

Valid Usage (Implicit)

- The XR_BD_spatial_audio_rendering extension **must** be enabled prior to calling xrSubmitSoundFieldBufferBD
- soundField **must** be a valid XrSoundFieldBD handle
- buffer **must** be a pointer to a valid XrAudioBufferBD structure

Thread Safety

- Access to soundField parameter by any other xrSubmitSoundFieldBufferBD call, as well as the renderer parameter by any other xrWaitAudioPeriodBD or xrEndAudioPeriodBD call (if the renderer is the parent of soundField) **must** be externally synchronized

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_CALL_ORDER_INVALID
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The xrSubmitSoundObjectBufferBD function is defined as:

```
// Provided by XR_BD_spatial_audio_rendering
XrResult xrSubmitSoundObjectBufferBD(
    XrSoundObjectBD                             soundObject,
    const XrAudioBufferBD*                      buffer);
```

Parameter Descriptions

- soundObject is the XrSoundObjectBD to input audio signal.
- buffer is a pointer to an XrAudioBufferBD structure that contains the input audio signal.

This function submits an audio buffer to a sound object XrSoundObjectBD. The corresponding function for a sound field XrSoundFieldBD is xrSubmitSoundFieldBufferBD.

This function **must** only be called during an audio period, which is the time period after a successful, not timed out call to xrWaitAudioPeriodBD and before a successful call to xrEndAudioPeriodBD, otherwise the runtime **must** return XR_ERROR_CALL_ORDER_INVALID.

For each sound object, this function **must** not be called more than once per audio period, otherwise the runtime **must** return XR_ERROR_CALL_ORDER_INVALID.

The buffer **must**

- Have XrAudioBufferBD::bufferChannels set to 1.
- Have an XrAudioBufferBD::bufferLength equal to the XrSpatialAudioRendererCreateInfoBD::framesPerBuffer specified when creating the spatial audio renderer with xrCreateSpatialAudioRendererBD.

If any of the above conditions are not met, the runtime **must** return XR_ERROR_VALIDATION_FAILURE.

When valid audio is submitted, the runtime **must** correctly process and render the spatial audio.

Applications **must** not call this function concurrently with xrWaitAudioPeriodBD or xrEndAudioPeriodBD. Doing so **may** result in undefined behavior.

Valid Usage (Implicit)

- The XR_BD_spatial_audio_rendering extension **must** be enabled prior to calling xrSubmitSoundObjectBufferBD
- soundObject **must** be a valid XrSoundObjectBD handle
- buffer **must** be a pointer to a valid XrAudioBufferBD structure

Thread Safety

- Access to soundObject parameter by any other xrSubmitSoundObjectBufferBD call, as well as the renderer parameter by any other xrWaitAudioPeriodBD or xrEndAudioPeriodBD call (if the renderer is the parent of soundObject) **must** be externally synchronized

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_CALL_ORDER_INVALID
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

12.88.6. Issues

This extension does not define separate structures for each audio frame type. Instead, it uses a flat array of floats for audio samples within XrAudioBufferBD.

Rationale:

- Using a flat array allows zero-copy or minimal-copy paths between application and runtime, improving data transfer efficiency and cache locality.
- The flat representation aligns with widely used audio APIs and formats (e.g. PCM float arrays), minimizing conversion overhead and easing interoperability.
- Requirements that would normally vary by frame type (channel count, layout, frames per buffer) are already expressed by fields in XrAudioBufferBD and XrSpatialAudioRendererCreateInfoBD, avoiding the need for per-type structs.
- All frame types share a common floating-point sample representation. Defining separate structures would introduce redundant fields and unnecessary complexity.

This design choice favors clarity and performance without reducing functionality.

12.88.7. Example code

```
XrInstance instance; // previously initialized
XrSession session;   // previously initialized
XrSpace space; //previously initialized
/*************************************************/
```

```
/* xrCreateSpatialAudioRendererBD */
/************************************************/

PFN_xrQueryFramesPerBufferRangeBD xrQueryFramesPerBufferRangeBD;
CHK_XR(xrGetInstanceProcAddr(instance, "xrQueryFramesPerBufferRangeBD",
                             reinterpret_cast<PFN_xrVoidFunction*>(
                             &xrQueryFramesPerBufferRangeBD)));

PFN_xrEnumerateSupportedAudioSampleRateBD xrEnumerateSupportedAudioSampleRateBD;
CHK_XR(xrGetInstanceProcAddr(instance, "xrEnumerateSupportedAudioSampleRateBD",
                             reinterpret_cast<PFN_xrVoidFunction*>(
                             &xrEnumerateSupportedAudioSampleRateBD)));

PFN_xrCreateSpatialAudioRendererBD xrCreateSpatialAudioRendererBD;
CHK_XR(xrGetInstanceProcAddr(instance, "xrCreateSpatialAudioRendererBD",
                             reinterpret_cast<PFN_xrVoidFunction*>(
                             &xrCreateSpatialAudioRendererBD)));

uint32_t sampleRateCount = 0;
CHK_XR(xrEnumerateSupportedAudioSampleRateBD(session, 0, &sampleRateCount, NULL));
std::vector<XrAudioSampleRateBD> sampleRates;
if (sampleRateCount != 0) {
    sampleRates.resize(sampleRateCount);
    CHK_XR(xrEnumerateSupportedAudioSampleRateBD(session, sampleRateCount,
&sampleRateCount, sampleRates.data()));
}

uint32_t minFramePerBuffer = 0;
uint32_t maxFramePerBuffer = 0;
CHK_XR(xrQueryFramesPerBufferRangeBD(session, sampleRates[0], &minFramePerBuffer,
&maxFramePerBuffer));

uint32_t framePerBuffer = 4096;
framePerBuffer = framePerBuffer < minFramePerBuffer ? minFramePerBuffer : framePerBuffer;
framePerBuffer = framePerBuffer > maxFramePerBuffer ? maxFramePerBuffer : framePerBuffer;

// Setup context config struct
XrSpatialAudioRendererCreateInfoBD createInfo{
    XR_TYPE_SPATIAL_AUDIO_RENDERER_CREATE_INFO_BD,
    NULL,
    framePerBuffer,  //  frames_per_buffer
    sampleRates[0]  //  sample rate
};

// Create and initialize spatial audio renderer for session
XrSpatialAudioRendererBD renderer;
CHK_XR(xrCreateSpatialAudioRendererBD(session, &createInfo, &renderer));
```

```
/*****************************************************************************/
/* Setting-up free-field spatialization for sound fields. The following functions are
used: */
/* - xrCreateSoundFieldBD */
/* - xrUpdateSoundFieldConfigBD */
/* - xrSubmitSoundFieldBufferBD */
/*****************************************************************************/

// Setup a 1st order ambisonic sound field config struct
XrSoundFieldChannelDefinitionAmbixBD ambix {
    XR_TYPE_SOUND_FIELD_CHANNEL_DEFINITION_AMBIX_BD,
    nullptr,
    XR_SOUND_FIELD_CHANNEL_MASK_AMBIX_1ST_ORDER_BD
};

XrSoundFieldConfigBD soundFieldConfig{
    XR_TYPE_SOUND_FIELD_CONFIG_BD,
    &ambix,
    true,
    {0.0f, 0.0f, 0.0f, 1.0f},  // orientation
    space,
    1,  //  main volume
    1  //  LFE gain
};


// Add sound field to the scene
PFN_xrCreateSoundFieldBD xrCreateSoundFieldBD;
CHK_XR(xrGetInstanceProcAddr(instance, "xrCreateSoundFieldBD",
                             reinterpret_cast<PFN_xrVoidFunction*>(
                             &xrCreateSoundFieldBD)));
XrSoundFieldBD soundField;
CHK_XR(xrCreateSoundFieldBD(renderer, &soundFieldConfig, &soundField));

// Submit a sound field buffer
/// Wait for the next audio period to begin
PFN_xrWaitAudioPeriodBD xrWaitAudioPeriodBD;
CHK_XR(xrGetInstanceProcAddr(instance, "xrWaitAudioPeriodBD",
                             reinterpret_cast<PFN_xrVoidFunction*>(
                             &xrWaitAudioPeriodBD)));
CHK_XR(xrWaitAudioPeriodBD(renderer, XR_INFINITE_DURATION));

/// Submit a sound field buffer
PFN_xrSubmitSoundFieldBufferBD xrSubmitSoundFieldBufferBD;
CHK_XR(xrGetInstanceProcAddr(instance, "xrSubmitSoundFieldBufferBD",
                             reinterpret_cast<PFN_xrVoidFunction*>(
                             &xrSubmitSoundFieldBufferBD)));
float mcbuffer[4*256];
```

```
XrAudioBufferBD soundFieldBuffer{
    XR_TYPE_AUDIO_BUFFER_BD,
    nullptr,
    XR_AUDIO_BUFFER_CHANNEL_LAYOUT_PLANAR_BD,
    4,  //  num_channels
    4*256,  //  buffer_length
    mcbuffer
};
CHK_XR(xrSubmitSoundFieldBufferBD(soundField, &soundFieldBuffer));

/// Signal the end of the current audio period
PFN_xrEndAudioPeriodBD xrEndAudioPeriodBD;
CHK_XR(xrGetInstanceProcAddr(instance, "xrEndAudioPeriodBD",
                             reinterpret_cast<PFN_xrVoidFunction*>(
                             &xrEndAudioPeriodBD)));
CHK_XR(xrEndAudioPeriodBD(renderer));

// We should now be able to hear added sound field in device audio output

// Change sound field config
soundFieldConfig.orientation = {0.0f, 0.38268343f, 0.0f, 0.92387953f}; // rotate 45
degrees around y axis
PFN_xrUpdateSoundFieldConfigBD xrUpdateSoundFieldConfigBD;
CHK_XR(xrGetInstanceProcAddr(instance, "xrUpdateSoundFieldConfigBD",
                             reinterpret_cast<PFN_xrVoidFunction*>(
                             &xrUpdateSoundFieldConfigBD)));
CHK_XR(xrUpdateSoundFieldConfigBD(soundField, &soundFieldConfig,
XR_SOUND_FIELD_ORIENTATION_BIT_BD));

// Submit input signal again
/// Wait for the next audio period to begin
CHK_XR(xrWaitAudioPeriodBD(renderer, XR_INFINITE_DURATION));

/// Submit a sound field buffer
CHK_XR(xrSubmitSoundFieldBufferBD(soundField, &soundFieldBuffer));

/// Signal the end of the current audio period
CHK_XR(xrEndAudioPeriodBD(renderer));

// We should now be able to hear the sound field being rotated in device audio output

/*****************************************************************************/
/* Setting-up free-field spatialization for object audio. The following functions are
used: */
/* - xrCreateSoundObjectBD */
/* - xrUpdateSoundObjectConfigBD */
/* - xrSubmitSoundObjectBufferBD */
/*****************************************************************************/
```

```
// Add object to the scene
PFN_xrCreateSoundObjectBD xrCreateSoundObjectBD;
CHK_XR(xrGetInstanceProcAddr(instance, "xrCreateSoundObjectBD",
                             reinterpret_cast<PFN_xrVoidFunction*>(
                             &xrCreateSoundObjectBD)));
XrSoundObjectBD objectId;
// Construct polar pattern
XrSoundObjectDirectivityCardioidBD objectDirectivityCardioid{
    XR_TYPE_SOUND_OBJECT_DIRECTIVITY_CARDIOID_BD,
    NULL,
    0.5f,    //  alpha
    1.0f     //  order
};

// Construct shape
XrSoundObjectShapeSphereBD objectShapeSphere{
    XR_TYPE_SOUND_OBJECT_SHAPE_SPHERE_BD,
    &objectDirectivityCardioid,
    0.1f  //  radius of 0.1 meter
};

// Construct inverse square distance attenuation
XrSoundObjectDistanceAttenuationBD objectDistanceAttenuation{
    XR_TYPE_SOUND_OBJECT_DISTANCE_ATTENUATION_BD,
    NULL,
    XR_SOUND_OBJECT_DISTANCE_ATTENUATION_TYPE_INVERSE_SQUARE_BD,   //  distance
attenuation type
    0.1f,   //  min distance
    100.0f,    //  max distance
    1.0f  //  rolloff factor
};

// Construct object config
XrSoundObjectConfigBD objectConfig{
    XR_TYPE_SOUND_OBJECT_CONFIG_BD,
    &objectShapeSphere,
    true,
    {{0.0f, 0.0f, 0.0f, 1.0f}, {0.0f, 0.0f, 1.0f}}, //   Pose
    space,
    1.0f,   //  main volume
    1.0f,   //  reflection gain
    true,   //  is Doppler enabled
    &objectDistanceAttenuation,     //  distance attenuation for direct sound
    &objectDistanceAttenuation      //  distance attenuation for indirect sound
};

// Add object to the scene
```

```
CHK_XR(xrCreateSoundObjectBD(renderer, &objectConfig, &objectId));

// Submit a sound object buffer
/// Wait for the next audio period to begin
CHK_XR(xrWaitAudioPeriodBD(renderer, XR_INFINITE_DURATION));

/// Submit a sound object buffer
PFN_xrSubmitSoundObjectBufferBD xrSubmitSoundObjectBufferBD;
CHK_XR(xrGetInstanceProcAddr(instance, "xrSubmitSoundObjectBufferBD",
                             reinterpret_cast<PFN_xrVoidFunction*>(
                             &xrSubmitSoundObjectBufferBD)));
float buffer[256];
XrAudioBufferBD objBuffer{
    XR_TYPE_AUDIO_BUFFER_BD,
    nullptr,
    XR_AUDIO_BUFFER_CHANNEL_LAYOUT_INTERLEAVED_BD,
    1,  //  num_channels
    256,  //  buffer_length
    buffer
};

// submit the buffer
CHK_XR(xrSubmitSoundObjectBufferBD(objectId, &objBuffer));

/// Signal the end of the current audio period
CHK_XR(xrEndAudioPeriodBD(renderer));

// We should now be able to hear added sound object in device audio output

// Change object config
objectConfig.pose.position = {1.0f, 0.0f, 1.0f}; // move sound object 1 meter towards
positive direction of x axis
PFN_xrUpdateSoundObjectConfigBD xrUpdateSoundObjectConfigBD;
CHK_XR(xrGetInstanceProcAddr(instance, "xrUpdateSoundObjectConfigBD",
                             reinterpret_cast<PFN_xrVoidFunction*>(
                             &xrUpdateSoundObjectConfigBD)));
CHK_XR(xrUpdateSoundObjectConfigBD(objectId, &objectConfig,
XR_SOUND_OBJECT_POSE_BIT_BD));


// Submit input signal again and get output signal with a different sound field position.
/// Wait for the next audio period to begin
CHK_XR(xrWaitAudioPeriodBD(renderer, XR_INFINITE_DURATION));

/// Submit a sound object buffer
CHK_XR(xrSubmitSoundObjectBufferBD(objectId, &objBuffer));

/// Signal the end of the current audio period
```

```
CHK_XR(xrEndAudioPeriodBD(renderer));

// We should now be able to hear that the position of the sound object is changed

/******************************************************************************/
/* Adding sound obstacles to render room acoustics. The following functions are used: */
/* - xrCreateSoundObstacleBD */
/* - xrSubmitObjectBufferBD */
/* - xrUpdateSoundObstacleConfigBD */
/* - xrDestroySoundObstacleMeshBD */
/******************************************************************************/

// Setup triangle mesh of a rectangular room
XrVector3f vertices[] = {{5.0f, 5.0f, 5.0f}, {-5.0f, 5.0f, 5.0f}, {-5.0f, 5.0f, -5.0f},
{5.0f, 5.0f,-5.0f},
                         {5.0f, -5.0f, 5.0f}, {-5.0f, -5.0f, 5.0f}, {-5.0f, -5.0f, -
5.0f}, {5.0f, -5.0f, -5.0f}};
uint32_t indices[] = {0, 1, 2, 0, 2, 3, 1, 5, 2, 2, 5, 6, 2, 6, 3, 3, 6, 7, 1, 0, 4, 1,
4, 5, 0, 3, 7, 0, 7, 4, 5, 7, 6, 5, 4, 7};
XrSoundTriangleMeshBD mesh{
    XR_TYPE_SOUND_TRIANGLE_MESH_BD,NULL,
    8,    //  vertexCount
    vertices,
    12*3,   //  indexCount
    indices
};

// Setup acoustic material
float freqCenter[] = {500.0f, 1000.0f, 2000.0f, 4000.0f};
float absorption[] = {0.1f, 0.12f, 0.15f, 0.2f};
float scattering[] = {0.2f, 0.23f, 0.26f, 0.3f};
float transmission[] = {0.5f, 0.5f, 0.5f, 0.5f};
XrSoundObstacleMaterialConfigBD materialConfig{
    XR_TYPE_SOUND_OBSTACLE_MATERIAL_CONFIG_BD,
    NULL,
    XR_SOUND_OBSTACLE_MATERIAL_TYPE_CUSTOM_BD, //  material type
    4,  //  frequency bandCount
    freqCenter,       //  bandFrequencies (frequency center of bands)
    absorption,       //  bandAbsorptions
    scattering,       //  bandScatterings
    transmission      //  bandTransmissions
};

PFN_xrCreateSoundObstacleMaterialBD xrCreateSoundObstacleMaterialBD;
CHK_XR(xrGetInstanceProcAddr(instance, "xrCreateSoundObstacleMaterialBD",
                             reinterpret_cast<PFN_xrVoidFunction*>(
                             &xrCreateSoundObstacleMaterialBD)));
XrSoundObstacleMaterialBD materialHandle;
```

```
CHK_XR(xrCreateSoundObstacleMaterialBD(renderer, &materialConfig, &materialHandle));

// Setup obstacle config
XrSoundObstacleMaterialBD materials[8] = {materialHandle, materialHandle, materialHandle,
materialHandle,
    materialHandle, materialHandle, materialHandle, materialHandle};
XrSoundObstacleConfigBD obstacleConfig{
    XR_TYPE_SOUND_OBSTACLE_CONFIG_BD,
    NULL,
    true,   //  sound obstacle enabled
    {{0.0f, 0.0f, 0.0f, 1.0f}, {0.0f, 0.0f, 1.0f}}, //  Pose
    space,
    8,    //  materialCount, which is equal to vertexCount
    materials    //  material for each vertex
};

//  Submit sound obstacle to the scene
PFN_xrCreateSoundObstacleBD xrCreateSoundObstacleBD;
CHK_XR(xrGetInstanceProcAddr(instance, "xrCreateSoundObstacleBD",
                             reinterpret_cast<PFN_xrVoidFunction*>(
                             &xrCreateSoundObstacleBD)));
XrSoundObstacleBD soundObstacleHandle;
CHK_XR(xrCreateSoundObstacleBD(renderer, &obstacleConfig, &mesh, &soundObstacleHandle));


// Input object audio
/// Wait for the next audio period to begin
CHK_XR(xrWaitAudioPeriodBD(renderer, XR_INFINITE_DURATION));

/// Submit a sound object buffer
CHK_XR(xrSubmitSoundObjectBufferBD(objectId, &objBuffer));

/// Signal the end of the current audio period
CHK_XR(xrEndAudioPeriodBD(renderer));

// We should now be able to hear the sound object
// being reflected by the sound obstacle we just added in the form of echo and reverb

// Change sound obstacle config
obstacleConfig.enabled = false;
PFN_xrUpdateSoundObstacleConfigBD xrUpdateSoundObstacleConfigBD;
CHK_XR(xrGetInstanceProcAddr(instance, "xrUpdateSoundObstacleConfigBD",
                             reinterpret_cast<PFN_xrVoidFunction*>(
                             &xrUpdateSoundObstacleConfigBD)));
CHK_XR(xrUpdateSoundObstacleConfigBD(soundObstacleHandle, &obstacleConfig, NULL,
XR_SOUND_OBSTACLE_ENABLED_BIT_BD));
```

```
// Input object audio
/// Wait for the next audio period to begin
CHK_XR(xrWaitAudioPeriodBD(renderer, XR_INFINITE_DURATION));

/// Submit a sound object buffer
CHK_XR(xrSubmitSoundObjectBufferBD(objectId, &objBuffer));

/// Signal the end of the current audio period
CHK_XR(xrEndAudioPeriodBD(renderer));

// We should now be able to hear the echo and reverb goes away
// since the only sound obstacle is disabled

/*********************************************************/
/* Destroying spatial audio renderer for specified session */
/*********************************************************/

// Destroy spatial audio renderer
PFN_xrDestroySpatialAudioRendererBD xrDestroySpatialAudioRendererBD;
CHK_XR(xrGetInstanceProcAddr(instance, "xrDestroySpatialAudioRendererBD",
                             reinterpret_cast<PFN_xrVoidFunction*>(
                             &xrDestroySpatialAudioRendererBD)));
CHK_XR(xrDestroySpatialAudioRendererBD(renderer));
```

12.88.8. New Object Types

- XrSoundFieldBD
- XrSoundObjectBD
- XrSoundObstacleBD
- XrSoundObstacleMaterialBD
- XrSpatialAudioRendererBD

12.88.9. New Commands

- xrCreateSoundFieldBD
- xrCreateSoundObjectBD
- xrCreateSoundObstacleBD
- xrCreateSoundObstacleMaterialBD
- xrCreateSpatialAudioRendererBD
- xrDestroySoundFieldBD
- xrDestroySoundObjectBD

- xrDestroySoundObstacleBD
- xrDestroySoundObstacleMaterialBD
- xrDestroySpatialAudioRendererBD
- xrEndAudioPeriodBD
- xrEnumerateSupportedAudioSampleRateBD
- xrQueryFramesPerBufferRangeBD
- xrSubmitSoundFieldBufferBD
- xrSubmitSoundObjectBufferBD
- xrUpdateSoundFieldConfigBD
- xrUpdateSoundObjectConfigBD
- xrUpdateSoundObstacleConfigBD
- xrUpdateSoundObstacleMaterialConfigBD
- xrWaitAudioPeriodBD

12.88.10. New Structures

- XrAttenuationCurvePointBD
- XrAudioBufferBD
- XrSoundFieldConfigBD
- XrSoundObjectConfigBD
- XrSoundObjectDistanceAttenuationBD
- XrSoundObjectDistanceAttenuationCurveBD
- XrSoundObstacleConfigBD
- XrSoundObstacleMaterialConfigBD
- XrSoundTriangleMeshBD
- XrSpatialAudioRendererCreateInfoBD
- Extending XrSoundFieldConfigBD:
  - XrSoundFieldChannelDefinitionAmbixBD
  - XrSoundFieldChannelDefinitionFumaBD
  - XrSoundFieldChannelDefinitionSurroundBD
- Extending XrSoundObjectConfigBD:
  - XrSoundObjectDirectivityCardioidBD
  - XrSoundObjectShapeSphereBD

12.88.11. New Enums

- XrAudioBufferChannelLayoutBD
- XrAudioSampleRateBD
- XrSoundFieldChannelMaskAmbixBD
- XrSoundFieldChannelMaskFumaBD
- XrSoundFieldChannelMaskSurroundBD
- XrSoundFieldChannelMaskSurroundBD
- XrSoundFieldFlagBitsBD
- XrSoundObjectDistanceAttenuationTypeBD
- XrSoundObjectFlagBitsBD
- XrSoundObstacleFlagBitsBD
- XrSoundObstacleMaterialTypeBD

12.88.12. New Bitmasks

- XrSoundFieldFlagsBD
- XrSoundObjectFlagsBD
- XrSoundObstacleFlagsBD

12.88.13. New Enum Constants

- `XR_BD_SPATIAL_AUDIO_RENDERING_EXTENSION_NAME`
- `XR_BD_spatial_audio_rendering_SPEC_VERSION`
- Extending XrObjectType:
  - `XR_OBJECT_TYPE_SOUND_FIELD_BD`
  - `XR_OBJECT_TYPE_SOUND_OBJECT_BD`
  - `XR_OBJECT_TYPE_SOUND_OBSTACLE_BD`
  - `XR_OBJECT_TYPE_SOUND_OBSTACLE_MATERIAL_BD`
  - `XR_OBJECT_TYPE_SPATIAL_AUDIO_RENDERER_BD`
- Extending XrStructureType:
  - `XR_TYPE_AUDIO_BUFFER_BD`
  - `XR_TYPE_SOUND_FIELD_CHANNEL_DEFINITION_AMBIX_BD`
  - `XR_TYPE_SOUND_FIELD_CHANNEL_DEFINITION_FUMA_BD`
  - `XR_TYPE_SOUND_FIELD_CHANNEL_DEFINITION_SURROUND_BD`

- XR_TYPE_SOUND_FIELD_CONFIG_BD
- XR_TYPE_SOUND_OBJECT_CONFIG_BD
- XR_TYPE_SOUND_OBJECT_DIRECTIVITY_CARDIOID_BD
- XR_TYPE_SOUND_OBJECT_DISTANCE_ATTENUATION_BD
- XR_TYPE_SOUND_OBJECT_DISTANCE_ATTENUATION_CURVE_BD
- XR_TYPE_SOUND_OBJECT_SHAPE_SPHERE_BD
- XR_TYPE_SOUND_OBSTACLE_CONFIG_BD
- XR_TYPE_SOUND_OBSTACLE_MATERIAL_CONFIG_BD
- XR_TYPE_SOUND_TRIANGLE_MESH_BD
- XR_TYPE_SPATIAL_AUDIO_RENDERER_CREATE_INFO_BD

12.88.14. Version History

- Revision 1, 2025-01-22 (Xuzhou Ye)
  - Initial extension description

12.89. XR_BD_spatial_mesh

Name String

XR_BD_spatial_mesh

Extension Type

Instance extension

Registered Extension Number

394

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

XR_BD_spatial_sensing

Last Modified Date

2025-03-26

IP Status

No known IP claims.

Contributors

Zhipeng Liu, ByteDance
Zhanrui Jia, ByteDance
Xu Yang, ByteDance
Jun Yan, ByteDance
Jimmy Alamparambil, ByteDance

12.89.1. Overview

This extension allows applications to request the runtime to detect and track spatial scene meshes of the physical environment.

This extension requires `XR_BD_spatial_sensing` to be enabled.

12.89.2. Permission

A runtime on an AOSP-based platform **must** verify that applications have the `com.picovr.permission.SPATIAL_DATA` permission both listed in their manifest and granted to use spatial mesh. Without it, runtime **must** set XrFutureCompletionEXT::`futureResult` to `XR_ERROR_PERMISSION_INSUFFICIENT` when the xrStartSenseDataProviderCompleteBD is called.

This is an auto-requested permission: if it is listed in the manifest but not yet granted or denied, the runtime **must** prompt the user to grant or deny the permission when xrCreateSenseDataProviderBD is called with a provider type that requires it.

This permission is also used by `XR_BD_spatial_anchor`.

This permission is also used by `XR_BD_spatial_scene`.

This permission is also used by `XR_BD_spatial_plane`.

12.89.3. Inspect System Capability

The XrSystemSpatialMeshPropertiesBD structure is defined as:

```
// Provided by XR_BD_spatial_mesh
typedef struct XrSystemSpatialMeshPropertiesBD {
    XrStructureType    type;
    void*              next;
    XrBool32           supportsSpatialMesh;
} XrSystemSpatialMeshPropertiesBD;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. For more details about the structure chain, see the structure being extended (XrSystemProperties).
- `supportsSpatialMesh` is an `XrBool32`, indicating if current system is capable of spatial mesh functionality.

An application **can** inspect whether the system is capable of spatial mesh functionality by chaining an XrSystemSpatialMeshPropertiesBD structure to the XrSystemProperties::`next` chain when calling xrGetSystemProperties.

If a runtime returns `XR_FALSE` for `supportsSpatialMesh`, the system does not support spatial mesh functionality, and the runtime **must** return `XR_ERROR_FEATURE_UNSUPPORTED` from xrCreateSenseDataProviderBD when called with `XR_SENSE_DATA_PROVIDER_TYPE_MESH_BD`. The application **should** avoid using spatial mesh functionality when `supportsSpatialMesh` is `XR_FALSE`.

If XrSystemSpatialSensingPropertiesBD::`supportsSpatialSensing` is `XR_FALSE`, then `supportsSpatialMesh` **must** also be `XR_FALSE`.

If a runtime returns `XR_TRUE` for `supportsSpatialMesh`, the system supports spatial mesh functionality. This implies that XrSystemSpatialSensingPropertiesBD::`supportsSpatialSensing` **must** also be `XR_TRUE`.

Note that `supportsSpatialMesh` **may** be `XR_TRUE` even if running on an Android-based platform and the application does not have the required com.picovr.permission.SPATIAL_DATA permission both declared in the manifest and granted at runtime. Evaluation of permissions takes place later, in the asynchronous operation started by xrStartSenseDataProviderAsyncBD.

Valid Usage (Implicit)

- The `XR_BD_spatial_mesh` extension **must** be enabled prior to using XrSystemSpatialMeshPropertiesBD
- `type` **must** be `XR_TYPE_SYSTEM_SPATIAL_MESH_PROPERTIES_BD`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

12.89.4. Create Spatial Mesh Provider

The XrSenseDataProviderCreateInfoSpatialMeshBD structure is defined as:

```
// Provided by XR_BD_spatial_mesh
typedef struct XrSenseDataProviderCreateInfoSpatialMeshBD {
    XrStructureType               type;
    const void*                   next;
    XrSpatialMeshConfigFlagsBD    configFlags;
    XrSpatialMeshLodBD            lod;
} XrSenseDataProviderCreateInfoSpatialMeshBD;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `configFlags` is the XrSpatialMeshConfigFlagBitsBD of the configurations.
- `lod` is the XrSpatialMeshLodBD which represent spatial mesh level.

An application creates an XrSenseDataProviderBD handle representing a a spatial mesh provider by calling xrCreateSenseDataProviderBD, setting XrSenseDataProviderCreateInfoBD::`providerType` equal to the XrSenseDataProviderTypeBD value `XR_SENSE_DATA_PROVIDER_TYPE_MESH_BD`, and chaining XrSenseDataProviderCreateInfoSpatialMeshBD to XrSenseDataProviderCreateInfoBD::`next`. Both the XrSenseDataProviderTypeBD value and the chained XrSenseDataProviderCreateInfoSpatialMeshBD structure are required. If XrSenseDataProviderCreateInfoBD::`providerType` is equal to `XR_SENSE_DATA_PROVIDER_TYPE_MESH_BD` but XrSenseDataProviderCreateInfoSpatialMeshBD is not in the `next` chain, the runtime **must** return `XR_ERROR_VALIDATION_FAILURE`.

An application uses such a provider to obtain the spatial mesh info that is detected and tracked by the runtime. This data generally consists of spatial entities with at least a XrSpatialEntityComponentDataTriangleMeshBD component.

Valid Usage (Implicit)

- The `XR_BD_spatial_mesh` extension **must** be enabled prior to using XrSenseDataProviderCreateInfoSpatialMeshBD
- `type` **must** be `XR_TYPE_SENSE_DATA_PROVIDER_CREATE_INFO_SPATIAL_MESH_BD`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `configFlags` **must** be `0` or a valid combination of XrSpatialMeshConfigFlagBitsBD values
- `lod` **must** be a valid XrSpatialMeshLodBD value

The enumeration XrSpatialMeshConfigFlagsBD is defined as:

```
// Provided by XR_BD_spatial_mesh
typedef XrFlags64 XrSpatialMeshConfigFlagsBD;
```

The XrSenseDataProviderCreateInfoSpatialMeshBD::configFlags member is of the type XrSpatialMeshConfigFlagsBD, and contains a bitwise-OR of zero or more of the bits defined in XrSpatialMeshConfigFlagBitsBD.

Valid bits for XrSpatialMeshConfigFlagsBD are defined by XrSpatialMeshConfigFlagBitsBD, which is specified as:

```
// Provided by XR_BD_spatial_mesh
// Flag bits for XrSpatialMeshConfigFlagsBD
static const XrSpatialMeshConfigFlagsBD XR_SPATIAL_MESH_CONFIG_SEMANTIC_BIT_BD =
0x00000001;
static const XrSpatialMeshConfigFlagsBD
XR_SPATIAL_MESH_CONFIG_ALIGN_SEMANTIC_WITH_VERTEX_BIT_BD = 0x00000002;
```

Flag Descriptions

- XR_SPATIAL_MESH_CONFIG_SEMANTIC_BIT_BD — Detect and include semantic label component data in the spatial entity.
- XR_SPATIAL_MESH_CONFIG_ALIGN_SEMANTIC_WITH_VERTEX_BIT_BD — Semantic labels are associated with each vertex, otherwise, semantic labels are associated with each triangle face by default. It only takes effect if XR_SPATIAL_MESH_CONFIG_SEMANTIC_BIT_BD is set

If XR_SPATIAL_MESH_CONFIG_SEMANTIC_BIT_BD is not set, the XR_SPATIAL_MESH_CONFIG_ALIGN_SEMANTIC_WITH_VERTEX_BIT_BD takes no effect. If XR_SPATIAL_MESH_CONFIG_ALIGN_SEMANTIC_WITH_VERTEX_BIT_BD is not set, each semantic label corresponds to each vertex. If XR_SPATIAL_MESH_CONFIG_ALIGN_SEMANTIC_WITH_VERTEX_BIT_BD is set, each semantic label corresponds to three indices.

The XrSpatialMeshLodBD enumeration identifies the different LOD levels.

```
// Provided by XR_BD_spatial_mesh
typedef enum XrSpatialMeshLodBD {
    XR_SPATIAL_MESH_LOD_COARSE_BD = 0,
    XR_SPATIAL_MESH_LOD_MEDIUM_BD = 1,
    XR_SPATIAL_MESH_LOD_FINE_BD = 2,
    XR_SPATIAL_MESH_LOD_MAX_ENUM_BD = 0x7FFFFFFF
} XrSpatialMeshLodBD;
```

Enum	Description
XR_SPATIAL_MESH_LOD_COARSE_BD	Coarse level for the mesh with less details.
XR_SPATIAL_MESH_LOD_MEDIUM_BD	Medium level for the mesh.
XR_SPATIAL_MESH_LOD_FINE_BD	Fine level for the mesh with more details.

12.89.5. Start Spatial Mesh Provider

Applications start the spatial mesh data provider by calling xrStartSenseDataProviderAsyncBD after it is successfully created. To check the data provider state, call xrGetSenseDataProviderStateBD.

Subsequent application operations using this handle **must** not be performed unless the `futureResult` is `XR_SUCCESS`. If the `futureResult` returns an error code and the application needs to use the provider, the application **must** take appropriate action and try to call xrStartSenseDataProviderAsyncBD again before using the handle.

Detailed definitions and usage details are described in `XR_BD_spatial_sensing`.

12.89.6. Get Spatial Mesh Data

Applications query the latest detected spatial meshes from the spatial mesh data provider by calling xrQuerySenseDataAsyncBD. The runtime generates a snapshot of the spatial mesh information, from which the application **can** obtain detailed spatial mesh information. The mesh information is presented in the form of spatial entities which the application queries by calling xrGetQueriedSenseDataBD. Use xrEnumerateSpatialEntityComponentTypesBD to get the types of components contained within these entities, such as location, semantics, and mesh vertices and indices. To further retrieve component data information, call xrGetSpatialEntityComponentDataBD.

When the spatial mesh information changes, the runtime **must** queue an XrEventDataSenseDataUpdatedBD event with the handle of the spatial mesh provider.

When the application receives this event, it means the sense data provider has updated mesh data. A new query request for this sense data provider will get the latest, updated mesh data. This is the recommended way to get updated data.

Alternatively, the application **may** query the latest spatial mesh information at a time that suits its needs, independent of the update events.

All the functions to get spatial mesh info are defined in `XR_BD_spatial_sensing`.

12.89.7. New Structures

- Extending XrSenseDataProviderCreateInfoBD:
  - XrSenseDataProviderCreateInfoSpatialMeshBD
- Extending XrSystemProperties:

    - XrSystemSpatialMeshPropertiesBD

12.89.8. New Enums

- XrSpatialMeshConfigFlagBitsBD
- XrSpatialMeshLodBD

12.89.9. New Bitmasks

- XrSpatialMeshConfigFlagsBD

12.89.10. New Enum Constants

- `XR_BD_SPATIAL_MESH_EXTENSION_NAME`
- `XR_BD_spatial_mesh_SPEC_VERSION`
- Extending XrSenseDataProviderTypeBD:
    - `XR_SENSE_DATA_PROVIDER_TYPE_MESH_BD`
- Extending XrStructureType:
    - `XR_TYPE_SENSE_DATA_PROVIDER_CREATE_INFO_SPATIAL_MESH_BD`
    - `XR_TYPE_SYSTEM_SPATIAL_MESH_PROPERTIES_BD`

12.89.11. Issues

12.89.12. Version History

- Revision 1, 2024-05-06 (Zhipeng Liu)
    - Initial extension description

12.90. XR_BD_spatial_plane

Name String

`XR_BD_spatial_plane`

Extension Type

Instance extension

Registered Extension Number

397

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

XR_BD_spatial_sensing

Last Modified Date

2025-03-31

IP Status

No known IP claims.

Contributors

Xu Yang, ByteDance
Zhipeng Liu, ByteDance

12.90.1. Overview

This extension allows applications to request the runtime to detect and track planes in the physical environment. This extension requires XR_BD_spatial_sensing to be enabled.

12.90.2. Permission

A runtime on an AOSP-based platform **must** verify that applications have the com.picovr.permission.SPATIAL_DATA permission both listed in their manifest and granted to use spatial plane. Without it, runtime **must** set XrFutureCompletionEXT::futureResult to XR_ERROR_PERMISSION_INSUFFICIENT when the xrStartSenseDataProviderCompleteBD is called.

This is an auto-requested permission: if it is listed in the manifest but not yet granted or denied, the runtime **must** prompt the user to grant or deny the permission when xrCreateSenseDataProviderBD is called with a provider type that requires it.

This permission is also used by XR_BD_spatial_anchor.

This permission is also used by XR_BD_spatial_mesh.

This permission is also used by XR_BD_spatial_scene.

12.90.3. Inspect System Capability

The XrSystemSpatialPlanePropertiesBD structure is defined as:

```
// Provided by XR_BD_spatial_plane
typedef struct XrSystemSpatialPlanePropertiesBD {
    XrStructureType    type;
    void*              next;
    XrBool32           supportsSpatialPlane;
} XrSystemSpatialPlanePropertiesBD;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `supportsSpatialPlane` is an `XrBool32`, indicating if the current system is capable of spatial plane functionality.

An application **can** inspect whether the system is capable of spatial plane functionality by chaining an XrSystemSpatialPlanePropertiesBD structure to the XrSystemProperties::`next` chain when calling xrGetSystemProperties.

If a runtime returns `XR_FALSE` for `supportsSpatialPlane`, the system does not support spatial plane functionality, and the runtime **must** return `XR_ERROR_FEATURE_UNSUPPORTED` from xrCreateSenseDataProviderBD when called with `XR_SENSE_DATA_PROVIDER_TYPE_PLANE_BD`.

The application **should** avoid using spatial plane functionality when `supportsSpatialPlane` is `XR_FALSE`.

If XrSystemSpatialSensingPropertiesBD::`supportsSpatialSensing` is `XR_FALSE`, then `supportsSpatialPlane` **must** also be `XR_FALSE`.

If the runtime returns `XR_TRUE` for `supportsSpatialPlane`, the system supports spatial plane functionality. This implies that XrSystemSpatialSensingPropertiesBD::`supportsSpatialSensing` **must** also be `XR_TRUE`.

Note that `supportsSpatialPlane` **may** be `XR_TRUE` even if running on an Android-based platform and the application does not have the required com.picovr.permission.SPATIAL_DATA permission both declared in the manifest and granted at runtime. Evaluation of permissions takes place later, in the asynchronous operation started by xrStartSenseDataProviderAsyncBD.

Valid Usage (Implicit)

- The XR_BD_spatial_plane extension **must** be enabled prior to using XrSystemSpatialPlanePropertiesBD
- type **must** be XR_TYPE_SYSTEM_SPATIAL_PLANE_PROPERTIES_BD
- next **must** be NULL or a valid pointer to the next structure in a structure chain

12.90.4. Create Spatial Plane Provider

Applications create an XrSenseDataProviderBD handle representing a spatial plane provider by calling xrCreateSenseDataProviderBD, and setting XrSenseDataProviderCreateInfoBD::providerType to the XrSenseDataProviderTypeBD value XR_SENSE_DATA_PROVIDER_TYPE_PLANE_BD.

Applications use this provider handle to obtain information about spatial planes that are detected and tracked by the runtime. This data generally consists of spatial entities with at least a plane orientation component which corresponds to an XrSpatialEntityComponentTypeBD value of XR_SPATIAL_ENTITY_COMPONENT_TYPE_PLANE_ORIENTATION_BD and a triangle mesh component which corresponds to an XrSpatialEntityComponentTypeBD value of XR_SPATIAL_ENTITY_COMPONENT_TYPE_TRIANGLE_MESH_BD.

This provider type does not define any configuration and does not require a chained structure.

12.90.5. Start Spatial Plane Provider

Applications start the spatial plane data provider by calling xrStartSenseDataProviderAsyncBD after it has been successfully created. To check the data provider state, call xrGetSenseDataProviderStateBD.

Subsequent application operations using this provider handle **must** not be performed unless the data provider state is XR_SENSE_DATA_PROVIDER_STATE_RUNNING_BD. If the data provider state is not XR_SENSE_DATA_PROVIDER_STATE_RUNNING_BD and the application intends to use the provider, the application **must** take appropriate action and call xrStartSenseDataProviderAsyncBD again before using the handle.

Detailed definitions and usage details are described in XR_BD_spatial_sensing.

12.90.6. Get Spatial Plane Data

Applications query the latest detected spatial planes from the spatial plane data provider by calling xrQuerySenseDataAsyncBD. The runtime generates a snapshot of the spatial plane information, from which the application **can** obtain detailed spatial plane information. The plane information is presented in the form of spatial entities which the application queries by calling xrGetQueriedSenseDataBD. Use xrEnumerateSpatialEntityComponentTypesBD to get the types of components contained within these entities, such as location, semantics, vertices and indices. To further retrieve component data information, call xrGetSpatialEntityComponentDataBD.

When the runtime's understanding of the detected spatial planes changes, the runtime **must** queue an XrEventDataSenseDataUpdatedBD event with the handle of the spatial plane provider.

Receiving this event signals to the application that updated plane data is available to query from the sense data provider. A new query request for this sense data provider will get the latest, updated plane data. This is the recommended way to get updated data.

The application **may** also initiate a new query request at any time without receiving this event, but the queried data **may** be unmodified.

All the functions to get spatial plane info are defined in XR_BD_spatial_sensing.

12.90.7. Acquire Plane Orientation

The XrPlaneOrientationBD enumeration identifies the different orientation of a spatial plane.

```
// Provided by XR_BD_spatial_plane
typedef enum XrPlaneOrientationBD {
    XR_PLANE_ORIENTATION_HORIZONTAL_UPWARD_BD = 0,
    XR_PLANE_ORIENTATION_HORIZONTAL_DOWNWARD_BD = 1,
    XR_PLANE_ORIENTATION_VERTICAL_BD = 2,
    XR_PLANE_ORIENTATION_ARBITRARY_BD = 3,
    XR_PLANE_ORIENTATION_MAX_ENUM_BD = 0x7FFFFFFF
} XrPlaneOrientationBD;
```

Enum	Description
XR_PLANE_ORIENTATION_HORIZONTAL_UPWARD_BD	The detected plane is horizontal and faces upward (e.g. floor).
XR_PLANE_ORIENTATION_HORIZONTAL_DOWNWARD_BD	The detected plane is horizontal and faces downward (e.g. ceiling).
XR_PLANE_ORIENTATION_VERTICAL_BD	The detected plane is vertical (e.g. wall).
XR_PLANE_ORIENTATION_ARBITRARY_BD	The detected plane has an arbitrary, non-vertical and non-horizontal orientation.

The XrSpatialEntityComponentDataPlaneOrientationBD structure is defined as:

```
// Provided by XR_BD_spatial_plane
typedef struct XrSpatialEntityComponentDataPlaneOrientationBD {
    XrStructureType         type;
    void*                   next;
    XrPlaneOrientationBD    orientation;
} XrSpatialEntityComponentDataPlaneOrientationBD;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `orientation` is the pointer to XrPlaneOrientationBD.

XrSpatialEntityComponentDataPlaneOrientationBD is an output structure for getting the plane orientation component data from a snapshot. This corresponds to an XrSpatialEntityComponentTypeBD value of `XR_SPATIAL_ENTITY_COMPONENT_TYPE_PLANE_ORIENTATION_BD`.

Valid Usage (Implicit)

- The `XR_BD_spatial_plane` extension **must** be enabled prior to using XrSpatialEntityComponentDataPlaneOrientationBD
- `type` **must** be `XR_TYPE_SPATIAL_ENTITY_COMPONENT_DATA_PLANE_ORIENTATION_BD`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `orientation` **must** be a valid XrPlaneOrientationBD value

12.90.8. Spatial Plane Component Orientation Filter

The XrSenseDataFilterPlaneOrientationBD structure is defined as:

```
// Provided by XR_BD_spatial_plane
typedef struct XrSenseDataFilterPlaneOrientationBD {
    XrStructureType          type;
    const void*              next;
    uint32_t                 orientationCount;
    XrPlaneOrientationBD*    orientations;
} XrSenseDataFilterPlaneOrientationBD;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `orientationCount` is the count of XrPlaneOrientationBD.
- `orientations` is a pointer to the XrPlaneOrientationBD array.

The XrSenseDataFilterPlaneOrientationBD structure contains a list of plane orientations. When the

application passes this filter, all sense data that matches the criteria in the filter is included in the result returned. The runtime **must** not include sense data that does not match the provided filter.

Valid Usage (Implicit)

- The `XR_BD_spatial_plane` extension **must** be enabled prior to using XrSenseDataFilterPlaneOrientationBD
- `type` **must** be `XR_TYPE_SENSE_DATA_FILTER_PLANE_ORIENTATION_BD`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `orientations` **must** be a pointer to an array of `orientationCount` XrPlaneOrientationBD values
- The `orientationCount` parameter **must** be greater than `0`

12.90.9. New Structures

- Extending XrSenseDataQueryInfoBD:
  - XrSenseDataFilterPlaneOrientationBD
- Extending XrSpatialEntityComponentDataBaseHeaderBD:
  - XrSpatialEntityComponentDataPlaneOrientationBD
- Extending XrSystemProperties:
  - XrSystemSpatialPlanePropertiesBD

12.90.10. New Enums

- XrPlaneOrientationBD

12.90.11. New Enum Constants

- `XR_BD_SPATIAL_PLANE_EXTENSION_NAME`
- `XR_BD_spatial_plane_SPEC_VERSION`
- Extending XrSenseDataProviderTypeBD:
  - `XR_SENSE_DATA_PROVIDER_TYPE_PLANE_BD`
- Extending XrSpatialEntityComponentTypeBD:
  - `XR_SPATIAL_ENTITY_COMPONENT_TYPE_PLANE_ORIENTATION_BD`
- Extending XrStructureType:
  - `XR_TYPE_SENSE_DATA_FILTER_PLANE_ORIENTATION_BD`
  - `XR_TYPE_SPATIAL_ENTITY_COMPONENT_DATA_PLANE_ORIENTATION_BD`
  - `XR_TYPE_SYSTEM_SPATIAL_PLANE_PROPERTIES_BD`

12.90.12. Issues

12.90.13. Version History

- Revision 1, 2025-03-31 (Xu Yang)
    - Initial extension description

12.91. XR_BD_spatial_scene

Name String

`XR_BD_spatial_scene`

Extension Type

Instance extension

Registered Extension Number

393

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

`XR_BD_spatial_sensing`

Last Modified Date

2025-03-26

IP Status

No known IP claims.

Contributors

Zhipeng Liu, ByteDance
Zhao Li, ByteDance
Zijian Wang, ByteDance
Jun Yan, ByteDance
Jimmy Alamparambil, ByteDance

12.91.1. Overview

This extension allows applications to request the runtime to start capturing spatial scene information from the physical environment and query the information that the system has captured and stored.

Different objects in the scene are be represented by different component data. For example, 2D bounding boxes or polygons might be associated with walls, floors, ceilings, doors, and windows; 3D bounding boxes might be associated with tables, chairs and beds.

The general workflow for applications to use the scene capture process includes the following typical steps:

1. Initiate the scene capture process by invoking xrCaptureSceneAsyncBD. Once the call to xrCaptureSceneAsyncBD is successful, poll the returned future by using xrPollFutureEXT.
2. Typically, upon successful initiation of the scene capture process, the current application will be sent to the background. Therefore, the application **should** call xrEndSession to stop the current session upon receiving the XrEventDataSessionStateChanged event with XrEventDataSessionStateChanged::`state` set to `XR_SESSION_STATE_STOPPING`.
3. After the scene capture process is completed, the application will be brought back to the foreground. At this point, the application **should** call xrBeginSession to resume the session, when it receives the XrEventDataSessionStateChanged event with XrEventDataSessionStateChanged::`state` set to `XR_SESSION_STATE_READY`.
4. Generally, the future will be ready once the session has been restarted. If an error occurs during the scene capture process, the future result is an error code. For instance, if the scene capture process exits abnormally, an `XR_ERROR_SCENE_CAPTURE_FAILURE_BD` future result will be returned.
5. After receiving an XrEventDataSenseDataUpdatedBD, query the updated spatial scene data by calling xrQuerySenseDataAsyncBD. Subsequently, the application **can** obtain the updated data by following the steps outlined in `XR_BD_spatial_sensing`.

This extension requires `XR_BD_spatial_sensing` to also be enabled.

12.91.2. Permission

A runtime on an AOSP-based platform **must** verify that applications have the `com.picovr.permission.SPATIAL_DATA` permission both listed in their manifest and granted to use spatial scene capture. Without it, runtime **must** set XrFutureCompletionEXT::`futureResult` to `XR_ERROR_PERMISSION_INSUFFICIENT` when the xrStartSenseDataProviderCompleteBD is called.

This is an auto-requested permission: if it is listed in the manifest but not yet granted or denied, the runtime **must** prompt the user to grant or deny the permission when xrCreateSenseDataProviderBD is called with a provider type that requires it.

This permission is also used by `XR_BD_spatial_anchor`.

This permission is also used by `XR_BD_spatial_mesh`.

This permission is also used by `XR_BD_spatial_plane`.

12.91.3. Inspect System Capability

The XrSystemSpatialScenePropertiesBD structure is defined as:

```
// Provided by XR_BD_spatial_scene
typedef struct XrSystemSpatialScenePropertiesBD {
    XrStructureType    type;
    void*              next;
    XrBool32           supportsSpatialScene;
} XrSystemSpatialScenePropertiesBD;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. For more details about the structure chain, see the structure being extended (XrSystemProperties).
- `supportsSpatialScene` is an `XrBool32`, indicating if the current system is capable of spatial scene functionality.

An application **can** inspect whether the system is capable of spatial scene functionality by chaining an XrSystemSpatialScenePropertiesBD structure to the XrSystemProperties::`next` chain when calling xrGetSystemProperties.

If a runtime returns `XR_FALSE` for `supportsSpatialScene`, the system does not support spatial scene functionality, and **must** return `XR_ERROR_FEATURE_UNSUPPORTED` from xrCreateSenseDataProviderBD when passing the XrSenseDataProviderTypeBD value `XR_SENSE_DATA_PROVIDER_TYPE_SCENE_BD`.

The application **should** avoid using spatial scene functionality when `supportsSpatialScene` is `XR_FALSE`.

If XrSystemSpatialSensingPropertiesBD::`supportsSpatialSensing` is `XR_FALSE`, then `supportsSpatialScene` **must** also be `XR_FALSE`.

If a runtime returns `XR_TRUE` for `supportsSpatialScene`, the system supports spatial scene functionality. This implies that XrSystemSpatialSensingPropertiesBD::`supportsSpatialSensing` **must** also be `XR_TRUE`.

Note that `supportsSpatialScene` **may** be `XR_TRUE` even if running on an Android-based platform and the application does not have the required com.picovr.permission.SPATIAL_DATA permission both declared in the manifest and granted at runtime. Evaluation of permissions takes place later, in the asynchronous operation started by xrStartSenseDataProviderAsyncBD.

Valid Usage (Implicit)

- The XR_BD_spatial_scene extension **must** be enabled prior to using XrSystemSpatialScenePropertiesBD
- type **must** be XR_TYPE_SYSTEM_SPATIAL_SCENE_PROPERTIES_BD
- next **must** be NULL or a valid pointer to the next structure in a structure chain

12.91.4. Create Spatial Scene Provider

An application creates an XrSenseDataProviderBD handle representing a spatial scene data provider by calling xrCreateSenseDataProviderBD after setting XrSenseDataProviderCreateInfoBD::providerType equal to the XrSenseDataProviderTypeBD value XR_SENSE_DATA_PROVIDER_TYPE_SCENE_BD.

An application uses such a provider to obtain the spatial scene info that is captured and stored by the system scene capture process.

This provider type does not define any configuration and does not require a chained structure.

12.91.5. Start Spatial Scene Provider

Applications start the spatial scene data provider by calling xrStartSenseDataProviderAsyncBD after it is successfully created. To check the data provider state, call xrGetSenseDataProviderStateBD.

Subsequent application operations using this provider handle **must** not be performed unless the data provider state is XR_SENSE_DATA_PROVIDER_STATE_RUNNING_BD. If the data provider state is not XR_SENSE_DATA_PROVIDER_STATE_RUNNING_BD and the application needs to use the provider, the application **must** take appropriate action and try to call xrStartSenseDataProviderAsyncBD again before using the handle.

Detailed definitions and usage details are described in XR_BD_spatial_sensing.

12.91.6. Get Captured Scene Info

Applications query the latest captured scene information from the spatial scene data provider by calling xrQuerySenseDataAsyncBD. The runtime generates a snapshot of the spatial scene information, from which the application **can** obtain detailed scene information. The scene information is presented in the form of spatial entities which the application queries by calling xrGetQueriedSenseDataBD. Use xrEnumerateSpatialEntityComponentTypesBD to get the types of components contained within these entities, such as location, semantics, and bounding boxes. To further retrieve component information, call xrGetSpatialEntityComponentDataBD.

When the scene information changes, the runtime **must** queue an XrEventDataSenseDataUpdatedBD event with the handle of the spatial scene provider.

When the application receives this event, it means the sense data provider has updated scene data. A new query request for this sense data provider will get the latest, updated scene data. This is the recommended way to get updated data. Without receiving this event, the application also **may** initiate a new query request at any time, but the queried data **may** be unmodified.

All the functions to get captured scene info are defined in `XR_BD_spatial_sensing`.

12.91.7. Start Scene Capture

The xrCaptureSceneAsyncBD function is defined as:

```
// Provided by XR_BD_spatial_scene
XrResult xrCaptureSceneAsyncBD(
    XrSenseDataProviderBD                       provider,
    const XrSceneCaptureInfoBD*                 info,
    XrFutureEXT*                                future);
```

Parameter Descriptions

- `provider` is the previously created XrSenseDataProviderBD for spatial scene.
- `info` is a pointer to the XrSceneCaptureInfoBD.
- `future` is a pointer to the `XrFutureEXT`.

The application starts the scene capture process, which guides the user to capture the scene of the physical environment, by calling xrCaptureSceneAsyncBD.

This function starts an asynchronous operation and creates a corresponding `XrFutureEXT`, usable with xrPollFutureEXT and related functions. The return value of this function only indicates whether the parameters were acceptable to schedule the asynchronous operation. The corresponding completion function is xrStartSenseDataProviderCompleteBD, usable when a future from this function is in the READY state, with outputs populated by that function in the completion structure XrFutureCompletionEXT.

Note that scene capture **may** involve interaction by the user with system user interfaces. After calling this function, the system **may** switch the current application to the scene capture process. Accordingly, the runtime **may** emit an XrEventDataSessionStateChanged event with XrEventDataSessionStateChanged::`state` set to `XR_SESSION_STATE_STOPPING`, for the application to handle as usual, before scene capture results arrive.

Valid Usage (Implicit)

- The XR_BD_spatial_scene extension **must** be enabled prior to calling xrCaptureSceneAsyncBD
- provider **must** be a valid XrSenseDataProviderBD handle
- info **must** be a pointer to a valid XrSceneCaptureInfoBD structure
- future **must** be a pointer to an XrFutureEXT value

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FEATURE_UNSUPPORTED
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_SPATIAL_SENSING_SERVICE_UNAVAILABLE_BD
- XR_ERROR_VALIDATION_FAILURE

The XrSceneCaptureInfoBD structure is defined as:

```
// Provided by XR_BD_spatial_scene
typedef struct XrSceneCaptureInfoBD {
    XrStructureType    type;
    const void*        next;
} XrSceneCaptureInfoBD;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.

This structure is defined for future extension.

Valid Usage (Implicit)

- The XR_BD_spatial_scene extension **must** be enabled prior to using XrSceneCaptureInfoBD
- type **must** be XR_TYPE_SCENE_CAPTURE_INFO_BD
- next **must** be NULL or a valid pointer to the next structure in a structure chain

The xrCaptureSceneCompleteBD function is defined as:

```
// Provided by XR_BD_spatial_scene
XrResult xrCaptureSceneCompleteBD(
    XrSenseDataProviderBD                       provider,
    XrFutureEXT                                 future,
    XrFutureCompletionEXT*                      completion);
```

Parameter Descriptions

- provider is the XrSenseDataProviderBD.
- future is the XrFutureEXT.
- completion is a pointer to the XrFutureCompletionEXT.

The application obtains the scene capture starting result using xrCaptureSceneCompleteBD.

This is the completion function corresponding to the operation started by xrCaptureSceneAsyncBD. Do not call until the future is READY.

The XrFutureCompletionEXT structure is defined in XR_EXT_future.

Future Return Codes

XrFutureCompletionEXT::`futureResult` values:

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_SCENE_CAPTURE_FAILURE_BD`

Valid Usage (Implicit)

- The `XR_BD_spatial_scene` extension **must** be enabled prior to calling xrCaptureSceneCompleteBD
- `provider` **must** be a valid XrSenseDataProviderBD handle
- `completion` **must** be a pointer to an XrFutureCompletionEXT structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FEATURE_UNSUPPORTED
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_FUTURE_INVALID_EXT
- XR_ERROR_FUTURE_PENDING_EXT
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_SPATIAL_SENSING_SERVICE_UNAVAILABLE_BD
- XR_ERROR_VALIDATION_FAILURE

12.91.8. New Commands

- xrCaptureSceneAsyncBD
- xrCaptureSceneCompleteBD

12.91.9. New Structures

- XrSceneCaptureInfoBD
- Extending XrSystemProperties:
  - XrSystemSpatialScenePropertiesBD

12.91.10. New Enum Constants

- XR_BD_SPATIAL_SCENE_EXTENSION_NAME
- XR_BD_spatial_scene_SPEC_VERSION
- Extending XrResult:
  - XR_ERROR_SCENE_CAPTURE_FAILURE_BD
- Extending XrSenseDataProviderTypeBD:

    - XR_SENSE_DATA_PROVIDER_TYPE_SCENE_BD
- Extending XrStructureType:
    - XR_TYPE_SCENE_CAPTURE_INFO_BD
    - XR_TYPE_SYSTEM_SPATIAL_SCENE_PROPERTIES_BD

12.91.11. Issues

12.91.12. Version History

- Revision 1, 2024-05-06 (Zhipeng Liu)
    - Initial extension description

12.92. XR_BD_spatial_sensing

Name String

XR_BD_spatial_sensing

Extension Type

Instance extension

Registered Extension Number

390

Revision

2

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0
and
XR_EXT_future

Last Modified Date

2025-08-05

IP Status

No known IP claims.

Contributors

Zhipeng Liu, ByteDance
Ya Huang, ByteDance

Xiangxin Liu, ByteDance
Zijian Wang, ByteDance
Zhao Li, ByteDance
Zhanrui Jia, ByteDance
Xu Yang, ByteDance
Jun Yan, ByteDance
Jimmy Alamparambil, ByteDance

12.92.1. Overview

This extension introduces a spatial sensing API framework which contains a set of common functions and data structures for various spatial sensing capabilities. In this extension, the EC (Entity-Component) pattern is used to denote the sensed object in the physical environment via spatial entity.

A spatial entity **may** be points of interest, physical plane surfaces, physical objects, mesh blocks, images, etc., and it **may** have different *components*. Having different components indicates that different data are attached to the spatial entity and different operations **can** be performed on it. Applications **can** get the component types that a given spatial entity has as well as the information of a specific component.

Different kinds of spatial sensing data are provided via different sense data providers. Applications **can** get the sensed spatial entity details from sense data providers.

The general workflow for applications to includes the following typical steps:

1. Create an XrSenseDataProviderBD with relevant configuration information via xrCreateSenseDataProviderBD.
2. Start the XrSenseDataProviderBD to let the runtime provide sensed data for that provider via xrStartSenseDataProviderAsyncBD.
3. Get the state of XrSenseDataProviderBD via xrGetSenseDataProviderStateBD, and make sure it is in the running state.
4. Initiate a query request to the XrSenseDataProviderBD via xrQuerySenseDataAsyncBD after receiving an XrEventDataSenseDataUpdatedBD, and the runtime will produce an XrSenseDataSnapshotBD, which contains the most recent data associated with this provider.
5. Retrieve the spatial entity info as well as the detailed component data from the XrSenseDataSnapshotBD via xrGetQueriedSenseDataBD and xrGetSpatialEntityComponentDataBD.
6. Destroy the XrSenseDataSnapshotBD after all the data infos are retrieved via xrDestroySenseDataSnapshotBD to release related resources. Applications **can** repeat step 4-6 to get the updated sense data if necessary.
7. Stop the XrSenseDataProviderBD via xrStopSenseDataProviderBD.
8. Destroy the XrSenseDataProviderBD via xrDestroySenseDataProviderBD to release the system resources associated with the provider.



Note

If any function returns an XR_ERROR_SPATIAL_SENSING_SERVICE_UNAVAILABLE_BD, the service is unable to recover by itself, and the application **should** destroy the current XrInstance and create it again to use the service.

12.92.2. Inspect System Capability

The XrSystemSpatialSensingPropertiesBD structure is defined as:

```
// Provided by XR_BD_spatial_sensing
typedef struct XrSystemSpatialSensingPropertiesBD {
    XrStructureType    type;
    void*              next;
    XrBool32           supportsSpatialSensing;
} XrSystemSpatialSensingPropertiesBD;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. For more details about the structure chain, see the structure being extended (XrSystemProperties).
- supportsSpatialSensing is an XrBool32, indicating if current system is capable of spatial sensing.

An application **can** inspect whether the system is capable of spatial sensing by chaining an XrSystemSpatialSensingPropertiesBD structure to the XrSystemProperties::next chain when calling xrGetSystemProperties.

If a runtime returns XR_FALSE for supportsSpatialSensing, the system does not support spatial sensing, and the runtime **must** return XR_ERROR_FEATURE_UNSUPPORTED from xrCreateSenseDataProviderBD.

If a runtime returns XR_TRUE for supportsSpatialSensing, the system supports spatial sensing.

Valid Usage (Implicit)

- The XR_BD_spatial_sensing extension **must** be enabled prior to using XrSystemSpatialSensingPropertiesBD
- type **must** be XR_TYPE_SYSTEM_SPATIAL_SENSING_PROPERTIES_BD
- next **must** be NULL or a valid pointer to the next structure in a structure chain

12.92.3. Create Sense Data Provider

The XrSenseDataProviderBD handle is defined as:

```
// Provided by XR_BD_spatial_sensing
XR_DEFINE_HANDLE(XrSenseDataProviderBD)
```

XrSenseDataProviderBD is a handle which the application creates to do spatial sensing for the user's physical environment.

The xrCreateSenseDataProviderBD function is defined as:

```
// Provided by XR_BD_spatial_sensing
XrResult xrCreateSenseDataProviderBD(
    XrSession                                   session,
    const XrSenseDataProviderCreateInfoBD*      createInfo,
    XrSenseDataProviderBD*                      provider);
```

Parameter Descriptions

- `session` is the XrSession.
- `createInfo` is a pointer to a valid XrSenseDataProviderCreateInfoBD.
- `provider` is a pointer to the XrSenseDataProviderBD that the runtime created.

The application **can** create a sense data provider using xrCreateSenseDataProviderBD. Different providers are associated with different XrSenseDataProviderTypeBD provider type values in XrSenseDataProviderCreateInfoBD::`providerType`. Some require additional parameters in structure types chained on to `createInfo`.

Valid Usage (Implicit)

- The `XR_BD_spatial_sensing` extension **must** be enabled prior to calling xrCreateSenseDataProviderBD
- `session` **must** be a valid XrSession handle
- `createInfo` **must** be a pointer to a valid XrSenseDataProviderCreateInfoBD structure
- `provider` **must** be a pointer to an XrSenseDataProviderBD handle

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FEATURE_UNSUPPORTED
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_SPATIAL_SENSING_SERVICE_UNAVAILABLE_BD
- XR_ERROR_VALIDATION_FAILURE

The XrSenseDataProviderCreateInfoBD structure is defined as:

```
// Provided by XR_BD_spatial_sensing
typedef struct XrSenseDataProviderCreateInfoBD {
    XrStructureType              type;
    const void*                  next;
    XrSenseDataProviderTypeBD    providerType;
} XrSenseDataProviderCreateInfoBD;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain.
- providerType is an XrSenseDataProviderTypeBD indicating the type of provider to create.

When the application creates an XrSenseDataProviderBD, it **must** specify the type of provider via providerType. If the value of providerType is not valid given the current enabled extensions, the runtime **must** return XR_ERROR_VALIDATION_FAILURE. Some provider types have additional parameters passed via the XrSenseDataProviderCreateInfoBD::next chain.

If a valid provider type is specified in `providerType` but that type requires an additional configuration structure that is not provided in the `next` chain, the runtime **must** return `XR_ERROR_VALIDATION_FAILURE`. Note that in accordance with Valid Usage for Structure Pointer Chains, configuration structures in the `next` chain for provider types other than the one indicated by `providerType` are ignored.

Valid Usage (Implicit)

- The `XR_BD_spatial_sensing` extension **must** be enabled prior to using XrSenseDataProviderCreateInfoBD
- `type` **must** be `XR_TYPE_SENSE_DATA_PROVIDER_CREATE_INFO_BD`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain. See also: XrSenseDataProviderCreateInfoDynamicObjectBD, XrSenseDataProviderCreateInfoSpatialMeshBD
- `providerType` **must** be a valid XrSenseDataProviderTypeBD value

The XrSenseDataProviderTypeBD enumeration is defined as:

```
// Provided by XR_BD_spatial_sensing
typedef enum XrSenseDataProviderTypeBD {
  // Provided by XR_BD_spatial_anchor
    XR_SENSE_DATA_PROVIDER_TYPE_ANCHOR_BD = 1000390000,
  // Provided by XR_BD_spatial_scene
    XR_SENSE_DATA_PROVIDER_TYPE_SCENE_BD = 1000392000,
  // Provided by XR_BD_spatial_mesh
    XR_SENSE_DATA_PROVIDER_TYPE_MESH_BD = 1000393000,
  // Provided by XR_BD_spatial_plane
    XR_SENSE_DATA_PROVIDER_TYPE_PLANE_BD = 1000396000,
  // Provided by XR_BD_dynamic_object_tracking
    XR_SENSE_DATA_PROVIDER_TYPE_DYNAMIC_OBJECT_BD = 1000746000,
    XR_SENSE_DATA_PROVIDER_TYPE_MAX_ENUM_BD = 0x7FFFFFFF
} XrSenseDataProviderTypeBD;
```

Enum	Description
`XR_SENSE_DATA_PROVIDER_TYPE_ANCHOR_BD`	Create arbitrary spatial anchors. (Added by the `XR_BD_spatial_anchor` extension)
`XR_SENSE_DATA_PROVIDER_TYPE_SCENE_BD`	Access spatial scene capture data. (Added by the `XR_BD_spatial_scene` extension)
`XR_SENSE_DATA_PROVIDER_TYPE_MESH_BD`	Capture spatial mesh data. (Added by the `XR_BD_spatial_mesh` extension)

Enum	Description
XR_SENSE_DATA_PROVIDER_TYPE_PLANE_BD	Capture spatial plane data. (Added by the XR_BD_spatial_plane extension)
XR_SENSE_DATA_PROVIDER_TYPE_DYNAMIC_OBJECT_BD	Capture dynamic object data. (Added by the XR_BD_dynamic_object_tracking extension)

12.92.4. Start Sense Data Provider

The xrStartSenseDataProviderAsyncBD function is defined as:

```
// Provided by XR_BD_spatial_sensing
XrResult xrStartSenseDataProviderAsyncBD(
    XrSenseDataProviderBD                       provider,
    const XrSenseDataProviderStartInfoBD*       startInfo,
    XrFutureEXT*                                future);
```

Parameter Descriptions

- provider is the sense data provider handle to be started.
- startInfo is a pointer to the XrSenseDataProviderStartInfoBD.
- future is a pointer to the XrFutureEXT.

The application **can** start a previously created sense data provider using xrStartSenseDataProviderAsyncBD when it is in either the XR_SENSE_DATA_PROVIDER_STATE_INITIALIZED_BD or the XR_SENSE_DATA_PROVIDER_STATE_STOPPED_BD state.

This function starts an asynchronous operation and creates a corresponding XrFutureEXT, usable with xrPollFutureEXT and related functions. The return value of this function only indicates whether the parameters were acceptable to schedule the asynchronous operation. The corresponding completion function is xrStartSenseDataProviderCompleteBD, usable when a future from this function is in the READY state, with outputs populated by that function in the completion structure XrFutureCompletionEXT.

If the sense data provider is already in the XR_SENSE_DATA_PROVIDER_STATE_RUNNING_BD state, calling this function does not change the provider state. In this case, the runtime **must** still set the return value of xrStartSenseDataProviderAsyncBD to XR_SUCCESS, and the future state will be XR_FUTURE_STATE_READY_EXT immediately. The runtime **must** set the corresponding future result returned by xrStartSenseDataProviderCompleteBD to XR_SUCCESS.

Individual sense data provider types **may** specify required a permission or permissions to use their capabilities on AOSP-based platforms. If the specified provider type requires a permission and that permission is not listed in the application's manifest, or the permission is listed but denied, the future

state will be XR_FUTURE_STATE_READY_EXT immediately and the runtime **must** set the corresponding future result returned by xrStartSenseDataProviderCompleteBD to XR_ERROR_PERMISSION_INSUFFICIENT.

Valid Usage (Implicit)

- The XR_BD_spatial_sensing extension **must** be enabled prior to calling xrStartSenseDataProviderAsyncBD
- provider **must** be a valid XrSenseDataProviderBD handle
- startInfo **must** be a pointer to a valid XrSenseDataProviderStartInfoBD structure
- future **must** be a pointer to an XrFutureEXT value

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FEATURE_UNSUPPORTED
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_SPATIAL_SENSING_SERVICE_UNAVAILABLE_BD
- XR_ERROR_VALIDATION_FAILURE

The XrSenseDataProviderStartInfoBD structure is defined as:

```
// Provided by XR_BD_spatial_sensing
typedef struct XrSenseDataProviderStartInfoBD {
    XrStructureType    type;
    const void*        next;
} XrSenseDataProviderStartInfoBD;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.

This structure is defined for future extension.

Valid Usage (Implicit)

- The `XR_BD_spatial_sensing` extension **must** be enabled prior to using XrSenseDataProviderStartInfoBD
- `type` **must** be `XR_TYPE_SENSE_DATA_PROVIDER_START_INFO_BD`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

The xrStartSenseDataProviderCompleteBD function is defined as:

```
// Provided by XR_BD_spatial_sensing
XrResult xrStartSenseDataProviderCompleteBD(
    XrSession                                   session,
    XrFutureEXT                                 future,
    XrFutureCompletionEXT*                      completion);
```

Parameter Descriptions

- `session` is the XrSession.
- `future` is the `XrFutureEXT` previously returned by xrStartSenseDataProviderAsyncBD.
- `completion` is a pointer to the XrFutureCompletionEXT.

The application **can** obtain the sense data provider start result using xrStartSenseDataProviderCompleteBD.

This is the completion function corresponding to the operation started by xrStartSenseDataProviderAsyncBD. Do not call until the `future` is READY.

The XrFutureCompletionEXT structure is defined in `XR_EXT_future`.

Future Return Codes

XrFutureCompletionEXT::`futureResult` values:

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_OUT_OF_MEMORY`
- `XR_ERROR_PERMISSION_INSUFFICIENT`

Subsequent application operations using this handle **must** not be performed unless the XrFutureCompletionEXT::`futureResult` is `XR_SUCCESS`. For example, if the XrFutureCompletionEXT::`futureResult` is `XR_ERROR_PERMISSION_INSUFFICIENT`, it indicates that the application does not have sufficient permissions and needs to apply for the corresponding permissions.

Valid Usage (Implicit)

- The `XR_BD_spatial_sensing` extension **must** be enabled prior to calling xrStartSenseDataProviderCompleteBD
- `session` **must** be a valid XrSession handle
- `completion` **must** be a pointer to an XrFutureCompletionEXT structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FEATURE_UNSUPPORTED
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_FUTURE_INVALID_EXT
- XR_ERROR_FUTURE_PENDING_EXT
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_SPATIAL_SENSING_SERVICE_UNAVAILABLE_BD
- XR_ERROR_VALIDATION_FAILURE

12.92.5. Get Sense Data Provider State

The xrGetSenseDataProviderStateBD function is defined as:

```
// Provided by XR_BD_spatial_sensing
XrResult xrGetSenseDataProviderStateBD(
    XrSenseDataProviderBD                       provider,
    XrSenseDataProviderStateBD*                 state);
```

Parameter Descriptions

- provider is the XrSenseDataProviderBD handle.
- state is a pointer to the XrSenseDataProviderStateBD.

The application **can** get a sense data provider's state by calling xrGetSenseDataProviderStateBD.



Figure 29. XrSenseDataProviderStateBD

Valid Usage (Implicit)

- The `XR_BD_spatial_sensing` extension **must** be enabled prior to calling xrGetSenseDataProviderStateBD
- `provider` **must** be a valid XrSenseDataProviderBD handle
- `state` **must** be a pointer to an XrSenseDataProviderStateBD value

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_FEATURE_UNSUPPORTED`
- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_SPATIAL_SENSING_SERVICE_UNAVAILABLE_BD`
- `XR_ERROR_VALIDATION_FAILURE`

The XrSenseDataProviderStateBD enumeration identifies the various states of the sense data provider.

```
// Provided by XR_BD_spatial_sensing
typedef enum XrSenseDataProviderStateBD {
    XR_SENSE_DATA_PROVIDER_STATE_INITIALIZED_BD = 0,
    XR_SENSE_DATA_PROVIDER_STATE_RUNNING_BD = 1,
    XR_SENSE_DATA_PROVIDER_STATE_STOPPED_BD = 2,
    XR_SENSE_DATA_PROVIDER_STATE_MAX_ENUM_BD = 0x7FFFFFFF
} XrSenseDataProviderStateBD;
```

Enum	Description
`XR_SENSE_DATA_PROVIDER_STATE_INITIALIZED_BD`	The state after the provider is successfully created, which means the provider is ready to start.

Enum	Description
XR_SENSE_DATA_PROVIDER_STATE_RUNNING_BD	The state when the provider is running normally.
XR_SENSE_DATA_PROVIDER_STATE_STOPPED_BD	The state after the provider is successfully stopped, or when an unexpected error occurs.

The XrEventDataSenseDataProviderStateChangedBD structure is defined as:

```
// Provided by XR_BD_spatial_sensing
typedef struct XrEventDataSenseDataProviderStateChangedBD {
    XrStructureType               type;
    const void*                   next;
    XrSenseDataProviderBD         provider;
    XrSenseDataProviderStateBD    newState;
} XrEventDataSenseDataProviderStateChangedBD;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- provider is the XrSenseDataProviderBD handle.
- newState is the new XrSenseDataProviderStateBD of the provider.

This event indicates the state change of the sense data provider. The runtime **must** queue an event of type XR_TYPE_EVENT_DATA_SENSE_DATA_PROVIDER_STATE_CHANGED_BD when a specific sense data provider changes from one state to another state. When the application calls xrCreateSenseDataProviderBD and the return code is XR_SUCCESS, the runtime **must** set the provider's state to XR_SENSE_DATA_PROVIDER_STATE_INITIALIZED_BD without queuing an event of type XR_TYPE_EVENT_DATA_SENSE_DATA_PROVIDER_STATE_CHANGED_BD.

Valid Usage (Implicit)

- The XR_BD_spatial_sensing extension **must** be enabled prior to using XrEventDataSenseDataProviderStateChangedBD
- type **must** be XR_TYPE_EVENT_DATA_SENSE_DATA_PROVIDER_STATE_CHANGED_BD
- next **must** be NULL or a valid pointer to the next structure in a structure chain

12.92.6. Query Sense Data

When a sense data provider is running, the application **can** initiate a query that creates a snapshot of the sense data from the current sense data provider.

The XrSenseDataSnapshotBD handle is defined as:

```
// Provided by XR_BD_spatial_sensing
XR_DEFINE_HANDLE(XrSenseDataSnapshotBD)
```

The xrQuerySenseDataAsyncBD function is defined as:

```
// Provided by XR_BD_spatial_sensing
XrResult xrQuerySenseDataAsyncBD(
    XrSenseDataProviderBD                       provider,
    const XrSenseDataQueryInfoBD*               queryInfo,
    XrFutureEXT*                                future);
```

Parameter Descriptions

- `provider` is the XrSenseDataProviderBD handle to query.
- `queryInfo` is the XrSenseDataQueryInfoBD.
- `future` is the pointer to the returned `XrFutureEXT`.

The application **can** query a sense data provider by calling xrQuerySenseDataAsyncBD.

The application **must** not call this function to initiate a query request for the spatial sense data unless the xrStartSenseDataProviderCompleteBD has been called successfully, the future completed, and returned a future result of `XR_SUCCESS`. Otherwise, the runtime **must** return `XR_ERROR_CALL_ORDER_INVALID`. The query result is a snapshot of the sense data from the current sense data provider.

This function starts an asynchronous operation and creates a corresponding `XrFutureEXT`, usable with xrPollFutureEXT and related functions. The return value of this function only indicates whether the parameters were acceptable to schedule the asynchronous operation. The corresponding completion function is xrQuerySenseDataCompleteBD, usable when a future from this function is in the READY state, with outputs populated by that function in the completion structure XrSenseDataQueryCompletionBD.

Valid Usage (Implicit)

- The `XR_BD_spatial_sensing` extension **must** be enabled prior to calling xrQuerySenseDataAsyncBD
- `provider` **must** be a valid XrSenseDataProviderBD handle
- `queryInfo` **must** be a pointer to a valid XrSenseDataQueryInfoBD structure
- `future` **must** be a pointer to an `XrFutureEXT` value

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_CALL_ORDER_INVALID`
- `XR_ERROR_FEATURE_UNSUPPORTED`
- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_SPATIAL_SENSING_SERVICE_UNAVAILABLE_BD`
- `XR_ERROR_VALIDATION_FAILURE`

The XrSenseDataQueryInfoBD structure is defined as:

```
// Provided by XR_BD_spatial_sensing
typedef struct XrSenseDataQueryInfoBD {
    XrStructureType    type;
    const void*        next;
} XrSenseDataQueryInfoBD;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.

When the application initiates a query request, it **can** chain a `XrSenseDataFilter*` structure to XrSenseDataQueryInfoBD::`next`. Then all sense data that matches the criteria in the filter is included in the result returned. The runtime **must** not include sense data that does not match the provided filter.

If no `XrSenseDataFilter*` structure is chained to XrSenseDataQueryInfoBD::`next`, it means no filter is applied, and runtime **must** include all currently detected sense data of this provider in the result.



Note

Currently, only the first filter in the XrSenseDataQueryInfoBD::`next` chain takes effect.

Valid Usage (Implicit)

- The `XR_BD_spatial_sensing` extension **must** be enabled prior to using XrSenseDataQueryInfoBD
- `type` **must** be `XR_TYPE_SENSE_DATA_QUERY_INFO_BD`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain. See also: XrSenseDataFilterDynamicObjectTypeBD, XrSenseDataFilterPlaneOrientationBD, XrSenseDataFilterSemanticBD, XrSenseDataFilterUuidBD

Some commonly used filters are defined in this extension.

The XrSenseDataFilterUuidBD structure is defined as:

```
// Provided by XR_BD_spatial_sensing
typedef struct XrSenseDataFilterUuidBD {
    XrStructureType     type;
    const void*         next;
    uint32_t            uuidCount;
    const XrUuidEXT*    uuids;
} XrSenseDataFilterUuidBD;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `uuidCount` is the count of XrUuidEXT.
- `uuids` is a pointer to the XrUuidEXT array.

The XrSenseDataFilterUuidBD contains a list of UUIDs. When the application passes this filter, the query result will only contain entities with UUIDs that appear in this list.

Valid Usage (Implicit)

- The `XR_BD_spatial_sensing` extension **must** be enabled prior to using XrSenseDataFilterUuidBD
- `type` **must** be `XR_TYPE_SENSE_DATA_FILTER_UUID_BD`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `uuids` **must** be a pointer to an array of `uuidCount` XrUuidEXT structures
- The `uuidCount` parameter **must** be greater than `0`

The XrSenseDataFilterSemanticBD structure is defined as:

```
// Provided by XR_BD_spatial_sensing
typedef struct XrSenseDataFilterSemanticBD {
    XrStructureType             type;
    const void*                 next;
    uint32_t                    labelCount;
    const XrSemanticLabelBD*    labels;
} XrSenseDataFilterSemanticBD;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `labelCount` is the number of XrSemanticLabelBD elements in `labels`.
- `labels` is a pointer to the XrSemanticLabelBD array.

The XrSenseDataFilterSemanticBD contains a list of semantic labels. When the application passes this

filter, the query result will only contain entities with semantic labels that appear in this list.

Valid Usage (Implicit)

- The XR_BD_spatial_sensing extension **must** be enabled prior to using XrSenseDataFilterSemanticBD
- `type` **must** be `XR_TYPE_SENSE_DATA_FILTER_SEMANTIC_BD`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `labels` **must** be a pointer to an array of `labelCount` valid XrSemanticLabelBD values
- The `labelCount` parameter **must** be greater than `0`

The xrQuerySenseDataCompleteBD function is defined as:

```
// Provided by XR_BD_spatial_sensing
XrResult xrQuerySenseDataCompleteBD(
    XrSenseDataProviderBD                       provider,
    XrFutureEXT                                 future,
    XrSenseDataQueryCompletionBD*               completion);
```

Parameter Descriptions

- `provider` is the XrSenseDataProviderBD.
- `future` is the `XrFutureEXT` previously returned by xrQuerySenseDataAsyncBD.
- `completion` is the XrSenseDataQueryCompletionBD.

This is the completion function corresponding to xrQuerySenseDataAsyncBD. It completes the asynchronous operation and returns the results. Do not call until the `future` is READY.

Valid Usage (Implicit)

- The XR_BD_spatial_sensing extension **must** be enabled prior to calling xrQuerySenseDataCompleteBD
- `provider` **must** be a valid XrSenseDataProviderBD handle
- `completion` **must** be a pointer to an XrSenseDataQueryCompletionBD structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FEATURE_UNSUPPORTED
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_FUTURE_INVALID_EXT
- XR_ERROR_FUTURE_PENDING_EXT
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_SPATIAL_SENSING_SERVICE_UNAVAILABLE_BD
- XR_ERROR_VALIDATION_FAILURE

The XrSenseDataQueryCompletionBD structure is defined as:

```
// Provided by XR_BD_spatial_sensing
typedef struct XrSenseDataQueryCompletionBD {
    XrStructureType          type;
    void*                    next;
    XrResult                 futureResult;
    XrSenseDataSnapshotBD    snapshot;
} XrSenseDataQueryCompletionBD;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- futureResult is the XrResult of the future.
- snapshot is the XrSenseDataSnapshotBD handle of the query result.

If the futureResult is not XR_SUCCESS, the runtime **must** set snapshot to XR_NULL_HANDLE.

Future Return Codes

`futureResult` values:

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_OUT_OF_MEMORY`
- `XR_ERROR_LIMIT_REACHED`

Valid Usage (Implicit)

- The `XR_BD_spatial_sensing` extension **must** be enabled prior to using XrSenseDataQueryCompletionBD
- `type` **must** be `XR_TYPE_SENSE_DATA_QUERY_COMPLETION_BD`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `futureResult` **must** be a valid XrResult value
- `snapshot` **must** be a valid XrSenseDataSnapshotBD handle

The xrDestroySenseDataSnapshotBD function is defined as:

```
// Provided by XR_BD_spatial_sensing
XrResult xrDestroySenseDataSnapshotBD(
    XrSenseDataSnapshotBD                       snapshot);
```

Parameter Descriptions

- `snapshot` is the XrSenseDataSnapshotBD to be destroyed.

The application **can** destroy a sense data snapshot using xrDestroySenseDataSnapshotBD.

After all the spatial entity info are retrieved from the snapshot, application **should** destroy the

XrSenseDataSnapshotBD handle to release related resources. The runtime **may** have an upper limit on the number of snapshots it supports. When the application exceeds the runtime's limit, the runtime **must** set XrSenseDataQueryCompletionBD::`futureResult` to `XR_ERROR_LIMIT_REACHED` when xrQuerySenseDataCompleteBD is called. To obtain new snapshots once the limit is reached, destroy at least one of the previously obtained snapshots.

Valid Usage (Implicit)

- The `XR_BD_spatial_sensing` extension **must** be enabled prior to calling xrDestroySenseDataSnapshotBD
- `snapshot` **must** be a valid XrSenseDataSnapshotBD handle

Thread Safety

- Access to `snapshot`, and any child handles, **must** be externally synchronized

Return Codes

Success

- `XR_SUCCESS`

Failure

- `XR_ERROR_FEATURE_UNSUPPORTED`
- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SPATIAL_SENSING_SERVICE_UNAVAILABLE_BD`

12.92.7. Spatial Entity

The `XrSpatialEntityIdBD` atom is defined as:

```
// Provided by XR_BD_spatial_sensing
XR_DEFINE_ATOM(XrSpatialEntityIdBD)
```

The `XrSpatialEntityIdBD` is used to represent the spatial entities that are sensed by the runtime in the user's physical environment. The application **can** obtain spatial entity information from the XrSenseDataSnapshotBD. This atom is retrieved from XrSenseDataSnapshotBD and shares its lifetime.

The application **must** not use an XrSpatialEntityIdBD with a snapshot it was not retrieved from. The same numerical value of ID **may** be reused between different snapshots for different entities. See xrGetSpatialEntityUuidBD for a more stable ID.

12.92.8. Spatial Entity UUID

An XrUuidEXT is generated by the runtime when it creates a spatial entity, and the runtime **must** guarantee the UUID refers to the same spatial entity for the whole life of the spatial entity. For spatial entities that are persisted, the runtime **must** guarantee the UUID remain unchanged across application sessions.

The xrGetSpatialEntityUuidBD function is defined as:

```
// Provided by XR_BD_spatial_sensing
XrResult xrGetSpatialEntityUuidBD(
    XrSenseDataSnapshotBD                       snapshot,
    XrSpatialEntityIdBD                         entityId,
    XrUuidEXT*                                  uuid);
```

Parameter Descriptions

- `snapshot` is the XrSenseDataSnapshotBD handle.
- `entityId` is the XrSpatialEntityIdBD of a spatial entity.
- `uuid` is an output parameter XrUuidEXT pointer to the spatial entity's UUID.

Applications **can** get the UUID for a given spatial entity by calling xrGetSpatialEntityUuidBD.

Valid Usage (Implicit)

- The `XR_BD_spatial_sensing` extension **must** be enabled prior to calling xrGetSpatialEntityUuidBD
- `snapshot` **must** be a valid XrSenseDataSnapshotBD handle
- `uuid` **must** be a pointer to an XrUuidEXT structure

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_FEATURE_UNSUPPORTED`
- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_SPATIAL_ENTITY_ID_INVALID_BD`
- `XR_ERROR_SPATIAL_SENSING_SERVICE_UNAVAILABLE_BD`
- `XR_ERROR_VALIDATION_FAILURE`

12.92.9. Get Sense Data Info

The xrGetQueriedSenseDataBD function is defined as:

```
// Provided by XR_BD_spatial_sensing
XrResult xrGetQueriedSenseDataBD(
    XrSenseDataSnapshotBD                       snapshot,
    XrQueriedSenseDataGetInfoBD*                getInfo,
    XrQueriedSenseDataBD*                       queriedSenseData);
```

Parameter Descriptions

- `snapshot` is the XrSenseDataSnapshotBD.
- `getInfo` is a pointer to the XrQueriedSenseDataGetInfoBD.
- `queriedSenseData` is a pointer to the XrQueriedSenseDataBD.

The application **can** get queried sense data from the XrSenseDataSnapshotBD using xrGetQueriedSenseDataBD. `queriedSenseData` contains an application-allocated array that is populated by the runtime, after capacity negotiation using the two-call idiom, with entity ID, UUID, and last update times for each entity in the snapshot.

Valid Usage (Implicit)

- The XR_BD_spatial_sensing extension **must** be enabled prior to calling xrGetQueriedSenseDataBD
- `snapshot` **must** be a valid XrSenseDataSnapshotBD handle
- `getInfo` **must** be a pointer to an XrQueriedSenseDataGetInfoBD structure
- `queriedSenseData` **must** be a pointer to an XrQueriedSenseDataBD structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FEATURE_UNSUPPORTED
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_SIZE_INSUFFICIENT
- XR_ERROR_SPATIAL_SENSING_SERVICE_UNAVAILABLE_BD
- XR_ERROR_VALIDATION_FAILURE

The XrQueriedSenseDataGetInfoBD structure is defined as:

```
// Provided by XR_BD_spatial_sensing
typedef struct XrQueriedSenseDataGetInfoBD {
    XrStructureType    type;
    const void*        next;
} XrQueriedSenseDataGetInfoBD;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.

Valid Usage (Implicit)

- The `XR_BD_spatial_sensing` extension **must** be enabled prior to using XrQueriedSenseDataGetInfoBD
- `type` **must** be `XR_TYPE_QUERIED_SENSE_DATA_GET_INFO_BD`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

The XrQueriedSenseDataBD structure is defined as:

```
// Provided by XR_BD_spatial_sensing
typedef struct XrQueriedSenseDataBD {
    XrStructureType            type;
    void*                      next;
    uint32_t                   stateCapacityInput;
    uint32_t                   stateCountOutput;
    XrSpatialEntityStateBD*    states;
} XrQueriedSenseDataBD;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `stateCapacityInput` is the capacity of the `states` array, or 0 to indicate a request to retrieve the required capacity.
- `stateCountOutput` is a pointer to the count of `states` written, or a pointer to the required capacity in the case that `stateCapacityInput` is insufficient.
- `states` is an array of XrSpatialEntityStateBD filled in by the runtime, specifying the spatial entities in the snapshot.
- See Buffer Size Parameters chapter for a detailed description of retrieving the required `states` size.

Valid Usage (Implicit)

- The XR_BD_spatial_sensing extension **must** be enabled prior to using XrQueriedSenseDataBD
- type **must** be XR_TYPE_QUERIED_SENSE_DATA_BD
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- If stateCapacityInput is not 0, states **must** be a pointer to an array of stateCapacityInput XrSpatialEntityStateBD structures

The XrSpatialEntityStateBD structure is defined as:

```
// Provided by XR_BD_spatial_sensing
typedef struct XrSpatialEntityStateBD {
    XrStructureType        type;
    void*                  next;
    XrSpatialEntityIdBD    entityId;
    XrTime                 lastUpdateTime;
    XrUuidEXT              uuid;
} XrSpatialEntityStateBD;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- entityId is the XrSpatialEntityIdBD of a spatial entity.
- lastUpdateTime is the last XrTime timestamp at which any of the spatial entity's component data was updated.
- uuid is the XrUuidEXT of the spatial entity.

The XrSpatialEntityStateBD contains the general info of the spatial entity.

Valid Usage (Implicit)

- The XR_BD_spatial_sensing extension **must** be enabled prior to using XrSpatialEntityStateBD
- type **must** be XR_TYPE_SPATIAL_ENTITY_STATE_BD
- next **must** be NULL or a valid pointer to the next structure in a structure chain

12.92.10. Spatial Entity Component Type

A spatial entity **may** have several components which provide different data for the entity.

The xrEnumerateSpatialEntityComponentTypesBD function is defined as:

```
// Provided by XR_BD_spatial_sensing
XrResult xrEnumerateSpatialEntityComponentTypesBD(
    XrSenseDataSnapshotBD                       snapshot,
    XrSpatialEntityIdBD                         entityId,
    uint32_t                                    componentTypeCapacityInput,
    uint32_t*                                   componentTypeCountOutput,
    XrSpatialEntityComponentTypeBD*             componentTypes);
```

Parameter Descriptions

- `snapshot` is the XrSenseDataSnapshotBD handle.
- `entityId` is the `XrSpatialEntityIdBD` whose spatial entity component types will be enumerated.
- `componentTypeCapacityInput` is the capacity of the `componentTypes` array, or 0 to indicate a request to retrieve the required capacity.
- `componentTypeCountOutput` is the number of component types, or the required capacity in the case that `componentTypeCapacityInput` is insufficient.
- `componentTypes` is an array of XrSpatialEntityComponentTypeBD. It **can** be `NULL` if `componentTypeCapacityInput` is 0.
- See Buffer Size Parameters chapter for a detailed description of retrieving the required `componentTypes` size.

The application inspects the component types for a given `XrSpatialEntityIdBD` using xrEnumerateSpatialEntityComponentTypesBD. The application **may** skip enumerating component types and proceed directly to attempting to access component data.

Valid Usage (Implicit)

- The `XR_BD_spatial_sensing` extension **must** be enabled prior to calling xrEnumerateSpatialEntityComponentTypesBD
- `snapshot` **must** be a valid XrSenseDataSnapshotBD handle
- `componentTypeCountOutput` **must** be a pointer to a `uint32_t` value
- If `componentTypeCapacityInput` is not `0`, `componentTypes` **must** be a pointer to an array of `componentTypeCapacityInput` XrSpatialEntityComponentTypeBD values

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_FEATURE_UNSUPPORTED`
- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_SIZE_INSUFFICIENT`
- `XR_ERROR_SPATIAL_ENTITY_ID_INVALID_BD`
- `XR_ERROR_SPATIAL_SENSING_SERVICE_UNAVAILABLE_BD`
- `XR_ERROR_VALIDATION_FAILURE`

The XrSpatialEntityComponentTypeBD enumeration identifies the different types of components that spatial entities **may** support.

```
// Provided by XR_BD_spatial_sensing
typedef enum XrSpatialEntityComponentTypeBD {
    XR_SPATIAL_ENTITY_COMPONENT_TYPE_LOCATION_BD = 0,
    XR_SPATIAL_ENTITY_COMPONENT_TYPE_SEMANTIC_BD = 1,
    XR_SPATIAL_ENTITY_COMPONENT_TYPE_BOUNDING_BOX_2D_BD = 2,
    XR_SPATIAL_ENTITY_COMPONENT_TYPE_POLYGON_BD = 3,
    XR_SPATIAL_ENTITY_COMPONENT_TYPE_BOUNDING_BOX_3D_BD = 4,
    XR_SPATIAL_ENTITY_COMPONENT_TYPE_TRIANGLE_MESH_BD = 5,
    XR_SPATIAL_ENTITY_COMPONENT_TYPE_SPHERE_BD = 6,
  // Provided by XR_BD_spatial_plane
    XR_SPATIAL_ENTITY_COMPONENT_TYPE_PLANE_ORIENTATION_BD = 1000396000,
  // Provided by XR_BD_dynamic_object_tracking
    XR_SPATIAL_ENTITY_COMPONENT_TYPE_DYNAMIC_OBJECT_BD = 1000746000,
    XR_SPATIAL_ENTITY_COMPONENT_TYPE_MAX_ENUM_BD = 0x7FFFFFFF
} XrSpatialEntityComponentTypeBD;
```

Enum	Description
XR_SPATIAL_ENTITY_COMPONENT_TYPE_LOCATION_BD	The location including position and rotation. Corresponds to component data structure XrSpatialEntityComponentDataLocationBD.
XR_SPATIAL_ENTITY_COMPONENT_TYPE_SEMANTIC_BD	The semantic label. Corresponds to component data structure XrSpatialEntityComponentDataSemanticBD.
XR_SPATIAL_ENTITY_COMPONENT_TYPE_BOUNDING_BOX_2D_BD	The two-dimensional bounding box. Corresponds to component data structure XrSpatialEntityComponentDataBoundingBox2DBD.
XR_SPATIAL_ENTITY_COMPONENT_TYPE_POLYGON_BD	The two-dimensional polygon. Corresponds to component data structure XrSpatialEntityComponentDataPolygonBD.
XR_SPATIAL_ENTITY_COMPONENT_TYPE_BOUNDING_BOX_3D_BD	The three-dimensional bounding box. Corresponds to component data structure XrSpatialEntityComponentDataBoundingBox3DBD.
XR_SPATIAL_ENTITY_COMPONENT_TYPE_TRIANGLE_MESH_BD	The triangle mesh. Corresponds to component data structure XrSpatialEntityComponentDataTriangleMeshBD.
XR_SPATIAL_ENTITY_COMPONENT_TYPE_SPHERE_BD	The Sphere. Corresponds to component data structure XrSpatialEntityComponentDataSphereBD.

Enum	Description
XR_SPATIAL_ENTITY_COMPONENT_TYPE_PLANE_ORIENTATION_BD	The plane orientation. Corresponds to component data structure XrSpatialEntityComponentDataPlaneOrientationBD. (Added by the XR_BD_spatial_plane extension)
XR_SPATIAL_ENTITY_COMPONENT_TYPE_DYNAMIC_OBJECT_BD	The dynamic object component. Corresponds to component data structure XrSpatialEntityComponentDataDynamicObjectBD. (Added by the XR_BD_dynamic_object_tracking extension)

12.92.11. Get Spatial Entity Component Data

Applications **can** get the spatial entity component data of a given spatial entity.

The xrGetSpatialEntityComponentDataBD function is defined as:

```
// Provided by XR_BD_spatial_sensing
XrResult xrGetSpatialEntityComponentDataBD(
    XrSenseDataSnapshotBD                       snapshot,
    const XrSpatialEntityComponentGetInfoBD*    getInfo,
    XrSpatialEntityComponentDataBaseHeaderBD*   componentData);
```

Parameter Descriptions

- snapshot is the XrSenseDataSnapshotBD handle.
- getInfo is a pointer to the XrSpatialEntityComponentGetInfoBD identifying the component data to retrieve.
- componentData is a pointer to a valid structure based on XrSpatialEntityComponentDataBaseHeaderBD.

The application **can** use xrGetSpatialEntityComponentDataBD to get the component data of a spatial entity.

The runtime **must** return XR_ERROR_VALIDATION_FAILURE if the component type is not present on a given entity.

The XrSpatialEntityComponentGetInfoBD::componentType **must** match the type of the output XrSpatialEntityComponentData* structure passed as componentData when querying certain component data with xrGetSpatialEntityComponentDataBD. Otherwise, the runtime **must** return XR_ERROR_VALIDATION_FAILURE from xrGetSpatialEntityComponentDataBD.

If a given component type requires passing additional input through the XrSpatialEntityComponentGetInfoBD::`next` chain, but that input is not present, the runtime **must** return `XR_ERROR_VALIDATION_FAILURE`.

Some component data types (types passed to `componentData`) use the structure form of the two call idiom to populate buffers of variable sizes.

The runtime **must** not change the component data in the snapshot until the snapshot is destroyed.

Valid Usage (Implicit)

- The `XR_BD_spatial_sensing` extension **must** be enabled prior to calling xrGetSpatialEntityComponentDataBD
- `snapshot` **must** be a valid XrSenseDataSnapshotBD handle
- `getInfo` **must** be a pointer to a valid XrSpatialEntityComponentGetInfoBD structure
- `componentData` **must** be a pointer to an XrSpatialEntityComponentDataBaseHeaderBD-based structure

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_FEATURE_UNSUPPORTED`
- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_SIZE_INSUFFICIENT`
- `XR_ERROR_SPATIAL_ENTITY_ID_INVALID_BD`
- `XR_ERROR_SPATIAL_SENSING_SERVICE_UNAVAILABLE_BD`
- `XR_ERROR_VALIDATION_FAILURE`

The XrSpatialEntityComponentGetInfoBD structure is defined as:

```
// Provided by XR_BD_spatial_sensing
typedef struct XrSpatialEntityComponentGetInfoBD {
    XrStructureType                   type;
    const void*                       next;
    XrSpatialEntityIdBD               entityId;
    XrSpatialEntityComponentTypeBD    componentType;
} XrSpatialEntityComponentGetInfoBD;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `entityId` is the `XrSpatialEntityIdBD` to get the component info.
- `componentType` is the XrSpatialEntityComponentTypeBD to get.

The XrSpatialEntityComponentGetInfoBD is a common structure that is used to get component data for a given component type. For some component types, applications **may** need to chain additional `getInfo` structures on the XrSpatialEntityComponentGetInfoBD::`next` chain, when passing it to the xrGetSpatialEntityComponentDataBD function.

Valid Usage (Implicit)

- The `XR_BD_spatial_sensing` extension **must** be enabled prior to using XrSpatialEntityComponentGetInfoBD
- `type` **must** be `XR_TYPE_SPATIAL_ENTITY_COMPONENT_GET_INFO_BD`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain. See also: XrSpatialEntityLocationGetInfoBD
- `componentType` **must** be a valid XrSpatialEntityComponentTypeBD value

The XrSpatialEntityComponentDataBaseHeaderBD structure is defined as:

```
// Provided by XR_BD_spatial_sensing
typedef struct XrSpatialEntityComponentDataBaseHeaderBD {
    XrStructureType    type;
    void*              next;
} XrSpatialEntityComponentDataBaseHeaderBD;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.

The XrSpatialEntityComponentDataBaseHeaderBD is a base structure that is not intended to be directly used, but forms a basis for specific component info types. All component info structures begin with the elements described in the XrSpatialEntityComponentDataBaseHeaderBD, and a component info pointer **must** be cast to a pointer to XrSpatialEntityComponentDataBaseHeaderBD when passing it to the xrGetSpatialEntityComponentDataBD function.

Valid Usage (Implicit)

- The `XR_BD_spatial_sensing` extension **must** be enabled prior to using XrSpatialEntityComponentDataBaseHeaderBD
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain. See also: XrSpatialEntityComponentDataBoundingBox2DBD, XrSpatialEntityComponentDataBoundingBox3DBD, XrSpatialEntityComponentDataDynamicObjectBD, XrSpatialEntityComponentDataLocationBD, XrSpatialEntityComponentDataPlaneOrientationBD, XrSpatialEntityComponentDataPolygonBD, XrSpatialEntityComponentDataSemanticBD, XrSpatialEntityComponentDataSphereBD, XrSpatialEntityComponentDataTriangleMeshBD

There are some commonly used component data structures defined in this extension which provide access to different spatial sensing features.

The XrSpatialEntityLocationGetInfoBD structure is defined as:

```
// Provided by XR_BD_spatial_sensing
typedef struct XrSpatialEntityLocationGetInfoBD {
    XrStructureType    type;
    const void*        next;
    XrSpace            baseSpace;
} XrSpatialEntityLocationGetInfoBD;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `baseSpace` is an XrSpace within which the spatial entity will be located.

Chaining this struct to XrSpatialEntityComponentGetInfoBD::`next` is required when querying certain component data with xrGetSpatialEntityComponentDataBD. Otherwise, the runtime **must** return `XR_ERROR_VALIDATION_FAILURE` from xrGetSpatialEntityComponentDataBD. See the `XrSpatialEntityComponentData*` struct details for more information which components require this struct to be chained.

Valid Usage (Implicit)

- The `XR_BD_spatial_sensing` extension **must** be enabled prior to using XrSpatialEntityLocationGetInfoBD
- `type` **must** be `XR_TYPE_SPATIAL_ENTITY_LOCATION_GET_INFO_BD`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `baseSpace` **must** be a valid XrSpace handle

The XrSpatialEntityComponentDataLocationBD structure is defined as:

```
// Provided by XR_BD_spatial_sensing
typedef struct XrSpatialEntityComponentDataLocationBD {
    XrStructureType    type;
    void*              next;
    XrSpaceLocation    location;
} XrSpatialEntityComponentDataLocationBD;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `location` is the XrSpaceLocation structure.

The XrSpatialEntityComponentDataLocationBD is an output struct for getting the location component

data from the snapshot. This corresponds to an XrSpatialEntityComponentTypeBD value of XR_SPATIAL_ENTITY_COMPONENT_TYPE_LOCATION_BD. The application **must** chain XrSpatialEntityLocationGetInfoBD to XrSpatialEntityComponentGetInfoBD::next when querying XrSpatialEntityComponentDataLocationBD. Otherwise, the runtime **must** return XR_ERROR_VALIDATION_FAILURE from xrGetSpatialEntityComponentDataBD.

Valid Usage (Implicit)

- The XR_BD_spatial_sensing extension **must** be enabled prior to using XrSpatialEntityComponentDataLocationBD
- type **must** be XR_TYPE_SPATIAL_ENTITY_COMPONENT_DATA_LOCATION_BD
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- location **must** be a valid XrSpaceLocation structure

The XrSpatialEntityComponentDataSemanticBD structure is defined as:

```
// Provided by XR_BD_spatial_sensing
typedef struct XrSpatialEntityComponentDataSemanticBD {
    XrStructureType       type;
    void*                 next;
    uint32_t              labelCapacityInput;
    uint32_t              labelCountOutput;
    XrSemanticLabelBD*    labels;
} XrSpatialEntityComponentDataSemanticBD;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- labelCapacityInput is the capacity of the labels array, or 0 to indicate a request to retrieve the required capacity.
- labelCountOutput is the number of semantic labels, or the required capacity in the case that labelCapacityInput is insufficient.
- labels is an array of XrSemanticLabelBD. It **can** be NULL if labelCapacityInput is 0.
- See Buffer Size Parameters chapter for a detailed description of retrieving the required labels size.

The XrSpatialEntityComponentDataSemanticBD is an output struct for getting the semantic labels

component data from an entity in the snapshot. This corresponds to an XrSpatialEntityComponentTypeBD value of XR_SPATIAL_ENTITY_COMPONENT_TYPE_SEMANTIC_BD.

Valid Usage (Implicit)

- The XR_BD_spatial_sensing extension **must** be enabled prior to using XrSpatialEntityComponentDataSemanticBD
- type **must** be XR_TYPE_SPATIAL_ENTITY_COMPONENT_DATA_SEMANTIC_BD
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- If labelCapacityInput is not 0, labels **must** be a pointer to an array of labelCapacityInput XrSemanticLabelBD values

Specify the semantic labels of spatial entity.

```
// Provided by XR_BD_spatial_sensing
typedef enum XrSemanticLabelBD {
    XR_SEMANTIC_LABEL_UNKNOWN_BD = 0,
    XR_SEMANTIC_LABEL_FLOOR_BD = 1,
    XR_SEMANTIC_LABEL_CEILING_BD = 2,
    XR_SEMANTIC_LABEL_WALL_BD = 3,
    XR_SEMANTIC_LABEL_DOOR_BD = 4,
    XR_SEMANTIC_LABEL_WINDOW_BD = 5,
    XR_SEMANTIC_LABEL_OPENING_BD = 6,
    XR_SEMANTIC_LABEL_TABLE_BD = 7,
    XR_SEMANTIC_LABEL_SOFA_BD = 8,
    XR_SEMANTIC_LABEL_CHAIR_BD = 9,
    XR_SEMANTIC_LABEL_HUMAN_BD = 10,
    XR_SEMANTIC_LABEL_BEAM_BD = 11,
    XR_SEMANTIC_LABEL_COLUMN_BD = 12,
    XR_SEMANTIC_LABEL_CURTAIN_BD = 13,
    XR_SEMANTIC_LABEL_CABINET_BD = 14,
    XR_SEMANTIC_LABEL_BED_BD = 15,
    XR_SEMANTIC_LABEL_PLANT_BD = 16,
    XR_SEMANTIC_LABEL_SCREEN_BD = 17,
    XR_SEMANTIC_LABEL_VIRTUAL_WALL_BD = 18,
    XR_SEMANTIC_LABEL_REFRIGERATOR_BD = 19,
    XR_SEMANTIC_LABEL_WASHING_MACHINE_BD = 20,
    XR_SEMANTIC_LABEL_AIR_CONDITIONER_BD = 21,
    XR_SEMANTIC_LABEL_LAMP_BD = 22,
    XR_SEMANTIC_LABEL_WALL_ART_BD = 23,
    XR_SEMANTIC_LABEL_STAIRWAY_BD = 24,
    XR_SEMANTIC_LABEL_MAX_ENUM_BD = 0x7FFFFFFF
} XrSemanticLabelBD;
```

Enum	Description
XR_SEMANTIC_LABEL_UNKNOWN_BD	Semantic label that the runtime does not know.
XR_SEMANTIC_LABEL_FLOOR_BD	Semantic label of floor.
XR_SEMANTIC_LABEL_CEILING_BD	Semantic label of ceiling.
XR_SEMANTIC_LABEL_WALL_BD	Semantic label of wall.
XR_SEMANTIC_LABEL_DOOR_BD	Semantic label of door.
XR_SEMANTIC_LABEL_WINDOW_BD	Semantic label of window.
XR_SEMANTIC_LABEL_OPENING_BD	Semantic label of opening, usually refers to a space that something or someone can pass through.
XR_SEMANTIC_LABEL_TABLE_BD	Semantic label of table.
XR_SEMANTIC_LABEL_SOFA_BD	Semantic label of sofa, usually refers to a seat that multiple people can sit on.
XR_SEMANTIC_LABEL_CHAIR_BD	Semantic label of chair, usually refers to a seat that for one person.
XR_SEMANTIC_LABEL_HUMAN_BD	Semantic label of human.
XR_SEMANTIC_LABEL_BEAM_BD	Semantic label of beam, which usually supports weight in a building or other structure.
XR_SEMANTIC_LABEL_COLUMN_BD	Semantic label of column, which is vertical and used as a support for the roof of a building.
XR_SEMANTIC_LABEL_CURTAIN_BD	Semantic label of curtain.
XR_SEMANTIC_LABEL_CABINET_BD	Semantic label of cabinet.
XR_SEMANTIC_LABEL_BED_BD	Semantic label of bed.
XR_SEMANTIC_LABEL_PLANT_BD	Semantic label of plant.
XR_SEMANTIC_LABEL_SCREEN_BD	Semantic label of screen.
XR_SEMANTIC_LABEL_VIRTUAL_WALL_BD	Semantic label of virtual wall, which is generated by the system scene capture app in order to create a closed space for users.
XR_SEMANTIC_LABEL_REFRIGERATOR_BD	Semantic label of refrigerator.
XR_SEMANTIC_LABEL_WASHING_MACHINE_BD	Semantic label of washing machine.
XR_SEMANTIC_LABEL_AIR_CONDITIONER_BD	Semantic label of air conditioner.
XR_SEMANTIC_LABEL_LAMP_BD	Semantic label of lamp.
XR_SEMANTIC_LABEL_WALL_ART_BD	Semantic label of wall art, like a painting or a photo frame.

Enum	Description
XR_SEMANTIC_LABEL_STAIRWAY_BD	Semantic label of stairway.

The XrSpatialEntityComponentDataBoundingBox2DBD structure is defined as:

```
// Provided by XR_BD_spatial_sensing
typedef struct XrSpatialEntityComponentDataBoundingBox2DBD {
    XrStructureType    type;
    void*              next;
    XrRect2Df          boundingBox2D;
} XrSpatialEntityComponentDataBoundingBox2DBD;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- boundingBox2D is an XrRect2Df defining the offset and extent of the bounding box along the x-axis (width) and y-axis (height).

The XrSpatialEntityComponentDataBoundingBox2DBD is an output struct for getting the bounding box 2D component data from the snapshot. This corresponds to an XrSpatialEntityComponentTypeBD value of XR_SPATIAL_ENTITY_COMPONENT_TYPE_BOUNDING_BOX_2D_BD.

The x-axis and y-axis that describe boundingBox2D are defined by the spatial entity's coordinate space.

Valid Usage (Implicit)

- The XR_BD_spatial_sensing extension **must** be enabled prior to using XrSpatialEntityComponentDataBoundingBox2DBD
- type **must** be XR_TYPE_SPATIAL_ENTITY_COMPONENT_DATA_BOUNDING_BOX_2D_BD
- next **must** be NULL or a valid pointer to the next structure in a structure chain

The XrSpatialEntityComponentDataPolygonBD structure is defined as:

```
// Provided by XR_BD_spatial_sensing
typedef struct XrSpatialEntityComponentDataPolygonBD {
    XrStructureType    type;
    void*              next;
    uint32_t           vertexCapacityInput;
    uint32_t           vertexCountOutput;
    XrVector2f*        vertices;
} XrSpatialEntityComponentDataPolygonBD;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `vertexCapacityInput` is the capacity of the `vertices` array, or 0 to indicate a request to retrieve the required capacity.
- `vertexCountOutput` is the number of elements populated in the `vertices` array, or the required capacity in the case that `vertexCapacityInput` is insufficient.
- `vertices` is an array of XrVector2f. It **can** be `NULL` if `vertexCapacityInput` is 0.
- See Buffer Size Parameters chapter for a detailed description of retrieving the required `vertices` size.

The XrSpatialEntityComponentDataPolygonBD is an output struct for getting the polygon component data from an entity in the snapshot. This corresponds to an XrSpatialEntityComponentTypeBD value of `XR_SPATIAL_ENTITY_COMPONENT_TYPE_POLYGON_BD`.

The `vertices` is a list of vertices described on the XY plane, which is defined by the spatial entity's coordinate space.

Valid Usage (Implicit)

- The `XR_BD_spatial_sensing` extension **must** be enabled prior to using XrSpatialEntityComponentDataPolygonBD
- `type` **must** be `XR_TYPE_SPATIAL_ENTITY_COMPONENT_DATA_POLYGON_BD`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- If `vertexCapacityInput` is not `0`, `vertices` **must** be a pointer to an array of `vertexCapacityInput` XrVector2f structures

The XrSpatialEntityComponentDataBoundingBox3DBD structure is defined as:

```
// Provided by XR_BD_spatial_sensing
typedef struct XrSpatialEntityComponentDataBoundingBox3DBD {
    XrStructureType    type;
    void*              next;
    XrBoxf             boundingBox3D;
} XrSpatialEntityComponentDataBoundingBox3DBD;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `boundingBox3D` is an XrBoxf defining the center position and orientation and the extent of a bounding box 3D relative to its spatial entity's location.

The XrSpatialEntityComponentDataBoundingBox3DBD is an output struct for getting the bounding box 3D component data from an entity in the snapshot. This corresponds to an XrSpatialEntityComponentTypeBD value of `XR_SPATIAL_ENTITY_COMPONENT_TYPE_BOUNDING_BOX_3D_BD`.

The x-axis, y-axis, and z-axis that describe `boundingBox3D` are defined by the spatial entity's coordinate space.

Valid Usage (Implicit)

- The `XR_BD_spatial_sensing` extension **must** be enabled prior to using XrSpatialEntityComponentDataBoundingBox3DBD
- `type` **must** be `XR_TYPE_SPATIAL_ENTITY_COMPONENT_DATA_BOUNDING_BOX_3D_BD`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

The XrSpatialEntityComponentDataSphereBD structure is defined as:

```
// Provided by XR_BD_spatial_sensing
typedef struct XrSpatialEntityComponentDataSphereBD {
    XrStructureType    type;
    void*              next;
    XrSpheref          sphere;
} XrSpatialEntityComponentDataSphereBD;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `sphere` is an XrSpheref defining the center position and orientation and the radius of a sphere bounds relative to the spatial entity.

The XrSpatialEntityComponentDataSphereBD is an output struct for getting the sphere component data from an entity in the snapshot. This corresponds to an XrSpatialEntityComponentTypeBD value of `XR_SPATIAL_ENTITY_COMPONENT_TYPE_SPHERE_BD`.

Valid Usage (Implicit)

- The `XR_BD_spatial_sensing` extension **must** be enabled prior to using XrSpatialEntityComponentDataSphereBD
- `type` **must** be `XR_TYPE_SPATIAL_ENTITY_COMPONENT_DATA_SPHERE_BD`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

The XrSpatialEntityComponentDataTriangleMeshBD structure is defined as:

```
// Provided by XR_BD_spatial_sensing
typedef struct XrSpatialEntityComponentDataTriangleMeshBD {
    XrStructureType    type;
    void*              next;
    uint32_t           vertexCapacityInput;
    uint32_t           vertexCountOutput;
    XrVector3f*        vertices;
    uint32_t           indexCapacityInput;
    uint32_t           indexCountOutput;
    uint16_t*          indices;
} XrSpatialEntityComponentDataTriangleMeshBD;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `vertexCapacityInput` is the capacity of the `vertices` array, or 0 to indicate a request to retrieve the required capacity.
- `vertexCountOutput` is the number of elements populated in the `vertices` array, or the required capacity in the case that `vertexCapacityInput` is insufficient.
- `vertices` is an array of XrVector3f. It **can** be `NULL` if `vertexCapacityInput` is 0.
- `indexCapacityInput` is the capacity of the `indices` array, or 0 to indicate a request to retrieve the required capacity.
- `indexCountOutput` is the number of elements in the `indices` array, or the required capacity in the case that `indexCapacityInput` is insufficient.
- `indices` is an array of indices. It **can** be `NULL` if `indexCapacityInput` is 0.
- See Buffer Size Parameters chapter for a detailed description of retrieving the required `vertices` and `indices` size.

The XrSpatialEntityComponentDataTriangleMeshBD is an output struct for getting the triangle mesh component data from the snapshot. This corresponds to an XrSpatialEntityComponentTypeBD value of `XR_SPATIAL_ENTITY_COMPONENT_TYPE_TRIANGLE_MESH_BD`.

The `vertices` array is a list of vertices described by the spatial entity's coordinate space. The triangle vertices are in counter-clockwise order as viewed from the user perspective.

The `indices` array defines the topology of the triangle mesh. Each triplet of consecutive elements points to three vertices in the `vertices` array and thus form a triangle.

Valid Usage (Implicit)

- The `XR_BD_spatial_sensing` extension **must** be enabled prior to using XrSpatialEntityComponentDataTriangleMeshBD
- `type` **must** be `XR_TYPE_SPATIAL_ENTITY_COMPONENT_DATA_TRIANGLE_MESH_BD`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- If `vertexCapacityInput` is not `0`, `vertices` **must** be a pointer to an array of `vertexCapacityInput` XrVector3f structures
- If `indexCapacityInput` is not `0`, `indices` **must** be a pointer to an array of `indexCapacityInput` `uint16_t` values

12.92.12. Get Sense Data Update Event

The XrEventDataSenseDataUpdatedBD structure is defined as:

```
// Provided by XR_BD_spatial_sensing
typedef struct XrEventDataSenseDataUpdatedBD {
    XrStructureType          type;
    const void*              next;
    XrSenseDataProviderBD    provider;
} XrEventDataSenseDataUpdatedBD;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `provider` is the XrSenseDataProviderBD in which the sense data updated.

When the application receives this event, it means one of the sense data providers has updated sense data, and the application **can** initiate a new query request to this sense data provider to get the latest sense data, as a new XrSenseDataSnapshotBD. This is the recommended way to get updated data. The application **may** initiate a new query request even without receiving this event, but the sense data provider **may** not have acquired new data and therefore the contents of the snapshot **may** be functionally identical.

Valid Usage (Implicit)

- The `XR_BD_spatial_sensing` extension **must** be enabled prior to using XrEventDataSenseDataUpdatedBD
- `type` **must** be `XR_TYPE_EVENT_DATA_SENSE_DATA_UPDATED_BD`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

12.92.13. Stop Sense Data Provider

The xrStopSenseDataProviderBD function is defined as:

```
// Provided by XR_BD_spatial_sensing
XrResult xrStopSenseDataProviderBD(
    XrSenseDataProviderBD                       provider);
```

Parameter Descriptions

- `provider` is the XrSenseDataProviderBD to stop.

The application **can** stop a sense data provider using xrStopSenseDataProviderBD if it no longer needs to obtain new sense data from the sense data provider. Stopping the provider **must** not affect the usage of existing valid XrSenseDataSnapshotBD handles and their corresponding entity IDs.

Valid Usage (Implicit)

- The `XR_BD_spatial_sensing` extension **must** be enabled prior to calling xrStopSenseDataProviderBD
- `provider` **must** be a valid XrSenseDataProviderBD handle

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_FEATURE_UNSUPPORTED`
- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_SPATIAL_SENSING_SERVICE_UNAVAILABLE_BD`
- `XR_ERROR_VALIDATION_FAILURE`

12.92.14. Destroy Sense Data Provider

The xrDestroySenseDataProviderBD function is defined as:

```
// Provided by XR_BD_spatial_sensing
XrResult xrDestroySenseDataProviderBD(
    XrSenseDataProviderBD                       provider);
```

Parameter Descriptions

- `provider` is the XrSenseDataProviderBD to destroy.

The application **should** destroy a sense data provider using xrDestroySenseDataProviderBD to release the resources related to it.

Valid Usage (Implicit)

- The `XR_BD_spatial_sensing` extension **must** be enabled prior to calling xrDestroySenseDataProviderBD
- `provider` **must** be a valid XrSenseDataProviderBD handle

Thread Safety

- Access to `provider`, and any child handles, **must** be externally synchronized

Return Codes

Success

- `XR_SUCCESS`

Failure

- `XR_ERROR_FEATURE_UNSUPPORTED`
- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SPATIAL_SENSING_SERVICE_UNAVAILABLE_BD`

12.92.15. Anchor

The XrAnchorBD handle is defined as:

```
// Provided by XR_BD_spatial_sensing
XR_DEFINE_HANDLE(XrAnchorBD)
```

An anchor is bound to a point in the real physical world, or to a spatial entity.

The XrAnchorBD handle is alive until it is explicitly destroyed or the XrSenseDataProviderBD is destroyed.

Create an Anchor from a Spatial Entity

The xrCreateSpatialEntityAnchorBD function is defined as:

```
// Provided by XR_BD_spatial_sensing
XrResult xrCreateSpatialEntityAnchorBD(
    XrSenseDataProviderBD                          provider,
    const XrSpatialEntityAnchorCreateInfoBD*       createInfo,
    XrAnchorBD*                                    anchor);
```

Parameter Descriptions

- `provider` is the XrSenseDataProviderBD.
- `createInfo` is a pointer to the XrSpatialEntityAnchorCreateInfoBD.
- `anchor` is a pointer to the created XrAnchorBD handle.

Some spatial entities **may** have frequently changing poses, requiring real-time acquisition of their poses. The application **can** create the XrAnchorBD handle bound to the spatial entity through xrCreateSpatialEntityAnchorBD, and then create the XrSpace from it using xrCreateAnchorSpaceBD, so that the application **can** obtain its latest pose in real time through xrLocateSpace or xrLocateSpaces.

This function is not supported for all types of sense data providers. Whether this is supported in a type of sense data provider will be explained in the extension that defines the specific type of sense data provider. Otherwise, the runtime **must** return `XR_ERROR_ANCHOR_NOT_SUPPORTED_FOR_ENTITY_BD`.

This handle and associated handles continue to be valid and usable even if the XrSenseDataSnapshotBD used at its creation has since been destroyed.

Valid Usage (Implicit)

- The `XR_BD_spatial_sensing` extension **must** be enabled prior to calling xrCreateSpatialEntityAnchorBD
- `provider` **must** be a valid XrSenseDataProviderBD handle
- `createInfo` **must** be a pointer to a valid XrSpatialEntityAnchorCreateInfoBD structure
- `anchor` **must** be a pointer to an XrAnchorBD handle

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FEATURE_UNSUPPORTED
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_SPATIAL_SENSING_SERVICE_UNAVAILABLE_BD
- XR_ERROR_VALIDATION_FAILURE

The XrSpatialEntityAnchorCreateInfoBD structure is defined as:

```
// Provided by XR_BD_spatial_sensing
typedef struct XrSpatialEntityAnchorCreateInfoBD {
    XrStructureType          type;
    const void*              next;
    XrSenseDataSnapshotBD    snapshot;
    XrSpatialEntityIdBD      entityId;
} XrSpatialEntityAnchorCreateInfoBD;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- snapshot is the XrSenseDataSnapshotBD handle.
- entityId is the XrSpatialEntityIdBD to create from.

Valid Usage (Implicit)

- The XR_BD_spatial_sensing extension **must** be enabled prior to using XrSpatialEntityAnchorCreateInfoBD
- type **must** be XR_TYPE_SPATIAL_ENTITY_ANCHOR_CREATE_INFO_BD
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- snapshot **must** be a valid XrSenseDataSnapshotBD handle

12.92.16. Get Anchor UUID

The xrGetAnchorUuidBD function is defined as:

```
// Provided by XR_BD_spatial_sensing
XrResult xrGetAnchorUuidBD(
    XrAnchorBD                                  anchor,
    XrUuidEXT*                                  uuid);
```

Parameter Descriptions

- anchor is the XrAnchorBD handle.
- uuid is a pointer to the XrUuid.

The application **can** get the anchor UUID using xrGetAnchorUuidBD. If the XrAnchorBD is created from a spatial entity, the anchor's UUID is the same as the spatial entity's UUID. .Valid Usage (Implicit)

- The XR_BD_spatial_sensing extension **must** be enabled prior to calling xrGetAnchorUuidBD
- anchor **must** be a valid XrAnchorBD handle
- uuid **must** be a pointer to an XrUuidEXT structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FEATURE_UNSUPPORTED
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_SPATIAL_SENSING_SERVICE_UNAVAILABLE_BD
- XR_ERROR_VALIDATION_FAILURE

12.92.17. Locate an Anchor with Anchor Spaces

Locating an anchor relative to a base space is performed similarly to locating other spatial objects: through use of an XrSpace handle and functions like xrLocateSpace and xrLocateSpaces. To locate an anchor in a base space, first create an XrSpace handle for that anchor using xrCreateAnchorSpaceBD.

The xrCreateAnchorSpaceBD function is defined as:

```
// Provided by XR_BD_spatial_sensing
XrResult xrCreateAnchorSpaceBD(
    XrSession                                   session,
    const XrAnchorSpaceCreateInfoBD*            createInfo,
    XrSpace*                                    space);
```

Parameter Descriptions

- session is a handle to the parent XrSession.
- createInfo is a pointer to an XrAnchorSpaceCreateInfoBD structure containing information about how to create the anchor.
- space is a pointer to a handle in which the created XrSpace is returned.

The application **can** create an XrSpace for an anchor using xrCreateAnchorSpaceBD. Using this handle,

the application **can** use calls like xrLocateSpace and xrLocateSpaces to locate the anchor in a given base space at a given time. The `createInfo` parameter contains the XrAnchorBD as well as a pose offset to apply.

Multiple XrSpace handles for a given XrAnchorBD **may** exist simultaneously, up to some limit imposed by the runtime. The XrSpace handle **must** be eventually freed via the xrDestroySpace function or by destroying the parent XrSession handle.

As the parent of all XrSpace handles, including those created with this function, is an XrSession handle, an anchor space **may** outlive the XrAnchorBD handle used to create it. Additionally, the ability to locate anchor spaces depends on spatial sensing being active. A valid XrSpace handle created for an XrAnchorBD **must** be unlocatable unless the associated XrSenseDataProviderBD has been started without since being stopped.

Such an unlocatable anchor space behaves the same as an unlocatable action space as discussed in Action Spaces Lifetime.

Note that destroying the XrAnchorBD used in creating an anchor space does not itself make the anchor space unlocatable; it only prevents creation of additional anchor spaces from that anchor. This mirrors the behavior of action spaces and destruction of their corresponding pose actions.

Valid Usage (Implicit)

- The `XR_BD_spatial_sensing` extension **must** be enabled prior to calling xrCreateAnchorSpaceBD
- `session` **must** be a valid XrSession handle
- `createInfo` **must** be a pointer to a valid XrAnchorSpaceCreateInfoBD structure
- `space` **must** be a pointer to an XrSpace handle

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FEATURE_UNSUPPORTED
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_POSE_INVALID
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_SPATIAL_SENSING_SERVICE_UNAVAILABLE_BD
- XR_ERROR_VALIDATION_FAILURE

The XrAnchorSpaceCreateInfoBD structure is defined as:

```
// Provided by XR_BD_spatial_sensing
typedef struct XrAnchorSpaceCreateInfoBD {
    XrStructureType    type;
    const void*        next;
    XrAnchorBD         anchor;
    XrPosef            poseInAnchorSpace;
} XrAnchorSpaceCreateInfoBD;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `anchor` is a handle to an XrAnchorBD.
- `poseInAnchorSpace` is an XrPosef describing the pose of the new space's origin relative to the pose of the anchor.

Valid Usage (Implicit)

- The `XR_BD_spatial_sensing` extension **must** be enabled prior to using XrAnchorSpaceCreateInfoBD
- `type` **must** be `XR_TYPE_ANCHOR_SPACE_CREATE_INFO_BD`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `anchor` **must** be a valid XrAnchorBD handle

12.92.18. Destroy an Anchor

The xrDestroyAnchorBD function is defined as:

```
// Provided by XR_BD_spatial_sensing
XrResult xrDestroyAnchorBD(
    XrAnchorBD                                  anchor);
```

Parameter Descriptions

- `anchor` is the XrAnchorBD handle.

The application **can** destroy an anchor using xrDestroyAnchorBD.

Valid Usage (Implicit)

- The `XR_BD_spatial_sensing` extension **must** be enabled prior to calling xrDestroyAnchorBD
- `anchor` **must** be a valid XrAnchorBD handle

Thread Safety

- Access to `anchor`, and any child handles, **must** be externally synchronized

Return Codes

Success

- `XR_SUCCESS`

Failure

- `XR_ERROR_FEATURE_UNSUPPORTED`
- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SPATIAL_SENSING_SERVICE_UNAVAILABLE_BD`

12.92.19. New Base Types

- `XrSpatialEntityIdBD`

12.92.20. New Object Types

- XrAnchorBD
- XrSenseDataProviderBD
- XrSenseDataSnapshotBD

12.92.21. New Commands

- xrCreateAnchorSpaceBD
- xrCreateSenseDataProviderBD
- xrCreateSpatialEntityAnchorBD
- xrDestroyAnchorBD
- xrDestroySenseDataProviderBD
- xrDestroySenseDataSnapshotBD
- xrEnumerateSpatialEntityComponentTypesBD
- xrGetAnchorUuidBD
- xrGetQueriedSenseDataBD

- xrGetSenseDataProviderStateBD
- xrGetSpatialEntityComponentDataBD
- xrGetSpatialEntityUuidBD
- xrQuerySenseDataAsyncBD
- xrQuerySenseDataCompleteBD
- xrStartSenseDataProviderAsyncBD
- xrStartSenseDataProviderCompleteBD
- xrStopSenseDataProviderBD

12.92.22. New Structures

- XrAnchorSpaceCreateInfoBD
- XrEventDataSenseDataProviderStateChangedBD
- XrEventDataSenseDataUpdatedBD
- XrQueriedSenseDataBD
- XrQueriedSenseDataGetInfoBD
- XrSenseDataProviderCreateInfoBD
- XrSenseDataProviderStartInfoBD
- XrSenseDataQueryCompletionBD
- XrSenseDataQueryInfoBD
- XrSpatialEntityAnchorCreateInfoBD
- XrSpatialEntityComponentDataBaseHeaderBD
- XrSpatialEntityComponentGetInfoBD
- XrSpatialEntityStateBD
- Extending XrSenseDataQueryInfoBD:
  - XrSenseDataFilterSemanticBD
  - XrSenseDataFilterUuidBD
- Extending XrSpatialEntityComponentDataBaseHeaderBD:
  - XrSpatialEntityComponentDataBoundingBox2DBD
  - XrSpatialEntityComponentDataBoundingBox3DBD
  - XrSpatialEntityComponentDataLocationBD
  - XrSpatialEntityComponentDataPolygonBD
  - XrSpatialEntityComponentDataSemanticBD
  - XrSpatialEntityComponentDataSphereBD

    - XrSpatialEntityComponentDataTriangleMeshBD
- Extending XrSpatialEntityComponentGetInfoBD:
    - XrSpatialEntityLocationGetInfoBD
- Extending XrSystemProperties:
    - XrSystemSpatialSensingPropertiesBD

12.92.23. New Enums

- XrSemanticLabelBD
- XrSenseDataProviderStateBD
- XrSenseDataProviderTypeBD
- XrSpatialEntityComponentTypeBD

12.92.24. New Enum Constants

- XR_BD_SPATIAL_SENSING_EXTENSION_NAME
- XR_BD_spatial_sensing_SPEC_VERSION
- Extending XrObjectType:
    - XR_OBJECT_TYPE_ANCHOR_BD
    - XR_OBJECT_TYPE_SENSE_DATA_PROVIDER_BD
    - XR_OBJECT_TYPE_SENSE_DATA_SNAPSHOT_BD
- Extending XrResult:
    - XR_ERROR_ANCHOR_NOT_SUPPORTED_FOR_ENTITY_BD
    - XR_ERROR_SPATIAL_ENTITY_ID_INVALID_BD
    - XR_ERROR_SPATIAL_SENSING_SERVICE_UNAVAILABLE_BD
- Extending XrStructureType:
    - XR_TYPE_ANCHOR_SPACE_CREATE_INFO_BD
    - XR_TYPE_EVENT_DATA_SENSE_DATA_PROVIDER_STATE_CHANGED_BD
    - XR_TYPE_EVENT_DATA_SENSE_DATA_UPDATED_BD
    - XR_TYPE_QUERIED_SENSE_DATA_BD
    - XR_TYPE_QUERIED_SENSE_DATA_GET_INFO_BD
    - XR_TYPE_SENSE_DATA_FILTER_SEMANTIC_BD
    - XR_TYPE_SENSE_DATA_FILTER_UUID_BD
    - XR_TYPE_SENSE_DATA_PROVIDER_CREATE_INFO_BD
    - XR_TYPE_SENSE_DATA_PROVIDER_START_INFO_BD

- XR_TYPE_SENSE_DATA_QUERY_COMPLETION_BD
- XR_TYPE_SENSE_DATA_QUERY_INFO_BD
- XR_TYPE_SPATIAL_ENTITY_ANCHOR_CREATE_INFO_BD
- XR_TYPE_SPATIAL_ENTITY_COMPONENT_DATA_BOUNDING_BOX_2D_BD
- XR_TYPE_SPATIAL_ENTITY_COMPONENT_DATA_BOUNDING_BOX_3D_BD
- XR_TYPE_SPATIAL_ENTITY_COMPONENT_DATA_LOCATION_BD
- XR_TYPE_SPATIAL_ENTITY_COMPONENT_DATA_POLYGON_BD
- XR_TYPE_SPATIAL_ENTITY_COMPONENT_DATA_SEMANTIC_BD
- XR_TYPE_SPATIAL_ENTITY_COMPONENT_DATA_SPHERE_BD
- XR_TYPE_SPATIAL_ENTITY_COMPONENT_DATA_TRIANGLE_MESH_BD
- XR_TYPE_SPATIAL_ENTITY_COMPONENT_GET_INFO_BD
- XR_TYPE_SPATIAL_ENTITY_LOCATION_GET_INFO_BD
- XR_TYPE_SPATIAL_ENTITY_STATE_BD
- XR_TYPE_SYSTEM_SPATIAL_SENSING_PROPERTIES_BD

12.92.25. Sample Code

Create sense data provider & query component data

The following example code demonstrates how to create and start spatial sense data provider for capability "Foo", as well as how to query its component "Bar" data.

```
XrInstance instance; // previously initialized
XrSystemId systemId; // previously initialized
XrSession session;   // previously initialized

// The function pointers are previously initialized using xrGetInstanceProcAddr.
PFN_xrCreateSenseDataProviderBD xrCreateSenseDataProviderBD;                    //
previously initialized
PFN_xrStartSenseDataProviderAsyncBD xrStartSenseDataProviderAsyncBD;            //
previously initialized
PFN_xrStartSenseDataProviderCompleteBD xrStartSenseDataProviderCompleteBD;      //
previously initialized
PFN_xrStopSenseDataProviderBD xrStopSenseDataProviderBD;                        //
previously initialized
PFN_xrDestroySenseDataProviderBD xrDestroySenseDataProviderBD;                  //
previously initialized
PFN_xrQuerySenseDataAsyncBD xrQuerySenseDataAsyncBD;                            //
previously initialized
PFN_xrQuerySenseDataCompleteBD xrQuerySenseDataCompleteBD;                      //
previously initialized
```

```
PFN_xrGetSenseDataProviderStateBD xrGetSenseDataProviderStateBD;                    //
previously initialized
PFN_xrGetQueriedSenseDataBD xrGetQueriedSenseDataBD;                                //
previously initialized
PFN_xrEnumerateSpatialEntityComponentTypesBD xrEnumerateSpatialEntityComponentTypesBD; //
previously initialized
PFN_xrGetSpatialEntityComponentDataBD xrGetSpatialEntityComponentDataBD;            //
previously initialized
PFN_xrDestroySenseDataSnapshotBD xrDestroySenseDataSnapshotBD;                      //
previously initialized
PFN_xrPollFutureEXT xrPollFutureEXT;                                                //
previously initialized

// Define structure to create data provider for spatial sensing capability foo
#define XR_TYPE_SENSE_DATA_PROVIDER_CREATE_INFO_FOO_BD ((XrStructureType)1000389000U)

// Derives from XrSenseDataProviderCreateInfoBD
typedef struct XrSenseDataProviderCreateInfoFooBD
{
    XrStructureType type;
    const void *XR_MAY_ALIAS next;
} XrSenseDataProviderCreateInfoFooBD;

// Define type for component bar and data structure to get component bar data
#define XR_SPATIAL_ENTITY_COMPONENT_TYPE_BAR_BD
((XrSpatialEntityComponentTypeBD)1000389001U)
#define XR_TYPE_SPATIAL_ENTITY_COMPONENT_DATA_BAR_BD ((XrStructureType)1000389002U)

// Derives from XrSpatialEntityComponentDataBaseHeaderBD
typedef struct XrSpatialEntityComponentDataBarBD
{
    XrStructureType type;
    const void *XR_MAY_ALIAS next;
    uint32_t barData;
} XrSpatialEntityComponentDataBarBD;

// Create data provider for spatial sensing capability foo
XrSenseDataProviderBD fooProvider;
XrFutureEXT providerCreateFuture;

XrSenseDataProviderCreateInfoFooBD providerCreateInfoFoo {
    .type = XR_TYPE_SENSE_DATA_PROVIDER_CREATE_INFO_FOO_BD,
};
XrSenseDataProviderCreateInfoBD providerCreateInfo {
    .type = XR_TYPE_SENSE_DATA_PROVIDER_CREATE_INFO_BD,
    .next = &providerCreateInfoFoo,
};
```

```
CHK_XR(xrCreateSenseDataProviderBD(session, &providerCreateInfo, &fooProvider));

auto waitUntilFutureReady = [&](XrFutureEXT future)
{
    XrFuturePollInfoEXT pollInfo {
        .type = XR_TYPE_FUTURE_POLL_INFO_EXT,
        .future = future,
    };
    XrFuturePollResultEXT pollResult {
        .type = XR_TYPE_FUTURE_POLL_RESULT_EXT,
    };
    do
    {
        // sleep(1);
        CHK_XR(xrPollFutureEXT(instance, &pollInfo, &pollResult));
    } while (pollResult.state != XR_FUTURE_STATE_READY_EXT);
};

// Start foo provider
XrFutureEXT providerStartFuture {XR_NULL_FUTURE_EXT};
XrSenseDataProviderStartInfoBD startInfo {
    .type = XR_TYPE_SENSE_DATA_PROVIDER_START_INFO_BD,
};
CHK_XR(xrStartSenseDataProviderAsyncBD(fooProvider, &startInfo, &providerStartFuture));

waitUntilFutureReady(providerStartFuture);

XrFutureCompletionEXT completion {
    .type = XR_TYPE_FUTURE_COMPLETION_EXT,
};

XrResult result = xrStartSenseDataProviderCompleteBD(session, providerStartFuture,
&completion);
CHK_XR(result); // Result of the complete function
CHK_XR(completion.futureResult);

// Check the provider state.
XrSenseDataProviderStateBD providerState;
CHK_XR(xrGetSenseDataProviderStateBD(fooProvider, &providerState));
if (providerState != XR_SENSE_DATA_PROVIDER_STATE_RUNNING_BD)
{
    // Provider not running, handle the case: start the provider again, or quit
    // ...
    return;
}

// Query foo spatial entities
auto queryFooSpatialEntities = [&]()
```

```
{
    XrSenseDataSnapshotBD querySnapshotHandle{XR_NULL_HANDLE};

    XrFutureEXT queryFuture;
    XrSenseDataQueryInfoBD queryInfo {
        .type = XR_TYPE_SENSE_DATA_QUERY_INFO_BD,
    };

    CHK_XR(xrQuerySenseDataAsyncBD(fooProvider, &queryInfo, &queryFuture));

    waitUntilFutureReady(queryFuture);

    XrSenseDataQueryCompletionBD completion {
        .type = XR_TYPE_SENSE_DATA_QUERY_COMPLETION_BD,
    };

    result = xrQuerySenseDataCompleteBD(fooProvider, queryFuture, &completion);
    CHK_XR(result); // Result of the complete function
    CHK_XR(completion.futureResult);

    querySnapshotHandle = completion.snapshot;

    // Retrieve queried sense data
    XrQueriedSenseDataGetInfoBD queriedDataGetInfo {
        .type = XR_TYPE_QUERIED_SENSE_DATA_GET_INFO_BD,
    };

    XrQueriedSenseDataBD queriedSenseData {
        .type = XR_TYPE_QUERIED_SENSE_DATA_BD,
        .next = nullptr,
        .stateCapacityInput = 0,
        .stateCountOutput = 0,
        .states = nullptr,
    };

    // First call to get spatial entity count
    CHK_XR(xrGetQueriedSenseDataBD(querySnapshotHandle, &queriedDataGetInfo,
&queriedSenseData));

    if (queriedSenseData.stateCountOutput > 0)
    {
        std::vector<XrSpatialEntityStateBD> queriedFooStates(queriedSenseData
.stateCountOutput);
        queriedSenseData.stateCapacityInput = queriedSenseData.stateCountOutput;
        queriedSenseData.states = queriedFooStates.data();

        // Second call to get the spatial entities in the provider's query result
        CHK_XR(xrGetQueriedSenseDataBD(querySnapshotHandle, &queriedDataGetInfo,
```

```
&queriedSenseData));

        // Process the data
        for (int i = 0; i < queriedSenseData.stateCountOutput; ++i)
        {
            // Process the spatial entities, and get spatial entity component data
            XrSpatialEntityIdBD entityId = queriedFooStates[i].entityId;
            XrUuidEXT uuid = queriedFooStates[i].uuid;

            // Enumerate the component types of the entity
            uint32_t componentTypeCountOutput = 0;
            CHK_XR(xrEnumerateSpatialEntityComponentTypesBD(querySnapshotHandle,
entityId, 0, &componentTypeCountOutput, nullptr));

            if (componentTypeCountOutput > 0)
            {
                std::vector<XrSpatialEntityComponentTypeBD> entityComponentTypes
(componentTypeCountOutput);
                uint32_t componentTypeCapacityInput = componentTypeCountOutput;
                CHK_XR(xrEnumerateSpatialEntityComponentTypesBD(querySnapshotHandle,
entityId, componentTypeCapacityInput,
                                                                &
componentTypeCountOutput, entityComponentTypes.data()));

                // Check the entity component types and get the bar component data
                for (int j = 0; j < componentTypeCountOutput; ++j)
                {
                    if (entityComponentTypes[j] ==
XR_SPATIAL_ENTITY_COMPONENT_TYPE_BAR_BD)
                    {
                        XrSpatialEntityComponentGetInfoBD componentBarGetInfo {
                            .type = XR_TYPE_SPATIAL_ENTITY_COMPONENT_GET_INFO_BD,
                            .next = nullptr,
                            .entityId = entityId,
                            .componentType = XR_SPATIAL_ENTITY_COMPONENT_TYPE_BAR_BD,
                        };
                        XrSpatialEntityComponentDataBarBD componentBarData {
                            .type = XR_TYPE_SPATIAL_ENTITY_COMPONENT_DATA_BAR_BD,
                        };

                        CHK_XR(xrGetSpatialEntityComponentDataBD(querySnapshotHandle,
&componentBarGetInfo,
                                                                 reinterpret_cast
<XrSpatialEntityComponentDataBaseHeaderBD *>(&componentBarData)));
                        // Component bar data now in componentBarData
                        // ...
                    }
                }
```

```
            }
        }
    }
    // Destroy query result
    CHK_XR(xrDestroySenseDataSnapshotBD(querySnapshotHandle));
};

auto processSenseDataEvent = [&]()
{
    while (true)
    {
        XrEventDataBuffer event {
            .type = XR_TYPE_EVENT_DATA_BUFFER,
        };
        XrResult result = xrPollEvent(instance, &event);
        if (result == XR_EVENT_UNAVAILABLE)
        {
            // No event available, quit the loop.
            break;
        }

        if (result == XR_SUCCESS)
        {
            // Process the event
            switch (event.type)
            {
            case XR_TYPE_EVENT_DATA_SENSE_DATA_UPDATED_BD:
            {
                const XrEventDataSenseDataUpdatedBD &senseDataUpdatedEvent =
                    *reinterpret_cast<XrEventDataSenseDataUpdatedBD *>(&event);
                if (senseDataUpdatedEvent.provider == fooProvider)
                {
                    // Sense data update in foo provider
                    queryFooSpatialEntities();
                }
                break;
            }

            default:
                break;
            }
        }
    }
};

////////////////////////////////////
// Per-frame work                 //
////////////////////////////////////
```

```
while (1)
{
    //...
    processSenseDataEvent();
    //...

    // ...
    // Finish loop
    // ...
}

// Stop and destroy the provider
CHK_XR(xrStopSenseDataProviderBD(fooProvider));
CHK_XR(xrDestroySenseDataProviderBD(fooProvider));
```

Get location component data and other component data coordinate with location

The following example code demonstrates how to get location component data from a snapshot, and how to get the bounding box 2D component data as well as the triangle mesh component data that are defined in the spatial entity's location component data's coordinate system.

```
XrSenseDataProviderBD provider;             // previously returned from
xrCreateSenseDataProviderBD
XrSenseDataSnapshotBD querySnapshotHandle;  // previously returned from
xrQuerySenseDataCompleteBD
XrSpace localSpace;                         // previously initialized, e.g. from
XR_REFERENCE_SPACE_TYPE_LOCAL
XrFrameState frameState;                    // previously returned from xrWaitFrame

PFN_xrGetQueriedSenseDataBD xrGetQueriedSenseDataBD;                                //
previously initialized
PFN_xrEnumerateSpatialEntityComponentTypesBD xrEnumerateSpatialEntityComponentTypesBD; //
previously initialized
PFN_xrGetSpatialEntityComponentDataBD xrGetSpatialEntityComponentDataBD;            //
previously initialized

auto hasComponentType = [&](std::vector<XrSpatialEntityComponentTypeBD>& typeList,
XrSpatialEntityComponentTypeBD type)->bool
{
    if(typeList.empty())
    {
        return false;
    }

    for(int i = 0; i < typeList.size(); i++)
    {
```

```
        if(typeList[i] == type)
        {
            return true;
        }
    }

    return false;
};

XrQueriedSenseDataGetInfoBD queriedDataGetInfo {
    .type = XR_TYPE_QUERIED_SENSE_DATA_GET_INFO_BD,
};

XrQueriedSenseDataBD queriedSenseData {
    .type = XR_TYPE_QUERIED_SENSE_DATA_BD,
};

// First call to get spatial entity count
CHK_XR(xrGetQueriedSenseDataBD(querySnapshotHandle, &queriedDataGetInfo,
&queriedSenseData));

if (queriedSenseData.stateCountOutput > 0)
{
    std::vector<XrSpatialEntityStateBD> queriedEntityStates(queriedSenseData
.stateCountOutput);
    queriedSenseData.stateCapacityInput = queriedSenseData.stateCountOutput;
    queriedSenseData.states = queriedEntityStates.data();

    // Second call to get the spatial entities in the provider's query result
    CHK_XR(xrGetQueriedSenseDataBD(querySnapshotHandle, &queriedDataGetInfo,
&queriedSenseData));

    // Process the data
    for (int i = 0; i < queriedSenseData.stateCountOutput; ++i)
    {
        // Process the spatial entities, and get spatial entity component data
        XrSpatialEntityIdBD entityId = queriedEntityStates[i].entityId;
        XrUuidEXT uuid = queriedEntityStates[i].uuid;

        // Enumerate the component types of the entity
        uint32_t componentTypeCountOutput = 0;
        CHK_XR(xrEnumerateSpatialEntityComponentTypesBD(querySnapshotHandle, entityId, 0,
&componentTypeCountOutput, nullptr));

        if (componentTypeCountOutput > 0)
        {
            std::vector<XrSpatialEntityComponentTypeBD> entityComponentTypes
(componentTypeCountOutput);
```

```
            uint32_t componentTypeCapacityInput = componentTypeCountOutput;
            CHK_XR(xrEnumerateSpatialEntityComponentTypesBD(querySnapshotHandle,
entityId, componentTypeCapacityInput,
                                                            &componentTypeCountOutput,
entityComponentTypes.data()));

            // Check the entity component types and get the location component data
            if(hasComponentType(entityComponentTypes,
XR_SPATIAL_ENTITY_COMPONENT_TYPE_LOCATION_BD))
            {
                XrSpatialEntityLocationGetInfoBD locationGetInfo {
                    .type = XR_TYPE_SPATIAL_ENTITY_LOCATION_GET_INFO_BD,
                    .baseSpace = localSpace,
                };

                // Chain the XrSpatialEntityLocationGetInfoBD struct to
XrSpatialEntityComponentGetInfoBD when getting location component data.
                XrSpatialEntityComponentGetInfoBD componentGetInfo {
                    .type = XR_TYPE_SPATIAL_ENTITY_COMPONENT_GET_INFO_BD,
                    .next = &locationGetInfo,
                    .entityId = entityId,
                    .componentType = XR_SPATIAL_ENTITY_COMPONENT_TYPE_LOCATION_BD,
                };

                XrSpatialEntityComponentDataLocationBD componentLocationData {
                    .type = XR_TYPE_SPATIAL_ENTITY_COMPONENT_DATA_LOCATION_BD,
                };

                CHK_XR(xrGetSpatialEntityComponentDataBD(querySnapshotHandle,
&componentGetInfo,
                                                         reinterpret_cast
<XrSpatialEntityComponentDataBaseHeaderBD *>(&componentLocationData)));
                // The location data now in componentLocationData
                // ...
            }

            // Check the entity component types and get the bounding box 2D component
data
            if(hasComponentType(entityComponentTypes,
XR_SPATIAL_ENTITY_COMPONENT_TYPE_BOUNDING_BOX_2D_BD))
            {
                XrSpatialEntityComponentGetInfoBD componentGetInfo {
                    .type = XR_TYPE_SPATIAL_ENTITY_COMPONENT_GET_INFO_BD,
                    .next = nullptr,
                    .entityId = entityId,
                    .componentType = XR_SPATIAL_ENTITY_COMPONENT_TYPE_BOUNDING_BOX_2D_BD,
                };
```

```
                XrSpatialEntityComponentDataBoundingBox2DBD componentBoundingBox2DData {
                    .type = XR_TYPE_SPATIAL_ENTITY_COMPONENT_DATA_BOUNDING_BOX_2D_BD,
                };

                CHK_XR(xrGetSpatialEntityComponentDataBD(querySnapshotHandle,
&componentGetInfo,
                                                         reinterpret_cast
<XrSpatialEntityComponentDataBaseHeaderBD *>(&componentBoundingBox2DData)));
                // The bounding box 2D data now in componentBoundingBox2DData
                // The bounding box 2D data is an XrRect2Df defining the offset and
extent along the x-axis (width) and y-axis (height).
                // The x-axis and y-axis are defined by the spatial entity's location
component data

            }

            // Check the entity component types and get the triangle mesh component data
            if(hasComponentType(entityComponentTypes,
XR_SPATIAL_ENTITY_COMPONENT_TYPE_TRIANGLE_MESH_BD))
            {
                XrSpatialEntityComponentGetInfoBD componentGetInfo {
                    .type = XR_TYPE_SPATIAL_ENTITY_COMPONENT_GET_INFO_BD,
                    .next = nullptr,
                    .entityId = entityId,
                    .componentType = XR_SPATIAL_ENTITY_COMPONENT_TYPE_TRIANGLE_MESH_BD,
                };

                XrSpatialEntityComponentDataTriangleMeshBD componentTriangleMeshData {
                    .type = XR_TYPE_SPATIAL_ENTITY_COMPONENT_DATA_TRIANGLE_MESH_BD,
                    .next = nullptr,
                    .vertexCapacityInput = 0,
                    .indexCapacityInput = 0,
                };

                // First call to get vertex and index counts.
                CHK_XR(xrGetSpatialEntityComponentDataBD(querySnapshotHandle,
&componentGetInfo,
                                                         reinterpret_cast
<XrSpatialEntityComponentDataBaseHeaderBD *>(&componentTriangleMeshData)));

                if(componentTriangleMeshData.vertexCountOutput > 0 &&
componentTriangleMeshData.indexCountOutput > 0)
                {
                    std::vector<XrVector3f> vertices(componentTriangleMeshData
.vertexCountOutput);
                    std::vector<uint16_t> indices(componentTriangleMeshData
.indexCountOutput);
```

```
                    componentTriangleMeshData.vertexCapacityInput =
componentTriangleMeshData.vertexCountOutput;
                    componentTriangleMeshData.vertices = vertices.data();
                    componentTriangleMeshData.indexCapacityInput =
componentTriangleMeshData.indexCountOutput;
                    componentTriangleMeshData.indices = indices.data();

                    // Second call to get vertices and indices .
                    CHK_XR(xrGetSpatialEntityComponentDataBD(querySnapshotHandle,
&componentGetInfo,
                                                             reinterpret_cast
<XrSpatialEntityComponentDataBaseHeaderBD *>(&componentTriangleMeshData)));

                    // The triangle mesh data now in componentTriangleMeshData
                    // The vertices are XrVector3f structs defining the positions along
the x-axis, y-axis and z-axis.
                    // The x-axis, y-axis and z-axis are defined by the spatial entity's
location component data.
                    // ...

                }

            }
        }
    }
}
```

12.92.26. Issues

12.92.27. Version History

- Revision 1, 2024-05-06 (Zhipeng Liu)
  - Initial extension description
- Revision 2, 2025-08-05 (Zijian Wang)
  - Add the sphere component data struct

12.93. XR_BD_ultra_controller_interaction

Name String

`XR_BD_ultra_controller_interaction`

Extension Type

Instance extension

Registered Extension Number

404

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

API Interactions

- Interacts with `XR_EXT_dpad_binding`
- Interacts with `XR_EXT_hand_interaction`
- Interacts with `XR_EXT_palm_pose`

Last Modified Date

2025-04-15

IP Status

No known IP claims.

Contributors

Shanliang Xu, ByteDance
Shuai Liu, ByteDance
Chenxi Bao,ByteDance
Chen Han,ByteDance
Zhao Jiabin,ByteDance

12.93.1. Overview

This extension defines the interaction profile for PICO Ultra Controllers.

Interaction profile path for PICO Ultra Controllers:

- */interaction_profiles/bytedance/pico_ultra_controller_bd*

Valid for user paths for pico_ultra_controller_bd:

- */user/hand/left*
- */user/hand/right*

Supported component paths for pico_ultra_controller_bd:

- On /user/hand/left only:
  - *.../input/x/click*
  - *.../input/x/touch*
  - *.../input/y/click*
  - *.../input/y/touch*
  - *.../input/menu/click*
- On /user/hand/right only:
  - *.../input/a/click*
  - *.../input/a/touch*
  - *.../input/b/click*
  - *.../input/b/touch*
- *.../input/system/click* (**may** not be available for application use)
- *.../input/trigger/click*
- *.../input/trigger/value*
- *.../input/trigger/touch*
- *.../input/thumbstick/y*
- *.../input/thumbstick/x*
- *.../input/thumbstick/click*
- *.../input/thumbstick/touch*
- *.../input/squeeze/click*
- *.../input/squeeze/value*
- *.../input/grip/pose*
- *.../input/grip_surface/pose*
- *.../input/aim/pose*
- *.../output/haptic*

Be careful with the following component:

- Ultra Controller support *.../input/menu/click* only on */user/hand/left*.



Note

When the `XR_EXT_palm_pose` extension is available and enabled, this interaction profile **must** also support

- *.../input/palm_ext/pose*



Note

When the `XR_EXT_hand_interaction` extension is available and enabled, this interaction profile **must** also support

- *.../input/pinch_ext/pose*
- *.../input/poke_ext/pose*

12.93.2. New Enum Constants

- `XR_BD_ULTRA_CONTROLLER_INTERACTION_EXTENSION_NAME`
- `XR_BD_ultra_controller_interaction_SPEC_VERSION`

12.93.3. Issues

12.93.4. Version History

- Revision 1, 2025-04-15 (Shanliang Xu)
  - Initial extension description

12.94. XR_EPIC_view_configuration_fov

Name String

`XR_EPIC_view_configuration_fov`

Extension Type

Instance extension

Registered Extension Number

60

Revision

2

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Last Modified Date

2020-03-05

IP Status

No known IP claims.

Contributors

Jules Blok, Epic Games

Overview

This extension allows the application to retrieve the recommended and maximum field-of-view using xrEnumerateViewConfigurationViews. These field-of-view parameters can be used during initialization of the application before creating a session.

The field-of-view given here **should** not be used for rendering, see xrLocateViews to retrieve the field-of-view for rendering.

For views with `fovMutable` set to `XR_TRUE` the maximum field-of-view **should** specify the upper limit that runtime can support. If the view has `fovMutable` set to `XR_FALSE` the runtime **must** set `maxMutableFov` to be the same as `recommendedFov`.

New Object Types

New Flag Types

New Enum Constants

New Enums

New Structures

The XrViewConfigurationViewFovEPIC structure is an output struct which can be added to the next chain of XrViewConfigurationView to retrieve the field-of-view for that view.

```
// Provided by XR_EPIC_view_configuration_fov
typedef struct XrViewConfigurationViewFovEPIC {
    XrStructureType    type;
    const void*        next;
    XrFovf             recommendedFov;
    XrFovf             maxMutableFov;
} XrViewConfigurationViewFovEPIC;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `recommendedFov` is the recommended field-of-view based on the current user IPD.
- `maxMutableFov` is the maximum field-of-view that the runtime can display.

Valid Usage (Implicit)

- The `XR_EPIC_view_configuration_fov` extension **must** be enabled prior to using XrViewConfigurationViewFovEPIC
- `type` **must** be `XR_TYPE_VIEW_CONFIGURATION_VIEW_FOV_EPIC`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

New Functions

Issues

Version History

- Revision 2, 2020-06-04 (Jules Blok)
  - Fixed incorrect member name.
- Revision 1, 2020-03-05 (Jules Blok)
  - Initial version.

12.95. XR_FB_android_surface_swapchain_create

Name String

`XR_FB_android_surface_swapchain_create`

Extension Type

Instance extension

Registered Extension Number

71

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0
and
XR_KHR_android_surface_swapchain

Contributors

Cass Everitt, Facebook
Gloria Kennickell, Facebook
Tomislav Novak, Facebook

Overview

This extension provides support for the specification of Android Surface specific swapchain create flags.

These additional create flags are specified by attaching a XrAndroidSurfaceSwapchainCreateInfoFB structure to the `next` chain of an XrSwapchainCreateInfo structure.

New Object Types

New Flag Types

```
// Provided by XR_FB_android_surface_swapchain_create
typedef XrFlags64 XrAndroidSurfaceSwapchainFlagsFB;
```

```
// Provided by XR_FB_android_surface_swapchain_create
// Flag bits for XrAndroidSurfaceSwapchainFlagsFB
static const XrAndroidSurfaceSwapchainFlagsFB
XR_ANDROID_SURFACE_SWAPCHAIN_SYNCHRONOUS_BIT_FB = 0x00000001;
static const XrAndroidSurfaceSwapchainFlagsFB
XR_ANDROID_SURFACE_SWAPCHAIN_USE_TIMESTAMPS_BIT_FB = 0x00000002;
```

Flag Descriptions

- XR_ANDROID_SURFACE_SWAPCHAIN_SYNCHRONOUS_BIT_FB indicates the underlying BufferQueue should be created in synchronous mode, allowing multiple buffers to be queued instead of always replacing the last buffer. Buffers are retired in order, and the producer may block until a new buffer is available.
- XR_ANDROID_SURFACE_SWAPCHAIN_USE_TIMESTAMPS_BIT_FB indicates the compositor should acquire the most recent buffer whose presentation timestamp is not greater than the expected display time of the final composited frame.

New Enum Constants

XrStructureType enumeration is extended with:

- XR_TYPE_ANDROID_SURFACE_SWAPCHAIN_CREATE_INFO_FB

New Enums

- XR_ANDROID_SURFACE_SWAPCHAIN_SYNCHRONOUS_BIT_FB
- XR_ANDROID_SURFACE_SWAPCHAIN_USE_TIMESTAMPS_BIT_FB

New Structures

The XrAndroidSurfaceSwapchainCreateInfoFB structure is defined as:

```
// Provided by XR_FB_android_surface_swapchain_create
typedef struct XrAndroidSurfaceSwapchainCreateInfoFB {
    XrStructureType                       type;
    const void*                           next;
    XrAndroidSurfaceSwapchainFlagsFB      createFlags;
} XrAndroidSurfaceSwapchainCreateInfoFB;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- createFlags is 0 or one or more XrAndroidSurfaceSwapchainFlagBitsFB which indicate various characteristics desired for the Android Surface Swapchain.

XrAndroidSurfaceSwapchainCreateInfoFB contains additional Android Surface specific create flags when calling xrCreateSwapchainAndroidSurfaceKHR. The XrAndroidSurfaceSwapchainCreateInfoFB

structure **must** be provided in the `next` chain of the XrSwapchainCreateInfo structure when calling xrCreateSwapchainAndroidSurfaceKHR.

Valid Usage (Implicit)

- The `XR_FB_android_surface_swapchain_create` extension **must** be enabled prior to using XrAndroidSurfaceSwapchainCreateInfoFB
- `type` **must** be `XR_TYPE_ANDROID_SURFACE_SWAPCHAIN_CREATE_INFO_FB`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `createFlags` **must** be a valid combination of XrAndroidSurfaceSwapchainFlagBitsFB values
- `createFlags` **must** not be `0`

New Functions

Issues

Version History

- Revision 1, 2020-12-10 (Gloria Kennickell)
  - Initial draft

12.96. XR_FB_body_tracking

Name String

`XR_FB_body_tracking`

Extension Type

Instance extension

Registered Extension Number

77

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Last Modified Date

2022-07-18

IP Status

No known IP claims.

Contributors

Giancarlo Di Biase, Meta
Dikpal Reddy, Meta
Igor Tceglevskii, Meta

12.96.1. Overview

This extension enables applications to locate the individual body joints that represent the estimated position of the user of the device. It enables applications to render the upper body in XR experiences.

12.96.2. Inspect system capability

An application **can** inspect whether the system is capable of body tracking by extending the XrSystemProperties with XrSystemBodyTrackingPropertiesFB structure when calling xrGetSystemProperties.

```
// Provided by XR_FB_body_tracking
typedef struct XrSystemBodyTrackingPropertiesFB {
    XrStructureType    type;
    void*              next;
    XrBool32           supportsBodyTracking;
} XrSystemBodyTrackingPropertiesFB;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension. For more details about the structure chain, see the structure being extended (XrSystemProperties).
- `supportsBodyTracking` is an `XrBool32`, indicating if current system is capable of receiving body tracking input.

If a runtime returns `XR_FALSE` for `supportsBodyTracking`, the runtime **must** return `XR_ERROR_FEATURE_UNSUPPORTED` from xrCreateBodyTrackerFB.

Valid Usage (Implicit)

- The XR_FB_body_tracking extension **must** be enabled prior to using XrSystemBodyTrackingPropertiesFB
- type **must** be XR_TYPE_SYSTEM_BODY_TRACKING_PROPERTIES_FB
- next **must** be NULL or a valid pointer to the next structure in a structure chain

12.96.3. Create a body tracker handle

The XrBodyTrackerFB handle represents the resources for body tracking.

```
// Provided by XR_FB_body_tracking
XR_DEFINE_HANDLE(XrBodyTrackerFB)
```

This handle **can** be used to locate body joints using xrLocateBodyJointsFB function.

A body tracker provides joint locations with an unobstructed range of human body motion.

It also provides the estimated scale of this body.

An application **can** create an XrBodyTrackerFB handle using xrCreateBodyTrackerFB function.

```
// Provided by XR_FB_body_tracking
XrResult xrCreateBodyTrackerFB(
    XrSession                                   session,
    const XrBodyTrackerCreateInfoFB*            createInfo,
    XrBodyTrackerFB*                            bodyTracker);
```

Parameter Descriptions

- session is an XrSession in which the body tracker will be active.
- createInfo is the XrBodyTrackerCreateInfoFB used to specify the body tracker.
- bodyTracker is the returned XrBodyTrackerFB handle.

If the system does not support body tracking, the runtime **must** return XR_ERROR_FEATURE_UNSUPPORTED from xrCreateBodyTrackerFB. In this case, the runtime **must** return XR_FALSE for XrSystemBodyTrackingPropertiesFB::supportsBodyTracking when the function xrGetSystemProperties is called, so that the application **can** avoid creating a body tracker.

Valid Usage (Implicit)

- The XR_FB_body_tracking extension **must** be enabled prior to calling xrCreateBodyTrackerFB
- session **must** be a valid XrSession handle
- createInfo **must** be a pointer to a valid XrBodyTrackerCreateInfoFB structure
- bodyTracker **must** be a pointer to an XrBodyTrackerFB handle

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FEATURE_UNSUPPORTED
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The XrBodyTrackerCreateInfoFB structure describes the information to create an XrBodyTrackerFB handle.

```
// Provided by XR_FB_body_tracking
typedef struct XrBodyTrackerCreateInfoFB {
    XrStructureType     type;
    const void*         next;
    XrBodyJointSetFB    bodyJointSet;
} XrBodyTrackerCreateInfoFB;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `bodyJointSet` is an XrBodyJointSetFB that describes the set of body joints to retrieve.

Valid Usage (Implicit)

- The `XR_FB_body_tracking` extension **must** be enabled prior to using XrBodyTrackerCreateInfoFB
- `type` **must** be `XR_TYPE_BODY_TRACKER_CREATE_INFO_FB`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `bodyJointSet` **must** be a valid XrBodyJointSetFB value

The XrBodyJointSetFB enum describes the set of body joints to track when creating an XrBodyTrackerFB.

```
// Provided by XR_FB_body_tracking
typedef enum XrBodyJointSetFB {
    XR_BODY_JOINT_SET_DEFAULT_FB = 0,
  // Provided by XR_META_body_tracking_full_body
    XR_BODY_JOINT_SET_FULL_BODY_META = 1000274000,
    XR_BODY_JOINT_SET_MAX_ENUM_FB = 0x7FFFFFFF
} XrBodyJointSetFB;
```

Enumerant Descriptions

- `XR_BODY_JOINT_SET_DEFAULT_FB` — Indicates that the created XrBodyTrackerFB tracks the set of body joints described by XrBodyJointFB enum, i.e. the xrLocateBodyJointsFB function returns an array of joint locations with the count of `XR_BODY_JOINT_COUNT_FB` and can be indexed using XrBodyJointFB. This includes the upper body, both hands, and torso.
- `XR_BODY_JOINT_SET_FULL_BODY_META` — Indicates that the created XrBodyTrackerFB tracks the set of body joints described by XrFullBodyJointMETA enum, i.e. the xrLocateBodyJointsFB function returns an array of joint locations with the count of `XR_FULL_BODY_JOINT_COUNT_META` and can be indexed using XrFullBodyJointMETA. This extends the joints of `XR_BODY_JOINT_SET_DEFAULT_FB` by also including the lower body. (Added by the `XR_META_body_tracking_full_body` extension)

xrDestroyBodyTrackerFB function releases the bodyTracker and the underlying resources when the body tracking experience is over.

```
// Provided by XR_FB_body_tracking
XrResult xrDestroyBodyTrackerFB(
    XrBodyTrackerFB                             bodyTracker);
```

Parameter Descriptions

- bodyTracker is an XrBodyTrackerFB previously created by xrCreateBodyTrackerFB.

Valid Usage (Implicit)

- The XR_FB_body_tracking extension **must** be enabled prior to calling xrDestroyBodyTrackerFB
- bodyTracker **must** be a valid XrBodyTrackerFB handle

Thread Safety

- Access to bodyTracker, and any child handles, **must** be externally synchronized

Return Codes

Success

- XR_SUCCESS

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_RUNTIME_FAILURE

12.96.4. Locate body joints

The xrLocateBodyJointsFB function locates an array of body joints to a base space at a given time.

```
// Provided by XR_FB_body_tracking
XrResult xrLocateBodyJointsFB(
    XrBodyTrackerFB                             bodyTracker,
    const XrBodyJointsLocateInfoFB*             locateInfo,
    XrBodyJointLocationsFB*                     locations);
```

Parameter Descriptions

- `bodyTracker` is an XrBodyTrackerFB previously created by xrCreateBodyTrackerFB.
- `locateInfo` is a pointer to XrBodyJointsLocateInfoFB describing information to locate body joints.
- `locations` is a pointer to XrBodyJointLocationsFB receiving the returned body joint locations.

Valid Usage (Implicit)

- The `XR_FB_body_tracking` extension **must** be enabled prior to calling xrLocateBodyJointsFB
- `bodyTracker` **must** be a valid XrBodyTrackerFB handle
- `locateInfo` **must** be a pointer to a valid XrBodyJointsLocateInfoFB structure
- `locations` **must** be a pointer to an XrBodyJointLocationsFB structure

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_TIME_INVALID`
- `XR_ERROR_VALIDATION_FAILURE`

The XrBodyJointsLocateInfoFB structure describes the information to locate body joints.

```
// Provided by XR_FB_body_tracking
typedef struct XrBodyJointsLocateInfoFB {
    XrStructureType    type;
    const void*        next;
    XrSpace            baseSpace;
    XrTime             time;
} XrBodyJointsLocateInfoFB;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `baseSpace` is an XrSpace within which the returned body joint locations will be represented.
- `time` is an `XrTime` at which to locate the body joints.

Callers **should** request a time equal to the predicted display time for the rendered frame. The system will employ appropriate modeling to support body tracking at this time.

Valid Usage (Implicit)

- The `XR_FB_body_tracking` extension **must** be enabled prior to using XrBodyJointsLocateInfoFB
- `type` **must** be `XR_TYPE_BODY_JOINTS_LOCATE_INFO_FB`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `baseSpace` **must** be a valid XrSpace handle

XrBodyJointLocationsFB structure returns the state of the body joint locations.

```
// Provided by XR_FB_body_tracking
typedef struct XrBodyJointLocationsFB {
    XrStructureType           type;
    void*                     next;
    XrBool32                  isActive;
    float                     confidence;
    uint32_t                  jointCount;
    XrBodyJointLocationFB*    jointLocations;
    uint32_t                  skeletonChangedCount;
    XrTime                    time;
} XrBodyJointLocationsFB;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `isActive` is an `XrBool32` indicating if the body tracker is actively tracking.
- `confidence` is a `float` between 0 and 1 which represents the confidence for the returned body pose. A value of 0 means there is no confidence in the pose returned, and a value of 1 means maximum confidence in the returned body pose.
- `jointCount` is a `uint32_t` describing the count of elements in `jointLocations` array.
- `jointLocations` is an application-allocated array of XrBodyJointLocationFB that will be filled with joint locations.
- `skeletonChangedCount` is an output `uint32_t` incremental counter indicating that the skeleton scale proportions have changed. xrGetBodySkeletonFB **can** be called when this counter increases to get the latest body proportions/scale.
- `time` is an `XrTime` time at which the returned joints are tracked. Equals the time at which the joints were requested if the interpolation at the time was successful.

The runtime **must** return `XR_ERROR_VALIDATION_FAILURE` if `jointCount` does not equal to the number of joints defined by the XrBodyJointSetFB used to create the XrBodyTrackerFB.

The runtime **must** return `jointLocations` representing the range of human body motion, without any obstructions. Input systems that either obstruct the movement of the user's body (for example, a held controller preventing the user from making a fist) or input systems that have only limited ability to track finger positions **must** use the information available to them to emulate an unobstructed range of motion.

The runtime **must** update the `jointLocations` array ordered so that it is indexed using the corresponding body joint enum (e.g. XrBodyJointFB) as described by XrBodyJointSetFB when creating the XrBodyTrackerFB. For example, when the XrBodyTrackerFB is created with `XR_BODY_JOINT_SET_DEFAULT_FB`, the application **must** set the `jointCount` to `XR_BODY_JOINT_COUNT_FB`, and the runtime **must** fill the `jointLocations` array ordered so that it is indexed by the XrBodyJointFB enum.

If the returned `isActive` is true, the runtime **must** return all joint locations with both `XR_SPACE_LOCATION_POSITION_VALID_BIT` and `XR_SPACE_LOCATION_ORIENTATION_VALID_BIT` set. However, in this case, some joint space locations **may** be untracked (i.e. `XR_SPACE_LOCATION_POSITION_TRACKED_BIT` or `XR_SPACE_LOCATION_ORIENTATION_TRACKED_BIT` is unset).

If the returned `isActive` is false, it indicates that the body tracker did not detect the body input, the application lost input focus, or the consent for body tracking was denied by the user. In this case, the runtime **must** return all `jointLocations` with neither `XR_SPACE_LOCATION_POSITION_VALID_BIT` nor

XR_SPACE_LOCATION_ORIENTATION_VALID_BIT set.

Valid Usage (Implicit)

- The XR_FB_body_tracking extension **must** be enabled prior to using XrBodyJointLocationsFB
- type **must** be XR_TYPE_BODY_JOINT_LOCATIONS_FB
- next **must** be NULL or a valid pointer to the next structure in a structure chain. See also: XrBodyTrackingCalibrationStatusMETA, XrBodyTrackingFidelityStatusMETA

XrBodyJointLocationFB structure describes the position, orientation, and radius of a body joint.

```
// Provided by XR_FB_body_tracking
typedef struct XrBodyJointLocationFB {
    XrSpaceLocationFlags    locationFlags;
    XrPosef                 pose;
} XrBodyJointLocationFB;
```

Member Descriptions

- locationFlags is a bitfield, with bit masks defined in XrSpaceLocationFlagBits, to indicate which members contain valid data. If none of the bits are set, no other fields in this structure **should** be considered to be valid or meaningful.
- pose is an XrPosef defining the position and orientation of the origin of a body joint within the reference frame of the corresponding XrBodyJointsLocateInfoFB::baseSpace.

Valid Usage (Implicit)

- The XR_FB_body_tracking extension **must** be enabled prior to using XrBodyJointLocationFB
- locationFlags **must** be a valid combination of XrSpaceLocationFlagBits values
- locationFlags **must** not be 0

12.96.5. Retrieve body skeleton

The xrGetBodySkeletonFB function returns the body skeleton in T-pose.

```
// Provided by XR_FB_body_tracking
XrResult xrGetBodySkeletonFB(
    XrBodyTrackerFB                             bodyTracker,
    XrBodySkeletonFB*                           skeleton);
```

Parameter Descriptions

- `bodyTracker` is an XrBodyTrackerFB previously created by xrCreateBodyTrackerFB.
- `skeleton` is a pointer to XrBodySkeletonFB receiving the returned body skeleton hierarchy.

This function **can** be used to query the skeleton scale and proportions in conjunction with XrBodyJointLocationsFB::`skeletonChangedCount`. XrBodyJointLocationsFB::`skeletonChangedCount` is incremented whenever the tracking auto-calibrates the user skeleton scale and proportions.

Valid Usage (Implicit)

- The `XR_FB_body_tracking` extension **must** be enabled prior to calling xrGetBodySkeletonFB
- `bodyTracker` **must** be a valid XrBodyTrackerFB handle
- `skeleton` **must** be a pointer to an XrBodySkeletonFB structure

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_VALIDATION_FAILURE`

The XrBodySkeletonFB structure is a container to represent the body skeleton in T-pose including the joint hierarchy.

```
// Provided by XR_FB_body_tracking
typedef struct XrBodySkeletonFB {
    XrStructureType           type;
    void*                     next;
    uint32_t                  jointCount;
    XrBodySkeletonJointFB*    joints;
} XrBodySkeletonFB;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `jointCount` is an `uint32_t` describing the count of elements in `joints` array.
- `joints` is an application-allocated array of XrBodySkeletonJointFB that will be filled with skeleton joint elements.

The runtime **must** return `XR_ERROR_VALIDATION_FAILURE` if `jointCount` does not equal to the number of joints defined by the XrBodyJointSetFB used to create the XrBodyTrackerFB.

The runtime **must** return `joints` representing the default pose of the current estimation regarding the user's skeleton.

Valid Usage (Implicit)

- The `XR_FB_body_tracking` extension **must** be enabled prior to using XrBodySkeletonFB
- `type` **must** be `XR_TYPE_BODY_SKELETON_FB`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `joints` **must** be a pointer to an array of `jointCount` XrBodySkeletonJointFB structures
- The `jointCount` parameter **must** be greater than `0`

XrBodySkeletonJointFB structure describes the position, orientation of the joint in space, and position of the joint in the skeleton hierarchy.

```
// Provided by XR_FB_body_tracking
typedef struct XrBodySkeletonJointFB {
    int32_t    joint;
    int32_t    parentJoint;
    XrPosef    pose;
} XrBodySkeletonJointFB;
```

Member Descriptions

- `joint` is an index of a joint using the corresponding body joint enum (e.g. XrBodyJointFB).
- `parentJoint` is an index of a parent joint of that joint, using the corresponding body joint enum (e.g. XrBodyJointFB).
- `pose` is an XrPosef defining the position and orientation of the origin of a body joint within the reference frame of the corresponding XrBodyJointsLocateInfoFB::`baseSpace`.

Valid Usage (Implicit)

- The `XR_FB_body_tracking` extension **must** be enabled prior to using XrBodySkeletonJointFB

12.96.6. Example code for locating body joints

The following example code demonstrates how to locate all body joints relatively to a base space.

```
XrInstance instance; // previously initialized
XrSystemId systemId; // previously initialized
XrSession session;   // previously initialized
XrSpace baseSpace;  // previously initialized, e.g. from
                     // XR_REFERENCE_SPACE_TYPE_LOCAL

// Inspect body tracking system properties
XrSystemBodyTrackingPropertiesFB bodyTrackingSystemProperties{
    .type = XR_TYPE_SYSTEM_BODY_TRACKING_PROPERTIES_FB,
};
XrSystemProperties systemProperties{
    .type = XR_TYPE_SYSTEM_PROPERTIES,
    .next = &bodyTrackingSystemProperties,
};
CHK_XR(xrGetSystemProperties(instance, systemId, &systemProperties));
if (!bodyTrackingSystemProperties.supportsBodyTracking) {
    // The system does not support body tracking
    return;
```

```
}

// Get function pointer for xrCreateBodyTrackerFB
PFN_xrCreateBodyTrackerFB pfnCreateBodyTrackerFB = nullptr;
CHK_XR(xrGetInstanceProcAddr(instance, "xrCreateBodyTrackerFB",
                             reinterpret_cast<PFN_xrVoidFunction*>(
                             &pfnCreateBodyTrackerFB)));

// Create a body tracker that tracks default set of body joints.
XrBodyTrackerFB bodyTracker{XR_NULL_HANDLE};
{
    XrBodyTrackerCreateInfoFB createInfo{
        .type = XR_TYPE_BODY_TRACKER_CREATE_INFO_FB,
        .next = nullptr,
        .bodyJointSet = XR_BODY_JOINT_SET_DEFAULT_FB,
    };
    CHK_XR(pfnCreateBodyTrackerFB(session, &createInfo, &bodyTracker));
}

// Allocate buffers to receive joint location data before frame
// loop starts.
XrBodyJointLocationFB jointLocations[XR_BODY_JOINT_COUNT_FB];
XrBodyJointLocationsFB locations{
    .type = XR_TYPE_BODY_JOINT_LOCATIONS_FB,
    .next = nullptr,
    .jointCount = XR_BODY_JOINT_COUNT_FB,
    .jointLocations = jointLocations,
};

// Get function pointer for xrLocateBodyJointsFB.
PFN_xrLocateBodyJointsFB pfnLocateBodyJointsFB = nullptr;
CHK_XR(xrGetInstanceProcAddr(instance, "xrLocateBodyJointsFB",
                             reinterpret_cast<PFN_xrVoidFunction*>(
                             &pfnLocateBodyJointsFB)));
while (1) {
    // ...
    // For every frame in the frame loop
    // ...
    XrFrameState frameState; // previously returned from xrWaitFrame
    const XrTime time = frameState.predictedDisplayTime;

    XrBodyJointsLocateInfoFB locateInfo{
        .type = XR_TYPE_BODY_JOINTS_LOCATE_INFO_FB,
        .next = nullptr,
        .baseSpace = baseSpace,
        .time = time,
    };
```

```
    CHK_XR(pfnLocateBodyJointsFB(bodyTracker, &locateInfo, &locations));

    if (locations.isActive) {
        // The returned joint location array is directly indexed with
        // XrBodyJointFB enum.
        const XrPosef &indexTip =
            jointLocations[XR_BODY_JOINT_LEFT_HAND_INDEX_TIP_FB].pose;
    }
}
```

12.96.7. Conventions of body joints

This extension defines 70 joints for body tracking: 18 core body joints + 52 hand joints.

```
// Provided by XR_FB_body_tracking
typedef enum XrBodyJointFB {
    XR_BODY_JOINT_ROOT_FB = 0,
    XR_BODY_JOINT_HIPS_FB = 1,
    XR_BODY_JOINT_SPINE_LOWER_FB = 2,
    XR_BODY_JOINT_SPINE_MIDDLE_FB = 3,
    XR_BODY_JOINT_SPINE_UPPER_FB = 4,
    XR_BODY_JOINT_CHEST_FB = 5,
    XR_BODY_JOINT_NECK_FB = 6,
    XR_BODY_JOINT_HEAD_FB = 7,
    XR_BODY_JOINT_LEFT_SHOULDER_FB = 8,
    XR_BODY_JOINT_LEFT_SCAPULA_FB = 9,
    XR_BODY_JOINT_LEFT_ARM_UPPER_FB = 10,
    XR_BODY_JOINT_LEFT_ARM_LOWER_FB = 11,
    XR_BODY_JOINT_LEFT_HAND_WRIST_TWIST_FB = 12,
    XR_BODY_JOINT_RIGHT_SHOULDER_FB = 13,
    XR_BODY_JOINT_RIGHT_SCAPULA_FB = 14,
    XR_BODY_JOINT_RIGHT_ARM_UPPER_FB = 15,
    XR_BODY_JOINT_RIGHT_ARM_LOWER_FB = 16,
    XR_BODY_JOINT_RIGHT_HAND_WRIST_TWIST_FB = 17,
    XR_BODY_JOINT_LEFT_HAND_PALM_FB = 18,
    XR_BODY_JOINT_LEFT_HAND_WRIST_FB = 19,
    XR_BODY_JOINT_LEFT_HAND_THUMB_METACARPAL_FB = 20,
    XR_BODY_JOINT_LEFT_HAND_THUMB_PROXIMAL_FB = 21,
    XR_BODY_JOINT_LEFT_HAND_THUMB_DISTAL_FB = 22,
    XR_BODY_JOINT_LEFT_HAND_THUMB_TIP_FB = 23,
    XR_BODY_JOINT_LEFT_HAND_INDEX_METACARPAL_FB = 24,
    XR_BODY_JOINT_LEFT_HAND_INDEX_PROXIMAL_FB = 25,
    XR_BODY_JOINT_LEFT_HAND_INDEX_INTERMEDIATE_FB = 26,
    XR_BODY_JOINT_LEFT_HAND_INDEX_DISTAL_FB = 27,
    XR_BODY_JOINT_LEFT_HAND_INDEX_TIP_FB = 28,
    XR_BODY_JOINT_LEFT_HAND_MIDDLE_METACARPAL_FB = 29,
```

```
    XR_BODY_JOINT_LEFT_HAND_MIDDLE_PROXIMAL_FB = 30,
    XR_BODY_JOINT_LEFT_HAND_MIDDLE_INTERMEDIATE_FB = 31,
    XR_BODY_JOINT_LEFT_HAND_MIDDLE_DISTAL_FB = 32,
    XR_BODY_JOINT_LEFT_HAND_MIDDLE_TIP_FB = 33,
    XR_BODY_JOINT_LEFT_HAND_RING_METACARPAL_FB = 34,
    XR_BODY_JOINT_LEFT_HAND_RING_PROXIMAL_FB = 35,
    XR_BODY_JOINT_LEFT_HAND_RING_INTERMEDIATE_FB = 36,
    XR_BODY_JOINT_LEFT_HAND_RING_DISTAL_FB = 37,
    XR_BODY_JOINT_LEFT_HAND_RING_TIP_FB = 38,
    XR_BODY_JOINT_LEFT_HAND_LITTLE_METACARPAL_FB = 39,
    XR_BODY_JOINT_LEFT_HAND_LITTLE_PROXIMAL_FB = 40,
    XR_BODY_JOINT_LEFT_HAND_LITTLE_INTERMEDIATE_FB = 41,
    XR_BODY_JOINT_LEFT_HAND_LITTLE_DISTAL_FB = 42,
    XR_BODY_JOINT_LEFT_HAND_LITTLE_TIP_FB = 43,
    XR_BODY_JOINT_RIGHT_HAND_PALM_FB = 44,
    XR_BODY_JOINT_RIGHT_HAND_WRIST_FB = 45,
    XR_BODY_JOINT_RIGHT_HAND_THUMB_METACARPAL_FB = 46,
    XR_BODY_JOINT_RIGHT_HAND_THUMB_PROXIMAL_FB = 47,
    XR_BODY_JOINT_RIGHT_HAND_THUMB_DISTAL_FB = 48,
    XR_BODY_JOINT_RIGHT_HAND_THUMB_TIP_FB = 49,
    XR_BODY_JOINT_RIGHT_HAND_INDEX_METACARPAL_FB = 50,
    XR_BODY_JOINT_RIGHT_HAND_INDEX_PROXIMAL_FB = 51,
    XR_BODY_JOINT_RIGHT_HAND_INDEX_INTERMEDIATE_FB = 52,
    XR_BODY_JOINT_RIGHT_HAND_INDEX_DISTAL_FB = 53,
    XR_BODY_JOINT_RIGHT_HAND_INDEX_TIP_FB = 54,
    XR_BODY_JOINT_RIGHT_HAND_MIDDLE_METACARPAL_FB = 55,
    XR_BODY_JOINT_RIGHT_HAND_MIDDLE_PROXIMAL_FB = 56,
    XR_BODY_JOINT_RIGHT_HAND_MIDDLE_INTERMEDIATE_FB = 57,
    XR_BODY_JOINT_RIGHT_HAND_MIDDLE_DISTAL_FB = 58,
    XR_BODY_JOINT_RIGHT_HAND_MIDDLE_TIP_FB = 59,
    XR_BODY_JOINT_RIGHT_HAND_RING_METACARPAL_FB = 60,
    XR_BODY_JOINT_RIGHT_HAND_RING_PROXIMAL_FB = 61,
    XR_BODY_JOINT_RIGHT_HAND_RING_INTERMEDIATE_FB = 62,
    XR_BODY_JOINT_RIGHT_HAND_RING_DISTAL_FB = 63,
    XR_BODY_JOINT_RIGHT_HAND_RING_TIP_FB = 64,
    XR_BODY_JOINT_RIGHT_HAND_LITTLE_METACARPAL_FB = 65,
    XR_BODY_JOINT_RIGHT_HAND_LITTLE_PROXIMAL_FB = 66,
    XR_BODY_JOINT_RIGHT_HAND_LITTLE_INTERMEDIATE_FB = 67,
    XR_BODY_JOINT_RIGHT_HAND_LITTLE_DISTAL_FB = 68,
    XR_BODY_JOINT_RIGHT_HAND_LITTLE_TIP_FB = 69,
    XR_BODY_JOINT_COUNT_FB = 70,
    XR_BODY_JOINT_NONE_FB = -1,
    XR_BODY_JOINT_MAX_ENUM_FB = 0x7FFFFFFF
} XrBodyJointFB;
```

The backward (+Z) direction is parallel to the corresponding bone and points away from the finger tip. The up (+Y) direction is pointing out of the back of and perpendicular to the corresponding finger nail

at the fully opened hand pose. The X direction is perpendicular to Y and Z and follows the right hand rule.

The wrist joint is located at the pivot point of the wrist, which is location invariant when twisting the hand without moving the forearm. The backward (+Z) direction is parallel to the line from wrist joint to middle finger metacarpal joint, and points away from the finger tips. The up (+Y) direction points out towards back of the hand and perpendicular to the skin at wrist. The X direction is perpendicular to the Y and Z directions and follows the right hand rule.

The palm joint is located at the center of the middle finger's metacarpal bone. The backward (+Z) direction is parallel to the middle finger's metacarpal bone, and points away from the finger tips. The up (+Y) direction is perpendicular to palm surface and pointing towards the back of the hand. The X direction is perpendicular to the Y and Z directions and follows the right hand rule.

Body skeleton has the full set of body joints (e.g. defined by XrBodyJointFB), organized in a hierarchy with a default T-shape body pose.

The purpose of the skeleton is to provide data about the body size. Coordinates are relative to each other, so there is no any relation to any space.

The calculation of the body size **may** be updated during a session. Each time the calculation of the size is changed, `skeletonChangedCount` of XrBodyJointLocationsFB is changed to indicate that a new skeleton **may** be retrieved.

New Object Types

- XrBodyTrackerFB

New Flag Types

New Enum Constants

- `XR_BODY_JOINT_COUNT_FB`

XrObjectType enumeration is extended with:

- `XR_OBJECT_TYPE_BODY_TRACKER_FB`

XrStructureType enumeration is extended with:

- `XR_TYPE_SYSTEM_BODY_TRACKING_PROPERTIES_FB`
- `XR_TYPE_BODY_TRACKER_CREATE_INFO_FB`
- `XR_TYPE_BODY_JOINTS_LOCATE_INFO_FB`
- `XR_TYPE_BODY_JOINT_LOCATIONS_FB`
- `XR_TYPE_BODY_SKELETON_FB`

New Enums

- XrBodyJointFB
- XrBodyJointSetFB

New Structures

- XrSystemBodyTrackingPropertiesFB
- XrBodyTrackerCreateInfoFB
- XrBodyJointsLocateInfoFB
- XrBodyJointLocationFB
- XrBodyJointLocationsFB
- XrBodySkeletonJointFB
- XrBodySkeletonFB

New Functions

- xrCreateBodyTrackerFB
- xrDestroyBodyTrackerFB
- xrLocateBodyJointsFB
- xrGetBodySkeletonFB

Issues

Version History

- Revision 1, 2022-07-18 (Igor Tceglevskii)
  - Initial extension description

12.97. XR_FB_color_space

Name String

`XR_FB_color_space`

Extension Type

Instance extension

Registered Extension Number

109

Revision

3

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Contributors

Volga Aksoy, Facebook
Cass Everitt, Facebook
Gloria Kennickell, Facebook

Overview

XR devices may use a color space that is different from many monitors used in development. Application developers may desire to specify the color space in which they have authored their application so appropriate colors are shown when the application is running on the XR device.

This extension allows:

- An application to get the native color space of the XR device.
- An application to enumerate the supported color spaces for the session.
- An application to set the color space for the session.

New Object Types

New Flag Types

New Enum Constants

XrStructureType enumeration is extended with:

- `XR_TYPE_SYSTEM_COLOR_SPACE_PROPERTIES_FB`

XrResult enumeration is extended with:

- `XR_ERROR_COLOR_SPACE_UNSUPPORTED_FB`

New Enums

The possible color spaces are specified by the XrColorSpaceFB enumeration.

```
// Provided by XR_FB_color_space
typedef enum XrColorSpaceFB {
    XR_COLOR_SPACE_UNMANAGED_FB = 0,
    XR_COLOR_SPACE_REC2020_FB = 1,
    XR_COLOR_SPACE_REC709_FB = 2,
    XR_COLOR_SPACE_RIFT_CV1_FB = 3,
    XR_COLOR_SPACE_RIFT_S_FB = 4,
    XR_COLOR_SPACE_QUEST_FB = 5,
    XR_COLOR_SPACE_P3_FB = 6,
    XR_COLOR_SPACE_ADOBE_RGB_FB = 7,
    XR_COLOR_SPACE_MAX_ENUM_FB = 0x7FFFFFFF
} XrColorSpaceFB;
```

Enumerant Descriptions

- XR_COLOR_SPACE_UNMANAGED_FB. No color correction, not recommended for production use.
- XR_COLOR_SPACE_REC2020_FB. Standard Rec. 2020 chromacities with D65 white point.
- XR_COLOR_SPACE_REC709_FB. Standard Rec. 709 chromaticities, similar to sRGB.
- XR_COLOR_SPACE_RIFT_CV1_FB. Unique color space, between P3 and Adobe RGB using D75 white point. This is the preferred color space for standardized color across all Oculus HMDs.

  Color Space Details with Chromacity Primaries in CIE 1931 xy:

  - Red: (0.666, 0.334)
  - Green: (0.238, 0.714)
  - Blue: (0.139, 0.053)
  - White: (0.298, 0.318)
- XR_COLOR_SPACE_RIFT_S_FB. Unique color space. Similar to Rec 709 using D75.

  Color Space Details with Chromacity Primaries in CIE 1931 xy:

  - Red: (0.640, 0.330)
  - Green: (0.292, 0.586)
  - Blue: (0.156, 0.058)
  - White: (0.298, 0.318)
- XR_COLOR_SPACE_QUEST_FB. Unique color space. Similar to Rift CV1 using D75 white point

  Color Space Details with Chromacity Primaries in CIE 1931 xy:

  - Red: (0.661, 0.338)
  - Green: (0.228, 0.718)
  - Blue: (0.142, 0.042)
  - White: (0.298, 0.318)
- XR_COLOR_SPACE_P3_FB. Similar to DCI-P3, but uses D65 white point instead.

  Color Space Details with Chromacity Primaries in CIE 1931 xy:

  - Red: (0.680, 0.320)
  - Green: (0.265, 0.690)
  - Blue: (0.150, 0.060)
  - White: (0.313, 0.329)
- XR_COLOR_SPACE_ADOBE_RGB_FB. Standard Adobe chromacities.

New Structures

An application may inspect the native color space of the system by chaining an XrSystemColorSpacePropertiesFB structure to the XrSystemProperties when calling xrGetSystemProperties.

The XrSystemColorSpacePropertiesFB structure is defined as:

```
// Provided by XR_FB_color_space
typedef struct XrSystemColorSpacePropertiesFB {
    XrStructureType    type;
    void*              next;
    XrColorSpaceFB     colorSpace;
} XrSystemColorSpacePropertiesFB;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension. For more details about the structure chain, see the structure being extended (XrSystemProperties).
- `colorSpace` is the native color space of the XR device.

Valid Usage (Implicit)

- The `XR_FB_color_space` extension **must** be enabled prior to using XrSystemColorSpacePropertiesFB
- `type` **must** be `XR_TYPE_SYSTEM_COLOR_SPACE_PROPERTIES_FB`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

New Functions

The xrEnumerateColorSpacesFB function is defined as:

```
// Provided by XR_FB_color_space
XrResult xrEnumerateColorSpacesFB(
    XrSession                                   session,
    uint32_t                                    colorSpaceCapacityInput,
    uint32_t*                                   colorSpaceCountOutput,
    XrColorSpaceFB*                             colorSpaces);
```

Parameter Descriptions

- `session` is the session that enumerates the supported color spaces.
- `colorSpaceCapacityInput` is the capacity of the `colorSpaces` array, or 0 to retrieve the required capacity.
- `colorSpaceCountOutput` is a pointer to the count of XrColorSpaceFB `colorSpaces` written, or a pointer to the required capacity in the case that `colorSpaceCapacityInput` is insufficient.
- `colorSpaces` is a pointer to an array of XrColorSpaceFB color spaces, but **can** be `NULL` if `colorSpaceCapacityInput` is `0`.
- See the Buffer Size Parameters section for a detailed description of retrieving the required `colorSpaces` size.

xrEnumerateColorSpacesFB enumerates the color spaces supported by the current session. Runtimes **must** always return identical buffer contents from this enumeration for the lifetime of the session.

Valid Usage (Implicit)

- The `XR_FB_color_space` extension **must** be enabled prior to calling xrEnumerateColorSpacesFB
- `session` **must** be a valid XrSession handle
- `colorSpaceCountOutput` **must** be a pointer to a `uint32_t` value
- If `colorSpaceCapacityInput` is not `0`, `colorSpaces` **must** be a pointer to an array of `colorSpaceCapacityInput` XrColorSpaceFB values

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_SIZE_INSUFFICIENT
- XR_ERROR_VALIDATION_FAILURE

The xrSetColorSpaceFB function is defined as:

```
// Provided by XR_FB_color_space
XrResult xrSetColorSpaceFB(
    XrSession                                   session,
    const XrColorSpaceFB                        colorSpace);
```

Parameter Descriptions

- session is a valid XrSession handle.
- colorSpace is a supported color space. Supported color spaces are indicated by xrEnumerateColorSpacesFB.

xrSetColorSpaceFB provides a mechanism for an application to specify the color space used in the final rendered frame. If this function is not called, the session will use the color space deemed appropriate by the runtime. Oculus HMDs for both PC and Mobile product lines default to XR_COLOR_SPACE_RIFT_CV1_FB. The runtime **must** return XR_ERROR_COLOR_SPACE_UNSUPPORTED_FB if colorSpace is not one of the values enumerated by xrEnumerateColorSpacesFB.

Formal definitions of color spaces contain a number of aspects such as gamma correction, max luminance and more. However, xrSetColorSpaceFB will only affect the color gamut of the output by transforming the color gamut from the source (defined by the colorSpace parameter) to the HMD display's color gamut (defined by the hardware internally). This call will not affect gamma correction, leaving that to follow the GPU texture format standards. Luminance, tonemapping, and other aspects

of the color space will also remain unaffected.

For more info on color management in Oculus HMDs, please refer to this guide: Color Management in Oculus Headsets

Valid Usage (Implicit)

- The `XR_FB_color_space` extension **must** be enabled prior to calling xrSetColorSpaceFB
- `session` **must** be a valid XrSession handle
- `colorSpace` **must** be a valid XrColorSpaceFB value

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_COLOR_SPACE_UNSUPPORTED_FB`
- `XR_ERROR_FEATURE_UNSUPPORTED`
- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_VALIDATION_FAILURE`

Issues

Version History

- Revision 1, 2020-11-09 (Gloria Kennickell)
  - Initial extension description
- Revision 2, 2021-09-28 (Rylie Pavlik, Collabora, Ltd.)
  - Fix XML markup to indicate that `XrSystemColorSpacePropertiesFB` is chained to `XrSystemProperties`.
- Revision 3, 2022-09-01 (Rylie Pavlik, Collabora, Ltd.)

- Fix XML markup to indicate that `XrSystemColorSpacePropertiesFB` is returned-only.

12.98. XR_FB_composition_layer_alpha_blend

Name String

`XR_FB_composition_layer_alpha_blend`

Extension Type

Instance extension

Registered Extension Number

42

Revision

3

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Contributors

Cass Everitt, Facebook
Gloria Kennickell, Facebook
Johannes Schmid, Facebook

Overview

This extension provides explicit control over source and destination blend factors, with separate controls for color and alpha. When specified, these blend controls supersede the behavior of `XR_COMPOSITION_LAYER_BLEND_TEXTURE_SOURCE_ALPHA_BIT`, as well as the layer blending operation which defined in the section of Composition Layer Blending.

When `XR_COMPOSITION_LAYER_UNPREMULTIPLIED_ALPHA_BIT` is specified, the source color is unpremultiplied alpha.

Like color, destination alpha is initialized to 0 before composition begins.

These blend factors are specified by attaching a XrCompositionLayerAlphaBlendFB structure to the `next` chain of a layer structure derived from XrCompositionLayerBaseHeader.

Warning

Please note that using this extension is mutually exclusive with accessing an environment blend mode other than XR_ENVIRONMENT_BLEND_MODE_OPAQUE. The latter **must** be the only mode returned from xrEnumerateEnvironmentBlendModes if this extension is enabled. Application **should** not enable this extension if a different value of XrEnvironmentBlendMode is required.

New Object Types

New Flag Types

New Enum Constants

XrStructureType enumeration is extended with:

- XR_TYPE_COMPOSITION_LAYER_ALPHA_BLEND_FB

New Enums

The possible blend factors are specified by the XrBlendFactorFB enumeration.

```
// Provided by XR_FB_composition_layer_alpha_blend
typedef enum XrBlendFactorFB {
    XR_BLEND_FACTOR_ZERO_FB = 0,
    XR_BLEND_FACTOR_ONE_FB = 1,
    XR_BLEND_FACTOR_SRC_ALPHA_FB = 2,
    XR_BLEND_FACTOR_ONE_MINUS_SRC_ALPHA_FB = 3,
    XR_BLEND_FACTOR_DST_ALPHA_FB = 4,
    XR_BLEND_FACTOR_ONE_MINUS_DST_ALPHA_FB = 5,
    XR_BLEND_FACTOR_MAX_ENUM_FB = 0x7FFFFFFF
} XrBlendFactorFB;
```

New Structures

The XrCompositionLayerAlphaBlendFB structure is defined as:

```
// Provided by XR_FB_composition_layer_alpha_blend
typedef struct XrCompositionLayerAlphaBlendFB {
    XrStructureType    type;
    void*              next;
    XrBlendFactorFB    srcFactorColor;
    XrBlendFactorFB    dstFactorColor;
    XrBlendFactorFB    srcFactorAlpha;
    XrBlendFactorFB    dstFactorAlpha;
} XrCompositionLayerAlphaBlendFB;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `srcFactorColor` specifies the source color blend factor.
- `dstFactorColor` specifies the destination color blend factor.
- `srcFactorAlpha` specifies the source alpha blend factor.
- `dstFactorAlpha` specifies the destination alpha blend factor.

XrCompositionLayerAlphaBlendFB provides applications with explicit control over source and destination blend factors.

The XrCompositionLayerAlphaBlendFB structure **must** be provided in the `next` chain of the XrCompositionLayerBaseHeader structure.

Valid Usage (Implicit)

- The `XR_FB_composition_layer_alpha_blend` extension **must** be enabled prior to using XrCompositionLayerAlphaBlendFB
- `type` **must** be `XR_TYPE_COMPOSITION_LAYER_ALPHA_BLEND_FB`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `srcFactorColor` **must** be a valid XrBlendFactorFB value
- `dstFactorColor` **must** be a valid XrBlendFactorFB value
- `srcFactorAlpha` **must** be a valid XrBlendFactorFB value
- `dstFactorAlpha` **must** be a valid XrBlendFactorFB value

New Functions

Issues

- Should we add separate blend controls for color and alpha?
  - Yes. New use cases necessitated adding separate blend controls for color and alpha.

Version History

- Revision 1, 2020-06-22 (Gloria Kennickell)
  - Initial draft
- Revision 2, 2020-06-22 (Gloria Kennickell)
  - Provide separate controls for color and alpha blend factors.
- Revision 3, 2024-03-04 (Xiang Wei)
  - Clarify the superseding of layer blending operation.
  - Add warning about the exclusive usage with non-opaque environment blend modes.

12.99. XR_FB_composition_layer_depth_test

Name String

`XR_FB_composition_layer_depth_test`

Extension Type

Instance extension

Registered Extension Number

213

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Contributors

Guodong Rong, Meta
Cass Everitt, Meta
Jian Zhang, Meta

Overview

This extension enables depth-tested layer composition. The compositor will maintain a depth buffer in

addition to a color buffer. The depth buffer is cleared to a depth corresponding to the infinitely far distance at the beginning of composition.

When composing each layer, if depth testing is requested, the incoming layer depths are transformed into the compositor window space depth and compared to the depth stored in the frame buffer. After the transformation, incoming depths that are outside of the range of the compositor window space depth **must** be clamped. If the depth test fails, the fragment is discarded. If the depth test passes the depth buffer is updated if depth writes are enabled, and color processing continues.

Depth testing requires depth values for the layer. For projection layers, this can be supplied via the XR_KHR_composition_layer_depth extension. For geometric primitive layers, the runtime computes the depth of the sample directly from the layer parameters. An XrCompositionLayerDepthTestFB chained to layers without depth **must** be ignored.

New Object Types

New Flag Types

New Enum Constants

XrStructureType enumeration is extended with:

- XR_TYPE_COMPOSITION_LAYER_DEPTH_TEST_FB

New Enums

The possible comparison operations are specified by the XrCompareOpFB enumeration.

```
// Provided by XR_FB_composition_layer_depth_test
typedef enum XrCompareOpFB {
    XR_COMPARE_OP_NEVER_FB = 0,
    XR_COMPARE_OP_LESS_FB = 1,
    XR_COMPARE_OP_EQUAL_FB = 2,
    XR_COMPARE_OP_LESS_OR_EQUAL_FB = 3,
    XR_COMPARE_OP_GREATER_FB = 4,
    XR_COMPARE_OP_NOT_EQUAL_FB = 5,
    XR_COMPARE_OP_GREATER_OR_EQUAL_FB = 6,
    XR_COMPARE_OP_ALWAYS_FB = 7,
    XR_COMPARE_OP_MAX_ENUM_FB = 0x7FFFFFFF
} XrCompareOpFB;
```

Enumerant Descriptions

- XR_COMPARE_OP_NEVER_FB — Comparison is never true.
- XR_COMPARE_OP_LESS_FB — Comparison is true if source less than is destination.
- XR_COMPARE_OP_EQUAL_FB — Comparison is true if source is equal to destination.
- XR_COMPARE_OP_LESS_OR_EQUAL_FB — Comparison is true if source is less than or equal to destination.
- XR_COMPARE_OP_GREATER_FB — Comparison is true if source is greater than destination.
- XR_COMPARE_OP_NOT_EQUAL_FB — Comparison is true if source is not equal to destination.
- XR_COMPARE_OP_GREATER_OR_EQUAL_FB — Comparison is true if source is greater than or equal to destination.
- XR_COMPARE_OP_ALWAYS_FB — Comparison is always true.

New Structures

The XrCompositionLayerDepthTestFB structure is defined as:

```
// Provided by XR_FB_composition_layer_depth_test
typedef struct XrCompositionLayerDepthTestFB {
    XrStructureType    type;
    const void*        next;
    XrBool32           depthMask;
    XrCompareOpFB      compareOp;
} XrCompositionLayerDepthTestFB;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain.
- depthMask is a boolean indicating whether writes to the composition depth buffer are enabled.
- compareOp is an enum that indicates which compare operation is used in the depth test.

To specify that a layer should be depth tested, a XrCompositionLayerDepthTestFB structure **must** be passed via the polymorphic XrCompositionLayerBaseHeader structure's next parameter chain.

Valid Usage (Implicit)

- The `XR_FB_composition_layer_depth_test` extension **must** be enabled prior to using XrCompositionLayerDepthTestFB
- `type` **must** be `XR_TYPE_COMPOSITION_LAYER_DEPTH_TEST_FB`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `compareOp` **must** be a valid XrCompareOpFB value

New Functions

Issues

Version History

- Revision 1, 2022-02-17 (Cass Everitt)
  - Initial draft

12.100. XR_FB_composition_layer_image_layout

Name String

`XR_FB_composition_layer_image_layout`

Extension Type

Instance extension

Registered Extension Number

41

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Contributors

Cass Everitt, Facebook
Gloria Kennickell, Facebook

Overview

This extension does not define a new composition layer type, but rather it defines parameters that change the interpretation of the image layout, where the default image layout is dictated by the Graphics API.

New Object Types

New Flag Types

```
// Provided by XR_FB_composition_layer_image_layout
typedef XrFlags64 XrCompositionLayerImageLayoutFlagsFB;
```

```
// Provided by XR_FB_composition_layer_image_layout
// Flag bits for XrCompositionLayerImageLayoutFlagsFB
static const XrCompositionLayerImageLayoutFlagsFB
XR_COMPOSITION_LAYER_IMAGE_LAYOUT_VERTICAL_FLIP_BIT_FB = 0x00000001;
```

Flag Descriptions

- XR_COMPOSITION_LAYER_IMAGE_LAYOUT_VERTICAL_FLIP_BIT_FB indicates the coordinate origin must be considered flipped vertically.

New Enum Constants

XrStructureType enumeration is extended with:

- XR_TYPE_COMPOSITION_LAYER_IMAGE_LAYOUT_FB

New Enums

- XR_COMPOSITION_LAYER_IMAGE_LAYOUT_VERTICAL_FLIP_BIT_FB

New Structures

The XrCompositionLayerImageLayoutFB structure is defined as:

```
// Provided by XR_FB_composition_layer_image_layout
typedef struct XrCompositionLayerImageLayoutFB {
    XrStructureType                         type;
    void*                                   next;
    XrCompositionLayerImageLayoutFlagsFB    flags;
} XrCompositionLayerImageLayoutFB;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `flags` is a bitmask of XrCompositionLayerImageLayoutFlagBitsFB.

XrCompositionLayerImageLayoutFB contains additional flags used to change the interpretation of the image layout for a composition layer.

To specify the additional flags, you **must** create a XrCompositionLayerImageLayoutFB structure and pass it via the XrCompositionLayerBaseHeader structure's `next` parameter.

Valid Usage (Implicit)

- The `XR_FB_composition_layer_image_layout` extension **must** be enabled prior to using XrCompositionLayerImageLayoutFB
- `type` **must** be `XR_TYPE_COMPOSITION_LAYER_IMAGE_LAYOUT_FB`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `flags` **must** be `0` or a valid combination of XrCompositionLayerImageLayoutFlagBitsFB values

New Functions

Issues

Version History

- Revision 1, 2020-07-06 (Gloria Kennickell)
  - Initial draft

12.101. XR_FB_composition_layer_secure_content

Name String

`XR_FB_composition_layer_secure_content`

Extension Type

Instance extension

Registered Extension Number

73

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Contributors

Cass Everitt, Facebook
Gloria Kennickell, Facebook

Overview

This extension does not define a new composition layer type, but rather it provides support for the application to specify an existing composition layer type has secure content and whether it must be completely excluded from external outputs, like video or screen capture, or if proxy content must be rendered in its place.

New Object Types

New Flag Types

```
// Provided by XR_FB_composition_layer_alpha_blend
typedef XrFlags64 XrCompositionLayerSecureContentFlagsFB;
```

```
// Provided by XR_FB_composition_layer_alpha_blend
// Flag bits for XrCompositionLayerSecureContentFlagsFB
static const XrCompositionLayerSecureContentFlagsFB
XR_COMPOSITION_LAYER_SECURE_CONTENT_EXCLUDE_LAYER_BIT_FB = 0x00000001;
static const XrCompositionLayerSecureContentFlagsFB
XR_COMPOSITION_LAYER_SECURE_CONTENT_REPLACE_LAYER_BIT_FB = 0x00000002;
```

Flag Descriptions

- XR_COMPOSITION_LAYER_SECURE_CONTENT_EXCLUDE_LAYER_BIT_FB — Indicates the layer will only be visible inside the HMD, and not visible to external sources
- XR_COMPOSITION_LAYER_SECURE_CONTENT_REPLACE_LAYER_BIT_FB — Indicates the layer will be displayed inside the HMD, but replaced by proxy content when written to external sources

New Enum Constants

XrStructureType enumeration is extended with:

- XR_TYPE_COMPOSITION_LAYER_SECURE_CONTENT_FB

New Enums

- XR_COMPOSITION_LAYER_SECURE_CONTENT_EXCLUDE_LAYER_BIT_FB
- XR_COMPOSITION_LAYER_SECURE_CONTENT_REPLACE_LAYER_BIT_FB

New Structures

The XrCompositionLayerSecureContentFB structure is defined as:

```
// Provided by XR_FB_composition_layer_secure_content
typedef struct XrCompositionLayerSecureContentFB {
    XrStructureType                           type;
    const void*                               next;
    XrCompositionLayerSecureContentFlagsFB    flags;
} XrCompositionLayerSecureContentFB;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- flags is a bitmask of XrCompositionLayerSecureContentFlagBitsFB.

XrCompositionLayerSecureContentFB contains additional flags to indicate a composition layer contains secure content and must not be written to external outputs.

If both XR_COMPOSITION_LAYER_SECURE_CONTENT_EXCLUDE_LAYER_BIT_FB and XR_COMPOSITION_LAYER_SECURE_CONTENT_REPLACE_LAYER_BIT_FB are set, XR_COMPOSITION_LAYER_SECURE_CONTENT_EXCLUDE_LAYER_BIT_FB will take precedence.

To specify the additional flags, you **must** create a XrCompositionLayerSecureContentFB structure and pass it via the XrCompositionLayerBaseHeader structure's next parameter.

Valid Usage (Implicit)

- The `XR_FB_composition_layer_secure_content` extension **must** be enabled prior to using XrCompositionLayerSecureContentFB
- `type` **must** be `XR_TYPE_COMPOSITION_LAYER_SECURE_CONTENT_FB`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `flags` **must** be a valid combination of XrCompositionLayerSecureContentFlagBitsFB values
- `flags` **must** not be `0`

New Functions

Issues

Version History

- Revision 1, 2020-06-16 (Gloria Kennickell)
  - Initial draft

12.102. XR_FB_composition_layer_settings

Name String

`XR_FB_composition_layer_settings`

Extension Type

Instance extension

Registered Extension Number

205

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Contributors

Grant Yang, Meta Platforms

Overview

This extension allows applications to request the use of processing options such as sharpening or super-sampling on a composition layer.

New Object Types

New Flag Types

```
// Provided by XR_FB_composition_layer_settings
typedef XrFlags64 XrCompositionLayerSettingsFlagsFB;
```

```
// Provided by XR_FB_composition_layer_settings
// Flag bits for XrCompositionLayerSettingsFlagsFB
static const XrCompositionLayerSettingsFlagsFB
XR_COMPOSITION_LAYER_SETTINGS_NORMAL_SUPER_SAMPLING_BIT_FB = 0x00000001;
static const XrCompositionLayerSettingsFlagsFB
XR_COMPOSITION_LAYER_SETTINGS_QUALITY_SUPER_SAMPLING_BIT_FB = 0x00000002;
static const XrCompositionLayerSettingsFlagsFB
XR_COMPOSITION_LAYER_SETTINGS_NORMAL_SHARPENING_BIT_FB = 0x00000004;
static const XrCompositionLayerSettingsFlagsFB
XR_COMPOSITION_LAYER_SETTINGS_QUALITY_SHARPENING_BIT_FB = 0x00000008;
  // Provided by XR_META_automatic_layer_filter
static const XrCompositionLayerSettingsFlagsFB
XR_COMPOSITION_LAYER_SETTINGS_AUTO_LAYER_FILTER_BIT_META = 0x00000020;
```

Flag Descriptions

- XR_COMPOSITION_LAYER_SETTINGS_NORMAL_SUPER_SAMPLING_BIT_FB — Indicates compositor **may** use layer texture supersampling.
- XR_COMPOSITION_LAYER_SETTINGS_QUALITY_SUPER_SAMPLING_BIT_FB — Indicates compositor **may** use high quality layer texture supersampling.
- XR_COMPOSITION_LAYER_SETTINGS_NORMAL_SHARPENING_BIT_FB — Indicates compositor **may** use layer texture sharpening.
- XR_COMPOSITION_LAYER_SETTINGS_QUALITY_SHARPENING_BIT_FB — Indicates compositor **may** use high quality layer texture sharpening.
- XR_COMPOSITION_LAYER_SETTINGS_AUTO_LAYER_FILTER_BIT_META — Indicates compositor **may** automatically toggle a texture filtering mechanism to improve visual quality of layer. This **must** not be the only bit set. (Added by XR_META_automatic_layer_filter) (Added by the XR_META_automatic_layer_filter extension)

New Enum Constants

XrStructureType enumeration is extended with:

- XR_TYPE_COMPOSITION_LAYER_SETTINGS_FB

New Enums

- XR_COMPOSITION_LAYER_SETTINGS_NORMAL_SUPER_SAMPLING_BIT_FB
- XR_COMPOSITION_LAYER_SETTINGS_QUALITY_SUPER_SAMPLING_BIT_FB
- XR_COMPOSITION_LAYER_SETTINGS_NORMAL_SHARPENING_BIT_FB
- XR_COMPOSITION_LAYER_SETTINGS_QUALITY_SHARPENING_BIT_FB

New Structures

The XrCompositionLayerSettingsFB structure is defined as:

```
// Provided by XR_FB_composition_layer_settings
typedef struct XrCompositionLayerSettingsFB {
    XrStructureType                       type;
    const void*                           next;
    XrCompositionLayerSettingsFlagsFB     layerFlags;
} XrCompositionLayerSettingsFB;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- layerFlags is a bitmask of XrCompositionLayerSettingsFlagBitsFB.

XrCompositionLayerSettingsFB contains additional flags to indicate which processing steps to perform on a composition layer.

If both XR_COMPOSITION_LAYER_SETTINGS_NORMAL_SUPER_SAMPLING_BIT_FB and XR_COMPOSITION_LAYER_SETTINGS_QUALITY_SUPER_SAMPLING_BIT_FB are set, XR_COMPOSITION_LAYER_SETTINGS_NORMAL_SUPER_SAMPLING_BIT_FB will take precedence.

If both XR_COMPOSITION_LAYER_SETTINGS_NORMAL_SHARPENING_BIT_FB and XR_COMPOSITION_LAYER_SETTINGS_QUALITY_SHARPENING_BIT_FB are set, XR_COMPOSITION_LAYER_SETTINGS_NORMAL_SHARPENING_BIT_FB will take precedence.

To specify the additional flags, create an XrCompositionLayerSettingsFB structure and pass it via the XrCompositionLayerBaseHeader structure's next parameter.

Valid Usage (Implicit)

- The `XR_FB_composition_layer_settings` extension **must** be enabled prior to using XrCompositionLayerSettingsFB
- `type` **must** be `XR_TYPE_COMPOSITION_LAYER_SETTINGS_FB`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `layerFlags` **must** be a valid combination of XrCompositionLayerSettingsFlagBitsFB values
- `layerFlags` **must** not be `0`

New Functions

Issues

Version History

- Revision 1, 2022-03-08 (Grant Yang)
  - Initial draft

12.103. XR_FB_display_refresh_rate

Name String

`XR_FB_display_refresh_rate`

Extension Type

Instance extension

Registered Extension Number

102

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

IP Status

No known IP claims.

Contributors

Cass Everitt, Facebook
Gloria Kennickell, Facebook

Overview

On platforms which support dynamically adjusting the display refresh rate, application developers may request a specific display refresh rate in order to improve the overall user experience, examples include:

- A video application may choose a display refresh rate which better matches the video content playback rate in order to achieve smoother video frames.
- An application which can support a higher frame rate may choose to render at the higher rate to improve the overall perceptual quality, for example, lower latency and less flicker.

This extension allows:

- An application to identify what display refresh rates the session supports and the current display refresh rate.
- An application to request a display refresh rate to indicate its preference to the runtime.
- An application to receive notification of changes to the display refresh rate which are delivered via events.

New Object Types

New Flag Types

New Enum Constants

XrStructureType enumeration is extended with:

- `XR_TYPE_EVENT_DATA_DISPLAY_REFRESH_RATE_CHANGED_FB`

XrResult enumeration is extended with:

- `XR_ERROR_DISPLAY_REFRESH_RATE_UNSUPPORTED_FB`

New Enums

New Structures

Receiving the XrEventDataDisplayRefreshRateChangedFB event structure indicates that the display refresh rate has changed.

The XrEventDataDisplayRefreshRateChangedFB structure is defined as:

```
// Provided by XR_FB_display_refresh_rate
typedef struct XrEventDataDisplayRefreshRateChangedFB {
    XrStructureType    type;
    const void*        next;
    float              fromDisplayRefreshRate;
    float              toDisplayRefreshRate;
} XrEventDataDisplayRefreshRateChangedFB;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `fromDisplayRefreshRate` is the previous display refresh rate.
- `toDisplayRefreshRate` is the new display refresh rate.

Valid Usage (Implicit)

- The `XR_FB_display_refresh_rate` extension **must** be enabled prior to using XrEventDataDisplayRefreshRateChangedFB
- `type` **must** be `XR_TYPE_EVENT_DATA_DISPLAY_REFRESH_RATE_CHANGED_FB`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

New Functions

The xrEnumerateDisplayRefreshRatesFB function is defined as:

```
// Provided by XR_FB_display_refresh_rate
XrResult xrEnumerateDisplayRefreshRatesFB(
    XrSession                                   session,
    uint32_t                                    displayRefreshRateCapacityInput,
    uint32_t*                                   displayRefreshRateCountOutput,
    float*                                      displayRefreshRates);
```

Parameter Descriptions

- `session` is the session that enumerates the supported display refresh rates.
- `displayRefreshRateCapacityInput` is the capacity of the `displayRefreshRates`, or 0 to retrieve the required capacity.
- `displayRefreshRateCountOutput` is a pointer to the count of `float displayRefreshRates` written, or a pointer to the required capacity in the case that `displayRefreshRateCapacityInput` is insufficient.
- `displayRefreshRates` is a pointer to an array of `float` display refresh rates, but **can** be `NULL` if `displayRefreshRateCapacityInput` is `0`.
- See the Buffer Size Parameters section for a detailed description of retrieving the required `displayRefreshRates` size.

xrEnumerateDisplayRefreshRatesFB enumerates the display refresh rates supported by the current session. Display refresh rates **must** be in order from lowest to highest supported display refresh rates. Runtimes **must** always return identical buffer contents from this enumeration for the lifetime of the session.

Valid Usage (Implicit)

- The `XR_FB_display_refresh_rate` extension **must** be enabled prior to calling xrEnumerateDisplayRefreshRatesFB
- `session` **must** be a valid XrSession handle
- `displayRefreshRateCountOutput` **must** be a pointer to a `uint32_t` value
- If `displayRefreshRateCapacityInput` is not `0`, `displayRefreshRates` **must** be a pointer to an array of `displayRefreshRateCapacityInput` `float` values

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_SIZE_INSUFFICIENT
- XR_ERROR_VALIDATION_FAILURE

The xrGetDisplayRefreshRateFB function is defined as:

```
// Provided by XR_FB_display_refresh_rate
XrResult xrGetDisplayRefreshRateFB(
    XrSession                                   session,
    float*                                      displayRefreshRate);
```

Parameter Descriptions

- session is the XrSession to query.
- displayRefreshRate is a pointer to a float into which the current display refresh rate will be placed.

xrGetDisplayRefreshRateFB retrieves the current display refresh rate.

Valid Usage (Implicit)

- The XR_FB_display_refresh_rate extension **must** be enabled prior to calling xrGetDisplayRefreshRateFB
- session **must** be a valid XrSession handle
- displayRefreshRate **must** be a pointer to a float value

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The xrRequestDisplayRefreshRateFB function is defined as:

```
// Provided by XR_FB_display_refresh_rate
XrResult xrRequestDisplayRefreshRateFB(
    XrSession                                   session,
    float                                       displayRefreshRate);
```

Parameter Descriptions

- session is a valid XrSession handle.
- displayRefreshRate is 0.0f or a supported display refresh rate. Supported display refresh rates are indicated by xrEnumerateDisplayRefreshRatesFB.

xrRequestDisplayRefreshRateFB provides a mechanism for an application to request the system to dynamically change the display refresh rate to the application preferred value. The runtime **must** return XR_ERROR_DISPLAY_REFRESH_RATE_UNSUPPORTED_FB if displayRefreshRate is not either 0.0f or one of the values enumerated by xrEnumerateDisplayRefreshRatesFB. A display refresh rate of 0.0f indicates the application has no preference.

Note that this is only a request and does not guarantee the system will switch to the requested display refresh rate.

Valid Usage (Implicit)

- The `XR_FB_display_refresh_rate` extension **must** be enabled prior to calling xrRequestDisplayRefreshRateFB
- `session` **must** be a valid XrSession handle

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_DISPLAY_REFRESH_RATE_UNSUPPORTED_FB`
- `XR_ERROR_FEATURE_UNSUPPORTED`
- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_VALIDATION_FAILURE`

Issues

Changing the display refresh rate from its system default does not come without trade-offs. Increasing the display refresh rate puts more load on the entire system and can lead to thermal degradation. Conversely, lowering the display refresh rate can provide better thermal sustainability but at the cost of more perceptual issues, like higher latency and flickering.

Version History

- Revision 1, 2020-10-05 (Gloria Kennickell)
  - Initial extension description

12.104. XR_FB_eye_tracking_social

Name String

`XR_FB_eye_tracking_social`

Extension Type

Instance extension

Registered Extension Number

203

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Last Modified Date

2022-07-17

IP Status

No known IP claims.

Contributors

Scott Ramsby, Meta
Dikpal Reddy, Meta
Igor Tceglevskii, Meta

12.104.1. Overview

This extension enables applications to obtain position and orientation of the user's eyes. It enables applications to render eyes in XR experiences.

This extension is intended to drive animation of avatar eyes. So, for that purpose, the runtimes **may** filter the poses in ways that are suitable for avatar eye interaction but detrimental to other use cases. This extension **should** not be used for other eye tracking purposes. For interaction, `XR_EXT_eye_gaze_interaction` **should** be used.

Eye tracking data is sensitive personal information and is closely linked to personal privacy and integrity. It is strongly recommended that applications that store or transfer eye tracking data always ask the user for active and specific acceptance to do so.

If a runtime supports a permission system to control application access to the eye tracker, then the runtime **must** set the `isValid` field to `XR_FALSE` on the supplied XrEyeGazeFB structure until the application has been allowed access to the eye tracker. When the application access has been allowed, the runtime **may** set `isValid` on the supplied XrEyeGazeFB structure to `XR_TRUE`.

12.104.2. Inspect system capability

The XrSystemEyeTrackingPropertiesFB structure is defined as:

```
// Provided by XR_FB_eye_tracking_social
typedef struct XrSystemEyeTrackingPropertiesFB {
    XrStructureType    type;
    void*              next;
    XrBool32           supportsEyeTracking;
} XrSystemEyeTrackingPropertiesFB;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension. For more details about the structure chain, see the structure being extended (XrSystemProperties).
- `supportsEyeTracking` is an `XrBool32`, indicating if the current system is capable of receiving eye tracking input.

An application **can** inspect whether the system is capable of eye tracking input by extending the XrSystemProperties with XrSystemEyeTrackingPropertiesFB structure when calling xrGetSystemProperties.

If a runtime returns `XR_FALSE` for `supportsEyeTracking`, the runtime **must** return `XR_ERROR_FEATURE_UNSUPPORTED` from xrCreateEyeTrackerFB.

Valid Usage (Implicit)

- The `XR_FB_eye_tracking_social` extension **must** be enabled prior to using XrSystemEyeTrackingPropertiesFB
- `type` **must** be `XR_TYPE_SYSTEM_EYE_TRACKING_PROPERTIES_FB`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

12.104.3. Create an eye tracker handle

The XrEyeTrackerFB handle represents the resources for eye tracking.

```
// Provided by XR_FB_eye_tracking_social
XR_DEFINE_HANDLE(XrEyeTrackerFB)
```

This handle is used for getting eye gaze using xrGetEyeGazesFB function.

An eye tracker provides eye gaze directions.

An application creates an XrEyeTrackerFB handle using xrCreateEyeTrackerFB function.

```
// Provided by XR_FB_eye_tracking_social
XrResult xrCreateEyeTrackerFB(
    XrSession                                   session,
    const XrEyeTrackerCreateInfoFB*             createInfo,
    XrEyeTrackerFB*                             eyeTracker);
```

Parameter Descriptions

- session is an XrSession in which the eye tracker will be active.
- createInfo is the XrEyeTrackerCreateInfoFB used to specify the eye tracker.
- eyeTracker is the returned XrEyeTrackerFB handle.

If the system does not support eye tracking, the runtime **must** return XR_ERROR_FEATURE_UNSUPPORTED from xrCreateEyeTrackerFB. In this case, the runtime **must** return XR_FALSE for XrSystemEyeTrackingPropertiesFB::supportsEyeTracking when the function xrGetSystemProperties is called, so that the application **can** avoid creating an eye tracker.

Valid Usage (Implicit)

- The XR_FB_eye_tracking_social extension **must** be enabled prior to calling xrCreateEyeTrackerFB
- session **must** be a valid XrSession handle
- createInfo **must** be a pointer to a valid XrEyeTrackerCreateInfoFB structure
- eyeTracker **must** be a pointer to an XrEyeTrackerFB handle

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FEATURE_UNSUPPORTED
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The XrEyeTrackerCreateInfoFB structure is defined as:

```
// Provided by XR_FB_eye_tracking_social
typedef struct XrEyeTrackerCreateInfoFB {
    XrStructureType    type;
    const void*        next;
} XrEyeTrackerCreateInfoFB;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.

The XrEyeTrackerCreateInfoFB structure describes the information to create an XrEyeTrackerFB handle.

Valid Usage (Implicit)

- The XR_FB_eye_tracking_social extension **must** be enabled prior to using XrEyeTrackerCreateInfoFB
- `type` **must** be `XR_TYPE_EYE_TRACKER_CREATE_INFO_FB`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

12.104.4. Destroy an eye tracker handle

xrDestroyEyeTrackerFB function releases the `eyeTracker` and the underlying resources when the eye tracking experience is over.

```
// Provided by XR_FB_eye_tracking_social
XrResult xrDestroyEyeTrackerFB(
    XrEyeTrackerFB                              eyeTracker);
```

Parameter Descriptions

- `eyeTracker` is an XrEyeTrackerFB previously created by xrCreateEyeTrackerFB.

Valid Usage (Implicit)

- The XR_FB_eye_tracking_social extension **must** be enabled prior to calling xrDestroyEyeTrackerFB
- `eyeTracker` **must** be a valid XrEyeTrackerFB handle

Thread Safety

- Access to `eyeTracker`, and any child handles, **must** be externally synchronized

Return Codes

Success

- XR_SUCCESS

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_RUNTIME_FAILURE

12.104.5. Get eye gaze

The xrGetEyeGazesFB function is defined as:

```
// Provided by XR_FB_eye_tracking_social
XrResult xrGetEyeGazesFB(
    XrEyeTrackerFB                              eyeTracker,
    const XrEyeGazesInfoFB*                     gazeInfo,
    XrEyeGazesFB*                               eyeGazes);
```

Parameter Descriptions

- eyeTracker is an XrEyeTrackerFB previously created by xrCreateEyeTrackerFB.
- gazeInfo is the information to get eye gaze.
- eyeGazes is a pointer to XrEyeGazesFB receiving the returned eye poses and confidence.

The xrGetEyeGazesFB function obtains pose for a user's eyes at a specific time and within a specific coordinate system.

Valid Usage (Implicit)

- The XR_FB_eye_tracking_social extension **must** be enabled prior to calling xrGetEyeGazesFB
- eyeTracker **must** be a valid XrEyeTrackerFB handle
- gazeInfo **must** be a pointer to a valid XrEyeGazesInfoFB structure
- eyeGazes **must** be a pointer to an XrEyeGazesFB structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_TIME_INVALID
- XR_ERROR_VALIDATION_FAILURE

The XrEyeGazesInfoFB structure describes the information to get eye gaze directions.

```
// Provided by XR_FB_eye_tracking_social
typedef struct XrEyeGazesInfoFB {
    XrStructureType    type;
    const void*        next;
    XrSpace            baseSpace;
    XrTime             time;
} XrEyeGazesInfoFB;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- baseSpace is an XrSpace within which the returned eye poses will be represented.
- time is an XrTime at which the eye gaze information is requested.

The application **should** request a time equal to the predicted display time for the rendered frame. The system will employ appropriate modeling to provide eye gaze at this time.

Valid Usage (Implicit)

- The XR_FB_eye_tracking_social extension **must** be enabled prior to using XrEyeGazesInfoFB
- type **must** be XR_TYPE_EYE_GAZES_INFO_FB
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- baseSpace **must** be a valid XrSpace handle

XrEyeGazesFB structure returns the state of the eye gaze directions.

```
// Provided by XR_FB_eye_tracking_social
typedef struct XrEyeGazesFB {
    XrStructureType    type;
    void*              next;
    XrEyeGazeFB        gaze[XR_EYE_POSITION_COUNT_FB];
    XrTime             time;
} XrEyeGazesFB;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- gaze is an array of XrEyeGazeFB receiving the returned eye gaze directions.
- time is an XrTime time at which the returned eye gaze is tracked or extrapolated to. Equals the time for which the eye gaze was requested if the interpolation at the time was successful.

Valid Usage (Implicit)

- The XR_FB_eye_tracking_social extension **must** be enabled prior to using XrEyeGazesFB
- type **must** be XR_TYPE_EYE_GAZES_FB
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- Any given element of gaze **must** be a valid XrEyeGazeFB structure

XrEyeGazeFB structure describes the validity, direction, and confidence of a social eye gaze observation.

```
// Provided by XR_FB_eye_tracking_social
typedef struct XrEyeGazeFB {
    XrBool32    isValid;
    XrPosef     gazePose;
    float       gazeConfidence;
} XrEyeGazeFB;
```

Member Descriptions

- `isValid` is an `XrBool32` indicating if the returned gazePose is valid. Callers **should** check the validity of pose prior to use.
- `gazePose` is an `XrPosef` describing the position and orientation of the user's eye. The pose is represented in the coordinate system provided by `XrEyeGazesInfoFB`::`baseSpace`.
- `gazeConfidence` is a `float` value between 0 and 1 that represents the confidence for eye pose. A value of 0 represents no confidence in the pose returned, and a value of 1 means maximum confidence in the returned eye pose.

If the returned `isValid` is true, the runtime **must** return `gazePose` and `gazeConfidence`.

If the returned `isValid` is false, it indicates either the eye tracker did not detect the eye gaze or the application lost input focus.

The eye gaze pose is natively oriented with +Y up, +X to the right, and -Z forward and not gravity-aligned, similar to the `XR_REFERENCE_SPACE_TYPE_VIEW`.

Valid Usage (Implicit)

- The `XR_FB_eye_tracking_social` extension **must** be enabled prior to using `XrEyeGazeFB`

The `XrEyePositionFB` describes which eye in the specific position of the `gaze` is in the `XrEyeGazesFB`.

```
// Provided by XR_FB_eye_tracking_social
typedef enum XrEyePositionFB {
    XR_EYE_POSITION_LEFT_FB = 0,
    XR_EYE_POSITION_RIGHT_FB = 1,
    XR_EYE_POSITION_COUNT_FB = 2,
    XR_EYE_POSITION_MAX_ENUM_FB = 0x7FFFFFFF
} XrEyePositionFB;
```

Enumerant Descriptions

- `XR_EYE_POSITION_LEFT_FB` — Specifies the position of the left eye.
- `XR_EYE_POSITION_RIGHT_FB` — Specifies the position of the right eye.
- `XR_EYE_POSITION_COUNT_FB` — Specifies the number of eyes tracked.

12.104.6. Example code for locating eye gaze

The following example code demonstrates how to locate eye gaze relative to a world space.

```
XrInstance instance; // previously initialized
XrSystemId systemId; // previously initialized
XrSession session;   // previously initialized
XrSpace worldSpace;  // previously initialized, e.g. from
                     // XR_REFERENCE_SPACE_TYPE_LOCAL

XrSystemEyeTrackingPropertiesFB eyeTrackingSystemProperties{
    .type = XR_TYPE_SYSTEM_EYE_TRACKING_PROPERTIES_FB,
};
XrSystemProperties systemProperties{
    .type = XR_TYPE_SYSTEM_PROPERTIES,
    .next = &eyeTrackingSystemProperties,
};
CHK_XR(xrGetSystemProperties(instance, systemId, &systemProperties));
if (!eyeTrackingSystemProperties.supportsEyeTracking) {
    // The system does not support eye tracking.
    return;
}

// Get function pointer for xrCreateEyeTrackerFB.
PFN_xrCreateEyeTrackerFB pfnCreateEyeTrackerFB = nullptr;
CHK_XR(xrGetInstanceProcAddr(instance, "xrCreateEyeTrackerFB",
                             reinterpret_cast<PFN_xrVoidFunction*>(
                             &pfnCreateEyeTrackerFB)));

// Create an eye tracker.
XrEyeTrackerFB eyeTracker{XR_NULL_HANDLE};
{
    XrEyeTrackerCreateInfoFB createInfo{
        .type = XR_TYPE_EYE_TRACKER_CREATE_INFO_FB,
    };
    CHK_XR(pfnCreateEyeTrackerFB(session, &createInfo, &eyeTracker));
}
```

```
// Allocate buffers to receive eyes pose and confidence data before frame
// the loop starts.
XrEyeGazesFB eyeGazes{
    .type = XR_TYPE_EYE_GAZES_FB,
    .next = nullptr,
};

// Get function pointer for xrGetEyeGazesFB.
PFN_xrGetEyeGazesFB pfnGetEyeGazesFB = nullptr;
CHK_XR(xrGetInstanceProcAddr(instance, "xrGetEyeGazesFB",
                             reinterpret_cast<PFN_xrVoidFunction*>(
                             &pfnGetEyeGazesFB)));
while (1) {
    // ...
    // For every frame in frame loop
    // ...
    XrFrameState frameState; // previously returned from xrWaitFrame
    const XrTime time = frameState.predictedDisplayTime;

    XrEyeGazesInfoFB gazesInfo{
        .type = XR_TYPE_EYE_GAZES_INFO_FB,
        .next = nullptr,
        .baseSpace = worldSpace,
        .time = time,
    };

    CHK_XR(pfnGetEyeGazesFB(eyeTracker, &gazesInfo, &eyeGazes));

    if (eyeGazes.gaze[XR_EYE_POSITION_LEFT_FB].isValid) {
        // ....
    }
}
```

New Object Types

- XrEyeTrackerFB

New Flag Types

New Enum Constants

XrObjectType enumeration is extended with:

- XR_OBJECT_TYPE_EYE_TRACKER_FB

XrStructureType enumeration is extended with:

- XR_TYPE_SYSTEM_EYE_TRACKING_PROPERTIES_FB

- `XR_TYPE_EYE_TRACKER_CREATE_INFO_FB`
- `XR_TYPE_EYE_GAZES_INFO_FB`
- `XR_TYPE_EYE_GAZES_FB`

New Enums

- XrEyePositionFB

New Structures

- XrSystemEyeTrackingPropertiesFB
- XrEyeTrackerCreateInfoFB
- XrEyeGazesInfoFB
- XrEyeGazeFB
- XrEyeGazesFB

New Functions

- xrCreateEyeTrackerFB
- xrDestroyEyeTrackerFB
- xrGetEyeGazesFB

Issues

Version History

- Revision 1, 2022-07-17 (Igor Tceglevskii)
  - Initial extension description

12.105. XR_FB_face_tracking

Name String

`XR_FB_face_tracking`

Extension Type

Instance extension

Registered Extension Number

202

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Last Modified Date

2022-07-15

IP Status

No known IP claims.

Contributors

Jaebong Lee, Meta
Dikpal Reddy, Meta
Igor Tceglevskii, Meta

12.105.1. Overview

This extension enables applications to get weights of blend shapes. It also enables applications to render facial expressions in XR experiences.

Face tracking data is sensitive personal information and is closely linked to personal privacy and integrity. It is strongly recommended that applications storing or transferring face tracking data always ask the user for active and specific acceptance to do so.

If a runtime supports a permission system to control application access to the face tracker, then the runtime **must** set the `isValid` field to `XR_FALSE` on the supplied XrFaceExpressionStatusFB structure until the user allows the application to access the face tracker. When the application access has been allowed, the runtime **may** set `isValid` on the supplied XrFaceExpressionStatusFB structure to `XR_TRUE`.

Some permission systems **may** control access to the eye tracking separately from access to the face tracking, even though the eyes are part of the face. In case the user denied tracking of the eyes, yet, allowed tracking of the face, then the runtime **must** set the `isEyeFollowingBlendshapesValid` field to `XR_FALSE` on the supplied XrFaceExpressionStatusFB for indicating that eye tracking data is not available, but at the same time **may** set the `isValid` field to `XR_TRUE` on the supplied XrFaceExpressionStatusFB for indicating that another part of the face is tracked properly.

12.105.2. Inspect system capability

```
// Provided by XR_FB_face_tracking
typedef struct XrSystemFaceTrackingPropertiesFB {
    XrStructureType    type;
    void*              next;
    XrBool32           supportsFaceTracking;
} XrSystemFaceTrackingPropertiesFB;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension. For more details about the structure chain, see the structure being extended (XrSystemProperties).
- `supportsFaceTracking` is an `XrBool32`, indicating if current system is capable of receiving face tracking input.

An application **can** inspect whether the system is capable of receiving face tracking input by extending the XrSystemProperties with XrSystemFaceTrackingPropertiesFB structure when calling xrGetSystemProperties.

If a runtime returns `XR_FALSE` for `supportsFaceTracking`, the runtime **must** return `XR_ERROR_FEATURE_UNSUPPORTED` from xrCreateFaceTrackerFB.

Valid Usage (Implicit)

- The `XR_FB_face_tracking` extension **must** be enabled prior to using XrSystemFaceTrackingPropertiesFB
- `type` **must** be `XR_TYPE_SYSTEM_FACE_TRACKING_PROPERTIES_FB`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

12.105.3. Create a face tracker handle

The XrFaceTrackerFB handle represents the resources for face tracking.

```
// Provided by XR_FB_face_tracking
XR_DEFINE_HANDLE(XrFaceTrackerFB)
```

This handle is used to obtain blend shapes using the xrGetFaceExpressionWeightsFB function.

The xrCreateFaceTrackerFB function is defined as:

```
// Provided by XR_FB_face_tracking
XrResult xrCreateFaceTrackerFB(
    XrSession                                   session,
    const XrFaceTrackerCreateInfoFB*            createInfo,
    XrFaceTrackerFB*                            faceTracker);
```

Parameter Descriptions

- `session` is an XrSession in which the face tracker will be active.
- `createInfo` is the XrFaceTrackerCreateInfoFB used to specify the face tracker.
- `faceTracker` is the returned XrFaceTrackerFB handle.

An application **can** create an XrFaceTrackerFB handle using xrCreateFaceTrackerFB function.

If the system does not support face tracking, the runtime **must** return `XR_ERROR_FEATURE_UNSUPPORTED` from xrCreateFaceTrackerFB. In this case, the runtime **must** return `XR_FALSE` for XrSystemFaceTrackingPropertiesFB::`supportsFaceTracking` when the function xrGetSystemProperties is called, so that the application **can** avoid creating a face tracker.

Valid Usage (Implicit)

- The `XR_FB_face_tracking` extension **must** be enabled prior to calling xrCreateFaceTrackerFB
- `session` **must** be a valid XrSession handle
- `createInfo` **must** be a pointer to a valid XrFaceTrackerCreateInfoFB structure
- `faceTracker` **must** be a pointer to an XrFaceTrackerFB handle

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FEATURE_UNSUPPORTED
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The XrFaceTrackerCreateInfoFB structure is described as follows:

```
// Provided by XR_FB_face_tracking
typedef struct XrFaceTrackerCreateInfoFB {
    XrStructureType          type;
    const void*              next;
    XrFaceExpressionSetFB    faceExpressionSet;
} XrFaceTrackerCreateInfoFB;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- faceExpressionSet is an XrFaceExpressionSetFB that describe the set of blend shapes to retrieve.

The XrFaceTrackerCreateInfoFB structure describes the information to create an XrFaceTrackerFB handle.

Valid Usage (Implicit)

- The XR_FB_face_tracking extension **must** be enabled prior to using XrFaceTrackerCreateInfoFB
- type **must** be XR_TYPE_FACE_TRACKER_CREATE_INFO_FB
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- faceExpressionSet **must** be a valid XrFaceExpressionSetFB value

The XrFaceExpressionSetFB enum describes the set of blend shapes of a facial expression to track when creating an XrFaceTrackerFB.

```
// Provided by XR_FB_face_tracking
typedef enum XrFaceExpressionSetFB {
    XR_FACE_EXPRESSION_SET_DEFAULT_FB = 0,
    XR_FACE_EXPRESSION_SET_MAX_ENUM_FB = 0x7FFFFFFF
} XrFaceExpressionSetFB;
```

Enumerant Descriptions

- XR_FACE_EXPRESSION_SET_DEFAULT_FB — indicates that the created XrFaceTrackerFB tracks the set of blend shapes described by XrFaceExpressionFB enum, i.e. the xrGetFaceExpressionWeightsFB function returns an array of blend shapes with the count of XR_FACE_EXPRESSION_COUNT_FB and **can** be indexed using XrFaceExpressionFB.

```
// Provided by XR_FB_face_tracking
#define XR_FACE_EXPRESSSION_SET_DEFAULT_FB XR_FACE_EXPRESSION_SET_DEFAULT_FB
```

The XR_FACE_EXPRESSSION_SET_DEFAULT_FB is an alias for XR_FACE_EXPRESSION_SET_DEFAULT_FB for backward compatibility, deprecated and **should** not be used.

12.105.4. Delete a face tracker handle

The xrDestroyFaceTrackerFB function releases the faceTracker and the underlying resources when face tracking experience is over.

```
// Provided by XR_FB_face_tracking
XrResult xrDestroyFaceTrackerFB(
    XrFaceTrackerFB                             faceTracker);
```

Parameter Descriptions

- `faceTracker` is an XrFaceTrackerFB previously created by xrCreateFaceTrackerFB.

Valid Usage (Implicit)

- The `XR_FB_face_tracking` extension **must** be enabled prior to calling xrDestroyFaceTrackerFB
- `faceTracker` **must** be a valid XrFaceTrackerFB handle

Thread Safety

- Access to `faceTracker`, and any child handles, **must** be externally synchronized

Return Codes

Success

- `XR_SUCCESS`

Failure

- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_RUNTIME_FAILURE`

12.105.5. Obtain facial expressions

The xrGetFaceExpressionWeightsFB function return blend shapes of facial expression at a given time.

```
// Provided by XR_FB_face_tracking
XrResult xrGetFaceExpressionWeightsFB(
    XrFaceTrackerFB                             faceTracker,
    const XrFaceExpressionInfoFB*               expressionInfo,
    XrFaceExpressionWeightsFB*                  expressionWeights);
```

Parameter Descriptions

- `faceTracker` is an XrFaceTrackerFB previously created by xrCreateFaceTrackerFB.
- `expressionInfo` is a pointer to XrFaceExpressionInfoFB describing information to obtain face expression.
- `expressionWeights` is a pointer to XrFaceExpressionWeightsFB receiving the returned facial expression weights.

Valid Usage (Implicit)

- The `XR_FB_face_tracking` extension **must** be enabled prior to calling xrGetFaceExpressionWeightsFB
- `faceTracker` **must** be a valid XrFaceTrackerFB handle
- `expressionInfo` **must** be a pointer to a valid XrFaceExpressionInfoFB structure
- `expressionWeights` **must** be a pointer to an XrFaceExpressionWeightsFB structure

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_TIME_INVALID`
- `XR_ERROR_VALIDATION_FAILURE`

The XrFaceExpressionInfoFB structure describes the information to obtain facial expression.

```
// Provided by XR_FB_face_tracking
typedef struct XrFaceExpressionInfoFB {
    XrStructureType    type;
    const void*        next;
    XrTime             time;
} XrFaceExpressionInfoFB;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `time` is an `XrTime` at which the facial expression weights are requested.

Callers **should** request a time equal to the predicted display time for the rendered frame. The system will employ appropriate modeling to provide expressions for this time.

Valid Usage (Implicit)

- The `XR_FB_face_tracking` extension **must** be enabled prior to using XrFaceExpressionInfoFB
- `type` **must** be `XR_TYPE_FACE_EXPRESSION_INFO_FB`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

XrFaceExpressionWeightsFB structure returns the facial expression.

```
// Provided by XR_FB_face_tracking
typedef struct XrFaceExpressionWeightsFB {
    XrStructureType             type;
    void*                       next;
    uint32_t                    weightCount;
    float*                      weights;
    uint32_t                    confidenceCount;
    float*                      confidences;
    XrFaceExpressionStatusFB    status;
    XrTime                      time;
} XrFaceExpressionWeightsFB;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `weightCount` is a `uint32_t` describing the count of elements in `weights` array.
- `weights` is a pointer to an application-allocated array of `float` that will be filled with weights of facial expression blend shapes.
- `confidenceCount` is a `uint32_t` describing the count of elements in `confidences` array.
- `confidences` is a pointer to an application-allocated array of `float` that will be filled with confidence of tracking specific parts of a face.
- `status` is the XrFaceExpressionStatusFB of validity status of the expression weights.
- `time` is an `XrTime` time at which the returned expression weights are tracked or extrapolated to. Equals the time at which the expression weights were requested if the extrapolating at the time was successful.

The runtime **must** return `XR_ERROR_VALIDATION_FAILURE` if `weightCount` is not equal to the number of blend shapes defined by the XrFaceExpressionSetFB used to create the XrFaceTrackerFB.

The runtime **must** return `XR_ERROR_VALIDATION_FAILURE` if `confidenceCount` is not equal to the number of confidence areas defined by the XrFaceExpressionSetFB used to create the XrFaceTrackerFB.

The runtime **must** return `weights` representing the weights of blend shapes of current facial expression.

The runtime **must** update the `weights` array ordered so that the application **can** index elements using the corresponding facial expression enum (e.g. XrFaceExpressionFB) as described by XrFaceExpressionSetFB when creating the XrFaceTrackerFB. For example, when the XrFaceTrackerFB is created with `XR_FACE_EXPRESSION_SET_DEFAULT_FB`, the application sets the `weightCount` to `XR_FACE_EXPRESSION_COUNT_FB`, and the runtime **must** fill the `weights` array ordered so that it **can** be indexed by the XrFaceExpressionFB enum.

The runtime **must** update the `confidences` array ordered so that the application **can** index elements using the corresponding confidence area enum (e.g. XrFaceConfidenceFB) as described by XrFaceExpressionSetFB when creating the XrFaceTrackerFB. For example, when the XrFaceTrackerFB is created with `XR_FACE_EXPRESSION_SET_DEFAULT_FB`, the application sets the `confidenceCount` to `XR_FACE_CONFIDENCE_COUNT_FB`, and the runtime **must** fill the `confidences` array ordered so that it **can** be indexed by the XrFaceConfidenceFB enum.

Valid Usage (Implicit)

- The XR_FB_face_tracking extension **must** be enabled prior to using XrFaceExpressionWeightsFB
- type **must** be XR_TYPE_FACE_EXPRESSION_WEIGHTS_FB
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- weights **must** be a pointer to an array of weightCount float values
- confidences **must** be a pointer to an array of confidenceCount float values
- status **must** be a valid XrFaceExpressionStatusFB structure
- The weightCount parameter **must** be greater than 0
- The confidenceCount parameter **must** be greater than 0

XrFaceExpressionStatusFB structure describes the validity of facial expression weights.

```
// Provided by XR_FB_face_tracking
typedef struct XrFaceExpressionStatusFB {
    XrBool32    isValid;
    XrBool32    isEyeFollowingBlendshapesValid;
} XrFaceExpressionStatusFB;
```

Member Descriptions

- isValid is an XrBool32 which indicates that the tracked expression weights are valid.
- isEyeFollowingBlendshapesValid is an XrBool32 which indicates if the 8 expression weights with prefix XR_FACE_EXPRESSION_EYES_LOOK_* are valid.

If the returned isValid is XR_FALSE, then it indicates that the face tracker failed to track or lost track of the face, or the application lost focus, or the consent for face tracking was denied.

If the returned isValid is XR_TRUE, the runtime **must** return all weights (or all weights except eyes related weights, see isEyeFollowingBlendshapesValid).

If the returned isEyeFollowingBlendshapesValid is XR_FALSE, then it indicates that the eye tracking driving blendshapes with prefix XR_FACE_EXPRESSION_EYES_LOOK_* lost track or the consent for eye tracking was denied.

Valid Usage (Implicit)

- The XR_FB_face_tracking extension **must** be enabled prior to using XrFaceExpressionStatusFB

12.105.6. Example code for obtaining facial expression

The following example code demonstrates how to obtain all weights for facial expression blend shapes.

```
XrInstance instance; // previously initialized
XrSystemId systemId; // previously initialized
XrSession session;   // previously initialized

// Confirm face tracking system support.
XrSystemFaceTrackingPropertiesFB faceTrackingSystemProperties{
    .type = XR_TYPE_SYSTEM_FACE_TRACKING_PROPERTIES_FB,
};
XrSystemProperties systemProperties{
    .type = XR_TYPE_SYSTEM_PROPERTIES,
    .next = &faceTrackingSystemProperties,
};
CHK_XR(xrGetSystemProperties(instance, systemId, &systemProperties));
if (!faceTrackingSystemProperties.supportsFaceTracking) {
    // The system does not support face tracking
    return;
}

// Get function pointer for xrCreateFaceTrackerFB.
PFN_xrCreateFaceTrackerFB pfnCreateFaceTrackerFB = nullptr;
CHK_XR(xrGetInstanceProcAddr(instance, "xrCreateFaceTrackerFB",
                             reinterpret_cast<PFN_xrVoidFunction*>(
                             &pfnCreateFaceTrackerFB)));

// Create a face tracker for default set of facial expressions.
XrFaceTrackerFB faceTracker{XR_NULL_HANDLE};
{
    XrFaceTrackerCreateInfoFB createInfo{
        .type = XR_TYPE_FACE_TRACKER_CREATE_INFO_FB,
        .next = nullptr,
        .faceExpressionSet = XR_FACE_EXPRESSION_SET_DEFAULT_FB,
    };
    CHK_XR(pfnCreateFaceTrackerFB(session, &createInfo, &faceTracker));
}

// Allocate buffers to receive facial expression data before frame
// loop starts.
```

```
float weights[XR_FACE_EXPRESSION_COUNT_FB];
float confidences[XR_FACE_CONFIDENCE_COUNT_FB];

XrFaceExpressionWeightsFB expressionWeights{
    .type = XR_TYPE_FACE_EXPRESSION_WEIGHTS_FB,
    .next = nullptr,
    .weightCount = XR_FACE_EXPRESSION_COUNT_FB,
    .weights = weights,
    .confidenceCount = XR_FACE_CONFIDENCE_COUNT_FB,
    .confidences = confidences,
};

// Get function pointer for xrGetFaceExpressionWeightsFB.
PFN_xrGetFaceExpressionWeightsFB pfnGetFaceExpressionWeights = nullptr;
CHK_XR(xrGetInstanceProcAddr(instance, "xrGetFaceExpressionWeightsFB",
                             reinterpret_cast<PFN_xrVoidFunction*>(
                             &pfnGetFaceExpressionWeights)));
while (1) {
    // ...
    // For every frame in the frame loop
    // ...
    XrFrameState frameState; // previously returned from xrWaitFrame
    const XrTime time = frameState.predictedDisplayTime;

    XrFaceExpressionInfoFB expressionInfo{
        .type = XR_TYPE_FACE_EXPRESSION_INFO_FB,
        .next = nullptr,
        .time = time,
    };

    CHK_XR(pfnGetFaceExpressionWeights(faceTracker, &expressionInfo, &
expressionWeights));

    if (expressionWeights.status.isValid) {
        for (uint32_t i = 0; i < XR_FACE_EXPRESSION_COUNT_FB; ++i) {
            // weights[i] contains a weight of specific blend shape
        }
    }
}
```

12.105.7. Conventions of blend shapes

This extension defines 63 blend shapes for tracking facial expressions.

```
// Provided by XR_FB_face_tracking
typedef enum XrFaceExpressionFB {
```

```
    XR_FACE_EXPRESSION_BROW_LOWERER_L_FB = 0,
    XR_FACE_EXPRESSION_BROW_LOWERER_R_FB = 1,
    XR_FACE_EXPRESSION_CHEEK_PUFF_L_FB = 2,
    XR_FACE_EXPRESSION_CHEEK_PUFF_R_FB = 3,
    XR_FACE_EXPRESSION_CHEEK_RAISER_L_FB = 4,
    XR_FACE_EXPRESSION_CHEEK_RAISER_R_FB = 5,
    XR_FACE_EXPRESSION_CHEEK_SUCK_L_FB = 6,
    XR_FACE_EXPRESSION_CHEEK_SUCK_R_FB = 7,
    XR_FACE_EXPRESSION_CHIN_RAISER_B_FB = 8,
    XR_FACE_EXPRESSION_CHIN_RAISER_T_FB = 9,
    XR_FACE_EXPRESSION_DIMPLER_L_FB = 10,
    XR_FACE_EXPRESSION_DIMPLER_R_FB = 11,
    XR_FACE_EXPRESSION_EYES_CLOSED_L_FB = 12,
    XR_FACE_EXPRESSION_EYES_CLOSED_R_FB = 13,
    XR_FACE_EXPRESSION_EYES_LOOK_DOWN_L_FB = 14,
    XR_FACE_EXPRESSION_EYES_LOOK_DOWN_R_FB = 15,
    XR_FACE_EXPRESSION_EYES_LOOK_LEFT_L_FB = 16,
    XR_FACE_EXPRESSION_EYES_LOOK_LEFT_R_FB = 17,
    XR_FACE_EXPRESSION_EYES_LOOK_RIGHT_L_FB = 18,
    XR_FACE_EXPRESSION_EYES_LOOK_RIGHT_R_FB = 19,
    XR_FACE_EXPRESSION_EYES_LOOK_UP_L_FB = 20,
    XR_FACE_EXPRESSION_EYES_LOOK_UP_R_FB = 21,
    XR_FACE_EXPRESSION_INNER_BROW_RAISER_L_FB = 22,
    XR_FACE_EXPRESSION_INNER_BROW_RAISER_R_FB = 23,
    XR_FACE_EXPRESSION_JAW_DROP_FB = 24,
    XR_FACE_EXPRESSION_JAW_SIDEWAYS_LEFT_FB = 25,
    XR_FACE_EXPRESSION_JAW_SIDEWAYS_RIGHT_FB = 26,
    XR_FACE_EXPRESSION_JAW_THRUST_FB = 27,
    XR_FACE_EXPRESSION_LID_TIGHTENER_L_FB = 28,
    XR_FACE_EXPRESSION_LID_TIGHTENER_R_FB = 29,
    XR_FACE_EXPRESSION_LIP_CORNER_DEPRESSOR_L_FB = 30,
    XR_FACE_EXPRESSION_LIP_CORNER_DEPRESSOR_R_FB = 31,
    XR_FACE_EXPRESSION_LIP_CORNER_PULLER_L_FB = 32,
    XR_FACE_EXPRESSION_LIP_CORNER_PULLER_R_FB = 33,
    XR_FACE_EXPRESSION_LIP_FUNNELER_LB_FB = 34,
    XR_FACE_EXPRESSION_LIP_FUNNELER_LT_FB = 35,
    XR_FACE_EXPRESSION_LIP_FUNNELER_RB_FB = 36,
    XR_FACE_EXPRESSION_LIP_FUNNELER_RT_FB = 37,
    XR_FACE_EXPRESSION_LIP_PRESSOR_L_FB = 38,
    XR_FACE_EXPRESSION_LIP_PRESSOR_R_FB = 39,
    XR_FACE_EXPRESSION_LIP_PUCKER_L_FB = 40,
    XR_FACE_EXPRESSION_LIP_PUCKER_R_FB = 41,
    XR_FACE_EXPRESSION_LIP_STRETCHER_L_FB = 42,
    XR_FACE_EXPRESSION_LIP_STRETCHER_R_FB = 43,
    XR_FACE_EXPRESSION_LIP_SUCK_LB_FB = 44,
    XR_FACE_EXPRESSION_LIP_SUCK_LT_FB = 45,
    XR_FACE_EXPRESSION_LIP_SUCK_RB_FB = 46,
    XR_FACE_EXPRESSION_LIP_SUCK_RT_FB = 47,
```

```
    XR_FACE_EXPRESSION_LIP_TIGHTENER_L_FB = 48,
    XR_FACE_EXPRESSION_LIP_TIGHTENER_R_FB = 49,
    XR_FACE_EXPRESSION_LIPS_TOWARD_FB = 50,
    XR_FACE_EXPRESSION_LOWER_LIP_DEPRESSOR_L_FB = 51,
    XR_FACE_EXPRESSION_LOWER_LIP_DEPRESSOR_R_FB = 52,
    XR_FACE_EXPRESSION_MOUTH_LEFT_FB = 53,
    XR_FACE_EXPRESSION_MOUTH_RIGHT_FB = 54,
    XR_FACE_EXPRESSION_NOSE_WRINKLER_L_FB = 55,
    XR_FACE_EXPRESSION_NOSE_WRINKLER_R_FB = 56,
    XR_FACE_EXPRESSION_OUTER_BROW_RAISER_L_FB = 57,
    XR_FACE_EXPRESSION_OUTER_BROW_RAISER_R_FB = 58,
    XR_FACE_EXPRESSION_UPPER_LID_RAISER_L_FB = 59,
    XR_FACE_EXPRESSION_UPPER_LID_RAISER_R_FB = 60,
    XR_FACE_EXPRESSION_UPPER_LIP_RAISER_L_FB = 61,
    XR_FACE_EXPRESSION_UPPER_LIP_RAISER_R_FB = 62,
    XR_FACE_EXPRESSION_COUNT_FB = 63,
    XR_FACE_EXPRESSION_MAX_ENUM_FB = 0x7FFFFFFF
} XrFaceExpressionFB;
```

12.105.8. Conventions of confidence areas

This extension defines two separate areas of confidence.

```
// Provided by XR_FB_face_tracking
typedef enum XrFaceConfidenceFB {
    XR_FACE_CONFIDENCE_LOWER_FACE_FB = 0,
    XR_FACE_CONFIDENCE_UPPER_FACE_FB = 1,
    XR_FACE_CONFIDENCE_COUNT_FB = 2,
    XR_FACE_CONFIDENCE_MAX_ENUM_FB = 0x7FFFFFFF
} XrFaceConfidenceFB;
```

The "upper face" area represents everything above the upper lip, including eye, eyebrows + cheek, and nose. The "lower face" area represents everything under eyes, including mouth, chin + cheek, and nose. Cheek and nose areas contribute to both "upper face" and "lower face" areas.

New Object Types

- XrFaceTrackerFB

New Flag Types

New Enum Constants

XrObjectType enumeration is extended with:

- XR_OBJECT_TYPE_FACE_TRACKER_FB

XrStructureType enumeration is extended with:

- XR_TYPE_SYSTEM_FACE_TRACKING_PROPERTIES_FB
- XR_TYPE_FACE_TRACKER_CREATE_INFO_FB
- XR_TYPE_FACE_EXPRESSION_INFO_FB
- XR_TYPE_FACE_EXPRESSION_WEIGHTS_FB

New Enums

- XrFaceExpressionFB
- XrFaceExpressionSetFB
- XrFaceConfidenceFB

New Structures

- XrSystemFaceTrackingPropertiesFB
- XrFaceTrackerCreateInfoFB
- XrFaceExpressionInfoFB
- XrFaceExpressionStatusFB
- XrFaceExpressionWeightsFB

New Functions

- xrCreateFaceTrackerFB
- xrDestroyFaceTrackerFB
- xrGetFaceExpressionWeightsFB

Issues

Version History

- Revision 1, 2022-07-15 (Igor Tceglevskii)
  - Initial extension description

12.106. XR_FB_face_tracking2

Name String

XR_FB_face_tracking2

Extension Type

Instance extension

Registered Extension Number

288

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Last Modified Date

2023-10-06

IP Status

No known IP claims.

Contributors

Jaebong Lee, Meta
Dikpal Reddy, Meta
Igor Tceglevskii, Meta
Bill Orr, Meta
Scott Ramsby, Meta

12.106.1. Overview

This extension enables applications to get weights of blend shapes. It also enables applications to render facial expressions in XR experiences.

It is recommended to choose this extension over the `XR_FB_face_tracking` extension, if it is supported by the runtime, because this extension provides the following two additional capabilities to the application:

- This extension provides additional seven blend shapes that estimate tongue movement.
- This extension allows an application and the runtime to communicate about the data sources that are used to estimate facial expression in a cooperative manner.

Face tracking data is sensitive personal information and is closely linked to personal privacy and integrity. Applications storing or transferring face tracking data **should** always ask the user for active and specific acceptance to do so.

If the runtime supports a permission system to control application access to the face tracker, then the runtime **must** set the `isValid` field to `XR_FALSE` on the supplied XrFaceExpressionWeights2FB structure until the user allows the application to access the face tracker. When the application access has been

allowed, the runtime **should** set `isValid` on the supplied XrFaceExpressionWeights2FB structure to `XR_TRUE`.

Some permission systems **may** control access to the eye tracking separately from access to the face tracking, even though the eyes are part of the face. In case the user denied tracking of the eyes, yet, allowed tracking of the face, then the runtime **must** set the `isEyeFollowingBlendshapesValid` field to `XR_FALSE` on the supplied XrFaceExpressionWeights2FB for indicating that eye tracking data is not available, but at the same time **may** set the `isValid` field to `XR_TRUE` on the supplied XrFaceExpressionWeights2FB for indicating that another part of the face is tracked properly.

12.106.2. Inspect system capability

```
// Provided by XR_FB_face_tracking2
typedef struct XrSystemFaceTrackingProperties2FB {
    XrStructureType    type;
    void*              next;
    XrBool32           supportsVisualFaceTracking;
    XrBool32           supportsAudioFaceTracking;
} XrSystemFaceTrackingProperties2FB;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension. For more details about the structure chain, see the structure being extended (XrSystemProperties).
- `supportsVisualFaceTracking` is an `XrBool32`, indicating if the current system is capable of receiving face tracking input that is estimated based on visual data source.
- `supportsAudioFaceTracking` is an `XrBool32`, indicating if the current system is capable of receiving face tracking input that is estimated based on audio data source.

An application **can** inspect whether the system is capable of receiving face tracking input by extending the XrSystemProperties with XrSystemFaceTrackingProperties2FB structure when calling xrGetSystemProperties.

If an application calls xrCreateFaceTracker2FB only with unsupported XrFaceTrackerCreateInfo2FB ::`requestedDataSources`, the runtime **must** return `XR_ERROR_FEATURE_UNSUPPORTED` from xrCreateFaceTracker2FB. For example, if an application calls xrCreateFaceTracker2FB only with `XR_FACE_TRACKING_DATA_SOURCE2_AUDIO_FB` in XrFaceTrackerCreateInfo2FB::`requestedDataSources` when the runtime returns `XR_FALSE` for `supportsAudioFaceTracking`, the runtime **must** return `XR_ERROR_FEATURE_UNSUPPORTED` from xrCreateFaceTracker2FB.

Valid Usage (Implicit)

- The XR_FB_face_tracking2 extension **must** be enabled prior to using XrSystemFaceTrackingProperties2FB
- type **must** be XR_TYPE_SYSTEM_FACE_TRACKING_PROPERTIES2_FB
- next **must** be NULL or a valid pointer to the next structure in a structure chain

12.106.3. Create a face tracker handle

The XrFaceTracker2FB handle represents the resources for face tracking.

```
// Provided by XR_FB_face_tracking2
XR_DEFINE_HANDLE(XrFaceTracker2FB)
```

This handle is used to obtain blend shapes using the xrGetFaceExpressionWeights2FB function.

The xrCreateFaceTracker2FB function is defined as:

```
// Provided by XR_FB_face_tracking2
XrResult xrCreateFaceTracker2FB(
    XrSession                                   session,
    const XrFaceTrackerCreateInfo2FB*           createInfo,
    XrFaceTracker2FB*                           faceTracker);
```

Parameter Descriptions

- session is an XrSession in which the face tracker will be active.
- createInfo is the XrFaceTrackerCreateInfo2FB used to specify the face tracker.
- faceTracker is the returned XrFaceTracker2FB handle.

An application **can** create an XrFaceTracker2FB handle using xrCreateFaceTracker2FB function.

If the system does not support face tracking, the runtime **must** return XR_ERROR_FEATURE_UNSUPPORTED from xrCreateFaceTracker2FB. In this case, the runtime **must** return XR_FALSE for both XrSystemFaceTrackingProperties2FB::supportsVisualFaceTracking and XrSystemFaceTrackingProperties2FB::supportsAudioFaceTracking when the function xrGetSystemProperties is called, so that the application **can** avoid creating a face tracker.

Valid Usage (Implicit)

- The XR_FB_face_tracking2 extension **must** be enabled prior to calling xrCreateFaceTracker2FB
- session **must** be a valid XrSession handle
- createInfo **must** be a pointer to a valid XrFaceTrackerCreateInfo2FB structure
- faceTracker **must** be a pointer to an XrFaceTracker2FB handle

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FEATURE_UNSUPPORTED
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The XrFaceTrackerCreateInfo2FB structure is described as follows:

```
// Provided by XR_FB_face_tracking2
typedef struct XrFaceTrackerCreateInfo2FB {
    XrStructureType                 type;
    const void*                     next;
    XrFaceExpressionSet2FB          faceExpressionSet;
    uint32_t                        requestedDataSourceCount;
    XrFaceTrackingDataSource2FB*    requestedDataSources;
} XrFaceTrackerCreateInfo2FB;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `faceExpressionSet` is an XrFaceExpressionSet2FB that describes the set of blend shapes to retrieve.
- `requestedDataSourceCount` is the number of elements in the `requestedDataSources` array.
- `requestedDataSources` is an array of XrFaceTrackingDataSource2FB that the application accepts. The order of values in the array has no significance.

The XrFaceTrackerCreateInfo2FB structure describes the information to create an XrFaceTracker2FB handle.

Runtimes **may** support a variety of data sources for estimations of facial expression, and some runtimes and devices **may** use data from multiple data sources. The application tells the runtime all data sources that the runtime **may** use to provide facial expressions for the application.

Because the device setting **may** change during a running session, the runtime **may** return a valid XrFaceTracker2FB handle even if the device is unable to estimate facial expression using the data sources requested by the application's call to xrCreateFaceTracker2FB. The runtime **must** instead return `XR_ERROR_FEATURE_UNSUPPORTED` from xrCreateFaceTracker2FB, if for example the runtime believes it will never be able to satisfy the request.

If `requestedDataSourceCount` is `0`, the runtime **may** choose any supported data source, preferably one that is more expressive than the others.

If any value in `requestedDataSources` is duplicated the runtime **must** return `XR_ERROR_VALIDATION_FAILURE` from the call to xrCreateFaceTracker2FB.

Valid Usage (Implicit)

- The `XR_FB_face_tracking2` extension **must** be enabled prior to using XrFaceTrackerCreateInfo2FB
- `type` **must** be `XR_TYPE_FACE_TRACKER_CREATE_INFO2_FB`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `faceExpressionSet` **must** be a valid XrFaceExpressionSet2FB value
- If `requestedDataSourceCount` is not `0`, `requestedDataSources` **must** be a pointer to an array of `requestedDataSourceCount` XrFaceTrackingDataSource2FB values

The XrFaceExpressionSet2FB enum describes the set of blend shapes of a facial expression to track when creating an XrFaceTracker2FB.

```
// Provided by XR_FB_face_tracking2
typedef enum XrFaceExpressionSet2FB {
    XR_FACE_EXPRESSION_SET2_DEFAULT_FB = 0,
    XR_FACE_EXPRESSION_SET_2FB_MAX_ENUM_FB = 0x7FFFFFFF
} XrFaceExpressionSet2FB;
```

Enumerant Descriptions

- XR_FACE_EXPRESSION_SET2_DEFAULT_FB — indicates that the created XrFaceTracker2FB tracks the set of blend shapes described by XrFaceExpression2FB enum, i.e. the xrGetFaceExpressionWeights2FB function returns an array of blend shapes with the count of XR_FACE_EXPRESSION2_COUNT_FB and **can** be indexed using XrFaceExpression2FB.

The XrFaceTrackingDataSource2FB enumeration is defined as:

```
// Provided by XR_FB_face_tracking2
typedef enum XrFaceTrackingDataSource2FB {
    XR_FACE_TRACKING_DATA_SOURCE2_VISUAL_FB = 0,
    XR_FACE_TRACKING_DATA_SOURCE2_AUDIO_FB = 1,
    XR_FACE_TRACKING_DATA_SOURCE_2FB_MAX_ENUM_FB = 0x7FFFFFFF
} XrFaceTrackingDataSource2FB;
```

Enumerant Descriptions

- XR_FACE_TRACKING_DATA_SOURCE2_VISUAL_FB - This value indicates that the face tracking data source supports using visual data to estimate facial expression. The runtime **may** also use audio to further improve the quality of the tracking.
- XR_FACE_TRACKING_DATA_SOURCE2_AUDIO_FB - This value indicates that the face tracking data source supports using audio data to estimate facial expression. The runtime **must** not use visual data for this data source.

12.106.4. Delete a face tracker handle

The xrDestroyFaceTracker2FB function is defined as:

```
// Provided by XR_FB_face_tracking2
XrResult xrDestroyFaceTracker2FB(
    XrFaceTracker2FB                            faceTracker);
```

Parameter Descriptions

- `faceTracker` is an XrFaceTracker2FB previously created by xrCreateFaceTracker2FB.

The xrDestroyFaceTracker2FB function releases the `faceTracker` and the underlying resources when face tracking experience is over.

Valid Usage (Implicit)

- The `XR_FB_face_tracking2` extension **must** be enabled prior to calling xrDestroyFaceTracker2FB
- `faceTracker` **must** be a valid XrFaceTracker2FB handle

Thread Safety

- Access to `faceTracker`, and any child handles, **must** be externally synchronized

Return Codes

Success

- `XR_SUCCESS`

Failure

- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_RUNTIME_FAILURE`

12.106.5. Obtain facial expressions

The xrGetFaceExpressionWeights2FB function is defined as:

```
// Provided by XR_FB_face_tracking2
XrResult xrGetFaceExpressionWeights2FB(
    XrFaceTracker2FB                            faceTracker,
    const XrFaceExpressionInfo2FB*              expressionInfo,
    XrFaceExpressionWeights2FB*                 expressionWeights);
```

Parameter Descriptions

- `faceTracker` is an XrFaceTracker2FB previously created by xrCreateFaceTracker2FB.
- `expressionInfo` is a pointer to XrFaceExpressionInfo2FB describing information to obtain face expression.
- `expressionWeights` is a pointer to XrFaceExpressionWeights2FB receiving the returned facial expression weights.

The xrGetFaceExpressionWeights2FB function return blend shapes of facial expression at a given time.

Valid Usage (Implicit)

- The `XR_FB_face_tracking2` extension **must** be enabled prior to calling xrGetFaceExpressionWeights2FB
- `faceTracker` **must** be a valid XrFaceTracker2FB handle
- `expressionInfo` **must** be a pointer to a valid XrFaceExpressionInfo2FB structure
- `expressionWeights` **must** be a pointer to an XrFaceExpressionWeights2FB structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_TIME_INVALID
- XR_ERROR_VALIDATION_FAILURE

The XrFaceExpressionInfo2FB structure is defined as:

```
// Provided by XR_FB_face_tracking2
typedef struct XrFaceExpressionInfo2FB {
    XrStructureType    type;
    const void*        next;
    XrTime             time;
} XrFaceExpressionInfo2FB;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- time is an XrTime at which the facial expression weights are requested.

The XrFaceExpressionInfo2FB structure describes the information to obtain facial expression. The application **should** pass a time equal to the predicted display time for the rendered frame. The system **must** employ appropriate modeling to provide expressions for this time.

Valid Usage (Implicit)

- The XR_FB_face_tracking2 extension **must** be enabled prior to using XrFaceExpressionInfo2FB
- type **must** be XR_TYPE_FACE_EXPRESSION_INFO2_FB
- next **must** be NULL or a valid pointer to the next structure in a structure chain

The XrFaceExpressionWeights2FB structure is defined as:

```
// Provided by XR_FB_face_tracking2
typedef struct XrFaceExpressionWeights2FB {
    XrStructureType                type;
    void*                          next;
    uint32_t                       weightCount;
    float*                         weights;
    uint32_t                       confidenceCount;
    float*                         confidences;
    XrBool32                       isValid;
    XrBool32                       isEyeFollowingBlendshapesValid;
    XrFaceTrackingDataSource2FB    dataSource;
    XrTime                         time;
} XrFaceExpressionWeights2FB;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `weightCount` is a `uint32_t` describing the count of elements in `weights` array.
- `weights` is a pointer to an application-allocated array of `float` that will be filled with weights of facial expression blend shapes.
- `confidenceCount` is a `uint32_t` describing the count of elements in `confidences` array.
- `confidences` is a pointer to an application-allocated array of `float` that will be filled with confidence of tracking specific parts of a face.
- `isValid` is an `XrBool32` which indicates that the tracked expression weights are valid.
- `isEyeFollowingBlendshapesValid` is an `XrBool32` which indicates if the 8 expression weights with prefix `XR_FACE_EXPRESSION2_EYES_LOOK_*` are valid.
- `dataSource` is an XrFaceTrackingDataSource2FB which indicates the data source that was used to estimate the facial expression.
- `time` is an `XrTime` time at which the returned expression weights are tracked or extrapolated to. Equals the time at which the expression weights were requested if the extrapolating at the time was successful.

XrFaceExpressionWeights2FB structure returns the facial expression.

The runtime **must** return `XR_ERROR_VALIDATION_FAILURE` if `weightCount` is not equal to the number of blend shapes defined by the XrFaceExpressionSet2FB used to create the XrFaceTracker2FB.

The runtime **must** return `XR_ERROR_VALIDATION_FAILURE` if `confidenceCount` is not equal to the number of confidence areas defined by the XrFaceExpressionSet2FB used to create the XrFaceTracker2FB.

A special case applies when XrFaceTrackingVisemesMETA is added to the `next` chain. In this scenario, `weightCount` and `confidenceCount` can be set to 0 to indicate that only visemes are requested. In such cases, it is not necessary to assign pointers to the `weights` and `confidences`.

The runtime **must** return `weights` representing the weights of blend shapes of current facial expression. Each weight is a normalized value in the range [0, 1], where 0 indicates the blend shape is not active and 1 indicates the blend shape is fully active.

The runtime **must** update the `weights` array ordered so that the application **can** index elements using the corresponding facial expression enum (e.g. XrFaceExpression2FB) as described by XrFaceExpressionSet2FB when creating the XrFaceTracker2FB. For example, when the XrFaceTracker2FB is created with `XR_FACE_EXPRESSION_SET2_DEFAULT_FB`, the application sets the `weightCount` to `XR_FACE_EXPRESSION2_COUNT_FB`, and the runtime **must** fill the `weights` array ordered so

that it **can** be indexed by the XrFaceExpression2FB enum.

The runtime **must** update the `confidences` array ordered so that the application **can** index elements using the corresponding confidence area enum (e.g. XrFaceConfidence2FB) as described by XrFaceExpressionSet2FB when creating the XrFaceTracker2FB. For example, when the XrFaceTracker2FB is created with `XR_FACE_EXPRESSION_SET2_DEFAULT_FB`, the application sets the `confidenceCount` to `XR_FACE_CONFIDENCE2_COUNT_FB`, and the runtime **must** fill the `confidences` array ordered so that it **can** be indexed by the XrFaceConfidence2FB enum.

The runtime **must** set `isValid` to `XR_FALSE` and it **must** also set all elements of `weights` to zero, if one of the following is true:

- the face tracker failed to track or lost track of the face
- the application lost focus
- the consent for face tracking was denied
- the runtime is unable to estimate facial expression from the data sources specified when xrCreateFaceTracker2FB function was called

If the returned `isValid` is `XR_TRUE`, the runtime **must** return all weights (or all weights except eyes related weights, see `isEyeFollowingBlendshapesValid`).

The runtime **must** set `isEyeFollowingBlendshapesValid` to `XR_FALSE` and it **must** also set 8 expression weights with prefix `XR_FACE_EXPRESSION2_EYES_LOOK_*` to zero, if one of the following is true:

- the eye tracking driving blendshapes with prefix `XR_FACE_EXPRESSION2_EYES_LOOK_*` lost track
- the consent for eye tracking was denied

Valid Usage (Implicit)

- The `XR_FB_face_tracking2` extension **must** be enabled prior to using XrFaceExpressionWeights2FB
- `type` **must** be `XR_TYPE_FACE_EXPRESSION_WEIGHTS2_FB`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain. See also: XrFaceTrackingVisemesMETA
- `weights` **must** be a pointer to an array of `weightCount` `float` values
- `confidences` **must** be a pointer to an array of `confidenceCount` `float` values
- `dataSource` **must** be a valid XrFaceTrackingDataSource2FB value
- The `weightCount` parameter **must** be greater than `0`
- The `confidenceCount` parameter **must** be greater than `0`

12.106.6. Example code for obtaining facial expression

The following example code demonstrates how to obtain all weights for facial expression blend shapes.

```
XrInstance instance; // previously initialized
XrSystemId systemId; // previously initialized
XrSession session;   // previously initialized

// Confirm face tracking system support.
XrSystemFaceTrackingProperties2FB faceTrackingSystemProperties{
    .type = XR_TYPE_SYSTEM_FACE_TRACKING_PROPERTIES2_FB,
};
XrSystemProperties systemProperties{
    .type = XR_TYPE_SYSTEM_PROPERTIES,
    .next = &faceTrackingSystemProperties,
};
CHK_XR(xrGetSystemProperties(instance, systemId, &systemProperties));
if (!faceTrackingSystemProperties.supportsVisualFaceTracking &&
    !faceTrackingSystemProperties.supportsAudioFaceTracking) {
    // The system does not support face tracking
    return;
}

// Get function pointer for xrCreateFaceTracker2FB.
PFN_xrCreateFaceTracker2FB pfnCreateFaceTracker2FB = nullptr;
CHK_XR(xrGetInstanceProcAddr(instance, "xrCreateFaceTracker2FB",
                             reinterpret_cast<PFN_xrVoidFunction*>(
                             &pfnCreateFaceTracker2FB)));

// Create a face tracker for default set of facial expressions.
XrFaceTracker2FB faceTracker{XR_NULL_HANDLE};
{
    XrFaceTrackingDataSource2FB dataSources[2] = {
        XR_FACE_TRACKING_DATA_SOURCE2_VISUAL_FB,
        XR_FACE_TRACKING_DATA_SOURCE2_AUDIO_FB};

    XrFaceTrackerCreateInfo2FB createInfo{
        .type = XR_TYPE_FACE_TRACKER_CREATE_INFO2_FB,
        .next = nullptr,
        .faceExpressionSet = XR_FACE_EXPRESSION_SET2_DEFAULT_FB,
        // This tells the runtime that the application can take
        // facial expression from any of two data sources.
        .requestedDataSourceCount = 2,
        .requestedDataSources = dataSources,
    };
    CHK_XR(pfnCreateFaceTracker2FB(session, &createInfo, &faceTracker));
}
```

```
// Allocate buffers to receive facial expression data before frame
// loop starts.
float weights[XR_FACE_EXPRESSION2_COUNT_FB];
float confidences[XR_FACE_CONFIDENCE2_COUNT_FB];

XrFaceExpressionWeights2FB expressionWeights{
    .type = XR_TYPE_FACE_EXPRESSION_WEIGHTS2_FB,
    .next = nullptr,
    .weightCount = XR_FACE_EXPRESSION2_COUNT_FB,
    .weights = weights,
    .confidenceCount = XR_FACE_CONFIDENCE2_COUNT_FB,
    .confidences = confidences,
};

// Get function pointer for xrGetFaceExpressionWeights2FB.
PFN_xrGetFaceExpressionWeights2FB pfnGetFaceExpressionWeights = nullptr;
CHK_XR(xrGetInstanceProcAddr(instance, "xrGetFaceExpressionWeights2FB",
                             reinterpret_cast<PFN_xrVoidFunction*>(
                             &pfnGetFaceExpressionWeights)));
while (1) {
    // ...
    // For every frame in the frame loop
    // ...
    XrFrameState frameState; // previously returned from xrWaitFrame
    const XrTime time = frameState.predictedDisplayTime;

    XrFaceExpressionInfo2FB expressionInfo{
        .type = XR_TYPE_FACE_EXPRESSION_INFO2_FB,
        .next = nullptr,
        .time = time,
    };

    CHK_XR(pfnGetFaceExpressionWeights(faceTracker, &expressionInfo, &
expressionWeights));

    if (expressionWeights.isValid) {
        // If you want to do something depending on the data source.
        if (expressionWeights.dataSource == XR_FACE_TRACKING_DATA_SOURCE2_VISUAL_FB) {
            // do something when visual or audiovisual data source was used.
        } else if (expressionWeights.dataSource ==
XR_FACE_TRACKING_DATA_SOURCE2_AUDIO_FB) {
            // do something when audio data source was used.
        }

        for (uint32_t i = 0; i < XR_FACE_EXPRESSION2_COUNT_FB; ++i) {
            // weights[i] contains a weight of specific blend shape
        }
```

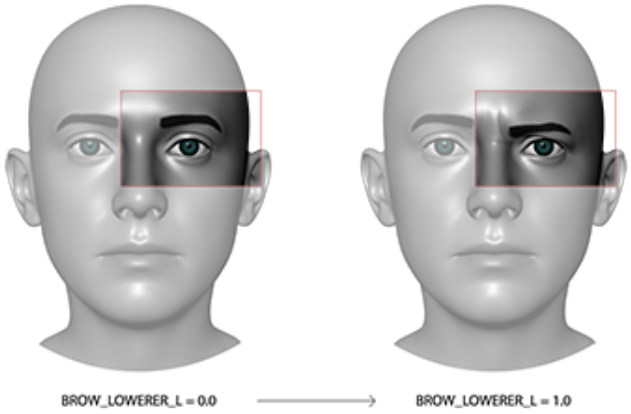
```
    }
}
```

12.106.7. Conventions of blend shapes

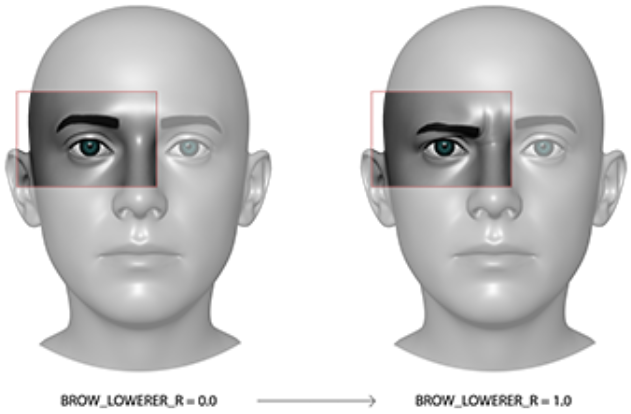
This extension defines 70 blend shapes for tracking facial expressions.

```
// Provided by XR_FB_face_tracking2
typedef enum XrFaceExpression2FB {
    XR_FACE_EXPRESSION2_BROW_LOWERER_L_FB = 0,
    XR_FACE_EXPRESSION2_BROW_LOWERER_R_FB = 1,
    XR_FACE_EXPRESSION2_CHEEK_PUFF_L_FB = 2,
    XR_FACE_EXPRESSION2_CHEEK_PUFF_R_FB = 3,
    XR_FACE_EXPRESSION2_CHEEK_RAISER_L_FB = 4,
    XR_FACE_EXPRESSION2_CHEEK_RAISER_R_FB = 5,
    XR_FACE_EXPRESSION2_CHEEK_SUCK_L_FB = 6,
    XR_FACE_EXPRESSION2_CHEEK_SUCK_R_FB = 7,
    XR_FACE_EXPRESSION2_CHIN_RAISER_B_FB = 8,
    XR_FACE_EXPRESSION2_CHIN_RAISER_T_FB = 9,
    XR_FACE_EXPRESSION2_DIMPLER_L_FB = 10,
    XR_FACE_EXPRESSION2_DIMPLER_R_FB = 11,
    XR_FACE_EXPRESSION2_EYES_CLOSED_L_FB = 12,
    XR_FACE_EXPRESSION2_EYES_CLOSED_R_FB = 13,
    XR_FACE_EXPRESSION2_EYES_LOOK_DOWN_L_FB = 14,
    XR_FACE_EXPRESSION2_EYES_LOOK_DOWN_R_FB = 15,
    XR_FACE_EXPRESSION2_EYES_LOOK_LEFT_L_FB = 16,
    XR_FACE_EXPRESSION2_EYES_LOOK_LEFT_R_FB = 17,
    XR_FACE_EXPRESSION2_EYES_LOOK_RIGHT_L_FB = 18,
    XR_FACE_EXPRESSION2_EYES_LOOK_RIGHT_R_FB = 19,
    XR_FACE_EXPRESSION2_EYES_LOOK_UP_L_FB = 20,
    XR_FACE_EXPRESSION2_EYES_LOOK_UP_R_FB = 21,
    XR_FACE_EXPRESSION2_INNER_BROW_RAISER_L_FB = 22,
    XR_FACE_EXPRESSION2_INNER_BROW_RAISER_R_FB = 23,
    XR_FACE_EXPRESSION2_JAW_DROP_FB = 24,
    XR_FACE_EXPRESSION2_JAW_SIDEWAYS_LEFT_FB = 25,
    XR_FACE_EXPRESSION2_JAW_SIDEWAYS_RIGHT_FB = 26,
    XR_FACE_EXPRESSION2_JAW_THRUST_FB = 27,
    XR_FACE_EXPRESSION2_LID_TIGHTENER_L_FB = 28,
    XR_FACE_EXPRESSION2_LID_TIGHTENER_R_FB = 29,
    XR_FACE_EXPRESSION2_LIP_CORNER_DEPRESSOR_L_FB = 30,
    XR_FACE_EXPRESSION2_LIP_CORNER_DEPRESSOR_R_FB = 31,
    XR_FACE_EXPRESSION2_LIP_CORNER_PULLER_L_FB = 32,
    XR_FACE_EXPRESSION2_LIP_CORNER_PULLER_R_FB = 33,
    XR_FACE_EXPRESSION2_LIP_FUNNELER_LB_FB = 34,
    XR_FACE_EXPRESSION2_LIP_FUNNELER_LT_FB = 35,
    XR_FACE_EXPRESSION2_LIP_FUNNELER_RB_FB = 36,
```

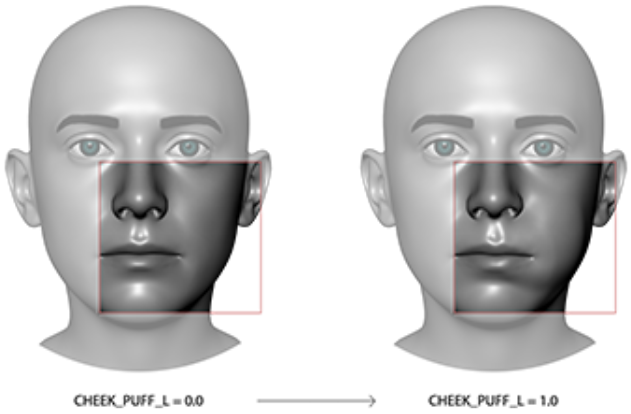
```
    XR_FACE_EXPRESSION2_LIP_FUNNELER_RT_FB = 37,
    XR_FACE_EXPRESSION2_LIP_PRESSOR_L_FB = 38,
    XR_FACE_EXPRESSION2_LIP_PRESSOR_R_FB = 39,
    XR_FACE_EXPRESSION2_LIP_PUCKER_L_FB = 40,
    XR_FACE_EXPRESSION2_LIP_PUCKER_R_FB = 41,
    XR_FACE_EXPRESSION2_LIP_STRETCHER_L_FB = 42,
    XR_FACE_EXPRESSION2_LIP_STRETCHER_R_FB = 43,
    XR_FACE_EXPRESSION2_LIP_SUCK_LB_FB = 44,
    XR_FACE_EXPRESSION2_LIP_SUCK_LT_FB = 45,
    XR_FACE_EXPRESSION2_LIP_SUCK_RB_FB = 46,
    XR_FACE_EXPRESSION2_LIP_SUCK_RT_FB = 47,
    XR_FACE_EXPRESSION2_LIP_TIGHTENER_L_FB = 48,
    XR_FACE_EXPRESSION2_LIP_TIGHTENER_R_FB = 49,
    XR_FACE_EXPRESSION2_LIPS_TOWARD_FB = 50,
    XR_FACE_EXPRESSION2_LOWER_LIP_DEPRESSOR_L_FB = 51,
    XR_FACE_EXPRESSION2_LOWER_LIP_DEPRESSOR_R_FB = 52,
    XR_FACE_EXPRESSION2_MOUTH_LEFT_FB = 53,
    XR_FACE_EXPRESSION2_MOUTH_RIGHT_FB = 54,
    XR_FACE_EXPRESSION2_NOSE_WRINKLER_L_FB = 55,
    XR_FACE_EXPRESSION2_NOSE_WRINKLER_R_FB = 56,
    XR_FACE_EXPRESSION2_OUTER_BROW_RAISER_L_FB = 57,
    XR_FACE_EXPRESSION2_OUTER_BROW_RAISER_R_FB = 58,
    XR_FACE_EXPRESSION2_UPPER_LID_RAISER_L_FB = 59,
    XR_FACE_EXPRESSION2_UPPER_LID_RAISER_R_FB = 60,
    XR_FACE_EXPRESSION2_UPPER_LIP_RAISER_L_FB = 61,
    XR_FACE_EXPRESSION2_UPPER_LIP_RAISER_R_FB = 62,
    XR_FACE_EXPRESSION2_TONGUE_TIP_INTERDENTAL_FB = 63,
    XR_FACE_EXPRESSION2_TONGUE_TIP_ALVEOLAR_FB = 64,
    XR_FACE_EXPRESSION2_TONGUE_FRONT_DORSAL_PALATE_FB = 65,
    XR_FACE_EXPRESSION2_TONGUE_MID_DORSAL_PALATE_FB = 66,
    XR_FACE_EXPRESSION2_TONGUE_BACK_DORSAL_VELAR_FB = 67,
    XR_FACE_EXPRESSION2_TONGUE_OUT_FB = 68,
    XR_FACE_EXPRESSION2_TONGUE_RETREAT_FB = 69,
    XR_FACE_EXPRESSION2_COUNT_FB = 70,
    XR_FACE_EXPRESSION_2FB_MAX_ENUM_FB = 0x7FFFFFFF
} XrFaceExpression2FB;
```



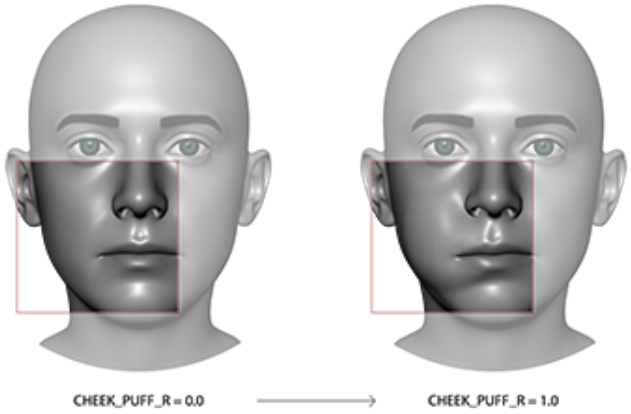
`XR_FACE_EXPRESSION2_BROW_LOWERER_L_FB` knits and lowers the left brow area and lowers central forehead.



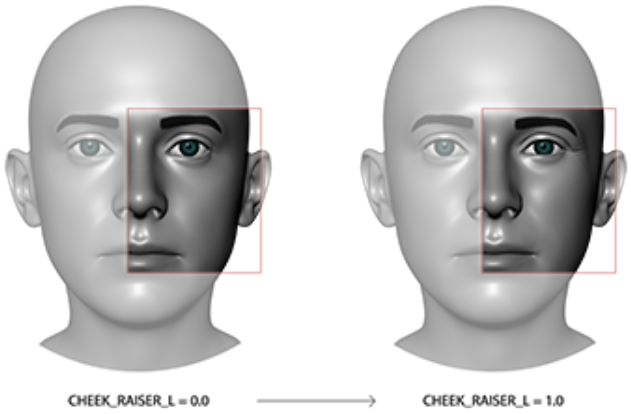
`XR_FACE_EXPRESSION2_BROW_LOWERER_R_FB` knits and lowers the right brow area and lowers central forehead.



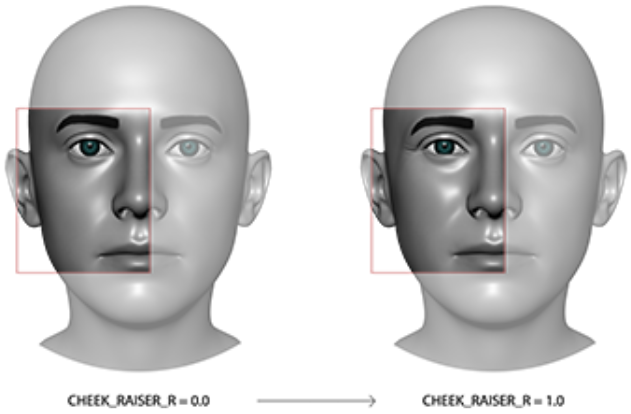
`XR_FACE_EXPRESSION2_CHEEK_PUFF_L_FB` fills the left cheek with air causing them to round and extend outward.



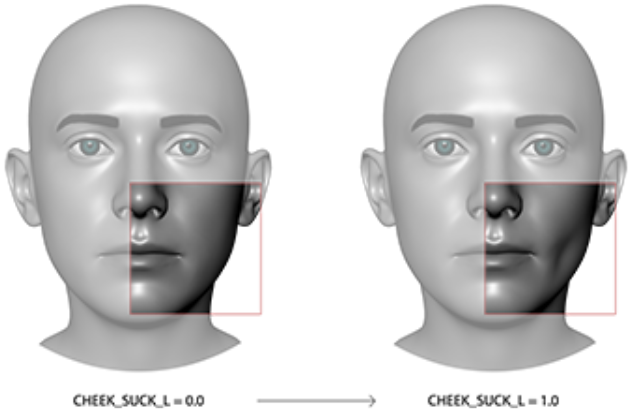
XR_FACE_EXPRESSION2_CHEEK_PUFF_R_FB fills the right cheek with air causing them to round and extend outward.



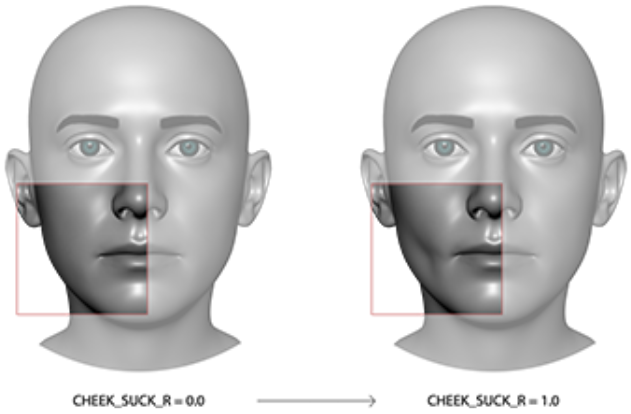
XR_FACE_EXPRESSION2_CHEEK_RAISER_L_FB tightens the outer rings of the left eye orbit and squeezes the lateral left eye corners.



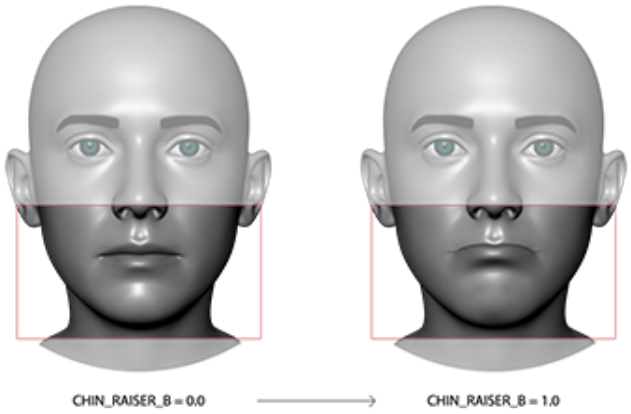
XR_FACE_EXPRESSION2_CHEEK_RAISER_R_FB tightens the outer rings of the right eye orbit and squeezes the lateral right eye corners.



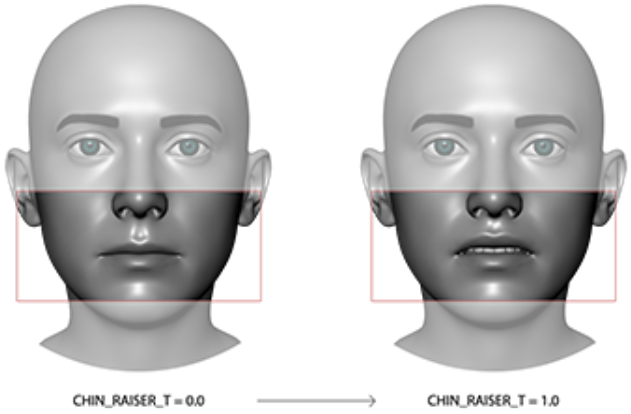
`XR_FACE_EXPRESSION2_CHEEK_SUCK_L_FB` sucks the left cheek inward and against the teeth to create a hollow effect in the cheek.



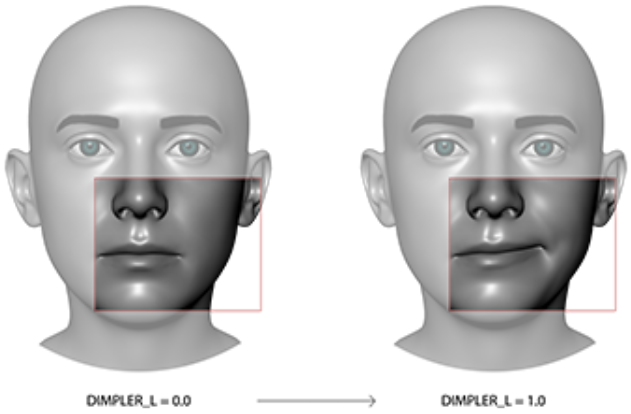
`XR_FACE_EXPRESSION2_CHEEK_SUCK_R_FB` sucks the right cheek inward and against the teeth to create a hollow effect in the cheek.



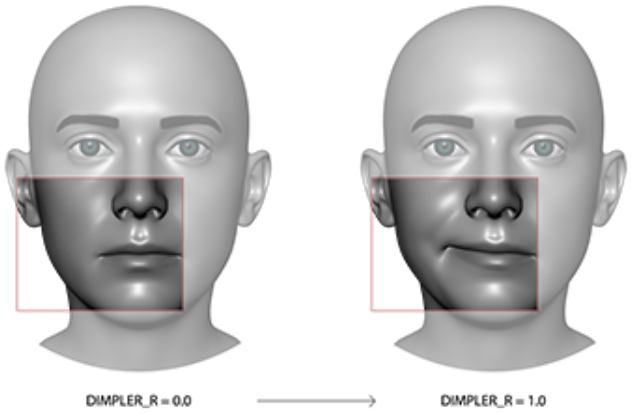
`XR_FACE_EXPRESSION2_CHIN_RAISER_B_FB` pushes the skin of the chin and the lower lip upward.



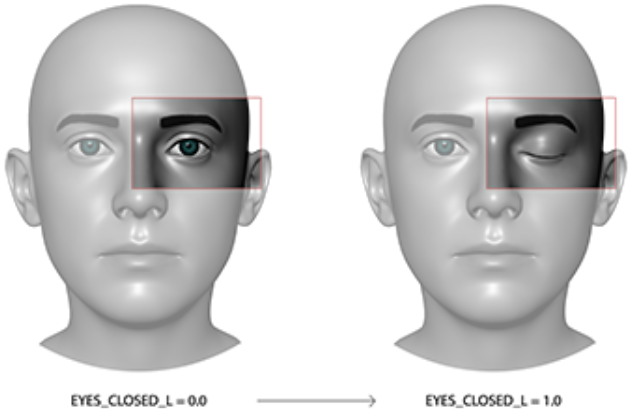
`XR_FACE_EXPRESSION2_CHIN_RAISER_T_FB` pushes up the top lip. This is induced by the upward force from `XR_FACE_EXPRESSION2_CHIN_RAISER_B_FB`.



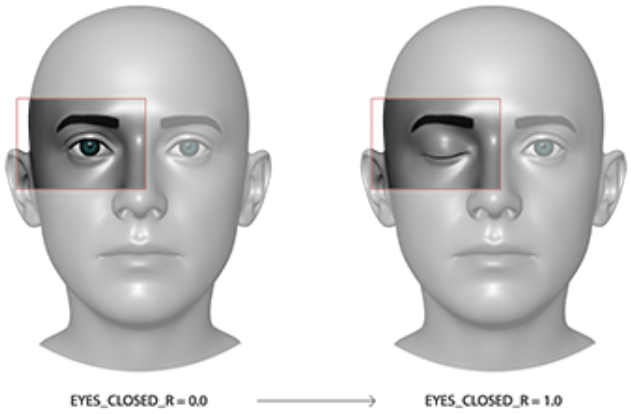
`XR_FACE_EXPRESSION2_DIMPLER_L_FB` pinches the left lip corner against the teeth, drawing them slightly backward and often upward in the process.



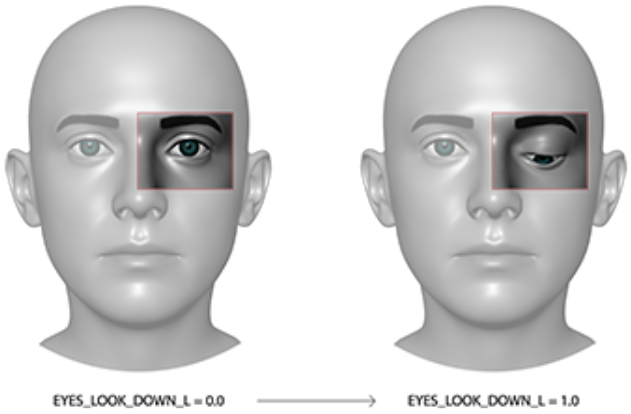
`XR_FACE_EXPRESSION2_DIMPLER_R_FB` pinches the right lip corner against the teeth, drawing them slightly backward and often upward in the process.



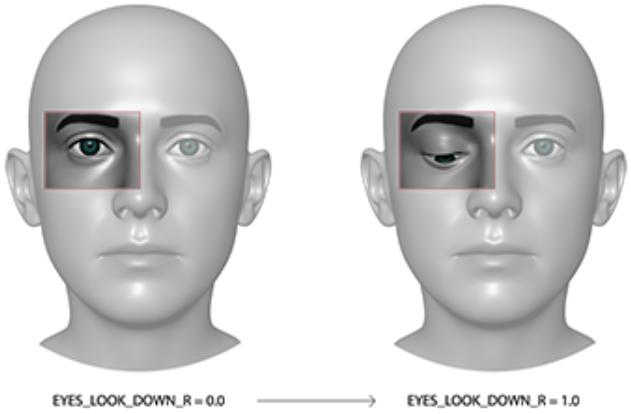
`XR_FACE_EXPRESSION2_EYES_CLOSED_L_FB` lowers the top eyelid to cover the left eye.



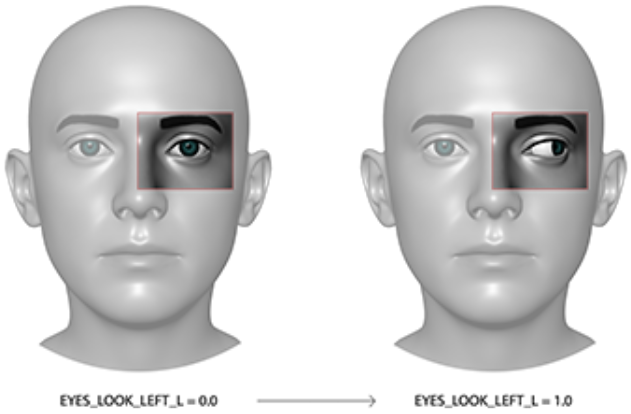
`XR_FACE_EXPRESSION2_EYES_CLOSED_R_FB` lowers the top eyelid to cover the right eye.



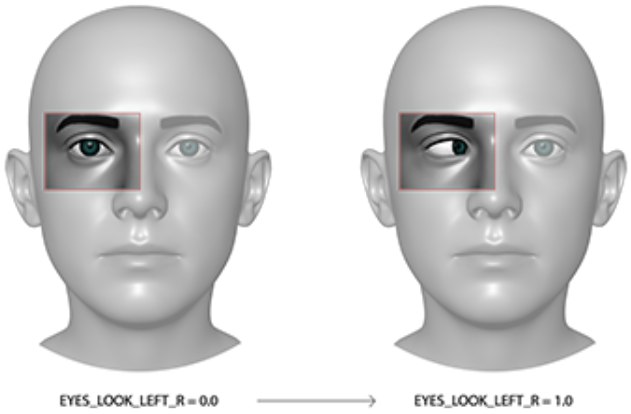
`XR_FACE_EXPRESSION2_EYES_LOOK_DOWN_L_FB` moves the left eye consistent with downward gaze.



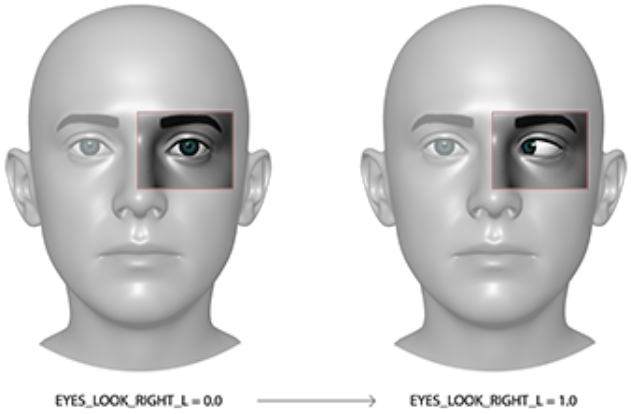
XR_FACE_EXPRESSION2_EYES_LOOK_DOWN_R_FB moves the right eye consistent with downward gaze.



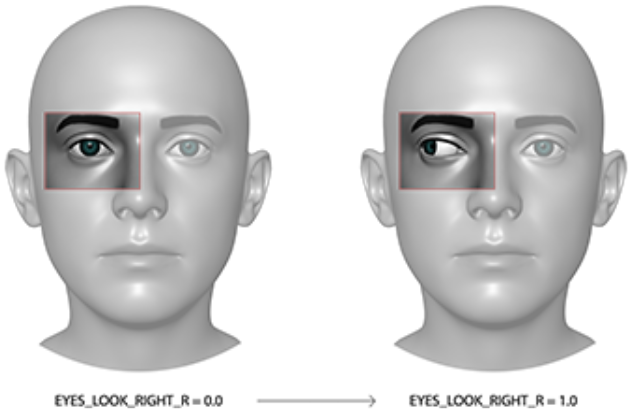
XR_FACE_EXPRESSION2_EYES_LOOK_LEFT_L_FB moves the left eye consistent with leftward gaze.



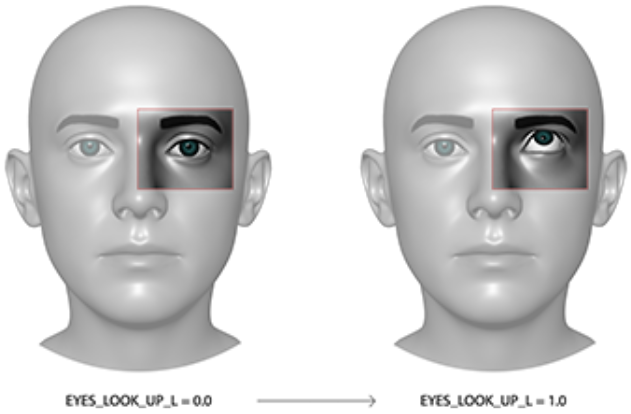
XR_FACE_EXPRESSION2_EYES_LOOK_LEFT_R_FB moves the right eye consistent with leftward gaze.



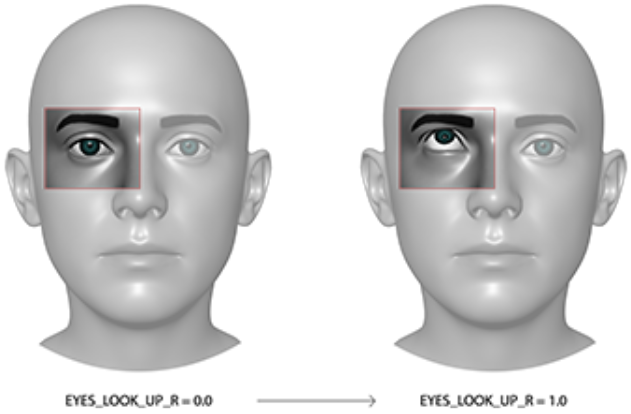
`XR_FACE_EXPRESSION2_EYES_LOOK_RIGHT_L_FB` moves the left eye consistent with rightward gaze.



`XR_FACE_EXPRESSION2_EYES_LOOK_RIGHT_R_FB` moves the right eye consistent with rightward gaze.

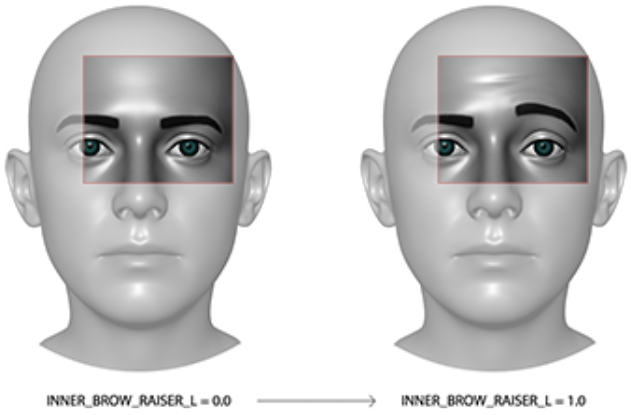


`XR_FACE_EXPRESSION2_EYES_LOOK_UP_L_FB` moves the left eye consistent with upward gaze.

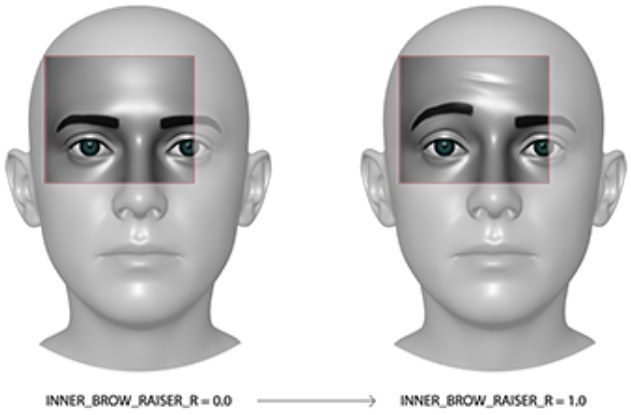


`XR_FACE_EXPRESSION2_EYES_LOOK_UP_R_FB` moves the right eye consistent with upward gaze.

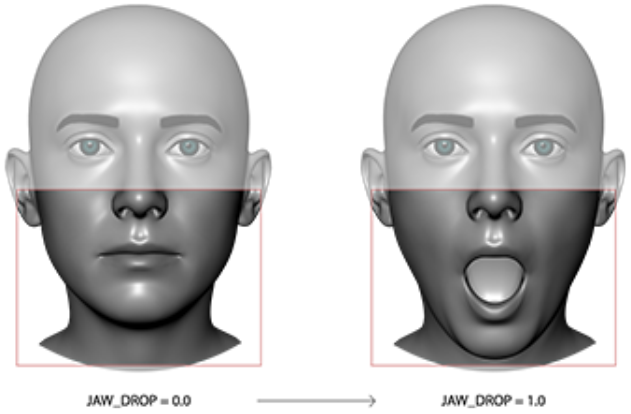
The eight `XR_FACE_EXPRESSION2_EYES_LOOK_*` blend shapes represent eye rotation as a linear mapping from angle to weight. A weight of 0 corresponds to the eye at its neutral forward position and a weight of 1 corresponds to the eye rotated to its maximum angle in the given direction. For opposing directions (e.g. left vs right), only one of the pair will be non-zero at any given time.



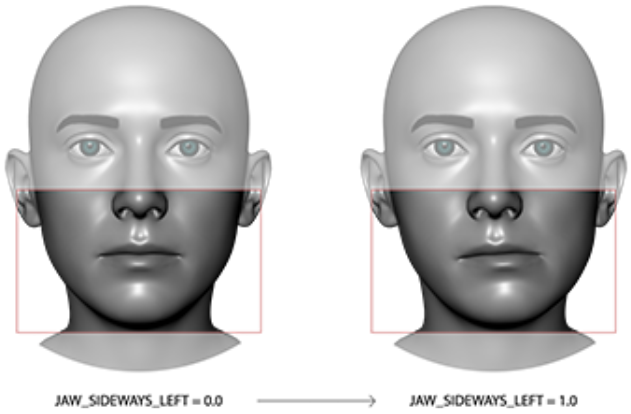
`XR_FACE_EXPRESSION2_INNER_BROW_RAISER_L_FB` lifts the left medial brow and forehead area.



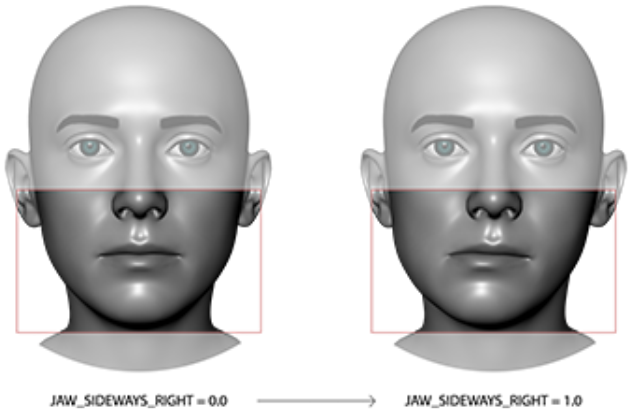
`XR_FACE_EXPRESSION2_INNER_BROW_RAISER_R_FB` lifts the right medial brow and forehead area.



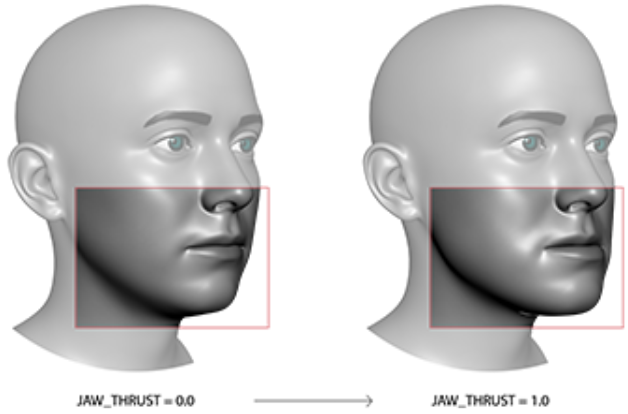
`XR_FACE_EXPRESSION2_JAW_DROP_FB` moves the lower mandible downward and toward the neck.



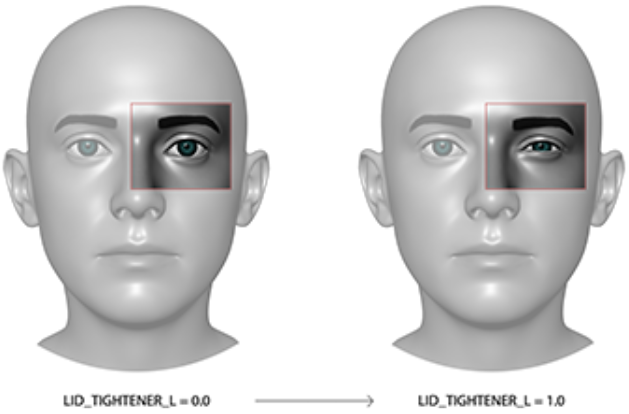
`XR_FACE_EXPRESSION2_JAW_SIDEWAYS_LEFT_FB` moves the lower mandible leftward.



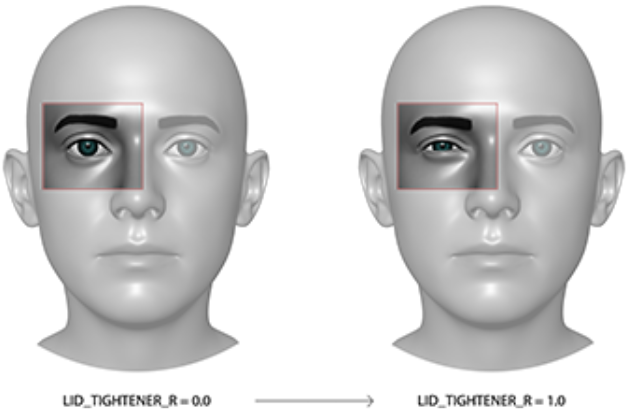
`XR_FACE_EXPRESSION2_JAW_SIDEWAYS_RIGHT_FB` moves the lower mandible rightward.



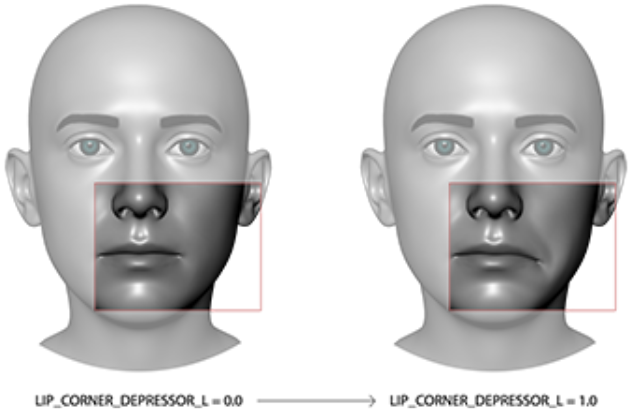
`XR_FACE_EXPRESSION2_JAW_THRUST_FB` projects the lower mandible forward.



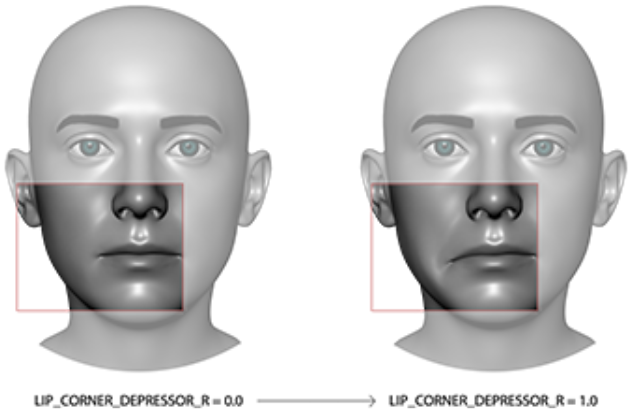
`XR_FACE_EXPRESSION2_LID_TIGHTENER_L_FB` tightens the rings around the left eyelid and pushes the lower eyelid skin toward the inner eye corners.



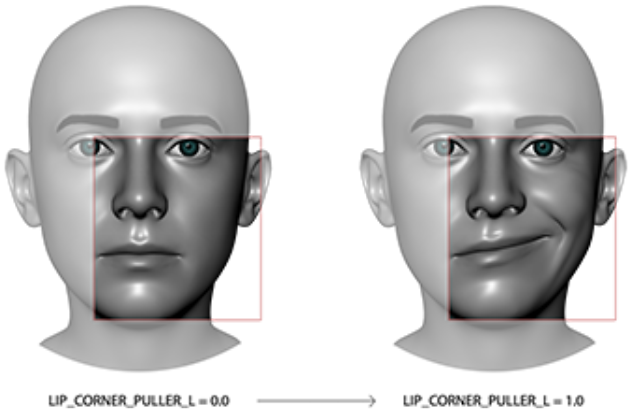
`XR_FACE_EXPRESSION2_LID_TIGHTENER_R_FB` tightens the rings around the right eyelid and pushes the lower eyelid skin toward the inner eye corners.



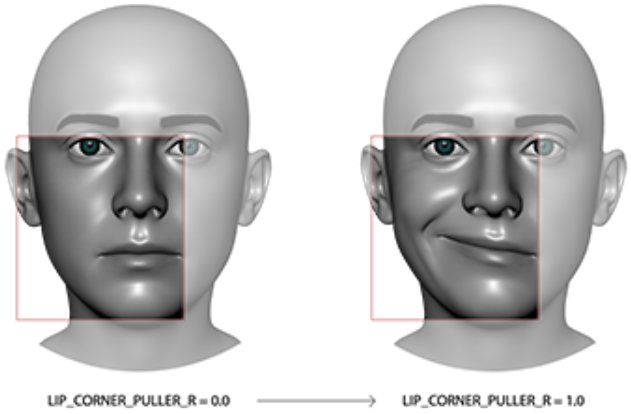
`XR_FACE_EXPRESSION2_LIP_CORNER_DEPRESSOR_L_FB` draws the left lip corner downward.



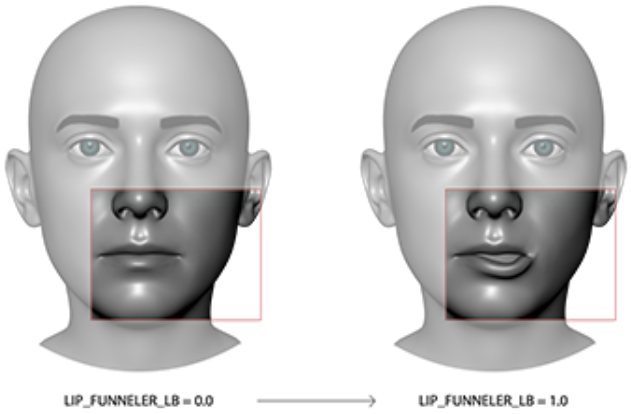
`XR_FACE_EXPRESSION2_LIP_CORNER_DEPRESSOR_R_FB` draws the right lip corner downward.



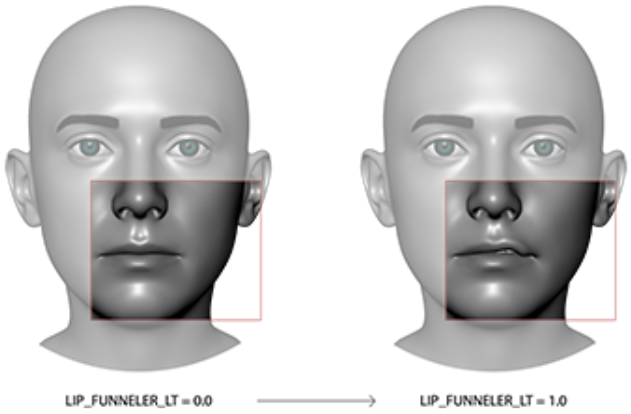
`XR_FACE_EXPRESSION2_LIP_CORNER_PULLER_L_FB` draws the left lip corners up, back, and laterally.



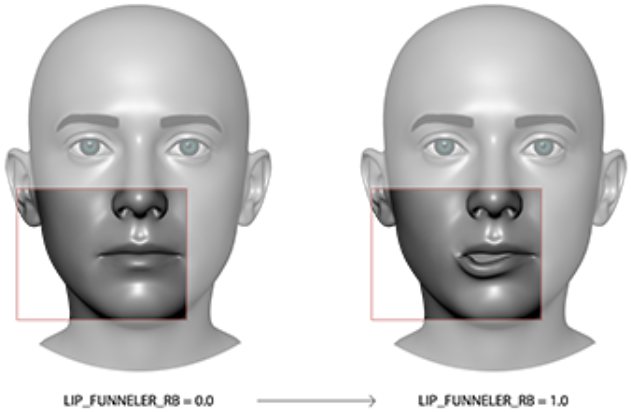
`XR_FACE_EXPRESSION2_LIP_CORNER_PULLER_R_FB` draws the right lip corners up, back, and laterally.



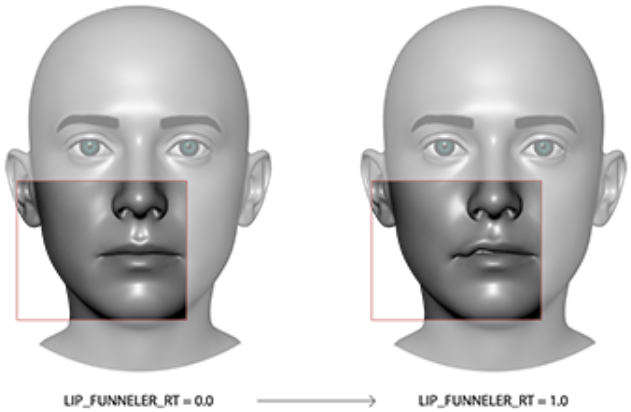
`XR_FACE_EXPRESSION2_LIP_FUNNELER_LB_FB` fans the left bottom lip outward in a forward projection, often rounding the mouth and separating the lips.



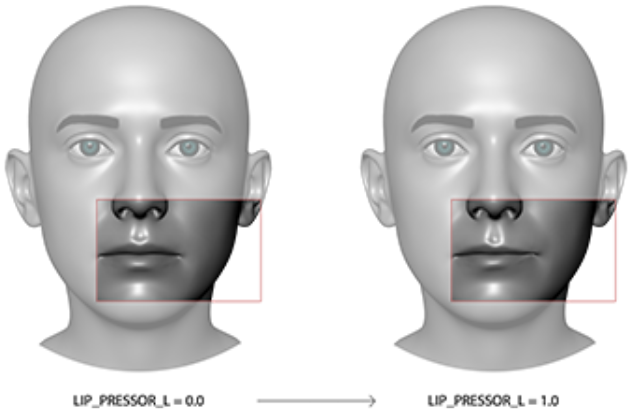
`XR_FACE_EXPRESSION2_LIP_FUNNELER_LT_FB` fans the left top lip outward in a forward projection, often rounding the mouth and separating the lips.



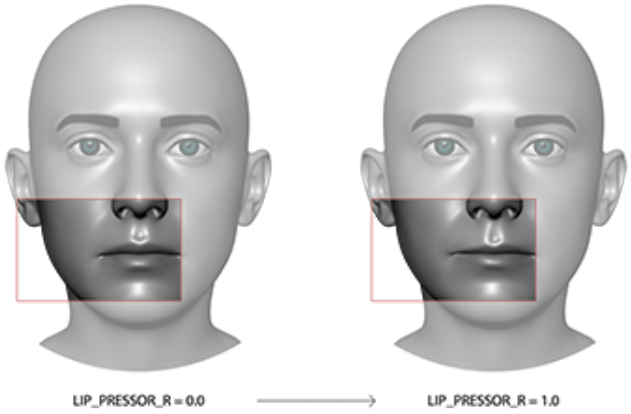
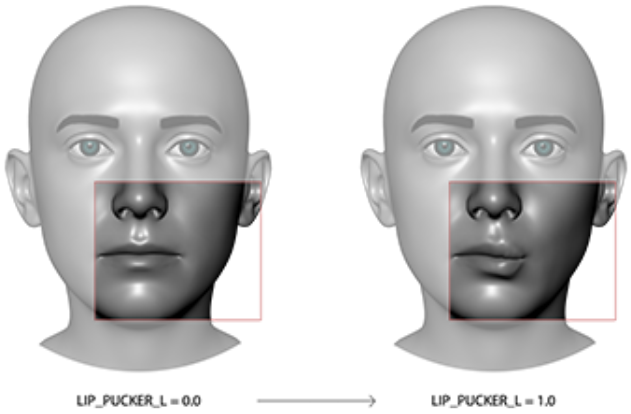
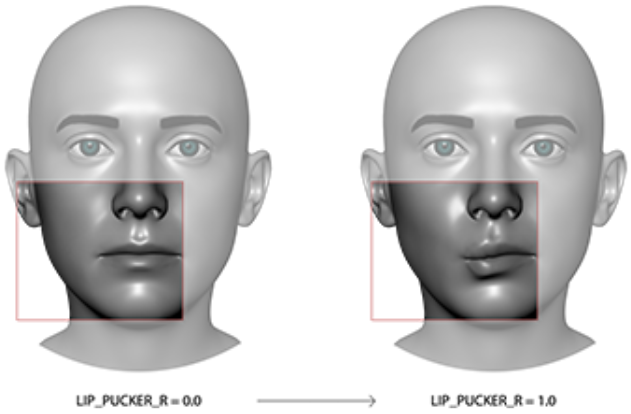
`XR_FACE_EXPRESSION2_LIP_FUNNELER_RB_FB` fans the right bottom lip outward in a forward projection, often rounding the mouth and separating the lips.

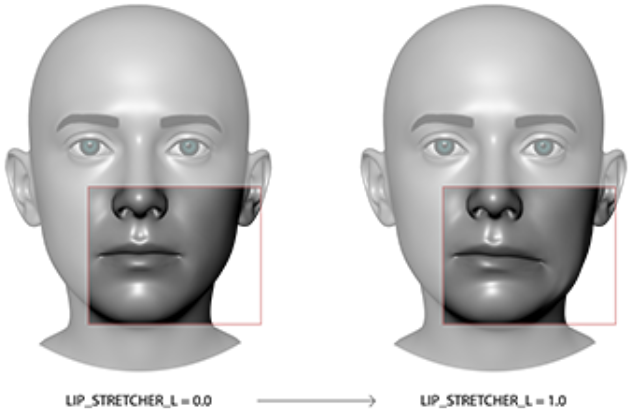


`XR_FACE_EXPRESSION2_LIP_FUNNELER_RT_FB` fans the right top lip outward in a forward projection, often rounding the mouth and separating the lips.

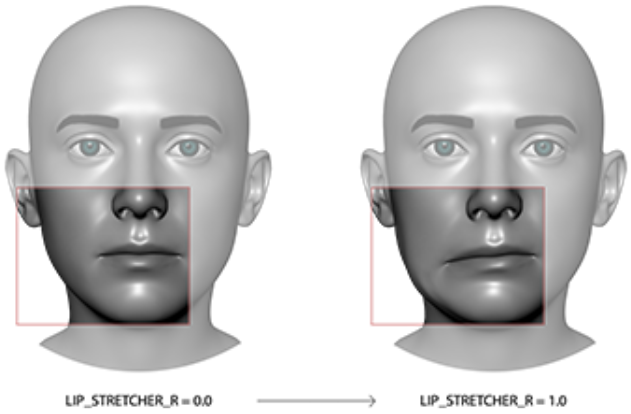


`XR_FACE_EXPRESSION2_LIP_PRESSOR_L_FB` presses the left upper and left lower lips against one another.

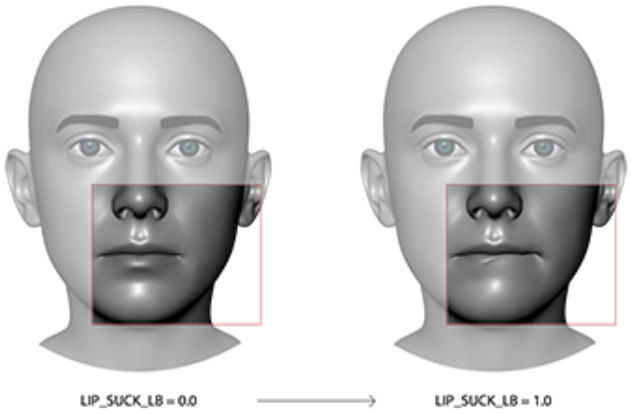
	`XR_FACE_EXPRESSION2_LIP_PRESSOR_R_FB` presses the right upper and right lower lips against one another.
	`XR_FACE_EXPRESSION2_LIP_PUCKER_L_FB` draws the left lip corners medially causing the lips protrude in the process.
	`XR_FACE_EXPRESSION2_LIP_PUCKER_R_FB` draws the right lip corners medially causing the lips protrude in the process.



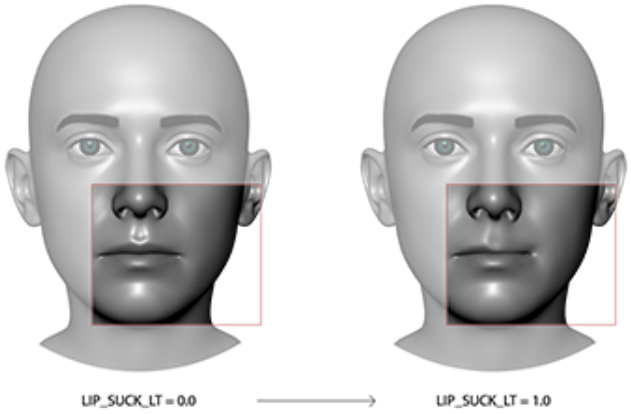
`XR_FACE_EXPRESSION2_LIP_STRETCHER_L_FB` draws the left lip corners laterally, stretching the lips and widening the jawline.



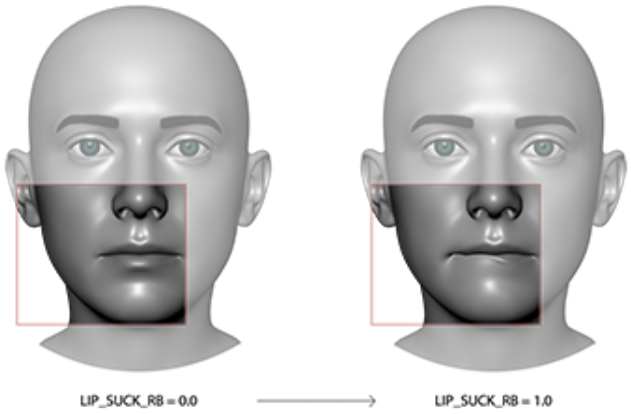
`XR_FACE_EXPRESSION2_LIP_STRETCHER_R_FB` draws the right lip corners laterally, stretching the lips and widening the jawline.



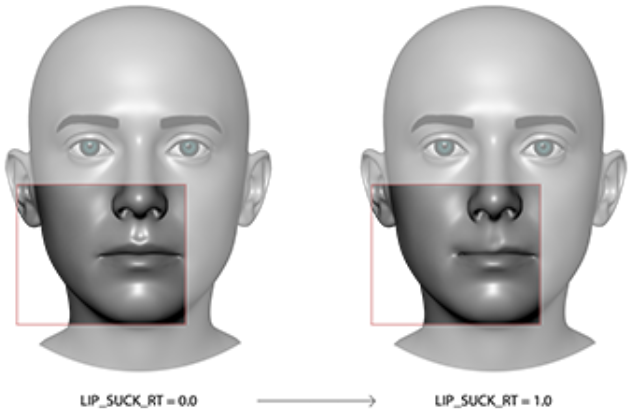
`XR_FACE_EXPRESSION2_LIP_SUCK_LB_FB` sucks the left bottom lip toward the inside of the mouth.



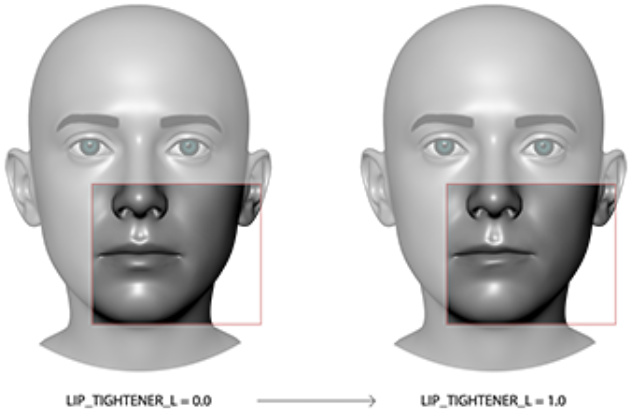
XR_FACE_EXPRESSION2_LIP_SUCK_LT_FB sucks the left top lip toward the inside of the mouth.



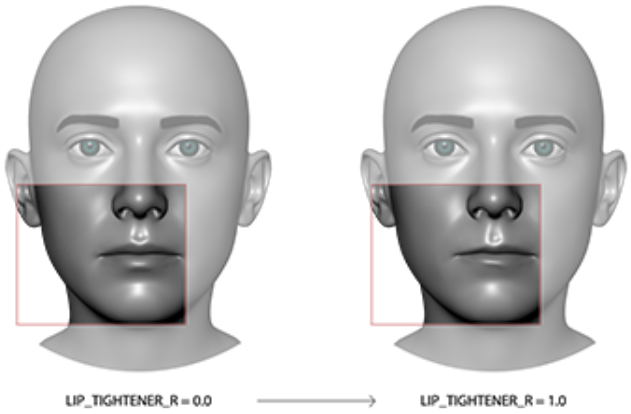
XR_FACE_EXPRESSION2_LIP_SUCK_RB_FB sucks the right bottom lip toward the inside of the mouth.



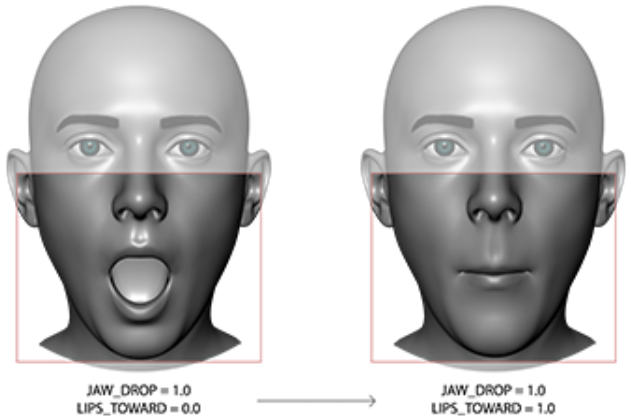
XR_FACE_EXPRESSION2_LIP_SUCK_RT_FB sucks the right top lip toward the inside of the mouth.



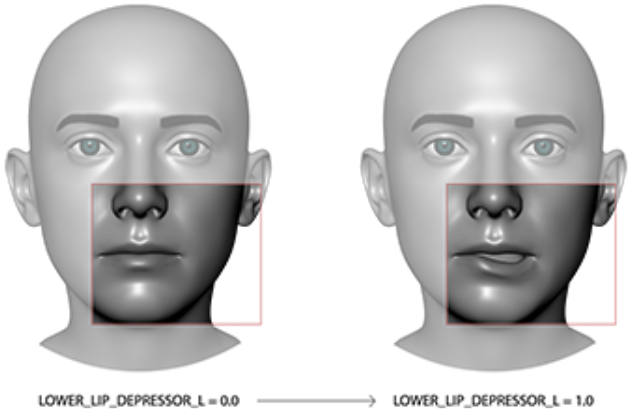
`XR_FACE_EXPRESSION2_LIP_TIGHTENER_L_FB` narrows or constricts the left lips on a horizontal plane.



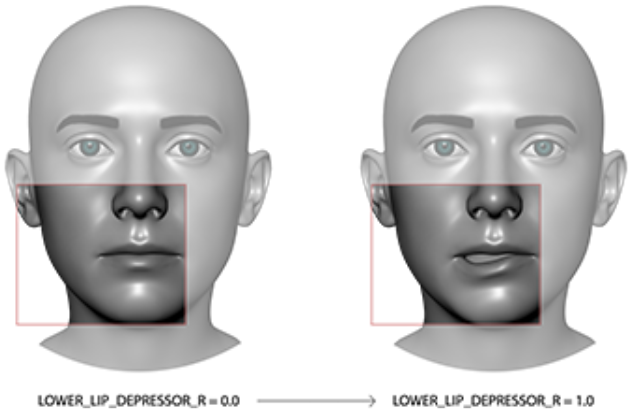
`XR_FACE_EXPRESSION2_LIP_TIGHTENER_R_FB` narrows or constricts the right lips on a horizontal plane.



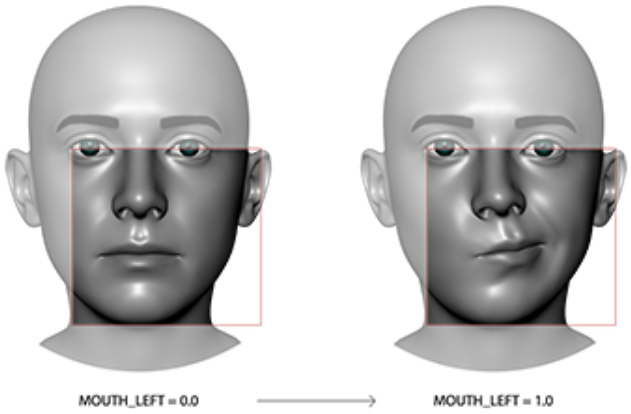
`XR_FACE_EXPRESSION2_LIPS_TOWARD_FB` forces contact between top and bottom lips to keep the mouth closed regardless of the position of the jaw.



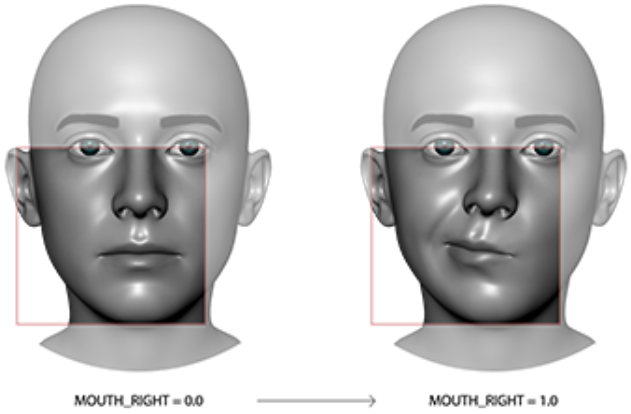
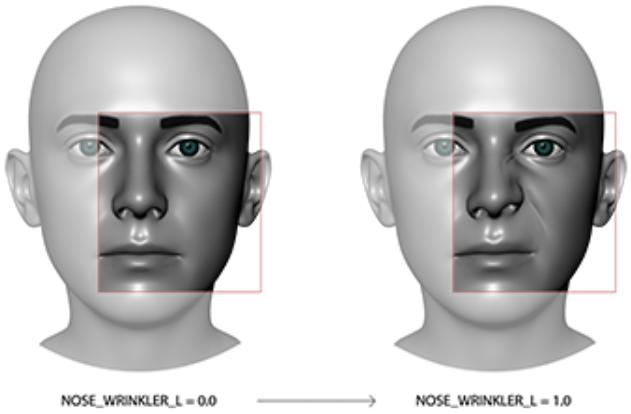
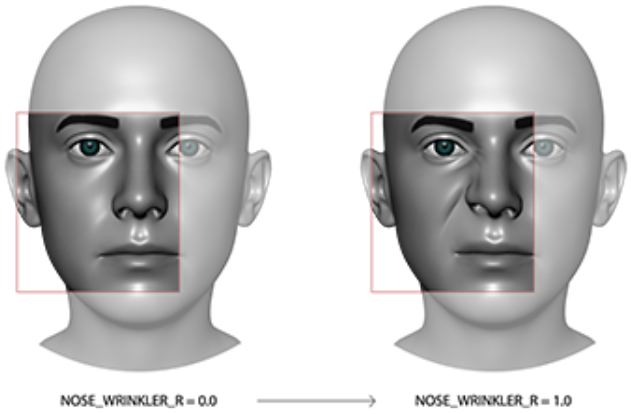
`XR_FACE_EXPRESSION2_LOWER_LIP_DEPRESSOR_L_FB` draws the left lower lip downward and slightly laterally.

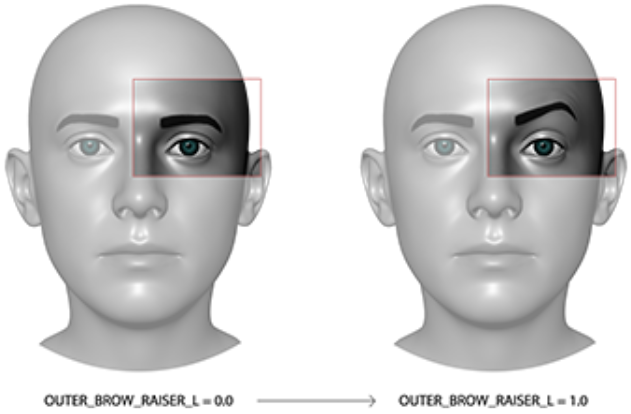


`XR_FACE_EXPRESSION2_LOWER_LIP_DEPRESSOR_R_FB` draws the right lower lip downward and slightly laterally.

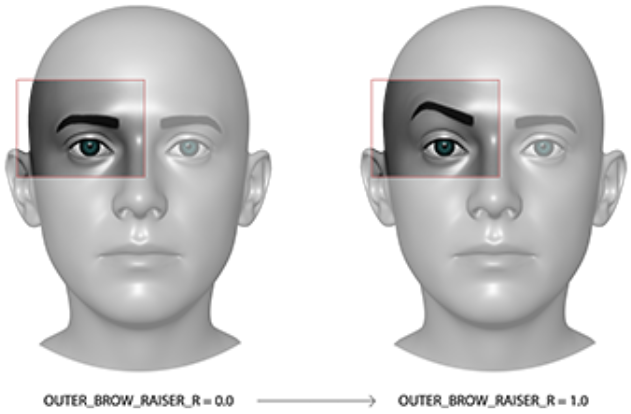


`XR_FACE_EXPRESSION2_MOUTH_LEFT_FB` pulls the left lip corner leftward and pushes the right side of the mouth toward the left lip corner.

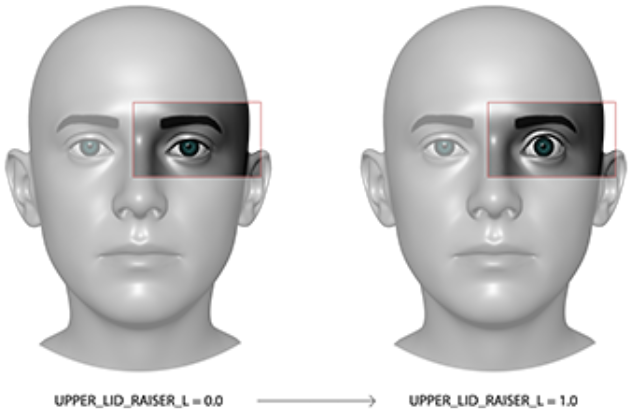
	`XR_FACE_EXPRESSION2_MOUTH_RIGHT_FB` pulls the right lip corner rightward and pushes the left side of the mouth toward the right lip corner.
	`XR_FACE_EXPRESSION2_NOSE_WRINKLER_L_FB` lifts the left sides of the nose, nostrils, and central upper lip area. Often pairs with brow lowering muscles to lower the medial brow tips.
	`XR_FACE_EXPRESSION2_NOSE_WRINKLER_R_FB` lifts the right sides of the nose, nostrils, and central upper lip area. Often pairs with brow lowering muscles to lower the medial brow tips.



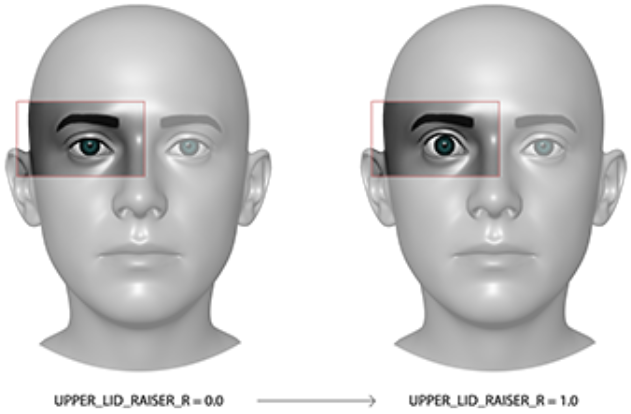
`XR_FACE_EXPRESSION2_OUTER_BROW_RAISER_L_FB` lifts the lateral left brow and forehead areas.



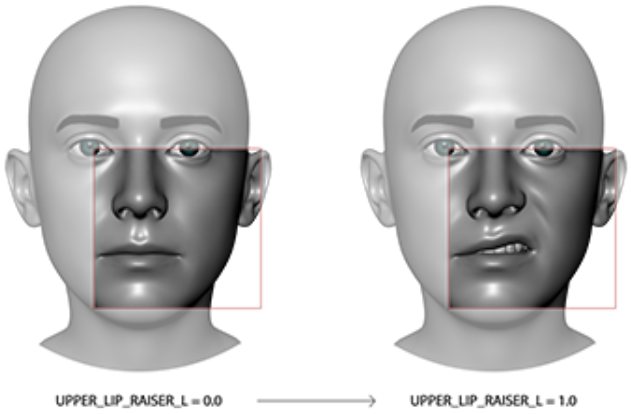
`XR_FACE_EXPRESSION2_OUTER_BROW_RAISER_R_FB` lifts the lateral right brow and forehead areas.



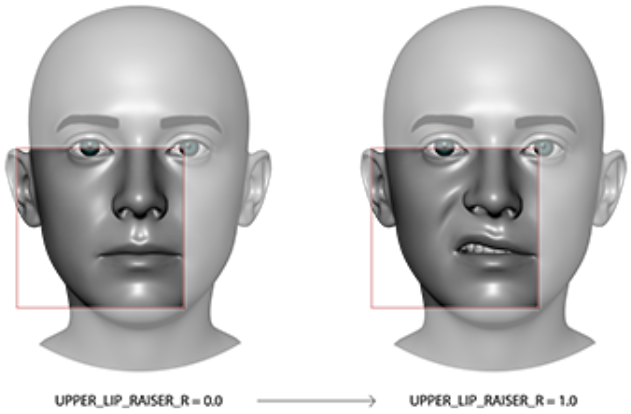
`XR_FACE_EXPRESSION2_UPPER_LID_RAISER_L_FB` pulls the top left eyelid up and back to widen eyes.



`XR_FACE_EXPRESSION2_UPPER_LID_RAISER_R_FB` pulls the top right eyelid up and back to widen eyes.



`XR_FACE_EXPRESSION2_UPPER_LIP_RAISER_L_FB` lifts the top left lip (in a more lateral manner than nose wrinkler).



`XR_FACE_EXPRESSION2_UPPER_LIP_RAISER_R_FB` lifts the top right lip (in a more lateral manner than nose wrinkler).

	`XR_FACE_EXPRESSION2_TONGUE_TIP_INTERDENTAL_FB` raises the tip of the tongue to touch the top teeth like with the viseme "TH". The tongue is visible and slightly sticks out past the teeth line.
	`XR_FACE_EXPRESSION2_TONGUE_TIP_ALVEOLAR_FB` raises the tip of tongue to touch the back of the top teeth like in the viseme "NN".
	`XR_FACE_EXPRESSION2_TONGUE_FRONT_DORSAL_PALATE_FB` makes the front part of the tongue to press against the palate like in the viseme "CH".
	`XR_FACE_EXPRESSION2_TONGUE_MID_DORSAL_PALATE_FB` presses the middle of the tongue against the palate like in the viseme "DD".

	XR_FACE_EXPRESSION2_TONGUE_BACK_DORSAL_VELAR_FB presses the back of the tongue against the palate like in the viseme "KK".
	XR_FACE_EXPRESSION2_TONGUE_OUT_FB sticks the tongue out.
	XR_FACE_EXPRESSION2_TONGUE_RETREAT_FB pulls the tongue back in the throat and makes the tongue to stay down like in the viseme "AA".

12.106.8. Conventions of confidence areas

This extension defines two separate areas of confidence.

```
// Provided by XR_FB_face_tracking2
typedef enum XrFaceConfidence2FB {
    XR_FACE_CONFIDENCE2_LOWER_FACE_FB = 0,
    XR_FACE_CONFIDENCE2_UPPER_FACE_FB = 1,
    XR_FACE_CONFIDENCE2_COUNT_FB = 2,
    XR_FACE_CONFIDENCE_2FB_MAX_ENUM_FB = 0x7FFFFFFF
} XrFaceConfidence2FB;
```

The "upper face" area represents everything above the upper lip, including the eyes and eyebrows. The "lower face" area represents everything under the eyes, including the mouth and chin. Cheek and nose

areas contribute to both "upper face" and "lower face" areas.

New Object Types

- XrFaceTracker2FB

New Flag Types

New Enum Constants

XrObjectType enumeration is extended with:

- `XR_OBJECT_TYPE_FACE_TRACKER2_FB`

XrStructureType enumeration is extended with:

- `XR_TYPE_SYSTEM_FACE_TRACKING_PROPERTIES2_FB`
- `XR_TYPE_FACE_TRACKER_CREATE_INFO2_FB`
- `XR_TYPE_FACE_EXPRESSION_INFO2_FB`
- `XR_TYPE_FACE_EXPRESSION_WEIGHTS2_FB`

New Enums

- XrFaceExpression2FB
- XrFaceExpressionSet2FB
- XrFaceConfidence2FB
- XrFaceTrackingDataSource2FB

New Structures

- XrSystemFaceTrackingProperties2FB
- XrFaceTrackerCreateInfo2FB
- XrFaceExpressionInfo2FB
- XrFaceExpressionWeights2FB

New Functions

- xrCreateFaceTracker2FB
- xrDestroyFaceTracker2FB
- xrGetFaceExpressionWeights2FB

Issues

- Should we add the tongue shapes to `XR_FB_face_tracking` as a new enum value in

XrFaceExpressionSetFB?

- **Resolved.** We expect that all applications should use `XR_FB_face_tracking2` in the future and that `XR_FB_face_tracking` will ultimately be replaced by this extension.

Version History

- Revision 1, 2023-10-06 (Jaebong Lee)
  - Initial extension description

12.107. XR_FB_foveation

Name String

`XR_FB_foveation`

Extension Type

Instance extension

Registered Extension Number

115

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0
and
`XR_FB_swapchain_update_state`

Contributors

Kevin Xiao, Facebook
Ross Ning, Facebook
Remi Palandri, Facebook
Cass Everitt, Facebook
Gloria Kennickell, Facebook

Overview

Foveation in the context of XR is a rendering technique that allows the area of an image near the focal point or fovea of the eye to be displayed at higher resolution than areas in the periphery. This trades some visual fidelity in the periphery, where it is less noticeable for the user, for improved rendering performance, most notably regarding the fragment shader, as fewer pixels or subpixels in the

periphery need to be shaded and processed. On platforms which support foveation patterns and features tailored towards the optical properties, performance profiles, and hardware support of specific HMDs, application developers may request and use available foveation profiles from the runtime. Foveation profiles refer to a set of properties describing how, when, and where foveation will be applied.

This extension allows:

- An application to create swapchains that can support foveation for its graphics API.
- An application to request foveation profiles supported by the runtime and apply them to foveation-supported swapchains.

New Object Types

```
// Provided by XR_FB_foveation
XR_DEFINE_HANDLE(XrFoveationProfileFB)
```

XrFoveationProfileFB represents a set of properties and resources that define a foveation pattern for the runtime, which **can** be applied to individual swapchains.

New Flag Types

```
// Provided by XR_FB_foveation
typedef XrFlags64 XrSwapchainCreateFoveationFlagsFB;
```

```
// Provided by XR_FB_foveation
// Flag bits for XrSwapchainCreateFoveationFlagsFB
static const XrSwapchainCreateFoveationFlagsFB
XR_SWAPCHAIN_CREATE_FOVEATION_SCALED_BIN_BIT_FB = 0x00000001;
static const XrSwapchainCreateFoveationFlagsFB
XR_SWAPCHAIN_CREATE_FOVEATION_FRAGMENT_DENSITY_MAP_BIT_FB = 0x00000002;
```

Flag Descriptions

- XR_SWAPCHAIN_CREATE_FOVEATION_SCALED_BIN_BIT_FB — Explicitly create the swapchain with scaled bin foveation support. The application must ensure that the swapchain is using the OpenGL graphics API and that the QCOM_texture_foveated extension is supported and enabled.
- XR_SWAPCHAIN_CREATE_FOVEATION_FRAGMENT_DENSITY_MAP_BIT_FB — Explicitly create the swapchain with fragment density map foveation support. The application must ensure that the swapchain is using the Vulkan graphics API and that the VK_EXT_fragment_density_map extension is supported and enabled.

```
// Provided by XR_FB_foveation
typedef XrFlags64 XrSwapchainStateFoveationFlagsFB;
```

```
// Provided by XR_FB_foveation
// Flag bits for XrSwapchainStateFoveationFlagsFB
```

There are currently no foveation swapchain state flags. This is reserved for future use.

New Enum Constants

XrObjectType enumeration is extended with:

- XR_OBJECT_TYPE_FOVEATION_PROFILE_FB

XrStructureType enumeration is extended with:

- XR_TYPE_FOVEATION_PROFILE_CREATE_INFO_FB
- XR_TYPE_SWAPCHAIN_CREATE_INFO_FOVEATION_FB
- XR_TYPE_SWAPCHAIN_STATE_FOVEATION_FB

New Enums

New Structures

XrFoveationProfileCreateInfoFB **must** be provided when calling xrCreateFoveationProfileFB. The runtime **must** interpret XrFoveationProfileCreateInfoFB without any additional structs in its `next` chain as a request to create a foveation profile that will apply no foveation to any area of the swapchain.

The XrFoveationProfileCreateInfoFB structure is defined as:

```
// Provided by XR_FB_foveation
typedef struct XrFoveationProfileCreateInfoFB {
    XrStructureType    type;
    void*              next;
} XrFoveationProfileCreateInfoFB;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.

Valid Usage (Implicit)

- The `XR_FB_foveation` extension **must** be enabled prior to using XrFoveationProfileCreateInfoFB
- `type` **must** be `XR_TYPE_FOVEATION_PROFILE_CREATE_INFO_FB`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain. See also: XrFoveationLevelProfileCreateInfoFB

XrSwapchainCreateInfoFoveationFB **can** be provided in the `next` chain of XrSwapchainCreateInfo when calling xrCreateSwapchain to indicate to the runtime that the swapchain **must** be created with foveation support in the corresponding graphics API. XrSwapchainCreateInfoFoveationFB contains additional foveation-specific flags for swapchain creation.

The XrSwapchainCreateInfoFoveationFB structure is defined as:

```
// Provided by XR_FB_foveation
typedef struct XrSwapchainCreateInfoFoveationFB {
    XrStructureType                       type;
    void*                                 next;
    XrSwapchainCreateFoveationFlagsFB     flags;
} XrSwapchainCreateInfoFoveationFB;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `flags` is a bitmask of XrSwapchainCreateFoveationFlagBitsFB which indicate various characteristics for how foveation is enabled on the swapchain.

Valid Usage (Implicit)

- The `XR_FB_foveation` extension **must** be enabled prior to using XrSwapchainCreateInfoFoveationFB
- `type` **must** be `XR_TYPE_SWAPCHAIN_CREATE_INFO_FOVEATION_FB`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `flags` **must** be `0` or a valid combination of XrSwapchainCreateFoveationFlagBitsFB values

XrSwapchainStateFoveationFB **can** be provided in place of XrSwapchainStateBaseHeaderFB when calling xrUpdateSwapchainFB to update the foveation properties of the swapchain. XrSwapchainCreateInfoFoveationFB contains the desired foveation profile and additional foveation specific flags for updating the swapchain.

The XrSwapchainStateFoveationFB structure is defined as:

```
// Provided by XR_FB_foveation
typedef struct XrSwapchainStateFoveationFB {
    XrStructureType                       type;
    void*                                 next;
    XrSwapchainStateFoveationFlagsFB      flags;
    XrFoveationProfileFB                  profile;
} XrSwapchainStateFoveationFB;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `flags` is a bitmask of XrSwapchainStateFoveationFlagBitsFB which indicate various characteristics of how and when the foveation properties of the swapchain **must** be updated.
- `profile` is an XrFoveationProfileFB defining the desired foveation properties to be applied to the swapchain.

Valid Usage (Implicit)

- The `XR_FB_foveation` extension **must** be enabled prior to using XrSwapchainStateFoveationFB
- `type` **must** be `XR_TYPE_SWAPCHAIN_STATE_FOVEATION_FB`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `flags` **must** be `0`
- `profile` **must** be a valid XrFoveationProfileFB handle

New Functions

The xrCreateFoveationProfileFB function is defined as:

```
// Provided by XR_FB_foveation
XrResult xrCreateFoveationProfileFB(
    XrSession                                   session,
    const XrFoveationProfileCreateInfoFB*       createInfo,
    XrFoveationProfileFB*                       profile);
```

Parameter Descriptions

- `session` is the XrSession that created the swapchains to which this foveation profile will be applied.
- `createInfo` is a pointer to an XrFoveationProfileCreateInfoFB structure containing parameters to be used to create the foveation profile.
- `profile` is a pointer to a handle in which the created XrFoveationProfileFB is returned.

Creates an XrFoveationProfileFB handle. The returned foveation profile handle **may** be subsequently

used in API calls.

Valid Usage (Implicit)

- The XR_FB_foveation extension **must** be enabled prior to calling xrCreateFoveationProfileFB
- session **must** be a valid XrSession handle
- createInfo **must** be a pointer to a valid XrFoveationProfileCreateInfoFB structure
- profile **must** be a pointer to an XrFoveationProfileFB handle

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The xrDestroyFoveationProfileFB function is defined as:

```
// Provided by XR_FB_foveation
XrResult xrDestroyFoveationProfileFB(
    XrFoveationProfileFB                        profile);
```

Parameter Descriptions

- profile is the XrFoveationProfileFB to destroy.

XrFoveationProfileFB handles are destroyed using xrDestroyFoveationProfileFB. A

XrFoveationProfileFB **may** be safely destroyed after being applied to a swapchain state using xrUpdateSwapchainFB without affecting the foveation parameters of the swapchain. The application is responsible for ensuring that it has no calls using profile in progress when the foveation profile is destroyed.

Valid Usage (Implicit)

- The `XR_FB_foveation` extension **must** be enabled prior to calling xrDestroyFoveationProfileFB
- `profile` **must** be a valid XrFoveationProfileFB handle

Thread Safety

- Access to `profile`, and any child handles, **must** be externally synchronized

Return Codes

Success

- `XR_SUCCESS`

Failure

- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_RUNTIME_FAILURE`

Issues

Version History

- Revision 1, 2021-05-13 (Kevin Xiao)
  - Initial extension description

12.108. XR_FB_foveation_configuration

Name String

`XR_FB_foveation_configuration`

Extension Type

Instance extension

Registered Extension Number

116

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0
and
`XR_FB_foveation`

Contributors

Kevin Xiao, Facebook
Ross Ning, Facebook
Remi Palandri, Facebook
Cass Everitt, Facebook
Gloria Kennickell, Facebook

Overview

On Facebook HMDs, developers **may** create foveation profiles generated by the runtime for the optical properties and performance profile of the specific HMD.

This extension allows:

- An application to request foveation profiles generated by the runtime for the current HMD.

New Object Types

New Flag Types

New Enum Constants

XrStructureType enumeration is extended with:

- `XR_TYPE_FOVEATION_LEVEL_PROFILE_CREATE_INFO_FB`

New Enums

The possible foveation levels are specified by the XrFoveationLevelFB enumeration:

```
// Provided by XR_FB_foveation_configuration
typedef enum XrFoveationLevelFB {
    XR_FOVEATION_LEVEL_NONE_FB = 0,
    XR_FOVEATION_LEVEL_LOW_FB = 1,
    XR_FOVEATION_LEVEL_MEDIUM_FB = 2,
    XR_FOVEATION_LEVEL_HIGH_FB = 3,
    XR_FOVEATION_LEVEL_MAX_ENUM_FB = 0x7FFFFFFF
} XrFoveationLevelFB;
```

Enumerant Descriptions

- XR_FOVEATION_LEVEL_NONE_FB — No foveation
- XR_FOVEATION_LEVEL_LOW_FB — Less foveation (higher periphery visual fidelity, lower performance)
- XR_FOVEATION_LEVEL_MEDIUM_FB — Medium foveation (medium periphery visual fidelity, medium performance)
- XR_FOVEATION_LEVEL_HIGH_FB — High foveation (lower periphery visual fidelity, higher performance)

The possible foveation levels are specified by the XrFoveationDynamicFB enumeration:

```
// Provided by XR_FB_foveation_configuration
typedef enum XrFoveationDynamicFB {
    XR_FOVEATION_DYNAMIC_DISABLED_FB = 0,
    XR_FOVEATION_DYNAMIC_LEVEL_ENABLED_FB = 1,
    XR_FOVEATION_DYNAMIC_MAX_ENUM_FB = 0x7FFFFFFF
} XrFoveationDynamicFB;
```

Enumerant Descriptions

- XR_FOVEATION_DYNAMIC_DISABLED_FB — Static foveation at the maximum desired level
- XR_FOVEATION_DYNAMIC_LEVEL_ENABLED_FB — Dynamic changing foveation based on performance headroom available up to the maximum desired level

New Structures

XrFoveationLevelProfileCreateInfoFB **can** be provided in the `next` chain of XrFoveationProfileCreateInfoFB when calling xrCreateFoveationProfileFB. The runtime **must** interpret XrSwapchainCreateInfoFoveationFB with XrFoveationLevelProfileCreateInfoFB in its `next` chain as a request to create a foveation profile that will apply a fixed foveation pattern according to the

parameters defined in the XrFoveationLevelProfileCreateInfoFB.

The XrFoveationLevelProfileCreateInfoFB structure is defined as:

```
// Provided by XR_FB_foveation_configuration
typedef struct XrFoveationLevelProfileCreateInfoFB {
    XrStructureType         type;
    void*                   next;
    XrFoveationLevelFB      level;
    float                   verticalOffset;
    XrFoveationDynamicFB    dynamic;
} XrFoveationLevelProfileCreateInfoFB;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `level` is the maximum desired foveation level.
- `verticalOffset` is the desired vertical offset in degrees for the center of the foveation pattern.
- `dynamic` is the desired dynamic foveation setting.

Valid Usage (Implicit)

- The `XR_FB_foveation_configuration` extension **must** be enabled prior to using XrFoveationLevelProfileCreateInfoFB
- `type` **must** be `XR_TYPE_FOVEATION_LEVEL_PROFILE_CREATE_INFO_FB`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain. See also: XrFoveationEyeTrackedProfileCreateInfoMETA
- `level` **must** be a valid XrFoveationLevelFB value
- `dynamic` **must** be a valid XrFoveationDynamicFB value

New Functions

Issues

Version History

- Revision 1, 2021-05-13 (Kevin Xiao)

- Initial extension description

12.109. XR_FB_foveation_vulkan

Name String

`XR_FB_foveation_vulkan`

Extension Type

Instance extension

Registered Extension Number

161

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0
and
`XR_FB_foveation`

Contributors

Kevin Xiao, Facebook
Ross Ning, Facebook
Remi Palandri, Facebook
Cass Everitt, Facebook
Gloria Kennickell, Facebook

Overview

The Vulkan graphics API requires an image to be applied to the swapchain to apply a foveation pattern.

This extension allows:

- An application to obtain foveation textures or constructs needed for foveated rendering in Vulkan.

New Object Types

New Flag Types

New Enum Constants

XrStructureType enumeration is extended with:

- XR_TYPE_SWAPCHAIN_IMAGE_FOVEATION_VULKAN_FB

New Enums

New Structures

XrSwapchainImageFoveationVulkanFB **can** be provided in the next chain of XrSwapchainImageVulkanKHR when calling xrEnumerateSwapchainImages on a swapchain created with xrCreateSwapchain, if XrSwapchainCreateInfoFoveationFB was in the next chain of XrSwapchainCreateInfo and XrSwapchainCreateInfoFoveationFB had the XR_SWAPCHAIN_CREATE_FOVEATION_FRAGMENT_DENSITY_MAP_BIT_FB flag set. The image, width, and height will be populated by xrEnumerateSwapchainImages to be compatible with the corresponding XrSwapchainImageVulkanKHR.

The XrSwapchainImageFoveationVulkanFB structure is defined as:

```
// Provided by XR_FB_foveation_vulkan
typedef struct XrSwapchainImageFoveationVulkanFB {
    XrStructureType    type;
    void*              next;
    VkImage            image;
    uint32_t           width;
    uint32_t           height;
} XrSwapchainImageFoveationVulkanFB;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- image is a valid Vulkan VkImage to use.
- width is the horizontal width in pixels of the image.
- height is the vertical height in pixels of the image.

Valid Usage (Implicit)

- The `XR_FB_foveation_vulkan` extension **must** be enabled prior to using XrSwapchainImageFoveationVulkanFB
- `type` **must** be `XR_TYPE_SWAPCHAIN_IMAGE_FOVEATION_VULKAN_FB`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

New Functions

Issues

Version History

- Revision 1, 2021-05-26 (Kevin Xiao)
  - Initial extension description

12.110. XR_FB_hand_tracking_aim

Name String

`XR_FB_hand_tracking_aim`

Extension Type

Instance extension

Registered Extension Number

112

Revision

2

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0
and
`XR_EXT_hand_tracking`

Contributors

Federico Schliemann, Facebook
James Hillery, Facebook
Gloria Kennickell, Facebook

Overview

The XR_EXT_hand_tracking extension provides a list of hand joint poses which represent the current configuration of the tracked hands. This extension adds a layer of gesture recognition that is used by the system.

This extension allows:

- An application to get a set of basic gesture states for the hand when using the XR_EXT_hand_tracking extension.

New Object Types

New Flag Types

```
// Provided by XR_FB_hand_tracking_aim
typedef XrFlags64 XrHandTrackingAimFlagsFB;
```

```
// Provided by XR_FB_hand_tracking_aim
// Flag bits for XrHandTrackingAimFlagsFB
static const XrHandTrackingAimFlagsFB XR_HAND_TRACKING_AIM_COMPUTED_BIT_FB = 0x00000001;
static const XrHandTrackingAimFlagsFB XR_HAND_TRACKING_AIM_VALID_BIT_FB = 0x00000002;
static const XrHandTrackingAimFlagsFB XR_HAND_TRACKING_AIM_INDEX_PINCHING_BIT_FB =
0x00000004;
static const XrHandTrackingAimFlagsFB XR_HAND_TRACKING_AIM_MIDDLE_PINCHING_BIT_FB =
0x00000008;
static const XrHandTrackingAimFlagsFB XR_HAND_TRACKING_AIM_RING_PINCHING_BIT_FB =
0x00000010;
static const XrHandTrackingAimFlagsFB XR_HAND_TRACKING_AIM_LITTLE_PINCHING_BIT_FB =
0x00000020;
static const XrHandTrackingAimFlagsFB XR_HAND_TRACKING_AIM_SYSTEM_GESTURE_BIT_FB =
0x00000040;
static const XrHandTrackingAimFlagsFB XR_HAND_TRACKING_AIM_DOMINANT_HAND_BIT_FB =
0x00000080;
static const XrHandTrackingAimFlagsFB XR_HAND_TRACKING_AIM_MENU_PRESSED_BIT_FB =
0x00000100;
```

Flag Descriptions

- XR_HAND_TRACKING_AIM_COMPUTED_BIT_FB — Aiming data is computed from additional sources beyond the hand data in the base structure
- XR_HAND_TRACKING_AIM_VALID_BIT_FB — Aiming data is valid
- XR_HAND_TRACKING_AIM_INDEX_PINCHING_BIT_FB — Index finger pinch discrete signal
- XR_HAND_TRACKING_AIM_MIDDLE_PINCHING_BIT_FB — Middle finger pinch discrete signal
- XR_HAND_TRACKING_AIM_RING_PINCHING_BIT_FB — Ring finger pinch discrete signal
- XR_HAND_TRACKING_AIM_LITTLE_PINCHING_BIT_FB — Little finger pinch discrete signal
- XR_HAND_TRACKING_AIM_SYSTEM_GESTURE_BIT_FB — System gesture is active
- XR_HAND_TRACKING_AIM_DOMINANT_HAND_BIT_FB — Hand is currently marked as dominant for the system
- XR_HAND_TRACKING_AIM_MENU_PRESSED_BIT_FB — System menu gesture is active

New Enum Constants

XrStructureType enumeration is extended with:

- XR_TYPE_HAND_TRACKING_AIM_STATE_FB

New Enums

New Structures

XrHandTrackingAimStateFB **can** be provided in the `next` chain of XrHandJointLocationsEXT when calling xrLocateHandJointsEXT to request aiming gesture information associated with this hand.

The XrHandTrackingAimStateFB structure is defined as:

```
// Provided by XR_FB_hand_tracking_aim
typedef struct XrHandTrackingAimStateFB {
    XrStructureType             type;
    void*                       next;
    XrHandTrackingAimFlagsFB    status;
    XrPosef                     aimPose;
    float                       pinchStrengthIndex;
    float                       pinchStrengthMiddle;
    float                       pinchStrengthRing;
    float                       pinchStrengthLittle;
} XrHandTrackingAimStateFB;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `status` is a bitmask of XrHandTrackingAimFlagBitsFB describing the availability and state of other signals.
- `aimPose` is a system-determined "aim" pose, similar in intent and convention to the aim poses used with the action system, based on hand data.
- `pinchStrengthIndex` is the current pinching strength for the index finger of this hand. Range is 0.0 to 1.0, with 1.0 meaning index and thumb are fully touching.
- `pinchStrengthMiddle` is the current pinching strength for the middle finger of this hand. Range is 0.0 to 1.0, with 1.0 meaning middle and thumb are fully touching.
- `pinchStrengthRing` is the current pinching strength for the ring finger of this hand. Range is 0.0 to 1.0, with 1.0 meaning ring and thumb are fully touching.
- `pinchStrengthLittle` is the current pinching strength for the little finger of this hand. Range is 0.0 to 1.0, with 1.0 meaning little and thumb are fully touching.

Valid Usage (Implicit)

- The `XR_FB_hand_tracking_aim` extension **must** be enabled prior to using XrHandTrackingAimStateFB
- `type` **must** be `XR_TYPE_HAND_TRACKING_AIM_STATE_FB`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

New Functions

Issues

Version History

- Revision 1, 2021-07-07 (Federico Schliemann)
  - Initial extension description
- Revision 2, 2022-04-20 (John Kearney)
  - Correct next chain parent for `XrHandTrackingAimStateFB` to `XrHandJointLocationsEXT`

12.111. XR_FB_hand_tracking_capsules

Name String

XR_FB_hand_tracking_capsules

Extension Type

Instance extension

Registered Extension Number

113

Revision

3

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0
and
XR_EXT_hand_tracking

Contributors

Federico Schliemann, Facebook
James Hillery, Facebook
Gloria Kennickell, Facebook

Overview

The XR_EXT_hand_tracking extension provides a list of hand joint poses which include a collision sphere for each joint. However some physics systems prefer to use capsules as a collision stand in for the hands.

This extension allows:

- An application to get a list of capsules that represent the volume of the hand when using the XR_EXT_hand_tracking extension.

New Object Types

New Flag Types

New Enum Constants

- XR_HAND_TRACKING_CAPSULE_POINT_COUNT_FB
  - XR_FB_HAND_TRACKING_CAPSULE_POINT_COUNT was the original name, and is still provided as an alias

for backward compatibility.

- XR_HAND_TRACKING_CAPSULE_COUNT_FB
  - XR_FB_HAND_TRACKING_CAPSULE_COUNT was the original name, and is still provided as an alias for backward compatibility.

XrStructureType enumeration is extended with:

- XR_TYPE_HAND_TRACKING_CAPSULES_STATE_FB

New Enums

New Structures

The XrHandCapsuleFB structure is defined as:

```
// Provided by XR_FB_hand_tracking_capsules
typedef struct XrHandCapsuleFB {
    XrVector3f        points[XR_HAND_TRACKING_CAPSULE_POINT_COUNT_FB];
    float             radius;
    XrHandJointEXT    joint;
} XrHandCapsuleFB;
```

It describes a collision capsule associated with a hand joint.

Member Descriptions

- points are the two points defining the capsule length.
- radius is the radius of the capsule.
- joint is the hand joint that drives this capsule's transform. Multiple capsules **may** be attached to the same joint.

Valid Usage (Implicit)

- The XR_FB_hand_tracking_capsules extension **must** be enabled prior to using XrHandCapsuleFB

XrHandTrackingCapsulesStateFB **can** be provided in the next chain of XrHandJointLocationsEXT when calling xrLocateHandJointsEXT to request collision capsule information associated with this hand.

The XrHandTrackingCapsulesStateFB structure is defined as:

```
// Provided by XR_FB_hand_tracking_capsules
typedef struct XrHandTrackingCapsulesStateFB {
    XrStructureType    type;
    void*              next;
    XrHandCapsuleFB    capsules[XR_HAND_TRACKING_CAPSULE_COUNT_FB];
} XrHandTrackingCapsulesStateFB;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `capsules` is an array of capsules.

Valid Usage (Implicit)

- The `XR_FB_hand_tracking_capsules` extension **must** be enabled prior to using XrHandTrackingCapsulesStateFB
- `type` **must** be `XR_TYPE_HAND_TRACKING_CAPSULES_STATE_FB`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

New Functions

Issues

Version History

- Revision 1, 2021-07-07 (Federico Schliemann)
  - Initial extension description
- Revision 2, 2021-11-18 (Rylie Pavlik, Collabora, Ltd.)
  - Fix typos/naming convention errors: rename `XR_FB_HAND_TRACKING_CAPSULE_POINT_COUNT` to `XR_HAND_TRACKING_CAPSULE_POINT_COUNT_FB` and `XR_FB_HAND_TRACKING_CAPSULE_COUNT` to `XR_HAND_TRACKING_CAPSULE_COUNT_FB`, providing the old names as compatibility aliases.
- Revision 3, 2022-04-20 (John Kearney)
  - Correct next chain parent for `XrHandTrackingCapsulesStateFB` to `XrHandJointLocationsEXT`

12.112. XR_FB_hand_tracking_mesh

Name String

XR_FB_hand_tracking_mesh

Extension Type

Instance extension

Registered Extension Number

111

Revision

3

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0
and
XR_EXT_hand_tracking

Contributors

Federico Schliemann, Facebook
James Hillery, Facebook
Gloria Kennickell, Facebook

Overview

The XR_EXT_hand_tracking extension provides a list of hand joint poses but no mechanism to render a skinned hand mesh.

This extension allows:

- An application to get a skinned hand mesh and a bind pose skeleton that **can** be used to render a hand object driven by the joints from the XR_EXT_hand_tracking extension.
- Control the scale of the hand joints returned by XR_EXT_hand_tracking.

New Object Types

New Flag Types

New Enum Constants

XrStructureType enumeration is extended with:

- XR_TYPE_HAND_TRACKING_MESH_FB
- XR_TYPE_HAND_TRACKING_SCALE_FB

New Enums

New Structures

The XrVector4sFB structure is defined as:

```
// Provided by XR_FB_hand_tracking_mesh
typedef struct XrVector4sFB {
    int16_t    x;
    int16_t    y;
    int16_t    z;
    int16_t    w;
} XrVector4sFB;
```

This is a short integer, four component vector type, used for per-vertex joint indexing for mesh skinning.

Member Descriptions

- x is the x component of the vector.
- y is the y component of the vector.
- z is the z component of the vector.
- w is the w component of the vector.

Valid Usage (Implicit)

- The XR_FB_hand_tracking_mesh extension **must** be enabled prior to using XrVector4sFB

The XrHandTrackingMeshFB structure contains three sets of parallel, application-allocated arrays: one with per-joint data, one with vertex data, and one with index data.

The XrHandTrackingMeshFB structure is defined as:

```
// Provided by XR_FB_hand_tracking_mesh
typedef struct XrHandTrackingMeshFB {
    XrStructureType    type;
    void*              next;
    uint32_t           jointCapacityInput;
    uint32_t           jointCountOutput;
    XrPosef*           jointBindPoses;
    float*             jointRadii;
    XrHandJointEXT*    jointParents;
    uint32_t           vertexCapacityInput;
    uint32_t           vertexCountOutput;
    XrVector3f*        vertexPositions;
    XrVector3f*        vertexNormals;
    XrVector2f*        vertexUVs;
    XrVector4sFB*      vertexBlendIndices;
    XrVector4f*        vertexBlendWeights;
    uint32_t           indexCapacityInput;
    uint32_t           indexCountOutput;
    int16_t*           indices;
} XrHandTrackingMeshFB;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `jointCapacityInput` is the capacity of the joint data arrays in this structure, or `0` to indicate a request to retrieve the required capacity.
- `jointCountOutput` is filled in by the runtime with the count of joint data elements written, or the required capacity in the case that any of `jointCapacityInput`, `vertexCapacityInput`, or `indexCapacityInput` is insufficient.
- `jointBindPoses` is an array of poses that matches what is returned by xrLocateHandJointsEXT which describes the hand skeleton's bind pose.
- `jointRadii` is an array of joint radii at bind pose.
- `jointParents` is an array of joint parents to define a bone hierarchy for the hand skeleton.
- `vertexCapacityInput` is the capacity of the vertex data arrays in this structure, or `0` to indicate a request to retrieve the required capacity.
- `vertexCountOutput` is filled in by the runtime with the count of vertex data elements written, or the required capacity in the case that any of `jointCapacityInput`, `vertexCapacityInput`, or `indexCapacityInput` is insufficient.
- `vertexPositions` is an array of 3D vertex positions.
- `vertexNormals` is an array of 3D vertex normals.
- `vertexUVs` is an array of texture coordinates for this vertex.
- `vertexBlendIndices` is an array of bone blend indices.
- `vertexBlendWeights` is an array of bone blend weights.
- `indexCapacityInput` is the capacity of the index data arrays in this structure, or `0` to indicate a request to retrieve the required capacity.
- `indexCountOutput` is filled in by the runtime with the count of index data elements written, or the required capacity in the case that any of `jointCapacityInput`, `vertexCapacityInput`, or `indexCapacityInput` is insufficient.
- `indices` is an array of triangle indices.
- See the Buffer Size Parameters section for a detailed description of retrieving the array sizes in the "struct form" as used here.

All arrays are application-allocated, and all **may** be `NULL` if any of `jointCapacityInput`, `vertexCapacityInput`, or `indexCapacityInput` is `0`.

The data in a fully-populated XrHandTrackingMeshFB is immutable during the lifetime of the

corresponding XrInstance, and is intended to be retrieved once then used in combination with data changing per-frame retrieved from xrLocateHandJointsEXT.

Valid Usage (Implicit)

- The XR_FB_hand_tracking_mesh extension **must** be enabled prior to using XrHandTrackingMeshFB
- type **must** be XR_TYPE_HAND_TRACKING_MESH_FB
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- If jointCapacityInput is not 0, jointBindPoses **must** be a pointer to an array of jointCapacityInput XrPosef structures
- If jointCapacityInput is not 0, jointRadii **must** be a pointer to an array of jointCapacityInput float values
- If jointCapacityInput is not 0, jointParents **must** be a pointer to an array of jointCapacityInput XrHandJointEXT values
- If vertexCapacityInput is not 0, vertexPositions **must** be a pointer to an array of vertexCapacityInput XrVector3f structures
- If vertexCapacityInput is not 0, vertexNormals **must** be a pointer to an array of vertexCapacityInput XrVector3f structures
- If vertexCapacityInput is not 0, vertexUVs **must** be a pointer to an array of vertexCapacityInput XrVector2f structures
- If vertexCapacityInput is not 0, vertexBlendIndices **must** be a pointer to an array of vertexCapacityInput XrVector4sFB structures
- If vertexCapacityInput is not 0, vertexBlendWeights **must** be a pointer to an array of vertexCapacityInput XrVector4f structures
- If indexCapacityInput is not 0, indices **must** be a pointer to an array of indexCapacityInput int16_t values

XrHandTrackingScaleFB **can** be provided in the next chain of XrHandJointLocationsEXT when calling xrLocateHandJointsEXT to indicate to the runtime that the requested joints need to be scaled to a different size and to query the existing scale value. This is useful in breaking up the overall scale out of the skinning transforms.

The XrHandTrackingScaleFB structure is defined as:

```
// Provided by XR_FB_hand_tracking_mesh
typedef struct XrHandTrackingScaleFB {
    XrStructureType    type;
    void*              next;
    float              sensorOutput;
    float              currentOutput;
    XrBool32           overrideHandScale;
    float              overrideValueInput;
} XrHandTrackingScaleFB;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `sensorOutput` is an output value: the currently measured scale as otherwise applied without passing this structure.
- `currentOutput` is an output value: the effective output that the bind skeleton is getting on the current call, which **may** be subject to filtering, scaling, or validation.
- `overrideHandScale` indicates whether the runtime **must** scale the output of this xrLocateHandJointsEXT call according to `overrideValueInput`
- `overrideValueInput` is an **optional** input value, enabled only when the `overrideHandScale` parameter is set. Setting this to 1.0 and setting `overrideHandScale` to `true` will give the joints in mesh binding scale.

Valid Usage (Implicit)

- The `XR_FB_hand_tracking_mesh` extension **must** be enabled prior to using XrHandTrackingScaleFB
- `type` **must** be `XR_TYPE_HAND_TRACKING_SCALE_FB`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

New Functions

The xrGetHandMeshFB function is defined as:

```
// Provided by XR_FB_hand_tracking_mesh
XrResult xrGetHandMeshFB(
    XrHandTrackerEXT                            handTracker,
    XrHandTrackingMeshFB*                       mesh);
```

Parameter Descriptions

- `handTracker` is the XrHandTrackerEXT that is associated with a particular hand.
- `mesh` is the XrHandTrackingMeshFB output structure.

The xrGetHandMeshFB function populates an XrHandTrackingMeshFB structure with enough information to render a skinned mesh driven by the hand joints. As discussed in the specification for that structure, the data enumerated by this call is constant during the lifetime of an XrInstance.

Valid Usage (Implicit)

- The `XR_FB_hand_tracking_mesh` extension **must** be enabled prior to calling xrGetHandMeshFB
- `handTracker` **must** be a valid XrHandTrackerEXT handle
- `mesh` **must** be a pointer to an XrHandTrackingMeshFB structure

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_FEATURE_UNSUPPORTED`
- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_SIZE_INSUFFICIENT`
- `XR_ERROR_VALIDATION_FAILURE`

Issues

Version History

- Revision 1, 2021-07-07 (Federico Schliemann)
  - Initial extension description
- Revision 2, 2022-04-20 (John Kearney)
  - Correct next chain parent for `XrHandTrackingScaleFB` to `XrHandJointLocationsEXT`
- Revision 3, 2022-07-07 (Rylie Pavlik, Collabora, Ltd.)
  - Correct markup and thus generated valid usage for two-call idiom.

12.113. XR_FB_haptic_amplitude_envelope

Name String

`XR_FB_haptic_amplitude_envelope`

Extension Type

Instance extension

Registered Extension Number

174

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Last Modified Date

2022-06-27

IP Status

No known IP claims.

Contributors

Aanchal Dalmia, Meta
Federico Schliemann, Meta

12.113.1. Overview

This extension enables applications to trigger haptic effect using an Amplitude Envelope buffer.

Trigger haptics

An application can trigger an amplitude envelope haptic effect by creating a XrHapticAmplitudeEnvelopeVibrationFB structure and calling xrApplyHapticFeedback.

The XrHapticAmplitudeEnvelopeVibrationFB structure is defined as:

```
// Provided by XR_FB_haptic_amplitude_envelope
typedef struct XrHapticAmplitudeEnvelopeVibrationFB {
    XrStructureType    type;
    const void*        next;
    XrDuration         duration;
    uint32_t           amplitudeCount;
    const float*       amplitudes;
} XrHapticAmplitudeEnvelopeVibrationFB;
```

This structure describes an amplitude envelope haptic effect.

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `duration` is the duration of the haptic effect in nanoseconds. See Duration for more details.
- `amplitudeCount` is the number of samples in the buffer.
- `amplitudes` is the pointer to a float array that contains the samples.

The runtime **should** resample the provided samples in the `amplitudes`, and maintain an internal buffer which **should** be of XR_MAX_HAPTIC_AMPLITUDE_ENVELOPE_SAMPLES_FB length. The resampling **should** happen based on the `duration`, `amplitudeCount`, and the device's sample rate.

Valid Usage (Implicit)

- The `XR_FB_haptic_amplitude_envelope` extension **must** be enabled prior to using XrHapticAmplitudeEnvelopeVibrationFB
- `type` **must** be `XR_TYPE_HAPTIC_AMPLITUDE_ENVELOPE_VIBRATION_FB`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `amplitudes` **must** be a pointer to an array of `amplitudeCount` `float` values
- The `amplitudeCount` parameter **must** be greater than `0`

New Object Types

New Flag Types

New Enum Constants

- `XR_TYPE_HAPTIC_AMPLITUDE_ENVELOPE_VIBRATION_FB`

New Defines

```
// Provided by XR_FB_haptic_amplitude_envelope
#define XR_MAX_HAPTIC_AMPLITUDE_ENVELOPE_SAMPLES_FB 4000u
```

XR_MAX_HAPTIC_AMPLITUDE_ENVELOPE_SAMPLES_FB defines the maximum number of sample the runtime **should** store in memory.

New Enums

New Structures

- XrHapticAmplitudeEnvelopeVibrationFB

New Functions

Issues

Version History

- Revision 1, 2022-06-27 (Aanchal Dalmia)
  - Initial extension description

12.114. XR_FB_haptic_pcm

Name String

`XR_FB_haptic_pcm`

Extension Type

Instance extension

Registered Extension Number

210

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Last Modified Date

2022-06-27

IP Status

No known IP claims.

Contributors

Aanchal Dalmia, Meta
Adam Bengis, Meta

12.114.1. Overview

This extension enables applications to trigger haptic effects using Pulse Code Modulation (PCM) buffers.

Trigger haptics

An application **can** trigger PCM haptic effect by creating a XrHapticPcmVibrationFB structure and calling xrApplyHapticFeedback.

The XrHapticPcmVibrationFB structure is defined as:

```
// Provided by XR_FB_haptic_pcm
typedef struct XrHapticPcmVibrationFB {
    XrStructureType    type;
    const void*        next;
    uint32_t           bufferSize;
    const float*       buffer;
    float              sampleRate;
    XrBool32           append;
    uint32_t*          samplesConsumed;
} XrHapticPcmVibrationFB;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `bufferSize` is the number of samples in the buffer.
- `buffer` is a pointer to a float array representing the PCM samples. If you consider the haptic effect as a sampled analog audio, then this buffer will contain the samples representing that effect. The values in this buffer are expected to be in the range [-1.0, 1.0].
- `sampleRate` is the number of samples to be played per second, this is used to determine the duration of the haptic effect.
- `append` if set to `XR_FALSE`, any existing samples will be cleared and a new haptic effect will begin, if `XR_TRUE`, samples will be appended to the currently playing effect
- `samplesConsumed` is a pointer to an unsigned integer; it is populated by runtime, to tell the application about how many samples were consumed from the input `buffer`

This structure describes a PCM haptic effect.

The runtime **may** resample the provided samples in the `buffer`, and maintain an internal buffer which **should** be of XR_MAX_HAPTIC_PCM_BUFFER_SIZE_FB length. The resampling **should** happen based on the `sampleRate` and the device's sample rate.

If `append` is `XR_TRUE` and a preceding XrHapticPcmVibrationFB haptic effect on this action has not yet completed, then the runtime **must** finish playing the preceding samples and then play the new haptic effect. If a preceding haptic event on this action has not yet completed, and either the preceding effect is not an XrHapticPcmVibrationFB haptic effect or `append` is `XR_FALSE`, the runtime **must** cancel the preceding incomplete effects on that action and start playing the new haptic effect, as usual for the core specification.

When `append` is true and a preceding XrHapticPcmVibrationFB haptic effect on this action has not yet

completed, then the application can provide a different sampleRate in the new haptic effect.

The runtime **must** populate the samplesConsumed with the count of the samples from buffer which were consumed. The samplesConsumed is populated before the xrApplyHapticFeedback returns.

Valid Usage (Implicit)

- The XR_FB_haptic_pcm extension **must** be enabled prior to using XrHapticPcmVibrationFB
- type **must** be XR_TYPE_HAPTIC_PCM_VIBRATION_FB
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- buffer **must** be a pointer to an array of bufferSize float values
- samplesConsumed **must** be a pointer to a uint32_t value
- The bufferSize parameter **must** be greater than 0

Get the device sample rate

An application **can** use the xrGetDeviceSampleRateFB function to get the sample rate of the currently bound device on which the haptic action is triggered. If the application does not want any resampling to occur, then it can use this function to know the currently bound device sample rate, and pass that value in sampleRate of XrHapticPcmVibrationFB.

```
// Provided by XR_FB_haptic_pcm
XrResult xrGetDeviceSampleRateFB(
    XrSession                                   session,
    const XrHapticActionInfo*                   hapticActionInfo,
    XrDevicePcmSampleRateGetInfoFB*             deviceSampleRate);
```

Parameter Descriptions

- session is the specified XrSession.
- hapticActionInfo is the XrHapticActionInfo used to provide action and subaction paths
- deviceSampleRate is a pointer to XrDevicePcmSampleRateStateFB which is populated by the runtime.

The runtime **must** use the hapticActionInfo to get the sample rate of the currently bound device on which haptics is triggered and populate the deviceSampleRate structure. The device is determined by the XrHapticActionInfo::action and XrHapticActionInfo::subactionPath. If the hapticActionInfo is bound to more than one device, then runtime **should** assume that the all these bound devices have the same deviceSampleRate and the runtime **should** return the sampleRate for any of those bound devices. If the device is invalid, the runtime **must** populate the deviceSampleRate of XrDevicePcmSampleRateStateFB

as 0. A device can be invalid if the runtime does not find any device (which can play haptics) connected to the headset, or if the device does not support PCM haptic effect.

Valid Usage (Implicit)

- The XR_FB_haptic_pcm extension **must** be enabled prior to calling xrGetDeviceSampleRateFB
- session **must** be a valid XrSession handle
- hapticActionInfo **must** be a pointer to a valid XrHapticActionInfo structure
- deviceSampleRate **must** be a pointer to an XrDevicePcmSampleRateGetInfoFB structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_ACTIONSET_NOT_ATTACHED
- XR_ERROR_ACTION_TYPE_MISMATCH
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_PATH_INVALID
- XR_ERROR_PATH_UNSUPPORTED
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The XrDevicePcmSampleRateStateFB structure is defined as:

```
// Provided by XR_FB_haptic_pcm
typedef struct XrDevicePcmSampleRateStateFB {
    XrStructureType    type;
    void*              next;
    float              sampleRate;
} XrDevicePcmSampleRateStateFB;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `sampleRate` is the sample rate of the currently bound device which can play a haptic effect

Valid Usage (Implicit)

- The `XR_FB_haptic_pcm` extension **must** be enabled prior to using XrDevicePcmSampleRateStateFB
- `type` **must** be `XR_TYPE_DEVICE_PCM_SAMPLE_RATE_STATE_FB`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

New Object Types

New Flag Types

New Enum Constants

XrStructureType enumeration is extended with:

- `XR_TYPE_HAPTIC_PCM_VIBRATION_FB`
- `XR_TYPE_DEVICE_PCM_SAMPLE_RATE_STATE_FB`

New Defines

```
// Provided by XR_FB_haptic_pcm
#define XR_MAX_HAPTIC_PCM_BUFFER_SIZE_FB 4000
```

XR_MAX_HAPTIC_PCM_BUFFER_SIZE_FB defines the maximum number of samples the runtime can store.

New Enums

New Structures

- XrHapticPcmVibrationFB
- XrDevicePcmSampleRateStateFB

New Functions

- xrGetDeviceSampleRateFB

Issues

Version History

- Revision 1, 2022-06-27 (Aanchal Dalmia)
  - Initial extension description

12.115. XR_FB_keyboard_tracking

Name String

`XR_FB_keyboard_tracking`

Extension Type

Instance extension

Registered Extension Number

117

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Contributors

Federico Schliemann, Facebook
Robert Memmott, Facebook
Cass Everitt, Facebook

Overview

This extension allows the application to query the system for a supported trackable keyboard type and obtain an XrSpace handle to track it. It also provides relevant metadata about the keyboard itself, including bounds and a human readable identifier.

New Object Types

New Flag Types

```
// Provided by XR_FB_keyboard_tracking
typedef XrFlags64 XrKeyboardTrackingFlagsFB;
```

```
// Provided by XR_FB_keyboard_tracking
// Flag bits for XrKeyboardTrackingFlagsFB
static const XrKeyboardTrackingFlagsFB XR_KEYBOARD_TRACKING_EXISTS_BIT_FB = 0x00000001;
static const XrKeyboardTrackingFlagsFB XR_KEYBOARD_TRACKING_LOCAL_BIT_FB = 0x00000002;
static const XrKeyboardTrackingFlagsFB XR_KEYBOARD_TRACKING_REMOTE_BIT_FB = 0x00000004;
static const XrKeyboardTrackingFlagsFB XR_KEYBOARD_TRACKING_CONNECTED_BIT_FB =
0x00000008;
```

Flag Descriptions

- XR_KEYBOARD_TRACKING_EXISTS_BIT_FB — indicates that the system has a physically tracked keyboard to report. If not set then no other bits should be considered to be valid or meaningful. If set either XR_KEYBOARD_TRACKING_LOCAL_BIT_FB or XR_KEYBOARD_TRACKING_REMOTE_BIT_FB must also be set.
- XR_KEYBOARD_TRACKING_LOCAL_BIT_FB — indicates that the physically tracked keyboard is intended to be used in a local pairing with the system. Mutually exclusive with XR_KEYBOARD_TRACKING_REMOTE_BIT_FB.
- XR_KEYBOARD_TRACKING_REMOTE_BIT_FB — indicates that the physically tracked keyboard is intended to be used while paired to a separate remote computing device. Mutually exclusive with XR_KEYBOARD_TRACKING_LOCAL_BIT_FB.
- XR_KEYBOARD_TRACKING_CONNECTED_BIT_FB — indicates that the physically tracked keyboard is actively connected to the headset and capable of sending key data

```
// Provided by XR_FB_keyboard_tracking
typedef XrFlags64 XrKeyboardTrackingQueryFlagsFB;
```

```
// Provided by XR_FB_keyboard_tracking
// Flag bits for XrKeyboardTrackingQueryFlagsFB
static const XrKeyboardTrackingQueryFlagsFB XR_KEYBOARD_TRACKING_QUERY_LOCAL_BIT_FB =
0x00000002;
static const XrKeyboardTrackingQueryFlagsFB XR_KEYBOARD_TRACKING_QUERY_REMOTE_BIT_FB =
0x00000004;
```

Flag Descriptions

- XR_KEYBOARD_TRACKING_QUERY_LOCAL_BIT_FB — indicates the query is for the physically tracked keyboard that is intended to be used in a local pairing with the System. Mutually exclusive with XR_KEYBOARD_TRACKING_QUERY_REMOTE_BIT_FB.
- XR_KEYBOARD_TRACKING_QUERY_REMOTE_BIT_FB — indicates the query is for the physically tracked keyboard that may be connected to a separate remote computing device. Mutually exclusive with XR_KEYBOARD_TRACKING_QUERY_LOCAL_BIT_FB.

New Enum Constants

- XR_MAX_KEYBOARD_TRACKING_NAME_SIZE_FB

XrStructureType enumeration is extended with:

- XR_TYPE_KEYBOARD_SPACE_CREATE_INFO_FB
- XR_TYPE_KEYBOARD_TRACKING_QUERY_FB
- XR_TYPE_SYSTEM_KEYBOARD_TRACKING_PROPERTIES_FB

New Enums

New Structures

The XrSystemKeyboardTrackingPropertiesFB structure is defined as:

```
// Provided by XR_FB_keyboard_tracking
typedef struct XrSystemKeyboardTrackingPropertiesFB {
    XrStructureType    type;
    void*              next;
    XrBool32           supportsKeyboardTracking;
} XrSystemKeyboardTrackingPropertiesFB;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain.
- supportsKeyboardTracking defines whether the system supports the tracked keyboard feature.

XrSystemKeyboardTrackingPropertiesFB is populated with information from the system about tracked keyboard support.

Valid Usage (Implicit)

- The XR_FB_keyboard_tracking extension **must** be enabled prior to using XrSystemKeyboardTrackingPropertiesFB
- type **must** be XR_TYPE_SYSTEM_KEYBOARD_TRACKING_PROPERTIES_FB
- next **must** be NULL or a valid pointer to the next structure in a structure chain

The XrKeyboardTrackingQueryFB structure is defined as:

```
// Provided by XR_FB_keyboard_tracking
typedef struct XrKeyboardTrackingQueryFB {
    XrStructureType                   type;
    void*                             next;
    XrKeyboardTrackingQueryFlagsFB    flags;
} XrKeyboardTrackingQueryFB;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain.
- flags is a bitmask of XrKeyboardTrackingQueryFlagsFB.

XrKeyboardTrackingQueryFB specifies input data needed to determine which type of tracked keyboard to query for.

Valid Usage (Implicit)

- The XR_FB_keyboard_tracking extension **must** be enabled prior to using XrKeyboardTrackingQueryFB
- type **must** be XR_TYPE_KEYBOARD_TRACKING_QUERY_FB
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- flags **must** be a valid combination of XrKeyboardTrackingQueryFlagBitsFB values
- flags **must** not be 0

The XrKeyboardTrackingDescriptionFB structure is defined as:

```
// Provided by XR_FB_keyboard_tracking
typedef struct XrKeyboardTrackingDescriptionFB {
    uint64_t                    trackedKeyboardId;
    XrVector3f                  size;
    XrKeyboardTrackingFlagsFB   flags;
    char                        name[XR_MAX_KEYBOARD_TRACKING_NAME_SIZE_FB];
} XrKeyboardTrackingDescriptionFB;
```

Member Descriptions

- `trackedKeyboardId` abstract identifier describing the type of keyboard.
- `size` bounding box.
- `flags` additional information on the type of keyboard available. If `XR_KEYBOARD_TRACKING_EXISTS_BIT_FB` is not set there is no keyboard.
- `name` human readable keyboard identifier.

XrKeyboardTrackingDescriptionFB describes a trackable keyboard and its associated metadata.

Valid Usage (Implicit)

- The `XR_FB_keyboard_tracking` extension **must** be enabled prior to using XrKeyboardTrackingDescriptionFB

The XrKeyboardSpaceCreateInfoFB structure is defined as:

```
// Provided by XR_FB_keyboard_tracking
typedef struct XrKeyboardSpaceCreateInfoFB {
    XrStructureType    type;
    void*              next;
    uint64_t           trackedKeyboardId;
} XrKeyboardSpaceCreateInfoFB;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `trackedKeyboardId` abstract identifier describing the type of keyboard to track.

XrKeyboardSpaceCreateInfoFB describes a request for the system needed to create a trackable XrSpace associated with the keyboard.

Valid Usage (Implicit)

- The XR_FB_keyboard_tracking extension **must** be enabled prior to using XrKeyboardSpaceCreateInfoFB
- type **must** be XR_TYPE_KEYBOARD_SPACE_CREATE_INFO_FB
- next **must** be NULL or a valid pointer to the next structure in a structure chain

New Functions

The xrQuerySystemTrackedKeyboardFB function is defined as:

```
// Provided by XR_FB_keyboard_tracking
XrResult xrQuerySystemTrackedKeyboardFB(
    XrSession                                   session,
    const XrKeyboardTrackingQueryFB*            queryInfo,
    XrKeyboardTrackingDescriptionFB*            keyboard);
```

Parameter Descriptions

- session is the session that will be associated with a keyboard space.
- queryInfo is the XrKeyboardTrackingQueryFB that describes the type of keyboard to return. queryInfo must have either XR_KEYBOARD_TRACKING_QUERY_LOCAL_BIT_FB or XR_KEYBOARD_TRACKING_QUERY_REMOTE_BIT_FB set.
- keyboard is the XrKeyboardTrackingDescriptionFB output structure.

The xrQuerySystemTrackedKeyboardFB function populates an XrKeyboardTrackingDescriptionFB structure with enough information to describe a keyboard that the system can locate.

Valid Usage (Implicit)

- The XR_FB_keyboard_tracking extension **must** be enabled prior to calling xrQuerySystemTrackedKeyboardFB
- session **must** be a valid XrSession handle
- queryInfo **must** be a pointer to a valid XrKeyboardTrackingQueryFB structure
- keyboard **must** be a pointer to an XrKeyboardTrackingDescriptionFB structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FEATURE_UNSUPPORTED
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The xrCreateKeyboardSpaceFB function is defined as:

```
// Provided by XR_FB_keyboard_tracking
XrResult xrCreateKeyboardSpaceFB(
    XrSession                                   session,
    const XrKeyboardSpaceCreateInfoFB*          createInfo,
    XrSpace*                                    keyboardSpace);
```

Parameter Descriptions

- session is the session that will be associated with the returned keyboard space.
- createInfo is the XrKeyboardSpaceCreateInfoFB that describes the type of keyboard to track.
- keyboardSpace is the XrSpace output structure.

The xrCreateKeyboardSpaceFB function returns an XrSpace that can be used to locate a physical keyboard in space. The origin of the created XrSpace is located in the center of the bounding box in the x and z axes, and at the top of the y axis (meaning the keyboard is located entirely in negative y).

Valid Usage (Implicit)

- The XR_FB_keyboard_tracking extension **must** be enabled prior to calling xrCreateKeyboardSpaceFB
- session **must** be a valid XrSession handle
- createInfo **must** be a pointer to a valid XrKeyboardSpaceCreateInfoFB structure
- keyboardSpace **must** be a pointer to an XrSpace handle

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FEATURE_UNSUPPORTED
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

Issues

Version History

- Revision 1, 2021-08-27 (Federico Schliemann)
  - Initial extension description

12.116. XR_FB_passthrough

Name String

XR_FB_passthrough

Extension Type

Instance extension

Registered Extension Number

119

Revision

5

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Contributors

Anton Vaneev, Meta Platforms
Cass Everitt, Meta Platforms
Federico Schliemann, Meta Platforms
Johannes Schmid, Meta Platforms
Ante Trbojevic, Meta Platforms

Overview

Passthrough is a way to show a user their physical environment in a light-blocking VR headset. Applications may use passthrough in a multitude of ways, including:

- Creating AR-like experiences, where virtual objects augment the user's environment.
- Bringing real objects into a VR experience.
- Mapping the playspace such that a VR experience is customized to it.

This extension allows:

- An application to request passthrough to be composited with the application content.
- An application to specify the compositing and blending rules between passthrough and VR content.
- An application to apply styles, such as color mapping and edge rendering, to passthrough.
- An application to provide a geometry to be used in place of the user's physical environment. Camera images will be projected onto the surface provided by the application. In some cases where a part of the environment, such as a desk, can be approximated well, this provides better visual experience.

New Object Types

```
// Provided by XR_FB_passthrough
XR_DEFINE_HANDLE(XrPassthroughFB)
```

XrPassthroughFB represents a passthrough feature.

```
// Provided by XR_FB_passthrough
XR_DEFINE_HANDLE(XrPassthroughLayerFB)
```

XrPassthroughLayerFB represents a layer of passthrough content.

```
// Provided by XR_FB_passthrough
XR_DEFINE_HANDLE(XrGeometryInstanceFB)
```

XrGeometryInstanceFB represents a geometry instance used in a passthrough layer.

New Flag Types

```
// Provided by XR_FB_passthrough
typedef XrFlags64 XrPassthroughFlagsFB;
```

Specify additional creation behavior.

```
// Provided by XR_FB_passthrough
// Flag bits for XrPassthroughFlagsFB
static const XrPassthroughFlagsFB XR_PASSTHROUGH_IS_RUNNING_AT_CREATION_BIT_FB =
0x00000001;
static const XrPassthroughFlagsFB XR_PASSTHROUGH_LAYER_DEPTH_BIT_FB = 0x00000002;
```

Flag Descriptions

- XR_PASSTHROUGH_IS_RUNNING_AT_CREATION_BIT_FB — The object (passthrough, layer) is running at creation.
- XR_PASSTHROUGH_LAYER_DEPTH_BIT_FB — The passthrough system sends depth information to the compositor. Only applicable to layer objects.

```
// Provided by XR_FB_passthrough
typedef XrFlags64 XrPassthroughStateChangedFlagsFB;
```

Specify additional state change behavior.

```
// Provided by XR_FB_passthrough
// Flag bits for XrPassthroughStateChangedFlagsFB
static const XrPassthroughStateChangedFlagsFB
XR_PASSTHROUGH_STATE_CHANGED_REINIT_REQUIRED_BIT_FB = 0x00000001;
static const XrPassthroughStateChangedFlagsFB
XR_PASSTHROUGH_STATE_CHANGED_NON_RECOVERABLE_ERROR_BIT_FB = 0x00000002;
static const XrPassthroughStateChangedFlagsFB
XR_PASSTHROUGH_STATE_CHANGED_RECOVERABLE_ERROR_BIT_FB = 0x00000004;
static const XrPassthroughStateChangedFlagsFB
XR_PASSTHROUGH_STATE_CHANGED_RESTORED_ERROR_BIT_FB = 0x00000008;
```

Flag Descriptions

- XR_PASSTHROUGH_STATE_CHANGED_REINIT_REQUIRED_BIT_FB — Passthrough system requires reinitialization.
- XR_PASSTHROUGH_STATE_CHANGED_NON_RECOVERABLE_ERROR_BIT_FB — Non-recoverable error has occurred. A device reboot or a firmware update may be required.
- XR_PASSTHROUGH_STATE_CHANGED_RECOVERABLE_ERROR_BIT_FB — A recoverable error has occurred. The runtime will attempt to recover, but some functionality may be temporarily unavailable.
- XR_PASSTHROUGH_STATE_CHANGED_RESTORED_ERROR_BIT_FB — The runtime has recovered from a previous error and is functioning normally.

```
// Provided by XR_FB_passthrough
typedef XrFlags64 XrPassthroughCapabilityFlagsFB;
```

Specify passthrough system capabilities.

```
// Provided by XR_FB_passthrough
// Flag bits for XrPassthroughCapabilityFlagsFB
static const XrPassthroughCapabilityFlagsFB XR_PASSTHROUGH_CAPABILITY_BIT_FB =
0x00000001;
static const XrPassthroughCapabilityFlagsFB XR_PASSTHROUGH_CAPABILITY_COLOR_BIT_FB =
0x00000002;
static const XrPassthroughCapabilityFlagsFB XR_PASSTHROUGH_CAPABILITY_LAYER_DEPTH_BIT_FB
= 0x00000004;
```

Flag Descriptions

- XR_PASSTHROUGH_CAPABILITY_BIT_FB — The system supports passthrough.
- XR_PASSTHROUGH_CAPABILITY_COLOR_BIT_FB — The system can show passthrough with realistic colors. XR_PASSTHROUGH_CAPABILITY_BIT_FB **must** be set if XR_PASSTHROUGH_CAPABILITY_COLOR_BIT_FB is set.
- XR_PASSTHROUGH_CAPABILITY_LAYER_DEPTH_BIT_FB — The system supports passthrough layers composited using depth testing. XR_PASSTHROUGH_CAPABILITY_BIT_FB **must** be set if XR_PASSTHROUGH_CAPABILITY_LAYER_DEPTH_BIT_FB is set.

New Enum Constants

- XR_PASSTHROUGH_COLOR_MAP_MONO_SIZE_FB

XrStructureType enumeration is extended with:

- XR_TYPE_SYSTEM_PASSTHROUGH_PROPERTIES_FB
- XR_TYPE_PASSTHROUGH_CREATE_INFO_FB
- XR_TYPE_PASSTHROUGH_LAYER_CREATE_INFO_FB
- XR_TYPE_COMPOSITION_LAYER_PASSTHROUGH_FB
- XR_TYPE_GEOMETRY_INSTANCE_CREATE_INFO_FB
- XR_TYPE_GEOMETRY_INSTANCE_TRANSFORM_FB
- XR_TYPE_PASSTHROUGH_STYLE_FB
- XR_TYPE_PASSTHROUGH_COLOR_MAP_MONO_TO_RGBA_FB
- XR_TYPE_PASSTHROUGH_COLOR_MAP_MONO_TO_MONO_FB
- XR_TYPE_PASSTHROUGH_BRIGHTNESS_CONTRAST_SATURATION_FB
- XR_TYPE_EVENT_DATA_PASSTHROUGH_STATE_CHANGED_FB

XrResult enumeration is extended with:

- XR_ERROR_UNEXPECTED_STATE_PASSTHROUGH_FB The state of an object for which a function is called is not one of the expected states for that function.
- XR_ERROR_FEATURE_ALREADY_CREATED_PASSTHROUGH_FB An application attempted to create a feature when one has already been created and only one can exist.
- XR_ERROR_FEATURE_REQUIRED_PASSTHROUGH_FB A feature is required before the function can be called.
- XR_ERROR_NOT_PERMITTED_PASSTHROUGH_FB Operation is not permitted.
- XR_ERROR_INSUFFICIENT_RESOURCES_PASSTHROUGH_FB The runtime does not have sufficient resources to perform the operation. Either the object being created is too large, or too many objects of a specific

kind have been created.

New Enums

Specify the kind of passthrough behavior the layer provides.

```
// Provided by XR_FB_passthrough
typedef enum XrPassthroughLayerPurposeFB {
    XR_PASSTHROUGH_LAYER_PURPOSE_RECONSTRUCTION_FB = 0,
    XR_PASSTHROUGH_LAYER_PURPOSE_PROJECTED_FB = 1,
  // Provided by XR_FB_passthrough_keyboard_hands
    XR_PASSTHROUGH_LAYER_PURPOSE_TRACKED_KEYBOARD_HANDS_FB = 1000203001,
  // Provided by XR_FB_passthrough_keyboard_hands
    XR_PASSTHROUGH_LAYER_PURPOSE_TRACKED_KEYBOARD_MASKED_HANDS_FB = 1000203002,
    XR_PASSTHROUGH_LAYER_PURPOSE_MAX_ENUM_FB = 0x7FFFFFFF
} XrPassthroughLayerPurposeFB;
```

Enumerant Descriptions

- XR_PASSTHROUGH_LAYER_PURPOSE_RECONSTRUCTION_FB — Reconstruction passthrough (full screen environment)
- XR_PASSTHROUGH_LAYER_PURPOSE_PROJECTED_FB — Projected passthrough (using a custom surface)
- XR_PASSTHROUGH_LAYER_PURPOSE_TRACKED_KEYBOARD_HANDS_FB — Passthrough layer purpose for keyboard hands presence. (Added by the XR_FB_passthrough_keyboard_hands extension)
- XR_PASSTHROUGH_LAYER_PURPOSE_TRACKED_KEYBOARD_MASKED_HANDS_FB — Passthrough layer purpose for keyboard hands presence with keyboard masked hand transitions (i.e passthrough hands rendered only when they are over the keyboard). (Added by the XR_FB_passthrough_keyboard_hands extension)

New Structures

The XrSystemPassthroughPropertiesFB structure is defined as:

```
// Provided by XR_FB_passthrough
typedef struct XrSystemPassthroughPropertiesFB {
    XrStructureType    type;
    const void*        next;
    XrBool32           supportsPassthrough;
} XrSystemPassthroughPropertiesFB;
```

It describes a passthrough system property.

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension. For more details about the structure chain, see the structure being extended (XrSystemProperties).
- `supportsPassthrough` defines whether the system supports the passthrough feature.

Valid Usage (Implicit)

- The `XR_FB_passthrough` extension **must** be enabled prior to using XrSystemPassthroughPropertiesFB
- `type` **must** be `XR_TYPE_SYSTEM_PASSTHROUGH_PROPERTIES_FB`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

New Structures

The XrSystemPassthroughProperties2FB structure is defined as:

```
// Provided by XR_FB_passthrough
typedef struct XrSystemPassthroughProperties2FB {
    XrStructureType                   type;
    const void*                       next;
    XrPassthroughCapabilityFlagsFB    capabilities;
} XrSystemPassthroughProperties2FB;
```

Applications **can** pass this structure in a call to xrGetSystemProperties to query passthrough system properties. Applications **should** verify that the runtime implements `XR_FB_passthrough` spec version 3 or newer before doing so. In older versions, this structure is not supported and will be left unpopulated. Applications **should** use XrSystemPassthroughPropertiesFB in that case.

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension. For more details about the structure chain, see the structure being extended (XrSystemProperties).
- `capabilities` defines a set of features supported by the passthrough system.

Valid Usage (Implicit)

- The XR_FB_passthrough extension **must** be enabled prior to using XrSystemPassthroughProperties2FB
- type **must** be XR_TYPE_SYSTEM_PASSTHROUGH_PROPERTIES2_FB
- next **must** be NULL or a valid pointer to the next structure in a structure chain

The XrPassthroughCreateInfoFB structure is defined as:

```
// Provided by XR_FB_passthrough
typedef struct XrPassthroughCreateInfoFB {
    XrStructureType         type;
    const void*             next;
    XrPassthroughFlagsFB    flags;
} XrPassthroughCreateInfoFB;
```

It contains parameters used to specify a new passthrough feature.

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- flags is a bitmask of XrPassthroughFlagBitsFB that specify additional behavior.

Valid Usage (Implicit)

- The XR_FB_passthrough extension **must** be enabled prior to using XrPassthroughCreateInfoFB
- type **must** be XR_TYPE_PASSTHROUGH_CREATE_INFO_FB
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- flags **must** be 0 or a valid combination of XrPassthroughFlagBitsFB values

The XrPassthroughLayerCreateInfoFB structure is defined as:

```
// Provided by XR_FB_passthrough
typedef struct XrPassthroughLayerCreateInfoFB {
    XrStructureType                type;
    const void*                    next;
    XrPassthroughFB                passthrough;
    XrPassthroughFlagsFB           flags;
    XrPassthroughLayerPurposeFB    purpose;
} XrPassthroughLayerCreateInfoFB;
```

It contains parameters used to specify a new passthrough layer.

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `passthrough` an XrPassthroughFB handle.
- `flags` XrPassthroughFlagsFB that specify additional behavior.
- `purpose` XrPassthroughLayerPurposeFB that specifies the layer's purpose.

Valid Usage (Implicit)

- The `XR_FB_passthrough` extension **must** be enabled prior to using XrPassthroughLayerCreateInfoFB
- `type` **must** be `XR_TYPE_PASSTHROUGH_LAYER_CREATE_INFO_FB`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `passthrough` **must** be a valid XrPassthroughFB handle
- `flags` **must** be `0` or a valid combination of XrPassthroughFlagBitsFB values
- `purpose` **must** be a valid XrPassthroughLayerPurposeFB value

The XrCompositionLayerPassthroughFB structure is defined as:

```
// Provided by XR_FB_passthrough
typedef struct XrCompositionLayerPassthroughFB {
    XrStructureType            type;
    const void*                next;
    XrCompositionLayerFlags    flags;
    XrSpace                    space;
    XrPassthroughLayerFB       layerHandle;
} XrCompositionLayerPassthroughFB;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `flags` is a bitmask of XrCompositionLayerFlagBits that specify additional behavior.
- `space` is the XrSpace that specifies the layer's space - **must** be XR_NULL_HANDLE.
- `layerHandle` is the XrPassthroughLayerFB that defines this layer's behavior.

It is a composition layer type that may be submitted in xrEndFrame where an XrCompositionLayerBaseHeader is specified, as a stand-in for the actual passthrough contents.

Errata: the third field of this structure is named `flags` rather than `layerFlags` as expected and as documented for for the parent type XrCompositionLayerBaseHeader.

Valid Usage (Implicit)

- The `XR_FB_passthrough` extension **must** be enabled prior to using XrCompositionLayerPassthroughFB
- `type` **must** be `XR_TYPE_COMPOSITION_LAYER_PASSTHROUGH_FB`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `flags` **must** be a valid combination of XrCompositionLayerFlagBits values
- `flags` **must** not be `0`
- If `space` is not XR_NULL_HANDLE, `space` **must** be a valid XrSpace handle
- `layerHandle` **must** be a valid XrPassthroughLayerFB handle
- Both of `layerHandle` and `space` that are valid handles **must** have been created, allocated, or retrieved from the same XrSession

The XrGeometryInstanceCreateInfoFB structure is defined as:

```
// Provided by XR_FB_passthrough
typedef struct XrGeometryInstanceCreateInfoFB {
    XrStructureType         type;
    const void*             next;
    XrPassthroughLayerFB    layer;
    XrTriangleMeshFB        mesh;
    XrSpace                 baseSpace;
    XrPosef                 pose;
    XrVector3f              scale;
} XrGeometryInstanceCreateInfoFB;
```

It contains parameters to specify a new geometry instance.

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `layer` is the XrPassthroughLayerFB.
- `mesh` is the XrTriangleMeshFB.
- `baseSpace` is the XrSpace that defines the geometry instance's base space for transformations.
- `pose` is the XrPosef that defines the geometry instance's pose.
- `scale` is the XrVector3f that defines the geometry instance's scale.

Valid Usage (Implicit)

- The `XR_FB_passthrough` extension **must** be enabled prior to using XrGeometryInstanceCreateInfoFB
- `type` **must** be `XR_TYPE_GEOMETRY_INSTANCE_CREATE_INFO_FB`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `layer` **must** be a valid XrPassthroughLayerFB handle
- `mesh` **must** be a valid XrTriangleMeshFB handle
- `baseSpace` **must** be a valid XrSpace handle
- Each of `baseSpace`, `layer`, and `mesh` **must** have been created, allocated, or retrieved from the same XrSession

The XrGeometryInstanceTransformFB structure is defined as:

```
// Provided by XR_FB_passthrough
typedef struct XrGeometryInstanceTransformFB {
    XrStructureType    type;
    const void*        next;
    XrSpace            baseSpace;
    XrTime             time;
    XrPosef            pose;
    XrVector3f         scale;
} XrGeometryInstanceTransformFB;
```

It describes a transformation for a geometry instance.

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `baseSpace` is the XrSpace that defines the geometry instance's base space for transformations.
- `time` is the `XrTime` that define the time at which the transform is applied.
- `pose` is the XrPosef that defines the geometry instance's pose.
- `scale` is the XrVector3f that defines the geometry instance's scale.

Valid Usage (Implicit)

- The `XR_FB_passthrough` extension **must** be enabled prior to using XrGeometryInstanceTransformFB
- `type` **must** be `XR_TYPE_GEOMETRY_INSTANCE_TRANSFORM_FB`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `baseSpace` **must** be a valid XrSpace handle

The XrPassthroughStyleFB structure is defined as:

```
// Provided by XR_FB_passthrough
typedef struct XrPassthroughStyleFB {
    XrStructureType    type;
    const void*        next;
    float              textureOpacityFactor;
    XrColor4f          edgeColor;
} XrPassthroughStyleFB;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `textureOpacityFactor` is the opacity of the passthrough imagery in the range [0, 1].
- `edgeColor` is the XrColor4f that defines the edge rendering color. Edges are detected in the original passthrough imagery and rendered on top of it. Edge rendering is disabled when the alpha value of `edgeColor` is zero.

XrPassthroughStyleFB lets applications customize the appearance of passthrough layers. In addition to the parameters specified here, applications **may** add one of the following structures to the structure chain: XrPassthroughColorMapMonoToRgbaFB, XrPassthroughColorMapMonoToMonoFB, XrPassthroughBrightnessContrastSaturationFB. These structures are mutually exclusive. The runtime **must** return `XR_ERROR_VALIDATION_FAILURE` if more than one of them are present in the structure chain.

Valid Usage (Implicit)

- The `XR_FB_passthrough` extension **must** be enabled prior to using XrPassthroughStyleFB
- `type` **must** be `XR_TYPE_PASSTHROUGH_STYLE_FB`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain. See also: XrPassthroughBrightnessContrastSaturationFB, XrPassthroughColorMapInterpolatedLutMETA, XrPassthroughColorMapLutMETA, XrPassthroughColorMapMonoToMonoFB, XrPassthroughColorMapMonoToRgbaFB

The XrPassthroughColorMapMonoToRgbaFB structure is defined as:

```
// Provided by XR_FB_passthrough
typedef struct XrPassthroughColorMapMonoToRgbaFB {
    XrStructureType    type;
    const void*        next;
    XrColor4f          textureColorMap[XR_PASSTHROUGH_COLOR_MAP_MONO_SIZE_FB];
} XrPassthroughColorMapMonoToRgbaFB;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `textureColorMap` is an array of XrColor4f colors to which the passthrough imagery luminance values are mapped.

XrPassthroughColorMapMonoToRgbaFB lets applications define a map which replaces each input luminance value in the passthrough imagery with an RGBA color value. The map is applied before any additional effects (such as edges) are rendered on top.

XrPassthroughColorMapMonoToRgbaFB is provided in the `next` chain of XrPassthroughStyleFB.

Valid Usage (Implicit)

- The `XR_FB_passthrough` extension **must** be enabled prior to using XrPassthroughColorMapMonoToRgbaFB
- `type` **must** be `XR_TYPE_PASSTHROUGH_COLOR_MAP_MONO_TO_RGBA_FB`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

The XrPassthroughColorMapMonoToMonoFB structure is defined as:

```
// Provided by XR_FB_passthrough
typedef struct XrPassthroughColorMapMonoToMonoFB {
    XrStructureType    type;
    const void*        next;
    uint8_t            textureColorMap[XR_PASSTHROUGH_COLOR_MAP_MONO_SIZE_FB];
} XrPassthroughColorMapMonoToMonoFB;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `textureColorMap` is an array of `uint8_t` grayscale color values to which the passthrough luminance values are mapped.

XrPassthroughColorMapMonoToMonoFB lets applications define a map which replaces each input luminance value in the passthrough imagery with a grayscale color value defined in `textureColorMap`. The map is applied before any additional effects (such as edges) are rendered on top.

XrPassthroughColorMapMonoToMonoFB is provided in the `next` chain of XrPassthroughStyleFB.

Valid Usage (Implicit)

- The `XR_FB_passthrough` extension **must** be enabled prior to using XrPassthroughColorMapMonoToMonoFB
- `type` **must** be `XR_TYPE_PASSTHROUGH_COLOR_MAP_MONO_TO_MONO_FB`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

The XrPassthroughBrightnessContrastSaturationFB structure is defined as:

```
// Provided by XR_FB_passthrough
typedef struct XrPassthroughBrightnessContrastSaturationFB {
    XrStructureType    type;
    const void*        next;
    float              brightness;
    float              contrast;
    float              saturation;
} XrPassthroughBrightnessContrastSaturationFB;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `brightness` is the brightness adjustment value in the range [-100, 100]. The neutral element is 0.
- `contrast` is the contrast adjustment value in the range [0, Infinity]. The neutral element is 1.
- `saturation` is the saturation adjustment value in the range [0, Infinity]. The neutral element is 1.

XrPassthroughBrightnessContrastSaturationFB lets applications adjust the brightness, contrast, and saturation of passthrough layers. The adjustments only are applied before any additional effects (such as edges) are rendered on top.

The adjustments are applied in CIELAB color space (white point D65) using the following formulas:

- L*' = clamp((L* - 50) × contrast + 50, 0, 100)
- L*'' = clamp(L*' + brightness, 0, 100)
- (a*', b*') = (a*, b*) × saturation
- Resulting color: (L*'', a*', b*')

XrPassthroughBrightnessContrastSaturationFB is provided in the `next` chain of XrPassthroughStyleFB.

Valid Usage (Implicit)

- The `XR_FB_passthrough` extension **must** be enabled prior to using XrPassthroughBrightnessContrastSaturationFB
- `type` **must** be `XR_TYPE_PASSTHROUGH_BRIGHTNESS_CONTRAST_SATURATION_FB`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

The XrEventDataPassthroughStateChangedFB structure is defined as:

```
// Provided by XR_FB_passthrough
typedef struct XrEventDataPassthroughStateChangedFB {
    XrStructureType                     type;
    const void*                         next;
    XrPassthroughStateChangedFlagsFB    flags;
} XrEventDataPassthroughStateChangedFB;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `flags` XrPassthroughStateChangedFlagsFB that specify additional behavior.

It describes event data for state changes returned by xrPollEvent.

Valid Usage (Implicit)

- The `XR_FB_passthrough` extension **must** be enabled prior to using XrEventDataPassthroughStateChangedFB
- `type` **must** be `XR_TYPE_EVENT_DATA_PASSTHROUGH_STATE_CHANGED_FB`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

New Functions

The xrCreatePassthroughFB function is defined as:

```
// Provided by XR_FB_passthrough
XrResult xrCreatePassthroughFB(
    XrSession                                   session,
    const XrPassthroughCreateInfoFB*            createInfo,
    XrPassthroughFB*                            outPassthrough);
```

Parameter Descriptions

- `session` is the XrSession.
- `createInfo` is the XrPassthroughCreateInfoFB.
- `outPassthrough` is the XrPassthroughFB.

Creates an XrPassthroughFB handle. The returned passthrough handle **may** be subsequently used in API calls. A single XrSession allows only one XrPassthroughFB. If an XrPassthroughFB handle has been already created for this XrSession, the runtime **must** return `XR_ERROR_FEATURE_ALREADY_CREATED_PASSTHROUGH_FB`. If an app requires another feature, then the previous one **must** be destroyed by the app with xrDestroyPassthroughFB first.

Valid Usage (Implicit)

- The XR_FB_passthrough extension **must** be enabled prior to calling xrCreatePassthroughFB
- session **must** be a valid XrSession handle
- createInfo **must** be a pointer to a valid XrPassthroughCreateInfoFB structure
- outPassthrough **must** be a pointer to an XrPassthroughFB handle

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FEATURE_ALREADY_CREATED_PASSTHROUGH_FB
- XR_ERROR_FEATURE_UNSUPPORTED
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_NOT_PERMITTED_PASSTHROUGH_FB
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_UNKNOWN_PASSTHROUGH_FB
- XR_ERROR_VALIDATION_FAILURE

The xrDestroyPassthroughFB function is defined as:

```
// Provided by XR_FB_passthrough
XrResult xrDestroyPassthroughFB(
    XrPassthroughFB                             passthrough);
```

Parameter Descriptions

- `passthrough` is the XrPassthroughFB to be destroyed.

Destroys an XrPassthroughFB handle.

The application is responsible for ensuring that it has no calls using feature handles in progress when the XrPassthroughFB is destroyed.

Valid Usage (Implicit)

- The `XR_FB_passthrough` extension **must** be enabled prior to calling xrDestroyPassthroughFB
- `passthrough` **must** be a valid XrPassthroughFB handle

Thread Safety

- Access to `passthrough`, and any child handles, **must** be externally synchronized

Return Codes

Success

- `XR_SUCCESS`

Failure

- `XR_ERROR_FEATURE_UNSUPPORTED`
- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_RUNTIME_FAILURE`

The xrPassthroughStartFB function is defined as:

```
// Provided by XR_FB_passthrough
XrResult xrPassthroughStartFB(
    XrPassthroughFB                             passthrough);
```

Parameter Descriptions

- `passthrough` is the XrPassthroughFB to be started.

Starts an XrPassthroughFB feature. If the feature is not started, either explicitly with a call to xrPassthroughStartFB, or implicitly at creation using the behavior flags, it is considered paused. When the feature is paused, runtime will stop rendering and compositing all passthrough layers produced on behalf of the application, and may free up some or all the resources used to produce passthrough until xrPassthroughStartFB is called.

Valid Usage (Implicit)

- The `XR_FB_passthrough` extension **must** be enabled prior to calling xrPassthroughStartFB
- `passthrough` **must** be a valid XrPassthroughFB handle

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_FEATURE_UNSUPPORTED`
- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_UNEXPECTED_STATE_PASSTHROUGH_FB`
- `XR_ERROR_VALIDATION_FAILURE`

The xrPassthroughPauseFB function is defined as:

```
// Provided by XR_FB_passthrough
XrResult xrPassthroughPauseFB(
    XrPassthroughFB                             passthrough);
```

Parameter Descriptions

- `passthrough` is the XrPassthroughFB to be paused.

Pauses an XrPassthroughFB feature. When the feature is paused, runtime will stop rendering and compositing all passthrough layers produced on behalf of the application, and may free up some or all the resources used to produce passthrough until xrPassthroughStartFB is called.

Valid Usage (Implicit)

- The `XR_FB_passthrough` extension **must** be enabled prior to calling xrPassthroughPauseFB
- `passthrough` **must** be a valid XrPassthroughFB handle

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_FEATURE_UNSUPPORTED`
- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_UNEXPECTED_STATE_PASSTHROUGH_FB`
- `XR_ERROR_VALIDATION_FAILURE`

The xrCreatePassthroughLayerFB function is defined as:

```
// Provided by XR_FB_passthrough
XrResult xrCreatePassthroughLayerFB(
    XrSession                                   session,
    const XrPassthroughLayerCreateInfoFB*       createInfo,
    XrPassthroughLayerFB*                       outLayer);
```

Parameter Descriptions

- `session` is the XrSession.
- `createInfo` is the XrPassthroughLayerCreateInfoFB.
- `outLayer` is the XrPassthroughLayerFB.

Creates an XrPassthroughLayerFB handle. The returned layer handle **may** be subsequently used in API calls. Layer objects may be used to specify rendering properties of the layer, such as styles, and compositing rules.

If XrPassthroughFB has not be created for XrSession before calling this operation, then the runtime **must** return `XR_ERROR_FEATURE_REQUIRED_PASSTHROUGH_FB`.

Valid Usage (Implicit)

- The `XR_FB_passthrough` extension **must** be enabled prior to calling xrCreatePassthroughLayerFB
- `session` **must** be a valid XrSession handle
- `createInfo` **must** be a pointer to a valid XrPassthroughLayerCreateInfoFB structure
- `outLayer` **must** be a pointer to an XrPassthroughLayerFB handle

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FEATURE_REQUIRED_PASSTHROUGH_FB
- XR_ERROR_FEATURE_UNSUPPORTED
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_INSUFFICIENT_RESOURCES_PASSTHROUGH_FB
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_UNKNOWN_PASSTHROUGH_FB
- XR_ERROR_VALIDATION_FAILURE

The xrDestroyPassthroughLayerFB function is defined as:

```
// Provided by XR_FB_passthrough
XrResult xrDestroyPassthroughLayerFB(
    XrPassthroughLayerFB                        layer);
```

Parameter Descriptions

- layer is the XrPassthroughLayerFB to be destroyed.

Destroys an XrPassthroughLayerFB handle.

Valid Usage (Implicit)

- The XR_FB_passthrough extension **must** be enabled prior to calling xrDestroyPassthroughLayerFB
- `layer` **must** be a valid XrPassthroughLayerFB handle

Thread Safety

- Access to `layer`, and any child handles, **must** be externally synchronized

Return Codes

Success

- XR_SUCCESS

Failure

- XR_ERROR_FEATURE_UNSUPPORTED
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_RUNTIME_FAILURE

The xrPassthroughLayerPauseFB function is defined as:

```
// Provided by XR_FB_passthrough
XrResult xrPassthroughLayerPauseFB(
    XrPassthroughLayerFB                        layer);
```

Parameter Descriptions

- `layer` is the XrPassthroughLayerFB to be paused.

Pauses an XrPassthroughLayerFB layer. Runtime will not render or composite paused layers.

Valid Usage (Implicit)

- The XR_FB_passthrough extension **must** be enabled prior to calling xrPassthroughLayerPauseFB
- `layer` **must** be a valid XrPassthroughLayerFB handle

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FEATURE_UNSUPPORTED
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_UNEXPECTED_STATE_PASSTHROUGH_FB
- XR_ERROR_VALIDATION_FAILURE

The xrPassthroughLayerResumeFB function is defined as:

```
// Provided by XR_FB_passthrough
XrResult xrPassthroughLayerResumeFB(
    XrPassthroughLayerFB                        layer);
```

Parameter Descriptions

- `layer` is the XrPassthroughLayerFB to be resumed.

Resumes an XrPassthroughLayerFB layer.

Valid Usage (Implicit)

- The XR_FB_passthrough extension **must** be enabled prior to calling xrPassthroughLayerResumeFB
- `layer` **must** be a valid XrPassthroughLayerFB handle

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FEATURE_UNSUPPORTED
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_UNEXPECTED_STATE_PASSTHROUGH_FB
- XR_ERROR_VALIDATION_FAILURE

The xrPassthroughLayerSetStyleFB function is defined as:

```
// Provided by XR_FB_passthrough
XrResult xrPassthroughLayerSetStyleFB(
    XrPassthroughLayerFB                        layer,
    const XrPassthroughStyleFB*                 style);
```

Parameter Descriptions

- `layer` is the XrPassthroughLayerFB to get the style.
- `style` is the XrPassthroughStyleFB to be set.

Sets an XrPassthroughStyleFB style on an XrPassthroughLayerFB layer.

Valid Usage (Implicit)

- The XR_FB_passthrough extension **must** be enabled prior to calling xrPassthroughLayerSetStyleFB
- `layer` **must** be a valid XrPassthroughLayerFB handle
- `style` **must** be a pointer to a valid XrPassthroughStyleFB structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FEATURE_UNSUPPORTED
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The xrCreateGeometryInstanceFB function is defined as:

```
// Provided by XR_FB_passthrough
XrResult xrCreateGeometryInstanceFB(
    XrSession                                   session,
    const XrGeometryInstanceCreateInfoFB*       createInfo,
    XrGeometryInstanceFB*                       outGeometryInstance);
```

Parameter Descriptions

- `session` is the XrSession.
- `createInfo` is the XrGeometryInstanceCreateInfoFB.
- `outGeometryInstance` is the XrGeometryInstanceFB.

Creates an XrGeometryInstanceFB handle. Geometry instance functionality requires XR_FB_triangle_mesh extension to be enabled. An XrGeometryInstanceFB connects a layer, a mesh, and a transformation, with the semantics that a specific mesh will be instantiated in a specific layer with a specific transformation. A mesh can be instantiated multiple times, in the same or in different layers.

Valid Usage (Implicit)

- The XR_FB_passthrough extension **must** be enabled prior to calling xrCreateGeometryInstanceFB
- `session` **must** be a valid XrSession handle
- `createInfo` **must** be a pointer to a valid XrGeometryInstanceCreateInfoFB structure
- `outGeometryInstance` **must** be a pointer to an XrGeometryInstanceFB handle

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FEATURE_UNSUPPORTED
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_INSUFFICIENT_RESOURCES_PASSTHROUGH_FB
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_POSE_INVALID
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The xrDestroyGeometryInstanceFB function is defined as:

```
// Provided by XR_FB_passthrough
XrResult xrDestroyGeometryInstanceFB(
    XrGeometryInstanceFB                        instance);
```

Parameter Descriptions

- `instance` is the XrGeometryInstanceFB to be destroyed.

Destroys an XrGeometryInstanceFB handle. Destroying an XrGeometryInstanceFB does not destroy a mesh and does not free mesh resources. Destroying a layer invalidates all geometry instances attached to it. Destroying a mesh invalidates all its instances.

Valid Usage (Implicit)

- The `XR_FB_passthrough` extension **must** be enabled prior to calling xrDestroyGeometryInstanceFB
- `instance` **must** be a valid XrGeometryInstanceFB handle

Thread Safety

- Access to `instance`, and any child handles, **must** be externally synchronized

Return Codes

Success

- `XR_SUCCESS`

Failure

- `XR_ERROR_FEATURE_UNSUPPORTED`
- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_RUNTIME_FAILURE`

The xrGeometryInstanceSetTransformFB function is defined as:

```
// Provided by XR_FB_passthrough
XrResult xrGeometryInstanceSetTransformFB(
    XrGeometryInstanceFB                        instance,
    const XrGeometryInstanceTransformFB*        transformation);
```

Parameter Descriptions

- `instance` is the XrGeometryInstanceFB to get the transform.
- `transformation` is the XrGeometryInstanceTransformFB to be set.

Sets an XrGeometryInstanceTransformFB transform on an XrGeometryInstanceFB geometry instance.

Valid Usage (Implicit)

- The `XR_FB_passthrough` extension **must** be enabled prior to calling xrGeometryInstanceSetTransformFB
- `instance` **must** be a valid XrGeometryInstanceFB handle
- `transformation` **must** be a pointer to a valid XrGeometryInstanceTransformFB structure

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_FEATURE_UNSUPPORTED`
- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_POSE_INVALID`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_TIME_INVALID`
- `XR_ERROR_VALIDATION_FAILURE`

Issues * Is XrPassthroughFB the parent for XrPassthroughLayerFB? **From OpenXR conventions, it is clear that the XrSession is the parent of XrPassthroughLayerFB, however XrPassthroughLayerFB is also strongly associated to XrPassthroughFB and a reasonable implementation could result in it be invalidated after XrPassthroughFB is destroyed.** In retrospect, (i.e. if we were going to design this extension again), we would explicitly define XrPassthroughFB as the first parameter (and therefore parent, instead of XrSession) in xrCreatePassthroughLayerFB and thus explicitly define XrPassthroughFB as the parent of XrPassthroughLayerFB. ** To avoid potential issues, applications are recommended not to use XrPassthroughLayerFB after XrPassthroughFB has been destroyed. That means that before destroying feature XrPassthroughFB with xrDestroyPassthroughFB, an app should destroy all XrPassthroughLayerFB handles first, by calling xrDestroyPassthroughLayerFB for each such handle.

Version History

- Revision 1, 2021-09-01 (Anton Vaneev)
  - Initial extension description
- Revision 2, 2022-03-16 (Johannes Schmid)
  - Introduce XrPassthroughBrightnessContrastSaturationFB.
  - Revise the documentation of XrPassthroughStyleFB and its descendants.
- Revision 3, 2022-07-14 (Johannes Schmid)
  - Introduce a new struct for querying passthrough system capabilities: XrSystemPassthroughProperties2FB.
  - Introduce a new flag bit that enables submission of depth maps for compositing: `XR_PASSTHROUGH_LAYER_DEPTH_BIT_FB`.
- Revision 4, 2024-06-03 (Rylie Pavlik, Collabora)
  - Correct registry for `XrCompositionLayerPassthroughFB`, note errata regarding field name.
- Revision 5, 2025-01-15 (Ante Trbojevic)
  - Explicitly define that only one XrPassthroughFB handle per XrSession is allowed.
  - Update Issues section with "Is XrPassthroughFB the parent for XrPassthroughLayerFB?"
  - Change wording "Facebook" to "Meta Platforms"

12.117. XR_FB_passthrough_keyboard_hands

Name String

`XR_FB_passthrough_keyboard_hands`

Extension Type

Instance extension

Registered Extension Number

204

Revision

2

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0
and
XR_FB_passthrough

Contributors

Ante Trbojevic, Facebook
Cass Everitt, Facebook
Federico Schliemann, Facebook
Anton Vaneev, Facebook
Johannes Schmid, Facebook

Overview

This extension enables applications to show passthrough hands when hands are placed over the tracked keyboard. It enables users to see their hands over the keyboard in a mixed reality application. This extension is dependent on XR_FB_passthrough extension which can be used to create a passthrough layer for hand presence use-case.

The extension supports a single pair of hands (one left and one right hand), multiple pair of hands are not supported.

This extension allows:

- Creation of keyboard hands passthrough layer using xrCreatePassthroughLayerFB
- Setting the level of intensity for the hand mask in a passthrough layer with purpose XrPassthroughLayerPurposeFB as XR_PASSTHROUGH_LAYER_PURPOSE_TRACKED_KEYBOARD_HANDS_FB or XR_PASSTHROUGH_LAYER_PURPOSE_TRACKED_KEYBOARD_MASKED_HANDS_FB

New Enum Constants

XrPassthroughLayerPurposeFB enumeration is extended with a new constant:

- XR_PASSTHROUGH_LAYER_PURPOSE_TRACKED_KEYBOARD_HANDS_FB - It defines a keyboard hands presence purpose of passthrough layer (i.e. basic mode, without hand transitions).
- XR_PASSTHROUGH_LAYER_PURPOSE_TRACKED_KEYBOARD_MASKED_HANDS_FB - It defines a keyboard hands presence purpose of passthrough layer with keyboard masked hand transitions. A hand mask will

be visible only when hands are inside the region of VR keyboard (i.e. hands over the keyboard).

XrStructureType enumeration is extended with:

- XR_TYPE_PASSTHROUGH_KEYBOARD_HANDS_INTENSITY_FB

New Structures

The XrPassthroughKeyboardHandsIntensityFB structure is defined as:

```
// Provided by XR_FB_passthrough_keyboard_hands
typedef struct XrPassthroughKeyboardHandsIntensityFB {
    XrStructureType    type;
    const void*        next;
    float              leftHandIntensity;
    float              rightHandIntensity;
} XrPassthroughKeyboardHandsIntensityFB;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- leftHandIntensity defines an intensity for the left tracked hand.
- rightHandIntensity defines an intensity for the right tracked hand.

XrPassthroughKeyboardHandsIntensityFB describes intensities of passthrough hands, and is used as a parameter to xrPassthroughLayerSetKeyboardHandsIntensityFB.

Each of the intensity values leftHandIntensity and rightHandIntensity **must** be in the range [0.0, 1.0]. The hand intensity value represents the level of visibility of rendered hand, the minimal value of the intensity 0.0 represents the fully transparent hand (not visible), the maximal value of 1.0 represented fully opaque hands (maximal visibility).

If either leftHandIntensity or rightHandIntensity is outside the range [0.0, 1.0], the runtime must return XR_ERROR_VALIDATION_FAILURE.

Valid Usage (Implicit)

- The XR_FB_passthrough_keyboard_hands extension **must** be enabled prior to using XrPassthroughKeyboardHandsIntensityFB
- type **must** be XR_TYPE_PASSTHROUGH_KEYBOARD_HANDS_INTENSITY_FB
- next **must** be NULL or a valid pointer to the next structure in a structure chain

New Functions

The xrPassthroughLayerSetKeyboardHandsIntensityFB function is defined as:

```
// Provided by XR_FB_passthrough_keyboard_hands
XrResult xrPassthroughLayerSetKeyboardHandsIntensityFB(
    XrPassthroughLayerFB                        layer,
    const XrPassthroughKeyboardHandsIntensityFB* intensity);
```

Parameter Descriptions

- layer is the XrPassthroughLayerFB to apply the intensity.
- intensity is the XrPassthroughKeyboardHandsIntensityFB to be set.

Sets an XrPassthroughKeyboardHandsIntensityFB intensity on an XrPassthroughLayerFB layer.

Valid Usage (Implicit)

- The XR_FB_passthrough_keyboard_hands extension **must** be enabled prior to calling xrPassthroughLayerSetKeyboardHandsIntensityFB
- layer **must** be a valid XrPassthroughLayerFB handle
- intensity **must** be a pointer to a valid XrPassthroughKeyboardHandsIntensityFB structure

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_FEATURE_UNSUPPORTED`
- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_VALIDATION_FAILURE`

Issues

Version History

- Revision 1, 2021-11-23 (Ante Trbojevic)
  - Initial extension description
- Revision 2, 2022-03-16 (Ante Trbojevic)
  - Introduce `XR_PASSTHROUGH_LAYER_PURPOSE_TRACKED_KEYBOARD_MASKED_HANDS_FB`

12.118. XR_FB_render_model

Name String

`XR_FB_render_model`

Extension Type

Instance extension

Registered Extension Number

120

Revision

4

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Contributors

Leonard Tsai, Meta
Xiang Wei, Meta
Robert Memmott, Meta

Overview

This extension allows applications to request GLTF models for certain connected devices supported by the runtime. Paths that correspond to these devices will be provided through the extension and can be used to get information about the models as well as loading them.

New Flag Types

```
// Provided by XR_FB_render_model
typedef XrFlags64 XrRenderModelFlagsFB;
```

```
// Provided by XR_FB_render_model
// Flag bits for XrRenderModelFlagsFB
static const XrRenderModelFlagsFB XR_RENDER_MODEL_SUPPORTS_GLTF_2_0_SUBSET_1_BIT_FB =
0x00000001;
static const XrRenderModelFlagsFB XR_RENDER_MODEL_SUPPORTS_GLTF_2_0_SUBSET_2_BIT_FB =
0x00000002;
```

Flag Descriptions

- XR_RENDER_MODEL_SUPPORTS_GLTF_2_0_SUBSET_1_BIT_FB — Minimal level of support. Can only contain a single mesh. Can only contain a single texture. Can not contain transparency. Assumes unlit rendering. Requires Extension KHR_texturebasisu.
- XR_RENDER_MODEL_SUPPORTS_GLTF_2_0_SUBSET_2_BIT_FB — All of XR_RENDER_MODEL_SUPPORTS_GLTF_2_0_SUBSET_1_BIT_FB support plus: Multiple meshes. Multiple Textures. Texture Transparency.

Render Model Support Levels: An application **should** request a model of a certain complexity via the XrRenderModelCapabilitiesRequestFB on the structure chain of XrRenderModelPropertiesFB passed into xrGetRenderModelPropertiesFB. The flags on the XrRenderModelCapabilitiesRequestFB are an acknowledgement of the application's ability to render such a model. Multiple values of XrRenderModelFlagBitsFB can be set on this variable to indicate acceptance of different support levels. The flags parameter on the XrRenderModelPropertiesFB will indicate what capabilities the model in the runtime actually requires. It will be set to a single value of XrRenderModelFlagBitsFB.

New Enum Constants

XrStructureType enumeration is extended with:

- XR_TYPE_SYSTEM_RENDER_MODEL_PROPERTIES_FB
- XR_TYPE_RENDER_MODEL_PATH_INFO_FB
- XR_TYPE_RENDER_MODEL_PROPERTIES_FB
- XR_TYPE_RENDER_MODEL_BUFFER_FB
- XR_TYPE_RENDER_MODEL_LOAD_INFO_FB
- XR_MAX_RENDER_MODEL_NAME_SIZE_FB

New Defines

```
// Provided by XR_FB_render_model
#define XR_NULL_RENDER_MODEL_KEY_FB 0
```

XR_NULL_RENDER_MODEL_KEY_FB defines an invalid model key atom.

New Base Types

```
// Provided by XR_FB_render_model
XR_DEFINE_ATOM(XrRenderModelKeyFB)
```

The unique model key used to retrieve the data for the render model that is valid across multiple instances and installs. The application can use this key along with the model version to update its cached or saved version of the model.

New Structures

The XrSystemRenderModelPropertiesFB structure is defined as:

```
// Provided by XR_FB_render_model
typedef struct XrSystemRenderModelPropertiesFB {
    XrStructureType    type;
    void*              next;
    XrBool32           supportsRenderModelLoading;
} XrSystemRenderModelPropertiesFB;
```

It describes a render model system property.

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension. For more details about the structure chain, see the structure being extended (XrSystemProperties).
- `supportsRenderModelLoading` defines whether the system supports loading render models.

Valid Usage (Implicit)

- The `XR_FB_render_model` extension **must** be enabled prior to using XrSystemRenderModelPropertiesFB
- `type` **must** be `XR_TYPE_SYSTEM_RENDER_MODEL_PROPERTIES_FB`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

The XrRenderModelPathInfoFB structure is defined as:

```
// Provided by XR_FB_render_model
typedef struct XrRenderModelPathInfoFB {
    XrStructureType    type;
    void*              next;
    XrPath             path;
} XrRenderModelPathInfoFB;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `path` is a valid `XrPath` used for retrieving model properties from xrGetRenderModelPropertiesFB.

XrRenderModelPathInfoFB contains a model path supported by the device when returned from xrEnumerateRenderModelPathsFB. This path can be used to request information about the render model for the connected device that the path represents using xrGetRenderModelPropertiesFB.

Possible Render Model Paths

- Controller models with origin at the grip pose.
  - */model_fb/controller/left*
  - */model_fb/controller/right*
- Keyboard models with origin at the center of its bounding box.
  - */model_fb/keyboard/local*
  - */model_fb/keyboard/remote*
  - */model_meta/keyboard/virtual*
    (if the `XR_META_virtual_keyboard` extension is enabled)

Valid Usage (Implicit)

- The `XR_FB_render_model` extension **must** be enabled prior to using XrRenderModelPathInfoFB
- `type` **must** be `XR_TYPE_RENDER_MODEL_PATH_INFO_FB`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

The XrRenderModelPropertiesFB structure is defined as:

```
// Provided by XR_FB_render_model
typedef struct XrRenderModelPropertiesFB {
    XrStructureType         type;
    void*                   next;
    uint32_t                vendorId;
    char                    modelName[XR_MAX_RENDER_MODEL_NAME_SIZE_FB];
    XrRenderModelKeyFB      modelKey;
    uint32_t                modelVersion;
    XrRenderModelFlagsFB    flags;
} XrRenderModelPropertiesFB;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. XrRenderModelCapabilitiesRequestFB is a structure in this structure chain and **should** be linked when this structure is passed to xrGetRenderModelPropertiesFB.
- `vendorId` is the vendor id of the model.
- `modelName` is the name of the model.
- `modelKey` is the unique model key used to load the model in xrLoadRenderModelFB.
- `modelVersion` is the version number of the model.
- `flags` is a bitmask of XrRenderModelFlagsFB. After a successful call to xrGetRenderModelPropertiesFB, flags must contain the support level of the model and no other support levels.

XrRenderModelPropertiesFB contains information about the render model for a device. XrRenderModelPropertiesFB **must** be provided when calling xrGetRenderModelPropertiesFB. The `XrRenderModelKeyFB` included in the properties is a unique key for each render model that is valid across multiple instances and installs.

If the application decides to cache or save the render model in any way, `modelVersion` can be used to determine if the render model has changed. The application **should** then update its cached or saved version.

Valid Usage (Implicit)

- The `XR_FB_render_model` extension **must** be enabled prior to using XrRenderModelPropertiesFB
- `type` **must** be `XR_TYPE_RENDER_MODEL_PROPERTIES_FB`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain. See also: XrRenderModelCapabilitiesRequestFB
- `modelName` **must** be a null-terminated UTF-8 string whose length is less than or equal to `XR_MAX_RENDER_MODEL_NAME_SIZE_FB`
- `flags` **must** be a valid combination of XrRenderModelFlagBitsFB values
- `flags` **must** not be `0`

The XrRenderModelCapabilitiesRequestFB structure is defined as:

```
// Provided by XR_FB_render_model
typedef struct XrRenderModelCapabilitiesRequestFB {
    XrStructureType         type;
    void*                   next;
    XrRenderModelFlagsFB    flags;
} XrRenderModelCapabilitiesRequestFB;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `flags` is a bit mask of the model complexities that the application is able to support.

XrRenderModelCapabilitiesRequestFB contains information about the render capabilities requested for a model. XrRenderModelCapabilitiesRequestFB **must** be set in the structure chain of the `next` pointer on the XrRenderModelPropertiesFB passed into the xrGetRenderModelPropertiesFB call. The `flags` on XrRenderModelCapabilitiesRequestFB represent an acknowledgement of being able to handle the individual model capability levels. If no XrRenderModelCapabilitiesRequestFB is on the structure chain then the runtime **should** treat it as if a value of `XR_RENDER_MODEL_SUPPORTS_GLTF_2_0_SUBSET_1_BIT_FB` was set. If the runtime does not have a model available that matches any of the supports flags set, then it **must** return a `XR_RENDER_MODEL_UNAVAILABLE_FB` result.

Valid Usage (Implicit)

- The `XR_FB_render_model` extension **must** be enabled prior to using XrRenderModelCapabilitiesRequestFB
- `type` **must** be `XR_TYPE_RENDER_MODEL_CAPABILITIES_REQUEST_FB`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `flags` **must** be a valid combination of XrRenderModelFlagBitsFB values
- `flags` **must** not be `0`

The XrRenderModelLoadInfoFB structure is defined as:

```
// Provided by XR_FB_render_model
typedef struct XrRenderModelLoadInfoFB {
    XrStructureType       type;
    void*                 next;
    XrRenderModelKeyFB    modelKey;
} XrRenderModelLoadInfoFB;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `modelKey` is the unique model key for a connected device.

XrRenderModelLoadInfoFB is used to provide information about which render model to load. XrRenderModelLoadInfoFB **must** be provided when calling xrLoadRenderModelFB.

Valid Usage (Implicit)

- The `XR_FB_render_model` extension **must** be enabled prior to using XrRenderModelLoadInfoFB
- `type` **must** be `XR_TYPE_RENDER_MODEL_LOAD_INFO_FB`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

The XrRenderModelBufferFB structure is defined as:

```
// Provided by XR_FB_render_model
typedef struct XrRenderModelBufferFB {
    XrStructureType    type;
    void*              next;
    uint32_t           bufferCapacityInput;
    uint32_t           bufferCountOutput;
    uint8_t*           buffer;
} XrRenderModelBufferFB;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `bufferCapacityInput` is the capacity of the `buffer`, or `0` to retrieve the required capacity.
- `bufferCountOutput` is the count of `uint8_t buffer` written, or the required capacity in the case that `bufferCapacityInput` is insufficient.
- `buffer` is a pointer to an application-allocated array that will be filled with the render model binary data.
- See the Buffer Size Parameters section for a detailed description of retrieving the required `buffer` size.

XrRenderModelBufferFB is used when loading the binary data for a render model. XrRenderModelBufferFB **must** be provided when calling xrLoadRenderModelFB.

Valid Usage (Implicit)

- The `XR_FB_render_model` extension **must** be enabled prior to using XrRenderModelBufferFB
- `type` **must** be `XR_TYPE_RENDER_MODEL_BUFFER_FB`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- If `bufferCapacityInput` is not `0`, `buffer` **must** be a pointer to an array of `bufferCapacityInput` `uint8_t` values

New Functions

The xrEnumerateRenderModelPathsFB function is defined as:

```
// Provided by XR_FB_render_model
XrResult xrEnumerateRenderModelPathsFB(
    XrSession                                   session,
    uint32_t                                    pathCapacityInput,
    uint32_t*                                   pathCountOutput,
    XrRenderModelPathInfoFB*                    paths);
```

Parameter Descriptions

- `session` is the specified XrSession.
- `pathCapacityInput` is the capacity of the `paths`, or 0 to retrieve the required capacity.
- `pathCountOutput` is a pointer to the count of `float paths` written, or a pointer to the required capacity in the case that `pathCapacityInput` is insufficient.
- `paths` is a pointer to an application-allocated array that will be filled with XrRenderModelPathInfoFB values that are supported by the runtime, but **can** be `NULL` if `pathCapacityInput` is `0`
- See the Buffer Size Parameters section for a detailed description of retrieving the required `paths` size.

The application **must** call xrEnumerateRenderModelPathsFB to enumerate the valid render model paths that are supported by the runtime before calling xrGetRenderModelPropertiesFB. The paths returned **may** be used later in xrGetRenderModelPropertiesFB.

Valid Usage (Implicit)

- The `XR_FB_render_model` extension **must** be enabled prior to calling xrEnumerateRenderModelPathsFB
- `session` **must** be a valid XrSession handle
- `pathCountOutput` **must** be a pointer to a `uint32_t` value
- If `pathCapacityInput` is not `0`, `paths` **must** be a pointer to an array of `pathCapacityInput` XrRenderModelPathInfoFB structures

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The xrGetRenderModelPropertiesFB function is defined as:

```
// Provided by XR_FB_render_model
XrResult xrGetRenderModelPropertiesFB(
    XrSession                                   session,
    XrPath                                      path,
    XrRenderModelPropertiesFB*                  properties);
```

Parameter Descriptions

- session is the specified XrSession.
- path is the path of the render model to get the properties for.
- properties is a pointer to the XrRenderModelPropertiesFB to write the render model information to.

xrGetRenderModelPropertiesFB is used for getting information for a render model using a path retrieved from xrEnumerateRenderModelPathsFB. The information returned will be for the connected device that corresponds to the path given. For example, using */model_fb/controller/left* will return information for the left controller that is currently connected and will change if a different device that also represents a left controller is connected.

The runtime **must** return XR_ERROR_CALL_ORDER_INVALID if xrGetRenderModelPropertiesFB is called with render model paths before calling xrEnumerateRenderModelPathsFB. The runtime **must** return

XR_ERROR_PATH_INVALID if a path not given by xrEnumerateRenderModelPathsFB is used.

If xrGetRenderModelPropertiesFB returns a success code of XR_RENDER_MODEL_UNAVAILABLE_FB and has a XrRenderModelPropertiesFB::modelKey of XR_NULL_RENDER_MODEL_KEY_FB, this indicates that the model for the device is unavailable. The application **may** keep calling xrGetRenderModelPropertiesFB because the model **may** become available later when a device is connected.

Valid Usage (Implicit)

- The XR_FB_render_model extension **must** be enabled prior to calling xrGetRenderModelPropertiesFB
- session **must** be a valid XrSession handle
- properties **must** be a pointer to an XrRenderModelPropertiesFB structure

Return Codes

Success

- XR_SUCCESS
- XR_RENDER_MODEL_UNAVAILABLE_FB
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_CALL_ORDER_INVALID
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_PATH_INVALID
- XR_ERROR_PATH_UNSUPPORTED
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The xrLoadRenderModelFB function is defined as:

```
// Provided by XR_FB_render_model
XrResult xrLoadRenderModelFB(
    XrSession                                   session,
    const XrRenderModelLoadInfoFB*              info,
    XrRenderModelBufferFB*                      buffer);
```

Parameter Descriptions

- `session` is the specified XrSession.
- `info` is a pointer to the XrRenderModelLoadInfoFB structure.
- `buffer` is a pointer to the XrRenderModelBufferFB structure to write the binary data into.

xrLoadRenderModelFB is used to load the GLTF model data using a valid XrRenderModelLoadInfoFB::`modelKey`. xrLoadRenderModelFB loads the model as a byte buffer containing the GLTF in the binary format (GLB). The GLB data **must** conform to the glTF 2.0 format defined at https://registry.khronos.org/glTF/specs/2.0/glTF-2.0.html. The GLB **may** contain texture data in a format that requires the use of the `KHR_texture_basisu` GLTF extension defined at https://github.com/KhronosGroup/glTF/tree/main/extensions/2.0/Khronos/KHR_texture_basisu. Therefore, the application **should** ensure it can handle this extension.

If the device for the requested model is disconnected or does not match the XrRenderModelLoadInfoFB::`modelKey` provided, xrLoadRenderModelFB **must** return `XR_RENDER_MODEL_UNAVAILABLE_FB` as well as an XrRenderModelBufferFB::`bufferCountOutput` value of 0 indicating that the model was not available.

The xrLoadRenderModelFB function **may** be slow, therefore applications **should** call it from a non-time sensitive thread.

Valid Usage (Implicit)

- The `XR_FB_render_model` extension **must** be enabled prior to calling xrLoadRenderModelFB
- `session` **must** be a valid XrSession handle
- `info` **must** be a pointer to a valid XrRenderModelLoadInfoFB structure
- `buffer` **must** be a pointer to an XrRenderModelBufferFB structure

Return Codes

Success

- XR_SUCCESS
- XR_RENDER_MODEL_UNAVAILABLE_FB
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RENDER_MODEL_KEY_INVALID_FB
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

Issues

Version History

- Revision 1, 2021-08-17 (Leonard Tsai)
  - Initial extension description
- Revision 2, 2022-05-03 (Robert Memmott)
  - Render Model Support Subsets
- Revision 3, 2022-07-07 (Rylie Pavlik, Collabora, Ltd.)
  - Fix implicit valid usage for XrRenderModelCapabilitiesRequestFB
- Revision 4, 2023-04-14 (Peter Chan)
  - Add possible render model path for XR_META_virtual_keyboard

12.119. XR_FB_scene

Name String

XR_FB_scene

Extension Type

Instance extension

Registered Extension Number

176

Revision

4

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0
and
XR_FB_spatial_entity

Contributors

John Schofield, Facebook
Andrew Kim, Facebook
Yuichi Taguchi, Facebook
Cass Everitt, Facebook

Overview

This extension expands on the concept of spatial entities to include a way for a spatial entity to represent rooms, objects, or other boundaries in a scene.

New Object Types

New Flag Types

```
// Provided by XR_FB_scene
typedef XrFlags64 XrSemanticLabelsSupportFlagsFB;
```

```
// Provided by XR_FB_scene
// Flag bits for XrSemanticLabelsSupportFlagsFB
static const XrSemanticLabelsSupportFlagsFB
XR_SEMANTIC_LABELS_SUPPORT_MULTIPLE_SEMANTIC_LABELS_BIT_FB = 0x00000001;
static const XrSemanticLabelsSupportFlagsFB
XR_SEMANTIC_LABELS_SUPPORT_ACCEPT_DESK_TO_TABLE_MIGRATION_BIT_FB = 0x00000002;
static const XrSemanticLabelsSupportFlagsFB
XR_SEMANTIC_LABELS_SUPPORT_ACCEPT_INVISIBLE_WALL_FACE_BIT_FB = 0x00000004;
```

Flag Descriptions

- XR_SEMANTIC_LABELS_SUPPORT_MULTIPLE_SEMANTIC_LABELS_BIT_FB — If set, and the runtime reports the extensionVersion as 2 or greater, the runtime **may** return multiple semantic labels separated by a comma without spaces. Otherwise, the runtime **must** return a single semantic label.
- XR_SEMANTIC_LABELS_SUPPORT_ACCEPT_DESK_TO_TABLE_MIGRATION_BIT_FB — If set, and the runtime reports the extensionVersion as 3 or greater, the runtime **must** return "TABLE" instead of "DESK" as a semantic label to the application. Otherwise, the runtime **must** return "DESK" instead of "TABLE" as a semantic label to the application, when applicable.
- XR_SEMANTIC_LABELS_SUPPORT_ACCEPT_INVISIBLE_WALL_FACE_BIT_FB — If set, and the runtime reports the extensionVersion as 4 or greater, the runtime **may** return "INVISIBLE_WALL_FACE" instead of "WALL_FACE" as a semantic label to the application in order to represent an invisible wall used to conceptually separate a space (e.g., separate a living space from a kitchen space in an open floor plan house even though there is no real wall between the two spaces) instead of a real wall. Otherwise, the runtime **must** return "WALL_FACE" as a semantic label to the application in order to represent both an invisible and real wall, when applicable.

New Enum Constants

XrStructureType enumeration is extended with:

- XR_TYPE_SEMANTIC_LABELS_FB
- XR_TYPE_ROOM_LAYOUT_FB
- XR_TYPE_BOUNDARY_2D_FB
- XR_TYPE_SEMANTIC_LABELS_SUPPORT_INFO_FB

New Enums

New Structures

The XrExtent3DfFB structure is defined as:

```
// Provided by XR_FB_scene
// XrExtent3DfFB is an alias for XrExtent3Df
typedef struct XrExtent3Df {
    float    width;
    float    height;
    float    depth;
} XrExtent3Df;

typedef XrExtent3Df XrExtent3DfFB;
```

Member Descriptions

- `width` is the floating-point width of the extent.
- `height` is the floating-point height of the extent.
- `depth` is the floating-point depth of the extent.

This structure is used for component values that may be fractional (floating-point). If used to represent physical distances, values must be in meters. The width, height, and depth values must be non-negative.

Valid Usage (Implicit)

- The `XR_FB_scene` extension **must** be enabled prior to using XrExtent3DfFB

The XrOffset3DfFB structure is defined as:

```
// Provided by XR_FB_scene
typedef struct XrOffset3DfFB {
    float    x;
    float    y;
    float    z;
} XrOffset3DfFB;
```

Member Descriptions

- `x` is the floating-point offset in the x direction.
- `y` is the floating-point offset in the y direction.
- `z` is the floating-point offset in the z direction.

This structure is used for component values that may be fractional (floating-point). If used to represent physical distances, values must be in meters.

Valid Usage (Implicit)

- The XR_FB_scene extension **must** be enabled prior to using XrOffset3DfFB

The XrRect3DfFB structure is defined as:

```
// Provided by XR_FB_scene
typedef struct XrRect3DfFB {
    XrOffset3DfFB    offset;
    XrExtent3DfFB    extent;
} XrRect3DfFB;
```

Member Descriptions

- `offset` is the XrOffset3DfFB specifying the rectangle offset.
- `extent` is the XrExtent3DfFB specifying the rectangle extent.

This structure is used for component values that may be fractional (floating-point).

The bounding box is defined by an `offset` and `extent`. The `offset` refers to the coordinate of the minimum corner of the box in the local space of the XrSpace; that is, the corner whose coordinate has the minimum value on each axis. The `extent` refers to the dimensions of the box along each axis. The maximum corner can therefore be computed as `offset` + `extent`.

Valid Usage (Implicit)

- The XR_FB_scene extension **must** be enabled prior to using XrRect3DfFB

The XrSemanticLabelsFB structure is defined as:

```
// Provided by XR_FB_scene
typedef struct XrSemanticLabelsFB {
    XrStructureType    type;
    const void*        next;
    uint32_t           bufferCapacityInput;
    uint32_t           bufferCountOutput;
    char*              buffer;
} XrSemanticLabelsFB;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain, such as XrSemanticLabelsSupportInfoFB.
- `bufferCapacityInput` is the capacity of the `buffer` array, in bytes, or 0 to indicate a request to retrieve the required capacity.
- `bufferCountOutput` is the count of bytes written, or the required capacity in the case that `bufferCapacityInput` is insufficient.
- `buffer` is a pointer to an array of bytes, but can be `NULL` if `bufferCapacityInput` is 0. Multiple labels represented by raw string, separated by a comma without spaces.
- See the Buffer Size Parameters section for a detailed description of retrieving the required `buffer` size.

This structure is used by the xrGetSpaceSemanticLabelsFB function to provide the application with the intended usage of the spatial entity.

Valid Usage (Implicit)

- The `XR_FB_scene` extension **must** be enabled prior to using XrSemanticLabelsFB
- `type` **must** be `XR_TYPE_SEMANTIC_LABELS_FB`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain. See also: XrSemanticLabelsSupportInfoFB
- If `bufferCapacityInput` is not `0`, `buffer` **must** be a pointer to an array of `bufferCapacityInput` char values

The XrRoomLayoutFB structure is defined as:

```
// Provided by XR_FB_scene
typedef struct XrRoomLayoutFB {
    XrStructureType    type;
    const void*        next;
    XrUuidEXT          floorUuid;
    XrUuidEXT          ceilingUuid;
    uint32_t           wallUuidCapacityInput;
    uint32_t           wallUuidCountOutput;
    XrUuidEXT*         wallUuids;
} XrRoomLayoutFB;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `floorUuid` is the UUID of the spatial entity representing the room floor
- `ceilingUuid` is the UUID of the spatial entity representing the room ceiling
- `wallUuidCapacityInput` is the capacity of the `wallUuids` array, in number of UUIDs, or 0 to indicate a request to retrieve the required capacity.
- `wallUuidCountOutput` is the count of XrUuidEXT handles written, or the required capacity in the case that `wallUuidCapacityInput` is insufficient.
- `wallUuids` is a pointer to an array of XrUuidEXT handles, but can be `NULL` if `wallUuidCapacityInput` is 0.
- See the Buffer Size Parameters section for a detailed description of retrieving the required `wallUuids` array size.

This structure is used by the xrGetSpaceRoomLayoutFB function to provide the application with the XrUuidEXT handles representing the various surfaces of a room.

Valid Usage (Implicit)

- The `XR_FB_scene` extension **must** be enabled prior to using XrRoomLayoutFB
- `type` **must** be `XR_TYPE_ROOM_LAYOUT_FB`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- If `wallUuidCapacityInput` is not `0`, `wallUuids` **must** be a pointer to an array of `wallUuidCapacityInput` XrUuidEXT structures

The XrBoundary2DFB structure is defined as:

```
// Provided by XR_FB_scene
typedef struct XrBoundary2DFB {
    XrStructureType    type;
    const void*        next;
    uint32_t           vertexCapacityInput;
    uint32_t           vertexCountOutput;
    XrVector2f*        vertices;
} XrBoundary2DFB;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `vertexCapacityInput` is the capacity of the `vertices` array, in number of vertices, or 0 to indicate a request to retrieve the required capacity.
- `vertexCountOutput` is the count of XrVector2f written, or the required capacity in the case that `vertexCapacityInput` is insufficient.
- `vertices` is a pointer to an array of XrVector2f, but **can** be `NULL` if `vertexCapacityInput` is 0.
- See the Buffer Size Parameters section for a detailed description of retrieving the required `vertices` array size.

This structure is used by the xrGetSpaceBoundary2DFB function to provide the application with the XrVector2f vertices representing the a spatial entity with a boundary.

Valid Usage (Implicit)

- The `XR_FB_scene` extension **must** be enabled prior to using XrBoundary2DFB
- `type` **must** be `XR_TYPE_BOUNDARY_2D_FB`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- If `vertexCapacityInput` is not `0`, `vertices` **must** be a pointer to an array of `vertexCapacityInput` XrVector2f structures

The XrSemanticLabelsSupportInfoFB structure is defined as:

```
// Provided by XR_FB_scene
typedef struct XrSemanticLabelsSupportInfoFB {
    XrStructureType                   type;
    const void*                       next;
    XrSemanticLabelsSupportFlagsFB    flags;
    const char*                       recognizedLabels;
} XrSemanticLabelsSupportInfoFB;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `flags` is a bitmask of XrSemanticLabelsSupportFlagBitsFB that specifies additional behaviors.
- `recognizedLabels` is a `NULL` terminated string that indicates a set of semantic labels recognized by the application. Each semantic label **must** be represented as a string and be separated by a comma without spaces. This field **must** include at least "OTHER" and **must** not be `NULL`.

The XrSemanticLabelsSupportInfoFB structure **may** be specified in the `next` chain of XrSemanticLabelsFB to specify additional behaviors of the xrGetSpaceSemanticLabelsFB function. The runtime **must** follow the behaviors specified in `flags` according to the descriptions of XrSemanticLabelsSupportFlagBitsFB. The runtime **must** return any semantic label that is not included in `recognizedLabels` as "OTHER" to the application. The runtime **must** follow this direction only if the runtime reports the XrExtensionProperties::`extensionVersion` as 2 or greater, otherwise the runtime **must** ignore this as an unknown chained structure.

If the XrSemanticLabelsSupportInfoFB structure is not present in the `next` chain of XrSemanticLabelsFB, the runtime **may** return any semantic labels to the application.

Valid Usage (Implicit)

- The `XR_FB_scene` extension **must** be enabled prior to using XrSemanticLabelsSupportInfoFB
- `type` **must** be `XR_TYPE_SEMANTIC_LABELS_SUPPORT_INFO_FB`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `flags` **must** be `0` or a valid combination of XrSemanticLabelsSupportFlagBitsFB values
- `recognizedLabels` **must** be a null-terminated UTF-8 string

New Functions

The xrGetSpaceBoundingBox2DFB function is defined as:

```
// Provided by XR_FB_scene
XrResult xrGetSpaceBoundingBox2DFB(
    XrSession                                   session,
    XrSpace                                     space,
    XrRect2Df*                                  boundingBox2DOutput);
```

Parameter Descriptions

- `session` is a handle to an XrSession.
- `space` is the XrSpace handle to the spatial entity.
- `boundingBox2DOutput` is an output parameter pointing to the structure containing the 2D bounding box for `space`.

Gets the 2D bounding box for a spatial entity with the `XR_SPACE_COMPONENT_TYPE_BOUNDED_2D_FB` component type enabled.

The bounding box is defined by an XrRect2Df::`offset` and XrRect2Df::`extent`. The XrRect2Df::`offset` refers to the coordinate of the minimum corner of the box in the x-y plane of the given XrSpace's coordinate system; that is, the corner whose coordinate has the minimum value on each axis. The XrRect2Df::`extent` refers to the dimensions of the box along each axis. The maximum corner can therefore be computed as XrRect2Df::`offset` + XrRect2Df::`extent`.

Valid Usage (Implicit)

- The `XR_FB_scene` extension **must** be enabled prior to calling xrGetSpaceBoundingBox2DFB
- `session` **must** be a valid XrSession handle
- `space` **must** be a valid XrSpace handle
- `boundingBox2DOutput` **must** be a pointer to an XrRect2Df structure
- `space` **must** have been created, allocated, or retrieved from `session`

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FEATURE_UNSUPPORTED
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_SPACE_COMPONENT_NOT_ENABLED_FB
- XR_ERROR_VALIDATION_FAILURE

The xrGetSpaceBoundingBox3DFB function is defined as:

```
// Provided by XR_FB_scene
XrResult xrGetSpaceBoundingBox3DFB(
    XrSession                                   session,
    XrSpace                                     space,
    XrRect3DfFB*                                boundingBox3DOutput);
```

Parameter Descriptions

- session is a handle to an XrSession.
- space is the XrSpace handle to the spatial entity.
- boundingBox3DOutput is an output parameter pointing to the structure containing the 3D bounding box for space.

Gets the 3D bounding box for a spatial entity with the XR_SPACE_COMPONENT_TYPE_BOUNDED_3D_FB component type enabled.

Valid Usage (Implicit)

- The XR_FB_scene extension **must** be enabled prior to calling xrGetSpaceBoundingBox3DFB
- session **must** be a valid XrSession handle
- space **must** be a valid XrSpace handle
- boundingBox3DOutput **must** be a pointer to an XrRect3DfFB structure
- space **must** have been created, allocated, or retrieved from session

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FEATURE_UNSUPPORTED
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_SPACE_COMPONENT_NOT_ENABLED_FB
- XR_ERROR_VALIDATION_FAILURE

The xrGetSpaceSemanticLabelsFB function is defined as:

```
// Provided by XR_FB_scene
XrResult xrGetSpaceSemanticLabelsFB(
    XrSession                                   session,
    XrSpace                                     space,
    XrSemanticLabelsFB*                         semanticLabelsOutput);
```

Parameter Descriptions

- `session` is a handle to an XrSession.
- `space` is the XrSpace handle to the spatial entity.
- `semanticLabelsOutput` is an output parameter pointing to the structure containing the XrSemanticLabelsFB for `space`.

Gets the semantic labels for a spatial entity with the `XR_SPACE_COMPONENT_TYPE_SEMANTIC_LABELS_FB` component type enabled.

Valid Usage (Implicit)

- The `XR_FB_scene` extension **must** be enabled prior to calling xrGetSpaceSemanticLabelsFB
- `session` **must** be a valid XrSession handle
- `space` **must** be a valid XrSpace handle
- `semanticLabelsOutput` **must** be a pointer to an XrSemanticLabelsFB structure
- `space` **must** have been created, allocated, or retrieved from `session`

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_FEATURE_UNSUPPORTED`
- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_SIZE_INSUFFICIENT`
- `XR_ERROR_SPACE_COMPONENT_NOT_ENABLED_FB`
- `XR_ERROR_VALIDATION_FAILURE`

The xrGetSpaceBoundary2DFB function is defined as:

```
// Provided by XR_FB_scene
XrResult xrGetSpaceBoundary2DFB(
    XrSession                                   session,
    XrSpace                                     space,
    XrBoundary2DFB*                             boundary2DOutput);
```

Parameter Descriptions

- `session` is a handle to an XrSession.
- `space` is the XrSpace handle to the spatial entity.
- `boundary2DOutput` is an output parameter pointing to the structure containing the XrBoundary2DFB for `space`.

Gets the 2D boundary, specified by vertices, for a spatial entity with the `XR_SPACE_COMPONENT_TYPE_BOUNDED_2D_FB` component type enabled.

Valid Usage (Implicit)

- The `XR_FB_scene` extension **must** be enabled prior to calling xrGetSpaceBoundary2DFB
- `session` **must** be a valid XrSession handle
- `space` **must** be a valid XrSpace handle
- `boundary2DOutput` **must** be a pointer to an XrBoundary2DFB structure
- `space` **must** have been created, allocated, or retrieved from `session`

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FEATURE_UNSUPPORTED
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_SIZE_INSUFFICIENT
- XR_ERROR_SPACE_COMPONENT_NOT_ENABLED_FB
- XR_ERROR_VALIDATION_FAILURE

The xrGetSpaceRoomLayoutFB function is defined as:

```
// Provided by XR_FB_scene
XrResult xrGetSpaceRoomLayoutFB(
    XrSession                                   session,
    XrSpace                                     space,
    XrRoomLayoutFB*                             roomLayoutOutput);
```

Parameter Descriptions

- session is a handle to an XrSession.
- space is the XrSpace handle to the spatial entity.
- roomLayoutOutput is an output parameter pointing to the structure containing the XrRoomLayoutFB for space.

Gets the room layout, specified by UUIDs for each surface, for a spatial entity with the XR_SPACE_COMPONENT_TYPE_ROOM_LAYOUT_FB component type enabled.

If the XrRoomLayoutFB::wallUuidCapacityInput field is zero (indicating a request to retrieve the required capacity for the XrRoomLayoutFB::wallUuids array), or if xrGetSpaceRoomLayoutFB returns failure, then the values of floorUuid and ceilingUuid are unspecified and should not be used.

Valid Usage (Implicit)

- The XR_FB_scene extension **must** be enabled prior to calling xrGetSpaceRoomLayoutFB
- session **must** be a valid XrSession handle
- space **must** be a valid XrSpace handle
- roomLayoutOutput **must** be a pointer to an XrRoomLayoutFB structure
- space **must** have been created, allocated, or retrieved from session

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FEATURE_UNSUPPORTED
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_SIZE_INSUFFICIENT
- XR_ERROR_SPACE_COMPONENT_NOT_ENABLED_FB
- XR_ERROR_VALIDATION_FAILURE

Issues

Version History

- Revision 1, 2022-03-09 (John Schofield)
  - Initial draft
- Revision 2, 2023-04-03 (Yuichi Taguchi)
  - Introduce XrSemanticLabelsSupportInfoFB.
- Revision 3, 2023-04-03 (Yuichi Taguchi)
  - Introduce XR_SEMANTIC_LABELS_SUPPORT_ACCEPT_DESK_TO_TABLE_MIGRATION_BIT_FB.

- Revision 4, 2023-06-12 (Yuichi Taguchi)
    - Introduce `XR_SEMANTIC_LABELS_SUPPORT_ACCEPT_INVISIBLE_WALL_FACE_BIT_FB`.

12.120. XR_FB_scene_capture

Name String

`XR_FB_scene_capture`

Extension Type

Instance extension

Registered Extension Number

199

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Contributors

John Schofield, Facebook
Andrew Kim, Facebook
Yuichi Taguchi, Facebook
Cass Everitt, Facebook

Overview

This extension allows an application to request that the system begin capturing information about what is in the environment around the user.

New Object Types

New Flag Types

New Enum Constants

XrStructureType enumeration is extended with:

- `XR_TYPE_SCENE_CAPTURE_REQUEST_INFO_FB`
- `XR_TYPE_EVENT_DATA_SCENE_CAPTURE_COMPLETE_FB`

New Enums

New Structures

The XrSceneCaptureRequestInfoFB structure is defined as:

```
// Provided by XR_FB_scene_capture
typedef struct XrSceneCaptureRequestInfoFB {
    XrStructureType    type;
    const void*        next;
    uint32_t           requestByteCount;
    const char*        request;
} XrSceneCaptureRequestInfoFB;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `requestByteCount` is byte length of the `request` parameter.
- `request` is a string which the application **can** use to specify which type of scene capture should be initiated by the runtime. The contents of buffer pointed to by the `request` parameter is runtime-specific.

The XrSceneCaptureRequestInfoFB structure is used by an application to instruct the system what to look for during a scene capture. If the `request` parameter is `NULL`, then the runtime **must** conduct a default scene capture.

Valid Usage (Implicit)

- The `XR_FB_scene_capture` extension **must** be enabled prior to using XrSceneCaptureRequestInfoFB
- `type` **must** be `XR_TYPE_SCENE_CAPTURE_REQUEST_INFO_FB`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- If `requestByteCount` is not `0`, `request` **must** be a pointer to an array of `requestByteCount` char values

The XrEventDataSceneCaptureCompleteFB structure is defined as:

```
// Provided by XR_FB_scene_capture
typedef struct XrEventDataSceneCaptureCompleteFB {
    XrStructureType       type;
    const void*           next;
    XrAsyncRequestIdFB    requestId;
    XrResult              result;
} XrEventDataSceneCaptureCompleteFB;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `requestId` is the ID of the asynchronous query request.
- `result` is an XrResult that indicates if the request succeeded or if an error occurred.

The XrEventDataSceneCaptureCompleteFB structure is used by an application to instruct the system what to look for during a scene capture.

Valid Usage (Implicit)

- The `XR_FB_scene_capture` extension **must** be enabled prior to using XrEventDataSceneCaptureCompleteFB
- `type` **must** be `XR_TYPE_EVENT_DATA_SCENE_CAPTURE_COMPLETE_FB`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

New Functions

The xrRequestSceneCaptureFB function is defined as:

```
// Provided by XR_FB_scene_capture
XrResult xrRequestSceneCaptureFB(
    XrSession                                   session,
    const XrSceneCaptureRequestInfoFB*          info,
    XrAsyncRequestIdFB*                         requestId);
```

Parameter Descriptions

- `session` is a handle to an XrSession.
- `info` is an XrSceneCaptureRequestInfoFB which specifies how the scene capture should occur.
- `requestId` is the output parameter that points to the ID of this asynchronous request.

The xrRequestSceneCaptureFB function is used by an application to begin capturing the scene around the user. This is an asynchronous operation.

Valid Usage (Implicit)

- The `XR_FB_scene_capture` extension **must** be enabled prior to calling xrRequestSceneCaptureFB
- `session` **must** be a valid XrSession handle
- `info` **must** be a pointer to a valid XrSceneCaptureRequestInfoFB structure
- `requestId` **must** be a pointer to an `XrAsyncRequestIdFB` value

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_FEATURE_UNSUPPORTED`
- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_VALIDATION_FAILURE`

Issues

Version History

- Revision 1, 2022-03-09 (John Schofield)
    - Initial draft

12.121. XR_FB_spatial_entity

Name String

`XR_FB_spatial_entity`

Extension Type

Instance extension

Registered Extension Number

114

Revision

3

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Contributors

John Schofield, Facebook
Andrew Kim, Facebook
Yuichi Taguchi, Facebook
Cass Everitt, Facebook
Curtis Arink, Facebook

Overview

This extension enables applications to use spatial entities to specify world-locked frames of reference. It enables applications to persist the real world location of content over time and contains definitions for the Entity-Component System. All Facebook spatial entity and scene extensions are dependent on this one.

We use OpenXR XrSpace handles to give applications access to spatial entities such as Spatial Anchors. In other words, any operation which involves spatial entities uses XrSpace handles to identify the affected spatial entities.

This extension allows:

- An application to create a Spatial Anchor (a type of spatial entity).
- An application to enumerate supported components for a given spatial entity.

- An application to enable or disable a component for a given spatial entity.
- An application to get the status of a component for a given spatial entity.

New Object Types

New Flag Types

New Enum Constants

XrStructureType enumeration is extended with:

- XR_TYPE_SYSTEM_SPATIAL_ENTITY_PROPERTIES_FB
- XR_TYPE_SPATIAL_ANCHOR_CREATE_INFO_FB
- XR_TYPE_SPACE_COMPONENT_STATUS_SET_INFO_FB
- XR_TYPE_SPACE_COMPONENT_STATUS_FB
- XR_TYPE_EVENT_DATA_SPATIAL_ANCHOR_CREATE_COMPLETE_FB
- XR_TYPE_EVENT_DATA_SPACE_SET_STATUS_COMPLETE_FB

XrResult enumeration is extended with:

- XR_ERROR_SPACE_COMPONENT_NOT_SUPPORTED_FB
- XR_ERROR_SPACE_COMPONENT_NOT_ENABLED_FB
- XR_ERROR_SPACE_COMPONENT_STATUS_PENDING_FB
- XR_ERROR_SPACE_COMPONENT_STATUS_ALREADY_SET_FB

New Enums

```
// Provided by XR_FB_spatial_entity
typedef enum XrSpaceComponentTypeFB {
    XR_SPACE_COMPONENT_TYPE_LOCATABLE_FB = 0,
    XR_SPACE_COMPONENT_TYPE_STORABLE_FB = 1,
    XR_SPACE_COMPONENT_TYPE_SHARABLE_FB = 2,
    XR_SPACE_COMPONENT_TYPE_BOUNDED_2D_FB = 3,
    XR_SPACE_COMPONENT_TYPE_BOUNDED_3D_FB = 4,
    XR_SPACE_COMPONENT_TYPE_SEMANTIC_LABELS_FB = 5,
    XR_SPACE_COMPONENT_TYPE_ROOM_LAYOUT_FB = 6,
    XR_SPACE_COMPONENT_TYPE_SPACE_CONTAINER_FB = 7,
  // Provided by XR_META_spatial_entity_mesh
    XR_SPACE_COMPONENT_TYPE_TRIANGLE_MESH_META = 1000269000,
  // Provided by XR_META_spatial_entity_room_mesh
    XR_SPACE_COMPONENT_TYPE_ROOM_MESH_META = 1000553000,
    XR_SPACE_COMPONENT_TYPE_MAX_ENUM_FB = 0x7FFFFFFF
} XrSpaceComponentTypeFB;
```

Specify the component interfaces attached to the spatial entity.

Enumerant Descriptions

- XR_SPACE_COMPONENT_TYPE_LOCATABLE_FB — Enables tracking the 6 DOF pose of the XrSpace with xrLocateSpace.
- XR_SPACE_COMPONENT_TYPE_STORABLE_FB — Enables persistence operations: save and erase.
- XR_SPACE_COMPONENT_TYPE_SHARABLE_FB — Enables sharing of spatial entities.
- XR_SPACE_COMPONENT_TYPE_BOUNDED_2D_FB — Bounded 2D component.
- XR_SPACE_COMPONENT_TYPE_BOUNDED_3D_FB — Bounded 3D component.
- XR_SPACE_COMPONENT_TYPE_SEMANTIC_LABELS_FB — Semantic labels component.
- XR_SPACE_COMPONENT_TYPE_ROOM_LAYOUT_FB — Room layout component.
- XR_SPACE_COMPONENT_TYPE_SPACE_CONTAINER_FB — Space container component.
- XR_SPACE_COMPONENT_TYPE_TRIANGLE_MESH_META — Triangle mesh component. (Added by the XR_META_spatial_entity_mesh extension)
- XR_SPACE_COMPONENT_TYPE_ROOM_MESH_META — Room mesh component. (Added by the XR_META_spatial_entity_room_mesh extension)

New Base Types

The XrAsyncRequestIdFB base type is defined as:

```
// Provided by XR_FB_spatial_entity
XR_DEFINE_ATOM(XrAsyncRequestIdFB)
```

Represents a request to the spatial entity system. Several functions in this and other extensions will populate an output variable of this type so that an application **can** use it when referring to a specific request.

New Structures

The XrSystemSpatialEntityPropertiesFB structure is defined as:

```
// Provided by XR_FB_spatial_entity
typedef struct XrSystemSpatialEntityPropertiesFB {
    XrStructureType    type;
    const void*        next;
    XrBool32           supportsSpatialEntity;
} XrSystemSpatialEntityPropertiesFB;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `supportsSpatialEntity` is a boolean value that determines if spatial entities are supported by the system.

An application **can** inspect whether the system is capable of spatial entity operations by extending the XrSystemProperties with XrSystemSpatialEntityPropertiesFB structure when calling xrGetSystemProperties.

If a runtime returns `XR_FALSE` for `supportsSpatialEntity`, the runtime **must** return `XR_ERROR_FEATURE_UNSUPPORTED` from xrGetSpaceUuidFB.

Valid Usage (Implicit)

- The `XR_FB_spatial_entity` extension **must** be enabled prior to using XrSystemSpatialEntityPropertiesFB
- `type` **must** be `XR_TYPE_SYSTEM_SPATIAL_ENTITY_PROPERTIES_FB`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

The XrSpatialAnchorCreateInfoFB structure is defined as:

```
// Provided by XR_FB_spatial_entity
typedef struct XrSpatialAnchorCreateInfoFB {
    XrStructureType    type;
    const void*        next;
    XrSpace            space;
    XrPosef            poseInSpace;
    XrTime             time;
} XrSpatialAnchorCreateInfoFB;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `space` is the XrSpace handle to the reference space that defines the `poseInSpace` of the anchor to be defined.
- `poseInSpace` is the XrPosef location and orientation of the Spatial Anchor in the specified reference space.
- `time` is the `XrTime` timestamp associated with the specified pose.

Parameters to create a new spatial anchor.

Valid Usage (Implicit)

- The `XR_FB_spatial_entity` extension **must** be enabled prior to using XrSpatialAnchorCreateInfoFB
- `type` **must** be `XR_TYPE_SPATIAL_ANCHOR_CREATE_INFO_FB`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `space` **must** be a valid XrSpace handle

The XrSpaceComponentStatusSetInfoFB structure is defined as:

```
// Provided by XR_FB_spatial_entity
typedef struct XrSpaceComponentStatusSetInfoFB {
    XrStructureType           type;
    const void*               next;
    XrSpaceComponentTypeFB    componentType;
    XrBool32                  enabled;
    XrDuration                timeout;
} XrSpaceComponentStatusSetInfoFB;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `componentType` is the component whose status is to be set.
- `enabled` is the value to set the component to.
- `timeout` is the number of nanoseconds before the operation should be cancelled. A value of XR_INFINITE_DURATION indicates to never time out. See Duration for more details.

Enables or disables the specified component for the specified spatial entity.

Valid Usage (Implicit)

- The `XR_FB_spatial_entity` extension **must** be enabled prior to using XrSpaceComponentStatusSetInfoFB
- `type` **must** be `XR_TYPE_SPACE_COMPONENT_STATUS_SET_INFO_FB`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `componentType` **must** be a valid XrSpaceComponentTypeFB value

The XrSpaceComponentStatusFB structure is defined as:

```
// Provided by XR_FB_spatial_entity
typedef struct XrSpaceComponentStatusFB {
    XrStructureType    type;
    void*              next;
    XrBool32           enabled;
    XrBool32           changePending;
} XrSpaceComponentStatusFB;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `enabled` is a boolean value that determines if a component is currently enabled or disabled.
- `changePending` is a boolean value that determines if the component's enabled state is about to change.

It holds information on the current state of a component.

Valid Usage (Implicit)

- The `XR_FB_spatial_entity` extension **must** be enabled prior to using XrSpaceComponentStatusFB
- `type` **must** be `XR_TYPE_SPACE_COMPONENT_STATUS_FB`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

The XrEventDataSpatialAnchorCreateCompleteFB structure is defined as:

```
// Provided by XR_FB_spatial_entity
typedef struct XrEventDataSpatialAnchorCreateCompleteFB {
    XrStructureType       type;
    const void*           next;
    XrAsyncRequestIdFB    requestId;
    XrResult              result;
    XrSpace               space;
    XrUuidEXT             uuid;
} XrEventDataSpatialAnchorCreateCompleteFB;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `requestId` is the ID of the asynchronous request used to create a new spatial anchor.
- `result` is an XrResult that determines if the request succeeded or if an error occurred.
- `space` is the XrSpace handle to the newly created spatial anchor.
- `uuid` is the UUID of the newly created spatial anchor.

It describes the result of a request to create a new spatial anchor. Once this event is posted, it is the applications responsibility to take ownership of the XrSpace. The XrSession passed into xrCreateSpatialAnchorFB is the parent handle of the newly created XrSpace.

Valid Usage (Implicit)

- The `XR_FB_spatial_entity` extension **must** be enabled prior to using XrEventDataSpatialAnchorCreateCompleteFB
- `type` **must** be `XR_TYPE_EVENT_DATA_SPATIAL_ANCHOR_CREATE_COMPLETE_FB`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

The XrEventDataSpaceSetStatusCompleteFB structure is defined as:

```
// Provided by XR_FB_spatial_entity
typedef struct XrEventDataSpaceSetStatusCompleteFB {
    XrStructureType           type;
    const void*               next;
    XrAsyncRequestIdFB        requestId;
    XrResult                  result;
    XrSpace                   space;
    XrUuidEXT                 uuid;
    XrSpaceComponentTypeFB    componentType;
    XrBool32                  enabled;
} XrEventDataSpaceSetStatusCompleteFB;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `requestId` is the ID of the asynchronous request used to enable or disable a component.
- `result` is an XrResult that describes whether the request succeeded or if an error occurred.
- `space` is the XrSpace handle to the spatial entity.
- `uuid` is the UUID of the spatial entity.
- `componentType` is the type of component being enabled or disabled.
- `enabled` is a boolean value indicating whether the component is now enabled or disabled.

It describes the result of a request to enable or disable a component of a spatial entity.

Valid Usage (Implicit)

- The `XR_FB_spatial_entity` extension **must** be enabled prior to using XrEventDataSpaceSetStatusCompleteFB
- `type` **must** be `XR_TYPE_EVENT_DATA_SPACE_SET_STATUS_COMPLETE_FB`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

New Functions

The xrCreateSpatialAnchorFB function is defined as:

```
// Provided by XR_FB_spatial_entity
XrResult xrCreateSpatialAnchorFB(
    XrSession                                   session,
    const XrSpatialAnchorCreateInfoFB*          info,
    XrAsyncRequestIdFB*                         requestId);
```

Parameter Descriptions

- `session` is a handle to an XrSession.
- `info` is a pointer to an XrSpatialAnchorCreateInfoFB structure containing information about how to create the anchor.
- `requestId` is the output parameter that points to the ID of this asynchronous request.

Creates a Spatial Anchor using the specified tracking origin and pose relative to the specified tracking origin. The anchor will be locatable at the time of creation, and the 6 DOF pose relative to the tracking origin **can** be queried using the xrLocateSpace method. This operation is asynchronous and the runtime **must** post an XrEventDataSpatialAnchorCreateCompleteFB event when the operation completes successfully or encounters an error. If this function returns a failure code, no event is posted. The `requestId` **can** be used to later refer to the request, such as identifying which request has completed when an XrEventDataSpatialAnchorCreateCompleteFB is posted to the event queue.

Valid Usage (Implicit)

- The `XR_FB_spatial_entity` extension **must** be enabled prior to calling xrCreateSpatialAnchorFB
- `session` **must** be a valid XrSession handle
- `info` **must** be a pointer to a valid XrSpatialAnchorCreateInfoFB structure
- `requestId` **must** be a pointer to an `XrAsyncRequestIdFB` value

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_FEATURE_UNSUPPORTED`
- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_LIMIT_REACHED`
- `XR_ERROR_OUT_OF_MEMORY`
- `XR_ERROR_POSE_INVALID`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_TIME_INVALID`
- `XR_ERROR_VALIDATION_FAILURE`

The xrGetSpaceUuidFB function is defined as:

```
// Provided by XR_FB_spatial_entity
XrResult xrGetSpaceUuidFB(
    XrSpace                                     space,
    XrUuidEXT*                                  uuid);
```

Parameter Descriptions

- `space` is the XrSpace handle of a spatial entity.
- `uuid` is an output parameter pointing to the entity's UUID.

Gets the UUID for a spatial entity. If this space was previously created as a spatial anchor, `uuid` **must** be equal to the XrEventDataSpatialAnchorCreateCompleteFB::`uuid` in the event corresponding to the creation of that space. Subsequent calls to xrGetSpaceUuidFB using the same XrSpace **must** return the same XrUuidEXT.

Valid Usage (Implicit)

- The `XR_FB_spatial_entity` extension **must** be enabled prior to calling xrGetSpaceUuidFB
- `space` **must** be a valid XrSpace handle
- `uuid` **must** be a pointer to an XrUuidEXT structure

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_FEATURE_UNSUPPORTED`
- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_VALIDATION_FAILURE`

The xrEnumerateSpaceSupportedComponentsFB function is defined as:

```
// Provided by XR_FB_spatial_entity
XrResult xrEnumerateSpaceSupportedComponentsFB(
    XrSpace                                     space,
    uint32_t                                    componentTypeCapacityInput,
    uint32_t*                                   componentTypeCountOutput,
    XrSpaceComponentTypeFB*                     componentTypes);
```

Parameter Descriptions

- `space` is the XrSpace handle to the spatial entity.
- `componentTypeCapacityInput` is the capacity of the `componentTypes` array, or 0 to indicate a request to retrieve the required capacity.
- `componentTypeCountOutput` is a pointer to the count of `componentTypes` written, or a pointer to the required capacity in the case that `componentTypeCapacityInput` is insufficient.
- `componentTypes` is a pointer to an array of XrSpaceComponentTypeFB values, but **can** be `NULL` if `componentTypeCapacityInput` is `0`.
- See the Buffer Size Parameters section for a detailed description of retrieving the required `componentTypes` size.

Lists any component types that an entity supports. The list of component types available for an entity depends on which extensions are enabled. Component types **must** not be enumerated unless the corresponding extension that defines them is also enabled.

Valid Usage (Implicit)

- The `XR_FB_spatial_entity` extension **must** be enabled prior to calling xrEnumerateSpaceSupportedComponentsFB
- `space` **must** be a valid XrSpace handle
- `componentTypeCountOutput` **must** be a pointer to a `uint32_t` value
- If `componentTypeCapacityInput` is not `0`, `componentTypes` **must** be a pointer to an array of `componentTypeCapacityInput` XrSpaceComponentTypeFB values

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FEATURE_UNSUPPORTED
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_SIZE_INSUFFICIENT
- XR_ERROR_VALIDATION_FAILURE

The xrSetSpaceComponentStatusFB function is defined as:

```
// Provided by XR_FB_spatial_entity
XrResult xrSetSpaceComponentStatusFB(
    XrSpace                                     space,
    const XrSpaceComponentStatusSetInfoFB*      info,
    XrAsyncRequestIdFB*                         requestId);
```

Parameter Descriptions

- space is the XrSpace handle to the spatial entity.
- info is a pointer to an XrSpaceComponentStatusSetInfoFB structure containing information about the component to be enabled or disabled.
- requestId is the output parameter that points to the ID of this asynchronous request.

Enables or disables the specified component for the specified entity. This operation is asynchronous and always returns immediately, regardless of the value of XrSpaceComponentStatusSetInfoFB ::timeout. The requestId **can** be used to later refer to the request, such as identifying which request has completed when an XrEventDataSpaceSetStatusCompleteFB is posted to the event queue. If this function returns a failure code, no event is posted. This function **must** return XR_ERROR_SPACE_COMPONENT_NOT_SUPPORTED_FB if the XrSpace does not support the specified component type.

Valid Usage (Implicit)

- The XR_FB_spatial_entity extension **must** be enabled prior to calling xrSetSpaceComponentStatusFB
- space **must** be a valid XrSpace handle
- info **must** be a pointer to a valid XrSpaceComponentStatusSetInfoFB structure
- requestId **must** be a pointer to an XrAsyncRequestIdFB value

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FEATURE_UNSUPPORTED
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_SPACE_COMPONENT_NOT_SUPPORTED_FB
- XR_ERROR_SPACE_COMPONENT_STATUS_ALREADY_SET_FB
- XR_ERROR_SPACE_COMPONENT_STATUS_PENDING_FB
- XR_ERROR_VALIDATION_FAILURE

The xrGetSpaceComponentStatusFB function is defined as:

```
// Provided by XR_FB_spatial_entity
XrResult xrGetSpaceComponentStatusFB(
    XrSpace                                     space,
    XrSpaceComponentTypeFB                      componentType,
    XrSpaceComponentStatusFB*                   status);
```

Parameter Descriptions

- `space` is the XrSpace handle of a spatial entity.
- `componentType` is the component type to query.
- `status` is an output parameter pointing to the structure containing the status of the component that was queried.

Gets the current status of the specified component for the specified entity. This function **must** return `XR_ERROR_SPACE_COMPONENT_NOT_SUPPORTED_FB` if the XrSpace does not support the specified component type.

Valid Usage (Implicit)

- The `XR_FB_spatial_entity` extension **must** be enabled prior to calling xrGetSpaceComponentStatusFB
- `space` **must** be a valid XrSpace handle
- `componentType` **must** be a valid XrSpaceComponentTypeFB value
- `status` **must** be a pointer to an XrSpaceComponentStatusFB structure

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_FEATURE_UNSUPPORTED`
- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_SPACE_COMPONENT_NOT_SUPPORTED_FB`
- `XR_ERROR_VALIDATION_FAILURE`

Issues

Version History

- Revision 1, 2022-01-22 (John Schofield)
  - Initial draft
- Revision 2, 2023-01-18 (Andrew Kim)
  - Added a new component enum value
- Revision 3, 2023-01-30 (Wenlin Mao)
  - Drop requirement for `XR_EXT_uuid` must be enabled

12.122. XR_FB_spatial_entity_container

Name String

`XR_FB_spatial_entity_container`

Extension Type

Instance extension

Registered Extension Number

200

Revision

2

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0
and
`XR_FB_spatial_entity`

Contributors

John Schofield, Facebook
Andrew Kim, Facebook
Yuichi Taguchi, Facebook

Overview

This extension expands on the concept of spatial entities to include a way for one spatial entity to contain multiple child spatial entities, forming a hierarchy.

New Object Types

New Flag Types

New Enum Constants

XrStructureType enumeration is extended with:

- XR_TYPE_SPACE_CONTAINER_FB

New Enums

New Structures

The XrSpaceContainerFB structure is defined as:

```
// Provided by XR_FB_spatial_entity_container
typedef struct XrSpaceContainerFB {
    XrStructureType    type;
    const void*        next;
    uint32_t           uuidCapacityInput;
    uint32_t           uuidCountOutput;
    XrUuidEXT*         uuids;
} XrSpaceContainerFB;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- uuidCapacityInput is the capacity of the uuids array, or 0 to indicate a request to retrieve the required capacity.
- uuidCountOutput is an output parameter which will hold the number of UUIDs included in the output list, or the required capacity in the case that uuidCapacityInput is insufficient
- uuids is an output parameter which will hold a list of space UUIDs contained by the space to which the component is attached.
- See the Buffer Size Parameters section for a detailed description of retrieving the required uuids size.

The XrSpaceContainerFB structure **can** be used by an application to perform the two calls required to obtain information about which spatial entities are contained by a specified spatial entity.

Valid Usage (Implicit)

- The XR_FB_spatial_entity_container extension **must** be enabled prior to using XrSpaceContainerFB
- type **must** be XR_TYPE_SPACE_CONTAINER_FB
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- If uuidCapacityInput is not 0, uuids **must** be a pointer to an array of uuidCapacityInput XrUuidEXT structures

New Functions

The xrGetSpaceContainerFB function is defined as:

```
// Provided by XR_FB_spatial_entity_container
XrResult xrGetSpaceContainerFB(
    XrSession                                   session,
    XrSpace                                     space,
    XrSpaceContainerFB*                         spaceContainerOutput);
```

Parameter Descriptions

- session is a handle to an XrSession.
- space is a handle to an XrSpace.
- spaceContainerOutput is the output parameter that points to an XrSpaceContainerFB containing information about which spaces are contained by space.

The xrGetSpaceContainerFB function is used by an application to perform the two calls required to obtain information about which spatial entities are contained by a specified spatial entity.

The XR_SPACE_COMPONENT_TYPE_SPACE_CONTAINER_FB component type **must** be enabled, otherwise this function will return XR_ERROR_SPACE_COMPONENT_NOT_ENABLED_FB.

Valid Usage (Implicit)

- The `XR_FB_spatial_entity_container` extension **must** be enabled prior to calling xrGetSpaceContainerFB
- `session` **must** be a valid XrSession handle
- `space` **must** be a valid XrSpace handle
- `spaceContainerOutput` **must** be a pointer to an XrSpaceContainerFB structure
- `space` **must** have been created, allocated, or retrieved from `session`

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_FEATURE_UNSUPPORTED`
- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_SIZE_INSUFFICIENT`
- `XR_ERROR_SPACE_COMPONENT_NOT_ENABLED_FB`
- `XR_ERROR_VALIDATION_FAILURE`

Issues

Version History

- Revision 1, 2022-03-09 (John Schofield)
  - Initial draft
- Revision 2, 2022-05-31 (John Schofield)
  - Fix types of `XrSpaceContainerFB` fields.

12.123. XR_FB_spatial_entity_query

Name String

`XR_FB_spatial_entity_query`

Extension Type

Instance extension

Registered Extension Number

157

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0
and
`XR_FB_spatial_entity_storage`

Contributors

John Schofield, Facebook
Andrew Kim, Facebook
Yuichi Taguchi, Facebook
Cass Everitt, Facebook
Curtis Arink, Facebook

Overview

This extension enables an application to discover persistent spatial entities in the area and restore them. Using the query system, the application **can** load persistent spatial entities from storage. The query system consists of a set of filters to define the spatial entity search query and an operation that needs to be performed on the search results.

New Object Types

New Flag Types

New Enum Constants

XrStructureType enumeration is extended with:

- `XR_TYPE_SPACE_QUERY_INFO_FB`
- `XR_TYPE_SPACE_QUERY_RESULTS_FB`

- XR_TYPE_SPACE_STORAGE_LOCATION_FILTER_INFO_FB
- XR_TYPE_SPACE_UUID_FILTER_INFO_FB
- XR_TYPE_SPACE_COMPONENT_FILTER_INFO_FB
- XR_TYPE_EVENT_DATA_SPACE_QUERY_RESULTS_AVAILABLE_FB
- XR_TYPE_EVENT_DATA_SPACE_QUERY_COMPLETE_FB

New Enums

```
// Provided by XR_FB_spatial_entity_query
typedef enum XrSpaceQueryActionFB {
    XR_SPACE_QUERY_ACTION_LOAD_FB = 0,
    XR_SPACE_QUERY_ACTION_MAX_ENUM_FB = 0x7FFFFFFF
} XrSpaceQueryActionFB;
```

Specify the type of query being performed.

Enumerant Descriptions

- XR_SPACE_QUERY_ACTION_LOAD_FB — Tells the query to perform a load operation on any XrSpace returned by the query.

New Structures

The XrSpaceQueryInfoBaseHeaderFB structure is defined as:

```
// Provided by XR_FB_spatial_entity_query
typedef struct XrSpaceQueryInfoBaseHeaderFB {
    XrStructureType    type;
    const void*        next;
} XrSpaceQueryInfoBaseHeaderFB;
```

Member Descriptions

- type is the XrStructureType of this structure. This base structure itself has no associated XrStructureType value.
- next is NULL or a pointer to the next structure in a structure chain. This base structure itself has no associated XrStructureType value.

The XrSpaceQueryInfoBaseHeaderFB is a base structure that is not intended to be directly used, but forms a basis for specific query info types. All query info structures begin with the elements described

in the XrSpaceQueryInfoBaseHeaderFB, and a query info pointer **must** be cast to a pointer to XrSpaceQueryInfoBaseHeaderFB when passing it to the xrQuerySpacesFB function.

Valid Usage (Implicit)

- The XR_FB_spatial_entity_query extension **must** be enabled prior to using XrSpaceQueryInfoBaseHeaderFB
- `type` **must** be `XR_TYPE_SPACE_QUERY_INFO_FB`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain. See also: XrSpaceGroupUuidFilterInfoMETA

The XrSpaceFilterInfoBaseHeaderFB structure is defined as:

```
// Provided by XR_FB_spatial_entity_query
typedef struct XrSpaceFilterInfoBaseHeaderFB {
    XrStructureType    type;
    const void*        next;
} XrSpaceFilterInfoBaseHeaderFB;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. This base structure itself has no associated XrStructureType value.

The XrSpaceFilterInfoBaseHeaderFB is a base structure that is not intended to be directly used, but forms a basis for specific filter info types. All filter info structures begin with the elements described in the XrSpaceFilterInfoBaseHeaderFB, and a filter info pointer **must** be cast to a pointer to XrSpaceFilterInfoBaseHeaderFB when populating XrSpaceQueryInfoFB::`filter` and XrSpaceQueryInfoFB::`excludeFilter` to pass to the xrQuerySpacesFB function.

Valid Usage (Implicit)

- The XR_FB_spatial_entity_query extension **must** be enabled prior to using XrSpaceFilterInfoBaseHeaderFB
- `type` **must** be one of the following XrStructureType values: `XR_TYPE_SPACE_COMPONENT_FILTER_INFO_FB`, `XR_TYPE_SPACE_UUID_FILTER_INFO_FB`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain. See also: XrSpaceStorageLocationFilterInfoFB

The XrSpaceQueryInfoFB structure is defined as:

```
// Provided by XR_FB_spatial_entity_query
typedef struct XrSpaceQueryInfoFB {
    XrStructureType                         type;
    const void*                             next;
    XrSpaceQueryActionFB                    queryAction;
    uint32_t                                maxResultCount;
    XrDuration                              timeout;
    const XrSpaceFilterInfoBaseHeaderFB*    filter;
    const XrSpaceFilterInfoBaseHeaderFB*    excludeFilter;
} XrSpaceQueryInfoFB;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `queryAction` is the type of query to perform.
- `maxResultCount` is the maximum number of entities to be found.
- `timeout` is the number of nanoseconds before the operation should time out. A value of XR_INFINITE_DURATION indicates no timeout.
- `filter` is NULL or a pointer to a valid structure based on XrSpaceFilterInfoBaseHeaderFB.
- `excludeFilter` is NULL or a pointer to a valid structure based on XrSpaceFilterInfoBaseHeaderFB.

May be used to query for spaces and perform a specific action on the spaces returned. The available actions are enumerated in XrSpaceQueryActionFB. The filter info provided to the `filter` member of the struct is used as an inclusive filter. The filter info provided to the `excludeFilter` member of the structure is used to exclude spaces from the results returned from the filter. All spaces that match the criteria in `filter`, and that do not match the criteria in `excludeFilter`, **must** be included in the results returned. This is to allow for a more selective style query.

Valid Usage (Implicit)

- The `XR_FB_spatial_entity_query` extension **must** be enabled prior to using XrSpaceQueryInfoFB
- `type` **must** be `XR_TYPE_SPACE_QUERY_INFO_FB`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `queryAction` **must** be a valid XrSpaceQueryActionFB value
- If `filter` is not `NULL`, `filter` **must** be a pointer to a valid XrSpaceFilterInfoBaseHeaderFB -based structure. See also: XrSpaceComponentFilterInfoFB, XrSpaceUuidFilterInfoFB
- If `excludeFilter` is not `NULL`, `excludeFilter` **must** be a pointer to a valid XrSpaceFilterInfoBaseHeaderFB-based structure. See also: XrSpaceComponentFilterInfoFB, XrSpaceUuidFilterInfoFB

The XrSpaceStorageLocationFilterInfoFB structure is defined as:

```
// Provided by XR_FB_spatial_entity_query
typedef struct XrSpaceStorageLocationFilterInfoFB {
    XrStructureType             type;
    const void*                 next;
    XrSpaceStorageLocationFB    location;
} XrSpaceStorageLocationFilterInfoFB;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `location` is the location to limit the query to.

Extends a query filter to limit a query to a specific storage location. Set the `next` pointer of an XrSpaceFilterInfoBaseHeaderFB to chain this extra filtering functionality.

Valid Usage (Implicit)

- The XR_FB_spatial_entity_query extension **must** be enabled prior to using XrSpaceStorageLocationFilterInfoFB
- type **must** be XR_TYPE_SPACE_STORAGE_LOCATION_FILTER_INFO_FB
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- location **must** be a valid XrSpaceStorageLocationFB value

The XrSpaceUuidFilterInfoFB structure is defined as:

```
// Provided by XR_FB_spatial_entity_query
typedef struct XrSpaceUuidFilterInfoFB {
    XrStructureType    type;
    const void*        next;
    uint32_t           uuidCount;
    XrUuidEXT*         uuids;
} XrSpaceUuidFilterInfoFB;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- uuidCount is the number of UUIDs to be matched.
- uuids is an array of XrUuidEXT that contains the UUIDs to be matched.

The XrSpaceUuidFilterInfoFB structure is a filter an application **can** use to find XrSpace entities that match specified UUIDs, to include or exclude them from a query.

Valid Usage (Implicit)

- The XR_FB_spatial_entity_query extension **must** be enabled prior to using XrSpaceUuidFilterInfoFB
- type **must** be XR_TYPE_SPACE_UUID_FILTER_INFO_FB
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- uuids **must** be a pointer to an array of uuidCount XrUuidEXT structures
- The uuidCount parameter **must** be greater than 0

The XrSpaceComponentFilterInfoFB structure is defined as:

```
// Provided by XR_FB_spatial_entity_query
typedef struct XrSpaceComponentFilterInfoFB {
    XrStructureType           type;
    const void*               next;
    XrSpaceComponentTypeFB    componentType;
} XrSpaceComponentFilterInfoFB;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `componentType` is the XrSpaceComponentTypeFB to query for.

The XrSpaceComponentFilterInfoFB structure is a filter an application **can** use to find XrSpace entities which have the `componentType` enabled, to include or exclude them from a query.

Valid Usage (Implicit)

- The `XR_FB_spatial_entity_query` extension **must** be enabled prior to using XrSpaceComponentFilterInfoFB
- `type` **must** be `XR_TYPE_SPACE_COMPONENT_FILTER_INFO_FB`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `componentType` **must** be a valid XrSpaceComponentTypeFB value

The XrSpaceQueryResultFB structure is defined as:

```
// Provided by XR_FB_spatial_entity_query
typedef struct XrSpaceQueryResultFB {
    XrSpace      space;
    XrUuidEXT    uuid;
} XrSpaceQueryResultFB;
```

Member Descriptions

- `space` is the XrSpace handle to the spatial entity found by the query.
- `uuid` is the UUID that identifies the entity.

The XrSpaceQueryResultFB structure is a query result returned in the xrRetrieveSpaceQueryResultsFB::`results` output parameter of the xrRetrieveSpaceQueryResultsFB function.

Valid Usage (Implicit)

- The `XR_FB_spatial_entity_query` extension **must** be enabled prior to using XrSpaceQueryResultFB

The XrSpaceQueryResultsFB structure is defined as:

```
// Provided by XR_FB_spatial_entity_query
typedef struct XrSpaceQueryResultsFB {
    XrStructureType          type;
    void*                    next;
    uint32_t                 resultCapacityInput;
    uint32_t                 resultCountOutput;
    XrSpaceQueryResultFB*    results;
} XrSpaceQueryResultsFB;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `resultCapacityInput` is the capacity of the `results` array, or 0 to indicate a request to retrieve the required capacity.
- `resultCountOutput` is an output parameter containing the count of results retrieved, or returns the required capacity in the case that `resultCapacityInput` is insufficient.
- `results` is a pointer to an array of results, but **can** be `NULL` if `resultCapacityInput` is 0.
- See the Buffer Size Parameters section for a detailed description of retrieving the required `results` size.

The XrSpaceQueryResultsFB structure is used by the xrRetrieveSpaceQueryResultsFB function to

retrieve query results.

Valid Usage (Implicit)

- The XR_FB_spatial_entity_query extension **must** be enabled prior to using XrSpaceQueryResultsFB
- `type` **must** be `XR_TYPE_SPACE_QUERY_RESULTS_FB`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- If `resultCapacityInput` is not `0`, `results` **must** be a pointer to an array of `resultCapacityInput` XrSpaceQueryResultFB structures

The XrEventDataSpaceQueryResultsAvailableFB structure is defined as:

```
// Provided by XR_FB_spatial_entity_query
typedef struct XrEventDataSpaceQueryResultsAvailableFB {
    XrStructureType       type;
    const void*           next;
    XrAsyncRequestIdFB    requestId;
} XrEventDataSpaceQueryResultsAvailableFB;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `requestId` is the ID of the asynchronous query request.

It indicates a query request has produced some number of results. If a query yields results this event **must** be delivered before the XrEventDataSpaceQueryCompleteFB event is delivered. Call xrRetrieveSpaceQueryResultsFB to retrieve those results.

Valid Usage (Implicit)

- The XR_FB_spatial_entity_query extension **must** be enabled prior to using XrEventDataSpaceQueryResultsAvailableFB
- `type` **must** be `XR_TYPE_EVENT_DATA_SPACE_QUERY_RESULTS_AVAILABLE_FB`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

The XrEventDataSpaceQueryCompleteFB structure is defined as:

```
// Provided by XR_FB_spatial_entity_query
typedef struct XrEventDataSpaceQueryCompleteFB {
    XrStructureType       type;
    const void*           next;
    XrAsyncRequestIdFB    requestId;
    XrResult              result;
} XrEventDataSpaceQueryCompleteFB;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `requestId` is the ID of the asynchronous query request.
- `result` is an XrResult that determines if the request succeeded or if an error occurred.

It indicates a query request has completed and specifies the request result. This event **must** be delivered when a query has completed, regardless of the number of results found. If any results have been found, then this event **must** be delivered after any XrEventDataSpaceQueryResultsAvailableFB events have been delivered.

Valid Usage (Implicit)

- The `XR_FB_spatial_entity_query` extension **must** be enabled prior to using XrEventDataSpaceQueryCompleteFB
- `type` **must** be `XR_TYPE_EVENT_DATA_SPACE_QUERY_COMPLETE_FB`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

New Functions

The xrQuerySpacesFB function is defined as:

```
// Provided by XR_FB_spatial_entity_query
XrResult xrQuerySpacesFB(
    XrSession                                   session,
    const XrSpaceQueryInfoBaseHeaderFB*         info,
    XrAsyncRequestIdFB*                         requestId);
```

Parameter Descriptions

- `session` is a handle to an XrSession.
- `info` is a pointer to the XrSpaceQueryInfoBaseHeaderFB structure.
- `requestId` is an output parameter, and the variable it points to will be populated with the ID of this asynchronous request.

The xrQuerySpacesFB function enables an application to find and retrieve spatial entities from storage. Cast an XrSpaceQueryInfoFB pointer to a XrSpaceQueryInfoBaseHeaderFB pointer to pass as `info`. The application **should** keep the returned `requestId` for the duration of the request as it is used to refer to the request when calling xrRetrieveSpaceQueryResultsFB and is used to map completion events to the request. This operation is asynchronous and the runtime **must** post an XrEventDataSpaceQueryCompleteFB event when the operation completes successfully or encounters an error. If this function returns a failure code, no event is posted. The runtime **must** post an XrEventDataSpaceQueryResultsAvailableFB before XrEventDataSpaceQueryCompleteFB if any results are found. Once an XrEventDataSpaceQueryResultsAvailableFB event has been posted, the application **may** call xrRetrieveSpaceQueryResultsFB to retrieve the available results.

Valid Usage (Implicit)

- The `XR_FB_spatial_entity_query` extension **must** be enabled prior to calling xrQuerySpacesFB
- `session` **must** be a valid XrSession handle
- `info` **must** be a pointer to a valid XrSpaceQueryInfoBaseHeaderFB-based structure. See also: XrSpaceQueryInfoFB
- `requestId` **must** be a pointer to an `XrAsyncRequestIdFB` value

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FEATURE_UNSUPPORTED
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The xrRetrieveSpaceQueryResultsFB function is defined as:

```
// Provided by XR_FB_spatial_entity_query
XrResult xrRetrieveSpaceQueryResultsFB(
    XrSession                                   session,
    XrAsyncRequestIdFB                          requestId,
    XrSpaceQueryResultsFB*                      results);
```

Parameter Descriptions

- session is the XrSession for which the in-progress query is valid.
- requestId is the XrAsyncRequestIdFB to enumerate results for.
- results is a pointer to an XrSpaceQueryResultsFB to populate with results.
- See the Buffer Size Parameters section for a detailed description of retrieving the required size of the results in this parameter.

Allows an application to retrieve all available results for a specified query. Call this function once to get the number of results found and then once more to copy the results into a buffer provided by the application. The number of results will not change between the two calls used to retrieve results. This function **must** only retrieve each query result once. After the application has used this function to retrieve a query result, the runtime frees its copy. The runtime **must** return XR_ERROR_VALIDATION_FAILURE if requestId refers to a request that is not yet complete, a request for which results have already been retrieved, or if requestId does not refer to a known request.

Valid Usage (Implicit)

- The XR_FB_spatial_entity_query extension **must** be enabled prior to calling xrRetrieveSpaceQueryResultsFB
- `session` **must** be a valid XrSession handle
- `results` **must** be a pointer to an XrSpaceQueryResultsFB structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FEATURE_UNSUPPORTED
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_SIZE_INSUFFICIENT
- XR_ERROR_VALIDATION_FAILURE

Issues

Version History

- Revision 1, 2022-01-22 (John Schofield)
  - Initial draft

12.124. XR_FB_spatial_entity_sharing

Name String

XR_FB_spatial_entity_sharing

Extension Type

Instance extension

Registered Extension Number

170

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0
and
XR_FB_spatial_entity

Contributors

John Schofield, Facebook
Andrew Kim, Facebook

Overview

This extension enables spatial entities to be shared between users. If the XR_SPACE_COMPONENT_TYPE_SHARABLE_FB component has been enabled on the spatial entity, application developers **may** share XrSpace entities between users.

New Object Types

New Flag Types

New Enum Constants

XrStructureType enumeration is extended with:

- XR_TYPE_SPACE_SHARE_INFO_FB
- XR_TYPE_EVENT_DATA_SPACE_SHARE_COMPLETE_FB

XrResult enumeration is extended with:

- XR_ERROR_SPACE_MAPPING_INSUFFICIENT_FB
- XR_ERROR_SPACE_LOCALIZATION_FAILED_FB
- XR_ERROR_SPACE_NETWORK_TIMEOUT_FB
- XR_ERROR_SPACE_NETWORK_REQUEST_FAILED_FB
- XR_ERROR_SPACE_CLOUD_STORAGE_DISABLED_FB

New Enums

New Base Types

New Structures

The XrSpaceShareInfoFB structure is defined as:

```
// Provided by XR_FB_spatial_entity_sharing
typedef struct XrSpaceShareInfoFB {
    XrStructureType    type;
    const void*        next;
    uint32_t           spaceCount;
    XrSpace*           spaces;
    uint32_t           userCount;
    XrSpaceUserFB*     users;
} XrSpaceShareInfoFB;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension. `spaceCount` is the number of elements in the `spaces` list. `spaces` is a list containing all spatial entities to be shared. `userCount` is the number of elements in the `users` list. `users` is a list of the users with which the `spaces` will: be shared.

The XrSpaceShareInfoFB structure describes a request to share one or more spatial entities with one or more users.

Valid Usage (Implicit)

- The `XR_FB_spatial_entity_sharing` extension **must** be enabled prior to using XrSpaceShareInfoFB
- `type` **must** be `XR_TYPE_SPACE_SHARE_INFO_FB`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `spaces` **must** be a pointer to an array of `spaceCount` XrSpace handles
- `users` **must** be a pointer to an array of `userCount` XrSpaceUserFB handles
- The `spaceCount` parameter **must** be greater than `0`
- The `userCount` parameter **must** be greater than `0`

The XrEventDataSpaceShareCompleteFB structure is defined as:

```
// Provided by XR_FB_spatial_entity_sharing
typedef struct XrEventDataSpaceShareCompleteFB {
    XrStructureType       type;
    const void*           next;
    XrAsyncRequestIdFB    requestId;
    XrResult              result;
} XrEventDataSpaceShareCompleteFB;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `requestId` is the ID of the asynchronous request used to share the spatial entities.
- `result` is an XrResult that describes whether the request succeeded or if an error occurred.

It indicates that the request to share one or more spatial entities has completed. The application **can** use `result` to check if the request was successful or if an error occurred.

Result Codes

Success

- `XR_SUCCESS`

Failure

- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SPACE_MAPPING_INSUFFICIENT_FB`
- `XR_ERROR_SPACE_LOCALIZATION_FAILED_FB`
- `XR_ERROR_SPACE_NETWORK_TIMEOUT_FB`
- `XR_ERROR_SPACE_NETWORK_REQUEST_FAILED_FB`
- `XR_ERROR_SPACE_CLOUD_STORAGE_DISABLED_FB`

Valid Usage (Implicit)

- The XR_FB_spatial_entity_sharing extension **must** be enabled prior to using XrEventDataSpaceShareCompleteFB
- type **must** be XR_TYPE_EVENT_DATA_SPACE_SHARE_COMPLETE_FB
- next **must** be NULL or a valid pointer to the next structure in a structure chain

New Functions

The xrShareSpacesFB function is defined as:

```
// Provided by XR_FB_spatial_entity_sharing
XrResult xrShareSpacesFB(
    XrSession                                   session,
    const XrSpaceShareInfoFB*                   info,
    XrAsyncRequestIdFB*                         requestId);
```

Parameter Descriptions

- session is a handle to an XrSession.
- info is a pointer to an XrSpaceShareInfoFB structure containing information about which spatial entities to share with which users.
- requestId is the output parameter that points to the ID of this asynchronous request.

This operation is asynchronous and the runtime **must** post an XrEventDataSpaceShareCompleteFB event when the operation completes successfully or encounters an error. If this function returns a failure code, no event is posted. The requestId **can** be used to later refer to the request, such as identifying which request has completed when an XrEventDataSpaceShareCompleteFB is posted to the event queue.

Valid Usage (Implicit)

- The XR_FB_spatial_entity_sharing extension **must** be enabled prior to calling xrShareSpacesFB
- session **must** be a valid XrSession handle
- info **must** be a pointer to a valid XrSpaceShareInfoFB structure
- requestId **must** be a pointer to an XrAsyncRequestIdFB value

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FEATURE_UNSUPPORTED
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_SPACE_CLOUD_STORAGE_DISABLED_FB
- XR_ERROR_SPACE_COMPONENT_NOT_ENABLED_FB
- XR_ERROR_SPACE_LOCALIZATION_FAILED_FB
- XR_ERROR_SPACE_MAPPING_INSUFFICIENT_FB
- XR_ERROR_SPACE_NETWORK_REQUEST_FAILED_FB
- XR_ERROR_SPACE_NETWORK_TIMEOUT_FB
- XR_ERROR_VALIDATION_FAILURE

Issues

Version History

- Revision 1, 2022-06-08 (John Schofield)
  - Initial draft

12.125. XR_FB_spatial_entity_storage

Name String

XR_FB_spatial_entity_storage

Extension Type

Instance extension

Registered Extension Number

159

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0
and
XR_FB_spatial_entity

Contributors

John Schofield, Facebook
Andrew Kim, Facebook
Yuichi Taguchi, Facebook
Cass Everitt, Facebook
Curtis Arink, Facebook

Overview

This extension enables spatial entities to be stored and persisted across sessions. If the XR_SPACE_COMPONENT_TYPE_STORABLE_FB component has been enabled on the spatial entity, application developers **may** save, load, and erase persisted XrSpace entities.

New Object Types

New Flag Types

New Enum Constants

XrStructureType enumeration is extended with:

- XR_TYPE_SPACE_SAVE_INFO_FB
- XR_TYPE_SPACE_ERASE_INFO_FB
- XR_TYPE_EVENT_DATA_SPACE_SAVE_COMPLETE_FB
- XR_TYPE_EVENT_DATA_SPACE_ERASE_COMPLETE_FB

New Enums

```
// Provided by XR_FB_spatial_entity_storage
typedef enum XrSpaceStorageLocationFB {
    XR_SPACE_STORAGE_LOCATION_INVALID_FB = 0,
    XR_SPACE_STORAGE_LOCATION_LOCAL_FB = 1,
    XR_SPACE_STORAGE_LOCATION_CLOUD_FB = 2,
    XR_SPACE_STORAGE_LOCATION_MAX_ENUM_FB = 0x7FFFFFFF
} XrSpaceStorageLocationFB;
```

The XrSpaceStorageLocationFB enumeration contains the storage locations used to store, erase, and query spatial entities.

Enumerant Descriptions

- XR_SPACE_STORAGE_LOCATION_INVALID_FB — Invalid storage location
- XR_SPACE_STORAGE_LOCATION_LOCAL_FB — Local device storage
- XR_SPACE_STORAGE_LOCATION_CLOUD_FB — Cloud storage

```
// Provided by XR_FB_spatial_entity_storage
typedef enum XrSpacePersistenceModeFB {
    XR_SPACE_PERSISTENCE_MODE_INVALID_FB = 0,
    XR_SPACE_PERSISTENCE_MODE_INDEFINITE_FB = 1,
    XR_SPACE_PERSISTENCE_MODE_MAX_ENUM_FB = 0x7FFFFFFF
} XrSpacePersistenceModeFB;
```

The XrSpacePersistenceModeFB enumeration specifies the persistence mode for the save operation.

Enumerant Descriptions

- XR_SPACE_PERSISTENCE_MODE_INVALID_FB — Invalid storage persistence
- XR_SPACE_PERSISTENCE_MODE_INDEFINITE_FB — Store XrSpace indefinitely, or until erased

New Structures

The XrSpaceSaveInfoFB structure is defined as:

```
// Provided by XR_FB_spatial_entity_storage
typedef struct XrSpaceSaveInfoFB {
    XrStructureType             type;
    const void*                 next;
    XrSpace                     space;
    XrSpaceStorageLocationFB    location;
    XrSpacePersistenceModeFB    persistenceMode;
} XrSpaceSaveInfoFB;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `space` is the XrSpace handle to the space of the entity to be saved.
- `location` is the storage location.
- `persistenceMode` is the persistence mode.

The XrSpaceSaveInfoFB structure contains information used to save the spatial entity.

Valid Usage (Implicit)

- The `XR_FB_spatial_entity_storage` extension **must** be enabled prior to using XrSpaceSaveInfoFB
- `type` **must** be `XR_TYPE_SPACE_SAVE_INFO_FB`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `space` **must** be a valid XrSpace handle
- `location` **must** be a valid XrSpaceStorageLocationFB value
- `persistenceMode` **must** be a valid XrSpacePersistenceModeFB value

The XrSpaceEraseInfoFB structure is defined as:

```
// Provided by XR_FB_spatial_entity_storage
typedef struct XrSpaceEraseInfoFB {
    XrStructureType             type;
    const void*                 next;
    XrSpace                     space;
    XrSpaceStorageLocationFB    location;
} XrSpaceEraseInfoFB;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `space` is the XrSpace handle to the reference space that defines the entity to be erased.
- `location` is the storage location.

The XrSpaceEraseInfoFB structure contains information used to erase the spatial entity.

Valid Usage (Implicit)

- The `XR_FB_spatial_entity_storage` extension **must** be enabled prior to using XrSpaceEraseInfoFB
- `type` **must** be `XR_TYPE_SPACE_ERASE_INFO_FB`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `space` **must** be a valid XrSpace handle
- `location` **must** be a valid XrSpaceStorageLocationFB value

The XrEventDataSpaceSaveCompleteFB structure is defined as:

```
// Provided by XR_FB_spatial_entity_storage
typedef struct XrEventDataSpaceSaveCompleteFB {
    XrStructureType             type;
    const void*                 next;
    XrAsyncRequestIdFB          requestId;
    XrResult                    result;
    XrSpace                     space;
    XrUuidEXT                   uuid;
    XrSpaceStorageLocationFB    location;
} XrEventDataSpaceSaveCompleteFB;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `requestId` is the ID of the asynchronous request to save an entity.
- `result` is an XrResult that describes whether the request succeeded or if an error occurred.
- `space` is the spatial entity being saved.
- `uuid` is the UUID for the spatial entity being saved.
- `location` is the location of the spatial entity being saved.

The save result event contains the success of the save/write operation to the specified location, as well as the XrSpace handle on which the save operation was attempted on, the unique UUID, and the triggered async request ID from the initial calling function.

Valid Usage (Implicit)

- The `XR_FB_spatial_entity_storage` extension **must** be enabled prior to using XrEventDataSpaceSaveCompleteFB
- `type` **must** be `XR_TYPE_EVENT_DATA_SPACE_SAVE_COMPLETE_FB`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

The XrEventDataSpaceEraseCompleteFB structure is defined as:

```
// Provided by XR_FB_spatial_entity_storage
typedef struct XrEventDataSpaceEraseCompleteFB {
    XrStructureType             type;
    const void*                 next;
    XrAsyncRequestIdFB          requestId;
    XrResult                    result;
    XrSpace                     space;
    XrUuidEXT                   uuid;
    XrSpaceStorageLocationFB    location;
} XrEventDataSpaceEraseCompleteFB;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `requestId` is the ID of the asynchronous request to erase an entity.
- `result` is an XrResult that describes whether the request succeeded or if an error occurred.
- `space` is the spatial entity being erased.
- `uuid` is the UUID for the spatial entity being erased.
- `location` is the location of the spatial entity being erased.

The erase result event contains the success of the erase operation from the specified storage location. It also provides the UUID of the entity and the async request ID from the initial calling function.

Valid Usage (Implicit)

- The `XR_FB_spatial_entity_storage` extension **must** be enabled prior to using XrEventDataSpaceEraseCompleteFB
- `type` **must** be `XR_TYPE_EVENT_DATA_SPACE_ERASE_COMPLETE_FB`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

New Functions

The xrSaveSpaceFB function is defined as:

```
// Provided by XR_FB_spatial_entity_storage
XrResult xrSaveSpaceFB(
    XrSession                                   session,
    const XrSpaceSaveInfoFB*                    info,
    XrAsyncRequestIdFB*                         requestId);
```

Parameter Descriptions

- `session` is a handle to an XrSession.
- `info` contains the parameters for the save operation.
- `requestId` is an output parameter, and the variable it points to will be populated with the ID of this asynchronous request.

The xrSaveSpaceFB function persists the spatial entity at the specified location with the specified mode. The runtime **must** return XR_ERROR_VALIDATION_FAILURE if XrSpaceSaveInfoFB::space is XR_NULL_HANDLE or otherwise invalid. The runtime **must** return XR_ERROR_VALIDATION_FAILURE if XrSpaceSaveInfoFB::location or XrSpaceSaveInfoFB::persistenceMode is invalid. This operation is asynchronous and the runtime **must** post an XrEventDataSpaceSaveCompleteFB event when the operation completes successfully or encounters an error. If this function returns a failure code, no event is posted.

Valid Usage (Implicit)

- The XR_FB_spatial_entity_storage extension **must** be enabled prior to calling xrSaveSpaceFB
- session **must** be a valid XrSession handle
- info **must** be a pointer to a valid XrSpaceSaveInfoFB structure
- requestId **must** be a pointer to an XrAsyncRequestIdFB value

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FEATURE_UNSUPPORTED
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_SPACE_COMPONENT_NOT_ENABLED_FB
- XR_ERROR_VALIDATION_FAILURE

The xrEraseSpaceFB function is defined as:

```
// Provided by XR_FB_spatial_entity_storage
XrResult xrEraseSpaceFB(
    XrSession                                   session,
    const XrSpaceEraseInfoFB*                   info,
    XrAsyncRequestIdFB*                         requestId);
```

Parameter Descriptions

- `session` is a handle to an XrSession.
- `info` contains the parameters for the erase operation.
- `requestId` is an output parameter, and the variable it points to will be populated with the ID of this asynchronous request.

The xrEraseSpaceFB function erases a spatial entity from storage at the specified location. The XrSpace remains valid in the current session until the application destroys it or the session ends. The runtime **must** return `XR_ERROR_VALIDATION_FAILURE` if XrSpaceEraseInfoFB::`space` is XR_NULL_HANDLE or otherwise invalid. The runtime **must** return `XR_ERROR_VALIDATION_FAILURE` if XrSpaceEraseInfoFB ::`location` is invalid. This operation is asynchronous and the runtime **must** post an XrEventDataSpaceEraseCompleteFB event when the operation completes successfully or encounters an error. If this function returns a failure code, no event is posted.

Valid Usage (Implicit)

- The `XR_FB_spatial_entity_storage` extension **must** be enabled prior to calling xrEraseSpaceFB
- `session` **must** be a valid XrSession handle
- `info` **must** be a pointer to a valid XrSpaceEraseInfoFB structure
- `requestId` **must** be a pointer to an `XrAsyncRequestIdFB` value

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_FEATURE_UNSUPPORTED`
- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_SPACE_COMPONENT_NOT_ENABLED_FB`
- `XR_ERROR_VALIDATION_FAILURE`

Issues

Version History

- Revision 1, 2022-01-22 (John Schofield)
  - Initial draft

12.126. XR_FB_spatial_entity_storage_batch

Name String

`XR_FB_spatial_entity_storage_batch`

Extension Type

Instance extension

Registered Extension Number

239

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0
and
XR_FB_spatial_entity_storage

Contributors

John Schofield, Facebook
Andrew Kim, Facebook

Overview

This extension enables multiple spatial entities at a time to be persisted across sessions. If the XR_SPACE_COMPONENT_TYPE_STORABLE_FB component has been enabled on the spatial entity, application developers **may** save and erase XrSpace entities.

New Object Types

New Flag Types

New Enum Constants

XrStructureType enumeration is extended with:

- XR_TYPE_SPACE_LIST_SAVE_INFO_FB
- XR_TYPE_EVENT_DATA_SPACE_LIST_SAVE_COMPLETE_FB

New Enums

New Structures

The XrSpaceListSaveInfoFB structure is defined as:

```
// Provided by XR_FB_spatial_entity_storage_batch
typedef struct XrSpaceListSaveInfoFB {
    XrStructureType             type;
    const void*                 next;
    uint32_t                    spaceCount;
    XrSpace*                    spaces;
    XrSpaceStorageLocationFB    location;
} XrSpaceListSaveInfoFB;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `spaceCount` is the number of spatial entities to save.
- `spaces` is a list of XrSpace handles for the entities to be saved.
- `location` is the storage location.

The XrSpaceListSaveInfoFB structure contains information used to save multiple spatial entities.

Valid Usage (Implicit)

- The `XR_FB_spatial_entity_storage_batch` extension **must** be enabled prior to using XrSpaceListSaveInfoFB
- `type` **must** be `XR_TYPE_SPACE_LIST_SAVE_INFO_FB`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `spaces` **must** be a pointer to an array of `spaceCount` XrSpace handles
- `location` **must** be a valid XrSpaceStorageLocationFB value
- The `spaceCount` parameter **must** be greater than `0`

The XrEventDataSpaceListSaveCompleteFB structure is defined as:

```
// Provided by XR_FB_spatial_entity_storage_batch
typedef struct XrEventDataSpaceListSaveCompleteFB {
    XrStructureType       type;
    const void*           next;
    XrAsyncRequestIdFB    requestId;
    XrResult              result;
} XrEventDataSpaceListSaveCompleteFB;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `requestId` is the ID of the asynchronous request to save an entity.
- `result` is an XrResult that describes whether the request succeeded or if an error occurred.

This completion event indicates that a request to save a list of XrSpace objects has completed. The application **can** use `result` to check if the request was successful or if an error occurred.

Result Codes

Success

- `XR_SUCCESS`

Failure

- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SPACE_MAPPING_INSUFFICIENT_FB`
- `XR_ERROR_SPACE_LOCALIZATION_FAILED_FB`
- `XR_ERROR_SPACE_NETWORK_TIMEOUT_FB`
- `XR_ERROR_SPACE_NETWORK_REQUEST_FAILED_FB`
- `XR_ERROR_SPACE_CLOUD_STORAGE_DISABLED_FB`

Valid Usage (Implicit)

- The `XR_FB_spatial_entity_storage_batch` extension **must** be enabled prior to using XrEventDataSpaceListSaveCompleteFB
- `type` **must** be `XR_TYPE_EVENT_DATA_SPACE_LIST_SAVE_COMPLETE_FB`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

New Functions

The xrSaveSpaceListFB function is defined as:

```
// Provided by XR_FB_spatial_entity_storage_batch
XrResult xrSaveSpaceListFB(
    XrSession                                   session,
    const XrSpaceListSaveInfoFB*                info,
    XrAsyncRequestIdFB*                         requestId);
```

Parameter Descriptions

- `session` is a handle to an XrSession.
- `info` contains the parameters for the save operation.
- `requestId` is an output parameter, and the variable it points to will be populated with the ID of this asynchronous request.

The xrSaveSpaceListFB function persists the specified spatial entities at the specified storage location. The runtime **must** return `XR_ERROR_VALIDATION_FAILURE` if XrSpaceSaveInfoFB::`location` is invalid. This operation is asynchronous and the runtime **must** post an XrEventDataSpaceListSaveCompleteFB event when the operation completes successfully or encounters an error. If this function returns a failure code, no event is posted.

Valid Usage (Implicit)

- The `XR_FB_spatial_entity_storage_batch` extension **must** be enabled prior to calling xrSaveSpaceListFB
- `session` **must** be a valid XrSession handle
- `info` **must** be a pointer to a valid XrSpaceListSaveInfoFB structure
- `requestId` **must** be a pointer to an `XrAsyncRequestIdFB` value

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FEATURE_UNSUPPORTED
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_SPACE_CLOUD_STORAGE_DISABLED_FB
- XR_ERROR_SPACE_COMPONENT_NOT_ENABLED_FB
- XR_ERROR_SPACE_LOCALIZATION_FAILED_FB
- XR_ERROR_SPACE_MAPPING_INSUFFICIENT_FB
- XR_ERROR_SPACE_NETWORK_REQUEST_FAILED_FB
- XR_ERROR_SPACE_NETWORK_TIMEOUT_FB
- XR_ERROR_VALIDATION_FAILURE

Issues

Version History

- Revision 1, 2022-06-08 (John Schofield)
  - Initial draft

12.127. XR_FB_spatial_entity_user

Name String

XR_FB_spatial_entity_user

Extension Type

Instance extension

Registered Extension Number

242

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Contributors

John Schofield, Facebook
Andrew Kim, Facebook
Andreas Selvik, Facebook

Overview

This extension enables creation and management of user objects which **can** be used by the application to reference a user other than the current user.

New Object Types

```
// Provided by XR_FB_spatial_entity_user
XR_DEFINE_HANDLE(XrSpaceUserFB)
```

Represents a user with which the application **can** interact using various extensions including `XR_FB_spatial_entity_sharing`. See xrCreateSpaceUserFB for how to declare a user.

New Flag Types

New Enum Constants

XrStructureType enumeration is extended with:

- `XR_TYPE_SPACE_USER_CREATE_INFO_FB`

New Enums

New Base Types

The `XrSpaceUserIdFB` type is defined as:

```
// Provided by XR_FB_spatial_entity_user
typedef uint64_t XrSpaceUserIdFB;
```

An implementation-defined ID of the underlying user.

New Structures

The XrSpaceUserCreateInfoFB structure is defined as:

```
// Provided by XR_FB_spatial_entity_user
typedef struct XrSpaceUserCreateInfoFB {
    XrStructureType    type;
    const void*        next;
    XrSpaceUserIdFB    userId;
} XrSpaceUserCreateInfoFB;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `userId` is the user ID with which the application can reference.

The XrSpaceUserCreateInfoFB structure describes a user with which the application **can** interact.

Valid Usage (Implicit)

- The `XR_FB_spatial_entity_user` extension **must** be enabled prior to using XrSpaceUserCreateInfoFB
- `type` **must** be `XR_TYPE_SPACE_USER_CREATE_INFO_FB`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

New Functions

The xrCreateSpaceUserFB function is defined as:

```
// Provided by XR_FB_spatial_entity_user
XrResult xrCreateSpaceUserFB(
    XrSession                                   session,
    const XrSpaceUserCreateInfoFB*              info,
    XrSpaceUserFB*                              user);
```

Parameter Descriptions

- `session` is a handle to an XrSession.
- `info` is a pointer to an XrSpaceUserCreateInfoFB structure containing information to create the user handle.
- `user` is the output parameter that points to the handle of the user being created.

The application **can** use this function to create a user handle with which it **can** then interact, such as sharing XrSpace objects.

Valid Usage (Implicit)

- The `XR_FB_spatial_entity_user` extension **must** be enabled prior to calling xrCreateSpaceUserFB
- `session` **must** be a valid XrSession handle
- `info` **must** be a pointer to a valid XrSpaceUserCreateInfoFB structure
- `user` **must** be a pointer to an XrSpaceUserFB handle

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_LIMIT_REACHED`
- `XR_ERROR_OUT_OF_MEMORY`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_VALIDATION_FAILURE`

The xrGetSpaceUserIdFB function is defined as:

```
// Provided by XR_FB_spatial_entity_user
XrResult xrGetSpaceUserIdFB(
    XrSpaceUserFB                               user,
    XrSpaceUserIdFB*                            userId);
```

Parameter Descriptions

- `user` is a handle to an XrSpaceUserFB.
- `userId` is the output parameter that points to the user ID of the user.

The application **can** use this function to retrieve the user ID of a given user handle.

Valid Usage (Implicit)

- The `XR_FB_spatial_entity_user` extension **must** be enabled prior to calling xrGetSpaceUserIdFB
- `user` **must** be a valid XrSpaceUserFB handle
- `userId` **must** be a pointer to an `XrSpaceUserIdFB` value

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_VALIDATION_FAILURE`

The xrDestroySpaceUserFB function is defined as:

```
// Provided by XR_FB_spatial_entity_user
XrResult xrDestroySpaceUserFB(
    XrSpaceUserFB                               user);
```

Parameter Descriptions

- `user` is a handle to the user object to be destroyed.

The application **should** use this function to release resources tied to a given XrSpaceUserFB once the application no longer needs to reference the user.

Valid Usage (Implicit)

- The `XR_FB_spatial_entity_user` extension **must** be enabled prior to calling xrDestroySpaceUserFB
- `user` **must** be a valid XrSpaceUserFB handle

Thread Safety

- Access to `user`, and any child handles, **must** be externally synchronized

Return Codes

Success

- `XR_SUCCESS`

Failure

- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_RUNTIME_FAILURE`

Issues

Version History

- Revision 1, 2022-07-28 (John Schofield)
  - Initial draft

12.128. XR_FB_swapchain_update_state

Name String

XR_FB_swapchain_update_state

Extension Type

Instance extension

Registered Extension Number

72

Revision

3

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Contributors

Cass Everitt, Facebook
Gloria Kennickell, Facebook

Overview

This extension enables the application to modify and query specific mutable state associated with a swapchain.

New Object Types

New Flag Types

New Enum Constants

New Enums

New Structures

The XrSwapchainStateBaseHeaderFB structure is defined as:

```
// Provided by XR_FB_swapchain_update_state
typedef struct XrSwapchainStateBaseHeaderFB {
    XrStructureType    type;
    void*              next;
} XrSwapchainStateBaseHeaderFB;
```

Member Descriptions

- `type` is the XrStructureType of this structure. This base structure itself has no associated XrStructureType value.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.

The XrSwapchainStateBaseHeaderFB is a base structure that can be overridden by a specific `XrSwapchainState*` child structure.

Valid Usage (Implicit)

- The `XR_FB_swapchain_update_state` extension **must** be enabled prior to using XrSwapchainStateBaseHeaderFB
- `type` **must** be one of the following XrStructureType values: `XR_TYPE_SWAPCHAIN_STATE_ANDROID_SURFACE_DIMENSIONS_FB`, `XR_TYPE_SWAPCHAIN_STATE_FOVEATION_FB`, `XR_TYPE_SWAPCHAIN_STATE_SAMPLER_OPENGL_ES_FB`, `XR_TYPE_SWAPCHAIN_STATE_SAMPLER_VULKAN_FB`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

New Functions

The xrUpdateSwapchainFB function is defined as:

```
// Provided by XR_FB_swapchain_update_state
XrResult xrUpdateSwapchainFB(
    XrSwapchain                                 swapchain,
    const XrSwapchainStateBaseHeaderFB*         state);
```

Parameter Descriptions

- `swapchain` is the XrSwapchain to update state for.
- `state` is a pointer to a `XrSwapchainState` structure based off of XrSwapchainStateBaseHeaderFB.

xrUpdateSwapchainFB provides support for an application to update specific mutable state associated with an XrSwapchain.

Valid Usage (Implicit)

- The `XR_FB_swapchain_update_state` extension **must** be enabled prior to calling xrUpdateSwapchainFB
- `swapchain` **must** be a valid XrSwapchain handle
- `state` **must** be a pointer to a valid XrSwapchainStateBaseHeaderFB-based structure. See also: XrSwapchainStateAndroidSurfaceDimensionsFB, XrSwapchainStateFoveationFB, XrSwapchainStateSamplerOpenGLESFB, XrSwapchainStateSamplerVulkanFB

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_VALIDATION_FAILURE`

The xrGetSwapchainStateFB function is defined as:

```
// Provided by XR_FB_swapchain_update_state
XrResult xrGetSwapchainStateFB(
    XrSwapchain                                 swapchain,
    XrSwapchainStateBaseHeaderFB*               state);
```

Parameter Descriptions

- `swapchain` is the XrSwapchain to update state for.
- `state` is a pointer to a `XrSwapchainState` structure based off of XrSwapchainStateBaseHeaderFB.

xrGetSwapchainStateFB provides support for an application to query specific mutable state associated with an XrSwapchain.

Valid Usage (Implicit)

- The `XR_FB_swapchain_update_state` extension **must** be enabled prior to calling xrGetSwapchainStateFB
- `swapchain` **must** be a valid XrSwapchain handle
- `state` **must** be a pointer to an XrSwapchainStateBaseHeaderFB-based structure. See also: XrSwapchainStateAndroidSurfaceDimensionsFB, XrSwapchainStateFoveationFB, XrSwapchainStateSamplerOpenGLESFB, XrSwapchainStateSamplerVulkanFB

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_VALIDATION_FAILURE`

Issues

- Should we add a method to query the current state?
  - Yes. Given that we allow mutable state to be updated by the application, it is useful to have a query mechanism to get the current state for all state structures.

Version History

- Revision 1, 2021-04-16 (Gloria Kennickell)
  - Initial extension description
- Revision 2, 2021-05-13 (Gloria Kennickell)
  - Add mechanism to query current state for all state structures.
- Revision 3, 2021-05-27 (Gloria Kennickell)
  - Move platform and graphics API specific structs into separate extensions.

12.129. XR_FB_swapchain_update_state_android_surface

Name String

`XR_FB_swapchain_update_state_android_surface`

Extension Type

Instance extension

Registered Extension Number

162

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0
and
`XR_KHR_android_surface_swapchain`
and
`XR_FB_swapchain_update_state`

Contributors

Cass Everitt, Facebook
Gloria Kennickell, Facebook

Overview

This extension enables the application to modify and query specific mutable state associated with an Android surface swapchain, examples include:

- A video application may need to update the default size of the image buffers associated with an Android Surface Swapchain.
- A video application may need to communicate a new width and height for an Android Surface Swapchain, as the surface dimensions may be implicitly updated by the producer during the life of the Swapchain. This is important for correct application of the non-normalized `imageRect` specified via XrSwapchainSubImage.

New Object Types

New Flag Types

New Enum Constants

XrStructureType enumeration is extended with:

- `XR_TYPE_SWAPCHAIN_STATE_ANDROID_SURFACE_DIMENSIONS_FB`

New Enums

New Structures

The XrSwapchainStateAndroidSurfaceDimensionsFB structure is defined as:

```
// Provided by XR_FB_swapchain_update_state_android_surface
typedef struct XrSwapchainStateAndroidSurfaceDimensionsFB {
    XrStructureType    type;
    void*              next;
    uint32_t           width;
    uint32_t           height;
} XrSwapchainStateAndroidSurfaceDimensionsFB;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `width` is the width of the image buffer, must not be greater than the graphics API's maximum limit.
- `height` is the height of the image buffer, must not be greater than the graphics API's maximum limit.

When XrSwapchainStateAndroidSurfaceDimensionsFB is specified in the call to xrUpdateSwapchainFB, the dimensions provided will be used to update the default size of the image buffers associated with the Android Surface swapchain.

Additionally, the dimensions provided will become the new source of truth for the swapchain width and height, affecting operations such as computing the normalized imageRect for the swapchain.

When XrSwapchainStateAndroidSurfaceDimensionsFB is specified in the call to xrGetSwapchainStateFB, the dimensions will be populated with the current swapchain width and height.

To use XrSwapchainStateAndroidSurfaceDimensionsFB, XR_USE_PLATFORM_ANDROID must be defined before including `openxr_platform.h`.

Valid Usage (Implicit)

- The `XR_FB_swapchain_update_state_android_surface` extension **must** be enabled prior to using XrSwapchainStateAndroidSurfaceDimensionsFB
- `type` **must** be `XR_TYPE_SWAPCHAIN_STATE_ANDROID_SURFACE_DIMENSIONS_FB`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

New Functions

Issues

Version History

- Revision 1, 2021-05-27 (Gloria Kennickell)
  - Initial draft

12.130. XR_FB_swapchain_update_state_opengl_es

Name String

XR_FB_swapchain_update_state_opengl_es

Extension Type

Instance extension

Registered Extension Number

163

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

XR_KHR_opengl_es_enable
and
XR_FB_swapchain_update_state

Contributors

Cass Everitt, Facebook
Gloria Kennickell, Facebook

Overview

This extension enables the application to modify and query OpenGL ES-specific mutable state associated with a swapchain, examples include:

- On platforms where composition runs in a separate process from the application, swapchains must be created in a cross-process friendly way. In such cases, the texture image memory may be shared between processes, but the texture state may not; and, an explicit mechanism to synchronize this texture state between the application and the compositor is required.

New Object Types

New Flag Types

New Enum Constants

XrStructureType enumeration is extended with:

- XR_TYPE_SWAPCHAIN_STATE_SAMPLER_OPENGL_ES_FB

New Enums

New Structures

The XrSwapchainStateSamplerOpenGLESFB structure is defined as:

```
// Provided by XR_FB_swapchain_update_state_opengl_es
typedef struct XrSwapchainStateSamplerOpenGLESFB {
    XrStructureType    type;
    void*              next;
    EGLenum            minFilter;
    EGLenum            magFilter;
    EGLenum            wrapModeS;
    EGLenum            wrapModeT;
    EGLenum            swizzleRed;
    EGLenum            swizzleGreen;
    EGLenum            swizzleBlue;
    EGLenum            swizzleAlpha;
    float              maxAnisotropy;
    XrColor4f          borderColor;
} XrSwapchainStateSamplerOpenGLESFB;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `minFilter` is a valid Android OpenGL ES `EGLenum`.
- `magFilter` is a valid Android OpenGL ES `EGLenum`.
- `wrapModeS` is a valid Android OpenGL ES `EGLenum`.
- `wrapModeT` is a valid Android OpenGL ES `EGLenum`.
- `swizzleRed` is a valid Android OpenGL ES `EGLenum`.
- `swizzleGreen` is a valid Android OpenGL ES `EGLenum`.
- `swizzleBlue` is a valid Android OpenGL ES `EGLenum`.
- `swizzleAlpha` is a valid Android OpenGL ES `EGLenum`.
- `maxAnisotropy` is a valid float used to represent max anisotropy.
- `borderColor` is an RGBA color to be used as border texels.

When XrSwapchainStateSamplerOpenGLESFB is specified in the call to xrUpdateSwapchainFB, texture sampler state for all images in the XrSwapchain will be updated for both the application and

compositor processes.

For most cases, the sampler state update is only required compositor-side, as that is where the swapchain images are sampled. For completeness, the application-side sampler state is additionally updated to support cases where the application may choose to directly sample the swapchain images.

Applications are expected to handle synchronization of the sampler state update with application-side rendering. Similarly, the compositor will synchronize the sampler state update with rendering of the next compositor frame.

An `EGLContext`, either the `EGLContext` bound during XrSwapchain creation or an `EGLContext` in the same share group, is required to be bound on the application calling thread. Current texture bindings may be altered by the call, including the active texture.

When XrSwapchainStateSamplerOpenGLESFB is specified in the call to xrGetSwapchainStateFB, the sampler state will be populated with the current swapchain sampler state.

To use XrSwapchainStateSamplerOpenGLESFB, XR_USE_GRAPHICS_API_OPENGL_ES must be defined before including `openxr_platform.h`.

Valid Usage (Implicit)

- The `XR_FB_swapchain_update_state_opengl_es` extension **must** be enabled prior to using XrSwapchainStateSamplerOpenGLESFB
- `type` **must** be `XR_TYPE_SWAPCHAIN_STATE_SAMPLER_OPENGL_ES_FB`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `minFilter` **must** be a valid `EGLenum` value
- `magFilter` **must** be a valid `EGLenum` value
- `wrapModeS` **must** be a valid `EGLenum` value
- `wrapModeT` **must** be a valid `EGLenum` value
- `swizzleRed` **must** be a valid `EGLenum` value
- `swizzleGreen` **must** be a valid `EGLenum` value
- `swizzleBlue` **must** be a valid `EGLenum` value
- `swizzleAlpha` **must** be a valid `EGLenum` value

New Functions

Issues

Version History

- Revision 1, 2021-05-27 (Gloria Kennickell)

- Initial draft

12.131. XR_FB_swapchain_update_state_vulkan

Name String

XR_FB_swapchain_update_state_vulkan

Extension Type

Instance extension

Registered Extension Number

164

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0
and
XR_KHR_vulkan_enable
and
XR_FB_swapchain_update_state

Contributors

Cass Everitt, Facebook
Gloria Kennickell, Facebook

Overview

This extension enables the application to modify and query Vulkan-specific mutable state associated with a swapchain, examples include:

- On platforms where composition runs in a separate process from the application, swapchains must be created in a cross-process friendly way. In such cases, the texture image memory may be shared between processes, but the texture state may not; and, an explicit mechanism to synchronize this texture state between the application and the compositor is required.

New Object Types

New Flag Types

New Enum Constants

XrStructureType enumeration is extended with:

- XR_TYPE_SWAPCHAIN_STATE_SAMPLER_VULKAN_FB

New Enums

New Structures

The XrSwapchainStateSamplerVulkanFB structure is defined as:

```
// Provided by XR_FB_swapchain_update_state_vulkan
typedef struct XrSwapchainStateSamplerVulkanFB {
    XrStructureType         type;
    void*                   next;
    VkFilter                minFilter;
    VkFilter                magFilter;
    VkSamplerMipmapMode     mipmapMode;
    VkSamplerAddressMode    wrapModeS;
    VkSamplerAddressMode    wrapModeT;
    VkComponentSwizzle      swizzleRed;
    VkComponentSwizzle      swizzleGreen;
    VkComponentSwizzle      swizzleBlue;
    VkComponentSwizzle      swizzleAlpha;
    float                   maxAnisotropy;
    XrColor4f               borderColor;
} XrSwapchainStateSamplerVulkanFB;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `minFilter` is a valid Vulkan `VkFilter`.
- `magFilter` is a valid Vulkan `VkFilter`.
- `mipmapMode` is a valid Vulkan `VkSamplerMipmapMode`.
- `wrapModeS` is a valid Vulkan `VkSamplerAddressMode`.
- `wrapModeT` is a valid Vulkan `VkSamplerAddressMode`.
- `swizzleRed` is a valid Vulkan `VkComponentSwizzle`.
- `swizzleGreen` is a valid Vulkan `VkComponentSwizzle`.
- `swizzleBlue` is a valid Vulkan `VkComponentSwizzle`.
- `swizzleAlpha` is a valid Vulkan `VkComponentSwizzle`.
- `maxAnisotropy` is a valid float used to represent max anisotropy.
- `borderColor` is an RGBA color to be used as border texels.

When XrSwapchainStateSamplerVulkanFB is specified in the call to xrUpdateSwapchainFB, texture sampler state for all images in the XrSwapchain will be updated for the compositor process. For most cases, the sampler state update is only required compositor-side, as that is where the swapchain images are sampled. If the application requires sampling of the swapchain images, the application will be responsible for updating the texture state using normal Vulkan mechanisms and synchronizing appropriately with application-side rendering.

When XrSwapchainStateSamplerVulkanFB is specified in the call to xrGetSwapchainStateFB, the sampler state will be populated with the current swapchain sampler state.

To use XrSwapchainStateSamplerVulkanFB, XR_USE_GRAPHICS_API_VULKAN must be defined before including `openxr_platform.h`.

Valid Usage (Implicit)

- The `XR_FB_swapchain_update_state_vulkan` extension **must** be enabled prior to using XrSwapchainStateSamplerVulkanFB
- `type` **must** be `XR_TYPE_SWAPCHAIN_STATE_SAMPLER_VULKAN_FB`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `minFilter` **must** be a valid `VkFilter` value
- `magFilter` **must** be a valid `VkFilter` value
- `mipmapMode` **must** be a valid `VkSamplerMipmapMode` value
- `wrapModeS` **must** be a valid `VkSamplerAddressMode` value
- `wrapModeT` **must** be a valid `VkSamplerAddressMode` value
- `swizzleRed` **must** be a valid `VkComponentSwizzle` value
- `swizzleGreen` **must** be a valid `VkComponentSwizzle` value
- `swizzleBlue` **must** be a valid `VkComponentSwizzle` value
- `swizzleAlpha` **must** be a valid `VkComponentSwizzle` value

New Functions

Issues

Version History

- Revision 1, 2021-05-27 (Gloria Kennickell)
  - Initial draft

12.132. XR_FB_triangle_mesh

Name String

`XR_FB_triangle_mesh`

Extension Type

Instance extension

Registered Extension Number

118

Revision

2

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Contributors

Anton Vaneev, Facebook
Cass Everitt, Facebook
Federico Schliemann, Facebook
Johannes Schmid, Facebook

Overview

Meshes may be useful in XR applications when representing parts of the environment. In particular, application may provide the surfaces of real-world objects tagged manually to the runtime, or obtain automatically detected environment contents.

This extension allows:

- An application to create a triangle mesh and specify the mesh data.
- An application to update mesh contents if a mesh is mutable.

New Object Types

```
// Provided by XR_FB_triangle_mesh
XR_DEFINE_HANDLE(XrTriangleMeshFB)
```

XrTriangleMeshFB represents a triangle mesh with its corresponding mesh data: a vertex buffer and an index buffer.

New Flag Types

```
// Provided by XR_FB_triangle_mesh
typedef XrFlags64 XrTriangleMeshFlagsFB;
```

```
// Provided by XR_FB_triangle_mesh
// Flag bits for XrTriangleMeshFlagsFB
static const XrTriangleMeshFlagsFB XR_TRIANGLE_MESH_MUTABLE_BIT_FB = 0x00000001;
```

Flag Descriptions

- XR_TRIANGLE_MESH_MUTABLE_BIT_FB — The triangle mesh is mutable (can be modified after it is created).

New Enum Constants

XrStructureType enumeration is extended with:

- XR_TYPE_TRIANGLE_MESH_CREATE_INFO_FB

New Enums

Applications may specify the triangle winding order of a mesh - whether the vertices of an outward-facing side of a triangle appear in clockwise or counter-clockwise order - using XrWindingOrderFB enumeration.

```
// Provided by XR_FB_triangle_mesh
typedef enum XrWindingOrderFB {
    XR_WINDING_ORDER_UNKNOWN_FB = 0,
    XR_WINDING_ORDER_CW_FB = 1,
    XR_WINDING_ORDER_CCW_FB = 2,
    XR_WINDING_ORDER_MAX_ENUM_FB = 0x7FFFFFFF
} XrWindingOrderFB;
```

Enumerant Descriptions

- XR_WINDING_ORDER_UNKNOWN_FB — Winding order is unknown and the runtime cannot make any assumptions on the triangle orientation
- XR_WINDING_ORDER_CW_FB — Clockwise winding order
- XR_WINDING_ORDER_CCW_FB — Counter-clockwise winding order

New Structures

XrTriangleMeshCreateInfoFB **must** be provided when calling xrCreateTriangleMeshFB.

The XrTriangleMeshCreateInfoFB structure is defined as:

```
// Provided by XR_FB_triangle_mesh
typedef struct XrTriangleMeshCreateInfoFB {
    XrStructureType          type;
    const void*              next;
    XrTriangleMeshFlagsFB    flags;
    XrWindingOrderFB         windingOrder;
    uint32_t                 vertexCount;
    const XrVector3f*        vertexBuffer;
    uint32_t                 triangleCount;
    const uint32_t*          indexBuffer;
} XrTriangleMeshCreateInfoFB;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `flags` is a bitmask of XrTriangleMeshFlagBitsFB that specify behavior.
- `windingOrder` is the XrWindingOrderFB value defining the winding order of the mesh triangles.
- `vertexCount` is the number of vertices in the mesh. In the case of the mutable mesh, the value is treated as the maximum number of vertices the mesh will be able to represent at any time in its lifecycle. The actual number of vertices can vary and is defined when xrTriangleMeshEndUpdateFB is called.
- `vertexBuffer` is a pointer to the vertex data. The size of the array must be `vertexCount` elements. When the mesh is mutable (`(flags & XR_TRIANGLE_MESH_MUTABLE_BIT_FB) != 0`), the `vertexBuffer` parameter **must** be `NULL` and mesh data **must** be populated separately.
- `triangleCount` is the number of triangles in the mesh. In the case of the mutable mesh, the value is treated as the maximum number of triangles the mesh will be able to represent at any time in its lifecycle. The actual number of triangles can vary and is defined when xrTriangleMeshEndUpdateFB is called.
- `indexBuffer` the triangle indices. The size of the array must be `triangleCount` elements. When the mesh is mutable (`(flags & XR_TRIANGLE_MESH_MUTABLE_BIT_FB) != 0`), the `indexBuffer` parameter **must** be `NULL` and mesh data **must** be populated separately.

Mesh buffers **can** be updated between xrTriangleMeshBeginUpdateFB and xrTriangleMeshEndUpdateFB calls.

If the mesh is non-mutable, `vertexBuffer` **must** be a pointer to an array of `vertexCount` XrVector3f structures. If the mesh is non-mutable, `indexBuffer` **must** be a pointer to an array of 3 * `triangleCount`

uint32_t vertex indices.

Valid Usage (Implicit)

- The XR_FB_triangle_mesh extension **must** be enabled prior to using XrTriangleMeshCreateInfoFB
- type **must** be XR_TYPE_TRIANGLE_MESH_CREATE_INFO_FB
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- flags **must** be 0 or a valid combination of XrTriangleMeshFlagBitsFB values
- windingOrder **must** be a valid XrWindingOrderFB value

Mutable Mesh Update States

Mutable meshes have a state machine controlling how they may be updated.

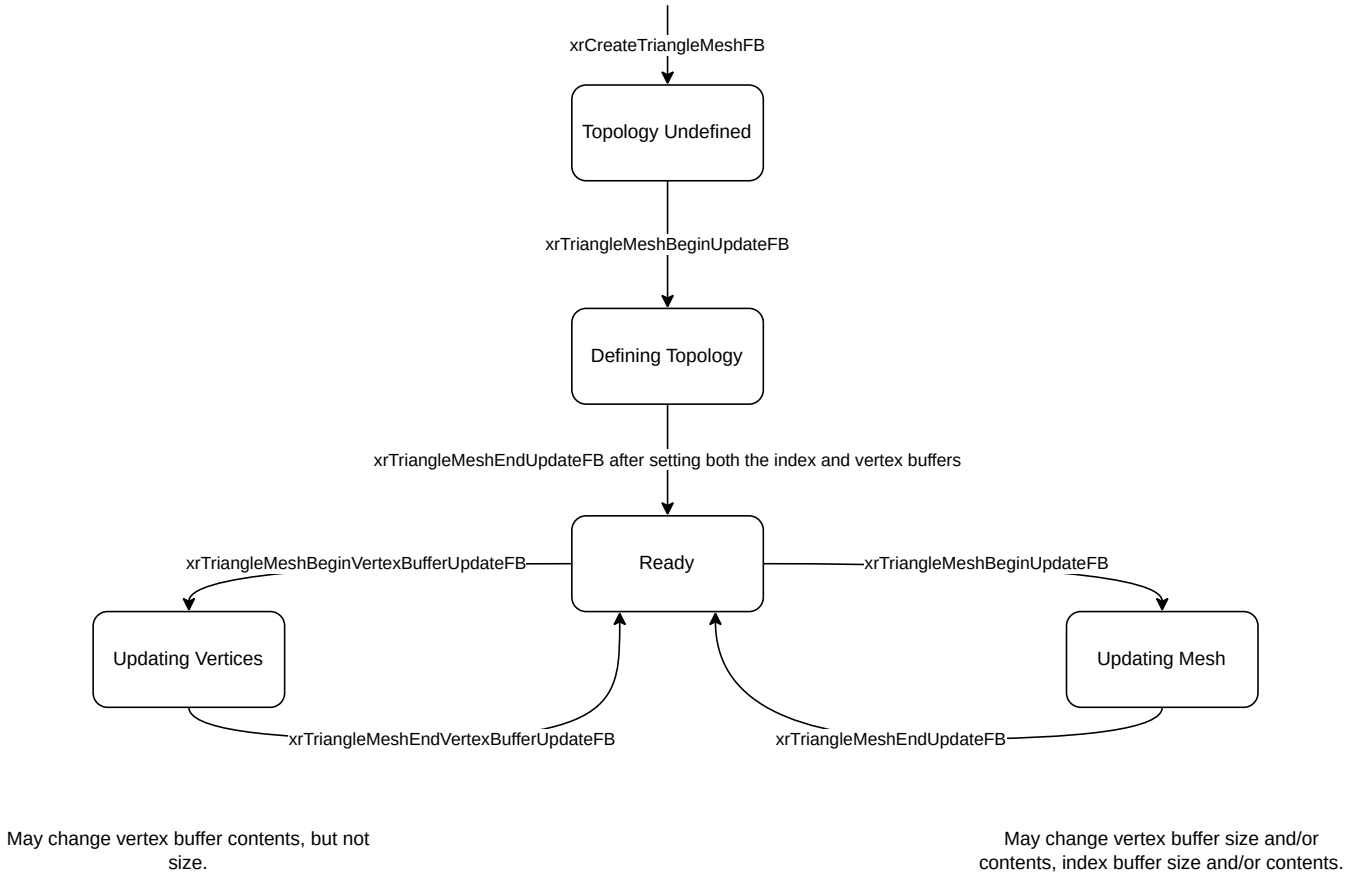


Figure 30. Mutable Triangle Mesh States

The states are as follows:

Undefined Topology

The default state immediately after creation of a mutable mesh. Move to Defining Topology by

calling xrTriangleMeshBeginUpdateFB.

Defining Topology

The application **must** set the initial vertex buffer and index buffer before moving to Ready by calling xrTriangleMeshEndUpdateFB.

Ready

In this state, the buffer contents/size **must** not be modified. To move to Updating Mesh call xrTriangleMeshBeginUpdateFB. To move to Updating Vertices call xrTriangleMeshBeginVertexBufferUpdateFB.

Updating Mesh

The application **may** modify the vertex buffer contents and/or the vertex count. The application **may** modify the index buffer contents and/or the index buffer element count. Move to Ready and commit changes by calling xrTriangleMeshEndUpdateFB.

Updating Vertices

The application **may** modify the vertex buffer contents, but not the vertex count. Move to Ready and commit changes by calling xrTriangleMeshEndVertexBufferUpdateFB.

New Functions

The xrCreateTriangleMeshFB function is defined as:

```
// Provided by XR_FB_triangle_mesh
XrResult xrCreateTriangleMeshFB(
    XrSession                                   session,
    const XrTriangleMeshCreateInfoFB*           createInfo,
    XrTriangleMeshFB*                           outTriangleMesh);
```

Parameter Descriptions

- `session` is the XrSession to which the mesh will belong.
- `createInfo` is a pointer to an XrTriangleMeshCreateInfoFB structure containing parameters to be used to create the mesh.
- `outTriangleMesh` is a pointer to a handle in which the created XrTriangleMeshFB is returned.

This creates an XrTriangleMeshFB handle. The returned triangle mesh handle **may** be subsequently used in API calls.

When the mesh is mutable (the `XR_TRIANGLE_MESH_MUTABLE_BIT_FB` bit is set in XrTriangleMeshCreateInfoFB::`flags`), the created triangle mesh starts in the Undefined Topology state.

Immutable meshes have no state machine; they may be considered to be in state Ready with no valid edges leaving that state.

Valid Usage (Implicit)

- The XR_FB_triangle_mesh extension **must** be enabled prior to calling xrCreateTriangleMeshFB
- session **must** be a valid XrSession handle
- createInfo **must** be a pointer to a valid XrTriangleMeshCreateInfoFB structure
- outTriangleMesh **must** be a pointer to an XrTriangleMeshFB handle

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FEATURE_UNSUPPORTED
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_INSUFFICIENT_RESOURCES_PASSTHROUGH_FB
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The xrDestroyTriangleMeshFB function is defined as:

```
// Provided by XR_FB_triangle_mesh
XrResult xrDestroyTriangleMeshFB(
    XrTriangleMeshFB                            mesh);
```

Parameter Descriptions

- `mesh` is the XrTriangleMeshFB to destroy.

XrTriangleMeshFB handles and their associated data are destroyed by xrDestroyTriangleMeshFB. The mesh buffers retrieved by xrTriangleMeshGetVertexBufferFB and xrTriangleMeshGetIndexBufferFB **must** not be accessed anymore after their parent mesh object has been destroyed.

Valid Usage (Implicit)

- The `XR_FB_triangle_mesh` extension **must** be enabled prior to calling xrDestroyTriangleMeshFB
- `mesh` **must** be a valid XrTriangleMeshFB handle

Thread Safety

- Access to `mesh`, and any child handles, **must** be externally synchronized
- Access to the buffers returned from calls to xrTriangleMeshGetVertexBufferFB and xrTriangleMeshGetIndexBufferFB on `mesh` **must** be externally synchronized

Return Codes

Success

- `XR_SUCCESS`

Failure

- `XR_ERROR_FEATURE_UNSUPPORTED`
- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_RUNTIME_FAILURE`

The xrTriangleMeshGetVertexBufferFB function is defined as:

```
// Provided by XR_FB_triangle_mesh
XrResult xrTriangleMeshGetVertexBufferFB(
    XrTriangleMeshFB                            mesh,
    XrVector3f**                                outVertexBuffer);
```

Parameter Descriptions

- `mesh` is the XrTriangleMeshFB to get the vertex buffer for.
- `outVertexBuffer` is a pointer to return the vertex buffer into.

Retrieves a pointer to the vertex buffer. The vertex buffer is structured as an array of XrVector3f. The size of the buffer is XrTriangleMeshCreateInfoFB::`vertexCount` elements. The buffer location is guaranteed to remain constant over the lifecycle of the mesh object.

A mesh **must** be mutable and in a specific state for the application to **modify** it through the retrieved vertex buffer.

- A mutable triangle mesh **must** be in state Defining Topology, Updating Mesh, or Updating Vertices to modify the **contents** of the vertex buffer retrieved by this function.
- A mutable triangle mesh **must** be in state Defining Topology or Updating Mesh to modify the **count** of elements in the vertex buffer retrieved by this function. The new count is passed as a parameter to xrTriangleMeshEndUpdateFB.

Valid Usage (Implicit)

- The `XR_FB_triangle_mesh` extension **must** be enabled prior to calling xrTriangleMeshGetVertexBufferFB
- `mesh` **must** be a valid XrTriangleMeshFB handle
- `outVertexBuffer` **must** be a pointer to a pointer to an XrVector3f structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FEATURE_UNSUPPORTED
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The xrTriangleMeshGetIndexBufferFB function is defined as:

```
// Provided by XR_FB_triangle_mesh
XrResult xrTriangleMeshGetIndexBufferFB(
    XrTriangleMeshFB                            mesh,
    uint32_t**                                  outIndexBuffer);
```

Parameter Descriptions

- mesh is the XrTriangleMeshFB to get the index buffer for.
- outIndexBuffer is a pointer to return the index buffer into.

Retrieves a pointer to the index buffer that defines the topology of the triangle mesh. Each triplet of consecutive elements points to three vertices in the vertex buffer and thus form a triangle. The size of the index buffer is 3 * XrTriangleMeshCreateInfoFB::triangleCount elements. The buffer location is guaranteed to remain constant over the lifecycle of the mesh object.

A triangle mesh **must** be mutable and in state Defining Topology or Updating Mesh for the application to **modify** the contents and/or triangle count in the index buffer retrieved by this function.

Valid Usage (Implicit)

- The XR_FB_triangle_mesh extension **must** be enabled prior to calling xrTriangleMeshGetIndexBufferFB
- mesh **must** be a valid XrTriangleMeshFB handle
- outIndexBuffer **must** be a pointer to a pointer to a uint32_t value

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FEATURE_UNSUPPORTED
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The xrTriangleMeshBeginUpdateFB function is defined as:

```
// Provided by XR_FB_triangle_mesh
XrResult xrTriangleMeshBeginUpdateFB(
    XrTriangleMeshFB                            mesh);
```

Parameter Descriptions

- mesh is the XrTriangleMeshFB to update.

Begins updating the mesh buffer data. The application **must** call this function before it makes any modifications to the buffers retrieved by xrTriangleMeshGetVertexBufferFB and xrTriangleMeshGetIndexBufferFB. If only the vertex buffer contents need to be updated, and the mesh is in state Ready, xrTriangleMeshBeginVertexBufferUpdateFB **may** be used instead. To commit the modifications, the application **must** call xrTriangleMeshEndUpdateFB.

The triangle mesh `mesh` **must** be mutable. The runtime **must** return `XR_ERROR_VALIDATION_FAILURE` if the mesh is immutable.

The triangle mesh `mesh` **must** be in state Undefined Topology or Ready.

- If the triangle mesh is in state Undefined Topology before this call, a successful call moves it to state Defining Topology.
- If the triangle mesh is in state Ready before this call, a successful call moves it to state Updating Mesh.

Valid Usage (Implicit)

- The `XR_FB_triangle_mesh` extension **must** be enabled prior to calling xrTriangleMeshBeginUpdateFB
- `mesh` **must** be a valid XrTriangleMeshFB handle

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_CALL_ORDER_INVALID`
- `XR_ERROR_FEATURE_UNSUPPORTED`
- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_VALIDATION_FAILURE`

The xrTriangleMeshEndUpdateFB function is defined as:

```
// Provided by XR_FB_triangle_mesh
XrResult xrTriangleMeshEndUpdateFB(
    XrTriangleMeshFB                            mesh,
    uint32_t                                    vertexCount,
    uint32_t                                    triangleCount);
```

Parameter Descriptions

- `mesh` is the XrTriangleMeshFB to update.
- `vertexCount` is the vertex count after the update.
- `triangleCount` is the triangle count after the update.

Signals to the runtime that the application has finished initially populating or updating the mesh buffers. `vertexCount` and `triangleCount` specify the actual number of primitives that make up the mesh after the update. They **must** be larger than zero but smaller or equal to the maximum counts defined at create time. The runtime **must** return `XR_ERROR_VALIDATION_FAILURE` if an invalid count is passed.

The triangle mesh `mesh` **must** be mutable. The runtime **must** return `XR_ERROR_VALIDATION_FAILURE` if the mesh is immutable.

The triangle mesh `mesh` **must** be in state Defining Topology or Updating Mesh.

A successful call moves `mesh` to state Ready.

Valid Usage (Implicit)

- The `XR_FB_triangle_mesh` extension **must** be enabled prior to calling xrTriangleMeshEndUpdateFB
- `mesh` **must** be a valid XrTriangleMeshFB handle

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_CALL_ORDER_INVALID
- XR_ERROR_FEATURE_UNSUPPORTED
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The xrTriangleMeshBeginVertexBufferUpdateFB function is defined as:

```
// Provided by XR_FB_triangle_mesh
XrResult xrTriangleMeshBeginVertexBufferUpdateFB(
    XrTriangleMeshFB                            mesh,
    uint32_t*                                   outVertexCount);
```

Parameter Descriptions

- mesh is the XrTriangleMeshFB to update.
- outVertexCount is a pointer to a value to populate with the current vertex count. The updated data must have the exact same number of vertices.

Begins an update of the vertex positions of a mutable triangle mesh. The vertex count returned through outVertexCount is defined by the last call to xrTriangleMeshEndUpdateFB. Once the modification is done, call xrTriangleMeshEndVertexBufferUpdateFB to commit the changes and move to state Ready.

The triangle mesh mesh **must** be mutable. The runtime **must** return XR_ERROR_VALIDATION_FAILURE if the mesh is immutable.

The triangle mesh mesh **must** be in state Ready.

A successful call moves `mesh` to state Updating Vertices.

Valid Usage (Implicit)

- The `XR_FB_triangle_mesh` extension **must** be enabled prior to calling xrTriangleMeshBeginVertexBufferUpdateFB
- `mesh` **must** be a valid XrTriangleMeshFB handle
- `outVertexCount` **must** be a pointer to a `uint32_t` value

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_CALL_ORDER_INVALID`
- `XR_ERROR_FEATURE_UNSUPPORTED`
- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_VALIDATION_FAILURE`

The xrTriangleMeshEndVertexBufferUpdateFB function is defined as:

```
// Provided by XR_FB_triangle_mesh
XrResult xrTriangleMeshEndVertexBufferUpdateFB(
    XrTriangleMeshFB                            mesh);
```

Parameter Descriptions

- `mesh` is the XrTriangleMeshFB to update.

Signals to the runtime that the application has finished updating the vertex buffer data following a call to xrTriangleMeshBeginVertexBufferUpdateFB.

The triangle mesh `mesh` **must** be mutable. The runtime **must** return `XR_ERROR_VALIDATION_FAILURE` if the mesh is immutable.

The triangle mesh `mesh` **must** be in state Updating Vertices.

A successful call moves `mesh` to state Ready.

Valid Usage (Implicit)

- The `XR_FB_triangle_mesh` extension **must** be enabled prior to calling xrTriangleMeshEndVertexBufferUpdateFB
- `mesh` **must** be a valid XrTriangleMeshFB handle

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_CALL_ORDER_INVALID`
- `XR_ERROR_FEATURE_UNSUPPORTED`
- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_VALIDATION_FAILURE`

Issues

Version History

- Revision 1, 2021-09-01 (Anton Vaneev)
  - Initial extension description
- Revision 2, 2022-01-07 (Rylie Pavlik, Collabora, Ltd.)
  - Add a state diagram to clarify valid usage, and allow `XR_ERROR_CALL_ORDER_INVALID`.

12.133. XR_HTC_anchor

Name String

XR_HTC_anchor

Extension Type

Instance extension

Registered Extension Number

320

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Last Modified Date

2023-09-14

IP Status

No known IP claims.

Contributors

CheHsuan Shu, HTC
Bill Chang, HTC

Overview

This extension allows an application to create a spatial anchor to track a point in the physical environment. The runtime adjusts the pose of the anchor over time to align it with the real world.

Inspect system capability

The XrSystemAnchorPropertiesHTC structure is defined as:

```
// Provided by XR_HTC_anchor
typedef struct XrSystemAnchorPropertiesHTC {
    XrStructureType    type;
    void*              next;
    XrBool32           supportsAnchor;
} XrSystemAnchorPropertiesHTC;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension. For more details about the structure chain, see the structure being extended (XrSystemProperties).
- `supportsAnchor` indicates if current system is capable of anchor functionality.

An application **can** inspect whether the system is capable of anchor functionality by chaining an XrSystemAnchorPropertiesHTC structure to the XrSystemProperties when calling xrGetSystemProperties. The runtime **must** return `XR_ERROR_FEATURE_UNSUPPORTED` if XrSystemAnchorPropertiesHTC::`supportsAnchor` was `XR_FALSE`.

Valid Usage (Implicit)

- The `XR_HTC_anchor` extension **must** be enabled prior to using XrSystemAnchorPropertiesHTC
- `type` **must** be `XR_TYPE_SYSTEM_ANCHOR_PROPERTIES_HTC`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

The xrCreateSpatialAnchorHTC function is defined as:

```
// Provided by XR_HTC_anchor
XrResult xrCreateSpatialAnchorHTC(
    XrSession                                   session,
    const XrSpatialAnchorCreateInfoHTC*         createInfo,
    XrSpace*                                    anchor);
```

Parameter Descriptions

- `session` is the XrSession to create the anchor in.
- `createInfo` is the XrSpatialAnchorCreateInfoHTC used to specify the anchor.
- `anchor` is the returned XrSpace handle.

The xrCreateSpatialAnchorHTC function creates a spatial anchor with specified base space and pose in the space. The anchor is represented by an XrSpace and its pose can be tracked via xrLocateSpace. Once the anchor is no longer needed, call xrDestroySpace to erase the anchor.

Valid Usage (Implicit)

- The XR_HTC_anchor extension **must** be enabled prior to calling xrCreateSpatialAnchorHTC
- session **must** be a valid XrSession handle
- createInfo **must** be a pointer to a valid XrSpatialAnchorCreateInfoHTC structure
- anchor **must** be a pointer to an XrSpace handle

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_NAME_INVALID
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_POSE_INVALID
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The XrSpatialAnchorCreateInfoHTC structure is defined as:

```
// Provided by XR_HTC_anchor
typedef struct XrSpatialAnchorCreateInfoHTC {
    XrStructureType           type;
    const void*               next;
    XrSpace                   space;
    XrPosef                   poseInSpace;
    XrSpatialAnchorNameHTC    name;
} XrSpatialAnchorCreateInfoHTC;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `space` is the XrSpace in which `poseInSpace` is specified.
- `poseInSpace` is the XrPosef specifying the point in the real world within `space`.
- `name` is the XrSpatialAnchorNameHTC containing the name of the anchor.

The `poseInSpace` is transformed into world space to specify the point in the real world. The anchor tracks changes of the reality and **may** not be affected by the changes of `space`.

Valid Usage (Implicit)

- The `XR_HTC_anchor` extension **must** be enabled prior to using XrSpatialAnchorCreateInfoHTC
- `type` **must** be `XR_TYPE_SPATIAL_ANCHOR_CREATE_INFO_HTC`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `space` **must** be a valid XrSpace handle
- `name` **must** be a valid XrSpatialAnchorNameHTC structure

The XrSpatialAnchorNameHTC structure is defined as:

```
// Provided by XR_HTC_anchor
typedef struct XrSpatialAnchorNameHTC {
    char    name[XR_MAX_SPATIAL_ANCHOR_NAME_SIZE_HTC];
} XrSpatialAnchorNameHTC;
```

Member Descriptions

- `name` is a null-terminated UTF-8 string whose length is less than or equal to `XR_MAX_SPATIAL_ANCHOR_NAME_SIZE_HTC`.

Valid Usage (Implicit)

- The XR_HTC_anchor extension **must** be enabled prior to using XrSpatialAnchorNameHTC
- name **must** be a null-terminated UTF-8 string whose length is less than or equal to XR_MAX_SPATIAL_ANCHOR_NAME_SIZE_HTC

The xrGetSpatialAnchorNameHTC function is defined as:

```
// Provided by XR_HTC_anchor
XrResult xrGetSpatialAnchorNameHTC(
    XrSpace                                     anchor,
    XrSpatialAnchorNameHTC*                     name);
```

Parameter Descriptions

- anchor is the XrSpace created by xrCreateSpatialAnchorHTC.
- name is a pointer to output XrSpatialAnchorNameHTC.

The xrGetSpatialAnchorNameHTC function gets the name of an anchor. If the provided anchor is a valid space handle but was **not** created with xrCreateSpatialAnchorHTC, the runtime **must** return XR_ERROR_NOT_AN_ANCHOR_HTC.

Valid Usage (Implicit)

- The XR_HTC_anchor extension **must** be enabled prior to calling xrGetSpatialAnchorNameHTC
- anchor **must** be a valid XrSpace handle
- name **must** be a pointer to an XrSpatialAnchorNameHTC structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_NOT_AN_ANCHOR_HTC
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

New Object Types

New Flag Types

New Enum Constants

- XR_MAX_SPATIAL_ANCHOR_NAME_SIZE_HTC

XrStructureType enumeration is extended with:

- XR_TYPE_SYSTEM_ANCHOR_PROPERTIES_HTC
- XR_TYPE_SPATIAL_ANCHOR_CREATE_INFO_HTC

XrResult enumeration is extended with:

- XR_ERROR_NOT_AN_ANCHOR_HTC

New Enums

New Structures

- XrSystemAnchorPropertiesHTC
- XrSpatialAnchorCreateInfoHTC
- XrSpatialAnchorNameHTC

New Functions

- xrCreateSpatialAnchorHTC
- xrGetSpatialAnchorNameHTC

Issues

Version History

- Revision 1, 2023-09-14 (CheHsuan Shu)
  - Initial extension description

12.134. XR_HTC_body_tracking

Name String

`XR_HTC_body_tracking`

Extension Type

Instance extension

Registered Extension Number

321

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Last Modified Date

2023-01-17

IP Status

No known IP claims.

Contributors

Kyle Chen, HTC
Chris Kuo, HTC

12.134.1. Overview

This extension allows an application to locate the user's individual body joints. It enables applications to render the full body in XR experience.

12.134.2. Inspect system capability

The XrSystemBodyTrackingPropertiesHTC structure is defined as:

```
// Provided by XR_HTC_body_tracking
typedef struct XrSystemBodyTrackingPropertiesHTC {
    XrStructureType    type;
    void*              next;
    XrBool32           supportsBodyTracking;
} XrSystemBodyTrackingPropertiesHTC;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. For more details about the structure chain, see the structure being extended (XrSystemProperties).
- `supportsBodyTracking` indicates if the current system is capable of track individual body joints.

An application **can** inspect whether the system is capable of body tracking by extending the XrSystemProperties with XrSystemBodyTrackingPropertiesHTC structure when calling xrGetSystemProperties.

If a runtime returns `XR_FALSE` for `supportsBodyTracking`, the runtime **must** return `XR_ERROR_FEATURE_UNSUPPORTED` from xrCreateBodyTrackerHTC.

Valid Usage (Implicit)

- The `XR_HTC_body_tracking` extension **must** be enabled prior to using XrSystemBodyTrackingPropertiesHTC
- `type` **must** be `XR_TYPE_SYSTEM_BODY_TRACKING_PROPERTIES_HTC`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

12.134.3. Create a body tracker handle

The XrBodyTrackerHTC handle represents the resources for a body tracker.

```
// Provided by XR_HTC_body_tracking
XR_DEFINE_HANDLE(XrBodyTrackerHTC)
```

An application **can** create an XrBodyTrackerHTC handle which is used to locate individual body joints

with an unobstructed range of motion using xrLocateBodyJointsHTC function.

The xrCreateBodyTrackerHTC function is defined as

```
// Provided by XR_HTC_body_tracking
XrResult xrCreateBodyTrackerHTC(
    XrSession                                   session,
    const XrBodyTrackerCreateInfoHTC*           createInfo,
    XrBodyTrackerHTC*                           bodyTracker);
```

Parameter Descriptions

- `session` is an XrSession in which the body tracking will be active.
- `createInfo` is the XrBodyTrackerCreateInfoHTC is used to specify the set of body joints.
- `bodyTracker` is the returned XrBodyTrackerHTC handle.

An application **can** create an XrBodyTrackerHTC handle using xrCreateBodyTrackerHTC.

If the system does not support body tracking, the runtime **must** return `XR_ERROR_FEATURE_UNSUPPORTED` from xrCreateBodyTrackerHTC. In this case, the runtime **must** return `XR_FALSE` for XrSystemBodyTrackingPropertiesHTC::`supportsBodyTracking` in XrSystemBodyTrackingPropertiesHTC when the function xrGetSystemProperties is called, so that the application avoids creating a body tracker.

Valid Usage (Implicit)

- The `XR_HTC_body_tracking` extension **must** be enabled prior to calling xrCreateBodyTrackerHTC
- `session` **must** be a valid XrSession handle
- `createInfo` **must** be a pointer to a valid XrBodyTrackerCreateInfoHTC structure
- `bodyTracker` **must** be a pointer to an XrBodyTrackerHTC handle

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FEATURE_UNSUPPORTED
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The XrBodyTrackerCreateInfoHTC structure is defined as:

```
// Provided by XR_HTC_body_tracking
typedef struct XrBodyTrackerCreateInfoHTC {
    XrStructureType      type;
    const void*          next;
    XrBodyJointSetHTC    bodyJointSet;
} XrBodyTrackerCreateInfoHTC;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- bodyJointSet is an XrBodyJointSetHTC value that describes the set of body joints to retrieve.

The XrBodyTrackerCreateInfoHTC structure describes the information to create an XrBodyTrackerHTC handle. If the supplied bodyJointSet is not valid, the runtime **must** return XR_ERROR_VALIDATION_FAILURE.
.Valid Usage (Implicit)

- The XR_HTC_body_tracking extension **must** be enabled prior to using XrBodyTrackerCreateInfoHTC
- type **must** be XR_TYPE_BODY_TRACKER_CREATE_INFO_HTC
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- bodyJointSet **must** be a valid XrBodyJointSetHTC value

The xrDestroyBodyTrackerHTC function is defined as:

```
// Provided by XR_HTC_body_tracking
XrResult xrDestroyBodyTrackerHTC(
    XrBodyTrackerHTC                            bodyTracker);
```

Parameter Descriptions

- bodyTracker is an XrBodyTrackerHTC previously created by xrCreateBodyTrackerHTC.

xrDestroyBodyTrackerHTC releases the bodyTracker and the underlying resources when finished with body tracking experiences.

Valid Usage (Implicit)

- The XR_HTC_body_tracking extension **must** be enabled prior to calling xrDestroyBodyTrackerHTC
- bodyTracker **must** be a valid XrBodyTrackerHTC handle

Thread Safety

- Access to bodyTracker, and any child handles, **must** be externally synchronized

Return Codes

Success

- XR_SUCCESS

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_RUNTIME_FAILURE

12.134.4. Locate body joints

The xrLocateBodyJointsHTC function is defined as:

```
// Provided by XR_HTC_body_tracking
XrResult xrLocateBodyJointsHTC(
    XrBodyTrackerHTC                            bodyTracker,
    const XrBodyJointsLocateInfoHTC*            locateInfo,
    XrBodyJointLocationsHTC*                    locations);
```

Parameter Descriptions

- bodyTracker is an XrBodyTrackerHTC previously created by xrCreateBodyTrackerHTC.
- locateInfo is a pointer to XrBodyJointsLocateInfoHTC describing information to locate body joints.
- locations is a pointer to XrBodyJointLocationsHTC receiving the returned body joint locations.

The xrLocateBodyJointsHTC function locates an array of body joints relative to a base space at a given time.

If XrBodyJointLocationsHTC::jointLocationCount does not match the value associated with the supplied XrBodyJointSetHTC value, the runtime **must** return XR_ERROR_VALIDATION_FAILURE.

Valid Usage (Implicit)

- The XR_HTC_body_tracking extension **must** be enabled prior to calling xrLocateBodyJointsHTC
- bodyTracker **must** be a valid XrBodyTrackerHTC handle
- locateInfo **must** be a pointer to a valid XrBodyJointsLocateInfoHTC structure
- locations **must** be a pointer to an XrBodyJointLocationsHTC structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_TIME_INVALID
- XR_ERROR_VALIDATION_FAILURE

The XrBodyJointsLocateInfoHTC structure is defined as:

```
// Provided by XR_HTC_body_tracking
typedef struct XrBodyJointsLocateInfoHTC {
    XrStructureType    type;
    const void*        next;
    XrSpace            baseSpace;
    XrTime             time;
} XrBodyJointsLocateInfoHTC;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `baseSpace` is an XrSpace within which the returned body joint locations will be represented.
- `time` is an `XrTime` at which to locate the body joints.

The XrBodyJointsLocateInfoHTC structure describes the information to locate individual body joints.

Valid Usage (Implicit)

- The `XR_HTC_body_tracking` extension **must** be enabled prior to using XrBodyJointsLocateInfoHTC
- `type` **must** be `XR_TYPE_BODY_JOINTS_LOCATE_INFO_HTC`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `baseSpace` **must** be a valid XrSpace handle

The XrBodyJointLocationsHTC structure is defined as:

```
// Provided by XR_HTC_body_tracking
typedef struct XrBodyJointLocationsHTC {
    XrStructureType             type;
    void*                       next;
    XrSpaceLocationFlags        combinedLocationFlags;
    XrBodyJointConfidenceHTC    confidenceLevel;
    uint32_t                    jointLocationCount;
    XrBodyJointLocationHTC*     jointLocations;
    uint32_t                    skeletonGenerationId;
} XrBodyJointLocationsHTC;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `combinedLocationFlags` is a bitfield, with bit masks defined in XrSpaceLocationFlagBits, containing the bitwise OR of the location flags of all body joints.
- `confidenceLevel` is an XrBodyJointConfidenceHTC value which indicates the confidence level for the returned body joint pose.
- `jointLocationCount` is a `uint32_t` describing the count of elements in `jointLocations` array.
- `jointLocations` is an application-allocated XrBodyJointLocationHTC array filled in by the runtime specifying the locations for individual body joints.
- `skeletonGenerationId` is a unique identifier, representing the skeleton which is generated when the tracking auto-calibrates the user skeleton scale and proportions.

The application **must** set `jointLocationCount` as appropriate for the chosen XrBodyJointSetHTC value when creating the XrBodyTrackerHTC. If `jointLocationCount` does not match the value associated with the supplied XrBodyJointSetHTC value, the runtime **must** return `XR_ERROR_VALIDATION_FAILURE` from xrLocateBodyJointsHTC.

An application **must** allocate the output `jointLocations` array with a minimum capacity of `jointLocationCount` of XrBodyJointLocationHTC elements. If the application supplies a `NULL` value for `jointLocations`, the runtime **must** return `XR_ERROR_VALIDATION_FAILURE`.

The runtime **must** update the `jointLocations` array elements indexed using the corresponding body joint enumeration (e.g. XrBodyJointHTC for the joint set `XR_BODY_JOINT_SET_FULL_HTC`) as described by XrBodyJointSetHTC when creating the XrBodyTrackerHTC. For example, when the XrBodyTrackerHTC is created with `XR_BODY_JOINT_SET_FULL_HTC`, the runtime **must** fill the `jointLocations` array with body joint data indexed by the XrBodyJointHTC enumeration.

If the runtime returns `combinedLocationFlags` with `XR_SPACE_LOCATION_POSITION_VALID_BIT` and `XR_SPACE_LOCATION_ORIENTATION_VALID_BIT` set, it indicates that the body tracker detects the joint space locations. If the runtime returns `combinedLocationFlags` with neither `XR_SPACE_LOCATION_POSITION_VALID_BIT` nor `XR_SPACE_LOCATION_ORIENTATION_VALID_BIT` set, it indicates that the body tracker did not detect the joint space locations.

The purpose of the skeleton is to provide data about the body size. The calculation of the body size **may** be updated during a session. Each time the calculation of the size is changed, XrBodyJointLocationsHTC::`skeletonGenerationId` is changed to indicate that a new skeleton **may** be retrieved. xrGetBodySkeletonHTC **can** be called with the specified skeletonGenerationId to get the corresponding skeleton.

Valid Usage (Implicit)

- The XR_HTC_body_tracking extension **must** be enabled prior to using XrBodyJointLocationsHTC
- type **must** be XR_TYPE_BODY_JOINT_LOCATIONS_HTC
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- combinedLocationFlags **must** be 0 or a valid combination of XrSpaceLocationFlagBits values
- confidenceLevel **must** be a valid XrBodyJointConfidenceHTC value
- jointLocations **must** be a pointer to an array of jointLocationCount XrBodyJointLocationHTC structures
- The jointLocationCount parameter **must** be greater than 0

The XrBodyJointConfidenceHTC enumeration is defined as:

```
// Provided by XR_HTC_body_tracking
typedef enum XrBodyJointConfidenceHTC {
    XR_BODY_JOINT_CONFIDENCE_NONE_HTC = 0,
    XR_BODY_JOINT_CONFIDENCE_LOW_HTC = 1,
    XR_BODY_JOINT_CONFIDENCE_HIGH_HTC = 2,
    XR_BODY_JOINT_CONFIDENCE_MAX_ENUM_HTC = 0x7FFFFFFF
} XrBodyJointConfidenceHTC;
```

Enumerant Descriptions

- XR_BODY_JOINT_CONFIDENCE_NONE_HTC — No confidence for the returned body joint pose.
- XR_BODY_JOINT_CONFIDENCE_LOW_HTC — Low confience for the returned body joint pose. It indicates some of the body joints may not be tracked.
- XR_BODY_JOINT_CONFIDENCE_HIGH_HTC — High confidence for the returned body joint pose. It indicates almost all of the critical body joints are tracked.

The XrBodyJointConfidenceHTC enumeration describes the confidence level for the returned body joint pose.

The XrBodyJointLocationHTC structure is defined as:

```
// Provided by XR_HTC_body_tracking
typedef struct XrBodyJointLocationHTC {
    XrSpaceLocationFlags    locationFlags;
    XrPosef                 pose;
} XrBodyJointLocationHTC;
```

Member Descriptions

- `locationFlags` is a bitfield, with bit masks defined in XrSpaceLocationFlagBits, to indicate which members contain valid data. For those bits that are not set, reading the relevant fields in this structure and its nested structures **must** be defined, but the values are unspecified and not meaningful.
- `pose` is an XrPosef describing the position and orientation of the origin of a body joint within the reference frame of the corresponding XrBodyJointsLocateInfoHTC::`baseSpace`.

XrBodyJointLocationHTC structure describes the position, orientation, and location flag of a body joint. It is populated by the runtime during a call to xrLocateBodyJointsHTC.

Valid Usage (Implicit)

- The `XR_HTC_body_tracking` extension **must** be enabled prior to using XrBodyJointLocationHTC

12.134.5. Get body skeleton

The xrGetBodySkeletonHTC function is defined as:

```
// Provided by XR_HTC_body_tracking
XrResult xrGetBodySkeletonHTC(
    XrBodyTrackerHTC                            bodyTracker,
    XrSpace                                     baseSpace,
    uint32_t                                    skeletonGenerationId,
    XrBodySkeletonHTC*                          skeleton);
```

Parameter Descriptions

- `bodyTracker` is an XrBodyTrackerHTC previously created by xrCreateBodyTrackerHTC.
- `baseSpace` is an XrSpace within which the returned body skeleton in T-pose will be represented.
- `skeletonGenerationId` is the skeleton id of the skeleton.
- `skeleton` is a pointer to XrBodySkeletonHTC receiving the returned body skeleton.

The xrGetBodySkeletonHTC function returns the body skeleton in T-pose. This function **can** be used to get body skeleton and infer the skeleton scale and proportions in conjunction with XrBodyJointLocationsHTC::`skeletonGenerationId`. XrBodyJointLocationsHTC::`skeletonGenerationId` is generated when the tracking auto-calibrates the user skeleton scale and proportions.

If the application supplies a `skeletonGenerationId` that does not match any value returned in XrBodyJointLocationsHTC::`skeletonGenerationId` during the current session, the runtime **must** return `XR_ERROR_VALIDATION_FAILURE`.

Valid Usage (Implicit)

- The `XR_HTC_body_tracking` extension **must** be enabled prior to calling xrGetBodySkeletonHTC
- `bodyTracker` **must** be a valid XrBodyTrackerHTC handle
- `baseSpace` **must** be a valid XrSpace handle
- `skeleton` **must** be a pointer to an XrBodySkeletonHTC structure
- Both of `baseSpace` and `bodyTracker` **must** have been created, allocated, or retrieved from the same XrSession

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The XrBodySkeletonHTC structure is defined as:

```
// Provided by XR_HTC_body_tracking
typedef struct XrBodySkeletonHTC {
    XrStructureType            type;
    void*                      next;
    uint32_t                   jointCount;
    XrBodySkeletonJointHTC*    joints;
} XrBodySkeletonHTC;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- jointCount is an uint32_t describing the count of elements in joints array.
- joints is an application-allocated array of XrBodySkeletonJointHTC that will be filled with skeleton joint elements.

The XrBodySkeletonHTC structure is a container to represent the body skeleton in T-pose including each joint pose. The runtime **must** return XR_ERROR_VALIDATION_FAILURE if jointCount does not equal the number of joints associated with the XrBodyJointSetHTC value used to create the XrBodyTrackerHTC.

The application **must** allocate an array of at least jointCount elements for joints, to be populated by

the runtime. If joints is NULL, the runtime **must** return XR_ERROR_VALIDATION_FAILURE.

The runtime **must** return joints representing the default pose of the current estimation regarding the user's skeleton.

The runtime **must** update the joints array ordered so that it is indexed using the corresponding body joint enumeration (e.g. XrBodyJointHTC for the joint set XR_BODY_JOINT_SET_FULL_HTC) as associated with the XrBodyJointSetHTC value used when creating the XrBodyTrackerHTC. For example, when the XrBodyTrackerHTC is created with XR_BODY_JOINT_SET_FULL_HTC, the runtime **must** fill the joints array indexed by the XrBodyJointHTC enumeration.

Valid Usage (Implicit)

- The XR_HTC_body_tracking extension **must** be enabled prior to using XrBodySkeletonHTC
- type **must** be XR_TYPE_BODY_SKELETON_HTC
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- joints **must** be a pointer to an array of jointCount XrBodySkeletonJointHTC structures
- The jointCount parameter **must** be greater than 0

The XrBodySkeletonJointHTC structure is defined as:

```
// Provided by XR_HTC_body_tracking
typedef struct XrBodySkeletonJointHTC {
    XrPosef    pose;
} XrBodySkeletonJointHTC;
```

Member Descriptions

- pose is an XrPosef defining the position and orientation of the origin of a body joint within the reference frame of the corresponding xrGetBodySkeletonHTC::baseSpace.

XrBodySkeletonJointHTC structure describes the position, orientation of the joint in space, and position of the joint in the skeleton.

Valid Usage (Implicit)

- The XR_HTC_body_tracking extension **must** be enabled prior to using XrBodySkeletonJointHTC

12.134.6. Conventions of body joints

The XrBodyJointSetHTC enumeration is defined as:

```
// Provided by XR_HTC_body_tracking
typedef enum XrBodyJointSetHTC {
    XR_BODY_JOINT_SET_FULL_HTC = 0,
    XR_BODY_JOINT_SET_MAX_ENUM_HTC = 0x7FFFFFFF
} XrBodyJointSetHTC;
```

Enumerant Descriptions

- XR_BODY_JOINT_SET_FULL_HTC indicates that the created XrBodyTrackerHTC tracks the set of body joints described by XrBodyJointHTC enum, i.e. the xrLocateBodyJointsHTC function returns an array of joint locations with the count of XR_BODY_JOINT_COUNT_HTC and is indexed using XrBodyJointHTC.

The XrBodyJointSetHTC enumeration describes the set of body joints to track when creating an XrBodyTrackerHTC.

The XrBodyJointHTC enumeration is defined as:

```
// Provided by XR_HTC_body_tracking
typedef enum XrBodyJointHTC {
    XR_BODY_JOINT_PELVIS_HTC = 0,
    XR_BODY_JOINT_LEFT_HIP_HTC = 1,
    XR_BODY_JOINT_LEFT_KNEE_HTC = 2,
    XR_BODY_JOINT_LEFT_ANKLE_HTC = 3,
    XR_BODY_JOINT_LEFT_FEET_HTC = 4,
    XR_BODY_JOINT_RIGHT_HIP_HTC = 5,
    XR_BODY_JOINT_RIGHT_KNEE_HTC = 6,
    XR_BODY_JOINT_RIGHT_ANKLE_HTC = 7,
    XR_BODY_JOINT_RIGHT_FEET_HTC = 8,
    XR_BODY_JOINT_WAIST_HTC = 9,
    XR_BODY_JOINT_SPINE_LOWER_HTC = 10,
    XR_BODY_JOINT_SPINE_MIDDLE_HTC = 11,
    XR_BODY_JOINT_SPINE_HIGH_HTC = 12,
    XR_BODY_JOINT_CHEST_HTC = 13,
    XR_BODY_JOINT_NECK_HTC = 14,
    XR_BODY_JOINT_HEAD_HTC = 15,
    XR_BODY_JOINT_LEFT_CLAVICLE_HTC = 16,
    XR_BODY_JOINT_LEFT_SCAPULA_HTC = 17,
    XR_BODY_JOINT_LEFT_ARM_HTC = 18,
    XR_BODY_JOINT_LEFT_ELBOW_HTC = 19,
    XR_BODY_JOINT_LEFT_WRIST_HTC = 20,
    XR_BODY_JOINT_RIGHT_CLAVICLE_HTC = 21,
    XR_BODY_JOINT_RIGHT_SCAPULA_HTC = 22,
    XR_BODY_JOINT_RIGHT_ARM_HTC = 23,
    XR_BODY_JOINT_RIGHT_ELBOW_HTC = 24,
    XR_BODY_JOINT_RIGHT_WRIST_HTC = 25,
    XR_BODY_JOINT_MAX_ENUM_HTC = 0x7FFFFFFF
} XrBodyJointHTC;
```

It is used to index into a joint location array when the joint set in use (XrBodyJointSetHTC) is XR_BODY_JOINT_SET_FULL_HTC.

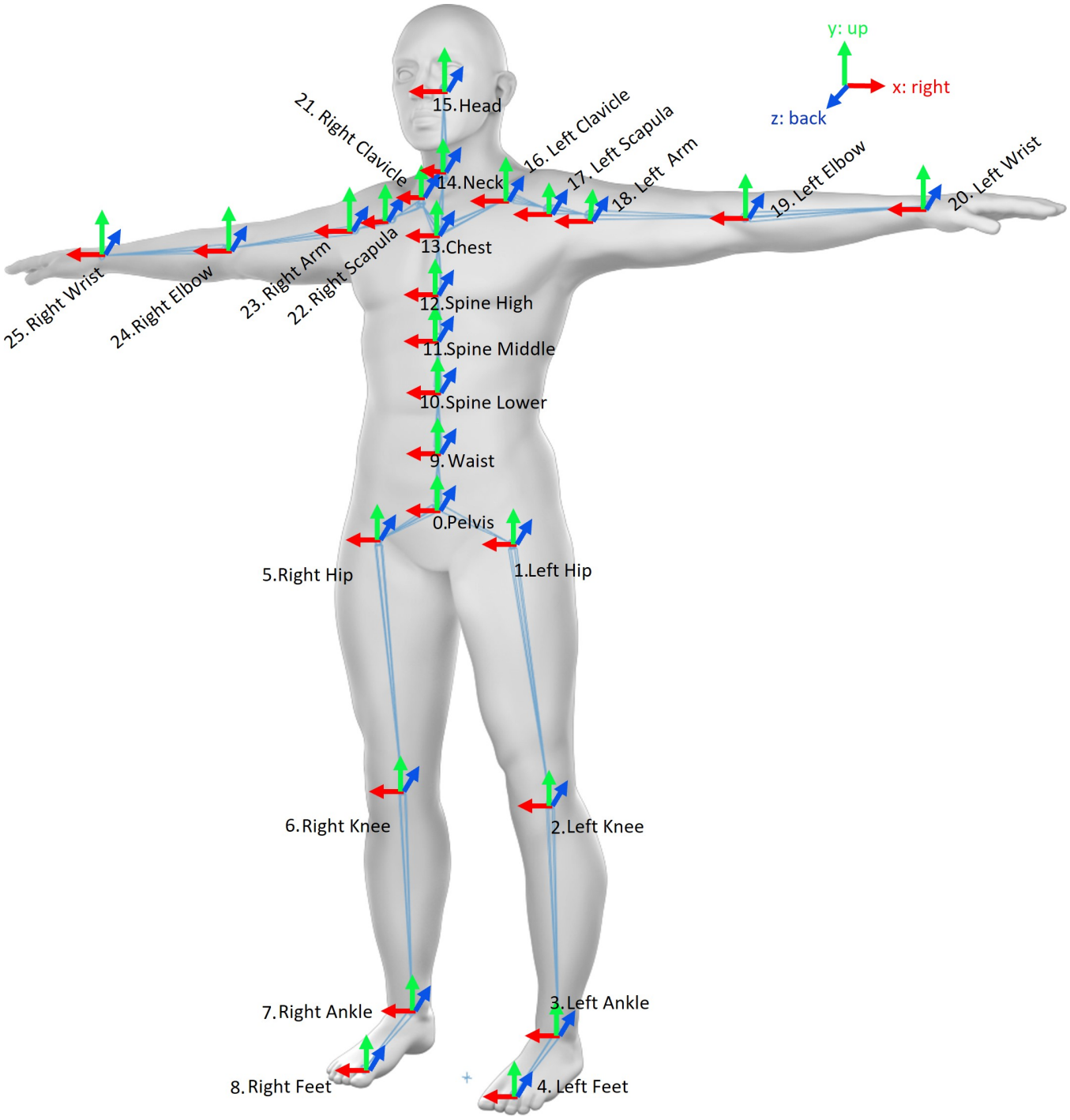
This extension defines 26 joints for body tracking: 6 joints for the torso, 5 joints for each arm, 4 joints for each leg, and the other 2 joints for the head and neck. The definitions of these joints are based on human skeletal joints.

As shown in the figure below, the following conventions are stated with a T-shape body pose in which the palms are facing down to the ground.

The right direction (+X) is pointing from left hand to right hand in T-pose.

The up direction (+Y) is pointing from foot to head in T-pose.

The Z direction is perpendicular to X and Y and follows the right hand rule in T-pose.




```
// Provided by XR_HTC_body_tracking
#define XR_BODY_JOINT_COUNT_HTC 26
```

XR_BODY_JOINT_COUNT_HTC defines the number of body joint enumerants defined in XrBodyJointHTC.

12.134.7. Example code for locating body joints

The following example code demonstrates how to locate all individual body joints relative to a world

space.

```
XrInstance instance; // previously initialized
XrSystemId systemId; // previously initialized
XrSession session;   // previously initialized
XrSpace worldSpace;  // previously initialized, e.g. from
                     // XR_REFERENCE_SPACE_TYPE_LOCAL

// Inspect body tracking system properties
XrSystemBodyTrackingPropertiesHTC bodyTrackingSystemProperties {
  .type = XR_TYPE_SYSTEM_BODY_TRACKING_PROPERTIES_HTC,
};
XrSystemProperties systemProperties {
  .type = XR_TYPE_SYSTEM_PROPERTIES,
  .next = &bodyTrackingSystemProperties,
};
CHK_XR(xrGetSystemProperties(instance, systemId, &systemProperties));
if (!bodyTrackingSystemProperties.supportsBodyTracking) {
    // The system does not support body tracking
    return;
}

// Get function pointer for xrCreateBodyTrackerHTC
PFN_xrCreateBodyTrackerHTC pfnCreateBodyTrackerHTC = nullptr;
CHK_XR(xrGetInstanceProcAddr(instance, "xrCreateBodyTrackerHTC",
                             reinterpret_cast<PFN_xrVoidFunction*>(
                             &pfnCreateBodyTrackerHTC)));

// Create a body tracker that tracks default set of individual body joints.
XrBodyTrackerHTC bodyTracker {XR_NULL_HANDLE};
{
    XrBodyTrackerCreateInfoHTC createInfo {
      .type = XR_TYPE_BODY_TRACKER_CREATE_INFO_HTC,
      .bodyJointSet = XR_BODY_JOINT_SET_FULL_HTC,
    };
    CHK_XR(pfnCreateBodyTrackerHTC(session, &createInfo, &bodyTracker));
}

// Allocate buffers to receive joint location before frame loop starts
XrBodyJointLocationHTC jointLocations[XR_BODY_JOINT_COUNT_HTC];
XrBodyJointLocationsHTC locations {
  .type = XR_TYPE_BODY_JOINT_LOCATIONS_HTC,
  .jointLocationCount = XR_BODY_JOINT_COUNT_HTC,
  .jointLocations = jointLocations,
};

// Get function pointer for xrLocateBodyJointsHTC
```

```
PFN_xrLocateBodyJointsHTC pfnLocateBodyJointsHTC = nullptr;
CHK_XR(xrGetInstanceProcAddr(instance, "xrLocateBodyJointsHTC",
                             reinterpret_cast<PFN_xrVoidFunction*>(
                             &pfnLocateBodyJointsHTC)));
while (1) {
    // ...
    // For every frame in frame loop
    // ...
    XrFrameState frameState; // previously returned from xrWaitFrame
    const XrTime time = frameState.predictedDisplayTime;

    XrBodyJointsLocateInfoHTC locateInfo {
      .type = XR_TYPE_BODY_JOINTS_LOCATE_INFO_HTC,
      .baseSpace = worldSpace,
      .time = time,
    };

    CHK_XR(pfnLocateBodyJointsHTC(bodyTracker, &locateInfo, &locations));

    // The returned joint location array is directly indexed with
    // XrBodyJointHTC enum.
    const XrPosef &pelvisInWorld =
        jointLocations[XR_BODY_JOINT_PELVIS_HTC].pose;
    const XrPosef &headInWorld =
        jointLocations[XR_BODY_JOINT_HEAD_HTC].pose;

}
```

12.134.8. New Macros

- XR_BODY_JOINT_COUNT_HTC

12.134.9. New Object Types

- XrBodyTrackerHTC

12.134.10. New Commands

- xrCreateBodyTrackerHTC
- xrDestroyBodyTrackerHTC
- xrGetBodySkeletonHTC
- xrLocateBodyJointsHTC

12.134.11. New Structures

- XrBodyJointLocationHTC
- XrBodyJointLocationsHTC
- XrBodyJointsLocateInfoHTC
- XrBodySkeletonHTC
- XrBodySkeletonJointHTC
- XrBodyTrackerCreateInfoHTC
- Extending XrSystemProperties:
  - XrSystemBodyTrackingPropertiesHTC

12.134.12. New Enums

- XrBodyJointConfidenceHTC
- XrBodyJointHTC
- XrBodyJointSetHTC

12.134.13. New Enum Constants

- `XR_HTC_BODY_TRACKING_EXTENSION_NAME`
- `XR_HTC_body_tracking_SPEC_VERSION`
- Extending XrObjectType:
  - `XR_OBJECT_TYPE_BODY_TRACKER_HTC`
- Extending XrStructureType:
  - `XR_TYPE_BODY_JOINTS_LOCATE_INFO_HTC`
  - `XR_TYPE_BODY_JOINT_LOCATIONS_HTC`
  - `XR_TYPE_BODY_SKELETON_HTC`
  - `XR_TYPE_BODY_TRACKER_CREATE_INFO_HTC`
  - `XR_TYPE_SYSTEM_BODY_TRACKING_PROPERTIES_HTC`

12.134.14. Version History

- Revision 1, 2023-01-17 (Kyle Chen)
  - Initial extension description

12.135. XR_HTC_facial_tracking

Name String

`XR_HTC_facial_tracking`

Extension Type

Instance extension

Registered Extension Number

105

Revision

3

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Last Modified Date

2024-07-26

IP Status

No known IP claims.

Contributors

Kyle Chen, HTC
Chris Kuo, HTC
Andy Chen, HTC

Overview

This extension allows an application to track and integrate users' eye and lip movements, empowering developers to read intention and model facial expressions.

Inspect system capability

XrSystemFacialTrackingPropertiesHTC is defined as:

```
// Provided by XR_HTC_facial_tracking
typedef struct XrSystemFacialTrackingPropertiesHTC {
    XrStructureType    type;
    void*              next;
    XrBool32           supportEyeFacialTracking;
    XrBool32           supportLipFacialTracking;
} XrSystemFacialTrackingPropertiesHTC;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension. For more details about the structure chain, see the structure being extended (XrSystemProperties).
- `supportEyeFacialTracking` indicates if the current system is capable of generating eye expressions.
- `supportLipFacialTracking` indicates if the current system is capable of generating lip expressions.

An application **can** inspect whether the system is capable of two of the facial tracking by extending the XrSystemProperties with XrSystemFacialTrackingPropertiesHTC structure when calling xrGetSystemProperties.

Valid Usage (Implicit)

- The `XR_HTC_facial_tracking` extension **must** be enabled prior to using XrSystemFacialTrackingPropertiesHTC
- `type` **must** be `XR_TYPE_SYSTEM_FACIAL_TRACKING_PROPERTIES_HTC`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

If a runtime returns `XR_FALSE` for `supportEyeFacialTracking`, the runtime **must** return `XR_ERROR_FEATURE_UNSUPPORTED` from xrCreateFacialTrackerHTC with `XR_FACIAL_TRACKING_TYPE_EYE_DEFAULT_HTC` set for XrFacialTrackingTypeHTC in XrFacialTrackerCreateInfoHTC. Similarly, if a runtime returns `XR_FALSE` for `supportLipFacialTracking` the runtime **must** return `XR_ERROR_FEATURE_UNSUPPORTED` from xrCreateFacialTrackerHTC with `XR_FACIAL_TRACKING_TYPE_LIP_DEFAULT_HTC` set for XrFacialTrackingTypeHTC in XrFacialTrackerCreateInfoHTC.

Create a facial tracker handle

The XrFacialTrackerHTC handle represents the resources for an facial tracker of the specific facial tracking type.

```
// Provided by XR_HTC_facial_tracking
XR_DEFINE_HANDLE(XrFacialTrackerHTC)
```

An application creates separate XrFacialTrackerHTC handles for eye tracker or lip tracker. This handle can be used to retrieve corresponding facial expressions using xrGetFacialExpressionsHTC function.

The xrCreateFacialTrackerHTC function is defined as

```
// Provided by XR_HTC_facial_tracking
XrResult xrCreateFacialTrackerHTC(
    XrSession                                   session,
    const XrFacialTrackerCreateInfoHTC*         createInfo,
    XrFacialTrackerHTC*                         facialTracker);
```

Parameter Descriptions

- `session` is an XrSession in which the facial expression will be active.
- `createInfo` is the XrFacialTrackerCreateInfoHTC used to specify the facial tracking type.
- `facialTracker` is the returned XrFacialTrackerHTC handle.

An application **can** create an XrFacialTrackerHTC handle using xrCreateFacialTrackerHTC.

If the system does not support eye tracking or lip tracking, runtime **must** return `XR_ERROR_FEATURE_UNSUPPORTED` from xrCreateFacialTrackerHTC according to the corresponding case. In this case, the runtime **must** return `XR_FALSE` for XrSystemFacialTrackingPropertiesHTC ::`supportEyeFacialTracking` or XrSystemFacialTrackingPropertiesHTC::`supportLipFacialTracking` when the function xrGetSystemProperties is called, so that the application **may** avoid creating a facial tracker.

Valid Usage (Implicit)

- The `XR_HTC_facial_tracking` extension **must** be enabled prior to calling xrCreateFacialTrackerHTC
- `session` **must** be a valid XrSession handle
- `createInfo` **must** be a pointer to a valid XrFacialTrackerCreateInfoHTC structure
- `facialTracker` **must** be a pointer to an XrFacialTrackerHTC handle

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FEATURE_UNSUPPORTED
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The XrFacialTrackerCreateInfoHTC structure is defined as:

```
// Provided by XR_HTC_facial_tracking
typedef struct XrFacialTrackerCreateInfoHTC {
    XrStructureType            type;
    const void*                next;
    XrFacialTrackingTypeHTC    facialTrackingType;
} XrFacialTrackerCreateInfoHTC;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- facialTrackingType is an XrFacialTrackingTypeHTC which describes which type of facial tracking should be used for this handle.

The XrFacialTrackerCreateInfoHTC structure describes the information to create an XrFacialTrackerHTC handle.

Valid Usage (Implicit)

- The XR_HTC_facial_tracking extension **must** be enabled prior to using XrFacialTrackerCreateInfoHTC
- type **must** be XR_TYPE_FACIAL_TRACKER_CREATE_INFO_HTC
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- facialTrackingType **must** be a valid XrFacialTrackingTypeHTC value

The XrFacialTrackingTypeHTC describes which type of tracking the XrFacialTrackerHTC is using.

```
// Provided by XR_HTC_facial_tracking
typedef enum XrFacialTrackingTypeHTC {
    XR_FACIAL_TRACKING_TYPE_EYE_DEFAULT_HTC = 1,
    XR_FACIAL_TRACKING_TYPE_LIP_DEFAULT_HTC = 2,
    XR_FACIAL_TRACKING_TYPE_MAX_ENUM_HTC = 0x7FFFFFFF
} XrFacialTrackingTypeHTC;
```

Enumerant Descriptions

- XR_FACIAL_TRACKING_TYPE_EYE_DEFAULT_HTC — Specifies this handle will observe eye expressions, with values indexed by XrEyeExpressionHTC whose count is XR_FACIAL_EXPRESSION_EYE_COUNT_HTC.
- XR_FACIAL_TRACKING_TYPE_LIP_DEFAULT_HTC — Specifies this handle will observe lip expressions, with values indexed by XrLipExpressionHTC whose count is XR_FACIAL_EXPRESSION_LIP_COUNT_HTC.

The xrDestroyFacialTrackerHTC function is defined as:

```
// Provided by XR_HTC_facial_tracking
XrResult xrDestroyFacialTrackerHTC(
    XrFacialTrackerHTC                          facialTracker);
```

Parameter Descriptions

- facialTracker is an XrFacialTrackerHTC previously created by xrCreateFacialTrackerHTC.

xrDestroyFacialTrackerHTC releases the facialTracker and the underlying resources when finished with facial tracking experiences.

Valid Usage (Implicit)

- The XR_HTC_facial_tracking extension **must** be enabled prior to calling xrDestroyFacialTrackerHTC
- facialTracker **must** be a valid XrFacialTrackerHTC handle

Thread Safety

- Access to facialTracker, and any child handles, **must** be externally synchronized

Return Codes

Success

- XR_SUCCESS

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_RUNTIME_FAILURE

Retrieve facial expressions

The xrGetFacialExpressionsHTC function is defined as:

```
// Provided by XR_HTC_facial_tracking
XrResult xrGetFacialExpressionsHTC(
    XrFacialTrackerHTC                          facialTracker,
    XrFacialExpressionsHTC*                     facialExpressions);
```

Parameter Descriptions

- facialTracker is an XrFacialTrackerHTC previously created by xrCreateFacialTrackerHTC.
- facialExpressions is a pointer to XrFacialExpressionsHTC receiving the returned facial expressions.

xrGetFacialExpressionsHTC retrieves an array of values of blend shapes for a facial expression on a given time.

Valid Usage (Implicit)

- The XR_HTC_facial_tracking extension **must** be enabled prior to calling xrGetFacialExpressionsHTC
- facialTracker **must** be a valid XrFacialTrackerHTC handle
- facialExpressions **must** be a pointer to an XrFacialExpressionsHTC structure

Return Codes

Success

- XR_SUCCESS

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_TIME_INVALID
- XR_ERROR_VALIDATION_FAILURE

The XrFacialExpressionsHTC structure is defined as:

```
// Provided by XR_HTC_facial_tracking
typedef struct XrFacialExpressionsHTC {
    XrStructureType    type;
    const void*        next;
    XrBool32           isActive;
    XrTime             sampleTime;
    uint32_t           expressionCount;
    float*             expressionWeightings;
} XrFacialExpressionsHTC;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `isActive` is an `XrBool32` indicating if the facial tracker is active.
- `sampleTime` is when in time the expression is expressed.
- `expressionCount` is a `uint32_t` describing the count of elements in `expressionWeightings` array.
- `expressionWeightings` is a `float` array filled in by the runtime, specifying the weightings for each blend shape.

XrFacialExpressionsHTC structure returns data of a lip facial expression or an eye facial expression.

An application **must** preallocate the output `expressionWeightings` array that can contain at least `expressionCount` of `float`. `expressionCount` **must** be at least XR_FACIAL_EXPRESSION_LIP_COUNT_HTC for `XR_FACIAL_TRACKING_TYPE_LIP_DEFAULT_HTC`, and at least XR_FACIAL_EXPRESSION_EYE_COUNT_HTC for `XR_FACIAL_TRACKING_TYPE_EYE_DEFAULT_HTC`.

The application **must** set `expressionCount` as described by the XrFacialTrackingTypeHTC when creating the XrFacialTrackerHTC otherwise the runtime **must** return `XR_ERROR_VALIDATION_FAILURE`.

The runtime **must** update the `expressionWeightings` array ordered so that the application can index elements using the corresponding facial tracker enum (e.g. XrEyeExpressionHTC or XrLipExpressionHTC) as described by XrFacialTrackingTypeHTC when creating the XrFacialTrackerHTC. For example, when the XrFacialTrackerHTC is created with XrFacialTrackerHTC ::`facialTrackingType` set to `XR_FACIAL_TRACKING_TYPE_EYE_DEFAULT_HTC`, the application **must** set the `expressionCount` to XR_FACIAL_EXPRESSION_EYE_COUNT_HTC, and the runtime **must** fill the `expressionWeightings` array ordered with eye expression data so that it can be indexed by the XrEyeExpressionHTC enum.

If the returned `isActive` is true, the runtime **must** fill the `expressionWeightings` array ordered.

If the returned `isActive` is false, it indicates the facial tracker did not detect the corresponding facial input or the application lost input focus.

If the input `expressionCount` is not sufficient to contain all output indices, the runtime **must** return `XR_ERROR_SIZE_INSUFFICIENT` on calls to xrGetFacialExpressionsHTC and not change the content in `expressionWeightings`.

Valid Usage (Implicit)

- The XR_HTC_facial_tracking extension **must** be enabled prior to using XrFacialExpressionsHTC
- type **must** be XR_TYPE_FACIAL_EXPRESSIONS_HTC
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- expressionWeightings **must** be a pointer to a float value

```
// Provided by XR_HTC_facial_tracking
#define XR_FACIAL_EXPRESSION_EYE_COUNT_HTC 14
```

The number of blend shapes in an expression of type XR_FACIAL_TRACKING_TYPE_EYE_DEFAULT_HTC.

```
// Provided by XR_HTC_facial_tracking
#define XR_FACIAL_EXPRESSION_LIP_COUNT_HTC 37
```

The number of blend shapes in an expression of type XR_FACIAL_TRACKING_TYPE_LIP_DEFAULT_HTC.

Facial Expression List

- **Eye Blend Shapes**

Through feeding the blend shape values of eye expression to an avatar, its facial expression can be animated with the player's eye movement. The following pictures show how the facial expression acts on the avatar according to each set of eye blend shape values.

```
// Provided by XR_HTC_facial_tracking
typedef enum XrEyeExpressionHTC {
    XR_EYE_EXPRESSION_LEFT_BLINK_HTC = 0,
    XR_EYE_EXPRESSION_LEFT_WIDE_HTC = 1,
    XR_EYE_EXPRESSION_RIGHT_BLINK_HTC = 2,
    XR_EYE_EXPRESSION_RIGHT_WIDE_HTC = 3,
    XR_EYE_EXPRESSION_LEFT_SQUEEZE_HTC = 4,
    XR_EYE_EXPRESSION_RIGHT_SQUEEZE_HTC = 5,
    XR_EYE_EXPRESSION_LEFT_DOWN_HTC = 6,
    XR_EYE_EXPRESSION_RIGHT_DOWN_HTC = 7,
    XR_EYE_EXPRESSION_LEFT_OUT_HTC = 8,
    XR_EYE_EXPRESSION_RIGHT_IN_HTC = 9,
    XR_EYE_EXPRESSION_LEFT_IN_HTC = 10,
    XR_EYE_EXPRESSION_RIGHT_OUT_HTC = 11,
    XR_EYE_EXPRESSION_LEFT_UP_HTC = 12,
    XR_EYE_EXPRESSION_RIGHT_UP_HTC = 13,
    XR_EYE_EXPRESSION_MAX_ENUM_HTC = 0x7FFFFFFF
} XrEyeExpressionHTC;
```

XR_EYE_EXPRESSION_LEFT_WIDE_HTC	
	**Description** This blend shape keeps left eye wide and at that time XR_EYE_EXPRESSION_LEFT_BLINK_HTC value is 0.

XR_EYE_EXPRESSION_RIGHT_WIDE_HTC	
	**Description** This blend shape keeps right eye wide and at that time XR_EYE_EXPRESSION_RIGHT_BLINK_HTC value is 0.

`XR_EYE_EXPRESSION_LEFT_BLINK_HTC`



Description
This blend shape influences blinking of the right eye. When this value goes higher, left eye approaches close.

`XR_EYE_EXPRESSION_RIGHT_BLINK_HTC`



Description
This blend shape influences blinking of the right eye. When this value goes higher, right eye approaches close.

`XR_EYE_EXPRESSION_LEFT_SQUEEZE_HTC`



Description
The blend shape closes eye tightly and at that time `XR_EYE_EXPRESSION_LEFT_BLINK_HTC` value is 1.

`XR_EYE_EXPRESSION_RIGHT_SQUEEZE_HTC`



Description
The blend shape closes eye tightly and at that time `XR_EYE_EXPRESSION_RIGHT_BLINK_HTC` value is 1.

XR_EYE_EXPRESSION_LEFT_DOWN_HTC	
	**Description** This blendShape influences the muscles around the left eye, moving these muscles further downward with a higher value.

XR_EYE_EXPRESSION_RIGHT_DOWN_HTC	
	**Description** This blendShape influences the muscles around the right eye, moving these muscles further downward with a higher value.

XR_EYE_EXPRESSION_LEFT_OUT_HTC	
	**Description** This blendShape influences the muscles around the left eye, moving these muscles further leftward with a higher value.

XR_EYE_EXPRESSION_RIGHT_IN_HTC	
	**Description** This blendShape influences the muscles around the right eye, moving these muscles further leftward with a higher value.

XR_EYE_EXPRESSION_LEFT_IN_HTC	
	**Description** This blendShape influences the muscles around the left eye, moving these muscles further rightward with a higher value.

XR_EYE_EXPRESSION_RIGHT_OUT_HTC	
	**Description** This blendShape influences the muscles around the right eye, moving these muscles further rightward with a higher value.

XR_EYE_EXPRESSION_LEFT_UP_HTC	
	**Description** This blendShape influences the muscles around the left eye, moving these muscles further upward with a higher value.

XR_EYE_EXPRESSION_RIGHT_UP_HTC	
	**Description** This blendShape influences the muscles around the right eye, moving these muscles further upward with a higher value.

- **Lip Blend Shapes**

Through feeding the blend shape values of lip expression to an avatar, its facial expression can be

animated with the player's lip movement. The following pictures show how the facial expression acts on the avatar according to each set of lip blend shape values.

```
// Provided by XR_HTC_facial_tracking
typedef enum XrLipExpressionHTC {
    XR_LIP_EXPRESSION_JAW_RIGHT_HTC = 0,
    XR_LIP_EXPRESSION_JAW_LEFT_HTC = 1,
    XR_LIP_EXPRESSION_JAW_FORWARD_HTC = 2,
    XR_LIP_EXPRESSION_JAW_OPEN_HTC = 3,
    XR_LIP_EXPRESSION_MOUTH_APE_SHAPE_HTC = 4,
    XR_LIP_EXPRESSION_MOUTH_UPPER_RIGHT_HTC = 5,
    XR_LIP_EXPRESSION_MOUTH_UPPER_LEFT_HTC = 6,
    XR_LIP_EXPRESSION_MOUTH_LOWER_RIGHT_HTC = 7,
    XR_LIP_EXPRESSION_MOUTH_LOWER_LEFT_HTC = 8,
    XR_LIP_EXPRESSION_MOUTH_UPPER_OVERTURN_HTC = 9,
    XR_LIP_EXPRESSION_MOUTH_LOWER_OVERTURN_HTC = 10,
    XR_LIP_EXPRESSION_MOUTH_POUT_HTC = 11,
    XR_LIP_EXPRESSION_MOUTH_RAISER_RIGHT_HTC = 12,
    XR_LIP_EXPRESSION_MOUTH_RAISER_LEFT_HTC = 13,
    XR_LIP_EXPRESSION_MOUTH_STRETCHER_RIGHT_HTC = 14,
    XR_LIP_EXPRESSION_MOUTH_STRETCHER_LEFT_HTC = 15,
    XR_LIP_EXPRESSION_CHEEK_PUFF_RIGHT_HTC = 16,
    XR_LIP_EXPRESSION_CHEEK_PUFF_LEFT_HTC = 17,
    XR_LIP_EXPRESSION_CHEEK_SUCK_HTC = 18,
    XR_LIP_EXPRESSION_MOUTH_UPPER_UPRIGHT_HTC = 19,
    XR_LIP_EXPRESSION_MOUTH_UPPER_UPLEFT_HTC = 20,
    XR_LIP_EXPRESSION_MOUTH_LOWER_DOWNRIGHT_HTC = 21,
    XR_LIP_EXPRESSION_MOUTH_LOWER_DOWNLEFT_HTC = 22,
    XR_LIP_EXPRESSION_MOUTH_UPPER_INSIDE_HTC = 23,
    XR_LIP_EXPRESSION_MOUTH_LOWER_INSIDE_HTC = 24,
    XR_LIP_EXPRESSION_MOUTH_LOWER_OVERLAY_HTC = 25,
    XR_LIP_EXPRESSION_TONGUE_LONGSTEP1_HTC = 26,
    XR_LIP_EXPRESSION_TONGUE_LEFT_HTC = 27,
    XR_LIP_EXPRESSION_TONGUE_RIGHT_HTC = 28,
    XR_LIP_EXPRESSION_TONGUE_UP_HTC = 29,
    XR_LIP_EXPRESSION_TONGUE_DOWN_HTC = 30,
    XR_LIP_EXPRESSION_TONGUE_ROLL_HTC = 31,
    XR_LIP_EXPRESSION_TONGUE_LONGSTEP2_HTC = 32,
    XR_LIP_EXPRESSION_TONGUE_UPRIGHT_MORPH_HTC = 33,
    XR_LIP_EXPRESSION_TONGUE_UPLEFT_MORPH_HTC = 34,
    XR_LIP_EXPRESSION_TONGUE_DOWNRIGHT_MORPH_HTC = 35,
    XR_LIP_EXPRESSION_TONGUE_DOWNLEFT_MORPH_HTC = 36,
  // Provided by XR_HTC_facial_tracking
    XR_LIP_EXPRESSION_MOUTH_SMILE_RIGHT_HTC = XR_LIP_EXPRESSION_MOUTH_RAISER_RIGHT_HTC,
  // Provided by XR_HTC_facial_tracking
    XR_LIP_EXPRESSION_MOUTH_SMILE_LEFT_HTC = XR_LIP_EXPRESSION_MOUTH_RAISER_LEFT_HTC,
  // Provided by XR_HTC_facial_tracking
```

```
    XR_LIP_EXPRESSION_MOUTH_SAD_RIGHT_HTC = XR_LIP_EXPRESSION_MOUTH_STRETCHER_RIGHT_HTC,
  // Provided by XR_HTC_facial_tracking
    XR_LIP_EXPRESSION_MOUTH_SAD_LEFT_HTC = XR_LIP_EXPRESSION_MOUTH_STRETCHER_LEFT_HTC,
    XR_LIP_EXPRESSION_MAX_ENUM_HTC = 0x7FFFFFFF
} XrLipExpressionHTC;
```

XR_LIP_EXPRESSION_JAW_LEFT_HTC	
	**Description** This blend shape moves the jaw further leftward with a higher value.

XR_LIP_EXPRESSION_JAW_RIGHT_HTC	
	**Description** This blend shape moves the jaw further rightward with a higher value.

XR_LIP_EXPRESSION_JAW_FORWARD_HTC	
	**Description** This blend shape moves the jaw forward with a higher value.

XR_LIP_EXPRESSION_JAW_OPEN_HTC	
	**Description** This blend shape opens the mouth further with a higher value.

XR_LIP_EXPRESSION_MOUTH_APE_SHAPE_HTC	
	**Description** This blend shape stretches the jaw further with a higher value.

XR_LIP_EXPRESSION_MOUTH_UPPER_LEFT_HTC	
	**Description** This blend shape moves your upper lip leftward.

XR_LIP_EXPRESSION_MOUTH_UPPER_RIGHT_HTC	
	**Description** This blend shape moves your upper lip rightward.

`XR_LIP_EXPRESSION_MOUTH_LOWER_LEFT_HTC`



Description
This blend shape moves your lower lip leftward.

`XR_LIP_EXPRESSION_MOUTH_LOWER_RIGHT_HTC`



Description
This blend shape moves your lower lip rightward.

`XR_LIP_EXPRESSION_MOUTH_UPPER_OVERTURN_HTC`



Description
This blend shape pouts your upper lip. Can be used with `XR_LIP_EXPRESSION_MOUTH_UPPER_UPRIGHT_HTC` and `XR_LIP_EXPRESSION_MOUTH_UPPER_UPLEFT_HTC` to complete upper O mouth shape.

`XR_LIP_EXPRESSION_MOUTH_LOWER_OVERTURN_HTC`



Description
This blend shape pouts your lower lip. Can be used with `XR_LIP_EXPRESSION_MOUTH_UPPER_UPRIGHT_HTC` and `XR_LIP_EXPRESSION_MOUTH_LOWER_DOWNRIGHT_HTC` to complete upper O mouth shape.

XR_LIP_EXPRESSION_MOUTH_POUT_HTC	
	**Description** This blend shape allows the lips to pout more with a higher value.

XR_LIP_EXPRESSION_MOUTH_RAISER_LEFT_HTC	
	**Description** This blend shape raises the left side of the mouth further with a higher value.

XR_LIP_EXPRESSION_MOUTH_RAISER_RIGHT_HTC	
	**Description** This blend shape raises the right side of the mouth further with a higher value.

XR_LIP_EXPRESSION_MOUTH_STRETCHER_LEFT_HTC	
	**Description** This blend shape lowers the left side of the mouth further with a higher value.

`XR_LIP_EXPRESSION_MOUTH_STRETCHER_RIGHT_HTC`



Description
This blend shape lowers the right side of the mouth further with a higher value.

`XR_LIP_EXPRESSION_CHEEK_PUFF_RIGHT_HTC`



Description
This blend shape puffs up the right side of the cheek further with a higher value.

`XR_LIP_EXPRESSION_CHEEK_PUFF_LEFT_HTC`



Description
This blend shape puffs up the left side of the cheek further with a higher value.

`XR_LIP_EXPRESSION_CHEEK_SUCK_HTC`



Description
This blend shape sucks in the cheeks on both sides further with a higher value.

`XR_LIP_EXPRESSION_MOUTH_UPPER_UPLEFT_HTC`



Description
This blend shape raises the left upper lip further with a higher value.

`XR_LIP_EXPRESSION_MOUTH_UPPER_UPRIGHT_HTC`



Description
This blend shape raises the right upper lip further with a higher value.

`XR_LIP_EXPRESSION_MOUTH_LOWER_DOWNLEFT_HTC`



Description
This blend shape lowers the left lower lip further with a higher value.

`XR_LIP_EXPRESSION_MOUTH_LOWER_DOWNRIGHT_HTC`



Description
This blend shape lowers the right lower lip further with a higher value.

`XR_LIP_EXPRESSION_MOUTH_LOWER_INSIDE_HTC`



Description
This blend shape rolls in the lower lip further with a higher value.

`XR_LIP_EXPRESSION_MOUTH_UPPER_INSIDE_HTC`



Description
This blend shape rolls in the upper lip further with a higher value.

`XR_LIP_EXPRESSION_MOUTH_LOWER_OVERLAY_HTC`



Description
This blend shape stretches the lower lip further and lays it on the upper lip further with a higher value.

`XR_LIP_EXPRESSION_TONGUE_LONGSTEP1_HTC`



Description
This blend shape sticks the tongue out slightly.
In step 1 of extending the tongue, the main action of the tongue is to lift up, and the elongated length only extends to a little bit beyond the teeth.

XR_LIP_EXPRESSION_TONGUE_LONGSTEP2_HTC	
	**Description** This blend shape sticks the tongue out extremely. Continuing the step 1, it extends the tongue to the longest.

XR_LIP_EXPRESSION_TONGUE_DOWN_HTC	
	**Description** This blend shape sticks the tongue out and down extremely. This example contains (XR_LIP_EXPRESSION_TONGUE_DOWN_HTC XR_LIP_EXPRESSION_TONGUE_LONGSTEP1_HTC + XR_LIP_EXPRESSION_TONGUE_LONGSTEP2_HTC).

XR_LIP_EXPRESSION_TONGUE_UP_HTC	
	**Description** This blend shape sticks the tongue out and up extremely. This example contains (XR_LIP_EXPRESSION_TONGUE_UP_HTC XR_LIP_EXPRESSION_TONGUE_LONGSTEP1_HTC + XR_LIP_EXPRESSION_TONGUE_LONGSTEP2_HTC).

XR_LIP_EXPRESSION_TONGUE_RIGHT_HTC	
	**Description** This blend shape sticks the tongue out and right extremely. This example contains (XR_LIP_EXPRESSION_TONGUE_RIGHT_HTC XR_LIP_EXPRESSION_TONGUE_LONGSTEP1_HTC + XR_LIP_EXPRESSION_TONGUE_LONGSTEP2_HTC).

XR_LIP_EXPRESSION_TONGUE_LEFT_HTC	
	**Description** This blend shape sticks the tongue out and left extremely. This example contains (XR_LIP_EXPRESSION_TONGUE_LEFT_HTC XR_LIP_EXPRESSION_TONGUE_LONGSTEP1_HTC + XR_LIP_EXPRESSION_TONGUE_LONGSTEP2_HTC).

XR_LIP_EXPRESSION_TONGUE_ROLL_HTC	
	**Description** This blend shape sticks the tongue out with roll type. This example contains (XR_LIP_EXPRESSION_TONGUE_ROLL_HTC XR_LIP_EXPRESSION_TONGUE_LONGSTEP1_HTC + XR_LIP_EXPRESSION_TONGUE_LONGSTEP2_HTC).

XR_LIP_EXPRESSION_TONGUE_UPRIGHT_MORPH_HTC	
	**Description** This blend shape does not make sense. When both the right and up blend shapes appear at the same time, the tongue will be deformed. (XR_LIP_EXPRESSION_TONGUE_RIGHT_HTC XR_LIP_EXPRESSION_TONGUE_UP_HTC + XR_LIP_EXPRESSION_TONGUE_LONGSTEP1_HTC XR_LIP_EXPRESSION_TONGUE_LONGSTEP2_HTC)

XR_LIP_EXPRESSION_TONGUE_UPRIGHT_MORPH_HTC	
	**Description** This blend shape fixes the deformation illustrated above. (XR_LIP_EXPRESSION_TONGUE_RIGHT_HTC XR_LIP_EXPRESSION_TONGUE_UP_HTC + XR_LIP_EXPRESSION_TONGUE_LONGSTEP1_HTC XR_LIP_EXPRESSION_TONGUE_LONGSTEP2_HTC + XR_LIP_EXPRESSION_TONGUE_UPRIGHT_MORPH_HTC)

XR_LIP_EXPRESSION_TONGUE_UPLEFT_MORPH_HTC	
	**Description** This blend shape does not make sense. When both the left and up blend shapes appear at the same time, the tongue will be deformed. (XR_LIP_EXPRESSION_TONGUE_LEFT_HTC XR_LIP_EXPRESSION_TONGUE_UP_HTC + XR_LIP_EXPRESSION_TONGUE_LONGSTEP1_HTC XR_LIP_EXPRESSION_TONGUE_LONGSTEP2_HTC)
	**Description** This blend shape fixes the deformation illustrated above. (XR_LIP_EXPRESSION_TONGUE_LEFT_HTC XR_LIP_EXPRESSION_TONGUE_UP_HTC + XR_LIP_EXPRESSION_TONGUE_LONGSTEP1_HTC XR_LIP_EXPRESSION_TONGUE_LONGSTEP2_HTC + XR_LIP_EXPRESSION_TONGUE_UPLEFT_MORPH_HTC)

XR_LIP_EXPRESSION_TONGUE_DOWNRIGHT_MORPH_HTC	
	**Description** This blend shape does not make sense. When both the right and down blend shapes appear at the same time, the tongue will be deformed. (XR_LIP_EXPRESSION_TONGUE_RIGHT_HTC XR_LIP_EXPRESSION_TONGUE_DOWN_HTC + XR_LIP_EXPRESSION_TONGUE_LONGSTEP1_HTC XR_LIP_EXPRESSION_TONGUE_LONGSTEP2_HTC)
	**Description** This blend shape fixes the deformation illustrated above. (XR_LIP_EXPRESSION_TONGUE_RIGHT_HTC XR_LIP_EXPRESSION_TONGUE_DOWN_HTC + XR_LIP_EXPRESSION_TONGUE_LONGSTEP1_HTC XR_LIP_EXPRESSION_TONGUE_LONGSTEP2_HTC + XR_LIP_EXPRESSION_TONGUE_DOWNRIGHT_MORPH_HTC)

XR_LIP_EXPRESSION_TONGUE_DOWNLEFT_MORPH_HTC	
	**Description** This blend shape does not make sense. When both the left and down blend shapes appear at the same time, the tongue will be deformed. (XR_LIP_EXPRESSION_TONGUE_LEFT_HTC XR_LIP_EXPRESSION_TONGUE_DOWN_HTC + XR_LIP_EXPRESSION_TONGUE_LONGSTEP1_HTC XR_LIP_EXPRESSION_TONGUE_LONGSTEP2_HTC)

XR_LIP_EXPRESSION_TONGUE_DOWNLEFT_MORPH_HTC	
	**Description** This blend shape fixes the deformation illustrated above. (XR_LIP_EXPRESSION_TONGUE_LEFT_HTC XR_LIP_EXPRESSION_TONGUE_DOWN_HTC + XR_LIP_EXPRESSION_TONGUE_LONGSTEP1_HTC XR_LIP_EXPRESSION_TONGUE_LONGSTEP2_HTC + XR_LIP_EXPRESSION_TONGUE_DOWNLEFT_MORPH_HTC)

O shape	
	**Description** The entire O-shaped mouth is formed by the combination of 6 blend shapes: (XR_LIP_EXPRESSION_MOUTH_UPPER_OVERTURN_HTC XR_LIP_EXPRESSION_MOUTH_LOWER_OVERTURN_HTC XR_LIP_EXPRESSION_MOUTH_UPPER_UPLEFT_HTC XR_LIP_EXPRESSION_MOUTH_UPPER_UPRIGHT_HTC XR_LIP_EXPRESSION_MOUTH_LOWER_DOWNLEFT_HTC XR_LIP_EXPRESSION_MOUTH_LOWER_DOWNRIGHT_HTC)

New Object Types

- XrFacialTrackerHTC

New Flag Types

New Enum Constants

XrObjectType enumeration is extended with:

- XR_OBJECT_TYPE_FACIAL_TRACKER_HTC

XrStructureType enumeration is extended with:

- XR_TYPE_SYSTEM_FACIAL_TRACKING_PROPERTIES_HTC

- XR_TYPE_FACIAL_TRACKER_CREATE_INFO_HTC
- XR_TYPE_FACIAL_EXPRESSIONS_HTC

New Enums

- XrFacialTrackingTypeHTC
- XrEyeExpressionHTC
- XrLipExpressionHTC

New Structures

- XrSystemFacialTrackingPropertiesHTC
- XrFacialTrackerCreateInfoHTC
- XrFacialExpressionsHTC

New Functions

- xrCreateFacialTrackerHTC
- xrDestroyFacialTrackerHTC
- xrGetFacialExpressionsHTC

Issues

Version History

- Revision 1, 2021-12-16 (Kyle Chen)
  - Initial extension description
- Revision 2, 2022-09-22 (Andy Chen)
  - Correct the range of the blink blend shapes.
- Revision 3, 2024-07-26 (Andy Chen)
  - Change expression naming convention: rename XR_LIP_EXPRESSION_MOUTH_SMILE_RIGHT_HTC to XR_LIP_EXPRESSION_MOUTH_RAISER_RIGHT_HTC, XR_LIP_EXPRESSION_MOUTH_SMILE_LEFT_HTC to XR_LIP_EXPRESSION_MOUTH_RAISER_LEFT_HTC, XR_LIP_EXPRESSION_MOUTH_SAD_RIGHT_HTC to XR_LIP_EXPRESSION_MOUTH_STRETCHER_RIGHT_HTC and XR_LIP_EXPRESSION_MOUTH_SAD_LEFT_HTC to XR_LIP_EXPRESSION_MOUTH_STRETCHER_LEFT_HTC, providing the old names as compatibility aliases.

12.136. XR_HTC_foveation

Name String

XR_HTC_foveation

Extension Type

Instance extension

Registered Extension Number

319

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Last Modified Date

2022-09-14

IP Status

No known IP claims.

Contributors

Billy Chang, HTC
Bill Chang, HTC

Overview

This extension enables an application to gain rendering performance improvement by reducing the pixel density of areas in the peripheral vision. The areas near the focal point still sustains the original pixel density than periphery.

The application **can** use this extension in the following steps:

1. Create an XrFoveationApplyInfoHTC structure with the desired foveation configurations.
2. Apply the foveation configuration by calling xrApplyFoveationHTC with desired XrFoveationApplyInfoHTC.



Note

This extension is recommended for XrSession whose XrViewConfigurationType is `XR_VIEW_CONFIGURATION_TYPE_PRIMARY_STEREO`.

Operate foveated rendering

The application **can** operate foveated rendering by calling xrApplyFoveationHTC with the corresponding foveation configuration and the specified XrSwapchainSubImage.

The xrApplyFoveationHTC function is defined as:

```
// Provided by XR_HTC_foveation
XrResult xrApplyFoveationHTC(
    XrSession                                   session,
    const XrFoveationApplyInfoHTC*              applyInfo);
```

Parameter Descriptions

- `session` is a handle to an XrSession in which the foveation will apply to.
- `applyInfo` is a pointer to an XrFoveationApplyInfoHTC structure containing information about the foveation configuration and applied XrSwapchainSubImage.

The foveation configuration will be applied after this call, and the state will persist until the next call to xrApplyFoveationHTC or the end of this XrSession, whichever comes first. You **should** not call xrApplyFoveationHTC during rendering to target image layer XrSwapchainSubImage in render loop.

Valid Usage (Implicit)

- The `XR_HTC_foveation` extension **must** be enabled prior to calling xrApplyFoveationHTC
- `session` **must** be a valid XrSession handle
- `applyInfo` **must** be a pointer to a valid XrFoveationApplyInfoHTC structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The XrFoveationApplyInfoHTC structure is defined as:

```
// Provided by XR_HTC_foveation
typedef struct XrFoveationApplyInfoHTC {
    XrStructureType         type;
    const void*             next;
    XrFoveationModeHTC      mode;
    uint32_t                subImageCount;
    XrSwapchainSubImage*    subImages;
} XrFoveationApplyInfoHTC;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain.
- mode is an XrFoveationModeHTC enum describing the foveation mode.
- subImageCount is the count of subImages in the subImages array. This **must** be equal to the number of view poses returned by xrLocateViews.
- subImages is an array of XrSwapchainSubImage to apply foveated rendering.

The application **should** set the following configurations in XrFoveationApplyInfoHTC:

- The foveation mode to be applied.

- The specified XrSwapchainSubImage to the corresponding view.

The XrSwapchain::faceCount of the swapchain in XrSwapchainSubImage **must** be 1 since this extension does not support cubemaps.

If mode is XR_FOVEATION_MODE_DYNAMIC_HTC, the next chain for this structure **must** include XrFoveationDynamicModeInfoHTC structure.

If mode is XR_FOVEATION_MODE_CUSTOM_HTC, the next chain for this structure **must** include XrFoveationCustomModeInfoHTC structure.

The order of subImages **must** be the same order as in XrCompositionLayerProjectionView when submitted in xrEndFrame.

Valid Usage (Implicit)

- The XR_HTC_foveation extension **must** be enabled prior to using XrFoveationApplyInfoHTC
- type **must** be XR_TYPE_FOVEATION_APPLY_INFO_HTC
- next **must** be NULL or a valid pointer to the next structure in a structure chain. See also: XrFoveationCustomModeInfoHTC, XrFoveationDynamicModeInfoHTC
- mode **must** be a valid XrFoveationModeHTC value
- subImages **must** be a pointer to an array of subImageCount XrSwapchainSubImage structures
- The subImageCount parameter **must** be greater than 0

XrFoveationModeHTC identifies the different foveation modes.

```
// Provided by XR_HTC_foveation
typedef enum XrFoveationModeHTC {
    XR_FOVEATION_MODE_DISABLE_HTC = 0,
    XR_FOVEATION_MODE_FIXED_HTC = 1,
    XR_FOVEATION_MODE_DYNAMIC_HTC = 2,
    XR_FOVEATION_MODE_CUSTOM_HTC = 3,
    XR_FOVEATION_MODE_MAX_ENUM_HTC = 0x7FFFFFFF
} XrFoveationModeHTC;
```

Enumerant Descriptions

- `XR_FOVEATION_MODE_DISABLE_HTC` — No foveation
- `XR_FOVEATION_MODE_FIXED_HTC` — Apply system default setting with fixed clear FOV and periphery quality.
- `XR_FOVEATION_MODE_DYNAMIC_HTC` — Allow system to set foveation dynamically according realtime system metric or other extensions.
- `XR_FOVEATION_MODE_CUSTOM_HTC` — Allow application to set foveation with desired clear FOV, periphery quality, and focal center offset.

Dynamic foveation mode

The application allows runtime to configure the foveation settings dynamically according to the system metrics or other extensions.

The XrFoveationDynamicModeInfoHTC structure is defined as:

```
// Provided by XR_HTC_foveation
typedef struct XrFoveationDynamicModeInfoHTC {
    XrStructureType               type;
    const void*                   next;
    XrFoveationDynamicFlagsHTC    dynamicFlags;
} XrFoveationDynamicModeInfoHTC;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `dynamicFlags` is a bitmask of XrFoveationDynamicFlagBitsHTC indicated which item **may** be changed during dynamic mode.

The application **must** chain an XrFoveationDynamicModeInfoHTC structure to XrFoveationApplyInfoHTC if dynamic mode is set.

Valid Usage (Implicit)

- The XR_HTC_foveation extension **must** be enabled prior to using XrFoveationDynamicModeInfoHTC
- type **must** be XR_TYPE_FOVEATION_DYNAMIC_MODE_INFO_HTC
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- dynamicFlags **must** be 0 or a valid combination of XrFoveationDynamicFlagBitsHTC values

```
// Provided by XR_HTC_foveation
typedef XrFlags64 XrFoveationDynamicFlagsHTC;
```

```
// Provided by XR_HTC_foveation
// Flag bits for XrFoveationDynamicFlagsHTC
static const XrFoveationDynamicFlagsHTC XR_FOVEATION_DYNAMIC_LEVEL_ENABLED_BIT_HTC =
0x00000001;
static const XrFoveationDynamicFlagsHTC XR_FOVEATION_DYNAMIC_CLEAR_FOV_ENABLED_BIT_HTC =
0x00000002;
static const XrFoveationDynamicFlagsHTC
XR_FOVEATION_DYNAMIC_FOCAL_CENTER_OFFSET_ENABLED_BIT_HTC = 0x00000004;
```

Flag Descriptions

- XR_FOVEATION_DYNAMIC_LEVEL_ENABLED_BIT_HTC — Allow system to set periphery pixel density dynamically.
- XR_FOVEATION_DYNAMIC_CLEAR_FOV_ENABLED_BIT_HTC — Allow system to set clear FOV degree dynamically.
- XR_FOVEATION_DYNAMIC_FOCAL_CENTER_OFFSET_ENABLED_BIT_HTC — Allow system to set focal center offset dynamically.

Custom foveation mode

The application **can** configure the foveation settings according to the preference of content.

The XrFoveationCustomModeInfoHTC structure is defined as:

```
// Provided by XR_HTC_foveation
typedef struct XrFoveationCustomModeInfoHTC {
    XrStructureType                         type;
    const void*                             next;
    uint32_t                                configCount;
    const XrFoveationConfigurationHTC*      configs;
} XrFoveationCustomModeInfoHTC;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `configCount` is a `uint32_t` describing the count of elements in the `configs` array, which **must** be the number of views.
- `configs` is an array of XrFoveationConfigurationHTC structure contains the custom foveation settings for the corresponding views.

The application **must** chain an XrFoveationCustomModeInfoHTC structure to XrFoveationApplyInfoHTC to customize foveation if custom mode is set.

Valid Usage (Implicit)

- The `XR_HTC_foveation` extension **must** be enabled prior to using XrFoveationCustomModeInfoHTC
- `type` **must** be `XR_TYPE_FOVEATION_CUSTOM_MODE_INFO_HTC`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `configs` **must** be a pointer to an array of `configCount` valid XrFoveationConfigurationHTC structures
- The `configCount` parameter **must** be greater than `0`

The XrFoveationConfigurationHTC structure is defined as:

```
// Provided by XR_HTC_foveation
typedef struct XrFoveationConfigurationHTC {
    XrFoveationLevelHTC    level;
    float                  clearFovDegree;
    XrVector2f             focalCenterOffset;
} XrFoveationConfigurationHTC;
```

Member Descriptions

- `level` is the pixel density drop level of periphery area specified by XrFoveationLevelHTC .
- `clearFovDegree` is the value indicating the total horizontal and vertical field angle with the original pixel density level. `clearFovDegree` **must** be specified in degree, and **must** be in the range [0, 180].
- `focalCenterOffset` is the desired center offset of the field of view in NDC(normalized device coordinates) space. The x and y of `focalCenterOffset` **must** be in the range [-1, 1].

Valid Usage (Implicit)

- The `XR_HTC_foveation` extension **must** be enabled prior to using XrFoveationConfigurationHTC
- `level` **must** be a valid XrFoveationLevelHTC value

```
// Provided by XR_HTC_foveation
typedef enum XrFoveationLevelHTC {
    XR_FOVEATION_LEVEL_NONE_HTC = 0,
    XR_FOVEATION_LEVEL_LOW_HTC = 1,
    XR_FOVEATION_LEVEL_MEDIUM_HTC = 2,
    XR_FOVEATION_LEVEL_HIGH_HTC = 3,
    XR_FOVEATION_LEVEL_MAX_ENUM_HTC = 0x7FFFFFFF
} XrFoveationLevelHTC;
```

Enumerant Descriptions

- `XR_FOVEATION_LEVEL_NONE_HTC` — No foveation
- `XR_FOVEATION_LEVEL_LOW_HTC` — Light periphery pixel density drop and lower performance gain.
- `XR_FOVEATION_LEVEL_MEDIUM_HTC` — Medium periphery pixel density drop and medium performance gain
- `XR_FOVEATION_LEVEL_HIGH_HTC` — Heavy periphery pixel density drop and higher performance gain

New Object Types

New Flag Types

XrFoveationDynamicFlagsHTC

New Enum Constants

XrStructureType enumeration is extended with:

- `XR_TYPE_FOVEATION_APPLY_INFO_HTC`
- `XR_TYPE_FOVEATION_DYNAMIC_MODE_INFO_HTC`
- `XR_TYPE_FOVEATION_CUSTOM_MODE_INFO_HTC`

New Enum Constants

New Enums

XrFoveationModeHTC

XrFoveationDynamicFlagBitsHTC

XrFoveationLevelHTC

New Structures

XrFoveationApplyInfoHTC

XrFoveationDynamicModeInfoHTC

XrFoveationCustomModeInfoHTC

New Functions

xrApplyFoveationHTC

Issues

Version History

- Revision 1, 2022-09-14 (Billy Chang)
  - Initial extension description

12.137. XR_HTC_hand_interaction

Name String

`XR_HTC_hand_interaction`

Extension Type

Instance extension

Registered Extension Number

107

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

API Interactions

- Interacts with `XR_EXT_hand_interaction`
- Interacts with `XR_EXT_palm_pose`

Last Modified Date

2022-05-27

IP Status

No known IP claims.

Contributors

Ria Hsu, HTC
Bill Chang, HTC

Overview

This extension defines a new interaction profile for tracked hands.

Hand interaction profile

Interaction profile path:

- */interaction_profiles/htc/hand_interaction*

> **Note**
>
> The interaction profile path */interaction_profiles/htc/hand_interaction* defined here does not follow current rules for interaction profile names. If this extension were introduced today, it would be called */interaction_profiles/htc/hand_interaction_htc*, to allow for modifications when promoted to a KHR extension or the core specification.

Valid for user paths:

- */user/hand_htc/left*
- */user/hand_htc/right*

This interaction profile represents basic pose and actions for interaction of tracked hands.

Supported component paths for far interaction:

- *.../input/select/value*
- *.../input/aim/pose*

The application **should** use *.../input/aim/pose* path to aim at objects in the world and use *.../input/select/value* path to decide user selection from pinch shape strength which the range of value is `0.0f` to `1.0f`, with `1.0f` meaning pinch fingers touched.

Supported component paths for near interaction:

- *.../input/squeeze/value*
- *.../input/grip/pose*

The application **should** use *.../input/grip/pose* path to interact with the nearby objects and locate the position of handheld objects, and use *.../input/squeeze/value* path to decide the hand picking up or holding the nearby objects from grip shape strength which the range of value is `0.0f` to `1.0f`, with `1.0f` meaning hand grip shape is closed.



Note

Far and near interaction depends on the support capabilities of hand tracking engine. The application **can** check isActive of XrActionStatePose of aim and grip to know far and near interaction supported or not then decide the interaction behavior in content.

Version History

- Revision 1, 2022-05-27 (Ria Hsu)
  - Initial extension description

12.138. XR_HTC_passthrough

Name String

`XR_HTC_passthrough`

Extension Type

Instance extension

Registered Extension Number

318

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Last Modified Date

2022-09-14

IP Status

No known IP claims.

Contributors

Livi Lin, HTC
Sacdar Hsu, HTC
Bill Chang, HTC

Overview

This extension enables an application to show the passthrough image to see the surrounding environment from the VR headset. The application is allowed to configure the passthrough image with the different appearances according to the demand of the application.

The passthrough configurations that runtime provides to applications contain:

- Decide the passthrough layer shown over or under the frame submitted by the application.
- Specify the passthrough form with full of the entire screen or projection onto the mesh specified by the application.
- Set the alpha blending level for the composition of the passthrough layer.

Create a passthrough handle

An application **can** create an XrPassthroughHTC handle by calling xrCreatePassthroughHTC. The returned passthrough handle **can** be subsequently used in API calls.

```
// Provided by XR_HTC_passthrough
XR_DEFINE_HANDLE(XrPassthroughHTC)
```

The xrCreatePassthroughHTC function is defined as:

```
// Provided by XR_HTC_passthrough
XrResult xrCreatePassthroughHTC(
    XrSession                                   session,
    const XrPassthroughCreateInfoHTC*           createInfo,
    XrPassthroughHTC*                           passthrough);
```

Parameter Descriptions

- `session` is an XrSession in which the passthrough will be active.
- `createInfo` is a pointer to an XrPassthroughCreateInfoHTC structure containing information about how to create the passthrough.
- `passthrough` is a pointer to a handle in which the created XrPassthroughHTC is returned.

Creates an XrPassthroughHTC handle.

If the function successfully returned, the output `passthrough` **must** be a valid handle.

Valid Usage (Implicit)

- The `XR_HTC_passthrough` extension **must** be enabled prior to calling xrCreatePassthroughHTC
- `session` **must** be a valid XrSession handle
- `createInfo` **must** be a pointer to a valid XrPassthroughCreateInfoHTC structure
- `passthrough` **must** be a pointer to an XrPassthroughHTC handle

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FEATURE_UNSUPPORTED
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The XrPassthroughCreateInfoHTC structure is defined as:

```
// Provided by XR_HTC_passthrough
typedef struct XrPassthroughCreateInfoHTC {
    XrStructureType         type;
    const void*             next;
    XrPassthroughFormHTC    form;
} XrPassthroughCreateInfoHTC;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- form XrPassthroughFormHTC that specifies the form of passthrough.

Valid Usage (Implicit)

- The XR_HTC_passthrough extension **must** be enabled prior to using XrPassthroughCreateInfoHTC
- `type` **must** be `XR_TYPE_PASSTHROUGH_CREATE_INFO_HTC`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `form` **must** be a valid XrPassthroughFormHTC value

The XrPassthroughFormHTC enumeration identifies the form of the passthrough, presenting the passthrough fill the full screen or project onto a specified mesh.

```
// Provided by XR_HTC_passthrough
typedef enum XrPassthroughFormHTC {
    XR_PASSTHROUGH_FORM_PLANAR_HTC = 0,
    XR_PASSTHROUGH_FORM_PROJECTED_HTC = 1,
    XR_PASSTHROUGH_FORM_MAX_ENUM_HTC = 0x7FFFFFFF
} XrPassthroughFormHTC;
```

Enumerant Descriptions

- `XR_PASSTHROUGH_FORM_PLANAR_HTC` — Presents the passthrough with full of the entire screen.
- `XR_PASSTHROUGH_FORM_PROJECTED_HTC` — Presents the passthrough projecting onto a custom mesh.

The xrDestroyPassthroughHTC function is defined as:

```
// Provided by XR_HTC_passthrough
XrResult xrDestroyPassthroughHTC(
    XrPassthroughHTC                            passthrough);
```

Parameter Descriptions

- `passthrough` is the XrPassthroughHTC to be destroyed.

The xrDestroyPassthroughHTC function releases the passthrough and the underlying resources.

Valid Usage (Implicit)

- The XR_HTC_passthrough extension **must** be enabled prior to calling xrDestroyPassthroughHTC
- passthrough **must** be a valid XrPassthroughHTC handle

Thread Safety

- Access to passthrough, and any child handles, **must** be externally synchronized

Return Codes

Success

- XR_SUCCESS

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_RUNTIME_FAILURE

Composite the passthrough layer

The XrCompositionLayerPassthroughHTC structure is defined as:

```
// Provided by XR_HTC_passthrough
typedef struct XrCompositionLayerPassthroughHTC {
    XrStructureType            type;
    const void*                next;
    XrCompositionLayerFlags    layerFlags;
    XrSpace                    space;
    XrPassthroughHTC           passthrough;
    XrPassthroughColorHTC      color;
} XrCompositionLayerPassthroughHTC;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain, such as XrPassthroughMeshTransformInfoHTC.
- `layerFlags` is a bitmask of XrCompositionLayerFlagBits describing flags to apply to the layer.
- `space` is the XrSpace that specifies the layer's space - **must** be XR_NULL_HANDLE.
- `passthrough` is the XrPassthroughHTC previously created by xrCreatePassthroughHTC.
- `color` is the XrPassthroughColorHTC describing the color information with the alpha value of the passthrough layer.

The application **can** create an XrCompositionLayerPassthroughHTC structure with the created `passthrough` and the corresponding information. A pointer to XrCompositionLayerPassthroughHTC **may** be submitted in xrEndFrame as a pointer to the base structure XrCompositionLayerBaseHeader, in the desired layer order, to request the runtime to composite a passthrough layer into the final frame output.

If the passthrough form specified to xrCreatePassthroughHTC is `XR_PASSTHROUGH_FORM_PROJECTED_HTC`, XrPassthroughMeshTransformInfoHTC **must** appear in the `next` chain. If they are absent, the runtime **must** return error `XR_ERROR_VALIDATION_FAILURE`.

Valid Usage (Implicit)

- The `XR_HTC_passthrough` extension **must** be enabled prior to using XrCompositionLayerPassthroughHTC
- `type` **must** be `XR_TYPE_COMPOSITION_LAYER_PASSTHROUGH_HTC`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain. See also: XrPassthroughMeshTransformInfoHTC
- `layerFlags` **must** be a valid combination of XrCompositionLayerFlagBits values
- `layerFlags` **must** not be `0`
- `space` **must** be a valid XrSpace handle
- `passthrough` **must** be a valid XrPassthroughHTC handle
- `color` **must** be a valid XrPassthroughColorHTC structure
- Both of `passthrough` and `space` **must** have been created, allocated, or retrieved from the same XrSession

The XrPassthroughColorHTC structure is defined as:

```
// Provided by XR_HTC_passthrough
typedef struct XrPassthroughColorHTC {
    XrStructureType    type;
    const void*        next;
    float              alpha;
} XrPassthroughColorHTC;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `alpha` is the alpha value of the passthrough in the range [0, 1].

The application **can** specify the XrPassthroughColorHTC to adjust the alpha value of the passthrough. The range is between 0.0f and 1.0f, 1.0f means opaque.

Valid Usage (Implicit)

- The `XR_HTC_passthrough` extension **must** be enabled prior to using XrPassthroughColorHTC
- `type` **must** be `XR_TYPE_PASSTHROUGH_COLOR_HTC`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

The XrPassthroughMeshTransformInfoHTC structure is defined as:

```
// Provided by XR_HTC_passthrough
typedef struct XrPassthroughMeshTransformInfoHTC {
    XrStructureType      type;
    const void*          next;
    uint32_t             vertexCount;
    const XrVector3f*    vertices;
    uint32_t             indexCount;
    const uint32_t*      indices;
    XrSpace              baseSpace;
    XrTime               time;
    XrPosef              pose;
    XrVector3f           scale;
} XrPassthroughMeshTransformInfoHTC;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `vertexCount` is the count of vertices array in the mesh.
- `vertices` is an array of XrVector3f. The size of the array **must** be equal to vertexCount.
- `indexCount` is the count of indices array in the mesh.
- `indices` is an array of triangle indices. The size of the array **must** be equal to indexCount.
- `baseSpace` is the XrSpace that defines the projected passthrough's base space for transformations.
- `time` is the `XrTime` that defines the time at which the transform is applied.
- `pose` is the XrPosef that defines the pose of the mesh
- `scale` is the XrVector3f that defines the scale of the mesh

The XrPassthroughMeshTransformInfoHTC structure describes the mesh and transformation.

The application **must** specify the XrPassthroughMeshTransformInfoHTC in the `next` chain of XrCompositionLayerPassthroughHTC if the specified form of passthrough layer previously created by xrCreatePassthroughHTC is `XR_PASSTHROUGH_FORM_PROJECTED_HTC`.

Passing XrPassthroughMeshTransformInfoHTC updates the projected mesh information in the runtime for passthrough layer composition.

If XrPassthroughMeshTransformInfoHTC is not set correctly, runtime **must** return error `XR_ERROR_VALIDATION_FAILURE` when xrEndFrame is called with composition layer XrCompositionLayerPassthroughHTC.

Valid Usage (Implicit)

- The XR_HTC_passthrough extension **must** be enabled prior to using XrPassthroughMeshTransformInfoHTC
- type **must** be XR_TYPE_PASSTHROUGH_MESH_TRANSFORM_INFO_HTC
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- vertices **must** be a pointer to an array of vertexCount XrVector3f structures
- indices **must** be a pointer to an array of indexCount uint32_t values
- baseSpace **must** be a valid XrSpace handle
- The vertexCount parameter **must** be greater than 0
- The indexCount parameter **must** be greater than 0

New Object Types

- XrPassthroughHTC

New Flag Types

New Enum Constants

XrObjectType enumeration is extended with:

- XR_OBJECT_TYPE_PASSTHROUGH_HTC

XrStructureType enumeration is extended with:

- XR_TYPE_PASSTHROUGH_CREATE_INFO_HTC
- XR_TYPE_PASSTHROUGH_COLOR_HTC
- XR_TYPE_PASSTHROUGH_MESH_TRANSFORM_INFO_HTC
- XR_TYPE_COMPOSITION_LAYER_PASSTHROUGH_HTC

New Enums

- XrPassthroughFormHTC

New Structures

- XrPassthroughCreateInfoHTC
- XrPassthroughColorHTC
- XrPassthroughMeshTransformInfoHTC
- XrCompositionLayerPassthroughHTC

New Functions

- xrCreatePassthroughHTC
- xrDestroyPassthroughHTC

Issues

Version History

- Revision 1, 2022-09-14 (Sacdar Hsu)
  - Initial extension description

12.139. XR_HTC_vive_wrist_tracker_interaction

Name String

`XR_HTC_vive_wrist_tracker_interaction`

Extension Type

Instance extension

Registered Extension Number

108

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Last Modified Date

2022-05-27

IP Status

No known IP claims.

Contributors

Ria Hsu, HTC
Bill Chang, HTC

Overview

This extension provides an `XrPath` for getting device input from a VIVE wrist tracker to enable its

interactions. VIVE wrist tracker is a tracked device mainly worn on user's wrist for pose tracking. Besides this use case, user also can tie it to a physical object to track its object pose, e.g. tie on a gun.

VIVE Wrist Tracker input

This extension exposes a new interaction profile path */interaction_profiles/htc/vive_wrist_tracker* that is valid for the user path

- */user/wrist_htc/left*
- */user/wrist_htc/right*

with supported input subpaths

- On */user/wrist_htc/left* only:
  - *.../input/menu/click*
  - *.../input/x/click*
- On */user/wrist_htc/right* only:
  - *.../input/system/click* (**may** not be available for application use)
  - *.../input/a/click*
- *.../input/entity_htc/pose*

> **Note**
>
> The interaction profile path */interaction_profiles/htc/vive_wrist_tracker* defined here does not follow current rules for interaction profile names. If this extension were introduced today, it would be called */interaction_profiles/htc/vive_wrist_tracker_htc*, to allow for modifications when promoted to a KHR extension or the core specification.

The entity_htc pose allows the applications to recognize the origin of a tracked input device, especially for the wearable devices which are not held in the user's hand. The entity_htc pose is defined as follows:

- The entity position: The center position of the tracked device.
- The entity orientation: Oriented with +Y up, +X to the right, and -Z forward.

Version History

- Revision 1, 2022-05-27 (Ria Hsu)
  - Initial extension description

12.140. XR_HUAWEI_controller_interaction

Name String

`XR_HUAWEI_controller_interaction`

Extension Type

Instance extension

Registered Extension Number

70

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

API Interactions

- Interacts with `XR_EXT_dpad_binding`
- Interacts with `XR_EXT_hand_interaction`
- Interacts with `XR_EXT_palm_pose`

Last Modified Date

2020-05-26

IP Status

No known IP claims.

Contributors

Guodong Chen, Huawei
Kai Shao, Huawei
Yang Tao, Huawei
Gang Shen, Huawei
Yihong Huang, Huawei

Overview

This extension defines a new interaction profile for the Huawei Controller, including but not limited to Huawei VR Glasses Controller.

Huawei Controller interaction profile

Interaction profile path:

- */interaction_profiles/huawei/controller*

> **Note**
>
> The interaction profile path */interaction_profiles/huawei/controller* defined here does not follow current rules for interaction profile names. If this extension were introduced today, it would be called */interaction_profiles/huawei/controller_huawei*, to allow for modifications when promoted to a KHR extension or the core specification.

Valid for user paths:

- */user/hand/left*
- */user/hand/right*

This interaction profile represents the input sources and haptics on the Huawei Controller.

Supported component paths:

- *.../input/home/click*
- *.../input/back/click*
- *.../input/volume_up/click*
- *.../input/volume_down/click*
- *.../input/trigger/value*
- *.../input/trigger/click*
- *.../input/trackpad/x*
- *.../input/trackpad/y*
- *.../input/trackpad/click*
- *.../input/trackpad/touch*
- *.../input/aim/pose*
- *.../input/grip/pose*
- *.../output/haptic*



Note

When the runtime supports `XR_VERSION_1_1` and use of OpenXR 1.1 is requested by the application, this interaction profile **must** also support

- *.../input/grip_surface/pose*



Note

When the `XR_KHR_maintenance1` extension is available and enabled, this interaction profile **must** also support

- *.../input/grip_surface/pose*



Note

When the `XR_EXT_palm_pose` extension is available and enabled, this interaction profile **must** also support

- *.../input/palm_ext/pose*



Note

When the `XR_EXT_hand_interaction` extension is available and enabled, this interaction profile **must** also support

- *.../input/pinch_ext/pose*
- *.../input/poke_ext/pose*

New Object Types

New Flag Types

New Enum Constants

New Enums

New Structures

New Functions

Issues

Version History

- Revision 1, 2020-04-28 (Yihong Huang)
  - Initial extension description

12.141. XR_LOGITECH_mx_ink_stylus_interaction

Name String

`XR_LOGITECH_mx_ink_stylus_interaction`

Extension Type

Instance extension

Registered Extension Number

746

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

API Interactions

- Interacts with `XR_EXT_hand_interaction`
- Interacts with `XR_EXT_palm_pose`

Last Modified Date

2024-06-02

IP Status

No known IP claims.

Contributors

Mario Gutierrez, Logitech
Aidan Kehoe, Logitech
Fabien Zellweger, Logitech

Overview

This extension defines a new interaction profile for the Logitech MX Ink, a 6-DOF tracked stylus.

12.141.1. Logitech MX Ink Stylus Interaction Profile

Interaction profile path:

- */interaction_profiles/logitech/mx_ink_stylus_logitech*

Valid for user paths:

- */user/hand/left*
- */user/hand/right*

This interaction profile provides inputs and outputs for a 6-DOF tracked controller shaped as a stylus. This controller allows for writing/sketching on 2D surfaces using an analog force sensor at the tip. In addition to the tip, the controller has an input cluster composed of three buttons. The middle button features a force sensor. The front and back buttons provide two different inputs: click and double tap detection. Double tap events are treated as virtual buttons. The stylus also includes a system button, situated at the rear end of the device.

Supported component paths:

- *.../input/system/click* (**may** not be available for application use)
- *.../input/grip/pose*
- *.../input/aim/pose*
- *.../input/cluster_front_logitech/click*
- *.../input/cluster_front_logitech/double_tap_logitech*
- *.../input/cluster_middle_logitech/force*
- *.../input/cluster_back_logitech/click*
- *.../input/cluster_back_logitech/double_tap_logitech*
- *.../input/dock_logitech/docked_logitech*
- *.../input/tip_logitech/force*
- *.../input/tip_logitech/pose*
- *.../output/haptic*

12.141.2. New Identifiers and Components

New Identifiers

- **tip_logitech**: The Logitech MX Ink Stylus features a pressure-sensitive tip. Its usage is intended to be primarily drawing/writing applications, along with other forms of close interaction. This pose corresponds to the position and orientation of the stylus **tip**. The origin of the local coordinate system is the sharpest point of the **tip**. The Z axis aligns with the main axis of the stylus, with -Z extending outward from the tip. The X axis points to the right, while the direction of the Y axis aligns with the surface normal of the input cluster.
- **cluster_front_logitech**: This is the front button (closest to the stylus tip) in the input cluster of the controller. Its usage is intended to be similar to *.../input/squeeze/click* on other controllers.
- **cluster_middle_logitech**: The middle button in the input cluster measures pressure/force levels. Its usage is intended to be controlling in-air drawing (together with "tip_logitech"), selection, and generic interactions, similar to the trigger in other XR controllers.
- **cluster_back_logitech**: This is the third button in the device input cluster, furthest from the tip. Usage of the click interaction with this button is intended to be similar to the A button on other controllers. Usage of the double tap input is intended to be similar to the B button on other controllers.
- **dock_logitech**: The Logitech MX Ink Stylus is associated with a docking station for battery charging when not in use. It is expected that users will frequently switch between the MX Ink stylus and hand tracking or other controller.

New Components

- **docked_logitech**: Boolean input component. This component becomes true when placed in the dock, immediately before the device becomes inactive and stops being tracked. A recommended usage pattern is to monitor this component to become true, then locate the device at the associated "last change" timestamp. This allows retrieving the last known position for display purposes, making it easier for the user to pick up again when needed.
- **double_tap_logitech**: Boolean input component. Double tap gesture detection intended for usage as a virtual button. The three buttons in the input cluster have capacitive sensors. However, to reduce complexity and standardize the UX, 'single' tap gestures are not exposed in this interaction profile. A tap event is when the user makes brief contact with the button surface and lifts their finger within a predefined time window. A double tap consists of two single taps performed in succession, where the time interval between the two taps does not exceed a maximum threshold. Time intervals are user-defined and managed at the platform level. The goal is to unify the UX and free developers from having to implement their own double tap detection algorithm.

Types of Holds

The device associated with this interaction profile is a stylus, and thus affords two basic ways to hold and use it. The device does **not** sense which hold is in use, but some pose input subpaths only make sense in the context of one of the two holds. Application developers **should** consider which hold is expected in a given setting when designing suggested bindings.

A "precise grip" is a pencil-like hold allowing precision control using the arm, wrist, hand, and fingers. A precise grip is the primary hold encouraged by the affordances of the device design. It provides fine control over close interaction. While the stylus shape allows several orientations of this hold about the major axis of the device, the design intent is for the control cluster buttons to be accessible to the index finger and/or thumb while maintaining a precise grip.

A "power grip", by contrast, is a closed-fist hold, providing more coarse control using the arm and wrist. The control cluster buttons **may** be inaccessible when a user is employing a power grip. This is a secondary method of using the device. Most other XR controllers with interaction profiles in this specification are designed primarily for a power grip, and the "grip" standard pose identifier is intended to be a power grip in all of those profiles.

- *.../input/tip_logitech/pose* is intended for close selection, interaction, and drawing. This pose input subpath is referenced to the physical hardware rather than an assumed or designed hold. Therefore, its pose is meaningful in **both precise grip and power grip**. However, it also serves as the primary interaction pose for close-range use in the precise grip.
- *.../input/aim/pose* is intended for selection and interaction at a distance according to platform conventions. When used with this interaction profile, the position and orientation of this subpath relative to the device assumes a **precise grip**. This pose input subpath is not necessarily suitable for use with a power grip.
- *.../input/grip/pose* in this interaction profile is for close, coarse interaction while using a **power grip**, or virtual tool/object holding in a power grip. The relative pose of this subpath is based on the design intent for holding in a power grip. Be aware that due to the shape of the device, a user's actual power grip **may** easily vary from the design intent by a rotation about the Z axis, as well as translation along it. As such, inferring hand position and orientation from this pose is not recommended. Discouraged from use when a precise grip is anticipated, as this pose has very little to no meaningful relationship to the device or the user's hand in a precise grip.
- *.../input/grip_surface/pose* (when available — see `XR_KHR_maintenance1` or `XR_VERSION_1_1`) in this interaction profile is established assuming a **power grip**, holding the stylus against the palm in a closed fist. The predecessor of this input subpath, *.../input/palm_ext/pose*, is identically established, when available (see `XR_EXT_palm_pose`). The descriptions of positions and axes in the "grip_surface" standard pose identifier definition are **not** valid when the device is held in a precise grip rather than a power grip.
- *.../input/poke_ext/pose* and *.../input/pinch_ext/pose* (when available — see `XR_EXT_hand_interaction`) in this interaction profile are established assuming a **precise grip** interacting using the stylus tip. Discouraged from use when a power grip is anticipated.
    - NB: The device associated with this interaction profile also supports use through the

/interaction_profile/ext/hand_interaction interaction profile provided by `XR_EXT_hand_interaction`.

Designing Suggesting Bindings

The table below provides recommended translations for suggested interaction bindings from other controllers to this interaction profile, serving as a starting point for development.

As this controller is ambidextrous, the runtime **should** provide a platform settings method to select the stylus handedness and map the input paths for the A/B and X/Y buttons according to the current handedness.

The runtime **may** provide a platform settings method to define on which button the double tap events will be detected, and which virtual button it is mapped to. It is recommended to suggest binding a single action to both double tap subpaths for consistent behavior regardless of user settings.

Input subpath on common controller designs	**Recommended input subpath to suggest when using with** */interaction_profiles/logitech/mx_ink_stylus_logitech*
.../input/squeeze/click	*.../input/cluster_front_logitech/click*
.../input/trigger/value	*.../input/cluster_middle_logitech/force*
.../input/a/click	*.../input/cluster_back_logitech/click*
.../input/b/click	*.../input/cluster_back_logitech/double_tap_logitech* or *.../input/cluster_front_logitech/double_tap_logitech*
.../input/x/click	*.../input/cluster_back_logitech/click*
.../input/y/click	*.../input/cluster_back_logitech/double_tap_logitech* or *.../input/cluster_front_logitech/double_tap_logitech*

12.141.3. New Enum Constants

- `XR_LOGITECH_MX_INK_STYLUS_INTERACTION_EXTENSION_NAME`
- `XR_LOGITECH_mx_ink_stylus_interaction_SPEC_VERSION`

12.141.4. Version History

- Revision 1, 2024-06-02 (Mario Gutierrez, Aidan Kehoe)
  - Initial extension description

12.142. XR_META_automatic_layer_filter

Name String

XR_META_automatic_layer_filter

Extension Type

Instance extension

Registered Extension Number

272

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0
and
XR_FB_composition_layer_settings

Contributors

Rohit Rao Padebettu, Meta
Grant Yang, Meta

Overview

This extension defines a new flag in XrCompositionLayerSettingsFlagBitsFB that allows applications to provide a hint to the runtime to automatically toggle a layer filtering mechanism. The layer filtering helps alleviate visual quality artifacts such as blur and flicker.

Note: The runtime **may** use any factors it wishes to apply a filter to the layer. These **may** include not only fixed factors such as screen resolution, HMD type, and swapchain resolution, but also dynamic ones such as layer pose and system-wide GPU utilization.

Automatic Layer Filtering

XrCompositionLayerSettingsFlagBitsFB is extended with XR_COMPOSITION_LAYER_SETTINGS_AUTO_LAYER_FILTER_BIT_META

To enable automatic selection of layer filtering method, XR_COMPOSITION_LAYER_SETTINGS_AUTO_LAYER_FILTER_BIT_META is passed to the runtime in XrCompositionLayerSettingsFB::layerFlags.

A candidate pool of preferred layer filtering methods from XrCompositionLayerSettingsFlagBitsFB

must be passed along with XR_COMPOSITION_LAYER_SETTINGS_AUTO_LAYER_FILTER_BIT_META. The runtime **may** apply the appropriate filter when rendering the layer. The runtime **must** return XR_ERROR_VALIDATION_FAILURE from xrEndFrame when an XrCompositionLayerSettingsFB structure is submitted with one or more of the layers if no other flag bits are supplied with XR_COMPOSITION_LAYER_SETTINGS_AUTO_LAYER_FILTER_BIT_META.

Version History

- Revision 1, 2023-04-21 (Rohit Rao Padebettu)
  - Initial extension description

12.143. XR_META_body_tracking_calibration

Name String

XR_META_body_tracking_calibration

Extension Type

Instance extension

Registered Extension Number

284

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

XR_FB_body_tracking

Last Modified Date

2025-07-23

IP Status

No known IP claims.

Contributors

Giancarlo Di Biase, Meta
Dikpal Reddy, Meta
Igor Tceglevskii, Meta
Bill Orr, Meta
Alexey Sidnev, Meta
Alexandra Movsesyan, Meta

12.143.1. Overview

This extension enables applications utilizing XR_FB_body_tracking to override automatic calibration, and to determine the current calibration status for automatic calibration.

Applications **can** call the xrSuggestBodyTrackingCalibrationOverrideMETA function to override calibration on request, for example, when the user indicates it is desired through an in-app UI.

Typical times to calibrate are during app startup (when body tracking is first initialized), on user request, when calibration is detected to be low quality, and on user switch.

Manual calibration is validated, with applications providing accurate values to the runtime.

12.143.2. Inspect system capability

The XrSystemPropertiesBodyTrackingCalibrationMETA structure is defined as:

```
// Provided by XR_META_body_tracking_calibration
typedef struct XrSystemPropertiesBodyTrackingCalibrationMETA {
    XrStructureType    type;
    void*              next;
    XrBool32           supportsHeightOverride;
} XrSystemPropertiesBodyTrackingCalibrationMETA;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain.
- supportsHeightOverride is an XrBool32, indicating if the current system is capable of supporting body tracking calibration height override.

An application **can** inspect whether the system supports body tracking and body calibration by extending the XrSystemProperties with XrSystemPropertiesBodyTrackingCalibrationMETA structure when calling xrGetSystemProperties.

If a runtime returns XR_FALSE for supportsHeightOverride, the runtime **must** return XR_ERROR_FEATURE_UNSUPPORTED from xrSuggestBodyTrackingCalibrationOverrideMETA and from xrResetBodyTrackingCalibrationMETA.

Valid Usage (Implicit)

- The XR_META_body_tracking_calibration extension **must** be enabled prior to using XrSystemPropertiesBodyTrackingCalibrationMETA
- type **must** be XR_TYPE_SYSTEM_PROPERTIES_BODY_TRACKING_CALIBRATION_META
- next **must** be NULL or a valid pointer to the next structure in a structure chain

12.143.3. Overriding the user height

The xrSuggestBodyTrackingCalibrationOverrideMETA function is defined as:

```
// Provided by XR_META_body_tracking_calibration
XrResult xrSuggestBodyTrackingCalibrationOverrideMETA(
    XrBodyTrackerFB                             bodyTracker,
    const XrBodyTrackingCalibrationInfoMETA*    calibrationInfo);
```

Parameter Descriptions

- bodyTracker is an XrBodyTrackerFB previously created by xrCreateBodyTrackerFB.
- calibrationInfo is an XrBodyTrackingCalibrationInfoMETA specifying the calibration parameters.

Applications **can** choose to call xrSuggestBodyTrackingCalibrationOverrideMETA on app startup, or implement a UI to allow users to specify when it should be called.

Runtimes **must** return XR_ERROR_VALIDATION_FAILURE if the specified height falls outside the range (0.5 meters, 3 meters).

Runtimes **should** use the provided calibration data as an input to calibrate any body tracking algorithms.

Valid Usage (Implicit)

- The XR_META_body_tracking_calibration extension **must** be enabled prior to calling xrSuggestBodyTrackingCalibrationOverrideMETA
- bodyTracker **must** be a valid XrBodyTrackerFB handle
- calibrationInfo **must** be a pointer to a valid XrBodyTrackingCalibrationInfoMETA structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FEATURE_UNSUPPORTED
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The XrBodyTrackingCalibrationInfoMETA structure is defined as:

```
// Provided by XR_META_body_tracking_calibration
typedef struct XrBodyTrackingCalibrationInfoMETA {
    XrStructureType    type;
    const void*        next;
    float              bodyHeight;
} XrBodyTrackingCalibrationInfoMETA;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain.
- bodyHeight is a float value specifying the height of the user to use for calibration (in meters).

The XrBodyTrackingCalibrationInfoMETA structure contains the user's height that the runtime **should** use to calibrate body tracking.

Valid Usage (Implicit)

- The `XR_META_body_tracking_calibration` extension **must** be enabled prior to using XrBodyTrackingCalibrationInfoMETA
- `type` **must** be `XR_TYPE_BODY_TRACKING_CALIBRATION_INFO_META`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

12.143.4. Reset body tracking calibration.

The xrResetBodyTrackingCalibrationMETA function is defined as:

```
// Provided by XR_META_body_tracking_calibration
XrResult xrResetBodyTrackingCalibrationMETA(
    XrBodyTrackerFB                             bodyTracker);
```

Parameter Descriptions

- `bodyTracker` is an XrBodyTrackerFB previously created by xrCreateBodyTrackerFB.

This function removes any height override previously set by xrSuggestBodyTrackingCalibrationOverrideMETA and allows the runtime to return to automatic calibration.

Valid Usage (Implicit)

- The `XR_META_body_tracking_calibration` extension **must** be enabled prior to calling xrResetBodyTrackingCalibrationMETA
- `bodyTracker` **must** be a valid XrBodyTrackerFB handle

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FEATURE_UNSUPPORTED
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

12.143.5. Body Tracking Calibration Status

The XrBodyTrackingCalibrationStateMETA enum describes the current calibration state.

```
// Provided by XR_META_body_tracking_calibration
typedef enum XrBodyTrackingCalibrationStateMETA {
    XR_BODY_TRACKING_CALIBRATION_STATE_VALID_META = 1,
    XR_BODY_TRACKING_CALIBRATION_STATE_CALIBRATING_META = 2,
    XR_BODY_TRACKING_CALIBRATION_STATE_INVALID_META = 3,
    XR_BODY_TRACKING_CALIBRATION_STATE_MAX_ENUM_META = 0x7FFFFFFF
} XrBodyTrackingCalibrationStateMETA;
```

Enum	Description
XR_BODY_TRACKING_CALIBRATION_STATE_VALID_META	Valid calibration, body tracking expected to be stable.
XR_BODY_TRACKING_CALIBRATION_STATE_CALIBRATING_META	Calibration is in progress, body joint poses from XR_FB_body_tracking are valid to read and internally consistent within a single call, but **may** be more variable than normal, shifting dramatically before calibration ends.

Enum	Description
XR_BODY_TRACKING_CALIBRATION_STATE_INVALID_META	Calibration is invalid, accessing the body joint poses from XR_FB_body_tracking is defined behavior but their contents are unspecified. Tracking data **may** be read but **must** not be interpreted as body tracking poses.

The XrBodyTrackingCalibrationStatusMETA structure is defined as:

```
// Provided by XR_META_body_tracking_calibration
typedef struct XrBodyTrackingCalibrationStatusMETA {
    XrStructureType                       type;
    void*                                 next;
    XrBodyTrackingCalibrationStateMETA    status;
} XrBodyTrackingCalibrationStatusMETA;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain.
- status is one of XrBodyTrackingCalibrationStateMETA indicating the current calibration status.

The XrBodyTrackingCalibrationStatusMETA structure contains a XrBodyTrackingCalibrationStateMETA that describes the current calibration status.

The application **can** obtain the calibration status by adding XrBodyTrackingCalibrationStatusMETA to the XrBodyJointLocationsFB::next chain when calling xrLocateBodyJointsFB.

Valid Usage (Implicit)

- The XR_META_body_tracking_calibration extension **must** be enabled prior to using XrBodyTrackingCalibrationStatusMETA
- type **must** be XR_TYPE_BODY_TRACKING_CALIBRATION_STATUS_META
- next **must** be NULL or a valid pointer to the next structure in a structure chain

12.143.6. New Commands

- xrResetBodyTrackingCalibrationMETA
- xrSuggestBodyTrackingCalibrationOverrideMETA

12.143.7. New Structures

- XrBodyTrackingCalibrationInfoMETA
- Extending XrBodyJointLocationsFB:
  - XrBodyTrackingCalibrationStatusMETA
- Extending XrSystemProperties:
  - XrSystemPropertiesBodyTrackingCalibrationMETA

12.143.8. New Enums

- XrBodyTrackingCalibrationStateMETA

12.143.9. New Enum Constants

- `XR_META_BODY_TRACKING_CALIBRATION_EXTENSION_NAME`
- `XR_META_body_tracking_calibration_SPEC_VERSION`
- Extending XrStructureType:
  - `XR_TYPE_BODY_TRACKING_CALIBRATION_INFO_META`
  - `XR_TYPE_BODY_TRACKING_CALIBRATION_STATUS_META`
  - `XR_TYPE_SYSTEM_PROPERTIES_BODY_TRACKING_CALIBRATION_META`

12.143.10. Version History

- Revision 1, 2025-07-23 (Bill Orr)
  - Initial extension description

12.144. XR_META_body_tracking_fidelity

Name String

`XR_META_body_tracking_fidelity`

Extension Type

Instance extension

Registered Extension Number

285

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0
and
XR_FB_body_tracking

Last Modified Date

2023-09-28

IP Status

No known IP claims.

Contributors

Giancarlo Di Biase, Meta
Dikpal Reddy, Meta
Igor Tceglevskii, Meta
Bill Orr, Meta

12.144.1. Overview

This extension enables applications to control tradeoffs with body tracking quality and performance.

This extension builds on top of the XR_FB_body_tracking extension which provides the actual body tracking output.

12.144.2. Interaction with other extensions

- XR_META_simultaneous_hands_and_controllers - Both high and low fidelity body tracking is able to work regardless of whether hands or controllers are in use. Body tracking hands **should** reflect the location of the user's hands.
- XR_META_hand_tracking_wide_motion_mode - Wide motion mode for hand tracking **may** use body tracking internally, and **may** force the system to use high fidelity. In this case, the system **may** use high fidelity tracking even when the application requests low fidelity.
- XR_META_detached_controllers - Both high and low fidelity body tracking is able to work regardless of whether hands or controllers are in use. Body tracking **should** not use the location of detached controllers to calculate its output.
- XR_META_body_tracking_full_body - Full body tracking **must** be compatible with both high fidelity and low fidelity body tracking. If a runtime does not support a low fidelity full body tracking system, the runtime **may** report that high fidelity is already in use when full body is in use.
- XR_EXT_hand_tracking_data_source - When the runtime is simulating hands via controllers, body tracking will similarly simulate hands via controllers.

12.144.3. Inspect system capability

The XrSystemPropertiesBodyTrackingFidelityMETA structure is defined as:

```
// Provided by XR_META_body_tracking_fidelity
typedef struct XrSystemPropertiesBodyTrackingFidelityMETA {
    XrStructureType    type;
    void*              next;
    XrBool32           supportsBodyTrackingFidelity;
} XrSystemPropertiesBodyTrackingFidelityMETA;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `supportsBodyTrackingFidelity` is an `XrBool32`, indicating if current system is capable of changing body tracking fidelity.

An application **can** inspect whether the system is capable of controlling body tracking fidelity by extending the XrSystemProperties with XrSystemPropertiesBodyTrackingFidelityMETA structure when calling xrGetSystemProperties.

If a runtime returns `XR_FALSE` for `supportsBodyTrackingFidelity`, the runtime **must** return `XR_ERROR_FEATURE_UNSUPPORTED` from xrRequestBodyTrackingFidelityMETA.

Valid Usage (Implicit)

- The `XR_META_body_tracking_fidelity` extension **must** be enabled prior to using XrSystemPropertiesBodyTrackingFidelityMETA
- `type` **must** be `XR_TYPE_SYSTEM_PROPERTIES_BODY_TRACKING_FIDELITY_META`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

12.144.4. Setting requested fidelity

The xrRequestBodyTrackingFidelityMETA function is defined as:

```
// Provided by XR_META_body_tracking_fidelity
XrResult xrRequestBodyTrackingFidelityMETA(
    XrBodyTrackerFB                         bodyTracker,
    const XrBodyTrackingFidelityMETA        fidelity);
```

Parameter Descriptions

- `bodyTracker` is an XrBodyTrackerFB used to specify which body tracker we want to configure fidelity.
- `fidelity` is the XrBodyTrackingFidelityMETA enum value specifying the desired fidelity.

An application **can** request a desired fidelity using the xrRequestBodyTrackingFidelityMETA function.

If the system does not support changing body tracking fidelity, the runtime **must** return `XR_ERROR_FEATURE_UNSUPPORTED` from xrRequestBodyTrackingFidelityMETA.

The xrRequestBodyTrackingFidelityMETA function allows applications to express a preference for body tracking fidelity, but this is a request that the runtime is allowed to ignore. The runtime **must** return `XR_SUCCESS` if the request is valid, even if the runtime will not honor the requested fidelity level.

The runtime **may** ignore the requested fidelity due to system load, thermal state, or when other active extensions require a higher fidelity level. When the application calls xrLocateBodyJointsFB with XrBodyTrackingFidelityStatusMETA in the next chain, the runtime will report the actual fidelity level in use.

Valid Usage (Implicit)

- The `XR_META_body_tracking_fidelity` extension **must** be enabled prior to calling xrRequestBodyTrackingFidelityMETA
- `bodyTracker` **must** be a valid XrBodyTrackerFB handle
- `fidelity` **must** be a valid XrBodyTrackingFidelityMETA value

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FEATURE_UNSUPPORTED
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

```
// Provided by XR_META_body_tracking_fidelity
typedef enum XrBodyTrackingFidelityMETA {
    XR_BODY_TRACKING_FIDELITY_LOW_META = 1,
    XR_BODY_TRACKING_FIDELITY_HIGH_META = 2,
    XR_BODY_TRACKING_FIDELITY_MAX_ENUM_META = 0x7FFFFFFF
} XrBodyTrackingFidelityMETA;
```

Enum	Description
XR_BODY_TRACKING_FIDELITY_LOW_META	This value indicates that the body tracking runtime uses a simpler algorithm or less input to estimate the body pose in a way that trades off pose quality for system performance. For example, low fidelity may not accurately track a user's elbows.
XR_BODY_TRACKING_FIDELITY_HIGH_META	This value indicates that the body tracking runtime uses more advanced algorithms or more inputs to estimate the body pose. For example high fidelity may track a user's elbows.

The runtime **may** support a variety of fidelity options for body tracking expression. The application **can** request a desired fidelity, and the runtime will attempt to use that fidelity to provide body tracking for this application.

The XrBodyTrackingFidelityStatusMETA structure is defined as:

```
// Provided by XR_META_body_tracking_fidelity
typedef struct XrBodyTrackingFidelityStatusMETA {
    XrStructureType               type;
    void*                         next;
    XrBodyTrackingFidelityMETA    fidelity;
} XrBodyTrackingFidelityStatusMETA;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `fidelity` is an XrBodyTrackingFidelityMETA that describes the fidelity in use by the runtime.

The XrBodyTrackingFidelityStatusMETA structure is used to obtain the fidelity of the body tracking system.

An application **can** inspect which fidelity is in use by the runtime by extending the XrBodyJointLocationsFB with XrBodyTrackingFidelityStatusMETA structure when calling xrLocateBodyJointsFB.

Valid Usage (Implicit)

- The `XR_META_body_tracking_fidelity` extension **must** be enabled prior to using XrBodyTrackingFidelityStatusMETA
- `type` **must** be `XR_TYPE_BODY_TRACKING_FIDELITY_STATUS_META`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

12.144.5. Example code for setting fidelity

The following example code demonstrates how to set and inspect the body tracking fidelity.

```
XrInstance instance; // previously initialized
XrSystemId systemId; // previously initialized
XrSession session;   // previously initialized
XrBodyTrackerFB bodyTracker; // previously initialized
PFN_xrLocateBodyJointsFB pfnLocateBodyJointsFB; // previously initialized
XrSpace stageSpace; // previously initialized

// Confirm body tracking fidelity system support.
XrSystemPropertiesBodyTrackingFidelityMETA bodyTrackingFidelitySystemProperties{
    .type = XR_TYPE_SYSTEM_PROPERTIES_BODY_TRACKING_FIDELITY_META,
```

```
};
XrSystemProperties systemProperties{
    .type = XR_TYPE_SYSTEM_PROPERTIES,
    .next = &bodyTrackingFidelitySystemProperties,
};
CHK_XR(xrGetSystemProperties(instance, systemId, &systemProperties));
if (!bodyTrackingFidelitySystemProperties.supportsBodyTrackingFidelity) {
    // The system does not support body tracking fidelity.
    return;
}

// Get function pointer for xrRequestBodyTrackingFidelityMETA.
PFN_xrRequestBodyTrackingFidelityMETA pfnRequestBodyTrackingFidelity = nullptr;
CHK_XR(xrGetInstanceProcAddr(instance, "xrRequestBodyTrackingFidelityMETA",
                             reinterpret_cast<PFN_xrVoidFunction*>(
                             &pfnRequestBodyTrackingFidelity)));
CHK_XR(pfnRequestBodyTrackingFidelity(bodyTracker, XR_BODY_TRACKING_FIDELITY_HIGH_META));

while (1) {
    // ...
    // For every frame in the frame loop
    // ...
    XrFrameState frameState; // previously returned from xrWaitFrame
    const XrTime time = frameState.predictedDisplayTime;

    // input
    XrBodyJointsLocateInfoFB locateInfo{
        .type = XR_TYPE_BODY_JOINTS_LOCATE_INFO_FB,
        .next = nullptr,
        .baseSpace = stageSpace,
        .time = time,
    };

    // output
    XrBodyTrackingFidelityStatusMETA bodyFidelityStatus{
        .type = XR_TYPE_BODY_TRACKING_FIDELITY_STATUS_META,
    };
    XrBodyJointLocationFB jointLocations[XR_BODY_JOINT_COUNT_FB];
    XrBodyJointLocationsFB bodyLocations{
        .type = XR_TYPE_BODY_JOINT_LOCATIONS_FB,
        .next = &bodyFidelityStatus,
        .jointCount = XR_BODY_JOINT_COUNT_FB,
        .jointLocations = jointLocations,
    };

    CHK_XR(pfnLocateBodyJointsFB(bodyTracker, &locateInfo, &bodyLocations));
    if (bodyFidelityStatus.fidelity != XR_BODY_TRACKING_FIDELITY_HIGH_META) {
        // App-specific logic.
```

```
    }
}
```

12.144.6. New Commands

- xrRequestBodyTrackingFidelityMETA

12.144.7. New Structures

- Extending XrBodyJointLocationsFB:
  - XrBodyTrackingFidelityStatusMETA
- Extending XrSystemProperties:
  - XrSystemPropertiesBodyTrackingFidelityMETA

12.144.8. New Enums

- XrBodyTrackingFidelityMETA

12.144.9. New Enum Constants

- `XR_META_BODY_TRACKING_FIDELITY_EXTENSION_NAME`
- `XR_META_body_tracking_fidelity_SPEC_VERSION`
- Extending XrStructureType:
  - `XR_TYPE_BODY_TRACKING_FIDELITY_STATUS_META`
  - `XR_TYPE_SYSTEM_PROPERTIES_BODY_TRACKING_FIDELITY_META`

12.144.10. Version History

- Revision 1, 2023-09-28 (Bill Orr)
  - Initial extension description

12.145. XR_META_body_tracking_full_body

Name String

`XR_META_body_tracking_full_body`

Extension Type

Instance extension

Registered Extension Number

275

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

XR_FB_body_tracking

Last Modified Date

2024-08-26

IP Status

No known IP claims.

Contributors

Giancarlo Di Biase, Meta
Dikpal Reddy, Meta
Igor Tceglevskii, Meta
Bill Orr, Meta

12.145.1. Overview

This extends body tracking to support the full body including the lower body.

This extension builds on top of the XR_FB_body_tracking extension, which only exposed the upper body, torso, and hands.

12.145.2. Inspect system capability

The XrSystemPropertiesBodyTrackingFullBodyMETA structure is defined as:

```
// Provided by XR_META_body_tracking_full_body
typedef struct XrSystemPropertiesBodyTrackingFullBodyMETA {
    XrStructureType    type;
    void*              next;
    XrBool32           supportsFullBodyTracking;
} XrSystemPropertiesBodyTrackingFullBodyMETA;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension. For more details about the structure chain, see the structure being extended (XrSystemProperties).
- `supportsFullBodyTracking` is an `XrBool32`, indicating if current system is capable of locating the full body tracking jointset.

An application **can** inspect whether the system is capable of full body tracking by extending the XrSystemProperties with XrSystemPropertiesBodyTrackingFullBodyMETA structure when calling xrGetSystemProperties.

If a runtime returns `XR_FALSE` for `supportsFullBodyTracking`, the runtime **must** return `XR_ERROR_VALIDATION_FAILURE` from xrCreateBodyTrackerFB when trying to create a body tracker for full body tracking using the joint set `XR_BODY_JOINT_SET_FULL_BODY_META`.

Valid Usage (Implicit)

- The `XR_META_body_tracking_full_body` extension **must** be enabled prior to using XrSystemPropertiesBodyTrackingFullBodyMETA
- `type` **must** be `XR_TYPE_SYSTEM_PROPERTIES_BODY_TRACKING_FULL_BODY_META`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

12.145.3. Creating body tracker for full body tracking

The XrBodyJointSetFB enum is extended with a new joint set `XR_BODY_JOINT_SET_FULL_BODY_META`, which can be set as the joint set in the XrBodyTrackerCreateInfoFB info parameter to xrCreateBodyTrackerFB.

```
// Provided by XR_META_body_tracking_full_body
typedef enum XrFullBodyJointMETA {
    XR_FULL_BODY_JOINT_ROOT_META = 0,
    XR_FULL_BODY_JOINT_HIPS_META = 1,
    XR_FULL_BODY_JOINT_SPINE_LOWER_META = 2,
    XR_FULL_BODY_JOINT_SPINE_MIDDLE_META = 3,
    XR_FULL_BODY_JOINT_SPINE_UPPER_META = 4,
    XR_FULL_BODY_JOINT_CHEST_META = 5,
    XR_FULL_BODY_JOINT_NECK_META = 6,
    XR_FULL_BODY_JOINT_HEAD_META = 7,
    XR_FULL_BODY_JOINT_LEFT_SHOULDER_META = 8,
```

```
    XR_FULL_BODY_JOINT_LEFT_SCAPULA_META = 9,
    XR_FULL_BODY_JOINT_LEFT_ARM_UPPER_META = 10,
    XR_FULL_BODY_JOINT_LEFT_ARM_LOWER_META = 11,
    XR_FULL_BODY_JOINT_LEFT_HAND_WRIST_TWIST_META = 12,
    XR_FULL_BODY_JOINT_RIGHT_SHOULDER_META = 13,
    XR_FULL_BODY_JOINT_RIGHT_SCAPULA_META = 14,
    XR_FULL_BODY_JOINT_RIGHT_ARM_UPPER_META = 15,
    XR_FULL_BODY_JOINT_RIGHT_ARM_LOWER_META = 16,
    XR_FULL_BODY_JOINT_RIGHT_HAND_WRIST_TWIST_META = 17,
    XR_FULL_BODY_JOINT_LEFT_HAND_PALM_META = 18,
    XR_FULL_BODY_JOINT_LEFT_HAND_WRIST_META = 19,
    XR_FULL_BODY_JOINT_LEFT_HAND_THUMB_METACARPAL_META = 20,
    XR_FULL_BODY_JOINT_LEFT_HAND_THUMB_PROXIMAL_META = 21,
    XR_FULL_BODY_JOINT_LEFT_HAND_THUMB_DISTAL_META = 22,
    XR_FULL_BODY_JOINT_LEFT_HAND_THUMB_TIP_META = 23,
    XR_FULL_BODY_JOINT_LEFT_HAND_INDEX_METACARPAL_META = 24,
    XR_FULL_BODY_JOINT_LEFT_HAND_INDEX_PROXIMAL_META = 25,
    XR_FULL_BODY_JOINT_LEFT_HAND_INDEX_INTERMEDIATE_META = 26,
    XR_FULL_BODY_JOINT_LEFT_HAND_INDEX_DISTAL_META = 27,
    XR_FULL_BODY_JOINT_LEFT_HAND_INDEX_TIP_META = 28,
    XR_FULL_BODY_JOINT_LEFT_HAND_MIDDLE_METACARPAL_META = 29,
    XR_FULL_BODY_JOINT_LEFT_HAND_MIDDLE_PROXIMAL_META = 30,
    XR_FULL_BODY_JOINT_LEFT_HAND_MIDDLE_INTERMEDIATE_META = 31,
    XR_FULL_BODY_JOINT_LEFT_HAND_MIDDLE_DISTAL_META = 32,
    XR_FULL_BODY_JOINT_LEFT_HAND_MIDDLE_TIP_META = 33,
    XR_FULL_BODY_JOINT_LEFT_HAND_RING_METACARPAL_META = 34,
    XR_FULL_BODY_JOINT_LEFT_HAND_RING_PROXIMAL_META = 35,
    XR_FULL_BODY_JOINT_LEFT_HAND_RING_INTERMEDIATE_META = 36,
    XR_FULL_BODY_JOINT_LEFT_HAND_RING_DISTAL_META = 37,
    XR_FULL_BODY_JOINT_LEFT_HAND_RING_TIP_META = 38,
    XR_FULL_BODY_JOINT_LEFT_HAND_LITTLE_METACARPAL_META = 39,
    XR_FULL_BODY_JOINT_LEFT_HAND_LITTLE_PROXIMAL_META = 40,
    XR_FULL_BODY_JOINT_LEFT_HAND_LITTLE_INTERMEDIATE_META = 41,
    XR_FULL_BODY_JOINT_LEFT_HAND_LITTLE_DISTAL_META = 42,
    XR_FULL_BODY_JOINT_LEFT_HAND_LITTLE_TIP_META = 43,
    XR_FULL_BODY_JOINT_RIGHT_HAND_PALM_META = 44,
    XR_FULL_BODY_JOINT_RIGHT_HAND_WRIST_META = 45,
    XR_FULL_BODY_JOINT_RIGHT_HAND_THUMB_METACARPAL_META = 46,
    XR_FULL_BODY_JOINT_RIGHT_HAND_THUMB_PROXIMAL_META = 47,
    XR_FULL_BODY_JOINT_RIGHT_HAND_THUMB_DISTAL_META = 48,
    XR_FULL_BODY_JOINT_RIGHT_HAND_THUMB_TIP_META = 49,
    XR_FULL_BODY_JOINT_RIGHT_HAND_INDEX_METACARPAL_META = 50,
    XR_FULL_BODY_JOINT_RIGHT_HAND_INDEX_PROXIMAL_META = 51,
    XR_FULL_BODY_JOINT_RIGHT_HAND_INDEX_INTERMEDIATE_META = 52,
    XR_FULL_BODY_JOINT_RIGHT_HAND_INDEX_DISTAL_META = 53,
    XR_FULL_BODY_JOINT_RIGHT_HAND_INDEX_TIP_META = 54,
    XR_FULL_BODY_JOINT_RIGHT_HAND_MIDDLE_METACARPAL_META = 55,
    XR_FULL_BODY_JOINT_RIGHT_HAND_MIDDLE_PROXIMAL_META = 56,
```

```
    XR_FULL_BODY_JOINT_RIGHT_HAND_MIDDLE_INTERMEDIATE_META = 57,
    XR_FULL_BODY_JOINT_RIGHT_HAND_MIDDLE_DISTAL_META = 58,
    XR_FULL_BODY_JOINT_RIGHT_HAND_MIDDLE_TIP_META = 59,
    XR_FULL_BODY_JOINT_RIGHT_HAND_RING_METACARPAL_META = 60,
    XR_FULL_BODY_JOINT_RIGHT_HAND_RING_PROXIMAL_META = 61,
    XR_FULL_BODY_JOINT_RIGHT_HAND_RING_INTERMEDIATE_META = 62,
    XR_FULL_BODY_JOINT_RIGHT_HAND_RING_DISTAL_META = 63,
    XR_FULL_BODY_JOINT_RIGHT_HAND_RING_TIP_META = 64,
    XR_FULL_BODY_JOINT_RIGHT_HAND_LITTLE_METACARPAL_META = 65,
    XR_FULL_BODY_JOINT_RIGHT_HAND_LITTLE_PROXIMAL_META = 66,
    XR_FULL_BODY_JOINT_RIGHT_HAND_LITTLE_INTERMEDIATE_META = 67,
    XR_FULL_BODY_JOINT_RIGHT_HAND_LITTLE_DISTAL_META = 68,
    XR_FULL_BODY_JOINT_RIGHT_HAND_LITTLE_TIP_META = 69,
    XR_FULL_BODY_JOINT_LEFT_UPPER_LEG_META = 70,
    XR_FULL_BODY_JOINT_LEFT_LOWER_LEG_META = 71,
    XR_FULL_BODY_JOINT_LEFT_FOOT_ANKLE_TWIST_META = 72,
    XR_FULL_BODY_JOINT_LEFT_FOOT_ANKLE_META = 73,
    XR_FULL_BODY_JOINT_LEFT_FOOT_SUBTALAR_META = 74,
    XR_FULL_BODY_JOINT_LEFT_FOOT_TRANSVERSE_META = 75,
    XR_FULL_BODY_JOINT_LEFT_FOOT_BALL_META = 76,
    XR_FULL_BODY_JOINT_RIGHT_UPPER_LEG_META = 77,
    XR_FULL_BODY_JOINT_RIGHT_LOWER_LEG_META = 78,
    XR_FULL_BODY_JOINT_RIGHT_FOOT_ANKLE_TWIST_META = 79,
    XR_FULL_BODY_JOINT_RIGHT_FOOT_ANKLE_META = 80,
    XR_FULL_BODY_JOINT_RIGHT_FOOT_SUBTALAR_META = 81,
    XR_FULL_BODY_JOINT_RIGHT_FOOT_TRANSVERSE_META = 82,
    XR_FULL_BODY_JOINT_RIGHT_FOOT_BALL_META = 83,
    XR_FULL_BODY_JOINT_COUNT_META = 84,
    XR_FULL_BODY_JOINT_NONE_META = 85,
    XR_FULL_BODY_JOINT_MAX_ENUM_META = 0x7FFFFFFF
} XrFullBodyJointMETA;
```

These joint enumeration values index into the array returned by body tracking when the joint set `XR_BODY_JOINT_SET_FULL_BODY_META` is used. There are a total of `XR_FULL_BODY_JOINT_COUNT_META` joints in this set.

The joint indices shared with XrBodyJointFB have the same semantic meaning. The meaning of joint index 0 through `XR_BODY_JOINT_COUNT_FB` - 1 (69) matches the corresponding index in XrBodyJointFB.

12.145.4. Example code for setting full body tracking

The following example code demonstrates how to create a body tracker with full body joint set.

```
XrInstance instance; // previously initialized
XrSystemId systemId; // previously initialized
XrSession session;   // previously initialized
```

```
PFN_xrCreateBodyTrackerFB pfnCreateBodyTrackerFB; // previously initialized
PFN_xrLocateBodyJointsFB pfnLocateBodyJointsFB; // previously initialized
XrSpace stageSpace; // previously initialized

// Confirm full body tracking system support.
XrSystemPropertiesBodyTrackingFullBodyMETA bodyTrackingFullBodySystemProperties{
    .type = XR_TYPE_SYSTEM_PROPERTIES_BODY_TRACKING_FULL_BODY_META,
};
XrSystemProperties systemProperties{
    .type = XR_TYPE_SYSTEM_PROPERTIES,
    .next = &bodyTrackingFullBodySystemProperties,
};
CHK_XR(xrGetSystemProperties(instance, systemId, &systemProperties));
if (!bodyTrackingFullBodySystemProperties.supportsFullBodyTracking) {
  // The system does not support full body tracking.
  return;
}


XrBodyTrackerCreateInfoFB createInfo{
    .type = XR_TYPE_BODY_TRACKER_CREATE_INFO_FB,
    .next =  nullptr,
    .bodyJointSet = XR_BODY_JOINT_SET_FULL_BODY_META,
};
XrBodyTrackerFB bodyTracker{XR_NULL_HANDLE};
CHK_XR(pfnCreateBodyTrackerFB(session, &createInfo, &bodyTracker));

while (1) {
  // ...
  // For every frame in the frame loop
  // ...
  XrFrameState frameState; // previously returned from xrWaitFrame
  const XrTime time = frameState.predictedDisplayTime;

  XrBodyJointLocationFB jointLocations[XR_FULL_BODY_JOINT_COUNT_META];
  XrBodyJointsLocateInfoFB locateInfo{
      .type = XR_TYPE_BODY_JOINTS_LOCATE_INFO_FB,
      .next = nullptr,
      .baseSpace = stageSpace,
      .time = time,
  };
  XrBodyJointLocationsFB bodyLocations{
      .type = XR_TYPE_BODY_JOINT_LOCATIONS_FB,
      .next = nullptr,
      .jointCount = XR_FULL_BODY_JOINT_COUNT_META,
      .jointLocations = jointLocations,
  };
```

```
    CHK_XR(pfnLocateBodyJointsFB(bodyTracker, &locateInfo, &bodyLocations));
}
```

12.145.5. New Structures

- Extending XrSystemProperties:
  - XrSystemPropertiesBodyTrackingFullBodyMETA

12.145.6. New Enums

- XrFullBodyJointMETA

12.145.7. New Enum Constants

- XR_META_BODY_TRACKING_FULL_BODY_EXTENSION_NAME
- XR_META_body_tracking_full_body_SPEC_VERSION
- Extending XrBodyJointSetFB:
  - XR_BODY_JOINT_SET_FULL_BODY_META
- Extending XrStructureType:
  - XR_TYPE_SYSTEM_PROPERTIES_BODY_TRACKING_FULL_BODY_META

Issues

Version History

- Revision 1, 2024-08-26 (Bill Orr)
  - Initial extension description

12.146. XR_META_boundary_visibility

Name String

XR_META_boundary_visibility

Extension Type

Instance extension

Registered Extension Number

529

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Last Modified Date

2024-02-22

IP Status

No known IP claims.

Contributors

Yuichi Taguchi, Meta Platforms
Jasmine Ou, Meta Platforms
Shen Li, Meta Platforms

12.146.1. Overview

Some applications include both MR/AR (users see their environment) and VR (users have immersive experiences) portions. In the MR/AR portion, the application might want to suppress the boundary visibility so that the user moves freely if there are no safety concerns. In the VR portion, the application might not want to suppress the boundary visibility so that the user stays safely within the boundary.

This extension enables the application to request the runtime to change the boundary visibility. The runtime **may** accept such a request depending on runtime-specific conditions.

12.146.2. Requesting the runtime to change the boundary visibility

The xrRequestBoundaryVisibilityMETA function is defined as:

```
// Provided by XR_META_boundary_visibility
XrResult xrRequestBoundaryVisibilityMETA(
    XrSession                                   session,
    XrBoundaryVisibilityMETA                    boundaryVisibility);
```

Parameter Descriptions

- `session` is a handle to an XrSession.
- `boundaryVisibility` is a valid XrBoundaryVisibilityMETA value.

The xrRequestBoundaryVisibilityMETA function is used by the application to request a specific

boundary visibility represented with XrBoundaryVisibilityMETA. The function returns XR_SUCCESS if the runtime accepted the request. The function returns XR_BOUNDARY_VISIBILITY_SUPPRESSION_NOT_ALLOWED_META if the runtime did not accept the request to suppress the boundary visibility depending on runtime-specific conditions. An example of the runtime-specific conditions preventing the boundary visibility from changing **may** be the application fully obscuring the real world environment. The application **can** request suppressing the boundary visibility later again, and the runtime **may** accept the new request when the conditions are met.

Valid Usage (Implicit)

- The XR_META_boundary_visibility extension **must** be enabled prior to calling xrRequestBoundaryVisibilityMETA
- session **must** be a valid XrSession handle
- boundaryVisibility **must** be a valid XrBoundaryVisibilityMETA value

Return Codes

Success

- XR_SUCCESS
- XR_BOUNDARY_VISIBILITY_SUPPRESSION_NOT_ALLOWED_META
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FEATURE_UNSUPPORTED
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The XrBoundaryVisibilityMETA enum is defined as:

```
// Provided by XR_META_boundary_visibility
typedef enum XrBoundaryVisibilityMETA {
    XR_BOUNDARY_VISIBILITY_NOT_SUPPRESSED_META = 1,
    XR_BOUNDARY_VISIBILITY_SUPPRESSED_META = 2,
    XR_BOUNDARY_VISIBILITY_MAX_ENUM_META = 0x7FFFFFFF
} XrBoundaryVisibilityMETA;
```

Enumerant Descriptions

- XR_BOUNDARY_VISIBILITY_NOT_SUPPRESSED_META — Boundary visibility is not suppressed.
- XR_BOUNDARY_VISIBILITY_SUPPRESSED_META — Boundary visibility is suppressed.

12.146.3. Event indicating that the boundary visibility has changed in the runtime

The XrEventDataBoundaryVisibilityChangedMETA structure is defined as:

```
// Provided by XR_META_boundary_visibility
typedef struct XrEventDataBoundaryVisibilityChangedMETA {
    XrStructureType             type;
    const void*                 next;
    XrBoundaryVisibilityMETA    boundaryVisibility;
} XrEventDataBoundaryVisibilityChangedMETA;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- boundaryVisibility is an XrBoundaryVisibilityMETA value to represent the boundary visibility changed in the runtime.

The XrEventDataBoundaryVisibilityChangedMETA structure represents an event indicating that the boundary visibility has changed in the runtime. This event **may** be a result of the xrRequestBoundaryVisibilityMETA function called from this application or some other application. This event **may** also be a result of runtime-specific conditions that changed the boundary visibility. An example of the runtime-specific conditions **may** be the application fully obscuring the real world environment, in which case the runtime **may** change the boundary visibility to XR_BOUNDARY_VISIBILITY_NOT_SUPPRESSED_META.

The runtime **must** queue this event upon a successful call to the xrBeginSession function, regardless of the value of `boundaryVisibility`, so that the application is in sync on the value of the boundary visibility when a session begins running.

The runtime **must** return a valid XrSession handle for a running session.

After the application calls xrEndSession, a running session is ended and the runtime **must** not enqueue this event anymore. Therefore, the application will no longer observe any changes of the `boundaryVisibility` until another running session.

Valid Usage (Implicit)

- The `XR_META_boundary_visibility` extension **must** be enabled prior to using XrEventDataBoundaryVisibilityChangedMETA
- `type` **must** be `XR_TYPE_EVENT_DATA_BOUNDARY_VISIBILITY_CHANGED_META`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

12.146.4. System properties

The XrSystemBoundaryVisibilityPropertiesMETA structure is defined as:

```
// Provided by XR_META_boundary_visibility
typedef struct XrSystemBoundaryVisibilityPropertiesMETA {
    XrStructureType    type;
    void*              next;
    XrBool32           supportsBoundaryVisibility;
} XrSystemBoundaryVisibilityPropertiesMETA;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `supportsBoundaryVisibility` is a boolean value that determines if this capability is supported by the runtime.

The application **can** inspect whether the runtime supports this capability by extending the XrSystemProperties with XrSystemBoundaryVisibilityPropertiesMETA structure when calling xrGetSystemProperties.

If the runtime returns `XR_FALSE` for `supportsBoundaryVisibility`, the runtime **must** return `XR_ERROR_FEATURE_UNSUPPORTED` from xrRequestBoundaryVisibilityMETA.

Valid Usage (Implicit)

- The XR_META_boundary_visibility extension **must** be enabled prior to using XrSystemBoundaryVisibilityPropertiesMETA
- type **must** be XR_TYPE_SYSTEM_BOUNDARY_VISIBILITY_PROPERTIES_META
- next **must** be NULL or a valid pointer to the next structure in a structure chain

12.146.5. New Commands

- xrRequestBoundaryVisibilityMETA

12.146.6. New Structures

- XrEventDataBoundaryVisibilityChangedMETA
- Extending XrSystemProperties:
  - XrSystemBoundaryVisibilityPropertiesMETA

12.146.7. New Enums

- XrBoundaryVisibilityMETA

12.146.8. New Enum Constants

- XR_META_BOUNDARY_VISIBILITY_EXTENSION_NAME
- XR_META_boundary_visibility_SPEC_VERSION
- Extending XrResult:
  - XR_BOUNDARY_VISIBILITY_SUPPRESSION_NOT_ALLOWED_META
- Extending XrStructureType:
  - XR_TYPE_EVENT_DATA_BOUNDARY_VISIBILITY_CHANGED_META
  - XR_TYPE_SYSTEM_BOUNDARY_VISIBILITY_PROPERTIES_META

12.146.9. Issues

12.146.10. Version History

- Revision 1, 2024-02-22 (Yuichi Taguchi)
  - Initial extension description.

12.147. XR_META_colocation_discovery

Name String

`XR_META_colocation_discovery`

Extension Type

Instance extension

Registered Extension Number

572

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Last Modified Date

2024-06-15

IP Status

No known IP claims.

Contributors

TJ Gilbrough, Meta Platforms
Lionel Reyero, Meta Platforms
Scott Dewald, Meta Platforms

12.147.1. Overview

Colocation discovery is a capability available through the `XR_META_colocation_discovery` extension that allows apps to discover physically colocated devices running the same app.

In the context of this extension, "the same application" means "bytewise identical Android package name" when running on an Android-based platform.

12.147.2. Check compatibility

The XrSystemColocationDiscoveryPropertiesMETA structure is defined as:

```
// Provided by XR_META_colocation_discovery
typedef struct XrSystemColocationDiscoveryPropertiesMETA {
    XrStructureType    type;
    void*              next;
    XrBool32           supportsColocationDiscovery;
} XrSystemColocationDiscoveryPropertiesMETA;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `supportsColocationDiscovery` is an `XrBool32` specifying if colocation discovery (and relatedly, colocation advertisement) is supported.

An application **can** inspect whether the system is capable of colocation advertisement and discovery by extending the XrSystemProperties with XrSystemColocationDiscoveryPropertiesMETA structure when calling xrGetSystemProperties.

If a runtime returns `XR_FALSE` for `supportsColocationDiscovery`, the runtime **must** return `XR_ERROR_FEATURE_UNSUPPORTED` for all functions in the `XR_META_colocation_discovery` extension.

Colocation advertisement controls whether a device is discoverable using colocation discovery, so the term "colocation discovery" on its own is used here to refer to the combined capability of colocation advertisement and colocation discovery.

Valid Usage (Implicit)

- The `XR_META_colocation_discovery` extension **must** be enabled prior to using XrSystemColocationDiscoveryPropertiesMETA
- `type` **must** be `XR_TYPE_SYSTEM_COLOCATION_DISCOVERY_PROPERTIES_META`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

12.147.3. Controlling Colocation Advertisement

The ability for other physically colocated devices running the same application to discover the current device is known as "colocation advertisement". The xrStartColocationAdvertisementMETA function requests starting colocation advertisement, while the xrStopColocationAdvertisementMETA requests that colocation advertisement stop. Both of these functions initiate an asynchronous operation similar to that found in extensions built on `XR_FB_spatial_entity`, with their asynchronous completion results returned in an event structure (XrEventDataStartColocationAdvertisementCompleteMETA and XrEventDataStopColocationAdvertisementCompleteMETA, respectively).

Colocation advertisement **may** stop without being explicitly requested for a variety of reasons. If it stops, whether subsequent to an xrStopColocationAdvertisementMETA call or not, an XrEventDataColocationAdvertisementCompleteMETA event is queued.

The following figures show examples of the advertisement process in two general circumstances: normal use and the case where the runtime stops advertisement before the application requests it.

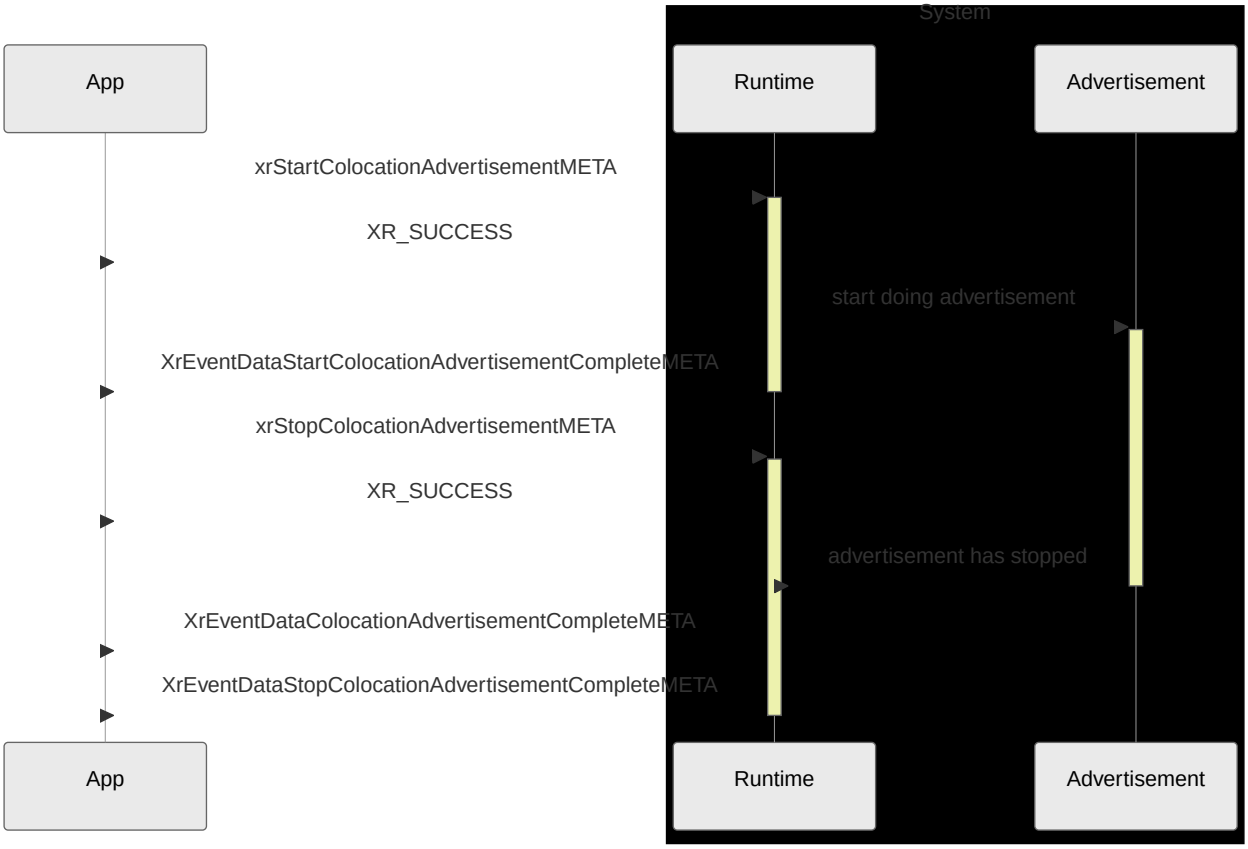


Figure 31. XR_META_colocation_discovery Advertisement Process - Normal

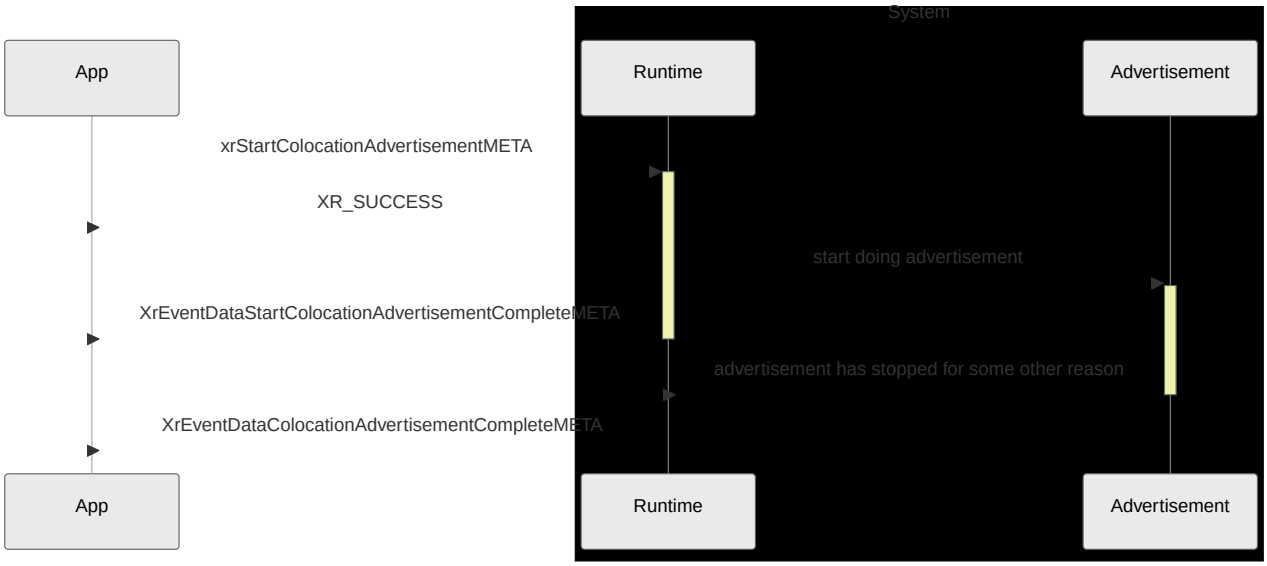


Figure 32. XR_META_colocation_discovery Advertisement Process - runtime needs to stop advertisement

The xrStartColocationAdvertisementMETA function is defined as:

```
// Provided by XR_META_colocation_discovery
XrResult xrStartColocationAdvertisementMETA(
    XrSession                                       session,
    const XrColocationAdvertisementStartInfoMETA* info,
    XrAsyncRequestIdFB*                             advertisementRequestId);
```

Parameter Descriptions

- session is an XrSession in which the colocation discovery will be active.
- info is a pointer to XrColocationAdvertisementStartInfoMETA structure to specify the visibility configuration.
- advertisementRequestId is an output parameter, and the variable it points to will be populated with the ID of this asynchronous request. Note that this ID is used for associating additional events with this original call, in addition to the typical completion event.

The xrStartColocationAdvertisementMETA function requests that the current device become discoverable by other physically colocated devices running the same application.

If the system does not support colocation advertisement and discovery, the runtime **must** return XR_ERROR_FEATURE_UNSUPPORTED from xrStartColocationAdvertisementMETA. In this case, the runtime **must** return XR_FALSE for XrSystemColocationDiscoveryPropertiesMETA::supportsColocationDiscovery when the function xrGetSystemProperties is called, so that the application knows to not use this functionality.

This is an asynchronous operation. Completion results are conveyed in the event XrEventDataStartColocationAdvertisementCompleteMETA.

If the asynchronous operation is scheduled successfully, the runtime **must** return XR_SUCCESS.

If and only if the runtime returns XR_SUCCESS, the runtime **must** queue a single XrEventDataStartColocationAdvertisementCompleteMETA event identified with a advertisementRequestId matching the advertisementRequestId value output by this function, referred to as the "corresponding completion event."

(This implies that if the runtime returns anything other than XR_SUCCESS, the runtime **must** not queue any XrEventDataStartColocationAdvertisementCompleteMETA events with advertisementRequestId field matching the advertisementRequestId populated by this function.)

If the asynchronous operation is successful, in the corresponding completion event, the runtime **must** set the XrEventDataStartColocationAdvertisementCompleteMETA::result field to XR_SUCCESS.

If the asynchronous operation is scheduled but not successful, in the corresponding completion event,

the runtime **must** set the XrEventDataStartColocationAdvertisementCompleteMETA::`result` field to an appropriate error code instead of `XR_SUCCESS`.

See Figure 31 and Figure 32 for sample flows incorporating use of xrStartColocationAdvertisementMETA.

Valid Usage (Implicit)

- The `XR_META_colocation_discovery` extension **must** be enabled prior to calling xrStartColocationAdvertisementMETA
- `session` **must** be a valid XrSession handle
- `info` **must** be a pointer to a valid XrColocationAdvertisementStartInfoMETA structure
- `advertisementRequestId` **must** be a pointer to an `XrAsyncRequestIdFB` value

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_EXTENSION_NOT_PRESENT`
- `XR_ERROR_FEATURE_UNSUPPORTED`
- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_RUNTIME_UNAVAILABLE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_VALIDATION_FAILURE`

The XrColocationAdvertisementStartInfoMETA structure is defined as:

```
// Provided by XR_META_colocation_discovery
typedef struct XrColocationAdvertisementStartInfoMETA {
    XrStructureType    type;
    const void*        next;
    uint32_t           bufferSize;
    uint8_t*           buffer;
} XrColocationAdvertisementStartInfoMETA;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR.
- `bufferSize` is the count of bytes used in the `buffer` array. If `bufferSize` is greater than `XR_MAX_COLOCATION_DISCOVERY_BUFFER_SIZE_META`, then the runtime **must** return `XR_ERROR_VALIDATION_FAILURE`. The maximum supported count is `XR_MAX_COLOCATION_DISCOVERY_BUFFER_SIZE_META`.
- `buffer` is a byte array which allows the application to define app buffer. It is the advertisement payload.

XrColocationAdvertisementStartInfoMETA is the input data for xrStartColocationAdvertisementMETA. Implicitly, while the application has an active advertisement, the runtime will retain a copy of the XrColocationAdvertisementStartInfoMETA submitted with xrStartColocationAdvertisementMETA.

Valid Usage (Implicit)

- The `XR_META_colocation_discovery` extension **must** be enabled prior to using XrColocationAdvertisementStartInfoMETA
- `type` **must** be `XR_TYPE_COLOCATION_ADVERTISEMENT_START_INFO_META`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `buffer` **must** be a pointer to an array of `bufferSize` `uint8_t` values
- The `bufferSize` parameter **must** be greater than `0`

```
#define XR_MAX_COLOCATION_DISCOVERY_BUFFER_SIZE_META 1024
```

`XR_MAX_COLOCATION_DISCOVERY_BUFFER_SIZE_META` is the maximum size of data supported in a colocation advertisement.

The XrEventDataStartColocationAdvertisementCompleteMETA event structure is defined as:

```
// Provided by XR_META_colocation_discovery
typedef struct XrEventDataStartColocationAdvertisementCompleteMETA {
    XrStructureType       type;
    const void*           next;
    XrAsyncRequestIdFB    advertisementRequestId;
    XrResult              result;
    XrUuid                advertisementUuid;
} XrEventDataStartColocationAdvertisementCompleteMETA;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR.
- `advertisementRequestId` is an `XrAsyncRequestIdFB` that identifies which request this event is in response to. The runtime **must** ensure this value matches a `advertisementRequestId` returned during an xrStartColocationAdvertisementMETA call.
- `result` is an XrResult that specifies the request result. The valid `result` values are: `XR_SUCCESS`, `XR_ERROR_RUNTIME_FAILURE`, `XR_COLOCATION_DISCOVERY_ALREADY_ADVERTISING_META`, `XR_ERROR_COLOCATION_DISCOVERY_NO_DISCOVERY_METHOD_META`, and `XR_ERROR_COLOCATION_DISCOVERY_NETWORK_FAILED_META`. If the application already has an active colocation advertisement, the runtime **must** return `XR_COLOCATION_DISCOVERY_ALREADY_ADVERTISING_META` in this field.
- `advertisementUuid` is an XrUuid which is discoverable by other physically colocated devices running the same application. The runtime **must** return a unique XrUuid which has not been returned in a previous XrEventDataStartColocationAdvertisementCompleteMETA event, if `result` is `XR_SUCCESS`.

This event conveys the results of the asynchronous operation started by xrStopColocationAdvertisementMETA.

Valid Usage (Implicit)

- The `XR_META_colocation_discovery` extension **must** be enabled prior to using XrEventDataStartColocationAdvertisementCompleteMETA
- `type` **must** be `XR_TYPE_EVENT_DATA_START_COLOCATION_ADVERTISEMENT_COMPLETE_META`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

The XrEventDataColocationAdvertisementCompleteMETA structure is defined as:

```
// Provided by XR_META_colocation_discovery
typedef struct XrEventDataColocationAdvertisementCompleteMETA {
    XrStructureType       type;
    const void*           next;
    XrAsyncRequestIdFB    advertisementRequestId;
    XrResult              result;
} XrEventDataColocationAdvertisementCompleteMETA;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR.
- `advertisementRequestId` is an `XrAsyncRequestIdFB` specifying the original start advertisement async request id. The runtime **must** return a value which matches a previously returned `advertisementRequestId` value from an xrStartColocationAdvertisementMETA request.
- `result` is an XrResult representing the advertisement process result. The valid `result` values are: `XR_SUCCESS`, `XR_ERROR_RUNTIME_FAILURE`, and `XR_ERROR_COLOCATION_DISCOVERY_NO_DISCOVERY_METHOD_META`.

The runtime **must** queue exactly one XrEventDataColocationAdvertisementCompleteMETA event whenever an active colocation advertisement is stopped. This includes if the colocation advertisement is stopped due to an application calling xrStopColocationAdvertisementMETA, or the runtime needs to stop the colocation advertisement for any reason. If the colocation advertisement is stopped due to an application calling xrStopColocationAdvertisementMETA, the runtime **must** queue the XrEventDataColocationAdvertisementCompleteMETA event before queuing the corresponding XrEventDataStopColocationAdvertisementCompleteMETA event. When the XrSession is destroyed, the runtime **must** stop all active advertisements started from the same XrSession.

See Figure 31 and Figure 32 for sample flows that show how XrEventDataColocationAdvertisementCompleteMETA is used.

Valid Usage (Implicit)

- The `XR_META_colocation_discovery` extension **must** be enabled prior to using XrEventDataColocationAdvertisementCompleteMETA
- `type` **must** be `XR_TYPE_EVENT_DATA_COLOCATION_ADVERTISEMENT_COMPLETE_META`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

The xrStopColocationAdvertisementMETA function is defined as:

```
// Provided by XR_META_colocation_discovery
XrResult xrStopColocationAdvertisementMETA(
    XrSession                                       session,
    const XrColocationAdvertisementStopInfoMETA* info,
    XrAsyncRequestIdFB*                             requestId);
```

Parameter Descriptions

- session is an XrSession in which the colocation discovery will be active.
- info is a pointer to XrColocationAdvertisementStopInfoMETA structure to specify the advertisement configuration.
- requestId is an output parameter, and the variable it points to will be populated with the ID of this asynchronous request.

The application **can** use the xrStopColocationAdvertisementMETA function to disable the ability for other physically colocated devices running the same application to discover the current device.

If the system does not support colocation advertisement and discovery, the runtime **must** return XR_ERROR_FEATURE_UNSUPPORTED from xrStopColocationAdvertisementMETA. In this case, the runtime **must** return XR_FALSE for XrSystemColocationDiscoveryPropertiesMETA::supportsColocationDiscovery when the function xrGetSystemProperties is called, so that the application knows to not use this functionality.

This is an asynchronous operation. Completion results are conveyed in the event XrEventDataStopColocationAdvertisementCompleteMETA.

If the asynchronous operation is scheduled successfully, the runtime **must** return XR_SUCCESS.

If and only if the runtime returns XR_SUCCESS, the runtime **must** queue a single XrEventDataStopColocationAdvertisementCompleteMETA event identified with a requestId matching the requestId value output by this function, referred to as the "corresponding completion event."

(This implies that if the runtime returns anything other than XR_SUCCESS, the runtime **must** not queue any XrEventDataStopColocationAdvertisementCompleteMETA events with requestId field matching the requestId populated by this function.)

If the asynchronous operation is successful, in the corresponding completion event, the runtime **must** set the XrEventDataStopColocationAdvertisementCompleteMETA::result field to XR_SUCCESS.

If the asynchronous operation is scheduled but not successful, in the corresponding completion event, the runtime **must** set the XrEventDataStopColocationAdvertisementCompleteMETA::result field to an appropriate error code instead of XR_SUCCESS.

See Figure 31 for a sample flow incorporating use of xrStopColocationAdvertisementMETA.

Valid Usage (Implicit)

- The `XR_META_colocation_discovery` extension **must** be enabled prior to calling xrStopColocationAdvertisementMETA
- `session` **must** be a valid XrSession handle
- `info` **must** be a pointer to a valid XrColocationAdvertisementStopInfoMETA structure
- `requestId` **must** be a pointer to an `XrAsyncRequestIdFB` value

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_EXTENSION_NOT_PRESENT`
- `XR_ERROR_FEATURE_UNSUPPORTED`
- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_RUNTIME_UNAVAILABLE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_VALIDATION_FAILURE`

The XrColocationAdvertisementStopInfoMETA structure is defined as:

```
// Provided by XR_META_colocation_discovery
typedef struct XrColocationAdvertisementStopInfoMETA {
    XrStructureType    type;
    const void*        next;
} XrColocationAdvertisementStopInfoMETA;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR.

Valid Usage (Implicit)

- The `XR_META_colocation_discovery` extension **must** be enabled prior to using XrColocationAdvertisementStopInfoMETA
- `type` **must** be `XR_TYPE_COLOCATION_ADVERTISEMENT_STOP_INFO_META`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

The XrEventDataStopColocationAdvertisementCompleteMETA event structure is defined as:

```
// Provided by XR_META_colocation_discovery
typedef struct XrEventDataStopColocationAdvertisementCompleteMETA {
    XrStructureType       type;
    const void*           next;
    XrAsyncRequestIdFB    requestId;
    XrResult              result;
} XrEventDataStopColocationAdvertisementCompleteMETA;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR.
- `requestId` is an `XrAsyncRequestIdFB` that identifies which request this event is in response to. The runtime **must** ensure this value matches a `requestId` returned during an xrStopColocationAdvertisementMETA call.
- `result` is an XrResult that specifies the request result. The valid `result` values are: `XR_SUCCESS`, `XR_ERROR_RUNTIME_FAILURE`, and `XR_ERROR_COLOCATION_DISCOVERY_NETWORK_FAILED_META`. If the application does not already have an active colocation advertisement, the runtime **must** return `XR_SUCCESS`.

This event conveys the results of the asynchronous operation started by xrStopColocationAdvertisementMETA.

Valid Usage (Implicit)

- The `XR_META_colocation_discovery` extension **must** be enabled prior to using XrEventDataStopColocationAdvertisementCompleteMETA
- `type` **must** be `XR_TYPE_EVENT_DATA_STOP_COLOCATION_ADVERTISEMENT_COMPLETE_META`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

12.147.4. Colocation Discovery

Discovering other physically colocated devices, running the same application and currently advertising, is known as "colocation discovery". It is a background process that is controlled by xrStartColocationDiscoveryMETA and xrStopColocationDiscoveryMETA. Both of these functions initiate an asynchronous operation similar to that found in extensions built on `XR_FB_spatial_entity`, with their asynchronous completion results returned in an event structure (XrEventDataStartColocationDiscoveryCompleteMETA and XrEventDataStopColocationAdvertisementCompleteMETA, respectively).

Results from colocation discovery, if it is successfully started, are returned through a sequence of XrEventDataColocationDiscoveryResultMETA events. When colocation discovery stops for any reason (application request or otherwise), an XrEventDataColocationDiscoveryCompleteMETA event is enqueued.

The following figures show examples of the discovery process in two general circumstances: normal use and the case where the runtime stops discovery before the application requests it.

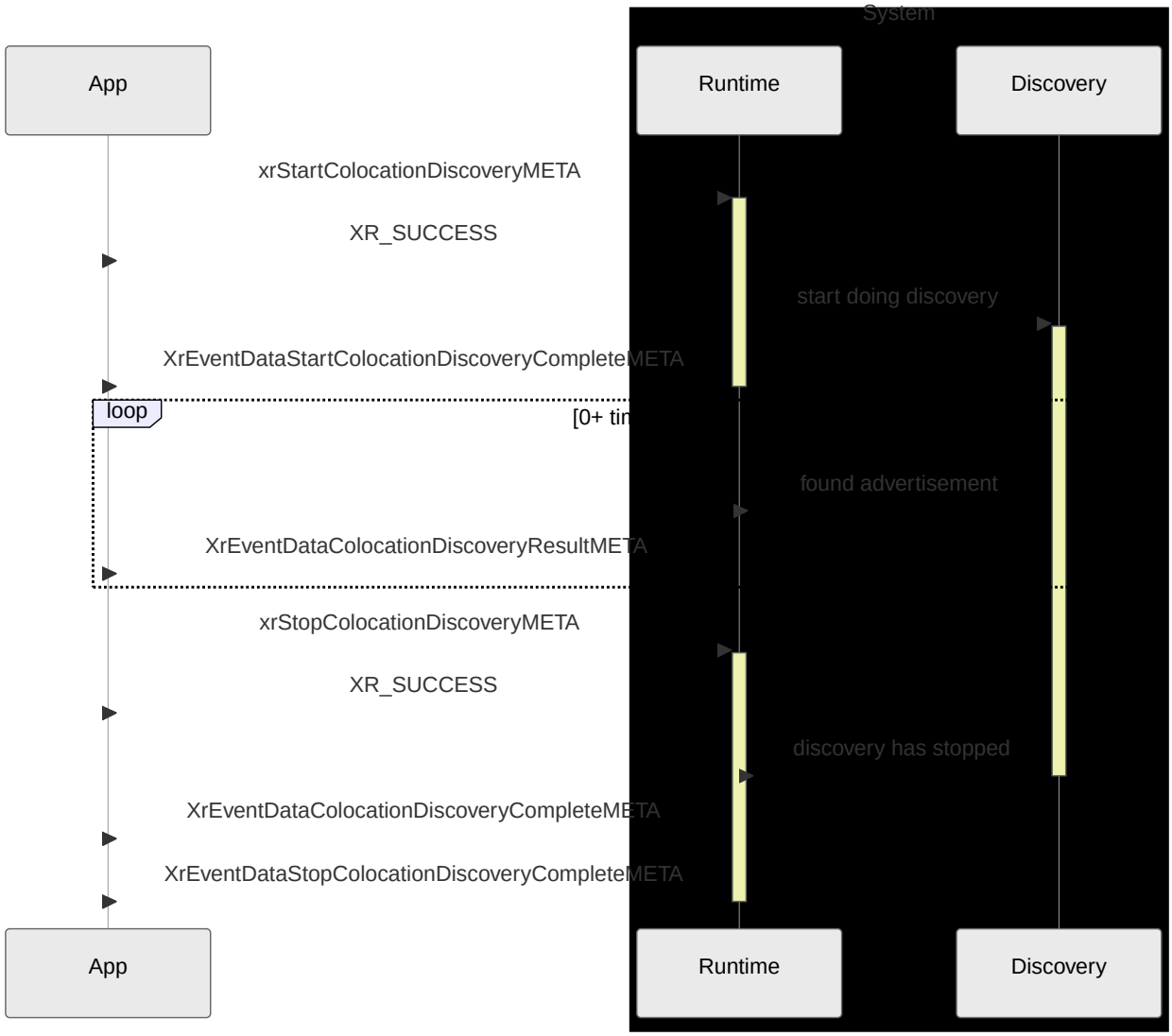


Figure 33. XR_META_colocation_discovery Discovery Process - Normal

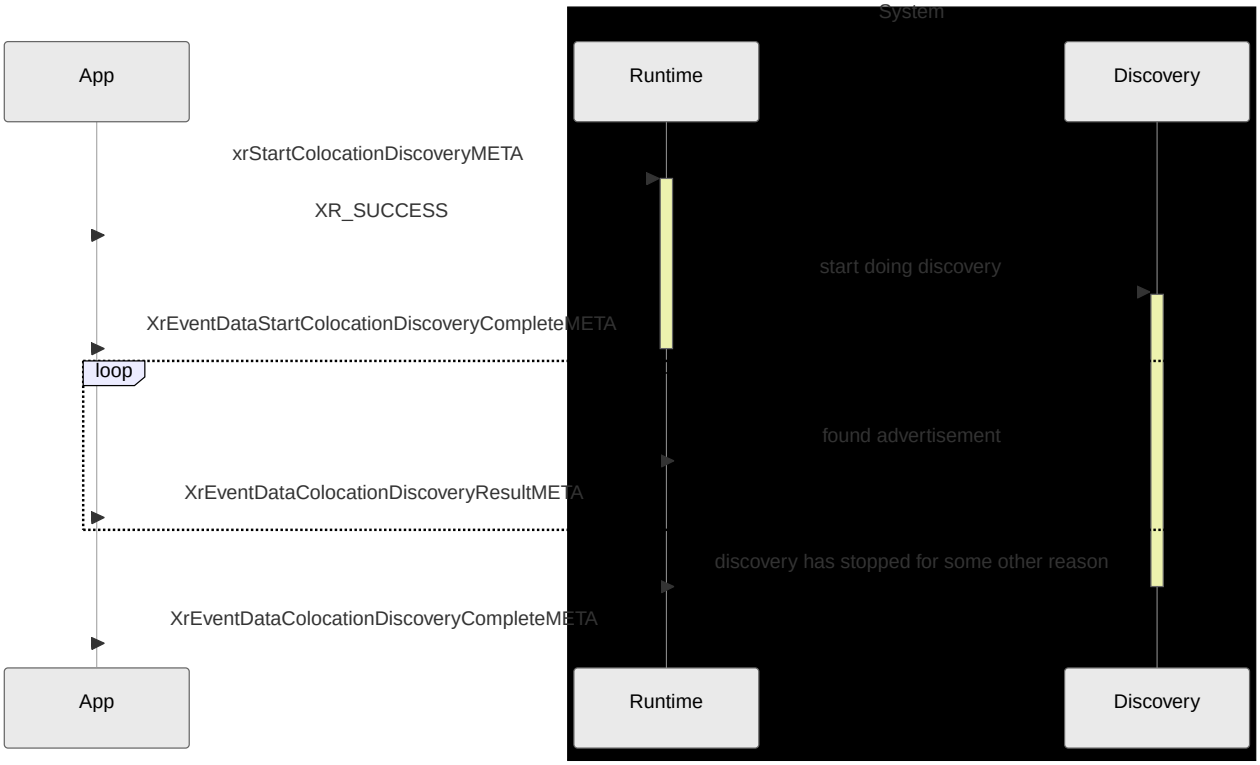


Figure 34. XR_META_colocation_discovery Discovery Process - runtime needs to stop discovery

The xrStartColocationDiscoveryMETA function is defined as:

```
// Provided by XR_META_colocation_discovery
XrResult xrStartColocationDiscoveryMETA(
    XrSession                                   session,
    const XrColocationDiscoveryStartInfoMETA*   info,
    XrAsyncRequestIdFB*                         discoveryRequestId);
```

Parameter Descriptions

- `session` is an XrSession in which the colocation discovery will be active.
- `info` is a pointer to XrColocationDiscoveryStartInfoMETA structure to specify the discovery request information.
- `discoveryRequestId` is an output parameter, and the variable it points to will be populated with the ID of this asynchronous request. Note that this ID is used for associating additional events with this original call, in addition to the typical completion event.

The application **can** call xrStartColocationDiscoveryMETA to start discovering physically colocated devices.

If the system does not support colocation advertisement and discovery, the runtime **must** return `XR_ERROR_FEATURE_UNSUPPORTED` from xrStartColocationDiscoveryMETA. In this case, the runtime **must** return `XR_FALSE` for XrSystemColocationDiscoveryPropertiesMETA::`supportsColocationDiscovery` when the function xrGetSystemProperties is called, so that the application knows to not use this functionality.

This is an asynchronous operation. Completion results are conveyed in the event XrEventDataStartColocationDiscoveryCompleteMETA.

If the asynchronous operation is scheduled successfully, the runtime **must** return `XR_SUCCESS`.

If and only if the runtime returns `XR_SUCCESS`, the runtime **must** queue a single XrEventDataStartColocationDiscoveryCompleteMETA event identified with a `discoveryRequestId` matching the `discoveryRequestId` value output by this function, referred to as the "corresponding completion event."

(This implies that if the runtime returns anything other than `XR_SUCCESS`, the runtime **must** not queue any XrEventDataStartColocationDiscoveryCompleteMETA events with `discoveryRequestId` field matching the `discoveryRequestId` populated by this function.)

If the asynchronous operation is successful, in the corresponding completion event, the runtime **must** set the XrEventDataStartColocationDiscoveryCompleteMETA::`result` field to `XR_SUCCESS`. The runtime **may** subsequently queue zero or more XrEventDataColocationDiscoveryResultMETA events asynchronously as the runtime discovers nearby advertisements. Once the application or runtime

stops the colocation discovery, the runtime **must** queue a single XrEventDataColocationDiscoveryCompleteMETA event. All XrEventDataColocationDiscoveryResultMETA and XrEventDataColocationDiscoveryCompleteMETA events will identified with `discoveryRequestId` matching the value populated in `discoveryRequestId` by xrStartColocationDiscoveryMETA.

If the asynchronous operation is scheduled but not successful, in the corresponding completion event, the runtime **must** set the XrEventDataStartColocationDiscoveryCompleteMETA::`result` field to an appropriate error code instead of `XR_SUCCESS`.

If the application already has an active colocation discovery, in the corresponding completion event, the runtime **must** set the XrEventDataStartColocationDiscoveryCompleteMETA::`result` field to `XR_COLOCATION_DISCOVERY_ALREADY_DISCOVERING_META`.

See Figure 33 and Figure 34 for sample flows incorporating use of xrStartColocationDiscoveryMETA.

Valid Usage (Implicit)

- The `XR_META_colocation_discovery` extension **must** be enabled prior to calling xrStartColocationDiscoveryMETA
- `session` **must** be a valid XrSession handle
- `info` **must** be a pointer to a valid XrColocationDiscoveryStartInfoMETA structure
- `discoveryRequestId` **must** be a pointer to an `XrAsyncRequestIdFB` value

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_EXTENSION_NOT_PRESENT
- XR_ERROR_FEATURE_UNSUPPORTED
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_RUNTIME_UNAVAILABLE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The XrColocationDiscoveryStartInfoMETA structure is defined as:

```
// Provided by XR_META_colocation_discovery
typedef struct XrColocationDiscoveryStartInfoMETA {
    XrStructureType    type;
    const void*        next;
} XrColocationDiscoveryStartInfoMETA;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR.

Valid Usage (Implicit)

- The XR_META_colocation_discovery extension **must** be enabled prior to using XrColocationDiscoveryStartInfoMETA
- type **must** be XR_TYPE_COLOCATION_DISCOVERY_START_INFO_META
- next **must** be NULL or a valid pointer to the next structure in a structure chain

The XrEventDataStartColocationDiscoveryCompleteMETA structure is defined as:

```
// Provided by XR_META_colocation_discovery
typedef struct XrEventDataStartColocationDiscoveryCompleteMETA {
    XrStructureType       type;
    const void*           next;
    XrAsyncRequestIdFB    discoveryRequestId;
    XrResult              result;
} XrEventDataStartColocationDiscoveryCompleteMETA;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR.
- discoveryRequestId is an XrAsyncRequestIdFB specifying the original start discovery async request id. The runtime **must** return a value which matches a previously returned discoveryRequestId value from an xrStartColocationDiscoveryMETA request.
- result is an XrResult representing the discovery process result. The valid result values are: XR_SUCCESS, XR_ERROR_RUNTIME_FAILURE, XR_ERROR_COLOCATION_DISCOVERY_NETWORK_FAILED_META, XR_ERROR_COLOCATION_DISCOVERY_NO_DISCOVERY_METHOD_META, and XR_COLOCATION_DISCOVERY_ALREADY_DISCOVERING_META. If the application already has an active colocation discovery, the runtime **must** return XR_COLOCATION_DISCOVERY_ALREADY_DISCOVERING_META in XrEventDataStartColocationDiscoveryCompleteMETA::result.

This event conveys the results of the asynchronous operation started by xrStartColocationDiscoveryMETA.

Valid Usage (Implicit)

- The XR_META_colocation_discovery extension **must** be enabled prior to using XrEventDataStartColocationDiscoveryCompleteMETA
- type **must** be XR_TYPE_EVENT_DATA_START_COLOCATION_DISCOVERY_COMPLETE_META
- next **must** be NULL or a valid pointer to the next structure in a structure chain

The XrEventDataColocationDiscoveryResultMETA structure is defined as:

```
// Provided by XR_META_colocation_discovery
typedef struct XrEventDataColocationDiscoveryResultMETA {
    XrStructureType       type;
    const void*           next;
    XrAsyncRequestIdFB    discoveryRequestId;
    XrUuid                advertisementUuid;
    uint32_t              bufferSize;
    uint8_t               buffer[XR_MAX_COLOCATION_DISCOVERY_BUFFER_SIZE_META];
} XrEventDataColocationDiscoveryResultMETA;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR.
- discoveryRequestId is an XrAsyncRequestIdFB specifying the original start discovery async request id. The runtime **must** return a value which matches a previously returned discoveryRequestId value from an xrStartColocationDiscoveryMETA request.
- advertisementUuid is an XrUuid of the discovered colocated application.
- bufferSize is the count of bytes used in the buffer array.
- buffer is a byte array which is the application **may** set when the application starts the advertisement.

advertisementUuid and buffer are both considered the payload of colocated advertisements. The value of advertisementUuid matches the value returned in XrEventDataStartColocationAdvertisementCompleteMETA::advertisementUuid on the advertising device.

See Figure 33 and Figure 34 for sample flows that show how XrEventDataColocationDiscoveryResultMETA is used.

Valid Usage (Implicit)

- The XR_META_colocation_discovery extension **must** be enabled prior to using XrEventDataColocationDiscoveryResultMETA
- type **must** be XR_TYPE_EVENT_DATA_COLOCATION_DISCOVERY_RESULT_META
- next **must** be NULL or a valid pointer to the next structure in a structure chain

The XrEventDataColocationDiscoveryCompleteMETA structure is defined as:

```
// Provided by XR_META_colocation_discovery
typedef struct XrEventDataColocationDiscoveryCompleteMETA {
    XrStructureType       type;
    const void*           next;
    XrAsyncRequestIdFB    discoveryRequestId;
    XrResult              result;
} XrEventDataColocationDiscoveryCompleteMETA;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR.
- discoveryRequestId is an XrAsyncRequestIdFB specifying the original start discovery async request id. The runtime **must** return a value which matches a previously returned discoveryRequestId value from an xrStartColocationDiscoveryMETA request.
- result is an XrResult representing the discovery process result. The valid result values are: XR_SUCCESS, XR_ERROR_RUNTIME_FAILURE, and XR_ERROR_COLOCATION_DISCOVERY_NO_DISCOVERY_METHOD_META.

The runtime **must** queue exactly one XrEventDataColocationDiscoveryCompleteMETA event whenever an active colocation discovery is stopped. This includes if the colocation discovery is stopped due to an application calling xrStopColocationDiscoveryMETA, or the runtime needs to stop the colocation discovery for any reason. When the XrSession is destroyed, the runtime **must** stop all active advertisements started from the same XrSession.

See Figure 33 and Figure 34 for sample flows that show how XrEventDataColocationDiscoveryCompleteMETA is used.

Valid Usage (Implicit)

- The XR_META_colocation_discovery extension **must** be enabled prior to using XrEventDataColocationDiscoveryCompleteMETA
- type **must** be XR_TYPE_EVENT_DATA_COLOCATION_DISCOVERY_COMPLETE_META
- next **must** be NULL or a valid pointer to the next structure in a structure chain

The xrStopColocationDiscoveryMETA function is defined as:

```
// Provided by XR_META_colocation_discovery
XrResult xrStopColocationDiscoveryMETA(
    XrSession                                   session,
    const XrColocationDiscoveryStopInfoMETA*    info,
    XrAsyncRequestIdFB*                         requestId);
```

Parameter Descriptions

- session is an XrSession in which the colocation discovery will be active.
- info is a pointer to XrColocationDiscoveryStopInfoMETA structure to specify the stop discovery request information.
- requestId is an output parameter, and the variable it points to will be populated with the ID of this asynchronous request.

The application **can** call xrStopColocationDiscoveryMETA to stop an ongoing discovery process that was started by xrStartColocationDiscoveryMETA.

If the system does not support colocation advertisement and discovery, the runtime **must** return XR_ERROR_FEATURE_UNSUPPORTED from xrStopColocationDiscoveryMETA. In this case, the runtime **must** return XR_FALSE for XrSystemColocationDiscoveryPropertiesMETA::supportsColocationDiscovery when the function xrGetSystemProperties is called, so that the application knows to not use this functionality.

This is an asynchronous operation. Completion results are conveyed in the event XrEventDataStopColocationDiscoveryCompleteMETA.

If the asynchronous operation is scheduled successfully, the runtime **must** return XR_SUCCESS.

If and only if the runtime returns XR_SUCCESS, the runtime **must** queue a single XrEventDataStopColocationDiscoveryCompleteMETA event identified with a requestId matching the requestId value output by this function, referred to as the "corresponding completion event."

(This implies that if the runtime returns anything other than XR_SUCCESS, the runtime **must** not queue

any XrEventDataStopColocationDiscoveryCompleteMETA events with `requestId` field matching the `requestId` populated by this function.)

If the asynchronous operation is successful, in the corresponding completion event, the runtime **must** set the XrEventDataStopColocationDiscoveryCompleteMETA::`result` field to `XR_SUCCESS`.

If the asynchronous operation is scheduled but not successful, in the corresponding completion event, the runtime **must** set the XrEventDataStopColocationDiscoveryCompleteMETA::`result` field to an appropriate error code instead of `XR_SUCCESS`.

See Figure 33 for a sample flow incorporating use of xrStopColocationDiscoveryMETA.

Valid Usage (Implicit)

- The `XR_META_colocation_discovery` extension **must** be enabled prior to calling xrStopColocationDiscoveryMETA
- `session` **must** be a valid XrSession handle
- `info` **must** be a pointer to a valid XrColocationDiscoveryStopInfoMETA structure
- `requestId` **must** be a pointer to an `XrAsyncRequestIdFB` value

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_EXTENSION_NOT_PRESENT`
- `XR_ERROR_FEATURE_UNSUPPORTED`
- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_RUNTIME_UNAVAILABLE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_VALIDATION_FAILURE`

The XrColocationDiscoveryStopInfoMETA structure is defined as:

```
// Provided by XR_META_colocation_discovery
typedef struct XrColocationDiscoveryStopInfoMETA {
    XrStructureType    type;
    const void*        next;
} XrColocationDiscoveryStopInfoMETA;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR.

Valid Usage (Implicit)

- The `XR_META_colocation_discovery` extension **must** be enabled prior to using XrColocationDiscoveryStopInfoMETA
- `type` **must** be `XR_TYPE_COLOCATION_DISCOVERY_STOP_INFO_META`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

The XrEventDataStopColocationDiscoveryCompleteMETA structure is defined as:

```
// Provided by XR_META_colocation_discovery
typedef struct XrEventDataStopColocationDiscoveryCompleteMETA {
    XrStructureType       type;
    const void*           next;
    XrAsyncRequestIdFB    requestId;
    XrResult              result;
} XrEventDataStopColocationDiscoveryCompleteMETA;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR.
- `requestId` is an `XrAsyncRequestIdFB` that identifies which request this event is in response to. The runtime **must** ensure this value matches a `requestId` returned during an xrStopColocationDiscoveryMETA call.
- `result` is an XrResult representing the discovery process result. The valid `result` values are: `XR_SUCCESS` and `XR_ERROR_RUNTIME_FAILURE`. If the application does not already have an active colocation discovery, the runtime **must** return `XR_SUCCESS`.

This event conveys the results of the asynchronous operation started by xrStopColocationDiscoveryMETA.

Valid Usage (Implicit)

- The `XR_META_colocation_discovery` extension **must** be enabled prior to using XrEventDataStopColocationDiscoveryCompleteMETA
- `type` **must** be `XR_TYPE_EVENT_DATA_STOP_COLOCATION_DISCOVERY_COMPLETE_META`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

12.147.5. New Commands

- xrStartColocationAdvertisementMETA
- xrStartColocationDiscoveryMETA
- xrStopColocationAdvertisementMETA
- xrStopColocationDiscoveryMETA

12.147.6. New Structures

- XrColocationAdvertisementStartInfoMETA
- XrColocationAdvertisementStopInfoMETA
- XrColocationDiscoveryStartInfoMETA
- XrColocationDiscoveryStopInfoMETA
- XrEventDataColocationAdvertisementCompleteMETA
- XrEventDataColocationDiscoveryCompleteMETA

- XrEventDataColocationDiscoveryResultMETA
- XrEventDataStartColocationAdvertisementCompleteMETA
- XrEventDataStartColocationDiscoveryCompleteMETA
- XrEventDataStopColocationAdvertisementCompleteMETA
- XrEventDataStopColocationDiscoveryCompleteMETA
- Extending XrSystemProperties:
  - XrSystemColocationDiscoveryPropertiesMETA

12.147.7. New Enum Constants

- XR_MAX_COLOCATION_DISCOVERY_BUFFER_SIZE_META
- XR_META_COLOCATION_DISCOVERY_EXTENSION_NAME
- XR_META_colocation_discovery_SPEC_VERSION
- Extending XrResult:
  - XR_COLOCATION_DISCOVERY_ALREADY_ADVERTISING_META
  - XR_COLOCATION_DISCOVERY_ALREADY_DISCOVERING_META
  - XR_ERROR_COLOCATION_DISCOVERY_NETWORK_FAILED_META
  - XR_ERROR_COLOCATION_DISCOVERY_NO_DISCOVERY_METHOD_META
- Extending XrStructureType:
  - XR_TYPE_COLOCATION_ADVERTISEMENT_START_INFO_META
  - XR_TYPE_COLOCATION_ADVERTISEMENT_STOP_INFO_META
  - XR_TYPE_COLOCATION_DISCOVERY_START_INFO_META
  - XR_TYPE_COLOCATION_DISCOVERY_STOP_INFO_META
  - XR_TYPE_EVENT_DATA_COLOCATION_ADVERTISEMENT_COMPLETE_META
  - XR_TYPE_EVENT_DATA_COLOCATION_DISCOVERY_COMPLETE_META
  - XR_TYPE_EVENT_DATA_COLOCATION_DISCOVERY_RESULT_META
  - XR_TYPE_EVENT_DATA_START_COLOCATION_ADVERTISEMENT_COMPLETE_META
  - XR_TYPE_EVENT_DATA_START_COLOCATION_DISCOVERY_COMPLETE_META
  - XR_TYPE_EVENT_DATA_STOP_COLOCATION_ADVERTISEMENT_COMPLETE_META
  - XR_TYPE_EVENT_DATA_STOP_COLOCATION_DISCOVERY_COMPLETE_META
  - XR_TYPE_SYSTEM_COLOCATION_DISCOVERY_PROPERTIES_META

Version History

- Revision 1, 2024-06-15 (TJ Gilbrough)

- Initial extension description

12.148. XR_META_detached_controllers

Name String

`XR_META_detached_controllers`

Extension Type

Instance extension

Registered Extension Number

241

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Contributors

Matthew Langille, Meta Platforms
Andreas Selvik, Meta Platforms
John Kearney, Meta Platforms

12.148.1. Overview

On platforms which support both hand tracking and controller input concurrently, application developers have various use cases that require the ability to track controllers when they are not being held.

- Controllers that are "not held" can have some meaning in-application, for example the ability to act as some sort of marker or anchor.
- In cases when neither controller is currently held, access to pose data for the not-held controllers allows the application to render them for the user to locate, without removing the headset or remembering their physical location in the real world.
- Asymmetric multi-user applications could be created where the user with the immersive display is not holding the controller, but a second person manipulates it.
- In applications that support a tracked hand in combination with a single held controller, a not-held controller might remain relevant to interaction for the application.

A runtime **may** require the app to enable `XR_META_simultaneous_hands_and_controllers` in order to fully

enable the functionality exposed by this extension. If that extension is not enabled, the runtime **must** still accept the bindings, but **may** choose to not report any current interaction profile for */user/detached_controller_meta* paths and not provide action data accordingly.

12.148.2. New Paths

This extension defines two new top level */user* paths:

- */user/detached_controller_meta/left*
- */user/detached_controller_meta/right*

Interaction profiles are extended as follows:

- For any interaction profile valid for */user/hand/left* for which *.../input/grip/pose* is a valid binding sub-path, the runtime **must** accept the same input/output source subpaths on */user/detached_controller_meta/left* in suggested bindings.
- For any interaction profile valid for */user/hand/right* for which *.../input/grip/pose* is a valid binding sub-path, the runtime **must** accept the same input/output source subpaths on */user/detached_controller_meta/right* in suggested bindings.

12.148.3. Beginning Detached Controller Interaction

In-hand detection is left to the implementing runtime. The runtime **may** use proximity data, orientation data, or any other data for both hands and both controllers for in-hand detection. Controllers with handedness **should** be considered "in hand" **only** when held by their corresponding hand (a left controller held in the right hand **should** be considered detached, as should a right controller held in the left hand).

The same interaction profile **may** be used to suggest bindings for top level */user* paths beginning with both */user/hand* and */user/detached_controller_meta* paths, but the runtime **must** only report any given interaction profile as current for **at most one** of these paired top level */user* paths at any given time. That is, if a given interaction profile is reported as current for */user/detached_controller_meta/left*, that same profile **must** not be current for */user/hand/left* (and vice versa), and similarly for */user/detached_controller_meta/right* and */user/hand/right*.

This is done individually such that any combination of held or not-held controllers is representable (both or neither controller held, or one controller held in one hand for left or right controllers).

12.148.4. Ending Detached Controller Interaction

If an interaction profile previously current on a detached path becomes current on the corresponding hand, the current interaction profile of the detached path **must** become XR_NULL_PATH.

However, if a (controller) interaction profile becomes current on a "detached" top level */user* path, a hand-related interaction profile **should** become current on the corresponding */user/hand/* path if the application has submitted a suggested binding for the corresponding */user/hand/* top level path.

An application **may** use the XR_TYPE_EVENT_DATA_INTERACTION_PROFILE_CHANGED and change in output value from xrGetCurrentInteractionProfile to determine when the user put down or picked up a controller.

12.148.5. Changes to Action Space Behavior

In some cases, this extension modifies the behavior specified in Resolving a single action bound to multiple inputs or outputs. Specifically, given the following:

- A pose action with both a path under */user/detached_controller_meta* as a sub-action path, as well as the corresponding path under */user/hand*,
- Suggested bindings for that pose action for a single interaction profile on both a binding path under */user/detached_controller_meta/* path and the corresponding binding path under */user/hand/* (e.g. */user/detached_controller_meta/right/input/grip/pose* and */user/hand/right/input/grip/pose*),
- And an action space created for the pose action **without** specifying a sub-action path,

the runtime **should** associate the action space with the appropriate pose on the **controller**, whether it is in-hand or detached. That is, such an action space "follows" the controller. The core specification does not specify any particular behavior for action spaces created without a sub-action path on actions created with sub-action paths, other than not changing the association between calls to xrSyncActions.

A more common practice of defining a pose action with both */user/hand/left* and */user/hand/right* as sub-action paths does **not** change behavior with this extension. That is, if the application creates an action space from such a pose action, without passing a sub-action path to xrCreateActionSpace, the runtime **must** choose a single pose source. However, it usually does not make sense to create an action space without specifying a sub-action path for such an action.

12.148.6. Example Usage

Detecting hand/controller modality change per hand:

```
const int NUM_HANDS = 2; // previously defined
XrInstance instance;  // previously initialized
XrSession session;  // previously initialized
XrPath path;  // previously initialized for path in question
XrInteractionProfileState currentState[NUM_HANDS];  // previously initialized for
NUM_HANDS

// In event polling code

// Initialize an event buffer to hold the output.
XrEventDataBuffer event = {
    .type = XR_TYPE_EVENT_DATA_BUFFER,
};
XrResult result = xrPollEvent(instance, &event);
if (result == XR_SUCCESS) {

  // ...

  switch (event.type) {
    case XR_TYPE_EVENT_DATA_INTERACTION_PROFILE_CHANGED: {
      for (int handIndex = 0; handIndex < NUM_HANDS; ++handIndex) {
        XrInteractionProfileState profileState {
            .type = XR_TYPE_INTERACTION_PROFILE_STATE,
        };
        CHK_XR(xrGetCurrentInteractionProfile(session, path, &profileState));

        // Inspect profileState to understand if this hand is now holding a
        // controller, or if the controller has been placed somewhere
        if (currentState[handIndex].interactionProfile != profileState.
interactionProfile) {

            // Log state change, etc
            // LOG_OUT("Detected state change in {} from {} to {}", handIndex,
            //         currentState[handIndex].interactionProfile,
            //         profileState.interactionProfile);

            // Do whatever processing is required when this state has changed
        }
      }

      break;
    }
  }
}
```

Accessing data is effectively the same as existing hands and controllers input code, except that we no

longer make the (invalid) assumption that both */user/hand/left* and */user/hand/right* are the same modality.



Detecting controller locations when not in hands

Suggest interaction profile bindings for the use cases your app cares about. For an app that cares about detached controllers, you can set up bindings for either of the in-hand case or detached case, or both together.

For example, for many controllers you might suggest the following bindings with XrInteractionProfileSuggestedBinding:

- `LeftControllerPose` → */user/hand/left/grip/pose*
- `LeftControllerPose` → */user/detached_controller_meta/left/grip/pose*
- `LeftControllerDetachedOnly` → */user/detached_controller_meta/left/grip/pose*

```
XrSpace leftControllerSpace = XR_NULL_HANDLE;
XrSpace detachedLeftControllerSpace = XR_NULL_HANDLE;
XrActionSpaceCreateInfo asci{
    .type = XR_TYPE_ACTION_SPACE_CREATE_INFO,
    .next = nullptr,
    .action = LeftControllerPose,
};
CHK_XR(xrCreateActionSpace(session, &asci, &leftControllerSpace));
asci.action = LeftControllerDetachedOnly;
CHK_XR(xrCreateActionSpace(session, &asci, &
detachedLeftControllerSpace));

// Will follow controller whether in hand or detached
CHK_XR(xrLocateSpace(leftControllerSpace, /* ... */));

// Only valid data when controller is detached:
CHK_XR(xrLocateSpace(detachedLeftControllerSpace, /* ... */));

// Note that any bindings that work on /user/hand/left will work
// for /user/detached_controller_meta/left, and they can be
// combined by the app on an action to action basis.
```

Using the same controller pose when in hand as when detached with hands

The two extensions can be brought together to have a controller pose that follows the controller when detached, and a hand pose that flips from tracking the controller to tracking the hand when the controller is put down

For example, suggest bindings for */interaction_profiles/oculus/touch_controller* as follows with xrSuggestInteractionProfileBindings:

- `LeftControllerPose` → */user/hand/left/grip/pose*
- `LeftControllerPose` → */user/detached_controller_meta/left/grip/pose*
- `LeftHandPose` → */user/hand/left/grip/pose*



Further, suggest bindings for */interaction_profiles/ext/hand_interaction* as follows with xrSuggestInteractionProfileBindings:

- `LeftHandPose` → */user/hand/left/grip/pose*

Proceed with usage similar to above. When controllers are in hand, the binding source for */user/hand/left/grip/pose* path provides controller-based data to both `LeftControllerPose` and `LeftHandPose`.

When controllers are placed on a surface, the application will receive the `XR_TYPE_EVENT_DATA_INTERACTION_PROFILE_CHANGED` event. From this point on, `LeftHandPose` will be powered by hand data bound to */user/hand/left/grip/pose*, and the `LeftControllerPose` will still be powered by controller data bound to */user/detached_controller_meta/left/grip/pose*

Note that the same can be done for actions; `ButtonClickAction` could seamlessly transition from a controller trigger to a finger pinch when you put down the controllers, for example.

12.148.7. New Enum Constants

- `XR_META_DETACHED_CONTROLLERS_EXTENSION_NAME`
- `XR_META_detached_controllers_SPEC_VERSION`

12.148.8. Issues

12.148.9. Version History

- Revision 1, 2023-05-25 (Matthew Langille)
  - Initial extension description

12.149. XR_META_environment_depth

Name String

XR_META_environment_depth

Extension Type

Instance extension

Registered Extension Number

292

Revision

2

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Last Modified Date

2025-09-19

IP Status

No known IP claims.

Contributors

Andreas Selvik, Meta Platforms
Cass Everitt, Meta Platforms
Daniel Henell, Meta Platforms
John Kearney, Meta Platforms
Urs Niesen, Meta Platforms
Martin Sherburn, Meta Platforms

12.149.1. Overview

This extension allows the application to request depth maps of the real-world environment around the headset. The depth maps are generated by the runtime and shared with the application using an XrEnvironmentDepthSwapchainMETA.

12.149.2. Inspect System Capability

The XrSystemEnvironmentDepthPropertiesMETA structure is defined as:

```
// Provided by XR_META_environment_depth
typedef struct XrSystemEnvironmentDepthPropertiesMETA {
    XrStructureType    type;
    void*              next;
    XrBool32           supportsEnvironmentDepth;
    XrBool32           supportsHandRemoval;
} XrSystemEnvironmentDepthPropertiesMETA;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `supportsEnvironmentDepth` is an `XrBool32` indicating if current system supports environment depth.
- `supportsHandRemoval` is an `XrBool32` indicating if current system supports hand removal.

An application **can** inspect whether the system is capable of supporting environment depth by extending the XrSystemProperties with XrSystemEnvironmentDepthPropertiesMETA structure when calling xrGetSystemProperties.

If and only if a runtime returns `XR_FALSE` for `supportsEnvironmentDepth`, the runtime **must** return `XR_ERROR_FEATURE_UNSUPPORTED` from xrCreateEnvironmentDepthProviderMETA.

If and only if a runtime returns `XR_FALSE` for `supportsHandRemoval`, the runtime **must** return `XR_ERROR_FEATURE_UNSUPPORTED` from xrSetEnvironmentDepthHandRemovalMETA.

Valid Usage (Implicit)

- The `XR_META_environment_depth` extension **must** be enabled prior to using XrSystemEnvironmentDepthPropertiesMETA
- `type` **must** be `XR_TYPE_SYSTEM_ENVIRONMENT_DEPTH_PROPERTIES_META`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

12.149.3. Creating and Destroying a Depth Provider

```
// Provided by XR_META_environment_depth
XR_DEFINE_HANDLE(XrEnvironmentDepthProviderMETA)
```

An XrEnvironmentDepthProviderMETA is a handle to a depth provider.

The xrCreateEnvironmentDepthProviderMETA function is defined as:

```
// Provided by XR_META_environment_depth
XrResult xrCreateEnvironmentDepthProviderMETA(
    XrSession                                   session,
    const XrEnvironmentDepthProviderCreateInfoMETA* createInfo,
    XrEnvironmentDepthProviderMETA*             environmentDepthProvider);
```

Parameter Descriptions

- `session` is the XrSession.
- `createInfo` is a pointer to an XrEnvironmentDepthProviderCreateInfoMETA containing creation options for the depth provider.
- `environmentDepthProvider` is the returned XrEnvironmentDepthProviderMETA handle for the created depth provider.

The xrCreateEnvironmentDepthProviderMETA function creates a depth provider instance.

Creating the depth provider **may** allocate resources, but **should** not incur any per-frame compute costs until the provider has been started.

- Runtimes **must** create the provider in a stopped state.
- Runtimes **may** limit the number of depth providers per XrInstance. If xrCreateEnvironmentDepthProviderMETA fails due to reaching this limit, the runtime **must** return `XR_ERROR_LIMIT_REACHED`.
- Runtimes **must** support at least 1 provider per XrInstance.
- Runtimes **may** return `XR_ERROR_NOT_PERMITTED_PASSTHROUGH_FB` if the app permissions have not been granted to the calling app.
- Applications **can** call xrStartEnvironmentDepthProviderMETA to start the generation of depth maps.

Valid Usage (Implicit)

- The XR_META_environment_depth extension **must** be enabled prior to calling xrCreateEnvironmentDepthProviderMETA
- session **must** be a valid XrSession handle
- createInfo **must** be a pointer to a valid XrEnvironmentDepthProviderCreateInfoMETA structure
- environmentDepthProvider **must** be a pointer to an XrEnvironmentDepthProviderMETA handle

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_NOT_PERMITTED_PASSTHROUGH_FB
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The XrEnvironmentDepthProviderCreateInfoMETA structure is defined as:

```
// Provided by XR_META_environment_depth
typedef struct XrEnvironmentDepthProviderCreateInfoMETA {
    XrStructureType                               type;
    const void*                                   next;
    XrEnvironmentDepthProviderCreateFlagsMETA     createFlags;
} XrEnvironmentDepthProviderCreateInfoMETA;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `createFlags` is `0` or one or more XrEnvironmentDepthProviderCreateFlagBitsMETA.

The XrEnvironmentDepthProviderCreateInfoMETA structure provides creation options for the XrEnvironmentDepthProviderMETA when passed to xrCreateEnvironmentDepthProviderMETA.

Valid Usage (Implicit)

- The `XR_META_environment_depth` extension **must** be enabled prior to using XrEnvironmentDepthProviderCreateInfoMETA
- `type` **must** be `XR_TYPE_ENVIRONMENT_DEPTH_PROVIDER_CREATE_INFO_META`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `createFlags` **must** be `0`

The XrEnvironmentDepthProviderCreateFlagsMETA specifies creation options for XrEnvironmentDepthProviderMETA.

```
// Provided by XR_META_environment_depth
typedef XrFlags64 XrEnvironmentDepthProviderCreateFlagsMETA;
```

Valid bits for XrEnvironmentDepthProviderCreateFlagsMETA are defined by XrEnvironmentDepthProviderCreateFlagBitsMETA, which is specified as:

```
// Provided by XR_META_environment_depth
// Flag bits for XrEnvironmentDepthProviderCreateFlagsMETA
```

There are currently no flag bits defined. This is reserved for future use.

The xrDestroyEnvironmentDepthProviderMETA function is defined as:

```
// Provided by XR_META_environment_depth
XrResult xrDestroyEnvironmentDepthProviderMETA(
    XrEnvironmentDepthProviderMETA              environmentDepthProvider);
```

Parameter Descriptions

- `environmentDepthProvider` is an XrEnvironmentDepthProviderMETA handle for the depth provider.

The xrDestroyEnvironmentDepthProviderMETA function destroys the depth provider. After this call the runtime **may** free all related memory and resources.

Valid Usage (Implicit)

- The `XR_META_environment_depth` extension **must** be enabled prior to calling xrDestroyEnvironmentDepthProviderMETA
- `environmentDepthProvider` **must** be a valid XrEnvironmentDepthProviderMETA handle

Thread Safety

- Access to `environmentDepthProvider`, and any child handles, **must** be externally synchronized

Return Codes

Success

- `XR_SUCCESS`

Failure

- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_RUNTIME_FAILURE`

12.149.4. Starting and Stopping a Depth Provider

The xrStartEnvironmentDepthProviderMETA function is defined as:

```
// Provided by XR_META_environment_depth
XrResult xrStartEnvironmentDepthProviderMETA(
    XrEnvironmentDepthProviderMETA              environmentDepthProvider);
```

Parameter Descriptions

- `environmentDepthProvider` is an XrEnvironmentDepthProviderMETA handle for the depth provider.

The xrStartEnvironmentDepthProviderMETA function starts the asynchronous generation of depth maps.

Starting the depth provider **may** use CPU and GPU resources.

Runtimes **must** return `XR_ERROR_UNEXPECTED_STATE_PASSTHROUGH_FB` if xrStartEnvironmentDepthProviderMETA is called on an already started XrEnvironmentDepthProviderMETA.

Valid Usage (Implicit)

- The `XR_META_environment_depth` extension **must** be enabled prior to calling xrStartEnvironmentDepthProviderMETA
- `environmentDepthProvider` **must** be a valid XrEnvironmentDepthProviderMETA handle

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_UNEXPECTED_STATE_PASSTHROUGH_FB`

The xrStopEnvironmentDepthProviderMETA function is defined as:

```
// Provided by XR_META_environment_depth
XrResult xrStopEnvironmentDepthProviderMETA(
    XrEnvironmentDepthProviderMETA              environmentDepthProvider);
```

Parameter Descriptions

- `environmentDepthProvider` is an XrEnvironmentDepthProviderMETA handle for the depth provider.

The xrStopEnvironmentDepthProviderMETA function stops the generation of depth maps. This stops all per frame computation of environment depth for the application.

Runtimes **must** return `XR_ERROR_UNEXPECTED_STATE_PASSTHROUGH_FB` if xrStopEnvironmentDepthProviderMETA is called on an already stopped XrEnvironmentDepthProviderMETA.

Valid Usage (Implicit)

- The `XR_META_environment_depth` extension **must** be enabled prior to calling xrStopEnvironmentDepthProviderMETA
- `environmentDepthProvider` **must** be a valid XrEnvironmentDepthProviderMETA handle

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_UNEXPECTED_STATE_PASSTHROUGH_FB`

12.149.5. Hand Removal

Runtimes **may** provide functionality to remove hands from the depth map and filling in estimated background depth values. This is useful to support other occlusion methods specialized for hands to coexist with the Environment Depth extension.

The xrSetEnvironmentDepthHandRemovalMETA function is defined as:

```
// Provided by XR_META_environment_depth
XrResult xrSetEnvironmentDepthHandRemovalMETA(
    XrEnvironmentDepthProviderMETA              environmentDepthProvider,
    const XrEnvironmentDepthHandRemovalSetInfoMETA* setInfo);
```

Parameter Descriptions

- `environmentDepthProvider` is an XrEnvironmentDepthProviderMETA handle for the depth provider.
- `setInfo` is a pointer to an XrEnvironmentDepthHandRemovalSetInfoMETA containing options for the hand removal.

The xrSetEnvironmentDepthHandRemovalMETA function sets hand removal options.

Runtimes **should** enable or disable the removal of the hand depths from the depth map. If enabled, the corresponding depth pixels **should** be replaced with the estimated background depth behind the hands. Runtimes **must** return `XR_ERROR_FEATURE_UNSUPPORTED` if and only if XrSystemEnvironmentDepthPropertiesMETA::`supportsHandRemoval` is `XR_FALSE`.

Valid Usage (Implicit)

- The `XR_META_environment_depth` extension **must** be enabled prior to calling xrSetEnvironmentDepthHandRemovalMETA
- `environmentDepthProvider` **must** be a valid XrEnvironmentDepthProviderMETA handle
- `setInfo` **must** be a pointer to a valid XrEnvironmentDepthHandRemovalSetInfoMETA structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FEATURE_UNSUPPORTED
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The XrEnvironmentDepthHandRemovalSetInfoMETA structure is defined as:

```
// Provided by XR_META_environment_depth
typedef struct XrEnvironmentDepthHandRemovalSetInfoMETA {
    XrStructureType    type;
    const void*        next;
    XrBool32           enabled;
} XrEnvironmentDepthHandRemovalSetInfoMETA;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- enabled is XR_TRUE or XR_FALSE to enable/disable hand removal from the depth map, respectively.

This structure contains options passed to xrSetEnvironmentDepthHandRemovalMETA.

Valid Usage (Implicit)

- The XR_META_environment_depth extension **must** be enabled prior to using XrEnvironmentDepthHandRemovalSetInfoMETA
- type **must** be XR_TYPE_ENVIRONMENT_DEPTH_HAND_REMOVAL_SET_INFO_META
- next **must** be NULL or a valid pointer to the next structure in a structure chain

12.149.6. Creating a Readable Depth Swapchain

The depth data is generated in the runtime and shared to the application though an XrEnvironmentDepthSwapchainMETA. This swapchain is different from regular swapchains in that it provides a data channel from the runtime to the application instead of the other way around.

```
// Provided by XR_META_environment_depth
XR_DEFINE_HANDLE(XrEnvironmentDepthSwapchainMETA)
```

XrEnvironmentDepthSwapchainMETA is a handle to a readable depth swapchain.

The xrCreateEnvironmentDepthSwapchainMETA function is defined as:

```
// Provided by XR_META_environment_depth
XrResult xrCreateEnvironmentDepthSwapchainMETA(
    XrEnvironmentDepthProviderMETA              environmentDepthProvider,
    const XrEnvironmentDepthSwapchainCreateInfoMETA* createInfo,
    XrEnvironmentDepthSwapchainMETA*            swapchain);
```

Parameter Descriptions

- environmentDepthProvider is an XrEnvironmentDepthProviderMETA handle for the depth provider.
- createInfo is a pointer to an XrEnvironmentDepthSwapchainCreateInfoMETA containing creation options for the swapchain.
- swapchain is the returned XrEnvironmentDepthSwapchainMETA handle for the created swapchain.

The xrCreateEnvironmentDepthSwapchainMETA function creates a readable swapchain, which is used for accessing the depth data.

The runtime decides on the resolution and length of the swapchain. Additional information about the swapchain **can** be accessed by calling xrGetEnvironmentDepthSwapchainStateMETA.

Runtimes **must** create a swapchain with array textures of length 2, which map to a left-eye and right-eye view. View index 0 **must** represent the left eye and view index 1 **must** represent the right eye. This is the same convention as for `XR_VIEW_CONFIGURATION_TYPE_PRIMARY_STEREO` in XrViewConfigurationType. Runtimes **must** create the swapchain with the following image formats depending on the graphics API associated with the session:

- OpenGL: `GL_DEPTH_COMPONENT16`
- Vulkan: `VK_FORMAT_D16_UNORM`
- Direct3D: `DXGI_FORMAT_D16_UNORM`

Runtimes **must** only allow maximum one swapchain to exist per depth provider at any given time, and must return `XR_ERROR_LIMIT_REACHED` if xrCreateEnvironmentDepthSwapchainMETA is called to create more. Applications **should** destroy the swapchain when no longer needed. Applications **must** be able to handle different swapchain lengths and resolutions.

Valid Usage (Implicit)

- The `XR_META_environment_depth` extension **must** be enabled prior to calling xrCreateEnvironmentDepthSwapchainMETA
- `environmentDepthProvider` **must** be a valid XrEnvironmentDepthProviderMETA handle
- `createInfo` **must** be a pointer to a valid XrEnvironmentDepthSwapchainCreateInfoMETA structure
- `swapchain` **must** be a pointer to an XrEnvironmentDepthSwapchainMETA handle

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The XrEnvironmentDepthSwapchainCreateInfoMETA structure is defined as:

```
// Provided by XR_META_environment_depth
typedef struct XrEnvironmentDepthSwapchainCreateInfoMETA {
    XrStructureType                                type;
    const void*                                    next;
    XrEnvironmentDepthSwapchainCreateFlagsMETA     createFlags;
} XrEnvironmentDepthSwapchainCreateInfoMETA;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- createFlags is a bitmask of XrEnvironmentDepthSwapchainCreateFlagBitsMETA.

XrEnvironmentDepthSwapchainCreateInfoMETA contains creation options for the readable depth swapchain, and is passed to xrCreateEnvironmentDepthSwapchainMETA.

Valid Usage (Implicit)

- The XR_META_environment_depth extension **must** be enabled prior to using XrEnvironmentDepthSwapchainCreateInfoMETA
- type **must** be XR_TYPE_ENVIRONMENT_DEPTH_SWAPCHAIN_CREATE_INFO_META
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- createFlags **must** be 0

The XrEnvironmentDepthSwapchainCreateFlagsMETA specifies creation options for XrEnvironmentDepthSwapchainCreateInfoMETA.

```
// Provided by XR_META_environment_depth
typedef XrFlags64 XrEnvironmentDepthSwapchainCreateFlagsMETA;
```

Valid bits for XrEnvironmentDepthProviderCreateFlagsMETA are defined by XrEnvironmentDepthSwapchainCreateFlagBitsMETA, which is specified as:

```
// Provided by XR_META_environment_depth
// Flag bits for XrEnvironmentDepthSwapchainCreateFlagsMETA
```

There are currently no flag bits defined. This is reserved for future use.

The xrGetEnvironmentDepthSwapchainStateMETA function is defined as:

```
// Provided by XR_META_environment_depth
XrResult xrGetEnvironmentDepthSwapchainStateMETA(
    XrEnvironmentDepthSwapchainMETA             swapchain,
    XrEnvironmentDepthSwapchainStateMETA*       state);
```

Parameter Descriptions

- swapchain is an XrEnvironmentDepthSwapchainMETA handle.
- state is a pointer to an XrEnvironmentDepthSwapchainStateMETA.

xrGetEnvironmentDepthSwapchainStateMETA retrieves information about the XrEnvironmentDepthSwapchainMETA. This information is constant throughout the lifetime of the XrEnvironmentDepthSwapchainMETA.

Valid Usage (Implicit)

- The XR_META_environment_depth extension **must** be enabled prior to calling xrGetEnvironmentDepthSwapchainStateMETA
- `swapchain` **must** be a valid XrEnvironmentDepthSwapchainMETA handle
- `state` **must** be a pointer to an XrEnvironmentDepthSwapchainStateMETA structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The XrEnvironmentDepthSwapchainStateMETA structure is defined as:

```
// Provided by XR_META_environment_depth
typedef struct XrEnvironmentDepthSwapchainStateMETA {
    XrStructureType    type;
    void*              next;
    uint32_t           width;
    uint32_t           height;
} XrEnvironmentDepthSwapchainStateMETA;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `width` is the width of the image.
- `height` is the height of the image.

Valid Usage (Implicit)

- The `XR_META_environment_depth` extension **must** be enabled prior to using XrEnvironmentDepthSwapchainStateMETA
- `type` **must** be `XR_TYPE_ENVIRONMENT_DEPTH_SWAPCHAIN_STATE_META`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

The xrDestroyEnvironmentDepthSwapchainMETA function is defined as:

```
// Provided by XR_META_environment_depth
XrResult xrDestroyEnvironmentDepthSwapchainMETA(
    XrEnvironmentDepthSwapchainMETA             swapchain);
```

Parameter Descriptions

- `swapchain` is the XrEnvironmentDepthSwapchainMETA to be destroyed.

The xrDestroyEnvironmentDepthSwapchainMETA function destroys a readable environment depth swapchain.

All submitted graphics API commands that refer to `swapchain` **must** have completed execution. Runtimes **may** continue to utilize swapchain images after xrDestroyEnvironmentDepthSwapchainMETA is called.

Valid Usage (Implicit)

- The `XR_META_environment_depth` extension **must** be enabled prior to calling xrDestroyEnvironmentDepthSwapchainMETA
- `swapchain` **must** be a valid XrEnvironmentDepthSwapchainMETA handle

Thread Safety

- Access to `swapchain`, and any child handles, **must** be externally synchronized

Return Codes

Success

- `XR_SUCCESS`

Failure

- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_RUNTIME_FAILURE`

12.149.7. Accessing the Readable Depth Swapchain During Rendering

The xrEnumerateEnvironmentDepthSwapchainImagesMETA function is defined as:

```
// Provided by XR_META_environment_depth
XrResult xrEnumerateEnvironmentDepthSwapchainImagesMETA(
    XrEnvironmentDepthSwapchainMETA             swapchain,
    uint32_t                                    imageCapacityInput,
    uint32_t*                                   imageCountOutput,
    XrSwapchainImageBaseHeader*                 images);
```

Parameter Descriptions

- `swapchain` is the XrEnvironmentDepthSwapchainMETA to get images from.
- `imageCapacityInput` is the capacity of the images array, or 0 to indicate a request to retrieve the required capacity.
- `imageCountOutput` is a pointer to the count of images written, or a pointer to the required capacity in the case that `imageCapacityInput` is insufficient.
- `images` is a pointer to an array of graphics API-specific XrSwapchainImage structures, all of the same type, based on XrSwapchainImageBaseHeader. It **can** be `NULL` if `imageCapacityInput` is 0.
- See the Buffer Size Parameters section for a detailed description of retrieving the required `images` size.

xrEnumerateEnvironmentDepthSwapchainImagesMETA fills an array of graphics API-specific XrSwapchainImage* structures derived from XrSwapchainImageBaseHeader. The resources **must** be constant and valid for the lifetime of the XrEnvironmentDepthSwapchainMETA. This function behaves analogously to xrEnumerateSwapchainImages.

Runtimes **must** always return identical buffer contents from this enumeration for the lifetime of the swapchain.

Note: images is a pointer to an array of structures of graphics API-specific type, not an array of structure pointers.

The pointer submitted as images will be treated as an array of the expected graphics API-specific type based on the graphics API used at session creation time. If the type member of any array element accessed in this way does not match the expected value, the runtime **must** return XR_ERROR_VALIDATION_FAILURE.

Valid Usage (Implicit)

- The XR_META_environment_depth extension **must** be enabled prior to calling xrEnumerateEnvironmentDepthSwapchainImagesMETA
- swapchain **must** be a valid XrEnvironmentDepthSwapchainMETA handle
- imageCountOutput **must** be a pointer to a uint32_t value
- If imageCapacityInput is not 0, images **must** be a pointer to an array of imageCapacityInput XrSwapchainImageBaseHeader-based structures. See also: XrSwapchainImageD3D11KHR, XrSwapchainImageD3D12KHR, XrSwapchainImageMetalKHR, XrSwapchainImageOpenGLESKHR, XrSwapchainImageOpenGLKHR, XrSwapchainImageVulkanKHR

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_SIZE_INSUFFICIENT
- XR_ERROR_VALIDATION_FAILURE

The xrAcquireEnvironmentDepthImageMETA function is defined as:

```
// Provided by XR_META_environment_depth
XrResult xrAcquireEnvironmentDepthImageMETA(
    XrEnvironmentDepthProviderMETA              environmentDepthProvider,
    const XrEnvironmentDepthImageAcquireInfoMETA* acquireInfo,
    XrEnvironmentDepthImageMETA*                environmentDepthImage);
```

Parameter Descriptions

- environmentDepthProvider is an XrEnvironmentDepthProviderMETA handle for the depth provider.
- acquireInfo is an XrEnvironmentDepthImageAcquireInfoMETA containing parameters for populating a depth swapchain image.
- environmentDepthImage is the returned XrEnvironmentDepthImageMETA containing information about the acquired depth image.

Acquires the latest available swapchain image that has been generated by the depth provider and ensures it is ready to be accessed by the application. The application **may** access and queue GPU operations using the acquired image until the next xrEndFrame call, when the image is released and the depth provider **may** write new depth data into it after completion of all work queued before the xrEndFrame call.

The returned XrEnvironmentDepthImageMETA contains the swapchain index into the array enumerated by xrEnumerateEnvironmentDepthSwapchainImagesMETA. It also contains other information such as the field of view and pose that are necessary to interpret the depth data.

There **must** be no more than one call to xrAcquireEnvironmentDepthImageMETA between any pair of corresponding xrBeginFrame and xrEndFrame calls in a session.

- The runtime **may** block if previously acquired swapchain images are still being used by the graphics API.
- The runtime **must** return `XR_ERROR_CALL_ORDER_INVALID` if xrAcquireEnvironmentDepthImageMETA is called before xrBeginFrame or after xrEndFrame.
- The runtime **must** return `XR_ERROR_CALL_ORDER_INVALID` if xrAcquireEnvironmentDepthImageMETA is called on a stopped XrEnvironmentDepthProviderMETA.
- The runtime **must** return `XR_ERROR_LIMIT_REACHED` if xrAcquireEnvironmentDepthImageMETA is called more than once per frame - i.e. in a running session, after a call to xrBeginFrame that has not had an associated xrEndFrame.
- Runtimes **must** return `XR_ENVIRONMENT_DEPTH_NOT_AVAILABLE_META` if no depth frame is available yet (i.e. the provider was recently started and did not yet have time to compute depth). Note that this is a success code. In this case the output parameters **must** be unchanged.
- The application **must** not utilize the swapchain image in calls to the graphics API after xrEndFrame has been called.
- A runtime **may** use the graphics API specific contexts provided to OpenXR. In particular:
  - For OpenGL, a runtime **may** use the OpenGL context specified in the call to xrCreateSession, which needs external synchronization.
  - For Vulkan, a runtime **may** use the `VkQueue` specified in the XrGraphicsBindingVulkan2KHR, which needs external synchronization.
  - For Direct3D12, a runtime **may** use the `ID3D12CommandQueue` specified in the XrGraphicsBindingD3D12KHR, which needs external synchronization.

Valid Usage (Implicit)

- The `XR_META_environment_depth` extension **must** be enabled prior to calling xrAcquireEnvironmentDepthImageMETA
- `environmentDepthProvider` **must** be a valid XrEnvironmentDepthProviderMETA handle
- `acquireInfo` **must** be a pointer to a valid XrEnvironmentDepthImageAcquireInfoMETA structure
- `environmentDepthImage` **must** be a pointer to an XrEnvironmentDepthImageMETA structure

Return Codes

Success

- XR_SUCCESS
- XR_ENVIRONMENT_DEPTH_NOT_AVAILABLE_META
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_CALL_ORDER_INVALID
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_TIME_INVALID
- XR_ERROR_VALIDATION_FAILURE

The XrEnvironmentDepthImageAcquireInfoMETA structure is defined as:

```
// Provided by XR_META_environment_depth
typedef struct XrEnvironmentDepthImageAcquireInfoMETA {
    XrStructureType    type;
    const void*        next;
    XrSpace            space;
    XrTime             displayTime;
} XrEnvironmentDepthImageAcquireInfoMETA;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `space` is an XrSpace defining the reference frame of the returned pose in XrEnvironmentDepthImageMETA.
- `displayTime` is an `XrTime` specifying the time used to compute the pose for the returned pose in XrEnvironmentDepthImageMETA. Clients **should** pass their predicted display time for the current frame.

Valid Usage (Implicit)

- The `XR_META_environment_depth` extension **must** be enabled prior to using XrEnvironmentDepthImageAcquireInfoMETA
- `type` **must** be `XR_TYPE_ENVIRONMENT_DEPTH_IMAGE_ACQUIRE_INFO_META`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `space` **must** be a valid XrSpace handle

The XrEnvironmentDepthImageViewMETA structure is defined as:

```
// Provided by XR_META_environment_depth
typedef struct XrEnvironmentDepthImageViewMETA {
    XrStructureType    type;
    const void*        next;
    XrFovf             fov;
    XrPosef            pose;
} XrEnvironmentDepthImageViewMETA;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `fov` is an XrFovf specifying the field of view used to generate this view. The view is never flipped horizontally nor vertically.
- `pose` is an XrPosef specifying the pose from which the depth map was rendered. The reference frame is specified in XrEnvironmentDepthImageAcquireInfoMETA.

Valid Usage (Implicit)

- The `XR_META_environment_depth` extension **must** be enabled prior to using XrEnvironmentDepthImageViewMETA
- `type` **must** be `XR_TYPE_ENVIRONMENT_DEPTH_IMAGE_VIEW_META`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

The XrEnvironmentDepthImageMETA structure is defined as:

```
// Provided by XR_META_environment_depth
typedef struct XrEnvironmentDepthImageMETA {
    XrStructureType                   type;
    const void*                       next;
    uint32_t                          swapchainIndex;
    float                             nearZ;
    float                             farZ;
    XrEnvironmentDepthImageViewMETA   views[2];
} XrEnvironmentDepthImageMETA;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `swapchainIndex` is the index of the acquired texture in the depth swapchain.
- `nearZ` is the distance to the near Z plane in meters.
- `farZ` is the distance to the far Z plane in meters.
- `views` is an array of two XrEnvironmentDepthImageViewMETA, one for each eye, where index 0 is left eye and index 1 is the right eye.

Depth is provided as textures in the same format as described in the `XR_KHR_composition_layer_depth` extension.

The frustum's Z-planes are placed at `nearZ` and `farZ` meters. When `farZ` is less than `nearZ`, an infinite projection matrix is used.

Valid Usage (Implicit)

- The `XR_META_environment_depth` extension **must** be enabled prior to using XrEnvironmentDepthImageMETA
- `type` **must** be `XR_TYPE_ENVIRONMENT_DEPTH_IMAGE_META`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain. See also: XrEnvironmentDepthImageTimestampMETA
- Any given element of `views` **must** be a valid XrEnvironmentDepthImageViewMETA structure

The XrEnvironmentDepthImageTimestampMETA structure is defined as:

```
// Provided by XR_META_environment_depth
typedef struct XrEnvironmentDepthImageTimestampMETA {
    XrStructureType    type;
    const void*        next;
    XrTime             captureTime;
} XrEnvironmentDepthImageTimestampMETA;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `captureTime` is an `XrTime` specifying when the depth image was captured.

The XrEnvironmentDepthImageTimestampMETA structure provides timestamp information for environment depth data. This structure **can** be chained to XrEnvironmentDepthImageMETA to provide temporal context for the depth information.

The `captureTime` field indicates the capture time of the images used to reconstruct the environment depth map.

Applications **can** use this timestamp information for latency measurements and synchronization with other time-based data such as passthrough camera.

If present in the structure chain, this structure **must** be populated by the runtime if and only if the runtime reports version 2 or greater of this extension. This is an "unknown structure" to runtimes reporting version 1 of this extension, and is therefore ignored/unmodified according to Valid Usage for Structure Pointer Chains.

Applications **should** initialize `captureTime` to 0 before chaining this structure and calling xrAcquireEnvironmentDepthImageMETA. After the call, a non-zero value in `captureTime` indicates that the runtime has populated the timestamp field.

Valid Usage (Implicit)

- The `XR_META_environment_depth` extension **must** be enabled prior to using XrEnvironmentDepthImageTimestampMETA
- `type` **must** be `XR_TYPE_ENVIRONMENT_DEPTH_IMAGE_TIMESTAMP_META`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

12.149.8. Vulkan Swapchain Image Layout

For an application using Vulkan, after a successful call to xrAcquireEnvironmentDepthImageMETA that does **not** return `XR_ENVIRONMENT_DEPTH_NOT_AVAILABLE_META`, the following conditions apply to the **runtime**:

- The runtime **must** ensure the acquired readable depth swapchain image has a memory layout compatible with `VK_IMAGE_LAYOUT_SHADER_READ_ONLY_OPTIMAL`. **Note** that this is different from xrAcquireSwapchainImage which guarantees `VK_IMAGE_LAYOUT_DEPTH_STENCIL_ATTACHMENT_OPTIMAL`.

- The runtime **must** ensure the VkQueue specified in XrGraphicsBindingVulkanKHR / XrGraphicsBindingVulkan2KHR has ownership of the acquired readable depth swapchain image.

Upon next calling xrEndFrame after such an acquire call, the following conditions apply to the **application**:

- The application **must** ensure that the readable depth swapchain image has a memory layout compatible with VK_IMAGE_LAYOUT_SHADER_READ_ONLY_OPTIMAL.
- The application **must** ensure that the readable depth swapchain image is owned by the VkQueue specified in XrGraphicsBindingVulkanKHR / XrGraphicsBindingVulkan2KHR.

The application is responsible for transitioning the swapchain image back to the image layout and queue ownership that the OpenXR runtime requires. If the image is not in a layout compatible with the above specifications, the runtime **may** exhibit undefined behavior.

12.149.9. Direct3D 12 Swapchain Image Resource State

For an application using D3D12, after a successful call to xrAcquireEnvironmentDepthImageMETA that does **not** return XR_ENVIRONMENT_DEPTH_NOT_AVAILABLE_META, the following conditions apply to the **runtime**:

- The runtime **must** ensure the acquired readable depth swapchain image has a resource state match with D3D12_RESOURCE_STATE_ALL_SHADER_RESOURCE. **Note** that this is different from xrAcquireSwapchainImage which guarantees D3D12_RESOURCE_STATE_DEPTH_WRITE for swapchain images with depth formats.
- The runtime **must** ensure that the ID3D12CommandQueue specified in XrGraphicsBindingD3D12KHR **may** read from the acquired readable depth swapchain image.

Upon next calling xrEndFrame after such an acquire call, the following conditions apply to the **application**:

- The application **must** ensure that the readable depth swapchain image has a resource state match with D3D12_RESOURCE_STATE_ALL_SHADER_RESOURCE.
- The application **must** ensure that the readable depth swapchain image is available for read/write on the ID3D12CommandQueue specified in XrGraphicsBindingD3D12KHR.

The application is responsible for transitioning the swapchain image back to the resource state and queue availability that the OpenXR runtime requires. If the image is not in a resource state match with the above specifications the runtime **may** exhibit undefined behavior.

Version History

- Revision 1, 2023-08-24 (Daniel Henell)
  - Initial extension description
- Revision 2, 2025-09-19 (Martin Sherburn)

- Add support for accessing capture time stamps for depth images.

12.150. XR_META_environment_raycast

Name String

`XR_META_environment_raycast`

Extension Type

Instance extension

Registered Extension Number

593

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

`XR_EXT_future`

Last Modified Date

2025-03-01

IP Status

No known IP claims.

Contributors

Matt Saraceno, Meta
Po-Kang Huang, Meta
Zhentian Qian, Meta
Jiawen Zhang, Meta
Chengyuan Lin, Meta

12.150.1. Overview

The `XR_META_environment_raycast` extension allows applications to perform raycast operations against the surrounding physical environment.

Given a ray origin and a direction, this extension searches along the ray to determine where the ray first intersects with the environment. Optionally, the application applies filters to refine the search results.

The application **must** also enable the `XR_EXT_future` extension in order to use this extension. Otherwise

xrCreateInstance **must** return XR_ERROR_EXTENSION_DEPENDENCY_NOT_ENABLED.

12.150.2. Inspect system capability

The XrSystemEnvironmentRaycastPropertiesMETA structure is defined as:

```
// Provided by XR_META_environment_raycast
typedef struct XrSystemEnvironmentRaycastPropertiesMETA {
    XrStructureType    type;
    void*              next;
    XrBool32           supportsEnvironmentRaycast;
} XrSystemEnvironmentRaycastPropertiesMETA;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- supportsEnvironmentRaycast is an XrBool32, indicating if current system is capable of performing environment raycast.

An application **can** inspect whether the system is capable of performing environment raycasts by extending the XrSystemProperties with an XrSystemEnvironmentRaycastPropertiesMETA structure when calling xrGetSystemProperties.

If and only if a runtime returns XR_FALSE for supportsEnvironmentRaycast, the runtime **must** return XR_ERROR_FEATURE_UNSUPPORTED from xrCreateEnvironmentRaycasterAsyncMETA.

Valid Usage (Implicit)

- The XR_META_environment_raycast extension **must** be enabled prior to using XrSystemEnvironmentRaycastPropertiesMETA
- type **must** be XR_TYPE_SYSTEM_ENVIRONMENT_RAYCAST_PROPERTIES_META
- next **must** be NULL or a valid pointer to the next structure in a structure chain

12.150.3. Create an environment raycaster

The XrEnvironmentRaycasterMETA handle represents the resources used to perform environment raycast operations. It is valid until destroyed via xrDestroyEnvironmentRaycasterMETA.

```
// Provided by XR_META_environment_raycast
XR_DEFINE_HANDLE(XrEnvironmentRaycasterMETA)
```

The runtime **must** have an active environment raycaster handle created by the application before providing any raycast hit info.

The xrCreateEnvironmentRaycasterAsyncMETA function is defined as:

```
// Provided by XR_META_environment_raycast
XrResult xrCreateEnvironmentRaycasterAsyncMETA(
    XrSession                                   session,
    const XrEnvironmentRaycasterCreateInfoMETA* info,
    XrFutureEXT*                                future);
```

Parameter Descriptions

- `session` is an XrSession in which the environment raycaster will be active.
- `info` is the XrEnvironmentRaycasterCreateInfoMETA used to create the raycaster.
- `future` is a pointer to an `XrFutureEXT`.

This function starts an asynchronous operation and creates a corresponding `XrFutureEXT`, usable with xrPollFutureEXT and related functions. The return value of this function only indicates whether the parameters were acceptable to schedule the asynchronous operation. The corresponding completion function is xrCreateEnvironmentRaycasterCompleteMETA, usable when a future from this function is in the READY state, with outputs populated by that function in the completion structure XrEnvironmentRaycasterCreateCompletionMETA.

Runtimes **may** initialize needed resources to perform raycast operations during this asynchronous call.

If the system does not support environment raycast operations, the runtime **must** return `XR_ERROR_FEATURE_UNSUPPORTED` from xrCreateEnvironmentRaycasterAsyncMETA. In this case, the runtime **must** return `XR_FALSE` for XrSystemEnvironmentRaycastPropertiesMETA ::`supportsEnvironmentRaycast` when the function xrGetSystemProperties is called, so that the application **can** avoid attempting to create an environment raycaster.

Valid Usage (Implicit)

- The `XR_META_environment_raycast` extension **must** be enabled prior to calling xrCreateEnvironmentRaycasterAsyncMETA
- `session` **must** be a valid XrSession handle
- If `info` is not `NULL`, `info` **must** be a pointer to a valid XrEnvironmentRaycasterCreateInfoMETA structure
- `future` **must** be a pointer to an `XrFutureEXT` value

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_FEATURE_UNSUPPORTED`
- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_LIMIT_REACHED`
- `XR_ERROR_OUT_OF_MEMORY`
- `XR_ERROR_PERMISSION_INSUFFICIENT`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_VALIDATION_FAILURE`

The XrEnvironmentRaycasterCreateInfoMETA structure is defined as:

```
// Provided by XR_META_environment_raycast
typedef struct XrEnvironmentRaycasterCreateInfoMETA {
    XrStructureType    type;
    const void*        next;
} XrEnvironmentRaycasterCreateInfoMETA;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.

The XrEnvironmentRaycasterCreateInfoMETA structure describes the information to create an environment raycaster and respective XrEnvironmentRaycasterMETA handle.

Valid Usage (Implicit)

- The `XR_META_environment_raycast` extension **must** be enabled prior to using XrEnvironmentRaycasterCreateInfoMETA
- `type` **must** be `XR_TYPE_ENVIRONMENT_RAYCASTER_CREATE_INFO_META`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

The xrCreateEnvironmentRaycasterCompleteMETA function is defined as:

```
// Provided by XR_META_environment_raycast
XrResult xrCreateEnvironmentRaycasterCompleteMETA(
    XrSession                                   session,
    XrFutureEXT                                 future,
    XrEnvironmentRaycasterCreateCompletionMETA* completion);
```

Parameter Descriptions

- `session` is an XrSession in which the environment raycaster will be active.
- `future` is the `XrFutureEXT` received from xrCreateEnvironmentRaycasterAsyncMETA.
- `completion` is a pointer to an XrEnvironmentRaycasterCreateCompletionMETA.

This is the completion function corresponding to xrCreateEnvironmentRaycasterAsyncMETA. It completes the asynchronous operation and returns the results. Do not call until the `future` is READY.

The runtime **must** return `XR_ERROR_FUTURE_PENDING_EXT` if `future` is not in the READY state. The runtime **must** return `XR_ERROR_FUTURE_INVALID_EXT` if `future` has already been completed or cancelled.

Valid Usage (Implicit)

- The `XR_META_environment_raycast` extension **must** be enabled prior to calling xrCreateEnvironmentRaycasterCompleteMETA
- `session` **must** be a valid XrSession handle
- `completion` **must** be a pointer to an XrEnvironmentRaycasterCreateCompletionMETA structure

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_FEATURE_UNSUPPORTED`
- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_FUTURE_INVALID_EXT`
- `XR_ERROR_FUTURE_PENDING_EXT`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INITIALIZATION_FAILED`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_LIMIT_REACHED`
- `XR_ERROR_OUT_OF_MEMORY`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_SESSION_NOT_RUNNING`
- `XR_ERROR_VALIDATION_FAILURE`

The XrEnvironmentRaycasterCreateCompletionMETA structure is defined as:

```
// Provided by XR_META_environment_raycast
typedef struct XrEnvironmentRaycasterCreateCompletionMETA {
    XrStructureType               type;
    void*                         next;
    XrResult                      futureResult;
    XrEnvironmentRaycasterMETA    environmentRaycaster;
} XrEnvironmentRaycasterCreateCompletionMETA;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `futureResult` is the XrResult of the environment raycaster creation operation.
- `environmentRaycaster` is the returned XrEnvironmentRaycasterMETA handle.

Future Return Codes

XrEnvironmentRaycasterCreateCompletionMETA::`futureResult` values:

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_OUT_OF_MEMORY`
- `XR_ERROR_FUNCTION_UNSUPPORTED`

The XrEnvironmentRaycasterCreateCompletionMETA structure stores the result of the environment raycaster creation.

Valid Usage (Implicit)

- The XR_META_environment_raycast extension **must** be enabled prior to using XrEnvironmentRaycasterCreateCompletionMETA
- type **must** be XR_TYPE_ENVIRONMENT_RAYCASTER_CREATE_COMPLETION_META
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- futureResult **must** be a valid XrResult value
- environmentRaycaster **must** be a valid XrEnvironmentRaycasterMETA handle

12.150.4. Performing an environment raycast operation

The xrPerformEnvironmentRaycastMETA function is defined as:

```
// Provided by XR_META_environment_raycast
XrResult xrPerformEnvironmentRaycastMETA(
    XrEnvironmentRaycasterMETA                environmentRaycaster,
    const XrEnvironmentRaycastHitGetInfoMETA*  info,
    XrEnvironmentRaycastHitMETA*               hitPoint);
```

Parameter Descriptions

- environmentRaycaster is an XrEnvironmentRaycasterMETA previously created by xrCreateEnvironmentRaycasterAsyncMETA.
- info is a pointer to an XrEnvironmentRaycastHitGetInfoMETA describing the ray to cast.
- hitPoint is a pointer to XrEnvironmentRaycastHitMETA that the runtime fills with information about where the closest hit is, if found.

The xrPerformEnvironmentRaycastMETA function is used by the application to perform an environment raycast operation.

Valid Usage (Implicit)

- The XR_META_environment_raycast extension **must** be enabled prior to calling xrPerformEnvironmentRaycastMETA
- environmentRaycaster **must** be a valid XrEnvironmentRaycasterMETA handle
- info **must** be a pointer to a valid XrEnvironmentRaycastHitGetInfoMETA structure
- hitPoint **must** be a pointer to an XrEnvironmentRaycastHitMETA structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FEATURE_UNSUPPORTED
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_TIME_INVALID
- XR_ERROR_VALIDATION_FAILURE

The XrEnvironmentRaycastHitGetInfoMETA structure is defined as:

```
// Provided by XR_META_environment_raycast
typedef struct XrEnvironmentRaycastHitGetInfoMETA {
    XrStructureType                                         type;
    const void*                                             next;
    XrSpace                                                 baseSpace;
    XrTime                                                  time;
    XrVector3f                                              origin;
    XrVector3f                                              direction;
    uint32_t                                                filterCount;
    const XrEnvironmentRaycastFilterBaseHeaderMETA* const *    filters;
} XrEnvironmentRaycastHitGetInfoMETA;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `baseSpace` is the XrSpace in which all the following locations in this struct will be located in.
- `time` is the `XrTime` at which all the following locations in this struct will be located.
- `origin` is an XrVector3f which describes the starting point for the ray.
- `direction` is an XrVector3f which describes the direction of the ray.
- `filterCount` is the number of filters that are passed in the `filters` parameter.
- `filters` is a pointer to an array of XrEnvironmentRaycastFilterBaseHeaderMETA pointers.

The XrEnvironmentRaycastHitGetInfoMETA structure describes a ray to cast.

Valid Usage (Implicit)

- The `XR_META_environment_raycast` extension **must** be enabled prior to using XrEnvironmentRaycastHitGetInfoMETA
- `type` **must** be `XR_TYPE_ENVIRONMENT_RAYCAST_HIT_GET_INFO_META`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `baseSpace` **must** be a valid XrSpace handle
- If `filterCount` is not `0`, `filters` **must** be a pointer to an array of `filterCount` valid XrEnvironmentRaycastFilterBaseHeaderMETA-based structures. See also: XrEnvironmentRaycastFilterDistanceMETA

The XrEnvironmentRaycastFilterBaseHeaderMETA structure is defined as:

```
// Provided by XR_META_environment_raycast
typedef struct XrEnvironmentRaycastFilterBaseHeaderMETA {
    XrStructureType    type;
    const void*        next;
} XrEnvironmentRaycastFilterBaseHeaderMETA;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` **must** be `NULL`

The XrEnvironmentRaycastFilterBaseHeaderMETA is meant to be used as the base header for filter types defined in this extension.

Valid Usage (Implicit)

- The `XR_META_environment_raycast` extension **must** be enabled prior to using XrEnvironmentRaycastFilterBaseHeaderMETA
- `type` **must** be `XR_TYPE_ENVIRONMENT_RAYCAST_FILTER_DISTANCE_META`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

One such filter type is defined in this extension: XrEnvironmentRaycastFilterDistanceMETA.

The XrEnvironmentRaycastFilterDistanceMETA structure is defined as:

```
// Provided by XR_META_environment_raycast
typedef struct XrEnvironmentRaycastFilterDistanceMETA {
    XrStructureType    type;
    const void*        next;
    float              maxDistance;
} XrEnvironmentRaycastFilterDistanceMETA;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` **must** be `NULL`
- `maxDistance` is the distance in meters to constrain the length of the ray from XrEnvironmentRaycastHitGetInfoMETA::`origin` along XrEnvironmentRaycastHitGetInfoMETA::`direction`.

The XrEnvironmentRaycastFilterDistanceMETA structure contains distance filter information for environment raycast.

If the application provides an XrEnvironmentRaycastFilterDistanceMETA, the runtime **must** constrain the length of the ray from XrEnvironmentRaycastHitGetInfoMETA::`origin` along

XrEnvironmentRaycastHitGetInfoMETA::direction to maxDistance in meters. This means that the runtime **must** not return any hitpoint that is more than XrEnvironmentRaycastFilterDistanceMETA ::maxDistance away from XrEnvironmentRaycastHitGetInfoMETA::origin.

Valid Usage (Implicit)

- The XR_META_environment_raycast extension **must** be enabled prior to using XrEnvironmentRaycastFilterDistanceMETA
- type **must** be XR_TYPE_ENVIRONMENT_RAYCAST_FILTER_DISTANCE_META
- next **must** be NULL or a valid pointer to the next structure in a structure chain

The XrEnvironmentRaycastHitMETA structure is defined as:

```
// Provided by XR_META_environment_raycast
typedef struct XrEnvironmentRaycastHitMETA {
    XrStructureType                     type;
    const void*                         next;
    XrEnvironmentRaycastHitStatusMETA   status;
    XrPosef                             pose;
} XrEnvironmentRaycastHitMETA;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next **must** be NULL
- status provides more info about whether a hit point was found, or why it was not found.
- pose is the XrPosef of the hit point in XrEnvironmentRaycastHitGetInfoMETA::baseSpace, if found.

The XrEnvironmentRaycastHitMETA contains information about where the closest hit was found.

The runtime **must** set the XrPosef::position of the pose to the point where the ray specified in XrEnvironmentRaycastHitGetInfoMETA intersected with the environment. If the ray intersects with the environment at multiple points, XrPosef::position **must** be set to the first point of intersection when traveling from XrEnvironmentRaycastHitGetInfoMETA::origin in the XrEnvironmentRaycastHitGetInfoMETA::direction direction.

The XrPosef::orientation of the pose **must** represent the best estimate of the of the normal of the surface hit by the ray. The normal vector is calculated by rotating {0, 0, 1} by this orientation. Rotating the vector {0, 1, 0} with XrPosef::orientation **must** result in a vector that roughly points in the direction of the ray origin.

Valid Usage (Implicit)

- The XR_META_environment_raycast extension **must** be enabled prior to using XrEnvironmentRaycastHitMETA
- type **must** be XR_TYPE_ENVIRONMENT_RAYCAST_HIT_META
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- status **must** be a valid XrEnvironmentRaycastHitStatusMETA value

The XrEnvironmentRaycastHitStatusMETA enumerates the possible status of an attempted environment raycast. XR_ENVIRONMENT_RAYCAST_HIT_STATUS_HIT_META indicates a successful environment raycast with a valid hit point found given the criteria specified in XrEnvironmentRaycastHitGetInfoMETA.

```
// Provided by XR_META_environment_raycast
typedef enum XrEnvironmentRaycastHitStatusMETA {
    XR_ENVIRONMENT_RAYCAST_HIT_STATUS_HIT_META = 1,
    XR_ENVIRONMENT_RAYCAST_HIT_STATUS_NO_HIT_META = 2,
    XR_ENVIRONMENT_RAYCAST_HIT_STATUS_HIT_POINT_OCCLUDED_META = 3,
    XR_ENVIRONMENT_RAYCAST_HIT_STATUS_HIT_POINT_OUTSIDE_OF_FOV_META = 4,
    XR_ENVIRONMENT_RAYCAST_HIT_STATUS_RAY_OCCLUDED_META = 5,
    XR_ENVIRONMENT_RAYCAST_HIT_STATUS_HIT_INVALID_ORIENTATION_META = 6,
    XR_ENVIRONMENT_RAYCAST_HIT_STATUS_MAX_ENUM_META = 0x7FFFFFFF
} XrEnvironmentRaycastHitStatusMETA;
```

Enum	Description
XR_ENVIRONMENT_RAYCAST_HIT_STATUS_HIT_META	The intersection with the environment is found.
XR_ENVIRONMENT_RAYCAST_HIT_STATUS_NO_HIT_META	The intersection with the environment is not found.
XR_ENVIRONMENT_RAYCAST_HIT_STATUS_HIT_POINT_OCCLUDED_META	The ray intersects with the environment, but the actual hit point is invisible.
XR_ENVIRONMENT_RAYCAST_HIT_STATUS_HIT_POINT_OUTSIDE_OF_FOV_META	The hit point can't be determined because it lies outside of the field of view.
XR_ENVIRONMENT_RAYCAST_HIT_STATUS_RAY_OCCLUDED_META	The ray is completely occluded by the environment.
XR_ENVIRONMENT_RAYCAST_HIT_STATUS_HIT_INVALID_ORIENTATION_META	The intersection with the environment is found, but the hit orientation cannot be determined.

XR_ENVIRONMENT_RAYCAST_HIT_STATUS_NO_HIT_META indicates a failed environment raycast, and none of the pose fields are valid. XR_ENVIRONMENT_RAYCAST_HIT_STATUS_HIT_POINT_OCCLUDED_META indicates the hit point

is occluded. For example, if you are looking at a screen, the hit point may be on the wall behind it. The pose is still valid and represents the best estimate of the hit point, but it may not be accurate. XR_ENVIRONMENT_RAYCAST_HIT_STATUS_HIT_POINT_OUTSIDE_OF_FOV_META indicates the hit point is outside camera's field of view. The pose is not valid, the application **should** not use it. XR_ENVIRONMENT_RAYCAST_HIT_STATUS_RAY_OCCLUDED_META indicates the input ray is occluded. The pose is not valid, the application **should** not use it. XR_ENVIRONMENT_RAYCAST_HIT_STATUS_HIT_INVALID_ORIENTATION_META indicates a partially successful environment raycast that only the XrPosef::position was determined, but the XrPosef::orientation was unable to be determined, it contains undefined values and the application **should** not use it.

When necessary, the application can instruct users to move to a different viewing point to achieve more accurate raycast results.

12.150.5. Destroy an environment raycaster handle

The xrDestroyEnvironmentRaycasterMETA function is defined as:

```
// Provided by XR_META_environment_raycast
XrResult xrDestroyEnvironmentRaycasterMETA(
    XrEnvironmentRaycasterMETA                  environmentRaycaster);
```

Parameter Descriptions

- environmentRaycaster is an XrEnvironmentRaycasterMETA previously created by xrCreateEnvironmentRaycasterAsyncMETA.

The xrDestroyEnvironmentRaycasterMETA function releases the environmentRaycaster and frees up associated system resources.

Valid Usage (Implicit)

- The XR_META_environment_raycast extension **must** be enabled prior to calling xrDestroyEnvironmentRaycasterMETA
- environmentRaycaster **must** be a valid XrEnvironmentRaycasterMETA handle

Thread Safety

- Access to environmentRaycaster, and any child handles, **must** be externally synchronized

Return Codes

Success

- XR_SUCCESS

Failure

- XR_ERROR_FEATURE_UNSUPPORTED
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_RUNTIME_FAILURE

12.150.6. Example Code

```
XrInstance instance; // previously initialized
XrSystemId systemId; // previously initialized
XrSession session; // previously initialized

// The function pointers are previously initialized using xrGetInstanceProcAddr
PFN_xrCreateEnvironmentRaycasterAsyncMETA xrCreateEnvironmentRaycasterAsyncMETA;
PFN_xrCreateEnvironmentRaycasterCompleteMETA xrCreateEnvironmentRaycasterCompleteMETA;
PFN_xrDestroyEnvironmentRaycasterMETA xrDestroyEnvironmentRaycasterMETA;
PFN_xrPerformEnvironmentRaycastMETA xrPerformEnvironmentRaycastMETA;
PFN_xrPollFutureEXT xrPollFutureEXT;

// Check device support
XrSystemEnvironmentRaycastPropertiesMETA environmentRaycastProperties{
    .type = XR_TYPE_SYSTEM_ENVIRONMENT_RAYCAST_PROPERTIES_META
};
XrSystemProperties systemProperties{
    .type = XR_TYPE_SYSTEM_PROPERTIES,
    .next = &environmentRaycastProperties,
};
CHK_XR(xrGetSystemProperties(instance, systemId, &systemProperties));
if (environmentRaycastProperties.supportsEnvironmentRaycast == XR_FALSE) {
    return;
}

// Create a new environment raycaster
XrFutureEXT createFuture{XR_NULL_HANDLE};
XrEnvironmentRaycasterCreateInfoMETA createInfo = {
    .type = XR_TYPE_ENVIRONMENT_RAYCASTER_CREATE_INFO_META,
};
CHK_XR(xrCreateEnvironmentRaycasterAsyncMETA(session, &createInfo, &createFuture));
```

```
// Wait for the future to be ready
bool futureReady = false;
while (!futureReady) {
    XrFuturePollInfoEXT pollInfo{
        .type = XR_TYPE_FUTURE_POLL_INFO_EXT,
        .future = createFuture,
    };
    XrFuturePollResultEXT pollResult{
        .type = XR_TYPE_FUTURE_POLL_RESULT_EXT,
    };
    CHK_XR(xrPollFutureEXT(instance, &pollInfo, &pollResult));

    if (pollResult.state == XR_FUTURE_STATE_READY_EXT) {
        futureReady = true;
    } else {
        // sleep(10);
    }
}

// Complete future and get creation result
XrEnvironmentRaycasterCreateCompletionMETA completion = {
    .type = XR_TYPE_ENVIRONMENT_RAYCASTER_CREATE_COMPLETION_META,
};
CHK_XR(xrCreateEnvironmentRaycasterCompleteMETA(session, createFuture, &completion));
CHK_XR(completion.futureResult); // Return code of the create function
XrEnvironmentRaycasterMETA raycaster = completion.environmentRaycaster;

// these are setup to match the location and time of the ray's origin and direction.
XrSpace localSpace; // previously initialized, e.g. from XR_REFERENCE_SPACE_TYPE_LOCAL
XrTime time; // previously initialized, e.g. from frameState.predictedDisplayTime
XrVector3f rayOrigin; // previously initialized
XrVector3f rayDirection; // previously initialized

// Perform an environment raycast operation
XrEnvironmentRaycastFilterDistanceMETA distanceFilter = {
    .type = XR_TYPE_ENVIRONMENT_RAYCAST_FILTER_DISTANCE_META,
    .maxDistance = 5.0f,
};
XrEnvironmentRaycastFilterBaseHeaderMETA* filters[] = {
    reinterpret_cast<XrEnvironmentRaycastFilterBaseHeaderMETA*>(&distanceFilter)};
XrEnvironmentRaycastHitGetInfoMETA info = {
    .type = XR_TYPE_ENVIRONMENT_RAYCAST_HIT_GET_INFO_META,
    .baseSpace = localSpace,
    .time = time,
    .origin = rayOrigin,
    .direction = rayDirection,
    .filterCount = 1,
```

```
    .filters = filters,
};
XrEnvironmentRaycastHitMETA hitPoint = {
    .type = XR_TYPE_ENVIRONMENT_RAYCAST_HIT_META,
};
CHK_XR(xrPerformEnvironmentRaycastMETA(raycaster, &info, &hitPoint));
if (hitPoint.status == XR_ENVIRONMENT_RAYCAST_HIT_STATUS_HIT_META) {
    // Do something with hitPoint, e.g. place a virtual object
}

// Destroy environment raycaster when no longer needed by the app
CHK_XR(xrDestroyEnvironmentRaycasterMETA(raycaster));
```

12.150.7. New Object Types

- XrEnvironmentRaycasterMETA

12.150.8. New Commands

- xrCreateEnvironmentRaycasterAsyncMETA
- xrCreateEnvironmentRaycasterCompleteMETA
- xrDestroyEnvironmentRaycasterMETA
- xrPerformEnvironmentRaycastMETA

12.150.9. New Structures

- XrEnvironmentRaycastFilterBaseHeaderMETA
- XrEnvironmentRaycastFilterDistanceMETA
- XrEnvironmentRaycastHitGetInfoMETA
- XrEnvironmentRaycastHitMETA
- XrEnvironmentRaycasterCreateCompletionMETA
- XrEnvironmentRaycasterCreateInfoMETA
- Extending XrSystemProperties:
  - XrSystemEnvironmentRaycastPropertiesMETA

12.150.10. New Enums

- XrEnvironmentRaycastHitStatusMETA

12.150.11. New Enum Constants

- XR_META_ENVIRONMENT_RAYCAST_EXTENSION_NAME
- XR_META_environment_raycast_SPEC_VERSION
- Extending XrObjectType:
  - XR_OBJECT_TYPE_ENVIRONMENT_RAYCASTER_META
- Extending XrStructureType:
  - XR_TYPE_ENVIRONMENT_RAYCASTER_CREATE_COMPLETION_META
  - XR_TYPE_ENVIRONMENT_RAYCASTER_CREATE_INFO_META
  - XR_TYPE_ENVIRONMENT_RAYCAST_FILTER_DISTANCE_META
  - XR_TYPE_ENVIRONMENT_RAYCAST_HIT_GET_INFO_META
  - XR_TYPE_ENVIRONMENT_RAYCAST_HIT_META
  - XR_TYPE_SYSTEM_ENVIRONMENT_RAYCAST_PROPERTIES_META

12.150.12. Issues

12.150.13. Version History

- Revision 1, 2025-03-01 (Chengyuan Lin)
  - Initial extension description

12.151. XR_META_face_tracking_visemes

Name String

XR_META_face_tracking_visemes

Extension Type

Instance extension

Registered Extension Number

542

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

and
XR_FB_face_tracking2

Last Modified Date

2024-08-05

IP Status

No known IP claims.

Contributors

Jaebong Lee, Meta
Bill Orr, Meta
Scott Ramsby, Meta
Igor Tceglevskii, Meta

12.151.1. Overview

Visemes are the visual representations of the face and mouth while speaking certain sounds in a spoken language. Visemes are used here as an alternative way of representing the output of the face tracking extension.

This extension augments the XR_FB_face_tracking2 extension to enable an alternative way of representing its output.

The application **must** also enable the XR_FB_face_tracking2 extension in order to use this extension.

12.151.2. Inspect system capability

The XrSystemFaceTrackingVisemesPropertiesMETA structure is defined as:

```
// Provided by XR_META_face_tracking_visemes
typedef struct XrSystemFaceTrackingVisemesPropertiesMETA {
    XrStructureType    type;
    void*              next;
    XrBool32           supportsVisemes;
} XrSystemFaceTrackingVisemesPropertiesMETA;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- supportsVisemes is an XrBool32 specifying if visemes are supported.

An application **can** inspect whether the system is capable of estimating visemes by extending the XrSystemProperties with XrSystemFaceTrackingVisemesPropertiesMETA structure when calling xrGetSystemProperties.

The runtime **must** set `supportsVisemes` to `XR_FALSE`, if both XrSystemFaceTrackingProperties2FB ::`supportsVisualFaceTracking` and XrSystemFaceTrackingProperties2FB::`supportsAudioFaceTracking` of XrSystemFaceTrackingProperties2FB are `XR_FALSE`.

Valid Usage (Implicit)

- The `XR_META_face_tracking_visemes` extension **must** be enabled prior to using XrSystemFaceTrackingVisemesPropertiesMETA
- `type` **must** be `XR_TYPE_SYSTEM_FACE_TRACKING_VISEMES_PROPERTIES_META`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

12.151.3. Obtain visemes

The XrFaceTrackingVisemesMETA structure is defined as:

```
// Provided by XR_META_face_tracking_visemes
typedef struct XrFaceTrackingVisemesMETA {
    XrStructureType    type;
    const void*        next;
    XrBool32           isValid;
    float              visemes[XR_FACE_TRACKING_VISEME_COUNT_META];
} XrFaceTrackingVisemesMETA;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `isValid` is an `XrBool32` which indicates that the visemes are valid.
- `visemes` is an array of `float` that will be filled with visemes.

The XrFaceTrackingVisemesMETA structure returns the visemes. The application **can** add XrFaceTrackingVisemesMETA to the `next` chain of the XrFaceExpressionWeights2FB structure when calling xrGetFaceExpressionWeights2FB.

The runtime **should** set `isValid` to `XR_FALSE`, if any of the following are true:

- the face tracker failed to estimate visemes

- the application lost focus
- the consent for audio recording was denied.

If `isValid` is `XR_FALSE`, the runtime **must** also set all elements of `visemes` to zero.

Otherwise, if the runtime sets `isValid` to `XR_TRUE`, the runtime **must** update the `visemes` array ordered so that the array elements are indexed using the corresponding viseme enumeration XrFaceTrackingVisemeMETA.

Each value in the `visemes` array **must** be between 0.0 and 1.0 (inclusive), and the values represent a probability. If `isValid` is `XR_TRUE`, the summation of all elements in the `visemes` array **must** be 1.0.

If the application only wants visemes without blend shapes, as a **change** in the behavior of the underlying extension, it **may** assign 0 to XrFaceExpressionWeights2FB::`weightCount` and XrFaceExpressionWeights2FB::`confidenceCount` when calling xrGetFaceExpressionWeights2FB. In this case, the application **can** assign `NULL` to XrFaceExpressionWeights2FB::`weights` and XrFaceExpressionWeights2FB::`confidences`.

Valid Usage (Implicit)

- The `XR_META_face_tracking_visemes` extension **must** be enabled prior to using XrFaceTrackingVisemesMETA
- `type` **must** be `XR_TYPE_FACE_TRACKING_VISEMES_META`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

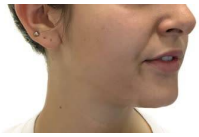
12.151.4. Conventions of visemes

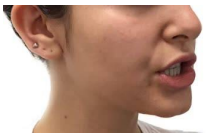
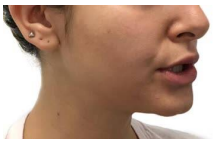
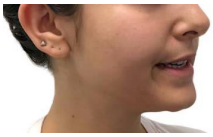
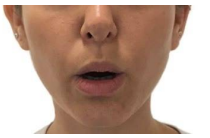
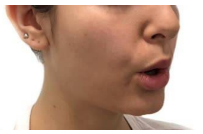
This extension defines 15 visemes and is designed to match existing 15-viseme standards as well as match the older Oculus Lipsync API's visemes.

```
#define XR_FACE_TRACKING_VISEME_COUNT_META 15
```

They are enumerated by XrFaceTrackingVisemeMETA.

```
// Provided by XR_META_face_tracking_visemes
typedef enum XrFaceTrackingVisemeMETA {
    XR_FACE_TRACKING_VISEME_SIL_META = 0,
    XR_FACE_TRACKING_VISEME_PP_META = 1,
    XR_FACE_TRACKING_VISEME_FF_META = 2,
    XR_FACE_TRACKING_VISEME_TH_META = 3,
    XR_FACE_TRACKING_VISEME_DD_META = 4,
    XR_FACE_TRACKING_VISEME_KK_META = 5,
    XR_FACE_TRACKING_VISEME_CH_META = 6,
    XR_FACE_TRACKING_VISEME_SS_META = 7,
    XR_FACE_TRACKING_VISEME_NN_META = 8,
    XR_FACE_TRACKING_VISEME_RR_META = 9,
    XR_FACE_TRACKING_VISEME_AA_META = 10,
    XR_FACE_TRACKING_VISEME_E_META = 11,
    XR_FACE_TRACKING_VISEME_IH_META = 12,
    XR_FACE_TRACKING_VISEME_OH_META = 13,
    XR_FACE_TRACKING_VISEME_OU_META = 14,
    XR_FACE_TRACKING_VISEME_MAX_ENUM_META = 0x7FFFFFFF
} XrFaceTrackingVisemeMETA;
```

Viseme	Phonemes	Examples	Mild Production	Emphasized Production	Rotated Image
SIL (XR_FACE_TRACKING_VISEME_SIL_META)	neutral			None	
PP (XR_FACE_TRACKING_VISEME_PP_META)	p, b, m	put, bat, mat			
FF (XR_FACE_TRACKING_VISEME_FF_META)	f, v	fat, vat			
TH (XR_FACE_TRACKING_VISEME_TH_META)	th	think, that			
DD (XR_FACE_TRACKING_VISEME_DD_META)	t, d	tip, doll			

Viseme	Phonemes	Examples	Mild Production	Emphasized Production	Rotated Image
KK (XR_FACE_TRACKING_VISEME_KK_META)	k, g	call, gas			
CH (XR_FACE_TRACKING_VISEME_CH_META)	tS, dZ, S	chair, join, she			
SS (XR_FACE_TRACKING_VISEME_SS_META)	s, z	sir, zeal			
NN (XR_FACE_TRACKING_VISEME_NN_META)	n, l	lot, not			
RR (XR_FACE_TRACKING_VISEME_RR_META)	r	red			
AA (XR_FACE_TRACKING_VISEME_AA_META)	A:	car			
E (XR_FACE_TRACKING_VISEME_E_META)	e	bed			
IH (XR_FACE_TRACKING_VISEME_IH_META)	ih	tip			
OH (XR_FACE_TRACKING_VISEME_OH_META)	oh	toe			

Viseme	Phonemes	Examples	Mild Production	Emphasized Production	Rotated Image
OU (XR_FACE_TRACKING_VISEME_OU_META)	ou	book			

12.151.5. Example code for obtaining visemes

The following example code demonstrates how to obtain all visemes.

```
XrInstance instance; // previously initialized
XrSystemId systemId; // previously initialized
XrSession session;   // previously initialized

// Confirm face tracking system support.
XrSystemFaceTrackingVisemesPropertiesMETA faceTrackingVisemesSystemProperties{
    .type = XR_TYPE_SYSTEM_FACE_TRACKING_VISEMES_PROPERTIES_META,
};
XrSystemProperties systemProperties{
    .type = XR_TYPE_SYSTEM_PROPERTIES,
    .next = &faceTrackingVisemesSystemProperties,
};
CHK_XR(xrGetSystemProperties(instance, systemId, &systemProperties));
if (!faceTrackingVisemesSystemProperties.supportsVisemes) {
    // The system does not support visemes
    return;
}

// Get function pointer for xrCreateFaceTracker2FB.
PFN_xrCreateFaceTracker2FB pfnCreateFaceTracker2FB = nullptr;
CHK_XR(xrGetInstanceProcAddr(instance, "xrCreateFaceTracker2FB",
                             reinterpret_cast<PFN_xrVoidFunction*>(
                             &pfnCreateFaceTracker2FB)));

// Create a face tracker for default set of facial expressions.
XrFaceTracker2FB faceTracker{XR_NULL_HANDLE};
{
    XrFaceTrackingDataSource2FB dataSources[2] = {
        XR_FACE_TRACKING_DATA_SOURCE2_VISUAL_FB,
        XR_FACE_TRACKING_DATA_SOURCE2_AUDIO_FB};
    XrFaceTrackerCreateInfo2FB createInfo{
        .type = XR_TYPE_FACE_TRACKER_CREATE_INFO2_FB,
        .next = nullptr,
        .faceExpressionSet = XR_FACE_EXPRESSION_SET2_DEFAULT_FB,
        // This tells the runtime that the application accepts
```

```
        // facial expression data from either of two data sources.
        .requestedDataSourceCount = 2,
        .requestedDataSources = dataSources,
    };
    CHK_XR(pfnCreateFaceTracker2FB(session, &createInfo, &faceTracker));
}

XrFaceTrackingVisemesMETA visemeInfo{
    .type = XR_TYPE_FACE_TRACKING_VISEMES_META,
};

// Set 0 to weightCount and confidenceCount to get visemes only
XrFaceExpressionWeights2FB expressionWeights{
    .type = XR_TYPE_FACE_EXPRESSION_WEIGHTS2_FB,
    .next = &visemeInfo,
    .weightCount = 0,
    .confidenceCount = 0,
};

// Get function pointer for xrGetFaceExpressionWeights2FB.
PFN_xrGetFaceExpressionWeights2FB pfnGetFaceExpressionWeights = nullptr;
CHK_XR(xrGetInstanceProcAddr(instance, "xrGetFaceExpressionWeights2FB",
                             reinterpret_cast<PFN_xrVoidFunction*>(
                             &pfnGetFaceExpressionWeights)));
while (1) {
    // ...
    // For every frame in the frame loop
    // ...
    XrFrameState frameState; // previously returned from xrWaitFrame
    const XrTime time = frameState.predictedDisplayTime;

    XrFaceExpressionInfo2FB expressionInfo{
        .type = XR_TYPE_FACE_EXPRESSION_INFO2_FB,
        .time = time,
    };

    CHK_XR(pfnGetFaceExpressionWeights(faceTracker, &expressionInfo, &
expressionWeights));

    if (visemeInfo.isValid) {
        for (uint32_t i = 0; i < XR_FACE_TRACKING_VISEME_COUNT_META; ++i) {
            // visemeInfo.visemes[i] contains a weight of specific visemes
        }
    }
}
```

12.151.6. New Structures

- Extending XrFaceExpressionWeights2FB:
  - XrFaceTrackingVisemesMETA
- Extending XrSystemProperties:
  - XrSystemFaceTrackingVisemesPropertiesMETA

12.151.7. New Enums

- XrFaceTrackingVisemeMETA

12.151.8. New Enum Constants

- XR_FACE_TRACKING_VISEME_COUNT_META
- XR_META_FACE_TRACKING_VISEMES_EXTENSION_NAME
- XR_META_face_tracking_visemes_SPEC_VERSION
- Extending XrStructureType:
  - XR_TYPE_FACE_TRACKING_VISEMES_META
  - XR_TYPE_SYSTEM_FACE_TRACKING_VISEMES_PROPERTIES_META

12.151.9. Issues

- Does this extension operate independently of XR_FB_face_tracking2?
  - Resolved.
  - Answer: Yes. The extension allows the application to query visemes data without returning blend weights (from XR_FB_face_tracking2) but it is expected that an application could use the two datasets together and that a runtime could generate the two datasets together.

12.151.10. Version History

- Revision 1, 2024-08-05 (Jaebong Lee)
  - Initial extension description

12.152. XR_META_foveation_eye_tracked

Name String

XR_META_foveation_eye_tracked

Extension Type

Instance extension

Registered Extension Number

201

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0
and
XR_FB_foveation
and
XR_FB_foveation_configuration

Contributors

Ross Ning, Facebook
Kevin Xiao, Facebook
Remi Palandri, Facebook
Jian Zhang, Facebook
Neel Bedekar, Facebook

Overview

Eye tracked foveated rendering renders lower pixel density in the periphery of the user's gaze, taking advantage of low peripheral acuity.

This extension allows:

- An application to query eye tracked foveation availability.
- An application to request eye tracked foveation profile supported by the runtime and apply them to foveation-supported swapchains.
- An application to query foveation center position every frame.
- An application to request a foveation pattern update from the runtime. As a consequence, runtime knows how to adjust the eye tracking camera exposure start time in order to optimize the total pipeline latency.

New Object Types

New Flag Types

```
// Provided by XR_META_foveation_eye_tracked
typedef XrFlags64 XrFoveationEyeTrackedProfileCreateFlagsMETA;
```

```
// Provided by XR_META_foveation_eye_tracked
// Flag bits for XrFoveationEyeTrackedProfileCreateFlagsMETA
```

There are currently no eye tracked profile create flags. This is reserved for future use.

```
// Provided by XR_META_foveation_eye_tracked
typedef XrFlags64 XrFoveationEyeTrackedStateFlagsMETA;
```

```
// Provided by XR_META_foveation_eye_tracked
// Flag bits for XrFoveationEyeTrackedStateFlagsMETA
static const XrFoveationEyeTrackedStateFlagsMETA
XR_FOVEATION_EYE_TRACKED_STATE_VALID_BIT_META = 0x00000001;
```

Flag Descriptions

- XR_FOVEATION_EYE_TRACKED_STATE_VALID_BIT_META — Indicates whether or not foveation data is valid. This can happen if the eye tracker is obscured, the camera has dirt, or eye lid is closed, etc.

New Enum Constants

XrStructureType enumeration is extended with:

- XR_TYPE_FOVEATION_EYE_TRACKED_PROFILE_CREATE_INFO_META
- XR_TYPE_FOVEATION_EYE_TRACKED_STATE_META
- XR_TYPE_SYSTEM_FOVEATION_EYE_TRACKED_PROPERTIES_META

New Enums

New Structures

The XrFoveationEyeTrackedProfileCreateInfoMETA structure is defined as:

```
// Provided by XR_META_foveation_eye_tracked
typedef struct XrFoveationEyeTrackedProfileCreateInfoMETA {
    XrStructureType                                 type;
    const void*                                     next;
    XrFoveationEyeTrackedProfileCreateFlagsMETA    flags;
} XrFoveationEyeTrackedProfileCreateInfoMETA;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `flags` is a bitmask of XrFoveationEyeTrackedProfileCreateFlagBitsMETA which indicate various characteristics for how eye tracked foveation is enabled on the swapchain.

XrFoveationEyeTrackedProfileCreateInfoMETA **can** be added to the `next` chain of XrFoveationLevelProfileCreateInfoFB in order to enable eye tracked foveation. The runtime **must** apply an eye tracked foveation pattern according to the parameters defined in the XrFoveationLevelProfileCreateInfoFB.

Valid Usage (Implicit)

- The `XR_META_foveation_eye_tracked` extension **must** be enabled prior to using XrFoveationEyeTrackedProfileCreateInfoMETA
- `type` **must** be `XR_TYPE_FOVEATION_EYE_TRACKED_PROFILE_CREATE_INFO_META`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `flags` **must** be `0`

The XrFoveationEyeTrackedStateMETA structure is defined as:

```
// Provided by XR_META_foveation_eye_tracked
typedef struct XrFoveationEyeTrackedStateMETA {
    XrStructureType                         type;
    void*                                   next;
    XrVector2f                              foveationCenter[
XR_FOVEATION_CENTER_SIZE_META];
    XrFoveationEyeTrackedStateFlagsMETA     flags;
} XrFoveationEyeTrackedStateMETA;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `foveationCenter` is the center of the foveal region defined in NDC space in the range of -1 to 1 for both eyes.
- `flags` is a bitmask of XrFoveationEyeTrackedStateFlagBitsMETA which indicates various characteristics for current foveation state.

XrFoveationEyeTrackedStateMETA **must** be provided when calling xrGetFoveationEyeTrackedStateMETA. The runtime **must** interpret XrFoveationEyeTrackedStateMETA without any additional structs in its `next` chain in order to query eye tracked foveation state, e.g. the center of the foveal region.

Valid Usage (Implicit)

- The `XR_META_foveation_eye_tracked` extension **must** be enabled prior to using XrFoveationEyeTrackedStateMETA
- `type` **must** be `XR_TYPE_FOVEATION_EYE_TRACKED_STATE_META`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

The XrSystemFoveationEyeTrackedPropertiesMETA structure is defined as:

```
// Provided by XR_META_foveation_eye_tracked
typedef struct XrSystemFoveationEyeTrackedPropertiesMETA {
    XrStructureType    type;
    void*              next;
    XrBool32           supportsFoveationEyeTracked;
} XrSystemFoveationEyeTrackedPropertiesMETA;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `supportsFoveationEyeTracked` indicates if the current system is capable of eye tracked foveation.

An application **can** inspect whether the system is capable of eye tracked foveation by extending the XrSystemProperties with XrSystemFoveationEyeTrackedPropertiesMETA structure when calling xrGetSystemProperties.

Valid Usage (Implicit)

- The `XR_META_foveation_eye_tracked` extension **must** be enabled prior to using XrSystemFoveationEyeTrackedPropertiesMETA
- `type` **must** be `XR_TYPE_SYSTEM_FOVEATION_EYE_TRACKED_PROPERTIES_META`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

New Functions

The xrGetFoveationEyeTrackedStateMETA function is defined as:

```
// Provided by XR_META_foveation_eye_tracked
XrResult xrGetFoveationEyeTrackedStateMETA(
    XrSession                                   session,
    XrFoveationEyeTrackedStateMETA*             foveationState);
```

Parameter Descriptions

- `session` is the XrSession in which the eye tracked foveation profile is applied.
- `foveationState` is a pointer to an XrFoveationEyeTrackedStateMETA structure returning the current eye tracked foveation state.

The xrGetFoveationEyeTrackedStateMETA function returns the current eye tracked foveation state including the center of the foveal region, validity of the foveation data, etc.

Note that xrUpdateSwapchainFB **should** be called right before the xrGetFoveationEyeTrackedStateMETA function in order to (1) request a foveation pattern update by the runtime (2) optionally instruct the runtime to adjust the eye tracking camera capture start time in order to optimize for pipeline latency.

Valid Usage (Implicit)

- The `XR_META_foveation_eye_tracked` extension **must** be enabled prior to calling xrGetFoveationEyeTrackedStateMETA
- `session` **must** be a valid XrSession handle
- `foveationState` **must** be a pointer to an XrFoveationEyeTrackedStateMETA structure

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_FEATURE_UNSUPPORTED`
- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_VALIDATION_FAILURE`

Issues

Version History

- Revision 1, 2022-04-08 (Ross Ning)
  - Initial extension description

12.153. XR_META_hand_tracking_frequency_hint

Name String

`XR_META_hand_tracking_frequency_hint`

Extension Type

Instance extension

Registered Extension Number

695

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0
and
XR_EXT_hand_tracking

Last Modified Date

2025-09-30

IP Status

No known IP claims.

Contributors

Robert Memmott, Meta
John Kearney, Meta
Andreas Selvik, Meta

12.153.1. Overview

This extension augments the XR_EXT_hand_tracking extension. If the application passes XR_META_hand_tracking_frequency_hint but does not pass XR_EXT_hand_tracking then xrCreateInstance **must** return XR_ERROR_EXTENSION_DEPENDENCY_NOT_ENABLED.

This extension allows applications to provide a frequency hint to the runtime to indicate the desired hand tracking update frequency. Applications **can** suggest that the runtime use a higher tracking frequency when needed for low-latency scenarios, or use the default frequency for normal operation.

12.153.2. Setting the hand tracking frequency hint

The xrSetHandTrackingFrequencyHintMETA function provides a frequency hint to the runtime to suggest the desired hand tracking update frequency for the session.

```
// Provided by XR_META_hand_tracking_frequency_hint
XrResult xrSetHandTrackingFrequencyHintMETA(
    XrSession                                   session,
    XrHandTrackingFrequencyHintMETA             frequencyHint);
```

Parameter Descriptions

- `session` is an XrSession handle previously created with xrCreateSession.
- `frequencyHint` is the frequency hint that the application suggests to the runtime.

xrSetHandTrackingFrequencyHintMETA provides a frequency hint to the runtime to suggest the desired hand tracking update frequency for the session. This is a suggestion mechanism only, and the runtime **may** choose to ignore the hint based on user preferences, system constraints, power management policies, or other considerations. Applications **should** not rely on the runtime honoring the frequency hint and **should** be prepared to handle hand tracking data at any supported frequency.

The frequency hint remains active until changed by another call to xrSetHandTrackingFrequencyHintMETA or until the session is destroyed.

Applications **can** call this function multiple times during a session to adjust the frequency hint based on changing application requirements.

The runtime **may** change the actual hand tracking frequency at any time, regardless of the frequency hint, and applications **should** be prepared to handle hand tracking data at any supported frequency.

Valid Usage (Implicit)

- The `XR_META_hand_tracking_frequency_hint` extension **must** be enabled prior to calling xrSetHandTrackingFrequencyHintMETA
- `session` **must** be a valid XrSession handle
- `frequencyHint` **must** be a valid XrHandTrackingFrequencyHintMETA value

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

12.153.3. Hand Tracking Frequency Hint

The XrHandTrackingFrequencyHintMETA enumerates the frequency hints that applications **can** provide to suggest the desired hand tracking update frequency to the runtime.

```
// Provided by XR_META_hand_tracking_frequency_hint
typedef enum XrHandTrackingFrequencyHintMETA {
    XR_HAND_TRACKING_FREQUENCY_HINT_DEFAULT_META = 1,
    XR_HAND_TRACKING_FREQUENCY_HINT_HIGH_META = 2,
    XR_HAND_TRACKING_FREQUENCY_HINT_MAX_ENUM_META = 0x7FFFFFFF
} XrHandTrackingFrequencyHintMETA;
```

Enumerant Descriptions

- XR_HAND_TRACKING_FREQUENCY_HINT_DEFAULT_META - This value indicates that the application suggests the runtime use the default hand tracking frequency. This is typically the most power-efficient frequency that provides adequate tracking quality for general use cases.
- XR_HAND_TRACKING_FREQUENCY_HINT_HIGH_META - This value indicates that the application suggests the runtime use a higher hand tracking frequency when possible. This **may** provide more responsive tracking for performance-critical applications, but at higher framerates, the effectiveness of temporal smoothing algorithms is reduced, which **can** result in increased jitter and less visually smooth hand tracking.

12.153.4. Sample code

```
XrSession session; // previously initialized
PFN_xrSetHandTrackingFrequencyHintMETA setHandTrackingFrequencyHint; // previously
initialized

/// request the default hand tracking frequency
CHK_XR(setHandTrackingFrequencyHint(session,
XR_HAND_TRACKING_FREQUENCY_HINT_DEFAULT_META));

/// request a high hand tracking frequency
CHK_XR(setHandTrackingFrequencyHint(session, XR_HAND_TRACKING_FREQUENCY_HINT_HIGH_META));
```

12.153.5. Issues

1) Should the frequency hint be per-handTracker or per-session?

RESOLVED: Per-session. Most applications that require high-frequency hand tracking will need it for both hands simultaneously. A per-session approach simplifies the API and reduces the complexity for both applications and runtimes.

2) Should there be more granular frequency levels beyond DEFAULT and HIGH?

RESOLVED: No. A simple two-level hint system provides sufficient flexibility for most use cases while keeping the API simple. More granular control could be added in future extensions if needed.

12.153.6. Version History

- Revision 1, 2025-09-30 (Robert Memmott)
  - Initial extension description

12.154. XR_META_hand_tracking_microgestures

Name String

`XR_META_hand_tracking_microgestures`

Extension Type

Instance extension

Registered Extension Number

253

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0
and
`XR_EXT_hand_interaction`

Last Modified Date

2024-09-03

IP Status

No known IP claims.

Contributors

Matthew Langille, Meta
Kenrick Kin, Meta
Chengde Wan, Meta
Ken Koh, Meta
Necati Cihan Camgoz, Meta
Shugao Ma, Meta
Andrei Marin, Meta
Eric Sauser, Meta
Muzaffer Akbay, Meta
Fengyang Zhang, Meta
Jingming Dong, Meta
Yujun Cai, Meta
Matthew Longest, Meta

12.154.1. Overview

Microgestures expand the capabilities of hand tracking by enabling low-calorie thumb tap and swipe motions to trigger discrete D-pad-like directional commands.

The hand pose and motion of the thumb is as follows: initially, the user **must** raise their thumb above their index finger (not touching the index finger). For best results, the user **should** slightly curl the other fingers as in the following illustration; i.e. not too extended, nor completely curled into a fist.

A **tap** is performed by touching the middle segment of the index finger with the thumb, and then lifting the thumb.

The four directional thumb swipes performed on the surface of the index finger are:

Left swipe

a swipe towards the index fingertip on the right hand, and away from the index fingertip on the left

hand. On the right hand for example, the motion is as follows: the thumb starts raised above the index finger, touches the middle segment of the index finger, slides towards the index fingertip, and lifts.

Right swipe

the same motion as the left swipe, but in the opposite direction. On the right hand for example, the thumb starts raised above the index finger, touches the middle segment of the index finger, slides away from the index fingertip, and lifts.

Forward swipe

the thumb starts raised above the index finger, touches the middle segment of the index finger, slides forward, and lifts.

Backward swipe

the thumb starts raised above the index finger, touches the middle segment of the index finger, slides backward/downward, and lifts.

Note that the motions are performed at moderate to quick speeds, and are intended to be performed in one smooth motion. The detection of the gesture happens at the end of the motion, regardless of speed.

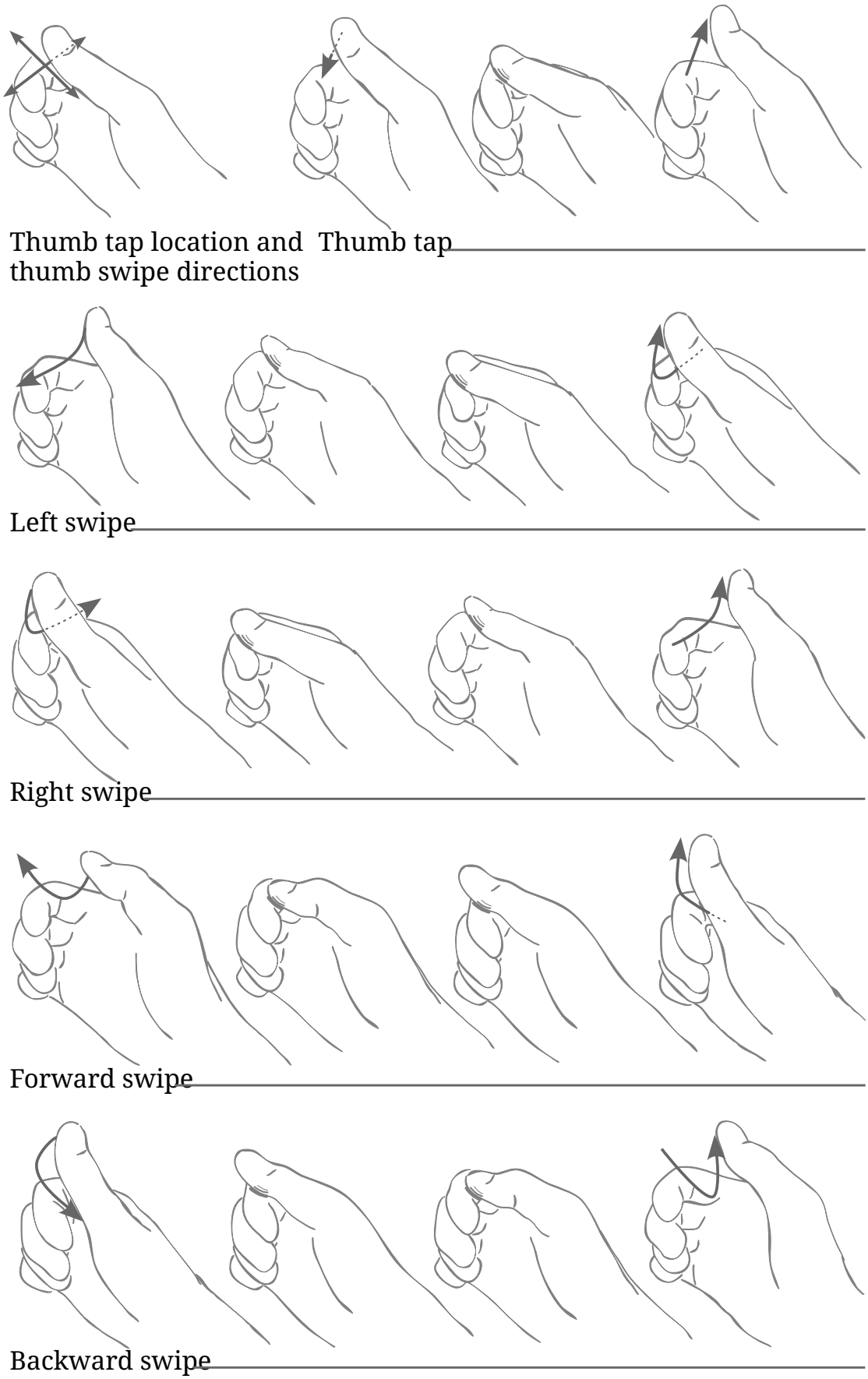


Figure 35. Microgestures

This extension exposes these discrete signals through the OpenXR action system. It augments XR_EXT_hand_interaction by adding a series of component paths to the interaction profile.

12.154.2. Enabling Microgestures

In order to use the binding paths defined in this extension in addition to those already present in XR_EXT_hand_interaction, applications **must** enable **both** XR_EXT_hand_interaction and XR_META_hand_tracking_microgestures.

If the application passes XR_META_hand_tracking_microgestures but does not pass XR_EXT_hand_interaction then xrCreateInstance **must** return XR_ERROR_EXTENSION_DEPENDENCY_NOT_ENABLED.

12.154.3. Action paths for Microgestures

Interaction profile path:

- */interaction_profiles/ext/hand_interaction_ext*

Valid for top level user path:

- */user/hand/left*
- */user/hand/right*

Additional supported component paths:

- *.../input/swipe_left_meta/click*
- *.../input/swipe_right_meta/click*
- *.../input/swipe_forward_meta/click*
- *.../input/swipe_backward_meta/click*
- *.../input/tap_thumb_meta/click*

All listed inputs are boolean that become XR_TRUE once the corresponding gesture has been completed and recognized.

Corresponding inputs are XR_FALSE otherwise, even during the progress of a gesture.

12.154.4. New Enum Constants

- XR_META_HAND_TRACKING_MICROGESTURES_EXTENSION_NAME
- XR_META_hand_tracking_microgestures_SPEC_VERSION

12.154.5. Issues

When the `XR_EXT_hand_interaction` and `XR_META_hand_tracking_microgestures` extensions are available and enabled, the runtime **should** avoid interferences between the detection of pinches and microgestures, gestures that are similar in nature. Specifically, a swipe towards the tip of the index finger **should** not be misclassified as a pinch.

12.154.6. Version History

- Revision 1, 2024-09-03 (Andrei Marin)
  - Initial API version

12.155. XR_META_hand_tracking_unextrapolated_poses

Name String

`XR_META_hand_tracking_unextrapolated_poses`

Extension Type

Instance extension

Registered Extension Number

694

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0
and
`XR_EXT_hand_tracking`

Last Modified Date

2025-09-30

IP Status

No known IP claims.

Contributors

Robert Memmott, Meta
John Kearney, Meta
Andreas Selvik, Meta

12.155.1. Overview

This extension augments the XR_EXT_hand_tracking extension. If the application passes XR_META_hand_tracking_unextrapolated_poses but does not pass XR_EXT_hand_tracking then xrCreateInstance **must** return XR_ERROR_EXTENSION_DEPENDENCY_NOT_ENABLED.

This extension allows applications to request unextrapolated hand tracking poses and obtain information about the capture time of the data used to generate the hand poses. This is useful for applications that want to perform their own extrapolation using additional context information that is not available to the runtime.

When the application chains the XrHandTrackingUnextrapolatedPosesRequestMETA structure to the XrHandJointsLocateInfoEXT structure when calling xrLocateHandJointsEXT, the runtime **must** provide the latest calculated hand poses without temporal extrapolation. The time parameter in XrHandJointsLocateInfoEXT is still used for determining the reference space transformation, but **must** be ignored for determining the hand joint locations themselves.

The XrHandTrackingUnextrapolatedPosesRequestMETA structure **can** be used to request unextrapolated hand tracking poses.

```
// Provided by XR_META_hand_tracking_unextrapolated_poses
typedef struct XrHandTrackingUnextrapolatedPosesRequestMETA {
    XrStructureType    type;
    const void*        next;
} XrHandTrackingUnextrapolatedPosesRequestMETA;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain.

The XrHandTrackingUnextrapolatedPosesRequestMETA is a structure that an application **can** chain to XrHandJointsLocateInfoEXT::next when calling xrLocateHandJointsEXT to request that the runtime provide unextrapolated hand tracking poses.

Valid Usage (Implicit)

- The XR_META_hand_tracking_unextrapolated_poses extension **must** be enabled prior to using XrHandTrackingUnextrapolatedPosesRequestMETA
- type **must** be XR_TYPE_HAND_TRACKING_UNEXTRAPOLATED_POSES_REQUEST_META
- next **must** be NULL or a valid pointer to the next structure in a structure chain

The XrHandTrackingUnextrapolatedPosesMETA is a structure that an application **can** chain to XrHandJointLocationsEXT::next when calling xrLocateHandJointsEXT to receive information about the unextrapolated hand tracking poses.

```
// Provided by XR_META_hand_tracking_unextrapolated_poses
typedef struct XrHandTrackingUnextrapolatedPosesMETA {
    XrStructureType    type;
    void*              next;
    XrTime             captureTime;
} XrHandTrackingUnextrapolatedPosesMETA;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain.
- captureTime is an XrTime representing when the data used to generate the returned hand poses were captured.

The runtime **must** populate this structure when the XrHandTrackingUnextrapolatedPosesRequestMETA structure is chained to the XrHandJointsLocateInfoEXT input structure. The runtime **must** not modify this structure if the XrHandTrackingUnextrapolatedPosesRequestMETA structure is not chained to the XrHandJointsLocateInfoEXT input structure.

The captureTime field indicates when the source data used to generate the hand poses were captured by the tracking system. This timestamp **can** be used by applications to understand the age of the tracking data and implement appropriate extrapolation or prediction algorithms. If there are no valid poses the captureTime **must** be zero.

Valid Usage (Implicit)

- The XR_META_hand_tracking_unextrapolated_poses extension **must** be enabled prior to using XrHandTrackingUnextrapolatedPosesMETA
- type **must** be XR_TYPE_HAND_TRACKING_UNEXTRAPOLATED_POSES_META
- next **must** be NULL or a valid pointer to the next structure in a structure chain

12.155.2. Sample code

```
XrSession session; // previously initialized
XrSpace worldSpace; // previously initialized, e.g. from
  // XR_REFERENCE_SPACE_TYPE_LOCAL
XrFrameState frameState; // previously returned from xrWaitFrame
XrHandTrackerEXT handTracker; // previously initialized
PFN_xrLocateHandJointsEXT locateHandJoints; // previously initialized


/// unextrapolated poses request variable
XrHandTrackingUnextrapolatedPosesRequestMETA unextrapolatedPosesRequestMETA{
    .type = XR_TYPE_HAND_TRACKING_UNEXTRAPOLATED_POSES_REQUEST_META,
};

XrHandJointsLocateInfoEXT locateInfo{
    .type = XR_TYPE_HAND_JOINTS_LOCATE_INFO_EXT,
    .next = &unextrapolatedPosesRequestMETA,
    .baseSpace = worldSpace,
    .time = frameState.predictedDisplayTime,
};

/// unextrapolated poses response variable
XrHandTrackingUnextrapolatedPosesMETA unextrapolatedPosesMETA{
    .type = XR_TYPE_HAND_TRACKING_UNEXTRAPOLATED_POSES_META,
};

XrHandJointLocationEXT jointLocations[XR_HAND_JOINT_COUNT_EXT];
XrHandJointLocationsEXT locations{
    .type = XR_TYPE_HAND_JOINT_LOCATIONS_EXT,
    .next = &unextrapolatedPosesMETA,
    .jointCount = XR_HAND_JOINT_COUNT_EXT,
    .jointLocations = jointLocations,
};


CHK_XR(locateHandJoints(handTracker, &locateInfo, &locations));

// jointLocations contains the unextrapolated hand poses
// unextrapolatedPosesMETA.captureTime the capture time of the hand poses
```

12.155.3. Issues

1) Should the extension validate that both request and response structures are chained together?

RESOLVED: No strict validation required. The runtime will populate the response structure when the request structure is present, but applications **may** choose to chain only the request structure if they do not need the capture time information.

12.155.4. Version History

- Revision 1, 2025-09-30 (Robert Memmott)
  - Initial extension description

12.156. XR_META_hand_tracking_wide_motion_mode2

Name String

`XR_META_hand_tracking_wide_motion_mode2`

Extension Type

Instance extension

Registered Extension Number

696

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0
and
`XR_EXT_hand_tracking`
and
`XR_EXT_hand_tracking_data_source`

Last Modified Date

2025-09-30

IP Status

No known IP claims.

Contributors

Robert Memmott, Meta
John Kearney, Meta
Andreas Selvik, Meta

12.156.1. Overview

This extension augments the `XR_EXT_hand_tracking_data_source` extension. If the application passes `XR_META_hand_tracking_wide_motion_mode2` but does not pass `XR_EXT_hand_tracking_data_source` then

xrCreateInstance **must** return XR_ERROR_EXTENSION_DEPENDENCY_NOT_ENABLED.

This extension adds a new data source value to XrHandTrackingDataSourceEXT, which applications **can** use to request hand tracking poses that utilize wide motion mode inference when hands are not directly visible to the tracking cameras. This data source represents inferred hand poses generated when the hands are outside the normal camera tracking volume. When active, the runtime employs inference algorithms to provide hand pose estimates based on previous tracking data, body motion, or other contextual information when the hands are not directly observable by the tracking system. Applications **can** use this data source both to request wide motion mode tracking and to query whether returned hand poses were generated using the wide motion inference system.

Enumerant Descriptions

- XR_HAND_TRACKING_DATA_SOURCE_UNOBSTRUCTED_WIDE_MOTION_META - This data source value indicates that the hand tracking system **should** use wide motion mode inference to provide hand poses when the hands are not directly visible to the tracking cameras. When this data source is active, the runtime is providing inferred hand poses based on contextual information such as previous tracking data, body motion, or other available sensors when hands are outside the normal tracking volume. Applications **can** use this data source value in XrHandTrackingDataSourceInfoEXT to request wide motion mode tracking, and **can** query XrHandTrackingDataSourceStateEXT to determine if returned poses were generated using the wide motion inference system. The quality and accuracy of poses returned via this data source **may** be reduced compared to direct optical tracking, particularly when hands have been out of view for extended periods.

12.156.2. Sample Code

```
XrSession session; // previously initialized
XrSpace worldSpace; // previously initialized, e.g. from
  // XR_REFERENCE_SPACE_TYPE_LOCAL
PFN_xrCreateHandTrackerEXT pfnCreateHandTracker; // previously initialized
PFN_xrLocateHandJointsEXT pfnLocateHandJoints; // previously initialized

/// create hand tracker that contains the unobstructed wide motion mode data source
XrHandTrackerEXT handTracker = XR_NULL_HANDLE;

XrHandTrackerCreateInfoEXT createInfo{
    .type = XR_TYPE_HAND_TRACKER_CREATE_INFO_EXT,
    .hand = XR_HAND_LEFT_EXT,
    .handJointSet = XR_HAND_JOINT_SET_DEFAULT_EXT,
};

std::array<XrHandTrackingDataSourceEXT, 2> dataSourcesWMM = {
    XR_HAND_TRACKING_DATA_SOURCE_UNOBSTRUCTED_EXT,
```

```
    XR_HAND_TRACKING_DATA_SOURCE_UNOBSTRUCTED_WIDE_MOTION_META,
};

XrHandTrackingDataSourceInfoEXT usageInfo{
    .type = XR_TYPE_HAND_TRACKING_DATA_SOURCE_INFO_EXT,
    .requestedDataSourceCount = static_cast<uint32_t>(dataSourcesWMM.size()),
    .requestedDataSources = dataSourcesWMM.data(),
};

createInfo.next = &usageInfo;
CHK_XR(pfnCreateHandTracker(session, &createInfo, &handTracker));

XrFrameState frameState; // previously returned from xrWaitFrame
const XrTime time = frameState.predictedDisplayTime;

/// locate hand tracker using the unobstructed wide motion mode data source
XrHandJointsLocateInfoEXT locateInfo{
    .type = XR_TYPE_HAND_JOINTS_LOCATE_INFO_EXT,
    .baseSpace = worldSpace,
    .time = time,
};

XrHandJointLocationEXT jointLocations[XR_HAND_JOINT_COUNT_EXT];
XrHandTrackingDataSourceStateEXT dataSourceState{
    .type = XR_TYPE_HAND_TRACKING_DATA_SOURCE_STATE_EXT,
};
XrHandJointLocationsEXT locations{
    .type = XR_TYPE_HAND_JOINT_LOCATIONS_EXT,
    .next = &dataSourceState,
    .jointCount = XR_HAND_JOINT_COUNT_EXT,
    .jointLocations = jointLocations,
};


CHK_XR(pfnLocateHandJoints(handTracker, &locateInfo, &locations));
if(dataSourceState.dataSource ==
XR_HAND_TRACKING_DATA_SOURCE_UNOBSTRUCTED_WIDE_MOTION_META){
    // the hand poses were generated using the wide motion mode inference system
}
```

Version History

- Revision 1, 2025-09-30 (Robert Memmott)
  - Initial extension description

12.157. XR_META_headset_id

Name String

`XR_META_headset_id`

Extension Type

Instance extension

Registered Extension Number

246

Revision

2

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Last Modified Date

2022-08-11

IP Status

No known IP claims.

Contributors

Wenlin Mao, Meta Platforms
Andreas Loeve Selvik, Meta Platforms
Rémi Palandri, Meta Platforms
John Kearney, Meta Platforms
Jonathan Wright, Meta Platforms

Contacts

Wenlin Mao, Meta Platforms



Note

Using the headset ID to alter application behavior is discouraged, as it interferes with compatibility with current and future headsets. The OpenXR specification is designed with the goal of avoiding the need for explicit per-device logic. If the use of this extension is required, it is encouraged to let the OpenXR working group know about the use case, through a communication channel like email or GitHub. While this usage is discouraged, applications that need this functionality are encouraged to use this extension instead of the `systemName` field in XrSystemProperties. Game engines and similar middleware **should** not enable this extension by default. This extension will be deprecated and no longer exposed once the remaining use cases are resolved in a more portable way.

The XrSystemHeadsetIdPropertiesMETA structure is defined as:

```
// Provided by XR_META_headset_id
typedef struct XrSystemHeadsetIdPropertiesMETA {
    XrStructureType    type;
    void*              next;
    XrUuidEXT          id;
} XrSystemHeadsetIdPropertiesMETA;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `id` is the XrUuidEXT corresponding to the headset model.

An application **can** get a corresponding headset UUID of the headset model by chaining an XrSystemHeadsetIdPropertiesMETA structure to the XrSystemProperties when calling xrGetSystemProperties.

The UUID returned in the XrSystemHeadsetIdPropertiesMETA structure is an opaque UUID that identifies a runtime / headset model combo.

The runtime **should** always return the same UUID for a given headset model for the entire lifetime of that product.

The runtime **may** report a different UUID to some applications for compatibility purposes.

This is in contrast to the XrSystemProperties::`systemName` field which is not required to be consistent across product renames.

This is intended to be a temporary feature that will be deprecated along with its extension as soon as motivating use cases are resolved in a better way. See the disclaimer at the start of the XR_META_headset_id extension documentation for more details.

Valid Usage (Implicit)

- The XR_META_headset_id extension **must** be enabled prior to using XrSystemHeadsetIdPropertiesMETA
- type **must** be XR_TYPE_SYSTEM_HEADSET_ID_PROPERTIES_META
- next **must** be NULL or a valid pointer to the next structure in a structure chain

New Object Types

New Atom

New Flag Types

New Enum Constants

XrStructureType enumeration is extended with:

- XR_TYPE_SYSTEM_HEADSET_ID_PROPERTIES_META

New Enums

New Structures

- XrSystemHeadsetIdPropertiesMETA

New Functions

Issues

Version History

- Revision 1, 2022-08-11 (Wenlin Mao)
  - Initial extension description
- Revision 2, 2023-01-30 (Wenlin Mao)
  - Drop requirement for XR_EXT_uuid being enabled

12.158. XR_META_local_dimming

Name String

XR_META_local_dimming

Extension Type

Instance extension

Registered Extension Number

217

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Last Modified Date

2022-05-05

IP Status

No known IP claims.

Contributors

Ross Ning, Meta Platforms
Haomiao Jiang, Meta Platforms
Remi Palandri, Meta Platforms
Xiang Wei, Meta Platforms

Overview

Local dimming allows to adjust backlight intensity of dark areas on the screen in order to increase content dynamic range. Local dimming feature is not intended for optical see-through HMDs.

An application **can** request the local dimming mode on a frame basis by chaining an XrLocalDimmingFrameEndInfoMETA structure to the XrFrameEndInfo.

- Using XrFrameEndInfoLocalDimmingFB is considered as a hint and will not trigger xrEndFrame errors whether or not the requested dimming mode is fulfilled by the runtime.
- The runtime will have full control of the local dimming mode and **may** disregard app requests. For example, the runtime **may** allow only one primary client to control the local dimming mode.

New Object Types

New Flag Types

New Enum Constants

XrStructureType enumeration is extended with:

- XR_TYPE_LOCAL_DIMMING_FRAME_END_INFO_META

New Enums

The local dimming mode is specified by the XrLocalDimmingModeMETA enumeration:

```
// Provided by XR_META_local_dimming
typedef enum XrLocalDimmingModeMETA {
    XR_LOCAL_DIMMING_MODE_OFF_META = 0,
    XR_LOCAL_DIMMING_MODE_ON_META = 1,
    XR_LOCAL_DIMMING_MODE_MAX_ENUM_META = 0x7FFFFFFF
} XrLocalDimmingModeMETA;
```

Enumerant Descriptions

- XR_LOCAL_DIMMING_MODE_OFF_META — Local dimming is turned off by default for the current submitted frame. This is the same as not chaining XrLocalDimmingModeMETA.
- XR_LOCAL_DIMMING_MODE_ON_META — Local dimming is turned on for the current submitted frame.

New Structures

The XrLocalDimmingFrameEndInfoMETA structure is defined as:

```
// Provided by XR_META_local_dimming
typedef struct XrLocalDimmingFrameEndInfoMETA {
    XrStructureType           type;
    const void*               next;
    XrLocalDimmingModeMETA    localDimmingMode;
} XrLocalDimmingFrameEndInfoMETA;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- localDimmingMode is the local dimming mode for current submitted frame.

The XrLocalDimmingFrameEndInfoMETA is a structure that an application **can** chain in

XrFrameEndInfo in order to request a local dimming mode.

Valid Usage (Implicit)

- The XR_META_local_dimming extension **must** be enabled prior to using XrLocalDimmingFrameEndInfoMETA
- type **must** be XR_TYPE_LOCAL_DIMMING_FRAME_END_INFO_META
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- localDimmingMode **must** be a valid XrLocalDimmingModeMETA value

New Functions

Issues

Version History

- Revision 1, 2022-05-05 (Ross Ning)
  - Initial draft

12.159. XR_META_passthrough_color_lut

Name String

XR_META_passthrough_color_lut

Extension Type

Instance extension

Registered Extension Number

267

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0
and
XR_FB_passthrough

Last Modified Date

2022-11-28

IP Status

No known IP claims.

Contributors

Andreas Loeve Selvik, Meta Platforms
Johannes Schmid, Meta Platforms
John Kearney, Meta Platforms

Overview

This extension adds the capability to define and apply RGB to RGB(A) color look-up tables (LUTs) to passthrough layers created using XR_FB_passthrough.

Color LUTs are 3-dimensional arrays which map each input color to a different output color. When applied to a Passthrough layer, the runtime **must** transform Passthrough camera images according to this map before display. Color LUTs **may** be used to achieve effects such as color grading, level control, color filtering, or chroma keying.

Color LUTs **must** be created using xrCreatePassthroughColorLutMETA before they **can** be applied to a Passthrough layer in a call to xrPassthroughLayerSetStyleFB (as a part of XrPassthroughColorMapLutMETA or XrPassthroughColorMapInterpolatedLutMETA). A color LUT **may** be applied to multiple Passthrough layers simultaneously.

New Object Types

```
// Provided by XR_META_passthrough_color_lut
XR_DEFINE_HANDLE(XrPassthroughColorLutMETA)
```

XrPassthroughColorLutMETA represents the definition and data for a color LUT which **may** be applied to a passthrough layer using xrPassthroughLayerSetStyleFB.

New Enum Constants

XrStructureType enumeration is extended with:

- XR_TYPE_SYSTEM_PASSTHROUGH_COLOR_LUT_PROPERTIES_META
- XR_TYPE_PASSTHROUGH_COLOR_LUT_CREATE_INFO_META
- XR_TYPE_PASSTHROUGH_COLOR_LUT_UPDATE_INFO_META
- XR_TYPE_PASSTHROUGH_COLOR_MAP_LUT_META
- XR_TYPE_PASSTHROUGH_COLOR_MAP_INTERPOLATED_LUT_META

New Enums

Specify the color channels contained in the color LUT.

```
// Provided by XR_META_passthrough_color_lut
typedef enum XrPassthroughColorLutChannelsMETA {
    XR_PASSTHROUGH_COLOR_LUT_CHANNELS_RGB_META = 1,
    XR_PASSTHROUGH_COLOR_LUT_CHANNELS_RGBA_META = 2,
    XR_PASSTHROUGH_COLOR_LUT_CHANNELS_MAX_ENUM_META = 0x7FFFFFFF
} XrPassthroughColorLutChannelsMETA;
```

New Structures

The XrSystemPassthroughColorLutPropertiesMETA structure is defined as:

```
// Provided by XR_META_passthrough_color_lut
typedef struct XrSystemPassthroughColorLutPropertiesMETA {
    XrStructureType    type;
    const void*        next;
    uint32_t           maxColorLutResolution;
} XrSystemPassthroughColorLutPropertiesMETA;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `maxColorLutResolution` Maximum value for XrPassthroughColorLutCreateInfoMETA ::`resolution` supported by the system. Runtimes implementing this extension **must** support a value of at least 32 for this property.

When the `XR_META_passthrough_color_lut` extension is enabled, an application **may** pass in an XrSystemPassthroughColorLutPropertiesMETA structure in next chain structure when calling xrGetSystemProperties to acquire information about the connected system.

The runtime **must** populate the XrSystemPassthroughColorLutPropertiesMETA structure with the relevant information to the XrSystemProperties returned by the xrGetSystemProperties call.

Valid Usage (Implicit)

- The `XR_META_passthrough_color_lut` extension **must** be enabled prior to using XrSystemPassthroughColorLutPropertiesMETA
- `type` **must** be `XR_TYPE_SYSTEM_PASSTHROUGH_COLOR_LUT_PROPERTIES_META`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

The XrPassthroughColorLutDataMETA structure is defined as:

```
// Provided by XR_META_passthrough_color_lut
typedef struct XrPassthroughColorLutDataMETA {
    uint32_t          bufferSize;
    const uint8_t*    buffer;
} XrPassthroughColorLutDataMETA;
```

Member Descriptions

- `bufferSize` is the number of bytes contained in the buffer data.
- `buffer` is a pointer to a memory block of `bufferSize` bytes that contains the LUT data.

XrPassthroughColorLutDataMETA defines the LUT data for a color LUT. This structure is used when creating and updating color LUTs.

Valid Usage (Implicit)

- The `XR_META_passthrough_color_lut` extension **must** be enabled prior to using XrPassthroughColorLutDataMETA
- `buffer` **must** be a pointer to an array of `bufferSize` `uint8_t` values
- The `bufferSize` parameter **must** be greater than `0`

The XrPassthroughColorLutCreateInfoMETA structure is defined as:

```
// Provided by XR_META_passthrough_color_lut
typedef struct XrPassthroughColorLutCreateInfoMETA {
    XrStructureType                     type;
    const void*                         next;
    XrPassthroughColorLutChannelsMETA   channels;
    uint32_t                            resolution;
    XrPassthroughColorLutDataMETA       data;
} XrPassthroughColorLutCreateInfoMETA;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `channels` defines the color channels expected in one LUT element. The number of bytes expected per LUT element is 3 for `XR_PASSTHROUGH_COLOR_LUT_CHANNELS_RGB_META` and 4 for `XR_PASSTHROUGH_COLOR_LUT_CHANNELS_RGBA_META`.
- `resolution` is the number of LUT elements per input channel. The total number of elements in the LUT is $resolution^3$.
- `data` contains the data the LUT is initialized with.

`resolution` **must** be a power of 2, otherwise the runtime **must** return `XR_ERROR_VALIDATION_FAILURE`. The runtime **may** impose a limit on the maximum supported resolution, which is indicated in XrSystemPassthroughColorLutPropertiesMETA. If `resolution` exceeds that limit, the runtime **must** return `XR_ERROR_VALIDATION_FAILURE`.

`data` contains a 3-dimensional array which defines an output color for each RGB input color. The input color is scaled to be in the range [0, `resolution`]. For an RGBA LUT, the RGBA tuple of output colors for an input color (R_{in}, G_{in}, B_{in}) is found in the four bytes starting at the offset 4 * (R_{in} + G_{in} * `resolution` + B_{in} * $resolution^2$). For an RGB LUT, the RGB tuple of output colors for an input color (R_{in}, G_{in}, B_{in}) is found in the three bytes starting at the offset 3 * (R_{in} + G_{in} * `resolution` + B_{in} * $resolution^2$).

Color LUT data **must** be specified and interpreted in sRGB color space.

Runtimes **must** employ trilinear interpolation of neighboring color values if the resolution of the color LUT is smaller than the bit depth of the input colors.

The value of XrPassthroughColorLutDataMETA::`bufferSize` in `data` **must** be equal to $resolution^3$ * bytesPerElement, where bytesPerElement is either 3 or 4 depending on `channels`. Otherwise, the runtime **must** return `XR_ERROR_PASSTHROUGH_COLOR_LUT_BUFFER_SIZE_MISMATCH_META`.

Valid Usage (Implicit)

- The `XR_META_passthrough_color_lut` extension **must** be enabled prior to using XrPassthroughColorLutCreateInfoMETA
- `type` **must** be `XR_TYPE_PASSTHROUGH_COLOR_LUT_CREATE_INFO_META`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `channels` **must** be a valid XrPassthroughColorLutChannelsMETA value
- `data` **must** be a valid XrPassthroughColorLutDataMETA structure

The XrPassthroughColorLutUpdateInfoMETA structure is defined as:

```
// Provided by XR_META_passthrough_color_lut
typedef struct XrPassthroughColorLutUpdateInfoMETA {
    XrStructureType                 type;
    const void*                     next;
    XrPassthroughColorLutDataMETA    data;
} XrPassthroughColorLutUpdateInfoMETA;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `data` contains the updated LUT data.

The LUT data **may** be updated for an existing color LUT, while channels and resolution remain constant after creation. Hence, the value of XrPassthroughColorLutDataMETA::`bufferSize` in `data` **must** be equal to the buffer size specified at creation. Otherwise, the runtime **must** return `XR_ERROR_PASSTHROUGH_COLOR_LUT_BUFFER_SIZE_MISMATCH_META`.

Valid Usage (Implicit)

- The `XR_META_passthrough_color_lut` extension **must** be enabled prior to using XrPassthroughColorLutUpdateInfoMETA
- `type` **must** be `XR_TYPE_PASSTHROUGH_COLOR_LUT_UPDATE_INFO_META`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `data` **must** be a valid XrPassthroughColorLutDataMETA structure

The XrPassthroughColorMapLutMETA structure is defined as:

```
// Provided by XR_META_passthrough_color_lut
typedef struct XrPassthroughColorMapLutMETA {
    XrStructureType                 type;
    const void*                     next;
    XrPassthroughColorLutMETA    colorLut;
    float                           weight;
} XrPassthroughColorMapLutMETA;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `colorLut` is an XrPassthroughColorLutMETA.
- `weight` is a factor in the range [0, 1] which defines the linear blend between the original and the mapped colors for the output color.

XrPassthroughColorMapLutMETA lets applications apply a color LUT to a passthrough layer. Other Passthrough style elements (such as edges) **must** not be affected by color LUTs.

Applications **may** use `weight` to efficiently blend between the original colors and the mapped colors. The blend is computed as (1 - `weight`) * C_{in} + `weight` * `colorLut` [C_{in}].

XrPassthroughColorMapLutMETA is provided in the `next` chain of XrPassthroughStyleFB when calling xrPassthroughLayerSetStyleFB. Subsequent calls to xrPassthroughLayerSetStyleFB with XrPassthroughColorMapLutMETA in the `next` chain update the color LUT for that layer. Subsequent calls to xrPassthroughLayerSetStyleFB without this XrPassthroughColorMapLutMETA (or XrPassthroughColorMapInterpolatedLutMETA) in the next chain disable color LUTs for that layer.

Valid Usage (Implicit)

- The `XR_META_passthrough_color_lut` extension **must** be enabled prior to using XrPassthroughColorMapLutMETA
- `type` **must** be `XR_TYPE_PASSTHROUGH_COLOR_MAP_LUT_META`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `colorLut` **must** be a valid XrPassthroughColorLutMETA handle

The XrPassthroughColorMapInterpolatedLutMETA structure is defined as:

```
// Provided by XR_META_passthrough_color_lut
typedef struct XrPassthroughColorMapInterpolatedLutMETA {
    XrStructureType             type;
    const void*                 next;
    XrPassthroughColorLutMETA   sourceColorLut;
    XrPassthroughColorLutMETA   targetColorLut;
    float                       weight;
} XrPassthroughColorMapInterpolatedLutMETA;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `sourceColorLut` is the initial XrPassthroughColorLutMETA.
- `targetColorLut` is the final XrPassthroughColorLutMETA.
- `weight` is a factor in the range [0, 1] which defines the linear blend between the initial and the final color LUT.

XrPassthroughColorMapInterpolatedLutMETA lets applications apply the interpolation between two color LUTs to a passthrough layer. Applications **may** use this feature to smoothly transition between two color LUTs. Other Passthrough style elements (such as edges) **must** not be affected by color LUTs.

The blend between `sourceColorLut` and `targetColorLut` is computed as (1 - `weight`) * `sourceColorLut` [C_{in}] + `weight` * `targetColorLut` [C_{in}].

XrPassthroughColorMapInterpolatedLutMETA is provided in the `next` chain of XrPassthroughStyleFB when calling xrPassthroughLayerSetStyleFB. Subsequent calls to xrPassthroughLayerSetStyleFB with XrPassthroughColorMapInterpolatedLutMETA in the next chain update the color LUT for that layer. Subsequent calls to xrPassthroughLayerSetStyleFB without this XrPassthroughColorMapInterpolatedLutMETA (or XrPassthroughColorMapLutMETA) in the next chain disable color LUTs for that layer.

Valid Usage (Implicit)

- The `XR_META_passthrough_color_lut` extension **must** be enabled prior to using XrPassthroughColorMapInterpolatedLutMETA
- `type` **must** be `XR_TYPE_PASSTHROUGH_COLOR_MAP_INTERPOLATED_LUT_META`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `sourceColorLut` **must** be a valid XrPassthroughColorLutMETA handle
- `targetColorLut` **must** be a valid XrPassthroughColorLutMETA handle
- Both of `sourceColorLut` and `targetColorLut` **must** have been created, allocated, or retrieved from the same XrPassthroughFB

New Functions

The xrCreatePassthroughColorLutMETA function is defined as:

```
// Provided by XR_META_passthrough_color_lut
XrResult xrCreatePassthroughColorLutMETA(
    XrPassthroughFB                                 passthrough,
    const XrPassthroughColorLutCreateInfoMETA*  createInfo,
    XrPassthroughColorLutMETA*                  colorLut);
```

Parameter Descriptions

- `passthrough` is the XrPassthroughFB this color LUT is created for.
- `createInfo` is the XrPassthroughColorLutCreateInfoMETA.
- `colorLut` is the resulting XrPassthroughColorLutMETA.

Creates a passthrough color LUT. The resulting XrPassthroughColorLutMETA **may** be referenced in XrPassthroughColorMapLutMETA and XrPassthroughColorMapInterpolatedLutMETA in subsequent calls to xrPassthroughLayerSetStyleFB.

Valid Usage (Implicit)

- The `XR_META_passthrough_color_lut` extension **must** be enabled prior to calling xrCreatePassthroughColorLutMETA
- `passthrough` **must** be a valid XrPassthroughFB handle
- `createInfo` **must** be a pointer to a valid XrPassthroughColorLutCreateInfoMETA structure
- `colorLut` **must** be a pointer to an XrPassthroughColorLutMETA handle

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FEATURE_UNSUPPORTED
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_PASSTHROUGH_COLOR_LUT_BUFFER_SIZE_MISMATCH_META
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The xrDestroyPassthroughColorLutMETA function is defined as:

```
// Provided by XR_META_passthrough_color_lut
XrResult xrDestroyPassthroughColorLutMETA(
    XrPassthroughColorLutMETA                   colorLut);
```

Parameter Descriptions

- colorLut is the XrPassthroughColorLutMETA to be destroyed.

Destroys a passthrough color LUT. If the color LUT is still in use (i.e. if for at least one passthrough layer, xrPassthroughLayerSetStyleFB has last been called with an instance of XrPassthroughColorMapLutMETA or XrPassthroughColorMapInterpolatedLutMETA in the next chain that references this color LUT), the runtime **must** retain the color LUT data and continue applying it to the affected passthrough layer until a different style is applied.

Valid Usage (Implicit)

- The `XR_META_passthrough_color_lut` extension **must** be enabled prior to calling xrDestroyPassthroughColorLutMETA
- `colorLut` **must** be a valid XrPassthroughColorLutMETA handle

Thread Safety

- Access to `colorLut`, and any child handles, **must** be externally synchronized

Return Codes

Success

- `XR_SUCCESS`

Failure

- `XR_ERROR_FEATURE_UNSUPPORTED`
- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_RUNTIME_FAILURE`

The xrUpdatePassthroughColorLutMETA function is defined as:

```
// Provided by XR_META_passthrough_color_lut
XrResult xrUpdatePassthroughColorLutMETA(
    XrPassthroughColorLutMETA                   colorLut,
    const XrPassthroughColorLutUpdateInfoMETA*  updateInfo);
```

Parameter Descriptions

- `colorLut` is the XrPassthroughColorLutMETA to be updated.
- `updateInfo` is the XrPassthroughColorLutUpdateInfoMETA.

Updates the LUT data of a passthrough color LUT. The data type of the color LUT (resolution and channels) is immutable. The provided data in this call **must** therefore match the data type specified at creation time. Specifically, XrPassthroughColorLutDataMETA::`bufferSize` of the new data **must** be equal to the XrPassthroughColorLutDataMETA::`bufferSize` specified during creation. Otherwise, the

runtime **must** return XR_ERROR_VALIDATION_FAILURE.

The runtime **must** reflect changes to color LUT data on all Passthrough layers the color LUT is currently applied to.

Valid Usage (Implicit)

- The XR_META_passthrough_color_lut extension **must** be enabled prior to calling xrUpdatePassthroughColorLutMETA
- colorLut **must** be a valid XrPassthroughColorLutMETA handle
- updateInfo **must** be a pointer to a valid XrPassthroughColorLutUpdateInfoMETA structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FEATURE_UNSUPPORTED
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_PASSTHROUGH_COLOR_LUT_BUFFER_SIZE_MISMATCH_META
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

Version History

- Revision 1, 2022-12-08 (Johannes Schmid)
  - Initial extension description

12.160. XR_META_passthrough_layer_resumed_event

Name String

XR_META_passthrough_layer_resumed_event

Extension Type

Instance extension

Registered Extension Number

283

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0
and
XR_FB_passthrough

Last Modified Date

2023-12-12

IP Status

No known IP claims.

Contributors

Ante Trbojevic, Meta Platforms

12.160.1. Overview

This extension defines an event that is emitted once a passthrough layer (created using XR_FB_passthrough) is resumed and ready for displaying after a resume command or when the passthrough layer was created with XR_PASSTHROUGH_IS_RUNNING_AT_CREATION_BIT_FB.

The functions xrPassthroughLayerResumeFB and xrCreatePassthroughLayerFB from XR_FB_passthrough are asynchronous without any guarantees on when their effect will be visible on the display. Runtimes **may** asynchronously perform operations which **may** take several frames to complete, such as turning on sensor hardware. Runtimes queue this event under the aforementioned conditions when this extension is requested during instance creation.

Unlike most extensions, to start receiving the event, an app only needs to enable this extension.

This extension depends on XR_FB_passthrough.

12.160.2. New Event

The XrEventDataPassthroughLayerResumedMETA structure is defined as:

```
// Provided by XR_META_passthrough_layer_resumed_event
typedef struct XrEventDataPassthroughLayerResumedMETA {
    XrStructureType         type;
    const void*             next;
    XrPassthroughLayerFB    layer;
} XrEventDataPassthroughLayerResumedMETA;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `layer` is an XrPassthroughLayerFB for which the event has been triggered.

Runtimes **must** queue the event exactly once when first presenting passthrough after an app successfully calls one of the following:

- xrPassthroughLayerResumeFB
- xrCreatePassthroughLayerFB with flag `XR_PASSTHROUGH_IS_RUNNING_AT_CREATION_BIT_FB` set

The passthrough layer state is reset when the app calls xrPassthroughLayerPauseFB.

Runtimes **must** queue the event again, if xrPassthroughLayerPauseFB is followed by xrPassthroughLayerResumeFB. During the transition from paused to resumed state, the event is queued exactly once when passthrough has been presented for the first time. If the passthrough feature is not active during the transition, for example because it has been paused using xrPassthroughPauseFB, the event is queued when passthrough becomes active.

Valid Usage (Implicit)

- The `XR_META_passthrough_layer_resumed_event` extension **must** be enabled prior to using XrEventDataPassthroughLayerResumedMETA
- `type` **must** be `XR_TYPE_EVENT_DATA_PASSTHROUGH_LAYER_RESUMED_META`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

12.160.3. New Structures

- XrEventDataPassthroughLayerResumedMETA

12.160.4. New Enum Constants

- XR_META_PASSTHROUGH_LAYER_RESUMED_EVENT_EXTENSION_NAME
- XR_META_passthrough_layer_resumed_event_SPEC_VERSION
- Extending XrStructureType:
  - XR_TYPE_EVENT_DATA_PASSTHROUGH_LAYER_RESUMED_META

Version History

- Revision 1, 2023-05-31 (Ante Trbojevic)
  - Initial extension description

12.161. XR_META_passthrough_preferences

Name String

XR_META_passthrough_preferences

Extension Type

Instance extension

Registered Extension Number

218

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Last Modified Date

2023-04-25

IP Status

No known IP claims.

Contributors

Johannes Schmid, Meta Platforms

Overview

This extension provides applications with access to system preferences concerning passthrough. For

more information on how applications can control the display of passthrough, see XR_FB_passthrough.

New Flag Types

```
// Provided by XR_META_passthrough_preferences
typedef XrFlags64 XrPassthroughPreferenceFlagsMETA;
```

```
// Provided by XR_META_passthrough_preferences
// Flag bits for XrPassthroughPreferenceFlagsMETA
static const XrPassthroughPreferenceFlagsMETA
XR_PASSTHROUGH_PREFERENCE_DEFAULT_TO_ACTIVE_BIT_META = 0x00000001;
```

Flag Descriptions

- XR_PASSTHROUGH_PREFERENCE_DEFAULT_TO_ACTIVE_BIT_META — Indicates that the runtime recommends apps to default to a mixed reality experience with passthrough (if supported).

New Enum Constants

XrStructureType enumeration is extended with:

- XR_TYPE_PASSTHROUGH_PREFERENCES_META

New Structures

The XrPassthroughPreferencesMETA structure is defined as:

```
// Provided by XR_META_passthrough_preferences
typedef struct XrPassthroughPreferencesMETA {
    XrStructureType                    type;
    const void*                        next;
    XrPassthroughPreferenceFlagsMETA   flags;
} XrPassthroughPreferencesMETA;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `flags` is a bitmask of XrPassthroughPreferenceFlagBitsMETA describing boolean passthrough preferences.

The runtime **must** populate the XrPassthroughPreferencesMETA structure with the relevant information when the app calls xrGetPassthroughPreferencesMETA.

Presence of the bit flag `XR_PASSTHROUGH_PREFERENCE_DEFAULT_TO_ACTIVE_BIT_META` does not indicate a guarantee that applications **can** enable and use passthrough in practice. The runtime **may** impose restrictions on passthrough usage (e.g. based on hardware availability or permission models) independently of the state of this flag bit. Apps **should** test for this flag explicitly, as more flag bits **may** be introduced in the future.

Valid Usage (Implicit)

- The `XR_META_passthrough_preferences` extension **must** be enabled prior to using XrPassthroughPreferencesMETA
- `type` **must** be `XR_TYPE_PASSTHROUGH_PREFERENCES_META`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

New Functions

The xrGetPassthroughPreferencesMETA function is defined as:

```
// Provided by XR_META_passthrough_preferences
XrResult xrGetPassthroughPreferencesMETA(
    XrSession                                   session,
    XrPassthroughPreferencesMETA*               preferences);
```

Parameter Descriptions

- `session` is the XrSession.
- `preferences` points to an instance of XrPassthroughPreferencesMETA structure, that will be filled with returned information

An application **can** call xrGetPassthroughPreferencesMETA to retrieve passthrough-related preferences from the system.

Valid Usage (Implicit)

- The `XR_META_passthrough_preferences` extension **must** be enabled prior to calling xrGetPassthroughPreferencesMETA
- `session` **must** be a valid XrSession handle
- `preferences` **must** be a pointer to an XrPassthroughPreferencesMETA structure

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_VALIDATION_FAILURE`

Version History

- Revision 1, 2023-04-25 (Johannes Schmid)
  - Initial extension description

12.162. XR_META_performance_metrics

Name String

`XR_META_performance_metrics`

Extension Type

Instance extension

Registered Extension Number

233

Revision

2

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Contributors

Xiang Wei, Meta Platforms

Overview

This extension provides APIs to enumerate and query performance metrics counters of the current XR device and XR application. Developers **can** perform performance analysis and do targeted optimization to the XR application using the performance metrics counters being collected. The application **should** not change its behavior based on the counter reads.

The performance metrics counters are organized into predefined `XrPath` values, under the root path */perfmetrics_meta.* An application **can** query the available counters through xrEnumeratePerformanceMetricsCounterPathsMETA. Here is a list of the performance metrics counter paths that **may** be provided on Meta devices:

- */perfmetrics_meta/app/cpu_frametime*
- */perfmetrics_meta/app/gpu_frametime*
- */perfmetrics_meta/app/motion_to_photon_latency*
- */perfmetrics_meta/compositor/cpu_frametime*
- */perfmetrics_meta/compositor/gpu_frametime*
- */perfmetrics_meta/compositor/dropped_frame_count*
- */perfmetrics_meta/compositor/spacewarp_mode*
- */perfmetrics_meta/device/cpu_utilization_average*
- */perfmetrics_meta/device/cpu_utilization_worst*
- */perfmetrics_meta/device/gpu_utilization*
- */perfmetrics_meta/device/cpu0_utilization* through */perfmetrics_meta/device/cpuX_utilization*

After a session is created, an application **can** use xrSetPerformanceMetricsStateMETA to enable the performance metrics system for that session. An application **can** use xrQueryPerformanceMetricsCounterMETA to query a performance metrics counter on a session that has the performance metrics system enabled, or use xrGetPerformanceMetricsStateMETA to query if the performance metrics system is enabled.

Note: the measurement intervals of individual performance metrics counters are defined by the OpenXR runtime. The application **must** not make assumptions or change its behavior at runtime by measuring them.

New Flag Types

```
// Provided by XR_META_performance_metrics
typedef XrFlags64 XrPerformanceMetricsCounterFlagsMETA;
```

```
// Provided by XR_META_performance_metrics
// Flag bits for XrPerformanceMetricsCounterFlagsMETA
static const XrPerformanceMetricsCounterFlagsMETA
XR_PERFORMANCE_METRICS_COUNTER_ANY_VALUE_VALID_BIT_META = 0x00000001;
static const XrPerformanceMetricsCounterFlagsMETA
XR_PERFORMANCE_METRICS_COUNTER_UINT_VALUE_VALID_BIT_META = 0x00000002;
static const XrPerformanceMetricsCounterFlagsMETA
XR_PERFORMANCE_METRICS_COUNTER_FLOAT_VALUE_VALID_BIT_META = 0x00000004;
```

Flag Descriptions

- XR_PERFORMANCE_METRICS_COUNTER_ANY_VALUE_VALID_BIT_META — Indicates any of the values in XrPerformanceMetricsCounterMETA is valid.
- XR_PERFORMANCE_METRICS_COUNTER_UINT_VALUE_VALID_BIT_META — Indicates the uintValue in XrPerformanceMetricsCounterMETA is valid.
- XR_PERFORMANCE_METRICS_COUNTER_FLOAT_VALUE_VALID_BIT_META — Indicates the floatValue in XrPerformanceMetricsCounterMETA is valid.

New Enum Constants

XrStructureType enumeration is extended with:

- XR_TYPE_PERFORMANCE_METRICS_STATE_META
- XR_TYPE_PERFORMANCE_METRICS_COUNTER_META

New Enums

```
// Provided by XR_META_performance_metrics
typedef enum XrPerformanceMetricsCounterUnitMETA {
    XR_PERFORMANCE_METRICS_COUNTER_UNIT_GENERIC_META = 0,
    XR_PERFORMANCE_METRICS_COUNTER_UNIT_PERCENTAGE_META = 1,
    XR_PERFORMANCE_METRICS_COUNTER_UNIT_MILLISECONDS_META = 2,
    XR_PERFORMANCE_METRICS_COUNTER_UNIT_BYTES_META = 3,
    XR_PERFORMANCE_METRICS_COUNTER_UNIT_HERTZ_META = 4,
    XR_PERFORMANCE_METRICS_COUNTER_UNIT_MAX_ENUM_META = 0x7FFFFFFF
} XrPerformanceMetricsCounterUnitMETA;
```

Enum	Description
XR_PERFORMANCE_METRICS_COUNTER_UNIT_GENERIC_META	the performance counter unit is generic (unspecified).
XR_PERFORMANCE_METRICS_COUNTER_UNIT_PERCENTAGE_META	the performance counter unit is percentage (%).
XR_PERFORMANCE_METRICS_COUNTER_UNIT_MILLISECONDS_META	the performance counter unit is millisecond.
XR_PERFORMANCE_METRICS_COUNTER_UNIT_BYTES_META	the performance counter unit is byte.
XR_PERFORMANCE_METRICS_COUNTER_UNIT_HERTZ_META	the performance counter unit is hertz (Hz).

New Structures

The XrPerformanceMetricsStateMETA structure is defined as:

```
// Provided by XR_META_performance_metrics
typedef struct XrPerformanceMetricsStateMETA {
    XrStructureType    type;
    const void*        next;
    XrBool32           enabled;
} XrPerformanceMetricsStateMETA;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- enabled is set to XR_TRUE to indicate the performance metrics system is enabled, XR_FALSE otherwise, when getting state. When setting state, set to XR_TRUE to enable the performance metrics system and XR_FALSE to disable it.

XrPerformanceMetricsStateMETA is provided as input when calling xrSetPerformanceMetricsStateMETA to enable or disable the performance metrics system. XrPerformanceMetricsStateMETA is populated as an output parameter when calling xrGetPerformanceMetricsStateMETA to query if the performance metrics system is enabled.

Valid Usage (Implicit)

- The XR_META_performance_metrics extension **must** be enabled prior to using XrPerformanceMetricsStateMETA
- type **must** be XR_TYPE_PERFORMANCE_METRICS_STATE_META
- next **must** be NULL or a valid pointer to the next structure in a structure chain

The XrPerformanceMetricsCounterMETA structure is defined as:

```
// Provided by XR_META_performance_metrics
typedef struct XrPerformanceMetricsCounterMETA {
    XrStructureType                         type;
    const void*                             next;
    XrPerformanceMetricsCounterFlagsMETA    counterFlags;
    XrPerformanceMetricsCounterUnitMETA     counterUnit;
    uint32_t                                uintValue;
    float                                   floatValue;
} XrPerformanceMetricsCounterMETA;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- counterFlags is a bitmask of XrPerformanceMetricsCounterFlagBitsMETA describing the validity of value members.
- counterUnit is a enum of XrPerformanceMetricsCounterUnitMETA describing the measurement unit.
- uintValue is the counter value in uint32_t format. It is valid if counterFlags contains XR_PERFORMANCE_METRICS_COUNTER_UINT_VALUE_VALID_BIT_META.
- floatValue is the counter value in float format. It is valid if counterFlags contains XR_PERFORMANCE_METRICS_COUNTER_FLOAT_VALUE_VALID_BIT_META.

XrPerformanceMetricsCounterMETA is populated by calling xrQueryPerformanceMetricsCounterMETA to query real-time performance metrics counter

information.

Valid Usage (Implicit)

- The XR_META_performance_metrics extension **must** be enabled prior to using XrPerformanceMetricsCounterMETA
- type **must** be XR_TYPE_PERFORMANCE_METRICS_COUNTER_META
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- counterFlags **must** be 0 or a valid combination of XrPerformanceMetricsCounterFlagBitsMETA values
- counterUnit **must** be a valid XrPerformanceMetricsCounterUnitMETA value

New Functions

The xrEnumeratePerformanceMetricsCounterPathsMETA function enumerates all performance metrics counter paths that supported by the runtime, it is defined as:

```
// Provided by XR_META_performance_metrics
XrResult xrEnumeratePerformanceMetricsCounterPathsMETA(
    XrInstance                                  instance,
    uint32_t                                    counterPathCapacityInput,
    uint32_t*                                   counterPathCountOutput,
    XrPath*                                     counterPaths);
```

Parameter Descriptions

- instance is an XrInstance handle previously created with xrCreateInstance.
- counterPathCapacityInput is the capacity of the counterPaths array, or 0 to indicate a request to retrieve the required capacity.
- counterPathCountOutput is filled in by the runtime with the count of counterPaths written or the required capacity in the case that counterPathCapacityInput is insufficient.
- counterPaths is an array of XrPath filled in by the runtime which contains all the available performance metrics counters, but **can** be NULL if counterPathCapacityInput is 0.
- See the Buffer Size Parameters section for a detailed description of retrieving the required counterPaths size.

Valid Usage (Implicit)

- The XR_META_performance_metrics extension **must** be enabled prior to calling xrEnumeratePerformanceMetricsCounterPathsMETA
- instance **must** be a valid XrInstance handle
- counterPathCountOutput **must** be a pointer to a uint32_t value
- If counterPathCapacityInput is not 0, counterPaths **must** be a pointer to an array of counterPathCapacityInput XrPath values

Return Codes

Success

- XR_SUCCESS

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SIZE_INSUFFICIENT
- XR_ERROR_VALIDATION_FAILURE

The xrSetPerformanceMetricsStateMETA function is defined as:

```
// Provided by XR_META_performance_metrics
XrResult xrSetPerformanceMetricsStateMETA(
    XrSession                                   session,
    const XrPerformanceMetricsStateMETA*        state);
```

Parameter Descriptions

- session is an XrSession handle previously created with xrCreateSession.
- state is a pointer to an XrPerformanceMetricsStateMETA structure.

The xrSetPerformanceMetricsStateMETA function enables or disables the performance metrics system.

Valid Usage (Implicit)

- The XR_META_performance_metrics extension **must** be enabled prior to calling xrSetPerformanceMetricsStateMETA
- session **must** be a valid XrSession handle
- state **must** be a pointer to a valid XrPerformanceMetricsStateMETA structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The xrGetPerformanceMetricsStateMETA function is defined as:

```
// Provided by XR_META_performance_metrics
XrResult xrGetPerformanceMetricsStateMETA(
    XrSession                                   session,
    XrPerformanceMetricsStateMETA*              state);
```

Parameter Descriptions

- session is an XrSession handle previously created with xrCreateSession.
- state is a pointer to an XrPerformanceMetricsStateMETA structure.

The xrGetPerformanceMetricsStateMETA function gets the current state of the performance metrics system.

Valid Usage (Implicit)

- The XR_META_performance_metrics extension **must** be enabled prior to calling xrGetPerformanceMetricsStateMETA
- session **must** be a valid XrSession handle
- state **must** be a pointer to an XrPerformanceMetricsStateMETA structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The xrQueryPerformanceMetricsCounterMETA function is defined as:

```
// Provided by XR_META_performance_metrics
XrResult xrQueryPerformanceMetricsCounterMETA(
    XrSession                                   session,
    XrPath                                      counterPath,
    XrPerformanceMetricsCounterMETA*            counter);
```

Parameter Descriptions

- session is an XrSession handle previously created with xrCreateSession.
- counterPath is a valid performance metrics counter path.
- counter is a pointer to an XrPerformanceMetricsCounterMETA structure.

The xrQueryPerformanceMetricsCounterMETA function queries a performance metrics counter.

The application **should** enable the performance metrics system (by calling xrSetPerformanceMetricsStateMETA) before querying metrics using xrQueryPerformanceMetricsCounterMETA. If the performance metrics system has not been enabled before calling xrQueryPerformanceMetricsCounterMETA, the runtime **must** return XR_ERROR_VALIDATION_FAILURE.

If counterPath is not in the list returned by xrEnumeratePerformanceMetricsCounterPathsMETA, the runtime must return XR_ERROR_PATH_UNSUPPORTED.

Valid Usage (Implicit)

- The XR_META_performance_metrics extension **must** be enabled prior to calling xrQueryPerformanceMetricsCounterMETA
- session **must** be a valid XrSession handle
- counter **must** be a pointer to an XrPerformanceMetricsCounterMETA structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_PATH_INVALID
- XR_ERROR_PATH_UNSUPPORTED
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

Issues

Version History

- Revision 1, 2022-04-28 (Xiang Wei)
  - Initial extension description
- Revision 2, 2022-09-16 (John Kearney)

- Clarification of error codes

12.163. XR_META_recommended_layer_resolution

Name String

`XR_META_recommended_layer_resolution`

Extension Type

Instance extension

Registered Extension Number

255

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Contributors

Rohit Rao Padebettu, Meta
Remi Palandri, Meta
Ben Cumings, Meta

Overview

The extension allows an application to request a recommended swapchain resolution from the runtime, in order to either allocate a swapchain of a more appropriate size, or to render into a smaller image rect according to the recommendation. For layers with multiple views such as XrCompositionLayerProjection, the application **may** scale the individual views to match the scaled swapchain resolution.

The runtime **may** use any factors to drive the recommendation it wishes to return to the application. Those include static properties such as screen resolution and HMD type, but also dynamic ones such as layer positioning and system-wide GPU utilization.

Application **may** also use this extension to allocate the swapchain by passing in a layer with a swapchain handle XR_NULL_HANDLE.

New Structures

The XrRecommendedLayerResolutionMETA structure is defined as:

```
// Provided by XR_META_recommended_layer_resolution
typedef struct XrRecommendedLayerResolutionMETA {
    XrStructureType    type;
    void*              next;
    XrExtent2Di        recommendedImageDimensions;
    XrBool32           isValid;
} XrRecommendedLayerResolutionMETA;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `recommendedImageDimensions` is the XrExtent2Di recommended image dimensions of the layer.
- `isValid` is the `XrBool32` boolean returned by the runtime which indicates whether the runtime returned a valid recommendation or does not have any recommendations to make.

If the runtime does not wish to make a recommendation, `isValid` **must** be `XR_FALSE` and `recommendedImageDimensions` **must** be {0,0}.

Valid Usage (Implicit)

- The `XR_META_recommended_layer_resolution` extension **must** be enabled prior to using XrRecommendedLayerResolutionMETA
- `type` **must** be `XR_TYPE_RECOMMENDED_LAYER_RESOLUTION_META`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

The XrRecommendedLayerResolutionGetInfoMETA structure is defined as:

```
// Provided by XR_META_recommended_layer_resolution
typedef struct XrRecommendedLayerResolutionGetInfoMETA {
    XrStructureType                         type;
    const void*                             next;
    const XrCompositionLayerBaseHeader*     layer;
    XrTime                                  predictedDisplayTime;
} XrRecommendedLayerResolutionGetInfoMETA;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `layer` is a pointer to a structure based on XrCompositionLayerBaseHeader, describing the layer for which the application wants a runtime-recommended swapchain resolution. Layers with multiple views **may** scale the views to match the scaled swapchain resolution.
- `predictedDisplayTime` is the `XrTime` that the application intends to submit the layer for.

If `predictedDisplayTime` is older than the predicted display time returned from most recent xrWaitFrame then, the runtime **must** return `XR_ERROR_TIME_INVALID`.

Valid Usage (Implicit)

- The `XR_META_recommended_layer_resolution` extension **must** be enabled prior to using XrRecommendedLayerResolutionGetInfoMETA
- `type` **must** be `XR_TYPE_RECOMMENDED_LAYER_RESOLUTION_GET_INFO_META`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `layer` **must** be a pointer to a valid XrCompositionLayerBaseHeader-based structure. See also: XrCompositionLayerCubeKHR, XrCompositionLayerCylinderKHR, XrCompositionLayerEquirect2KHR, XrCompositionLayerEquirectKHR, XrCompositionLayerPassthroughANDROID, XrCompositionLayerPassthroughFB, XrCompositionLayerPassthroughHTC, XrCompositionLayerProjection, XrCompositionLayerQuad

New Functions

The xrGetRecommendedLayerResolutionMETA function is defined as:

```
// Provided by XR_META_recommended_layer_resolution
XrResult xrGetRecommendedLayerResolutionMETA(
    XrSession                                   session,
    const XrRecommendedLayerResolutionGetInfoMETA* info,
    XrRecommendedLayerResolutionMETA*           resolution);
```

Parameter Descriptions

- `session` is the XrSession in which the recommendation is made.
- `info` is a pointer to an XrRecommendedLayerResolutionGetInfoMETA structure containing the details of the layer for which the application is requesting a recommendation.
- `resolution` is a pointer to an XrRecommendedLayerResolutionMETA that the runtime will populate.

The xrGetRecommendedLayerResolutionMETA function returns the recommendation that the runtime wishes to make to the application for the layer provided in the XrRecommendedLayerResolutionGetInfoMETA structure. Application **may** choose to reallocate their swapchain or scale view resolution accordingly. Applications rendering multiple views into the swapchain **may** scale individual views to match the recommended swapchain resolution.

The runtime **may** not wish to make any recommendation, in which case it **must** return an XrRecommendedLayerResolutionMETA::`isValid` value of `XR_FALSE`.

If the XrRecommendedLayerResolutionGetInfoMETA::`layer` attribute of the `info` argument of the function contains valid swapchain handles in all fields where required, the runtime **must** return a resolution recommendation which is less than or equal to the size of that swapchain, so that the application **may** render into an existing swapchain or swapchains without reallocation. As an exception to valid usage, an otherwise-valid structure passed as XrRecommendedLayerResolutionGetInfoMETA::`layer` **may** contain XR_NULL_HANDLE in place of valid XrSwapchain handle(s) for this function only, to obtain a recommended resolution resolution for the purpose of allocating a swapchain. If at least one otherwise-required XrSwapchain handle within XrRecommendedLayerResolutionGetInfoMETA::`layer` is XR_NULL_HANDLE, the runtime **must** interpret this as a request for recommended resolution without limitation to the allocated size of any existing swapchain.

If the runtime makes a recommendation, it **should** make a recommendation that is directly usable by the application to render its frames without creating adverse visual effects for the user.

Issues

1. Should this extension be leveraging events instead of being queried potentially every frame?

   RESOLVED: Yes.

   We want to provide the runtime the flexibility to smoothly transition the application from one resolution to another in a dynamic resolution usecase without any reallocation. To do so with an event system would send an event every frame which we preferred to avoid.

Version History

- Revision 1, 2023-12-10 (Remi Palandri)
    - Initial extension description

12.164. XR_META_simultaneous_hands_and_controllers

Name String

`XR_META_simultaneous_hands_and_controllers`

Extension Type

Instance extension

Registered Extension Number

533

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Last Modified Date

2025-05-07

IP Status

No known IP claims.

Contributors

Matthew Langille, Meta Platforms

12.164.1. Overview

Some XR systems have the ability to track both hands and controllers simultaneously (commonly referred to as multimodal hands and controllers input), but this may consume additional power and system resources. This extension defines two new functions that applications **can** use to control when the simultaneous hands and controller tracking is activated.

12.164.2. Inspect System Capability

The XrSystemSimultaneousHandsAndControllersPropertiesMETA structure is defined as:

```
// Provided by XR_META_simultaneous_hands_and_controllers
typedef struct XrSystemSimultaneousHandsAndControllersPropertiesMETA {
    XrStructureType    type;
    void*              next;
    XrBool32           supportsSimultaneousHandsAndControllers;
} XrSystemSimultaneousHandsAndControllersPropertiesMETA;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension. For more details about the structure chain, see the structure being extended (XrSystemProperties).
- `supportsSimultaneousHandsAndControllers` is an `XrBool32`, indicating if current system is capable of simultaneous hands and controller tracking.

An application **can** inspect whether the system is capable of enabling simultaneous hands and controller tracking by extending the XrSystemProperties with XrSystemSimultaneousHandsAndControllersPropertiesMETA structure when calling xrGetSystemProperties.

If and only if a runtime returns `XR_FALSE` for `supportsSimultaneousHandsAndControllers`, the runtime **must** return `XR_ERROR_FEATURE_UNSUPPORTED` from xrResumeSimultaneousHandsAndControllersTrackingMETA and xrPauseSimultaneousHandsAndControllersTrackingMETA.

Valid Usage (Implicit)

- The `XR_META_simultaneous_hands_and_controllers` extension **must** be enabled prior to using XrSystemSimultaneousHandsAndControllersPropertiesMETA
- `type` **must** be `XR_TYPE_SYSTEM_SIMULTANEOUS_HANDS_AND_CONTROLLERS_PROPERTIES_META`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

12.164.3. Enable Simultaneous Tracking

The xrResumeSimultaneousHandsAndControllersTrackingMETA function is defined as:

```
// Provided by XR_META_simultaneous_hands_and_controllers
XrResult xrResumeSimultaneousHandsAndControllersTrackingMETA(
    XrSession                                           session,
    const XrSimultaneousHandsAndControllersTrackingResumeInfoMETA* resumeInfo);
```

Parameter Descriptions

- `session` is an XrSession handle previously created with xrCreateSession
- `resumeInfo` is a pointer to an instance of the XrSimultaneousHandsAndControllersTrackingResumeInfoMETA structure containing parameters affecting how the simultaneous hands and controllers mode is enabled

An application **can** call xrResumeSimultaneousHandsAndControllersTrackingMETA to enable simultaneous hands and controller tracking.

Runtimes **must** initialize the simultaneous tracking feature in a paused state, so applications call the resume function for the simultaneous tracking to start.

If xrResumeSimultaneousHandsAndControllersTrackingMETA is called when the feature is already in a resumed state, the runtime **must** return `XR_SUCCESS`.

If a system supports detection of whether a controller is currently held by the user, the runtime **should** represent this transition by switching the active interaction profile in the relevant hand from the active controller interaction profile to an interaction profile representing hands if available.

For example, the returned interaction from xrGetCurrentInteractionProfile on */user/hand/left* might change from */interaction_profiles/facebook/touch_controller_pro* to */interaction_profiles/ext/hand_interaction_ext*, generating an `XR_TYPE_EVENT_DATA_INTERACTION_PROFILE_CHANGED` (XrEventDataInteractionProfileChanged) event, assuming that bindings were suggested for both of those interaction profiles on that path.

This would indicate that the user put down the controller and is no longer holding a controller in their left hand. Note that in this example, if the user is holding a controller in their right hand, xrGetCurrentInteractionProfile on */user/hand/right* still returns */interaction_profiles/facebook/touch_controller_pro*.

Also note that if the `XR_META_detached_controllers` extension is used alongside `XR_META_simultaneous_hands_and_controllers`, then when a controller leaves the hand, it **may** report that interaction profile as current for the corresponding */user/detached_controller_meta* path if bindings have been suggested for that path. See `XR_META_simultaneous_hands_and_controllers` for details.

When simultaneous tracking is resumed, runtimes **should** allow xrGetCurrentInteractionProfile to return different interaction profiles for different top level */user* paths (e.g. */user/hand/left* and

/user/hand/right). Note that this behavior is already allowed by the specification, even without this extension, but runtimes exposing this extension **may** choose to not expose different interaction profiles for different top level */user* paths unless it is enabled.

When a held controller transitions to an unheld state, the current interaction profile for the relevant top level path **must** change to an interaction profile representing hand tracking, if available.

Valid Usage (Implicit)

- The `XR_META_simultaneous_hands_and_controllers` extension **must** be enabled prior to calling xrResumeSimultaneousHandsAndControllersTrackingMETA
- `session` **must** be a valid XrSession handle
- `resumeInfo` **must** be a pointer to a valid XrSimultaneousHandsAndControllersTrackingResumeInfoMETA structure

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_FEATURE_UNSUPPORTED`
- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_VALIDATION_FAILURE`

The XrSimultaneousHandsAndControllersTrackingResumeInfoMETA structure is defined as:

```
// Provided by XR_META_simultaneous_hands_and_controllers
typedef struct XrSimultaneousHandsAndControllersTrackingResumeInfoMETA {
    XrStructureType    type;
    const void*        next;
} XrSimultaneousHandsAndControllersTrackingResumeInfoMETA;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.

This structure only exists to point to future extension structures.

Valid Usage (Implicit)

- The `XR_META_simultaneous_hands_and_controllers` extension **must** be enabled prior to using XrSimultaneousHandsAndControllersTrackingResumeInfoMETA
- `type` **must** be `XR_TYPE_SIMULTANEOUS_HANDS_AND_CONTROLLERS_TRACKING_RESUME_INFO_META`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

12.164.4. Disable Simultaneous Tracking

The xrPauseSimultaneousHandsAndControllersTrackingMETA function is defined as:

```
// Provided by XR_META_simultaneous_hands_and_controllers
XrResult xrPauseSimultaneousHandsAndControllersTrackingMETA(
    XrSession                                   session,
    const XrSimultaneousHandsAndControllersTrackingPauseInfoMETA* pauseInfo);
```

Parameter Descriptions

- `session` is an XrSession handle previously created with xrCreateSession
- `pauseInfo` is a pointer to an instance of the XrSimultaneousHandsAndControllersTrackingPauseInfoMETA structure containing parameters affecting how the simultaneous hands and controllers mode is to be paused

An application **can** call xrPauseSimultaneousHandsAndControllersTrackingMETA to disable simultaneous hands and controller tracking.

If xrPauseSimultaneousHandsAndControllersTrackingMETA is called when the feature is not running, the runtime **must** return `XR_SUCCESS`.

Tracking systems consume system resources and it is desirable to be able to stop them when they are not in use; a strong motivation for this extension is that it provides the ability for clients to dynamically switch to a multiple tracking system operating mode only as needed, thus preserving

system resources and improving battery performance.

Valid Usage (Implicit)

- The XR_META_simultaneous_hands_and_controllers extension **must** be enabled prior to calling xrPauseSimultaneousHandsAndControllersTrackingMETA
- session **must** be a valid XrSession handle
- pauseInfo **must** be a pointer to a valid XrSimultaneousHandsAndControllersTrackingPauseInfoMETA structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FEATURE_UNSUPPORTED
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The XrSimultaneousHandsAndControllersTrackingPauseInfoMETA structure is defined as:

```
// Provided by XR_META_simultaneous_hands_and_controllers
typedef struct XrSimultaneousHandsAndControllersTrackingPauseInfoMETA {
    XrStructureType    type;
    const void*        next;
} XrSimultaneousHandsAndControllersTrackingPauseInfoMETA;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.

This structure only exists to point to future extension structures.

Valid Usage (Implicit)

- The `XR_META_simultaneous_hands_and_controllers` extension **must** be enabled prior to using XrSimultaneousHandsAndControllersTrackingPauseInfoMETA
- `type` **must** be `XR_TYPE_SIMULTANEOUS_HANDS_AND_CONTROLLERS_TRACKING_PAUSE_INFO_META`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

12.164.5. New Commands

- xrPauseSimultaneousHandsAndControllersTrackingMETA
- xrResumeSimultaneousHandsAndControllersTrackingMETA

12.164.6. New Structures

- XrSimultaneousHandsAndControllersTrackingPauseInfoMETA
- XrSimultaneousHandsAndControllersTrackingResumeInfoMETA
- Extending XrSystemProperties:
  - XrSystemSimultaneousHandsAndControllersPropertiesMETA

12.164.7. New Enum Constants

- `XR_META_SIMULTANEOUS_HANDS_AND_CONTROLLERS_EXTENSION_NAME`
- `XR_META_simultaneous_hands_and_controllers_SPEC_VERSION`
- Extending XrStructureType:
  - `XR_TYPE_SIMULTANEOUS_HANDS_AND_CONTROLLERS_TRACKING_PAUSE_INFO_META`
  - `XR_TYPE_SIMULTANEOUS_HANDS_AND_CONTROLLERS_TRACKING_RESUME_INFO_META`
  - `XR_TYPE_SYSTEM_SIMULTANEOUS_HANDS_AND_CONTROLLERS_PROPERTIES_META`

12.164.8. Issues

12.164.9. Version History

- Revision 1, 2025-05-07 (Matthew Langille)
  - Initial extension description

12.165. XR_META_spatial_entity_discovery

Name String

`XR_META_spatial_entity_discovery`

Extension Type

Instance extension

Registered Extension Number

248

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Last Modified Date

2024-11-10

IP Status

No known IP claims.

Contributors

Natalie Fleury, Meta Platforms
Abhishek Shrivastava, Meta Platforms
Adrian Mancilla, Meta Platforms

12.165.1. Overview

The `XR_META_spatial_entity_discovery` extension supports spatial entity retrieval in larger areas. It offers entity filters like component and UUID filters. This allows for more efficient and targeted discovery of spatial entities.

Technical overview

This extension enables finding and loading persisted Spatial Entities which **can** then be tracked across different sessions and over time by applications.

If the XR_SPACE_COMPONENT_TYPE_STORABLE_FB component has been enabled on a space, and that space has previously been persisted, application developers **can** discover and then track this XrSpace entity. XR_META_spatial_entity_discovery is expected to be used alongside XR_META_spatial_entity_persistence for space persistence and storage management.



XR_META_spatial_entity_discovery is not expected to be used alongside XR_FB_spatial_entity_sharing, and applications should not try to retrieve shared Spatial Entities via XR_META_spatial_entity_discovery, and should instead use XR_FB_spatial_entity_query to retrieve shared Spatial Entities.

12.165.2. Inspect System Capability

The XrSystemSpaceDiscoveryPropertiesMETA structure is defined as:

```
// Provided by XR_META_spatial_entity_discovery
typedef struct XrSystemSpaceDiscoveryPropertiesMETA {
    XrStructureType    type;
    const void*        next;
    XrBool32           supportsSpaceDiscovery;
} XrSystemSpaceDiscoveryPropertiesMETA;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension. For more details about the structure chain, see the structure being extended (XrSystemProperties).
- supportsSpaceDiscovery is an XrBool32, indicating if current system supports space discovery.

An application **can** inspect whether the system is capable of supporting space discovery by extending the XrSystemProperties with XrSystemSpaceDiscoveryPropertiesMETA structure when calling xrGetSystemProperties.

If and only if a runtime returns XR_FALSE for supportsSpaceDiscovery, the runtime **must** return XR_ERROR_FEATURE_UNSUPPORTED from xrDiscoverSpacesMETA and xrRetrieveSpaceDiscoveryResultsMETA.

Valid Usage (Implicit)

- The `XR_META_spatial_entity_discovery` extension **must** be enabled prior to using XrSystemSpaceDiscoveryPropertiesMETA
- `type` **must** be `XR_TYPE_SYSTEM_SPACE_DISCOVERY_PROPERTIES_META`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

12.165.3. Discover Spaces

The xrDiscoverSpacesMETA function is defined as:

```
// Provided by XR_META_spatial_entity_discovery
XrResult xrDiscoverSpacesMETA(
    XrSession                                   session,
    const XrSpaceDiscoveryInfoMETA*             info,
    XrAsyncRequestIdFB*                         requestId);
```

Parameter Descriptions

- `session` is a handle to an XrSession.
- `info` contains the input struct XrSpaceDiscoveryInfoMETA for the Discovery operation.
- `requestId` is a pointer to an `XrAsyncRequestIdFB` value and is an output parameter. The variable it points to is populated with the ID of this asynchronous request.

The xrDiscoverSpacesMETA function discovers spaces that were persisted previously, and which comply with the filters passed in the XrSpaceDiscoveryInfoMETA `info` structure.

This operation is asynchronous.

If xrDiscoverSpacesMETA returns a XR_FAILED result, no discovery operation takes place and no events will be queued for this operation.

If the asynchronous operation is scheduled successfully, the runtime **must** return `XR_SUCCESS` and the asynchronous discovery operation will queue at least 1 event.

If Spatial Entities have been discovered and are ready for retrieval, the runtime **must** queue an XrEventDataSpaceDiscoveryResultsAvailableMETA event. The runtime **may** queue 0, 1, or more XrEventDataSpaceDiscoveryResultsAvailableMETA events depending on the Spatial Entities found.

If and only if the runtime returns `XR_SUCCESS`, the runtime **must** queue a single XrEventDataSpaceDiscoveryCompleteMETA event identified with a

XrEventDataSpaceDiscoveryCompleteMETA::`requestId` matching the `requestId` value output by this function, referred to as the "corresponding completion event." The XrEventDataSpaceDiscoveryCompleteMETA event is queued after all XrEventDataSpaceDiscoveryResultsAvailableMETA events for this operation have been queued.

Completion results are conveyed in the event XrEventDataSpaceDiscoveryCompleteMETA, while availability of output for xrRetrieveSpaceDiscoveryResultsMETA is signaled by either this completion event or the event XrEventDataSpaceDiscoveryResultsAvailableMETA.

If the asynchronous operation is successful, in the corresponding completion event, the runtime **must** set the XrEventDataSpaceDiscoveryCompleteMETA::`result` field to `XR_SUCCESS`.

If the asynchronous operation is scheduled but not successful, in the corresponding completion event, the runtime **must** set the XrEventDataSpaceDiscoveryCompleteMETA::`result` field to an appropriate error code instead of `XR_SUCCESS`.

Valid Usage (Implicit)

- The `XR_META_spatial_entity_discovery` extension **must** be enabled prior to calling xrDiscoverSpacesMETA
- `session` **must** be a valid XrSession handle
- `info` **must** be a pointer to a valid XrSpaceDiscoveryInfoMETA structure
- `requestId` **must** be a pointer to an `XrAsyncRequestIdFB` value

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INITIALIZATION_FAILED
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_SESSION_NOT_RUNNING
- XR_ERROR_SIZE_INSUFFICIENT
- XR_ERROR_SPACE_RATE_LIMITED_META
- XR_ERROR_VALIDATION_FAILURE

The XrSpaceDiscoveryInfoMETA structure is defined as:

```
// Provided by XR_META_spatial_entity_discovery
typedef struct XrSpaceDiscoveryInfoMETA {
    XrStructureType                                 type;
    const void*                                     next;
    uint32_t                                        filterCount;
    const XrSpaceFilterBaseHeaderMETA* const *      filters;
} XrSpaceDiscoveryInfoMETA;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- filterCount is the number of filters that are passed in the filters parameter.
- filters is a pointer to an array of XrSpaceFilterBaseHeaderMETA pointers.

The XrSpaceDiscoveryInfoMETA structure contains information used to discover space(s).

Valid Usage (Implicit)

- The XR_META_spatial_entity_discovery extension **must** be enabled prior to using XrSpaceDiscoveryInfoMETA
- type **must** be XR_TYPE_SPACE_DISCOVERY_INFO_META
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- If filterCount is not 0, filters **must** be a pointer to an array of filterCount valid XrSpaceFilterBaseHeaderMETA-based structures. See also: XrSpaceFilterComponentMETA, XrSpaceFilterUuidMETA

The XrSpaceFilterBaseHeaderMETA structure is defined as:

```
// Provided by XR_META_spatial_entity_discovery
typedef struct XrSpaceFilterBaseHeaderMETA {
    XrStructureType    type;
    const void*        next;
} XrSpaceFilterBaseHeaderMETA;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next **must** be NULL

The XrSpaceFilterBaseHeaderMETA structure is meant to be used as the base header for filter types defined in this extension.

Valid Usage (Implicit)

- The XR_META_spatial_entity_discovery extension **must** be enabled prior to using XrSpaceFilterBaseHeaderMETA
- type **must** be one of the following XrStructureType values: XR_TYPE_SPACE_FILTER_COMPONENT_META, XR_TYPE_SPACE_FILTER_UUID_META
- next **must** be NULL or a valid pointer to the next structure in a structure chain

Two such filter types are defined in this extension: XrSpaceFilterUuidMETA and XrSpaceFilterComponentMETA.

The XrSpaceFilterUuidMETA structure is defined as:

```
// Provided by XR_META_spatial_entity_discovery
typedef struct XrSpaceFilterUuidMETA {
    XrStructureType     type;
    const void*         next;
    uint32_t            uuidCount;
    const XrUuidEXT*    uuids;
} XrSpaceFilterUuidMETA;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` **must** be `NULL`
- `uuidCount` is the number of IDs that **should** be in the uuids array.
- `uuids` is the XrUuidEXT array (pointer to first element) to the IDs to be discovered.

The XrSpaceFilterUuidMETA structure contains information used to discover space(s) by ID.

Valid Usage (Implicit)

- The `XR_META_spatial_entity_discovery` extension **must** be enabled prior to using XrSpaceFilterUuidMETA
- `type` **must** be `XR_TYPE_SPACE_FILTER_UUID_META`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `uuids` **must** be a pointer to an array of `uuidCount` XrUuidEXT structures
- The `uuidCount` parameter **must** be greater than `0`

The XrSpaceFilterComponentMETA structure is defined as:

```
// Provided by XR_META_spatial_entity_discovery
typedef struct XrSpaceFilterComponentMETA {
    XrStructureType           type;
    const void*               next;
    XrSpaceComponentTypeFB    componentType;
} XrSpaceFilterComponentMETA;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` **must** be `NULL`
- `componentType` is the XrSpaceComponentTypeFB component that is expected to be enabled for the spaces returned by discovery.

The XrSpaceFilterComponentMETA structure contains information used to discover Spatial Entities by components enabled.

Valid Usage (Implicit)

- The `XR_META_spatial_entity_discovery` extension **must** be enabled prior to using XrSpaceFilterComponentMETA
- `type` **must** be `XR_TYPE_SPACE_FILTER_COMPONENT_META`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `componentType` **must** be a valid XrSpaceComponentTypeFB value

The XrEventDataSpaceDiscoveryResultsAvailableMETA structure is defined as:

```
// Provided by XR_META_spatial_entity_discovery
typedef struct XrEventDataSpaceDiscoveryResultsAvailableMETA {
    XrStructureType       type;
    const void*           next;
    XrAsyncRequestIdFB    requestId;
} XrEventDataSpaceDiscoveryResultsAvailableMETA;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `requestId` is the `XrAsyncRequestIdFB` of the asynchronous request to discover spaces.

The XrEventDataSpaceDiscoveryResultsAvailableMETA event indicates there are results ready to be retrieved. Any number of these XrEventDataSpaceDiscoveryResultsAvailableMETA events **may** be queued, until the XrEventDataSpaceDiscoveryCompleteMETA event is queued. Once the XrEventDataSpaceDiscoveryCompleteMETA event is queued, there **must** not be any more

XrEventDataSpaceDiscoveryResultsAvailableMETA events queued.

Valid Usage (Implicit)

- The XR_META_spatial_entity_discovery extension **must** be enabled prior to using XrEventDataSpaceDiscoveryResultsAvailableMETA
- type **must** be XR_TYPE_EVENT_DATA_SPACE_DISCOVERY_RESULTS_AVAILABLE_META
- next **must** be NULL or a valid pointer to the next structure in a structure chain

The XrEventDataSpaceDiscoveryCompleteMETA structure is defined as:

```
// Provided by XR_META_spatial_entity_discovery
typedef struct XrEventDataSpaceDiscoveryCompleteMETA {
    XrStructureType       type;
    const void*           next;
    XrAsyncRequestIdFB    requestId;
    XrResult              result;
} XrEventDataSpaceDiscoveryCompleteMETA;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- requestId is the XrAsyncRequestIdFB of the asynchronous request to discover spaces.
- result the XrResult indicating the success, success with warnings, or failure of the Discovery operation.

The XrEventDataSpaceDiscoveryCompleteMETA event indicates that there **must** be no more XrEventDataSpaceDiscoveryResultsAvailableMETA queued and that Discovery has completed execution.

Potential XrEventDataSpaceDiscoveryCompleteMETA::result values include the following XrResult enumerants:

- XR_SUCCESS
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SPACE_INSUFFICIENT_RESOURCES_META
- XR_ERROR_SPACE_INSUFFICIENT_VIEW_META

- XR_ERROR_SPACE_PERMISSION_INSUFFICIENT_META
- XR_ERROR_SPACE_RATE_LIMITED_META
- XR_ERROR_SPACE_TOO_DARK_META
- XR_ERROR_SPACE_TOO_BRIGHT_META

Valid Usage (Implicit)

- The XR_META_spatial_entity_discovery extension **must** be enabled prior to using XrEventDataSpaceDiscoveryCompleteMETA
- type **must** be XR_TYPE_EVENT_DATA_SPACE_DISCOVERY_COMPLETE_META
- next **must** be NULL or a valid pointer to the next structure in a structure chain

12.165.4. Retrieve Discovered Spaces

The xrRetrieveSpaceDiscoveryResultsMETA function is defined as:

```
// Provided by XR_META_spatial_entity_discovery
XrResult xrRetrieveSpaceDiscoveryResultsMETA(
    XrSession                                   session,
    XrAsyncRequestIdFB                          requestId,
    XrSpaceDiscoveryResultsMETA*                results);
```

Parameter Descriptions

- session is a handle to an XrSession.
- requestId is the XrAsyncRequestIdFB returned as part of the XrEventDataSpaceDiscoveryResultsAvailableMETA or XrEventDataSpaceDiscoveryCompleteMETA events.
- results is the XrSpaceDiscoveryResultsMETA structure used to get the number of results then to populate those results.

The xrRetrieveSpaceDiscoveryResultsMETA function is synchronous, and follows the 2-call idiom, where the first call is made to determine the number of results, populated in the XrSpaceDiscoveryResultsMETA::resultCountOutput. The application uses this value to initialize the XrSpaceDiscoveryResultsMETA::resultCapacityInput and XrSpaceDiscoveryResultsMETA::results fields. The second call to xrRetrieveSpaceDiscoveryResultsMETA **must** then populate the results field with whatever results are available. See Buffer Size Parameters for a detailed description of retrieving the required results size.

Note that after any results have been retrieved, those specific results will be unavailable for retrieval again. Application developers **can** choose to retrieve discovered Spatial Entities either after receiving an XrEventDataSpaceDiscoveryResultsAvailableMETA event or after receiving an XrEventDataSpaceDiscoveryCompleteMETA event. If application developers choose to retrieve after XrEventDataSpaceDiscoveryResultsAvailableMETA events (before the XrEventDataSpaceDiscoveryCompleteMETA event), more results **may** be discovered between the first and the second call to xrRetrieveSpaceDiscoveryResultsMETA. If this occurs, and if the application chose the result array capacity to match the XrSpaceDiscoveryResultsMETA::`resultCountOutput`, that capacity is no longer sufficient to receive all available results, so the second call will fail due to insufficient size. This will not lose results, and the application **can** begin the two-call process for xrRetrieveSpaceDiscoveryResultsMETA again.

xrPollEvent and xrRetrieveSpaceDiscoveryResultsMETA **may** be called simultaneously (without external synchronization).

Valid Usage (Implicit)

- The `XR_META_spatial_entity_discovery` extension **must** be enabled prior to calling xrRetrieveSpaceDiscoveryResultsMETA
- `session` **must** be a valid XrSession handle
- `results` **must** be a pointer to an XrSpaceDiscoveryResultsMETA structure

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_SIZE_INSUFFICIENT`
- `XR_ERROR_VALIDATION_FAILURE`

The XrSpaceDiscoveryResultsMETA structure is defined as:

```
// Provided by XR_META_spatial_entity_discovery
typedef struct XrSpaceDiscoveryResultsMETA {
    XrStructureType                type;
    const void*                    next;
    uint32_t                       resultCapacityInput;
    uint32_t                       resultCountOutput;
    XrSpaceDiscoveryResultMETA*    results;
} XrSpaceDiscoveryResultsMETA;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `resultCapacityInput` is the capacity of the results array, or 0 to indicate a request to retrieve the required capacity.
- `resultCountOutput` is an output parameter containing the count of results for retrieval, or returns the required capacity in the case that resultCapacityInput is insufficient.
- `results` is a pointer to an array of XrSpaceDiscoveryResultMETA structures to populate with the spaces retrieved, but **can** be `NULL` if `resultCapacityInput` is `0`.
- See the Buffer Size Parameters section for a detailed description of retrieving the required `results` size.

The XrSpaceDiscoveryResultsMETA structure is used to retrieve all results from a Discovery operation.

Valid Usage (Implicit)

- The `XR_META_spatial_entity_discovery` extension **must** be enabled prior to using XrSpaceDiscoveryResultsMETA
- `type` **must** be `XR_TYPE_SPACE_DISCOVERY_RESULTS_META`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- If `resultCapacityInput` is not `0`, `results` **must** be a pointer to an array of `resultCapacityInput` XrSpaceDiscoveryResultMETA structures

The XrSpaceDiscoveryResultMETA structure is defined as:

```
// Provided by XR_META_spatial_entity_discovery
typedef struct XrSpaceDiscoveryResultMETA {
    XrSpace      space;
    XrUuidEXT    uuid;
} XrSpaceDiscoveryResultMETA;
```

Member Descriptions

- `space` is a space that complies with all the filters provided to Discovery call.
- `uuid` is the XrUuidEXT of the space.

The XrSpaceDiscoveryResultMETA structure contains a single Space result retrieved after Discovery returns results.

Valid Usage (Implicit)

- The `XR_META_spatial_entity_discovery` extension **must** be enabled prior to using XrSpaceDiscoveryResultMETA

12.165.5. New Commands

- xrDiscoverSpacesMETA
- xrRetrieveSpaceDiscoveryResultsMETA

12.165.6. New Structures

- XrEventDataSpaceDiscoveryCompleteMETA
- XrEventDataSpaceDiscoveryResultsAvailableMETA
- XrSpaceDiscoveryInfoMETA
- XrSpaceDiscoveryResultMETA
- XrSpaceDiscoveryResultsMETA
- XrSpaceFilterBaseHeaderMETA
- XrSpaceFilterComponentMETA
- XrSpaceFilterUuidMETA
- Extending XrSystemProperties:
  - XrSystemSpaceDiscoveryPropertiesMETA

12.165.7. New Enum Constants

- XR_META_SPATIAL_ENTITY_DISCOVERY_EXTENSION_NAME
- XR_META_spatial_entity_discovery_SPEC_VERSION
- Extending XrStructureType:
  - XR_TYPE_EVENT_DATA_SPACE_DISCOVERY_COMPLETE_META
  - XR_TYPE_EVENT_DATA_SPACE_DISCOVERY_RESULTS_AVAILABLE_META
  - XR_TYPE_SPACE_DISCOVERY_INFO_META
  - XR_TYPE_SPACE_DISCOVERY_RESULTS_META
  - XR_TYPE_SPACE_DISCOVERY_RESULT_META
  - XR_TYPE_SPACE_FILTER_COMPONENT_META
  - XR_TYPE_SPACE_FILTER_UUID_META
  - XR_TYPE_SYSTEM_SPACE_DISCOVERY_PROPERTIES_META

12.165.8. Issues

12.165.9. Version History

- Revision 1, 2024-11-10 (Natalie Fleury)
  - Initial draft

12.166. XR_META_spatial_entity_group_sharing

Name String

XR_META_spatial_entity_group_sharing

Extension Type

Instance extension

Registered Extension Number

573

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

API Interactions

- Interacts with `XR_FB_spatial_entity_query`
- Interacts with `XR_META_spatial_entity_sharing`

Last Modified Date

2024-06-14

IP Status

No known IP claims.

Contributors

TJ Gilbrough, Meta Platforms
Jiawen Zhang, Meta Platforms
Scott Dewald, Meta Platforms
Ribel Fares, Meta Platforms

12.166.1. Overview

The `XR_META_spatial_entity_group_sharing` extension enables applications to share spatial entities to an application-specified group UUID. An application **can** share spatial entities with one or more group UUIDs, and query for spatial entities previously shared with a group UUID. A Group UUID is any application provided UUID. The Group will be established for the application simply by sharing spatial entities to it.

12.166.2. Check compatibility

The XrSystemSpatialEntityGroupSharingPropertiesMETA structure is defined as:

```
// Provided by XR_META_spatial_entity_group_sharing
typedef struct XrSystemSpatialEntityGroupSharingPropertiesMETA {
    XrStructureType    type;
    void*              next;
    XrBool32           supportsSpatialEntityGroupSharing;
} XrSystemSpatialEntityGroupSharingPropertiesMETA;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `supportsSpatialEntityGroupSharing` is an `XrBool32` specifying if group sharing is supported.

An application **can** inspect whether the system is capable of group based sharing by extending the

XrSystemProperties with XrSystemSpatialEntityGroupSharingPropertiesMETA structure when calling xrGetSystemProperties.

In order to use XrShareSpacesRecipientGroupsMETA with xrShareSpacesMETA, the system **must** also support XR_META_spatial_entity_sharing. Please see XR_META_spatial_entity_sharing's section for how to check if XR_META_spatial_entity_sharing is supported.

In order to use XrSpaceGroupUuidFilterInfoMETA with xrQuerySpacesFB, the system **must** also support XR_FB_spatial_entity_query. Please see XR_FB_spatial_entity_query's section for how to check if XR_FB_spatial_entity_query is supported.

Valid Usage (Implicit)

- The XR_META_spatial_entity_group_sharing extension **must** be enabled prior to using XrSystemSpatialEntityGroupSharingPropertiesMETA
- type **must** be XR_TYPE_SYSTEM_SPATIAL_ENTITY_GROUP_SHARING_PROPERTIES_META
- next **must** be NULL or a valid pointer to the next structure in a structure chain

12.166.3. Sharing Spaces to a Group UUID

An application **can** share spatial entities with any application provided UUID.

Once spatial entities are shared with this group UUID, the application **can** query for spatial entities previously shared with this group UUID.

Applications **can** share multiple spatial entities to the same group UUID.

Spatial entities will remain shared with the Group UUID for 30 days since the last successful share.

Any logged-in user using the same application may query for spatial entities shared with the Group UUID.

The XrShareSpacesRecipientGroupsMETA structure is defined as:

```
// Provided by XR_META_spatial_entity_group_sharing with XR_META_spatial_entity_sharing
typedef struct XrShareSpacesRecipientGroupsMETA {
    XrStructureType    type;
    const void*        next;
    uint32_t           groupCount;
    XrUuid*            groups;
} XrShareSpacesRecipientGroupsMETA;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR.
- `groupCount` is the number of elements in the `groups` list. The maximum supported count in a single request is 16.
- `groups` is a list of XrUuid defined by the application, which the spaces will be shared with.

XrShareSpacesRecipientGroupsMETA implements the XrShareSpacesRecipientBaseHeaderMETA base type. Where xrShareSpacesMETA specifies that a valid structure based on XrShareSpacesRecipientBaseHeaderMETA is to be passed, a valid XrShareSpacesRecipientGroupsMETA structure **may** be passed.

Valid Usage (Implicit)

- The `XR_META_spatial_entity_group_sharing` extension **must** be enabled prior to using XrShareSpacesRecipientGroupsMETA
- `type` **must** be `XR_TYPE_SHARE_SPACES_RECIPIENT_GROUPS_META`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `groups` **must** be a pointer to an array of `groupCount` XrUuid structures
- The `groupCount` parameter **must** be greater than `0`

12.166.4. Query Spaces Shared with a Group UUID

The XrSpaceGroupUuidFilterInfoMETA structure is defined as:

```
// Provided by XR_FB_spatial_entity_query with XR_META_spatial_entity_group_sharing
typedef struct XrSpaceGroupUuidFilterInfoMETA {
    XrStructureType    type;
    const void*        next;
    XrUuid             groupUuid;
} XrSpaceGroupUuidFilterInfoMETA;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `groupUuid` is an XrUuid specifying which group UUID that the application wants to query for spaces shared within.

This structure is a space query filter for use with query functions introduced in the `XR_FB_spatial_entity_query` extension.

To query spaces shared with a group, the application **can** include the XrSpaceGroupUuidFilterInfoMETA filter in the query filters when calling xrQuerySpacesFB.

If XrSpaceGroupUuidFilterInfoMETA is passed into xrQuerySpacesFB and the group UUID is not found by the runtime, the runtime **must** return an `XR_ERROR_SPACE_GROUP_NOT_FOUND_META` as the XrEventDataSpaceQueryCompleteFB::`result`.

Valid Usage (Implicit)

- The `XR_META_spatial_entity_group_sharing` extension **must** be enabled prior to using XrSpaceGroupUuidFilterInfoMETA
- `type` **must** be `XR_TYPE_SPACE_GROUP_UUID_FILTER_INFO_META`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

12.166.5. New Structures

- Extending XrSystemProperties:
  - XrSystemSpatialEntityGroupSharingPropertiesMETA

If `XR_FB_spatial_entity_query` is supported:

- Extending XrSpaceQueryInfoBaseHeaderFB:
  - XrSpaceGroupUuidFilterInfoMETA

If `XR_META_spatial_entity_sharing` is supported:

- XrShareSpacesRecipientGroupsMETA

12.166.6. New Enum Constants

- `XR_META_SPATIAL_ENTITY_GROUP_SHARING_EXTENSION_NAME`

- XR_META_spatial_entity_group_sharing_SPEC_VERSION
- Extending XrResult:
  - XR_ERROR_SPACE_GROUP_NOT_FOUND_META
- Extending XrStructureType:
  - XR_TYPE_SHARE_SPACES_RECIPIENT_GROUPS_META
  - XR_TYPE_SPACE_GROUP_UUID_FILTER_INFO_META
  - XR_TYPE_SYSTEM_SPATIAL_ENTITY_GROUP_SHARING_PROPERTIES_META

Version History

- Revision 1, 2024-06-14 (TJ Gilbrough)
  - Initial extension description

12.167. XR_META_spatial_entity_mesh

Name String

XR_META_spatial_entity_mesh

Extension Type

Instance extension

Registered Extension Number

270

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0
and
XR_FB_spatial_entity

Last Modified Date

2023-06-12

IP Status

No known IP claims.

Contributors

Yuichi Taguchi, Meta Platforms
Anton Vaneev, Meta Platforms
Andreas Loeve Selvik, Meta Platforms
John Kearney, Meta Platforms

12.167.1. Overview

This extension expands on the concept of spatial entities to include a way for a spatial entity to represent a triangle mesh that describes 3D geometry of the spatial entity in a scene. Spatial entities are defined in `XR_FB_spatial_entity` extension using the Entity-Component System. The triangle mesh is a component type that **may** be associated to a spatial entity.

12.167.2. Retrieving a triangle mesh

The xrGetSpaceTriangleMeshMETA function is defined as:

```
// Provided by XR_META_spatial_entity_mesh
XrResult xrGetSpaceTriangleMeshMETA(
    XrSpace                                     space,
    const XrSpaceTriangleMeshGetInfoMETA*       getInfo,
    XrSpaceTriangleMeshMETA*                    triangleMeshOutput);
```

Parameter Descriptions

- `space` is a handle to an XrSpace.
- `getInfo` exists for extensibility purposes. It is `NULL` or a pointer to a valid XrSpaceTriangleMeshGetInfoMETA.
- `triangleMeshOutput` is the output parameter that points to an XrSpaceTriangleMeshMETA.

The xrGetSpaceTriangleMeshMETA function is used by the application to perform the two calls required to obtain a triangle mesh associated to a spatial entity specified by `space`.

The spatial entity `space` **must** have the `XR_SPACE_COMPONENT_TYPE_TRIANGLE_MESH_META` component type enabled, otherwise this function will return `XR_ERROR_SPACE_COMPONENT_NOT_ENABLED_FB`.

Valid Usage (Implicit)

- The XR_META_spatial_entity_mesh extension **must** be enabled prior to calling xrGetSpaceTriangleMeshMETA
- space **must** be a valid XrSpace handle
- getInfo **must** be a pointer to a valid XrSpaceTriangleMeshGetInfoMETA structure
- triangleMeshOutput **must** be a pointer to an XrSpaceTriangleMeshMETA structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FEATURE_UNSUPPORTED
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_SIZE_INSUFFICIENT
- XR_ERROR_SPACE_COMPONENT_NOT_ENABLED_FB
- XR_ERROR_VALIDATION_FAILURE

The XrSpaceTriangleMeshGetInfoMETA structure is defined as:

```
// Provided by XR_META_spatial_entity_mesh
typedef struct XrSpaceTriangleMeshGetInfoMETA {
    XrStructureType    type;
    const void*        next;
} XrSpaceTriangleMeshGetInfoMETA;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.

Valid Usage (Implicit)

- The `XR_META_spatial_entity_mesh` extension **must** be enabled prior to using XrSpaceTriangleMeshGetInfoMETA
- `type` **must** be `XR_TYPE_SPACE_TRIANGLE_MESH_GET_INFO_META`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

The XrSpaceTriangleMeshMETA structure is defined as:

```
// Provided by XR_META_spatial_entity_mesh
typedef struct XrSpaceTriangleMeshMETA {
    XrStructureType    type;
    void*              next;
    uint32_t           vertexCapacityInput;
    uint32_t           vertexCountOutput;
    XrVector3f*        vertices;
    uint32_t           indexCapacityInput;
    uint32_t           indexCountOutput;
    uint32_t*          indices;
} XrSpaceTriangleMeshMETA;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `vertexCapacityInput` is an input parameter for the application to specify the capacity of the `vertices` array, or 0 to indicate a request to retrieve the required capacity.
- `vertexCountOutput` is an output parameter that will hold the number of vertices written in the output array, or the required capacity in the case that `vertexCapacityInput` is insufficient. The returned value **must** be equal to or larger than 3.
- `vertices` is a pointer to an array of XrVector3f, but **can** be `NULL` if `vertexCapacityInput` is 0. The vertices are defined in the coordinate frame of XrSpace to which this struct is associated.
- `indexCapacityInput` is an input parameter for the application to specify the capacity of the `indices` array, or 0 to indicate a request to retrieve the required capacity.
- `indexCountOutput` is an output parameter that will hold the number of indices written in the output array, or the required capacity in the case that `indexCapacityInput` is insufficient. The returned value **must** be a multiple of 3.
- `indices` is a pointer to an array of `uint32_t`, but **can** be `NULL` if `indexCapacityInput` is 0. Each element refers to a vertex in `vertices`.
- See the Buffer Size Parameters section for a detailed description of retrieving the required `vertices` and `indices` array sizes.

The XrSpaceTriangleMeshMETA structure **can** be used by the application to perform the two calls required to obtain a triangle mesh associated to a specified spatial entity.

The output values written in the `indices` array represent indices of vertices: Three consecutive elements represent a triangle with a counter-clockwise winding order.

Valid Usage (Implicit)

- The `XR_META_spatial_entity_mesh` extension **must** be enabled prior to using XrSpaceTriangleMeshMETA
- `type` **must** be `XR_TYPE_SPACE_TRIANGLE_MESH_META`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

New Object Types

New Atom

New Flag Types

New Enum Constants

XrSpaceComponentTypeFB enumeration is extended with:

- `XR_SPACE_COMPONENT_TYPE_TRIANGLE_MESH_META`

XrStructureType enumeration is extended with:

- `XR_TYPE_SPACE_TRIANGLE_MESH_GET_INFO_META`
- `XR_TYPE_SPACE_TRIANGLE_MESH_META`

New Enums

New Structures

- XrSpaceTriangleMeshGetInfoMETA
- XrSpaceTriangleMeshMETA

New Functions

- xrGetSpaceTriangleMeshMETA

Issues

Version History

- Revision 1, 2023-06-12 (Yuichi Taguchi)
  - Initial extension description.

12.168. XR_META_spatial_entity_persistence

Name String

`XR_META_spatial_entity_persistence`

Extension Type

Instance extension

Registered Extension Number

260

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Last Modified Date

2024-11-09

IP Status

No known IP claims.

Contributors

Natalie Fleury, Meta Platforms
Abhishek Shrivastava, Meta Platforms

12.168.1. Overview

XR_META_spatial_entity_persistence enables saving and erasing Spatial Entities, allowing them to be retrieved and tracked across different sessions and over time.

Technical overview

XR_META_spatial_entity_persistence is the next generation of Meta spatial entity storage management, following the previous generation offered through XR_FB_spatial_entity_storage and XR_FB_spatial_entity_storage_batch which are now obsolete.

If the XR_SPACE_COMPONENT_TYPE_STORABLE_FB component is enabled on a space, as defined in XR_FB_spatial_entity, application developers **may** save and erase app-created XrSpace corresponding to Meta spatial entities. XR_META_spatial_entity_persistence is expected to be used alongside XR_META_spatial_entity_discovery for spatial entity discovery/loading and retrieval.

12.168.2. Inspect System Capability

The XrSystemSpacePersistencePropertiesMETA structure is defined as:

```
// Provided by XR_META_spatial_entity_persistence
typedef struct XrSystemSpacePersistencePropertiesMETA {
    XrStructureType    type;
    const void*        next;
    XrBool32           supportsSpacePersistence;
} XrSystemSpacePersistencePropertiesMETA;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension. For more details about the structure chain, see the structure being extended (XrSystemProperties).
- `supportsSpacePersistence` is an `XrBool32`, indicating if current system supports space persistence.

An application **can** inspect whether the system is capable of supporting space persistence by extending the XrSystemProperties with XrSystemSpacePersistencePropertiesMETA structure when calling xrGetSystemProperties.

If and only if a runtime returns `XR_FALSE` for `supportsSpacePersistence`, the runtime **must** return `XR_ERROR_FEATURE_UNSUPPORTED` from xrSaveSpacesMETA and xrEraseSpacesMETA.

Valid Usage (Implicit)

- The `XR_META_spatial_entity_persistence` extension **must** be enabled prior to using XrSystemSpacePersistencePropertiesMETA
- `type` **must** be `XR_TYPE_SYSTEM_SPACE_PERSISTENCE_PROPERTIES_META`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

Save Spaces

The xrSaveSpacesMETA function is defined as:

```
// Provided by XR_META_spatial_entity_persistence
XrResult xrSaveSpacesMETA(
    XrSession                                   session,
    const XrSpacesSaveInfoMETA*                 info,
    XrAsyncRequestIdFB*                         requestId);
```

Parameter Descriptions

- `session` is a handle to an XrSession.
- `info` contains the input struct XrSpacesSaveInfoMETA for the save operation.
- `requestId` is a pointer to an `XrAsyncRequestIdFB` value and is an output parameter. The variable it points to is populated with the ID of this asynchronous request.

The xrSaveSpacesMETA function persists the space(s) provided. The scope of the save operation is same-user, same-device, same-app. This is an asynchronous operation. Completion results are conveyed in the event XrEventDataSpacesSaveResultMETA.

The runtime **must** return XR_ERROR_HANDLE_INVALID from xrSaveSpacesMETA if any of the XrSpacesSaveInfoMETA::spaces are XR_NULL_HANDLE or otherwise invalid (e.g. not a Meta Spatial Entity XrSpace). Note that saving an entity which has already been saved previously is valid and a no-op.

The runtime **must** return XR_ERROR_VALIDATION_FAILURE from xrSaveSpacesMETA if either one of the following is true:

- The XrSpacesSaveInfoMETA::spaces is NULL.
- The XrSpacesSaveInfoMETA::spaceCount is 0.

The length of the array XrSpacesSaveInfoMETA::spaces **must** have a size of at least XrSpacesSaveInfoMETA::spaceCount.

When initiated unsuccessfully (i.e. the immediate return value of the xrSaveSpacesMETA call is an error), the save operation terminates without saving any entities, and no save result event is queued.

When initiated successfully (i.e. the immediate return value of the xrSaveSpacesMETA call is not an error), the full save operation is asynchronous. The runtime **must** queue an XrEventDataSpacesSaveResultMETA event when the save operation completes, either successfully, qualified success (warning), or with an error. See the XrEventDataSpacesSaveResultMETA for which XrResult enumerants **may** be returned in the event.

Note that if the XrEventDataSpacesSaveResultMETA::result is an error, it is possible that any subset of the Spatial Entity Spaces were saved.

Valid Usage (Implicit)

- The XR_META_spatial_entity_persistence extension **must** be enabled prior to calling xrSaveSpacesMETA
- session **must** be a valid XrSession handle
- info **must** be a pointer to a valid XrSpacesSaveInfoMETA structure
- requestId **must** be a pointer to an XrAsyncRequestIdFB value

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING
- XR_SESSION_NOT_FOCUSED

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INITIALIZATION_FAILED
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_SESSION_NOT_RUNNING
- XR_ERROR_SPACE_COMPONENT_NOT_ENABLED_FB
- XR_ERROR_SPACE_RATE_LIMITED_META
- XR_ERROR_VALIDATION_FAILURE

The XrSpacesSaveInfoMETA structure is defined as:

```
// Provided by XR_META_spatial_entity_persistence
typedef struct XrSpacesSaveInfoMETA {
    XrStructureType    type;
    const void*        next;
    uint32_t           spaceCount;
    XrSpace*           spaces;
} XrSpacesSaveInfoMETA;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- spaceCount is the number of spaces in the spaces array.
- spaces is the XrSpace array (pointer to first element) of spaces to be saved.

The XrSpacesSaveInfoMETA structure contains information used to save one or more spaces with xrSaveSpacesMETA.

Valid Usage (Implicit)

- The XR_META_spatial_entity_persistence extension **must** be enabled prior to using XrSpacesSaveInfoMETA
- type **must** be XR_TYPE_SPACES_SAVE_INFO_META
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- spaces **must** be a pointer to an array of spaceCount XrSpace handles
- The spaceCount parameter **must** be greater than 0

The XrEventDataSpacesSaveResultMETA structure is defined as:

```
// Provided by XR_META_spatial_entity_persistence
typedef struct XrEventDataSpacesSaveResultMETA {
    XrStructureType       type;
    const void*           next;
    XrAsyncRequestIdFB    requestId;
    XrResult              result;
} XrEventDataSpacesSaveResultMETA;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- requestId is the XrAsyncRequestIdFB of the asynchronous request to save spaces.
- result is an XrResult that describes whether the request succeeded, had any warnings, or if an error occurred.

This event conveys the results of the asynchronous operation started by xrSaveSpacesMETA.

The XrEventDataSpacesSaveResultMETA event contains the result of the save/write operation, as well as the XrAsyncRequestIdFB of the operation.

Potential XrEventDataSpacesSaveResultMETA::result values include the following XrResult enumerants:

Async Return Codes

XrEventDataSpacesSaveResultMETA::`result` values:

Success

- `XR_SUCCESS`

Failure

- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SPACE_INSUFFICIENT_RESOURCES_META`
- `XR_ERROR_SPACE_STORAGE_AT_CAPACITY_META`
- `XR_ERROR_SPACE_INSUFFICIENT_VIEW_META`
- `XR_ERROR_SPACE_PERMISSION_INSUFFICIENT_META`
- `XR_ERROR_SPACE_RATE_LIMITED_META`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_SPACE_RATE_LIMITED_META`
- `XR_ERROR_VALIDATION_FAILURE`

Valid Usage (Implicit)

- The `XR_META_spatial_entity_persistence` extension **must** be enabled prior to using XrEventDataSpacesSaveResultMETA
- `type` **must** be `XR_TYPE_EVENT_DATA_SPACES_SAVE_RESULT_META`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

Erase Spaces

The xrEraseSpacesMETA function is defined as:

```
// Provided by XR_META_spatial_entity_persistence
XrResult xrEraseSpacesMETA(
    XrSession                                   session,
    const XrSpacesEraseInfoMETA*                info,
    XrAsyncRequestIdFB*                         requestId);
```

Parameter Descriptions

- `session` is a handle to an XrSession.
- `info` contains the input struct XrSpacesEraseInfoMETA for the erase operation.
- `requestId` is a pointer to an `XrAsyncRequestIdFB` value and is an output parameter. The variable it points to is populated with the ID of this asynchronous request.

The xrEraseSpacesMETA function erases space(s) from storage. The scope of the erase operation is same-user, same-device, same-app. After a successful erase operation, the XrSpace remains valid in the current session until the application destroys the space handle or its parent, the session handle. That is, this does not destroy spaces from tracking, but if erase is successful, the spaces **must** be ephemeral again (undiscoverable across sessions).

This is an asynchronous operation. Completion results are conveyed in the event XrEventDataSpacesEraseResultMETA.

The runtime **must** return `XR_ERROR_HANDLE_INVALID` from xrEraseSpacesMETA if any of the XrSpacesEraseInfoMETA::`spaces` are XR_NULL_HANDLE or otherwise invalid (e.g. not a Meta Spatial Entity XrSpace). Note that it is valid for a Meta Spatial Entity space that has not previously been saved to be included in an erase operation; that portion of the operation is a no-op.

At least one of XrSpacesEraseInfoMETA::`uuids` or XrSpacesEraseInfoMETA::`spaces` **must** be populated. The runtime **must** return `XR_ERROR_VALIDATION_FAILURE` from xrEraseSpacesMETA if either of the following are true:

- The XrSpacesEraseInfoMETA::`spaces` are `NULL` and the XrSpacesEraseInfoMETA::`uuids` are `NULL`.
- The XrSpacesEraseInfoMETA::`spaceCount` is 0 and the XrSpacesEraseInfoMETA::`uuidCount` is 0.

The lengths of the arrays **must** equal the corresponding counts (e.g. `spaceCount` **must** equal the length of the `spaces` array).

When initiated unsuccessfully (i.e. the immediate return value of the xrEraseSpacesMETA call is an error), the erase operation terminates without erasing any Spatial Entities, and no erase result event is queued.

When initiated successfully (i.e. the immediate return value of the xrEraseSpacesMETA call is not an error), the full erase operation is asynchronous. The runtime **must** queue an XrEventDataSpacesEraseResultMETA event when the erase operation completes, either successfully, qualified success (warning), or with an error. See the XrEventDataSpacesEraseResultMETA for which XrResult enumerants **may** be returned in the event.

Note that if the XrEventDataSpacesEraseResultMETA::`result` is an error, it is possible that any subset of the Spatial Entity Spaces were erased.

Valid Usage (Implicit)

- The XR_META_spatial_entity_persistence extension **must** be enabled prior to calling xrEraseSpacesMETA
- `session` **must** be a valid XrSession handle
- `info` **must** be a pointer to a valid XrSpacesEraseInfoMETA structure
- `requestId` **must** be a pointer to an `XrAsyncRequestIdFB` value

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING
- XR_SESSION_NOT_FOCUSED

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INITIALIZATION_FAILED
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_SESSION_NOT_RUNNING
- XR_ERROR_SPACE_COMPONENT_NOT_ENABLED_FB
- XR_ERROR_SPACE_RATE_LIMITED_META
- XR_ERROR_VALIDATION_FAILURE

The XrSpacesEraseInfoMETA structure is defined as:

```
// Provided by XR_META_spatial_entity_persistence
typedef struct XrSpacesEraseInfoMETA {
    XrStructureType    type;
    const void*        next;
    uint32_t           spaceCount;
    XrSpace*           spaces;
    uint32_t           uuidCount;
    XrUuidEXT*         uuids;
} XrSpacesEraseInfoMETA;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `spaceCount` is the number of spaces in the `spaces` array.
- `spaces` is the XrSpace array (pointer to first element) of spaces to be erased.
- `uuidCount` is the number of UUIDs in the `uuids` array.
- `uuids` is the XrUuidEXT array (pointer to first element) of spaces to be erased.

The XrSpacesEraseInfoMETA structure contains information used to erase one or more spaces with xrEraseSpacesMETA. Both the `spaces` and `uuids` arrays are optional, but at least one of them must contain at least one element. See xrEraseSpacesMETA for required validation behavior by the runtime.

Valid Usage (Implicit)

- The `XR_META_spatial_entity_persistence` extension **must** be enabled prior to using XrSpacesEraseInfoMETA
- `type` **must** be `XR_TYPE_SPACES_ERASE_INFO_META`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- If `spaceCount` is not `0`, `spaces` **must** be a pointer to an array of `spaceCount` XrSpace handles
- If `uuidCount` is not `0`, `uuids` **must** be a pointer to an array of `uuidCount` XrUuidEXT structures

The XrEventDataSpacesEraseResultMETA structure is defined as:

```
// Provided by XR_META_spatial_entity_persistence
typedef struct XrEventDataSpacesEraseResultMETA {
    XrStructureType       type;
    const void*           next;
    XrAsyncRequestIdFB    requestId;
    XrResult              result;
} XrEventDataSpacesEraseResultMETA;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `requestId` is the `XrAsyncRequestIdFB` of the asynchronous request to erase spaces.
- `result` is an XrResult that describes whether the request succeeded, had any warnings, or if an error occurred.

The XrEventDataSpacesEraseResultMETA event contains the result of the erase/write operation, as well as the `XrAsyncRequestIdFB` of the operation.

Potential XrEventDataSpacesEraseResultMETA::`result` values include the following XrResult enumerants:

Async Return Codes

XrEventDataSpacesEraseResultMETA::`result` values:

Success

- `XR_SUCCESS`

Failure

- `XR_ERROR_SPACE_INSUFFICIENT_RESOURCES_META`
- `XR_ERROR_SPACE_PERMISSION_INSUFFICIENT_META`
- `XR_ERROR_SPACE_RATE_LIMITED_META`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_SPACE_RATE_LIMITED_META`
- `XR_ERROR_VALIDATION_FAILURE`

Valid Usage (Implicit)

- The XR_META_spatial_entity_persistence extension **must** be enabled prior to using XrEventDataSpacesEraseResultMETA
- type **must** be XR_TYPE_EVENT_DATA_SPACES_ERASE_RESULT_META
- next **must** be NULL or a valid pointer to the next structure in a structure chain

12.168.3. New Commands

- xrEraseSpacesMETA
- xrSaveSpacesMETA

12.168.4. New Structures

- XrEventDataSpacesEraseResultMETA
- XrEventDataSpacesSaveResultMETA
- XrSpacesEraseInfoMETA
- XrSpacesSaveInfoMETA
- Extending XrSystemProperties:
  - XrSystemSpacePersistencePropertiesMETA

12.168.5. New Enum Constants

- XR_META_SPATIAL_ENTITY_PERSISTENCE_EXTENSION_NAME
- XR_META_spatial_entity_persistence_SPEC_VERSION
- Extending XrResult:
  - XR_ERROR_SPACE_INSUFFICIENT_RESOURCES_META
  - XR_ERROR_SPACE_INSUFFICIENT_VIEW_META
  - XR_ERROR_SPACE_PERMISSION_INSUFFICIENT_META
  - XR_ERROR_SPACE_RATE_LIMITED_META
  - XR_ERROR_SPACE_STORAGE_AT_CAPACITY_META
  - XR_ERROR_SPACE_TOO_BRIGHT_META
  - XR_ERROR_SPACE_TOO_DARK_META
- Extending XrStructureType:
  - XR_TYPE_EVENT_DATA_SPACES_ERASE_RESULT_META
  - XR_TYPE_EVENT_DATA_SPACES_SAVE_RESULT_META

- `XR_TYPE_SPACES_ERASE_INFO_META`
- `XR_TYPE_SPACES_SAVE_INFO_META`
- `XR_TYPE_SYSTEM_SPACE_PERSISTENCE_PROPERTIES_META`

12.168.6. Issues

12.168.7. Version History

- Revision 1, 2024-11-09 (Natalie Fleury)
  - Initial draft

12.169. XR_META_spatial_entity_room_mesh

Name String

`XR_META_spatial_entity_room_mesh`

Extension Type

Instance extension

Registered Extension Number

554

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0
and
`XR_FB_spatial_entity`
and
`XR_META_spatial_entity_semantic_label`

Last Modified Date

2024-10-04

IP Status

No known IP claims.

Contributors

Yuichi Taguchi, Meta Platforms

Chen Liu, Meta Platforms
Markus Schlaefli, Meta Platforms
Martin Sherburn, Meta Platforms

12.169.1. Overview

This extension expands on the concept of spatial entities to include a way for a spatial entity to represent a room mesh. A room mesh provides 3D geometry and semantic information of a room in a scene. Spatial entities are defined in the `XR_FB_spatial_entity` extension using the Entity-Component System. A spatial entity **may** have the room mesh component type associated with it.

12.169.2. Retrieving a room mesh

The xrGetSpaceRoomMeshMETA function is defined as:

```
// Provided by XR_META_spatial_entity_room_mesh
XrResult xrGetSpaceRoomMeshMETA(
    XrSpace                                     space,
    const XrSpaceRoomMeshGetInfoMETA*           getInfo,
    XrRoomMeshMETA*                             roomMeshOutput);
```

Parameter Descriptions

- `space` is a handle to an XrSpace.
- `getInfo` is the input parameter that points to a valid XrSpaceRoomMeshGetInfoMETA.
- `roomMeshOutput` is the output parameter that points to a valid XrRoomMeshMETA.

The xrGetSpaceRoomMeshMETA function is used by the application to perform the two calls required to obtain a room mesh associated with a spatial entity specified by `space`.

If the spatial entity `space` does not have the `XR_SPACE_COMPONENT_TYPE_ROOM_MESH_META` component type enabled, this function **must** return `XR_ERROR_SPACE_COMPONENT_NOT_ENABLED_FB`.

Valid Usage (Implicit)

- The `XR_META_spatial_entity_room_mesh` extension **must** be enabled prior to calling xrGetSpaceRoomMeshMETA
- `space` **must** be a valid XrSpace handle
- `getInfo` **must** be a pointer to a valid XrSpaceRoomMeshGetInfoMETA structure
- `roomMeshOutput` **must** be a pointer to an XrRoomMeshMETA structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FEATURE_UNSUPPORTED
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_SIZE_INSUFFICIENT
- XR_ERROR_SPACE_COMPONENT_NOT_ENABLED_FB
- XR_ERROR_VALIDATION_FAILURE

The XrSpaceRoomMeshGetInfoMETA structure is defined as:

```
// Provided by XR_META_spatial_entity_room_mesh
typedef struct XrSpaceRoomMeshGetInfoMETA {
    XrStructureType               type;
    const void*                   next;
    uint32_t                      recognizedSemanticLabelCount;
    const XrSemanticLabelMETA*    recognizedSemanticLabels;
} XrSpaceRoomMeshGetInfoMETA;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `recognizedSemanticLabelCount` is an input parameter for the application to specify the number of elements in the `recognizedSemanticLabels` array.
- `recognizedSemanticLabels` is an input parameter for the application to specify an array of XrSemanticLabelMETA. The runtime **must** return only the semantic labels specified in this list in XrRoomMeshFaceMETA::`semanticLabel`. The runtime **must** return semantic labels that are not specified in this list as `XR_SEMANTIC_LABEL_UNKNOWN_META`.

Valid Usage (Implicit)

- The `XR_META_spatial_entity_room_mesh` extension **must** be enabled prior to using XrSpaceRoomMeshGetInfoMETA
- `type` **must** be `XR_TYPE_SPACE_ROOM_MESH_GET_INFO_META`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- If `recognizedSemanticLabelCount` is not `0`, `recognizedSemanticLabels` **must** be a pointer to an array of `recognizedSemanticLabelCount` valid XrSemanticLabelMETA values

The XrRoomMeshMETA structure is defined as:

```
// Provided by XR_META_spatial_entity_room_mesh
typedef struct XrRoomMeshMETA {
    XrStructureType        type;
    void*                  next;
    uint32_t               vertexCapacityInput;
    uint32_t               vertexCountOutput;
    XrVector3f*            vertices;
    uint32_t               faceCapacityInput;
    uint32_t               faceCountOutput;
    XrRoomMeshFaceMETA*    faces;
} XrRoomMeshMETA;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `vertexCapacityInput` is an input parameter for the application to specify the capacity of the `vertices` array, or 0 to indicate a request to retrieve the required capacity.
- `vertexCountOutput` is an output parameter that will hold the number of vertices written in the output array, or the required capacity in the case that `vertexCapacityInput` is insufficient. The returned value **must** be equal to or larger than 4.
- `vertices` is a pointer to an array of XrVector3f, but **can** be `NULL` if `vertexCapacityInput` is 0. The vertices are defined in the coordinate frame of XrSpace to which this struct is associated.
- `faceCapacityInput` is an input parameter for the application to specify the capacity of the `faces` array, or 0 to indicate a request to retrieve the required capacity.
- `faceCountOutput` is an output parameter that will hold the number of faces written in the output array, or the required capacity in the case that `faceCapacityInput` is insufficient.
- `faces` is a pointer to an array of XrRoomMeshFaceMETA, but **can** be `NULL` if `faceCapacityInput` is 0.
- See the Buffer Size Parameters section for a detailed description of retrieving the required `vertices` and `faces` array sizes.

The application uses the XrRoomMeshMETA structure to perform the two calls required to obtain a room mesh associated with a specified spatial entity.

The output values written in the `faces` array represent faces of the room mesh. Each face is represented with an XrRoomMeshFaceMETA structure as well as an XrRoomMeshFaceIndicesMETA structure that **can** be separately obtained by using the xrGetSpaceRoomMeshFaceIndicesMETA function. The faces of the room mesh **must** define a closed 3D volume. The faces **must** not have overlaps with each other.

Valid Usage (Implicit)

- The XR_META_spatial_entity_room_mesh extension **must** be enabled prior to using XrRoomMeshMETA
- type **must** be XR_TYPE_ROOM_MESH_META
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- If vertexCapacityInput is not 0, vertices **must** be a pointer to an array of vertexCapacityInput XrVector3f structures
- If faceCapacityInput is not 0, faces **must** be a pointer to an array of faceCapacityInput XrRoomMeshFaceMETA structures

The XrRoomMeshFaceMETA structure is defined as:

```
// Provided by XR_META_spatial_entity_room_mesh
typedef struct XrRoomMeshFaceMETA {
    XrUuid                 uuid;
    XrUuid                 parentUuid;
    XrSemanticLabelMETA    semanticLabel;
} XrRoomMeshFaceMETA;
```

Member Descriptions

- uuid is a valid XrUuid of this face.
- parentUuid is a valid XrUuid of the parent face of this face if the parent face exists; otherwise, it is an all-zero XrUuid.
- semanticLabel is the semantic label of this face, which **must** be one of the semantic labels specified in XrSpaceRoomMeshGetInfoMETA::recognizedSemanticLabels or XR_SEMANTIC_LABEL_UNKNOWN_META.

Valid Usage (Implicit)

- The XR_META_spatial_entity_room_mesh extension **must** be enabled prior to using XrRoomMeshFaceMETA
- semanticLabel **must** be a valid XrSemanticLabelMETA value

12.169.3. Retrieving indices of a face of a room mesh

The xrGetSpaceRoomMeshFaceIndicesMETA function is defined as:

```
// Provided by XR_META_spatial_entity_room_mesh
XrResult xrGetSpaceRoomMeshFaceIndicesMETA(
    XrSpace                                     space,
    const XrUuid*                               faceUuid,
    XrRoomMeshFaceIndicesMETA*                  roomMeshFaceIndicesOutput);
```

Parameter Descriptions

- `space` is a handle to an XrSpace.
- `faceUuid` is the input parameter that points to a valid XrUuid obtained from XrRoomMeshFaceMETA::`uuid`.
- `roomMeshFaceIndicesOutput` is the output parameter that points to a valid XrRoomMeshFaceIndicesMETA.

The xrGetSpaceRoomMeshFaceIndicesMETA function is used by the application to perform the two calls required to obtain indices of a face specified by `faceUuid` of a room mesh associated with a spatial entity specified by `space`.

If the spatial entity `space` does not have the `XR_SPACE_COMPONENT_TYPE_ROOM_MESH_META` component type enabled, this function **must** return `XR_ERROR_SPACE_COMPONENT_NOT_ENABLED_FB`.

Valid Usage (Implicit)

- The `XR_META_spatial_entity_room_mesh` extension **must** be enabled prior to calling xrGetSpaceRoomMeshFaceIndicesMETA
- `space` **must** be a valid XrSpace handle
- `faceUuid` **must** be a pointer to a valid XrUuid structure
- `roomMeshFaceIndicesOutput` **must** be a pointer to an XrRoomMeshFaceIndicesMETA structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FEATURE_UNSUPPORTED
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_SIZE_INSUFFICIENT
- XR_ERROR_SPACE_COMPONENT_NOT_ENABLED_FB
- XR_ERROR_VALIDATION_FAILURE

The XrRoomMeshFaceIndicesMETA structure is defined as:

```
// Provided by XR_META_spatial_entity_room_mesh
typedef struct XrRoomMeshFaceIndicesMETA {
    XrStructureType    type;
    void*              next;
    uint32_t           indexCapacityInput;
    uint32_t           indexCountOutput;
    uint32_t*          indices;
} XrRoomMeshFaceIndicesMETA;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `indexCapacityInput` is an input parameter for the application to specify the capacity of the `indices` array, or 0 to indicate a request to retrieve the required capacity.
- `indexCountOutput` is an output parameter that will hold the number of indices written in the output array, or the required capacity in the case that `indexCapacityInput` is insufficient. The returned value **must** be a multiple of 3.
- `indices` is a pointer to an array of `uint32_t`, but **can** be `NULL` if `indexCapacityInput` is 0.
- See the Buffer Size Parameters section for a detailed description of retrieving the required `indices` array sizes.

The XrRoomMeshFaceIndicesMETA structure **can** be used by the application to perform the two calls required to obtain indices of a face of a room mesh associated with a specified spatial entity.

The geometry of a face is represented as a set of triangles. The output values written in the `indices` array represent indices of the vertices obtained in XrRoomMeshMETA::`vertices`. Three consecutive elements represent a triangle with a counter-clockwise winding order. Each face of a room mesh **must** be a planar face; i.e. the vertices of a face **must** be coplanar.

Valid Usage (Implicit)

- The `XR_META_spatial_entity_room_mesh` extension **must** be enabled prior to using XrRoomMeshFaceIndicesMETA
- `type` **must** be `XR_TYPE_ROOM_MESH_FACE_INDICES_META`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- If `indexCapacityInput` is not `0`, `indices` **must** be a pointer to an array of `indexCapacityInput` `uint32_t` values

12.169.4. New Commands

- xrGetSpaceRoomMeshFaceIndicesMETA
- xrGetSpaceRoomMeshMETA

12.169.5. New Structures

- XrRoomMeshFaceIndicesMETA

- XrRoomMeshFaceMETA
- XrRoomMeshMETA
- XrSpaceRoomMeshGetInfoMETA

12.169.6. New Enum Constants

- XR_META_SPATIAL_ENTITY_ROOM_MESH_EXTENSION_NAME
- XR_META_spatial_entity_room_mesh_SPEC_VERSION
- Extending XrSpaceComponentTypeFB:
  - XR_SPACE_COMPONENT_TYPE_ROOM_MESH_META
- Extending XrStructureType:
  - XR_TYPE_ROOM_MESH_FACE_INDICES_META
  - XR_TYPE_ROOM_MESH_META
  - XR_TYPE_SPACE_ROOM_MESH_GET_INFO_META

12.169.7. Issues

12.169.8. Version History

- Revision 1, 2024-10-04 (Yuichi Taguchi)
  - Initial extension description.

12.170. XR_META_spatial_entity_semantic_label

Name String

XR_META_spatial_entity_semantic_label

Extension Type

Instance extension

Registered Extension Number

553

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Last Modified Date

2024-10-04

IP Status

No known IP claims.

Contributors

Yuichi Taguchi, Meta Platforms
Chen Liu, Meta Platforms
Markus Schlaefli, Meta Platforms
Martin Sherburn, Meta Platforms

12.170.1. Overview

This extension defines semantic labels to describe semantic meanings of an associated spatial entity, or parts of an associated spatial entity. Spatial entities are defined in the `XR_FB_spatial_entity` extension using the Entity-Component System.

12.170.2. Label enumeration

The XrSemanticLabelMETA enumeration is defined as:

```
// Provided by XR_META_spatial_entity_semantic_label
typedef enum XrSemanticLabelMETA {
    XR_SEMANTIC_LABEL_UNKNOWN_META = 0,
    XR_SEMANTIC_LABEL_FLOOR_META = 1,
    XR_SEMANTIC_LABEL_CEILING_META = 2,
    XR_SEMANTIC_LABEL_WALL_FACE_META = 3,
    XR_SEMANTIC_LABEL_INNER_WALL_FACE_META = 4,
    XR_SEMANTIC_LABEL_INVISIBLE_WALL_FACE_META = 5,
    XR_SEMANTIC_LABEL_DOOR_FRAME_META = 6,
    XR_SEMANTIC_LABEL_WINDOW_FRAME_META = 7,
    XR_SEMANTIC_LABEL_MAX_ENUM_META = 0x7FFFFFFF
} XrSemanticLabelMETA;
```

Enumerant Descriptions

- `XR_SEMANTIC_LABEL_UNKNOWN_META` — Unknown. This is a valid label that is used when the runtime is not able to classify the entity using another label, or when the application does not recognize the label.
- `XR_SEMANTIC_LABEL_FLOOR_META` — Floor of a room or space.
- `XR_SEMANTIC_LABEL_CEILING_META` — Ceiling of a room or space.
- `XR_SEMANTIC_LABEL_WALL_FACE_META` — Wall face of a room or space. Wall faces, along with invisible wall faces, are used to define the outer boundary of a room.
- `XR_SEMANTIC_LABEL_INNER_WALL_FACE_META` — Inner wall face, which is a wall face that exists inside a room and is not connected to the outer boundary of the room. For example, a pillar that exists at the center of a room **may** be represented by using four inner wall faces.
- `XR_SEMANTIC_LABEL_INVISIBLE_WALL_FACE_META` — Invisible wall face, which is used to conceptually separate a space (e.g., separate a living space from a kitchen space in an open floor plan house even though there is no real wall between the two spaces).
- `XR_SEMANTIC_LABEL_DOOR_FRAME_META` — Door frame, which usually exists on a wall face.
- `XR_SEMANTIC_LABEL_WINDOW_FRAME_META` — Window frame, which usually exists on a wall face.

12.170.3. New Enums

- XrSemanticLabelMETA

12.170.4. New Enum Constants

- `XR_META_SPATIAL_ENTITY_SEMANTIC_LABEL_EXTENSION_NAME`
- `XR_META_spatial_entity_semantic_label_SPEC_VERSION`

12.170.5. Issues

12.170.6. Version History

- Revision 1, 2024-10-04 (Yuichi Taguchi)
  - Initial extension description.

12.171. XR_META_spatial_entity_sharing

Name String

`XR_META_spatial_entity_sharing`

Extension Type

Instance extension

Registered Extension Number

291

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

`XR_FB_spatial_entity`

Last Modified Date

2024-06-14

IP Status

No known IP claims.

Contributors

TJ Gilbrough, Meta Platforms
Jiawen Zhang, Meta Platforms
Scott Dewald, Meta Platforms

12.171.1. Overview

The `XR_META_spatial_entity_sharing` extension enables applications to share spatial entities.

This base extension provides a generic space sharing endpoint. This extension depends on other extensions (such as `XR_META_spatial_entity_group_sharing`) to define concrete "recipient info" structures, which are passed into the generic endpoint introduced in this extension.

The scope/lifetime of the sharing action is dependent on the recipient type of the sharing. Therefore, the scope/lifetime of the sharing action is defined in the extensions which provide concrete "recipient info" structures.

`XR_META_spatial_entity_sharing` is a more generic and extendable alternative to `XR_FB_spatial_entity_sharing` (which is tightly coupled with XrSpaceUserFB).

12.171.2. Check compatibility

The XrSystemSpatialEntitySharingPropertiesMETA structure is defined as:

```
// Provided by XR_META_spatial_entity_sharing
typedef struct XrSystemSpatialEntitySharingPropertiesMETA {
    XrStructureType    type;
    void*              next;
    XrBool32           supportsSpatialEntitySharing;
} XrSystemSpatialEntitySharingPropertiesMETA;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `supportsSpatialEntitySharing` is an `XrBool32` specifying if the spatial entity sharing is supported.

An application **can** inspect whether the system is capable of Spatial Entity Sharing by extending the XrSystemProperties with XrSystemSpatialEntitySharingPropertiesMETA structure when calling xrGetSystemProperties.

If a runtime returns `XR_FALSE` for `supportsSpatialEntitySharing`, the runtime **must** return `XR_ERROR_FEATURE_UNSUPPORTED` from xrShareSpacesMETA.

Valid Usage (Implicit)

- The `XR_META_spatial_entity_sharing` extension **must** be enabled prior to using XrSystemSpatialEntitySharingPropertiesMETA
- `type` **must** be `XR_TYPE_SYSTEM_SPATIAL_ENTITY_SHARING_PROPERTIES_META`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

12.171.3. Sharing Spaces

The xrShareSpacesMETA function is defined as:

```
// Provided by XR_META_spatial_entity_sharing
XrResult xrShareSpacesMETA(
    XrSession                                   session,
    const XrShareSpacesInfoMETA*                info,
    XrAsyncRequestIdFB*                         requestId);
```

Parameter Descriptions

- `session` is an XrSession in which the spatial entity will be active.
- `info` is a pointer to XrShareSpacesInfoMETA structure to specify the spatial entity sharing details.
- `requestId` is a pointer to an `XrAsyncRequestIdFB` variable in which the generated async request ID will be stored. It is used to identify the corresponding XrEventDataShareSpacesCompleteMETA completion event.

The application **may** use the xrShareSpacesMETA function to share spaces (XrSpace) if the `XR_SPACE_COMPONENT_TYPE_SHARABLE_FB` component has been enabled on the space.

This is an asynchronous operation. Completion results are conveyed in the event XrEventDataShareSpacesCompleteMETA.

If the asynchronous operation is scheduled successfully, the runtime **must** return `XR_SUCCESS`.

If and only if the runtime returns `XR_SUCCESS`, the runtime **must** queue a single XrEventDataShareSpacesCompleteMETA event identified with a `requestId` field matching the value output by this function, referred to as the "corresponding completion event."

(If the runtime returns anything other than `XR_SUCCESS`, the runtime **must** not queue any XrEventDataShareSpacesCompleteMETA events with `requestId` field matching the `requestId` populated by this function.)

If the asynchronous operation is successful, the runtime **must** set the XrEventDataShareSpacesCompleteMETA::`result` field to `XR_SUCCESS` in the corresponding completion event.

If the asynchronous operation is scheduled but not successful, in the corresponding completion event, the runtime **must** set the XrEventDataShareSpacesCompleteMETA::`result` field to an appropriate error code instead of `XR_SUCCESS`.

Valid Usage (Implicit)

- The `XR_META_spatial_entity_sharing` extension **must** be enabled prior to calling xrShareSpacesMETA
- `session` **must** be a valid XrSession handle
- `info` **must** be a pointer to a valid XrShareSpacesInfoMETA structure
- `requestId` **must** be a pointer to an `XrAsyncRequestIdFB` value

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FEATURE_UNSUPPORTED
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The XrShareSpacesInfoMETA structure is defined as:

```
// Provided by XR_META_spatial_entity_sharing
typedef struct XrShareSpacesInfoMETA {
    XrStructureType                                 type;
    const void*                                     next;
    uint32_t                                        spaceCount;
    XrSpace*                                        spaces;
    const XrShareSpacesRecipientBaseHeaderMETA*     recipientInfo;
} XrShareSpacesInfoMETA;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR.
- spaceCount is the count spaces populated in the spaces field of this request.
- spaces is an array pointer of type XrSpace containing all spaces to be shared
- recipientInfo is a structure based on XrShareSpacesRecipientBaseHeaderMETA, defining the target recipient of the shared spaces.

Valid Usage (Implicit)

- The XR_META_spatial_entity_sharing extension **must** be enabled prior to using XrShareSpacesInfoMETA
- type **must** be XR_TYPE_SHARE_SPACES_INFO_META
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- spaces **must** be a pointer to an array of spaceCount XrSpace handles
- recipientInfo **must** be a pointer to a valid XrShareSpacesRecipientBaseHeaderMETA-based structure. See also: XrShareSpacesRecipientGroupsMETA
- The spaceCount parameter **must** be greater than 0

The XrShareSpacesRecipientBaseHeaderMETA structure is defined as:

```
// Provided by XR_META_spatial_entity_sharing
typedef struct XrShareSpacesRecipientBaseHeaderMETA {
    XrStructureType    type;
    const void*        next;
} XrShareSpacesRecipientBaseHeaderMETA;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR.

XrShareSpacesRecipientBaseHeaderMETA is designed to be an abstract base struct which is to be extended by other structures.

Any valid structure that identifies XrShareSpacesRecipientBaseHeaderMETA as its parent structure **may** be provided anywhere a valid XrShareSpacesRecipientBaseHeaderMETA is specified to be passed.

Valid Usage (Implicit)

- The XR_META_spatial_entity_sharing extension **must** be enabled prior to using XrShareSpacesRecipientBaseHeaderMETA
- type **must** be XR_TYPE_SHARE_SPACES_RECIPIENT_GROUPS_META
- next **must** be NULL or a valid pointer to the next structure in a structure chain

The XrEventDataShareSpacesCompleteMETA event structure is defined as:

```
// Provided by XR_META_spatial_entity_sharing
typedef struct XrEventDataShareSpacesCompleteMETA {
    XrStructureType       type;
    const void*           next;
    XrAsyncRequestIdFB    requestId;
    XrResult              result;
} XrEventDataShareSpacesCompleteMETA;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR.
- `requestId` is an `XrAsyncRequestIdFB` that identifies which request this event is referring to.
- `result` is an XrResult that specifies the request result. The valid `result` values are: `XR_SUCCESS`, `XR_ERROR_RUNTIME_FAILURE`, `XR_ERROR_SPACE_NETWORK_TIMEOUT_FB`, and `XR_ERROR_SPACE_NETWORK_REQUEST_FAILED_FB`.

This event conveys the results of the asynchronous operation started by xrShareSpacesMETA.

Valid Usage (Implicit)

- The `XR_META_spatial_entity_sharing` extension **must** be enabled prior to using XrEventDataShareSpacesCompleteMETA
- `type` **must** be `XR_TYPE_EVENT_DATA_SHARE_SPACES_COMPLETE_META`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

12.171.4. New Commands

- xrShareSpacesMETA

12.171.5. New Structures

- XrEventDataShareSpacesCompleteMETA
- XrShareSpacesInfoMETA
- XrShareSpacesRecipientBaseHeaderMETA
- Extending XrSystemProperties:

    - XrSystemSpatialEntitySharingPropertiesMETA

12.171.6. New Enum Constants

- XR_MAX_SPACES_PER_SHARE_REQUEST_META
- XR_META_SPATIAL_ENTITY_SHARING_EXTENSION_NAME
- XR_META_spatial_entity_sharing_SPEC_VERSION
- Extending XrStructureType:
    - XR_TYPE_EVENT_DATA_SHARE_SPACES_COMPLETE_META
    - XR_TYPE_SHARE_SPACES_INFO_META
    - XR_TYPE_SYSTEM_SPATIAL_ENTITY_SHARING_PROPERTIES_META

Version History

- Revision 1, 2024-06-14 (TJ Gilbrough)
    - Initial extension description

12.172. XR_META_tile_properties_hint

Name String

XR_META_tile_properties_hint

Extension Type

Instance extension

Registered Extension Number

610

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Last Modified Date

2025-06-06

IP Status

No known IP claims.

Contributors

Brian McNamee, Meta Platforms

12.172.1. Overview

The XR_META_tile_properties extension allows the application to provide a hint to the OpenXR implementation about the tile properties of the application's main render pass.

OpenXR runtimes do not have access to the application's frame buffers and therefore cannot get the tile properties without providing an API for the application to pass those tile properties to the OpenXR runtime.

12.172.2. Passing Tile Properties

Applications **should** use the returned tile properties from functions provided by the VK_QCOM_tile_properties Vulkan extension to get the tile properties of the application's main render pass and copy those properties to an XrTilePropertiesMETA structure to use in a call to xrSetTilePropertiesHintMETA. These tile properties **may** be used for various optimizations in the OpenXR implementation, such as providing tile aligned resolutions for XR_META_recommended_layer_resolution and optimized fragment density maps when foveated rendering is enabled.

```
// Provided by XR_META_tile_properties_hint
XrResult xrSetTilePropertiesHintMETA(
    XrSession                                   session,
    const XrTilePropertiesHintMETA*             properties);
```

Parameter Descriptions

- session is an XrSession handle previously created with xrCreateSession.
- properties is an XrTilePropertiesHintMETA that provides the tile property data.

xrSetTilePropertiesHintMETA allows the application to provide a hint to the OpenXR runtime about the GPU tiling properties to be used for internal optimizations, such as providing tile aligned resolutions from xrGetRecommendedLayerResolutionMETA and tile aligned fragment density maps for foveated swapchains.

Valid Usage (Implicit)

- The XR_META_tile_properties_hint extension **must** be enabled prior to calling xrSetTilePropertiesHintMETA
- `session` **must** be a valid XrSession handle
- `properties` **must** be a pointer to a valid XrTilePropertiesHintMETA structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The XrTilePropertiesHintMETA structure is defined as:

```
// Provided by XR_META_tile_properties_hint
typedef struct XrTilePropertiesHintMETA {
    XrStructureType                type;
    void*                          next;
    uint32_t                       propertiesCount;
    const XrTilePropertiesMETA*    properties;
} XrTilePropertiesHintMETA;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `propertiesCount` is the number of properties in the `properties` array.
- `properties` is an array of XrTilePropertiesMETA corresponding to a single render pass.

Each XrTilePropertiesMETA in the XrTilePropertiesHintMETA::properties array **should** correspond to a `VkTilePropertiesQCOM` passed by `vkGetDynamicRenderingTilePropertiesQCOM` or `vkGetFramebufferTilePropertiesQCOM`.

Valid Usage (Implicit)

- The `XR_META_tile_properties_hint` extension **must** be enabled prior to using XrTilePropertiesHintMETA
- `type` **must** be `XR_TYPE_TILE_PROPERTIES_HINT_META`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `properties` **must** be a pointer to an array of `propertiesCount` valid XrTilePropertiesMETA structures
- The `propertiesCount` parameter **must** be greater than `0`

The XrTilePropertiesMETA structure is defined as:

```
// Provided by XR_META_tile_properties_hint
typedef struct XrTilePropertiesMETA {
    XrStructureType    type;
    void*              next;
    XrExtent3DiMETA    tileDimensions;
    XrExtent2Di        apronDimensions;
    XrOffset2Di        origin;
} XrTilePropertiesMETA;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `tileDimensions` is the dimension of GPU tiles for the render pass.
- `apronDimensions` is the dimension of the tile apron for the render pass.
- `origin` is the coordinates of the top left corner of the first tile in attachment space.

XrTilePropertiesMETA is used to specify the tile properties for a single render pass provided in the XrTilePropertiesHintMETA structure. These values **should** be provided from the `VK_QCOM_tile_properties` extension from `vkGetDynamicRenderingTilePropertiesQCOM` or `vkGetFramebufferTilePropertiesQCOM`.

Valid Usage (Implicit)

- The `XR_META_tile_properties_hint` extension **must** be enabled prior to using XrTilePropertiesMETA
- `type` **must** be `XR_TYPE_TILE_PROPERTIES_META`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

The XrExtent3DiMETA structure is defined as:

```
// Provided by XR_META_tile_properties_hint
typedef struct XrExtent3DiMETA {
    int32_t    width;
    int32_t    height;
    int32_t    depth;
} XrExtent3DiMETA;
```

Member Descriptions

- `width` is the width of the extent.
- `height` is the height of the extent.
- `depth` is the depth of the extent.

Valid Usage (Implicit)

- The `XR_META_tile_properties_hint` extension **must** be enabled prior to using XrExtent3DiMETA

12.172.3. Example code

```
XrInstance instance; // previously initialized
XrSession session; // previously initialized
VkDevice device; // previously initialized
VkFramebuffer framebuffer; // previously initialized
PFN_vkGetFramebufferTilePropertiesQCOM vkGetFrameBufferTilePropertiesQCOM; //
previously initialized

// Get function pointer for xrCreateFaceTrackerFB.
PFN_xrSetTilePropertiesHintMETA pfnSetTilePropertiesHintMETA = nullptr;
CHK_XR(xrGetInstanceProcAddr(instance, "xrSetTilePropertiesHintMETA",
                             reinterpret_cast<PFN_xrVoidFunction*>(
                             &pfnSetTilePropertiesHintMETA)));

// Get tile properties from vulkan
uint32_t propertyCount = 0;
vkGetFrameBufferTilePropertiesQCOM(device, framebuffer, &propertyCount, nullptr);

if (propertyCount == 0) {
  return;
}

std::vector<VkTilePropertiesQCOM> qcomProperties(propertyCount);
vkGetFrameBufferTilePropertiesQCOM(device, framebuffer, &propertyCount,
qcomProperties.data());

// Convert VkTilePropertiesQCOM to XrTilePropertiesMETA
std::vector<XrTilePropertiesMETA> xrProperties(propertyCount, {
    .type = XR_TYPE_TILE_PROPERTIES_META,
});
for (uint32_t i = 0; i < propertyCount; ++i) {
  xrProperties[i].type = XR_TYPE_TILE_PROPERTIES_META;
  xrProperties[i].tileDimensions.width = qcomProperties[i].tileSize.width;
  xrProperties[i].tileDimensions.height = qcomProperties[i].tileSize.height;
  xrProperties[i].tileDimensions.depth = qcomProperties[i].tileSize.depth;
  xrProperties[i].apronDimensions.width = qcomProperties[i].apronSize.width;
  xrProperties[i].apronDimensions.height = qcomProperties[i].apronSize.height;
  xrProperties[i].origin.x = qcomProperties[i].origin.x;
  xrProperties[i].origin.y = qcomProperties[i].origin.y;
}

// Send the hint
XrTilePropertiesHintMETA hint{
    .type = XR_TYPE_TILE_PROPERTIES_HINT_META,
    .propertiesCount = propertyCount,
    .properties = xrProperties.data(),
};
CHK_XR(pfnSetTilePropertiesHintMETA(session, &hint));
```

12.172.4. New Commands

- xrSetTilePropertiesHintMETA

12.172.5. New Structures

- XrExtent3DiMETA
- XrTilePropertiesHintMETA
- XrTilePropertiesMETA

12.172.6. New Enum Constants

- XR_META_TILE_PROPERTIES_HINT_EXTENSION_NAME
- XR_META_tile_properties_hint_SPEC_VERSION
- Extending XrStructureType:
  - XR_TYPE_TILE_PROPERTIES_HINT_META
  - XR_TYPE_TILE_PROPERTIES_META

12.172.7. Version History

- Revision 1, 2025-06-06 (Brian McNamee)
  - Initial extension description

12.173. XR_META_virtual_keyboard

Name String

XR_META_virtual_keyboard

Extension Type

Instance extension

Registered Extension Number

220

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Last Modified Date

2023-04-14

IP Status

No known IP claims.

Contributors

Brent Housen, Meta Platforms
Chiara Coetzee, Meta Platforms
Juan Pablo León, Meta Platforms
Peter Chan, Meta Platforms

Contacts

Brent Housen, Meta Platforms
Peter Chan, Meta Platforms

12.173.1. Overview

The virtual keyboard extension provides a system-driven localized keyboard that the application has full control over in terms of positioning and rendering.

This is achieved by giving the application the data required to drive rendering and animation of the keyboard in response to interaction data passed from the application to the runtime.

This approach is an alternative to a potential system keyboard overlay solution and provides a keyboard that can seamlessly blend into the application environment, since it is rendered by the same system, and avoids input focus issues that might come with a system overlay.

The API is also designed to work with custom hand and/or controller models in various games and applications.

Virtual Keyboard Integration Summary

Before explaining the individual API functions, types, and events, here is an overview on how to integrate the virtual keyboard in an application.

Note that this is purely informational and does not serve as binding requirements for the runtime or the application.

App Startup

- Check if your device supports the virtual keyboard with xrGetSystemProperties.
- Create a new keyboard with xrCreateVirtualKeyboardMETA.
- Give it a location with xrCreateVirtualKeyboardSpaceMETA, and keep a reference to the returned XrSpace.
- Load the virtual keyboard glTF model using `XR_FB_render_model`:

- Query the render model key for path */model_meta/keyboard/virtual*.
  - Using xrEnumerateRenderModelPathsFB and xrGetRenderModelPropertiesFB.
  - Make sure to set the support level to `XR_RENDER_MODEL_SUPPORTS_GLTF_2_0_SUBSET_2_BIT_FB`.
- Load the render model glTF data with the given key with xrLoadRenderModelFB.
- Load the glTF data into an extendable glTF renderer (see `Extend glTF render model support`). Note that this render model is hidden by default.

Update Tick

- When the application wants to show the keyboard, call xrSetVirtualKeyboardModelVisibilityMETA to request the runtime to update the model visibility.
  - The application **should** wait for the XrEventDataVirtualKeyboardShownMETA event as confirmation that the runtime is ready to show the keyboard.
- The application **can** move the keyboard by calling xrSuggestVirtualKeyboardLocationMETA to update the saved XrSpace.
- Then for every active input type feed the keyboard input with xrSendVirtualKeyboardInputMETA:
  - For each hand/controller, use:
    - `XR_VIRTUAL_KEYBOARD_INPUT_SOURCE_*_RAY_*` for far input
    - `XR_VIRTUAL_KEYBOARD_INPUT_SOURCE_*_DIRECT_*` for direct/near input
    - If both near and far input types are sent, the runtime **may** decide which one is the most appropriate to use.
  - Passing in a value for the input devices interactorRoot as well, i.e. the wrist root for hands.
  - The runtime will modify the `interactorRootPose` to poke limit direct interaction.
    - If poke limiting is desired, the application **should** reposition input render models with the modified root pose.
- Then get the runtime keyboard pose and scale:
  - Using xrLocateSpace on the saved keyboardSpace.
  - Using xrGetVirtualKeyboardScaleMETA to get the scale.
- Then check if the virtual keyboard glTF model has any textures that need to be updated with xrGetVirtualKeyboardDirtyTexturesMETA.
  - For every dirty texture, call xrGetVirtualKeyboardTextureDataMETA to get the RGBA texture data.
  - And then updating the texture in the glTF model that matches the given texture id.
- Then apply any glTF model animations using xrGetVirtualKeyboardModelAnimationStatesMETA to get updated animation indices and fraction values for each animation.

On Events

- XrEventDataVirtualKeyboardCommitTextMETA / XrEventDataVirtualKeyboardBackspaceMETA / XrEventDataVirtualKeyboardEnterMETA
  - Applications **can** pipe these events to a focused input field, or whatever they are expecting to handle the virtual keyboard's input.
- XrEventDataVirtualKeyboardShownMETA & XrEventDataVirtualKeyboardHiddenMETA
  - Signaled when the virtual keyboard render model animation system is hiding or showing the keyboard.

App Shutdown

- Destroy the keyboard with xrDestroyVirtualKeyboardMETA.

12.173.2. Extend glTF render model support

The virtual keyboard glTF model uses a custom texture URI for textures that the application needs to update dynamically. The application **should** implement a custom URI handler when loading the glTF model to check for these URIs and create writable textures identified by the corresponding texture ids.

The runtime **must** refer to these textures in the returned glTF model by URIs in the following format: `metaVirtualKeyboard://texture/{textureID}?w={width}&h={height}&fmt=RGBA32`

The application **should** retrieve new pixel data from the runtime with xrGetVirtualKeyboardDirtyTexturesMETA and xrGetVirtualKeyboardTextureDataMETA and apply them to the corresponding textures that are used to render the glTF model.

Furthermore, the runtime **may** use additive morph target animations to control vertex coordinates and modify UVs. The application **should** check the "extras" property when loading a glTF animation channel for an integer field named "additiveWeightIndex". If present, this value indicates the morph target index that the animation weight should be applied to, or apply all weights if the value is -1.

The application **should** check for any glTF animations to apply to the model each frame with xrGetVirtualKeyboardModelAnimationStatesMETA.

12.173.3. Collision Handling

Even though the runtime will handle any user interaction with the keyboard based on the input sent by the application, the application is responsible for managing how the keyboard should collide with other objects in the scene. To do this, the application **can** look for a node named "collision" in the loaded glTF model and use its mesh geometry and bound to define colliders that can be used by the application's choice of physics system.

12.173.4. Check device compatibility

When the `XR_META_virtual_keyboard` extension is enabled, an application **can** pass in an XrSystemVirtualKeyboardPropertiesMETA structure in the XrSystemProperties::`next` chain when calling xrGetSystemProperties to acquire information about the virtual keyboard's availability.

The XrSystemVirtualKeyboardPropertiesMETA structure is defined as:

```
// Provided by XR_META_virtual_keyboard
typedef struct XrSystemVirtualKeyboardPropertiesMETA {
    XrStructureType    type;
    void*              next;
    XrBool32           supportsVirtualKeyboard;
} XrSystemVirtualKeyboardPropertiesMETA;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension. For more details about the structure chain, see the structure being extended (XrSystemProperties).
- `supportsVirtualKeyboard` is an `XrBool32` indicating if virtual keyboard is supported.

The struct is used for checking virtual keyboard support.

Valid Usage (Implicit)

- The `XR_META_virtual_keyboard` extension **must** be enabled prior to using XrSystemVirtualKeyboardPropertiesMETA
- `type` **must** be `XR_TYPE_SYSTEM_VIRTUAL_KEYBOARD_PROPERTIES_META`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

12.173.5. Create a virtual keyboard

An application **can** create a virtual keyboard by calling xrCreateVirtualKeyboardMETA.

The xrCreateVirtualKeyboardMETA function is defined as:

```
// Provided by XR_META_virtual_keyboard
XrResult xrCreateVirtualKeyboardMETA(
    XrSession                                   session,
    const XrVirtualKeyboardCreateInfoMETA*      createInfo,
    XrVirtualKeyboardMETA*                      keyboard);
```

Parameter Descriptions

- `session` is the XrSession.
- `createInfo` is the XrVirtualKeyboardCreateInfoMETA.
- `keyboard` is the returned XrVirtualKeyboardMETA.

xrCreateVirtualKeyboardMETA creates an XrVirtualKeyboardMETA handle and establishes a keyboard within the runtime XrSession. The returned virtual keyboard handle **may** be subsequently used in API calls.

Valid Usage (Implicit)

- The `XR_META_virtual_keyboard` extension **must** be enabled prior to calling xrCreateVirtualKeyboardMETA
- `session` **must** be a valid XrSession handle
- `createInfo` **must** be a pointer to a valid XrVirtualKeyboardCreateInfoMETA structure
- `keyboard` **must** be a pointer to an XrVirtualKeyboardMETA handle

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_FEATURE_UNSUPPORTED`
- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_LIMIT_REACHED`
- `XR_ERROR_OUT_OF_MEMORY`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_VALIDATION_FAILURE`

The XrVirtualKeyboardCreateInfoMETA structure is defined as:

```
// Provided by XR_META_virtual_keyboard
typedef struct XrVirtualKeyboardCreateInfoMETA {
    XrStructureType    type;
    const void*        next;
} XrVirtualKeyboardCreateInfoMETA;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.

The struct is used for keyboard creation. Empty with the intention of future extension.

The runtime **must** return `XR_ERROR_FEATURE_UNSUPPORTED` if XrSystemVirtualKeyboardPropertiesMETA ::`supportsVirtualKeyboard` is `XR_FALSE` when checking the device compatibility.

Valid Usage (Implicit)

- The `XR_META_virtual_keyboard` extension **must** be enabled prior to using XrVirtualKeyboardCreateInfoMETA
- `type` **must** be `XR_TYPE_VIRTUAL_KEYBOARD_CREATE_INFO_META`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

12.173.6. Destroy the virtual keyboard

An application **can** destroy a virtual keyboard by calling xrDestroyVirtualKeyboardMETA.

The xrDestroyVirtualKeyboardMETA function is defined as:

```
// Provided by XR_META_virtual_keyboard
XrResult xrDestroyVirtualKeyboardMETA(
    XrVirtualKeyboardMETA                       keyboard);
```

Parameter Descriptions

- `keyboard` is the XrVirtualKeyboardMETA handle to the keyboard to destroy.

Valid Usage (Implicit)

- The XR_META_virtual_keyboard extension **must** be enabled prior to calling xrDestroyVirtualKeyboardMETA
- keyboard **must** be a valid XrVirtualKeyboardMETA handle

Thread Safety

- Access to keyboard, and any child handles, **must** be externally synchronized

Return Codes

Success

- XR_SUCCESS

Failure

- XR_ERROR_FEATURE_UNSUPPORTED
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_RUNTIME_FAILURE

12.173.7. Place the virtual keyboard

To place the keyboard, an application **can** create a virtual keyboard space by calling xrCreateVirtualKeyboardSpaceMETA.

The xrCreateVirtualKeyboardSpaceMETA function is defined as:

```
// Provided by XR_META_virtual_keyboard
XrResult xrCreateVirtualKeyboardSpaceMETA(
    XrSession                                   session,
    XrVirtualKeyboardMETA                       keyboard,
    const XrVirtualKeyboardSpaceCreateInfoMETA* createInfo,
    XrSpace*                                    keyboardSpace);
```

Parameter Descriptions

- `session` is the XrSession.
- `keyboard` is the XrVirtualKeyboardMETA handle.
- `createInfo` is the XrVirtualKeyboardSpaceCreateInfoMETA.
- `keyboardSpace` is the returned space handle.

Creates an XrSpace handle and places the keyboard in this space. The returned space handle **may** be subsequently used in API calls.

Once placed, the application **should** query the keyboard's location each frame using xrLocateSpace. It is important to do this every frame as the runtime is in control of the keyboard's movement.

The runtime **must** return `XR_ERROR_HANDLE_INVALID` if `session` is different than what is used to create `keyboard`.

Valid Usage (Implicit)

- The `XR_META_virtual_keyboard` extension **must** be enabled prior to calling xrCreateVirtualKeyboardSpaceMETA
- `session` **must** be a valid XrSession handle
- `keyboard` **must** be a valid XrVirtualKeyboardMETA handle
- `createInfo` **must** be a pointer to a valid XrVirtualKeyboardSpaceCreateInfoMETA structure
- `keyboardSpace` **must** be a pointer to an XrSpace handle
- `keyboard` **must** have been created, allocated, or retrieved from `session`

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FEATURE_UNSUPPORTED
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_POSE_INVALID
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The XrVirtualKeyboardSpaceCreateInfoMETA structure is defined as:

```
// Provided by XR_META_virtual_keyboard
typedef struct XrVirtualKeyboardSpaceCreateInfoMETA {
    XrStructureType                     type;
    const void*                         next;
    XrVirtualKeyboardLocationTypeMETA   locationType;
    XrSpace                             space;
    XrPosef                             poseInSpace;
} XrVirtualKeyboardSpaceCreateInfoMETA;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `locationType` is an XrVirtualKeyboardLocationTypeMETA enum providing the location type.
- `space` is an XrSpace previously created by a function such as xrCreateReferenceSpace.
- `poseInSpace` is the desired pose if `locationType` is `XR_VIRTUAL_KEYBOARD_LOCATION_TYPE_CUSTOM_META`.

If `locationType` is set to `XR_VIRTUAL_KEYBOARD_LOCATION_TYPE_CUSTOM_META`, the runtime **must** use the value `poseInSpace` set by the application. Otherwise, the runtime **must** provide a default pose and ignore `poseInSpace`. In all cases, the runtime **must** default the scale to 1.0.

Valid Usage (Implicit)

- The `XR_META_virtual_keyboard` extension **must** be enabled prior to using XrVirtualKeyboardSpaceCreateInfoMETA
- `type` **must** be `XR_TYPE_VIRTUAL_KEYBOARD_SPACE_CREATE_INFO_META`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `locationType` **must** be a valid XrVirtualKeyboardLocationTypeMETA value
- `space` **must** be a valid XrSpace handle

12.173.8. Move and scale the virtual keyboard

After creating a keyboard and a space, an application **can** request to move its location or change its scale. The application **can** suggest a new location or scale by calling xrSuggestVirtualKeyboardLocationMETA.

The xrSuggestVirtualKeyboardLocationMETA function is defined as:

```
// Provided by XR_META_virtual_keyboard
XrResult xrSuggestVirtualKeyboardLocationMETA(
    XrVirtualKeyboardMETA                       keyboard,
    const XrVirtualKeyboardLocationInfoMETA*    locationInfo);
```

Parameter Descriptions

- `keyboard` is the XrVirtualKeyboardMETA handle.
- `locationInfo` is the desired XrVirtualKeyboardLocationInfoMETA.

Valid Usage (Implicit)

- The `XR_META_virtual_keyboard` extension **must** be enabled prior to calling xrSuggestVirtualKeyboardLocationMETA
- `keyboard` **must** be a valid XrVirtualKeyboardMETA handle
- `locationInfo` **must** be a pointer to a valid XrVirtualKeyboardLocationInfoMETA structure

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_FEATURE_UNSUPPORTED`
- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_POSE_INVALID`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_VALIDATION_FAILURE`

The XrVirtualKeyboardLocationInfoMETA structure is defined as:

```
// Provided by XR_META_virtual_keyboard
typedef struct XrVirtualKeyboardLocationInfoMETA {
    XrStructureType                       type;
    const void*                           next;
    XrVirtualKeyboardLocationTypeMETA     locationType;
    XrSpace                               space;
    XrPosef                               poseInSpace;
    float                                 scale;
} XrVirtualKeyboardLocationInfoMETA;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `locationType` is an XrVirtualKeyboardLocationTypeMETA enum providing the location type.
- `space` is an XrSpace previously created by a function such as xrCreateReferenceSpace.
- `poseInSpace` is the desired pose if `locationType` is `XR_VIRTUAL_KEYBOARD_LOCATION_TYPE_CUSTOM_META`.
- `scale` is a `float` value of the desired multiplicative scale between 0.0 and 1.0 if `locationType` is `XR_VIRTUAL_KEYBOARD_LOCATION_TYPE_CUSTOM_META`.

If `locationType` is set to `XR_VIRTUAL_KEYBOARD_LOCATION_TYPE_CUSTOM_META`, the runtime **must** use the values `poseInSpace` and `scale` set by the application. Otherwise, the runtime **must** provide a default pose and scale and ignore `poseInSpace` and `scale`.

Valid Usage (Implicit)

- The `XR_META_virtual_keyboard` extension **must** be enabled prior to using XrVirtualKeyboardLocationInfoMETA
- `type` **must** be `XR_TYPE_VIRTUAL_KEYBOARD_LOCATION_INFO_META`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `locationType` **must** be a valid XrVirtualKeyboardLocationTypeMETA value
- `space` **must** be a valid XrSpace handle

12.173.9. Get the virtual keyboard scale

Since xrLocateSpace only handles the pose, the application **should** also get the scale every frame by

calling xrGetVirtualKeyboardScaleMETA.

The xrGetVirtualKeyboardScaleMETA function is defined as:

```
// Provided by XR_META_virtual_keyboard
XrResult xrGetVirtualKeyboardScaleMETA(
    XrVirtualKeyboardMETA                       keyboard,
    float*                                      scale);
```

Parameter Descriptions

- `keyboard` is the XrVirtualKeyboardMETA handle.
- `scale` is a float value of the current scale of the keyboard.

With both the pose and scale, the application has all the information to draw the virtual keyboard render model.

Valid Usage (Implicit)

- The `XR_META_virtual_keyboard` extension **must** be enabled prior to calling xrGetVirtualKeyboardScaleMETA
- `keyboard` **must** be a valid XrVirtualKeyboardMETA handle
- `scale` **must** be a pointer to a `float` value

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FEATURE_UNSUPPORTED
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

12.173.10. Show and hide the virtual keyboard

The runtime is in control of the keyboard's visibility to decide when to process input and reset the keyboard states. By default the keyboard render model is hidden. An application **can** update the render model visibility by calling xrSetVirtualKeyboardModelVisibilityMETA.

The xrSetVirtualKeyboardModelVisibilityMETA function is defined as:

```
// Provided by XR_META_virtual_keyboard
XrResult xrSetVirtualKeyboardModelVisibilityMETA(
    XrVirtualKeyboardMETA                       keyboard,
    const XrVirtualKeyboardModelVisibilitySetInfoMETA* modelVisibility);
```

Parameter Descriptions

- keyboard is the XrVirtualKeyboardMETA handle.
- modelVisibility is the XrVirtualKeyboardModelVisibilitySetInfoMETA.

Note that the runtime has final control of the model visibility. The runtime **may** also change the visible state in certain situations. To get the actual visibility state of the render model, the application **should** wait for the XrEventDataVirtualKeyboardShownMETA and XrEventDataVirtualKeyboardHiddenMETA events.

Valid Usage (Implicit)

- The XR_META_virtual_keyboard extension **must** be enabled prior to calling xrSetVirtualKeyboardModelVisibilityMETA
- keyboard **must** be a valid XrVirtualKeyboardMETA handle
- modelVisibility **must** be a pointer to a valid XrVirtualKeyboardModelVisibilitySetInfoMETA structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FEATURE_UNSUPPORTED
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The XrVirtualKeyboardModelVisibilitySetInfoMETA structure is defined as:

```
// Provided by XR_META_virtual_keyboard
typedef struct XrVirtualKeyboardModelVisibilitySetInfoMETA {
    XrStructureType    type;
    const void*        next;
    XrBool32           visible;
} XrVirtualKeyboardModelVisibilitySetInfoMETA;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `visible` an `XrBool32` that controls whether to show or hide the keyboard.

Valid Usage (Implicit)

- The `XR_META_virtual_keyboard` extension **must** be enabled prior to using XrVirtualKeyboardModelVisibilitySetInfoMETA
- `type` **must** be `XR_TYPE_VIRTUAL_KEYBOARD_MODEL_VISIBILITY_SET_INFO_META`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

12.173.11. Update render model textures

Each frame update the application **should** check for any textures that are updated by the runtime (e.g. when new swipe suggestion words are available). The application **should** first get the texture IDs that have updated contents (are "dirty") by calling xrGetVirtualKeyboardDirtyTexturesMETA. Then for each texture ID received, the application **should** create a XrVirtualKeyboardTextureDataMETA structure and call xrGetVirtualKeyboardTextureDataMETA to get the pixel data to update the corresponding texture created by the render system using the id reference.

The xrGetVirtualKeyboardDirtyTexturesMETA function is defined as:

```
// Provided by XR_META_virtual_keyboard
XrResult xrGetVirtualKeyboardDirtyTexturesMETA(
    XrVirtualKeyboardMETA                       keyboard,
    uint32_t                                    textureIdCapacityInput,
    uint32_t*                                   textureIdCountOutput,
    uint64_t*                                   textureIds);
```

Parameter Descriptions

- `keyboard` is the XrVirtualKeyboardMETA handle.
- `textureIdCapacityInput` is the capacity of the `textureIds` array, or 0 to indicate a request to retrieve the required capacity.
- `textureIdCountOutput` is filled in by the runtime with the count of texture IDs written or the required capacity in the case that `textureIdCapacityInput` is insufficient.
- `textureIds` is the array of texture IDs that need to be updated.

This function follows the two-call idiom for filling the `textureIds` array. Note that new texture data may be added after the runtime processes inputs from xrSendVirtualKeyboardInputMETA. Therefore, after sending new keyboard inputs the application **should** query the buffer size again before getting any texture data.

Valid Usage (Implicit)

- The `XR_META_virtual_keyboard` extension **must** be enabled prior to calling xrGetVirtualKeyboardDirtyTexturesMETA
- `keyboard` **must** be a valid XrVirtualKeyboardMETA handle
- `textureIdCountOutput` **must** be a pointer to a `uint32_t` value
- If `textureIdCapacityInput` is not `0`, `textureIds` **must** be a pointer to an array of `textureIdCapacityInput` `uint64_t` values

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FEATURE_UNSUPPORTED
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_SIZE_INSUFFICIENT
- XR_ERROR_VALIDATION_FAILURE

The xrGetVirtualKeyboardTextureDataMETA function is defined as:

```
// Provided by XR_META_virtual_keyboard
XrResult xrGetVirtualKeyboardTextureDataMETA(
    XrVirtualKeyboardMETA                       keyboard,
    uint64_t                                    textureId,
    XrVirtualKeyboardTextureDataMETA*           textureData);
```

Parameter Descriptions

- keyboard is the XrVirtualKeyboardMETA handle.
- textureId is the ID of the texture that the application is querying data for.
- textureData is the returned XrVirtualKeyboardTextureDataMETA.

This function follows the two-call idiom for filling the textureData array in the XrVirtualKeyboardTextureDataMETA structure. Note that new texture data may be added after the runtime processes inputs from xrSendVirtualKeyboardInputMETA. Therefore, after sending new keyboard inputs the application **should** query the buffer size again before getting any texture data.

Valid Usage (Implicit)

- The XR_META_virtual_keyboard extension **must** be enabled prior to calling xrGetVirtualKeyboardTextureDataMETA
- keyboard **must** be a valid XrVirtualKeyboardMETA handle
- textureData **must** be a pointer to an XrVirtualKeyboardTextureDataMETA structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FEATURE_UNSUPPORTED
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_SIZE_INSUFFICIENT
- XR_ERROR_VALIDATION_FAILURE

The XrVirtualKeyboardTextureDataMETA structure is defined as:

```
// Provided by XR_META_virtual_keyboard
typedef struct XrVirtualKeyboardTextureDataMETA {
    XrStructureType    type;
    void*              next;
    uint32_t           textureWidth;
    uint32_t           textureHeight;
    uint32_t           bufferCapacityInput;
    uint32_t           bufferCountOutput;
    uint8_t*           buffer;
} XrVirtualKeyboardTextureDataMETA;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `textureWidth` is the pixel width of the texture to be updated.
- `textureHeight` is the pixel height of the texture to be updated.
- `bufferCapacityInput` is the capacity of `buffer`, or 0 to indicate a request to retrieve the required capacity.
- `bufferCountOutput` is filled in by the runtime with the byte count written or the required capacity in the case that `bufferCapacityInput` is insufficient.
- `buffer` is the pixel data in linear color space, RGBA 8-bit unsigned normalized integer format (i.e. GL_RGBA8 in OpenGL, VK_FORMAT_R8G8B8A8_UNORM in Vulkan).

Valid Usage (Implicit)

- The `XR_META_virtual_keyboard` extension **must** be enabled prior to using XrVirtualKeyboardTextureDataMETA
- `type` **must** be `XR_TYPE_VIRTUAL_KEYBOARD_TEXTURE_DATA_META`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- If `bufferCapacityInput` is not `0`, `buffer` **must** be a pointer to an array of `bufferCapacityInput` `uint8_t` values

12.173.12. Update render model animations

Besides checking for texture updates, each frame the application **should** also check for any animations to be applied to the render model. The runtime **may** use these animations to control the visibility of different keys, layout changes, and even modify key sizes and texture coordinates via morph targets. The application **can** get the animation states to be applied by calling xrGetVirtualKeyboardModelAnimationStatesMETA. This will return an array of XrVirtualKeyboardAnimationStateMETA which the application **should** apply to the render model, indexed by the GLTF animation array index order.

The xrGetVirtualKeyboardModelAnimationStatesMETA function is defined as:

```
// Provided by XR_META_virtual_keyboard
XrResult xrGetVirtualKeyboardModelAnimationStatesMETA(
    XrVirtualKeyboardMETA                       keyboard,
    XrVirtualKeyboardModelAnimationStatesMETA*  animationStates);
```

Parameter Descriptions

- `keyboard` is the XrVirtualKeyboardMETA handle.
- `animationStates` is the XrVirtualKeyboardModelAnimationStatesMETA.

This function follows the two-call idiom for filling the `animationStates` array in the XrVirtualKeyboardModelAnimationStatesMETA structure. Note that new animations may be added after the runtime processes inputs from xrSendVirtualKeyboardInputMETA. Therefore, after sending new keyboard inputs the application **should** query the buffer size again before getting any animation data.

Valid Usage (Implicit)

- The `XR_META_virtual_keyboard` extension **must** be enabled prior to calling xrGetVirtualKeyboardModelAnimationStatesMETA
- `keyboard` **must** be a valid XrVirtualKeyboardMETA handle
- `animationStates` **must** be a pointer to an XrVirtualKeyboardModelAnimationStatesMETA structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FEATURE_UNSUPPORTED
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_SIZE_INSUFFICIENT
- XR_ERROR_VALIDATION_FAILURE

The XrVirtualKeyboardAnimationStateMETA structure is defined as:

```
// Provided by XR_META_virtual_keyboard
typedef struct XrVirtualKeyboardAnimationStateMETA {
    XrStructureType    type;
    void*              next;
    int32_t            animationIndex;
    float              fraction;
} XrVirtualKeyboardAnimationStateMETA;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- animationIndex is the index of the animation to use for the render model.
- fraction is the normalized value between the start and end time of the animation.

Valid Usage (Implicit)

- The XR_META_virtual_keyboard extension **must** be enabled prior to using XrVirtualKeyboardAnimationStateMETA
- type **must** be XR_TYPE_VIRTUAL_KEYBOARD_ANIMATION_STATE_META
- next **must** be NULL or a valid pointer to the next structure in a structure chain

The XrVirtualKeyboardModelAnimationStatesMETA structure is defined as:

```
// Provided by XR_META_virtual_keyboard
typedef struct XrVirtualKeyboardModelAnimationStatesMETA {
    XrStructureType                           type;
    void*                                     next;
    uint32_t                                  stateCapacityInput;
    uint32_t                                  stateCountOutput;
    XrVirtualKeyboardAnimationStateMETA*      states;
} XrVirtualKeyboardModelAnimationStatesMETA;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- stateCapacityInput is the capacity of the states array, or 0 to indicate a request to retrieve the required capacity.
- stateCountOutput is filled in by the runtime with the count of XrVirtualKeyboardAnimationStateMETA written or the required capacity in the case that stateCapacityInput is insufficient.
- states is the array of XrVirtualKeyboardAnimationStateMETA to apply to the model.

Valid Usage (Implicit)

- The XR_META_virtual_keyboard extension **must** be enabled prior to using XrVirtualKeyboardModelAnimationStatesMETA
- type **must** be XR_TYPE_VIRTUAL_KEYBOARD_MODEL_ANIMATION_STATES_META
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- If stateCapacityInput is not 0, states **must** be a pointer to an array of stateCapacityInput XrVirtualKeyboardAnimationStateMETA structures

12.173.13. Send user input and text context

Since the application has control over how collision should be handled between the keyboard and other objects in the scene, it is up to the application to decide when to send input to the virtual keyboard. Per frame, for every input source the application wants to be applied to the keyboard, the application **should** create a XrVirtualKeyboardInputInfoMETA and call xrSendVirtualKeyboardInputMETA while also supplying the root pose of the interaction source.

The runtime **may** modify with an offset the given interactorRootPose if the given input is puncturing the keyboard. This is to give the effect that the virtual object cannot push through the keyboard and improves keyboard input perception. This is sometimes referred to as poke limiting.

To aid features like auto complete or whole word deletion, before sending input applications **should** populate a XrVirtualKeyboardTextContextChangeInfoMETA structure and call xrChangeVirtualKeyboardTextContextMETA to supply the runtime with the application's text context prior to the input cursor.

The xrSendVirtualKeyboardInputMETA function is defined as:

```
// Provided by XR_META_virtual_keyboard
XrResult xrSendVirtualKeyboardInputMETA(
    XrVirtualKeyboardMETA                       keyboard,
    const XrVirtualKeyboardInputInfoMETA*       info,
    XrPosef*                                    interactorRootPose);
```

Parameter Descriptions

- keyboard is the XrVirtualKeyboardMETA handle.
- info is the XrVirtualKeyboardInputInfoMETA detailing the input being sent to the runtime.
- interactorRootPose is an XrPosef defining the root pose of the input source. The runtime **may** modify this value to aid keyboard input perception.

The application **can** use values like a pointer pose as the `interactorRootPose` for `XR_VIRTUAL_KEYBOARD_INPUT_SOURCE_CONTROLLER_RAY_*` or `XR_VIRTUAL_KEYBOARD_INPUT_SOURCE_HAND_RAY_*` input sources, a point on a controller model for `XR_VIRTUAL_KEYBOARD_INPUT_SOURCE_CONTROLLER_DIRECT_*` input sources and the hand index tip pose for `XR_VIRTUAL_KEYBOARD_INPUT_SOURCE_HAND_DIRECT_INDEX_TIP_*`. Different input poses can be used to accommodate application specific controller or hand models.

Valid Usage (Implicit)

- The `XR_META_virtual_keyboard` extension **must** be enabled prior to calling xrSendVirtualKeyboardInputMETA
- `keyboard` **must** be a valid XrVirtualKeyboardMETA handle
- `info` **must** be a pointer to a valid XrVirtualKeyboardInputInfoMETA structure
- `interactorRootPose` **must** be a pointer to an XrPosef structure

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_FEATURE_UNSUPPORTED`
- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_POSE_INVALID`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_VALIDATION_FAILURE`

The XrVirtualKeyboardInputInfoMETA structure is defined as:

```
// Provided by XR_META_virtual_keyboard
typedef struct XrVirtualKeyboardInputInfoMETA {
    XrStructureType                           type;
    const void*                               next;
    XrVirtualKeyboardInputSourceMETA          inputSource;
    XrSpace                                   inputSpace;
    XrPosef                                   inputPoseInSpace;
    XrVirtualKeyboardInputStateFlagsMETA      inputState;
} XrVirtualKeyboardInputInfoMETA;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `inputSource` is an enum of XrVirtualKeyboardInputSourceMETA describing the source device and input mode type.
- `inputSpace` is an XrSpace previously created by a function such as xrCreateReferenceSpace.
- `inputPoseInSpace` is an XrPosef defining the position and orientation of the input's source pose within the natural reference frame of the input space.
- `inputState` is a bitmask of XrVirtualKeyboardInputStateFlagsMETA describing the button or pinch state of the `inputSource`.

Valid Usage (Implicit)

- The `XR_META_virtual_keyboard` extension **must** be enabled prior to using XrVirtualKeyboardInputInfoMETA
- `type` **must** be `XR_TYPE_VIRTUAL_KEYBOARD_INPUT_INFO_META`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `inputSource` **must** be a valid XrVirtualKeyboardInputSourceMETA value
- If `inputSpace` is not XR_NULL_HANDLE, `inputSpace` **must** be a valid XrSpace handle
- `inputState` **must** be `0` or a valid combination of XrVirtualKeyboardInputStateFlagBitsMETA values

The xrChangeVirtualKeyboardTextContextMETA function is defined as:

```
// Provided by XR_META_virtual_keyboard
XrResult xrChangeVirtualKeyboardTextContextMETA(
    XrVirtualKeyboardMETA                       keyboard,
    const XrVirtualKeyboardTextContextChangeInfoMETA* changeInfo);
```

Parameter Descriptions

- `keyboard` is the XrVirtualKeyboardMETA handle.
- `changeInfo` is the XrVirtualKeyboardTextContextChangeInfoMETA detailing prior input text context to the runtime.

Valid Usage (Implicit)

- The `XR_META_virtual_keyboard` extension **must** be enabled prior to calling xrChangeVirtualKeyboardTextContextMETA
- `keyboard` **must** be a valid XrVirtualKeyboardMETA handle
- `changeInfo` **must** be a pointer to a valid XrVirtualKeyboardTextContextChangeInfoMETA structure

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_FEATURE_UNSUPPORTED`
- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_VALIDATION_FAILURE`

The XrVirtualKeyboardTextContextChangeInfoMETA structure is defined as:

```
// Provided by XR_META_virtual_keyboard
typedef struct XrVirtualKeyboardTextContextChangeInfoMETA {
    XrStructureType    type;
    const void*        next;
    const char*        textContext;
} XrVirtualKeyboardTextContextChangeInfoMETA;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `textContext` is a pointer to a `char` buffer, should contain prior input text context terminated with a null character.

Valid Usage (Implicit)

- The `XR_META_virtual_keyboard` extension **must** be enabled prior to using XrVirtualKeyboardTextContextChangeInfoMETA
- `type` **must** be `XR_TYPE_VIRTUAL_KEYBOARD_TEXT_CONTEXT_CHANGE_INFO_META`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `textContext` **must** be a null-terminated UTF-8 string

12.173.14. Handling events

Each frame the application **should** listen for the following events sent by the runtime that reflects the state of the keyboard.

The XrEventDataVirtualKeyboardCommitTextMETA structure is defined as:

```
// Provided by XR_META_virtual_keyboard
typedef struct XrEventDataVirtualKeyboardCommitTextMETA {
    XrStructureType          type;
    const void*              next;
    XrVirtualKeyboardMETA    keyboard;
    char                     text[XR_MAX_VIRTUAL_KEYBOARD_COMMIT_TEXT_SIZE_META];
} XrEventDataVirtualKeyboardCommitTextMETA;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `keyboard` is the XrVirtualKeyboardMETA this event belongs to.
- `text` is the text string input by the keyboard.

The XrEventDataVirtualKeyboardCommitTextMETA event **must** be sent by the runtime when a character or string is input by the keyboard. The application **should** append to the text field that the keyboard is editing.

Valid Usage (Implicit)

- The `XR_META_virtual_keyboard` extension **must** be enabled prior to using XrEventDataVirtualKeyboardCommitTextMETA
- `type` **must** be `XR_TYPE_EVENT_DATA_VIRTUAL_KEYBOARD_COMMIT_TEXT_META`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

The XrEventDataVirtualKeyboardBackspaceMETA structure is defined as:

```
// Provided by XR_META_virtual_keyboard
typedef struct XrEventDataVirtualKeyboardBackspaceMETA {
    XrStructureType          type;
    const void*              next;
    XrVirtualKeyboardMETA    keyboard;
} XrEventDataVirtualKeyboardBackspaceMETA;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `keyboard` is the XrVirtualKeyboardMETA this event belongs to.

The XrEventDataVirtualKeyboardBackspaceMETA event **must** be sent by the runtime when the [Backspace] key is pressed. The application **should** update the text field that the keyboard is editing.

Valid Usage (Implicit)

- The XR_META_virtual_keyboard extension **must** be enabled prior to using XrEventDataVirtualKeyboardBackspaceMETA
- type **must** be XR_TYPE_EVENT_DATA_VIRTUAL_KEYBOARD_BACKSPACE_META
- next **must** be NULL or a valid pointer to the next structure in a structure chain

The XrEventDataVirtualKeyboardEnterMETA structure is defined as:

```
// Provided by XR_META_virtual_keyboard
typedef struct XrEventDataVirtualKeyboardEnterMETA {
    XrStructureType          type;
    const void*              next;
    XrVirtualKeyboardMETA    keyboard;
} XrEventDataVirtualKeyboardEnterMETA;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- keyboard is the XrVirtualKeyboardMETA this event belongs to.

The XrEventDataVirtualKeyboardEnterMETA event **must** be sent by the runtime when the [Enter] key is pressed. The application **should** respond accordingly (e.g. newline, accept, etc).

Valid Usage (Implicit)

- The XR_META_virtual_keyboard extension **must** be enabled prior to using XrEventDataVirtualKeyboardEnterMETA
- type **must** be XR_TYPE_EVENT_DATA_VIRTUAL_KEYBOARD_ENTER_META
- next **must** be NULL or a valid pointer to the next structure in a structure chain

The XrEventDataVirtualKeyboardShownMETA structure is defined as:

```
// Provided by XR_META_virtual_keyboard
typedef struct XrEventDataVirtualKeyboardShownMETA {
    XrStructureType          type;
    const void*              next;
    XrVirtualKeyboardMETA    keyboard;
} XrEventDataVirtualKeyboardShownMETA;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `keyboard` is the XrVirtualKeyboardMETA this event belongs to.

The XrEventDataVirtualKeyboardShownMETA event **must** be sent when the runtime has shown the keyboard render model (via animation). The application **should** update its state accordingly (e.g. update UI, pause simulation, etc).

Valid Usage (Implicit)

- The `XR_META_virtual_keyboard` extension **must** be enabled prior to using XrEventDataVirtualKeyboardShownMETA
- `type` **must** be `XR_TYPE_EVENT_DATA_VIRTUAL_KEYBOARD_SHOWN_META`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

The XrEventDataVirtualKeyboardHiddenMETA structure is defined as:

```
// Provided by XR_META_virtual_keyboard
typedef struct XrEventDataVirtualKeyboardHiddenMETA {
    XrStructureType          type;
    const void*              next;
    XrVirtualKeyboardMETA    keyboard;
} XrEventDataVirtualKeyboardHiddenMETA;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `keyboard` is the XrVirtualKeyboardMETA this event belongs to.

The XrEventDataVirtualKeyboardHiddenMETA event **must** be sent when the keyboard render model is hidden by the runtime (via animation). The application **should** update its state accordingly (e.g. update UI, resume simulation, etc).

Valid Usage (Implicit)

- The `XR_META_virtual_keyboard` extension **must** be enabled prior to using XrEventDataVirtualKeyboardHiddenMETA
- `type` **must** be `XR_TYPE_EVENT_DATA_VIRTUAL_KEYBOARD_HIDDEN_META`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

12.173.15. Example code for using virtual keyboard

The following example code demonstrates how to create and use the virtual keyboard.

```
XrInstance instance;  // previously initialized
XrSystemId system;    // previously initialized
XrSession session;    // previously initialized
XrSpace localSpace;   // previously initialized
XrPosef poseIdentity; // previously initialized

// XR_FB_render_model API previously initialized with xrGetInstanceProcAddr
PFN_xrEnumerateRenderModelPathsFB xrEnumerateRenderModelPathsFB = nullptr;
PFN_xrGetRenderModelPropertiesFB xrGetRenderModelPropertiesFB = nullptr;
PFN_xrLoadRenderModelFB xrLoadRenderModelFB = nullptr;

// XR_META_virtual_keyboard API previously initialized with xrGetInstanceProcAddr
PFN_xrCreateVirtualKeyboardMETA xrCreateVirtualKeyboardMETA = nullptr;
PFN_xrDestroyVirtualKeyboardMETA xrDestroyVirtualKeyboardMETA = nullptr;
PFN_xrCreateVirtualKeyboardSpaceMETA xrCreateVirtualKeyboardSpaceMETA = nullptr;
PFN_xrSuggestVirtualKeyboardLocationMETA xrSuggestVirtualKeyboardLocationMETA = nullptr;
PFN_xrGetVirtualKeyboardScaleMETA xrGetVirtualKeyboardScaleMETA = nullptr;
PFN_xrSetVirtualKeyboardModelVisibilityMETA xrSetVirtualKeyboardModelVisibilityMETA =
nullptr;
PFN_xrGetVirtualKeyboardModelAnimationStatesMETA
```

```
xrGetVirtualKeyboardModelAnimationStatesMETA = nullptr;
PFN_xrGetVirtualKeyboardDirtyTexturesMETA xrGetVirtualKeyboardDirtyTexturesMETA =
nullptr;
PFN_xrGetVirtualKeyboardTextureDataMETA xrGetVirtualKeyboardTextureDataMETA = nullptr;
PFN_xrSendVirtualKeyboardInputMETA xrSendVirtualKeyboardInputMETA = nullptr;

XrVirtualKeyboardMETA keyboardHandle{XR_NULL_HANDLE};
XrSpace keyboardSpace{XR_NULL_HANDLE};
XrRenderModelKeyFB keyboardModelKey{XR_NULL_RENDER_MODEL_KEY_FB};

/// Check virtual keyboard support
XrSystemVirtualKeyboardPropertiesMETA virtualKeyboardProps{
    .type = XR_TYPE_SYSTEM_VIRTUAL_KEYBOARD_PROPERTIES_META,
};
XrSystemProperties systemProperties{
    .type = XR_TYPE_SYSTEM_PROPERTIES,
    .next = &virtualKeyboardProps,
};
CHK_XR(xrGetSystemProperties(instance, system, &systemProperties));
if (virtualKeyboardProps.supportsVirtualKeyboard == XR_FALSE) {
  return; // Virtual keyboard not supported
}

/// Create virtual keyboard and space
XrVirtualKeyboardCreateInfoMETA createInfo{
    .type = XR_TYPE_VIRTUAL_KEYBOARD_CREATE_INFO_META,
};
CHK_XR(xrCreateVirtualKeyboardMETA(session, &createInfo, &keyboardHandle));

XrVirtualKeyboardSpaceCreateInfoMETA spaceCreateInfo{
    .type = XR_TYPE_VIRTUAL_KEYBOARD_SPACE_CREATE_INFO_META,
    .locationType = XR_VIRTUAL_KEYBOARD_LOCATION_TYPE_CUSTOM_META,
    .space = localSpace,
    .poseInSpace = poseIdentity,
};
CHK_XR(xrCreateVirtualKeyboardSpaceMETA(session, keyboardHandle, &spaceCreateInfo,
&keyboardSpace));

/// Get render model key
uint32_t pathCount = 0;
CHK_XR(xrEnumerateRenderModelPathsFB(session, pathCount, &pathCount, nullptr));
std::vector<XrRenderModelPathInfoFB> pathInfos(pathCount, {
    .type = XR_TYPE_RENDER_MODEL_PATH_INFO_FB,
});
CHK_XR(xrEnumerateRenderModelPathsFB(session, pathCount, &pathCount, pathInfos.data()));

for (const auto& info : pathInfos) {
  char pathString[XR_MAX_PATH_LENGTH];
```

```
  uint32_t countOutput = 0;
  CHK_XR(xrPathToString(instance, info.path, XR_MAX_PATH_LENGTH, &countOutput,
pathString));

  if (strcmp(pathString, "/model_meta/keyboard/virtual") == 0) {
    XrRenderModelCapabilitiesRequestFB capReq{
        .type = XR_TYPE_RENDER_MODEL_CAPABILITIES_REQUEST_FB,
        .flags = XR_RENDER_MODEL_SUPPORTS_GLTF_2_0_SUBSET_2_BIT_FB,
    };
    XrRenderModelPropertiesFB prop{
        .type = XR_TYPE_RENDER_MODEL_PROPERTIES_FB,
        .next = &capReq,
    };
    CHK_XR(xrGetRenderModelPropertiesFB(session, info.path, &prop));
    keyboardModelKey = prop.modelKey;
    break;
  }
}

if (keyboardModelKey == XR_NULL_RENDER_MODEL_KEY_FB) {
  return; // Model not available
}

/// Load render model
XrRenderModelLoadInfoFB loadInfo{
    .type = XR_TYPE_RENDER_MODEL_LOAD_INFO_FB,
    .modelKey = keyboardModelKey,
};
XrRenderModelBufferFB renderModelbuffer{
    .type = XR_TYPE_RENDER_MODEL_BUFFER_FB,
};
CHK_XR((xrLoadRenderModelFB(session, &loadInfo, &renderModelbuffer)));
std::vector<uint8_t> modelBuffer(renderModelbuffer.bufferCountOutput);
renderModelbuffer.buffer = modelBuffer.data();
renderModelbuffer.bufferCapacityInput = renderModelbuffer.bufferCountOutput;
CHK_XR((xrLoadRenderModelFB(session, &loadInfo, &renderModelbuffer)));
// >>> Application loads the glTF model in `modelBuffer`, keeping a reference to the
model animations and any textures with a URI texture id. See `Extend glTF render model
support`.

/// Show render model
XrVirtualKeyboardModelVisibilitySetInfoMETA modelVisibility{
    .type = XR_TYPE_VIRTUAL_KEYBOARD_MODEL_VISIBILITY_SET_INFO_META,
    .visible = XR_TRUE,
};
CHK_XR(xrSetVirtualKeyboardModelVisibilityMETA(keyboardHandle, &modelVisibility));

while (!quit) {
```

```
  // ...
  // For every frame in frame loop
  // ...
  XrFrameState frameState; // previously returned from xrWaitFrame
  const XrTime time = frameState.predictedDisplayTime;

  XrVirtualKeyboardLocationInfoMETA locationInfo{
      .type = XR_TYPE_VIRTUAL_KEYBOARD_LOCATION_INFO_META,
  };
  // >>> Application sets desired location and scale in `locationInfo`
  CHK_XR(xrSuggestVirtualKeyboardLocationMETA(keyboardHandle, &locationInfo));

  // For each input source:
  {
    XrVirtualKeyboardInputInfoMETA inputInfo{
        .type = XR_TYPE_VIRTUAL_KEYBOARD_INPUT_INFO_META,
    };
    // >>> Application sets input source data in `inputInfo`
    XrPosef interactorRootPose;
    CHK_XR(xrSendVirtualKeyboardInputMETA(keyboardHandle, &inputInfo, &
interactorRootPose));
    // >>> Application uses `interactorRootPose` as feedback for poke limiting
  }

  uint32_t textureIdCountOutput = 0;
  CHK_XR(xrGetVirtualKeyboardDirtyTexturesMETA(keyboardHandle, 0, &textureIdCountOutput,
nullptr));
  std::vector<uint64_t> dirtyTextureIds(textureIdCountOutput);
  CHK_XR(xrGetVirtualKeyboardDirtyTexturesMETA(keyboardHandle, textureIdCountOutput,
&textureIdCountOutput, dirtyTextureIds.data()));
  for (const uint64_t textureId : dirtyTextureIds) {
    XrVirtualKeyboardTextureDataMETA textureData{
        .type = XR_TYPE_VIRTUAL_KEYBOARD_TEXTURE_DATA_META,
    };
    CHK_XR(xrGetVirtualKeyboardTextureDataMETA(keyboardHandle, textureId, &textureData));
    std::vector<uint8_t> textureDataBuffer(textureData.bufferCountOutput);
    textureData.bufferCapacityInput = textureData.bufferCountOutput;
    textureData.buffer = textureDataBuffer.data();
    CHK_XR(xrGetVirtualKeyboardTextureDataMETA(keyboardHandle, textureId, &textureData));
    // >>> Application applies `textureData` to the glTF texture referenced by
`textureId`
  }

  XrVirtualKeyboardModelAnimationStatesMETA animationStates{
      .type = XR_TYPE_VIRTUAL_KEYBOARD_MODEL_ANIMATION_STATES_META,
  };
  CHK_XR(xrGetVirtualKeyboardModelAnimationStatesMETA(keyboardHandle, &animationStates));
  std::vector<XrVirtualKeyboardAnimationStateMETA> animationStatesBuffer(animationStates
```

```
.stateCountOutput, {
      .type = XR_TYPE_VIRTUAL_KEYBOARD_ANIMATION_STATE_META,
  });
  animationStates.stateCapacityInput = animationStates.stateCountOutput;
  animationStates.states = animationStatesBuffer.data();
  CHK_XR(xrGetVirtualKeyboardModelAnimationStatesMETA(keyboardHandle, &animationStates));
  for (uint32_t i = 0; i < animationStates.stateCountOutput; ++i) {
    const auto& animationState = animationStates.states[i];
    // >>> Application applies `animationState` to the corresponding glTF model animation
  }

  XrSpaceLocation keyboardLocation{
      .type = XR_TYPE_SPACE_LOCATION,
  };
  CHK_XR(xrLocateSpace(keyboardSpace, localSpace, time, &keyboardLocation));
  float keyboardScale;
  CHK_XR(xrGetVirtualKeyboardScaleMETA(keyboardHandle, &keyboardScale));
  // >>> Application renders model with `keyboardLocation` and `keyboardScale`
}

CHK_XR(xrDestroyVirtualKeyboardMETA(keyboardHandle));
```

New Object Types

```
// Provided by XR_META_virtual_keyboard
XR_DEFINE_HANDLE(XrVirtualKeyboardMETA)
```

XrVirtualKeyboardMETA represents a virtual keyboard instance.

New Flag Types

```
// Provided by XR_META_virtual_keyboard
typedef XrFlags64 XrVirtualKeyboardInputStateFlagsMETA;
```

```
// Provided by XR_META_virtual_keyboard
// Flag bits for XrVirtualKeyboardInputStateFlagsMETA
static const XrVirtualKeyboardInputStateFlagsMETA
XR_VIRTUAL_KEYBOARD_INPUT_STATE_PRESSED_BIT_META = 0x00000001;
```

Flag Descriptions

- XR_VIRTUAL_KEYBOARD_INPUT_STATE_PRESSED_BIT_META — If the input source is considered 'pressed' at all. Pinch for hands, Primary button for controllers.

New Enum Constants

- XR_MAX_VIRTUAL_KEYBOARD_COMMIT_TEXT_SIZE_META

XrStructureType enumeration is extended with:

- XR_TYPE_SYSTEM_VIRTUAL_KEYBOARD_PROPERTIES_META
- XR_TYPE_VIRTUAL_KEYBOARD_CREATE_INFO_META
- XR_TYPE_VIRTUAL_KEYBOARD_SPACE_CREATE_INFO_META
- XR_TYPE_VIRTUAL_KEYBOARD_LOCATION_INFO_META
- XR_TYPE_VIRTUAL_KEYBOARD_MODEL_VISIBILITY_SET_INFO_META
- XR_TYPE_VIRTUAL_KEYBOARD_ANIMATION_STATE_META
- XR_TYPE_VIRTUAL_KEYBOARD_MODEL_ANIMATION_STATES_META
- XR_TYPE_VIRTUAL_KEYBOARD_TEXTURE_DATA_META
- XR_TYPE_VIRTUAL_KEYBOARD_INPUT_INFO_META
- XR_TYPE_VIRTUAL_KEYBOARD_TEXT_CONTEXT_CHANGE_INFO_META
- XR_TYPE_EVENT_DATA_VIRTUAL_KEYBOARD_COMMIT_TEXT_META
- XR_TYPE_EVENT_DATA_VIRTUAL_KEYBOARD_BACKSPACE_META
- XR_TYPE_EVENT_DATA_VIRTUAL_KEYBOARD_ENTER_META
- XR_TYPE_EVENT_DATA_VIRTUAL_KEYBOARD_SHOWN_META
- XR_TYPE_EVENT_DATA_VIRTUAL_KEYBOARD_HIDDEN_META

New Defines

New Enums

The possible location types are specified by the XrVirtualKeyboardLocationTypeMETA enumeration:

```
// Provided by XR_META_virtual_keyboard
typedef enum XrVirtualKeyboardLocationTypeMETA {
    XR_VIRTUAL_KEYBOARD_LOCATION_TYPE_CUSTOM_META = 0,
    XR_VIRTUAL_KEYBOARD_LOCATION_TYPE_FAR_META = 1,
    XR_VIRTUAL_KEYBOARD_LOCATION_TYPE_DIRECT_META = 2,
    XR_VIRTUAL_KEYBOARD_LOCATION_TYPE_MAX_ENUM_META = 0x7FFFFFFF
} XrVirtualKeyboardLocationTypeMETA;
```

Enumerant Descriptions

- XR_VIRTUAL_KEYBOARD_LOCATION_TYPE_CUSTOM_META
  Indicates that the application will provide the position and scale of the keyboard.
- XR_VIRTUAL_KEYBOARD_LOCATION_TYPE_FAR_META
  Indicates that the runtime will set the position and scale for far field keyboard.
- XR_VIRTUAL_KEYBOARD_LOCATION_TYPE_DIRECT_META
  Indicates that the runtime will set the position and scale for direct interaction keyboard.

The possible input sources are specified by the XrVirtualKeyboardInputSourceMETA enumeration:

```
// Provided by XR_META_virtual_keyboard
typedef enum XrVirtualKeyboardInputSourceMETA {
    XR_VIRTUAL_KEYBOARD_INPUT_SOURCE_CONTROLLER_RAY_LEFT_META = 1,
    XR_VIRTUAL_KEYBOARD_INPUT_SOURCE_CONTROLLER_RAY_RIGHT_META = 2,
    XR_VIRTUAL_KEYBOARD_INPUT_SOURCE_HAND_RAY_LEFT_META = 3,
    XR_VIRTUAL_KEYBOARD_INPUT_SOURCE_HAND_RAY_RIGHT_META = 4,
    XR_VIRTUAL_KEYBOARD_INPUT_SOURCE_CONTROLLER_DIRECT_LEFT_META = 5,
    XR_VIRTUAL_KEYBOARD_INPUT_SOURCE_CONTROLLER_DIRECT_RIGHT_META = 6,
    XR_VIRTUAL_KEYBOARD_INPUT_SOURCE_HAND_DIRECT_INDEX_TIP_LEFT_META = 7,
    XR_VIRTUAL_KEYBOARD_INPUT_SOURCE_HAND_DIRECT_INDEX_TIP_RIGHT_META = 8,
    XR_VIRTUAL_KEYBOARD_INPUT_SOURCE_MAX_ENUM_META = 0x7FFFFFFF
} XrVirtualKeyboardInputSourceMETA;
```

Enum	Description
XR_VIRTUAL_KEYBOARD_INPUT_SOURCE_CONTROLLER_RAY_LEFT_META	Left controller ray.
XR_VIRTUAL_KEYBOARD_INPUT_SOURCE_CONTROLLER_RAY_RIGHT_META	Right controller ray.
XR_VIRTUAL_KEYBOARD_INPUT_SOURCE_HAND_RAY_LEFT_META	Left hand ray.
XR_VIRTUAL_KEYBOARD_INPUT_SOURCE_HAND_RAY_RIGHT_META	Right hand ray.

Enum	Description
XR_VIRTUAL_KEYBOARD_INPUT_SOURCE_CONTROLLER_DIRECT_LEFT_META	Left controller direct touch.
XR_VIRTUAL_KEYBOARD_INPUT_SOURCE_CONTROLLER_DIRECT_RIGHT_META	Right controller direct touch.
XR_VIRTUAL_KEYBOARD_INPUT_SOURCE_HAND_DIRECT_INDEX_TIP_LEFT_META	Left hand direct touch.
XR_VIRTUAL_KEYBOARD_INPUT_SOURCE_HAND_DIRECT_INDEX_TIP_RIGHT_META	Right hand direct touch.

New Structures

- XrSystemVirtualKeyboardPropertiesMETA
- XrVirtualKeyboardCreateInfoMETA
- XrVirtualKeyboardSpaceCreateInfoMETA
- XrVirtualKeyboardLocationInfoMETA
- XrVirtualKeyboardModelVisibilitySetInfoMETA
- XrVirtualKeyboardAnimationStateMETA
- XrVirtualKeyboardModelAnimationStatesMETA
- XrVirtualKeyboardTextureDataMETA
- XrVirtualKeyboardInputInfoMETA
- XrVirtualKeyboardTextContextChangeInfoMETA
- XrEventDataVirtualKeyboardCommitTextMETA
- XrEventDataVirtualKeyboardBackspaceMETA
- XrEventDataVirtualKeyboardEnterMETA
- XrEventDataVirtualKeyboardShownMETA
- XrEventDataVirtualKeyboardHiddenMETA

New Functions

- xrCreateVirtualKeyboardMETA
- xrDestroyVirtualKeyboardMETA
- xrCreateVirtualKeyboardSpaceMETA
- xrSuggestVirtualKeyboardLocationMETA
- xrGetVirtualKeyboardScaleMETA
- xrSetVirtualKeyboardModelVisibilityMETA
- xrGetVirtualKeyboardModelAnimationStatesMETA

- xrGetVirtualKeyboardDirtyTexturesMETA
- xrGetVirtualKeyboardTextureDataMETA
- xrSendVirtualKeyboardInputMETA
- xrChangeVirtualKeyboardTextContextMETA

Issues

Version History

- Revision 1, 2023-04-14 (Peter Chan, Brent Housen)
  - Initial extension description

12.174. XR_META_vulkan_swapchain_create_info

Name String

`XR_META_vulkan_swapchain_create_info`

Extension Type

Instance extension

Registered Extension Number

228

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Last Modified Date

2022-05-19

IP Status

No known IP claims.

Contributors

John Kearney, Meta Platforms
Andreas L. Selvik, Meta Platforms
Jakob Bornecrantz, Collabora
Ross Ning, Meta Platforms

Overview

Using this extension, a Vulkan-based application **can** pass through additional VkImageCreateFlags or VkImageUsageFlags by chaining an XrVulkanSwapchainCreateInfoMETA structure to the XrSwapchainCreateInfo when calling xrCreateSwapchain.

The application is still encouraged to use the common bits like XR_SWAPCHAIN_USAGE_TRANSFER_SRC_BIT defined in XrSwapchainUsageFlags. However, the application **may** present both XR_SWAPCHAIN_USAGE_TRANSFER_SRC_BIT in XrSwapchainUsageFlags and VK_IMAGE_USAGE_TRANSFER_SRC_BIT in VkImageUsageFlags at the same time.

The application **must** enable the corresponding Vulkan extensions before requesting additional Vulkan flags. For example, VK_EXT_fragment_density_map device extension **must** be enabled if an application requests VK_IMAGE_CREATE_SUBSAMPLED_BIT_EXT bit. Otherwise, it **may** cause undefined behavior, including an application crash.

Runtimes that implement this extension **must** support the XR_KHR_vulkan_enable or the XR_KHR_vulkan_enable2 extension.

New Object Types

New Flag Types

New Enum Constants

New Enums

New Structures

```
// Provided by XR_META_vulkan_swapchain_create_info
typedef struct XrVulkanSwapchainCreateInfoMETA {
    XrStructureType       type;
    const void*           next;
    VkImageCreateFlags    additionalCreateFlags;
    VkImageUsageFlags     additionalUsageFlags;
} XrVulkanSwapchainCreateInfoMETA;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `additionalCreateFlags` is a bitmask of `VkImageCreateFlags` describing additional parameters of an image.
- `additionalUsageFlags` is a bitmask of `VkImageUsageFlags` describing additional parameters of an image.

The runtime **must** return `XR_ERROR_FEATURE_UNSUPPORTED` if any bit of either `additionalCreateFlags` or `additionalUsageFlags` is not supported.

Valid Usage (Implicit)

- The `XR_META_vulkan_swapchain_create_info` extension **must** be enabled prior to using XrVulkanSwapchainCreateInfoMETA
- `type` **must** be `XR_TYPE_VULKAN_SWAPCHAIN_CREATE_INFO_META`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `additionalCreateFlags` **must** be a valid `VkImageCreateFlags` value
- `additionalUsageFlags` **must** be a valid `VkImageUsageFlags` value

New Functions

Issues

Version History

- Revision 1, 2022-05-05 (Ross Ning)
  - Initial draft

12.175. XR_ML_compat

Name String

`XR_ML_compat`

Extension Type

Instance extension

Registered Extension Number

138

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Last Modified Date

2022-11-08

Contributors

Ron Bessems, Magic Leap

Overview

This extension provides functionality to facilitate transitioning from Magic Leap SDK to OpenXR SDK, most notably interoperability between Coordinate Frame UUIDs and XrSpace.

New Enum Constants

XrStructureType enumeration is extended with:

- `XR_TYPE_COORDINATE_SPACE_CREATE_INFO_ML`

New Structures

The XrCoordinateSpaceCreateInfoML structure is defined as:

```
// Provided by XR_ML_compat
typedef struct XrCoordinateSpaceCreateInfoML {
    XrStructureType         type;
    const void*             next;
    MLCoordinateFrameUID    cfuid;
    XrPosef                 poseInCoordinateSpace;
} XrCoordinateSpaceCreateInfoML;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `cfuid` is the `MLCoordinateFrameUID` as generated by the non-OpenXR API in the Magic Leap SDK.
- `poseInCoordinateSpace` is an XrPosef defining the position and orientation of the new space's origin within the natural reference frame of the `cfuid`.

XrCoordinateSpaceCreateInfoML is provided as input when calling xrCreateSpaceFromCoordinateFrameUIDML to convert a Magic Leap SDK generated `MLCoordinateFrameUID` to an XrSpace. The conversion only needs to be done once even if the underlying `MLCoordinateFrameUID` changes its pose.

Valid Usage (Implicit)

- The `XR_ML_compat` extension **must** be enabled prior to using XrCoordinateSpaceCreateInfoML
- `type` **must** be `XR_TYPE_COORDINATE_SPACE_CREATE_INFO_ML`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `cfuid` **must** be a valid `MLCoordinateFrameUID` value

New Functions

The xrCreateSpaceFromCoordinateFrameUIDML function is defined as:

```
// Provided by XR_ML_compat
XrResult xrCreateSpaceFromCoordinateFrameUIDML(
    XrSession                                   session,
    const XrCoordinateSpaceCreateInfoML *       createInfo,
    XrSpace*                                    space);
```

Parameter Descriptions

- `session` is a handle to an XrSession previously created with xrCreateSession.
- `createInfo` is the XrCoordinateSpaceCreateInfoML used to specify the space.
- `space` is the returned space handle.

The service that created the underlying XrCoordinateSpaceCreateInfoML::`cfuid` **must** remain active for

the lifetime of the XrSpace. If xrLocateSpace is called on a space created from an XrCoordinateSpaceCreateInfoML::cfuid from a no-longer-active service, the runtime **may** set XrSpaceLocation::locationFlags to 0.

XrSpace handles are destroyed using xrDestroySpace.

Valid Usage (Implicit)

- The XR_ML_compat extension **must** be enabled prior to calling xrCreateSpaceFromCoordinateFrameUIDML
- session **must** be a valid XrSession handle
- createInfo **must** be a pointer to a valid XrCoordinateSpaceCreateInfoML structure
- space **must** be a pointer to an XrSpace handle

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_POSE_INVALID
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

Issues

Version History

- Revision 1, 2022-11-08 (Ron Bessems)
  - Initial extension description

12.176. XR_ML_facial_expression

Name String

`XR_ML_facial_expression`

Extension Type

Instance extension

Registered Extension Number

483

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Last Modified Date

2023-11-22

Contributors

Dushan Vasilevski, Magic Leap
Karthik Kadappan, Magic Leap
Ron Bessems, Magic Leap
Johannes Fung, Magic Leap

12.176.1. Overview

This extension provides the weights of facial blend shapes usable for a variety of purposes, such as mood monitoring or interpolating the expression of an avatar's face.



Permissions

Android applications **must** have the com.magicleap.permission.FACIAL_EXPRESSION permission listed in their manifest and granted to use this extension, otherwise xrCreateFacialExpressionClientML will return a `XR_ERROR_FACIAL_EXPRESSION_PERMISSION_DENIED_ML` error. (protection level: dangerous)

12.176.2. Inspect system capability

The XrSystemFacialExpressionPropertiesML structure is defined as:

```
// Provided by XR_ML_facial_expression
typedef struct XrSystemFacialExpressionPropertiesML {
    XrStructureType    type;
    void*              next;
    XrBool32           supportsFacialExpression;
} XrSystemFacialExpressionPropertiesML;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension. For more details about the structure chain, see the structure being extended (XrSystemProperties).
- `supportsFacialExpression` is an `XrBool32`, indicating if current system is capable of parsing facial expressions enumerated in XrFacialBlendShapeML

An application **can** inspect whether the system is capable of parsing facial blend shapes by extending the XrSystemProperties with XrSystemFacialExpressionPropertiesML structure when calling xrGetSystemProperties.

If a runtime returns `XR_FALSE` for `supportsFacialExpression`, the runtime **must** return `XR_ERROR_FEATURE_UNSUPPORTED` from xrCreateFacialExpressionClientML.

Valid Usage (Implicit)

- The `XR_ML_facial_expression` extension **must** be enabled prior to using XrSystemFacialExpressionPropertiesML
- `type` **must** be `XR_TYPE_SYSTEM_FACIAL_EXPRESSION_PROPERTIES_ML`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

12.176.3. Create a facial expression client handle

The XrFacialExpressionClientML handle represents the resources for parsing facial expressions.

```
// Provided by XR_ML_facial_expression
XR_DEFINE_HANDLE(XrFacialExpressionClientML)
```

This handle is used to obtain blend shapes using the xrGetFacialExpressionBlendShapePropertiesML function.

The xrCreateFacialExpressionClientML function is defined as:

```
// Provided by XR_ML_facial_expression
XrResult xrCreateFacialExpressionClientML(
    XrSession                                   session,
    const XrFacialExpressionClientCreateInfoML* createInfo,
    XrFacialExpressionClientML*                 facialExpressionClient);
```

Parameter Descriptions

- `session` is an XrSession in which the facial expression client will be active.
- `createInfo` is a pointer to the XrFacialExpressionClientCreateInfoML used to specify the facial expression client.
- `facialExpressionClient` is a pointer to the returned XrFacialExpressionClientML handle.

An application **can** create an XrFacialExpressionClientML handle using xrCreateFacialExpressionClientML function.

If the system does not support parsing facial expressions, the runtime **must** return `XR_ERROR_FEATURE_UNSUPPORTED` from xrCreateFacialExpressionClientML. In this case, the runtime **must** also return `XR_FALSE` for XrSystemFacialExpressionPropertiesML::`supportsFacialExpression` when the function xrGetSystemProperties is called.

Valid Usage (Implicit)

- The `XR_ML_facial_expression` extension **must** be enabled prior to calling xrCreateFacialExpressionClientML
- `session` **must** be a valid XrSession handle
- `createInfo` **must** be a pointer to a valid XrFacialExpressionClientCreateInfoML structure
- `facialExpressionClient` **must** be a pointer to an XrFacialExpressionClientML handle

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FACIAL_EXPRESSION_PERMISSION_DENIED_ML
- XR_ERROR_FEATURE_UNSUPPORTED
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The XrFacialExpressionClientCreateInfoML structure is defined as follows:

```
// Provided by XR_ML_facial_expression
typedef struct XrFacialExpressionClientCreateInfoML {
    XrStructureType                type;
    const void*                    next;
    uint32_t                       requestedCount;
    const XrFacialBlendShapeML*    requestedFacialBlendShapes;
} XrFacialExpressionClientCreateInfoML;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- requestedCount is the size of the requestedFacialBlendShapes array
- requestedFacialBlendShapes is a pointer to an application defined array containing the blend shapes for the runtime to target.

Note that although the naming convention for `requestedCount` does not align with `requestedFacialBlendShapes`, they are coupled together.

The XrFacialExpressionClientCreateInfoML structure describes the information to create an XrFacialExpressionClientML handle.

An application specifies the blend shapes they want to query by creating an array of type XrFacialBlendShapeML and passing it to `requestedFacialBlendShapes` along with the corresponding `requestedCount`.

The application **can** also pass in `NULL` into `requestedFacialBlendShapes` to capture the entirety of XrFacialBlendShapeML. However, for performance reasons, it **may** be better to be explicit about what blend shapes to query for performance reasons since some blend shapes **may** be queried by the runtime at a greater frequency than other blend shapes.

Valid Usage (Implicit)

- The `XR_ML_facial_expression` extension **must** be enabled prior to using XrFacialExpressionClientCreateInfoML
- `type` **must** be `XR_TYPE_FACIAL_EXPRESSION_CLIENT_CREATE_INFO_ML`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- If `requestedCount` is not `0`, `requestedFacialBlendShapes` **must** be a pointer to an array of `requestedCount` valid XrFacialBlendShapeML values

12.176.4. Destroy a facial expression client handle

The xrDestroyFacialExpressionClientML function is defined as:

```
// Provided by XR_ML_facial_expression
XrResult xrDestroyFacialExpressionClientML(
    XrFacialExpressionClientML                  facialExpressionClient);
```

Parameter Descriptions

- `facialExpressionClient` is an XrFacialExpressionClientML previously created by xrCreateFacialExpressionClientML.

The xrDestroyFacialExpressionClientML function releases the `facialExpressionClient` and the underlying resources.

Valid Usage (Implicit)

- The XR_ML_facial_expression extension **must** be enabled prior to calling xrDestroyFacialExpressionClientML
- facialExpressionClient **must** be a valid XrFacialExpressionClientML handle

Thread Safety

- Access to facialExpressionClient, and any child handles, **must** be externally synchronized

Return Codes

Success

- XR_SUCCESS

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_RUNTIME_FAILURE

12.176.5. Obtain facial expression blend shapes

The xrGetFacialExpressionBlendShapePropertiesML function is defined as:

```
// Provided by XR_ML_facial_expression
XrResult xrGetFacialExpressionBlendShapePropertiesML(
    XrFacialExpressionClientML                  facialExpressionClient,
    const XrFacialExpressionBlendShapeGetInfoML* blendShapeGetInfo,
    uint32_t                                    blendShapeCount,
    XrFacialExpressionBlendShapePropertiesML*   blendShapes);
```

Parameter Descriptions

- `facialExpressionClient` is an XrFacialExpressionClientML previously created by xrCreateFacialExpressionClientML.
- `blendShapeGetInfo` is a pointer to XrFacialExpressionBlendShapeGetInfoML describing information to obtain facial expression.
- `blendShapeCount` is a `uint32_t` specifying the size of the `blendShapes` array.
- `blendShapes` is an application allocated array of XrFacialExpressionBlendShapePropertiesML that contains the blend shapes information. This array acts as both input and output for this function. The array **may** be a subset of blend shapes.

XrFacialExpressionBlendShapePropertiesML is better thought of as a mutable state rather than an immutable property. In general, OpenXR convention tries to keep property data types to be immutable.

The xrGetFacialExpressionBlendShapePropertiesML function returns the XrFacialExpressionBlendShapePropertiesML of each blend shape requested in XrFacialExpressionClientCreateInfoML

Each XrFacialExpressionBlendShapePropertiesML in `blendShapes` **must** have its `requestedFacialBlendShape` member variable initialized before being passed into xrGetFacialExpressionBlendShapePropertiesML.

If a blend shape in `blendShapes` is not enabled in xrCreateFacialExpressionClientML, the runtime **must** return `XR_ERROR_VALIDATION_FAILURE`.

For unsupported blend shapes, the runtime **must** clear XrFacialExpressionBlendShapePropertiesML ::`flags` and the runtime **must** also return `XR_SUCCESS`.

Valid Usage (Implicit)

- The `XR_ML_facial_expression` extension **must** be enabled prior to calling xrGetFacialExpressionBlendShapePropertiesML
- `facialExpressionClient` **must** be a valid XrFacialExpressionClientML handle
- `blendShapeGetInfo` **must** be a pointer to a valid XrFacialExpressionBlendShapeGetInfoML structure
- `blendShapes` **must** be a pointer to an array of `blendShapeCount` XrFacialExpressionBlendShapePropertiesML structures
- The `blendShapeCount` parameter **must** be greater than `0`

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_TIME_INVALID
- XR_ERROR_VALIDATION_FAILURE

The XrFacialExpressionBlendShapeGetInfoML structure is defined as:

```
// Provided by XR_ML_facial_expression
typedef struct XrFacialExpressionBlendShapeGetInfoML {
    XrStructureType    type;
    const void*        next;
} XrFacialExpressionBlendShapeGetInfoML;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.

The XrFacialExpressionBlendShapeGetInfoML structure specifies properties about blend shapes desired by an application. It **must** be passed into xrGetFacialExpressionBlendShapePropertiesML and is currently empty for future extensibility.

Valid Usage (Implicit)

- The XR_ML_facial_expression extension **must** be enabled prior to using XrFacialExpressionBlendShapeGetInfoML
- type **must** be XR_TYPE_FACIAL_EXPRESSION_BLEND_SHAPE_GET_INFO_ML
- next **must** be NULL or a valid pointer to the next structure in a structure chain

The XrFacialExpressionBlendShapePropertiesML structure is defined as:

```
// Provided by XR_ML_facial_expression
typedef struct XrFacialExpressionBlendShapePropertiesML {
    XrStructureType                                 type;
    void*                                           next;
    XrFacialBlendShapeML                            requestedFacialBlendShape;
    float                                           weight;
    XrFacialExpressionBlendShapePropertiesFlagsML   flags;
    XrTime                                          time;
} XrFacialExpressionBlendShapePropertiesML;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- requestedFacialBlendShape is the type of XrFacialBlendShapeML to query the runtime for.
- weight is a float describing the weight of a facial expression's blend shape.
- flags is an XrFacialExpressionBlendShapePropertiesFlagsML whose bits are specified in XrFacialExpressionBlendShapePropertiesFlagBitsML
- time is the XrTime that corresponds to the moment at which the requestedFacialBlendShape is captured.

XrFacialExpressionBlendShapePropertiesML structure holds the facial expression.

If requestedFacialBlendShape does not correspond to any XrFacialBlendShapeML passed into xrCreateFacialExpressionClientML then the XR_FACIAL_EXPRESSION_BLEND_SHAPE_PROPERTIES_VALID_BIT_ML and XR_FACIAL_EXPRESSION_BLEND_SHAPE_PROPERTIES_TRACKED_BIT_ML of flags **must** be unset. If the requestedFacialBlendShape is not available at sample time time then XR_FACIAL_EXPRESSION_BLEND_SHAPE_PROPERTIES_TRACKED_BIT_ML **must** be set to false.

The runtime **must** populate weight with the weight of the queried blend shape.

Valid Usage (Implicit)

- The XR_ML_facial_expression extension **must** be enabled prior to using XrFacialExpressionBlendShapePropertiesML
- type **must** be XR_TYPE_FACIAL_EXPRESSION_BLEND_SHAPE_PROPERTIES_ML
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- requestedFacialBlendShape **must** be a valid XrFacialBlendShapeML value

12.176.6. Conventions of blend shapes

This extension defines the following blend shapes for tracking facial expressions.

```
// Provided by XR_ML_facial_expression
typedef enum XrFacialBlendShapeML {
    XR_FACIAL_BLEND_SHAPE_BROW_LOWERER_L_ML = 0,
    XR_FACIAL_BLEND_SHAPE_BROW_LOWERER_R_ML = 1,
    XR_FACIAL_BLEND_SHAPE_CHEEK_RAISER_L_ML = 2,
    XR_FACIAL_BLEND_SHAPE_CHEEK_RAISER_R_ML = 3,
    XR_FACIAL_BLEND_SHAPE_CHIN_RAISER_ML = 4,
    XR_FACIAL_BLEND_SHAPE_DIMPLER_L_ML = 5,
    XR_FACIAL_BLEND_SHAPE_DIMPLER_R_ML = 6,
    XR_FACIAL_BLEND_SHAPE_EYES_CLOSED_L_ML = 7,
    XR_FACIAL_BLEND_SHAPE_EYES_CLOSED_R_ML = 8,
    XR_FACIAL_BLEND_SHAPE_INNER_BROW_RAISER_L_ML = 9,
    XR_FACIAL_BLEND_SHAPE_INNER_BROW_RAISER_R_ML = 10,
    XR_FACIAL_BLEND_SHAPE_JAW_DROP_ML = 11,
    XR_FACIAL_BLEND_SHAPE_LID_TIGHTENER_L_ML = 12,
    XR_FACIAL_BLEND_SHAPE_LID_TIGHTENER_R_ML = 13,
    XR_FACIAL_BLEND_SHAPE_LIP_CORNER_DEPRESSOR_L_ML = 14,
    XR_FACIAL_BLEND_SHAPE_LIP_CORNER_DEPRESSOR_R_ML = 15,
    XR_FACIAL_BLEND_SHAPE_LIP_CORNER_PULLER_L_ML = 16,
    XR_FACIAL_BLEND_SHAPE_LIP_CORNER_PULLER_R_ML = 17,
    XR_FACIAL_BLEND_SHAPE_LIP_FUNNELER_LB_ML = 18,
    XR_FACIAL_BLEND_SHAPE_LIP_FUNNELER_LT_ML = 19,
    XR_FACIAL_BLEND_SHAPE_LIP_FUNNELER_RB_ML = 20,
    XR_FACIAL_BLEND_SHAPE_LIP_FUNNELER_RT_ML = 21,
    XR_FACIAL_BLEND_SHAPE_LIP_PRESSOR_L_ML = 22,
    XR_FACIAL_BLEND_SHAPE_LIP_PRESSOR_R_ML = 23,
    XR_FACIAL_BLEND_SHAPE_LIP_PUCKER_L_ML = 24,
    XR_FACIAL_BLEND_SHAPE_LIP_PUCKER_R_ML = 25,
    XR_FACIAL_BLEND_SHAPE_LIP_STRETCHER_L_ML = 26,
    XR_FACIAL_BLEND_SHAPE_LIP_STRETCHER_R_ML = 27,
    XR_FACIAL_BLEND_SHAPE_LIP_SUCK_LB_ML = 28,
    XR_FACIAL_BLEND_SHAPE_LIP_SUCK_LT_ML = 29,
```

```
    XR_FACIAL_BLEND_SHAPE_LIP_SUCK_RB_ML = 30,
    XR_FACIAL_BLEND_SHAPE_LIP_SUCK_RT_ML = 31,
    XR_FACIAL_BLEND_SHAPE_LIP_TIGHTENER_L_ML = 32,
    XR_FACIAL_BLEND_SHAPE_LIP_TIGHTENER_R_ML = 33,
    XR_FACIAL_BLEND_SHAPE_LIPS_TOWARD_ML = 34,
    XR_FACIAL_BLEND_SHAPE_LOWER_LIP_DEPRESSOR_L_ML = 35,
    XR_FACIAL_BLEND_SHAPE_LOWER_LIP_DEPRESSOR_R_ML = 36,
    XR_FACIAL_BLEND_SHAPE_NOSE_WRINKLER_L_ML = 37,
    XR_FACIAL_BLEND_SHAPE_NOSE_WRINKLER_R_ML = 38,
    XR_FACIAL_BLEND_SHAPE_OUTER_BROW_RAISER_L_ML = 39,
    XR_FACIAL_BLEND_SHAPE_OUTER_BROW_RAISER_R_ML = 40,
    XR_FACIAL_BLEND_SHAPE_UPPER_LID_RAISER_L_ML = 41,
    XR_FACIAL_BLEND_SHAPE_UPPER_LID_RAISER_R_ML = 42,
    XR_FACIAL_BLEND_SHAPE_UPPER_LIP_RAISER_L_ML = 43,
    XR_FACIAL_BLEND_SHAPE_UPPER_LIP_RAISER_R_ML = 44,
    XR_FACIAL_BLEND_SHAPE_TONGUE_OUT_ML = 45,
    XR_FACIAL_BLEND_SHAPE_MAX_ENUM_ML = 0x7FFFFFFF
} XrFacialBlendShapeML;
```

The XrFacialExpressionBlendShapePropertiesML::`flags` member is of the following type, and contains a bitwise-OR of zero or more bits defined in XrFacialExpressionBlendShapePropertiesFlagBitsML.

```
// Provided by XR_ML_facial_expression
typedef XrFlags64 XrFacialExpressionBlendShapePropertiesFlagsML;
```

Valid bits for XrFacialExpressionBlendShapePropertiesFlagsML are defined by XrFacialExpressionBlendShapePropertiesFlagBitsML and are specified as:

```
// Provided by XR_ML_facial_expression
// Flag bits for XrFacialExpressionBlendShapePropertiesFlagsML
static const XrFacialExpressionBlendShapePropertiesFlagsML
XR_FACIAL_EXPRESSION_BLEND_SHAPE_PROPERTIES_VALID_BIT_ML = 0x00000001;
static const XrFacialExpressionBlendShapePropertiesFlagsML
XR_FACIAL_EXPRESSION_BLEND_SHAPE_PROPERTIES_TRACKED_BIT_ML = 0x00000002;
```

The flag bits' meaning are described as below:

Flag Descriptions

- XR_FACIAL_EXPRESSION_BLEND_SHAPE_PROPERTIES_VALID_BIT_ML — Indicates whether the blend shape is valid.
- XR_FACIAL_EXPRESSION_BLEND_SHAPE_PROPERTIES_TRACKED_BIT_ML — Indicates whether the blend shape is being tracked by the runtime.

12.176.7. Example code for obtaining facial expression information

The following example code demonstrates how to obtain weights for facial expression blend shapes.

```
XrInstance instance; // previously initialized
XrSystemId systemId; // previously initialized
XrSession session;   // previously initialized

// Confirm face tracking system support.
XrSystemFacialExpressionPropertiesML systemFacialExpressionClientProperties {
    .type = XR_TYPE_SYSTEM_FACIAL_EXPRESSION_PROPERTIES_ML,
};
XrSystemProperties systemProperties {
    .type = XR_TYPE_SYSTEM_PROPERTIES,
    .next = &systemFacialExpressionClientProperties,
};
CHK_XR(xrGetSystemProperties(instance, systemId, &systemProperties));
if (!systemFacialExpressionClientProperties.supportsFacialExpression) {
    // The system does not support face tracking
    return;
}

// Get function pointer for xrCreateFacialExpressionClientML.
PFN_xrCreateFacialExpressionClientML pfnCreateFacialExpressionClientML = nullptr;
CHK_XR(xrGetInstanceProcAddr(instance, "xrCreateFacialExpressionClientML",
                             reinterpret_cast<PFN_xrVoidFunction*>(
                             &pfnCreateFacialExpressionClientML)));

// Create a client that queries for default set of facial expressions.
XrFacialExpressionClientML facialExpressionClient = {};
  XrFacialExpressionClientCreateInfoML createInfo {
      .type = XR_TYPE_FACIAL_EXPRESSION_CLIENT_CREATE_INFO_ML,
  };
  CHK_XR(pfnCreateFacialExpressionClientML(session, &createInfo, &
facialExpressionClient));

// Allocate buffers to receive facial expression data before frame
// loop starts.
```

```
const uint32_t num_blend_shapes = 2;
XrFacialExpressionBlendShapePropertiesML blendShapes[num_blend_shapes];
// User must explicitly request what facial expression blend shape to query
blendShapes[0].requestedFacialBlendShape = XR_FACIAL_BLEND_SHAPE_BROW_LOWERER_L_ML;
blendShapes[1].requestedFacialBlendShape = XR_FACIAL_BLEND_SHAPE_BROW_LOWERER_R_ML;

// Get function pointer for xrGetFacialExpressionBlendShapePropertiesML.
PFN_xrGetFacialExpressionBlendShapePropertiesML pfnGetFacialExpressionBlendShapesML =
nullptr;
CHK_XR(xrGetInstanceProcAddr(instance, "xrGetFacialExpressionBlendShapePropertiesML",
                             reinterpret_cast<PFN_xrVoidFunction*>(
                             &pfnGetFacialExpressionBlendShapesML)));
while (1) {
    // ...
    // For every frame in the frame loop
    // ...
    XrFrameState frameState; // previously returned from xrWaitFrame
    const XrTime time = frameState.predictedDisplayTime;

    XrFacialExpressionBlendShapeGetInfoML expressionInfo {
        .type = XR_TYPE_FACIAL_EXPRESSION_BLEND_SHAPE_GET_INFO_ML,
    };

    CHK_XR(pfnGetFacialExpressionBlendShapesML(facialExpressionClient, &expressionInfo,
num_blend_shapes, blendShapes));

    for (uint32_t i = 0; i < num_blend_shapes; ++i) {
        // blendShapes[i] contains the properties of specific blend shape
    }
}
```

12.176.8. New Object Types

- XrFacialExpressionClientML

12.176.9. New Commands

- xrCreateFacialExpressionClientML
- xrDestroyFacialExpressionClientML
- xrGetFacialExpressionBlendShapePropertiesML

12.176.10. New Structures

- XrFacialExpressionBlendShapeGetInfoML
- XrFacialExpressionBlendShapePropertiesML

- XrFacialExpressionClientCreateInfoML
- Extending XrSystemProperties:
  - XrSystemFacialExpressionPropertiesML

12.176.11. New Enums

- XrFacialBlendShapeML
- XrFacialExpressionBlendShapePropertiesFlagBitsML

12.176.12. New Bitmasks

- XrFacialExpressionBlendShapePropertiesFlagsML

12.176.13. New Enum Constants

- `XR_ML_FACIAL_EXPRESSION_EXTENSION_NAME`
- `XR_ML_facial_expression_SPEC_VERSION`
- Extending XrObjectType:
  - `XR_OBJECT_TYPE_FACIAL_EXPRESSION_CLIENT_ML`
- Extending XrResult:
  - `XR_ERROR_FACIAL_EXPRESSION_PERMISSION_DENIED_ML`
- Extending XrStructureType:
  - `XR_TYPE_FACIAL_EXPRESSION_BLEND_SHAPE_GET_INFO_ML`
  - `XR_TYPE_FACIAL_EXPRESSION_BLEND_SHAPE_PROPERTIES_ML`
  - `XR_TYPE_FACIAL_EXPRESSION_CLIENT_CREATE_INFO_ML`
  - `XR_TYPE_SYSTEM_FACIAL_EXPRESSION_PROPERTIES_ML`

12.176.14. Issues

12.176.15. Version History

- Revision 1, 2023-11-22 (Johannes Fung)
  - Initial extension description

12.177. XR_ML_frame_end_info

Name String

`XR_ML_frame_end_info`

Extension Type

Instance extension

Registered Extension Number

136

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Last Modified Date

2022-10-26

Contributors

Ron Bessems, Magic Leap

Overview

This extension provides access to Magic Leap specific extensions to frame settings like focus distance, vignette, and protection.

New Flag Types

The XrFrameEndInfoML::`flags` member is of the following type, and contains a bitwise-OR of zero or more of the bits defined in XrFrameEndInfoFlagBitsML.

```
// Provided by XR_ML_frame_end_info
typedef XrFlags64 XrFrameEndInfoFlagsML;
```

Valid bits for XrFrameEndInfoFlagsML are defined by XrFrameEndInfoFlagBitsML, which is specified as:

```
// Provided by XR_ML_frame_end_info
// Flag bits for XrFrameEndInfoFlagsML
static const XrFrameEndInfoFlagsML XR_FRAME_END_INFO_PROTECTED_BIT_ML = 0x00000001;
static const XrFrameEndInfoFlagsML XR_FRAME_END_INFO_VIGNETTE_BIT_ML = 0x00000002;
```

The flag bits have the following meanings:

Flag Descriptions

- `XR_FRAME_END_INFO_PROTECTED_BIT_ML` — Indicates that the content for this frame is protected and should not be recorded or captured outside the graphics system.
- `XR_FRAME_END_INFO_VIGNETTE_BIT_ML` — Indicates that a soft fade to transparent should be added to the frame in the compositor to blend any hard edges at the FOV limits.

New Enum Constants

XrStructureType enumeration is extended with:

- `XR_TYPE_FRAME_END_INFO_ML`

New Structures

The XrFrameEndInfoML structure is defined as:

```
// Provided by XR_ML_frame_end_info
typedef struct XrFrameEndInfoML {
    XrStructureType          type;
    const void*              next;
    float                    focusDistance;
    XrFrameEndInfoFlagsML    flags;
} XrFrameEndInfoML;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `focusDistance` is the distance, in meters, to defined focus point for the client content. The focus distance is interpreted as the positive distance to the client-determined object of interest (relative to the forward vector of the Lightwear).
- `flags` is a bitmask of XrFrameEndInfoFlagsML

Valid Usage (Implicit)

- The `XR_ML_frame_end_info` extension **must** be enabled prior to using XrFrameEndInfoML
- `type` **must** be `XR_TYPE_FRAME_END_INFO_ML`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `flags` **must** be `0` or a valid combination of XrFrameEndInfoFlagBitsML values

Version History

- Revision 1, 2022-10-26 (Ron Bessems)
  - Initial extension description

12.178. XR_ML_global_dimmer

Name String

`XR_ML_global_dimmer`

Extension Type

Instance extension

Registered Extension Number

137

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Last Modified Date

2022-10-25

Contributors

Ron Bessems, Magic Leap
Michał Kulągowski, Magic Leap

Overview

This extension provides control over the global dimmer panel of the Magic Leap 2. The Global

Dimming™ feature dims the entire display without dimming digital content to make text and images more solid and precise.

Note that when using the XR_ENVIRONMENT_BLEND_MODE_ALPHA_BLEND mode the alpha channel of the color swapchain image is combined with the global dimmer value. The global dimmer however is able to address the whole panel whereas the alpha channel covers the video addressable portion.

New Flag Types

The XrGlobalDimmerFrameEndInfoML::flags member is of the following type, and contains a bitwise-OR of zero or more of the bits defined in XrFrameEndInfoFlagBitsML.

```
// Provided by XR_ML_global_dimmer
typedef XrFlags64 XrGlobalDimmerFrameEndInfoFlagsML;
```

Valid bits for XrGlobalDimmerFrameEndInfoFlagsML are defined by XrGlobalDimmerFrameEndInfoFlagBitsML, which is specified as:

```
// Provided by XR_ML_global_dimmer
// Flag bits for XrGlobalDimmerFrameEndInfoFlagsML
static const XrGlobalDimmerFrameEndInfoFlagsML
XR_GLOBAL_DIMMER_FRAME_END_INFO_ENABLED_BIT_ML = 0x00000001;
```

The flag bits have the following meanings:

Flag Descriptions

- XR_GLOBAL_DIMMER_FRAME_END_INFO_ENABLED_BIT_ML — Indicates that the global dimmer **should** be enabled and controlled by XrGlobalDimmerFrameEndInfoML::dimmerValue.

New Enum Constants

XrStructureType enumeration is extended with:

- XR_TYPE_GLOBAL_DIMMER_FRAME_END_INFO_ML

New Structures

The XrGlobalDimmerFrameEndInfoML structure is defined as:

```
// Provided by XR_ML_global_dimmer
typedef struct XrGlobalDimmerFrameEndInfoML {
    XrStructureType                       type;
    const void*                           next;
    float                                 dimmerValue;
    XrGlobalDimmerFrameEndInfoFlagsML     flags;
} XrGlobalDimmerFrameEndInfoML;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `dimmerValue` is a value between 0.0 (transparent) and 1.0 (opaque). The runtime **may** adjust the dimmerValue used during composition at the runtime's discretion. This **may** be done for user safety, display performance, or other reasons. Values outside of the range are silently clamped.
- `flags` is a bitmask of XrGlobalDimmerFrameEndInfoFlagsML

Valid Usage (Implicit)

- The `XR_ML_global_dimmer` extension **must** be enabled prior to using XrGlobalDimmerFrameEndInfoML
- `type` **must** be `XR_TYPE_GLOBAL_DIMMER_FRAME_END_INFO_ML`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `flags` **must** be `0` or a valid combination of XrGlobalDimmerFrameEndInfoFlagBitsML values

Version History

- Revision 1, 2022-10-25 (Ron Bessems)
  - Initial extension description

12.179. XR_ML_localization_map

Name String

`XR_ML_localization_map`

Extension Type

Instance extension

Registered Extension Number

140

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0
and
`XR_EXT_uuid`

Last Modified Date

2023-09-14

Contributors

Ron Bessems, Magic Leap
Karthik Kadappan, Magic Leap

12.179.1. Overview

A Magic Leap localization map is a container that holds metadata about the scanned environment. It is a digital copy of a physical place. A localization map holds spatial anchors, dense mesh, planes, feature points, and positional data.

- Spatial anchors - Used for persistent placement of content.
- Dense mesh - 3D triangulated geometry representing Magic Leap device understanding of the real-world geometry of an area.
- Planes - Large, flat surfaces derived from dense mesh data.

Localization maps **can** be created on device or in the Magic Leap AR Cloud. There are two types - "On Device" and "Cloud".

- "On Device" for OpenXR (local space for MagicLeap) - are for a single device and **can** be shared via the export/import mechanism.
- "Cloud" for OpenXR (shared space for MagicLeap) - **can** be shared across multiple MagicLeap devices in the AR Cloud.



Note

Localization Maps are called Spaces in the Magic Leap C-API.

Permissions

Android applications **must** have the com.magicleap.permission.SPACE_MANAGER permission listed in their manifest to use these functions:

- xrQueryLocalizationMapsML
- xrRequestMapLocalizationML

(protection level: normal)

Android applications **must** have the com.magicleap.permission.SPACE_IMPORT_EXPORT permission listed in their manifest and granted to use these functions:

- xrImportLocalizationMapML
- xrCreateExportedLocalizationMapML

(protection level: dangerous)

12.179.2. Current Localization Map Information

Applications **can** receive notifications when the current localization map changes by calling xrPollEvent and handling the XrEventDataLocalizationChangedML type. To enable these events call xrEnableLocalizationEventsML.

The XrEventDataLocalizationChangedML structure is defined as:

```
// Provided by XR_ML_localization_map
typedef struct XrEventDataLocalizationChangedML {
     XrStructureType                 type;
    const void*                      next;
    XrSession                        session;
    XrLocalizationMapStateML         state;
    XrLocalizationMapML              map;
    XrLocalizationMapConfidenceML    confidence;
    XrLocalizationMapErrorFlagsML    errorFlags;
} XrEventDataLocalizationChangedML;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `session` is the session to which this change event applies.
- `state` is the current XrLocalizationMapStateML of the map.
- `map` is the XrLocalizationMapML of the current map.
- `confidence` is the XrLocalizationMapConfidenceML of the current map.
- `errorFlags` is a a bitwise-OR of zero or more of the bits defined in XrLocalizationMapErrorFlagBitsML in the case that the localization map has low confidence.

By default the runtime does not send these events but calling xrEnableLocalizationEventsML function enables the events. When this function is called the XrEventDataLocalizationChangedML event will always be posted to the event queue, regardless of whether the map localization state has changed. This allows the application to synchronize with the current state.



Note

The arrival of the event is asynchronous to this call.

Valid Usage (Implicit)

- The `XR_ML_localization_map` extension **must** be enabled prior to using XrEventDataLocalizationChangedML
- `type` **must** be `XR_TYPE_EVENT_DATA_LOCALIZATION_CHANGED_ML`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

The bitmask type XrLocalizationMapErrorFlagsML is defined as:

```
// Provided by XR_ML_localization_map
typedef XrFlags64 XrLocalizationMapErrorFlagsML;
```

As used in XrEventDataLocalizationChangedML::`errorFlags` field, XrLocalizationMapErrorFlagsML contains a bitwise-OR of zero or more of the bits defined in XrLocalizationMapErrorFlagBitsML.

```
// Provided by XR_ML_localization_map
// Flag bits for XrLocalizationMapErrorFlagsML
static const XrLocalizationMapErrorFlagsML XR_LOCALIZATION_MAP_ERROR_UNKNOWN_BIT_ML =
0x00000001;
static const XrLocalizationMapErrorFlagsML
XR_LOCALIZATION_MAP_ERROR_OUT_OF_MAPPED_AREA_BIT_ML = 0x00000002;
static const XrLocalizationMapErrorFlagsML
XR_LOCALIZATION_MAP_ERROR_LOW_FEATURE_COUNT_BIT_ML = 0x00000004;
static const XrLocalizationMapErrorFlagsML
XR_LOCALIZATION_MAP_ERROR_EXCESSIVE_MOTION_BIT_ML = 0x00000008;
static const XrLocalizationMapErrorFlagsML XR_LOCALIZATION_MAP_ERROR_LOW_LIGHT_BIT_ML =
0x00000010;
static const XrLocalizationMapErrorFlagsML XR_LOCALIZATION_MAP_ERROR_HEADPOSE_BIT_ML =
0x00000020;
```

The flag bits have the following meanings:

Flag Descriptions

- XR_LOCALIZATION_MAP_ERROR_UNKNOWN_BIT_ML — Localization failed for an unknown reason.
- XR_LOCALIZATION_MAP_ERROR_OUT_OF_MAPPED_AREA_BIT_ML — Localization failed because the user is outside of the mapped area.
- XR_LOCALIZATION_MAP_ERROR_LOW_FEATURE_COUNT_BIT_ML — There are not enough features in the environment to successfully localize.
- XR_LOCALIZATION_MAP_ERROR_EXCESSIVE_MOTION_BIT_ML — Localization failed due to excessive motion.
- XR_LOCALIZATION_MAP_ERROR_LOW_LIGHT_BIT_ML — Localization failed because the lighting levels are too low in the environment.
- XR_LOCALIZATION_MAP_ERROR_HEADPOSE_BIT_ML — A headpose failure caused localization to be unsuccessful.

The xrEnableLocalizationEventsML function is defined as:

```
// Provided by XR_ML_localization_map
XrResult xrEnableLocalizationEventsML(
    XrSession                                   session,
    const XrLocalizationEnableEventsInfoML *    info);
```

Parameter Descriptions

- `session` is a handle to an XrSession previously created with xrCreateSession.
- `info` is a pointer to an XrLocalizationEnableEventsInfoML structure.

Valid Usage (Implicit)

- The `XR_ML_localization_map` extension **must** be enabled prior to calling xrEnableLocalizationEventsML
- `session` **must** be a valid XrSession handle
- `info` **must** be a pointer to a valid XrLocalizationEnableEventsInfoML structure

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_LOCALIZATION_MAP_PERMISSION_DENIED_ML`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_VALIDATION_FAILURE`

The XrLocalizationEnableEventsInfoML structure is defined as:

```
// Provided by XR_ML_localization_map
typedef struct XrLocalizationEnableEventsInfoML {
    XrStructureType    type;
    const void*        next;
    XrBool32           enabled;
} XrLocalizationEnableEventsInfoML;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `enabled` is the flag to enable/disable localization status events.

Valid Usage (Implicit)

- The `XR_ML_localization_map` extension **must** be enabled prior to using XrLocalizationEnableEventsInfoML
- `type` **must** be `XR_TYPE_LOCALIZATION_ENABLE_EVENTS_INFO_ML`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

The XrLocalizationMapML structure is defined as:

```
// Provided by XR_ML_localization_map
typedef struct XrLocalizationMapML {
    XrStructureType            type;
    void*                      next;
    char                       name[XR_MAX_LOCALIZATION_MAP_NAME_LENGTH_ML];
    XrUuidEXT                  mapUuid;
    XrLocalizationMapTypeML    mapType;
} XrLocalizationMapML;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `name` is a human readable name of the localization map, as a null terminated UTF-8 string. This name is set outside of this extension.
- `mapUuid` is the XrUuidEXT of the localization map.
- `mapType` is the XrLocalizationMapTypeML of the map.

Valid Usage (Implicit)

- The XR_ML_localization_map extension **must** be enabled prior to using XrLocalizationMapML
- type **must** be XR_TYPE_LOCALIZATION_MAP_ML
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- name **must** be a null-terminated UTF-8 string whose length is less than or equal to XR_MAX_LOCALIZATION_MAP_NAME_LENGTH_ML
- If mapType is not 0, mapType **must** be a valid XrLocalizationMapTypeML value

12.179.3. Listing Localization Maps

Localization maps available to the application **can** be queried using xrQueryLocalizationMapsML.

The xrQueryLocalizationMapsML function is defined as:

```
// Provided by XR_ML_localization_map
XrResult xrQueryLocalizationMapsML(
    XrSession                                   session,
    const XrLocalizationMapQueryInfoBaseHeaderML* queryInfo,
    uint32_t                                    mapCapacityInput,
    uint32_t *                                  mapCountOutput,
    XrLocalizationMapML*                        maps);
```

Parameter Descriptions

- session is a handle to an XrSession previously created with xrCreateSession.
- queryInfo is an **optional** enumeration filter based on XrLocalizationMapQueryInfoBaseHeaderML to use.
- mapCapacityInput is the capacity of the maps array, or 0 to indicate a request to retrieve the required capacity.
- mapCountOutput is filled in by the runtime with the count of maps written or the required capacity in the case that mapCapacityInput is insufficient.
- maps is an array of XrLocalizationMapML filled in by the runtime, but **can** be NULL if mapCapacityInput is 0.
- See the Buffer Size Parameters section for a detailed description of retrieving the required maps size.

The list of localization maps returned will depend on the current device mapping mode. Only the localization maps associated with the current mapping mode will be returned by this call. Device

mapping mode (e.g. XR_LOCALIZATION_MAP_TYPE_ON_DEVICE_ML or XR_LOCALIZATION_MAP_TYPE_CLOUD_ML) **can** only be changed via the system application(s).

The list of maps known to the runtime **may** change between the two calls to xrQueryLocalizationMapsML. This is however a rare occurrence and the application **may** retry the call again if it receives XR_ERROR_SIZE_INSUFFICIENT.

Valid Usage (Implicit)

- The XR_ML_localization_map extension **must** be enabled prior to calling xrQueryLocalizationMapsML
- session **must** be a valid XrSession handle
- If queryInfo is not NULL, queryInfo **must** be a pointer to a valid XrLocalizationMapQueryInfoBaseHeaderML-based structure
- mapCountOutput **must** be a pointer to a uint32_t value
- If mapCapacityInput is not 0, maps **must** be a pointer to an array of mapCapacityInput XrLocalizationMapML structures

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LOCALIZATION_MAP_PERMISSION_DENIED_ML
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The XrLocalizationMapQueryInfoBaseHeaderML structure is defined as:

```
// Provided by XR_ML_localization_map
typedef struct XrLocalizationMapQueryInfoBaseHeaderML {
    XrStructureType    type;
    const void*        next;
} XrLocalizationMapQueryInfoBaseHeaderML;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.

Currently no filters are available.

Valid Usage (Implicit)

- The `XR_ML_localization_map` extension **must** be enabled prior to using XrLocalizationMapQueryInfoBaseHeaderML
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

12.179.4. Request Localization Map

Applications **can** change the current map by calling xrRequestMapLocalizationML.

The xrRequestMapLocalizationML function is defined as:

```
// Provided by XR_ML_localization_map
XrResult xrRequestMapLocalizationML(
    XrSession                                   session,
    const XrMapLocalizationRequestInfoML*       requestInfo);
```

Parameter Descriptions

- `session` is a handle to an XrSession previously created with xrCreateSession.
- `requestInfo` contains XrMapLocalizationRequestInfoML on the localization map to request.

This is an asynchronous request. Listen for XrEventDataLocalizationChangedML events to get the results of the localization. A new request for localization will override all the past requests for localization that are yet to be completed.

The runtime **must** return XR_ERROR_LOCALIZATION_MAP_UNAVAILABLE_ML if the requested is not a map known to the runtime.

Valid Usage (Implicit)

- The XR_ML_localization_map extension **must** be enabled prior to calling xrRequestMapLocalizationML
- session **must** be a valid XrSession handle
- requestInfo **must** be a pointer to a valid XrMapLocalizationRequestInfoML structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LOCALIZATION_MAP_FAIL_ML
- XR_ERROR_LOCALIZATION_MAP_PERMISSION_DENIED_ML
- XR_ERROR_LOCALIZATION_MAP_UNAVAILABLE_ML
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The XrMapLocalizationRequestInfoML structure is defined as:

```
// Provided by XR_ML_localization_map
typedef struct XrMapLocalizationRequestInfoML {
    XrStructureType    type;
    const void*        next;
    XrUuidEXT          mapUuid;
} XrMapLocalizationRequestInfoML;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `mapUuid` is the XrUuidEXT of the localization map to request. This mapUuid **can** be obtained via xrQueryLocalizationMapsML.

Valid Usage (Implicit)

- The `XR_ML_localization_map` extension **must** be enabled prior to using XrMapLocalizationRequestInfoML
- `type` **must** be `XR_TYPE_MAP_LOCALIZATION_REQUEST_INFO_ML`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

12.179.5. Import and Exporting

This API supports exporting and importing of device localization maps. The runtime **must** not export AR Cloud maps and **must** return `XR_ERROR_LOCALIZATION_MAP_CANNOT_EXPORT_CLOUD_MAP_ML` if the application attempts to do so.

The format of the exported localization map data **can** change with OS version updates.

- Backwards compatibility: exports using OS version n **should** work on OS versions up to and including OS version n-4.
- Forwards compatibility: exports using OS version n is not guaranteed to work on OS versions > n.

Developers are strongly encouraged to encrypt the exported localization maps.

The xrImportLocalizationMapML function is defined as:

```
// Provided by XR_ML_localization_map
XrResult xrImportLocalizationMapML(
    XrSession                                   session,
    const XrLocalizationMapImportInfoML*        importInfo,
    XrUuidEXT*                                  mapUuid);
```

Parameter Descriptions

- `session` is a handle to an XrSession previously created with xrCreateSession.
- `importInfo` contains XrLocalizationMapImportInfoML on the localization map to import.
- `mapUuid` is the XrUuidEXT of the newly imported localization map filled in by the runtime.

The runtime **must** return `XR_ERROR_LOCALIZATION_MAP_ALREADY_EXISTS_ML` if the map that is being imported already exists. The runtime **must** return `XR_ERROR_LOCALIZATION_MAP_INCOMPATIBLE_ML` if the map being imported is not compatible.

xrImportLocalizationMapML **may** take a long time to complete; as such applications **should** not call this from the frame loop.

Valid Usage (Implicit)

- The `XR_ML_localization_map` extension **must** be enabled prior to calling xrImportLocalizationMapML
- `session` **must** be a valid XrSession handle
- `importInfo` **must** be a pointer to a valid XrLocalizationMapImportInfoML structure
- If `mapUuid` is not `NULL`, `mapUuid` **must** be a pointer to an XrUuidEXT structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LOCALIZATION_MAP_ALREADY_EXISTS_ML
- XR_ERROR_LOCALIZATION_MAP_IMPORT_EXPORT_PERMISSION_DENIED_ML
- XR_ERROR_LOCALIZATION_MAP_INCOMPATIBLE_ML
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The XrLocalizationMapImportInfoML structure is defined as:

```
// Provided by XR_ML_localization_map
typedef struct XrLocalizationMapImportInfoML {
    XrStructureType    type;
    const void*        next;
    uint32_t           size;
    char*              data;
} XrLocalizationMapImportInfoML;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain.
- size is the size in bytes of the data member.
- data is the byte data of the previously exported localization map.

Valid Usage (Implicit)

- The XR_ML_localization_map extension **must** be enabled prior to using XrLocalizationMapImportInfoML
- type **must** be XR_TYPE_LOCALIZATION_MAP_IMPORT_INFO_ML
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- data **must** be a pointer to an array of size char values
- The size parameter **must** be greater than 0

Exporting

The xrCreateExportedLocalizationMapML function is defined as:

```
// Provided by XR_ML_localization_map
XrResult xrCreateExportedLocalizationMapML(
    XrSession                                   session,
    const XrUuidEXT*                            mapUuid,
    XrExportedLocalizationMapML*                map);
```

Parameter Descriptions

- session is a handle to an XrSession previously created with xrCreateSession.
- mapUuid is a pointer to the uuid of the map to export.
- map is a pointer to a map handle filled in by the runtime.

xrCreateExportedLocalizationMapML creates a frozen copy of the mapUuid localization map that **can** be exported using xrGetExportedLocalizationMapDataML. Applications **should** call xrDestroyExportedLocalizationMapML once they are done with the data.

Valid Usage (Implicit)

- The XR_ML_localization_map extension **must** be enabled prior to calling xrCreateExportedLocalizationMapML
- session **must** be a valid XrSession handle
- mapUuid **must** be a pointer to a valid XrUuidEXT structure
- map **must** be a pointer to an XrExportedLocalizationMapML handle

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_LOCALIZATION_MAP_CANNOT_EXPORT_CLOUD_MAP_ML
- XR_ERROR_LOCALIZATION_MAP_IMPORT_EXPORT_PERMISSION_DENIED_ML
- XR_ERROR_LOCALIZATION_MAP_UNAVAILABLE_ML
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The xrDestroyExportedLocalizationMapML function is defined as:

```
// Provided by XR_ML_localization_map
XrResult xrDestroyExportedLocalizationMapML(
    XrExportedLocalizationMapML                 map);
```

Parameter Descriptions

- map is the map to destroy.

Valid Usage (Implicit)

- The XR_ML_localization_map extension **must** be enabled prior to calling xrDestroyExportedLocalizationMapML
- map **must** be a valid XrExportedLocalizationMapML handle

Thread Safety

- Access to `map`, and any child handles, **must** be externally synchronized

Return Codes

Success

- `XR_SUCCESS`

Failure

- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_RUNTIME_FAILURE`

The xrGetExportedLocalizationMapDataML function is defined as:

```
// Provided by XR_ML_localization_map
XrResult xrGetExportedLocalizationMapDataML(
    XrExportedLocalizationMapML                 map,
    uint32_t                                    bufferCapacityInput,
    uint32_t*                                   bufferCountOutput,
    char*                                       buffer);
```

Parameter Descriptions

- `map` is the map to export.
- `bufferCapacityInput` is the capacity of the buffer array, or 0 to indicate a request to retrieve the required capacity.
- `bufferCountOutput` is filled in by the runtime with the count of bytes written or the required capacity in the case that bufferCapacityInput is insufficient.
- `buffer` is an array of bytes filled in by the runtime.

xrGetExportedLocalizationMapDataML **may** take a long time to complete; as such applications **should** not call this from the frame loop.

Valid Usage (Implicit)

- The `XR_ML_localization_map` extension **must** be enabled prior to calling xrGetExportedLocalizationMapDataML
- `map` **must** be a valid XrExportedLocalizationMapML handle
- `bufferCountOutput` **must** be a pointer to a `uint32_t` value
- If `bufferCapacityInput` is not `0`, `buffer` **must** be a pointer to an array of `bufferCapacityInput` char values

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_VALIDATION_FAILURE`

12.179.6. Reference Space

Applications localized into the same localization map **can** use this reference space to place virtual content in the same physical location.

`XR_REFERENCE_SPACE_TYPE_LOCALIZATION_MAP_ML` is the reference space of the current localization map. Creating a space is done via xrCreateReferenceSpace.

The runtime **must** emit the XrEventDataReferenceSpaceChangePending event if the reference space is changing due to a localization map change.

The runtime **may** move the physical location of the origin of this space as it updates its understanding of the physical space to maintain consistency without sending the XrEventDataReferenceSpaceChangePending event.

For a given XrUuidEXT the runtime **must** keep the position and orientation of this space identical across more than one XrInstance, including for different users and different hardware.

The runtime **must** create this reference space as gravity-aligned to exclude pitch and roll, with +Y up.

12.179.7. Example code

The following code shows how to list the currently available localization maps.

```
uint32_t mapCount = 0;
CHK_XR(xrQueryLocalizationMapsML(session, nullptr, 0, &mapCount, nullptr));

std::vector<XrLocalizationMapML> maps(mapCount, {
    .type = XR_TYPE_LOCALIZATION_MAP_ML,
});

CHK_XR(xrQueryLocalizationMapsML(session, nullptr, static_cast<uint32_t>(maps.size()),
&mapCount, maps.data()));
```

This code shows how to poll for localization events.

```
XrEventDataBuffer event {
    .type = XR_TYPE_EVENT_DATA_BUFFER,
};
XrResult result = xrPollEvent(instance, &event);
if (result == XR_SUCCESS) {
    switch (event.type) {
        case XR_TYPE_EVENT_DATA_LOCALIZATION_CHANGED_ML: {
            const auto& localization_event =
              *reinterpret_cast<XrEventDataLocalizationChangedML*>(&event);
            // Use the data in localization_event.
            break;
        }
        // Handle other events as well as usual.
    }
}
```

12.179.8. Constants

New Object Types

```
// Provided by XR_ML_localization_map
XR_DEFINE_HANDLE(XrExportedLocalizationMapML)
```

XrExportedLocalizationMapML represents a frozen exported localization map.

New Enum Constants

XrStructureType enumeration is extended with:

- XR_TYPE_LOCALIZATION_MAP_ML
- XR_TYPE_EVENT_DATA_LOCALIZATION_CHANGED_ML
- XR_TYPE_MAP_LOCALIZATION_REQUEST_INFO_ML
- XR_TYPE_LOCALIZATION_MAP_IMPORT_INFO_ML
- XR_TYPE_LOCALIZATION_ENABLE_EVENTS_INFO_ML

XrResult enumeration is extended with:

- XR_ERROR_LOCALIZATION_MAP_INCOMPATIBLE_ML
- XR_ERROR_LOCALIZATION_MAP_UNAVAILABLE_ML
- XR_ERROR_LOCALIZATION_MAP_IMPORT_EXPORT_PERMISSION_DENIED_ML
- XR_ERROR_LOCALIZATION_MAP_PERMISSION_DENIED_ML
- XR_ERROR_LOCALIZATION_MAP_ALREADY_EXISTS_ML
- XR_ERROR_LOCALIZATION_MAP_CANNOT_EXPORT_CLOUD_MAP_ML
- XR_ERROR_LOCALIZATION_MAP_FAIL_ML

New Enums

```
// Provided by XR_ML_localization_map
typedef enum XrLocalizationMapStateML {
    XR_LOCALIZATION_MAP_STATE_NOT_LOCALIZED_ML = 0,
    XR_LOCALIZATION_MAP_STATE_LOCALIZED_ML = 1,
    XR_LOCALIZATION_MAP_STATE_LOCALIZATION_PENDING_ML = 2,
    XR_LOCALIZATION_MAP_STATE_LOCALIZATION_SLEEPING_BEFORE_RETRY_ML = 3,
    XR_LOCALIZATION_MAP_STATE_MAX_ENUM_ML = 0x7FFFFFFF
} XrLocalizationMapStateML;
```

Enum	Description
XR_LOCALIZATION_MAP_STATE_NOT_LOCALIZED_ML	The system is not localized into a map. Features like Spatial Anchors relying on localization will not work.
XR_LOCALIZATION_MAP_STATE_LOCALIZED_ML	The system is localized into a map.
XR_LOCALIZATION_MAP_STATE_LOCALIZATION_PENDING_ML	The system is localizing into a map.
XR_LOCALIZATION_MAP_STATE_LOCALIZATION_SLEEPING_BEFORE_RETRY_ML	Initial localization failed, the system will retry localization.

```
// Provided by XR_ML_localization_map
typedef enum XrLocalizationMapConfidenceML {
    XR_LOCALIZATION_MAP_CONFIDENCE_POOR_ML = 0,
    XR_LOCALIZATION_MAP_CONFIDENCE_FAIR_ML = 1,
    XR_LOCALIZATION_MAP_CONFIDENCE_GOOD_ML = 2,
    XR_LOCALIZATION_MAP_CONFIDENCE_EXCELLENT_ML = 3,
    XR_LOCALIZATION_MAP_CONFIDENCE_MAX_ENUM_ML = 0x7FFFFFFF
} XrLocalizationMapConfidenceML;
```

Enum	Description
XR_LOCALIZATION_MAP_CONFIDENCE_POOR_ML	The localization map has poor confidence, systems relying on the localization map are likely to have poor performance.
XR_LOCALIZATION_MAP_CONFIDENCE_FAIR_ML	The confidence is fair, current environmental conditions may adversely affect localization.
XR_LOCALIZATION_MAP_CONFIDENCE_GOOD_ML	The confidence is high, persistent content should be stable.
XR_LOCALIZATION_MAP_CONFIDENCE_EXCELLENT_ML	This is a very high-confidence localization, persistent content will be very stable.

```
// Provided by XR_ML_localization_map
typedef enum XrLocalizationMapTypeML {
    XR_LOCALIZATION_MAP_TYPE_ON_DEVICE_ML = 0,
    XR_LOCALIZATION_MAP_TYPE_CLOUD_ML = 1,
    XR_LOCALIZATION_MAP_TYPE_MAX_ENUM_ML = 0x7FFFFFFF
} XrLocalizationMapTypeML;
```

Enum	Description
XR_LOCALIZATION_MAP_TYPE_ON_DEVICE_ML	The system is localized into an On-Device map, published anchors are not shared between different devices.
XR_LOCALIZATION_MAP_TYPE_CLOUD_ML	The system is localized into a Cloud Map, anchors are shared per cloud account settings.

New Enum Constants

XrReferenceSpaceType enumeration is extended with:

- XR_REFERENCE_SPACE_TYPE_LOCALIZATION_MAP_ML

New Defines

Version History

- Revision 1, 2023-06-23 (Ron Bessems)
  - Initial extension description

12.180. XR_ML_marker_understanding

Name String

`XR_ML_marker_understanding`

Extension Type

Instance extension

Registered Extension Number

139

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Last Modified Date

2023-05-18

Contributors

Robbie Bridgewater, Magic Leap
Ron Bessems, Magic Leap
Karthik Kadappan, Magic Leap

12.180.1. Overview

This extension **can** be used to track and query fiducial markers like QR codes, AprilTag markers, and ArUco markers, and detect, but not locate, 1D barcodes like Code 128, UPC-A.



Permissions

Android applications **must** have the `com.magicleap.permission.MARKER_TRACKING` permission listed in their manifest to use this extension. (protection level: normal)

12.180.2. Creating a Marker Detector

```
// Provided by XR_ML_marker_understanding
XR_DEFINE_HANDLE(XrMarkerDetectorML)
```

The XrMarkerDetectorML handle represents the resources for detecting one or more markers.

A marker detector handle detects a single type of marker, specified by a value of XrMarkerTypeML. To detect more than one marker type, a runtime **may** support creating multiple marker detector handles.

This handle **can** be used to detect markers using other functions in this extension.

The xrCreateMarkerDetectorML function is defined as:

```
// Provided by XR_ML_marker_understanding
XrResult xrCreateMarkerDetectorML(
    XrSession                                   session,
    const XrMarkerDetectorCreateInfoML*         createInfo,
    XrMarkerDetectorML*                         markerDetector);
```

Parameter Descriptions

- `session` is an XrSession in which the marker detection will be active.
- `createInfo` is the XrMarkerDetectorCreateInfoML used to specify the marker detection.
- `markerDetector` is the returned XrMarkerDetectorML handle.

An application creates an XrMarkerDetectorML handle using the xrCreateMarkerDetectorML function. If `createInfo` contains mutually exclusive contents, the runtime **must** return `XR_ERROR_MARKER_DETECTOR_INVALID_CREATE_INFO_ML`.

If a runtime is unable to create a marker detector due to some internal limit, the runtime **must** return `XR_ERROR_LIMIT_REACHED`.

Valid Usage (Implicit)

- The `XR_ML_marker_understanding` extension **must** be enabled prior to calling xrCreateMarkerDetectorML
- `session` **must** be a valid XrSession handle
- `createInfo` **must** be a pointer to a valid XrMarkerDetectorCreateInfoML structure
- `markerDetector` **must** be a pointer to an XrMarkerDetectorML handle

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_MARKER_DETECTOR_INVALID_CREATE_INFO_ML
- XR_ERROR_MARKER_DETECTOR_PERMISSION_DENIED_ML
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The XrMarkerDetectorCreateInfoML structure is defined as:

```
// Provided by XR_ML_marker_understanding
typedef struct XrMarkerDetectorCreateInfoML {
    XrStructureType              type;
    const void*                  next;
    XrMarkerDetectorProfileML    profile;
    XrMarkerTypeML               markerType;
} XrMarkerDetectorCreateInfoML;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain.
- profile is the marker tracker profile to be used.
- markerType is the detector type that this tracker enables.

Valid Usage (Implicit)

- The XR_ML_marker_understanding extension **must** be enabled prior to using XrMarkerDetectorCreateInfoML
- type **must** be XR_TYPE_MARKER_DETECTOR_CREATE_INFO_ML
- next **must** be NULL or a valid pointer to the next structure in a structure chain. See also: XrMarkerDetectorAprilTagInfoML, XrMarkerDetectorArucoInfoML, XrMarkerDetectorCustomProfileInfoML, XrMarkerDetectorSizeInfoML
- profile **must** be a valid XrMarkerDetectorProfileML value
- markerType **must** be a valid XrMarkerTypeML value

The possible premade profiles for an XrMarkerDetectorML are specified by the XrMarkerDetectorProfileML enumeration:

```
// Provided by XR_ML_marker_understanding
typedef enum XrMarkerDetectorProfileML {
    XR_MARKER_DETECTOR_PROFILE_DEFAULT_ML = 0,
    XR_MARKER_DETECTOR_PROFILE_SPEED_ML = 1,
    XR_MARKER_DETECTOR_PROFILE_ACCURACY_ML = 2,
    XR_MARKER_DETECTOR_PROFILE_SMALL_TARGETS_ML = 3,
    XR_MARKER_DETECTOR_PROFILE_LARGE_FOV_ML = 4,
    XR_MARKER_DETECTOR_PROFILE_CUSTOM_ML = 5,
    XR_MARKER_DETECTOR_PROFILE_MAX_ENUM_ML = 0x7FFFFFFF
} XrMarkerDetectorProfileML;
```

Enumerant Descriptions

- XR_MARKER_DETECTOR_PROFILE_DEFAULT_ML — Tracker profile that covers standard use cases. If this does not suite the needs of the application try the other profiles listed below.
- XR_MARKER_DETECTOR_PROFILE_SPEED_ML — Optimized for speed. Use this profile to reduce the compute load and increase detection/tracker speed. This can result in low accuracy poses.
- XR_MARKER_DETECTOR_PROFILE_ACCURACY_ML — Optimized for accuracy. Use this profile to optimize for accurate marker poses. This can cause increased load on the compute.
- XR_MARKER_DETECTOR_PROFILE_SMALL_TARGETS_ML — Optimized for small targets. Use this profile to optimize for markers that are small or for larger markers that need to be detected from afar.
- XR_MARKER_DETECTOR_PROFILE_LARGE_FOV_ML — Optimized for FoV. Use this profile to be able to detect markers across a larger FoV. The marker tracker system will attempt to use multiple cameras to detect the markers.
- XR_MARKER_DETECTOR_PROFILE_CUSTOM_ML — Custom Tracker Profile. The application can define a custom tracker profile. See XrMarkerDetectorCustomProfileInfoML for more details.

The type of marker to be tracked is specified via XrMarkerDetectorML:

```
// Provided by XR_ML_marker_understanding
typedef enum XrMarkerTypeML {
    XR_MARKER_TYPE_ARUCO_ML = 0,
    XR_MARKER_TYPE_APRIL_TAG_ML = 1,
    XR_MARKER_TYPE_QR_ML = 2,
    XR_MARKER_TYPE_EAN_13_ML = 3,
    XR_MARKER_TYPE_UPC_A_ML = 4,
    XR_MARKER_TYPE_CODE_128_ML = 5,
    XR_MARKER_TYPE_MAX_ENUM_ML = 0x7FFFFFFF
} XrMarkerTypeML;
```

Enumerant Descriptions

- `XR_MARKER_TYPE_ARUCO_ML` — Aruco Marker detection and localization. The marker id of the Aruco marker is available via xrGetMarkerNumberML.
- `XR_MARKER_TYPE_APRIL_TAG_ML` — AprilTag detection and localization. The marker id of the AprilTags is available via xrGetMarkerNumberML.
- `XR_MARKER_TYPE_QR_ML` — QR code detection and localization. The contents of the QR code is available via xrGetMarkerStringML.
- `XR_MARKER_TYPE_EAN_13_ML` — EAN-13, detection only, not locatable. The contents of the barcode is available via xrGetMarkerStringML.
- `XR_MARKER_TYPE_UPC_A_ML` — UPC-A, detection only, not locatable. The contents of the barcode is available via xrGetMarkerStringML.
- `XR_MARKER_TYPE_CODE_128_ML` — Code 128, detection only, not locatable. The contents of the barcode is available via xrGetMarkerStringML.

An application specifies details of the type of marker to be tracked by chaining an `XrMarkerDetector*InfoML` structure to XrMarkerDetectorCreateInfoML. Some of these structure types **must** be included to enable detection or locating, depending on the marker type.

The following structures are used by the ArUco, AprilTag, and QR code detectors:

Marker Type	Structures
ArUco	XrMarkerDetectorArucoInfoML XrMarkerDetectorSizeInfoML
AprilTag	XrMarkerDetectorAprilTagInfoML XrMarkerDetectorSizeInfoML
QR Code	XrMarkerDetectorSizeInfoML

The XrMarkerDetectorSizeInfoML **may** be optional depending on runtime support for estimating marker size. A higher localization accuracy **may** be obtained by specifying the marker size. If the runtime does not support estimating marker size it **must** return `XR_ERROR_VALIDATION_FAILURE` if XrMarkerDetectorSizeInfoML is omitted.

The XrMarkerDetectorArucoInfoML structure extends XrMarkerDetectorCreateInfoML and is defined as:

```
// Provided by XR_ML_marker_understanding
typedef struct XrMarkerDetectorArucoInfoML {
    XrStructureType        type;
    const void*            next;
    XrMarkerArucoDictML    arucoDict;
} XrMarkerDetectorArucoInfoML;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `arucoDict` is the ArUco dictionary name from which markers will be detected.

This structure is required by the `XR_MARKER_TYPE_ARUCO_ML` detector.

Valid Usage (Implicit)

- The `XR_ML_marker_understanding` extension **must** be enabled prior to using XrMarkerDetectorArucoInfoML
- `type` **must** be `XR_TYPE_MARKER_DETECTOR_ARUCO_INFO_ML`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `arucoDict` **must** be a valid XrMarkerArucoDictML value

The XrMarkerArucoDictML enumeration is defined as:

```
// Provided by XR_ML_marker_understanding
typedef enum XrMarkerArucoDictML {
    XR_MARKER_ARUCO_DICT_4X4_50_ML = 0,
    XR_MARKER_ARUCO_DICT_4X4_100_ML = 1,
    XR_MARKER_ARUCO_DICT_4X4_250_ML = 2,
    XR_MARKER_ARUCO_DICT_4X4_1000_ML = 3,
    XR_MARKER_ARUCO_DICT_5X5_50_ML = 4,
    XR_MARKER_ARUCO_DICT_5X5_100_ML = 5,
    XR_MARKER_ARUCO_DICT_5X5_250_ML = 6,
    XR_MARKER_ARUCO_DICT_5X5_1000_ML = 7,
    XR_MARKER_ARUCO_DICT_6X6_50_ML = 8,
    XR_MARKER_ARUCO_DICT_6X6_100_ML = 9,
    XR_MARKER_ARUCO_DICT_6X6_250_ML = 10,
    XR_MARKER_ARUCO_DICT_6X6_1000_ML = 11,
    XR_MARKER_ARUCO_DICT_7X7_50_ML = 12,
    XR_MARKER_ARUCO_DICT_7X7_100_ML = 13,
    XR_MARKER_ARUCO_DICT_7X7_250_ML = 14,
    XR_MARKER_ARUCO_DICT_7X7_1000_ML = 15,
    XR_MARKER_ARUCO_DICT_MAX_ENUM_ML = 0x7FFFFFFF
} XrMarkerArucoDictML;
```

Supported predefined ArUco dictionary:

Enumerant Descriptions

- XR_MARKER_ARUCO_DICT_4X4_50_ML — 4 by 4 pixel Aruco marker dictionary with 50 IDs.
- XR_MARKER_ARUCO_DICT_4X4_100_ML — 4 by 4 pixel Aruco marker dictionary with 100 IDs.
- XR_MARKER_ARUCO_DICT_4X4_250_ML — 4 by 4 pixel Aruco marker dictionary with 250 IDs.
- XR_MARKER_ARUCO_DICT_4X4_1000_ML — 4 by 4 pixel Aruco marker dictionary with 1000 IDs.
- XR_MARKER_ARUCO_DICT_5X5_50_ML — 5 by 5 pixel Aruco marker dictionary with 50 IDs.
- XR_MARKER_ARUCO_DICT_5X5_100_ML — 5 by 5 pixel Aruco marker dictionary with 100 IDs.
- XR_MARKER_ARUCO_DICT_5X5_250_ML — 5 by 5 pixel Aruco marker dictionary with 250 IDs.
- XR_MARKER_ARUCO_DICT_5X5_1000_ML — 5 by 5 pixel Aruco marker dictionary with 1000 IDs.
- XR_MARKER_ARUCO_DICT_6X6_50_ML — 6 by 6 pixel Aruco marker dictionary with 50 IDs.
- XR_MARKER_ARUCO_DICT_6X6_100_ML — 6 by 6 pixel Aruco marker dictionary with 100 IDs.
- XR_MARKER_ARUCO_DICT_6X6_250_ML — 6 by 6 pixel Aruco marker dictionary with 250 IDs.
- XR_MARKER_ARUCO_DICT_6X6_1000_ML — 6 by 6 pixel Aruco marker dictionary with 1000 IDs.
- XR_MARKER_ARUCO_DICT_7X7_50_ML — 7 by 7 pixel Aruco marker dictionary with 50 IDs.
- XR_MARKER_ARUCO_DICT_7X7_100_ML — 7 by 7 pixel Aruco marker dictionary with 100 IDs.
- XR_MARKER_ARUCO_DICT_7X7_250_ML — 7 by 7 pixel Aruco marker dictionary with 250 IDs.
- XR_MARKER_ARUCO_DICT_7X7_1000_ML — 7 by 7 pixel Aruco marker dictionary with 1000 IDs.

The XrMarkerDetectorAprilTagInfoML structure extends XrMarkerDetectorCreateInfoML and is defined as:

```
// Provided by XR_ML_marker_understanding
typedef struct XrMarkerDetectorAprilTagInfoML {
    XrStructureType           type;
    const void*               next;
    XrMarkerAprilTagDictML    aprilTagDict;
} XrMarkerDetectorAprilTagInfoML;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain.
- aprilTagDict AprilTag Dictionary name from which markers will be detected.

This structure is required by the XR_MARKER_TYPE_APRIL_TAG_ML detector.

Valid Usage (Implicit)

- The XR_ML_marker_understanding extension **must** be enabled prior to using XrMarkerDetectorAprilTagInfoML
- type **must** be XR_TYPE_MARKER_DETECTOR_APRIL_TAG_INFO_ML
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- aprilTagDict **must** be a valid XrMarkerAprilTagDictML value

The XrMarkerAprilTagDictML enumeration is defined as:

```
// Provided by XR_ML_marker_understanding
typedef enum XrMarkerAprilTagDictML {
    XR_MARKER_APRIL_TAG_DICT_16H5_ML = 0,
    XR_MARKER_APRIL_TAG_DICT_25H9_ML = 1,
    XR_MARKER_APRIL_TAG_DICT_36H10_ML = 2,
    XR_MARKER_APRIL_TAG_DICT_36H11_ML = 3,
    XR_MARKER_APRIL_TAG_DICT_MAX_ENUM_ML = 0x7FFFFFFF
} XrMarkerAprilTagDictML;
```

Supported predefined AprilTag dictionary:

Enumerant Descriptions

- XR_MARKER_APRIL_TAG_DICT_16H5_ML — 4 by 4 bits, minimum Hamming distance between any two codes = 5, 30 codes.
- XR_MARKER_APRIL_TAG_DICT_25H9_ML — 5 by 5 bits, minimum Hamming distance between any two codes = 9, 35 codes.
- XR_MARKER_APRIL_TAG_DICT_36H10_ML — 6 by 6 bits, minimum Hamming distance between any two codes = 10, 2320 codes.
- XR_MARKER_APRIL_TAG_DICT_36H11_ML — 6 by 6 bits, minimum Hamming distance between any two codes = 11, 587 codes.

The XrMarkerDetectorSizeInfoML structure extends XrMarkerDetectorCreateInfoML and is defined as:

```
// Provided by XR_ML_marker_understanding
typedef struct XrMarkerDetectorSizeInfoML {
    XrStructureType    type;
    const void*        next;
    float              markerLength;
} XrMarkerDetectorSizeInfoML;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `markerLength` is the physical length of one side of a marker.

Pose estimation accuracy depends on the accuracy of the specified `markerLength`.

This structure is used by `XR_MARKER_TYPE_ARUCO_ML`, `XR_MARKER_TYPE_APRIL_TAG_ML`, and `XR_MARKER_TYPE_QR_ML` detectors.

Valid Usage (Implicit)

- The `XR_ML_marker_understanding` extension **must** be enabled prior to using XrMarkerDetectorSizeInfoML
- `type` **must** be `XR_TYPE_MARKER_DETECTOR_SIZE_INFO_ML`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

The xrDestroyMarkerDetectorML function is defined as:

```
// Provided by XR_ML_marker_understanding
XrResult xrDestroyMarkerDetectorML(
    XrMarkerDetectorML                          markerDetector);
```

Parameter Descriptions

- `markerDetector` object to destroy.

Destroy a marker detection handle.

Valid Usage (Implicit)

- The XR_ML_marker_understanding extension **must** be enabled prior to calling xrDestroyMarkerDetectorML
- markerDetector **must** be a valid XrMarkerDetectorML handle

Thread Safety

- Access to markerDetector, and any child handles, **must** be externally synchronized

Return Codes

Success

- XR_SUCCESS

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_RUNTIME_FAILURE

Using a custom profile

The XrMarkerDetectorCustomProfileInfoML structure extends XrMarkerDetectorCreateInfoML and is defined as:

```
// Provided by XR_ML_marker_understanding
typedef struct XrMarkerDetectorCustomProfileInfoML {
    XrStructureType                          type;
    const void*                              next;
    XrMarkerDetectorFpsML                    fpsHint;
    XrMarkerDetectorResolutionML             resolutionHint;
    XrMarkerDetectorCameraML                 cameraHint;
    XrMarkerDetectorCornerRefineMethodML     cornerRefineMethod;
    XrBool32                                 useEdgeRefinement;
    XrMarkerDetectorFullAnalysisIntervalML   fullAnalysisIntervalHint;
} XrMarkerDetectorCustomProfileInfoML;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `fpsHint` is a suggestion of the category of frame rate for the detector to use.
- `resolutionHint` is a suggestion of the category of camera resolution for the detector to use.
- `cameraHint` is a suggestion of the camera set for the detector to use
- `cornerRefineMethod` selects a method for corner refinement for ArUco/AprilTag detectors. This member is ignored for detectors of other marker types.
- `useEdgeRefinement` specifies whether to run a refinement step that uses marker edges to generate even more accurate corners, but slow down tracking rate overall by consuming more compute. It affects ArUco/AprilTag markers only: this member is ignored for detectors of other marker types.
- `fullAnalysisIntervalHint` is the suggested interval between fully analyzed frames that introduce new detected markers, in addition to updating the state of already detected markers.

All marker detectors share some underlying hardware and resources, and thus not all combinations of profiles between multiple detectors are possible. If a profile (preset or custom) specified during marker detector creation is different from those used by existing marker detectors the runtime will attempt to honor the highest frame rate and fps requested.

CPU load due to marker tracking is a function of the chosen XrMarkerTypeML, XrMarkerDetectorFpsML, and XrMarkerDetectorResolutionML.

Valid Usage (Implicit)

- The `XR_ML_marker_understanding` extension **must** be enabled prior to using XrMarkerDetectorCustomProfileInfoML
- `type` **must** be `XR_TYPE_MARKER_DETECTOR_CUSTOM_PROFILE_INFO_ML`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `fpsHint` **must** be a valid XrMarkerDetectorFpsML value
- `resolutionHint` **must** be a valid XrMarkerDetectorResolutionML value
- `cameraHint` **must** be a valid XrMarkerDetectorCameraML value
- `cornerRefineMethod` **must** be a valid XrMarkerDetectorCornerRefineMethodML value
- `fullAnalysisIntervalHint` **must** be a valid XrMarkerDetectorFullAnalysisIntervalML value

The XrMarkerDetectorFpsML enumeration is defined as:

```
// Provided by XR_ML_marker_understanding
typedef enum XrMarkerDetectorFpsML {
    XR_MARKER_DETECTOR_FPS_LOW_ML = 0,
    XR_MARKER_DETECTOR_FPS_MEDIUM_ML = 1,
    XR_MARKER_DETECTOR_FPS_HIGH_ML = 2,
    XR_MARKER_DETECTOR_FPS_MAX_ML = 3,
    XR_MARKER_DETECTOR_FPS_MAX_ENUM_ML = 0x7FFFFFFF
} XrMarkerDetectorFpsML;
```

Used to hint to the back-end the max frames per second that **should** be analyzed.

Enumerant Descriptions

- XR_MARKER_DETECTOR_FPS_LOW_ML — Low FPS.
- XR_MARKER_DETECTOR_FPS_MEDIUM_ML — Medium FPS.
- XR_MARKER_DETECTOR_FPS_HIGH_ML — High FPS.
- XR_MARKER_DETECTOR_FPS_MAX_ML — Max possible FPS.

The XrMarkerDetectorResolutionML enumeration is defined as:

```
// Provided by XR_ML_marker_understanding
typedef enum XrMarkerDetectorResolutionML {
    XR_MARKER_DETECTOR_RESOLUTION_LOW_ML = 0,
    XR_MARKER_DETECTOR_RESOLUTION_MEDIUM_ML = 1,
    XR_MARKER_DETECTOR_RESOLUTION_HIGH_ML = 2,
    XR_MARKER_DETECTOR_RESOLUTION_MAX_ENUM_ML = 0x7FFFFFFF
} XrMarkerDetectorResolutionML;
```

Used to hint to the back-end the resolution that **should** be used. CPU load is a combination of chosen XrMarkerTypeML, XrMarkerDetectorFpsML, and XrMarkerDetectorResolutionML.

Enumerant Descriptions

- XR_MARKER_DETECTOR_RESOLUTION_LOW_ML — Low Resolution.
- XR_MARKER_DETECTOR_RESOLUTION_MEDIUM_ML — Medium Resolution.
- XR_MARKER_DETECTOR_RESOLUTION_HIGH_ML — High Resolution.

The XrMarkerDetectorCameraML enumeration is defined as:

```
// Provided by XR_ML_marker_understanding
typedef enum XrMarkerDetectorCameraML {
    XR_MARKER_DETECTOR_CAMERA_RGB_CAMERA_ML = 0,
    XR_MARKER_DETECTOR_CAMERA_WORLD_CAMERAS_ML = 1,
    XR_MARKER_DETECTOR_CAMERA_MAX_ENUM_ML = 0x7FFFFFFF
} XrMarkerDetectorCameraML;
```

The XrMarkerDetectorCameraML enum values are used to hint to the camera that **should** be used. This is set in the XrMarkerDetectorCustomProfileInfoML.

The RGB camera has a higher resolution than world cameras and is better suited for use cases where the target to be tracked is small or needs to be detected from far away.

XR_MARKER_DETECTOR_CAMERA_WORLD_CAMERAS_ML make use of multiple cameras to improve accuracy and increase the FoV for detection.

Enumerant Descriptions

- XR_MARKER_DETECTOR_CAMERA_RGB_CAMERA_ML — Single RGB camera.
- XR_MARKER_DETECTOR_CAMERA_WORLD_CAMERAS_ML — One or more world cameras.

The XrMarkerDetectorCornerRefineMethodML enumeration is defined as:

```
// Provided by XR_ML_marker_understanding
typedef enum XrMarkerDetectorCornerRefineMethodML {
    XR_MARKER_DETECTOR_CORNER_REFINE_METHOD_NONE_ML = 0,
    XR_MARKER_DETECTOR_CORNER_REFINE_METHOD_SUBPIX_ML = 1,
    XR_MARKER_DETECTOR_CORNER_REFINE_METHOD_CONTOUR_ML = 2,
    XR_MARKER_DETECTOR_CORNER_REFINE_METHOD_APRIL_TAG_ML = 3,
    XR_MARKER_DETECTOR_CORNER_REFINE_METHOD_MAX_ENUM_ML = 0x7FFFFFFF
} XrMarkerDetectorCornerRefineMethodML;
```

The ArUco/AprilTag detector comes with several corner refinement methods. Choosing the right corner refinement method has an impact on the accuracy and speed trade-off that comes with each detection pipeline.

Enumerant Descriptions

- XR_MARKER_DETECTOR_CORNER_REFINE_METHOD_NONE_ML — No refinement. Inaccurate corners.
- XR_MARKER_DETECTOR_CORNER_REFINE_METHOD_SUBPIX_ML — Subpixel refinement. Corners have subpixel coordinates. High detection rate, very fast, reasonable accuracy.
- XR_MARKER_DETECTOR_CORNER_REFINE_METHOD_CONTOUR_ML — Contour refinement. High detection rate, fast, reasonable accuracy.
- XR_MARKER_DETECTOR_CORNER_REFINE_METHOD_APRIL_TAG_ML — AprilTag refinement. Reasonable detection rate, slowest, but very accurate. Only valid with AprilTags.

The XrMarkerDetectorFullAnalysisIntervalML enumeration is defined as:

```
// Provided by XR_ML_marker_understanding
typedef enum XrMarkerDetectorFullAnalysisIntervalML {
    XR_MARKER_DETECTOR_FULL_ANALYSIS_INTERVAL_MAX_ML = 0,
    XR_MARKER_DETECTOR_FULL_ANALYSIS_INTERVAL_FAST_ML = 1,
    XR_MARKER_DETECTOR_FULL_ANALYSIS_INTERVAL_MEDIUM_ML = 2,
    XR_MARKER_DETECTOR_FULL_ANALYSIS_INTERVAL_SLOW_ML = 3,
    XR_MARKER_DETECTOR_FULL_ANALYSIS_INTERVAL_MAX_ENUM_ML = 0x7FFFFFFF
} XrMarkerDetectorFullAnalysisIntervalML;
```

In order to improve performance, the detectors do not always run on the full frame. Full frame analysis is however necessary to detect new markers that were not detected before. Use this option to control how often the detector **should** detect new markers and its impact on tracking performance.

Enumerant Descriptions

- XR_MARKER_DETECTOR_FULL_ANALYSIS_INTERVAL_MAX_ML — Detector analyzes every frame fully.
- XR_MARKER_DETECTOR_FULL_ANALYSIS_INTERVAL_FAST_ML — Detector analyzes frame fully very often.
- XR_MARKER_DETECTOR_FULL_ANALYSIS_INTERVAL_MEDIUM_ML — Detector analyzes frame fully a few times per second.
- XR_MARKER_DETECTOR_FULL_ANALYSIS_INTERVAL_SLOW_ML — Detector analyzes frame fully about every second.

12.180.3. Scanning for markers

The xrSnapshotMarkerDetectorML function is defined as:

```
// Provided by XR_ML_marker_understanding
XrResult xrSnapshotMarkerDetectorML(
    XrMarkerDetectorML                          markerDetector,
    XrMarkerDetectorSnapshotInfoML*             snapshotInfo);
```

Parameter Descriptions

- `markerDetector` object to issue a snapshot request to.
- `snapshotInfo` is a pointer to XrMarkerDetectorSnapshotInfoML containing marker snapshot parameters.

Collects the latest marker detector state and makes it ready for inspection. This function only snapshots the non-pose state of markers. Once called, and if a new snapshot is not yet available a runtime **must** set the state of the marker detector to `XR_MARKER_DETECTOR_STATUS_PENDING_ML`. If a new state is available the runtime **must** set the state to `XR_MARKER_DETECTOR_STATUS_READY_ML`. If an error occurred the runtime **must** set the state to `XR_MARKER_DETECTOR_STATUS_ERROR_ML`. The application **may** attempt the snapshot again.

Once the application has inspected the state it is interested in it **can** call this function again and the state is set to `XR_MARKER_DETECTOR_STATUS_PENDING_ML` until a new state has been snapshotted. After each snapshot, only the currently detected markers are available for inspection, though the same marker **may** repeatedly be detected across snapshots.

Valid Usage (Implicit)

- The `XR_ML_marker_understanding` extension **must** be enabled prior to calling xrSnapshotMarkerDetectorML
- `markerDetector` **must** be a valid XrMarkerDetectorML handle
- `snapshotInfo` **must** be a pointer to an XrMarkerDetectorSnapshotInfoML structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The XrMarkerDetectorSnapshotInfoML structure is defined as:

```
// Provided by XR_ML_marker_understanding
typedef struct XrMarkerDetectorSnapshotInfoML {
    XrStructureType    type;
    const void*        next;
} XrMarkerDetectorSnapshotInfoML;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.

Valid Usage (Implicit)

- The XR_ML_marker_understanding extension **must** be enabled prior to using XrMarkerDetectorSnapshotInfoML
- type **must** be XR_TYPE_MARKER_DETECTOR_SNAPSHOT_INFO_ML
- next **must** be NULL or a valid pointer to the next structure in a structure chain

The xrGetMarkerDetectorStateML function is defined as:

```
// Provided by XR_ML_marker_understanding
XrResult xrGetMarkerDetectorStateML(
    XrMarkerDetectorML                          markerDetector,
    XrMarkerDetectorStateML*                    state);
```

Parameter Descriptions

- `markerDetector` object to retrieve state information from.
- `state` points to an XrMarkerDetectorStateML in which the current state of the marker detector is returned.

xrGetMarkerDetectorStateML is used after calling xrSnapshotMarkerDetectorML to check the current status of the snapshot in progress. When XrMarkerDetectorStateML::`state` == `XR_MARKER_DETECTOR_STATUS_READY_ML`, the detector is ready to be queried, while `XR_MARKER_DETECTOR_STATUS_PENDING_ML` indicates the snapshot is still in progress. `XR_MARKER_DETECTOR_STATUS_ERROR_ML` indicates that the runtime has encountered an error getting a snapshot for the requested detector, which **may** require user intervention to solve.

If xrSnapshotMarkerDetectorML has not yet been called for the `markerDetector`, the runtime **must** return `XR_ERROR_CALL_ORDER_INVALID`.

Valid Usage (Implicit)

- The `XR_ML_marker_understanding` extension **must** be enabled prior to calling xrGetMarkerDetectorStateML
- `markerDetector` **must** be a valid XrMarkerDetectorML handle
- `state` **must** be a pointer to an XrMarkerDetectorStateML structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_CALL_ORDER_INVALID
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The XrMarkerDetectorStateML structure is defined as:

```
// Provided by XR_ML_marker_understanding
typedef struct XrMarkerDetectorStateML {
    XrStructureType             type;
    void*                       next;
    XrMarkerDetectorStatusML    state;
} XrMarkerDetectorStateML;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- state is the current state of the marker detector.

Valid Usage (Implicit)

- The XR_ML_marker_understanding extension **must** be enabled prior to using XrMarkerDetectorStateML
- type **must** be XR_TYPE_MARKER_DETECTOR_STATE_ML
- next **must** be NULL or a valid pointer to the next structure in a structure chain

The XrMarkerDetectorStatusML enumeration is defined as:

```
// Provided by XR_ML_marker_understanding
typedef enum XrMarkerDetectorStatusML {
    XR_MARKER_DETECTOR_STATUS_PENDING_ML = 0,
    XR_MARKER_DETECTOR_STATUS_READY_ML = 1,
    XR_MARKER_DETECTOR_STATUS_ERROR_ML = 2,
    XR_MARKER_DETECTOR_STATUS_MAX_ENUM_ML = 0x7FFFFFFF
} XrMarkerDetectorStatusML;
```

The XrMarkerDetectorStatusML enumeration describes the current state of the marker detector. It is queried via xrGetMarkerDetectorStateML to determine if the marker tracker is currently available for inspection.

Enumerant Descriptions

- XR_MARKER_DETECTOR_STATUS_PENDING_ML — The marker detector is working on a new snapshot.
- XR_MARKER_DETECTOR_STATUS_READY_ML — The marker detector is ready to be inspected.
- XR_MARKER_DETECTOR_STATUS_ERROR_ML — The marker detector has encountered a fatal error.

12.180.4. Getting Marker Results

The xrGetMarkersML function is defined as:

```
// Provided by XR_ML_marker_understanding
XrResult xrGetMarkersML(
    XrMarkerDetectorML                          markerDetector,
    uint32_t                                    markerCapacityInput,
    uint32_t*                                   markerCountOutput,
    XrMarkerML*                                 markers);
```

Parameter Descriptions

- `markerDetector` is the detector object to retrieve marker information from.
- `markerCapacityInput` is the capacity of the `markers` array or `0` to indicate a request to retrieve the required capacity.
- `markerCountOutput` is filled in by the runtime with the count of marker atoms written or the required capacity in the case that `markerCapacityInput` is insufficient.
- `markers` is a pointer to an array of `XrMarkerML` atoms, but **can** be `NULL` if propertyCapacityInput is `0`.
- See the Buffer Size Parameters section for a detailed description of retrieving the required `markers` size.

Get the list of current snapshotted marker atoms, **must** only be called when the state of the detector is `XR_MARKER_DETECTOR_STATUS_READY_ML`.

If xrGetMarkerDetectorStateML has not been called and returned `XR_MARKER_DETECTOR_STATUS_READY_ML` since the last invocation of xrSnapshotMarkerDetectorML, the runtime **must** return `XR_ERROR_CALL_ORDER_INVALID`.

The returned atoms are only valid while in the `XR_MARKER_DETECTOR_STATUS_READY_ML` state. The runtime **must** return the same atom value for the same uniquely identifiable marker across successive snapshots. It is unspecified what happens if the detector is observing two markers with the same identification patterns.

Assuming the same set of markers are in view across several snapshots, the runtime **should** return the same set of atoms. An application **can** use the list of atoms as a simple test for if a particular marker has gone in or out of view.

Note that `XrMarkerML` atoms are only usable with the XrMarkerDetectorML that returned them.

This function follows the two-call idiom for filling the `markers`.

Valid Usage (Implicit)

- The `XR_ML_marker_understanding` extension **must** be enabled prior to calling xrGetMarkersML
- `markerDetector` **must** be a valid XrMarkerDetectorML handle
- `markerCountOutput` **must** be a pointer to a `uint32_t` value
- If `markerCapacityInput` is not `0`, `markers` **must** be a pointer to an array of `markerCapacityInput` `XrMarkerML` values

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_CALL_ORDER_INVALID
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

```
// Provided by XR_ML_marker_understanding
XR_DEFINE_ATOM(XrMarkerML)
```

The unique marker key used to retrieve the data about detected markers. For an XrMarkerDetectorML a runtime **must** use the same value of XrMarkerML each time a marker is detected in a snapshot, but an application **cannot** use a cached atom if it was not present in the most recent snapshot.

The xrGetMarkerNumberML function is defined as:

```
// Provided by XR_ML_marker_understanding
XrResult xrGetMarkerNumberML(
    XrMarkerDetectorML                          markerDetector,
    XrMarkerML                                  marker,
    uint64_t*                                   number);
```

Parameter Descriptions

- markerDetector is the detector object to retrieve marker information from.
- marker is the marker atom to be examined.
- number points to a float in which the numerical value associated with the marker is returned.

Get the numerical value of a marker, such as the ArUco ID. xrGetMarkerNumberML **must** only be

called when the state of the detector is XR_MARKER_DETECTOR_STATUS_READY_ML. If the marker does not have an associated numerical value, the runtime **must** return XR_ERROR_MARKER_DETECTOR_INVALID_DATA_QUERY_ML.

If xrGetMarkerDetectorStateML has not been called and returned XR_MARKER_DETECTOR_STATUS_READY_ML since the last invocation of xrSnapshotMarkerDetectorML, the runtime **must** return XR_ERROR_CALL_ORDER_INVALID.

The runtime **must** return XR_ERROR_MARKER_INVALID_ML if the marker atom is invalid.

Valid Usage (Implicit)

- The XR_ML_marker_understanding extension **must** be enabled prior to calling xrGetMarkerNumberML
- markerDetector **must** be a valid XrMarkerDetectorML handle
- number **must** be a pointer to a uint64_t value

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_MARKER_DETECTOR_INVALID_DATA_QUERY_ML
- XR_ERROR_MARKER_INVALID_ML
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The xrGetMarkerStringML function is defined as:

```
// Provided by XR_ML_marker_understanding
XrResult xrGetMarkerStringML(
    XrMarkerDetectorML                          markerDetector,
    XrMarkerML                                  marker,
    uint32_t                                    bufferCapacityInput,
    uint32_t*                                   bufferCountOutput,
    char*                                       buffer);
```

Parameter Descriptions

- `markerDetector` is the detector object to retrieve marker information from.
- `marker` is the marker atom to be examined.
- `bufferCapacityInput` is the capacity of the buffer, or 0 to indicate a request to retrieve the required capacity.
- `bufferCountOutput` is a pointer to the count of characters written to buffer (including the terminating '\0'), or a pointer to the required capacity in the case that bufferCapacityInput is insufficient.
- `buffer` is a pointer to an application-allocated buffer that **should** be filled with the QR code's contents. It **can** be NULL if bufferCapacityInput is 0.
- See the Buffer Size Parameters section for a detailed description of retrieving the required `buffer` size.

Get the string value of a marker, such as the QR encoded string. xrCreateMarkerSpaceML **must** only be called when the state of the detector is `XR_MARKER_DETECTOR_STATUS_READY_ML`.

If the marker does not have an associated string value, the runtime **must** return `XR_ERROR_MARKER_DETECTOR_INVALID_DATA_QUERY_ML`.

If xrGetMarkerDetectorStateML has not been called and returned `XR_MARKER_DETECTOR_STATUS_READY_ML` since the last invocation of xrSnapshotMarkerDetectorML, the runtime **must** return `XR_ERROR_CALL_ORDER_INVALID`.

This function follows the two-call idiom for filling the `buffer`.

The runtime **must** return `XR_ERROR_MARKER_INVALID_ML` if the marker atom is invalid.

Valid Usage (Implicit)

- The XR_ML_marker_understanding extension **must** be enabled prior to calling xrGetMarkerStringML
- markerDetector **must** be a valid XrMarkerDetectorML handle
- bufferCountOutput **must** be a pointer to a uint32_t value
- If bufferCapacityInput is not 0, buffer **must** be a pointer to an array of bufferCapacityInput char values

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_MARKER_DETECTOR_INVALID_DATA_QUERY_ML
- XR_ERROR_MARKER_INVALID_ML
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The xrGetMarkerReprojectionErrorML function is defined as:

```
// Provided by XR_ML_marker_understanding
XrResult xrGetMarkerReprojectionErrorML(
    XrMarkerDetectorML                          markerDetector,
    XrMarkerML                                  marker,
    float*                                      reprojectionErrorMeters);
```

Parameter Descriptions

- `markerDetector` is the detector object to retrieve marker information from.
- `marker` is the marker atom to be examined.
- `reprojectionErrorMeters` points to a `float` in which the estimated reprojection error in meters is returned.

Get the reprojection error of a marker, only available for certain types of markers. **must** only be called when the state of the detector is `XR_MARKER_DETECTOR_STATUS_READY_ML`.

If xrGetMarkerDetectorStateML has not been called and returned `XR_MARKER_DETECTOR_STATUS_READY_ML` since the last invocation of xrSnapshotMarkerDetectorML, the runtime **must** return `XR_ERROR_CALL_ORDER_INVALID`.

A high reprojection error means that the estimated pose of the marker does not match well with the 2D detection on the processed video frame and thus the pose **may** be inaccurate. The error is given in meters, representing the displacement between real marker and its estimated pose. This means this is a normalized number, independent of marker distance or length.

The runtime **must** return `XR_ERROR_MARKER_INVALID_ML` if the marker atom is invalid.

Valid Usage (Implicit)

- The `XR_ML_marker_understanding` extension **must** be enabled prior to calling xrGetMarkerReprojectionErrorML
- `markerDetector` **must** be a valid XrMarkerDetectorML handle
- `reprojectionErrorMeters` **must** be a pointer to a `float` value

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_CALL_ORDER_INVALID
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_MARKER_INVALID_ML
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The xrGetMarkerLengthML function is defined as:

```
// Provided by XR_ML_marker_understanding
XrResult xrGetMarkerLengthML(
    XrMarkerDetectorML                          markerDetector,
    XrMarkerML                                  marker,
    float*                                      meters);
```

Parameter Descriptions

- markerDetector is the detector object to retrieve marker information from.
- marker is the marker atom to be examined.
- meters points to a float in which the size per side of the queried marker is returned.

Get the size of the marker, defined as the length in meters per side. If the application created the detector while passing in a XrMarkerDetectorSizeInfoML, this query **may** be redundant. xrGetMarkerLengthML is primarily intended to query for a runtime estimated size when an application did not indicate the expected size via XrMarkerDetectorSizeInfoML.

xrGetMarkerLengthML **must** only be called when the state of the detector is XR_MARKER_DETECTOR_STATUS_READY_ML. If xrGetMarkerDetectorStateML has not been called and returned XR_MARKER_DETECTOR_STATUS_READY_ML since the last invocation of xrSnapshotMarkerDetectorML, the

runtime **must** return XR_ERROR_CALL_ORDER_INVALID.

The runtime **must** return XR_ERROR_MARKER_INVALID_ML if the marker atom is invalid.

Valid Usage (Implicit)

- The XR_ML_marker_understanding extension **must** be enabled prior to calling xrGetMarkerLengthML
- markerDetector **must** be a valid XrMarkerDetectorML handle
- meters **must** be a pointer to a float value

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_CALL_ORDER_INVALID
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_MARKER_INVALID_ML
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

12.180.5. Getting an XrSpace from Marker Results

The xrCreateMarkerSpaceML function is defined as:

```
// Provided by XR_ML_marker_understanding
XrResult xrCreateMarkerSpaceML(
    XrSession                                   session,
    const XrMarkerSpaceCreateInfoML*            createInfo,
    XrSpace*                                    space);
```

Parameter Descriptions

- `session` is the session that will own the created space.
- `createInfo` is a pointer to the XrMarkerSpaceCreateInfoML used to specify the space creation parameters.
- `space` points to an XrSpace handle in which the resulting space is returned.

Creates an XrSpace from a currently snapshotted marker. The space **may** still be used even if the marker is later not in the FOV, or even if the marker detector has been destroyed. In such a scenario, the `XR_SPACE_LOCATION_ORIENTATION_TRACKED_BIT` and `XR_SPACE_LOCATION_POSITION_TRACKED_BIT` **must** be false, but `XR_SPACE_LOCATION_POSITION_VALID_BIT` and `XR_SPACE_LOCATION_ORIENTATION_VALID_BIT` **may** be set as appropriate to the last known location.

Once an application has created a space, it **may** stop calling xrSnapshotMarkerDetectorML, and the position of the marker **must** still be updated by the runtime whenever it is aware of a more up to date location.

If a runtime is unable to spatially locate a snapshotted marker, it **may** return `XR_ERROR_MARKER_DETECTOR_LOCATE_FAILED_ML`. This is most likely to happen if significant time has passed since the snapshot of markers was acquired, and the marker in question is no longer in the user's FOV. Thus, an application **should** call xrCreateMarkerSpaceML immediately after examining a snapshot, but **should** also be prepared to try again if needed.

must only be called when the state of the detector is `XR_MARKER_DETECTOR_STATUS_READY_ML`.

If xrGetMarkerDetectorStateML has not been called and returned `XR_MARKER_DETECTOR_STATUS_READY_ML` since the last invocation of xrSnapshotMarkerDetectorML, the runtime **must** return `XR_ERROR_CALL_ORDER_INVALID`.

`session` must be the same session that created the XrMarkerSpaceCreateInfoML::`markerDetector`, else the runtime **must** return `XR_ERROR_HANDLE_INVALID`.

The runtime **must** return `XR_ERROR_MARKER_INVALID_ML` if the marker atom is invalid.

The XrSpace origin **must** be located at the marker's center. The X-Y plane of the XrSpace **must** be aligned with the plane of the marker with the positive Z axis coming out of the marker face.

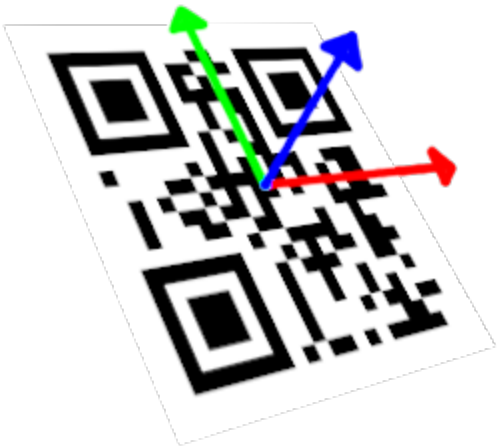


Figure 36. QR code marker with axis

Valid Usage (Implicit)

- The `XR_ML_marker_understanding` extension **must** be enabled prior to calling xrCreateMarkerSpaceML
- `session` **must** be a valid XrSession handle
- `createInfo` **must** be a pointer to a valid XrMarkerSpaceCreateInfoML structure
- `space` **must** be a pointer to an XrSpace handle

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_CALL_ORDER_INVALID
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_MARKER_DETECTOR_LOCATE_FAILED_ML
- XR_ERROR_MARKER_INVALID_ML
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_POSE_INVALID
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The XrMarkerSpaceCreateInfoML structure is defined as:

```
// Provided by XR_ML_marker_understanding
typedef struct XrMarkerSpaceCreateInfoML {
    XrStructureType       type;
    const void*           next;
    XrMarkerDetectorML    markerDetector;
    XrMarkerML            marker;
    XrPosef               poseInMarkerSpace;
} XrMarkerSpaceCreateInfoML;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `markerDetector` is the detector object to retrieve marker information from.
- `marker` is the marker atom to be examined.
- `poseInMarkerSpace` is the offset from the marker's origin of the new XrSpace. The origin of each marker is located at its center.

Valid Usage (Implicit)

- The `XR_ML_marker_understanding` extension **must** be enabled prior to using XrMarkerSpaceCreateInfoML
- `type` **must** be `XR_TYPE_MARKER_SPACE_CREATE_INFO_ML`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `markerDetector` **must** be a valid XrMarkerDetectorML handle

12.180.6. Example code for locating a marker

The following example code demonstrates how to detect a marker relative to a local space, and query the contents.

```
XrInstance instance;  // previously initialized
XrSystemId systemId;  // previously initialized
XrSession session;    // previously initialized
XrSpace localSpace;   // previously initialized, e.g. from
                      // XR_REFERENCE_SPACE_TYPE_LOCAL
XrSpace viewSpace;    // previously initialized, e.g. from
                      // XR_REFERENCE_SPACE_TYPE_VIEW

// The function pointers are previously initialized using
// xrGetInstanceProcAddr.
PFN_xrCreateMarkerDetectorML xrCreateMarkerDetectorML; // previously initialized
PFN_xrDestroyMarkerDetectorML xrDestroyMarkerDetectorML; // previously initialized
PFN_xrSnapshotMarkerDetectorML xrSnapshotMarkerDetectorML; // previously initialized
PFN_xrGetMarkerDetectorStateML xrGetMarkerDetectorStateML; // previously initialized
PFN_xrGetMarkersML xrGetMarkersML; // previously initialized
PFN_xrGetMarkerReprojectionErrorML xrGetMarkerReprojectionErrorML; // previously
initialized
```

```
PFN_xrGetMarkerLengthML xrGetMarkerLengthML; // previously initialized
PFN_xrGetMarkerNumberML xrGetMarkerNumberML; // previously initialized
PFN_xrGetMarkerStringML xrGetMarkerStringML; // previously initialized
PFN_xrCreateMarkerSpaceML xrCreateMarkerSpaceML; // previously initialized


// Initialize marker detector handle
XrMarkerDetectorML markerDetector = XR_NULL_HANDLE;

XrMarkerDetectorCreateInfoML createInfo{ XR_TYPE_MARKER_DETECTOR_CREATE_INFO_ML };
createInfo.profile = XR_MARKER_DETECTOR_PROFILE_CUSTOM_ML;
createInfo.markerType = XR_MARKER_TYPE_ARUCO_ML;

// Passing a non-custom profile allows you to leave next == nullptr
XrMarkerDetectorCustomProfileInfoML customProfile{
XR_TYPE_MARKER_DETECTOR_CUSTOM_PROFILE_INFO_ML };
customProfile.fpsHint = XR_MARKER_DETECTOR_FPS_LOW_ML;
customProfile.resolutionHint = XR_MARKER_DETECTOR_RESOLUTION_HIGH_ML;
customProfile.cameraHint = XR_MARKER_DETECTOR_CAMERA_RGB_CAMERA_ML;
customProfile.cornerRefineMethod = XR_MARKER_DETECTOR_CORNER_REFINE_METHOD_CONTOUR_ML;
customProfile.useEdgeRefinement = true;
customProfile.fullAnalysisIntervalHint =
XR_MARKER_DETECTOR_FULL_ANALYSIS_INTERVAL_SLOW_ML;

createInfo.next = &customProfile;

// Elect to use ArUco marker tracking, providing required dictionary
XrMarkerDetectorArucoInfoML arucoCreateInfo{ XR_TYPE_MARKER_DETECTOR_ARUCO_INFO_ML };
arucoCreateInfo.arucoDict = XR_MARKER_ARUCO_DICT_6X6_100_ML;

customProfile.next = &arucoCreateInfo;

// Specify the size of the marker to improve tracking quality
XrMarkerDetectorSizeInfoML sizeCreateInfo{ XR_TYPE_MARKER_DETECTOR_SIZE_INFO_ML };
sizeCreateInfo.markerLength = 0.2f;

arucoCreateInfo.next = &sizeCreateInfo;

CHK_XR(xrCreateMarkerDetectorML(session, &createInfo, &markerDetector));

bool queryRunning = false;

std::unordered_map <uint64_t, XrSpace> markerSpaceMap;

auto processMarkers = [&]() {
    // 2 call idiom to get the markers from runtime
    uint32_t markerCount;
    CHK_XR(xrGetMarkersML(markerDetector, 0, &markerCount, nullptr));
```

```
    std::vector<XrMarkerML> markers(markerCount);
    CHK_XR(xrGetMarkersML(markerDetector, markerCount, &markerCount, markers.data()));

    for(uint32_t i = 0; i < markerCount; ++i)
    {
        uint64_t number;
        CHK_XR(xrGetMarkerNumberML(markerDetector, markers[i], &number));
        // Track every marker we find.
        if(markerSpaceMap.find(number) == markerSpaceMap.end())
        {
            // New entry
            XrSpace space;
            XrMarkerSpaceCreateInfoML spaceCreateInfo{
XR_TYPE_MARKER_SPACE_CREATE_INFO_ML };
            spaceCreateInfo.markerDetector = markerDetector;
            spaceCreateInfo.marker = markers[i];
            spaceCreateInfo.poseInMarkerSpace = { {0, 0, 0, 1}, {0, 0, 0} };

            CHK_XR(xrCreateMarkerSpaceML(session, &spaceCreateInfo, &space));
            markerSpaceMap[number] = space;
        }

        // This will not work in this example with ArUco markers, but had we configured
        // a marker with string content such as QR or Code 128, this is how to use it.
        // uint32_t stringSize;
        // CHK_XR(xrGetMarkerStringML(markerDetector, markers[i], 0, &stringSize,
nullptr));
        // std::string markerString(stringSize, ' ');
        // CHK_XR(xrGetMarkerStringML(markerDetector, markers[i], stringSize,
&stringSize, markerString.data()));
    }
};

// Must be initialized to true, otherwise in the loop below, there will
// be an XR_ERROR_CALL_ORDER_INVALID due to xrSnapshotMarkerDetectorML
// not being called first
bool isReadyForSnapshot = true;

while (1) {
    // ...
    // For every frame in frame loop
    // ...

    // We have this if/else block set up so that xrSnapshotMarkerDetectorML
    // is not captured per frame since the marker detector snapshot
    // might still be in the midst of being processed by the runtime
    if (isReadyForSnapshot) {
      // Call the first snapshot
```

```
      XrMarkerDetectorSnapshotInfoML detectorInfo{
XR_TYPE_MARKER_DETECTOR_SNAPSHOT_INFO_ML };
      CHK_XR(xrSnapshotMarkerDetectorML(markerDetector, &detectorInfo));
      isReadyForSnapshot = false;
    } else {
      XrMarkerDetectorStateML state{ XR_TYPE_MARKER_DETECTOR_STATE_ML };
      CHK_XR(xrGetMarkerDetectorStateML(markerDetector, &state));
      // For simplicity, this example will assume that the marker detector will not
      // be in an erroneous state
      if (state.state == XR_MARKER_DETECTOR_STATUS_READY_ML) {
        processMarkers();
        isReadyForSnapshot = true;
      }
    }


    // Draw the markers as needed from markerSpaceMap.
    // drawMarkers(markerSpaceMap);

    // ...
    // ...
}
// Cleanup
CHK_XR(xrDestroyMarkerDetectorML(markerDetector));
```

New Enum Constants

XrStructureType enumeration is extended with:

- XR_TYPE_SYSTEM_MARKER_UNDERSTANDING_PROPERTIES_ML
- XR_TYPE_MARKER_DETECTOR_CREATE_INFO_ML
- XR_TYPE_MARKER_DETECTOR_ARUCO_INFO_ML
- XR_TYPE_MARKER_DETECTOR_APRIL_TAG_INFO_ML
- XR_TYPE_MARKER_DETECTOR_CUSTOM_PROFILE_INFO_ML
- XR_TYPE_MARKER_DETECTOR_SNAPSHOT_INFO_ML
- XR_TYPE_MARKER_DETECTOR_STATE_ML
- XR_TYPE_MARKER_SPACE_CREATE_INFO_ML

the XrResult enumeration is extended with:

- XR_ERROR_MARKER_DETECTOR_PERMISSION_DENIED_ML
- XR_ERROR_MARKER_DETECTOR_LOCATE_FAILED_ML
- XR_ERROR_MARKER_DETECTOR_INVALID_DATA_QUERY_ML

- XR_ERROR_MARKER_DETECTOR_INVALID_CREATE_INFO_ML
- XR_ERROR_MARKER_INVALID_ML

New Structures

The XrSystemMarkerUnderstandingPropertiesML structure is defined as:

```
// Provided by XR_ML_marker_understanding
typedef struct XrSystemMarkerUnderstandingPropertiesML {
    XrStructureType    type;
    void*              next;
    XrBool32           supportsMarkerUnderstanding;
} XrSystemMarkerUnderstandingPropertiesML;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. For more details about the structure chain, see the structure being extended (XrSystemProperties).
- supportsMarkerUnderstanding indicates whether marker detection and tracking is supported by this system.

Valid Usage (Implicit)

- The XR_ML_marker_understanding extension **must** be enabled prior to using XrSystemMarkerUnderstandingPropertiesML
- type **must** be XR_TYPE_SYSTEM_MARKER_UNDERSTANDING_PROPERTIES_ML
- next **must** be NULL or a valid pointer to the next structure in a structure chain

Version History

- Revision 1, 2023-05-18 (Robbie Bridgewater)
  - Initial extension skeleton

12.181. XR_ML_spatial_anchors

Name String

XR_ML_spatial_anchors

Extension Type

Instance extension

Registered Extension Number

141

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

`XR_EXT_future`

Last Modified Date

2023-06-09

Contributors

Ron Bessems, Magic Leap
Karthik Kadappan, Magic Leap

12.181.1. Overview

Spatial anchors are XrSpace entities tied to a physical location. This allows the developer to place virtual content in real world locations.

The runtime **should** then adjust the XrSpace over time as needed, independently of all other spaces and anchors, to ensure that it maintains its original mapping to the real world.



Caution

If head pose is lost and regained, spatial anchors **may** also be lost. It is therefore strongly recommended that once an anchor is created, it is also persisted using the `XR_ML_spatial_anchors_storage` extension.



Permissions

Android applications **must** have the com.magicleap.permission.SPATIAL_ANCHOR permission listed in their manifest to use this extension. (protection level: normal)

12.181.2. Begin spatial anchor creation

xrCreateSpatialAnchorsAsyncML is used to create spatial anchors. It **can** create anchors in different ways depending on the parameter passed in. This extension defines one way to create single anchors using the XrSpatialAnchorsCreateInfoFromPoseML structure. `XR_ML_spatial_anchors_storage` extends

this to also create spatial anchors from a persistent storage using their XrUuidEXT.

The xrCreateSpatialAnchorsAsyncML function is defined as:

```
// Provided by XR_ML_spatial_anchors
XrResult xrCreateSpatialAnchorsAsyncML(
    XrSession                                       session,
    const XrSpatialAnchorsCreateInfoBaseHeaderML* createInfo,
    XrFutureEXT*                                    future);
```

Parameter Descriptions

- `session` is a handle to an XrSession previously created with xrCreateSession.
- `createInfo` is a pointer to an XrSpatialAnchorsCreateInfoBaseHeaderML derived structure.
- `future` is a pointer to an `XrFutureEXT`.

This function starts an asynchronous spatial anchor creation. Call one of the xrPollFutureEXT functions to check the ready state on the future. Once the future is in ready state, call xrCreateSpatialAnchorsCompleteML to retrieve the results.

Valid Usage (Implicit)

- The `XR_ML_spatial_anchors` extension **must** be enabled prior to calling xrCreateSpatialAnchorsAsyncML
- `session` **must** be a valid XrSession handle
- `createInfo` **must** be a pointer to a valid XrSpatialAnchorsCreateInfoBaseHeaderML-based structure. See also: XrSpatialAnchorsCreateInfoFromPoseML, XrSpatialAnchorsCreateInfoFromUuidsML
- `future` **must** be a pointer to an `XrFutureEXT` value

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_POSE_INVALID
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_SPATIAL_ANCHORS_PERMISSION_DENIED_ML
- XR_ERROR_SPATIAL_ANCHORS_SPACE_NOT_LOCATABLE_ML
- XR_ERROR_TIME_INVALID
- XR_ERROR_VALIDATION_FAILURE

The XrSpatialAnchorsCreateInfoFromPoseML structure **can** be used to create an spatial anchor from a pose in an XrSpace.

```
// Provided by XR_ML_spatial_anchors
typedef struct XrSpatialAnchorsCreateInfoFromPoseML {
    XrStructureType    type;
    const void*        next;
    XrSpace            baseSpace;
    XrPosef            poseInBaseSpace;
    XrTime             time;
} XrSpatialAnchorsCreateInfoFromPoseML;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `baseSpace` XrSpace used to calculate the position of the anchor.
- `poseInBaseSpace` XrPosef of the anchor to create.
- `time` `XrTime` to resolve the pose in the base space.

Note that xrCreateSpatialAnchorsCompleteML **must** be called with XrCreateSpatialAnchorsCompletionML::`spaceCount` set to 1 when using XrSpatialAnchorsCreateInfoFromPoseML to create a spatial anchor.

Valid Usage (Implicit)

- The `XR_ML_spatial_anchors` extension **must** be enabled prior to using XrSpatialAnchorsCreateInfoFromPoseML
- `type` **must** be `XR_TYPE_SPATIAL_ANCHORS_CREATE_INFO_FROM_POSE_ML`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `baseSpace` **must** be a valid XrSpace handle

The base structure for XrSpatialAnchorsCreateInfoFromPoseML is XrSpatialAnchorsCreateInfoBaseHeaderML.

The XrSpatialAnchorsCreateInfoBaseHeaderML structure is defined as:

```
// Provided by XR_ML_spatial_anchors
typedef struct XrSpatialAnchorsCreateInfoBaseHeaderML {
    XrStructureType    type;
    const void*        next;
} XrSpatialAnchorsCreateInfoBaseHeaderML;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.

This structure is not directly used in the API, please see XrSpatialAnchorsCreateInfoFromPoseML for an example of a child structure.

Valid Usage (Implicit)

- The XR_ML_spatial_anchors extension **must** be enabled prior to using XrSpatialAnchorsCreateInfoBaseHeaderML
- type **must** be one of the following XrStructureType values: XR_TYPE_SPATIAL_ANCHORS_CREATE_INFO_FROM_POSE_ML, XR_TYPE_SPATIAL_ANCHORS_CREATE_INFO_FROM_UUIDS_ML
- next **must** be NULL or a valid pointer to the next structure in a structure chain

12.181.3. Complete spatial anchor operation

xrCreateSpatialAnchorsCompleteML completes the asynchronous operation started by xrCreateSpatialAnchorsAsyncML. The XrFutureEXT **must** be in ready state before calling the completion function.

The xrCreateSpatialAnchorsCompleteML function is defined as:

```
// Provided by XR_ML_spatial_anchors
XrResult xrCreateSpatialAnchorsCompleteML(
    XrSession                                   session,
    XrFutureEXT                                 future,
    XrCreateSpatialAnchorsCompletionML*         completion);
```

Parameter Descriptions

- session is a handle to an XrSession previously created with xrCreateSession.
- future is the XrFutureEXT to complete.
- completion is a pointer to an XrCreateSpatialAnchorsCompletionML filled in by the runtime.

Valid Usage (Implicit)

- The XR_ML_spatial_anchors extension **must** be enabled prior to calling xrCreateSpatialAnchorsCompleteML
- session **must** be a valid XrSession handle
- completion **must** be a pointer to an XrCreateSpatialAnchorsCompletionML structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_FUTURE_INVALID_EXT
- XR_ERROR_FUTURE_PENDING_EXT
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The completion structure is XrCreateSpatialAnchorsCompletionML.

The XrCreateSpatialAnchorsCompletionML structure is defined as:

```
// Provided by XR_ML_spatial_anchors
typedef struct XrCreateSpatialAnchorsCompletionML {
    XrStructureType    type;
    void*              next;
    XrResult           futureResult;
    uint32_t           spaceCount;
    XrSpace*           spaces;
} XrCreateSpatialAnchorsCompletionML;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `futureResult` is the XrResult of the create spatial anchor operation.
- `spaceCount` is the number of elements in the `spaces` array. This **must** match the number of spaces requested to be created.
- `spaces` is an array of XrSpace values to populate with the results of the anchor creation. If creation failed, the XrSpace values **must** be set to XR_NULL_HANDLE.

Future Return Codes

`futureResult` values:

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_OUT_OF_MEMORY`
- `XR_ERROR_LIMIT_REACHED`
- `XR_ERROR_SPATIAL_ANCHORS_SPACE_NOT_LOCATABLE_ML`
- `XR_ERROR_SPATIAL_ANCHORS_OUT_OF_MAP_BOUNDS_ML`

Valid Usage (Implicit)

- The `XR_ML_spatial_anchors` extension **must** be enabled prior to using XrCreateSpatialAnchorsCompletionML
- `type` **must** be `XR_TYPE_CREATE_SPATIAL_ANCHORS_COMPLETION_ML`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- If `futureResult` is not `0`, `futureResult` **must** be a valid XrResult value
- If `spaceCount` is not `0`, `spaces` **must** be a pointer to an array of `spaceCount` XrSpace handles

12.181.4. Retrieve spatial anchor state

Spatial anchor state **can** be queried using xrGetSpatialAnchorStateML.

The xrGetSpatialAnchorStateML function is defined as:

```
// Provided by XR_ML_spatial_anchors
XrResult xrGetSpatialAnchorStateML(
    XrSpace                                     anchor,
    XrSpatialAnchorStateML*                     state);
```

Parameter Descriptions

- `anchor` is a handle to an XrSpace corresponding to a spatial anchor.
- `state` is a pointer to an XrSpatialAnchorStateML structure to populate with the spatial anchor state.

The runtime **must** return `XR_ERROR_VALIDATION_FAILURE` if the XrSpace is not an spatial anchor.

Valid Usage (Implicit)

- The `XR_ML_spatial_anchors` extension **must** be enabled prior to calling xrGetSpatialAnchorStateML
- `anchor` **must** be a valid XrSpace handle
- `state` **must** be a pointer to an XrSpatialAnchorStateML structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The XrSpatialAnchorStateML structure is defined as:

```
// Provided by XR_ML_spatial_anchors
typedef struct XrSpatialAnchorStateML {
    XrStructureType                type;
    void*                          next;
    XrSpatialAnchorConfidenceML    confidence;
} XrSpatialAnchorStateML;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain.
- confidence is the XrSpatialAnchorConfidenceML of the anchor.

Valid Usage (Implicit)

- The XR_ML_spatial_anchors extension **must** be enabled prior to using XrSpatialAnchorStateML
- type **must** be XR_TYPE_SPATIAL_ANCHOR_STATE_ML
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- If confidence is not 0, confidence **must** be a valid XrSpatialAnchorConfidenceML value

```
// Provided by XR_ML_spatial_anchors
typedef enum XrSpatialAnchorConfidenceML {
    XR_SPATIAL_ANCHOR_CONFIDENCE_LOW_ML = 0,
    XR_SPATIAL_ANCHOR_CONFIDENCE_MEDIUM_ML = 1,
    XR_SPATIAL_ANCHOR_CONFIDENCE_HIGH_ML = 2,
    XR_SPATIAL_ANCHOR_CONFIDENCE_MAX_ENUM_ML = 0x7FFFFFFF
} XrSpatialAnchorConfidenceML;
```

Enum	Description
XR_SPATIAL_ANCHOR_CONFIDENCE_LOW_ML	Low quality, this anchor can be expected to move significantly.
XR_SPATIAL_ANCHOR_CONFIDENCE_MEDIUM_ML	Medium quality, this anchor may move slightly.
XR_SPATIAL_ANCHOR_CONFIDENCE_HIGH_ML	High quality, this anchor is stable and suitable for digital content attachment.

12.181.5. Example code

This code shows how to create a spatial anchor in a synchronous manner. This **can** be changed to be completely asynchronous by changing the waitInfo.duration to 0 and checking it during the frame loop until the function returns ready.

```
XrInstance instance;                // previously initialized
XrSession session;                  // previously initialized

// these are setup to match the location and time
// of the position in the world you wish to set the
// spatial anchor for.
XrSpace baseSpace;                  // previously initialized
XrTime time;                        // previously initialized
XrPosef pose;                       // previously initialized

// Get function pointer for xrCreateSpatialAnchorsAsyncML
```

```
PFN_xrCreateSpatialAnchorsAsyncML xrCreateSpatialAnchorsAsyncML = nullptr;
CHK_XR(xrGetInstanceProcAddr(instance, "xrCreateSpatialAnchorsAsyncML",
                             reinterpret_cast<PFN_xrVoidFunction*>(
                             &xrCreateSpatialAnchorsAsyncML)))

// Get function pointer for xrCreateSpatialAnchorsCompleteML
PFN_xrCreateSpatialAnchorsCompleteML xrCreateSpatialAnchorsCompleteML = nullptr;
CHK_XR(xrGetInstanceProcAddr(instance, "xrCreateSpatialAnchorsCompleteML",
                             reinterpret_cast<PFN_xrVoidFunction*>(
                             &xrCreateSpatialAnchorsCompleteML)));


// Get function pointer for xrPollFutureEXT
PFN_xrPollFutureEXT xrPollFutureEXT = nullptr;
CHK_XR(xrGetInstanceProcAddr(instance, "xrPollFutureEXT",
                             reinterpret_cast<PFN_xrVoidFunction*>(
                             &xrPollFutureEXT)));

XrFutureEXT future {XR_NULL_FUTURE_EXT};
XrSpatialAnchorsCreateInfoFromPoseML createInfo {
    .type = XR_TYPE_SPATIAL_ANCHORS_CREATE_INFO_FROM_POSE_ML,
    .baseSpace = baseSpace,
    .poseInBaseSpace = pose,
    .time = time,
};

CHK_XR(xrCreateSpatialAnchorsAsyncML(session, reinterpret_cast<const
XrSpatialAnchorsCreateInfoBaseHeaderML*>(&createInfo), &future));

XrFuturePollInfoEXT pollInfo {
    .type = XR_TYPE_FUTURE_POLL_INFO_EXT,
    .future = future,
};
XrFuturePollResultEXT pollResult {
    .type = XR_TYPE_FUTURE_POLL_RESULT_EXT,
    .state = XR_FUTURE_STATE_PENDING_EXT,
};

while(pollResult.state==XR_FUTURE_STATE_PENDING_EXT) {
  // Ideally this check is done in your game loop
  // instead of busy waiting, this is just an
  // example.
  // If you do choose busy wait sleep to avoid
  // CPU overloading.
  CHK_XR(xrPollFutureEXT(instance, &pollInfo, &pollResult));
}

XrSpace anchor;
```

```
XrCreateSpatialAnchorsCompletionML completion {
    .type = XR_TYPE_CREATE_SPATIAL_ANCHORS_COMPLETION_ML,
    .spaceCount = 1,
    .spaces = &anchor,
};

CHK_XR(xrCreateSpatialAnchorsCompleteML(session, future, &completion));
// Check the future completion result as well!
CHK_XR(completion.futureResult);

// Now the anchor is usable!
```

12.181.6. New Commands

- xrCreateSpatialAnchorsAsyncML
- xrCreateSpatialAnchorsCompleteML
- xrGetSpatialAnchorStateML

12.181.7. New Structures

- XrCreateSpatialAnchorsCompletionML
- XrSpatialAnchorStateML
- XrSpatialAnchorsCreateInfoBaseHeaderML
- XrSpatialAnchorsCreateInfoFromPoseML

12.181.8. New Enums

- XrSpatialAnchorConfidenceML

12.181.9. New Enum Constants

- XR_ML_SPATIAL_ANCHORS_EXTENSION_NAME
- XR_ML_spatial_anchors_SPEC_VERSION
- Extending XrResult:
  - XR_ERROR_SPATIAL_ANCHORS_NOT_LOCALIZED_ML
  - XR_ERROR_SPATIAL_ANCHORS_OUT_OF_MAP_BOUNDS_ML
  - XR_ERROR_SPATIAL_ANCHORS_PERMISSION_DENIED_ML
  - XR_ERROR_SPATIAL_ANCHORS_SPACE_NOT_LOCATABLE_ML
- Extending XrStructureType:
  - XR_TYPE_CREATE_SPATIAL_ANCHORS_COMPLETION_ML

- XR_TYPE_SPATIAL_ANCHORS_CREATE_INFO_FROM_POSE_ML
- XR_TYPE_SPATIAL_ANCHOR_STATE_ML

12.181.10. Version history

- Revision 1, 2023-06-13 (Ron Bessems)
  - Initial extension description

12.182. XR_ML_spatial_anchors_storage

Name String

XR_ML_spatial_anchors_storage

Extension Type

Instance extension

Registered Extension Number

142

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

XR_EXT_future
and
XR_ML_spatial_anchors

Last Modified Date

2023-08-24

Contributors

Ron Bessems, Magic Leap
Karthik Kadappan, Magic Leap

12.182.1. Overview

This extension allows spatial anchors created by XR_ML_spatial_anchors to be persisted beyond the head pose session.

Spatial anchor management is closely tied to the selected mapping mode on the device. The modes are mutually exclusive and affect the functionality of these APIs. The available mapping modes are:

On-Device Mode

A persistent mode in which anchors are persisted locally and will be available across multiple sessions when localized to the same localization map in which they were published.

AR Cloud Mode

A persistent mode in which anchors are persisted in the cloud environment and will be available across multiple sessions and across multiple devices that are localized to the same localization map in which they were published.

For more details on mapping modes refer to the `XR_ML_localization_map` extension.



Permissions

Android applications **must** have the com.magicleap.permission.SPATIAL_ANCHOR permission listed in their manifest to use this extension. (protection level: normal)

12.182.2. Storage

```
// Provided by XR_ML_spatial_anchors_storage
XR_DEFINE_HANDLE(XrSpatialAnchorsStorageML)
```

The XrSpatialAnchorsStorageML handle maintains the connection with the backend service. This **may** be on device or in cloud depending on the localization map that is active. Use the `XR_ML_localization_map` extension to deduce the current mode if the application needs to know this.

The XrSpatialAnchorsStorageML handle represents the resources for storing spatial anchors.

The xrCreateSpatialAnchorsStorageML function is defined as:

```
// Provided by XR_ML_spatial_anchors_storage
XrResult xrCreateSpatialAnchorsStorageML(
    XrSession                                   session,
    const XrSpatialAnchorsCreateStorageInfoML*  createInfo,
    XrSpatialAnchorsStorageML*                  storage);
```

Parameter Descriptions

- `session` is a handle to an XrSession previously created with xrCreateSession.
- `createInfo` is a pointer to an XrSpatialAnchorsCreateInfoBaseHeaderML derived structure.
- `storage` is a pointer to an XrSpatialAnchorsStorageML where the created storage is returned.

The xrCreateSpatialAnchorsStorageML function is used to create a XrSpatialAnchorsStorageML.

Valid Usage (Implicit)

- The `XR_ML_spatial_anchors_storage` extension **must** be enabled prior to calling xrCreateSpatialAnchorsStorageML
- `session` **must** be a valid XrSession handle
- `createInfo` **must** be a pointer to a valid XrSpatialAnchorsCreateStorageInfoML structure
- `storage` **must** be a pointer to an XrSpatialAnchorsStorageML handle

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_SPATIAL_ANCHORS_PERMISSION_DENIED_ML
- XR_ERROR_VALIDATION_FAILURE

The XrSpatialAnchorsCreateStorageInfoML structure is defined as:

```
// Provided by XR_ML_spatial_anchors_storage
typedef struct XrSpatialAnchorsCreateStorageInfoML {
    XrStructureType    type;
    const void*        next;
} XrSpatialAnchorsCreateStorageInfoML;
```

Currently no extra information is needed to create this structure.

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.

Valid Usage (Implicit)

- The `XR_ML_spatial_anchors_storage` extension **must** be enabled prior to using XrSpatialAnchorsCreateStorageInfoML
- `type` **must** be `XR_TYPE_SPATIAL_ANCHORS_CREATE_STORAGE_INFO_ML`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

The xrDestroySpatialAnchorsStorageML function is defined as:

```
// Provided by XR_ML_spatial_anchors_storage
XrResult xrDestroySpatialAnchorsStorageML(
    XrSpatialAnchorsStorageML                   storage);
```

Parameter Descriptions

- `storage` is the XrSpatialAnchorsStorageML to destroy.

Valid Usage (Implicit)

- The `XR_ML_spatial_anchors_storage` extension **must** be enabled prior to calling xrDestroySpatialAnchorsStorageML
- `storage` **must** be a valid XrSpatialAnchorsStorageML handle

Thread Safety

- Access to `storage`, and any child handles, **must** be externally synchronized

Return Codes

Success

- XR_SUCCESS

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE

12.182.3. Query Stored Anchors

To find out which spatial anchors are stored near a certain XrPosef, this extension provides a query system. This function is asynchronous and uses the XR_EXT_future extension.

The xrQuerySpatialAnchorsAsyncML function is defined as:

```
// Provided by XR_ML_spatial_anchors_storage
XrResult xrQuerySpatialAnchorsAsyncML(
    XrSpatialAnchorsStorageML                    storage,
    const XrSpatialAnchorsQueryInfoBaseHeaderML* queryInfo,
    XrFutureEXT*                                 future);
```

Parameter Descriptions

- storage is the XrSpatialAnchorsStorageML to use.
- queryInfo is the XrSpatialAnchorsQueryInfoBaseHeaderML structure used to define the query. XrSpatialAnchorsQueryInfoRadiusML allows querying around a pose.
- future is a pointer the created XrFutureEXT.

If the space was not locatable during the query the runtime **must** return XR_ERROR_SPACE_NOT_LOCATABLE_EXT in XrSpatialAnchorsQueryCompletionML::futureResult.

Valid Usage (Implicit)

- The `XR_ML_spatial_anchors_storage` extension **must** be enabled prior to calling xrQuerySpatialAnchorsAsyncML
- `storage` **must** be a valid XrSpatialAnchorsStorageML handle
- `queryInfo` **must** be a pointer to a valid XrSpatialAnchorsQueryInfoBaseHeaderML-based structure. See also: XrSpatialAnchorsQueryInfoRadiusML
- `future` **must** be a pointer to an `XrFutureEXT` value

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_LIMIT_REACHED`
- `XR_ERROR_OUT_OF_MEMORY`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_SPATIAL_ANCHORS_SPACE_NOT_LOCATABLE_ML`
- `XR_ERROR_TIME_INVALID`
- `XR_ERROR_VALIDATION_FAILURE`

The XrSpatialAnchorsQueryInfoRadiusML structure is defined as:

```
// Provided by XR_ML_spatial_anchors_storage
typedef struct XrSpatialAnchorsQueryInfoRadiusML {
    XrStructureType    type;
    const void*        next;
    XrSpace            baseSpace;
    XrVector3f         center;
    XrTime             time;
    float              radius;
} XrSpatialAnchorsQueryInfoRadiusML;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `baseSpace` is the XrSpace to create the anchor relative to.
- `center` is the center of the query relative to the baseSpace.
- `time` is the time at which the pose **should** be resolved.
- `radius` is the radius of the search area in meters.

Valid Usage (Implicit)

- The `XR_ML_spatial_anchors_storage` extension **must** be enabled prior to using XrSpatialAnchorsQueryInfoRadiusML
- `type` **must** be `XR_TYPE_SPATIAL_ANCHORS_QUERY_INFO_RADIUS_ML`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `baseSpace` **must** be a valid XrSpace handle

The XrSpatialAnchorsQueryInfoBaseHeaderML structure is defined as:

```
// Provided by XR_ML_spatial_anchors_storage
typedef struct XrSpatialAnchorsQueryInfoBaseHeaderML {
    XrStructureType    type;
    const void*        next;
} XrSpatialAnchorsQueryInfoBaseHeaderML;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.

Valid Usage (Implicit)

- The `XR_ML_spatial_anchors_storage` extension **must** be enabled prior to using XrSpatialAnchorsQueryInfoBaseHeaderML
- `type` **must** be `XR_TYPE_SPATIAL_ANCHORS_QUERY_INFO_RADIUS_ML`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

The xrQuerySpatialAnchorsCompleteML function is defined as:

```
// Provided by XR_ML_spatial_anchors_storage
XrResult xrQuerySpatialAnchorsCompleteML(
    XrSpatialAnchorsStorageML                   storage,
    XrFutureEXT                                 future,
    XrSpatialAnchorsQueryCompletionML*          completion);
```

Parameter Descriptions

- `storage` is the XrSpatialAnchorsStorageML used to start the query.
- `future` is the `XrFutureEXT` of the future to complete.
- `completion` is a pointer to an XrSpatialAnchorsQueryCompletionML structure containing the result of the operation.

Once the `XrFutureEXT` has completed xrQuerySpatialAnchorsCompleteML **must** be called to retrieve the XrUuidEXT values of the found anchors.

Valid Usage (Implicit)

- The `XR_ML_spatial_anchors_storage` extension **must** be enabled prior to calling xrQuerySpatialAnchorsCompleteML
- `storage` **must** be a valid XrSpatialAnchorsStorageML handle
- `completion` **must** be a pointer to an XrSpatialAnchorsQueryCompletionML structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_FUTURE_INVALID_EXT
- XR_ERROR_FUTURE_PENDING_EXT
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The XrSpatialAnchorsQueryCompletionML structure is defined as:

```
// Provided by XR_ML_spatial_anchors_storage
typedef struct XrSpatialAnchorsQueryCompletionML {
    XrStructureType    type;
    void*              next;
    XrResult           futureResult;
    uint32_t           uuidCapacityInput;
    uint32_t           uuidCountOutput;
    XrUuidEXT*         uuids;
} XrSpatialAnchorsQueryCompletionML;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `futureResult` is the XrResult of the asynchronous operation.
- `uuidCapacityInput` is the capacity of the `uuids` array, or 0 to indicate a request to retrieve the required capacity.
- `uuidCountOutput` is the number of elements populated in `uuids`, or the required capacity in the case that `uuidCapacityInput` is insufficient.
- `uuids` is an array of XrUuidEXT.

Future Return Codes

`futureResult` values:

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_OUT_OF_MEMORY`
- `XR_ERROR_LIMIT_REACHED`
- `XR_ERROR_VALIDATION_FAILURE`
- `XR_ERROR_SPACE_NOT_LOCATABLE_EXT`
- `XR_ERROR_SPATIAL_ANCHORS_SPACE_NOT_LOCATABLE_ML`
- `XR_ERROR_SPATIAL_ANCHORS_NOT_LOCALIZED_ML`

Valid Usage (Implicit)

- The `XR_ML_spatial_anchors_storage` extension **must** be enabled prior to using XrSpatialAnchorsQueryCompletionML
- `type` **must** be `XR_TYPE_SPATIAL_ANCHORS_QUERY_COMPLETION_ML`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- If `futureResult` is not `0`, `futureResult` **must** be a valid XrResult value
- If `uuidCapacityInput` is not `0`, `uuids` **must** be a pointer to an array of `uuidCapacityInput` XrUuidEXT structures

12.182.4. Publish Anchors

If the application needs to persist an anchor beyond the current head pose session it **should** publish the anchor. Publishing is an asynchronous operation and **can** publish multiple anchors at the same time.

The xrPublishSpatialAnchorsAsyncML function is defined as:

```
// Provided by XR_ML_spatial_anchors_storage
XrResult xrPublishSpatialAnchorsAsyncML(
    XrSpatialAnchorsStorageML                   storage,
    const XrSpatialAnchorsPublishInfoML*        publishInfo,
    XrFutureEXT*                                future);
```

Parameter Descriptions

- `storage` is the XrSpatialAnchorsStorageML to use.
- `publishInfo` is the XrSpatialAnchorsPublishInfoML structure use to specify the anchors to publish.
- `future` is a pointer the created `XrFutureEXT`.

Valid Usage (Implicit)

- The `XR_ML_spatial_anchors_storage` extension **must** be enabled prior to calling xrPublishSpatialAnchorsAsyncML
- `storage` **must** be a valid XrSpatialAnchorsStorageML handle
- `publishInfo` **must** be a pointer to a valid XrSpatialAnchorsPublishInfoML structure
- `future` **must** be a pointer to an `XrFutureEXT` value

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_LIMIT_REACHED`
- `XR_ERROR_OUT_OF_MEMORY`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_VALIDATION_FAILURE`

The XrSpatialAnchorsPublishInfoML structure is defined as:

```
// Provided by XR_ML_spatial_anchors_storage
typedef struct XrSpatialAnchorsPublishInfoML {
    XrStructureType    type;
    const void*        next;
    uint32_t           anchorCount;
    const XrSpace*     anchors;
    uint64_t           expiration;
} XrSpatialAnchorsPublishInfoML;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `anchorCount` is the number of anchors in the anchors array.
- `anchors` is an array of XrSpace.
- `expiration` is an `uint64_t`, is the time in **seconds** since epoch after which these anchors **may** expire. The system **may** retain the anchors longer but **should** retain the anchors at least until this timestamp. Use 0 or XR_INFINITE_DURATION to create an anchor that does not expire.

Valid Usage (Implicit)

- The `XR_ML_spatial_anchors_storage` extension **must** be enabled prior to using XrSpatialAnchorsPublishInfoML
- `type` **must** be `XR_TYPE_SPATIAL_ANCHORS_PUBLISH_INFO_ML`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `anchors` **must** be a pointer to an array of `anchorCount` valid XrSpace handles
- The `anchorCount` parameter **must** be greater than `0`

Once the `XrFutureEXT` has completed xrPublishSpatialAnchorsCompleteML **must** be called to retrieve the XrUuidEXT values of the published anchors.

The xrPublishSpatialAnchorsCompleteML function is defined as:

```
// Provided by XR_ML_spatial_anchors_storage
XrResult xrPublishSpatialAnchorsCompleteML(
    XrSpatialAnchorsStorageML                   storage,
    XrFutureEXT                                 future,
    XrSpatialAnchorsPublishCompletionML*        completion);
```

Parameter Descriptions

- `storage` is the XrSpatialAnchorsStorageML used to start the publish operation.
- `future` is the `XrFutureEXT` of the future to complete.
- `completion` is a pointer to a XrSpatialAnchorsPublishCompletionML structure containing the result of the operation.

Valid Usage (Implicit)

- The XR_ML_spatial_anchors_storage extension **must** be enabled prior to calling xrPublishSpatialAnchorsCompleteML
- `storage` **must** be a valid XrSpatialAnchorsStorageML handle
- `completion` **must** be a pointer to an XrSpatialAnchorsPublishCompletionML structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_FUTURE_INVALID_EXT
- XR_ERROR_FUTURE_PENDING_EXT
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The XrSpatialAnchorsPublishCompletionML structure is defined as:

```
// Provided by XR_ML_spatial_anchors_storage
typedef struct XrSpatialAnchorsPublishCompletionML {
    XrStructureType    type;
    void*              next;
    XrResult           futureResult;
    uint32_t           uuidCount;
    XrUuidEXT*         uuids;
} XrSpatialAnchorsPublishCompletionML;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `futureResult` is the XrResult of the asynchronous operation.
- `uuidCount` is the capacity of the `uuids` array and **must** match the number of anchors passed to xrPublishSpatialAnchorsAsyncML
- `uuids` is an array of XrUuidEXT.

Future Return Codes

`futureResult` values:

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_OUT_OF_MEMORY`
- `XR_ERROR_LIMIT_REACHED`
- `XR_ERROR_VALIDATION_FAILURE`
- `XR_ERROR_SPATIAL_ANCHORS_NOT_LOCALIZED_ML`
- `XR_ERROR_SPATIAL_ANCHORS_OUT_OF_MAP_BOUNDS_ML`

Valid Usage (Implicit)

- The `XR_ML_spatial_anchors_storage` extension **must** be enabled prior to using XrSpatialAnchorsPublishCompletionML
- `type` **must** be `XR_TYPE_SPATIAL_ANCHORS_PUBLISH_COMPLETION_ML`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain. See also: XrSpatialAnchorsPublishCompletionDetailsML
- `futureResult` **must** be a valid XrResult value
- `uuids` **must** be a pointer to an array of `uuidCount` XrUuidEXT structures
- The `uuidCount` parameter **must** be greater than `0`

The XrSpatialAnchorsPublishCompletionDetailsML structure is defined as:

```
// Provided by XR_ML_spatial_anchors_storage
typedef struct XrSpatialAnchorsPublishCompletionDetailsML {
    XrStructureType                       type;
    void*                                 next;
    uint32_t                              resultCount;
    XrSpatialAnchorCompletionResultML*    results;
} XrSpatialAnchorsPublishCompletionDetailsML;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `resultCount` is the capacity of the `results` array and **must** match the number of anchors passed to xrPublishSpatialAnchorsAsyncML.
- `results` is the per UUID status of the publish operation. The order of results in this array correspond to the async request order.

Valid Usage (Implicit)

- The `XR_ML_spatial_anchors_storage` extension **must** be enabled prior to using XrSpatialAnchorsPublishCompletionDetailsML
- `type` **must** be `XR_TYPE_SPATIAL_ANCHORS_PUBLISH_COMPLETION_DETAILS_ML`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `results` **must** be a pointer to an array of `resultCount` XrSpatialAnchorCompletionResultML structures
- The `resultCount` parameter **must** be greater than `0`

The XrSpatialAnchorCompletionResultML structure is defined as:

```
// Provided by XR_ML_spatial_anchors_storage
typedef struct XrSpatialAnchorCompletionResultML {
    XrUuidEXT    uuid;
    XrResult     result;
} XrSpatialAnchorCompletionResultML;
```

Member Descriptions

- `uuid` is the UUID of the anchor.
- `result` is the result of the operation.

Anchor Operation Result Codes

`result` values:

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_OUT_OF_MEMORY`
- `XR_ERROR_LIMIT_REACHED`
- `XR_ERROR_VALIDATION_FAILURE`
- `XR_ERROR_SPATIAL_ANCHORS_NOT_LOCALIZED_ML`
- `XR_ERROR_SPATIAL_ANCHORS_OUT_OF_MAP_BOUNDS_ML`

Valid Usage (Implicit)

- The `XR_ML_spatial_anchors_storage` extension **must** be enabled prior to using XrSpatialAnchorCompletionResultML
- `result` **must** be a valid XrResult value

12.182.5. Delete Published Anchors

To delete anchors from the storage, xrDeleteSpatialAnchorsAsyncML **can** be used.

The xrDeleteSpatialAnchorsAsyncML function is defined as:

```
// Provided by XR_ML_spatial_anchors_storage
XrResult xrDeleteSpatialAnchorsAsyncML(
    XrSpatialAnchorsStorageML                   storage,
    const XrSpatialAnchorsDeleteInfoML*         deleteInfo,
    XrFutureEXT*                                future);
```

Parameter Descriptions

- `storage` is the XrSpatialAnchorsStorageML to use.
- `deleteInfo` is the XrSpatialAnchorsDeleteInfoML structure used to specify the anchors to delete.
- `future` is a pointer the created `XrFutureEXT`.

Valid Usage (Implicit)

- The `XR_ML_spatial_anchors_storage` extension **must** be enabled prior to calling xrDeleteSpatialAnchorsAsyncML
- `storage` **must** be a valid XrSpatialAnchorsStorageML handle
- `deleteInfo` **must** be a pointer to a valid XrSpatialAnchorsDeleteInfoML structure
- `future` **must** be a pointer to an `XrFutureEXT` value

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_LIMIT_REACHED`
- `XR_ERROR_OUT_OF_MEMORY`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_VALIDATION_FAILURE`

The XrSpatialAnchorsDeleteInfoML structure is defined as:

```
// Provided by XR_ML_spatial_anchors_storage
typedef struct XrSpatialAnchorsDeleteInfoML {
    XrStructureType     type;
    const void*         next;
    uint32_t            uuidCount;
    const XrUuidEXT*    uuids;
} XrSpatialAnchorsDeleteInfoML;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `uuidCount` is the number of UUIDs in the `uuids` array.
- `uuids` is an array of XrUuidEXT.

Valid Usage (Implicit)

- The `XR_ML_spatial_anchors_storage` extension **must** be enabled prior to using XrSpatialAnchorsDeleteInfoML
- `type` **must** be `XR_TYPE_SPATIAL_ANCHORS_DELETE_INFO_ML`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `uuids` **must** be a pointer to an array of `uuidCount` XrUuidEXT structures
- The `uuidCount` parameter **must** be greater than `0`

Once the `XrFutureEXT` has completed xrPublishSpatialAnchorsCompleteML **must** be called to retrieve the status of the delete operation.

The xrDeleteSpatialAnchorsCompleteML function is defined as:

```
// Provided by XR_ML_spatial_anchors_storage
XrResult xrDeleteSpatialAnchorsCompleteML(
    XrSpatialAnchorsStorageML                   storage,
    XrFutureEXT                                 future,
    XrSpatialAnchorsDeleteCompletionML*         completion);
```

Member Descriptions

- `storage` is the XrSpatialAnchorsStorageML used to start the publish operation.
- `future` is the `XrFutureEXT` of the future to complete.
- `completion` is a pointer to a XrSpatialAnchorsDeleteCompletionML structure containing the result of the operation.

Valid Usage (Implicit)

- The `XR_ML_spatial_anchors_storage` extension **must** be enabled prior to calling xrDeleteSpatialAnchorsCompleteML
- `storage` **must** be a valid XrSpatialAnchorsStorageML handle
- `completion` **must** be a pointer to an XrSpatialAnchorsDeleteCompletionML structure

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_FUTURE_INVALID_EXT`
- `XR_ERROR_FUTURE_PENDING_EXT`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_LIMIT_REACHED`
- `XR_ERROR_OUT_OF_MEMORY`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_VALIDATION_FAILURE`

The XrSpatialAnchorsDeleteCompletionML structure is defined as:

```
// Provided by XR_ML_spatial_anchors_storage
typedef struct XrSpatialAnchorsDeleteCompletionML {
    XrStructureType    type;
    void*              next;
    XrResult           futureResult;
} XrSpatialAnchorsDeleteCompletionML;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `futureResult` is the XrResult of the asynchronous operation.

Future Return Codes

`futureResult` values:

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_SPATIAL_ANCHORS_ANCHOR_NOT_FOUND_ML`
- `XR_ERROR_SPATIAL_ANCHORS_NOT_LOCALIZED_ML`

Valid Usage (Implicit)

- The `XR_ML_spatial_anchors_storage` extension **must** be enabled prior to using XrSpatialAnchorsDeleteCompletionML
- `type` **must** be `XR_TYPE_SPATIAL_ANCHORS_DELETE_COMPLETION_ML`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain. See also: XrSpatialAnchorsDeleteCompletionDetailsML
- `futureResult` **must** be a valid XrResult value

The XrSpatialAnchorsDeleteCompletionDetailsML structure is defined as:

```
// Provided by XR_ML_spatial_anchors_storage
typedef struct XrSpatialAnchorsDeleteCompletionDetailsML {
    XrStructureType                         type;
    void*                                   next;
    uint32_t                                resultCount;
    XrSpatialAnchorCompletionResultML*      results;
} XrSpatialAnchorsDeleteCompletionDetailsML;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `resultCount` is the capacity of the `results` array and **must** match the number of anchors passed to xrDeleteSpatialAnchorsAsyncML
- `results` is the per UUID status of the delete operation. The order of results in this array correspond to the async request order.

Valid Usage (Implicit)

- The `XR_ML_spatial_anchors_storage` extension **must** be enabled prior to using XrSpatialAnchorsDeleteCompletionDetailsML
- `type` **must** be `XR_TYPE_SPATIAL_ANCHORS_DELETE_COMPLETION_DETAILS_ML`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `results` **must** be a pointer to an array of `resultCount` XrSpatialAnchorCompletionResultML structures
- The `resultCount` parameter **must** be greater than `0`

12.182.6. Update Published Anchors Expiration

To update the expiration time on anchors xrUpdateSpatialAnchorsExpirationAsyncML **can** be used.

The xrUpdateSpatialAnchorsExpirationAsyncML function is defined as:

```
// Provided by XR_ML_spatial_anchors_storage
XrResult xrUpdateSpatialAnchorsExpirationAsyncML(
    XrSpatialAnchorsStorageML                    storage,
    const XrSpatialAnchorsUpdateExpirationInfoML* updateInfo,
    XrFutureEXT*                                 future);
```

Parameter Descriptions

- `storage` is the XrSpatialAnchorsStorageML to use.
- `updateInfo` is a pointer to XrSpatialAnchorsUpdateExpirationInfoML structure used to specify the anchors to update.
- `future` is a pointer to the created `XrFutureEXT`.

Valid Usage (Implicit)

- The `XR_ML_spatial_anchors_storage` extension **must** be enabled prior to calling xrUpdateSpatialAnchorsExpirationAsyncML
- `storage` **must** be a valid XrSpatialAnchorsStorageML handle
- `updateInfo` **must** be a pointer to a valid XrSpatialAnchorsUpdateExpirationInfoML structure
- `future` **must** be a pointer to an `XrFutureEXT` value

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_OUT_OF_MEMORY`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_VALIDATION_FAILURE`

The XrSpatialAnchorsUpdateExpirationInfoML structure is defined as:

```
// Provided by XR_ML_spatial_anchors_storage
typedef struct XrSpatialAnchorsUpdateExpirationInfoML {
    XrStructureType     type;
    const void*         next;
    uint32_t            uuidCount;
    const XrUuidEXT*    uuids;
    uint64_t            expiration;
} XrSpatialAnchorsUpdateExpirationInfoML;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `uuidCount` is the number of anchors in the anchors array.
- `uuids` is an array of XrUuidEXT.
- `expiration` is an `uint64_t`, is the time in seconds since epoch after which these anchors **may** expire.

Valid Usage (Implicit)

- The `XR_ML_spatial_anchors_storage` extension **must** be enabled prior to using XrSpatialAnchorsUpdateExpirationInfoML
- `type` **must** be `XR_TYPE_SPATIAL_ANCHORS_UPDATE_EXPIRATION_INFO_ML`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `uuids` **must** be a pointer to an array of `uuidCount` XrUuidEXT structures
- The `uuidCount` parameter **must** be greater than `0`

Once the `XrFutureEXT` has completed xrUpdateSpatialAnchorsExpirationCompleteML **must** be called to retrieve the status of the publish operation.

The xrUpdateSpatialAnchorsExpirationCompleteML function is defined as:

```
// Provided by XR_ML_spatial_anchors_storage
XrResult xrUpdateSpatialAnchorsExpirationCompleteML(
    XrSpatialAnchorsStorageML                    storage,
    XrFutureEXT                                  future,
    XrSpatialAnchorsUpdateExpirationCompletionML* completion);
```

Member Descriptions

- `storage` is the XrSpatialAnchorsStorageML used to start the publish operation.
- `future` is the `XrFutureEXT` of the future to complete.
- `completion` is a pointer to a XrSpatialAnchorsUpdateExpirationCompletionML structure containing the result of the operation.

Valid Usage (Implicit)

- The `XR_ML_spatial_anchors_storage` extension **must** be enabled prior to calling xrUpdateSpatialAnchorsExpirationCompleteML
- `storage` **must** be a valid XrSpatialAnchorsStorageML handle
- `completion` **must** be a pointer to an XrSpatialAnchorsUpdateExpirationCompletionML structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_FUTURE_INVALID_EXT
- XR_ERROR_FUTURE_PENDING_EXT
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The XrSpatialAnchorsUpdateExpirationCompletionML structure is defined as:

```
// Provided by XR_ML_spatial_anchors_storage
typedef struct XrSpatialAnchorsUpdateExpirationCompletionML {
    XrStructureType    type;
    void*              next;
    XrResult           futureResult;
} XrSpatialAnchorsUpdateExpirationCompletionML;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain.
- futureResult is the XrResult of the asynchronous operation.

Future Return Codes

`futureResult` values:

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_SPATIAL_ANCHORS_ANCHOR_NOT_FOUND_ML`
- `XR_ERROR_SPATIAL_ANCHORS_NOT_LOCALIZED_ML`

Valid Usage (Implicit)

- The `XR_ML_spatial_anchors_storage` extension **must** be enabled prior to using XrSpatialAnchorsUpdateExpirationCompletionML
- `type` **must** be `XR_TYPE_SPATIAL_ANCHORS_UPDATE_EXPIRATION_COMPLETION_ML`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain. See also: XrSpatialAnchorsUpdateExpirationCompletionDetailsML
- `futureResult` **must** be a valid XrResult value

The XrSpatialAnchorsUpdateExpirationCompletionDetailsML structure is defined as:

```
// Provided by XR_ML_spatial_anchors_storage
typedef struct XrSpatialAnchorsUpdateExpirationCompletionDetailsML {
    XrStructureType                       type;
    void*                                 next;
    uint32_t                              resultCount;
    XrSpatialAnchorCompletionResultML*    results;
} XrSpatialAnchorsUpdateExpirationCompletionDetailsML;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `resultCount` is the capacity of the `results` array and **must** match the number of anchors passed to xrUpdateSpatialAnchorsExpirationAsyncML
- `results` is the per UUID status of the update operation. The order of anchors in this array **must** match the async request order.

Valid Usage (Implicit)

- The `XR_ML_spatial_anchors_storage` extension **must** be enabled prior to using XrSpatialAnchorsUpdateExpirationCompletionDetailsML
- `type` **must** be `XR_TYPE_SPATIAL_ANCHORS_UPDATE_EXPIRATION_COMPLETION_DETAILS_ML`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `results` **must** be a pointer to an array of `resultCount` XrSpatialAnchorCompletionResultML structures
- The `resultCount` parameter **must** be greater than `0`

12.182.7. Create Spatial Anchors from Storage

Spatial anchors **can** be created from the storage XrUuidEXT by passing the XrSpatialAnchorsCreateInfoFromUuidsML structure to xrCreateSpatialAnchorsAsyncML.

The XrSpatialAnchorsCreateInfoFromUuidsML structure is defined as:

```
// Provided by XR_ML_spatial_anchors_storage
typedef struct XrSpatialAnchorsCreateInfoFromUuidsML {
    XrStructureType              type;
    const void*                  next;
    XrSpatialAnchorsStorageML    storage;
    uint32_t                     uuidCount;
    const XrUuidEXT*             uuids;
} XrSpatialAnchorsCreateInfoFromUuidsML;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `storage` is the XrSpatialAnchorsStorageML to use.
- `uuidCount` is the number of elements in the uuids array.
- `uuids` is pointer to an array of XrUuidEXT.

The XrSpace handle or handles returned via XrCreateSpatialAnchorsCompletionML::`spaces` **must** be in the same order as `uuids`.

The XrCreateSpatialAnchorsCompletionML::`spaceCount` field **must** match `uuidCount`. If not the runtime **must** return `XR_ERROR_VALIDATION_FAILURE` in XrCreateSpatialAnchorsCompletionML::`futureResult`.

If an anchor with a given UUID is not found, the runtime **must** return XR_NULL_HANDLE for the corresponding XrSpace handle(s) and return `XR_SUCCESS` in XrCreateSpatialAnchorsCompletionML ::`futureResult`.

Valid Usage (Implicit)

- The `XR_ML_spatial_anchors_storage` extension **must** be enabled prior to using XrSpatialAnchorsCreateInfoFromUuidsML
- `type` **must** be `XR_TYPE_SPATIAL_ANCHORS_CREATE_INFO_FROM_UUIDS_ML`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `storage` **must** be a valid XrSpatialAnchorsStorageML handle
- `uuids` **must** be a pointer to an array of `uuidCount` XrUuidEXT structures
- The `uuidCount` parameter **must** be greater than `0`

12.182.8. Examples

This example shows how to persist a list of anchors.

```
XrInstance instance;                // previously initialized
XrSession session;                  // previously initialized

XrSpace anchor1; // previously initialized
XrSpace anchor2; // previously initialized

std::vector<XrSpace> anchors{anchor1, anchor2};
```

```
XrSpatialAnchorsCreateStorageInfoML storageCreateInfo {
    .type = XR_TYPE_SPATIAL_ANCHORS_CREATE_STORAGE_INFO_ML,
};
XrSpatialAnchorsStorageML storage;

CHK_XR(xrCreateSpatialAnchorsStorageML(session, &storageCreateInfo, &storage));

XrSpatialAnchorsPublishInfoML publishInfo {
    .type = XR_TYPE_SPATIAL_ANCHORS_PUBLISH_INFO_ML,
    .anchorCount = static_cast<uint32_t>(anchors.size()),
    .anchors = anchors.data(),
    .expiration = 0,
};

XrFutureEXT future {XR_NULL_FUTURE_EXT};
CHK_XR(xrPublishSpatialAnchorsAsyncML(storage, &publishInfo, &future));

XrFuturePollInfoEXT pollInfo {
    .type = XR_TYPE_FUTURE_POLL_INFO_EXT,
    .future = future,
};
XrFuturePollResultEXT pollResult {
    .type = XR_TYPE_FUTURE_POLL_RESULT_EXT,
    .state = XR_FUTURE_STATE_PENDING_EXT,
};

// Ideally this is done once in your
// game loop instead of a busy wait.
while(pollResult.state==XR_FUTURE_STATE_PENDING_EXT) {
  CHK_XR(xrPollFutureEXT(instance, &pollInfo, &pollResult));
}

std::vector<XrUuidEXT> uuids(anchors.size());
XrSpatialAnchorsPublishCompletionML completion {
    .type = XR_TYPE_SPATIAL_ANCHORS_PUBLISH_COMPLETION_ML,
    .uuidCount = static_cast<uint32_t>(uuids.size()),
    .uuids = uuids.data(),
};
CHK_XR(xrPublishSpatialAnchorsCompleteML(storage, future, &completion));
CHK_XR(completion.futureResult);

// completion.uuid will now contain the UUID of the corresponding spatial anchors.

CHK_XR(xrDestroySpatialAnchorsStorageML(storage));
```

This example shows how to query for anchors.

```
XrTime currentTime;                // previously initialized
XrVector3f center;                      // previously initialized

XrSpatialAnchorsCreateStorageInfoML storageCreateInfo {
    .type = XR_TYPE_SPATIAL_ANCHORS_CREATE_STORAGE_INFO_ML,
};
XrSpatialAnchorsStorageML storage;

CHK_XR(xrCreateSpatialAnchorsStorageML(session, &storageCreateInfo, &storage));

// set up a query around a previously initialized center position with a radius of 10
meters.
XrSpatialAnchorsQueryInfoRadiusML queryInfo {
    .type = XR_TYPE_SPATIAL_ANCHORS_QUERY_INFO_RADIUS_ML,
    .baseSpace = viewSpace, // using viewspace, but can: be any space.
    .center = center,
    .time = currentTime,
    .radius = 10.0f,
};

XrFutureEXT future {XR_NULL_FUTURE_EXT};
CHK_XR(xrQuerySpatialAnchorsAsyncML(storage, reinterpret_cast
<XrSpatialAnchorsQueryInfoBaseHeaderML*>(&queryInfo), &future));

XrFuturePollInfoEXT pollInfo {
    .type = XR_TYPE_FUTURE_POLL_INFO_EXT,
    .future = future,
};
XrFuturePollResultEXT pollResult {
    .type = XR_TYPE_FUTURE_POLL_RESULT_EXT,
    .state = XR_FUTURE_STATE_PENDING_EXT,
};

// Ideally this is done once in your
// frame loop instead of a busy wait.
while(pollResult.state==XR_FUTURE_STATE_PENDING_EXT) {
  CHK_XR(xrPollFutureEXT(instance, &pollInfo, &pollResult));
}

XrSpatialAnchorsQueryCompletionML completion {
    .type = XR_TYPE_SPATIAL_ANCHORS_QUERY_COMPLETION_ML,
};
CHK_XR(xrQuerySpatialAnchorsCompleteML(storage, future, &completion));
CHK_XR(completion.futureResult);

std::vector<XrUuidEXT> uuids(completion.uuidCountOutput);
completion.uuidCapacityInput = static_cast<uint32_t>(uuids.size());
completion.uuids = uuids.data();
```

```
CHK_XR(xrQuerySpatialAnchorsCompleteML(storage, future, &completion));

// completion.uuid will now contain the UUID of the corresponding spatial anchors.

CHK_XR(xrDestroySpatialAnchorsStorageML(storage));
```

12.182.9. New Object Types

- XrSpatialAnchorsStorageML

12.182.10. New Commands

- xrCreateSpatialAnchorsStorageML
- xrDeleteSpatialAnchorsAsyncML
- xrDeleteSpatialAnchorsCompleteML
- xrDestroySpatialAnchorsStorageML
- xrPublishSpatialAnchorsAsyncML
- xrPublishSpatialAnchorsCompleteML
- xrQuerySpatialAnchorsAsyncML
- xrQuerySpatialAnchorsCompleteML
- xrUpdateSpatialAnchorsExpirationAsyncML
- xrUpdateSpatialAnchorsExpirationCompleteML

12.182.11. New Structures

- XrSpatialAnchorCompletionResultML
- XrSpatialAnchorsCreateInfoFromUuidsML
- XrSpatialAnchorsCreateStorageInfoML
- XrSpatialAnchorsDeleteCompletionML
- XrSpatialAnchorsDeleteInfoML
- XrSpatialAnchorsPublishCompletionML
- XrSpatialAnchorsPublishInfoML
- XrSpatialAnchorsQueryCompletionML
- XrSpatialAnchorsQueryInfoBaseHeaderML
- XrSpatialAnchorsQueryInfoRadiusML
- XrSpatialAnchorsUpdateExpirationCompletionML

- XrSpatialAnchorsUpdateExpirationInfoML
- Extending XrSpatialAnchorsDeleteCompletionML:
  - XrSpatialAnchorsDeleteCompletionDetailsML
- Extending XrSpatialAnchorsPublishCompletionML:
  - XrSpatialAnchorsPublishCompletionDetailsML
- Extending XrSpatialAnchorsUpdateExpirationCompletionML:
  - XrSpatialAnchorsUpdateExpirationCompletionDetailsML

12.182.12. New Enum Constants

- `XR_ML_SPATIAL_ANCHORS_STORAGE_EXTENSION_NAME`
- `XR_ML_spatial_anchors_storage_SPEC_VERSION`
- Extending XrObjectType:
  - `XR_OBJECT_TYPE_SPATIAL_ANCHORS_STORAGE_ML`
- Extending XrResult:
  - `XR_ERROR_SPATIAL_ANCHORS_ANCHOR_NOT_FOUND_ML`
- Extending XrStructureType:
  - `XR_TYPE_SPATIAL_ANCHORS_CREATE_INFO_FROM_UUIDS_ML`
  - `XR_TYPE_SPATIAL_ANCHORS_CREATE_STORAGE_INFO_ML`
  - `XR_TYPE_SPATIAL_ANCHORS_DELETE_COMPLETION_DETAILS_ML`
  - `XR_TYPE_SPATIAL_ANCHORS_DELETE_COMPLETION_ML`
  - `XR_TYPE_SPATIAL_ANCHORS_DELETE_INFO_ML`
  - `XR_TYPE_SPATIAL_ANCHORS_PUBLISH_COMPLETION_DETAILS_ML`
  - `XR_TYPE_SPATIAL_ANCHORS_PUBLISH_COMPLETION_ML`
  - `XR_TYPE_SPATIAL_ANCHORS_PUBLISH_INFO_ML`
  - `XR_TYPE_SPATIAL_ANCHORS_QUERY_COMPLETION_ML`
  - `XR_TYPE_SPATIAL_ANCHORS_QUERY_INFO_RADIUS_ML`
  - `XR_TYPE_SPATIAL_ANCHORS_UPDATE_EXPIRATION_COMPLETION_DETAILS_ML`
  - `XR_TYPE_SPATIAL_ANCHORS_UPDATE_EXPIRATION_COMPLETION_ML`
  - `XR_TYPE_SPATIAL_ANCHORS_UPDATE_EXPIRATION_INFO_ML`

Version History

- Revision 1, 2023-06-22 (Ron Bessems)
  - Initial extension description

12.183. XR_ML_system_notifications

Name String

`XR_ML_system_notifications`

Extension Type

Instance extension

Registered Extension Number

474

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Last Modified Date

2023-09-06

Contributors

Michał Kulągowski, Magic Leap
Ron Bessems, Magic Leap

12.183.1. Overview

This extension provides control over the system notifications. This extension allows system notifications that might obscure the field of view to be disabled.

Note that even when all system notifications have been suppressed, developers **can** still intercept certain events that allow them to properly react to the underlying reason of system notifications.



Permissions

Android applications **must** have the com.magicleap.permission.SYSTEM_NOTIFICATION permission listed in their manifest to use this extension. (protection level: normal)

12.183.2. Suppressing All System Notifications

Applications **can** suppress system notifications from being shown while the application has focus by calling xrSetSystemNotificationsML with the properly filled XrSystemNotificationsSetInfoML structure.

The xrSetSystemNotificationsML function is defined as:

```
// Provided by XR_ML_system_notifications
XrResult xrSetSystemNotificationsML(
    XrInstance                                  instance,
    const XrSystemNotificationsSetInfoML*       info);
```

Parameter Descriptions

- `instance` is XrInstance.
- `info` is a pointer to an XrSystemNotificationsSetInfoML structure.

This API will work only on certain SKUs. When called on an incompatible SKU the `XR_ERROR_SYSTEM_NOTIFICATION_INCOMPATIBLE_SKU_ML` error **must** be returned.

If the com.magicleap.permission.SYSTEM_NOTIFICATION permission is not granted, the runtime **must** return `XR_ERROR_SYSTEM_NOTIFICATION_PERMISSION_DENIED_ML`.

Valid Usage (Implicit)

- The `XR_ML_system_notifications` extension **must** be enabled prior to calling xrSetSystemNotificationsML
- `instance` **must** be a valid XrInstance handle
- `info` **must** be a pointer to a valid XrSystemNotificationsSetInfoML structure

Return Codes

Success

- XR_SUCCESS

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SYSTEM_NOTIFICATION_INCOMPATIBLE_SKU_ML
- XR_ERROR_SYSTEM_NOTIFICATION_PERMISSION_DENIED_ML
- XR_ERROR_VALIDATION_FAILURE

The XrSystemNotificationsSetInfoML structure is defined as:

```
// Provided by XR_ML_system_notifications
typedef struct XrSystemNotificationsSetInfoML {
    XrStructureType    type;
    const void*        next;
    XrBool32           suppressNotifications;
} XrSystemNotificationsSetInfoML;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain.
- suppressNotifications is a boolean flag for suppressing all system notifications.

Valid Usage (Implicit)

- The XR_ML_system_notifications extension **must** be enabled prior to using XrSystemNotificationsSetInfoML
- type **must** be XR_TYPE_SYSTEM_NOTIFICATIONS_SET_INFO_ML
- next **must** be NULL or a valid pointer to the next structure in a structure chain

12.183.3. New Commands

- xrSetSystemNotificationsML

12.183.4. New Structures

- XrSystemNotificationsSetInfoML

12.183.5. New Enum Constants

- `XR_ML_SYSTEM_NOTIFICATIONS_EXTENSION_NAME`
- `XR_ML_system_notifications_SPEC_VERSION`
- Extending XrResult:
  - `XR_ERROR_SYSTEM_NOTIFICATION_INCOMPATIBLE_SKU_ML`
  - `XR_ERROR_SYSTEM_NOTIFICATION_PERMISSION_DENIED_ML`
- Extending XrStructureType:
  - `XR_TYPE_SYSTEM_NOTIFICATIONS_SET_INFO_ML`

Version History

- Revision 1, 2023-09-06 (Michał Kulągowski)
  - Initial extension description

12.184. XR_ML_user_calibration

Name String

`XR_ML_user_calibration`

Extension Type

Instance extension

Registered Extension Number

473

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Last Modified Date

2023-08-21

Contributors

Karthik Kadappan, Magic Leap
Ron Bessems, Magic Leap

12.184.1. Overview

This extension **can** be used to determine how well the device is calibrated for the current user of the device. The extension provides two events for this purpose:

1. Headset Fit: Provides the quality of the fit of the headset on the user.
2. Eye Calibration: Provides the quality of the user's eye calibration.

12.184.2. Enabling user calibration events

User calibration events are requested by calling xrEnableUserCalibrationEventsML. When this function is called, each of the user calibration events **must** be posted to the event queue once, regardless of whether there were any changes to the event data. This allows the application to synchronize with the current state.

The xrEnableUserCalibrationEventsML function is defined as:

```
// Provided by XR_ML_user_calibration
XrResult xrEnableUserCalibrationEventsML(
    XrInstance                                  instance,
    const XrUserCalibrationEnableEventsInfoML*  enableInfo);
```

Parameter Descriptions

- `instance` is a handle to an XrInstance previously created with xrCreateInstance.
- `enableInfo` is the XrUserCalibrationEnableEventsInfoML that enables or disables user calibration events.

Valid Usage (Implicit)

- The `XR_ML_user_calibration` extension **must** be enabled prior to calling xrEnableUserCalibrationEventsML
- `instance` **must** be a valid XrInstance handle
- `enableInfo` **must** be a pointer to a valid XrUserCalibrationEnableEventsInfoML structure

Return Codes

Success

- `XR_SUCCESS`

Failure

- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_VALIDATION_FAILURE`

The XrUserCalibrationEnableEventsInfoML structure is defined as:

```
// Provided by XR_ML_user_calibration
typedef struct XrUserCalibrationEnableEventsInfoML {
    XrStructureType    type;
    const void*        next;
    XrBool32           enabled;
} XrUserCalibrationEnableEventsInfoML;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `enabled` is the flag to enable/disable user calibration events.

Valid Usage (Implicit)

- The XR_ML_user_calibration extension **must** be enabled prior to using XrUserCalibrationEnableEventsInfoML
- type **must** be XR_TYPE_USER_CALIBRATION_ENABLE_EVENTS_INFO_ML
- next **must** be NULL or a valid pointer to the next structure in a structure chain

12.184.3. Headset Fit Events

Receiving an XrEventDataHeadsetFitChangedML event from xrPollEvent notifies the application of headset fit changes. To enable these events call xrEnableUserCalibrationEventsML and set XrUserCalibrationEnableEventsInfoML::enabled to true. Headset fit is evaluated continuously and the runtime **must** post events anytime it detects a change in the headset fit state.

The XrEventDataHeadsetFitChangedML structure is defined as:

```
// Provided by XR_ML_user_calibration
typedef struct XrEventDataHeadsetFitChangedML {
    XrStructureType         type;
    const void*             next;
    XrHeadsetFitStatusML    status;
    XrTime                  time;
} XrEventDataHeadsetFitChangedML;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain.
- status is the XrHeadsetFitStatusML headset fit status.
- time is the XrTime at which the status was captured.

Valid Usage (Implicit)

- The XR_ML_user_calibration extension **must** be enabled prior to using XrEventDataHeadsetFitChangedML
- type **must** be XR_TYPE_EVENT_DATA_HEADSET_FIT_CHANGED_ML
- next **must** be NULL or a valid pointer to the next structure in a structure chain

```
// Provided by XR_ML_user_calibration
typedef enum XrHeadsetFitStatusML {
    XR_HEADSET_FIT_STATUS_UNKNOWN_ML = 0,
    XR_HEADSET_FIT_STATUS_NOT_WORN_ML = 1,
    XR_HEADSET_FIT_STATUS_GOOD_FIT_ML = 2,
    XR_HEADSET_FIT_STATUS_BAD_FIT_ML = 3,
    XR_HEADSET_FIT_STATUS_MAX_ENUM_ML = 0x7FFFFFFF
} XrHeadsetFitStatusML;
```

Enum	Description
XR_HEADSET_FIT_STATUS_UNKNOWN_ML	Headset fit status not available for unknown reason.
XR_HEADSET_FIT_STATUS_NOT_WORN_ML	Headset not worn.
XR_HEADSET_FIT_STATUS_GOOD_FIT_ML	Good fit.
XR_HEADSET_FIT_STATUS_BAD_FIT_ML	Bad fit.

12.184.4. Eye Calibration Events

Receiving an XrEventDataEyeCalibrationChangedML event from xrPollEvent notifies the application of eye calibration changes. To enable these events call xrEnableUserCalibrationEventsML and set XrUserCalibrationEnableEventsInfoML::enabled to true. Runtime **must** post events anytime it detects a change in the eye calibration. The user needs to calibrate the eyes using the system app provided for this. There is no support for in-app eye calibration in this extension.

The XrEventDataEyeCalibrationChangedML structure is defined as:

```
// Provided by XR_ML_user_calibration
typedef struct XrEventDataEyeCalibrationChangedML {
    XrStructureType             type;
    const void*                 next;
    XrEyeCalibrationStatusML    status;
} XrEventDataEyeCalibrationChangedML;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain.
- status is the XrEyeCalibrationStatusML eye calibration status.

Valid Usage (Implicit)

- The XR_ML_user_calibration extension **must** be enabled prior to using XrEventDataEyeCalibrationChangedML
- type **must** be XR_TYPE_EVENT_DATA_EYE_CALIBRATION_CHANGED_ML
- next **must** be NULL or a valid pointer to the next structure in a structure chain

```
// Provided by XR_ML_user_calibration
typedef enum XrEyeCalibrationStatusML {
    XR_EYE_CALIBRATION_STATUS_UNKNOWN_ML = 0,
    XR_EYE_CALIBRATION_STATUS_NONE_ML = 1,
    XR_EYE_CALIBRATION_STATUS_COARSE_ML = 2,
    XR_EYE_CALIBRATION_STATUS_FINE_ML = 3,
    XR_EYE_CALIBRATION_STATUS_MAX_ENUM_ML = 0x7FFFFFFF
} XrEyeCalibrationStatusML;
```

Enum	Description
XR_EYE_CALIBRATION_STATUS_UNKNOWN_ML	Eye calibration status not available for unknown reason.
XR_EYE_CALIBRATION_STATUS_NONE_ML	User has not performed the eye calibration step. Use system provided app to perform eye calibration.
XR_EYE_CALIBRATION_STATUS_COARSE_ML	Eye calibration is of lower accuracy.
XR_EYE_CALIBRATION_STATUS_FINE_ML	Eye calibration is of higher accuracy.

12.184.5. New Enum Constants

XrStructureType enumeration is extended with:

- XR_TYPE_EVENT_DATA_HEADSET_FIT_CHANGED_ML
- XR_TYPE_EVENT_DATA_EYE_CALIBRATION_CHANGED_ML
- XR_TYPE_USER_CALIBRATION_ENABLE_EVENTS_INFO_ML

Version History

- Revision 1, 2023-06-20 (Karthik Kadappan)
  - Initial extension description

12.185. XR_ML_view_configuration_depth_range_change

Name String

XR_ML_view_configuration_depth_range_change

Extension Type

Instance extension

Registered Extension Number

484

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

XR_EXT_view_configuration_depth_range

Last Modified Date

2023-11-09

IP Status

No known IP claims.

Contributors

Ron Bessems, Magic Leap
Andrei Aristarkhov, Magic Leap

12.185.1. Overview

There is a desire for runtimes to be able to inform applications of changes in clipping planes during the lifetime of an XrInstance.

12.185.2. Background

The XrViewConfigurationDepthRangeEXT structure is used to inform applications of the clipping plane values. However, since this information is obtained via xrEnumerateViewConfigurations its contents **must** not change.

12.185.3. Behavior change

When this extension is enabled, the runtime **may** change the contents of XrViewConfigurationDepthRangeEXT during the lifetime of an XrInstance.

Applications **should** track changes in the clipping plane values as provided by the runtime every frame.

Be aware that unlike most OpenXR extensions, enabling this extension is all that is required for runtime behavior to change, which **may** have impacts on modular applications, including applications built on engines.

12.185.4. New Enum Constants

- `XR_ML_VIEW_CONFIGURATION_DEPTH_RANGE_CHANGE_EXTENSION_NAME`
- `XR_ML_view_configuration_depth_range_change_SPEC_VERSION`

Version History

- Revision 1, 2023-11-09 (Ron Bessems)
  - Initial extension description

12.186. XR_ML_world_mesh_detection

Name String

`XR_ML_world_mesh_detection`

Extension Type

Instance extension

Registered Extension Number

475

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

`XR_EXT_future`

Last Modified Date

2023-08-29

Contributors

Ron Bessems, Magic Leap
Karthik Kadappan, Magic Leap

12.186.1. Overview

This extension provides the functionality to detect the world mesh.

12.186.2. Creating a world mesh detector

The XrWorldMeshDetectorML handle is defined as:

```
// Provided by XR_ML_world_mesh_detection
XR_DEFINE_HANDLE(XrWorldMeshDetectorML)
```

XrWorldMeshDetectorML is created by xrCreateWorldMeshDetectorML.

The xrCreateWorldMeshDetectorML function is defined as:

```
// Provided by XR_ML_world_mesh_detection
XrResult xrCreateWorldMeshDetectorML(
    XrSession                                   session,
    const XrWorldMeshDetectorCreateInfoML*      createInfo,
    XrWorldMeshDetectorML*                      detector);
```

Parameter Descriptions

- `session` is a handle to an XrSession.
- `createInfo` is a pointer to an XrWorldMeshDetectorCreateInfoML structure.
- `detector` is a pointer to XrWorldMeshDetectorML, returned by the runtime.

Valid Usage (Implicit)

- The `XR_ML_world_mesh_detection` extension **must** be enabled prior to calling xrCreateWorldMeshDetectorML
- `session` **must** be a valid XrSession handle
- `createInfo` **must** be a pointer to a valid XrWorldMeshDetectorCreateInfoML structure
- `detector` **must** be a pointer to an XrWorldMeshDetectorML handle

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE
- XR_ERROR_WORLD_MESH_DETECTOR_PERMISSION_DENIED_ML



Permissions

Android applications **must** have the com.magicleap.permission.SPATIAL_MAPPING permission listed in their manifest to use this extension. (protection level: dangerous)

The XrWorldMeshDetectorCreateInfoML structure is defined as:

```
// Provided by XR_ML_world_mesh_detection
typedef struct XrWorldMeshDetectorCreateInfoML {
    XrStructureType    type;
    const void*        next;
} XrWorldMeshDetectorCreateInfoML;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain.

Valid Usage (Implicit)

- The XR_ML_world_mesh_detection extension **must** be enabled prior to using XrWorldMeshDetectorCreateInfoML
- type **must** be XR_TYPE_WORLD_MESH_DETECTOR_CREATE_INFO_ML
- next **must** be NULL or a valid pointer to the next structure in a structure chain

12.186.3. Destroying a world mesh detector

The xrDestroyWorldMeshDetectorML function is defined as:

```
// Provided by XR_ML_world_mesh_detection
XrResult xrDestroyWorldMeshDetectorML(
    XrWorldMeshDetectorML                       detector);
```

Parameter Descriptions

- detector is the XrWorldMeshDetectorML to destroy.

Valid Usage (Implicit)

- The XR_ML_world_mesh_detection extension **must** be enabled prior to calling xrDestroyWorldMeshDetectorML
- detector **must** be a valid XrWorldMeshDetectorML handle

Thread Safety

- Access to detector, and any child handles, **must** be externally synchronized

Return Codes

Success

- XR_SUCCESS

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_RUNTIME_FAILURE

12.186.4. Detecting the World Mesh

Detecting the world mesh is done by blocks. Instead of returning the whole world mesh as one entity, this system returns the world mesh in chunks called blocks. To retrieve the currently detected chunks use xrRequestWorldMeshStateAsyncML.

The xrRequestWorldMeshStateAsyncML function is defined as:

```
// Provided by XR_ML_world_mesh_detection
XrResult xrRequestWorldMeshStateAsyncML(
    XrWorldMeshDetectorML                       detector,
    const XrWorldMeshStateRequestInfoML*        stateRequest,
    XrFutureEXT*                                future);
```

Parameter Descriptions

- detector is the XrWorldMeshDetectorML to use.
- stateRequest is the XrWorldMeshStateRequestInfoML request information.
- future is a pointer to a runtime created XrFutureEXT.

Valid Usage (Implicit)

- The XR_ML_world_mesh_detection extension **must** be enabled prior to calling xrRequestWorldMeshStateAsyncML
- detector **must** be a valid XrWorldMeshDetectorML handle
- stateRequest **must** be a pointer to a valid XrWorldMeshStateRequestInfoML structure
- future **must** be a pointer to an XrFutureEXT value

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_POSE_INVALID
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_TIME_INVALID
- XR_ERROR_VALIDATION_FAILURE
- XR_ERROR_WORLD_MESH_DETECTOR_SPACE_NOT_LOCATABLE_ML

The XrWorldMeshStateRequestInfoML structure is defined as:

```
// Provided by XR_ML_world_mesh_detection
typedef struct XrWorldMeshStateRequestInfoML {
    XrStructureType    type;
    const void*        next;
    XrSpace            baseSpace;
    XrTime             time;
    XrPosef            boundingBoxCenter;
    XrExtent3DfEXT     boundingBoxExtents;
} XrWorldMeshStateRequestInfoML;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `baseSpace` is the base space used by the bounding box.
- `time` is the time at which the bounding box location is resolved.
- `boundingBoxCenter` the XrPosef of the bounding box center.
- `boundingBoxExtents` the XrExtent3DfEXT of the bounding box. These extents indicate the full size of the box, so an XrExtent3DfEXT of 1.0,1.0,1.0 has a volume of 1 cubic meter and the box's center is at `boundingBoxCenter`.

Each mesh block **may** have a 'skirt' if `XR_WORLD_MESH_DETECTOR_REMOVE_MESH_SKIRT_BIT_ML` was not specified during the creation of the XrWorldMeshDetectorML. A skirt provides some overlap between adjacent mesh blocks. While a skirt improves coverage between blocks, it does not create a watertight mesh.

Valid Usage (Implicit)

- The `XR_ML_world_mesh_detection` extension **must** be enabled prior to using XrWorldMeshStateRequestInfoML
- `type` **must** be `XR_TYPE_WORLD_MESH_STATE_REQUEST_INFO_ML`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `baseSpace` **must** be a valid XrSpace handle

```
// Provided by XR_ML_world_mesh_detection
typedef XrFlags64 XrWorldMeshDetectorFlagsML;
```

```
// Provided by XR_ML_world_mesh_detection
// Flag bits for XrWorldMeshDetectorFlagsML
static const XrWorldMeshDetectorFlagsML XR_WORLD_MESH_DETECTOR_POINT_CLOUD_BIT_ML =
0x00000001;
static const XrWorldMeshDetectorFlagsML XR_WORLD_MESH_DETECTOR_COMPUTE_NORMALS_BIT_ML =
0x00000002;
static const XrWorldMeshDetectorFlagsML XR_WORLD_MESH_DETECTOR_COMPUTE_CONFIDENCE_BIT_ML
= 0x00000004;
static const XrWorldMeshDetectorFlagsML XR_WORLD_MESH_DETECTOR_PLANARIZE_BIT_ML =
0x00000008;
static const XrWorldMeshDetectorFlagsML XR_WORLD_MESH_DETECTOR_REMOVE_MESH_SKIRT_BIT_ML =
0x00000010;
static const XrWorldMeshDetectorFlagsML XR_WORLD_MESH_DETECTOR_INDEX_ORDER_CW_BIT_ML =
0x00000020;
```

Flag Descriptions

- XR_WORLD_MESH_DETECTOR_POINT_CLOUD_BIT_ML — If set, will return a point cloud instead of a triangle mesh.
- XR_WORLD_MESH_DETECTOR_COMPUTE_NORMALS_BIT_ML — If set, the system will compute the normals for the triangle vertices.
- XR_WORLD_MESH_DETECTOR_COMPUTE_CONFIDENCE_BIT_ML — If set, the system will compute the confidence values.
- XR_WORLD_MESH_DETECTOR_PLANARIZE_BIT_ML — If set, the system will planarize the returned mesh (planar regions will be smoothed out).
- XR_WORLD_MESH_DETECTOR_REMOVE_MESH_SKIRT_BIT_ML — If set, the mesh skirt (overlapping area between two mesh blocks) will be removed.
- XR_WORLD_MESH_DETECTOR_INDEX_ORDER_CW_BIT_ML — If set, winding order of indices will be be changed from counter clockwise to clockwise.

xrRequestWorldMeshStateAsyncML is an asynchronous function and xrRequestWorldMeshStateCompleteML retrieves the data once the future is ready.

The xrRequestWorldMeshStateCompleteML function is defined as:

```
// Provided by XR_ML_world_mesh_detection
XrResult xrRequestWorldMeshStateCompleteML(
    XrWorldMeshDetectorML                       detector,
    XrFutureEXT                                 future,
    XrWorldMeshStateRequestCompletionML*        completion);
```

Parameter Descriptions

- `detector` is the XrWorldMeshDetectorML to use.
- `future` is a the `XrFutureEXT` to complete.
- `completion` is a pointer to the XrWorldMeshStateRequestCompletionML structure.

Valid Usage (Implicit)

- The `XR_ML_world_mesh_detection` extension **must** be enabled prior to calling xrRequestWorldMeshStateCompleteML
- `detector` **must** be a valid XrWorldMeshDetectorML handle
- `completion` **must** be a pointer to an XrWorldMeshStateRequestCompletionML structure

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_FUTURE_INVALID_EXT`
- `XR_ERROR_FUTURE_PENDING_EXT`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_SIZE_INSUFFICIENT`
- `XR_ERROR_VALIDATION_FAILURE`

The XrWorldMeshStateRequestCompletionML structure is defined as:

```
// Provided by XR_ML_world_mesh_detection
typedef struct XrWorldMeshStateRequestCompletionML {
    XrStructureType             type;
    void*                       next;
    XrResult                    futureResult;
    XrTime                      timestamp;
    uint32_t                    meshBlockStateCapacityInput;
    uint32_t                    meshBlockStateCountOutput;
    XrWorldMeshBlockStateML*    meshBlockStates;
} XrWorldMeshStateRequestCompletionML;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `futureResult` is the XrResult of the asynchronous operation.
- `timestamp` is the `XrTime` of the request.
- `meshBlockStateCapacityInput` is the capacity of the `meshBlockStates` array, or 0 to indicate a request to retrieve the required capacity.
- `meshBlockStateCountOutput` is filled in by the runtime with the count of mesh states written or the required capacity in the case that `meshBlockStateCapacityInput` is insufficient.
- `meshBlockStates` is a pointer to an array of XrWorldMeshBlockStateML.

Future Return Codes

`futureResult` values:

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_OUT_OF_MEMORY`
- `XR_ERROR_LIMIT_REACHED`
- `XR_ERROR_WORLD_MESH_DETECTOR_SPACE_NOT_LOCATABLE_ML`

Valid Usage (Implicit)

- The `XR_ML_world_mesh_detection` extension **must** be enabled prior to using XrWorldMeshStateRequestCompletionML
- `type` **must** be `XR_TYPE_WORLD_MESH_STATE_REQUEST_COMPLETION_ML`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `futureResult` **must** be a valid XrResult value
- If `meshBlockStateCapacityInput` is not `0`, `meshBlockStates` **must** be a pointer to an array of `meshBlockStateCapacityInput` XrWorldMeshBlockStateML structures

The XrWorldMeshBlockStateML structure is defined as:

```
// Provided by XR_ML_world_mesh_detection
typedef struct XrWorldMeshBlockStateML {
    XrStructureType             type;
    void*                       next;
    XrUuidEXT                   uuid;
    XrPosef                     meshBoundingBoxCenter;
    XrExtent3DfEXT              meshBoundingBoxExtents;
    XrTime                      lastUpdateTime;
    XrWorldMeshBlockStatusML    status;
} XrWorldMeshBlockStateML;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `uuid` is the XrUuidEXT of the mesh block.
- `meshBoundingBoxCenter` the XrPosef of the mesh's bounding box center.
- `meshBoundingBoxExtents` the XrExtent3DfEXT of the mesh's bounding box.
- `lastUpdateTime` is the `XrTime` at which this mesh was last updated.
- `status` is an XrWorldMeshBlockStatusML indicating the status of the mesh block.

Valid Usage (Implicit)

- The `XR_ML_world_mesh_detection` extension **must** be enabled prior to using XrWorldMeshBlockStateML
- `type` **must** be `XR_TYPE_WORLD_MESH_BLOCK_STATE_ML`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `status` **must** be a valid XrWorldMeshBlockStatusML value

```
// Provided by XR_ML_world_mesh_detection
typedef enum XrWorldMeshBlockStatusML {
    XR_WORLD_MESH_BLOCK_STATUS_NEW_ML = 0,
    XR_WORLD_MESH_BLOCK_STATUS_UPDATED_ML = 1,
    XR_WORLD_MESH_BLOCK_STATUS_DELETED_ML = 2,
    XR_WORLD_MESH_BLOCK_STATUS_UNCHANGED_ML = 3,
    XR_WORLD_MESH_BLOCK_STATUS_MAX_ENUM_ML = 0x7FFFFFFF
} XrWorldMeshBlockStatusML;
```

Enum	Description
`XR_WORLD_MESH_BLOCK_STATUS_NEW_ML`	The mesh block has been created.
`XR_WORLD_MESH_BLOCK_STATUS_UPDATED_ML`	The mesh block has been updated.
`XR_WORLD_MESH_BLOCK_STATUS_DELETED_ML`	The mesh block has been deleted.
`XR_WORLD_MESH_BLOCK_STATUS_UNCHANGED_ML`	The mesh block is unchanged.

12.186.5. Allocate Mesh Block Memory

The next step is to retrieve the actual vertex data. This operation will require a buffer to be available

for the duration of the asynchronous operation and for as long as the application needs access to XrWorldMeshRequestCompletionML.

This buffer must be allocated by the application, the system provides recommended buffer size using the xrGetWorldMeshBufferRecommendSizeML function.

The xrGetWorldMeshBufferRecommendSizeML function is defined as:

```
// Provided by XR_ML_world_mesh_detection
XrResult xrGetWorldMeshBufferRecommendSizeML(
    XrWorldMeshDetectorML                       detector,
    const XrWorldMeshBufferRecommendedSizeInfoML* sizeInfo,
    XrWorldMeshBufferSizeML*                    size);
```

Parameter Descriptions

- `detector` is the XrWorldMeshDetectorML to use.
- `sizeInfo` is a pointer to an XrWorldMeshBufferRecommendedSizeInfoML structure.
- `size` is the XrWorldMeshBufferSizeML, this will be filled by the runtime with the size to allocate for the xrRequestWorldMeshAsyncML function. `size` **must** be larger than 0.

Errata: This function is called `xrGetWorldMeshBufferRecommendSizeML` rather than the expected `xrGetWorldMeshBufferRecommendedSizeML`.

Valid Usage (Implicit)

- The `XR_ML_world_mesh_detection` extension **must** be enabled prior to calling xrGetWorldMeshBufferRecommendSizeML
- `detector` **must** be a valid XrWorldMeshDetectorML handle
- `sizeInfo` **must** be a pointer to a valid XrWorldMeshBufferRecommendedSizeInfoML structure
- `size` **must** be a pointer to an XrWorldMeshBufferSizeML structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The XrWorldMeshBufferRecommendedSizeInfoML structure is defined as:

```
// Provided by XR_ML_world_mesh_detection
typedef struct XrWorldMeshBufferRecommendedSizeInfoML {
    XrStructureType    type;
    const void*        next;
    uint32_t           maxBlockCount;
} XrWorldMeshBufferRecommendedSizeInfoML;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain.
- maxBlockCount is a uint32_t of the maximum number of blocks in the request.

The value for maxBlockCount **should** be populated XrWorldMeshStateRequestCompletionML ::meshBlockStateCountOutput. As long as the maxBlockCount is equal or larger to this XrWorldMeshStateRequestCompletionML::meshBlockStateCountOutput, a memory block **may** be reused for new requests.

Valid Usage (Implicit)

- The XR_ML_world_mesh_detection extension **must** be enabled prior to using XrWorldMeshBufferRecommendedSizeInfoML
- type **must** be XR_TYPE_WORLD_MESH_BUFFER_RECOMMENDED_SIZE_INFO_ML
- next **must** be NULL or a valid pointer to the next structure in a structure chain

The XrWorldMeshBufferSizeML structure is defined as:

```
// Provided by XR_ML_world_mesh_detection
typedef struct XrWorldMeshBufferSizeML {
    XrStructureType    type;
    void*              next;
    uint32_t           size;
} XrWorldMeshBufferSizeML;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain.
- size is a uint32_t indicating the number of bytes recommended.

Valid Usage (Implicit)

- The XR_ML_world_mesh_detection extension **must** be enabled prior to using XrWorldMeshBufferSizeML
- type **must** be XR_TYPE_WORLD_MESH_BUFFER_SIZE_ML
- next **must** be NULL or a valid pointer to the next structure in a structure chain

Some runtimes have optimized memory available that avoids memory copies and provides the fastest way to get the vertex data. Applications **should** use the xrAllocateWorldMeshBufferML function to reserve memory for the vertex data. The application however **may** choose to allocate its own memory using alternative methods.

The xrAllocateWorldMeshBufferML function is defined as:

```
// Provided by XR_ML_world_mesh_detection
XrResult xrAllocateWorldMeshBufferML(
    XrWorldMeshDetectorML                       detector,
    const XrWorldMeshBufferSizeML*              size,
    XrWorldMeshBufferML*                        buffer);
```

Parameter Descriptions

- `detector` is the XrWorldMeshDetectorML to use.
- `size` is the XrWorldMeshBufferSizeML request information previously retrieved via xrGetWorldMeshBufferRecommendSizeML.
- `buffer` is a pointer to an XrWorldMeshBufferML that will be filled by the runtime with a pointer to a memory block and size of the buffer.

Valid Usage (Implicit)

- The `XR_ML_world_mesh_detection` extension **must** be enabled prior to calling xrAllocateWorldMeshBufferML
- `detector` **must** be a valid XrWorldMeshDetectorML handle
- `size` **must** be a pointer to a valid XrWorldMeshBufferSizeML structure
- `buffer` **must** be a pointer to an XrWorldMeshBufferML structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The XrWorldMeshBufferML structure is defined as:

```
// Provided by XR_ML_world_mesh_detection
typedef struct XrWorldMeshBufferML {
    XrStructureType    type;
    void*              next;
    uint32_t           bufferSize;
    void*              buffer;
} XrWorldMeshBufferML;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain.
- bufferSize is a uint32_t indicating the size of the buffer in bytes.
- buffer is a void memory block. This memory **may** be allocated using xrAllocateWorldMeshBufferML or otherwise.

Valid Usage (Implicit)

- The XR_ML_world_mesh_detection extension **must** be enabled prior to using XrWorldMeshBufferML
- type **must** be XR_TYPE_WORLD_MESH_BUFFER_ML
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- buffer **must** be a pointer to an array of bufferSize bytes
- The bufferSize parameter **must** be greater than 0

Memory blocks allocated with xrAllocateWorldMeshBufferML **must** be freed using xrFreeWorldMeshBufferML.

The xrFreeWorldMeshBufferML function is defined as:

```
// Provided by XR_ML_world_mesh_detection
XrResult xrFreeWorldMeshBufferML(
    XrWorldMeshDetectorML                       detector,
    const XrWorldMeshBufferML*                  buffer);
```

Parameter Descriptions

- detector is the XrWorldMeshDetectorML to use.
- buffer is a pointer to XrWorldMeshBufferML to be freed.

Valid Usage (Implicit)

- The XR_ML_world_mesh_detection extension **must** be enabled prior to calling xrFreeWorldMeshBufferML
- detector **must** be a valid XrWorldMeshDetectorML handle
- buffer **must** be a pointer to a valid XrWorldMeshBufferML structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

12.186.6. Start Mesh Data Query

Once a buffer has been allocated the mesh data retrieval **may** be started using xrRequestWorldMeshAsyncML.

The xrRequestWorldMeshAsyncML function is defined as:

```
// Provided by XR_ML_world_mesh_detection
XrResult xrRequestWorldMeshAsyncML(
    XrWorldMeshDetectorML                       detector,
    const XrWorldMeshGetInfoML*                 getInfo,
    XrWorldMeshBufferML*                        buffer,
    XrFutureEXT*                                future);
```

Parameter Descriptions

- detector is the XrWorldMeshDetectorML to use.
- getInfo is a pointer to an XrWorldMeshGetInfoML structure.
- buffer is a pointer to an XrWorldMeshBufferML to be used. This buffer **must** be kept available for the duration of the request and for however long the application needs access to the vertex buffer.
- future is a pointer to an XrFutureEXT to be filled by the runtime.

Valid Usage (Implicit)

- The `XR_ML_world_mesh_detection` extension **must** be enabled prior to calling xrRequestWorldMeshAsyncML
- `detector` **must** be a valid XrWorldMeshDetectorML handle
- `getInfo` **must** be a pointer to a valid XrWorldMeshGetInfoML structure
- `buffer` **must** be a pointer to an XrWorldMeshBufferML structure
- `future` **must** be a pointer to an `XrFutureEXT` value

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_VALIDATION_FAILURE`

The XrWorldMeshGetInfoML structure is defined as:

```
// Provided by XR_ML_world_mesh_detection
typedef struct XrWorldMeshGetInfoML {
    XrStructureType               type;
    const void*                   next;
    XrWorldMeshDetectorFlagsML    flags;
    float                         fillHoleLength;
    float                         disconnectedComponentArea;
    uint32_t                      blockCount;
    XrWorldMeshBlockRequestML*    blocks;
} XrWorldMeshGetInfoML;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `flags` are the detector flags defined by XrWorldMeshDetectorFlagBitsML.
- `fillHoleLength` Any hole which has perimeter (in meters) less than `fillHoleLength` will be filled. The maximum hole perimeter is limited to 5 meters, the runtime **must** clamp the value between 0 and 5 meters. Also the maximum area of the hole **should** not exceed 0.5 square meters. If the hole overlaps with neighboring mesh block(s), it will not be filled in.
- `disconnectedComponentArea` Any component that is disconnected from the main mesh which has an area (in square meters) less than `disconnectedComponentArea` will be removed. The maximum area is limited to 2 square meters, and the area needs to exist fully within a mesh block boundary without skirt. The runtime **must** clamp this value between 0 and 2 square meters.
- `blockCount` is a `uint32_t` indicating the number of elements in the `blocks` array.
- `blocks` is an array of XrWorldMeshBlockRequestML.

Valid Usage (Implicit)

- The `XR_ML_world_mesh_detection` extension **must** be enabled prior to using XrWorldMeshGetInfoML
- `type` **must** be `XR_TYPE_WORLD_MESH_GET_INFO_ML`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `flags` **must** be `0` or a valid combination of XrWorldMeshDetectorFlagBitsML values
- `blocks` **must** be a pointer to an array of `blockCount` XrWorldMeshBlockRequestML structures
- The `blockCount` parameter **must** be greater than `0`

The XrWorldMeshBlockRequestML structure is defined as:

```
// Provided by XR_ML_world_mesh_detection
typedef struct XrWorldMeshBlockRequestML {
    XrStructureType             type;
    void*                       next;
    XrUuidEXT                   uuid;
    XrWorldMeshDetectorLodML    lod;
} XrWorldMeshBlockRequestML;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `uuid` is the XrUuidEXT of the mesh block to retrieve.
- `lod` is the XrWorldMeshDetectorLodML level of detail of the mesh to retrieve.

Valid Usage (Implicit)

- The `XR_ML_world_mesh_detection` extension **must** be enabled prior to using XrWorldMeshBlockRequestML
- `type` **must** be `XR_TYPE_WORLD_MESH_BLOCK_REQUEST_ML`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `lod` **must** be a valid XrWorldMeshDetectorLodML value

```
// Provided by XR_ML_world_mesh_detection
typedef enum XrWorldMeshDetectorLodML {
    XR_WORLD_MESH_DETECTOR_LOD_MINIMUM_ML = 0,
    XR_WORLD_MESH_DETECTOR_LOD_MEDIUM_ML = 1,
    XR_WORLD_MESH_DETECTOR_LOD_MAXIMUM_ML = 2,
    XR_WORLD_MESH_DETECTOR_LOD_MAX_ENUM_ML = 0x7FFFFFFF
} XrWorldMeshDetectorLodML;
```

Enum	Description
`XR_WORLD_MESH_DETECTOR_LOD_MINIMUM_ML`	Minimum Level of Detail (LOD) for the mesh.
`XR_WORLD_MESH_DETECTOR_LOD_MEDIUM_ML`	Medium Level of Detail (LOD) for the mesh.
`XR_WORLD_MESH_DETECTOR_LOD_MAXIMUM_ML`	Maximum Level of Detail (LOD) for the mesh.

12.186.7. Complete Mesh Data Query

To complete the previously started mesh data query xrRequestWorldMeshCompleteML is used.

The xrRequestWorldMeshCompleteML function is defined as:

```
// Provided by XR_ML_world_mesh_detection
XrResult xrRequestWorldMeshCompleteML(
    XrWorldMeshDetectorML                       detector,
    const XrWorldMeshRequestCompletionInfoML*   completionInfo,
    XrFutureEXT                                 future,
    XrWorldMeshRequestCompletionML*             completion);
```

Parameter Descriptions

- `detector` is the XrWorldMeshDetectorML to use.
- `future` is an `XrFutureEXT` to complete.
- `completionInfo` is a pointer to XrWorldMeshRequestCompletionInfoML.
- `completion` is a pointer to XrWorldMeshRequestCompletionML containing the completion result.

Valid Usage (Implicit)

- The `XR_ML_world_mesh_detection` extension **must** be enabled prior to calling xrRequestWorldMeshCompleteML
- `detector` **must** be a valid XrWorldMeshDetectorML handle
- `completionInfo` **must** be a pointer to a valid XrWorldMeshRequestCompletionInfoML structure
- `completion` **must** be a pointer to an XrWorldMeshRequestCompletionML structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_FUTURE_INVALID_EXT
- XR_ERROR_FUTURE_PENDING_EXT
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_TIME_INVALID
- XR_ERROR_VALIDATION_FAILURE
- XR_ERROR_WORLD_MESH_DETECTOR_SPACE_NOT_LOCATABLE_ML

The XrWorldMeshRequestCompletionInfoML structure is defined as:

```
// Provided by XR_ML_world_mesh_detection
typedef struct XrWorldMeshRequestCompletionInfoML {
    XrStructureType    type;
    const void*        next;
    XrSpace            meshSpace;
    XrTime             meshSpaceLocateTime;
} XrWorldMeshRequestCompletionInfoML;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain.
- meshSpace the space used to express the vertex data in.
- meshSpaceLocateTime the time used to locate the meshSpace.

Valid Usage (Implicit)

- The XR_ML_world_mesh_detection extension **must** be enabled prior to using XrWorldMeshRequestCompletionInfoML
- type **must** be XR_TYPE_WORLD_MESH_REQUEST_COMPLETION_INFO_ML
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- meshSpace **must** be a valid XrSpace handle

The XrWorldMeshRequestCompletionML structure is defined as:

```
// Provided by XR_ML_world_mesh_detection
typedef struct XrWorldMeshRequestCompletionML {
    XrStructureType        type;
    void*                  next;
    XrResult               futureResult;
    uint32_t               blockCount;
    XrWorldMeshBlockML*    blocks;
} XrWorldMeshRequestCompletionML;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain.
- futureResult is the XrResult of the asynchronous operation.
- blockCount is a uint32_t that **must** match XrWorldMeshGetInfoML::blockCount parameter.
- blocks is an array of XrWorldMeshBlockML structures containing the vertex data pointers.

Future Return Codes

`futureResult` values:

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_OUT_OF_MEMORY`
- `XR_ERROR_LIMIT_REACHED`

Valid Usage (Implicit)

- The `XR_ML_world_mesh_detection` extension **must** be enabled prior to using XrWorldMeshRequestCompletionML
- `type` **must** be `XR_TYPE_WORLD_MESH_REQUEST_COMPLETION_ML`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `futureResult` **must** be a valid XrResult value
- `blocks` **must** be a pointer to an array of `blockCount` XrWorldMeshBlockML structures
- The `blockCount` parameter **must** be greater than `0`

The XrWorldMeshBlockML structure is defined as:

```
// Provided by XR_ML_world_mesh_detection
typedef struct XrWorldMeshBlockML {
    XrStructureType               type;
    void*                         next;
    XrUuidEXT                     uuid;
    XrWorldMeshBlockResultML      blockResult;
    XrWorldMeshDetectorLodML      lod;
    XrWorldMeshDetectorFlagsML    flags;
    uint32_t                      indexCount;
    uint16_t*                     indexBuffer;
    uint32_t                      vertexCount;
    XrVector3f*                   vertexBuffer;
    uint32_t                      normalCount;
    XrVector3f*                   normalBuffer;
    uint32_t                      confidenceCount;
    float*                        confidenceBuffer;
} XrWorldMeshBlockML;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `uuid` is the XrUuidEXT of the mesh block
- `blockResult` is the XrWorldMeshBlockResultML of the mesh block.
- `lod` is the XrWorldMeshDetectorLodML used to generated this mesh block.
- `flags` are the detector flags defined by XrWorldMeshDetectorFlagBitsML.
- `indexCount` is the number of indices in the `indexBuffer`.
- `indexBuffer` is the index buffer of type `uint16_t`. In the indexBuffer each value is the index of a vertex in the vertexBuffer. Three indices define one triangle. For example, the first triangle will have the vertices: `vertexBuffer[index[0]]`, `vertexBuffer[index[1]]`, `vertexBuffer[index[2]]`. Index order is defined by the `XR_WORLD_MESH_DETECTOR_INDEX_ORDER_CW_BIT_ML` flag.
- `vertexCount` is the number of vertices in the `vertexBuffer`.
- `vertexBuffer` is the vertex buffer of type XrVector3f. Vertex data will be provided in the XrWorldMeshStateRequestInfoML::`baseSpace` at time XrWorldMeshBlockStateML ::`lastUpdateTime`.
- `normalCount` is the number of normals in the `normalBuffer`.
- `normalBuffer` is the normal buffer of type XrVector3f.
- `confidenceCount` is the number of confidence values in the `confidenceBuffer`.
- `confidenceBuffer` is the confidence buffer of type `float` with range 0.0 - 1.0.

`normalCount` **must** be equal to `vertexCount` if `XR_WORLD_MESH_DETECTOR_COMPUTE_NORMALS_BIT_ML` was specified during XrWorldMeshDetectorML creation, otherwise 0.

`confidenceCount` **must** be equal to `vertexCount` if `XR_WORLD_MESH_DETECTOR_COMPUTE_CONFIDENCE_BIT_ML` was specified during XrWorldMeshDetectorML creation, otherwise 0.

Valid Usage (Implicit)

- The XR_ML_world_mesh_detection extension **must** be enabled prior to using XrWorldMeshBlockML
- type **must** be XR_TYPE_WORLD_MESH_BLOCK_ML
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- blockResult **must** be a valid XrWorldMeshBlockResultML value
- lod **must** be a valid XrWorldMeshDetectorLodML value
- flags **must** be a valid combination of XrWorldMeshDetectorFlagBitsML values
- flags **must** not be 0
- indexBuffer **must** be a pointer to an array of indexCount uint16_t values
- vertexBuffer **must** be a pointer to an array of vertexCount XrVector3f structures
- normalBuffer **must** be a pointer to an array of normalCount XrVector3f structures
- confidenceBuffer **must** be a pointer to an array of confidenceCount float values
- The indexCount parameter **must** be greater than 0
- The vertexCount parameter **must** be greater than 0
- The normalCount parameter **must** be greater than 0
- The confidenceCount parameter **must** be greater than 0

```
// Provided by XR_ML_world_mesh_detection
typedef enum XrWorldMeshBlockResultML {
    XR_WORLD_MESH_BLOCK_RESULT_SUCCESS_ML = 0,
    XR_WORLD_MESH_BLOCK_RESULT_FAILED_ML = 1,
    XR_WORLD_MESH_BLOCK_RESULT_PENDING_ML = 2,
    XR_WORLD_MESH_BLOCK_RESULT_PARTIAL_UPDATE_ML = 3,
    XR_WORLD_MESH_BLOCK_RESULT_MAX_ENUM_ML = 0x7FFFFFFF
} XrWorldMeshBlockResultML;
```

Enum	Description
XR_WORLD_MESH_BLOCK_RESULT_SUCCESS_ML	Mesh request has succeeded.
XR_WORLD_MESH_BLOCK_RESULT_FAILED_ML	Mesh request has failed.
XR_WORLD_MESH_BLOCK_RESULT_PENDING_ML	Mesh request is pending.
XR_WORLD_MESH_BLOCK_RESULT_PARTIAL_UPDATE_ML	There are partial updates on the mesh request.

12.186.8. Sample code

```
class MeshDetector {
  private:

    enum State {
      INFO_START, INFO_WAIT_COMPLETE, MESH_START, MESH_WAIT_COMPLETE, DONE
    };

    XrInstance m_Instance; // previously initialized.
    XrSession m_Session; // previously initialized.
    XrSpace m_ViewSpace; // previously initialized.
    XrSpace m_LocalSpace; // previously initialized.

    State m_State{INFO_START};
    XrFutureEXT m_Future{XR_NULL_FUTURE_EXT};
    XrWorldMeshDetectorML m_Detector;
    std::vector<XrWorldMeshBlockStateML> m_MeshBlocks;

    std::array<uint32_t,2> m_MaxBlockCounts{0};
    std::array<XrWorldMeshBufferML, 2> m_WorldMeshBuffers{{
      {
        .type = XR_TYPE_WORLD_MESH_BUFFER_ML,
      },
      {
        .type = XR_TYPE_WORLD_MESH_BUFFER_ML,
      },
    }};
    uint32_t m_QueryBuffer{0};
    std::vector<XrWorldMeshBlockML> m_WorldMeshBlocks;

    bool m_ApplicationCreatedMemory{false};


    bool StartInfoQuery(XrTime displayTime) {
      XrWorldMeshStateRequestInfoML requestInfo {
        .type = XR_TYPE_WORLD_MESH_STATE_REQUEST_INFO_ML,
        .baseSpace = m_ViewSpace,
        .time = displayTime,
        .boundingBoxCenter = {
          .orientation = {
            .w = 1.0f,
          },
        },
        .boundingBoxExtents = {10.0f, 10.0f, 10.0f},
      };
      return xrRequestWorldMeshStateAsyncML(m_Detector, &requestInfo, &m_Future)
```

```
==XR_SUCCESS;
    }

    bool CompleteInfoQuery() {
      XrWorldMeshStateRequestCompletionML completion {
        .type = XR_TYPE_WORLD_MESH_STATE_REQUEST_COMPLETION_ML,
      };
      if (xrRequestWorldMeshStateCompleteML(m_Detector, m_Future, &completion) !=
XR_SUCCESS) {
        return false;
      }
      if (completion.futureResult!=XR_SUCCESS) {
        return false;
      }
      m_MeshBlocks.resize(completion.meshBlockStateCountOutput, {
        .type = XR_TYPE_WORLD_MESH_BLOCK_STATE_ML,
      });
      completion.meshBlockStateCapacityInput = completion.meshBlockStateCountOutput;
      completion.meshBlockStates = m_MeshBlocks.data();
      CHK_XR(xrRequestWorldMeshStateCompleteML(m_Detector, m_Future, &completion));

      if (completion.meshBlockStateCountOutput==0) {
        return false; // start a new query.
      }

      // switch to next buffer.
      m_QueryBuffer = ( m_QueryBuffer + 1 ) % 2;

      if (completion.meshBlockStateCountOutput>m_MaxBlockCounts[m_QueryBuffer]) {
        m_MaxBlockCounts[m_QueryBuffer] = completion.meshBlockStateCountOutput;

        XrWorldMeshBufferRecommendedSizeInfoML sizeInfo {
          .type = XR_TYPE_WORLD_MESH_BUFFER_RECOMMENDED_SIZE_INFO_ML,
        };
        XrWorldMeshBufferSizeML bufferSize {
          .type = XR_TYPE_WORLD_MESH_BUFFER_SIZE_ML,
        };

        sizeInfo.maxBlockCount = m_MaxBlockCounts[m_QueryBuffer];
        CHK_XR(xrGetWorldMeshBufferRecommendSizeML(m_Detector, &sizeInfo, &bufferSize ));

        if (m_ApplicationCreatedMemory) {
          // It may be advantageous to use memory allocated
          // specific to the use case. For example shared graphics
          // memory may provide some performance benefits by avoiding
          // extra copying.
          m_WorldMeshBuffers[m_QueryBuffer].bufferSize = bufferSize.size;
          m_WorldMeshBuffers[m_QueryBuffer].buffer = malloc(bufferSize.size);
```

```
        } else {
          CHK_XR(xrAllocateWorldMeshBufferML(m_Detector, &bufferSize, &
m_WorldMeshBuffers[m_QueryBuffer]));
        }
      }
      return true;
    }

    bool StartMeshQuery() {
      std::vector<XrWorldMeshBlockRequestML> blockRequests(m_MeshBlocks.size());
      for (size_t i = 0; i< m_MeshBlocks.size();i++) {
        blockRequests[i].type = XR_TYPE_WORLD_MESH_BLOCK_REQUEST_ML;
        blockRequests[i].next = nullptr;
        blockRequests[i].uuid = m_MeshBlocks[i].uuid;
        blockRequests[i].lod = XR_WORLD_MESH_DETECTOR_LOD_MEDIUM_ML;
      }

      XrWorldMeshGetInfoML getInfo {
        .type = XR_TYPE_WORLD_MESH_GET_INFO_ML,
        .flags = 0,
        .fillHoleLength = 0.5f,
        .disconnectedComponentArea = 1.0f,
        .blockCount = static_cast<uint32_t>(blockRequests.size()),
        .blocks = blockRequests.data(),
      };
      CHK_XR(xrRequestWorldMeshAsyncML(m_Detector, &getInfo, &m_WorldMeshBuffers
[m_QueryBuffer], &m_Future));
      return true;
    }

    bool CompleteMeshQuery(XrTime displayTime) {
      m_WorldMeshBlocks.resize(m_MeshBlocks.size());
      XrWorldMeshRequestCompletionML completion {
        .type = XR_TYPE_WORLD_MESH_REQUEST_COMPLETION_ML,
        .blockCount = static_cast<uint32_t>(m_WorldMeshBlocks.size()),
        .blocks = m_WorldMeshBlocks.data(),
      };
      XrWorldMeshRequestCompletionInfoML completionInfo {
        .type = XR_TYPE_WORLD_MESH_REQUEST_COMPLETION_INFO_ML,
        .meshSpace = m_LocalSpace,
        .meshSpaceLocateTime = displayTime,
      };
      CHK_XR(xrRequestWorldMeshCompleteML(m_Detector, &completionInfo, m_Future,
&completion));
      CHK_XR(completion.futureResult);

      // The vertex data is now usable.
      // the backing buffer double-buffered, so the vertex data remains valid
```

```
    // even though a new request might be processing.


    return true;
  }

public:

  MeshDetector() {
    XrWorldMeshDetectorCreateInfoML createInfo {
      .type = XR_TYPE_WORLD_MESH_DETECTOR_CREATE_INFO_ML,
    };
    CHK_XR(xrCreateWorldMeshDetectorML(m_Session,&createInfo, &m_Detector));
  }


  ~MeshDetector() {

    // Must ensure the future has finished before destroying.
    // std::assert(IsDone());

    for (auto &buffer : m_WorldMeshBuffers) {
      if ( buffer.buffer != nullptr ) {
        if (m_ApplicationCreatedMemory) {
          free(buffer.buffer);
        } else {
          xrFreeWorldMeshBufferML(m_Detector, &buffer);
        }
      }
    }

    xrDestroyWorldMeshDetectorML(m_Detector);
    m_Detector = XR_NULL_HANDLE;
  }

  void RequestShutdown() {
    if ( m_Future != XR_NULL_FUTURE_EXT) {
      XrFutureCancelInfoEXT cancelInfo {
        .type = XR_TYPE_FUTURE_CANCEL_INFO_EXT,
        .future = m_Future,
      };
      xrCancelFutureEXT(m_Instance, &cancelInfo);
      m_State = DONE;
    } else {
      m_State = DONE;
    }
  }
```

```
    bool IsDone() {
      return m_State == DONE;
    }

    void FrameLoop(XrTime displayTime) {

      if (m_Future == XR_NULL_FUTURE_EXT) {
        return;
      }

      XrFuturePollResultEXT pollResult {
        .type = XR_TYPE_FUTURE_POLL_RESULT_EXT,
        .state = XR_FUTURE_STATE_PENDING_EXT,
      };

      XrFuturePollInfoEXT pollInfo {
        .type = XR_TYPE_FUTURE_POLL_INFO_EXT,
        .future = m_Future,
      };
      CHK_XR(xrPollFutureEXT(m_Instance, &pollInfo, &pollResult));

      switch (m_State) {

        case INFO_START:
          if (StartInfoQuery(displayTime)) {
            m_State = INFO_WAIT_COMPLETE;
          }
          break;

        case INFO_WAIT_COMPLETE:
          if (pollResult.state==XR_FUTURE_STATE_READY_EXT) {
            if (CompleteInfoQuery()) {
              m_State = MESH_START;
            } else {
              m_State = INFO_START;
            }
          }
          break;

        case MESH_START:
          if (StartMeshQuery()) {
            m_State = MESH_WAIT_COMPLETE;
          }
          break;

        case MESH_WAIT_COMPLETE:
          if (pollResult.state==XR_FUTURE_STATE_READY_EXT) {
            if (CompleteMeshQuery(displayTime)) {
```

```
                m_State = INFO_START;
              }
            }
            break;
          case DONE:
            break;
        }

    }

};
```

12.186.9. New Object Types

- XrWorldMeshDetectorML

12.186.10. New Commands

- xrAllocateWorldMeshBufferML
- xrCreateWorldMeshDetectorML
- xrDestroyWorldMeshDetectorML
- xrFreeWorldMeshBufferML
- xrGetWorldMeshBufferRecommendSizeML
- xrRequestWorldMeshAsyncML
- xrRequestWorldMeshCompleteML
- xrRequestWorldMeshStateAsyncML
- xrRequestWorldMeshStateCompleteML

12.186.11. New Structures

- XrWorldMeshBlockML
- XrWorldMeshBlockRequestML
- XrWorldMeshBlockStateML
- XrWorldMeshBufferML
- XrWorldMeshBufferRecommendedSizeInfoML
- XrWorldMeshBufferSizeML
- XrWorldMeshDetectorCreateInfoML
- XrWorldMeshGetInfoML
- XrWorldMeshRequestCompletionInfoML

- XrWorldMeshRequestCompletionML
- XrWorldMeshStateRequestCompletionML
- XrWorldMeshStateRequestInfoML

12.186.12. New Enums

- XrWorldMeshBlockResultML
- XrWorldMeshBlockStatusML
- XrWorldMeshDetectorFlagBitsML
- XrWorldMeshDetectorLodML

12.186.13. New Bitmasks

- XrWorldMeshDetectorFlagsML

12.186.14. New Enum Constants

- XR_ML_WORLD_MESH_DETECTION_EXTENSION_NAME
- XR_ML_world_mesh_detection_SPEC_VERSION
- Extending XrObjectType:
  - XR_OBJECT_TYPE_WORLD_MESH_DETECTOR_ML
- Extending XrResult:
  - XR_ERROR_WORLD_MESH_DETECTOR_PERMISSION_DENIED_ML
  - XR_ERROR_WORLD_MESH_DETECTOR_SPACE_NOT_LOCATABLE_ML
- Extending XrStructureType:
  - XR_TYPE_WORLD_MESH_BLOCK_ML
  - XR_TYPE_WORLD_MESH_BLOCK_REQUEST_ML
  - XR_TYPE_WORLD_MESH_BLOCK_STATE_ML
  - XR_TYPE_WORLD_MESH_BUFFER_ML
  - XR_TYPE_WORLD_MESH_BUFFER_RECOMMENDED_SIZE_INFO_ML
  - XR_TYPE_WORLD_MESH_BUFFER_SIZE_ML
  - XR_TYPE_WORLD_MESH_DETECTOR_CREATE_INFO_ML
  - XR_TYPE_WORLD_MESH_GET_INFO_ML
  - XR_TYPE_WORLD_MESH_REQUEST_COMPLETION_INFO_ML
  - XR_TYPE_WORLD_MESH_REQUEST_COMPLETION_ML
  - XR_TYPE_WORLD_MESH_STATE_REQUEST_COMPLETION_ML

    - XR_TYPE_WORLD_MESH_STATE_REQUEST_INFO_ML

Issues

Version History

- Revision 1, 2023-08-29
    - Initial Revision

12.187. XR_MND_headless

Name String

XR_MND_headless

Extension Type

Instance extension

Registered Extension Number

43

Revision

3

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Last Modified Date

2025-08-20

IP Status

No known IP claims.

Contributors

Rylie Pavlik, Collabora

Overview

Some applications may wish to access XR interaction devices without presenting any image content on the display(s). This extension provides a mechanism for writing such an application using the OpenXR API. It modifies the specification in the following ways, without adding any new named entities.

- When this extension is enabled, an application **may** call xrCreateSession without having made a

call to xrGet*GraphicsRequirements, and without an XrGraphicsBinding* structure in the next chain of XrSessionCreateInfo.

- If an application does not include an XrGraphicsBinding* structure in the next chain of XrSessionCreateInfo, the runtime **must** create a "headless" session that does not interact with the display.
- In a headless session, the session state **should** proceed to XR_SESSION_STATE_READY directly from XR_SESSION_STATE_IDLE.
- In a headless session, the XrSessionBeginInfo::primaryViewConfigurationType **must** be ignored and **may** be 0.
- In a headless session, the session state proceeds to XR_SESSION_STATE_SYNCHRONIZED, then XR_SESSION_STATE_VISIBLE and XR_SESSION_STATE_FOCUSED, after the call to xrBeginSession. The application does not need to call xrWaitFrame, xrBeginFrame, or xrEndFrame, unlike with non-headless sessions.
- In a headless session, xrEnumerateSwapchainFormats **must** return XR_SUCCESS but enumerate 0 formats.
- xrWaitFrame **must** set XrFrameState::shouldRender to XR_FALSE in a headless session. The VISIBLE and FOCUSED states are only used for their input-related semantics, not their rendering-related semantics, and these functions are permitted to allow minimal change between headless and non-headless code if desired.

Because xrWaitFrame is not required, an application using a headless session **should** sleep periodically to avoid consuming all available system resources in a busy-wait loop.

New Object Types

New Flag Types

New Enum Constants

New Enums

New Structures

New Functions

Issues

- Not all devices with which this would be useful fit into one of the existing XrFormFactor values.

Version History

- Revision 1, 2019-07-25 (Rylie Pavlik, Collabora, Ltd.)
  - Initial version reflecting Monado prototype.
- Revision 2, 2019-10-22 (Rylie Pavlik, Collabora, Ltd.)

    - Clarify that `xrWaitFrame` is permitted and should set `shouldRender` to false.
- Revision 3, 2025-08-20 (Dan Willmott, Valve Corporation)
    - Clarify that apps do not need to call `xrGet*GraphicsRequirements` before calling `xrCreateSession` when using this extension.

12.188. XR_OCULUS_audio_device_guid

Name String

`XR_OCULUS_audio_device_guid`

Extension Type

Instance extension

Registered Extension Number

160

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Overview

This extension enables the querying of audio device information associated with an OpenXR instance.

On Windows, there may be multiple audio devices available on the system. This extensions allows applications to query the runtime for the appropriate audio devices for the active HMD.

New Object Types

New Flag Types

New Enum Constants

- `XR_MAX_AUDIO_DEVICE_STR_SIZE_OCULUS`

New Enums

New Structures

New Functions

```
// Provided by XR_OCULUS_audio_device_guid
XrResult xrGetAudioOutputDeviceGuidOculus(
    XrInstance                                  instance,
    wchar_t                                     buffer
[XR_MAX_AUDIO_DEVICE_STR_SIZE_OCULUS]);
```

Parameter Descriptions

- `instance` is the XrInstance to query the audio device state in.
- `buffer` is a fixed size buffer which will contain the audio device GUID. The format of this data matches the IMMDevice::GetId API.

Valid Usage (Implicit)

- The `XR_OCULUS_audio_device_guid` extension **must** be enabled prior to calling xrGetAudioOutputDeviceGuidOculus
- `instance` **must** be a valid XrInstance handle
- `buffer` **must** be a wide character array of length `XR_MAX_AUDIO_DEVICE_STR_SIZE_OCULUS`

Return Codes

Success

- `XR_SUCCESS`

Failure

- `XR_ERROR_FEATURE_UNSUPPORTED`
- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_VALIDATION_FAILURE`

```
// Provided by XR_OCULUS_audio_device_guid
XrResult xrGetAudioInputDeviceGuidOculus(
    XrInstance                                  instance,
    wchar_t                                     buffer
[XR_MAX_AUDIO_DEVICE_STR_SIZE_OCULUS]);
```

Parameter Descriptions

- `instance` is the XrInstance to query the audio device state in.
- `buffer` is a fixed size buffer which will contain the audio device GUID. The format of this data matches the IMMDevice::GetId API.

Valid Usage (Implicit)

- The `XR_OCULUS_audio_device_guid` extension **must** be enabled prior to calling xrGetAudioInputDeviceGuidOculus
- `instance` **must** be a valid XrInstance handle
- `buffer` **must** be a wide character array of length `XR_MAX_AUDIO_DEVICE_STR_SIZE_OCULUS`

Return Codes

Success

- `XR_SUCCESS`

Failure

- `XR_ERROR_FEATURE_UNSUPPORTED`
- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_VALIDATION_FAILURE`

Issues

1. The intent of xrGetAudioInputDeviceGuidOculus and xrGetAudioOutputDeviceGuidOculus is to provide GUIDs associated with the active head mounted device. In OpenXR, a particular system is represented by an `XrSystemId`. If these commands were to be reimplemented, they should also pass

a `XrSystemId` parameter to allow the head mounted device system to be specified.

2. The names xrGetAudioInputDeviceGuidOculus and xrGetAudioOutputDeviceGuidOculus do not match the convention for OpenXR command names. If these commands were to be reimplemented, `Oculus` should be replaced by `OCULUS`.

Version History

- Revision 1, 2021-05-13 (John Kearney)
  - Initial extension description

12.189. XR_OCULUS_external_camera

Name String

`XR_OCULUS_external_camera`

Extension Type

Instance extension

Registered Extension Number

227

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Overview

This extension enables the querying of external camera information for a session. This extension is intended to enable mixed reality capture support for applications.

This extension does not provide a mechanism for supplying external camera information to the runtime. If external camera information is not supplied to the runtime before using this extension, no camera information will be returned.

This API supports returning camera intrinsics and extrinsics:

- Camera intrinsics are the attributes of the camera: resolution, field of view, etc.
- Camera extrinsics are everything external to the camera: relative pose, attached to, etc.
- We do not expect the camera intrinsics to change frequently. We expect the camera extrinsics to

change frequently.

New Object Types

New Flag Types

```
// Provided by XR_OCULUS_external_camera
typedef XrFlags64 XrExternalCameraStatusFlagsOCULUS;
```

```
// Provided by XR_OCULUS_external_camera
// Flag bits for XrExternalCameraStatusFlagsOCULUS
static const XrExternalCameraStatusFlagsOCULUS
XR_EXTERNAL_CAMERA_STATUS_CONNECTED_BIT_OCULUS = 0x00000001;
static const XrExternalCameraStatusFlagsOCULUS
XR_EXTERNAL_CAMERA_STATUS_CALIBRATING_BIT_OCULUS = 0x00000002;
static const XrExternalCameraStatusFlagsOCULUS
XR_EXTERNAL_CAMERA_STATUS_CALIBRATION_FAILED_BIT_OCULUS = 0x00000004;
static const XrExternalCameraStatusFlagsOCULUS
XR_EXTERNAL_CAMERA_STATUS_CALIBRATED_BIT_OCULUS = 0x00000008;
static const XrExternalCameraStatusFlagsOCULUS
XR_EXTERNAL_CAMERA_STATUS_CAPTURING_BIT_OCULUS = 0x00000010;
```

Flag Descriptions

- XR_EXTERNAL_CAMERA_STATUS_CONNECTED_BIT_OCULUS — External camera is connected
- XR_EXTERNAL_CAMERA_STATUS_CALIBRATING_BIT_OCULUS — External camera is undergoing calibration
- XR_EXTERNAL_CAMERA_STATUS_CALIBRATION_FAILED_BIT_OCULUS — External camera has tried and failed calibration
- XR_EXTERNAL_CAMERA_STATUS_CALIBRATED_BIT_OCULUS — External camera has tried and passed calibration
- XR_EXTERNAL_CAMERA_STATUS_CAPTURING_BIT_OCULUS — External camera is capturing

New Enum Constants

XR_MAX_EXTERNAL_CAMERA_NAME_SIZE_OCULUS defines the length of the field XrExternalCameraOCULUS ::name.

```
#define XR_MAX_EXTERNAL_CAMERA_NAME_SIZE_OCULUS 32
```

XrStructureType enumeration is extended with:

- XR_TYPE_EXTERNAL_CAMERA_OCULUS

New Enums

```
// Provided by XR_OCULUS_external_camera
typedef enum XrExternalCameraAttachedToDeviceOCULUS {
    XR_EXTERNAL_CAMERA_ATTACHED_TO_DEVICE_NONE_OCULUS = 0,
    XR_EXTERNAL_CAMERA_ATTACHED_TO_DEVICE_HMD_OCULUS = 1,
    XR_EXTERNAL_CAMERA_ATTACHED_TO_DEVICE_LTOUCH_OCULUS = 2,
    XR_EXTERNAL_CAMERA_ATTACHED_TO_DEVICE_RTOUCH_OCULUS = 3,
    XR_EXTERNAL_CAMERA_ATTACHED_TO_DEVICE_MAX_ENUM_OCULUS = 0x7FFFFFFF
} XrExternalCameraAttachedToDeviceOCULUS;
```

Enum	Description
XR_EXTERNAL_CAMERA_ATTACHED_TO_DEVICE_NONE_OCULUS	External camera is at a fixed point in LOCAL space
XR_EXTERNAL_CAMERA_ATTACHED_TO_DEVICE_HMD_OCULUS	External camera is attached to the HMD
XR_EXTERNAL_CAMERA_ATTACHED_TO_DEVICE_LTOUCH_OCULUS	External camera is attached to a left Touch controller
XR_EXTERNAL_CAMERA_ATTACHED_TO_DEVICE_RTOUCH_OCULUS	External camera is attached to a right Touch controller

New Structures

The XrExternalCameraIntrinsicsOCULUS structure is defined as:

```
// Provided by XR_OCULUS_external_camera
typedef struct XrExternalCameraIntrinsicsOCULUS {
    XrTime         lastChangeTime;
    XrFovf         fov;
    float          virtualNearPlaneDistance;
    float          virtualFarPlaneDistance;
    XrExtent2Di    imageSensorPixelResolution;
} XrExternalCameraIntrinsicsOCULUS;
```

Member Descriptions

- `lastChangeTime` is the `XrTime` when this camera's intrinsics last changed.
- `fov` is the XrFovf for this camera's viewport.
- `virtualNearPlaneDistance` is the near plane distance of the virtual camera used to match the external camera
- `virtualFarPlaneDistance` is the far plane distance of the virtual camera used to match the external camera
- `imageSensorPixelResolution` is the XrExtent2Di specifying the camera's resolution (in pixels).

Valid Usage (Implicit)

- The `XR_OCULUS_external_camera` extension **must** be enabled prior to using XrExternalCameraIntrinsicsOCULUS

The XrExternalCameraExtrinsicsOCULUS structure is defined as:

```
// Provided by XR_OCULUS_external_camera
typedef struct XrExternalCameraExtrinsicsOCULUS {
    XrTime                                  lastChangeTime;
    XrExternalCameraStatusFlagsOCULUS       cameraStatusFlags;
    XrExternalCameraAttachedToDeviceOCULUS  attachedToDevice;
    XrPosef                                 relativePose;
} XrExternalCameraExtrinsicsOCULUS;
```

Member Descriptions

- `lastChangeTime` is the `XrTime` when this camera's extrinsics last changed.
- `cameraStatusFlags` is the XrExternalCameraStatusFlagsOCULUS for this camera's status.
- `attachedToDevice` is the XrExternalCameraAttachedToDeviceOCULUS for the device this camera is attached to
- `relativePose` is the XrPosef for offset of the camera from the device that the camera is attached to

Valid Usage (Implicit)

- The XR_OCULUS_external_camera extension **must** be enabled prior to using XrExternalCameraExtrinsicsOCULUS
- cameraStatusFlags **must** be 0 or a valid combination of XrExternalCameraStatusFlagBitsOCULUS values
- attachedToDevice **must** be a valid XrExternalCameraAttachedToDeviceOCULUS value

The XrExternalCameraOCULUS structure is defined as:

```
// Provided by XR_OCULUS_external_camera
typedef struct XrExternalCameraOCULUS {
    XrStructureType                     type;
    const void*                         next;
    char                                name[XR_MAX_EXTERNAL_CAMERA_NAME_SIZE_OCULUS];
    XrExternalCameraIntrinsicsOCULUS    intrinsics;
    XrExternalCameraExtrinsicsOCULUS    extrinsics;
} XrExternalCameraOCULUS;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- name is a null-terminated UTF-8 string containing a camera identifier: VID (vendor ID), PID (product ID), and serial number
- intrinsics is the XrExternalCameraIntrinsicsOCULUS for the camera
- extrinsics is the XrExternalCameraExtrinsicsOCULUS for the camera

Valid Usage (Implicit)

- The XR_OCULUS_external_camera extension **must** be enabled prior to using XrExternalCameraOCULUS
- type **must** be XR_TYPE_EXTERNAL_CAMERA_OCULUS
- next **must** be NULL or a valid pointer to the next structure in a structure chain

New Functions

The xrEnumerateExternalCamerasOCULUS function enumerates all the external cameras that are supported by the runtime, it is defined as:

```
// Provided by XR_OCULUS_external_camera
XrResult xrEnumerateExternalCamerasOCULUS(
    XrSession                                   session,
    uint32_t                                    cameraCapacityInput,
    uint32_t*                                   cameraCountOutput,
    XrExternalCameraOCULUS*                     cameras);
```

Parameter Descriptions

- `session` is the XrSession to query the external cameras in
- `cameraCapacityInput` is the capacity of the `cameras` array, or 0 to indicate a request to retrieve the required capacity.
- `cameraCountOutput` is filled in by the runtime with the count of `cameras` written or the required capacity in the case that `cameraCapacityInput` is insufficient.
- `cameras` is an array of XrExternalCameraOCULUS filled in by the runtime which contains all the available external cameras, but **can** be `NULL` if `cameraCapacityInput` is `0`.
- See the Buffer Size Parameters section for a detailed description of retrieving the required `cameras` size.

Valid Usage (Implicit)

- The `XR_OCULUS_external_camera` extension **must** be enabled prior to calling xrEnumerateExternalCamerasOCULUS
- `session` **must** be a valid XrSession handle
- `cameraCountOutput` **must** be a pointer to a `uint32_t` value
- If `cameraCapacityInput` is not `0`, `cameras` **must** be a pointer to an array of `cameraCapacityInput` XrExternalCameraOCULUS structures

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_OUT_OF_MEMORY`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_SIZE_INSUFFICIENT`
- `XR_ERROR_VALIDATION_FAILURE`

Issues

Version History

- Revision 1, 2022-08-31 (John Kearney)
  - Initial extension description

12.190. XR_OPPO_controller_interaction

Name String

`XR_OPPO_controller_interaction`

Extension Type

Instance extension

Registered Extension Number

454

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

API Interactions

- Interacts with `XR_EXT_dpad_binding`
- Interacts with `XR_EXT_hand_interaction`
- Interacts with `XR_EXT_palm_pose`

Contributors

Haomiao Jiang, OPPO
Buyi Xu, OPPO
Yebao Cai, OPPO

Overview

This extension defines a new interaction profile for the OPPO Controller, including but not limited to OPPO MR Glasses Controller.

OPPO Controller interaction profile

Interaction profile path:

- */interaction_profiles/oppo/mr_controller_oppo*

Valid for user paths:

- */user/hand/left*
- */user/hand/right*

This interaction profile represents the input sources and haptics on the OPPO Controller.

Supported component paths:

- On /user/hand/left only:
  - *.../input/x/click*
  - *.../input/x/touch*
  - *.../input/y/click*
  - *.../input/y/touch*
  - *.../input/menu/click*
  - *.../input/heartrate_oppo/value*
- On /user/hand/right only:
  - *.../input/a/click*

    - *.../input/a/touch*
    - *.../input/b/click*
    - *.../input/b/touch*
    - *.../input/home/click* (**may** not be available for application use)
- *.../input/squeeze/value*
- *.../input/trigger/touch*
- *.../input/trigger/value*
- *.../input/grip/pose*
- *.../input/aim/pose*
- *.../input/thumbstick/click*
- *.../input/thumbstick/touch*
- *.../input/thumbstick*
- *.../input/thumbstick/x*
- *.../input/thumbstick/y*
- *.../output/haptic*

New Identifiers

- **heartrate_oppo**: OPPO MR Controller adds an optional heart rate sensor to monitor the heart beat rate of the user.

Input Path Descriptions

- **/input/heartrate_oppo/value** : Allow developers to access the heart beat per minute (BPM) of the user. The data would only be available with user's active consent.



Note

When the runtime supports `XR_VERSION_1_1` and use of OpenXR 1.1 is requested by the application, this interaction profile **must** also support

- *.../input/grip_surface/pose*



Note

When the `XR_KHR_maintenance1` extension is available and enabled, this interaction profile **must** also support

- *.../input/grip_surface/pose*



Note

When the `XR_EXT_palm_pose` extension is available and enabled, this interaction profile **must** also support

- *.../input/palm_ext/pose*



Note

When the `XR_EXT_hand_interaction` extension is available and enabled, this interaction profile **must** also support

- *.../input/pinch_ext/pose*
- *.../input/poke_ext/pose*

Version History

- Revision 1, Haomiao Jiang
  - Initial extension description

12.191. XR_QCOM_tracking_optimization_settings

Name String

`XR_QCOM_tracking_optimization_settings`

Extension Type

Instance extension

Registered Extension Number

307

Revision

2

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Last Modified Date

2022-06-02

Contributors

Daniel Guttenberg, Qualcomm
Martin Renschler, Qualcomm
Karthik Nagarajan, Qualcomm
Kurtis Eveleigh, Qualcomm

12.191.1. Overview

This extension defines an API for the application to give optimization hints to the runtime for tracker domains.

For example, an application might be interested in tracking targets that are at a far distance from the camera which **may** increase tracking latency, while another application might be interested in minimizing power consumption at the cost of tracking accuracy. Targets are domains which are defined in XrTrackingOptimizationSettingsDomainQCOM.

This allows the application to tailor the tracking algorithms to specific use-cases and scene-scales in order to provide the best experience possible.

Summary: provide domain hints to the run-time about which parameters to optimize tracking for.

12.191.2. Setting Tracking Optimization Hints

The tracking optimization hints are expressed as a hint XrTrackingOptimizationSettingsHintQCOM.

```
// Provided by XR_QCOM_tracking_optimization_settings
typedef enum XrTrackingOptimizationSettingsDomainQCOM {
    XR_TRACKING_OPTIMIZATION_SETTINGS_DOMAIN_ALL_QCOM = 1,
    XR_TRACKING_OPTIMIZATION_SETTINGS_DOMAIN_MAX_ENUM_QCOM = 0x7FFFFFFF
} XrTrackingOptimizationSettingsDomainQCOM;
```

Enumerant Descriptions

- XR_TRACKING_OPTIMIZATION_SETTINGS_DOMAIN_ALL_QCOM — Setting applies to all QCOM tracking extensions.

```
// Provided by XR_QCOM_tracking_optimization_settings
typedef enum XrTrackingOptimizationSettingsHintQCOM {
    XR_TRACKING_OPTIMIZATION_SETTINGS_HINT_NONE_QCOM = 0,
    XR_TRACKING_OPTIMIZATION_SETTINGS_HINT_LONG_RANGE_PRIORITIZATION_QCOM = 1,
    XR_TRACKING_OPTIMIZATION_SETTINGS_HINT_CLOSE_RANGE_PRIORITIZATION_QCOM = 2,
    XR_TRACKING_OPTIMIZATION_SETTINGS_HINT_LOW_POWER_PRIORITIZATION_QCOM = 3,
    XR_TRACKING_OPTIMIZATION_SETTINGS_HINT_HIGH_POWER_PRIORITIZATION_QCOM = 4,
  // Provided by XR_QCOM_tracking_optimization_settings
    XR_TRACKING_OPTIMIZATION_SETTINGS_HINT_LONG_RANGE_PRIORIZATION_QCOM =
XR_TRACKING_OPTIMIZATION_SETTINGS_HINT_LONG_RANGE_PRIORITIZATION_QCOM,
  // Provided by XR_QCOM_tracking_optimization_settings
    XR_TRACKING_OPTIMIZATION_SETTINGS_HINT_CLOSE_RANGE_PRIORIZATION_QCOM =
XR_TRACKING_OPTIMIZATION_SETTINGS_HINT_CLOSE_RANGE_PRIORITIZATION_QCOM,
  // Provided by XR_QCOM_tracking_optimization_settings
    XR_TRACKING_OPTIMIZATION_SETTINGS_HINT_LOW_POWER_PRIORIZATION_QCOM =
XR_TRACKING_OPTIMIZATION_SETTINGS_HINT_LOW_POWER_PRIORITIZATION_QCOM,
  // Provided by XR_QCOM_tracking_optimization_settings
    XR_TRACKING_OPTIMIZATION_SETTINGS_HINT_HIGH_POWER_PRIORIZATION_QCOM =
XR_TRACKING_OPTIMIZATION_SETTINGS_HINT_HIGH_POWER_PRIORITIZATION_QCOM,
    XR_TRACKING_OPTIMIZATION_SETTINGS_HINT_MAX_ENUM_QCOM = 0x7FFFFFFF
} XrTrackingOptimizationSettingsHintQCOM;
```

Enumerant Descriptions

- XR_TRACKING_OPTIMIZATION_SETTINGS_HINT_NONE_QCOM — Used by the application to indicate that it does not have a preference to optimize for. The run-time is understood to choose a balanced approach.
- XR_TRACKING_OPTIMIZATION_SETTINGS_HINT_LONG_RANGE_PRIORITIZATION_QCOM — Used by the application to indicate that it prefers tracking to be optimized for long range, possibly at the expense of competing interests.
- XR_TRACKING_OPTIMIZATION_SETTINGS_HINT_CLOSE_RANGE_PRIORITIZATION_QCOM — Used by the application to indicate that it prefers tracking to be optimized for close range, possibly at the expense of competing interests.
- XR_TRACKING_OPTIMIZATION_SETTINGS_HINT_LOW_POWER_PRIORITIZATION_QCOM — Used by the application to indicate that it prefers tracking to be optimized for low power consumption, possibly at the expense of competing interests.
- XR_TRACKING_OPTIMIZATION_SETTINGS_HINT_HIGH_POWER_PRIORITIZATION_QCOM — Used by the application to indicate that it prefers tracking to be optimized for increased tracking performance, possibly at the cost of increased power consumption.
- XR_TRACKING_OPTIMIZATION_SETTINGS_HINT_LONG_RANGE_PRIORIZATION_QCOM — Backward-compatibility alias containing typo. (Added by the XR_QCOM_tracking_optimization_settings extension)
- XR_TRACKING_OPTIMIZATION_SETTINGS_HINT_CLOSE_RANGE_PRIORIZATION_QCOM — Backward-compatibility alias containing typo. (Added by the XR_QCOM_tracking_optimization_settings extension)
- XR_TRACKING_OPTIMIZATION_SETTINGS_HINT_LOW_POWER_PRIORIZATION_QCOM — Backward-compatibility alias containing typo. (Added by the XR_QCOM_tracking_optimization_settings extension)
- XR_TRACKING_OPTIMIZATION_SETTINGS_HINT_HIGH_POWER_PRIORIZATION_QCOM — Backward-compatibility alias containing typo. (Added by the XR_QCOM_tracking_optimization_settings extension)

The xrSetTrackingOptimizationSettingsHintQCOM function is defined as:

```
// Provided by XR_QCOM_tracking_optimization_settings
XrResult xrSetTrackingOptimizationSettingsHintQCOM(
    XrSession                                   session,
    XrTrackingOptimizationSettingsDomainQCOM    domain,
    XrTrackingOptimizationSettingsHintQCOM      hint);
```

Parameter Descriptions

- `session` is a valid XrSession handle.
- `domain` is the tracking domain for which the hint is applied
- `hint` is the hint to be applied

The XR runtime behaves as if `XR_TRACKING_OPTIMIZATION_SETTINGS_HINT_NONE_QCOM` was submitted if the application does not provide a hint.

The XR runtime **must** return `XR_ERROR_VALIDATION_FAILURE` if the application sets a domain or hint not part of XrTrackingOptimizationSettingsDomainQCOM or XrTrackingOptimizationSettingsHintQCOM.

A hint is typically set before a domain handle is created. If hints are set more than once from one or concurrent sessions, the runtime **may** accommodate the first hint it received and return `XR_ERROR_HINT_ALREADY_SET_QCOM` for any subsequent calls made.

If the application destroys the active domain handle associated with the hint, the runtime **may** behave as if `XR_TRACKING_OPTIMIZATION_SETTINGS_HINT_NONE_QCOM` was set. In this scenario, the runtime **should** accommodate new valid hints that **may** be set for the same domain.

Valid Usage (Implicit)

- The `XR_QCOM_tracking_optimization_settings` extension **must** be enabled prior to calling xrSetTrackingOptimizationSettingsHintQCOM
- `session` **must** be a valid XrSession handle
- `domain` **must** be a valid XrTrackingOptimizationSettingsDomainQCOM value
- `hint` **must** be a valid XrTrackingOptimizationSettingsHintQCOM value

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_HINT_ALREADY_SET_QCOM
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

12.191.3. Example of setting a tracking optimization hint

```
XrInstance instance;  // previously initialized
XrSession session;    // previously initialized

// Get function pointer for xrSetTrackingOptimizationSettingsHintQCOM
PFN_xrSetTrackingOptimizationSettingsHintQCOM pfnSetTrackingOptimizationSettingsHintQCOM;
CHK_XR(xrGetInstanceProcAddr(instance, "xrSetTrackingOptimizationSettingsHintQCOM",
    (PFN_xrVoidFunction*)(&pfnSetTrackingOptimizationSettingsHintQCOM)));

pfnSetTrackingOptimizationSettingsHintQCOM(session,
    XR_TRACKING_OPTIMIZATION_SETTINGS_DOMAIN_ALL_QCOM,
    XR_TRACKING_OPTIMIZATION_SETTINGS_HINT_LONG_RANGE_PRIORITIZATION_QCOM);

// perform tracking while prioritizing long range tracking
```

12.191.4. New Commands

- xrSetTrackingOptimizationSettingsHintQCOM

12.191.5. New Enums

- XrTrackingOptimizationSettingsDomainQCOM
- XrTrackingOptimizationSettingsHintQCOM

12.191.6. New Enum Constants

- `XR_QCOM_TRACKING_OPTIMIZATION_SETTINGS_EXTENSION_NAME`
- `XR_QCOM_tracking_optimization_settings_SPEC_VERSION`
- Extending XrResult:
  - `XR_ERROR_HINT_ALREADY_SET_QCOM`
- Extending XrTrackingOptimizationSettingsHintQCOM:
  - `XR_TRACKING_OPTIMIZATION_SETTINGS_HINT_CLOSE_RANGE_PRIORIZATION_QCOM`
  - `XR_TRACKING_OPTIMIZATION_SETTINGS_HINT_HIGH_POWER_PRIORIZATION_QCOM`
  - `XR_TRACKING_OPTIMIZATION_SETTINGS_HINT_LONG_RANGE_PRIORIZATION_QCOM`
  - `XR_TRACKING_OPTIMIZATION_SETTINGS_HINT_LOW_POWER_PRIORIZATION_QCOM`

12.191.7. Issues

12.191.8. Version History

- Revision 1, 2022-06-02
  - Initial extension description
- Revision 2, 2025-02-20 (Kurtis Eveleigh)
  - Fix typos: rename XrTrackingOptimizationSettingsHintQCOM enum values to `PRIORITIZATION`, providing the old names as compatibility aliases.

12.192. XR_SONY_swapchain_color_space

Name String

`XR_SONY_swapchain_color_space`

Extension Type

Instance extension

Registered Extension Number

777

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Last Modified Date

2026-02-10

IP Status

No known IP claims.

Contributors

Kota Suzuki, Sony Corporation
Kenji Tokutake, Sony Corporation

12.192.1. Overview

This extension allows applications to specify the color space of swapchain images and to enumerate the color spaces supported by the session.

12.192.2. Enumerating Color Spaces

The xrEnumerateColorSpacesSONY function is defined as:

```
// Provided by XR_SONY_swapchain_color_space
XrResult xrEnumerateColorSpacesSONY(
    XrSession                                   session,
    const XrColorSpacesEnumerateInfoSONY*       enumerateInfo,
    uint32_t                                    colorSpaceCapacityInput,
    uint32_t*                                   colorSpaceCountOutput,
    XrColorSpaceSONY*                           colorSpaces);
```

Parameter Descriptions

- `session` is the session that enumerates the supported color spaces.
- `enumerateInfo` is a pointer to an XrColorSpacesEnumerateInfoSONY that specifies the swapchain format to query.
- `colorSpaceCapacityInput` is the capacity of the `colorSpaces` array, or 0 to retrieve the required capacity.
- `colorSpaceCountOutput` is a pointer to the count of XrColorSpaceSONY `colorSpaces` written, or a pointer to the required capacity in the case that `colorSpaceCapacityInput` is insufficient.
- `colorSpaces` is a pointer to an array of XrColorSpaceSONY color spaces, but **can** be `NULL` if `colorSpaceCapacityInput` is 0.
- See the Buffer Size Parameters section for a detailed description of retrieving the required `colorSpaces` size.

xrEnumerateColorSpacesSONY enumerates the color spaces that the session supports for the swapchain format specified in the XrColorSpacesEnumerateInfoSONY.

The runtime **may** return different color spaces depending on the swapchain format.

The runtime **must** not return any color space other than `XR_COLOR_SPACE_SRGB_NONLINEAR_SONY` for API-specific sRGB formats (e.g. `DXGI_FORMAT_R8G8B8A8_UNORM_SRGB`, `GL_SRGB8_ALPHA8`, `VK_FORMAT_R8G8B8A8_SRGB`).

The runtime **must** return 0 in `colorSpaceCountOutput` for swapchain image formats that do not have a meaningful color space (e.g. depth or stencil image formats).

The runtime **must** return `XR_ERROR_SWAPCHAIN_FORMAT_UNSUPPORTED` if the swapchain format is unsupported.

The runtime **must** always return identical buffer contents for a given swapchain format from this enumeration for the lifetime of the session.

Valid Usage (Implicit)

- The `XR_SONY_swapchain_color_space` extension **must** be enabled prior to calling xrEnumerateColorSpacesSONY
- `session` **must** be a valid XrSession handle
- `enumerateInfo` **must** be a pointer to a valid XrColorSpacesEnumerateInfoSONY structure
- `colorSpaceCountOutput` **must** be a pointer to a `uint32_t` value
- If `colorSpaceCapacityInput` is not `0`, `colorSpaces` **must** be a pointer to an array of `colorSpaceCapacityInput` XrColorSpaceSONY values

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_SIZE_INSUFFICIENT
- XR_ERROR_SWAPCHAIN_FORMAT_UNSUPPORTED
- XR_ERROR_VALIDATION_FAILURE

The XrColorSpacesEnumerateInfoSONY structure is defined as:

```
// Provided by XR_SONY_swapchain_color_space
typedef struct XrColorSpacesEnumerateInfoSONY {
    XrStructureType    type;
    const void*        next;
    int64_t            format;
} XrColorSpacesEnumerateInfoSONY;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- format is the swapchain image format to query. format **must** be one of the formats enumerated by xrEnumerateSwapchainFormats.

XrColorSpacesEnumerateInfoSONY specifies the swapchain image format when calling xrEnumerateColorSpacesSONY.

Valid Usage (Implicit)

- The XR_SONY_swapchain_color_space extension **must** be enabled prior to using XrColorSpacesEnumerateInfoSONY
- type **must** be XR_TYPE_COLOR_SPACES_ENUMERATE_INFO_SONY
- next **must** be NULL or a valid pointer to the next structure in a structure chain

The possible color spaces are specified by the XrColorSpaceSONY enumeration.

```
// Provided by XR_SONY_swapchain_color_space
typedef enum XrColorSpaceSONY {
    XR_COLOR_SPACE_SRGB_NONLINEAR_SONY = 0,
    XR_COLOR_SPACE_DISPLAY_P3_LINEAR_SONY = 1,
    XR_COLOR_SPACE_DISPLAY_P3_NONLINEAR_SONY = 2,
    XR_COLOR_SPACE_DCI_P3_LINEAR_SONY = 3,
    XR_COLOR_SPACE_DCI_P3_NONLINEAR_SONY = 4,
    XR_COLOR_SPACE_EXTENDED_SRGB_LINEAR_SONY = 5,
    XR_COLOR_SPACE_BT709_LINEAR_SONY = 6,
    XR_COLOR_SPACE_BT709_NONLINEAR_SONY = 7,
    XR_COLOR_SPACE_BT2020_LINEAR_SONY = 8,
    XR_COLOR_SPACE_BT2020_PQ_SONY = 9,
    XR_COLOR_SPACE_BT2020_HLG_SONY = 10,
    XR_COLOR_SPACE_MAX_ENUM_SONY = 0x7FFFFFFF
} XrColorSpaceSONY;
```

The enumerants have the following values:

Enum	Description
XR_COLOR_SPACE_SRGB_NONLINEAR_SONY	sRGB color space, encoded according to the sRGB specification.
XR_COLOR_SPACE_DISPLAY_P3_LINEAR_SONY	Display-P3 color space, encoded using a linear transfer function.
XR_COLOR_SPACE_DISPLAY_P3_NONLINEAR_SONY	Display-P3 color space, encoded according to the Display-P3 specification.
XR_COLOR_SPACE_DCI_P3_LINEAR_SONY	DCI-P3 color space, encoded using a linear transfer function.
XR_COLOR_SPACE_DCI_P3_NONLINEAR_SONY	DCI-P3 color space, encoded according to the DCI-P3 specification.
XR_COLOR_SPACE_EXTENDED_SRGB_LINEAR_SONY	Extended sRGB color space, encoded using a linear transfer function.

Enum	Description
XR_COLOR_SPACE_BT709_LINEAR_SONY	ITU-R BT.709 color space, encoded using a linear transfer function.
XR_COLOR_SPACE_BT709_NONLINEAR_SONY	ITU-R BT.709 color space, encoded according to the BT.709 specification.
XR_COLOR_SPACE_BT2020_LINEAR_SONY	ITU-R BT.2020 color space, encoded using a linear transfer function.
XR_COLOR_SPACE_BT2020_PQ_SONY	ITU-R BT.2020 color space, encoded according to the SMPTE ST 2084 Perceptual Quantizer (PQ) specification.
XR_COLOR_SPACE_BT2020_HLG_SONY	ITU-R BT.2020 color space, encoded according to the Hybrid Log Gamma (HLG) specification.

12.192.3. Specifying Swapchain Color Space

The XrSwapchainCreateInfoColorSpaceSONY structure is defined as:

```
// Provided by XR_SONY_swapchain_color_space
typedef struct XrSwapchainCreateInfoColorSpaceSONY {
    XrStructureType     type;
    const void*         next;
    XrColorSpaceSONY    colorSpace;
} XrSwapchainCreateInfoColorSpaceSONY;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- colorSpace is the color space of this swapchain. colorSpace **must** be one of the color spaces enumerated by xrEnumerateColorSpacesSONY for the swapchain image format specified by XrSwapchainCreateInfo::format.

XrSwapchainCreateInfoColorSpaceSONY **can** be provided in the next chain of XrSwapchainCreateInfo when calling xrCreateSwapchain to specify the color space and encoding for the new swapchain.

The color space specified by XrSwapchainCreateInfoColorSpaceSONY does not affect the encoding of the alpha component.

The color space specified at creation time is constant for the lifetime of the swapchain.

The runtime **must** return XR_ERROR_VALIDATION_FAILURE from xrCreateSwapchain if the colorSpace is not enumerated by xrEnumerateColorSpacesSONY for the swapchain image format specified by XrSwapchainCreateInfo::format.

Valid Usage (Implicit)

- The XR_SONY_swapchain_color_space extension **must** be enabled prior to using XrSwapchainCreateInfoColorSpaceSONY
- type **must** be XR_TYPE_SWAPCHAIN_CREATE_INFO_COLOR_SPACE_SONY
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- colorSpace **must** be a valid XrColorSpaceSONY value

12.192.4. New Commands

- xrEnumerateColorSpacesSONY

12.192.5. New Structures

- XrColorSpacesEnumerateInfoSONY
- Extending XrSwapchainCreateInfo:
  - XrSwapchainCreateInfoColorSpaceSONY

12.192.6. New Enums

- XrColorSpaceSONY

12.192.7. New Enum Constants

- XR_SONY_SWAPCHAIN_COLOR_SPACE_EXTENSION_NAME
- XR_SONY_swapchain_color_space_SPEC_VERSION
- Extending XrStructureType:
  - XR_TYPE_COLOR_SPACES_ENUMERATE_INFO_SONY
  - XR_TYPE_SWAPCHAIN_CREATE_INFO_COLOR_SPACE_SONY

12.192.8. Issues

12.192.9. Version History

- Revision 1, 2026-02-10 (Kota Suzuki)
  - Initial draft

12.193. XR_ULTRALEAP_hand_tracking_forearm

Name String

XR_ULTRALEAP_hand_tracking_forearm

Extension Type

Instance extension

Registered Extension Number

150

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0
and
XR_EXT_hand_tracking

Last Modified Date

2022-04-19

IP Status

No known IP claims.

Contributors

Robert Blenkinsopp, Ultraleap
Adam Harwood, Ultraleap

Overview

This extension augments the XR_EXT_hand_tracking extension to enable applications to request the default set of 26 hand joints, with the addition of a joint representing the user's elbow.

The application **must** also enable the XR_EXT_hand_tracking extension in order to use this extension.

New joint set

This extension extends the XrHandJointSetEXT enumeration with a new member XR_HAND_JOINT_SET_HAND_WITH_FOREARM_ULTRALEAP. This joint set is the same as the XR_HAND_JOINT_SET_DEFAULT_EXT, plus a joint representing the user's elbow, XR_HAND_FOREARM_JOINT_ELBOW_ULTRALEAP.

```
// Provided by XR_ULTRALEAP_hand_tracking_forearm
typedef enum XrHandForearmJointULTRALEAP {
    XR_HAND_FOREARM_JOINT_PALM_ULTRALEAP = 0,
    XR_HAND_FOREARM_JOINT_WRIST_ULTRALEAP = 1,
    XR_HAND_FOREARM_JOINT_THUMB_METACARPAL_ULTRALEAP = 2,
    XR_HAND_FOREARM_JOINT_THUMB_PROXIMAL_ULTRALEAP = 3,
    XR_HAND_FOREARM_JOINT_THUMB_DISTAL_ULTRALEAP = 4,
    XR_HAND_FOREARM_JOINT_THUMB_TIP_ULTRALEAP = 5,
    XR_HAND_FOREARM_JOINT_INDEX_METACARPAL_ULTRALEAP = 6,
    XR_HAND_FOREARM_JOINT_INDEX_PROXIMAL_ULTRALEAP = 7,
    XR_HAND_FOREARM_JOINT_INDEX_INTERMEDIATE_ULTRALEAP = 8,
    XR_HAND_FOREARM_JOINT_INDEX_DISTAL_ULTRALEAP = 9,
    XR_HAND_FOREARM_JOINT_INDEX_TIP_ULTRALEAP = 10,
    XR_HAND_FOREARM_JOINT_MIDDLE_METACARPAL_ULTRALEAP = 11,
    XR_HAND_FOREARM_JOINT_MIDDLE_PROXIMAL_ULTRALEAP = 12,
    XR_HAND_FOREARM_JOINT_MIDDLE_INTERMEDIATE_ULTRALEAP = 13,
    XR_HAND_FOREARM_JOINT_MIDDLE_DISTAL_ULTRALEAP = 14,
    XR_HAND_FOREARM_JOINT_MIDDLE_TIP_ULTRALEAP = 15,
    XR_HAND_FOREARM_JOINT_RING_METACARPAL_ULTRALEAP = 16,
    XR_HAND_FOREARM_JOINT_RING_PROXIMAL_ULTRALEAP = 17,
    XR_HAND_FOREARM_JOINT_RING_INTERMEDIATE_ULTRALEAP = 18,
    XR_HAND_FOREARM_JOINT_RING_DISTAL_ULTRALEAP = 19,
    XR_HAND_FOREARM_JOINT_RING_TIP_ULTRALEAP = 20,
    XR_HAND_FOREARM_JOINT_LITTLE_METACARPAL_ULTRALEAP = 21,
    XR_HAND_FOREARM_JOINT_LITTLE_PROXIMAL_ULTRALEAP = 22,
    XR_HAND_FOREARM_JOINT_LITTLE_INTERMEDIATE_ULTRALEAP = 23,
    XR_HAND_FOREARM_JOINT_LITTLE_DISTAL_ULTRALEAP = 24,
    XR_HAND_FOREARM_JOINT_LITTLE_TIP_ULTRALEAP = 25,
    XR_HAND_FOREARM_JOINT_ELBOW_ULTRALEAP = 26,
    XR_HAND_FOREARM_JOINT_MAX_ENUM_ULTRALEAP = 0x7FFFFFFF
} XrHandForearmJointULTRALEAP;
```



Note

The first XR_HAND_JOINT_COUNT_EXT members of XrHandForearmJointULTRALEAP are identical to the members of XrHandJointEXT and **can** be used interchangeably.

The XR_HAND_FOREARM_JOINT_ELBOW_ULTRALEAP joint represents the center of an elbow and is orientated with the backwards (+Z) direction parallel to the forearm and points away from the hand.

The up (+Y) direction is pointing out of the dorsal side of the forearm. The X direction is perpendicular to Y and Z and follows the right hand rule.

```
// Provided by XR_ULTRALEAP_hand_tracking_forearm
#define XR_HAND_FOREARM_JOINT_COUNT_ULTRALEAP 27
```

XR_HAND_FOREARM_JOINT_COUNT_ULTRALEAP defines the number of hand joint enumerants defined in XrHandForearmJointULTRALEAP.

New Object Types

New Flag Types

New Enum Constants

- XR_HAND_FOREARM_JOINT_COUNT_ULTRALEAP

XrHandJointSetEXT enumeration is extended with:

- `XR_HAND_JOINT_SET_HAND_WITH_FOREARM_ULTRALEAP`

New Enums

- XrHandForearmJointULTRALEAP

New Structures

New Functions

Issues

Version History

- Revision 1, 2022-04-19 (Robert Blenkinsopp)
  - Initial version

12.194. XR_VALVE_analog_threshold

Name String

`XR_VALVE_analog_threshold`

Extension Type

Instance extension

Registered Extension Number

80

Revision

2

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Last Modified Date

2021-06-09

IP Status

No known IP claims.

Contributors

Joe Ludwig, Valve
Rune Berg, Valve
Andres Rodriguez, Valve

Overview

This extension allows the application to control the threshold and haptic feedback applied to an analog to digital conversion. See XrInteractionProfileAnalogThresholdVALVE for more information.

Applications **should** also enable the `XR_KHR_binding_modification` extension to be able to define multiple thresholds.

New Object Types

New Flag Types

New Enum Constants

New Enums

New Structures

The XrInteractionProfileAnalogThresholdVALVE structure is an input struct that defines thresholds and haptic feedback behavior for action bindings and **should** be added to the XrBindingModificationsKHR::`bindingModifications` array of the XrBindingModificationsKHR structure (See `XR_KHR_binding_modification` extension).

```
// Provided by XR_VALVE_analog_threshold
typedef struct XrInteractionProfileAnalogThresholdVALVE {
    XrStructureType             type;
    const void*                 next;
    XrAction                    action;
    XrPath                      binding;
    float                       onThreshold;
    float                       offThreshold;
    const XrHapticBaseHeader*   onHaptic;
    const XrHapticBaseHeader*   offHaptic;
} XrInteractionProfileAnalogThresholdVALVE;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `action` is the handle of an action in the suggested binding list.
- `binding` is the input path used for the specified action in the suggested binding list.
- `onThreshold` is the value between 0.0 and 1.0 at which the runtime **must** consider the binding to be true. The binding must remain true until the input analog value falls below `offThreshold`.
- `offThreshold` is the value between 0.0 and 1.0 at which the runtime **must** consider the binding to be false if it was previous true.
- `onHaptic` is the haptic output that the runtime **must** trigger when the binding changes from false to true. If this field is NULL, the runtime **must** not trigger any haptic output on the threshold. This field **can** point to any supported sub-type of XrHapticBaseHeader.
- `offHaptic` is the haptic output that the runtime **must** trigger when the binding changes from true to false. If this field is NULL, the runtime **must** not trigger any haptic output on the threshold. This field **can** point to any supported sub-type of XrHapticBaseHeader.

Applications can also chain a single XrInteractionProfileAnalogThresholdVALVE structure on the next chain of any xrSuggestInteractionProfileBindings call. Runtimes **must** support this kind of chaining. This method of specifying analog thresholds is deprecated however, and should not be used by any new applications.

If a threshold struct is present for a given conversion, the runtime **must** use those thresholds instead of applying its own whenever it is using the binding suggested by the application.

`onThreshold` and `offThreshold` permit allow the application to specify that it wants hysteresis to be applied to the threshold operation. If `onThreshold` is smaller than `offThreshold`, the runtime **must**

return XR_ERROR_VALIDATION_FAILURE.

onHaptic and offHaptic allow the application to specify that it wants automatic haptic feedback to be generated when the boolean output of the threshold operation changes from false to true or vice versa. If these fields are not NULL, the runtime **must** trigger a haptic output with the specified characteristics. If the device has multiple haptic outputs, the runtime **should** use the haptic output that is most appropriate for the specified input path.

If a suggested binding with action and binding is not in the binding list for this interaction profile, the runtime **must** return XR_ERROR_PATH_UNSUPPORTED.

Valid Usage (Implicit)

- The XR_VALVE_analog_threshold extension **must** be enabled prior to using XrInteractionProfileAnalogThresholdVALVE
- type **must** be XR_TYPE_INTERACTION_PROFILE_ANALOG_THRESHOLD_VALVE
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- action **must** be a valid XrAction handle
- If onHaptic is not NULL, onHaptic **must** be a pointer to a valid XrHapticBaseHeader-based structure. See also: XrHapticAmplitudeEnvelopeVibrationFB, XrHapticParametricVibrationEXT, XrHapticPcmVibrationFB, XrHapticVibration
- If offHaptic is not NULL, offHaptic **must** be a pointer to a valid XrHapticBaseHeader-based structure. See also: XrHapticAmplitudeEnvelopeVibrationFB, XrHapticParametricVibrationEXT, XrHapticPcmVibrationFB, XrHapticVibration

New Functions

Issues

Version History

- Revision 1, 2020-06-29 (Joe Ludwig)
  - Initial version.
- Revision 2, 2021-07-28 (Rune Berg)
  - Deprecate chaining of struct in XrInteractionProfileSuggestedBinding, applications should use XrBindingModificationsKHR defined in the XR_KHR_binding_modification extension instead.

12.195. XR_VARJO_composition_layer_depth_test

Name String

XR_VARJO_composition_layer_depth_test

Extension Type

Instance extension

Registered Extension Number

123

Revision

2

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0
and
`XR_KHR_composition_layer_depth`

Last Modified Date

2021-07-15

IP Status

No known IP claims.

Contributors

Sergiy Dubovik, Varjo Technologies
Antti Hirvonen, Varjo Technologies
Rémi Arnaud, Varjo Technologies

Overview

This extension enables depth-based layer composition inside the compositor.

Core OpenXR specifies that layer compositing must happen in the layer submission order (as described in Compositing). However, an application may want to composite the final image against the other layers based on depth information for proper occlusion. Layers can now provide depth information that will be used to calculate occlusion between those layers, as well as with the environment depth estimator (`XR_VARJO_environment_depth_estimation`) when enabled.

This extension defines a new type, XrCompositionLayerDepthTestVARJO, which can be chained to XrCompositionLayerProjection in order to activate this functionality. An application must also specify a range where depth testing will happen, potentially covering only a subset of the full depth range.

Composition

Layer composition rules change when this extension is enabled.

If the application does not chain XrCompositionLayerDepthTestVARJO, "painter's algorithm" such as described in Compositing must be used for layer composition.

Overall, composition should be performed in the following way:

1. Layers must be composited in the submission order. The compositor must track the depth value nearest to the virtual camera. Initial value for the nearest depth should be infinity.
2. If the currently processed layer does not contain depth, compositor should composite the layer against the previous layers with "painter's algorithm" and move to the next layer.
3. If the layer depth or the active nearest depth fall inside the depth test range of the layer, the compositor must perform depth test against the layer and active depth. If the layer depth is less or equal than the active depth, layer is composited normally with the previous layers and active depth is updated to match the layer depth. Otherwise the layer pixel is discarded, and compositor should move to composite the next layer.

Example

Mixed reality applications may want to show hands on top of the rendered VR content. For this purpose the application should enable environment depth estimation (see `XR_VARJO_environment_depth_estimation` extension) and depth testing with range 0m to 1m.

The following code illustrates how to enable depth testing:

```
XrCompositionLayerProjection layer; // previously populated

XrCompositionLayerDepthTestVARJO depthTest {
    .type = XR_TYPE_COMPOSITION_LAYER_DEPTH_TEST_VARJO,
    .next = layer.next,
    .depthTestRangeNearZ = 0.0f, // in meters
    .depthTestRangeFarZ = 1.0f, // in meters
};
layer.next = &depthTest;
```

New Structures

Applications **can** enable depth testing by adding XrCompositionLayerDepthTestVARJO to the `next` chain for all XrCompositionLayerProjectionView structures in the given layer in addition to XrCompositionLayerDepthInfoKHR. Missing XrCompositionLayerDepthInfoKHR automatically disables the depth testing functionality.

The XrCompositionLayerDepthTestVARJO structure is defined as:

```
// Provided by XR_VARJO_composition_layer_depth_test
typedef struct XrCompositionLayerDepthTestVARJO {
    XrStructureType    type;
    const void*        next;
    float              depthTestRangeNearZ;
    float              depthTestRangeFarZ;
} XrCompositionLayerDepthTestVARJO;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `depthTestRangeNearZ` in a non-negative distance in meters that specifies the lower bound of the range where depth testing should be performed. Must be less than `depthTestRangeFarZ`. Value of zero means that there is no lower bound.
- `depthTestRangeFarZ` is a positive distance in meters that specifies the upper bound of the range where depth testing should be performed. Must be greater than `depthTestRangeNearZ`. Value of floating point positive infinity means that there is no upper bound.

Valid Usage (Implicit)

- The `XR_VARJO_composition_layer_depth_test` extension **must** be enabled prior to using XrCompositionLayerDepthTestVARJO
- `type` **must** be `XR_TYPE_COMPOSITION_LAYER_DEPTH_TEST_VARJO`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

New Enum Constants

XrStructureType enumeration is extended with:

- `XR_TYPE_COMPOSITION_LAYER_DEPTH_TEST_VARJO`

Version History

- Revision 1, 2021-02-16 (Sergiy Dubovik)
  - Initial extension description
- Revision 2, 2021-07-15 (Rylie Pavlik, Collabora, Ltd., and Sergiy Dubovik)
  - Update sample code so it is buildable

12.196. XR_VARJO_environment_depth_estimation

Name String

XR_VARJO_environment_depth_estimation

Extension Type

Instance extension

Registered Extension Number

124

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Last Modified Date

2021-02-17

IP Status

No known IP claims.

Contributors

Sergiy Dubovik, Varjo Technologies
Antti Hirvonen, Varjo Technologies
Rémi Arnaud, Varjo Technologies

Overview

This extension provides a mechanism for enabling depth estimation of the environment in the runtime-supplied compositor. This is an extension to XR_ENVIRONMENT_BLEND_MODE_ALPHA_BLEND mode to not only use the color but also depth for composition of the final image.

Mixed reality applications might want to mix real and virtual content based on the depth information for proper occlusion. XR hardware and runtime may offer various ways to estimate the depth of the environment inside the compositor. When this estimation is enabled, the compositor can generate properly occluded final image when layers are submitted with depth information (both XR_KHR_composition_layer_depth and XR_VARJO_composition_layer_depth_test).

This extension defines a new function, xrSetEnvironmentDepthEstimationVARJO, which **can** be used to toggle environment depth estimation in the compositor. Toggling depth estimation is an asynchronous

operation and the feature **may** not be activated immediately. Function can be called immediately after the session is created. Composition of the environment layer follows the rules as described in `XR_VARJO_composition_layer_depth_test`.

New Structures

The xrSetEnvironmentDepthEstimationVARJO function is defined as:

```
// Provided by XR_VARJO_environment_depth_estimation
XrResult xrSetEnvironmentDepthEstimationVARJO(
    XrSession                                   session,
    XrBool32                                    enabled);
```

Parameter Descriptions

- `session` is an XrSession handle previously created with xrCreateSession.
- `enabled` is a boolean that specifies whether depth estimation functionality should be activated. Compositor will disable depth estimation functionality if environment blend mode is not `XR_ENVIRONMENT_BLEND_MODE_ALPHA_BLEND` and will enable the functionality when environment blend mode is set to `XR_ENVIRONMENT_BLEND_MODE_ALPHA_BLEND`.

Valid Usage (Implicit)

- The `XR_VARJO_environment_depth_estimation` extension **must** be enabled prior to calling xrSetEnvironmentDepthEstimationVARJO
- `session` **must** be a valid XrSession handle

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FEATURE_UNSUPPORTED
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST

New Functions

Version History

- Revision 1, 2021-02-16 (Sergiy Dubovik)
  - Initial extension description

12.197. XR_VARJO_foveated_rendering

Name String

XR_VARJO_foveated_rendering

Extension Type

Instance extension

Registered Extension Number

122

Revision

3

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0
and

XR_VARJO_quad_views

Last Modified Date

2021-04-13

IP Status

No known IP claims.

Contributors

Sergiy Dubovik, Varjo Technologies
Rémi Arnaud, Varjo Technologies
Antti Hirvonen, Varjo Technologies

12.197.1. Overview

Varjo headsets provide extremely high pixel density displays in the center area of the display, blended with a high density display covering the rest of the field of view. If the application has to provide a single image per eye, that would cover the entire field of view, at the highest density it would be extremely resource intensive, and in fact impossible for the most powerful desktop GPUs to render in real time. So instead Varjo introduced the XR_VARJO_quad_views extension enabling the application to provide two separate images for the two screen areas, resulting in a significant reduction in processing, for pixels that could not even been seen.

This extension goes a step further by enabling the application to only generate the density that can be seen by the user, which is another big reduction compared to the density that can be displayed, using dedicated eye tracking.

This extension requires XR_VARJO_quad_views extension to be enabled.

An application using this extension to enable foveated rendering will take the following steps to prepare:

1. Enable XR_VARJO_quad_views and XR_VARJO_foveated_rendering extensions.
2. Query system properties in order to determine if system supports foveated rendering.
3. Query texture sizes for foveated rendering.

In the render loop, for each frame, an application using this extension **should**

1. Check if rendering gaze is available using xrLocateSpace.
2. Enable foveated rendering when xrLocateViews is called.

12.197.2. Inspect system capability

An application **can** inspect whether the system is capable of foveated rendering by chaining an XrSystemFoveatedRenderingPropertiesVARJO structure to the XrSystemProperties structure when

calling xrGetSystemProperties.

```
// Provided by XR_VARJO_foveated_rendering
typedef struct XrSystemFoveatedRenderingPropertiesVARJO {
    XrStructureType    type;
    void*              next;
    XrBool32           supportsFoveatedRendering;
} XrSystemFoveatedRenderingPropertiesVARJO;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension. For more details about the structure chain, see the structure being extended (XrSystemProperties).
- `supportsFoveatedRendering` is an `XrBool32`, indicating if current system is capable of performoning foveated rendering.

The runtime **should** return `XR_TRUE` for `supportsFoveatedRendering` when rendering gaze is available in the system. An application **should** avoid using foveated rendering functionality when `supportsFoveatedRendering` is `XR_FALSE`.

Valid Usage (Implicit)

- The `XR_VARJO_foveated_rendering` extension **must** be enabled prior to using XrSystemFoveatedRenderingPropertiesVARJO
- `type` **must** be `XR_TYPE_SYSTEM_FOVEATED_RENDERING_PROPERTIES_VARJO`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

12.197.3. Determine foveated texture sizes

Foveated textures **may** have different sizes and aspect ratio compared to non-foveated textures. In order to determine recommended foveated texture size, an application **can** chain XrFoveatedViewConfigurationViewVARJO to XrViewConfigurationView and set `foveatedRenderingActive` to `XR_TRUE`. Since an application using foveated rendering with this extension has to render four views, `XR_VARJO_quad_views` **must** be enabled along with this extension when XrInstance is created.

First and second views are non foveated views (covering whole field of view of HMD), third (left eye) and fourth (right eye) are foveated e.g. following gaze.

```
// Provided by XR_VARJO_foveated_rendering
typedef struct XrFoveatedViewConfigurationViewVARJO {
    XrStructureType    type;
    void*              next;
    XrBool32           foveatedRenderingActive;
} XrFoveatedViewConfigurationViewVARJO;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `foveatedRenderingActive` is an `XrBool32`, indicating if the runtime should return foveated view configuration view.

Valid Usage (Implicit)

- The `XR_VARJO_foveated_rendering` extension **must** be enabled prior to using XrFoveatedViewConfigurationViewVARJO
- `type` **must** be `XR_TYPE_FOVEATED_VIEW_CONFIGURATION_VIEW_VARJO`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

For example:

```
XrInstance instance; // previously populated
XrSystemId systemId; // previously populated
XrViewConfigurationType viewConfigType; // Select
XR_VIEW_CONFIGURATION_TYPE_PRIMARY_QUAD_VARJO

XrSystemFoveatedRenderingPropertiesVARJO foveatedRenderingProperties {
  .type = XR_TYPE_SYSTEM_FOVEATED_RENDERING_PROPERTIES_VARJO,
};
XrSystemProperties systemProperties {
  .type = XR_TYPE_SYSTEM_PROPERTIES,
  .next = &foveatedRenderingProperties,
};
CHK_XR(xrGetSystemProperties(instance, systemId, &systemProperties));

uint32_t viewCount = 0;
CHK_XR(xrEnumerateViewConfigurationViews(instance, systemId, viewConfigType, 0,
&viewCount, nullptr));
// Non-foveated rendering views dimensions
std::vector<XrViewConfigurationView> configViews(viewCount, {
  .type = XR_TYPE_VIEW_CONFIGURATION_VIEW,
});
CHK_XR(xrEnumerateViewConfigurationViews(instance, systemId, viewConfigType, viewCount,
&viewCount, configViews.data()));

// Foveated rendering views dimensions
std::vector<XrViewConfigurationView> foveatedViews;
if (foveatedRenderingProperties.supportsFoveatedRendering && viewConfigType ==
XR_VIEW_CONFIGURATION_TYPE_PRIMARY_QUAD_VARJO) {
  std::vector<XrFoveatedViewConfigurationViewVARJO> requestFoveatedConfig(4, {
    .type = XR_TYPE_FOVEATED_VIEW_CONFIGURATION_VIEW_VARJO,
    .next = nullptr,
    .foveatedRenderingActive = XR_TRUE,
  });
  foveatedViews = std::vector<XrViewConfigurationView>(4, {
    .type = XR_TYPE_VIEW_CONFIGURATION_VIEW,
  });
  for (size_t i = 0; i < 4; i++) {
    foveatedViews[i].next = &requestFoveatedConfig[i];
  }
  CHK_XR(xrEnumerateViewConfigurationViews(instance, systemId, viewConfigType, viewCount,
&viewCount, foveatedViews.data()));
}
```

Example 3. Note

Applications using this extension are encouraged to create two sets of swapchains or one big enough set of swapchains and two sets of viewports. One set will be used when rendering gaze is not available and other one will be used when foveated rendering and rendering gaze is available. Using foveated textures **may** not provide optimal visual quality when rendering gaze is not available.

12.197.4. Rendering gaze status

Extension defines new reference space type - `XR_REFERENCE_SPACE_TYPE_COMBINED_EYE_VARJO` which **should** be used to determine whether rendering gaze is available. After calling xrLocateSpace, application **should** inspect `XR_SPACE_LOCATION_ORIENTATION_TRACKED_BIT` bit. If it's set, rendering gaze is available otherwise not.

```
XrSession session; // previously populated

// Create needed spaces
XrSpace viewSpace {XR_NULL_HANDLE};
XrReferenceSpaceCreateInfo createViewSpaceInfo {
  .type = XR_TYPE_REFERENCE_SPACE_CREATE_INFO,
  .referenceSpaceType = XR_REFERENCE_SPACE_TYPE_VIEW,
  .poseInReferenceSpace = {
    .orientation = {
      .w = 1.0f,
    },
  },
};
CHK_XR(xrCreateReferenceSpace(session, &createViewSpaceInfo, &viewSpace));

XrSpace renderGazeSpace {XR_NULL_HANDLE};
XrReferenceSpaceCreateInfo createReferenceSpaceInfo {
  .type = XR_TYPE_REFERENCE_SPACE_CREATE_INFO,
  .referenceSpaceType = XR_REFERENCE_SPACE_TYPE_COMBINED_EYE_VARJO,
  .poseInReferenceSpace = {
    .orientation = {
      .w = 1.0f,
    },
  },
};
CHK_XR(xrCreateReferenceSpace(session, &createReferenceSpaceInfo, &renderGazeSpace));

// ...
// in frame loop
// ...
```

```
XrFrameState frameState; // previously populated by xrWaitFrame

// Query rendering gaze status
XrSpaceLocation renderGazeLocation {
  .type = XR_TYPE_SPACE_LOCATION,
};
CHK_XR(xrLocateSpace(renderGazeSpace, viewSpace, frameState.predictedDisplayTime,
&renderGazeLocation));

const bool foveationActive = (renderGazeLocation.locationFlags &
XR_SPACE_LOCATION_ORIENTATION_TRACKED_BIT) != 0;

if (foveationActive) {
  // Rendering gaze is available
} else {
  // Rendering gaze is not available
}
```

12.197.5. Request foveated field of view

For each frame, the application indicates if the runtime will return foveated or non-foveated field of view. This is done by chaining XrViewLocateFoveatedRenderingVARJO to XrViewLocateInfo.

```
// Provided by XR_VARJO_foveated_rendering
typedef struct XrViewLocateFoveatedRenderingVARJO {
    XrStructureType    type;
    const void*        next;
    XrBool32           foveatedRenderingActive;
} XrViewLocateFoveatedRenderingVARJO;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `foveatedRenderingActive` is an `XrBool32`, indicating if runtime should return foveated FoV.

The runtime **must** return foveated field of view when `foveatedRenderingActive` is `XR_TRUE`.

Valid Usage (Implicit)

- The XR_VARJO_foveated_rendering extension **must** be enabled prior to using XrViewLocateFoveatedRenderingVARJO
- type **must** be XR_TYPE_VIEW_LOCATE_FOVEATED_RENDERING_VARJO
- next **must** be NULL or a valid pointer to the next structure in a structure chain

```
// ...
// in frame loop
// ...

XrSession session; // previously populated
XrSpace appSpace; // previously populated
XrFrameState frameState; // previously populated by xrWaitFrame
XrViewConfigurationType viewConfigType; // previously populated
std::vector<XrView> views; // previously populated/resized to the correct size
bool foveationActive; // previously populated, as in the previous example

XrViewState viewState {
  .type = XR_TYPE_VIEW_STATE,
};
uint32_t viewCapacityInput = static_cast<uint32_t>(views.size());
uint32_t viewCountOutput;

XrViewLocateFoveatedRenderingVARJO viewLocateFoveatedRendering {
  .type = XR_TYPE_VIEW_LOCATE_FOVEATED_RENDERING_VARJO,
  .foveatedRenderingActive = foveationActive,
};

XrViewLocateInfo viewLocateInfo {
  .type = XR_TYPE_VIEW_LOCATE_INFO,
  .next = &viewLocateFoveatedRendering,
  .viewConfigurationType = viewConfigType,
  .displayTime = frameState.predictedDisplayTime,
  .space = appSpace,
};

CHK_XR(xrLocateViews(session, &viewLocateInfo, &viewState, viewCapacityInput,
&viewCountOutput, views.data()));
```

New Structures

- XrViewLocateFoveatedRenderingVARJO

- XrFoveatedViewConfigurationViewVARJO
- XrSystemFoveatedRenderingPropertiesVARJO

New Enum Constants

XrStructureType enumeration is extended with:

- `XR_TYPE_VIEW_LOCATE_FOVEATED_RENDERING_VARJO`
- `XR_TYPE_FOVEATED_VIEW_CONFIGURATION_VIEW_VARJO`
- `XR_TYPE_SYSTEM_FOVEATED_RENDERING_PROPERTIES_VARJO`

XrReferenceSpaceType enumeration is extended with:

- `XR_REFERENCE_SPACE_TYPE_COMBINED_EYE_VARJO`

Version History

- Revision 1, 2020-12-16 (Sergiy Dubovik)
  - Initial extension description
- Revision 2, 2021-04-13 (Rylie Pavlik, Collabora, Ltd., and Sergiy Dubovik)
  - Update sample code so it is buildable
- Revision 3, 2022-02-21 (Denny Rönngren)
  - Update sample code with a missing struct field initialization

12.198. XR_VARJO_marker_tracking

Name String

`XR_VARJO_marker_tracking`

Extension Type

Instance extension

Registered Extension Number

125

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Last Modified Date

2021-09-30

IP Status

No known IP claims.

Contributors

Roman Golovanov, Varjo Technologies
Rémi Arnaud, Varjo Technologies
Sergiy Dubovik, Varjo Technologies

12.198.1. Overview

Varjo Markers are physical markers tracked by the video cameras of the HMD. Different types of markers **can** be used for different purposes. As an example, Varjo Markers **can** be used as cheap replacements for electronic trackers. The cost per printed tracker is significantly lower and the markers require no power to function.

This extension provides the tracking interface to a set of marker types and sizes. Markers **can** be printed out from the PDF documents and instructions freely available at https://developer.varjo.com/docs/get-started/varjo-markers#printing-varjo-markers. Note that the printed marker **must** have the exact physical size for its ID.

Object markers are used to track static or dynamic objects in the user environment. You **may** use object markers in both XR and VR applications. Each marker has a unique ID, and you **must** not use the same physical marker more than once in any given environment. For added precision, an application **may** use multiple markers to track a single object. For example, you could track a monitor by placing a marker in each corner.

There is a set of marker IDs recognized by runtime and if the application uses ID which is not in the set then runtime **must** return `XR_ERROR_MARKER_ID_INVALID_VARJO`.

New Object Types

New Flag Types

New Enums

New Functions

The xrSetMarkerTrackingVARJO function is defined as:

```
// Provided by XR_VARJO_marker_tracking
XrResult xrSetMarkerTrackingVARJO(
    XrSession                                   session,
    XrBool32                                    enabled);
```

Parameter Descriptions

- `session` is an XrSession handle previously created with xrCreateSession.
- `enabled` is the flag to enable or disable marker tracking.

The xrSetMarkerTrackingVARJO function enables or disables marker tracking functionality. As soon as feature is become disabled all trackable markers become inactive and corresponding events will be generated. An application **may** call any of the functions in this extension regardless if the marker tracking functionality is enabled or disabled.

Valid Usage (Implicit)

- The `XR_VARJO_marker_tracking` extension **must** be enabled prior to calling xrSetMarkerTrackingVARJO
- `session` **must** be a valid XrSession handle

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_FEATURE_UNSUPPORTED`
- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`

The xrSetMarkerTrackingTimeoutVARJO function is defined as:

```
// Provided by XR_VARJO_marker_tracking
XrResult xrSetMarkerTrackingTimeoutVARJO(
    XrSession                                   session,
    uint64_t                                    markerId,
    XrDuration                                  timeout);
```

Parameter Descriptions

- `session` is an XrSession handle previously created with xrCreateSession.
- `markerId` is the unique identifier of the marker for which the timeout will be updated.
- `timeout` is the desired lifetime duration for a specified marker.

The xrSetMarkerTrackingTimeoutVARJO function sets a desired lifetime duration for a specified marker. The default value is XR_NO_DURATION. Negative value will be clamped to XR_NO_DURATION. It defines the time period during which the runtime **must** keep returning poses of previously tracked markers. The tracking may be lost if the marker went outside of the trackable field of view. In this case the runtime still will try to predict marker's pose for the timeout period. The runtime **must** return `XR_ERROR_MARKER_ID_INVALID_VARJO` if the supplied `markerId` is invalid.

Valid Usage (Implicit)

- The `XR_VARJO_marker_tracking` extension **must** be enabled prior to calling xrSetMarkerTrackingTimeoutVARJO
- `session` **must** be a valid XrSession handle

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FEATURE_UNSUPPORTED
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_MARKER_ID_INVALID_VARJO
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST

The xrSetMarkerTrackingPredictionVARJO function is defined as:

```
// Provided by XR_VARJO_marker_tracking
XrResult xrSetMarkerTrackingPredictionVARJO(
    XrSession                                   session,
    uint64_t                                    markerId,
    XrBool32                                    enable);
```

Parameter Descriptions

- session is an XrSession handle previously created with xrCreateSession.
- markerId is the unique identifier of the marker which should be tracked with prediction.
- enable is whether to enable the prediction feature.

The xrSetMarkerTrackingPredictionVARJO function enables or disables the prediction feature for a specified marker. By default, markers are created with disabled prediction. This works well for markers that are supposed to be stationary. The prediction **can** be used to improve tracking of movable markers. The runtime **must** return XR_ERROR_MARKER_ID_INVALID_VARJO if the supplied markerId is invalid.

Valid Usage (Implicit)

- The XR_VARJO_marker_tracking extension **must** be enabled prior to calling xrSetMarkerTrackingPredictionVARJO
- session **must** be a valid XrSession handle

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FEATURE_UNSUPPORTED
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_MARKER_ID_INVALID_VARJO
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST

The xrGetMarkerSizeVARJO function is defined as:

```
// Provided by XR_VARJO_marker_tracking
XrResult xrGetMarkerSizeVARJO(
    XrSession                                   session,
    uint64_t                                    markerId,
    XrExtent2Df*                                size);
```

Parameter Descriptions

- session is an XrSession handle previously created with xrCreateSession.
- markerId is the unique identifier of the marker for which size is requested.
- size is pointer to the size to populate by the runtime with the physical size of plane marker in meters.

The xrGetMarkerSizeVARJO function retrieves the height and width of an active marker. The runtime

must return XR_ERROR_MARKER_NOT_TRACKED_VARJO if marker tracking functionality is disabled or the marker with given markerId is inactive. The runtime **must** return XR_ERROR_MARKER_ID_INVALID_VARJO if the supplied markerId is invalid.

Valid Usage (Implicit)

- The XR_VARJO_marker_tracking extension **must** be enabled prior to calling xrGetMarkerSizeVARJO
- session **must** be a valid XrSession handle
- size **must** be a pointer to an XrExtent2Df structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FEATURE_UNSUPPORTED
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_MARKER_ID_INVALID_VARJO
- XR_ERROR_MARKER_NOT_TRACKED_VARJO
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST

The xrCreateMarkerSpaceVARJO function is defined as:

```
// Provided by XR_VARJO_marker_tracking
XrResult xrCreateMarkerSpaceVARJO(
    XrSession                                   session,
    const XrMarkerSpaceCreateInfoVARJO*         createInfo,
    XrSpace*                                    space);
```

Parameter Descriptions

- `session` is an XrSession handle previously created with xrCreateSession.
- `createInfo` is the structure containing information about how to create the space based on marker.
- `space` is a pointer to a handle in which the created XrSpace is returned.

The xrCreateMarkerSpaceVARJO function creates marker XrSpace for pose relative to the marker specified in XrMarkerSpaceCreateInfoVARJO. The runtime **must** return `XR_ERROR_MARKER_ID_INVALID_VARJO` if the supplied XrMarkerSpaceCreateInfoVARJO::`markerId` is invalid.

Valid Usage (Implicit)

- The `XR_VARJO_marker_tracking` extension **must** be enabled prior to calling xrCreateMarkerSpaceVARJO
- `session` **must** be a valid XrSession handle
- `createInfo` **must** be a pointer to a valid XrMarkerSpaceCreateInfoVARJO structure
- `space` **must** be a pointer to an XrSpace handle

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FEATURE_UNSUPPORTED
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_MARKER_ID_INVALID_VARJO
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_POSE_INVALID
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

New Structures

The XrSystemMarkerTrackingPropertiesVARJO structure is defined as:

```
// Provided by XR_VARJO_marker_tracking
typedef struct XrSystemMarkerTrackingPropertiesVARJO {
    XrStructureType    type;
    void*              next;
    XrBool32           supportsMarkerTracking;
} XrSystemMarkerTrackingPropertiesVARJO;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain.
- supportsMarkerTracking is an XrBool32, indicating if current system is capable of performing marker tracking.

An application **may** inspect whether the system is capable of marker tracking by chaining an XrSystemMarkerTrackingPropertiesVARJO structure to the XrSystemProperties structure when calling xrGetSystemProperties.

The runtime **should** return XR_TRUE for supportsMarkerTracking when marker tracking is available in the system, otherwise XR_FALSE. Marker tracking calls **must** return XR_ERROR_FEATURE_UNSUPPORTED if marker tracking is not available in the system.

Valid Usage (Implicit)

- The XR_VARJO_marker_tracking extension **must** be enabled prior to using XrSystemMarkerTrackingPropertiesVARJO
- type **must** be XR_TYPE_SYSTEM_MARKER_TRACKING_PROPERTIES_VARJO
- next **must** be NULL or a valid pointer to the next structure in a structure chain

The XrEventDataMarkerTrackingUpdateVARJO structure is defined as:

```
// Provided by XR_VARJO_marker_tracking
typedef struct XrEventDataMarkerTrackingUpdateVARJO {
    XrStructureType    type;
    const void*        next;
    uint64_t           markerId;
    XrBool32           isActive;
    XrBool32           isPredicted;
    XrTime             time;
} XrEventDataMarkerTrackingUpdateVARJO;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- markerId unique identifier of the marker that has been updated.
- isActive the tracking state of the marker.
- isPredicted the prediction state of the marker.
- time the time of the marker update.

Receiving the XrEventDataMarkerTrackingUpdateVARJO event structure indicates that the tracking information has changed. The runtime **must** not send more than one event per frame per marker. The runtime **must** send an event if the marker has changed its state (active or inactive). The runtime **must**

send an event if it has detected pose change of the active marker.

Valid Usage (Implicit)

- The XR_VARJO_marker_tracking extension **must** be enabled prior to using XrEventDataMarkerTrackingUpdateVARJO
- type **must** be XR_TYPE_EVENT_DATA_MARKER_TRACKING_UPDATE_VARJO
- next **must** be NULL or a valid pointer to the next structure in a structure chain

The XrMarkerSpaceCreateInfoVARJO structure is defined as:

```
// Provided by XR_VARJO_marker_tracking
typedef struct XrMarkerSpaceCreateInfoVARJO {
    XrStructureType    type;
    const void*        next;
    uint64_t           markerId;
    XrPosef            poseInMarkerSpace;
} XrMarkerSpaceCreateInfoVARJO;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- markerId unique identifier of the marker.
- poseInMarkerSpace is an XrPosef defining the position and orientation of the new space's origin relative to the marker's natural origin.

Valid Usage (Implicit)

- The XR_VARJO_marker_tracking extension **must** be enabled prior to using XrMarkerSpaceCreateInfoVARJO
- type **must** be XR_TYPE_MARKER_SPACE_CREATE_INFO_VARJO
- next **must** be NULL or a valid pointer to the next structure in a structure chain

New Enum Constants

XrStructureType enumeration is extended with:

- XR_TYPE_SYSTEM_MARKER_TRACKING_PROPERTIES_VARJO
- XR_TYPE_EVENT_DATA_MARKER_TRACKING_UPDATE_VARJO
- XR_TYPE_MARKER_SPACE_CREATE_INFO_VARJO

XrResult enumeration is extended with:

- XR_ERROR_MARKER_ID_INVALID_VARJO
- XR_ERROR_MARKER_NOT_TRACKED_VARJO

Issues

Version History

- Revision 1, 2021-09-30 (Roman Golovanov)
  - Initial extension description

12.198.2. Example

The example below represents the routine which enables marker tracking feature and then polls events. The event type XR_TYPE_EVENT_DATA_MARKER_TRACKING_UPDATE_VARJO has a special handler to process marker state change.

```
XrSession session; // previously initialized
if(XR_SUCCESS != xrSetMarkerTrackingVARJO(session, XR_TRUE)) {
    return;
}

XrInstance instance; // previously initialized
XrFrameState frameState; // previously initialized
XrSpace baseSpace; // previously initialized
XrSpaceLocation location; // previously initialized

// Collection of tracked markers and their space handlers
std::unordered_map<uint64_t, XrSpace> markerSpaces;
// Initialize an event buffer to hold the output.
XrEventDataBuffer event {
    .type = XR_TYPE_EVENT_DATA_BUFFER,
};
XrResult result = xrPollEvent(instance, &event);
if (result == XR_SUCCESS) {
    switch (event.type) {
        case XR_TYPE_EVENT_DATA_MARKER_TRACKING_UPDATE_VARJO: {
            const auto& marker_update =
              *reinterpret_cast<XrEventDataMarkerTrackingUpdateVARJO*>(&event);

```

```
            const auto id = marker_update.markerId;

            // If marker appeared for the first time then set some settings and
            // add it to collection
            if(0 == markerSpaces.count(id)) {
                XrMarkerSpaceCreateInfoVARJO spaceInfo {
                    .type = XR_TYPE_MARKER_SPACE_CREATE_INFO_VARJO,
                    .markerId = id,
                    .poseInMarkerSpace = {
                        .orientation = {
                            .w = 1.0f,
                        },
                    },
                };
                XrSpace markerSpace {XR_NULL_HANDLE};
                // Set 1 second timeout
                if(XR_SUCCESS != xrSetMarkerTrackingTimeoutVARJO(
                                    session, id, 1000000000))
                {
                    break;
                }
                // Enable prediction for markers with `odd` ids.
                if(XR_SUCCESS != xrSetMarkerTrackingPredictionVARJO(
                                    session, id, id % 2))
                {
                    break;
                }
                if(XR_SUCCESS != xrCreateMarkerSpaceVARJO(session, &spaceInfo,
                                    &markerSpace)) {
                    break;
                }
                markerSpaces[id] = markerSpace;
            }

            if(marker_update.isActive) {
                if(XR_SUCCESS != xrLocateSpace(markerSpaces.at(id), baseSpace,
                                    frameState.predictedDisplayTime, &location)){
                   break;
                }
                if(marker_update.isPredicted) {
                    // Process marker as dynamic
                } else {
                    // Process marker as stationary
                }

            } else {
                // Remove previously tracked marker
                markerSpaces.erase(id);
```

```
72             }
73
74             // ...
75             break;
76         }
77     }
78 }
```

12.199. XR_VARJO_view_offset

Name String

XR_VARJO_view_offset

Extension Type

Instance extension

Registered Extension Number

126

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Last Modified Date

2021-09-30

IP Status

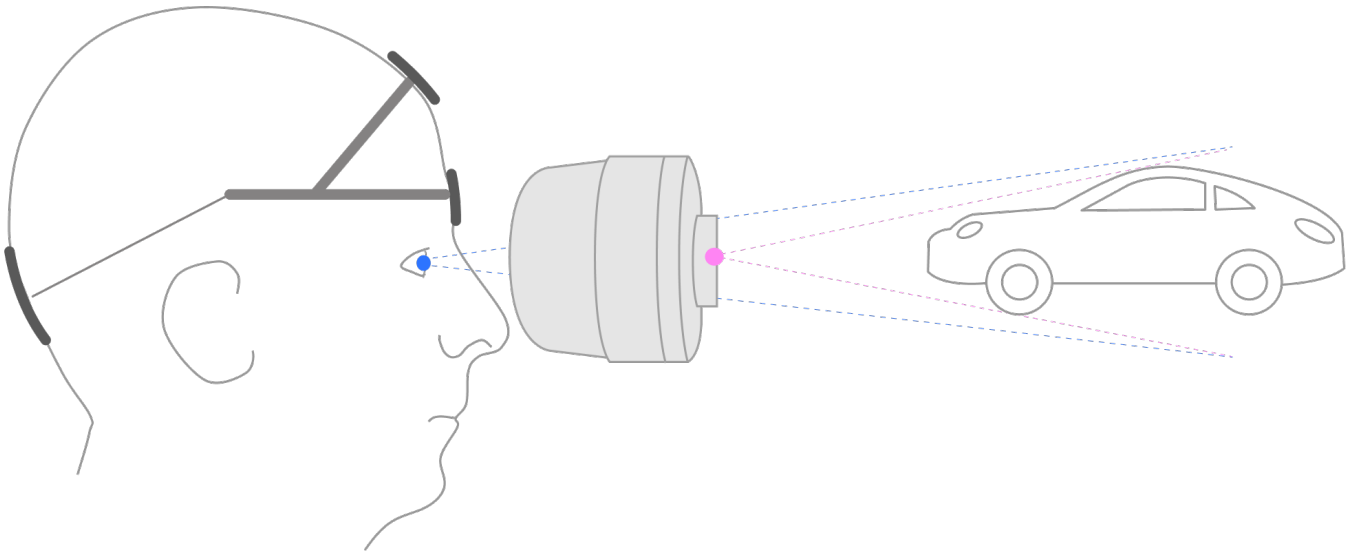
No known IP claims.

Contributors

Rémi Arnaud, Varjo Technologies

Overview

Varjo headsets use video pass-through cameras to create the mixed reality (MR) image. The cameras are located around 10 cm (3.9 inches) in front of the user's eyes, which leads to an offset in depth perception so that real-world objects in the video pass-through image appear larger than they are in real life. The image below gives a visualization of the difference between what the camera sees and what the user would see in real life.



This magnification effect is pronounced for objects that are close to the user – for example, their hands may appear unnaturally large in the image. The effect decreases with distance, so that objects at a distance of 2 meters already appear close to their actual size, and the sizes eventually converge at infinity. Note that while the objects' sizes may differ, their geometry, relative sizes, locations, etc. remain accurate. The extent of the magnification effect ultimately depends both on the application itself and the user's physiology, as the human visual system is highly adaptive in this type of setting.

When blending the video pass-through image with virtual content, it is important that their relative geometries – position, size, and disparity – match one another. To achieve this, Varjo's runtime automatically places the virtual reality cameras in the same position as the physical cameras when the video pass-through feature is enabled (see `XR_ENVIRONMENT_BLEND_MODE_ALPHA_BLEND`). This allows virtual and real-world content to appear at the same distance and on the same plane when viewed together. While this can be observed as an apparent jump in the location of virtual objects compared to VR-only content, this does not cause any distortion in the object geometry or location; it is only the viewer's location that changes.

In some cases, moving the VR content to match the real-world position may not be desirable. This extension enable the application to control where the VR content is rendered from the location of the user's eyes while the video pass-through image uses the camera locations. For example, if the virtual object is close the user, or if the application is switching between VR and MR modes. Offset values between 0.0 and 1.0 are supported. You can use this to create a smooth, animated transition between the two rendering positions in case you need to change from one to the other during a session.

New Functions

The xrSetViewOffsetVARJO function is defined as:

```
// Provided by XR_VARJO_view_offset
XrResult xrSetViewOffsetVARJO(
    XrSession                                   session,
    float                                       offset);
```

Parameter Descriptions

- `session` is an XrSession handle previously created with xrCreateSession.
- `offset` is the view offset to be applied. Must be between 0 and 1.

The xrSetViewOffsetVARJO function takes a float between 0.0 and 1.0. 0.0 means the pose returned by xrLocateViews will be at the eye location, a value of 1.0 means the pose will be at the camera location. A value between 0.0 and 1.0 will interpolate the pose to be in between the eye and the camera location. A value less than 0.0 or more than 1.0 will fail and return error `XR_ERROR_VALIDATION_FAILURE`.

Note that by default the offset is set to 0 if the pass-through cameras are not active, a.k.a. in VR (`XR_ENVIRONMENT_BLEND_MODE_OPAQUE`), and 1 if the cameras are active, a.k.a. in MR (`XR_ENVIRONMENT_BLEND_MODE_ALPHA_BLEND` or `XR_ENVIRONMENT_BLEND_MODE_ADDITIVE`).

Valid Usage (Implicit)

- The `XR_VARJO_view_offset` extension **must** be enabled prior to calling xrSetViewOffsetVARJO
- `session` **must** be a valid XrSession handle

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FEATURE_UNSUPPORTED
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

Version History

- Revision 1, 2022-02-08 (Remi Arnaud)
  - extension specification

12.200. XR_VARJO_xr4_controller_interaction

Name String

XR_VARJO_xr4_controller_interaction

Extension Type

Instance extension

Registered Extension Number

130

Revision

2

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

API Interactions

- Interacts with `XR_EXT_dpad_binding`
- Interacts with `XR_EXT_hand_interaction`
- Interacts with `XR_EXT_palm_pose`

Last Modified Date

2024-09-17

IP Status

No known IP claims.

Contributors

Denny Rönngren, Varjo Technologies
Szymon Policht, Varjo Technologies
Roman Golovanov, Varjo Technologies
Jussi Karhu, Varjo Technologies
Fabian Wahlster, Varjo Technologies

Overview

This extension adds a new interaction profile for the Varjo Controllers compatible with the Varjo XR-4 headset.

Interaction profile path:

- */interaction_profiles/varjo/xr-4_controller*

> **Note**
>
> The interaction profile path */interaction_profiles/varjo/xr-4_controller* defined here does not follow current rules for interaction profile names. If this extension were introduced today, it would be called */interaction_profiles/varjo/xr-4_controller_varjo*, to allow for modifications when promoted to a KHR extension or the core specification.

Valid for the user paths:

- */user/hand/left*
- */user/hand/right*

Supported component paths for */user/hand/left* only:

- *.../input/menu/click*

Supported component paths for */user/hand/right* only:

- *.../input/system/click* (**may** not be available for application use)

Supported component paths on both pathnames:

- *.../input/a/click*
- *.../input/a/touch*
- *.../input/b/click*
- *.../input/b/touch*
- *.../input/squeeze/click*
- *.../input/squeeze/touch*
- *.../input/trigger/value*
- *.../input/trigger/touch*
- *.../input/thumbstick/x*
- *.../input/thumbstick/y*
- *.../input/thumbstick/click*
- *.../input/thumbstick/touch*
- *.../input/grip/pose*
- *.../input/aim/pose*
- *.../output/haptic*



Note

When the runtime supports `XR_VERSION_1_1` and use of OpenXR 1.1 is requested by the application, this interaction profile **must** also support

- *.../input/grip_surface/pose*



Note

When the `XR_KHR_maintenance1` extension is available and enabled, this interaction profile **must** also support

- *.../input/grip_surface/pose*



Note

When the `XR_EXT_palm_pose` extension is available and enabled, this interaction profile **must** also support

- *.../input/palm_ext/pose*

New Object Types

New Flag Types

New Enum Constants

New Enums

New Structures

New Functions

Issues

Version History

- Revision 1, 2023-12-06 (Denny Rönngren)
  - Initial extension description
- Revision 2, 2024-09-17 (Fabian Wahlster)
  - Interacting extensions description

12.201. XR_YVR_controller_interaction

Name String

`XR_YVR_controller_interaction`

Extension Type

Instance extension

Registered Extension Number

498

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

API Interactions

- Interacts with `XR_EXT_dpad_binding`
- Interacts with `XR_EXT_hand_interaction`
- Interacts with `XR_EXT_palm_pose`

Last Modified Date

2023-07-12

IP Status

No known IP claims.

Contributors

Pengpeng Zhang, YVR
Xuanyu Chen, YVR

Overview

This extension defines a new interaction profile for the YVR Controller, including but not limited to YVR1 and YVR2 Controller.

YVR Controller interaction profile

Interaction profile path:

- */interaction_profiles/yvr/touch_controller_yvr*

Valid for user paths:

- */user/hand/left*
- */user/hand/right*

This interaction profile represents the input sources and haptics on the YVR Controller.

Supported component paths:

- On */user/hand/left* only:
  - *.../input/x/click*
  - *.../input/x/touch*
  - *.../input/y/click*
  - *.../input/y/touch*
  - *.../input/menu/click*
- On */user/hand/right* only:
  - *.../input/a/click*
  - *.../input/a/touch*
  - *.../input/b/click*
  - *.../input/b/touch*
  - *.../input/system/click* (**may** not be available for application use)

- On both:
  - *.../input/squeeze/click*
  - *.../input/trigger/value*
  - *.../input/trigger/touch*
  - *.../input/thumbstick/x*
  - *.../input/thumbstick/y*
  - *.../input/thumbstick/click*
  - *.../input/thumbstick/touch*
  - *.../input/grip/pose*
  - *.../input/aim/pose*
  - *.../output/haptic*



Note

When the runtime supports `XR_VERSION_1_1` and use of OpenXR 1.1 is requested by the application, this interaction profile **must** also support

- *.../input/grip_surface/pose*



Note

When the `XR_KHR_maintenance1` extension is available and enabled, this interaction profile **must** also support

- *.../input/grip_surface/pose*



Note

When the `XR_EXT_palm_pose` extension is available and enabled, this interaction profile **must** also support

- *.../input/palm_ext/pose*



Note

When the `XR_EXT_hand_interaction` extension is available and enabled, this interaction profile **must** also support

- *.../input/pinch_ext/pose*
- *.../input/poke_ext/pose*

New Object Types

New Flag Types

New Enum Constants

New Enums

New Structures

New Functions

Issues

Version History

- Revision 1, 2023-07-12 (Pengpeng Zhang)
  - Initial extension description

Chapter 13. List of Provisional Extensions

- XR_EXTX_overlay
- XR_HTCX_vive_tracker_interaction
- XR_MNDX_egl_enable
- XR_MNDX_force_feedback_curl

13.1. XR_EXTX_overlay

Name String

`XR_EXTX_overlay`

Extension Type

Instance extension

Registered Extension Number

34

Revision

5

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Last Modified Date

2021-01-13

IP Status

No known IP claims.

Contributors

Mark Young, LunarG
Jules Blok, Epic
Jared Cheshier, Pluto VR
Nick Whiting, Epic
Brad Grantham, LunarG

Overview

Application developers may desire to implement an OpenXR application that renders content on top of another OpenXR application. These additional applications will execute in a separate process, create a separate session, generate separate content, but want the OpenXR runtime to composite their content on top of the main OpenXR application. Examples of these applications might include:

- A debug environment outputting additional content
- A Store application that hovers to one side of the user's view
- A interactive HUD designed to expose additional chat features

This extension introduces the concept of "Overlay Sessions" in order to expose this usage model.

This extension allows:

- An application to identify when the current sessions composition layers will be applied during composition
- The ability for an overlay session to get information about what is going on with the main application

To create an overlay session, an application **must** pass an XrSessionCreateInfoOverlayEXTX structure to xrCreateSession via the XrSessionCreateInfo structure's `next` parameter.

An overlay application should not assume that the values returned to it by xrWaitFrame in `predictedDisplayTime` in XrFrameState will be the same as the values returned to the main application or even correlated.

13.1.1. Overlay Session Layer Placement

Since one or more sessions may be active at the same time, this extension provides the ability for the application to identify when the frames of the current session will be composited into the final frame.

The XrSessionCreateInfoOverlayEXTX `sessionLayersPlacement` parameter provides information on when the sessions composition layers should be applied to the final composition frame. The larger the value passed into `sessionLayersPlacement`, the closer to the front this session's composition layers will appear (relative to other overlay session's composition layers). The smaller the value of `sessionLayersPlacement`, the further to the back this session's composition's layers will appear. The main session's composition layers will always be composited first, resulting in any overlay content being composited on top of the main application's content.

If `sessionLayersPlacement` is 0, then the runtime will always attempt to composite that session's composition layers first. If `sessionLayersPlacement` is UINT32_MAX, then the runtime will always attempt to composite that session's composition layers last. If two or more overlay sessions are created with the same `sessionLayersPlacement` value, then the newer session's will be treated as if they had a slightly higher value of `sessionLayersPlacement` than the previous sessions with the same value. This should result in the newest overlay session being composited closer to the user than the older session.

The following image hopefully will provide any further clarification you need:

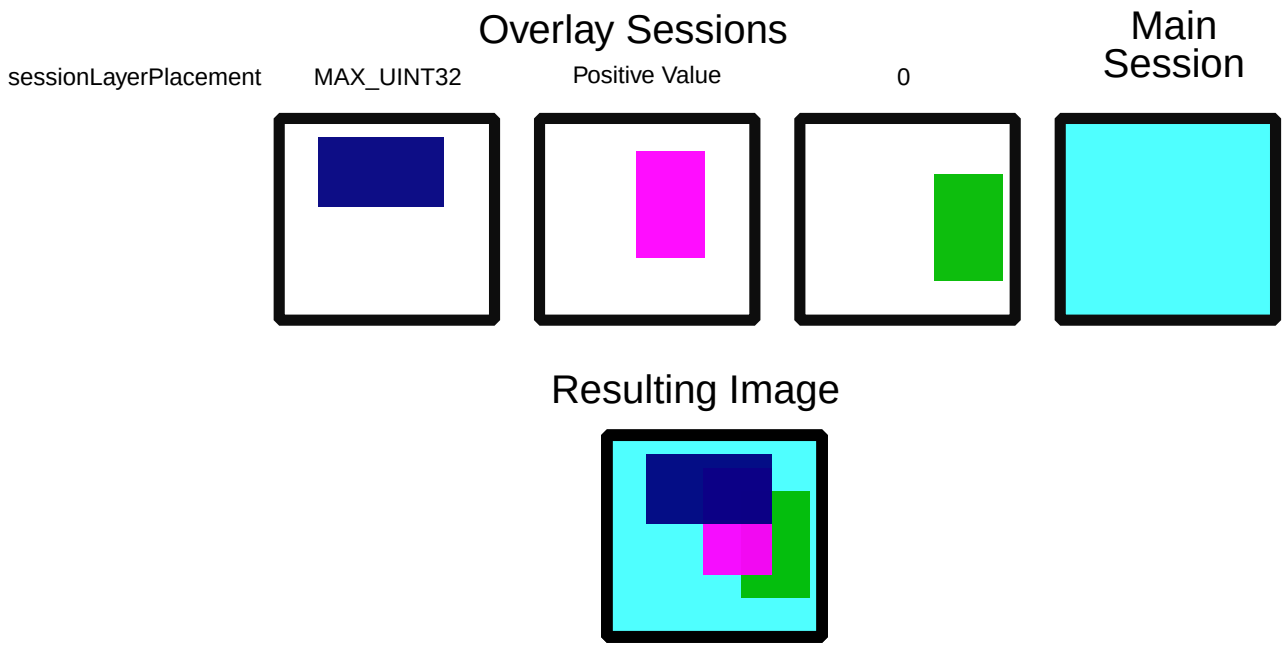


Figure 37. Overlay Composition Order

13.1.2. Main Session Behavior Event

Since an overlay session's intends to work in harmony with a main session, some information needs to be provided from that main session to the overlay session.

The XrEventDataMainSessionVisibilityChangedEXTX event structure provides information on the visibility of the main session as well as some additional flags which can be used to adjust overlay behavior.

If `XR_KHR_composition_layer_depth` is enabled in the main session, then XrEventDataMainSessionVisibilityChangedEXTX `flags` **should** contain the value: `XR_OVERLAY_MAIN_SESSION_ENABLED_COMPOSITION_LAYER_INFO_DEPTH_BIT_EXTX`. If the overlay session also enables `XR_KHR_composition_layer_depth`, then when both sessions are visible, the runtime can integrate their projection layer content together using depth information as described in the extension. However, if either the main session or the overlay do not enable the extension, then composition behavior will continue as if neither one enabled the extension.

13.1.3. Modifications to the OpenXR Specification

When this extension is enabled, certain core behaviors defined in the OpenXR specification must change as defined below:

Modifications to Composition

The Compositing section description of the composition process will be changed if this extension is enabled. If this extension is enabled, and there is only one active session, then there is no change. However, if this extension is enabled, and there are multiple active sessions, then the composition will occur in order based on the overlay session's XrSessionCreateInfoOverlayEXTX::`sessionLayersPlacement` value as described in the table below:

Table 6. Overlay Session Composition Order

Session Type	**XrSessionCreateInfoOverlayEXTX**::`sessionLayersPlacement`	Composited
Overlay Session	UINT32_MAX	*Composited last, appears in front of all other XrSessions*
Overlay Session	<Positive value>	
Overlay Session	0	
Non-overlay Session	N/A	*Composited first, appears behind all other XrSessions*

The above change only applies to when a session's composition layers are applied to the resulting image. The order in which composition layers are handled internal to a session does not change. However, once the sessions have been properly ordered, the runtime should behave as if all the composition layers have been placed into a single list (maintaining the separation of viewport images) and treat them as if they were from one original session. From this point forward, the composition behavior of the resulting composition layers is the same whether or not this extension is enabled.

If the overlay session is created as part of an XrInstance which has enabled the `XR_KHR_composition_layer_depth` extension, and a XrCompositionLayerDepthInfoKHR structure has been provided to one or more composition layers, then it intends for those layers to be composited into the final image using that depth information. This composition occurs as defined in the `XR_KHR_composition_layer_depth` extension. However, this is only possible if the main session has provided depth buffer information as part of its swapchain. In the event that a main session does not provide depth buffer information as part of its swapchain, then overlay application's composition layers containing depth information will be composited as if they did not contain that information.

Modifications to xrEndFrame Behavior

Frame Submission currently states that if xrEndFrame is called with no layers, then the runtime should clear the VR display.

If this extension is enabled, the above statement is now only true if the session is not an overlay session. If the session is an overlay session, and it provides 0 layers in the call to xrEndFrame, then the runtime will just ignore the overlay session for the current frame.

Modifications to Input Synchronization

If a runtime supports this extension, it **must** separate input tracking on a per-session basis. This means that reading the input from one active session does not disturb the input information that can be read by another active session. This may require duplicating events to more than one session.

New Object Types

None

New Flag Types

```
// Provided by XR_EXTX_overlay
typedef XrFlags64 XrOverlayMainSessionFlagsEXTX;
```

```
// Provided by XR_EXTX_overlay
// Flag bits for XrOverlayMainSessionFlagsEXTX
static const XrOverlayMainSessionFlagsEXTX
XR_OVERLAY_MAIN_SESSION_ENABLED_COMPOSITION_LAYER_INFO_DEPTH_BIT_EXTX = 0x00000001;
```

```
// Provided by XR_EXTX_overlay
typedef XrFlags64 XrOverlaySessionCreateFlagsEXTX;
```

```
// Provided by XR_EXTX_overlay
// Flag bits for XrOverlaySessionCreateFlagsEXTX
```

New Enum Constants

XrStructureType enumeration is extended with:

- XR_TYPE_SESSION_CREATE_INFO_OVERLAY_EXTX
- XR_TYPE_EVENT_DATA_MAIN_SESSION_VISIBILITY_CHANGED_EXTX

New Enums

- XR_OVERLAY_MAIN_SESSION_ENABLED_COMPOSITION_LAYER_INFO_DEPTH_BIT_EXTX

New Structures

```
// Provided by XR_EXTX_overlay
typedef struct XrSessionCreateInfoOverlayEXTX {
    XrStructureType                    type;
    const void*                        next;
    XrOverlaySessionCreateFlagsEXTX    createFlags;
    uint32_t                           sessionLayersPlacement;
} XrSessionCreateInfoOverlayEXTX;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `createFlags` is `0` or one or more XrOverlaySessionCreateFlagBitsEXTX which indicate various characteristics desired for the overlay session.
- `sessionLayersPlacement` is a value indicating the desired placement of the session's composition layers in terms of other sessions.

Valid Usage (Implicit)

- The `XR_EXTX_overlay` extension **must** be enabled prior to using XrSessionCreateInfoOverlayEXTX
- `type` **must** be `XR_TYPE_SESSION_CREATE_INFO_OVERLAY_EXTX`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `createFlags` **must** be `0`

The XrEventDataMainSessionVisibilityChangedEXTX structure is defined as:

```
// Provided by XR_EXTX_overlay
typedef struct XrEventDataMainSessionVisibilityChangedEXTX {
    XrStructureType                  type;
    const void*                      next;
    XrBool32                         visible;
    XrOverlayMainSessionFlagsEXTX    flags;
} XrEventDataMainSessionVisibilityChangedEXTX;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `visible` is an `XrBool32` which indicates if the main session is now visible or is not.
- `flags` is 0 or one or more XrOverlayMainSessionFlagBitsEXTX which indicates various state information for the main session.

Receiving the XrEventDataMainSessionVisibilityChangedEXTX event structure indicates that the main session has gained or lost visibility. This can occur in many cases, one typical example is when a user switches from one OpenXR application to another. See XrEventDataMainSessionVisibilityChangedEXTX for more information on the standard behavior. This structure contains additional information on the main session including `flags` which indicate additional state information of the main session. Currently, the only flag value supplied is `XR_OVERLAY_MAIN_SESSION_ENABLED_COMPOSITION_LAYER_INFO_DEPTH_BIT_EXTX` which indicates if the main session has enabled the `XR_KHR_composition_layer_depth` extension.

Valid Usage (Implicit)

- The `XR_EXTX_overlay` extension **must** be enabled prior to using XrEventDataMainSessionVisibilityChangedEXTX
- `type` **must** be `XR_TYPE_EVENT_DATA_MAIN_SESSION_VISIBILITY_CHANGED_EXTX`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

New Functions

None

New Function Pointers

None

Issues

None

Version History

- Revision 1, 2018-11-05 (Mark Young)
  - Initial draft

- Revision 2, 2020-02-12 (Brad Grantham)
  - Name change, remove overlay bool, add flags
- Revision 3, 2020-03-05 (Brad Grantham)
  - Name change
- Revision 4, 2020-03-23 (Brad Grantham)
  - Fix enums
- Revision 5, 2021-01-13 (Brad Grantham)
  - Remove bit requesting synchronized display times

13.2. XR_HTCX_vive_tracker_interaction

Name String

`XR_HTCX_vive_tracker_interaction`

Extension Type

Instance extension

Registered Extension Number

104

Revision

3

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

API Interactions

- Interacts with `XR_EXT_dpad_binding`

Last Modified Date

2023-07-14

IP Status

No known IP claims.

Contributors

Kyle Chen, HTC
Chris Kuo, HTC

Overview

This extension defines a new interaction profile for HTC VIVE Tracker. HTC VIVE Tracker is a generic tracked device which can be attached to anything to make them trackable. For example, it can be attached to user's hands or feet to track the motion of human body. It can also be attached to any other devices the user wants to track and interact with.

This extension allows:

- An application to enumerate the subpaths of all current connected VIVE trackers.
- An application to receive notification of the top level paths of a VIVE tracker when it is connected.

The paths of a VIVE tracker contains two paths below:

- VIVE tracker persistent path indicate a specific tracker whose lifetime lasts longer than an instance, which means it **must** not change during its hardware lifetime. The format of this path string is unspecified and should be treated as an opaque string.
- VIVE tracker role path **may** be constructed as "/user/vive_tracker_htcx/role/*ROLE_VALUE*", where *ROLE_VALUE* takes one of the following values. The role path **may** be assigned from the tool provided by the runtime and is XR_NULL_PATH if it has not been assigned. If this role path refers to more than one tracker, the runtime **should** choose one of them to be currently active. The role path **may** be changed during the lifetime of instance. Whenever it is changed, the runtime **must** send event `XR_TYPE_EVENT_DATA_VIVE_TRACKER_CONNECTED_HTCX` to provide the new role path of that tracker.

  ROLE_VALUE
  - `XR_NULL_PATH`
  - `handheld_object`
  - `left_foot`
  - `right_foot`
  - `left_shoulder`
  - `right_shoulder`
  - `left_elbow`
  - `right_elbow`
  - `left_knee`
  - `right_knee`
  - `left_wrist` (rev: 3)
  - `right_wrist` (rev: 3)
  - `left_ankle` (rev: 3)
  - `right_ankle` (rev: 3)
  - `waist`

    - `chest`
    - `camera`
    - `keyboard`

- Either the persistent path or the role path can be be passed as a subaction path to indicate a specific tracker. For example, XrActionCreateInfo::`subactionPaths` into function xrCreateAction or XrActionSpaceCreateInfo::`subactionPath` into function xrCreateActionSpace. Please see Example 1 below.

As with other controllers, if a VIVE tracker is connected and bound to a top-level user path, or disconnected while bound to top-level user path, the runtime **must** send event `XR_TYPE_EVENT_DATA_INTERACTION_PROFILE_CHANGED`, and the application **may** call xrGetCurrentInteractionProfile to check if the tracker is active or not.



The device that a tracker is attached to probably has a different motion model than what the tracker assumes. The motion tracking might not be as expected in this case.

VIVE Tracker interaction profile

Interaction profile path:

- */interaction_profiles/htc/vive_tracker_htcx*

This interaction profile represents the input sources and haptics on the VIVE Tracker.

Supported component paths:

- *.../input/system/click* (**may** not be available for application use)
- *.../input/menu/click*
- *.../input/trigger/click*
- *.../input/squeeze/click*
- *.../input/trigger/value*
- *.../input/trackpad/x*
- *.../input/trackpad/y*
- *.../input/trackpad/click*
- *.../input/trackpad/touch*
- *.../input/grip/pose*
- *.../output/haptic*

New Object Types

New Flag Types

New Enum Constants

XrStructureType enumeration is extended with:

- XR_TYPE_VIVE_TRACKER_PATHS_HTCX
- XR_TYPE_EVENT_DATA_VIVE_TRACKER_CONNECTED_HTCX

New Enums

New Structures

The XrViveTrackerPathsHTCX structure is defined as:

```
// Provided by XR_HTCX_vive_tracker_interaction
typedef struct XrViveTrackerPathsHTCX {
    XrStructureType    type;
    void*              next;
    XrPath             persistentPath;
    XrPath             rolePath;
} XrViveTrackerPathsHTCX;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- persistentPath is the unique path of the VIVE tracker which is persistent over the lifetime of the hardware.
- rolePath is the path of the VIVE tracker role. This **may** be XR_NULL_PATH if the role is not assigned.

The XrViveTrackerPathsHTCX structure contains two paths of VIVE tracker.

Valid Usage (Implicit)

- The XR_HTCX_vive_tracker_interaction extension **must** be enabled prior to using XrViveTrackerPathsHTCX
- type **must** be XR_TYPE_VIVE_TRACKER_PATHS_HTCX
- next **must** be NULL or a valid pointer to the next structure in a structure chain

The XrEventDataViveTrackerConnectedHTCX structure is defined as:

```
// Provided by XR_HTCX_vive_tracker_interaction
typedef struct XrEventDataViveTrackerConnectedHTCX {
    XrStructureType            type;
    const void*                next;
    XrViveTrackerPathsHTCX*    paths;
} XrEventDataViveTrackerConnectedHTCX;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `paths` contains two paths of the connected VIVE tracker.

Receiving the XrEventDataViveTrackerConnectedHTCX event structure indicates that a new VIVE tracker was connected or its role changed. It is received via xrPollEvent.

Valid Usage (Implicit)

- The `XR_HTCX_vive_tracker_interaction` extension **must** be enabled prior to using XrEventDataViveTrackerConnectedHTCX
- `type` **must** be `XR_TYPE_EVENT_DATA_VIVE_TRACKER_CONNECTED_HTCX`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

New Functions

The xrEnumerateViveTrackerPathsHTCX function is defined as:

```
// Provided by XR_HTCX_vive_tracker_interaction
XrResult xrEnumerateViveTrackerPathsHTCX(
    XrInstance                                  instance,
    uint32_t                                    pathCapacityInput,
    uint32_t*                                   pathCountOutput,
    XrViveTrackerPathsHTCX*                     paths);
```

Parameter Descriptions

- `instance` is an instance previously created.
- `pathCapacityInput` is the capacity of the `paths`, or `0` to retrieve the required capacity.
- `pathCountOutput` is a pointer to the count of XrViveTrackerPathsHTCX `paths` written, or a pointer to the required capacity in the case that `pathCapacityInput` is insufficient.
- `paths` is a pointer to an array of XrViveTrackerPathsHTCX VIVE tracker paths, but **can** be `NULL` if `pathCapacityInput` is `0`.
- See the Buffer Size Parameters section for a detailed description of retrieving the required `paths` size.

xrEnumerateViveTrackerPathsHTCX enumerates all connected VIVE trackers to retrieve their paths under current instance.

Valid Usage (Implicit)

- The `XR_HTCX_vive_tracker_interaction` extension **must** be enabled prior to calling xrEnumerateViveTrackerPathsHTCX
- `instance` **must** be a valid XrInstance handle
- `pathCountOutput` **must** be a pointer to a `uint32_t` value
- If `pathCapacityInput` is not `0`, `paths` **must** be a pointer to an array of `pathCapacityInput` XrViveTrackerPathsHTCX structures

Return Codes

Success

- `XR_SUCCESS`

Failure

- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SIZE_INSUFFICIENT`
- `XR_ERROR_VALIDATION_FAILURE`

Examples

Example 1

This example illustrates how to locate a VIVE tracker which is attached on the chest. First of all, create an action with */user/vive_tracker_htcx/role/chest* as the subaction path. Then, submit a suggested binding for that action to the role path plus *.../input/grip/pose*, for the interaction profile */interaction_profiles/htc/vive_tracker_htcx*, using xrSuggestInteractionProfileBindings. To locate the tracker, create an action space from that action, with */user/vive_tracker_htcx/role/chest* once again specified as the subaction path.

```
extern XrInstance instance;  // previously initialized
extern XrSession session;  // previously initialized
extern XrActionSet actionSet;  // previously initialized

// Create the action with subaction path
XrPath chestTrackerRolePath {XR_NULL_PATH};
CHK_XR(xrStringToPath(instance, "/user/vive_tracker_htcx/role/chest",
    &chestTrackerRolePath));

XrAction chestPoseAction {XR_NULL_HANDLE};
XrActionCreateInfo actionInfo {
    .type = XR_TYPE_ACTION_CREATE_INFO,
    .actionType = XR_ACTION_TYPE_POSE_INPUT,
    .countSubactionPaths = 1,
    .subactionPaths = &chestTrackerRolePath,
};
CHK_XR(xrCreateAction(actionSet, &actionInfo, &chestPoseAction));

// Describe a suggested binding for that action and subaction path.
XrPath suggestedBindingPath {XR_NULL_PATH};
CHK_XR(xrStringToPath(instance,
    "/user/vive_tracker_htcx/role/chest/input/grip/pose",
    &suggestedBindingPath));

std::vector<XrActionSuggestedBinding> actionSuggBindings;
XrActionSuggestedBinding actionSuggBinding {
    .action = chestPoseAction,
    .binding = suggestedBindingPath,
};
actionSuggBindings.push_back(actionSuggBinding);

// Suggest that binding for the VIVE tracker interaction profile
XrPath viveTrackerInteractionProfilePath {XR_NULL_PATH};
CHK_XR(xrStringToPath(instance, "/interaction_profiles/htc/vive_tracker_htcx",
    &viveTrackerInteractionProfilePath));

XrInteractionProfileSuggestedBinding profileSuggBindings {
    .type = XR_TYPE_INTERACTION_PROFILE_SUGGESTED_BINDING,
```

```
    .interactionProfile = viveTrackerInteractionProfilePath,
    .countSuggestedBindings = static_cast<uint32_t>(actionSuggBindings.size()),
    .suggestedBindings = actionSuggBindings.data(),
};
CHK_XR(xrSuggestInteractionProfileBindings(instance, &profileSuggBindings));

// Create action space for locating tracker
XrSpace chestTrackerSpace {XR_NULL_HANDLE};
XrActionSpaceCreateInfo actionSpaceInfo {
    .type = XR_TYPE_ACTION_SPACE_CREATE_INFO,
    .action = chestPoseAction,
    .subactionPath = chestTrackerRolePath,
};
CHK_XR(xrCreateActionSpace(session, &actionSpaceInfo, &chestTrackerSpace));
```

Example 2

This example illustrates how to handle the VIVE tracker when it is connected or disconnected. When a VIVE tracker is connected or its role changed, event `XR_TYPE_EVENT_DATA_VIVE_TRACKER_CONNECTED_HTCX` will be received. The role path and persistent path of this tracker can be retrieved with this event. When a VIVE tracker is connected or disconnected, event `XR_TYPE_EVENT_DATA_INTERACTION_PROFILE_CHANGED` will also be received. The XrInteractionProfileState ::`interactionProfile` will be XR_NULL_PATH if the tracker represented by that top level path is not connected.

```
extern XrInstance instance;  // previously initialized
extern XrSession session;  // previously initialized
extern XrEventDataBuffer xrEvent; // previously received from xrPollEvent

switch ( xrEvent.type )
{
    case XR_TYPE_EVENT_DATA_VIVE_TRACKER_CONNECTED_HTCX: {
        const XrEventDataViveTrackerConnectedHTCX& viveTrackerConnected =
            *reinterpret_cast<XrEventDataViveTrackerConnectedHTCX*>(&xrEvent);
        uint32_t nCount;
        char sPersistentPath[XR_MAX_PATH_LENGTH];
        CHK_XR(xrPathToString(instance,
            viveTrackerConnected.paths->persistentPath,
            sizeof(sPersistentPath), &nCount, sPersistentPath));

        std::printf("Vive Tracker connected: %s \n", sPersistentPath);
        if (viveTrackerConnected.paths->rolePath != XR_NULL_PATH) {
            char sRolePath[XR_MAX_PATH_LENGTH];
            CHK_XR(xrPathToString(instance,
                viveTrackerConnected.paths->rolePath, sizeof(sRolePath),
                &nCount, sRolePath));

            std::printf(" New role is: %s\n\n", sRolePath);
        } else {
            std::printf(" No role path.\n\n");
        }
        break;
    }

    case XR_TYPE_EVENT_DATA_INTERACTION_PROFILE_CHANGED: {
        XrPath chestTrackerRolePath {XR_NULL_PATH};
        XrInteractionProfileState xrInteractionProfileState {
            .type = XR_TYPE_INTERACTION_PROFILE_STATE,
        };

        CHK_XR(xrStringToPath(instance, "/user/vive_tracker_htcx/role/chest",
            &chestTrackerRolePath));
        CHK_XR(xrGetCurrentInteractionProfile(session, chestTrackerRolePath,
            &xrInteractionProfileState));
        break;
    }
}
```

Issues

Version History

- Revision 1, 2021-09-23 (Kyle Chen)
  - Initial extension description.
- Revision 2, 2022-09-08 (Rylie Pavlik, Collabora, Ltd.)
  - Mark event type as returned-only, updating the implicit valid usage.
- Revision 3, 2022-05-19 (Rune Berg, Valve Corporation)
  - Add new wrist and ankle roles to match additional openvr roles.

13.3. XR_MNDX_egl_enable

Name String

`XR_MNDX_egl_enable`

Extension Type

Instance extension

Registered Extension Number

49

Revision

2

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Last Modified Date

2023-12-02

IP Status

No known IP claims.

Contributors

Jakob Bornecrantz, Collabora
Drew DeVault, Individual
Simon Ser, Individual

Overview

This extension must be provided by runtimes supporting applications using the EGL API to create rendering contexts.

- XR_USE_PLATFORM_EGL

New Object Types

New Flag Types

New Enum Constants

XrStructureType enumeration is extended with:

- XR_TYPE_GRAPHICS_BINDING_EGL_MNDX

New Enums

New Structures

The XrGraphicsBindingEGLMNDX structure is defined as:

```
// Provided by XR_MNDX_egl_enable
typedef struct XrGraphicsBindingEGLMNDX {
    XrStructureType                 type;
    const void*                     next;
    PFN_xrEglGetProcAddressMNDX     getProcAddress;
    EGLDisplay                      display;
    EGLConfig                       config;
    EGLContext                      context;
} XrGraphicsBindingEGLMNDX;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `getProcAddress` is a valid function pointer to `eglGetProcAddress`.
- `display` is a valid EGL `EGLDisplay`.
- `config` is a valid EGL `EGLConfig`.
- `context` is a valid EGL `EGLContext`.

When creating an EGL based XrSession, the application will provide a pointer to an XrGraphicsBindingEGLMNDX structure in the `next` chain of the XrSessionCreateInfo.

The required window system configuration define to expose this structure type is XR_USE_PLATFORM_EGL.

Valid Usage (Implicit)

- The XR_MNDX_egl_enable extension **must** be enabled prior to using XrGraphicsBindingEGLMNDX
- type **must** be XR_TYPE_GRAPHICS_BINDING_EGL_MNDX
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- getProcAddress **must** be a valid PFN_xrEglGetProcAddressMNDX value
- display **must** be a valid EGLDisplay value
- config **must** be a valid EGLConfig value
- context **must** be a valid EGLContext value

New Functions

New Function Pointers

```
// Provided by XR_MNDX_egl_enable
typedef PFN_xrVoidFunction (*PFN_xrEglGetProcAddressMNDX)(const char *name);
```

Parameter Descriptions

- name specifies the name of the function to return.

eglGetProcAddress returns the address of the client API or EGL function named by procname. For details please see https://registry.khronos.org/EGL/sdk/docs/man/html/eglGetProcAddress.xhtml

Issues

Version History

- Revision 1, 2020-05-20 (Jakob Bornecrantz)
  - Initial draft
- Revision 2, 2023-12-02
  - Use PFN_xrEglGetProcAddressMNDX to replace PFNEGLGETPROCADDRESSPROC (for eglGetProcAddress). Note this does change function pointer attributes on some platforms.

13.4. XR_MNDX_force_feedback_curl

Name String

`XR_MNDX_force_feedback_curl`

Extension Type

Instance extension

Registered Extension Number

376

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0
and
`XR_EXT_hand_tracking`

Last Modified Date

2022-11-18

IP Status

No known IP claims.

Contributors

Daniel Willmott
Moses Turner (Collabora, Ltd.)
Christoph Haagch (Collabora, Ltd.)
Jakob Bornecrantz (Collabora, Ltd.)

Overview

This extension provides APIs for force feedback devices capable of restricting physical movement in a single direction along a single dimension.

The intended use for this extension is to provide simple force feedback capabilities to restrict finger movement for VR Gloves.

The application **must** also enable the `XR_EXT_hand_tracking` extension in order to use this extension.

The XrForceFeedbackCurlLocationMNDX describes which location to apply force feedback.

```
// Provided by XR_MNDX_force_feedback_curl
typedef enum XrForceFeedbackCurlLocationMNDX {
    XR_FORCE_FEEDBACK_CURL_LOCATION_THUMB_CURL_MNDX = 0,
    XR_FORCE_FEEDBACK_CURL_LOCATION_INDEX_CURL_MNDX = 1,
    XR_FORCE_FEEDBACK_CURL_LOCATION_MIDDLE_CURL_MNDX = 2,
    XR_FORCE_FEEDBACK_CURL_LOCATION_RING_CURL_MNDX = 3,
    XR_FORCE_FEEDBACK_CURL_LOCATION_LITTLE_CURL_MNDX = 4,
    XR_FORCE_FEEDBACK_CURL_LOCATION_MAX_ENUM_MNDX = 0x7FFFFFFF
} XrForceFeedbackCurlLocationMNDX;
```

Enumerant Descriptions

- XR_FORCE_FEEDBACK_CURL_LOCATION_THUMB_CURL_MNDX — force feedback for thumb curl
- XR_FORCE_FEEDBACK_CURL_LOCATION_INDEX_CURL_MNDX — force feedback for index finger curl
- XR_FORCE_FEEDBACK_CURL_LOCATION_MIDDLE_CURL_MNDX — force feedback for middle finger curl
- XR_FORCE_FEEDBACK_CURL_LOCATION_RING_CURL_MNDX — force feedback for ring finger curl
- XR_FORCE_FEEDBACK_CURL_LOCATION_LITTLE_CURL_MNDX — force feedback for little finger curl

New Object Types

New Flag Types

New Enum Constants

XrStructureType enumeration is extended with:

- XR_TYPE_SYSTEM_FORCE_FEEDBACK_CURL_PROPERTIES_MNDX
- XR_TYPE_FORCE_FEEDBACK_CURL_APPLY_LOCATIONS_MNDX

New Enums

- XrForceFeedbackCurlLocationMNDX

New Structures

The XrSystemForceFeedbackCurlPropertiesMNDX structure is defined as:

```
// Provided by XR_MNDX_force_feedback_curl
typedef struct XrSystemForceFeedbackCurlPropertiesMNDX {
    XrStructureType    type;
    void*              next;
    XrBool32           supportsForceFeedbackCurl;
} XrSystemForceFeedbackCurlPropertiesMNDX;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `supportsForceFeedbackCurl` is an `XrBool32`, indicating if the current system is capable of performing force feedback.

An application **may** inspect whether the system is capable of force feedback by chaining an XrSystemForceFeedbackCurlPropertiesMNDX structure to the XrSystemProperties structure when calling xrGetSystemProperties.

The runtime **should** return `XR_TRUE` for `supportsForceFeedbackCurl` when force feedback is available in the system, otherwise `XR_FALSE`. Force feedback calls **must** return `XR_ERROR_FEATURE_UNSUPPORTED` if force feedback is not available in the system.

Valid Usage (Implicit)

- The `XR_MNDX_force_feedback_curl` extension **must** be enabled prior to using XrSystemForceFeedbackCurlPropertiesMNDX
- `type` **must** be `XR_TYPE_SYSTEM_FORCE_FEEDBACK_CURL_PROPERTIES_MNDX`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

The XrForceFeedbackCurlApplyLocationsMNDX structure is defined as:

```
// Provided by XR_MNDX_force_feedback_curl
typedef struct XrForceFeedbackCurlApplyLocationsMNDX {
    XrStructureType                         type;
    const void*                             next;
    uint32_t                                locationCount;
    XrForceFeedbackCurlApplyLocationMNDX*   locations;
} XrForceFeedbackCurlApplyLocationsMNDX;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `locationCount` is the number of elements in the `locations` array.
- `locations` is a pointer to an array of locations to apply force feedback.

Contains an array of XrForceFeedbackCurlApplyLocationMNDX that contains information on locations to apply force feedback to.

Valid Usage (Implicit)

- The `XR_MNDX_force_feedback_curl` extension **must** be enabled prior to using XrForceFeedbackCurlApplyLocationsMNDX
- `type` **must** be `XR_TYPE_FORCE_FEEDBACK_CURL_APPLY_LOCATIONS_MNDX`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `locations` **must** be a pointer to an array of `locationCount` XrForceFeedbackCurlApplyLocationMNDX structures
- The `locationCount` parameter **must** be greater than `0`

The XrForceFeedbackCurlApplyLocationMNDX structure is defined as:

```
// Provided by XR_MNDX_force_feedback_curl
typedef struct XrForceFeedbackCurlApplyLocationMNDX {
    XrForceFeedbackCurlLocationMNDX    location;
    float                              value;
} XrForceFeedbackCurlApplyLocationMNDX;
```

Member Descriptions

- `location` represents the location to apply force feedback to.
- `value` is a value from 0-1 representing the amount of force feedback to apply. The range of the value should represent the entire range the location is capable of moving through, with 1 representing making the location incapable of movement, and 0 being fully flexible. For example, in the case of a finger curl, setting `value` to 1 would prevent the finger from curling at all (fully extended), and 0 would allow the finger to have free range of movement, being able to curl fully.

`value` is specified as a limit in a single direction. For example, if the value specified is 0.5, a location **must** have free movement from the point where it would be incapable of movement if `value` was 1, to 0.5 of the range the location is capable of moving.

Valid Usage (Implicit)

- The `XR_MNDX_force_feedback_curl` extension **must** be enabled prior to using XrForceFeedbackCurlApplyLocationMNDX
- `location` **must** be a valid XrForceFeedbackCurlLocationMNDX value

New Functions

The xrApplyForceFeedbackCurlMNDX function is defined as:

```
// Provided by XR_MNDX_force_feedback_curl
XrResult xrApplyForceFeedbackCurlMNDX(
    XrHandTrackerEXT                            handTracker,
    const XrForceFeedbackCurlApplyLocationsMNDX* locations);
```

Parameter Descriptions

- `handTracker` is an XrHandTrackerEXT handle previously created with xrCreateHandTrackerEXT.
- `locations` is an XrForceFeedbackCurlApplyLocationsMNDX containing a set of locations to apply force feedback to.

The xrApplyForceFeedbackCurlMNDX function applies force feedback to the set locations listed in XrForceFeedbackCurlApplyLocationsMNDX.

xrApplyForceFeedbackCurlMNDX **should** be called every time an application wishes to update a set of

force feedback locations.

Submits a request for force feedback for a set of locations. The runtime **should** deliver this request to the `handTracker` device. If the `handTracker` device is not available, the runtime **may** ignore this request for force feedback.

If the session associated with `handTracker` is not focused, the runtime **must** return `XR_SESSION_NOT_FOCUSED`, and not apply force feedback.

When an application submits force feedback for a set of locations, the runtime **must** update the set of locations to that specified by the application. A runtime **must** set any locations not specified by the application when submitting force feedback to 0.

The runtime **may** discontinue force feedback if the application that set it loses focus. An application **should** call the function again after regaining focus if force feedback is still desired.

Valid Usage (Implicit)

- The `XR_MNDX_force_feedback_curl` extension **must** be enabled prior to calling xrApplyForceFeedbackCurlMNDX
- `handTracker` **must** be a valid XrHandTrackerEXT handle
- `locations` **must** be a pointer to a valid XrForceFeedbackCurlApplyLocationsMNDX structure

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`
- `XR_SESSION_NOT_FOCUSED`

Failure

- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_VALIDATION_FAILURE`

Issues

Version History

- Revision 1, 2022-09-07 (Daniel Willmott)
    - Initial version

Chapter 14. List of Deprecated Extensions

- XR_KHR_locate_spaces (promoted to core)
- XR_KHR_maintenance1 (promoted to core)
- XR_EXT_hp_mixed_reality_controller (promoted to core)
- XR_EXT_local_floor (promoted to core)
- XR_EXT_palm_pose (promoted to core)
- XR_EXT_plane_detection
- XR_EXT_samsung_odyssey_controller (promoted to core)
- XR_EXT_uuid (promoted to core)
- XR_ANDROID_device_anchor_persistence
- XR_ANDROID_raycast
- XR_ANDROID_trackables
- XR_ANDROID_trackables_marker
- XR_ANDROID_trackables_object
- XR_ANDROID_trackables_qr_code
- XR_BD_controller_interaction (promoted to core)
- XR_FB_space_warp
- XR_FB_touch_controller_pro (promoted to core)
- XR_FB_touch_controller_proximity
- XR_HTC_vive_cosmos_controller_interaction (promoted to core)
- XR_HTC_vive_focus3_controller_interaction (promoted to core)
- XR_META_touch_controller_plus (promoted to core)
- XR_ML_ml2_controller_interaction (promoted to core)
- XR_MND_swapchain_usage_input_attachment_bit
- XR_MSFT_composition_layer_reprojection
- XR_MSFT_controller_model
- XR_MSFT_first_person_observer
- XR_MSFT_hand_interaction
- XR_MSFT_hand_tracking_mesh
- XR_MSFT_holographic_window_attachment
- XR_MSFT_perception_anchor_interop

- XR_MSFT_scene_marker
- XR_MSFT_scene_understanding
- XR_MSFT_scene_understanding_serialization
- XR_MSFT_secondary_view_configuration
- XR_MSFT_spatial_anchor
- XR_MSFT_spatial_anchor_persistence
- XR_MSFT_spatial_graph_bridge
- XR_MSFT_unbounded_reference_space
- XR_OCULUS_android_session_state_enable
- XR_QCOM_hand_tracking_gesture
- XR_VARJO_quad_views (promoted to core)

14.1. XR_KHR_locate_spaces

Name String

`XR_KHR_locate_spaces`

Extension Type

Instance extension

Registered Extension Number

472

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

OpenXR 1.0

Deprecation State

- *Promoted* to OpenXR 1.1

Last Modified Date

2026-07-10

IP Status

No known IP claims.

Contributors

Yin Li, Microsoft
Bryce Hutchings, Microsoft
Andreas Loeve Selvik, Meta Platforms
John Kearney, Meta Platforms
Robert Blenkinsopp, Ultraleap
Rylie Pavlik, Collabora
Ron Bessems, Magic Leap
Jakob Bornecrantz, NVIDIA

14.1.1. Overview

This extension introduces the xrLocateSpacesKHR function, which enables applications to locate an array of spaces in a single function call. Runtimes **may** provide performance benefits for applications that use many spaces.

Compared to the xrLocateSpace function, the new xrLocateSpacesKHR function also provides extensible input parameters for future extensions to extend using additional chained structures.

14.1.2. Locate spaces

Applications **can** use xrLocateSpacesKHR function to locate an array of spaces.

The xrLocateSpacesKHR function is defined as:

```
// Provided by XR_KHR_locate_spaces
XrResult xrLocateSpacesKHR(
    XrSession                                   session,
    const XrSpacesLocateInfo*                   locateInfo,
    XrSpaceLocations*                           spaceLocations);
```

Parameter Descriptions

- `session` is an XrSession handle previously created with xrCreateSession.
- `locateInfo` is a pointer to an XrSpacesLocateInfoKHR that provides the input information to locate spaces.
- `spaceLocations` is a pointer to an XrSpaceLocationsKHR for the runtime to return the locations of the specified spaces in the base space.

xrLocateSpacesKHR provides the physical location of one or more spaces in a base space at a specified time, if currently known by the runtime.

The XrSpacesLocateInfoKHR::`time`, the XrSpacesLocateInfoKHR::`baseSpace`, and each space in XrSpacesLocateInfoKHR::`spaces`, in the `locateInfo` parameter, all follow the same specifics as the corresponding inputs to the xrLocateSpace function.

Valid Usage (Implicit)

- The `XR_KHR_locate_spaces` extension **must** be enabled prior to calling xrLocateSpacesKHR
- `session` **must** be a valid XrSession handle
- `locateInfo` **must** be a pointer to a valid XrSpacesLocateInfo structure
- `spaceLocations` **must** be a pointer to an XrSpaceLocations structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_SIZE_INSUFFICIENT
- XR_ERROR_TIME_INVALID
- XR_ERROR_VALIDATION_FAILURE

The XrSpacesLocateInfoKHR structure is defined as:

```
// Provided by XR_KHR_locate_spaces
// XrSpacesLocateInfoKHR is an alias for XrSpacesLocateInfo
typedef struct XrSpacesLocateInfo {
    XrStructureType    type;
    const void*        next;
    XrSpace            baseSpace;
    XrTime             time;
    uint32_t           spaceCount;
    const XrSpace*     spaces;
} XrSpacesLocateInfo;

typedef XrSpacesLocateInfo XrSpacesLocateInfoKHR;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `baseSpace` identifies the underlying space in which to locate `spaces`.
- `time` is the time for which the location is requested.
- `spaceCount` is a `uint32_t` specifying the count of elements in the `spaces` array.
- `spaces` is an array of valid XrSpace handles to be located.

The `time`, the `baseSpace`, and each space in `spaces` all follow the same specifics as the corresponding inputs to the xrLocateSpace function.

The `baseSpace` and all of the XrSpace handles in the `spaces` array **must** be valid and share the same parent XrSession.

If the `time` is invalid, the xrLocateSpacesKHR **must** return `XR_ERROR_TIME_INVALID`.

The `spaceCount` **must** be a positive number, i.e. the array `spaces` **must** not be empty. Otherwise, the runtime **must** return `XR_ERROR_VALIDATION_FAILURE`.

Valid Usage (Implicit)

- The `XR_KHR_locate_spaces` extension **must** be enabled prior to using XrSpacesLocateInfoKHR
- **Note:** XrSpacesLocateInfoKHR is an alias for XrSpacesLocateInfo, so the following items replicate the implicit valid usage for XrSpacesLocateInfo
- `type` **must** be `XR_TYPE_SPACES_LOCATE_INFO`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `baseSpace` **must** be a valid XrSpace handle
- `spaces` **must** be a pointer to an array of `spaceCount` valid XrSpace handles
- The `spaceCount` parameter **must** be greater than `0`
- Both of `baseSpace` and the elements of `spaces` **must** have been created, allocated, or retrieved from the same XrSession

The XrSpaceLocationsKHR structure is defined as:

```
// Provided by XR_KHR_locate_spaces
// XrSpaceLocationsKHR is an alias for XrSpaceLocations
typedef struct XrSpaceLocations {
    XrStructureType         type;
    void*                   next;
    uint32_t                locationCount;
    XrSpaceLocationData*    locations;
} XrSpaceLocations;

typedef XrSpaceLocations XrSpaceLocationsKHR;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain, such as XrSpaceVelocitiesKHR.
- `locationCount` is a `uint32_t` specifying the count of elements in the `locations` array.
- `locations` is an array of XrSpaceLocationsKHR for the runtime to populate with the locations of the specified spaces in the XrSpacesLocateInfoKHR::`baseSpace` at the specified XrSpacesLocateInfoKHR::`time`.

The XrSpaceLocationsKHR structure contains an array of space locations in the member `locations`, to be used as output for xrLocateSpacesKHR. The application **must** allocate this array to be populated with the function output. The `locationCount` value **must** be the same as XrSpacesLocateInfoKHR::`spaceCount`, otherwise, the xrLocateSpacesKHR function **must** return `XR_ERROR_VALIDATION_FAILURE`.

Valid Usage (Implicit)

- The `XR_KHR_locate_spaces` extension **must** be enabled prior to using XrSpaceLocationsKHR
- **Note:** XrSpaceLocationsKHR is an alias for XrSpaceLocations, so the following items replicate the implicit valid usage for XrSpaceLocations
- `type` **must** be `XR_TYPE_SPACE_LOCATIONS`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain. See also: XrSpaceVelocities
- `locations` **must** be a pointer to an array of `locationCount` XrSpaceLocationData structures
- The `locationCount` parameter **must** be greater than `0`

The XrSpaceLocationDataKHR structure is defined as:

```
// Provided by XR_KHR_locate_spaces
// XrSpaceLocationDataKHR is an alias for XrSpaceLocationData
typedef struct XrSpaceLocationData {
    XrSpaceLocationFlags    locationFlags;
    XrPosef                 pose;
} XrSpaceLocationData;

typedef XrSpaceLocationData XrSpaceLocationDataKHR;
```

Member Descriptions

- `locationFlags` is a bitfield, with bit masks defined in XrSpaceLocationFlagBits. It behaves the same as XrSpaceLocation::`locationFlags`.
- `pose` is an XrPosef that behaves the same as XrSpaceLocation::`pose`.

This is a single element of the array in XrSpaceLocationsKHR::`locations`, and is used to return the pose and location flags for a single space with respect to the specified base space from a call to xrLocateSpacesKHR. It does not accept chained structures to allow for easier use in dynamically allocated container datatypes. Chained structures are possible with the XrSpaceLocationsKHR that describes an array of these elements.

Valid Usage (Implicit)

- The `XR_KHR_locate_spaces` extension **must** be enabled prior to using XrSpaceLocationDataKHR

14.1.3. Locate space velocities

Applications **can** request the velocities of spaces by chaining the XrSpaceVelocitiesKHR structure to the next pointer of XrSpaceLocationsKHR when calling xrLocateSpacesKHR.

The XrSpaceVelocitiesKHR structure is defined as:

```
// Provided by XR_KHR_locate_spaces
// XrSpaceVelocitiesKHR is an alias for XrSpaceVelocities
typedef struct XrSpaceVelocities {
    XrStructureType         type;
    void*                   next;
    uint32_t                velocityCount;
    XrSpaceVelocityData*    velocities;
} XrSpaceVelocities;

typedef XrSpaceVelocities XrSpaceVelocitiesKHR;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `velocityCount` is a `uint32_t` specifying the count of elements in the `velocities` array.
- `velocities` is an array of XrSpaceVelocityDataKHR for the runtime to populate with the velocities of the specified spaces in the XrSpacesLocateInfoKHR::`baseSpace` at the specified XrSpacesLocateInfoKHR::`time`.

The `velocities` member contains an array of space velocities in the member `velocities`, to be used as output for xrLocateSpacesKHR. The application **must** allocate this array to be populated with the function output. The `velocityCount` value **must** be the same as XrSpacesLocateInfoKHR::`spaceCount`, otherwise, the xrLocateSpacesKHR function **must** return `XR_ERROR_VALIDATION_FAILURE`.

Valid Usage (Implicit)

- The `XR_KHR_locate_spaces` extension **must** be enabled prior to using XrSpaceVelocitiesKHR
- **Note:** XrSpaceVelocitiesKHR is an alias for XrSpaceVelocities, so the following items replicate the implicit valid usage for XrSpaceVelocities
- `type` **must** be `XR_TYPE_SPACE_VELOCITIES`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `velocities` **must** be a pointer to an array of `velocityCount` XrSpaceVelocityData structures
- The `velocityCount` parameter **must** be greater than `0`

The XrSpaceVelocityDataKHR structure is defined as:

```
// Provided by XR_KHR_locate_spaces
// XrSpaceVelocityDataKHR is an alias for XrSpaceVelocityData
typedef struct XrSpaceVelocityData {
    XrSpaceVelocityFlags    velocityFlags;
    XrVector3f              linearVelocity;
    XrVector3f              angularVelocity;
} XrSpaceVelocityData;

typedef XrSpaceVelocityData XrSpaceVelocityDataKHR;
```

Member Descriptions

- `velocityFlags` is a bitfield, with bit values defined in XrSpaceVelocityFlagBits. It behaves the same as XrSpaceVelocity::`velocityFlags`.
- `linearVelocity` is an XrVector3f. It behaves the same as XrSpaceVelocity::`linearVelocity`.
- `angularVelocity` is an XrVector3f. It behaves the same as XrSpaceVelocity::`angularVelocity`.

This is a single element of the array in XrSpaceVelocitiesKHR::`velocities`, and is used to return the linear and angular velocity and velocity flags for a single space with respect to the specified base space from a call to xrLocateSpacesKHR. It does not accept chained structures to allow for easier use in dynamically allocated container datatypes.

Valid Usage (Implicit)

- The `XR_KHR_locate_spaces` extension **must** be enabled prior to using XrSpaceVelocityDataKHR

14.1.4. Example code for xrLocateSpacesKHR

The following example code shows how an application retrieves both the location and velocity of one or more spaces in a base space at a given time using the xrLocateSpacesKHR function.

```
XrInstance instance; // previously initialized
XrSession session; // previously initialized
XrSpace baseSpace; // previously initialized
std::vector<XrSpace> spacesToLocate;  // previously initialized

// Prepare output buffers to receive data and get reused in frame loop.
std::vector<XrSpaceLocationDataKHR> locationBuffer(spacesToLocate.size());
std::vector<XrSpaceVelocityDataKHR> velocityBuffer(spacesToLocate.size());

// Get function pointer for xrLocateSpacesKHR.
```

```
PFN_xrLocateSpacesKHR xrLocateSpacesKHR;
CHK_XR(xrGetInstanceProcAddr(instance, "xrLocateSpacesKHR",
                             reinterpret_cast<PFN_xrVoidFunction*>(
                             &xrLocateSpacesKHR)));

// application frame loop
while (1) {
    // Typically the time is the predicted display time returned from xrWaitFrame.
    XrTime displayTime; // previously initialized.

    XrSpacesLocateInfoKHR locateInfo {
        .type = XR_TYPE_SPACES_LOCATE_INFO_KHR,
        .baseSpace = baseSpace,
        .time = displayTime,
        .spaceCount = static_cast<uint32_t>(spacesToLocate.size()),
        .spaces = spacesToLocate.data(),
    };

    XrSpaceVelocitiesKHR velocities {
        .type = XR_TYPE_SPACE_VELOCITIES_KHR,
        .velocityCount = static_cast<uint32_t>(velocityBuffer.size()),
        .velocities = velocityBuffer.data(),
    };

    XrSpaceLocationsKHR locations {
        .type = XR_TYPE_SPACE_LOCATIONS_KHR,
        .next = &velocities,
        .locationCount = static_cast<uint32_t>(locationBuffer.size()),
        .locations = locationBuffer.data(),
    };

    CHK_XR(xrLocateSpacesKHR(session, &locateInfo, &locations));

    for (uint32_t i = 0; i < spacesToLocate.size(); i++) {
        const auto positionAndOrientationTracked =
          XR_SPACE_LOCATION_POSITION_TRACKED_BIT |
XR_SPACE_LOCATION_ORIENTATION_TRACKED_BIT;
        const auto orientationOnlyTracked = XR_SPACE_LOCATION_ORIENTATION_TRACKED_BIT;

        if ((locationBuffer[i].locationFlags & positionAndOrientationTracked) ==
positionAndOrientationTracked) {
            // if the location is 6dof tracked
            do_something(locationBuffer[i].pose.position);
            do_something(locationBuffer[i].pose.orientation);

            const auto velocityValidBits =
              XR_SPACE_VELOCITY_LINEAR_VALID_BIT | XR_SPACE_VELOCITY_ANGULAR_VALID_BIT;
            if ((velocityBuffer[i].velocityFlags & velocityValidBits) ==
```

```
velocityValidBits) {
                do_something(velocityBuffer[i].linearVelocity);
                do_something(velocityBuffer[i].angularVelocity);
            }
        }
        else if ((locationBuffer[i].locationFlags & orientationOnlyTracked) ==
orientationOnlyTracked) {
            // if the location is 3dof tracked
            do_something(locationBuffer[i].pose.orientation);

            if ((velocityBuffer[i].velocityFlags & XR_SPACE_VELOCITY_ANGULAR_VALID_BIT)
== XR_SPACE_VELOCITY_ANGULAR_VALID_BIT) {
                do_something(velocityBuffer[i].angularVelocity);
            }
        }
    }
}
```

New Object Types

New Flag Types

New Enum Constants

XrStructureType enumeration is extended with:

- XR_TYPE_SPACES_LOCATE_INFO_KHR
- XR_TYPE_SPACE_LOCATIONS_KHR
- XR_TYPE_SPACE_VELOCITIES_KHR

New Enums

New Structures

- XrSpacesLocateInfoKHR
- XrSpaceLocationsKHR
- XrSpaceLocationDataKHR
- XrSpaceVelocitiesKHR
- XrSpaceVelocityDataKHR

New Functions

- xrLocateSpacesKHR

Issues

Version History

- Revision 1, 2023-04-22 (Yin LI)
  - Initial extension description

14.2. XR_KHR_maintenance1

Name String

`XR_KHR_maintenance1`

Extension Type

Instance extension

Registered Extension Number

711

Revision

1

Ratification Status

Ratified

Extension and Version Dependencies

OpenXR 1.0

API Interactions

- Interacts with `XR_BD_controller_interaction`
- Interacts with `XR_EXT_hand_interaction`
- Interacts with `XR_EXT_hp_mixed_reality_controller`
- Interacts with `XR_EXT_samsung_odyssey_controller`
- Interacts with `XR_FB_touch_controller_pro`
- Interacts with `XR_HTCX_vive_tracker_interaction`
- Interacts with `XR_HTC_hand_interaction`
- Interacts with `XR_HTC_vive_cosmos_controller_interaction`
- Interacts with `XR_HTC_vive_focus3_controller_interaction`
- Interacts with `XR_HUAWEI_controller_interaction`
- Interacts with `XR_LOGITECH_mx_ink_stylus_interaction`
- Interacts with `XR_META_touch_controller_plus`
- Interacts with `XR_ML_ml2_controller_interaction`

- Interacts with XR_MSFT_hand_interaction
- Interacts with XR_OPPO_controller_interaction
- Interacts with XR_VARJO_xr4_controller_interaction
- Interacts with XR_YVR_controller_interaction

Deprecation State

- *Promoted* to OpenXR 1.1

Last Modified Date

2023-10-25

IP Status

No known IP claims.

Contributors

Ron Bessems, Magic Leap
Karthik Kadappan, Magic Leap
Rylie Pavlik, Collabora
Nihav Jain, Google
Lachlan Ford, Google
John Kearney, Meta
Yin Li, Microsoft
Robert Blenkinsopp, Ultraleap

14.2.1. Overview

XR_KHR_maintenance1 adds a collection of minor features that were intentionally left out or overlooked from the original OpenXR 1.0 release. All are promoted to the OpenXR 1.1 release.

```
// Provided by XR_KHR_maintenance1
// XrColor3fKHR is an alias for XrColor3f
typedef struct XrColor3f {
    float    r;
    float    g;
    float    b;
} XrColor3f;

typedef XrColor3f XrColor3fKHR;
```

```
// Provided by XR_KHR_maintenance1
// XrExtent3DfKHR is an alias for XrExtent3Df
typedef struct XrExtent3Df {
    float    width;
    float    height;
    float    depth;
} XrExtent3Df;

typedef XrExtent3Df XrExtent3DfKHR;
```

```
// Provided by XR_KHR_maintenance1
// XrSpherefKHR is an alias for XrSpheref
typedef struct XrSpheref {
    XrPosef    center;
    float      radius;
} XrSpheref;

typedef XrSpheref XrSpherefKHR;
```

```
// Provided by XR_KHR_maintenance1
// XrBoxfKHR is an alias for XrBoxf
typedef struct XrBoxf {
    XrPosef        center;
    XrExtent3Df    extents;
} XrBoxf;

typedef XrBoxf XrBoxfKHR;
```

```
// Provided by XR_KHR_maintenance1
// XrFrustumfKHR is an alias for XrFrustumf
typedef struct XrFrustumf {
    XrPosef    pose;
    XrFovf     fov;
    float      nearZ;
    float      farZ;
} XrFrustumf;

typedef XrFrustumf XrFrustumfKHR;
```

14.2.2. New Structures

- XrBoxfKHR

- XrColor3fKHR
- XrExtent3DfKHR
- XrFrustumfKHR
- XrSpherefKHR

14.2.3. New Enum Constants

- `XR_KHR_MAINTENANCE1_EXTENSION_NAME`
- `XR_KHR_maintenance1_SPEC_VERSION`
- Extending XrResult:
  - `XR_ERROR_EXTENSION_DEPENDENCY_NOT_ENABLED_KHR`
  - `XR_ERROR_PERMISSION_INSUFFICIENT_KHR`

14.2.4. Version History

- Revision 1, 2023-10-25 (Ron Bessems)
  - Initial extension description

14.3. XR_EXT_hp_mixed_reality_controller

Name String

`XR_EXT_hp_mixed_reality_controller`

Extension Type

Instance extension

Registered Extension Number

96

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

API Interactions

- Interacts with `XR_EXT_dpad_binding`
- Interacts with `XR_EXT_hand_interaction`

- Interacts with `XR_EXT_palm_pose`

Deprecation State

- *Promoted* to OpenXR 1.1

Last Modified Date

2020-06-08

IP Status

No known IP claims.

Contributors

Alain Zanchetta, Microsoft
Lachlan Ford, Microsoft
Alex Turner, Microsoft
Yin Li, Microsoft
Nathan Nuber, HP Inc.

Overview

This extension added a new interaction profile path for the HP Reverb G2 Controllers:

- */interaction_profiles/hp/mixed_reality_controller*

> **Note**
>
> The interaction profile path */interaction_profiles/hp/mixed_reality_controller* defined here does not follow current rules for interaction profile names. If this extension were introduced today, it would be called */interaction_profiles/hp/mixed_reality_controller_hp*, to allow for modifications when promoted to a KHR extension or the core specification.

Valid for the user paths

- */user/hand/left*
- */user/hand/right*

Supported component paths:

- On /user/hand/left only
  - *.../input/x/click*
  - *.../input/y/click*
- On /user/hand/right only
  - *.../input/a/click*

    - *.../input/b/click*
- On both hands
    - *.../input/menu/click*
    - *.../input/squeeze/value*
    - *.../input/trigger/value*
    - *.../input/thumbstick/x*
    - *.../input/thumbstick/y*
    - *.../input/thumbstick/click*
    - *.../input/grip/pose*
    - *.../input/aim/pose*
    - *.../output/haptic*



Note

When the runtime supports `XR_VERSION_1_1` and use of OpenXR 1.1 is requested by the application, this interaction profile **must** also support

- *.../input/grip_surface/pose*



Note

When the `XR_KHR_maintenance1` extension is available and enabled, this interaction profile **must** also support

- *.../input/grip_surface/pose*



Note

When the `XR_EXT_palm_pose` extension is available and enabled, this interaction profile **must** also support

- *.../input/palm_ext/pose*



Note

When the `XR_EXT_hand_interaction` extension is available and enabled, this interaction profile **must** also support

- *.../input/pinch_ext/pose*
- *.../input/poke_ext/pose*

Version History

- Revision 1, 2020-06-08 (Yin Li)

- Initial extension proposal

14.4. XR_EXT_local_floor

Name String

`XR_EXT_local_floor`

Extension Type

Instance extension

Registered Extension Number

427

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Deprecation State

- *Promoted* to OpenXR 1.1

Last Modified Date

2022-11-28

IP Status

No known IP claims.

Contributors

John Kearney, Meta
Alex Turner, Microsoft
Yin Li, Microsoft
Cass Everitt, Meta

Contacts

John Kearney, Meta

Overview

The core OpenXR spec contains two world-locked reference space XrSpace types in XrReferenceSpaceType, `XR_REFERENCE_SPACE_TYPE_LOCAL` and `XR_REFERENCE_SPACE_TYPE_STAGE` with a design goal that `LOCAL` space gets the user positioned correctly in `XZ` space and `STAGE` gets the user

positioned correctly in Y space.

As defined in the core OpenXR spec, LOCAL space is useful when an application needs to render **seated-scale** content that is not positioned relative to the physical floor and STAGE space is useful when an application needs to render **standing-scale** content that is relative to the physical floor.

The core OpenXR specification describes that **standing-scale** experiences are meant to use the STAGE reference space. However, using the STAGE forces the user to move to the stage space in order to operate their experience, rather than just standing locally where they are.

Definition of the space

Similar to LOCAL space, the LOCAL_FLOOR reference space (XR_REFERENCE_SPACE_TYPE_LOCAL_FLOOR_EXT) establishes a world-locked origin, gravity-aligned to exclude pitch and roll, with +Y up, +X to the right, and -Z forward.

The location of the origin of the LOCAL_FLOOR space **must** match the LOCAL space in the X and Z coordinates but not in the Y coordinate.

The orientation of the LOCAL_FLOOR space **must** match the LOCAL space.

If the STAGE space is supported, then the floor level (Y coordinate) of the LOCAL_FLOOR space and the STAGE space **must** match.

If the STAGE space is not supported, then the runtime **must** give a best estimate of the floor level.

Note: The LOCAL_FLOOR space could be implemented by an application without support from the runtime by using the difference between in the Y coordinate of the pose of the LOCAL and STAGE reference spaces.

When this extension is enabled, a runtime **must** support XR_REFERENCE_SPACE_TYPE_LOCAL_FLOOR_EXT (in xrEnumerateReferenceSpaces).

When a user needs to recenter LOCAL space, the LOCAL_FLOOR space will also be recentered.

When such a recentering occurs, the runtime **must** queue the XrEventDataReferenceSpaceChangePending event, with the recentered LOCAL_FLOOR space origin only taking effect for xrLocateSpace or xrLocateViews calls whose XrTime parameter is greater than or equal to the changeTime provided in that event. Additionally, when the runtime changes the floor level (or the floor level estimate), the runtime **must** queue this event.

New Object Types

New Flag Types

New Enum Constants

XrReferenceSpaceType enumeration is extended with:

- XR_REFERENCE_SPACE_TYPE_LOCAL_FLOOR_EXT

New Enums

New Structures

Examples

If a runtime does not support the local floor extension, an application **can** construct an equivalent space using the LOCAL and STAGE spaces.

```
extern XrSession session;
extern bool supportsStageSpace;
extern bool supportsLocalFloorExtension;
extern XrTime curtime; // previously initialized

XrSpace localFloorSpace {XR_NULL_HANDLE};

if (supportsLocalFloorExtension)
{
    XrReferenceSpaceCreateInfo localFloorCreateInfo {
        .type = XR_TYPE_REFERENCE_SPACE_CREATE_INFO,
        .referenceSpaceType = XR_REFERENCE_SPACE_TYPE_LOCAL_FLOOR_EXT,
        .poseInReferenceSpace = {{0.f, 0.f, 0.f, 1.f}, {0.f, 0.f, 0.f}},
    };
    CHK_XR(xrCreateReferenceSpace(session, &localFloorCreateInfo, &localFloorSpace));
}
else if (supportsStageSpace)
{
    XrSpace localSpace {XR_NULL_HANDLE};
    XrSpace stageSpace {XR_NULL_HANDLE};

    XrReferenceSpaceCreateInfo createInfo {
        .type = XR_TYPE_REFERENCE_SPACE_CREATE_INFO,
        .poseInReferenceSpace = {
            .orientation = {
                .w = 1.f,
            },
        },
    };

    createInfo.referenceSpaceType = XR_REFERENCE_SPACE_TYPE_LOCAL;
    CHK_XR(xrCreateReferenceSpace(session, &createInfo, &localSpace));

    createInfo.referenceSpaceType = XR_REFERENCE_SPACE_TYPE_STAGE;
    CHK_XR(xrCreateReferenceSpace(session, &createInfo, &stageSpace));
```

```
    XrSpaceLocation stageLoc {
        .type = XR_TYPE_SPACE_LOCATION,
    };
    CHK_XR(xrLocateSpace(stageSpace, localSpace, curtime, &stageLoc));

    CHK_XR(xrDestroySpace(localSpace));
    CHK_XR(xrDestroySpace(stageSpace));

    float floorOffset = stageLoc.pose.position.y;

    XrReferenceSpaceCreateInfo localFloorCreateInfo {
        .type = XR_TYPE_REFERENCE_SPACE_CREATE_INFO,
        .referenceSpaceType = XR_REFERENCE_SPACE_TYPE_LOCAL,
        .poseInReferenceSpace = {{0.f, 0.f, 0.f, 1.f}, {0.f, floorOffset, 0.f}},
    };
    CHK_XR(xrCreateReferenceSpace(session, &localFloorCreateInfo, &localFloorSpace));
}
else
{
    // We do not support local floor or stage - make an educated guess
    float floorOffset = -1.5;

    XrReferenceSpaceCreateInfo localFloorCreateInfo {
        .type = XR_TYPE_REFERENCE_SPACE_CREATE_INFO,
        .referenceSpaceType = XR_REFERENCE_SPACE_TYPE_LOCAL,
        .poseInReferenceSpace = {{0.f, 0.f, 0.f, 1.f}, {0.f, floorOffset, 0.f}},
    };
    CHK_XR(xrCreateReferenceSpace(session, &localFloorCreateInfo, &localFloorSpace));
}
```

Issues

None

Version History

- Revision 1, 2022-11-28 (John Kearney)
  - Initial draft

14.5. XR_EXT_palm_pose

Name String

`XR_EXT_palm_pose`

Extension Type

Instance extension

Registered Extension Number

177

Revision

3

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Deprecation State

- *Promoted* to OpenXR 1.1

Last Modified Date

2022-05-23

IP Status

No known IP claims.

Contributors

Jack Pritz, Unity Technologies
Joe Ludwig, Valve
Rune Berg, Valve
John Kearney, Facebook
Peter Kuhn, Unity Technologies
Lachlan Ford, Microsoft

Overview

This extension defines a new "standard pose identifier" for interaction profiles, named "palm_ext". The new identifier is a pose that can be used to place application-specific visual content such as avatar visuals that may or may not match human hands. This extension also adds a new input component path using this "palm_ext" pose identifier to existing interaction profiles when active.

The application **can** use the *.../input/palm_ext/pose* component path to place visual content representing the user's physical hand location. Application visuals may depict, for example, realistic human hands that are very simply animated or creative depictions such as an animal, an alien, or robot limb extremity.

Note that this is not intended to be an alternative to extensions that perform hand tracking for more complex use cases: the use of "palm" in the name is to reflect that it is a user-focused pose rather than a held-object-focused pose.



Note

OpenXR 1.1 replaces *.../input/palm_ext/pose* with *.../input/grip_surface/pose.* The definitions of both poses are identical.

Pose Identifier

When this extension is active, a runtime **must** behave as if the following were added to the list of Standard pose identifiers.

- palm_ext - a pose that allows applications to reliably anchor visual content relative to the user's physical hand, whether the user's hand is tracked directly or its position and orientation is inferred by a physical controller. The palm pose is defined as follows:
  - The palm position: The user's physical palm centroid, at the surface of the palm.
  - The palm orientation's +X axis: When a user is holding the controller and straightens their index finger, the ray that is normal to the user's palm (away from the palm in the left hand, into the palm in the right hand).
  - The palm orientation's -Z axis: When a user is holding the controller and straightens their index finger, the ray that is parallel to their finger's pointing direction.
  - The palm orientation's +Y axis: orthogonal to +Z and +X using the right-hand rule.

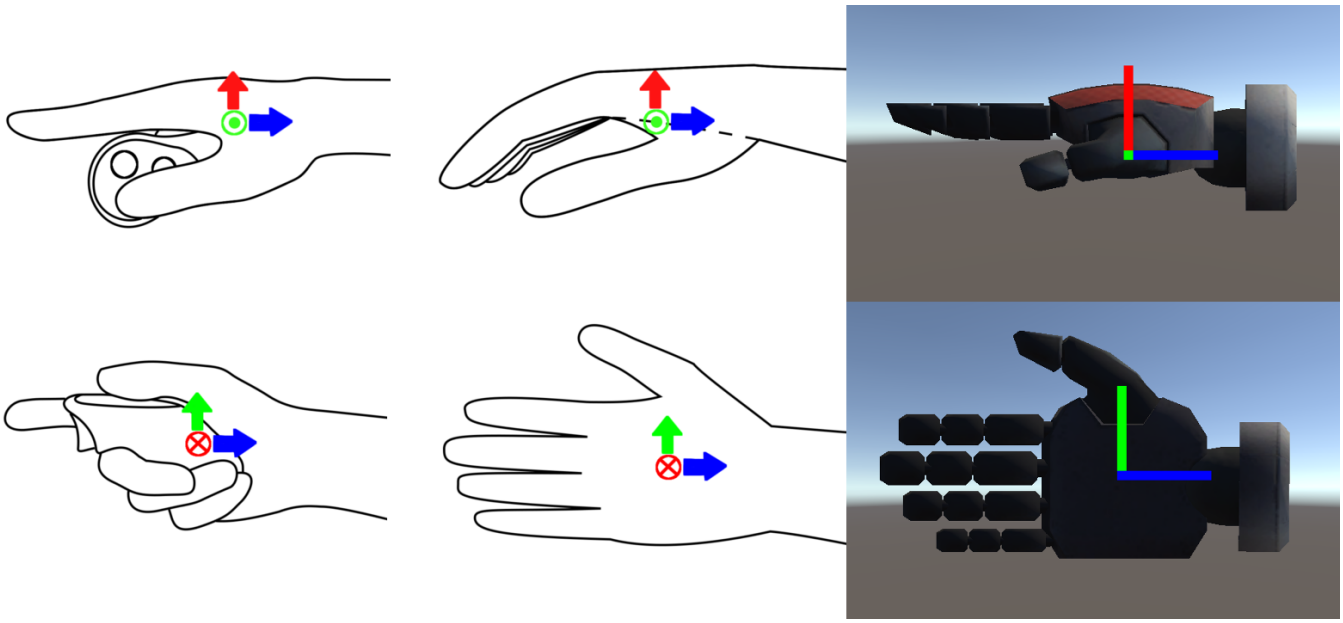


Figure 38. Example palm pose for (from left to right) a generic motion controller, tracked hand, and a digital hand avatar). The X axis is depicted in red. The Y axis is depicted in green. The Z axis is depicted in blue.

This pose is explicitly static for rigid controller type devices. The pose of *.../input/palm_ext/pose* and *.../input/grip_surface/pose* **must** be identical.

Interaction Profile Additions

When this extension is active, a runtime **must** accept the *.../input/palm_ext/pose* component path for all

interaction profiles that are valid for at least one of the user paths listed below listed below, including those interaction profiles enabled through extensions. Actions bound to such palm input component paths **must** behave as though those paths were listed in the original definition of an interaction profile.

Valid for the user paths

- */user/hand/left*
- */user/hand/right*

Supported component paths:

- On both user paths
  - *.../input/palm_ext/pose*



Note

While this extension itself does not add the *.../input/palm_ext/pose* input component path to interaction profiles defined in extensions, extension authors **may** update existing extensions to add this path, or submit new extensions defining new interaction profiles using this pose identifier and component path. For consistency, it is recommended that the *.../input/palm_ext/pose* path in extension-defined interaction profiles be specified as only valid when this `XR_EXT_palm_pose` extension is also enabled.

This extension does pose a challenge to API layer implementers attempting to provide interaction profile support through their layer. If a runtime implements `XR_EXT_palm_pose`, and an application enables it, but such an API layer is unaware of it, the runtime may "accept" (not error) the additional suggested binding but the layer will not know to provide data or indicate an active binding. This behavior, while unexpected, does not violate the specification, and does not substantially increase the difficulty of providing additional input support using an API layer.

Version History

- Revision 1, 2020-07-26 (Jack Pritz)
  - Initial extension proposal
- Revision 2, 2022-05-18 (Lachlan Ford)
  - Modification and cleanup of extension proposal based on working group discussion.
- Revision 3, 2023-11-16 (Ron Bessems)
  - Notes and clarification for the addition of *.../input/grip_surface/pose* to the core spec in OpenXR 1.1.

14.6. XR_EXT_plane_detection

Name String

`XR_EXT_plane_detection`

Extension Type

Instance extension

Registered Extension Number

430

Revision

2

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Deprecation State

- *Deprecated* by `XR_EXT_spatial_plane_tracking` extension

Last Modified Date

2024-05-23

Contributors

Aitor Font, Qualcomm
Daniel Guttenberg, Qualcomm
Maximilian Mayer, Qualcomm
Martin Renschler, Qualcomm
Karthik Nagarajan, Qualcomm
Ron Bessems, Magic Leap
Karthik Kadappan, Magic Leap

14.6.1. Overview

This extension enables applications to detect planes in the scene.

14.6.2. Runtime support

To determine if this runtime supports detecting planes xrGetSystemProperties **can** be used.

XrSystemPlaneDetectionPropertiesEXT provides information on the features supported by the runtime.

```
// Provided by XR_EXT_plane_detection
typedef struct XrSystemPlaneDetectionPropertiesEXT {
    XrStructureType                       type;
    void*                                 next;
    XrPlaneDetectionCapabilityFlagsEXT    supportedFeatures;
} XrSystemPlaneDetectionPropertiesEXT;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. For more details about the structure chain, see the structure being extended (XrSystemProperties).
- `supportedFeatures` is a bitfield, with bit masks defined in XrPlaneDetectionCapabilityFlagBitsEXT.

Valid Usage (Implicit)

- The `XR_EXT_plane_detection` extension **must** be enabled prior to using XrSystemPlaneDetectionPropertiesEXT
- `type` **must** be `XR_TYPE_SYSTEM_PLANE_DETECTION_PROPERTIES_EXT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

The XrSystemPlaneDetectionPropertiesEXT::`supportedFeatures` member is of the following type, and contains a bitwise-OR of zero or more of the bits defined in XrPlaneDetectionCapabilityFlagBitsEXT.

```
// Provided by XR_EXT_plane_detection
typedef XrFlags64 XrPlaneDetectionCapabilityFlagsEXT;
```

Valid bits for XrPlaneDetectionCapabilityFlagsEXT are defined by XrPlaneDetectionCapabilityFlagBitsEXT, which is specified as:

```
// Provided by XR_EXT_plane_detection
// Flag bits for XrPlaneDetectionCapabilityFlagsEXT
static const XrPlaneDetectionCapabilityFlagsEXT
XR_PLANE_DETECTION_CAPABILITY_PLANE_DETECTION_BIT_EXT = 0x00000001;
static const XrPlaneDetectionCapabilityFlagsEXT
XR_PLANE_DETECTION_CAPABILITY_PLANE_HOLES_BIT_EXT = 0x00000002;
static const XrPlaneDetectionCapabilityFlagsEXT
XR_PLANE_DETECTION_CAPABILITY_SEMANTIC_CEILING_BIT_EXT = 0x00000004;
static const XrPlaneDetectionCapabilityFlagsEXT
XR_PLANE_DETECTION_CAPABILITY_SEMANTIC_FLOOR_BIT_EXT = 0x00000008;
static const XrPlaneDetectionCapabilityFlagsEXT
XR_PLANE_DETECTION_CAPABILITY_SEMANTIC_WALL_BIT_EXT = 0x00000010;
static const XrPlaneDetectionCapabilityFlagsEXT
XR_PLANE_DETECTION_CAPABILITY_SEMANTIC_PLATFORM_BIT_EXT = 0x00000020;
static const XrPlaneDetectionCapabilityFlagsEXT
XR_PLANE_DETECTION_CAPABILITY_ORIENTATION_BIT_EXT = 0x00000040;
```

The flag bits have the following meanings:

Flag Descriptions

- `XR_PLANE_DETECTION_CAPABILITY_PLANE_DETECTION_BIT_EXT` — plane detection is supported
- `XR_PLANE_DETECTION_CAPABILITY_PLANE_HOLES_BIT_EXT` — polygon buffers for holes in planes can be generated
- `XR_PLANE_DETECTION_CAPABILITY_SEMANTIC_CEILING_BIT_EXT` — plane detection supports ceiling semantic classification
- `XR_PLANE_DETECTION_CAPABILITY_SEMANTIC_FLOOR_BIT_EXT` — plane detection supports floor semantic classification
- `XR_PLANE_DETECTION_CAPABILITY_SEMANTIC_WALL_BIT_EXT` — plane detection supports wall semantic classification
- `XR_PLANE_DETECTION_CAPABILITY_SEMANTIC_PLATFORM_BIT_EXT` — plane detection supports platform semantic classification (for example table tops)
- `XR_PLANE_DETECTION_CAPABILITY_ORIENTATION_BIT_EXT` — plane detection supports plane orientation classification. If not supported planes are always classified as ARBITRARY.

14.6.3. Create a plane detection handle

```
// Provided by XR_EXT_plane_detection
XR_DEFINE_HANDLE(XrPlaneDetectorEXT)
```

The XrPlaneDetectorEXT handle represents the resources for detecting one or more planes.

An application **may** create separate XrPlaneDetectorEXT handles for different sets of planes. This handle **can** be used to detect planes using other functions in this extension.

Plane detection provides locations of planes in the scene.

The xrCreatePlaneDetectorEXT function is defined as:

```
// Provided by XR_EXT_plane_detection
XrResult xrCreatePlaneDetectorEXT(
    XrSession                                   session,
    const XrPlaneDetectorCreateInfoEXT*         createInfo,
    XrPlaneDetectorEXT*                         planeDetector);
```

Parameter Descriptions

- `session` is an XrSession in which the plane detection will be active.
- `createInfo` is the XrPlaneDetectorCreateInfoEXT used to specify the plane detection.
- `planeDetector` is the returned XrPlaneDetectorEXT handle.

An application creates an XrPlaneDetectorEXT handle using xrCreatePlaneDetectorEXT function.

If the system does not support plane detection, the runtime **must** return `XR_ERROR_FEATURE_UNSUPPORTED` from xrCreatePlaneDetectorEXT.

Valid Usage (Implicit)

- The `XR_EXT_plane_detection` extension **must** be enabled prior to calling xrCreatePlaneDetectorEXT
- `session` **must** be a valid XrSession handle
- `createInfo` **must** be a pointer to a valid XrPlaneDetectorCreateInfoEXT structure
- `planeDetector` **must** be a pointer to an XrPlaneDetectorEXT handle

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FEATURE_UNSUPPORTED
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_PLANE_DETECTION_PERMISSION_DENIED_EXT
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The XrPlaneDetectorCreateInfoEXT structure is defined as:

```
// Provided by XR_EXT_plane_detection
typedef struct XrPlaneDetectorCreateInfoEXT {
    XrStructureType            type;
    const void*                next;
    XrPlaneDetectorFlagsEXT    flags;
} XrPlaneDetectorCreateInfoEXT;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- flags **must** be a valid combination of XrPlaneDetectorFlagsEXT flags or zero.

The XrPlaneDetectorCreateInfoEXT structure describes the information to create an XrPlaneDetectorEXT handle.

Valid Usage (Implicit)

- The XR_EXT_plane_detection extension **must** be enabled prior to using XrPlaneDetectorCreateInfoEXT
- type **must** be XR_TYPE_PLANE_DETECTOR_CREATE_INFO_EXT
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- flags **must** be 0 or a valid combination of XrPlaneDetectorFlagBitsEXT values

The XrPlaneDetectorCreateInfoEXT::flags member is of the following type, and contains a bitwise-OR of zero or more of the bits defined in XrPlaneDetectorFlagBitsEXT.

```
// Provided by XR_EXT_plane_detection
typedef XrFlags64 XrPlaneDetectorFlagsEXT;
```

Valid bits for XrPlaneDetectorFlagsEXT are defined by XrPlaneDetectorFlagBitsEXT, which is specified as:

```
// Provided by XR_EXT_plane_detection
// Flag bits for XrPlaneDetectorFlagsEXT
static const XrPlaneDetectorFlagsEXT XR_PLANE_DETECTOR_ENABLE_CONTOUR_BIT_EXT =
0x00000001;
```

The flag bits have the following meanings:

Flag Descriptions

- XR_PLANE_DETECTOR_ENABLE_CONTOUR_BIT_EXT — populate the plane contour information

The xrDestroyPlaneDetectorEXT function is defined as:

```
// Provided by XR_EXT_plane_detection
XrResult xrDestroyPlaneDetectorEXT(
    XrPlaneDetectorEXT                          planeDetector);
```

Parameter Descriptions

- planeDetector is an XrPlaneDetectorEXT previously created by xrCreatePlaneDetectorEXT.

xrDestroyPlaneDetectorEXT function releases the planeDetector and the underlying resources when finished with plane detection experiences.

Valid Usage (Implicit)

- The XR_EXT_plane_detection extension **must** be enabled prior to calling xrDestroyPlaneDetectorEXT
- planeDetector **must** be a valid XrPlaneDetectorEXT handle

Thread Safety

- Access to planeDetector, and any child handles, **must** be externally synchronized

Return Codes

Success

- XR_SUCCESS

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_RUNTIME_FAILURE

14.6.4. Detecting planes

The xrBeginPlaneDetectionEXT function is defined as:

```
// Provided by XR_EXT_plane_detection
XrResult xrBeginPlaneDetectionEXT(
    XrPlaneDetectorEXT                          planeDetector,
    const XrPlaneDetectorBeginInfoEXT*          beginInfo);
```

Parameter Descriptions

- planeDetector is an XrPlaneDetectorEXT previously created by xrCreatePlaneDetectorEXT.
- beginInfo is a pointer to XrPlaneDetectorBeginInfoEXT containing plane detection parameters.

The xrBeginPlaneDetectionEXT function begins the detection of planes in the scene. Detecting planes in a scene is an asynchronous operation. xrGetPlaneDetectionStateEXT **can** be used to determine if the query has finished. Once it has finished the results **may** be retrieved via xrGetPlaneDetectionsEXT. If a detection has already been started on a plane detector handle, calling xrBeginPlaneDetectionEXT again on the same handle will cancel the operation in progress and start a new detection with the new filter parameters.

The bounding volume is resolved and fixed relative to LOCAL space at the time of the call to xrBeginPlaneDetectionEXT using XrPlaneDetectorBeginInfoEXT::baseSpace, XrPlaneDetectorBeginInfoEXT::time, XrPlaneDetectorBeginInfoEXT::boundingBoxPose and XrPlaneDetectorBeginInfoEXT::boundingBoxExtent. The runtime **must** resolve the location defined by XrPlaneDetectorBeginInfoEXT::baseSpace at the time of the call. The XrPlaneDetectorBeginInfoEXT ::boundingBoxPose is the pose of the center of the box defined by XrPlaneDetectorBeginInfoEXT ::boundingBoxExtent.

The runtime **must** return XR_ERROR_SPACE_NOT_LOCATABLE_EXT if the XrPlaneDetectorBeginInfoEXT ::baseSpace is not locatable at the time of the call.

Valid Usage (Implicit)

- The XR_EXT_plane_detection extension **must** be enabled prior to calling xrBeginPlaneDetectionEXT
- planeDetector **must** be a valid XrPlaneDetectorEXT handle
- beginInfo **must** be a pointer to a valid XrPlaneDetectorBeginInfoEXT structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_POSE_INVALID
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_SPACE_NOT_LOCATABLE_EXT
- XR_ERROR_TIME_INVALID
- XR_ERROR_VALIDATION_FAILURE

The XrPlaneDetectorBeginInfoEXT structure describes the information to detect planes.

```
// Provided by XR_EXT_plane_detection
typedef struct XrPlaneDetectorBeginInfoEXT {
    XrStructureType                           type;
    const void*                               next;
    XrSpace                                   baseSpace;
    XrTime                                    time;
    uint32_t                                  orientationCount;
    const XrPlaneDetectorOrientationEXT*      orientations;
    uint32_t                                  semanticTypeCount;
    const XrPlaneDetectorSemanticTypeEXT*     semanticTypes;
    uint32_t                                  maxPlanes;
    float                                     minArea;
    XrPosef                                   boundingBoxPose;
    XrExtent3DfEXT                            boundingBoxExtent;
} XrPlaneDetectorBeginInfoEXT;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `baseSpace` is the XrSpace that the `boundingBoxPose` is defined in.
- `time` is an `XrTime` at which to detect the planes.
- `orientationCount` the number of elements in the `orientations`.
- `orientations` an array of XrPlaneDetectorOrientationEXT. If this field is null no orientation filtering is applied. If any orientations are present only planes with any of the orientation listed are returned.
- `semanticTypeCount` the number of elements in the `semanticTypes`.
- `semanticTypes` an array of XrPlaneDetectorSemanticTypeEXT. If this field is null no semantic type filtering is applied. If any semantic types are present only planes with matching semantic types are returned.
- `maxPlanes` is the maximum number of planes the runtime **may** return. This number **must** be larger than 0. If the number is 0 the runtime **must** return `XR_ERROR_VALIDATION_FAILURE`.
- `minArea` is the minimum area in square meters a plane **must** have to be returned. A runtime **may** have a lower limit under which planes are not detected regardless of `minArea` and silently drop planes lower than the internal minimum.
- `boundingBoxPose` is the pose of the center of the bounding box of the volume to use for detection in `baseSpace`.
- `boundingBoxExtent` is the extent of the bounding box to use for detection. If any part of a plane falls within the bounding box it **should** be considered for inclusion subject to the other filters. This means that planes **may** extend beyond the bounding box. A runtime **may** have an upper limit on the detection range and silently clip the results to that internally.

Valid Usage (Implicit)

- The XR_EXT_plane_detection extension **must** be enabled prior to using XrPlaneDetectorBeginInfoEXT
- type **must** be XR_TYPE_PLANE_DETECTOR_BEGIN_INFO_EXT
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- baseSpace **must** be a valid XrSpace handle
- If orientationCount is not 0, orientations **must** be a pointer to an array of orientationCount valid XrPlaneDetectorOrientationEXT values
- If semanticTypeCount is not 0, semanticTypes **must** be a pointer to an array of semanticTypeCount valid XrPlaneDetectorSemanticTypeEXT values

The xrGetPlaneDetectionStateEXT function is defined as:

```
// Provided by XR_EXT_plane_detection
XrResult xrGetPlaneDetectionStateEXT(
    XrPlaneDetectorEXT                          planeDetector,
    XrPlaneDetectionStateEXT*                   state);
```

Parameter Descriptions

- planeDetector is an XrPlaneDetectorEXT previously created by xrCreatePlaneDetectorEXT.
- state is a pointer to XrPlaneDetectionStateEXT.

The xrGetPlaneDetectionStateEXT function retrieves the state of the plane query and **must** be called before calling xrGetPlaneDetectionsEXT.

If the plane detection has not yet finished state **must** be XR_PLANE_DETECTION_STATE_PENDING_EXT. If the plane detection has finished state **must** be XR_PLANE_DETECTION_STATE_DONE_EXT. If no plane detection was previously started XR_PLANE_DETECTION_STATE_NONE_EXT **must** be returned. For all three states the function **must** return XR_SUCCESS.

When a query error occurs the function **must** return XR_SUCCESS and the appropriate error state value **must** be set.

Valid Usage (Implicit)

- The XR_EXT_plane_detection extension **must** be enabled prior to calling xrGetPlaneDetectionStateEXT
- `planeDetector` **must** be a valid XrPlaneDetectorEXT handle
- `state` **must** be a pointer to an XrPlaneDetectionStateEXT value

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The xrGetPlaneDetectionsEXT function is defined as:

```
// Provided by XR_EXT_plane_detection
XrResult xrGetPlaneDetectionsEXT(
    XrPlaneDetectorEXT                          planeDetector,
    const XrPlaneDetectorGetInfoEXT*            info,
    XrPlaneDetectorLocationsEXT*                locations);
```

Parameter Descriptions

- `planeDetector` is an XrPlaneDetectorEXT previously created by xrCreatePlaneDetectorEXT.
- `info` is a pointer to XrPlaneDetectorGetInfoEXT.
- `locations` is a pointer to XrPlaneDetectorLocationsEXT receiving the returned plane locations.

xrGetPlaneDetectionsEXT **must** return XR_ERROR_CALL_ORDER_INVALID if the detector state reported by

xrGetPlaneDetectionStateEXT is not XR_PLANE_DETECTION_STATE_DONE_EXT for the current query started by xrBeginPlaneDetectionEXT.

If the XrPlaneDetectorGetInfoEXT::baseSpace is not locatable XR_ERROR_SPACE_NOT_LOCATABLE_EXT **must** be returned.

Once xrBeginPlaneDetectionEXT is called again, the previous results for that handle are no longer available. The application **should** cache them before calling xrBeginPlaneDetectionEXT again if it needs access to that data while waiting for updated detection results.

Upon the completion of a detection cycle (xrBeginPlaneDetectionEXT, xrGetPlaneDetectionStateEXT to xrGetPlaneDetectionsEXT) the runtime **must** keep a snapshot of the plane data and no data **may** be modified. Calling xrGetPlaneDetectionsEXT multiple times with the same baseSpace and time **must** return the same plane pose data.

The current snapshot, if any, **must** be discarded upon calling xrBeginPlaneDetectionEXT.

If the XrEventDataReferenceSpaceChangePending is queued and the changeTime elapsed while the application is holding cached data the application **may** use the event data to adjust the poses accordingly.

Valid Usage (Implicit)

- The XR_EXT_plane_detection extension **must** be enabled prior to calling xrGetPlaneDetectionsEXT
- `planeDetector` **must** be a valid XrPlaneDetectorEXT handle
- `info` **must** be a pointer to a valid XrPlaneDetectorGetInfoEXT structure
- `locations` **must** be a pointer to an XrPlaneDetectorLocationsEXT structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_CALL_ORDER_INVALID
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_SPACE_NOT_LOCATABLE_EXT
- XR_ERROR_TIME_INVALID
- XR_ERROR_VALIDATION_FAILURE

XrPlaneDetectorGetInfoEXT structure contains the information required to retrieve the detected planes.

```
// Provided by XR_EXT_plane_detection
typedef struct XrPlaneDetectorGetInfoEXT {
    XrStructureType    type;
    const void*        next;
    XrSpace            baseSpace;
    XrTime             time;
} XrPlaneDetectorGetInfoEXT;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain.
- baseSpace the plane pose will be relative to this XrSpace at time.
- time is the XrTime at which to evaluate the coordinates relative to the baseSpace.

Valid Usage (Implicit)

- The XR_EXT_plane_detection extension **must** be enabled prior to using XrPlaneDetectorGetInfoEXT
- type **must** be XR_TYPE_PLANE_DETECTOR_GET_INFO_EXT
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- baseSpace **must** be a valid XrSpace handle

XrPlaneDetectorLocationsEXT structure contains information on the detected planes.

```
// Provided by XR_EXT_plane_detection
typedef struct XrPlaneDetectorLocationsEXT {
    XrStructureType                 type;
    void*                           next;
    uint32_t                        planeLocationCapacityInput;
    uint32_t                        planeLocationCountOutput;
    XrPlaneDetectorLocationEXT*     planeLocations;
} XrPlaneDetectorLocationsEXT;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain.
- planeLocationCapacityInput is the capacity of the array, or 0 to indicate a request to retrieve the required capacity.
- planeLocationCountOutput is the number of planes, or the required capacity in the case that planeLocationCapacityInput is insufficient.
- planeLocations is an array of XrPlaneDetectorLocationEXT. It **can** be NULL if planeLocationCapacityInput is 0.
- See the Buffer Size Parameters section for a detailed description of retrieving the required planeLocations size.

Valid Usage (Implicit)

- The XR_EXT_plane_detection extension **must** be enabled prior to using XrPlaneDetectorLocationsEXT
- type **must** be XR_TYPE_PLANE_DETECTOR_LOCATIONS_EXT
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- If planeLocationCapacityInput is not 0, planeLocations **must** be a pointer to an array of planeLocationCapacityInput XrPlaneDetectorLocationEXT structures

XrPlaneDetectorLocationEXT structure describes the position and orientation of a plane.

```
// Provided by XR_EXT_plane_detection
typedef struct XrPlaneDetectorLocationEXT {
    XrStructureType                   type;
    void*                             next;
    uint64_t                          planeId;
    XrSpaceLocationFlags              locationFlags;
    XrPosef                           pose;
    XrExtent2Df                       extents;
    XrPlaneDetectorOrientationEXT     orientation;
    XrPlaneDetectorSemanticTypeEXT    semanticType;
    uint32_t                          polygonBufferCount;
} XrPlaneDetectorLocationEXT;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `planeId` is a `uint64_t` unique identifier of the plane. The planeId **should** remain the same for the duration of the XrPlaneDetectorEXT handle for a physical plane. A runtime on occasion **may** assign a different id to the same physical plane, for example when several planes merge into one plane. `planeId` **must** remain valid until the next call to xrBeginPlaneDetectionEXT or xrDestroyPlaneDetectorEXT. This id is used by xrGetPlanePolygonBufferEXT.
- `locationFlags` is a bitfield, with bit masks defined in XrSpaceLocationFlagBits, to indicate which members contain valid data. If none of the bits are set, no other fields in this structure **should** be considered to be valid or meaningful.
- `pose` is an XrPosef defining the position and orientation of the origin of a plane within the reference frame of the corresponding XrPlaneDetectorGetInfoEXT::`baseSpace`.
- `extents` is the extent of the plane along the x-axis (`extents.width`) and y-axis (`extents.height`) centered on the `pose`.
- `orientation` is the detected orientation of the plane.
- `semanticType` XrPlaneDetectorSemanticTypeEXT type of the plane.
- `polygonBufferCount` is the number of polygon buffers associated with this plane. If this is zero no polygon buffer was generated. The first polygon buffer is always the outside contour. If contours are requested with `XR_PLANE_DETECTOR_ENABLE_CONTOUR_BIT_EXT` this value **must** always be at least 1.

Valid Usage (Implicit)

- The `XR_EXT_plane_detection` extension **must** be enabled prior to using XrPlaneDetectorLocationEXT
- `type` **must** be `XR_TYPE_PLANE_DETECTOR_LOCATION_EXT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `locationFlags` **must** be `0` or a valid combination of XrSpaceLocationFlagBits values
- If `orientation` is not `0`, `orientation` **must** be a valid XrPlaneDetectorOrientationEXT value
- If `semanticType` is not `0`, `semanticType` **must** be a valid XrPlaneDetectorSemanticTypeEXT value

The XrPlaneDetectorOrientationEXT enumeration identifies the different general categories of orientations of detected planes.

```
// Provided by XR_EXT_plane_detection
typedef enum XrPlaneDetectorOrientationEXT {
    XR_PLANE_DETECTOR_ORIENTATION_HORIZONTAL_UPWARD_EXT = 0,
    XR_PLANE_DETECTOR_ORIENTATION_HORIZONTAL_DOWNWARD_EXT = 1,
    XR_PLANE_DETECTOR_ORIENTATION_VERTICAL_EXT = 2,
    XR_PLANE_DETECTOR_ORIENTATION_ARBITRARY_EXT = 3,
    XR_PLANE_DETECTOR_ORIENTATION_MAX_ENUM_EXT = 0x7FFFFFFF
} XrPlaneDetectorOrientationEXT;
```

The enums have the following meanings:

Enum	Description
XR_PLANE_DETECTOR_ORIENTATION_HORIZONTAL_UPWARD_EXT	The detected plane is horizontal and faces upward (e.g. floor).
XR_PLANE_DETECTOR_ORIENTATION_HORIZONTAL_DOWNWARD_EXT	The detected plane is horizontal and faces downward (e.g. ceiling).
XR_PLANE_DETECTOR_ORIENTATION_VERTICAL_EXT	The detected plane is vertical (e.g. wall).
XR_PLANE_DETECTOR_ORIENTATION_ARBITRARY_EXT	The detected plane has an arbitrary, non-vertical and non-horizontal orientation.

The XrPlaneDetectorSemanticTypeEXT enumeration identifies the different semantic types of detected planes.

```
// Provided by XR_EXT_plane_detection
typedef enum XrPlaneDetectorSemanticTypeEXT {
    XR_PLANE_DETECTOR_SEMANTIC_TYPE_UNDEFINED_EXT = 0,
    XR_PLANE_DETECTOR_SEMANTIC_TYPE_CEILING_EXT = 1,
    XR_PLANE_DETECTOR_SEMANTIC_TYPE_FLOOR_EXT = 2,
    XR_PLANE_DETECTOR_SEMANTIC_TYPE_WALL_EXT = 3,
    XR_PLANE_DETECTOR_SEMANTIC_TYPE_PLATFORM_EXT = 4,
    XR_PLANE_DETECTOR_SEMANTIC_TYPE_MAX_ENUM_EXT = 0x7FFFFFFF
} XrPlaneDetectorSemanticTypeEXT;
```

The enums have the following meanings:

Enum	Description
XR_PLANE_DETECTOR_SEMANTIC_TYPE_UNDEFINED_EXT	The runtime was unable to classify this plane.
XR_PLANE_DETECTOR_SEMANTIC_TYPE_CEILING_EXT	The detected plane is a ceiling.
XR_PLANE_DETECTOR_SEMANTIC_TYPE_FLOOR_EXT	The detected plane is a floor.
XR_PLANE_DETECTOR_SEMANTIC_TYPE_WALL_EXT	The detected plane is a wall.

Enum	Description
XR_PLANE_DETECTOR_SEMANTIC_TYPE_PLATFORM_EXT	The detected plane is a platform, like a table.

The XrPlaneDetectionStateEXT enumeration identifies the possible states of the plane detector.

```
// Provided by XR_EXT_plane_detection
typedef enum XrPlaneDetectionStateEXT {
    XR_PLANE_DETECTION_STATE_NONE_EXT = 0,
    XR_PLANE_DETECTION_STATE_PENDING_EXT = 1,
    XR_PLANE_DETECTION_STATE_DONE_EXT = 2,
    XR_PLANE_DETECTION_STATE_ERROR_EXT = 3,
    XR_PLANE_DETECTION_STATE_FATAL_EXT = 4,
    XR_PLANE_DETECTION_STATE_MAX_ENUM_EXT = 0x7FFFFFFF
} XrPlaneDetectionStateEXT;
```

Enumerant Descriptions

- XR_PLANE_DETECTION_STATE_NONE_EXT - The plane detector is not actively looking for planes; call xrBeginPlaneDetectionEXT to start detection.
- XR_PLANE_DETECTION_STATE_PENDING_EXT - This plane detector is currently looking for planes but not yet ready with results; call xrGetPlaneDetectionsEXT again, or call xrBeginPlaneDetectionEXT to restart with new filter parameters.
- XR_PLANE_DETECTION_STATE_DONE_EXT - This plane detector has finished and results **may** now be retrieved. The results are valid until xrBeginPlaneDetectionEXT or xrDestroyPlaneDetectorEXT are called.
- XR_PLANE_DETECTION_STATE_ERROR_EXT - An error occurred. The query **may** be tried again.
- XR_PLANE_DETECTION_STATE_FATAL_EXT - An error occurred. The query **must** not be tried again.

14.6.5. Read plane polygon vertices

The xrGetPlanePolygonBufferEXT function is defined as:

```
// Provided by XR_EXT_plane_detection
XrResult xrGetPlanePolygonBufferEXT(
    XrPlaneDetectorEXT                          planeDetector,
    uint64_t                                    planeId,
    uint32_t                                    polygonBufferIndex,
    XrPlaneDetectorPolygonBufferEXT*            polygonBuffer);
```

Parameter Descriptions

- `planeDetector` is an XrPlaneDetectorEXT previously created by xrCreatePlaneDetectorEXT.
- `planeId` is the XrPlaneDetectorLocationEXT::`planeId`.
- `polygonBufferIndex` is the index of the polygon contour buffer to retrieve. This **must** be a number from 0 to XrPlaneDetectorLocationEXT:polygonBufferCount -1. Index 0 retrieves the outside contour, larger indexes retrieve holes in the plane.
- `polygonBuffer` is a pointer to XrPlaneDetectorPolygonBufferEXT receiving the returned plane polygon buffer.

The xrGetPlanePolygonBufferEXT function retrieves the plane's polygon buffer for the given `planeId` and `polygonBufferIndex`. Calling xrGetPlanePolygonBufferEXT with `polygonBufferIndex` equal to 0 **must** return the outside contour, if available. Calls with non-zero indices less than XrPlaneDetectorLocationEXT::`polygonBufferCount` **must** return polygons corresponding to holes in the plane. This feature **may** not be supported by all runtimes, check the XrSystemPlaneDetectionPropertiesEXT::`supportedFeatures` for support.

Outside contour polygon vertices **must** be ordered in counter clockwise order. Vertices of holes **must** be ordered in clockwise order. The right-hand rule is used to determine the direction of the normal of this plane. The polygon contour data is relative to the pose of the plane and coplanar with it.

This function only retrieves polygons, which means that it needs to be converted to a regular mesh to be rendered.

Valid Usage (Implicit)

- The `XR_EXT_plane_detection` extension **must** be enabled prior to calling xrGetPlanePolygonBufferEXT
- `planeDetector` **must** be a valid XrPlaneDetectorEXT handle
- `polygonBuffer` **must** be a pointer to an XrPlaneDetectorPolygonBufferEXT structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

XrPlaneDetectorPolygonBufferEXT is an input/output structure for reading plane contour polygon vertices.

```
// Provided by XR_EXT_plane_detection
typedef struct XrPlaneDetectorPolygonBufferEXT {
    XrStructureType    type;
    void*              next;
    uint32_t           vertexCapacityInput;
    uint32_t           vertexCountOutput;
    XrVector2f*        vertices;
} XrPlaneDetectorPolygonBufferEXT;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain.
- vertexCapacityInput is the capacity of the array, or 0 to indicate a request to retrieve the required capacity.
- vertexCountOutput is the count of vertices written, or the required capacity in the case that vertexCapacityInput is insufficient.
- vertices is an array of XrVector2f that **must** be filled by the runtime with the positions of the polygon vertices relative to the plane's pose.

Valid Usage (Implicit)

- The XR_EXT_plane_detection extension **must** be enabled prior to using XrPlaneDetectorPolygonBufferEXT
- type **must** be XR_TYPE_PLANE_DETECTOR_POLYGON_BUFFER_EXT
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- If vertexCapacityInput is not 0, vertices **must** be a pointer to an array of vertexCapacityInput XrVector2f structures

The XrExtent3DfEXT structure is defined as:

```
// Provided by XR_EXT_plane_detection
// XrExtent3DfEXT is an alias for XrExtent3Df
typedef struct XrExtent3Df {
    float    width;
    float    height;
    float    depth;
} XrExtent3Df;

typedef XrExtent3Df XrExtent3DfEXT;
```

Member Descriptions

- width the floating-point width of the extent.
- height the floating-point height of the extent.
- depth the floating-point depth of the extent.

The XrExtent3DfEXT structure describes a axis aligned three-dimensional floating-point extent: This structure is used for component values that **may** be fractional (floating-point). If used to represent physical distances, values **must** be in meters.

The width (X), height (Y) and depth (Z) values **must** be non-negative.

14.6.6. Example code for locating planes

The following example code demonstrates how to detect planes relative to a local space.

```
XrInstance instance;  // previously initialized
XrSystemId systemId;  // previously initialized
XrSession session;    // previously initialized
```

```
XrSpace localSpace;   // previously initialized, e.g. from
                      // XR_REFERENCE_SPACE_TYPE_LOCAL
XrSpace viewSpace;    // previously initialized, e.g. from
                      // XR_REFERENCE_SPACE_TYPE_VIEW

// The function pointers are previously initialized using
// xrGetInstanceProcAddr.
PFN_xrCreatePlaneDetectorEXT xrCreatePlaneDetectorEXT; // previously initialized
PFN_xrBeginPlaneDetectionEXT xrBeginPlaneDetectionEXT; // previously initialized
PFN_xrGetPlaneDetectionStateEXT xrGetPlaneDetectionStateEXT; // previously initialized
PFN_xrGetPlaneDetectionsEXT xrGetPlaneDetectionsEXT; // previously initialized
PFN_xrGetPlanePolygonBufferEXT xrGetPlanePolygonBufferEXT; // previously initialized


XrSystemPlaneDetectionPropertiesEXT planeDetectionProperties {
  .type = XR_TYPE_SYSTEM_PLANE_DETECTION_PROPERTIES_EXT,
};
XrSystemProperties systemProperties {
  .type = XR_TYPE_SYSTEM_PROPERTIES,
  .next = &planeDetectionProperties,
};

CHK_XR(xrGetSystemProperties(instance, systemId, &systemProperties));
if (!(planeDetectionProperties.supportedFeatures &
XR_PLANE_DETECTION_CAPABILITY_PLANE_DETECTION_BIT_EXT )) {
  // plane detection is not supported.
  return;
}

// Create a plane detection
XrPlaneDetectorEXT planeDetector{};
{
    XrPlaneDetectorCreateInfoEXT createInfo {
      .type = XR_TYPE_PLANE_DETECTOR_CREATE_INFO_EXT,
      .flags = XR_PLANE_DETECTOR_ENABLE_CONTOUR_BIT_EXT,
    };
    CHK_XR(xrCreatePlaneDetectorEXT(session, &createInfo, &planeDetector));
}


bool queryRunning = false;

std::vector<XrPlaneDetectorOrientationEXT> orientations;
orientations.push_back(XR_PLANE_DETECTOR_ORIENTATION_HORIZONTAL_UPWARD_EXT);
orientations.push_back(XR_PLANE_DETECTOR_ORIENTATION_HORIZONTAL_DOWNWARD_EXT);

std::vector<XrPlaneDetectorLocationEXT> cachedPlaneLocations;
```

```
auto processPlanes = [&](const XrTime time) {

    if (!queryRunning) {
        XrPlaneDetectorBeginInfoEXT beginInfo{
          .type = XR_TYPE_PLANE_DETECTOR_BEGIN_INFO_EXT,
          .baseSpace = viewSpace,
          .time = time,
          .orientationCount = static_cast<uint32_t>(orientations.size()),
          .orientations = orientations.data(),
          .boundingBoxPose = {
            .orientation = {
              .w = 1.0f,
            },
          },
          .boundingBoxExtent = {
            .width = 10.0f,
            .height = 10.0f,
            .depth = 10.0f,
          }
        };
        CHK_XR(xrBeginPlaneDetectionEXT(planeDetector, &beginInfo));
        queryRunning = true;
        return;
    } else {
        XrPlaneDetectionStateEXT planeDetectionState;
        if (xrGetPlaneDetectionStateEXT(planeDetector, &planeDetectionState)!=XR_SUCCESS)
{
          queryRunning = false;
          return;
        }

        switch(planeDetectionState) {
          case XR_PLANE_DETECTION_STATE_DONE_EXT:
            // query has finished, process the results.
            break;
          case XR_PLANE_DETECTION_STATE_ERROR_EXT:
            // something temporary went wrong, just
            // retry
            queryRunning = false;
            return;
          case XR_PLANE_DETECTION_STATE_FATAL_EXT:
            // there was something wrong with the query
            // do not retry.
            // exit();
            return;
          case XR_PLANE_DETECTION_STATE_PENDING_EXT:
            // query is still processing, come back on the next loop.
            return;
```

```
          default:
            // restart the query.
            queryRunning = false;
            return;
        }


        XrPlaneDetectorGetInfoEXT planeGetInfo{
          .type = XR_TYPE_PLANE_DETECTOR_GET_INFO_EXT,
          .baseSpace = localSpace,
          .time = time,
        };

        XrPlaneDetectorLocationsEXT planeLocations{
          .type = XR_TYPE_PLANE_DETECTOR_LOCATIONS_EXT,
          .planeLocationCapacityInput = 0,
          .planeLocations = nullptr,
        };

        if (xrGetPlaneDetectionsEXT(planeDetector, &planeGetInfo, &planeLocations) !=
XR_SUCCESS ) {
          queryRunning = false;
          return;
        }

        if (planeLocations.planeLocationCountOutput > 0) {
            queryRunning = false;
            std::vector<XrPlaneDetectorLocationEXT>
                locationsBuffer(planeLocations.planeLocationCountOutput,
                {
                  .type = XR_TYPE_PLANE_DETECTOR_LOCATION_EXT,
                });
            planeLocations.planeLocationCapacityInput =
                planeLocations.planeLocationCountOutput;
            planeLocations.planeLocations = locationsBuffer.data();

            CHK_XR(xrGetPlaneDetectionsEXT(planeDetector, &planeGetInfo, &
planeLocations));

            cachedPlaneLocations = locationsBuffer;

            for (int i = 0; i < planeLocations.planeLocationCountOutput; ++i) {
                const XrPosef& planeInLocalSpace = planeLocations.planeLocations[i].pose;
                auto planeId =
                    planeLocations.planeLocations[i].planeId;
                auto polygonBufferCount =
                    planeLocations.planeLocations[i].polygonBufferCount;
```

```
                for (uint32_t polygonBufferIndex=0; polygonBufferIndex <
polygonBufferCount; polygonBufferIndex++) {
                    // polygonBufferIndex = 0 -> outside contour CCW
                    // polygonBufferIndex > 0 -> holes CW
                    XrPlaneDetectorPolygonBufferEXT polygonBuffer{};
                    polygonBuffer.vertexCapacityInput = 0;

                    CHK_XR(xrGetPlanePolygonBufferEXT(planeDetector,
                        planeId, polygonBufferIndex, &polygonBuffer));

                    // allocate space and use buffer
                }
                // plane planeInLocalSpace, planeType
            }
        }
    }
};

while (1) {
    // ...
    // For every frame in frame loop
    // ...

    XrFrameState frameState;  // previously returned from xrWaitFrame
    const XrTime time = frameState.predictedDisplayTime;

    processPlanes(time);

    // Draw the planes as needed from cachedPlaneLocations.
    // drawPlanes(cachedPlaneLocations);

    // ...
    // Finish frame loop
    // ...
}
```

New Object Types

- XrPlaneDetectorEXT

New Enum Constants

XrObjectType enumeration is extended with:

- XR_OBJECT_TYPE_PLANE_DETECTOR_EXT

XrStructureType enumeration is extended with:

- XR_TYPE_PLANE_DETECTOR_CREATE_INFO_EXT
- XR_TYPE_PLANE_DETECTOR_BEGIN_INFO_EXT
- XR_TYPE_PLANE_DETECTOR_GET_INFO_EXT
- XR_TYPE_PLANE_DETECTOR_LOCATION_EXT
- XR_TYPE_PLANE_DETECTOR_POLYGON_BUFFER_EXT
- XR_TYPE_SYSTEM_PLANE_DETECTION_PROPERTIES_EXT

the XrResult enumeration is extended with:

- XR_ERROR_SPACE_NOT_LOCATABLE_EXT
- XR_ERROR_PLANE_DETECTION_PERMISSION_DENIED_EXT

New Enums

- XrPlaneDetectorOrientationEXT
- XrPlaneDetectorFlagsEXT
- XrPlaneDetectionStateEXT
- XrPlaneDetectionCapabilityFlagsEXT
- XrPlaneDetectorSemanticTypeEXT

New Structures

- XrSystemPlaneDetectionPropertiesEXT
- XrPlaneDetectorCreateInfoEXT
- XrPlaneDetectorBeginInfoEXT
- XrPlaneDetectorGetInfoEXT
- XrPlaneDetectorLocationEXT
- XrPlaneDetectorPolygonBufferEXT
- XrExtent3DfEXT

New Functions

- xrCreatePlaneDetectorEXT
- xrDestroyPlaneDetectorEXT
- xrBeginPlaneDetectionEXT
- xrGetPlaneDetectionStateEXT
- xrGetPlaneDetectionsEXT
- xrGetPlanePolygonBufferEXT

Version History

- Revision 1, 2023-06-26 (Ron Bessems)
- Revision 2, 2024-05-23 (Ron Bessems)
  - Fix extents description and plane axis to match CTS and implementations.

14.7. XR_EXT_samsung_odyssey_controller

Name String

`XR_EXT_samsung_odyssey_controller`

Extension Type

Instance extension

Registered Extension Number

95

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

API Interactions

- Interacts with `XR_EXT_dpad_binding`
- Interacts with `XR_EXT_hand_interaction`
- Interacts with `XR_EXT_palm_pose`

Deprecation State

- *Promoted* to OpenXR 1.1

Last Modified Date

2020-06-08

IP Status

No known IP claims.

Contributors

Lachlan Ford, Microsoft
Alex Turner, Microsoft

Yin Li, Microsoft
Philippe Harscoet, Samsung Electronics

Overview

This extension enables the application to differentiate the newer form factor of motion controller released with the Samsung Odyssey headset. It enables the application to customize the appearance and experience of the controller differently from the original mixed reality motion controller.

This extension added a new interaction profile */interaction_profiles/samsung/odyssey_controller* to describe the Odyssey controller. The action bindings of this interaction profile work exactly the same as the */interaction_profiles/microsoft/motion_controller* in terms of valid user paths and supported input and output component paths.

Note

The interaction profile path */interaction_profiles/samsung/odyssey_controller* defined here does not follow current rules for interaction profile names. If this extension were introduced today, it would be called */interaction_profiles/samsung/odyssey_controller_samsung*, to allow for modifications when promoted to a KHR extension or the core specification.

If the application does not do its own custom rendering for specific motion controllers, it **should** avoid using this extension and instead just use *.../microsoft/motion_controller*, as runtimes **should** treat both controllers equally when applications declare action bindings only for that profile.

If the application wants to customize rendering for specific motion controllers, it **should** setup the suggested bindings for *.../samsung/odyssey_controller* the same as *.../microsoft/motion_controller* when calling xrSuggestInteractionProfileBindings, and expect the same action bindings. Then the application **can** listen to the XrEventDataInteractionProfileChanged event and inspect the returned interaction profile from xrGetCurrentInteractionProfile to differentiate which controller is being used by the user, and hence customize the appearance or experience of the motion controller specifically for the form factor of *.../samsung/odyssey_controller*.



Note

When the runtime supports `XR_VERSION_1_1` and use of OpenXR 1.1 is requested by the application, this interaction profile **must** also support

- *.../input/grip_surface/pose*



Note

When the `XR_KHR_maintenance1` extension is available and enabled, this interaction profile **must** also support

- *.../input/grip_surface/pose*



Note

When the `XR_EXT_palm_pose` extension is available and enabled, this interaction profile **must** also support

- *.../input/palm_ext/pose*



Note

When the `XR_EXT_hand_interaction` extension is available and enabled, this interaction profile **must** also support

- *.../input/pinch_ext/pose*
- *.../input/poke_ext/pose*

Version History

- Revision 1, 2020-06-08 (Yin Li)
  - Initial extension proposal

14.8. XR_EXT_uuid

Name String

`XR_EXT_uuid`

Extension Type

Instance extension

Registered Extension Number

300

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Deprecation State

- *Promoted* to OpenXR 1.1

Last Modified Date

2021-10-27

IP Status

No known IP claims.

Contributors

Darryl Gough, Microsoft
Yin Li, Microsoft
Alex Turner, Microsoft
David Fields, Microsoft

Overview

This extension defines a Universally Unique Identifier that follows RFC 4122.

The XrUuidEXT structure is a 128-bit Universally Unique Identifier and is defined as:

```
// Provided by XR_EXT_uuid
// XrUuidEXT is an alias for XrUuid
typedef struct XrUuid {
    uint8_t    data[XR_UUID_SIZE];
} XrUuid;

typedef XrUuid XrUuidEXT;
```

Member Descriptions

- `data` is a 128-bit Universally Unique Identifier.

The structure is composed of 16 octets, with the size and order of the fields defined in RFC 4122 section 4.1.2.

Valid Usage (Implicit)

- The `XR_FB_spatial_entity` extension **must** be enabled prior to using XrUuidEXT

New Object Types

New Flag Types

New Enum Constants

- `XR_UUID_SIZE_EXT`

New Enums

New Structures

- XrUuidEXT

New Functions

Issues

Version History

- Revision 1, 2021-10-27 (Darryl Gough)
  - Initial extension description

14.9. XR_ANDROID_device_anchor_persistence

Name String

`XR_ANDROID_device_anchor_persistence`

Extension Type

Instance extension

Registered Extension Number

458

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

`XR_EXT_uuid`
and
`XR_ANDROID_trackables`

Deprecation State

- *Deprecated* by `XR_EXT_spatial_persistence` , `XR_EXT_spatial_persistence_operations` extensions

Last Modified Date

2025-07-23

IP Status

No known IP claims.

Contributors

Nihav Jain, Google
Levana Chen, Google
Spencer Quin, Google
Kenny Vercaemer, Google

14.9.1. Overview

This extension allows the application to persist, retrieve, and unpersist anchors on the current device for the current user, across application and device sessions. The anchors are persisted per app, as identified by their Android package name.



Permissions

Android applications **must** have the android.permission.SCENE_UNDERSTANDING_COARSE permission listed in their manifest as this extension depends on `XR_ANDROID_trackables` and exposes the geometry of the environment. The android.permission.SCENE_UNDERSTANDING_COARSE permission is considered a dangerous permission. The application **must** request the permission at runtime to use these functions:

- xrCreateDeviceAnchorPersistenceANDROID

(protection level: dangerous)

14.9.2. Deprecation note

The `XR_ANDROID_device_anchor_persistence` extension is now deprecated. Unlike typical OpenXR deprecations, its features are not replaced by a single successor. Instead, its functionality has been distributed across several different extensions.

System support, persistence data loading, and enumeration of persisted anchors is replaced by `XR_EXT_spatial_persistence`:

`XR_ANDROID_device_anchor_persistence` **functionality**	**Now available in**
XrSystemDeviceAnchorPersistencePropertiesANDROID & xrEnumerateSupportedPersistenceAnchorTypesANDROID	xrEnumerateSpatialPersistenceScopesEXT
XrDeviceAnchorPersistenceANDROID	XrSpatialPersistenceContextEXT
xrCreateDeviceAnchorPersistenceANDROID	xrCreateSpatialPersistenceContextAsyncEXT & xrCreateSpatialPersistenceContextCompleteEXT

XR_ANDROID_device_anchor_persistence **functionality**	**Now available in**
xrDestroyDeviceAnchorPersistenceANDROID	xrDestroySpatialPersistenceContextEXT
xrEnumeratePersistedAnchorsANDROID	Discover all persisted entities

Functions to persist and unpersist anchors are replaced by XR_EXT_spatial_persistence_operations:

XR_ANDROID_device_anchor_persistence **functionality**	**Now available in**
xrPersistAnchorANDROID & xrGetAnchorPersistStateANDROID	xrPersistSpatialEntityAsyncEXT & xrPersistSpatialEntityCompleteEXT
xrUnpersistAnchorANDROID	xrUnpersistSpatialEntityAsyncEXT & xrUnpersistSpatialEntityCompleteEXT

14.9.3. Inspect system capability

The XrSystemDeviceAnchorPersistencePropertiesANDROID structure is defined as:

```
// Provided by XR_ANDROID_device_anchor_persistence
typedef struct XrSystemDeviceAnchorPersistencePropertiesANDROID {
    XrStructureType    type;
    void*              next;
    XrBool32           supportsAnchorPersistence;
} XrSystemDeviceAnchorPersistencePropertiesANDROID;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension. For more details about the structure chain, see the structure being extended (XrSystemProperties).
- supportsAnchorPersistence is an XrBool32 indicating if the current system supports anchor persistence for spatial anchors.

An application inspects whether the system is capable of persisting spatial anchors (see xrCreateAnchorSpaceANDROID) by extending the XrSystemProperties with XrSystemDeviceAnchorPersistencePropertiesANDROID structure when calling xrGetSystemProperties.

To query the supported types for the supported trackable anchors an application calls xrEnumerateSupportedPersistenceAnchorTypesANDROID.

If and only if a runtime returns XR_FALSE for supportsAnchorPersistence, the runtime **must** return XR_ERROR_FEATURE_UNSUPPORTED from device anchor persistence functions that operate on a spatial anchor.

Valid Usage (Implicit)

- The XR_ANDROID_device_anchor_persistence extension **must** be enabled prior to using XrSystemDeviceAnchorPersistencePropertiesANDROID
- type **must** be XR_TYPE_SYSTEM_DEVICE_ANCHOR_PERSISTENCE_PROPERTIES_ANDROID
- next **must** be NULL or a valid pointer to the next structure in a structure chain

The xrEnumerateSupportedPersistenceAnchorTypesANDROID function is defined as:

```
// Provided by XR_ANDROID_device_anchor_persistence
XrResult xrEnumerateSupportedPersistenceAnchorTypesANDROID(
    XrInstance                                  instance,
    XrSystemId                                  systemId,
    uint32_t                                    trackableTypeCapacityInput,
    uint32_t*                                   trackableTypeCountOutput,
    XrTrackableTypeANDROID*                     trackableTypes);
```

Parameter Descriptions

- instance is the XrInstance that creates the XrDeviceAnchorPersistenceANDROID.
- systemId is the XrSystemId that is enumerated for supported types.
- trackableTypeCapacityInput is the capacity of the trackableTypes, or 0 to retrieve the required capacity.
- trackableTypeCountOutput is a pointer to the count of trackableTypes written, or a pointer to the required capacity in the case that trackableTypeCapacityInput is insufficient.
- trackableTypes is a pointer to an array of XrTrackableTypeANDROID, but **can** be NULL if trackableTypeCapacityInput is 0.
- See the Buffer Size Parameters section for a detailed description of retrieving the required trackableTypes size.

To check for support of anchor persistence on other XrTrackableTypeANDROID trackables the application calls xrEnumerateSupportedPersistenceAnchorTypesANDROID.

If and only if a runtime does not return a given XrTrackableTypeANDROID in the trackableTypes array, the runtime **must** return XR_ERROR_FEATURE_UNSUPPORTED from device anchor persistence functions that operate on an anchor of that type.

Valid Usage (Implicit)

- The `XR_ANDROID_device_anchor_persistence` extension **must** be enabled prior to calling xrEnumerateSupportedPersistenceAnchorTypesANDROID
- `instance` **must** be a valid XrInstance handle
- `trackableTypeCountOutput` **must** be a pointer to a `uint32_t` value
- If `trackableTypeCapacityInput` is not `0`, `trackableTypes` **must** be a pointer to an array of `trackableTypeCapacityInput` XrTrackableTypeANDROID values

Return Codes

Success

- `XR_SUCCESS`

Failure

- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SIZE_INSUFFICIENT`
- `XR_ERROR_SYSTEM_INVALID`
- `XR_ERROR_VALIDATION_FAILURE`

14.9.4. Create a device anchor persistence handle

An XrDeviceAnchorPersistenceANDROID is a handle that represents the resources required to persist anchors.

```
// Provided by XR_ANDROID_device_anchor_persistence
XR_DEFINE_HANDLE(XrDeviceAnchorPersistenceANDROID)
```

The xrCreateDeviceAnchorPersistenceANDROID function is defined as:

```
// Provided by XR_ANDROID_device_anchor_persistence
XrResult xrCreateDeviceAnchorPersistenceANDROID(
    XrSession                                       session,
    const XrDeviceAnchorPersistenceCreateInfoANDROID* createInfo,
    XrDeviceAnchorPersistenceANDROID*               outHandle);
```

Parameter Descriptions

- `session` is the XrSession that creates the XrDeviceAnchorPersistenceANDROID.
- `createInfo` is the XrDeviceAnchorPersistenceCreateInfoANDROID containing parameters for the XrDeviceAnchorPersistenceANDROID.
- `outHandle` is a pointer to a handle in which the created XrDeviceAnchorPersistenceANDROID is returned.

An application creates an XrDeviceAnchorPersistenceANDROID handle by calling xrCreateDeviceAnchorPersistenceANDROID.

This function starts off an asynchronous loading of the application's persisted data. All functions of this extensions, except xrDestroyDeviceAnchorPersistenceANDROID, **must** return `XR_ERROR_PERSISTED_DATA_NOT_READY_ANDROID` if the data loading is not complete. The application retries those functions at a later time.

The XrDeviceAnchorPersistenceANDROID handle **must** be eventually freed via the xrDestroyDeviceAnchorPersistenceANDROID function or by destroying the parent XrSession handle.

Valid Usage (Implicit)

- The `XR_ANDROID_device_anchor_persistence` extension **must** be enabled prior to calling xrCreateDeviceAnchorPersistenceANDROID
- `session` **must** be a valid XrSession handle
- `createInfo` **must** be a pointer to a valid XrDeviceAnchorPersistenceCreateInfoANDROID structure
- `outHandle` **must** be a pointer to an XrDeviceAnchorPersistenceANDROID handle

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The XrDeviceAnchorPersistenceCreateInfoANDROID structure is defined as:

```
// Provided by XR_ANDROID_device_anchor_persistence
typedef struct XrDeviceAnchorPersistenceCreateInfoANDROID {
    XrStructureType    type;
    const void*        next;
} XrDeviceAnchorPersistenceCreateInfoANDROID;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.

The XrDeviceAnchorPersistenceCreateInfoANDROID structure provides creation options for the XrDeviceAnchorPersistenceANDROID when passed to xrCreateDeviceAnchorPersistenceANDROID.

Valid Usage (Implicit)

- The XR_ANDROID_device_anchor_persistence extension **must** be enabled prior to using XrDeviceAnchorPersistenceCreateInfoANDROID
- `type` **must** be `XR_TYPE_DEVICE_ANCHOR_PERSISTENCE_CREATE_INFO_ANDROID`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

The xrDestroyDeviceAnchorPersistenceANDROID function is defined as:

```
// Provided by XR_ANDROID_device_anchor_persistence
XrResult xrDestroyDeviceAnchorPersistenceANDROID(
    XrDeviceAnchorPersistenceANDROID            handle);
```

Parameter Descriptions

- `handle` is an XrDeviceAnchorPersistenceANDROID handle previously created by xrCreateDeviceAnchorPersistenceANDROID.

The xrDestroyDeviceAnchorPersistenceANDROID function destroys the device anchor persistence handle. After this call the runtime **may** free all related memory and resources.

Valid Usage (Implicit)

- The XR_ANDROID_device_anchor_persistence extension **must** be enabled prior to calling xrDestroyDeviceAnchorPersistenceANDROID
- `handle` **must** be a valid XrDeviceAnchorPersistenceANDROID handle

Thread Safety

- Access to `handle`, and any child handles, **must** be externally synchronized

Return Codes

Success

- XR_SUCCESS

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_RUNTIME_FAILURE

14.9.5. Persist an anchor

The xrPersistAnchorANDROID function is defined as:

```
// Provided by XR_ANDROID_device_anchor_persistence
XrResult xrPersistAnchorANDROID(
    XrDeviceAnchorPersistenceANDROID            handle,
    const XrPersistedAnchorSpaceInfoANDROID*    persistedInfo,
    XrUuidEXT*                                  anchorIdOutput);
```

Parameter Descriptions

- handle is the XrDeviceAnchorPersistenceANDROID.
- persistedInfo is the XrPersistedAnchorSpaceInfoANDROID containing parameters for the anchor persistence.
- anchorIdOutput is a pointer to an XrUuidEXT in which the UUID of the persisted anchor is returned.

The application requests anchors to be persisted by calling xrPersistAnchorANDROID. The application **must** not assume a success return value means the anchor is immediately persisted. The application **should** use xrGetAnchorPersistStateANDROID to check the persist state of the anchor using the returned anchor XrUuidEXT.

- The runtime **must** return XR_ERROR_PERSISTED_DATA_NOT_READY_ANDROID if the persisted data loading for handle is not yet complete.
- The runtime **must** return XR_SUCCESS once the anchor has been queued for persistence.

To unpersist the anchor, the application calls xrUnpersistAnchorANDROID.

Valid Usage (Implicit)

- The XR_ANDROID_device_anchor_persistence extension **must** be enabled prior to calling xrPersistAnchorANDROID
- `handle` **must** be a valid XrDeviceAnchorPersistenceANDROID handle
- `persistedInfo` **must** be a pointer to a valid XrPersistedAnchorSpaceInfoANDROID structure
- `anchorIdOutput` **must** be a pointer to an XrUuidEXT structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_ANCHOR_NOT_OWNED_BY_CALLER_ANDROID
- XR_ERROR_ANCHOR_NOT_TRACKING_ANDROID
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_PERSISTED_DATA_NOT_READY_ANDROID
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The XrPersistedAnchorSpaceInfoANDROID structure is defined as:

```
// Provided by XR_ANDROID_device_anchor_persistence
typedef struct XrPersistedAnchorSpaceInfoANDROID {
    XrStructureType    type;
    const void*        next;
    XrSpace            anchor;
} XrPersistedAnchorSpaceInfoANDROID;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `anchor` is an anchor XrSpace previously created by xrCreateAnchorSpaceANDROID to persist.

Valid Usage (Implicit)

- The `XR_ANDROID_device_anchor_persistence` extension **must** be enabled prior to using XrPersistedAnchorSpaceInfoANDROID
- `type` **must** be `XR_TYPE_PERSISTED_ANCHOR_SPACE_INFO_ANDROID`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `anchor` **must** be a valid XrSpace handle

The xrGetAnchorPersistStateANDROID function is defined as:

```
// Provided by XR_ANDROID_device_anchor_persistence
XrResult xrGetAnchorPersistStateANDROID(
    XrDeviceAnchorPersistenceANDROID            handle,
    const XrUuidEXT*                            anchorId,
    XrAnchorPersistStateANDROID*                persistState);
```

Parameter Descriptions

- `handle` is the XrDeviceAnchorPersistenceANDROID.
- `anchorId` is the XrUuidEXT of the anchor.
- `persistState` is a pointer to an XrAnchorPersistStateANDROID in which the state of the anchor is returned.

- The runtime **must** set `persistState` to `XR_ANCHOR_PERSIST_STATE_PERSIST_NOT_REQUESTED_ANDROID` if the anchor XrUuidEXT has not been requested for persistence.
- The runtime **must** return `XR_ERROR_PERSISTED_DATA_NOT_READY_ANDROID` if the persisted data of `anchorId` is not ready.

Valid Usage (Implicit)

- The XR_ANDROID_device_anchor_persistence extension **must** be enabled prior to calling xrGetAnchorPersistStateANDROID
- `handle` **must** be a valid XrDeviceAnchorPersistenceANDROID handle
- `anchorId` **must** be a pointer to a valid XrUuidEXT structure
- `persistState` **must** be a pointer to an XrAnchorPersistStateANDROID value

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_ANCHOR_ID_NOT_FOUND_ANDROID
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_PERSISTED_DATA_NOT_READY_ANDROID
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The XrAnchorPersistStateANDROID enum is defined as:

```
// Provided by XR_ANDROID_device_anchor_persistence
typedef enum XrAnchorPersistStateANDROID {
    XR_ANCHOR_PERSIST_STATE_PERSIST_NOT_REQUESTED_ANDROID = 0,
    XR_ANCHOR_PERSIST_STATE_PERSIST_PENDING_ANDROID = 1,
    XR_ANCHOR_PERSIST_STATE_PERSISTED_ANDROID = 2,
    XR_ANCHOR_PERSIST_STATE_MAX_ENUM_ANDROID = 0x7FFFFFFF
} XrAnchorPersistStateANDROID;
```

Enum	Description
XR_ANCHOR_PERSIST_STATE_PERSIST_NOT_REQUESTED_ANDROID	Anchor has not been requested to be persisted by the app.

Enum	Description
XR_ANCHOR_PERSIST_STATE_PERSIST_PENDING_ANDROID	Anchor has been requested to be persisted but not persisted yet.
XR_ANCHOR_PERSIST_STATE_PERSISTED_ANDROID	Anchor has been successfully persisted by the runtime.

14.9.6. Enumerate persisted anchors

The xrEnumeratePersistedAnchorsANDROID function is defined as:

```
// Provided by XR_ANDROID_device_anchor_persistence
XrResult xrEnumeratePersistedAnchorsANDROID(
    XrDeviceAnchorPersistenceANDROID            handle,
    uint32_t                                    anchorIdCapacityInput,
    uint32_t*                                   anchorIdCountOutput,
    XrUuidEXT*                                  anchorIds);
```

Parameter Descriptions

- handle is the XrDeviceAnchorPersistenceANDROID.
- anchorIdCapacityInput is the capacity of the anchorIds array, or 0 to indicate a request to retrieve the required capacity.
- anchorIdCountOutput is a pointer to the count of anchorIds written, or a pointer to the required capacity in the case that anchorIdCapacityInput is insufficient.
- anchorIds is a pointer to an array of XrUuidEXT structures. It **can** be NULL if anchorIdCapacityInput is 0.
- See the Buffer Size Parameters section for a detailed description of retrieving the required anchorIds size.

To enumerate all current persisted anchors, the application calls xrEnumeratePersistedAnchorsANDROID. anchorIds will hold the UUIDs of the persisted anchors up to the capacity of the array. Since anchors are persisted asynchronously, the count of persisted anchors **may** change in between calls, causing anchorIdCountOutput to differ as well. If the capacity is insufficient, the runtime **must** truncate the values to fit the output array and makes no guarantees about which anchors are returned. The runtime **must** return XR_ERROR_PERSISTED_DATA_NOT_READY_ANDROID if the persisted data loading for handle is not yet complete.

Valid Usage (Implicit)

- The `XR_ANDROID_device_anchor_persistence` extension **must** be enabled prior to calling xrEnumeratePersistedAnchorsANDROID
- `handle` **must** be a valid XrDeviceAnchorPersistenceANDROID handle
- `anchorIdCountOutput` **must** be a pointer to a `uint32_t` value
- If `anchorIdCapacityInput` is not `0`, `anchorIds` **must** be a pointer to an array of `anchorIdCapacityInput` XrUuidEXT structures

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_PERSISTED_DATA_NOT_READY_ANDROID`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_SIZE_INSUFFICIENT`
- `XR_ERROR_VALIDATION_FAILURE`

14.9.7. Create an anchor from persisted data

The xrCreatePersistedAnchorSpaceANDROID function is defined as:

```
// Provided by XR_ANDROID_device_anchor_persistence
XrResult xrCreatePersistedAnchorSpaceANDROID(
    XrDeviceAnchorPersistenceANDROID            handle,
    const XrPersistedAnchorSpaceCreateInfoANDROID* createInfo,
    XrSpace*                                    anchorOutput);
```

Parameter Descriptions

- `handle` is the XrDeviceAnchorPersistenceANDROID.
- `createInfo` is the XrPersistedAnchorSpaceCreateInfoANDROID containing parameters for the anchor creation.
- `anchorOutput` is a pointer to an XrSpace in which the new anchor space is returned.

The application creates an XrSpace anchor from a previously persisted anchor by calling xrCreatePersistedAnchorSpaceANDROID with the same XrUuidEXT. This is another way of creating anchors as defined in `XR_ANDROID_trackables`.

- The runtime **must** return `XR_ERROR_PERSISTED_DATA_NOT_READY_ANDROID` if the persisted data loading for `handle` is not yet complete.
- The runtime **must** return `XR_ERROR_ANCHOR_ID_NOT_FOUND_ANDROID` if the anchor XrUuidEXT is not found.
- If the XrUuidEXT refers to an anchor that has not reached the `XR_ANCHOR_PERSIST_STATE_PERSISTED_ANDROID` state yet, the runtime **must** return `XR_ERROR_ANCHOR_ID_NOT_FOUND_ANDROID`.
- Despite the first parameter of this function being an XrDeviceAnchorPersistenceANDROID, the parent of the created XrSpace is the XrSession that is the parent of `handle`. The runtime **must** track the anchor until `anchorOutput` is destroyed.

Valid Usage (Implicit)

- The `XR_ANDROID_device_anchor_persistence` extension **must** be enabled prior to calling xrCreatePersistedAnchorSpaceANDROID
- `handle` **must** be a valid XrDeviceAnchorPersistenceANDROID handle
- `createInfo` **must** be a pointer to a valid XrPersistedAnchorSpaceCreateInfoANDROID structure
- `anchorOutput` **must** be a pointer to an XrSpace handle

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_ANCHOR_ID_NOT_FOUND_ANDROID
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_PERSISTED_DATA_NOT_READY_ANDROID
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The XrPersistedAnchorSpaceCreateInfoANDROID structure is defined as:

```
// Provided by XR_ANDROID_device_anchor_persistence
typedef struct XrPersistedAnchorSpaceCreateInfoANDROID {
    XrStructureType    type;
    const void*        next;
    XrUuidEXT          anchorId;
} XrPersistedAnchorSpaceCreateInfoANDROID;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- anchorId is the XrUuidEXT of the persisted anchor to create an anchor XrSpace from.

The XrPersistedAnchorSpaceCreateInfoANDROID structure provides creation options for the anchor when passed to xrCreateDeviceAnchorPersistenceANDROID.

Valid Usage (Implicit)

- The XR_ANDROID_device_anchor_persistence extension **must** be enabled prior to using XrPersistedAnchorSpaceCreateInfoANDROID
- type **must** be XR_TYPE_PERSISTED_ANCHOR_SPACE_CREATE_INFO_ANDROID
- next **must** be NULL or a valid pointer to the next structure in a structure chain

14.9.8. Unpersist a persisted anchor

The xrUnpersistAnchorANDROID function is defined as:

```
// Provided by XR_ANDROID_device_anchor_persistence
XrResult xrUnpersistAnchorANDROID(
    XrDeviceAnchorPersistenceANDROID            handle,
    const XrUuidEXT*                            anchorId);
```

Parameter Descriptions

- handle is the XrDeviceAnchorPersistenceANDROID.
- anchorId is the XrUuidEXT of the anchor to unpersist.

The application unpersists a persisted anchor by calling xrUnpersistAnchorANDROID and passing the anchor XrUuidEXT of the anchor to unpersist. The runtime **may** take some time to unpersist the anchor. Applications **should** use xrEnumeratePersistedAnchorsANDROID to verify that an anchor has been unpersisted before exiting. Anchor spaces that have been created from persisted anchors are unaffected by unpersisting and keep tracking as normal anchors.

- The runtime **must** return XR_ERROR_PERSISTED_DATA_NOT_READY_ANDROID if the persisted data is not ready.
- The runtime **must** return XR_ERROR_ANCHOR_ID_NOT_FOUND_ANDROID if the anchor XrUuidEXT is not found.
- The runtime **must** guarantee that the anchor will be unpersisted, regardless if the anchor at the time of the call is in either pending or persisted state. I.e. A call to xrPersistAnchorANDROID followed by a call to xrUnpersistAnchorANDROID **must** always result in the anchor being unpersisted, or never persisted.

Valid Usage (Implicit)

- The `XR_ANDROID_device_anchor_persistence` extension **must** be enabled prior to calling xrUnpersistAnchorANDROID
- `handle` **must** be a valid XrDeviceAnchorPersistenceANDROID handle
- `anchorId` **must** be a pointer to a valid XrUuidEXT structure

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_ANCHOR_ID_NOT_FOUND_ANDROID`
- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_LIMIT_REACHED`
- `XR_ERROR_OUT_OF_MEMORY`
- `XR_ERROR_PERSISTED_DATA_NOT_READY_ANDROID`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_VALIDATION_FAILURE`

14.9.9. Example code for anchor persistence

The following example code demonstrates how to inspect system capability, persist, enumerate and unpersist anchors, as well as creating an anchor from persisted anchor XrUuidEXT.

```
XrSession session; // previously initialized
XrSpace anchor; // previously initialized
XrSpace appSpace; // previously initialized

// The function pointers are previously initialized using xrGetInstanceProcAddr.
PFN_xrEnumerateSupportedPersistenceAnchorTypesANDROID
xrEnumerateSupportedPersistenceAnchorTypesANDROID; // previously initialized
PFN_xrCreateDeviceAnchorPersistenceANDROID xrCreateDeviceAnchorPersistenceANDROID; //
```

```
previously initialized
PFN_xrDestroyDeviceAnchorPersistenceANDROID xrDestroyDeviceAnchorPersistenceANDROID; //
previously initialized
PFN_xrPersistAnchorANDROID xrPersistAnchorANDROID; // previously initialized
PFN_xrGetAnchorPersistStateANDROID xrGetAnchorPersistStateANDROID; // previously
initialized
PFN_xrCreatePersistedAnchorSpaceANDROID xrCreatePersistedAnchorSpaceANDROID; //
previously initialized
PFN_xrEnumeratePersistedAnchorsANDROID xrEnumeratePersistedAnchorsANDROID; // previously
initialized
PFN_xrUnpersistAnchorANDROID xrUnpersistAnchorANDROID; // previously initialized

// Create a device anchor persistence handle
XrDeviceAnchorPersistenceCreateInfoANDROID persistenceHandleCreateInfo {
  .type = XR_TYPE_DEVICE_ANCHOR_PERSISTENCE_CREATE_INFO_ANDROID,
  .next = nullptr,
};

XrDeviceAnchorPersistenceANDROID persistenceHandle;
CHK_XR(xrCreateDeviceAnchorPersistenceANDROID(session, &persistenceHandleCreateInfo,
&persistenceHandle));

/// Persist an anchor
XrPersistedAnchorSpaceInfoANDROID anchorSpaceInfo {
  .type = XR_TYPE_PERSISTED_ANCHOR_SPACE_INFO_ANDROID,
  .next = nullptr,
  .anchor = anchor,
};

XrUuidEXT anchorId;
XrResult result;
do {
  result = xrPersistAnchorANDROID(
    persistenceHandle, &anchorSpaceInfo, &anchorId);
} while (result == XR_ERROR_PERSISTED_DATA_NOT_READY_ANDROID ||
         result == XR_ERROR_ANCHOR_NOT_TRACKING_ANDROID);
if (result != XR_SUCCESS) {
  // Handle errors
}

// ... Update loop ...
// Poll for anchor persist state to confirm if it was successfully persisted
XrAnchorPersistStateANDROID persistState;
CHK_XR(xrGetAnchorPersistStateANDROID(persistenceHandle, &anchorId, &persistState));
if (persistState == XR_ANCHOR_PERSIST_STATE_PERSISTED_ANDROID)  {
  // The anchor was persisted successfully
}
```

```
// Enumerate all persisted anchors
uint32_t anchorCountOutput = 0;
std::vector<XrUuidEXT> allAnchors;

CHK_XR(xrEnumeratePersistedAnchorsANDROID(
  persistenceHandle,
  anchorCountOutput,
  &anchorCountOutput,
  nullptr
));
allAnchors.resize(anchorCountOutput, XrUuidEXT{.data={0}});

// Fetch the actual anchors in an appropriately resized array.
CHK_XR(xrEnumeratePersistedAnchorsANDROID(
  persistenceHandle,
  anchorCountOutput,
  &anchorCountOutput,
  allAnchors.data()
));

// Creating an anchor from a previously persisted anchor using its UUID
XrTime updateTime; // Time used for the current frame's simulation update.
anchorId = allAnchors[0];

XrPersistedAnchorSpaceCreateInfoANDROID createInfo {
  .type = XR_TYPE_PERSISTED_ANCHOR_SPACE_CREATE_INFO_ANDROID,
  .next = nullptr,
  .anchorId = anchorId,
};

XrSpace anchorSpace = XR_NULL_HANDLE;
CHK_XR(xrCreatePersistedAnchorSpaceANDROID(
  persistenceHandle,
  &createInfo,
  &anchorSpace
));

// The anchor was found and retrieved from the local device successfully.
XrSpaceLocation anchorLocation {
  .type = XR_TYPE_SPACE_LOCATION,
};
CHK_XR(xrLocateSpace(anchorSpace, appSpace, updateTime, &anchorLocation));
XrPosef pose = anchorLocation.pose;

// Unpersist the anchor
do {
  result = xrUnpersistAnchorANDROID(persistenceHandle, &anchorId);
} while (result == XR_ERROR_PERSISTED_DATA_NOT_READY_ANDROID);
```

```
if (result != XR_SUCCESS) {
  // Handle errors
}

// Once app is done with all persistence related tasks
CHK_XR(xrDestroySpace(anchorSpace));
CHK_XR(xrDestroyDeviceAnchorPersistenceANDROID(persistenceHandle));
```

14.9.10. New Object Types

- XrDeviceAnchorPersistenceANDROID

14.9.11. New Commands

- xrCreateDeviceAnchorPersistenceANDROID
- xrCreatePersistedAnchorSpaceANDROID
- xrDestroyDeviceAnchorPersistenceANDROID
- xrEnumeratePersistedAnchorsANDROID
- xrEnumerateSupportedPersistenceAnchorTypesANDROID
- xrGetAnchorPersistStateANDROID
- xrPersistAnchorANDROID
- xrUnpersistAnchorANDROID

14.9.12. New Structures

- XrDeviceAnchorPersistenceCreateInfoANDROID
- XrPersistedAnchorSpaceCreateInfoANDROID
- XrPersistedAnchorSpaceInfoANDROID
- Extending XrSystemProperties:
  - XrSystemDeviceAnchorPersistencePropertiesANDROID

14.9.13. New Enums

- XrAnchorPersistStateANDROID

14.9.14. New Enum Constants

- `XR_ANDROID_DEVICE_ANCHOR_PERSISTENCE_EXTENSION_NAME`
- `XR_ANDROID_device_anchor_persistence_SPEC_VERSION`
- Extending XrObjectType:

    - XR_OBJECT_TYPE_DEVICE_ANCHOR_PERSISTENCE_ANDROID
- Extending XrResult:
    - XR_ERROR_ANCHOR_ALREADY_PERSISTED_ANDROID
    - XR_ERROR_ANCHOR_ID_NOT_FOUND_ANDROID
    - XR_ERROR_ANCHOR_NOT_TRACKING_ANDROID
    - XR_ERROR_PERSISTED_DATA_NOT_READY_ANDROID
- Extending XrStructureType:
    - XR_TYPE_DEVICE_ANCHOR_PERSISTENCE_CREATE_INFO_ANDROID
    - XR_TYPE_PERSISTED_ANCHOR_SPACE_CREATE_INFO_ANDROID
    - XR_TYPE_PERSISTED_ANCHOR_SPACE_INFO_ANDROID
    - XR_TYPE_SYSTEM_DEVICE_ANCHOR_PERSISTENCE_PROPERTIES_ANDROID

14.9.15. Issues

14.9.16. Version History

- Revision 1, 2025-07-23 (Kenny Vercaemer)
    - Initial extension description

14.10. XR_ANDROID_raycast

Name String

XR_ANDROID_raycast

Extension Type

Instance extension

Registered Extension Number

464

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

XR_ANDROID_trackables

Deprecation State

- *Deprecated* by `XR_ANDROID_spatial_discovery_raycast` extension

Last Modified Date

2025-07-23

IP Status

No known IP claims.

Contributors

Spencer Quin, Google
Nihav Jain, Google
John Pursey, Google
Jared Finder, Google
Levana Chen, Google
Kenny Vercaemer, Google

14.10.1. Overview

This extension allows the application to perform raycasts against trackables in the environment.

Raycasts are useful for detecting objects in the environment that are in the trajectory of a ray from a given origin. For example:

- To determine where a floating object will fall when dropped, by using a vertical raycast.
- To determine where a user is looking, by using a forward-facing raycast.



Permissions

Android applications **must** have the android.permission.SCENE_UNDERSTANDING_COARSE permission listed in their manifest as this extension depends on `XR_ANDROID_trackables` and exposes the geometry of the environment. The android.permission.SCENE_UNDERSTANDING_COARSE permission is considered a dangerous permission.

(protection level: dangerous)

14.10.2. Query supported raycast capabilities

The xrEnumerateRaycastSupportedTrackableTypesANDROID function is defined as:

```
// Provided by XR_ANDROID_raycast
XrResult xrEnumerateRaycastSupportedTrackableTypesANDROID(
    XrInstance                                  instance,
    XrSystemId                                  systemId,
    uint32_t                                    trackableTypeCapacityInput,
    uint32_t*                                   trackableTypeCountOutput,
    XrTrackableTypeANDROID*                     trackableTypes);
```

Parameter Descriptions

- `instance` is the XrInstance from which `systemId` was retrieved.
- `systemId` is the `XrSystemId` whose supported trackable types for raycasting are being enumerated.
- `trackableTypeCapacityInput` is the capacity of the `trackableTypes`, or 0 to retrieve the required capacity.
- `trackableTypeCountOutput` is a pointer to the count of the array, or a pointer to the required capacity in the case that `trackableTypeCapacityInput` is insufficient.
- `trackableTypes` is a pointer to an array of XrTrackableTypeANDROID, but **can** be `NULL` if `trackableTypeCapacityInput` is `0`.
- See the Buffer Size Parameters section for a detailed description of retrieving the required `trackableTypes` size.

xrEnumerateRaycastSupportedTrackableTypesANDROID enumerates which trackable types the runtime supports raycasting for.

Valid Usage (Implicit)

- The `XR_ANDROID_raycast` extension **must** be enabled prior to calling xrEnumerateRaycastSupportedTrackableTypesANDROID
- `instance` **must** be a valid XrInstance handle
- `trackableTypeCountOutput` **must** be a pointer to a `uint32_t` value
- If `trackableTypeCapacityInput` is not `0`, `trackableTypes` **must** be a pointer to an array of `trackableTypeCapacityInput` XrTrackableTypeANDROID values

Return Codes

Success

- XR_SUCCESS

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SIZE_INSUFFICIENT
- XR_ERROR_SYSTEM_INVALID
- XR_ERROR_VALIDATION_FAILURE

14.10.3. Performing a raycast

The xrRaycastANDROID function is defined as:

```
// Provided by XR_ANDROID_raycast
XrResult xrRaycastANDROID(
    XrSession                                   session,
    const XrRaycastInfoANDROID*                 rayInfo,
    XrRaycastHitResultsANDROID*                 results);
```

Parameter Descriptions

- session is the XrSession that the raycast is performed in.
- rayInfo is the XrRaycastInfoANDROID that describes the ray to cast.
- results is a pointer to the XrRaycastHitResultsANDROID that receives the results of the raycast.

To perform raycasts, the application calls xrRaycastANDROID.

- If a raycast intersects more trackables than XrRaycastInfoANDROID::maxResults, the runtime **must** return the hit results that are closest to the XrRaycastInfoANDROID::origin of the ray.
- If a raycast intersects a trackable of type XR_TRACKABLE_TYPE_PLANE_ANDROID, that is subsumed by another plane, the runtime **must** return the hit result for the subsuming plane only.
- The runtime **must** return the hit results in closest-to-farthest order from XrRaycastInfoANDROID

::origin along the XrRaycastInfoANDROID::trajectory vector.

- The runtime **must** return XR_ERROR_TRACKABLE_TYPE_NOT_SUPPORTED_ANDROID if the trackable type corresponding to the XrTrackableTrackerANDROID handles in XrRaycastInfoANDROID::trackers are not enumerated by xrEnumerateRaycastSupportedTrackableTypesANDROID.

Valid Usage (Implicit)

- The XR_ANDROID_raycast extension **must** be enabled prior to calling xrRaycastANDROID
- session **must** be a valid XrSession handle
- rayInfo **must** be a pointer to a valid XrRaycastInfoANDROID structure
- results **must** be a pointer to an XrRaycastHitResultsANDROID structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FEATURE_UNSUPPORTED
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_POSE_INVALID
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_TIME_INVALID
- XR_ERROR_TRACKABLE_TYPE_NOT_SUPPORTED_ANDROID
- XR_ERROR_VALIDATION_FAILURE

The XrRaycastInfoANDROID structure is defined as:

```
// Provided by XR_ANDROID_raycast
typedef struct XrRaycastInfoANDROID {
    XrStructureType                     type;
    const void*                         next;
    uint32_t                            maxResults;
    uint32_t                            trackerCount;
    const XrTrackableTrackerANDROID*    trackers;
    XrVector3f                          origin;
    XrVector3f                          trajectory;
    XrSpace                             space;
    XrTime                              time;
} XrRaycastInfoANDROID;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `maxResults` is the `uint32_t` maximum number of results to return.
- `trackerCount` is the `uint32_t` count of the `trackers` array.
- `trackers` is the array of XrTrackableTrackerANDROID that the casted ray **should** be tested against.
- `origin` is the XrVector3f that the ray is cast from.
- `trajectory` is the XrVector3f that the ray is targeted at.
- `space` is the XrSpace that the ray is cast in.
- `time` is the `XrTime` the ray is cast at.

The XrRaycastInfoANDROID structure describes the ray to cast.

- The XrRaycastInfoANDROID::`trackers` array **may** contain trackers of different types.
- The XrRaycastInfoANDROID::`trackers` array **must** not contain multiple trackers of the same type, otherwise the runtime **must** return `XR_ERROR_VALIDATION_FAILURE`.

Valid Usage (Implicit)

- The XR_ANDROID_raycast extension **must** be enabled prior to using XrRaycastInfoANDROID
- type **must** be XR_TYPE_RAYCAST_INFO_ANDROID
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- trackers **must** be a pointer to an array of trackerCount valid XrTrackableTrackerANDROID handles
- space **must** be a valid XrSpace handle
- The trackerCount parameter **must** be greater than 0
- Both of space and the elements of trackers **must** have been created, allocated, or retrieved from the same XrSession

The XrRaycastHitResultsANDROID structure is defined as:

```
// Provided by XR_ANDROID_raycast
typedef struct XrRaycastHitResultsANDROID {
    XrStructureType               type;
    void*                         next;
    uint32_t                      resultsCapacityInput;
    uint32_t                      resultsCountOutput;
    XrRaycastHitResultANDROID*    results;
} XrRaycastHitResultsANDROID;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- resultsCapacityInput is the capacity of the results array, or 0 to indicate a request to retrieve the required capacity.
- resultsCountOutput is a pointer to the count of results written, or a pointer to the required capacity in the case that resultsCapacityInput is insufficient.
- results is a pointer to an array of XrRaycastHitResultANDROID structures. It **can** be NULL if resultsCapacityInput is 0.
- See the Buffer Size Parameters section for a detailed description of retrieving the required results size.

The XrRaycastHitResultsANDROID contains the array of hits of a raycast.

The runtime **must** set `resultsCountOutput` to be less than or equal to XrRaycastInfoANDROID ::`maxResults`.

Valid Usage (Implicit)

- The `XR_ANDROID_raycast` extension **must** be enabled prior to using XrRaycastHitResultsANDROID
- `type` **must** be `XR_TYPE_RAYCAST_HIT_RESULTS_ANDROID`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- If `resultsCapacityInput` is not `0`, `results` **must** be a pointer to an array of `resultsCapacityInput` XrRaycastHitResultANDROID-based structures

The XrRaycastHitResultANDROID structure is defined as:

```
// Provided by XR_ANDROID_raycast
typedef struct XrRaycastHitResultANDROID {
    XrTrackableTypeANDROID    type;
    XrTrackableANDROID        trackable;
    XrPosef                   pose;
} XrRaycastHitResultANDROID;
```

Member Descriptions

- `type` is the XrTrackableTypeANDROID of the trackable that the raycast hit.
- `trackable` is the `XrTrackableANDROID` that the raycast hit, or XR_NULL_TRACKABLE_ANDROID if the trackable `type` was `XR_TRACKABLE_TYPE_DEPTH_ANDROID`.
- `pose` is the XrPosef that the raycast hit.

The XrRaycastHitResultANDROID contains the details of a raycast hit.

The XrRaycastHitResultANDROID::`pose` for a plane hit **must** be such that X+ is perpendicular to the cast ray and parallel to the physical surface centered around the hit, Y+ points along the estimated surface normal, and Z+ points roughly toward the ray origin.

The XrRaycastHitResultANDROID::`pose` for a depth hit is analogous to a plane hit, using an estimated surface normal. X+ is perpendicular to the cast ray and parallel to the physical surface centered around the hit, Y+ points along the estimated surface normal, and Z+ points roughly toward the ray origin.

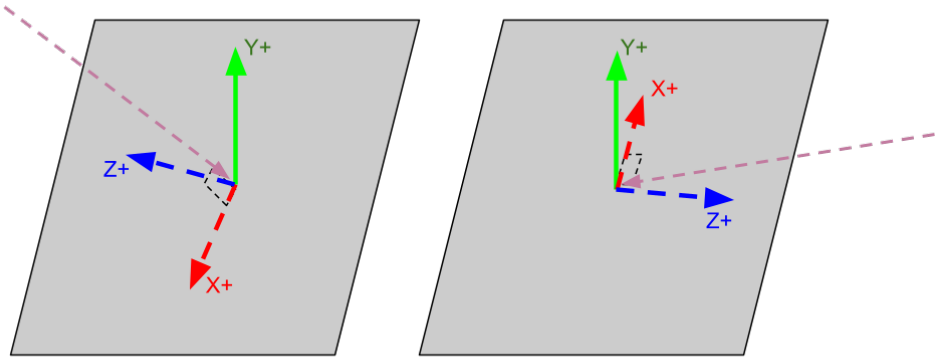


Figure 39. Example raycast hit orientation.

Type of trackable hit	Description
`XR_TRACKABLE_TYPE_PLANE_ANDROID`	Hits horizontal and/or vertical surfaces to determine a point's correct depth and orientation.
`XR_TRACKABLE_TYPE_DEPTH_ANDROID`	Uses depth information from the entire scene to determine a point's correct depth and orientation.

Other extensions **may** implement raycasting for other types of trackables.

Valid Usage (Implicit)

- The `XR_ANDROID_raycast` extension **must** be enabled prior to using XrRaycastHitResultANDROID

14.10.4. Example code for raycasting

The following example code demonstrates how to perform raycasts.

```
XrSession session; // previously initialized
XrTime updateTime; // previously initialized
XrSpace appSpace;  // space created for XR_REFERENCE_SPACE_TYPE_LOCAL.
XrPosef headPose;  // latest pose of the HMD.
XrTrackableTrackerANDROID planeTracker; // tracker for plane trackables.
XrTrackableTrackerANDROID depthTracker; // tracker for depth trackables.

PFN_xrRaycastANDROID xrRaycastANDROID; // previously initialized
XrVector3f CalculateForwardDirectionFromHeadPose(XrPosef); // defined elsewhere

// Perform a raycast against multiple trackers.
constexpr uint32_t NUM_DESIRED_RESULTS = 2;
XrTrackableTrackerANDROID trackers[] = {
  planeTracker,
  depthTracker,
};
XrRaycastInfoANDROID rayInfo = {
  .type = XR_TYPE_RAYCAST_INFO_ANDROID,
  .maxResults = NUM_DESIRED_RESULTS,
  .trackerCount = sizeof(trackers) / sizeof(XrTrackableTrackerANDROID),
  .trackers = trackers,
  .origin = headPose.position,
  .trajectory = CalculateForwardDirectionFromHeadPose(headPose),
  .space = appSpace,
  .time = updateTime,
};

uint32_t totalHitResults = 0;
XrRaycastHitResultANDROID hitResult[NUM_DESIRED_RESULTS]{};
XrRaycastHitResultsANDROID hitResults = {
  .type = XR_TYPE_RAYCAST_HIT_RESULTS_ANDROID,
  .resultsCapacityInput = NUM_DESIRED_RESULTS,
  .results = hitResult,
};
XrResult result = xrRaycastANDROID(session, &rayInfo, &hitResults);

if (result == XR_SUCCESS && hitResults.resultsCountOutput >= 1) {
  // Hit results are returned in closest-to-farthest order in
  // hitResults.results[0] .. hitResults.results[hitResults.resultsCountOutput - 1]
}
```

14.10.5. New Commands

- xrEnumerateRaycastSupportedTrackableTypesANDROID
- xrRaycastANDROID

14.10.6. New Structures

- XrRaycastHitResultANDROID
- XrRaycastHitResultsANDROID
- XrRaycastInfoANDROID

14.10.7. New Enum Constants

- `XR_ANDROID_RAYCAST_EXTENSION_NAME`
- `XR_ANDROID_raycast_SPEC_VERSION`
- Extending XrStructureType:
  - `XR_TYPE_RAYCAST_HIT_RESULTS_ANDROID`
  - `XR_TYPE_RAYCAST_INFO_ANDROID`
- Extending XrTrackableTypeANDROID:
  - `XR_TRACKABLE_TYPE_DEPTH_ANDROID`

14.10.8. Issues

14.10.9. Version History

- Revision 1, 2025-07-23 (Kenny Vercaemer)
  - Initial extension description

14.11. XR_ANDROID_trackables

Name String

`XR_ANDROID_trackables`

Extension Type

Instance extension

Registered Extension Number

456

Revision

2

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Deprecation State

- *Deprecated* by `XR_EXT_spatial_entity`, `XR_EXT_spatial_plane_tracking`, `XR_ANDROID_spatial_component_subsumed_by`, `XR_EXT_spatial_anchor`, `XR_ANDROID_spatial_entity_bound_anchor`, `XR_ANDROID_spatial_anchor_space` extensions

Last Modified Date

2025-07-21

IP Status

No known IP claims.

Contributors

Spencer Quin, Google
Nihav Jain, Google
John Pursey, Google
Jared Finder, Google
Levana Chen, Google
Kenny Vercaemer, Google

14.11.1. Overview

This extension allows the application to access trackables from the physical environment, and create anchors attached to a trackable. It also allows applications to create world-locked spatial anchors.

A trackable is something that is tracked in the physical environment:

- a plane (e.g. wall, floor, ceiling, table)
- an object (e.g. keyboard, mouse, laptop)

See also XrTrackableTypeANDROID.

This extension defines plane trackables.

Additional trackable types are added by other extensions. For example, `XR_ANDROID_trackables_object` adds object trackables, and `XR_ANDROID_raycast` adds depth trackables that allow raycasting to arbitrary points in the environment.



Permissions

Android applications **must** have the android.permission.SCENE_UNDERSTANDING_COARSE permission listed in their manifest as this extension exposes the geometry of the environment. The android.permission.SCENE_UNDERSTANDING_COARSE permission is considered a dangerous permission. The application **must** request the permission at runtime to use these functions:

- xrCreateTrackableTrackerANDROID

(protection level: dangerous)

14.11.2. Deprecation note

The `XR_ANDROID_trackables` extension is now deprecated. Unlike typical OpenXR deprecations, its features are not replaced by a single successor. Instead, its functionality has been distributed across several different extensions.

Trackable foundations are replaced by `XR_EXT_spatial_entity`:

`XR_ANDROID_trackables` **functionality**	**Now available in**
XrTrackableTypeANDROID	XrSpatialCapabilityEXT
xrEnumerateSupportedTrackableTypesANDROID	xrEnumerateSpatialCapabilitiesEXT
XrTrackableTrackerANDROID	XrSpatialContextEXT
xrCreateTrackableTrackerANDROID	xrCreateSpatialContextAsyncEXT & xrCreateSpatialContextCompleteEXT
xrDestroyTrackableTrackerANDROID	xrDestroySpatialContextEXT
`XrTrackableANDROID`	`XrSpatialEntityIdEXT`
XrTrackingStateANDROID	XrSpatialEntityTrackingStateEXT
xrGetAllTrackablesANDROID	xrCreateSpatialDiscoverySnapshotAsyncEXT, xrCreateSpatialDiscoverySnapshotCompleteEXT, & xrQuerySpatialComponentDataEXT (XrSpatialComponentDataQueryResultEXT::`entityIds`)

Plane trackables are replaced by `XR_EXT_spatial_plane_tracking` and `XR_ANDROID_spatial_component_subsumed_by`:

`XR_ANDROID_trackables` **functionality**	**Now available in**
XrPlaneTypeANDROID	XrSpatialPlaneAlignmentEXT
XrPlaneLabelANDROID	XrSpatialPlaneSemanticLabelEXT

XR_ANDROID_trackables functionality	Now available in
xrGetTrackablePlaneANDROID	xrQuerySpatialComponentDataEXT
XrTrackablePlaneANDROID::centerPose & XrTrackablePlaneANDROID::extents	Provided by Bounded 2D component
XrTrackablePlaneANDROID::vertices	Provided by Polygon 2D Component
XrTrackablePlaneANDROID::subsumedByPlane	Provided by Subsumed By Component

Anchors are replaced by XR_EXT_spatial_anchor, XR_ANDROID_spatial_entity_bound_anchor, and XR_ANDROID_spatial_anchor_space:

XR_ANDROID_trackables functionality	Now available in
xrEnumerateSupportedAnchorTrackableTypesANDROID	xrEnumerateSpatialAnchorAttachableComponentsANDROID
xrCreateAnchorSpaceANDROID	xrCreateSpatialAnchorEXT or xrCreateSpatialAnchorSpaceANDROID
XrAnchorSpaceCreateInfoANDROID::trackable	XrSpatialAnchorParentANDROID::parentId

14.11.3. Inspect system capability

The XrSystemTrackablesPropertiesANDROID structure is defined as:

```
// Provided by XR_ANDROID_trackables
typedef struct XrSystemTrackablesPropertiesANDROID {
    XrStructureType    type;
    const void*        next;
    XrBool32           supportsAnchor;
    uint32_t           maxAnchors;
} XrSystemTrackablesPropertiesANDROID;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension. For more details about the structure chain, see the structure being extended (XrSystemProperties).
- supportsAnchor is an XrBool32 indicating if current system supports anchor creation via xrCreateAnchorSpaceANDROID for spatial anchors.
- maxAnchors is the total maximum number of anchors supported by the runtime to be created via xrCreateAnchorSpaceANDROID of all types combined at any given time.

To inspect whether the system is capable of creating spatial anchors an application calls xrCreateAnchorSpaceANDROID by extending the XrSystemProperties with XrSystemTrackablesPropertiesANDROID structure when calling xrGetSystemProperties. The runtime **must** return XR_ERROR_FEATURE_UNSUPPORTED for spatial anchor creation if and only if supportsAnchor is false.

If a runtime supports anchors, it **must** support maxAnchors active anchors at any given time.

To determine which trackable types are supported by the runtime for tracking and for anchor creation respectively an application calls xrEnumerateSupportedTrackableTypesANDROID and xrEnumerateSupportedAnchorTrackableTypesANDROID.

Valid Usage (Implicit)

- The XR_ANDROID_trackables extension **must** be enabled prior to using XrSystemTrackablesPropertiesANDROID
- type **must** be XR_TYPE_SYSTEM_TRACKABLES_PROPERTIES_ANDROID
- next **must** be NULL or a valid pointer to the next structure in a structure chain

The xrEnumerateSupportedTrackableTypesANDROID function is defined as:

```
// Provided by XR_ANDROID_trackables
XrResult xrEnumerateSupportedTrackableTypesANDROID(
    XrInstance                                  instance,
    XrSystemId                                  systemId,
    uint32_t                                    trackableTypeCapacityInput,
    uint32_t*                                   trackableTypeCountOutput,
    XrTrackableTypeANDROID*                     trackableTypes);
```

Parameter Descriptions

- `instance` is the XrInstance that enumerates the supported trackable types.
- `systemId` is the `XrSystemId` of the system to query.
- `trackableTypeCapacityInput` is the capacity of the `trackableTypes`, or 0 to retrieve the required capacity.
- `trackableTypeCountOutput` is a pointer to the count of `uint64_t trackableTypes` written, or a pointer to the required capacity in the case that `trackableTypeCapacityInput` is insufficient.
- `trackableTypes` is a pointer to an array of XrTrackableTypeANDROID, but **can** be `NULL` if `trackableTypeCapacityInput` is `0`.
- See the Buffer Size Parameters section for a detailed description of retrieving the required `trackableTypes` size.

xrEnumerateSupportedTrackableTypesANDROID enumerates the trackable types supported by the runtime for xrCreateTrackableTrackerANDROID.

If and only if a trackable type X is not returned by xrEnumerateSupportedTrackableTypesANDROID, then the runtime **must** return `XR_ERROR_FEATURE_UNSUPPORTED` when calling xrCreateTrackableTrackerANDROID with X as the trackable type.

Valid Usage (Implicit)

- The `XR_ANDROID_trackables` extension **must** be enabled prior to calling xrEnumerateSupportedTrackableTypesANDROID
- `instance` **must** be a valid XrInstance handle
- `trackableTypeCountOutput` **must** be a pointer to a `uint32_t` value
- If `trackableTypeCapacityInput` is not `0`, `trackableTypes` **must** be a pointer to an array of `trackableTypeCapacityInput` XrTrackableTypeANDROID values

Return Codes

Success

- XR_SUCCESS

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SIZE_INSUFFICIENT
- XR_ERROR_SYSTEM_INVALID
- XR_ERROR_VALIDATION_FAILURE

The xrEnumerateSupportedAnchorTrackableTypesANDROID function is defined as:

```
// Provided by XR_ANDROID_trackables
XrResult xrEnumerateSupportedAnchorTrackableTypesANDROID(
    XrInstance                                  instance,
    XrSystemId                                  systemId,
    uint32_t                                    trackableTypeCapacityInput,
    uint32_t*                                   trackableTypeCountOutput,
    XrTrackableTypeANDROID*                     trackableTypes);
```

Parameter Descriptions

- instance is the XrInstance that enumerates the supported anchor trackable types.
- systemId is the XrSystemId of the system to query.
- trackableTypeCapacityInput is the capacity of the trackableTypes, or 0 to retrieve the required capacity.
- trackableTypeCountOutput is a pointer to the count of uint64_t trackableTypes written, or a pointer to the required capacity in the case that trackableTypeCapacityInput is insufficient.
- trackableTypes is a pointer to an array of XrTrackableTypeANDROID, but **can** be NULL if trackableTypeCapacityInput is 0.
- See the Buffer Size Parameters section for a detailed description of retrieving the required trackableTypes size.

xrEnumerateSupportedAnchorTrackableTypesANDROID enumerates the trackable types supported by the runtime for anchor creation.

If and only if a trackable type X is not returned by xrEnumerateSupportedAnchorTrackableTypesANDROID, then the runtime **must** return XR_ERROR_FEATURE_UNSUPPORTED when calling xrCreateAnchorSpaceANDROID with X as the trackable type.

Valid Usage (Implicit)

- The XR_ANDROID_trackables extension **must** be enabled prior to calling xrEnumerateSupportedAnchorTrackableTypesANDROID
- `instance` **must** be a valid XrInstance handle
- `trackableTypeCountOutput` **must** be a pointer to a `uint32_t` value
- If `trackableTypeCapacityInput` is not `0`, `trackableTypes` **must** be a pointer to an array of `trackableTypeCapacityInput` XrTrackableTypeANDROID values

Return Codes

Success

- XR_SUCCESS

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SIZE_INSUFFICIENT
- XR_ERROR_SYSTEM_INVALID
- XR_ERROR_VALIDATION_FAILURE

14.11.4. Creating a trackable tracker

An XrTrackableTrackerANDROID is a handle that represents the resources required to discover and update trackables of a given XrTrackableTypeANDROID in the environment.

```
// Provided by XR_ANDROID_trackables
XR_DEFINE_HANDLE(XrTrackableTrackerANDROID)
```

The xrCreateTrackableTrackerANDROID function is defined as:

```
// Provided by XR_ANDROID_trackables
XrResult xrCreateTrackableTrackerANDROID(
    XrSession                                   session,
    const XrTrackableTrackerCreateInfoANDROID*  createInfo,
    XrTrackableTrackerANDROID*                  trackableTracker);
```

Parameter Descriptions

- `session` is the XrSession that creates the tracker.
- `createInfo` is a pointer to an XrTrackableTrackerCreateInfoANDROID structure containing parameters to be used to create the tracker.
- `trackableTracker` is a pointer to a handle in which the created XrTrackableTrackerANDROID is returned.

The application creates trackable trackers with xrCreateTrackableTrackerANDROID.

- The runtime **must** return `XR_ERROR_FEATURE_UNSUPPORTED` if the system does not support trackables of the specified XrTrackableTrackerCreateInfoANDROID::`trackableType` as returned by xrEnumerateSupportedTrackableTypesANDROID.
- The runtime **must** return `XR_ERROR_PERMISSION_INSUFFICIENT` if the required permissions have not been granted to the calling app.

The XrTrackableTrackerANDROID handle **must** be eventually freed via the xrDestroyTrackableTrackerANDROID function or by destroying the parent XrSession handle.

The runtime **may** use the creation of an XrTrackableTrackerANDROID to prepare itself for discovering trackables of the selected XrTrackableTrackerCreateInfoANDROID::`trackableType`. For example, the runtime **may** only begin its plane tracking system when a trackable tracker handle for `XR_TRACKABLE_TYPE_PLANE_ANDROID` is created by the application.

Valid Usage (Implicit)

- The `XR_ANDROID_trackables` extension **must** be enabled prior to calling xrCreateTrackableTrackerANDROID
- `session` **must** be a valid XrSession handle
- `createInfo` **must** be a pointer to a valid XrTrackableTrackerCreateInfoANDROID structure
- `trackableTracker` **must** be a pointer to an XrTrackableTrackerANDROID handle

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FEATURE_UNSUPPORTED
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The XrTrackableTrackerCreateInfoANDROID structure is defined as:

```
// Provided by XR_ANDROID_trackables
typedef struct XrTrackableTrackerCreateInfoANDROID {
    XrStructureType           type;
    const void*               next;
    XrTrackableTypeANDROID    trackableType;
} XrTrackableTrackerCreateInfoANDROID;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- trackableType is the XrTrackableTypeANDROID that the tracker will track.

The XrTrackableTrackerCreateInfoANDROID structure is passed to xrCreateTrackableTrackerANDROID and provides creation options for the XrTrackableTrackerANDROID.

Extensions **may** define structures to be provided in the slinkXrTrackableTrackerCreateInfoANDROID::next chain to allow additional configuration for the

trackable trackers.

Valid Usage (Implicit)

- The XR_ANDROID_trackables extension **must** be enabled prior to using XrTrackableTrackerCreateInfoANDROID
- type **must** be XR_TYPE_TRACKABLE_TRACKER_CREATE_INFO_ANDROID
- next **must** be NULL or a valid pointer to the next structure in a structure chain. See also: XrTrackableMarkerConfigurationANDROID, XrTrackableObjectConfigurationANDROID, XrTrackableQrCodeConfigurationANDROID
- trackableType **must** be a valid XrTrackableTypeANDROID value

The XrTrackableTypeANDROID enum is defined as:

```
// Provided by XR_ANDROID_trackables
typedef enum XrTrackableTypeANDROID {
    XR_TRACKABLE_TYPE_NOT_VALID_ANDROID = 0,
    XR_TRACKABLE_TYPE_PLANE_ANDROID = 1,
  // Provided by XR_ANDROID_raycast
    XR_TRACKABLE_TYPE_DEPTH_ANDROID = 1000463000,
  // Provided by XR_ANDROID_trackables_object
    XR_TRACKABLE_TYPE_OBJECT_ANDROID = 1000466000,
  // Provided by XR_ANDROID_trackables_marker
    XR_TRACKABLE_TYPE_MARKER_ANDROID = 1000707000,
  // Provided by XR_ANDROID_trackables_qr_code
    XR_TRACKABLE_TYPE_QR_CODE_ANDROID = 1000708000,
  // Provided by XR_ANDROID_trackables_image
    XR_TRACKABLE_TYPE_IMAGE_ANDROID = 1000709000,
    XR_TRACKABLE_TYPE_MAX_ENUM_ANDROID = 0x7FFFFFFF
} XrTrackableTypeANDROID;
```

Enum	Description
XR_TRACKABLE_TYPE_NOT_VALID_ANDROID	Indicates that the trackable is not valid.
XR_TRACKABLE_TYPE_PLANE_ANDROID	Indicates that the trackable is a plane.
XR_TRACKABLE_TYPE_DEPTH_ANDROID	Indicates that the trackable is the perception depth buffer. (Added by the XR_ANDROID_raycast extension)
XR_TRACKABLE_TYPE_OBJECT_ANDROID	Indicates that the trackable is the object. (Added by the XR_ANDROID_trackables_object extension)

Enum	Description
XR_TRACKABLE_TYPE_MARKER_ANDROID	Indicates that the trackable is a marker. (Added by the XR_ANDROID_trackables_marker extension)
XR_TRACKABLE_TYPE_QR_CODE_ANDROID	Indicates that the trackable is a QR code. (Added by the XR_ANDROID_trackables_qr_code extension)
XR_TRACKABLE_TYPE_IMAGE_ANDROID	Indicates that the trackable is an image. (Added by the XR_ANDROID_trackables_image extension)

The xrDestroyTrackableTrackerANDROID function is defined as:

```
// Provided by XR_ANDROID_trackables
XrResult xrDestroyTrackableTrackerANDROID(
    XrTrackableTrackerANDROID                   trackableTracker);
```

Parameter Descriptions

- trackableTracker is an XrTrackableTrackerANDROID handle previously created by xrCreateTrackableTrackerANDROID.

The xrDestroyTrackableTrackerANDROID function destroys the trackable tracker. After this call the runtime **may** free all related memory and resources.

If there is no other valid XrTrackableTrackerANDROID that was created with the same XrTrackableTypeANDROID, the runtime **may** disable the tracking services required for that trackable type to save system resources.

Valid Usage (Implicit)

- The XR_ANDROID_trackables extension **must** be enabled prior to calling xrDestroyTrackableTrackerANDROID
- trackableTracker **must** be a valid XrTrackableTrackerANDROID handle

Thread Safety

- Access to trackableTracker, and any child handles, **must** be externally synchronized

Return Codes

Success

- XR_SUCCESS

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_RUNTIME_FAILURE

14.11.5. Get all trackables

The XrTrackableANDROID atom is defined as:

```
// Provided by XR_ANDROID_trackables
XR_DEFINE_ATOM(XrTrackableANDROID)
```

XrTrackableANDROID is used to represent a single trackable and is valid only within the lifecycle of its associated XrTrackableTrackerANDROID. The runtime **must** not reuse the same XrTrackableANDROID for different trackables within the same XrTrackableTrackerANDROID.

The xrGetAllTrackablesANDROID function is defined as:

```
// Provided by XR_ANDROID_trackables
XrResult xrGetAllTrackablesANDROID(
    XrTrackableTrackerANDROID                   trackableTracker,
    uint32_t                                    trackableCapacityInput,
    uint32_t*                                   trackableCountOutput,
    XrTrackableANDROID*                         trackables);
```

Parameter Descriptions

- `trackableTracker` is the XrTrackableTrackerANDROID to query.
- `trackableCapacityInput` is the capacity of the `trackables` array, or 0 to indicate a request to retrieve the required capacity.
- `trackableCountOutput` is a pointer to the count of `trackables` written, or a pointer to the required capacity in the case that `trackables` is insufficient.
- `trackables` is a pointer to an array of `XrTrackableANDROID`. It **can** be `NULL` if `trackableCapacityInput` is 0.
- See the Buffer Size Parameters section for a detailed description of retrieving the required `trackables` size.

xrGetAllTrackablesANDROID fills an array of `XrTrackableANDROID` representing the trackables found in the environment. The XrTrackableTypeANDROID of the returned `trackables` **must** match the XrTrackableTypeANDROID of the `trackableTracker`.

14.11.6. Get trackable plane

The xrGetTrackablePlaneANDROID function is defined as:

```
// Provided by XR_ANDROID_trackables
XrResult xrGetTrackablePlaneANDROID(
    XrTrackableTrackerANDROID                   trackableTracker,
    const XrTrackableGetInfoANDROID*            getInfo,
    XrTrackablePlaneANDROID*                    planeOutput);
```

Parameter Descriptions

- `trackableTracker` is the XrTrackableTrackerANDROID to query.
- `getInfo` is the XrTrackableGetInfoANDROID with the information used to get the trackable plane.
- `planeOutput` is a pointer to the XrTrackablePlaneANDROID structure in which the trackable plane is returned.

The xrGetTrackablePlaneANDROID function returns details about the trackable plane — such as its geometry, orientation, and tracking state — at a given point in time.

The plane information returned is a best effort approximation of its state at the given XrTrackableGetInfoANDROID::`time`, relative to the XrTrackableGetInfoANDROID::`baseSpace`.

The runtime **must** return XR_ERROR_MISMATCHING_TRACKABLE_TYPE_ANDROID if the trackable type of the XrTrackableGetInfoANDROID::trackable is not XR_TRACKABLE_TYPE_PLANE_ANDROID.

Valid Usage (Implicit)

- The XR_ANDROID_trackables extension **must** be enabled prior to calling xrGetTrackablePlaneANDROID
- trackableTracker **must** be a valid XrTrackableTrackerANDROID handle
- getInfo **must** be a pointer to a valid XrTrackableGetInfoANDROID structure
- planeOutput **must** be a pointer to an XrTrackablePlaneANDROID structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_MISMATCHING_TRACKABLE_TYPE_ANDROID
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_TIME_INVALID
- XR_ERROR_VALIDATION_FAILURE

The XrTrackableGetInfoANDROID structure is defined as:

```
// Provided by XR_ANDROID_trackables
typedef struct XrTrackableGetInfoANDROID {
    XrStructureType       type;
    const void*           next;
    XrTrackableANDROID    trackable;
    XrSpace               baseSpace;
    XrTime                time;
} XrTrackableGetInfoANDROID;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `trackable` is the `XrTrackableANDROID` plane to query.
- `baseSpace` the plane pose will be relative to this XrSpace at `time`.
- `time` is the `XrTime` at which to evaluate the coordinates relative to the `baseSpace`.

The XrTrackableGetInfoANDROID structure provides query options when passed to xrGetTrackablePlaneANDROID. The `trackable` **must** correspond to the `trackable` used in xrGetTrackablePlaneANDROID.

Valid Usage (Implicit)

- The `XR_ANDROID_trackables` extension **must** be enabled prior to using XrTrackableGetInfoANDROID
- `type` **must** be `XR_TYPE_TRACKABLE_GET_INFO_ANDROID`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `baseSpace` **must** be a valid XrSpace handle

The XrTrackablePlaneANDROID structure is defined as:

```
// Provided by XR_ANDROID_trackables
typedef struct XrTrackablePlaneANDROID {
    XrStructureType           type;
    void*                     next;
    XrTrackingStateANDROID    trackingState;
    XrPosef                   centerPose;
    XrExtent2Df               extents;
    XrPlaneTypeANDROID        planeType;
    XrPlaneLabelANDROID       planeLabel;
    XrTrackableANDROID        subsumedByPlane;
    XrTime                    lastUpdatedTime;
    uint32_t                  vertexCapacityInput;
    uint32_t*                 vertexCountOutput;
    XrVector2f*               vertices;
} XrTrackablePlaneANDROID;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `trackingState` is the XrTrackingStateANDROID of the plane.
- `centerPose` is an XrPosef defining the position and orientation of the plane within the reference frame of the corresponding XrTrackableGetInfoANDROID::`baseSpace`. An identity orientation here represents a set of axes with +Y parallel to the plane's normal.
- `extents` is the XrExtent2Df dimension of the plane.
- `planeType` is the XrPlaneTypeANDROID that the runtime has determined for this plane.
- `planeLabel` is the XrPlaneLabelANDROID that the runtime has determined for this plane.
- `subsumedByPlane` is the `XrTrackableANDROID` of the plane that subsumes this plane (XR_NULL_TRACKABLE_ANDROID if none exists).
- `lastUpdatedTime` is the `XrTime` of the last update of the plane, meaning any of its properties **may** be different, e.g. `planeLabel`.
- `vertexCapacityInput` is the capacity of the `vertices` array, or 0 to indicate a request to retrieve the required capacity.
- `vertexCountOutput` is the count of `vertices` written, or the required capacity in the case that `vertices` is insufficient.
- `vertices` is a pointer to an array of XrVector2f. It **can** be `NULL` if `vertexCapacityInput` is 0. The vertices are in counter-clockwise order. The polygon **may** be concave and **must** not be self-intersecting.
- See the Buffer Size Parameters section for a detailed description of retrieving the required `vertices` size. However, it is important to note that unlike the "structure form" example given in that section, `vertexCountOutput` is a pointer instead of a value type.

When the runtime has acquired enough environment information to detect that 2 tracked planes are actually the same plane, it **must** set the XrTrackablePlaneANDROID::`subsumedByPlane` of one of the planes to the handle of the other. When this happens, the plane information returned in both of the associated planes **must** be identical. The application **should** stop querying for information about planes that have been reported as subsumed.

Warning

Unlike the typical structure forms used for Buffer Size Parameters, XrTrackablePlaneANDROID ::`vertexCountOutput` is a pointer and not a value type.

Valid Usage (Implicit)

- The XR_ANDROID_trackables extension **must** be enabled prior to using XrTrackablePlaneANDROID
- type **must** be XR_TYPE_TRACKABLE_PLANE_ANDROID
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- trackingState **must** be a valid XrTrackingStateANDROID value
- planeType **must** be a valid XrPlaneTypeANDROID value
- planeLabel **must** be a valid XrPlaneLabelANDROID value
- If vertexCountOutput is not NULL, vertexCountOutput **must** be a pointer to a uint32_t value
- If vertexCapacityInput is not 0, vertices **must** be a pointer to an array of vertexCapacityInput XrVector2f structures

The XrTrackingStateANDROID enum describes the tracking state of an XrTrackableANDROID.

```
// Provided by XR_ANDROID_trackables
typedef enum XrTrackingStateANDROID {
    XR_TRACKING_STATE_PAUSED_ANDROID = 0,
    XR_TRACKING_STATE_STOPPED_ANDROID = 1,
    XR_TRACKING_STATE_TRACKING_ANDROID = 2,
    XR_TRACKING_STATE_MAX_ENUM_ANDROID = 0x7FFFFFFF
} XrTrackingStateANDROID;
```

Enum	Description
XR_TRACKING_STATE_PAUSED_ANDROID	Indicates that the trackable or anchor tracking is paused but may be resumed in the future.
XR_TRACKING_STATE_STOPPED_ANDROID	Tracking has stopped on this Trackable and will never be resumed.
XR_TRACKING_STATE_TRACKING_ANDROID	The object is currently tracked and its pose is current.

The XrPlaneTypeANDROID enum is the type of an XrTrackableANDROID plane.

```
// Provided by XR_ANDROID_trackables
typedef enum XrPlaneTypeANDROID {
    XR_PLANE_TYPE_HORIZONTAL_DOWNWARD_FACING_ANDROID = 0,
    XR_PLANE_TYPE_HORIZONTAL_UPWARD_FACING_ANDROID = 1,
    XR_PLANE_TYPE_VERTICAL_ANDROID = 2,
    XR_PLANE_TYPE_ARBITRARY_ANDROID = 3,
    XR_PLANE_TYPE_MAX_ENUM_ANDROID = 0x7FFFFFFF
} XrPlaneTypeANDROID;
```

Enum	Description
XR_PLANE_TYPE_HORIZONTAL_DOWNWARD_FACING_ANDROID	A horizontal plane facing downward (for example a ceiling).
XR_PLANE_TYPE_HORIZONTAL_UPWARD_FACING_ANDROID	A horizontal plane facing upward (for example a floor or tabletop).
XR_PLANE_TYPE_VERTICAL_ANDROID	A vertical plane (for example a wall).
XR_PLANE_TYPE_ARBITRARY_ANDROID	A plane with an arbitrary orientation.

The XrPlaneLabelANDROID enum is a label for an XrTrackableANDROID plane.

```
// Provided by XR_ANDROID_trackables
typedef enum XrPlaneLabelANDROID {
    XR_PLANE_LABEL_UNKNOWN_ANDROID = 0,
    XR_PLANE_LABEL_WALL_ANDROID = 1,
    XR_PLANE_LABEL_FLOOR_ANDROID = 2,
    XR_PLANE_LABEL_CEILING_ANDROID = 3,
    XR_PLANE_LABEL_TABLE_ANDROID = 4,
    XR_PLANE_LABEL_MAX_ENUM_ANDROID = 0x7FFFFFFF
} XrPlaneLabelANDROID;
```

Enum	Description
XR_PLANE_LABEL_UNKNOWN_ANDROID	It was not possible to label the plane
XR_PLANE_LABEL_WALL_ANDROID	The plane is a wall.
XR_PLANE_LABEL_FLOOR_ANDROID	The plane is a floor.
XR_PLANE_LABEL_CEILING_ANDROID	The plane is a ceiling.
XR_PLANE_LABEL_TABLE_ANDROID	The plane is a table.

14.11.7. Create anchor space

The xrCreateAnchorSpaceANDROID function is defined as:

```
// Provided by XR_ANDROID_trackables
XrResult xrCreateAnchorSpaceANDROID(
    XrSession                                   session,
    const XrAnchorSpaceCreateInfoANDROID*       createInfo,
    XrSpace*                                    anchorOutput);
```

Parameter Descriptions

- `session` is the XrSession that creates the anchor space.
- `createInfo` is a pointer to an XrAnchorSpaceCreateInfoANDROID structure containing parameters to be used to create the anchor space.
- `anchorOutput` is a pointer to a handle in which the created XrSpace is returned.

At any point in time both the position and direction of the anchor is tracked or untracked together. This means that the runtime **must** either set both `XR_SPACE_LOCATION_POSITION_TRACKED_BIT` and `XR_SPACE_LOCATION_ORIENTATION_TRACKED_BIT` or clear both `XR_SPACE_LOCATION_POSITION_TRACKED_BIT` and `XR_SPACE_LOCATION_ORIENTATION_TRACKED_BIT` when the application calls xrLocateSpace or xrLocateSpaces for `anchorOutput`.

The application **must** eventually free the returned XrSpace via xrDestroySpace.

- The runtime **must** return `XR_ERROR_FEATURE_UNSUPPORTED` if the runtime does not support anchors.
- The runtime **must** return `XR_ERROR_TRACKABLE_TYPE_NOT_SUPPORTED_ANDROID` if the specific anchor attachment is not supported for the type of the passed XrAnchorSpaceCreateInfoANDROID ::`trackable` as returned by xrEnumerateSupportedAnchorTrackableTypesANDROID.

Valid Usage (Implicit)

- The `XR_ANDROID_trackables` extension **must** be enabled prior to calling xrCreateAnchorSpaceANDROID
- `session` **must** be a valid XrSession handle
- `createInfo` **must** be a pointer to a valid XrAnchorSpaceCreateInfoANDROID structure
- `anchorOutput` **must** be a pointer to an XrSpace handle

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_POSE_INVALID
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_TIME_INVALID
- XR_ERROR_TRACKABLE_TYPE_NOT_SUPPORTED_ANDROID
- XR_ERROR_VALIDATION_FAILURE

The XrAnchorSpaceCreateInfoANDROID structure is defined as:

```
// Provided by XR_ANDROID_trackables
typedef struct XrAnchorSpaceCreateInfoANDROID {
    XrStructureType       type;
    const void*           next;
    XrSpace               space;
    XrTime                time;
    XrPosef               pose;
    XrTrackableANDROID    trackable;
} XrAnchorSpaceCreateInfoANDROID;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `space` is the XrSpace on which the anchor will be created.
- `time` is the `XrTime` at which `pose` is evaluated in `space`.
- `pose` is the XrPosef of the anchor.
- `trackable` is the `XrTrackableANDROID` on which the anchor will be attached. It **may** be XR_NULL_TRACKABLE_ANDROID to create a spatial anchor.

Valid Usage (Implicit)

- The `XR_ANDROID_trackables` extension **must** be enabled prior to using XrAnchorSpaceCreateInfoANDROID
- `type` **must** be `XR_TYPE_ANCHOR_SPACE_CREATE_INFO_ANDROID`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `space` **must** be a valid XrSpace handle

Example code for getting all trackables

The following example code demonstrates how to get all trackables of a given type.

```
XrSession session; // previously initialized

// The function pointers are previously initialized using xrGetInstanceProcAddr.
PFN_xrCreateTrackableTrackerANDROID xrCreateTrackableTrackerANDROID; // previously
initialized
PFN_xrGetAllTrackablesANDROID xrGetAllTrackablesANDROID; // previously initialized
PFN_xrDestroyTrackableTrackerANDROID xrDestroyTrackableTrackerANDROID; // previously
initialized

XrTrackableTrackerCreateInfoANDROID createInfo{
  .type = XR_TYPE_TRACKABLE_TRACKER_CREATE_INFO_ANDROID,
  .trackableType = XR_TRACKABLE_TYPE_PLANE_ANDROID,
};
XrTrackableTrackerANDROID planeTrackableTracker {XR_NULL_HANDLE};
XrResult result = xrCreateTrackableTrackerANDROID(
  session,
  &createInfo,
```

```
  &planeTrackableTracker);
if (result != XR_SUCCESS) { /* Handle failures. */ }

uint32_t trackableCountOutput = 0;
std::vector<XrTrackableANDROID> allPlaneTrackables;

// Query the number of trackables available.
result = xrGetAllTrackablesANDROID(
  planeTrackableTracker,
  0,
  &trackableCountOutput,
  nullptr
);

if (result == XR_SUCCESS) {
  allPlaneTrackables.resize(trackableCountOutput, XR_NULL_TRACKABLE_ANDROID);

  // Fetch the actual trackable handles in the appropriately resized array.
  result = xrGetAllTrackablesANDROID(
    planeTrackableTracker,
    trackableCountOutput,
    &trackableCountOutput,
    allPlaneTrackables.data());

  if (result == XR_SUCCESS) {
    for (XrTrackableANDROID trackable : allPlaneTrackables) {
      // You now have all trackables of the specified type.
    }
  }
}

// Release trackable tracker.
result = xrDestroyTrackableTrackerANDROID(planeTrackableTracker);
```

Example code for getting trackable plane

The following example code demonstrates how to get a trackable plane from an existing XrTrackableANDROID, obtained from a hit result (XR_ANDROID_raycast) or xrGetAllTrackablesANDROID.

```
XrTrackableTrackerANDROID planeTracker; // previously created

// The function pointers are previously initialized using xrGetInstanceProcAddr.
PFN_xrGetTrackablePlaneANDROID xrGetTrackablePlaneANDROID; // previously initialized

XrTime updateTime; // Time used for the current frame's simulation update.
XrSpace appSpace; // Space created for XR_REFERENCE_SPACE_TYPE_LOCAL.
XrTrackableANDROID planeTrackable; // Acquired from a hit result or
xrGetAllTrackablesANDROID().

XrTrackableGetInfoANDROID planeGetInfo {
  .type = XR_TYPE_TRACKABLE_GET_INFO_ANDROID,
  .next = nullptr,
  .trackable = planeTrackable,
  .baseSpace = appSpace,
  .time = updateTime,
};

XrTrackablePlaneANDROID plane {
  .type = XR_TYPE_TRACKABLE_PLANE_ANDROID,
};

XrResult result = xrGetTrackablePlaneANDROID(
  planeTracker,
  &planeGetInfo,
  &plane
);

if (result == XR_SUCCESS) {
  // Plane tracking state, center pose, extents, type now available in plane.
}
```

Example code for creating anchor space

The following example code demonstrates how to create an anchor space attached to a trackable.

```
XrSession session; // Created at app startup.
XrTime updateTime; // Time used for the current frame's simulation update.
XrSpace appSpace; // Space created for XR_REFERENCE_SPACE_TYPE_LOCAL.
XrTrackableANDROID planeTrackable; // Acquired from a hit result or
xrGetAllTrackablesANDROID().

PFN_xrCreateAnchorSpaceANDROID xrCreateAnchorSpaceANDROID; // Previously initialized.

// Create an anchor at (2, 2, 2) world-coordinates.
XrAnchorSpaceCreateInfoANDROID spatialAnchorCreateInfo {
```

```
  .type = XR_TYPE_ANCHOR_SPACE_CREATE_INFO_ANDROID,
  .next = nullptr,
  .space = appSpace,
  .time = updateTime,
  .pose = { { 0, 0, 0, 1 }, { 2, 2, 2 } },
};

XrSpace spatialAnchor {XR_NULL_HANDLE};
XrResult result = xrCreateAnchorSpaceANDROID(
  session,
  &spatialAnchorCreateInfo,
  &spatialAnchor
);

// Create an anchor attached to a trackable.
XrTrackablePlaneANDROID plane;
XrAnchorSpaceCreateInfoANDROID trackableAnchorCreateInfo {
  .type = XR_TYPE_ANCHOR_SPACE_CREATE_INFO_ANDROID,
  .next = nullptr,
  .space = appSpace,
  .pose = plane.centerPose,
  .trackable = planeTrackable,
};

XrSpace trackableAnchor = XR_NULL_HANDLE;
result = xrCreateAnchorSpaceANDROID(
  session,
  &trackableAnchorCreateInfo,
  &trackableAnchor
);


while (true) {
  // app update loop
  // ...

  // Get the current location of the anchor's space w.r.t the world.
  XrSpaceLocation anchorLocation {
    .type = XR_TYPE_SPACE_LOCATION,
  };
  result = xrLocateSpace(trackableAnchor, appSpace, updateTime, &anchorLocation);

  if (plane.trackingState == XR_TRACKING_STATE_TRACKING_ANDROID) {
    // Update anchor pose.
    /* doDrawingForAnchor(anchorLocation.pose); */
  } else {
    // ...
  }
```

```
}

// Cleanup - destroy the space, detach the anchor so its no longer tracked by the
// runtime and then release all resources held by it.
xrDestroySpace(spatialAnchor);
xrDestroySpace(trackableAnchor);
```

New Defines

```
// Provided by XR_ANDROID_trackables
#define XR_NULL_TRACKABLE_ANDROID 0
```

XR_NULL_TRACKABLE_ANDROID defines an invalid trackable atom.

14.11.8. New Macros

- XR_NULL_TRACKABLE_ANDROID

14.11.9. New Base Types

- XrTrackableANDROID

14.11.10. New Object Types

- XrTrackableTrackerANDROID

14.11.11. New Commands

- xrCreateAnchorSpaceANDROID
- xrCreateTrackableTrackerANDROID
- xrDestroyTrackableTrackerANDROID
- xrEnumerateSupportedAnchorTrackableTypesANDROID
- xrEnumerateSupportedTrackableTypesANDROID
- xrGetAllTrackablesANDROID
- xrGetTrackablePlaneANDROID

14.11.12. New Structures

- XrAnchorSpaceCreateInfoANDROID
- XrTrackableGetInfoANDROID
- XrTrackablePlaneANDROID

- XrTrackableTrackerCreateInfoANDROID
- Extending XrSystemProperties:
  - XrSystemTrackablesPropertiesANDROID

14.11.13. New Enums

- XrPlaneLabelANDROID
- XrPlaneTypeANDROID
- XrTrackableTypeANDROID
- XrTrackingStateANDROID

14.11.14. New Enum Constants

- `XR_ANDROID_TRACKABLES_EXTENSION_NAME`
- `XR_ANDROID_trackables_SPEC_VERSION`
- Extending XrObjectType:
  - `XR_OBJECT_TYPE_TRACKABLE_TRACKER_ANDROID`
- Extending XrResult:
  - `XR_ERROR_MISMATCHING_TRACKABLE_TYPE_ANDROID`
  - `XR_ERROR_TRACKABLE_TYPE_NOT_SUPPORTED_ANDROID`
- Extending XrStructureType:
  - `XR_TYPE_ANCHOR_SPACE_CREATE_INFO_ANDROID`
  - `XR_TYPE_SYSTEM_TRACKABLES_PROPERTIES_ANDROID`
  - `XR_TYPE_TRACKABLE_GET_INFO_ANDROID`
  - `XR_TYPE_TRACKABLE_PLANE_ANDROID`
  - `XR_TYPE_TRACKABLE_TRACKER_CREATE_INFO_ANDROID`

14.11.15. Issues

14.11.16. Version History

- Revision 1, 2025-07-21 (Kenny Vercaemer)
  - Initial extension description.
- Revision 2, 2025-09-10 (Nihav Jain)
  - Update XrTrackablePlaneANDROID::`vertexCountOutput` to a pointer.

14.12. XR_ANDROID_trackables_marker

Name String

`XR_ANDROID_trackables_marker`

Extension Type

Instance extension

Registered Extension Number

708

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

`XR_ANDROID_trackables`

Deprecation State

- *Deprecated* by `XR_EXT_spatial_marker_tracking` extension

Last Modified Date

2025-07-23

IP Status

No known IP claims.

Contributors

Christopher Doer, Google
Diego Tipaldi, Google
Levana Chen, Google
Jared Finder, Google
Spencer Quin, Google
Nihav Jain, Google
Ken Mackay, Google
Daniel Guttenberg, Qualcomm

14.12.1. Overview

This extension enables physical marker tracking, and enables applications to attach XR content to physical markers in an efficient way.

The extension supports well known marker types, specifically ArUco and April Tags. It enables

runtimes to optionally support marker size estimation.



Permissions

Android applications **must** have the android.permission.SCENE_UNDERSTANDING_COARSE permission listed in their manifest as this extension depends on `XR_ANDROID_trackables` and exposes the geometry of the environment. The android.permission.SCENE_UNDERSTANDING_COARSE permission is considered a dangerous permission i.e. applications **must** explicitly request the permission.

(protection level: dangerous)

14.12.2. Inspect system capability

The XrSystemMarkerTrackingPropertiesANDROID structure is defined as:

```
// Provided by XR_ANDROID_trackables_marker
typedef struct XrSystemMarkerTrackingPropertiesANDROID {
    XrStructureType    type;
    void*              next;
    XrBool32           supportsMarkerTracking;
    XrBool32           supportsMarkerSizeEstimation;
    uint16_t           maxMarkerCount;
} XrSystemMarkerTrackingPropertiesANDROID;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension. For more details about the structure chain, see the structure being extended (XrSystemProperties).
- `supportsMarkerTracking` is an `XrBool32` indicating if the current system provides marker tracking capability.
- `supportsMarkerSizeEstimation` is an `XrBool32` indicating if the current system provides marker size estimation.
- `maxMarkerCount` is the maximum number of markers that the runtime is able to track at the same time.

An application **can** inspect whether the system is capable of marker tracking by extending the XrSystemProperties with XrSystemMarkerTrackingPropertiesANDROID structure when calling xrGetSystemProperties. The runtime **must** return `XR_ERROR_FEATURE_UNSUPPORTED` for marker tracker

creation if and only if supportsMarkerTracking is XR_FALSE.

If a runtime supports marker tracking, maxMarkerCount **must** be at least 1.

Valid Usage (Implicit)

- The XR_ANDROID_trackables_marker extension **must** be enabled prior to using XrSystemMarkerTrackingPropertiesANDROID
- type **must** be XR_TYPE_SYSTEM_MARKER_TRACKING_PROPERTIES_ANDROID
- next **must** be NULL or a valid pointer to the next structure in a structure chain

14.12.3. Tracking markers

This extension adds XR_TRACKABLE_TYPE_MARKER_ANDROID to XrTrackableTypeANDROID.

The application creates an XrTrackableTrackerANDROID by calling xrCreateTrackableTrackerANDROID and specifying XR_TRACKABLE_TYPE_MARKER_ANDROID as the trackable type in XrTrackableTrackerCreateInfoANDROID::trackableType as well as setting a setting a valid configuration by adding a XrTrackableMarkerConfigurationANDROID to the next chain of XrTrackableTrackerCreateInfoANDROID.

The runtime **must** return XR_ERROR_FEATURE_UNSUPPORTED if XrTrackableTrackerCreateInfoANDROID::trackableType is XR_TRACKABLE_TYPE_MARKER_ANDROID and XrSystemMarkerTrackingPropertiesANDROID::supportsMarkerTracking returns XR_FALSE via xrGetSystemProperties.

The XrTrackableMarkerConfigurationANDROID structure is defined as:

```
// Provided by XR_ANDROID_trackables_marker
typedef struct XrTrackableMarkerConfigurationANDROID {
    XrStructureType                           type;
    void*                                     next;
    XrTrackableMarkerTrackingModeANDROID      trackingMode;
    uint32_t                                  databaseCount;
    const XrTrackableMarkerDatabaseANDROID*   databases;
} XrTrackableMarkerConfigurationANDROID;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `trackingMode` is an XrTrackableMarkerTrackingModeANDROID indicating the desired mode for tracking.
- `databaseCount` is a `uint32_t` describing the count of elements in the `databases` array.
- `databases` is a pointer to an array of XrTrackableMarkerDatabaseANDROID, each of which contains the desired markers from a given dictionary to track.

The application **must** set a valid configuration by adding a XrTrackableMarkerConfigurationANDROID to the XrTrackableTrackerCreateInfoANDROID::`next` chain when calling xrCreateTrackableTrackerANDROID with XrTrackableTrackerCreateInfoANDROID::`trackableType` set to `XR_TRACKABLE_TYPE_MARKER_ANDROID`. Otherwise, if the tracker type is set as above but the configuration structure is not present or not valid, the runtime **must** return `XR_ERROR_VALIDATION_FAILURE`.

If a runtime supports marker size estimation, the application **can** set XrTrackableMarkerDatabaseEntryANDROID::`edgeSize` to `0` in XrTrackableMarkerDatabaseANDROID::`entries` to indicate the usage of size estimation. Otherwise, the application **must** set XrTrackableMarkerDatabaseEntryANDROID::`edgeSize` to a positive value or the runtime **must** return `XR_ERROR_VALIDATION_FAILURE`.

The runtime **must** filter the output from xrGetAllTrackablesANDROID to match the `trackingMode` and XrTrackableMarkerDatabaseEntryANDROID::`edgeSize`.

Valid Usage (Implicit)

- The `XR_ANDROID_trackables_marker` extension **must** be enabled prior to using XrTrackableMarkerConfigurationANDROID
- `type` **must** be `XR_TYPE_TRACKABLE_MARKER_CONFIGURATION_ANDROID`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `trackingMode` **must** be a valid XrTrackableMarkerTrackingModeANDROID value
- `databases` **must** be a pointer to an array of `databaseCount` valid XrTrackableMarkerDatabaseANDROID structures
- The `databaseCount` parameter **must** be greater than `0`

The XrTrackableMarkerTrackingModeANDROID enum describes the supported tracking modes of markers.

```
// Provided by XR_ANDROID_trackables_marker
typedef enum XrTrackableMarkerTrackingModeANDROID {
    XR_TRACKABLE_MARKER_TRACKING_MODE_DYNAMIC_ANDROID = 0,
    XR_TRACKABLE_MARKER_TRACKING_MODE_STATIC_ANDROID = 1,
    XR_TRACKABLE_MARKER_TRACKING_MODE_MAX_ENUM_ANDROID = 0x7FFFFFFF
} XrTrackableMarkerTrackingModeANDROID;
```

Enumerant Descriptions

- XR_TRACKABLE_MARKER_TRACKING_MODE_DYNAMIC_ANDROID — Tracking dynamic markers. This mode has the highest accuracy and works on moving and static markers, but also has the highest power consumption.
- XR_TRACKABLE_MARKER_TRACKING_MODE_STATIC_ANDROID — Tracking static markers. This mode is primarily useful for markers that are known to be static, which leads to less power consumption in comparison to the dynamic mode.

The XrTrackableMarkerDatabaseANDROID structure defines a dictionary and corresponding marker ids to be tracked.

```
// Provided by XR_ANDROID_trackables_marker
typedef struct XrTrackableMarkerDatabaseANDROID {
    XrTrackableMarkerDictionaryANDROID              dictionary;
    uint32_t                                        entryCount;
    const XrTrackableMarkerDatabaseEntryANDROID*    entries;
} XrTrackableMarkerDatabaseANDROID;
```

Member Descriptions

- dictionary is the XrTrackableMarkerDictionaryANDROID that all entries belong to.
- entryCount is a uint32_t describing the count of elements in the entries array. The application **can** set entryCount 0 to track all markers in the dictionary.
- entries is a pointer to an array of XrTrackableMarkerDatabaseEntryANDROID, each of which contains the configuration of a marker to track.

Valid Usage (Implicit)

- The XR_ANDROID_trackables_marker extension **must** be enabled prior to using XrTrackableMarkerDatabaseANDROID
- dictionary **must** be a valid XrTrackableMarkerDictionaryANDROID value
- If entryCount is not 0, entries **must** be a pointer to an array of entryCount XrTrackableMarkerDatabaseEntryANDROID structures

The XrTrackableMarkerDictionaryANDROID enum describes the supported marker dictionaries.

```
// Provided by XR_ANDROID_trackables_marker
typedef enum XrTrackableMarkerDictionaryANDROID {
    XR_TRACKABLE_MARKER_DICTIONARY_ARUCO_4X4_50_ANDROID = 0,
    XR_TRACKABLE_MARKER_DICTIONARY_ARUCO_4X4_100_ANDROID = 1,
    XR_TRACKABLE_MARKER_DICTIONARY_ARUCO_4X4_250_ANDROID = 2,
    XR_TRACKABLE_MARKER_DICTIONARY_ARUCO_4X4_1000_ANDROID = 3,
    XR_TRACKABLE_MARKER_DICTIONARY_ARUCO_5X5_50_ANDROID = 4,
    XR_TRACKABLE_MARKER_DICTIONARY_ARUCO_5X5_100_ANDROID = 5,
    XR_TRACKABLE_MARKER_DICTIONARY_ARUCO_5X5_250_ANDROID = 6,
    XR_TRACKABLE_MARKER_DICTIONARY_ARUCO_5X5_1000_ANDROID = 7,
    XR_TRACKABLE_MARKER_DICTIONARY_ARUCO_6X6_50_ANDROID = 8,
    XR_TRACKABLE_MARKER_DICTIONARY_ARUCO_6X6_100_ANDROID = 9,
    XR_TRACKABLE_MARKER_DICTIONARY_ARUCO_6X6_250_ANDROID = 10,
    XR_TRACKABLE_MARKER_DICTIONARY_ARUCO_6X6_1000_ANDROID = 11,
    XR_TRACKABLE_MARKER_DICTIONARY_ARUCO_7X7_50_ANDROID = 12,
    XR_TRACKABLE_MARKER_DICTIONARY_ARUCO_7X7_100_ANDROID = 13,
    XR_TRACKABLE_MARKER_DICTIONARY_ARUCO_7X7_250_ANDROID = 14,
    XR_TRACKABLE_MARKER_DICTIONARY_ARUCO_7X7_1000_ANDROID = 15,
    XR_TRACKABLE_MARKER_DICTIONARY_APRILTAG_16H5_ANDROID = 16,
    XR_TRACKABLE_MARKER_DICTIONARY_APRILTAG_25H9_ANDROID = 17,
    XR_TRACKABLE_MARKER_DICTIONARY_APRILTAG_36H10_ANDROID = 18,
    XR_TRACKABLE_MARKER_DICTIONARY_APRILTAG_36H11_ANDROID = 19,
    XR_TRACKABLE_MARKER_DICTIONARY_MAX_ENUM_ANDROID = 0x7FFFFFFF
} XrTrackableMarkerDictionaryANDROID;
```

The XrTrackableMarkerDatabaseEntryANDROID structure configures a single marker id of a dictionary.

```
// Provided by XR_ANDROID_trackables_marker
typedef struct XrTrackableMarkerDatabaseEntryANDROID {
    int32_t    id;
    float      edgeSize;
} XrTrackableMarkerDatabaseEntryANDROID;
```

Member Descriptions

- `id` is the marker id as given in the dictionary.
- `edgeSize` represents the size of marker edge in meters. If the runtime supports marker size estimation, the application **can** set this to zero and the marker size will be estimated online. If this is set to zero but the runtime does not support marker size estimation, the runtime **must** return `XR_ERROR_VALIDATION_FAILURE`.

Valid Usage (Implicit)

- The `XR_ANDROID_trackables_marker` extension **must** be enabled prior to using XrTrackableMarkerDatabaseEntryANDROID

14.12.4. Get markers

The xrGetTrackableMarkerANDROID function is defined as:

```
// Provided by XR_ANDROID_trackables_marker
XrResult xrGetTrackableMarkerANDROID(
    XrTrackableTrackerANDROID                   tracker,
    const XrTrackableGetInfoANDROID*            getInfo,
    XrTrackableMarkerANDROID*                   markerOutput);
```

Parameter Descriptions

- `tracker` is the XrTrackableTrackerANDROID to query.
- `getInfo` is the XrTrackableGetInfoANDROID with the information used to get the trackable marker.
- `markerOutput` is a pointer to the XrTrackableMarkerANDROID structure in which the trackable marker is returned.

The runtime **must** return `XR_ERROR_MISMATCHING_TRACKABLE_TYPE_ANDROID` if the trackable type of the

XrTrackableANDROID is not XR_TRACKABLE_TYPE_MARKER_ANDROID, or if the trackable type of the XrTrackableTrackerANDROID is not XR_TRACKABLE_TYPE_MARKER_ANDROID.

Valid Usage (Implicit)

- The XR_ANDROID_trackables_marker extension **must** be enabled prior to calling xrGetTrackableMarkerANDROID
- `tracker` **must** be a valid XrTrackableTrackerANDROID handle
- `getInfo` **must** be a pointer to a valid XrTrackableGetInfoANDROID structure
- `markerOutput` **must** be a pointer to an XrTrackableMarkerANDROID structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_MISMATCHING_TRACKABLE_TYPE_ANDROID
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_TIME_INVALID
- XR_ERROR_VALIDATION_FAILURE

The XrTrackableMarkerANDROID structure is defined as:

```
// Provided by XR_ANDROID_trackables_marker
typedef struct XrTrackableMarkerANDROID {
    XrStructureType                         type;
    void*                                   next;
    XrTrackingStateANDROID                  trackingState;
    XrTime                                  lastUpdatedTime;
    XrTrackableMarkerDictionaryANDROID      dictionary;
    int32_t                                 markerId;
    XrPosef                                 centerPose;
    XrExtent2Df                             extents;
} XrTrackableMarkerANDROID;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `trackingState` is the XrTrackingStateANDROID of the marker.
- `lastUpdatedTime` is the `XrTime` of the last update of the marker.
- `dictionary` is the XrTrackableMarkerDictionaryANDROID of the marker.
- `markerId` is the marker id as given in the dictionary.
- `centerPose` is the XrPosef of the marker located in XrTrackableGetInfoANDROID::`baseSpace`. The marker lies in the XZ plane with X pointing to the right of the marker and Z pointing to its bottom and Y coming out of the marker as the normal.
- `extents` is the XrExtent2Df dimensions of the marker. The boundary of the bounding box is at points: `centerPose` +/- (`extents` / 2).

Valid Usage (Implicit)

- The `XR_ANDROID_trackables_marker` extension **must** be enabled prior to using XrTrackableMarkerANDROID
- `type` **must** be `XR_TYPE_TRACKABLE_MARKER_ANDROID`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `trackingState` **must** be a valid XrTrackingStateANDROID value
- `dictionary` **must** be a valid XrTrackableMarkerDictionaryANDROID value

14.12.5. Example code for getting trackable markers

The following example code demonstrates how to get trackable markers.

```
XrInstance instance; // previously initialized
XrSystemId systemId; // previously initialized
XrSession session;   // previously initialized

// The function pointers are previously initialized using xrGetInstanceProcAddr.
PFN_xrGetSystemProperties xrGetSystemProperties;                      // previously
initialized
PFN_xrCreateTrackableTrackerANDROID xrCreateTrackableTrackerANDROID;   // previously
initialized
PFN_xrGetAllTrackablesANDROID xrGetAllTrackablesANDROID;              // previously
initialized
PFN_xrGetTrackableMarkerANDROID xrGetTrackableMarkerANDROID;          // previously
initialized
PFN_xrDestroyTrackableTrackerANDROID xrDestroyTrackableTrackerANDROID; // previously
initialized

XrTime updateTime; // Time used for the current frame's simulation update.
XrSpace appSpace;  // Space created for XR_REFERENCE_SPACE_TYPE_LOCAL.

// Inspect system capability
XrSystemMarkerTrackingPropertiesANDROID markerProperty {
  .type = XR_TYPE_SYSTEM_MARKER_TRACKING_PROPERTIES_ANDROID,
  .next = nullptr,
};
XrSystemProperties systemProperties {
  .type = XR_TYPE_SYSTEM_PROPERTIES,
  .next = &markerProperty,
};
CHK_XR(xrGetSystemProperties(instance, systemId, &systemProperties));
if (!markerProperty.supportsMarkerTracking) {
    // Marker tracking is not supported.
    return;
}

// Create a trackable tracker for marker tracking.
// If the runtime does not support size estimation, configures marker edge size of 0.1m.
XrTrackableMarkerDatabaseEntryANDROID markerEntries {
  .id = 0,
  .edgeSize = markerProperty.supportsMarkerSizeEstimation ? 0.0f : 0.1f,
};
XrTrackableMarkerDatabaseANDROID markerDatabases {
  .dictionary = XR_TRACKABLE_MARKER_DICTIONARY_ARUCO_4X4_50_ANDROID,
  .entryCount = 1,
```

```
  .entries = &markerEntries,
};
XrTrackableMarkerConfigurationANDROID configuration {
  .type = XR_TYPE_TRACKABLE_MARKER_CONFIGURATION_ANDROID,
  .next = nullptr,
  .trackingMode = XR_TRACKABLE_MARKER_TRACKING_MODE_DYNAMIC_ANDROID,
  .databaseCount = 1,
  .databases = &markerDatabases,
};
XrTrackableTrackerCreateInfoANDROID createInfo {
  .type = XR_TYPE_TRACKABLE_TRACKER_CREATE_INFO_ANDROID,
  .next = &configuration,
  .trackableType = XR_TRACKABLE_TYPE_MARKER_ANDROID,
};
XrTrackableTrackerANDROID markerTracker;
auto res = xrCreateTrackableTrackerANDROID(session, &createInfo, &markerTracker);
if (res == XR_ERROR_PERMISSION_INSUFFICIENT) {
    // Handle permission requests.
}
CHK_XR(res);

// Get markers.
std::vector<XrTrackableANDROID> trackables(markerProperty.maxMarkerCount);
std::vector<XrTrackableMarkerANDROID> markers(markerProperty.maxMarkerCount, {
  .type = XR_TYPE_TRACKABLE_MARKER_ANDROID,
  .next = nullptr,
});
uint32_t markerSize = 0;
CHK_XR(xrGetAllTrackablesANDROID(markerTracker, markerProperty.maxMarkerCount,
&markerSize,
                                 trackables.data()));
for (int i = 0; i < markerSize; i++) {
    XrTrackableGetInfoANDROID getInfo {
      .type = XR_TYPE_TRACKABLE_GET_INFO_ANDROID,
      .next = nullptr,
      .trackable = trackables[i],
      .baseSpace = appSpace,
      .time = updateTime,
    };
    CHK_XR(xrGetTrackableMarkerANDROID(markerTracker, &getInfo, &markers[i]));
    // Handle markers.
}

// Release trackable tracker.
CHK_XR(xrDestroyTrackableTrackerANDROID(markerTracker));
```

14.12.6. New Commands

- xrGetTrackableMarkerANDROID

14.12.7. New Structures

- XrTrackableMarkerANDROID
- XrTrackableMarkerDatabaseANDROID
- XrTrackableMarkerDatabaseEntryANDROID
- Extending XrSystemProperties:
  - XrSystemMarkerTrackingPropertiesANDROID
- Extending XrTrackableTrackerCreateInfoANDROID:
  - XrTrackableMarkerConfigurationANDROID

14.12.8. New Enums

- XrTrackableMarkerDictionaryANDROID
- XrTrackableMarkerTrackingModeANDROID

14.12.9. New Enum Constants

- `XR_ANDROID_TRACKABLES_MARKER_EXTENSION_NAME`
- `XR_ANDROID_trackables_marker_SPEC_VERSION`
- Extending XrStructureType:
  - `XR_TYPE_SYSTEM_MARKER_TRACKING_PROPERTIES_ANDROID`
  - `XR_TYPE_TRACKABLE_MARKER_ANDROID`
  - `XR_TYPE_TRACKABLE_MARKER_CONFIGURATION_ANDROID`
- Extending XrTrackableTypeANDROID:
  - `XR_TRACKABLE_TYPE_MARKER_ANDROID`

14.12.10. Issues

14.12.11. Version History

- Revision 1, 2025-07-23 (Levana Chen)
  - Initial extension description.

14.13. XR_ANDROID_trackables_object

Name String

`XR_ANDROID_trackables_object`

Extension Type

Instance extension

Registered Extension Number

467

Revision

2

Ratification Status

Not ratified

Extension and Version Dependencies

`XR_ANDROID_trackables`

Deprecation State

- *Deprecated* by `XR_ANDROID_spatial_object_tracking` extension

Last Modified Date

2025-09-23

IP Status

No known IP claims.

Contributors

Diego Tipaldi, Google
David Joseph Tan, Google
Christopher Doer, Google
Levana Chen, Google
Spencer Quin, Google
Jared Finder, Google
Kenny Vercaemer, Google

14.13.1. Overview

This extension enables physical object tracking such as keyboards, mice, and other objects in the environment.



Permissions

Android applications **must** have the android.permission.SCENE_UNDERSTANDING_COARSE permission listed in their manifest as this extension depends on XR_ANDROID_trackables and exposes the geometry of the environment. The android.permission.SCENE_UNDERSTANDING_COARSE permission is considered a dangerous permission.

(protection level: dangerous)

14.13.2. Tracking objects

To track objects, the application creates an XrTrackableTrackerANDROID by calling xrCreateTrackableTrackerANDROID and specifying the XR_TRACKABLE_TYPE_OBJECT_ANDROID enumerant added to XrTrackableTypeANDROID by this extension as the trackable type in XrTrackableTrackerCreateInfoANDROID::trackableType.

The XrTrackableObjectConfigurationANDROID structure is defined as:

```
// Provided by XR_ANDROID_trackables_object
typedef struct XrTrackableObjectConfigurationANDROID {
    XrStructureType                type;
    void*                          next;
    uint32_t                       labelCount;
    const XrObjectLabelANDROID*    activeLabels;
} XrTrackableObjectConfigurationANDROID;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- labelCount is the count of activeLabels.
- activeLabels is a pointer to the array of XrObjectLabelANDROID indicating the active objects in tracking.

- The application **can** configure the XrTrackableTrackerANDROID to track only a subset of supported objects by adding a XrTrackableObjectConfigurationANDROID structure to the XrTrackableTrackerCreateInfoANDROID::next chain.
- The runtime **must** filter the output from xrGetAllTrackablesANDROID to match any of the activeLabels.

If the application does not add XrTrackableObjectConfigurationANDROID to the XrTrackableTrackerCreateInfoANDROID::next chain, the runtime **must** track all objects that the system has identified.

Valid Usage (Implicit)

- The XR_ANDROID_trackables_object extension **must** be enabled prior to using XrTrackableObjectConfigurationANDROID
- type **must** be XR_TYPE_TRACKABLE_OBJECT_CONFIGURATION_ANDROID
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- activeLabels **must** be a pointer to an array of labelCount valid XrObjectLabelANDROID values
- The labelCount parameter **must** be greater than 0

The XrObjectLabelANDROID enum describes the type of object that the system has identified.

```
// Provided by XR_ANDROID_trackables_object
typedef enum XrObjectLabelANDROID {
    XR_OBJECT_LABEL_UNKNOWN_ANDROID = 0,
    XR_OBJECT_LABEL_KEYBOARD_ANDROID = 1,
    XR_OBJECT_LABEL_MOUSE_ANDROID = 2,
    XR_OBJECT_LABEL_LAPTOP_ANDROID = 3,
    XR_OBJECT_LABEL_MAX_ENUM_ANDROID = 0x7FFFFFFF
} XrObjectLabelANDROID;
```

14.13.3. Get trackable object

The xrGetTrackableObjectANDROID function is defined as:

```
// Provided by XR_ANDROID_trackables_object
XrResult xrGetTrackableObjectANDROID(
    XrTrackableTrackerANDROID                   tracker,
    const XrTrackableGetInfoANDROID*            getInfo,
    XrTrackableObjectANDROID*                   objectOutput);
```

Parameter Descriptions

- `tracker` is the XrTrackableTrackerANDROID to query.
- `getInfo` is the XrTrackableGetInfoANDROID with the information used to get the trackable object.
- `objectOutput` is a pointer to the XrTrackableObjectANDROID structure in which the trackable object is returned.

The runtime **must** return `XR_ERROR_MISMATCHING_TRACKABLE_TYPE_ANDROID` if the trackable type of the `XrTrackableANDROID` is not `XR_TRACKABLE_TYPE_OBJECT_ANDROID`, or if the trackable type of the XrTrackableTrackerANDROID is not `XR_TRACKABLE_TYPE_OBJECT_ANDROID`.

Valid Usage (Implicit)

- The `XR_ANDROID_trackables_object` extension **must** be enabled prior to calling xrGetTrackableObjectANDROID
- `tracker` **must** be a valid XrTrackableTrackerANDROID handle
- `getInfo` **must** be a pointer to a valid XrTrackableGetInfoANDROID structure
- `objectOutput` **must** be a pointer to an XrTrackableObjectANDROID structure

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_MISMATCHING_TRACKABLE_TYPE_ANDROID`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_SESSION_NOT_RUNNING`
- `XR_ERROR_TIME_INVALID`
- `XR_ERROR_VALIDATION_FAILURE`

The XrTrackableObjectANDROID structure is defined as:

```
// Provided by XR_ANDROID_trackables_object
typedef struct XrTrackableObjectANDROID {
    XrStructureType           type;
    void*                     next;
    XrTrackingStateANDROID    trackingState;
    XrPosef                   centerPose;
    XrExtent3DfEXT            extents;
    XrObjectLabelANDROID      objectLabel;
    XrTime                    lastUpdatedTime;
} XrTrackableObjectANDROID;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `trackingState` is the XrTrackingStateANDROID of the object.
- `centerPose` is the XrPosef of the object located in XrTrackableGetInfoANDROID::`baseSpace`.
- `extents` is the XrExtent3DfEXT dimension of the object.
- `objectLabel` is the XrObjectLabelANDROID that the runtime has determined for this object.
- `lastUpdatedTime` is the `XrTime` of the last update of the object.

The runtime **must** only return objects that it successfully identified. I.e. XrTrackableObjectANDROID ::`objectLabel` **must** not be `XR_OBJECT_LABEL_UNKNOWN_ANDROID`.

Valid Usage (Implicit)

- The `XR_ANDROID_trackables_object` extension **must** be enabled prior to using XrTrackableObjectANDROID
- `type` **must** be `XR_TYPE_TRACKABLE_OBJECT_ANDROID`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `trackingState` **must** be a valid XrTrackingStateANDROID value
- `objectLabel` **must** be a valid XrObjectLabelANDROID value

14.13.4. Trackable object orientation

The orientation of the objects is shown in the following image (with xyz = rgb) originating at the centroid of the objects.

The 3D orientation of the mouse is aligned to the 3D orientation of the supporting plane. The extents enclose the keyboard and mouse completely and the base of the laptop.

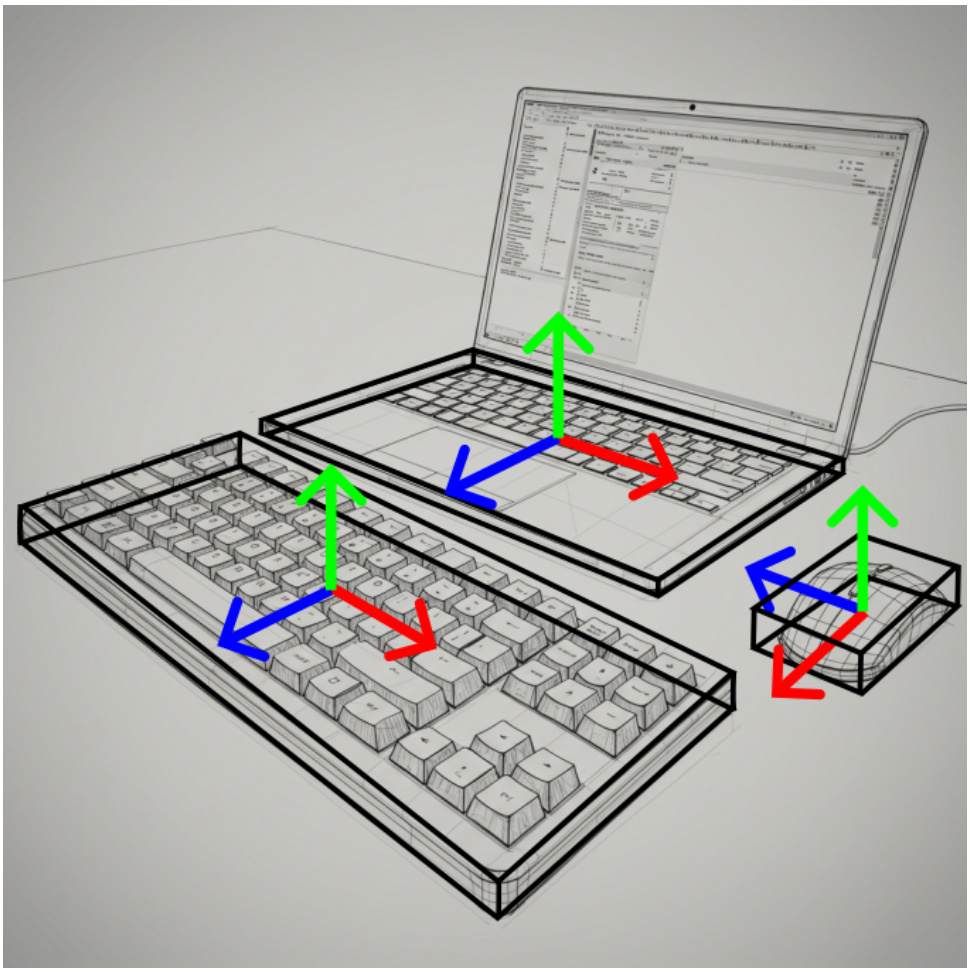


Figure 40. Example object orientation.

14.13.5. Example code for getting trackable objects

The following example code demonstrates how to get trackable objects.

```
XrSession session; // previously initialized

// The function pointers are previously initialized using xrGetInstanceProcAddr.
PFN_xrCreateTrackableTrackerANDROID xrCreateTrackableTrackerANDROID; // previously
initialized
PFN_xrGetAllTrackablesANDROID xrGetAllTrackablesANDROID; // previously initialized
PFN_xrGetTrackableObjectANDROID xrGetTrackableObjectANDROID; // previously initialized
PFN_xrDestroyTrackableTrackerANDROID xrDestroyTrackableTrackerANDROID; // previously
```

```
initialized

XrTime updateTime; // Time used for the current frame's simulation update.
XrSpace appSpace; // Space created for XR_REFERENCE_SPACE_TYPE_LOCAL.

XrTrackableTrackerCreateInfoANDROID createInfo {
  .type = XR_TYPE_TRACKABLE_TRACKER_CREATE_INFO_ANDROID,
  .trackableType = XR_TRACKABLE_TYPE_OBJECT_ANDROID,
};
XrTrackableTrackerANDROID objectTrackableTracker;
XrResult result = xrCreateTrackableTrackerANDROID(
  session,
  &createInfo,
  &objectTrackableTracker);
if (result != XR_SUCCESS) { /* Handle failures. */ }

uint32_t trackableCountOutput = 0;
std::vector<XrTrackableANDROID> allObjectTrackables;

// Query the number of trackables available.
result = xrGetAllTrackablesANDROID(
  objectTrackableTracker,
  0,
  &trackableCountOutput,
  nullptr
);

if (result == XR_SUCCESS) {
  allObjectTrackables.resize(trackableCountOutput, XR_NULL_TRACKABLE_ANDROID);

  // Fetch the actual trackable handles in the appropriately resized array.
  result = xrGetAllTrackablesANDROID(
    objectTrackableTracker,
    trackableCountOutput,
    &trackableCountOutput,
    allObjectTrackables.data());

  if (result == XR_SUCCESS) {
    for (XrTrackableANDROID trackable : allObjectTrackables) {

      // Object trackable query information
      XrTrackableGetInfoANDROID objectGetInfo {
        .type = XR_TYPE_TRACKABLE_GET_INFO_ANDROID,
        .next = nullptr,
        .trackable = trackable,
        .baseSpace = appSpace,
        .time = updateTime,
      };
```

```
      // Get the object trackable. Note that the tracker only returns object types.
      XrTrackableObjectANDROID object = { XR_TYPE_TRACKABLE_OBJECT_ANDROID };
      result = xrGetTrackableObjectANDROID(
        objectTrackableTracker,
        &objectGetInfo,
        &object
      );

      if (result == XR_SUCCESS) {
        /** Do Stuff with the object */
      }
    }
  }
}

// Release trackable tracker.
result = xrDestroyTrackableTrackerANDROID(objectTrackableTracker);
```

14.13.6. New Commands

- xrGetTrackableObjectANDROID

14.13.7. New Structures

- XrTrackableObjectANDROID
- Extending XrTrackableTrackerCreateInfoANDROID:
  - XrTrackableObjectConfigurationANDROID

14.13.8. New Enums

- XrObjectLabelANDROID

14.13.9. New Enum Constants

- XR_ANDROID_TRACKABLES_OBJECT_EXTENSION_NAME
- XR_ANDROID_trackables_object_SPEC_VERSION
- Extending XrStructureType:
  - XR_TYPE_TRACKABLE_OBJECT_ANDROID
  - XR_TYPE_TRACKABLE_OBJECT_CONFIGURATION_ANDROID
- Extending XrTrackableTypeANDROID:
  - XR_TRACKABLE_TYPE_OBJECT_ANDROID

14.13.10. Issues

14.13.11. Version History

- Revision 1, 2025-07-21 (Kenny Vercaemer)
  - Initial extension description.
- Revision 2, 2025-09-23 (Kenny Vercaemer)
  - Fix object orientation.

14.14. XR_ANDROID_trackables_qr_code

Name String

`XR_ANDROID_trackables_qr_code`

Extension Type

Instance extension

Registered Extension Number

709

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

`XR_ANDROID_trackables`

Deprecation State

- *Deprecated* by `XR_EXT_spatial_marker_tracking` extension

Last Modified Date

2025-02-05

IP Status

No known IP claims.

Contributors

Christopher Doer, Google
Levana Chen, Google
Jared Finder, Google
Spencer Quin, Google

Nihav Jain, Google
Diego Tipaldi, Google
Ken Mackay, Google
Daniel Guttenberg, Qualcomm

14.14.1. Overview

This extension enables physical QR code tracking and QR code data decoding.



Permissions

Android applications **must** have the android.permission.SCENE_UNDERSTANDING_COARSE permission listed in their manifest as this extension depends on `XR_ANDROID_trackables` and exposes the geometry of the environment. The android.permission.SCENE_UNDERSTANDING_COARSE permission is considered a dangerous permission.

(protection level: dangerous)

14.14.2. Inspect system capability

The XrSystemQrCodeTrackingPropertiesANDROID structure is defined as:

```
// Provided by XR_ANDROID_trackables_qr_code
typedef struct XrSystemQrCodeTrackingPropertiesANDROID {
    XrStructureType    type;
    void*              next;
    XrBool32           supportsQrCodeTracking;
    XrBool32           supportsQrCodeSizeEstimation;
    uint16_t           maxQrCodeCount;
} XrSystemQrCodeTrackingPropertiesANDROID;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `supportsQrCodeTracking` is an `XrBool32` indicating if current system provides QR code tracking capability.
- `supportsQrCodeSizeEstimation` is an `XrBool32` indicating if current system provides QR code size estimation.
- `maxQrCodeCount` is the total maximum number of QR codes that **can** be tracked at the same time.

An application **can** inspect whether the system is capable of QR code tracking by extending the XrSystemProperties with XrSystemQrCodeTrackingPropertiesANDROID structure when calling xrGetSystemProperties. The runtime **must** return `XR_ERROR_FEATURE_UNSUPPORTED` for QR code tracker creation if and only if `supportsQrCodeTracking` is `XR_FALSE`.

If a runtime supports QR code tracking, `maxQrCodeCount` **must** be at least 1. If a runtime does not support QR code tracking, `maxQrCodeCount` **must** be 0.

Valid Usage (Implicit)

- The `XR_ANDROID_trackables_qr_code` extension **must** be enabled prior to using XrSystemQrCodeTrackingPropertiesANDROID
- `type` **must** be `XR_TYPE_SYSTEM_QR_CODE_TRACKING_PROPERTIES_ANDROID`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

14.14.3. Tracking QR codes

This extension adds `XR_TRACKABLE_TYPE_QR_CODE_ANDROID` to XrTrackableTypeANDROID.

The application **may** create an XrTrackableTrackerANDROID by calling xrCreateTrackableTrackerANDROID and specifying `XR_TRACKABLE_TYPE_QR_CODE_ANDROID` as the trackable type in XrTrackableTrackerCreateInfoANDROID::`trackableType` to track QR codes.

The runtime **must** return `XR_ERROR_FEATURE_UNSUPPORTED` if XrTrackableTrackerCreateInfoANDROID::`trackableType` is `XR_TRACKABLE_TYPE_QR_CODE_ANDROID` and XrSystemQrCodeTrackingPropertiesANDROID::`supportsQrCodeTracking` returns `XR_FALSE` via xrGetSystemProperties.

The XrTrackableQrCodeConfigurationANDROID structure is defined as:

```
// Provided by XR_ANDROID_trackables_qr_code
typedef struct XrTrackableQrCodeConfigurationANDROID {
    XrStructureType                type;
    void*                          next;
    XrQrCodeTrackingModeANDROID    trackingMode;
    float                          qrCodeEdgeSize;
} XrTrackableQrCodeConfigurationANDROID;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `trackingMode` is an XrQrCodeTrackingModeANDROID indicating the desired mode for tracking.
- `qrCodeEdgeSize` indicates the size of the QR code edge in meters. If zero, the runtime estimates the QR code size online.

The application **must** set a valid configuration by adding a XrTrackableQrCodeConfigurationANDROID to the next chain of XrTrackableTrackerCreateInfoANDROID. Otherwise, the runtime **must** return `XR_ERROR_VALIDATION_FAILURE`.

If the runtime supports QR code size estimation, the application **may** set XrTrackableQrCodeConfigurationANDROID::`qrCodeEdgeSize` to `0.0` to indicate the usage of size estimation.

If the runtime does not support QR code size estimation, the application **must** set XrTrackableQrCodeConfigurationANDROID::`qrCodeEdgeSize` to a positive value, otherwise the runtime **must** return `XR_ERROR_VALIDATION_FAILURE`.

The runtime **must** filter the output from xrGetAllTrackablesANDROID to match the `trackingMode`. If XrTrackableQrCodeConfigurationANDROID::`qrCodeEdgeSize` is not set to `0.0` the runtime **must** only return QR codes that match this size. If XrTrackableQrCodeConfigurationANDROID::`qrCodeEdgeSize` is set to `0.0` the runtime **must** return all QR codes with estimated size.

Valid Usage (Implicit)

- The `XR_ANDROID_trackables_qr_code` extension **must** be enabled prior to using XrTrackableQrCodeConfigurationANDROID
- `type` **must** be `XR_TYPE_TRACKABLE_QR_CODE_CONFIGURATION_ANDROID`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `trackingMode` **must** be a valid XrQrCodeTrackingModeANDROID value

The XrQrCodeTrackingModeANDROID enum describes the supported tracking modes of QR codes.

```
// Provided by XR_ANDROID_trackables_qr_code
typedef enum XrQrCodeTrackingModeANDROID {
    XR_QR_CODE_TRACKING_MODE_DYNAMIC_ANDROID = 0,
    XR_QR_CODE_TRACKING_MODE_STATIC_ANDROID = 1,
    XR_QR_CODE_TRACKING_MODE_MAX_ENUM_ANDROID = 0x7FFFFFFF
} XrQrCodeTrackingModeANDROID;
```

Enumerant Descriptions

- `XR_QR_CODE_TRACKING_MODE_DYNAMIC_ANDROID` — Tracking dynamic QR codes. This mode has the highest accuracy and works on moving and static QR codes, but also has the highest power consumption.
- `XR_QR_CODE_TRACKING_MODE_STATIC_ANDROID` — Tracking static QR codes. This mode is primarily useful for QR codes that are known to be static, which leads to less power consumption in comparison to the dynamic mode.

14.14.4. Get QR codes

The xrGetTrackableQrCodeANDROID function is defined as:

```
// Provided by XR_ANDROID_trackables_qr_code
XrResult xrGetTrackableQrCodeANDROID(
    XrTrackableTrackerANDROID                   tracker,
    const XrTrackableGetInfoANDROID*            getInfo,
    XrTrackableQrCodeANDROID*                   qrCodeOutput);
```

Parameter Descriptions

- `tracker` is the XrTrackableTrackerANDROID to query.
- `getInfo` is the XrTrackableGetInfoANDROID with the information used to get the trackable QR code.
- `qrCodeOutput` is a pointer to the XrTrackableQrCodeANDROID structure in which the trackable QR code is returned.

The runtime **must** return `XR_ERROR_MISMATCHING_TRACKABLE_TYPE_ANDROID` if the trackable type of the `XrTrackableANDROID` is not `XR_TRACKABLE_TYPE_QR_CODE_ANDROID`, or if the trackable type of the XrTrackableTrackerANDROID is not `XR_TRACKABLE_TYPE_QR_CODE_ANDROID`.

Valid Usage (Implicit)

- The `XR_ANDROID_trackables_qr_code` extension **must** be enabled prior to calling xrGetTrackableQrCodeANDROID
- `tracker` **must** be a valid XrTrackableTrackerANDROID handle
- `getInfo` **must** be a pointer to a valid XrTrackableGetInfoANDROID structure
- `qrCodeOutput` **must** be a pointer to an XrTrackableQrCodeANDROID structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_MISMATCHING_TRACKABLE_TYPE_ANDROID
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_SIZE_INSUFFICIENT
- XR_ERROR_TIME_INVALID
- XR_ERROR_VALIDATION_FAILURE

The XrTrackableQrCodeANDROID structure is defined as:

```
// Provided by XR_ANDROID_trackables_qr_code
typedef struct XrTrackableQrCodeANDROID {
    XrStructureType           type;
    void*                     next;
    XrTrackingStateANDROID    trackingState;
    XrTime                    lastUpdatedTime;
    XrPosef                   centerPose;
    XrExtent2Df               extents;
    uint32_t                  bufferCapacityInput;
    uint32_t                  bufferCountOutput;
    char*                     buffer;
} XrTrackableQrCodeANDROID;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `trackingState` is the XrTrackingStateANDROID of the QR code.
- `lastUpdatedTime` is the `XrTime` of the last update of the QR code. If `lastUpdatedTime` is changed from the last call, all other fields **may** have changed.
- `centerPose` is the XrPosef of the QR code located in XrTrackableGetInfoANDROID::`baseSpace`. The QR code lies in the XZ plane with X pointing to the right of the QR code, Z pointing to its bottom and Y coming out of the QR code as the normal.
- `extents` is the XrExtent2Df dimensions of the QR code. The boundary of the bounding box is at points: `centerPose` +/- (`extents` / 2).
- `bufferCapacityInput` is the capability of the `buffer`, or `0` to retrieve the required capability.
- `bufferCountOutput` If the `bufferCapacityInput` is `0`, the runtime will write the required buffer size into `bufferCountOutput`. Otherwise, it contains the total elements written in `buffer`. If the QR code data has not been decoded yet, the runtime **must** set bufferCountOutput to 0.
- `buffer` is a pointer to an array of `char` to write the decoded QR code data. If the application does not care about the decoded QR code data it **can** pass `nullptr` and omit the second two-call call. The QR code data is returned as null-terminated UTF-8 string.
- See the Buffer Size Parameters section for a detailed description of retrieving the required `buffer` size.

Valid Usage (Implicit)

- The `XR_ANDROID_trackables_qr_code` extension **must** be enabled prior to using XrTrackableQrCodeANDROID
- `type` **must** be `XR_TYPE_TRACKABLE_QR_CODE_ANDROID`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `trackingState` **must** be a valid XrTrackingStateANDROID value
- If `bufferCapacityInput` is not `0`, `buffer` **must** be a pointer to an array of `bufferCapacityInput` char values

14.14.5. Example code for getting trackable QR codes

The following example code demonstrates how to get trackable QR codes.

```
XrInstance instance; // previously initialized
XrSystemId systemId; // previously initialized
XrSession session;   // previously initialized

// The function pointers are previously initialized using xrGetInstanceProcAddr.
PFN_xrGetSystemProperties xrGetSystemProperties;                      // previously
initialized
PFN_xrCreateTrackableTrackerANDROID xrCreateTrackableTrackerANDROID;   // previously
initialized
PFN_xrGetAllTrackablesANDROID xrGetAllTrackablesANDROID;               // previously
initialized
PFN_xrGetTrackableQrCodeANDROID xrGetTrackableQrCodeANDROID;           // previously
initialized
PFN_xrDestroyTrackableTrackerANDROID xrDestroyTrackableTrackerANDROID; // previously
initialized

XrTime updateTime; // Time used for the current frame's simulation update.
XrSpace appSpace;  // Space created for XR_REFERENCE_SPACE_TYPE_LOCAL.

// Inspect system capability
XrSystemQrCodeTrackingPropertiesANDROID qrCodeProperty {
  .type = XR_TYPE_SYSTEM_QR_CODE_TRACKING_PROPERTIES_ANDROID,
  .next = nullptr,
};
XrSystemProperties systemProperties {
  .type = XR_TYPE_SYSTEM_PROPERTIES,
  .next = &qrCodeProperty,
};
CHK_XR(xrGetSystemProperties(instance, systemId, &systemProperties));
if (!qrCodeProperty.supportsQrCodeTracking) {
    // QR code tracking is not supported.
    return;
}

// Create a trackable tracker for QR code tracking.
// If the runtime does not support size estimation, configures QR code edge size of 0.1m.
XrTrackableQrCodeConfigurationANDROID configuration {
  .type = XR_TYPE_TRACKABLE_QR_CODE_CONFIGURATION_ANDROID,
  .next = nullptr,
  .trackingMode = XR_QR_CODE_TRACKING_MODE_DYNAMIC_ANDROID,
  .qrCodeEdgeSize = qrCodeProperty.supportsQrCodeSizeEstimation ? 0.0f : 0.1f,
};
XrTrackableTrackerCreateInfoANDROID createInfo {
  .type = XR_TYPE_TRACKABLE_TRACKER_CREATE_INFO_ANDROID,
  .next = &configuration,
  .trackableType = XR_TRACKABLE_TYPE_QR_CODE_ANDROID
};
XrTrackableTrackerANDROID qrCodeTracker;
```

```
auto res = xrCreateTrackableTrackerANDROID(session, &createInfo, &qrCodeTracker);
if (res == XR_ERROR_PERMISSION_INSUFFICIENT) {
    // Handle permission requests.
}
CHK_XR(res);

// Get QR codes.
std::vector<XrTrackableANDROID> trackables(qrCodeProperty.maxQrCodeCount);
std::vector<XrTrackableQrCodeANDROID> qrCodes(qrCodeProperty.maxQrCodeCount, {
  .type = XR_TYPE_TRACKABLE_QR_CODE_ANDROID,
  .next = nullptr,
  .bufferCountOutput = 0,
});
uint32_t qrCodeSize = 0;
CHK_XR(xrGetAllTrackablesANDROID(qrCodeTracker, qrCodeProperty.maxQrCodeCount,
&qrCodeSize,
                                 trackables.data()));
for (int i = 0; i < qrCodeSize; i++) {
    XrTrackableGetInfoANDROID getInfo {
      .type = XR_TYPE_TRACKABLE_GET_INFO_ANDROID,
      .next = nullptr,
      .trackable = trackables.at(i),
      .baseSpace = appSpace,
      .time = updateTime,
    };
    CHK_XR(xrGetTrackableQrCodeANDROID(qrCodeTracker, &getInfo, &qrCodes[i]));
    if (qrCodes[i].bufferCountOutput > 0) {
        // Allocate the buffer if it is not already allocated.
        if (qrCodes[i].bufferCapacityInput == 0) {
            qrCodes[i].buffer = new char[qrCodes[i].bufferCountOutput];
            qrCodes[i].bufferCapacityInput = qrCodes[i].bufferCountOutput;
            CHK_XR(xrGetTrackableQrCodeANDROID(qrCodeTracker, &getInfo, &qrCodes[i]));
        }
    }
}

// Release trackable tracker.
CHK_XR(xrDestroyTrackableTrackerANDROID(qrCodeTracker));
```

14.14.6. New Commands

- xrGetTrackableQrCodeANDROID

14.14.7. New Structures

- XrTrackableQrCodeANDROID

- Extending XrSystemProperties:
  - XrSystemQrCodeTrackingPropertiesANDROID
- Extending XrTrackableTrackerCreateInfoANDROID:
  - XrTrackableQrCodeConfigurationANDROID

14.14.8. New Enums

- XrQrCodeTrackingModeANDROID

14.14.9. New Enum Constants

- `XR_ANDROID_TRACKABLES_QR_CODE_EXTENSION_NAME`
- `XR_ANDROID_trackables_qr_code_SPEC_VERSION`
- Extending XrStructureType:
  - `XR_TYPE_SYSTEM_QR_CODE_TRACKING_PROPERTIES_ANDROID`
  - `XR_TYPE_TRACKABLE_QR_CODE_ANDROID`
  - `XR_TYPE_TRACKABLE_QR_CODE_CONFIGURATION_ANDROID`
- Extending XrTrackableTypeANDROID:
  - `XR_TRACKABLE_TYPE_QR_CODE_ANDROID`

14.14.10. Issues

14.14.11. Version History

- Revision 1, 2025-02-05 (Levana Chen)
  - Initial extension description.

14.15. XR_BD_controller_interaction

Name String

`XR_BD_controller_interaction`

Extension Type

Instance extension

Registered Extension Number

385

Revision

2

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

API Interactions

- Interacts with `XR_EXT_dpad_binding`
- Interacts with `XR_EXT_hand_interaction`
- Interacts with `XR_EXT_palm_pose`

Deprecation State

- *Promoted* to OpenXR 1.1

Last Modified Date

2023-08-10

IP Status*

No known IP claims.

Contributors

Baolin Fu, ByteDance
Shanliang Xu, ByteDance
Zhanrui Jia, ByteDance

Overview

This extension defines the interaction profile for PICO Neo3, PICO 4, and PICO G3 Controllers.

BD(ByteDance) Controller interaction profile

Interaction profile path for PICO Neo3:

- */interaction_profiles/bytedance/pico_neo3_controller*

> **Note**
>
> The interaction profile path */interaction_profiles/bytedance/pico_neo3_controller* defined here does not follow current rules for interaction profile names. If this extension were introduced today, it would be called */interaction_profiles/bytedance/pico_neo3_controller_bd*, to allow for modifications when promoted to a KHR extension or the core specification.

Interaction profile path for PICO 4:

- */interaction_profiles/bytedance/pico4_controller*

> **Note**
>
> The interaction profile path */interaction_profiles/bytedance/pico4_controller* defined here does not follow current rules for interaction profile names. If this extension were introduced today, it would be called */interaction_profiles/bytedance/pico4_controller_bd*, to allow for modifications when promoted to a KHR extension or the core specification.

Interaction profile path for PICO G3:

- */interaction_profiles/bytedance/pico_g3_controller*

> **Note**
>
> The interaction profile path */interaction_profiles/bytedance/pico_g3_controller* defined here does not follow current rules for interaction profile names. If this extension were introduced today, it would be called */interaction_profiles/bytedance/pico_g3_controller_bd*, to allow for modifications when promoted to a KHR extension or the core specification.

Valid for user paths for pico_neo3_controller, pico4_controller, and pico_g3_controller:

- */user/hand/left*
- */user/hand/right*

Supported component paths for pico_neo3_controller:

- On /user/hand/left only:
  - *.../input/x/click*
  - *.../input/x/touch*
  - *.../input/y/click*
  - *.../input/y/touch*
- On /user/hand/right only:
  - *.../input/a/click*
  - *.../input/a/touch*
  - *.../input/b/click*
  - *.../input/b/touch*
- *.../input/menu/click*
- *.../input/system/click* (**may** not be available for application use)

- *.../input/trigger/click*
- *.../input/trigger/value*
- *.../input/trigger/touch*
- *.../input/thumbstick/y*
- *.../input/thumbstick/x*
- *.../input/thumbstick/click*
- *.../input/thumbstick/touch*
- *.../input/squeeze/click*
- *.../input/squeeze/value*
- *.../input/grip/pose*
- *.../input/aim/pose*
- *.../output/haptic*

Supported component paths for pico4_controller:

- On /user/hand/left only:
  - *.../input/x/click*
  - *.../input/x/touch*
  - *.../input/y/click*
  - *.../input/y/touch*
  - *.../input/menu/click*
- On /user/hand/right only:
  - *.../input/a/click*
  - *.../input/a/touch*
  - *.../input/b/click*
  - *.../input/b/touch*
- *.../input/system/click* (**may** not be available for application use)
- *.../input/trigger/click*
- *.../input/trigger/value*
- *.../input/trigger/touch*
- *.../input/thumbstick/y*
- *.../input/thumbstick/x*
- *.../input/thumbstick/click*
- *.../input/thumbstick/touch*

- *.../input/squeeze/click*
- *.../input/squeeze/value*
- *.../input/grip/pose*
- *.../input/aim/pose*
- *.../output/haptic*

Supported component paths for pico_g3_controller:

- *.../input/trigger/click*
- *.../input/trigger/value*
- *.../input/menu/click*
- *.../input/grip/pose*
- *.../input/aim/pose*
- *.../input/thumbstick*
- *.../input/thumbstick/click*

Be careful with the following difference:

- pico_neo3_controller supports *.../input/menu/click* both on */user/hand/left* and */user/hand/right*.
- pico4_controller supports *.../input/menu/click* only on */user/hand/left*.
- pico_g3_controller has only one physical controller. When designing suggested bindings for this interaction profile, you **may** suggest bindings for both */user/hand/left* and */user/hand/right*. However, only one of them will be active at a given time, so do not design interactions that require simultaneous use of both hands.



Note

When the runtime supports `XR_VERSION_1_1` and use of OpenXR 1.1 is requested by the application, this interaction profile **must** also support

- *.../input/grip_surface/pose*



Note

When the `XR_KHR_maintenance1` extension is available and enabled, this interaction profile **must** also support

- *.../input/grip_surface/pose*



Note

When the `XR_EXT_palm_pose` extension is available and enabled, this interaction profile **must** also support

- *.../input/palm_ext/pose*



Note

When the `XR_EXT_hand_interaction` extension is available and enabled, this interaction profile **must** also support

- *.../input/pinch_ext/pose*
- *.../input/poke_ext/pose*

New Object Types

New Flag Types

New Enum Constants

New Enums

New Structures

New Functions

Issues

Version History

- Revision 1, 2023-01-04 (Baolin Fu)
  - Initial extension description
- Revision 2, 2023-08-10 (Shanliang Xu)
  - Add support for G3 devices

14.16. XR_FB_space_warp

Name String

`XR_FB_space_warp`

Extension Type

Instance extension

Registered Extension Number

172

Revision

2

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Deprecation State

- *Deprecated* by `XR_EXT_frame_synthesis` extension

Contributors

Jian Zhang, Facebook
Neel Bedekar, Facebook
Xiang Wei, Facebook

Overview

This extension provides support to enable space warp technology on application. By feeding application generated motion vector and depth buffer images, the runtime can do high quality frame extrapolation and reprojection, allow applications to run at half fps but still providing smooth experience to users.



Note

This extension is independent of `XR_KHR_composition_layer_depth`, and both may be enabled and used at the same time, for different purposes. The XrCompositionLayerSpaceWarpInfoFB::`depthSubImage` depth data is dedicated for space warp, and its resolution is usually lower than XrCompositionLayerDepthInfoKHR::`subImage`. See XrSystemSpaceWarpPropertiesFB for suggested resolution of `depthSubImage`.

New Flag Types

```
// Provided by XR_FB_space_warp
typedef XrFlags64 XrCompositionLayerSpaceWarpInfoFlagsFB;
```

```
// Provided by XR_FB_space_warp
// Flag bits for XrCompositionLayerSpaceWarpInfoFlagsFB
static const XrCompositionLayerSpaceWarpInfoFlagsFB
XR_COMPOSITION_LAYER_SPACE_WARP_INFO_FRAME_SKIP_BIT_FB = 0x00000001;
```

Flag Descriptions

- XR_COMPOSITION_LAYER_SPACE_WARP_INFO_FRAME_SKIP_BIT_FB requests that the runtime skips space warp frame extrapolation for a particular frame. This can be used when the application has better knowledge the particular frame will be not a good fit for space warp frame extrapolation.

New Enum Constants

XrStructureType enumeration is extended with:

- XR_TYPE_COMPOSITION_LAYER_SPACE_WARP_INFO_FB
- XR_TYPE_SYSTEM_SPACE_WARP_PROPERTIES_FB

New Enums

- XR_COMPOSITION_LAYER_SPACE_WARP_INFO_FRAME_SKIP_BIT_FB

New Structures

When submitting motion vector buffer and depth buffers along with projection layers, add an XrCompositionLayerSpaceWarpInfoFB structure to the XrCompositionLayerProjectionView::next chain, for each XrCompositionLayerProjectionView structure in the given layer.

The XrCompositionLayerSpaceWarpInfoFB structure is defined as:

```
// Provided by XR_FB_space_warp
typedef struct XrCompositionLayerSpaceWarpInfoFB {
    XrStructureType                            type;
    const void*                                next;
    XrCompositionLayerSpaceWarpInfoFlagsFB     layerFlags;
    XrSwapchainSubImage                        motionVectorSubImage;
    XrPosef                                    appSpaceDeltaPose;
    XrSwapchainSubImage                        depthSubImage;
    float                                      minDepth;
    float                                      maxDepth;
    float                                      nearZ;
    float                                      farZ;
} XrCompositionLayerSpaceWarpInfoFB;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `layerFlags` is a bitmask of XrCompositionLayerSpaceWarpInfoFlagsFB.
- `motionVectorSubImage` identifies the motion vector image XrSwapchainSubImage to be associated with the submitted layer XrCompositionLayerProjection.
- `appSpaceDeltaPose` is the incremental application-applied transform, if any, since the previous frame that affects the view. When artificial locomotion (scripted movement, teleportation, etc.) happens, the application might transform the whole XrCompositionLayerProjection::space from one application space pose to another pose between frames. The pose should be identity when there is no XrCompositionLayerProjection::space transformation in application.
- `depthSubImage` identifies the depth image XrSwapchainSubImage to be associated with motionVectorSubImage. The swapchain should be created with `XR_SWAPCHAIN_USAGE_SAMPLED_BIT` | `XR_SWAPCHAIN_USAGE_DEPTH_STENCIL_ATTACHMENT_BIT`.
- `minDepth` and `maxDepth` are the range of depth values the depth swapchain could have, in the range of [0.0,1.0]. This is akin to min and max values of OpenGL's `glDepthRange`, but with the requirement here that maxDepth ≥ minDepth.
- `nearZ` is the positive distance in meters of the `minDepth` value in the depth swapchain. Applications **may** use a `nearZ` that is greater than `farZ` to indicate depth values are reversed. `nearZ` can be infinite.
- `farZ` is the positive distance in meters of the `maxDepth` value in the depth swapchain. `farZ` can be infinite.

The motion vector data is stored in the `motionVectorSubImage`'s RGB channels, defined in NDC (normalized device coordinates) space, for example, the same surface point's NDC is PrevNDC in previous frame, CurrNDC in current frame, then the motion vector value is "highp vec3 motionVector = (CurrNDC - PrevNDC).xyz;". Signed 16 bit float pixel format is recommended for this image.

The runtime **must** return error `XR_ERROR_VALIDATION_FAILURE` if `nearZ` == `farZ`.

Valid Usage (Implicit)

- The XR_FB_space_warp extension **must** be enabled prior to using XrCompositionLayerSpaceWarpInfoFB
- type **must** be XR_TYPE_COMPOSITION_LAYER_SPACE_WARP_INFO_FB
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- layerFlags **must** be 0 or a valid combination of XrCompositionLayerSpaceWarpInfoFlagBitsFB values
- motionVectorSubImage **must** be a valid XrSwapchainSubImage structure
- depthSubImage **must** be a valid XrSwapchainSubImage structure

When this extension is enabled, an application **can** pass in an XrSystemSpaceWarpPropertiesFB structure in the XrSystemProperties::next chain when calling xrGetSystemProperties to acquire information about recommended motion vector buffer resolution. The XrSystemSpaceWarpPropertiesFB structure is defined as:

```
// Provided by XR_FB_space_warp
typedef struct XrSystemSpaceWarpPropertiesFB {
    XrStructureType    type;
    void*              next;
    uint32_t           recommendedMotionVectorImageRectWidth;
    uint32_t           recommendedMotionVectorImageRectHeight;
} XrSystemSpaceWarpPropertiesFB;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain.
- recommendedMotionVectorImageRectWidth: recommended motion vector and depth image width
- recommendedMotionVectorImageRectHeight: recommended motion vector and depth image height

Valid Usage (Implicit)

- The XR_FB_space_warp extension **must** be enabled prior to using XrSystemSpaceWarpPropertiesFB
- type **must** be XR_TYPE_SYSTEM_SPACE_WARP_PROPERTIES_FB
- next **must** be NULL or a valid pointer to the next structure in a structure chain

Issues

Version History

- Revision 1, 2021-08-04 (Jian Zhang)
  - Initial extension description
- Revision 2, 2022-02-07 (Jian Zhang)
  - Add XR_COMPOSITION_LAYER_SPACE_WARP_INFO_FRAME_SKIP_BIT_FB

14.17. XR_FB_touch_controller_pro

Name String

XR_FB_touch_controller_pro

Extension Type

Instance extension

Registered Extension Number

168

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

API Interactions

- Interacts with XR_EXT_dpad_binding
- Interacts with XR_EXT_hand_interaction
- Interacts with XR_EXT_palm_pose

Deprecation State

- *Promoted* to OpenXR 1.1

Last Modified Date

2022-06-29

IP Status

No known IP claims.

Contributors

Aanchal Dalmia, Meta
Adam Bengis, Meta
Tony Targonski, Meta
Federico Schliemann, Meta

Overview

This extension defines a new interaction profile for the Meta Quest Touch Pro Controller.

Meta Quest Touch Pro Controller Profile Path:

- */interaction_profiles/facebook/touch_controller_pro*

> **Note**
>
> The interaction profile path */interaction_profiles/facebook/touch_controller_pro* defined here does not follow current rules for interaction profile names. If this extension were introduced today, it would be called */interaction_profiles/facebook/touch_controller_pro_fb*, to allow for modifications when promoted to a KHR extension or the core specification.

Valid for user paths:

- */user/hand/left*
- */user/hand/right*

This interaction profile provides inputs and outputs that are a superset of those available in the existing "Oculus Touch Controller" interaction profile:

- */interaction_profiles/oculus/touch_controller*

Supported component paths (Note that the paths which are marked as 'new' are enabled by Meta Quest Touch Pro Controller profile exclusively):

- On */user/hand/left* only:
  - *.../input/x/click*

    - *.../input/x/touch*
    - *.../input/y/click*
    - *.../input/y/touch*
    - *.../input/menu/click*
- On */user/hand/right* only:
    - *.../input/a/click*
    - *.../input/a/touch*
    - *.../input/b/click*
    - *.../input/b/touch*
    - *.../input/system/click* (**may** not be available for application use)
- On both:
    - *.../input/squeeze/value*
    - *.../input/trigger/value*
    - *.../input/trigger/touch*
    - *.../input/thumbstick*
    - *.../input/thumbstick/x*
    - *.../input/thumbstick/y*
    - *.../input/thumbstick/click*
    - *.../input/thumbstick/touch*
    - *.../input/thumbrest/touch*
    - *.../input/grip/pose*
    - *.../input/aim/pose*
    - *.../output/haptic*
    - *.../input/thumbrest/force* (new)
    - *.../input/stylus_fb/force* (new)
    - *.../input/trigger/curl_fb* (new)
    - *.../input/trigger/slide_fb* (new)
    - *.../input/trigger/proximity_fb* (new)
    - *.../input/thumb_fb/proximity_fb* (new)
    - *.../output/haptic_trigger_fb* (new)
    - *.../output/haptic_thumb_fb* (new)



Note

When the runtime supports `XR_VERSION_1_1` and use of OpenXR 1.1 is requested by the application, this interaction profile **must** also support

- *.../input/grip_surface/pose*



Note

When the `XR_KHR_maintenance1` extension is available and enabled, this interaction profile **must** also support

- *.../input/grip_surface/pose*



Note

When the `XR_EXT_palm_pose` extension is available and enabled, this interaction profile **must** also support

- *.../input/palm_ext/pose*



Note

When the `XR_EXT_hand_interaction` extension is available and enabled, this interaction profile **must** also support

- *.../input/pinch_ext/pose*
- *.../input/poke_ext/pose*

New Identifiers

- **stylus_fb**: Meta Quest Touch Pro Controller adds an optional stylus tip that can be interchanged with the lanyard. This tip can detect various pressure levels and could be used for writing or drawing.
- **thumb_fb**: Meta Quest Touch Pro Controller adds a 1-dimensional analog input value for the thumb. This is similar to other triggers on the controller like the fore trigger for the index finger and grip trigger for the middle finger.

Input Path Descriptions

- **/input/thumbrest/force** : Allow developers to access the normalized 1D force value associated with the thumb ranging from 0-6 Newtons: 0 = not pressed, 1 = fully pressed
- **/input/stylus_fb/force** : Allow developers to access the normalized 1D force value associated with the stylus ranging from ~0-2 Newtons: 0 = not pressed, 1 = fully pressed
- **/input/trigger/curl_fb** : This represents how pointed or curled the user's finger is on the trigger: 0 = fully pointed, 1 = finger flat on surface
- **/input/trigger/slide_fb**: This represents how far the user is sliding their index finger along the surface of the trigger: 0 = finger flat on the surface, 1 = finger fully drawn back
- **/input/trigger/proximity_fb** : Bit indicating whether the user's index finger is near the trigger
- **/input/thumb_fb/proximity_fb** : Bit indicating the user's thumb is near the touchpad

Output Path Descriptions

In addition to the VCM motor, Meta Quest Touch Pro Controller has two localized LRA haptics elements located in the fore trigger and under the touchpad.

- **/output/haptic_trigger_fb** represents the path to the haptic element in the trigger
- **/output/haptic_thumb_fb** represents the path to the haptic element under the touchpad

Version History

- Revision 1, 2022-06-29 (Aanchal Dalmia)
  - Initial extension proposal

14.18. XR_FB_touch_controller_proximity

Name String

`XR_FB_touch_controller_proximity`

Extension Type

Instance extension

Registered Extension Number

207

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Deprecation State

- *Obsoleted* by OpenXR 1.1

Last Modified Date

2022-09-12

IP Status

No known IP claims.

Contributors

Tony Targonski, Meta Platforms
Aanchal Dalmia, Meta Platforms
Andreas Loeve Selvik, Meta Platforms
John Kearney, Meta Platforms
James Hillery, Meta Platforms

14.18.1. Overview

This extension introduces a new component path, proximity_fb, and adds support for it for the */interaction_profiles/oculus/touch_controller* interaction profile.

14.18.2. New Interaction Profile Component Paths

- proximity_fb - The user is in physical proximity of input source. This **may** be present for any kind of input source representing a physical component, such as a button, if the device includes the necessary sensor. The state of a "proximity_fb" component **must** be `XR_TRUE` if the same input source is returning `XR_TRUE` for either a "touch" or any other component that implies physical contact. The runtime **may** return `XR_TRUE` for "proximity_fb" when "touch" returns `XR_FALSE`. This indicate that the user is hovering just above, but not touching the input source in question. "proximity_fb" components are always boolean.

14.18.3. Interaction Profile Changes

Interaction profile: */interaction_profiles/oculus/touch_controller*

Additional supported component paths for the above profile enabled by this extension:

Valid for user paths:

- */user/hand/left*

- */user/hand/right*

On both:

- *.../input/trigger/proximity_fb* This represents whether the user is in proximity of the trigger button, usually with their index finger.
- *.../input/thumb_fb/proximity_fb* This represents whether the user is in proximity of the input sources at the top of the controller, usually with their thumb.

14.18.4. Example code

The following example code demonstrates detecting when a user lifts their finger off the trigger button.

```
XrInstance instance;           // previously initialized
XrSession session;             // previously initialized
XrActionSet inGameActionSet;   // previously initialized

XrAction indexProximityAction; // previously initialized
XrAction indexTouchAction;     // previously initialized

//  ----------
//  Bind actions to trigger/proximity_fb and trigger/touch
//  ----------

XrPath indexProximityPath, indexTouchPath{XR_NULL_PATH};
// New component exposed by this extension:
CHK_XR(xrStringToPath(instance, "/user/hand/right/input/trigger/proximity_fb",
&indexProximityPath));
// Existing component that is useful together with proximity_fb
CHK_XR(xrStringToPath(instance, "/user/hand/right/input/trigger/touch", &indexTouchPath))

XrPath interactionProfilePath{XR_NULL_PATH};
CHK_XR(xrStringToPath(instance, "/interaction_profiles/oculus/touch_controller",
&interactionProfilePath));

XrActionSuggestedBinding bindings[2];
bindings[0].action = indexProximityAction;
bindings[0].binding = indexProximityPath;
bindings[1].action = indexTouchAction;
bindings[1].binding = indexTouchPath;

XrInteractionProfileSuggestedBinding suggestedBindings{
  .type = XR_TYPE_INTERACTION_PROFILE_SUGGESTED_BINDING,
  .interactionProfile = interactionProfilePath,
  .countSuggestedBindings = 2,
```

```
  .suggestedBindings = bindings,
};
CHK_XR(xrSuggestInteractionProfileBindings(instance, &suggestedBindings));

//  ----------
//  Application main loop
//  ----------

while (1)
{
    // ...

    //  ----------
    //  Query input state
    //  ----------

    XrActionStateBoolean indexTouchState{
        .type = XR_TYPE_ACTION_STATE_BOOLEAN,
    };
    XrActionStateBoolean indexProximityState{
        .type = XR_TYPE_ACTION_STATE_BOOLEAN,
    };
    XrActionStateGetInfo getInfo{
        .type = XR_TYPE_ACTION_STATE_GET_INFO,
    };

    getInfo.action = indexTouchAction;
    CHK_XR(xrGetActionStateBoolean(session, &getInfo, &indexTouchState));
    getInfo.action = indexProximityAction;
    CHK_XR(xrGetActionStateBoolean(session, &getInfo, &indexProximityState));

    //  ----------
    //  Proximity and touch logic
    //  ----------

    // There are only three valid combinations of the proximity and touch values
    if (!indexProximityState.currentState)
    {
        // Index is not in proximity of the trigger button (they might be pointing!)
        // Implies that TouchState.currentState == XR_FALSE
    }
    if (indexProximityState.currentState && !indexTouchState.currentState)
    {
        // Index finger of user is in proximity of, but not touching, the trigger button
        // i.e. they are hovering above the button
    }
    if (indexTouchState.currentState)
    {
```

```
        // Index finger of user is touching the trigger button
        // Implies that ProximityState.currentState == XR_TRUE
    }
}
```

New Object Types

New Flag Types

New Enum Constants

New Enums

New Structures

New Object Types

New Flag Types

New Enum Constants

New Enums

New Structures

Version History

- Revision 1, 2022-09-12 (Andreas Loeve Selvik)
  - Initial extension proposal

14.19. XR_HTC_vive_cosmos_controller_interaction

Name String

`XR_HTC_vive_cosmos_controller_interaction`

Extension Type

Instance extension

Registered Extension Number

103

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

API Interactions

- Interacts with `XR_EXT_dpad_binding`
- Interacts with `XR_EXT_hand_interaction`
- Interacts with `XR_EXT_palm_pose`

Deprecation State

- *Promoted* to OpenXR 1.1

Last Modified Date

2020-09-28

IP Status

No known IP claims.

Contributors

Chris Kuo, HTC
Kyle Chen, HTC

Overview

This extension defines a new interaction profile for the VIVE Cosmos Controller.

VIVE Cosmos Controller interaction profile

Interaction profile path:

- */interaction_profiles/htc/vive_cosmos_controller*

> **Note**
>
> The interaction profile path */interaction_profiles/htc/vive_cosmos_controller* defined here does not follow current rules for interaction profile names. If this extension were introduced today, it would be called */interaction_profiles/htc/vive_cosmos_controller_htc*, to allow for modifications when promoted to a KHR extension or the core specification.

Valid for user paths:

- */user/hand/left*
- */user/hand/right*

This interaction profile represents the input sources and haptics on the VIVE Cosmos Controller.

Supported component paths:

- On */user/hand/left* only:
  - *.../input/x/click*
  - *.../input/y/click*
  - *.../input/menu/click*
- On */user/hand/right* only:
  - *.../input/a/click*
  - *.../input/b/click*
  - *.../input/system/click* (**may** not be available for application use)
- *.../input/shoulder/click*
- *.../input/squeeze/click*
- *.../input/trigger/click*
- *.../input/trigger/value*
- *.../input/thumbstick/x*
- *.../input/thumbstick/y*
- *.../input/thumbstick/click*
- *.../input/thumbstick/touch*
- *.../input/grip/pose*
- *.../input/aim/pose*
- *.../output/haptic*



Note

When the runtime supports `XR_VERSION_1_1` and use of OpenXR 1.1 is requested by the application, this interaction profile **must** also support

- *.../input/grip_surface/pose*



Note

When the `XR_KHR_maintenance1` extension is available and enabled, this interaction profile **must** also support

- *.../input/grip_surface/pose*



Note

When the `XR_EXT_palm_pose` extension is available and enabled, this interaction profile **must** also support

- *.../input/palm_ext/pose*



Note

When the `XR_EXT_hand_interaction` extension is available and enabled, this interaction profile **must** also support

- *.../input/pinch_ext/pose*
- *.../input/poke_ext/pose*

New Object Types

New Flag Types

New Enum Constants

New Enums

New Structures

New Functions

Issues

Version History

- Revision 1, 2020-09-28 (Chris Kuo)
  - Initial extension description

14.20. XR_HTC_vive_focus3_controller_interaction

Name String

`XR_HTC_vive_focus3_controller_interaction`

Extension Type

Instance extension

Registered Extension Number

106

Revision

2

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

API Interactions

- Interacts with `XR_EXT_dpad_binding`
- Interacts with `XR_EXT_hand_interaction`
- Interacts with `XR_EXT_palm_pose`

Deprecation State

- *Promoted* to OpenXR 1.1

Last Modified Date

2022-04-29

IP Status

No known IP claims.

Contributors

Ria Hsu, HTC

Overview

This extension defines a new interaction profile for the VIVE Focus 3 Controller.

VIVE Focus 3 Controller interaction profile

Interaction profile path:

- */interaction_profiles/htc/vive_focus3_controller*

> **Note**
>
> The interaction profile path */interaction_profiles/htc/vive_focus3_controller* defined here does not follow current rules for interaction profile names. If this extension were introduced today, it would be called */interaction_profiles/htc/vive_focus3_controller_htc*, to allow for modifications when promoted to a KHR extension or the core specification.

Valid for user paths:

- */user/hand/left*
- */user/hand/right*

This interaction profile represents the input sources and haptics on the VIVE Focus 3 Controller.

Supported component paths:

- On */user/hand/left* only:
  - *.../input/x/click*
  - *.../input/y/click*
  - *.../input/menu/click*
- On */user/hand/right* only:
  - *.../input/a/click*
  - *.../input/b/click*
  - *.../input/system/click* (**may** not be available for application use)
- *.../input/squeeze/click*
- *.../input/squeeze/touch*
- *.../input/squeeze/value*
- *.../input/trigger/click*
- *.../input/trigger/touch*
- *.../input/trigger/value*
- *.../input/thumbstick/x*
- *.../input/thumbstick/y*
- *.../input/thumbstick/click*
- *.../input/thumbstick/touch*
- *.../input/thumbrest/touch*
- *.../input/grip/pose*
- *.../input/aim/pose*
- *.../output/haptic*



Note

When the runtime supports `XR_VERSION_1_1` and use of OpenXR 1.1 is requested by the application, this interaction profile **must** also support

- *.../input/grip_surface/pose*



Note

When the `XR_KHR_maintenance1` extension is available and enabled, this interaction profile **must** also support

- *.../input/grip_surface/pose*



Note

When the `XR_EXT_palm_pose` extension is available and enabled, this interaction profile **must** also support

- *.../input/palm_ext/pose*



Note

When the `XR_EXT_hand_interaction` extension is available and enabled, this interaction profile **must** also support

- *.../input/pinch_ext/pose*
- *.../input/poke_ext/pose*

New Object Types

New Flag Types

New Enum Constants

New Enums

New Structures

New Functions

Issues

Version History

- Revision 1, 2022-01-03 (Ria Hsu)
  - Initial extension description
- Revision 2, 2022-04-29 (Ria Hsu)
  - Support component path "/input/squeeze/value"

14.21. XR_META_touch_controller_plus

Name String

`XR_META_touch_controller_plus`

Extension Type

Instance extension

Registered Extension Number

280

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

API Interactions

- Interacts with `XR_EXT_dpad_binding`
- Interacts with `XR_EXT_hand_interaction`
- Interacts with `XR_EXT_palm_pose`

Deprecation State

- *Promoted* to OpenXR 1.1

Last Modified Date

2023-04-10

IP Status

No known IP claims.

Contributors

Aanchal Dalmia, Meta Platforms
Adam Bengis, Meta Platforms

Overview

This extension defines a new interaction profile for the Meta Quest Touch Plus Controller.

Meta Quest Touch Plus Controller interaction profile path:

- */interaction_profiles/meta/touch_controller_plus*

Valid for user paths:

- */user/hand/left*
- */user/hand/right*

This interaction profile provides inputs and outputs that are a superset of those available in the existing "Oculus Touch Controller" interaction profile, */interaction_profiles/oculus/touch_controller*

Supported component paths:

- On */user/hand/left* only:
  - *.../input/x/click*
  - *.../input/x/touch*
  - *.../input/y/click*
  - *.../input/y/touch*
  - *.../input/menu/click*
- On */user/hand/right* only:
  - *.../input/a/click*
  - *.../input/a/touch*
  - *.../input/b/click*
  - *.../input/b/touch*
  - *.../input/system/click* (**may** not be available for application use)
- On both:
  - *.../input/squeeze/value*
  - *.../input/trigger/value*
  - *.../input/trigger/touch*
  - *.../input/thumbstick*
  - *.../input/thumbstick/x*
  - *.../input/thumbstick/y*
  - *.../input/thumbstick/click*
  - *.../input/thumbstick/touch*
  - *.../input/thumbrest/touch*
  - *.../input/grip/pose*
  - *.../input/aim/pose*
  - *.../output/haptic*
  - *.../input/thumb_meta/proximity_meta*
  - *.../input/trigger/proximity_meta*
  - *.../input/trigger/curl_meta*
  - *.../input/trigger/slide_meta*

- */input/trigger/force*



Note

When the runtime supports `XR_VERSION_1_1` and use of OpenXR 1.1 is requested by the application, this interaction profile **must** also support

- */input/grip_surface/pose*



Note

When the `XR_KHR_maintenance1` extension is available and enabled, this interaction profile **must** also support

- */input/grip_surface/pose*



Note

When the `XR_EXT_palm_pose` extension is available and enabled, this interaction profile **must** also support

- */input/palm_ext/pose*



Note

When the `XR_EXT_hand_interaction` extension is available and enabled, this interaction profile **must** also support

- */input/pinch_ext/pose*
- */input/poke_ext/pose*

New Identifiers

- **thumb_meta**: Meta Quest Touch Plus Controller adds an input identifier for the user's thumb on the same hand currently holding the controller. Thumb input is not explicitly bound to any location on the controller.

Input Path Descriptions

- **/input/thumb_meta/proximity_meta** : Boolean indicating the user's thumb is near the inputs on the top face of the controller.
- **/input/trigger/proximity_meta** : Boolean indicating whether the user's index finger is near the trigger.
- **/input/trigger/curl_meta** : Float representing how pointed or curled the user's index finger is on the trigger: 0.0 = fully pointed, 1.0 = finger flat on the surface
- **/input/trigger/slide_meta** : Float representing how far the user is sliding the tip of their index finger along the surface of the trigger: 0.0 = finger flat on the surface, 1.0 = finger fully drawn back.
- **/input/trigger/force** : Float representing the amount of force being applied by the user to the trigger after it reaches the end of the range of travel: 0.0 = no additional pressure applied, 1.0 = maximum detectable pressure applied.

Version History

- Revision 1, 2023-04-10 (Adam Bengis)
  - Initial extension proposal

14.22. XR_ML_ml2_controller_interaction

Name String

`XR_ML_ml2_controller_interaction`

Extension Type

Instance extension

Registered Extension Number

135

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

API Interactions

- Interacts with `XR_EXT_dpad_binding`
- Interacts with `XR_EXT_hand_interaction`

- Interacts with `XR_EXT_palm_pose`

Deprecation State

- *Promoted* to OpenXR 1.1

Last Modified Date

2022-07-22

IP Status

No known IP claims.

Contributors

Ron Bessems, Magic Leap
Rafael Wiltz, Magic Leap

Overview

This extension defines the interaction profile for the Magic Leap 2 Controller.

Magic Leap 2 Controller interaction profile

This interaction profile represents the input sources and haptics on the Magic Leap 2 Controller.

Interaction profile path:

- */interaction_profiles/ml/ml2_controller*

> **Note**
>
> The interaction profile path */interaction_profiles/ml/ml2_controller* defined here does not follow current rules for interaction profile names. If this extension were introduced today, it would be called */interaction_profiles/ml/ml2_controller_ml*, to allow for modifications when promoted to a KHR extension or the core specification.

Valid for user paths:

- */user/hand/left*
- */user/hand/right*

Supported component paths:

- *.../input/menu/click*
- *.../input/home/click* (**may** not be available for application use)
- *.../input/trigger/click*

- *.../input/trigger/value*
- *.../input/trackpad/y*
- *.../input/trackpad/x*
- *.../input/trackpad/click*
- *.../input/trackpad/force*
- *.../input/trackpad/touch*
- *.../input/grip/pose*
- *.../input/aim/pose*
- *.../input/shoulder/click*
- *.../output/haptic*



Note

When the runtime supports `XR_VERSION_1_1` and use of OpenXR 1.1 is requested by the application, this interaction profile **must** also support

- *.../input/grip_surface/pose*



Note

When the `XR_KHR_maintenance1` extension is available and enabled, this interaction profile **must** also support

- *.../input/grip_surface/pose*



Note

When the `XR_EXT_palm_pose` extension is available and enabled, this interaction profile **must** also support

- *.../input/palm_ext/pose*



Note

When the `XR_EXT_hand_interaction` extension is available and enabled, this interaction profile **must** also support

- *.../input/pinch_ext/pose*
- *.../input/poke_ext/pose*

New Object Types

New Flag Types

New Enum Constants

New Enums

New Structures

New Functions

Issues

Version History

- Revision 1, 2022-07-22 (Ron Bessems)
  - Initial extension description

14.23. XR_MND_swapchain_usage_input_attachment_bit

Name String

`XR_MND_swapchain_usage_input_attachment_bit`

Extension Type

Instance extension

Registered Extension Number

97

Revision

2

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Deprecation State

- *Deprecated* by `XR_KHR_swapchain_usage_input_attachment_bit` extension

Last Modified Date

2020-07-24

IP Status

No known IP claims.

Contributors

Jakob Bornecrantz, Collabora

Overview

This extension enables an application to specify that swapchain images should be created in a way so that they can be used as input attachments. At the time of writing this bit only affects Vulkan swapchains.

New Object Types

New Flag Types

New Enum Constants

XrSwapchainUsageFlagBits enumeration is extended with:

- `XR_SWAPCHAIN_USAGE_INPUT_ATTACHMENT_BIT_MND`

New Enums

New Structures

New Functions

Issues

Version History

- Revision 1, 2020-07-23 (Jakob Bornecrantz)
  - Initial draft
- Revision 2, 2020-07-24 (Jakob Bornecrantz)
  - Added note about only affecting Vulkan
  - Changed from MNDX to MND

14.24. XR_MSFT_composition_layer_reprojection

Name String

`XR_MSFT_composition_layer_reprojection`

Extension Type

Instance extension

Registered Extension Number

67

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Deprecation State

- *Deprecated* without replacement

Last Modified Date

2020-06-20

IP Status

No known IP claims.

Contributors

Zonglin Wu, Microsoft
Bryce Hutchings, Microsoft
Alex Turner, Microsoft
Yin Li, Microsoft

Overview

This extension enables an application to provide additional reprojection information for a projection composition layer to help the runtime produce better hologram stability and visual quality.

First, the application uses xrEnumerateReprojectionModesMSFT to inspect what reprojection mode the view configuration supports.

The xrEnumerateReprojectionModesMSFT function returns the supported reprojection modes of the view configuration.

```
// Provided by XR_MSFT_composition_layer_reprojection
XrResult xrEnumerateReprojectionModesMSFT(
    XrInstance                                  instance,
    XrSystemId                                  systemId,
    XrViewConfigurationType                     viewConfigurationType,
    uint32_t                                    modeCapacityInput,
    uint32_t*                                   modeCountOutput,
    XrReprojectionModeMSFT*                     modes);
```

Parameter Descriptions

- `instance` is the instance from which `systemId` was retrieved.
- `systemId` is the `XrSystemId` whose reprojection modes will be enumerated.
- `viewConfigurationType` is the XrViewConfigurationType to enumerate.
- `modeCapacityInput` is the capacity of the array, or 0 to indicate a request to retrieve the required capacity.
- `modeCountOutput` is a pointer to the count of the array, or a pointer to the required capacity in the case that `modeCapacityInput` is insufficient.
- `modes` is a pointer to an application-allocated array that will be filled with the XrReprojectionModeMSFT values that are supported by the runtime. It **can** be `NULL` if `modeCapacityInput` is 0.
- See the Buffer Size Parameters section for a detailed description of retrieving the required `modes` size.

Valid Usage (Implicit)

- The `XR_MSFT_composition_layer_reprojection` extension **must** be enabled prior to calling xrEnumerateReprojectionModesMSFT
- `instance` **must** be a valid XrInstance handle
- `viewConfigurationType` **must** be a valid XrViewConfigurationType value
- `modeCountOutput` **must** be a pointer to a `uint32_t` value
- If `modeCapacityInput` is not `0`, `modes` **must** be a pointer to an array of `modeCapacityInput` XrReprojectionModeMSFT values

Return Codes

Success

- XR_SUCCESS

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SIZE_INSUFFICIENT
- XR_ERROR_SYSTEM_INVALID
- XR_ERROR_VALIDATION_FAILURE
- XR_ERROR_VIEW_CONFIGURATION_TYPE_UNSUPPORTED

A system **may** support different sets of reprojection modes for different view configuration types.

Then, the application **can** provide reprojection mode for the projection composition layer to inform the runtime that the XR experience **may** benefit from the provided reprojection mode.

An XrCompositionLayerReprojectionInfoMSFT structure **can** be added to the `next` chain of XrCompositionLayerProjection structure when calling xrEndFrame.

```
// Provided by XR_MSFT_composition_layer_reprojection
typedef struct XrCompositionLayerReprojectionInfoMSFT {
    XrStructureType           type;
    const void*               next;
    XrReprojectionModeMSFT    reprojectionMode;
} XrCompositionLayerReprojectionInfoMSFT;
```

Parameter Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `reprojectionMode` is an XrReprojectionModeMSFT enum providing a hint to the reprojection mode to the corresponding projection layer.

Valid Usage (Implicit)

- The XR_MSFT_composition_layer_reprojection extension **must** be enabled prior to using XrCompositionLayerReprojectionInfoMSFT
- type **must** be XR_TYPE_COMPOSITION_LAYER_REPROJECTION_INFO_MSFT
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- reprojectionMode **must** be a valid XrReprojectionModeMSFT value

When the application chained this structure when calling xrEndFrame, the reprojectionMode **must** be one of the supported XrReprojectionModeMSFT returned by xrEnumerateReprojectionModesMSFT function for the corresponding XrViewConfigurationType. Otherwise, the runtime **must** return error XR_ERROR_REPROJECTION_MODE_UNSUPPORTED_MSFT on the xrEndFrame function.

The runtime **must** only use the given information for the corresponding frame in xrEndFrame function, and it **must** not affect other frames.

The XrReprojectionModeMSFT describes the reprojection mode of a projection composition layer.

```
// Provided by XR_MSFT_composition_layer_reprojection
typedef enum XrReprojectionModeMSFT {
    XR_REPROJECTION_MODE_DEPTH_MSFT = 1,
    XR_REPROJECTION_MODE_PLANAR_FROM_DEPTH_MSFT = 2,
    XR_REPROJECTION_MODE_PLANAR_MANUAL_MSFT = 3,
    XR_REPROJECTION_MODE_ORIENTATION_ONLY_MSFT = 4,
    XR_REPROJECTION_MODE_MAX_ENUM_MSFT = 0x7FFFFFFF
} XrReprojectionModeMSFT;
```

- `XR_REPROJECTION_MODE_DEPTH_MSFT` indicates the corresponding layer **may** benefit from per-pixel depth reprojection provided by XrCompositionLayerDepthInfoKHR to the projection layer. This mode is typically used for world-locked content that should remain physically stationary as the user walks around.
- `XR_REPROJECTION_MODE_PLANAR_FROM_DEPTH_MSFT` indicates the corresponding layer **may** benefit from planar reprojection and the plane **can** be calculated from the corresponding depth information provided by XrCompositionLayerDepthInfoKHR to the projection layer. This mode works better when the application knows the content is mostly placed on a plane.
- `XR_REPROJECTION_MODE_PLANAR_MANUAL_MSFT` indicates that the corresponding layer **may** benefit from planar reprojection. The application **can** customize the plane by chaining an XrCompositionLayerReprojectionPlaneOverrideMSFT structure to the same layer. The app **can** also omit the plane override, indicating the runtime should use the default reprojection plane settings. This mode works better when the application knows the content is mostly placed on a plane, or when it cannot afford to submit depth information.
- `XR_REPROJECTION_MODE_ORIENTATION_ONLY_MSFT` indicates the layer should be stabilized only for changes to orientation, ignoring positional changes. This mode works better for body-locked content that should follow the user as they walk around, such as 360-degree video.

When the application passes `XR_REPROJECTION_MODE_DEPTH_MSFT` or `XR_REPROJECTION_MODE_PLANAR_FROM_DEPTH_MSFT` mode, it **should** also provide the depth buffer for the corresponding layer using XrCompositionLayerDepthInfoKHR in `XR_KHR_composition_layer_depth` extension. However, if the application does not submit this depth buffer, the runtime **must** apply a runtime defined fallback reprojection mode, and **must** not fail the xrEndFrame function because of this missing depth.

When the application passes `XR_REPROJECTION_MODE_PLANAR_MANUAL_MSFT` or `XR_REPROJECTION_MODE_ORIENTATION_ONLY_MSFT` mode, it **should** avoid providing a depth buffer for the corresponding layer using XrCompositionLayerDepthInfoKHR in `XR_KHR_composition_layer_depth` extension. However, if the application does submit this depth buffer, the runtime **must** not fail the xrEndFrame function because of this unused depth data.

When the application is confident that overriding the reprojection plane can benefit hologram stability, it **can** provide XrCompositionLayerReprojectionPlaneOverrideMSFT structure to further help the runtime to fine tune the reprojection details.

An application **can** add an XrCompositionLayerReprojectionPlaneOverrideMSFT structure to the `next` chain of XrCompositionLayerProjection structure.

The runtime **must** only use the given plane override for the corresponding frame in xrEndFrame function, and it **must** not affect other frames.

```
// Provided by XR_MSFT_composition_layer_reprojection
typedef struct XrCompositionLayerReprojectionPlaneOverrideMSFT {
    XrStructureType    type;
    const void*        next;
    XrVector3f         position;
    XrVector3f         normal;
    XrVector3f         velocity;
} XrCompositionLayerReprojectionPlaneOverrideMSFT;
```

Parameter Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `position` describes the position of the focus plane represented in the corresponding XrCompositionLayerProjection::`space`.
- `normal` is a unit vector describes the focus plane normal represented in the corresponding XrCompositionLayerProjection::`space`.
- `velocity` is a velocity of the position in the corresponding XrCompositionLayerProjection ::`space` measured in meters per second.

A runtime **must** return `XR_ERROR_VALIDATION_FAILURE` if the `normal` vector deviates by more than 1% from unit length.

Adding a reprojection plane override **may** benefit various reprojection modes including `XR_REPROJECTION_MODE_DEPTH_MSFT`, `XR_REPROJECTION_MODE_PLANAR_FROM_DEPTH_MSFT` and `XR_REPROJECTION_MODE_PLANAR_MANUAL_MSFT`.

When application choose `XR_REPROJECTION_MODE_ORIENTATION_ONLY_MSFT` mode, the reprojection plane override **may** be ignored by the runtime.

Valid Usage (Implicit)

- The `XR_MSFT_composition_layer_reprojection` extension **must** be enabled prior to using XrCompositionLayerReprojectionPlaneOverrideMSFT
- `type` **must** be `XR_TYPE_COMPOSITION_LAYER_REPROJECTION_PLANE_OVERRIDE_MSFT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

New Object Types

New Flag Types

New Enum Constants

XrStructureType enumeration is extended with:

- `XR_TYPE_COMPOSITION_LAYER_REPROJECTION_INFO_MSFT`
- `XR_TYPE_COMPOSITION_LAYER_REPROJECTION_PLANE_OVERRIDE_MSFT`

XrResult enumeration is extended with:

- `XR_ERROR_REPROJECTION_MODE_UNSUPPORTED_MSFT`

New Enums

- XrReprojectionModeMSFT

New Structures

- XrCompositionLayerReprojectionInfoMSFT
- XrCompositionLayerReprojectionPlaneOverrideMSFT

New Functions

- xrEnumerateReprojectionModesMSFT

Issues

Version History

- Revision 1, 2020-06-20 (Yin Li)
  - Initial extension proposal

14.25. XR_MSFT_controller_model

Name String

`XR_MSFT_controller_model`

Extension Type

Instance extension

Registered Extension Number

56

Revision

2

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Deprecation State

- *Obsoleted* by `XR_EXT_interaction_render_model` extension

Contributors

Bryce Hutchings, Microsoft
Darryl Gough, Microsoft
Yin Li, Microsoft
Lachlan Ford, Microsoft

Overview

This extension provides a mechanism to load a GLTF model for controllers. An application **can** render the controller model using the real time pose input from controller's grip action pose and animate controller parts representing the user's interactions, such as pressing a button, or pulling a trigger.

This extension supports any controller interaction profile that supports *.../grip/pose*. The returned controller model represents the physical controller held in the user's hands, and it **may** be different from the current interaction profile.

Query controller model key

xrGetControllerModelKeyMSFT retrieves the `XrControllerModelKeyMSFT` for a controller. This model key **may** later be used to retrieve the model data.

The xrGetControllerModelKeyMSFT function is defined as:

```
// Provided by XR_MSFT_controller_model
XrResult xrGetControllerModelKeyMSFT(
    XrSession                                   session,
    XrPath                                      topLevelUserPath,
    XrControllerModelKeyStateMSFT*              controllerModelKeyState);
```

Parameter Descriptions

- `session` is the specified XrSession.
- `topLevelUserPath` is the top level user path corresponding to the controller render model being queried (e.g. */user/hand/left* or */user/hand/right*).
- `controllerModelKeyState` is a pointer to the XrControllerModelKeyStateMSFT to write the model key state to.

Valid Usage (Implicit)

- The `XR_MSFT_controller_model` extension **must** be enabled prior to calling xrGetControllerModelKeyMSFT
- `session` **must** be a valid XrSession handle
- `controllerModelKeyState` **must** be a pointer to an XrControllerModelKeyStateMSFT structure

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_CONTROLLER_MODEL_KEY_INVALID_MSFT`
- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_OUT_OF_MEMORY`
- `XR_ERROR_PATH_INVALID`
- `XR_ERROR_PATH_UNSUPPORTED`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_VALIDATION_FAILURE`

The XrControllerModelKeyStateMSFT structure is defined as:

```
// Provided by XR_MSFT_controller_model
typedef struct XrControllerModelKeyStateMSFT {
    XrStructureType             type;
    void*                       next;
    XrControllerModelKeyMSFT    modelKey;
} XrControllerModelKeyStateMSFT;
```

Parameter Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `modelKey` is the model key corresponding to the controller render model being queried.

The `modelKey` value for the session represents a unique controller model that can be retrieved from xrLoadControllerModelMSFT function. Therefore, the application **can** use `modelKey` to cache the returned data from xrLoadControllerModelMSFT for the session.

A `modelKey` value of XR_NULL_CONTROLLER_MODEL_KEY_MSFT, represents an invalid model key and indicates there is no controller model yet available. The application **should** keep calling xrGetControllerModelKeyMSFT because the model **may** become available at a later point.

The returned `modelKey` value depends on an active action binding to the corresponding *.../grip/pose* of the controller. Therefore, the application **must** have provided a valid action set containing an action for *.../grip/pose*, and have successfully completed an xrSyncActions call, in order to obtain a valid `modelKey`.

Valid Usage (Implicit)

- The `XR_MSFT_controller_model` extension **must** be enabled prior to using XrControllerModelKeyStateMSFT
- `type` **must** be `XR_TYPE_CONTROLLER_MODEL_KEY_STATE_MSFT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

```
// Provided by XR_MSFT_controller_model
#define XR_NULL_CONTROLLER_MODEL_KEY_MSFT 0
```

XR_NULL_CONTROLLER_MODEL_KEY_MSFT defines an invalid model key value.

```
// Provided by XR_MSFT_controller_model
XR_DEFINE_ATOM(XrControllerModelKeyMSFT)
```

The controller model key used to retrieve the data for the renderable controller model and associated properties and state.

Load controller model as glTF 2.0 data

Once the application obtained a valid `modelKey`, it **can** use the xrLoadControllerModelMSFT function to load the GLB data for the controller model.

The xrLoadControllerModelMSFT function loads the controller model as a byte buffer containing a binary form of glTF (a.k.a GLB file format) for the controller. The binary glTF data **must** conform to glTF 2.0 format defined at https://registry.khronos.org/glTF/specs/2.0/glTF-2.0.html.

```
// Provided by XR_MSFT_controller_model
XrResult xrLoadControllerModelMSFT(
    XrSession                                   session,
    XrControllerModelKeyMSFT                    modelKey,
    uint32_t                                    bufferCapacityInput,
    uint32_t*                                   bufferCountOutput,
    uint8_t*                                    buffer);
```

Parameter Descriptions

- `session` is the specified XrSession.
- `modelKey` is the model key corresponding to the controller render model being queried.
- `bufferCapacityInput` is the capacity of the `buffer` array, or 0 to indicate a request to retrieve the required capacity.
- `bufferCountOutput` filled in by the runtime with the count of elements in `buffer` array, or returns the required capacity in the case that `bufferCapacityInput` is insufficient.
- `buffer` is a pointer to an application-allocated array of the model for the device that will be filled with the `uint8_t` values by the runtime. It **can** be `NULL` if `bufferCapacityInput` is 0.
- See the Buffer Size Parameters section for a detailed description of retrieving the required `buffer` size.

The xrLoadControllerModelMSFT function **may** be a slow operation and therefore **should** be invoked from a non-timing critical thread.

If the input `modelKey` is invalid, i.e. it is XR_NULL_CONTROLLER_MODEL_KEY_MSFT or not a key returned from XrControllerModelKeyStateMSFT, the runtime **must** return

XR_ERROR_CONTROLLER_MODEL_KEY_INVALID_MSFT.

Valid Usage (Implicit)

- The XR_MSFT_controller_model extension **must** be enabled prior to calling xrLoadControllerModelMSFT
- session **must** be a valid XrSession handle
- bufferCountOutput **must** be a pointer to a uint32_t value
- If bufferCapacityInput is not 0, buffer **must** be a pointer to an array of bufferCapacityInput uint8_t values

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_CONTROLLER_MODEL_KEY_INVALID_MSFT
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_SIZE_INSUFFICIENT
- XR_ERROR_VALIDATION_FAILURE

Animate controller parts

The application **can** animate parts of the glTF model to represent the user's interaction on the controller, such as pressing a button or pulling a trigger.

Once the application loads the glTF model of the controller, it **should** first get XrControllerModelPropertiesMSFT containing an array of node names in the glTF model that **can** be animated. These properties, including the order of these node names in the array, **must** be immutable for a valid modelKey in the session, and therefore **can** be cached. In the frame loop, the application **should** get XrControllerModelStateMSFT to retrieve the pose of each node representing user's

interaction on the controller and apply the transform to the corresponding node in the glTF model using application's glTF renderer.

The xrGetControllerModelPropertiesMSFT function returns the controller model properties for a given `modelKey`.

```
// Provided by XR_MSFT_controller_model
XrResult xrGetControllerModelPropertiesMSFT(
    XrSession                                   session,
    XrControllerModelKeyMSFT                    modelKey,
    XrControllerModelPropertiesMSFT*            properties);
```

Parameter Descriptions

- `session` is the specified XrSession.
- `modelKey` is a valid model key obtained from XrControllerModelKeyStateMSFT
- `properties` is an XrControllerModelPropertiesMSFT returning the properties of the controller model

The runtime **must** return the same data in XrControllerModelPropertiesMSFT for a valid `modelKey`. Therefore, the application **can** cache the returned XrControllerModelPropertiesMSFT using `modelKey` and reuse the data for each frame.

If the input `modelKey` is invalid, i.e. it is XR_NULL_CONTROLLER_MODEL_KEY_MSFT or not a key returned from XrControllerModelKeyStateMSFT, the runtime **must** return `XR_ERROR_CONTROLLER_MODEL_KEY_INVALID_MSFT`.

Valid Usage (Implicit)

- The `XR_MSFT_controller_model` extension **must** be enabled prior to calling xrGetControllerModelPropertiesMSFT
- `session` **must** be a valid XrSession handle
- `properties` **must** be a pointer to an XrControllerModelPropertiesMSFT structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_CONTROLLER_MODEL_KEY_INVALID_MSFT
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_SIZE_INSUFFICIENT
- XR_ERROR_VALIDATION_FAILURE

The XrControllerModelPropertiesMSFT structure describes the properties of a controller model including an array of XrControllerModelNodePropertiesMSFT.

```
// Provided by XR_MSFT_controller_model
typedef struct XrControllerModelPropertiesMSFT {
    XrStructureType                         type;
    void*                                   next;
    uint32_t                                nodeCapacityInput;
    uint32_t                                nodeCountOutput;
    XrControllerModelNodePropertiesMSFT*    nodeProperties;
} XrControllerModelPropertiesMSFT;
```

Parameter Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `nodeCapacityInput` is the capacity of the `nodeProperties` array, or 0 to indicate a request to retrieve the required capacity.
- `nodeCountOutput` filled in by the runtime with the count of elements in `nodeProperties` array, or returns the required capacity in the case that `nodeCapacityInput` is insufficient.
- `nodeProperties` is a pointer to an application-allocated array that will be filled with the XrControllerModelNodePropertiesMSFT values. It **can** be `NULL` if `nodeCapacityInput` is 0.
- See the Buffer Size Parameters section for a detailed description of retrieving the required `nodeProperties` size.

Valid Usage (Implicit)

- The `XR_MSFT_controller_model` extension **must** be enabled prior to using XrControllerModelPropertiesMSFT
- `type` **must** be `XR_TYPE_CONTROLLER_MODEL_PROPERTIES_MSFT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- If `nodeCapacityInput` is not `0`, `nodeProperties` **must** be a pointer to an array of `nodeCapacityInput` XrControllerModelNodePropertiesMSFT structures

The XrControllerModelNodePropertiesMSFT structure describes properties of animatable nodes, including the node name and parent node name to locate a glTF node in the controller model that **can** be animated based on user's interactions on the controller.

```
// Provided by XR_MSFT_controller_model
typedef struct XrControllerModelNodePropertiesMSFT {
    XrStructureType    type;
    void*              next;
    char               parentNodeName[XR_MAX_CONTROLLER_MODEL_NODE_NAME_SIZE_MSFT];
    char               nodeName[XR_MAX_CONTROLLER_MODEL_NODE_NAME_SIZE_MSFT];
} XrControllerModelNodePropertiesMSFT;
```

Parameter Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `parentNodeName` is the name of the parent node in the provided glTF file. The parent name **may** be empty if it should not be used to locate this node.
- `nodeName` is the name of this node in the provided glTF file.

The node can be located in the glTF node hierarchy by finding the node(s) with the matching node name and parent node name. If the `parentNodeName` is empty, the matching will be solely based on the `nodeName`.

If there are multiple nodes in the glTF file matches the condition above, the first matching node using depth-first traversal in the glTF scene **should** be animated and the rest **should** be ignored.

The runtime **must** not return any `nodeName` or `parentNodeName` that does not match any glTF nodes in the corresponding controller model.

Valid Usage (Implicit)

- The `XR_MSFT_controller_model` extension **must** be enabled prior to using XrControllerModelNodePropertiesMSFT
- `type` **must** be `XR_TYPE_CONTROLLER_MODEL_NODE_PROPERTIES_MSFT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `parentNodeName` **must** be a null-terminated UTF-8 string whose length is less than or equal to `XR_MAX_CONTROLLER_MODEL_NODE_NAME_SIZE_MSFT`
- `nodeName` **must** be a null-terminated UTF-8 string whose length is less than or equal to `XR_MAX_CONTROLLER_MODEL_NODE_NAME_SIZE_MSFT`

The xrGetControllerModelStateMSFT function returns the current state of the controller model representing user's interaction to the controller, such as pressing a button or pulling a trigger.

```
// Provided by XR_MSFT_controller_model
XrResult xrGetControllerModelStateMSFT(
    XrSession                                   session,
    XrControllerModelKeyMSFT                    modelKey,
    XrControllerModelStateMSFT*                 state);
```

Parameter Descriptions

- `session` is the specified XrSession.
- `modelKey` is the model key corresponding to the controller model being queried.
- `state` is a pointer to XrControllerModelStateMSFT and returns the current controller model state.

The runtime **may** return different state for a model key after each call to xrSyncActions, which represents the latest state of the user interactions.

If the input `modelKey` is invalid, i.e. it is XR_NULL_CONTROLLER_MODEL_KEY_MSFT or not a key returned from XrControllerModelKeyStateMSFT, the runtime **must** return `XR_ERROR_CONTROLLER_MODEL_KEY_INVALID_MSFT`.

Valid Usage (Implicit)

- The `XR_MSFT_controller_model` extension **must** be enabled prior to calling xrGetControllerModelStateMSFT
- `session` **must** be a valid XrSession handle
- `state` **must** be a pointer to an XrControllerModelStateMSFT structure

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_CONTROLLER_MODEL_KEY_INVALID_MSFT`
- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_OUT_OF_MEMORY`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_VALIDATION_FAILURE`

The XrControllerModelStateMSFT structure describes the state of a controller model, including an array of XrControllerModelNodeStateMSFT.

```
// Provided by XR_MSFT_controller_model
typedef struct XrControllerModelStateMSFT {
    XrStructureType                     type;
    void*                               next;
    uint32_t                            nodeCapacityInput;
    uint32_t                            nodeCountOutput;
    XrControllerModelNodeStateMSFT*     nodeStates;
} XrControllerModelStateMSFT;
```

Parameter Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `nodeCapacityInput` is the capacity of the `nodeStates` array, or 0 to indicate a request to retrieve the required capacity.
- `nodeCountOutput` filled in by the runtime with the count of elements in `nodeStates` array, or returns the required capacity in the case that `nodeCapacityInput` is insufficient.
- `nodeStates` is a pointer to an application-allocated array that will be filled with the XrControllerModelNodeStateMSFT values. It **can** be `NULL` if `nodeCapacityInput` is 0.
- See the Buffer Size Parameters section for a detailed description of retrieving the required `nodeStates` size.

Valid Usage (Implicit)

- The `XR_MSFT_controller_model` extension **must** be enabled prior to using XrControllerModelStateMSFT
- `type` **must** be `XR_TYPE_CONTROLLER_MODEL_STATE_MSFT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- If `nodeCapacityInput` is not `0`, `nodeStates` **must** be a pointer to an array of `nodeCapacityInput` XrControllerModelNodeStateMSFT structures

The XrControllerModelNodeStateMSFT structure describes the state of a node in a controller model.

```
// Provided by XR_MSFT_controller_model
typedef struct XrControllerModelNodeStateMSFT {
    XrStructureType    type;
    void*              next;
    XrPosef            nodePose;
} XrControllerModelNodeStateMSFT;
```

Parameter Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `nodePose` is an XrPosef of the node in its parent node space.

The state is corresponding to the glTF node identified by the XrControllerModelNodePropertiesMSFT::`nodeName` and XrControllerModelNodePropertiesMSFT::`parentNodeName` of the node property at the same array index in the XrControllerModelPropertiesMSFT::`nodeProperties` in XrControllerModelPropertiesMSFT.

The `nodePose` is based on the user's interaction on the controller at the latest xrSyncActions, represented as the XrPosef of the node in it's parent node space.

Valid Usage (Implicit)

- The `XR_MSFT_controller_model` extension **must** be enabled prior to using XrControllerModelNodeStateMSFT
- `type` **must** be `XR_TYPE_CONTROLLER_MODEL_NODE_STATE_MSFT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

New Object Types

New Flag Types

New Enum Constants

- `XR_MAX_CONTROLLER_MODEL_NODE_NAME_SIZE_MSFT`
- `XR_TYPE_CONTROLLER_MODEL_NODE_PROPERTIES_MSFT`
- `XR_TYPE_CONTROLLER_MODEL_PROPERTIES_MSFT`
- `XR_TYPE_CONTROLLER_MODEL_NODE_STATE_MSFT`
- `XR_TYPE_CONTROLLER_MODEL_STATE_MSFT`

- `XR_ERROR_CONTROLLER_MODEL_KEY_INVALID_MSFT`

New Enums

New Structures

- XrControllerModelKeyStateMSFT
- XrControllerModelNodePropertiesMSFT
- XrControllerModelPropertiesMSFT
- XrControllerModelNodeStateMSFT
- XrControllerModelStateMSFT

New Functions

- xrGetControllerModelKeyMSFT
- xrLoadControllerModelMSFT
- xrGetControllerModelPropertiesMSFT
- xrGetControllerModelStateMSFT

Issues

Version History

- Revision 1, 2020-03-12 (Yin Li)
  - Initial extension description
- Revision 2, 2020-08-12 (Bryce Hutchings)
  - Remove a possible error condition

14.26. XR_MSFT_first_person_observer

Name String

`XR_MSFT_first_person_observer`

Extension Type

Instance extension

Registered Extension Number

55

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0
and
`XR_MSFT_secondary_view_configuration`

Deprecation State

- *Deprecated* without replacement

Last Modified Date

2020-05-02

IP Status

No known IP claims.

Contributors

Yin Li, Microsoft
Zonglin Wu, Microsoft
Alex Turner, Microsoft

14.26.1. Overview

This first-person observer view configuration enables the runtime to request the application to render an additional first-person view of the scene to be composed onto video frames being captured from a camera attached to and moved with the primary display on the form factor, which is generally for viewing on a 2D screen by an external observer. This first-person camera will be facing forward with roughly the same perspective as the primary views, and so the application should render its view to show objects that surround the user and avoid rendering the user's body avatar. The runtime is responsible for composing the application's rendered observer view onto the camera frame based on the chosen environment blend mode for this view configuration, as this extension does not provide the associated camera frame to the application.

This extension requires the `XR_MSFT_secondary_view_configuration` extension to also be enabled.

`XR_VIEW_CONFIGURATION_TYPE_SECONDARY_MONO_FIRST_PERSON_OBSERVER_MSFT` requires one projection in each XrCompositionLayerProjection layer.

Runtimes **should** only make this view configuration active when the user or the application activates a runtime feature that will make use of the resulting composed camera frames, for example taking a mixed reality photo. Otherwise, the runtime **should** leave this view configuration inactive to avoid the application wasting CPU and GPU resources rendering unnecessarily for this extra view.

Because this is a first-person view of the scene, applications **can** share a common culling and instanced rendering pass with their primary view renders. However, the view state (pose and FOV) of the first-

person observer view will not match the view state of any of the primary views. Applications enabling this view configuration **must** call xrLocateViews a second time each frame to explicitly query the view state for the XR_VIEW_CONFIGURATION_TYPE_SECONDARY_MONO_FIRST_PERSON_OBSERVER_MSFT configuration.

This secondary view configuration **may** support a different set of environment blend modes than the primary view configuration. For example, a device that only supports additive blending for its primary display may support alpha-blending when composing the first-person observer view with camera frames. The application should render with assets and shaders that produce output acceptable to both the primary and observer view configuration's environment blend modes when sharing render passes across both view configurations.

New Object Types

New Flag Types

New Enum Constants

XrViewConfigurationType enumeration is extended with:

- XR_VIEW_CONFIGURATION_TYPE_SECONDARY_MONO_FIRST_PERSON_OBSERVER_MSFT

New Enums

New Structures

New Functions

Issues

Version History

- Revision 1, 2019-07-30 (Yin LI)
  - Initial extension description

14.27. XR_MSFT_hand_interaction

Name String

XR_MSFT_hand_interaction

Extension Type

Instance extension

Registered Extension Number

51

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

API Interactions

- Interacts with `XR_EXT_hand_interaction`
- Interacts with `XR_EXT_palm_pose`

Deprecation State

- *Promoted* to `XR_EXT_hand_interaction` extension

Contributors

Yin Li, Microsoft
Lachlan Ford, Microsoft
Alex Turner, Microsoft

Overview

This extension defines a new interaction profile for near interactions and far interactions driven by directly-tracked hands.

Hand interaction profile

Interaction profile path:

- */interaction_profiles/microsoft/hand_interaction*

> **Note**
>
> The interaction profile path */interaction_profiles/microsoft/hand_interaction* defined here does not follow current rules for interaction profile names. If this extension were introduced today, it would be called */interaction_profiles/microsoft/hand_interaction_msft*, to allow for modifications when promoted to a KHR extension or the core specification.

Valid for top level user path:

- */user/hand/left*
- */user/hand/right*

This interaction profile provides basic pose and actions for near and far interactions using hand tracking input.

Supported component paths:

- *.../input/select/value*
- *.../input/squeeze/value*
- *.../input/aim/pose*
- *.../input/grip/pose*



Note

When the runtime supports `XR_VERSION_1_1` and use of OpenXR 1.1 is requested by the application, this interaction profile **must** also support

- *.../input/grip_surface/pose*



Note

When the `XR_KHR_maintenance1` extension is available and enabled, this interaction profile **must** also support

- *.../input/grip_surface/pose*



Note

When the `XR_EXT_palm_pose` extension is available and enabled, this interaction profile **must** also support

- *.../input/palm_ext/pose*



Note

When the `XR_EXT_hand_interaction` extension is available and enabled, this interaction profile **must** also support

- *.../input/pinch_ext/pose*
- *.../input/poke_ext/pose*

The application **should** use the *.../select/value* and *.../aim/pose* paths for far hand interactions, such as using a virtual laser pointer to target and click a button on the wall. Here, *.../select/value* **can** be used as either a boolean or float action type, where the value `XR_TRUE` or `1.0f` represents a closed hand shape.

The application **should** use the *.../squeeze/value* and *.../grip/pose* for near hand interactions, such as picking up a virtual object within the user's reach from a table. Here, *.../squeeze/value* **can** be used as either a boolean or float action type, where the value `XR_TRUE` or `1.0f` represents a closed hand shape.

The runtime **may** trigger both "select" and "squeeze" actions for the same hand gesture if the user's hand gesture is able to trigger both near and far interactions. The application **should** not assume they are as independent as two buttons on a controller.

New Object Types

New Flag Types

New Enum Constants

New Enums

New Structures

New Functions

Issues

Version History

- Revision 1, 2019-09-16 (Yin Li)
  - Initial extension description

14.28. XR_MSFT_hand_tracking_mesh

Name String

`XR_MSFT_hand_tracking_mesh`

Extension Type

Instance extension

Registered Extension Number

53

Revision

4

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0
and
`XR_EXT_hand_tracking`

Deprecation State

- *Deprecated* without replacement

Last Modified Date

2021-10-20

IP Status

No known IP claims.

Contributors

Yin Li, Microsoft
Lachlan Ford, Microsoft
Alex Turner, Microsoft
Bryce Hutchings, Microsoft

14.28.1. Overview

This extension enables hand tracking inputs represented as a dynamic hand mesh. It enables applications to render hands in XR experiences and interact with virtual objects using hand meshes.

The application **must** also enable the `XR_EXT_hand_tracking` extension in order to use this extension.

Inspect system capability

An application **can** inspect whether the system is capable of hand tracking meshes by chaining an XrSystemHandTrackingMeshPropertiesMSFT structure to the XrSystemProperties when calling xrGetSystemProperties.

```
// Provided by XR_MSFT_hand_tracking_mesh
typedef struct XrSystemHandTrackingMeshPropertiesMSFT {
    XrStructureType    type;
    void*              next;
    XrBool32           supportsHandTrackingMesh;
    uint32_t           maxHandMeshIndexCount;
    uint32_t           maxHandMeshVertexCount;
} XrSystemHandTrackingMeshPropertiesMSFT;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension. For more details about the structure chain, see the structure being extended (XrSystemProperties).
- `supportsHandTrackingMesh` is an `XrBool32`, indicating if current system is capable of hand tracking mesh input.
- `maxHandMeshIndexCount` is a `uint32_t` returns the maximum count of indices that will be returned from the hand tracker.
- `maxHandMeshVertexCount` is a `uint32_t` returns the maximum count of vertices that will be returned from the hand tracker.

If a runtime returns `XR_FALSE` for `supportsHandTrackingMesh`, the system does not support hand tracking mesh input, and therefore **must** return `XR_ERROR_FEATURE_UNSUPPORTED` from xrCreateHandMeshSpaceMSFT and xrUpdateHandMeshMSFT. The application **should** avoid using hand mesh functionality when `supportsHandTrackingMesh` is `XR_FALSE`.

If a runtime returns `XR_TRUE` for `supportsHandTrackingMesh`, the system supports hand tracking mesh input. In this case, the runtime **must** return a positive number for `maxHandMeshIndexCount` and `maxHandMeshVertexCount`. An application **should** use `maxHandMeshIndexCount` and `maxHandMeshVertexCount` to preallocate hand mesh buffers and reuse them in their render loop when calling xrUpdateHandMeshMSFT every frame.

Valid Usage (Implicit)

- The `XR_MSFT_hand_tracking_mesh` extension **must** be enabled prior to using XrSystemHandTrackingMeshPropertiesMSFT
- `type` **must** be `XR_TYPE_SYSTEM_HAND_TRACKING_MESH_PROPERTIES_MSFT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

14.28.2. Obtain a hand tracker handle

An application first creates an XrHandTrackerEXT handle using the xrCreateHandTrackerEXT function for each hand. The application can also reuse the same XrHandTrackerEXT handle previously created for the hand joint tracking. When doing so, the hand mesh input is always in sync with hand joints input with the same XrHandTrackerEXT handle.

14.28.3. Create a hand mesh space

The application creates a hand mesh space using function xrCreateHandMeshSpaceMSFT. The position and normal of hand mesh vertices will be represented in this space.

```
// Provided by XR_MSFT_hand_tracking_mesh
XrResult xrCreateHandMeshSpaceMSFT(
    XrHandTrackerEXT                            handTracker,
    const XrHandMeshSpaceCreateInfoMSFT*        createInfo,
    XrSpace*                                    space);
```

Parameter Descriptions

- handTracker is an XrHandTrackerEXT handle previously created with the xrCreateHandTrackerEXT function.
- createInfo is the XrHandMeshSpaceCreateInfoMSFT used to specify the hand mesh space.
- space is the returned XrSpace handle of the new hand mesh space.

A hand mesh space location is specified by runtime preference to effectively represent hand mesh vertices without unnecessary transformations. For example, an optical hand tracking system **can** define the hand mesh space origin at the depth camera's optical center.

An application should create separate hand mesh space handles for each hand to retrieve the corresponding hand mesh data. The runtime **may** use the lifetime of this hand mesh space handle to manage the underlying device resources. Therefore, the application **should** destroy the hand mesh handle after it is finished using the hand mesh.

The hand mesh space can be related to other spaces in the session, such as view reference space, or grip action space from the */interaction_profiles/khr/simple_controller* interaction profile. The hand mesh space may be not locatable when the hand is outside of the tracking range, or if focus is removed from the application. In these cases, the runtime **must** not set the XR_SPACE_LOCATION_POSITION_VALID_BIT and XR_SPACE_LOCATION_ORIENTATION_VALID_BIT bits on calls to xrLocateSpace with the hand mesh space, and the application **should** avoid using the returned poses or query for hand mesh data.

If the underlying XrHandTrackerEXT is destroyed, the runtime **must** continue to support xrLocateSpace using the hand mesh space, and it **must** return space location with XR_SPACE_LOCATION_POSITION_VALID_BIT and XR_SPACE_LOCATION_ORIENTATION_VALID_BIT unset.

The application **may** create a mesh space for the reference hand by setting XrHandPoseTypeInfoMSFT::handPoseType to XR_HAND_POSE_TYPE_REFERENCE_OPEN_PALM_MSFT. Hand mesh spaces for the reference hand **must** only be locatable in reference to mesh spaces or joint spaces of the reference hand.

Valid Usage (Implicit)

- The XR_MSFT_hand_tracking_mesh extension **must** be enabled prior to calling xrCreateHandMeshSpaceMSFT
- handTracker **must** be a valid XrHandTrackerEXT handle
- createInfo **must** be a pointer to a valid XrHandMeshSpaceCreateInfoMSFT structure
- space **must** be a pointer to an XrSpace handle

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FEATURE_UNSUPPORTED
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_POSE_INVALID
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

```
// Provided by XR_MSFT_hand_tracking_mesh
typedef struct XrHandMeshSpaceCreateInfoMSFT {
    XrStructureType       type;
    const void*           next;
    XrHandPoseTypeMSFT    handPoseType;
    XrPosef               poseInHandMeshSpace;
} XrHandMeshSpaceCreateInfoMSFT;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `handPoseType` is an XrHandPoseTypeMSFT used to specify the type of hand this mesh is tracking. Indices and vertices returned from xrUpdateHandMeshMSFT for a hand type will be relative to the corresponding space create with the same hand type.
- `poseInHandMeshSpace` is an XrPosef defining the position and orientation of the new space's origin within the natural reference frame of the hand mesh space.

Valid Usage (Implicit)

- The `XR_MSFT_hand_tracking_mesh` extension **must** be enabled prior to using XrHandMeshSpaceCreateInfoMSFT
- `type` **must** be `XR_TYPE_HAND_MESH_SPACE_CREATE_INFO_MSFT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `handPoseType` **must** be a valid XrHandPoseTypeMSFT value

14.28.4. Locate the hand mesh

The application **can** use the xrUpdateHandMeshMSFT function to retrieve the hand mesh at a given timestamp. The hand mesh's vertices position and normal are represented in the hand mesh space created by xrCreateHandMeshSpaceMSFT with a same XrHandTrackerEXT.

```
// Provided by XR_MSFT_hand_tracking_mesh
XrResult xrUpdateHandMeshMSFT(
    XrHandTrackerEXT                            handTracker,
    const XrHandMeshUpdateInfoMSFT*             updateInfo,
    XrHandMeshMSFT*                             handMesh);
```

Parameter Descriptions

- `handTracker` is an XrHandTrackerEXT handle previously created with xrCreateHandTrackerEXT.
- `updateInfo` is an XrHandMeshUpdateInfoMSFT which contains information to query the hand mesh.
- `handMesh` is an XrHandMeshMSFT structure to receive the updates of hand mesh data.

The application **should** preallocate the index buffer and vertex buffer in XrHandMeshMSFT using the XrSystemHandTrackingMeshPropertiesMSFT::`maxHandMeshIndexCount` and XrSystemHandTrackingMeshPropertiesMSFT::`maxHandMeshVertexCount` from the XrSystemHandTrackingMeshPropertiesMSFT returned from the xrGetSystemProperties function.

The application **should** preallocate the XrHandMeshMSFT structure and reuse it for each frame so as to reduce the copies of data when underlying tracking data is not changed. The application should use XrHandMeshMSFT::`indexBufferChanged` and XrHandMeshMSFT::`vertexBufferChanged` in XrHandMeshMSFT to detect changes and avoid unnecessary data processing when there is no changes.

Valid Usage (Implicit)

- The `XR_MSFT_hand_tracking_mesh` extension **must** be enabled prior to calling xrUpdateHandMeshMSFT
- `handTracker` **must** be a valid XrHandTrackerEXT handle
- `updateInfo` **must** be a pointer to a valid XrHandMeshUpdateInfoMSFT structure
- `handMesh` **must** be a pointer to an XrHandMeshMSFT structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FEATURE_UNSUPPORTED
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_SIZE_INSUFFICIENT
- XR_ERROR_TIME_INVALID
- XR_ERROR_VALIDATION_FAILURE

A XrHandMeshUpdateInfoMSFT describes the information to update a hand mesh.

```
// Provided by XR_MSFT_hand_tracking_mesh
typedef struct XrHandMeshUpdateInfoMSFT {
    XrStructureType       type;
    const void*           next;
    XrTime                time;
    XrHandPoseTypeMSFT    handPoseType;
} XrHandMeshUpdateInfoMSFT;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- time is the XrTime that describes the time for which the application wishes to query the hand mesh state.
- handPoseType is an XrHandPoseTypeMSFT which describes the type of hand pose of the hand mesh to update.

A runtime **may** not maintain a full history of hand mesh data, therefore the returned XrHandMeshMSFT might return data that's not exactly corresponding to the `time` input. If the runtime cannot return any tracking data for the given `time` at all, it **must** set XrHandMeshMSFT::`isActive` to `XR_FALSE` for the call to xrUpdateHandMeshMSFT. Otherwise, if the runtime returns XrHandMeshMSFT::`isActive` as `XR_TRUE`, the data in XrHandMeshMSFT must be valid to use.

An application can choose different `handPoseType` values to query the hand mesh data. The returned hand mesh **must** be consistent to the hand joint space location on the same XrHandTrackerEXT when using the same XrHandPoseTypeMSFT.

Valid Usage (Implicit)

- The `XR_MSFT_hand_tracking_mesh` extension **must** be enabled prior to using XrHandMeshUpdateInfoMSFT
- `type` **must** be `XR_TYPE_HAND_MESH_UPDATE_INFO_MSFT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `handPoseType` **must** be a valid XrHandPoseTypeMSFT value

A XrHandMeshMSFT structure contains data and buffers to receive updates of hand mesh tracking data from xrUpdateHandMeshMSFT function.

```
// Provided by XR_MSFT_hand_tracking_mesh
typedef struct XrHandMeshMSFT {
    XrStructureType               type;
    void*                         next;
    XrBool32                      isActive;
    XrBool32                      indexBufferChanged;
    XrBool32                      vertexBufferChanged;
    XrHandMeshIndexBufferMSFT     indexBuffer;
    XrHandMeshVertexBufferMSFT    vertexBuffer;
} XrHandMeshMSFT;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `isActive` is an `XrBool32` indicating if the current hand tracker is active.
- `indexBufferChanged` is an `XrBool32` indicating if the `indexBuffer` content was changed during the update.
- `vertexBufferChanged` is an `XrBool32` indicating if the `vertexBuffer` content was changed during the update.
- `indexBuffer` is an XrHandMeshIndexBufferMSFT returns the index buffer of the tracked hand mesh.
- `vertexBuffer` is an XrHandMeshVertexBufferMSFT returns the vertex buffer of the tracked hand mesh.

When the returned `isActive` value is `XR_FALSE`, the runtime indicates the hand is not actively tracked, for example, the hand is outside of sensor's range, or the input focus is taken away from the application. When the runtime returns `XR_FALSE` to `isActive`, it **must** set `indexBufferChanged` and `vertexBufferChanged` to `XR_FALSE`, and **must** not change the content in `indexBuffer` or `vertexBuffer`,

When the returned `isActive` value is `XR_TRUE`, the hand tracking mesh represented in `indexBuffer` and `vertexBuffer` are updated to the latest data of the XrHandMeshUpdateInfoMSFT::`time` given to the xrUpdateHandMeshMSFT function. The runtime **must** set `indexBufferChanged` and `vertexBufferChanged` to reflect whether the index or vertex buffer's content are changed during the update. In this way, the application can easily avoid unnecessary processing of buffers when there's no new data.

The hand mesh is represented in triangle lists and each triangle's vertices are in clockwise order when looking from outside of the hand. When hand tracking is active, i.e. when `isActive` is returned as `XR_TRUE`, the returned `indexBuffer.indexCountOutput` value **must** be positive and multiple of 3, and `vertexBuffer.vertexCountOutput` value **must** be equal to or larger than 3.

Valid Usage (Implicit)

- The `XR_MSFT_hand_tracking_mesh` extension **must** be enabled prior to using XrHandMeshMSFT
- `type` **must** be `XR_TYPE_HAND_MESH_MSFT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `indexBuffer` **must** be a valid XrHandMeshIndexBufferMSFT structure
- `vertexBuffer` **must** be a valid XrHandMeshVertexBufferMSFT structure

A XrHandMeshIndexBufferMSFT structure includes an array of indices describing the triangle list of a hand mesh.

```
// Provided by XR_MSFT_hand_tracking_mesh
typedef struct XrHandMeshIndexBufferMSFT {
    uint32_t     indexBufferKey;
    uint32_t     indexCapacityInput;
    uint32_t     indexCountOutput;
    uint32_t*    indices;
} XrHandMeshIndexBufferMSFT;
```

Member Descriptions

- `indexBufferKey` is a `uint32_t` serving as the key of the returned index buffer content or 0 to indicate a request to retrieve the latest indices regardless of existing content in `indices`.
- `indexCapacityInput` is a positive `uint32_t` describes the capacity of the `indices` array.
- `indexCountOutput` is a `uint32_t` returned by the runtime with the count of indices written in `indices`.
- `indices` is an array of indices filled in by the runtime, specifying the indices of the triangles list in the vertex buffer.

An application **should** preallocate the indices array using the XrSystemHandTrackingMeshPropertiesMSFT::`maxHandMeshIndexCount` returned from xrGetSystemProperties. In this way, the application can avoid possible insufficient buffer sizees for each query, and therefore avoid reallocating memory each frame.

The input `indexCapacityInput` **must** not be 0, and `indices` **must** not be `NULL`, or else the runtime **must** return `XR_ERROR_VALIDATION_FAILURE` on calls to the xrUpdateHandMeshMSFT function.

If the input `indexCapacityInput` is not sufficient to contain all output indices, the runtime **must** return `XR_ERROR_SIZE_INSUFFICIENT` on calls to xrUpdateHandMeshMSFT, not change the content in `indexBufferKey` and `indices`, and return 0 for `indexCountOutput`.

If the input `indexCapacityInput` is equal to or larger than the XrSystemHandTrackingMeshPropertiesMSFT::`maxHandMeshIndexCount` returned from xrGetSystemProperties, the runtime **must** not return `XR_ERROR_SIZE_INSUFFICIENT` error on xrUpdateHandMeshMSFT because of insufficient index buffer size.

If the input `indexBufferKey` is 0, the capacity of indices array is sufficient, and hand mesh tracking is active, the runtime **must** return the latest non-zero `indexBufferKey`, and fill in `indexCountOutput` and `indices`.

If the input `indexBufferKey` is not 0, the runtime **can** either return without changing `indexCountOutput`

or content in indices, and return XR_FALSE for XrHandMeshMSFT::indexBufferChanged indicating the indices are not changed; or return a new non-zero indexBufferKey and fill in latest data in indexCountOutput and indices, and return XR_TRUE for XrHandMeshMSFT::indexBufferChanged indicating the indices are updated to a newer version.

An application **can** keep the XrHandMeshIndexBufferMSFT structure for each frame in a frame loop and use the returned indexBufferKey to identify different triangle list topology described in indices. The application can therefore avoid unnecessary processing of indices, such as coping them to GPU memory.

The runtime **must** return the same indexBufferKey for the same XrHandTrackerEXT at a given time, regardless of the input XrHandPoseTypeMSFT in XrHandMeshUpdateInfoMSFT. This ensures the index buffer has the same mesh topology and allows the application to reason about vertices across different hand pose types. For example, the application **can** build a procedure to perform UV mapping on vertices of a hand mesh using XR_HAND_POSE_TYPE_REFERENCE_OPEN_PALM_MSFT, and apply the resultant UV data on vertices to the mesh returned from the same hand tracker using XR_HAND_POSE_TYPE_TRACKED_MSFT.

Valid Usage (Implicit)

- The XR_MSFT_hand_tracking_mesh extension **must** be enabled prior to using XrHandMeshIndexBufferMSFT
- If indexCapacityInput is not 0, indices **must** be a pointer to an array of indexCapacityInput uint32_t values

A XrHandMeshVertexBufferMSFT structure includes an array of vertices of the hand mesh represented in the hand mesh space.

```
// Provided by XR_MSFT_hand_tracking_mesh
typedef struct XrHandMeshVertexBufferMSFT {
    XrTime                   vertexUpdateTime;
    uint32_t                 vertexCapacityInput;
    uint32_t                 vertexCountOutput;
    XrHandMeshVertexMSFT*    vertices;
} XrHandMeshVertexBufferMSFT;
```

Member Descriptions

- `vertexUpdateTime` is an `XrTime` representing the time when the runtime receives the vertex buffer content or 0 to indicate a request to retrieve latest vertices regardless of existing content in `vertices`.
- `vertexCapacityInput` is a positive `uint32_t` describes the capacity of the `vertices` array.
- `vertexCountOutput` is a `uint32_t` filled in by the runtime with the count of vertices written in `vertices`.
- `vertices` is an array of XrHandMeshVertexMSFT filled in by the runtime, specifying the vertices of the hand mesh including the position and normal vector in the hand mesh space.

An application **should** preallocate the vertices array using the XrSystemHandTrackingMeshPropertiesMSFT::`maxHandMeshVertexCount` returned from xrGetSystemProperties. In this way, the application can avoid possible insufficient buffer sizes for each query, and therefore avoid reallocating memory each frame.

The input `vertexCapacityInput` **must** not be 0, and `vertices` **must** not be `NULL`, or else the runtime **must** return `XR_ERROR_VALIDATION_FAILURE` on calls to the xrUpdateHandMeshMSFT function.

If the input `vertexCapacityInput` is not sufficient to contain all output vertices, the runtime **must** return `XR_ERROR_SIZE_INSUFFICIENT` on calls to the xrUpdateHandMeshMSFT, do not change content in `vertexUpdateTime` and `vertices`, and return 0 for `vertexCountOutput`.

If the input `vertexCapacityInput` is equal to or larger than the XrSystemHandTrackingMeshPropertiesMSFT::`maxHandMeshVertexCount` returned from xrGetSystemProperties, the runtime **must** not return `XR_ERROR_SIZE_INSUFFICIENT` on calls to the xrUpdateHandMeshMSFT because of insufficient vertex buffer size.

If the input `vertexUpdateTime` is 0, and the capacity of the vertices array is sufficient, and hand mesh tracking is active, the runtime **must** return the latest non-zero `vertexUpdateTime`, and fill in the `vertexCountOutput` and `vertices` fields.

If the input `vertexUpdateTime` is not 0, the runtime **can** either return without changing `vertexCountOutput` or the content in `vertices`, and return `XR_FALSE` for XrHandMeshMSFT::`vertexBufferChanged` indicating the vertices are not changed; or return a new non-zero `vertexUpdateTime` and fill in latest data in `vertexCountOutput` and `vertices` and return `XR_TRUE` for XrHandMeshMSFT::`vertexBufferChanged` indicating the vertices are updated to a newer version.

An application **can** keep the XrHandMeshVertexBufferMSFT structure for each frame in frame loop and use the returned `vertexUpdateTime` to detect the changes of the content in `vertices`. The application can therefore avoid unnecessary processing of vertices, such as coping them to GPU memory.

Valid Usage (Implicit)

- The XR_MSFT_hand_tracking_mesh extension **must** be enabled prior to using XrHandMeshVertexBufferMSFT
- If vertexCapacityInput is not 0, vertices **must** be a pointer to an array of vertexCapacityInput XrHandMeshVertexMSFT structures

Each XrHandMeshVertexMSFT includes the position and normal of a vertex of a hand mesh.

```
// Provided by XR_MSFT_hand_tracking_mesh
typedef struct XrHandMeshVertexMSFT {
    XrVector3f    position;
    XrVector3f    normal;
} XrHandMeshVertexMSFT;
```

Member Descriptions

- position is an XrVector3f structure representing the position of the vertex in the hand mesh space, measured in meters.
- normal is an XrVector3f structure representing the unweighted normal of the triangle surface at the vertex as a unit vector in hand mesh space.

Valid Usage (Implicit)

- The XR_MSFT_hand_tracking_mesh extension **must** be enabled prior to using XrHandMeshVertexMSFT

14.28.5. Example code for hand mesh tracking

Following example code demos preallocating hand mesh buffers and updating the hand mesh in rendering loop

```
XrInstance instance; // previously initialized
XrSystemId systemId; // previously initialized
XrSession session;   // previously initialized

// Inspect hand tracking mesh system properties
XrSystemHandTrackingMeshPropertiesMSFT handMeshSystemProperties {
    .type = XR_TYPE_SYSTEM_HAND_TRACKING_MESH_PROPERTIES_MSFT,
};
```

```
XrSystemProperties systemProperties {
    .type = XR_TYPE_SYSTEM_PROPERTIES,
    .next = &handMeshSystemProperties,
};
CHK_XR(xrGetSystemProperties(instance, systemId, &systemProperties));
if (!handMeshSystemProperties.supportsHandTrackingMesh) {
    // the system does not support hand mesh tracking
    return;
}

// Get function pointer for xrCreateHandTrackerEXT
PFN_xrCreateHandTrackerEXT pfnCreateHandTrackerEXT = nullptr;
CHK_XR(xrGetInstanceProcAddr(instance, "xrCreateHandTrackerEXT",
                             reinterpret_cast<PFN_xrVoidFunction*>(
                             &pfnCreateHandTrackerEXT)));

// Create a tracker for left hand.
XrHandTrackerEXT leftHandTracker{};
{
    XrHandTrackerCreateInfoEXT createInfo {
        .type = XR_TYPE_HAND_TRACKER_CREATE_INFO_EXT,
        .hand = XR_HAND_LEFT_EXT,
        .handJointSet = XR_HAND_JOINT_SET_DEFAULT_EXT,
    };
    CHK_XR(pfnCreateHandTrackerEXT(session, &createInfo, &leftHandTracker));
}

// Get function pointer for xrCreateHandMeshSpaceMSFT
PFN_xrCreateHandMeshSpaceMSFT pfnCreateHandMeshSpaceMSFT = nullptr;
CHK_XR(xrGetInstanceProcAddr(instance, "xrCreateHandMeshSpaceMSFT",
                             reinterpret_cast<PFN_xrVoidFunction*>(
                             &pfnCreateHandMeshSpaceMSFT)));

// Create the hand mesh spaces
XrSpace leftHandMeshSpace{};
{
    XrHandMeshSpaceCreateInfoMSFT createInfo {
        .type = XR_TYPE_HAND_MESH_SPACE_CREATE_INFO_MSFT,
        .poseInHandMeshSpace = {{0, 0, 0, 1}, {0, 0, 0}},
    };
    CHK_XR(pfnCreateHandMeshSpaceMSFT(leftHandTracker, &createInfo, &leftHandMeshSpace));
}

// Preallocate buffers for hand mesh indices and vertices
std::vector<uint32_t> handMeshIndices(handMeshSystemProperties.maxHandMeshIndexCount);
std::vector<XrHandMeshVertexMSFT> handMeshVertices(handMeshSystemProperties
.maxHandMeshVertexCount);
```

```
XrHandMeshMSFT leftHandMesh {
    .type = XR_TYPE_HAND_MESH_MSFT,
    .indexBuffer = {
        .indexCapacityInput = (uint32_t)handMeshIndices.size(),
        .indices = handMeshIndices.data(),
    },
    .vertexBuffer = {
        .vertexCapacityInput = (uint32_t)handMeshVertices.size(),
        .vertices = handMeshVertices.data(),
    },
};

// Get function pointer for xrUpdateHandMeshMSFT
PFN_xrUpdateHandMeshMSFT pfnUpdateHandMeshMSFT = nullptr;
CHK_XR(xrGetInstanceProcAddr(instance, "xrUpdateHandMeshMSFT",
                             reinterpret_cast<PFN_xrVoidFunction*>(
                             &pfnUpdateHandMeshMSFT)));
while(1){
    // ...
    // For every frame in frame loop
    // ...
    XrFrameState frameState;    // previously returned from xrWaitFrame
    const XrTime time = frameState.predictedDisplayTime;

    XrHandMeshUpdateInfoMSFT updateInfo {
        .type = XR_TYPE_HAND_MESH_UPDATE_INFO_MSFT,
        .time = time,
    };
    CHK_XR(pfnUpdateHandMeshMSFT(leftHandTracker, &updateInfo, &leftHandMesh));
    if (!leftHandMesh.isActive) {
        // Hand input is not focused or user's hand is out of tracking range.
        // Do not process or render hand mesh.
    } else {
        if (leftHandMesh.indexBufferChanged) {
            // Process indices in indexBuffer.indices
        }

        if (leftHandMesh.vertexBufferChanged) {
            // Process vertices in vertexBuffer.vertices and leftHandMeshSpace
        }
    }
}
```

14.28.6. Get hand reference poses

By default, an XrHandTrackerEXT tracks a default hand pose type, that is to provide best fidelity to the user's actual hand motion. This is the same with XR_HAND_POSE_TYPE_TRACKED_MSFT (i.e. value 0) in a

chained XrHandPoseTypeInfoMSFT structure to the next pointer of XrHandTrackerCreateInfoEXT when calling xrCreateHandTrackerEXT.

Some hand mesh visualizations may require an initial analysis or processing of the hand mesh relative to the joints of the hand. For example, a hand visualization may generate a UV mapping for the hand mesh vertices by raycasting outward from key joints against the mesh to find key vertices.

To avoid biasing such static analysis with the arbitrary tracked hand pose, an application **can** instead create a different XrHandTrackerEXT handle with a reference hand pose type when calling xrCreateHandTrackerEXT. This will instruct the runtime to provide a reference hand pose that is better suited for such static analysis.

An application can chain an XrHandPoseTypeInfoMSFT structure to the XrHandTrackerCreateInfoEXT::next pointer when calling xrCreateHandTrackerEXT to indicate the hand tracker to return the hand pose of specific XrHandPoseTypeMSFT.

```
// Provided by XR_MSFT_hand_tracking_mesh
typedef struct XrHandPoseTypeInfoMSFT {
    XrStructureType       type;
    const void*           next;
    XrHandPoseTypeMSFT    handPoseType;
} XrHandPoseTypeInfoMSFT;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- handPoseType is an XrHandPoseTypeMSFT that describes the type of hand pose of the hand tracking.

Valid Usage (Implicit)

- The XR_MSFT_hand_tracking_mesh extension **must** be enabled prior to using XrHandPoseTypeInfoMSFT
- type **must** be XR_TYPE_HAND_POSE_TYPE_INFO_MSFT
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- handPoseType **must** be a valid XrHandPoseTypeMSFT value

The XrHandPoseTypeMSFT describes the type of input hand pose from XrHandTrackerEXT.

```
// Provided by XR_MSFT_hand_tracking_mesh
typedef enum XrHandPoseTypeMSFT {
    XR_HAND_POSE_TYPE_TRACKED_MSFT = 0,
    XR_HAND_POSE_TYPE_REFERENCE_OPEN_PALM_MSFT = 1,
    XR_HAND_POSE_TYPE_MAX_ENUM_MSFT = 0x7FFFFFFF
} XrHandPoseTypeMSFT;
```

Enumerant Descriptions

- XR_HAND_POSE_TYPE_TRACKED_MSFT represents a hand pose provided by actual tracking of the user's hand.
- XR_HAND_POSE_TYPE_REFERENCE_OPEN_PALM_MSFT represents a stable reference hand pose in a relaxed open hand shape.

The XR_HAND_POSE_TYPE_TRACKED_MSFT input provides best fidelity to the user's actual hand motion. When the hand tracking input requires the user to be holding a controller in their hand, the hand tracking input will appear as the user virtually holding the controller. This input can be used to render the hand shape together with the controller in hand.

The XR_HAND_POSE_TYPE_REFERENCE_OPEN_PALM_MSFT input does not move with the user's actual hand. Through this reference hand pose, an application **can** get a stable hand joint and mesh that has the same mesh topology as the tracked hand mesh using the same XrHandTrackerEXT, so that the application can apply the data computed from a reference hand pose to the corresponding tracked hand.

Although a reference hand pose does not move with user's hand motion, the bone length and hand thickness **may** be updated, for example when tracking result refines, or a different user's hand is detected. The application **should** update reference hand joints and meshes when the tracked mesh's indexBufferKey is changed or when the isActive value returned from xrUpdateHandMeshMSFT changes from XR_FALSE to XR_TRUE. It can use the returned indexBufferKey and vertexUpdateTime from xrUpdateHandMeshMSFT to avoid unnecessary CPU or GPU work to process the neutral hand inputs.

14.28.7. Example code for reference hand mesh update

The following example code demonstrates detecting reference hand mesh changes and retrieving data for processing.

```
XrInstance instance;                // previously initialized
XrSession session;                  // previously initialized
XrHandTrackerEXT handTracker;       // previously initialized with handJointSet set to
XR_HAND_JOINT_SET_DEFAULT_MSFT
XrSpace handMeshReferenceSpace;     // previously initialized with handPoseType set to
XR_HAND_POSE_TYPE_REFERENCE_OPEN_PALM_MSFT
```

```
XrHandMeshMSFT referenceHandMesh;   // previously initialized with preallocated buffers

// Get function pointer for xrUpdateHandMeshMSFT
PFN_xrUpdateHandMeshMSFT pfnUpdateHandMeshMSFT = nullptr;
CHK_XR(xrGetInstanceProcAddr(instance, "xrUpdateHandMeshMSFT",
                             reinterpret_cast<PFN_xrVoidFunction*>(
                             &pfnUpdateHandMeshMSFT)));

// Get function pointer for xrCreateHandTrackerEXT
PFN_xrCreateHandTrackerEXT pfnCreateHandTrackerEXT = nullptr;
CHK_XR(xrGetInstanceProcAddr(instance, "xrCreateHandTrackerEXT",
                             reinterpret_cast<PFN_xrVoidFunction*>(
                             &pfnCreateHandTrackerEXT)));

// Get function pointer for xrLocateHandJointsEXT
PFN_xrLocateHandJointsEXT pfnLocateHandJointsEXT = nullptr;
CHK_XR(xrGetInstanceProcAddr(instance, "xrLocateHandJointsEXT",
                             reinterpret_cast<PFN_xrVoidFunction*>(
                             &pfnLocateHandJointsEXT)));
while(1){
    // ...
    // For every frame in frame loop
    // ...
    XrFrameState frameState;    // previously returned from xrWaitFrame
    const XrTime time = frameState.predictedDisplayTime;

    XrHandMeshUpdateInfoMSFT updateInfo {
        .type = XR_TYPE_HAND_MESH_UPDATE_INFO_MSFT,
        .time = time,
        .handPoseType = XR_HAND_POSE_TYPE_REFERENCE_OPEN_PALM_MSFT,
    };
    CHK_XR(pfnUpdateHandMeshMSFT(handTracker, &updateInfo, &referenceHandMesh));

    // Detect if reference hand mesh is changed.
    if (referenceHandMesh.indexBufferChanged || referenceHandMesh.vertexBufferChanged) {

        // Query the joint location using "open palm" reference hand pose.
        XrHandPoseTypeInfoMSFT handPoseTypeInfo {
            .type = XR_TYPE_HAND_POSE_TYPE_INFO_MSFT,
            .handPoseType = XR_HAND_POSE_TYPE_REFERENCE_OPEN_PALM_MSFT,
        };

        XrHandTrackerCreateInfoEXT createInfo {
            .type = XR_TYPE_HAND_TRACKER_CREATE_INFO_EXT,
            .next = &handPoseTypeInfo,
            .hand = XR_HAND_LEFT_EXT,
            .handJointSet = XR_HAND_JOINT_SET_DEFAULT_EXT,
        };
```

```
        XrHandTrackerEXT referenceHandTracker {XR_NULL_HANDLE};
        CHK_XR(pfnCreateHandTrackerEXT(session, &createInfo, &referenceHandTracker));

        XrHandJointsLocateInfoEXT locateInfo {
            .type = XR_TYPE_HAND_JOINTS_LOCATE_INFO_EXT,
            .next = &handPoseTypeInfo,
            .baseSpace = handMeshReferenceSpace,  // Query joint location relative to
hand mesh reference space
            .time = time,
        };

        std::array<XrHandJointLocationEXT, XR_HAND_JOINT_COUNT_EXT> jointLocations;
        XrHandJointLocationsEXT locations {
            .type = XR_TYPE_HAND_JOINT_LOCATIONS_EXT,
            .jointCount = jointLocations.size(),
            .jointLocations = jointLocations.data(),
        };

        CHK_XR(pfnLocateHandJointsEXT(referenceHandTracker, &locateInfo, &locations));

        // Generate UV map using tip/wrist location and referenceHandMesh.vertexBuffer
        // For example, gradually changes color from the tip of the hand to wrist.
    }
}
```

New Object Types

New Flag Types

New Enum Constants

XrStructureType enumeration is extended with:

- XR_TYPE_HAND_MESH_SPACE_CREATE_INFO_MSFT
- XR_TYPE_HAND_MESH_UPDATE_INFO_MSFT
- XR_TYPE_HAND_MESH_MSFT
- XR_TYPE_SYSTEM_HAND_TRACKING_MESH_PROPERTIES_MSFT
- XR_TYPE_HAND_POSE_TYPE_INFO_MSFT

New Enums

- XrHandPoseTypeMSFT

New Structures

- XrHandMeshSpaceCreateInfoMSFT
- XrHandMeshUpdateInfoMSFT
- XrHandMeshMSFT
- XrHandMeshIndexBufferMSFT
- XrHandMeshVertexBufferMSFT
- XrHandMeshVertexMSFT
- XrSystemHandTrackingMeshPropertiesMSFT
- XrHandPoseTypeInfoMSFT

New Functions

- xrCreateHandMeshSpaceMSFT
- xrUpdateHandMeshMSFT

Issues

Version History

- Revision 1, 2019-09-20 (Yin LI)
  - Initial extension description
- Revision 2, 2020-04-20 (Yin LI)
  - Change joint spaces to locate joints function.
- Revision 3, 2021-04-13 (Rylie Pavlik, Collabora, Ltd.)
  - Correctly show function pointer retrieval in sample code
- Revision 4, 2021-10-20 (Darryl Gough)
  - Winding order for hand mesh is corrected to clockwise to match runtime behavior.

14.29. XR_MSFT_holographic_window_attachment

Name String

`XR_MSFT_holographic_window_attachment`

Extension Type

Instance extension

Registered Extension Number

64

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Deprecation State

- *Deprecated* without replacement

Contributors

Bryce Hutchings, Microsoft
Yin Li, Microsoft
Alex Turner, Microsoft

Overview

This extension enables the runtime to attach to app-provided HolographicSpace and CoreWindow WinRT objects when an XrSession is created. Applications may use this extension to create and control the CoreWindow/App View objects, allowing the app to subscribe to keyboard input events and react to activation event arguments. These events and data would otherwise be inaccessible if the application simply managed the app state and lifetime exclusively through the OpenXR API. This extension is only valid to use where an application can create a CoreWindow, such as UWP applications on the HoloLens.

The XrHolographicWindowAttachmentMSFT structure is defined as:

```
// Provided by XR_MSFT_holographic_window_attachment
typedef struct XrHolographicWindowAttachmentMSFT {
    XrStructureType    type;
    const void*        next;
    IUnknown*          holographicSpace;
    IUnknown*          coreWindow;
} XrHolographicWindowAttachmentMSFT;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `holographicSpace` is a pointer to a valid `Windows`.Graphics.Holographic.HolographicSpace.
- `coreWindow` is a pointer to a valid `Windows`.UI.Core.CoreWindow.

When creating a holographic window-backed XrSession, the application provides a pointer to an XrHolographicWindowAttachmentMSFT in the `next` chain of the XrSessionCreateInfo.

The session state of a holographic window-backed XrSession will only reach `XR_SESSION_STATE_VISIBLE` when the provided CoreWindow is made visible. If the CoreWindow is for a secondary app view, the application must programmatically request to make the CoreWindow visible (e.g. with `ApplicationViewSwitcher.TryShowAsStandaloneAsync` or `ApplicationViewSwitcher.SwitchAsync`).

The app **must** not call xrCreateSession while the specified CoreWindow thread is blocked, otherwise the call **may** deadlock.

Valid Usage (Implicit)

- The `XR_MSFT_holographic_window_attachment` extension **must** be enabled prior to using XrHolographicWindowAttachmentMSFT
- `type` **must** be `XR_TYPE_HOLOGRAPHIC_WINDOW_ATTACHMENT_MSFT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `holographicSpace` **must** be a pointer to an `IUnknown` value
- `coreWindow` **must** be a pointer to an `IUnknown` value

14.29.1. Sample code

Following example demos the usage of holographic window attachment and use the attached CoreWindow to receive keyboard input, use `CoreTextEditContext` to handle text typing experience, and use `IActivatedEventArgs` to handle protocol launching arguments.

```
struct AppView : implements<AppView, IFrameworkView> {
    void Initialize(CoreApplicationView const& applicationView) {
        applicationView.Activated({this, &AppView::OnActivated});
    }

    void Load(winrt::hstring const& entryPoint) {
    }

    void Uninitialize() {
    }

    void Run() {
        // Creating a HolographicSpace before activating the CoreWindow to make it a holographic window
        CoreWindow window = CoreWindow::GetForCurrentThread();
        HolographicSpace holographicSpace = Windows::Graphics::Holographic::HolographicSpace::CreateForCoreWindow(window);
```

```
        window.Activate();

        // [xrCreateInstance, xrGetSystem, and create a graphics binding]

        XrHolographicWindowAttachmentMSFT holographicWindowAttachment {
            .type = XR_TYPE_ATTACHED_CORE_WINDOW_MSFT,
            .next = &graphicsBinding,
            .coreWindow = window.as<IUnknown>().get(),
            .holographicSpace = holographicSpace.as<IUnknown>().get(),
        };

        XrSessionCreateInfo sessionCreateInfo {
            .type = XR_TYPE_SESSION_CREATE_INFO,
            .next = &holographicWindowAttachment,
            .systemId = systemId,
        };

        XrSession session {XR_NULL_HANDLE};
        CHECK_XRCMD(xrCreateSession(instance, &sessionCreateInfo, &session));

        while (!m_windowClosed) {
            window.Dispatcher().ProcessEvents(CoreProcessEventsOption
::ProcessAllIfPresent);

            // [OpenXR calls: Poll events, sync actions, render, and submit frames].
        }
    }

    void SetWindow(CoreWindow const& window) {
        window.Closed({this, &AppView::OnWindowClosed});
        window.KeyDown({this, &AppView::OnKeyDown});

        // This sample customizes the text input pane with manual display policy and
email address scope.
        windows::CoreTextServicesManager manager = windows::CoreTextServicesManager
::GetForCurrentView();
        windows::CoreTextEditContext editingContext = manager.CreateEditContext();
        editingContext.InputPaneDisplayPolicy(windows::
CoreTextInputPaneDisplayPolicy::Manual);
        editingContext.InputScope(windows::CoreTextInputScope::EmailAddress);
    }

    void OnWindowClosed(CoreWindow const& sender, CoreWindowEventArgs const& args) {
        m_windowClosed = true;
    }

    void OnKeyDown(CoreWindow const& sender, KeyEventArgs const& args) {
        // [Process key down]
```

```
60      }
61
62      void OnActivated(CoreApplicationView const&, IActivatedEventArgs const& args) {
63          if (args.Kind() == windows::ActivationKind::Protocol) {
64              auto eventArgs{args.as<windows::ProtocolActivatedEventArgs>()};
65              // Use the protocol activation parameters in eventArgs.Uri();
66          }
67
68          // Inspecting whether the application is launched from within holographic
    shell or from desktop.
69          if (windows::HolographicApplicationPreview::IsHolographicActivation(args)) {
70              // App activation is targeted at the holographic shell.
71          } else {
72              // App activation is targeted at the desktop.
73          }
74
75          // NOTE: CoreWindow is activated later after the HolographicSpace has been
    created.
76      }
77
78      bool m_windowClosed{false};
79  };
80
81  struct AppViewSource : winrt::implements<AppViewSource, IFrameworkViewSource> {
82      windows::IFrameworkView CreateView() {
83          return winrt::make<AppView>();
84      }
85  };
86
87  int __stdcall wWinMain(HINSTANCE, HINSTANCE, PWSTR, int) {
88      CoreApplication::Run(make<AppViewSource>());
89  }
```

Version History

- Revision 1, 2020-05-18 (Bryce Hutchings)
  - Initial extension description

14.30. XR_MSFT_perception_anchor_interop

Name String

`XR_MSFT_perception_anchor_interop`

Extension Type

Instance extension

Registered Extension Number

57

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0
and
XR_MSFT_spatial_anchor

Deprecation State

- *Deprecated* without replacement

Last Modified Date

2020-06-16

IP Status

No known IP claims.

Contributors

Lachlan Ford, Microsoft
Bryce Hutchings, Microsoft
Yin Li, Microsoft

Overview

This extension supports conversion between XrSpatialAnchorMSFT and Windows.Perception.Spatial.SpatialAnchor. An application **can** use this extension to persist spatial anchors on the Windows device through SpatialAnchorStore or transfer spatial anchors between devices through SpatialAnchorTransferManager.

The xrCreateSpatialAnchorFromPerceptionAnchorMSFT function creates a XrSpatialAnchorMSFT handle from an IUnknown pointer to Windows.Perception.Spatial.SpatialAnchor.

```
// Provided by XR_MSFT_perception_anchor_interop
XrResult xrCreateSpatialAnchorFromPerceptionAnchorMSFT(
    XrSession                                   session,
    IUnknown*                                   perceptionAnchor,
    XrSpatialAnchorMSFT*                        anchor);
```

Parameter Descriptions

- `session` is the specified XrSession.
- `perceptionAnchor` is an IUnknown pointer to a Windows.Perception.Spatial.SpatialAnchor object.
- `anchor` is a pointer to XrSpatialAnchorMSFT to receive the returned anchor handle.

The input `perceptionAnchor` **must** support successful `QueryInterface` to Windows.Perception.Spatial.SpatialAnchor , otherwise the runtime **must** return `XR_ERROR_VALIDATION_FAILURE`.

If the function successfully returned, the output `anchor` **must** be a valid handle. This also increments the refcount of the `perceptionAnchor` object.

When application is done with the `anchor` handle, it **can** be destroyed using xrDestroySpatialAnchorMSFT function. This also decrements the refcount of underlying windows perception anchor object.

Valid Usage (Implicit)

- The `XR_MSFT_perception_anchor_interop` extension **must** be enabled prior to calling xrCreateSpatialAnchorFromPerceptionAnchorMSFT
- `session` **must** be a valid XrSession handle
- `perceptionAnchor` **must** be a pointer to an `IUnknown` value
- `anchor` **must** be a pointer to an XrSpatialAnchorMSFT handle

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The xrTryGetPerceptionAnchorFromSpatialAnchorMSFT function converts a XrSpatialAnchorMSFT handle into an IUnknown pointer to Windows.Perception.Spatial.SpatialAnchor.

```
// Provided by XR_MSFT_perception_anchor_interop
XrResult xrTryGetPerceptionAnchorFromSpatialAnchorMSFT(
    XrSession                                   session,
    XrSpatialAnchorMSFT                         anchor,
    IUnknown**                                  perceptionAnchor);
```

Parameter Descriptions

- session is the specified XrSession.
- anchor is a valid XrSpatialAnchorMSFT handle.
- perceptionAnchor is a valid pointer to IUnknown pointer to receive the output Windows.Perception.Spatial.SpatialAnchor object.

If the runtime can convert the anchor to a Windows.Perception.Spatial.SpatialAnchor object, this function **must** return XR_SUCCESS, and the output IUnknown in the pointer of perceptionAnchor **must** be not NULL. This also increments the refcount of the object. The application **can** then use QueryInterface to get the pointer for Windows.Perception.Spatial.SpatialAnchor object. The application **should** release the COM pointer after done with the object, or attach it to a smart COM pointer such as winrt::com_ptr.

If the runtime cannot convert the `anchor` to a Windows.Perception.Spatial.SpatialAnchor object, the function **must** return `XR_SUCCESS`, and the output `IUnknown` in the pointer of `perceptionAnchor` **must** be `NULL`.

Valid Usage (Implicit)

- The `XR_MSFT_perception_anchor_interop` extension **must** be enabled prior to calling xrTryGetPerceptionAnchorFromSpatialAnchorMSFT
- `session` **must** be a valid XrSession handle
- `anchor` **must** be a valid XrSpatialAnchorMSFT handle
- `perceptionAnchor` **must** be a pointer to a pointer to an `IUnknown` value
- `anchor` **must** have been created, allocated, or retrieved from `session`

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_OUT_OF_MEMORY`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_VALIDATION_FAILURE`

New Object Types

New Flag Types

New Enum Constants

New Enums

New Structures

New Functions

xrCreateSpatialAnchorFromPerceptionAnchorMSFT

xrTryGetPerceptionAnchorFromSpatialAnchorMSFT

Issues

Version History

- Revision 1, 2020-06-16 (Yin Li)
  - Initial extension proposal

14.31. XR_MSFT_scene_marker

Name String

`XR_MSFT_scene_marker`

Extension Type

Instance extension

Registered Extension Number

148

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0
and
`XR_MSFT_scene_understanding`

Deprecation State

- *Obsoleted* by `XR_EXT_spatial_marker_tracking` extension

Contributors

Alain Zanchetta, Microsoft
Yin Li, Microsoft
Alex Turner, Microsoft

14.31.1. Overview

This extension enables the application to observe the tracked markers, such as the QR Code markers in ISO/IEC 18004:2015. This extension also enables future extensions to easily add new types of marker

tracking.

The application **must** enable both XR_MSFT_scene_marker and XR_MSFT_scene_understanding in order to use this extension.

Note

A typical use of this extension is:

1. Verify if marker detection is supported by calling xrEnumerateSceneComputeFeaturesMSFT and validate that the returned supported features include XR_SCENE_COMPUTE_FEATURE_MARKER_MSFT.
2. If supported, create an XrSceneObserverMSFT handle.
3. Pass in XR_SCENE_COMPUTE_FEATURE_MARKER_MSFT as requested feature when starting the scene compute by calling xrComputeNewSceneMSFT function.
4. Inspect the completion of computation by polling xrGetSceneComputeStateMSFT.
5. Once compute is successfully completed, create an XrSceneMSFT handle to the result by calling xrCreateSceneMSFT.
6. Get the list of detected markers using xrGetSceneComponentsMSFT:
   - optionally: filter the type of the returned markers using XrSceneMarkerTypeFilterMSFT.
   - optionally: retrieve additional marker properties by chaining XrSceneMarkersMSFT and/or XrSceneMarkerQRCodesMSFT to the next pointer of XrSceneComponentsMSFT.
7. Get the data encoded in a marker using xrGetSceneMarkerDecodedStringMSFT or xrGetSceneMarkerRawDataMSFT.
8. Locate markers using xrLocateSceneComponentsMSFT.

14.31.2. Retrieve marker properties

The XrSceneMarkersMSFT structure is defined as:

```
// Provided by XR_MSFT_scene_marker
typedef struct XrSceneMarkersMSFT {
    XrStructureType       type;
    const void*           next;
    uint32_t              sceneMarkerCapacityInput;
    XrSceneMarkerMSFT*    sceneMarkers;
} XrSceneMarkersMSFT;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. See also: XrSceneComponentsMSFT, XrSceneMarkerQRCodesMSFT
- `sceneMarkerCapacityInput` is a `uint32_t` indicating the capacity of elements in the `sceneMarkers` array.
- `sceneMarkers` is an array of XrSceneMarkerMSFT to fill with the properties of the markers.

Once the application creates an XrSceneMSFT after a successful scene compute, it **can** retrieve the scene markers' properties by chaining XrSceneMarkersMSFT structure to the next pointer of XrSceneComponentsGetInfoMSFT when calling xrGetSceneComponentsMSFT.

xrGetSceneComponentsMSFT follows the two-call idiom for filling the XrSceneComponentsMSFT structure to which an XrSceneMarkersMSFT structure **can** be chained.

The input `sceneMarkerCapacityInput` **must** be equal to or greater than the corresponding XrSceneComponentsMSFT::componentCapacityInput, otherwise the runtime **must** return `XR_ERROR_SIZE_INSUFFICIENT`.

The actual count of elements returned in the array `sceneMarkers` is consistent with the extended XrSceneComponentsMSFT structure and returned in XrSceneComponentsMSFT::`componentCountOutput`.

Valid Usage (Implicit)

- The `XR_MSFT_scene_marker` extension **must** be enabled prior to using XrSceneMarkersMSFT
- `type` **must** be `XR_TYPE_SCENE_MARKERS_MSFT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- If `sceneMarkerCapacityInput` is not `0`, `sceneMarkers` **must** be a pointer to an array of `sceneMarkerCapacityInput` XrSceneMarkerMSFT structures

The XrSceneMarkerMSFT structure is defined as:

```
// Provided by XR_MSFT_scene_marker
typedef struct XrSceneMarkerMSFT {
    XrSceneMarkerTypeMSFT    markerType;
    XrTime                   lastSeenTime;
    XrOffset2Df              center;
    XrExtent2Df              size;
} XrSceneMarkerMSFT;
```

Member Descriptions

- `markerType` is an XrSceneMarkerTypeMSFT indicating the type of the marker.
- `lastSeenTime` is an `XrTime` indicating when the marker was seen last.
- `center` is an XrOffset2Df structure representing the location of the center of the axis-aligned bounding box of the marker in the XY plane of the marker's coordinate system.
- `size` is an XrExtent2Df structure representing the width and height of the axis-aligned bounding box of the marker in the XY plane of the marker's coordinate system.

The XrSceneMarkerMSFT structure is an element in the array of XrSceneMarkersMSFT::`sceneMarkers`.

Refer to the QR code convention for an example of marker's center and size in the context of a QR code.

When the runtime updates the location or properties of an observed marker, the runtime **must** set the XrSceneMarkerMSFT::`lastSeenTime` to the new timestamp of the update.

When the runtime cannot observe a previously observed XrSceneMarkerMSFT, the runtime **must** keep the previous `lastSeenTime` for the marker. Hence, the application **can** use the `lastSeenTime` to know how fresh the tracking information is for a given marker.

The `center` and `size` are measured in meters, relative to the XrPosef of the marker for the visual bound of the marker in XY plane, regardless of the marker type.

Valid Usage (Implicit)

- The `XR_MSFT_scene_marker` extension **must** be enabled prior to using XrSceneMarkerMSFT

The XrSceneMarkerTypeFilterMSFT structure is defined as:

```
// Provided by XR_MSFT_scene_marker
typedef struct XrSceneMarkerTypeFilterMSFT {
    XrStructureType           type;
    const void*               next;
    uint32_t                  markerTypeCount;
    XrSceneMarkerTypeMSFT*    markerTypes;
} XrSceneMarkerTypeFilterMSFT;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `markerTypeCount` is a `uint32_t` indicating the count of elements in the `markerTypes` array.
- `markerTypes` is an array of XrSceneMarkerTypeMSFT indicating the types of markers to return.

The application **can** filter the returned scene components to specific marker types by chaining XrSceneMarkerTypeFilterMSFT to the `next` pointer of XrSceneComponentsGetInfoMSFT when calling xrGetSceneComponentsMSFT.

When XrSceneMarkerTypeFilterMSFT is provided to xrGetSceneComponentsMSFT, the runtime **must** only return scene components that match the requested types.

The application **must** provide a non-empty array of unique `markerTypes`, i.e. the `markerTypeCount` **must** be positive and the elements in the `markerTypes` array **must** not have duplicated values. Otherwise, the runtime **must** return `XR_ERROR_VALIDATION_FAILURE` for xrGetSceneComponentsMSFT function.

Valid Usage (Implicit)

- The `XR_MSFT_scene_marker` extension **must** be enabled prior to using XrSceneMarkerTypeFilterMSFT
- `type` **must** be `XR_TYPE_SCENE_MARKER_TYPE_FILTER_MSFT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- If `markerTypeCount` is not `0`, `markerTypes` **must** be a pointer to an array of `markerTypeCount` XrSceneMarkerTypeMSFT values

The XrSceneMarkerTypeMSFT identifies the type of a scene marker.

```
// Provided by XR_MSFT_scene_marker
typedef enum XrSceneMarkerTypeMSFT {
    XR_SCENE_MARKER_TYPE_QR_CODE_MSFT = 1,
    XR_SCENE_MARKER_TYPE_MAX_ENUM_MSFT = 0x7FFFFFFF
} XrSceneMarkerTypeMSFT;
```

Enumerant Descriptions

- `XR_SCENE_MARKER_TYPE_QR_CODE_MSFT` represents a marker that follows the ISO standard for QR code in ISO/IEC 18004:2015.

14.31.3. Locate markers

Applications **can** use xrLocateSceneComponentsMSFT to locate an XrSceneMarkerMSFT.

The scene marker's locations are snapshots of the XrSceneMSFT, that do not change for the lifecycle of the result. To get updated tracking, the application **can** issue another xrComputeNewSceneMSFT and obtain a new XrSceneMSFT. The application **can** use the XrSceneComponentMSFT::`id` to correlate the same marker across multiple scene computes.

The pose and geometry of scene markers returned from this extension follows these general conventions:

- The marker image reside in the plane of X and Y axes.
- Z axis is perpendicular to the X and Y axes and follows the right hand rule. +Z is pointing into the marker image.
- The origin of the marker is runtime defined for the specific XrSceneMarkerTypeMSFT, and it typically represents the most stable and accurate point for tracking the marker. This allows the application to use the marker as a tracked point.
- In cases where the origin does not necessarily coincide with the center of the marker geometry, applications can obtain additional geometry information from the XrSceneMarkerMSFT structure. This information includes the center and size of the marker image in the X and Y plane.

The exact origin and geometry properties relative to the tracked marker image in physical world **must** be well defined and consistent for each XrSceneMarkerTypeMSFT, including the new marker types defined in future extensions.

14.31.4. The convention of QRCode marker location

For a marker with `XR_SCENE_MARKER_TYPE_QR_CODE_MSFT`, the origin is at the top left corner of the QR code image, where the orientation of the QR code image in the XY plane follows the convention in ISO/IEC 18004:2015. The X axis of QR code pose points to the right of the marker image, and the Z axis points inward to the marker image, as illustrated in following image.

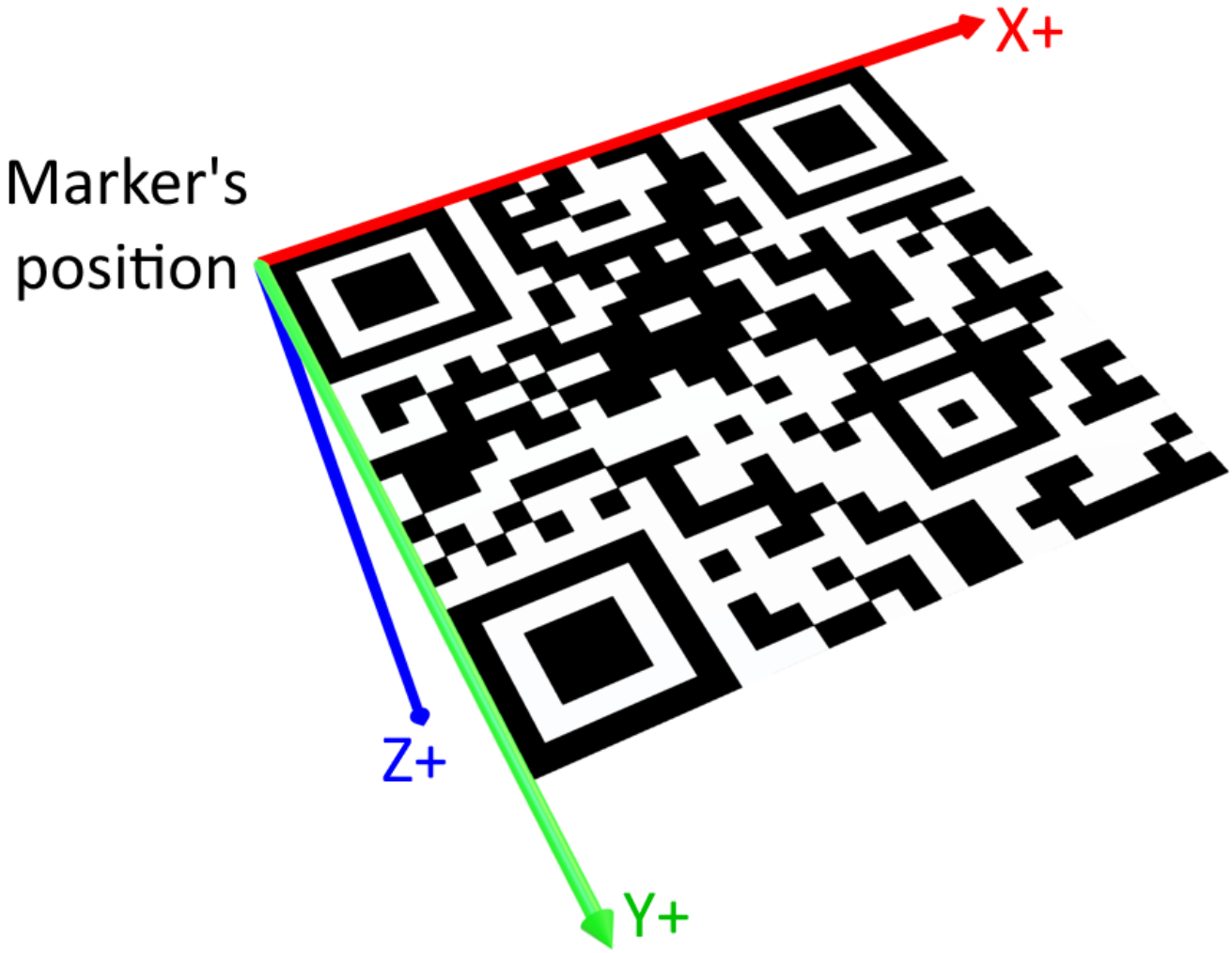


Figure 41. The pose convention of a QR code marker.

The QR Code marker's center and size are defined in the XY plane, as illustrated in following pictures.

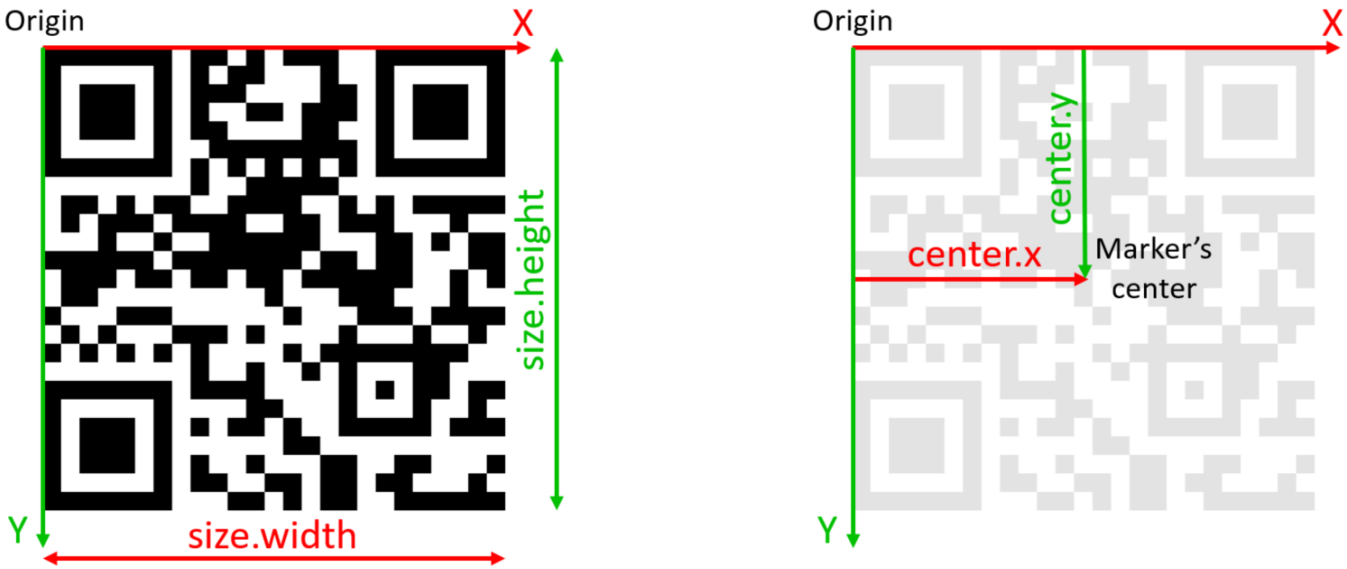


Figure 42. The center and size of QR Code marker.

14.31.5. Retrieving QRCode marker properties

The XrSceneMarkerQRCodesMSFT structure is defined as:

```
// Provided by XR_MSFT_scene_marker
typedef struct XrSceneMarkerQRCodesMSFT {
    XrStructureType             type;
    const void*                 next;
    uint32_t                    qrCodeCapacityInput;
    XrSceneMarkerQRCodeMSFT*    qrCodes;
} XrSceneMarkerQRCodesMSFT;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. See also: XrSceneComponentsMSFT, XrSceneMarkersMSFT
- `qrCodeCapacityInput` is a `uint32_t` indicating the count of elements in the `qrCodes` array.
- `qrCodes` is an array of XrSceneMarkerQRCodeMSFT for the runtime to fill with the properties of the QR Codes.

An XrSceneMarkerQRCodesMSFT structure can be chained to the `next` pointer of XrSceneComponentsMSFT when calling xrGetSceneComponentsMSFT function to retrieve the QR Code specific properties through an array of XrSceneMarkerQRCodeMSFT structures.

xrGetSceneComponentsMSFT follows the two-call idiom for filling the XrSceneComponentsMSFT structure to which an XrSceneMarkerQRCodesMSFT structure **can** be chained.

The `qrCodeCapacityInput` **must** be equal to or greater than the corresponding XrSceneComponentsMSFT::componentCapacityInput, otherwise the runtime **must** return the success code `XR_ERROR_SIZE_INSUFFICIENT` from xrGetSceneComponentsMSFT.

The actual count of elements returned in the array `qrCodes` is consistent to the extended XrSceneComponentsMSFT structure and returned in XrSceneComponentsMSFT::componentCountOutput.

Valid Usage (Implicit)

- The XR_MSFT_scene_marker extension **must** be enabled prior to using XrSceneMarkerQRCodesMSFT
- type **must** be XR_TYPE_SCENE_MARKER_QR_CODES_MSFT
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- If qrCodeCapacityInput is not 0, qrCodes **must** be a pointer to an array of qrCodeCapacityInput XrSceneMarkerQRCodeMSFT structures

The XrSceneMarkerQRCodeMSFT structure is defined as:

```
// Provided by XR_MSFT_scene_marker
typedef struct XrSceneMarkerQRCodeMSFT {
    XrSceneMarkerQRCodeSymbolTypeMSFT    symbolType;
    uint8_t                              version;
} XrSceneMarkerQRCodeMSFT;
```

Member Descriptions

- symbolType is an XrSceneMarkerQRCodeSymbolTypeMSFT indicating the symbol type of the QR Code.
- version is a uint8_t indicating the version of the QR Code

The XrSceneMarkerQRCodeMSFT structure contains the detailed QR Code symbol type and version according to ISO/IEC 18004:2015. The version **must** be in the range 1 to 40 inclusively for a QR Code and 1 to 4 inclusively for a Micro QR Code.

Valid Usage (Implicit)

- The XR_MSFT_scene_marker extension **must** be enabled prior to using XrSceneMarkerQRCodeMSFT

```
// Provided by XR_MSFT_scene_marker
typedef enum XrSceneMarkerQRCodeSymbolTypeMSFT {
    XR_SCENE_MARKER_QR_CODE_SYMBOL_TYPE_QR_CODE_MSFT = 1,
    XR_SCENE_MARKER_QR_CODE_SYMBOL_TYPE_MICRO_QR_CODE_MSFT = 2,
    XR_SCENE_MARKER_QRCODE_SYMBOL_TYPE_MAX_ENUM_MSFT = 0x7FFFFFFF
} XrSceneMarkerQRCodeSymbolTypeMSFT;
```

The XrSceneMarkerQRCodeSymbolTypeMSFT identifies the symbol type of the QR Code.

Enumerant Descriptions

- XR_SCENE_MARKER_QR_CODE_SYMBOL_TYPE_QR_CODE_MSFT if the marker is a QR Code.
- XR_SCENE_MARKER_QR_CODE_SYMBOL_TYPE_MICRO_QR_CODE_MSFT if the marker is a Micro QR Code.

The xrGetSceneMarkerDecodedStringMSFT function is defined as:

```
// Provided by XR_MSFT_scene_marker
XrResult xrGetSceneMarkerDecodedStringMSFT(
    XrSceneMSFT                                 scene,
    const XrUuidMSFT*                           markerId,
    uint32_t                                    bufferCapacityInput,
    uint32_t*                                   bufferCountOutput,
    char*                                       buffer);
```

Parameter Descriptions

- scene is an XrSceneMSFT previously created by xrCreateSceneMSFT.
- markerId is an XrUuidMSFT identifying the marker, returned previously from XrSceneComponentMSFT::id when calling xrGetSceneComponentsMSFT.
- bufferCapacityInput is the capacity of the string buffer, or 0 to indicate a request to retrieve the required capacity.
- bufferCountOutput is a pointer to the count of characters written (including the terminating '\0'), or a pointer to the required capacity in the case that bufferCapacityInput is insufficient.
- buffer is a pointer to an application-allocated buffer that will be filled with the string stored in the QR Code. It can be NULL if bufferCapacityInput is 0.
- See the Buffer Size Parameters section for a detailed description of retrieving the required buffer size.

The xrGetSceneMarkerDecodedStringMSFT function retrieves the string stored in the scene marker as an UTF-8 string, including the terminating '\0'. This function follows the two-call idiom for filling the buffer array.

If the stored data in the marker is not an encoded string, the runtime **must** return the success code XR_SCENE_MARKER_DATA_NOT_STRING_MSFT, set bufferCountOutput to 1, and make buffer an empty string.

Valid Usage (Implicit)

- The `XR_MSFT_scene_marker` extension **must** be enabled prior to calling xrGetSceneMarkerDecodedStringMSFT
- `scene` **must** be a valid XrSceneMSFT handle
- `markerId` **must** be a pointer to a valid XrUuidMSFT structure
- `bufferCountOutput` **must** be a pointer to a `uint32_t` value
- If `bufferCapacityInput` is not `0`, `buffer` **must** be a pointer to an array of `bufferCapacityInput` char values

Return Codes

Success

- `XR_SUCCESS`
- `XR_SCENE_MARKER_DATA_NOT_STRING_MSFT`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_OUT_OF_MEMORY`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SCENE_COMPONENT_ID_INVALID_MSFT`
- `XR_ERROR_SCENE_COMPONENT_TYPE_MISMATCH_MSFT`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_SIZE_INSUFFICIENT`
- `XR_ERROR_VALIDATION_FAILURE`

The xrGetSceneMarkerRawDataMSFT function is defined as:

```
// Provided by XR_MSFT_scene_marker
XrResult xrGetSceneMarkerRawDataMSFT(
    XrSceneMSFT                                 scene,
    const XrUuidMSFT*                           markerId,
    uint32_t                                    bufferCapacityInput,
    uint32_t*                                   bufferCountOutput,
    uint8_t*                                    buffer);
```

Parameter Descriptions

- `scene` is an XrSceneMSFT previously created by xrCreateSceneMSFT.
- `markerId` is an XrUuidMSFT identifying the marker, and it is returned previous from XrSceneComponentMSFT when calling xrGetSceneComponentsMSFT.
- `bufferCapacityInput` is the capacity of the buffer, or 0 to indicate a request to retrieve the required capacity.
- `bufferCountOutput` is a pointer to the count of bytes written, or a pointer to the required capacity in the case that bufferCapacityInput is insufficient.
- `buffer` is a pointer to an application-allocated buffer that will be filled with the data stored in the QR Code. It can be NULL if bufferCapacityInput is 0.
- See the Buffer Size Parameters section for a detailed description of retrieving the required `buffer` size.

The xrGetSceneMarkerRawDataMSFT function retrieves the data stored in the scene marker.

Valid Usage (Implicit)

- The `XR_MSFT_scene_marker` extension **must** be enabled prior to calling xrGetSceneMarkerRawDataMSFT
- `scene` **must** be a valid XrSceneMSFT handle
- `markerId` **must** be a pointer to a valid XrUuidMSFT structure
- `bufferCountOutput` **must** be a pointer to a `uint32_t` value
- If `bufferCapacityInput` is not `0`, `buffer` **must** be a pointer to an array of `bufferCapacityInput` `uint8_t` values

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SCENE_COMPONENT_ID_INVALID_MSFT
- XR_ERROR_SCENE_COMPONENT_TYPE_MISMATCH_MSFT
- XR_ERROR_SESSION_LOST
- XR_ERROR_SIZE_INSUFFICIENT
- XR_ERROR_VALIDATION_FAILURE

New Object Types

New Flag Types

New Enum Constants

XrSceneComputeFeatureMSFT enumeration is extended with:

- XR_SCENE_COMPUTE_FEATURE_MARKER_MSFT

XrSceneComponentTypeMSFT enumeration is extended with:

- XR_SCENE_COMPONENT_TYPE_MARKER_MSFT

XrStructureType enumeration is extended with:

- XR_TYPE_SCENE_MARKERS_MSFT
- XR_TYPE_SCENE_MARKER_TYPE_FILTER_MSFT
- XR_TYPE_SCENE_MARKER_QR_CODES_MSFT

XrResult enumeration is extended with:

- XR_SCENE_MARKER_DATA_NOT_STRING_MSFT

New Enums

- XrSceneMarkerTypeMSFT
- XrSceneMarkerQRCodeSymbolTypeMSFT

New Structures

- XrSceneMarkerMSFT
- XrSceneMarkersMSFT
- XrSceneMarkerTypeFilterMSFT
- XrSceneMarkerQRCodeMSFT
- XrSceneMarkerQRCodesMSFT

New Functions

- xrGetSceneMarkerRawDataMSFT
- xrGetSceneMarkerDecodedStringMSFT

Version History

- Revision 1, 2023-01-11 (Alain Zanchetta)
  - Initial extension description

14.32. XR_MSFT_scene_understanding

Name String

`XR_MSFT_scene_understanding`

Extension Type

Instance extension

Registered Extension Number

98

Revision

2

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Deprecation State

- *Obsoleted* by `XR_EXT_spatial_entity` extension

Last Modified Date

2021-05-03

IP Status

No known IP claims.

Contributors

Darryl Gough, Microsoft
Yin Li, Microsoft
Bryce Hutchings, Microsoft
Alex Turner, Microsoft
Simon Stachniak, Microsoft
David Fields, Microsoft

Overview

Scene understanding provides applications with a structured, high-level representation of the planes, meshes, and objects in the user's environment, enabling the development of spatially-aware applications.

The application requests computation of a scene, receiving the list of scene components observed in the environment around the user. These scene components contain information such as:

- The type of the discovered objects (wall, floor, ceiling, or other surface type).
- The planes and their bounds that represent the object.
- The visual and collider triangle meshes that represent the object.

The application can use this information to reason about the structure and location of the environment, to place holograms on surfaces, or render clues for grounding objects.

An application typically uses this extension in the following steps:

1. Create an XrSceneObserverMSFT handle to manage the system resource of the scene understanding compute.
2. Start the scene compute by calling xrComputeNewSceneMSFT with XrSceneBoundsMSFT to specify the scan range and a list of XrSceneComputeFeatureMSFT features.
3. Inspect the completion of computation by polling xrGetSceneComputeStateMSFT.
4. Once compute is completed, create an XrSceneMSFT handle to the result by calling xrCreateSceneMSFT.
5. Get properties of scene components using xrGetSceneComponentsMSFT.

6. Locate scene components using xrLocateSceneComponentsMSFT.

Create a scene observer handle

The XrSceneObserverMSFT handle represents the resources for computing scenes. It maintains a correlation of scene component identifiers across multiple scene computes.



Note

The application should destroy the XrSceneObserverMSFT handle when it is done with scene compute and scene component data to save system power consumption.

```
// Provided by XR_MSFT_scene_understanding

            XR_DEFINE_HANDLE(XrSceneObserverMSFT)
```

An XrSceneObserverMSFT handle is created using xrCreateSceneObserverMSFT.

```
// Provided by XR_MSFT_scene_understanding
XrResult xrCreateSceneObserverMSFT(
    XrSession                                   session,
    const XrSceneObserverCreateInfoMSFT*        createInfo,
    XrSceneObserverMSFT*                        sceneObserver);
```

Parameter Descriptions

- `session` is an XrSession in which the scene observer will be active.
- `createInfo` exists for extensibility purposes, it is `NULL` or a pointer to a valid XrSceneObserverCreateInfoMSFT structure.
- `sceneObserver` is the returned XrSceneObserverMSFT handle.

Valid Usage (Implicit)

- The `XR_MSFT_scene_understanding` extension **must** be enabled prior to calling xrCreateSceneObserverMSFT
- `session` **must** be a valid XrSession handle
- If `createInfo` is not `NULL`, `createInfo` **must** be a pointer to a valid XrSceneObserverCreateInfoMSFT structure
- `sceneObserver` **must** be a pointer to an XrSceneObserverMSFT handle

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The XrSceneObserverCreateInfoMSFT structure is defined as:

```
// Provided by XR_MSFT_scene_understanding
typedef struct XrSceneObserverCreateInfoMSFT {
    XrStructureType    type;
    const void*        next;
} XrSceneObserverCreateInfoMSFT;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.

Valid Usage (Implicit)

- The XR_MSFT_scene_understanding extension **must** be enabled prior to using XrSceneObserverCreateInfoMSFT
- type **must** be XR_TYPE_SCENE_OBSERVER_CREATE_INFO_MSFT
- next **must** be NULL or a valid pointer to the next structure in a structure chain

The xrDestroySceneObserverMSFT function releases the sceneObserver and the underlying resources.

```
// Provided by XR_MSFT_scene_understanding
XrResult xrDestroySceneObserverMSFT(
    XrSceneObserverMSFT                         sceneObserver);
```

Parameter Descriptions

- sceneObserver is an XrSceneObserverMSFT previously created by xrCreateSceneObserverMSFT.

Valid Usage (Implicit)

- The XR_MSFT_scene_understanding extension **must** be enabled prior to calling xrDestroySceneObserverMSFT
- sceneObserver **must** be a valid XrSceneObserverMSFT handle

Thread Safety

- Access to sceneObserver, and any child handles, **must** be externally synchronized

Return Codes

Success

- XR_SUCCESS

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_RUNTIME_FAILURE

Compute a new scene and wait for completion

The xrComputeNewSceneMSFT function begins the compute of a new scene and the runtime **must** return quickly without waiting for the compute to complete. The application **should** use xrGetSceneComputeStateMSFT to inspect the compute status.

The application **can** control the compute features by passing a list of XrSceneComputeFeatureMSFT via XrNewSceneComputeInfoMSFT::requestedFeatures.

- If XR_SCENE_COMPUTE_FEATURE_PLANE_MSFT is passed, but XR_SCENE_COMPUTE_FEATURE_PLANE_MESH_MSFT is not passed, then:
  - The application **may** be able to read XR_SCENE_COMPONENT_TYPE_PLANE_MSFT and XR_SCENE_COMPONENT_TYPE_OBJECT_MSFT scene components from the resulting XrSceneMSFT handle.
  - XrScenePlaneMSFT::meshBufferId **must** be zero to indicate that the plane scene component does not have a mesh buffer available to read.
- If XR_SCENE_COMPUTE_FEATURE_PLANE_MSFT and XR_SCENE_COMPUTE_FEATURE_PLANE_MESH_MSFT are passed, then:
  - the application **may** be able to read XR_SCENE_COMPONENT_TYPE_PLANE_MSFT and XR_SCENE_COMPONENT_TYPE_OBJECT_MSFT scene components from the resulting XrSceneMSFT handle
  - XrScenePlaneMSFT::meshBufferId **may** contain a non-zero mesh buffer identifier to indicate that the plane scene component has a mesh buffer available to read.
- If XR_SCENE_COMPUTE_FEATURE_VISUAL_MESH_MSFT is passed then:
  - the application **may** be able to read XR_SCENE_COMPONENT_TYPE_VISUAL_MESH_MSFT and XR_SCENE_COMPONENT_TYPE_OBJECT_MSFT scene components from the resulting XrSceneMSFT handle.
- If XR_SCENE_COMPUTE_FEATURE_COLLIDER_MESH_MSFT is passed then:
  - the application **may** be able to read XR_SCENE_COMPONENT_TYPE_COLLIDER_MESH_MSFT and XR_SCENE_COMPONENT_TYPE_OBJECT_MSFT scene components from the resulting XrSceneMSFT handle.

```
// Provided by XR_MSFT_scene_understanding
XrResult xrComputeNewSceneMSFT(
    XrSceneObserverMSFT                         sceneObserver,
    const XrNewSceneComputeInfoMSFT*            computeInfo);
```

Parameter Descriptions

- sceneObserver is a handle to an XrSceneObserverMSFT.
- computeInfo is a pointer to an XrNewSceneComputeInfoMSFT structure.

The runtime **must** return XR_ERROR_SCENE_COMPUTE_FEATURE_INCOMPATIBLE_MSFT if incompatible features were passed or no compatible features were passed.

The runtime **must** return XR_ERROR_SCENE_COMPUTE_FEATURE_INCOMPATIBLE_MSFT if XR_SCENE_COMPUTE_FEATURE_PLANE_MESH_MSFT was passed but XR_SCENE_COMPUTE_FEATURE_PLANE_MSFT was not passed.

The runtime **must** return XR_ERROR_COMPUTE_NEW_SCENE_NOT_COMPLETED_MSFT if xrComputeNewSceneMSFT is called while the scene computation is in progress.

An application that wishes to use XR_SCENE_COMPUTE_CONSISTENCY_OCCLUSION_OPTIMIZED_MSFT **must** create an XrSceneObserverMSFT handle that passes neither XR_SCENE_COMPUTE_CONSISTENCY_SNAPSHOT_COMPLETE_MSFT nor XR_SCENE_COMPUTE_CONSISTENCY_SNAPSHOT_INCOMPLETE_FAST_MSFT to xrComputeNewSceneMSFT for the lifetime of that XrSceneObserverMSFT handle. This allows the runtime to return occlusion mesh at a different cadence than non-occlusion mesh or planes.

- The runtime **must** return XR_ERROR_SCENE_COMPUTE_CONSISTENCY_MISMATCH_MSFT if:
  - XR_SCENE_COMPUTE_CONSISTENCY_OCCLUSION_OPTIMIZED_MSFT is passed to xrComputeNewSceneMSFT **and**
  - a previous call to xrComputeNewSceneMSFT did not pass XR_SCENE_COMPUTE_CONSISTENCY_OCCLUSION_OPTIMIZED_MSFT for the same XrSceneObserverMSFT handle.
- The runtime **must** return XR_ERROR_SCENE_COMPUTE_CONSISTENCY_MISMATCH_MSFT if:
  - XR_SCENE_COMPUTE_CONSISTENCY_OCCLUSION_OPTIMIZED_MSFT is not passed to xrComputeNewSceneMSFT **and**
  - a previous call to xrComputeNewSceneMSFT did pass XR_SCENE_COMPUTE_CONSISTENCY_OCCLUSION_OPTIMIZED_MSFT for the same XrSceneObserverMSFT handle.
- The runtime **must** return XR_ERROR_SCENE_COMPUTE_FEATURE_INCOMPATIBLE_MSFT if:
  - XR_SCENE_COMPUTE_CONSISTENCY_OCCLUSION_OPTIMIZED_MSFT is passed to xrComputeNewSceneMSFT **and**
  - neither XR_SCENE_COMPUTE_FEATURE_VISUAL_MESH_MSFT nor XR_SCENE_COMPUTE_FEATURE_COLLIDER_MESH_MSFT are also passed.
- The runtime **must** return XR_ERROR_SCENE_COMPUTE_FEATURE_INCOMPATIBLE_MSFT if:
  - XR_SCENE_COMPUTE_CONSISTENCY_OCCLUSION_OPTIMIZED_MSFT is passed to xrComputeNewSceneMSFT **and**
  - at least one of XR_SCENE_COMPUTE_FEATURE_SERIALIZE_SCENE_MSFT, XR_SCENE_COMPUTE_FEATURE_PLANE_MSFT, XR_SCENE_COMPUTE_FEATURE_PLANE_MESH_MSFT, or XR_SCENE_COMPUTE_FEATURE_SERIALIZE_SCENE_MSFT are also passed.

Valid Usage (Implicit)

- The `XR_MSFT_scene_understanding` extension **must** be enabled prior to calling xrComputeNewSceneMSFT
- `sceneObserver` **must** be a valid XrSceneObserverMSFT handle
- `computeInfo` **must** be a pointer to a valid XrNewSceneComputeInfoMSFT structure

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_COMPUTE_NEW_SCENE_NOT_COMPLETED_MSFT`
- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_OUT_OF_MEMORY`
- `XR_ERROR_POSE_INVALID`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SCENE_COMPUTE_CONSISTENCY_MISMATCH_MSFT`
- `XR_ERROR_SCENE_COMPUTE_FEATURE_INCOMPATIBLE_MSFT`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_TIME_INVALID`
- `XR_ERROR_VALIDATION_FAILURE`

An XrSceneMSFT handle represents the collection of scene components that were detected during the scene computation.

```
// Provided by XR_MSFT_scene_understanding

            XR_DEFINE_HANDLE(XrSceneMSFT)
```

The XrNewSceneComputeInfoMSFT structure is defined as:

```
// Provided by XR_MSFT_scene_understanding
typedef struct XrNewSceneComputeInfoMSFT {
    XrStructureType                       type;
    const void*                           next;
    uint32_t                              requestedFeatureCount;
    const XrSceneComputeFeatureMSFT*      requestedFeatures;
    XrSceneComputeConsistencyMSFT         consistency;
    XrSceneBoundsMSFT                     bounds;
} XrNewSceneComputeInfoMSFT;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `requestedFeatureCount` is the number of features.
- `requestedFeatures` is an array of XrSceneComputeFeatureMSFT.
- `consistency` indicates the requested XrSceneComputeConsistencyMSFT, trading off speed against the quality of the resulting scene.
- `bounds` is an XrSceneBoundsMSFT representing the culling volume. Scene components entirely outside this volume **should** culled.

Valid Usage (Implicit)

- The `XR_MSFT_scene_understanding` extension **must** be enabled prior to using XrNewSceneComputeInfoMSFT
- `type` **must** be `XR_TYPE_NEW_SCENE_COMPUTE_INFO_MSFT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain. See also: XrVisualMeshComputeLodInfoMSFT
- `requestedFeatures` **must** be a pointer to an array of `requestedFeatureCount` valid XrSceneComputeFeatureMSFT values
- `consistency` **must** be a valid XrSceneComputeConsistencyMSFT value
- `bounds` **must** be a valid XrSceneBoundsMSFT structure
- The `requestedFeatureCount` parameter **must** be greater than `0`

The XrSceneComputeFeatureMSFT enumeration identifies the different scene compute features that may be passed to xrComputeNewSceneMSFT.

```
// Provided by XR_MSFT_scene_understanding
typedef enum XrSceneComputeFeatureMSFT {
    XR_SCENE_COMPUTE_FEATURE_PLANE_MSFT = 1,
    XR_SCENE_COMPUTE_FEATURE_PLANE_MESH_MSFT = 2,
    XR_SCENE_COMPUTE_FEATURE_VISUAL_MESH_MSFT = 3,
    XR_SCENE_COMPUTE_FEATURE_COLLIDER_MESH_MSFT = 4,
  // Provided by XR_MSFT_scene_understanding_serialization
    XR_SCENE_COMPUTE_FEATURE_SERIALIZE_SCENE_MSFT = 1000098000,
  // Provided by XR_MSFT_scene_marker
    XR_SCENE_COMPUTE_FEATURE_MARKER_MSFT = 1000147000,
    XR_SCENE_COMPUTE_FEATURE_MAX_ENUM_MSFT = 0x7FFFFFFF
} XrSceneComputeFeatureMSFT;
```

Enumerant Descriptions

- `XR_SCENE_COMPUTE_FEATURE_PLANE_MSFT` specifies that plane data for objects should be included in the resulting scene.
- `XR_SCENE_COMPUTE_FEATURE_PLANE_MESH_MSFT` specifies that planar meshes for objects should be included in the resulting scene.
- `XR_SCENE_COMPUTE_FEATURE_VISUAL_MESH_MSFT` specifies that 3D visualization meshes for objects should be included in the resulting scene.
- `XR_SCENE_COMPUTE_FEATURE_COLLIDER_MESH_MSFT` specifies that 3D collider meshes for objects should be included in the resulting scene.

Note

Applications wanting to use the scene for analysis, or in a physics simulation should set `consistency` to `XR_SCENE_COMPUTE_CONSISTENCY_SNAPSHOT_COMPLETE_MSFT` in order to avoid physics objects falling through the gaps and escaping the scene.

Setting `consistency` to `XR_SCENE_COMPUTE_CONSISTENCY_SNAPSHOT_INCOMPLETE_FAST_MSFT` might speed up the compute but it will result in gaps in the scene.

Setting `consistency` to `XR_SCENE_COMPUTE_CONSISTENCY_OCCLUSION_OPTIMIZED_MSFT` should be done when the resulting mesh will only be used to occlude virtual objects that are behind real-world surfaces. This mode will be most efficient and have the lowest-latency, but will return meshes less suitable for analysis or visualization.

The XrSceneComputeConsistencyMSFT enumeration identifies the different scene compute consistencies that may be passed to xrComputeNewSceneMSFT.

```
// Provided by XR_MSFT_scene_understanding
typedef enum XrSceneComputeConsistencyMSFT {
    XR_SCENE_COMPUTE_CONSISTENCY_SNAPSHOT_COMPLETE_MSFT = 1,
    XR_SCENE_COMPUTE_CONSISTENCY_SNAPSHOT_INCOMPLETE_FAST_MSFT = 2,
    XR_SCENE_COMPUTE_CONSISTENCY_OCCLUSION_OPTIMIZED_MSFT = 3,
    XR_SCENE_COMPUTE_CONSISTENCY_MAX_ENUM_MSFT = 0x7FFFFFFF
} XrSceneComputeConsistencyMSFT;
```

Enumerant Descriptions

- XR_SCENE_COMPUTE_CONSISTENCY_SNAPSHOT_COMPLETE_MSFT. The runtime **must** return a scene that is a consistent and complete snapshot of the environment, inferring the size and shape of objects as needed where the objects were not directly observed, in order to generate a watertight representation of the scene.
- XR_SCENE_COMPUTE_CONSISTENCY_SNAPSHOT_INCOMPLETE_FAST_MSFT. The runtime **must** return a consistent snapshot of the scene with meshes that do not overlap adjacent meshes at their edges, but **may** skip returning objects with XrSceneObjectTypeMSFT of XR_SCENE_OBJECT_TYPE_INFERRED_MSFT in order to return the scene faster.
- XR_SCENE_COMPUTE_CONSISTENCY_OCCLUSION_OPTIMIZED_MSFT. The runtime **may** react to this value by computing scenes more quickly and reusing existing mesh buffer IDs more often to minimize app overhead, with potential tradeoffs such as returning meshes that are not watertight, meshes that overlap adjacent meshes at their edges to allow partial updates in the future, or other reductions in mesh quality that are less observable when mesh is used for occlusion only.

An application **can** pass one or more bounding volumes when calling xrComputeNewSceneMSFT. These bounding volumes are used to determine which scene components to include in the resulting scene. Scene components that intersect one or more of the bounding volumes **should** be included, and all other scene components **should** be excluded. If an application inputs no bounding volumes, then the runtime **must** not associate any scene components with the resulting XrSceneMSFT handle.

```
// Provided by XR_MSFT_scene_understanding
typedef struct XrSceneBoundsMSFT {
    XrSpace                                 space;
    XrTime                                  time;
    uint32_t                                sphereCount;
    const XrSceneSphereBoundMSFT*           spheres;
    uint32_t                                boxCount;
    const XrSceneOrientedBoxBoundMSFT*      boxes;
    uint32_t                                frustumCount;
    const XrSceneFrustumBoundMSFT*          frustums;
} XrSceneBoundsMSFT;
```

Member Descriptions

- `space` is a handle to the XrSpace in which the bounds are specified.
- `time` is the `XrTime` at which the bounds will be evaluated within `space`.
- `sphereCount` is the number of sphere bounds.
- `spheres` is an array of XrSceneSphereBoundMSFT.
- `boxCount` is the number of oriented box bounds.
- `boxes` is an array of XrSceneOrientedBoxBoundMSFT.
- `frustumCount` is the number of frustum bounds.
- `frustums` is an array of XrSceneFrustumBoundMSFT.

Valid Usage (Implicit)

- The `XR_MSFT_scene_understanding` extension **must** be enabled prior to using XrSceneBoundsMSFT
- `space` **must** be a valid XrSpace handle
- If `sphereCount` is not `0`, `spheres` **must** be a pointer to an array of `sphereCount` XrSceneSphereBoundMSFT structures
- If `boxCount` is not `0`, `boxes` **must** be a pointer to an array of `boxCount` XrSceneOrientedBoxBoundMSFT structures
- If `frustumCount` is not `0`, `frustums` **must** be a pointer to an array of `frustumCount` XrSceneFrustumBoundMSFT structures

An XrSceneSphereBoundMSFT structure describes the center and radius of a sphere bounds.

```
// Provided by XR_MSFT_scene_understanding
typedef struct XrSceneSphereBoundMSFT {
    XrVector3f    center;
    float         radius;
} XrSceneSphereBoundMSFT;
```

Member Descriptions

- `center` is an XrVector3f representing the center of the sphere bound within the reference frame of the corresponding XrSceneBoundsMSFT::`space`.
- `radius` is the finite positive radius of the sphere bound.

The runtime **must** return `XR_ERROR_VALIDATION_FAILURE` if `radius` is not a finite positive value.

Valid Usage (Implicit)

- The `XR_MSFT_scene_understanding` extension **must** be enabled prior to using XrSceneSphereBoundMSFT

An XrSceneOrientedBoxBoundMSFT structure describes the pose and extents of an oriented box bounds.

```
// Provided by XR_MSFT_scene_understanding
typedef struct XrSceneOrientedBoxBoundMSFT {
    XrPosef       pose;
    XrVector3f    extents;
} XrSceneOrientedBoxBoundMSFT;
```

Member Descriptions

- `pose` is an XrPosef defining the center position and orientation of the oriented bounding box bound within the reference frame of the corresponding XrSceneBoundsMSFT::`space`.
- `extents` is an XrVector3f defining the edge-to-edge length of the box along each dimension with `pose` as the center.

The runtime **must** return `XR_ERROR_VALIDATION_FAILURE` if any component of `extents` is not finite or less than or equal to zero.

Valid Usage (Implicit)

- The XR_MSFT_scene_understanding extension **must** be enabled prior to using XrSceneOrientedBoxBoundMSFT

An XrSceneFrustumBoundMSFT structure describes the pose, field of view, and far distance of a frustum bounds.

```
// Provided by XR_MSFT_scene_understanding
typedef struct XrSceneFrustumBoundMSFT {
    XrPosef    pose;
    XrFovf     fov;
    float      farDistance;
} XrSceneFrustumBoundMSFT;
```

Member Descriptions

- pose is an XrPosef defining the position and orientation of the tip of the frustum bound within the reference frame of the corresponding XrSceneBoundsMSFT::space.
- fov is an XrFovf for the four sides of the frustum bound where XrFovf::angleLeft and XrFovf ::angleRight are along the X axis and XrFovf::angleUp and XrFovf::angleDown are along the Y axis of the frustum bound space.
- farDistance is the positive distance of the far plane of the frustum bound along the -Z direction of the frustum bound space.

The runtime **must** return XR_ERROR_VALIDATION_FAILURE if farDistance is less than or equal to zero. The runtime **must** return XR_ERROR_VALIDATION_FAILURE if the fov angles are not between between -π/2 and π/2 exclusively.

Valid Usage (Implicit)

- The XR_MSFT_scene_understanding extension **must** be enabled prior to using XrSceneFrustumBoundMSFT

Applications **can** request a desired visual mesh level of detail by including XrVisualMeshComputeLodInfoMSFT in the XrNewSceneComputeInfoMSFT::next chain. If XrVisualMeshComputeLodInfoMSFT is not included in the XrNewSceneComputeInfoMSFT::next chain, then XR_MESH_COMPUTE_LOD_COARSE_MSFT **must** be used for the visual mesh level of detail.

The XrVisualMeshComputeLodInfoMSFT structure is defined as:

```
// Provided by XR_MSFT_scene_understanding
typedef struct XrVisualMeshComputeLodInfoMSFT {
    XrStructureType         type;
    const void*             next;
    XrMeshComputeLodMSFT    lod;
} XrVisualMeshComputeLodInfoMSFT;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `lod` is the requested mesh level of detail specified by XrMeshComputeLodMSFT.

Valid Usage (Implicit)

- The `XR_MSFT_scene_understanding` extension **must** be enabled prior to using XrVisualMeshComputeLodInfoMSFT
- `type` **must** be `XR_TYPE_VISUAL_MESH_COMPUTE_LOD_INFO_MSFT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `lod` **must** be a valid XrMeshComputeLodMSFT value

The XrMeshComputeLodMSFT enumeration identifies the level of detail of visual mesh compute.

```
// Provided by XR_MSFT_scene_understanding
typedef enum XrMeshComputeLodMSFT {
    XR_MESH_COMPUTE_LOD_COARSE_MSFT = 1,
    XR_MESH_COMPUTE_LOD_MEDIUM_MSFT = 2,
    XR_MESH_COMPUTE_LOD_FINE_MSFT = 3,
    XR_MESH_COMPUTE_LOD_UNLIMITED_MSFT = 4,
    XR_MESH_COMPUTE_LOD_MAX_ENUM_MSFT = 0x7FFFFFFF
} XrMeshComputeLodMSFT;
```

Enumerant Descriptions

- XR_MESH_COMPUTE_LOD_COARSE_MSFT. Coarse mesh compute level of detail will generate roughly 100 triangles per cubic meter.
- XR_MESH_COMPUTE_LOD_MEDIUM_MSFT. Medium mesh compute level of detail will generate roughly 400 triangles per cubic meter.
- XR_MESH_COMPUTE_LOD_FINE_MSFT. Fine mesh compute level of detail will generate roughly 2000 triangles per cubic meter.
- XR_MESH_COMPUTE_LOD_UNLIMITED_MSFT. Unlimited mesh compute level of detail. There is no guarantee as to the number of triangles returned.

The xrEnumerateSceneComputeFeaturesMSFT function enumerates the supported scene compute features of the given system.

This function follows the two-call idiom for filling the features array.

```
// Provided by XR_MSFT_scene_understanding
XrResult xrEnumerateSceneComputeFeaturesMSFT(
    XrInstance                                  instance,
    XrSystemId                                  systemId,
    uint32_t                                    featureCapacityInput,
    uint32_t*                                   featureCountOutput,
    XrSceneComputeFeatureMSFT*                  features);
```

Parameter Descriptions

- instance is a handle to an XrInstance.
- systemId is the XrSystemId whose scene compute features will be enumerated.
- featureCapacityInput is the capacity of the array, or 0 to indicate a request to retrieve the required capacity.
- featureCountOutput is a pointer to the count of scene compute features, or a pointer to the required capacity in the case that featureCapacityInput is insufficient.
- features is an array of XrSceneComputeFeatureMSFT.

Valid Usage (Implicit)

- The XR_MSFT_scene_understanding extension **must** be enabled prior to calling xrEnumerateSceneComputeFeaturesMSFT
- instance **must** be a valid XrInstance handle
- featureCountOutput **must** be a pointer to a uint32_t value
- If featureCapacityInput is not 0, features **must** be a pointer to an array of featureCapacityInput XrSceneComputeFeatureMSFT values

Return Codes

Success

- XR_SUCCESS

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SIZE_INSUFFICIENT
- XR_ERROR_SYSTEM_INVALID
- XR_ERROR_VALIDATION_FAILURE

An application **can** inspect the completion of the compute by polling xrGetSceneComputeStateMSFT. This function **should** typically be called once per frame per XrSceneObserverMSFT.

```
// Provided by XR_MSFT_scene_understanding
XrResult xrGetSceneComputeStateMSFT(
    XrSceneObserverMSFT                         sceneObserver,
    XrSceneComputeStateMSFT*                    state);
```

Parameter Descriptions

- sceneObserver is a handle to an XrSceneObserverMSFT.
- state is the returned XrSceneComputeStateMSFT value.

Valid Usage (Implicit)

- The XR_MSFT_scene_understanding extension **must** be enabled prior to calling xrGetSceneComputeStateMSFT
- `sceneObserver` **must** be a valid XrSceneObserverMSFT handle
- `state` **must** be a pointer to an XrSceneComputeStateMSFT value

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

XrSceneComputeStateMSFT identifies the different states of computing a new scene.

```
// Provided by XR_MSFT_scene_understanding
typedef enum XrSceneComputeStateMSFT {
    XR_SCENE_COMPUTE_STATE_NONE_MSFT = 0,
    XR_SCENE_COMPUTE_STATE_UPDATING_MSFT = 1,
    XR_SCENE_COMPUTE_STATE_COMPLETED_MSFT = 2,
    XR_SCENE_COMPUTE_STATE_COMPLETED_WITH_ERROR_MSFT = 3,
    XR_SCENE_COMPUTE_STATE_MAX_ENUM_MSFT = 0x7FFFFFFF
} XrSceneComputeStateMSFT;
```

Enumerant Descriptions

- XR_SCENE_COMPUTE_STATE_NONE_MSFT indicates that no scene is available, and that a scene is not being computed. The application **may** call xrComputeNewSceneMSFT to start computing a scene.
- XR_SCENE_COMPUTE_STATE_UPDATING_MSFT indicates that a new scene is being computed. Calling xrCreateSceneMSFT or xrComputeNewSceneMSFT **must** return the error XR_ERROR_COMPUTE_NEW_SCENE_NOT_COMPLETED_MSFT.
- XR_SCENE_COMPUTE_STATE_COMPLETED_MSFT indicates that a new scene has completed computing. The application **may** call xrCreateSceneMSFT to get the results of the query or the application **may** call xrComputeNewSceneMSFT to start computing a new scene.
- XR_SCENE_COMPUTE_STATE_COMPLETED_WITH_ERROR_MSFT indicates that the new scene computation completed with an error. Calling xrCreateSceneMSFT **must** return a valid XrSceneMSFT handle but calling xrGetSceneComponentsMSFT with that handle **must** return zero scene components. The runtime **must** allow the application to call xrComputeNewSceneMSFT to try computing a scene again, even if the last call to xrComputeNewSceneMSFT resulted in XR_SCENE_COMPUTE_STATE_COMPLETED_WITH_ERROR_MSFT.

- The xrGetSceneComputeStateMSFT function **must** return XR_SCENE_COMPUTE_STATE_NONE_MSFT if it is called before xrComputeNewSceneMSFT is called for the first time for the given XrSceneObserverMSFT handle.
- After calling xrComputeNewSceneMSFT but before the asynchronous operation has completed, any calls to xrGetSceneComputeStateMSFT **should** return XR_SCENE_COMPUTE_STATE_UPDATING_MSFT.
- Once the asynchronous operation has completed successfully, xrGetSceneComputeStateMSFT **must** return XR_SCENE_COMPUTE_STATE_COMPLETED_MSFT until xrComputeNewSceneMSFT is called again.

Create a scene handle after a new scene compute has completed

The xrCreateSceneMSFT functions creates an XrSceneMSFT handle. It can only be called after xrGetSceneComputeStateMSFT returns XR_SCENE_COMPUTE_STATE_COMPLETED_MSFT to indicate that the asynchronous operation has completed. The XrSceneMSFT handle manages the collection of scene components that represents the detected objects found during the query.

After an XrSceneMSFT handle is created, the handle and associated data **must** remain valid until destroyed, even after xrCreateSceneMSFT is called again to create the next scene. The runtime **must** keep alive any component data and mesh buffers relating to this historical scene until its handle is destroyed.

```
// Provided by XR_MSFT_scene_understanding
XrResult xrCreateSceneMSFT(
    XrSceneObserverMSFT                         sceneObserver,
    const XrSceneCreateInfoMSFT*                createInfo,
    XrSceneMSFT*                                scene);
```

Parameter Descriptions

- `sceneObserver` is a handle to an XrSceneObserverMSFT.
- `createInfo` exists for extensibility purposes, it is `NULL` or a pointer to a valid XrSceneCreateInfoMSFT structure.
- `scene` is the returned XrSceneMSFT handle.

Valid Usage (Implicit)

- The `XR_MSFT_scene_understanding` extension **must** be enabled prior to calling xrCreateSceneMSFT
- `sceneObserver` **must** be a valid XrSceneObserverMSFT handle
- If `createInfo` is not `NULL`, `createInfo` **must** be a pointer to a valid XrSceneCreateInfoMSFT structure
- `scene` **must** be a pointer to an XrSceneMSFT handle

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_COMPUTE_NEW_SCENE_NOT_COMPLETED_MSFT
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

Calling xrCreateSceneMSFT when xrGetSceneComputeStateMSFT returns XR_SCENE_COMPUTE_STATE_NONE_MSFT or XR_SCENE_COMPUTE_STATE_UPDATING_MSFT **must** return the error XR_ERROR_COMPUTE_NEW_SCENE_NOT_COMPLETED_MSFT.

The XrSceneCreateInfoMSFT structure is defined as:

```
// Provided by XR_MSFT_scene_understanding
typedef struct XrSceneCreateInfoMSFT {
    XrStructureType    type;
    const void*        next;
} XrSceneCreateInfoMSFT;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.

Valid Usage (Implicit)

- The XR_MSFT_scene_understanding extension **must** be enabled prior to using XrSceneCreateInfoMSFT
- type **must** be XR_TYPE_SCENE_CREATE_INFO_MSFT
- next **must** be NULL or a valid pointer to the next structure in a structure chain

The xrDestroySceneMSFT function releases the scene and the underlying resources.

```
// Provided by XR_MSFT_scene_understanding
XrResult xrDestroySceneMSFT(
    XrSceneMSFT                                 scene);
```

Parameter Descriptions

- scene is an XrSceneMSFT previously created by xrCreateSceneMSFT.

Valid Usage (Implicit)

- The XR_MSFT_scene_understanding extension **must** be enabled prior to calling xrDestroySceneMSFT
- scene **must** be a valid XrSceneMSFT handle

Thread Safety

- Access to scene, and any child handles, **must** be externally synchronized

Return Codes

Success

- XR_SUCCESS

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_RUNTIME_FAILURE

Scene component types and Universally Unique Identifiers

Each XrSceneMSFT may contain one or more scene components. Scene components are uniquely identified by a Universally Unique Identifier, represented by XrUuidMSFT. Each scene component belongs to one XrSceneComponentTypeMSFT. The XrSceneComponentTypeMSFT denotes which additional properties can be read for that scene component.

- Get a list of scene objects and their properties in the scene by calling xrGetSceneComponentsMSFT with XR_SCENE_COMPONENT_TYPE_OBJECT_MSFT and including XrSceneObjectsMSFT in the XrSceneComponentsMSFT::next chain.
- Get the list of scene planes and their properties in the scene if XR_SCENE_COMPUTE_FEATURE_PLANE_MSFT was passed to xrComputeNewSceneMSFT by calling xrGetSceneComponentsMSFT with XR_SCENE_COMPONENT_TYPE_PLANE_MSFT and including XrScenePlanesMSFT in the XrSceneComponentsMSFT::next chain.
- Get the list of scene visual meshes and their properties in the scene if XR_SCENE_COMPUTE_FEATURE_VISUAL_MESH_MSFT was passed to xrComputeNewSceneMSFT by calling xrGetSceneComponentsMSFT with XR_SCENE_COMPONENT_TYPE_VISUAL_MESH_MSFT and including XrSceneMeshesMSFT in the XrSceneComponentsMSFT::next chain.
- Get the list of scene collider meshes and their properties in the scene if XR_SCENE_COMPUTE_FEATURE_COLLIDER_MESH_MSFT was passed to xrComputeNewSceneMSFT by calling xrGetSceneComponentsMSFT with XR_SCENE_COMPONENT_TYPE_COLLIDER_MESH_MSFT and including XrSceneMeshesMSFT in the XrSceneComponentsMSFT::next chain.

The XrUuidMSFT structure is a 128-bit UUID (Universally Unique IDentifier) that follows RFC 4122 Variant 1. The structure is composed of 16 octets, typically with the sizes and order of the fields defined in RFC 4122 section 4.1.2. The XrUuidMSFT structure is defined as:

```
// Provided by XR_MSFT_scene_understanding
typedef struct XrUuidMSFT {
    uint8_t    bytes[16];
} XrUuidMSFT;
```

Member Descriptions

- bytes is a 128-bit Variant-1 Universally Unique Identifier.

Valid Usage (Implicit)

- The XR_MSFT_scene_understanding extension **must** be enabled prior to using XrUuidMSFT

The XrSceneComponentTypeMSFT enumeration identifies the scene component type.

```
// Provided by XR_MSFT_scene_understanding
typedef enum XrSceneComponentTypeMSFT {
    XR_SCENE_COMPONENT_TYPE_INVALID_MSFT = -1,
    XR_SCENE_COMPONENT_TYPE_OBJECT_MSFT = 1,
    XR_SCENE_COMPONENT_TYPE_PLANE_MSFT = 2,
    XR_SCENE_COMPONENT_TYPE_VISUAL_MESH_MSFT = 3,
    XR_SCENE_COMPONENT_TYPE_COLLIDER_MESH_MSFT = 4,
  // Provided by XR_MSFT_scene_understanding_serialization
    XR_SCENE_COMPONENT_TYPE_SERIALIZED_SCENE_FRAGMENT_MSFT = 1000098000,
  // Provided by XR_MSFT_scene_marker
    XR_SCENE_COMPONENT_TYPE_MARKER_MSFT = 1000147000,
    XR_SCENE_COMPONENT_TYPE_MAX_ENUM_MSFT = 0x7FFFFFFF
} XrSceneComponentTypeMSFT;
```

Enumerant Descriptions

- XR_SCENE_COMPONENT_TYPE_INVALID_MSFT indicates an invalid scene component type.
- XR_SCENE_COMPONENT_TYPE_OBJECT_MSFT indicates a discrete object detected in the world, such as a wall, floor, ceiling or table. Scene objects then provide their geometric representations such as planes and meshes as child scene components with the types below.
- XR_SCENE_COMPONENT_TYPE_PLANE_MSFT indicates a flat 2D representation of a surface in the world, such as a wall, floor, ceiling or table.
- XR_SCENE_COMPONENT_TYPE_VISUAL_MESH_MSFT indicates a visual mesh representation of an object in the world, optimized for visual quality when directly rendering a wireframe or other mesh visualization to the user. Visual mesh can also be used for rendering the silhouettes of objects. Applications can request varying levels of detail for visual meshes when calling xrComputeNewSceneMSFT using XrVisualMeshComputeLodInfoMSFT.
- XR_SCENE_COMPONENT_TYPE_COLLIDER_MESH_MSFT indicates a collider mesh representation of an object in the world, optimized to maintain the silhouette of an object while reducing detail on mostly-flat surfaces. Collider mesh is useful when calculating physics collisions or when rendering silhouettes of objects for occlusion.

Get scene components

Scene components are read from an XrSceneMSFT using xrGetSceneComponentsMSFT and passing one XrSceneComponentTypeMSFT. This function follows the two-call idiom for filling multiple buffers in a struct. Different scene component types **may** have additional properties that **can** be read by chaining additional structures to XrSceneComponentsMSFT. Those additional structures **must** have an array size that is at least as large as XrSceneComponentsMSFT::componentCapacityInput, otherwise the runtime **must** return XR_ERROR_SIZE_INSUFFICIENT.

- If XR_SCENE_COMPONENT_TYPE_OBJECT_MSFT is passed to xrGetSceneComponentsMSFT, then

XrSceneObjectsMSFT may be included in the XrSceneComponentsMSFT::next chain.

- If XR_SCENE_COMPONENT_TYPE_PLANE_MSFT is passed to xrGetSceneComponentsMSFT, then XrScenePlanesMSFT may be included in the XrSceneComponentsMSFT::next chain.
- If XR_SCENE_COMPONENT_TYPE_VISUAL_MESH_MSFT or XR_SCENE_COMPONENT_TYPE_COLLIDER_MESH_MSFT are passed to xrGetSceneComponentsMSFT, then XrSceneMeshesMSFT may be included in the XrSceneComponentsMSFT::next chain.

```
// Provided by XR_MSFT_scene_understanding
XrResult xrGetSceneComponentsMSFT(
    XrSceneMSFT                                 scene,
    const XrSceneComponentsGetInfoMSFT*         getInfo,
    XrSceneComponentsMSFT*                      components);
```

Parameter Descriptions

- scene is an XrSceneMSFT previously created by xrCreateSceneMSFT.
- getInfo is a pointer to an XrSceneComponentsGetInfoMSFT structure.
- components is the XrSceneComponentsMSFT output structure.

Valid Usage (Implicit)

- The XR_MSFT_scene_understanding extension **must** be enabled prior to calling xrGetSceneComponentsMSFT
- scene **must** be a valid XrSceneMSFT handle
- getInfo **must** be a pointer to a valid XrSceneComponentsGetInfoMSFT structure
- components **must** be a pointer to an XrSceneComponentsMSFT structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SCENE_COMPONENT_TYPE_MISMATCH_MSFT
- XR_ERROR_SESSION_LOST
- XR_ERROR_SIZE_INSUFFICIENT
- XR_ERROR_VALIDATION_FAILURE

An application **can** use XrSceneComponentsGetInfoMSFT to read the state of a specific component type using the xrGetSceneComponentsMSFT function. Applications can chain one or more of following extension structures to the XrSceneComponentsGetInfoMSFT::next chain to further narrow the returned components. The returned components **must** satisfy all conditions in the extension structs.

- XrSceneComponentParentFilterInfoMSFT to return only scene components that match the given parent object identifier.
- XrSceneObjectTypesFilterInfoMSFT to return only scene components that match any of the given XrSceneObjectTypeMSFT values or if a scene component does not have an XrSceneObjectTypeMSFT property then the parent's XrSceneObjectTypeMSFT property will be compared.
- XrScenePlaneAlignmentFilterInfoMSFT to return only scene components that match any of the given XrScenePlaneAlignmentTypeMSFT values.

The XrSceneComponentsGetInfoMSFT structure is defined as:

```
// Provided by XR_MSFT_scene_understanding
typedef struct XrSceneComponentsGetInfoMSFT {
    XrStructureType             type;
    const void*                 next;
    XrSceneComponentTypeMSFT    componentType;
} XrSceneComponentsGetInfoMSFT;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `componentType` is the scene component type requested.

Valid Usage (Implicit)

- The `XR_MSFT_scene_understanding` extension **must** be enabled prior to using XrSceneComponentsGetInfoMSFT
- `type` **must** be `XR_TYPE_SCENE_COMPONENTS_GET_INFO_MSFT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain. See also: XrSceneComponentParentFilterInfoMSFT, XrSceneMarkerTypeFilterMSFT, XrSceneObjectTypesFilterInfoMSFT, XrScenePlaneAlignmentFilterInfoMSFT
- `componentType` **must** be a valid XrSceneComponentTypeMSFT value

The XrSceneComponentsMSFT structure contains an array of XrSceneComponentMSFT returning the components that satisfy the conditions in xrGetSceneComponentsMSFT::`getInfo`. The XrSceneComponentsMSFT structure is defined as:

```
// Provided by XR_MSFT_scene_understanding
typedef struct XrSceneComponentsMSFT {
    XrStructureType          type;
    void*                    next;
    uint32_t                 componentCapacityInput;
    uint32_t                 componentCountOutput;
    XrSceneComponentMSFT*    components;
} XrSceneComponentsMSFT;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `componentCapacityInput` is the capacity of the array, or 0 to indicate a request to retrieve the required capacity.
- `componentCountOutput` is a pointer to the count of components, or a pointer to the required capacity in the case that `componentCapacityInput` is insufficient.
- `components` is an array of XrSceneComponentMSFT.
- See the Buffer Size Parameters section for a detailed description of retrieving the required `components` size.

Valid Usage (Implicit)

- The `XR_MSFT_scene_understanding` extension **must** be enabled prior to using XrSceneComponentsMSFT
- `type` **must** be `XR_TYPE_SCENE_COMPONENTS_MSFT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain. See also: XrSceneMarkerQRCodesMSFT, XrSceneMarkersMSFT, XrSceneMeshesMSFT, XrSceneObjectsMSFT, XrScenePlanesMSFT
- If `componentCapacityInput` is not `0`, `components` **must** be a pointer to an array of `componentCapacityInput` XrSceneComponentMSFT structures

The XrSceneComponentMSFT structure is defined as:

```
// Provided by XR_MSFT_scene_understanding
typedef struct XrSceneComponentMSFT {
    XrSceneComponentTypeMSFT    componentType;
    XrUuidMSFT                  id;
    XrUuidMSFT                  parentId;
    XrTime                      updateTime;
} XrSceneComponentMSFT;
```

Member Descriptions

- `componentType` is the XrSceneComponentTypeMSFT of the scene component.
- `id` is the XrUuidMSFT of the scene component.
- `parentId` is the XrUuidMSFT of the parent scene object. If the scene component does not have a parent, then `parentId` will be equal to zero.
- `updateTime` is the `XrTime` that this scene component was last updated.

The runtime **must** set `parentId` to either zero or a valid XrUuidMSFT that corresponds to a scene component of type `XR_SCENE_COMPONENT_TYPE_OBJECT_MSFT` that exists in the XrSceneMSFT.



Note

The parent scene object is intended to allow scene components to be grouped. For example, the scene object for a wall might have multiple scene component children like `XR_SCENE_COMPONENT_TYPE_PLANE_MSFT`, `XR_SCENE_COMPONENT_TYPE_VISUAL_MESH_MSFT`, and `XR_SCENE_COMPONENT_TYPE_COLLIDER_MESH_MSFT`. Those child scene components would be alternative representations of the same wall.

Valid Usage (Implicit)

- The `XR_MSFT_scene_understanding` extension **must** be enabled prior to using XrSceneComponentMSFT
- `componentType` **must** be a valid XrSceneComponentTypeMSFT value

Get scene components using filters

The scene components that are returned by xrGetSceneComponentsMSFT **can** be filtered by chaining optional structures to XrSceneComponentsGetInfoMSFT. The runtime **must** combine multiple filters with a logical AND.

The XrSceneComponentParentFilterInfoMSFT structure is defined as:

```
// Provided by XR_MSFT_scene_understanding
typedef struct XrSceneComponentParentFilterInfoMSFT {
    XrStructureType    type;
    const void*        next;
    XrUuidMSFT         parentId;
} XrSceneComponentParentFilterInfoMSFT;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `parentId` is the XrUuidMSFT of the parent scene component to filter by.

The runtime **must** return only scene components with matching `parentId`. If `parentId` is zero then the runtime **must** return only scene components that do not have a parent.

Valid Usage (Implicit)

- The `XR_MSFT_scene_understanding` extension **must** be enabled prior to using XrSceneComponentParentFilterInfoMSFT
- `type` **must** be `XR_TYPE_SCENE_COMPONENT_PARENT_FILTER_INFO_MSFT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

The XrSceneObjectTypesFilterInfoMSFT structure is defined as:

```
// Provided by XR_MSFT_scene_understanding
typedef struct XrSceneObjectTypesFilterInfoMSFT {
    XrStructureType                 type;
    const void*                     next;
    uint32_t                        objectTypeCount;
    const XrSceneObjectTypeMSFT*    objectTypes;
} XrSceneObjectTypesFilterInfoMSFT;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `objectTypeCount` is a `uint32_t` describing the count of elements in the `objectTypes` array.
- `objectTypes` is an array of XrSceneObjectTypeMSFT to filter by.

The runtime **must** return only scene components that match any of the XrSceneObjectTypeMSFT in `objectTypes`. If a scene component does not have an XrSceneObjectTypeMSFT then the parent's XrSceneObjectTypeMSFT value will be used for the comparison if it exists.

Valid Usage (Implicit)

- The XR_MSFT_scene_understanding extension **must** be enabled prior to using XrSceneObjectTypesFilterInfoMSFT
- type **must** be XR_TYPE_SCENE_OBJECT_TYPES_FILTER_INFO_MSFT
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- If objectTypeCount is not 0, objectTypes **must** be a pointer to an array of objectTypeCount valid XrSceneObjectTypeMSFT values

The XrScenePlaneAlignmentFilterInfoMSFT structure is defined as:

```
// Provided by XR_MSFT_scene_understanding
typedef struct XrScenePlaneAlignmentFilterInfoMSFT {
    XrStructureType                         type;
    const void*                             next;
    uint32_t                                alignmentCount;
    const XrScenePlaneAlignmentTypeMSFT*    alignments;
} XrScenePlaneAlignmentFilterInfoMSFT;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain.
- alignmentCount is a uint32_t describing the count of elements in the alignments array.
- alignments is an array of XrScenePlaneAlignmentTypeMSFT to filter by.

The runtime **must** return only scene components that match one of the XrScenePlaneAlignmentTypeMSFT values passed in alignments.

Valid Usage (Implicit)

- The XR_MSFT_scene_understanding extension **must** be enabled prior to using XrScenePlaneAlignmentFilterInfoMSFT
- type **must** be XR_TYPE_SCENE_PLANE_ALIGNMENT_FILTER_INFO_MSFT
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- If alignmentCount is not 0, alignments **must** be a pointer to an array of alignmentCount valid XrScenePlaneAlignmentTypeMSFT values

Get scene objects

The runtime **must** fill out the XrSceneObjectsMSFT structure when included in the XrSceneComponentsMSFT::`next` chain. The XrSceneComponentsGetInfoMSFT::componentType **must** be `XR_SCENE_COMPONENT_TYPE_OBJECT_MSFT` when XrSceneObjectsMSFT is included in the next chain. If it is not, the `XR_ERROR_SCENE_COMPONENT_TYPE_MISMATCH_MSFT` error **must** be returned.

The XrSceneObjectsMSFT structure is defined as:

```
// Provided by XR_MSFT_scene_understanding
typedef struct XrSceneObjectsMSFT {
    XrStructureType       type;
    void*                 next;
    uint32_t              sceneObjectCount;
    XrSceneObjectMSFT*    sceneObjects;
} XrSceneObjectsMSFT;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `sceneObjectCount` is a `uint32_t` describing the count of elements in the `sceneObjects` array.
- `sceneObjects` is an array of XrSceneObjectMSFT.

The runtime **must** only set XrSceneObjectMSFT::`objectType` to any of the following XrSceneObjectTypeMSFT values:

- `XR_SCENE_OBJECT_TYPE_UNCATEGORIZED_MSFT`
- `XR_SCENE_OBJECT_TYPE_BACKGROUND_MSFT`
- `XR_SCENE_OBJECT_TYPE_WALL_MSFT`
- `XR_SCENE_OBJECT_TYPE_FLOOR_MSFT`
- `XR_SCENE_OBJECT_TYPE_CEILING_MSFT`
- `XR_SCENE_OBJECT_TYPE_PLATFORM_MSFT`
- `XR_SCENE_OBJECT_TYPE_INFERRED_MSFT`

Valid Usage (Implicit)

- The XR_MSFT_scene_understanding extension **must** be enabled prior to using XrSceneObjectsMSFT
- type **must** be XR_TYPE_SCENE_OBJECTS_MSFT
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- If sceneObjectCount is not 0, sceneObjects **must** be a pointer to an array of sceneObjectCount XrSceneObjectMSFT structures

The XrSceneObjectMSFT structure represents the state of a scene object.

It is defined as:

```
// Provided by XR_MSFT_scene_understanding
typedef struct XrSceneObjectMSFT {
    XrSceneObjectTypeMSFT    objectType;
} XrSceneObjectMSFT;
```

Member Descriptions

- objectType is the type of the object specified by XrSceneObjectTypeMSFT.

Valid Usage (Implicit)

- The XR_MSFT_scene_understanding extension **must** be enabled prior to using XrSceneObjectMSFT
- objectType **must** be a valid XrSceneObjectTypeMSFT value

The XrSceneObjectTypeMSFT enumeration identifies the different types of scene objects.

```
// Provided by XR_MSFT_scene_understanding
typedef enum XrSceneObjectTypeMSFT {
    XR_SCENE_OBJECT_TYPE_UNCATEGORIZED_MSFT = -1,
    XR_SCENE_OBJECT_TYPE_BACKGROUND_MSFT = 1,
    XR_SCENE_OBJECT_TYPE_WALL_MSFT = 2,
    XR_SCENE_OBJECT_TYPE_FLOOR_MSFT = 3,
    XR_SCENE_OBJECT_TYPE_CEILING_MSFT = 4,
    XR_SCENE_OBJECT_TYPE_PLATFORM_MSFT = 5,
    XR_SCENE_OBJECT_TYPE_INFERRED_MSFT = 6,
    XR_SCENE_OBJECT_TYPE_MAX_ENUM_MSFT = 0x7FFFFFFF
} XrSceneObjectTypeMSFT;
```

Enumerant Descriptions

- XR_SCENE_OBJECT_TYPE_UNCATEGORIZED_MSFT. This scene object has yet to be classified and assigned a type. This should not be confused with background, as this object could be anything; the system has just not come up with a strong enough classification for it yet.
- XR_SCENE_OBJECT_TYPE_BACKGROUND_MSFT. The scene object is known to be not one of the other recognized types of scene object. This class should not be confused with uncategorized where background is known not to be wall/floor/ceiling etc. while uncategorized is not yet categorized.
- XR_SCENE_OBJECT_TYPE_WALL_MSFT. A physical wall. Walls are assumed to be immovable environmental structures.
- XR_SCENE_OBJECT_TYPE_FLOOR_MSFT. Floors are any surfaces on which one can walk. Note: stairs are not floors. Also note, that floors assume any walkable surface and therefore there is no explicit assumption of a singular floor. Multi-level structures, ramps, etc. should all classify as floor.
- XR_SCENE_OBJECT_TYPE_CEILING_MSFT. The upper surface of a room.
- XR_SCENE_OBJECT_TYPE_PLATFORM_MSFT. A large flat surface on which you could place holograms. These tend to represent tables, countertops, and other large horizontal surfaces.
- XR_SCENE_OBJECT_TYPE_INFERRED_MSFT. An imaginary object that was added to the scene in order to make the scene watertight and avoid gaps.

Get scene planes

The runtime **must** fill out the XrScenePlanesMSFT structure when included in the XrSceneComponentsMSFT::`next` chain. The XrSceneComponentsGetInfoMSFT::componentType **must** be XR_SCENE_COMPONENT_TYPE_PLANE_MSFT when XrScenePlanesMSFT is included in the next chain. If it is not, the XR_ERROR_SCENE_COMPONENT_TYPE_MISMATCH_MSFT error **must** be returned.

The XrScenePlanesMSFT structure is defined as:

```
// Provided by XR_MSFT_scene_understanding
typedef struct XrScenePlanesMSFT {
    XrStructureType      type;
    void*                next;
    uint32_t             scenePlaneCount;
    XrScenePlaneMSFT*    scenePlanes;
} XrScenePlanesMSFT;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `scenePlaneCount` is a `uint32_t` describing the count of elements in the XrScenePlaneMSFT array.
- `scenePlanes` is an array of XrScenePlaneMSFT.

Valid Usage (Implicit)

- The `XR_MSFT_scene_understanding` extension **must** be enabled prior to using XrScenePlanesMSFT
- `type` **must** be `XR_TYPE_SCENE_PLANES_MSFT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- If `scenePlaneCount` is not `0`, `scenePlanes` **must** be a pointer to an array of `scenePlaneCount` XrScenePlaneMSFT structures

The XrScenePlaneMSFT structure represents the state of a scene plane.

It is defined as:

```
// Provided by XR_MSFT_scene_understanding
typedef struct XrScenePlaneMSFT {
    XrScenePlaneAlignmentTypeMSFT    alignment;
    XrExtent2Df                      size;
    uint64_t                         meshBufferId;
    XrBool32                         supportsIndicesUint16;
} XrScenePlaneMSFT;
```

Member Descriptions

- `alignment` is the alignment type of the plane specified by XrScenePlaneAlignmentTypeMSFT.
- `size` is the 2D size of the plane's extent, where XrExtent2Df::`width` is the width of the plane along the X axis, and XrExtent2Df::`height` is the height of the plane along the Y axis.
- `meshBufferId` is the `uint64_t` identifier that specifies the scene mesh buffer of this plane's triangle mesh. If `meshBufferId` is zero then this plane does not have a mesh. The triangles in a planar mesh are coplanar.
- `supportsIndicesUint16` is `XR_TRUE` if the mesh supports reading 16-bit unsigned indices.

The `size` of a plane refers to the plane's size in the x-y plane of the plane's coordinate system. A plane with a position of {0,0,0}, rotation of {0,0,0,1} (no rotation), and an extent of {1,1} refers to a 1 meter x 1 meter plane centered at {0,0,0} with its front face normal vector pointing towards the +Z direction in the plane component's space. For planes with an alignment of `XR_SCENE_PLANE_ALIGNMENT_TYPE_VERTICAL_MSFT`, the +Y direction **must** point up away from the direction of gravity.

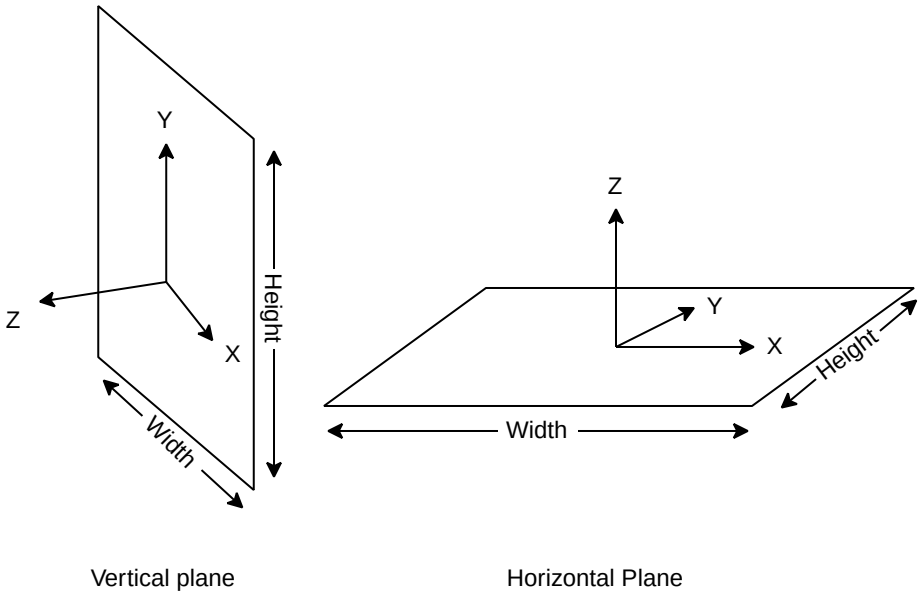


Figure 43. Scene Understanding Plane Coordinate System



Note

OpenXR uses an X-Y plane with +Z as the plane normal but other APIs may use an X-Z plane with +Y as the plane normal. The X-Y plane can be converted to an X-Z plane by rotating -π/2 radians around the +X axis.

Valid Usage (Implicit)

- The XR_MSFT_scene_understanding extension **must** be enabled prior to using XrScenePlaneMSFT
- `alignment` **must** be a valid XrScenePlaneAlignmentTypeMSFT value

XrScenePlaneAlignmentTypeMSFT identifies the different plane alignment types.

```
// Provided by XR_MSFT_scene_understanding
typedef enum XrScenePlaneAlignmentTypeMSFT {
    XR_SCENE_PLANE_ALIGNMENT_TYPE_NON_ORTHOGONAL_MSFT = 0,
    XR_SCENE_PLANE_ALIGNMENT_TYPE_HORIZONTAL_MSFT = 1,
    XR_SCENE_PLANE_ALIGNMENT_TYPE_VERTICAL_MSFT = 2,
    XR_SCENE_PLANE_ALIGNMENT_TYPE_MAX_ENUM_MSFT = 0x7FFFFFFF
} XrScenePlaneAlignmentTypeMSFT;
```

Enumerant Descriptions

- XR_SCENE_PLANE_ALIGNMENT_TYPE_NON_ORTHOGONAL_MSFT means the plane's normal is not orthogonal or parallel to the gravity direction.
- XR_SCENE_PLANE_ALIGNMENT_TYPE_HORIZONTAL_MSFT means the plane's normal is roughly parallel to the gravity direction.
- XR_SCENE_PLANE_ALIGNMENT_TYPE_VERTICAL_MSFT means the plane's normal is roughly orthogonal to the gravity direction.

Get scene mesh

The runtime **must** fill out the XrSceneMeshesMSFT structure when included in the XrSceneComponentsMSFT::`next` chain. The XrSceneComponentsGetInfoMSFT::componentType **must** be XR_SCENE_COMPONENT_TYPE_VISUAL_MESH_MSFT or XR_SCENE_COMPONENT_TYPE_COLLIDER_MESH_MSFT when XrSceneMeshesMSFT is included in the next chain. If it is not, the XR_ERROR_SCENE_COMPONENT_TYPE_MISMATCH_MSFT error **must** be returned.

The XrSceneMeshesMSFT structure is defined as:

```
// Provided by XR_MSFT_scene_understanding
typedef struct XrSceneMeshesMSFT {
    XrStructureType     type;
    void*               next;
    uint32_t            sceneMeshCount;
    XrSceneMeshMSFT*    sceneMeshes;
} XrSceneMeshesMSFT;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `sceneMeshCount` is a `uint32_t` describing the count of elements in the `sceneMeshes` array.
- `sceneMeshes` is an array of XrSceneMeshMSFT.

Valid Usage (Implicit)

- The `XR_MSFT_scene_understanding` extension **must** be enabled prior to using XrSceneMeshesMSFT
- `type` **must** be `XR_TYPE_SCENE_MESHES_MSFT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- If `sceneMeshCount` is not `0`, `sceneMeshes` **must** be a pointer to an array of `sceneMeshCount` XrSceneMeshMSFT structures

The XrSceneMeshMSFT structure represents the state of a scene component's mesh.

It is defined as:

```
// Provided by XR_MSFT_scene_understanding
typedef struct XrSceneMeshMSFT {
    uint64_t    meshBufferId;
    XrBool32    supportsIndicesUint16;
} XrSceneMeshMSFT;
```

Member Descriptions

- `meshBufferId` is the `uint64_t` identifier that specifies the scene mesh buffer. If `meshBufferId` is zero then this scene component does not have mesh data of corresponding XrSceneComponentTypeMSFT in xrGetSceneComponentsMSFT::`getInfo`.
- `supportsIndicesUint16` is `XR_TRUE` if the mesh supports reading 16-bit unsigned indices.

Valid Usage (Implicit)

- The `XR_MSFT_scene_understanding` extension **must** be enabled prior to using XrSceneMeshMSFT

Read scene mesh buffer

The xrGetSceneMeshBuffersMSFT function retrieves the scene mesh vertex buffer and index buffer for the given scene mesh buffer identifier.



Note

Applications may use the scene mesh buffer identifier as a key to cache the vertices and indices of a mesh for reuse within an XrSceneMSFT or across multiple XrSceneMSFT for the same XrSession.

Applications can avoid unnecessarily calling xrGetSceneMeshBuffersMSFT for a scene component if XrSceneComponentMSFT::`updateTime` is equal to the XrSceneComponentMSFT::`updateTime` value in the previous XrSceneMSFT. A scene component is uniquely identified by XrUuidMSFT.

This function follows the two-call idiom for filling multiple buffers in a struct.

The xrGetSceneMeshBuffersMSFT function is defined as:

```
// Provided by XR_MSFT_scene_understanding
XrResult xrGetSceneMeshBuffersMSFT(
    XrSceneMSFT                                 scene,
    const XrSceneMeshBuffersGetInfoMSFT*        getInfo,
    XrSceneMeshBuffersMSFT*                     buffers);
```

Parameter Descriptions

- `scene` is an XrSceneMSFT previously created by xrCreateSceneMSFT.
- `getInfo` is a pointer to an XrSceneMeshBuffersGetInfoMSFT structure.
- `buffers` is a pointer to an XrSceneMeshBuffersMSFT structure for reading a scene mesh buffer.

Applications **can** request the vertex buffer of the mesh by including XrSceneMeshVertexBufferMSFT in the XrSceneMeshBuffersMSFT::`next` chain. Runtimes **must** support requesting a 32-bit index buffer and **may** support requesting a 16-bit index buffer. Applications **can** request a 32-bit index buffer by including XrSceneMeshIndicesUint32MSFT in the XrSceneMeshBuffersMSFT::`next` chain. Applications **can** request a 16-bit index buffer by including XrSceneMeshIndicesUint16MSFT in the XrSceneMeshBuffersMSFT::`next` chain. If the runtime for the given scene mesh buffer does not support requesting a 16-bit index buffer then `XR_ERROR_VALIDATION_FAILURE` **must** be returned. The runtime **must** support reading a 16-bit index buffer for the given scene mesh buffer if XrScenePlaneMSFT:supportsIndicesUint16 or XrSceneMeshMSFT:supportsIndicesUint16 are `XR_TRUE` for the scene component that contained that scene mesh buffer identifier.

The runtime **must** return `XR_ERROR_SCENE_MESH_BUFFER_ID_INVALID_MSFT` if none of the scene components in the given XrSceneMSFT contain XrSceneMeshBuffersGetInfoMSFT::`meshBufferId`. The runtime **must** return `XR_ERROR_SCENE_MESH_BUFFER_ID_INVALID_MSFT` if XrSceneMeshBuffersGetInfoMSFT::`meshBufferId` is zero. The runtime **must** return `XR_ERROR_VALIDATION_FAILURE` if both XrSceneMeshIndicesUint32MSFT and XrSceneMeshIndicesUint16MSFT are included in the XrSceneMeshBuffersMSFT::`next` chain. The runtime **must** return `XR_ERROR_VALIDATION_FAILURE` if the XrSceneMeshBuffersMSFT::`next` does not contain at least one of XrSceneMeshVertexBufferMSFT, XrSceneMeshIndicesUint32MSFT or XrSceneMeshIndicesUint16MSFT.

The runtime **must** return the same vertices and indices for a given scene mesh buffer identifier and XrSession. A runtime **may** return zero vertices and indices if the underlying mesh data is no longer available.

Valid Usage (Implicit)

- The `XR_MSFT_scene_understanding` extension **must** be enabled prior to calling xrGetSceneMeshBuffersMSFT
- `scene` **must** be a valid XrSceneMSFT handle
- `getInfo` **must** be a pointer to a valid XrSceneMeshBuffersGetInfoMSFT structure
- `buffers` **must** be a pointer to an XrSceneMeshBuffersMSFT structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SCENE_COMPONENT_ID_INVALID_MSFT
- XR_ERROR_SCENE_MESH_BUFFER_ID_INVALID_MSFT
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

XrSceneMeshBuffersGetInfoMSFT is an input structure for the xrGetSceneMeshBuffersMSFT function.

```
// Provided by XR_MSFT_scene_understanding
typedef struct XrSceneMeshBuffersGetInfoMSFT {
    XrStructureType    type;
    const void*        next;
    uint64_t           meshBufferId;
} XrSceneMeshBuffersGetInfoMSFT;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- meshBufferId is the uint64_t identifier that specifies the scene mesh buffer to read.

Valid Usage (Implicit)

- The XR_MSFT_scene_understanding extension **must** be enabled prior to using XrSceneMeshBuffersGetInfoMSFT
- type **must** be XR_TYPE_SCENE_MESH_BUFFERS_GET_INFO_MSFT
- next **must** be NULL or a valid pointer to the next structure in a structure chain

XrSceneMeshBuffersMSFT is an input/output structure for reading scene mesh buffers.

```
// Provided by XR_MSFT_scene_understanding
typedef struct XrSceneMeshBuffersMSFT {
    XrStructureType    type;
    void*              next;
} XrSceneMeshBuffersMSFT;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain.

Valid Usage (Implicit)

- The XR_MSFT_scene_understanding extension **must** be enabled prior to using XrSceneMeshBuffersMSFT
- type **must** be XR_TYPE_SCENE_MESH_BUFFERS_MSFT
- next **must** be NULL or a valid pointer to the next structure in a structure chain. See also: XrSceneMeshIndicesUint16MSFT, XrSceneMeshIndicesUint32MSFT, XrSceneMeshVertexBufferMSFT

XrSceneMeshVertexBufferMSFT is an input/output structure for reading scene mesh buffer vertices.

```
// Provided by XR_MSFT_scene_understanding
typedef struct XrSceneMeshVertexBufferMSFT {
    XrStructureType    type;
    void*              next;
    uint32_t           vertexCapacityInput;
    uint32_t           vertexCountOutput;
    XrVector3f*        vertices;
} XrSceneMeshVertexBufferMSFT;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `vertexCapacityInput` is the capacity of the array, or 0 to indicate a request to retrieve the required capacity.
- `vertexCountOutput` is the count of vertices, or the required capacity in the case that `vertexCapacityInput` is insufficient.
- `vertices` is an array of XrVector3f filled in by the runtime returns the position of vertices in the mesh component's space.
- See the Buffer Size Parameters section for a detailed description of retrieving the required `vertices` size.

Valid Usage (Implicit)

- The `XR_MSFT_scene_understanding` extension **must** be enabled prior to using XrSceneMeshVertexBufferMSFT
- `type` **must** be `XR_TYPE_SCENE_MESH_VERTEX_BUFFER_MSFT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- If `vertexCapacityInput` is not `0`, `vertices` **must** be a pointer to an array of `vertexCapacityInput` XrVector3f structures

XrSceneMeshIndicesUint32MSFT is an input/output structure for reading 32-bit indices from a scene mesh buffer.

```
// Provided by XR_MSFT_scene_understanding
typedef struct XrSceneMeshIndicesUint32MSFT {
    XrStructureType    type;
    void*              next;
    uint32_t           indexCapacityInput;
    uint32_t           indexCountOutput;
    uint32_t*          indices;
} XrSceneMeshIndicesUint32MSFT;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `indexCapacityInput` is the capacity of the array, or 0 to indicate a request to retrieve the required capacity.
- `indexCountOutput` is the count of indices, or the required capacity in the case that `indexCapacityInput` is insufficient.
- `indices` is an array of triangle indices filled in by the runtime, specifying the indices of the scene mesh buffer in the vertices array. The triangle indices **must** be returned in counter-clockwise order and three indices denote one triangle.
- See the Buffer Size Parameters section for a detailed description of retrieving the required `indices` size.

Valid Usage (Implicit)

- The `XR_MSFT_scene_understanding` extension **must** be enabled prior to using XrSceneMeshIndicesUint32MSFT
- `type` **must** be `XR_TYPE_SCENE_MESH_INDICES_UINT32_MSFT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- If `indexCapacityInput` is not `0`, `indices` **must** be a pointer to an array of `indexCapacityInput` `uint32_t` values

XrSceneMeshIndicesUint16MSFT is an input/output structure for reading 16-bit indices from a scene mesh buffer.

```
// Provided by XR_MSFT_scene_understanding
typedef struct XrSceneMeshIndicesUint16MSFT {
    XrStructureType    type;
    void*              next;
    uint32_t           indexCapacityInput;
    uint32_t           indexCountOutput;
    uint16_t*          indices;
} XrSceneMeshIndicesUint16MSFT;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `indexCapacityInput` is the capacity of the array, or 0 to indicate a request to retrieve the required capacity.
- `indexCountOutput` is a pointer to the count of indices, or a pointer to the required capacity in the case that `indexCapacityInput` is insufficient.
- `indices` is an array of triangle indices filled in by the runtime, specifying the indices of the scene mesh buffer in the vertices array. The triangle indices **must** be returned in counter-clockwise order and three indices denote one triangle.
- See the Buffer Size Parameters section for a detailed description of retrieving the required `indices` size.

Valid Usage (Implicit)

- The `XR_MSFT_scene_understanding` extension **must** be enabled prior to using XrSceneMeshIndicesUint16MSFT
- `type` **must** be `XR_TYPE_SCENE_MESH_INDICES_UINT16_MSFT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- If `indexCapacityInput` is not `0`, `indices` **must** be a pointer to an array of `indexCapacityInput` `uint16_t` values

Locate scene objects

The xrLocateSceneComponentsMSFT function locates an array of scene components to a base space at a given time.

```
// Provided by XR_MSFT_scene_understanding
XrResult xrLocateSceneComponentsMSFT(
    XrSceneMSFT                                 scene,
    const XrSceneComponentsLocateInfoMSFT*      locateInfo,
    XrSceneComponentLocationsMSFT*              locations);
```

Parameter Descriptions

- `scene` is a handle to an XrSceneMSFT.
- `locateInfo` is a pointer to XrSceneComponentsLocateInfoMSFT describing information to locate scene components.
- `locations` is a pointer to XrSceneComponentLocationsMSFT receiving the returned scene component locations.

The runtime **must** return `XR_ERROR_SIZE_INSUFFICIENT` if XrSceneComponentLocationsMSFT::`locationCount` is less than XrSceneComponentsLocateInfoMSFT::`componentIdCount`.



Note

Similar to xrLocateSpace, apps should call xrLocateSceneComponentsMSFT each frame because the location returned by xrLocateSceneComponentsMSFT in later frames may change over time as the target space or the scene components may refine their locations.

Valid Usage (Implicit)

- The `XR_MSFT_scene_understanding` extension **must** be enabled prior to calling xrLocateSceneComponentsMSFT
- `scene` **must** be a valid XrSceneMSFT handle
- `locateInfo` **must** be a pointer to a valid XrSceneComponentsLocateInfoMSFT structure
- `locations` **must** be a pointer to an XrSceneComponentLocationsMSFT structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_SIZE_INSUFFICIENT
- XR_ERROR_TIME_INVALID
- XR_ERROR_VALIDATION_FAILURE

The XrSceneComponentsLocateInfoMSFT structure describes the information to locate scene components.

```
// Provided by XR_MSFT_scene_understanding
typedef struct XrSceneComponentsLocateInfoMSFT {
    XrStructureType      type;
    const void*          next;
    XrSpace              baseSpace;
    XrTime               time;
    uint32_t             componentIdCount;
    const XrUuidMSFT*    componentIds;
} XrSceneComponentsLocateInfoMSFT;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `baseSpace` is an XrSpace within which the scene components will be located.
- `time` is an `XrTime` at which to locate the scene components.
- `componentIdCount` is a `uint32_t` describing the count of elements in the `componentIds` array.
- `componentIds` is an array of XrUuidMSFT identifiers for the scene components to location.

Valid Usage (Implicit)

- The `XR_MSFT_scene_understanding` extension **must** be enabled prior to using XrSceneComponentsLocateInfoMSFT
- `type` **must** be `XR_TYPE_SCENE_COMPONENTS_LOCATE_INFO_MSFT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `baseSpace` **must** be a valid XrSpace handle
- If `componentIdCount` is not `0`, `componentIds` **must** be a pointer to an array of `componentIdCount` XrUuidMSFT structures

The XrSceneComponentLocationsMSFT structure returns scene component locations.

```
// Provided by XR_MSFT_scene_understanding
typedef struct XrSceneComponentLocationsMSFT {
    XrStructureType                 type;
    void*                           next;
    uint32_t                        locationCount;
    XrSceneComponentLocationMSFT*   locations;
} XrSceneComponentLocationsMSFT;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `locationCount` is a `uint32_t` describing the count of elements in the `locations` array.
- `locations` is an array of XrSceneComponentLocationMSFT scene component locations.

Valid Usage (Implicit)

- The `XR_MSFT_scene_understanding` extension **must** be enabled prior to using XrSceneComponentLocationsMSFT
- `type` **must** be `XR_TYPE_SCENE_COMPONENT_LOCATIONS_MSFT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- If `locationCount` is not `0`, `locations` **must** be a pointer to an array of `locationCount` XrSceneComponentLocationMSFT structures

The XrSceneComponentLocationMSFT structure describes the position and orientation of a scene component to space XrSceneComponentsLocateInfoMSFT::`baseSpace` at time XrSceneComponentsLocateInfoMSFT::`time`. If the scene component identified by XrUuidMSFT is not found, `flags` should be empty.

```
// Provided by XR_MSFT_scene_understanding
typedef struct XrSceneComponentLocationMSFT {
    XrSpaceLocationFlags    flags;
    XrPosef                 pose;
} XrSceneComponentLocationMSFT;
```

Member Descriptions

- `flags` is a bitfield, with bit masks defined in XrSpaceLocationFlagBits, to indicate which members contain valid data.
- `pose` is an XrPosef defining the position and orientation of the scene component within the reference frame of the corresponding XrSceneComponentsLocateInfoMSFT::`baseSpace`.

Valid Usage (Implicit)

- The `XR_MSFT_scene_understanding` extension **must** be enabled prior to using XrSceneComponentLocationMSFT
- `flags` **must** be `0` or a valid combination of XrSpaceLocationFlagBits values

New Object Types

- XrSceneObserverMSFT
- XrSceneMSFT

New Flag Types

New Enum Constants

XrObjectType enumeration is extended with:

- `XR_OBJECT_TYPE_SCENE_OBSERVER_MSFT`
- `XR_OBJECT_TYPE_SCENE_MSFT`

XrStructureType enumeration is extended with:

- `XR_TYPE_SCENE_OBSERVER_CREATE_INFO_MSFT`
- `XR_TYPE_SCENE_CREATE_INFO_MSFT`
- `XR_TYPE_NEW_SCENE_COMPUTE_INFO_MSFT`
- `XR_TYPE_VISUAL_MESH_COMPUTE_LOD_INFO_MSFT`
- `XR_TYPE_SCENE_COMPONENTS_MSFT`
- `XR_TYPE_SCENE_COMPONENTS_GET_INFO_MSFT`
- `XR_TYPE_SCENE_COMPONENT_LOCATIONS_MSFT`
- `XR_TYPE_SCENE_COMPONENTS_LOCATE_INFO_MSFT`
- `XR_TYPE_SCENE_OBJECTS_MSFT`
- `XR_TYPE_SCENE_COMPONENT_PARENT_FILTER_INFO_MSFT`
- `XR_TYPE_SCENE_OBJECT_TYPES_FILTER_INFO_MSFT`
- `XR_TYPE_SCENE_PLANES_MSFT`
- `XR_TYPE_SCENE_PLANE_ALIGNMENT_FILTER_INFO_MSFT`
- `XR_TYPE_SCENE_MESHES_MSFT`
- `XR_TYPE_SCENE_MESH_BUFFERS_GET_INFO_MSFT`
- `XR_TYPE_SCENE_MESH_BUFFERS_MSFT`

XrResult enumeration is extended with:

- `XR_ERROR_COMPUTE_NEW_SCENE_NOT_COMPLETED_MSFT`
- `XR_ERROR_SCENE_COMPONENT_ID_INVALID_MSFT`
- `XR_ERROR_SCENE_COMPONENT_TYPE_MISMATCH_MSFT`
- `XR_ERROR_SCENE_MESH_BUFFER_ID_INVALID_MSFT`
- `XR_ERROR_SCENE_COMPUTE_FEATURE_INCOMPATIBLE_MSFT`
- `XR_ERROR_SCENE_COMPUTE_CONSISTENCY_MISMATCH_MSFT`

New Enums

- XrSceneComputeFeatureMSFT
- XrSceneComputeConsistencyMSFT
- XrSceneObjectTypeMSFT
- XrScenePlaneAlignmentTypeMSFT
- XrSceneComputeStateMSFT
- XrSceneComponentTypeMSFT
- XrMeshComputeLodMSFT

New Structures

- XrSceneObserverCreateInfoMSFT
- XrSceneCreateInfoMSFT
- XrNewSceneComputeInfoMSFT
- XrUuidMSFT
- XrSceneObserverCreateInfoMSFT
- XrSceneCreateInfoMSFT
- XrNewSceneComputeInfoMSFT
- XrVisualMeshComputeLodInfoMSFT
- XrSceneSphereBoundMSFT
- XrSceneOrientedBoxBoundMSFT
- XrSceneFrustumBoundMSFT
- XrSceneBoundsMSFT
- XrSceneComponentMSFT
- XrSceneComponentsMSFT
- XrSceneComponentsGetInfoMSFT

- XrSceneComponentLocationMSFT
- XrSceneComponentLocationsMSFT
- XrSceneComponentsLocateInfoMSFT
- XrSceneObjectMSFT
- XrSceneObjectsMSFT
- XrSceneComponentParentFilterInfoMSFT
- XrSceneObjectTypesFilterInfoMSFT
- XrScenePlaneMSFT
- XrScenePlanesMSFT
- XrScenePlaneAlignmentFilterInfoMSFT
- XrSceneMeshMSFT
- XrSceneMeshesMSFT
- XrSceneMeshBuffersGetInfoMSFT
- XrSceneMeshBuffersMSFT

New Functions

- xrCreateSceneObserverMSFT
- xrDestroySceneObserverMSFT
- xrCreateSceneMSFT
- xrDestroySceneMSFT
- xrComputeNewSceneMSFT
- xrGetSceneComponentsMSFT
- xrLocateSceneComponentsMSFT
- xrGetSceneMeshBuffersMSFT

Issues

Version History

- Revision 1, 2021-05-03 (Darryl Gough)
  - Initial extension description
- Revision 2, 2022-06-29 (Darryl Gough)
  - Fix missing error codes

14.33. XR_MSFT_scene_understanding_serialization

Name String

`XR_MSFT_scene_understanding_serialization`

Extension Type

Instance extension

Registered Extension Number

99

Revision

2

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0
and
`XR_MSFT_scene_understanding`

Deprecation State

- *Deprecated* without replacement

Last Modified Date

2021-05-03

IP Status

No known IP claims.

Contributors

Darryl Gough, Microsoft
Yin Li, Microsoft
Bryce Hutchings, Microsoft
Alex Turner, Microsoft
Simon Stachniak, Microsoft
David Fields, Microsoft

Overview

This extension extends the scene understanding extension and enables scenes to be serialized or deserialized. It enables computing a new scene into a serialized binary stream and it enables deserializing a binary stream into an XrSceneMSFT handle.

Serialize a scene

This extension adds XR_SCENE_COMPUTE_FEATURE_SERIALIZE_SCENE_MSFT to XrSceneComputeFeatureMSFT, which **can** be passed to xrComputeNewSceneMSFT plus one or more of XR_SCENE_COMPUTE_FEATURE_PLANE_MSFT, XR_SCENE_COMPUTE_FEATURE_PLANE_MESH_MSFT, XR_SCENE_COMPUTE_FEATURE_VISUAL_MESH_MSFT or XR_SCENE_COMPUTE_FEATURE_COLLIDER_MESH_MSFT to inform the runtime that it **should** compute a serialized binary representation of the scene. If XR_SCENE_COMPUTE_FEATURE_SERIALIZE_SCENE_MSFT is the only XrSceneComputeFeatureMSFT passed to xrComputeNewSceneMSFT then XR_ERROR_SCENE_COMPUTE_FEATURE_INCOMPATIBLE_MSFT **must** be returned.

If an XrSceneMSFT was created using XR_SCENE_COMPUTE_FEATURE_SERIALIZE_SCENE_MSFT then XR_SCENE_COMPONENT_TYPE_SERIALIZED_SCENE_FRAGMENT_MSFT can be passed to the xrGetSceneComponentsMSFT function to read the list of serialized scene fragment XrUuidMSFT values from XrSceneComponentMSFT::id. The XrUuidMSFT of a scene fragment can be passed to xrGetSerializedSceneFragmentDataMSFT to read the binary data of the given scene fragment.

The application **can** call the xrGetSerializedSceneFragmentDataMSFT function to read the binary data of a serialized scene fragment from the XrSceneMSFT handle. This function follows the two-call idiom for filling the buffer.

The xrGetSerializedSceneFragmentDataMSFT function is defined as:

```
// Provided by XR_MSFT_scene_understanding_serialization
XrResult xrGetSerializedSceneFragmentDataMSFT(
    XrSceneMSFT                                 scene,
    const XrSerializedSceneFragmentDataGetInfoMSFT* getInfo,
    uint32_t                                    countInput,
    uint32_t*                                   readOutput,
    uint8_t*                                    buffer);
```

Parameter Descriptions

- scene is the XrSceneMSFT handle to read from.
- getInfo is a pointer to an XrSerializedSceneFragmentDataGetInfoMSFT structure.
- countInput is the number of bytes that should be read.
- readOutput is the number of bytes read.
- buffer is a pointer to the buffer where the data should be copied.

The runtime **must** return XR_ERROR_SCENE_COMPONENT_ID_INVALID_MSFT if the given scene fragment XrUuidMSFT was not found.

Valid Usage (Implicit)

- The XR_MSFT_scene_understanding_serialization extension **must** be enabled prior to calling xrGetSerializedSceneFragmentDataMSFT
- scene **must** be a valid XrSceneMSFT handle
- getInfo **must** be a pointer to a valid XrSerializedSceneFragmentDataGetInfoMSFT structure
- readOutput **must** be a pointer to a uint32_t value
- If countInput is not 0, buffer **must** be a pointer to an array of countInput uint8_t values

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SCENE_COMPONENT_ID_INVALID_MSFT
- XR_ERROR_SESSION_LOST
- XR_ERROR_SIZE_INSUFFICIENT
- XR_ERROR_VALIDATION_FAILURE

The XrSerializedSceneFragmentDataGetInfoMSFT structure is defined as:

```
// Provided by XR_MSFT_scene_understanding_serialization
typedef struct XrSerializedSceneFragmentDataGetInfoMSFT {
    XrStructureType    type;
    const void*        next;
    XrUuidMSFT         sceneFragmentId;
} XrSerializedSceneFragmentDataGetInfoMSFT;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `sceneFragmentId` is the XrUuidMSFT of the serialized scene fragment that was previously read from xrGetSceneComponentsMSFT with `XR_SCENE_COMPONENT_TYPE_SERIALIZED_SCENE_FRAGMENT_MSFT`.

Valid Usage (Implicit)

- The `XR_MSFT_scene_understanding_serialization` extension **must** be enabled prior to using XrSerializedSceneFragmentDataGetInfoMSFT
- `type` **must** be `XR_TYPE_SERIALIZED_SCENE_FRAGMENT_DATA_GET_INFO_MSFT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

Deserialize a scene

This extension enables an application to deserialize the binary representation of a scene that was previously serialized.

For a given XrSceneObserverMSFT handle, instead of calling xrComputeNewSceneMSFT, which computes the scene from the system's sensors, the application **can** use xrDeserializeSceneMSFT to produce a scene from the given binary scene fragment data.

The xrDeserializeSceneMSFT function is defined as:

```
// Provided by XR_MSFT_scene_understanding_serialization
XrResult xrDeserializeSceneMSFT(
    XrSceneObserverMSFT                         sceneObserver,
    const XrSceneDeserializeInfoMSFT*           deserializeInfo);
```

Parameter Descriptions

- `sceneObserver` is a handle to an XrSceneObserverMSFT previously created with xrCreateSceneObserverMSFT.
- `deserializeInfo` is a pointer to an XrSceneDeserializeInfoMSFT structure.

The xrDeserializeSceneMSFT function begins deserializing a list of serialized scene fragments. The

runtime **must** return quickly without waiting for the deserialization to complete. The application **should** use xrGetSceneComputeStateMSFT to inspect the completeness of the deserialization.

The runtime **must** return XR_ERROR_COMPUTE_NEW_SCENE_NOT_COMPLETED_MSFT if xrDeserializeSceneMSFT is called while the scene computation is in progress.

The xrGetSceneComputeStateMSFT function **must** return XR_SCENE_COMPUTE_STATE_UPDATING_MSFT while the deserialization is in progress, and XR_SCENE_COMPUTE_STATE_COMPLETED_MSFT when the deserialization has completed successfully. If the runtime fails to deserialize the binary stream, xrGetSceneComputeStateMSFT **must** return XR_SCENE_COMPUTE_STATE_COMPLETED_WITH_ERROR_MSFT to indicate that the deserialization has completed but an error occurred.

When xrGetSceneComputeStateMSFT returns XR_SCENE_COMPUTE_STATE_COMPLETED_MSFT, the application **may** call xrCreateSceneMSFT to create the XrSceneMSFT handle. If xrCreateSceneMSFT is called while xrGetSceneComputeStateMSFT returns XR_SCENE_COMPUTE_STATE_COMPLETED_WITH_ERROR_MSFT, a valid XrSceneMSFT handle **must** be returned, but that handle **must** contain zero scene components.

Valid Usage (Implicit)

- The XR_MSFT_scene_understanding_serialization extension **must** be enabled prior to calling xrDeserializeSceneMSFT
- sceneObserver **must** be a valid XrSceneObserverMSFT handle
- deserializeInfo **must** be a pointer to a valid XrSceneDeserializeInfoMSFT structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_COMPUTE_NEW_SCENE_NOT_COMPLETED_MSFT
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

XrSceneDeserializeInfoMSFT is an input structure that describes the array of serialized scene fragments that will be deserialized by the xrDeserializeSceneMSFT function.

```
// Provided by XR_MSFT_scene_understanding_serialization
typedef struct XrSceneDeserializeInfoMSFT {
    XrStructureType                           type;
    const void*                               next;
    uint32_t                                  fragmentCount;
    const XrDeserializeSceneFragmentMSFT*     fragments;
} XrSceneDeserializeInfoMSFT;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `fragmentCount` is the count of XrDeserializeSceneFragmentMSFT structures in the `fragments` array.
- `fragments` is an array of XrDeserializeSceneFragmentMSFT.

If the scene fragments are not in the same order as returned by xrGetSceneComponentsMSFT or the runtime failed to deserialized the binary data then xrGetSceneComputeStateMSFT **must** return `XR_SCENE_COMPUTE_STATE_COMPLETED_WITH_ERROR_MSFT`.

Valid Usage (Implicit)

- The `XR_MSFT_scene_understanding_serialization` extension **must** be enabled prior to using XrSceneDeserializeInfoMSFT
- `type` **must** be `XR_TYPE_SCENE_DESERIALIZE_INFO_MSFT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- If `fragmentCount` is not `0`, `fragments` **must** be a pointer to an array of `fragmentCount` valid XrDeserializeSceneFragmentMSFT structures

The XrDeserializeSceneFragmentMSFT structure represents a single fragment of a binary stream to be deserialized. It is defined as:

```
// Provided by XR_MSFT_scene_understanding_serialization
typedef struct XrDeserializeSceneFragmentMSFT {
    uint32_t          bufferSize;
    const uint8_t*    buffer;
} XrDeserializeSceneFragmentMSFT;
```

Member Descriptions

- `bufferSize` is the size of the `buffer` array.
- `buffer` is an array of `uint_8` data for the scene fragment to be deserialized.

Valid Usage (Implicit)

- The `XR_MSFT_scene_understanding_serialization` extension **must** be enabled prior to using XrDeserializeSceneFragmentMSFT
- If `bufferSize` is not `0`, `buffer` **must** be a pointer to an array of `bufferSize` `uint8_t` values

New Object Types

New Flag Types

New Enum Constants

XrSceneComponentTypeMSFT enumeration is extended with:

- `XR_SCENE_COMPONENT_TYPE_SERIALIZED_SCENE_FRAGMENT_MSFT`

XrSceneComputeFeatureMSFT enumeration is extended with:

- `XR_SCENE_COMPUTE_FEATURE_SERIALIZE_SCENE_MSFT`

XrStructureType enumeration is extended with:

- `XR_TYPE_SERIALIZED_SCENE_FRAGMENT_DATA_GET_INFO_MSFT`
- `XR_TYPE_SCENE_DESERIALIZE_INFO_MSFT`

New Enums

New Structures

- XrSerializedSceneFragmentDataGetInfoMSFT
- XrSceneDeserializeInfoMSFT

- XrDeserializeSceneFragmentMSFT

New Functions

- xrGetSerializedSceneFragmentDataMSFT
- xrDeserializeSceneMSFT

Issues

Version History

- Revision 1, 2021-05-03 (Darryl Gough)
  - Initial extension description
- Revision 2, 2022-06-29 (Darryl Gough)
  - Fix missing error codes

14.34. XR_MSFT_secondary_view_configuration

Name String

`XR_MSFT_secondary_view_configuration`

Extension Type

Instance extension

Registered Extension Number

54

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Deprecation State

- *Deprecated* without replacement

Last Modified Date

2020-05-02

IP Status

No known IP claims.

Contributors

Yin Li, Microsoft
Zonglin Wu, Microsoft
Alex Turner, Microsoft

14.34.1. Overview

This extension allows an application to enable support for one or more **secondary view configurations**. A secondary view configuration is a well-known set of views that the runtime can make active while a session is running. In a frame where a secondary view configuration is active, the application's single frame loop should additionally render into those active secondary views, sharing the frame waiting logic and update loop with the primary view configuration for that running session.

A proper secondary view configuration support includes following steps:

1. When calling xrCreateInstance, enable the `XR_MSFT_secondary_view_configuration` extension and the extension defines a concrete secondary view configuration type, for example, `XR_MSFT_first_person_observer`.
2. Inspect supported secondary view configurations using the xrEnumerateViewConfigurations function.
3. Enable supported secondary view configurations using the xrBeginSession function with an XrSecondaryViewConfigurationSessionBeginInfoMSFT chained extension structure.
4. Inspect if an enabled secondary view configuration is activated by the system or the user using the xrWaitFrame function with an XrSecondaryViewConfigurationFrameStateMSFT chained extension structure.
5. When a secondary view configuration is changed to active, get the latest view configuration properties using the xrGetViewConfigurationProperties and xrEnumerateViewConfigurationViews functions.
6. Create the swapchain images for the active secondary view configuration using the xrCreateSwapchain function with an XrSecondaryViewConfigurationSwapchainCreateInfoMSFT chained extension structure using `recommendedImageRectWidth` and `recommendedImageRectHeight` in the corresponding XrViewConfigurationView structure returned from xrEnumerateViewConfigurationViews.
7. Locate the secondary view configuration views using the xrLocateViews function with the active secondary view configuration type.
8. Submit the composition layers using the swapchain images for an active secondary view configuration using the xrEndFrame function with the XrSecondaryViewConfigurationFrameEndInfoMSFT chained extension structure.

14.34.2. Enumerate supported secondary view configurations

The first step is for the application to inspect if a runtime supports certain secondary view

configurations. The app uses the existing API xrEnumerateViewConfigurations for this.

For example, when the XR_MSFT_first_person_observer extension is enabled, the application will enumerate a view configuration of type XR_VIEW_CONFIGURATION_TYPE_SECONDARY_MONO_FIRST_PERSON_OBSERVER_MSFT, and can use this secondary view configuration type in later functions.

14.34.3. Secondary view configuration properties

The application can inspect the properties of a secondary view configuration through the existing xrGetViewConfigurationProperties, xrEnumerateViewConfigurationViews and xrEnumerateEnvironmentBlendModes functions using a supported secondary view configuration type.

The runtime **may** change the recommended properties, such as recommended image width or height, when the secondary view configuration becomes active. The application **should** use the latest recommended width and height when creating swapchain images and related resources for the active secondary view configuration.

When an application creates swapchain images for a secondary view configuration, it **can** chain a XrSecondaryViewConfigurationSwapchainCreateInfoMSFT structure to XrSwapchainCreateInfo when calling xrCreateSwapchain. This hints to the runtime that the created swapchain image will be submitted to the given secondary view configuration, allowing the runtime to make optimizations for such usage when there is opportunity.

```
// Provided by XR_MSFT_secondary_view_configuration
typedef struct XrSecondaryViewConfigurationSwapchainCreateInfoMSFT {
    XrStructureType           type;
    const void*               next;
    XrViewConfigurationType   viewConfigurationType;
} XrSecondaryViewConfigurationSwapchainCreateInfoMSFT;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- viewConfigurationType is the secondary view configuration type the application is intending to use this swapchain for.

If this structure is not present in the XrSwapchainCreateInfo next chain when calling xrCreateSwapchain, the runtime **should** optimize the created swapchain for the primary view configuration of the session.

If the application submits a swapchain image created with one view configuration type to a composition layer for another view configuration, the runtime **may** need to copy the resource across view configurations. However, the runtime **must** correctly compose the image regardless which view configuration type was hinted when swapchain image was created.

Valid Usage (Implicit)

- The `XR_MSFT_secondary_view_configuration` extension **must** be enabled prior to using XrSecondaryViewConfigurationSwapchainCreateInfoMSFT
- `type` **must** be `XR_TYPE_SECONDARY_VIEW_CONFIGURATION_SWAPCHAIN_CREATE_INFO_MSFT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `viewConfigurationType` **must** be a valid XrViewConfigurationType value

14.34.4. Enable secondary view configuration

The application indicates to the runtime which secondary view configurations it can support by chaining an XrSecondaryViewConfigurationSessionBeginInfoMSFT structure to the XrSessionBeginInfo::`next` pointer when calling xrBeginSession.

The XrSecondaryViewConfigurationSessionBeginInfoMSFT structure is used by the application to indicate the list of secondary XrViewConfigurationType to enable for this session.

It is defined as:

```
// Provided by XR_MSFT_secondary_view_configuration
typedef struct XrSecondaryViewConfigurationSessionBeginInfoMSFT {
    XrStructureType                     type;
    const void*                         next;
    uint32_t                            viewConfigurationCount;
    const XrViewConfigurationType*      enabledViewConfigurationTypes;
} XrSecondaryViewConfigurationSessionBeginInfoMSFT;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `viewConfigurationCount` is the number of elements in `enabledViewConfigurationTypes`
- `enabledViewConfigurationTypes` is an array of enabled secondary view configuration types that application supports.

If there are any duplicated view configuration types in the array of `enabledViewConfigurationTypes`, the runtime **must** return error `XR_ERROR_VALIDATION_FAILURE`.

If there are any primary view configuration types in the array of `enabledViewConfigurationTypes`, the runtime **must** return error `XR_ERROR_VALIDATION_FAILURE`.

If there are any secondary view configuration types not returned by xrEnumerateViewConfigurations in the array of `enabledViewConfigurationTypes`, the runtime **must** return error `XR_ERROR_VIEW_CONFIGURATION_TYPE_UNSUPPORTED`.

Valid Usage (Implicit)

- The `XR_MSFT_secondary_view_configuration` extension **must** be enabled prior to using XrSecondaryViewConfigurationSessionBeginInfoMSFT
- `type` **must** be `XR_TYPE_SECONDARY_VIEW_CONFIGURATION_SESSION_BEGIN_INFO_MSFT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `enabledViewConfigurationTypes` **must** be a pointer to an array of `viewConfigurationCount` valid XrViewConfigurationType values
- The `viewConfigurationCount` parameter **must** be greater than `0`

14.34.5. Per-frame active view configurations

The runtime then tells the application at each xrWaitFrame function call which of the enabled secondary view configurations are active for that frame. When extension structure XrSecondaryViewConfigurationFrameStateMSFT is chained to the XrFrameState::next pointer, the runtime writes into this structure the state of each enabled secondary view configuration.

The XrSecondaryViewConfigurationFrameStateMSFT structure returns whether the enabled view configurations are active or inactive.

It is defined as as:

```
// Provided by XR_MSFT_secondary_view_configuration
typedef struct XrSecondaryViewConfigurationFrameStateMSFT {
    XrStructureType                           type;
    void*                                     next;
    uint32_t                                  viewConfigurationCount;
    XrSecondaryViewConfigurationStateMSFT*    viewConfigurationStates;
} XrSecondaryViewConfigurationFrameStateMSFT;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `viewConfigurationCount` is the number of elements in `viewConfigurationStates`.
- `viewConfigurationStates` is an array of XrSecondaryViewConfigurationStateMSFT structures.

The array size `viewConfigurationCount` in the XrSecondaryViewConfigurationFrameStateMSFT structure **must** be the same as the array size enabled through XrSecondaryViewConfigurationSessionBeginInfoMSFT when calling xrBeginSession earlier, otherwise the runtime **must** return error `XR_ERROR_VALIDATION_FAILURE`.

Valid Usage (Implicit)

- The `XR_MSFT_secondary_view_configuration` extension **must** be enabled prior to using XrSecondaryViewConfigurationFrameStateMSFT
- `type` **must** be `XR_TYPE_SECONDARY_VIEW_CONFIGURATION_FRAME_STATE_MSFT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `viewConfigurationStates` **must** be a pointer to an array of `viewConfigurationCount` XrSecondaryViewConfigurationStateMSFT structures
- The `viewConfigurationCount` parameter **must** be greater than `0`

The XrSecondaryViewConfigurationStateMSFT structure returns the state of an enabled secondary view configurations.

```
// Provided by XR_MSFT_secondary_view_configuration
typedef struct XrSecondaryViewConfigurationStateMSFT {
    XrStructureType           type;
    void*                     next;
    XrViewConfigurationType   viewConfigurationType;
    XrBool32                  active;
} XrSecondaryViewConfigurationStateMSFT;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `viewConfigurationType` is an XrViewConfigurationType that represents the returned state.
- `active` is an `XrBool32` returns whether the secondary view configuration is active and displaying frames to users.

When a secondary view configuration becomes active, the application **should** render its secondary views as soon as possible, by getting their view transforms and FOV using xrLocateViews and then submitting composition layers to xrEndFrame through the XrSecondaryViewConfigurationFrameEndInfoMSFT extension structure. When a secondary view configuration changes from inactive to active, the runtime **may** change XrViewConfigurationView of the given view configuration such as the recommended image width or height. An application **should** query for latest XrViewConfigurationView through xrEnumerateViewConfigurationViews function for the secondary view configuration and consider recreating swapchain images if necessary. The runtime **must** not change the XrViewConfigurationView, including recommended image width and height of a secondary view configuration when `active` remains true until the secondary view configuration deactivated or the session has ended.

If necessary, the application **can** take longer than a frame duration to prepare by calling xrEndFrame without submitting layers for that secondary view configuration until ready. The runtime **should** delay the underlying scenario managed by the secondary view configuration until the application begins submitting frames with layers for that configuration. The active secondary view configuration composed output is undefined if the application stops submitting frames with layers for a secondary view configuration while `active` remains true.

When the runtime intends to conclude a secondary view configuration, for example when user stops video capture, the runtime makes the view configuration inactive by setting the corresponding `active` in the XrSecondaryViewConfigurationStateMSFT structure to false.

Valid Usage (Implicit)

- The `XR_MSFT_secondary_view_configuration` extension **must** be enabled prior to using XrSecondaryViewConfigurationStateMSFT
- `type` **must** be `XR_TYPE_SECONDARY_VIEW_CONFIGURATION_STATE_MSFT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `viewConfigurationType` **must** be a valid XrViewConfigurationType value

14.34.6. Locate and inspect view states of secondary view configurations

When the application calls xrLocateViews, it **can** use XrViewLocateInfo::viewConfigurationType field to query the view locations and projections for any enabled XrViewConfigurationType for the running session.

The runtime **must** return XR_ERROR_VIEW_CONFIGURATION_TYPE_UNSUPPORTED from xrLocateViews if the specified XrViewConfigurationType is not enabled for the running session using XrSecondaryViewConfigurationSessionBeginInfoMSFT when calling xrBeginSession.

If the view configuration is supported but not active, as indicated in XrSecondaryViewConfigurationFrameStateMSFT, xrLocateViews will successfully return, but the resulting XrViewState **may** have XR_VIEW_STATE_ORIENTATION_TRACKED_BIT and XR_VIEW_STATE_ORIENTATION_TRACKED_BIT unset.

14.34.7. Submit composition layers to secondary view configurations

The application **should** submit layers each frame for all active secondary view configurations using the xrEndFrame function, by chaining the XrSecondaryViewConfigurationFrameEndInfoMSFT structure to the next pointer of XrFrameEndInfo structure.

The XrSecondaryViewConfigurationFrameEndInfoMSFT structure is defined as as:

```
// Provided by XR_MSFT_secondary_view_configuration
typedef struct XrSecondaryViewConfigurationFrameEndInfoMSFT {
    XrStructureType                                 type;
    const void*                                     next;
    uint32_t                                        viewConfigurationCount;
    const XrSecondaryViewConfigurationLayerInfoMSFT*    viewConfigurationLayersInfo;
} XrSecondaryViewConfigurationFrameEndInfoMSFT;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- viewConfigurationCount is the number of elements in viewConfigurationLayersInfo.
- viewConfigurationLayersInfo is an array of XrSecondaryViewConfigurationLayerInfoMSFT, containing composition layers to be submitted for the specified active view configuration.

The view configuration type in each XrSecondaryViewConfigurationLayerInfoMSFT must be one of the view configurations enabled when calling xrBeginSession in XrSecondaryViewConfigurationSessionBeginInfoMSFT, or else the runtime **must** return error

XR_ERROR_SECONDARY_VIEW_CONFIGURATION_TYPE_NOT_ENABLED_MSFT.

The view configuration type in each XrSecondaryViewConfigurationLayerInfoMSFT must not be the primary view configuration in this session, or else the runtime **must** return error XR_ERROR_LAYER_INVALID. The primary view configuration layers continue to be submitted through XrFrameEndInfo directly.

If the view configuration is not active, as indicated in XrSecondaryViewConfigurationFrameStateMSFT, the composition layers submitted to this view configuration **may** be ignored by the runtime. Applications **should** avoid rendering into secondary views when the view configuration is inactive.

Valid Usage (Implicit)

- The XR_MSFT_secondary_view_configuration extension **must** be enabled prior to using XrSecondaryViewConfigurationFrameEndInfoMSFT
- type **must** be XR_TYPE_SECONDARY_VIEW_CONFIGURATION_FRAME_END_INFO_MSFT
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- viewConfigurationLayersInfo **must** be a pointer to an array of viewConfigurationCount valid XrSecondaryViewConfigurationLayerInfoMSFT structures
- The viewConfigurationCount parameter **must** be greater than 0

The application should submit an XrSecondaryViewConfigurationLayerInfoMSFT in XrSecondaryViewConfigurationFrameEndInfoMSFT for each active secondary view configuration type when calling xrEndFrame.

The XrSecondaryViewConfigurationLayerInfoMSFT structure is defined as as:

```
// Provided by XR_MSFT_secondary_view_configuration
typedef struct XrSecondaryViewConfigurationLayerInfoMSFT {
    XrStructureType                                 type;
    const void*                                     next;
    XrViewConfigurationType                         viewConfigurationType;
    XrEnvironmentBlendMode                          environmentBlendMode;
    uint32_t                                        layerCount;
    const XrCompositionLayerBaseHeader* const*      layers;
} XrSecondaryViewConfigurationLayerInfoMSFT;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `viewConfigurationType` is XrViewConfigurationType to which the composition layers will be displayed.
- `environmentBlendMode` is the XrEnvironmentBlendMode value representing the desired environment blend mode for this view configuration.
- `layerCount` is the number of composition layers in this frame for the secondary view configuration type. The maximum supported layer count is identified by XrSystemGraphicsProperties::maxLayerCount. If layerCount is greater than the maximum supported layer count then `XR_ERROR_LAYER_LIMIT_EXCEEDED` is returned.
- `layers` is a pointer to an array of XrCompositionLayerBaseHeader pointers.

This structure is similar to the XrFrameEndInfo structure, with an extra XrViewConfigurationType field to specify the view configuration for which the submitted layers will be rendered.

The application **should** render its content for both the primary and secondary view configurations using the same XrFrameState::`predictedDisplayTime` reported by xrWaitFrame. The runtime **must** treat both the primary views and secondary views as being submitted for the same XrViewLocateInfo ::`displayTime` specified in the call to xrEndFrame.

For layers such as quad layers whose content is identical across view configurations, the application **can** submit the same XrCompositionLayerBaseHeader structures to multiple view configurations in the same xrEndFrame function call.

For each frame, the application **should** only render and submit layers for the secondary view configurations that were active that frame, as indicated in the XrSecondaryViewConfigurationFrameStateMSFT filled in for that frame's xrWaitFrame call. The runtime **must** ignore composition layers submitted for an inactive view configuration.

Valid Usage (Implicit)

- The `XR_MSFT_secondary_view_configuration` extension **must** be enabled prior to using XrSecondaryViewConfigurationLayerInfoMSFT
- `type` **must** be `XR_TYPE_SECONDARY_VIEW_CONFIGURATION_LAYER_INFO_MSFT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `viewConfigurationType` **must** be a valid XrViewConfigurationType value
- `environmentBlendMode` **must** be a valid XrEnvironmentBlendMode value
- `layers` **must** be a pointer to an array of `layerCount` valid XrCompositionLayerBaseHeader -based structures. See also: XrCompositionLayerCubeKHR, XrCompositionLayerCylinderKHR, XrCompositionLayerEquirect2KHR, XrCompositionLayerEquirectKHR, XrCompositionLayerPassthroughANDROID, XrCompositionLayerPassthroughFB, XrCompositionLayerPassthroughHTC, XrCompositionLayerProjection, XrCompositionLayerQuad
- The `layerCount` parameter **must** be greater than `0`

New Object Types

New Flag Types

New Enum Constants

XrStructureType enumeration is extended with:

- `XR_TYPE_SECONDARY_VIEW_CONFIGURATION_SESSION_BEGIN_INFO_MSFT`
- `XR_TYPE_SECONDARY_VIEW_CONFIGURATION_STATE_MSFT`
- `XR_TYPE_SECONDARY_VIEW_CONFIGURATION_FRAME_STATE_MSFT`
- `XR_TYPE_SECONDARY_VIEW_CONFIGURATION_FRAME_END_INFO_MSFT`
- `XR_TYPE_SECONDARY_VIEW_CONFIGURATION_LAYER_INFO_MSFT`
- `XR_ERROR_SECONDARY_VIEW_CONFIGURATION_TYPE_NOT_ENABLED_MSFT`

New Enums

New Structures

- XrSecondaryViewConfigurationSessionBeginInfoMSFT
- XrSecondaryViewConfigurationStateMSFT
- XrSecondaryViewConfigurationFrameStateMSFT
- XrSecondaryViewConfigurationFrameEndInfoMSFT

- XrSecondaryViewConfigurationLayerInfoMSFT

New Functions

Issues

Version History

- Revision 1, 2019-07-30 (Yin Li)
  - Initial extension description

14.35. XR_MSFT_spatial_anchor

Name String

`XR_MSFT_spatial_anchor`

Extension Type

Instance extension

Registered Extension Number

40

Revision

2

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Deprecation State

- *Obsoleted* by `XR_EXT_spatial_anchor` extension

Overview

This extension allows an application to create a **spatial anchor**, an arbitrary freespace point in the user's physical environment that will then be tracked by the runtime. The runtime **should** then adjust the position and orientation of that anchor's origin over time as needed, independently of all other spaces and anchors, to ensure that it maintains its original mapping to the real world.

```
// Provided by XR_MSFT_spatial_anchor
XR_DEFINE_HANDLE(XrSpatialAnchorMSFT)
```

Spatial anchors are often used in combination with an UNBOUNDED_MSFT reference space. UNBOUNDED_MSFT reference spaces adjust their origin as necessary to keep the viewer's coordinates relative to the space's origin stable. Such adjustments maintain the visual stability of content currently near the viewer, but may cause content placed far from the viewer to drift in its alignment to the real world by the time the user moves close again. By creating an XrSpatialAnchorMSFT where a piece of content is placed and then always rendering that content relative to its anchor's space, an application can ensure that each piece of content stays at a fixed location in the environment.

The xrCreateSpatialAnchorMSFT function is defined as:

```
// Provided by XR_MSFT_spatial_anchor
XrResult xrCreateSpatialAnchorMSFT(
    XrSession                                   session,
    const XrSpatialAnchorCreateInfoMSFT*        createInfo,
    XrSpatialAnchorMSFT*                        anchor);
```

Parameter Descriptions

- session is a handle to an XrSession.
- createInfo is a pointer to an XrSpatialAnchorCreateInfoMSFT structure containing information about how to create the anchor.
- anchor is a pointer to a handle in which the created XrSpatialAnchorMSFT is returned.

Creates an XrSpatialAnchorMSFT handle representing a spatial anchor that will track a fixed location in the physical world over time. That real-world location is specified by the position and orientation of the specified XrSpatialAnchorCreateInfoMSFT::pose within XrSpatialAnchorCreateInfoMSFT::space at XrSpatialAnchorCreateInfoMSFT::time.

The runtime **must** avoid long blocking operations such as networking or disk operations for xrCreateSpatialAnchorMSFT function. The application **may** safely use this function in UI thread. Though, the created anchor handle **may** not be ready immediately for certain operations yet. For example, the corresponding anchor space **may** not return valid location, or its location **may** not be successfully saved in anchor store.

If XrSpatialAnchorCreateInfoMSFT::space cannot be located relative to the environment at the moment of the call to xrCreateSpatialAnchorMSFT, the runtime **must** return XR_ERROR_CREATE_SPATIAL_ANCHOR_FAILED_MSFT.

After the anchor is created, the runtime **should** then adjust its position and orientation over time relative to other spaces so as to maintain maximum alignment to its original real-world location, even if that changes the anchor's relationship to the original XrSpatialAnchorCreateInfoMSFT::space used to initialize it.

Valid Usage (Implicit)

- The XR_MSFT_spatial_anchor extension **must** be enabled prior to calling xrCreateSpatialAnchorMSFT
- `session` **must** be a valid XrSession handle
- `createInfo` **must** be a pointer to a valid XrSpatialAnchorCreateInfoMSFT structure
- `anchor` **must** be a pointer to an XrSpatialAnchorMSFT handle

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_CREATE_SPATIAL_ANCHOR_FAILED_MSFT
- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_POSE_INVALID
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_TIME_INVALID
- XR_ERROR_VALIDATION_FAILURE

The XrSpatialAnchorCreateInfoMSFT structure is defined as:

```
// Provided by XR_MSFT_spatial_anchor
typedef struct XrSpatialAnchorCreateInfoMSFT {
    XrStructureType    type;
    const void*        next;
    XrSpace            space;
    XrPosef            pose;
    XrTime             time;
} XrSpatialAnchorCreateInfoMSFT;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `space` is a handle to the XrSpace in which `pose` is specified.
- `pose` is the XrPosef within `space` at `time` that specifies the point in the real world used to initialize the new anchor.
- `time` is the `XrTime` at which `pose` will be evaluated within `space`.

Valid Usage (Implicit)

- The `XR_MSFT_spatial_anchor` extension **must** be enabled prior to using XrSpatialAnchorCreateInfoMSFT
- `type` **must** be `XR_TYPE_SPATIAL_ANCHOR_CREATE_INFO_MSFT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `space` **must** be a valid XrSpace handle

The xrCreateSpatialAnchorSpaceMSFT function is defined as:

```
// Provided by XR_MSFT_spatial_anchor
XrResult xrCreateSpatialAnchorSpaceMSFT(
    XrSession                                   session,
    const XrSpatialAnchorSpaceCreateInfoMSFT*   createInfo,
    XrSpace*                                    space);
```

Parameter Descriptions

- `session` is a handle to an XrSession.
- `createInfo` is a pointer to an XrSpatialAnchorSpaceCreateInfoMSFT structure containing information about how to create the anchor.
- `space` is a pointer to a handle in which the created XrSpace is returned.

Creates an XrSpace handle based on a spatial anchor. Application **can** provide an XrPosef to define the position and orientation of the new space's origin relative to the anchor's natural origin.

Multiple XrSpace handles may exist for a given XrSpatialAnchorMSFT simultaneously, up to some limit imposed by the runtime. The XrSpace handle must be eventually freed via the xrDestroySpace function or by destroying the parent XrSpatialAnchorMSFT handle.

Valid Usage (Implicit)

- The `XR_MSFT_spatial_anchor` extension **must** be enabled prior to calling xrCreateSpatialAnchorSpaceMSFT
- `session` **must** be a valid XrSession handle
- `createInfo` **must** be a pointer to a valid XrSpatialAnchorSpaceCreateInfoMSFT structure
- `space` **must** be a pointer to an XrSpace handle

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_POSE_INVALID
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The XrSpatialAnchorSpaceCreateInfoMSFT structure is defined as:

```
// Provided by XR_MSFT_spatial_anchor
typedef struct XrSpatialAnchorSpaceCreateInfoMSFT {
    XrStructureType        type;
    const void*            next;
    XrSpatialAnchorMSFT    anchor;
    XrPosef                poseInAnchorSpace;
} XrSpatialAnchorSpaceCreateInfoMSFT;
```

Member Descriptions

- type is the XrStructureType of this structure.
- next is NULL or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- anchor is a handle to an XrSpatialAnchorMSFT previously created with xrCreateSpatialAnchorMSFT.
- poseInAnchorSpace is an XrPosef defining the position and orientation of the new space's origin relative to the anchor's natural origin.

Valid Usage (Implicit)

- The XR_MSFT_spatial_anchor extension **must** be enabled prior to using XrSpatialAnchorSpaceCreateInfoMSFT
- type **must** be XR_TYPE_SPATIAL_ANCHOR_SPACE_CREATE_INFO_MSFT
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- anchor **must** be a valid XrSpatialAnchorMSFT handle

The xrDestroySpatialAnchorMSFT function is defined as:

```
// Provided by XR_MSFT_spatial_anchor
XrResult xrDestroySpatialAnchorMSFT(
    XrSpatialAnchorMSFT                         anchor);
```

Parameter Descriptions

- anchor is a handle to an XrSpatialAnchorMSFT previously created by xrCreateSpatialAnchorMSFT.

XrSpatialAnchorMSFT handles are destroyed using xrDestroySpatialAnchorMSFT. By destroying an anchor, the runtime **can** stop spending resources used to maintain tracking for that anchor's origin.

Valid Usage (Implicit)

- The XR_MSFT_spatial_anchor extension **must** be enabled prior to calling xrDestroySpatialAnchorMSFT
- anchor **must** be a valid XrSpatialAnchorMSFT handle

Thread Safety

- Access to anchor, and any child handles, **must** be externally synchronized

Return Codes

Success

- `XR_SUCCESS`

Failure

- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_RUNTIME_FAILURE`

New Object Types

XrSpatialAnchorMSFT

New Flag Types

New Enum Constants

XrObjectType enumeration is extended with:

- `XR_OBJECT_TYPE_SPATIAL_ANCHOR_MSFT`

XrStructureType enumeration is extended with:

- `XR_TYPE_SPATIAL_ANCHOR_CREATE_INFO_MSFT`
- `XR_TYPE_SPATIAL_ANCHOR_SPACE_CREATE_INFO_MSFT`

XrResult enumeration is extended with:

- `XR_ERROR_CREATE_SPATIAL_ANCHOR_FAILED_MSFT`

New Enums

New Structures

XrSpatialAnchorCreateInfoMSFT

XrSpatialAnchorSpaceCreateInfoMSFT

New Functions

xrCreateSpatialAnchorMSFT

xrCreateSpatialAnchorSpaceMSFT

xrDestroySpatialAnchorMSFT

Issues

Version History

- Revision 1, 2019-07-30 (Alex Turner)
  - Initial extension description
- Revision 2, 2021-06-02 (Rylie Pavlik, Collabora, Ltd.)
  - Note that the parameter to `xrDestroySpatialAnchorMSFT` must be externally synchronized

14.36. XR_MSFT_spatial_anchor_persistence

Name String

`XR_MSFT_spatial_anchor_persistence`

Extension Type

Instance extension

Registered Extension Number

143

Revision

2

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0
and
`XR_MSFT_spatial_anchor`

Deprecation State

- *Obsoleted* by `XR_EXT_spatial_persistence` extension

Last Modified Date

2021-07-15

IP Status

No known IP claims.

Contributors

Lachlan Ford, Microsoft
Yin Li, Microsoft
Norman Pohl, Microsoft

Alex Turner, Microsoft
Bryce Hutchings, Microsoft

14.36.1. Overview

This extension allows persistence and retrieval of spatial anchors sharing and localization across application sessions on a device. Spatial anchors persisted during an application session on a device will only be able to be retrieved during sessions of that same application on the same device. This extension requires `XR_MSFT_spatial_anchor` to also be enabled.

14.36.2. Spatial Anchor Store Connection

The XrSpatialAnchorStoreConnectionMSFT handle represents a connection to the spatial anchor store and is used by the application to perform operations on the spatial anchor store such as:

- Persisting and unpersisting of spatial anchors.
- Enumeration of currently persisted anchors.
- Clearing the spatial anchor store of all anchors.

```
// Provided by XR_MSFT_spatial_anchor_persistence
XR_DEFINE_HANDLE(XrSpatialAnchorStoreConnectionMSFT)
```

The application **can** use the xrCreateSpatialAnchorStoreConnectionMSFT function to create an handle to the spatial anchor store. The application **can** use this handle to interact with the spatial anchor store in order to persist anchors across application sessions.

The xrCreateSpatialAnchorStoreConnectionMSFT function **may** be a slow operation and therefore **should** be invoked from a non-timing critical thread.

```
// Provided by XR_MSFT_spatial_anchor_persistence
XrResult xrCreateSpatialAnchorStoreConnectionMSFT(
    XrSession                                   session,
    XrSpatialAnchorStoreConnectionMSFT*         spatialAnchorStore);
```

Parameter Descriptions

- `session` is the XrSession the anchor was created with.
- `spatialAnchorStore` is a pointer to the XrSpatialAnchorStoreConnectionMSFT handle.

Valid Usage (Implicit)

- The XR_MSFT_spatial_anchor_persistence extension **must** be enabled prior to calling xrCreateSpatialAnchorStoreConnectionMSFT
- session **must** be a valid XrSession handle
- spatialAnchorStore **must** be a pointer to an XrSpatialAnchorStoreConnectionMSFT handle

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The application **can** use the xrDestroySpatialAnchorStoreConnectionMSFT function to destroy an anchor store connection.

```
// Provided by XR_MSFT_spatial_anchor_persistence
XrResult xrDestroySpatialAnchorStoreConnectionMSFT(
    XrSpatialAnchorStoreConnectionMSFT          spatialAnchorStore);
```

Parameter Descriptions

- spatialAnchorStore is the XrSpatialAnchorStoreConnectionMSFT to be destroyed.

Valid Usage (Implicit)

- The XR_MSFT_spatial_anchor_persistence extension **must** be enabled prior to calling xrDestroySpatialAnchorStoreConnectionMSFT
- spatialAnchorStore **must** be a valid XrSpatialAnchorStoreConnectionMSFT handle

Thread Safety

- Access to spatialAnchorStore, and any child handles, **must** be externally synchronized

Return Codes

Success

- XR_SUCCESS

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE

14.36.3. Persist Spatial Anchor

The application **can** use the xrPersistSpatialAnchorMSFT function to persist a spatial anchor in the spatial anchor store for this application. The given XrSpatialAnchorPersistenceInfoMSFT ::spatialAnchorPersistenceName will be the string to retrieve the spatial anchor from the Spatial Anchor store or subsequently remove the record of this spatial anchor from the store. This name will uniquely identify the spatial anchor for the current application. If there is already a spatial anchor of the same name persisted in the spatial anchor store, the existing spatial anchor will be replaced and xrPersistSpatialAnchorMSFT **must** return XR_SUCCESS.

```
// Provided by XR_MSFT_spatial_anchor_persistence
XrResult xrPersistSpatialAnchorMSFT(
    XrSpatialAnchorStoreConnectionMSFT          spatialAnchorStore,
    const XrSpatialAnchorPersistenceInfoMSFT*   spatialAnchorPersistenceInfo);
```

Parameter Descriptions

- `spatialAnchorStore` is the XrSpatialAnchorStoreConnectionMSFT with which to persist the XrSpatialAnchorPersistenceInfoMSFT::`spatialAnchor`.
- `spatialAnchorPersistenceInfo` is a pointer to XrSpatialAnchorPersistenceInfoMSFT structure to specify the anchor and its name to persist.

Valid Usage (Implicit)

- The `XR_MSFT_spatial_anchor_persistence` extension **must** be enabled prior to calling xrPersistSpatialAnchorMSFT
- `spatialAnchorStore` **must** be a valid XrSpatialAnchorStoreConnectionMSFT handle
- `spatialAnchorPersistenceInfo` **must** be a pointer to a valid XrSpatialAnchorPersistenceInfoMSFT structure

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_OUT_OF_MEMORY`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_SPATIAL_ANCHOR_NAME_INVALID_MSFT`
- `XR_ERROR_VALIDATION_FAILURE`

The XrSpatialAnchorPersistenceNameMSFT structure is the name associated with the XrSpatialAnchorMSFT in the spatial anchor store. It is used to perform persist and unpersist on an `name` in the spatial anchor store.

The XrSpatialAnchorPersistenceNameMSFT structure is defined as:

```
// Provided by XR_MSFT_spatial_anchor_persistence
typedef struct XrSpatialAnchorPersistenceNameMSFT {
    char    name[XR_MAX_SPATIAL_ANCHOR_NAME_SIZE_MSFT];
} XrSpatialAnchorPersistenceNameMSFT;
```

Member Descriptions

- `name` is a null terminated character array of size `XR_MAX_SPATIAL_ANCHOR_NAME_SIZE_MSFT`.

If an XrSpatialAnchorPersistenceNameMSFT with an empty `name` value is passed to any function as a parameter, that function **must** return `XR_ERROR_SPATIAL_ANCHOR_NAME_INVALID_MSFT`.

Valid Usage (Implicit)

- The `XR_MSFT_spatial_anchor_persistence` extension **must** be enabled prior to using XrSpatialAnchorPersistenceNameMSFT
- `name` **must** be a null-terminated UTF-8 string whose length is less than or equal to `XR_MAX_SPATIAL_ANCHOR_NAME_SIZE_MSFT`

The XrSpatialAnchorPersistenceInfoMSFT structure is defined as:

```
// Provided by XR_MSFT_spatial_anchor_persistence
typedef struct XrSpatialAnchorPersistenceInfoMSFT {
    XrStructureType                     type;
    const void*                         next;
    XrSpatialAnchorPersistenceNameMSFT  spatialAnchorPersistenceName;
    XrSpatialAnchorMSFT                 spatialAnchor;
} XrSpatialAnchorPersistenceInfoMSFT;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR.
- `spatialAnchorPersistenceName` is an XrSpatialAnchorPersistenceNameMSFT containing the name associated with the XrSpatialAnchorMSFT in the spatial anchor store.
- `spatialAnchor` is the XrSpatialAnchorMSFT that the application wishes to perform persistence operations on.

Valid Usage (Implicit)

- The XR_MSFT_spatial_anchor_persistence extension **must** be enabled prior to using XrSpatialAnchorPersistenceInfoMSFT
- type **must** be XR_TYPE_SPATIAL_ANCHOR_PERSISTENCE_INFO_MSFT
- next **must** be NULL or a valid pointer to the next structure in a structure chain
- spatialAnchorPersistenceName **must** be a valid XrSpatialAnchorPersistenceNameMSFT structure
- spatialAnchor **must** be a valid XrSpatialAnchorMSFT handle

The application **can** use the xrEnumeratePersistedSpatialAnchorNamesMSFT function to enumerate the names of all spatial anchors currently persisted in the spatial anchor store for this application. This function follows the two-call idiom for filling the spatialAnchorNames.

```
// Provided by XR_MSFT_spatial_anchor_persistence
XrResult xrEnumeratePersistedSpatialAnchorNamesMSFT(
    XrSpatialAnchorStoreConnectionMSFT          spatialAnchorStore,
    uint32_t                                    spatialAnchorNameCapacityInput,
    uint32_t*                                   spatialAnchorNameCountOutput,
    XrSpatialAnchorPersistenceNameMSFT*         spatialAnchorNames);
```

Parameter Descriptions

- spatialAnchorStore is the XrSpatialAnchorStoreConnectionMSFT anchor store to perform the enumeration operation on.
- spatialAnchorNameCapacityInput is the capacity of the spatialAnchorNames array, or 0 to indicate a request to retrieve the required capacity.
- spatialAnchorNameCountOutput is filled in by the runtime with the count of anchor names written or the required capacity in the case that spatialAnchorNameCapacityInput is insufficient.
- spatialAnchorNames is a pointer to an array of XrSpatialAnchorPersistenceNameMSFT structures, but **can** be NULL if propertyCapacityInput is 0.
- See the Buffer Size Parameters section for a detailed description of retrieving the required spatialAnchorNames size.

Valid Usage (Implicit)

- The XR_MSFT_spatial_anchor_persistence extension **must** be enabled prior to calling xrEnumeratePersistedSpatialAnchorNamesMSFT
- spatialAnchorStore **must** be a valid XrSpatialAnchorStoreConnectionMSFT handle
- spatialAnchorNameCountOutput **must** be a pointer to a uint32_t value
- If spatialAnchorNameCapacityInput is not 0, spatialAnchorNames **must** be a pointer to an array of spatialAnchorNameCapacityInput XrSpatialAnchorPersistenceNameMSFT structures

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_SIZE_INSUFFICIENT
- XR_ERROR_VALIDATION_FAILURE

The application **can** use the xrCreateSpatialAnchorFromPersistedNameMSFT function to create a XrSpatialAnchorMSFT from the spatial anchor store. If the XrSpatialAnchorFromPersistedAnchorCreateInfoMSFT::spatialAnchorPersistenceName provided does not correspond to a currently stored anchor (i.e. the list of spatial anchor names returned from xrEnumeratePersistedSpatialAnchorNamesMSFT), the function **must** return XR_ERROR_SPATIAL_ANCHOR_NAME_NOT_FOUND_MSFT.

```
// Provided by XR_MSFT_spatial_anchor_persistence
XrResult xrCreateSpatialAnchorFromPersistedNameMSFT(
    XrSession                                       session,
    const XrSpatialAnchorFromPersistedAnchorCreateInfoMSFT* spatialAnchorCreateInfo,
    XrSpatialAnchorMSFT*                            spatialAnchor);
```

Parameter Descriptions

- `session` is a handle to an XrSession previously created with xrCreateSession.
- `spatialAnchorCreateInfo` is a pointer to the XrSpatialAnchorFromPersistedAnchorCreateInfoMSFT.
- `spatialAnchor` is a pointer to an XrSpatialAnchorMSFT handle that will be set by the runtime on successful load.

Valid Usage (Implicit)

- The `XR_MSFT_spatial_anchor_persistence` extension **must** be enabled prior to calling xrCreateSpatialAnchorFromPersistedNameMSFT
- `session` **must** be a valid XrSession handle
- `spatialAnchorCreateInfo` **must** be a pointer to a valid XrSpatialAnchorFromPersistedAnchorCreateInfoMSFT structure
- `spatialAnchor` **must** be a pointer to an XrSpatialAnchorMSFT handle

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_LIMIT_REACHED`
- `XR_ERROR_OUT_OF_MEMORY`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_SPATIAL_ANCHOR_NAME_INVALID_MSFT`
- `XR_ERROR_SPATIAL_ANCHOR_NAME_NOT_FOUND_MSFT`
- `XR_ERROR_VALIDATION_FAILURE`

The XrSpatialAnchorFromPersistedAnchorCreateInfoMSFT structure is defined as:

```
// Provided by XR_MSFT_spatial_anchor_persistence
typedef struct XrSpatialAnchorFromPersistedAnchorCreateInfoMSFT {
    XrStructureType                         type;
    const void*                             next;
    XrSpatialAnchorStoreConnectionMSFT    spatialAnchorStore;
    XrSpatialAnchorPersistenceNameMSFT    spatialAnchorPersistenceName;
} XrSpatialAnchorFromPersistedAnchorCreateInfoMSFT;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR.
- `spatialAnchorStore` is the XrSpatialAnchorStoreConnectionMSFT from which the spatial anchor will be loaded from.
- `spatialAnchorPersistenceName` is the XrSpatialAnchorPersistenceNameMSFT associated with the XrSpatialAnchorMSFT in the spatial anchor store. This name is used to create an XrSpatialAnchorMSFT handle from a spatial anchor persisted in the spatial anchor store.

The `spatialAnchorPersistenceName` is a character array of maximum size `XR_MAX_SPATIAL_ANCHOR_NAME_SIZE_MSFT`, which **must** include a null terminator and **must** not be empty (i.e. the first element is the null terminator). If an empty `spatialAnchorPersistenceName` value is passed to any function as a parameter, that function **must** return `XR_ERROR_SPATIAL_ANCHOR_NAME_INVALID_MSFT`.

Valid Usage (Implicit)

- The `XR_MSFT_spatial_anchor_persistence` extension **must** be enabled prior to using XrSpatialAnchorFromPersistedAnchorCreateInfoMSFT
- `type` **must** be `XR_TYPE_SPATIAL_ANCHOR_FROM_PERSISTED_ANCHOR_CREATE_INFO_MSFT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `spatialAnchorStore` **must** be a valid XrSpatialAnchorStoreConnectionMSFT handle
- `spatialAnchorPersistenceName` **must** be a valid XrSpatialAnchorPersistenceNameMSFT structure

The application **can** use the xrUnpersistSpatialAnchorMSFT function to remove the record of the anchor in the spatial anchor store. This operation will not affect any XrSpatialAnchorMSFT handles previously created. If the `spatialAnchorPersistenceName` provided does not correspond to a currently

stored anchor, the function **must** return XR_ERROR_SPATIAL_ANCHOR_NAME_NOT_FOUND_MSFT.

```
// Provided by XR_MSFT_spatial_anchor_persistence
XrResult xrUnpersistSpatialAnchorMSFT(
    XrSpatialAnchorStoreConnectionMSFT          spatialAnchorStore,
    const XrSpatialAnchorPersistenceNameMSFT*   spatialAnchorPersistenceName);
```

Parameter Descriptions

- spatialAnchorStore is an XrSpatialAnchorStoreConnectionMSFT anchor store to perform the unpersist operation on.
- spatialAnchorPersistenceName is a pointer to the XrSpatialAnchorPersistenceNameMSFT.

Valid Usage (Implicit)

- The XR_MSFT_spatial_anchor_persistence extension **must** be enabled prior to calling xrUnpersistSpatialAnchorMSFT
- spatialAnchorStore **must** be a valid XrSpatialAnchorStoreConnectionMSFT handle
- spatialAnchorPersistenceName **must** be a pointer to a valid XrSpatialAnchorPersistenceNameMSFT structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_SPATIAL_ANCHOR_NAME_INVALID_MSFT
- XR_ERROR_SPATIAL_ANCHOR_NAME_NOT_FOUND_MSFT
- XR_ERROR_VALIDATION_FAILURE

The application **can** use the xrClearSpatialAnchorStoreMSFT function to remove all spatial anchors from the spatial anchor store for this application. The function only removes the record of the spatial anchors in the store but does not affect any XrSpatialAnchorMSFT handles previously loaded in the current session.

```
// Provided by XR_MSFT_spatial_anchor_persistence
XrResult xrClearSpatialAnchorStoreMSFT(
    XrSpatialAnchorStoreConnectionMSFT          spatialAnchorStore);
```

Parameter Descriptions

- spatialAnchorStore is XrSpatialAnchorStoreConnectionMSFT to perform the clear operation on.

Valid Usage (Implicit)

- The XR_MSFT_spatial_anchor_persistence extension **must** be enabled prior to calling xrClearSpatialAnchorStoreMSFT
- spatialAnchorStore **must** be a valid XrSpatialAnchorStoreConnectionMSFT handle

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

New Object Types

- XrSpatialAnchorStoreConnectionMSFT

New Flag Types

New Enum Constants

- XR_TYPE_SPATIAL_ANCHOR_PERSISTENCE_INFO_MSFT
- XR_TYPE_SPATIAL_ANCHOR_FROM_PERSISTED_ANCHOR_CREATE_INFO_MSFT
- XR_ERROR_SPATIAL_ANCHOR_NAME_NOT_FOUND_MSFT
- XR_ERROR_SPATIAL_ANCHOR_NAME_INVALID_MSFT
- XR_MAX_SPATIAL_ANCHOR_NAME_SIZE_MSFT

New Enums

New Structures

- XrSpatialAnchorPersistenceNameMSFT
- XrSpatialAnchorPersistenceInfoMSFT
- XrSpatialAnchorFromPersistedAnchorCreateInfoMSFT

New Functions

- xrCreateSpatialAnchorStoreConnectionMSFT

- xrDestroySpatialAnchorStoreConnectionMSFT
- xrPersistSpatialAnchorMSFT
- xrEnumeratePersistedSpatialAnchorNamesMSFT
- xrCreateSpatialAnchorFromPersistedNameMSFT
- xrUnpersistSpatialAnchorMSFT
- xrClearSpatialAnchorStoreMSFT

Version History

- Revision 1, 2021-02-19 (Lachlan Ford)
  - Initial extension proposal
- Revision 2, 2021-07-15 (Yin Li)
  - Extension proposal to OpenXR working group

14.37. XR_MSFT_spatial_graph_bridge

Name String

`XR_MSFT_spatial_graph_bridge`

Extension Type

Instance extension

Registered Extension Number

50

Revision

2

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Deprecation State

- *Deprecated* without replacement

Contributors

Darryl Gough, Microsoft
Yin Li, Microsoft
Alex Turner, Microsoft
David Fields, Microsoft

Overview

This extension enables applications to interop between XrSpace handles and other Windows Mixed Reality device platform libraries or APIs. These libraries represent a spatially tracked point, also known as a "spatial graph node", with a GUID value. This extension enables applications to create XrSpace handles from spatial graph nodes. Applications can also try to get a spatial graph node from an XrSpace handle.

14.37.1. Create XrSpace from Spatial Graph Node

The xrCreateSpatialGraphNodeSpaceMSFT function creates an XrSpace handle for a given spatial graph node type and ID.

```
// Provided by XR_MSFT_spatial_graph_bridge
XrResult xrCreateSpatialGraphNodeSpaceMSFT(
    XrSession                                     session,
    const XrSpatialGraphNodeSpaceCreateInfoMSFT* createInfo,
    XrSpace*                                      space);
```

Parameter Descriptions

- `session` is the XrSession which will use the created space.
- `createInfo` is an XrSpatialGraphNodeSpaceCreateInfoMSFT specifying the space to be created.
- `space` is the returned XrSpace handle for the given spatial node ID.

Valid Usage (Implicit)

- The `XR_MSFT_spatial_graph_bridge` extension **must** be enabled prior to calling xrCreateSpatialGraphNodeSpaceMSFT
- `session` **must** be a valid XrSession handle
- `createInfo` **must** be a pointer to a valid XrSpatialGraphNodeSpaceCreateInfoMSFT structure
- `space` **must** be a pointer to an XrSpace handle

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_LIMIT_REACHED
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_POSE_INVALID
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

The XrSpatialGraphNodeSpaceCreateInfoMSFT structure is used with xrCreateSpatialGraphNodeSpaceMSFT to create an XrSpace handle for a given spatial node type and node ID.

```
// Provided by XR_MSFT_spatial_graph_bridge
typedef struct XrSpatialGraphNodeSpaceCreateInfoMSFT {
    XrStructureType               type;
    const void*                   next;
    XrSpatialGraphNodeTypeMSFT    nodeType;
    uint8_t                       nodeId[XR_GUID_SIZE_MSFT];
    XrPosef                       pose;
} XrSpatialGraphNodeSpaceCreateInfoMSFT;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain. No such structures are defined in core OpenXR or this extension.
- `nodeType` is an XrSpatialGraphNodeTypeMSFT specifying the spatial node type.
- `nodeId` is a global unique identifier (a.k.a. GUID or 16 byte array), representing the spatial node that is being tracked.
- `pose` is an XrPosef defining the position and orientation of the new space's origin within the natural reference frame of the spatial graph node.

Valid Usage (Implicit)

- The `XR_MSFT_spatial_graph_bridge` extension **must** be enabled prior to using XrSpatialGraphNodeSpaceCreateInfoMSFT
- `type` **must** be `XR_TYPE_SPATIAL_GRAPH_NODE_SPACE_CREATE_INFO_MSFT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `nodeType` **must** be a valid XrSpatialGraphNodeTypeMSFT value

The enum XrSpatialGraphNodeTypeMSFT describes the types of spatial graph nodes.

```
// Provided by XR_MSFT_spatial_graph_bridge
typedef enum XrSpatialGraphNodeTypeMSFT {
    XR_SPATIAL_GRAPH_NODE_TYPE_STATIC_MSFT = 1,
    XR_SPATIAL_GRAPH_NODE_TYPE_DYNAMIC_MSFT = 2,
    XR_SPATIAL_GRAPH_NODE_TYPE_MAX_ENUM_MSFT = 0x7FFFFFFF
} XrSpatialGraphNodeTypeMSFT;
```

There are two types of spatial graph nodes: static and dynamic.

Static spatial nodes track the pose of a fixed location in the world relative to reference spaces. The tracking of static nodes **may** slowly adjust the pose over time for better accuracy but the pose is relatively stable in the short term, such as between rendering frames. For example, a QR code tracking library can use a static node to represent the location of the tracked QR code. Static spatial nodes are represented by `XR_SPATIAL_GRAPH_NODE_TYPE_STATIC_MSFT`.

Dynamic spatial nodes track the pose of a physical object that moves continuously relative to reference spaces. The pose of dynamic spatial nodes **can** be very different within the duration of a rendering frame. It is important for the application to use the correct timestamp to query the space location using xrLocateSpace. For example, a color camera mounted in front of a HMD is also tracked by the HMD so

a web camera library can use a dynamic node to represent the camera location. Dynamic spatial nodes are represented by XR_SPATIAL_GRAPH_NODE_TYPE_DYNAMIC_MSFT.

14.37.2. Create Spatial Graph Node Binding from XrSpace

The XrSpatialGraphNodeBindingMSFT handle represents a binding to a spatial graph node. This handle allows an application to get a spatial graph node GUID from an XrSpace to use in other Windows Mixed Reality device platform libraries or APIs.

The runtime **must** remember the spatial graph node and track it for the lifetime of the XrSpatialGraphNodeBindingMSFT handle. When the XrSpatialGraphNodeBindingMSFT handle is destroyed then the runtime's tracking system **may** forget about the spatial graphic node and stop tracking it.

```
// Provided by XR_MSFT_spatial_graph_bridge
XR_DEFINE_HANDLE(XrSpatialGraphNodeBindingMSFT)
```

The xrTryCreateSpatialGraphStaticNodeBindingMSFT function tries to create a binding to the best spatial graph static node relative to the given location and returns an XrSpatialGraphNodeBindingMSFT handle.

```
// Provided by XR_MSFT_spatial_graph_bridge
XrResult xrTryCreateSpatialGraphStaticNodeBindingMSFT(
    XrSession                                       session,
    const XrSpatialGraphStaticNodeBindingCreateInfoMSFT* createInfo,
    XrSpatialGraphNodeBindingMSFT*                  nodeBinding);
```

Parameter Descriptions

- session is the specified XrSession.
- createInfo is the XrSpatialGraphStaticNodeBindingCreateInfoMSFT input structure.
- nodeBinding is the XrSpatialGraphNodeBindingMSFT output structure.

The runtime **may** return XR_SUCCESS and set nodeBinding to XR_NULL_HANDLE if it is unable to create a spatial graph static node binding. This may happen when the given XrSpace cannot be properly tracked at the moment. The application can retry creating the XrSpatialGraphNodeBindingMSFT handle again after a reasonable period of time when tracking is regained.

The xrTryCreateSpatialGraphStaticNodeBindingMSFT function **may** be a slow operation and therefore **should** be invoked from a non-timing critical thread.

Valid Usage (Implicit)

- The `XR_MSFT_spatial_graph_bridge` extension **must** be enabled prior to calling xrTryCreateSpatialGraphStaticNodeBindingMSFT
- `session` **must** be a valid XrSession handle
- `createInfo` **must** be a pointer to a valid XrSpatialGraphStaticNodeBindingCreateInfoMSFT structure
- `nodeBinding` **must** be a pointer to an XrSpatialGraphNodeBindingMSFT handle

Return Codes

Success

- `XR_SUCCESS`
- `XR_SESSION_LOSS_PENDING`

Failure

- `XR_ERROR_FUNCTION_UNSUPPORTED`
- `XR_ERROR_HANDLE_INVALID`
- `XR_ERROR_INSTANCE_LOST`
- `XR_ERROR_LIMIT_REACHED`
- `XR_ERROR_OUT_OF_MEMORY`
- `XR_ERROR_POSE_INVALID`
- `XR_ERROR_RUNTIME_FAILURE`
- `XR_ERROR_SESSION_LOST`
- `XR_ERROR_TIME_INVALID`
- `XR_ERROR_VALIDATION_FAILURE`

XrSpatialGraphStaticNodeBindingCreateInfoMSFT is an input structure for xrTryCreateSpatialGraphStaticNodeBindingMSFT.

```
// Provided by XR_MSFT_spatial_graph_bridge
typedef struct XrSpatialGraphStaticNodeBindingCreateInfoMSFT {
    XrStructureType    type;
    const void*        next;
    XrSpace            space;
    XrPosef            poseInSpace;
    XrTime             time;
} XrSpatialGraphStaticNodeBindingCreateInfoMSFT;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `space` is a handle to the XrSpace in which `poseInSpace` is specified.
- `poseInSpace` is the XrPosef within `space` at `time`.
- `time` is the `XrTime` at which `poseInSpace` will be evaluated within `space`.

Valid Usage (Implicit)

- The `XR_MSFT_spatial_graph_bridge` extension **must** be enabled prior to using XrSpatialGraphStaticNodeBindingCreateInfoMSFT
- `type` **must** be `XR_TYPE_SPATIAL_GRAPH_STATIC_NODE_BINDING_CREATE_INFO_MSFT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain
- `space` **must** be a valid XrSpace handle

The xrDestroySpatialGraphNodeBindingMSFT function releases the `nodeBinding` and the underlying resources.

```
// Provided by XR_MSFT_spatial_graph_bridge
XrResult xrDestroySpatialGraphNodeBindingMSFT(
    XrSpatialGraphNodeBindingMSFT               nodeBinding);
```

Parameter Descriptions

- `nodeBinding` is an XrSpatialGraphNodeBindingMSFT previously created by xrTryCreateSpatialGraphStaticNodeBindingMSFT.

Valid Usage (Implicit)

- The XR_MSFT_spatial_graph_bridge extension **must** be enabled prior to calling xrDestroySpatialGraphNodeBindingMSFT
- nodeBinding **must** be a valid XrSpatialGraphNodeBindingMSFT handle

Thread Safety

- Access to nodeBinding, and any child handles, **must** be externally synchronized

Return Codes

Success

- XR_SUCCESS

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_RUNTIME_FAILURE

Get spatial graph node binding properties

The xrGetSpatialGraphNodeBindingPropertiesMSFT function retrieves the spatial graph node GUID and the pose in the node space from an XrSpatialGraphNodeBindingMSFT handle.

```
// Provided by XR_MSFT_spatial_graph_bridge
XrResult xrGetSpatialGraphNodeBindingPropertiesMSFT(
    XrSpatialGraphNodeBindingMSFT                nodeBinding,
    const XrSpatialGraphNodeBindingPropertiesGetInfoMSFT* getInfo,
    XrSpatialGraphNodeBindingPropertiesMSFT*     properties);
```

Parameter Descriptions

- nodeBinding is an XrSpatialGraphNodeBindingMSFT previously created by xrTryCreateSpatialGraphStaticNodeBindingMSFT.
- getInfo is a pointer to an XrSpatialGraphNodeBindingPropertiesGetInfoMSFT input structure.
- properties is a pointer to an XrSpatialGraphNodeBindingPropertiesMSFT output structure.

Valid Usage (Implicit)

- The XR_MSFT_spatial_graph_bridge extension **must** be enabled prior to calling xrGetSpatialGraphNodeBindingPropertiesMSFT
- nodeBinding **must** be a valid XrSpatialGraphNodeBindingMSFT handle
- If getInfo is not NULL, getInfo **must** be a pointer to a valid XrSpatialGraphNodeBindingPropertiesGetInfoMSFT structure
- properties **must** be a pointer to an XrSpatialGraphNodeBindingPropertiesMSFT structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_OUT_OF_MEMORY
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_VALIDATION_FAILURE

XrSpatialGraphNodeBindingPropertiesGetInfoMSFT is an input structure for xrGetSpatialGraphNodeBindingPropertiesMSFT.

```
// Provided by XR_MSFT_spatial_graph_bridge
typedef struct XrSpatialGraphNodeBindingPropertiesGetInfoMSFT {
    XrStructureType    type;
    const void*        next;
} XrSpatialGraphNodeBindingPropertiesGetInfoMSFT;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.

Valid Usage (Implicit)

- The `XR_MSFT_spatial_graph_bridge` extension **must** be enabled prior to using XrSpatialGraphNodeBindingPropertiesGetInfoMSFT
- `type` **must** be `XR_TYPE_SPATIAL_GRAPH_NODE_BINDING_PROPERTIES_GET_INFO_MSFT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

XrSpatialGraphNodeBindingPropertiesMSFT is an output structure for xrGetSpatialGraphNodeBindingPropertiesMSFT.

```
// Provided by XR_MSFT_spatial_graph_bridge
typedef struct XrSpatialGraphNodeBindingPropertiesMSFT {
    XrStructureType    type;
    void*              next;
    uint8_t            nodeId[XR_GUID_SIZE_MSFT];
    XrPosef            poseInNodeSpace;
} XrSpatialGraphNodeBindingPropertiesMSFT;
```

Member Descriptions

- `type` is the XrStructureType of this structure.
- `next` is `NULL` or a pointer to the next structure in a structure chain.
- `nodeId` is a global unique identifier (a.k.a. GUID or 16 byte array), representing the spatial graph node.
- `poseInNodeSpace` is an XrPosef defining the pose in the underlying node's space.

Valid Usage (Implicit)

- The `XR_MSFT_spatial_graph_bridge` extension **must** be enabled prior to using XrSpatialGraphNodeBindingPropertiesMSFT
- `type` **must** be `XR_TYPE_SPATIAL_GRAPH_NODE_BINDING_PROPERTIES_MSFT`
- `next` **must** be `NULL` or a valid pointer to the next structure in a structure chain

New Object Types

- XrSpatialGraphNodeBindingMSFT

New Flag Types

New Enum Constants

XrObjectType enumeration is extended with:

- `XR_OBJECT_TYPE_SPATIAL_GRAPH_NODE_BINDING_MSFT`

XrStructureType enumeration is extended with:

- `XR_TYPE_SPATIAL_GRAPH_NODE_SPACE_CREATE_INFO_MSFT`
- `XR_TYPE_SPATIAL_GRAPH_STATIC_NODE_BINDING_CREATE_INFO_MSFT`
- `XR_TYPE_SPATIAL_GRAPH_NODE_BINDING_PROPERTIES_GET_INFO_MSFT`
- `XR_TYPE_SPATIAL_GRAPH_NODE_BINDING_PROPERTIES_MSFT`

New Enums

- XrSpatialGraphNodeTypeMSFT

New Structures

- XrSpatialGraphNodeSpaceCreateInfoMSFT
- XrSpatialGraphStaticNodeBindingCreateInfoMSFT
- XrSpatialGraphNodeBindingPropertiesGetInfoMSFT
- XrSpatialGraphNodeBindingPropertiesMSFT

New Functions

- xrTryCreateSpatialGraphStaticNodeBindingMSFT
- xrDestroySpatialGraphNodeBindingMSFT
- xrGetSpatialGraphNodeBindingPropertiesMSFT

Issues

Version History

- Revision 1, 2019-10-31 (Yin LI)
  - Initial extension description
- Revision 2, 2022-01-13 (Darryl Gough)
  - Added Spatial Graph Node Binding handle.

14.38. XR_MSFT_unbounded_reference_space

Name String

`XR_MSFT_unbounded_reference_space`

Extension Type

Instance extension

Registered Extension Number

39

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Deprecation State

- *Deprecated* without replacement

Overview

This extension allows an application to create an `UNBOUNDED_MSFT` reference space. This reference space enables the viewer to move freely through a complex environment, often many meters from where they started, while always optimizing for coordinate system stability near the viewer. This is done by allowing the origin of the reference space to drift as necessary to keep the viewer's coordinates relative to the space's origin stable.

To create an `UNBOUNDED_MSFT` reference space, the application **can** pass `XR_REFERENCE_SPACE_TYPE_UNBOUNDED_MSFT` to xrCreateReferenceSpace.

The `UNBOUNDED_MSFT` reference space establishes a world-locked origin, gravity-aligned to exclude pitch and roll, with +Y up, +X to the right, and -Z forward. This space begins with an arbitrary initial position

and orientation, which the runtime **may** define to be either the initial position at app launch or some other initial zero position. Unlike a `STAGE` reference space, the runtime **may** place the origin of an `UNBOUNDED_MSFT` reference space at any height, rather than fixing it at the floor. This is because the viewer may move through various rooms and levels of their environment, each of which has a different floor height. Runtimes **should** not automatically adjust the position of the origin when the viewer moves to a room with a different floor height.

`UNBOUNDED_MSFT` space is useful when an app needs to render **world-scale** content that spans beyond the bounds of a single `STAGE`, for example, an entire floor or multiple floors of a building.

An `UNBOUNDED_MSFT` space maintains stability near the viewer by slightly adjusting its origin over time. The runtime **must** not queue the XrEventDataReferenceSpaceChangePending event in response to these minor adjustments.

When views, controllers or other spaces experience tracking loss relative to the `UNBOUNDED_MSFT` space, runtimes **should** continue to provide inferred or last-known `position` and `orientation` values. These inferred poses can, for example, be based on neck model updates, inertial dead reckoning, or a last-known position, so long as it is still reasonable for the application to use that pose. While a runtime is providing position data, it **must** continue to set `XR_SPACE_LOCATION_POSITION_VALID_BIT` and `XR_VIEW_STATE_POSITION_VALID_BIT` but it **can** clear `XR_SPACE_LOCATION_POSITION_TRACKED_BIT` and `XR_VIEW_STATE_POSITION_TRACKED_BIT` to indicate that the position is inferred or last-known in this way.

When tracking is recovered, runtimes **should** snap the pose of other spaces back into position relative to the `UNBOUNDED_MSFT` space's original origin. However, if tracking recovers into a new tracking volume in which the original origin can no longer be located (e.g. the viewer moved through a dark hallway and regained tracking in a new room), the runtime **may** recenter the origin arbitrarily, for example moving the origin to coincide with the viewer. If such recentering occurs, the runtime **must** queue the XrEventDataReferenceSpaceChangePending event with `poseValid` set to false.

If the viewer moves far enough away from the origin of an `UNBOUNDED_MSFT` reference space that floating point error would introduce noticeable error when locating the viewer within that space, the runtime **may** recenter the space's origin to a new location closer to the viewer. If such recentering occurs, the runtime **must** queue the XrEventDataReferenceSpaceChangePending event with `poseValid` set to true.

Runtimes **must** support the `UNBOUNDED_MSFT` reference space when this extension is enabled.

New Object Types

New Flag Types

New Enum Constants

XrReferenceSpaceType enumeration is extended with:

- `XR_REFERENCE_SPACE_TYPE_UNBOUNDED_MSFT`

New Enums

New Structures

New Functions

Issues

Version History

- Revision 1, 2019-07-30 (Alex Turner)
  - Initial extension description

14.39. XR_OCULUS_android_session_state_enable

Name String

`XR_OCULUS_android_session_state_enable`

Extension Type

Instance extension

Registered Extension Number

45

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Deprecation State

- *Deprecated* without replacement

Overview

This extension enables the integration of the Android session lifecycle and an OpenXR runtime session state. Some OpenXR runtimes may require this extension to transition the application to the session READY or STOPPING state.

Applications that run on an Android system with this extension enabled have a different OpenXR Session state flow.

On Android, it is the Android Activity lifecycle that will dictate when the system is ready for the application to begin or end its session, not the runtime.

When XR_OCULUS_android_session_state is enabled, the following changes are made to Session State handling:

- The runtime does not determine when the application's session should be moved to the ready state, XR_SESSION_STATE_READY. The application should not wait to receive the XR_SESSION_STATE_READY session state changed event before beginning a session. Instead, the application should begin their session once there is a surface and the activity is resumed.
- The application should not call xrRequestExitSession to request the session move to the stopping state, XR_SESSION_STATE_STOPPING. xrRequestExitSession will return XR_ERROR_VALIDATION_FAILURE if called.
- The application should not wait to receive the XR_SESSION_STATE_STOPPING session state changed event before ending a session. Instead, the application should end its session once the surface is destroyed or the activity is paused.
- The runtime will not transition to XR_SESSION_STATE_READY or XR_SESSION_STATE_STOPPING as the state is implicit from the Android activity and surface lifecycles.

Android Activity life cycle

An Android Activity can only be in the session running state while the activity is in the resumed state. The following shows how beginning and ending an XR session fits into the Android Activity life cycle.

```
1.  VrActivity::onCreate() <---------+
2.  VrActivity::onStart() <-------+  |
3.  VrActivity::onResume() <---+  |  |
4.  xrBeginSession()           |  |  |
5.  xrEndSession()             |  |  |
6.  VrActivity::onPause() -----+  |  |
7.  VrActivity::onStop() ---------+  |
8.  VrActivity::onDestroy() ---------+
```

Android Surface life cycle

An Android Activity can only be in the session running state while there is a valid Android Surface. The following shows how beginning and ending an XR session fits into the Android Surface life cycle.

```
1.  VrActivity::surfaceCreated() <----+
2.  VrActivity::surfaceChanged()      |
3.  xrBeginSession()                  |
4.  xrEndSession()                    |
5.  VrActivity::surfaceDestroyed() ---+
```

Note that the life cycle of a surface is not necessarily tightly coupled with the life cycle of an activity. These two life cycles may interleave in complex ways. Usually surfaceCreated() is called after

onResume() and surfaceDestroyed() is called between onPause() and onDestroy(). However, this is not guaranteed and, for instance, surfaceDestroyed() may be called after onDestroy() or even before onPause().

An Android Activity is only in the resumed state with a valid Android Surface between surfaceChanged() or onResume(), whichever comes last, and surfaceDestroyed() or onPause(), whichever comes first. In other words, a XR application will typically begin the session from surfaceChanged() or onResume(), whichever comes last, and end the session from surfaceDestroyed() or onPause(), whichever comes first.

New Object Types

New Flag Types

New Enum Constants

New Enums

New Structures

New Functions

Issues

Version History

- Revision 1, 2019-08-16 (Cass Everitt)
  - Initial extension description

14.40. XR_QCOM_hand_tracking_gesture

Name String

`XR_QCOM_hand_tracking_gesture`

Extension Type

Instance extension

Registered Extension Number

311

Revision

1

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0
and
XR_EXT_hand_tracking

Deprecation State

- *Deprecated* by XR_EXT_hand_interaction extension

Last Modified Date

2025-09-17

IP Status

No known IP claims.

Contributors

Laurent Gaidon, Qualcomm
Ophir Paz, Qualcomm
Kurtis Eveleigh, Qualcomm

14.40.1. Overview

This extension enables applications to detect hand gestures.

14.40.2. Get gesture detection data

The xrGetHandGestureQCOM function is defined as:

```
// Provided by XR_QCOM_hand_tracking_gesture
XrResult xrGetHandGestureQCOM(
    XrHandTrackerEXT                            handTracker,
    XrTime                                      time,
    XrHandGestureQCOM*                          handGesture);
```

Parameter Descriptions

- handTracker is an XrHandTrackerEXT previously created by xrCreateHandTrackerEXT.
- time is the XrTime of gesture detection.
- handGesture is a pointer to XrHandGestureQCOM and returns the current gesture detection data.

The xrGetHandGestureQCOM function retrieves the hand gesture data at a given time.

Valid Usage (Implicit)

- The XR_QCOM_hand_tracking_gesture extension **must** be enabled prior to calling xrGetHandGestureQCOM
- handTracker **must** be a valid XrHandTrackerEXT handle
- handGesture **must** be a pointer to an XrHandGestureQCOM structure

Return Codes

Success

- XR_SUCCESS
- XR_SESSION_LOSS_PENDING

Failure

- XR_ERROR_FUNCTION_UNSUPPORTED
- XR_ERROR_HANDLE_INVALID
- XR_ERROR_INSTANCE_LOST
- XR_ERROR_RUNTIME_FAILURE
- XR_ERROR_SESSION_LOST
- XR_ERROR_TIME_INVALID

The XrHandGestureQCOM structure is defined as:

```
// Provided by XR_QCOM_hand_tracking_gesture
typedef struct XrHandGestureQCOM {
    XrHandGestureTypeQCOM    gesture;
    float                    gestureRatio;
    float                    flipRatio;
} XrHandGestureQCOM;
```

Member Descriptions

- `gesture` is the XrHandGestureTypeQCOM detected.
- `gestureRatio` is a value between 0 and 1 indicating how much the gesture is applied.
- `flipRatio` is a value between -1 and 1 indicating if the hand gesture is detected from the back, from the front, or in between. The value is 1 when the palm is facing the user and is -1 when the palm is directed toward the environment. It represents the normalized dot product between the palm normal and the palm-to-user vector.

XrHandGestureQCOM structure returns the state of the gesture detection.

Valid Usage (Implicit)

- The `XR_QCOM_hand_tracking_gesture` extension **must** be enabled prior to using XrHandGestureQCOM
- `gesture` **must** be a valid XrHandGestureTypeQCOM value

The XrHandGestureTypeQCOM enumeration identifies the different types of gestures.

```
// Provided by XR_QCOM_hand_tracking_gesture
typedef enum XrHandGestureTypeQCOM {
    XR_HAND_GESTURE_TYPE_UNKNOWN_QCOM = -1,
    XR_HAND_GESTURE_TYPE_OPEN_HAND_QCOM = 0,
    XR_HAND_GESTURE_TYPE_GRAB_QCOM = 2,
    XR_HAND_GESTURE_TYPE_PINCH_QCOM = 7,
    XR_HAND_GESTURE_TYPE_MAX_ENUM_QCOM = 0x7FFFFFFF
} XrHandGestureTypeQCOM;
```

Enum	Description
XR_HAND_GESTURE_TYPE_UNKNOWN_QCOM	Indicates that no gesture can be detected.
XR_HAND_GESTURE_TYPE_OPEN_HAND_QCOM	The open hand gesture is a neutral position. It is performed by extending the hand with the fingers opened and the palm away from the user.
XR_HAND_GESTURE_TYPE_GRAB_QCOM	The grab gesture is performed by placing the hand in front of the user and closing the hand into a fist.
XR_HAND_GESTURE_TYPE_PINCH_QCOM	The pinch gesture is performed by touching the tip of the thumb to the tip of the index finger while extending the remaining fingers.

14.40.3. New Commands

- xrGetHandGestureQCOM

14.40.4. New Structures

- XrHandGestureQCOM

14.40.5. New Enums

- XrHandGestureTypeQCOM

14.40.6. New Enum Constants

- `XR_QCOM_HAND_TRACKING_GESTURE_EXTENSION_NAME`
- `XR_QCOM_hand_tracking_gesture_SPEC_VERSION`

14.40.7. Issues

14.40.8. Version History

- Revision 1, 2022-08-08 (Laurent Gaidon)
  - Initial extension description

14.41. XR_VARJO_quad_views

Name String

`XR_VARJO_quad_views`

Extension Type

Instance extension

Registered Extension Number

38

Revision

2

Ratification Status

Not ratified

Extension and Version Dependencies

OpenXR 1.0

Deprecation State

- *Promoted* to OpenXR 1.1

Last Modified Date

2019-04-16

IP Status

No known IP claims.

Contributors

Sergiy Dubovik, Varjo Technologies
Rémi Arnaud, Varjo Technologies
Robert Menzel, NVIDIA

14.41.1. Overview

This extension adds a new view configuration type - `XR_VIEW_CONFIGURATION_TYPE_PRIMARY_QUAD_VARJO` to XrViewConfigurationType which can be returned by xrEnumerateViewConfigurations to indicate that the runtime supports 4 viewports.

In this configuration each eye consists of two viewports of which one is smaller (in terms of field of view) of the other and fully included inside of the larger FoV one. The small FoV viewport however can have a higher resolution with respect to the same field of view in the outer viewport. The motivation is special hardware which superimposes a smaller, high resolution screen for the fovea region onto a larger screen for the periphery.

The runtime guarantees that the inner viewport of each eye is fully inside of the outer viewport.

To enumerate the 4 views xrEnumerateViewConfigurationViews can be used. The first two views (XrViewConfigurationView) will be for the left and right eyes for the outer viewport. The views 2 and 3 are for the left and right eyes for the inner viewport.

The relative position of the inner views relative to the outer views can change at run-time.

The runtime **must** set `pose` for view 0 and 2 to be identical and **must** set `pose` for view 1 and 3 to be be identical when application calls xrLocateViews.

The runtime might blend between the views at the edges, so the application should not omit the inner field of view from being generated in the outer view.

New Object Types

New Flag Types

New Enum Constants

XrViewConfigurationType enumeration is extended with:

- `XR_VIEW_CONFIGURATION_TYPE_PRIMARY_QUAD_VARJO`

New Enums

New Structures

New Functions

Issues

Version History

- Revision 1, 2019-04-16 (Sergiy Dubovik)
  - Initial draft
- Revision 2, 2024-06-13 (Denny Rönngren)
  - Clarified that both views for each eye needs to have identical poses. This reflects the actual behavior in all known implementations.

Chapter 15. Core Revisions (Informative)

New minor versions of the OpenXR API are defined periodically by the Khronos OpenXR Working Group. These consist of some amount of additional functionality added to the core API, potentially including both new functionality and functionality promoted from extensions.

15.1. Version 1.1

15.1.1. OpenXR 1.1 Promotions

OpenXR version 1.1 promoted a number of key extensions into the core API:

- `XR_KHR_locate_spaces`
- `XR_KHR_maintenance1`
- `XR_EXT_hp_mixed_reality_controller`
- `XR_EXT_local_floor`
- `XR_EXT_palm_pose`
- `XR_EXT_samsung_odyssey_controller`
- `XR_EXT_uuid`
- `XR_BD_controller_interaction`
- `XR_FB_touch_controller_pro`
- `XR_HTC_vive_cosmos_controller_interaction`
- `XR_HTC_vive_focus3_controller_interaction`
- `XR_META_touch_controller_plus`
- `XR_ML_ml2_controller_interaction`
- `XR_VARJO_quad_views`

All differences in behavior between these extensions and the corresponding OpenXR 1.1 functionality are summarized below.

Differences Relative to `XR_EXT_local_floor`

The definition of this space was made more precise, and it was clarified that the mandatory support of this space does **not** dictate any particular quality of floor level estimation. Applications that can provide a head-relative interaction experience in the absence of a defined stage continue to use `LOCAL` space, while those that need higher quality assertions about floor level continue to use `STAGE` space or scene understanding extensions to detect floor level. The (mandatory) presence of this space when

enumerating reference spaces is a convenience for portability rather than an assertion that e.g. floor detection scene understanding has taken place or that the floor is inherently walkable.

Differences Relative to XR_EXT_palm_pose

The input identifier `palm_ext` defined in the extension has been renamed to `grip_surface` to more clearly describe its intended use and distinguish it from hand tracking.

Differences Relative to XR_VARJO_quad_views

The view configuration type enumerant `XR_VIEW_CONFIGURATION_TYPE_PRIMARY_QUAD_VARJO` was renamed to `XR_VIEW_CONFIGURATION_TYPE_PRIMARY_STEREO_WITH_FOVEATED_INSET`, to clarify that it is not vendor-specific nor the only way four views are possible. In OpenXR 1.1, a runtime **may** support `XR_VIEW_CONFIGURATION_TYPE_PRIMARY_STEREO_WITH_FOVEATED_INSET`, but this is **optional** like the other view configuration types. Use xrEnumerateViewConfigurations to determine if it is provided, rather than using the presence or absence of the extension.

Differences Relative to XR_FB_touch_controller_pro

The interaction profile path was changed from */interaction_profiles/facebook/touch_controller_pro* to */interaction_profiles/meta/touch_pro_controller*. Note the updated company name and different word order in the device name level.

The following input/output subpaths were renamed when changing to this new interaction profile path:

- *.../input/stylus_fb/force* → *.../input/stylus/force*
- *.../input/trigger/proximity_fb* → *.../input/trigger/proximity*
- *.../output/haptic_trigger_fb* → *.../output/haptic_trigger*
- *.../output/haptic_thumb_fb* → *.../output/haptic_thumb*
- *.../input/thumb_fb/proximity_fb* → *.../input/thumb_resting_surfaces/proximity*
  - Note this is a boolean-OR of the "thumbrest" location's proximity sensor as well as other proximity-sensitive regions.
- *.../input/trigger/curl_fb* → *.../input/trigger_curl/value*
- *.../input/trigger/slide_fb* → *.../input/trigger_slide/value*

The last two changes listed moved from being components on the trigger identifier to being independent identifiers in order to clarify how they relate to actions bound to other trigger components with regards to action priority.

Differences Relative to XR_META_touch_controller_plus

The interaction profile path was changed from */interaction_profiles/meta/touch_controller_plus* to */interaction_profiles/meta/touch_plus_controller*. Note the different word order in the device name

level.

The following input subpaths were renamed when changing to this new interaction profile path:

- *.../input/trigger/proximity_meta → .../input/trigger/proximity*
- *.../input/thumb_meta/proximity_meta → .../input/thumb_resting_surfaces/proximity*
  - Note this is a boolean-OR of the "thumbrest" location's proximity sensor as well as other proximity-sensitive regions.
- *.../input/trigger/curl_meta → .../input/trigger_curl/value*
- *.../input/trigger/slide_meta → .../input/trigger_slide/value*

15.1.2. Additional OpenXR 1.1 Changes

In addition to the promoted extensions described above, OpenXR 1.1 changed the following:

- Substantial clarifications in the input and fundamentals chapters, intended to be non-substantive.
- Added the following legacy interaction profiles to represent specific controllers shipped under the Oculus/Meta Touch name and previously grouped into a single Oculus Touch interaction profile:
  - */interaction_profiles/meta/touch_controller_rift_cv1* - Meta Touch Controller (Rift CV1) Profile
  - */interaction_profiles/meta/touch_controller_quest_1_rift_s* - Meta Touch Controller (Rift S / Quest 1) Profile
  - */interaction_profiles/meta/touch_controller_quest_2* - Meta Touch Controller (Quest 2) Profile

15.1.3. New Macros

- XR_API_VERSION_1_1

15.1.4. New Commands

- xrLocateSpaces

15.1.5. New Structures

- XrBoxf
- XrColor3f
- XrExtent3Df
- XrFrustumf
- XrSpaceLocationData
- XrSpaceLocations
- XrSpaceVelocityData

- XrSpacesLocateInfo
- XrSpheref
- XrUuid
- Extending XrSpaceLocations:
  - XrSpaceVelocities

15.1.6. New Enum Constants

- XR_UUID_SIZE
- Extending XrReferenceSpaceType:
  - XR_REFERENCE_SPACE_TYPE_LOCAL_FLOOR
- Extending XrResult:
  - XR_ERROR_EXTENSION_DEPENDENCY_NOT_ENABLED
  - XR_ERROR_PERMISSION_INSUFFICIENT
- Extending XrStructureType:
  - XR_TYPE_SPACES_LOCATE_INFO
  - XR_TYPE_SPACE_LOCATIONS
  - XR_TYPE_SPACE_VELOCITIES
- Extending XrViewConfigurationType:
  - XR_VIEW_CONFIGURATION_TYPE_PRIMARY_STEREO_WITH_FOVEATED_INSET

15.2. Loader Runtime and API Layer Negotiation Version 1.0

The OpenXR version 1.0.33 patch release included ratification of the runtime and API layer negotiation API, associated with the identifier XR_LOADER_VERSION_1_0, substantially unchanged from the unratified form previously described in the loader design document. This interface is intended for use only between the loader, runtimes, and API layers, and is not typically directly used by an application.

15.2.1. New Macros

- XR_API_LAYER_CREATE_INFO_STRUCT_VERSION
- XR_API_LAYER_INFO_STRUCT_VERSION
- XR_API_LAYER_MAX_SETTINGS_PATH_SIZE
- XR_API_LAYER_NEXT_INFO_STRUCT_VERSION
- XR_CURRENT_LOADER_API_LAYER_VERSION

- XR_CURRENT_LOADER_RUNTIME_VERSION
- XR_LOADER_INFO_STRUCT_VERSION
- XR_RUNTIME_INFO_STRUCT_VERSION

15.2.2. New Commands

- xrCreateApiLayerInstance
- xrNegotiateLoaderApiLayerInterface
- xrNegotiateLoaderRuntimeInterface

15.2.3. New Structures

- XrApiLayerCreateInfo
- XrApiLayerNextInfo
- XrNegotiateApiLayerRequest
- XrNegotiateLoaderInfo
- XrNegotiateRuntimeRequest

15.2.4. New Function Pointers

- PFN_xrCreateApiLayerInstance
- PFN_xrGetInstanceProcAddr

15.2.5. New Enums

- XrLoaderInterfaceStructs

15.3. Version 1.0

OpenXR version 1.0 defined the initial core API.

15.3.1. New Macros

- XR_API_VERSION_1_0
- XR_CURRENT_API_VERSION
- XR_DEFINE_ATOM
- XR_DEFINE_HANDLE
- XR_DEFINE_OPAQUE_64
- XR_EXTENSION_ENUM_BASE

- XR_EXTENSION_ENUM_STRIDE
- XR_FAILED
- XR_FREQUENCY_UNSPECIFIED
- XR_INFINITE_DURATION
- XR_MAKE_VERSION
- XR_MAX_EVENT_DATA_SIZE
- XR_MAY_ALIAS
- XR_MIN_COMPOSITION_LAYERS_SUPPORTED
- XR_MIN_HAPTIC_DURATION
- XR_NO_DURATION
- XR_NULL_HANDLE
- XR_NULL_PATH
- XR_NULL_SYSTEM_ID
- XR_SUCCEEDED
- XR_UNQUALIFIED_SUCCESS
- XR_VERSION_MAJOR
- XR_VERSION_MINOR
- XR_VERSION_PATCH

15.3.2. New Base Types

- `XrBool32`
- `XrDuration`
- `XrFlags64`
- `XrPath`
- `XrSystemId`
- `XrTime`
- `XrVersion`

15.3.3. New Object Types

- XrAction
- XrActionSet
- XrInstance
- XrSession

- XrSpace
- XrSwapchain

15.3.4. New Commands

- xrAcquireSwapchainImage
- xrApplyHapticFeedback
- xrAttachSessionActionSets
- xrBeginFrame
- xrBeginSession
- xrCreateAction
- xrCreateActionSet
- xrCreateActionSpace
- xrCreateInstance
- xrCreateReferenceSpace
- xrCreateSession
- xrCreateSwapchain
- xrDestroyAction
- xrDestroyActionSet
- xrDestroyInstance
- xrDestroySession
- xrDestroySpace
- xrDestroySwapchain
- xrEndFrame
- xrEndSession
- xrEnumerateApiLayerProperties
- xrEnumerateBoundSourcesForAction
- xrEnumerateEnvironmentBlendModes
- xrEnumerateInstanceExtensionProperties
- xrEnumerateReferenceSpaces
- xrEnumerateSwapchainFormats
- xrEnumerateSwapchainImages
- xrEnumerateViewConfigurationViews
- xrEnumerateViewConfigurations

- xrGetActionStateBoolean
- xrGetActionStateFloat
- xrGetActionStatePose
- xrGetActionStateVector2f
- xrGetCurrentInteractionProfile
- xrGetInputSourceLocalizedName
- xrGetInstanceProcAddr
- xrGetInstanceProperties
- xrGetReferenceSpaceBoundsRect
- xrGetSystem
- xrGetSystemProperties
- xrGetViewConfigurationProperties
- xrLocateSpace
- xrLocateViews
- xrPathToString
- xrPollEvent
- xrReleaseSwapchainImage
- xrRequestExitSession
- xrResultToString
- xrStopHapticFeedback
- xrStringToPath
- xrStructureTypeToString
- xrSuggestInteractionProfileBindings
- xrSyncActions
- xrWaitFrame
- xrWaitSwapchainImage

15.3.5. New Structures

- XrActionCreateInfo
- XrActionSetCreateInfo
- XrActionSpaceCreateInfo
- XrActionStateBoolean
- XrActionStateFloat

- XrActionStateGetInfo
- XrActionStatePose
- XrActionStateVector2f
- XrActionSuggestedBinding
- XrActionsSyncInfo
- XrActiveActionSet
- XrApiLayerProperties
- XrApplicationInfo
- XrBaseInStructure
- XrBaseOutStructure
- XrBoundSourcesForActionEnumerateInfo
- XrColor4f
- XrCompositionLayerBaseHeader
- XrCompositionLayerProjection
- XrCompositionLayerProjectionView
- XrCompositionLayerQuad
- XrEventDataBaseHeader
- XrEventDataBuffer
- XrEventDataEventsLost
- XrEventDataInstanceLossPending
- XrEventDataInteractionProfileChanged
- XrEventDataReferenceSpaceChangePending
- XrEventDataSessionStateChanged
- XrExtensionProperties
- XrExtent2Df
- XrExtent2Di
- XrFovf
- XrFrameBeginInfo
- XrFrameEndInfo
- XrFrameState
- XrFrameWaitInfo
- XrHapticActionInfo

- XrHapticBaseHeader
- XrHapticVibration
- XrInputSourceLocalizedNameGetInfo
- XrInstanceCreateInfo
- XrInstanceProperties
- XrInteractionProfileState
- XrInteractionProfileSuggestedBinding
- XrOffset2Df
- XrOffset2Di
- XrPosef
- XrQuaternionf
- XrRect2Df
- XrRect2Di
- XrReferenceSpaceCreateInfo
- XrSessionActionSetsAttachInfo
- XrSessionBeginInfo
- XrSessionCreateInfo
- XrSpaceLocation
- XrSwapchainCreateInfo
- XrSwapchainImageAcquireInfo
- XrSwapchainImageBaseHeader
- XrSwapchainImageReleaseInfo
- XrSwapchainImageWaitInfo
- XrSwapchainSubImage
- XrSystemGetInfo
- XrSystemGraphicsProperties
- XrSystemProperties
- XrSystemTrackingProperties
- XrVector2f
- XrVector3f
- XrVector4f
- XrView

- XrViewConfigurationProperties
- XrViewConfigurationView
- XrViewLocateInfo
- XrViewState
- Extending XrSpaceLocation:
  - XrSpaceVelocity

15.3.6. New Function Pointers

- PFN_xrVoidFunction

15.3.7. New Enums

- XrActionType
- XrCompositionLayerFlagBits
- XrEnvironmentBlendMode
- XrEyeVisibility
- XrFormFactor
- XrInputSourceLocalizedNameFlagBits
- XrInstanceCreateFlagBits
- XrObjectType
- XrReferenceSpaceType
- XrResult
- XrSessionCreateFlagBits
- XrSessionState
- XrSpaceLocationFlagBits
- XrSpaceVelocityFlagBits
- XrStructureType
- XrSwapchainCreateFlagBits
- XrSwapchainUsageFlagBits
- XrViewConfigurationType
- XrViewStateFlagBits

15.3.8. New Bitmasks

- XrCompositionLayerFlags

- XrInputSourceLocalizedNameFlags
- XrInstanceCreateFlags
- XrSessionCreateFlags
- XrSpaceLocationFlags
- XrSpaceVelocityFlags
- XrSwapchainCreateFlags
- XrSwapchainUsageFlags
- XrViewStateFlags

15.3.9. New Headers

- `openxr_platform_defines`

15.3.10. New Enum Constants

- `XR_FALSE`
- `XR_MAX_API_LAYER_DESCRIPTION_SIZE`
- `XR_MAX_API_LAYER_NAME_SIZE`
- `XR_MAX_APPLICATION_NAME_SIZE`
- `XR_MAX_ENGINE_NAME_SIZE`
- `XR_MAX_EXTENSION_NAME_SIZE`
- `XR_MAX_PATH_LENGTH`
- `XR_MAX_RESULT_STRING_SIZE`
- `XR_MAX_RUNTIME_NAME_SIZE`
- `XR_MAX_STRUCTURE_NAME_SIZE`
- `XR_MAX_SYSTEM_NAME_SIZE`
- `XR_TRUE`

Appendix

Code Style Conventions

These are the code style conventions used in this specification to define the API.

Conventions

- Enumerants and defines are all upper case with words separated by an underscore.
- Neither type, function or member names contain underscores.
- Structure members start with a lower case character and each consecutive word starts with a capital.
- A structure that has a pointer to an array includes a structure member named `fooCount` of type `uint32_t` to denote the number of elements in the array of `foo`.
- A structure that has a pointer to an array lists the `fooCount` member first and then the array pointer.
- Unless a negative value has a clearly defined meaning all `fooCount` variables are unsigned.
- Function parameters that are modified are always listed last.

Prefixes are used in the API to denote specific semantic meaning of names, or as a label to avoid name clashes, and are explained here:

Prefix	Description
`XR_`	Enumerants and defines are prefixed with these characters.
`Xr`	Non-function-pointer types are prefixed with these characters.
`xr`	Functions are prefixed with these characters.
`PFN_xr`	Function pointer types are prefixed with these characters.

Application Binary Interface

This section describes additional definitions and conventions that define the application binary interface.

Structure Types

```
// Provided by XR_VERSION_1_0
typedef enum XrStructureType {
    XR_TYPE_UNKNOWN = 0,
    XR_TYPE_API_LAYER_PROPERTIES = 1,
    XR_TYPE_EXTENSION_PROPERTIES = 2,
    XR_TYPE_INSTANCE_CREATE_INFO = 3,
    XR_TYPE_SYSTEM_GET_INFO = 4,
    XR_TYPE_SYSTEM_PROPERTIES = 5,
    XR_TYPE_VIEW_LOCATE_INFO = 6,
    XR_TYPE_VIEW = 7,
    XR_TYPE_SESSION_CREATE_INFO = 8,
    XR_TYPE_SWAPCHAIN_CREATE_INFO = 9,
    XR_TYPE_SESSION_BEGIN_INFO = 10,
    XR_TYPE_VIEW_STATE = 11,
    XR_TYPE_FRAME_END_INFO = 12,
    XR_TYPE_HAPTIC_VIBRATION = 13,
    XR_TYPE_EVENT_DATA_BUFFER = 16,
    XR_TYPE_EVENT_DATA_INSTANCE_LOSS_PENDING = 17,
    XR_TYPE_EVENT_DATA_SESSION_STATE_CHANGED = 18,
    XR_TYPE_ACTION_STATE_BOOLEAN = 23,
    XR_TYPE_ACTION_STATE_FLOAT = 24,
    XR_TYPE_ACTION_STATE_VECTOR2F = 25,
    XR_TYPE_ACTION_STATE_POSE = 27,
    XR_TYPE_ACTION_SET_CREATE_INFO = 28,
    XR_TYPE_ACTION_CREATE_INFO = 29,
    XR_TYPE_INSTANCE_PROPERTIES = 32,
    XR_TYPE_FRAME_WAIT_INFO = 33,
    XR_TYPE_COMPOSITION_LAYER_PROJECTION = 35,
    XR_TYPE_COMPOSITION_LAYER_QUAD = 36,
    XR_TYPE_REFERENCE_SPACE_CREATE_INFO = 37,
    XR_TYPE_ACTION_SPACE_CREATE_INFO = 38,
    XR_TYPE_EVENT_DATA_REFERENCE_SPACE_CHANGE_PENDING = 40,
    XR_TYPE_VIEW_CONFIGURATION_VIEW = 41,
    XR_TYPE_SPACE_LOCATION = 42,
    XR_TYPE_SPACE_VELOCITY = 43,
    XR_TYPE_FRAME_STATE = 44,
    XR_TYPE_VIEW_CONFIGURATION_PROPERTIES = 45,
    XR_TYPE_FRAME_BEGIN_INFO = 46,
    XR_TYPE_COMPOSITION_LAYER_PROJECTION_VIEW = 48,
    XR_TYPE_EVENT_DATA_EVENTS_LOST = 49,
    XR_TYPE_INTERACTION_PROFILE_SUGGESTED_BINDING = 51,
    XR_TYPE_EVENT_DATA_INTERACTION_PROFILE_CHANGED = 52,
    XR_TYPE_INTERACTION_PROFILE_STATE = 53,
    XR_TYPE_SWAPCHAIN_IMAGE_ACQUIRE_INFO = 55,
    XR_TYPE_SWAPCHAIN_IMAGE_WAIT_INFO = 56,
```

```
    XR_TYPE_SWAPCHAIN_IMAGE_RELEASE_INFO = 57,
    XR_TYPE_ACTION_STATE_GET_INFO = 58,
    XR_TYPE_HAPTIC_ACTION_INFO = 59,
    XR_TYPE_SESSION_ACTION_SETS_ATTACH_INFO = 60,
    XR_TYPE_ACTIONS_SYNC_INFO = 61,
    XR_TYPE_BOUND_SOURCES_FOR_ACTION_ENUMERATE_INFO = 62,
    XR_TYPE_INPUT_SOURCE_LOCALIZED_NAME_GET_INFO = 63,
  // Provided by XR_VERSION_1_1
    XR_TYPE_SPACES_LOCATE_INFO = 1000471000,
  // Provided by XR_VERSION_1_1
    XR_TYPE_SPACE_LOCATIONS = 1000471001,
  // Provided by XR_VERSION_1_1
    XR_TYPE_SPACE_VELOCITIES = 1000471002,
  // Provided by XR_KHR_composition_layer_cube
    XR_TYPE_COMPOSITION_LAYER_CUBE_KHR = 1000006000,
  // Provided by XR_KHR_android_create_instance
    XR_TYPE_INSTANCE_CREATE_INFO_ANDROID_KHR = 1000008000,
  // Provided by XR_KHR_composition_layer_depth
    XR_TYPE_COMPOSITION_LAYER_DEPTH_INFO_KHR = 1000010000,
  // Provided by XR_KHR_vulkan_swapchain_format_list
    XR_TYPE_VULKAN_SWAPCHAIN_FORMAT_LIST_CREATE_INFO_KHR = 1000014000,
  // Provided by XR_EXT_performance_settings
    XR_TYPE_EVENT_DATA_PERF_SETTINGS_EXT = 1000015000,
  // Provided by XR_KHR_composition_layer_cylinder
    XR_TYPE_COMPOSITION_LAYER_CYLINDER_KHR = 1000017000,
  // Provided by XR_KHR_composition_layer_equirect
    XR_TYPE_COMPOSITION_LAYER_EQUIRECT_KHR = 1000018000,
  // Provided by XR_EXT_debug_utils
    XR_TYPE_DEBUG_UTILS_OBJECT_NAME_INFO_EXT = 1000019000,
  // Provided by XR_EXT_debug_utils
    XR_TYPE_DEBUG_UTILS_MESSENGER_CALLBACK_DATA_EXT = 1000019001,
  // Provided by XR_EXT_debug_utils
    XR_TYPE_DEBUG_UTILS_MESSENGER_CREATE_INFO_EXT = 1000019002,
  // Provided by XR_EXT_debug_utils
    XR_TYPE_DEBUG_UTILS_LABEL_EXT = 1000019003,
  // Provided by XR_KHR_opengl_enable
    XR_TYPE_GRAPHICS_BINDING_OPENGL_WIN32_KHR = 1000023000,
  // Provided by XR_KHR_opengl_enable
    XR_TYPE_GRAPHICS_BINDING_OPENGL_XLIB_KHR = 1000023001,
  // Provided by XR_KHR_opengl_enable
    XR_TYPE_GRAPHICS_BINDING_OPENGL_XCB_KHR = 1000023002,
  // Provided by XR_KHR_opengl_enable
    XR_TYPE_GRAPHICS_BINDING_OPENGL_WAYLAND_KHR = 1000023003,
  // Provided by XR_KHR_opengl_enable
    XR_TYPE_SWAPCHAIN_IMAGE_OPENGL_KHR = 1000023004,
  // Provided by XR_KHR_opengl_enable
    XR_TYPE_GRAPHICS_REQUIREMENTS_OPENGL_KHR = 1000023005,
  // Provided by XR_KHR_opengl_es_enable
```

```
    XR_TYPE_GRAPHICS_BINDING_OPENGL_ES_ANDROID_KHR = 1000024001,
  // Provided by XR_KHR_opengl_es_enable
    XR_TYPE_SWAPCHAIN_IMAGE_OPENGL_ES_KHR = 1000024002,
  // Provided by XR_KHR_opengl_es_enable
    XR_TYPE_GRAPHICS_REQUIREMENTS_OPENGL_ES_KHR = 1000024003,
  // Provided by XR_KHR_vulkan_enable
    XR_TYPE_GRAPHICS_BINDING_VULKAN_KHR = 1000025000,
  // Provided by XR_KHR_vulkan_enable
    XR_TYPE_SWAPCHAIN_IMAGE_VULKAN_KHR = 1000025001,
  // Provided by XR_KHR_vulkan_enable
    XR_TYPE_GRAPHICS_REQUIREMENTS_VULKAN_KHR = 1000025002,
  // Provided by XR_KHR_D3D11_enable
    XR_TYPE_GRAPHICS_BINDING_D3D11_KHR = 1000027000,
  // Provided by XR_KHR_D3D11_enable
    XR_TYPE_SWAPCHAIN_IMAGE_D3D11_KHR = 1000027001,
  // Provided by XR_KHR_D3D11_enable
    XR_TYPE_GRAPHICS_REQUIREMENTS_D3D11_KHR = 1000027002,
  // Provided by XR_KHR_D3D12_enable
    XR_TYPE_GRAPHICS_BINDING_D3D12_KHR = 1000028000,
  // Provided by XR_KHR_D3D12_enable
    XR_TYPE_SWAPCHAIN_IMAGE_D3D12_KHR = 1000028001,
  // Provided by XR_KHR_D3D12_enable
    XR_TYPE_GRAPHICS_REQUIREMENTS_D3D12_KHR = 1000028002,
  // Provided by XR_KHR_metal_enable
    XR_TYPE_GRAPHICS_BINDING_METAL_KHR = 1000029000,
  // Provided by XR_KHR_metal_enable
    XR_TYPE_SWAPCHAIN_IMAGE_METAL_KHR = 1000029001,
  // Provided by XR_KHR_metal_enable
    XR_TYPE_GRAPHICS_REQUIREMENTS_METAL_KHR = 1000029002,
  // Provided by XR_EXT_eye_gaze_interaction
    XR_TYPE_SYSTEM_EYE_GAZE_INTERACTION_PROPERTIES_EXT = 1000030000,
  // Provided by XR_EXT_eye_gaze_interaction
    XR_TYPE_EYE_GAZE_SAMPLE_TIME_EXT = 1000030001,
  // Provided by XR_KHR_visibility_mask
    XR_TYPE_VISIBILITY_MASK_KHR = 1000031000,
  // Provided by XR_KHR_visibility_mask
    XR_TYPE_EVENT_DATA_VISIBILITY_MASK_CHANGED_KHR = 1000031001,
  // Provided by XR_EXTX_overlay
    XR_TYPE_SESSION_CREATE_INFO_OVERLAY_EXTX = 1000033000,
  // Provided by XR_EXTX_overlay
    XR_TYPE_EVENT_DATA_MAIN_SESSION_VISIBILITY_CHANGED_EXTX = 1000033003,
  // Provided by XR_KHR_composition_layer_color_scale_bias
    XR_TYPE_COMPOSITION_LAYER_COLOR_SCALE_BIAS_KHR = 1000034000,
  // Provided by XR_MSFT_spatial_anchor
    XR_TYPE_SPATIAL_ANCHOR_CREATE_INFO_MSFT = 1000039000,
  // Provided by XR_MSFT_spatial_anchor
    XR_TYPE_SPATIAL_ANCHOR_SPACE_CREATE_INFO_MSFT = 1000039001,
  // Provided by XR_FB_composition_layer_image_layout
```

```
    XR_TYPE_COMPOSITION_LAYER_IMAGE_LAYOUT_FB = 1000040000,
  // Provided by XR_FB_composition_layer_alpha_blend
    XR_TYPE_COMPOSITION_LAYER_ALPHA_BLEND_FB = 1000041001,
  // Provided by XR_EXT_view_configuration_depth_range
    XR_TYPE_VIEW_CONFIGURATION_DEPTH_RANGE_EXT = 1000046000,
  // Provided by XR_MNDX_egl_enable
    XR_TYPE_GRAPHICS_BINDING_EGL_MNDX = 1000048004,
  // Provided by XR_MSFT_spatial_graph_bridge
    XR_TYPE_SPATIAL_GRAPH_NODE_SPACE_CREATE_INFO_MSFT = 1000049000,
  // Provided by XR_MSFT_spatial_graph_bridge
    XR_TYPE_SPATIAL_GRAPH_STATIC_NODE_BINDING_CREATE_INFO_MSFT = 1000049001,
  // Provided by XR_MSFT_spatial_graph_bridge
    XR_TYPE_SPATIAL_GRAPH_NODE_BINDING_PROPERTIES_GET_INFO_MSFT = 1000049002,
  // Provided by XR_MSFT_spatial_graph_bridge
    XR_TYPE_SPATIAL_GRAPH_NODE_BINDING_PROPERTIES_MSFT = 1000049003,
  // Provided by XR_EXT_hand_tracking
    XR_TYPE_SYSTEM_HAND_TRACKING_PROPERTIES_EXT = 1000051000,
  // Provided by XR_EXT_hand_tracking
    XR_TYPE_HAND_TRACKER_CREATE_INFO_EXT = 1000051001,
  // Provided by XR_EXT_hand_tracking
    XR_TYPE_HAND_JOINTS_LOCATE_INFO_EXT = 1000051002,
  // Provided by XR_EXT_hand_tracking
    XR_TYPE_HAND_JOINT_LOCATIONS_EXT = 1000051003,
  // Provided by XR_EXT_hand_tracking
    XR_TYPE_HAND_JOINT_VELOCITIES_EXT = 1000051004,
  // Provided by XR_MSFT_hand_tracking_mesh
    XR_TYPE_SYSTEM_HAND_TRACKING_MESH_PROPERTIES_MSFT = 1000052000,
  // Provided by XR_MSFT_hand_tracking_mesh
    XR_TYPE_HAND_MESH_SPACE_CREATE_INFO_MSFT = 1000052001,
  // Provided by XR_MSFT_hand_tracking_mesh
    XR_TYPE_HAND_MESH_UPDATE_INFO_MSFT = 1000052002,
  // Provided by XR_MSFT_hand_tracking_mesh
    XR_TYPE_HAND_MESH_MSFT = 1000052003,
  // Provided by XR_MSFT_hand_tracking_mesh
    XR_TYPE_HAND_POSE_TYPE_INFO_MSFT = 1000052004,
  // Provided by XR_MSFT_secondary_view_configuration
    XR_TYPE_SECONDARY_VIEW_CONFIGURATION_SESSION_BEGIN_INFO_MSFT = 1000053000,
  // Provided by XR_MSFT_secondary_view_configuration
    XR_TYPE_SECONDARY_VIEW_CONFIGURATION_STATE_MSFT = 1000053001,
  // Provided by XR_MSFT_secondary_view_configuration
    XR_TYPE_SECONDARY_VIEW_CONFIGURATION_FRAME_STATE_MSFT = 1000053002,
  // Provided by XR_MSFT_secondary_view_configuration
    XR_TYPE_SECONDARY_VIEW_CONFIGURATION_FRAME_END_INFO_MSFT = 1000053003,
  // Provided by XR_MSFT_secondary_view_configuration
    XR_TYPE_SECONDARY_VIEW_CONFIGURATION_LAYER_INFO_MSFT = 1000053004,
  // Provided by XR_MSFT_secondary_view_configuration
    XR_TYPE_SECONDARY_VIEW_CONFIGURATION_SWAPCHAIN_CREATE_INFO_MSFT = 1000053005,
  // Provided by XR_MSFT_controller_model
```

```
    XR_TYPE_CONTROLLER_MODEL_KEY_STATE_MSFT = 1000055000,
  // Provided by XR_MSFT_controller_model
    XR_TYPE_CONTROLLER_MODEL_NODE_PROPERTIES_MSFT = 1000055001,
  // Provided by XR_MSFT_controller_model
    XR_TYPE_CONTROLLER_MODEL_PROPERTIES_MSFT = 1000055002,
  // Provided by XR_MSFT_controller_model
    XR_TYPE_CONTROLLER_MODEL_NODE_STATE_MSFT = 1000055003,
  // Provided by XR_MSFT_controller_model
    XR_TYPE_CONTROLLER_MODEL_STATE_MSFT = 1000055004,
  // Provided by XR_EPIC_view_configuration_fov
    XR_TYPE_VIEW_CONFIGURATION_VIEW_FOV_EPIC = 1000059000,
  // Provided by XR_MSFT_holographic_window_attachment
    XR_TYPE_HOLOGRAPHIC_WINDOW_ATTACHMENT_MSFT = 1000063000,
  // Provided by XR_MSFT_composition_layer_reprojection
    XR_TYPE_COMPOSITION_LAYER_REPROJECTION_INFO_MSFT = 1000066000,
  // Provided by XR_MSFT_composition_layer_reprojection
    XR_TYPE_COMPOSITION_LAYER_REPROJECTION_PLANE_OVERRIDE_MSFT = 1000066001,
  // Provided by XR_FB_android_surface_swapchain_create
    XR_TYPE_ANDROID_SURFACE_SWAPCHAIN_CREATE_INFO_FB = 1000070000,
  // Provided by XR_FB_composition_layer_secure_content
    XR_TYPE_COMPOSITION_LAYER_SECURE_CONTENT_FB = 1000072000,
  // Provided by XR_FB_body_tracking
    XR_TYPE_BODY_TRACKER_CREATE_INFO_FB = 1000076001,
  // Provided by XR_FB_body_tracking
    XR_TYPE_BODY_JOINTS_LOCATE_INFO_FB = 1000076002,
  // Provided by XR_FB_body_tracking
    XR_TYPE_SYSTEM_BODY_TRACKING_PROPERTIES_FB = 1000076004,
  // Provided by XR_FB_body_tracking
    XR_TYPE_BODY_JOINT_LOCATIONS_FB = 1000076005,
  // Provided by XR_FB_body_tracking
    XR_TYPE_BODY_SKELETON_FB = 1000076006,
  // Provided by XR_EXT_dpad_binding
    XR_TYPE_INTERACTION_PROFILE_DPAD_BINDING_EXT = 1000078000,
  // Provided by XR_VALVE_analog_threshold
    XR_TYPE_INTERACTION_PROFILE_ANALOG_THRESHOLD_VALVE = 1000079000,
  // Provided by XR_EXT_hand_joints_motion_range
    XR_TYPE_HAND_JOINTS_MOTION_RANGE_INFO_EXT = 1000080000,
  // Provided by XR_KHR_loader_init_android
    XR_TYPE_LOADER_INIT_INFO_ANDROID_KHR = 1000089000,
  // Provided by XR_KHR_vulkan_enable2
    XR_TYPE_VULKAN_INSTANCE_CREATE_INFO_KHR = 1000090000,
  // Provided by XR_KHR_vulkan_enable2
    XR_TYPE_VULKAN_DEVICE_CREATE_INFO_KHR = 1000090001,
  // Provided by XR_KHR_vulkan_enable2
    XR_TYPE_VULKAN_GRAPHICS_DEVICE_GET_INFO_KHR = 1000090003,
  // Provided by XR_KHR_composition_layer_equirect2
    XR_TYPE_COMPOSITION_LAYER_EQUIRECT2_KHR = 1000091000,
  // Provided by XR_MSFT_scene_understanding
```

```
    XR_TYPE_SCENE_OBSERVER_CREATE_INFO_MSFT = 1000097000,
  // Provided by XR_MSFT_scene_understanding
    XR_TYPE_SCENE_CREATE_INFO_MSFT = 1000097001,
  // Provided by XR_MSFT_scene_understanding
    XR_TYPE_NEW_SCENE_COMPUTE_INFO_MSFT = 1000097002,
  // Provided by XR_MSFT_scene_understanding
    XR_TYPE_VISUAL_MESH_COMPUTE_LOD_INFO_MSFT = 1000097003,
  // Provided by XR_MSFT_scene_understanding
    XR_TYPE_SCENE_COMPONENTS_MSFT = 1000097004,
  // Provided by XR_MSFT_scene_understanding
    XR_TYPE_SCENE_COMPONENTS_GET_INFO_MSFT = 1000097005,
  // Provided by XR_MSFT_scene_understanding
    XR_TYPE_SCENE_COMPONENT_LOCATIONS_MSFT = 1000097006,
  // Provided by XR_MSFT_scene_understanding
    XR_TYPE_SCENE_COMPONENTS_LOCATE_INFO_MSFT = 1000097007,
  // Provided by XR_MSFT_scene_understanding
    XR_TYPE_SCENE_OBJECTS_MSFT = 1000097008,
  // Provided by XR_MSFT_scene_understanding
    XR_TYPE_SCENE_COMPONENT_PARENT_FILTER_INFO_MSFT = 1000097009,
  // Provided by XR_MSFT_scene_understanding
    XR_TYPE_SCENE_OBJECT_TYPES_FILTER_INFO_MSFT = 1000097010,
  // Provided by XR_MSFT_scene_understanding
    XR_TYPE_SCENE_PLANES_MSFT = 1000097011,
  // Provided by XR_MSFT_scene_understanding
    XR_TYPE_SCENE_PLANE_ALIGNMENT_FILTER_INFO_MSFT = 1000097012,
  // Provided by XR_MSFT_scene_understanding
    XR_TYPE_SCENE_MESHES_MSFT = 1000097013,
  // Provided by XR_MSFT_scene_understanding
    XR_TYPE_SCENE_MESH_BUFFERS_GET_INFO_MSFT = 1000097014,
  // Provided by XR_MSFT_scene_understanding
    XR_TYPE_SCENE_MESH_BUFFERS_MSFT = 1000097015,
  // Provided by XR_MSFT_scene_understanding
    XR_TYPE_SCENE_MESH_VERTEX_BUFFER_MSFT = 1000097016,
  // Provided by XR_MSFT_scene_understanding
    XR_TYPE_SCENE_MESH_INDICES_UINT32_MSFT = 1000097017,
  // Provided by XR_MSFT_scene_understanding
    XR_TYPE_SCENE_MESH_INDICES_UINT16_MSFT = 1000097018,
  // Provided by XR_MSFT_scene_understanding_serialization
    XR_TYPE_SERIALIZED_SCENE_FRAGMENT_DATA_GET_INFO_MSFT = 1000098000,
  // Provided by XR_MSFT_scene_understanding_serialization
    XR_TYPE_SCENE_DESERIALIZE_INFO_MSFT = 1000098001,
  // Provided by XR_FB_display_refresh_rate
    XR_TYPE_EVENT_DATA_DISPLAY_REFRESH_RATE_CHANGED_FB = 1000101000,
  // Provided by XR_HTCX_vive_tracker_interaction
    XR_TYPE_VIVE_TRACKER_PATHS_HTCX = 1000103000,
  // Provided by XR_HTCX_vive_tracker_interaction
    XR_TYPE_EVENT_DATA_VIVE_TRACKER_CONNECTED_HTCX = 1000103001,
  // Provided by XR_HTC_facial_tracking
```

```
    XR_TYPE_SYSTEM_FACIAL_TRACKING_PROPERTIES_HTC = 1000104000,
  // Provided by XR_HTC_facial_tracking
    XR_TYPE_FACIAL_TRACKER_CREATE_INFO_HTC = 1000104001,
  // Provided by XR_HTC_facial_tracking
    XR_TYPE_FACIAL_EXPRESSIONS_HTC = 1000104002,
  // Provided by XR_FB_color_space
    XR_TYPE_SYSTEM_COLOR_SPACE_PROPERTIES_FB = 1000108000,
  // Provided by XR_FB_hand_tracking_mesh
    XR_TYPE_HAND_TRACKING_MESH_FB = 1000110001,
  // Provided by XR_FB_hand_tracking_mesh
    XR_TYPE_HAND_TRACKING_SCALE_FB = 1000110003,
  // Provided by XR_FB_hand_tracking_aim
    XR_TYPE_HAND_TRACKING_AIM_STATE_FB = 1000111001,
  // Provided by XR_FB_hand_tracking_capsules
    XR_TYPE_HAND_TRACKING_CAPSULES_STATE_FB = 1000112000,
  // Provided by XR_FB_spatial_entity
    XR_TYPE_SYSTEM_SPATIAL_ENTITY_PROPERTIES_FB = 1000113004,
  // Provided by XR_FB_spatial_entity
    XR_TYPE_SPATIAL_ANCHOR_CREATE_INFO_FB = 1000113003,
  // Provided by XR_FB_spatial_entity
    XR_TYPE_SPACE_COMPONENT_STATUS_SET_INFO_FB = 1000113007,
  // Provided by XR_FB_spatial_entity
    XR_TYPE_SPACE_COMPONENT_STATUS_FB = 1000113001,
  // Provided by XR_FB_spatial_entity
    XR_TYPE_EVENT_DATA_SPATIAL_ANCHOR_CREATE_COMPLETE_FB = 1000113005,
  // Provided by XR_FB_spatial_entity
    XR_TYPE_EVENT_DATA_SPACE_SET_STATUS_COMPLETE_FB = 1000113006,
  // Provided by XR_FB_foveation
    XR_TYPE_FOVEATION_PROFILE_CREATE_INFO_FB = 1000114000,
  // Provided by XR_FB_foveation
    XR_TYPE_SWAPCHAIN_CREATE_INFO_FOVEATION_FB = 1000114001,
  // Provided by XR_FB_foveation
    XR_TYPE_SWAPCHAIN_STATE_FOVEATION_FB = 1000114002,
  // Provided by XR_FB_foveation_configuration
    XR_TYPE_FOVEATION_LEVEL_PROFILE_CREATE_INFO_FB = 1000115000,
  // Provided by XR_FB_keyboard_tracking
    XR_TYPE_KEYBOARD_SPACE_CREATE_INFO_FB = 1000116009,
  // Provided by XR_FB_keyboard_tracking
    XR_TYPE_KEYBOARD_TRACKING_QUERY_FB = 1000116004,
  // Provided by XR_FB_keyboard_tracking
    XR_TYPE_SYSTEM_KEYBOARD_TRACKING_PROPERTIES_FB = 1000116002,
  // Provided by XR_FB_triangle_mesh
    XR_TYPE_TRIANGLE_MESH_CREATE_INFO_FB = 1000117001,
  // Provided by XR_FB_passthrough
    XR_TYPE_SYSTEM_PASSTHROUGH_PROPERTIES_FB = 1000118000,
  // Provided by XR_FB_passthrough
    XR_TYPE_PASSTHROUGH_CREATE_INFO_FB = 1000118001,
  // Provided by XR_FB_passthrough
```

```
    XR_TYPE_PASSTHROUGH_LAYER_CREATE_INFO_FB = 1000118002,
  // Provided by XR_FB_passthrough
    XR_TYPE_COMPOSITION_LAYER_PASSTHROUGH_FB = 1000118003,
  // Provided by XR_FB_passthrough
    XR_TYPE_GEOMETRY_INSTANCE_CREATE_INFO_FB = 1000118004,
  // Provided by XR_FB_passthrough
    XR_TYPE_GEOMETRY_INSTANCE_TRANSFORM_FB = 1000118005,
  // Provided by XR_FB_passthrough
    XR_TYPE_SYSTEM_PASSTHROUGH_PROPERTIES2_FB = 1000118006,
  // Provided by XR_FB_passthrough
    XR_TYPE_PASSTHROUGH_STYLE_FB = 1000118020,
  // Provided by XR_FB_passthrough
    XR_TYPE_PASSTHROUGH_COLOR_MAP_MONO_TO_RGBA_FB = 1000118021,
  // Provided by XR_FB_passthrough
    XR_TYPE_PASSTHROUGH_COLOR_MAP_MONO_TO_MONO_FB = 1000118022,
  // Provided by XR_FB_passthrough
    XR_TYPE_PASSTHROUGH_BRIGHTNESS_CONTRAST_SATURATION_FB = 1000118023,
  // Provided by XR_FB_passthrough
    XR_TYPE_EVENT_DATA_PASSTHROUGH_STATE_CHANGED_FB = 1000118030,
  // Provided by XR_FB_render_model
    XR_TYPE_RENDER_MODEL_PATH_INFO_FB = 1000119000,
  // Provided by XR_FB_render_model
    XR_TYPE_RENDER_MODEL_PROPERTIES_FB = 1000119001,
  // Provided by XR_FB_render_model
    XR_TYPE_RENDER_MODEL_BUFFER_FB = 1000119002,
  // Provided by XR_FB_render_model
    XR_TYPE_RENDER_MODEL_LOAD_INFO_FB = 1000119003,
  // Provided by XR_FB_render_model
    XR_TYPE_SYSTEM_RENDER_MODEL_PROPERTIES_FB = 1000119004,
  // Provided by XR_FB_render_model
    XR_TYPE_RENDER_MODEL_CAPABILITIES_REQUEST_FB = 1000119005,
  // Provided by XR_KHR_binding_modification
    XR_TYPE_BINDING_MODIFICATIONS_KHR = 1000120000,
  // Provided by XR_VARJO_foveated_rendering
    XR_TYPE_VIEW_LOCATE_FOVEATED_RENDERING_VARJO = 1000121000,
  // Provided by XR_VARJO_foveated_rendering
    XR_TYPE_FOVEATED_VIEW_CONFIGURATION_VIEW_VARJO = 1000121001,
  // Provided by XR_VARJO_foveated_rendering
    XR_TYPE_SYSTEM_FOVEATED_RENDERING_PROPERTIES_VARJO = 1000121002,
  // Provided by XR_VARJO_composition_layer_depth_test
    XR_TYPE_COMPOSITION_LAYER_DEPTH_TEST_VARJO = 1000122000,
  // Provided by XR_VARJO_marker_tracking
    XR_TYPE_SYSTEM_MARKER_TRACKING_PROPERTIES_VARJO = 1000124000,
  // Provided by XR_VARJO_marker_tracking
    XR_TYPE_EVENT_DATA_MARKER_TRACKING_UPDATE_VARJO = 1000124001,
  // Provided by XR_VARJO_marker_tracking
    XR_TYPE_MARKER_SPACE_CREATE_INFO_VARJO = 1000124002,
  // Provided by XR_ML_frame_end_info
```

```
    XR_TYPE_FRAME_END_INFO_ML = 1000135000,
  // Provided by XR_ML_global_dimmer
    XR_TYPE_GLOBAL_DIMMER_FRAME_END_INFO_ML = 1000136000,
  // Provided by XR_ML_compat
    XR_TYPE_COORDINATE_SPACE_CREATE_INFO_ML = 1000137000,
  // Provided by XR_ML_marker_understanding
    XR_TYPE_SYSTEM_MARKER_UNDERSTANDING_PROPERTIES_ML = 1000138000,
  // Provided by XR_ML_marker_understanding
    XR_TYPE_MARKER_DETECTOR_CREATE_INFO_ML = 1000138001,
  // Provided by XR_ML_marker_understanding
    XR_TYPE_MARKER_DETECTOR_ARUCO_INFO_ML = 1000138002,
  // Provided by XR_ML_marker_understanding
    XR_TYPE_MARKER_DETECTOR_SIZE_INFO_ML = 1000138003,
  // Provided by XR_ML_marker_understanding
    XR_TYPE_MARKER_DETECTOR_APRIL_TAG_INFO_ML = 1000138004,
  // Provided by XR_ML_marker_understanding
    XR_TYPE_MARKER_DETECTOR_CUSTOM_PROFILE_INFO_ML = 1000138005,
  // Provided by XR_ML_marker_understanding
    XR_TYPE_MARKER_DETECTOR_SNAPSHOT_INFO_ML = 1000138006,
  // Provided by XR_ML_marker_understanding
    XR_TYPE_MARKER_DETECTOR_STATE_ML = 1000138007,
  // Provided by XR_ML_marker_understanding
    XR_TYPE_MARKER_SPACE_CREATE_INFO_ML = 1000138008,
  // Provided by XR_ML_localization_map
    XR_TYPE_LOCALIZATION_MAP_ML = 1000139000,
  // Provided by XR_ML_localization_map
    XR_TYPE_EVENT_DATA_LOCALIZATION_CHANGED_ML = 1000139001,
  // Provided by XR_ML_localization_map
    XR_TYPE_MAP_LOCALIZATION_REQUEST_INFO_ML = 1000139002,
  // Provided by XR_ML_localization_map
    XR_TYPE_LOCALIZATION_MAP_IMPORT_INFO_ML = 1000139003,
  // Provided by XR_ML_localization_map
    XR_TYPE_LOCALIZATION_ENABLE_EVENTS_INFO_ML = 1000139004,
  // Provided by XR_ML_spatial_anchors
    XR_TYPE_SPATIAL_ANCHORS_CREATE_INFO_FROM_POSE_ML = 1000140000,
  // Provided by XR_ML_spatial_anchors
    XR_TYPE_CREATE_SPATIAL_ANCHORS_COMPLETION_ML = 1000140001,
  // Provided by XR_ML_spatial_anchors
    XR_TYPE_SPATIAL_ANCHOR_STATE_ML = 1000140002,
  // Provided by XR_ML_spatial_anchors_storage
    XR_TYPE_SPATIAL_ANCHORS_CREATE_STORAGE_INFO_ML = 1000141000,
  // Provided by XR_ML_spatial_anchors_storage
    XR_TYPE_SPATIAL_ANCHORS_QUERY_INFO_RADIUS_ML = 1000141001,
  // Provided by XR_ML_spatial_anchors_storage
    XR_TYPE_SPATIAL_ANCHORS_QUERY_COMPLETION_ML = 1000141002,
  // Provided by XR_ML_spatial_anchors_storage
    XR_TYPE_SPATIAL_ANCHORS_CREATE_INFO_FROM_UUIDS_ML = 1000141003,
  // Provided by XR_ML_spatial_anchors_storage
```

```
    XR_TYPE_SPATIAL_ANCHORS_PUBLISH_INFO_ML = 1000141004,
  // Provided by XR_ML_spatial_anchors_storage
    XR_TYPE_SPATIAL_ANCHORS_PUBLISH_COMPLETION_ML = 1000141005,
  // Provided by XR_ML_spatial_anchors_storage
    XR_TYPE_SPATIAL_ANCHORS_DELETE_INFO_ML = 1000141006,
  // Provided by XR_ML_spatial_anchors_storage
    XR_TYPE_SPATIAL_ANCHORS_DELETE_COMPLETION_ML = 1000141007,
  // Provided by XR_ML_spatial_anchors_storage
    XR_TYPE_SPATIAL_ANCHORS_UPDATE_EXPIRATION_INFO_ML = 1000141008,
  // Provided by XR_ML_spatial_anchors_storage
    XR_TYPE_SPATIAL_ANCHORS_UPDATE_EXPIRATION_COMPLETION_ML = 1000141009,
  // Provided by XR_ML_spatial_anchors_storage
    XR_TYPE_SPATIAL_ANCHORS_PUBLISH_COMPLETION_DETAILS_ML = 1000141010,
  // Provided by XR_ML_spatial_anchors_storage
    XR_TYPE_SPATIAL_ANCHORS_DELETE_COMPLETION_DETAILS_ML = 1000141011,
  // Provided by XR_ML_spatial_anchors_storage
    XR_TYPE_SPATIAL_ANCHORS_UPDATE_EXPIRATION_COMPLETION_DETAILS_ML = 1000141012,
  // Provided by XR_ML_user_calibration
    XR_TYPE_EVENT_DATA_HEADSET_FIT_CHANGED_ML = 1000472000,
  // Provided by XR_ML_user_calibration
    XR_TYPE_EVENT_DATA_EYE_CALIBRATION_CHANGED_ML = 1000472001,
  // Provided by XR_ML_user_calibration
    XR_TYPE_USER_CALIBRATION_ENABLE_EVENTS_INFO_ML = 1000472002,
  // Provided by XR_MSFT_spatial_anchor_persistence
    XR_TYPE_SPATIAL_ANCHOR_PERSISTENCE_INFO_MSFT = 1000142000,
  // Provided by XR_MSFT_spatial_anchor_persistence
    XR_TYPE_SPATIAL_ANCHOR_FROM_PERSISTED_ANCHOR_CREATE_INFO_MSFT = 1000142001,
  // Provided by XR_MSFT_scene_marker
    XR_TYPE_SCENE_MARKERS_MSFT = 1000147000,
  // Provided by XR_MSFT_scene_marker
    XR_TYPE_SCENE_MARKER_TYPE_FILTER_MSFT = 1000147001,
  // Provided by XR_MSFT_scene_marker
    XR_TYPE_SCENE_MARKER_QR_CODES_MSFT = 1000147002,
  // Provided by XR_FB_spatial_entity_query
    XR_TYPE_SPACE_QUERY_INFO_FB = 1000156001,
  // Provided by XR_FB_spatial_entity_query
    XR_TYPE_SPACE_QUERY_RESULTS_FB = 1000156002,
  // Provided by XR_FB_spatial_entity_query
    XR_TYPE_SPACE_STORAGE_LOCATION_FILTER_INFO_FB = 1000156003,
  // Provided by XR_FB_spatial_entity_query
    XR_TYPE_SPACE_UUID_FILTER_INFO_FB = 1000156054,
  // Provided by XR_FB_spatial_entity_query
    XR_TYPE_SPACE_COMPONENT_FILTER_INFO_FB = 1000156052,
  // Provided by XR_FB_spatial_entity_query
    XR_TYPE_EVENT_DATA_SPACE_QUERY_RESULTS_AVAILABLE_FB = 1000156103,
  // Provided by XR_FB_spatial_entity_query
    XR_TYPE_EVENT_DATA_SPACE_QUERY_COMPLETE_FB = 1000156104,
  // Provided by XR_FB_spatial_entity_storage
```

```
    XR_TYPE_SPACE_SAVE_INFO_FB = 1000158000,
  // Provided by XR_FB_spatial_entity_storage
    XR_TYPE_SPACE_ERASE_INFO_FB = 1000158001,
  // Provided by XR_FB_spatial_entity_storage
    XR_TYPE_EVENT_DATA_SPACE_SAVE_COMPLETE_FB = 1000158106,
  // Provided by XR_FB_spatial_entity_storage
    XR_TYPE_EVENT_DATA_SPACE_ERASE_COMPLETE_FB = 1000158107,
  // Provided by XR_FB_foveation_vulkan
    XR_TYPE_SWAPCHAIN_IMAGE_FOVEATION_VULKAN_FB = 1000160000,
  // Provided by XR_FB_swapchain_update_state_android_surface
    XR_TYPE_SWAPCHAIN_STATE_ANDROID_SURFACE_DIMENSIONS_FB = 1000161000,
  // Provided by XR_FB_swapchain_update_state_opengl_es
    XR_TYPE_SWAPCHAIN_STATE_SAMPLER_OPENGL_ES_FB = 1000162000,
  // Provided by XR_FB_swapchain_update_state_vulkan
    XR_TYPE_SWAPCHAIN_STATE_SAMPLER_VULKAN_FB = 1000163000,
  // Provided by XR_FB_spatial_entity_sharing
    XR_TYPE_SPACE_SHARE_INFO_FB = 1000169001,
  // Provided by XR_FB_spatial_entity_sharing
    XR_TYPE_EVENT_DATA_SPACE_SHARE_COMPLETE_FB = 1000169002,
  // Provided by XR_FB_space_warp
    XR_TYPE_COMPOSITION_LAYER_SPACE_WARP_INFO_FB = 1000171000,
  // Provided by XR_FB_space_warp
    XR_TYPE_SYSTEM_SPACE_WARP_PROPERTIES_FB = 1000171001,
  // Provided by XR_FB_haptic_amplitude_envelope
    XR_TYPE_HAPTIC_AMPLITUDE_ENVELOPE_VIBRATION_FB = 1000173001,
  // Provided by XR_FB_scene
    XR_TYPE_SEMANTIC_LABELS_FB = 1000175000,
  // Provided by XR_FB_scene
    XR_TYPE_ROOM_LAYOUT_FB = 1000175001,
  // Provided by XR_FB_scene
    XR_TYPE_BOUNDARY_2D_FB = 1000175002,
  // Provided by XR_FB_scene
    XR_TYPE_SEMANTIC_LABELS_SUPPORT_INFO_FB = 1000175010,
  // Provided by XR_ALMALENCE_digital_lens_control
    XR_TYPE_DIGITAL_LENS_CONTROL_ALMALENCE = 1000196000,
  // Provided by XR_FB_scene_capture
    XR_TYPE_EVENT_DATA_SCENE_CAPTURE_COMPLETE_FB = 1000198001,
  // Provided by XR_FB_scene_capture
    XR_TYPE_SCENE_CAPTURE_REQUEST_INFO_FB = 1000198050,
  // Provided by XR_FB_spatial_entity_container
    XR_TYPE_SPACE_CONTAINER_FB = 1000199000,
  // Provided by XR_META_foveation_eye_tracked
    XR_TYPE_FOVEATION_EYE_TRACKED_PROFILE_CREATE_INFO_META = 1000200000,
  // Provided by XR_META_foveation_eye_tracked
    XR_TYPE_FOVEATION_EYE_TRACKED_STATE_META = 1000200001,
  // Provided by XR_META_foveation_eye_tracked
    XR_TYPE_SYSTEM_FOVEATION_EYE_TRACKED_PROPERTIES_META = 1000200002,
  // Provided by XR_FB_face_tracking
```

```
    XR_TYPE_SYSTEM_FACE_TRACKING_PROPERTIES_FB = 1000201004,
  // Provided by XR_FB_face_tracking
    XR_TYPE_FACE_TRACKER_CREATE_INFO_FB = 1000201005,
  // Provided by XR_FB_face_tracking
    XR_TYPE_FACE_EXPRESSION_INFO_FB = 1000201002,
  // Provided by XR_FB_face_tracking
    XR_TYPE_FACE_EXPRESSION_WEIGHTS_FB = 1000201006,
  // Provided by XR_FB_eye_tracking_social
    XR_TYPE_EYE_TRACKER_CREATE_INFO_FB = 1000202001,
  // Provided by XR_FB_eye_tracking_social
    XR_TYPE_EYE_GAZES_INFO_FB = 1000202002,
  // Provided by XR_FB_eye_tracking_social
    XR_TYPE_EYE_GAZES_FB = 1000202003,
  // Provided by XR_FB_eye_tracking_social
    XR_TYPE_SYSTEM_EYE_TRACKING_PROPERTIES_FB = 1000202004,
  // Provided by XR_FB_passthrough_keyboard_hands
    XR_TYPE_PASSTHROUGH_KEYBOARD_HANDS_INTENSITY_FB = 1000203002,
  // Provided by XR_FB_composition_layer_settings
    XR_TYPE_COMPOSITION_LAYER_SETTINGS_FB = 1000204000,
  // Provided by XR_FB_haptic_pcm
    XR_TYPE_HAPTIC_PCM_VIBRATION_FB = 1000209001,
  // Provided by XR_FB_haptic_pcm
    XR_TYPE_DEVICE_PCM_SAMPLE_RATE_STATE_FB = 1000209002,
  // Provided by XR_EXT_frame_synthesis
    XR_TYPE_FRAME_SYNTHESIS_INFO_EXT = 1000211000,
  // Provided by XR_EXT_frame_synthesis
    XR_TYPE_FRAME_SYNTHESIS_CONFIG_VIEW_EXT = 1000211001,
  // Provided by XR_FB_composition_layer_depth_test
    XR_TYPE_COMPOSITION_LAYER_DEPTH_TEST_FB = 1000212000,
  // Provided by XR_META_local_dimming
    XR_TYPE_LOCAL_DIMMING_FRAME_END_INFO_META = 1000216000,
  // Provided by XR_META_passthrough_preferences
    XR_TYPE_PASSTHROUGH_PREFERENCES_META = 1000217000,
  // Provided by XR_META_virtual_keyboard
    XR_TYPE_SYSTEM_VIRTUAL_KEYBOARD_PROPERTIES_META = 1000219001,
  // Provided by XR_META_virtual_keyboard
    XR_TYPE_VIRTUAL_KEYBOARD_CREATE_INFO_META = 1000219002,
  // Provided by XR_META_virtual_keyboard
    XR_TYPE_VIRTUAL_KEYBOARD_SPACE_CREATE_INFO_META = 1000219003,
  // Provided by XR_META_virtual_keyboard
    XR_TYPE_VIRTUAL_KEYBOARD_LOCATION_INFO_META = 1000219004,
  // Provided by XR_META_virtual_keyboard
    XR_TYPE_VIRTUAL_KEYBOARD_MODEL_VISIBILITY_SET_INFO_META = 1000219005,
  // Provided by XR_META_virtual_keyboard
    XR_TYPE_VIRTUAL_KEYBOARD_ANIMATION_STATE_META = 1000219006,
  // Provided by XR_META_virtual_keyboard
    XR_TYPE_VIRTUAL_KEYBOARD_MODEL_ANIMATION_STATES_META = 1000219007,
  // Provided by XR_META_virtual_keyboard
```

```
    XR_TYPE_VIRTUAL_KEYBOARD_TEXTURE_DATA_META = 1000219009,
  // Provided by XR_META_virtual_keyboard
    XR_TYPE_VIRTUAL_KEYBOARD_INPUT_INFO_META = 1000219010,
  // Provided by XR_META_virtual_keyboard
    XR_TYPE_VIRTUAL_KEYBOARD_TEXT_CONTEXT_CHANGE_INFO_META = 1000219011,
  // Provided by XR_META_virtual_keyboard
    XR_TYPE_EVENT_DATA_VIRTUAL_KEYBOARD_COMMIT_TEXT_META = 1000219014,
  // Provided by XR_META_virtual_keyboard
    XR_TYPE_EVENT_DATA_VIRTUAL_KEYBOARD_BACKSPACE_META = 1000219015,
  // Provided by XR_META_virtual_keyboard
    XR_TYPE_EVENT_DATA_VIRTUAL_KEYBOARD_ENTER_META = 1000219016,
  // Provided by XR_META_virtual_keyboard
    XR_TYPE_EVENT_DATA_VIRTUAL_KEYBOARD_SHOWN_META = 1000219017,
  // Provided by XR_META_virtual_keyboard
    XR_TYPE_EVENT_DATA_VIRTUAL_KEYBOARD_HIDDEN_META = 1000219018,
  // Provided by XR_OCULUS_external_camera
    XR_TYPE_EXTERNAL_CAMERA_OCULUS = 1000226000,
  // Provided by XR_META_vulkan_swapchain_create_info
    XR_TYPE_VULKAN_SWAPCHAIN_CREATE_INFO_META = 1000227000,
  // Provided by XR_META_performance_metrics
    XR_TYPE_PERFORMANCE_METRICS_STATE_META = 1000232001,
  // Provided by XR_META_performance_metrics
    XR_TYPE_PERFORMANCE_METRICS_COUNTER_META = 1000232002,
  // Provided by XR_FB_spatial_entity_storage_batch
    XR_TYPE_SPACE_LIST_SAVE_INFO_FB = 1000238000,
  // Provided by XR_FB_spatial_entity_storage_batch
    XR_TYPE_EVENT_DATA_SPACE_LIST_SAVE_COMPLETE_FB = 1000238001,
  // Provided by XR_FB_spatial_entity_user
    XR_TYPE_SPACE_USER_CREATE_INFO_FB = 1000241001,
  // Provided by XR_META_headset_id
    XR_TYPE_SYSTEM_HEADSET_ID_PROPERTIES_META = 1000245000,
  // Provided by XR_META_spatial_entity_discovery
    XR_TYPE_SYSTEM_SPACE_DISCOVERY_PROPERTIES_META = 1000247000,
  // Provided by XR_META_spatial_entity_discovery
    XR_TYPE_SPACE_DISCOVERY_INFO_META = 1000247001,
  // Provided by XR_META_spatial_entity_discovery
    XR_TYPE_SPACE_FILTER_UUID_META = 1000247003,
  // Provided by XR_META_spatial_entity_discovery
    XR_TYPE_SPACE_FILTER_COMPONENT_META = 1000247004,
  // Provided by XR_META_spatial_entity_discovery
    XR_TYPE_SPACE_DISCOVERY_RESULT_META = 1000247005,
  // Provided by XR_META_spatial_entity_discovery
    XR_TYPE_SPACE_DISCOVERY_RESULTS_META = 1000247006,
  // Provided by XR_META_spatial_entity_discovery
    XR_TYPE_EVENT_DATA_SPACE_DISCOVERY_RESULTS_AVAILABLE_META = 1000247007,
  // Provided by XR_META_spatial_entity_discovery
    XR_TYPE_EVENT_DATA_SPACE_DISCOVERY_COMPLETE_META = 1000247008,
  // Provided by XR_META_recommended_layer_resolution
```

```
    XR_TYPE_RECOMMENDED_LAYER_RESOLUTION_META = 1000254000,
  // Provided by XR_META_recommended_layer_resolution
    XR_TYPE_RECOMMENDED_LAYER_RESOLUTION_GET_INFO_META = 1000254001,
  // Provided by XR_META_spatial_entity_persistence
    XR_TYPE_SYSTEM_SPACE_PERSISTENCE_PROPERTIES_META = 1000259000,
  // Provided by XR_META_spatial_entity_persistence
    XR_TYPE_SPACES_SAVE_INFO_META = 1000259001,
  // Provided by XR_META_spatial_entity_persistence
    XR_TYPE_EVENT_DATA_SPACES_SAVE_RESULT_META = 1000259002,
  // Provided by XR_META_spatial_entity_persistence
    XR_TYPE_SPACES_ERASE_INFO_META = 1000259003,
  // Provided by XR_META_spatial_entity_persistence
    XR_TYPE_EVENT_DATA_SPACES_ERASE_RESULT_META = 1000259004,
  // Provided by XR_META_passthrough_color_lut
    XR_TYPE_SYSTEM_PASSTHROUGH_COLOR_LUT_PROPERTIES_META = 1000266000,
  // Provided by XR_META_passthrough_color_lut
    XR_TYPE_PASSTHROUGH_COLOR_LUT_CREATE_INFO_META = 1000266001,
  // Provided by XR_META_passthrough_color_lut
    XR_TYPE_PASSTHROUGH_COLOR_LUT_UPDATE_INFO_META = 1000266002,
  // Provided by XR_META_passthrough_color_lut
    XR_TYPE_PASSTHROUGH_COLOR_MAP_LUT_META = 1000266100,
  // Provided by XR_META_passthrough_color_lut
    XR_TYPE_PASSTHROUGH_COLOR_MAP_INTERPOLATED_LUT_META = 1000266101,
  // Provided by XR_META_spatial_entity_mesh
    XR_TYPE_SPACE_TRIANGLE_MESH_GET_INFO_META = 1000269001,
  // Provided by XR_META_spatial_entity_mesh
    XR_TYPE_SPACE_TRIANGLE_MESH_META = 1000269002,
  // Provided by XR_META_body_tracking_full_body
    XR_TYPE_SYSTEM_PROPERTIES_BODY_TRACKING_FULL_BODY_META = 1000274000,
  // Provided by XR_META_passthrough_layer_resumed_event
    XR_TYPE_EVENT_DATA_PASSTHROUGH_LAYER_RESUMED_META = 1000282000,
  // Provided by XR_META_body_tracking_calibration
    XR_TYPE_BODY_TRACKING_CALIBRATION_INFO_META = 1000283002,
  // Provided by XR_META_body_tracking_calibration
    XR_TYPE_BODY_TRACKING_CALIBRATION_STATUS_META = 1000283003,
  // Provided by XR_META_body_tracking_calibration
    XR_TYPE_SYSTEM_PROPERTIES_BODY_TRACKING_CALIBRATION_META = 1000283004,
  // Provided by XR_META_body_tracking_fidelity
    XR_TYPE_BODY_TRACKING_FIDELITY_STATUS_META = 1000284000,
  // Provided by XR_META_body_tracking_fidelity
    XR_TYPE_SYSTEM_PROPERTIES_BODY_TRACKING_FIDELITY_META = 1000284001,
  // Provided by XR_FB_face_tracking2
    XR_TYPE_SYSTEM_FACE_TRACKING_PROPERTIES2_FB = 1000287013,
  // Provided by XR_FB_face_tracking2
    XR_TYPE_FACE_TRACKER_CREATE_INFO2_FB = 1000287014,
  // Provided by XR_FB_face_tracking2
    XR_TYPE_FACE_EXPRESSION_INFO2_FB = 1000287015,
  // Provided by XR_FB_face_tracking2
```

```
    XR_TYPE_FACE_EXPRESSION_WEIGHTS2_FB = 1000287016,
  // Provided by XR_META_spatial_entity_sharing
    XR_TYPE_SYSTEM_SPATIAL_ENTITY_SHARING_PROPERTIES_META = 1000290000,
  // Provided by XR_META_spatial_entity_sharing
    XR_TYPE_SHARE_SPACES_INFO_META = 1000290001,
  // Provided by XR_META_spatial_entity_sharing
    XR_TYPE_EVENT_DATA_SHARE_SPACES_COMPLETE_META = 1000290002,
  // Provided by XR_META_environment_depth
    XR_TYPE_ENVIRONMENT_DEPTH_PROVIDER_CREATE_INFO_META = 1000291000,
  // Provided by XR_META_environment_depth
    XR_TYPE_ENVIRONMENT_DEPTH_SWAPCHAIN_CREATE_INFO_META = 1000291001,
  // Provided by XR_META_environment_depth
    XR_TYPE_ENVIRONMENT_DEPTH_SWAPCHAIN_STATE_META = 1000291002,
  // Provided by XR_META_environment_depth
    XR_TYPE_ENVIRONMENT_DEPTH_IMAGE_ACQUIRE_INFO_META = 1000291003,
  // Provided by XR_META_environment_depth
    XR_TYPE_ENVIRONMENT_DEPTH_IMAGE_VIEW_META = 1000291004,
  // Provided by XR_META_environment_depth
    XR_TYPE_ENVIRONMENT_DEPTH_IMAGE_META = 1000291005,
  // Provided by XR_META_environment_depth
    XR_TYPE_ENVIRONMENT_DEPTH_HAND_REMOVAL_SET_INFO_META = 1000291006,
  // Provided by XR_META_environment_depth
    XR_TYPE_SYSTEM_ENVIRONMENT_DEPTH_PROPERTIES_META = 1000291007,
  // Provided by XR_META_environment_depth
    XR_TYPE_ENVIRONMENT_DEPTH_IMAGE_TIMESTAMP_META = 1000291008,
  // Provided by XR_EXT_render_model
    XR_TYPE_RENDER_MODEL_CREATE_INFO_EXT = 1000300000,
  // Provided by XR_EXT_render_model
    XR_TYPE_RENDER_MODEL_PROPERTIES_GET_INFO_EXT = 1000300001,
  // Provided by XR_EXT_render_model
    XR_TYPE_RENDER_MODEL_PROPERTIES_EXT = 1000300002,
  // Provided by XR_EXT_render_model
    XR_TYPE_RENDER_MODEL_SPACE_CREATE_INFO_EXT = 1000300003,
  // Provided by XR_EXT_render_model
    XR_TYPE_RENDER_MODEL_STATE_GET_INFO_EXT = 1000300004,
  // Provided by XR_EXT_render_model
    XR_TYPE_RENDER_MODEL_STATE_EXT = 1000300005,
  // Provided by XR_EXT_render_model
    XR_TYPE_RENDER_MODEL_ASSET_CREATE_INFO_EXT = 1000300006,
  // Provided by XR_EXT_render_model
    XR_TYPE_RENDER_MODEL_ASSET_DATA_GET_INFO_EXT = 1000300007,
  // Provided by XR_EXT_render_model
    XR_TYPE_RENDER_MODEL_ASSET_DATA_EXT = 1000300008,
  // Provided by XR_EXT_render_model
    XR_TYPE_RENDER_MODEL_ASSET_PROPERTIES_GET_INFO_EXT = 1000300009,
  // Provided by XR_EXT_render_model
    XR_TYPE_RENDER_MODEL_ASSET_PROPERTIES_EXT = 1000300010,
  // Provided by XR_EXT_interaction_render_model
```

```
    XR_TYPE_INTERACTION_RENDER_MODEL_IDS_ENUMERATE_INFO_EXT = 1000301000,
  // Provided by XR_EXT_interaction_render_model
    XR_TYPE_INTERACTION_RENDER_MODEL_SUBACTION_PATH_INFO_EXT = 1000301001,
  // Provided by XR_EXT_interaction_render_model
    XR_TYPE_EVENT_DATA_INTERACTION_RENDER_MODELS_CHANGED_EXT = 1000301002,
  // Provided by XR_EXT_interaction_render_model
    XR_TYPE_INTERACTION_RENDER_MODEL_TOP_LEVEL_USER_PATH_GET_INFO_EXT = 1000301003,
  // Provided by XR_HTC_passthrough
    XR_TYPE_PASSTHROUGH_CREATE_INFO_HTC = 1000317001,
  // Provided by XR_HTC_passthrough
    XR_TYPE_PASSTHROUGH_COLOR_HTC = 1000317002,
  // Provided by XR_HTC_passthrough
    XR_TYPE_PASSTHROUGH_MESH_TRANSFORM_INFO_HTC = 1000317003,
  // Provided by XR_HTC_passthrough
    XR_TYPE_COMPOSITION_LAYER_PASSTHROUGH_HTC = 1000317004,
  // Provided by XR_HTC_foveation
    XR_TYPE_FOVEATION_APPLY_INFO_HTC = 1000318000,
  // Provided by XR_HTC_foveation
    XR_TYPE_FOVEATION_DYNAMIC_MODE_INFO_HTC = 1000318001,
  // Provided by XR_HTC_foveation
    XR_TYPE_FOVEATION_CUSTOM_MODE_INFO_HTC = 1000318002,
  // Provided by XR_HTC_anchor
    XR_TYPE_SYSTEM_ANCHOR_PROPERTIES_HTC = 1000319000,
  // Provided by XR_HTC_anchor
    XR_TYPE_SPATIAL_ANCHOR_CREATE_INFO_HTC = 1000319001,
  // Provided by XR_HTC_body_tracking
    XR_TYPE_SYSTEM_BODY_TRACKING_PROPERTIES_HTC = 1000320000,
  // Provided by XR_HTC_body_tracking
    XR_TYPE_BODY_TRACKER_CREATE_INFO_HTC = 1000320001,
  // Provided by XR_HTC_body_tracking
    XR_TYPE_BODY_JOINTS_LOCATE_INFO_HTC = 1000320002,
  // Provided by XR_HTC_body_tracking
    XR_TYPE_BODY_JOINT_LOCATIONS_HTC = 1000320003,
  // Provided by XR_HTC_body_tracking
    XR_TYPE_BODY_SKELETON_HTC = 1000320004,
  // Provided by XR_EXT_active_action_set_priority
    XR_TYPE_ACTIVE_ACTION_SET_PRIORITIES_EXT = 1000373000,
  // Provided by XR_MNDX_force_feedback_curl
    XR_TYPE_SYSTEM_FORCE_FEEDBACK_CURL_PROPERTIES_MNDX = 1000375000,
  // Provided by XR_MNDX_force_feedback_curl
    XR_TYPE_FORCE_FEEDBACK_CURL_APPLY_LOCATIONS_MNDX = 1000375001,
  // Provided by XR_BD_body_tracking
    XR_TYPE_BODY_TRACKER_CREATE_INFO_BD = 1000385001,
  // Provided by XR_BD_body_tracking
    XR_TYPE_BODY_JOINTS_LOCATE_INFO_BD = 1000385002,
  // Provided by XR_BD_body_tracking
    XR_TYPE_BODY_JOINT_LOCATIONS_BD = 1000385003,
  // Provided by XR_BD_body_tracking
```

```
    XR_TYPE_SYSTEM_BODY_TRACKING_PROPERTIES_BD = 1000385004,
  // Provided by XR_BD_facial_simulation
    XR_TYPE_SYSTEM_FACIAL_SIMULATION_PROPERTIES_BD = 1000386001,
  // Provided by XR_BD_facial_simulation
    XR_TYPE_FACE_TRACKER_CREATE_INFO_BD = 1000386002,
  // Provided by XR_BD_facial_simulation
    XR_TYPE_FACIAL_SIMULATION_DATA_GET_INFO_BD = 1000386003,
  // Provided by XR_BD_facial_simulation
    XR_TYPE_FACIAL_SIMULATION_DATA_BD = 1000386004,
  // Provided by XR_BD_facial_simulation
    XR_TYPE_LIP_EXPRESSION_DATA_BD = 1000386005,
  // Provided by XR_BD_spatial_sensing
    XR_TYPE_SYSTEM_SPATIAL_SENSING_PROPERTIES_BD = 1000389000,
  // Provided by XR_BD_spatial_sensing
    XR_TYPE_SPATIAL_ENTITY_COMPONENT_GET_INFO_BD = 1000389001,
  // Provided by XR_BD_spatial_sensing
    XR_TYPE_SPATIAL_ENTITY_LOCATION_GET_INFO_BD = 1000389002,
  // Provided by XR_BD_spatial_sensing
    XR_TYPE_SPATIAL_ENTITY_COMPONENT_DATA_LOCATION_BD = 1000389003,
  // Provided by XR_BD_spatial_sensing
    XR_TYPE_SPATIAL_ENTITY_COMPONENT_DATA_SEMANTIC_BD = 1000389004,
  // Provided by XR_BD_spatial_sensing
    XR_TYPE_SPATIAL_ENTITY_COMPONENT_DATA_BOUNDING_BOX_2D_BD = 1000389005,
  // Provided by XR_BD_spatial_sensing
    XR_TYPE_SPATIAL_ENTITY_COMPONENT_DATA_POLYGON_BD = 1000389006,
  // Provided by XR_BD_spatial_sensing
    XR_TYPE_SPATIAL_ENTITY_COMPONENT_DATA_BOUNDING_BOX_3D_BD = 1000389007,
  // Provided by XR_BD_spatial_sensing
    XR_TYPE_SPATIAL_ENTITY_COMPONENT_DATA_TRIANGLE_MESH_BD = 1000389008,
  // Provided by XR_BD_spatial_sensing
    XR_TYPE_SENSE_DATA_PROVIDER_CREATE_INFO_BD = 1000389009,
  // Provided by XR_BD_spatial_sensing
    XR_TYPE_SENSE_DATA_PROVIDER_START_INFO_BD = 1000389010,
  // Provided by XR_BD_spatial_sensing
    XR_TYPE_EVENT_DATA_SENSE_DATA_PROVIDER_STATE_CHANGED_BD = 1000389011,
  // Provided by XR_BD_spatial_sensing
    XR_TYPE_EVENT_DATA_SENSE_DATA_UPDATED_BD = 1000389012,
  // Provided by XR_BD_spatial_sensing
    XR_TYPE_SENSE_DATA_QUERY_INFO_BD = 1000389013,
  // Provided by XR_BD_spatial_sensing
    XR_TYPE_SENSE_DATA_QUERY_COMPLETION_BD = 1000389014,
  // Provided by XR_BD_spatial_sensing
    XR_TYPE_SENSE_DATA_FILTER_UUID_BD = 1000389015,
  // Provided by XR_BD_spatial_sensing
    XR_TYPE_SENSE_DATA_FILTER_SEMANTIC_BD = 1000389016,
  // Provided by XR_BD_spatial_sensing
    XR_TYPE_QUERIED_SENSE_DATA_GET_INFO_BD = 1000389017,
  // Provided by XR_BD_spatial_sensing
```

```
    XR_TYPE_QUERIED_SENSE_DATA_BD = 1000389018,
  // Provided by XR_BD_spatial_sensing
    XR_TYPE_SPATIAL_ENTITY_STATE_BD = 1000389019,
  // Provided by XR_BD_spatial_sensing
    XR_TYPE_SPATIAL_ENTITY_ANCHOR_CREATE_INFO_BD = 1000389020,
  // Provided by XR_BD_spatial_sensing
    XR_TYPE_ANCHOR_SPACE_CREATE_INFO_BD = 1000389021,
  // Provided by XR_BD_spatial_sensing
    XR_TYPE_SPATIAL_ENTITY_COMPONENT_DATA_SPHERE_BD = 1000389022,
  // Provided by XR_BD_spatial_anchor
    XR_TYPE_SYSTEM_SPATIAL_ANCHOR_PROPERTIES_BD = 1000390000,
  // Provided by XR_BD_spatial_anchor
    XR_TYPE_SPATIAL_ANCHOR_CREATE_INFO_BD = 1000390001,
  // Provided by XR_BD_spatial_anchor
    XR_TYPE_SPATIAL_ANCHOR_CREATE_COMPLETION_BD = 1000390002,
  // Provided by XR_BD_spatial_anchor
    XR_TYPE_SPATIAL_ANCHOR_PERSIST_INFO_BD = 1000390003,
  // Provided by XR_BD_spatial_anchor
    XR_TYPE_SPATIAL_ANCHOR_UNPERSIST_INFO_BD = 1000390004,
  // Provided by XR_BD_spatial_anchor_sharing
    XR_TYPE_SYSTEM_SPATIAL_ANCHOR_SHARING_PROPERTIES_BD = 1000391000,
  // Provided by XR_BD_spatial_anchor_sharing
    XR_TYPE_SPATIAL_ANCHOR_SHARE_INFO_BD = 1000391001,
  // Provided by XR_BD_spatial_anchor_sharing
    XR_TYPE_SHARED_SPATIAL_ANCHOR_DOWNLOAD_INFO_BD = 1000391002,
  // Provided by XR_BD_spatial_scene
    XR_TYPE_SYSTEM_SPATIAL_SCENE_PROPERTIES_BD = 1000392000,
  // Provided by XR_BD_spatial_scene
    XR_TYPE_SCENE_CAPTURE_INFO_BD = 1000392001,
  // Provided by XR_BD_spatial_mesh
    XR_TYPE_SYSTEM_SPATIAL_MESH_PROPERTIES_BD = 1000393000,
  // Provided by XR_BD_spatial_mesh
    XR_TYPE_SENSE_DATA_PROVIDER_CREATE_INFO_SPATIAL_MESH_BD = 1000393001,
  // Provided by XR_BD_future_progress
    XR_TYPE_FUTURE_POLL_RESULT_PROGRESS_BD = 1000394001,
  // Provided by XR_BD_body_tracking_auxiliary_metrics
    XR_TYPE_BODY_TRACKING_POSTURE_DATA_BD = 1000395001,
  // Provided by XR_BD_body_tracking_auxiliary_metrics
    XR_TYPE_BODY_JOINT_VELOCITIES_BD = 1000395002,
  // Provided by XR_BD_body_tracking_auxiliary_metrics
    XR_TYPE_BODY_JOINT_ACCELERATIONS_BD = 1000395003,
  // Provided by XR_BD_body_tracking_auxiliary_metrics
    XR_TYPE_BODY_TRACKING_STATE_BD = 1000395004,
  // Provided by XR_BD_spatial_plane
    XR_TYPE_SYSTEM_SPATIAL_PLANE_PROPERTIES_BD = 1000396000,
  // Provided by XR_BD_spatial_plane
    XR_TYPE_SPATIAL_ENTITY_COMPONENT_DATA_PLANE_ORIENTATION_BD = 1000396001,
  // Provided by XR_BD_spatial_plane
```

```
    XR_TYPE_SENSE_DATA_FILTER_PLANE_ORIENTATION_BD = 1000396002,
  // Provided by XR_BD_spatial_audio_rendering
    XR_TYPE_SPATIAL_AUDIO_RENDERER_CREATE_INFO_BD = 1000409000,
  // Provided by XR_BD_spatial_audio_rendering
    XR_TYPE_AUDIO_BUFFER_BD = 1000409001,
  // Provided by XR_BD_spatial_audio_rendering
    XR_TYPE_SOUND_OBJECT_DIRECTIVITY_CARDIOID_BD = 1000409003,
  // Provided by XR_BD_spatial_audio_rendering
    XR_TYPE_SOUND_OBJECT_SHAPE_SPHERE_BD = 1000409004,
  // Provided by XR_BD_spatial_audio_rendering
    XR_TYPE_SOUND_OBJECT_DISTANCE_ATTENUATION_BD = 1000409005,
  // Provided by XR_BD_spatial_audio_rendering
    XR_TYPE_SOUND_OBJECT_DISTANCE_ATTENUATION_CURVE_BD = 1000409006,
  // Provided by XR_BD_spatial_audio_rendering
    XR_TYPE_SOUND_OBJECT_CONFIG_BD = 1000409007,
  // Provided by XR_BD_spatial_audio_rendering
    XR_TYPE_SOUND_FIELD_CONFIG_BD = 1000409008,
  // Provided by XR_BD_spatial_audio_rendering
    XR_TYPE_SOUND_FIELD_CHANNEL_DEFINITION_SURROUND_BD = 1000409009,
  // Provided by XR_BD_spatial_audio_rendering
    XR_TYPE_SOUND_FIELD_CHANNEL_DEFINITION_AMBIX_BD = 1000409010,
  // Provided by XR_BD_spatial_audio_rendering
    XR_TYPE_SOUND_FIELD_CHANNEL_DEFINITION_FUMA_BD = 1000409011,
  // Provided by XR_BD_spatial_audio_rendering
    XR_TYPE_SOUND_TRIANGLE_MESH_BD = 1000409012,
  // Provided by XR_BD_spatial_audio_rendering
    XR_TYPE_SOUND_OBSTACLE_CONFIG_BD = 1000409013,
  // Provided by XR_BD_spatial_audio_rendering
    XR_TYPE_SOUND_OBSTACLE_MATERIAL_CONFIG_BD = 1000409014,
  // Provided by XR_EXT_hand_tracking_data_source
    XR_TYPE_HAND_TRACKING_DATA_SOURCE_INFO_EXT = 1000428000,
  // Provided by XR_EXT_hand_tracking_data_source
    XR_TYPE_HAND_TRACKING_DATA_SOURCE_STATE_EXT = 1000428001,
  // Provided by XR_EXT_plane_detection
    XR_TYPE_PLANE_DETECTOR_CREATE_INFO_EXT = 1000429001,
  // Provided by XR_EXT_plane_detection
    XR_TYPE_PLANE_DETECTOR_BEGIN_INFO_EXT = 1000429002,
  // Provided by XR_EXT_plane_detection
    XR_TYPE_PLANE_DETECTOR_GET_INFO_EXT = 1000429003,
  // Provided by XR_EXT_plane_detection
    XR_TYPE_PLANE_DETECTOR_LOCATIONS_EXT = 1000429004,
  // Provided by XR_EXT_plane_detection
    XR_TYPE_PLANE_DETECTOR_LOCATION_EXT = 1000429005,
  // Provided by XR_EXT_plane_detection
    XR_TYPE_PLANE_DETECTOR_POLYGON_BUFFER_EXT = 1000429006,
  // Provided by XR_EXT_plane_detection
    XR_TYPE_SYSTEM_PLANE_DETECTION_PROPERTIES_EXT = 1000429007,
  // Provided by XR_ANDROID_trackables
```

```
    XR_TYPE_TRACKABLE_GET_INFO_ANDROID = 1000455000,
  // Provided by XR_ANDROID_trackables
    XR_TYPE_ANCHOR_SPACE_CREATE_INFO_ANDROID = 1000455001,
  // Provided by XR_ANDROID_trackables
    XR_TYPE_TRACKABLE_PLANE_ANDROID = 1000455003,
  // Provided by XR_ANDROID_trackables
    XR_TYPE_TRACKABLE_TRACKER_CREATE_INFO_ANDROID = 1000455004,
  // Provided by XR_ANDROID_trackables
    XR_TYPE_SYSTEM_TRACKABLES_PROPERTIES_ANDROID = 1000455005,
  // Provided by XR_ANDROID_eye_tracking
    XR_TYPE_EYES_ANDROID = 1000456000,
  // Provided by XR_ANDROID_eye_tracking
    XR_TYPE_EYE_TRACKER_CREATE_INFO_ANDROID = 1000456001,
  // Provided by XR_ANDROID_eye_tracking
    XR_TYPE_EYES_GET_INFO_ANDROID = 1000456002,
  // Provided by XR_ANDROID_eye_tracking
    XR_TYPE_SYSTEM_EYE_TRACKING_PROPERTIES_ANDROID = 1000456004,
  // Provided by XR_ANDROID_device_anchor_persistence
    XR_TYPE_PERSISTED_ANCHOR_SPACE_CREATE_INFO_ANDROID = 1000457001,
  // Provided by XR_ANDROID_device_anchor_persistence
    XR_TYPE_PERSISTED_ANCHOR_SPACE_INFO_ANDROID = 1000457002,
  // Provided by XR_ANDROID_device_anchor_persistence
    XR_TYPE_DEVICE_ANCHOR_PERSISTENCE_CREATE_INFO_ANDROID = 1000457003,
  // Provided by XR_ANDROID_device_anchor_persistence
    XR_TYPE_SYSTEM_DEVICE_ANCHOR_PERSISTENCE_PROPERTIES_ANDROID = 1000457004,
  // Provided by XR_ANDROID_face_tracking
    XR_TYPE_FACE_TRACKER_CREATE_INFO_ANDROID = 1000458000,
  // Provided by XR_ANDROID_face_tracking
    XR_TYPE_FACE_STATE_GET_INFO_ANDROID = 1000458001,
  // Provided by XR_ANDROID_face_tracking
    XR_TYPE_FACE_STATE_ANDROID = 1000458002,
  // Provided by XR_ANDROID_face_tracking
    XR_TYPE_SYSTEM_FACE_TRACKING_PROPERTIES_ANDROID = 1000458003,
  // Provided by XR_ANDROID_passthrough_camera_state
    XR_TYPE_PASSTHROUGH_CAMERA_STATE_GET_INFO_ANDROID = 1000460000,
  // Provided by XR_ANDROID_passthrough_camera_state
    XR_TYPE_SYSTEM_PASSTHROUGH_CAMERA_STATE_PROPERTIES_ANDROID = 1000460001,
  // Provided by XR_ANDROID_recommended_resolution
    XR_TYPE_EVENT_DATA_RECOMMENDED_RESOLUTION_CHANGED_ANDROID = 1000461000,
  // Provided by XR_ANDROID_composition_layer_passthrough_mesh
    XR_TYPE_PASSTHROUGH_LAYER_CREATE_INFO_ANDROID = 1000462000,
  // Provided by XR_ANDROID_composition_layer_passthrough_mesh
    XR_TYPE_PASSTHROUGH_LAYER_MESH_ANDROID = 1000462001,
  // Provided by XR_ANDROID_composition_layer_passthrough_mesh
    XR_TYPE_COMPOSITION_LAYER_PASSTHROUGH_ANDROID = 1000462002,
  // Provided by XR_ANDROID_composition_layer_passthrough_mesh
    XR_TYPE_SYSTEM_PASSTHROUGH_LAYER_PROPERTIES_ANDROID = 1000462003,
  // Provided by XR_ANDROID_raycast
```

```
    XR_TYPE_RAYCAST_INFO_ANDROID = 1000463000,
  // Provided by XR_ANDROID_raycast
    XR_TYPE_RAYCAST_HIT_RESULTS_ANDROID = 1000463001,
  // Provided by XR_ANDROID_performance_metrics
    XR_TYPE_PERFORMANCE_METRICS_STATE_ANDROID = 1000465000,
  // Provided by XR_ANDROID_performance_metrics
    XR_TYPE_PERFORMANCE_METRICS_COUNTER_ANDROID = 1000465001,
  // Provided by XR_ANDROID_trackables_object
    XR_TYPE_TRACKABLE_OBJECT_ANDROID = 1000466000,
  // Provided by XR_ANDROID_trackables_object
    XR_TYPE_TRACKABLE_OBJECT_CONFIGURATION_ANDROID = 1000466001,
  // Provided by XR_EXT_future
    XR_TYPE_FUTURE_CANCEL_INFO_EXT = 1000469000,
  // Provided by XR_EXT_future
    XR_TYPE_FUTURE_POLL_INFO_EXT = 1000469001,
  // Provided by XR_EXT_future
    XR_TYPE_FUTURE_COMPLETION_EXT = 1000469002,
  // Provided by XR_EXT_future
    XR_TYPE_FUTURE_POLL_RESULT_EXT = 1000469003,
  // Provided by XR_EXT_user_presence
    XR_TYPE_EVENT_DATA_USER_PRESENCE_CHANGED_EXT = 1000470000,
  // Provided by XR_EXT_user_presence
    XR_TYPE_SYSTEM_USER_PRESENCE_PROPERTIES_EXT = 1000470001,
  // Provided by XR_ML_system_notifications
    XR_TYPE_SYSTEM_NOTIFICATIONS_SET_INFO_ML = 1000473000,
  // Provided by XR_ML_world_mesh_detection
    XR_TYPE_WORLD_MESH_DETECTOR_CREATE_INFO_ML = 1000474001,
  // Provided by XR_ML_world_mesh_detection
    XR_TYPE_WORLD_MESH_STATE_REQUEST_INFO_ML = 1000474002,
  // Provided by XR_ML_world_mesh_detection
    XR_TYPE_WORLD_MESH_BLOCK_STATE_ML = 1000474003,
  // Provided by XR_ML_world_mesh_detection
    XR_TYPE_WORLD_MESH_STATE_REQUEST_COMPLETION_ML = 1000474004,
  // Provided by XR_ML_world_mesh_detection
    XR_TYPE_WORLD_MESH_BUFFER_RECOMMENDED_SIZE_INFO_ML = 1000474005,
  // Provided by XR_ML_world_mesh_detection
    XR_TYPE_WORLD_MESH_BUFFER_SIZE_ML = 1000474006,
  // Provided by XR_ML_world_mesh_detection
    XR_TYPE_WORLD_MESH_BUFFER_ML = 1000474007,
  // Provided by XR_ML_world_mesh_detection
    XR_TYPE_WORLD_MESH_BLOCK_REQUEST_ML = 1000474008,
  // Provided by XR_ML_world_mesh_detection
    XR_TYPE_WORLD_MESH_GET_INFO_ML = 1000474009,
  // Provided by XR_ML_world_mesh_detection
    XR_TYPE_WORLD_MESH_BLOCK_ML = 1000474010,
  // Provided by XR_ML_world_mesh_detection
    XR_TYPE_WORLD_MESH_REQUEST_COMPLETION_ML = 1000474011,
  // Provided by XR_ML_world_mesh_detection
```

```
    XR_TYPE_WORLD_MESH_REQUEST_COMPLETION_INFO_ML = 1000474012,
  // Provided by XR_ML_facial_expression
    XR_TYPE_SYSTEM_FACIAL_EXPRESSION_PROPERTIES_ML = 1000482004,
  // Provided by XR_ML_facial_expression
    XR_TYPE_FACIAL_EXPRESSION_CLIENT_CREATE_INFO_ML = 1000482005,
  // Provided by XR_ML_facial_expression
    XR_TYPE_FACIAL_EXPRESSION_BLEND_SHAPE_GET_INFO_ML = 1000482006,
  // Provided by XR_ML_facial_expression
    XR_TYPE_FACIAL_EXPRESSION_BLEND_SHAPE_PROPERTIES_ML = 1000482007,
  // Provided by XR_META_boundary_visibility
    XR_TYPE_SYSTEM_BOUNDARY_VISIBILITY_PROPERTIES_META = 1000528000,
  // Provided by XR_META_boundary_visibility
    XR_TYPE_EVENT_DATA_BOUNDARY_VISIBILITY_CHANGED_META = 1000528001,
  // Provided by XR_META_simultaneous_hands_and_controllers
    XR_TYPE_SYSTEM_SIMULTANEOUS_HANDS_AND_CONTROLLERS_PROPERTIES_META = 1000532001,
  // Provided by XR_META_simultaneous_hands_and_controllers
    XR_TYPE_SIMULTANEOUS_HANDS_AND_CONTROLLERS_TRACKING_RESUME_INFO_META = 1000532002,
  // Provided by XR_META_simultaneous_hands_and_controllers
    XR_TYPE_SIMULTANEOUS_HANDS_AND_CONTROLLERS_TRACKING_PAUSE_INFO_META = 1000532003,
  // Provided by XR_META_face_tracking_visemes
    XR_TYPE_FACE_TRACKING_VISEMES_META = 1000541000,
  // Provided by XR_META_face_tracking_visemes
    XR_TYPE_SYSTEM_FACE_TRACKING_VISEMES_PROPERTIES_META = 1000541001,
  // Provided by XR_META_spatial_entity_room_mesh
    XR_TYPE_ROOM_MESH_FACE_INDICES_META = 1000553000,
  // Provided by XR_META_spatial_entity_room_mesh
    XR_TYPE_SPACE_ROOM_MESH_GET_INFO_META = 1000553001,
  // Provided by XR_META_spatial_entity_room_mesh
    XR_TYPE_ROOM_MESH_META = 1000553002,
  // Provided by XR_META_colocation_discovery
    XR_TYPE_COLOCATION_DISCOVERY_START_INFO_META = 1000571010,
  // Provided by XR_META_colocation_discovery
    XR_TYPE_COLOCATION_DISCOVERY_STOP_INFO_META = 1000571011,
  // Provided by XR_META_colocation_discovery
    XR_TYPE_COLOCATION_ADVERTISEMENT_START_INFO_META = 1000571012,
  // Provided by XR_META_colocation_discovery
    XR_TYPE_COLOCATION_ADVERTISEMENT_STOP_INFO_META = 1000571013,
  // Provided by XR_META_colocation_discovery
    XR_TYPE_EVENT_DATA_START_COLOCATION_ADVERTISEMENT_COMPLETE_META = 1000571020,
  // Provided by XR_META_colocation_discovery
    XR_TYPE_EVENT_DATA_STOP_COLOCATION_ADVERTISEMENT_COMPLETE_META = 1000571021,
  // Provided by XR_META_colocation_discovery
    XR_TYPE_EVENT_DATA_COLOCATION_ADVERTISEMENT_COMPLETE_META = 1000571022,
  // Provided by XR_META_colocation_discovery
    XR_TYPE_EVENT_DATA_START_COLOCATION_DISCOVERY_COMPLETE_META = 1000571023,
  // Provided by XR_META_colocation_discovery
    XR_TYPE_EVENT_DATA_COLOCATION_DISCOVERY_RESULT_META = 1000571024,
  // Provided by XR_META_colocation_discovery
```

```
    XR_TYPE_EVENT_DATA_COLOCATION_DISCOVERY_COMPLETE_META = 1000571025,
  // Provided by XR_META_colocation_discovery
    XR_TYPE_EVENT_DATA_STOP_COLOCATION_DISCOVERY_COMPLETE_META = 1000571026,
  // Provided by XR_META_colocation_discovery
    XR_TYPE_SYSTEM_COLOCATION_DISCOVERY_PROPERTIES_META = 1000571030,
  // Provided by XR_META_spatial_entity_group_sharing
    XR_TYPE_SHARE_SPACES_RECIPIENT_GROUPS_META = 1000572000,
  // Provided by XR_META_spatial_entity_group_sharing
    XR_TYPE_SPACE_GROUP_UUID_FILTER_INFO_META = 1000572001,
  // Provided by XR_META_spatial_entity_group_sharing
    XR_TYPE_SYSTEM_SPATIAL_ENTITY_GROUP_SHARING_PROPERTIES_META = 1000572100,
  // Provided by XR_META_environment_raycast
    XR_TYPE_SYSTEM_ENVIRONMENT_RAYCAST_PROPERTIES_META = 1000592000,
  // Provided by XR_META_environment_raycast
    XR_TYPE_ENVIRONMENT_RAYCASTER_CREATE_INFO_META = 1000592001,
  // Provided by XR_META_environment_raycast
    XR_TYPE_ENVIRONMENT_RAYCASTER_CREATE_COMPLETION_META = 1000592002,
  // Provided by XR_META_environment_raycast
    XR_TYPE_ENVIRONMENT_RAYCAST_HIT_GET_INFO_META = 1000592003,
  // Provided by XR_META_environment_raycast
    XR_TYPE_ENVIRONMENT_RAYCAST_HIT_META = 1000592004,
  // Provided by XR_META_environment_raycast
    XR_TYPE_ENVIRONMENT_RAYCAST_FILTER_DISTANCE_META = 1000592005,
  // Provided by XR_META_tile_properties_hint
    XR_TYPE_TILE_PROPERTIES_META = 1000609000,
  // Provided by XR_META_tile_properties_hint
    XR_TYPE_TILE_PROPERTIES_HINT_META = 1000609001,
  // Provided by XR_META_hand_tracking_unextrapolated_poses
    XR_TYPE_HAND_TRACKING_UNEXTRAPOLATED_POSES_REQUEST_META = 1000693000,
  // Provided by XR_META_hand_tracking_unextrapolated_poses
    XR_TYPE_HAND_TRACKING_UNEXTRAPOLATED_POSES_META = 1000693001,
  // Provided by XR_ANDROID_light_estimation
    XR_TYPE_LIGHT_ESTIMATOR_CREATE_INFO_ANDROID = 1000700000,
  // Provided by XR_ANDROID_light_estimation
    XR_TYPE_LIGHT_ESTIMATE_GET_INFO_ANDROID = 1000700001,
  // Provided by XR_ANDROID_light_estimation
    XR_TYPE_LIGHT_ESTIMATE_ANDROID = 1000700002,
  // Provided by XR_ANDROID_light_estimation
    XR_TYPE_DIRECTIONAL_LIGHT_ANDROID = 1000700003,
  // Provided by XR_ANDROID_light_estimation
    XR_TYPE_SPHERICAL_HARMONICS_ANDROID = 1000700004,
  // Provided by XR_ANDROID_light_estimation
    XR_TYPE_AMBIENT_LIGHT_ANDROID = 1000700005,
  // Provided by XR_ANDROID_light_estimation
    XR_TYPE_SYSTEM_LIGHT_ESTIMATION_PROPERTIES_ANDROID = 1000700006,
  // Provided by XR_ANDROID_anchor_sharing_export
    XR_TYPE_ANCHOR_SHARING_INFO_ANDROID = 1000701000,
  // Provided by XR_ANDROID_anchor_sharing_export
```

```
    XR_TYPE_ANCHOR_SHARING_TOKEN_ANDROID = 1000701001,
  // Provided by XR_ANDROID_anchor_sharing_export
    XR_TYPE_SYSTEM_ANCHOR_SHARING_EXPORT_PROPERTIES_ANDROID = 1000701002,
  // Provided by XR_ANDROID_trackables_marker
    XR_TYPE_SYSTEM_MARKER_TRACKING_PROPERTIES_ANDROID = 1000707000,
  // Provided by XR_ANDROID_trackables_marker
    XR_TYPE_TRACKABLE_MARKER_CONFIGURATION_ANDROID = 1000707001,
  // Provided by XR_ANDROID_trackables_marker
    XR_TYPE_TRACKABLE_MARKER_ANDROID = 1000707002,
  // Provided by XR_ANDROID_trackables_qr_code
    XR_TYPE_SYSTEM_QR_CODE_TRACKING_PROPERTIES_ANDROID = 1000708000,
  // Provided by XR_ANDROID_trackables_qr_code
    XR_TYPE_TRACKABLE_QR_CODE_CONFIGURATION_ANDROID = 1000708001,
  // Provided by XR_ANDROID_trackables_qr_code
    XR_TYPE_TRACKABLE_QR_CODE_ANDROID = 1000708002,
  // Provided by XR_ANDROID_trackables_image
    XR_TYPE_SYSTEM_IMAGE_TRACKING_PROPERTIES_ANDROID = 1000709000,
  // Provided by XR_ANDROID_trackables_image
    XR_TYPE_TRACKABLE_IMAGE_DATABASE_ENTRY_ANDROID = 1000709001,
  // Provided by XR_ANDROID_trackables_image
    XR_TYPE_TRACKABLE_IMAGE_DATABASE_CREATE_INFO_ANDROID = 1000709002,
  // Provided by XR_ANDROID_trackables_image
    XR_TYPE_CREATE_TRACKABLE_IMAGE_DATABASE_COMPLETION_ANDROID = 1000709003,
  // Provided by XR_ANDROID_trackables_image
    XR_TYPE_TRACKABLE_IMAGE_CONFIGURATION_ANDROID = 1000709004,
  // Provided by XR_ANDROID_trackables_image
    XR_TYPE_TRACKABLE_IMAGE_ANDROID = 1000709005,
  // Provided by XR_ANDROID_trackables_image
    XR_TYPE_EVENT_DATA_IMAGE_TRACKING_LOST_ANDROID = 1000709006,
  // Provided by XR_ANDROID_scene_meshing
    XR_TYPE_SYSTEM_SCENE_MESHING_PROPERTIES_ANDROID = 1000718000,
  // Provided by XR_ANDROID_scene_meshing
    XR_TYPE_SCENE_MESHING_TRACKER_CREATE_INFO_ANDROID = 1000718001,
  // Provided by XR_ANDROID_scene_meshing
    XR_TYPE_SCENE_MESH_SNAPSHOT_CREATE_INFO_ANDROID = 1000718002,
  // Provided by XR_ANDROID_scene_meshing
    XR_TYPE_SCENE_MESH_SNAPSHOT_CREATION_RESULT_ANDROID = 1000718003,
  // Provided by XR_ANDROID_scene_meshing
    XR_TYPE_SCENE_SUBMESH_STATE_ANDROID = 1000718004,
  // Provided by XR_ANDROID_scene_meshing
    XR_TYPE_SCENE_SUBMESH_DATA_ANDROID = 1000718005,
  // Provided by XR_EXT_spatial_entity
    XR_TYPE_SPATIAL_CAPABILITY_COMPONENT_TYPES_EXT = 1000740000,
  // Provided by XR_EXT_spatial_entity
    XR_TYPE_SPATIAL_CONTEXT_CREATE_INFO_EXT = 1000740001,
  // Provided by XR_EXT_spatial_entity
    XR_TYPE_CREATE_SPATIAL_CONTEXT_COMPLETION_EXT = 1000740002,
  // Provided by XR_EXT_spatial_entity
```

```
    XR_TYPE_SPATIAL_DISCOVERY_SNAPSHOT_CREATE_INFO_EXT = 1000740003,
  // Provided by XR_EXT_spatial_entity
    XR_TYPE_CREATE_SPATIAL_DISCOVERY_SNAPSHOT_COMPLETION_INFO_EXT = 1000740004,
  // Provided by XR_EXT_spatial_entity
    XR_TYPE_CREATE_SPATIAL_DISCOVERY_SNAPSHOT_COMPLETION_EXT = 1000740005,
  // Provided by XR_EXT_spatial_entity
    XR_TYPE_SPATIAL_COMPONENT_DATA_QUERY_CONDITION_EXT = 1000740006,
  // Provided by XR_EXT_spatial_entity
    XR_TYPE_SPATIAL_COMPONENT_DATA_QUERY_RESULT_EXT = 1000740007,
  // Provided by XR_EXT_spatial_entity
    XR_TYPE_SPATIAL_BUFFER_GET_INFO_EXT = 1000740008,
  // Provided by XR_EXT_spatial_entity
    XR_TYPE_SPATIAL_COMPONENT_BOUNDED_2D_LIST_EXT = 1000740009,
  // Provided by XR_EXT_spatial_entity
    XR_TYPE_SPATIAL_COMPONENT_BOUNDED_3D_LIST_EXT = 1000740010,
  // Provided by XR_EXT_spatial_entity
    XR_TYPE_SPATIAL_COMPONENT_PARENT_LIST_EXT = 1000740011,
  // Provided by XR_EXT_spatial_entity
    XR_TYPE_SPATIAL_COMPONENT_MESH_3D_LIST_EXT = 1000740012,
  // Provided by XR_EXT_spatial_entity
    XR_TYPE_SPATIAL_ENTITY_FROM_ID_CREATE_INFO_EXT = 1000740013,
  // Provided by XR_EXT_spatial_entity
    XR_TYPE_SPATIAL_UPDATE_SNAPSHOT_CREATE_INFO_EXT = 1000740014,
  // Provided by XR_EXT_spatial_entity
    XR_TYPE_EVENT_DATA_SPATIAL_DISCOVERY_RECOMMENDED_EXT = 1000740015,
  // Provided by XR_EXT_spatial_entity
    XR_TYPE_SPATIAL_FILTER_TRACKING_STATE_EXT = 1000740016,
  // Provided by XR_EXT_spatial_plane_tracking
    XR_TYPE_SPATIAL_CAPABILITY_CONFIGURATION_PLANE_TRACKING_EXT = 1000741000,
  // Provided by XR_EXT_spatial_plane_tracking
    XR_TYPE_SPATIAL_COMPONENT_PLANE_ALIGNMENT_LIST_EXT = 1000741001,
  // Provided by XR_EXT_spatial_plane_tracking
    XR_TYPE_SPATIAL_COMPONENT_MESH_2D_LIST_EXT = 1000741002,
  // Provided by XR_EXT_spatial_plane_tracking
    XR_TYPE_SPATIAL_COMPONENT_POLYGON_2D_LIST_EXT = 1000741003,
  // Provided by XR_EXT_spatial_plane_tracking
    XR_TYPE_SPATIAL_COMPONENT_PLANE_SEMANTIC_LABEL_LIST_EXT = 1000741004,
  // Provided by XR_EXT_stationary_reference_space
    XR_TYPE_STATIONARY_REFERENCE_SPACE_GENERATION_ID_GET_INFO_EXT = 1000742001,
  // Provided by XR_EXT_stationary_reference_space
    XR_TYPE_STATIONARY_REFERENCE_SPACE_GENERATION_ID_RESULT_EXT = 1000742002,
  // Provided by XR_EXT_spatial_marker_tracking
    XR_TYPE_SPATIAL_CAPABILITY_CONFIGURATION_QR_CODE_EXT = 1000743000,
  // Provided by XR_EXT_spatial_marker_tracking
    XR_TYPE_SPATIAL_CAPABILITY_CONFIGURATION_MICRO_QR_CODE_EXT = 1000743001,
  // Provided by XR_EXT_spatial_marker_tracking
    XR_TYPE_SPATIAL_CAPABILITY_CONFIGURATION_ARUCO_MARKER_EXT = 1000743002,
  // Provided by XR_EXT_spatial_marker_tracking
```

```
    XR_TYPE_SPATIAL_CAPABILITY_CONFIGURATION_APRIL_TAG_EXT = 1000743003,
  // Provided by XR_EXT_spatial_marker_tracking
    XR_TYPE_SPATIAL_MARKER_SIZE_EXT = 1000743004,
  // Provided by XR_EXT_spatial_marker_tracking
    XR_TYPE_SPATIAL_MARKER_STATIC_OPTIMIZATION_EXT = 1000743005,
  // Provided by XR_EXT_spatial_marker_tracking
    XR_TYPE_SPATIAL_COMPONENT_MARKER_LIST_EXT = 1000743006,
  // Provided by XR_BD_dynamic_object_tracking
    XR_TYPE_SYSTEM_DYNAMIC_OBJECT_TRACKING_PROPERTIES_BD = 1000746000,
  // Provided by XR_BD_dynamic_object_tracking
    XR_TYPE_SENSE_DATA_PROVIDER_CREATE_INFO_DYNAMIC_OBJECT_BD = 1000746001,
  // Provided by XR_BD_dynamic_object_tracking
    XR_TYPE_SPATIAL_ENTITY_COMPONENT_DATA_DYNAMIC_OBJECT_BD = 1000746002,
  // Provided by XR_BD_dynamic_object_tracking
    XR_TYPE_DYNAMIC_OBJECT_DATA_BD = 1000746003,
  // Provided by XR_BD_dynamic_object_tracking
    XR_TYPE_SENSE_DATA_FILTER_DYNAMIC_OBJECT_TYPE_BD = 1000746004,
  // Provided by XR_BD_dynamic_object_keyboard
    XR_TYPE_SYSTEM_DYNAMIC_OBJECT_KEYBOARD_PROPERTIES_BD = 1000747000,
  // Provided by XR_BD_dynamic_object_mouse
    XR_TYPE_SYSTEM_DYNAMIC_OBJECT_MOUSE_PROPERTIES_BD = 1000748000,
  // Provided by XR_ANDROID_spatial_discovery_bounds
    XR_TYPE_SPATIAL_BOUNDS_SPHEREF_ANDROID = 1000761000,
  // Provided by XR_ANDROID_spatial_discovery_bounds
    XR_TYPE_SPATIAL_BOUNDS_BOXF_ANDROID = 1000761001,
  // Provided by XR_ANDROID_spatial_discovery_bounds
    XR_TYPE_SPATIAL_BOUNDS_FRUSTUMF_ANDROID = 1000761002,
  // Provided by XR_EXT_spatial_anchor
    XR_TYPE_SPATIAL_CAPABILITY_CONFIGURATION_ANCHOR_EXT = 1000762000,
  // Provided by XR_EXT_spatial_anchor
    XR_TYPE_SPATIAL_COMPONENT_ANCHOR_LIST_EXT = 1000762001,
  // Provided by XR_EXT_spatial_anchor
    XR_TYPE_SPATIAL_ANCHOR_CREATE_INFO_EXT = 1000762002,
  // Provided by XR_EXT_spatial_persistence
    XR_TYPE_SPATIAL_PERSISTENCE_CONTEXT_CREATE_INFO_EXT = 1000763000,
  // Provided by XR_EXT_spatial_persistence
    XR_TYPE_CREATE_SPATIAL_PERSISTENCE_CONTEXT_COMPLETION_EXT = 1000763001,
  // Provided by XR_EXT_spatial_persistence
    XR_TYPE_SPATIAL_CONTEXT_PERSISTENCE_CONFIG_EXT = 1000763002,
  // Provided by XR_EXT_spatial_persistence
    XR_TYPE_SPATIAL_DISCOVERY_PERSISTENCE_UUID_FILTER_EXT = 1000763003,
  // Provided by XR_EXT_spatial_persistence
    XR_TYPE_SPATIAL_COMPONENT_PERSISTENCE_LIST_EXT = 1000763004,
  // Provided by XR_EXT_haptic_parametric
    XR_TYPE_HAPTIC_PARAMETRIC_VIBRATION_EXT = 1000775000,
  // Provided by XR_EXT_haptic_parametric
    XR_TYPE_HAPTIC_PARAMETRIC_PROPERTIES_EXT = 1000775001,
  // Provided by XR_EXT_haptic_parametric
```

```
    XR_TYPE_SYSTEM_HAPTIC_PARAMETRIC_PROPERTIES_EXT = 1000775002,
  // Provided by XR_SONY_swapchain_color_space
    XR_TYPE_COLOR_SPACES_ENUMERATE_INFO_SONY = 1000776000,
  // Provided by XR_SONY_swapchain_color_space
    XR_TYPE_SWAPCHAIN_CREATE_INFO_COLOR_SPACE_SONY = 1000776001,
  // Provided by XR_EXT_spatial_persistence_operations
    XR_TYPE_SPATIAL_ENTITY_PERSIST_INFO_EXT = 1000781000,
  // Provided by XR_EXT_spatial_persistence_operations
    XR_TYPE_PERSIST_SPATIAL_ENTITY_COMPLETION_EXT = 1000781001,
  // Provided by XR_EXT_spatial_persistence_operations
    XR_TYPE_SPATIAL_ENTITY_UNPERSIST_INFO_EXT = 1000781002,
  // Provided by XR_EXT_spatial_persistence_operations
    XR_TYPE_UNPERSIST_SPATIAL_ENTITY_COMPLETION_EXT = 1000781003,
  // Provided by XR_ANDROID_spatial_object_tracking
    XR_TYPE_SPATIAL_CAPABILITY_CONFIGURATION_OBJECT_TRACKING_ANDROID = 1000785000,
  // Provided by XR_ANDROID_spatial_object_tracking
    XR_TYPE_SPATIAL_COMPONENT_OBJECT_SEMANTIC_LABEL_LIST_ANDROID = 1000785001,
  // Provided by XR_ANDROID_spatial_discovery_raycast
    XR_TYPE_SPATIAL_CAPABILITY_CONFIGURATION_DEPTH_RAYCAST_ANDROID = 1000786000,
  // Provided by XR_ANDROID_spatial_discovery_raycast
    XR_TYPE_SPATIAL_RAYCAST_INFO_ANDROID = 1000786001,
  // Provided by XR_ANDROID_spatial_discovery_raycast
    XR_TYPE_SPATIAL_COMPONENT_RAYCAST_RESULT_LIST_ANDROID = 1000786002,
  // Provided by XR_ANDROID_spatial_discovery_raycast
    XR_TYPE_SPATIAL_RAYCAST_SNAPSHOT_CREATE_INFO_ANDROID = 1000786003,
  // Provided by XR_ANDROID_google_cloud_auth
    XR_TYPE_GOOGLE_CLOUD_AUTH_INFO_API_KEY_ANDROID = 1000787000,
  // Provided by XR_ANDROID_google_cloud_auth
    XR_TYPE_GOOGLE_CLOUD_AUTH_INFO_TOKEN_ANDROID = 1000787001,
  // Provided by XR_ANDROID_google_cloud_auth
    XR_TYPE_GOOGLE_CLOUD_AUTH_INFO_KEYLESS_ANDROID = 1000787002,
  // Provided by XR_ANDROID_google_cloud_auth
    XR_TYPE_GOOGLE_CLOUD_AUTH_ERROR_RESULT_ANDROID = 1000787003,
  // Provided by XR_ANDROID_geospatial
    XR_TYPE_SYSTEM_GEOSPATIAL_PROPERTIES_ANDROID = 1000789000,
  // Provided by XR_ANDROID_geospatial
    XR_TYPE_GEOSPATIAL_TRACKER_CREATE_INFO_ANDROID = 1000789001,
  // Provided by XR_ANDROID_geospatial
    XR_TYPE_EVENT_DATA_GEOSPATIAL_TRACKER_STATE_CHANGED_ANDROID = 1000789002,
  // Provided by XR_ANDROID_geospatial
    XR_TYPE_GEOSPATIAL_POSE_FROM_POSE_LOCATE_INFO_ANDROID = 1000789003,
  // Provided by XR_ANDROID_geospatial
    XR_TYPE_GEOSPATIAL_POSE_RESULT_ANDROID = 1000789004,
  // Provided by XR_ANDROID_geospatial
    XR_TYPE_GEOSPATIAL_POSE_LOCATE_INFO_ANDROID = 1000789005,
  // Provided by XR_ANDROID_geospatial
    XR_TYPE_VPS_AVAILABILITY_CHECK_COMPLETION_ANDROID = 1000789006,
  // Provided by XR_ANDROID_spatial_entity_bound_anchor
```

```
    XR_TYPE_SPATIAL_ANCHOR_PARENT_ANDROID = 1000790000,
  // Provided by XR_ANDROID_spatial_component_subsumed_by
    XR_TYPE_SPATIAL_DISCOVERY_UNIQUE_ENTITIES_FILTER_ANDROID = 1000791001,
  // Provided by XR_ANDROID_spatial_component_subsumed_by
    XR_TYPE_SPATIAL_COMPONENT_SUBSUMED_BY_LIST_ANDROID = 1000791002,
  // Provided by XR_ANDROID_spatial_anchor_space
    XR_TYPE_SPATIAL_ANCHOR_SPACE_FROM_ID_CREATE_INFO_ANDROID = 1000795000,
  // Provided by XR_ANDROID_geospatial_anchor
    XR_TYPE_GEOSPATIAL_ANCHOR_CREATE_INFO_ANDROID = 1000797000,
  // Provided by XR_ANDROID_geospatial_anchor
    XR_TYPE_SURFACE_ANCHOR_CREATE_INFO_ANDROID = 1000797001,
  // Provided by XR_ANDROID_geospatial_anchor
    XR_TYPE_SURFACE_ANCHOR_CREATE_COMPLETION_ANDROID = 1000797002,
  // Provided by XR_ANDROID_geospatial_anchor
    XR_TYPE_SYSTEM_GEOSPATIAL_ANCHOR_PROPERTIES_ANDROID = 1000797003,
  // Provided by XR_ANDROID_geospatial_anchor
    XR_TYPE_GEOSPATIAL_TRACKER_ANCHOR_TRACKING_INFO_ANDROID = 1000797004,
  // Provided by XR_EXT_interaction_profile_battery_state_display
    XR_TYPE_BATTERY_STATE_DISPLAY_EXT = 1000836000,
  // Provided by XR_EXT_loader_init_properties
    XR_TYPE_LOADER_INIT_INFO_PROPERTIES_EXT = 1000838000,
  // Provided by XR_EXT_view_configuration_views_change
    XR_TYPE_EVENT_DATA_VIEW_CONFIGURATION_VIEWS_CHANGED_EXT = 1000839000,
  // Provided by XR_KHR_vulkan_enable2
    XR_TYPE_GRAPHICS_BINDING_VULKAN2_KHR = XR_TYPE_GRAPHICS_BINDING_VULKAN_KHR,
  // Provided by XR_KHR_vulkan_enable2
    XR_TYPE_SWAPCHAIN_IMAGE_VULKAN2_KHR = XR_TYPE_SWAPCHAIN_IMAGE_VULKAN_KHR,
  // Provided by XR_KHR_vulkan_enable2
    XR_TYPE_GRAPHICS_REQUIREMENTS_VULKAN2_KHR = XR_TYPE_GRAPHICS_REQUIREMENTS_VULKAN_KHR,
  // Provided by XR_FB_haptic_pcm
    XR_TYPE_DEVICE_PCM_SAMPLE_RATE_GET_INFO_FB = XR_TYPE_DEVICE_PCM_SAMPLE_RATE_STATE_FB,
  // Provided by XR_KHR_locate_spaces
    XR_TYPE_SPACES_LOCATE_INFO_KHR = XR_TYPE_SPACES_LOCATE_INFO,
  // Provided by XR_KHR_locate_spaces
    XR_TYPE_SPACE_LOCATIONS_KHR = XR_TYPE_SPACE_LOCATIONS,
  // Provided by XR_KHR_locate_spaces
    XR_TYPE_SPACE_VELOCITIES_KHR = XR_TYPE_SPACE_VELOCITIES,
    XR_STRUCTURE_TYPE_MAX_ENUM = 0x7FFFFFFF
} XrStructureType;
```

Most structures containing `type` members have a value of `type` matching the type of the structure, as described more fully in Valid Usage for Structure Types.

Note that all extension enums begin at the extension enum base of 10^9 (base 10). Each extension is assigned a block of 1000 enums, starting at the enum base and arranged by the extension's number.

```
// Provided by XR_VERSION_1_0
#define XR_EXTENSION_ENUM_BASE 1000000000
```

```
// Provided by XR_VERSION_1_0
#define XR_EXTENSION_ENUM_STRIDE 1000
```

For example, if extension number 5 wants to use an enum value of 3, the final enum is computed by:

enum = XR_EXTENSION_ENUM_BASE + (extension_number - 1) * XR_EXTENSION_ENUM_STRIDE + enum_value

1000004003 = 1000000000 + 4 * 1000 + 3

The maximum allowed enum value in an extension is 2,147,482,999, which belongs to extension number 1147483.

Flag Types

Flag types are all bitmasks aliasing the base type `XrFlags64` and with corresponding bit flag types defining the valid bits for that flag, as described in Valid Usage for Flags.

Flag types defined in the core specification were originally listed/defined here, but have been moved to be adjacent to their associated `FlagBits` type. See the Index for a list.

General Macro Definitions

This API is defined in C and uses "C" linkage. The `openxr.h` header file is opened with:

```
1 #ifdef __cplusplus
2 extern "C" {
3 #endif
```

and closed with:

```
1 #ifdef __cplusplus
2 }
3 #endif
```

The supplied `openxr.h` header defines a small number of C preprocessor macros that are described below.

Version Number Macros

Three version numbers are defined in `openxr.h`. Each is packed into a 64-bit integer as described in API Version Number Function-like Macros.

```
// Provided by XR_VERSION_1_0
// OpenXR current version number.
#define XR_CURRENT_API_VERSION XR_MAKE_VERSION(1, 1, 62)
```

XR_CURRENT_API_VERSION is the current version of the OpenXR API.

In many cases, XR_API_VERSION_1_0 or XR_API_VERSION_1_1 are preferred for source forward-compatibility.

```
// Provided by XR_VERSION_1_0
// OpenXR 1.0 version number
#define XR_API_VERSION_1_0 XR_MAKE_VERSION(1, 0,
XR_VERSION_PATCH(XR_CURRENT_API_VERSION))
```

XR_API_VERSION_1_0 is the version of the OpenXR 1.0 API. The "major" and "minor" components are always 1.0, while the "patch" component matches XR_CURRENT_API_VERSION.

```
// Provided by XR_VERSION_1_1
// OpenXR 1.1 version number
#define XR_API_VERSION_1_1 XR_MAKE_VERSION(1, 1,
XR_VERSION_PATCH(XR_CURRENT_API_VERSION))
```

XR_API_VERSION_1_1 is the version of the OpenXR 1.1 API. The "major" and "minor" components are always 1.1, while the "patch" component matches XR_CURRENT_API_VERSION.

API Version Number Function-like Macros

API Version Numbers are three components, packed into a single 64-bit integer. The following macros manipulate version components and packed version numbers.

```
// Provided by XR_VERSION_1_0
#define XR_MAKE_VERSION(major, minor, patch) \
    ((((major) & 0xffffULL) << 48) | (((minor) & 0xffffULL) << 32) | ((patch) &
0xffffffffULL))
```

Parameter Descriptions

- `major` is the major version number, packed into the most-significant 16 bits.
- `minor` is the minor version number, packed into the second-most-significant group of 16 bits.
- `patch` is the patch version number, in the least-significant 32 bits.

XR_MAKE_VERSION constructs a packed 64-bit integer API version number from three components. The format used is described in API Version Numbers and Semantics.

This macro **can** be used when constructing the XrApplicationInfo::`apiVersion` parameter passed to xrCreateInstance.

```
// Provided by XR_VERSION_1_0
#define XR_VERSION_MAJOR(version) (uint16_t)(((uint64_t)(version) >> 48)& 0xffffULL)
```

Parameter Descriptions

- `version` is a packed version number, such as those produced with XR_MAKE_VERSION.

XR_VERSION_MAJOR extracts the API major version number from a packed version number.

```
// Provided by XR_VERSION_1_0
#define XR_VERSION_MINOR(version) (uint16_t)(((uint64_t)(version) >> 32) & 0xffffULL)
```

Parameter Descriptions

- `version` is a packed version number, such as those produced with XR_MAKE_VERSION.

XR_VERSION_MINOR extracts the API minor version number from a packed version number.

```
// Provided by XR_VERSION_1_0
#define XR_VERSION_PATCH(version) (uint32_t)((uint64_t)(version) & 0xffffffffULL)
```

Parameter Descriptions

- `version` is a packed version number, such as those produced with XR_MAKE_VERSION.

XR_VERSION_PATCH extracts the API patch version number from a packed version number.

Handle and Atom Macros

```
// Provided by XR_VERSION_1_0
#if !defined(XR_DEFINE_HANDLE)
#if (XR_PTR_SIZE == 8)
    #define XR_DEFINE_HANDLE(object) typedef struct object##_T* object;
#else
    #define XR_DEFINE_HANDLE(object) typedef uint64_t object;
#endif
#endif
```

Parameter Descriptions

- object is the name of the resulting C type.

XR_DEFINE_HANDLE defines a handle type, which is an opaque 64 bit value, which **may** be implemented as an opaque, distinct pointer type on platforms with 64 bit pointers.

For further details, see Handles.

```
// Provided by XR_VERSION_1_0
#if !defined(XR_NULL_HANDLE)
#if (XR_PTR_SIZE == 8) && XR_CPP_NULLPTR_SUPPORTED
    #define XR_NULL_HANDLE nullptr
#else
    #define XR_NULL_HANDLE 0
#endif
#endif
```

XR_NULL_HANDLE is a reserved value representing a non-valid object handle. It **may** be passed to and returned from API functions only when specifically allowed.

```
// Provided by XR_VERSION_1_0
#if !defined(XR_DEFINE_ATOM)
    #define XR_DEFINE_ATOM(object) typedef uint64_t object;
#endif
```

Parameter Descriptions

- object is the name of the resulting C type.

XR_DEFINE_ATOM defines an atom type, which is an opaque 64 bit integer.

```
// Provided by XR_VERSION_1_0
#if !defined(XR_DEFINE_OPAQUE_64)
    #if (XR_PTR_SIZE == 8)
        #define XR_DEFINE_OPAQUE_64(object) typedef struct object##_T* object;
    #else
        #define XR_DEFINE_OPAQUE_64(object) typedef uint64_t object;
    #endif
#endif
```

Parameter Descriptions

- object is the name of the resulting C type.

XR_DEFINE_OPAQUE_64 defines an opaque 64 bit value, which **may** be implemented as an opaque, distinct pointer type on platforms with 64 bit pointers.

Platform-Specific Macro Definitions

Additional platform-specific macros and interfaces are defined using the included `openxr_platform.h` file. These macros are used to control platform-dependent behavior, and their exact definitions are under the control of specific platform implementations of the API.

Platform-Specific Calling Conventions

On many platforms the following macros are empty strings, causing platform- and compiler-specific default calling conventions to be used.

XRAPI_ATTR is a macro placed before the return type of an API function declaration. This macro controls calling conventions for C++11 and GCC/Clang-style compilers.

XRAPI_CALL is a macro placed after the return type of an API function declaration. This macro controls calling conventions for MSVC-style compilers.

XRAPI_PTR is a macro placed between the (and * in API function pointer declarations. This macro also controls calling conventions, and typically has the same definition as XRAPI_ATTR or XRAPI_CALL, depending on the compiler.

Examples:

Function declaration:

```
XRAPI_ATTR <return_type> XRAPI_CALL <function_name>(<function_parameters>);
```

Function pointer type declaration:

```
typedef <return_type> (XRAPI_PTR *PFN_<function_name>)(<function_parameters>);
```

Platform-Specific Header Control

If the XR_NO_STDINT_H macro is defined by the application at compile time, before including any OpenXR header, extended integer types normally found in `<stdint.h>` and used by the OpenXR headers, such as `uint8_t`, **must** also be defined (as `typedef` or with the preprocessor) before including any OpenXR header. Otherwise, `openxr.h` and related headers will not compile. If XR_NO_STDINT_H is not defined, the system-provided `<stdint.h>` is used to define these types. There is a fallback path for Microsoft Visual Studio version 2008 and earlier versions (which lack this header) that is automatically activated as needed.

Graphics API Header Control

Compile Time Symbol	Graphics API Name
XR_USE_GRAPHICS_API_OPENGL	OpenGL
XR_USE_GRAPHICS_API_OPENGL_ES	OpenGL ES
XR_USE_GRAPHICS_API_VULKAN	Vulkan
XR_USE_GRAPHICS_API_D3D11	Direct3D 11
XR_USE_GRAPHICS_API_D3D12	Direct3D 12

Window System Header Control

Compile Time Symbol	Window System Name
XR_USE_PLATFORM_WIN32	Microsoft Windows
XR_USE_PLATFORM_XLIB	X Window System Xlib
XR_USE_PLATFORM_XCB	X Window System XCB
XR_USE_PLATFORM_EGL	EGL (for OpenGL/OpenGL ES usage on any platform with EGL support)
XR_USE_PLATFORM_WAYLAND	Wayland (deprecated)
XR_USE_PLATFORM_ANDROID	Android Native

Wayland Notes

The Wayland part of `XR_KHR_opengl_enable` was never implemented by any runtime, and it was determined that as written the Wayland integration would not work well without making unsafe assumptions. It was decided to deprecate these parts. A future extension would be needed to provide functionality for a Wayland integration.

Android Notes

Android specific notes for using the OpenXR specification.

Android Runtime category tag for immersive mode selection

Android applications should add the `<category android:name="org.khronos.openxr.intent.category.IMMERSIVE_HMD" />` tag inside the intent-filter to indicate that the activity starts in an immersive OpenXR mode and will not touch the native Android 2D surface.

The HMD suffix indicates the preferred form-factor used by the application and can be used by launchers to filter applications listed.

For example:

```
<intent-filter>
   <action android:name="android.intent.action.MAIN" />
   <category android:name="android.intent.category.LAUNCHER" />
   <category android:name="org.khronos.openxr.intent.category.IMMERSIVE_HMD" />
</intent-filter>
```

Glossary

The terms defined in this section are used throughout this Specification. Capitalization is not significant for these definitions.

Term	Description
Application	The XR application which calls the OpenXR API to communicate with an OpenXR runtime.

Term	Description
Deprecated	A feature/extension is deprecated if it is no longer recommended as the correct or best way to achieve its intended purpose. Generally a newer feature/extension will have been created that solves the same problem - in cases where no newer alternative feature exists, justification should be provided.
Handle	An opaque integer or pointer value used to refer to an object. Each object type has a unique handle type.
Haptic	Haptic or kinesthetic communication recreates the sense of touch by applying forces, vibrations, or motions to the user.
In-Process	Something that executes in the application's process.
Instance	The top-level object, which represents the application's connection to the runtime. Represented by an XrInstance object.
Normalized	A value that is interpreted as being in the range [0,1], or a vector whose norm is in that range, as a result of being implicitly divided or scaled by some other value.
Out-Of-Process	Something that executes outside the application's process.
Promoted	A feature is promoted if it is taken from an older extension and made available as part of a new core version of the API, or a newer extension that is considered to be either as widely supported or more so. A promoted feature may have minor differences from the original such as: • It may be renamed • A small number of non-intrusive parameters may have been added • The feature may be advertised differently by device features • The author ID suffixes will be changed or removed as appropriate

Term	Description
Provisional	A feature is released provisionally in order to get wider feedback on the functionality before it is finalized. Provisional features may change in ways that break backwards compatibility, and thus are not recommended for use in production applications.
Required Extensions	Extensions that must be enabled alongside extensions dependent on them, or that must be enabled to use given hardware.
Runtime	The software which implements the OpenXR API and allows applications to interact with XR hardware.
Swapchain	A resource that represents a chain of images in device memory. Represented by an XrSwapchain object.
Swapchain Image	Each element in a swapchain. Commonly these are simple formatted 2D images, but in other cases they may be array images. Represented by a structure related to XrSwapchainImageBaseHeader.

Abbreviations

Abbreviations and acronyms are sometimes used in the API where they are considered clear and commonplace, and are defined here:

Abbreviation	Description
API	Application Programming Interface
AR	Augmented Reality
ER	Eye Relief
IAD	Inter Axial Distance
IPD	Inter Pupillary Distance
MR	Mixed Reality
OS	Operating System
TSG	Technical Sub-Group. A specialized sub-group within a Khronos Working Group (WG).
VR	Virtual Reality

Abbreviation	Description
WG	Working Group. An organized group of people working to define/augment an API.
XR	VR + AR + MR

Dedication (Informative)

In memory of Johannes van Waveren: a loving father, husband, son, brother, colleague, and dear friend.

Johannes, known to his friends as "JP", had a great sense of humor, fierce loyalty, intense drive, a love of rainbow unicorns, and deep disdain for processed American cheese. Perhaps most distinguishing of all, though, was his love of technology and his extraordinary technical ability.

JP's love of technology started at an early age --- instead of working on his homework, he built train sets, hovercrafts, and complex erector sets from scratch; fashioned a tool for grabbing loose change out of street grates; and played computer games. The passion for computer games continued at Delft University of Technology, where, armed with a T1 internet connection and sheer talent, he regularly destroyed his foes in arena matches without being seen, earning him the moniker "MrElusive". During this time, he wrote the Gladiator-bot AI, which earned him acclaim in the community and led directly to a job at the iconic American computer game company, id Software. From there, he quickly became an expert in every system he touched, contributing significantly to every facet of the technology: AI, path navigation, networking, skeletal animation, virtual texturing, advanced rendering, and physics. He became a master of all. He famously owned more lines of code than anyone else, but he was also a generous mentor, helping junior developers hone their skills and make their own contributions.

When the chance to work in the VR industry arose, he saw it as an opportunity to help shape the future. Having never worked on VR hardware did not phase him; he quickly became a top expert in the field. Many of his contributions directly moved the industry forward, most recently his work on asynchronous timewarp and open-standards development.

Time was not on his side. Even in his final days, JP worked tirelessly on the initial proposal for this specification. The treatments he had undergone took a tremendous physical toll, but he continued to work because of his love of technology, his dedication to the craft, and his desire to get OpenXR started on a solid footing. His focus was unwavering.

His proposal was unofficially adopted several days before his passing - and upon hearing, he mustered the energy for a smile. While it was his great dream to see this process through, he would be proud of the spirit of cooperation, passion, and dedication of the industry peers who took up the torch to drive this specification to completion.

JP lived a life full of accomplishment, as evidenced by many publications, credits, awards, and nominations where you will find his name. A less obvious accomplishment --- but of equal importance --- is the influence he had on people through his passionate leadership. He strove for excellence in everything that he did. He was always excited to talk about technology and share the discoveries made while working through complex problems. He created excitement and interest around engineering and technical excellence. He was a mentor and teacher who inspired those who knew him and many continue to benefit from his hard work and generosity.

JP was a rare gem; fantastically brilliant intellectually, but also warm, compassionate, generous, humble, and funny. Those of us lucky enough to have crossed paths with him knew what a privilege and great honor it was to know him. He is certainly missed.



Contributors (Informative)

OpenXR is the result of contributions from many people and companies participating in the Khronos OpenXR Working Group. Members of the Working Group, including the company that they represented at the time of their most recent contribution, are listed below.

Working Group Contributors to OpenXR

- Adam Gousetis, Google (version 1.0)
- Alain Zanchetta, Microsoft (version 1.1)
- Alex Turner, Microsoft (versions 1.0, 1.1)
- Alex Sink, HTC (version 1.1)
- Alfredo Muniz, XEED (version 1.1) (Working Group Chair)
- Andreas Loeve Selvik, Meta Platforms (versions 1.0, 1.1)
- Andres Rodriguez, Valve Software (version 1.0)
- Armelle Laine, Qualcomm Technologies (version 1.0)
- Attila Maczak, CTRL-labs (version 1.0)
- David Fields, Microsoft (version 1.1)
- Baolin Fu, ByteDance (version 1.1)
- Blake Taylor, Magic Leap (version 1.0)
- Brad Grantham, Google (version 1.0)
- Brandon Jones, Google (version 1.0)
- Brent E. Insko, Intel (version 1.0) (former Working Group Chair)
- Brent Wilson, Microsoft (version 1.0)
- Bryce Hutchings, Microsoft (versions 1.0, 1.1)
- Cass Everitt, Meta Platforms (versions 1.0, 1.1)
- Charles Egenbacher, Epic Games (version 1.0)
- Charlton Rodda, Collabora (version 1.1)
- Chris Kuo, HTC (version 1.1)
- Chris Osborn, CTRL-labs (version 1.0)
- Christine Perey, Perey Research & Consulting (version 1.0)
- Christoph Haag, Collabora (version 1.0, 1.1)
- Christopher Fiala, Epic Games (version 1.1)
- Craig Donner, Google (version 1.0)

- Dan Ginsburg, Valve Software (version 1.0)
- Dave Houlton, LunarG (version 1.0)
- Dave Shreiner, Unity Technologies (version 1.0)
- Darryl Gough, Microsoft (version 1.1)
- Denny Rönngren, Varjo (versions 1.0, 1.1)
- Dmitriy Vasilev, Samsung Electronics (version 1.0)
- Doug Twileager, ZSpace (version 1.0)
- Ed Hutchins, Meta Platforms (version 1.0)
- Eryk Pecyna, Meta Platforms (version 1.1)
- Frederic Plourde, Collabora (version 1.1)
- Gloria Kennickell, Meta Platforms (version 1.0)
- Gregory Greeby, AMD (version 1.0)
- Guodong Chen, Huawei (version 1.0)
- Jack Pritz, Unity Technologies (versions 1.0, 1.1)
- Jakob Bornecrantz, Collabora (versions 1.0, 1.1)
- Jared Cheshier, PlutoVR (versions 1.0, 1.1)
- Jared Finder, Google (version 1.1)
- Javier Martinez, Intel (version 1.0)
- Jeff Bellinghausen, Valve Software (version 1.0)
- Jiehua Guo, Huawei (version 1.0)
- Joe Ludwig, Valve Software (versions 1.0, 1.1)
- John Kearney, Meta Platforms (version 1.1)
- Johannes van Waveren, Meta Platforms (version 1.0)
- Jon Leech, Khronos (version 1.0)
- Jonas Pegerfalk, Tobii (version 1.1)
- Jonathan Wright, Meta Platforms (versions 1.0, 1.1)
- Juan Wee, Samsung Electronics (version 1.0)
- Jules Blok, Epic Games (version 1.0)
- Jun Yan, ByteDance (version 1.1)
- Karl Schultz, LunarG (version 1.0)
- Karthik Kadappan, Magic Leap (version 1.1)
- Karthik Nagarajan, Qualcomm Technologies (version 1.1)

- Kaye Mason, Google (version 1.0)
- Krzysztof Kosiński, Google (version 1.0)
- Kyle Chen, HTC (version 1.1)
- Lachlan Ford, Google (versions 1.0, 1.1)
- Lubosz Sarnecki, Collabora (version 1.0)
- Mark Young, LunarG (version 1.0)
- Martin Renschler, Qualcomm Technologies (version 1.0)
- Matias Koskela, Tampere University of Technology (version 1.0)
- Matt Wash, Arm (version 1.0)
- Mattias Brand, Tobii (version 1.0)
- Mattias O. Karlsson, Tobii (version 1.0)
- Michael Gatson, Dell (version 1.0)
- Minmin Gong, Microsoft (version 1.0)
- Mitch Singer, AMD (version 1.0)
- Nathan Nuber, Valve (version 1.1)
- Nell Waliczek, Microsoft (version 1.0)
- Nick Whiting, Epic Games (version 1.0) (former Working Group Chair)
- Nigel Williams, Sony (version 1.0)
- Nihav Jain, Google, Inc (version 1.1)
- Paul Pedriana, Meta Platforms (version 1.0)
- Paulo Gomes, Samsung Electronics (version 1.0)
- Peter Kuhn, Unity Technologies (versions 1.0, 1.1)
- Peter Peterson, HP Inc (version 1.0)
- Philippe Harscoet, Samsung Electronics (versions 1.0, 1.1)
- Pierre-Loup Griffais, Valve Software (version 1.0)
- Rafael Wiltz, Magic Leap (version 1.1)
- Rajeev Gupta, Sony (version 1.0)
- Remi Arnaud, Starbreeze (version 1.0)
- Remy Zimmerman, Logitech (version 1.0)
- Ria Hsu, HTC (version 1.1)
- River Gillis, Google (version 1.0)
- Robert Blenkinsopp, Ultraleap (version 1.1)

- Robert Memmott, Meta Platforms (version 1.0)
- Robert Menzel, NVIDIA (version 1.0)
- Robert Simpson, Qualcomm Technologies (version 1.0)
- Robin Bourianes, Starbreeze (version 1.0)
- Ron Bessems, Magic Leap (version 1.1) (Working Group Vice-Chair)
- Rune Berg, independent (version 1.1)
- Rylie Pavlik, Collabora (versions 1.0, 1.1) (Spec Editor)
- Ryan Vance, Epic Games (version 1.0)
- Sam Martin, Arm (version 1.0)
- Satish Salian, NVIDIA (version 1.0)
- Scott Flynn, Unity Technologies (version 1.0)
- Shanliang Xu, ByteDance (version 1.1)
- Sean Payne, CTRL-labs (version 1.0)
- Sophia Baldonado, PlutoVR (version 1.0)
- Steve Smith, Epic Games (version 1.0)
- Sungye Kim, Intel (version 1.0)
- Tom Flynn, Samsung Electronics (version 1.0)
- Trevor F. Smith, Mozilla (version 1.0)
- Victor Brodin, Epic Games (version 1.1)
- Vivek Viswanathan, Dell (version 1.0)
- Wenlin Mao, Meta Platforms (version 1.1)
- Xiang Wei, Meta Platforms (version 1.1)
- Yin Li, Microsoft (versions 1.0, 1.1)
- Yuval Boger, Sensics (version 1.0)
- Zhanrui Jia, ByteDance (version 1.1)
- Zheng Qin, Microsoft (version 1.0)

Index